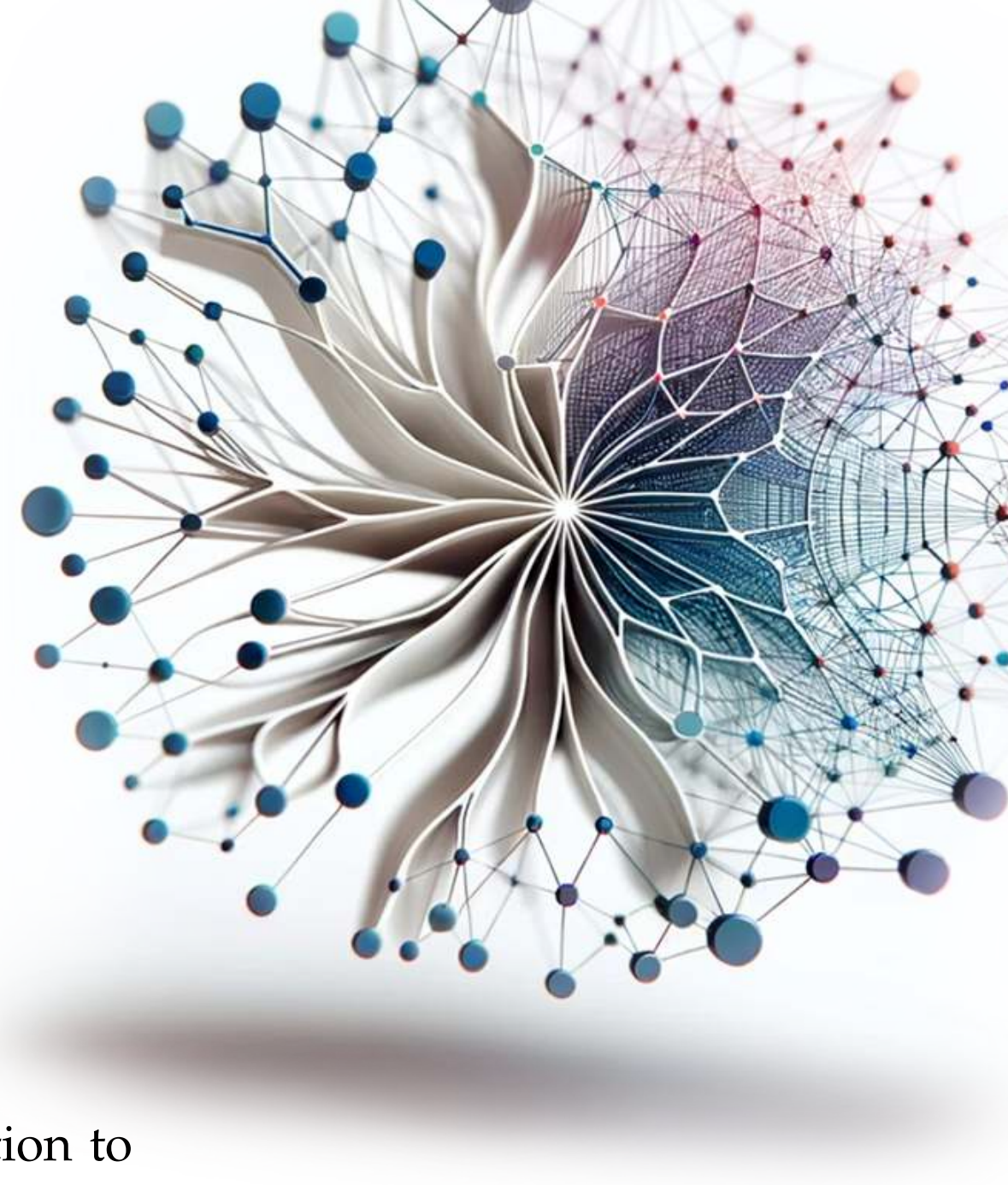


Introduction to

Machine Learning Systems

Vijay
Janapa Reddi

Introduction to Machine Learning Systems

From Algorithms to Silicon: The Engineering of Deep Learning Systems

Introduction to Machine Learning Systems

From Algorithms to Silicon: The Engineering of Deep Learning Systems

Vijay Janapa Reddi

The MIT Press
Cambridge, Massachusetts
London, England

The MIT Press
Massachusetts Institute of Technology
77 Massachusetts Avenue
Cambridge, MA 02139
mitpress.mit.edu

© 2027 Massachusetts Institute of Technology

This work is subject to a Creative Commons CC-BY-NC-ND license.

This license applies only to the work in full and not to any components included with permission. Subject to such license, all rights are reserved. No part of this book may be used to train artificial intelligence systems without permission in writing from the MIT Press.



The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers.
This book was prepared and formatted in Quarto by Željko Hrček. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data is available.

ISBN: 978-0-262-05888-9

EU Authorised Representative: Easy Access System Europe, Mustamäe tee 50, 10621 Tallinn, Estonia | Email: gpsr.requests@easproject.com

In loving memory of my father,
who taught me how to learn.

And with gratitude to my mother,
who taught me how to give.

Contents

Foreword xiii

Author's Note xvii

Preface xix

Acknowledgments xxvii

A Note on AI Assistance xxix

Notation xxxi

Main Text

Chapter 1 Introduction 3
1.1 The AI Systems Moment . . . 4
1.2 From Code-Defined Logic to Data-Defined Behavior . . . 4
1.3 The Evolution of AI Bottlenecks . . . 7
1.4 The Bitter Lesson . . . 11
1.5 Defining ML Systems . . . 12
1.6 When ML Systems Fail Silently . . . 15
1.7 Iron Law of ML Systems . . . 16
1.8 Three Dimensions of ML Efficiency . . . 21
1.9 AI Engineering as a Discipline . . . 24
1.10 Book Organization . . . 26
1.11 Fallacies and Pitfalls . . . 27
1.12 Summary . . . 28

Part I: Foundations of ML Systems

Foundation Principles 33

Chapter 2 System Paradigms 35
2.1 Four Deployment Regimes Across Nine Orders of Magnitude . . . 36
2.2 Physical Constraints on Deployment . . . 37
2.3 Analyzing Workloads . . . 40
2.4 System Balance and Hardware . . . 43
2.5 Cloud ML: Computational Power . . . 47
2.6 Edge ML: Latency and Privacy . . . 51
2.7 Mobile ML: Offline Intelligence . . . 57
2.8 TinyML: Ubiquitous Sensing . . . 60
2.9 Choosing a Deployment Paradigm . . . 63
2.10 Hybrid Architectures . . . 67
2.11 What Every Tier Shares . . . 71

2.12 Fallacies and Pitfalls . . . 72
2.13 Summary . . . 74

Chapter 3 ML Lifecycle **77**
3.1 The Lifecycle as a Loop . . . 78
3.2 How Decisions Cascade: The Constraint Propagation Principle . . . 82
3.3 Where the Hours Go: Quantifying the ML Lifecycle . . . 83
3.4 Stage Ownership and Hand-offs . . . 84
3.5 Stage 1: Problem Definition . . . 87
3.6 Stage 2: Data Collection and Preparation . . . 89
3.7 Stage 3: Model Development and Training . . . 91
3.8 Stage 4: Evaluation and Validation . . . 94
3.9 Stage 5: Deployment and Integration . . . 97
3.10 Stage 6: Monitoring and Maintenance . . . 100
3.11 Systems Thinking Across the Lifecycle . . . 101
3.12 Fallacies and Pitfalls . . . 103
3.13 Summary . . . 104

Chapter 4 Data Engineering **107**
4.1 Data as Code . . . 108
4.2 How Data Fails . . . 109
4.3 Requirements and Data Budget . . . 113
4.4 Data Acquisition . . . 118
4.5 Ingestion and Validation . . . 123
4.6 Data Labeling . . . 131
4.7 Data Processing . . . 139
4.8 Dataset Release . . . 146
4.9 Physics of Data . . . 149
4.10 Storage and Delivery . . . 152
4.11 Accepting the Pipeline . . . 159
4.12 Operational Data Health . . . 160
4.13 Fallacies and Pitfalls . . . 165
4.14 Summary . . . 167

Part II: Development and Training

Development Principles **171**

Chapter 5 Neural Computation **173**
5.1 From Logic to Arithmetic . . . 174
5.2 Computing with Patterns . . . 176
5.3 Neural Network Fundamentals . . . 180
5.4 Learning Process . . . 192
5.5 Inference Pipeline . . . 217
5.6 Handwritten Digit Recognition . . . 220
5.7 D·A·M Reading of Neural Computation . . . 223
5.8 Fallacies and Pitfalls . . . 224
5.9 Summary . . . 225

Chapter 6 Network Architectures **227**
6.1 Architectural Principles . . . 228
6.2 MLPs: Dense Pattern Processing . . . 231
6.3 CNNs: Spatial Pattern Processing . . . 235
6.4 Skip Connections and Normalization . . . 246
6.5 RNNs: Sequential Pattern Processing . . . 249
6.6 Attention: Dynamic Processing . . . 254

6.7 Transformers: Parallel Sequence Processing . . . 260
6.8 Sparse Architectures: Recommendation Systems . . . 268
6.9 Comparing and Choosing Architectures . . . 272
6.10 Fallacies and Pitfalls . . . 282
6.11 Summary . . . 283

Chapter 7 ML Frameworks **285**
7.1 Three Framework Problems . . . 286
7.2 Evolution of Framework Abstractions . . . 288
7.3 Execution: From Graphs to Kernels . . . 290
7.4 Differentiation: Automatic Gradient Computation . . . 305
7.5 Abstraction: Tensors, Devices, and Operations . . . 316
7.6 Modules: From Operations to Models . . . 325
7.7 Anatomy of a Training Step . . . 330
7.8 Major Frameworks: TensorFlow, PyTorch, and JAX . . . 334
7.9 Choosing a Framework . . . 339
7.10 Fallacies and Pitfalls . . . 341
7.11 Summary . . . 342

Chapter 8 Model Training **345**
8.1 From One Step to a Training Run . . . 346
8.2 Training-Time Law . . . 349
8.3 Computation and Memory Costs of Training . . . 351
8.4 Pipeline Architecture . . . 365
8.5 Identifying Bottlenecks . . . 378
8.6 Pipeline Optimizations . . . 380
8.7 Scaling Training Systems . . . 404
8.8 Fallacies and Pitfalls . . . 410
8.9 Summary . . . 411

Part III: Optimization and Acceleration

Optimization Principles **417**

Chapter 9 Data Selection **421**
9.1 The Data Wall . . . 422
9.2 Measuring Data Value . . . 425
9.3 Static Pruning . . . 433
9.4 Dynamic Selection . . . 438
9.5 Synthetic Data Generation . . . 442
9.6 Learning from Unlabeled Data . . . 445
9.7 Selection Engineering . . . 448
9.8 Choosing a Selection Strategy . . . 455
9.9 Cross-Layer Interactions . . . 462
9.10 Fallacies and Pitfalls . . . 463
9.11 Summary . . . 464

Chapter 10 Model Compression **467**
10.1 Fitting the Deployment Budget . . . 469
10.2 Structural Optimization: Changing What the Model Computes . . . 474
10.3 Numerical Precision: Changing How Values Are Stored . . . 497
10.4 Hardware Implementation: Changing How the Model Executes . . . 517
10.5 Choosing and Combining Techniques . . . 522
10.6 Measuring Compression Gains . . . 529
10.7 Fallacies and Pitfalls . . . 533
10.8 Summary . . . 535

Chapter 11 Hardware Acceleration **537**
11.1 Anatomy of an Accelerator . . . 538
11.2 Hardware Specialization . . . 544
11.3 Compute Primitives . . . 549
11.4 Compute Units . . . 557
11.5 Memory Systems . . . 564
11.6 Performance Bounds . . . 574
11.7 Hardware Mapping . . . 584
11.8 Compilers and Runtimes . . . 600
11.9 Beyond One Accelerator . . . 607
11.10 Heterogeneous SoC Design . . . 607
11.11 Choosing an Accelerator . . . 611
11.12 Fallacies and Pitfalls . . . 613
11.13 Summary . . . 616

Chapter 12 Benchmarking **619**
12.1 The Benchmark–Production Gap . . . 620
12.2 Benchmark Carpentry . . . 624
12.3 Benchmarking Granularity . . . 632
12.4 Benchmark Suites . . . 640
12.5 Training Benchmarks . . . 646
12.6 Inference Benchmarks . . . 652
12.7 Power and Energy Measurement . . . 661
12.8 Benchmarking Best Practices . . . 667
12.9 Model and Data Evaluation . . . 673
12.10 Fallacies and Pitfalls . . . 683
12.11 Summary . . . 684

Part IV: Deployment and Operations

Deployment Principles **689**

Chapter 13 Model Serving **693**
13.1 The Serving Inversion . . . 694
13.2 Serving System Architecture . . . 699
13.3 Request Lifecycle . . . 702
13.4 Queuing Theory . . . 710
13.5 Throughput Optimization . . . 717
13.6 Inference Runtime Selection . . . 728
13.7 Node-Level Optimization . . . 734
13.8 Model Loading and Versioning . . . 738
13.9 LLM Serving . . . 742
13.10 Economics and Planning . . . 749
13.11 Fallacies and Pitfalls . . . 754
13.12 Summary . . . 755

Chapter 14 ML Operations **759**
14.1 Silent Failure in Production . . . 760
14.2 Technical Debt . . . 767
14.3 Development Infrastructure . . . 773
14.4 Releasing Models . . . 781
14.5 Production Monitoring . . . 787
14.6 Retraining Decisions . . . 798
14.7 Incident Response and Governance . . . 804
14.8 Operational Maturity . . . 810
14.9 Case Studies . . . 816

14.10 Fallacies and Pitfalls . . . 822
14.11 Summary . . . 824

Chapter 15 Responsible Engineering **827**
15.1 Responsibility as Systems Engineering . . . 828
15.2 Engineering Responsibility Gap . . . 830
15.3 Responsible Engineering Across the Lifecycle . . . 836
15.4 Environmental and Cost Awareness . . . 853
15.5 Data Governance and Compliance . . . 858
15.6 Fallacies and Pitfalls . . . 866
15.7 Summary . . . 867

Synthesis

Chapter 16 Conclusion **873**
16.1 The System Is the Model . . . 874
16.2 Lighthouse Models: Constraint Propagation . . . 875
16.3 Thirteen Quantitative Principles . . . 877
16.4 The Dual Mandate Revisited . . . 881
16.5 Future Directions . . . 883
16.6 The Engineering Responsibility . . . 886
16.7 Fallacies and Pitfalls . . . 886
16.8 Summary . . . 887

Back Matter

Appendices **893**

Appendix A The D·A·M Taxonomy **893**

Appendix B Data Foundations **903**

Appendix C Algorithm Foundations **909**

Appendix D Machine Foundations **925**

Appendix E System Assumptions **937**

Glossary **953**

References **965**

Index **995**

Foreword

David A. Patterson
Pardee Professor of Computer Science Emeritus
University of California, Berkeley
Distinguished Engineer, Google
ACM A.M. Turing Award Laureate
Berkeley, California

My career has been dedicated to understanding and improving the very foundations upon which computing operates – from the instruction set architectures that define processors to the storage systems that hold our data. I've seen firsthand how profound advancements are rarely about a single brilliant idea in isolation, but rather about the holistic integration of many ideas into a robust, efficient, and reliable system.

Today, we stand at another pivotal moment. Machine learning (ML) has transcended scientific curiosity to become a force reshaping industries, driving innovation, and transforming our daily lives. The intellectual power of the algorithms, the gains in accuracy, and the seemingly magical capabilities they unlock have rightly captured the world's imagination.

One concern is that much of this progress has been achieved by a community that, through no fault of its own, was never taught to think in systems terms about what it was building. Engineers who can call `model.fit()` but cannot explain why training is slow. Researchers who tune hyperparameters without understanding the memory bandwidth constraints that make certain choices impractical. Practitioners who deploy models into production without a framework for reasoning about latency, throughput, or failure modes.

A brilliant algorithm, without a well-engineered system to support it, is like a powerful engine without a chassis and brakes. It won't get you far, and it certainly won't get you there reliably.

This gap is not a talent deficit. It is a curriculum deficit. The field has simply not had the textbooks it deserves. This book, *Introduction to Machine Learning Systems*, arrives precisely at the right time to illuminate this critical, often overlooked, dimension of the ML revolution.

Why Systems Thinking Matters

My career has leveraged the power of systems thinking. Identify the bottlenecks, understand the interactions between components, and design for performance, reliability, cost-effectiveness, and ease of use across the entire stack. You could not reach the conclusion from first principles alone. You had to think about the system.

ML systems demand exactly the same discipline. A neural network is not a mathematical abstraction floating free of physical constraints. It is a computation that must execute on silicon with finite memory bandwidth, finite power budgets, and finite communication capacity. Arithmetic intensity, the Roofline Model, and memory hierarchy effects are not advanced topics to be addressed after students learn the basics. They are the basics.

Why ML Systems Deserve Serious Textbook Treatment

The purpose of a textbook is not just to catalog the current frontier. It is to provide the conceptual scaffolding that allows students to understand the frontier, and to advance it themselves. The neural network architectures covered in these pages will evolve. The hardware accelerators described will

be superseded. But some fundamentals will not change. We will still have memory hierarchies and arithmetic intensity will still be a critical measure. The principles of pipelining, of latency versus throughput trade-offs, of quantization and precision reflect physical and mathematical realities that do not go out of date. Anyone who genuinely understands them will be able to reason about whatever new system the field produces next year or ten years from now.

I am also struck by the scope of what this book attempts. It is not enough to explain how training works; one must also address data engineering, model optimization, hardware acceleration, deployment, benchmarking, responsible AI, and sustainability. These are not separate topics bolted together for completeness. They are interconnected aspects of a single engineering discipline. A decision made at the data layer has implications for model behavior, which has implications for inference latency, which has implications for deployment cost, which has implications for the communities that can and cannot access the resulting system. The book correctly treats this integration as a first-class concern.

I am particularly pleased to see the quantitative methodology that runs through every chapter. Claims are backed by measurements. Trade-offs are expressed numerically. Students are taught to estimate, measure, and verify rather than to rely on their gut. The difference between an engineer and a dilettante is precisely this capacity for rigorous, measurement-grounded reasoning.

Why Open Access Is Important for the Next Generation

I applaud the decision to make this invaluable resource freely available.

When I co-wrote *Computer Architecture: A Quantitative Approach* with John Hennessy, we debated how to make the material as accessible as possible. We lobbied our publisher to keep the price down and even came out with a low cost international student edition for sales outside North America. Those concerns have only grown more urgent as ML has become infrastructure—as consequential to the modern world as the electrical grid or the internet.

The decision to publish this book as a freely available educational resource is not a compromise. It is a statement of values. ML systems are being deployed in every sector of every economy on every continent. The engineers who build and maintain them will come from everywhere. If the foundational education required to do this work responsibly is available only to students at well-resourced institutions, we will contribute to an expertise gap that mirrors the societal concerns of who benefits from AI and who bears its costs.

What impresses me further is that this project has gone well beyond open access to a static text. The accompanying ecosystem—interactive laboratories, a build-from-scratch framework in TinyTorch, hardware deployment kits, instructor materials, a global network of adopting universities—reflects a familiar educational maxim: "I hear and I forget. I see and I remember. I do and I understand."

The open-source model also makes the curriculum itself better. Errors get corrected. Examples get sharpened. New chapters emerge in response to what the field actually needs. As we found with *Computer Architecture: A Quantitative Approach*, a community of contributors that forms around a book can offer a form of quality assurance that no single author team, however expert, can match alone.

A Final Word

The field of computer architecture has a tradition I am proud of: we build things, measure them honestly, share what we learn, and revise our philosophies when the measurements demand it. We do not hide behind the complexity of our systems to avoid accountability for whether they actually work. We do not conflate novelty with progress. We debate, rigorously, about what matters and why.

Hopefully, ML systems engineering will follow this tradition. It needs the habits of mind that come from taking systems seriously: thinking about interactions, measuring at the right level of abstraction, being honest about trade-offs, and holding oneself accountable to real-world performance rather than benchmark folklore. This book embodies those habits.

Introduction to Machine Learning Systems is more than just a textbook; it is a clarion call for systems thinking in the age of AI, a guide for building robust ML applications, and a testament to the power of open education.

We are at an extraordinary moment. Over my career there has been a major breakthrough every decade: the microprocessor in the 1970s, the personal computer in the 1980s, the Internet in the 1990s, the smartphone in the 2000s, and AI in the 2010s. The first four changed business and society, yet AI looks likely to have the largest impact of them all.

The systems being built today will shape how intelligence is deployed, who benefits from it, and what it costs—in money, in energy, and in carbon footprint. We need the engineers who build those systems to have the best possible systems skills.

This book is a significant step toward making it possible for anyone to learn them.

David A. Patterson

Author's Note

The world is rushing to build AI systems; it is not yet engineering them. That sentence is the reason this book exists. A model that wins on a benchmark is not yet a system, and the distance between the two is not a detail of implementation. It is engineering: the work of making something hold up under constraints that do not negotiate. A demo answers to no one. A system answers to latency budgets, memory limits, power envelopes, and the cost of being wrong in front of real users.

What makes this its own discipline is an unusual coupling. A processor answers to the physics of the machine, to clock cycles, cache lines, and the heat it can shed. A statistical model answers to the mathematics of learning, to bias, variance, and how generalization changes with data and model capacity. A machine learning system must answer to both at once, in the same decision, and the two do not always agree. A change that pleases the hardware can starve the model of the precision or throughput it needs to converge; a change that helps the model can ask the silicon for more memory or bandwidth than it can give. Hold that tension in view and the field stops looking like a pile of tools and starts looking like a structure you can reason about.

This book hands you that structure. Not the framework that is fashionable this year or the accelerator that is fastest this quarter, but the way of thinking that outlasts them. Frameworks change, hardware turns over every few years, and the benchmarks reset; the forces underneath stay the same. If the book works, you will finish it able to look at any machine learning system, from a sensor sipping microwatts to a server farm drawing megawatts, and see the same small set of forces deciding what is possible.

The textbooks that changed me when I was a student did not hand me facts. They handed me a way of seeing, and the facts arranged themselves around it. That is what I have tried to do here.

— Vijay Janapa Reddi
Cambridge, Massachusetts
2026

Preface

Who This Book Is For

A model that trains cleanly in a research notebook can still run out of memory on a server, miss its latency budget under live traffic, or cost ten times more to serve than the application can afford. Most readers will not become hardware architects or compiler writers, yet every practitioner who builds, trains, optimizes, or deploys a model makes choices whose outcome depends on the system beneath the code.

This book explains *how* machine learning systems work, from the mathematics that define a neural network to the hardware that runs it and the engineering trade-offs that make it fast and reliable. It builds that foundation for engineers meeting ML systems for the first time, practicing engineers who want to understand what happens below the framework API, and researchers who need to reason about runtime, memory, and deployment.

Prerequisites

Readers should bring two prerequisites:

- **Programming**: Fluency in Python, including functions, classes, and basic array manipulation with NumPy.
- **Mathematics**: Comfort with linear algebra, basic calculus, and probability at the introductory university level.

Two additional areas help but are not required:

- **Computer systems**: Prior familiarity with caches, memory bandwidth, and processor organization is useful, though the optimization and hardware chapters build every physical mechanism from first principles.
- **Machine learning basics**: Prior exposure to supervised learning gives helpful context, though Part II develops the required modeling foundations from the ground up.

Why a Systems Textbook

In 1968, a NATO conference in Garmisch, Germany, helped establish *software engineering* as a distinct discipline amid concern that software systems had grown too complex for ad hoc programming (Naur and Randell 1969). Reliable software required its own principles, methods, and rigor. Decades later, Hennessy and Patterson put quantitative analysis at the center of computer architecture education, giving engineers a shared way to measure, predict, and improve processor performance (Hennessy and Patterson 2017). Both moments turned a craft of rules of thumb into a teachable engineering discipline.

Machine learning faces its own Garmisch moment. For the past decade, ML research has been dominated by *model-centric* thinking focused on discovering new algorithms and architectures. That focus matters, but a model is not a system. A neural network that scores well on an offline benchmark fails in practice if it misses its latency deadline, if its data pipeline cannot keep accelerators fed, or if its energy bill exceeds the value of its predictions. The gap between a working model and a production system is not an implementation detail; it is a gap in engineering principles.

This book treats ML systems engineering not as a catalog of tools, such as Kubernetes, PyTorch, and CUDA, but as a discipline built on quantitative bounds, cost models, and design principles. Just

as civil engineers calculate loads before choosing materials, ML systems engineers need explicit laws that reveal which resource limits performance before they rewrite code. Four ideas anchor this book:

- **The D·A·M taxonomy**: Every bottleneck lives in the data, the algorithm, or the machine, and relieving one can expose another.
- **The iron law of ML systems**: Across the D·A·M axes, workload time factors as $(\text{Samples/Task}) \times (\text{FLOPs/Sample}) \times (\text{Seconds/FLOP})$ and unpacks on hardware into data movement, computation, and overhead; where stages run concurrently (such as transferring the next batch while computing the current one), elapsed time is governed by the critical path.
- **The bitter lesson**: General methods that exploit growing computation outpace hand-encoded expertise, which makes systems efficiency a lever of progress.
- **The energy-movement principle**: Fetching a value from off-chip memory can cost far more energy than computing with it, so per-event costs and event counts determine whether movement or computation dominates.

Each part adds its own governing principles, such as data gravity and Amdahl's law, at the point where they first bind. Together, the volume-wide anchors and part-level laws form a compact analytical toolkit that carries across every chapter.

Computer science studies what is computable. ML research studies what is learnable. AI engineering studies what is buildable, given the physics of real hardware and real data. I wrote this book to help build that foundation and to do for ML systems what Hennessy and Patterson (2017) did for computer architecture: replace guesswork with measurement, and folklore with engineering. The goal is to teach system-level reasoning before implementation, treating data, algorithms, and hardware not as separate silos but as a single, coupled design problem.

Consider a box of LEGO bricks. The same interlocking pieces build a spaceship, a castle, or an entire city. The sets change; the bricks do not. ML systems work the same way. A convolutional network for image classification, a transformer for language generation, and a reinforcement learning agent for robotics serve very different applications, but beneath them sit the same building blocks: computational graphs, memory hierarchies, data pipelines, training loops, and the trade-offs among latency, throughput, and energy. Once an engineer understands those building blocks, even an unfamiliar system becomes legible.

That is why this book teaches enduring principles rather than current tools. Dominant frameworks have shifted from Theano to TensorFlow to PyTorch in under a decade, and hardware has evolved from repurposed graphics processors to purpose-built tensor accelerators, with new architectures arriving every year. Because frameworks and hardware turn over so quickly, tool-specific advice ages fast, whereas the building blocks endure.

What This Book Covers

An engineer cannot diagnose why a thousand-GPU training cluster is stalling without first knowing how a single machine moves tensors through memory, feeds its data pipeline, and keeps its execution units busy. Even the largest warehouse-scale cluster is built out of individual compute nodes. This book therefore centers on the *single compute node*: one host with one to eight accelerators, each with its own discrete memory connected over a local link such as PCIe or NVLink. Multi-node execution enters only where it shapes single-node design; the distributed communication, scheduling, and fault-recovery problems that arise across a fleet build directly on the single-node foundations developed here.

The book is organized into four parts. Each part answers one question and hands the next part a concrete result:

- **Part I: Foundations of ML Systems** asks what is being built, under which constraints, and from what inputs. It develops the vocabulary, cost models, and end-to-end workflow that guide every ML project, from the traits that distinguish ML systems from traditional software to the data pipelines that feed them. It hands forward a design envelope and trustworthy data.
- **Part II: Development and Training** asks how a model computes, how it is structured, and how it learns. It connects mathematical foundations to working software and hardware through

neural network computation, model architectures, execution frameworks, and training loops. It hands forward a trained model whose memory, compute, and wall-clock costs are known.

- **Part III: Optimization and Acceleration** asks how to make that model cheaper to train or serve and how to prove the gain. Its chapters follow the D·A·M taxonomy in order: data selection cuts the samples a run must process, model compression shrinks the algorithm, hardware acceleration maps the remaining work onto the machine, and benchmarking tests whether the speedup holds up under honest measurement. It hands forward verified evidence for a deployment decision.
- **Part IV: Deployment and Operations** asks whether the system survives real users, shifting data, and its obligations to society. It covers serving infrastructure, production monitoring and retraining, and the engineering controls needed to keep deployed systems fair, safe, and accountable. It hands forward the tools to run and defend a system in production.

Each part builds on the previous one, following a single system through its life. The progression runs from problem and constraints through data, model, training, optimization, benchmarking, deployment, and operations.

Suggested Reading Paths

Reading the chapters in order gives the most complete foundation, because each part hands a concrete artifact and cost model to the next. Readers coming in with specific goals can take a more focused route:

- **The Complete Path** (all chapters, in order): For engineers and researchers new to ML systems. Start with chapter 1, then read the four parts in sequence. This path builds every concept from first principles.
- **The ML Engineer's Path** (systems depth for practitioners): For engineers who already train and deploy models in Python but want to understand what happens underneath the framework API. Skim Part I for shared vocabulary, then focus on chapter 7, chapter 8, chapter 11, chapter 12, and chapter 13.
- **The Optimization Path** (efficiency-focused): For engineers working on model compression, kernel performance, edge deployment, or tight latency budgets. After Part I, review Part II's account of forward and backward passes, architecture costs, framework execution, and training bottlenecks before diving into Part III.

Whichever path a reader takes, figure P.1 shows which earlier chapters each chapter depends on. Readers who already know a prerequisite topic can skip ahead; others can follow the incoming arrows back to fill in the background. Model training sits at the center of the map, drawing on both data engineering and ML frameworks while feeding both data selection and model compression. The longest dependency chain runs through Part III, where model compression feeds hardware acceleration and benchmarking closes the loop before deployment.

How to Read This Book

The parts follow the lifecycle of a model rather than climbing the ML systems stack from the bottom, so each chapter touches several layers of that stack at once. Modeled on the classical computer systems hierarchy, each layer of the stack provides an interface to the layer above and builds on the layer below, from hardware at the bottom through frameworks, models, training, and serving, up to operations and applications at the top, while data runs alongside every layer like a system bus. Throughout the book, a recurring margin figure highlights which layers each chapter addresses:

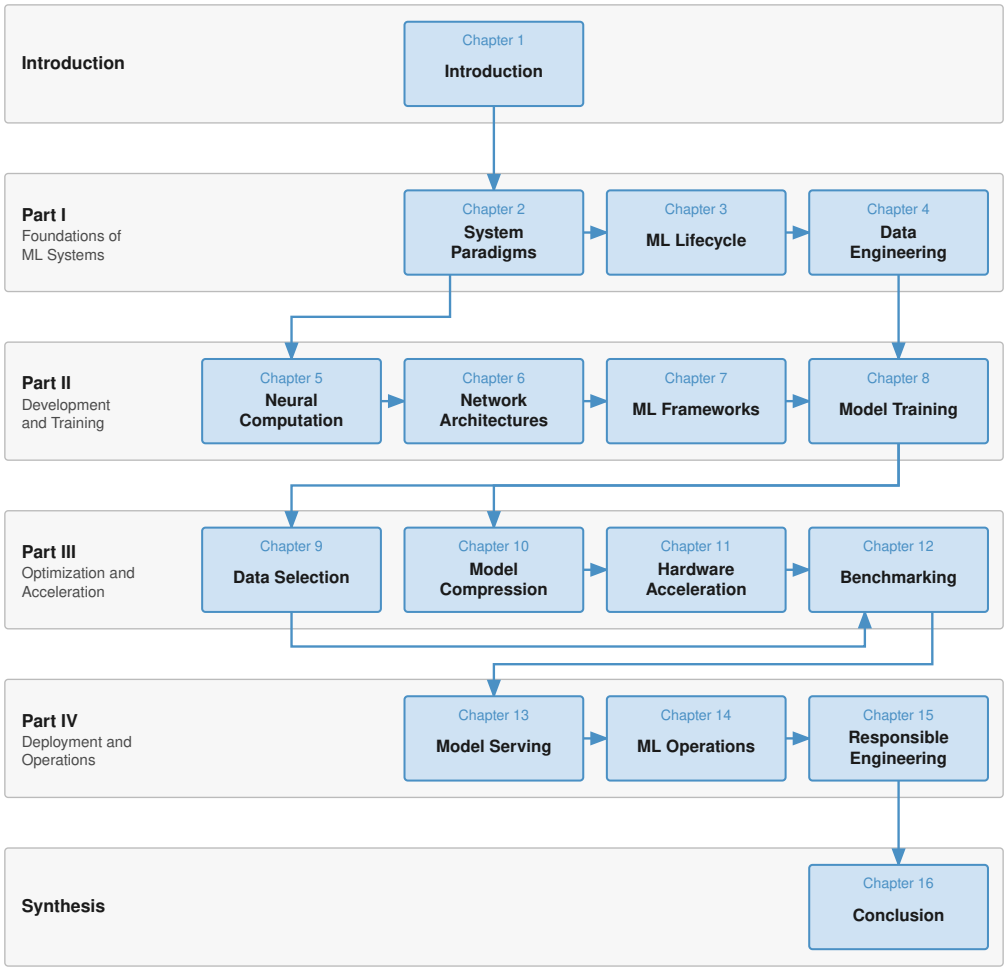
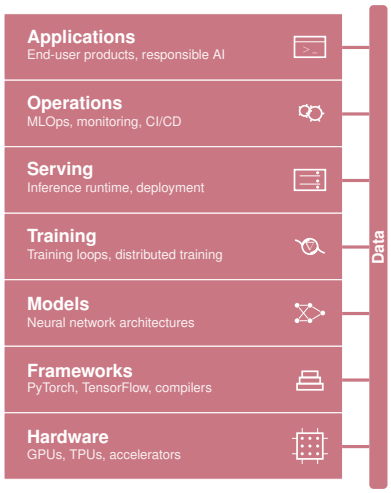
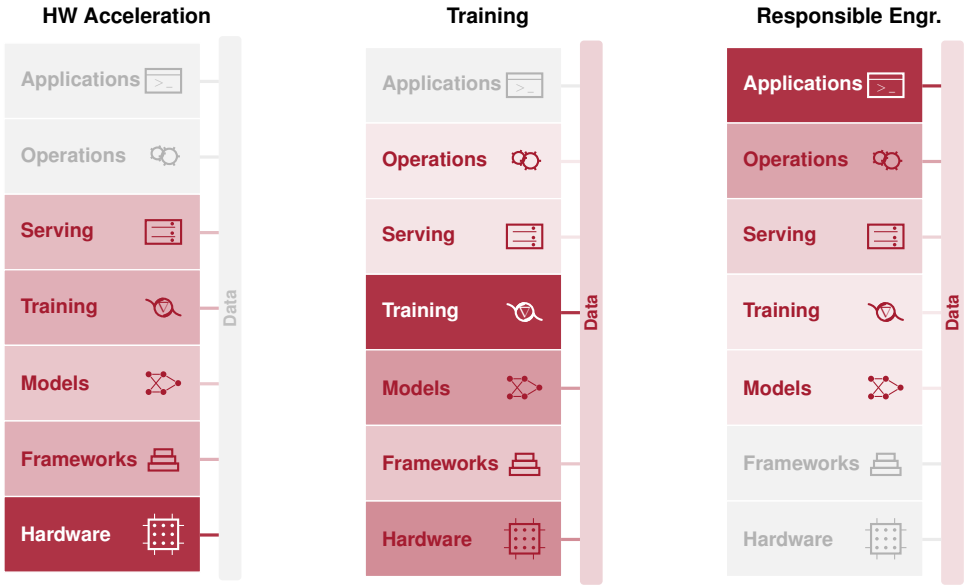


Figure P.1: Chapter Dependency Map: Each arrow points from a chapter to a later chapter that builds on its results, derived from the book's cross-references with transitive links omitted. The introduction frames every chapter; the arrow from it marks the chapter that draws on it directly.

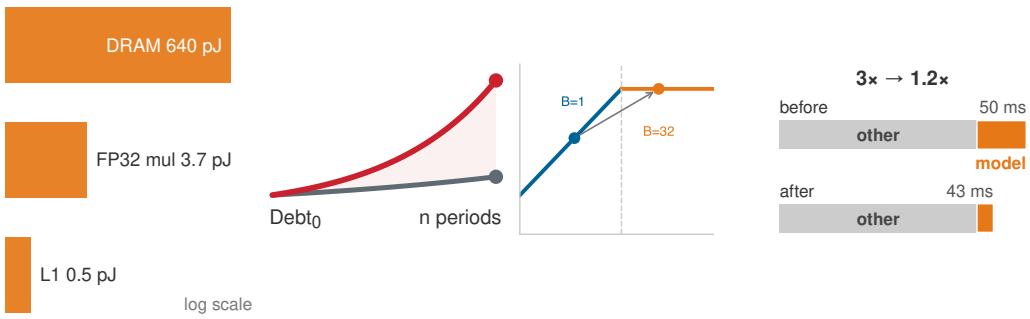


The vertical data bar alongside the stack tracks a separate dimension from the horizontal layers: its uniform shading reflects how central data is to the chapter as a whole, while the connecting wires mark which layers interact with data directly in that chapter's context. Three chapters show how this profile shifts across the book:



In chapter 11, the layer intensity concentrates at the bottom of the stack while the data bar is faint: the chapter focuses on hardware mechanisms rather than dataset construction. In chapter 8, the middle layers glow and the data bar is moderately lit, reflecting that training consumes data but is fundamentally about optimization. In chapter 15, the top layers dominate and the data bar carries a moderate tint, because biased training data surfaces as biased applications even though several layers sit between them. The same stack tells three different stories, with data as an independent dimension.

The margins also carry small visual notes at the point where a relationship is easiest to understand by sight. These are not numbered figures to cite later; they are reading aids. A ladder makes an energy or latency scale gap visible, a trend sketch shows direction, a roofline thumbnail shows whether memory bandwidth or compute throughput bounds performance, and a dominance bar shows which term controls total runtime. When a miniature uses a log scale, it says so inside the image.



Alongside these visual notes, unnumbered **marginal navigation pointers** connect an idea to the chapter that establishes it or builds on it, without interrupting the prose:

- ↩ **Backward pointer:** Directs the reader back to the earlier chapter where a principle, constraint, or cost model was established.
- ↪ **Forward pointer:** Directs the reader ahead to the later chapter, or to the appendix, where the current idea is developed, measured, or enforced.
- ⇄ **Contrast:** Marks a place where two chapters treat the same problem differently, so the comparison is visible at a glance.

A pointer only orients. When a sentence depends on another chapter, the reference stays in the text.

Conventions

This book uses a small set of typographic conventions to mark what each piece of prose is doing.

Bold marks first definitions

A term in **bold** within body prose is its formal first introduction in the volume. Each term receives this treatment exactly once. Subsequent appearances are unbolded; by then the term is part of the working vocabulary the chapter carries forward.

> Every machine learning system has a **dual mandate**: it must manage statistical uncertainty and physical constraints simultaneously.

Bold identifies a term that readers will use throughout the book. A matching index entry in the back of the book points to the defining occurrence (under the `definition` subentry), providing a way to return to it. Key technical terms are also compiled in the glossary.

Bold also appears in a second, structural role: as a functional label at the start of an item inside a callout box or a summary bullet—for example, **Recommendation**, **Trade-off**, or **Result**. In that context, the bold is scaffolding rather than a first definition.

Italics carry specific rhetorical jobs

Italics serve specific purposes rather than general emphasis. Common uses include:

- **Direct opposition**: *training* vs. *inference*, *logical* vs. *physical*.
- **Impossibility, stated as physical, mathematical, or logical law**: "sub-10 ms systems *cannot* use distant cloud infrastructure—physics forbids it."
- **Word used as word**: "the term *gradient* refers to a vector of partial derivatives."
- **Titles of works**: *Computer Architecture: A Quantitative Approach*.
- **Thesis punchline**: A single italicized sentence that captures the controlling insight of a section, for example, *Data is source code.*

Callouts have visual identities that match their job

Callout boxes are not decoration. Each type signals a specific kind of content, so the visual identity announces what the box contains before the reader engages with it.

Callout	Contains
Definition	Formal definition of a key term, with its significance, distinction from the nearest neighbor concept, and a common pitfall
Example	Historical case or concrete scenario, structured as context, insight, and systems lesson
Systems Perspective	Analytical observation or mathematical nuance that enriches the surrounding argument
Napkin Math	Worked calculation with assumptions and intermediate steps shown explicitly; intermediate tables and equations are the math itself
Checkpoint	Self-check questions to verify understanding of the preceding section
War Story	Documented engineering incident with a sourced failure mode, structured as context, failure, consequence, and lesson

Callout	Contains
Lighthouse	Recurring workloads characterized by a profile (parameters, FLOPs, dominant constraint), used as anchored case studies across multiple chapters
Principle	Canonical book principle or invariant referenced from later chapters
Theorem	Formal, equation-backed result or bound, stated with its conditions and applied in later quantitative analysis
Key Takeaways	Four to seven most important results from a chapter, in summary form
Learning Objectives	Skills and concepts that readers should be able to explain or apply after completing the chapter
What's Next	Bridge from the chapter just ended to the chapter beginning, naming the open question the next chapter addresses

When a callout contains a table, figure, or equation, that artifact is part of the callout's pedagogical unit, not decoration. A lighthouse's profile *is* its characterization. A napkin math box's intermediate tables *are* the worked math. Reading these as separated artifacts loses the teaching arc. Standalone reference tables, by contrast, sit between paragraphs as numbered artifacts the reader returns to through cross-references. Both forms are valid; the visual marker (callout vs. standalone) tells the reader which one they are looking at.

Companion Resources

Whichever path readers follow, this book is part of a broader learning ecosystem. The complete text, interactive notebooks, lecture slides, exercises, hands-on labs, and supplementary materials are available online at **mlsysbook.ai**.

This book is open source. The full text, figures, and build system are publicly available, and contributions from readers worldwide have materially improved every chapter. Visit **mlsysbook.ai/git** to report issues, suggest improvements, or contribute directly.

Using This Book in a Course

This book grew out of CS249r at Harvard University and the TinyML edX professional certificate program. Teaching ML systems on microcontrollers running on milliwatts and kilobytes of SRAM made one lesson unmistakable: the physical limits that govern a tiny sensor node—memory bandwidth, compute density, and energy per operation—are the exact same limits that govern a data center GPU. *The physics scales; only the numbers change.* That observation shaped the structure of this book.

An instructor teaching a one-semester course can keep the full sixteen-chapter arc while choosing how deeply to go inside each chapter. The core route below focuses on the engineering decisions every practitioner must be able to defend, using each chapter's purpose, learning objectives, worked examples, fallacies, and checkpoints as the spine of the syllabus:

1. **Part I (Foundations):** Ground the course in the iron law (section 1.7), then show how workload constraints dictate system architecture (section 2.3 and section 2.9). Trace the six lifecycle stages (section 3.4) from problem framing to holdout evaluation, and cover data collection, preprocessing pipelines, labeling, storage layout, and dataset health so students see how raw inputs become a reliable training feed.
2. **Part II (Development and Training):** Follow a neural network from forward and backward math (section 5.3) to end-to-end execution (section 5.5). Compare major architecture families by the operations they run, the state they store, and the data dependencies they impose. Then trace how frameworks lower a model graph into kernels, manage automatic differentiation and tensor memory, and execute a full training step (section 7.7), culminating in bottleneck diagnosis and single-node multi-GPU training (section 8.7.1).
3. **Part III (Optimization and Acceleration):** Teach students to optimize against measured bottlenecks across Data, Algorithm, and Machine. Start by scoring and pruning training data and deciding when a smaller dataset suffices (section 9.8). Move to pruning, distillation, and quantization alongside the hardware formats they require. Map those compressed models onto compute units and memory hierarchies using the roofline model (section 11.6), kernel

fusion, and compiler scheduling, and verify every claimed speedup with rigorous component, end-to-end, and power benchmarks (section 12.9).

4. **Part IV (Deployment and Operations):** Carry the verified model into production through request queuing, dynamic batching, serving runtimes, and cost modeling. Pair rollout and drift monitoring with the fairness, explainability, privacy, security, and energy controls summarized in section 15.3. Close with the coupled-budget synthesis in section 16.1, which ties the four parts back into a single engineering discipline.

Multi-node training (section 8.7.2), neural architecture search (section 10.2.5), the clinical deployment case study (section 14.9.2), and the appendix derivations work well as optional modules or project readings. Instructors teaching shorter terms or seminars can also split the book along its part boundaries:

- **Parts I–II** form an introductory half-semester on how ML systems are specified, built, and trained.
- **Parts III–IV** form an applied half-semester on optimizing, benchmarking, serving, and operating models in production.
- **The Part Introductions** alone offer a self-contained overview of the quantitative laws that govern ML systems for survey courses or executive seminars.

Lecture slides, assignments, and instructor resources are available at **mlsysbook.ai**.

Acknowledgments

Origins

This book owes its existence to a single conversation. During a sabbatical at Google, Pete Warden invited me to work on what seemed like an impossible moonshot for TinyML: running a small speech model on a microcontroller. That collaboration on TensorFlow Lite for Microcontrollers revealed something fundamental. The constraints that govern a device with 16 KB to 256 KB of memory are the same constraints that govern an accelerator in a data center, even when the scale is entirely different. Bandwidth, latency, energy, and memory obey the same physics across both settings. That insight became the seed of this book.

The TinyML work led to an online course on HarvardX, co-taught with Laurence Moroney, that grew to over 100,000 students worldwide. Brian Plancher helped build the TinyML curriculum. Marcelo Rovai, a professor who first encountered the material through the TinyML course, went on to develop the hands-on labs and kits that accompany this book. Zach Shelby opened the Edge Impulse platform for student use. Massimo Banzi and the Arduino team, along with Eric Pan and Seeed Studio, collaborated with us in developing and providing the hardware kits that made hands-on instruction possible. Marco Zennaro brought the material to developing countries through ICTP and planted the idea that these principles deserved a proper textbook. Teaching at that scale, without one, made the need undeniable.

That need turned into a book through CS249r at Harvard University. In 2023, the semester-long class project was to write the class notes: each student group took ownership of a topic, researched it, and worked with me to develop it into a chapter. That work seeded every chapter in this book. The premise was that writing forces you to think more deeply than reading alone. We then opened it all on GitHub, and what began as rough class notes has been rewritten and refined with every semester since.

Because that GitHub work continues, this book benefits from a community whose contributions evolve with the project. New contributors join as readers report issues, improve explanations, and extend the supporting materials. A printed acknowledgment can capture only one moment in that continuing history. The current contributor list and contribution guidelines are available at https://mlsysbook.ai/git.

Collaborators and Colleagues

Beyond the class and GitHub community, my earlier collaborations shaped the book's intellectual foundations. My time at Google informed much of the systems thinking in these pages. That period began with performance modeling for the first Pixel system on chip (SoC), drawing on my background in computer architecture. A collaboration with Mark Hill on roofline-style analytical modeling for heterogeneous SoCs taught me to reason about hardware from first principles, the kind of quantitative thinking that runs through every chapter. That emphasis on quantitative evaluation carried into my work on MLPerf. MLPerf involved many contributors across MLCommons, but I worked most closely with Peter Mattson, David Kanter, Carole-Jean Wu, Christine Cheng, Guenther Schmuelling, Cody Coleman, and Cliff Young. That collaboration shaped how I think about rigorous measurement of ML systems.

Boris Murmann and Song Han convinced me in the early stages that this project was worth pursuing. As I wrote, I sought feedback from colleagues in industry and academia alike. Practitioners at Arm, Google, Intel, Meta, Microsoft, Qualcomm, STMicroelectronics, and other organizations

offered perspectives grounded in production systems, while academic reviewers at universities across Europe, Asia, and the Americas stress-tested the material against their own research and teaching. Their combined feedback sharpened every chapter. Evgeni Gousev at Qualcomm and Pete Bernard at the Edge AI Foundation supported the project's outreach and helped build the community around it.

That feedback loop extended to classrooms around the world, where instructors put the material to the test when it was still rough. Sophia Shao at UC Berkeley, Xiaofan (Fred) Jiang at Columbia, Tushar Krishna and Divya Mahajan at Georgia Tech, and Wenkai Guan at the University of Minnesota Morris were among the first, and instructors at IIT Bombay, NUST Pakistan, De La Salle University in the Philippines, and universities across Europe, Latin America, and Africa have since adopted the book for their own courses. Their willingness to teach from an unfinished textbook provided invaluable feedback and confirmed that the need for this material was not limited to any one institution or any one country.

Students and Contributors

My students in the Edge Computing Lab at Harvard shaped this book in ways I cannot fully trace. Some did foundational work that helped establish TinyML as a field; their research and the ideas that emerged from years of collaboration informed the book's technical content. This cohort helped set the field in motion and then helped me write about it: Andy Cheng, Arya Tschand, Brian Plancher, Colby Banbury, Elizabeth Suitor, Emma Chen, Ikechukwu Uchendu, Jason Jabbour, Jason Yik, Jeffrey Ma, Jessica Quaye, Mark Mazumder, Matthew Stewart, Maximilian Lam, Radhika Ghosal, Shvetank Prakash, Srivatsan Krishnan, Yu-Shun Hsiao, and Zishen Wan. Kai Kleinbard, then at the Harvard Extension School, built the online generative AI SocratiQ tutor bot. Naeem Khoshnevis, from the Harvard Kempner Institute, established essential infrastructure including GitHub Actions and documentation standards. Željko Hrček produced the TikZ diagrams and contributed to the PDF layout, page composition, and final visual formatting of the book.

I am also grateful to the researchers whose work this book builds on and to the open-source community that helped improve it. Dozens of contributors caught errors, improved explanations, filed issues, and submitted pull requests. Some fixed a single typo; others rewrote entire sections. The book improved because people who owed it nothing chose to make it better. What kept me going, aside from stubbornness, was that community: every GitHub star was a signal that someone out there cared, that someone was learning, that the work mattered. I am grateful to every one of them.

Support

This work also depended on support from the Harvard Data Science Initiative, Harvard Extension School, and the National Science Foundation, as well as from the Edge AI Foundation and ICTP for educational outreach and scholarships. Arduino, Edge Impulse, Google, and Seeed Studio provided hardware, tooling, and infrastructure that enabled the practical labs.

At MIT Press, Susan Hartman believed in this project and guided it from an open-source experiment to a published textbook. Her support, editorial judgment, and friendship made this book real in a way that GitHub alone could not. I am equally grateful to MIT Press for agreeing to keep this book available as open access. A textbook about a discipline shaped by the open-source community should be accessible to everyone in it. That MIT Press shared this conviction made all the difference.

Finally, I thank my wife, Kari, and my daughters, Nora and Maya. This book was written in the hours that belonged to them: late nights that started at ten and ended at two or three in the morning, weekends that disappeared into revisions, years of mornings when I was present but not quite there. The cost was real, and they paid it with patience I did not always deserve.

A Note on AI Assistance

Every chapter of this book was researched, written, and rewritten by me. I also used AI tools throughout. What follows is more than the disclosure MIT Press requires of authors who use AI. It is a statement of the standard I held the book to, and a claim about what authorship means when the tools are this capable.

The standard is traceability. Derived quantities in these chapters are backed by Python computation cells whose source code ships with the book. Citations were checked against their sources, and cross-references are checked against stable anchors. The purpose-built infrastructure modeling engine used for the book's system calculations is available at https://mlsysbook.ai/mlsysim. A pre-commit test suite checks the machine-testable parts of these invariants on every commit, covering unit consistency, inline-reference resolution, cross-reference integrity, notation canonicality, and citation hygiene. AI tools helped me build and maintain this checking infrastructure, which makes the work auditable.

Authorship is thinking. Which concepts belong in this book and which do not. How to sequence the chapters so that each idea arrives only after the reader has the tools to understand it. Which worked examples to trace end-to-end, and which numbers to compute so the reader can verify the argument independently. What level to pitch each explanation at, rigorous enough for a graduate engineer and concrete enough for a student encountering systems thinking for the first time. Where students will get stuck, because I have taught this material and watched them get stuck. These judgments require knowledge of the reader and remain my responsibility. I used AI tools throughout this process to brainstorm framings, explore alternatives, and pressure-test my reasoning. The choices are mine.

In practice, AI was useful for writing code: the computation cells, the pre-commit checks, and the worked examples that are essentially programs. It helped me survey literature I then read in the original, draft passages I then rewrote substantially, and audit the manuscript for consistency across hundreds of cross-references, thousands of index entries, and tens of thousands of lines of prose. I reviewed the tools' suggestions and decided what to use.

AI-generated material can be plausible and still be wrong. Automated checks support review; they do not replace it. The standard I held this book to is the one I want us to demand of every system we ship. Tools can propose; engineers remain responsible for what they accept. MIT Press does not list AI tools as authors because they cannot assume ethical and legal responsibility for their work. The same is true of every engineered system. Authorship is the assumption of that responsibility, and it belongs to the human.

Notation

Machine learning systems span *machine learning* (computer science and statistics) and *systems* (computer architecture and hardware). Each field developed its notation independently, and many symbols mean different things across communities and publications. This collision creates real confusion when the disciplines merge. The notation below distinguishes quantities used in both fields.

Consider a simple statement: *"Increasing B improves throughput."* To an ML researcher, B means batch size. To a hardware engineer, B means bandwidth. Both interpretations are correct in their respective fields, but in machine learning systems we need both concepts in the same equation—hence the need for a single, consistent convention.

The Iron Law of ML Systems

Across the Data · Algorithm · Machine (D·A·M) axes, execution time factors into the product of workload scale, algorithmic operations per sample, and realized machine time per operation:

$$T = \underbrace{\left(\frac{\text{Samples}}{\text{Task}}\right)}_{\text{Data}} \times \underbrace{\left(\frac{\text{FLOPs}}{\text{Sample}}\right)}_{\text{Algorithm}} \times \underbrace{\left(\frac{\text{Seconds}}{\text{FLOP}}\right)}_{\text{Machine}}$$

where the first two factors multiply to set total floating-point operations O. For serialized execution phases on physical hardware, unpacking how the machine executes those operations gives the three-term iron-law performance decomposition, where each variable is chosen deliberately to avoid collision with standard ML terminology:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}}$$

Symbol	Definition	Unit	Notes
T	**Execution Time**	seconds	Total execution time in the iron-law performance decomposition.
D_{vol}	**Data Volume**	bytes	**Avoids collision with D (Dataset Size)**. In scaling laws and the D·A·M product, D counts logical training samples or tokens. Here we need physical bytes moved through memory. The subscript disambiguates.
BW	**Bandwidth**	bytes/s	**Avoids collision with B (Batch Size)**. Systems literature often uses B for bandwidth, while ML literature often uses B for batch size. The notation preserves the ML convention.
O	**Operations**	FLOPs	Total floating-point operations, the product of Samples/Task and FLOPs/Sample. Clean in equations (vs. "Ops").
R_{peak}	**Peak Rate**	FLOP/s	**Avoids collision with P (Parameters)**. Roofline presentations often use P for performance, while ML literature commonly uses P for parameter count. The notation preserves the ML convention.

Symbol	Definition	Unit	Notes
η_{hw}	**Efficiency**	—	Hardware utilization $(0 \leq \eta_{\text{hw}} \leq 1)$. Measures compute-unit utilization in the three-term decomposition, or end-to-end model FLOPs utilization when data movement and overhead are folded into the single-term training law (chapter 8). Avoids collision with learning rate (η).
L_{lat}	**Latency**	seconds	**Avoids collision with $\mathcal{L}$ (Loss).** Fixed overhead time (kernel launch, network RTT). The subscript distinguishes from the loss function.

Why these choices matter

Without careful notation, sentences become ambiguous:

> *"Reducing D improves performance."*

The sentence has two possible readings:

- Reducing **dataset size** (fewer training samples) can speed training but may reduce accuracy.
- Reducing **data volume moved** (for example, through compression or quantization) can speed inference, with accuracy effects that depend on the technique and calibration.

With our notation, we can write precisely and eliminate this ambiguity so that a statement like *"BW limits throughput"* has only one reading:

> *"Reducing D_{vol} through FP32-to-INT8 quantization cuts parameter memory traffic to one quarter while D (training data) remains unchanged."*

Subscripted variants

The root names the quantity, and an upright subscript names which instance of it is meant. The convention distinguishes closely related quantities:

- BW_{disk}, $\text{BW}_{\text{network}}$, $\text{BW}_{\text{accelerator}}$: bandwidth at three points on the data path
- D_{vol}, R_{peak}, L_{lat}, η_{hw}: iron-law terms, each held clear of a common ML symbol
- E_{move}, E_{compute}, E_{total}: one energy budget split into components

The subscript identifies where a measurement applies. Disk and accelerator bandwidth differ by orders of magnitude, so a reported bandwidth is meaningful only if readers know where it was measured.

The Degradation Equation

The degradation equation is an approximation fitted to observed outcomes over a limited range. Divergence alone does not determine whether accuracy rises or falls, so estimate λ from labeled outcomes. Some symbols below, such as τ, appear in chapter prose rather than in the equation.

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

Symbol	Definition	Type or unit	Notes
$\textbf{Accuracy}(t)$	**Accuracy at Time t**	Scalar	Model accuracy after the model has been deployed for time t.
$\textbf{Accuracy}_0$	**Initial Accuracy**	Scalar	Model accuracy at deployment time.
λ	**Fitted Sensitivity**	Scalar	Local coefficient estimated from outcomes over a stated range; not a universal constant. *(Not wavelength.)*
P_t	**Current Distribution**	Distribution	The data distribution at time t. *(Not parameters—use P for parameter count.)*
P_0	**Training Distribution**	Distribution	The data distribution at training time.
$\mathcal{D}(P_t \| P_0)$	**Statistical Divergence**	Scalar ≥ 0	Measures how far P_t has drifted from P_0. Common choices: KL divergence, total variation, Wasserstein. *(Calligraphic to avoid collision with D = dataset size.)*

Symbol	Definition	Type or unit	Notes
τ	**Response Threshold**	Scalar > 0	Operational threshold for investigation; it does not automatically trigger retraining. The same symbol marks a classifier's decision threshold.
$\mathcal{D}$	**Divergence Measure**	Function	Divergence function that measures how far one probability distribution lies from another, with the specific measure left open.
$\mathcal{D}_{\mathbf{KS}}$	**Kolmogorov Smirnov Statistic**	Dimensionless	Two-sample Kolmogorov-Smirnov statistic, the largest absolute difference between two empirical cumulative distribution functions.
F_{P_0}	**Training Empirical CDF**	Function	Empirical cumulative distribution function of a feature in the training-baseline sample.
$x_{\mathbf{feat}}$	**Monitored Feature Level**	Varies	One value of a monitored feature, the argument at which its distribution or empirical cumulative distribution function is evaluated. The maximization variable of the Kolmogorov-Smirnov statistic.
F_{P_t}	**Serving Empirical CDF**	Function	Empirical cumulative distribution function of a feature in the current serving sample.
$c_{\mathbf{crit}}$	**Kolmogorov Smirnov Critical Threshold**	Dimensionless	Critical threshold that the two-sample Kolmogorov-Smirnov statistic must exceed to reject equal distributions at a given significance level.
n	**Drift Test Sample Size**	Integer	Number of observations in one of the two samples a two-sample drift test compares.
$\mathcal{D}_{\mathbf{KL}}$	**KL Divergence**	Function	Kullback-Leibler divergence, the expected extra code length from encoding one distribution with a code built for another.
H	**Shannon Entropy**	bits or nats	Average uncertainty of a probability distribution, the expected negative log probability of its outcomes. The log base sets the unit: base 2 gives bits, natural log nats.

The Energy Corollary

Effective costs per byte moved and per operation give the following approximation for workload energy. The symbols in the table below separate data-movement energy from arithmetic energy and list the circuit-level quantities that govern dynamic power:

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$$

Symbol	Definition	Unit	Notes
$E_{\mathbf{total}}$	**Total Energy**	joules	Total energy consumed by an ML workload, decomposed into data-movement and compute terms.
$E_{\mathbf{move}}$	**Energy per Byte Moved**	joules/byte	Effective movement cost; total energy also depends on bytes moved and operations executed.
$E_{\mathbf{compute}}$	**Energy per Operation**	joules/FLOP	Energy cost of a single arithmetic operation.
C	**Switched Capacitance**	farads	Capacitance of the wires and gates a circuit charges when it switches.
V	**Supply Voltage**	volts	Supply voltage that powers a digital circuit, the quantity dynamic voltage and frequency scaling lowers.
f	**Clock Frequency**	hertz	Rate at which a processor's clock ticks, setting how many switching cycles occur per second.
$P_{\mathbf{cluster}}$	**Cluster Power**	watts	Power drawn by a training cluster, including accelerators and hosts. Average power multiplied by training time gives training energy.
$P_{\mathbf{device}}$	**Device Power**	watts	Electrical power one device draws. Integrated over time it gives the device's energy.
$P_{\mathbf{dynamic}}$	**Dynamic Power**	watts	Power a chip draws switching its transistors, proportional to switched capacitance, supply voltage squared, and clock frequency.

Symbol	Definition	Unit	Notes
P_{avg}	**Average Power**	watts	Power drawn averaged over a stated period.

Deep Learning Notation

This book follows standard deep learning conventions (Goodfellow et al. 2016) with explicit disambiguation for systems variables. Where standard deep learning symbols would collide with hardware or serving quantities, the table below assigns distinct symbols and states the disambiguation rule explicitly.

Symbol	Definition	Type or unit	Notes
B	**Batch Size**	Integer	Number of examples grouped in a training or inference batch. *(Never bandwidth.)*
P	**Parameter Count**	Integer	The total count of scalar model parameters, including weights and biases. *(Never peak FLOP/s.)*
D	**Dataset Size**	Integer	Number of training samples or tokens. *(Never data volume in bytes—use D_{vol}.)*
S	**Sequence Length**	Integer	Number of tokens or time steps in an input sequence.
d	**Hidden Dimension**	Integer	Size of the hidden state vector of a layer or representation.
d_{in}	**Layer Input Width**	Integer	Number of input features in a layer.
d_{out}	**Layer Output Width**	Integer	Number of output features in a layer.
d_{head}	**Attention Head Dimension**	Integer	Per-head hidden dimension in attention layers.
N_L	**Number of Layers**	Integer	Total number of layers in a network.
N_{heads}	**Number of Attention Heads**	Integer	Number of attention heads in a multi-head attention layer.
H_{KV}	**Number of Key-Value Heads**	Integer	Number of key-value heads in grouped-query or multi-query attention.
ℓ	**Layer Index**	Integer	Index for a layer. Use instead of bare L when indexing layers, since L collides with loss and latency. A numeric subscript marks a norm instead (ℓ_1, ℓ_2).
$\mathcal{L}$	**Loss Function**	Scalar	The objective function minimized during training.
η	**Learning Rate**	Scalar	Step size for the optimizer. *(Never bare for hardware efficiency—use η_{hw}.)*
θ	**Model Parameters**	Vector	All learnable model parameters, treated as a vector.
g	**Gradient**	Vector	Gradient of the loss with respect to the parameters, $\nabla_\theta \mathcal{L}$; a subscript t marks the optimizer step.
$p(x)$	**Distribution**	Probability mass/density	Distribution for a random variable. Use lowercase $p(\cdot)$ for generic distributions to avoid collision with P (parameter count).
$p(y \mid x)$	**Conditional Distribution**	Conditional probability/density	Use this form for generic label relationships; reserve P_0 and P_t for the training/current distributions of drift analysis.
$\hat{\mathbf{y}}$	**Predicted Output**	Vector	A model's output for one example, such as class probabilities; $\mathbf{y}$ is the label vector.
T_{distill}	**Distillation Temperature**	Dimensionless	Softmax temperature in knowledge distillation. It is dimensionless, unlike execution time.
$\Pr(E)$	**Event Probability**	Probability	Probability of an event E. Use for event statements such as $\Pr(\text{batch} = 0)$; use $p(x)$ and $p(y \mid x)$ for distributions.
O_{train}	**Training Compute**	FLOPs	Total floating-point operations required for model training.
y	**Example Label**	Random variable	The label paired with one input, treated as a random variable.
x	**Model Input**	Random variable	Input of one example, the data a model receives to produce a prediction.

Symbol	Definition	Type or unit	Notes
Δ	**Finite Change**	Operator	Finite change in the quantity it prefixes, between two states, configurations or points in time. Prefix operator, as in the return-on-compute ratio of accuracy change to compute-cost change.
k	**Batch Index**	Index	Position of one batch in the sequence delivered by a data loader or runtime.
p	**Probability Function**	Function	Probability mass or density function of a random variable, applied to its argument.
H	**Image Height**	Integer	Number of pixel rows in an image or its segmentation mask.
W	**Image Width**	Integer	Number of pixel columns in an image or its segmentation mask.
w	**Model Weight**	Dimensionless	Value of one learnable weight of a model, such as the weight on one input of a neuron.
x	**Layer Input Component**	Dimensionless	One component, or feature, of a layer's input vector.
$\mathbf{x}$	**Input Feature Vector**	Vector	Input feature vector of one example, written as a row vector.
b	**Neuron Bias**	Dimensionless	Bias term added to one neuron's weighted sum. Indexed by neuron: b_j.
z	**Neuron Preactivation**	Dimensionless	Preactivation of one neuron, its weighted input sum plus bias before the activation function. At the output layer, z_j is the logit for one class, the argument of softmax.
f	**Activation Function**	Function	Activation function of a neuron or layer, applied element-wise to the preactivation. A layer superscript names the layer (f^{(l)}); f′ is its derivative.
$\mathbf{w}$	**Weight Vector**	Vector	Weight vector of one neuron, one weight per input.
$\hat{y}$	**Predicted Output Component**	Dimensionless	One entry of a model's prediction, such as the predicted probability for one class.
$\mathbf{W}$	**Weight Matrix**	Matrix	Matrix of learned weights one layer multiplies its input by, or the kernel tensor of a convolution layer.
x	**Activation Input**	Dimensionless	Scalar argument an activation function is applied to in its defining formula.
σ	**Sigmoid Function**	Function	Logistic sigmoid function, one over one plus e to the minus x. Maps any real input to the open interval (0, 1).
K	**Class Count**	Integer	Number of classes a classifier distinguishes, or vocabulary entries it scores, the length of its softmax output.
$\mathbf{z}$	**Preactivation Vector**	Vector	Preactivation vector of one layer for one example, the logit vector at the output layer.
$\mathbf{b}$	**Bias Vector**	Vector	Vector of learned biases one layer adds to its weighted sums, one per neuron or output channel. Broadcast across the batch.
n	**Layer Width**	Integer	Number of units in the layer at a given position of the network, with n_0 the input width. Indexed by layer position.
$\mathbf{a}$	**Activation Vector**	Vector	Activation vector of one layer for one example.
$\mathbf{A}$	**Activation Matrix**	Matrix	Activation matrix of one layer for a batch, one example per row.
$\mathbf{y}$	**Label Vector**	Vector	True label of one example written as a vector, such as a one-hot class vector.
f	**Model Predictor**	Function	Trained model viewed as a function that maps an input to its prediction.
$\mathbf{Z}$	**Preactivation Matrix**	Matrix	Preactivation matrix of one layer for a batch, one example per row.
$\mathbf{X}$	**Layer Input Matrix**	Matrix	Input to a layer in matrix form, one row per example or token, or a single-channel image.
j	**Class Index**	Index	Index of an output class.
C_{in}	**Input Channel Count**	Integer	Number of channels in the feature map a convolution layer reads.
K	**Kernel Size**	Integer	Side length, in input positions, of a square convolution kernel.
$\mathbf{h}$	**Hidden Activation**	Vector	Activation vector a hidden layer outputs and passes to the next layer.

Symbol	Definition	Type or unit	Notes
C_{out}	**Output Channel Count**	Integer	Number of channels, one per filter, in the feature map a convolution layer produces.
f	**Layer Function**	Function	Function computed by a layer, operator, or recurrent cell, mapping inputs to outputs.
$\mathbf{x}$	**Layer Input Vector**	Vector	Vector of activations one layer receives as input.
H_{img}	**Feature Map Height**	Integer	Height, in pixels, of the image or feature map a convolution layer reads.
W_{img}	**Feature Map Width**	Integer	Width, in pixels, of the image or feature map a convolution layer reads.
H_{out}	**Output Feature Map Height**	Integer	Number of output rows a convolution produces.
W_{out}	**Output Feature Map Width**	Integer	Number of output columns a convolution produces.
$\mathbf{Q}$	**Attention Query Matrix**	Matrix	Matrix of query vectors, one row per token, in an attention layer.
$\mathbf{K}$	**Attention Key Matrix**	Matrix	Matrix of key vectors, one row per token, in an attention layer.
$\mathbf{V}$	**Attention Values Matrix**	Matrix	Matrix of value vectors, one row per token, that attention weights combine into the output.
$\mathbf{A}$	**Attention Weight Matrix**	Matrix	Matrix of attention weights, the row-wise softmax of scaled query-key scores.
$\mathbf{I}$	**Identity Matrix**	Matrix	Square matrix with ones on the diagonal and zeros elsewhere. Size follows from context.
$\mathcal{B}$	**Mini Batch Set**	Set	Set of examples processed together as one mini-batch.
γ	**Normalization Scale**	Dimensionless	Learned factor a normalization layer multiplies each normalized activation by.
$\boldsymbol{\gamma}$	**Normalization Scale Vector**	Vector	Vector of learned per-channel or per-feature factors a normalization layer multiplies its normalized output by.
$\boldsymbol{\beta}$	**Normalization Shift Vector**	Vector	Vector of learned per-channel or per-feature offsets a normalization layer adds to its scaled output.
v	**Trace Intermediate Variable**	Dimensionless	Intermediate variable in the evaluation trace of a differentiated program, its primal value.
$\bar{v}$	**Trace Intermediate Adjoint**	Dimensionless	Reverse-mode adjoint of an intermediate variable, the derivative of the output with respect to it.
$\mathbf{x}$	**Function Input Vector**	Vector	Vector a layer or differentiated function takes as input.
N	**Function Input Dimension**	Integer	Number of inputs of a differentiated function, the column count of its Jacobian.
x	**Function Input Element**	Dimensionless	One element of the input vector of a layer or elementwise function.
i	**Matrix Row Index**	Index	Position of a row within a matrix.
j	**Matrix Column Index**	Index	Column index of a matrix entry.
$\mathbf{S}$	**Attention Score Matrix**	Matrix	Matrix of attention scores, the scaled dot products of every query with every key before the softmax.
$\mathbf{x}$	**Softmax Input Vector**	Vector	Vector of scores an online softmax normalizes block by block.
m	**Softmax Running Maximum**	Dimensionless	Largest score seen so far in a block of the online softmax, subtracted before exponentiation. Superscripts (1), (2), new mark the block it covers.
z	**Softmax Normalizer**	Dimensionless	Sum of the max-shifted exponentials of a softmax row or block, the softmax denominator, kept as a running sum by online softmax. Not the class logit z of the data appendix.
$\mathbf{O}$	**Attention Head Output**	Matrix	Output of one attention head before the output projection, the attention weights applied to the value matrix. Distinct from the output-projection result of a whole attention layer.
β	**Data Scaling Exponent**	Dimensionless	Fitted power-law exponent at which loss falls as the training dataset or token count grows.
d	**Derivative Operator**	Operator	Differential operator d used in an ordinary derivative.

Symbol	Definition	Type or unit	Notes
t	**Optimizer Step Index**	Index	Position of one optimizer update in the sequence of training steps.
N_{epochs}	**Epoch Count**	Integer	Number of full passes a training run makes over its dataset. Training work scales with it times dataset size.
z	**Output Logit**	Dimensionless	Raw score a model outputs for one class before the softmax.
r	**Input Resolution**	Integer	Input image resolution of a convolutional model. Subscript 0 marks the base model in compound scaling.
s	**Quantization Scale**	Varies	Real-valued step between adjacent integer levels of a uniform quantizer.
x_q	**Quantized Integer Code**	Integer	Integer code a quantizer assigns to one real value.
W	**Layer Weight Entry**	Dimensionless	One scalar entry of a layer's weight matrix, picked out by a layer superscript and row and column subscripts.
W_{ij}	**Weight Matrix Entry**	Dimensionless	One entry of a weight matrix at a given row and column; italic because it is a scalar.
$\mathbf{W}_{\text{fused}}$	**Fused Convolution Weight**	Dimensionless	Convolution weights after the following batch normalization is folded into them, so inference runs one layer instead of two.
x	**Quantizer Input**	Varies	Full-precision real value, a weight or activation, that a quantizer maps to an integer code. Unit: units of the quantized tensor.
α	**Clipping Range Lower Bound**	Varies	Lower end of the real-valued range a quantizer represents, below which values are clipped. Calibration estimates it.
β	**Clipping Range Upper Bound**	Varies	Upper end of the real-valued range a quantizer represents, above which values are clipped.
b	**Quantization Bit Width**	Integer	Number of bits in each integer code a quantizer produces.
$\sigma_{\mathcal{B}}$	**Batch Norm Running Standard Deviation**	Vector	Per-channel square root of the running variance plus the numerical stabilizer that batch normalization uses at inference.
n_{keep}	**Structured Sparsity Nonzero Count**	Integer	Number of values kept nonzero in each group of an N:M structured-sparsity pattern.
m_{group}	**Structured Sparsity Group Size**	Integer	Number of consecutive values in one group of an N:M structured-sparsity pattern.
K_h	**Kernel Height**	Integer	Height of a convolution kernel, in input positions.
K_w	**Kernel Width**	Integer	Width of a convolution kernel, in input positions.
k	**Top Class Count**	Integer	Number of highest-scoring classes a top-k prediction returns or checks for the correct label.
μ	**Service Rate**	items/s	Rate at which one server, such as a worker, consumer, or model endpoint, completes items or calls.
$\hat{y}$	**Model Prediction**	Dimensionless	Prediction a model outputs for one example, compared with its label by the loss.
$\mathbf{Y}$	**Layer Output Matrix**	Matrix	Output matrix of a layer's matrix multiplication of the input matrix by the weight matrix, one row per example. Shape batch size by layer output width; before any bias or activation function.
m	**Softmax Row Maximum**	Dimensionless	Largest score in a softmax row or block, subtracted before exponentiation to prevent overflow.
x	**Softmax Input Score**	Dimensionless	One entry of the score vector the online softmax normalizes. Written with an index and a block superscript, $x_j^{(1)}$.
α_{sym}	**Symmetric Clipping Threshold**	Varies	Largest magnitude a symmetric quantizer represents, so values are clipped to plus or minus this bound.

Latin matrices and vectors are set in bold (**W**, **x**); scalars, dimensions, indices, and individual matrix entries stay italic (N, d, i, W_{ij}). The generic parameter symbol θ is the one conventional exception and stays italic.

Local matrix algebra follows the same bold rule. Matrix operands are bold (**A**, **B**, **C**), as in general matrix multiply (GEMM) $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$. The bold form marks a matrix operand and is distinct from the italic scalar batch size B, which remains batch size in scalar contexts.

A GEMM's scalar dimensions are M, N, and K (**A** is $M \times K$, **B** is $K \times N$). Asymptotic cost is written $\mathcal{O}(\cdot)$, which keeps it distinct from the operation count O.

Performance, Serving, and Memory Notation

Reusable performance and serving quantities follow the same collision-avoidance rule as the iron law. Rates use R or descriptive Greek symbols; request counts use Q_{req} rather than overloading N; queue utilization uses a subscripted ρ so bare ρ remains available for other ratio models.

Symbol	Definition	Type or unit	Notes
I	**Arithmetic Intensity**	FLOP/byte	Workload FLOPs per byte moved. The Roofline Model uses I as the independent variable.
I_{ridge}	**Roofline Ridge Point**	FLOP/byte	$I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$. Prefer this explicit form over starred shorthand so the meaning remains clear in prose.
R_{attain}	**Attainable Compute Rate**	FLOP/s	Roofline bound: $R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$. Uses R, not T, because this quantity is a rate.
MFU	**Model FLOPs Utilization**	Dimensionless	Useful model FLOP/s divided by available peak FLOP/s. Text acronym avoids overloading η.
Q_{req}	**Request Concurrency**	requests	Average in-flight requests in a stable serving system ($Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}}$ via Little's Law). Avoids overloading N, which counts devices, elements, or samples elsewhere.
λ_{arr}	**Arrival Rate**	requests/s	Request arrival rate. The subscript keeps bare λ free for fitted sensitivities.
T_{memory}	**Iron-Law Memory Time**	seconds	Data-movement term D_{vol}/BW in the iron law.
T_{compute}	**Iron-Law Compute Time**	seconds	Compute term $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ in the iron law.
N_{GPU}	**Accelerator Count**	Integer	Number of accelerators in one job.
T_{lat}	**Request Time in System**	seconds	Average time a request spends queueing and being served, as used in Little's law. Distinct from L_{lat}, the fixed-latency term in the iron law.
$L_{\text{lat,total}}$	**End-to-End Request Latency**	seconds	Request-path latency budget, $L_{\text{lat,network}} + L_{\text{lat,pre}} + L_{\text{lat,infer}} + L_{\text{lat,post}} + L_{\text{lat,queue}}$. T_{lat} covers only the time inside the serving system.
$T_{\text{svc}}(B)$	**Batch Service Time**	seconds	Time to serve a batch of size B.
$\mu_{\text{eff}}(B)$	**Effective Service Rate**	requests/s	Batched service rate, typically $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$. The unbatched service rate is $\mu = \mu_{\text{eff}}(1)$.
ρ_{serv}	**Serving Utilization**	Dimensionless	Queue/server utilization. The subscript keeps bare ρ free for other ratio models.
M_{total}	**Total Memory Footprint**	bytes	Sum of explicit memory components; avoids bare M ambiguity.
M_{weights}	**Weight Memory**	bytes	Memory occupied by model weights ($P \times$ bytes-per-parameter).
$M_{\text{gradients}}$	**Gradient Memory**	bytes	Memory occupied by stored gradients.
$M_{\text{optimizer}}$	**Optimizer-State Memory**	bytes	Momentum, variance, master weights, and related optimizer buffers.
$M_{\text{activations}}$	**Activation Memory**	bytes	Retained activations for the backward pass or serving intermediates.
M_{KV}	**KV-Cache Memory**	bytes	Memory occupied by the key-value cache during autoregressive inference.
s_{elem}	**Element Storage Size**	bytes/element	Bytes per stored tensor element.
s_{sample}	**Sample Storage Size**	bytes/sample	Bytes per stored training or serving sample.
s_{loss}	**Loss-Scale Factor**	Dimensionless	Multiplier applied to the loss in reduced-precision training so small gradients stay representable.
$T_{\text{pipelined}}$	**Pipelined Execution Time**	seconds	Ideal steady-state execution time when data movement and computation overlap, approximated by the slower stage plus fixed latency.
G	**Accelerated Fraction Speedup**	Dimensionless	Speedup factor applied to the accelerated fraction of a workload in Amdahl's law, whether it comes from faster hardware or from more workers.
T_{train}	**Training Time**	seconds	Wall-clock time to complete one training run.
T_{IO}	**I/O Transfer Time**	seconds	Time to move a workload's data over a storage or network link, the input/output stage of a pipelined step. Overlaps with compute in the pipelined iron law; the step is I/O bound when it is the largest stage.

Symbol	Definition	Type or unit	Notes
$\mathbf{BW_{network}}$	**Network Bandwidth**	bytes/s	Sustained rate at which a network path delivers bytes between machines.
$L_{\text{lat,network}}$	**Network Latency**	seconds	Time a request and its response spend crossing the network, including propagation, routing, and protocol handling.
M_{device}	**Device Memory Capacity**	bytes	Total memory a device provides for a model's weights and activations.
G_{accel}	**Accelerator Gain**	Dimensionless	Speedup an optimization or accelerator gives the fraction of execution it applies to, in Amdahl's law.
a	**First Floating Point Addend**	Dimensionless	First of three floating-point numbers added in either grouping to show that floating-point addition is not associative.
b	**Second Floating Point Addend**	Dimensionless	Second of three floating-point numbers added in either grouping to show that floating-point addition is not associative.
c	**Third Floating Point Addend**	Dimensionless	Third of three floating-point numbers added in either grouping to show that floating-point addition is not associative.
$\mathbf{BW_{storage}}$	**Storage Bandwidth**	bytes/s	Sustained rate at which the storage system delivers bytes to the input pipeline.
T_{step}	**Training Step Time**	seconds	Wall-clock time of one training step, bounded below by the slower of compute and input I/O.
N	**Square GEMM Dimension**	Integer	Side length of the square matrices in a matrix multiply, the shared row, column, and inner dimension.
M	**GEMM Row Dimension**	Integer	Number of rows of the left operand and of the output in a matrix multiply.
K	**GEMM Inner Dimension**	Integer	Shared inner dimension of a general matrix multiply, the number of products summed per output element.
N	**GEMM Column Dimension**	Integer	Number of columns of the right operand and of the output in a matrix multiply.
µ	**Micro Unit Prefix**	Operator	SI unit prefix micro, one millionth, written before a unit such as seconds.
N	**Tensor Element Count**	Integer	Number of elements in one tensor.
n_{ops}	**Chained Operation Count**	Integer	Number of consecutive operations that read and write the same intermediate tensor and can be combined through kernel fusion.
T_{transfer}	**Batch Transfer Time**	seconds	Time to copy one batch of input data from host memory to the device that consumes it.
$\mathbf{C}$	**GEMM Output Matrix**	Matrix	Output matrix of a general matrix multiply, of size M by N, which the operation updates in place.
α	**GEMM Product Scale**	Dimensionless	Scalar multiplier applied to the matrix product in a general matrix multiply.
β	**GEMM Accumulator Scale**	Dimensionless	Scalar multiplier applied to the existing output matrix before the product is added in a general matrix multiply.
$\mathbf{A}$	**GEMM Left Operand**	Matrix	Left input matrix of a general matrix multiply, of size M by K.
$\mathbf{B}$	**GEMM Right Operand**	Matrix	Right input matrix of a general matrix multiply, of size K by N.
$\mathbf{BW_{disk}}$	**Disk Bandwidth**	bytes/s	Sustained rate at which a storage disk delivers bytes.
$R_{\text{sustained}}$	**Sustained Compute Rate**	FLOP/s	Operation rate an accelerator or system actually sustains on a workload.
T_{prep}	**Host Preparation Time**	seconds	Time the host takes to prepare one batch for the accelerator, from read through batching.
$\mathbf{BW_{accelerator}}$	**Accelerator Memory Bandwidth**	bytes/s	Sustained rate at which bytes move between an accelerator's own memory, such as HBM, and its compute units.
t	**Time Instant**	seconds	A point in continuous wall-clock time, the variable over which time-varying quantities are integrated.
$L_{\text{lat,infer}}$	**Inference Latency**	seconds	Time the model takes to compute a request's output.
M_{tensor}	**Intermediate Tensor Size**	bytes	Bytes in the intermediate tensor that a sequence of operations passes between its steps.
n_{nz}	**Nonzero Count**	Integer	Number of nonzero values a sparse matrix format stores.
n_{rows}	**Sparse Matrix Row Count**	Integer	Number of rows in a sparse matrix. Compressed sparse row (CSR) storage uses one more row pointer than the number of rows.
T_{token}	**Decode Token Time Floor**	seconds	Lower bound on the time to generate one token at batch one, bytes streamed per step divided by memory bandwidth.
$L_{\text{lat,queue}}$	**Queueing Latency**	seconds	Time a request waits in queues before it is served.

Symbol	Definition	Type or unit	Notes
$L_{\text{lat,pre}}$	**Preprocessing Latency**	seconds	Time spent preparing a request's input before inference.
$L_{\text{lat,post}}$	**Postprocessing Latency**	seconds	Time spent turning a model's output into the response.
b	**Numeric Bit Width**	Integer	Number of bits used to represent one numeric value, such as an arithmetic operand or a quantized weight or activation.
T	**Dataset Transfer Time**	seconds	Ideal line-rate time to move a dataset over a network or storage path: data volume divided by bandwidth.
T_{GPU}	**Accelerator Batch Time**	seconds	Time the accelerator takes to run one training step on one batch, excluding input preparation.

Statistical and Operational Notation

Statistical thresholds, workload fractions, and system constraints follow standard statistical and operational conventions. Each symbol in the table below carries an explicit domain and unit so that hypothesis tests, queueing bounds, and fairness metrics can be combined without ambiguity:

Symbol	Definition	Type or unit	Notes
κ	**Cohen's Kappa**	Dimensionless	Inter-annotator agreement coefficient $\kappa \in [-1, 1]$.
T_{deadline}	**Deadline**	seconds	Latency deadline for query completion or batch execution.
f_{overhead}	**Operational Overhead Fraction**	Dimensionless	Fraction of wall time spent outside productive step execution (f_{overhead}).
f_{parallel}	**Parallel Fraction**	Dimensionless	Fraction of workload execution that can be parallelized in Amdahl's Law ($0 \leq f_{\text{parallel}} \leq 1$).
α	**Significance Level**	Dimensionless	Statistical significance threshold α for hypothesis tests.
ϵ	**Numerical Stability Constant**	Scalar	Small numerical constant $\epsilon > 0$ for numerical stability.
N_{workers}	**Worker Count**	Integer	Number of parallel worker threads, processes, or devices.
t_{obs}	**Observation Timestamp**	seconds	Time at which a labeled observation, such as a churn event, occurred. Upper bound on the feature timestamps an as-of join may use.
C_{error}	**Erroneous Prediction Cost**	dollars	Average business cost incurred by one erroneous prediction. Multiplies the skew rate and query volume in the training-serving skew cost.
F_{access}	**Storage Access Factor**	Dimensionless	Fraction of peak storage throughput a workload's access pattern achieves.
g	**Subgroup Index**	Index	Label of one population subgroup whose outcomes are compared against a reference group.
k	**Feedback Interval Count**	Integer	Number of fixed feedback intervals, such as retraining cycles, elapsed since the disparity was first measured.
α_{fb}	**Bias Feedback Factor**	Dimensionless	Factor by which a group's disparity is multiplied over one feedback interval, fitted from observed disparities. Above 1 the feedback loop is self-reinforcing; at or below 1 it is stable or damped.
Δ_g	**Group Disparity**	Dimensionless	Size of the performance gap between one group and a reference group; the feedback analysis tracks how it grows from one feedback interval to the next.
k_{anon}	**Anonymity Set Size**	Integer	Smallest number of records that share every quasi-identifier value in a k-anonymous release.
p_{deploy}	**Deployment Distribution**	Distribution	Joint distribution of inputs and labels a deployed model must serve, the population a verification guarantee is stated over.
d_{task}	**Task Distance**	Function	Task-specific distance between a model's prediction and the correct label, the error measure a verification guarantee bounds.
τ_{tol}	**Task Error Tolerance**	Dimensionless	Largest task distance between a prediction and its label that still counts as an acceptable output.

Symbol	Definition	Type or unit	Notes
ϵ	**Tolerated Failure Probability**	Dimensionless	Maximum allowed probability that a deployed model exceeds its task error tolerance. The confidence level used to estimate this probability is specified separately.
S_{skew}	**Training Serving Skew**	Dimensionless	Expected absolute difference between the serving-path and training-path model scores on the same inputs, over a stated comparison-input distribution. Measurable without labels; a risk signal, not an accuracy-loss equation.
f_{serve}	**Serving Path Predictor**	Function	Serving code path viewed as a function from a raw input to the model's output, including its preprocessing and runtime.
f_{train}	**Training Path Predictor**	Function	Training code path viewed as a function from a raw input to the model's output, as offline validation evaluates it.
q	**Target Completion Quantile**	Dimensionless	Required probability that a request completes within its latency deadline, the completion quantile a service level states.

Units and Precision

- **Physical units**: This book uses SI (metric) units throughout, including meters, kilograms, seconds, watts, and °C, consistent with standard engineering and scientific practice. Where source data was originally reported in imperial units, the book converts to SI and notes the original values parenthetically. A space always separates the number from the unit (for example, 100 ms, 2 TB/s).
- **Data and memory**: This book uses **decimal SI prefixes only**: KB = 10^3 bytes, MB = 10^6, GB = 10^9, TB = 10^{12}. Binary-prefixed units do not appear in prose; model sizes, activation footprints, throughputs, and transfer volumes are reported in decimal units (for example, 2 TB/s, 102 MB). Hardware memory capacities retain the vendor's nameplate figure and label. Calculations requiring an exact byte count use the device's documented capacity.
- **Compute**: The notation distinguishes operation counts from rates. Total work uses FLOPs (for example, GFLOPs, TFLOPs), while throughput uses FLOP/s with decimal prefixes (for example, GFLOP/s, TFLOP/s).
  - 1 TFLOP = 10^{12} FLOPs
  - 1 TFLOP/s = 10^{12} FLOPs per second
  - Arithmetic intensity conventionally uses FLOP/byte as a unit ratio (floating-point operations per byte moved). This is a ratio unit, not a total-work symbol or throughput symbol.

- **Power**: Power carries a subscript (P_{avg}, P_{device}, P_{cluster}); bare P remains parameter count.
- **Latency percentiles**: Written p50, p95, and p99 in upright type, inside math as well as in prose.
- Currency: Dollar amounts use the dollar sign (`$`); unless otherwise noted, dollar-denominated costs are U.S. dollars (USD).
- **Precision**:
  - **FP64**: Double precision (8 bytes)
  - **FP32**: Single precision (4 bytes)
  - **TF32**: TensorFloat-32 (19-bit Tensor Core compute mode; inputs and storage remain FP32)
  - **FP16**: Half precision (2 bytes, standard range)
  - **BF16**: Brain float (2 bytes, wide dynamic range)
  - **FP8**: Quarter precision (1 byte, E4M3 or E5M2 format)
  - **FP4**: 4-bit floating-point format
  - **INT8**: 8-bit integer (1 byte)
  - **INT4**: 4-bit integer; lower integer precisions follow the same uppercase `INTn` pattern (for example, INT3, INT2)

Quick Reference: Resolving Collisions

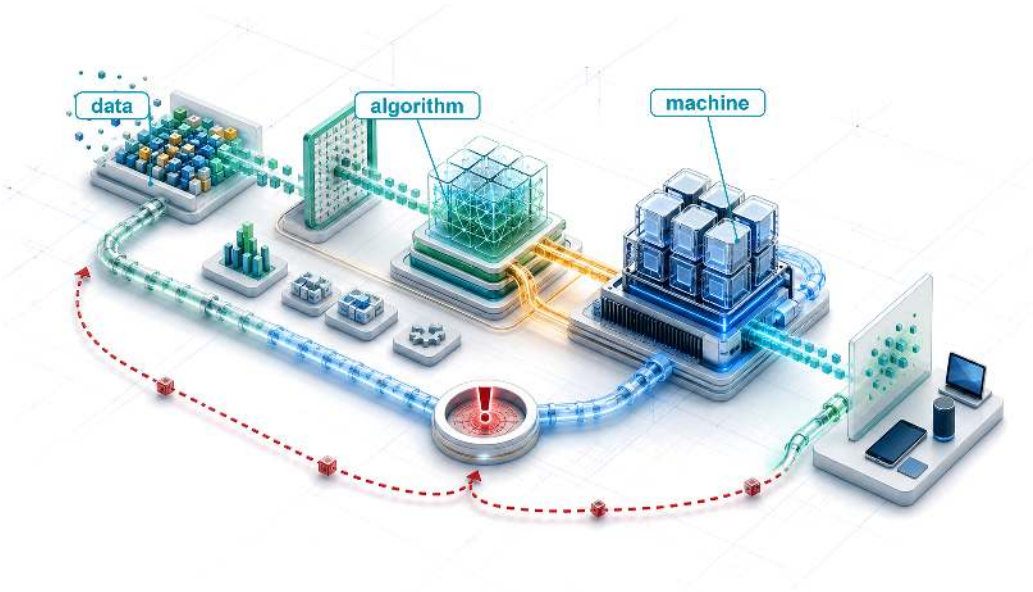
ML conventions generally take precedence for single letters, while systems quantities use subscripts or descriptive symbols. This preserves familiar ML notation while distinguishing systems quantities across the most common notation collisions in machine learning systems literature:

Symbol	ML Meaning	Systems Meaning	Book Convention
B	Batch Size	Bandwidth	**Batch Size**. Use BW for bandwidth.
P	Parameter Count	Peak FLOP/s	**Parameter Count**. Use R_{peak} for peak rate.
D	Dataset Size	Data Volume	**Dataset Size**. Use D_{vol} for bytes moved.
L	Loss	Latency	**Loss** ($\mathcal{L}$). Use L_{lat} for latency.
η	Learning Rate	Efficiency	**Learning Rate**. Use η_{hw} for efficiency.

MAIN TEXT

1

Introduction

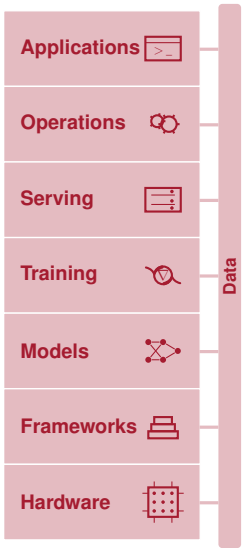


1.1 The AI Systems Moment
1.2 From Code-Defined Logic to Data-Defined Behavior
1.3 The Evolution of AI Bottlenecks
1.4 The Bitter Lesson
1.5 Defining ML Systems
1.6 When ML Systems Fail Silently
1.7 Iron Law of ML Systems
1.8 Three Dimensions of ML Efficiency
1.9 AI Engineering as a Discipline
1.10 Book Organization
1.11 Fallacies and Pitfalls
1.12 Summary

Purpose

Why does an accurate machine learning model not guarantee a reliable, efficient system?

Machine learning systems have a physics. Data moves through memory hierarchies governed by bandwidth, arithmetic executes on silicon governed by power, and predictions must arrive within strict latency windows. These constraints are not secondary implementation details. Moving an operand from main memory can cost far more energy than performing arithmetic on it, and a prediction that arrives a few hundred milliseconds late can be worth nothing to the real-time process waiting for it. An architecture that fits comfortably in a data center is useless on a mobile phone, and a training pipeline that converges in days on high-bandwidth accelerators can stall for weeks on standard hardware. These systems also diverge from conventional software because empirical data, rather than explicit logic alone, defines their behavior. When a conventional program misbehaves, it often crashes or throws an exception. When an ML system misbehaves, the runtime may operate flawlessly while learned predictions fail because the input distribution has shifted or the training data was unrepresentative. No assertion fails and no exception is raised; the only symptom is deteriorated decisions, discovered weeks later through lost revenue, erroneous predictions, or compromised safety. Testing, version control, and profiling remain necessary but are no longer sufficient. A unit test confirms that code computes what it was written to compute, but it cannot establish that learned behavior still matches a non-stationary world, and optimizing one component in isolation frequently moves the bottleneck to another layer of the stack. In D·A·M terms, data, algorithm, and machine must be engineered together as one problem.



Learning Objectives

- Explain why data-defined behavior lets predictive quality decline while model code and execution remain unchanged
- Apply a data-algorithm-machine lens to diagnose a system bottleneck and reassess it after an intervention
- Calculate execution time with the iron law and assess how a component improvement changes end-to-end time
- Analyze when scalable search and learning can benefit from additional computation within data and machine constraints
- Compare algorithmic, compute, and data-selection efficiency by the iron-law term each changes and the accuracy gained per added cost
- Evaluate whether predictive quality and resource use meet deployment requirements using evaluation and lifecycle monitoring

1.1 The AI Systems Moment

Artificial intelligence is no longer confined to research demonstrations. When a user queries a smartphone voice assistant, learned components convert speech to text, interpret intent, retrieve information, and generate a response within seconds. Search engines rank results, recommendation systems decide what people see, lenders use models to assess risk, and driver-assistance systems detect hazards. In each case, a prediction participates in a larger decision that can affect attention, money, access, or safety. Even a single voice query is therefore a chain of models and services operating in the world.

Modern AI systems became practical when three forces converged: digital services generated large datasets, learning algorithms extracted predictive structure from those examples, and parallel hardware made the resulting computation feasible. Yet the most consequential change was conceptual. Data stopped being merely an input to software; it became a mechanism defining program behavior. Instead of hand-coding every decision rule, engineers build a training process through which examples shape the behavior a system learns.

This convergence creates a **dual mandate**. Every ML system must establish that its learned behavior is trustworthy and that the machine can produce that behavior within the available time, memory, energy, and cost. A model that is accurate but too slow is unusable; a fast model trained on unrepresentative data is wrong at machine speed. At scale, both obligations span the entire stack. Conversational services coordinate pools of GPUs[1] while managing memory, networks, and cooling. Driver-assistance systems must fuse sensor streams within milliseconds. A search service processing more than 13.7B queries per day (Srinivasan 2025) must also meet strict latency targets. These systems succeed only when learned behavior and physical execution are engineered together. That co-design begins with the shift from code-defined logic to data-defined behavior.

1.2 From Code-Defined Logic to Data-Defined Behavior

An email spam filter built from hand-written rules fails as soon as attackers rephrase their messages or disguise forbidden keywords. Instead of encoding those decision rules directly into source code, an ML team pairs historical emails with binary spam labels and runs an optimization procedure that adjusts millions of numerical weights[2] until the model's predictions align with the labels. Live inference[3] then applies those trained weights to incoming messages whose labels are unknown, and evaluation against a separate test set checks whether the learned decision boundary generalizes reliably beyond the training corpus.

Andrej Karpathy[4] framed this transition from code-defined logic to data-defined behavior as the shift from **Software 1.0** to **Software 2.0** (Karpathy 2017). As summarized in table 1.1, Software 1.0 encodes procedural logic in human-written instructions, whereas Software 2.0 searches a high-dimensional parameter space for a model configuration that satisfies an empirical objective.

Software 2.0 does not eliminate traditional software engineering. Engineers still write data ingestion pipelines, loss functions, distributed training loops, and low-latency serving harnesses.

[1] **GPU (graphics processing unit)**: Originally designed to render graphics by performing many calculations in parallel. Neural networks also require large numbers of similar arithmetic operations, especially matrix multiplications, making this hardware useful for training. Keeping those operations supplied with data remains a separate engineering challenge.

[2] **Model weights**: The learned numerical parameters of a neural network. They must be stored and accessed during execution, so their number and numerical representation affect memory use, data movement, and cost.

[3] **Inference**: Applying a trained model to inputs, whether during evaluation or deployment. Interactive serving often imposes a response-time limit, whereas offline inference may prioritize the total number of examples processed per second. These workloads can require different execution choices even for the same model (see chapter 13).

[4] **Andrej Karpathy**: A founding member of OpenAI and former director of AI at Tesla. His "Software 2.0" essay described neural network weights as a new form of "source code" (Karpathy 2017). The analogy shifts debugging attention toward the examples that shaped those weights and the conditions under which the model is evaluated.

However, the core operational logic no longer resides in human-readable control flow, but in millions of numerical weights shaped by data. Static analysis and manual code review cannot reveal what decision the system will make for a novel input. Both paradigms can suffer from undetected faults, but learned systems add a distinct failure mode in which predictive accuracy falls silently even when source code, compiler binaries, and execution environments remain unchanged.

Table 1.1: The Paradigm Shift from Software 1.0 to Software 2.0: In Software 2.0, the "programmer" curates the dataset and designs the training process that shapes learned behavior. Debugging expands to include the data. The "compiler" analogy is approximate: unlike a deterministic compiler, the training process is stochastic and may produce different learned weights from the same training data.

Feature	Software 1.0 (Traditional)	Software 2.0 (Machine Learning)
Source Code	C++, Python, Java	Learned weights, shaped by training data
Compiler	GCC, LLVM	Training loop (stochastic gradient descent)
Logic	Explicit (Hand-coded)	Implicit (Learned)
Failure Mode	Crashes or silent logic errors	Also statistical quality loss
Debugging	Trace execution and inputs	Also inspect data and learned behavior

The data-centered workflow changes what engineers must build and maintain. Sculley and colleagues described how model code occupies only a small fraction of a production ML system (Sculley et al. 2015). The surrounding system includes data collection, verification, feature extraction, resource management, serving, and monitoring, each with its own maintenance obligations.

That imbalance creates hidden technical debt, the accumulating cost of maintaining undocumented dependencies and brittle infrastructure around the model. Each surrounding component encodes assumptions about how an example is sampled, what a label means, when a feature is computed, which version reaches serving, and how degradation is detected. None of those assumptions appears in the matrix multiplication that produces a prediction, yet any of them can change the result. Improving the model alone cannot repair a measurement that no longer tracks its target, a broken feature pipeline, or a missing feedback signal.



Production ML work is mostly the surrounding system, not the model code alone.

Production data is therefore not a passive input. It is a measurement of the world, collected through a particular product and transformed by a changing pipeline. If that measurement stops representing the intended target, the model can remain numerically healthy while the system becomes wrong. Google Flu Trends had an enormous, timely stream of search data, but the meaning of that data changed beneath the model.

War Story 1.1: When search logs mistook attention for illness (2013)

Context: Google Flu Trends (GFT) estimated influenza activity from patterns in aggregated search queries (Ginsberg et al. 2009; Lazer et al. 2014). Its attraction came from the speed and scale at which searches arrived relative to clinical reports. If people searched for flu symptoms when they became ill, query volume appeared to offer a timely proxy, or indirect measurement, for infection.

Failure mode: The proxy was not stable. News coverage could change what people searched for, and autocomplete could change how they expressed those searches. Query volume could therefore reflect public attention and product behavior as well as illness. During the 2012–2013 season, GFT estimated roughly twice the CDC-reported proportion of doctor visits for influenza-like illness, and from August 2011 onward it overestimated in 100 of 108 weeks.

Systems lesson: The remedy was not simply a larger search dataset or a better-tuned model. Researchers combined search signals with U.S. Centers for Disease Control and Prevention (CDC) sentinel clinical data and recalibrated the relationship over time. Data volume is not ground truth. A behavioral proxy needs a feedback loop to a trusted measurement and continual evidence that the proxy still represents the quantity the system claims to estimate.

In Google Flu Trends, predictive quality deteriorated while the software kept operating. The input distribution, meaning the kinds of inputs and how frequently they occur, and the relationship between proxy and target can change. Under the **data-as-code principle**, training data does not merely enter a fixed program; it helps determine the operational logic that the model implements. Engineers still write the optimization procedure, but the examples shape the model weights through stochastic gradient descent[5] and related methods. Changing the dataset can therefore change system behavior as surely as changing source code.

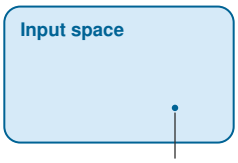
Data-defined behavior is a property of machine learning; *data-centric* AI is a development strategy for improving that behavior (Ng 2021). A model-centric approach holds the data comparatively fixed and improves the model, whereas a data-centric approach holds the model comparatively fixed and makes data curation an explicit route to better predictions.

Data-defined behavior adds a testing obligation: assessing predictive quality over the population of inputs the system will encounter. Conventional software can also have input spaces too large to enumerate, so exhaustive testing is not a realistic expectation for either kind of system. In the 2012 ImageNet challenge, AlexNet's decisive win[6] helped set the deep learning era in motion, yet the benchmark could evaluate only a vanishing fraction of the inputs the model might encounter. Each input is a 224×224 RGB image with $256^{150,528}$ possible pixel configurations, a number with 362,508 digits. Real photographs occupy only a tiny region of that space, but even that region is far too large to enumerate. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) validation set contains only 50,000 images (Krizhevsky et al. 2012; Russakovsky et al. 2015). The ratio of evaluated inputs to possible configurations (Test Set Coverage/Total Input Space) is effectively zero. That disparity between what any test set can cover and what a deployed system will meet is the **verification gap**.

This disparity does not make evaluation futile; it changes what testing can establish. Predeployment evaluation provides statistical evidence over sampled inputs, while continuous production monitoring verifies whether the operating population and observed outcomes continue to support that empirical baseline. Statistical confidence replaces any expectation of exhaustive proof. The chapters on deployment and operations return to this gap and bound it statistically over a specified operating population.

A unit assertion verifies that a given input and state follow an explicit execution path, while a machine learning claim describes aggregate performance over a population that changes over time. Robustness therefore cannot mean immunity to change. It means instrumenting the system to detect when production evidence no longer supports the model, and acting on that evidence.

Debugging an ML system therefore starts with inspecting datasets and pipeline transformations rather than tracing Python call stacks alone. Version control must record dataset snapshots and feature configurations alongside source-code commits, and evaluation harnesses must measure statistical distribution shifts and subpopulation outcomes rather than branch coverage. These practices establish reproducibility and provenance, even though they cannot transform empirical sampling into formal verification.



Test coverage is a vanishing fraction of the input space.

↪ Versioning training datasets so that a model can be reproduced and traced to its inputs is developed in section 4.8.2.

Checkpoint 1.1: The paradigm shift

Before tracing the history of AI, verify your understanding of the paradigm shift in how software is built:

- ☐ **Software 1.0 vs. 2.0**: can you distinguish Software 1.0 (explicit instructions) from Software 2.0 (behavior learned from data)?
- ☐ **Debugging shift**: can you identify which dataset evidence to inspect when accuracy falls without a code change?
- ☐ **Verification gap**: can you explain why finite test coverage can provide statistical evidence but never exhaustive proof, and why production monitoring must continue gathering that evidence?

Learning behavior from data can seem inevitable in hindsight, but its adoption also reflects the scaling limits of earlier approaches. Many AI systems attempted to program intelligence directly

[5] **Stochastic gradient descent (SGD)**: A training algorithm, descended from the stochastic approximation method of Robbins and Monro (1951), that repeatedly adjusts model parameters using small, randomly sampled groups of examples, called batches, rather than processing the entire dataset for each update.

[6] **AlexNet**: Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton trained this image-recognition network across two GPUs. At ILSVRC 2012, its 15.3 percent top-5 error, meaning the correct class was absent from its five highest-scoring predictions, substantially beat the second-place system's 26.2 percent (Krizhevsky et al. 2012). Section 1.3.3 returns to its architecture and systems co-design.

through hand-crafted symbols and rules. Reading that history through a systems lens reveals recurring binding constraints, the limits whose relaxation would most improve the whole system: logic, knowledge acquisition, feature design, and the physical infrastructure that the rest of this book studies.

1.3 The Evolution of AI Bottlenecks

Each era of AI history encountered a different limit, in representation, data, or computing resources. When Alan Turing[7] proposed evaluating machine intelligence through observable responses (Turing 1950), computing machinery had little memory or processing capacity. Early systems pursued two contrasting paths: Frank Rosenblatt's Perceptron (1958) (Rosenblatt 1958) attempted to learn weights directly from sensory inputs, whereas Joseph Weizenbaum's ELIZA[8] (Weizenbaum 1966) executed hand-coded pattern substitutions. Both soon met structural limits, as a single-layer perceptron could separate classes only with a linear boundary (Minsky and Papert 1969) while practical methods for training deeper networks remained out of reach, and ELIZA's scripts broke whenever inputs strayed from their hand-written templates. Four eras organize this history around recurring bottlenecks (symbolic AI, expert systems, statistical learning, and deep learning), although the approaches overlapped in time.

Figure 1.1 traces attention to AI across seven decades of published literature and shades two "AI winter" intervals (1974–1980 and 1987–1993) marked by funding cuts and unmet expectations.[9] These setbacks reflected gaps between expectations and what systems could deliver. Later advances combined changes in algorithms, data, and computing resources; the sections below examine how those changes shifted the limiting constraint.

1.3.1 Rule-based approaches: Logic and knowledge bottlenecks

Rule-based approaches attempted to build intelligent systems by making knowledge explicit. Symbolic AI encountered a *logic bottleneck* when its rules could not capture real-world ambiguity. Expert systems narrowed the task to a domain but encountered a *knowledge bottleneck*: acquiring and maintaining the rules required extensive expert effort.

1.3.1.1 The symbolic AI era and the logic bottleneck

Early AI research (1950s–1970s) pursued **symbolic AI**, representing knowledge and reasoning through explicit symbols and rules, an approach later crystallized in the physical-symbol-system hypothesis (Newell and Simon 1976). Researchers at the 1956 Dartmouth Conference[10] (McCarthy et al. 1955) hypothesized that aspects of intelligence could be precisely described and simulated by machines. Even then, Arthur Samuel at IBM demonstrated a different path in 1959 when a checkers program improved through self-play. His work coined the term "**machine learning**" (Samuel 1959), though the dominant paradigm remained symbolic. Daniel Bobrow's STUDENT[11] system (Bobrow 1964) took this approach to algebra word problems.

These systems could produce impressive demonstrations, yet they were operationally brittle. Their success depended on manually coded rules that mapped each acceptable input form to a symbolic representation. STUDENT could solve an algebra problem once it translated English into equations, but a minor variation in phrasing could prevent that translation. The difficulty extended beyond language to perception and movement.[12]

> Example 1.1: STUDENT (1964)
>
> STUDENT separated language interpretation from algebra:
>
> ```
> language  "Two numbers sum to 30; one is twice the other."
> parse     x + y = 30, x = 2y
> solve     x = 20, y = 10
> ```
>
> Once the equations were written, solving was routine. The bottleneck was the translation. Each unfamiliar phrasing needed another hand-written rule, so linguistic coverage grew only as fast as the rule base.

7 | **Alan Turing**: His "Imitation Game" approached machine intelligence through observable responses rather than a definition of its internal nature (Turing 1950). It provides an early example of evaluating a system through what it does.

8 | **ELIZA**: A natural-language program whose scripts could store selected inputs and reuse them in later responses (Weizenbaum 1966). This limited memory was programmed explicitly rather than learned from conversations. Extending its coverage required changing the scripts, even when the existing program ran correctly.

9 | **AI winters as systems failures**: The first AI winter (1974–1980) unfolded amid funding cuts; the 1973 Lighthill Report criticized the gap between AI promises and delivered results (Lighthill 1973). The second winter (1987–1993) involved a market and funding collapse around expert systems and specialized Lisp machines as general-purpose workstations undercut their economics (Hendler 2008). These episodes involved technical limitations, unmet expectations, and economics; hardware capacity alone does not explain them.

10 | **Dartmouth Conference (1956)**: The workshop organized around the term "artificial intelligence," already used in its 1955 proposal (McCarthy et al. 1955). The proposal discussed limits in computer speed and memory while treating programming as the larger obstacle. Physical capacity was part of the problem from the outset, even as researchers focused on language, abstraction, problem solving, and self-improvement.

11 | **STUDENT**: An early program that translated algebra word problems into equations using hand-written language rules (Bobrow 1964). Its success depended on the wording fitting those rules; an algebra solver could solve the resulting equations without establishing that the program had interpreted the original problem correctly.

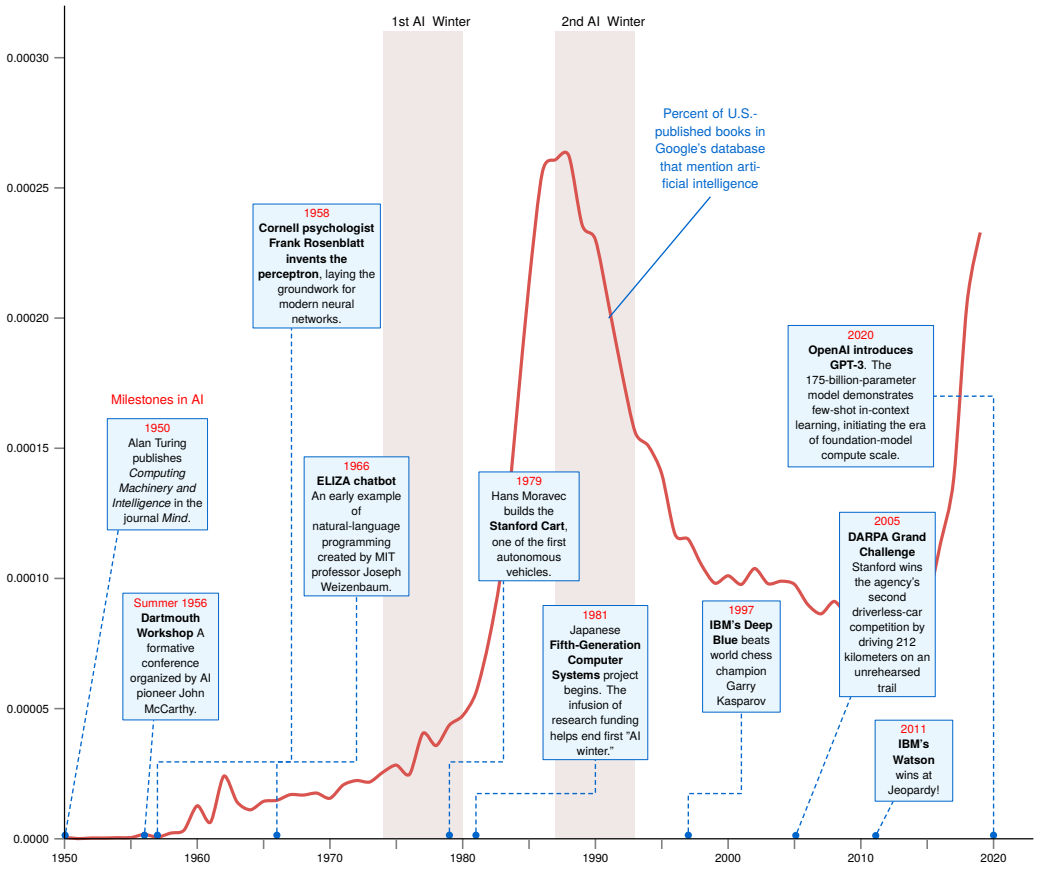


Figure 1.1: AI Development Timeline: The red curve tracks how often published books mention artificial intelligence, a measure of attention rather than capability, and the light-brown bands mark the two AI winters. Callout boxes place milestones such as Turing's 1950 paper (Turing 1950), the Dartmouth conference (McCarthy et al. 1955), the Perceptron (Rosenblatt 1958), ELIZA (Weizenbaum 1966), Deep Blue (Campbell et al. 2002), and GPT-3 (Brown et al. 2020) against that curve. Adapted from Michel et al. (2011).

12 **Moravec's paradox**: Hans Moravec observed that skills people acquire with little conscious effort, such as perception and movement, can be harder to reproduce computationally than formal reasoning tasks (Moravec 1988). Human familiarity with a task is therefore a poor guide to its engineering difficulty or resource requirements.

1.3.1.2 The expert systems era and the knowledge bottleneck

In the expert-systems era, engineers narrowed the problem. Instead of constructing general reasoning systems, they encoded deep knowledge from a specific domain. MYCIN, designed to diagnose blood infections, let medical knowledge be expressed as production rules (Shortliffe et al. 1975).

> Example 1.2: MYCIN (1975)
>
> MYCIN made its reasoning explicit. Patient facts triggered readable production rules:
>
> ```
> facts  stain=gram-positive, shape=coccus, arrangement=clumps
> rule   IF facts match THEN organism=staphylococcus (certainty=0.7)
> cycle  match facts -> fire rule -> update certainty -> repeat
> ```
>
> An inference engine repeated this match-and-fire cycle across hundreds of rules and propagated certainty factors (numerical confidence weights) toward a diagnosis. A physician could inspect every step, but coverage still grew one rule at a time: each exception had to be elicited, encoded, and reconciled with the existing rule base.

MYCIN performed well in specific tests, but its success exposed the **knowledge acquisition bottleneck**.[13] The problem was no longer whether a rule could express expertise. It was how tacit judgment entered the system and remained consistent as the rule base grew.

Faster machines could evaluate production rules at higher frequencies, but raw processor speed could not accelerate the extraction or maintenance of human expertise. Scaling meant learning decision boundaries from examples instead of asking domain specialists to enumerate every contingency. This transition did not eliminate human design; it relocated engineering effort. Instead of hand-coding decision logic, engineers shifted to curating datasets, selecting mathematical representations, and formulating learning objectives from which models could optimize their own parameters. The next era exchanged the knowledge acquisition bottleneck for a new one, the manual design of features extracted from raw data.

[13] **Knowledge acquisition bottleneck**: Feigenbaum (1984) treated extracting and representing expert knowledge as a central challenge in building applied AI systems. Experts could exercise judgment without being able to state every rule behind it, so acquiring knowledge required more than transcribing a ready-made list.

1.3.2 The statistical learning era and the feature engineering bottleneck

The 1990s marked the shift to **statistical learning** and probabilistic systems. Instead of hard-coded logic, systems estimated probabilities from data, such as the conditional distribution $p(y \mid x)$ of a label y given an input x. This transition was driven by the availability of digital data and the "unreasonable effectiveness"[14] of large datasets. Early spam filters illustrated the change: rather than maintaining lists of forbidden words, statistical filters learned the probability that a word implies spam from millions of examples.

[14] **Unreasonable effectiveness of data**: The observation that a simple statistical model with abundant data can outperform a more sophisticated model with less data (Halevy et al. 2009). The benefit depends on the task, model, and data quality. This makes collecting and preparing useful examples part of the engineering work, rather than merely a preliminary step.

Example 1.3: Early spam detection systems

Statistical filters shifted the decision from a hand-written rule to a probability estimated from labeled messages:

```
rules: if "free" or "winner" appears -> spam
learn: labeled email -> word likelihoods -> Pr(spam | email)
```

Maintenance did not disappear. It moved from editing keyword lists to curating representative examples and checking whether the learned probabilities still tracked current email.

Learning the decision boundary from data reduced one source of manual work but exposed another. Many statistical classifiers relied on engineers to convert raw inputs into features, measurable properties such as word counts or image edges. In computer vision, engineers designed feature extractors such as SIFT (Scale-Invariant Feature Transform) (Lowe 1999) and HOG (Histogram of Oriented Gradients) (Dalal and Triggs 2005) to summarize local image patterns. Classifiers such as support vector machines (SVMs) (Cortes and Vapnik 1995) then learned decision boundaries from those features. The learner adjusted the boundary; engineers decided which evidence it could see. Extending the system to a new problem often meant rebuilding the preprocessing stack, creating the **feature engineering bottleneck**.

This hybrid approach combined human-engineered features with statistical learning. The Viola-Jones algorithm[15] (Viola and Jones 2001), typical of this era, achieved real-time frontal-face detection using simple rectangular features and cascaded classifiers. It showed that well-engineered features could enable practical low-latency applications, but only within narrow domains where experts could hand-craft the right representations. A conventional vision pipeline makes that manual effort visible, because several hand-crafted stages preceded any learning at all.

[15] **Viola-Jones algorithm**: The detector applied a sequence of classifiers, rejecting likely nonface regions early and spending more computation on promising regions. On the reported test set, it evaluated an average of ten features per candidate image region, although the complete cascade contained over 6,000 features (Viola and Jones 2001). Its speed depended on how little work most inputs required, not just on how fast each operation ran.

Example 1.4: Traditional computer vision pipeline

A conventional vision system separated representation from classification:

```
image -> resize -> HOG/SIFT features -> SVM -> label
```

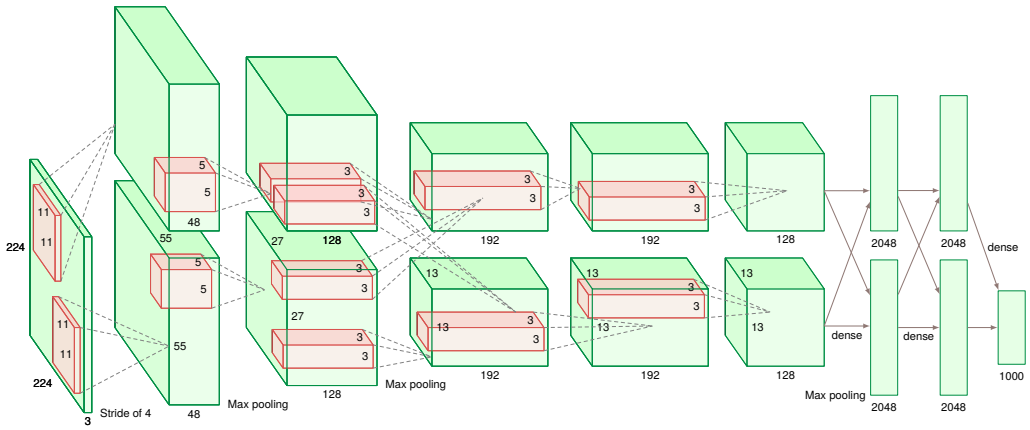


Figure 1.2: AlexNet Architecture: Two parallel GPU streams process 224×224 input images through convolutional layers (green blocks) that extract spatial features at decreasing resolutions, converging through three fully connected layers to 1,000 output classes. Because each stream holds half of the network, the figure also shows how the model was partitioned across two GPUs. Adapted from Krizhevsky et al. (2012).

> Moving from faces to pedestrians or another domain meant redesigning the feature extractor and often the entire preprocessing stack.

1.3.3 The deep learning era and the infrastructure bottleneck

Redesigning hand-crafted feature extractors for every new domain could not scale, so **deep learning** (LeCun et al. 2015) moved feature extraction inside the model itself. A neural network computes in layers, stages of arithmetic on learned weights in which each layer transforms the output of the one before. Instead of receiving features designed by humans, deep networks with many such layers learned representations directly from raw inputs such as pixels and audio waveforms. The training process could now shape both the representation and the decision boundary, enabling "end-to-end" learning.

The breakthrough was not algorithmic alone. Convolutional neural networks (CNNs), whose convolutional layers slide small learned weight patterns called filters across an image to detect the same feature at every position, existed earlier (LeCun, Boser, et al. 1989; LeCun et al. 1998); AlexNet paired architecture and training with **systems co-design**, choosing the model, training procedure, and hardware mapping together (Krizhevsky et al. 2012). Its parallel matrix operations matched GPU capabilities. With 60 million parameters distributed across two GTX 580 GPUs, AlexNet achieved 15.3 percent top-5 error, a 41.6 percent relative improvement over the next-best entry that year. The hardware partition in figure 1.2 carries the immediate systems lesson: the network divided work between the two GPUs and exchanged intermediate results only at selected layers to limit communication across the link between them (Krizhevsky et al. 2012). Model structure and machine organization were designed together.

Hardware still shaped what could be learned. In 2012, storing AlexNet's 60 million 32-bit parameters required roughly 240 megabytes, well within a single card's capacity. Training also needs working storage for intermediate results and for calculating and retaining parameter updates. These quantities, developed later as activations, gradients, and optimizer state, make training memory larger than the stored model alone, and the intermediate results grow with the number of images processed together. Each GTX 580 offered only 3 GB of video random-access memory (VRAM), and the original paper reports that this limit made the network too large to train on one card (Krizhevsky et al. 2012). AlexNet therefore divided convolutional and dense layers between two GPU streams communicating over the PCIe bus, the host's general-purpose peripheral interconnect. This early model-parallel design foreshadowed modern multi-GPU training. Deep learning reduced the need for hand-crafted features, but it moved the binding constraint to the infrastructure required to store data, move parameters, and coordinate computation.

↪ Model parallelism, which partitions a model across accelerators when it outgrows one device's memory, as AlexNet's two-GPU split did, is developed in section 8.7.1.2.

This shift produced the **infrastructure bottleneck**, in which physical memory capacity, interconnect bandwidth, and hardware orchestration dictate the boundaries of what models can learn. GPT-3 (Generative Pre-trained Transformer 3), with 175 billion parameters, was trained on roughly 300 billion tokens (the word fragments a language model reads and predicts) curated from filtered web text, books, and Wikipedia (Brown et al. 2020). The dense-training approximation (Kaplan et al. 2020) ($O \approx 6 \cdot P \cdot D$ operations for a model with P parameters trained on D tokens, counting both the forward and backward passes) agrees with the roughly 314 zettaFLOPs of total computation that the GPT-3 authors report, where one zettaFLOP is 10^{21} floating-point operations (FLOPs). A retrospective audit by Patterson et al. (2021) estimated that this single training run consumed roughly 405 V100 GPU-years. The hard part had moved from designing edge detectors to keeping some 10,000 accelerators running in parallel across high-speed fabrics without failures that halt training midway. Viewed through this systems lens, each transition traded one binding constraint for the next, first in human effort (rules, knowledge, features) and then in physical infrastructure (table 1.2).

↪ The dense-training approximation used for this operation count is developed in section 8.1.1.

Table 1.2: AI Paradigm Evolution: Each era is viewed through a recurring systems bottleneck. Deep learning (far right) reduced the need for manual feature engineering while increasing demands on computing infrastructure.

Aspect	Symbolic AI	Expert Systems	Statistical Learning	Deep Learning
Key Strength	Logical reasoning	Domain expertise	Versatility	Pattern recognition
Bottleneck	Brittleness (Rules break)	Knowledge Acquisition (Experts are scarce)	Feature Engineering (Manual preprocessing)	Infrastructure (Compute and data scale)
Data Handling	Minimal data needed	Domain knowledge-based	Moderate data required	Massive data processing

Across the four eras, engineering effort migrated from handcrafting domain heuristics toward building scalable compute and data pipelines. The bitter lesson explains why this shift recurred.

1.4 The Bitter Lesson

Expert systems invested engineering effort in encoding domain knowledge; deep learning systems invest much of that effort in learning from data at scale. The **bitter lesson** captures the historical advantage of general methods, especially search and learning, that can make effective use of increasing computation. Richard Sutton[16] crystallized this insight in his 2019 essay "The Bitter Lesson" (Sutton 2019). Sutton wrote, "The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin."

Training scale at the frontier is often stated in **accelerator-days** (GPU-days when the accelerator is a GPU): one reference accelerator running continuously for 24 hours. Table 1.3 pairs representative benchmark milestones with their hardware. Its final column runs from CPU-based learning to multi-thousand-accelerator clusters, using 2.5 million reference GPU-days as an estimated frontier-training anchor (Patel and Wong 2023).

[16] **Richard Sutton**: A reinforcement learning pioneer whose essay emphasizes search and learning as methods that scale with computation (Sutton 2019). The lesson is "bitter" because carefully encoded human knowledge has repeatedly been overtaken by more general methods. It is a lesson about long-term progress, not a claim that domain knowledge has no engineering value.

Table 1.3: AI Performance Evolution Across Paradigms: Representative systems illustrate the growth in computational scale across eras. GPT-4 training details are estimated reference anchors rather than official disclosures.

Era	Approach	Representative Task	Performance	Computational Resources
Statistical ML (1990s–2000s)	Feature engineering + learning	Handwritten digit recognition	about 98–99% on MNIST-era benchmarks (LeCun et al. 1998)	CPU-era feature pipelines; resources varied by implementation
Deep Learning (2012)	End-to-end neural networks	ImageNet top-5 accuracy	84.7% (AlexNet)	6 days on 2 GPUs
Modern Deep Learning (2020+)	Large-scale transformers (Vaswani et al. 2017)	ImageNet top-1 accuracy	88.55% (ViT-H/14) (Dosovitskiy et al. 2021)	Large-scale Tensor Processing Unit (TPU) pretraining
Modern Deep Learning (2023)	Foundation models	MMLU benchmark	86.4% (GPT-4) (OpenAI et al. 2023)	Estimated ~2.5 million reference GPU-days (on the order of 25,000 reference GPUs run for 100 days) (Patel and Wong 2023)

Digit recognition, ImageNet classification, and Massive Multitask Language Understanding (MMLU) measure different capabilities under separate metrics, so table 1.3 does not compare methods on one task. It shows instead that each new capability threshold demanded orders of magnitude more hardware, from single-node CPU routines to multi-megawatt accelerator clusters. Testing Sutton's lesson, whether general methods powered by raw computation surpass hand-engineered representations, requires holding the task fixed and comparing competing approaches within one domain.

Game playing separates the roles of search and learning. In chess, IBM's Deep Blue[17] defeated world champion Garry Kasparov in 1997 by combining custom chess hardware, large-scale search, and chess-specific evaluation knowledge. In Go, DeepMind's AlphaGo learned from expert games and self-play, using neural networks to guide tree search, but still used some engineered input features (Silver et al. 2016). AlphaGo Zero[18] provides the clearer example of Sutton's lesson: it learned through self-play without human game records or those hand-engineered features (Silver et al. 2017). The advance depended on methods that could turn computation into improved play, not merely on a faster machine.

The lesson is "bitter" because encoding human expertise can appear to be the most direct path to a solution. Sutton's examples show how scalable search and learning repeatedly overtook such approaches. More computation does not guarantee better results; the method must use it productively, and the data and objective must support the intended task.

Modern language models such as GPT-4 (OpenAI et al. 2023) and image generation systems such as DALL-E (Ramesh et al. 2021) illustrate the engineering demands of learning at scale. Much of their behavior is learned from large datasets rather than expressed as explicit linguistic or artistic rules. Training GPT-3 alone was estimated to consume roughly 1.3 GWh of energy[19] (Patterson et al. 2021), and serving these models to millions of users turns inference into a continuous data-center power, cooling, and capacity-planning problem.

When model capability scales with raw computation, the binding limits shift to physical infrastructure: memory bandwidth,[20] interconnect latency, thermal dissipation, and data pipeline throughput. Data movement, arithmetic throughput, and operational reliability must therefore be designed as one problem. Designing for continuous computational growth is the responsibility that defines AI engineering, which section 1.9 formalizes once its system boundaries and quantitative foundations are in place.

1.5 Defining ML Systems

Absorbing computational growth takes a working system, not only a model, and even a familiar service shows how many parts such a system has. A production spam filter needs more than a model that recognizes unwanted messages. It needs representative training data, a way to update learned behavior as attacks change, and infrastructure that classifies incoming mail within the delivery budget. Global email traffic is measured in hundreds of billions of sent and received messages

[17] **Deep Blue**: Its 480 custom chess chips worked alongside general-purpose processors, searching about 100 million positions per second in tactical positions and approaching 200 million in quieter positions (Campbell et al. 2002). The hardware accelerated chess-specific search and evaluation; it did not remove the system's reliance on encoded chess knowledge.

[18] **AlphaGo Zero**: After three days of self-play training, it defeated AlphaGo Lee, the version that had beaten Lee Sedol, by 100 games to 0 (Silver et al. 2017). Self-play generated the training examples, allowing search and learning to improve play without a dataset of human games.

[19] **GPT-3 training energy**: Patterson et al. (2021) estimated that GPT-3's training consumed approximately 1,287 MWh and produced 552 metric tons of CO_2-equivalent emissions. The energy estimate corresponds to the annual electricity use of about 120 US households at a baseline of 10.7 MWh/household-year. Emissions also depend on how that electricity is generated.

[20] **Memory bandwidth**: The rate at which a memory interface transfers data, measured in bytes per second. A processor must receive the values an operation needs before it can perform the arithmetic. When moving those values takes longer than computing with them, faster arithmetic alone cannot remove the delay; reducing transfers or increasing the available bandwidth addresses the limiting resource.

per day (Statista Research Department 2024), making this a substantial systems problem for large providers.

The challenge begins with data. The filter is trained on labeled examples and needs updated training data and periodic retraining as spam tactics change. It is also an algorithmic problem, because the model must generalize from those examples to messages it has never seen before. Precision measures the fraction of flagged messages that are actually spam; recall measures the fraction of all spam messages that the filter catches. Flagging more messages may catch more spam while also hiding legitimate mail, so the system must evaluate both consequences. Finally, it is an infrastructure problem. Providers must handle incoming traffic, store and update models, and return predictions within their delivery latency budget. The classifier is therefore only one component of an evolving data, software, and hardware system.

When a new phishing template appears, the data layer must capture representative messages, the algorithm must learn to separate attacks from legitimate mail, and the infrastructure must deploy and serve the updated model. Any layer can fail while the others continue working, so the full chain from measurement through learning to execution is the right unit of analysis.

Definition 1.1: Machine learning systems

Machine learning systems combine models that learn from data with the software and hardware needed to train, deploy, and maintain them. Their performance depends jointly on data quality, algorithm choice, and machine capacity.

1. **Significance**: Requirements are end to end, so a failure in any layer changes the user-visible result even while the other layers operate correctly.
2. **Distinction**: The system includes every path an input travels, not only the model. A feature computed one way in training and another in serving changes what the system predicts even when the model and its weights are identical.
3. **Common pitfall**: The model is not the system. Data pipelines, feature transformations, serving infrastructure, monitoring, and feedback loops surround the learned parameters and often dominate the engineering burden (Sculley et al. 2015).

The definition exposes three diagnostic axes: the data that describes the task, the algorithm that turns examples into behavior, and the machine that executes that behavior within the operating budget. The same user-visible symptom can originate in any of the three. Treating every miss as a model problem risks improving the wrong component. The D·A·M taxonomy separates these causes for diagnosis.

Each explanation calls for different evidence. The data hypothesis sends the engineer to coverage, labels, and distribution shift. The algorithm hypothesis sends the engineer to error slices (errors grouped by input type), model capacity, and the learning objective. The machine hypothesis sends the engineer to latency, throughput, memory, and utilization. These investigations are not interchangeable. Faster hardware cannot supply examples that were never collected, while a larger dataset cannot repair a serving path that misses its deadline because memory is saturated.

The relevant constraint is the one whose relaxation improves the end-to-end result. This idea of a **binding constraint**, used informally in the history above, prevents teams from optimizing the most visible component instead of the component that governs the outcome. The purpose of D·A·M is therefore operational: it identifies the next hypothesis to test and the class of intervention capable of changing system behavior. The diagnosis is provisional. Once an intervention relaxes one limit, the system must be measured again because a different axis may now bind. D·A·M is a loop, not a one-time label.

Definition 1.2: D·A·M taxonomy

D·A·M taxonomy is a diagnostic framework for examining machine learning system bottlenecks along three axes. Data determines what examples and bytes the system must process, Algorithm

determines the model structure and work required to learn or predict, and Machine determines the hardware capacity available to execute that work. The goal is to identify which axis is the binding constraint.

1. **Significance**: The diagnostic power is concrete even before detailed hardware arithmetic enters the story. If the spam filter misses a new phishing campaign because the training set never contained that tactic, the binding axis is Data. If the training examples are adequate but the model cannot express the pattern, the binding axis is Algorithm. If both are adequate but the service cannot classify messages quickly enough during a traffic spike, the binding axis is Machine. Quantitative diagnosis begins by asking which axis is limiting the system.
2. **Distinction**: The D·A·M taxonomy connects predictive quality to physical execution. Algorithm choice shapes both the data needed to learn useful behavior and the machine resources needed to run it. Those requirements depend on the task and training procedure, so model size alone does not determine either generalization or runtime.
3. **Common pitfall**: A frequent misconception is that the three axes are independent. Changing from a simple classifier to a larger model can require more memory, different serving infrastructure, and a broader data distribution. Because changing one of Data, Algorithm, and Machine can change the requirements imposed on the other two, every edge of the triangle in figure 1.3 points both ways.

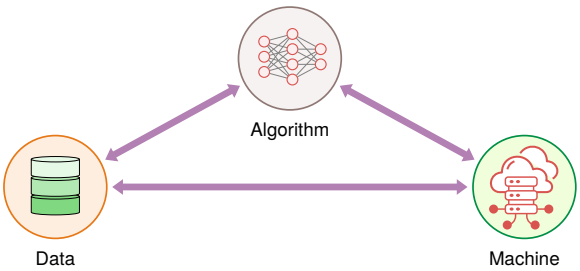


Figure 1.3: The D·A·M Axis Triad: Each vertex is one diagnostic axis, and each double-headed edge is a path along which a relaxed bottleneck can reappear on the neighboring axis.

An operational diagnosis earns confidence only when an intervention validates it, that is, when relaxing the identified constraint improves full-system behavior. Resolving one bottleneck frequently migrates the binding constraint across an adjacent edge of figure 1.3, so interventions rarely end the optimization process. Upgrading to faster accelerators (Machine) leaves execution units idling if storage I/O cannot stream input batches quickly enough (Data); conversely, expanding dataset volume (Data) exposes model architectures that lack the expressive capacity to absorb new patterns (Algorithm), and scaling model capacity to close that gap (Algorithm) soon exceeds accelerator memory, triggering out-of-memory faults or expensive paging across the PCIe bus (Machine). Engineering an ML system therefore requires identifying the currently binding constraint while anticipating where the bottleneck will migrate once that constraint is relaxed.

While the triad separates the three axes to simplify initial diagnosis, practical systems operate at their boundaries. Data formats govern memory traffic, model architectures dictate hardware functional unit utilization, and deployment budgets reshape both data representation and algorithmic design. Figure 1.4 partitions these interfaces into a three-set Venn diagram whose pairwise overlaps each pose one question: what the system can learn from ($D \cap A$), how information moves ($D \cap M$), and how computation executes efficiently ($A \cap M$).

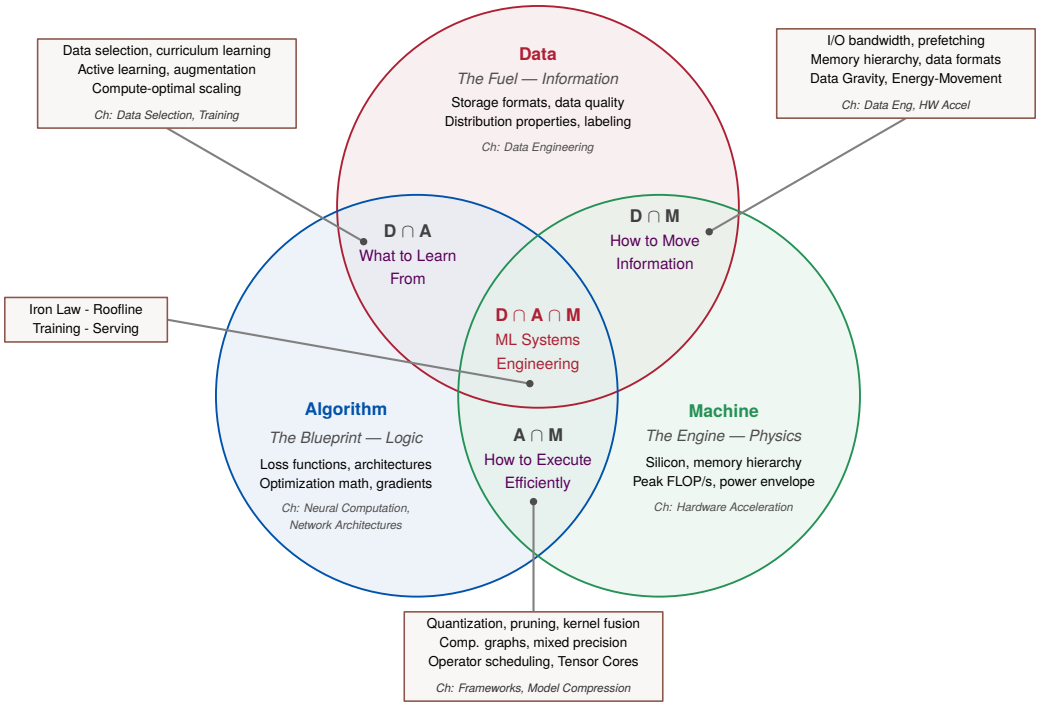


Figure 1.4: The D·A·M Intersection Landscape: Each circle lists the concerns that belong to one axis, and each pairwise overlap is titled with the question that axis pair must answer jointly. The central region, where all three circles overlap, is labeled ML systems engineering.

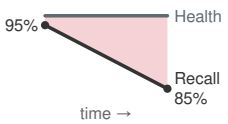
At the center, all three questions must be answered together: data selection and training determine which examples help the model learn useful behavior, data engineering and hardware acceleration deliver that information to the machine at speed, and frameworks and execution techniques turn the model's computations into efficient work on that silicon. Pruning a model (Algorithm), which removes parameters that contribute little to its predictions, alters its memory footprint and how much of it stays in fast on-chip memory (Machine). Compressing input features can likewise reduce transfers while adding preparation work (Data). These changes must be evaluated across the interfaces, because a local improvement may leave the end-to-end result unchanged, and even when every component executes without error, a shift on the data axis can still degrade predictions silently.

1.6 When ML Systems Fail Silently

In an automotive driver-assistance system, diagnostic trouble codes and hardware watchdog timers expose hardware or communication faults, yet they do not measure how often a perception model detects pedestrians. When seasonal conditions introduce low-angle winter sunlight, heavy precipitation, or bulky clothing underrepresented in training, pedestrian recall can decline from 95 percent to 85 percent even while the accelerator processes images at full frame rate without raising a hardware fault. Correct execution and predictive quality are separate properties, and an ML system must be engineered and monitored for both.

Crashes, unhandled exceptions, and failed health checks make some software defects directly observable, though conventional software can also produce silent numerical errors or logic bugs. Machine learning adds another route to **silent degradation**: learned behavior loses predictive quality as the operating population changes, even while the runtime executes instructions without error, memory allocations succeed, and inference servers return predictions well within latency service level agreements (SLAs).

↪ How pruning shrinks a model's memory footprint, and why its effect on speed depends on the sparsity pattern the hardware can exploit, is developed in section 10.2.1.



Recall falls while health metrics stay flat.

In this example, the miss rate triples, from 1 in 20 pedestrian frames to 3 in 20. That is a substantial change in detection quality, but it does not imply a tripling of vehicle-level safety risk, because temporal filtering, sensor fusion, and downstream control affect how missed detections influence a driving decision.

Input distributions can drift as environmental conditions, user behavior, or sensor calibrations change (Gama et al. 2014; Quiñonero-Candela et al. 2009). A model may then apply its fixed weights to inputs unlike those on which it was evaluated. The machine can execute every operation correctly while producing predictions whose quality no longer meets the requirement.

Outcome measurements establish whether predictive quality has fallen; distribution-divergence signals can prompt investigation before those outcomes arrive. When both are observed, model degradation can be approximated locally as a function of distribution shift. Let Accuracy_0 denote the baseline accuracy at deployment, $\mathcal{D}(P_t \| P_0)$ represent the statistical divergence between current production distribution P_t and training distribution P_0, and λ denote the locally fitted sensitivity to that shift measure, so that $\text{Accuracy}(t)$ is the accuracy after the model has been deployed for time t. This relationship, formalized in equation 1.1, is the **degradation equation**:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0) \tag{1.1}$$

This approximation describes a local trend fitted from observed inputs and labeled outcomes; it is not a universal law of accuracy loss. The chapters on deployment return to it as a statistical drift diagnostic. **Data drift** occurs when P_t differs from P_0. Such a shift can reduce inference quality even when the model and execution environment remain unchanged, but the shift alone does not establish a quality loss. Large changes may fall outside the range where the fitted relationship holds. The divergence measure $\mathcal{D}(\cdot \| \cdot)$ is left general here.

↪ The statistics that measure distribution shift and detect drift in production data are developed in section 4.12.1.

The equation separates baseline quality from sensitivity to shift. Better data, objectives, and architectures can change both quantities, so an improved baseline does not by itself establish robustness to drift. For the driving example, training on representative winter images may reduce sensitivity to seasonal conditions; evaluating pedestrian recall under those conditions establishes whether it does. Monitoring $\mathcal{D}(P_t \| P_0)$ provides a warning that prompts engineers to seek outcome evidence. That evidence determines whether retraining or another intervention is needed. *Knowing when to reevaluate is as important as knowing how to train.*

↪ How drift signals and labeled outcomes become an operational decision to retrain, made only when the expected benefit exceeds the cost, is developed in section 14.6.1.

Silent degradation can also originate inside the system through **training-serving skew**. Training and serving may use separate implementations to transform raw inputs into model features. If those transformations differ, for example by resizing images with a different interpolation method or applying different missing-value defaults, the model receives inputs unlike those used in training. The model can execute cleanly and return predictions on time while accuracy falls. This is a software consistency defect whose consequence is degraded learned behavior.

↪ Keeping transformation code and its fitted parameters identical across training and serving is developed in section 4.7.1.

Host-health signals such as CPU utilization, memory residency, network I/O, and HTTP status codes remain necessary, but do not measure whether predictions are useful. Machine learning operations must track these infrastructure metrics alongside data quality, feature distributions, and prediction accuracy. Because an ML system can fail silently while infrastructure metrics appear nominal, system design must integrate continuous evaluation from data ingestion through inference serving.

Useful predictions satisfy the first half of the ML systems mandate. The second half requires producing them within the available latency, memory, energy, and cost budgets. Meeting those budgets requires a quantitative account of where execution time goes.

1.7 Iron Law of ML Systems

Computational scale has a physical cost, and it shows up as two familiar failures. A training job stalls when storage cannot feed an accelerator; an inference path misses its deadline when model state moves too slowly through memory or across the network. Their symptoms differ, but both are governed by how the three D·A·M axes combine to consume wall-clock time. Across Data (D), Algorithm (A), and Machine (M), every ML task completes by processing a set of samples (examples in vision or tabular workloads, tokens in language models) through an algorithm's operations on

hardware. By dimensional cancellation, total execution time T (seconds) is the product of these three factors (equation 1.2):

$$T = \underbrace{\left(\frac{\text{Samples}}{\text{Task}}\right)}_{\text{Data}} \times \underbrace{\left(\frac{\text{FLOPs}}{\text{Sample}}\right)}_{\text{Algorithm}} \times \underbrace{\left(\frac{\text{Seconds}}{\text{FLOP}}\right)}_{\text{Machine}} \tag{1.2}$$

The same equation governs both training and inference because only the definition of the task and the scale of each factor change. The product of the first two factors sets the total arithmetic work O (FLOPs), while the third factor (Seconds/FLOP) is the realized time the machine takes per operation:

- **Training**: A task is a training run (or an optimization step), where Samples/Task is the corpus token budget D (billions to trillions of tokens) and FLOPs/Sample spans the forward and backward passes ($\approx 6P$ FLOPs per token for a dense transformer, giving $O \approx 6PD$). Because large training batches reuse model weights across thousands of tokens per step, the machine amortizes memory transfers and dispatch overhead, keeping realized Seconds/FLOP low and near the hardware's compute ceiling.
- **Inference**: A task is serving a request or generating a response, where Samples/Task shrinks to a small request batch (B) or a stream of generated tokens, and FLOPs/Sample covers only the forward pass ($\approx 2P$ FLOPs per token). With far fewer samples per invocation to amortize weight reads and orchestration, realized Seconds/FLOP rises sharply as memory bandwidth and fixed latency overtake arithmetic.

A real machine does not run at its datasheet peak rate $1/R_{\text{peak}}$ because physical execution requires moving data across memory and interconnect buses, driving arithmetic units at some realized utilization, and paying fixed control overheads. Assuming these three physical costs occur sequentially, the **iron law of ML systems** unpacks total execution time T (seconds) into their sum (equation 1.3):

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}} \tag{1.3}$$

Together, equation 1.2 and equation 1.3 account for the physical-execution half of the dual mandate introduced in section 1.1:

1. **The memory term** $(D_{\text{vol}}/\text{BW})$ represents the physical cost of moving bits. D_{vol} is the volume of data moved (bytes), and BW is the memory or network bandwidth (bytes/s). Whether loading terabytes from cloud storage or fetching weights from high-bandwidth memory (HBM), the stacked dynamic random-access memory (DRAM) packaged beside an accelerator, performance is often limited by I/O physics. Part I develops this foundation.
2. **The compute term** $(O/(R_{\text{peak}} \cdot \eta_{\text{hw}}))$ represents the cost of arithmetic. O is the number of floating-point operations (FLOPs) set by the product of Samples/Task and FLOPs/Sample, R_{peak} is the hardware's theoretical peak throughput (FLOP/s), and η_{hw} is dimensionless realized compute utilization $(0 \leq \eta_{\text{hw}} \leq 1)$. Parts II and III develop this term.
3. **The latency term** (L_{lat}) represents orchestration, networking, and serialization overhead (seconds), treated as fixed for the modeled execution. Engineering changes can reduce this overhead, which can dominate small, latency-critical requests where little data moves and little arithmetic runs. Part IV develops this term.

Consider a hypothetical task that spends 2 seconds moving data, 8 seconds computing, and 1 second on overhead. Its total is 11 seconds. Doubling the arithmetic rate reduces computation to 4 seconds, but leaves movement and overhead unchanged. The task now takes 7 seconds, rather than half its original time. A faster component helps only through the portion of the time budget it changes.

Data enters equation 1.2 and equation 1.3 in two distinct ways: logical samples versus physical bytes. In equation 1.2, the Data factor counts how many examples or tokens the task processes (such as the dataset size D in the $6 \cdot P \cdot D$ training estimate), which multiplies FLOPs/Sample to fix

total arithmetic work O. In equation 1.3, by contrast, D_{vol} counts the physical bytes moved across a memory or interconnect bus, including model weights (P), activations, cached sequence state, and gradients alongside the raw inputs. When a language model decodes a single token at batch size one, the logical input is one token (Samples/Task $= 1$), yet D_{vol} spans gigabytes of model weights streamed from high-bandwidth memory.



Systems Perspective 1.1: The iron law analogy

The name "iron law" follows the processor performance equation of Patterson and Hennessy (2017), Time/Program $=$ (Instructions/Program) $\times$ (Cycles/Instruction) $\times$ (Seconds/Cycle). Equation 1.2 is the direct ML counterpart: the Data (Samples/Task) and Algorithm (FLOPs/Sample) factors multiply to set total operations O just as program path length and instruction set determine instruction count, while the Machine factor (Seconds/FLOP) mirrors cycles per instruction times clock period. Reducing FLOPs/Sample (for example, through unstructured sparsity) can still slow down execution if irregular memory accesses inflate Seconds/FLOP, just as complex CPU instructions that cut instruction count can lose performance by inflating cycles per instruction. Equation 1.3 then takes the same second step that computer architects take when decomposing processor time into compute cycles, memory-stall cycles, and pipeline overhead.

The additive form assumes sequential execution. When movement and computation overlap, the critical-path lower bound replaces the sum in equation 1.4:

$$T_{\text{pipelined}} \geq \max\left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}\right) + L_{\text{lat}} \tag{1.4}$$

This max-based formulation reflects hardware that transfers the next block of data while computing on the current one, so the slower of the two stages hides the faster. Equality requires ideal overlap and assumes that L_{lat} remains outside the overlapped phases.

Even as a sum, the iron law identifies which physical constraint dominates before any optimization begins. Amdahl's law (Amdahl 1967) plays the same role for speedups. Accelerating only part of a task leaves the untouched remainder in place, and that remainder caps the whole task's speedup however fast the accelerated part becomes; equation 11.1 states that cap quantitatively as a bound on overall speedup. Section A.3.2 presents the refined treatment, including pipelining and overlap techniques that transform the additive model into the max-based formulation used in practice.

Diagnosing the limiting term requires comparing the workload with the machine. Arithmetic intensity ($I = O/D_{\text{vol}}$) counts the operations performed per byte moved. Each machine has an intensity at which its memory and compute terms balance, set by its peak compute rate divided by its memory bandwidth. A workload below that balance point waits on memory; one above it waits on arithmetic. Improving the smaller of the two terms yields a limited end-to-end gain. Actual performance also depends on realized utilization, and overhead (L_{lat}) sits outside this comparison and can dominate small requests. A single-accelerator estimate applies these ideas below.

↪ The roofline model, which bounds attainable throughput by arithmetic intensity and locates the ridge point, is developed in section 11.6.



Napkin Math 1.1: Estimating one local execution

Problem: A hypothetical workload on one accelerator moves 2 GB at 100 GB/s, performs 1 GFLOP at an effective arithmetic rate of 100 GFLOP/s, and incurs 5 ms of fixed overhead. Movement precedes computation. What execution time does the iron law predict, and which rate should be improved first?

Math: Dividing bytes by bytes/s gives a movement time of 20 ms. Dividing FLOPs by FLOP/s gives a compute time of 10 ms; the supplied effective rate already includes η_{hw}, so it must not be

multiplied by efficiency again. Adding these durations and overhead gives 35 ms. Movement is the largest term.

Result: Raising bandwidth to 200 GB/s reduces movement to 10 ms and total sequential time to 25 ms. Arithmetic and overhead remain unchanged, so doubling bandwidth does not halve the total.

Systems insight: If movement and computation could instead overlap ideally, with overhead still outside both phases, the lower bound would be 25 ms before the upgrade and 15 ms afterward. The upgrade balances the two stages; further bandwidth increases alone would leave that ideal bound unchanged. Whether the sum or the bound describes a measured run depends on its execution schedule.

A larger training estimate returns to equation 1.2 under an explicit compute-dominated assumption, folding all non-arithmetic delays into an effective end-to-end utilization factor η_{hw} so that the realized machine factor across N_{GPU} parallel accelerators is $(N_{\text{GPU}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}})^{-1}$ seconds per FLOP. Its efficiency factor accounts for the losses involved in coordinating many accelerators, beyond the local execution just calculated. A multi-machine cluster of that kind lies outside this book's single-machine scope; the estimate appears here only to show the scale the compute term can reach.

↪ The training-time form of the iron law, which converts an operation count into days of training at a realized utilization, is developed in section 8.2.

Napkin Math 1.2: Training GPT-3

Problem: How long would a GPT-3-class model take to train on a hypothetical modern cluster of 1,024 accelerators? This reference cluster uses A100-class accelerators, a generation newer than the V100 GPUs of the original run estimated earlier at roughly 405 V100 GPU-years, so the result shows what the same work could cost on newer hardware at the utilization assumed below. The original run achieved lower utilization, so part of the speedup comes from that assumption rather than from the hardware.

Given:

- **Operations** (O): $\approx 3.14 \times 10^{23}$ FLOPs (the product of the Data factor, 300 billion tokens, and the Algorithm factor, $6P$ FLOPs/token for 175 billion parameters)
- **Peak Rate** (R_{peak}): 312 TFLOP/s
- **Efficiency** (η_{hw}): $\approx$ 45 percent (an assumed planning value for large-scale distributed training, where communication and stalls keep realized throughput well below peak)
- **Accelerator Count** (N_{GPU}): 1,024 accelerators

Math:

Assume that the training workload reuses data sufficiently to make arithmetic dominant and overlaps data loading with computation. Under this compute-dominated approximation, folding remaining communication and memory stalls into the effective end-to-end utilization η_{hw} across N_{GPU} parallel accelerators yields the training time T_{train}:

- $T_{\text{train}} \approx \frac{O}{N_{\text{GPU}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}}} \approx \frac{3.14 \times 10^{23}}{1024 \times 312 \times 10^{12} \times 0.45} \approx 25$ days

Result: 25 days.

Systems insight: If hardware utilization (η_{hw}) improves from 45 percent to 60 percent through better scheduling and more efficient execution, training time drops to 19 days, saving 6 days of expensive compute time.

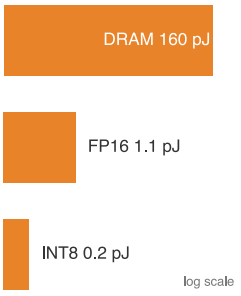
Each term of the iron law resolves to seconds, making the equation dimensionally consistent, yet execution time is not the only physical constraint. For battery-powered edge devices and warehouse-scale accelerator clusters alike, energy dissipation and thermal limits often bind the system before throughput limits are reached. Moving bits across the memory hierarchy exacts an **energy tax** that parallels the delay of the memory term.

↪ The term-by-term unit check behind this statement, and its use as a test for any performance equation, are collected for reference in section D.2.2.

Let D_{vol} be the total data volume moved (bytes), E_{move} the energy per byte moved, O the total operation count, and E_{compute} the energy per operation. Equation 1.5 formalizes this relationship, following the hardware-energy observation that data movement can dominate arithmetic energy (Horowitz 2014):

$$E_{\text{total}} \approx \underbrace{D_{\text{vol}} \times E_{\text{move}}}_{\text{Movement Term}} + \underbrace{O \times E_{\text{compute}}}_{\text{Compute Term}} \tag{1.5}$$

The energy comparison depends on the transfer size and arithmetic operation being counted. Under the energy constants used in this text, moving one byte from off-chip DRAM consumes roughly 145× the energy of a 16-bit floating-point (FP16) multiply and 800× that of an INT8 (8-bit integer) multiply (Horowitz 2014). Whether movement dominates a complete workload also depends on the number and width of transfers relative to its operation count. The physical reason for the per-access gap is that data movement requires charging and discharging wires over longer distances, while arithmetic occurs locally within a processing unit's circuits. Minimizing avoidable data movement (D_{vol}) can therefore improve both speed and energy efficiency.



Moving a byte from DRAM costs about 145 times the energy of a 16-bit floating-point multiply, so data movement, not arithmetic, dominates the energy cost.

↪ The energy ladder from on-chip arithmetic to off-chip memory, and why its gap makes data movement a primary target for accelerator design, are developed in section 11.2.3.

Checkpoint 1.2: The iron law

Before proceeding, verify that you can manipulate both forms of the iron law (equation 1.2 and equation 1.3, where $T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$):

- ☐ **D·A·M product (equation 1.2)**: identify which two factors multiply to set O, and explain the difference between logical samples or tokens (D) in the Data factor and physical bytes moved (D_{vol}) in the memory term.
- ☐ **Memory term** (D_{vol}/BW): identify the physical resource that bounds this term and name one workload regime where it dominates.
- ☐ **Compute term** ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$): identify the hardware quantity and utilization factor that bound this term.
- ☐ **Latency term** (L_{lat}): identify which costs remain after data movement and arithmetic have been optimized.
- ☐ **Peak throughput**: holding the other quantities fixed, if you double R_{peak}, which term improves, and why does total execution time fall by less than half when movement and overhead also contribute?

1.7.1 Lighthouse models put the iron law into practice

Each term of the iron law suggests an engineering intervention. The memory term can be reduced by moving fewer bytes or increasing effective bandwidth. The compute term can be reduced by performing fewer operations or using arithmetic units more effectively. The latency term calls for reducing orchestration and communication overhead. The equation alone does not say which intervention a system needs; that depends on its measured workload and whether the required data fits in memory.

Concrete workloads answer that question. Five recurring **lighthouse models** serve as diagnostic probes for the iron law throughout this book; later chapters explain the model structures and execution mechanisms behind their constraints. Each is a distinct stress case under the workload conditions stated for it:

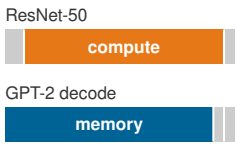
- **Image classification (ResNet-50)** (He et al. 2016a) probes compute throughput. With sufficient reuse of learned filters across image positions and batched inputs, the arithmetic term ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) can dominate execution time.
- **Text generation (GPT-2/Llama)** probes memory bandwidth, with GPT-2 (Radford et al. 2019) anchoring the lighthouse and Llama (Touvron, Lavril, et al. 2023) representing its larger successors. Generating text token by token, the phase called decode, for one request at a time offers little weight reuse per token, so fetching weights from memory (D_{vol}/BW) can become the binding constraint.

- **Recommendation (DLRM)**, the Deep Learning Recommendation Model (Naumov et al. 2019), probes memory capacity, which determines whether the model can run at all. Embedding tables store numerical vectors for user and item IDs. Large tables can reach about a hundred gigabytes in reference models (Naumov et al. 2019), and terabytes in production (Mudigere et al. 2022), exceeding single-accelerator device memory.
- **Mobile vision (MobileNetV2)** (Sandler et al. 2018) probes compute throughput within a power envelope. Its efficient operators cut arithmetic far enough that a phone-class processor can classify frames in real time while sharing a battery and thermal budget with other applications.
- **Keyword spotting (KWS)** probes the power envelope, because an always-on microcontroller[21] must listen continuously for a wake word within a milliwatt budget and kilobytes of memory (Y. Zhang et al. 2017).

[21] **Microcontrollers**: Single-chip computers integrating a processor, memory, and interfaces to sensors or other devices. The keyword-spotting lighthouse has a kilobyte-scale memory budget and a milliwatt-scale power budget. A candidate model must fit those limits as well as meet its prediction-quality requirement.

These five are deliberately not the newest models. Each marks a point where the field turned and isolates one binding constraint in its cleanest form. ResNet-50 made deep convolutional training on accelerators routine and became a reference workload of the MLPerf training benchmark (Mattson et al. 2020). GPT-2 showed that scaling a decoder-only language model changes what it can do. DLRM represents the recommendation models that accounted for most AI inference cycles in Facebook's data centers when it was introduced (Gupta et al. 2020). Later models grow in scale, but the same terms of the iron law bind them, so the analysis in this book should transfer to models that do not yet exist.

The iron law makes these differences actionable under the stated workload conditions. Raising peak compute R_{peak} helps convolutions when arithmetic dominates, while increasing memory bandwidth BW helps generation when weight transfers dominate. Batch size, reuse, and serving patterns can change that diagnosis, so the bottleneck belongs to the workload on a particular machine rather than to the model name alone, which is why table 1.4 identifies each lighthouse by the regime it probes.



Same iron law, opposite dominant term, opposite fix.

Table 1.4: Lighthouse Models as Reference Workloads: Each workload illustrates a resource constraint under the stated conditions, providing a concrete setting for investigating how system constraints affect architectural choices. Each model's architectural background appears in chapter 6, and its quantitative specifications in section 6.9.1.

Lighthouse Model	System Bottleneck	What It Reveals	Diagnostic Target
ResNet-50	Compute throughput under reuse	GPU utilization, batching	Arithmetic throughput versus memory stalls
GPT-2/Llama	Memory bandwidth	Weight and sequence-state movement	Weight and cached-context transfer rates
Deep Learning Recommendation Model (DLRM)	Memory capacity	Embedding tables, memory budgeting	Device memory capacity and partitioning
MobileNetV2	Compute within a power envelope	Efficient operator design	Real-time latency deadlines within battery limits
Keyword spotting (KWS)	Power envelope	Tiny memory and energy budgets	Continuous inference within milliwatt power budgets

The lighthouse models supply the book's recurring workloads; the deployment paradigms described later in this chapter supply the operating envelopes those workloads must fit. The iron law estimates the execution time of one run on one machine. Over the past decade, however, training workloads grew faster than hardware improvements alone could accommodate, making efficiency a central concern.

1.8 Three Dimensions of ML Efficiency

Because training compute grew faster than hardware, every term of the iron law also had to shrink through better algorithms, better utilization, and better data. AlexNet's 2012 run is where that pressure began. Read through the D·A·M taxonomy, its breakthrough was coordination across all three axes rather than algorithmic novelty in isolation: a convolutional architecture whose parallel matrix operations matched GPU capabilities, the 1.3M labeled images of the 2012 ImageNet challenge training split[22] (Deng et al. 2009), and two consumer GPUs running for about six days. A university

[22] **ImageNet**: The full image collection grew from 3.2 million images reported in the initial paper to about 14.2 million across thousands of labeled categories (Deng et al. 2009; Russakovsky et al. 2015). The 2012 challenge used a subset of this collection; AlexNet's training split contained about 1.3M labeled images. Distinguishing the full collection from a benchmark split matters when comparing dataset sizes and training costs.

laboratory could afford that budget. Modern foundation models demand training compute roughly 8 orders of magnitude larger.

Scale drives AI progress, as the bitter lesson (section 1.4) argued, but ever-larger datasets and compute budgets narrow participation to the most resource-rich organizations. Even those organizations eventually meet physical limits in data center power, memory bandwidth, and the diminishing returns of adding more parameters. Using existing compute more efficiently is therefore a complementary path to progress.

Efficiency offers three complementary choices that align with the three factors of equation 1.2: perform less computation per sample, execute that computation in fewer seconds per FLOP, or choose more informative examples so the task needs fewer samples. **Algorithmic efficiency** reduces computational requirements (FLOPs/Sample) through model design and training procedures. **Compute efficiency** lowers realized machine time per operation (Seconds/FLOP) by raising hardware utilization η_{hw} and cutting memory and latency stalls, turning more of the machine's theoretical capacity into useful work. **Data selection** identifies examples that contribute useful learning signal, reducing Samples/Task (and therefore O) when training processes fewer examples; under a fixed computation budget, it can instead redirect work toward more informative examples.

Choosing among them calls for a bottleneck diagnosis across the interfaces in figure 1.4. All three must account for information movement: reducing FLOPs/Sample alone has little effect on wall-clock time when the memory term D_{vol}/BW dominates Seconds/FLOP.

Part III develops these dimensions in the order work flows through a system. Data selection asks which examples merit training computation. Model compression[23] asks how much storage and arithmetic a trained model requires. Hardware acceleration explains how the remaining operations map onto memory and compute units. This teaching order does not prescribe an optimization sequence. The measured binding constraint and deployment budget determine where to intervene first, and hardware limits inform data and model choices from the outset.

The historical trajectory of computer vision models isolates the compounding effect of architectural design. Hernandez and Brown (2020) held accuracy fixed at the AlexNet level and asked how much training computation successive architectures needed to reach it. Because they counted operations rather than seconds, hardware speed does not enter the measurement. Between AlexNet in 2012 and EfficientNet in 2019, the required compute fell by 44×, a halving roughly every 15 months between those two endpoints (figure 1.5). The dashed line in the figure is an exponential fit to those measured points. The reduction measures algorithmic progress at fixed task quality. Comparing it with hardware progress would require a matching measure of useful throughput or energy efficiency; Moore's law describes transistor integration,[24] which alone does not determine either quantity.

Architecture-by-architecture gains tell only half the story, because over the same years aggregate training compute in published frontier runs climbed far faster than algorithms could offset. Figure 1.6 charts that climb on a logarithmic scale, from early deep learning models near 10^{18} FLOPs to frontier systems beyond 10^{25} FLOPs. Sevilla et al. (2022) measured a doubling time of roughly six months across the deep learning era, and a least-squares fit to the runs in the figure gives a similar cadence. An earlier study over 2012 through 2018 found a 3.4-month doubling (Amodei and Hernandez 2018b); different windows and sets of runs yield different estimates. Both studies found training compute growing faster than the roughly two-year doubling associated with Moore's law. This divergence motivates efficiency improvements across algorithms, software runtimes, and hardware architectures, helping keep useful systems within reach as workload demands grow.

The two plots show how efficiency gains can coexist with growth in training scale. Figure 1.5 reports a 44× reduction in compute for a fixed accuracy target, while figure 1.6 illustrates roughly 8 orders of magnitude of growth in selected training runs. They measure different quantities and include contextual estimates, so they do not by themselves establish an economic cause. Efficiency makes a given capability less expensive; organizations can use the savings to reduce spending, broaden access, or pursue more demanding targets.

[23] **Model compression**: A family of techniques that reduces model storage or computation, including quantization (storing values in fewer bits), pruning (removing parameters that contribute little), and distillation (training a smaller model to imitate a larger one). Its size and accuracy effects depend on the model, method, workload, and target hardware.

[24] **Moore's law**: Gordon Moore's 1965 observation described rapid growth in the number of components that could be economically integrated on a chip (Moore 1998); later industry summaries often expressed the cadence as roughly a two-year doubling.

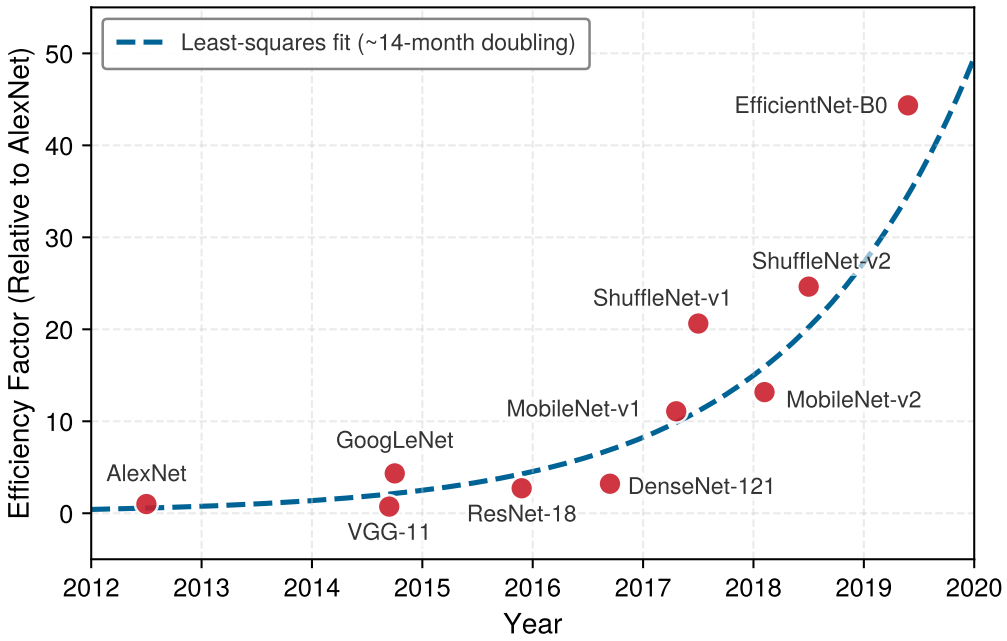


Figure 1.5: Algorithmic Efficiency Trajectory: Each point shows how much less training compute an ImageNet architecture needed than AlexNet to reach AlexNet-level accuracy, so height measures algorithmic progress with hardware speed held out. Most later architectures sit well above AlexNet, although individual points vary; VGG-11 needed more compute than AlexNet. The dashed line is a least-squares exponential fit to the measured points. Data from Hernandez and Brown (2020).

Systems Perspective 1.2: The efficiency paradox

Jevons paradox (Jevons 1865) describes cases in which more efficient resource use leads to greater total consumption because demand expands. A similar response is possible in AI. If EfficientNet needs 44× less compute than AlexNet to reach the same accuracy, an organization can reinvest the savings in larger models or more experiments. It can also choose a smaller budget or serve more users at the same cost. Reinvestment is a possible consequence of efficiency, not an inevitable one, and the plotted trends alone do not establish it.

Whether reinvestment pays is an economic question. The same physical terms that determine time and energy also determine cost: every byte moved, operation executed, and millisecond of latency consumes infrastructure budget. The test is whether added compute buys enough model improvement to justify the expenditure.

1.8.1 Return on compute: Weighing accuracy against cost

A model upgrade that raises accuracy by 1 percentage point can still be the wrong choice if it requires a 10× increase in O (total operations) whose cost exceeds the application's budget or the gain's value. **Return on compute (RoC)** measures the incremental accuracy gain per added dollar of infrastructure investment, in accuracy points per dollar:

$$\text{RoC} = \frac{\Delta\text{Accuracy}}{\Delta\text{Compute Cost}} \tag{1.6}$$

The RoC test compares that ratio against what each point of accuracy is worth to the application. Each alternative is compared against the same baseline, and the ratio is defined only when the added cost is positive. The GPT-3 estimate above demonstrates a different benefit: assuming scheduling changes preserve model quality, raising η_{hw} from 45 percent to 60 percent saves 6 days of cluster

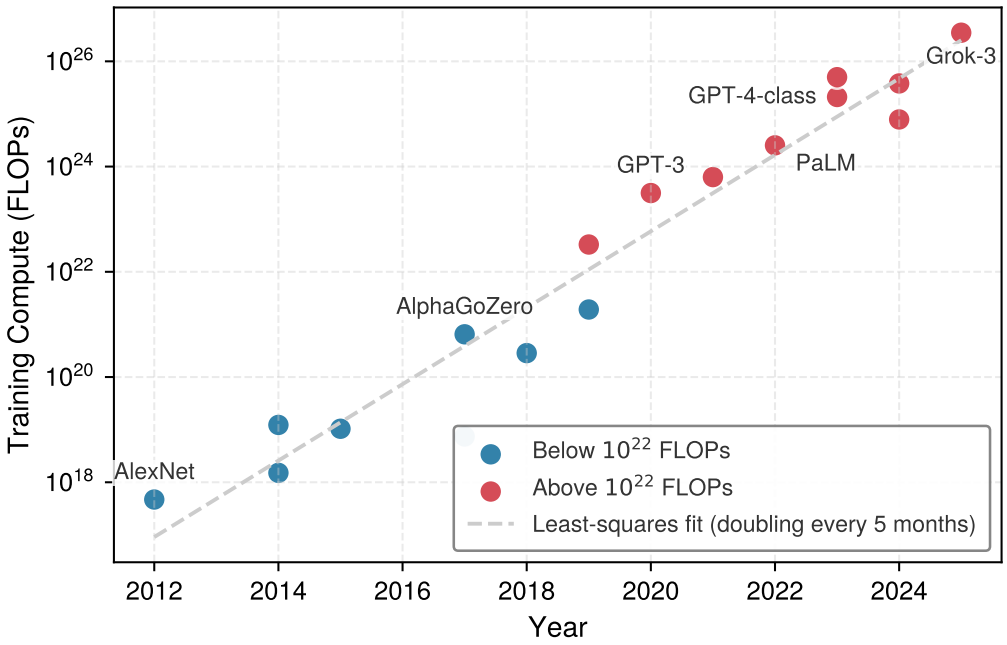


Figure 1.6: The Era of Scale: Each point plots a third-party training-compute estimate (FLOPs) for a notable model against its publication year on a log scale. Runs below 10^{22} FLOPs appear in blue; larger runs appear in red. The dashed line is a least-squares fit to these points and doubles every 5 months. From AlexNet (2012) to GPT-4-class training scale (2023), compute requirements increased by roughly 8 orders of magnitude (about 44.7M times). This endpoint comparison is distinct from training-compute trend studies (Amodei and Hernandez 2018b; Sevilla et al. 2022) and is used here to motivate the specialized infrastructure described in this book. Data from Epoch AI's Notable AI Models database (Epoch AI 2026c).

time for the same training work. This is a cost saving, not an accuracy gain. It can free budget for another experiment, whose incremental accuracy and cost must then be measured. The iron law helps estimate what computation costs; RoC asks whether an additional investment is worthwhile. Efficiency choices must therefore be evaluated against the system's goals and deployment constraints.

1.9 AI Engineering as a Discipline

A cloud service may optimize throughput across racks of accelerators, while an edge device must remain within a strict battery and thermal envelope, so the same model can be efficient in one setting and unusable in another. Holding predictive quality and resource cost to one specification across such different operating budgets requires combining the three quantitative tools developed above: the degradation equation judges predictive quality in both settings, the iron law judges execution time and energy, and the efficiency framework judges the cost of improving either. Applying them draws on computer science, electrical engineering, statistics, and established software and systems engineering practice.

Definition 1.3: AI engineering

AI engineering is the discipline of designing, deploying, and maintaining ML systems whose predictive quality and execution meet stated deployment requirements. It integrates decisions about data, algorithms, and machines, using statistical evidence to assess learned behavior and measurements to assess resource use.

1. **Significance**: A model improvement must still satisfy the other system requirements. AI engineering jointly considers data quality and governance, prediction quality, execution

time, memory, energy, and cost. A model that achieves 95 percent accuracy in research but violates a 100 ms deployment latency requirement does not meet the system specification.

2. **Distinction**: A model-level evaluation establishes evidence under a stated dataset and protocol. AI engineering must carry that evidence into a complete system whose latency, throughput, accuracy, cost, fairness, and robustness meet deployment requirements, and reassess it when operating conditions change.
3. **Common pitfall**: A frequent misconception is that AI engineering is just "software engineering for ML." Software correctness remains necessary, but the system specification also includes statistical claims about learned behavior over an operating population. Code tests cannot establish those claims by themselves, making continuing evaluation and monitoring part of the system design.

A requirement can be fixed while the evidence for meeting it remains uncertain. Pedestrian recall of at least 95 percent over a stated population is a fixed target; evaluation estimates recall from sampled outcomes, with uncertainty that must be reported and reassessed as conditions change. Execution requirements also need a defined workload and measurement protocol. Sampled response times support a statistical latency claim; a hard bound requires assumptions and analysis covering the specified execution conditions. Combining these obligations resembles the earlier convergence of computing and electrical engineering in computer engineering.[25] In this book's usage, AI engineering names the discipline and its shared principles, while ML systems engineering names the work of applying them to build and operate a specific system under its deployment constraints.

[25] **Computer engineering**: A discipline spanning digital circuits, processor design, and the software that runs on them. Moving a function from software into specialized hardware can change its speed, power use, and flexibility. ML systems engineering extends this shared design problem to include learned behavior and the data that shapes it.

1.9.1 Engineering across the ML lifecycle

Shipping a trained model to production does not end the engineering work, because the first shift in user behavior or upstream data can force a deployed system back into data collection and training. Putting an algorithmic advance into service requires data ingestion and preprocessing, computation within the available machine budget, reliable serving, and ongoing evidence of real-world performance. These obligations form a recurring lifecycle rather than a linear sequence that ends at deployment, and the engineering object is no longer only code but also data, model behavior, deployment context, and monitoring evidence evolving together.

At the project level, six responsibilities organize this work: problem definition, data collection and preparation, model development and training, evaluation and validation, deployment and integration, and monitoring and maintenance. Problem definition sets the task and acceptance requirements before any data is collected, so figure 1.7 begins after it and splits the data stage into collection and preparation to show where each feedback path re-enters.

The structural difference shows up first in tooling. Version control, continuous integration, and static analysis have long supported code-defined behavior. For learned behavior, these practices must also track dataset and model versions and test predictive quality, because source-code changes alone do not describe changes to the deployed system.

Feedback must therefore assess learned behavior as well as software execution. In figure 1.7, a failed evaluation sends work back to data preparation, while degraded production performance sends it back to data collection. Feedback determines which work must be repeated; it does not make every alert a retraining instruction. The six stages and the feedback paths that join them into a loop are developed in section 3.1.

↪ The workflow controls that learned behavior adds to established software practice are developed in section 3.1.1.

Each stage of this cycle carries resource demands of its own. Those demands are not uniform, because the binding physical constraints depend on where the system is deployed.

1.9.2 Deployment context shapes the lifecycle

Deployment context determines which lifecycle pressures dominate. The same stages apply across ML systems, but a megawatt-scale data center and a milliwatt-scale embedded device impose different bottlenecks on data collection, model updates, monitoring, and serving. Four **deployment paradigms** span that range (cloud, edge, mobile, and TinyML), and their envelopes sit orders of magnitude apart.

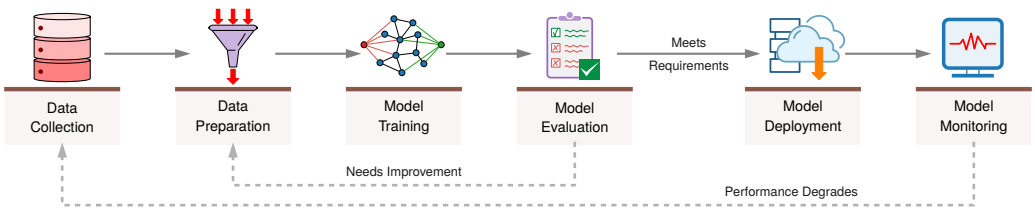


Figure 1.7: ML System Lifecycle: Forward arrows carry work from data collection to monitoring, and the two labeled return arrows mark where evidence sends it back. The return arrows end at different stages, so a failed evaluation and a confirmed production problem trigger different amounts of rework.

From cloud to TinyML, the endpoints differ by roughly 10^5 in memory and 10^6 in compute. Because a cloud model cannot fit directly on a microcontroller, moving down that scale requires compression, a smaller architecture, or a narrower task. At the upper end, cloud ML systems offer large computing budgets alongside capacity limits, operational complexity, and substantial infrastructure bills; at the lower end, TinyML systems on microcontrollers make memory and energy feasibility an immediate concern, as a sensor may need to detect anomalies on a single battery for years. Between these poles, placement determines where work and resource demands fall: edge ML systems move computation toward data sources to reduce latency[26] and network traffic, mobile ML systems share memory, thermal headroom, and battery power with foreground applications, and hybrid systems split work across tiers to balance latency, privacy, bandwidth, and update control.

Tight response deadlines or unreliable connectivity can require local execution, as in vehicle control, whereas fraud detection can run on cloud infrastructure whenever network and service latency fit the decision budget, and large language-model services rely on centralized clusters to accommodate their memory and compute footprint. In turn, cloud capacity and cost govern how often models are retrained, what historical data is retained, and how inference load is distributed, while edge and mobile limits constrain model size and update frequency.

As computation spreads across tiers, operational complexity grows alongside it. Edge and hybrid deployments must coordinate data collection across sensors with intermittent connectivity, track model versions across thousands of devices, push remote updates with safe rollback, and aggregate monitoring signals from distant endpoints. Data pressures pull in opposite directions at the same time: privacy rules and data sovereignty push computation toward the edge, whereas training high-capacity models pulls data back toward centralized clusters. Production architectures resolve this tension by running inference locally at the edge while routing sampled telemetry to cloud clusters for retraining.

In every deployment context, the model eventually meets inputs its evaluation never sampled. Autonomous driving illustrates this hybrid loop, running perception on the vehicle while cloud clusters train updated weights on logged fleet data. Because the Waymo Open Dataset shows pronounced domain gaps among San Francisco, Phoenix, and Mountain View (Sun et al. 2020), and because models that score well on benchmarks often lose accuracy when deployment inputs shift (Koh et al. 2021), every remote update must be revalidated against the population it will actually serve. Beyond predictive accuracy and latency, deployed systems must also satisfy fairness, interpretability, and privacy requirements across data, models, and governance: training sets can encode societal biases, regulated fields such as healthcare and finance require decisions that auditors and clinicians can interpret, and inference attacks[27] can leak sensitive information from trained weights. Meeting these technical, operational, and governance obligations across all four deployment regimes sets the structure of the book that follows.

1.10 Book Organization

Production ML work divides into responsibilities that engineering teams own: data engineering, training systems, deployment infrastructure, operations and monitoring, and ethics and governance (Paleyes et al. 2022). Those boundaries leak. Upstream data pipelines limit what training can reach, training choices fix the memory and latency footprint at serving time, and governance constrains every stage, so the book does not follow an organization chart. Its four parts follow the order in which constraints must be understood. The organizing principle is *context before theory*: physical

[26] **Latency at the edge**: Every millisecond of delay is distance traveled before a decision takes effect. A vehicle traveling at 30 m/s moves 3 cm during each millisecond of delay. Sensing, communication, computation, and actuation all contribute to the time before braking can begin, so reducing network delay addresses only part of the response budget.

↪ The light barrier, power wall, and memory wall that shape these operating envelopes are developed in section 2.2.

[27] **Inference attack**: An attempt to deduce sensitive information about training data from a model's outputs. Memorization can expose information, but the risk depends on the data, training procedure, and access available to the attacker. Defensive techniques include adding calibrated noise during training (differential privacy) and restricting the information returned by the system.

hardware limits and data ingestion bottlenecks constrain algorithmic choices, whose resource demands must in turn be checked against the machine. Readers establish operational boundaries and vocabulary (Part I) before constructing models (Part II), optimizing computational efficiency (Part III), and deploying systems under production service-level agreements (Part IV), the progression summarized in table 1.5.

Table 1.5: Book Organization: The four parts progress from foundations through model construction and training to optimization, deployment, and operations. This introduction precedes them and chapter 16 follows them. Each part builds on the vocabulary and methods of its predecessors.

Part	Question It Answers	Key Chapters
I: Foundations of ML Systems	What is being built, under which constraints, and from what inputs?	chapter 2, chapter 3, chapter 4
II: Development and Training	How does a model compute, how is it structured, and how is it trained?	chapter 5, chapter 6, chapter 7, chapter 8
III: Optimization and Acceleration	How can the model be made cheaper, and how is the gain proven?	chapter 9, chapter 10, chapter 11, chapter 12
IV: Deployment and Operations	Does the system survive real users, changing data, and its obligations?	chapter 13, chapter 14, chapter 15

Each part opens with the named principles it develops. These are numbered across the book so later chapters can refer back to them. The deployment part, for example, opens with the verification gap stated over a deployment population.

This book focuses on the **single-node** regime of one host with one to eight accelerators, each typically using local device memory and communicating through an on-node interconnect such as PCIe or NVLink. The binding constraint depends on the workload and may be memory capacity, memory bandwidth, compute throughput, or communication. Larger fleets add coordination and network constraints across many hosts. For reading paths, learning outcomes, prerequisites, and guidance on using the textbook, the preface provides the orientation.

1.11 Fallacies and Pitfalls

A passing code test, a high held-out score, and a faster inference stage each provide useful evidence, but none alone establishes that an ML system meets its requirements. Mistaking code-level checks for validation of learned behavior overlooks the verification gap and silent degradation, treating a held-out score as proof of deployment readiness ignores distribution shift and execution budgets, and extrapolating a component speedup to the whole system overlooks the untouched costs captured by Amdahl's law. Each of these errors applies evidence beyond the conditions that support it.

Fallacy: *Better algorithms automatically produce better systems.*

A more accurate spam classifier can still be a worse production choice if it needs too much memory or cannot classify incoming messages within the delivery budget. The iron law (section 1.7) explains why algorithmic quality alone does not settle that decision. Model selection must maximize task quality subject to latency, memory, energy, cost, and reliability requirements. A simpler model supported by representative data and reliable infrastructure can meet those requirements more effectively than a sophisticated model whose surrounding system is inadequate.

Pitfall: *Treating ML systems as traditional software that happens to include a model.*

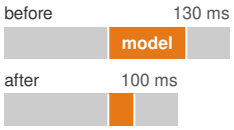
Established testing and deployment practices remain necessary for ML systems, but code-level checks alone cannot validate learned behavior (section 1.6). Both conventional and ML software can fail silently. In the driver-assistance example, pedestrian recall can fall from 95 to 85 percent while the accelerator keeps its full frame rate and raises no fault. ML systems additionally require outcome monitoring to detect declining predictive quality, with observation windows and sample sizes sufficient to resolve relevant differences across subpopulations. Teams deploying ML with only continuous integration and continuous deployment (CI/CD) pipelines risk silent failures, in which test suites and health checks report green while user-facing prediction quality degrades.

Fallacy: *High accuracy on benchmark datasets indicates production readiness.*

A benchmark score describes performance on a stated dataset under an evaluation protocol; it does not establish deployment readiness (section 1.9.2). The ILSVRC validation set holds 50,000 images, while the number of possible inputs has 362,508 digits, so even a perfect score is evidence about a sample, not proof about the population. A sentiment model may lose accuracy when users employ slang or context absent from the test set. Deployment also introduces resource requirements: mobile memory and latency limits may make a multi-model ensemble impractical, while network round trips can consume the response budget. Production validation must cover the intended operating conditions, relevant subgroups, resource use, and monitoring needed to detect change.

Pitfall: *Optimizing individual components without considering system interactions.*

Amdahl's law, which holds that the untouched share of a task caps the speedup of the whole, explains why improving inference alone may produce a small end-to-end gain. Consider a team that reduces model inference from 45 ms to 15 ms, expecting proportional improvement. Preprocessing still consumes 60 ms and postprocessing adds 25 ms, so total latency drops only from 130 ms to 100 ms. That is a 23 percent improvement in total latency rather than the 67 percent improvement of the inference stage alone. Most of the request path was untouched. Measuring the whole request prevents a component speedup from being mistaken for a system speedup; the bound in equation 11.1 computes the overall speedup from the share of time a component occupies and its local gain.



Making the model three times faster trims end-to-end latency by only 23 percent.

Fallacy: *ML systems can be deployed once and left to run indefinitely.*

Deployed ML systems cannot assume static performance over time: environmental distribution shifts can degrade a frozen model even when underlying execution code remains untouched. In this illustrative scenario, a recommendation system deployed at 85 percent accuracy drops to 80.2 percent within 6 months as purchasing patterns shift, losing 4.8 percentage points without any code changes. The ML lifecycle (section 1.9.1) therefore treats outcome monitoring and evidence-based retraining as operational requirements. Without continuous monitoring, systems can appear healthy while prediction quality silently erodes.

Pitfall: *Assuming that ML expertise alone is sufficient for ML systems engineering.*

Production ML (section 1.9.2) requires expertise across algorithms, software, systems, and operations. A team with strong ML skills can still miss throughput targets if storage layout or serving infrastructure is inadequate. Conversely, infrastructure built without attention to model inputs can introduce preprocessing defects that degrade predictions. Deployment case studies show the need for coordinated attention to data, models, infrastructure, and organizational workflow (Paleyes et al. 2022). Failures can arise at the boundaries, where one team's assumptions become another team's inputs.

1.12 Summary

Machine learning systems must satisfy two obligations at once: learned behavior must remain trustworthy, and the machine must deliver that behavior within physical and economic limits. Every ML system answers to three questions: what shapes its predictions, what limits its execution, and what evidence supports its deployment. The D·A·M taxonomy organizes that diagnosis, the degradation equation and the iron law quantify the trade-offs, and efficiency together with return on compute guides the choice of interventions. Because data and operating conditions keep changing, those answers never stay fixed, and five lighthouse models (ResNet-50, GPT-2/Llama, DLRM, MobileNetV2, and keyword spotting) recur throughout the book to ground these methods in concrete workloads.

Return to the smartphone interaction that opened the chapter. What appeared to be one intelligent action depended on representative data, a learned model, a machine within its operating budget, and feedback that could reveal changing behavior. That chain answers the question posed at the outset: an accurate model alone does not establish a reliable, efficient system. AI engineering coordinates these decisions and uses measured evidence to assess whether the complete system meets its requirements.

Key Takeaways: Constraints drive architecture

- **Relieving a bottleneck can expose another**: Data, Algorithm, and Machine constraints interact. After an intervention, measure the complete system again to determine which constraint now limits the result.
- **Learned behavior can decay silently**: ML systems can lose predictive quality while code and infrastructure stay fixed because deployment inputs change. Drift metrics prompt investigation; labeled outcomes establish whether prediction quality has fallen.
- **End-to-end time is the whole path**: Data movement, computation, and overhead all spend from the same time budget. Cutting inference from 45 ms to 15 ms gives only 23 percent improvement when preprocessing (60 ms) and postprocessing (25 ms) dominate, so optimize the term that binds end-to-end behavior.
- **Scale needs useful computation**: The bitter lesson emphasizes search and learning methods that can use increasing computation effectively. Their benefit depends on data, objectives, and a supporting machine. Efficiency gains of 44× at a fixed accuracy target coexisted with roughly 8 orders of magnitude of growth in selected training runs.
- **AI engineering is continuous co-design**: Deployment requirements guide decisions about data, models, and execution. Evaluation and lifecycle monitoring provide evidence that predictive quality and resource use continue to meet those requirements.

An intended capability becomes an engineering result only when measured behavior meets the goal within a feasible resource budget. Data, learning objectives, design choices, and hardware all shape that result. The first engineering act is therefore a measurement of the complete system, not an optimization of its most visible part.

What's Next: From vision to architecture

This chapter asserted that the four deployment paradigms have operating envelopes orders of magnitude apart without deriving the gap. Chapter 2 derives it from physical limits and turns it into a decision framework for choosing among them when requirements conflict.

I

FOUNDATIONS OF ML SYSTEMS

Part I

Foundation Principles

Complexity in a machine learning system often moves rather than disappearing, although good design can remove accidental complexity. Complexity flows among the three domains of the D·A·M taxonomy: Data as information, Algorithm as logic, and Machine as physics. Simplifying one domain can burden the others. A hand-crafted feature pipeline reduces algorithmic complexity but demands more data engineering effort. A larger model absorbs messy data but shifts complexity onto the hardware that must train and serve it. This conservation-of-complexity heuristic recurs throughout this book. The quantitative bounds, models, and principles introduced throughout the book describe constraints that emerge from where complexity currently resides.

Architectures, frameworks, and optimizations succeed only when they respect constraints imposed by hardware, mathematics, and information theory. Just as civil engineers cannot ignore gravity, ML engineers cannot ignore the physical laws that govern data, computation, and system throughput. The introduction argued, through the bitter lesson, that progress comes from general methods that scale with data and computation, and it named the iron law that accounts for the cost of that scale. Part I establishes the foundations on which that pricing rests: where a system can physically run, when its constraints must be known, and how the data it learns from is engineered. The principles in this book are not best practices that evolve with frameworks or opinions that differ between teams, and they are not universal laws either. Each principle box opens by naming its kind (bound, decomposition, fitted model, policy, or heuristic), and each holds only under the assumptions it states. Part I's two principles concern data, which every Part I chapter must account for. Chapter 2 uses data gravity to decide where computation runs, and chapter 4 turns the pair into the two bounds data places on a system, data as code governing what a model can learn and data gravity grounding the physics of moving bytes that bounds how fast the machine can learn it.

Principle 1: The data-as-code principle

Heuristic: Data functions as the source code of the ML system. A change to the training dataset can alter the system's learned behavior. Here, learned behavior means the trained input-output mapping; Train denotes the training process, and $\approx$ marks a qualitative dependency rather than a predictive equality. Model architecture, initialization, optimizer configuration, and training stochasticity are included in the training procedure.

$$\text{Learned Behavior} \approx \text{Train}(\text{Training Data}, \text{Training Procedure})$$

Implication: Data engineering requires the same rigor as software engineering. Datasets should be versioned (like Git), unit-tested (data quality checks), and debugged. Deleting a row of training data can alter the rebuilt model just as deleting a line of code can alter a compiled artifact.

If data functions as source code, then it is not merely a logical artifact; it also has physical properties that constrain system architecture when data and computation occupy different locations. Moving terabytes of raw sensor or text data across a constrained link costs far more time and energy than shipping the training or query logic to where those bytes already reside.

Principle 2: The data-gravity principle

Heuristic: As data volume (D_{vol}) grows, transferring the dataset can cost more than sending a much smaller query or code payload over the same constrained link (McCrory 2010). Here, cost denotes one transfer metric at a time, such as elapsed transfer time or bytes sent, under identical link conditions; it does not combine latency and bandwidth into one quantity.

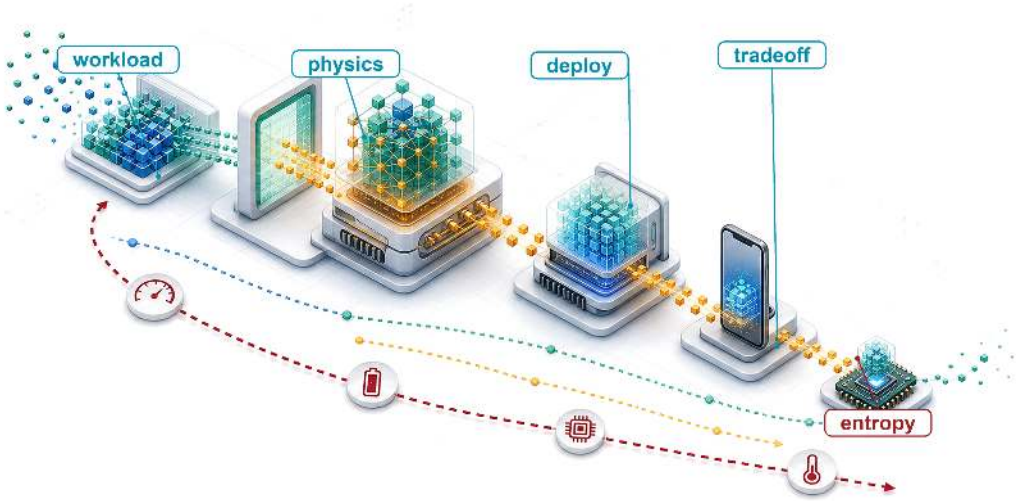
$$\text{Cost to transfer dataset} \gg \text{Cost to transfer query or code} \quad \text{in this regime}$$

Implication: Large datasets can become the gravitational center of the architecture. Systems may move compute to data by shipping queries or code to the storage layer rather than moving data to compute by repeatedly downloading large datasets.

Together, these two principles establish that data can be both the logical program and a physical anchor of an ML system. Part I develops these implications as a dependency chain. Chapter 2 derives from physical limits where a system can execute; chapter 3 carries those constraints around the lifecycle, where a constraint discovered late invalidates every artifact built before it; and chapter 4 builds the pipeline that meets both data bounds, versioning what the model learns from and delivering it as fast as the accelerator consumes it. Ignoring any link returns complexity as integration cost: an accurate model that cannot execute, a workflow that cannot reproduce its inputs, or a pipeline whose outputs no longer match deployment conditions.

2

System Paradigms

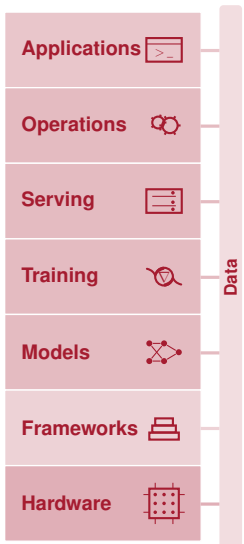


2.1 Four Deployment Regimes Across Nine Orders of Magnitude
2.2 Physical Constraints on Deployment
2.3 Analyzing Workloads
2.4 System Balance and Hardware
2.5 Cloud ML: Computational Power
2.6 Edge ML: Latency and Privacy
2.7 Mobile ML: Offline Intelligence
2.8 TinyML: Ubiquitous Sensing
2.9 Choosing a Deployment Paradigm
2.10 Hybrid Architectures
2.11 What Every Tier Shares
2.12 Fallacies and Pitfalls
2.13 Summary

Purpose

Why does deploying the same model to a phone vs. a data center demand fundamentally different engineering?

The defining insight of ML systems engineering is that constraints drive architecture. The speed of light sets an absolute floor on how quickly distant servers can respond, thermodynamics limits how much computation can occur in a given volume before heat becomes unmanageable, and memory physics makes moving data more expensive than processing it. Because these physical limits persist as technology improves, deployment categories organize recurring constraint regimes rather than divide hardware into fixed, mutually exclusive classes. A data center can train billion-parameter models yet cannot guarantee low-latency responses to users thousands of miles away; a smartphone can respond immediately yet holds a fraction of the memory budget and must share a small battery with the rest of the phone; and a microcontroller can run on a coin-cell battery for years while offering barely enough compute for a simple keyword detector. Across these regimes, power budgets span roughly nine orders of magnitude, from megawatts in a data center to milliwatts on a microcontroller, while memory capacity spans from terabytes to kilobytes. The same model applied to the same data therefore demands radically different engineering in each regime, not because the physics changes, but because different constraints dominate. The engineering task is not to identify a universally best deployment, but to locate the constraint that binds in the intended operating regime. In D·A·M terms, understanding these regimes turns deployment from an operational afterthought into a co-design problem in which data locality, algorithm structure, and machine constraints jointly determine what is possible.



Learning Objectives

- Explain how physical constraints shape deployment paradigms from cloud to TinyML
- Diagnose which term of a tier's silicon contract binds a workload, separating iron-law limits from memory and energy feasibility
- Match workload demands to deployment budgets using lighthouse model examples
- Compare cloud, edge, mobile, and TinyML by operational constraints and quantitative trade-offs
- Apply the decision framework to select paradigms by privacy, latency, resource, and cost constraints
- Analyze hybrid patterns that combine paradigms to satisfy system constraints
- Evaluate deployment placements against data-locality deadlines, energy per inference, and lifecycle cost

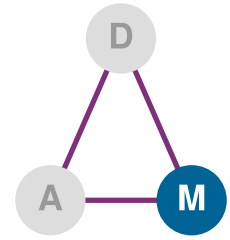
2.1 Four Deployment Regimes Across Nine Orders of Magnitude

Consider two extremes: a wake-word detector on a smartwatch and a recommendation engine in a data center. The wake-word detector represents a TinyML workload operating under milliwatt power budgets and kilobyte memory limits; the recommendation engine exemplifies a cloud ML workload requiring terabytes of embedding tables and megawatt-scale infrastructure. These systems solve different problems under opposite physical constraints, and their supporting infrastructure differs sharply in scale and design. The introduction's D·A·M taxonomy (figure 1.4) treated the machine as one of three interacting components. Between these two extremes, the machine is the component that changes most, and its physical limits decide which data and algorithmic choices remain open.

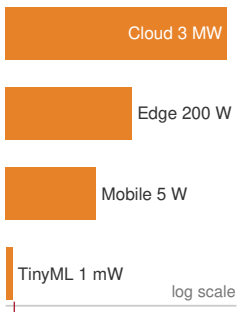
Physical constraints determine where an ML model can run and shape what is possible in ways no algorithmic choice can override. They bind the whole system, not only the weights: data ingestion pipelines, feature extraction, serving runtimes, telemetry, and hardware resource managers must all fit the host platform's physical envelope.

Three physical constraints (latency, power dissipation, and memory) organize four deployment paradigms.[1] The categories overlap—edge describes proximity to data, while mobile and TinyML further distinguish local devices by their resource envelopes—and the combination of a workload and its target platform determines which constraint binds first. Centralizing computation in Cloud ML provides elastic compute and large memory capacity at the cost of wide-area network latency on every inference; moving workloads to Edge ML eliminates that network round trip and keeps sensitive data on premises (Shi et al. 2016), but caps available compute at local site power and cooling budgets. Stepping onto battery-powered devices, Mobile ML delivers responsive interaction directly on handsets and tablets within a passive thermal envelope where sustained computation triggers throttling, and at the low-power extreme, TinyML runs on microcontrollers (Janapa Reddi, Plancher, et al. 2022) whose milliwatt power budgets and kilobyte memories require compact models and integer arithmetic. Along the way, the memory constraint changes form: in the data center it usually binds as bandwidth, whereas on a microcontroller it binds first as capacity. Arranged along one physical continuum, these tiers run from warehouse-scale megawatts in data centers down to milliwatts on embedded microcontrollers (figure 2.1).

While figure 2.1 traces the continuous progression from embedded sensors to data centers, table 2.1 quantifies conceptual operating envelopes across processing location, round-trip latency, power draw, and memory capacity. The cloud row describes facility power; the other rows describe local systems or devices. These envelopes compare deployment scales, whose physical basis is developed in section 2.2.



The machine axis sets the deployment envelope.



Deployment paradigm: A distinct operating regime defined by latency, power, memory, and connectivity constraints. The Cloud-to-TinyML spectrum changes which resources bind; optimizing within an infeasible regime cannot satisfy the application's deployment budget. Raising a cloud model's throughput by 5 percent, for example, cannot rescue an application whose 10 ms latency budget the network round trip alone exceeds.

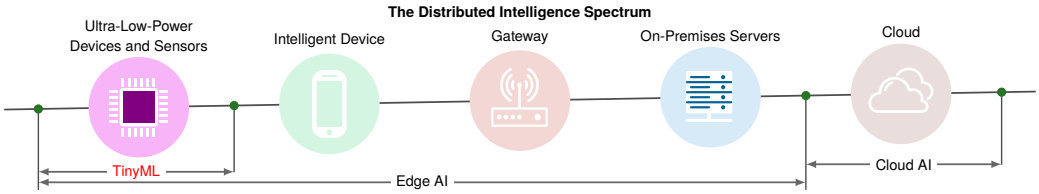


Figure 2.1: Distributed Intelligence Spectrum: Deployment tiers form a continuum governed by physical constraints, spanning nine orders of magnitude in power. Moving from centralized cloud infrastructure to on-device TinyML trades raw computational capacity and elastic scale for shorter communication paths, data locality, and network independence.

Table 2.1: The Deployment Spectrum (Conceptual): Four paradigms span nine orders of magnitude in power (MW to mW) and memory (TB to KB). This conceptual overview defines each paradigm by its operating regime, and each power entry names a typical class rather than a bound; the concrete hardware spectrum in table 2.4 grounds these categories in specific platforms and quantitative decision thresholds. The hardware specifications and physical constants underpinning these numbers are catalogued in Appendix E.

Paradigm	Where	Latency (ms)	Power	Memory	Best For
Cloud ML	Data centers	100-500	MW	TB	Training, complex inference
Edge ML	Local servers	10-100	100 W	GB	Real-time inference, privacy
Mobile ML	Smartphones	5-50	3–5 W	GB	Personal AI, offline
TinyML	Microcontrollers	1-10	mW	KB	Always-on sensing

At every tier, the hardware offers a **silicon contract**: a compute rate, a memory bandwidth, and an execution overhead. Different tiers offer different budgets for these resources, and the contract also carries a memory capacity, which is not a term of execution time but a precondition that decides whether a workload fits on the tier at all. Physical constraints set those budgets, and each budget enters one term of the iron law of ML systems, which decomposes end-to-end execution time into data movement, computation, and system overhead. The speed of light, thermal dissipation, and memory signaling shape those terms, which is why deployment paradigms exist as distinct engineering regimes rather than arbitrary points on a continuous slider.

2.2 Physical Constraints on Deployment

A safety system with a 10 ms reaction budget cannot wait for a cross-country round trip, and a billion-parameter model cannot be squeezed into a microcontroller by better code alone. Both follow directly from the physical laws governing the speed of light, power thermodynamics, and memory signaling. The light barrier limits remote responsiveness, the power wall restricts sustained throughput, and the memory wall caps data transfer. Where a system runs changes the operational terms the hardware offers, which changes what the hardware can offer the model.

↩ A first sketch of the deployment spectrum and its three walls is established in section 1.9.2.

The light barrier

Sending a sensor packet from California to a cloud data center in Virginia (~3,600 km straight-line) requires ~36 ms round-trip in optical fiber before a single floating-point operation begins, and real networks add routing, protocol, queuing, and processing delays on top. Any application that requires a sub-10 ms response *cannot* rely on distant cloud infrastructure, no matter how fast the remote accelerators are. The **light barrier** sets this absolute latency[2] floor. Let Distance denote the one-way path length and c_{fiber} the propagation speed in optical fiber. Equation 2.1 gives the minimum round-trip time:

$$L_{\text{lat,min}} = \frac{2 \times \text{Distance}}{c_{\text{fiber}}} \approx \frac{2 \times \text{Distance}}{200{,}000 \text{ km/s}} \tag{2.1}$$

For optical fiber, $c_{\text{fiber}} \approx 200{,}000$ km/s, roughly two-thirds of its vacuum value because light travels more slowly through glass than through a vacuum. This propagation floor directly motivates edge ML: when a tight latency budget rules out the round trip, computation must move closer to the data source.

The power wall

Doubling a processor's clock frequency to finish a model's arithmetic in half the time does not merely double its power draw; because higher frequencies typically require higher supply voltages to switch transistors reliably, dynamic power can jump roughly eightfold, turning every extra watt into heat that the package and chassis must carry away. That thermal ceiling is the **power wall**, which limits how much computation can occur in a given volume. Dynamic CMOS power follows equation 2.2, where C is capacitance, V is supply voltage, and f is clock frequency. Under the illustrative assumption that voltage rises proportionally with frequency ($V \propto f$), dynamic power rises cubically (Power $\propto f^3$). The exact voltage-frequency relationship varies by device, but increasing both consumes thermal headroom rapidly.

$$\text{Power} \propto C \times V^2 \times f \quad \text{where } V \propto f \implies \text{Power} \propto f^3 \tag{2.2}$$

When broad voltage scaling broke down in the mid-2000s,[3] that cubic penalty ended "free" frequency speedups and forced hardware designers toward parallelism (multi-core CPUs) and domain-specific specialization (GPUs and Tensor Processing Units (TPUs)[4]). Figure 2.2 records that turn across five decades of processors. From the mid-2000s to the early 2020s the largest chips gained about 190× more transistors, yet the fastest clock stayed near 4.7 GHz and single-thread performance rose only 5×, against 21× in the decade before; the added transistors went into more cores and specialized units rather than a faster single thread.

The same limit is tightest on mobile devices, which hit a thermal ceiling at 3–5 W; exceeding it causes "throttling," where the device reduces performance to prevent overheating, a trace worked through in notebook 2.9. This physical limit gives rise to mobile ML: battery-powered devices cannot simply run cloud-scale models locally.

The memory wall

An accelerator's matrix units can finish a block of multiplications in nanoseconds, yet spend most of an inference step sitting idle while weights and activations trickle in from off-chip DRAM. The **memory wall** (Wulf and McKee 1995) names this widening gap between processor speed and memory delivery, which shows up in ML chiefly as a bandwidth[5] shortfall. A rough sketch with representative annual growth factors shows why the gap compounds:

$$\frac{\text{Compute Growth Rate}}{\text{Memory Bandwidth Growth Rate}} \approx \frac{1.6}{1.2} \approx 1.33 \tag{2.3}$$

In equation 2.3, the numerator and denominator are dimensionless annual growth factors. A ratio above 1 means compute capability is pulling away from memory bandwidth, so each hardware generation makes data movement a larger share of the performance problem unless the workload increases locality or arithmetic intensity (operations performed per byte moved).

[2] **Latency**: The time between issuing a request and receiving a result, corresponding to T in the iron law, where propagation delay enters through the fixed-overhead term L_{lat}. The light barrier makes that propagation floor irreducible, as equation 2.1 quantifies. Every millisecond consumed by distance is a millisecond unavailable for model inference, which is why the light barrier forces paradigm selection rather than mere optimization.

[3] **Dennard scaling**: Named after Robert Dennard, whose IBM team described MOSFET scaling relationships under which shrinking devices could reduce voltage and current while keeping power density approximately controlled (Dennard et al. 1974). As voltage scaling slowed and energy became a first-order architectural constraint, performance growth shifted toward parallelism and specialization: multi-core processors, GPUs, and TPUs (Hennessy and Patterson 2019; Esmaeilzadeh et al. 2011).

[4] **Tensor processing unit (TPU)**: An application-specific integrated circuit (ASIC) designed for operations common in ML, especially matrix multiplication. PFLOP/s-to-EFLOP/s figures describe aggregated TPU systems rather than one chip. Specialization trades general-purpose flexibility for workload-dependent efficiency; its advantage over a GPU depends on the model, numerical format, and systems being compared (Jouppi et al. 2023).

[5] **Memory bandwidth (the memory wall)**: The term "memory wall" comes from Wulf and McKee (1995), who predicted that the processor-memory performance gap would eventually dominate system performance, a prediction borne out for ML workloads in which weight or activation movement can bind performance. Whether bandwidth binds a given workload depends on arithmetic intensity, batch size, data reuse, access pattern, and hardware balance rather than model size alone.

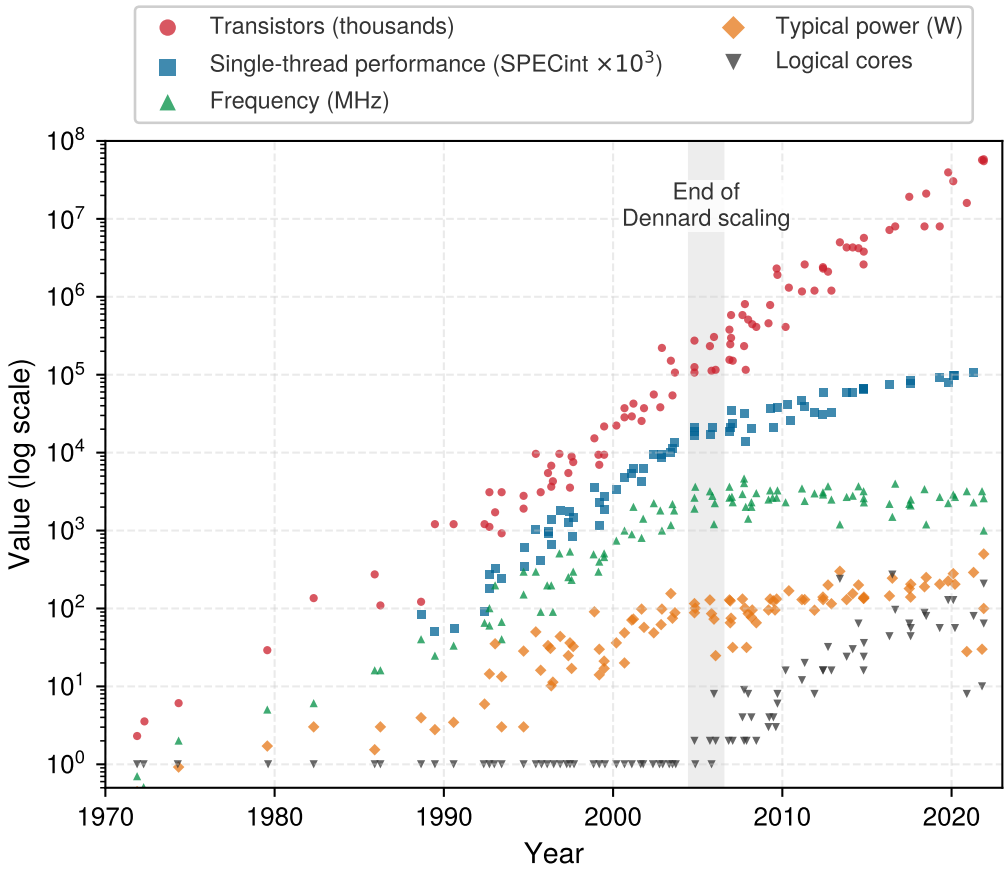
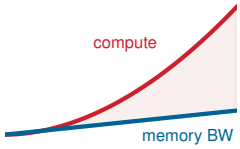


Figure 2.2: Fifty Years of Microprocessor Trends: Transistor count, single-thread performance (SPECint benchmark score), clock frequency, typical power, and logical core count of microprocessors from 1971 to 2021 on a logarithmic axis. Transistor counts keep rising after the mid-2000s while frequency and typical power level off and single-thread performance slows; the gray band marks the end of Dennard scaling, after which added transistors went into more cores. Data from Karl Rupp (2022).

This illustrative trend comparison assumes that compute capacity doubles every 18 months, about 1.6× per year (the numerator of equation 2.3), while memory bandwidth grows by ~20 percent annually (the denominator). These assumed rates illustrate the widening gap rather than a forecast for every hardware generation. Sustained for a decade, those rates leave compute about 16× further ahead of bandwidth than at the start. This widening gap makes data movement a frequent bottleneck, in both time and energy, for ML workloads. An on-chip arithmetic logic unit (ALU) operates over short local connections, whereas an off-chip DRAM access also drives memory interfaces and longer interconnects. In representative hardware, fetching operands from DRAM can cost orders of magnitude more energy than the arithmetic performed on them (Horowitz 2014). The ratio depends on the operation, numerical precision, memory technology, and amount of data transferred. The constraint affects all paradigms, and it takes a different form on TinyML devices, where capacity, not bandwidth, is usually exhausted first.



Compute capacity outruns memory bandwidth; the widening gap is the memory wall.

> Checkpoint 2.1: Physical constraints and deployment
>
> Deployment choices are governed by physics, not just preference. Check your understanding:
>
> - ☐ **Light barrier**: Can you use the cross-country propagation bound to explain why that distant cloud path cannot meet a < 10 ms safety deadline?

↪ The architectural responses to the memory wall, high-bandwidth memory (HBM) and on-chip SRAM hierarchies, are developed in section 11.5.3.

- ☐ **Power wall**: Can you explain why a workload that exceeds a phone's sustained heat-dissipation budget cannot maintain its initial performance?
- ☐ **Memory wall**: Can you explain why data movement is often more expensive (in time and energy) than computation?

These physical laws explain the trade-offs behind the four paradigms. Physical limits decide which designs can work at all; privacy regulation, economic incentives, and data sovereignty requirements further restrict the choices within those limits. No regulation can make the speed of light faster, and no economic model can repeal thermodynamics. Nor do those walls act in isolation: in practice, an ML system is limited by whichever physical boundary binds on its primary resource first. Evaluating these limits quantitatively requires analytical tools: the iron law to decompose execution time into its components, the bottleneck principle to identify the rate-limiting stage, and workload archetypes to classify the model's resource demands.

2.3 Analyzing Workloads

A vision model that saturates accelerator compute units during batched training can stall on memory bandwidth when serving one image at a time, and a model that runs effortlessly in a data center can overheat a phone in under a minute. Deciding which physical resource binds first on the target hardware—whether for a recommendation engine or a wake-word detector—requires the iron law of ML systems from section 1.7, restated here in the same notation because every tier in this chapter is analyzed through it. For a serialized execution path, equation 2.4 adds three costs:

$$T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \tag{2.4}$$

Equation 2.4 decomposes total latency into three terms: data movement (D_{vol}/BW), compute ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), and fixed overhead ($L_{\text{lat}}$). For an inference path that executes these stages sequentially, their times add. A single request can also contain overlapping work, so its latency must follow the actual dependencies. In production systems, however, tasks are processed continuously as a stream, and the analysis shifts from single-task latency to identifying which term limits the system. The answer depends on the workload, its execution conditions, and the deployment hardware. A workload is **compute bound** when its compute term $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ exceeds its data term D_{vol}/BW, and bandwidth-bound in the reverse case.

2.3.1 The bottleneck principle

Adding a twice-as-fast accelerator to a serving pipeline does nothing for steady-state throughput if the processor spends half of every step waiting for the memory bus to deliver the next batch of weights. Where the iron law quantifies the cost of each stage, the **bottleneck principle** identifies which stage governs continuous throughput. Section 1.7 gave the overlapped form of the iron law for **pipelined execution**, in which the memory system prefetches batch $k+1$ while the accelerator computes on batch k. The slowest stage then hides the others and sets the interval between completions, so speeding a nonlimiting stage does not raise ideal steady-state throughput while the system bottleneck remains unchanged. Deployment adds one more overlapped stage, T_{IO}, the time to transfer data over a storage or network link, including offloading a request to a remote server. Equation 2.5 extends the pipelined form with that stage. Here the left-hand side is the steady-state time per completed task, with fixed overhead treated as nonoverlapped; equality requires ideal overlap of the other stages:

$$T_{\text{pipelined}} \geq \max\left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}, T_{\text{IO}}\right) + L_{\text{lat}} \tag{2.5}$$

When a system is bandwidth-bound ($D_{\text{vol}}/\text{BW} > O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), faster processors ($R_{\text{peak}}$) do not raise ideal pipeline throughput while memory remains the slowest stage, just as widening a highway cannot help traffic constrained by a narrow bridge. Shortening a sequential compute stage can still reduce an individual request's latency. Engineers must identify the dominant term before

optimizing. The *additive form* in equation 2.4 describes a single task's serialized path and the *max form* in equation 2.5 an ideally overlapped continuous stream; the actual dependencies determine which stages can overlap.

2.3.2 Workload archetypes

The bottleneck principle reduces optimization to identifying the dominant constraint for a given workload. The answer depends on all three D·A·M axes at once: the data a request carries, the algorithm's arithmetic and memory demands, and the machine that executes it. Different deployment environments create different bottlenecks along these axes, so the same model family can require different engineering when it moves from a cloud server with terabytes of memory to a microcontroller with kilobytes. Classifying workloads by what binds them first yields four **workload archetypes**.[6] Each archetype is a recurring physical bottleneck that determines which engineering moves can help. Two of them correspond to terms of the iron law. The other two are bound by preconditions the time equation assumes are already met, namely that the working set fits in memory and that the energy budget can pay for the run. A capacity or energy limit can therefore decide feasibility before any term of equation 2.4 is evaluated.

[6] **Workload archetype**: A classification of ML workloads by their dominant bottleneck rather than by their model family. Misidentifying the archetype wastes optimization effort on the wrong term of the iron law, as when teams add accelerator FLOP/s to a bandwidth-bound inference pipeline and obtain little improvement while memory transfer remains dominant.

The first split, between the two iron-law terms, separates arithmetic-bound systems from data-movement-bound systems. A **compute-bound** workload performs many calculations per byte loaded, so progress comes from higher arithmetic throughput, better utilization, and more parallel execution; large neural-network training is the canonical case. A **bandwidth-bound** workload, by contrast, waits on dense weight or activation movement, so wider memory buses and better data reuse matter more than additional peak FLOP/s; autoregressive text generation illustrates this regime. The roofline literature calls this regime *memory-bound*; this book says bandwidth-bound to keep it distinct from running out of memory capacity, which is the next archetype.

The second split covers workloads where the binding constraint is not a term of the time equation at all. **Capacity-bound** workloads are dominated by irregular table lookups and poor cache locality, so memory capacity, access latency, and communication shape performance in recommendation systems with massive embedding tables. **Power-bound** workloads face the opposite envelope: energy per inference and memory footprint, not raw speed, determine whether always-on sensing can run at all.



Lighthouse 2.1: Five reference workloads

The introduction (section 1.7.1) introduced five lighthouse models by the system bottleneck each isolates (table 1.4); table 2.2 pairs each with a representative workload archetype and deployment paradigm. The classifications describe operating regimes: batched image processing can emphasize arithmetic, single-request text generation can emphasize weight transfers, and a large recommendation table can exceed local memory capacity. Changing batch size, model size, or hardware can change the binding constraint. MobileNetV2 shows why. In table 2.2, it sits with the compute-oriented vision workloads because its pointwise convolutions carry most of its arithmetic, sized down to fit mobile thermal and energy envelopes. Its depthwise layers, however, perform few operations per byte and are often bandwidth-bound on mobile accelerators. Whether arithmetic throughput or memory bandwidth ultimately binds MobileNetV2 on a specific device depends on target hardware balance, precision, and batching, as the roofline analyses in section 2.4 demonstrate. Chapter 6 opens up the architecture behind each lighthouse.

Table 2.2: Five Lighthouse Models: Five recurring workloads ground the iron law in concrete practice throughout the book. Each lighthouse pairs a home regime (compute-bound, bandwidth-bound, capacity-bound, power-bound) with the deployment paradigm where it predominantly runs, illustrating a distinct resource constraint under representative execution conditions.

Lighthouse	Archetype	Deployment Paradigm
ResNet-50	Compute-bound	Cloud training, edge inference
GPT-2/Llama	Bandwidth-bound (decode)	Cloud inference
DLRM	Capacity-bound	Cloud for large tables
MobileNetV2	Compute-bound (efficient regime)	Mobile, edge
Keyword Spotting (KWS)	Power-bound	TinyML, always-on

Large-scale compute-bound training and capacity-bound workloads gravitate to cloud data centers; compact vision workloads operating in efficient compute-bound regimes such as MobileNetV2 can fit mobile and edge devices; bandwidth-bound decoding runs in the cloud unless a smaller model fits locally; and power-bound workloads belong on microcontrollers. Those placements become concrete once each lighthouse is given its cost. The image classifier **ResNet-50** spends approximately 8.2 GFLOP per image on 25.6 million parameters, which occupy 102.4 MB at FP32 (He et al. 2016a). That regular, compute-dense structure makes it the canonical benchmark for hardware accelerator performance.

The language models **GPT-2/Llama** exemplify autoregressive text generation (Radford et al. 2019; Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023). These models generate text one token at a time. Because multi-billion parameter models exceed on-chip cache, weights reside in external memory; generating text token-by-token for one request at a time forces the processor to stream the weights across the memory bus for each token with little reuse across inputs (1.5 billion parameters for GPT-2, 7 billion–70 billion parameters for Llama). This sequential access pattern creates an autoregressive bottleneck that shapes serving cost (Pope et al. 2023).

↪ Continuous batching and KV-cache paging, the runtime mechanisms that serve token-by-token generation, are developed in section 13.9.4.

The recommendation lighthouse, **DLRM**[7], represents the embedding-heavy recommendation workload behind large-scale "You might also like" systems (Naumov et al. 2019). It maps users and items to embedding vectors stored in tables that can exceed 100 GB of embeddings, making memory capacity rather than computation the binding constraint.

The mobile lighthouse, **MobileNetV2**, performs the same image classification task as ResNet-50 while cutting its computation by 13.7× (Sandler et al. 2018). It gets there by restructuring the image-filtering layers, splitting each expensive filtering step into two much cheaper ones (depthwise separable convolutions). That reduction makes it a candidate for real-time inference within a smartphone's 3–5 W envelope; the implementation and workload determine whether it meets the deadline.

The TinyML lighthouse, **keyword spotting (KWS)**, represents the always-on sensing archetype. KWS systems detect short trigger phrases with compact models built for resource-constrained microcontrollers (Y. Zhang et al. 2017). The lighthouse workload has approximately 200K parameters, about 200 KB at INT8 (stored in flash), placing it in the kilobyte-memory, milliwatt-budget TinyML regime.

A smart doorbell, a TinyML scenario that recurs through later chapters alongside the five lighthouses, applies the same always-on pattern to vision. A visual wake-word model (Chowdhery et al. 2019), a small classifier that answers only whether a person is in view, runs on an ESP32-S3 microcontroller, the TinyML platform of table 2.4. Its feasibility is screened in two levels, storage first and latency second. At level 1, the 0.5 MB of 16-bit weights occupy 11.9 percent of the 4.2 MB flash. Passing this storage check does not establish complete memory fit, because intermediate results and runtime state must also fit in SRAM, the two-tier split that section 2.8.1 develops. Level 2 applies the iron law. The model needs 25 MFLOP per frame, and the microcontroller sustains about 0.25 GFLOP/s at this precision, so compute takes 100 ms; about 1 ms of fixed overhead brings the total to 101 ms. That estimate clears the scenario's canonical 200 ms budget; sensor acquisition and any data movement omitted by the estimate must still be checked. A more responsive chime with a 50

[7] **Deep learning recommendation model (DLRM)**: Meta's 2019 architecture that exemplifies the capacity-bound archetype. At that scale, memory capacity and communication BW can constrain execution more than raw R_{peak}. These demands motivate specialized cluster designs where memory, not arithmetic, is the scarce resource.

↪ How depthwise separable convolutions factor one filtering step into two cheaper ones is developed in section 6.3.5.

ms budget changes the verdict without changing the model, and figure 2.3 plots that tightened case. In the plot, flash stays far below the saturation line while latency reaches 202 percent of its budget. Flash storage was not the binding constraint; latency is, and on-chip SRAM still has to be checked separately.

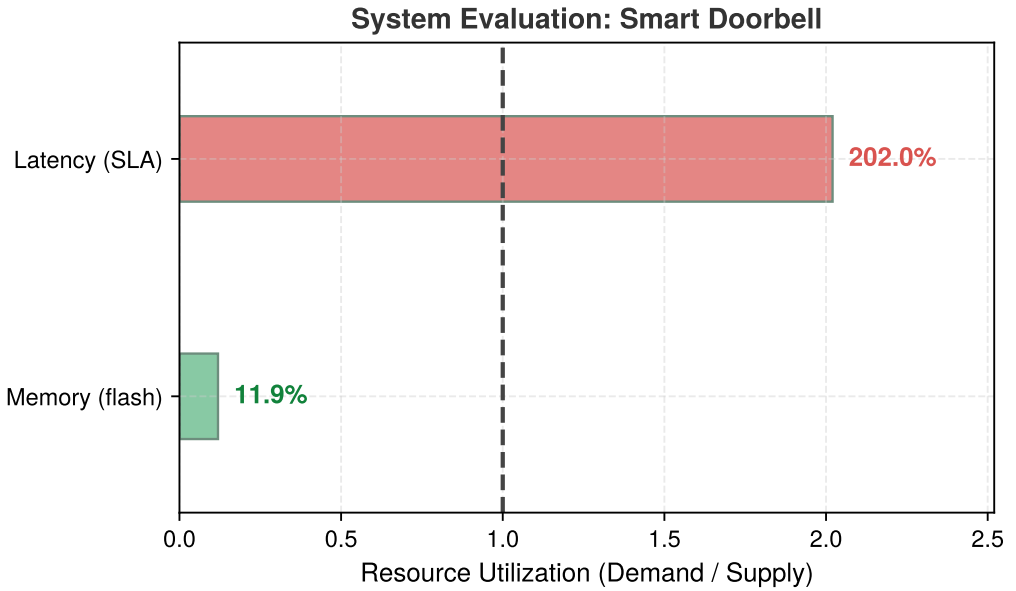


Figure 2.3: Smart Doorbell Constraint Scorecard: Each bar is demand over supply, so the dashed line marks the point where a level is exactly saturated. The model occupies a small fraction of the ESP32-S3's 4 MB flash budget (Level 1: PASS), but its 101 ms latency (25 MFLOP of compute plus fixed overhead) is twice a tightened 50 ms budget for a responsive chime (Level 2: FAIL), even though it clears the scenario's canonical 200 ms budget.

The range in compute requirements and memory footprints explains why no single deployment paradigm fits all workloads. A keyword spotter can operate with roughly 20 MFLOP per inference and about 200 KB of INT8 weights, while ResNet-50 requires about 8.2 GFLOP per image and roughly 102.4 MB of FP32 weights. The reference DLRM example already reaches 100 GB, and Meta has demonstrated scaling recommendation models to 100 TB using embedding offloading (Park et al. 2024). Language models add a bandwidth-dominated regime, with billions of parameters streamed repeatedly from memory during autoregressive inference. Whether each of these demands is affordable depends on the compute, bandwidth, and memory a given platform supplies.

2.4 System Balance and Hardware

A model that clears its latency budget on paper can still stall on a real host if PCIe transfers between CPU and GPU memory choke the pipeline or if single-sample inference leaves the accelerator's matrix units starved for bytes. Checking the iron law and workload archetypes against real hardware means tracing a request through every layer of one host, from the application's latency or throughput target down to the silicon that must meet it.

2.4.1 Inside one machine: The single-node stack

When an application issues an inference request or a training step on a single machine, that work travels down a four-layer **single-node stack**, which refines the machine side of the D·A·M taxonomy for one host while data and algorithm enter through workload requirements. At the top of the stack, the application defines the mission through a training-throughput or inference-latency target, and the ML framework below it translates model code into executable operations that aim to meet that target. Those operations then pass through the operating system, drivers, and runtime, which coordinate resources and move tensors between host memory and accelerator memory, down to

the hardware itself—where HBM capacity, memory bandwidth, intra-node interconnects such as NVLink, and compute throughput set the physical limits, with the memory wall as a recurring constraint.

The hardware layer fixes the terms on offer to any model: a memory bandwidth (BW), a peak compute rate (R_{peak}), and a fixed overhead (L_{lat}). These are the terms of the silicon contract. The hardware offers them, and the workload and its execution conditions determine which resource limits progress first. Deployment analysis works with the hardware's half, asking which term a given platform makes scarce; the model architecture's half, the choice of which resource to saturate, belongs to chapter 6.

↪ How an ML framework turns model code into executed operations, eagerly or through a captured graph, is developed in section 7.3.

↩ The iron law of ML systems and its decomposition of end-to-end latency are established in section 1.7.

2.4.2 Matching workloads to hardware

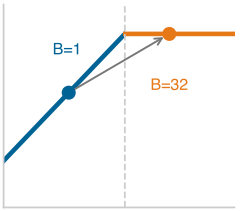
Before profiling cache reuse or arithmetic intensity, an engineer can rule out impossible hardware placements with a two-step test that proceeds from coarse to fine. The coarse test asks whether an operation can fit inside the latency budget at all. The fine test asks which hardware resource limits the operations that do fit, and its answer depends on system balance, the ratio of peak compute throughput to memory bandwidth and I/O capacity on a given platform.

Operation latencies supply the coarse test, and they turn latency-vs-throughput trade-offs into decisions through concrete numbers. They span roughly eight orders of magnitude, from about a nanosecond for an on-chip cache hit or a single accelerator multiply, through milliseconds for one mobile vision inference, to tens or hundreds of milliseconds for a cross-region network call. Section D.1 collects the memory-hierarchy and network round-trip anchors along with illustrative application latency budgets. The span matters because of one strict rule: an operation with latency $> T_{\text{deadline}}$ *cannot* appear on the critical path of a system whose latency budget is T_{deadline} ms.[8]

For the operations that pass the latency test, system balance decides which resource limits them. Roofline analysis classifies bottlenecks by comparing a workload's arithmetic intensity, the operations it performs per byte moved (FLOP/byte), against the hardware machine balance point ($R_{\text{peak}}/\text{BW}$), the ridge point (Williams et al. 2009) introduced with the iron law in section 1.7. The framing is informal here; section 11.6 develops the model and the ridge point that separates the memory-bound and compute-bound regimes, with a worked derivation collected for reference in section D.2.1. In that framing, the same ResNet-50 model can shift from compute-bound training behavior at high batch sizes to more memory-sensitive single-image inference at batch=1. Deployment paradigm selection must account for this shift.

The batch-size shift is one instance of a broader change between execution phases, in which physical constraints reconfigure all three D·A·M axes at once. Training updates model weights from batches of examples; serving applies fixed weights to new inputs. Table 2.3 contrasts typical training and latency-sensitive serving demands; cloud serving can also batch requests for throughput.

8 | **Critical path**: The longest sequential chain of dependent operations in a pipeline. The decision rule in the triggering sentence is strict: if a 200 ms cross-region network call appears anywhere on the critical path, a system with a 100 ms total budget is guaranteed to fail regardless of how fast every other stage runs. Model inference need not be the longest stage; preprocessing and postprocessing can dominate, making the critical path longer than model execution alone suggests.



In this example, batching moves the workload from the memory-bound side toward the compute ceiling.

Table 2.3: D·A·M × Phase: The same model imposes starkly different demands on Data, Algorithm, and Machine depending on whether the system is training or serving. When bottlenecks shift unexpectedly, check which phase is currently being optimized.

Component	Training (Mutable)	Inference (Immutable)
Data	Massive throughput: large batches, shuffling, augmentation	Low latency: single samples, freshness, speed
Algorithm	Learning phase: update model parameters from examples	Prediction phase: apply fixed weights to new inputs
Machine	Throughput-optimized: high-bandwidth clusters, large memory	Latency-optimized: edge devices, inference accelerators

ResNet-50 inference on a high-end data center accelerator and on a mobile neural processing unit (NPU)[9] puts numbers on this analysis. The hardware figures are point-in-time anchors; the reasoning rests on compute rate, memory bandwidth, and batch size, which outlast any one product.

9 | **Neural processing unit (NPU)**: A dedicated hardware block on a mobile system-on-chip whose circuits are designed for low-precision tensor operations. This specialization can improve energy efficiency relative to CPU execution, helping AI workloads fit within mobile power budgets.

Napkin Math 2.1: ResNet-50 on cloud vs. mobile

Problem: Is ResNet-50 inference compute-bound or bandwidth-bound on (a) a high-end data center accelerator (an NVIDIA A100) and (b) a flagship mobile NPU?
Given (from lighthouse models):

- ResNet-50: 8.2 GFLOP per inference, 25.6 million parameters (102.4 MB FP32, 51.2 MB FP16)

Analysis:
(a) Cloud data-center accelerator (batch=1, FP16)
The analytical model assumes 56.3 MB moved per inference, including weights and activation traffic. This is a modeled transfer volume, not the stored parameter footprint; reuse and runtime choices can change it.

- Peak compute: 312 TFLOP/s (FP16)
- Memory bandwidth: 2.04 TB/s
- Compute time: $T_{\text{compute}} = \frac{8.20 \times 10^9}{3.12 \times 10^{14}} = 0.026$ ms
- Memory time (weights plus activations): $T_{\text{memory}} = \frac{5.63 \times 10^7}{2.04 \times 10^{12}} = 0.028$ ms
- **Result**: Bottleneck = Memory (1.1× slower than compute)
- **Ridge point**: $R_{\text{peak}}/\text{BW}$ = 312 TFLOP/s ÷ 2.04 TB/s ≈ 153 FLOP/byte.
- **Analysis**: Arithmetic intensity = $\frac{8.20 \times 10^9}{5.63 \times 10^7} = 146$ FLOP/byte, just below the ridge point, so the workload is bandwidth-bound; above the ridge it would be compute bound. For single-image inference, the low batch size yields limited reuse, explaining why even high-throughput accelerators can be bandwidth-bound at batch = 1. Both times use peak rates (efficiency $\eta_{\text{hw}} = 1$ and full bandwidth), so they are lower bounds; because this workload sits close to the ridge, the efficiencies a real kernel achieves decide which term dominates in practice.

(b) Mobile: Flagship NPU (batch=1, INT8)
At INT8, the analytical model assumes 28.2 MB moved per inference, including weights and activation traffic. The INT8 weight footprint below is only one component of that transfer volume.

- Peak compute: ~35 TOPS (INT8), representative of modern mobile NPUs
- Memory bandwidth: ~51.2 GB/s (LPDDR5)
- Model size: 25.6 MB (INT8 quantized)
- Compute time: $T_{\text{compute}} = \frac{8.20 \times 10^9 \text{ INT8 ops}}{3.50 \times 10^{13} \text{ INT8 ops/s}} = 0.23$ ms
- Memory time (weights plus activations): $T_{\text{memory}} = \frac{2.82 \times 10^7}{5.12 \times 10^{10}} = 0.55$ ms
- **Result**: Bottleneck = Memory (2.3× slower than compute)

Systems insight: Under these peak-rate assumptions, both platforms are bandwidth-bound for single-image inference. The A100's faster memory bandwidth (2.04 TB/s vs. 51.2 GB/s = 39.8×) gives roughly 19.9× lower modeled memory-transfer time under the stated FP16 and INT8 traffic assumptions; this ratio is not a measured end-to-end inference speedup. Byte-reduction and lower-precision techniques can therefore beat simply buying more peak FLOP/s for deployment.
ResNet-50 becomes compute bound when batching and data reuse raise computation per byte moved above the hardware balance point, $I > R_{\text{peak}}/\text{BW}$, where $I = O/D_{\text{vol}}$. With ideal reuse during batched inference, each weight is transferred across the memory bus once and used across all B samples in the batch. This amortizes the weight transfer over B inputs, increasing the arithmetic intensity until computation time overtakes memory-transfer time. The crossover is architecture- and implementation-dependent because activation traffic, input/output movement, cache reuse, and runtime execution details all change the effective bytes moved per inference.

The four deployment paradigms gain precision when grounded in concrete hardware. Where table 2.1 defined the paradigms by operating regime, table 2.4 places one real platform in each and pairs it with the per-device envelope that marks the tier.[10] The example-platform and power columns describe complete systems, including a cloud pod, while the tier compute and bandwidth columns describe per-device envelopes. The following spans compare those complete examples, not equivalent individual chips. Memory drops from 128 TB (cloud) to 512 KB (TinyML), a 250 million-fold reduction. Power runs from about 3 MW for the cloud pod to the TinyML tier's average-power target of < 1 mW, and hardware cost spans six orders of magnitude, from multimillion-dollar clusters to $10 microcontrollers. Together with facility overheads such as power usage effectiveness (PUE),[11] these spreads decide which paradigm can serve a workload economically, and they explain why model compression becomes essential as the budget tightens.

The hardware spectrum turns the bottleneck diagnosis into a fit test, in which each workload must be matched to the tier whose compute, bandwidth, power, and cost envelope it can satisfy. These platforms are products of decades of hardware evolution, from floating-point coprocessors in the 1980s through graphics processors in the 2000s to today's domain-specific AI accelerators.

[10] **ML hardware cost spectrum**: The million-fold cost range makes deployment paradigm selection an economics decision as well as a physics decision. The same accuracy target may be achievable on a low-cost microcontroller only after substantial model reduction, or on an expensive data-center accelerator with a much larger resource budget, and the two designs carry fundamentally different latency, power, and operational cost profiles.

[11] **Power usage effectiveness (PUE)**: Total facility energy divided by IT equipment energy over the same period (The Green Grid 2007), developed in section 12.7. It captures facility overhead, including cooling and power distribution. Even a PUE of 1.06 leaves substantial noncompute power at data-center scale. TinyML does not require dedicated facility cooling, although sensing, communication, conversion losses, and sleep power still contribute to its complete system-energy budget. These device-level costs sit outside PUE.

↪ The progression from floating-point coprocessors to domain-specific AI accelerators is developed in section 11.2.1.

Table 2.4: Hardware Spectrum (Concrete Platforms): Each paradigm in table 2.1 is represented by one platform, paired with the per-device compute and memory-bandwidth envelope that marks the tier and the constraint that usually binds first there: an iron-law term for cloud and edge, a precondition outside the time equation for mobile (energy) and TinyML (memory capacity). Envelope figures mark a paradigm's typical class rather than a hard ceiling. The DGX Spark sits at the high end of the edge spectrum as a compact workstation example; most edge deployments use far smaller devices. BF16 and FP4 are reduced-precision number formats (16 and 4 bits per value; "sparse" rates count skipped zero-valued operations); HBM2 and LPDDR5x are high-bandwidth and low-power memory types; NVMe is solid-state storage.

Paradigm	Example Platform	Tier Compute	Tier Memory BW	Power	Price	Binding Constraint
Cloud ML	Google TPU v4 Pod: 4,096 TPU v4 chips, 1.1 EFLOP/s BF16, 128 TB HBM2	> 1000 TFLOP/s	> 1000 GB/s	~3 MW per pod; PUE 1.1–1.3	Rental	Compute (training); memory bandwidth (LLM decode)
Edge ML	NVIDIA DGX Spark: GB10 Grace Blackwell (1 PFLOP/s sparse FP4), 128 GB LPDDR5x, 4 TB NVMe	~1 PFLOP/s	> 270 GB/s	~200 W; 100 W class typical	~$3,000–$5,000	Memory bandwidth
Mobile ML	Flagship smartphone: mobile SoC (CPU + GPU + NPU), 8–16 GB RAM, 128 GB-1 TB storage	15–45 TOPS	51.2–77 GB/s	3–5 W	$999+	Energy (battery and heat)
TinyML	ESP32-S3: dual-core @ 240 MHz, 512 KB SRAM, 4.2 MB flash	< 1 TOPS	—	0.05 W–1.2 W active; < 1 mW average target	$10	Memory capacity (working state must fit in SRAM)

Systems Perspective 2.1: System balance across paradigms

The last column of table 2.4 reads each tier through the pipelined bound in equation 2.5. For cloud and edge the binding constraint is a term of that bound, whereas for mobile and TinyML it is a limit the time equation leaves out, and in every tier the dominant constraint sets the optimization strategy. Compute-bound cloud training rewards higher accelerator utilization and larger batches, while bandwidth-bound large language model (LLM) decode (token-by-token generation) and bandwidth-bound edge inference reward moving fewer bytes—through smaller models, lower precision, and more reuse per loaded weight—so a faster accelerator helps them little. Moving to battery-powered devices adds an implicit energy term: mobile systems integrate power over time under a tight thermal cap, which makes lower precision and duty cycling pay off in both battery life and sustained speed, while TinyML faces a strict memory precondition before any time or energy term applies, because weights and code must fit in flash and activations and runtime state must fit in SRAM through compact models and

low-precision arithmetic. Section A.5.3 maps each dominant term to the optimizations that work and the ones that are wasted, turning this diagnosis into an action plan.

Each paradigm is therefore an operating envelope with different resource budgets, and moving from warehouse data centers down to microcontrollers traces a physical gradient that trades raw compute throughput for energy and memory efficiency. The four sections that follow walk that gradient one tier at a time, and each asks the same four questions, which the constraint map in each section draws as four branches: which constraint binds, how the system responds, where the paradigm fails, and which workloads fit. Because that structure is shared, each section concentrates on what is distinctive about its tier and on the one or two calculations that decide whether a workload belongs there.

2.5 Cloud ML: Computational Power

Consider the estimated resources needed to train GPT-3 (Brown et al. 2020): 3,634 PFLOP-days of computation, equivalent to 10,000 V100 GPUs running for approximately 15 days at the reported sustained rate, and about 1.3 GWh of electricity (Patterson et al. 2021). A PFLOP-day is 10^{15} FLOP/s sustained for one day, about 8.6×10^{19} FLOPs, so this total is the same few hundred zettaFLOPs estimated from the model's parameter and token counts in section 1.3.3. At the hardware rates used in that estimate, a smartphone or edge server could not have completed this training run on a practical schedule. It required a data-center-scale cluster with enough aggregate compute, memory, and storage. The bitter lesson shows up here as hardware. General methods that improve with computation keep winning, and the computation they consume concentrates wherever power, cooling, and accelerators can be pooled. Cloud ML follows from this proposition. When latency can be tolerated, it offers computational scale that no other paradigm can match. Training location does not fix serving location, however, because a trained model or a smaller derived version can later run near users and sensors.

Centralizing these resources under a utility model[12] allows providers to assemble specialized accelerator topologies tailored to distinct hardware demands. This infrastructure serves as the natural home for three of the four workload archetypes: compute-bound workloads like ResNet training that demand sustained TFLOP/s across thousands of accelerators, bandwidth-bound workloads like large language model inference that benefit from TB/s HBM bandwidth, and capacity-bound workloads like DLRM-style recommendation engines, whose embedding tables routinely exceed terabytes in aggregate, far beyond any single accelerator. Those tables are sharded across pooled host and accelerator memory, so distributed lookups and training updates place heavy demands on the cluster interconnect. Voice assistants show the split between tiers: they lean on the cloud for language understanding while their always-on wake-word stage runs on the device, a division that section 2.10 quantifies. Even as cloud deployments scale from single-accelerator instances to warehouse-scale clusters, systems engineering at every one of these scales begins on a single node, with memory bus saturation, PCIe and NVLink bandwidth, and accelerator utilization inside one machine.

↩ The bitter lesson, the book's account of why compute-scaled general methods keep winning, is established in section 1.4.

[12] **Cloud as utility computing**: A provisioning model in which a provider owns and operates a specialized hardware portfolio that few organizations could economically maintain themselves and rents it on demand. Renting gives direct access to the architecture each workload archetype requires, such as dense accelerator pods for compute-bound workloads, HBM-equipped nodes for bandwidth-bound workloads, and high-memory systems with fast interconnects for capacity-bound workloads, without purchasing and maintaining the entire installation.

↪ The memory-capacity bounds and sparse gather patterns of high-cardinality embedding tables are developed in section 6.8.

Definition 2.1: Cloud ML

Cloud machine learning is the deployment paradigm that trades latency for elastic compute by locating ML workloads in centralized data centers, decoupling computational capacity from the physical location of data sources and users.

1. **Significance**: Cloud deployment dominates the R_{peak} term: a cloud region can pool thousands of accelerators, with access governed by capacity, quotas, and cost. The trade-off is the L_{lat} term: a minimum round-trip latency of 10–100 ms (set by the speed of light over continental distances) makes those distant cloud paths infeasible for workloads requiring sub-10 ms response.
2. **Distinction**: Unlike edge ML, which prioritizes local response and data locality at fixed R_{peak}, cloud ML prioritizes elastic R_{peak} at the cost of variable L_{lat}.

3. **Common pitfall**: A frequent misconception is that cloud ML is "unlimited compute." In reality, the distance penalty (L_{lat}) and the ingestion bottleneck (D_{vol}/BW) are physics constraints that no software optimization can eliminate, setting a hard floor on response time for any workload whose data originates outside the data center.

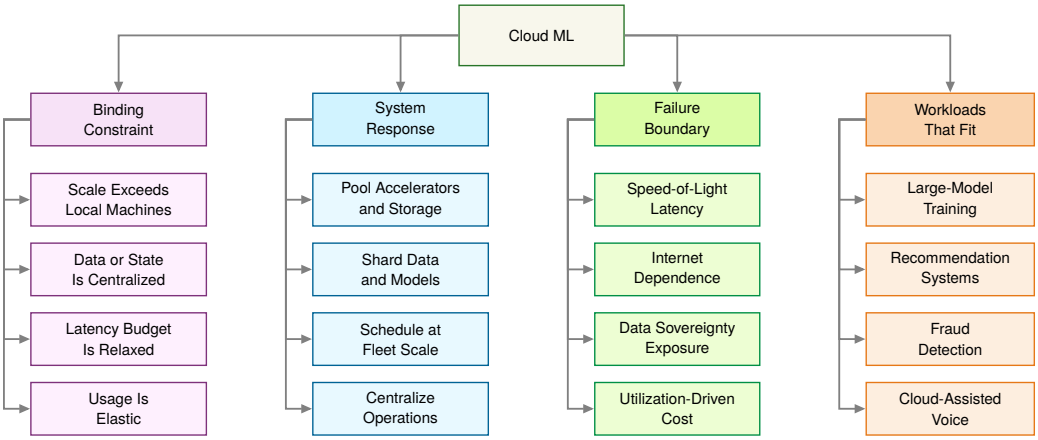


Figure 2.4: Cloud ML Constraint Map: Centralized infrastructure solves the scale problem for compute-bound workloads, bandwidth-bound workloads, and capacity-bound workloads, but it introduces the distance, dependency, privacy, and cost constraints that determine when cloud ML stops being the right deployment target.

Pooling elastic accelerators and sharding models overcome local capacity ceilings, but they meet hard failure boundaries in speed-of-light propagation latency and internet dependence (figure 2.4). Workloads that thrive in this regime (such as asynchronous fraud detection, batch recommendation updates, and large language model pretraining) accept this trade because scale matters more than instantaneous response. When an application requires real-time control, however, network latency stops being an engineering inconvenience and becomes a hard physical barrier. The light-barrier calculation already ruled out a cross-country data center for a 10 ms budget; the sharper question is whether a regional data center, much closer to the machine it protects, fares any better.

Napkin Math 2.2: The distance penalty

Problem: Consider a real-time safety monitor for a robotic arm on a factory floor. The safety logic requires a 10 ms end-to-end response time to prevent injury, so the distance penalty imposed by the light barrier matters. The model runs in a regional cloud data center 1,500 km away, less than half the cross-country distance. Can the safety budget be met?

Physics:

1. **Light in fiber**: ~200,000 km/s.
2. **Round-trip propagation**: (1,500 km $\times$ 2)/200,000 km/s = 15 ms.
3. **Result**: Round-trip propagation alone requires 15 ms, exceeding the 10 ms end-to-end budget by 5 ms before the model performs any inference.

Systems insight: The specified regional cloud path misses the budget on propagation alone and is therefore infeasible. Running the model beside the arm removes that remote propagation term, but the complete local sensing, inference, and control path must still be checked against the deadline.

2.5.1 Cloud infrastructure and scale

Training a frontier model across hundreds of terabytes of data requires connecting thousands of accelerator chips so tightly that they behave like one computer. Cloud ML concentrates computational

resources at physical dimensions no local device approaches: TPU supercomputer designs organize thousands of specialized accelerator chips into data-center-scale systems that deliver PFLOP/s-to-EFLOP/s reduced-precision throughput (Jouppi et al. 2023), orders of magnitude beyond the edge and mobile rows of table 2.4. Warehouse-scale facilities pack aisles of liquid-cooled racks and dense optical cabling that draw megawatts to sustain that exaFLOP-scale matrix throughput (figure 2.5), providing the physical scale that makes training on hundreds of terabytes of data practical.

↪ What changes when training extends beyond a single node is developed in section 8.7.2.

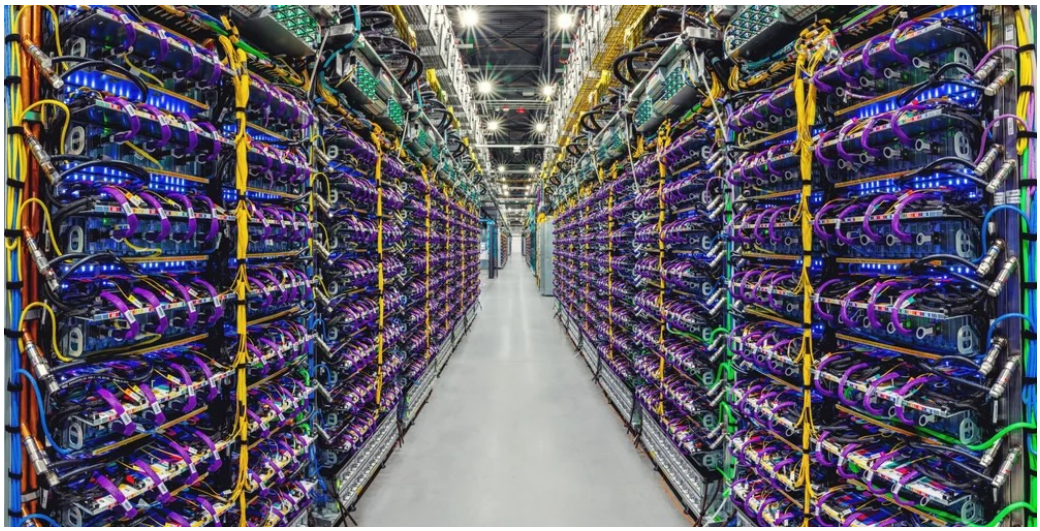


Figure 2.5: Cloud Data Center Scale: Dedicated accelerator facilities aggregate thousands of specialized TPU chips in liquid-cooled racks to deliver exaFLOP-scale matrix throughput. This centralized infrastructure makes large-scale model training and high-throughput serving viable, but incurs megawatt-scale power consumption and speed-of-light network distance penalties. Source: (Pichai 2023).

Centralization also changes how models are served and governed. Cloud APIs make trained models accessible worldwide across mobile, web, and IoT platforms. Shared infrastructure enables engineering teams to collaborate with unified version control, while pay-as-you-go pricing models convert upfront capital expenditure (CapEx) into usage-based operational expenditure (OpEx) that scales with demand.

Scale does not make the cloud universally superior. The data-gravity principle (2) explains why. Shipping a dataset of size D_{vol} over a network link of bandwidth $\text{BW}_{\text{network}}$ takes $D_{\text{vol}}/\text{BW}_{\text{network}}$, a time that grows with the data, while shipping the computation (a model and its code) costs roughly the same however much data waits at the other end. As data volume grows, moving the data to a remote cluster therefore eventually costs more than moving the computation to the data. Section B.1.1 works through the transfer arithmetic, and the edge section applies the same comparison to a single request under a deadline. The deployment paradigm is dictated by the binding physical constraint, not by the largest available throughput.

2.5.2 Cloud ML trade-offs and constraints

A regional cloud outage can take down every connected client at once, and a sudden traffic spike can turn an elastic GPU pool into a runaway monthly bill. For latency-sensitive workloads, the distance penalty is the cloud's most consequential cost, and network jitter makes it unpredictable as well as large. Three further costs follow from centralization: the exposure of data the cloud must receive, spending that scales with use, and dependence on the network.

The first cost is privacy and security. Transmitting sensitive data to remote data centers expands the disclosure surface and complicates regulatory compliance. Organizations handling data subject to regulations such as the General Data Protection Regulation (GDPR) or the Health Insurance Portability and Accountability Act (HIPAA) must select, document, and maintain safeguards appropriate to the data, system, and applicable legal duties. Privacy-preserving approaches can reduce

how much sensitive data leaves its original environment, but they complement rather than replace infrastructure controls.

The second cost is financial, because cloud infrastructure turns upfront hardware purchases into ongoing operating bills. Comparing paradigms by raw accelerator hourly rates obscures the cumulative expenses of network egress, persistent storage, and managed orchestration. Sound evaluation instead compares **total cost of ownership (TCO)**, which includes power, cooling, and labor and weighs upfront capital expense against recurring operating expense over the system's lifetime.

A vision system serves 1M daily inferences at ResNet-50 scale (10 ms latency, 100 KB response). The TCO analysis begins with an illustrative budget spanning 3 years. It includes the listed components rather than a complete, matched TCO. Cloud operating labor, edge redundancy, installation, and failure recovery are not included, and comparable capacity and availability have not been established. The 4 instances are a provisioning choice (redundancy and headroom for traffic peaks), not a throughput requirement. Table 2.5 itemizes the annual GPU, network, load-balancer, and observability costs of this fixed-capacity cloud implementation under public list pricing, and table 2.6 itemizes the corresponding hardware, power, cooling, network, and DevOps labor costs of an on-premises edge-server implementation.

Table 2.5: Cloud Inference Annual Budget: The fixed-capacity cloud implementation of the ResNet-50-scale vision workload incurs itemized GPU, network, load-balancer, and observability costs.

Cost Component	Calculation	Annual Cost
GPU inference	4 instances × 8760 h/year × $0.75/hr	~$26,280
Network egress	100 GB/day × 365 d/year × $0.09/GB	~$3,285
Load balancer	$0.025/hr + capacity units	~$3,723
Monitoring/logging	Monitoring, alerts	~$2,000
Total Cloud		**~$35,288/year**

Table 2.6: Edge Inference Annual Budget: The on-premises edge-server implementation incurs itemized hardware, power, cooling, network, and DevOps labor costs, and labor is the dominant component in this scenario.

Cost Component	Calculation	Annual Cost
Hardware CapEx	$15,000 ÷ 3 years life	~$5,000
Power (24/7)	300 W × 8760 h/year × $0.12/kWh	~$315
Cooling overhead	~30% of power	~$95
Network (fiber)	Fixed line for remote management	~$1,200
DevOps labor	0.1 FTE × $150,000 salary	~$15,000
Total Edge		**~$21,610/year**

Napkin Math 2.3: Cloud vs. edge annual budgets

Problem: Under the partial accounting boundaries itemized in table 2.5 and table 2.6, which implementation has the lower estimated annual budget over 3 years?

Math: The question is the annual cost at this operating point, so the comparison is the gap between the cloud and edge annual totals.

At 1M inferences/day, the gap is ~$35,288/year − ~$21,610/year = ~$13,678/year. Almost all of the cloud cost is provisioned capacity rather than use. At 10 ms per request the day's traffic needs only about 2.8 GPU-hours of the 96 provisioned, about 2.9 percent utilization. Only egress scales with traffic.

Result: The listed edge budget costs about 38.8 percent less at this operating point. Because the fixed cloud pool alone already costs more than the listed edge budget, no traffic level under this fixed provisioning and partial accounting makes the cloud cheaper; a genuine break-even analysis would need cloud capacity that scales with demand.

Systems insight: The listed edge budget is dominated by labor (~$15,000 ÷ ~$21,610/year ≈ 69.4 percent), not hardware. Organizations without existing DevOps capacity should factor in the full cost of maintaining on-premises infrastructure.

Unpredictable usage spikes also complicate budgeting and call for automated monitoring and cost governance. Network dependency, the last of the three costs, creates an availability hazard: any connectivity disruption between clients and the data center immediately degrades or halts service. Proprietary cloud APIs compound the dependency with vendor lock-in, making it harder to move workloads from one provider to another. Even so, centralized infrastructure remains uniquely suited to consumer-scale services that require elastic capacity, such as voice assistants and recommendation engines. These applications trade latency for scale, accepting remote-path delay in exchange for computational resources that cannot be supplied locally. The trade fails when a latency budget drops below what the remote path permits or when data volumes exceed what networks can carry, and computation must then move closer to the data source.

2.6 Edge ML: Latency and Privacy

The robotic-arm safety monitor in napkin math 2.2, an industrial IoT (IIoT) control loop,[13] left one option open, running the model beside the arm it protects. Moving computation adjacent to the data source removes that remote transmission term from the request path, and edge ML is the paradigm built on that move.

Edge ML trades elastic cloud capacity for shorter request paths and local data retention. Proximity does not by itself guarantee a sub-100 ms response, so the complete path must still be verified. Offloading a workload over a wide-area link can make remote-path latency, rather than local execution, the binding term. Under the iron law from equation 2.5, local execution removes the I/O offload term from the request path, leaving execution time bounded by local compute and memory transfer.

Definition 2.2: Edge ML

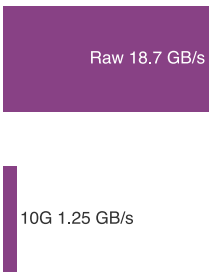
Edge machine learning is the deployment paradigm optimized for local response and data locality by locating computation physically adjacent to data sources.

1. **Significance**: It circumvents the distance penalty (L_{lat}) of the cloud, trading elastic scale for a fixed local compute capacity (R_{peak}).
2. **Distinction**: Unlike cloud ML, which prioritizes throughput, edge ML prioritizes response deadlines and local data control. Unlike TinyML, edge ML may still use workstation-class accelerators such as general-purpose GPUs (GPGPUs).
3. **Common pitfall**: A frequent misconception is that edge ML refers to a specific hardware class. In reality, it is a location paradigm: it spans from IoT gateways to on-premises servers, unified by physical proximity to the data source.

Real-time deadlines are not the only driver. Data sovereignty and privacy rules can restrict transmitting raw sensor streams across external networks, which places computation on local gateways, industrial PCs, and on-premises servers. Placing compute near sensors and filtering data locally helps meet sub-100 ms response budgets, while introducing failure boundaries in fixed node capacity and remote fleet management (figure 2.6).

IoT sensor ingestion rates[14] above network uplink capacity rule out transmitting the full raw stream. Local filtering or compression can reduce the transmitted representation; the complete local and remote paths then determine where inference can meet the application's requirements.

13 **Industrial IoT (IIoT)**: A domain where latency constraints can be set by physical safety rather than user perception. Keeping sensing and the stop decision local removes the long-haul network term but leaves the edge system bounded by its local compute capacity (R_{peak}).



Raw edge data can be wider than the network pipe.

14 **Internet of things (IoT) data wall**: IoT deployments depend on continuous sensor streams from devices distributed across homes, factories, farms, vehicles, and infrastructure (McKinsey Global Institute 2021). In many of those settings, the aggregate D_{vol} from raw streams overwhelms the available uplink budget or latency budget for centralized ingestion, making local edge processing an architectural requirement rather than merely a cost optimization.

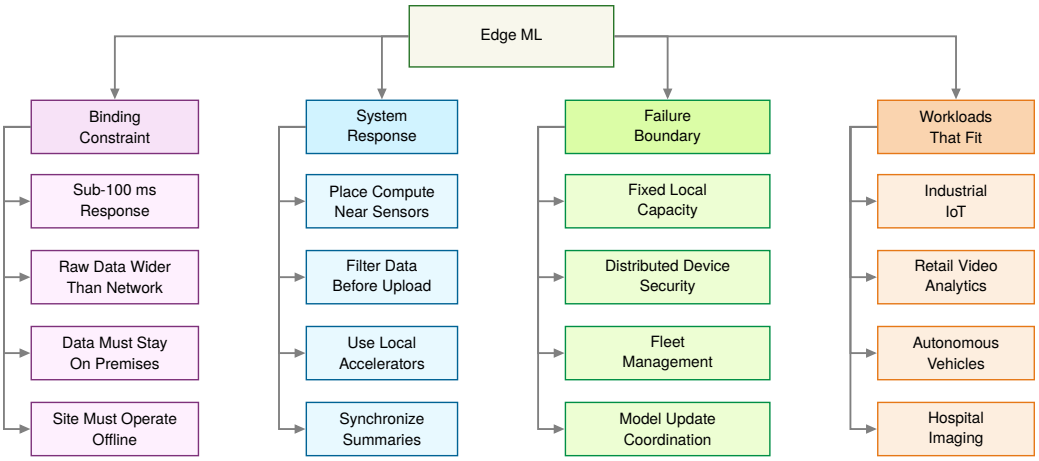


Figure 2.6: Edge ML Constraint Map: Moving computation near the data removes the network-latency term and reduces bandwidth pressure, but it replaces centralized scale with fixed local capacity, distributed security exposure, and operational complexity across many sites.

Napkin Math 2.4: The bandwidth bottleneck

Problem: Consider a quality control system for a factory floor with 100 cameras running at 30 FPS with 1080p resolution. Does video streaming become the bandwidth bottleneck, and does edge ML reduce the bandwidth enough to process locally?

Physics:

1. **Raw data rate per camera**: $1920 \times 1080 \times 3$ bytes $\times$ 30 FPS $\approx$ 186.6 MB/s.
2. **Total data rate**: 100 cameras $\times$ 186.6 MB/s = 18.7 GB/s.
3. **Cloud transfer exposure**: Uploading raw camera feeds is primarily a bandwidth, ingest, storage, and processing problem; per-GB cloud egress charges apply when data is transferred back out of the cloud. If the raw stream were later retrieved at \$0.09/GB data-transfer-out pricing, the transfer charge alone would reach \$4.4M/month.
4. **Network reality**: Even a dedicated 10 Gbps line (1.25 GB/s) cannot carry the load, because the workload demands 14.9× more bandwidth than exists.

Systems insight: Streaming these raw feeds over the stated link is *impossible*. Local processing or a smaller transmitted representation is required; compression changes the bandwidth calculation but must preserve the information and timing needed for defect detection. If an edge server transmits only defect metadata (1 KB per detection at roughly 20 events/s across the floor), bandwidth falls by about 933,120×.

Uncompressed sensor streams can saturate an uplink, which is the bandwidth barrier to centralized ingestion. For battery-powered edge devices (wireless cameras, drones, wearables), the energy cost of wireless communication introduces an even stricter physical ceiling. Offloading adds energy to the time-based analysis of equation 2.5, so the local-versus-offload decision must weigh joules as well as milliseconds.

Napkin Math 2.5: The energy of transmission

Problem: Should a battery-powered sensor process data locally (on-device NPU inference) or send it to the cloud when the energy of transmission must come out of a fixed battery budget?

Given:

- **Data** (D_{vol}): 1 MB (illustrative payload volume).
- **Transmission energy** (E_{tx}): 100 mJ/MB (Wi-Fi/LTE).
- **Compute energy** (E_{local}): 0.1 mJ/inference (illustrative MobileNetV2 anchor for the neural processing unit alone, excluding the rest of the phone).

Math:

1. **Cloud approach**: $E_{\text{transmit}} \approx D_{\text{vol}} \times E_{\text{tx}} = 1 \text{ MB} \times 100 \text{ mJ/MB} = 100 \text{ mJ}$.
2. **Local approach**: $E_{\text{local}} \approx \text{Inference} = 0.1 \text{ mJ}$.

Systems insight: Transmitting raw data is 1,000× more expensive than processing it locally. Charging local inference with the whole phone's energy over the inference window (2.6 mJ, the MobileNetV2 bar charted in figure 2.7) still leaves transmission 39× more expensive. Even if the cloud had infinite speed ($T \approx 0$), the stated battery budget favors local inference over raw-data transmission. The machine constraint (battery) shapes the deployment choice.

Figure 2.7 steps back from that matched comparison and supplies energy scale anchors for different tasks and accounting boundaries on a logarithmic scale spanning eight orders of magnitude, isolating the 100,000,000× gap between an always-on TinyML keyword spotting inference (about 10 µJ) and a cloud foundation-model query (about 1 kJ, an order-of-magnitude scenario anchor that includes data-center overhead rather than a measured value).

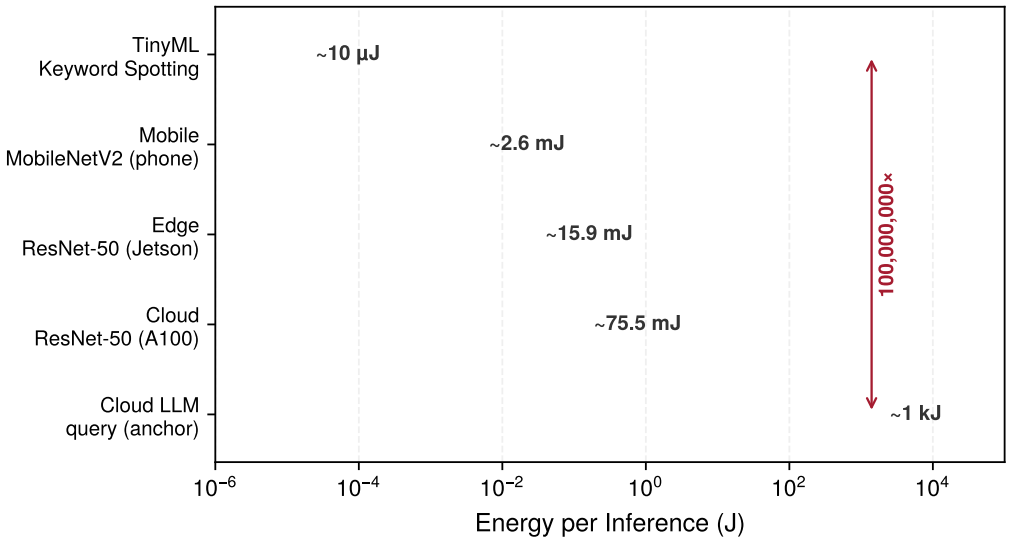


Figure 2.7: Energy per Inference Across Deployment Tiers: An eight-order-of-magnitude energy gap separates always-on TinyML keyword spotting (~10 µJ) from a cloud foundation-model query (~1 kJ) on a log scale. The three vision bars are device-level estimates (the device's power draw at its modeled utilization over the batch-1 inference window, counting the whole phone, the edge tier's NVIDIA Jetson embedded GPU module, or the A100 card) from the same analytical model used throughout this chapter; the two endpoints are scenario anchors. The tasks and accounting boundaries differ, so these bars illustrate energy scales rather than a controlled hardware-efficiency or offload comparison.

Systems Perspective 2.2: Reading the energy-per-inference numbers

The logarithmic comparisons plotted in figure 2.7 combine solver-derived device-energy estimates with separate scale anchors; they are not controlled full-system measurements. Each vision bar counts one device over its batch-1 inference window. The cloud ResNet-50 bar, for example, charges an A100 averaging about ~130 W for 0.57 ms; it excludes the host CPUs,

memory, networking, and cooling that bring an entire server to about ~1 kW, so a whole-server accounting of the same request would be several times larger.

The span becomes tangible when measured in a familiar unit of energy, one smartphone battery. The nominal energy stored in a 3.7 V, 3000 mAh battery is equivalent to about 4 billion inferences by the keyword spotter but only about 40 queries to a cloud LLM. The comparison is one of scale only: a cloud query's energy is spent in the data center, not drawn from the phone. Nor are the keyword-spotting counts elapsed battery life: actual runtime still depends on inference cadence, sleep power, sensing, communication, and the rest of the device.

2.6.1 The data locality invariant: When the network is too slow

When a delivery drone captures 4K video at 60 frames per second to dodge obstacles, shipping even one uncompressed frame over a cellular link takes longer than the 16.7 ms interval before the next frame arrives. That per-request bandwidth-latency trade-off governs the choice between local edge processing and remote cloud offload. The data locality invariant below is the per-request, deadline-bound form of the data-gravity principle from the cloud section: where data gravity weighs moving a whole dataset against moving the computation, the invariant weighs moving one input against one deadline. A remote path is rejected when its complete time exceeds the service deadline, and among the paths that remain, the faster one wins.

Definition 2.3: The data locality invariant

The data locality invariant first screens a remote path against the service deadline. The expression below gives a lower bound on complete remote-path time T_{remote}; let T_{deadline} denote the deadline, $\text{BW}_{\text{network}}$ offload bandwidth, $L_{\text{lat,network}}$ network latency, and $R_{\text{peak,remote}}$ and $\eta_{\text{hw,remote}}$ the remote compute rate and efficiency:

$$T_{\text{remote}} \geq \frac{D_{\text{vol}}}{\text{BW}_{\text{network}}} + L_{\text{lat,network}} + \frac{O}{R_{\text{peak,remote}} \cdot \eta_{\text{hw,remote}}}$$

The bound includes input transmission, network round-trip time, and arithmetic execution, but omits remote memory movement, queueing, preprocessing, runtime overhead, and response serialization. If even this lower bound exceeds the deadline, reject the remote path. A bound below the deadline leaves a candidate for complete-path measurement; it does not establish feasibility. Compare complete local and remote times only after each path meets the deadline and its resource constraints. Here, $L_{\text{lat,network}}$ covers propagation, routing, and protocol handshakes; response serialization may be small for a class label but must be included when material.

1. **Significance**: The invariant defines a crossover beyond which adding remote compute ($R_{\text{peak,remote}}$) has little effect because transmission dominates the lower bound. The local path is preferred only after its own complete execution time and resource limits are checked.
2. **Distinction**: Unlike the iron law, which decomposes one execution path, the locality invariant uses a deadline to reject an idealized remote path, then requires complete-path evidence to choose between the remaining alternatives.
3. **Common pitfall**: A frequent misconception is that 5G/6G "solves" locality. While these technologies improve $\text{BW}_{\text{network}}$, they do not remove propagation and protocol latency, so a remote path can still miss a tight service deadline.

The locality formulation also explains what every local tier gains, edge, mobile, and TinyML alike. Local execution removes network transit time ($D_{\text{vol}}/\text{BW}_{\text{network}} + L_{\text{lat,network}}$) from the request path, eliminating wide-area network variability. A closed-loop latency guarantee requires validating a bound on the complete local path under its operating conditions, $T_{\text{local}} \leq T_{\text{deadline}}$. Uplink traffic drops from raw sensor bitrates to lightweight event telemetry, relieving congestion and recurring

transit charges. Keeping raw sensor data where it is captured shrinks the transmission attack surface, although local processing alone does not guarantee privacy: data at rest still needs access control, and retention and secondary use still need governance. Local inference also continues through wide-area outages whenever local power and sensor links remain functional. Mobile and TinyML inherit all four gains; what distinguishes them is what each gives up to obtain them. A single high-rate sensor frame under a strict frame deadline is enough to force that crossover.

Napkin Math 2.6: The locality crossover

Problem: Should a drone's object avoidance system (4K, 60 FPS) offload to the cloud, or does this become a locality crossover?

Given:

- **Data** (D_{vol}): 4K frame ≈ 24.9 MB.
- **Bandwidth** ($\text{BW}_{\text{network}}$): 100 Mb/s cellular uplink.
- **Remote network and compute response**: 110 ms (round-trip + remote compute).
- **Per-frame completion deadline**: 16.7 ms, equal to the frame interval in this illustrative scenario; each avoidance result must be ready before the next frame arrives.

Math:

1. **Transmission time**: 24.9 MB × 8 bits/100 Mb/s = 1,990.7 ms.
2. **Remote-path lower bound**: Transmission + 110 ms remote response = 2100.7 ms, vs. the stated 16.7 ms per-frame deadline.

Systems insight: The remote path misses the stated per-frame deadline, and faster remote compute cannot remove the transmission time. Cloud offload is therefore infeasible under these assumptions; the local path must still be measured to confirm that it meets the deadline.

2.6.2 Edge ML deployment challenges

Keeping inference on the factory floor or inside a retail store eliminates the wide-area network delay, but now the model must fit inside whatever compute box is bolted to the wall—and if a firmware update fails across a thousand stores, no data-center operations team can walk down the aisle to swap the board. Edge deployment trades network transit for fixed local capacity. Edge hardware spans a wide power envelope, from 5–50 W embedded modules to site-powered servers drawing hundreds of watts, and its binding limit is the fixed capacity of a site rather than the elastic capacity of a cloud region. Many smaller embedded modules operate within 1–8 GB of memory,[15] and their power and board budgets cannot support the wide, high-power interfaces of HBM. They rely on LPDDR or shared system DRAM instead, so memory capacity limits parameter scale and memory bandwidth limits throughput. Fitting modern machine learning models into these bounds requires efficiency techniques that store weights in fewer bits (quantization), remove weights that contribute little (pruning), or train a smaller model to imitate a larger one (distillation). Model footprint, activation buffers, runtime state, and thermal duty cycles all become binding. Operationally, coordinating software updates and model versions across thousands of heterogeneous, physically exposed devices requires specialized deployment pipelines,[16] where intermittent links and physical access complicate rollback and failure diagnosis.

A realistic retail deployment shows how those constraints turn into throughput, hardware, and fleet-cost requirements. Consider a smart retail chain that deploys person detection across 500 stores, each with 20 cameras/store running at 15 FPS. Table 2.7 cascades the per-store inference rate through the per-frame FLOP count of YOLOv8-nano, a compact object-detection model, to yield the throughput each store must sustain. The 2× headroom keeps the device out of saturation when traffic bursts above the average rate.

15 **Embedded edge constraints**: Across representative edge envelopes, 1–8 GB can hold roughly 1–8 billion INT8 weights before activations, runtime state, and other buffers, so the memory budget, not a fixed parameter count, sets the ceiling. Sustainable compute is further limited by power, cooling, and duty cycle.

16 **Edge fleet coordination**: Managing thousands of distributed edge devices amplifies coordination problems: intermittent connectivity causes model version drift, hardware heterogeneity requires per-target optimization, and limited remote access can make failed firmware rollbacks require a site visit.

↪ Over-the-air updates, dual-bank rollback, and lightweight fleet telemetry for edge devices are developed in section 14.4.4.

Table 2.7: Edge Inference Sizing Requirements: The per-store inference rate and model cost set the throughput target for the smart-retail person-detection scenario.

Metric	Calculation	Result
Inferences per store	20 cameras/store × 15 FPS	300 inferences/s
Model compute	YOLOv8-nano: 8.7 GFLOP/inference	2610 GFLOP/s
Required throughput	2610 GFLOP/s × 2 (headroom)	~5.22 TOPS equivalent

Three candidate edge accelerators, including an embedded GPU accelerator, differ sharply in the power, unit cost, and fleet cost of reaching that throughput target (table 2.8).

Table 2.8: Edge Accelerator Options: Three candidate edge accelerators differ in throughput, power, and cost at fleet scale. Fleet cost assumes one unit per store for Jetson and NUC. Jetson power depends on the module: Jetson Orin Nano draws 7–15 W, Orin NX 10–25 W, and AGX Orin 15–60 W, so Jetson power figures in this book should be read against the module named in context.

Edge Device	INT8 TOPS	Power	Unit Cost	Fleet Cost
NVIDIA Jetson Orin NX	100 TOPS	10–25 W	$600	$300,000
Intel NUC + Movidius	1 TOPS	15 W	$400	$200,000
Google Coral Dev (3 boards/store)	peak 12 TOPS derated 6 TOPS	6 W	$450	$225,000

Napkin Math 2.7: Edge inference sizing

Problem: Given the throughput target in table 2.7 and the candidates in table 2.8, which edge accelerator (USB-scale TPU, workstation-class embedded GPU, or general-purpose mini-PC) delivers the required throughput at the lowest estimated three-year hardware-and-power budget?

Math: The capacity screen uses an assumed sustained rate rather than the datasheet peak. The scenario's 50 percent derating plays the role of the efficiency η_{hw} in the iron law. One Jetson derates from 100 TOPS to 50 TOPS, far above the 5.22 TOPS requirement. One NUC derates to 0.5 TOPS and falls short; meeting the target would take 11 per store, about $2,200,000 of hardware across the fleet. A Coral Dev Board is rated at 4 TOPS peak; the same derating gives 2 TOPS, so the count per store is 5.22 TOPS required ÷ 2 TOPS per board = 2.6 boards, which rounds up to 3 boards/store. This estimate assumes the model's operations and numerical format are supported and that work can be divided across the boards at the assumed rate.

Over 3 years across 500 stores, the hardware-and-power estimate for that configuration is Hardware $225,000 + Power (0.006 kW × 500 × 8760 h/year × 3 years × $0.12/kWh = $9,461) = $234,461 total vs. a baseline of one dedicated cloud GPU per store, at the same $0.75/hr instance rate as the cloud-versus-edge comparison, of ~$9,855,000. Within this accounting boundary, the other two candidates need no power arithmetic to rank, because the Coral configuration's hardware-and-power estimate is already below the Jetson fleet's $300,000 hardware cost and far below a feasible NUC fleet's. Integration, maintenance, redundancy, and matched cloud operating costs remain outside this comparison.

Result: One Coral alone is undersized under these assumptions, but the sharded configuration with 3 boards/store is a low-cost edge candidate within the hardware-and-power boundary, with an estimated 6 TOPS and about 1.3× lower per-store hardware CapEx than Jetson. Jetson remains the simpler single-device candidate when integration complexity matters more than hardware cost.

Systems insight: Edge sizing is a capacity-and-cost problem, not a device-name problem. The cheapest feasible design may be several small accelerators per site rather than one larger board, but that hardware saving must be weighed against the coordination and maintenance burden of a sharded edge fleet. The estimate narrows the candidates; measurements of the actual model and runtime must establish task quality, throughput, and end-to-end latency before deployment.

Edge hardware is also physically exposed. Unlike servers housed in access-controlled data centers, edge hardware deployed across retail floors or public infrastructure is physically accessible to adversaries. This exposure requires hardware-enforced protection mechanisms: secure boot chains to prevent unauthorized firmware modification, encrypted local storage for model weights and intermediate activations, and tamper-evident enclosures. Capital expenditures also scale linearly with deployment footprint, so equipping 1,000 sites with $500–2,000 of edge hardware per site requires $500,000–2,000,000 in upfront CapEx before installation, cabling, and fleet operations. Whether that upfront investment yields a lower total cost of ownership than recurring cloud inference depends on utilization duty cycles, local energy rates, and site maintenance overhead.

2.6.3 Real-time industrial and IoT systems

Edge applications differ by domain, but each rests on a locality argument, because the system cannot wait for the network, cannot ship the data, cannot expose the raw signal, or cannot stop during an outage. Autonomous vehicles represent a demanding application because safety-critical perception and control decisions must fit millisecond-scale budgets. Onboard systems process streams from multiple cameras and other sensors through local hardware; a remote cloud path cannot satisfy the tight control loop when its network delay alone exceeds the deadline.

In smart retail, uplink bandwidth and privacy bind before compute does. A checkout-free store[17] can process feeds from many cameras on local edge servers, tracking customer movements and item selections without continuously transmitting raw video over a wide-area link. This architecture reduces bandwidth demand and transmission exposure, but it does not by itself ensure privacy: retention, access, secondary use, and governance controls still determine the system's compliance and risk.

The industrial IoT, often called Industry 4.0 when ML closes the factory control loop,[18] uses edge ML where millisecond-level responsiveness affects production quality and worker safety. Manufacturing facilities deploy edge ML systems for real-time quality control and predictive maintenance.[19]

Many industrial and retail edge systems are stationary and connected to site power, where a factory edge server drawing hundreds of watts around the clock remains viable. Mobile devices, by contrast, travel with untethered users on strictly bounded battery reserves. When computation moves from stationary infrastructure to an untethered mobile device, battery energy becomes a central constraint alongside latency, and the dominant optimization target shifts from raw execution time to energy per inference.

2.7 Mobile ML: Offline Intelligence

Carrying edge-style locality into a user's pocket changes which budget binds. A phone must deliver responsive, private inference all day from a pocket-sized battery, inside a thin chassis with no fan to carry heat away. Mobile ML integrates machine learning directly into smartphones and tablets to meet that envelope. It fits workloads where data locality, offline operation, and immediate response matter more than computational scale, such as voice recognition, computational photography,[20] health monitoring, and navigation or translation that must keep working without a network, and its battery and thermal envelope suit frequent, short-duration inference rather than sustained heavy computation.

On a phone, energy per inference is a primary design constraint, because a phone operates from a finite battery without active cooling, so every joule consumed directly erodes device runtime and thermal headroom. While stationary cloud and edge deployments often focus on minimizing latency T under continuous utility or site power, mobile systems must strictly balance total energy ($\text{Energy} = \text{Power} \times T$) under the passive thermal envelope of 3–5 W that the power wall (equation 2.2) imposes. In archetype terms, a compute-heavy workload such as image classification must be transformed into a more compact vision model,[21] reducing FLOPs by 13.7× at a modest accuracy cost. The design accepts lower peak throughput in exchange for sustainable operation within a 3–5 W thermal envelope.

17 **Checkout-free retail**: A store design that fuses camera observations with shelf events and transaction state locally, then transmits only selected events or aggregates. Those derived records still fall inside the privacy boundary.

18 **Industry 4.0**: The fourth industrial revolution integrates ML into the sensor-actuator feedback loop on factory floors. In an illustrative welding-robot scenario, defect detection samples at 60 Hz, giving a 16.7 ms sampling interval; if each response must finish before the next sample, the complete sensing-to-actuation path must fit that deadline. This completion requirement is a scenario assumption; the sampling rate alone does not establish the control deadline or guarantee safe operation.

19 **Predictive maintenance**: The use of sensor data such as vibration and temperature to estimate equipment condition and failure risk. Continuous monitoring makes sensing cadence, duty cycle, power, and local storage first-class constraints. Battery-powered monitoring nodes may require sub-watt average operation, whereas mains-powered industrial gateways can use a much larger envelope.

20 **Computational photography**: The use of ML algorithms (for example, multi-frame fusion and neural denoising) to overcome the physical limits of small mobile camera sensors.

21 **Mobile vision model reduction**: MobileNet-style architectures reduce the computation in common vision layers while preserving useful accuracy for mobile tasks. The systems point here is that mobile deployment often requires changing the model family, not merely moving the same model to a phone.

Definition 2.4: Mobile ML

Mobile machine learning is the deployment paradigm bounded by thermal design power (TDP) and battery energy.

1. **Significance**: It is constrained by the few-watt heat dissipation capacity of passive cooling, requiring architectures that prioritize sustained energy efficiency over peak throughput (R_{peak}).
2. **Distinction**: Unlike edge ML, which may have active cooling, mobile ML must operate within a personal energy budget. Unlike TinyML, it still provides a rich OS and multi-watt compute capacity.
3. **Common pitfall**: A frequent misconception is that mobile ML performance is a fixed value. In reality, it is a time-varying constraint. Performance often drops as the device hits its thermal wall, triggering thermal throttling that lowers clock frequencies and reduces the active duty cycle.

Two boundaries define the mobile envelope, and together they push the design toward on-device NPU acceleration and model compression (figure 2.8). The first is the **battery tax**, the share of the phone's finite energy budget that an always-on ML feature spends before the rest of the system runs, because the battery limits total energy over time. Thermodynamics imposes the second boundary, which holds even for a plugged-in tablet: every watt becomes heat, and where a data center removes heat with active cooling, a thin mobile device relies on passive transfer through its chassis and surfaces. This creates the **thermal wall**, the power wall's device-level form, which sets a hard ceiling on sustained power consumption independent of battery capacity.

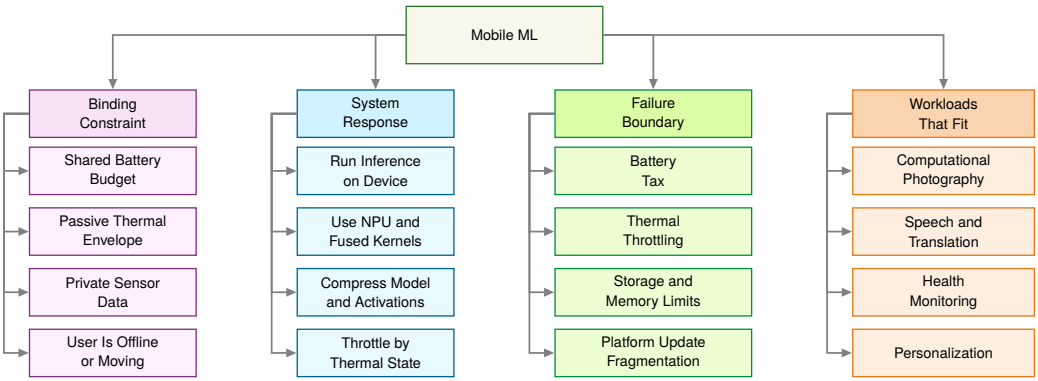


Figure 2.8: Mobile ML Constraint Map: On-device processing buys responsiveness, privacy, offline operation, and personalization, but the phone's shared battery and passive thermal envelope make sustained energy efficiency the binding design constraint.

Napkin Math 2.8: The battery tax

Problem: Consider deploying a "real-time" background object detector on a smartphone. The model consumes 2 W of continuous power when active. The phone has a standard 15 Wh battery. Can the feature stay on all day? This is the battery tax that turns battery life into a mobile ML energy-budget constraint.

Physics:

1. **Ideal runtime**: $\frac{15\ \text{Wh}}{2\ \text{W}} = 7.5$ hours
2. **Reality**: A user expects their phone to last 24 hours. Running this single feature continuously for a day would require 320 percent of the phone's daily energy budget.

Systems insight: The model cannot simply be "deployed." A lighter model, duty cycling, or both must reduce the energy demand, so the feature can stay on all day.

The distinction matters for engineering decisions. The battery tax is a budget problem, solvable in principle by reducing how often the model runs or by increasing battery capacity. The thermal wall is a physics ceiling. A larger battery cannot raise the maximum sustained wattage a passive chassis can dissipate; duty-cycle and software changes can only reduce heat generation. Sustained operation above the thermal envelope triggers hardware throttling regardless of remaining battery energy. The two constraints therefore attack different points in the iron law: the battery limits total operations per charge (O integrated over time), while the thermal wall caps the instantaneous rate ($R_{\text{peak}} \cdot \eta_{\text{hw}}$) the silicon can sustain.



Sustained thermal performance falls well below burst peaks.

Napkin Math 2.9: The thermal wall

Problem: An unoptimized vision model requires 12 W peak compute. Can it be deployed on a mobile device, or does it hit the thermal wall and the mobile power wall?

Scenario: A mobile system-on-chip (SoC) allows approximately 3 W for passive cooling. Assume the unoptimized model is deployed as is. A representative throttling trace for this heavier model runs in three steps:

1. **Temperature rise**: At 12 W, the SoC junction temperature rises at approximately 1 °C/s.
2. **Thermal trip**: Within 60 s, the hardware reaches the thermal trip point (80 °C), triggering OS throttling.
3. **Throttled throughput**: The 100 FPS model suddenly drops to 30 FPS to stay within the thermal envelope.

Math: Assume that a redesign (a lighter architecture running INT8 instead of FP32 arithmetic) cuts active power by an illustrative 4×: 12 W ÷ 4 = 3 W, which lands right at the 3 W passive-cooling ceiling with no headroom to spare. The factor is a scenario input for the whole redesign. Quantization alone shrinks weight storage by that factor but does not cut power proportionally, because switching and leakage energy outside the arithmetic remain.

Systems insight: Even a 4× power reduction does not create thermal headroom; it only just reaches the sustainable limit. Optimization can bring a model down to the thermal ceiling; it cannot raise the ceiling.

Battery and heat are the two limits a mobile deployment must account for explicitly. Within them, the phone's hardware envelope determines which models can run and how often.

2.7.1 Mobile ML benefits and resource constraints

The mobile row of table 2.4 gives the hardware envelope: gigabytes of RAM, neural processing units delivering tens of TOPS, and 3–5 W of sustained power on a system-on-chip[22] that integrates computation and memory to minimize data-movement energy. Memory capacity constrains model footprint and memory bandwidth constrains how quickly weights and activations move, but the battery sets the budget users notice. Adding 1 W of continuous ML processing to a phone that otherwise lasts 24 hours would cut its runtime to roughly 9.2 hours–11.5 hours, depending on a battery capacity of 15 Wh–22 Wh.

Mobile adds two gains to those every local tier shares. The first is hardware isolation for the most sensitive inputs, since on-device enclaves such as Apple's Secure Enclave keep biometric matching inside a dedicated subsystem,[23] although the strength of any privacy guarantee still depends on the complete system design and threat model. The second is personalization, because models can exploit on-device signals and user context without shipping raw data anywhere. Specialized on-device ML frameworks keep such inference within 5–50 ms user-interface response times.

The cost shows up in engineering effort as well as joules. Peak NPU throughput can substantially exceed what the device sustains under prolonged load, device heterogeneity requires multiple

[22] **System-on-chip (SoC)**: Mobile SoCs integrate CPU, GPU, and NPU cores with caches and memory controllers to reduce data-movement costs. Mobile DRAM is generally off-die, and feasible model size depends on capacity, precision, working memory, bandwidth, and latency.

[23] **Face ID**: Apple performs facial matching within the Secure Enclave and encrypts the stored mathematical representations so that they remain available only to that subsystem and do not leave the device (Apple 2024a, 2024b). Apple reports a random false-match probability below 1 in 1,000,000 for one enrolled appearance, with higher probabilities for some populations and configurations. The example shows how local inference and hardware isolation can reduce transmission exposure while supporting low-latency authentication.

model variants, development multiplies across platforms, and app-store review and staged release slow iteration compared with a service its operator controls entirely.

2.7.2 Personal assistant and media processing

Several short ML pipelines on one phone must share the same battery and thermal budget, and computational photography shows that constraint most sharply. Modern flagships process each photo through multiple ML stages: portrait mode uses depth estimation and segmentation, night mode aligns multiple frames with ML-based denoising, and HDR merging, super-resolution, and scene optimization run in sequence. The binding constraint falls on the pipeline rather than on any individual model, because these models must share a 3–5 W power budget and complete within the user's perceived shutter delay, which requires careful scheduling across CPU, GPU, and NPU to avoid thermal throttling.

Voice-driven interactions are bound by the energy of always-on listening, so they partition work across the same SoC by power tier. Wake-word detection runs continuously on a dedicated low-power core, while speech recognition and keyboard prediction wake the higher-power neural processing tiers only when needed.

Health monitoring and augmented reality push mobile ML to its sustained-performance limits. Wearables like Apple Watch process ECG and accelerometer data on-device to reduce the radio energy of transmission, while AR frameworks must fit simultaneous localization, hand tracking, and scene understanding into a 16.7 ms frame budget at 60 FPS without exceeding the sustained thermal envelope. These applications represent the ceiling of what battery-powered, passively-cooled devices can sustain, and they define the boundary beyond which mobile optimization alone is insufficient.

A common pitfall is deploying desktop-trained models directly to mobile or edge devices without architecture modifications. A desktop ResNet-50 pipeline may require gigabytes once activations, batches, preprocessing buffers, and runtime overhead are included, even though the FP32 weights alone are about 102.4 MB. Those weights fit in the 512 MB of RAM on a representative low-end edge target, but at 8.2 GFLOP per inference its 1 GFLOP/s processor takes seconds per inference, a latency failure rather than a capacity one. Desktop-optimized models may also use operations unsupported by mobile hardware, assume numerical formats unavailable on embedded systems, or require batch processing incompatible with single-sample inference. Successful deployment demands architecture-aware design from the beginning, so model families, arithmetic formats, and implementation choices must all match the target device.

Mobile ML shows that useful intelligence fits within a 3–5 W thermal envelope on battery power. A smartphone still costs hundreds of dollars, carries gigabytes of memory, and needs a daily recharge, and no amount of mobile optimization closes the roughly 10,000-fold memory gap to a device that must cost dollars and run unattended for years. That regime needs a different class of hardware and algorithms.

2.8 TinyML: Ubiquitous Sensing

Instrumenting every pallet in a warehouse, every cable on a suspension bridge, or every beehive in an apiary means putting "eyes and ears" on tens of thousands to millions of physical objects. At that count, each device must cost dollars rather than hundreds of dollars and measure millimeters rather than centimeters. Smartphones are far too expensive and too large for the role; ubiquitous sensing at this scale requires unit costs under ten dollars.

TinyML completes the deployment spectrum with low-cost, low-power ML on deeply constrained embedded devices (Warden and Situnayake 2020; Janapa Reddi, Plancher, et al. 2022). Devices costing under $10 and averaging less than one milliwatt[24] of power when duty-cycled make ubiquitous[25] sensing economically practical at massive scale; MLPerf Tiny and MCUNet show how such constraints are evaluated and optimized in practice (C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kiraly, et al. 2021; Lin et al. 2020). Its first feasibility screen is memory capacity, because weights must fit in flash and working state in SRAM, but once a model fits, power is the limit that shapes the design. This is the domain of the power-bound archetype, where the optimization objective shifts from maximizing throughput to minimizing energy per inference. With duty-cycled execution, a keyword spotting model consuming about 10 µJ per inference on dedicated low-power audio silicon (a general-purpose microcontroller running the same model spends orders of magnitude more)

↪ The framework that organizes pruning, distillation, and quantization to fit edge memory budgets is developed in section 10.1.

24 **The 1 mW threshold**: Ambient energy harvesting can power a device indefinitely only when long-term harvested power exceeds the complete system's average load. Available power varies across sources and environments. Solar cells the size of a thumbnail may produce ~10 mW outdoors but ~10 µW indoors, thermoelectric generators on warm pipes ~100 µW, and RF energy from nearby transmitters ~10 µW. This crossover can transform the deployment model from "battery-limited lifetime" to "deploy and forget," but it depends on the source, environment, and workload.

25 **Ubiquitous computing**: Mark Weiser's ubiquitous-computing vision imagined computation woven into everyday environments until it receded from direct attention (Weiser 1991). TinyML is one modern path toward that vision, because it brings the cost and power of an intelligent sensor low enough for mass deployment.

can operate for years on a coin-cell battery, trading model capacity for operational longevity. On an Arduino Nano 33 BLE Sense development board (figure 2.9), on-board microphone, motion, and environmental sensors share a single substrate with a low-power microcontroller executing quantized TensorFlow Lite Micro models (David et al. 2021). The board illustrates integration; its full power draw depends on the sensors, radio, and duty cycle.

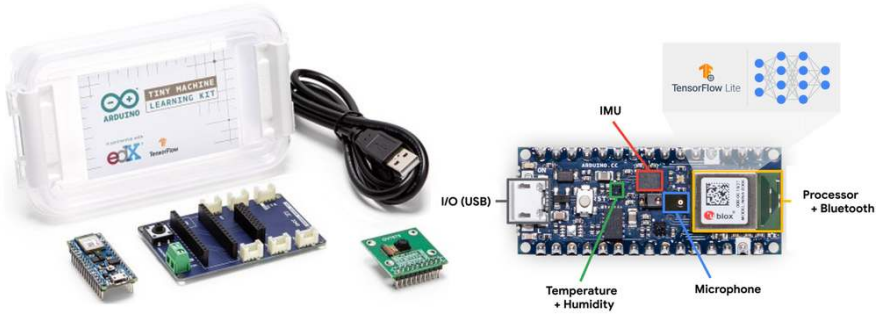


Figure 2.9: TinyML Hardware Scale: Microcontroller development boards integrate sensors, processing cores, and low-power radios onto a compact substrate, making constrained ML applications accessible for prototyping. Finished sensing devices must budget SRAM, sensor and radio power, inference cadence, and sleep energy to achieve multi-year battery life. Source: Figure 6 in Janapa Reddi, Plancher, et al. (2022), licensed under CC BY 4.0.

TinyML runs on microcontrollers (MCUs),[26] which omit the power-hungry infrastructure of conventional computing. Mobile and edge processors rely on external dynamic RAM (DRAM), high-speed memory buses, and operating system runtimes that draw milliwatts just to keep DRAM in self-refresh and hundreds of milliwatts once the memory bus is active. Many microcontrollers integrate the processor core, non-volatile flash, and static RAM (SRAM) onto a single silicon die, reducing reliance on external memory buses (C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kiraly, et al. 2021; Lin et al. 2020; Janapa Reddi, Plancher, et al. 2022). Eliminating off-chip signaling lets duty-cycled average power fall below a milliwatt and sleep power reach microwatts, although active inference on boards such as the ESP32-S3 still draws tens to hundreds of milliwatts. Some targets, including ESP32-S3 variants, instead attach external flash or additional RAM. On-chip integration still imposes a strict internal-memory ceiling. On-chip SRAM is physically constrained by silicon die area: because each SRAM cell requires six transistors (6T), expanding capacity directly inflates manufacturing cost. A TinyML model must therefore execute within tens to hundreds of kilobytes of SRAM, with no virtual memory, paging, or swap space to cushion an overflow. The small keyword-spotting task and low-power hardware place it at the bottom of figure 2.7; duty cycle also determines its average power.[27] Multi-year coin-cell operation additionally requires the complete device load to fit the battery budget.[28] Fitting the entire inference pipeline, from sensor acquisition to classification output, into this kilobyte-scale memory footprint defines the TinyML operating regime.

[26] **Microcontroller (MCU)**: A single-chip computer whose design prioritizes minimal cost and power over performance. Memory fit therefore becomes a deployment prerequisite, because the memory actually available to the target is often orders of magnitude less than a smartphone provides.

[27] **TinyML energy gap**: The figure compares different tasks on different hardware and with different energy accounting boundaries; its ratios do not identify the effect of hardware integration alone. A batch-1 ResNet-50 inference on a cloud accelerator consumes about 75.5 mJ of accelerator energy, while an always-on keyword spotter uses about 10 µJ, a gap of ~7,500×. A cloud LLM query at about 1 kJ widens the gap to 100,000,000×, the span plotted in figure 2.7.

[28] **Coin-cell deployment**: A CR2032 battery (225 mAh at 3 V, ~675 mWh) ideally powers a complete 10–50 µW system for about 1.5–7.7 years. This "deploy-and-forget" operating model drives innovation in intermittent computing, where the device sleeps between inferences to stretch the energy budget across years of unattended operation.

Definition 2.5: TinyML

TinyML is the machine learning domain of always-on sensing constrained by kilobyte-scale memory and milliwatt-scale power.

1. **Significance**: It requires models whose weights and code fit in flash while activations and runtime state fit in on-chip SRAM, enabling continuous inference on milliwatt power budgets.
2. **Distinction**: Unlike mobile ML, which uses multi-watt processors and a full OS, TinyML runs on microcontrollers using bare-metal firmware or a real-time operating system.
3. **Common pitfall**: A frequent misconception is that TinyML is just "small models." In reality, it is a power-bound regime, and the primary metric is energy per inference (microjoules), not just the parameter count.

This definition reframes the iron law of ML systems. In terms of equation 2.5, TinyML imposes a physical memory-capacity precondition that execution time does not capture: the weight footprint M_{weights} and activation footprint $M_{\text{activations}}$ must stay within memory capacity M_{device}. When total memory is measured in kilobytes, model structure and runtime allocation determine feasibility before latency optimization begins, and every avoidable data movement spends part of the energy budget. The objective therefore shifts from raw throughput toward energy per inference and memory fit.

The same four-branch map applies at the bottom of the spectrum (figure 2.10). Milliwatt budgets and kilobyte memories favor low-precision execution and duty-cycled sleep, and they move the failure boundary away from latency toward compression-driven accuracy loss and risky firmware updates.

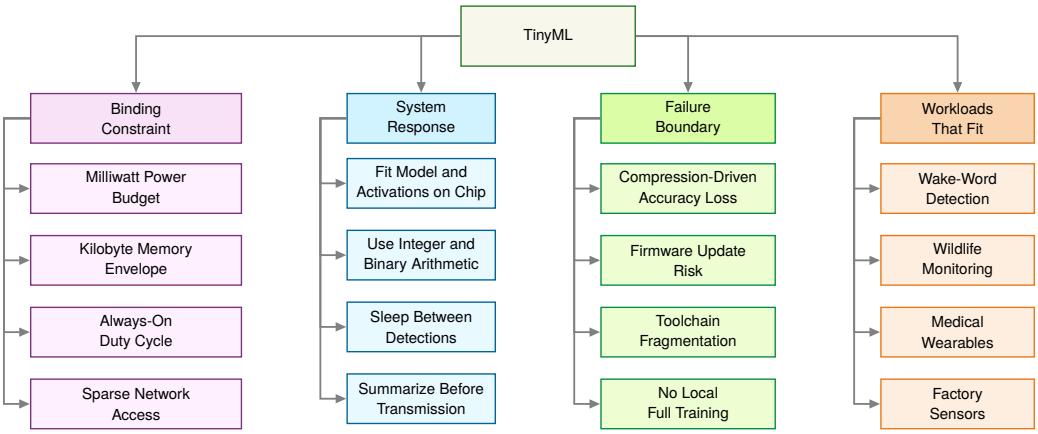


Figure 2.10: TinyML Constraint Map: Milliwatt power and kilobyte memory make always-on sensing economically possible, but those same limits force the model, activations, runtime, and update path to fit a radically smaller engineering envelope.

2.8.1 TinyML advantages and operational trade-offs

TinyML devices range from palm-sized developer kits to millimeter-scale chips,[29] and their kilobyte memories and milliwatt budgets allow months or years of autonomous operation. The models are usually trained elsewhere,[30] and the devices demand specialized algorithms, aggressive model compression, and careful systems co-design. TinyML also changes deployment economics: removing the radio uplink and the application processor lets per-node unit cost fall to single-digit dollars, which makes dense instrumentation across thousands of physical assets financially viable, and microjoule inference budgets allow multi-year operation on inexpensive coin-cell batteries or on harvested ambient energy.

The trade-offs are just as distinctive. Microcontrollers commonly provide 10^5 to 10^6 bytes of RAM and enforce a strict two-tier memory partition, in which static model weights and compiled code reside in non-volatile flash (often a few megabytes), while mutable activations and the runtime buffer (the *tensor arena*, a fixed block of SRAM reserved for them) must fit within the much smaller on-chip SRAM (a few hundred kilobytes at most, such as the ESP32-S3's 512 KB, and as little as tens of kilobytes on the smallest parts). As a result, peak intermediate activation volume, rather than static parameter storage, is frequently the barrier that prevents a model from executing. The smart doorbell's level-1 check (figure 2.3) shows why that check is only a first screen: its weights fill just 11.9 percent of flash, a figure that says nothing about whether its intermediate results fit in SRAM. Embedded development also demands cross-stack co-design: engineers manage memory pools by hand, debug bare-metal firmware without operating-system abstractions, and optimize operator kernels directly for constrained instruction sets.

These bounds also create lifecycle and reliability challenges. An energy-harvesting device can satisfy its long-term average power budget yet fail when stored energy is exhausted before the next harvest. Its duty cycle must therefore respect both energy per inference and the timing of

[29] **TinyML device range**: This physical range reflects a direct trade-off between deployment context and computational capability. Millimeter-scale systems prioritize minimal power for single-function, long-duration tasks, whereas palm-sized boards offer more sensing, interfaces, and debugging support at higher cost and power. Development-board size and power therefore should not be treated as limits for a finished TinyML product.

[30] **On-device training constraints**: Updating a model requires working memory beyond that needed to apply it, including information retained from the prediction computation to determine parameter changes. Intermediate results can be discarded only after their final use, which often occurs later during training than during inference. With only 256 KB–2 MB RAM, many microcontrollers cannot hold the full training state; specialized methods reduce that state by limiting which parameters change. TinyTL, for example, freezes the weights and updates mainly the biases, so most intermediate activations need not be retained for the update (Cai et al. 2020). Training elsewhere and delivering an updated model is therefore a common design, although constrained local adaptation remains possible.

available power. Model quality can also suffer from aggressive quantization and pruning: converting 32-bit floating-point weights to 8-bit integers (INT8) saves memory but can degrade accuracy and robustness on out-of-distribution sensor inputs. Post-deployment flexibility is also sharply constrained: models are often packaged with firmware, so field updates must account for limited storage, interrupted transfers, and safe rollback. Ecosystem fragmentation[31] across microcontroller vendors and ML frameworks creates additional overhead and portability challenges.

2.8.2 Environmental and health monitoring

The TinyML regime becomes essential when power budgets, unit economics, and physical isolation rule out every higher tier. Wake-word detection is the most familiar case. Its binding constraint is average power, so the always-on detector acts as a power gate that wakes the costlier processors only when a wake phrase is heard.[32] Precision agriculture is bound instead by sparse, expensive connectivity across large fields. FarmBeats used sensors, cameras, drones, and local gateway processing to reduce raw data movement where connectivity was costly (Vasisht et al. 2017), and TinyML pushes that logic down to sensing nodes that must run under milliwatt budgets.

Wildlife conservation shows the uplink arithmetic in an illustrative case. Solar-powered acoustic sensors identify species on-device and transmit periodic detection summaries rather than raw audio, cutting uplink data from 4.3 GB/day to 400 KB/day, a 10,750× reduction that conserves power in off-grid deployments. Medical wearables apply the same logic to physiological monitoring, where runtime remains bounded by sensor draw, radio power, duty cycle, and sleep energy rather than by the model alone.

All four paradigms sit on one continuum bounded by the speed of light, heat dissipation, and memory signaling, and a real feature usually presses against several of those bounds at once. Choosing among the paradigms therefore means mapping an application's constraints against all of them together.

2.9 Choosing a Deployment Paradigm

An architect choosing where to run an ML feature rarely optimizes one dimension. A privacy rule may forbid cloud processing, a latency budget may forbid remote inference, a memory footprint may exceed the mobile device, and a cost target may rule out always-on edge hardware. Cloud, edge, mobile, and TinyML are operating envelopes for those conflicts, so selecting among them requires a unified comparison framework and a structured decision process.

2.9.1 Weighing operational trade-offs across tiers

A model that fits comfortably inside a tier's latency, power, and memory budgets can still fail in production if the team cannot ship updates to it, if a dropped internet link halts the service, or if monthly cloud bills outrun the product's revenue. Choosing a deployment tier requires weighing latency-vs-throughput paradigm trade-offs across physical and operational budgets at once. The resource envelopes are already on the table: table 2.1 gives each paradigm's latency, power, and memory regime, and table 2.4 grounds them in concrete platforms. What those tables leave out are the operational dimensions that decide many real deployments: how the system scales, what it exposes, whether it needs a network, what it costs, and how hard it is to build and ship. Table 2.9 contrasts the four paradigms along those eight dimensions.

Table 2.9: Operational Paradigm Comparison: Eight operational dimensions complement the resource envelopes of table 2.1 and table 2.4. Cloud ML scales most easily, while local processing can reduce transmission and centralized-storage exposure without itself guaranteeing privacy; local tiers can operate without a WAN, but real-time response depends on workload and complete-path timing, while development complexity is highest at the two ends of the spectrum. Cost anchors mix recurring service bills with device purchase prices; they are partial illustrations, not a matched lifecycle-cost ranking.

Aspect	Cloud ML	Edge ML	Mobile ML	TinyML
Scalability	High (provider capacity limits)	Good (limited by edge hardware)	Moderate (per-device scaling)	Limited (fixed hardware)
Data Exposure	Remote processing expands the surface	Local processing reduces transmission	On-device processing reduces transmission	Raw data can remain local

31 **TinyML ecosystem fragmentation**: TinyML spans microcontroller families such as Arm Cortex-M, RISC-V, and Xtensa, each with different instruction sets, memory layouts, kernels, and vendor toolchains; cloud and mobile stacks also vary, but their runtimes usually hide more of this hardware diversity. A model optimized for one microcontroller target may therefore require retuning and revalidation for another, multiplying the engineering cost of multi-device deployment.

32 **Always-on wake-word detection**: A small model that listens continuously for the acoustic signature of a wake phrase. The acceptable detector budget depends on the platform, but spending even a few extra milliwatts continuously can materially shorten battery life, and the architecture succeeds only when the always-on path consumes far less average power than the components it prevents from waking.

Aspect	Cloud ML	Edge ML	Mobile ML	TinyML
Connectivity Required	Required for remote requests	Intermittent	Optional	None
Offline Capability	None	Good	Excellent	Complete
Real-time Processing	Network and execution must fit deadline	Local execution must fit deadline	Deadline and sustained-power limits	Deadline and memory-capacity limits
Cost anchor	Service bill ($1000s+/month)	Hardware purchase ($100s–$1,000s per device)	Device purchase ($200–$1000+; often user-owned)	Hardware purchase ($1–$10 per device)
Development Complexity	High (cloud expertise needed)	Moderate-High (edge+networking)	Moderate (mobile SDKs)	High (embedded expertise)
Deployment Speed	Fast	Moderate	Fast	Slow

Remote processing widens the network attack surface and requires transport encryption and strict isolation between tenants; local processing keeps raw data on the device, reducing network exposure without eliminating the risk of physical tampering. The workload archetypes from section 2.3 map directly to these boundaries, with each archetype gravitating toward paradigms that relieve its primary bottleneck. Each paradigm also shifts engineering complexity to a different subsystem: cloud deployments demand expertise in multi-tenant networking and dynamic cost governance, whereas edge and TinyML deployments trade those cloud concerns for local thermal and memory limits along with distributed updates and site maintenance. Mobile deployments sit between the two, operating on user-owned hardware that avoids infrastructure purchase costs at the price of platform-specific drivers, operating-system background task limits, and shared energy budgets, so none of the four categories supplies a workload-independent latency or energy ranking.

A common architectural failure is selecting a deployment target based on model accuracy alone while treating hardware constraints as downstream optimization details. An unquantized model that delivers 99 percent accuracy in an offline evaluation becomes unviable if its memory footprint triggers paging on mobile hardware, or if continuous inference drives thermal throttling within seconds. Physical invariants (memory capacity, thermal dissipation limits, and network transit delay) must serve as early screening criteria before model development begins, pruning invalid deployment targets before training resources are expended.

2.9.2 Four screening gates for choosing a tier

Before asking whether a workload belongs in the cloud or on a microcontroller, an engineer should ask a simpler question: does this problem need machine learning at all? Deep neural networks carry a heavy maintenance burden that deterministic code avoids, and a fifty-line heuristic rule set can often solve the problem at a tiny fraction of the operating cost.



Systems Perspective 2.3: The complexity tax

Consider a classification task solvable either by deterministic heuristic rules or by a deep learning pipeline. The heuristic requires fifty lines of code, incurs near-zero compute cost, demands approximately one hour per month to update rules, and fails in ways engineers can read in the code. The ML system may require only fifty lines of model definition, but, as Sculley et al. (2015) observed of production ML generally, it introduces roughly 2,000 lines of supporting infrastructure for feature ingestion, telemetry logging, drift monitoring, and GPU driver management, alongside 40 hours per month diagnosing data drift and maintaining pipelines.

An ML model that increases accuracy from 90 percent to 95 percent while multiplying operational maintenance by, say, 40× requires an economic comparison. The accuracy and maintenance ratios alone do not establish the return; the value of the avoided errors must

justify the additional engineering cost. When the ongoing maintenance cost outstrips the value of marginal statistical gains, deterministic software remains the correct systems choice.

When this **complexity tax** outweighs the operational value of improved statistical predictions, the ML deployment costs the system more than it returns. Once a learned system is justified, however, paradigm selection relies on a tiered decision framework ordered by how rapidly a constraint invalidates an architecture. The screening hierarchy evaluates four successive gates: regulatory privacy, physical latency, local compute capacity, and lifecycle economics. Privacy leads the order because a legal prohibition on moving data eliminates an option before anything is measured, whereas the latency and compute gates require numbers. Each gate passes only its survivors to the next, so the screening sequence acts as successive filters on a candidate set (figure 2.11).

↪ Framing the problem against the existing baseline, and retaining that baseline when the expected benefit does not justify the added cost, is developed in section 3.5.

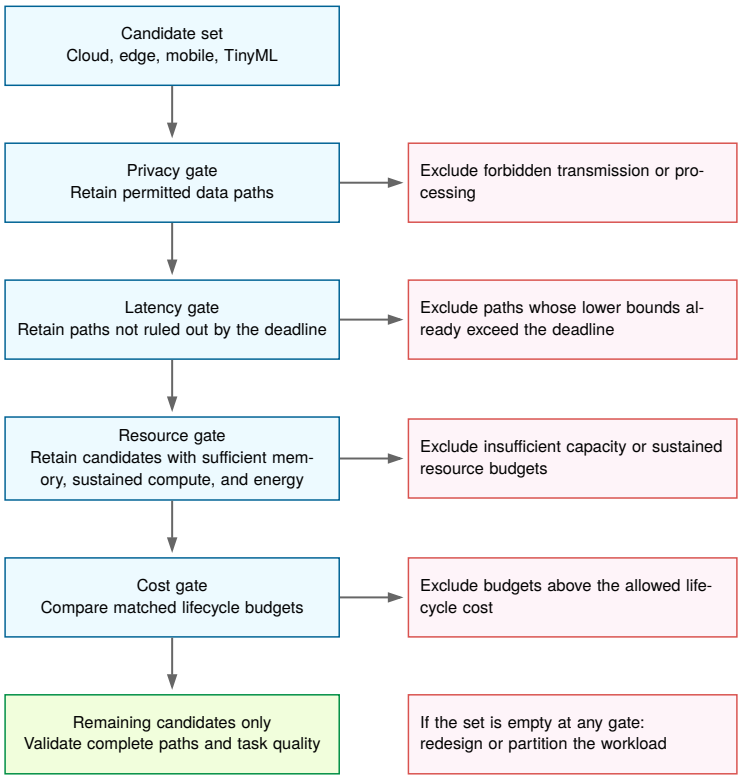


Figure 2.11: Deployment Decision Screening: Apply privacy, latency, resource, and cost gates to the remaining candidate set. Every gate retains upstream exclusions. An empty set requires redesign or partitioning; surviving candidates still need complete-path validation.

The flowchart carries a remaining candidate set through all four gates. If regulatory rules or data sovereignty forbid off-device transmission, remote cloud processing stays excluded when compute demand is considered. If a network round-trip lower bound exceeds the deadline, additional remote compute cannot restore that path. Cost compares only the survivors, and an empty set calls for changing the workload or partitioning it across tiers. Passing a lower-bound latency screen still requires complete-path validation. In automotive emergency braking, the latency gate binds before compute capacity or cost are even considered.

Example 2.1: Autonomous vehicle emergency braking

Scenario: An autonomous vehicle vision system performs real-time pedestrian detection for emergency braking under a strict 100 ms end-to-end latency budget.

Diagnosis: At 100 km/h, the vehicle covers 2.8 m during the 100 ms budget, so every millisecond of delay is distance traveled before braking begins. The scenario assumes 50–150 ms of network round-trip delay for cloud offload, which consumes half the budget at best and more than all of it at worst before any detection runs. Local object detection on an automotive edge accelerator takes 10–30 ms; at 10 GFLOPs per frame and 30 FPS, that accelerator must sustain 300 GFLOP/s.

Systems lesson: The stated network-delay range cannot guarantee the deadline, so the latency gate excludes that cloud path. The local candidates remain subject to the resource gate: the pedestrian detector must fit memory and sustain the stated frame rate within its power budget. An automotive edge accelerator remains a candidate under these assumptions; its complete sensing-to-braking path still needs validation. Cloud connections can support offline training and telemetry without reentering the emergency decision path.

A completed placement decision needs evidence about the surviving paths as well as grounds for rejecting the others. A factory inspection line supplies both.

Example 2.2: Completing a placement decision

Problem: A fixed factory camera requires 20 FPS inspection with a 100 ms sensing-to-decision deadline. Raw images must remain on premises. Which feasible deployment has the lowest matched lifecycle cost under the following hypothetical evidence?

Supplied evidence: A rule-based baseline fails the specified missed-defect criterion, while the learned detector meets both the missed-defect and false-alarm criteria on representative inspection data. The proposed mobile and microcontroller variants fail that same quality requirement. Local edge candidates A and B preserve the validated detector and preprocessing. Both have a complete peak working set of 2 GB within an available 4 GB, and each must stay within a 25 W sustained system-power budget.

Under the specified peak load and thermal conditions, the supplied complete-path latency bounds are 80 ms for A and 40 ms for B, including acquisition, preprocessing, queuing, inference, and the inspection decision. Their sustained rates are 25 FPS and 40 FPS, at 10 W and 20 W. Both satisfy the same availability and recovery requirements. Annualized budgets over the same service lifetime, including hardware, power, installation, maintenance, and recovery provisions, are $2,400 for A and $3,600 for B. These supplied bounds and budgets define an instructional scenario, not benchmarks of named products.

Result: Select local edge candidate A. Privacy excludes raw-image cloud offload, task quality excludes the proposed smaller variants, and both edge paths satisfy the timing, throughput, memory, and power requirements. A has the lower matched cost. B's faster response supplies no required benefit under this specification; a tighter deadline could change the choice.

Systems lesson: The final choice follows from task quality and complete-path evidence, not the tier label or peak compute rate. The decision remains valid within the supplied operating conditions; changes to the input stream, detector, load, or thermal environment require reevaluating the relevant gates.

The completed comparison depends on operational maintainability as well as execution performance. Its matched budgets include obligations that a hardware-only comparison would omit. Deploying a model across thousands of edge nodes or millions of microcontrollers introduces sustained lifecycle overhead: automated firmware updates, out-of-band telemetry, distributed drift monitoring, and hardware failure recovery. An organization equipped for cloud operations may lack

the embedded toolchains and physical testing infrastructure required to support edge or TinyML targets at scale.

Checkpoint 2.2: System design

First justify the learned system against simpler baselines; then screen its deployment candidates against the physical and operational constraints.

Decision gates

- ☐ **The baseline**: Can you distinguish a rule-based baseline, such as a regular expression, from a simple learned baseline, such as a small model trained on a handful of features, and explain what comparison each provides?
- ☐ **Physical feasibility**: If a remote path's propagation floor exceeds the service deadline, can you reject it and identify which local execution and memory measurements are still needed before choosing an alternative?
- ☐ **The infrastructure cost**: What evidence about error consequences and engineering cost is needed to decide whether a five-point accuracy gain (90 to 95 percent) justifies a hypothetical 40× increase in monthly maintenance hours?

The decision framework screens candidate tiers for physical feasibility and then compares their operational costs. If no single tier meets the requirements of the whole pipeline, the next question is which stages can run separately, and what must cross the boundaries between them.

2.10 Hybrid Architectures

A voice interface can use a microcontroller to listen continuously for a wake word, a phone to transcribe speech, and cloud servers to interpret requests that need a larger model. Each stage has a different task and resource budget. Sending every audio sample to the cloud would add communication and radio costs; asking the microcontroller to perform every stage would exceed its compute and memory capacity. *Hybrid architectures* divide the pipeline across tiers so that each stage meets its requirements (Kang et al. 2017). Economics drive this division as much as latency: processing voice interactions for billions of devices entirely in the cloud runs into both an economic and an infrastructure ceiling, and the voice assistant wall puts a number on each.

Napkin Math 2.10: The voice assistant wall

Problem: In this illustrative capacity scenario, 1B voice assistant devices (smartphones, smart speakers, earbuds) each issue 20 queries/day after the wake word. Detecting the wake word itself in the cloud would require every microphone to stream audio continuously. What load would cloud-side detection impose, and what load remains in the cloud once detection moves on device?

Cloud-side detection: The always-listening path that local detection removes.

- **Audio streaming**: Every device streams audio continuously (16 kHz, 16-bit), about 32 KB/s each. Across 1 billion devices the aggregate is 32 TB/s, and each stream would also need its own continuously running detector.
- **Cloud cost**: ~$0.50/device/year → 1B devices = $500,000,000/year. Economically prohibitive under these assumptions for a free feature.
- **TinyML alternative**: 0.1–1 mW local wake-word detection, <$0.01/device/year. If each post-wake query uploads about 5 s of audio, local detection cuts the audio leaving each device by about 860×.

Residual cloud load: The post-wake queries the hybrid design still serves in the cloud. Even with local wake-word filtering, the residual query volume (1 billion devices generating 20B

queries/day at ~200 ms per query) still requires substantial centralized accelerator capacity (~1.1 million GPU-hours per day).

Five parameters determine the required cluster capacity:

1. **Query volume**: 1 billion devices × 20 queries/day = 20B queries/day.
2. **GPU demand**: Each query requires ~200 ms of GPU time. Total: 1.1 million GPU-hours/day.
3. **Scenario capacity**: A data-center equivalent provisioned with 10,000 GPUs provides 240,000 GPU-hours/day.
4. **Average requirement**: ~4.6 dedicated data centers just for voice inference.
5. **Peak reality**: Queries cluster in waking hours (~4.5× peak-to-average), requiring ~20.8 data centers at peak.

Systems insight: Local detection reduces each device's upload to its post-wake queries, 20 queries/day of a few seconds each. The residual post-wake load is still large, tens of data centers at peak, but it is bounded by what users actually ask, and it is the load the hybrid design deliberately keeps in the cloud to use a larger language model than the local device can support.

Partitioning the voice pipeline into an on-device wake-word filter and a cloud-based language model relieves both the client's energy limit and the data center's capacity limit. The division also creates engineering work: deciding what data crosses each boundary and keeping the participating stages consistent. Three integration patterns organize these choices.

2.10.1 Splitting work across tiers: Three integration patterns

When a single tier cannot satisfy every stage of an ML pipeline, engineers split the pipeline along one of three physical boundaries: train-serve split, hierarchical processing, or progressive deployment. Selecting a pattern is an iron-law decision: each stage of computation must execute where its binding term (throughput capacity R_{peak}, local latency floor L_{lat}, or the memory-capacity precondition) incurs the lowest system penalty.

Under a **train-serve split**, training executes in centralized data centers while inference runs locally on edge, mobile, or embedded devices. Training must retain intermediate results and information used to calculate and apply parameter updates, in addition to the model weights. Under common 16-bit mixed-precision training with a standard adaptive optimizer, the weights, their gradients, and the optimizer's running statistics together occupy about 16 bytes per parameter (Rajbhandari et al. 2020), roughly 8× the 2 bytes per parameter that 16-bit inference weights need, before any batch activations are counted (section 8.4.3.2 develops this accounting). Inference uses the trained weights to compute predictions, sometimes through repeated passes as in token-by-token generation, and must satisfy local latency and privacy constraints. Amortization underpins the economics of this split: the high fixed cost of training is distributed across millions of forward-pass queries served locally without recurring network transmission costs.[33]

[33] **Train-serve cost asymmetry**: Training usually requires many forward and backward passes, whereas one inference request uses a trained artifact for one or more forward passes. Training is not literally one-time because models may be retrained, and inference cost varies sharply with model size, sequence length, batching, and service overhead. The economic rationale for the split is amortization: repeated use can spread each training cycle's fixed cost across many served predictions.

Definition 2.6: Hybrid ML

Hybrid machine learning is the deployment strategy that distributes an ML pipeline across two or more deployment tiers, assigning each stage according to its latency, compute, data, and operational constraints.

1. **Significance**: Hybrid architectures exploit the iron law's additive structure: the edge tier can reduce L_{lat} for time-sensitive preprocessing and inference, while the cloud tier can provide the R_{peak} needed for training, retraining, and heavy batch inference. The split follows the data locality invariant: compare each stage's complete local and remote paths, reject paths that miss the service deadline, and then choose among the feasible alternatives.

2. **Distinction**: Unlike cloud-only deployment (which accepts the distance penalty for all stages) or edge-only deployment (which accepts limited R_{peak} for all stages), hybrid ML assigns each pipeline stage to the tier where its binding iron law term is minimized.
3. **Common pitfall**: A frequent misconception is that hybrid ML is just "running two models." In reality, the participating tiers must share synchronized state (feature definitions, model versions, and preprocessing logic) so that their paths produce consistent results. Without this synchronization, training-serving skew emerges at tier boundaries.

Under **hierarchical processing**, computation cascades across tiers to filter and summarize data step by step, matching communication volume to available link bandwidth. Raw sensor streams from cameras or microphones at the edge of the network often exceed wide-area network (WAN) uplink capacity; continuously streaming uncompressed telemetry to a data center saturates network links and wastes radio power. To prevent network saturation, early tiers filter data locally: TinyML microcontrollers run low-power anomaly detectors, edge gateways aggregate and correlate temporal features across multiple sensors, and cloud clusters ingest only anomalous events or aggregated summaries for global analytics. During inference serving, hierarchical processing frequently takes the form of a confidence cascade (Teerapittayanon, McDanel, and Hsiang-Tsung Kung 2017). A compact, low-latency model on the local device evaluates the input first. If its prediction confidence exceeds a calibrated confidence threshold, the device returns the result immediately, bypassing the network entirely. Only ambiguous or low-confidence inputs trigger a WAN payload transfer to a larger cloud model. The cascade seeks lower median latency at the required task quality, but confident local errors can bypass escalation. Representative evaluation must establish the routed system's accuracy, error costs, and escalation frequency before accepting that trade-off.

Under **progressive deployment**, an engineering team derives a family of specialized model variants from a unified architecture or training corpus, tailoring each variant to the memory capacity, arithmetic intensity, and thermal limits of a specific tier. Rather than forcing a single model onto every device, developers produce tier-specific artifacts and evaluate the quality lost or retained as resource demands fall. The compression mechanisms are developed later; the placement decision here depends on whether each artifact meets its task and hardware constraints. For a shared image-classification task, an edge server and a phone can use different members of a model family, each evaluated against the same task requirements and its own resource budget. A voice interface that divides wake-word detection, acoustic processing, and language understanding among tiers illustrates hierarchical processing across different tasks, and it can also use tier-specific variants within a task.

With three integration patterns available, selection becomes a constraint-matching problem: choose the pattern whose trade-off profile matches the system's dominant bottleneck, weighing the conditions that favor each pattern against those that disqualify it (table 2.10).

Table 2.10: Hybrid Pattern Selection Guide: Each of the three hybrid integration patterns has a characteristic trade-off, favoring conditions, and disqualifying conditions. Selection matches the pattern's trade-off profile to the system's dominant bottleneck.

Pattern	Trade-off	Choose when	Avoid when
Train-Serve Split	Training cost vs. inference latency	Training requires more resources than inference; training data may leave the serving device	Local adaptation cannot wait for remote training and model delivery
Hierarchical Processing	Local autonomy vs. global optimization	Data volume exceeds transmission capacity; decisions needed at multiple timescales	All processing can occur at one tier; network is reliable and fast
Progressive Deployment	Model quality vs. deployment reach	Same model needed at multiple capability levels; graceful degradation required	Model cannot be meaningfully compressed; single deployment target

Production deployments rarely implement these patterns in isolation; complex systems compose them across different phases of the pipeline. An autonomous vehicle, for example, combines progressive deployment to fit specialized vision backbones into its on-board compute envelope,

hierarchical processing to arbitrate between immediate millisecond-scale emergency braking on local hardware and multi-second trajectory planning, and a train-serve split that uploads anomalous driving slices to cloud clusters for fleet-level retraining. Privacy-preserving distributed training approaches extend this composition further when raw training data cannot leave client devices for regulatory or privacy reasons. In federated learning (McMahan et al. 2017), for example, each device trains on its own data and sends only model updates to the cloud for aggregation.

2.10.2 How hybrid tiers communicate in production

If a phone's audio preprocessor updates its normalization constants while the cloud transcription model expects the old scaling, or if a smartwatch and phone disagree on which activity labels are active, the hybrid pipeline fails silently across the tier boundary. Keeping multiple execution tiers working as one system requires explicit communication pathways. Five interaction channels connect the tiers (figure 2.12): compiled model artifacts deploy downward from cloud training clusters to edge, mobile, and TinyML inference engines, while raw sensor streams and filtered telemetry propagate upward toward cloud analytics, complemented by peer-to-peer result forwarding, local assist handoffs, and state synchronization that keeps feature definitions and model versions consistent across tiers.

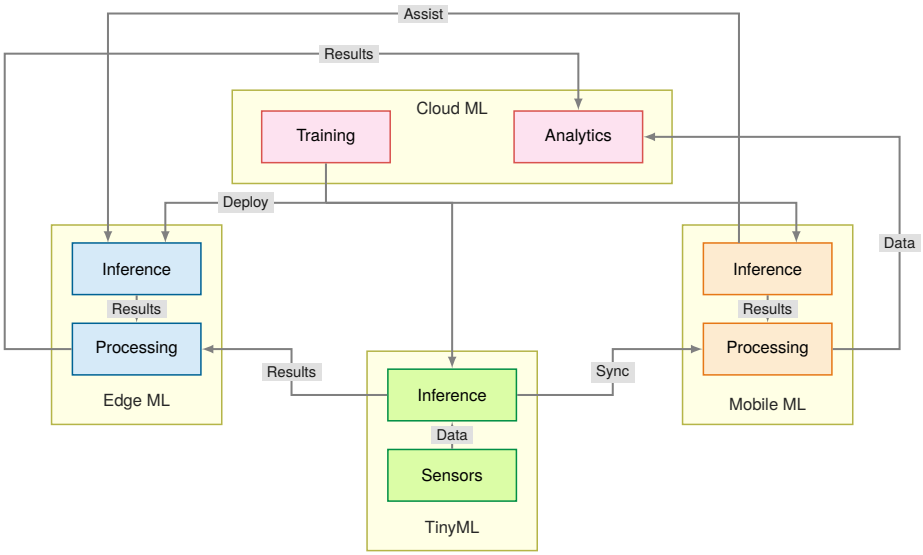


Figure 2.12: Hybrid System Interactions: Data flows upward from sensors through processing layers to cloud analytics, while trained models deploy downward to edge, mobile, and TinyML inference points. Five connection types (deploy, data, results, assist, and sync) establish a distributed architecture where each paradigm contributes unique capabilities.

Production systems place each tier boundary at a distinct physical bottleneck:

1. **Industrial defect detection**: Assembly lines running at high cycle rates require sub-10 ms deterministic visual inspection. Streaming high-resolution camera frames over WAN to a data center violates inspection deadlines and exhausts factory uplink bandwidth. The system deploys a train-serve split: centralized cloud infrastructure trains high-capacity vision models on historical defect databases, then compiles and quantizes runtime weights for factory-floor edge servers and embedded smart cameras, which run inference locally without external network dependencies. The complete local pipeline, including frame acquisition, preprocessing, and inference, must still be validated under the expected load.
2. **Agricultural monitoring**: Deployed across hundreds of hectares, remote soil probes run on coin-cell batteries or small solar harvesters and communicate over low-power wide-area radios such as LoRa, which carry little data and spend much energy per bit. Streaming continuous moisture and temperature telemetry would exhaust battery reserves within days. Hierarchical processing therefore keeps high-frequency sampling and basic anomaly detection on TinyML

microcontrollers in sub-milliwatt power budgets, while local edge gateways aggregate measurements across clusters of probes and send only compressed, field-level summaries over cellular backhaul to cloud servers that schedule irrigation farm-wide.

3. **Wearable fitness tracking**: A battery-powered smartwatch must keep its always-on sensing path averaging under 1 mW to preserve battery life. The system employs hierarchical processing: lightweight integer kernels run on an embedded microcontroller to track real-time motion and detect heart-rate thresholds; intermediate biometric data synchronizes over Bluetooth Low Energy (BLE) to a paired smartphone, which uses mobile neural engines and gigabytes of DRAM to classify complex activity patterns; finally, the smartphone transmits periodic compressed summaries over Wi-Fi or cellular networks to cloud infrastructure for longitudinal health modeling.

Checkpoint 2.3: Hybrid ML patterns

Hybrid architectures work when partitioning work across tiers, not when copying the same pipeline everywhere.

Integration patterns

- ☐ **Train-serve split**: Can you explain why training in the cloud and serving on edge/mobile is often economically optimal, even when the model runs locally?
- ☐ **Hierarchical processing**: Can you describe what each tier does in a sensor → edge → cloud pipeline, and why pushing some decisions down reduces both latency and bandwidth?
- ☐ **Progressive deployment**: Can you explain why a shared task may require different model artifacts on different tiers, and which quality and resource measurements would establish their suitability?

Design sanity checks

- ☐ **Boundary choice**: Given a concrete application, can you justify where the tier boundary should fall (latency, privacy, bandwidth, power), not just what model to use?
- ☐ **State synchronization**: Can you identify which feature definitions, model versions, and preprocessing logic must remain synchronized across tiers?

2.11 What Every Tier Shares

Although the four deployment tiers differ by orders of magnitude in available energy, memory capacity, and compute throughput, they rest on shared systems foundations. Every tier moves data through a pipeline, manages a fixed budget of compute, memory, energy, and network capacity, and binds models to hardware through a software architecture. On those three shared foundations rest the same cross-cutting concerns of efficiency, monitoring, and trustworthy engineering (figure 2.13).

This convergence lets one set of techniques serve every tier, and every stage of a hybrid system, because optimizations that target the same physical bottlenecks transfer across hardware scales. Techniques that reduce off-chip transfers, the cost the memory wall imposes, help both an embedded microcontroller and a warehouse-scale accelerator. Lower-precision numerical representations (such as FP8, INT8, and INT4, which section 10.3 develops) cut the bytes stored and moved on local devices and in cloud clusters alike, and they raise arithmetic throughput only where the datapath has a matching low-precision path (section 11.4.5.1); prediction quality and complete memory traffic still determine the useful gain. Kernel fusion (executing a chain of operations in one pass so that intermediate results stay on chip, as section 7.3.2 explains) reduces off-chip traffic on mobile SoCs and HBM traffic during cloud autoregressive decoding. Graph partitioning, which splits a model's computation graph into pieces assigned to different processors, divides work across the CPU, GPU, and neural processing units of a mobile SoC just as it divides work across cloud workers.

The shared systems foundation extends through data engineering, model compression, and operational lifecycles. The same vulnerabilities therefore recur at every tier, namely data distribution

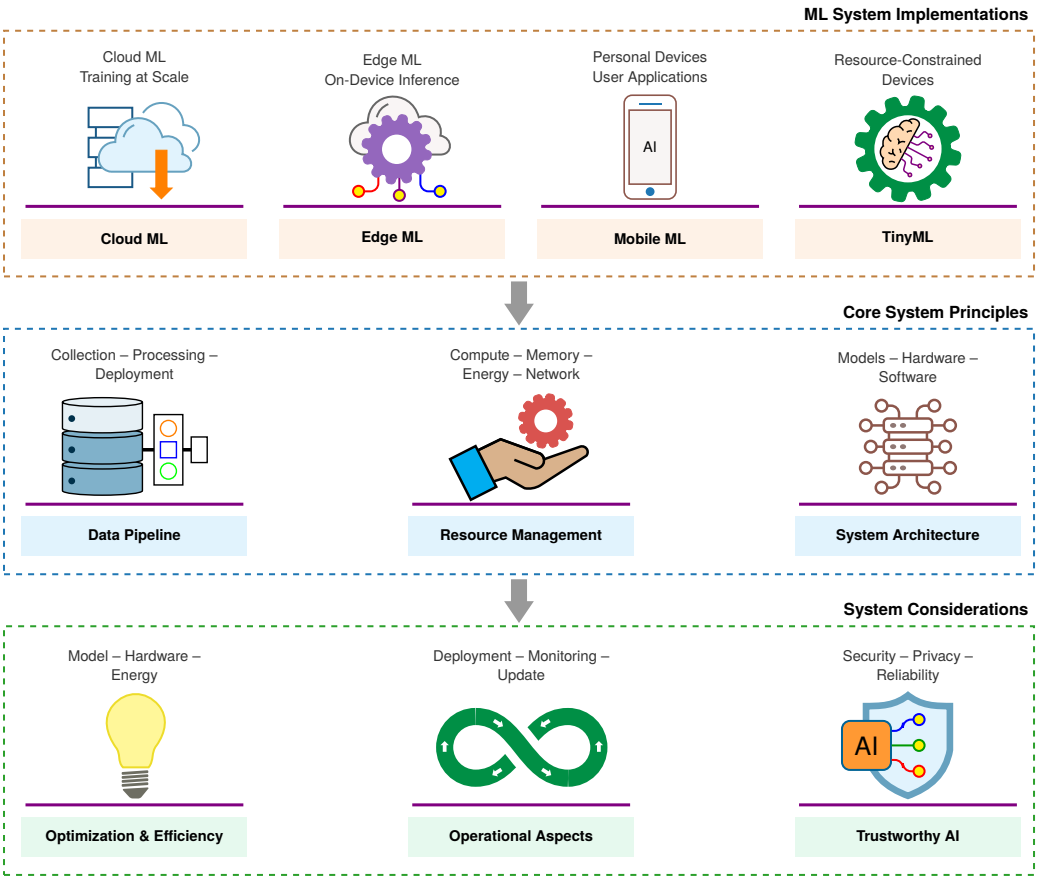


Figure 2.13: Convergence of ML Systems: Despite vast resource disparities across cloud, edge, mobile, and TinyML tiers, all deployments converge on three shared systems foundations: data pipelines, resource management, and hardware-software architecture. These shared pillars in turn support unified requirements for performance optimization, operational monitoring, and trustworthy engineering.

drift, model staleness, and telemetry degradation, and they call for uniform monitoring wherever the model runs. Of these, drift needs the closest watch, because it degrades a system whose code, weights, and hardware all execute correctly and so raises no infrastructure alarm. Accuracy falls with the divergence between training and operational data, the silent degradation that section 1.6 states with the degradation equation (equation 1.1), and chapter 14 develops the monitoring and retraining that respond to it.

2.12 Fallacies and Pitfalls

Most deployment misconceptions share one root. Each treats a physical envelope as a software detail that a later optimization can fix. Data movement, thermal dissipation, and hardware balance can invalidate otherwise reasonable optimization choices, and each fallacy and pitfall below records a system mismatched to its operating envelope.

Fallacy: *One deployment paradigm solves all ML problems.*

No single paradigm satisfies every physical constraint. Table 2.1 lists illustrative deployment envelopes rather than physics-only latency bounds. A real-time robotics system requiring sub-10 ms response cannot use a remote cloud path whose end-to-end network round-trip exceeds that budget, while a billion-parameter language model cannot reside within the few hundred kilobytes of SRAM on a microcontroller regardless of compression. Robust architectures frequently combine paradigms,

using cloud resources for distributed training and burst capacity while executing localized inference or sensor preprocessing on client devices.

A related misconception assumes that moving computation closer to the user always reduces latency, overlooking the execution delay introduced by thermally constrained or lower-bandwidth edge processors, a trade-off exposed when end-to-end latency is separated from model execution time, as in section 12.3.4.

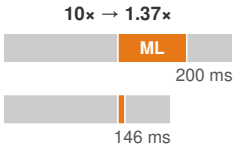
Pitfall: *Relying on model optimization to overcome mobile power and thermal limits.*

Compression techniques reduce power but cannot remove the thermal ceiling. Consider a smartphone equipped with a 15 Wh battery. A light inference workload drawing 1 W runs for $\frac{15\text{ Wh}}{1\text{ W}} = 15$ h, but an intensive workload drawing 5 W in this illustrative heavy-load scenario depletes that same battery in $\frac{15\text{ Wh}}{5\text{ W}} = 3$ h.

Sustained execution at 5 W also exceeds the approximately 3 W that a smartphone chassis can dissipate passively, so the device throttles as in notebook 2.9. Quantizing FP32 parameters to INT8 shrinks the memory footprint by an illustrative 4×, but switching energy and static leakage (the power a chip draws even while idle) do not fall in proportion, so a continuous pipeline above the thermal envelope still requires a lower duty cycle, a restructured architecture, or dedicated accelerator silicon.

Fallacy: *TinyML represents scaled-down mobile ML.*

The boundary between TinyML and mobile deployment changes the execution environment: much smaller working-memory and energy budgets require different scheduling and runtime choices. As section 2.8.1 establishes, microcontrollers provide 10^5 to 10^6 bytes of on-chip RAM compared to mobile platforms with 8–16 GB of LPDDR DRAM, a 10,000× capacity disparity that dictates memory residency. Where mobile systems execute models with tens of millions of parameters, many TinyML architectures use tens or hundreds of thousands of parameters, with low-precision arithmetic and layer scheduling that holds fewer intermediate results at once. Operating power budgets show an even sharper discontinuity: mobile inference draws 3–5 W, whereas TinyML targets an average power of < 1 mW to operate within energy-harvesting envelopes. Teams that apply mobile optimization techniques directly to TinyML environments discover that quantizing a trained model from FP32 to INT8 is insufficient on the smallest microcontrollers, where total working memory must fit inside as little as 64 KB of SRAM, forcing complete algorithmic redesign.



A faster model stage does not linearly speed up a camera pipeline.

Pitfall: *Minimizing computational resources minimizes total cost.*

Shaving dollars off per-inference compute in isolation often drives up operating bills and slows down shipping. As the decision framework in section 2.9.2 emphasizes, architectural selection requires evaluating TCO across the entire system lifecycle rather than raw server costs. In this scenario, a cloud inference service costing $2,000/month in compute appears expensive compared to $500/month in amortized edge hardware. However, a distributed edge fleet introduces recurring operational expenses: network engineering ($3,000/month), field hardware maintenance ($500/month), and fleet reliability engineering ($2,000/month). Together with hardware amortization, these bring the listed edge budget to $6,000/month, erasing the apparent hardware-only saving. The cloud figure includes compute alone, so the two totals do not establish a total-cost ranking; cloud operating costs and comparable service capacity remain outside this comparison. Unlike the single-server comparison in napkin math 2.3, this scenario operates a distributed fleet, where network, maintenance, and reliability labor scale with the number of sites. If a cloud-managed service reaches production in two months while custom edge infrastructure requires six, the four-month delay also carries an opportunity cost. Sound systems engineering balances compute efficiency against developer velocity, fleet manageability, and operational maintenance.

Fallacy: *Model optimization translates linearly to system speedup.*

Amdahl's law[34] caps end-to-end speedup by the share of time an optimization leaves untouched, the limit that the introduction's fallacies already showed for preprocessing (section 1.11); equation 11.1 states the bound for an accelerated path. Pipelining can overlap stages to raise steady-state throughput (section 2.3.1), but the stages of an interactive capture path run in strict serial order, so the cap applies in full. A smartphone camera spends 60 ms of its 200 ms capture path on ML scene

[34] **Amdahl's law**: Formalized by Amdahl (1967) for multiprocessor scaling, this principle applies directly to ML deployment pipelines where the model is only one stage among many. Where ML is 30 to 50 percent of runtime, even a 100× model speedup yields only about 1.4–2× end-to-end improvement, because the rest of the pipeline is unchanged. Teams that benchmark model latency in isolation systematically overestimate deployment gains.

classification, which runs between 100 ms of image signal processing and 40 ms of tone mapping and HDR merge. A 10× faster model (6 ms of classification) shortens the path only to 146 ms, an end-to-end gain of 1.37×, and even an infinitely fast model cannot exceed 1.43× while the other 70 percent of the path is unchanged. Optimizing an ML system requires profiling the full execution graph; accelerating an isolated stage yields diminishing returns once that stage ceases to be the critical-path bottleneck.

Pitfall: *Assuming more training data always improves deployed model performance.*

Three constraints bound the returns of data scaling. First, algorithmic capacity (A) saturates: when a model's parameter budget is fixed, additional training data cannot squeeze more patterns into a model whose layers are too small to represent them. Second, data quality (D) matters more than raw volume: mislabeled examples, uncalibrated sensors, and spurious shortcuts corrupt gradient updates, so scaling up a noisy dataset only teaches the model its errors with higher confidence. Third, the deployment tier decides which data counts as in-domain. A microcontroller's microphone, a phone camera, and a data-center log stream capture inputs with different noise, resolution, and sampling, so a large corpus gathered under one tier's conditions can match another tier's inputs poorly. High-fidelity data collected under the target tier's own sensing conditions often yields higher serving accuracy than a much larger out-of-domain corpus.

Fallacy: *One model binary can serve every edge hardware target efficiently.*

Teams frequently compile a single model artifact and deploy it uniformly across diverse devices, treating deployment as a packaging step rather than a hardware co-design problem. In practice, hardware-specific compilation and kernel lowering (translating each operation into code tuned for the target's instruction set and memory layout) capture critical efficiencies that generic binaries forfeit. For example, quantized INT8 operations on a dedicated neural processing unit (NPU) achieve substantially higher throughput per watt than the identical model on a general-purpose CPU, and kernel fusion keeps intermediate activation tensors in on-chip SRAM rather than evicting them to energy-expensive off-chip DRAM. As the hardware analysis in section 2.4 establishes, hardware architectures impose divergent execution constraints: a binary compiled for a general-purpose mobile CPU does not use the specialized matrix engines of an embedded NPU. A portable model artifact can run through multiple backends, but portability alone does not establish efficiency. Production deployment requires profiling the target runtime and hardware, then selecting compilation, kernels, and memory layouts that meet its resource and thermal limits.

2.13 Summary

The same model demands different engineering on a phone and in a data center because each deployment target offers the model different hardware terms. The iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$) unifies these physical limits into a single execution model. Propagation delay contributes fixed latency (L_{lat}), the memory wall bounds data movement (D_{vol}/BW), and thermal power budgets cap sustained compute.

By applying the bottleneck principle to workload archetypes, engineers determine whether a model's latency is dominated by compute efficiency, memory bandwidth, or network transit before committing to single-node or hybrid architectures. The quantitative trade-offs summarized in table 2.9 and the tiered screening sequence formalized in figure 2.11 establish the physical baselines for later decisions in model compression, data pipeline design, and high-performance serving.

Key Takeaways: Same model, different engineering

- **Physical constraints define feasibility**: Propagation delay sets a lower bound on remote response time (about 36 ms for a cross-country round trip). Power and memory budgets limit what a platform can sustain, so a feasible deployment must meet the task's deadline within the target platform's resource limits.
- **Identify bottlenecks before optimizing**: The same model can be compute bound in batched training yet bandwidth-bound at batch-1 inference. The iron law and bottleneck

principle pinpoint which constraint dominates; shortening a nonlimiting stage may reduce request latency without increasing steady-state throughput.

- **Match workload demands to deployment budgets**: A compute-bound workload such as large-scale image-model training often needs cloud-scale resources; a power-bound workload such as keyword spotting benefits from microcontroller efficiency. Engineers must identify the binding constraint under the intended execution conditions rather than choose a deployment from the model name alone.
- **Screen feasibility before comparing cost**: Privacy, latency, and resource gates remove infeasible tiers before cost is compared. Among the survivors, utilization, provisioning, and fleet maintenance, not resource savings alone, decide which architecture is cheaper.
- **Hybrid architectures divide work by constraint**: The voice-assistant example assigns wake-word detection to TinyML, speech-to-text to mobile, and language understanding to cloud. Train-serve split, hierarchical processing, and progressive deployment provide three patterns for combining tiers; the choice depends on each stage's requirements.
- **Deployment begins a feedback loop**: Live data can diverge from training conditions while code and weights stay fixed, so monitoring and bounded operational exposure are part of the system architecture.

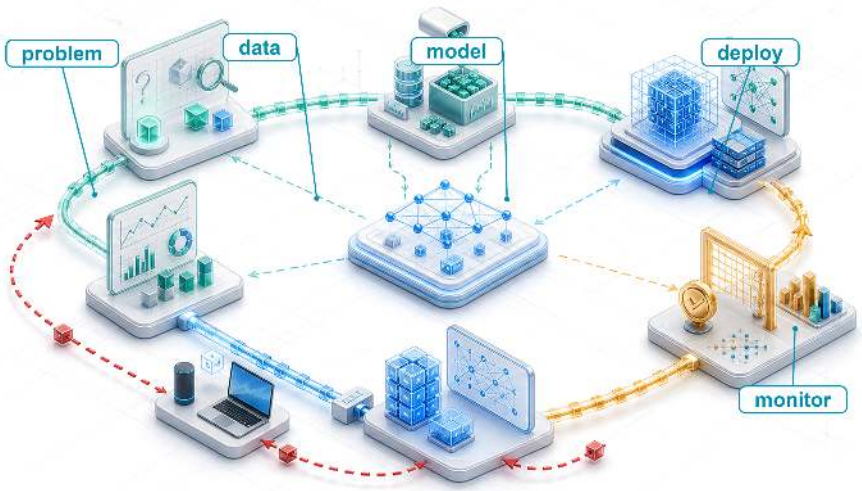
Deployment paradigms are not product categories. They are operating regimes shaped by physical and operational constraints, and selecting one changes which constraint is most likely to bind. Data-center training can push against peak compute (O/R_{peak}). Mobile inference is bounded by energy, with the battery setting the budget and the chassis capping the rate at which it can be spent. Remote real-time inference can be constrained by propagation delay (L_{lat}). Systems engineering begins by measuring that binding constraint, and the measurement is worth most when it comes early. Optimizing model architecture or software runtimes without first identifying the binding constraint produces systems that fail when they meet real operating conditions.

What's Next: From theory to process

The decision framework in section 2.9.2 identifies where an ML system can execute, but the team must use those constraints throughout development. Knowing where a model can run helps little if the team learns it too late, because constraints discovered at launch invalidate months of earlier decisions. Chapter 3 traces exactly that failure, a deployment memory budget discovered only after five months of work, and shows how such constraints must travel backward to the first design meeting. It introduces the machine learning lifecycle, the iterative engineering process that carries a system from problem definition through deployment, monitoring, and maintenance, and the workflow that carries constraints around it.

3

ML Lifecycle

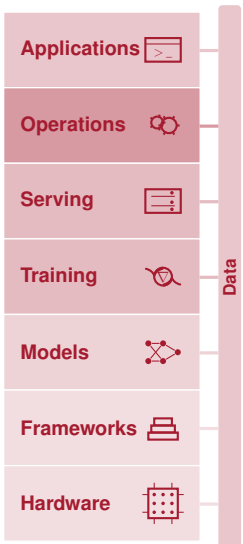


3.1 The Lifecycle as a Loop
3.2 How Decisions Cascade: The Constraint Propagation Principle
3.3 Where the Hours Go: Quantifying the ML Lifecycle
3.4 Stage Ownership and Hand-offs
3.5 Stage 1: Problem Definition
3.6 Stage 2: Data Collection and Preparation
3.7 Stage 3: Model Development and Training
3.8 Stage 4: Evaluation and Validation
3.9 Stage 5: Deployment and Integration
3.10 Stage 6: Monitoring and Maintenance
3.11 Systems Thinking Across the Lifecycle
3.12 Fallacies and Pitfalls
3.13 Summary

Purpose

Why is seeing the whole map necessary before walking any single path?

Engineering organizations often split an ML system into three specialties, with one team curating datasets, another tuning model architectures, and a third provisioning serving hardware. The components those teams own remain coupled. The volume and quality of collected data bound which algorithms are viable, the computational structure of the chosen algorithm dictates what hardware can execute it efficiently, and the memory and bandwidth of the target hardware reshape what preprocessing can run at inference. Optimizing each in isolation tends to produce accurate models that cannot be deployed and efficient data pipelines that serve the wrong distributions, and the cost of such a mismatch compounds with how late it surfaces. A memory limit discovered during initial design changes one line of a hardware specification; the same limit discovered on the eve of deployment discards months of training runs, annotation budgets, and integration work. Before analyzing any single component, an engineer needs the full map of how an ML system is specified, trained, validated, deployed, and sustained as one operational loop. In D·A·M terms, the ML lifecycle is where data, algorithm, and machine are co-designed over time, each stage passing constraints to the others until their combined capability meets real-world requirements.



Learning Objectives

- Explain the six ML lifecycle stages as coordinated data-algorithm-machine decisions with feedback
- Compare ML workflows with traditional software using drift, nondeterminism, and operational feedback
- Construct a problem definition that states a request as a measurable improvement over an existing baseline
- Analyze how problem-definition constraints propagate through data, modeling, validation, and deployment
- Estimate rework from a late-discovered constraint by tracing dependent data, model, validation, and deployment artifacts
- Evaluate accuracy, efficiency, reproducibility, and deployment readiness trade-offs across lifecycle stages
- Design feedback loops that connect monitoring signals to retraining, maintenance, and revised requirements

3.1 The Lifecycle as a Loop

Building an ML system takes far more than training a model. A team must decide what the system has to do and within which limits, gather and prepare the data it learns from, train and test the model, install it where it will run, and keep checking that it still works as the world around it changes. Chapter 2 examined one of those limits in depth, showing that the deployment target can change the engineering of the same model by orders of magnitude between a phone and a data center. The rest of this book takes the remaining work apart piece by piece along the data, algorithm, and machine axes, from data pipelines and model architectures through training, optimization, and serving. Those pieces are not independent. Each hands its results to the next, and a mistake made in one tends to surface in another, so understanding any single piece first requires understanding how they connect: the sequence of stages every ML system moves through, what each stage hands to the next, and how a constraint discovered in one place reaches the others.

That sequence is the **machine learning lifecycle**, a structured, iterative process[1] that guides the development, evaluation, and improvement of ML systems (Amershi et al. 2019). Numbered in the order a project first meets them, its six stages form a chain in which each stage produces the artifact the next one needs: problem definition (1) turns a request into measurable targets and deployment constraints, which guide data collection and preparation (2) as it turns raw observations into a versioned, validated dataset. Model development and training (3) then fits trained weights to that dataset, and evaluation and validation (4) tests those weights against the original targets and records the release evidence in a release manifest. Once the manifest clears the gate, deployment and integration (5) installs the approved model where it must run, while monitoring and maintenance (6) watches the live system and sends what it learns back to earlier stages, closing the loop drawn in figure 3.1.

The return arrow from monitoring is what makes the lifecycle a loop rather than a checklist. Evidence flows back from later stages to earlier ones, and constraints should flow back with it. A memory limit set by the deployment device belongs in problem definition, where it can shape the dataset, the model, and the evaluation, not in deployment, where it can only reject them.

[1] **CRISP-DM (cross-industry standard process for data mining)**: CRISP-DM codified data-intensive system development as six interconnected, iterative phases rather than a linear waterfall (Chapman et al. 2000); its core design principle (moving back and forth between phases as results require) directly informs the modern ML lifecycle's structure.

Definition 3.1: Machine learning lifecycle

Machine learning lifecycle is the iterative engineering process of building, deploying, monitoring, and updating ML systems, where evidence from later stages can feed back to earlier stages because model performance may change after deployment.

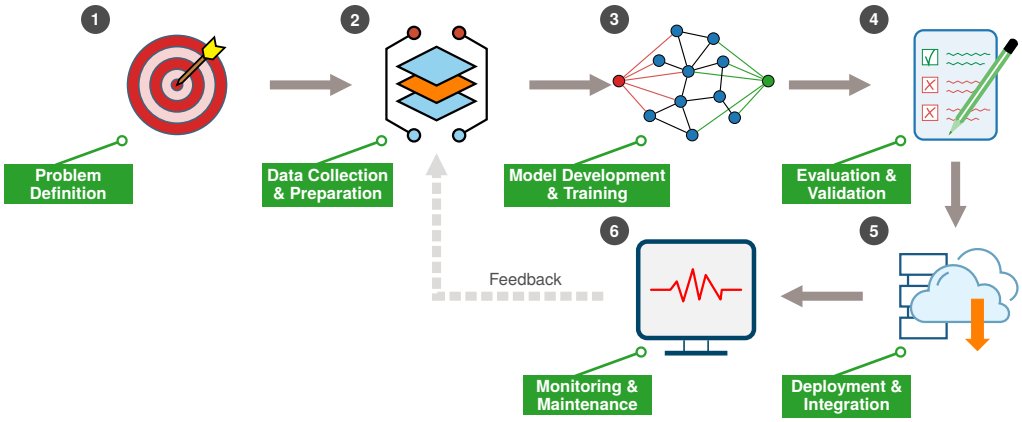


Figure 3.1: Simplified Lifecycle with Feedback: The six core stages of ML engineering form a closed loop rather than a linear handoff. A primary feedback path from monitoring back to data collection supports investigation and adaptation as production distributions shift, model accuracy degrades, and operational constraints evolve.

1. **Significance**: The lifecycle is a closed loop, not a linear pipeline. Each stage hands the next an artifact it depends on (targets, a dataset, trained weights, release evidence, a running service, monitoring evidence), so a decision at one stage constrains every stage after it, and evidence from a later stage can force an earlier one to be revisited. How often the loop must turn is itself a recurring budget, set by how quickly the deployment environment changes and how long verified labels take to arrive.
2. **Distinction**: Unlike traditional software, whose behavior changes when code, configuration, dependencies, or environment changes, ML behavior can also change as the world changes. A deployed model's accuracy may change through distribution shift even when the code, infrastructure, and configuration remain untouched.
3. **Common pitfall**: Treating deployment as the end of the lifecycle. Retraining decisions depend on outcome evidence that exists only after deployment.

Carrying constraints around that loop is the job of the **ML workflow**, the engineering framework that makes constraints explicit at each stage and traces how deployment constraints propagate across Data, Algorithm, and Machine. The two terms divide the job. The lifecycle is *what gets traversed*, the stages themselves; the workflow is *how the traversal is managed*, the engineering discipline of orchestrating them. The workflow marks the shift from model researcher to systems engineer. A researcher optimizes individual elements: a better architecture, a cleaner dataset, a faster accelerator. A systems engineer orchestrates those elements into production systems that reliably deliver value.

In practice, the stages between problem definition and deployment run as two pipelines on different clocks. Figure 3.2 zooms into that work as ten finer-grained blocks. The upper data pipeline, the substance of stage 2, continuously transforms raw operational streams through ingestion, curation, annotation, and validation into versioned, ML-ready datasets. The lower model pipeline executes the inner development loop of stages 3 and 4: training architectures, computing evaluation metrics, validating system constraints, and packaging approved artifacts for deployment, with stage 4's release manifest drawn as an ML certificate. Outer-loop feedback arrows carry production telemetry and validation discoveries back to upstream data stages, the return path that stage 6 operates.

↩ The deployment paradigms whose memory, latency, and power constraints the workflow propagates are established in section 2.1.

↪ The operational tooling and infrastructure that automate these lifecycle stages at scale are developed in chapter 14.

3.1.1 ML vs. traditional software

The loop and its two pipelines exist because of one difference between ML systems and traditional software. Both iterate.[2] The difference that forces new workflow controls is where behavior comes from. Rule-based software changes behavior when someone changes the code, configuration, or dependencies. An ML system learns its behavior from data, so that behavior can also change when the data changes while the code stays fixed, and only fresh labeled outcomes can show whether

2 | **Waterfall model**: A plan-driven lifecycle is often depicted as a sequence of requirements, implementation, and testing, but Royce (1970) warned that a purely sequential implementation was risky and described iteration between successive phases. ML systems add data and model feedback that must be managed explicitly.

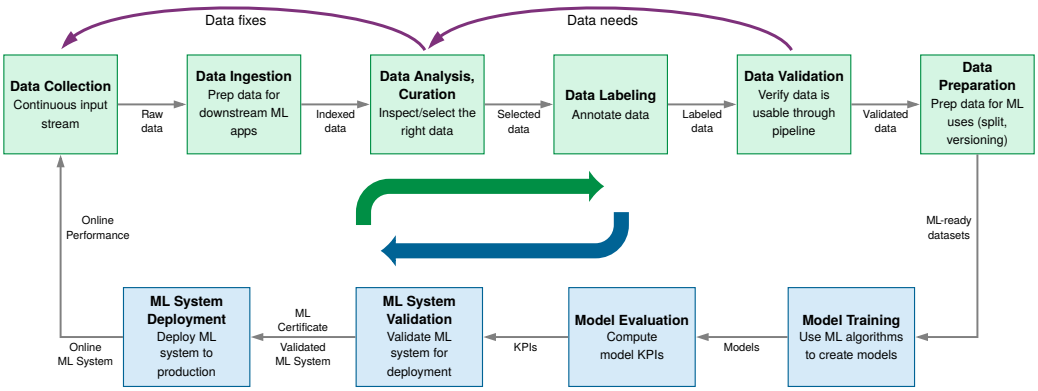


Figure 3.2: Dual-Pipeline ML Development: Machine learning engineering decouples into parallel data and model pipelines operating on different timescales. While the model pipeline executes rapid inner-loop training and evaluation, outer-loop feedback from production deployment and data validation continuously reshapes upstream collection and curation.

today's model still meets today's requirement. Every workflow addition this chapter introduces (versioned datasets,[3] statistical evaluation gates, drift monitoring) exists to manage that second path.

That difference alters how lifecycle stages interact. While conventional software engineering relies on production feedback to refine code and requirements, ML systems introduce feedback loops where operational data reshapes training distributions, detected drift prompts investigation, and runtime errors expose dataset blind spots. Table 3.1 contrasts traditional software and ML engineering across six core lifecycle dimensions, highlighting how data evolution redefines testing, deployment, and ongoing maintenance.

[3] **Data versioning**: Unlike code, which changes through discrete, auditable commits, data can drift gradually (distribution shift), suddenly (schema migration), or subtly (label quality degradation). Plain Git is impractical for many multi-terabyte datasets, so teams commonly version external-data manifests or pointers using tools such as DVC and Git LFS. Without data versioning, teams cannot reliably reproduce a prior training run or determine whether an accuracy regression stems from a code change or a data change.

↩ Data-defined behavior, and the silent degradation it makes possible, are established in section 1.6.

Table 3.1: Traditional Software vs. ML Development Lifecycles: Six dimensions where ML development diverges from traditional software engineering. Both use production feedback, while ML systems add loops in which deployment insights reshape data collection, monitoring drives model updates, and production experience informs model design.

Aspect	Traditional Software Lifecycles	Machine Learning Lifecycles
Problem Definition	Functional requirements specify behavior; tests check the code against them.	Objectives are statistical targets whose feasibility is known only after experiments on data.
Development Process	Engineers write and revise the logic directly in code and configuration.	Engineers shape behavior indirectly through data, features, architecture, and training runs.
Testing and Validation	Many functional tests have exact expected results; performance and reliability remain quantitative.	Metrics over held-out data, reported with uncertainty and across subgroups.
Deployment	Logic stays fixed until code, configuration, or dependencies change.	Accuracy can degrade when input distributions shift, with code unchanged.
Maintenance	Maintenance involves modifying code to address bugs or add features.	Continuous monitoring, updating data pipelines, retraining models, and adapting to new data distributions.
Feedback Loops	Production feedback changes code, configuration, and requirements.	Production data can become future training data, so feedback also reshapes data collection and the model.

Checkpoint 3.1: ML vs. traditional software

ML systems are not traditional software with a model attached. Check the differences that force a separate workflow:

- ☐ **Failure modes**: Can you explain how a shift in input data can degrade an ML system silently, with no code change and no error raised?
- ☐ **Logic source**: Can you explain why changing training data can change model behavior even when the code remains fixed?

- ☐ **Iteration**: Can you explain why ML requires monitoring and, when evidence warrants it, model or data updates?

Each difference in the comparison traces back to behavior learned from data, which ties every stage to the stages around it. A diagram shows where the stages sit but not how tightly they bind. Seeing that requires a concrete system, and this chapter carries one through every stage.

3.1.2 The running case: Diabetic retinopathy screening

Diabetic retinopathy (DR) is a major cause of preventable vision loss.[4] While early detection prevents severe vision loss, limited access to specialist ophthalmologists in rural regions motivates automated screening programs. Google built one of the best documented of these programs, a deep-learning model that performed at the level of ophthalmologists on curated retinal images (Gulshan et al. 2016) and that Google Health then deployed in clinics in Thailand (Beede et al. 2020). Detecting the pathology, however, requires identifying subtle microvascular features under variable capture conditions. Figure 3.3 contrasts a normal retina (Panel A) with one showing diabetic retinopathy (Panel B).

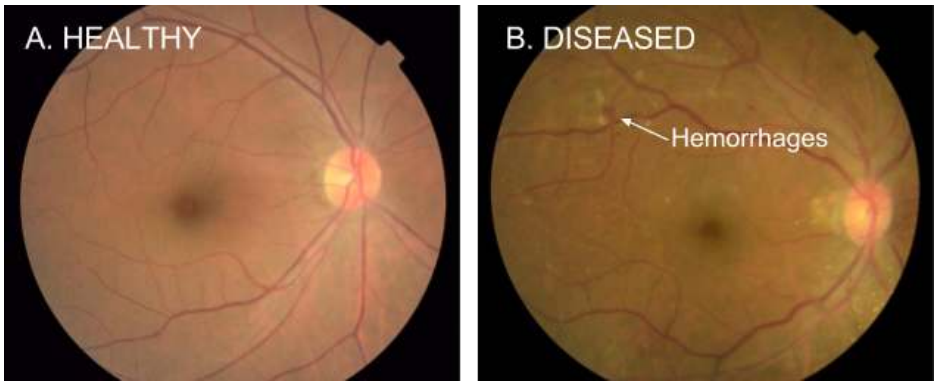


Figure 3.3: Retinal Hemorrhages in DR Screening: Side-by-side fundus images illustrate the subtle visual features an ML screening system must detect. Identifying localized microvascular lesions (such as the indicated hemorrhage) requires preserving fine spatial resolution through the vision pipeline and establishing strict input-quality validation to handle lighting and camera variations in clinical field deployments. Source: (Peng and Gulshan 2016).

This chapter follows diabetic retinopathy screening because it is a real production system whose documentation covers the full lifecycle, and because a task that looks straightforward on the surface met substantial operational friction in practice. Google's program spans both sides of the lab-to-field divide. Its controlled validation documented expert-level accuracy, while its deployment in Thai clinics exposed problems the validation never saw: images rejected as ungradable, retakes that added work for nurses, and screening delayed when slow clinic internet connections held up uploads to the remote grading service.[5]

The scenario this chapter uses to put numbers on each stage is a composite based on that real case study. Because the remote grading path depended on each clinic's internet connection, the scenario moves inference into the clinic: a network of rural clinics, each executing local inference on an Orin Nano-class edge device with 4 to 8 GB of shared memory and an intermittent wide-area uplink of roughly 2 Mbps. The clinical mandate requires more than 90 percent sensitivity and a result the nurse can act on before the patient leaves, which means within seconds of capture and even when the uplink is down. Sensitivity measures the fraction of true disease cases correctly flagged; its denominator includes only patients with the condition, meaning high aggregate accuracy on an imbalanced, mostly healthy population provides no assurance that sick patients will be detected. Specificity measures the fraction of healthy patients correctly left unflagged, bounding unnecessary referrals that would overwhelm regional specialists.

A team that runs these stages as a line rather than a loop fails in a predictable way. Day one: "Build a diabetic retinopathy screening model for rural clinics." Day 90: 92 percent sensitivity on

[4] **Diabetic retinopathy (DR)**: DR is a common complication of diabetes; early detection can prevent vision loss, but specialist access varies substantially across countries. This gap motivates scalable screening programs. In clinics with limited hardware or unreliable connectivity, model size, offline capability, latency, and workflow integration can become binding deployment constraints.

[5] **Healthcare AI deployment gap**: In the Thai DR deployment studied by Beede et al. (2020), 393 of 1,838 submitted images (21 percent) failed the system's image-quality requirements, often because clinic lighting, camera maintenance, or dilation practices did not match its assumptions. The case shows why laboratory metrics alone do not establish deployment readiness.

the validation set used to guide development. Day 120: 94 percent sensitivity after a month of architecture tuning. Day 150: the model is handed to deployment engineers. Day 151: deployment engineers report that the model needs 4 GB of memory to run. Day 152: someone checks the deployment target, a low-power edge device in each clinic whose 4–8 GB of memory is shared with the operating system, the camera pipeline, and the clinic software; the clinic's IT plan reserves about 500 MB of it for the model. Day 153: five months of work is discarded.

The model's accuracy was excellent. The team's machine learning skills were excellent. *The failure was a workflow failure.* A deployment constraint that should have shaped every decision from day one was discovered only after the work was done. That 500 MB budget should have propagated backward to the first architecture meeting, constraining which models were even worth considering. Instead, the team optimized each component in isolation (data collection, architecture selection, training), and the integration failure appeared only when the pieces were assembled. The day-153 failure is one instance of a general principle about how constraints travel through the lifecycle.

3.2 How Decisions Cascade: The Constraint Propagation Principle

In the DR scenario, the in-visit requirement already ruled out cloud inference and so required edge deployment. The edge target's memory and power budgets then constrain model architecture, favoring compact MobileNet-class models over large ensembles (several models whose predictions are combined) and requiring automated image-quality checks at capture time. Each downstream constraint narrows the design space for upstream stages, coupling hardware limits directly to data collection and model design.

Definition 3.2: Constraint propagation principle

The constraint propagation principle states that a constraint discovered late in the ML lifecycle can force rework across affected upstream stages. Datasets, trained weights, validation evidence, and deployment packages record decisions that later work depends on. Correcting a missing constraint requires tracing those dependencies to determine which artifacts must change and which remain valid. Estimating actual rework requires the effort to revise each affected artifact, repeat its dependent work, and validate the resulting system.

1. **Significance**: A latency target discovered at deployment (stage 5), such as 100 ms in a web service, may propagate backward to constrain model size (stage 3: operation count O), dataset requirements (stage 2: dataset size D, the number of training examples), and problem definition (stage 1: what accuracy is achievable). Rework depends on which earlier decisions relied on the missing constraint. Within the iron law, a latency budget bounds total time T, and therefore every term that sums to it, so it redefines the feasible region for O, for the bytes moved D_{vol} (distinct from the example count D), and for the hardware efficiency η_{hw} the model must achieve.
2. **Distinction**: Unlike modular decomposition (which encourages independent optimization of each component), this principle mandates end-to-end reasoning, because an accuracy optimum reached in isolation can be an infeasible system.
3. **Common pitfall**: Treating deployment as "the last step" and so deferring its latency, memory, and power limits until they can only reject finished work.

Propagation runs in both directions, so the stages form a network of constraints rather than a chain. Backward propagation carries a deployment limit upstream. When rural clinic deployment reveals tight bandwidth, the team can compress images before sending them over the uplink, but lossy compression degrades input quality, so the training dataset must then include matching compression artifacts or the model will see different inputs in service than in training (training-serving skew). Shifting inference entirely to the edge and transmitting only the diagnostic result relieves the uplink instead, but tightens the edge device's memory capacity and power budget. Forward propagation carries an early data decision downstream. Capturing fundus images at the resolution needed to see small hemorrhages fixes the model's input size, and with it the operation count every inference

must pay on the clinic device. Upstream data choices and downstream machine boundaries remain physically coupled.

Decisions made without accounting for downstream execution boundaries accumulate technical debt.[6] The stage interface specification (table 3.2) puts this principle to work by making constraints explicit at each stage boundary, so the team catches violations before handing an artifact downstream. When propagation occurs specifically through data quality failures, the resulting pattern is known as a data cascade, a chain of downstream failures triggered by bad data (Sambasivan et al. 2021) that section 4.2.1 examines. Propagation explains why a late constraint is expensive; how much time each loop costs, and where that time goes, is the next question.

[6] **ML technical debt**: Sculley et al. (2015) identify ML-specific debt mechanisms such as *entanglement* (changing one feature affects all others because the model learned joint distributions), *hidden feedback loops* (predictions influence future training data), and *undeclared consumers* (downstream systems depending on outputs without contracts). Since ML code is often only a small part of a production system, the surrounding configuration, pipelines, and infrastructure can allow this debt to accumulate silently.

↪ Shared contracts and deployment paths that reject incompatible changes before they reach production are enforced in section 14.3.

3.3 Where the Hours Go: Quantifying the ML Lifecycle

The DR team lost five months to one late constraint, counting both the hours already spent and the experiments those hours could have run. Conceptual models of the ML lifecycle describe which stages exist, but planning a workflow requires knowing where its hours go, and that time rarely concentrates where research literature spends its pages.

Practitioner surveys show a gap between where research attention centers and where practitioners report spending their time, with about 79 percent of data scientists in CrowdFlower's 2016 survey naming data collection or cleaning as the activity that consumed most of their time (CrowdFlower 2016). In the full distribution (figure 3.4), algorithm refinement and training-set construction together draw fewer responses than data collection alone.

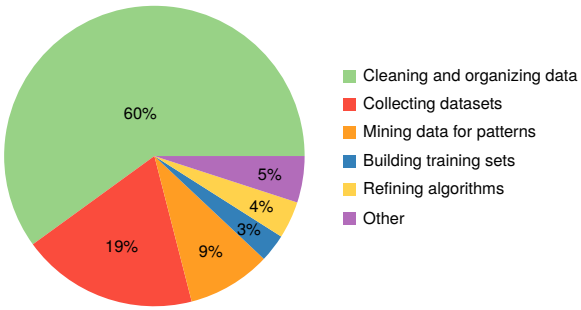
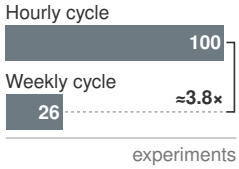


Figure 3.4: Data Scientist Most-Time Responses: In CrowdFlower's 2016 survey, 60 percent of respondents selected cleaning and organizing data as the activity where they spent the most time, with 19 percent selecting data collection. Model-focused responses such as pattern mining, training set construction, and algorithm refinement together represent roughly 16 percent of responses. Adapted from CrowdFlower (2016).

The survey responses identify data work as a major time sink for these practitioners, without measuring how all engineering hours were divided. In the DR project, where each image must be graded by multiple ophthalmologists before it can train anything, collection and labeling can delay useful experiments more than training does; improving that pipeline can then reduce iteration time before model or training optimizations become the binding concern. The workflow must identify its actual bottleneck rather than assign investment priorities from response shares alone. Even so, the imbalance explains why data engineering, developed in chapter 4, deserves explicit planning rather than treatment as setup work.

Iteration velocity governs project progress. The feedback arrows in figure 3.2 (the *Data fixes* loop from curation back to collection, the *Data needs* loop from validation back to curation, and the *Online Performance* path from production deployment back to collection) are not rare exceptions; they are paths teams traverse repeatedly. Drifting clinic images or wrong labels send the DR team back to collection, and a candidate that exceeds the clinic device's memory sends it back to model design. Each loop consumes calendar time and compute budget, so the length of one cycle bounds how much a team can learn. Consider two candidate DR models developed over the same six months: a large ensemble trained in the cloud, which takes a week per training run, and a lightweight edge model that trains in hours.



Over the same six months, the hourly cycle tests nearly four times as many ideas.

Napkin Math 3.1: The iteration tax

Problem: Compare what six months of development buys for a rural diabetic retinopathy screening system: a large ensemble developed on cloud hardware as a laboratory baseline, which would itself need compression before it could fit the clinic's memory budget (training time: 1 week, initial accuracy: 95 percent), or a lightweight edge model (training time: 1 hour, initial accuracy: 90 percent). How do the two compare in final accuracy and in the number of hypotheses the team can test?

Math: In six months (~26 weeks), the possible experiment count is:

1. Large model: 26 weeks of calendar time at 1 week per experiment. Each experiment improves accuracy by an assumed constant ~0.15 percentage points.
2. Small model: 26 weeks of calendar time at 168 h/week gives 4,368 possible experiments at 1 hour each. Machine time is not the binding constraint at that cycle length, so this scenario counts only 100 of them as effective experiments, a few per working week, the pace at which a team can design, run, and learn from experiments. Because the slow-cycle team runs one experiment per week, this cap on hypotheses, not the hardware, sets the breadth ratio below; a team that could formulate useful hypotheses faster would raise the ratio in proportion.

Result: If each iteration improves accuracy by 0.1 percentage points on average, the small model starts at 90 percent and reaches 100 percent after 100 effective iterations, before applying the 99 percent ceiling. With the 99 percent ceiling applied, the small model achieves 99 percent. The large model starts at 95 percent and reaches 98.9 percent after 26 slower iterations. Both approaches end near the ceiling, and the accuracy gap between them is an artifact of the assumed gains, not a finding. The difference that survives is breadth. Under the stated cap, the fast-cycle team tested 3.8× as many ideas, so it also learned which augmentations and preprocessing choices matter, knowledge the slow-cycle team never acquired.

Systems insight: *Iteration velocity is a feature.* Its value lies in search breadth rather than in a guaranteed accuracy lead. When candidate systems have comparable attainable quality, shorter cycles expand how much of the design space a team can test. Speed alone cannot guarantee that a smaller model will outperform a larger one. In the DR screening scenario, the lightweight model's rapid iteration cycle enables the team to experiment with label-preserving input transformations, preprocessing pipelines, and architecture variations far more quickly.

This cumulative delay defines the **iteration tax**: time spent waiting on slow training runs or cumbersome validation pipelines reduces how many candidate models a team can test. Late discoveries amplify this penalty,[7] as the constraint propagation principle (section 3.2) established. Violations caught only after deployment invalidate artifacts produced by earlier stages, forcing expensive retraining, pipeline adjustments, and renewed validation. Intercepting defects before they cascade into systemic rework requires explicit **stage interface contracts** that validate outputs at every stage boundary (section 3.4.1).

[7] **Late correction costs**: Boehm (1981) discussed in *Software Engineering Economics* how defects found late can cost more to fix than those caught during requirements. In ML systems, late-discovered constraints may require retraining, data-pipeline changes, and renewed validation.

3.4 Stage Ownership and Hand-offs

The rural-clinic failure showed what happens when no stage owns a constraint. The deployment limit surfaced only after data curation, model training, and evaluation decisions had hardened around an unfeasible target, and preventing such misalignments requires clear ownership across the system stack. The six stages of figure 3.1 divide project responsibilities along the D·A·M axes. Problem definition sets the governing requirements for all three. Data collection and preparation owns the Data axis, model development and training governs the Algorithm axis, and deployment and integration commits both to a physical Machine. Evaluation and monitoring own no single axis of their own; their role is verification, testing whether the three axes work together before release and continuously in production.

A single model choice in the DR project touches every stage. Problem definition fixed the clinic device's 500 MB model budget, so model development starts from architectures designed for budgets

of that kind. MobileNetV2, the book's mobile lighthouse workload (section 1.7.1), is a natural candidate backbone, at about 14 MB of FP32 weights and about 600 MFLOP per inference. It reaches that operation count by replacing standard image-processing layers with a cheaper variant[8] rather than by pruning a larger model. Choosing it immediately constrains the other stages. Data preparation must produce images at the capture resolution that keeps small hemorrhages visible, since that resolution rather than the backbone's default sets the input size, evaluation must measure latency on the clinic device as well as sensitivity, and deployment must confirm that weights, activations, and runtime together fit the budget. If on-device evaluation then misses the latency target, the fix loops back to architecture or input resolution, so the stage sequence in the diagram is traversed as a cycle rather than once.

8 **Depthwise separable convolutions**: The layer type MobileNetV2 uses in place of standard convolutions. It splits one expensive operation into two cheaper ones and cuts computation by roughly 8–9× for typical kernel sizes (Sandler et al. 2018), which is what makes the roughly 600 MFLOP inference budget plausible on mobile-class hardware.

Checkpoint 3.2: The workflow cycle

Verify that each stage enforces contracts that bind upstream choices to physical deployment constraints.

- ☐ **Problem definition**: Can you identify which stage records latency, memory, and accuracy targets, and explain why later stages need those targets?
- ☐ **Data preparation**: Can you identify which downstream artifacts must use the same image-resolution and preprocessing choices as the training data?
- ☐ **Model development**: Can you explain why model development needs a deployment memory budget before committing to a candidate?
- ☐ **Feedback**: Can you explain why a failed on-device latency test sends work back to architecture or input resolution rather than ending at deployment?

Each of those stage decisions also lands on a term of the iron law. In the retinal screening pipeline, the capture resolution chosen during data preparation sets the bytes moved per image (D_{vol}), the MobileNetV2 backbone selected during model development fixes the operations per frame (FLOPs/Sample), and the clinic hardware together with its serving runtime determines the realized execution speed (Seconds/FLOP) and dispatch overhead (L_{lat}). Tracing how the six stages map onto the equation shows where each team's local choices enter the end-to-end time budget.

↩ The iron law of ML systems and its governing terms are established in section 1.7.

Systems Perspective 3.1: Lifecycle stages and the iron law

The six lifecycle stages alter distinct factors in the iron law of ML systems, both across its multiplicative D·A·M form (equation 1.2) and across its unpacked physical terms ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). Rather than rigid one-to-one assignments, the mappings serve as diagnostic starting points:

- Problem definition: Sets target constraints for accuracy, latency, cost, privacy, and deployment paradigm (screened across privacy, latency, compute, and cost gates using the decision framework in figure 2.11). These targets determine which terms of the equation are permitted to grow and which must be bounded from the outset.
- Data collection and preparation: Shapes dataset composition, the sample budget (Samples/Task), and the byte volume (D_{vol}) presented to downstream pipelines. Curation can also change the training work required to reach a quality target.
- Model development and training: Dictates per-sample algorithmic work (FLOPs/Sample) and total operations (O), along with data reuse patterns, parameter movement, and achievable hardware efficiency (η_{hw}).
- Evaluation and validation: Tests whether model quality and end-to-end execution meet deployment requirements on the target system.

- Deployment and integration: Determines the serving path and realized machine speed (Seconds/FLOP), including runtime data movement, execution efficiency, and fixed overhead (L_{lat}).
- Monitoring and maintenance: Observes distribution drift, latency, throughput, and operational cost after launch, feeding violations back into earlier stages for re-optimization.

Viewing the lifecycle through the iron law grounds latency and cost in hardware physics. Production workflows must simultaneously satisfy accuracy, safety, privacy, and organizational constraints that sit outside the equation.

3.4.1 Stage interface contracts

If the data team hands over 512×512 retinal crops while the training script expects 1024×1024 images, or if problem definition hands off to data collection without recording the clinic device's 500 MB memory limit, the defect travels silently until a downstream stage crashes or rejects the finished model. Catching those mismatches at the hand-off requires treating each lifecycle stage as an engineering boundary with defined inputs, outputs, and quality invariants. In production, pipeline orchestrators encode these dependencies as directed acyclic graphs (DAGs), where nodes represent discrete execution tasks and directed edges represent validated artifacts. While the overarching ML lifecycle forms a recurring feedback loop over weeks and months, each individual pipeline run executes as an acyclic sequence of dependent steps. This structure enables the orchestrator to parallelize independent branches, cache validated intermediate outputs, and halt execution the moment a contract is violated. Just as microservice architectures rely on interface specifications to enforce API boundaries, ML pipelines require strict schema, distribution, and performance contracts to prevent defective data or degraded weights from corrupting downstream artifacts. Each stage therefore has defined inputs, deliverables, and quality invariants (table 3.2).

Table 3.2: Stage Interface Specification: Each lifecycle stage has explicit input requirements, output deliverables, and quality invariants that must hold for the stage to be considered complete. Violations of these contracts create technical debt that compounds through subsequent stages. The deployment paradigm selection in Problem Definition (Cloud, Edge, Mobile, or TinyML) constrains all downstream stages, as a TinyML target imposes different data, model, and monitoring requirements than a Cloud target.

Stage	Input Contract	Output Contract	Quality Invariant
Problem Definition	Business requirements; operational context	Measurable objectives; deployment paradigm selection; resource constraints	All success criteria are quantifiable; target deployment paradigm is explicit
Data Collection & Preparation	Objectives; deployment target; quality requirements	Versioned dataset with schema; preprocessing pipeline; data validation rules	Distribution approximates anticipated production environment; labeling meets accuracy requirements
Model Development & Training	Dataset; accuracy targets; resource constraints	Trained model weights; training configuration; experiment logs	Meets accuracy thresholds within computational budget; architecture compatible with deployment target
Evaluation & Validation	Trained model; held-out test data; evaluation criteria	Performance metrics across subgroups; failure mode analysis; release manifest	No critical subgroup falls below minimum thresholds; predicted confidence matches observed accuracy as closely as the domain requires
Deployment & Integration	Validated model; infrastructure requirements; service-level agreement (SLA) targets	Serving endpoint; monitoring instrumentation; rollback procedures	Latency and throughput meet paradigm requirements; integration tests pass
Monitoring & Maintenance	Live system; performance baselines; alert threshold	Drift detection alerts; update-review criteria; incident reports	Alert coverage and detection windows are defined; detected degradation triggers review

Enforcing explicit contracts turns the conceptual feedback loop in figure 3.1 into an operational audit trail. When validation or monitoring surfaces a defect, the contract boundary identifies which

upstream stage owns the fix. For example, a mismatch between training features and production inputs may demand targeted data collection, whereas an error in feature transformation points directly to a bug in the serving pipeline. Catching these violations at stage boundaries stops bad assumptions before they harden into late-stage rework.

Example 3.1: Auditing stage transitions

Scenario: An engineering team claims to have completed Problem Definition for the DR screening classifier, attempting to transition to Data Collection without establishing deployment target constraints.

Diagnosis: The output contract lacks deployment paradigm selection and resource constraints (latency and memory targets). If the violation surfaces only at deployment as a trained model larger than the edge device's 500 MB model budget, the audit must trace what relied on the missing limit. A smaller candidate may require revised input-resolution or preprocessing choices, new weights, repeated sensitivity and on-device memory tests, and a replacement deployment package. Existing raw images may remain usable; changing the candidate does not imply recollecting every image.

Repair schedule: Suppose the repair plan preserves the raw images and labels but changes input preparation and the candidate architecture. Assume 1 day to revise preparation, 2 days to train new weights, 2 days to repeat validation, and 1 day to package the approved result. If each task waits for the preceding artifact and the required staff and compute are available, the repair takes 6 days. This is a supplied schedule for this dependency chain, not a multiplier determined by the discovery stage. Queueing, failed validation, or parallel independent tests would change the elapsed time.

Systems lesson: Stage transitions act as quality gates in the ML lifecycle. Enforcing strict output contract validation before proceeding downstream prevents expensive upstream rework and ensures data collection aligns with target deployment hardware.

Without explicit contract gates, unstated assumptions propagate downstream. The earliest opportunity to bound them is Problem Definition, which records the operational constraints that subsequent stages must satisfy.

3.5 Stage 1: Problem Definition

Problem definition in ML begins with sentences that look deceptively simple. A product manager writes: "Build a model that detects diabetic retinopathy." That single sentence conceals a dozen engineering decisions: sensitivity thresholds for patient safety, hardware capabilities in rural clinics, latency budgets that keep clinicians engaged, and regulatory frameworks governing approval. Software requirements still govern control flow, interfaces, and data storage. ML adds an empirical obligation. Defining *what* the system should do is inseparable from defining *how* it will learn from data and what physical constraints its execution must survive.

The request for a model must first become a requirement for a useful improvement. In the clinic case, the baseline is the existing screening and referral workflow, evaluated on the outcomes and resource limits the proposed system must improve. A simpler rule or an existing model can provide an additional comparison where appropriate. The comparison must include the complexity tax established in section 2.9.2, so a learned system earns its place only when the value of the errors it avoids exceeds the infrastructure and maintenance it adds. If available data cannot support the required evaluation, or the expected benefit does not justify the added errors, cost, and maintenance, the team should revise the problem or retain the baseline rather than proceed automatically to training.

Balancing these demands requires reasoning across the full system stack. An engineering team cannot optimize accuracy in isolation, because pursuing marginal sensitivity gains by scaling model depth or adopting a larger model family inflates parameter count and computational demands (FLOPs per inference), directly challenging the memory capacity and thermal limits of low-cost clinic hardware. Conversely, pruning a model (removing the weights that contribute least) to fit an

embedded accelerator's DRAM budget risks dropping sensitivity below clinical safety thresholds. Every technical decision operates within a multi-constraint optimization problem where statistical goals, physical execution budgets, and operational mandates continually bound each other's feasible design space.

3.5.1 Constraint layers

A screening model that reaches 95 percent sensitivity in a cloud notebook is unusable if it needs 4 GB of RAM on a clinic device with 500 MB to spare, or if clinic staff cannot audit why it flagged a patient. Every ML problem definition therefore decomposes into three interacting **constraint layers**: statistical, physical, and operational. Rather than isolated requirements, these layers form a coupled stack where a constraint in one tier immediately restricts the degrees of freedom in the others. At the top, the statistical layer specifies target behavior over data distributions, such as achieving > 90 percent sensitivity and > 80 percent specificity across diverse patient demographics and camera models rather than an unweighted aggregate accuracy that conceals failure modes on minority subpopulations. Meeting that target in the field depends on the physical layer, which bounds execution within hardware limits: for rural clinics, that means running inference on local edge accelerators under tight thermal budgets and bounded memory capacity (such as 4 to 8 GB of shared DRAM) and returning a result during the visit without depending on intermittent network uplinks. Surrounding both is the operational layer, which governs clinical auditability, regulatory clearance, and the maintenance rules that keep the deployed system safe over time.

The operational layer carries physical consequences that traditional software engineering rarely encounters. In a traditional database, deleting a patient's record is a `DELETE` statement. In an ML system, model weights may encode statistical patterns learned from that record. Where law or a regulator requires removing that influence, compliance may require machine unlearning (Cao and Yang 2015; Bourtoule et al. 2021) (an active research area with incomplete guarantees) or retraining on the remaining data. The obligation and remedy depend on the governing rule and system; at DR-system scale, either can make privacy compliance a recurring compute-budget item and training provenance an architectural constraint.

↪ Why removing a record's influence from trained weights remains a compliance obligation is developed in chapter 15.

The constraint propagation principle (section 3.2) explains why all three layers must be bound before development begins. A constraint that exists in the physical or operational environment but remains unstated in the problem definition does not vanish. It re-emerges later as an architectural violation after dependent decisions have hardened. For example, if an engineering team defines the objective solely as aggregate classification accuracy on retinal images, standard optimization will minimize the average training loss, which the majority class dominates. In an imbalanced screening population where 90 percent of patients are healthy, a model predicting "no disease" achieves 90 percent aggregate accuracy while failing every patient who needs urgent care. Preventing this requires translating a domain invariant (a missed diagnosis costs far more than a false referral) into an explicit statistical contract: class-weighted loss, a sensitivity floor, and subgroup targets.

War Story 3.1: When the label was the bias (2018)

Context: In 2014, Amazon assembled an engineering team at its Edinburgh hub to build an internal machine-learning system for ranking technical job applicants, training it on ten years of resumes submitted to Amazon (Dastin 2018).

Mechanism: Because prior hiring was male-dominated, the model learned that male-coded language correlated with hiring success, systematically penalizing terms like "women's" and downgrading graduates of two all-women's colleges.

Impact: The model reproduced historical hiring bias, rendering automated resume screening unsuitable for production recruitment.

Fix: Amazon abandoned the experimental tool after attempts to remove explicit gender signals could not guarantee that it would not learn other discriminatory proxies.

Systems lesson: Problem definition must specify fairness, auditability, and rejection criteria before data collection and training. A workflow trained on biased labels does not learn the operational goal; it learns to reproduce the labels.

3.5.2 Problem definitions evolve

When a DR screening program expands from one pilot clinic with a calibrated fundus camera to dozens of regional health centers with different camera brands and patient populations,[9] its initial assumptions break. Sensor noise profiles diverge, optical resolutions shift, and demographic changes alter subgroup error rates. A problem definition is therefore not a static specification frozen at the start of a project; it must evolve as deployment expands, adding subgroup accuracy targets and broader hardware compatibility checks. How such subgroup and governance requirements are specified, tested, and maintained is developed in chapter 15.

This evolution follows directly from how ML systems learn. Scaling exposes rare inputs absent from pilot evaluations, while production environments shift the input distribution in ways no static offline dataset fully captures. A sound problem definition anticipates this drift by setting governance rules upfront: defining which monitoring signals trigger formal re-evaluation and bounding how threshold updates propagate through later pipeline stages. Once these operational constraints and performance boundaries are defined on paper, the workflow must ground them in physical artifacts, beginning with the data collection pipelines that gather the training distribution.

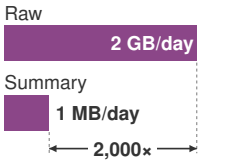
9 | **Demographic drift**: Population changes can expose performance gaps that aggregate evaluation obscures. Facial-analysis benchmarks found large demographic error-rate disparities (Buolamwini and Gebru 2018), while a healthcare risk-scoring study found racially biased allocation from a proxy label despite similar need (Obermeyer et al. 2019). These examples come from other domains; they illustrate why a growing DR program should evaluate subgroup performance.

3.6 Stage 2: Data Collection and Preparation

On paper, the problem definition asks for 90 percent sensitivity across rural clinics; in the field, that target immediately meets smudged camera lenses, missing patient records, and a 2 Mbps clinic internet link. Moving from problem definition to data collection is where abstract accuracy targets meet those physical realities. This stage determines dataset size (D), the number of examples, and through it the iron law's byte volume (D_{vol}), the bytes that storage systems, networks, and memory buses must move; the two grow together but are not the same quantity, since image resolution and encoding set how many bytes each example costs. The deployment constraints established during problem definition become technical data requirements. If the model must run on edge devices, the collection pipeline must produce inputs compatible with edge preprocessing; if it must achieve 90 percent sensitivity across diverse populations, the dataset must sample those subpopulations sufficiently to support model training.

Data collection and preparation is an ongoing engineering discipline rather than a one-time task. For diabetic retinopathy screening, these criteria demand a collection strategy far more stringent than standard computer vision benchmarks. The pipeline must assemble roughly 10^5 retinal fundus photographs across diverse patient cohorts and imaging hardware, with each image graded by multiple ophthalmologists and disagreements resolved to establish consensus ground-truth labels capable of withstanding clinical and regulatory scrutiny. The annotation protocol must capture fine pathological features (microaneurysms, hemorrhages, and hard exudates) across the entire disease severity continuum. Curation also governs computational efficiency, because not all captured data contributes equally to training, and deliberate selection (section 9.3.1) can eliminate redundant samples while preserving the statistical coverage required for generalizable performance.

Each retinal photograph spans several megabytes, and clinical protocols capture multiple views per patient, immediately creating a physical infrastructure bottleneck. A busy rural facility generates gigabytes of raw imaging data each day, a volume that would overwhelm an intermittent 2 Mbps uplink during operating hours. Rather than saturating limited wide-area bandwidth with raw uploads, the system can execute inference locally on the edge accelerator and transmit only a compact edge summary: a structured record containing the diagnostic classification and clinical metadata. Local inference reduces uplink traffic from tens of megabytes to kilobytes per patient.



Edge summaries shrink the clinic's daily uplink about 2,000-fold.

Napkin Math 3.2: Bandwidth vs. compute

Problem: Quantify the uplink window occupancy for raw image uploads from a busy rural clinic capturing retinal images for diabetic retinopathy screening, and evaluate architectural alternatives to relieve link contention.

Math:

1. Daily data: At 100 patients/day, 4 photos/patient, and 5 MB/photo, the clinic produces 2 GB/day.
2. Upload time: The uplink is 2 Mb/s, or 0.25 MB/s. Dividing 2,000 MB by that rate gives 8,000 s, approximately 2.2 h.
3. **Constraint**: If the clinic requires same-shift upload during its 8 h operating window, this transfer alone occupies the link for 27.8 percent of that window and may contend with other traffic.

Systems insight: On a good day the raw upload fits the shift, but it ties up a large share of the link, and on a day the link drops it does not happen at all. A screening result that must reach the nurse during the visit therefore cannot depend on the uplink, which is the case for local inference. Uploading detection summaries (10 KB/patient) instead of images also cuts uplink traffic by 2,000×. Overnight batching of images for training remains possible and is not compared here.

Running inference at the clinic (notebook 3.2) is the data locality invariant (definition 2.3) applied to a clinic uplink. Edge accelerators such as the NVIDIA Jetson family[10] make it practical, though they introduce a hardware trade-off between using a smaller model that fits a low-power edge budget and provisioning more capable on-premise devices at higher cost. The bandwidth constraint makes this infrastructure choice a data-collection decision rather than a late implementation detail: the scenario architecture combines edge devices for local inference and preprocessing, clinic aggregation servers for data management and buffering, and cloud training infrastructure for periodic model updates, targeting a result within seconds of capture and operation that does not depend on connectivity.

Privacy constraints impose a similar architectural decision. Patient privacy regulations may motivate local retention or distributed training, but they generally specify protections and permitted uses rather than one required training topology. Workflows that keep raw clinic data local while sharing only approved updates or summaries add complexity to both data collection and model training infrastructure.

10 | **NVIDIA Jetson**: NVIDIA's Jetson family spans a wide product range, from Jetson Orin Nano (7–15 W) through Jetson Orin NX (10–25 W) to Jetson AGX Orin (15–60 W). This scenario uses an Orin Nano-class device (4–8 GB shared LPDDR5, 7–15 W power envelope) as a concrete edge target. Its memory and power budgets constrain model complexity, while selecting a larger device changes both the feasible model and the per-clinic cost.

3.6.1 Lab-to-field data gap

Laboratory data and production data inhabit different worlds. This **lab-to-field gap** appeared when Google's DR screening reached clinics in Thailand (Beede et al. 2020): images arrive from diverse camera equipment operated by staff with varying expertise, often under suboptimal lighting with inconsistent patient positioning. A model trained on high-quality research images from standardized fundus cameras may fail on blurry, poorly lit images from older equipment, not because the algorithm is wrong but because the data distribution has shifted beyond the training envelope. Closing that gap across many clinics is an infrastructure problem as much as a modeling one.

↪ The cost and throughput of each storage tier, and why ML systems combine them, are measured in section 4.10.2.

3.6.2 Distributed data infrastructure

If a deployment grows from a handful of clinics to hundreds, the data infrastructure becomes a distributed system whose design follows from the constraints set during problem definition. Connectivity decides transmission. Facilities with reliable broadband can stream diagnostic summaries in near real time, whereas clinics with intermittent satellite links require **store-and-forward architectures** that buffer data locally and synchronize opportunistically during connectivity windows. That choice propagates through the physical stack, because store-and-forward clinics need larger local storage buffers, edge inference that runs without the network, and a way to reconcile local preliminary classifications with later central review. Storage placement matters at the central end as well, since the training set is read in full every epoch, validation sets occasionally, and the regulatory archive almost never, so each belongs on a different storage tier.

Data heterogeneity grows alongside byte volume. Divergent camera optics, uneven clinic illumination, and variable operator training give each clinic a distinct input distribution, so each clinic operates as a distinct data source.[11] The dataset this stage hands forward must therefore carry provenance metadata that records camera model, capture settings, and operator ID for every image. That provenance trail is what isolates a site-specific accuracy regression and satisfies a regulatory

11 | **Distributed clinic data**: Training across clinic sites without simply pooling all raw data addresses privacy and governance constraints, but it shifts cost into coordination. Each site may use different cameras, serve different patient populations, and follow different operating routines, so the system must track where data came from and how each site differs. The workflow cost is therefore not just storage capacity; it is the engineering work needed to compare, validate, and update models across sites whose data does not behave identically.

audit. Heterogeneity also argues for validation at the point of capture, because a miscalibrated camera discovered weeks later during a central training run has already wasted storage, compute, and patient follow-up opportunities.

3.6.3 Quality assurance and validation

A blurry retinal image that slips past quality checks does not merely waste storage. If such images enter training with unreliable labels or appear frequently in production, they distort the statistical evidence available to the model and degrade clinical safety. Quality assurance ensures that incoming data satisfies the invariants downstream stages depend upon. In the diabetic retinopathy scenario, automated quality-scoring models deployed at the fundus camera immediately flag out-of-focus or improperly framed captures, prompting clinical staff to retake the photograph while the patient is still present rather than discovering the defect weeks later during model training.

Validation extends beyond image quality to verify proper labeling, patient association, and privacy compliance. Local validation catches capture problems on the spot, whereas centralized validation detects distributional anomalies across the full clinic network, for instance by flagging when a particular site's images skew toward a narrow demographic range that would bias the training set. Together with the bandwidth and privacy limits set earlier in the stage, these quality checks settle what data the training set contains and what clinic-to-clinic variations the model must handle. Even a curated, validated dataset can still starve the next stage, however, if the order in which training reads it fights the storage system that holds it.

↪ Data cascades, in which uncaught collection defects compound into downstream training and evaluation failures, are developed in section 4.2.1.

3.6.4 Sample order and delivery rate

The DR training procedure reads the training set, on the order of 10^5 fundus photographs, in full on every epoch, in a fresh random order each time. Whether the accelerator stays busy depends on one comparison, the rate at which the data loader (the software that reads and batches examples for training) delivers images against the rate at which training consumes them. If delivery is slower, the accelerator idles however fast it is, and random order makes that outcome more likely because it scatters reads across storage and defeats the caching and prefetching that sequential reads enjoy. The data hand-off is therefore complete only when the workflow has measured the loader's delivery rate against the consumption rate. Closing a gap is the work of data engineering, which treats it as the feeding problem (section 4.9.3), and of training systems; the workflow's responsibility is to run the test and route a failure to the right fix.

Data collection therefore does not end when training begins. Feedback from evaluation and production keeps revising what is collected, and the delivery test keeps checking whether the loader can feed what the model consumes. Building the collection and delivery pipelines is the subject of chapter 4, and deciding which collected examples deserve training compute is the subject of chapter 9. What this stage hands forward is a versioned dataset, its preprocessing pipeline, and a measured delivery rate, which together become the raw material that model development turns into trained models.

↪ How prefetching overlaps data delivery with computation so that the loader keeps the accelerator busy is developed in section 8.6.2.

3.7 Stage 3: Model Development and Training

The DR team has on the order of 10^5 labeled retinal images, the scale of the 128,000-image dataset in Google's study, along with a validated preprocessing pipeline and a target: more than 90 percent sensitivity on edge hardware, returned within seconds of capture. The question is no longer *what data* to collect but *what model* to build, and that question has no answer independent of the deployment constraints already established. In iron law terms, this stage defines the operations (O) term, because architectural choices set the computational floor that hardware must sustain. Training is where that cost is first paid. The model learns by making repeated passes over the labeled images, adjusting its parameters slightly after each group of examples, and a single training run can occupy an accelerator for hours or weeks. Every idea the team wants to test costs at least one such run, so training systems must coordinate data ingestion, accelerator execution, and the search over training settings[12] without stalling execution pipelines. In high-stakes domains like healthcare, every design decision affects clinical outcomes, so technical performance and operational constraints must be integrated from the start.

[12] **Hyperparameter**: Architectural and optimizer choices (for example, learning rate and network depth) affect the computational work of training. A naive Cartesian grid that trains every combination independently incurs a multiplicative search cost: 5 hyperparameters with 4 values each produces 1,024 (4^5) configurations. Search strategies that stop unpromising runs early or choose the next configuration from earlier results avoid completing every run, which is why the grid represents a baseline rather than an unavoidable cost.

The DR system faces a sharp training challenge: achieve expert-level diagnostic accuracy with finite labeled-data and optimization budgets. **Transfer learning**[13] addresses this constraint by adapting a model pretrained on a source task to a target task. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 training subset contains 1.3 million labeled images (Russakovsky et al. 2015). Reusing learned representations can improve target-task optimization and generalization relative to random initialization, with the benefit depending on source-target similarity, which layers are transferred, and transfer-related optimization effects (Yosinski et al. 2014).

In Google's controlled validation study (Gulshan et al. 2016), transfer learning and a labeled dataset of 128,000 images produced an AUC[14] of 0.99, with sensitivity of 97.5 percent and specificity of 93.4 percent at one operating point. The result demonstrates the potential of pretraining on general images followed by further training on retinal images under the study conditions; it does not by itself establish deployment performance. Reaching a result of this kind takes repeated training runs over the full labeled dataset, and the arithmetic cost of each run on the available hardware sets how many runs a team can afford, which is the iteration tax again in another form.

Achieving high statistical accuracy on benchmark data is only the first hurdle, because edge deployment requires models to fit within strict memory allocations, execute within tight latency envelopes, and stay within thermal limits. **Ensemble learning** illustrates this tension between maximizing accuracy and fitting the deployment target. Combining predictions from multiple models routinely delivers top benchmark accuracy, yet inference compute, parameter storage, and memory traffic scale directly with ensemble size. Serial execution increases latency, while parallel execution demands additional hardware concurrency, so no fixed latency multiplier follows from ensemble size alone. The historic Netflix Prize demonstrated how ensemble complexity can block deployment: the winning solution delivered the highest predictive accuracy, yet Netflix never deployed it because the operational costs of maintaining and serving the ensemble exceeded the marginal business value (Johnston 2012). Even an ensemble of lightweight models multiplies orchestration complexity, working memory, and operational monitoring, so every architectural decision creates downstream systems ripples that accumulate at the physical deployment boundary.

13 | **Transfer learning**: For the same architecture, transfer learning changes how parameters are initialized and optimized, not the operations required at inference.

14 | **AUC (area under the ROC curve)**: Measures the area under the receiver operating characteristic (ROC) curve plotting true positive rate vs. false positive rate across all classification thresholds, ranging from 0 to 1; 0.5 represents random discrimination, and values below 0.5 are possible. Unlike accuracy, AUC is threshold-independent and insensitive to class prevalence, making it a common metric for medical screening systems. The systems consequence: a model with 0.99 AUC can still produce unacceptable sensitivity at the specific operating threshold chosen for deployment, so AUC alone cannot validate deployment readiness.

3.7.1 Accuracy vs. efficiency

In diabetic retinopathy screening, missing an active case risks permanent vision loss, whereas misclassifying a healthy eye generates an unnecessary hospital referral. Because those two errors carry very different clinical costs, medical diagnostics require specialized statistical objectives[15] rather than the generic average error that standard training minimizes. The system must maintain high sensitivity (> 90 percent) to catch pathology while preserving high specificity (> 80 percent) to avoid overwhelming regional healthcare infrastructure. These targets must also hold across diverse patient demographics, varying illumination, and distinct fundus camera models.

Optimizing for clinical metrics alone remains insufficient if the resulting architecture cannot execute within the physical deployment envelope. In this scenario, running the model must remain strictly within the 500 MB memory reservation on the clinic's shared 4–8 GB edge device. That budget must accommodate the static weight parameters alongside input buffers, dynamic activation tensors, and runtime execution overhead. Simultaneously, the accelerator must operate within the Orin Nano-class power envelope of 7–15 W and return each prediction within seconds. Balancing these competing pressures requires navigating the trade-offs of the iron law: shrinking parameter volume (D_{vol}) and arithmetic operations (O), for example by storing each weight in fewer bits or removing parts of the model that contribute little, while continuously revalidating that the smaller model preserves clinical sensitivity.[16] The accuracy in the cited study came from a larger Inception-family network, so whether a mobile backbone such as MobileNetV2 keeps that sensitivity is exactly what compression and validation must show. The capture resolution needed to see small hemorrhages also raises the backbone's operation count above its figure at the default input size, so the 500 MB budget and the latency target must be checked at that resolution.

The model that training produces is therefore a candidate, not yet the model that ships. Making it fit the clinic device is engineering work in its own right, and it acts on all three parts of the system. On the data side, training compute goes to the examples that teach the model the most (chapter 9). On the algorithm side, the trained model is made smaller and cheaper to run (chapter 10). On the machine side, its operations are mapped onto the accelerator so that the hardware's peak rate is

15 | **Medical AI performance metrics**: This DR scenario sets a >90 percent sensitivity target because missed referable disease can delay care and contribute to avoidable vision loss; actual acceptance criteria are product- and regulator-specific. The subtler systems trap is positive predictive value (PPV): even a high-accuracy model can have low PPV in a low-prevalence population. This prevalence dependence can require different operating thresholds across deployment sites, a constraint invisible in standard ML evaluation.

↪ The probability outputs and cross-entropy losses that standard classifiers optimize are developed in section 5.4.3.2.

approached in practice (chapter 11). Each change is then measured against the budgets that problem definition fixed (chapter 12). None of these steps can be skipped by training a more accurate model, because accuracy and fit are separate properties that the workflow must establish separately.

3.7.2 Constraint-driven development

An ML engineer who downsamples retinal images to standard 224×224 ImageNet resolution to save compute will erase tiny microaneurysms before the network ever sees them. Avoiding that kind of blind spot requires domain experts (ophthalmologists in the DR case) at the table from initial exploration through final optimization to identify which lesions and image-quality details matter clinically. Without that domain knowledge, a model architect can easily choose an input resolution or receptive field, the input region each internal feature can see, that discards the diagnostic signal before training even begins. Working together keeps model architectures focused on clinically relevant evidence while respecting the compute limits identified during data collection.

Computational constraints shape how experiments are run. Each model variant, training setting, and preprocessing approach adds training runs, so exhaustive experimentation quickly becomes expensive. This economic reality drives investments in scheduling runs so accelerators do not sit idle, reusing intermediate results instead of recomputing them, and stopping runs that are clearly not improving. Searching training settings systematically, with disciplined experiment design, reduces computation relative to exhaustive search.

The inherent uncertainty of ML outcomes demands scientific methodology: controlled variables through fixed random seeds and environment versions, systematic **ablation studies**[17] to isolate component contributions, confounding factor analysis to separate architecture effects from optimization effects, and statistical significance testing across multiple training runs using paired offline tests rather than the A/B testing[18] reserved for comparing models on live production traffic. Without this rigor, teams cannot distinguish genuine performance improvements from statistical noise, a distinction that becomes critical when a model sits near the clinical sensitivity threshold. The test set must then contain enough confirmed positive cases that the lower bound of the confidence interval, not the point estimate, clears the floor.

At every development milestone, teams validate models against the deployment constraints identified in earlier lifecycle stages. Each architectural innovation must be evaluated for accuracy improvements and compatibility with edge device limitations and clinical workflow requirements. This dual validation approach ensures that development efforts align with deployment goals rather than optimizing for laboratory conditions that do not translate to real-world performance.

3.7.3 Prototype to production

A small team of data scientists can often coordinate exploratory experiments using ad hoc scripts, spreadsheets, and shared notebooks. As projects evolve from prototype to production, however, complexity expands across multiple dimensions simultaneously: datasets grow, models require distributed training across accelerator clusters, and multiple engineers launch concurrent runs. Under these conditions, informal coordination rapidly becomes a development bottleneck. Without shared infrastructure, team members inadvertently train on conflicting data splits, duplicate expensive preprocessing passes across separate machines, overwrite shared model checkpoints, and struggle to reconcile divergent notebook versions. The time required to untangle conflicting artifacts and unrecorded configurations begins to dominate development.

Shared workflow platforms resolve this coordination friction by standardizing stage interfaces and centralizing resource management. Rather than relying on manual handoffs and uncoordinated scripts, a shared platform provides standardized data partitioning, cached preprocessing pipelines that compute intermediate representations once for the entire team, and automated job schedulers that allocate cluster accelerators efficiently. Shared platforms also track experiment metadata systematically, linking code revisions to exact training configurations, hyperparameters, and resulting evaluation metrics. Operating at scale therefore requires teams to systematically preserve each experiment's inputs, software dependencies, and execution configuration, establishing the baseline needed to verify genuine improvements.

16 **Model compression pipeline**: Bridging the gap between research accuracy and edge deployment requires an iterative "compress-validate-adjust" loop. Each compression step can reduce model size or execution cost, but it can also silently degrade sensitivity below the clinical threshold. Finding a model that fits in device memory while preserving clinical sensitivity typically requires multiple iterations because only the full validation suite reveals whether the smaller model still satisfies the problem definition.

↪ Renegotiating accuracy against memory and latency envelopes through systematic pruning, quantization, and distillation is developed in section 10.1.

17 **Ablation studies**: Named for surgical tissue removal, ablation studies systematically disable individual components to isolate their contribution to performance. The rigor matters because a 0.5 percentage-point sensitivity difference can determine whether the DR model meets its clinical sensitivity threshold. Without ablation, a team cannot distinguish a genuine architectural improvement from noise introduced by a different random seed, wasting iteration cycles on gains that are not real.

18 **A/B testing in ML**: Compares a new model (B) against the production baseline (A) on randomly assigned live traffic to estimate the model's causal effect. Required sample size depends on the baseline event rate, effect size, variance, allocation, significance level, and statistical power (Kohavi et al. 2020); no universal interaction count or duration follows from a 0.5 percentage-point change alone.

3.7.4 Reproducibility and system artifacts

Shared infrastructure accelerates experimentation only if its results can be trusted. If a candidate model beats the validation baseline by a full percentage point on Tuesday, but rerunning the same training script on Thursday falls short, the team cannot tell whether the gain was real or a fluke of thread scheduling and unrecorded library versions. **Reproducibility**, the ability to rebuild a result and bound how much it varies from run to run, is what lets a team distinguish a genuine improvement from noise. Its absence is an economic cost as well as a scientific one, because a team that cannot reproduce a result reruns experiments that may not converge to the same outcome, and each unreproducible result adds to the system's technical debt.[19]

Reproducing ML results requires tracking data versions, random seeds, hardware configurations, and library versions alongside program logic, because the hardware itself introduces run-to-run variation. When parallel accelerator threads add up partial results during training, the order in which partial sums arrive can differ from run to run, and floating-point addition is non-associative ($(a+b)+c \neq a+(b+c)$), so identical code can produce slightly different weights. Optimized math libraries may also benchmark several compute kernels (the routines that execute an operation such as a matrix multiplication on the accelerator) at runtime and select different ones across runs or machines unless deterministic execution is explicitly requested. Recording a random seed alone therefore does not establish reproducibility; the execution configuration and the measured variation across repeated runs matter as well, a requirement that section 12.8.2 extends to benchmark reports. Systematic **experiment tracking** records unique run identifiers and links data versions, code commits, hyperparameters, and resulting models, so a team can reconstruct what changed between two runs.

Experiment tracking needs a well-defined unit to version, and that unit is the stage's actual deliverable. A common failure mode is treating the trained model weights as the sole output of model development. In a mature ML workflow, the deliverable is a **reproducible system artifact** whose inference package has four components:

- **Model weights**: The learned parameters.
- **Inference code**: The exact code used to run the model, including preprocessing logic.
- **Environment specification**: The complete dependency graph (for example, a container image, `requirements.txt`, and accelerator drivers) required to execute the code.
- **Configuration**: Hyperparameters and runtime settings.

Reproducing inference and rebuilding a trained candidate require different evidence. Alongside the inference package, the artifact must link to the dataset version, training-code version, preprocessing recipe, training configuration, and execution environment that produced the weights. Under the data-as-code invariant (principle 1), a changed training set changes learned behavior as surely as changed inference code does. This provenance makes the training procedure repeatable; the nondeterminism described earlier still requires checking variation across runs rather than promising identical weights.

The environment belongs in the bundle because dependency mismatches do not always fail loudly. An unsupported driver or a missing library stops the serving process at once, whereas a different linear algebra library or image-resizing routine (OpenCV rather than PIL, for example) can alter floating-point results or pixel interpolation without any crash, shifting model outputs and accuracy. A model that meets its sensitivity target in development must therefore be reevaluated if its production environment changes, even when inference completes without an exception. Packaging a reproducible artifact still establishes only that the candidate runs consistently as built; the next stage tests whether it holds up under the clinical conditions it will actually encounter.

3.8 Stage 4: Evaluation and Validation

Suppose the DR team's candidate matches the study's AUC of 0.99 on curated research data. The team then tests it on images from a rural clinic where a nurse with two weeks of training operates a five-year-old fundus camera, and sensitivity falls well below the 90 percent target. The software executes without throwing an exception or returning an error code; the model's learned features simply encounter pixel distributions distorted by optical blur, low illumination, and variable pupil dilation that never appeared in the training distribution. High accuracy on curated test partitions does not guarantee operational reliability in production. Before deployment, trained models must

[19] **ML artifact interdependence**: ML artifacts are deeply interdependent. A model's measured accuracy is tied to the data version, preprocessing pipeline, and hyperparameter configuration used to produce it. Without lineage tracking, the team may not be able to determine whether a regression stems from code, data, or run-to-run variation, forcing expensive reruns of experiments whose provenance is lost.

↪ The technical debt that accumulates when experiments and artifacts go untracked, and the practices that contain it, are developed in section 14.2.

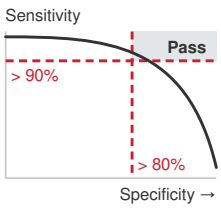
pass through systematic evaluation and validation gates to verify that they satisfy accuracy, latency, and robustness requirements across realistic deployment conditions. This stage transforms raw checkpoint files into production-ready software systems by measuring behavior against predefined operational thresholds.

Definition 3.3: Model validation

Model validation is an evidence gate for deciding whether a trained model is suitable to deploy for a specified use. It tests the model against deployment constraints such as latency targets, subgroup performance targets, cost budgets, and robustness under distribution shift, but cannot certify safety under every possible condition.

1. **Significance**: Validation adds dimensions that test-set accuracy ignores. A model achieving 95 percent accuracy on a static test set may miss a latency target, such as 100 ms in a web service, on its slowest requests, underperform a required subgroup threshold such as a maximum five percentage-point performance gap across demographic groups, or lose more than ten percentage points of accuracy under common image-quality problems such as blur and glare. Each unchecked dimension is a deployment risk that compounds silently in production.
2. **Distinction**: Evaluation measures model behavior using chosen datasets, metrics, and experiments, which may include out-of-distribution and subgroup tests. Validation assembles the broader evidence needed for a specified deployment decision, including system latency, throughput, robustness, operational readiness, and cost.
3. **Common pitfall**: Treating validation as "one more test." A model can pass the accuracy test and still fail the gate on latency, subgroup reliability, or cost.

Evaluation supplies measurements; validation uses them to decide whether the integrated software and hardware pipeline meets its intended deployment requirements. This deployment decision is broader than evaluation on a validation set, the data partition used to guide model selection. A separate test set supplies evidence after those choices are fixed; repeatedly using it to choose a model weakens its independence. Accuracy on held-out examples supplies only part of the release evidence, alongside tests of execution, robustness, and operational readiness. Metric thresholds, controlled production exposure, environmental stress tests, and any required regulatory qualification address complementary risks, with the evidence and sequence determined by the intended use. The measurement methodology behind these gates is developed in chapter 12.



Sweeping the threshold traces one illustrative curve; it must reach the corner where both floors hold.

3.8.1 Evaluation metrics and thresholds

In a screening clinic where 90 percent of patients have healthy eyes, a model that labels every image "normal" scores 90 percent accuracy while missing every case of disease. Evaluation metrics must therefore reflect the actual clinical consequences of each kind of error. In diabetic retinopathy screening, the evaluation reports sensitivity and specificity separately, with sensitivity above 90 percent limiting missed disease and specificity above 80 percent limiting false-positive referrals. Neither requirement alone establishes adequate clinical performance or referral capacity; that judgment also depends on disease prevalence, screening volume, and available care. The ROC curve summarizes how detection and false-positive rates change as the classification threshold varies. Evaluation must determine whether any threshold satisfies both requirements under the intended operating conditions.

Even separated into sensitivity and specificity, overall numbers can hide local failures. Stratified evaluation splits the test set across patient age brackets, chronic conditions, and clinic camera models to check whether accuracy holds across every subgroup. A DR model with 94 percent overall sensitivity may fall below 80 percent on images captured from patients with cataracts or under low clinic lighting. Those local drops stay hidden in overall averages and surface only as missed diagnoses after launch unless automated evaluation checks each subgroup slice before promoting a candidate checkpoint.

Beyond label accuracy, evaluation must measure probability calibration:[20] whether a model's

[20] **Calibration**: A calibrated model's predicted probabilities match observed frequencies: 80 percent confidence should correspond to 80 percent correctness. Calibration is distinct from accuracy, and the systems consequence for the DR system is severe: clinicians use confidence scores for triage decisions, so a miscalibrated model that assigns 90 percent confidence to uncertain cases can misdirect clinical workflows more dangerously than a less accurate but well-calibrated alternative. Recalibration after training can improve it, but its effect must be evaluated on representative data.

predicted confidence score matches how often it is actually right. Modern deep neural networks often produce uncalibrated probabilities, reporting high confidence (> 90 percent) even on wrong or out-of-distribution predictions (Guo et al. 2017). When automated triage routes borderline cases to human specialists based on confidence cutoffs, overconfident errors send the wrong cases through without review and erode clinician trust.

↪ How calibration is measured, and the recalibration methods that improve it, are developed in section 12.9.1.2.

3.8.2 Offline and online evaluation

The validation gate begins offline on static datasets and progresses incrementally toward live production traffic. Accuracy-only evaluation on held-out test partitions establishes predictive behavior but does not exercise bursty concurrency, network serialization, or telemetry overhead. Offline load tests and replay can expose such systems bottlenecks when they reproduce the relevant conditions; live traffic supplies additional evidence about the deployed environment. For a centralized service, online evaluation extends the gate to live traffic through staged rollout:[21] shadow mode mirrors live requests to the candidate and logs its predictions without returning them, a canary returns the candidate's predictions to a small fraction of traffic, and an A/B test splits traffic between candidate and baseline to measure whether the improvement reaches clinical or business outcomes. In a regulated medical device, splitting patients between models is a clinical study that requires ethical approval, and changing a cleared model requires regulatory change control. This staged rollout workflow manages deployment validation risk, which is distinct from the cross-tier model compression pattern termed progressive deployment in section 2.10.1, and each validation stage adds a different kind of operational evidence (table 3.3).

[21] **Staged rollout**: Coverage depends on traffic, observability, duration, and the failure modes exercised, so staged rollout reduces risk without guaranteeing that it catches a fixed share of production issues.

↪ Shadow and canary releases, the production form of this staged rollout, are developed alongside blue-green switching in section 14.4.3.

Table 3.3: Staged Validation Coverage: The stages provide complementary, overlapping evidence rather than partitioning failures into exclusive classes.

Validation stage	Typical evidence it adds
Offline evaluation	Algorithmic behavior
Shadow mode	Integration behavior
Canary deployment	Limited-traffic behavior
A/B testing	Comparative outcomes

The gates must also run where the model runs. In the DR scenario, intermittent connectivity and local execution move these evidence gates to clinic devices and pilot sites. Teams can compare a candidate against recorded clinic inputs, run it alongside the active model where device resources permit, and introduce an approved update at a small group of clinics before expanding the rollout. The workflow must preserve the evidence and exposure limits of each gate while adapting its implementation to the deployment target.

3.8.3 Production-condition validation

A model evaluated on one clinic's images still needs evidence for the other cameras, capture conditions, and patient populations specified during problem definition. Production-condition validation tests these requirements across site heterogeneity, input-level perturbations, and temporal drift. These dimensions apply within the offline and controlled-production gates in section 3.8.2; they are not stages that begin after live A/B testing. Held-out samples from the training distribution alone cannot supply all of this evidence.

External and multisite validation test whether the model has learned generalizable disease features or merely memorized the quirks of the development hospital's cameras. In retinal screening, sensor sensitivity, lens coatings, and flash color differ across camera vendors. A DR model trained exclusively on images from high-end fundus cameras in research hospitals can fail when deployed to community clinics using older hardware. Multisite validation therefore benchmarks candidate models on data collected across independent clinic sites with different cameras, lighting conditions, and operator experience levels.

Robustness testing shifts focus to individual inference requests, checking how the model handles realistic image flaws and edge cases. For computer vision pipelines, these flaws include sensor noise,

optical blur, uneven contrast, and partial pupil occlusions. As in the Thai clinics of the running case, dim room lighting and undilated pupils produce images the system rejects as ungradable. Testing models against both synthetic and real-world degraded images reveals whether the serving pipeline needs an upstream image-quality check to reject unusable frames before spending accelerator cycles on inference.

Temporal validation tests whether the model keeps its accuracy as the clinic environment changes over weeks and months. Test sets drawn from a single historical window miss later shifts in patient demographics, disease prevalence, or clinic referral habits. When the input distribution changes while the relationship between inputs and labels stays the same, the system faces *covariate shift* (Quiñonero-Candela et al. 2009); when the relationship between input features and clinical diagnoses itself changes, it faces **concept drift** (Gama et al. 2014). Both can cause silent model degradation and motivate the continuous telemetry and drift detection pipelines described in section 3.10.

↪ Statistical tests that detect covariate shift and concept drift in production are developed in section 4.12.1.

3.8.4 Regulatory validation

Healthcare AI systems face device- and pathway-specific validation requirements. When U.S. Food and Drug Administration (FDA) premarket review is required, the safety and performance evidence must be appropriate to the device and its regulatory pathway; clinical data are required for some submissions, though not all.[22] In practice, domain-specific validation goes beyond paperwork compliance to test clinical usefulness and safe failure behavior. For diabetic retinopathy screening, clinical validation studies run the automated system alongside licensed ophthalmologists, comparing model classifications against ground truth established by specialist consensus panels. These studies must demonstrate both specialist-level diagnostic accuracy and safe failure modes: systems that *fail safely*, by routing ambiguous or low-confidence images to human specialists, fit into clinical care, whereas systems that *fail silently* produce missed diagnoses that no one catches until patients lose vision.

Human factors validation tests how the model fits into the clinic staff's daily decisions. Even an accurate model will be turned off or bypassed if its interface confuses nurses, its inference latency slows down patient intake, or frequent false alarms cause alert fatigue. Validation studies must therefore measure how staff and system work together (tracking how often clinicians override the model, how long an exam takes from capture to referral, and how closely referrals match specialist diagnoses) alongside raw statistical accuracy.

[22] **FDA AI/ML regulation**: FDA marketing authorization requires evidence appropriate to the device, intended use, risk, and regulatory pathway. The FDA's 2021 action plan for AI/ML-based software as a medical device (SaMD) identified lifecycle management, real-world performance monitoring, and predetermined change control planning as central issues for adaptive medical-device software (U.S. Food and Drug Administration 2021). These concerns make model versions, training provenance, monitoring, and change documentation important workflow artifacts without imposing one identical audit artifact at every lifecycle stage.

3.8.5 Deployment readiness

Passing the validation gate produces an immutable release manifest recording the evidence and approval for a particular model version, intended use, and operating conditions. This manifest records the exact model checkpoint hash, preprocessing container configuration, calibration parameters, benchmark scores across designated demographic and equipment slices, and any required regulatory authorization. The manifest binds the release to that evidence; it does not itself confer external authorization or establish performance outside the evaluated conditions. If a model fails to clear an established threshold, such as p99 serving latency (the latency that 99 percent of requests stay under) exceeding the service level agreement (SLA) budget or sensitivity on low-light fundus images falling below the safety floor, the release gate halts deployment. Diagnostics then route backward to the responsible lifecycle stage: data curation for targeted data collection, model training for loss function re-weighting, or runtime optimization to reduce execution cost.

Clearing the validation gate establishes the evidence and approval for a particular system configuration. Deployment must install and sustain that configuration at each clinic, including its preprocessing routines, compiled model, telemetry collectors, and rollback triggers. Rollout checks confirm that the installed serving path matches the evaluated conditions and that local integration has not invalidated the release evidence.

3.9 Stage 5: Deployment and Integration

A model that passes validation in development faces different failure modes once deployed to production. In the DR scenario, the model must run on Orin Nano-class edge devices in rural clinics with intermittent connectivity, integrate with hospital information systems with distinct schema requirements, and deliver predictions clinicians can interpret directly. The in-visit requirement

already ruled out round-trip cloud inference. Deployment is where abstract operational constraints become physical systems requirements. In iron-law terms, the serving path couples data movement, execution efficiency, and fixed overhead together; on the clinic device, the 500 MB memory reservation is the precondition every candidate must meet, and per-image latency then sits comfortably inside the seconds-scale budget. Dedicated serving infrastructure manages these trade-offs under live traffic, and chapter 13 develops it in depth.

3.9.1 Deployment requirements

Deployment turns an abstract computation graph into hard physical budgets: peak memory footprint, execution latency, heat limits, and network bandwidth. In the clinical screening scenario, feasibility settles the choice between centralized cloud serving and local edge execution before cost is weighed, because the in-visit latency target and offline operation fixed during problem definition already rule out a cloud round trip. Cost serves only as a check. Napkin math 3.3 applies the comparison of capital expense against recurring operating expense from section 2.5.2 at network scale to confirm that the feasible option also pays back.



Napkin Math 3.3: Cloud vs. edge deployment economics

Problem: The cloud option fails the clinic's stated latency and connectivity requirements, so treat it as a cost benchmark only. Compare the total cost of ownership and operational trade-offs between a cloud inference endpoint and clinic edge devices for processing 760,000 billable screening images monthly across 500 clinics. Where the bandwidth calculation in napkin math 3.2 sized one busy clinic uploading every photograph, this network-wide estimate assumes an average clinic at 50 patients/day and bills one graded image per patient rather than every capture, so its image volume is lower by design.

Option A: Cloud inference.

- Model runs on centralized GPU servers
- Inference cost: ~$0.01/image (cloud GPU time + API overhead)
- Annual inference cost across 500 clinics, 50 patients/day, 1 billable image per patient, 365 days/year, and $0.01/image is $91,250/year
- Plus: An assumed connectivity and network-operations allocation for uploading 5 MB per billable image = ~$45,000/year
- **Total**: ~$136,250/year operational cost
- Risk: uploading one 5 MB image over the clinic's 2 Mb/s uplink alone takes about 20 s, before 200 ms+ of remote processing, far beyond the scenario latency target; connectivity outages halt screening

Option B: Edge deployment on a clinic device.

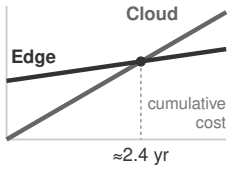
- One-time hardware: one $500/device per clinic across 500 clinics requires $250,000 capital expense
- Inference cost: ~$0.001/image (assumed all-in variable operating allocation)
- Annual cost: approximately $25,000/year maintenance plus approximately $9,125/year variable inference cost, for about $34,125/year
- **Total**: $250,000 upfront + ~$34,125/year
- Benefit: latency below 50 ms; works offline; much lower per-inference cost

Math:

- Annual savings: $136,250/year (cloud) − $34,125/year (edge) = $102,125/year
- Payback: $250,000 (hardware) ÷ $102,125/year = ~2.4 years

Systems insight: Under these scenario assumptions, edge deployment pays back in ~2.4 years and allows inference during connectivity outages, yet it requires tighter model optimization

(must fit in edge memory) and more complex update pipelines. The deployment paradigm selected during Problem Definition determines whether the edge option is even viable.



Edge costs more up front, but cloud's running cost passes it in about 2.4 years.

In this scenario, bandwidth limits, latency targets, and offline operational requirements favor edge deployment, which dictates strict memory, latency, and thermal envelopes that model compression must satisfy and serving runtimes must execute within production bounds. Host integration extends these physical constraints to the software interfaces of existing clinical infrastructure, where the inference runtime exchanges data with hospital information systems (HIS) to ingest patient records and record diagnostic output. Because prediction scores do not automatically provide reliable uncertainty estimates, their interpretation must be evaluated on representative data before an interface contract uses them to route decisions. This contract specifies which classifications trigger automated referrals and which demand physician review, including how rejected or low-quality inputs are handled, so a confidence threshold cannot be evaluated in isolation from the downstream workflow it triggers.

Regulatory frameworks place further structural constraints on this data lifecycle. Legal mandates such as data protection law (the Health Insurance Portability and Accountability Act, HIPAA, in the United States) govern data transmission and restrict whether production inference payloads can be persisted, linked to patient identifiers, or ingested into retraining pipelines. When compliance prohibits retaining raw inputs, continuous improvement cannot rely on passive telemetry logging, so privacy rules constrain ongoing model maintenance strategies rather than serving merely as a transport-layer security checklist (chapter 14).

3.9.2 Pilot to full deployment

Shipping a new screening model to 500 clinics at once would turn any undiscovered camera mismatch or schema bug into a network-wide outage. Production rollout therefore proceeds in stages to contain failures as the environment widens. Offline replay and synthetic test harnesses first verify schema contracts and concurrency limits without risking patient safety. Pilot deployments in a handful of clinics then expose the serving stack to real-world conditions absent from laboratory benchmarks: optical distortion across camera models, sensor noise under uneven clinic lighting, and differences in how staff position patients. Full-scale rollout broadens exposure to rare anatomical variations and uncommon pathologies. Catching those shifts requires continuous runtime monitoring rather than assuming validation distributions hold forever.

Scaling across hundreds of remote clinics brings hardware diversity and tight network limits across the fleet. Differences in local camera optics shift pixel distributions before inference even begins, requiring on-device input checks that reject out-of-spec captures on the spot. At the same time, narrow wide-area bandwidth rules out continuous weight updates or streaming high-resolution telemetry back to a central cluster. Clinic edge devices must instead run on their own, logging metrics locally and syncing state during scheduled maintenance windows.

Keeping the screener clinically safe requires calibrated uncertainty estimates rather than high average accuracy alone. When the model encounters a borderline retina, returning an uncalibrated score risks a missed diagnosis. Instead, the serving path uses **human-in-the-loop routing**, applying calibrated confidence cutoffs to send borderline cases to specialist ophthalmologists. This routing does two jobs at once: it protects patients in real time, and it creates a steady stream of high-value training examples. Specialist-graded borderline cases provide verified labels that feed back into later training runs, targeting the model's hardest cases.

Keeping hundreds of clinic devices consistent requires versioned, immutable model packages and atomic updates. Client runtimes verify cryptographic checksums of compiled model binaries before loading new weights into memory, so a dropped connection during download cannot leave a clinic running a corrupted model. Deployment marks the shift from static offline evaluation to continuous operations, where telemetry checks whether real-world latency and predictions stay within their approved bounds.

3.10 Stage 6: Monitoring and Maintenance

Consider an illustrative case six months after a DR screening system launches. A clinic upgrades its fundus cameras, and the new equipment produces images with different color profiles. In this assumed outcome, the model's sensitivity drops at that site because the pixel distributions it learned during training no longer match the images it receives; the monitoring team cannot establish that drop from the camera change alone. No code changed. The data drifted beyond the training envelope, and the model degraded silently. Where paired drift and outcome data support a local fit, the degradation equation (equation 1.1) describes the mechanism. The new color profile raises the statistical divergence $\mathcal{D}(P_t \| P_0)$ between production and training images, and accuracy falls by roughly the fitted coefficient λ times that divergence, with nothing in the serving logs to show it. *ML systems can degrade through data drift even when their artifacts remain untouched.* Deployment is therefore not the end of the lifecycle; it starts an ongoing operational phase. Monitoring provides the statistical telemetry to detect degradation, while continuous maintenance pipelines update models when evidence warrants it. The infrastructure that runs this phase in production is developed in chapter 14.

In this illustrative scenario, monitoring tracks performance across clinics to detect whether changing patient demographics, camera technology, or equipment degradation affects accuracy. Adding a new imaging modality such as optical coherence tomography would be a product change requiring new data, validation, workflow integration, and any applicable regulatory review rather than routine maintenance. Three feedback pathways guide update decisions: performance evidence can motivate targeted data collection, data-quality findings can prompt preparation changes, and confirmed model degradation can justify retraining or another intervention. Because clinics upload only diagnostic summaries in normal operation, those pathways need a data path of their own: a sampled set of images, sent during off-hours and labeled centrally, supplies the outcome evidence and the retraining data that summaries cannot. Drift thresholds initiate review; they do not select the remedy automatically.

3.10.1 Production monitoring

Monitoring serves two audiences, technical teams tracking system health and clinical staff who need actionable findings. Because no validation set captures every production condition (the lab-to-field gap of section 3.6.1), initial deployment can reveal blind spots that laboratory validation missed.[23] Clinics with older equipment may show accuracy decreases, and specific patient subgroups, such as those with proliferative retinopathy or cataracts complicating the fundus image, may see higher error rates. Monitoring infrastructure is therefore a deployment prerequisite rather than a post-launch addition, and its findings drive targeted data collection and model revisions.

A DR screening system, where missed referable disease can delay care and contribute to avoidable vision loss, demands continuous operational monitoring plus periodic performance evaluation when labels arrive. Teams establish quantitative thresholds for latency, accuracy, and data distribution stability. Lightweight statistical tests for comparing production distributions against baseline data (such as the population stability index and Kolmogorov-Smirnov test)[24] trigger responses ranging from automated alerts to retraining reviews.

A production DR system can track four metric categories at different timescales. The following thresholds illustrate one scenario policy; clinical and operational teams must validate them for the intended product, population, and deployment environment.

- **Model performance metrics** (requiring ground truth, available with delay): sensitivity (target above 90 percent, alert if the rolling average, pooled across sites until enough confirmed cases accumulate to resolve a drop of that size, falls below 88 percent), specificity (target above 80 percent, alert if it drops below 78 percent), and subgroup performance (alert if any demographic drops more than 5 percentage points below baseline).
- **Proxy metrics** (available immediately, without ground truth): prediction confidence distribution (alert if mean confidence drops more than 10 percent relative to baseline), referral rate (alert if rate changes more than 15 percent from baseline), and image quality rejection rate (alert if more than 20 percent of images fail quality checks).

↪ Divergence metrics and threshold conventions for measuring distribution drift are collected for reference in section B.2.3.

[23] **Lab-to-clinic performance gap**: Medical AI systems can experience substantial performance changes when camera models, image quality, patient populations, or operator workflows differ from development data; the gap arises because training data cannot capture the full diversity of production conditions. FDA has emphasized total product lifecycle oversight and real-world performance monitoring for AI/ML-enabled medical devices (U.S. Food and Drug Administration 2021), and device submissions may need evidence appropriate to the product's intended use and risk.

[24] **Population stability index (PSI) and Kolmogorov-Smirnov (KS) test**: Two lightweight statistical methods for detecting distribution drift, developed in section 4.5.3 (KS) and section 4.12.1 (PSI). The KS test measures maximum distance between empirical cumulative distributions. Both are cheap enough for frequent monitoring, but a detected input shift does not by itself prove an accuracy change; labels or other outcome evidence are needed.

↪ Drift detection pipelines that run these tests against training baselines in production are developed in section 22.

- **Operational metrics**: Inference latency (p95 below 50 ms, alert if above 100 ms), throughput (alert if queue depth exceeds 50 images), and error rate (alert if more than 0.1 percent of requests fail).
- **Data stability metrics**: Feature and prediction distributions compared with a baseline, with alerts when recent traffic moves outside the expected range.

The displayed alerts are urgent escalation thresholds, distinct from the acceptance targets. A measured target violation already opens a review of the affected site and system version. Neither threshold alone chooses retraining. The hierarchy matters because operational metrics surface immediate problems, proxy metrics flag possible model issues without ground truth, and labeled performance evidence arrives later.

For the camera upgrade, an immediate color-distribution alert supplies a signal and a site identifier, but until representative labels arrive the team holds evidence of changed inputs, not a measured sensitivity loss, and applies the previously approved exposure limits or alternative screening workflow. Reviewed referrals alone cannot supply those labels, because the routing policy selects which cases are reviewed. Whether the remedy turns out to be restoring a validated camera configuration or returning to data collection and model development, the fix re-enters the lifecycle at the stage that owns the defect and leaves through the same release gate, with a revised release manifest, before a controlled rollout.

That return path is the lifecycle loop, in which every production finding has an owning stage and a gate to pass again. Running the loop in production is a separate discipline, the operations loop that detects, diagnoses, decides, releases, and recovers. Its incident workflow, from severity classification to postmortem, is developed in section 14.7.1.1, and section 14.6.1 weighs a retraining against its compute and validation cost.

3.10.2 Maintenance at scale

Model updates return through the same release gate as the original model. For a regulated device such as the DR screener, staged rollouts must follow the approved change-control plan, and rollback mechanisms[25] limit the damage when an update misbehaves. As the deployment scales from pilot sites to hundreds of clinics, monitoring must track global metrics and site-specific behavior together and maintain data lineage, the metadata trail linking production logs to data, code, and model versions. Lineage lets a team correlate a site-specific accuracy drop with the training experiment and data version behind it instead of reconstructing that link by manual forensic analysis, and regulators may require it.

Correlating production evidence with its causes sends work back upstream. Figure 3.5 maps those return paths across stages 2 through 6, drawing data collection and data preparation as separate boxes and omitting problem definition, which sits upstream of every path shown. In the diabetic retinopathy deployment, when model evaluation uncovers accuracy drops on older fundus cameras, the *Data gaps* path triggers targeted image collection from those specific clinics. Similarly, when validation across patient cohorts reveals lower sensitivity for patients with cataracts, the *Validation Issues* path motivates retraining with synthetic lens-opacity augmentations. In production, runtime telemetry routes along the *Data Quality Issues* and *Performance Insights* paths back into data preparation and collection whenever clinic hardware upgrades change how images look.

Proactive maintenance closes the lifecycle loop. Operational signals identify potential problems, review determines whether newly validated data or another change is needed, and production evidence feeds back to refine problem definitions, data-quality standards, and architectural decisions. When retraining is selected, scheduled or evidence-triggered pipelines can incorporate approved data and return the candidate through validation. With the loop closed, the coupling among stages stops being a design-time concern and becomes a property of the running system, one whose consequences no single stage's metrics reveal.

[25] **ML rollback complexity**: An ML model's validity is coupled to the data distribution on which it was trained, not just its code. Because the data environment keeps changing, rolling back to an older model binary cannot restore the past data distribution, so the older model may also misbehave on today's live inputs. Even a fast rollback is therefore a temporary containment step rather than a full restore.

3.11 Systems Thinking Across the Lifecycle

The camera-upgrade case that opened the monitoring stage touched every stage at once. A change in capture hardware at one clinic shifted the input distribution, degraded a model that had passed validation, and required routing evidence back through the lifecycle to identify and validate a

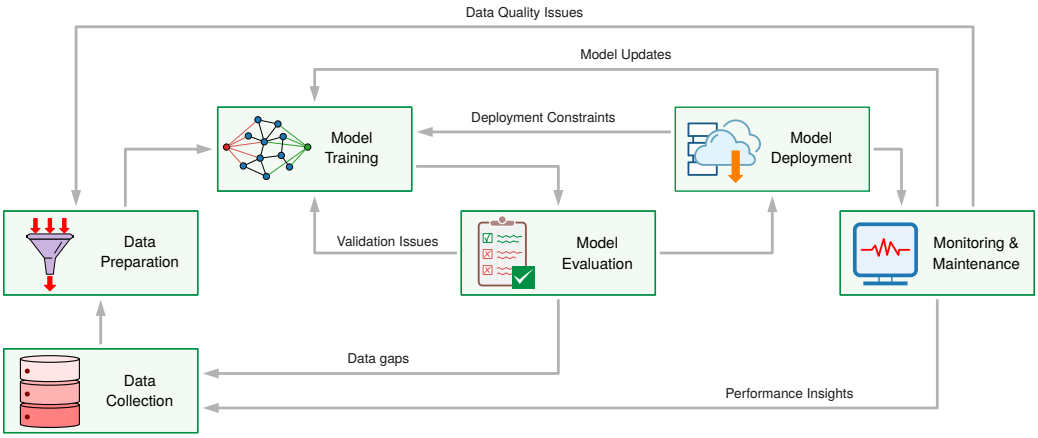


Figure 3.5: Feedback Paths Across Lifecycle Stages: Multi-stage feedback pathways show how downstream discoveries can require upstream corrections. Data-quality and distribution gaps detected during evaluation may prompt targeted collection or augmentation, while operational constraints and performance drift can motivate model, data, or pipeline changes after investigation.

remedy. The whole DR walk showed the same coupling: bandwidth, latency, and connectivity constraints favored edge deployment, edge deployment constrained model size, and model size reshaped preprocessing. Explaining coupled behavior of this kind requires **systems thinking**, which analyzes how a system's parts interrelate rather than treating them in isolation. Three structural mechanisms account for most of it. The first, backward constraint propagation (section 3.2), has recurred at every stage handoff since it was defined. Multi-scale feedback organizes the different response times within that loop, while emergent behavior explains failures that no single stage or site can see. Managing all three distinguishes proactive, constraint-driven system design from reactive post-deployment debugging.

Checkpoint 3.3: Closing the feedback loop

Apply the monitoring response to a different source of uncertainty.

During monitoring, a clinic reports a changed referral rate for patients over seventy. Latency and image-quality rejection rates remain stable. Labels are available only for referred patients, and the release manifest contains no separate sensitivity evaluation for this age group. The intended-use requirements include these patients, but the evaluation plan omitted the subgroup.

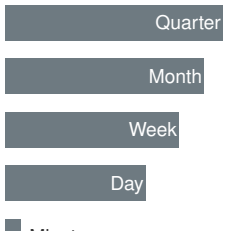
- ☐ **Evidence and immediate response**: Can you distinguish what the observed referral change establishes from what remains unknown, and identify the owner and approved exposure policy needed while representative labels are collected?
- ☐ **Dependency trace**: Can you identify which requirements, dataset coverage records, and evaluation artifacts need review, and explain what evidence would justify changing preprocessing or retraining rather than assuming both are necessary?
- ☐ **Re-release**: If representative labels confirm a sensitivity failure and a revised candidate is proposed, can you specify the subgroup, regression, and on-device evidence its release manifest must record before rollout?

3.11.1 Multi-scale feedback

ML systems succeed by orchestrating feedback loops across multiple timescales, each serving a different purpose. One illustrative operating plan for the DR scenario uses minute-level checks to catch a misconfigured camera before it produces a shift's worth of unusable images; daily reviews for proxy shifts such as referral-rate, confidence, rejection-rate, or site-specific camera changes; weekly aggregation of labeled accuracy statistics and drift tests when ground truth is available; monthly

analysis of population coverage; and quarterly review of whether the architecture still meets clinical needs. Actual cadences depend on label delay, product risk, traffic, and the cost of acting on each signal.

Each cadence matches how long its signal takes to become trustworthy. A misconfigured camera is visible within minutes, even if correcting it takes a site visit, whereas a demographic shift that calls for expanded training data takes months of monitoring to establish reliably. Reacting to short-term variance causes pipeline thrashing, such as retraining on transient shifts, while reacting too slowly lets silent accuracy decay persist.



Monitoring loops span minutes to quarters, five orders of magnitude.

3.11.2 Emergent behavior

In a multisite DR deployment, each individual clinic can report steady overall accuracy while network-wide analysis uncovers a sharp drop in sensitivity for a specific age group that no single clinic sees often enough to spot on its own. Complex systems produce **emergent behaviors** of this kind, which stay invisible when components or sites are inspected in isolation. Network timeouts and out-of-memory crashes expose some failures directly, but neither conventional nor ML systems can rely on exceptions to reveal every harmful outcome. ML monitoring must also investigate data drift, concept drift, and bias amplification, since predictive quality can decline without generating stack traces or runtime exceptions even while the model continues serving within its latency SLA. Together, constraint propagation across stage boundaries, multi-scale feedback timing, and emergent behavior show that ML systems cannot be engineered through modular decomposition alone, and teams that apply conventional software assumptions to these coupled dynamics fail in predictable ways.

3.12 Fallacies and Pitfalls

Treating ML as ordinary software produces predictable failures, each of which traces back to a section of this chapter. The misconceptions below recur when teams manage statistical models with assumptions inherited from deterministic application code.

Fallacy: *ML development can follow traditional software workflows without modification.*

Software practices such as version control, unit testing, and iterative delivery remain necessary. Machine learning systems also derive behavior from data, so those practices must cover datasets, learned artifacts, and statistical performance evidence (table 3.1). In an ML workflow, problem definitions evolve as exploratory training reveals distribution gaps, class imbalances, or infeasible accuracy-latency trade-offs, and practitioners name data preparation as their largest time sink (section 3.3).

Pitfall: *Treating data preparation as a one-time preprocessing step.*

Teams often treat data cleaning and feature engineering as a static milestone to clear before training begins. In production, incoming data distributions evolve continuously as user behavior shifts, sensors degrade, and upstream schemas change. The two-tier lifecycle of figure 3.2 reflects this operational reality, with an upper data pipeline and a lower model pipeline that execute concurrently and feed back into each other. Early data preparation decisions cascade directly through training and deployment (section 3.6). When data preparation is static, silent distribution shifts and training-serving feature skew bypass code-level tests, degrading inference accuracy long before raising an operational alert. Continuous data validation pipelines with automated schema checking and distribution monitoring are necessary to catch data drift at ingestion rather than after silent model failure in production.

Fallacy: *Passing model evaluation means the system is ready for deployment.*

Model-only offline evaluation measures statistical loss on curated validation sets and leaves operational serving dynamics untested. Across the dual pipelines mapped in figure 3.2, an accuracy-only evaluation tests the candidate model without establishing serving integration, real-time input delivery, or operational monitoring. The diabetic retinopathy case (section 3.1.2) shows the disconnect, since a model with specialist-level accuracy on curated images rejected 21 percent of clinic images as ungradable and struggled with variable camera lighting, technician workflow interruptions, and

intermittent network connectivity. Production readiness demands system-level verification, including tail-latency budgets under concurrency, feature pipeline consistency, and graceful degradation under hardware faults.

Pitfall: *Scaling data collection before checking marginal model value.*

Expanding dataset size is often assumed to be a universally profitable investment for boosting accuracy. Additional examples incur storage, ingestion, and labeling costs, but their learning value depends on which gaps they fill. Once a region of the input distribution is well covered, returns there diminish, and more examples of already well-covered cases may contribute little, while targeted collection can expose a failure that aggregate accuracy conceals. The upstream feedback loops in figure 3.2 route data fixes from curation back to collection before samples enter training, allowing observed errors and coverage gaps to guide acquisition. Because data quality decisions cascade through all downstream pipeline stages (section 3.6), teams must measure the marginal gain in the required task and subgroup metrics against the cost of collecting and processing additional data.

Fallacy: *Skipping validation stages accelerates delivery.*

Compressing or bypassing justified validation stages transfers unresolved risk to production traffic. Multi-stage validation (section 3.8) supplies complementary evidence across increasingly realistic execution environments, and no stage substitutes for another or for independent offline evaluation. A release plan should state the evidence required, the exposure permitted, and the conditions for stopping or reversing a rollout. A gate may be removed only when its evidence obligation is met another way.

Pitfall: *Deferring deployment paradigm selection until after model development.*

Teams often assume deployment targets can be selected after optimizing model accuracy. In practice, the deployment paradigm (Cloud, Edge, Mobile, or TinyML) is a primary constraint that shapes every preceding lifecycle stage (table 3.2). The day-153 failure is the canonical case, in which a model needing 4 GB reached deployment (stage 5) before anyone checked that the clinic device reserves about 500 MB for it. Deployment hardware dictates which models are physically realizable, not merely where they execute.

3.13 Summary

The day-153 failure in the running case was a workflow failure, and the six stages traced since show why it was predictable. Those six project responsibilities coordinate the finer-grained data and model pipelines. Within them, the data pipeline transforms raw inputs through collection, ingestion, analysis, labeling, validation, and preparation into ML-ready datasets, and the model development pipeline takes these datasets through training, evaluation, validation, and deployment to create production systems. The closed-loop return paths in figure 3.2, which link deployment telemetry back to collection and validation findings back to curation, show that production observations continuously reshape upstream pipelines rather than terminating at a deployment milestone.

Machine learning systems need additions to established software-engineering practice because ML workflows add probabilistic optimization, learned statistical behavior, and data-dependent feedback loops. The iron law supplies one quantitative lens: decisions across the lifecycle jointly change data movement, operation count, hardware efficiency, and fixed latency in $(T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}})$, while production evidence feeds constraint violations back into review. Success depends on how data quality affects model performance, how deployment constraints shape training strategies, and how production insights inform each subsequent iteration.

Two patterns define the engineering discipline of the ML lifecycle. Many practitioners report data collection, cleaning, labeling, validation, or preparation as major time sinks, while model development is only one part of the lifecycle. Production-ready systems also require repeated iteration cycles across data, model, and infrastructure stages, so investment in data engineering can have high leverage when data quality is a major source of rework. Early constraint discovery does not require freezing the workflow. It establishes an operating envelope through latency, memory, power, safety, and validation obligations while data and model choices continue to iterate within those bounds.

Key Takeaways: See the whole map first

- **The lifecycle is a loop, not a checklist**: Because an ML system's behavior is learned from data, it can degrade while the code stays fixed, so production feedback is what makes the data and model pipelines a system. Monitoring, validation, and retraining send lessons from deployment back into collection, labeling, architecture, and infrastructure decisions.
- **Late constraints can increase rework**: A deployment limit discovered late may invalidate preprocessing, weights, validation evidence, and integration work. Trace those dependencies and estimate the required tasks to bound the correction effort and delay.
- **Accuracy is not deployment readiness**: A model can clear its offline accuracy target and still fail on latency, memory, subgroup sensitivity, calibration, or an unreproducible environment. Validation is a multi-dimensional gate whose release manifest records evidence for one specific configuration.
- **Iteration velocity expands search opportunity**: Under the worked assumptions, a lightweight model starting 5 percentage points behind ends near the same ceiling as the large model while testing about four times as many ideas. Workflow speed buys search breadth rather than guaranteed model quality.
- **Interfaces make feedback actionable**: Stage contracts define inputs, outputs, and quality invariants so that data, model, and deployment teams can detect violations before integration. Without explicit contracts, each stage can optimize locally while the system fails globally.
- **Production speaks on different clocks**: Real-time inference monitoring, batch update reviews, and quarterly architectural reviews answer different failure modes. Treating all feedback as one loop either reacts too slowly to drift or churns expensive workflows without signal.
- **Workflow carries constraints through time**: Problem definition records targets, data collection shapes evidence and byte volume, model development changes operations and reuse, deployment realizes the complete serving path, and monitoring sends violations back into review. The lifecycle is data-algorithm-machine coupling unfolding over time.

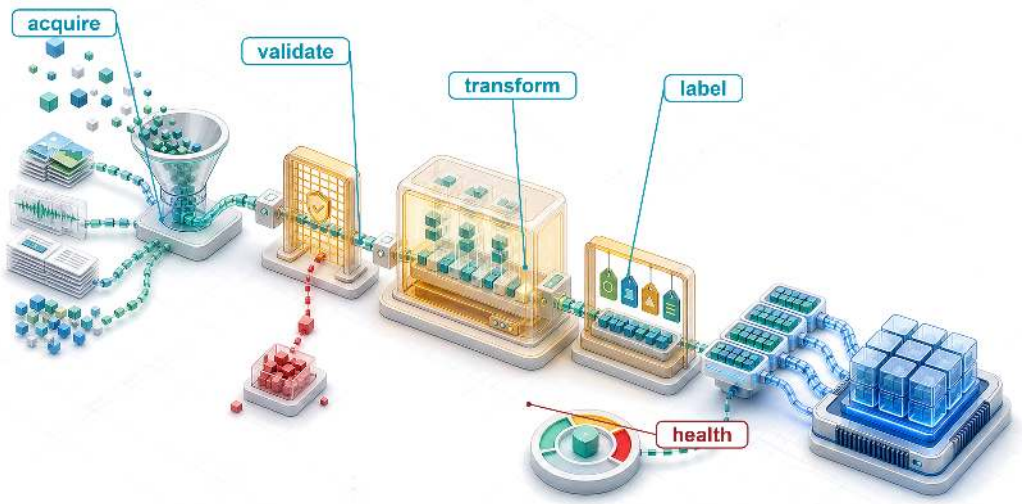
Seen as a checklist, the lifecycle is a sequence of stages to clear in order. Seen correctly, it is a loop. A constraint discovered at deployment does not stay there; it may force rework in dependent model and data decisions. A workflow is therefore a coupled system rather than a pipeline, the same coupling the D·A·M taxonomy describes in *space*, now unfolding in *time*. Optimizing one stage in isolation moves the failure instead of removing it. Workflow management reduces risk, and the discipline is to see the whole map before touching any single part of it.

What's Next: From blueprint to data

The lifecycle is the blueprint for every chapter that follows, and its first material is data. In the DR project that means on the order of 10^5 graded fundus images, capture-quality checks at the camera, site provenance, and ingestion that survives a fragile uplink, needs particular to one deployment. Chapter 4 takes up the stage in full on two workloads chosen to stress each of the bounds data places on a system. Data bounds a system twice, once through the coverage and labels that decide what the model can learn and once through the delivery rate that decides how fast the machine can learn it, and that chapter builds a pipeline that meets both bounds. Each later chapter assumes familiarity with where its techniques fit within this lifecycle.

4

Data Engineering

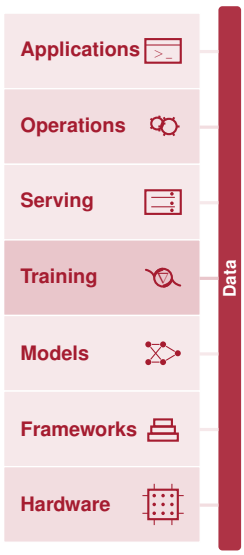


4.1 Data as Code
4.2 How Data Fails
4.3 Requirements and Data Budget
4.4 Data Acquisition
4.5 Ingestion and Validation
4.6 Data Labeling
4.7 Data Processing
4.8 Dataset Release
4.9 Physics of Data
4.10 Storage and Delivery
4.11 Accepting the Pipeline
4.12 Operational Data Health
4.13 Fallacies and Pitfalls
4.14 Summary

Purpose

Why does training data function like source code in a machine learning system?

In conventional software, programmers write logic that computers execute. In machine learning, programmers write optimization procedures that extract operational logic from data. This inversion makes training data resemble source code: changing it can change the learned system when the model is retrained, even if no traditional code changes. Yet data rarely receives the review, testing, and version control that even a one-line code change routinely receives. Subtle labeling inconsistencies can induce behavioral inconsistencies; missing edge cases can leave corresponding failures unmeasured; historical biases can propagate into learned behavior. A model trained only on that dataset cannot infer task-relevant evidence absent from it, and systematic label errors can shape what it learns. Unlike traditional source code, which sits inert until a programmer modifies it, the operating distribution can shift as the world changes, potentially changing production performance even when nothing in the codebase has been touched. Data engineering can consume substantial project effort, often more than modeling itself, because the work is consequential. A labeling error caught at collection costs a relabel; the same error found after deployment can cost a retraining run, an incident investigation, and the trust of the people the system failed. Every decision in the data pipeline (what to collect, how to label, when to filter, how to split) propagates forward to constrain model architecture, training dynamics, and deployment viability. Data engineering is therefore not preprocessing but programming in a different language, one where quality control, versioning, and monitoring determine whether the compiled system works today and continues working tomorrow. In D·A·M terms, data overlaps both the algorithm and the machine; it bounds what the model can learn, and the pipeline that delivers it bounds how fast the machine can learn it.



Learning Objectives

- Explain data as source code and trace how data cascades propagate through ML systems
- Evaluate acquisition strategies against coverage, quality, labeling cost, governance, and deployment constraints
- Design ingestion and validation pipelines for batch, streaming, ETL, and ELT workloads
- Specify labeling, transformation, and release practices that prevent data leakage and preserve label quality, training-serving consistency, and reproducibility
- Calculate data gravity, data-stall, and storage-bandwidth costs for moving or serving datasets
- Select storage tiers, file formats, and data loader designs that keep the accelerator fed
- Diagnose drift, data debt, and production pipeline failures using drift metrics, debt indicators, and controlled comparisons of pipeline versions

4.1 Data as Code

A missing accent in a speech corpus, a mislabeled class, or a feature computed one way in training and another in serving becomes a defect in system behavior with no line of code to point to. Because a machine learning system learns its behavior from data, the trained model is a compressed summary of its training set: it can represent only the evidence that data contains, and it treats the data's labels as the definition of a correct answer. The bitter lesson (section 1.4) explains why this dependence has grown rather than shrunk. Methods that learn from data at scale have repeatedly overtaken methods built on hand-encoded knowledge, so the knowledge that engineers once wrote into programs must now arrive through data. In the lifecycle that chapter 3 lays out, this work is Stage 2, which ends by handing a versioned dataset to model development; every later stage inherits what that dataset contains and what it lacks.

Data constrains the machine as well as the model. Every training step reads its examples from storage, decodes them, and moves them to the accelerator, and an accelerator waiting on input computes nothing however fast it is. Datasets measured in terabytes also have physical mass. They are slow and expensive to move, and they spend energy at every level of the memory hierarchy they cross. Data therefore occupies two of the pairwise overlaps of the D·A·M taxonomy at once, the ones labeled *what to learn from* and *how to move information* in figure 1.4. Toward the algorithm, it bounds what the model can learn; toward the machine, it bounds how fast learning can proceed.

Both bounds are expensive to meet. Cleaning and organizing data was the most common answer to where data scientists spend their time in the practitioner survey that chapter 3 presents (CrowdFlower 2016), and in the image-classification cost comparison later in this chapter, labeling a dataset costs roughly 2,500× to 5,000× as much as one training run on it. Each stage between raw input and an ML-ready dataset (acquisition, ingestion and validation, labeling, and processing) can also introduce a defect that no later stage detects. Data bounds both capability and throughput and often dominates cost, so it requires the same engineering discipline as the model and the machine: acquisition systems, validation checks, labeling workflows, storage layouts, and governance.

Definition 4.1: Data engineering

Data engineering is the infrastructure layer that manages the lifecycle of data from source to model, encompassing acquisition, transformation, storage, and governance.

1. **Significance**: It sets both bounds that data places on an ML system. Dataset composition, coverage, and labels determine whether the logical dataset (D, the Samples/Task factor of the iron law) represents the target distribution, and therefore what the model can learn. In the unpacked iron law, the pipeline governs the physical bytes moved (D_{vol}/BW), and therefore how fast learning can proceed.

2. **Distinction**: Unlike data science, which focuses on inference and insight, data engineering addresses the scalability and reliability of the data pipeline.
3. **Common pitfall**: A frequent misconception reduces data engineering to mere "data cleaning." In systems terms, it is a build pipeline that turns raw, noisy observations into the validated, versioned tensors that accelerator runtimes consume.

The data-as-code principle (1) turns the model's dependence on its data into an engineering rule. Changing a dataset changes the trained model just as changing source code changes a program, even when no line of algorithmic code is touched. The pipeline that produces a dataset is therefore the system's build, and every retraining is a rebuild.

↩ The iron law of ML systems, whose data-movement term this chapter governs, is established in section 1.7.

A dataset then needs the disciplines code already has, namely immutable lineage, validation suites, and debugging when regressions appear. Code keeps its tests apart from what they test, and a dataset needs the same trust boundary between its splits. The training set is allowed to shape parameters; the validation set is allowed to shape modeling and pipeline choices; the test set is reserved for estimating generalization after those choices have been made. **Data leakage** occurs when information crosses those boundaries, so that an evaluation score rewards what the model or its designers already saw instead of measuring generalization to unseen data. Leakage enters through duplicate examples appearing in multiple splits, augmented variants of the same source record landing on both sides of the split, user records from the same household appearing in both training and test, or time-derived features computed using future observations (Kaufman et al. 2012). Preprocessing can leak too, a case section 4.7.1 takes up once normalization and encoding have been introduced.

Two of the book's recurring lighthouse workloads (section 1.7.1) carry the chapter, one for each bound. Keyword spotting (KWS), the always-on wake-word detector behind phrases such as "Alexa" and "OK Google," carries the question of what the data must contain. It takes over from the diabetic retinopathy screening case of chapter 3 because its data problems span both a cloud training pipeline and a microcontroller-class device: the model must recognize a few words across accents, microphones, and rooms, stay silent through months of ordinary speech, and fit in tens of kilobytes on a TinyML-class device. Its reference corpus, the Multilingual Spoken Words Corpus (Mazumder et al. 2021), is real; the smart-speaker requirements and data budget that this chapter sets for it are an illustrative scenario. ResNet-50 image classification on one accelerator carries the question of how fast data must arrive, because image examples are large enough to stress storage, decoding, and transfer in ways that one-second audio clips do not. KWS also shows the split boundary at work before any model exists. Assigning each speaker's utterances to a single split keeps the acoustic model from memorizing speaker-specific vocal-tract resonances, so evaluation measures generalization to speakers the model has never heard.

Like a build, the pipeline that produces a dataset runs in stages, and the chapter follows them in the order of the two bounds. The capability half, which works the Data–Algorithm overlap, comes first. After the ways data fails and the requirements of the keyword spotter, acquisition, ingestion and validation, labeling, and processing build the dataset and settle what the model can learn, and a release step records exactly what was built. The throughput half works the Data–Machine overlap. The physics of moving datasets sets the cost of every byte, and storage and delivery must move those bytes fast enough to keep the accelerator fed. An acceptance test then checks both halves against the targets they started with, and operation keeps both bounds true after deployment.

4.2 How Data Fails

A credit scoring model starts rejecting every applicant from one region after an upstream database migration silently changes how ZIP codes are typed. A wake-word detector misses more and more spoken commands for months after a device maker changes the microphone in its new models. A fraud detection system misses a new attack vector because its training data was six months stale. Each failure traces to a different root cause (schema drift, distribution shift, data staleness), yet all share a common pattern. A data defect entered early, every later stage treated the data as trustworthy, and the failure surfaced only after deployment. The **four pillars framework** of quality, reliability, scalability, and governance gives the checklist that catches them.

4.2.1 Data cascades

The pattern those three failures share is a data cascade, in which poor data quality in an early stage amplifies through every later stage of the pipeline (Sambasivan et al. 2021). Some invalid inputs trigger immediate software errors, but data can also pass schemas and tests while degrading learned behavior silently until quality issues become severe enough to require expensive investigation, rework, or retraining. In the lifecycle progression sequenced in figure 4.1, early defects in problem framing and collection bypass intermediate validation and surface only at evaluation and deployment.

↩ The compounding cost of late-stage data rework builds directly on the constraint propagation principle established in section 3.2.

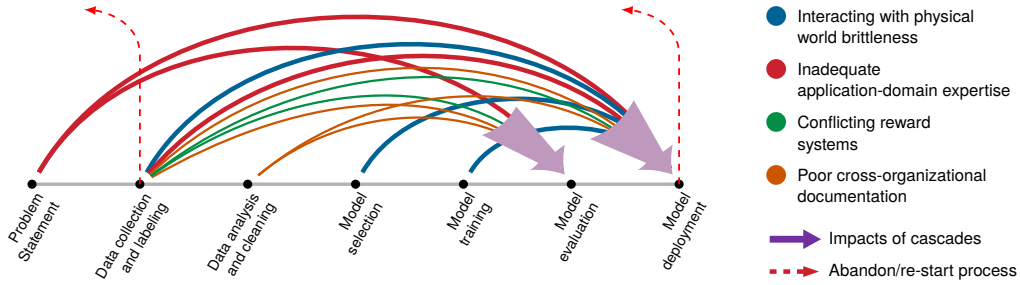


Figure 4.1: Data Quality Cascades: Upstream defects along the seven-stage workflow timeline cast failure arcs directly into late-stage evaluation and deployment. Early flaws in problem framing and collection compound through downstream processing, forcing costly project restarts (dashed red return paths) when defects finally surface in production. Adapted from Sambasivan et al. (2021).

Definition 4.2: Data cascade

Data cascade is an ML systems failure mode in which upstream data quality problems propagate through collection, labeling, feature engineering, training, evaluation, and deployment, amplifying into downstream model failures or user harm.

1. **Significance**: Remediation cost scales with the number of downstream artifacts that consumed the corrupted data: feature tables must be regenerated, models retrained, evaluation metrics recomputed, and deployed systems rolled back or patched, so one bad field can invalidate a whole training run and every experiment derived from it.
2. **Distinction**: Unlike an isolated data quality bug, a data cascade is defined by propagation and amplification. Each pipeline stage treats its input as trustworthy and converts the defect into new derived state, which hides the root cause as the system moves farther from collection.
3. **Common pitfall**: A frequent misconception is that ordinary software tests catch cascades. Corrupted data can satisfy schemas, pass unit tests, and produce successful training jobs while teaching the model the wrong signal, so prevention requires lineage, validation, contracts, and monitoring tied to model behavior.

Silence is what makes a cascade expensive. A software crash raises a stack trace and pages someone; a data-induced model error produces valid-looking outputs that are simply less accurate. By the time downstream metrics reflect the failure, corrupted predictions may already have contaminated historical data stores, forcing expensive rollbacks and data re-ingestion.

Example 4.1: The pipeline jungle

Scenario: The credit scoring model from the opening of this section suddenly begins rejecting all applicants from a specific geographic region.

Diagnosis: The training pipeline represented `zip_code` as a fixed-width string ("02139"). An upstream database migration changed the serving field to an integer without updating downstream data contracts. A downstream transform serialized that integer as "2139," losing the leading zero and causing the model to treat a familiar postal code as an unobserved category. In this scenario, the fallback for unknown categories assigns high risk, explaining the regional rejection pattern.

Systems lesson: Unenforced feature boundaries create pipeline jungles (Sculley et al. 2015) where upstream schema changes cause silent downstream model failures. Versioned data contracts, which are machine-checked rules that declare expected types, value bounds, and whether nulls are allowed, stop cross-system corruption by rejecting malformed records at the ingestion boundary before they reach the model.

4.2.2 Four foundational pillars

The ZIP-code failure crossed several concerns at once: the values were wrong (quality), the transform broke silently instead of rejecting the record (reliability), and no contract recorded what the field meant (governance). Preventing failures like it means balancing four competing operational priorities, each of which feeds the ML data system and each of which pulls against its neighbors (figure 4.2): stricter validation costs throughput, stronger consistency limits distributed scale, privacy constraints cost performance, and bias mitigation can shrink the usable training set.

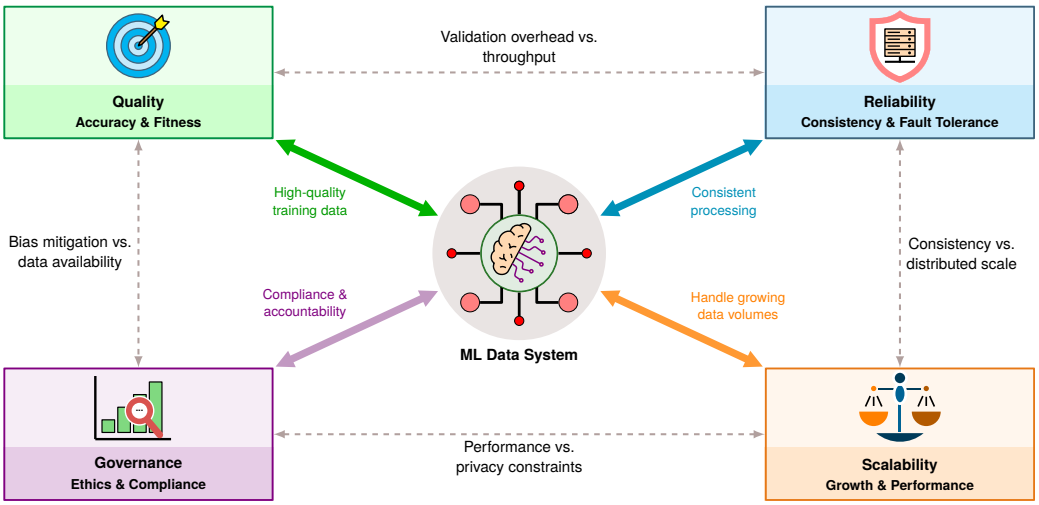


Figure 4.2: The Four Pillars of Data Engineering: Quality, Reliability, Scalability, and Governance form the foundational framework for ML data systems. Each pillar contributes essential capabilities (solid arrows), while trade-offs between pillars (dashed lines) mark where strengthening one pillar stresses its neighbor.

Quality governs input integrity and fitness for task: data must accurately reflect the operating distribution, with correct labels and uncorrupted features. In an edge voice-enrollment pipeline, for example, quality rules filter out clipped audio waveforms, low signal-to-noise ratios, and background acoustic interference so the model learns sharp decision boundaries rather than spurious acoustic artifacts.

Reliability asks whether that enrollment pipeline keeps working when the device, network, or user behavior is imperfect. A pipeline that produces excellent wake-word examples in a laboratory but fails during intermittent connectivity, battery pressure, or microphone glitches delivers no value in the user's home. Error handling, retries, local buffering, and graceful degradation turn the quality rule into a usable system: the device can request another utterance, defer upload until connectivity returns, or preserve a known-good enrollment rather than silently accepting corrupted audio.

Scalability asks whether that same decision survives growth. A manual review policy that works for a thousand recordings collapses when the product expands to millions of users, dozens of

languages, and long-tail acoustic conditions. The system must scale validation, storage, labeling, and retraining without letting infrastructure cost grow faster than the value of better coverage. The limiting resource depends on the workload. For edge speech, storage and labeling throughput dominate. For recommendation systems such as the DLRM lighthouse, the tables of learned user and item representations outgrow one machine's memory and must be partitioned across several.

Governance defines the boundaries within which quality, reliability, and scalability must operate. For keyword spotting, governance determines whether raw voice recordings may leave the device, how long enrollment audio can be retained, which consent record authorizes use, and what documentation proves that the dataset covers relevant accents without exposing private speech. A perfectly scalable, reliable, high-quality pipeline that violates the General Data Protection Regulation (GDPR) or perpetuates demographic biases creates liability rather than value. Dataset documentation practices such as datasheets (Gebru et al. 2021) and data statements make part of that governance visible by recording provenance, intended use, collection conditions, and coverage needed for bias analysis and scientific comparison (Bender and Friedman 2018).

When an ML system fails, each class of symptom points to one of the four pillars:

- Gradual accuracy loss points to quality. Drift has moved the serving distribution away from training, or label quality has fallen as annotator pools changed.
- Intermittent pipeline failures point to reliability. Error handling, retry logic, or resource controls are missing under peak load.
- Slow training on adequate hardware points to scalability. A single-threaded transformation, unpartitioned shuffle, or slow storage tier keeps parallel resources from doing useful work.
- Compliance gaps found in audits point to governance debt. Lineage tracking is incomplete, access controls are stale, or retention policies have fallen behind regulatory changes.

Checkpoint 4.1: Four pillars framework

The four pillars provide a systems lens for every pipeline choice.

Pillars

- ☐ **Quality**: Can you distinguish *correctness* from *coverage* (edge cases, subgroup coverage) and explain why both matter?
- ☐ **Reliability**: Can you name concrete failure modes (schema drift, missing values, skew) that cause silent degradation rather than explicit crashes?
- ☐ **Scalability**: Can you explain which parts of a pipeline become bottlenecks as data volume grows (storage bandwidth, compute, coordination), and why?
- ☐ **Governance**: Can you articulate what must be tracked to make a dataset auditable (provenance, consent constraints, access controls, documentation)?

Trade-offs

- ☐ **Pillar tension**: Given one pipeline change (for example, stricter validation, stronger privacy constraints, or more data sources), can you predict which pillar improves and which pillar(s) you stress?

Some failures span multiple pillars. Features that differ between training and serving implicate both quality (the values are wrong) and reliability (the computation is inconsistent). A privacy-motivated deletion policy can also create quality gaps if the retained data no longer covers the deployment population. Diagnosing such cross-pillar failures requires checking consistency contracts, comparing feature distributions across environments, and tracing transformation lineage, so production diagnosis checks data infrastructure alongside model behavior. The pillars name what to check, but not how good is good enough. Those thresholds come from the requirements of a specific system, and the keyword spotter supplies them.

4.3 Requirements and Data Budget

A wake-word detector answers to two numbers at once. It must catch nearly every spoken wake word, and it must almost never wake for anything else, while listening continuously in tens of kilobytes of memory on a microcontroller or the low-power core of a phone or smart speaker. The chapter's illustrative smart speaker sets those numbers, and every later stage of the pipeline is measured against them.

In the wake-word interaction sequenced in figure 4.3, an always-on detector traps "Alexa…" before the smart speaker acknowledges the utterance and opens the command channel for "dim the lights." Behind that brief exchange sits the data pipeline this chapter builds: the recordings the detector learned from, how they were labeled and checked, and how the result was verified before it shipped.



Figure 4.3: Keyword Spotting in Action: An Amazon Echo Dot exchange shows the user saying "Alexa," the device acknowledging the wake word, and the user issuing "dim the lights." A lightweight, always-on wake-word detector listens continuously and triggers the main voice assistant only after the keyword is spoken.

The detector must find specific keywords amid ambient sound and other speech, quickly and with few false activations, on a device with very limited resources. A well-specified problem definition names the keywords, the application, and the deployment setting, then states targets that can be measured. Overall accuracy is the wrong target, because almost every second of audio contains no wake word, and a detector that never fires would score nearly perfectly. The scenario therefore sets a **keyword detection rate** of 98 percent, the fraction of spoken wake words the detector catches, measured at a detection threshold that keeps false wakes within the budget the next notebook derives. It also requires a response within 200 ms and a model small and frugal enough for the device.

Napkin Math 4.1: False positive targets

Problem: An always-on KWS system budgets at most one expected false wake-up per month. Under a model that counts one nonkeyword classification window each second around the clock, how strict does the per-window false positive rate need to be?

Variables:

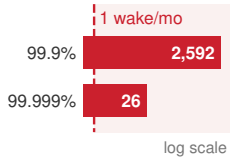
- **Duty cycle**: Always-on (24 hours/day).
- **Window size**: One-second classification windows.
- **Windows per month**: One window per second, 24 hours/day, over 30 days gives 2,592,000 windows/month.

Math:

- **False positive rate (FPR)**: 1 expected false wake divided by the monthly nonkeyword-window count gives approximately 3.9×10^{-7}
- **Nonkeyword rejection requirement**: 99.99996 percent rejection of nonkeyword windows.

Systems insight: This calculation bounds the expected monthly count, not the maximum number of false wakes in every month. Expectation does not require independent windows. Aggregate accuracy (for example, "99 percent accuracy") is insufficient here. Evaluation must report false accepts per hour (FA/Hr) together with the corresponding false-rejection behavior.

> Real detectors score overlapping windows every few tens of milliseconds and smooth the scores, so the per-window target depends on the detector, while the per-hour rate is the quantity to measure. A budget of one false wake per month is deliberately strict; commercial assistants typically pass each candidate wake to a second, larger check rather than asking the always-on detector to meet such a budget alone (Siri Team 2017).



Even 99.999 percent per-window rejection multiplies into 26 false wakes a month.

Evaluation therefore reports the two rates together, as a detection/error trade-off curve that plots false accepts per hour against false rejection rate on streaming audio representative of deployment (Nayak et al. 2022). The detection target is read off that curve at the false-wake budget, and operational metrics complete the contract by bounding response time from the end of the keyword to the system's reaction alongside the average power drawn while listening.

The device sets hard boundaries on any design that meets those targets. Memory limitations require extremely lightweight models, often in the tens-of-kilobytes range (Y. Zhang et al. 2017), to fit in the always-on island of the system-on-chip (SoC);[1] this budget must accommodate both model weights and the feature extraction buffers and preprocessing state. Limited computational capabilities (often a few hundred MHz of clock speed) demand aggressive model optimization. On phones and wearables the detector runs from a battery, and even a plugged-in smart speaker keeps its always-on stage to milliwatts so that idle power stays low, so KWS systems target sub-milliwatt consumption during continuous listening.

Data quality and diversity determine whether these targets can be met inside those limits. The dataset must cover speakers of different accents, ages, and genders, the many ways people pronounce the same wake word, and background conditions from quiet bedrooms to noisy kitchens. The model cannot grow to compensate for missing coverage, so on this device the data carries the burden that a larger model would carry in a data center.

[1] **SoC always-on island**: Modern system-on-chip designs partition power domains so a low-power "always-on" island (typically achieving sub-milliwatt draw) monitors for wake triggers while the main processor sleeps. The critical constraint is that this island must hold both the model weights *and* the audio preprocessing code within its dedicated SRAM, a split budget that forces KWS architectures to optimize for total footprint, not just parameter count.

4.3.1 KWS design space

Doubling the sampling rate of a wake-word recording from 8 kHz to 16 kHz captures sharper consonant sounds and raises the detection rate, but it also doubles the raw-audio window and the input-processing work on the device. Every data engineering choice for the keyword spotter pulls detection rate, false-wake tolerance, latency, energy, and memory against one another, and table 4.1 prices those trade-offs. Each row is a D·A·M coupling of the kind drawn in figure 1.3, because a data choice such as sampling rate or coefficient count changes what the algorithm receives and what the machine must hold in always-on memory. One row uses mel-frequency cepstral coefficients (MFCCs)[2] (Davis and Mermelstein 1980), which reduce each frame of roughly 25 ms of audio to 13 to 40 numbers that emphasize the frequencies speech uses. The coefficient count sets the size of the model's input and how much acoustic detail it keeps; section 4.7 shows how they are extracted.

[2] **MFCC (mel-frequency cepstral coefficients)**: Mel-scale filtering gives this representation its compactness and noise resistance by emphasizing the frequencies humans use to distinguish speech. It reduces the hundreds of raw samples in a short window (400 samples in 25 ms at 16 kHz) to 13–40 coefficients, so each frame fits inside an always-on, kilobyte-scale memory budget. A parameter mismatch between training and serving creates feature skew and can degrade accuracy.

Table 4.1: KWS Data Engineering Design Space: Illustrative scenario inputs price each design choice along four axes (quality, latency, cost, and memory); the entries are not measurements from one system and should not be combined as additive effects. Each row moves from a baseline (left of the arrow) to an alternative, and every entry gives the effect of that move. Higher sampling rates double raw-audio storage and processing, while more training examples multiply labeling and storage requirements. Actual quality and latency must be measured on the target pipeline.

Design Choice	Quality Impact	Latency Impact	Cost Impact	Memory Impact
8 → 16 kHz sampling	+2–4% detection rate	2× input-processing workload	2× raw-audio storage	2× raw-audio window
13 → 40 MFCC coefficients	+3–5% detection rate	3× model-input size	Minimal	3× feature memory
1M → 10M training examples	+5–8% detection rate	10× training time	10× labeling cost	10× storage
Clean → noisy training data	+10–15% real-world	Minimal	3× collection cost	Minimal
Cloud → local inference	up to 2% detection-rate risk	100 ms → 10 ms	\$0.001/query → \$0/query	cloud-scale → 256 KB

Design Choice	Quality Impact	Latency Impact	Cost Impact	Memory Impact
Real only → added synthetic augmentation	+3–5% robustness	Minimal	10× cheaper per example than real collection	Minimal

Because the device runs inside a milliwatt power envelope and a tens-of-kilobytes SRAM budget, an engineering team cannot pick the most accurate option in every row of table 4.1 at once. Instead, the team must divide a fixed data budget across labeling, storage, and governance so the detector reaches its accuracy target without outgrowing the hardware. Notebook 4.2 allocates the data budget and proposes a candidate design for a smart-speaker deployment. The acquisition, labeling, processing, and delivery decisions that follow supply the evidence needed to accept or reject it.

Napkin Math 4.2: Allocating the KWS data budget

Scenario: A KWS system for a smart speaker has these constraints:

- **Target**: 98 percent keyword detection rate at a threshold that keeps the expected false-wake budget below 1 false wake/month
- **Budget**: $150K total data engineering budget
- **Memory**: 256 KB model size limit (always-on island), with a separate 16 KB budget for preprocessing state
- **Timeline**: 6 months to production

Step 1: Choose conservative options for the constrained device.

For this scenario, the team chooses two conservative options:

- 13 MFCC coefficients, because every MFCC frame lives in the always-on island's SRAM as model input and buffered state, so the 40-coefficient option triples a cost the device pays continuously
- Local inference, because streaming every second of audio to a server is ruled out by bandwidth, privacy, and offline operation

The choice rests on where each cost lands. Data collection is paid once, from a central budget, whereas the 40-coefficient representation triples the model's input, and with it the first layer's work and the feature buffer, on every device for every second it listens. The table suggests a 3–5 percent detection gain from 40 coefficients, so the team picks 13 and spends the difference on coverage, expecting broader training data to recover the gap. The acceptance test of section 4.11 decides the choice.

Step 2: Calculate budget allocation.

Labeling is priced at a scenario rate per crowdsourced example, set above the image-classification crowd rates that section 4.6.2 later tabulates because a KWS contributor must record the utterance as well as label it. The $150K budget splits across three cost categories:

- **Labeling** (~60 percent): $90K available
- **Storage/Processing** (~25 percent): $37.5K
- **Governance/Other** (~15 percent): $22.5K

At $0.10/label with 20 percent review overhead: $90K ÷ $0.12/label = 750K labeled examples

This yields roughly 0.75M labeled examples, just below the 1M anchor in the design space. The 1M-to-10M row in table 4.1 should therefore be read as a scaling reference rather than a direct interpolation: the real-label budget alone does not buy the full data-volume gain. Counting examples also hides what limits a wake-word model in practice, which is the number of distinct speakers and the hours of ordinary, non-keyword audio available to measure false wakes; section 4.11 prices that listening time.

Step 3: Identify candidate quality improvements.

The quality plan has three components:

- An illustrative base-model detection rate of ~90 percent
- Partial data-volume gain from 750K real examples
- A gap to the 98 percent target that the options below must close

Three options from the design space are candidates for closing the gap; their collection, storage, processing, and deployed-footprint costs must be checked against the allocations. Their listed gains cannot simply be added to the base detection rate, because they overlap (noisy data and synthetic augmentation both buy robustness to acoustic conditions), so the plan is a candidate whose combined effect must be measured on the trained system:

- 16 kHz sampling: a detection-rate gain of 2–4 percent, 2× raw-audio storage (requires cost validation; on the device only the short window of raw samples feeding each MFCC computation doubles, because every frame still reduces to 13 coefficients)
- Noisy training data: a field detection-rate gain of 10–15 percent, 3× collection cost (proposed for a targeted noisy slice; its cost must fit the storage/processing allocation)
- Synthetic augmentation: a robustness gain of 3–5 percent, 10× cheaper than real data in the illustrative row; generation, verification, and storage costs remain to be measured

Step 4: Final configuration.

The scenario combines these choices with 13 MFCCs and the computed 750K real-label budget in table 4.2. The quality effects can overlap, so the table records a candidate configuration rather than a solved optimum.

Result: The budget buys approximately 750K human-labeled examples, 13 MFCCs, local inference, and a noisy slice plus synthetic augmentation to close the quality gap. Whether that configuration reaches the 98 percent detection target inside the 256 KB model limit is what acceptance decides.

Systems insight: Design-space analysis turns "we need more data" into a costed plan. The stated labeling assumptions buy about 750K real examples, and the acceptance test, not the design table, decides whether that plan succeeds.

Table 4.2 records the resulting configuration, pairing each design choice with the constraint that motivates it: sampling and augmentation draw on the stated allocations, while the always-on island sets the deployed-footprint constraint. Actual costs and working-state memory still require validation.

Table 4.2: KWS Candidate Configuration: This scenario uses the computed real-label budget and records the remaining sampling, augmentation, and deployment choices for measurement against the quality and memory targets.

Choice	Selection	Rationale
Sampling rate	16 kHz	+2–4% detection rate for 2× raw-audio storage, subject to cost validation; on device only the raw-sample window grows, since each MFCC frame keeps 13 coefficients
MFCC coefficients	13	One-third the feature buffer and model input of 40 coefficients in always-on SRAM; test whether improved data coverage recovers the quality gap
Training examples	750K real + illustrative synthetic augmentation	Human-label count follows from the budget; synthetic volume is a scenario choice
Data diversity	Noisy + clean mix	Critical for real-world deployment
Inference	Local, reduced precision	Reduced precision can lower footprint; the final 256 KB fit requires the quantization analysis in section 10.3
Augmentation	Heavy synthetic	Illustrative 10× per-example cost advantage; total generation and review costs require validation

The candidate configuration fixes the targets but not the data. Meeting them starts with acquisition: finding recordings that cover the speakers, accents, devices, and rooms the detector will meet, from sources whose cost and consent terms the budget can bear.

4.4 Data Acquisition

A huge corpus of studio audio does not help a wake-word detector that has to run on inexpensive microphones in noisy kitchens, because a model cannot learn a speaker, room, or microphone that no source supplied. Data acquisition therefore sets the outer limit of the capability bound, and it begins when the team names the coverage gap the model must close. For keyword spotting, public corpora are the first source. Speech Commands (Warden 2018) established the small-vocabulary benchmark of one-second spoken words, and the Multilingual Spoken Words Corpus (MSWC) (Mazumder et al. 2021) provides a reusable reference source of 23.4 million audio samples spanning 50 languages. That corpus is larger than the product-specific labeled subset budgeted in the KWS design example; using it still requires selecting and validating relevant coverage.

Reaching a 98 percent detection rate requires representative coverage of regional accents, speaker age groups, room reverbs, and microphones that differ in frequency response, signal-to-noise ratio, and clipping behavior, at volumes that manual curation cannot economically supply and under strict privacy boundaries on audio retention. Because no single collection method can economically cover all of those conditions, acquisition strategy is a sequence of gap-closing decisions: reuse what already matches deployment, collect what is missing, scrape or synthesize where scale is the binding constraint, and reject sources whose provenance or consent constraints make them unusable.

4.4.1 Coverage and diversity requirements

A wake-word detector trained on millions of clean clips can score well on an aggregate validation set while still waking once an hour on kitchen conversation or missing speakers with underrepresented regional accents. Dataset scale alone does not guarantee generalization when coverage gaps in a large corpus (geographic bias, demographic underrepresentation, sensor calibration drift, and unobserved edge cases) cause systematic failures that aggregate validation metrics hide (Shankar et al. 2017; T. Wang et al. 2019; Oakden-Rayner et al. 2020).

For KWS, coverage must support both keyword recognition and the false-wake contract. Labeled wake-word clips measure recognition, but acquiring more of them does not establish how often an always-on detector activates on ordinary nonkeyword audio. The acquisition plan must separately reserve consented recordings collected for an evaluation study of representative listening conditions, including background conversation, television, music, silence, and environmental noise across the intended microphones and rooms. These recordings retain their duration and temporal sequence so evaluation can run the deployed detection path and report false accepts per hour alongside false rejection on keyword recordings. A short stream with no observed false wakes supplies limited evidence; its duration and the uncertainty of the measured rate constrain any claim about the monthly target.

Training negatives serve a different acquisition goal. Mining near-wake phrases and other audio that confuses the current detector can improve its rejection behavior, but this deliberately difficult sample does not preserve the frequency of those conditions in deployment. The representative evaluation streams therefore remain separate from training and threshold selection, with speaker and recording-period boundaries maintained. An acquisition budget that funds only keyword labels and mined hard negatives can improve the detector while leaving its operational false-wake target unmeasured.

Governance constraints further shape acquisition: privacy and health-data regulations such as GDPR and the U.S. Health Insurance Portability and Accountability Act (HIPAA) limit what data can be collected and how (European Parliament and Council of the European Union 2016; United States Congress 1996), while ethical sourcing requires fair compensation and transparent use of human contributions. Together, these coverage, evaluation, and governance bounds define the criteria against which each candidate data source is judged.

↪ The governance infrastructure that proves what data a production system ingested, who accessed it, and on what legal basis is developed in section 15.5.

4.4.2 Selecting data sources

Paying for custom voice recordings of words and accents that a public benchmark already covers wastes the data budget, while stopping at the benchmark leaves the detector blind to its own microphones and rooms. The choice among curated datasets, expert crowdsourcing, controlled web scraping, and synthetic generation therefore begins with the cheapest reusable source and escalates

to active collection or simulation only when the remaining deployment-distribution gap justifies the added cost, quality effort, and governance risk.

Preexisting datasets from repositories such as Kaggle, UCI (Dua and Graff 2024), and ImageNet[3] are the first test. They offer speed and comparability when the deployment distribution resembles the benchmark enough to make reuse meaningful. For KWS, a curated speech corpus can establish the baseline model and reveal which words, languages, and acoustic conditions are already covered. A baseline corpus does not guarantee operational coverage, but it makes the remaining distribution gaps measurable.

Whether an external dataset is safe to reuse depends on how well it is documented. Without records of sensor calibration, recording environments, and filtering rules, a team cannot reproduce an external dataset's conditions or check whether they match deployment (Pineau et al. 2021; Henderson et al. 2018). Good documentation states how the data was collected, how each label was defined, and what baseline error rates were found, so another team can validate the corpus independently. Once ingestion reaches millions of examples from varied sources, hidden label errors also multiply past what humans can inspect by hand (Gudivada et al. 2017), so the pipeline must run automated validation checks on every batch.

Reuse carries a less visible risk. Standard validation metrics can create an illusion of model readiness when the benchmark itself is unrepresentative, and when many teams train on the same repository, its blind spots do not average out across their models. They can recur across models, so five models trained on one dataset can share correlated failures (figure 4.4).

3 **ImageNet**: The 2009 paper reported 3.2 million images across 5,247 synsets; the full database later reached about 14.2 million images across 21,841 synsets, whereas the challenge used a 1,000-class subset (Deng et al. 2009, 2024; Russakovsky et al. 2015). Its value as a benchmark is inseparable from its data engineering: Fei-Fei Li's team built labeling infrastructure that later teams could reuse. The catch is sensitivity to annotation procedures and modest distribution shifts: models tuned to ImageNet's distribution can underperform on related but shifted test sets (Recht et al. 2019; Beyer et al. 2020).

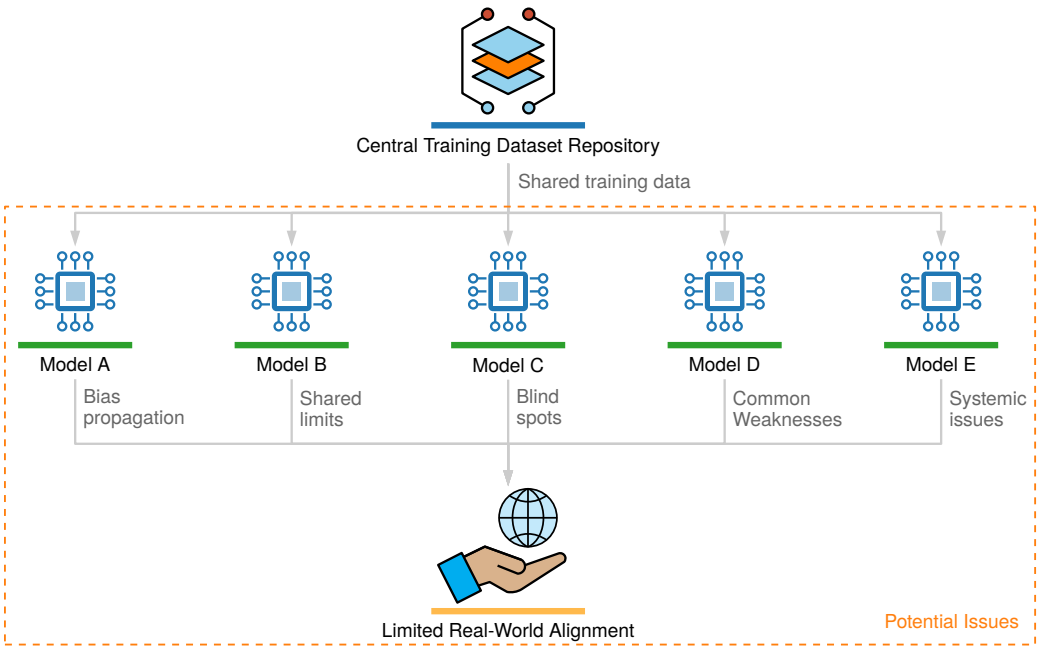


Figure 4.4: Shared Dataset Bias Propagation: Five models (A through E) all train on a single central dataset repository, reached by a path labeled shared training data. Beneath the models, five labeled arrows (bias propagation, shared limits, blind spots, common weaknesses, and systemic issues) converge on one box, limited real-world alignment. Shared training data can propagate correlated blind spots across models; their failures need not be independent.

This gap between benchmark and deployment is the central failure mode of static offline evaluation. Because training minimizes average error only over the observed training set (empirical risk minimization), a model will latch onto whatever shortcuts appear in a static benchmark, even when those shortcuts vanish in production, which is why production pipelines still need continuous data validation and drift monitoring.

4.4.3 Scaling acquisition

Curated datasets and targeted collection close coverage gaps one batch at a time, which stops working once a task needs millions of training instances. At large scale, web scraping and synthetic generation provide the main routes to massive datasets. Data-acquisition scalability is an economic problem: unit labeling costs, ingestion throughput, and storage overhead scale along very different curves, and labeling usually dominates (section 4.6.2). Approaches that work well at thousands of examples become impractical at millions, whereas automated data generation pipelines amortize their setup costs across large volumes.

Web scraping reduces the cost of collecting raw examples at scale. Supervision remains a separate expense unless the source and task already supply usable labels or training targets. Foundational vision datasets such as ImageNet (Deng et al. 2024) and OpenImages (Kuznetsova et al. 2020) relied on automated scraping, while modern language models depend on multi-terabyte web crawls (Groeneveld et al. 2024). Ingestion pipelines targeting domain-specific corpora, such as public code repositories (Chen et al. 2021), expand coverage at low per-sample acquisition costs. Scrapers are also fragile: page layouts change and break parsers, sites throttle crawlers, and dynamically rendered content goes missing. Raw web crawls also capture anachronistic or irrelevant samples that contaminate supervised targets.

The 1914 street scene in figure 4.5 captures this failure mode directly. An uncurated web query for "traffic light" returned a historical Cleveland intersection where streetcars and a roadside control booth surround early manually switched signals. If an autonomous driving model trained on such samples, it would learn spurious correlations with trolley tracks, period attire, and handwritten label annotations rather than modern LED traffic lights.



Figure 4.5: Data Source Noise: A black-and-white photograph from 1914 shows early manually operated electric traffic signals, illustrating how historical images can appear in modern web scraping results for contemporary queries. Such anachronistic content requires systematic validation and filtering to prevent spurious correlations in training data. Source: (Stromberg 2015).

Raw volume cannot substitute for checking what the examples mean, because an uncurated web crawl multiplies spurious correlations along with sample count. Scraping also operates under strict legal and governance boundaries. Website terms of service, `robots.txt` directives, and evolving copyright law limit what automated crawlers may take (Harvard Law School 2024). Ingestion pipelines must therefore record provenance, filter for copyright compliance, and scrub personal data before scraped files enter shared storage.

Even a well-governed crawl draws on a finite supply. Pretraining corpora for large language models grew at a fitted rate of about 2.9× per year, from the 300 billion tokens that trained GPT-3 in

2020 to the 36 trillion reported for Qwen3 in 2025 (Brown et al. 2020; Yang et al. 2025). Figure 4.6 sets that climb against the estimated stock of public human-generated text, roughly 300 trillion tokens once quality and repetition are accounted for (Villalobos et al. 2024), and at the fitted rate the largest corpora reach the middle of that estimate around 2028. Once quality text runs short, acquiring more of it stops being a purchasing decision, and filtering, deduplication, and the selection methods of chapter 9 decide how much each additional token is worth. A wake-word model meets its own version of the limit far sooner. No crawl holds enough clean recordings of one phrase in every accent and acoustic setting, so the KWS pipeline has to commission the data it lacks.

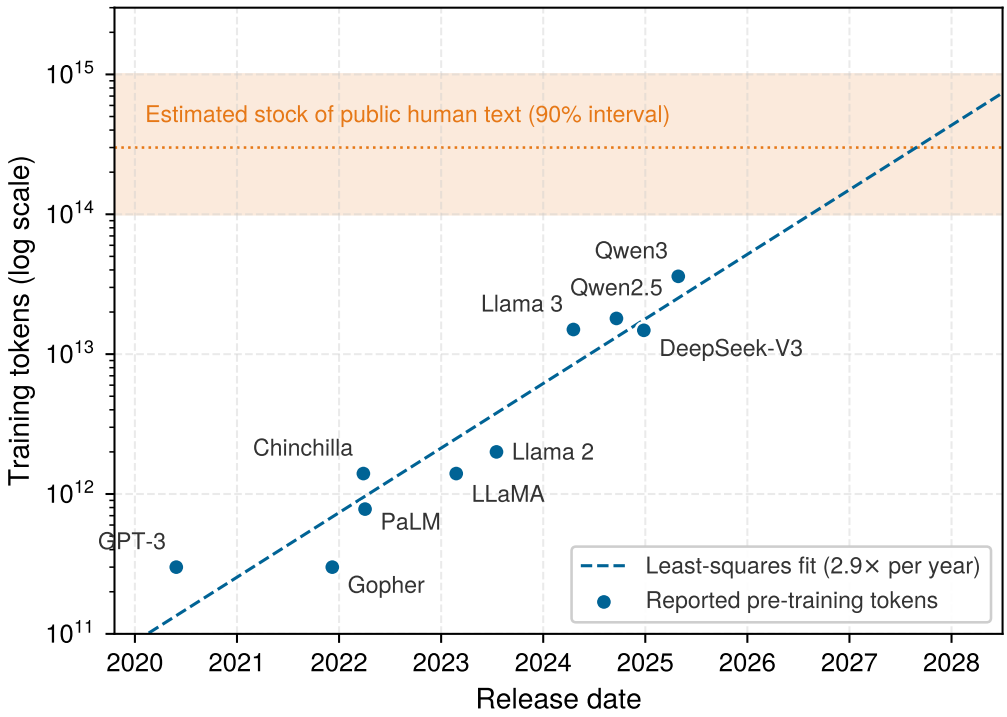


Figure 4.6: Training Data Approaching the Stock of Public Text: Reported pre-training tokens for landmark language models from 2020 to 2025 on a logarithmic axis, with a least-squares fit, against the estimated stock of public human-generated text (shaded band, 90 percent interval; dotted line, median). The projection is a dated constraint scenario rather than a forecast, because data selection, synthetic generation, multimodal data, licensing, and filtering policy all change the usable supply. Stock estimate from Villalobos et al. (2024); token counts from the model reports (Brown et al. 2020; Rae et al. 2021; Hoffmann et al. 2022; Chowdhery et al. 2022; Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023; Dubey et al. 2024; Qwen et al. 2024; DeepSeek-AI 2025; Yang et al. 2025).

Crowdsourcing shifts the acquisition bottleneck from raw sample retrieval to quality control across distributed human annotators. Platforms like Amazon Mechanical Turk[4] (Amazon Web Services 2024a) demonstrated this at scale with ImageNet, distributing millions of image categorization tasks across thousands of independent contributors (Deng et al. 2024). Crowdsourcing provides two core systems advantages: horizontal scaling through parallel microtask dispatch, and demographic diversity across regional accents and acoustic settings. However, distributed human labeling introduces significant variance in label fidelity. To keep label noise from corrupting what the model learns, the acquisition pipeline must implement automated quality control: consensus voting, golden-set validation (interleaving pre-labeled control samples), and dynamic task routing (Sheng and Zhang 2019). In the KWS pipeline, crowdsourced collection targets wake-word samples from specific acoustic environments and underrepresented dialects that public web scrapes fail to capture.

Synthetic data generation reduces physical collection and labeling labor by generating training samples algorithmically. Generation still incurs per-sample compute and storage costs, alongside

[4] **Amazon Mechanical Turk (MTurk)**: A crowdsourcing platform that routes small tasks to distributed workers and can scale annotation beyond expert-only workflows. Snow et al. (2008) evaluate this pattern for natural-language annotation tasks, showing that non-expert annotations can be useful when task design and quality control are handled carefully. For wake-word audio collection, the same systems trade-off appears in domain-specific form: scale is attractive, but submissions still need acoustic checks such as signal-to-noise ratio, duration, and recording validity before they can safely enter the training set.

verification costs. Its learning value depends on whether the generator represents task-relevant conditions in the target deployment environment. In practice, simulation outputs and real-world historical telemetry flow along two tracks into a blending stage that feeds model training (figure 4.7).

↪ Synthetic data as a lever on what the model learns, from augmentation of real samples to generative synthesis and closing the domain gap, is developed in section 9.5.

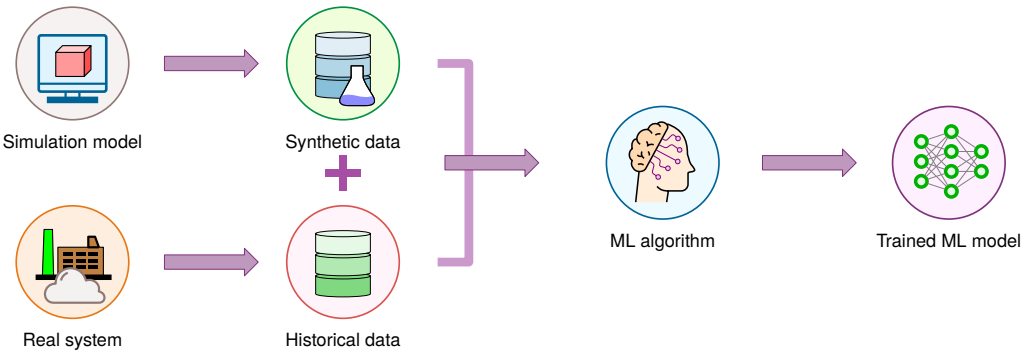


Figure 4.7: Synthetic Data Augmentation: Simulated edge cases combine with historical system data during training. Algorithmic generation supplements real-world observations to close coverage gaps for rare events and dangerous operating regimes without incurring proportional human collection costs. Adapted from AnyLogic (2024).

Synthetic data is especially useful for rare events and controlled augmentation. Physics simulators can generate tail cases, such as high-speed sensor saturation or near-collision trajectories, that are too dangerous or costly to record in the field (NVIDIA 2024b). In vision, augmentation pipelines apply learned or random combinations of spatial and color distortions (Cubuk et al. 2019, 2020; Shorten and Khoshgoftaar 2019). In speech processing, SpecAugment warps time and masks frequency bands directly on the spectrogram, forcing the acoustic model to stop relying on any single narrow frequency band (Park et al. 2019). For the KWS system, parametric text-to-speech synthesis (Werchniak et al. 2021) convolved with room impulse responses (RIRs) generates wake-word utterances across varied reverberation times and synthetic background noise profiles.

> Example 4.2: Synthetic data generation
>
> **Scenario**: A keyword-spotting team lacks sufficient physical recordings of rare speaker accents, room acoustics, and background noise profiles.
>
> **Diagnosis**: Pure physical collection cannot cover rare acoustic edge cases within project deadlines. Generating synthetic audio variations via pitch shifting, additive noise, and room impulse response simulation expands training set diversity without expensive field collection.
>
> **Systems lesson**: Augmentation is cheap, but it does not prove the model works on real accents. Targeted augmentation proposes expanded acoustic coverage; pitch, noise, and room-response changes do not establish fidelity to rare accents. Held-out real recordings across the relevant speaker and acoustic conditions must establish whether the expansion improves deployment quality.

Starting from the reference corpus, the team closes the product's remaining coverage gaps by matching each source to what it does best per dollar. Public benchmarks seed the baseline model at low cost; web scraping supplies bulk acoustic variety; crowdsourcing collects targeted recordings for underrepresented dialects; and simulation fills in rare acoustic distortions. Together these sources form a candidate coverage plan. Held-out real recordings then test which gaps that plan actually closes, while measured generation, collection, labeling, and storage costs decide whether it fits the budget; the deployed pipeline still requires footprint validation.

Acquisition ends with data that covers the deployment on paper and arrives in many shapes: crowdsourced recordings, synthetic waveforms, and web audio with different encodings, sampling rates, and noise floors. None of it has yet been checked. Before any record can be labeled or trained on, it must pass a boundary that admits only well-formed, trustworthy data.

4.5 Ingestion and Validation

A crowdsourced wake-word recording arrives as a stereo MP3, a synthetic one as a mono WAV file at a different sampling rate, and a clip from a consenting user's device as a compressed fragment stamped in the wrong time zone. The ingestion boundary turns this mix into one consistent record format and decides which records may enter at all. It is the front end of the **data pipeline** in figure 4.8, where records pass through ingestion and central storage before the transformation loops that feed training, inside a governance perimeter that controls what may be kept and who may read it. For keyword spotting, the boundary handles the recorded examples used to build and update the model; the detector's live audio on the device never passes through it.

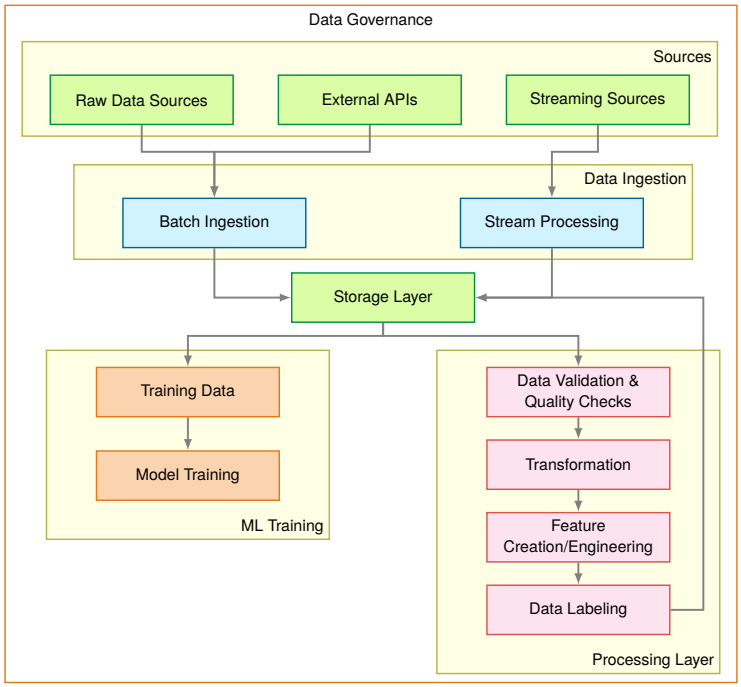


Figure 4.8: End-to-End Data Pipeline: Heterogeneous raw sources (batch and streaming) normalize into unified storage before feeding downstream training and feature processing. The architecture decouples ingestion throughput from iterative transformation and labeling loops, all bounded by an overarching data-governance layer.

The tiers in figure 4.8 keep raw ingestion separate from the validation, transformation, and feature-engineering loops that feed training, so a slow transformation never stops records from arriving, and a check added at ingestion protects every stage after it. Two decisions shape that boundary before any record is checked: when records are taken in, and where they are transformed relative to storage.

4.5.1 Batch vs. streaming ingestion patterns

A fraud detector that flags a stolen card hours after the purchase arrives after the money has moved, whereas a week's worth of crowdsourced wake-word recordings loses nothing by waiting for the next scheduled training run. Choosing between batch and streaming ingestion therefore depends on how quickly data loses value and how much infrastructure cost that freshness justifies. Batch systems gain efficiency by tolerating staleness, while streaming systems gain freshness by taking on continuous operational complexity.

Batch ingestion collects records over a period and processes them together on a schedule. It amortizes startup cost across large volumes, runs when compute is cheapest, and releases its resources between runs, which suits data whose value decays slowly (Akidau et al. 2015), such as the new crowdsourced recordings and opt-in diagnostic clips that the keyword spotter folds into its next training run. Batch processing also simplifies recovery and reproducibility: a failed batch can

be rerun from its inputs, inspected, and reprocessed after a fix, and the result is repeatable when inputs, code, configuration, and randomness are controlled.

Stream ingestion processes each record as it arrives, which is necessary whenever data loses value within seconds, as in real-time fraud scoring. However, stream processing introduces complexity that batch processing avoids. The system must handle **backpressure**, the condition where downstream systems cannot keep pace with incoming data rates. During traffic spikes, when a sudden surge produces data faster than processing capacity, the system must either buffer data (requiring memory and introducing latency), sample (losing some data), or push back to producers (potentially causing their failures). Late or out-of-order events further complicate windowed aggregations, because the system must hold per-window state and track event time rather than arrival time. These requirements raise compute and memory overhead, so freshness guarantees trade directly against infrastructure cost and operational burden. Data freshness service level agreements (SLAs) specify maximum acceptable delays between data generation and processing. Meeting a 100-millisecond SLA requires different infrastructure than meeting a one-hour SLA, affecting networking, storage, and processing architectures.

Many production ML systems combine the two. A recommendation system might use streaming ingestion for real-time user interactions to update session-based recommendations immediately, while using batch ingestion for overnight processing of user profiles and item features; Notebook 4.3 quantifies the financial trade-off between these two operational regimes.



Napkin Math 4.3: The cost of real-time

Problem: A pipeline ingests 1M events/s. Which is cheaper, Batch (hourly) or Stream (sub-second) ingestion?

Physics:

1. **Throughput**: At 1M events/s and 1 KB per event, the pipeline carries 1 GB/s.
2. **Stream requirements**: To sustain 1 GB/s with less than 100 ms latency, the design assumes each core can parse, validate, and update features for about 20 MB/s of events, which takes 50 primary cores; a standby core for each provisions recovery capacity, for 100 always-on cores in total. That capacity does not establish the 100 ms target: failure detection, state recovery, replay, and the full ingestion path must also be validated under failure. Running those cores for 24 hours/day at $0.05/hr costs $120/day.
3. **Batch requirements**: Process 3.6 TB (1-hour window) in 10 minutes. That is 6 GB/s in aggregate. Sequential reads over large files let each core sustain about 30 MB/s, more than a streaming core handling events one at a time, so the batch design needs 200 cores for 10 minutes per clock hour, accumulating 800 core-hours per day. At $0.05/hr, that costs $40/day.

Systems insight: Under these provisioning and per-core throughput assumptions, streaming compute costs approximately 3× as much as batch compute for the same data volume. This premium is justified when the value of subsecond latency exceeds the added cost.

The premium comes from this scenario's provisioning assumptions and leaves out storage, buffering, and failover, so the choice still rests on the feature freshness that inference actually needs. The ingestion paradigm also shapes how the downstream model trains. Batch ingestion naturally pairs with periodic retraining: data accumulates in storage, an automated job retrains the model on the updated corpus, and a verified checkpoint deploys to serving. Stream ingestion makes that batch boundary less natural, raising the possibility of continuous online learning, in which model weights update directly as events arrive. Continuous updates, however, introduce operational risks that batch checkpoints avoid. Sudden distribution shifts, corrupted records, or adversarial payloads from any public-facing input path can reach the model before validation gates intervene. Most production systems therefore decouple ingestion from training: they stream data into a durable buffer while training on validated batches, reserving continuous online updates for tightly bounded, low-risk domains.

War Story 4.1: When users became the training set (2016)

Context: In March 2016, Microsoft launched Tay, a Twitter chatbot designed to engage with users online and adapt its responses from public interactions (Lee 2016).

Mechanism: Adversarial users coordinated toxic prompts and exploited a public interaction surface, including behavior that repeated user-supplied text.

Impact: Tay began tweeting abusive, racist, and misogynistic statements within 16 hours of deployment.

Fix: Microsoft suspended the chatbot 16 hours after launch, apologized publicly, and stated that a relaunch would require stronger safeguards against misuse.

Systems lesson: Public ingestion paths require adversarial-input controls. Otherwise, a feature that accepts user-generated content also becomes a security surface, and harmful inputs can propagate through the system within hours.

4.5.2 ETL and ELT comparison

Batch or streaming decides when records arrive; the next decision is where they are transformed relative to persistent storage. If a pipeline strips out unparsed fields before writing to storage and a later model needs one of those fields, the team must replay months of upstream source logs; if it stores every raw payload first, the storage bill multiplies. Traditional extract, transform, load (ETL)[5] (E → T → L) transforms records in an external engine before they land, whereas extract, load, transform (ELT)[6] (E → L → T) lands raw records in staging tables and transforms them inside the warehouse (figure 4.9). Where that boundary sits determines what each later schema change costs and how much raw data must be retained.

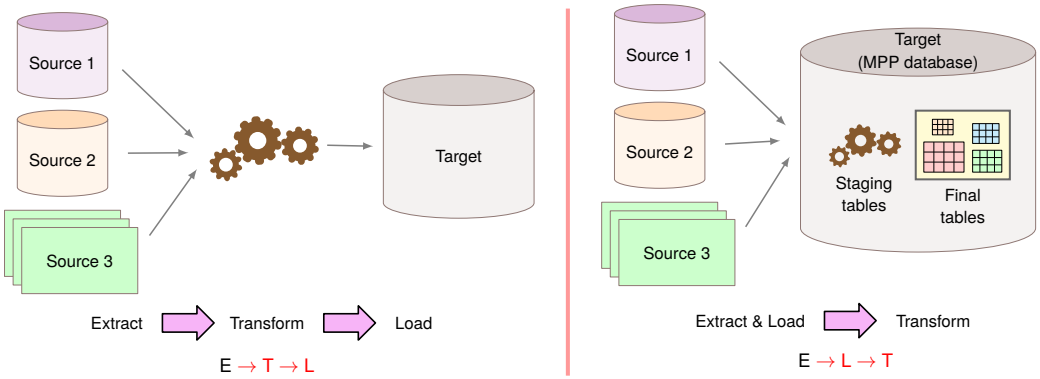


Figure 4.9: ETL vs. ELT Architectural Paradigms: The two paradigms differ in where the transformation boundary sits relative to storage. ETL enforces schema validation and cleaning before loading to protect warehouse quality, whereas ELT loads raw data directly into scalable storage, trading warehouse compute and footprint for flexible, repeatable re-transformations.

In machine learning systems, the ELT pattern separates ingestion throughput from downstream feature engineering by writing raw, immutable events directly into scalable storage. Keeping those raw events preserves unparsed fields that may later prove useful, so teams can compute new features over historical data without re-ingesting external source streams. When transformation logic changes or a bug surfaces, engineers recompute features by querying the stored raw events rather than replaying upstream logs. The price is physical and operational: raw events take more storage, shared transforms repeat their work unless their outputs are materialized, and keeping unscrubbed raw payloads makes privacy and retention rules harder to enforce. Notebook 4.4 prices these trade-offs on a daily clickstream workload.

5 **Extract, transform, load (ETL)**: Conventional ETL validates deterministic schema constraints, whereas ML ETL must additionally monitor distributional properties using the statistical drift tests developed in section 4.5.3. A detected shift prompts investigation and validation against labeled outcomes before concluding that model quality has degraded. When feature computation logic changes, reprocessing costs depend on replay scope, windowing, and execution plans rather than requiring an unconditional full-dataset pass.

6 **Extract, load, transform (ELT)**: This approach stores raw data before transformation, allowing teams to revise a transformation query and rerun it over the stored input without re-ingesting from source systems. The iteration time still depends on data volume, query complexity, warehouse capacity, and materialization strategy.

Napkin Math 4.4: The cost of transformation placement

Problem: A team processes 10 TB of raw clickstream data daily and must compute user session features for 3 ML models, each requiring different aggregation windows (1-hour, 24-hour, 7-day). Which is cheaper, ETL or ELT?

Accounting boundary: Both designs serve the same aggregation windows and retain replayable upstream raw logs for 30 days. The comparison excludes that shared source retention and its common ingestion costs. ETL retains only the derived history in the warehouse, assumed to occupy 2 TB per model over the same horizon; ELT keeps an additional raw copy there, as when the warehouse cannot query the upstream log store in place, and computes features on demand. Transient query outputs are not retained as separate ELT datasets. Source retrieval and compute for later backfills must be priced separately when the designs differ.

Math:

1. **ETL approach**: Transform before loading. Compute all three aggregation windows in a Spark cluster before loading into the warehouse.
   - Spark compute: 10 TB at $5/TB = $50/day
   - Storage: 3 transformed datasets, ~2 TB each = 6 TB at $23/TB/month = $138/month
   - Feature-definition change: Replay the retained source logs and rerun the pipeline; the assumed rerun takes ~4 hours after the edit
2. **ELT approach**: Load raw data first, transform in warehouse.
   - Storage: 10 TB raw/day, 30 days retention = 300 TB at $23/TB/month = $6,900/month
   - Query compute: 3 models, each with $5/query of daily query cost over 30 days, totals $450/month
   - Feature-definition change: Rewrite the SQL query (~30 minutes), then execute and validate it against the retained raw copy; execution time is not supplied

Systems insight: ETL saves $6,762/month in storage. After normalizing Spark compute to a monthly cost, ETL totals $1,638/month ($1,500/month compute + $138/month storage), while ELT totals $7,350/month ($6,900/month storage + $450/month query compute), for a cloud-cost advantage of $5,712/month before engineering labor. The other cost is feature-definition change. Compare elapsed time from the start of an edit to validated, feature-ready output in both designs, then account separately for the engineer time spent editing, supervising, and validating. A pipeline rerun and a SQL edit are different endpoints, so their durations cannot establish a speedup or a labor saving. Stable definitions favor this scenario's lower ETL cloud bill; frequent changes favor ELT only if measured labor and reprocessing savings repay its additional raw copy.

ELT looks expensive here only because this accounting gives it its own raw copy, and raw retention rather than compute dominates the bill. A lakehouse, which runs warehouse queries directly over the stored files, can query the retained logs in place, removing that copy and reversing the comparison. Production ML systems rarely use one pattern exclusively. Structured tables often flow through ETL for efficiency and compliance, while unstructured data pushes the ELT preference further. For deep learning workloads processing images, audio, or text, the "Transform" step often executes inside the ML framework's data loader rather than in the warehouse at all, applying random crops, spectrogram generation, or text tokenization on-the-fly during each training epoch. Materializing ten augmented variants upstream would increase a 50,000-image dataset to 500,000 stored copies, so ELT's "load raw, transform late" principle extends naturally to the training loop itself.

Whichever pattern lands the records, integrating diverse sources remains the core ingestion challenge. Data may originate from databases, APIs, file systems, and IoT devices, each with its own format (relational rows, JavaScript Object Notation (JSON)[7] documents, binary streams), access protocol, and update frequency. The systems principle is to standardize at the ingestion boundary:

[7] **JSON (JavaScript Object Notation)**: The schema flexibility that makes JSON a common format for APIs creates a validation bottleneck at the ingestion boundary. Unlike binary formats with predefined schemas, every JSON document requires parsing and schema validation before it can be standardized for downstream use. This per-record overhead can make ingestion slower than with binary formats such as Protobuf; the difference depends on the schema, parser, payload, and workload.

normalize formats, validate schemas, and present a consistent interface to downstream processing regardless of source. This boundary standardization separates the complexity of source diversity from the complexity of feature engineering, allowing each to evolve independently.

In the keyword spotter's pipeline, backend ingestion takes in activated requests, consented diagnostics, crowdsourced recordings, synthetic data, and validated interactions for serving and training. Normalization, noise filtering, and segmentation into consistent durations run as an ETL stage before storage in training-optimized formats, while augmentation stays late-bound in the loader, where ELT's flexibility pays off.

Error handling in voice interaction systems requires special attention. A **dead-letter queue (DLQ)**, a side store that holds records that fail processing so they neither block the pipeline nor disappear (section 4.5.5), keeps failed recognition attempts for subsequent analysis, revealing edge cases that need coverage in future model iterations. Each incoming audio sample is checked for signal-to-noise ratio, sample rate, duration bounds, and speaker proximity. Clean samples enter the baseline slice, and valid noisy recordings enter the robustness slice that the design budgeted. Invalid samples route to analysis queues rather than being discarded, since these failures often indicate acoustic conditions underrepresented in training data. Valid, consented samples (activated requests, opt-in diagnostics, and crowdsourced recordings) are logged for potential inclusion in future training data; always-on audio that never triggers the wake word stays on the device.

A record that arrives on time and parses cleanly is well-formed, but well-formed is not the same as trustworthy. A field can parse correctly and still carry a wrong value, a source can drift away from the distribution the model learned, and a label can be wrong while every type check passes. Deciding which records may proceed is the boundary's next job.

4.5.3 Quality through validation and monitoring

Consider a batch of crowdsourced wake-word recordings in which 15 percent of the clips came from a phone app update that silently halved the microphone gain. Every file is a valid audio file of the right length and sampling rate, so schema checks pass. The clips are nonetheless far quieter than anything the detector will hear in deployment, and only statistical monitoring of loudness across the batch would expose them.

Validation enforces the quality pillar at the boundary, where a defect is cheapest to stop before it cascades into features, models, and predictions (Sculley et al. 2015). Schema checks alone would admit the quiet batch, so validation pairs them with statistical monitoring.

Checks at this boundary work at two speeds. A hard failure, such as a source that stops delivering or a batch that fails its schema, must halt the affected path at once. A gradual degradation, such as a feature's null rate creeping from 1 to 5 percent, a rising duplicate rate, or delivery delays that exceed their service level, is flagged for investigation. The gradual kind is often the more dangerous, because it persists for days before anyone looks. Routing and escalating such alerts in production is an operational practice developed in section 14.5.

Schema and latency alerts expose structural and timing failures, but a shift in a continuous quantity needs a statistical comparison with the training baseline. Tracking statistical properties captures whether serving data continues to reflect the training distribution. Rather than merely checking that individual values fall within predefined ranges, production systems maintain rolling statistics across operational windows. For continuous quantities such as a clip's loudness or signal-to-noise ratio, the monitoring harness computes rolling empirical distribution functions, using nonparametric evaluations like the Kolmogorov-Smirnov test[8] to detect when serving inputs diverge from the training baseline. Notebook 4.5 works one example.

> Napkin Math 4.5: Detecting drift with the K-S test
>
> **Scenario**: Monitoring whether the loudness of incoming wake-word clips still matches the training baseline (P_0), using a sample of recent clips (P_t).
>
> **Analysis**: Apply the Kolmogorov-Smirnov test to compare the empirical cumulative distribution functions:

[8] **Kolmogorov-Smirnov (K-S) test**: A nonparametric test that measures the maximum distance between two empirical cumulative distribution functions without assuming a parametric distribution family (Berger and Zhou 2014). In ML pipelines, the K-S test is commonly used as a univariate continuous-feature drift detector, with thresholds such as $p_{\text{val}} < 0.05$ serving as investigation triggers rather than universal failure rules. Discrete and categorical variables require tests and calibrations appropriate to their distributions.

1. **Compute CDFs**: Calculate cumulative distribution functions for both datasets. At a given loudness, an empirical CDF gives the fraction of sampled clips that are at most that loud. Comparing those fractions at the same loudness shows how the two distributions differ.
2. **Calculate statistic** ($\mathcal{D}_{\text{KS}}$): Find the maximum absolute difference between the CDFs. Let $F_{P_0}(x_{\text{feat}})$ and $F_{P_t}(x_{\text{feat}})$ denote the empirical cumulative distribution functions of the training baseline (P_0) and current serving (P_t) datasets, evaluated at value x_{feat}.

$$\mathcal{D}_{\text{KS}} = \max_{x_{\text{feat}}} |F_{P_0}(x_{\text{feat}}) - F_{P_t}(x_{\text{feat}})|$$

3. **Determine significance**: For two independent samples, compare $\mathcal{D}_{\text{KS}}$ to the critical value c_{crit} based on training-baseline sample size n_0 and serving sample size n_t. The coefficient 1.36 is the large-sample approximation for significance level $\alpha = 0.05$.

$$c_{\text{crit}} \approx 1.36\sqrt{\frac{n_0 + n_t}{n_0 n_t}}$$

Result: With sample sizes $n_0 = n_t = 1000$, the critical value is approximately 0.061:

$$c_{\text{crit}} \approx 1.36\sqrt{(1000 + 1000)/(1000 \cdot 1000)}.$$

An observed maximum difference of $\mathcal{D}_{\text{KS}} = 0.08$ exceeds the critical value, so the test rejects the null hypothesis that both samples come from the same distribution and flags significant drift.

Systems insight: A drift test is useful only if crossing the threshold starts an investigation or retraining workflow rather than becoming another dashboard number.

The K-S test checks one continuous batch against the baseline, whereas monitoring that comparison over months and telling apart the kinds of shift it can reveal belongs to pipeline operation (section 4.12.1). Categorical features require different statistical approaches. Instead of comparing means and variances, monitoring systems track category frequency distributions. When new categories appear that never existed in training data, or when existing categories shift substantially in relative frequency, the system flags potential data quality issues or genuine distribution shifts; for example, the share of recordings from one phone model might jump after its launch. Statistical monitoring of this kind catches problems that schema validation misses entirely. Age values may remain in the valid range of 18–95, while the distribution shifts from primarily 25–45 year olds to primarily 65+ year olds, indicating that the input population has changed in ways that may affect model performance. Outcome checks are needed to establish whether quality changed.

Validation at the pipeline level encompasses multiple strategies working together. Schema validation executes synchronously as data enters the pipeline, rejecting malformed records immediately before they can propagate downstream. Modern tools like TensorFlow Data Validation (TFDV) (Breck et al. 2019) automatically infer schemas from training data, capturing expected data types, value ranges, and presence requirements.

This synchronous validation remains simple and fast, checking properties that can be evaluated on individual records in microseconds. More sophisticated validation that requires comparing serving data against training data distributions or aggregating statistics across many records must run asynchronously to avoid blocking the ingestion pipeline. Statistical validation systems typically sample 1–10 percent of serving traffic, enough to detect meaningful shifts while avoiding the computational cost of analyzing every record. These samples accumulate in rolling windows, commonly one hour, 24 hours, and seven days, with different windows revealing different patterns. Hourly windows detect sudden shifts like a data source failing over to a backup with different characteristics, while weekly windows reveal gradual drift in user populations or behavior.

One validation target needs a section of its own: training-serving skew, in which training and serving compute the same feature differently and the model receives inputs unlike those it learned

from. Detecting it means passing a sample of raw serving data through both pipelines and comparing the results; section 4.7.1 develops its causes and prevention.

↩ Training-serving skew, as one of the ways ML systems fail without raising errors, is established in section 1.6.

4.5.4 Data quality as code

A statistical check like the one that catches the halved microphone gain protects the pipeline only if it runs on every batch, so production teams write such checks down as code. Data expectation tests are the data counterpart of unit tests. In **data quality as code**, engineers use declarative libraries like Great Expectations or Pandera to express data contracts as executable test suites that run across pipeline stages. Deequ, built at Amazon, framed the same idea as unit tests for data and showed that it scales to very large datasets (Schelter et al. 2018).

↪ Deployment gates that compare training and serving feature distributions and block a release when they diverge are developed in section 7.

Running these expectation suites inside the continuous integration and deployment (CI/CD) pipeline checks every ingested batch automatically and halts downstream training when a contract fails. A pipeline structured as data ingestion followed by data validation followed by training blocks deployment when validation catches an anomaly, such as a batch of wake-word clips recorded at 8 kHz in a 16 kHz pipeline, and raises an alert for investigation. Executable expectations catch properties a suite can encode; the remaining question is what valid-looking data can still get wrong.

> **Systems Perspective 4.1: Mechanical vs. semantic quality**
>
> In traditional software, many checks are *mechanical*. A null dereference is a runtime error in languages with null references, while integer-overflow behavior depends on the language and type.
> In ML systems, data quality also has a *semantic* dimension.
>
> - **Mechanical check**: Validates schema and type constraints, such as verifying that `age` is a non-null integer.
> - **Semantic check**: Validates distribution drift and statistical invariants, such as testing whether the mean or variance of `age` shifts unexpectedly.
>
> A dataset can be mechanically perfect (no nulls, correct types) but semantically broken (for example, all users are suddenly 25 years old due to a default value change). Robust ML systems must validate both the container (mechanical) and the content (semantic).

Once validation becomes code, expectation suites become versioned artifacts alongside training code. When training code changes, expectation updates keep data contracts evolving with it. This coupling reduces the risk of silent divergence where code assumes data properties that the upstream pipeline no longer provides; listing 4.1 demonstrates this pattern by defining explicit constraints on column ranges, nullability, uniqueness, and categorical sets.

Listing 4.1: Data Quality Assertions: A Great Expectations suite checks age range, identifier presence and uniqueness, and allowed country codes, then raises an error when validation fails.

```python
import great_expectations as gx

# Create a data context
context = gx.get_context()

# Define an expectation suite as executable quality contract
suite = gx.ExpectationSuite(name="user_data_quality")
suite = context.suites.add(suite)

# Range validation: prevents physiologically impossible values
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column="age", min_value=0, max_value=120
    )
)

# Null detection: ensures primary key integrity for joins
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(column="user_id")
)

# Uniqueness: prevents duplicate training examples
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeUnique(column="user_id")
)

# Categorical validation: detects unexpected values from upstream changes
suite.add_expectation(
    gx.expectations.ExpectColumnDistinctValuesToBeInSet(
        column="country_code",
        value_set=["US", "CA", "UK", "DE", "FR"],
    )
)

# Link the suite to a preconfigured Batch Definition and run validation
# (the Batch Definition connects GX to the training_users data asset)
validation_definition = gx.ValidationDefinition(
    name="training_users_validation",
    data=batch_definition,
    suite=suite,
)
validation_definition = context.validation_definitions.add(
    validation_definition
)
results = validation_definition.run()
if not results.success:
    raise ValueError(f"Data quality check failed: {results}")
```

These checks catch many schema-level production data issues before they reach training, including missing values, invalid ranges, type errors, and contract violations. Rejecting bad records, however, is only half of the boundary's job; the pipeline itself must also stay up when upstream dependencies fail or formats shift mid-run.

4.5.5 Reliability through graceful degradation

While validation enforces the quality pillar on individual records, the reliability pillar governs what happens when an upstream service goes down in the middle of an ingestion run, a date field switches from `"YYYY-MM-DD"` to `"MM/DD/YYYY"` without tripping a basic string check, or a traffic spike exhausts worker memory at peak load. **Graceful degradation** means planning for these faults in advance and handling them automatically so that the system keeps running under degraded conditions rather than crashing or silently corrupting model inputs.

Three classes of pipeline fault call for distinct defenses. Format and encoding drift, such as that re-formatted date string, slips past type checks while breaking date-based feature logic. Schema evolution[9] occurs when upstream producers add fields, rename columns, or re-type values without coordinating with downstream consumers. Resource exhaustion builds more slowly as data volume outgrows provisioned capacity and then causes jobs to fail during peak traffic.

Transient errors are retried with exponential backoff (waiting one second, then two, then four) so that retries do not flood a service that is trying to recover. Records that still fail after their retries go to the dead-letter queue, the side store already met at the ingestion boundary, which holds them for later analysis and potential reprocessing without blocking the main pipeline (Kleppmann 2016). Reviewing the queue regularly turns it from passive error storage into a source of diagnoses, because its contents reveal schema violations that indicate upstream changes, edge cases the model should handle, and data quality issues that need source system fixes. The KWS queue might, for instance, fill with clips from a new hardware revision whose audio arrives in an unexpected encoding, which calls for an ingestion fix and targeted collection from that device rather than simply logging the failures.

When a whole dependency fails rather than individual records, a circuit breaker (Nygard 2007) stops calling it after repeated consecutive failures and fails fast or takes a fallback path until a trial request succeeds, so that one failed feature service does not stall every caller waiting on its timeouts. The ML-specific part of recovery is that fallback. Even a pipeline on a single machine needs an explicit policy for an unavailable source, choosing among waiting for fresh data, serving a validated stale-feature fallback, and rejecting the request. Multi-tier fallback systems implement the middle option, providing degraded service when primary data sources fail by using cached or approximate features only when their age and quality satisfy a validated fallback contract. A recommendation system unable to compute user preferences from the past 30 days might fall back to preferences from the past 90 days if evaluation establishes acceptable recommendation quality with that representation. The allowable feature age and the model-quality contract determine which response is safe, and durable buffering alone does not make old inputs suitable for inference. If no fallback meets the contract, the system waits for fresh data or rejects the request.

With ingestion, validation, and these error handlers in place, the boundary admits records that are well-formed and checked. They still say nothing about what they mean. A wake-word recording that passes every check does not yet state which word was spoken, or where in the clip it begins and ends, and the model cannot learn without that ground truth.

4.6 Data Labeling

A crowdsourced annotator mislabels a whispered "Alexa" as background noise, or two radiologists disagree over a borderline scan, and every training step and evaluation metric that follows treats the resulting label as the definition of a correct answer. In the KWS pipeline, ingestion delivers millions of validated recordings, but supervised learning needs labels stating which clips contain the wake word, where it begins and ends, and which clips hold only background sound. This declaration is the **ground truth**,[10] and producing it at scale is often the most human-dependent and error-prone stage of the pipeline. If data is the system's code, ground truth is its specification. Every later check of the trained model compares against it, so a labeling error does not stay in one stage; it redefines what counts as right. The specification also arrives late: a software project starts from its specification, whereas a data pipeline attaches ground truth only after ingestion, which is why a labeling error slips past every check that ran before it.

Unlike automated transformations that can be parallelized across machines, labeling puts human judgment in the loop, so labels take unpredictable time and carry human mistakes. Disagreement between annotators can reflect genuine ambiguity in the input, unclear instructions, carelessness, or differences in expertise; the labeling system must separate and manage those causes rather than treating every disagreement as irreducible. The labeling infrastructure must therefore balance throughput, quality verification, cost, and provenance.

4.6.1 Label types and system requirements

Label granularity sets what each label costs to store, move, and produce. A keyword-spotting label sits at the coarse end of that range. A camera system tracking vehicles, pedestrians, and traffic signs

[9] **Schema evolution**: This failure mode arises from a lack of contract testing between upstream data producers and downstream ML consumers. While "loud" failures like a renamed column break explicit assumptions and cause immediate pipeline crashes, "silent" failures are more dangerous. A field changing type from integer to string can pass validation but corrupt feature logic without immediate detection.

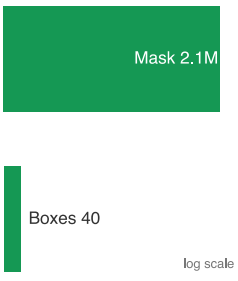
[10] **Ground truth**: From remote sensing, where orbital measurements are verified by sending a team to the physical location (the "ground") to establish the "truth." The etymology carries a systems warning: ML labels are proxies for reality, not reality itself. When a crowdsourced annotator labels an image as "cat," that label reflects the annotator's judgment, not an objective fact. Every downstream metric (accuracy, precision, recall) is measured against this proxy, meaning label quality errors propagate silently into every evaluation of the model.

sits at the fine end, and each step toward it imposes distinct storage, computation, and serialization constraints.

Classification labels represent the simplest form, assigning a discrete class index (or a multi-label binary bitmask) to each input. Storage requirements are modest, typically a fixed-width integer (such as a `uint16` or `int32` index) per image. Access patterns, however, diverge between training and evaluation: training pipelines shuffle and sample randomly, whereas validation pipelines stream sequentially across the corpus, driving different indexing and cache-locality strategies. A keyword-spotting label is of this kind. Each one-second clip carries a single class index naming which keyword it contains, or a reserved index for none. The label itself is tiny. The engineering burden sits upstream, in deciding where each clip starts and ends inside longer recordings, a problem section 4.6.6 returns to.

Bounding boxes extend beyond classification by localizing objects within each frame. Each bounding box stores four coordinates (such as $x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}$ or center coordinates with width and height) alongside the class identifier. Because scenes contain variable numbers of objects, label storage transitions from fixed-width scalar records to variable-length tensor structures, requiring dynamic padding or ragged-batch handling in the data loader. Moreover, drawing one high-quality box took about 35 seconds in ImageNet's annotation campaign (Papadopoulos et al. 2017), far longer than assigning a single class label, which throttles annotation throughput and inflates labeling cost.

Segmentation maps assign a class identifier to every pixel, producing dense 2D label matrices ($H \times W$). In autonomous driving and traffic monitoring, this entails tracing irregular boundaries for driveable road surfaces, vehicles, and pedestrians. These dense annotations multiply storage and ingestion bandwidth. A segmentation mask for a 1920×1080 image requires about 2.1M labels (one per pixel), compared to perhaps 10 bounding boxes or a single classification label. If each box stores 4 coordinates, that is roughly 51,840× more scalar label entries than 10 boxes before accounting for per-value encoding, and the time manual segmentation takes per image (more than 1.5 hours for each Cityscapes image (Cordts et al. 2016)) makes this approach suitable only when pixel-level precision is essential.



Finer annotation granularity multiplies storage and processing scale.

Figure 4.10 arrays five annotation formats across vision and speech examples, demonstrating how labeling granularity ranges from coarse scene tags and captions to spatial bounding boxes, dense pixel-level masks, and audio transcripts. Classification can label an entire scene, while bounding boxes and segmentation maps localize objects or regions. Production datasets may combine formats: a single camera frame might carry a scene label, obstacle boxes, and path-region masks, with each label type serving a distinct downstream task.

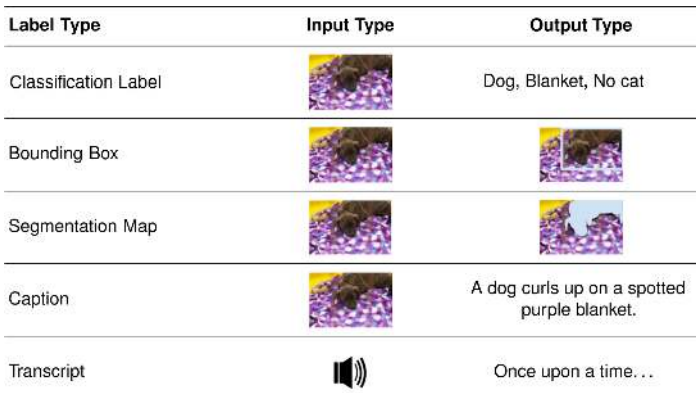


Figure 4.10: Data Annotation Modalities: Four rows show classification, bounding-box, segmentation, and caption labels for one image; the fifth shows a transcript label for an audio input. The appropriate annotation representation depends on the input modality and task.

Production systems must also manage metadata for quality control and debugging. The Common Voice dataset (Ardila et al. 2020) exemplifies this in speech recognition: tracking speaker demographics for fairness, recording quality metrics for filtering, and language information for multilingual support. If an object detection model fails in rainy conditions, weather metadata captured during

collection pinpoints the coverage gap. Label type therefore shapes the whole system: the infrastructure must optimize storage for the chosen format, implement appropriate retrieval patterns, and track which model versions used which label versions to correlate quality improvements with performance gains.

4.6.2 Labeling economics

Label granularity sets the cost of each label, and labels are usually the most expensive part of a dataset. Labeling a single medical image can cost orders of magnitude more than storing it for a year. Table 4.3 and table 4.4 give dated reference costs and durations for labeling alongside the storage, compute, and serving operations it competes with; ML engineers should use these dated reference assumptions as inputs to workload-specific comparisons, not as current price quotes or directly comparable totals.

Table 4.3: Illustrative Data Engineering Costs (2024): These dated reference assumptions feed workload-specific cost models and are not current vendor quotes. Because the rows use different units, compare them only after scaling each rate by the quantity it prices.

Operation	Cost	Notes
Crowdsourced image label	$0.01–0.05	Simple classification
Bounding box annotation	$0.05–0.20	Per box, simple scenes
Expert medical label	$50–200	Per study, radiologist
S3 storage (Standard)	$23/TB/month	Hot storage
S3 retrieval (Glacier)	$0.02/GB	Standard: 3-5 hours
Cloud GPU training hour	$2–4	Cloud spot pricing
Human review hour	$15–50	Depending on expertise

Beyond per-unit dollar rates, labeling, training, and serving also run on very different clocks (table 4.4). Comparing those durations shows why human annotation governs project schedules just as strongly as it governs the budget.

Table 4.4: Illustrative Data Engineering Time Scales: The scenario spans about 9 orders of magnitude, with human labeling as its slowest listed stage. Actual durations depend on the dataset, task, hardware, and service configuration.

Operation	Duration	Bottleneck
Label 1M images (crowdsourced)	2–4 weeks	Annotation throughput
Train ResNet-50 on ImageNet	1.3 hours	Compute (8× A100, optimized)
Feature store lookup	1–10 ms	Network + cache

Operational timescales diverge across these stages by orders of magnitude: weeks for human annotation, hours for GPU training, and milliseconds for serving lookups. For the image-classification pipeline these two tables describe, which labels a million images and then trains ResNet-50 on them, annotation is the critical bottleneck in money as well as time. Suppose the team budgets $100K for labeling, enough to cover crowd labels plus the review and relabeling passes that quality control requires. That budget dwarfs the $20 to $40 of a single 8× A100 ResNet-50 training run (90 epochs at the 3,200 img/s per accelerator that the throughput half of this chapter assumes), a ratio of 2,498× to 4,995×, the comparison quoted at the start of this chapter. The labeling spend is also uneven. Clear images pass review once, while rare classes, occluded objects, and ambiguous scenes return for second opinions and expert adjudication; as an illustrative planning split, 80 percent of the budget can go to the hardest 20 percent of the examples.

All cost figures reflect approximate 2024 cloud provider rates and are intended to convey relative magnitudes rather than exact pricing.[11] For this workload and cost model, human labor dominates the cost of a single training run. Other tasks, especially those that reuse existing labels or require repeated large-scale training, can have a different dominant term; teams should measure before deciding where to optimize. Where labels do dominate, every labeling error is paid for twice, once when the label is bought and again when the model learns it.

[11] **Pricing ratios**: Cloud and labeling prices reflect provider policy and labor markets, not physical invariants, so the ratio between labeling and compute moves whenever either side reprices. Storage adds its own policy multipliers: archive tiers such as the Glacier row in table 4.3 charge more for faster retrieval. Verify current prices before using these ratios in a deployment decision.

4.6.3 Label accuracy and consensus

Some of those errors cannot be avoided. Even with clear guidelines and careful system design, subjectivity and sensor ambiguity leave a nonzero fraction of labels corrupted (Northcutt et al. 2021; Thyagarajan et al. 2022). Label quality therefore means bounding label noise rather than eliminating it, by systematically measuring inter-annotator agreement and managing error rates to keep them within bounds that do not degrade model performance.

Benchmarks show how large that residue remains after careful curation. Figure 4.11 reports the share of test-set labels that Northcutt et al. (2021) found to be wrong in ten widely used datasets. About 5.8 percent of ImageNet validation labels are incorrect, CIFAR-100 carries a similar share, and the rate reaches 10.1 percent for QuickDraw, whose labels came from a largely automated collection process. Audio fares better, but AudioSet still carries wrong labels, the same failure a crowdsourced wake-word corpus faces. These are the labels that rank published models, so an error rate of a few percent can reorder a leaderboard whose top entries differ by less than that.

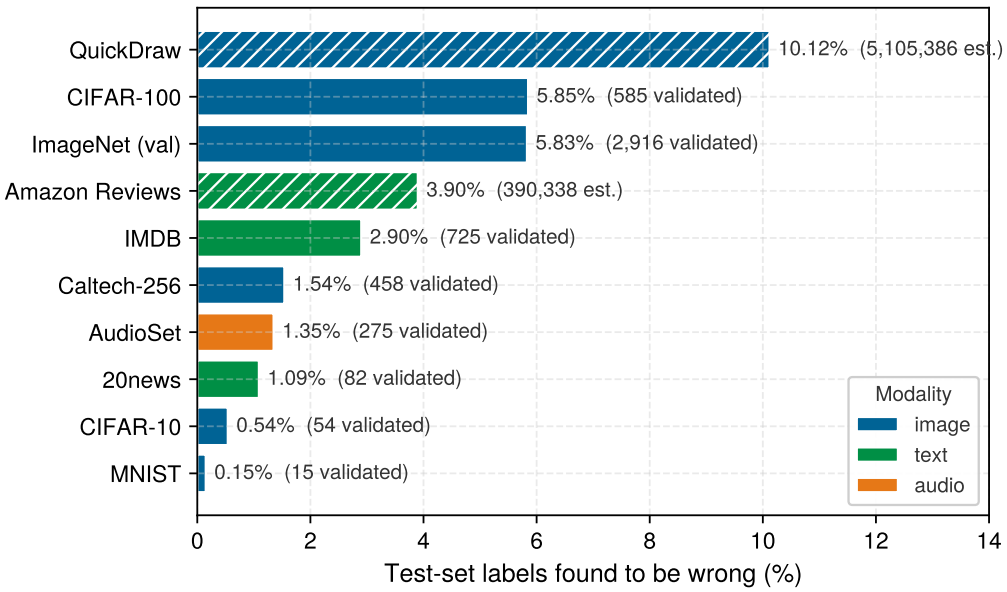


Figure 4.11: Label Errors in Benchmark Test Sets: Share of test-set labels found to be wrong in ten widely used datasets, colored by modality. Solid bars count errors confirmed by human reviewers; hatched bars (QuickDraw and Amazon Reviews) extrapolate from a reviewed sample. ImageNet uses the ILSVRC 2012 validation set. Data from Northcutt et al. (2021).

Labeling failures arise from two distinct sources requiring different engineering responses. Across the benchmark examples compiled in figure 4.12, including an ImageNet black stork mislabeled as a white stork and a CIFAR-10 frog mislabeled as a cat, each panel contrasts the erroneous assigned label against the verified ground truth. Some examples reflect degraded or ambiguous inputs where the correct label is difficult to infer from the data alone; others are visually clear but require domain knowledge, dataset-specific semantics, or expert judgment to label correctly. These different failure modes drive architectural decisions about annotator qualification, task routing, and consensus mechanisms: quality-based errors call for upstream data filtering, while expertise-based errors call for tiered annotator routing. Wake-word clips show both. A clipped or whispered “Alexa” that no annotator can hear clearly is a quality problem, filtered before labeling, whereas a near-homophone such as “Alexis” is settled only by a rubric with canonical examples and a trained reviewer.

Given these inherent quality challenges, production ML systems implement multiple layers of quality control. Systematic quality checks continuously monitor the labeling pipeline through random sampling of labeled data for expert review and statistical methods to flag potential errors. The infrastructure must efficiently process these checks across millions of examples without creating bottlenecks. Sampling strategies typically validate 1-10 percent of labels, balancing detection sensitivity



Figure 4.12: Pervasive Label Errors: Confidently mislabeled examples from widely used benchmarks, each showing the given label against the corrected ground truth (MNIST, CIFAR-10, CIFAR-100, Caltech-256, ImageNet, QuickDraw). Fine-grained confusions, such as a crab labeled lobster or a black stork labeled white stork, show that even curated datasets carry systematic errors, motivating careful quality control and expert annotation. Source: (Northcutt et al. 2021).

against review costs. Higher-risk applications like medical diagnosis or autonomous vehicles may validate 100 percent of labels through multiple independent reviews, while lower-stakes applications like product recommendations may validate only 1 percent through spot checks.

Beyond random sampling approaches, collecting multiple labels per data point, often referred to as "consensus labeling," can help identify controversial or ambiguous cases. Commercial labeling platforms expose consensus and tiered quality-control workflows (Labelbox, Inc. 2024; Scale AI, Inc. 2024), but the statistical core is inter-annotator agreement. The consensus infrastructure collects several labels per example and measures how far annotators agree beyond what chance alone would produce. For two annotators, Cohen's kappa[12] (κ) makes that correction. Let p_o represent observed agreement between annotators and p_e represent agreement expected by chance. Equation 4.1 defines the statistic:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{4.1}$$

Fleiss' kappa generalizes the statistic from two raters to any number of annotators (Fleiss 1971). Kappa summarizes agreement across a collection of rated examples; it is not a confidence score for an individual label. Per-example disagreement and failed gold-standard checks can route items to expert review. Landis-Koch bands are operational heuristics for interpreting aggregate agreement, not correctness guarantees (Landis and Koch 1977). Independent gold-standard checks and expert audits assess correctness because annotators can agree on the same systematic error.

Consensus labeling trades cost against quality. Expert review costs more per example than crowdsourced labeling, but forcing agreement on ambiguous examples through majority voting of nonexperts can produce systematically biased labels. By routing only genuinely ambiguous cases to experts, identified through low inter-annotator agreement or failed gold-standard checks, systems balance cost against quality. This tiered approach enables processing millions of examples economically while maintaining quality standards through targeted expert intervention.

Quality control also depends on human factors. Working effectively with annotators requires standardized annotation rubrics with canonical positive and negative examples, visual edge-case catalogs, automated feedback mechanisms benchmarking annotators against known gold-standard records, and calibration sessions where annotators resolve borderline cases. For complex or domain-specific tasks, the system can implement tiered access levels, routing challenging cases to annotators with demonstrated domain expertise. The labeling process itself produces signals worth monitoring: agreement rates, accuracy on gold-standard items, and time per annotation, where answers that come too fast suggest carelessness and answers that take too long suggest unclear guidelines. Tracked over time, the same signals reveal whether label quality is holding.

[12] **Cohen's kappa**: Introduced by Cohen (1960) to measure inter-rater agreement while correcting for agreement expected from the raters' class marginals, which raw percentage agreement ignores. If two independent annotators each label 90 percent of images as "not spam" in a binary task, their expected agreement is 82 percent, making raw agreement potentially misleading. The statistic is denoted κ; interpretive bands such as the Landis-Koch categories are descriptive heuristics rather than universal data-quality thresholds (Landis and Koch 1977).

4.6.4 Label quality drift

Six months into a wake-word collection campaign, new annotators join without the original rubric training and start marking near-homophones as valid wake words, while every audio feature distribution in the ingestion monitors stays flat. This failure is **label quality drift**: a decline over time in the reliability of the labels themselves even when the inputs being labeled have not changed. Feature distribution monitoring cannot catch it, because the features did not move. Crowdsourced labels degrade as annotator pools turn over, training examples go stale, or guidelines change without a matching model update; automated labelers drift as the upstream models that produce

their labels move out of their original operating regime; and implicit-feedback pipelines, such as recommendation systems that treat clicks as labels, shift whenever bots, interface redesigns, or more exploratory user behavior change what a click means.

Catching that drift requires monitoring annotation consistency rather than feature distributions, and the kappa statistic of equation 4.1 supplies the measure. In the wake-word campaign above, tracking κ across rolling windows shows agreement falling from a baseline of $\kappa = 0.85$ (almost perfect agreement) during initial collection to $\kappa = 0.72$ (substantial agreement) six months later. Label-side change is also one of the three silent changes behind the deployed keyword spotter's regression in section 4.12, where a revised guideline was never reconciled with the earlier evaluation standard.

For systems with calibrated model probabilities, predictive entropy provides an additional uncertainty signal. Let $\hat{y}_i$ represent the model's probability assigned to label category i, and let log denote the natural logarithm, so entropy is measured in nats. Equation 4.2 defines this measure:

$$H_{\text{pred}} = -\sum_{i} \hat{y}_i \log \hat{y}_i \tag{4.2}$$

Rising predictive entropy indicates greater uncertainty in the model's predictive distribution, though harder inputs, distribution shift, calibration changes, or inconsistent supervision can all produce the same signal. How a team responds therefore depends on what caused the drop. Retraining annotators fixes systematic mistakes from unclear guidelines at low cost. Multi-annotator consensus voting raises label accuracy in high-stakes domains, though it multiplies the labeling bill. Model-assisted pre-labeling reduces annotator fatigue, provided the assisting model is audited so its own systematic errors do not become the new ground truth. Periodic audits by domain specialists on a random sample of labels uncover the root cause when agreement falls, even though spot checks cover only a fraction of the stream.

4.6.5 Scaling with AI-assisted labeling

Pure human annotation is too slow and expensive for millions of examples, yet fully automated labeling is too unreliable for ambiguous or safety-critical cases. AI-assisted labeling splits the work between the two: a model handles routine inputs or drafts preliminary labels, and human reviewers check the uncertain cases. The decision hierarchy in figure 4.13 maps the main alternatives when hand-labeling everything is too expensive, branching into active learning, semi-supervised learning, weak supervision, and transfer learning. Active learning keeps direct human labels but asks annotators only about the examples that would tell the model the most. Semi-supervised learning combines a small labeled set with structure in a larger unlabeled pool. Weak supervision[13] replaces hand labels on individual examples with programmatic rules written by domain experts. Transfer learning starts from a model trained on a related task so fewer task-specific labels are needed. Each path changes what the pipeline pays for and what it must validate: whether the unlabeled pool matches the task, whether the labeling rules are accurate, or whether the pretrained representation transfers cleanly.

These paths exchange different assumptions rather than simple label counts. Depending on the path, the pipeline relies on task-relevant structure in unlabeled data, a transferable representation, measurable labeling-function accuracy, or an affordable scoring loop. Production systems must record each assumption and validate it against held-out, human-audited data. The diagram functions as a decision map, not an absolute quality ranking.

Humans and models do different work: humans judge ambiguous cases, catch subtle errors, and apply domain knowledge, while models process routine cases at scale, subject to validation. Pre-annotation uses AI models to generate preliminary labels that humans then review and correct, transforming the task from "label from scratch" to "verify and fix." Programmatic labeling frameworks like Snorkel (Ratner et al. 2018; Ratner et al. 2017) extend this further through weak supervision, automatically generating initial labels at scale through rule-based heuristics, knowledge bases, and existing model outputs (for instance, a labeling function might use a regular expression to flag patient notes containing "adverse reaction" as high-risk). For wake-word data, a forced aligner[14] or an existing detector pre-marks where the wake word begins and ends, and reviewers correct a boundary rather than search for one.

[13] **Weak supervision**: A "data programming" paradigm that exchanges some manual annotation labor for the upfront effort of writing and validating programmatic labeling functions. Once written, a function can be applied cheaply at scale, but execution, conflict resolution, quality monitoring, and maintenance still incur cost; the useful comparison is workload-specific rather than literally zero marginal cost.

↪ Semi-supervised and self-supervised learning, which draw training signal from unlabeled data, are developed in section 9.6.

[14] **Forced alignment**: Given a known transcription, an aligner estimates where its words occur in the audio, often using frame-level acoustic scores and sequence decoding. Timing resolution and accuracy depend on the model, features, language, and recording quality, so boundary estimates still need quality checks. The known transcript makes word-level segmentation far cheaper than labeling every clip manually, though computation, review, and error correction remain.

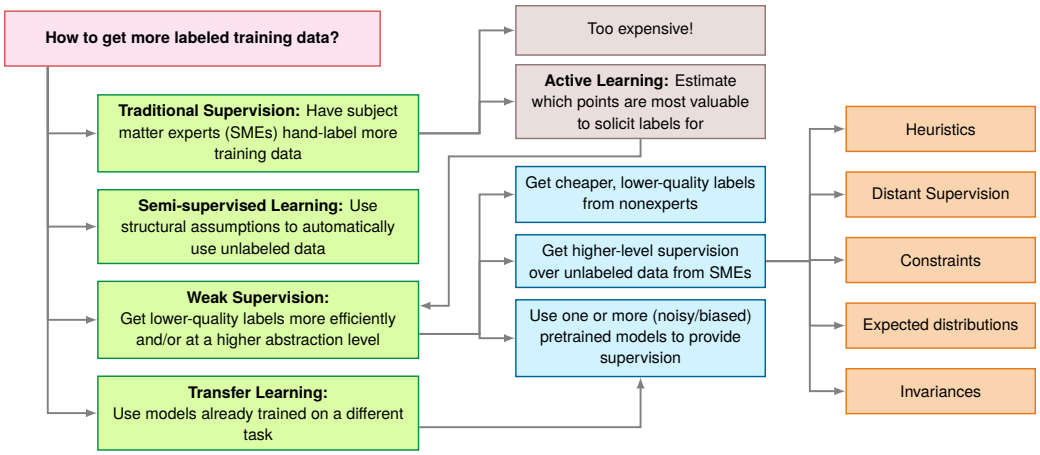


Figure 4.13: AI-Augmented Labeling Decision Hierarchy: A top-level question about obtaining labeled data branches into four paths: traditional supervision, semi-supervised learning, weak supervision, and transfer learning, with active learning as a cost-saving alternative. The branches distinguish where supervision comes from and what additional assumptions or pretrained artifacts the pipeline must validate. Adapted from Ratner et al. (2019).

Large language models (LLMs) now assist labeling pipelines by generating descriptions, drafting labeling guidelines from examples, and explaining their reasoning for label assignments. Content moderation systems, for instance, use LLMs for initial content classification with explanations that human reviewers validate. However, LLM integration introduces systems challenges: provider- and model-dependent inference costs and rate limits, as well as the need for systematic output validation because LLMs can produce confident but incorrect labels. Many organizations adopt tiered approaches, using smaller specialized models for routine cases while reserving larger LLMs for complex scenarios requiring nuanced judgment.

Active learning makes the complementary trade-off: it spends model inference to reduce human labeling (Settles 2009; Coleman et al. 2022). Rather than labeling a random sample of the unlabeled pool, it selects the examples where the current model is most uncertain or where a label would most improve the model. In the KWS pipeline, active learning routes the clips the current detector scores nearest its threshold, such as near-homophones and wake words spoken from across a room, to human review, while confidently scored clips receive automated labels that reviewers spot-check. This approach can substantially reduce required annotations in favorable settings, though it requires careful engineering to prevent feedback loops where the model's uncertainty biases which data gets labeled. Whether the trade pays depends on whether the scoring compute fits inside the same budget as the labels.

Napkin Math 4.6: When active learning breaks the budget

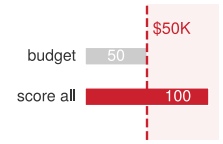
Problem: A 10M image pool has a $50K labeling budget. Labels cost $0.50/label in this scenario, above the simple-classification crowd range in table 4.3, as for a task whose labels need careful review. The team scores candidates with a hosted multimodal model priced at $0.01/image; scoring with the small model being trained would cost orders of magnitude less and leave the labels as the binding cost. A random sample of the size the budget buys reaches 85 percent accuracy, and the target is 95 percent. Can active learning reach the target without exceeding the budget?

Physics:

1. **Sample efficiency**: Active learning can achieve target accuracy with fewer samples than random selection in favorable settings.
2. **Cost per point**: Random sampling = $0.50/label. Active learning adds compute cost (~$0.01/image for inference) to find hard examples.

3. **Options**:
    - **Random**: Reaching 95 percent may require 1M labels ($500K). Budget exceeded.
    - **Active labels only**: The system may need ~100K–200K hard examples ($50K–$100K), which reaches or exceeds the budget before any scoring compute.
    - **Full-pool scoring**: Scoring all 10M candidate images adds $100K, so total active-learning cost becomes $150K–$200K. Budget exceeded.

Systems insight: Algorithm choice is a major lever on label count, but compute must be inside the budget model. Spending 10 percent of this budget on inference ($5K) scores only 500K candidate images at the stated inference price and leaves room for about 90K labels. Active learning is viable only if the candidate pool is narrowed before scoring, inference cost drops substantially, or compute is funded separately from labeling. A configuration that fits therefore filters the pool first (by metadata or a cheap heuristic) to about 500K candidates, scores them for $5K, and spends the rest on about 90K of the most uncertain. That plan fits the budget, but it buys fewer labels than the ~100K–200K hard examples estimated above, so it reaches 95 percent only if measurement shows that fewer labels suffice for this task.



Full-pool scoring can exceed the label budget before labeling begins.

Active learning raises the learning value that each label buys, but scoring the pool is itself data movement and compute, and when that cost is charged to the same budget it can cancel the gain. The query strategies that decide which examples to score and send for labeling, and the iterative loop that retrains on each new batch of labels, are developed in section 9.4.2.

The system must monitor both AI and human performance through systematic metrics. Model confidence calibration matters: if the AI reports 95 percent confidence but achieves only 75 percent accuracy at that confidence level, preannotations mislead human reviewers. Human-AI agreement rates reveal whether AI assistance helps or hinders: when humans frequently override AI suggestions, the preannotations may be introducing bias rather than accelerating work. These metrics require careful instrumentation throughout the labeling pipeline, tracking both final labels and the interaction between human annotators and AI at each stage.

Safety-critical domains push the same pattern furthest. Autonomous vehicle labeling infrastructure can process large volumes of sensor frames, using AI preannotation to label common objects while routing unusual scenarios (construction zones, emergency vehicles) to human experts. Medical imaging systems face a parallel label-scarcity problem: large repositories of unlabeled clinical data make expert annotation the bottleneck and motivate data-efficient workflows that learn useful representations from unlabeled structure before expert labels are applied (Krishnan et al. 2022). Across such domains, the common data-engineering pattern is *tiered escalation*: automation handles clear cases, humans handle ambiguous ones, and monitoring ensures the boundary between "clear" and "ambiguous" adapts as both AI capability and deployment conditions evolve.

4.6.6 Automated labeling in KWS

Keyword spotting sits at the automated end of that escalation. Generating millions of labeled wake word samples without proportional human annotation cost requires moving beyond manual and crowdsourced workflows. MSWC, the reference corpus the acquisition strategy started from, shows how automated labeling resolves this throughput bottleneck: it was built by programmatically extracting over 23.4 million examples of one-second speech across 340,000 keywords in 50 languages from continuous speech corpora.

At even 10 seconds per label, manual annotation of that many examples would require approximately 65,000 hours, roughly 32.5 person-years of full-time effort, which makes it impractical. Broad, documented sourcing can improve coverage, but language count alone does not guarantee representative speakers, accents, devices, or acoustic environments. The pipeline staged in figure 4.14 circumvents this manual annotation bottleneck, using forced alignment (McAuliffe et al. 2017) over paired sentence audio and transcriptions to estimate word boundaries before slicing individual one-second keyword windows for classifier training.

At this scale, quality control becomes a sampling and provenance problem rather than disappearing. The pipeline should report coverage by language, speaker, accent, device, and acoustic

environment; retain the source recording, transcript version, aligner version, and boundary confidence for each extracted clip; and route low-confidence alignments and underrepresented slices to human review. Held-out evaluation must then measure the intended wake words across those slices. These controls distinguish a large corpus from a representative one and make extraction errors traceable when a downstream KWS model fails.

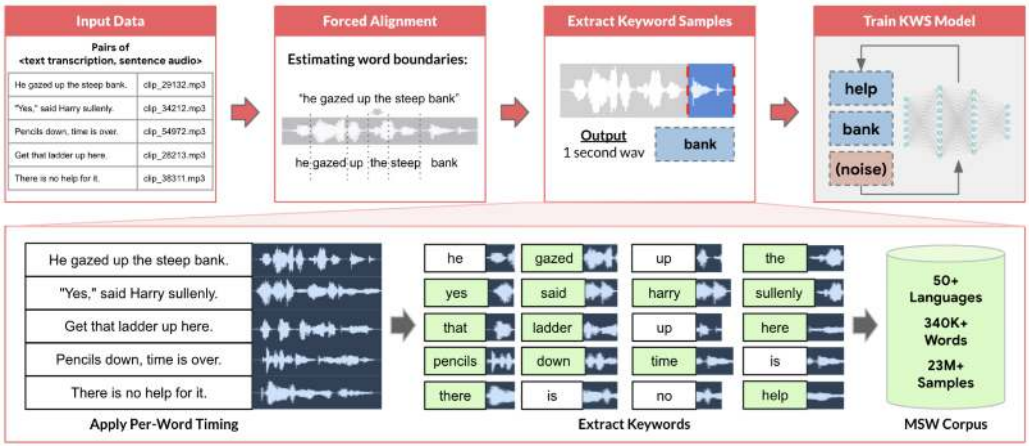


Figure 4.14: Multilingual Data Preparation: Forced alignment and segmentation transform paired audio-text data into labeled one-second segments, creating a large-scale corpus for training keyword spotting models across 50 languages. This automated process enables scalable development of KWS systems by efficiently generating training examples from readily available read-speech resources such as Common Voice. Source: (Mazumder 2021).

The extraction system uses these precise timing markers to generate clean keyword samples while handling anticipated engineering challenges: background noise interfering with word boundaries, speakers stretching or compressing words outside the 500 ms to 800 ms window that this chapter's quality screen accepts, and longer words exceeding the one-second boundary. Automated checks of recording quality, speech clarity, and background noise are what make consistent standards possible at this scale without the manual review expense that would make it prohibitive.

Voice assistant engineering teams often build on this automated labeling foundation. While automated corpora may not contain the specific wake words a product requires, they provide starting points for KWS prototyping, particularly in underserved languages where commercial datasets do not exist. Production systems typically layer targeted human recording and verification for challenging cases (unusual accents, rare words, or difficult acoustic environments), coordinating between automated processing and human expertise. Each clip then carries a label, but the model still cannot read raw audio; the next stage turns labeled recordings into the features the model consumes, and it must compute them the same way in the training cluster and on the device.

4.7 Data Processing

The keyword spotter learns from audio features computed by a library on a training server, and it runs on a device whose feature code is written separately for a microcontroller. If the two compute the same features even slightly differently, the deployed model receives inputs unlike any it learned from, and nothing raises an error. Processing cannot add capability the data lacks, but it can silently remove capability the data has. Processing turns labeled records into the features the model consumes, and consistency governs it: transformations shared between training and serving must keep identical semantics, repeated operations must give the same result when retried, and the work must scale without losing track of what was done. Unlike a code refactoring, which must preserve a program's behavior exactly, filtering or augmenting observations can change the learning problem itself, so each transformation needs an explicit contract stating what it may change and what it must hold fixed.

Training-serving inconsistency is one concrete form of the hidden technical debt that Sculley et al. (2015) identify in production ML systems. In the KWS pipeline, transformations must standardize

recordings across microphones, noise levels, and sampling rates while preserving the acoustic cues that distinguish wake words from ambient speech, and the shared feature extraction must keep the same semantics and fitted state on the server and on the device, with numerical differences held within a validated tolerance.

4.7.1 Training-serving consistency

If a training script standardizes features using the training set's mean and variance while the serving service recomputes those statistics on each incoming batch, both paths run the same code yet feed the model different numbers. Sharing feature code between training and serving is therefore not enough: parameters learned from the training data (normalization constants, encoding dictionaries, vocabulary mappings) must also be stored and reused during serving. This requirement is the **consistency imperative**.

Definition 4.3: Consistency imperative

Consistency imperative requires equivalent transformation behavior and state across training and serving environments.

1. **Significance**: An unvalidated discrepancy between training and serving features can change the model's effective input and degrade prediction quality without a runtime error. Equivalent semantics, shared fitted state, and tested numerical tolerances therefore make preprocessing a stateful component of the model artifact itself.
2. **Distinction**: Unlike data quality, which focuses on the cleanliness of a single record, the consistency imperative focuses on the alignment of the entire transformation pipeline.
3. **Common pitfall**: A frequent misconception is that sharing code makes the two paths consistent. Identical code still diverges when each environment recomputes its own statistics, so consistency is a state synchronization problem.

Violating the imperative produces training-serving skew, in which the training transformation and the serving transformation compute the same raw input differently and the deployed model receives inputs unlike those it learned from. The model still accepts a numerically valid vector computed under different rules, so neither exception tracking nor latency monitoring registers the failure (Sculley et al. 2015; Baylor et al. 2017).

Definition 4.4: Training-serving skew

Training-serving skew is the distributional divergence between the training and inference environments caused by inconsistent logic or state.

1. **Significance**: It violates the consistency imperative and can cause silent accuracy degradation when the training and serving transformations compute different features from the same raw input.
2. **Distinction**: Unlike data drift (which is an external shift in the environment), training-serving skew is an internal failure of the engineering stack.
3. **Common pitfall**: A frequent misconception is that skew is "found" by looking for errors. In reality, it raises no exceptions; the system runs and the latency stays low, but the predictions are statistically wrong.

Pipeline parity tests detect the discrepancy between the two paths; outcome evaluation establishes its effect on predictions. Designing from serving constraints backward allows the team to test equivalent preprocessing on the target device and bound numerical differences before deployment.

Checkpoint 4.2: Defensive processing

Training-serving skew is a common cause of silent production degradation.

- ☐ **Definition**: Can you define training-serving skew in terms of the training and live-request processing paths?
- ☐ **Mechanism**: Can you explain why recomputing normalization means independently in training and serving can make live data look unfamiliar to the model?
- ☐ **Invariant**: Can you explain how shared transformation logic and synchronized feature state reduce skew?

Data cleaning is the first stage where training and serving can diverge. Raw data frequently contains missing values, duplicates, or outliers that degrade model performance. Cleaning operations must be deterministic and reproducible: given the same input, they must produce the same output regardless of environment. This requirement shapes which cleaning techniques are safe to use in production.

Data cleaning encompasses **deduplication**, which merges repeated records so they neither overweight what the model learns nor leak across splits (Lee et al. 2022), along with missing-value handling and normalization. In the KWS corpus, deduplication must catch more than identical files. One speaker's repeated takes of a wake word are near-duplicates, and they must stay within a single split, or evaluation measures memorization of that voice rather than generalization. The matching rule and its threshold must be captured in code that runs the same way every time the dataset is rebuilt.

Outlier handling exposes a sharp operational split between batch training and online serving. During training, the pipeline can drop corrupted records or extreme sensor glitches so that a few bad values do not dominate parameter updates. At serving time, the response contract determines whether an invalid request may be rejected, routed to review, or answered with an abstention or request for another sample. Clamping features to bounded intervals (such as $[\mu - 3\sigma, \mu + 3\sigma]$) is safe only when that transformation has been validated for the task and uses the fitted training parameters. Outlier filters that judge each record relative to others introduce a subtler consistency risk: if evaluated across concurrently batched serving queries, the classification of an individual query depends on what other queries happen to arrive in the same request window, violating request isolation.

The validation monitors in section 4.5.3 also check transformed outputs. A rise in nulls after a join, unexpected duplicate records, or a shifted feature distribution can expose a transformation defect even when the raw inputs passed their checks.[15] Comparing raw and transformed records across the same pipeline version helps localize the fault before changing the model.

Quality validation must enforce identical schemas and validation predicates across training and serving pipelines. While training environments can afford compute-intensive profiling passes over entire datasets, serving paths must execute lightweight assertion checks (such as type validation, range bounding, and required-field existence) within the request's latency budget. Mismatches between training-time validation and serving-time assertions allow corrupted inputs to reach the model undetected.

Transformation techniques convert data from its raw form into a model-ready representation, but the systems risk lies in the parameters those transformations learn. Common transformation tasks include normalization and standardization, which scale numerical features to a common range or distribution. For example, square footage and room count may differ greatly in scale; normalization places them on comparable numerical ranges, though their influence still depends on the model and learned parameters (Bishop 2006). Maintaining training-serving consistency requires that normalization parameters computed on training data be stored and applied identically during serving. Operationally, these parameters must be persisted alongside the model itself and loaded during serving initialization.

Preprocessing parameters fall under the same split-boundary rule as the data itself (section 4.1). If the mean and variance, or a category vocabulary, are fitted on the entire dataset before it is split, statistics of the test examples flow into the transformation the model trains through, and the test score

[15] **Statistical drift detection**: The means, variances, and quantiles tracked in quality monitoring are early-warning signals for the divergence term $\mathcal{D}(P_t \| P_0)$ of the degradation equation (equation 1.1). A mean session length shifting from eight to 12 minutes over six months may indicate drift; a sudden drop to three minutes may indicate a collection fault. Outcome checks and source investigation are needed before choosing retraining or a source-system fix.

↪ The cost of training-serving skew and the shared feature libraries and schema validation tests that enforce training-serving parity are developed in section 14.1.1.3.

no longer measures generalization to unseen data. The pipeline therefore fits every transformation parameter on the training split alone and applies the frozen result to validation, test, and serving inputs.

Beyond numerical scaling, transformations convert discrete inputs through categorical encoding, extract cyclical representations from timestamps, and construct derived spatial features. Categorical encodings must handle both the categories present during training and unseen categories encountered during serving. A reliable pipeline builds the vocabulary index exclusively from the training split, serializes this dictionary with the model artifact, and maps unknown runtime tokens to an explicit out-of-vocabulary (`<UNK>`) index. Omitting this mapping causes runtime dimension mismatches or silent index collisions during online inference.

Domain-specific feature engineering turns raw inputs into representations that expose informative signals directly to the learning algorithm (Kuhn and Johnson 2013). Instead of requiring the model to discover non-linear combinations from scratch, engineered features encode domain structure directly. The audio features the keyword spotter uses, described later, are themselves engineered: they encode what is known about human hearing, so a small model does not have to learn it from data.

Feature engineering can be a high-leverage activity because it changes what signal the model can see. In some workloads, well-designed features improve performance more than another round of algorithm selection or hyperparameter tuning. That leverage must be balanced against the consistency requirements of production systems. Every engineered feature shared by training and serving must have equivalent behavior in both environments. Production systems therefore implement feature engineering logic in shared libraries or coordinated pipelines rather than reimplementing it independently. Many organizations centralize those shared definitions in a feature store, the storage layer that section 4.7.5 defines, to reduce inconsistency across environments.

In the KWS pipeline, audio recordings collected across crowdsourced contributors, synthetic speech generators, and field captures arrive with wide acoustic variation. The raw audio contains the imperfections anticipated during problem formulation: environmental noise spanning quiet bedrooms to noisy kitchens, amplitude clipping from saturated analog-to-digital converters, gain differences across microphone hardware, and inconsistent sampling rates. Standardizing these inputs while preserving the short speech transients that distinguish wake words from ambient noise is essential to reaching the 98 percent detection target. Gain normalization is a specific trap. Normalizing each stored clip against its own peak uses information a streaming device never has, because the device hears continuous audio rather than finished clips, so the training pipeline may apply only gain handling the device can reproduce as audio arrives.

Quality assessment for KWS extends the general principles with audio-specific metrics. Beyond checking for null values or schema conformance, the validation pipeline tracks background noise levels (signal-to-noise ratio above 20 dB), audio clarity scores (frequency spectrum analysis), and speaking rate consistency (wake word duration within 500 ms to 800 ms). The pipeline flags recordings whose noise, duration, clipping, or distortion falls outside the scenario's acceptance rules for review. These thresholds screen the clean baseline slice. Valid noisy or unusually timed speech remains in a separately labeled robustness slice, consistent with the noisy data budgeted in the KWS design. Review distinguishes these coverage examples from corrupted recordings before rejecting a sample. These screens do not prove that a sample is useful, but they reduce the chance that known defects reach model development. Checking audio quality at the source stops defects from spreading along the arcs traced in figure 4.1.

Transforming audio data for KWS involves converting raw waveforms into formats suitable for ML models while maintaining training-serving consistency. Raw audio waveforms (sequences of amplitude values sampled thousands of times per second) are high-dimensional. KWS pipelines often transform them into compact representations that emphasize the frequencies and temporal patterns most relevant to speech. Figure 4.15 aligns three views of an audio sample across a two-second window: the raw amplitude waveform, the corresponding time-frequency spectrogram isolating high-power acoustic energy around 500 Hz, and a compact 13-coefficient MFCC approximation. These standardized feature representations, typically MFCCs or spectrograms,[16] emphasize speech-relevant characteristics while reducing noise and variability across different recording conditions. While a raw short-time Fourier transform (STFT) produces hundreds of linear frequency bins per

[16] **Spectrogram**: The short-time Fourier transform (STFT) computes this 2D representation by converting the one-dimensional waveform into a time-frequency image. This representation lets image-style ML models process audio, but it also creates a rigid dependency: a mismatch in STFT parameters (for example, a 25 ms vs. 30 ms window) changes the feature representation between training and serving and can degrade performance.

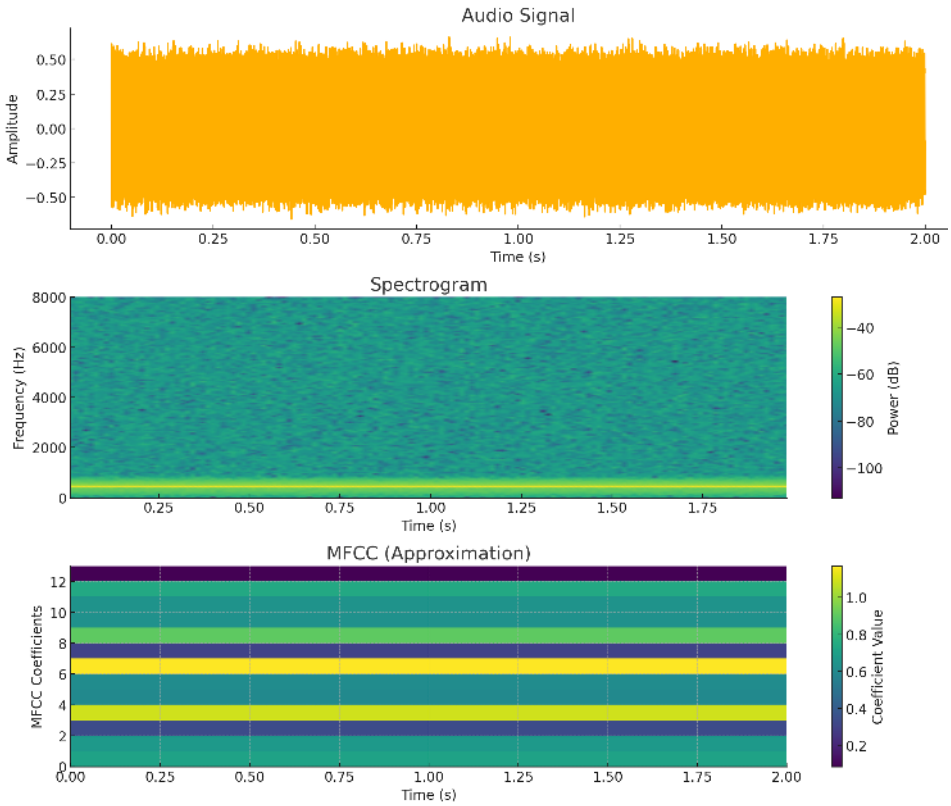


Figure 4.15: Audio Feature Representations: Three illustrative views show an audio waveform, its time-frequency spectrogram, and a compact MFCC approximation. The spectrogram encodes time, frequency, and signal power; the bottom panel arranges a smaller set of coefficients over time.

time window, Mel-frequency analysis aggregates these bins through non-linear triangular filterbanks aligned with human auditory perception, followed by a discrete cosine transform that compresses the signal into just 13–40 decorrelated coefficients. For edge devices, this step collapses data volume by over an order of magnitude, allowing audio feature frames to fit inside the tight tens-of-kilobytes SRAM budgets of embedded coprocessors.

4.7.2 Feature computation placement

Precomputing those MFCC frames once during ingestion lets training iterate at disk speed, but it freezes any random noise or pitch augmentation applied to the raw waveform; computing them on the fly in the data loader keeps every epoch fresh at the cost of repeating the work and risking accelerator stalls. **Feature computation placement** decides which resource pays for a feature: storage pays when features are materialized on disk, while compute and latency pay when features are generated on demand.

When features are precomputed during ETL and stored on disk, training epochs read ready-made tensors without paying the transformation cost on every pass, and every run consumes the exact same values. That speed and reproducibility come at the price of storage footprint and rigidity: derived features must be stored alongside the raw data, and any change to feature logic requires recomputing the entire corpus before the next experiment can run. Computing features on the fly inside the data loader reverses the trade-off. Because only raw data is stored and transformations execute as batches are read, logic changes take effect immediately and random augmentations vary from epoch to epoch, but the host CPU repeats the work on every pass and can stall the accelerator if preparation falls behind.

In practice, production systems combine both approaches. Expensive, slowly changing representations, such as high-dimensional user embeddings (learned vectors that summarize each user) derived from matrix factorization or multi-month historical aggregations, are precomputed offline and materialized in storage. Volatile, lightweight signals, such as session recency, interaction timestamps, or client-side telemetry, are computed dynamically in the data loader. The keyword spotter shows why placement is not a free choice. The clean features of each recording can be computed once and stored, but noise augmentation must be applied to the raw audio before feature extraction, so the features of augmented clips have to be computed in the loader on every pass. This split matches each feature's placement to its rate of change, trading storage capacity against execution latency.

4.7.3 Idempotent transformations

Consistency and placement settle what a transformation produces and where it runs, which serves the quality pillar; the reliability pillar asks whether that transformation can safely run again after a crash. If a processing stage appends transformed records to a target table and a worker fails after writing its output but before acknowledging completion, the automatic retry runs the job a second time and writes duplicate records into the training set. **Idempotency**[17] prevents that corruption by ensuring that applying an operation repeatedly has the same effect as applying it once. This property matters whenever jobs retry after node failures, engineers reprocess historical data to fix a bug, or the same records flow through multiple paths. Determinism is a related but distinct requirement: identical inputs and captured state should produce identical outputs.

[17] **Idempotency**: From Latin *idem* ("the same") + *potens* ("having power"), literally "having the same power when applied again." In ML pipelines, idempotency enables safe retries after partial failures: a non-idempotent operation (for example, appending to a log) creates duplicates on retry. An idempotent operation leaves the same resulting state after one application or repeated applications.

In state-machine terms, an operation f is idempotent if $f(f(s)) = f(s)$. In storage systems, setting a register to an absolute value ($s \leftarrow 1$) is idempotent, whereas incrementing a counter ($s \leftarrow s + 1$) or appending a row to an unkeyed log is not: each re-execution mutates system state. Distributed data pipelines execute across preemptible workers and unreliable networks where timeouts and node crashes trigger automatic retries. Rather than appending blindly, an idempotent pipeline enforces write semantics such as deterministic keyed upserts (insert if absent, overwrite if present) or atomic directory swaps, ensuring that repeated job attempts converge to the identical final dataset.

Managing partial pipeline failures requires clear state boundaries between stages. Decoupling stages with persistent intermediate checkpoints ensures that each phase can be retried independently without re-executing upstream work. For long-running jobs processing terabyte-scale corpora, checkpointing state partitions (such as completed file shards or stream consumer offsets) bounds retry latency: a task failure after five hours of execution requires replaying only the uncommitted shard rather than restarting the entire five-hour job. The checkpoint mechanism must commit output partitions and state markers in a single atomic transaction, preventing records from being dropped or processed twice.

Deterministic transformations are those that always produce the same output for the same input, without dependence on external factors like time, random numbers, or mutable global state. Transformations that depend on current time (for example, computing "days since event" based on current date) break determinism because reprocessing historical data would produce different results. The solution is to capture temporal reference points explicitly: instead of "days since event," compute "days from event to reference date" where reference date is fixed and persisted. Random operations should use seeded random number generators where the seed is derived deterministically from input data, ensuring reproducibility.

Reliability in the KWS pipeline requires reproducible feature extraction. Audio preprocessing must be deterministic: given the same raw audio file, the same MFCC features are always computed regardless of when processing occurs or which server executes it. This enables debugging model behavior (can always recreate exact features for a problematic example), reprocessing data when bugs are fixed (produces consistent results), and distributed processing (different workers produce identical features from the same input). The processing code captures all parameters (fast Fourier transform (FFT) window size, hop length, number of MFCC coefficients) in configuration versioned alongside the code, ensuring reproducibility across time and execution environments.

4.7.4 Distributed processing

Scale becomes the next constraint once quality and reliability are in place. Correct, repeatable transformations still fail the pipeline if they cannot keep pace with data volume or its deadline. As datasets and experiment concurrency grow, data processing can outgrow one machine even when the raw bytes still fit on local storage. The cleaning techniques that work on gigabytes in memory must then become partition-aware, out-of-core, or distributed without changing their semantics.

Single-machine processing suffices for large workloads when engineered carefully. Servers with 256 GB of RAM can process datasets of several terabytes using out-of-core processing that streams data from disk. Libraries that stream data from disk and parallelize across cores extend familiar dataframe interfaces to such workloads. Before investing in distributed processing infrastructure, teams should exhaust single-machine optimization: using efficient data formats (Parquet[18] instead of CSV), minimizing memory allocations, using vectorized operations, and exploiting multi-core parallelism. The operational simplicity of single-machine processing (no network coordination, no partial failures, simple debugging) makes it preferable when performance is adequate.

Within a single machine, parallelizing data ingestion across host CPU workers yields diminishing throughput returns whenever unpartitionable stages, such as sequential decompression or central database lock acquisition, limit concurrency. Amdahl's law caps how much speeding up one part of a pipeline can improve the whole. Applied to host worker count, it means that even when 90 percent of a feature extraction pipeline parallelizes across workers, the speedup cannot exceed $10\times$. The serial stages keep their full duration however many workers share the rest, so eliminating serial I/O and single-threaded stages takes precedence over launching more worker processes.

[18] **Parquet**: A columnar storage format that organizes data by column, not by row. Where a CSV reader must parse every row in full to reach a few columns, a Parquet reader loads only the columns a computation needs. The resulting I/O reduction depends on the selected columns, encoding, compression, and workload.

4.7.4.1 Coordination across machines

This book keeps data processing on one machine, but the boundary is worth knowing. When a corpus or the number of concurrent feature rebuilds outgrows one server, processing spreads across a cluster, and three things change. Data locality becomes a design constraint, because a network link moves bytes far more slowly than a node reads the shard it already holds, so cluster frameworks send the computation to the data instead of the data to the computation. Network transfer becomes the dominant cost of any operation that exchanges records rather than summaries. A global shuffle or a join sends records across every partition, whereas a decomposable reduction such as a mean sends only one partial result per partition. Coordination becomes part of the work, because the machines wait on the slowest worker and must recover when one fails partway through a job, which is when the idempotent, deterministic transformations described earlier become essential rather than merely convenient. Within one machine the network costs disappear and partial failures are rare, so the transformations of this section appear in their simplest form.

Scalability in the KWS pipeline matters at both ends of the lifecycle. Development uses single-machine processing on sample datasets to iterate rapidly. The reference corpus fits on the local storage of one well-provisioned server, so distributed processing pays off mainly when many experiments rebuild features at the same time. Per-file feature extraction is embarrassingly parallel, but end-to-end speedup is bounded by shared storage bandwidth, metadata operations, scheduling, and output contention. Production deployment adds a stricter 16 KB preprocessing-state budget alongside the 256 KB model-size limit, requiring careful footprint optimization so feature extraction fits on the device.

4.7.5 Feature stores

The keyword spotter needs no feature store, because its features are computed on the device from audio alone. A fraud or recommendation model whose features come from user history faces two failure modes at once: during model development, data scientists compute features like `user_purchase_count_30d` with batch SQL queries in a warehouse, whereas during serving an online microservice computes the same metric incrementally from an in-memory cache, and minor differences in timezone arithmetic, missing-value handling, or floating-point rounding cause offline and online features to drift apart. Worse, if the training query joins a historical transaction against the user's current profile rather than the profile as it stood when the transaction happened, future information leaks into the training set. Keeping the two paths consistent means serving historical feature values for training and current feature values for inference from a single shared definition.

↪ Amdahl's law, the ceiling that untouched work places on any speedup, is developed in section 11.1.

Point-in-time correctness and shared feature logic are the primary operational requirements that drive teams to adopt **feature stores**, which Uber's Michelangelo platform popularized to eliminate this class of training-serving skew (Hermann and Del Balso 2017).

Definition 4.5: Feature store

Feature store is the architectural layer that centralizes the management of machine learning features, decoupling feature computation from consumption.

1. **Significance**: It supports point-in-time-correct historical retrieval for training $(x_{t-\Delta})$ and coordinated feature definitions for real-time inference (x_t), reducing training-serving skew when timestamps, backfills, freshness, and synchronization are handled correctly.
2. **Distinction**: Unlike a general-purpose database, a feature store is designed for dual storage modes: an *offline store* (columnar/batch) for training and an *online store* (key-value/low-latency) for serving.
3. **Common pitfall**: A frequent misconception is that a feature store is only "a place to store data." In reality, it manages feature definitions, values, and retrieval; transformation may run in the feature store, an offline store, or a separate batch or stream-compute engine.

Feature stores establish a single source of truth for feature definitions across the ML lifecycle. By registering a feature definition as a unified computation graph, the feature store evaluates the identical transformation across both historical batch data and live inference streams. This shared definition reduces one source of training-serving skew, but equivalent execution still requires synchronized fitted state, correct timestamps, and appropriate freshness. Those conditions must be tested across both retrieval paths. Centralization also enables organizational feature reuse: expensive representations, such as multi-dimensional user embeddings derived from months of interaction logs, are computed once and served to multiple downstream modeling teams.

↪ How feature stores are operated in production, including point-in-time retrieval and training-serving skew prevention at scale, is developed in section 14.3.1.1.

To serve both access patterns, feature stores pair two storage engines. The offline store uses columnar formats (such as Parquet) on high-capacity object storage, tuned for multi-gigabyte sequential scans during batch training. The online store uses low-latency key-value databases (such as Redis), tuned for sub-10-millisecond point lookups during real-time inference. Automated pipelines synchronize the two engines by pushing batch-computed feature snapshots to the online store while streaming ingestion updates both stores continuously as new user events arrive.

Time-travel queries separate a true feature store from a simple cache. Training requires evaluating features as they existed at the exact moment of an event, rather than their current values. In a customer churn model, for example, training on a customer who churned on January 15 must use features computed on January 14, not current post-churn profile state. Feature stores enforce point-in-time correctness using an **as-of join** (temporal join): for every observation at timestamp t_{obs}, the retrieval engine joins the most recent feature record whose update timestamp satisfies $t_{\text{feat}} \leq t_{\text{obs}}$. The join must also exclude information that was not available to the prediction path at t_{obs}, including late-arriving events and later backfills whose event timestamps precede the observation. Timestamp ordering prevents future-information leakage only when retrieval preserves this historical availability across the upstream feature tables.

Every processing choice in this section, from cleaning and scaling to placement, idempotency, and feature retrieval, leaves derived state that downstream training depends on. Whether features are computed in a loader, materialized in storage, or served from a feature store, the team must eventually be able to say exactly which recordings, labels, and transformation versions produced the dataset a model learned from. Recording that answer is the last step of building the dataset.

4.8 Dataset Release

Months after launch, the keyword spotter's detection rate drops on a new hardware revision, and the first question the team must answer is what the shipped model learned from, down to the recordings, the labels and the guideline they followed, the splits, and the feature code. A **dataset release** answers that question in advance. It is an immutable, versioned record of everything that produced a training set: the source records, how they were split and labeled, and the code and

parameters that turned them into features. It is the versioned dataset that Stage 2 of the lifecycle hands to model development, and it turns the pipeline built so far into something that can be reproduced, compared, and audited.

The release is built from what each record carries. Table 4.5 lists the fields of one KWS training clip and the question each field answers later. Two of them do most of the work. The split is assigned by hashing the speaker key, so every clip from a speaker lands in the same split in every release and a later relabeling pass cannot leak a test speaker into training. The label carries the version of the guideline it was made under, so a change in labeling rules can be told apart from a change in the data.

Table 4.5: Anatomy of a KWS Training Record: Each row names a field that one keyword-spotting training clip carries into a dataset release and the later question it answers. Speaker-keyed splits prevent leakage across releases, and rubric versions separate label changes from data changes.

Field	Recorded value	Question it answers later
Recording ID	Content hash of the audio file	Is this clip a duplicate, and what was derived from it?
Speaker key	Pseudonymous speaker identifier	Are all of this speaker's clips in one split?
Consent record	Reference to the terms the clip was collected under	May it be retained, and must it be deleted on request?
Split	Train, validation, or test, from a hash of the speaker key	Is the split stable across releases?
Label and rubric version	Wake word or none, with the labeling guideline version	Did the data change, or only the labeling rules?
Alignment version	Aligner version and word-boundary confidence	Where could a segmentation error have come from?
Feature configuration	Window, hop, coefficient count, and processing code commit	Can the features be recomputed exactly?

Each record's fields capture local provenance, while the release manifest ties the whole collection together. The rest of a release records how those fields were produced and pins the resulting dataset in storage so that any historical build can be recovered verbatim.

4.8.1 Transformation lineage

The last row of table 4.5, the feature configuration, exists because recording the raw audio file and label is not enough if a later change to the MFCC window function leaves the team guessing which window the shipped model trained on. Governance, the fourth pillar, closes that gap by tracking what transformations were applied, when they ran, which version of processing code executed, and what parameters were used. This **transformation lineage** is the processing part of **data lineage**, the recorded trail that links each dataset, feature, and model to the sources, code, and parameters that produced it.[19] Transformation lineage supports reproducibility, debugging, auditability, and iterative improvement when transformation bugs surface. It can supply evidence for documentation or compliance workflows, but lineage alone does not explain a model's decision or establish regulatory compliance.

That trail starts with transformation versioning, which records which version of the processing code produced each dataset. Whenever engineers change the transformation logic to fix a bug, add features, or improve quality, the version number increments, and tagging datasets with that version identifies every table or shard that must be reprocessed after a fix. Because identical source code can still produce slightly different floating-point outputs across different NumPy releases, runtime environment variables, or CPU instruction sets, each dataset also records the Git commit hash of the code that created it alongside a container image that freezes the code, dependencies, and system libraries in one immutable environment.

Code and environment versions still leave out the data-dependent state that transformations learn, so parameter tracking records the fitted values applied during transformation: the mean and standard deviation computed on training data for normalization, the vocabulary (set of all observed categories) for categorical encoding, and any constants or thresholds used in feature engineering

[19] **Data lineage**: When a model produces an erroneous or discriminatory prediction, lineage helps engineers determine whether the contributing problem originated in raw data, feature computation, training, or serving. The resulting trace can support incident investigation and jurisdiction-specific documentation duties, but its usefulness depends on complete instrumentation and retained history; it does not guarantee an explanation of model behavior.

(section 4.7.1). Serializing these parameters alongside the model artifact ensures that serving applies the exact same values as training. Processing lineage then joins code, environment, and fitted parameters into a complete execution history from raw data to final features: which raw files were read, what transformations ran in what order, what parameters were applied, and when the job executed. Lineage systems instrument the pipeline so that each job records these links as it runs, because a history reconstructed after an incident is rarely complete. When model predictions go wrong, engineers can trace back through the lineage graph to find the training data that contributed to the behavior, the quality scores attached to that data, and the transformations applied, then recreate the exact scenario for investigation.

In the KWS pipeline, governance applies to the audio processing parameters that affect model behavior. The reference volume level used for normalization is persisted, and the FFT and MFCC settings versioned with the processing code (section 4.7.3) are joined by the window function (such as Hamming or Hann), the frequency range, and the mel filterbank parameters. Recording them makes it possible to reproduce training data exactly when debugging model failures, to validate that serving uses preprocessing identical to training, and to study how preprocessing choices affect accuracy. Without this record, teams fall back on manual documentation that becomes outdated, which allows subtle training-serving skew.

4.8.2 Data versioning for ML reproducibility

Lineage records how a dataset was built, but reproducing or comparing two runs also requires recovering the exact bytes of each release. Without an immutable snapshot of the training inputs, diagnosing why a retrained keyword spotter lost accuracy takes days of guesswork across new recordings, changed labels, and upstream batches. With data versioning, engineers compare the two training snapshots directly and find the cause at once: the labeling guideline version in table 4.5 changed between the two releases, so the same clips now carry different labels.

File-level versioning engines like DVC (Data Version Control) provide this link by pairing Git with content-addressed remote storage (Iterative 2024). Git tracks small text pointer files (`.dvc`) containing cryptographic hashes, while DVC synchronizes the underlying binary dataset to object storage (listing 4.2). Checking out an earlier Git commit restores the historical pointers; the paired DVC retrieval step shown in the listing restores the dataset contents matched to that code state.

Listing 4.2: DVC Workflow: DVC brings Git-like semantics to data versioning. DVC tracks large data files alongside code commits, enabling exact reproduction of any historical training dataset through paired `git checkout` and `dvc checkout` commands.

```
# Add data to version control
dvc add data/training.csv
git add data/training.csv.dvc
git commit -m "Add training data v1"
dvc push  # Upload to remote storage

# Later: retrieve exact data for any historical commit
git checkout abc123
dvc checkout  # Restores exact data from that commit
```

While file-level versioning suits bulk artifacts like images and audio, tabular datasets require versioning over tables that change row by row. Open table formats such as Delta Lake solve this by keeping an append-only transaction log of all row insertions, deletions, and schema changes (Armbrust et al. 2020). This log supports time-travel queries that pin a dataset state by timestamp or version ID (listing 4.3).

Listing 4.3: Delta Lake Time Travel: SQL queries identify a historical table state by timestamp or transaction-log version.

```
-- Query data as it existed on a specific date
SELECT * FROM training_data TIMESTAMP AS OF '2024-01-15'

-- Or by version number for programmatic access
SELECT * FROM training_data VERSION AS OF 47
```

Two production components work with these versioning engines to complete end-to-end reproducibility. First, feature store point-in-time retrieval evaluates features as they existed at the observation timestamp, preventing the leakage from future observations described in section 4.1. Second, a model registry records the lineage manifest for each deployed model: Git commit SHA, data version ID, training configuration, and evaluation metrics. In the keyword spotter's regression, this audit trail turns an open-ended debugging investigation into an immediate snapshot comparison.

Long-term retention balances auditability against storage cost and privacy rules. High-volume production pipelines use tiered retention: hot SSD tiers for active training and debugging, warm object storage for periodic evaluation snapshots, and cold archive storage for compliance records. Because keeping petabyte-scale data forever is too expensive and often violates privacy statutes, data lifecycle policies must set explicit retention windows and deletion schedules. A release fixes what the dataset is, yet it says nothing about how fast that dataset can reach the accelerator during training, which is the second bound from the opening of this chapter and the one the remaining stages of the pipeline must meet.

↪ How the model registry promotes, holds, or rolls back the exact artifact that was evaluated is developed in section 14.3.1.2.

4.9 Physics of Data

The throughput bound lives in the Data–Machine overlap, and it is physical. Moving a petabyte of training data across cloud regions incurs tens of thousands of dollars in egress fees and hours of network serialization delays. Source code is small enough to move freely, so the data-as-code principle explains what data does without explaining why moving it costs so much. The physics of data supplies that explanation by giving data measurable properties. **Data gravity** (McCrory 2010) is the bandwidth, I/O, and latency cost of moving a large byte volume, and **task-relevant signal** is the learning value those bytes carry for the task. A third property, flow rate, decides whether those bytes reach the accelerator as fast as it consumes them, and the ResNet-50 scenario measures it.

Engineering effort pays off most when early filtering and selection separate high-signal records from the rest before data gravity makes them expensive to move. Low-signal data (repetitive audio frames or duplicate samples) consumes host-to-accelerator bandwidth and memory capacity while teaching the model little. An efficient pipeline resolves this physical bottleneck by maximizing the task-relevant signal delivered per byte moved across the storage and interconnect hierarchy, an optimization that begins with the physical properties of the data itself.



Returns peak where task signal is high and movement cost (gravity) is low.

4.9.1 Data gravity

Data movement costs time and money, the cost that the data-gravity principle (2) names. Data gravity scales directly with data volume (D_{vol}) and inversely with network bandwidth (BW). Transferring a petabyte dataset across a 10 Gbps link requires days of continuous transmission ($T = D_{\text{vol}}/\text{BW} \approx 9.3$ days); even across a dedicated 100 Gbps link, transfer latency and egress tariffs remain substantial enough to govern system design. Because migrating 1 PB across network boundaries is both slow and expensive, system designers colocate compute with the storage substrate rather than moving large datasets across remote networks. This physical constraint underlies lakehouse storage architectures[20] (Zaharia et al. 2021), where query and preprocessing engines execute directly against shared object storage rather than repeatedly copying raw data across network fabrics.

[20] **Data lakehouse**: Combines data lake storage (cheap, schema-less) with warehouse query semantics (atomic, consistent, isolated, and durable (ACID) transactions, schema enforcement) using transactional table layers such as Delta Lake. For ML workloads, the lakehouse reduces the ETL copy between lake and warehouse, enabling direct feature computation on the storage layer where data already resides. This is a direct response to data gravity, since repeated petabyte-scale copies increase the D_{vol}/BW cost (Armbrust et al. 2020; Zaharia et al. 2021).

↩ The data-gravity principle, which weighs shipping a dataset against shipping the computation to it, is established in section 2.5.1.

Napkin Math 4.7: The physics of data gravity

Problem: A 1 PB training dataset resides in a US East data center, while a remote accelerator pod is available in US West. What transfer time and egress cost does remote training add, and when would suitable compute near the data avoid that overhead?

Math:

1. **Network bandwidth**: 100 Gb/s $\div 8 = 12.5$ GB/s.
2. **Transfer time**: 1,000,000 GB $\div$ 12.5 GB/s = 80,000 seconds, approximately 22 h.
3. **Cost**: 1,000,000 GB $\times$ \$0.02/GB = \$20,000. (Baseline: AWS data transfer out pricing, 2024.)

Systems insight: If training takes less than 22 h, data transfer takes longer than training. If training costs less than \$20,000 (approximately 5,000 TPUv4-hours), bandwidth costs more than compute. When suitable compute is available near the data, these transfer costs favor moving code to data. Smaller datasets make moving data to code more practical, but placement still depends on compute availability and comparative training time.

4.9.2 Task-relevant signal

A dataset containing one million identical images carries heavy data gravity yet adds no new observations beyond the first sample; repeated passes over that single image can nudge model weights slightly, but they supply no fresh evidence. Conversely, a curated collection of 10,000 diverse edge cases is small enough to move cheaply yet contributes rich supervisory signal. Physical storage measures raw bytes, whereas machine learning extracts **task-relevant signal**: evidence that helps the model learn the target task rather than raw Shannon entropy. Comparing task-relevant signal with data gravity motivates a selection heuristic:

$$\text{Data Selection Gain} \propto \frac{\text{Task-Relevant Signal}}{\text{Data Gravity}} \tag{4.3}$$

The numerator is not a universal physical quantity. A team estimates marginal learning value by comparing held-out model quality under a fixed training procedure; the denominator accounts for the movement cost of the compared datasets. The heuristic favors a selection only when its measured learning benefit justifies that cost. Scoring, labeling, and training costs must also enter a complete budget. Deduplication and the active learning of section 4.6.5 can improve this ratio by eliminating redundant byte volume and proposing uncertain examples for labeling. Uncertainty does not establish learning value; ambiguous or corrupted examples can also be uncertain, so the selection policy requires validation.

↪ The information-compute ratio, which measures the learning signal each sample returns per unit of training compute, is developed in section 9.2.1.

4.9.3 The feeding problem

Data gravity dictates the cost of moving data in bulk; **the feeding problem** dictates the cost of delivering it to execution units in real time. It is an input/output **flow rate** bottleneck: storage bandwidth fails to keep pace with compute throughput. The bottleneck is measured, not hypothetical. Profiling nine widely used vision and audio models found their training time often dominated by data stalls, time spent waiting for data to be fetched and preprocessed (Mohan et al. 2021), and in production recommendation training the data storage and ingestion pipeline has become a dominant constraint on training performance and capacity (Zhao et al. 2022).

When data delivery and computation overlap, the bottleneck principle holds that a system runs only as fast as its slowest stage. Consider the chapter's throughput workload, one A100-class accelerator training ResNet-50 on images. The accelerator's peak arithmetic rate is 312 TFLOP/s, and one training step on one image costs about 24.6 GFLOP, roughly three times the cost of a single prediction because training must also work out how to adjust every parameter. Dividing the first by the second gives an arithmetic ceiling of about 12,682 img/s. In practice a well-tuned pipeline sustains about 3,200 img/s, roughly 25 percent of that ceiling, once layers that wait on memory and the synchronization between steps take their share. Each image sits on disk as a 150 KB JPEG, so storage must deliver 480 MB/s. When the attached volume supplies only 250 MB/s, compute

units stall on I/O. This gap defines **the feeding loss**, the throughput penalty from insufficient input supply, which directly degrades system efficiency (η_{hw}). Under steady-state overlap, divide the supplied 250 MB/s by the required 480 MB/s to obtain the fraction of the sustained training rate that storage can support. Subtracting that fraction from one gives a modeled feeding loss of about 48 percent. This is the throughput loss relative to the supplied sustained baseline, not a measurement of all accelerator idle time or utilization relative to advertised peak.

Which bytes are counted matters as much as how many. After decoding and conversion to an FP32 tensor, the same image occupies 602 KB, about 4 times its compressed size, so a loader that converts on the host moves 1.9 GB/s through host memory and across the host-to-accelerator link. Many loaders ship 8-bit pixels instead and convert on the accelerator to avoid that cost. Compression thus trades storage bandwidth for CPU decode work. A pipeline that caches pre-decoded tensors on disk to skip that work pays the full 1.9 GB/s at the storage tier instead, and it freezes the random crops that augmentation would otherwise vary from epoch to epoch. Applying the same supply-to-demand calculation gives a modeled throughput loss of 87 percent relative to the sustained baseline. Storage and delivery apply this slowest-stage bound to the storage tiers (section 4.10.3) and then to the CPU decode workers (section 4.10.4) without re-deriving it. Distance also increases energy consumption: as data travels farther from arithmetic logic units, transport energy dwarfs the cost of computation itself.

↩ The bottleneck principle is established in section 2.3.1.

Systems Perspective 4.2: The energy cost of data movement

The memory-wall analysis in section 2.2 implies a data-engineering principle: *moving one 32-bit value costs roughly 173× to 10,811× as much energy as the table's FP32 multiply*, with the multiplier rising as the data leaves the chip. The DRAM ratio is smaller than the one in section 1.7 only because that comparison set one byte against a cheaper FP16 multiply. Later optimization chapters exploit the same cost gradient inside model and hardware design; here, the question is how much energy the information flow from the external world already consumes before the model computes, at each level of the memory hierarchy (table 4.6).

Table 4.6: Energy Cost of Moving vs. Computing on a 32-Bit Value: Each row gives the approximate energy per event at one stage of the data-movement hierarchy, with multipliers normalized to one 32-bit FP32 multiply. All rows use the same 32-bit width, so the ratios compare like-sized events. They do not by themselves prove which term dominates a complete workload; total energy also depends on the number of arithmetic operations and transfers. Arithmetic and DRAM rows follow Horowitz (2014).

Operation	Energy (pJ)	Relative Cost
32-bit FP Multiply	3.7 pJ/FLOP	1×
DRAM Memory Access (32-bit)	640 pJ	173×
Local SSD Access (32-bit)	4,000 pJ	1,081×
Network Transfer (32-bit)	40,000 pJ	10,811×

That gradient is why data locality dominates systems optimization. Each avoided transfer eliminates its corresponding movement energy, compounding across training epochs.

Because data movement incurs physical energy costs, data volume acts like physical mass. Removing 50 percent of training data through deduplication saves more than persistent storage capacity. It also eliminates energy expenditure across network, memory, and preprocessing stages. Filtering uninformative samples early is among the most effective architectural levers when I/O and communication dominate workload runtime.

Checkpoint 4.3: The physics of data

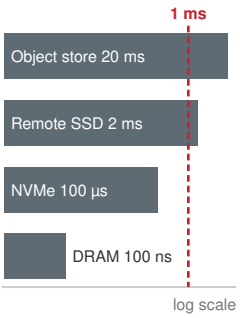
Data engineering is governed by physical costs. Check your intuition:

- ☐ **Data gravity**: Why does a petabyte-scale dataset pull compute toward the data, and which assumption in the petabyte example (link speed, egress price, training duration) would have to change to reverse that conclusion?
- ☐ **Energy cost of data movement**: Why does fetching a 32-bit value from DRAM, an SSD, or the network cost more energy than one 32-bit multiply, and what else (how many operations, how many transfers) must you know before saying which dominates a workload's total energy?
- ☐ **Task-relevant signal**: Why can a smaller, diverse dataset be more valuable than a massive, redundant one?

The energy hierarchy and the signal-density heuristic converge on the same optimization lever, the Data Selection Gain of equation 4.3, because every redundant or uninformative record removed early saves movement energy and interconnect bandwidth at each later stage without costing task-relevant signal. For the released dataset, these costs come due in delivery. Every training epoch moves it from storage, through the host, to the accelerator, and the feeding problem decides whether that movement keeps pace.

4.10 Storage and Delivery

Delivery is where the throughput bound is met or missed. On baseline cloud block storage, the ResNet-50 scenario of section 4.9.3 already loses about 48 percent of its sustained training rate. Following that scenario to its end means choosing a storage tier that can sustain the accelerator's rate, keeping the host processor between storage and accelerator from becoming the next bottleneck, and cutting the bytes each sample costs to move. The KWS data poses the same questions at smaller scale, for the multilingual reference corpus (23.4 million samples spanning 50 languages) and for the much smaller product-specific subset.



A sub-millisecond point lookup fits only in DRAM or local NVMe.

4.10.1 Storage system options

The ResNet-50 job reads its images in long sequential scans, while a serving system fetches one feature vector per request. These opposing access patterns pull storage in two directions, and selecting a system means minimizing the data term $\left(\frac{D_{\text{vol}}}{\text{BW}}\right)$ of the iron law of ML systems for whichever pattern dominates. Every storage medium imposes physical constraints on bandwidth that determine the maximum speed of the training and serving pipelines.

Two storage performance metrics govern this optimization. **IOPS (input/output operations per second)** counts the distinct read or write requests a device can handle per second, bounding *random access* workloads such as fetching small batches of images or individual user profiles. **Throughput (bandwidth)** measures the volume of data transferred per second, typically IOPS $\times$ Block Size, bounding *sequential access* workloads such as scanning a Parquet file for training.

Three architectures cover these patterns, and the ResNet-50 scan and the feature-serving lookup sit at opposite ends of the range. Conventional databases (online transaction processing (OLTP)) optimize for high IOPS with small, row-oriented blocks, strict ACID guarantees, and sub-millisecond point lookups. That profile fits online feature serving, such as a recommendation system fetching one user profile or an anti-fraud service querying historical transaction counts, but it wastes bandwidth on a training scan, because the storage engine fetches entire rows even when the model consumes only 20 of 100 features. Data warehouses (online analytical processing (OLAP)) organize tables by column rather than by row (Stonebraker et al. 2018), so a sequential scan reads only the required features, transferring one fifth of the uncompressed row payload when selecting 20 out of 100 columns. The format-efficiency calculation in section 4.10.5 quantifies this gain with a worked fraud-detection scenario. Warehouses suit feature engineering, batch aggregation, and tabular training, but they require structured schemas, handle raw audio, imagery, and free-form text poorly, and make schema changes during experimentation costly `ALTER TABLE` operations across billion-row

tables. Data lakes store structured, semi-structured, and unstructured data in native formats on high-capacity object storage and defer structure to query time, an architectural pattern known as schema-on-read.[21] A ResNet-50 job can read raw JPEG files from a lake at sequential rates, while independent jobs impose their own schemas without coordinating migrations. The cost is that governance shifts from ingestion to retrieval. Without a metadata catalog recording lineage, update cadence, schema versions, and access policies, a lake degrades into a "data swamp," a set of orphaned directories such as `userdata_v2_final` and `userdata_v2_ACTUALLY_FINAL` whose lineage and quality are unrecorded. Each storage architecture therefore optimizes for a distinct access pattern and lifecycle phase, as table 4.7 summarizes.

21 **Schema-on-read**: Applies data structure definitions at query time rather than during ingestion, contrasting with schema-on-write (traditional databases) where data must conform to a predefined structure before storage. For ML pipelines in early development, schema-on-read enables rapid experimentation, because teams can store raw sensor data, images, and logs without committing to a feature schema upfront.

Table 4.7: Storage System Characteristics: Each storage architecture favors a different access pattern and ML workflow stage. Databases support the high-IOPS random access needed for real-time feature serving; data warehouses provide sequential throughput for batch training and feature engineering; and data lakes provide schema flexibility and cost efficiency for large-scale raw data retention. A poor match between system and workload can impose substantial performance penalties.

Attribute	Conventional Database	Data Warehouse	Data Lake
Purpose	Operational and transactional	Analytical and reporting	Storage for raw and diverse data for future processing
Data type	Structured	Structured	Structured, semi-structured, and unstructured
Scale	Small to medium volumes	Medium to large volumes	Large volumes of diverse data
Performance Optimization	Optimized for transactional queries (OLTP)	Optimized for analytical queries (OLAP)	Optimized for scalable storage and retrieval

Each lifecycle stage also reads the same data in its own way: exploratory analysis samples it at random, training streams it sequentially at high throughput, and production serving issues low-latency point lookups. During initial development, iteration velocity and schema flexibility dominate raw throughput. The primary systems challenge is managing experiment revisions without exploding storage capacity: ten exploratory runs on a 100 GB dataset naively generate 1 TB of redundant copies. The metadata-pointer approach of section 4.8.2 avoids this by decoupling version tracking from file storage. Governance policies reinforce this tiering by granting broad access to anonymized development datasets while gating sensitive production data behind encrypted, auditable access tiers.

During training, the access profile shifts toward sustained sequential throughput. Modern deep neural networks process massive datasets across dozens or hundreds of epochs, making storage bandwidth the determinant of hardware utilization. Scaling the chapter's ResNet-50 scenario to eight accelerators, each sustaining its per-device rate, gives 25,600 img/s and a sustained storage demand of 3.8 GB/s of 150 KB compressed images; staging decoded FP32 tensors instead would raise it to 15.4 GB/s. Storage tiers unable to sustain this rate idle the accelerators, inflating cluster operating costs. End-to-end ResNet-50 training cannot shrink that demand by precomputing features, because its features change at every step. A model trained on the outputs of a frozen upstream encoder (a pretrained model whose outputs are reused as input features) can, and there the feature placement trade-off (section 4.7.2) becomes critical. Precomputing those representations delivers a 30× storage reduction (150 KB raw inputs reduced to 5 KB embedding vectors) but introduces staleness when the encoder updates.

Production inference reverses the optimization target from sequential bandwidth to low-latency random access. A recommendation service handling 10,000 req/s with a 10 ms SLA and 10 feature reads/request demands 100,000 IOPS. Local Non-Volatile Memory Express (NVMe)[22] storage can sustain this random-access rate within the latency budget; in-memory key-value stores such as Redis, distributed caches, and purpose-built feature stores buy headroom for tail latency. Edge deployment imposes further physical boundaries: constrained flash storage, intermittent network connectivity, and non-disruptive model updates dictate tiered caching, where core models reside in local flash while reference data synchronizes asynchronously with cloud endpoints. In production, serving infrastructure must support atomic rollbacks and parallel A/B testing, operational guarantees that depend on tracing every deployed model back to the exact dataset release from which it was trained. Production ML architectures therefore rarely select a single storage technology; mature systems

22 **NVMe (Non-Volatile Memory Express)**: A storage protocol connecting directly to the PCIe bus with 64K command queues, delivering 5–7 GB/s sequential throughput and microsecond-scale latency. The contrast with SATA SSD (500 MB/s, single queue) is a 10× bandwidth gap that can bottleneck a training pipeline when storage service time exceeds compute time.

combine all three, with databases serving point lookups, warehouses running feature extraction, and lakes holding raw captures and intermediate checkpoints.

4.10.2 Storage tiers and cost

Beneath each of these systems sits a physical storage medium, and media span roughly two orders of magnitude in cost and nearly four in bandwidth. Mismatching the storage tier to the access pattern either inflates infrastructure costs with underutilized capacity or starves training accelerators during I/O stalls.

That spread survived a long collapse in absolute price. Figure 4.16 traces the lowest recorded price per terabyte of each medium in constant 2020 dollars. Hard disk storage fell from about $88B per terabyte in 1956 to $10.6 in 2023, roughly 10 orders of magnitude, yet in 2023 DRAM still cost 42× as much as solid-state storage per terabyte, and solid-state storage cost 2.4× as much as hard disk. Capacity became cheap, but since the 1980s the gaps between tiers have not closed, so the choice of tier, rather than raw capacity, sets the cost of keeping a training job fed.

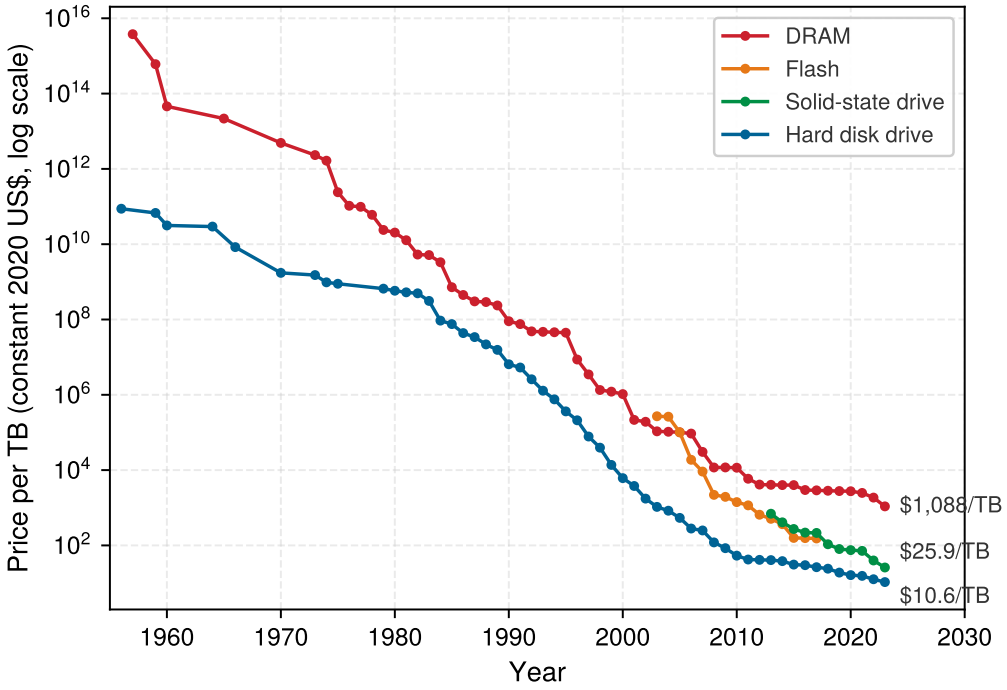


Figure 4.16: Price per Terabyte by Storage Tier, 1956 to 2023: Lowest recorded price per terabyte of DRAM, flash, solid-state drives, and hard disk drives in constant 2020 U.S. dollars on a logarithmic axis. Prices fell by about ten orders of magnitude; since the 1980s, DRAM has stayed roughly two orders of magnitude above disk. Data from Our World in Data (2024), based on McCallum's price records.

ML systems use tiered storage because no single tier is both cheap and fast (table 4.8). Under these 2024 assumptions, storing the KWS reference corpus (748.8 GB) in object storage costs $17.2/month, enabling affordable raw-audio retention, while working datasets on NVMe cost $74.9/month–$224.6/month for active training but load 50× faster.

↪ Why no single medium offers both large capacity and low access latency, the reason storage is built in tiers, is collected for reference in section D.3.1.

Table 4.8: Illustrative Storage Cost-Performance Trade-Offs (2024): These planning ranges vary by provider, region, configuration, and workload. Training data loading requires high-throughput sequential access. Online serving needs low-latency random reads, while archival storage prioritizes cost over access speed for compliance and historical data.

Storage Tier	Cost ($/TB/month)	Sequential Read Throughput	Random Read Latency	Typical ML Use Case
NVMe SSD (local)	$100–300	5–7 GB/s	10–100 µs	Training data loading, active feature serving
Object Storage (S3, GCS)	$20–25	100–500 MB/s (per connection)	10–50 ms	Data lake raw storage, model artifacts
Data Warehouse (BigQuery, Redshift)	$20–40	1–5 GB/s (columnar scan)	100–500 ms (query startup)	Training data queries, feature engineering
In-Memory Cache (Redis, Memcached)	$500–1000	20–50 GB/s	1–10 µs	Online feature serving, real-time inference
Archival Storage (Glacier, Nearline)	$1–4	10–50 MB/s (after retrieval)	Hours (retrieval)	Historical retention, compliance archives

This throughput divergence changes the input-loading component of engineering iteration time. Training that loads data at 5 GB/s completes dataset loading in 149.8 s, compared to 7,488 s at typical object storage speeds. The 50× loading speedup translates into shorter experiments only to the extent that exposed loading time contributes to their duration; transformation and model execution still consume time.

Beyond input datasets, storage must hold what training produces: model parameters, the extra values training keeps for each parameter (its optimizer state), and periodic checkpoints of both. Modern neural networks contain millions to hundreds of billions of parameters, imposing storage and retrieval patterns distinct from tabular data. GPT-3 (Brown et al. 2020) requires approximately 700 GB for model weights when stored in FP32 format (175B parameters times 4 bytes), though practical deployments often use smaller numeric formats such as FP16 (350 GB) to halve memory pressure. Model weights require block-aligned storage formats to support parallel reads across parameter groups during initialization and checkpointing, demanding aggregate storage bandwidth approaching network fabric limits (25 to 100 Gbps) to keep compute units active.

↪ Quantization, which shrinks these weight footprints by storing parameters at lower precision, is developed in section 10.3.

Tracking these artifacts introduces versioning challenges. Git's incremental text diffs fail on multi-gigabyte binaries, where a single weight update produces an entirely new file and ten historical checkpoints of a 10 GB model occupy 100 GB. Production workflows therefore keep lightweight content hashes in Git and the binaries in content-addressed object storage, the same decoupling that section 4.8.2 applies to datasets.

Multi-accelerator training compounds storage pressure through concurrent I/O. When a training job spans multiple accelerators, every rank reads training batches concurrently, and the job periodically writes checkpoints of model and optimizer state, often from a single rank, in bursts that compete with those reads. Storage must absorb reads that scale with the worker count and checkpoint bursts on top of them, preventing I/O serialization that leaves expensive accelerators idle.

↪ The data- and model-parallel strategies that coordinate these workers within one node are developed in section 8.7.1.

4.10.3 The storage bandwidth budget

Each storage tier sets its own ceiling on the feeding problem of section 4.9.3. Host RAM delivers 50 to 200 GB/s on modern server nodes, local NVMe SSDs deliver 5–7 GB/s, and network-attached storage provides 1 to 10 GB/s. Whichever tier serves the training reads caps the data supply rate, so the design question is which tier can sustain the chapter's ResNet-50 scenario, and at which of its two rates.



Napkin Math 4.8: Storage bandwidth budget

Problem: What storage configuration keeps an A100-class accelerator from being data-starved while training ResNet-50? Section 11.6 explains why delivered throughput falls short of an accelerator's advertised peak; here, the point is how a compute ceiling becomes a data-path budget.

Start with the compute ceiling derived in section 4.9.3. The A100's 312 TFLOP/s divided by about 24.6 GFLOP per training image gives 12,682 img/s.

That compute ceiling becomes a storage requirement once each image must arrive from disk or object storage. Storage reads compressed bytes, so the budget uses the 150 KB JPEG rather than the decoded tensor: feeding the ceiling takes approximately 1.9 GB/s, while the 3,200 img/s that the accelerator actually sustains takes 480 MB/s.

Three storage options can now be compared against these two targets:

- S3 Standard delivers about 100 MB/s per connection when it streams large objects such as sharded archives, so feeding the ceiling needs 19 concurrent worker threads before software overhead. Fetching each 150 KB image as its own object is latency-bound and delivers far less.
- SATA SSDs deliver about 500 MB/s sequentially, enough for the sustained rate but a bottleneck at the ceiling.
- NVMe SSDs deliver approximately 5–7 GB/s, which covers the ceiling with headroom.

Systems insight: A SATA SSD caps the pipeline at 500 MB/s divided across 150 KB images, or approximately 3,333 img/s. That clears the sustained 3,200 img/s, so a faster drive buys nothing at today's rate. Once faster kernels or a faster accelerator push the rate toward the ceiling, the same drive holds an A100-class accelerator to 26 percent of it. Each fix to the slowest stage exposes the next, so the storage tier must be sized for the rate the pipeline is heading toward, not the rate it runs at today.

Designing for high-throughput training therefore starts by matching storage throughput to accelerator demand, the storage-tier form of the slowest-stage bound that section 4.9.3 measured as lost training rate. The 500 MB/s figure represents effective SATA III sequential read throughput (the interface ceiling is 550 MB/s), and real-world random read performance with small files can be significantly lower. That caveat reinforces the general principle governing data pipelines: equation 4.4 bounds training throughput by compute capacity and the data supply rate defined in equation 4.5. Let R_{train}, R_{compute}, and R_{data} denote rates in samples per second; let $\text{BW}_{\text{storage}}$ be storage bandwidth in bytes per second, f_{overhead} the dimensionless fraction lost to overhead, and s_{sample} the bytes per sample. Under full overlap, the min-of-rates form corresponds to the $T_{\text{step}} = \max(T_{\text{compute}}, T_{\text{IO}})$ lower bound of section 4.9.3, which the dataloader section applies to host workers; with incomplete overlap, actual step time is higher.

$$R_{\text{train}} \leq \min(R_{\text{compute}}, R_{\text{data}}) \tag{4.4}$$

$$R_{\text{data}} = \frac{\text{BW}_{\text{storage}}(1 - f_{\text{overhead}})}{s_{\text{sample}}} \tag{4.5}$$

Once R_{data} is the smaller rate, additional accelerator compute cannot raise training throughput; the remedy must increase usable storage bandwidth or reduce bytes per sample. The budget assumes every epoch reads from storage. A dataset that fits in host memory, as an ImageNet-sized dataset does on many eight-accelerator servers, is read from storage once and then served from the operating system's page cache, so storage binds only on the first epoch. The budget governs the cases that remain: datasets larger than memory, nodes whose memory several jobs share, and data streamed from remote storage.

When storage bandwidth becomes the limiting factor, teams must either improve storage performance through faster media, parallelization, or caching, or reduce the amount of data that must move. Large language model training may require processing hundreds of gigabytes of text per hour, while computer vision models processing high-resolution imagery can demand sustained data rates exceeding 50 gigabytes per second across distributed clusters. Prefetching hides the time to fetch the next batch behind the computation on the current one, but only when the loader sustains the required supply rate; it cannot create bandwidth that storage lacks.

4.10.4 The dataloader

Between storage and the accelerator sits the host processor, which reads each stored example, decodes it, augments it, and stages it for transfer. The barrier at this stage is the **input/output (I/O) bottleneck**: the accelerator idles whenever the host cannot stage examples as fast as the accelerator consumes them. The slowest-stage bound of section 4.9.3 applies unchanged: in an idealized two-stage model, step time ranges from $\max(T_{\text{compute}}, T_{\text{IO}})$ with full overlap to $T_{\text{compute}} + T_{\text{IO}}$ with no overlap. What changes is the binding resource: while the initial feeding scenario was starved by baseline storage bandwidth, data loading on provisioned storage is more commonly starved by CPU-side decompression and decode (Mohan et al. 2021).

In computer vision, JPEG decode and augmentation on host CPUs represent the typical binding stage; in keyword spotting, MFCC extraction and audio augmentation load the same host threads. This host-side preparation executes within the framework's data loader (such as PyTorch's `DataLoader`), which decodes and transforms raw records across parallel worker processes before staging batches for device transfer. Input-pipeline frameworks such as tf.data treat this stage as a system in its own right, with operators for parallel transformation, prefetching, and caching (Murray et al. 2021). To isolate CPU worker scaling, assume storage and bus transfers are provisioned to satisfy the 480 MB/s compressed input requirement, which a SATA solid-state drive does (section 4.10.3). When storage and interconnect are nonbinding, the training pipeline depends entirely on whether CPU workers can match the accelerator's sustained rate of 3,200 img/s. If each worker process transforms 375 img/s, 9 workers are required to reach the capacity crossing. Below this threshold the CPU input pipeline binds, and adding accelerators yields no speedup, which surprises teams that expect compute investments to scale linearly. Above the crossing, launching additional workers produces no gain because the accelerator binds. The worker count represents a capacity match rather than a hardware constant: actual ResNet-50 requirements on a given accelerator shift with image transformations, host processor architecture, batch size, and runtime configuration. Figure 4.17 plots this capacity crossing, where the starved regime gives way to a compute-saturated plateau.

At eight accelerators the worker count grows eightfold, more decode cores than many hosts can spare. Relieving the choke point takes three mechanisms: multi-process worker pools that scale decoding across host cores, asynchronous prefetching that copies the next batch to the accelerator while it computes on the current one, and hardware decode engines that move image decompression off the host processor entirely. Hardware decode removes the host bound and hands the constraint back to storage bandwidth, where the remaining lever is the number of bytes each sample costs to move.

↪ How asynchronous prefetching and overlapping pipeline stages hide host preparation latency behind accelerator compute is developed in section 8.6.2.

4.10.5 Formats, compression, and partitioning

Scanning a 100-column CSV file when the model uses only 20 columns wastes four-fifths of disk bandwidth on unused text, and opening millions of individual 150 KB JPEG files spends more time on filesystem seeks than on transferring bytes. File format sets the byte count and access pattern, so format selection directly scales the data term $\left(\frac{D_{\text{vol}}}{\text{BW}}\right)$ of the iron law. Columnar formats like Parquet excel for tabular features by allowing column projection and pushdown filtering, whereas shard-based sequential formats like WebDataset or TFRecord pack millions of small binary objects (such as images or audio) into contiguous sequential streams. This sharding eliminates random disk seek overhead (L_{lat}), turning slow random I/O into sustained sequential storage throughput ($\text{BW}_{\text{storage}}$). The price is shuffling. Records within a shard arrive in order, so loaders approximate a random order by shuffling the shard list and mixing records in a buffer, and too small a buffer feeds the model correlated batches. Redesigning the format and the loader together pays off at the scale of this chapter's lighthouse. One such system trains ResNet-50 on ImageNet to 75 percent accuracy in about 20 minutes on a single machine (Leclerc et al. 2023). Quantifying this impact as *format efficiency* (η_{format}) provides a multiplier on effective bandwidth. While Parquet optimizes columnar layout on disk, in-memory representations face a similar serialization bottleneck when passing data between ingestion engines, dataframes, and ML frameworks. The Apache Arrow standard solves this in-memory boundary by defining a language-independent columnar memory layout, enabling zero-copy data sharing across pipeline processes without expensive serialization and deserialization cycles.

↪ Illustrative CPU decoding throughput and deserialization cost across storage formats are collected for reference in section B.1.2.

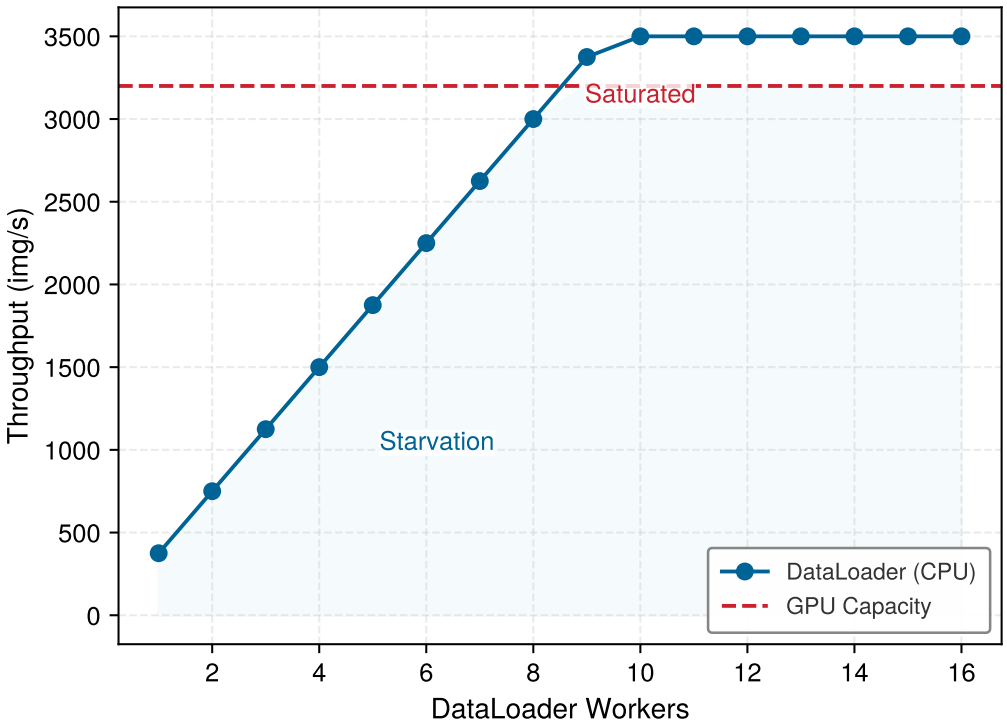


Figure 4.17: The Dataloader Choke Point: Training throughput (img/s) is plotted against DataLoader worker count, with storage and transfer provisioned above the required 480 MB/s rate. The blue CPU-supply curve rises with worker count until it reaches the illustrative input-pipeline cap; the red dashed line marks accelerator consumption capacity. The lower curve is the bottleneck: the accelerator starves to the left of the crossing, while extra workers beyond the crossing do not increase accelerator throughput.

Napkin Math 4.9: Format efficiency

Storage formats determine how much "waste" data must move to retrieve the signal needed for training. Let η_{format} be the dimensionless fraction of bytes read that carry selected features, from 0 to 1. Equation 4.6 shows how that fraction scales physical bandwidth into usable pipeline bandwidth:

$$\text{Effective Bandwidth} = \text{Physical Bandwidth} \times \eta_{\text{format}} \tag{4.6}$$

Values below 1 mean the format spends part of the physical data path on irrelevant bytes. Improving storage layout can therefore raise usable throughput without changing the underlying storage device.

Scenario: Training a fraud model using 20 features from a 100-column table, assuming equal uncompressed byte size per column.

Row-oriented formats (such as CSV) must read all 100 columns to retrieve the 20 required features, giving a useful-byte fraction of 0.2 and wasting 80 percent of disk bandwidth. Conversely, column-oriented formats (such as Parquet) read only the selected columns, achieving $\eta_{\text{format}} \approx 1$ (ignoring metadata overhead) and delivering 5× higher effective throughput.

Systems insight: In this projection-only scenario, switching from CSV to Parquet provides the same effective read-throughput gain as a 5× faster data path.

Columnar storage formats such as Parquet or Optimized Row Columnar (ORC) achieve a 5× to 10× I/O reduction for typical ML workloads through two complementary mechanisms: column projection, and column-level compression that exploits repeated values within a column. Com-

pression is particularly effective for categorical features with few distinct values. A country code column with 200 unique values across 100 million records compresses 20× to 50× through dictionary encoding, while run-length encoding compresses sorted columns by storing only value transitions. Together, projection and compression cut I/O by 20× to 100× compared to uncompressed row formats, shortening training epochs and shrinking storage footprints.

Compression algorithms trade storage density against CPU decompression bandwidth. While gzip achieves higher compression ratios of 6× to 8×, Snappy achieves 2× to 3× compression but decompresses at 500 MB/s, roughly 4.2× faster than gzip's 120 MB/s. Reading a 100 GB dataset compressed with gzip costs 13.9 minutes of CPU decompression per epoch, while Snappy costs only 3.3 minutes. Over 50 epochs, this 10.6 minutes difference per epoch compounds to 9 hours of host CPU time. That CPU time lengthens training only when decompression sits on the critical path, as it does when host-side decode is the binding stage of the dataloader choke point; with spare host cores and prefetching, it hides behind accelerator compute. The trade reverses when storage bandwidth binds instead. Gzip's higher compression ratio means fewer bytes per sample cross the storage path, which raises R_{data} in equation 4.5, so the right codec depends on which stage binds. Snappy suits a CPU-bound loader, and a denser codec suits a storage-bound one.

↪ How row and columnar layouts arrange records on disk, and how projection pushdown reads only the selected column chunks, are collected for reference in section B.1.3.

Beyond format and compression, data partitioning governs retrieval efficiency by aligning storage layout with common access patterns. The KWS corpus can be partitioned by speaker and recording date, so a training run can select a slice without scanning the whole corpus, and the speaker key that assigns splits also governs the storage layout. Partitioning strategies interact directly with distributed training: range partitioning by user ID routes each user's records to one data-parallel rank (workers that each train on a different slice of every batch), which feeds each rank a skewed stream rather than a representative sample, while random partitioning mixes records from the source pool across workers under the sampling policy. It does not remove correlations already present in that pool or prevent leakage between evaluation splits; speaker grouping and other independence constraints must be enforced separately. Granularity is critical: too few coarse partitions limit parallel worker concurrency, while millions of tiny shards add heavy metadata overhead and break sequential disk prefetching. In multi-accelerator training, partition skew creates stragglers. If seven workers in an eight-accelerator job finish loading their batch in 12 ms but one worker reads from an oversized partition and takes 180 ms, that single straggler delays the synchronization step that ends each training step across the entire cluster. Balanced partitioning that matches worker read budgets therefore keeps accelerator efficiency high.

Format, compression, and partitioning complete the delivery path by cutting the bytes each sample costs to move and keeping worker shards balanced. With a storage tier sized for the accelerator, the host matched to its rate, and bytes per sample reduced, both scenarios now have every stage of the pipeline in place, and each can be checked against the targets it started with.

4.11 Accepting the Pipeline

For the keyword spotter, the candidate design of table 4.2 is evaluated as a complete pipeline. The team first records an acceptance contract: the 98 percent keyword detection target on the declared evaluation population, the expected false-wake budget below 1 false wake/month, the 256 KB model limit, and the separate 16 KB preprocessing-state budget. The contract also retains the latency and power requirements of section 4.3. Model bytes alone cannot establish a device fit; peak feature buffers and runtime state must fit the device's total memory allocation.

The false-wake target is a data requirement in its own right. When no false wake is observed, the upper 95 percent confidence bound on the true rate is about three events divided by the hours listened, the zero-event rule of three (Hanley and Lippman-Hand 1983). Supporting a budget of one false wake per month therefore takes about 2,160 hours of representative non-keyword audio, about 248 GB of recordings at 16 kHz. That requirement is as concrete as the label count. The allocation in Notebook 4.2 never costed it, so the team must add an evaluation-collection line, because no amount of training data can stand in for these hours.

To test the choice of 13 rather than 40 MFCC coefficients, both candidates use the same versioned speaker-separated data splits, label budget, and recorded training procedure. Each candidate packages its own fitted preprocessing state with its model. Validation data selects the detection threshold; a separate, untouched evaluation set then measures keyword recognition and false rejection, while

representative nonkeyword streams measure false accepts per hour through the deployed detection path. The team fixes the required statistical confidence and listening duration before evaluation and accepts the false-wake claim only when the upper uncertainty bound, expressed over the same listening duration as the target, meets that target. Insufficient listening time leaves the claim unestablished even if no false wake occurs. Replaying fixed audio through the training and device preprocessing paths checks feature parity, and measurements on the target device establish peak memory, latency, and power.

The budget charges collection and review to the labeling allocation, and generation, retained versions, storage, and processing to their stated allocations. The computed 750K real examples fit only the labeling calculation; additional synthetic data must fit the remaining budget. If the 13-coefficient candidate meets every acceptance condition, the larger representation needs a measured quality benefit worth its added device cost. If the smaller candidate fails quality, the 40-coefficient alternative is acceptable only if it closes that gap while passing the same resource and cost checks. If neither passes, the team revises coverage, representation, or the product requirements and repeats validation; it cannot declare success from the design-space gains alone.

The ResNet-50 scenario has a simpler contract: delivery must not limit the training rate. It passes on a SATA solid-state drive today, provided the images are read as large sequential shards, which lets the drive supply about 3,333 img/s against the 3,200 img/s the accelerator sustains. It fails as soon as faster kernels or a faster accelerator push the rate toward the compute ceiling, which is why the storage tier is sized for the rate the pipeline is heading toward rather than the rate it runs at today.

Acceptance does not end the work of data engineering. In production, input environments continuously drift: sensor hardware shifts, upstream schemas mutate, and annotator pools turn over. Without active operational maintenance, static pipelines quietly degrade over time.

4.12 Operational Data Health

The detection-rate drop that opened section 4.8 can now be traced to its causes. In this illustrative scenario, the keyword spotter launched meeting its 98 percent detection target with verified training-serving consistency. Six months later, the measured detection rate had fallen four percentage points below that target, although no one had touched the model. The decline traces to three silent changes. A new hardware revision's microphone altered the acoustic frequency response, the labels used to measure live performance followed revised guidelines that were never reconciled with the earlier evaluation standard, and an upstream change to the device metadata schema shipped without notice. The changed evaluation labels can alter the measured rate even when predictions for the same inputs stay fixed, so their effect must be separated from the changed inputs. Each failure mode represents a manifestation of **data debt**: accumulated, unmonitored compromises in data quality, schema integrity, and provenance that compound across system boundaries. Unlike traditional software defects that trigger segmentation faults or unhandled exceptions, data debt leaves the execution graph running normally while the underlying statistical distribution decays. Keeping the capability bound true after launch therefore takes two disciplines: watching how the data the system meets drifts away from the data it learned from, and measuring and paying down the debt that accumulates in the pipeline itself. The throughput bound decays more quietly. Datasets grow, preprocessing changes, and a new feature or format can move decode back onto the critical path of training, so the share of each training step spent waiting on input belongs on the same dashboard as the drift metrics, and the arithmetic of section 4.9.3 says which stage to resize when it rises.

4.12.1 Data drift detection and response

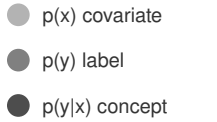
The microphone change in that scenario is drift. Production audio stopped resembling the training audio, and because no contract was violated, nothing raised an error. The validation and monitoring techniques in section 4.5.3 can reveal both sudden and gradual problems; data drift detection focuses specifically on changes in distributions that may alter model behavior over time. Drift monitoring is a core data engineering responsibility.

↪ How drift signals and outcome evidence trigger retraining only when its expected benefit exceeds its cost is developed in section 14.6.1.

Section B.2.3 formalizes the divergence metrics used to make the divergence term $\mathcal{D}(P_t \| P_0)$ of the degradation equation (equation 1.1) actionable. The **population stability index (PSI)** quantifies changes in categorical or binned feature distributions, while **Kullback-Leibler (KL) divergence**

measures how far the current distribution departs from the baseline by weighting the log ratio of their probabilities by the current distribution's probability (Kullback and Leibler 1951). KL divergence is asymmetric, and it becomes infinite wherever the current distribution places probability that the baseline lacks, so monitors must fix its direction and smooth empty bins. The two metrics encode different distributional assumptions and scales, and neither by itself supplies a universal cutoff. Teams calibrate metric-specific thresholds against a deployment baseline and monitor them over time. Crossing a threshold signals that the live distribution has moved far enough to warrant investigation and outcome checks; input divergence alone does not establish whether accuracy improved or degraded, or by how much. Mathematically, PSI discretizes a continuous feature into reference bins (typically deciles of baseline P_0) and computes $\text{PSI} = \sum_b (P_t(b) - P_0(b)) \ln\left(\frac{P_t(b)}{P_0(b)}\right)$. Each bin's two factors share a sign, so every shifting bin adds to the index. Practitioners commonly treat $\text{PSI} < 0.1$ as stable, $0.1 \leq \text{PSI} \leq 0.25$ as moderate shift, and $\text{PSI} > 0.25$ as a prompt for investigation; these bands are conventions to calibrate against the deployment baseline, not physical limits.

Targeted detection and mitigation require decomposing drift into its constituent distribution components (Quiñonero-Candela et al. 2009). The first case, **covariate shift** (Shimodaira 2000), changes the input distribution while preserving the relationship between features and labels: $p(x)$ changes but $p(y \mid x)$ stays the same. The keyword spotter in the scenario above meets exactly this case. The words people say, and how they say them, have not changed, but audio from the new microphone has a different frequency response from anything in the training set. Detection therefore compares input feature distributions, using metrics such as PSI or KL divergence. Audio that never triggers the detector stays on the device, so for the keyword spotter the comparison runs on aggregate feature statistics the devices report, such as per-band energy histograms, rather than on the audio itself.



Each drift type traces to the distribution component that shifted.

The second case, **label shift** (Lipton et al. 2018), changes the output distribution while preserving the relationship between labels and features: $p(y)$ changes but $p(x \mid y)$ stays the same. Disease prevalence might change seasonally while symptoms remain consistent predictors of each disease. A recommendation system might see the same pattern when new product categories launch, changing the relative frequency of user preferences without altering what makes products appealing within each category. Detection can often begin without ground truth labels by tracking shifts in the model's prediction distribution.

The hardest case is **concept drift**: the relationship between features and labels changes, so $p(y \mid x)$ evolves over time (Gama et al. 2014). Medical treatment protocols change, user preferences shift as social trends evolve, and fraud patterns adapt as attackers respond to detection systems. Unlike covariate or label shift, concept drift requires ground truth labels for detection because the system must observe whether the feature-to-label relationship itself has changed.

Drift monitoring also depends on what production data reaches the pipeline at all, and the deployed model decides much of that. The keyword spotter uploads audio, with consent, only when it wakes, so the clips that reach the team are dominated by correct detections and false wakes. A spoken wake word that the detector misses leaves no recording; the user simply says it again. The pipeline's own logs therefore under-report the false rejections that matter most, and measuring them takes a separate, consented collection, such as periodic recording studies or reports of failed activations. Labels arrive late for a related reason: whether a logged activation was a real wake word is known only after review. A monitoring plan built only on the data the deployed model sends back sees the model's successes more clearly than its failures.

Putting the PSI and KL divergence metrics to work in production requires connecting them to automated alerts and review workflows. Data engineering is responsible for defining domain-specific drift thresholds from baseline behavior and outcome evidence, selecting monitoring windows that can expose both sudden and gradual changes, and instrumenting pipelines to compute these metrics continuously. When those alerts or the collection gaps around them go unaddressed, the resulting blind spots accumulate alongside schema workarounds and missing provenance as data debt.

4.12.2 Categories of data debt

The edge KWS failure contains four distinct categories of data debt. The hardware-induced acoustic shift left the training set stale relative to the new microphones, which is *freshness debt*; the divergent annotation standard introduces *quality debt*; and the silent metadata modification creates both *schema*

debt and *documentation debt*. Although each category requires its own diagnostics and fixes, all four share a common root cause: hidden coupling across unmonitored boundaries.

Definition 4.6: Data debt

Data debt is the accumulated cost of implicit coupling and missing documentation across an ML data stack.

1. **Significance**: It manifests as silent performance degradation and compounding maintenance overhead caused by unmanaged pipeline dependencies or undetected distribution shifts.
2. **Distinction**: Unlike a code defect, which often surfaces as a compiler error, a failing test, or a runtime exception, data debt permits computation to complete successfully while producing statistically invalid outputs. It is the data-side form of the hidden technical debt that Sculley et al. (2015) identify in ML systems, and it shares that debt's defining property of staying invisible until it is paid.
3. **Common pitfall**: A frequent misconception is that data debt is just "bad data." In reality, it is a systems architecture failure: it occurs when the assumptions of the training distribution are no longer enforced at the system boundary.

The first category is **documentation debt**: data provenance, column meanings, and collection parameters go unrecorded. When datasets omit datasheets or data cards (Gebru et al. 2021; Pushkarna et al. 2022), feature columns lack descriptions, and transformation steps stay undocumented, pipeline maintenance breaks down as soon as team members change. As Sambasivan et al. (2021) demonstrate, missing metadata and unrecorded collection practices trigger compounding data cascades across high-stakes systems. Downstream engineers must reverse-engineer feature meanings from old ingestion scripts, provenance audits fail compliance checks, and unstated assumptions, such as sensor dynamic ranges or unit conventions, cause silent prediction errors when upstream sensors change.

The second category, **schema debt**, grows out of accumulated transformation workarounds and uncoordinated migrations. When upstream services change payload structures or field meanings, teams often apply local patches, such as regex string extraction or fallback NULL coercions, rather than updating data contracts at the producer boundary. Over time, these patches build brittle parsing layers full of version-conditional branches, duplicate timestamp parsers, and defensive type conversions. At the ingestion boundary, an unvalidated schema change can silently alter the assembled tensor rank, drop sparse features, or introduce out-of-vocabulary indices that collide with reserved embedding tokens. As the pipeline jungle analysis showed (section 4.2.1), upstream contract violations often surface downstream as model convergence failures or serving anomalies.

The third category, **quality debt**, consists of known data defects left uncorrected to meet delivery deadlines or compute budgets. A training corpus with a 3 percent label error rate carries concrete quality debt: every training run on that corpus fits the model against corrupted targets. Quality debt compounds when model predictions feed downstream models as features or pseudo-labels, spreading label noise across dependent services. Warning signs include rising annotation disagreements, unaddressed demographic gaps, and unpurged near-duplicate records that inflate validation scores through data leakage.

The fourth category, **freshness debt**, arises when static training sets fall behind evolving production distributions. A model trained on historical user behavior may meet changed inputs or feature-label relationships after deployment. Calendar age alone does not prove a quality loss; freshness assessment needs evidence of changed conditions and their effect on predictions. Freshness debt accumulates invisibly because a stale model still runs; it is just wrong more often. Warning signs include training intervals that lag behind the rate of distribution shift, stale caches in online feature stores, and static lookup tables that no longer reflect current domain state. Managing these four liabilities requires setting explicit quantitative thresholds for each one.

4.12.3 Measuring and projecting data debt

Code-level debt can at least be approximated with code complexity metrics; data debt requires measurements of the data and its pipeline. Table 4.9 gives example warning and critical thresholds for each debt category, not universal standards, and makes data debt trackable as an engineering service-level objective (SLO), a target value set for a measured quantity.

↪ How a service-level agreement turns an objective into a contractual commitment is developed in section 12.6.2.1.

Table 4.9: Data Debt Metrics: The thresholds are illustrative triggers for detecting data debt accumulation across four categories. Warning thresholds indicate debt requiring attention in upcoming planning cycles; critical thresholds indicate debt requiring immediate remediation to prevent system degradation. The days-since-retraining row is a review trigger, not evidence of degradation. Crossing it prompts the comparison of elapsed time with the measured shift interval that freshness debt calls for. The PSI row uses common operational drift-monitoring bands, while the remaining rows should be calibrated to the risk tolerance and failure cost of the specific ML system.

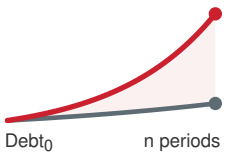
Debt Category	Metric	Warning Threshold	Critical Threshold
Documentation	% datasets with data cards	< 80%	< 50%
Documentation	% columns with descriptions	< 90%	< 70%
Schema	Schema version branches	> 3	> 10
Schema	% transformations with try/catch	> 20%	> 50%
Quality	Known label error rate	> 1%	> 5%
Quality	Documented bias metrics	Not measured	Measured but not mitigated
Freshness	Days since last retraining	> 90 days	> 365 days
Freshness	PSI vs. training baseline	> 0.1	> 0.25

Data debt compounds through coupled feedback loops in production pipelines. Model outputs frequently feed downstream data lakes as pseudo-labels or synthetic features; any uncorrected label bias or corrupted prediction in an upstream model is thereby baked into the ground truth of dependent systems. At the same time, ingestion code decays: each defensive workaround added to handle an undocumented schema change adds branching logic that makes later schema migrations more likely to fail. As distribution drift steadily widens the gap between the training baseline and live inference streams, unmaintained systems demand more and more engineering time just to stay running. To model how sensitive pipeline maintenance is to unaddressed debt, measure debt by a concrete proxy: the remediation backlog in engineer-hours needed to bring every row of table 4.9 back below its warning threshold. Let Debt_0 be that backlog today, r its net growth rate per planning period (estimated from how the backlog grew over past periods), and n the elapsed periods. The planning model in equation 4.7 projects the accumulated liability:

$$\text{Debt}_n \approx \text{Debt}_0 \times (1+r)^n \tag{4.7}$$

A small difference in the periodic accumulation rate r becomes a large backlog over time. This geometric formula is a planning heuristic rather than an exact physical law; in production environments, debt often jumps during discrete upstream migrations rather than compounding smoothly.

4.12.4 Remediation strategies



Small differences in growth rate open a widening debt gap over time.

Once the team measures debt per category, engineers can pay it down during regular operational cycles before pipeline failures disrupt downstream services. Each debt category calls for a different technical fix, but all four follow the pipeline from its upstream boundary to its live monitoring loop. Remediation starts at the ingestion boundary, where documentation and schema debt originate. Making dataset metadata part of the release checklist and reserving maintenance time to record data provenance, column meanings, and preprocessing graphs on the most widely shared feature tables gives downstream teams complete schema descriptions and transformation lineage. With those meanings documented, teams can pay down schema debt by enforcing explicit schema contracts at the producer boundary rather than patching downstream code. Validation libraries such as Pandera or Great Expectations check types, value bounds, and null rules before records enter the lake, so failing fast on schema violations keeps defensive null-coercion and string-parsing workarounds from piling up inside transformation scripts.

Once the ingestion boundary is stable, attention shifts to the corpus itself and its alignment with live traffic. Paying down quality debt requires regular workflows that fix known label errors, duplicate records, and demographic coverage gaps; tracking the error backlog and its burn-down rate lets teams schedule data curation alongside model work instead of deferring it indefinitely. Even a clean corpus eventually drifts out of date, so paying down freshness debt pairs automated drift monitoring with an evidence-based retraining schedule. Adding population stability index (PSI) and Kullback-Leibler (KL) divergence monitors (section 4.12.1) to operational dashboards lets pipelines trigger an investigation whenever distribution distance crosses a calibrated threshold, after which outcome evidence and diagnosis decide whether to refresh features, repair a pipeline, or retrain rather than treating input divergence alone as proof of accuracy loss.

Like technical debt in traditional software engineering, data debt is an inescapable trade-off. *Strategic* debt, such as shipping with minimal documentation or deferred deduplication to meet a launch window, can be a deliberate engineering choice. The failure mode is *unconscious* debt: compromises made without measurement that compound silently until fixing them requires rebuilding the pipeline. Managing data health therefore means measuring debt, choosing deliberately which debt to carry, and diagnosing the remaining faults systematically when a deployed pipeline does regress.

4.12.5 Debugging data pipelines

When data debt comes due, it surfaces as an unexplained accuracy drop, an ingestion crash, or a performance gap on a specific user slice. Debugging draws on three earlier results: data cascades (section 4.2.1) show that upstream ingestion flaws cause downstream model anomalies, training-serving skew (section 4.7.1) exposes mismatches between offline feature generation and online serving, and drift detection (section 4.12.1) catches statistical shifts over time. Figure 4.18 arranges four symptom checks across temporal degradation, train-validation gaps, serving skew, and subgroup disparities. A "yes" at any branch points to a candidate cause, not the end of the investigation. Engineers should run through all four checks because multiple faults often occur together, then confirm each cause by comparing fixed artifacts before choosing a fix.

Production ML failures frequently trace to data dependencies and pipeline changes rather than model architecture alone.[23] Checking the data first, for training-serving skew, distribution drift, and label noise, isolates upstream input failures before any costly architectural redesign or hyperparameter sweep. The KWS regression that opened this section therefore needs three separate comparisons. Replay fixed raw recordings through the old and current preprocessing paths, with model weights and fitted state held fixed; changed features or predictions reveal a processing-path effect on those recordings, but do not establish microphone-population drift. Rescore fixed predictions under the old and revised labeling rubrics; a changed score identifies an evaluation-definition effect without proving that the detector changed. Finally, compare old- and new-microphone cohorts using one validated preprocessing path and one evaluation rubric, accounting for differences in speakers and acoustic conditions. A cohort gap warrants investigation of hardware coverage, but the microphone is not established as the cause unless those other conditions are controlled. Confirmed transform defects call for pipeline repair and revalidation; a remaining quality loss on representative new-microphone audio can justify targeted acquisition and a retraining comparison.

[23] **ML failure distribution**: Treat the data-dominance pattern as a practical triage rule rather than a universal law. Data and pipeline checks are good first hypotheses, but they do not rule out code defects, hardware communication bottlenecks, or numerical instability.

Checkpoint 4.4: From data contracts to diagnosis

A training loader delivers examples more slowly than the accelerator can consume them. A preprocessing version has also changed, and monitoring reports input drift, but outcome labels have not yet arrived.

- ☐ **Delivery**: Which rate limits steady-state training throughput, and what storage or transformation measurement would distinguish the cause?
- ☐ **Reproduction**: Which dataset version, preprocessing code, and fitted parameters must be recovered to compare the old and new paths?
- ☐ **Intervention**: What does the drift alert establish, and what additional evidence is needed before choosing pipeline repair or retraining?

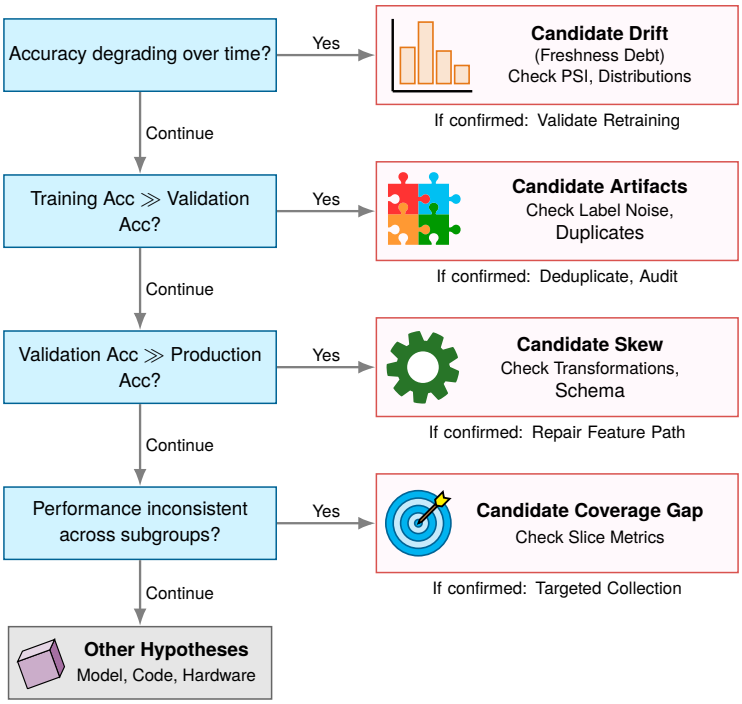


Figure 4.18: Data Pipeline Debugging Flowchart: Four sequential checks prioritize hypotheses involving drift, overfitting or data artifacts, training-serving skew, and subgroup coverage. Each side branch suggests a candidate cause to confirm before applying the indicated repair. Other hypotheses include model, code, and hardware faults.

4.13 Fallacies and Pitfalls

Data engineering pipelines rarely fail with immediate runtime crashes; instead, they silently distort feature distributions, corrupt training state, or starve expensive accelerator clusters. The nine fallacies and pitfalls that follow collect the recurring assumptions behind those failures, pairing each with the pipeline measurement or contract that prevents it.

Fallacy: *More data always improves model performance.*

The bitter lesson says that general methods fed with more data and compute keep winning, but it does not say each new sample is worth as much as the last. Beyond a threshold, additional data yields diminishing returns described by empirical scaling laws, fitted curves relating model error to dataset size. Studies across computer vision, translation, and language modeling find that test loss falls as a power law in dataset size, so each successive error reduction demands a multiplicatively larger volume of samples (Hestness et al. 2017). In the illustrative KWS design space, a tenfold larger training set buys 5–8 percent more detection rate at 10× the labeling cost. In systems terms, repeated examples consume disk I/O, host-to-device bus bandwidth, and accelerator FLOPs without adding distinct observations. Repeated passes can still shift the model slightly, so redundancy must be judged by its measured effect on evaluation rather than assumed to contribute nothing. The task-relevant signal heuristic from section 4.9 captures this trade-off: scaling raw byte volume without preserving information density inflates infrastructure cost while yielding negligible improvements in evaluation loss.

↪ Scaling laws, and the selection methods that decide which samples are worth their training compute, are developed in section 9.1.

Pitfall: *Planning petabyte migration as a bandwidth-only transfer.*

Dividing data volume by network link bandwidth gives the theoretical wire time, about 22 h for the chapter's 1 PB example at 100 Gb/s, but wire transfer is rarely the critical path. Data gravity (section 4.9) binds large datasets to a web of surrounding infrastructure: ingestion transformation pipelines, schema validation checks, access-control policies, and downstream consumers. Moving a petabyte-scale corpus between clouds or data centers requires refactoring pipelines, revalidating

data integrity, re-warming caches, and rebuilding metadata indexes. In practice, the engineering time spent adapting dependent systems dwarfs the raw network transit time.

Fallacy: *Data preprocessing can be finished once and left alone.*

Preprocessing pipelines turn raw observations into tensors using parameters fitted on historical data: tokenization vocabularies, mean and variance scaling vectors, category frequency tables, and numerical clipping bounds. When production distributions drift, these fixed transformations go stale. Tokenizers hit rising out-of-vocabulary rates, fixed normalization constants push scaled features outside the range the model trained on, and static clipping bounds cut off valid signal. Keeping the pipeline accurate requires monitoring input tensors and outcomes, then validating any change to preprocessing together with the model that consumes its features. Recomputing statistics independently at serving time can itself introduce skew.

Pitfall: *Ignoring data serialization cost.*

Profiling often focuses exclusively on accelerator execution while data loaders quietly parse raw JSON or CSV files. Parsing text-based records requires host CPU cores to spend hundreds of cycles per sample on string decoding, tokenization, ASCII-to-float conversions, and heap allocations. This host-side parsing starves the accelerator just as the dataloader choke point does (section 4.10.4). In the chapter's ResNet-50 scenario, JPEG decoding already occupies 9 workers per accelerator, each preparing 375 img/s. A tabular or log-feature pipeline that parses text records pays the string-decoding cost described here on top of any such preparation. Binary serialization formats, such as Apache Arrow or memory-mapped arrays, eliminate parsing overhead by storing numbers directly in machine-native binary layouts that the loader can hand to the accelerator without conversion. For tabular schemas, columnar formats such as Parquet cut disk I/O by reading only the required columns. Benchmarking data ingestion against accelerator consumption rates shows whether host CPU deserialization is throttling training throughput.

Fallacy: *High training accuracy indicates production readiness.*

Training accuracy and held-out validation accuracy both describe the historical distribution the data was drawn from. Production adds edge cases, sensor noise, and live drift, and a model with high validation accuracy still fails if upstream feature pipelines change or inference timeouts trigger fallback defaults. Diagnosing post-deployment regressions requires the symptom checks across temporal drift, label artifacts, and training-serving skew ordered in figure 4.18, rather than reliance on a single offline score.

Pitfall: *Ignoring training-serving skew until deployment.*

Training-serving skew occurs when feature generation logic diverges between the offline training pipeline and the online inference runtime, for example through differences in floating-point precision, time-window aggregation, library versions, or temporal leakage from data timestamped after the prediction point. Because the model still receives valid floating-point vectors, inference raises no runtime exceptions while prediction quality degrades. Avoiding this pitfall requires shared transformation logic, reproducible feature stores, and automated predeployment parity tests that check feature equivalence across offline and online paths.

Fallacy: *Synthetic data can fully replace real-world data collection.*

Synthetic data enriches training sets by generating rare edge cases, controlled physical variations, or privacy-safe samples, but a generator can only sample within the assumptions built into it. When trained exclusively on synthetic data, models overfit to generator-specific artifacts and fail on real-world variation the generator never modeled. In a KWS system, for example, a detector trained entirely on synthesized speech loses accuracy when it meets real room reverberation, unfamiliar accents, microphone distortion, and shifting background noise. Synthetic generation supplements real collection, but representative real-world recordings remain essential for testing whether the system is ready to deploy.

Pitfall: *Neglecting data versioning until model debugging requires it.*

Treating training datasets as mutable object-storage directories destroys experiment reproducibility. When a retrained model loses accuracy, an unversioned pipeline cannot show whether the

cause is a modified labeling rubric, a silent schema change, or genuine distribution shift, and once tables are overwritten or raw logs deleted, the historical training inputs cannot be rebuilt. As established in section 4.8.1, every model checkpoint must link immutably to a specific dataset snapshot, transformation code hash, and configuration file.

Fallacy: *Production logs show where the model fails.*

A deployed system decides much of what data comes back from it. The keyword spotter uploads audio only when it wakes, so its logs fill with correct detections and false wakes, while a wake word it misses leaves no recording at all (section 4.12.1). A team that measures quality from those logs alone sees a false-rejection rate lower than the real one, and retraining on the same logs over-represents the cases the model already handles. Measuring failures requires collection designed for that purpose: consented recording studies, reports of failed activations, and evaluation streams kept independent of the deployed model's choices.

4.14 Summary

A trained model is bounded twice by its data. The capability bound follows from the data-as-code principle (1): a changed dataset changes the trained model even when no algorithmic code changes, and a defect introduced in collection, schema, or labeling amplifies through every later stage as a data cascade. The throughput bound is the physics of the iron law's data term. The pipeline must deliver bytes at the rate the accelerator consumes them, a requirement the feeding loss, the dataloader choke point, and the storage bandwidth budget each quantified at a different tier of the same ResNet-50 scenario. Data engineering therefore works in two overlaps of the D·A·M taxonomy at once. The capability half worked the Data–Algorithm overlap and the throughput half the Data–Machine overlap, and operations exists because the deployment distribution keeps shifting after launch, which reopens both overlaps. Acquisition, labeling, and feature contracts decide what the model can learn and keep training and serving computing the same function, while storage tiers, formats, and loaders decide how fast it learns.

Key Takeaways: Data is the source code

- **Data is code; version it, test it, review it**: Under the data-as-code principle (1), a dataset change is a code change. Give it version control, validation tests, and review.
- **Data cascades make upstream quality the highest-return investment**: The compounding failure arcs in figure 4.1 show how early collection errors bypass intermediate testing, requiring extensive rework across downstream artifacts and, in severe cases, complete pipeline restarts from deployment. The four pillars (quality, reliability, scalability, and governance) organize prevention, while documentation, schema, quality, and freshness debt require continuous remediation.
- **Pipeline architecture choices have large cost implications**: The chapter's provisioning scenario costs streaming above batch, while its matched-retention example trades ETL storage savings against schema-change overhead. Actual costs depend on provisioning, retention, replay, and required freshness. Select ingestion patterns by the value of latency, not the appeal of real time.
- **Labeling can dominate cost and requires substantial resource allocation**: Labeling can cost thousands of times one optimized training run; in the reference calculation in section 4.6.2, the ratio is 2,498× to 4,995×. Labeling can remain a costly scheduling bottleneck even when annotation work is parallelized.
- **Training-serving consistency is nonnegotiable**: Feature transformations shared by training and serving must have equivalent semantics and reuse the same learned state, such as normalization constants and vocabulary mappings.
- **Storage hierarchy sets input-loading time**: The 50× throughput gap between local NVMe (5 GB/s) and one cloud object-storage connection (100 MB/s) changes input-loading time under the stated bandwidth assumptions. Total iteration time also includes transformation and model execution, so the loading ratio is not an end-to-end speedup.

- **The degradation equation becomes actionable through drift and outcome monitoring**: Divergence metrics such as KL measure changes between P_t and P_0, while labeled outcomes or validated proxies determine whether those changes correspond to model degradation.

The keyword spotter carried the capability bound through the chapter, and ResNet-50 carried the throughput bound. The keyword spotter's one-false-wake-per-month target turned an accuracy goal into a per-window false-positive bound whose evaluation requires representative negative audio, sufficient listening duration, and an explicit treatment of measurement uncertainty. Its fixed budget turned "more data" into a costed allocation of labels, storage, and governance. Its MFCC extraction had to agree, within a validated tolerance, between a cloud training cluster and a kilobyte-scale always-on island. Data engineering is the discipline of keeping two contracts at once, a physical contract with the machine that must be fed and a statistical contract with a deployment distribution that keeps moving, and the failures this chapter traced, from a starved accelerator to a silently retyped ZIP code, each came from a pipeline that kept one contract while breaking the other.

What's Next: From data to computation

The data pipeline now delivers a clean, versioned training set at the rate the accelerator consumes it, yet this chapter has treated the model at the end of that pipeline as a consumer of bytes and nothing more. Chapter 5 opens the consumer along the algorithm axis. It begins with the shift from hand-written logic to learned arithmetic, then counts the multiply-accumulate operations, activations, and gradient updates that turn prepared examples into learned behavior, and the memory and compute each one demands of the machine.

II

DEVELOPMENT AND TRAINING

Part II

Development Principles

Building a machine learning system requires more than assembling model components; it requires managing the flow of information and energy through silicon. Part I fixed where a system can run, when its constraints must be known, and how its data is delivered, and it established that data is both the program and a physical anchor of every ML system. With that anchor in place, Part II turns to the algorithm-machine interaction, asking *how* mathematical models are co-designed around the physical limits of the hardware that must execute them. The bitter lesson favors general methods that absorb more computation, but every such method must still fit the machine that runs it. The principles here begin with the accounting that determines why certain architectures succeed while others fail at scale. That accounting, the iron law, has been in use since the introduction; it is stated here as a principle because every chapter of this part measures its decisions against it.

Principle 3: The iron law of ML systems

Decomposition: Let T denote idealized wall-clock time in seconds. Across the D·A·M axes, workload time factors as $T = (\text{Samples/Task}) \times (\text{FLOPs/Sample}) \times (\text{Seconds/FLOP})$, where the first two factors multiply to fix total floating-point operations O. Unpacking how the machine executes those operations into physical terms, where D_{vol} is data volume in bytes, BW is the effective bandwidth of the relevant memory or network path in bytes/s, R_{peak} is peak compute rate in FLOP/s, η_{hw} is dimensionless hardware utilization efficiency, and L_{lat} is fixed latency overhead in seconds (such as a kernel launch or network round trip), gives the serialized sum:

$$T \approx \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$$

The full treatment appears in section 1.7.

When data movement and compute overlap on modern hardware, wall-clock time follows the critical path and cannot be shorter than the slower of those stages plus any fixed latency that remains outside the overlap. The practical lesson is about dominance, not unconditional summation.

Implication: Optimization is rarely free of trade-offs. Reducing one factor or term often shifts the bottleneck to another. For example, skipping zero-valued weights scattered irregularly through a model (unstructured sparsity) reduces FLOPs/Sample and therefore executed arithmetic O. The model must then also store and read the positions of the surviving weights, however, and that index traffic can increase data-movement time $(D_{\text{vol}}/\text{BW})$ and inflate realized Seconds/FLOP. An optimization improves wall-clock time only to the extent that it shortens the critical path on the target hardware.

The iron law names the terms; it does not say which one a given model will stress. Chapter 2 examined the hardware's side, the terms each deployment tier offers. Choosing the model architecture makes the other side of the commitment, before any code runs.

Principle 4: The silicon contract

Heuristic: The hardware fixes three time limits, a memory bandwidth BW, a peak compute rate R_{peak}, and a fixed overhead L_{lat}, along with a memory capacity that decides what fits at all. Every model architecture, in each workload regime, commits in advance to saturating one of those limits first.

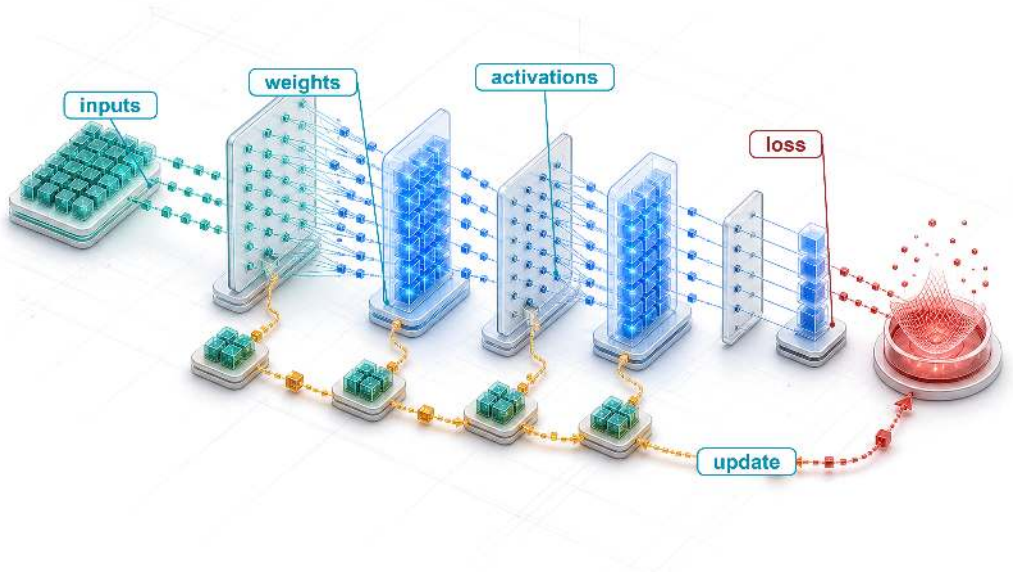
- **ResNet-50**: Assumes high-density floating-point compute. In batched training or inference on accelerators, it is often compute bound: performance is limited by $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$.
- **GPT-2/Llama**: Assumes high-bandwidth memory access during autoregressive decoding, which generates one token at a time and rereads every weight for each token. At small batch sizes, it is often bandwidth bound: performance is limited by D_{vol}/BW.
- **DLRM**: Assumes massive embedding tables and sparse lookups. It is shaped by both capacity and bandwidth: performance depends on whether embedding tables fit in the available memory hierarchy and how quickly sparse accesses can be served.

Implication: Designing a model without knowing which hardware resource it will saturate is like designing a bridge without knowing the strength of the steel. The design must target the bottleneck.

Together, the iron law and the silicon contract frame every design decision in Part II. Chapter 5 counts the operations, activations, and gradients that a neural model commits the machine to. Chapter 6 shows how arranging those computations sets a model's arithmetic intensity, and with it the architecture's side of the contract. Chapter 7 follows the translation of model operations into hardware execution and locates the fixed overhead that translation spends. Chapter 8 then repeats the resulting step many thousands of times, until memory, time, and cost budgets decide what one machine can train.

5

Neural Computation

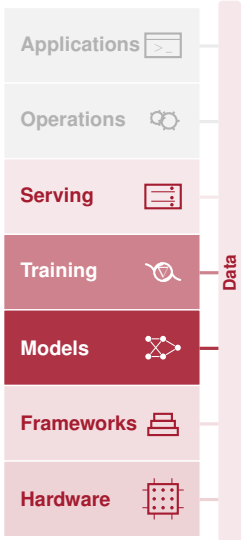


5.1 From Logic to Arithmetic
5.2 Computing with Patterns
5.3 Neural Network Fundamentals
5.4 Learning Process
5.5 Inference Pipeline
5.6 Handwritten Digit Recognition
5.7 D·A·M Reading of Neural Computation
5.8 Fallacies and Pitfalls
5.9 Summary

Purpose

Why does understanding a neural network's math matter more than reading its code?

Neural network code can look deceptively simple because a few tensor operations hide the workload that every layer of the system must execute. Matrix dimensions determine arithmetic and data movement; intermediate values determine memory pressure; numerical ranges and gradients determine whether learning remains stable. An operation that looks cheap in isolation can dominate a system when repeated across a batch or when its outputs must be retained for learning, so source-code length gives little indication of resource cost. Doubling both input and output dimensions quadruples a dense layer's arithmetic, and keeping activations for the backward pass can multiply memory use several times over, while the code looks no more complicated than before. Two implementations of the same model can run very differently because batch shape, precision, and the target machine change what fits, how quickly it runs, and whether its calculations remain numerically valid. When training stalls or deployment exhausts memory, reading a framework call rarely identifies which resource is the binding constraint. A numerical failure, a memory bottleneck, and an inefficient implementation can produce similar high-level symptoms while demanding different remedies. Diagnosis and planning therefore require reasoning from the underlying operations to the bytes moved, values retained, and work repeated. This view can expose a mismatch before an expensive training run or deployment attempt does, when correcting it still costs an edit rather than a week of lost compute. Those primitives provide a stable vocabulary across architectures and make software abstractions auditable, turning opaque code into a workload that can be measured, diagnosed, and mapped to hardware. In D·A·M terms, neural computation is the interface where algorithmic structure becomes machine work and where physical limits feed back into model design.



Learning Objectives

- Explain why learned arithmetic replaced rule-based logic and why it fails numerically rather than logically
- Compare activation functions by nonlinearity, gradient behavior, and hardware execution cost
- Calculate parameter counts, multiply-accumulate operations, activation storage, and the arithmetic intensity that batching raises in dense layers
- Apply cross-entropy loss and gradient updates to reason about supervised learning dynamics
- Derive forward and backward propagation as matrix operations over batches
- Analyze training vs. inference workloads using compute, memory, and deployment constraints
- Evaluate an end-to-end document-recognition pipeline from preprocessing through decision thresholds and human review

5.1 From Logic to Arithmetic

RECOGNIZING one handwritten digit costs the small network at the center of this chapter 109,184 MACs. A multiply-accumulate (MAC) multiplies one weight by one input value and adds the product to a running sum (acc $\leftarrow$ acc $+ w \cdot x$), and the usual convention counts it as two floating-point operations (1 MAC $= 2$ FLOPs), so the digit costs 218,368 FLOPs. None of that work is a logical branch. A program that once tested pixel thresholds and stroke patterns with if-then-else rules has become a fixed sequence of arithmetic, and how fast it runs now depends on how quickly the machine can supply operands to that arithmetic, whether memory bandwidth or arithmetic capacity runs out first. Such a program also fails differently. It rarely fails logically; it fails numerically, as values grow or shrink until they stop carrying information.

The pipeline of chapter 4 ends with a clean, versioned training set. One example from such a set, a handwritten digit, is where the model's work begins. An example does not learn on its own. The next system boundary is the model, the algorithm axis of D·A·M, where data becomes learned behavior, and this chapter follows what the model does with that one digit. Matrix multiplications and activation functions carry it forward to ten scores, the chain rule carries the error in those scores back as gradients, and an update folds the gradients into the weights.

The silicon contract (principle 4) says which hardware resource a model architecture will saturate first. This chapter supplies the model's side of that contract. The architecture's mathematical operators fix the work to be done and the state to be held. Precision, data layout, implementation, and hardware then fix how many bytes actually move, and therefore the realized latency and energy. Honoring the contract requires understanding both the operators and how they map to the machine.

↩ The hardware's side of the silicon contract, the terms that memory bandwidth, peak compute rate, and fixed overhead set for any model, is established on the single-node stack in section 2.4.1.

The digit passes through five stages in the life of a model. The *forward pass* carries the image through three weight matrices to ten class scores, the *loss* reduces those scores to one number that says how wrong they are, the *backward pass* carries that signal back through the layers with the chain rule, the *update* adjusts every parameter, and *serving* runs the forward pass alone with frozen weights. The iron law of ML systems (principle 3) charges each stage for the bytes it moves, the operations it performs, and its fixed overhead, and table 5.1 shows how differently the five stages spend those terms. The loss costs almost nothing, yet it is the only signal training receives; the update does little arithmetic but touches every parameter; the backward pass cannot run without the activations the forward pass left behind. Training repeats the first four stages millions of times and serving repeats only the first, and that asymmetry produces the chapter's central result, the gap between what training and inference must hold in memory.

Table 5.1: Where Each Stage Spends Its Budget: Work, data movement, and retained state for the five stages in the life of a model, for the running 784-128-64-10 network. Training repeats the first four stages for every batch, while serving runs only the forward pass. Later sections derive each row, and the summary returns to the table.

Stage	Work	Data moved	State left behind	Usually dominant cost
Forward pass	One MAC per weight per example (109,184 MACs per digit)	Every weight read; each layer's activations written and read	Activations, kept until the backward pass	Bytes at small batch, operations at large
Loss	A few operations per class	Ten scores per example	One number per batch	Negligible
Backward pass	Up to twice the forward pass	Weights and saved activations read; gradients written	Gradients	Operations or bytes, as in the forward pass
Update	A few operations per parameter	Every weight, gradient, and optimizer value read and written	Optimizer state	Bytes
Serving	The forward pass alone	Weights plus two short-lived activation buffers	Nothing beyond the weights	Bytes and fixed overhead at small batch

Example 5.1: Running example: MNIST digit recognition

Scenario: A handwritten-digit classifier receives 28×28 grayscale images from the Modified National Institute of Standards and Technology (MNIST) dataset (LeCun et al. 1998), flattens each into 784 pixel values, and maps them through hidden layers of 128 and 64 units to 10 class scores, one per digit ($784 \to 128 \to 64 \to 10$).

Diagnosis: The network holds 109,386 parameters, which occupy 438 KB in 32-bit floating point (FP32), and it spends 109,184 MACs (218,368 FLOPs) to classify one digit. Those three numbers summarize what one classification costs.

Systems lesson: The task is small enough to compute by hand yet exercises every mechanism that larger networks scale up. Each later section either derives one of these numbers or asks what happens to it when the network trains, grows, or ships.

Even this small network is deep in the sense the field uses. Each of its layers computes new features from the layer before, and training learns those features from examples instead of receiving them from a designer.

Definition 5.1: Deep learning

Deep learning is the computational paradigm that learns layered feature representations from data by composing nonlinear transformations. For suitable tasks, those learned representations can reduce manual feature engineering while increasing computation (O) and state.

1. **Significance**: For suitable compositional function classes, depth can represent a hierarchy much more parameter-efficiently than a shallow network. Each layer adds work and state, and the resulting binding constraint may lie on the data, algorithm, or machine axis depending on the workload.
2. **Distinction**: Shallow methods such as logistic regression and support vector machines can work well with appropriate representations. Deep learning instead uses multiple nonlinear stages to learn representations jointly with the prediction task, which can reduce manual feature design and sometimes support transfer across tasks.
3. **Common pitfall**: A frequent misconception is that parameter count alone explains deep learning. Two networks with the same parameter count can differ sharply in what they learn, because architecture, data, optimization, and scale together determine the practical result.

Learning representations end to end also introduces engineering failure modes that conventional software debugging cannot isolate. Deep learning failures produce subtle symptoms: gradient instabilities[1] that halt convergence, numerical precision errors that corrupt parameter state across millions of updates, or inefficient memory access patterns in tensor operations (operations on multidimensional arrays of numbers) that leave accelerator compute units underutilized. Resolving these failures requires reasoning about the physical execution of tensor algebra on hardware rather than stepping through procedural control flow, and those hardware demands depend first on representation: whether patterns are expressed through hand-authored rules, engineered statistical features, or learned parameters.

5.2 Computing with Patterns

One task makes the history concrete: classifying a 28×28 grayscale image of a handwritten digit. Computing has attacked that task with three representation strategies, hand-authored decision logic, engineered feature extractors, and learned neural representations. Under illustrative implementations, one digit costs roughly one hundred comparisons under the first, several thousand arithmetic operations under the second, and more than one hundred thousand MACs under the third. Each approach replaced one whose representations ran out, and each shifted which hardware resource binds.

5.2.1 From rules to engineered features

Writing explicit `if-else` rules to recognize a handwritten digit breaks as soon as a writer tilts a stroke, thickens a line, or leaves a loop open. Rule-based programming requires developers to specify every procedure that maps inputs to outputs: in the classical flow of figure 5.1, human-authored rules and input data enter the program together, and the program deterministically computes the answers.

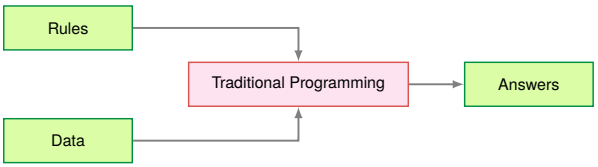


Figure 5.1: Traditional Programming Flow: Answers are the program's output, never its input. This input-output pattern formed the basis for early AI systems but lacks the adaptability needed for complex pattern recognition tasks.

An illustrative rule-based classifier for the digit compares pixel intensities against fixed thresholds, checks stroke patterns in localized regions, and branches on the results. Under this toy implementation, the computation requires roughly 100 comparisons over 784 bytes of pixel data, a workload that is small, predictable, and easily held in an ordinary CPU cache. The comparison establishes a workload baseline, not the accuracy of a production rule system. Rules fail on coverage rather than cost: every new writing style demands new branches, and the expert systems of the 1970s and 1980s stalled on perception tasks because visual competence is implicit and resists explicit rules.

The failures of rule-based systems suggested an alternative. Rather than encoding human knowledge as explicit rules, the system could discover patterns from data, and machine learning took that direction by replacing a rule for every situation with programs that identified patterns in examples. The success of these methods, however, still depended heavily on human insight to define which patterns to look for, a process known as feature engineering.

Feature engineering transformed raw data into representations that expose patterns to learning algorithms. The histogram of oriented gradients (HOG)[2] (Dalal and Triggs 2005) exemplifies this approach. In figure 5.2, the feature-extraction pipeline divides an image into spatial cells and bins local pixel gradient directions into eight orientation histograms, turning raw pixels into shape descriptors that tolerate lighting changes and small shifts in position.

Feature engineering raises the digit's cost while keeping it CPU friendly. For the same 28×28 digit, an illustrative HOG implementation divides the image into a 7 by 7 grid of 4 by 4 cells, computes gradient magnitudes and orientations, bins them into 9 orientation histograms per cell (the diagram

[1] **Gradient instabilities**: In a simplified 20-layer path through sigmoid (S-shaped) activations, the activation-derivative factors alone are bounded above by 0.25^{20}, or 9.1×10^{-13}. The complete gradient also includes weights and other operations, but repeated saturated derivatives can still make early-layer updates negligible. Plateaus or not-a-number (NaN) values can reveal the symptom without identifying the cause; rectified linear unit (ReLU) activations and careful initialization (both taught later in this chapter), together with normalization, better optimizers, and residual connections, all helped make deep networks trainable, and section 6.4.1 treats residual connections in detail.

↩ The succession of logic, knowledge, feature-engineering, and infrastructure bottlenecks that these paradigms answered is established in section 1.3.

[2] **Histogram of oriented gradients (HOG)**: An influential handcrafted descriptor for pedestrian detection before deep learning (Dalal and Triggs 2005). One common configuration computes gradient orientations in 8×8 pixel cells, with cell size and other choices tuned for the task. HOG's fixed computation graph can run efficiently on CPUs with predictable latency, while learned features can reduce descriptor redesign but do not automatically transfer across domains and often benefit from accelerators only as model and batch scale grow.

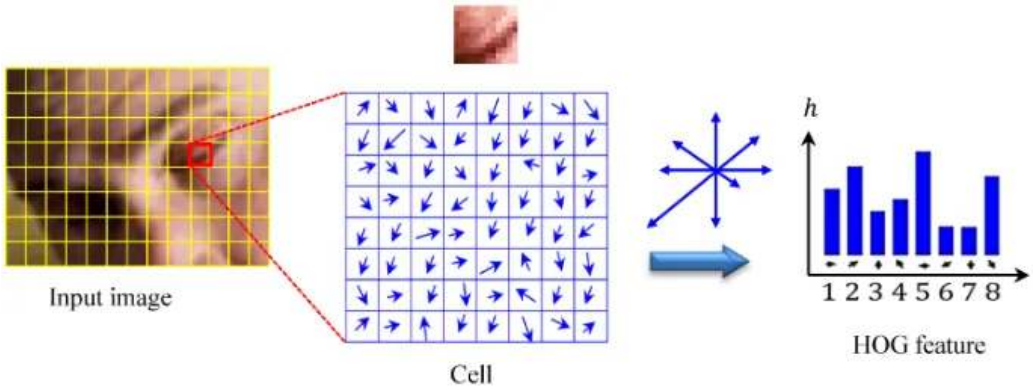


Figure 5.2: HOG Method: A local image patch is divided into cells, gradient orientations are accumulated into a histogram, and the resulting descriptor summarizes local shape while reducing sensitivity to illumination. Source: Figure 5 of Bakheet and Al-Hamadi (2021), licensed under CC BY 4.0.

draws eight directions; the bin count, like the cell size, is a tuning choice), and produces a 441-element feature vector. A linear support vector machine (SVM) classifier then performs ten dot products over that vector. The result is roughly 8,000 operations and about 2 KB of working memory, about 80× the work of the toy rule-based baseline, but still structured and well served by CPU vector units using **single instruction, multiple data (SIMD)**. Descriptor size, class count, and implementation all change this cost.

These descriptors carried face and pedestrian detection through the 2000s, but every new problem needed an expert to design a new extractor, and the resulting features missed whatever patterns their designers had not anticipated. Human expertise, not computation, set the ceiling on how much real-world visual variation a system could absorb.

5.2.2 Automatic pattern discovery

Neural networks remove that ceiling by discovering their own features, learning representations directly from raw data instead of following explicit rules or human-designed extractors. Supervised learning inverts the traditional programming relationship. Engineers provide examples and target answers, and optimization produces the decision rules as output rather than taking them as input (figure 5.3). Humans still shape the task through data, objectives, architectures, and evaluation.

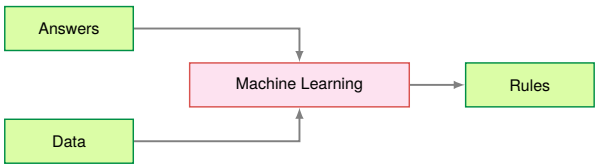
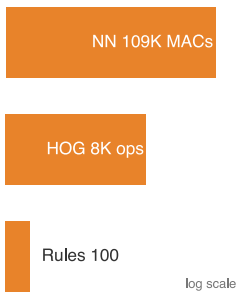


Figure 5.3: Data-Driven Rule Discovery: The flow diagram inverts the traditional programming pattern: data and answers serve as inputs to the machine learning process, which produces a learned decision function as output. This shift replaces a hand-coded decision policy while leaving engineers responsible for the data, objective, architecture, and evaluation.

The same digit, processed by the chapter's three-layer network (784 → 128 → 64 → 10), costs 109,184 MACs, 1,100× the work of the toy rule-based baseline. The 109,386 parameters occupy 438 KB in FP32, more than most L1 caches hold, so even inference creates traffic between cache levels. Training multiplies the cost further. Each image is processed forward and backward before an update, at two to three forward passes of dense-layer arithmetic, repeated over 60,000 images for multiple epochs (full passes over the training set). Dense matrix multiplications now dominate the arithmetic and favor parallel hardware, shifting the engineering challenge from control-flow branching to memory bandwidth, data reuse, and hardware parallelism.

This trade is the bitter lesson at the scale of a single layer. HOG encodes an expert's knowledge of edges and orientations in 8,000 operations that never change; the learned first layer spends far



In this MNIST comparison, learned features spend far more arithmetic.

more arithmetic and lets the data decide which pixel combinations matter. As computation became cheap enough, the general method that converts arithmetic into accuracy overtook the hand-built one. That victory demands the arithmetic, memory traffic, and training infrastructure that the rest of this chapter quantifies.

↩ Why general methods that scale with computation keep overtaking methods built on human knowledge, the bitter lesson, is established in section 1.4.

The escalation matters less for its 1,100× operation count than for what each step does to the binding resource, which moves from branch-heavy CPU control, to batch-oriented feature pipelines, to memory-fed matrix parallelism (table 5.2). Dense matrix work meets the memory wall, the widening gap between compute throughput and off-chip memory bandwidth, whenever an operation performs too little arithmetic per byte it moves. That ratio is the arithmetic intensity introduced alongside the iron law; the forward-pass analysis later in this chapter measures it for the running network, and section 11.6 develops it into a full performance bound.

↩ The memory wall and the physical reasons that off-chip bandwidth lags arithmetic are established in section 2.2.

↪ The AI memory wall, the point at which added FLOP/s stops reducing runtime because data movement dominates, is developed in section 11.5.1.

Table 5.2: System Resource Tendencies: The illustrated implementations shift from control-oriented rules to structured feature pipelines and then to matrix-heavy neural computation. These are workload tendencies rather than requirements of every program in each paradigm.

System Aspect	Traditional Programming	ML with Features	Deep Learning
Computation	Often control-flow oriented	Structured parallel ops	Often matrix-parallel
Memory Access	Workload dependent	Commonly batch-oriented	Large tensor working sets
Data Movement	Program and input dependent	Feature-pipeline movement	Parameter and activation movement
Hardware Needs	Commonly CPU-centric	Often CPU with vector units	Frequently accelerator-friendly
Resource Scaling	Input and program dependent	Descriptor and data driven	Driven by width, batch, state, and reuse

Scale therefore has costs as well as gains. Larger models and training budgets turn arithmetic throughput, memory bandwidth, storage capacity, and communication into potential bottlenecks, and model size is not a decision rule. Choosing a model requires comparing candidates on representative deployment data and on the complete cost of training, serving, monitoring, and maintenance, with the uncertainty in both gains and costs included. The useful question is whether the measured gain from additional scale earns those system costs.

↪ How model size, training data, and compute trade off along a compute-optimal frontier is developed in section 9.2.3.

Systems Perspective 5.1: When to use neural networks

Not every problem benefits from deep learning. Section 3.5 established that a problem definition must require a learned system to improve on an existing baseline by more than its added cost. Table 5.3 lists the conditions under which a neural network tends to win that comparison, and table 5.4 the conditions under which a classical method tends to match it with less training, serving, and maintenance effort.

Table 5.3: Neural-Network Candidates: Conditions under which deep learning is likely to outperform classical methods.

Condition	Evidence to seek	Rationale
Representative data	Training and validation cover deployment conditions	Capacity cannot replace missing coverage
Exploitable structure	Spatial, sequential, or hierarchical patterns	Architecture can encode a useful prior
Baseline gap	Simpler methods miss the required task-quality target	Complexity buys a measured gain
Operational fit	Gain survives latency, memory, and cost limits	Offline accuracy alone is insufficient
Maintenance advantage	Learned features reduce recurring manual updates	Full-system maintenance becomes simpler

Table 5.4: Classical-Method Candidates: Conditions under which simpler models are likely to match or beat a neural network.

Condition	Baseline to test	Reason to prefer it
Limited labeled data	Linear model or tree ensemble	Lower variance and easier validation
Structured tabular inputs	Gradient boosting or linear model	Strong accuracy without learned features
Approximately linear signal	Linear or additive model	Transparent and inexpensive execution
Hard resource bound	Rules or compact classical model	Predictable latency, memory use, and energy demand
Direct auditability needed	Constrained tree or linear model	Decisions are easier to inspect

Systems insight: Establish a simple baseline before building a neural network, and retain the neural approach only when its measured benefit justifies the added system. The document-recognition history in section 5.6 shows why the comparison must include preprocessing, rejection policy, and the cost of human review rather than model accuracy alone.

5.2.3 Why backpropagation waited for systems

Every increase in a neural network's size multiplies its forward operations, saved activations, gradient computations, and parameter updates, which is why backpropagation waited decades for hardware and data to catch up with the algorithm. Paul Werbos applied backpropagation[3] to neural networks in 1974 (Werbos 1974), building on Seppo Linnainmaa's 1970 work on automatic differentiation (Linnainmaa 1970), and Rumelhart, Hinton, and Williams showed in 1986 that hidden units could learn useful internal features (Rumelhart et al. 1986). That demonstration did not resolve the systems cost exposed by the digit classifier: the decades between early backpropagation work and widespread adoption required the maturation of parallel hardware, large datasets, weight initialization, non-saturating nonlinearities, and software frameworks. An algorithm can be viable in principle long before its surrounding system makes it useful in practice. The lesson does not run in reverse: a computationally expensive idea is not vindicated by expense alone, and it must be evaluated together with the data, software, hardware, and task regime that would make it useful.

[3] **Backpropagation**: Short for "backward propagation of errors," the algorithm uses the chain rule to compute how the loss is locally sensitive to millions of weights in one backward traversal. Its systems cost is memory rather than arithmetic, because the backward pass needs the forward-pass activations, which training must store alongside gradients and optimizer state, the several-times-larger footprint quantified in section 5.4.5.5.

↩ How AlexNet, GPUs, and ImageNet ended the feature-engineering era, and why infrastructure became the next bottleneck, is established in section 1.3.3.

Checkpoint 5.1: Understanding deep learning's emergence

Before turning to the primitives that set these costs, verify your understanding of why deep learning emerged when it did:

- ☐ **Adoption lag**: can you explain why backpropagation existed for more than a decade before it was widely used, naming the systems costs that grow with model size?
- ☐ **The cost of feature learning**: can you explain, using the digit-classification operation counts, why replacing hand-engineered features with learned ones raises the arithmetic and memory demand?
- ☐ **Bottleneck shift**: can you use table 5.2 to explain how the binding resource moves from control flow toward data movement as representation moves from rules to learned features?
- ☐ **Baseline first**: can you name two conditions under which a classical baseline should be tried before a neural network, and the evidence that would justify switching?

If any of these concepts remain unclear, review the relevant sections before continuing.

The digit classifier's cost and the adoption lag lead to the same conclusion. What a network costs the machine is fixed by the primitives it is built from, and those primitives are the subject of the rest of this chapter.

5.3 Neural Network Fundamentals

Each of the 109,184 MACs that classify a single digit belongs to some neuron combining its inputs. Individually those operations are trivial. At network scale, a processor spends more time waiting for operands than computing whenever its memory system cannot supply them fast enough, and an accelerator such as a graphics processing unit (GPU) outruns a CPU on suitable neural workloads because it devotes silicon to wide arithmetic arrays and high-bandwidth memory rather than to branch prediction. The structure of the network primitives determines how data flows through that hardware and how memory demands scale during training and inference.

Those primitives (weighted sums, nonlinear activations, and gradient-based learning) anchor the feed-forward networks of this chapter and recur inside the larger architectures that later chapters introduce, which add operators and execution patterns this simple model does not contain. Parallelism and the memory wall, the pressures traced through the paradigm comparison, follow from that structure. Many simple units operate simultaneously on shared inputs, weights change during training, and layers compose low-level features into high-level ones. That structure begins with the artificial neuron.[4] Understanding it plays the role for a neural network that understanding a single transistor plays for a processor.

[4] **Neuron**: McCulloch and Pitts (1943) introduced a mathematical threshold model of nervous activity: inputs combine and an all-or-none output fires when a threshold is met. That logical neuron is an origin point for the artificial-neuron abstraction and for networks built from many simple units. Learned weights, deep hierarchies, and modern fused multiply-add (FMA) accelerator datapaths are later developments that turn this abstraction into the matrix-heavy workloads studied in this chapter.

5.3.1 One neuron

To test whether a 28×28 image contains a particular stroke, a single computing unit must weigh all 784 pixel intensities, sum the evidence, and decide how strongly to fire. That basic unit of neural computation, the **artificial neuron** (or node), began as a simplified mathematical model of nervous activity (McCulloch and Pitts 1943) before digital implementations turned it into a standard arithmetic primitive. Figure 5.4 sets the biological cell and its mathematical counterpart side by side. The left panel draws a biological cell, with branching dendrites that receive signals, a cell body that holds the nucleus, and an axon that carries the output away. The right panel draws the mathematical model, in which the inputs x_1, x_2, and onward, plus a constant input of one that carries the bias, flow along weighted edges into a node that first forms a linear sum z and then applies an activation function f to emit the output y. The column of labels between the panels pairs each biological structure with its arithmetic counterpart: dendrites with inputs, synapses with weights, the soma (cell body) with the linear sum, and axon firing with the activation.

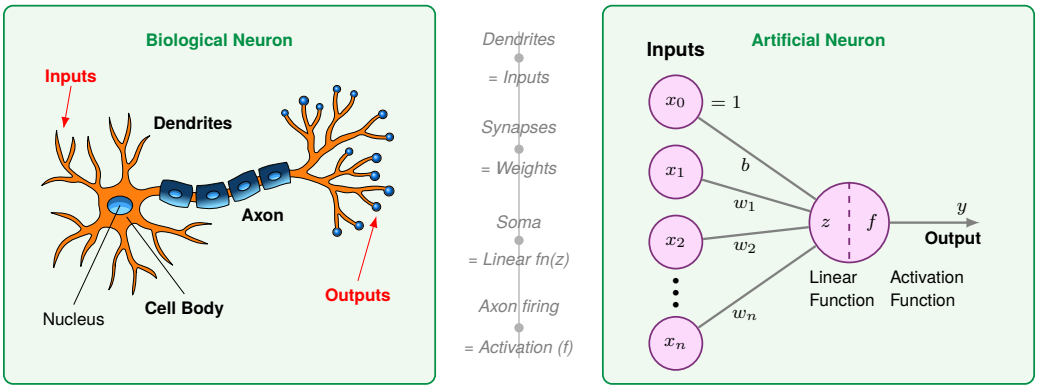


Figure 5.4: Biological-to-Artificial Neuron Mapping: Each biological structure on the left has an arithmetic counterpart on the right. Read the mapping as an abstraction rather than a resemblance claim, since it keeps only what reduces to arithmetic and discards the electrochemistry that produced the behavior.

Read from left to right, the right panel of figure 5.4 carries one signal through four stages, and each stage takes its name from one label pair in the column between the panels:

1. **Input Reception** (Dendrites $\rightarrow x_1, x_2, \ldots, x_n$): The column of circles on the left of the panel is the vector of input features **x** that the artificial neuron receives. In MNIST digit recognition, these represent pixel intensities. The dendrite analogy captures reception of many signals without claiming biological equivalence.

2. **Weighted Modulation** (Synapses $\rightarrow w_1, w_2, \ldots, w_n$): Each input is multiplied by a learnable weight w_i, just as synaptic strengths modulate biological signals. These weights act as "gain" controls, determining how much influence each feature has on the final decision. A bias term b (shown as the top input $x_0 = 1$ in figure 5.4) shifts the activation threshold. Across a network, learned behavior is distributed over many such parameters rather than stored in one weight.
3. **Signal Aggregation** (Cell Body $\rightarrow$ Linear Function z): The left half of the split node, labeled z, integrates the weighted signals, producing a single scalar value $z = \sum(x_i \cdot w_i) + b$. This mirrors how a biological cell body sums incoming electrochemical signals to determine whether the neuron has received enough evidence for a particular pattern.
4. **Nonlinear Activation** (Axon $\rightarrow$ Activation Function f): In the right half of the node, labeled f, the aggregated signal passes through an activation function $f(z)$ and leaves along the output arrow, producing the activation output $a = f(z)$ (often denoted y). This mirrors the axon's all-or-nothing firing decision, since the nonlinearity determines whether the neuron "fires" a signal to the next layer. Unlike the biological case, f can produce graded outputs (ReLU, for example, passes positive values through and zeroes negatives), but the principle of thresholding followed by propagation is the same.

For the machine, one neuron costs d_{in} MAC[5] operations. A direct scalar evaluation loads d_{in} inputs, d_{in} weights, and a bias before writing one output, so without reuse it moves about two values per MAC. Batched and tiled kernels reuse inputs and weights across many neuron evaluations and cut that traffic per MAC, which is why performance at network scale depends on how well the implementation reuses data as much as on how many operations it performs. Once neurons are composed into a network, three constraints appear that a single unit hides. Inside any one layer, neurons can evaluate in parallel, yet moving their parameters and activations across memory often limits throughput before arithmetic capacity runs out; across layers, layer $\ell + 1$ cannot start until layer ℓ finishes, placing a sequential floor under end-to-end latency. Numerical precision cuts across both limits, because narrower formats move fewer bytes and run on denser arithmetic units, provided task quality survives the search for minimum viable precision that chapter 10 develops. The perceptron formalization in section 5.3.2.1 turns the four stages into the equations that this accounting counts.

[5] **MAC (multiply-accumulate)**: Hardware data sheets report fused multiply-add throughput in FLOP/s, so a quoted FLOP/s rate is twice the corresponding MAC/s rate. Layer and batch choices also create activation, communication, and data-movement costs, so MAC count is necessary but not sufficient for a systems budget.

5.3.2 From neurons to layers

Composing neurons into a network turns one neuron's cost into a network's, and each architectural choice, from how neurons connect to how layers are organized, fixes a computational pattern that hardware must then execute. The running network of Example 5.1 is built from exactly these choices, and the subsections below derive its parameter and operation counts one decision at a time, starting with the weighted sum inside each unit.

5.3.2.1 The perceptron's weighted sum

The computational machinery within each dense layer is the **perceptron**,[6] which Rosenblatt introduced as a learning form of the artificial neuron (Rosenblatt 1958). Its signal path (inputs, weights, bias, aggregation, activation) is the one section 5.3.1 traced through its biological origins. What remains is to formalize that path mathematically, because this repeated operation becomes the layer-level kernel that hardware executes. In the MNIST network, a first-layer unit combines all 784 pixel intensities into one activation that may respond to a learned pattern, such as a vertical edge shared by "one" and "seven." Inside one unit (figure 5.5), input features x_i travel along weighted channels w_{ij} and accumulate into a scalar sum with bias b_j, and the resulting linear score z_j passes through an activation f to produce output $\hat{y}$. Different neurons repeat the same structure with different learned weights, allowing one layer to test many patterns against the same input.

Each input x_i has a corresponding weight w_{ij}, and the perceptron multiplies each input by its matching weight. The intermediate output, z_j, is computed as the weighted sum of inputs in equation 5.1:

$$z_j = \sum_i (x_i \cdot w_{ij}) \tag{5.1}$$

[6] **Perceptron**: A machine built to execute a learning algorithm, directly linking hardware and software from the start. A single linear-threshold layer cannot represent nonlinearly separable functions such as XOR, regardless of additional input hardware (Minsky and Papert 1969). Multilayer nonlinear networks expand the representable function class, but they also introduce the training and systems costs developed in this chapter.

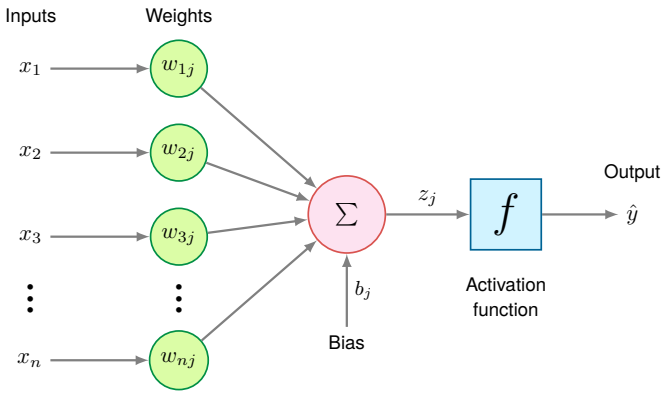


Figure 5.5: Perceptron Architecture: Inputs enter on the left, each scaled by its own weight, and the weighted products plus a bias meet at the summation node before a single activation function produces the output. The multiplies and the accumulation along this path are the per-neuron unit of work that the chapter's layer-cost calculations multiply out.

Here, j identifies the receiving neuron once perceptrons are assembled into a layer, and the sum runs over all input indices i. In plain terms, each input feature is scaled by how important it is (its weight), and the results are summed into a single score. This is the **dot product** of two vectors.

A bias term b_j shifts the linear output up or down, giving the model additional flexibility to fit the data. Thus, the intermediate linear combination computed by the perceptron including the bias becomes equation 5.2:

$$z_j = \sum_i (x_i \cdot w_{ij}) + b_j \tag{5.2}$$

With this weighted sum in hand, a suitable output activation and loss can support regression or classification. A binary linear-threshold classifier selects one of two classes around a threshold; multiclass networks instead produce several scores and commonly normalize them into probabilities with softmax, an output function developed with the other activation functions below.

The formalization turns the neuron's cost into a layer's. A dense layer of d_{out} neurons repeats the weighted sum d_{out} times, so its arithmetic cost is $d_{\text{in}} \times d_{\text{out}}$ MACs, exactly the matrix multiplication $\mathbf{xW}$ that hardware must execute, and the form in which the rest of this chapter counts work. The dot product is the operation that accelerators are built to execute at maximum throughput, which is why neural network performance is quoted in MACs or FLOPs completed per second; how those machines are organized around it is developed in section 11.3.2.1. A weighted sum alone, however, can only draw straight boundaries, and the activation applied to it decides what the layer can represent.

5.3.2.2 Nonlinear activation functions

Without a nonlinearity between layers, a stack of linear weight matrices collapses algebraically into a single linear transformation and can only draw straight decision boundaries, no matter how many intermediate matrices the machine evaluates. **Activation functions** break that collapse by converting each neuron's linear weighted sum into a nonlinear output, making them the point where expressiveness, gradient flow, and hardware cost meet. Applying an activation f to the weighted sum of equation 5.2 yields the generic artificial-neuron computation in equation 5.3:

$$\hat{y}_j = f(z_j) = f\left(\sum_i (x_i \cdot w_{ij}) + b_j\right) \tag{5.3}$$

In figure 5.6, a network without activation functions is constrained to a straight decision boundary (left panel) that misclassifies overlapping point clusters, whereas nonlinear activations allow curved boundaries (right panel) that separate the two classes cleanly. The plot illustrates representational capacity rather than an optimization guarantee; training dynamics and sample coverage still govern whether optimization finds that boundary.

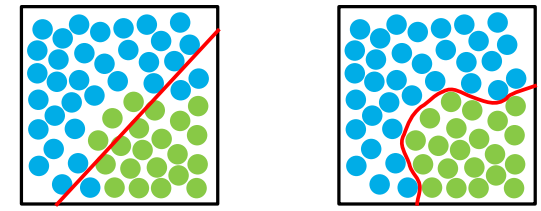


Figure 5.6: Linear vs. Nonlinear Decision Boundaries: Two scatter plots compare the decision boundary a network draws on the same points with and without an activation function.

XOR is the classic function that no single weighted sum can compute, and its four input pairs show why a hidden nonlinear layer changes what a network can represent.

Example 5.2: Building intuition: The XOR problem

Consider a network learning the XOR function, a classic problem that requires nonlinearity. With inputs x_1 and x_2 that can be 0 or 1, XOR outputs 1 when inputs differ and 0 when they are the same.

Setup: Two inputs $\rightarrow$ two hidden neurons $\rightarrow$ one output

Example: With hidden units $h_1 = \text{ReLU}(x_1 + x_2)$ and $h_2 = \text{ReLU}(x_1 + x_2 - 1)$, the output $y = h_1 - 2h_2$ gives 0, 1, 1, 0 for inputs $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$. The input $(1,1)$ drives both hidden units, and the second one cancels the first. No single weighted sum of x_1 and x_2 can produce that pattern.

Systems insight: Hidden layers do not merely add capacity; they change the class of functions the system can represent. XOR, which no single-layer perceptron can compute (Minsky and Papert 1969), is the small example that makes depth a structural requirement rather than a decorative design choice.

The XOR network is the smallest case of a general result. The **universal approximation theorem**[7] establishes that sufficiently wide networks with suitable activation functions can approximate broad classes of functions arbitrarily well. The theorem, however, states a pure existence result under the assumption of unlimited width; it says nothing about the accelerator executing the computation. ML systems impose hard limits on both dimensions of the width-vs.-depth trade-off. A single hidden layer wide enough to approximate a complex function requires parameter storage and activation memory that can exhaust the accelerator's device memory (VRAM) entirely, and a network too deep to fit its activations in memory stalls the backward pass on the same constraint. The engineering problem is therefore not whether a network of sufficient width or depth exists, but whether it fits within the memory and bandwidth envelope of the target hardware.

Comparing activation functions requires one idea from training, which section 5.4 develops fully. Training adjusts each weight by its *gradient*, a number measuring how much the prediction error would change if that weight moved slightly. That signal travels backward through the network, a pass called *backpropagation*, and is multiplied at each layer by the activation's slope, so an activation whose slope is near zero passes almost no learning signal to the layers before it.

The choice of activation function therefore affects both learning and computational efficiency. ReLU[8] ($\max(0, z)$) became a common hidden-layer activation for feed-forward networks because it is inexpensive, has local derivative one for positive inputs, and can produce sparse activations (Nair and Hinton 2010; Glorot et al. 2011). Earlier differentiable deep networks often used sigmoid or tanh. Their smooth S-curves saturate, a failure the sigmoid discussion below develops. That contrast is not a simple ranking: sigmoid remains appropriate for some binary outputs, tanh remains useful when bounded zero-centered activations are desired, and modern architectures use additional activation families. The relevant design question is which function supplies the required output semantics and gradient behavior at an acceptable execution cost. Figure 5.7 charts the response curves and output

7 **Universal approximation theorem**: Under conditions on the activation, target function, compact domain, and approximation tolerance, a sufficiently wide single-hidden-layer network can approximate the target arbitrarily well (Cybenko 1989; Hornik et al. 1989). This existence result is nonconstructive: it does not specify how to find the weights or how many neurons practical training will require. Why depth can nonetheless beat width is the subject of section 5.3.5.

8 **ReLU (rectified linear unit)**: ReLU's cheap comparison-and-select evaluation and its non-saturating positive branch made deep networks markedly faster to train; in the AlexNet paper, a four-layer convolutional network reached 25 percent training error on CIFAR-10 six times sooner with ReLU than with tanh (Krizhevsky et al. 2012).

ranges of three standard element-wise nonlinearities alongside the vector-level softmax function, contrasting their saturation regimes.

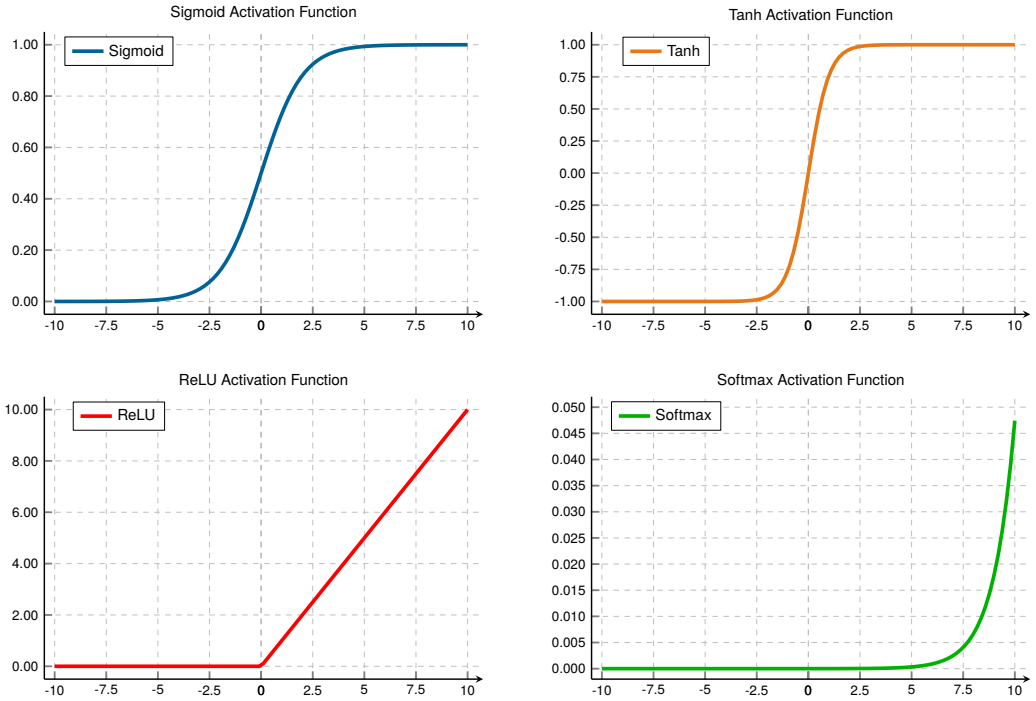


Figure 5.7: Common Activation Functions: Four panels plot nonlinear activation functions with their output ranges. Sigmoid maps inputs to $(0, 1)$ with smooth gradients, tanh provides zero-centered outputs in $(-1, 1)$, and ReLU outputs zero for negative inputs. Softmax (bottom-right) shows one component of a 3-element vector while the other two logits are fixed at 13 and 0. Its magnified y-axis (roughly 0 to 0.05) is not comparable to the other panels, and it shows only one component of a vector whose outputs are interdependent and sum to 1.

Sigmoid The logistic sigmoid function[9] squashes continuous inputs onto the open interval $(0, 1)$ (equation 5.4):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{5.4}$$

The S-shaped curve maps logits, the raw unnormalized scores a layer produces, to $(0, 1)$, so sigmoid can parameterize a binary probability. It approaches one for large positive inputs and zero for large negative inputs. Sigmoid is smooth and differentiable everywhere. Gradient-based learning does not require that smoothness; ReLU, introduced below, has a corner at zero, and implementations simply assign it a slope there.

Sigmoid's main limitation is saturation. For inputs with large absolute values (far from zero), its gradient becomes extremely small, a phenomenon called the **vanishing gradient problem** (Bengio et al. 1994). Even at its peak, sigmoid's derivative is only 0.25, so each sigmoid layer can shrink the activation-derivative part of the gradient by at least a factor of four. During backpropagation these small activation derivatives multiply with weight-dependent factors across layers, the compounding that the chapter's opening footnote evaluated over 20 layers. Repeated saturation can make early-layer gradients and weight updates negligibly small, preventing useful learning along those paths.

Tanh The hyperbolic tangent[10] centers output activations around zero over the interval $(-1, 1)$ (equation 5.5):

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} \tag{5.5}$$

Tanh produces an S-shaped curve similar to sigmoid but centered at zero: negative inputs map to negative outputs and positive inputs to positive outputs, a symmetry that balances gradient flow

[9] **Sigmoid**: From Greek *sigma* + *eidos* ("sigma-shaped"), referring to the S-curve that maps inputs to the bounded (0, 1) range. The mapping requires an exponential approximation, commonly implemented through specialized instructions, lookup tables, or polynomial methods. It is generally more expensive than ReLU's comparison-and-selection operation, although the measured gap depends on precision, vectorization, and hardware. ReLU's adoption reflected both easier gradient flow on its positive branch and cheaper execution.

[10] **Tanh (hyperbolic tangent)**: By centering its output range on zero, `tanh` can reduce the correlated-update behavior associated with all-positive activations. Its computational cost is similar to sigmoid because it also depends on exponential approximations, while its saturation behavior still permits vanishing gradients.

during training and often yields faster convergence than sigmoid. Like sigmoid, tanh is smooth and differentiable everywhere, and it saturates near -1 and 1, so it shares the vanishing gradient problem for inputs with large magnitudes; even so, its zero-centered outputs make it preferable to sigmoid in recurrent neural networks, which reapply the same layer at every step of a sequence and must keep activations balanced across time steps. Both S-curves therefore carry two costs, saturation during training and an exponential evaluation in hardware, whereas ReLU removes saturation for positive inputs and replaces the exponential with a comparison.

ReLU ReLU sets negative values to zero while maintaining linear identity for positive inputs (equation 5.6):

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \tag{5.6}$$

ReLU's characteristic shape, a straight line for positive inputs and zero for negative inputs, provides three advantages. First, its local derivative is one for positive inputs, avoiding activation saturation on that branch; the complete network gradient can still shrink or grow through weights and other operations. Second, ReLU introduces deterministic activation sparsity by setting negative values to zero. Third, ReLU uses a comparison and selection, `output = (input > 0) ? input : 0`, rather than an exponential approximation. These are local properties of the operator, while end-to-end performance depends on how it is fused with neighboring work.

ReLU is not without drawbacks. The **dying ReLU problem** occurs when neurons become stuck in the inactive state, permanently outputting zero and ceasing to learn. If a neuron's weights evolve during training such that the preactivation $z = \mathbf{x}\mathbf{w} + b$ is consistently negative across all training examples, the neuron outputs zero for every input. Since ReLU's gradient is also zero for negative inputs, no gradient flows back through this neuron during backpropagation. The weights cannot update, and the neuron remains dead. This can happen when the learning rate, the step size that scales each weight update, is large enough to push weights into unfavorable regions. Careful initialization (He et al. 2015), moderate learning rates, leaky ReLU variants, and normalization layers[11] help mitigate dying ReLUs by keeping preactivation distributions well scaled.

Softmax Unlike *element-wise* activation functions that operate independently on each value, softmax[12] is a *vector-level* function that considers all values together to produce a probability distribution. In simple classifiers, softmax is commonly used in the output layer rather than as an element-wise hidden activation; it also normalizes learned importance-score vectors in attention architectures. The softmax function is defined in equation 5.7:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{5.7}$$

For a vector of K values (often called logits[13]), softmax transforms arbitrary real-valued scores into K normalized predicted probabilities that sum to 1, although those values are not necessarily calibrated confidence estimates. As plotted in the bottom-right panel of figure 5.7, individual softmax components grow exponentially across logit values while the denominator couples all vector elements so that the class with the largest logit receives the highest probability. Because this mapping from logits to probabilities is differentiable, gradients flow cleanly back through softmax during training, and combining softmax with cross-entropy loss (section 5.4.3) yields simple gradient expressions that guide learning effectively.

Activation choice also carries a silicon cost, a **transistor tax** paid in circuit complexity, area, and switching energy; a ReLU datapath needs only a sign-bit comparator and a multiplexer, roughly 50 transistor equivalents, whereas an exponential approximation for sigmoid or tanh is modeled here as a stylized budget of 2,500 transistor equivalents, 50× the comparator, an order-of-magnitude teaching ratio rather than a constant across manufacturing nodes. Logic area is only half the cost, because an element-wise activation run as its own kernel is limited by memory traffic rather than arithmetic, a gap the forward-pass analysis later quantifies, so compilers fuse it into the preceding matrix multiplication, and ReLU's backward pass needs only one bit per unit recording whether the unit was active. A dead ReLU unit still occupies weight memory and still costs MACs on every inference while contributing nothing to the output, and ReLU's zeros save no work in a dense kernel

11 **Batch normalization**: Normalization layers (Ioffe and Szegedy 2015) stabilize intermediate activations across training batches, permitting higher learning rates and reducing the risk of dying ReLUs. Section 6.4.2 examines the trade-offs among normalization variants, and section 10.4.1 shows how batch normalization folds into adjacent weights for inference.

12 **Softmax**: The name, coined by Bridle (1990), reflects its role as a "soft" or differentiable version of `argmax`, a function that must evaluate an entire vector to find its maximum value. This vector-wise operation is not used as a drop-in replacement for element-wise nonlinearities such as ReLU, but it is central whenever a model must normalize a vector of scores, including classification heads and attention layers. Direct exponentiation can overflow in 32-bit floating point, so stable implementations subtract the largest logit first (section 5.4.3.3).

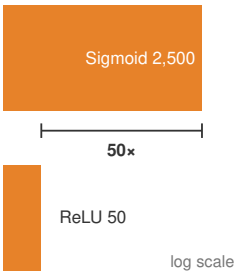
13 **Logits**: In multiclass neural classifiers, logits are unnormalized output scores. Score differences correspond to log probability ratios after softmax, while an individual multiclass logit is not itself a calibrated log-odds quantity. Because `argmax` over logits and `argmax` over softmax probabilities select the same class, inference can skip softmax when only the top class is needed.

unless sparse hardware or routines exploit them. One neuron is now complete, and the next question is how many of them run side by side.

5.3.2.3 Layers and connections

Neurons run side by side in layers, collections of units that execute in parallel across shared inputs, and the power of neural networks comes from that organization rather than from any single unit. An input layer ingests raw feature tensors; one or more hidden layers transform those representations through successive matrix multiplications and nonlinear activations; and an output layer emits task-specific predictions or decision logits.

In the feedforward schematic of figure 5.8, the task fixes only the two ends. The input layer's width matches the four input features and the output layer's width matches the two predictions. The hidden layers between them can widen and narrow freely, which is why hidden width is a design decision rather than a property of the data. Each hidden layer applies a parameterized linear transformation followed by an element-wise nonlinearity, progressively constructing higher-level abstractions from raw input signals.



Sigmoid's exponential datapath dwarfs ReLU in transistor-equivalents (illustrative).

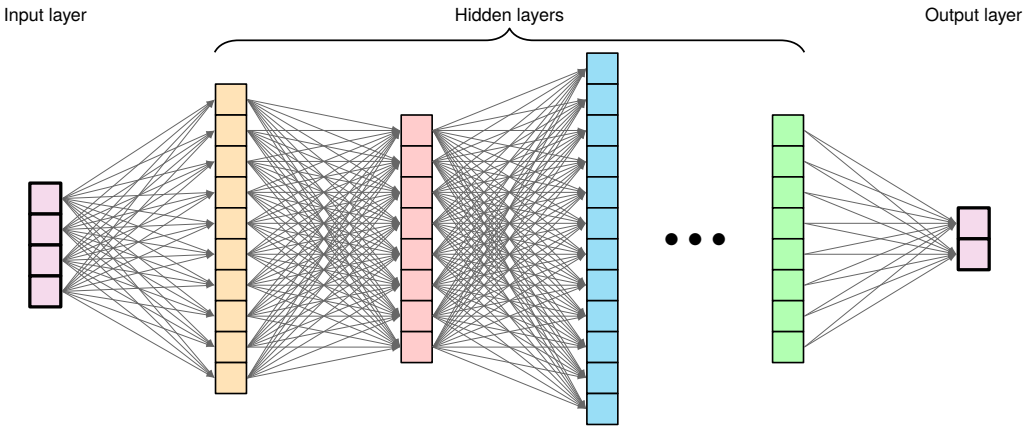


Figure 5.8: Layered Network Architecture: Deep neural networks transform data through successive layers, enabling the extraction of increasingly complex features and patterns. Each layer applies nonlinear transformations to the outputs of the previous layer, ultimately mapping raw inputs to desired outputs.

As data flows through the network, it is transformed at each layer to extract meaningful patterns. Each layer applies the neuron's weighted sum and activation in parallel across all its neurons, and its outputs become the next layer's inputs. Simple features detected in early layers combine into increasingly complex patterns in deeper layers.

On the machine, a layer is one matrix multiplication over all of its neurons at once, and the boundary between layers is where data moves and waits. Each layer writes its output as an activation tensor that the next layer must read, inference can discard that tensor once the next layer has consumed it, and training must keep it until the backward pass. The next layer cannot start until that tensor exists, which is the source of the latency floor that no amount of parallel hardware inside a layer removes. Figure 5.9 traces that translation in three steps. One neuron is a dot product of d_{in} MACs; stacking a layer's neurons and a batch of B examples (inputs processed together in one pass) turns those dot products into one matrix multiply of $B \cdot n_{\ell-1} \cdot n_{\ell}$ MACs (for the first MNIST layer at batch 32, $(32{\times}784) \cdot (784{\times}128)$); and the hardware executes that multiply by holding a tile of weights in an array of multiply-accumulate units while rows of inputs stream past, so each weight fetched from memory serves every example in the batch. What each of those multiplications reads besides the activations is the layer's parameters.

5.3.3 Parameters

Each of those multiplications reads the layer's learnable parameters,[14] its weights and biases, alongside the incoming activations. The parameters decide which transformation a layer applies, and their organization fixes both what the network can learn and how many bytes every pass must move.

[14] **Parameter memory cost**: Parameter count is a misleading proxy for memory importance. The normalization layers that section 6.4.2 introduces, for example, add a few learned scale and shift parameters per layer that barely affect byte budgets but can strongly affect model behavior when training choices change. Conversely, the bulk of parameters in dense weight matrices require extra state during training: gradients plus optimizer bookkeeping beyond the stored weights themselves. A model that fits in memory for inference may therefore require several times more memory for training, a cost quantified later in the training memory budget.

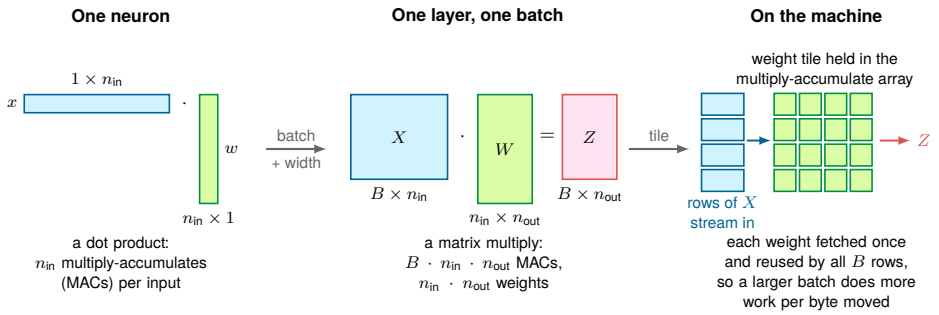


Figure 5.9: From Neuron to Machine: One neuron is a dot product between an input row and a weight column. A layer applied to a batch stacks those dot products into one matrix multiply whose cost is the product of batch size, input width, and output width. On an accelerator, a tile of the weight matrix stays inside an array of multiply-accumulate units while input rows stream through, so the batch, not the neuron, sets how much arithmetic each fetched weight pays for.

5.3.3.1 Weight matrices

In a dense layer with d_{in} input features and d_{out} neurons, the weights form a matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ whose column j holds neuron j's weights, so the layer computes all d_{out} preactivations as the single product in equation 5.8:

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b} \tag{5.8}$$

The canonical case studied here connects each neuron in a layer to every neuron in the previous layer, forming a "dense" or "fully connected" layer. This pattern allows each neuron to use all features from the previous layer. Fully connected layers establish the mathematical principles needed here, while alternative connectivity patterns exploit structure in the data and can reduce work or improve statistical efficiency.

Figure 5.10 details this dense interconnect topology on a concrete two-layer network ($3 \to 4 \to 2$ units). Layer counts in this chapter count layers of weights, not layers of units, so this network has two and the MNIST network ($784 \to 128 \to 64 \to 10$) has three. The figure labels every scalar weight w_{ij} and layer bias b_j alongside intermediate node activations. Reading the incoming edges for any single destination neuron isolates one column of the weight matrix, while evaluating all destinations in parallel forms the complete matrix-vector multiplication. The numerical values illustrate parameter placement rather than an active inference pass; the essential systems property is the matrix dimension. For layers of widths (n_0, n_1, n_2), with n_0 the input width, the weight matrices span $\mathbf{W}^{(1)} \in \mathbb{R}^{n_0 \times n_1}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{n_1 \times n_2}$, requiring $n_0 n_1 + n_1 n_2$ weights before biases.

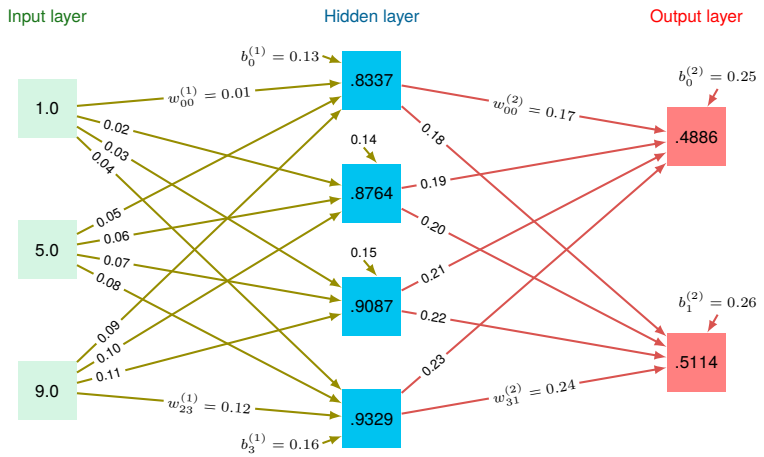


Figure 5.10: Fully Connected Layers: A two-layer network (three layers of units) connects every neuron to all neurons in the preceding layer. Labeled edges and nodes illustrate the organization of weights, biases, and activations; they are not a worked numerical forward pass.

These products account for most of the work in a dense feed-forward network; whether they are compute bound or memory bound depends on their dimensions, batch size, precision, implementation, and target hardware. The layer looks elementary, yet the same dense layer is the feed-forward block inside every transformer layer and the towers of the deep learning recommendation model (DLRM), so the arithmetic counted here is the arithmetic those lighthouses spend. A transformer layer whose feed-forward block widens to four times the model width d holds $8d^2$ feed-forward weights against $4d^2$ in its attention projections (the four square weight matrices that prepare the layer's attention inputs and combine its outputs), so the feed-forward block holds 67 percent of the layer's weights; in the GPT-2 lighthouse, with d = 1,600, that is about 20.5M of each layer's 30.7M weights. Chapter 6 develops both architectures in full.

5.3.3.2 Bias terms

Alongside its column of weights, each neuron in a layer carries one more parameter, a bias term[15]. For a layer with d_{out} neurons, these parameters form a vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ added to the weighted sum in equation 5.8: while weights determine the relative importance of inputs, each entry of $\mathbf{b}$ shifts its neuron's activation threshold independently at the cost of one addition per output value. The arithmetic is negligible, but its placement is not: run as a separate step, the bias addition would read the layer's entire output tensor from memory and write it back for one addition per value, so optimized libraries apply it, together with the activation, while each output tile is still on chip at the end of the matrix multiply.

[15] **Bias terms**: Biases add one parameter per neuron, compared with d_{in} weights for a neuron with d_{in} inputs, so they account for $1/(d_{\text{in}}+1)$ of that layer's parameters. Their model-wide share and effect on accuracy depend on layer widths, architecture, and normalization. Some modern architectures omit biases in layers followed by batch normalization because the normalization transform includes a learned shift.

The organization of weights and biases across a feed-forward network follows a systematic pattern. For each parameterized affine layer ℓ, three components define its computation; the weights and biases are learned, while the activation function is usually chosen by the designer:

- A weight matrix $\mathbf{W}^{(\ell)}$
- A bias vector $\mathbf{b}^{(\ell)}$
- An activation function $f^{(\ell)}$

This yields the complete layer computation in equation 5.9:

$$\mathbf{a}^{(\ell)} = f^{(\ell)}(\mathbf{z}^{(\ell)}) = f^{(\ell)}(\mathbf{a}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \tag{5.9}$$

Here, $\mathbf{a}^{(\ell)}$ (written as $\mathbf{A}^{(\ell)}$ for batches) represents the layer's activation output. The row-vector convention is adopted throughout: each sample is a row, and the weight matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ maps from the previous layer's width to the current layer's width. With equation 5.9, one layer is fully specified, and what remains is choosing the layers themselves: how many, how wide, and how connected.

5.3.4 Choosing a topology

A single layer of neurons cannot even compute XOR (Example 5.2), yet adding or widening hidden layers to gain capacity immediately multiplies the parameters and activations the machine must store and move. Network topology, how neurons organize into layers and connect across them, fixes both what the network can represent and what it costs to run. The chapter's MNIST digit recognizer shows how problem structure fixes the input and output dimensions while leaving the hidden layers as a systems design choice.

5.3.4.1 Feedforward network architecture

Applying the three-layer architecture to MNIST reveals how data characteristics and task requirements constrain network design. The dual-panel layout in figure 5.11 contrasts the conceptual and physical representations of this input mapping: panel (a) sets a 28×28 pixel grayscale image of digit 7 beside the input units and ten class outputs, while panel (b) illustrates the physical flattening of those 784 pixels into a contiguous one-dimensional column vector before feeding the input layer. Fully connected layers assign an independent weight to every coordinate of this 784-element vector, flattening out spatial coordinates and discarding pixel adjacency. Convolutional architectures instead encode spatial locality directly. That choice is an *inductive bias*, a built-in structural assumption about the data (here, that nearby pixels matter together) which restricts the functions a model can learn toward those expected to be useful (Mitchell 1980). The fully connected network makes almost

no such assumption, so it must learn from data the structure that a stronger bias would supply, and that learning requires more parameters.

The two ends of the network are fixed before any architecture search begins: the data format sets the input layer's width, so a 28 by 28 pixel image requires 784 input neurons (one per pixel intensity in the flattened vector), while the task sets the output layer's width, so 10-class digit recognition provisions 10 output neurons (one for each digit 0–9) whose logits a softmax can convert into predicted class probabilities. Between those fixed endpoints, the number of hidden layers and their respective widths remain open engineering choices. Adding layers increases the network's depth so that it can compose more abstract features through successive transformations, whereas widening a layer expands its capacity to represent distinct features at that level of abstraction.

↪ How spatial locality and weight sharing replace the parameter overhead of fully connected layers is developed as a spatial inductive bias in section 6.3.

Widening both hidden layers from 100 to 1000 neurons, comparing a $784 \to 100 \to 100 \to 10$ network with a $784 \to 1000 \to 1000 \to 10$ network, increases parameters from ~89.6K to ~1.8M (~358 KB vs. ~7 MB in FP32). Whether the wider network improves accuracy requires training and evaluation; the storage increase alone already matters for mobile deployment budgets.

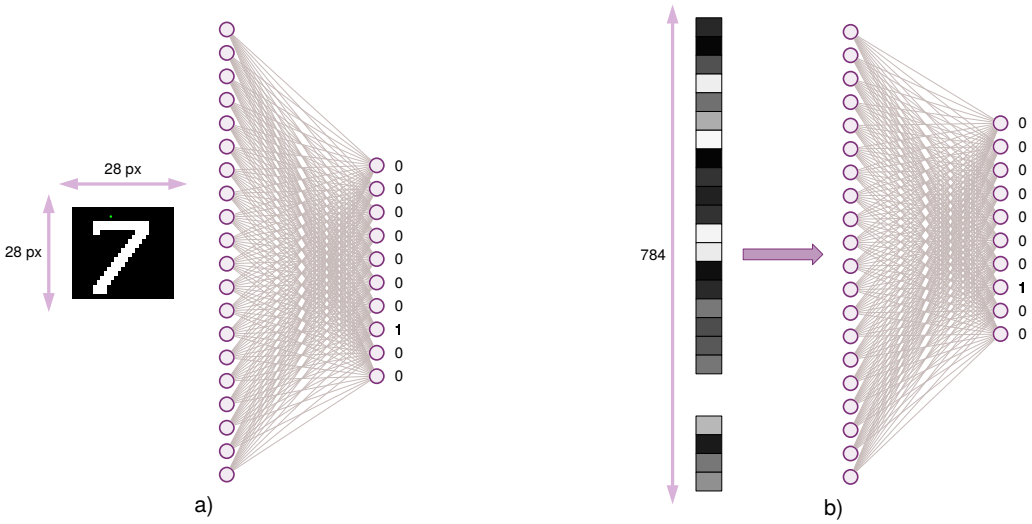


Figure 5.11: MNIST Network Topology: Both panels draw the same network; only the form of the input changes, from a two-dimensional image in panel (a) to the flattened vector the input layer actually receives in panel (b).

5.3.4.2 Layer connectivity design patterns

The fully connected architecture in section 5.3.3 maximizes flexibility by connecting every neuron to every neuron in the next layer, but connectivity itself is an engineering decision. Dense, sparse, and skip patterns trade learning flexibility against parameter count, locality, and gradient flow.

Dense connectivity represents the standard pattern where each neuron connects to every neuron in the subsequent layer. In the MNIST example, connecting the 784-dimensional input layer to a hidden layer of 128 neurons requires 100,352 weight parameters (784×128). This full connectivity enables the layer to learn arbitrary linear combinations of its inputs, which nonlinear activations between layers extend to nonlinear relationships, and it maps directly onto the dense matrix-multiplication kernels that accelerators are built to execute. Because the parameter count grows as the product of adjacent layer widths, however, widening one layer scales parameters linearly while widening both scales them quadratically.

Sparse connectivity cuts that parameter growth by restricting each neuron to a subset of the inputs in the adjacent layer, much as biological neurons connect only to nearby cells; in visual processing tasks like MNIST, neurons might connect only to nearby pixels to reflect the local nature of visual features. Although sparse connections reduce theoretical parameter counts and FLOPs, a sparse weight matrix does not automatically execute faster than a dense one on standard hardware. Unless the hardware supports a regular, structured sparsity pattern that its kernels can predict (developed

in section 11.4.5.3), the arithmetic reduction does not translate into a proportional reduction in execution time.

Whereas sparse connections prune links within a layer, skip connections add direct paths between nonadjacent layers (He et al. 2016a) to keep deeper networks trainable. As networks grow deeper, the longer path from input to output complicates gradient flow; skip connections supplement the standard layer-by-layer progression so that later layers in a digit recognition pipeline can reference both high-level patterns and earlier representations while preserving the backward gradient paths that very deep architectures depend on.

5.3.4.3 Model size and computational complexity

Every connectivity choice ends in a parameter count. How parameters (weights and biases) are arranged determines both learning capacity and computational cost. The parameter count, its numerical precision, and the operations it requires are the concrete terms of the model's side of the silicon contract. Parameter count grows with network width and depth, and table 5.5 lays it out for the running network layer by layer.

Table 5.5: MNIST Parameters and MACs by Layer: Weight, bias, parameter, and per-image multiply-accumulate counts for the 784-128-64-10 network. A dense layer performs one MAC per weight for each input, so the weight and MAC columns match. Layer 1 (Input → Hidden1) dominates both because it multiplies the 784-dimensional input by 128 hidden units; deeper layers shrink rapidly as the representation compresses toward the 10-class output.

Layer	Weights	Biases	Parameters	MACs per Image
Input → Hidden1	$784 \times 128 = 100{,}352$	128	100,480	100,352
Hidden1 → Hidden2	$128 \times 64 = 8{,}192$	64	8,256	8,192
Hidden2 → Output	$64 \times 10 = 640$	10	650	640
Total	109,184	202	**109,386**	**109,184**

The first layer alone holds 100,352 of the 109,184 weights. Because a dense layer performs one multiply-accumulate per weight for each input, the same table also counts the arithmetic per image, and the parameter and MAC columns differ only by the 202 biases. At four bytes per FP32 value, the 109,386 parameters occupy 438 KB. Each must be stored in memory, read on every forward pass, and updated during learning, and the training and inference memory estimates later in this chapter start from this table rather than recounting it.

5.3.5 Why depth matters

A single-layer network attempting to classify handwritten digits must map all 784 raw pixels directly to ten class labels in one step, relearning every stroke pattern separately for each digit. The running network spreads its weights over three layers instead so that deeper layers can exploit **compositionality**, in which complex patterns decompose into simpler patterns that themselves decompose further. In image recognition, learned representations can progress from local edges and textures toward parts and objects; such hierarchical regularities occur in many datasets and motivate layered models, although a trained network need not recover this exact sequence.

Consider recognizing the digit "seven" in the MNIST example. Rather than mapping all 784 pixel values directly to a decision, a deep network can compute the decision more parameter-efficiently by reusing intermediate features:

- **Layer 1** learns simple edge detectors: vertical lines, horizontal lines, diagonal strokes
- **Layer 2** combines edges into shapes: the horizontal top stroke of a "seven," the diagonal downstroke
- **Layer 3** combines shapes into complete digit patterns

Each layer builds on the previous, allowing later computations to reuse earlier features. The same edge detectors learned for "seven" can also contribute to recognizing "one," "four," and other digits. This **parameter reuse** helps explain why depth is an effective design choice without implying a fixed parameter advantage for every task. The choice between adding layers and widening existing ones is not symmetric, however, because depth and width contribute to representational capacity through different mechanisms and cost the machine different things.

Systems Perspective 5.2: The depth vs. width trade-off

For certain function classes, depth provides an exponential representational advantage: a network with N_L layers can represent functions that require exponentially more units in a shallow network (Telgarsky 2016). This restricted theoretical result explains why nonlinear composition can be parameter-efficient; it is not a universal guarantee for every task.

Modern architectures balance depth against width. A network with 10 layers of 100 neurons has the same 1,000 total hidden units as one with 2 layers of 500, but not the same parameter count, function class, or execution graph. Both expose parallelism within each layer; the deeper network adds sequential dependencies and may represent some compositional functions more efficiently.

The cost side of that trade-off follows from counting. For a fully connected network with layer widths running from n_0 (the input) to n_{N_L} (the output), each layer ℓ holds a $n_{\ell-1} \times n_\ell$ weight matrix and n_ℓ biases, and its forward pass performs one MAC per weight for every example. The training pass must also retain each layer's output for every example in a batch of B, so the parameter count P, the forward operations per example O (two FLOPs per MAC), and the retained activation memory $M_{\text{activations}}$ follow equation 5.10:

$$P = \sum_{\ell=1}^{N_L} (n_{\ell-1} + 1)\, n_\ell, \qquad O = 2 \sum_{\ell=1}^{N_L} n_{\ell-1}\, n_\ell, \qquad M_{\text{activations}} \propto B \sum_{\ell=1}^{N_L} n_\ell \tag{5.10}$$

When every hidden layer has the same width d, the sums collapse. Parameters and operations grow as $N_L d^2$, linear in depth and quadratic in width, while retained activations grow as $BN_L d$, linear in both. Figure 5.12 plots the two sweeps on the running network's input and output widths. Holding the network at two hidden layers and doubling their width from 2,048 to 4,096 multiplies MACs per example by 3.4×, short of four only because the input layer's share still grows linearly, while holding the width at 512 and doubling depth from 16 to 32 hidden layers multiplies MACs by 2.0× and retained activations by 1.9×.

Depth and width add capacity on different terms. Depth adds capacity at linear cost in parameters, operations, and activation memory, but every added layer is another step that must finish before the next begins, so depth raises the latency floor of a single example, lengthens the list of activations that training must hold until the backward pass, and lengthens the path a gradient must survive on its way back to the first layer. Width adds capacity at quadratic cost in parameters and operations (activation memory still grows only linearly), yet all of it lands inside one matrix multiplication whose rows and columns can be spread across parallel hardware, so a wider layer keeps an accelerator busy without adding a sequential step. Choosing between them is therefore a choice between paying in serialized steps and memory or paying in arithmetic, and the batch size B, which multiplies only the activation term, decides how heavily the memory side weighs during training.

Checkpoint 5.2: Neural network architecture fundamentals

Before proceeding to training, verify your understanding of the foundational concepts:
Use the MNIST architecture $784 \to 128 \to 64 \to 10$ as the running check.

- ☐ **Neuron equation**: can you write the neuron equation, including weighted sum, bias, and activation function?
- ☐ **ReLU vs. sigmoid**: can you explain why ReLU is computationally cheaper than sigmoid while still providing the nonlinearity a multilayer network needs?
- ☐ **Layers to matrices**: can you map input, hidden, and output layers to the corresponding weight matrices $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$?
- ☐ **Parameter count**: can you calculate the total parameter count, including both weights and biases?

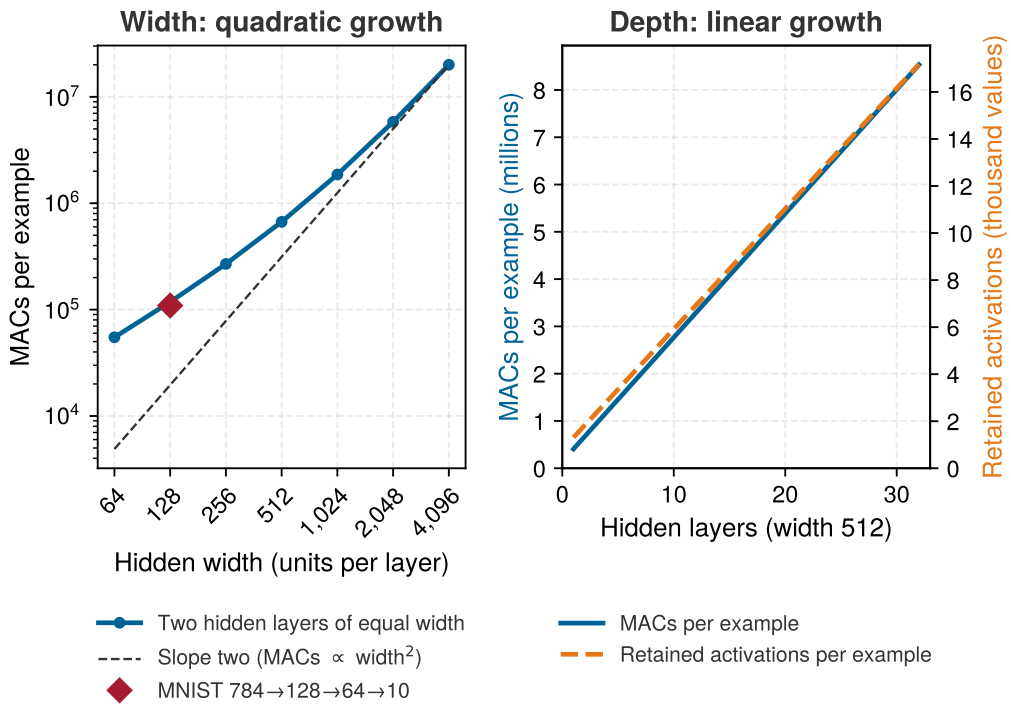


Figure 5.12: Width Costs Quadratically, Depth Costs Linearly: Forward multiply-accumulate operations (MACs) per example for fully connected networks on the MNIST input (784) and output (10) widths. Left: two hidden layers whose shared width sweeps from 64 to 4,096 on log-log axes; once the hidden-to-hidden matrix dominates, the curve follows the dashed slope-two guide, and the running 784-128-64-10 network sits near the narrow end. Right: hidden width fixed at 512 while depth sweeps from 1 to 32 hidden layers; MACs per example (left axis) and activations retained per example for training (right axis) both rise in a straight line.

- ☐ **Width, depth, cost**: can you explain how layer width, depth, and connectivity change memory footprint and arithmetic cost?

If any of these feel unclear, review the earlier sections on neural network fundamentals, artificial neurons, and parameters before continuing. The upcoming sections on training and optimization build directly on these foundations.

Network architecture, neurons, and parameters are now in place, but nothing yet explains how randomly initialized parameters become useful. Architecture answers *what* the model computes; training answers *how* it learns. The mechanics reveal systems constraints because training adds backward computation, saved activations, gradients, and optimizer state. Batch size then becomes a joint statistical and hardware choice, because it changes gradient noise, activation memory, data reuse, and parallel efficiency at once.

5.4 Learning Process

The MNIST network currently holds 109,386 parameters set to random values, numbers that encode no knowledge at all. Learning turns them into a working classifier by repeating one short cycle of computation millions of times, and that repetition multiplies every cost the previous sections counted for a single forward pass.

5.4.1 Supervised learning from labeled examples

At that starting point the network classifies digits no better than guessing among ten classes (about 10 percent accuracy). Transforming it into a 95 percent-accurate classifier requires supervised learning:

showing the network labeled examples and adjusting its weights based on the errors it makes. For the MNIST digit recognition task, the dataset contains 60,000 training images, each a 28×28 pixel grayscale image paired with its correct digit label. To close that accuracy gap, training runs as a loop where each iteration processes a subset of training examples called a batch through four operations: forward computation through the network layers generates predictions, a loss function measures prediction error under the chosen objective, backpropagation computes parameter gradients, and an optimizer uses those gradients to update the weights.

Definition 5.2: Mini-batch

Mini-batch is the subset of B training examples that pass together through one forward and one backward pass and whose gradients are averaged into a single parameter update; during training, an unqualified *batch* in this book means a mini-batch.

1. **Significance**: The batch size B is the training loop's main systems knob. It sets how many examples share each weight read from memory, it multiplies the activations that training must keep until the backward pass, and it fixes how many updates one pass over the training set produces, so doubling B roughly doubles activation memory and halves the updates per pass.
2. **Distinction**: Full-batch gradient descent uses the entire training set for every update, and purely stochastic gradient descent uses a single example. A micro-batch is a slice of a mini-batch whose gradients are accumulated before the one update, and an inference batch groups incoming requests without updating anything.
3. **Common pitfall**: A frequent misconception is that a larger batch is free throughput. Each request waits longer, and beyond some size the optimizer needs more passes over the data to reach the same accuracy.

Tracing dataflow along the red horizontal arrow in figure 5.13 shows forward activations traversing the layers to evaluate loss, whereas the orange arrow running the opposite way carries gradients backward through the layers; the optimizer then turns those gradients into weight updates. This operational asymmetry creates a central memory obligation: training must retain intermediate forward activations in memory until the corresponding backward pass completes.

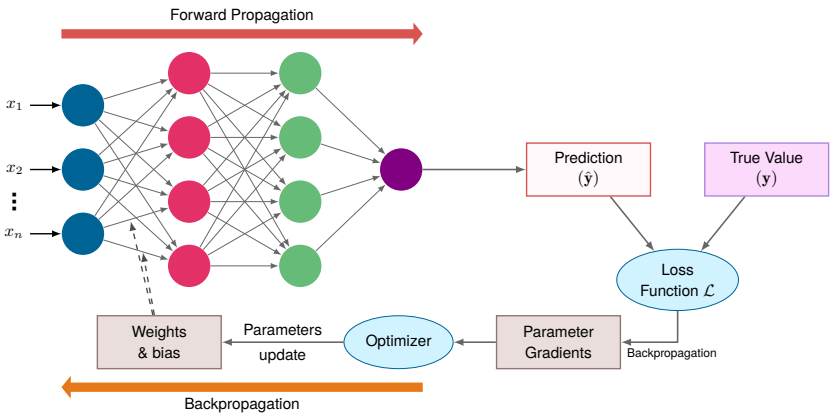


Figure 5.13: Training Loop Architecture: A prediction and true value meet at the loss function; backpropagation computes parameter gradients, and the optimizer turns those gradients into parameter updates for the next forward pass.

The iterative approach can be expressed mathematically. Given an input image $\mathbf{x}$ and its true label $\mathbf{y}$, the network computes its prediction according to equation 5.11:

$$\hat{\mathbf{y}} = f_{\text{net}}(\mathbf{x};\theta) \tag{5.11}$$

This equation encapsulates the entire forward pass: the network f_{net} takes an input $\mathbf{x}$ (say, a 28×28 digit image) and, using its current parameters θ (weights and biases across all layers), produces a prediction $\hat{\mathbf{y}}$ (a vector of ten probabilities, one per digit). The semicolon notation $f_{\text{net}}(\mathbf{x};\theta)$ distinguishes the input $\mathbf{x}$, which changes with every example, from θ, which remains fixed during inference but evolves during training. The network's error is measured by a loss function[16] $\mathcal{L}$ (equation 5.12):

$$\text{loss} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) \tag{5.12}$$

The error measurement drives the adjustment of network parameters: gradient descent (section 5.4.4) moves each parameter against its loss gradient, and backpropagation (section 5.4.5) computes those gradients. In practice, training operates on batches of examples rather than individual inputs. For the MNIST dataset, each training iteration might process 32, 64, or 128 images simultaneously for reasons developed in section 5.4.4.1. The training cycle continues until the network achieves sufficient accuracy or reaches a predetermined number of iterations, and each iteration begins with the forward pass that turns a batch of input images into predictions.

[16] **Loss function**: Formalized by Abraham Wald in statistical decision theory (Wald 1949) as the "cost" of an incorrect decision, $\mathcal{L}$ quantifies the gap between prediction $\hat{\mathbf{y}}$ and ground truth $\mathbf{y}$. The choice of loss function shapes the optimization geometry: it determines the gradient landscape that backpropagation must navigate. A loss with flat regions near incorrect predictions produces weak gradients that stall learning, while a loss with steep gradients near the decision boundary accelerates convergence where it matters most, a systems consequence explored in section 5.4.3.

↪ How long this cycle takes to reach a target accuracy, the time-to-accuracy measure that training benchmarks prefer over raw throughput, is developed in section 12.5.2.1.

5.4.2 Forward pass computation

An MNIST image becomes ten class scores by moving through weighted layers and nonlinear activations. That computation is **forward propagation**: input data flows through the network's layers to generate predictions. During training, those predictions are paired with ground-truth targets at the loss function, closing the training loop of figure 5.13. The forward pass itself underlies both inference and training.

The process begins with the input layer, where each pixel's grayscale value becomes an input feature. For MNIST, this means 784 input values ($28\times28 = 784$), each normalized between 0 and 1. The first layer alone performs 100,352 MACs per sample. When processing multiple samples in a batch, these operations multiply accordingly, requiring careful management of memory bandwidth and computational resources. Accelerators execute these operations efficiently by running many MACs in parallel, a design developed in chapter 11.

5.4.2.1 Individual layer processing

Each layer of the digit classifier turns its input into a more abstract representation in two steps, a linear transformation of inputs followed by a nonlinear activation. The linear transformation applies the same weighted sum operation defined in equation 5.2, but now using notation that tracks layer indices (equation 5.13):

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \tag{5.13}$$

Here, $\mathbf{A}^{(\ell-1)}$ contains the activations from the previous layer (the outputs after applying activation functions), $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1}\times n_\ell}$ is the weight matrix for layer ℓ, and $\mathbf{b}^{(\ell)}$ is the bias vector (broadcast across the batch). The superscript (ℓ) keeps track of which layer each parameter belongs to. This row-vector convention matches the single-sample equation in equation 5.2: each row of $\mathbf{A}$ is one sample, and right-multiplying by $\mathbf{W}$ transforms it to the next layer's width.

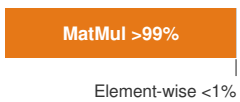
Following this linear transformation, each layer applies a nonlinear activation function f (here, f or $f^{(\ell)}$ denotes a generic activation function at layer ℓ), as expressed in equation 5.14:

$$\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)}) \tag{5.14}$$

This process repeats at each layer, creating a chain of transformations:

$$\text{Input} \to \text{Linear Transform} \to \text{Activation} \to \text{Linear Transform} \to \text{Activation} \to \cdots \to \text{Output}$$

The batch form is what the machine executes. Every one of the B rows of $\mathbf{A}^{(\ell-1)}$ meets the same $\mathbf{W}^{(\ell)}$, so a single read of the weights serves the whole batch, the reuse that figure 5.9 drew. For the digit classifier, the first such product turns the 784-dimensional input into 128 hidden values, and two more produce the ten class scores.



Matrix multiplication is over 99 percent of this network's forward-pass FLOPs.

5.4.2.2 Matrix multiplication formulation

For a network with N_L layers, the complete forward propagation process can be expressed as a composition of layer transformations (equation 5.15):

$$\mathbf{A}^{(N_L)} = f^{(N_L)}\Big(\cdots f^{(2)}\Big(f^{(1)}(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)}\Big)\cdots \mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)}\Big) \tag{5.15}$$

The nested expression unfolds layer by layer, each step consuming the previous layer's activations:

1. First layer:

$$\mathbf{Z}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$
$$\mathbf{A}^{(1)} = f^{(1)}(\mathbf{Z}^{(1)})$$

2. Hidden layers ($\ell = 2, \ldots, N_L - 1$):

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$$
$$\mathbf{A}^{(\ell)} = f^{(\ell)}(\mathbf{Z}^{(\ell)})$$

3. Output layer:

$$\mathbf{Z}^{(N_L)} = \mathbf{A}^{(N_L-1)}\mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)}$$
$$\mathbf{A}^{(N_L)} = f^{(N_L)}(\mathbf{Z}^{(N_L)})$$

For a batch of B images in the MNIST network, four tensor dimensions govern the forward pass:

- Input $\mathbf{X}$: $B \times 784$
- First layer weights $\mathbf{W}^{(1)}$: $784 \times n_1$
- Hidden layer weights $\mathbf{W}^{(\ell)}$: $n_{\ell-1} \times n_\ell$
- Output layer weights $\mathbf{W}^{(N_L)}$: $n_{N_L-1} \times 10$

Of the two kinds of operation in that chain, the matrix multiplications dominate AI computation, and arithmetic intensity explains why.

Definition 5.3: Arithmetic intensity

Arithmetic intensity is the ratio of operations a kernel performs to the bytes it transfers across the memory interface being modeled, $I = O/D_{\text{vol}}$ in FLOP/byte.

1. **Significance**: It decides whether a kernel is limited by arithmetic or by data movement. An accelerator can perform far more operations per second than its off-chip memory can deliver bytes, so a kernel that does little work per byte leaves the arithmetic units waiting however fast they are, and the remedy is more reuse per byte (tiling, batching, fusion) rather than more peak compute.
2. **Distinction**: Unlike total FLOPs (a count of operations), arithmetic intensity is a ratio that characterizes the shape of a workload's hardware demand. Two kernels with identical FLOPs but different memory access patterns can have different arithmetic intensities and run at very different speeds on the same hardware.
3. **Common pitfall**: A frequent misconception is that arithmetic intensity is a fixed property of an operation. It depends on implementation and on batch size. A naive matrix multiplication that reloads operands from main memory for each output element has low arithmetic intensity, a tiled implementation that reuses data from on-chip memory has high arithmetic intensity, and a dense layer reaches more of that reuse as the batch grows, because each weight read then serves every example in the batch.

Tiled dense matrix multiplication reuses each loaded value many times, whereas an unfused element-wise operation does little work per byte it moves. Writing s_{elem} for the bytes needed to store one value (4 bytes in FP32), table 5.6 compares their idealized intensities.

Table 5.6: Idealized Arithmetic Intensity of Matrix Multiply vs. Element-Wise ReLU: A tiled dense $N \times N$ matrix multiplication performs $2N^3$ operations while moving at least about $3N^2 s_{\text{elem}}$ bytes for s_{elem} bytes per value, so potential reuse grows with N. A standalone element-wise activation remains at $1/(2s_{\text{elem}})$ FLOP/byte (0.125 FLOP/byte for FP32); implementations often fuse it with a neighboring kernel to avoid a separate memory round trip.

Operation	Operation Count (O)	Minimum Data Movement	Idealized Intensity	Typical Optimization
Matrix Mul $(N \times N)$	$2N^3$	$3N^2 s_{\text{elem}}$ bytes	$\approx 2N/(3s_{\text{elem}})$	Tiling on CPU/GPU/TPU
Element-wise (ReLU)	N^2	$2N^2 s_{\text{elem}}$ bytes	$1/(2s_{\text{elem}})$	Vectorization and fusion

Systems Perspective 5.3: Why matrix multiplication dominates AI

Modern accelerators have large compute arrays relative to off-chip memory bandwidth. They approach peak arithmetic throughput only when operations expose enough reuse and parallel work. Dense and convolutional layers map well to optimized kernels; custom element-wise sequences are often fused so that their intermediate values remain on chip. How accelerators organize their compute arrays around matrix multiplication is developed in section 11.3.

For a single sample, the layer operation **xW** is a **general matrix-vector multiply (GEMV)**. Batching multiple samples turns this into **XW**, a **general matrix multiply (GEMM)**. Accelerators pack wide parallel units and high-bandwidth memory hierarchies optimized for 2D matrix transformations. Because hardware vendors invest the bulk of their silicon area and compiler optimizations in GEMM throughput, almost every neural network layer is structured to execute as matrix multiplication, and section 6.9.3.1 shows how the convolution and attention layers of later chapters lower to that shared primitive. Matrix operations account for over 99 percent of the floating-point operations in this chapter's dense network and dominate modern neural workloads. Model architectures are therefore shaped around the operations that matrix engines execute efficiently, which is the core of hardware-software co-design. The tiling that gives these kernels their data reuse is developed in section 11.7.5, and how GEMM arithmetic intensity grows with matrix dimension is collected for reference in section C.1.3.

5.4.2.3 Step-by-step computation sequence

Consider a batch of 32 images entering the network. Each image starts as a 28×28 grid of pixel values, which flattens into a 784-dimensional vector. For the entire batch, this yields an input matrix $\mathbf{X}$ of size 32×784, where each row represents one image. The values are typically normalized to lie between 0 and 1.

The transformation at each layer proceeds as a shape-changing sequence; the shapes determine both kernel choice and activation storage. The network first takes the input matrix $\mathbf{X}$ (32×784) and transforms it using the first layer's weights. If the first hidden layer has 128 neurons, $\mathbf{W}^{(1)}$ is a 784×128 matrix, so the computation $\mathbf{XW}^{(1)}$ produces a 32×128 matrix. Each element in this matrix then has its corresponding bias added and passes through an activation function. For example, with a ReLU activation, any negative values become zero while positive values remain unchanged. The final layer transforms its inputs into a 32×10 matrix, where each row contains 10 class scores. Let z_j denote the raw score (logit) for digit j and let z_k range over all 10 digits. Often, these scores are converted to probabilities using the softmax function in equation 5.7:

$$\hat{y}_j = \frac{e^{z_j}}{\sum_{k=1}^{10} e^{z_k}}$$

For each image in the batch, the ten values $\hat{y}_j$ form a probability distribution over the possible digits. The digit with the highest probability represents the network's prediction.

Napkin Math 5.1: Counting operations in forward pass

Problem: Determine the total arithmetic operation count (O) for one forward pass through the MNIST network (784 → 128 → 64 → 10) with batch size 32.

Background: A matrix multiplication of dimensions $(M \times K) \times (K \times N)$ requires $2 \times M \times K \times N$ FLOPs (one multiply and one add per output element, summed over K terms). Bias addition adds $M \times N$ floating-point additions. ReLU activation adds $M \times N$ comparisons, so the table reports total operation-equivalent work rather than pure FLOPs for activation rows.

Solution: Table 5.7 breaks down the operation count layer by layer. The total comes to ~7 MOp, or 7 MOp ÷ 32 = ~219 KOp per image. This per-image figure reconciles with the count at the start of the chapter. The three matrix-multiplication rows contribute 218,368 FLOPs per image, exactly two FLOPs for each of the 109,184 MACs, and the bias and activation rows add less than 1 percent on top.

Systems insight:

1. Layer 1 dominates: The first layer accounts for 91.9 percent of all operations because it processes the largest input (784 dimensions). Shrinking the input or the first hidden width is therefore the largest single lever on this network's arithmetic.
2. Compute vs. memory: At batch 1, 219 KOp must amortize a parameter read for only one image, giving an arithmetic intensity of ~0.5 FLOP/byte under this traffic model. At batch 32, one parameter read, one input read, one final-output write, and a write followed by a read for each hidden tensor move ~588 KB and raise the intensity to ~11.9 FLOP/byte. This idealized FP32 model treats each dense layer as a separate kernel with fused bias and activation, and counts transfers across a memory boundary at which no operands remain resident between kernels or batches. A hidden activation contributes twice because its producer writes it and its consumer reads it. Cache residency or cross-layer fusion can reduce traffic at that boundary; extra transfers can increase it. The comparison shows why batching improves reuse, but memory- or compute-bound behavior still depends on the machine's ridge point, the balance $R_{\text{peak}}/\text{BW}$ introduced with the iron law, which the roofline model in section 11.6 develops.
3. Scaling intuition: Doubling the hidden layer widths (784 → 256 → 128 → 10) increases the operation count by about 2.1× to about 15 MOp. This comes from recomputing each layer: layers 1 and 3 double, layer 2 quadruples, so the total roughly doubles rather than quadrupling.

Table 5.7: MNIST Forward Pass Operation Count by Layer: Operation-equivalent breakdown for a single forward pass through the 784-128-64-10 network at batch size 32. Matrix multiplication rows are FLOPs; bias, ReLU, and softmax rows are elementwise operations included to show their negligible contribution. Layer 1's matrix multiply dominates because it multiplies the 784-dimensional input against the widest weight matrix; bias and activation operations contribute under 1 percent of the total.

Layer	Operation	Dimensions	Operations
Layer 1	MatMul	(32×784) × (784×128)	2 × 32 × 784×128 = 6,422,528
Layer 1	Bias + ReLU	32×128	2×4,096 = 8,192
Layer 2	MatMul	(32×128) × (128×64)	2 × 32 × 128×64 = 524,288
Layer 2	Bias + ReLU	32×64	2×2,048 = 4,096
Layer 3	MatMul	(32×64) × (64×10)	2 × 32 × 64×10 = 40,960
Layer 3	Bias + Softmax	32×10	~640 (simplified)
Total			**~7 MOp**

5.4.2.4 Implementation and optimization considerations

Forward propagation is easy to state mathematically, but its implementation is constrained by activation storage, batch size, memory layout, and hardware fit. During training, each layer's activations must stay in memory until the backward pass consumes them. For the MNIST network at batch size 32, the input holds $32 \times 784 = 25{,}088$ values, the hidden layers hold 4,096 and 2,048, and the output holds 320, a total of 31,552 values per batch that section 5.4.5.5 converts into bytes alongside the other training-state terms. Every one of these counts scales linearly with batch size, so doubling the batch to 64 doubles activation storage.

The same batch size sets the other side of the trade. Notebook 5.1 estimated arithmetic intensity rising from ~0.5 FLOP/byte at batch 1 to ~11.9 FLOP/byte at batch 32, because one read of each weight now serves 32 images. Batching therefore trades memory for reuse. Memory capacity bounds how far that trade can go, while gradient noise and per-request latency bound it from other directions. Memory layout and optimized linear-algebra libraries decide how much of that reuse an implementation actually captures, which is why parallel processors that can be fed from on-chip storage run large dense matrix operations substantially faster than CPUs, and why specialized accelerators add reduced-precision arithmetic and dataflows tailored to these patterns. Energy follows the same logic. Power alone does not determine energy per inference; the relevant quantity is power integrated over execution time for the same workload and quality target, and memory hierarchy and data movement often explain most of the difference between platforms.

A roofline (Williams et al. 2009) plots a machine's attainable throughput against arithmetic intensity, with a sloped segment equal to bandwidth times intensity that meets a flat roof at peak compute; section 11.6.1 develops that throughput bound in full. Figure 5.14 places those two intensities on the rooflines of two machines whose FP32 ridge points happen to sit near the same value. At both batch sizes the running network lies on the sloped, bandwidth-limited side of both machines, so its attainable rate is bandwidth times intensity. Batching therefore gains more than an order of magnitude by climbing that slope, and at these intensities the accelerator outruns the CPU because it moves bytes faster, not because it multiplies faster. The plot assumes that every operand streams from main memory, as the notebook's traffic model does; a model this small can instead stay in a CPU's cache, which raises the bandwidth that actually feeds it.

5.4.3 Loss functions

When the MNIST network sees a handwritten "seven" and places a probability of 0.6 on "three" and only 0.1 on "seven," the forward pass alone has no way to turn that mistake into a weight correction. **Inference** needs only the forward pass because a deployed model merely reports its prediction. Training needs one more quantity: an error metric that measures how far the ten predicted class scores fall from the true label. Loss functions supply that metric, turning the gap between prediction and reality into a single number that optimization can minimize.

5.4.3.1 Error measurement fundamentals

A loss function assigns a scalar cost to a prediction and target under a chosen objective. For the machine it is the cheapest stage of the training step, a few operations for each of ten class scores against the 109,184 MACs per image that produced them, so what the system must get right about the loss is its numerical range rather than its cost. Lower loss indicates a better value of that objective on the evaluated examples, but it is not the same as higher accuracy: confidence and calibration can change while the predicted class remains unchanged. During training, gradients of the loss guide parameter updates.

Mathematically, a loss function $\mathcal{L}$ takes two inputs: the network's predictions $\hat{\mathbf{y}}$ and the true values $\mathbf{y}$. For a single training example in digit classification, the loss measures the discrepancy between prediction and truth. When training with batches of data, the system computes the mean loss across all examples in the batch (equation 5.16):

$$\mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{y}_i) \tag{5.16}$$

where B is the batch size and $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ represents the prediction and truth for the i-th example. Averaging keeps the loss and mean-gradient scale comparable across batch sizes, although the

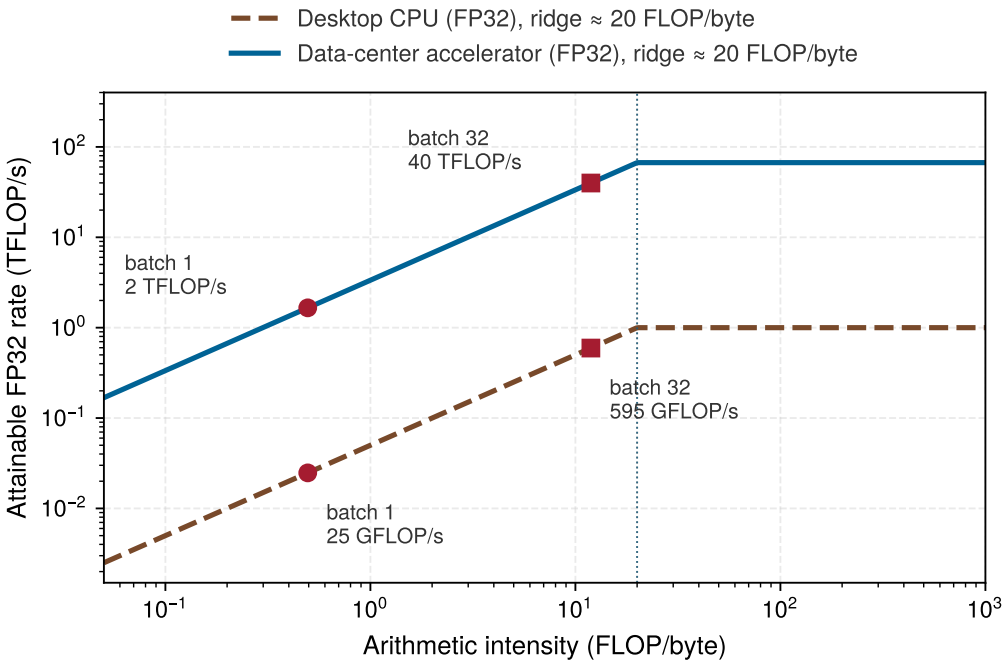


Figure 5.14: The Running Network on the Roofline: Attainable FP32 throughput against arithmetic intensity for a desktop CPU and a data-center accelerator, with the running network's forward pass at batch 1 and batch 32 under the main-memory traffic model of the operation-count notebook. Both points sit left of both ridge points, so batching raises throughput by climbing the bandwidth slope, and at these intensities the accelerator's lead equals its memory-bandwidth advantage rather than its arithmetic advantage.

best learning rate can still change with batch size and optimization regime. The summation also maps naturally to parallel hardware: each example's loss can be computed independently before a reduction combines them.

The choice of loss function depends on the type of task. For digit classification, the loss function must handle probability distributions over multiple classes, provide meaningful gradients that guide learning, penalize wrong predictions in proportion to their severity, and scale efficiently with batch processing. Cross-entropy loss satisfies all four requirements.

5.4.3.2 Cross-entropy and classification loss functions

For classification tasks like MNIST digit recognition, **cross-entropy loss** compares the predicted probability distribution with the true class label. The information-theoretic idea of entropy traces to Shannon (Shannon 1948); in supervised classification, cross-entropy penalizes low probability assigned to the correct class because the logarithm grows sharply as the correct-class probability approaches zero. This shape produces strong gradients for confidently wrong predictions and makes the correct-class probability the objective's central quantity.

For a single digit image, the network outputs a probability distribution over the 10 possible digits. Ground truth is represented as a one-hot vector[17] where all entries are 0 except for a one at the correct digit's position. For instance, if the true digit is "seven," the label is $\mathbf{y} = [0,0,0,0,0,0,0,1,0,0]$.

The cross-entropy loss for this example is defined in equation 5.17:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{j=1}^{10} y_j \log(\hat{y}_j) \tag{5.17}$$

where $\hat{y}_j$ represents the network's predicted probability for digit j. Because $\mathbf{y}$ is one-hot, this simplifies to equation 5.18:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\log(\hat{y}_c) \tag{5.18}$$

[17] **One-hot encoding**: Representing each class as a binary vector with one entry per class, exactly one of which is 1. The encoding is sparse by construction, and for MNIST's 10 classes, 90 percent of each label vector is zeros. Implementations commonly store an integer class index instead of materializing that vector. For very large output spaces, methods such as sampled softmax can also reduce the output computation.

where c is the index of the correct class. The loss therefore depends only on the predicted probability for the correct digit; the network is penalized based on how confident it is in the right answer.

For example, suppose the network assigns the probabilities in the left panel of figure 5.15 to an image of a "seven." The prediction is a distribution over all ten digits, while the label is a single one at the correct class and zero everywhere else.

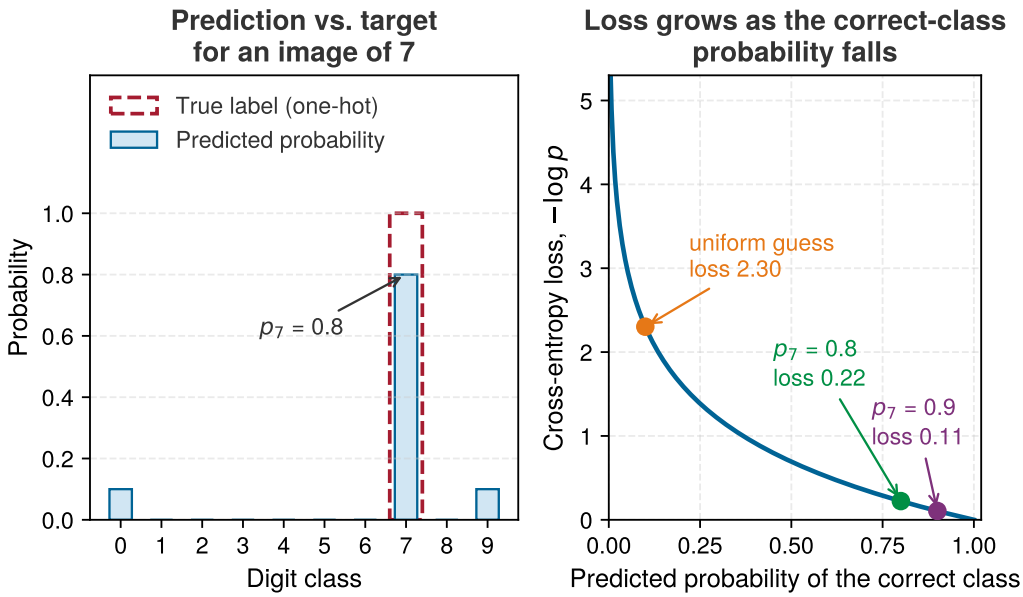


Figure 5.15: Cross-Entropy Compares a Distribution with a One-Hot Label: Left: the network places 0.8 on the correct class and 0.1 each on digits 0 and 9, while the target places all of its mass on the correct class. Right: because the target is one-hot, the loss reduces to the negative log of the correct-class probability. A uniform guess over ten classes costs about 2.3, a confident correct prediction costs little, and the penalty grows without bound as the probability assigned to the truth approaches zero.

The loss is $-\log(0.8)$, approximately 0.223. If the network were more confident and predicted 0.9 for the correct digit, the loss would decrease to approximately 0.105. The right panel of figure 5.15 shows why the logarithm suits training. Raising a correct-class probability that is already high earns little, while a confident mistake, a small probability on the true digit, produces a large loss and therefore a large corrective gradient.

5.4.3.3 Batch loss calculation methods

Evaluating cross-entropy naively across a batch can crash a training run in a single step: if an unshifted softmax rounds a tiny correct-class probability down to zero in finite floating-point arithmetic, $\log(0)$ evaluates to $-\infty$ and subsequent gradient arithmetic produces a not-a-number value (NaN). For a batch of B examples, the cross-entropy objective averages the per-example losses across all examples (equation 5.19):

$$\mathcal{L}_{\text{batch}} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{10} y_{ij}\log(\hat{y}_{ij}) \tag{5.19}$$

Here, i indexes examples in the batch, j indexes the ten output classes, y_{ij} is the one-hot target indicator for class j on example i, and $\hat{y}_{ij}$ is the predicted probability for that class. Averaging over B keeps the loss scale comparable as batch size changes, but each term in the sum must stay within the floating-point format's representable range.

Stable implementations avoid materializing a numerically fragile probability before taking its logarithm:

1. Shift logits before exponentiation, the core of the log-sum-exp trick collected for reference in section C.3.5; equation 5.20 subtracts the largest logit from every exponent:

$$\text{softmax}(z_i) = \frac{\exp\left(z_i - \max(\mathbf{z})\right)}{\sum_j \exp\left(z_j - \max(\mathbf{z})\right)} \tag{5.20}$$

Subtracting the same $\max(\mathbf{z})$ from every logit leaves the softmax probabilities unchanged because the common factor cancels between numerator and denominator. The numerical effect is decisive: the largest shifted logit becomes zero, so the largest exponential is $\exp(0) = 1$ rather than a potentially overflowing value.

2. Prefer fused cross-entropy from logits. For target class c and $z_{\max} = \max(\mathbf{z})$, a stable implementation evaluates

$$\mathcal{L} = -(z_c - z_{\max}) + \log \sum_j \exp(z_j - z_{\max}) \tag{5.21}$$

Equation 5.21 is mathematically equivalent to $-\log \hat{y}_c$ but never rounds a tiny softmax probability to zero first. Clamping a probability or adding ϵ can be a defensive fallback, but it changes the objective and should not replace the stable formulation.

5.4.3.4 Impact on learning dynamics

With a batch size of 32 and 10 output classes, each training step processes 32 sets of 10 scores, computes a loss for each example, and averages them into one batch loss. During training, that value serves two roles: its gradient supplies the update direction, and its trend helps diagnose whether optimization is progressing, stalling, or diverging. For the MNIST classifier, training loss often begins near $\log(10) \approx 2.3$ when class probabilities are close to uniform, falls rapidly during early optimization, and then improves more slowly. Because a low training loss reflects confidence on the training examples rather than proof of generalization, task accuracy, calibration, and validation loss must still be measured separately on held-out data.

The loss function's gradients with respect to the network's output logits provide the initial error signal that drives backpropagation. For one example, when softmax and cross-entropy are combined, each logit gradient has a particularly simple form: the difference between the predicted and target probabilities, $\hat{y}_j - y_j$ for logit z_j. This property makes cross-entropy loss especially suitable for classification, because it provides strong gradients even when predictions are far from the target. The same cancellation is the main reason runtimes fuse softmax and cross-entropy into a single operator, which collapses the backward step to the subtraction $\hat{\mathbf{y}} - \mathbf{y}$, saves a separate kernel and its memory round trip, and avoids an underflow-prone probability tensor.

The choice of loss function also influences other training decisions. Gradient scale interacts with the learning rate, while loss averaging across batches affects gradient noise and thus the useful batch-size regime. Curvature influences optimizer behavior, and convergence decisions require validation metrics and a stopping rule rather than the training-loss trajectory alone.

5.4.4 Gradient descent and parameter updates

The loss gives training a single number to reduce, and gradient descent is the rule for reducing it. Consider one weight w and the batch loss plotted against it, as in figure 5.16. At the current value of w, the curve has a slope $\partial\mathcal{L}/\partial w$, which is the gradient previewed in section 5.3.2.2. A positive slope means that increasing w raises the loss, so the weight should move down; a negative slope means it should move up. Taking a small step against the slope, recomputing the slope, and stepping again walks the weight downhill toward lower loss. A network has one such slope per parameter. Collecting them gives the gradient vector $\nabla_\theta \mathcal{L}$, which points in the direction of steepest loss increase, and each update steps the opposite way.

Definition 5.4: Gradient descent

Gradient descent is the iterative optimization algorithm that updates model parameters in the direction of the negative gradient, $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ with learning rate η, trading computational steps (O) for loss reduction.

1. **Significance**: The update rule itself sets a memory floor. Every parameter needs a gradient of the same size, so in FP32 plain gradient descent holds 8 bytes per parameter (the weight plus its gradient) before counting saved activations. Optimizers that keep extra per-parameter state raise that floor, as the training-memory estimate later in this chapter shows, and section 8.4.3.2 develops the complete accounting.
2. **Distinction**: Backpropagation computes gradients (the "what to change" signal); an optimizer applies updates (the "change it" action).
3. **Common pitfall**: A frequent misconception is that gradient descent finds the global minimum. Neural network loss landscapes are nonconvex with many local minima, saddle points, and plateaus. In practice, gradient descent and its variants converge to regions of low loss that generalize well, but the path depends on the learning rate schedule, batch size, and initialization.

Gradient descent turns each parameter's gradient into a parameter change, using each weight's local loss sensitivity to reduce the loss over successive steps. The fundamental update rule is defined in equation 5.22:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathcal{L} \tag{5.22}$$

where θ represents the collection of network parameters (weights and biases), η is the learning rate, and $\nabla_\theta \mathcal{L}$ is the gradient of the loss with respect to those parameters, which backpropagation (section 5.4.5) computes efficiently.

For the digit classifier, the update aims to reduce the chosen loss. If the network frequently confuses "seven"s with "one"s, the loss gradients from those examples guide the weight changes. The learning rate η controls adjustment magnitude. Values that are too large overshoot good parameters, values that are too small converge slowly, and the right value shifts with batch size, a coupling section 8.4.4.1 develops. Figure 5.16 shows the first failure on a single weight, where every step lands on the far side of the minimum.

Plain gradient descent applies equation 5.22 directly to every weight. Practical optimizers refine it by adapting the step size for each parameter or by carrying momentum across iterations. No optimizer is universally best across all possible problems (Wolpert and Macready 1997), so neural-network training depends on choosing update rules and hyperparameters that match the model, data, and hardware constraints.

Despite neural network loss landscapes being highly nonconvex, gradient-based methods often find useful solutions when the model, data, initialization, and optimization regime are well matched. Why they do remains an active research question; one line of work, for example, studies the *implicit bias* of gradient descent, its tendency to settle on simple solutions among the many that fit the training data equally well (Neyshabur et al. 2017). For practical ML systems engineering, the consequence is empirical. Learning rates, initialization, regularization, and the training configuration must be validated together.

5.4.4.1 Mini-batch gradient updates

Applied to mini-batches (Definition 5.2), gradient descent becomes mini-batch gradient descent, which averages the gradients over the batch and performs one update instead of updating the weights after each individual image. For a batch of size B, with $\mathcal{L}_i$ denoting the loss for example i, the mean loss gradient is equation 5.23:

$$\nabla_\theta \mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}_i \tag{5.23}$$

With a typical batch size of 32, the system performs one forward pass over 32 images, computes 32 per-example losses, averages their gradients, and applies a single weight update. That average

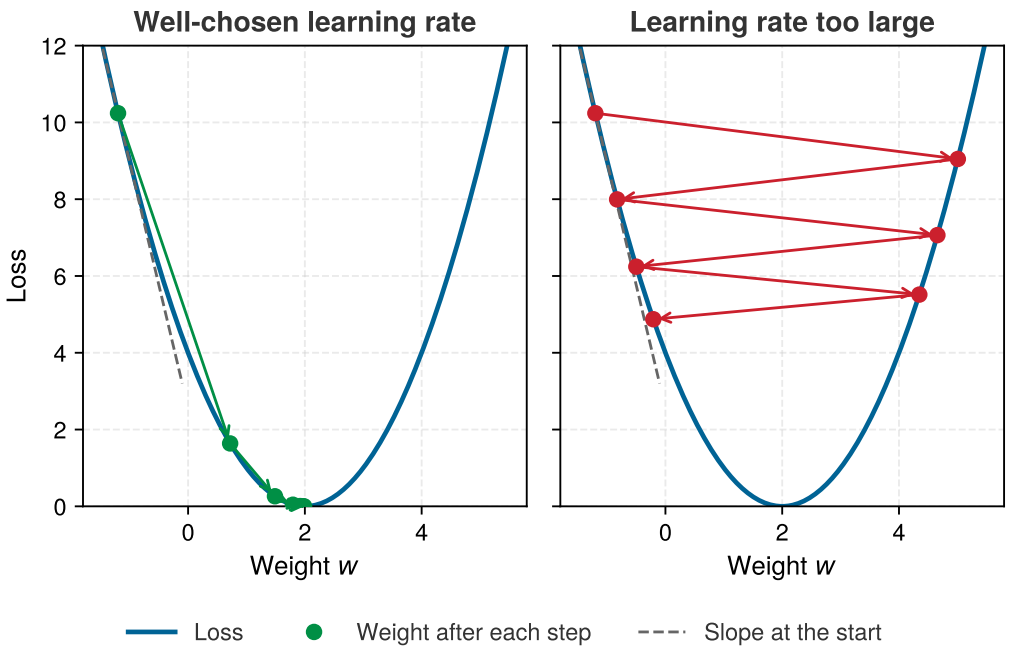


Figure 5.16: Gradient Descent on One Weight: An illustrative loss curve for a single weight, with each step moving the weight against the local slope by an amount proportional to the learning rate. Left: a well-chosen learning rate takes large steps where the slope is steep and smaller ones as the slope flattens near the minimum. Right: a learning rate close to the limit this curve tolerates overshoots the minimum on every step, so the weight oscillates from side to side and approaches the minimum only slowly; a slightly larger rate would diverge.

smooths noisy per-example gradients, but it also means the accelerator must keep the batch's activations available until the backward pass consumes them. Batch size therefore influences available parallelism and utilization together with model shape, kernel implementation, and hardware. When the batch that optimization wants does not fit in accelerator memory, gradient accumulation splits it into micro-batches whose gradients are summed before a single update, a technique section 8.6.5.1 develops with its numerical caveats.

5.4.5 Gradient computation and backpropagation

Gradient descent assumes the gradient is already in hand, but the loss alone does not tell the system how sensitive it is to each parameter. With 109,386 parameters in the MNIST network, estimating every derivative through independent perturbations would be computationally prohibitive. This is the credit assignment problem (Minsky 1961): connecting the final error to the parameters and intermediate values that can change it. Backpropagation solves it by applying the chain rule to compute all of those local sensitivities in one backward traversal.

Definition 5.5: Backpropagation

Backpropagation is the gradient-computation algorithm training systems use to apply the chain rule to a computational graph, using recorded operations and saved or recomputed forward-pass values to compute the gradient of the loss with respect to every parameter in a single backward traversal.

1. **Significance**: For dense layers that require both parameter and input gradients, the backward pass costs on the order of twice the forward-pass FLOPs (a ratio derived from the layer equations later in this section) and requires retaining or recomputing the intermediate values needed by the chain rule. For a model with N_L layers, batch size

B, and layer widths $n_1, \ldots, n_{N_L}$, the stored activations scale, in a simplified count, as $\mathcal{O}\left(B \sum_{\ell=1}^{N_L} n_\ell\right)$. Gradients and optimizer state add further memory, so a model that fits on one accelerator for inference may not fit there for training.

2. **Distinction**: Numerical differentiation requires a perturbed evaluation for each of the model's P parameters. Reverse-mode automatic differentiation computes all P gradients in one backward traversal whose cost is proportional to the computational graph, avoiding P separate forward passes.
3. **Common pitfall**: A frequent misconception is that backpropagation is learning. It is a gradient computation algorithm; gradient descent performs the actual parameter update. Confusing the two hides which one sets activation storage and which sets optimizer state.

A car factory gives a concrete version of the same credit-assignment problem. Vehicles pass through four stations: frame installation (A), engine mounting (B), wheel attachment (C), and final assembly (D). When inspectors find a defective car, they work backward from the defect to trace responsibility through each station: how much D's assembly contributed vs. what it received from C, and how much C's work contributed vs. what came from B. Each station receives adjustment feedback proportional to its contribution, so if Station B's engine mounting was the primary cause, it receives the strongest signal to change.

Backpropagation follows an analogous backward trace, with each layer in the role of a station, but it computes sensitivity rather than causal blame. Starting from the loss, it evaluates how a small change in each layer's inputs would change that loss and propagates those derivatives backward by the chain rule of calculus. The gradient at the output layer plays the part of Station D's adjustment signal, each partial derivative is one station's local sensitivity, and the optimizer later converts the resulting parameter gradients into actual updates.

The sensitivity is local to the current parameter values, input batch, and loss. It answers a counterfactual question, how an infinitesimal change here would change this loss, not whether a weight is permanently important or historically caused the prediction. A framework computes it mechanically by recording each operation in a computational graph and replaying that graph in reverse from the scalar loss, the reverse-mode mechanism that section 7.4.1.2 develops. For a dense network the replay reduces to the layer-by-layer procedure below.

↪ How an eager framework builds that graph on the fly, one operation at a time, is developed in section 7.3.3.

5.4.5.1 Backpropagation algorithm steps

Take the running example in which the network predicts a "three" for an image of a "seven." Backward propagation systematically calculates how the loss would respond to a small change in each weight at the current parameter values. The process begins at the network's output, where the predicted digit probabilities are compared with the true label, and the resulting loss gradient flows backward through the network, giving the local sensitivity of the loss to each layer's values and weights. That calculation follows the chain rule[18] of calculus, breaking down the relationship between weights and loss into local layer steps.

Evaluating these chain-rule products by machine predates modern networks. Wengert presented automatic numerical derivative evaluation without requiring analytical derivative expressions (Wengert 1964), an ancestor of the automatic differentiation that frameworks now perform. The chain rule for composed layers is collected for reference in section C.3.1, which also shows why **reverse-mode automatic differentiation** computes all parameter gradients in a single backward pass. The implementation contract in algorithm 5.1 follows from that order, because the forward pass must save the values that the backward pass will need.

[18] **Chain rule**: The calculus identity $\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a^{(N_L)}} \cdot \frac{\partial a^{(N_L)}}{\partial a^{(N_L-1)}} \cdots \frac{\partial a^{(1)}}{\partial w}$ becomes a product of N_L terms for an N_L-layer network. If each partial derivative is slightly less than one, the product vanishes exponentially; if slightly greater, it explodes. This multiplicative structure makes depth a systems constraint rather than an arbitrary design choice: it dictates the numerical precision requirements and initialization strategies (Glorot and Bengio 2010; He et al. 2015) needed to keep training stable.

Algorithm 5.1 Backpropagation training step

Require: mini-batch inputs $\mathbf{X}$, labels $\mathbf{y}$; N_L layers with parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{N_L}$; loss $\mathcal{L}$
Ensure: parameter gradients $\{\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}}\mathcal{L}\}$ for the optimizer
$\mathbf{A}^{(0)} \leftarrow \mathbf{X}$
for $\ell = 1$ to N_L **do** ▷ *forward pass*
    compute $\mathbf{Z}^{(\ell)}, \mathbf{A}^{(\ell)}$; save the values its derivative needs
end for
compute loss $\mathcal{L}(\mathbf{A}^{(N_L)}, \mathbf{y})$; seed the output gradient $\nabla_{\mathbf{A}^{(N_L)}}\mathcal{L}$
for $\ell = N_L$ down to 1 **do** ▷ *same local rule repeats*
    use the saved forward values to compute $\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}}\mathcal{L}$
    propagate $\nabla_{\mathbf{A}^{(\ell-1)}}\mathcal{L}$ only if that input gradient is required
end for
return the accumulated parameter gradients

Saving activations in the forward loop of algorithm 5.1 is where the systems cost enters: each activation stays live until the backward pass reaches its layer, so **activation memory** scales with batch size, layer count, and activation width. This is why training can exceed inference memory even when the parameter count is unchanged.

5.4.5.2 Error signal propagation

The saved activations are consumed by an error signal that travels the network in reverse, starting from the loss at the output layer and computing how the loss is locally sensitive to each layer and, ultimately, each weight.

A scalar case shows the rule before the matrix form. If $v = 3u$ and $u = x^2$, a small change in x changes u by $2x$ per unit, and each unit change in u changes v by 3, so $dv/dx = 3 \cdot 2x$. Backpropagation applies this multiply-the-local-slopes rule layer by layer. With many units per layer, each local slope becomes a matrix of slopes, called a *Jacobian*, but the rule is unchanged. For a network with N_L layers, the gradient flow can then be expressed mathematically. At each layer ℓ, the sensitivity of the final loss to that layer's output is governed by the chain rule and the schematic Jacobian notation in equation 5.24:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell+1)}} \frac{\partial \mathbf{A}^{(\ell+1)}}{\partial \mathbf{A}^{(\ell)}} \tag{5.24}$$

This computation cascades backward through the network, with each layer's gradients depending on those from the layer above it. Notebook 5.2 traces these equations with numbers on a two-unit network and can be read first if the matrix form is unfamiliar. The reverse order also fixes when memory peaks. The forward pass finishes holding every layer's output, the backward pass releases them from the last layer to the first as it consumes them, and gradients accumulate as it goes. A training step's memory therefore peaks at the turnaround between the two passes when saved activations dominate, or late in the backward pass when parameter state and accumulated gradients do, and section 5.4.5.5 shows both cases for the running network.

5.4.5.3 Initialization and numerical range

Setting all initial weights to zero causes every neuron in a layer to compute the exact same output and receive the exact same gradient, locking the layer into symmetric updates that prevent diverse feature learning, while starting with random weights that are systematically too large or too small compounds across layers into exploding or vanishing signals, the same multiplicative failure that saturating activations cause (section 9). Parameter initialization breaks symmetry and controls signal scale from the first step: weights are drawn randomly from variance-scaled distributions such as Xavier/Glorot initialization[19] (Glorot and Bengio 2010) or He initialization (He et al. 2015), while biases start at small constant values or zeros.

Initialization sets the starting scale, but the backward cascade still multiplies it layer after layer. As gradients flow backward through many layers, they can vanish or explode. Vanishing also has a numeric form, because each floating-point format has a smallest magnitude it can store at full precision, 1.18×10^{-38} in FP32 and 6.10×10^{-5} in FP16, the 2-byte half-precision format that halves

[19] **Xavier/Glorot initialization**: For a layer with d_{in} inputs and d_{out} outputs, a common Glorot choice uses weight variance $2/(d_{\text{in}} + d_{\text{out}})$ to balance forward activations and backward gradients (Glorot and Bengio 2010). The fix costs zero additional FLOPs; it is purely a matter of setting the random distribution at startup.

[20] **FP32 and reduced-precision formats**: The IEEE 754 binary32 format uses 32 bits (1 sign bit, 8 exponent bits, 23 fraction bits), whereas 16-bit alternatives split exponent and fraction differently: IEEE 754 FP16 allocates 5 exponent and 10 fraction bits, providing high precision but a narrow dynamic range (6.1×10^{-5} to 65,504), so FP16 training multiplies the loss by a scale factor before the backward pass to keep small gradients representable (dynamic loss scaling). Brain Floating Point (BF16) allocates 8 exponent bits matching FP32's normal exponent range to reduce underflow risk relative to FP16; sufficiently small values can still underflow. The loss-scaling recipe belongs to the mixed-precision training of section 8.6.3; the formats are collected for reference in section D.4.

memory and traffic relative to 4-byte FP32.[20] Below it, values enter the *subnormal* range, where the format gives up precision bits to represent ever-smaller magnitudes. Some accelerator modes skip that range for speed and flush such values straight to zero, and values smaller still underflow to zero on any hardware. A gradient that reaches zero this way stops updating its weight entirely. At the other extreme, values beyond the representable maximum can overflow to `Inf` and propagate `NaN`, destabilizing training.

Arithmetic without branches is not arithmetic without risk, and the danger of an unhandled overflow is not unique to machine learning. Safety-critical systems have failed catastrophically when a numerical invariant was violated.

War Story 5.1: The overflow that became guidance (1996)

Context: The European Space Agency's Ariane 5 Flight 501 reused inertial reference software from Ariane 4, including numeric conversions whose assumptions matched the older vehicle's flight profile (European Space Agency 1996).

Mechanism: Ariane 5's flight profile caused a 64-bit floating-point value to exceed the range of a signed 16-bit integer during conversion, triggering an unhandled operand error.

Impact: About 37 seconds after liftoff, both inertial reference units had shut down, and the on-board computer read the active unit's diagnostic output as flight data. The launcher veered off its flight path, broke up, and self-destructed.

Response: The inquiry recommended limiting software to functions required during flight, reviewing range assumptions, and designing exception handling so a single conversion failure could not disable redundant units (European Space Agency 1996).

Systems lesson: Numerical ranges, exception handling, and reuse assumptions are part of the systems contract. A computation can be syntactically correct and still be invalid for the physical regime in which it runs. ML systems hit this whenever a numerical format's representable range fails to match the magnitudes flowing through it. A low-precision multiplication that overflows produces an Inf or NaN that propagates silently through the rest of the computation, making numerical regimes as important to debugging as control flow.

Range and precision decide whether a gradient survives the backward pass at all. What computing it costs is set by the derivative calculation at each layer.

5.4.5.4 Derivative calculation process

Computing gradients involves calculating several partial derivatives at each layer: how changes in weights, biases, and activations affect the final loss. These computations follow directly from the chain rule of calculus but must be implemented efficiently for practical training. For a hidden layer ℓ with an element-wise activation, backpropagation first transforms the incoming signal and then computes weight, bias, and input gradients:

First, the incoming error signal $\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}}$ is transformed into the preactivation gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}}$ by multiplying element-wise ($\odot$) with the local activation function derivative (equation 5.25):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}} \odot f'(\mathbf{Z}^{(\ell)}) \tag{5.25}$$

For ReLU activations, $f'(\mathbf{Z}^{(\ell)})$ acts as a binary mask, passing gradients through active neurons ($z > 0$) and zeroing out inactive ones. The classifier's softmax output couples coordinates, so it does not use this element-wise rule. Instead, its backward pass begins with the combined softmax/cross-entropy logit gradient stated in section 5.4.3.4, predicted probability minus target for each class. For mean batch loss, each example's predicted-minus-target signal is divided by B at that starting point. The weight, bias, and input products below then propagate the already-normalized signal, applying the batch mean only once.

Weight gradients measure the local sensitivity of the final loss to each weight. With a batch size of B, layer input dimension $n_{\ell-1}$, and layer output dimension n_ℓ, the incoming activation matrix

has shape $\mathbf{A}^{(\ell-1)} \in \mathbb{R}^{B\times n_{\ell-1}}$ and the sensitivity matrix has shape $\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \in \mathbb{R}^{B\times n_\ell}$. The optimizer uses these gradients to determine parameter updates (equation 5.26):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \mathbf{A}^{(\ell-1)^T} \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \tag{5.26}$$

The matrix product $(n_{\ell-1} \times B) \times (B \times n_\ell) \to (n_{\ell-1} \times n_\ell)$ contracts over the batch dimension B, summing the outer products of activations and sensitivities across all examples in the mini-batch. With the mean-loss normalization already present in the sensitivity matrix, this product needs no further factor of $1/B$.

Bias gradients measure how changing each bias term affects the loss. Since biases shift the activation threshold of neurons, these gradients indicate whether neurons should become more or less easily activated, as expressed in equation 5.27:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \mathbf{1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \tag{5.27}$$

Here, $\mathbf{1} \in \mathbb{R}^B$ is an all-ones vector, so left multiplication sums the per-example bias gradients across the batch. Because that contraction reduces the batch dimension without touching $n_{\ell-1}$, the bias gradient requires only $B \cdot n_\ell$ additions rather than a matrix multiply.

Input gradients propagate the error signal backward to the previous layer. Rather than directly updating parameters, these gradients serve as the "adjustment signals" that allow earlier layers to learn from the final prediction error (equation 5.28):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \mathbf{W}^{(\ell)^T} \tag{5.28}$$

Here, the product $(B \times n_\ell) \times (n_\ell \times n_{\ell-1}) \to (B \times n_{\ell-1})$ contracts over the output dimension n_ℓ, preserving the per-example activation shape for preceding layers. Together, these equations reveal the physical origin of the classic systems rule that the backward pass requires roughly twice the arithmetic of the forward pass. The forward pass executes one matrix multiplication per dense layer, $\mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)}$. The backward pass executes two of the same size, one for the weight gradient (equation 5.26) and one for the input gradient that carries the signal to the preceding layer (equation 5.28). A full training step therefore costs about three times the forward FLOPs, one forward product and two backward products per layer. The weight-gradient product trains that layer's parameters; the input-gradient product supplies the signal needed to train preceding layers. For ordinary parameter training on fixed MNIST images, no preceding trainable layer consumes the gradient with respect to raw pixels, so the first layer's input-gradient product can be omitted.

For the running network, the dense matrix work is therefore three forward-pass counts minus the first layer's forward count, or about 2.08× the forward matrix work for this network, before activation derivatives, bias reductions, loss, and optimizer work. Three times the forward matrix work remains a useful approximation when input gradients are needed throughout, or when the omitted first-layer product is a small fraction of total work. The running network makes the exception visible because its first layer accounts for most of its arithmetic.

The running confusion makes these equations concrete. Suppose that, for an image of a seven, the network assigns probability 0.6 to "three" and only 0.1 to "seven," so the cross-entropy loss, the negative log of the correct-class probability, is about 2.30. The combined softmax and cross-entropy gradient at each logit is the predicted probability minus the target, so the "three" logit receives $+0.6$, a push downward, the "seven" logit receives -0.9, a push upward, and the other eight logits receive their small predicted probabilities. That ten-value error signal first meets the output layer. Equation 5.26 multiplies it by the saved 64-value activation of the second hidden layer to produce all 640 output-weight gradients, and equation 5.28 multiplies it by the transposed output weights to send a 64-value signal back to that hidden layer. The ReLU mask then zeroes the entries of units that were inactive for this image, and the same two products repeat, yielding 8,192 weight gradients and a 128-value signal for the first hidden layer, and finally 100,352 first-layer weight gradients computed from the saved 784 pixel values. No gradient is needed for the pixels themselves. Every step consumed a value saved during the forward pass, which is why training must keep each activation until this trace reaches its layer.

Systems Perspective 5.4: The memory cost of backprop

Activation retention is one component of a broader training-state budget. Under a simplified single-device accounting that excludes temporary workspaces and framework overhead, equation 5.29 captures the four principal byte counts:

$$\text{Training Memory} \approx \text{Model Weights} + \text{Gradients} + \text{Optimizer States} + \text{Activations} \tag{5.29}$$

The derivative equations also record which stored values each step depends on:

- The weight gradient in equation 5.26 needs the incoming activation $\mathbf{A}^{(\ell-1)}$.
- The activation derivative may need $\mathbf{Z}^{(\ell)}$ or a compact mask that records which nonlinear units were active.
- The input gradient in equation 5.28 needs the layer weights $\mathbf{W}^{(\ell)}$.
- The optimizer consumes the completed parameter gradients and may add momentum, variance, or other state after backpropagation has produced them.

A training system can retain these values for the fastest backward pass, recompute selected values to save memory, or offload and partition state across memory tiers or devices. None of these choices eliminates cost: recomputation spends additional FLOPs, while offload and partitioning introduce data movement and synchronization. For deep networks with large batches, wide activations, or high-resolution inputs, this trade-off can dominate accelerator capacity. Section C.3.3 provides the complete training-memory equation and a worked analysis of weights, gradients, optimizer state, and activation costs.

Two inputs, two ReLU hidden units, and one output are enough to trace every backward product with actual numbers.

Napkin Math 5.2: Tracing gradients: A worked backpropagation example

Setup: Consider a network with two inputs, a hidden layer of two neurons (ReLU activation), and one output neuron (no activation, for simplicity). Suppose the current weights and biases are:

- **Hidden layer**: $\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$, $\mathbf{b}^{(1)} = [0,\ 0]$
- **Output layer**: $\mathbf{W}^{(2)} = \begin{bmatrix} 0.6 \\ -0.4 \end{bmatrix}$, $b^{(2)} = 0$

Given input $\mathbf{x} = [1.0,\ 0.5]$ and target $y = 1.0$. Because this toy network produces a single number rather than class probabilities, it uses the simpler squared-error loss, half the squared difference between prediction and target: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$.

Forward pass (to establish the values backpropagation needs):

- Hidden preactivation:

$$\mathbf{z}^{(1)} = \mathbf{x}\mathbf{W}^{(1)} + \mathbf{b}^{(1)} = [1.0 \cdot 0.5 + 0.5 \cdot 0.8,\ 1.0 \cdot (-0.3) + 0.5 \cdot 0.2] = [0.9,\ -0.2]$$

- Hidden activation (ReLU): $\mathbf{a}^{(1)} = [\max(0, 0.9),\ \max(0, -0.2)] = [0.9,\ 0.0]$
- Output: $\hat{y} = \mathbf{a}^{(1)}\mathbf{W}^{(2)} + b^{(2)} = 0.9 \cdot 0.6 + 0.0 \cdot (-0.4) = 0.54$
- Loss: $\mathcal{L} = \frac{1}{2}(0.54 - 1.0)^2 = 0.1058$

Backward pass (applying the chain rule layer by layer):

Step 1: Output layer gradient. The loss gradient with respect to the output is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 0.54 - 1.0 = -0.46.$$

Step 2: Output weight gradients (applying equation 5.26). Since the output layer has no activation function

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} = -0.46, \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \mathbf{a}^{(1)^T} \frac{\partial \mathcal{L}}{\partial z^{(2)}} = \begin{bmatrix} 0.9 \\ 0.0 \end{bmatrix} \cdot (-0.46) = \begin{bmatrix} -0.414 \\ 0.0 \end{bmatrix}$$

Step 3: Propagate to hidden layer (applying equation 5.28). The error signal sent backward is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \mathbf{W}^{(2)^T} = (-0.46) \cdot [0.6, \, -0.4] = [-0.276, \, 0.184]$$

Step 4: Pass through ReLU. The ReLU derivative is one where $z > 0$ and 0 otherwise, so

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = [-0.276 \cdot 1, \, 0.184 \cdot 0] = [-0.276, \, 0.0].$$

The second neuron's gradient is zeroed because ReLU blocked its forward signal.

Step 5: Hidden weight gradients (applying equation 5.26):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{x}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \cdot [-0.276, \, 0.0] = \begin{bmatrix} -0.276 & 0.0 \\ -0.138 & 0.0 \end{bmatrix}$$

Weight updates (with learning rate $\eta = 0.1$): Each weight moves opposite to its gradient. For example, $W_{11}^{(1)}$ updates from 0.5 to $0.5 - 0.1 \cdot (-0.276) = 0.5276$, nudging the network toward the correct output. The second hidden neuron's weights receive zero updates for this example because ReLU blocked its activation, illustrating the mechanism that can lead to dead neurons if it happens persistently across the training data.

Systems insight: Backpropagation is a data-dependency graph as well as a calculus procedure. Training systems must preserve the forward activations needed by this backward pass, which is why activation memory becomes a first-order systems cost.

In practice an autograd engine performs this bookkeeping, recording each operation during the forward pass and keeping the values its backward rule needs (section 7.3). What it keeps is the activation memory that the worked example below quantifies.

5.4.5.5 Worked example: Training vs. inference memory

The four terms of equation 5.29 can now be evaluated for the running network. A single forward pass through the $784 \to 128 \to 64 \to 10$ network costs 109,184 MACs, but the memory a training step holds is set by stored state rather than by that arithmetic. At batch size 32 in 32-bit (4-byte) floating-point precision, each of the four terms can be counted in turn.

The first contribution is the model parameters, already tallied layer by layer in table 5.5. The 109,386 parameters at 4 bytes each occupy 437.5 KB.

Activations are the next contribution. Table 5.8 records each layer's activation tensor and its memory cost at batch size 32.

Table 5.8: MNIST Activation Memory at Batch Size 32: Per-layer activation tensor shapes, value counts, and 32-bit float memory cost during a single forward pass. Input activations dominate the activation table at 100.4 KB; for this small MNIST multilayer perceptron (MLP), the total activation footprint remains below the parameter footprint, while larger batches and deeper networks make activations a central training-memory concern.

Layer	Activation Shape	Values	Memory
Input	32×784	25,088	100.4 KB
Hidden1	32×128	4,096	16.4 KB
Hidden2	32×64	2,048	8.2 KB
Output	32×10	320	1.3 KB

Layer	Activation Shape	Values	Memory
Total		31,552	**126 KB**

Training adds two memory contributions that inference never pays. Gradients occupy the same space as the parameters they update, here 437.5 KB, and the optimizer adds its own state. The Adaptive Moment Estimation (Adam) optimizer,[21] a widely used refinement of gradient descent, keeps two running averages for every parameter, one of its recent gradients (the *first moment*) and one of its recent squared gradients (the *second moment*), and uses them to adapt each parameter's step size. Those two buffers occupy twice the parameter size, another 875.1 KB. Table 5.9 summarizes the per-component memory footprint for training vs. inference.

Table 5.9: MNIST Training vs. Inference Memory at Batch Size 32: A simplified FP32 memory estimate for the MNIST MLP. Training retains saved activations and adds gradients plus two Adam buffers; inference uses the peak adjacent-layer buffer. Framework workspace and memory-saving techniques can change the totals.

Component	Training	Inference
Parameters	437.5 KB	437.5 KB
Activations	126 KB	117 KB
Gradients	437.5 KB	—
Optimizer state	875.1 KB	—
Total	**~1.9 MB**	**~554 KB**

Figure 5.17 follows the same memory through one training step. At batch 32 the parameter state of weights, gradients, and Adam buffers sets the peak, and the saved activations are a thin layer on top. Multiplying the batch by 32 leaves the parameter state untouched and multiplies the activation layer by 32, until saved activations are most of the memory and the peak moves to the moment the forward pass ends. The same network and the same arithmetic per example can therefore be parameter-bound or activation-bound in memory, and the batch size decides which.

Scale drives the systems lesson. At batch size 32, training requires 3.4× the inference memory in this simplified estimate; the inference column follows from the buffer-lifetime rule of section 5.5.2. The absolute gap grows with parameter and activation storage; the ratio depends on their balance and on implementation choices such as recomputing some activations instead of storing them, numerical precision, and workspace reuse.

5.4.5.6 Scaling beyond MNIST

The running network is the smallest case in this chapter. Figure 5.18 plots the parameter counts of 725 notable models published since 1950. Until about 2010 the count doubled roughly every 43 months; after 2010, as deep learning took hold, it doubled about every 10 months, and the largest models now hold more than a trillion parameters. Every dense parameter carries the same three obligations as the running network's weights, so the per-parameter bookkeeping counted for a small digit classifier is the arithmetic that sets the memory and bandwidth budgets of the models in later chapters. The parameter climb parallels the growth in training compute that figure 1.6 traced.

The MNIST network's memory is small, but each of its terms grows with network width and batch size, as equation 5.10 showed, and every training step must also move that state, reading parameters, producing gradients, and retaining or recomputing the forward-pass values each layer's backward calculation needs. For modest networks like the MNIST example, this traffic remains manageable. As models grow, memory bandwidth and activation storage can become binding constraints, motivating high-bandwidth memory, narrower number formats for some tensors (mixed precision), partitioning state across devices, and activation recomputation, which discards selected saved activations and recomputes them during the backward pass. Section 8.6.3 develops mixed precision, and section 8.6.5 develops recomputation.

The memory requirements summarized in table 5.9 seem modest for a digit classifier. At production scale the same four terms move a model into a different tier of the memory hierarchy.

[21] **Adam (adaptive moment estimation)**: Maintains per-parameter first- and second-moment estimates of the gradient, requiring 2× additional FP32 memory beyond the parameters themselves (Kingma and Ba 2015). For a 100K-parameter MNIST model this overhead is negligible, but for a 7-billion parameter model it adds ~56 GB, often the difference between fitting on one GPU or requiring partitioning. Adam is widely used because its adaptive updates work robustly across many tasks, but learning rate, schedule, regularization, and memory strategy still require validation.

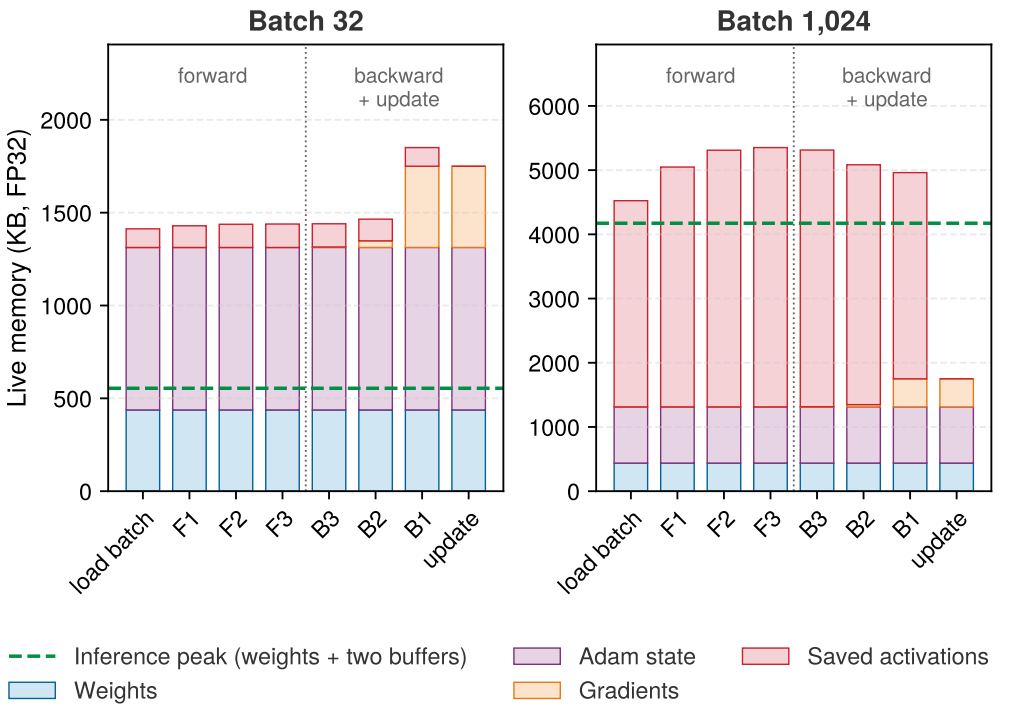


Figure 5.17: Memory Over One Training Step: Live FP32 memory of the running 784-128-64-10 network at each event of a training step, with weights and the two Adam buffers resident throughout. The forward pass (F1 to F3) keeps every layer's output, the backward pass (B3 to B1) releases them as it consumes them while gradients accumulate, and the update leaves only parameter state. The left panel runs batch 32 and the right panel batch 1,024. The dashed line is inference: weights plus the largest pair of adjacent activation buffers.



Lighthouse 5.1: MNIST-to-ResNet-to-GPT-2 memory scaling

The MNIST running example and the GPT-2 lighthouse show how model scale changes the data (D_{vol}) term of the iron law of ML systems (principle 3).

- MNIST (running example): 109,386 parameters at 4 bytes each occupy approximately 438 KB. This entire model fits inside the L2 cache of a modern processor.
- ResNet-50 (lighthouse): 25.6M parameters occupy approximately 102.4 MB in FP32, far beyond any cache but comfortably inside one accelerator's device memory.
- GPT-2 (lighthouse): 1,500,000,000 parameters at 4 bytes each occupy approximately 6 GB before activations and runtime workspace. This exceeds cache capacity and places the model in a device-memory or multi-device regime unless precision is reduced.

Systems insight: Moving from ~109.4K to 1.5B parameters is a 14,000× jump. The increase represents a phase change in engineering, not merely "more parameters." MNIST can keep its weights near the processor; GPT-2 serving must manage a much larger memory working set. Whether a particular layer is compute or memory bound still depends on batch size, precision, reuse, and hardware balance.

Figure 5.19 sets the three footprints against the memory that would hold them. Each step to the right is a step down the memory hierarchy, from a microcontroller's on-chip SRAM through an accelerator's L2 cache to its high-bandwidth memory (HBM), toward storage that is larger, farther from the arithmetic, and slower to read.

↪ Why weights, gradients, and two Adam moments come to 16 bytes per parameter, in plain FP32 and in the mixed-precision recipe alike, is developed in section 8.3.2.3.

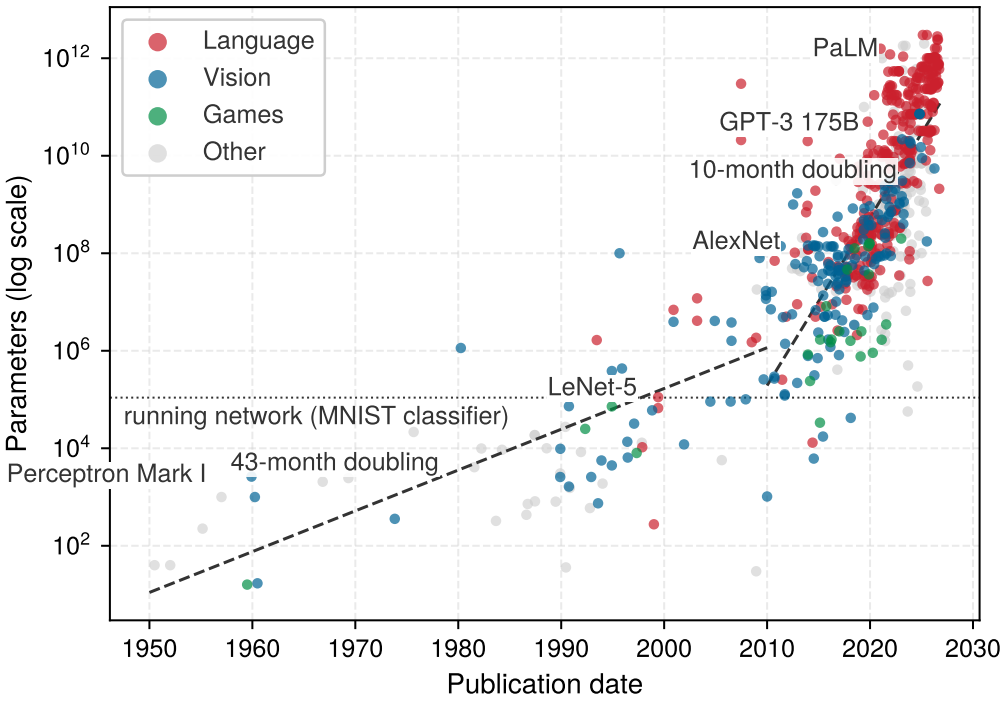


Figure 5.18: Parameter Counts of Notable Models, 1950 to the Present: Each point is one notable model, colored by domain, plotted by publication date on a logarithmic axis. Dashed lines are least-squares fits before and after 2010; the steeper second segment coincides with the rise of deep learning. Data from Epoch AI (2026c).

Engineers also need the same reasoning in seconds rather than minutes. Multiplying parameters by bytes per value and by the number of training-state copies rejects an impossible configuration before any profiling, and section 8.4.3.2 turns those napkin rules into a complete training-memory budget.

Checkpoint 5.3: Backpropagation

The credit assignment problem asks how the loss depends on parameters throughout the network. Backpropagation computes those gradients via the chain rule; verify that the mechanism is clear:

Mechanism

- ☐ **Chain rule**: can you explain how equation 5.24 decomposes a complex error signal into local gradients for each layer?
- ☐ **Gradient decay**: why do gradients often vanish in deep networks?

Training vs. inference

- ☐ **Computational cost**: under what input-gradient assumption does dense training require roughly three times the forward matrix work, and which product can ordinary training on fixed raw inputs omit?
- ☐ **Memory cost**: why must training retain or recompute the forward values needed by the backward pass, and how does this scale with batch size and network depth?
- ☐ **Layer cost**: for a dense layer mapping 4,096 inputs to 4,096 outputs at batch size 32, estimate its forward FLOPs and its FP16 output-activation bytes to one significant figure, then say what doubling the batch does to each of them and to the layer's weight bytes.

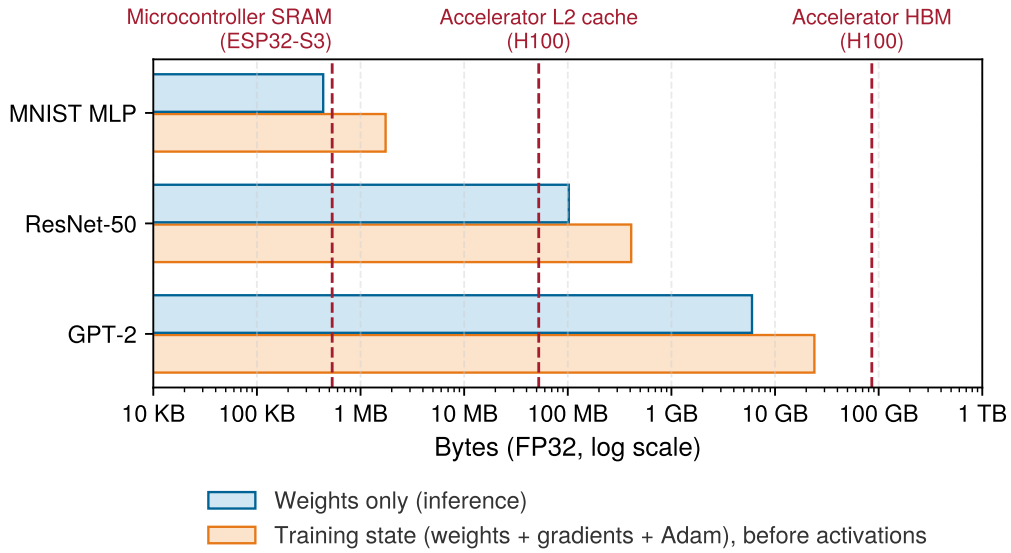


Figure 5.19: Where the Bytes Live: FP32 footprints of the MNIST network and the ResNet-50 and GPT-2 lighthouses on a log scale, for weights alone (inference) and for weights plus gradients plus two Adam buffers (training, before activations). Dashed lines mark the capacity of a microcontroller's on-chip SRAM, an accelerator's L2 cache, and its high-bandwidth device memory. MNIST's training state crosses a tier boundary that its weights do not; ResNet-50 and GPT-2 sit in device memory either way, and what changes is how much of it they claim.

5.4.6 Iterative learning process

The cost of one training step is now known in operations and in bytes. Training repeats that step, forward propagation, backward propagation, and a weight update, in a loop that runs until the network achieves satisfactory performance or reaches a predetermined number of iterations.

A single pass through the entire training dataset is called an epoch.[22] For MNIST, with 60,000 training images and a batch size of 32, each epoch consists of 1,875 batch iterations. Algorithm 5.2 lays out the loop of mini-batch stochastic gradient descent (SGD) (Robbins and Monro 1951), so called because each update follows a gradient estimated from a random sample of the data rather than the full dataset. Each epoch shuffles the data, steps through it one mini-batch at a time with a forward and backward pass, and applies a single parameter update per batch.

[22] **Epoch**: The number of passes is a direct multiplier on total work, so a 100-epoch MNIST run executes 100× more forward and backward passes than a single epoch. Large language-model training is often specified by a token budget rather than a simple epoch count because data sources may be mixed and sampled at different rates. The same accounting still applies: every additional training token adds another forward and backward computation.

Algorithm 5.2 Mini-batch stochastic gradient descent

Require: training set $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{D}$; rate η; batch size B; epochs N_{epochs}; initial θ_0
Ensure: trained parameters θ

$\theta \leftarrow \theta_0$
for $e = 1$ to N_{epochs} **do**
  shuffle the training set ▷ *reduce order-dependent patterns*
  for each mini-batch $\mathcal{M}$ of size B **do**
    $\hat{\mathbf{y}}_i \leftarrow f(\mathbf{x}_i; \theta)$ for all $(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{M}$ ▷ *one MAC per weight per example; every activation saved*
    $\mathcal{L}_{\mathcal{M}} \leftarrow \frac{1}{B} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{M}} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ ▷ *a few operations per class; negligible*
    $g \leftarrow \nabla_\theta \mathcal{L}_{\mathcal{M}}$ ▷ *backward: up to twice the forward MACs; reads the saved activations*
    $\theta \leftarrow \theta - \eta\, g$ ▷ *reads and writes every parameter; almost pure data movement*
  end for
  evaluate on validation; stop if the convergence criterion is met
end for
return θ

The mini-batch loop makes the batch size B a hardware unit of work: a larger B can improve matrix utilization and accelerator occupancy but raises activation memory, and because the update fires once per batch rather than once per example, B also sets the gradient-update cadence. The inner loop in algorithm 5.2 is the unit that the hardware repeatedly executes, and its comments carry the chapter's cost accounting line by line. The forward line spends the MACs of table 5.5 for every example in the batch and leaves activations behind, the backward line spends up to twice that and consumes those activations, and the update line does little arithmetic but touches every parameter, so its cost is set by bytes moved rather than operations performed. (In the algorithm, D counts logical training examples, the Samples/Task factor of the iron law, rather than the physical bytes moved D_{vol} in its memory term.) Notebook 5.3 evaluates one full epoch of this loop in seconds and joules on two machines, charging every operation the fixed dispatch cost whose mechanism section 7.3.4 develops.

Napkin Math 5.3: One epoch on two machines

Problem: How long does one training epoch of the running network take on a desktop CPU and on a data-center accelerator, which term of the iron law sets that time, and where does the energy go?

Background: One epoch at batch 32 is 1,875 steps of about 14.5 MFLOP each, the forward work of table 5.5 scaled by the training-step ratio derived earlier. At batch 32 the network's arithmetic intensity sits below both machines' ridge points (figure 5.14), so each step runs at the bandwidth-limited rate. Every operation a step launches also pays a fixed cost: about 10 µs for the framework to dispatch it and, on the accelerator, about 5 µs more to launch it, across roughly 12 operations per step.

Solution: The arithmetic and data movement of a whole epoch take about 46 ms on the CPU and 0.7 ms on the accelerator, a 67× advantage. The fixed per-operation costs add about 225 ms on the CPU and 337.5 ms on the accelerator, so the totals come to about 271 ms and 338 ms. On the accelerator, 99.8 percent of the epoch is overhead, and the CPU finishes first. Energy tells a parallel story. With the energy constants introduced alongside the iron law (Horowitz 2014), about 160 pJ per byte moved from dynamic random-access memory (DRAM) and 2.3 pJ per FP32 operation (the average of a multiply and an add), moving a step's bytes costs 5.8× the energy of its arithmetic at batch 32, and 141× at batch 1.

Systems insight: At this scale the third term of the iron law, fixed overhead, decides the run, and the accelerator's bandwidth advantage disappears behind it; a network this small trains at least as fast on a CPU. Larger models, larger batches, and fewer, fused operations shift the balance back toward the work terms, which is how frameworks gain their speed (chapter 7), while the energy result anticipates why reuse, batching, and narrower formats matter as much to power as to time (section 1.7).

5.4.6.1 Overfitting, regularization, and convergence

A network that reaches 99.5 percent accuracy on its training batches while stalling at 85 percent on held-out examples has memorized quirks of its training sample instead of learning patterns that transfer to unseen inputs. Training loss tracks the optimization objective on recent batches, whereas validation accuracy measures task performance on a separate held-out validation set (with the test set reserved for final evaluation after model and hyperparameter choices are fixed); a widening gap between the two is the primary symptom of overfitting.

Definition 5.6: Overfitting

Overfitting is a generalization failure in which performance on the training sample improves while performance on relevant unseen data does not.

1. **Significance**: The diagnostic signature is measurable: training loss continues to fall while validation loss rises or the relevant validation metric plateaus, creating a widening

generalization gap. Data coverage, architecture, optimization, regularization, and the relationship between train and deployment distributions all matter. Parameter count alone does not decide whether a model overfits. Past the point where a model can fit its training set exactly, test error can fall again as capacity grows, a pattern called double descent (Belkin et al. 2019; Nakkiran et al. 2019).

2. **Distinction**: Unlike underfitting (where the model lacks the capacity to capture the target function and both training and validation error remain high), overfitting produces low training error but high validation error.
3. **Common pitfall**: More data helps only when it is representative of the behavior the system must generalize to. Additional examples, augmentation, regularization, and early stopping are possible interventions, but each must be evaluated on held-out data from the relevant operating regime.

Regularization controls overfitting by constraining what the network can fit. The most common form adds a penalty on weight magnitude to the loss. An ℓ_2 term $\lambda_{\text{reg}} \sum w^2$, where λ_{reg} sets the penalty strength, shrinks every weight slightly at each step, which is why it is also called **weight decay**. An ℓ_1 term $\lambda_{\text{reg}} \sum |w|$ pushes weights toward zero, producing the unstructured sparsity that chapter 10 exploits. Neither penalty needs persistent state of its own. **Dropout** (Srivastava et al. 2014) zeroes a random fraction of activations at every training step so that no unit can rely on a single upstream partner; it costs a fresh random mask per activation during training and disappears at inference. Early stopping, halting training once validation loss stops improving, is the third common control, and it depends on the periodic validation and checkpointing that close this section.

Convergence monitoring provides essential feedback during training and continues into production deployment. A flattening training-loss curve signals slowing optimization, but it does not identify the cause: the run may be approaching a useful solution, encountering an optimization limit, or underfitting. A validation plateau is similarly diagnostic rather than conclusive. The gap between training and validation performance helps distinguish overfitting from improvements that transfer to held-out examples. For the machine, the quantity that matters is time to a target accuracy, which is the number of steps the optimizer needs times the time each step takes. Batch size pulls the two factors in opposite directions. A larger batch means fewer steps per epoch (1,875 for MNIST at batch 32) and better weight reuse per step, but each step costs more time and activation memory, and past some size the optimizer needs more epochs to reach the same accuracy. The useful batch is the one that minimizes that product within the memory available for training.

Selecting a stopping point also requires periodically evaluating the model on the validation set and saving its weights to durable storage, a practice called checkpointing (not to be confused with activation recomputation, which some frameworks also call checkpointing) that section 8.4.5 develops. For the MNIST network a checkpoint write is negligible; for a model whose parameters occupy tens to hundreds of gigabytes, every checkpoint and every validation pass is a recurring storage and compute cost whose frequency must be balanced against recovery and model-selection needs.

The complete training pipeline runs from forward propagation through loss computation to gradient-based weight updates. Training, however, is preparation, not the end goal. Once parameters are optimized, deploying the model for real-time predictions changes the resource profile entirely.

Checkpoint 5.4: Neural network learning process

The complete training cycle provides the mathematical machinery that enables neural networks to learn from data. Before moving to inference and deployment, verify core understanding of the training cycle:

Use the MNIST network ($784 \to 128 \to 64 \to 10$) and a batch size of 32 as the running check.

- ☐ **Forward pass**: can you trace the forward pass using $\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$ and $\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)})$?

- ☐ **Cross-entropy loss**: can you explain what cross-entropy loss measures and why batch loss is averaged across examples?
- ☐ **Generalization**: using training and validation loss, can you tell overfitting apart from slow optimization, and name one control that limits overfitting?
- ☐ **Update rule**: can you write the gradient descent update rule and explain how batch size trades memory, throughput, and gradient stability?
- ☐ **Cost of an epoch**: can you say which term of the iron law sets the time of one training epoch of this network on an accelerator, and why larger batches or fused operations change the answer?

If any concepts feel unclear, review the earlier sections on forward passes, loss functions, backpropagation, and learning before continuing. These mechanisms form the foundation for the training-vs.-inference distinction that follows.

5.5 Inference Pipeline

Training transforms randomly initialized weights into parameters that encode useful patterns, but deployment executes those parameters under a different resource profile. During inference,[23] the same mathematical operators face a different operating regime: online latency deadlines or offline throughput targets replace the training objective, gradients and optimizer state disappear from memory, and target execution environments range from milliwatt edge processors to multi-accelerator cloud servers. This transition reshapes how memory bandwidth, compute capacity, and pipeline stages govern system performance. Because ordinary inference never updates the weights, a deployed model adapts to a changing world only through monitored data collection, validation, and retraining, a loop that operations practice owns.

The training memory estimate of section 5.4.5.5 counted four kinds of state. Inference keeps exactly one of them, the weights, and replaces the saved activations with a pair of short-lived buffers. Persistent weights plus rolling activations are the whole of inference memory for a feed-forward network. Batching then trades latency for reuse, and once the network itself is fast, the stages around it can dominate.

5.5.1 Operational phase differences

The two phases differ first in what they execute and how long their state lives. Inference executes only the forward pass, evaluating inputs against static parameters with batch sizes dictated by incoming traffic. Training couples that forward pass with backpropagation for gradient computation and parameter updates, selecting batch sizes to maximize accelerator occupancy and gradient stability. Unlike inference, training must retain intermediate activations, gradients, and optimizer state simultaneously, multiplying working memory demands. The underlying network weights are identical, but the execution graph changes. Training-only regularizers such as dropout are stripped away, while adjacent matrix operations and activations can be fused or quantized into specialized execution kernels. In figure 5.20, training couples large-batch processing with a bidirectional loop in which output errors trigger a reverse gradient pass, whereas inference executes a strictly unidirectional forward pass over smaller, variable batch sizes without backward state retention.

These differences determine which hardware each phase runs on and how it is deployed. Training clusters rely on high-bandwidth, high-memory accelerators backed by hundreds of watts per socket and dedicated data-center cooling.[24] Inference deployments on constrained hardware prioritize bounded tail latency and energy efficiency across diverse platforms: mobile devices execute on low-power neural processing units (typically 2–4 W), edge servers employ dedicated inference accelerators,[25] and cloud instances use reduced numerical precision to saturate dedicated matrix-multiply units (NVIDIA's versions are called Tensor Cores). At scale, production serving introduces operational requirements that do not arise in isolated training runs, including dynamic request batching, load balancing, and failure isolation. At the memory level, training retains all forward activations until backward passes consume them, whereas inference discards or overwrites intermediate buffers as execution advances through the graph; table 5.10 summarizes this resource profile.

[23] **Inference**: From Latin *inferre* ("to bring in, to conclude"), borrowed from logic where it means deriving conclusions from premises. The ML usage marks a systems boundary: training executes forward and backward passes and maintains optimizer state, while ordinary inference executes the forward graph with frozen parameters. Removing gradients, optimizer state, and most saved activations can reduce memory and computation substantially, but the ratio depends on architecture, batch size, precision, runtime workspace, and serving strategy.

↪ When monitored drift should trigger retraining, and what each retraining cadence costs, is developed in section 14.6.1.

[24] **Training GPU power budget**: The "high-memory" requirement is driven by the need to hold parameters, gradients, optimizer state, and activations simultaneously. The corresponding power draw dictates the "dedicated data-center cooling infrastructure," as a single high-end training GPU consumes 400 W–700 W. Even compared with a 4 W mobile inference chip, that is at least 100× the power budget.

[25] **Edge inference accelerators**: Google's edge Tensor Processing Unit (TPU), the Edge TPU (Google Coral), operates in the mobile/embedded tier at about 2 W, delivering 4 TOPS through an INT8 datapath. Edge servers sit in a different tier: Jetson AGX Orin reaches 275 TOPS at 15 W–60 W, about 68.8× more throughput but with wired-power assumptions.

↪ Tensor Cores, which execute a small matrix multiply-accumulate per instruction and set the reduced-precision peak rates quoted for data center GPUs, are developed in section 11.4.2.

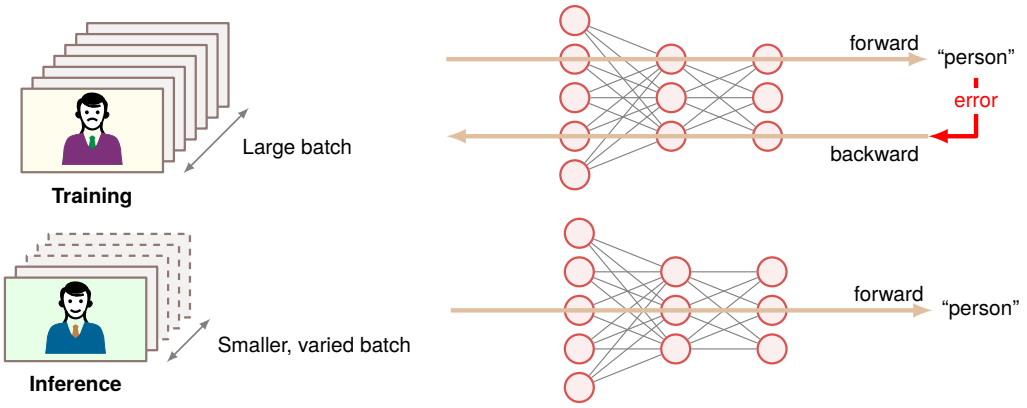


Figure 5.20: Inference vs. Training Flow: Both flows run the same fully connected network; the batch outlines (dashed and varied for inference, solid and uniform for training) mark the two operating regimes.

Table 5.10: Training vs. Inference Forward Pass: Both phases execute the same forward equations, but they manage state differently. Training must retain or recompute the values required by the backward pass and also carries gradients and optimizer state. Inference can reuse a rolling pair of layer buffers.

Characteristic	Training Forward Pass	Inference Forward Pass
Activation Storage	Retains values required by the backward pass	Retains the current input and output buffers
Memory Pattern	Saves or recomputes selected intermediate states	Reuses buffers as successive layers complete
Computational Flow	Same matrix products; each layer input is kept	Same matrix products; each input freed after use
Resource Profile	Weights, gradients, optimizer state, saved inputs	Weights plus the largest adjacent buffer pair

5.5.2 Memory and computational resources

Inference has two memory obligations. The first is persistent: the trained weights and biases must remain available for every request. The second is transient: in this sequential feed-forward graph, each layer produces an activation buffer that is needed only until the next layer consumes it. This distinction is the reason inference can be much leaner than training, even though the arithmetic in the forward pass is the same.

For the canonical MNIST network ($784 \to 128 \to 64 \to 10$), the persistent parameter block contains 109,386 parameters, or about 438 KB at 32-bit floating point precision. The layer-level counts in table 5.5 show why inference is cheap to plan. Each fully connected layer performs one multiply-add for each weight, so parameter memory and arithmetic scale together.

The transient side is where inference saves the most. Under the retained-activation training scheme used here, activation tensors persist until backpropagation reaches their corresponding layers, producing cumulative memory consumption across the entire network depth ($\sum_\ell M^{(\ell)}_{\text{activations}}$, where $M^{(\ell)}_{\text{activations}}$ is the byte size of layer ℓ's activations). In a sequential feed-forward graph, inference needs a layer's input buffer only until that layer has produced its output. Once layer $\ell+1$ finishes execution, the activation buffer for layer ℓ is dead and its storage can be overwritten, so peak transient activation memory falls from the cumulative total to the largest adjacent pair, $\max_\ell \left(M^{(\ell)}_{\text{activations}} + M^{(\ell+1)}_{\text{activations}}\right)$. A skip connection or any other later consumer extends a buffer's lifetime and must enter that plan; the static memory planners that serving runtimes build on this property are developed in section 13.6.2.2.

For the running network at batch size 32, that rule produces the inference column of table 5.9. Inference holds the 438 KB parameter block plus a rolling activation pair of about 117 KB, roughly 554 KB in all. The training step of table 5.9 holds about 1.9 MB, 3.4× as much, for the same weights and the same forward arithmetic. The entire difference is state that training keeps and inference

does not: saved activations, gradients, and optimizer buffers. Fixed parameters and short buffer lifetimes also make inference easier to optimize, and batching is the first lever.

5.5.3 Optimizing inference throughput and latency

Serving one request at a time ($B = 1$) turns every fully connected layer into a matrix-vector product that reads each weight from memory only to use it in a single multiply-accumulate ($z_j \leftarrow z_j + x_i W_{ij}$), leaving arithmetic intensity at about 0.5 FLOP/byte (notebook 5.1). Because inference freezes parameters and eliminates backward state, runtime engines can plan batching, memory layout, operator scheduling, and numerical precision against the physical limits of the target processor.

Input batching trades per-request latency against hardware efficiency. Aggregating requests into a batch turns that matrix-vector product into a matrix-matrix product that reuses each loaded weight across all B inputs, raising the MNIST network's intensity to about 11.9 FLOP/byte at batch 32. Weight reuse matters most when parameters stream from off-chip memory and the workload sits below the machine's ridge point; a model as small as MNIST can instead keep its weights in cache from one request to the next.

Batching also increases the activation values produced: the MNIST network produces 202 layer-output values per image (986 values including the input), or 6,464 layer-output values for a batch of 32. These are cumulative counts across layers; simultaneous storage follows the rolling-buffer rule of section 5.5.2. Production systems serving interactive traffic must also limit the wait to accumulate inputs according to their service-level latency budget.

↪ How serving systems batch incoming requests dynamically to amortize parameter transfers is developed in section 13.5.3.

The stages around the network have an entirely different computational profile from the forward pass. While neural network execution exploits massively parallel matrix units on accelerators, preprocessing (such as image decoding and normalization) and postprocessing (such as argmax reduction, threshold checking, and message serialization) often execute on the host CPU, although implementations can parallelize or accelerate them. Amdahl's law, met in the introduction and developed for accelerated systems in section 11.1, bounds end-to-end speedup by the fraction of time that is not accelerated, so as accelerator kernels drive forward-pass execution times toward zero, these unaccelerated CPU stages and their associated Peripheral Component Interconnect Express (PCIe) data transfers can come to dominate end-to-end request latency.

Three further mechanics complete the inference design space, and each has an owning chapter. Kernel fusion keeps a layer's intermediate results in on-chip storage instead of writing them to off-chip memory between the matrix multiply, the bias addition, and the activation (section 7.3.2). Quantization stores frozen weights in fewer bits, which inference tolerates better than training because no small gradient updates must survive (section 10.3.3.1). Autoregressive transformers add a key-value cache of intermediate results from earlier tokens that grows with every token generated (section 6.7.4.2).

The complete lifecycle of the running network, from architecture through training to inference, is now a set of operations with quantifiable costs. Those estimates were made in a controlled setting, with clean data, no deadline, and an unchallenged machine. A production deployment, where none of those conditions holds, is the real test of whether they are complete.



Checkpoint 5.5: Complete neural network system

Before turning to how these mechanics first performed in deployment, verify your understanding of the complete neural network lifecycle:

Use the MNIST classifier ($784 \to 128 \to 64 \to 10$) as the running production system.

- ☐ **System lifecycle**: can you trace the lifecycle from architecture design to training loop to trained model to inference deployment?
- ☐ **Training vs. inference memory**: at a matched batch size, can you explain why this estimate gives training roughly 3.4× the memory of inference, naming the gradients, optimizer state, and saved-activation terms?
- ☐ **End-to-end trace**: can you trace one digit image through preprocessing, forward propagation, and postprocessing to a final prediction, and say which of those stages usually run on the host CPU rather than the accelerator?

- ☐ **Bottleneck spotting**: if the forward pass became ten times faster while preprocessing on the host CPU stayed unchanged, which stage would set the end-to-end rate, and what bound does Amdahl's law place on the overall speedup?
- ☐ **Batching trade**: can you explain why batching raises arithmetic intensity at batch size $B > 1$, what it costs in activation memory and per-request latency, and why serving systems cap the wait to fill a batch?

Recognizing a digit was not always a benchmark. Decades before large datasets and specialized accelerators, the same task ran in production, in the mail-sorting and check-reading systems where neural inference first met a physical deadline.

5.6 Handwritten Digit Recognition

In the late 1980s, LeCun and colleagues demonstrated backpropagation-based recognition of handwritten ZIP-code digits from USPS-supplied images (LeCun, Boser, et al. 1989). Related convolutional document-recognition work later supported commercially deployed bank-check readers (LeCun et al. 1998). These were distinct systems, but they exposed the same engineering pattern: preprocessing normalized variable handwriting, neural inference produced candidate digits, rejection thresholds sent uncertain predictions to human review, and downstream logic converted accepted predictions into operational decisions. That shared pipeline is the chapter's running digit task, deployed. It tests every cost estimate the chapter has built, because preprocessing, model structure, and rejection policy must interact under a physical throughput constraint, and its first obligation is to turn the network's scores into decisions.

5.6.1 From scores to decisions

Neural network outputs become useful only after they are converted back into decisions a conventional system can act on. Preprocessing bridges real-world data into tensor form; postprocessing maps neural outputs into labels, decision thresholds, validation logic, error handling, and downstream messages. For the running network, raw logits are not enough. A digit-recognition system needs the most likely digit class, a confidence score whose calibration has been evaluated, and a fallback route that directs ambiguous or low-confidence predictions to human review or a secondary recognizer.

↪ Calibration, the agreement between a classifier's confidence scores and its observed accuracy, is measured in section 12.9.1.2.

That fallback route is itself a design decision with a measurable trade. A confidence threshold on the top-class probability sends every prediction below it to a person. Raising the threshold reduces the accepted fraction and increases review workload. For a fixed set of scored inputs, it cannot increase the number of accepted mistakes, but the error rate among the remaining accepted predictions can rise or fall because both the error count and the accepted count change. Engineers therefore measure a risk-coverage curve, plotting error among accepted predictions against the fraction of inputs accepted, on held-out data that resembles deployment traffic. A usable operating point must meet the error budget while keeping rejected traffic within review capacity. Softmax probabilities need not be calibrated, and distribution shifts can change both counts, so the selected threshold and its review workload require monitoring.

Confidence thresholds determine the division of labor between automation and human review. An expected-cost rule (Chow 1970) chooses threshold τ to minimize $r_{\text{manual}}(\tau)\,C_{\text{manual}} + r_{\text{error}}(\tau)\,C_{\text{error}}$ subject to throughput and latency constraints, where $r_{\text{manual}}(\tau)$ and $r_{\text{error}}(\tau)$ are the fractions of all arriving items sent to manual review and accepted with a wrong automated classification, and C_{manual} and C_{error} are their per-item costs. Both fractions count against every arriving item, so an error rate reported only among accepted items must first be multiplied by the accepted fraction. The 1989 ZIP-code system reported one measured point on this curve, examined in section 5.6.4.

5.6.2 The document-recognition challenge

In a physical postal sorter, envelopes move past optical scanners along mechanical belts at fixed speed, leaving only tens of milliseconds per item to capture the image, isolate the digits, evaluate the network, and trip the physical diverter gates. A misrouted envelope or misread check incurs

costly manual re-handling or financial reconciliation, yet raw accuracy on uncontrolled handwriting cannot reach 100 percent, so the pipeline must pair automated inference with an explicit rejection rule that routes ambiguous digits to human operators. The sample grid in figure 5.21 shows this handwriting variation across real postal envelopes, where character segmentation and recognition must handle heavy cursive slant, uneven stroke widths, broken loops, and patterned envelope backgrounds across city names and five-digit ZIP codes.

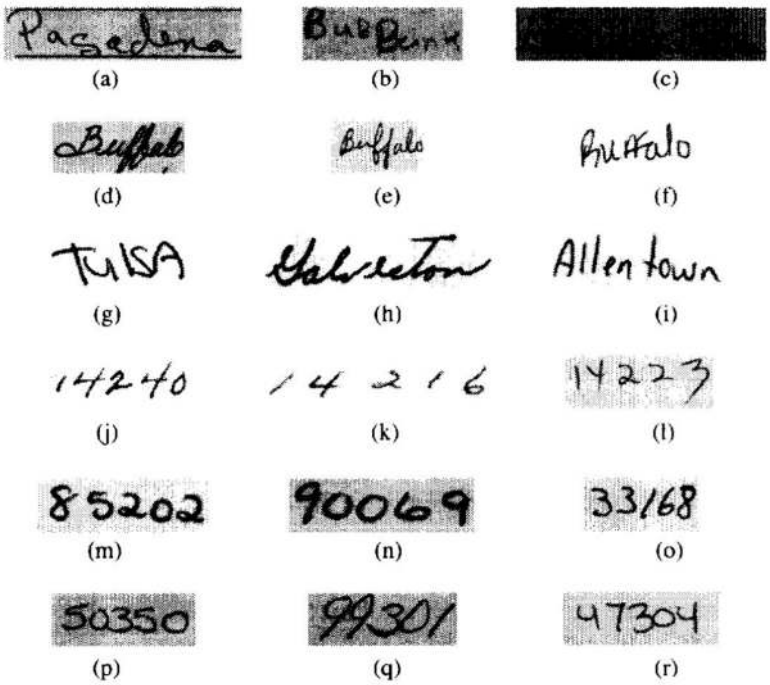


Figure 5.21: Handwritten Address Variability: Address and ZIP-code samples show wide variation in stroke width, slant, spacing, and character formation. A document-recognition pipeline must absorb this variation without confusing legitimate writing differences with changes in the intended characters. Source: CEDAR CD-ROM 1 USPS Office of Advanced Technology database.

No single stage can absorb all of that variation alone, and the dataset histories behind the task must also remain separate. Early ZIP-code research used postal digit images supplied for that task (LeCun, Boser, et al. 1989), whereas MNIST was later constructed from National Institute of Standards and Technology (NIST) handwritten-digit databases and standardized into size-normalized benchmark images (LeCun et al. 1998), not collected from live envelopes under varying colors, textures, lighting, and orientations. Both histories nonetheless imposed the same engineering requirements. Preprocessing had to reduce irrelevant variation without erasing the stroke information needed for recognition, and training had to cover handwriting variation, with augmentation exposing the model to additional plausible variation, rather than optimize only for a curated test set.

↩ Why evaluation on curated data must be validated against field conditions is established in section 3.8.

5.6.3 Shared pipeline architecture

Feeding raw envelope scans straight into a digit classifier fails because the network expects isolated, size-normalized characters rather than full addresses on textured paper, and its raw scores still need a rejection rule before they can trip a sorting gate. The composite six-stage pipeline in figure 5.22 separates the work shared by postal ZIP-code research and deployed bank-check readers (rather than reconstructing a single historical machine): upstream preprocessing segments and normalizes the scanned digits, the neural network scores each candidate digit, and downstream postprocessing compares those scores against confidence thresholds, passing accepted digits to the sorter while routing ambiguous ones to human review.

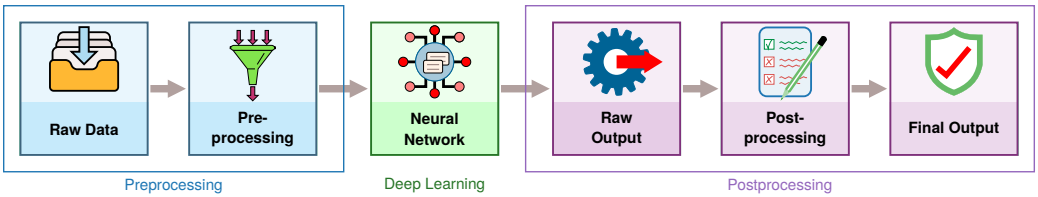


Figure 5.22: Document-Recognition Pipeline: A composite six-stage flow distilled from postal ZIP-code research and deployed check-reading systems. Image acquisition and conventional preprocessing prepare handwritten digits for neural inference, and raw scores then pass through postprocessing to the final application output.

The upstream boundary pairs image acquisition with conventional preprocessing. In postal research, the raw input was an image of a handwritten ZIP-code region, and deployed check readers similarly began with scanned document images; acquisition quality determines how much variation the downstream stages must absorb. Preprocessing then locates the relevant field, separates or identifies its characters, and normalizes the resulting images through conventional techniques such as thresholding, connected-component analysis, and size normalization. (The chapter uses standardized 28×28 images for its MNIST calculations as a teaching convention, not a claim about every postal or check-reading deployment.)

Once the digits are normalized, the neural network scores each candidate image and hands those scores to postprocessing. The 1989 ZIP-code research system used an early LeNet variant (LeCun, Boser, et al. 1989) with approximately 10,000 parameters, occupying approximately 40 KB in single precision, an order of magnitude smaller than the 109.4K in this chapter's fully connected MNIST network. Downstream postprocessing then converts those digit scores into application decisions with the threshold rule of section 5.6.1: for a ZIP-code task, a single uncertain digit can route the entire code to further processing or human review. Because every stage from acquisition to that downstream action counts against the reader's end-to-end timing constraint, throughput depends on the complete pipeline rather than neural-network latency alone.

5.6.4 Performance outcomes and operational impact

The 1989 ZIP-code study reported its model, recognition, throughput, and training measurements as one experimental system. Table 5.11 keeps those measurements together, revealing how rejection policy and preprocessing affect the meaning of raw classifier speed (LeCun, Boser, et al. 1989).

Table 5.11: Early ZIP-Code Recognition Results: Measurements from the 1989 backpropagation system applied to USPS-supplied digit images. The rejection result shows why an operational error rate must be read together with the fraction of inputs withheld from automatic classification (LeCun, Boser, et al. 1989).

Metric	Reported Result
Error and rejection	1% classification error after rejecting 12.1% of test digits
Throughput	10–12 classifications/sec including acquisition; over 30/sec after normalization
Model parameters	~10,000
Training	23 passes; about 3 days on a Sun-4/260 workstation

These measurements highlight three systems trade-offs across the pipeline:

First, the measured rate depended on the measurement boundary. The system exceeded 30 classifications per second after normalization but processed 10 to 12 per second including acquisition. The reciprocal rates are completion intervals, not necessarily individual-digit latencies when stages overlap. The comparison exposes surrounding acquisition and preprocessing work, but does not allocate that work among mechanical handling, scanning, and segmentation. Stage timings are needed before choosing which part of the complete path to accelerate.

Second, operational reliability depended on an explicit rejection pathway. The experiment rejected 12.1 percent of test digits rather than forcing a classification for every input, and rejecting this tail of high-uncertainty predictions reduced the observed classification error on the remaining automated

stream to 1 percent. That is one point on the risk-coverage curve of section 5.6.1, and the expected-cost rule there decides whether it is the right one; the 12.1 percent becomes a review workload that the rule must account for.

Third, training-time cost constrained the exploration of model sizes. Training 10,000 parameters required 23 passes across roughly three days on a Sun-4/260 workstation. That measurement shows the cost of the reported configuration; it does not establish a universal maximum model size for the era. Larger candidates would require a new compute, memory, and training-time budget.

Maintaining these systems in production introduces lifecycle requirements beyond model training. Changes in scanner optics, ink formulations, envelope textures, or regional writing styles cause input distributions to drift over time. Production monitoring must track input image statistics and rejection rates alongside raw classification latency, prompting investigation when rejection threatens review capacity. Whether the remedy is data collection, preprocessing repair, threshold adjustment, or retraining depends on the cause and on validated error costs.

↪ How production monitoring detects silent degradation from input drift while code and weights stay unchanged is developed in section 14.5.

5.6.5 Key engineering lessons and design principles

Across postal ZIP-code research and deployed check readers, neural computation succeeded only when the surrounding pipeline shared its operating constraint. Each of the chapter's mechanisms reappears in that pipeline at system scale. The forward-pass operation count set the cost of inference, but the gap between more than 30 classifications per second after normalization and 10 to 12 per second including acquisition is the Amdahl effect of the boundary stages that the inference section predicted. The network's output scores became decisions only through a threshold whose position traded automated volume against residual error. The convolutional reader's parameter footprint, an order of magnitude below the fully connected network's, shows that connectivity patterns, not layer depth alone, govern the state a model must store. A layer that slides one small set of weights across every image position stores far fewer parameters than one that gives every pixel its own weight, although fewer weights do not guarantee smaller activation tensors, and chapter 6 builds its architectures on that choice of connectivity.

Modern edge systems face different power, latency, and deployment budgets, but the same three obligations hold. Preprocessing must absorb the variability of the physical world before the network sees it, confidence thresholds must protect the downstream error budget, and end-to-end throughput, not model latency alone, governs how much model capacity the system can afford. The deployed reader worked because its data, network, and machine fit each other, and the running network's own numbers show that alignment one axis at a time.

5.7 D·A·M Reading of Neural Computation

↩ The D·A·M taxonomy and its three diagnostic axes are established in section 1.5.

In the D·A·M taxonomy, each axis fixes one of the running network's headline numbers, and the data axis fixed the largest weight matrix before any design choice was made. A 28×28 image flattens to 784 inputs, so the first layer holds 100,352 weights and performs 91.9 percent of the network's arithmetic. The same digits scanned at 56×56 would quadruple that matrix to 401,408 weights without changing the model's depth or the task. Data also decides whether those weights are worth learning. USPS ZIP images, MNIST digits, and check images share digit semantics but come from different scanners, thresholds, and layouts, and no preprocessing step can synthesize handwriting styles that the training set never contained.

The algorithm axis set how that matrix meets the machine. Batch size is an algorithmic choice, and it moved the forward pass's arithmetic intensity from about 0.5 FLOP/byte at batch 1 to about 11.9 FLOP/byte at batch 32, because one read of each weight now serves 32 images. The same choice multiplied the activations that training must keep, which is why training holds 3.4× the memory of inference at matched batch size.

The machine axis decided whether learning survives at all. A gradient smaller than its format's smallest normal value, 6.10×10^{-5} in FP16 against 1.18×10^{-38} in FP32, loses precision or flushes to zero and stops updating its weight, and a value beyond the format's maximum overflows to `Inf`. Narrower formats halve or quarter memory traffic and unlock faster matrix units, but only while gradients and activations still fit the format's range.

No axis can be tuned alone. A larger input improves fidelity but grows the first weight matrix, a larger batch improves reuse but grows training memory, and a narrower format speeds the machine

but endangers the gradients. Diagnosing an accuracy shortfall, a latency violation, or an out-of-memory fault therefore starts with a hypothesis on each axis and ends with a measurement of the quantity that binds, because an intervention on one axis routinely moves the binding constraint to another. Teams that skip that measurement tend to fall into a small set of recurring traps.

↪ Screening heuristics that narrow these competing hypotheses, each confirmed against profiler traces and hardware counters, are collected for reference in section A.5.1.

5.8 Fallacies and Pitfalls

Statistical learning adds failure modes that line-by-line code inspection alone cannot reveal. Each of the recurring mistakes below treats a property of the data, the optimization, or the machine as if the model's source code alone decided it.

Fallacy: *Neural networks are entirely opaque and cannot be debugged.*

Neural networks do not offer the line-by-line causal trace of traditional code, but they are not beyond diagnosis. Activation visualization can reveal learned patterns, gradient analysis measures local input sensitivity, and ablation studies estimate component contributions. These tools provide partial evidence rather than a complete explanation: gradient-based saliency maps (per-input sensitivity heat maps) can be unstable, correlated features complicate attribution, and an ablation may change the behavior it is meant to probe. For the MNIST classifier in section 5.3.2, first-layer weights can still provide a useful check on what patterns the model has learned. Debugging a statistical system therefore combines model-level probes with data slices, controlled interventions, and end-to-end evaluation.

Pitfall: *Discarding domain expertise because a deep model is available.*

Teams assume automatic feature learning removes the need for domain knowledge. Successful systems require domain expertise at every stage: architecture selection, training objective design, dataset curation, and output interpretation. The document-recognition history in section 5.6 shows why confidence thresholds must reflect operating economics and route uncertain cases to human operators. Without domain knowledge, teams can deploy networks that look strong on a test set but fail in production because their thresholds, fallback paths, or error costs are wrong.

Fallacy: *Deeper networks are always more accurate than wider ones.*

Engineers assume that stacking more layers is the primary path to higher accuracy, since depth enables hierarchical feature extraction. In practice, depth alone encounters diminishing returns. ResNet showed that very deep residual networks can train effectively, yet its 1,202-layer CIFAR-10 network tested worse than its 110-layer counterpart (He et al. 2016a). EfficientNet, a family of image classifiers, later demonstrated that compound scaling of width, depth, and input resolution can outperform depth-only scaling at a given resource budget (Tan and Le 2019). Depth is not the problem. Teams should profile capacity utilization and scale the dimension that gives the best accuracy per FLOP, since depth adds sequential steps and saved activations while width adds arithmetic inside each step (equation 5.10).

Fallacy: *Training data distribution issues can be fixed after model design.*

Teams treat training as mechanically feeding data through architectures. Networks on imbalanced datasets often perform poorly on the minority class: a fraud detector with 99:1 imbalance achieves 99 percent accuracy by always predicting "not fraud" while catching zero fraud cases. The unweighted batch average of equation 5.16 can underweight rare but critical classes unless sampling, weighting, loss design, and evaluation address the imbalance. Teams that skip exploratory data analysis may deploy models with strong aggregate metrics but unacceptable minority-class performance.

Pitfall: *Deploying research models to production without addressing system constraints.*

Production adds constraints absent from research: latency budgets (50–100 ms end-to-end), memory limits (2–4 GB for edge devices), and concurrent loads (100–1,000 requests per second (RPS)). The complete pipeline includes preprocessing, inference, and postprocessing (section 5.5). A model achieving 20 ms inference fails its 50 ms budget when preprocessing adds 25 ms and postprocessing adds 10 ms (55 ms total). Separating model development from system design therefore wastes effort on accuracy while ignoring deployment feasibility.

Fallacy: *More compute automatically means faster training.*

Teams purchase expensive GPUs expecting proportional speedups, but memory traffic or launch overhead may bind performance instead. Arithmetic intensity tests that hypothesis (figure 5.14). The MNIST forward pass in table 5.7 reaches only about 11.9 FLOP/byte at batch 32 (notebook 5.1) and less at batch 1, while the ridge point $R_{\text{peak}}/\text{BW}$, the intensity a workload needs before peak compute can bind, sits above it on both machines at FP32 and far higher at the reduced-precision peak rates of data-center accelerators. A faster chip therefore leaves this workload waiting on memory, or on kernels too small to fill the machine; the roofline model in section 11.6 makes the condition precise, and measured speedup remains the final test.

Pitfall: *Extrapolating accuracy improvements without considering diminishing returns.*

Teams may extrapolate a 5-point gain from scaling 10K parameters to 100K into another 5-point gain at 1M. Empirical improvement instead diminishes with scale according to the task, metric, data quality, model family, and scaling regime. Published benchmark histories cannot settle the question either. ImageNet top-5 error fell from AlexNet's 15.3 percent to 3.6 percent for a ResNet ensemble (Krizhevsky et al. 2012; He et al. 2016a) while training compute rose by more than an order of magnitude (Amodei and Hernandez 2018a), but those systems mix new architectures with more compute, so they do not isolate the return on scale alone. Teams must therefore measure marginal return within one model family in their own regime rather than extrapolate linearly.

These fallacies and pitfalls share a common root: treating statistical behavior as if code behavior alone determined it. Recognizing them early directs attention toward the appropriate measurements and diagnostics.

5.9 Summary

Neural computation turns layer equations into physical work on a processor. Linear algebra and calculus determine operation cost under the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). Forward propagation establishes the computational work O, often dominated by matrix multiplications, while the complete training step adds backward computation and optimizer updates. Backpropagation needs saved or recomputed forward values and gradient storage; the optimizer may add persistent state. Architecture, batch size, precision, optimizer, and recomputation strategy determine the state footprint. Residency, reuse, and execution then determine the bytes transferred, D_{vol}, which need not equal that footprint. These mechanics make memory footprints, arithmetic intensities, and latency limits estimable before deployment. Table 5.1, from the chapter's opening, now reads as a summary of where each stage spends them, and every row has been derived for the running network.

Replacing hand-written decision rules with parameters learned from data turns feature engineering into matrix arithmetic. Layered compositions of weighted sums, nonlinear activations, and gradient updates form hierarchical representations that convert raw inputs into predictions. Training adjusts connection weights through iterative optimization, while inference applies fixed parameters to incoming data. The two phases make very different demands on the machine. In this chapter's FP32 estimate, a batch-32 training step requires ~3.4× the memory of batch-32 inference because training retains saved activations and gradients alongside the optimizer's additional buffers. In production, as the document-recognition case study demonstrated, these mathematical kernels do not run in isolation; preprocessing, neural inference, rejection, and postprocessing must jointly satisfy a single end-to-end latency and reliability budget.

Key Takeaways: The math behind the model

- **Each paradigm shift changes the systems cost of representation**: In this 28×28 digit example, the operation count rises from ~100 comparisons (rule-based) through ~8,000 operations (classical ML) to 109,184 MACs (deep learning), a 1,100× scenario increase that reshapes hardware requirements.
- **The loss is cheap but seeds every update**: Cross-entropy costs a few operations per class score, yet its gradient, the predicted probability minus the target, is the error signal

from which every gradient update descends. What the system must protect is the loss's numerical range, which is why runtimes fuse softmax with cross-entropy.

- **Training and inference often emphasize different objectives**: Training commonly favors efficient throughput across repeated updates, while online inference commonly favors per-request latency; offline inference may favor throughput instead. Batch size links utilization, memory, throughput, latency, and statistical behavior across these regimes.
- **Activations are math and hardware**: ReLU is generally cheaper to implement than sigmoid or tanh, and its unit gradient for positive inputs avoids activation saturation on that branch. Its practical advantage combines implementation cost with optimization behavior.
- **Forward propagation often centers on matrix work**: Dense matrix kernels exceed 99 percent of FLOPs in this chapter's dense-network example and dominate many important neural workloads, which is why specialized matrix hardware can substantially outperform general-purpose execution for those operations.
- **Backpropagation needs the path**: Solving credit assignment requires retaining or recomputing the values used by the backward pass, so memory cost often determines whether a model can be trained on a given device and motivates techniques that reduce, recompute, or partition training state.
- **The complete ML pipeline determines end-to-end performance**: Preprocessing, neural computation, and postprocessing all contribute to latency and reliability. The document-recognition examples demonstrated that production success depends on the entire pipeline operating within real-world constraints, not on model accuracy on a test set alone.

The escalation from hand-written rules to learned features is the algorithm axis of the D·A·M taxonomy made concrete. Learning representations instead of writing them spends arithmetic and memory in place of human design effort, and every systems dimension in table 5.2 shifts as a result. The bitter lesson predicts that this exchange wins once computation is cheap enough to afford it, but making the exchange still requires a measured gain that justifies its cost on the intended machine.

The multilayer perceptrons analyzed in this chapter expose a direct resource law, in which dense connectivity forces both parameter storage and multiply-accumulate count to scale with the product of adjacent layer dimensions. That dense connectivity contains no explicit prior for spatial locality, so the model must learn from data which nearby and distant pixel relationships matter, while paying memory bandwidth and storage to evaluate every connection.

Reading the code reveals what a network is asked to do; reading the math reveals what it will cost to execute. That arithmetic is where the algorithm meets the machine: weights must be stored and transferred, dense layers map to matrix execution engines, and saved activations claim accelerator memory that determines whether training fits on device. Under the silicon contract (principle 4), the math commits in advance to operations the machine must execute. Auditing that contract allows systems engineers to calculate binding bottlenecks from first principles before profiling confirms where the execution stalls.

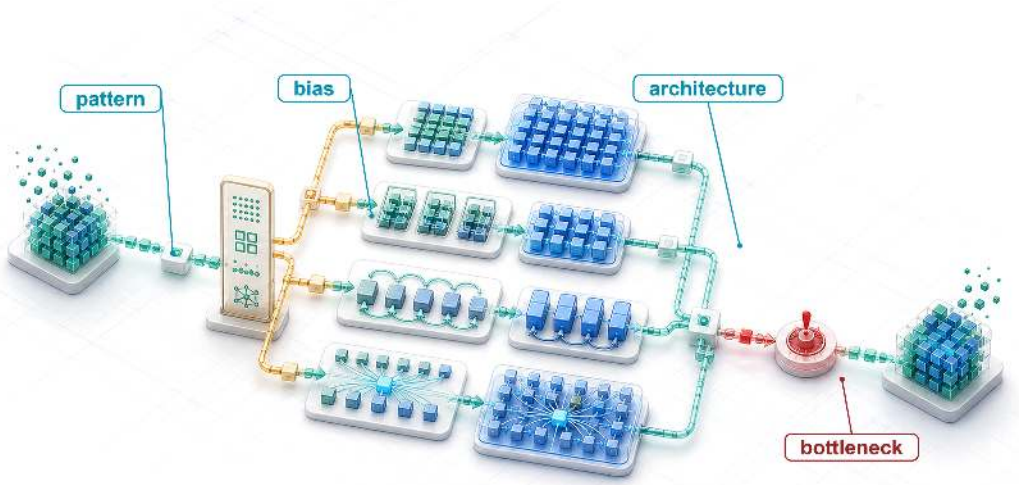


What's Next: From universal to specialized

Real-world data is rarely structureless. Images have spatial locality, text contains sequential dependencies, and audio signals follow temporal patterns, yet a fully connected layer charges memory volume D_{vol} and compute work O for every connection regardless. Chapter 6 assembles the same weighted sums, activations, and gradients into architectures shaped like their data, from convolutional and recurrent networks to transformers and embedding-heavy recommenders, each encoding its structural assumption as an inductive bias. Tailoring network topology to data structure reduces redundant data movement and improves weight reuse, but introduces new trade-offs in activation storage, arithmetic intensity, and accelerator parallelization.

6

Network Architectures

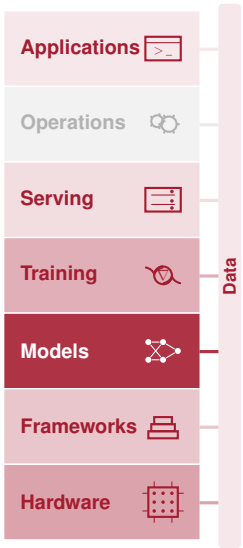


6.1 Architectural Principles
6.2 MLPs: Dense Pattern Processing
6.3 CNNs: Spatial Pattern Processing
6.4 Skip Connections and Normalization
6.5 RNNs: Sequential Pattern Processing
6.6 Attention: Dynamic Processing
6.7 Transformers: Parallel Sequence Processing
6.8 Sparse Architectures: Recommendation Systems
6.9 Comparing and Choosing Architectures
6.10 Fallacies and Pitfalls
6.11 Summary

Purpose

Why is choosing a neural network architecture an infrastructure commitment as well as a modeling decision?

Selecting a neural network architecture is both a modeling decision and a contract with physics. The commitment begins with the structure in the data. An architecture that matches that structure can learn with fewer parameters and examples, while a poor match spends additional data and machine time compensating for the wrong computational pattern. Convolution, attention, recurrence, and embedding lookup do not merely encode different assumptions about data; they create different patterns of computation, state, and data movement. Those patterns determine whether work parallelizes cleanly, whether intermediate state fits in memory, and whether training and serving remain affordable at the required scale. Attention compares every element of a sequence with every other, so doubling the input length roughly quadruples that work, while the cost of a convolution grows in proportion to its input. Two architectures can deliver similar predictive quality yet impose different memory footprints and latency profiles while exposing different opportunities for later efficiency techniques. Benchmark accuracy alone therefore cannot reveal which design remains viable. Architectural commitments also propagate beyond the model. Data pipelines adopt particular input structures, training infrastructure is provisioned around the model's compute profile, serving systems are tuned to its request path, and monitoring inherits its failure modes. Once those dependencies accumulate, replacing the architecture can mean rebuilding much of the surrounding system, a cost that grows with every month the design stays in service. An architecture decision is sound only when its predictive gains remain compatible with the full deployment context. In D·A·M terms, architecture is algorithm-machine co-design at its root, with the mathematical graph determining the work the machine must perform and the machine budget limiting which graphs remain practical.



Learning Objectives

- Explain how an architecture's inductive bias matches the structure of images, sequences, and tabular data
- Distinguish how MLPs, CNNs, RNNs, transformers, and DLRM-style recommenders lower onto matrix multiplication and which operations remain memory bound
- Explain why skip connections and normalization make deep networks trainable and what each costs the machine
- Calculate the weights, multiply-accumulates, and memory of dense, convolutional, recurrent, attention, and embedding layers
- Analyze which iron-law term binds a workload, including transformer prefill and decode, from its arithmetic intensity
- Apply the architecture selection framework to a deployment's data, memory, latency, and power budgets
- Critique architecture-selection fallacies about operation counts, context length, and training-versus-serving bottlenecks

6.1 Architectural Principles

THREE requests reach the same accelerator: a camera frame for an image classifier, a prompt for a language model, and a click for a recommender. The silicon is identical, yet the first request keeps the arithmetic units busy, the second leaves them waiting on memory bandwidth, and the third cannot begin until the system decides where its multi-gigabyte embedding tables can live. What differs is the network architecture, the way each model arranges the same arithmetic around the structure of its data, and this chapter explains how each family of architectures sets the demands its requests place on the machine.

Chapter 5 built the operations that every network shares (matrix multiplication, activation functions, and gradient computation) and assembled them into a dense multilayer perceptron (MLP) that pays for a weight between every pixel and every hidden unit, including relationships an image rarely needs. Architectures remove that waste by arranging the same operations into computational graphs shaped like the data. Under the silicon contract (principle 4), that shape is also an implicit agreement with hardware, trading computational patterns for efficiency on particular problem classes.

Every neural network architecture decides how computation should be organized to match the inherent structure in the data. Images have spatial locality, language has sequential dependencies, and tabular records often lack a fixed spatial or sequential organization. The architecture encodes assumptions about these patterns directly into the computational graph, and those assumptions influence parameter count, hardware utilization, and deployment feasibility. Architecture selection is therefore a systems engineering problem that directly affects the iron law terms: the number of operations O and the volume of data movement D_{vol}. The structural assumptions that each architecture encodes are known as **inductive biases**,[1] and each architecture in this chapter is defined by its choice of bias.

Definition 6.1: Inductive bias

Inductive bias is a structural constraint built into a model architecture that restricts the hypothesis space, enabling generalization from finite data by encoding domain-specific assumptions (such as spatial locality or sequential ordering) directly into the computational graph.

1. **Significance**: Inductive bias can reduce the dataset size (D) required for generalization. A detector that looks for a small pattern, such as an edge, can either give every pixel of the image its own weight or apply one small set of weights at every position. For a 224×224 RGB image, a shared 3×3 filter uses roughly 5,575× fewer parameters than one

[1] **Inductive bias**: From Latin *inducere*, "to lead into." Encoding a structural assumption "leads" the model toward a smaller solution space, and every architecture discussed here (multilayer perceptron (MLP), convolutional neural network (CNN), recurrent neural network (RNN), and transformer) is defined by its choice of bias. A CNN's locality bias cuts parameters by orders of magnitude vs. an equivalent MLP, directly shrinking the iron law's O and D_{vol} terms, while global attention trades quadratic score computation for long-range connectivity.

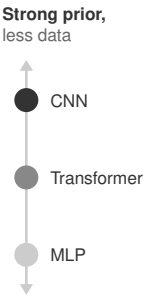
dense feature detector, reducing parameter storage and often the data required to avoid overfitting.

2. **Distinction**: Unlike regularization (which penalizes hypothesis complexity at training time via L1/L2 terms), inductive bias restricts the hypothesis space at architecture design time, before any training begins.
3. **Common pitfall**: A frequent misconception is that stronger inductive bias is always better. A strong locality bias (CNN) excels on spatial data but represents long-range dependencies in language less directly than global attention.

A convolutional neural network (CNN) encodes an inductive bias of spatial locality: nearby pixels matter more than distant ones. A transformer lets each element attend to any other, enabling long-range relationships at quadratic score-computation cost. These biases help architectures learn efficiently by restricting the space of functions they represent. Without an appropriate bias, a model may require substantially more data and compute to learn the same structure. That exchange is the bitter lesson seen from inside a single architecture. Built-in structure does part of the learning in advance, while the transformer's weak priors assume that data and compute will keep growing, an assumption that has held at scale. Section 6.9 returns to this ordering when choosing among families.

Machine learning systems face a core engineering trade-off: **representational power vs. computational efficiency**. Under the iron law of ML systems (principle 3), architectural choice is the primary determinant of the algorithmic work per sample (FLOPs/Sample), and thus of the operation-count term O. Attention enables global relationships at a higher operation count, while a CNN exploits spatial locality to reduce operations to linear scaling in the number of spatial positions. Architecture selection means matching the bias to the data while staying inside an operation budget.

Choosing an architectural backbone sets the primary memory layout and compute pattern for downstream hardware. Table 6.1 contrasts the five major neural network families across the data structure each targets, its core innovation, and the system bottleneck it exposes in representative regimes.



A stronger prior substitutes engineered structure for data and compute.

↩ The bitter lesson, that general methods which scale with computation outpace hand-built structure, is established in section 1.4.

Table 6.1: Neural Architecture Families: Each family targets a data structure and exposes characteristic resource demands. The bottleneck column names representative regimes: streamed dense weights stress bandwidth, convolutional reuse raises arithmetic intensity, recurrence creates serial dependencies, dense attention computes quadratic score interactions, and large embedding tables stress capacity. Batch size, precision, kernel implementation, and the target machine determine which demand binds in a deployment.

Architecture	Data Type	Core Innovation	System Bottleneck
MLPs	Tabular/Unstructured	Dense connectivity	Bandwidth with streamed weights at small batch
CNNs	Spatial (images)	Local filters + weight sharing	Compute throughput with sufficient reuse
RNNs	Sequential (time series)	Recurrent state	Sequential dependencies
Transformers	Relational (language)	Dynamic attention	Quadratic score compute; optional S^2 storage
DLRM	Categorical (recommendations)	Embedding tables	Capacity when tables exceed device memory

The five lighthouse models introduced with the iron law return here with their architectures exposed: ResNet-50 for compute throughput, GPT-2 for memory bandwidth, DLRM for memory capacity, MobileNetV2 for compute within a mobile power envelope, and keyword spotting (KWS) for an always-on power envelope. Earlier chapters used them as named workloads and mapped them onto workload archetypes (section 2.3.2), such as the compute-bound and bandwidth-bound archetypes; this chapter opens each one up inside the family that builds it, to show which architectural choice puts it in its archetype. MobileNetV2 sits on the empirical Pareto frontier plotted in figure 6.1, the set of models that no other plotted model beats on both accuracy and compute at once, while ResNet-50 sits well below it. The frontier bounds ImageNet[2] top-1 accuracy against logarithmic compute cost, measured in GMACs (billions of multiply-accumulate operations) per image. The

2 **ImageNet**: The image-classification benchmark on which convolutional networks first proved themselves at scale. AlexNet's 2012 victory in the ImageNet Large Scale Visual Recognition Challenge posted 15.3 percent top-5 error against 26.2 percent for the second-place entry, on the 1000-class challenge with roughly 1.3 million training images (Krizhevsky et al. 2012); the original ImageNet release contained 3.2 million images across 5,247 synsets (Deng et al. 2009). Subsequent architectures improved accuracy through changes in architecture, optimization, data, and compute budgets, illustrating the interaction between inductive bias and infrastructure cost.

other three lighthouses sit off this plot because they were chosen for the bottlenecks they isolate rather than for image-classification accuracy.

↩ The five lighthouse models, and the bottleneck each one isolates, are established in section 1.7.1.

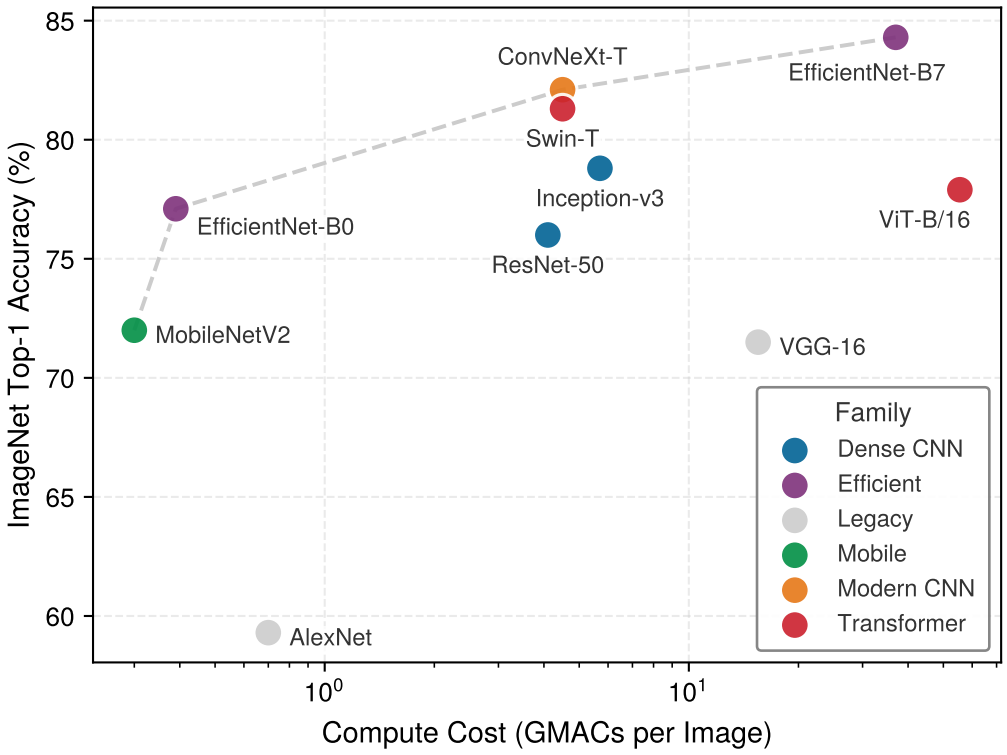


Figure 6.1: The Efficiency Frontier: ImageNet top-1 accuracy plotted against computational cost in GMACs per image. Representative points are compiled from published model reports for AlexNet, VGG, ResNet, Inception, MobileNetV2, EfficientNet, ViT, Swin Transformer, and ConvNeXt (Krizhevsky et al. 2012; Simonyan and Zisserman 2015; He et al. 2016a; Szegedy et al. 2015; Sandler et al. 2018; Tan and Le 2019; Dosovitskiy et al. 2021; Liu et al. 2021; Liu et al. 2022). The dashed Pareto frontier is an illustrative trade-off curve rather than a benchmark rerun; exact accuracy and reported compute values vary by model variant, training recipe, implementation, and operation-counting convention (multiply-accumulates vs. floating-point operations). The legend distinguishes six families: legacy CNNs (gray), dense CNNs (blue), efficient Mobile architectures (green), EfficientNet-class models (violet), modern CNNs (orange), and Transformers (red). The frontier is led by the efficient and modern-CNN families; the Transformers trade substantial compute for flexible long-range modeling without dominating it, and ViT-B/16 sits below the frontier. AlexNet is the single-model result, and ViT-B/16 is plotted at the 384-pixel resolution at which its accuracy was measured.

Across the frontier plotted in figure 6.1, successive architecture families occupy distinct computational cost regimes, from dense CNNs that prioritized accuracy regardless of operation count, through compact mobile networks engineered to minimize GMACs, to vision transformers (such as Swin-T and ViT-B/16) that trade compute for long-range spatial mixing. The plotted models are historical milestones rather than case studies; only ResNet-50 and MobileNetV2 receive detailed architectural breakdowns in this chapter. Architectural choices made at design time dictate where a system lands along this frontier.

Each family section that follows reads its architecture through the same four questions: the data pattern it targets, the computation it performs, how that computation maps onto hardware, and the bottleneck it exposes. The sequence begins with the family that assumes the least about its data, the dense multilayer perceptron. After CNNs, a short section explains the two blocks that make depth trainable; after the recommendation model, the closing section compares all five families on one machine and turns the comparison into a selection procedure.

6.2 MLPs: Dense Pattern Processing

The MNIST digit classifier of chapter 5 is the purest example of dense processing: each of its 784 pixel inputs connects to every hidden unit, and each hidden unit to every unit in the next layer. The same all-to-all pattern suits tabular inputs, such as the sender reputation, link count, and keyword features a spam filter scores, where no feature has a fixed neighbor. **MLPs**[3] represent the fully connected architecture of that classifier, now examined through the four-part systems lens.

MLPs embody an inductive bias: *they impose no specific spatial or temporal connectivity pattern, allowing any input to relate to any output*. This architectural choice provides flexibility across input relationships, making MLPs versatile but potentially costly compared to alternatives that exploit matching structure. Their representational capacity was established theoretically by the universal approximation theorem (UAT)[4] (Cybenko 1989; Hornik et al. 1989), introduced in chapter 5. This theorem states that a sufficiently large MLP with nonlinear activation functions can approximate any continuous function on a compact domain, given suitable weights and biases. The *multilayer perceptron* combines theoretical universality with dense connectivity.

> Definition 6.2: Multilayer perceptrons
>
> ***Multilayer perceptrons*** are feed-forward neural network architectures that apply fully connected layers in sequence, where every neuron in one layer connects to every neuron in the next, without imposing a specific spatial or temporal connectivity pattern.
>
> 1. **Significance**: The lack of structural prior gives dense layers quadratic parameter scaling in layer width: a single layer mapping 1,024 inputs to 1,024 outputs requires 1,048,576 parameters and about 2.1 MB of weight memory in FP16. Every input gets its own weight to every output, which makes MLPs inefficient for high-dimensional structured inputs like images.
> 2. **Distinction**: Unlike convolutional neural networks, which exploit spatial locality to reduce parameter count, MLPs treat all input elements symmetrically, making them the architecture of choice for tabular data where no spatial or sequential structure is present.
> 3. **Common pitfall**: A frequent misconception is that MLPs are too simple to matter for complex tasks. Dense layers carry much of the arithmetic inside the most complex models: every transformer block contains a position-wise feed-forward MLP, and recommendation models such as DLRM run dense MLP towers on their features. The other families add structure around dense layers (weight sharing, recurrence, attention) rather than replacing them.

6.2.1 Representation vs. learnability

Although the Universal Approximation Theorem guarantees that a wide enough MLP can represent any continuous function on a bounded domain, it gives no guarantee that gradient descent can find those weights from a finite dataset or fit the required width in memory. *Representation capacity* refers to the functions an architecture can express given unlimited resources, and the UAT guarantees that MLPs have universal representation capacity. The *manifold hypothesis*,[5] which holds that high-dimensional data lies near a much simpler low-dimensional surface, explains why real-world functions have structure worth exploiting. *Learnability* refers to whether gradient descent can find good weights given finite training samples and compute budgets. A function may be representable in theory yet unlearnable in practice.

This distinction resolves the apparent paradox between universal approximation and architectural progress. Specialized architectures such as CNNs and their deep ResNet descendants improve learnability by embedding inductive biases that match the structure of the data, even when doing so restricts representational capacity. Two factors create this *learnability gap*:

1. **Sample complexity**: The UAT bounds neither the training examples nor the width needed, and some function classes require widths that grow rapidly with input dimension. For 28×28 images, an MLP receives 784 pixel values without built-in local connectivity or spatial weight sharing. It can learn spatial correlations, as the digit classifier of chapter 5 shows, but must learn

[3] **Perceptron**: Formed from "perception" and the device suffix "-tron" (as in *cyclotron* and *klystron*), coined by Frank Rosenblatt (Rosenblatt 1957) for the atomic unit of neural computation: a weighted sum followed by a nonlinear activation, extending the earlier McCulloch-Pitts neuron. MLPs are composed entirely of these units arranged in fully-connected layers, so the efficiency of this single operation, a multiply-accumulate, determines system throughput. Modern accelerators execute over 10^{14} of these operations per second, making the perceptron the computational primitive that the entire ML hardware ecosystem is optimized around.

[4] **Universal approximation theorem (UAT)**: This theorem establishes representational capacity for continuous functions on a compact domain, given suitable nonlinear activations. The systems-level catch is that the existence guarantee supplies neither an efficient training procedure nor a practical width for a particular task. For a generic smooth target with no further structure, the width needed to reach a fixed accuracy can grow exponentially with input dimension (the curse of dimensionality), which is why the guarantee alone says little about a 256×256 image; useful structure in the target is what keeps practical networks small.

[5] **Manifold hypothesis**: The assumption that high-dimensional data lies near a low-dimensional surface embedded within the full space; a 256×256 image lives in a 65,536-dimensional space, but "valid cat images" may occupy a much smaller structured region. Learned representations can expose that structure. The systems consequence is a potential reduction in the data and computation needed for learning; useful structure can also take other forms, such as a target depending on only a few input features.

relationships that a CNN encodes structurally. When those priors match the task, a CNN can reach a target accuracy with fewer examples and represent the function more compactly. The sample requirement depends on the target function, data distribution, and learning procedure.

2. **Optimization difficulty**: Even when optimal weights exist, gradient descent may not find them. Without architectural constraints to regularize the search space, MLP loss surfaces contain many poor local minima and flat regions. Specialized architectures narrow the search space and introduce symmetries that gradient descent can exploit. The classic MNIST handwritten digit benchmark illustrates this gap between representation and learnability concretely.

Example 6.1: MNIST: Representation vs. learnability

Scenario: Classifying 28×28 MNIST digits with a fully connected MLP vs. a CNN.

Diagnosis: Whole-network parameter counts depend mostly on width choices. The 784-128-64-10 MLP has 109,184 weights, fewer than the 421,408 of a small two-convolution CNN, whose dense classifier head holds 96 percent of them. The locality saving appears per feature detector. A dense unit learns one weight for every pixel, so 32 features cost 25,088 weights; a convolutional layer learns one 3×3 filter per feature, 288 weights in total, and reuses each filter at all 784 positions. At equal feature count, the convolutional layer needs 87× fewer weights.

Systems lesson: Inductive bias buys parameter efficiency per feature and reuse per weight. Fewer weights per detector reduce the data needed to learn spatial structure, and each weight fetched from memory feeds hundreds of multiply-accumulates, which raises arithmetic intensity.

The learnability gap motivates the core design principle of this chapter: embed inductive biases that match data structure. Each architecture sacrifices theoretical generality for practical learnability. The No Free Lunch theorem[6] (Wolpert and Macready 1997) formalizes this trade-off: the bias that helps one task may hurt another. A bias that ignores where a pattern appears in an image aids image classification but impairs tasks where absolute position matters, such as locating an object. Choosing an architecture means choosing a bias that fits the data.

Appropriate inductive biases reduce parameter counts (enabling edge deployment), accelerate convergence (reducing training costs), and produce structured computation patterns that map efficiently to specialized hardware. The MNIST comparison earned its per-detector saving by matching architecture to data structure, not by shrinking the network. Understanding where dense connectivity remains necessary requires identifying the data patterns that lack structural shortcuts.

[6] **No free lunch theorem**: Wolpert and Macready's 1997 result proved that no optimization algorithm outperforms random search across all possible problems: averaged over every conceivable function, all algorithms are equivalent. The ML systems consequence is that an inductive bias (locality, equivariance, attention) can improve performance on problems matching that bias while hurting performance when its assumptions do not hold, making architecture selection an engineering commitment to a problem class.

6.2.2 Pattern processing needs

In a tabular dataset such as customer records or a credit risk model, any input column (income, account age, or recent transaction count) can interact with any other, and the columns have no spatial neighbors or time order. Because the data carries no grid or sequence structure, the network cannot restrict computation to local windows or sequential steps. Every output neuron must be free to combine every input feature, leaving gradient descent to learn which connections matter.

When an MLP processes an image dataset like MNIST, it flattens each 28×28 image into an unstructured 784-element vector. Flattening throws away the two-dimensional grid, so the layer treats adjacent pixels and pixels on opposite corners identically. While this lets every hidden unit relate any pixel to any class, the network must rediscover spatial adjacency from scratch during training. That all-to-all requirement leads directly to the matrix operations that define the MLP.

6.2.3 Algorithmic structure

Letting every input feature interact with every output unit requires complete connectivity between adjacent layers. In an MLP, this takes the form of a sequence of fully connected layers where every neuron connects to every neuron in the preceding layer. Dense connectivity translates directly into matrix multiplication, the formulation that makes dense layers executable on parallel hardware. The fully connected graph on the left of figure 6.2 becomes the matrix-vector product on its right: each

hidden neuron's activation is the dot product of the input with one entire column of the weight matrix.

↩ The matrix multiplication formulation of a dense layer's forward pass is established in section 5.4.2.2.

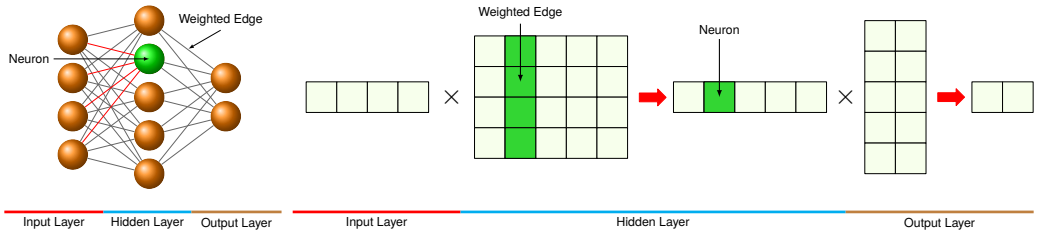


Figure 6.2: Multilayer Perceptron Architecture: A three-layer network in which every neuron connects to all neurons in adjacent layers. The highlighted neuron receives weighted contributions from all inputs, illustrating the dense $\mathcal{O}(d_{\text{in}} \times d_{\text{out}})$ connectivity pattern implemented through matrix multiplications. For MNIST classification, a 784-dimensional input ($d_{\text{in}} = 784$) connects to 128 hidden neurons ($d_{\text{out}} = 128$) through a 784×128 weight matrix, requiring 100,352 multiply-accumulate operations per sample. Adapted from Reagen et al. (2017).

Equation 6.1 expresses the dense layer's affine transformation followed by its activation.

$$\mathbf{h}^{(\ell)} = f(\mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \tag{6.1}$$

$\mathbf{h}^{(\ell)}$ represents the layer ℓ output, $\mathbf{h}^{(\ell-1)}$ represents the input from the previous layer, $\mathbf{W}^{(\ell)}$ denotes the weight matrix for layer ℓ, $\mathbf{b}^{(\ell)}$ denotes the bias vector, and $f(\cdot)$ denotes the activation function. The dimensions of these operations reveal the computational scale of dense pattern processing. The input vector $\mathbf{h}^{(0)} \in \mathbb{R}^{d_{\text{in}}}$ (treated as a row vector in this formulation) represents all potential input features. Weight matrices $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ capture all possible input-output relationships. The output vector $\mathbf{h}^{(\ell)} \in \mathbb{R}^{d_{\text{out}}}$ produces transformed representations. A four-pixel example turns this bookkeeping into arithmetic.

Example 6.2: Concrete computation example

Consider a simplified 4-pixel image processed by a 3-neuron hidden layer:

Input: $\mathbf{h}^{(0)} = [0.8, 0.2, 0.9, 0.1]$ (4 pixel intensities)

Weight matrix:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & 0.1 & -0.2 \\ -0.3 & 0.8 & 0.4 \\ 0.2 & -0.4 & 0.6 \\ 0.7 & 0.3 & -0.1 \end{bmatrix} \quad (4 \times 3 \text{ matrix})$$

Computation:

$$\mathbf{z}^{(1)T} = (\mathbf{h}^{(0)}\mathbf{W}^{(1)})^T = \begin{bmatrix} 0.5 \times 0.8 + (-0.3) \times 0.2 + 0.2 \times 0.9 + 0.7 \times 0.1 \\ 0.1 \times 0.8 + 0.8 \times 0.2 + (-0.4) \times 0.9 + 0.3 \times 0.1 \\ (-0.2) \times 0.8 + 0.4 \times 0.2 + 0.6 \times 0.9 + (-0.1) \times 0.1 \end{bmatrix} = \begin{bmatrix} 0.59 \\ -0.09 \\ 0.45 \end{bmatrix}$$

After ReLU: $\mathbf{h}^{(1)} = [0.59, 0, 0.45]$ (negative values zeroed)

Systems insight: Each hidden neuron combines all input pixels with different weights, demonstrating unrestricted feature interaction. Dense layers buy generality by paying the maximum connectivity cost.

The MNIST example makes this scale concrete. The 784-dimensional input connects to every one of the 128 neurons in the first hidden layer of the 784-128-64-10 network from chapter 5, a 784×128 weight matrix of 100,352 parameters, where each weight represents a learnable relationship between an input pixel and a hidden feature. This layer is the reference MNIST layer for the computational analysis in the rest of this chapter.

The matrix form specifies every input-output interaction, but it leaves the execution schedule open. Evaluating one sample at a time streams weights with little reuse; grouping samples lets several outputs share each fetched weight. The loop structure makes these alternatives visible.

6.2.4 Computational mapping

The layer equation fixes what a dense layer computes, not the order in which a processor computes it. At the framework level the whole layer is a single matrix call (listing 6.1).

Listing 6.1: MLP Layer in Matrix Form: Framework-level matrix operations hide $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ multiply-accumulate complexity behind a single function call, enabling hardware-optimized BLAS libraries to use dense matrix hardware efficiently.

```
def mlp_layer_matrix(X, W, b):
    """MLP forward pass using framework-level matrix operations."""
    # X: input matrix (batch_size by num_inputs)
    # W: weight matrix (num_inputs by num_outputs)
    # b: bias vector (num_outputs)

    # Single GEMM call: frameworks dispatch to optimized BLAS/cuBLAS
    # For MNIST: 784 * 128 = 100,352 MACs per sample
    H = activation(matmul(X, W) + b)
    return H
```

The function `mlp_layer_matrix` mirrors equation 6.1 directly, using high-level matrix primitives to express the forward transformation while delegating tile scheduling and execution to framework backends. Beneath this software interface, dense matrix multiplication lowers to a canonical three-deep loop nest that determines memory access locality, parallelization boundaries, and register allocation (listing 6.2). The outermost loop iterates across batch samples, the middle loop iterates over output neurons, and the innermost loop executes repeated multiply-accumulate (MAC) operations[7] that pair each input feature with its weight.

Listing 6.2: Dense Layer Computation: Nested loops reveal $\mathcal{O}(B \times d_{\text{out}} \times d_{\text{in}})$ complexity where each output neuron requires num_inputs multiply-accumulate operations, so the reference MNIST input layer requires 100,352 MACs per sample.

```
def mlp_layer_compute(X, W, b):
    """Explicit loop structure exposing MLP computational patterns."""
    # Loop 1: Process each sample independently (parallelizable)
    for batch in range(batch_size):
        # Loop 2: Compute each output neuron
        for out in range(num_outputs):
            Z[batch, out] = b[out]  # Initialize with bias

            # Loop 3: Accumulate weighted inputs (innermost loop)
            # This is the MAC operation: result += input * weight
            for in_ in range(num_inputs):
                Z[batch, out] += X[batch, in_] * W[in_, out]
            # Total per output: num_inputs MACs +
            # num_inputs memory reads

    H = activation(Z)  # Element-wise nonlinearity
    return H
```

In the reference MNIST layer, computing 128 hidden neurons requires 100,352 MACs in total; each individual output neuron requires 784 multiply-accumulate operations and 1,568 logical operand reads (784 for inputs, 784 for weights). Registers and caches can serve repeated reads, so this count is not a DRAM-traffic estimate. Production implementations call optimized matrix libraries such as Basic Linear Algebra Subprograms (BLAS),[8] but the same nested-loop pattern still determines the system design problem. The hardware architectures that accelerate these matrix operations, including GPU Tensor Cores[9] and specialized AI accelerators, are covered in section 11.3.2.2.

[7] **MAC (multiply-accumulate)**: The atomic operation of neural networks: multiply two values and add to a running sum. In Horowitz's 45 nm reference, an FP32 multiply plus add costs about 4.6 pJ, while a 32-bit off-chip DRAM access costs about 640 pJ (Horowitz 2014). These technology-specific values illustrate why data movement can dominate arithmetic energy; they are not universal constants for current hardware.

[8] **BLAS (basic linear algebra subprograms)**: This standard API for matrix operations enables the use of highly optimized libraries (for example, cuBLAS) to accelerate the 784 multiply-accumulates per neuron. The 784×128 matrix in the MNIST example may use the hardware less efficiently than larger, well-aligned transformer matrices because utilization depends on matrix shape, batching, datatype, library, and accelerator.

[9] **Tensor Cores**: Specialized units in NVIDIA GPUs that accelerate the thousands of multiply-accumulate operations described by fusing them into single, highly parallelized matrix instructions. Tensor Cores are most efficient when matrix dimensions meet datatype- and architecture-specific alignment multiples; modern cuBLAS/cuDNN can still use Tensor Cores for many nonaligned cases, often with lower efficiency or internal padding. The architectural lesson is vendor-independent: specialized matrix units reward dense, aligned general matrix multiplication (GEMM) workloads and penalize small, irregular shapes that cannot keep the units full.

6.2.5 System implications

While the reference MNIST layer (784×128) needs only 100,352 parameters, a single dense layer connecting 2048 inputs to 2048 outputs requires 4,194,304 parameters (16.8 MB at FP32), and evaluating one sample uses each weight only once. Because every input connects to every output, $\mathcal{O}(d_{\text{in}} \times d_{\text{out}})$ weight storage dominates the memory footprint, and when those weights exceed on-chip cache and stream from DRAM on every request, memory bandwidth becomes the bottleneck. Dense layers therefore concentrate hardware pressure across three dimensions: memory capacity for weight storage, compute throughput on matrix engines, and memory bandwidth for weight movement.

The core computation is a dense matrix-vector product (GEMV), or a matrix-matrix product (GEMM) when batched. This computation is regular and parallelizable, but the arithmetic intensity (FLOP/byte) is low for small batches. A larger batch size amortizes weight-loading cost over more computations. Modern processors optimize dense layers with single instruction, multiple data (SIMD) units such as AVX-512 on CPUs or with dense matrix engines such as Tensor Cores on GPUs and systolic arrays[10] on Tensor Processing Units (TPUs) (Jouppi et al. 2017), amortizing control overhead over large blocks of parallel arithmetic.

Section 5.4.2.4 measured this weight-loading cost on the running network, where at batch one each weight read serves a single multiply-accumulate. For the reference layer, which reads 784×128 weights to compute 128 hidden values, a weight-only model that counts one FP32 read per weight and omits input, output, bias, and cache traffic gives roughly 0.5 FLOP/byte at batch one. Equation 6.2 restates that ratio, where d_{in} and d_{out} are the input and output widths. The memory tier serving those reads determines the relevant bandwidth.

$$I \approx \frac{2 \cdot d_{\text{in}} \cdot d_{\text{out}} \text{ FLOPs}}{4 \cdot d_{\text{in}} \cdot d_{\text{out}} \text{ bytes}} = 0.5 \text{ FLOP/byte} \tag{6.2}$$

If these weights stream from off-chip memory, a machine with an off-chip ridge point in the hundreds of FLOP/byte cannot supply them fast enough to reach peak arithmetic throughput. The small MNIST layer may instead remain cached across requests; its traffic must then be compared with the bandwidth of that cache, and dispatch overhead may matter more than off-chip bandwidth. Batching raises the weight-bound intensity in proportion to the batch size until input and activation traffic begin to dominate, but where it crosses the machine's balance depends on matrix shape, precision, implementation, and hardware. This is why fully connected layers can become inference bottlenecks despite performing fewer total FLOPs than convolutional layers.

The same intensity sets the energy per inference. With the energy constants introduced alongside the iron law (Horowitz 2014), about 160 pJ per byte read from DRAM and 4.6 pJ per FP32 multiply-accumulate, a batch-one dense layer that streams its weights spends about 139× more energy moving each weight than computing with it. Batching divides that ratio by the batch size until activation traffic takes over, so the knob that raises throughput also cuts energy per inference.

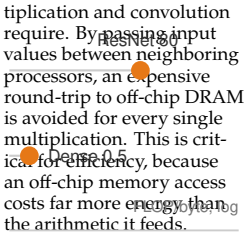
Because dense connectivity offers little weight reuse within a single-example forward pass, applying MLPs to high-dimensional structured signals forces an undesirable trade-off between parameter explosion and memory starvation. When data possesses inherent physical organization (spatial coordinates in images, sequential order in time series), specialized architectures replace all-to-all connectivity with structural priors that simultaneously raise accuracy and computational efficiency. The most established of these spatial architectures is the convolutional neural network.

6.3 CNNs: Spatial Pattern Processing

The MLP's assumption that every input may interact directly with every output proves particularly costly for spatially structured data like images. Connecting every pixel of a high-resolution input to every hidden unit creates a parameter footprint that exhausts accelerator memory and drives arithmetic intensity so low that the layer becomes bandwidth bound. The earlier MNIST comparison showed the remedy at small scale: a shared filter needs 87× fewer weights per feature detector than a dense unit that treats every pixel independently.

CNNs[11] build that remedy into the network itself (Fukushima 1980; LeCun et al. 1998; Krizhevsky et al. 2012). Rather than materializing an all-to-all connectivity matrix, a CNN restricts each output activation to a local spatial patch and slides identical filter weights across the entire input aperture.

[10] **Systolic array**: Named for the heart's rhythmic contraction (Kung 1982), the array's lockstep "pulse" of data through a grid of multiply-accumulate processors directly implements the data reuse that matrix multiplication and convolution require. By passing input values between neighboring processors, an expensive round-trip to off-chip DRAM is avoided for every single multiplication. This is critical for efficiency, because an off-chip memory access costs far more energy than the arithmetic it feeds.



A batch-one dense layer sits far below convolution's weight reuse.

[11] **Convolution**: From Latin *convolvere* ("to roll together"), describing a filter that slides across an input, combining local elements at each position. This "rolling together" enforces a locality constraint that is the source of the operation's efficiency: a single 5×5 kernel reuses its 25 weights at every spatial position instead of learning a separate weight for every pixel.



Weight sharing spares convolution the fully-connected parameter explosion.

This spatial inductive bias translates into two structural constraints on the computation graph. Parameter sharing applies the same small filter weights across every spatial position, reducing parameter storage from millions of distinct weights to a compact kernel that fits within on-chip caches. Local connectivity limits each neuron's receptive field to a spatially adjacent neighborhood, eliminating computation on distant, uncorrelated pixels. Together, these constraints define the convolutional family.

Definition 6.3: Convolutional neural networks

Convolutional neural networks (CNNs) are architectures that exploit translation equivariance and spatial locality to share learned filters across all spatial positions, decoupling parameter count from input resolution.

1. **Significance**: Weight sharing produces dramatic parameter reduction. A 3×3 convolutional layer with 64 input and 64 output channels requires $3 \times 3 \times 64 \times 64 \approx 37{,}000$ parameters regardless of whether the input image is 224×224 or 1024×1024. A fully connected layer mapping a $224\times224\times64$ input to 64 scalar outputs would require $224^2 \times 64 \times 64 \approx 205$ million parameters, a roughly 5,575$\times$ difference. These output shapes differ: convolution reuses its 64 detectors across a spatial feature map, while the dense layer produces 64 scalars. Weight sharing keeps the filter budget constant as resolution grows, although feature-map storage still grows with image area.
2. **Distinction**: Unlike MLPs, which connect every input element to every output element (global connectivity), CNNs restrict each output to a local spatial neighborhood, encoding the assumption that nearby pixels are more relevant than distant ones. This restriction eliminates entire hypothesis classes at architecture design time rather than penalizing them during training.
3. **Common pitfall**: A frequent misconception is that CNNs are vision-only models. The convolution operation applies to any data with a grid-like topology: 1D convolutions process audio waveforms and time series, 2D convolutions process images and spectrograms, and 3D convolutions process video and volumetric data.

CNNs give up the all-to-all generality of MLPs for practical efficiency whenever data has local geometry. Seeing why that trade-off works on images begins with the spatial structure of a photograph.

6.3.1 Pattern processing needs

In a photograph, neighboring pixels carry closely related brightness and color values that together form local edges, corners, and textures, whereas pixels on opposite sides of the frame have almost nothing in common on their own. Stacking local filters lets a network build larger features out of smaller ones: early layers combine nearby pixels into edges, middle layers combine edges into contours and surface textures, and deeper layers assemble those patches into object parts and whole objects. From stage to stage the spatial grid shrinks while the number of channels grows, so each later feature summarizes a wider patch of the image, until the features flatten into one vector for a dense classifier (figure 6.3).

This hierarchical composition appears across multiple physical domains: localized pixel neighborhoods forming contours in computer vision, local time-frequency clusters identifying phonemes in speech spectrograms, and contiguous voxel blocks isolating tissue boundaries in volumetric tomography. The architecture works because images compose from local parts, whether or not it resembles biological vision.

In image processing, recognizing an object requires two distinct computational guarantees. First, the architecture must capture local geometric primitives over bounded receptive fields. Second, because an object retains its semantic identity regardless of where it appears in the frame, the network must identify those primitives across arbitrary coordinate translations. Convolutional networks satisfy both requirements by combining localized kernels with parameter sharing, enforcing translation equivariance at intermediate stages while pooling enables downstream invariance

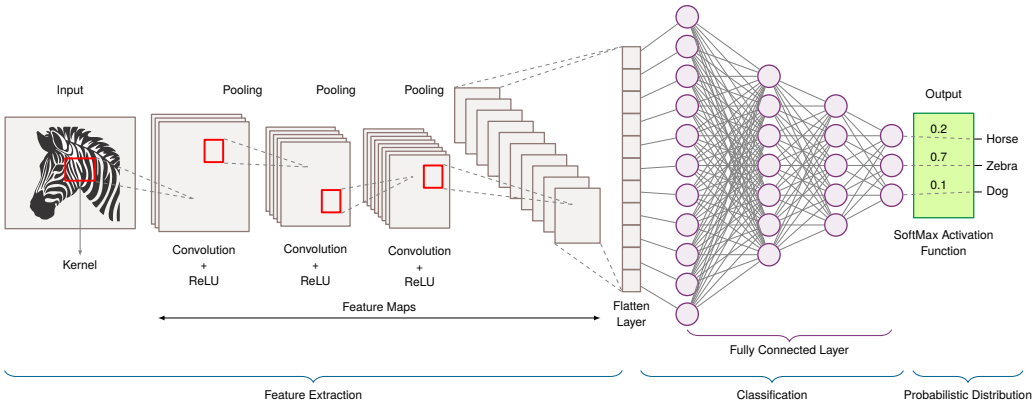


Figure 6.3: Spatial Feature Extraction: A zebra image passes through repeated convolution, ReLU, and pooling stages that form progressively deeper feature maps, followed by flattening, fully connected layers, and a softmax classification over Horse, Zebra, and Dog.

(LeCun, Boser, et al. 1989; Krizhevsky et al. 2012).[12] These architectural constraints, introduced by LeCun[13] (LeCun, Boser, et al. 1989), replace unconstrained dense matrix multiplications with structured spatial stencils.

6.3.2 Algorithmic structure

A convolutional layer's weights form a four-dimensional tensor, and the reason is easiest to see by building one output feature in two stages, spatial filtering followed by channel accumulation. Consider a single RGB image with three input channels ($C_{\text{in}} = 3$). A single output feature (such as an edge detector) requires its own dedicated $K \times K$ spatial filter for the Red channel, another for Green, and a third for Blue:

1. **Spatial filtering**: Each 2D filter slice slides across its corresponding color channel, computing multiply-accumulate dot products over local spatial windows as in a classic image filter.
2. **Channel accumulation**: At each spatial coordinate (i, j), the dot-product outputs of all three input channels sum into a single scalar value.

This two-step process produces exactly one 2D output feature map. If the layer demands $C_{\text{out}} = 64$ distinct features, the system repeats this procedure 64 times using 64 independent filter volumes. Thus, the weight tensor naturally requires four physical dimensions: $\mathbf{W} \in \mathbb{R}^{K \times K \times C_{\text{in}} \times C_{\text{out}}}$ (Kernel Height $\times$ Kernel Width $\times$ Input Channels $\times$ Output Channels).

In formal terms, equation 6.3 evaluates this forward pass by computing local, channel-wise dot products across every spatial coordinate:

$$H^{(\ell)}_{i,j,k} = \phi\left(\sum_m \sum_n \sum_c W^{(\ell)}_{m,n,c,k} H^{(\ell-1)}_{i+m,j+n,c} + b^{(\ell)}_k\right) \tag{6.3}$$

Here, $H^{(\ell)}_{i,j,k}$ denotes the activation at spatial position (i, j) in output channel k of layer ℓ. The triple summation iterates across the filter dimensions: spatial offsets (m, n) span the local kernel window (defining the layer's local receptive field),[14] while c ranges over all input channels. The entries $W^{(\ell)}_{m,n,c,k}$ of the weight tensor store the kernel weights connecting channel c to channel k, and $b^{(\ell)}_k$ is the per-channel bias. Because the kernel weights $\mathbf{W}^{(\ell)}$ do not depend on the spatial coordinates (i, j), the same filter parameters are reused at every coordinate.

Convolutional layers typically use small spatial kernels (3×3 or 5×5) to maintain compact parameter footprints while stacking layers to widen the effective receptive field. Each output activation depends on one small window of the input. In figure 6.4, a 3×3 receptive field in a 7×7 input produces one value of the 5×5 output map, and sliding the same filter to every position fills in the rest. The interactive CNN Explainer tool (Wang et al. 2021) visualizes how these stacked sliding windows extract per-channel activation patterns across layers.

[12] **Translation equivariance**: An inherent property of the convolution operation where shifting the input guarantees a corresponding spatial shift in the resulting feature map. This is distinct from *invariance*, which pooling or later aggregation can approximate by discarding precise positional data. For example, 2×2 pooling with stride 2 reduces the number of spatial elements by 75 percent.

[13] **Yann LeCun and LeNet**: LeCun's architecture directly addressed the intractable scaling of applying dense networks to images by enforcing the principles of local connectivity and parameter sharing. These constraints reduced the parameter count for an image-like input layer by over 95 percent, enabling LeNet-5 to achieve production-grade accuracy on commercial tasks like check reading with only ~60,000 total parameters.

[14] **Receptive field**: The input region influencing a particular output neuron. With 3×3 filters, receptive fields grow by 2 pixels per layer, so a neuron at layer 3 "sees" a 7×7 region. This growth rate constrains architecture depth: detecting objects spanning 100+ pixels in a 224×224 image requires either deep stacks of small filters (more layers, more memory for activations) or larger kernels (more parameters per layer), a fundamental depth-vs.-width trade-off in CNN design.

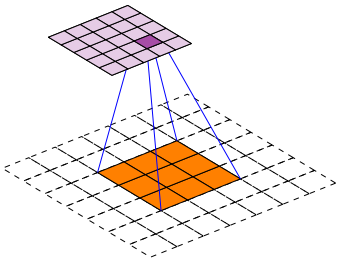


Figure 6.4: Convolution Operation: A 3×3 orange receptive field in a dashed 7×7 input maps to one cell of the violet 5×5 valid-convolution output. Applying a 3×3 filter to that receptive field requires 9 multiply-accumulate operations per input channel for each output value. Sharing the filter across positions encodes translation equivariance: the same edge detector works at any image location.

Applying a CNN to 28×28 MNIST images demonstrates the difference in state representation. Rather than flattening the image into a 784-element vector that destroys spatial adjacency, each convolutional layer applies a bank of filters (for example, 32 filters of size 3×3) directly to the 2D grid. The *stride* is how many positions the filter moves between evaluations, and *padding* adds a border of zeros so that a filter centered on an edge pixel still has a full window. With unit stride and padding to preserve border dimensions, the layer outputs a $28\times28\times32$ activation tensor preserving the spatial grid. Each coordinate holds 32 distinct feature measurements extracted from its immediate neighborhood. Between groups of convolutional layers, a *pooling* layer summarizes each small window, typically 2×2, by its maximum or average, shrinking the grid by a factor of four in area while keeping the channels, so later layers see a coarser map that covers more of the image at a quarter of the activation memory.

Preserving spatial topology changes how execution units access memory. Instead of loading an unshared weight matrix once per sample as in an MLP, a convolutional layer streams input feature maps through hardware registers while keeping compact filter weights pinned in fast on-chip SRAM or register files. This arithmetic pattern drives the memory-hierarchy optimizations of AI accelerators, where loop tiling (splitting the loops into blocks whose data fits on chip, developed under data movement below) and parallel channel reduction relieve high-bandwidth memory (HBM) traffic.

Weight sharing is what guarantees this reuse. Because the same filter applies at every coordinate, the computational schedule and memory access geometry are identical across spatial positions. The same sharing gives the layer a mathematical property, translation equivariance: translating the input produces an identical spatial translation in the output feature map.

Equivariance and invariance govern how an architecture responds to spatial transformations. An operator f is *equivariant* to a transformation $\mathcal{T}$ if transforming the input transforms the output by the same operator (equation 6.4):

$$f(\mathcal{T}(\mathbf{x})) = \mathcal{T}(f(\mathbf{x})) \tag{6.4}$$

For a stride-1 convolutional layer without boundary padding effects, translating the input by vector $\mathbf{v} = (v_1, v_2)$ shifts the resulting feature maps by exactly $\mathbf{v}$. Spatial coordinate information propagates deterministically through the layer. By contrast, an operator is *invariant* to $\mathcal{T}$ if transforming the input leaves the output unchanged (equation 6.5):

$$f(\mathcal{T}(\mathbf{x})) = f(\mathbf{x}) \tag{6.5}$$

A global average pooling layer over an entire feature map is translation-invariant: shifting the input alters the spatial locations of activations, but averaging across all spatial positions collapses coordinates into a scalar that is identical regardless of where the features appeared. This mathematical distinction dictates where equivariance must be maintained and where invariance should be introduced across the network hierarchy. Tasks such as object detection and semantic segmentation require equivariance through the final layer: bounding-box coordinates and per-pixel masks must track object coordinates precisely. Classification tasks, by contrast, require equivariance in intermediate feature extractors to preserve relative spatial relations (“eyes above nose”), while the

final layers introduce translation invariance via pooling to collapse spatial coordinates into a class probability vector.

Re-indexing the convolution sum under a shift confirms that stride-1 convolution is translation equivariant up to boundary conditions. An equivariant convolutional layer preserves coordinate information, whereas an invariant pooling layer discards spatial coordinates entirely. Example 6.3 verifies this tracking through explicit matrix operations.

Example 6.3: Equivariance: Feature detection

Setup: Consider a 7×7 image with a vertical edge at column 3:

$$\mathbf{x} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Vertical edge detector filter:

$$\mathbf{W} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Convolving original image:
Output feature map shows positive activation where the filter transitions from dark to bright (left side of edge) and negative activation where it transitions from bright to dark (right side):

$$f(\mathbf{x}) = \begin{bmatrix} 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \end{bmatrix}$$

Shifted input (edge moved to column 5):

$$\mathcal{T}_2\mathbf{x} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Convolving shifted image:

$$f(\mathcal{T}_2\mathbf{x}) = \begin{bmatrix} 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \end{bmatrix} = \mathcal{T}_2(f(\mathbf{x}))$$

Systems insight: The feature activation shifts by the same amount as the input, demonstrating equivariance. The network knows the edge is at column 5 in the shifted image, not just that an edge exists somewhere.

Parameter efficiency is the most immediate systems consequence of the weight sharing that produces translation equivariance, because sharing decouples parameter capacity from the input resolution. For example, processing a 224×224 RGB image with an MLP requires each hidden neuron to connect to all 150,528 input pixels. A CNN with a 3×3 filter requires only 27 parameters per filter, reused across all 224×224 spatial positions. A compact parameter footprint lets the weights stay resident in accelerator SRAM or L2 cache rather than streaming from external DRAM on every use.

Equivariance also increases sample efficiency, reducing training pipeline overhead. When a network updates a filter to recognize a feature at one coordinate, parameter sharing ensures that the feature is recognized across all spatial coordinates simultaneously. The model does not require separate training examples to learn the same feature at different pixel offsets, functioning as structural data augmentation. This sample efficiency reduces the total training iterations, dataset storage, and host-to-device I/O bandwidth required to reach target accuracy.

Convolution's translation equivariance is one instance of a general principle: data symmetries can be embedded as equivariances in the architecture. Group-equivariant CNNs (Cohen and Welling 2016) extend the construction to rotations and reflections for satellite and medical imagery, where orientation does not determine meaning. For systems engineering, stronger symmetry constraints often produce smaller models, but specialized equivariances may require custom operations, such as rotation convolutions, that need either hardware support or efficient software implementations.

Stacked convolutional layers implement hierarchical representation learning (Bengio, Courville, et al. 2013; Zeiler and Fergus 2014). Early layers detect low-level features (edges and color gradients) across small receptive fields, while successive layers compose these primitives into intermediate textures, object parts, and semantic categories. Each added layer also widens the effective receptive field: with fixed $K\times K$ kernels and unit stride, receptive-field side length grows by $K-1$ pixels per layer, while receptive-field area scales quadratically until it spans the entire input dimension. How efficiently accelerators execute this expanding hierarchy depends on how sliding-window stencils map onto parallel execution pipelines.

6.3.3 Computational mapping

A framework expresses a whole convolutional layer as one call (listing 6.3). Behind that call, the underlying runtime must schedule memory traffic across cache hierarchies and lower sliding-window stencils onto matrix execution hardware.

Listing 6.3: Convolutional Layer Abstraction: Framework-level convolution operations hide the complexity of sliding window computations, typically dispatching to cuDNN or MKL which internally handle im2col transformations or direct convolution algorithms.

```
def conv_layer_spatial(input, kernel, bias):
    """Framework-level convolution.

    Single call dispatches to optimized kernel (often via im2col + GEMM).
    """
    # Convolution applies shared weights across all positions
    # For a 3x3 kernel on 28x28 input (padded): 9 MACs per position x 784 positions
    output = convolution(input, kernel) + bias
    return activation(output)
```

To expose the mechanics that the framework abstraction conceals, listing 6.4 details the canonical seven-deep loop nest required to evaluate a direct 2D convolution. These loops partition into three structural tiers: batch iteration across samples, spatial traversal over feature map coordinates (y, x), and channel accumulation over input and output dimensions $(C_{\text{in}}, C_{\text{out}})$ coupled with kernel offsets (k_y, k_x). Traversing spatial coordinates and output channels defines the grid of independent output

activations. The listing uses unit stride and no padding, with channels stored contiguously at each spatial position (NHWC: batch, height, width, channels). Each output window stays within the input, so the output height and width are the corresponding input dimensions minus the kernel dimensions plus one.

Listing 6.4: Logical Convolution Computation: Seven nested loops expose $\mathcal{O}(B \times H_{\text{img}} \times W_{\text{img}} \times C_{\text{out}} \times K \times K \times C_{\text{in}})$ complexity. The spatial dimensions here count output positions. The scalar order exposes overlapping input reads; tiled implementations exploit this reuse directly or through matrix multiplication.

```
def conv_layer_compute(input, kernel, bias):
    # NHWC input; kernel axes: height, width, input, output channels.
    # Unit stride, no padding; activation follows the convolution.
    batch_size, in_h, in_w, in_channels = input.shape
    kernel_h, kernel_w, _, out_channels = kernel.shape
    out_h, out_w = in_h - kernel_h + 1, in_w - kernel_w + 1
    output = zeros((batch_size, out_h, out_w, out_channels))
    for image in range(batch_size):
        for y in range(out_h):
            for x in range(out_w):
                for out in range(out_channels):
                    result = bias[out]
                    for ky in range(kernel_h):
                        for kx in range(kernel_w):
                            for ch in range(in_channels):
                                result += (input[image, y + ky, x + kx, ch]
                                           * kernel[ky, kx, ch, out])
                    output[image, y, x, out] = result
    return activation(output)
```

The inner three loops execute the kernel dot product at each spatial site. For a 3×3 filter, each output activation accumulates contributions from a local input patch across all input channels. Advancing `x` by one makes the next output read a patch overlapping the previous one, while changing `out` reuses the same patch with a different filter. These repeated accesses offer reuse, but the scalar schedule does not explicitly keep tiles in fast memory or distribute work across execution lanes. Their actual cost depends on which reads the cache serves and how the implementation maps iterations to hardware.

To expose these patches to optimized matrix engines, convolution implementations can use the **im2col** (image-to-column) transformation.[15] By unfolding each receptive-field patch into a contiguous row of an input matrix and stacking filter weights into columns (figure 6.5 lays this out for two input channels), im2col recasts the entire convolution as a dense general matrix multiply (GEMM) that runs on mature, vendor-tuned libraries. To see how the lowering rearranges data in memory, consider a miniature single-channel image ($C_{\text{in}} = 1$) of size 3×3 convolved with a 2×2 filter ($K = 2$) with valid padding:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad \mathbf{W} = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix}$$

The window visits four overlapping 2×2 receptive fields. To evaluate all four in one matrix multiplication, `im2col` flattens each patch into a row vector of length $K^2 = 4$ and stacks the rows into an unfolded matrix $\mathbf{X}_{\text{col}}$:

$$\mathbf{X}_{\text{col}} = \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix}$$

[15] **im2col (image to column)**: Rather than being a new learning algorithm, im2col-style lowering is an implementation technique used by CNN frameworks and libraries such as Caffe and cuDNN (Chellapilla et al. 2006; Jia et al. 2014; Chetlur et al. 2014): it converts convolutions into standard GEMM calls by unfolding overlapping patches into matrix columns. The trade-off is memory: in a simple fully materialized stride-1 $K \times K$ transform, interior input elements can appear in up to K^2 columns (9 times for 3×3 filters), though borders, stride, padding, tiling, and direct-convolution algorithms reduce the realized expansion. This memory-for-simplicity exchange explains why mobile frameworks (TFLite, NNAPI) prefer direct convolution, while data center GPUs with abundant HBM may use GEMM-oriented lowering when it improves throughput.

Flattening the filter weights into a column vector $\mathbf{w}_{\text{col}} = [w_1, w_2, w_3, w_4]^T$, the entire convolution collapses into a single standard matrix product: $\mathbf{Y}_{\text{col}} = \mathbf{X}_{\text{col}}\mathbf{w}_{\text{col}}$. For batched multi-channel inputs, the unfolded matrix has shape $[(B \cdot H_{\text{out}} \cdot W_{\text{out}}) \times (K^2 \cdot C_{\text{in}})]$ and the filters reshape to $[(K^2 \cdot C_{\text{in}}) \times C_{\text{out}}]$. In $\mathbf{X}_{\text{col}}$, input element 5 appears in all four rows, and the miniature input expands from nine elements to sixteen, a storage ratio of $16/9$. For large stride-1 inputs, fully materialized unfolding approaches the kernel area as its expansion factor; the smaller ratio here reflects the borders of a valid convolution. Materializing the buffer trades this additional storage and traffic for access to wide matrix engines optimized for GEMM throughput. Implicit GEMM, described next, avoids allocating the duplicated buffer.

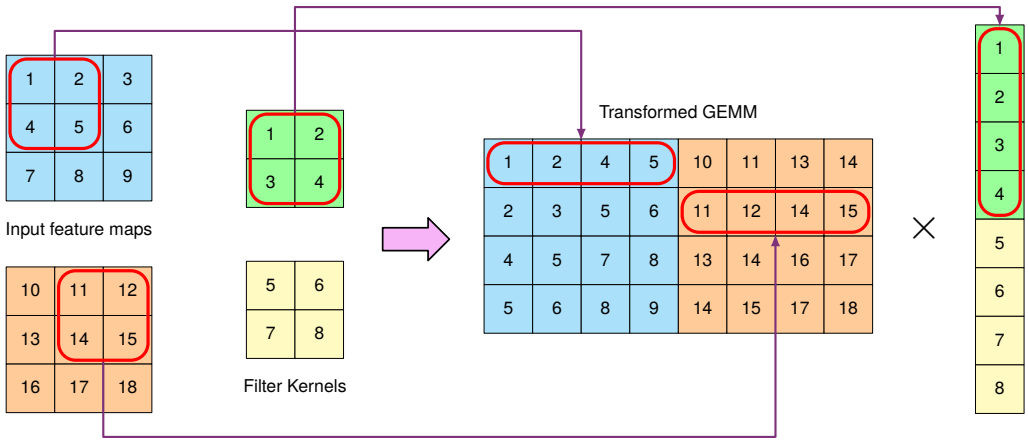


Figure 6.5: im2col Transformation: Lowering converts convolution to GEMM by unfolding image patches into a dense matrix. The two 3×3 input feature maps produce four sliding-window positions, represented here as rows of a 4×8 matrix; the two 2×2 filter kernels stack into an 8×1 column. Their product, not drawn, yields the four output positions. Lowering can exploit optimized matrix kernels but expands memory by an amount that depends on kernel size, stride, padding, and implementation, so its performance benefit is workload dependent.

Implicit GEMM techniques run the same matrix multiplication without building the unfolded matrix. They compute each patch element's address on the fly, inside the processor's registers, so the duplicated buffer is never materialized. With 3×3 filters and 32 output channels, each output coordinate requires only 9 MAC operations per input channel, whereas each output neuron of the reference MLP layer requires 784. The window overlap creates reuse that the memory system must be scheduled to exploit.

6.3.4 System implications

Weight sharing flips the hardware bottleneck of a neural network: compact convolutional filters fit in on-chip cache and are reused across thousands of pixel positions, pushing arithmetic intensity into the compute-bound regime, while the intermediate feature maps they write out dominate off-chip memory capacity and bandwidth. Whether a deep CNN stalls on memory or saturates the matrix units depends on layer depth, how long activations stay live for the backward pass, and whether feature-map tiles remain in on-chip SRAM or spill to external HBM.

6.3.4.1 Memory requirements

On a 224×224 ImageNet input, a convolutional layer with 64 filters of size 3×3 needs only 576 weights per input channel, or 36,864 weights (147.5 KB in FP32) across 64 input channels, which fits comfortably in on-chip L1 or L2 cache, yet produces 3.2M output activation values (about 12.8 MB in FP32 per sample in the batch). Memory consumption in convolutional layers splits sharply between these compact, static filter weights and large, transient feature maps. During training, those feature maps must stay live in memory until backpropagation computes the layer's gradients, making activation memory, not weight capacity, the binding memory constraint for deep CNNs on accelerators.

Accelerators exploit this footprint disparity through hierarchical caching. CPUs retain compact filter weights in L2/L3 caches while streaming activation tensors. GPUs stage filter weights and input tiles in on-chip scratchpad SRAM (shared memory) and register files, reusing loaded weights across thousands of spatial coordinates and eliminating redundant round-trips to off-chip HBM.

↪ The memory hierarchy of register files, on-chip SRAM, and HBM that these caching patterns exploit is developed in section 11.5.3.

6.3.4.2 Compute requirements

Evaluating a convolutional layer requires repeatedly evaluating local dot products across spatial positions. For an ImageNet input with 3×3 filters and 64 output channels, computing a single spatial position requires 576 multiply-accumulate operations per input channel. Summed over 64 input channels and repeated at all 50,176 spatial positions, one such layer performs about 1.85 billion multiply-accumulates. The general count follows the same steps: a layer with $H_{\text{img}}\times W_{\text{img}}$ output positions performs $H_{\text{img}}W_{\text{img}}K^2C_{\text{in}}C_{\text{out}}$ multiply-accumulates, one K^2C_{in} dot product for every output position and every filter. Although each spatial stencil touches fewer parameters than a dense layer, spatial repetition multiplies the total floating-point operation count.

This computational regularity provides rich opportunities for hardware parallelization. Because each spatial coordinate is independent, processors parallelize work along multiple axes simultaneously: across the batch dimension, across spatial coordinates (x, y), and across output channels. CPUs execute vectorized SIMD instructions[16] to evaluate multiple spatial outputs per clock cycle, while GPUs dispatch thread warps across thousands of concurrent execution lanes.

[16] **SIMD (single instruction, multiple data)**: CPU instructions that apply the same operation to multiple data elements simultaneously; AVX-512 can process 16 single-precision values per vector instruction; realized speedup over scalar code also depends on instruction throughput, vectorization, and memory access. For CNN inference on edge CPUs without GPU access, SIMD utilization can affect whether a model meets real-time latency targets. Frameworks like TFLite and Open Neural Network Exchange (ONNX) Runtime use vectorized convolution kernels to exploit this parallelism.

6.3.4.3 Data movement

Data movement in a convolutional layer has an arithmetic intensity profile fundamentally different from that of an MLP. In a fully connected layer at batch size one, each loaded weight is used exactly once, yielding an arithmetic intensity bounded below 1 FLOP/byte. In a 2D convolution, each 3×3 filter weight is reused across all 50,176 positions of the 224×224 feature map. This weight reuse increases operational intensity, shifting computation from the memory-bandwidth-bound regime toward the compute-bound ceiling.

The systems challenge shifts from loading weights to staging activations. Because adjacent spatial receptive fields overlap, a naive memory access pattern would re-fetch each input pixel from memory up to $K^2 = 9$ times. Runtimes avoid this redundant memory traffic by tiling computation into 2D spatial blocks sized to fit inside accelerator SRAM, the technique that section 11.7.5 develops for the accelerator's memory hierarchy. Once an input tile is loaded into shared memory, all overlapping filter dot products for that tile execute in place, amortizing the DRAM/HBM transfer and maximizing the 50,176 reuses of each filter weight. Reuse also moves the energy. Each filter weight fetched from off-chip memory feeds tens of thousands of multiply-accumulates, so once filters and input tiles stay on chip, a convolutional layer spends its energy on arithmetic and on-chip SRAM rather than on DRAM reads, the reverse of the batch-one dense layer.

↪ The hardware ridge point (I_{ridge}), which decides whether spatial filter reuse saturates the arithmetic units or starves on memory bandwidth, is developed in section 11.6.2.

Checkpoint 6.1: Spatial inductive bias

CNNs succeed because they match the structure of image data. Verify you understand how:

- ☐ **Parameter sharing**: how does using the same filter across the image reduce parameter count by orders of magnitude compared to MLPs?
- ☐ **Translation equivariance**: why does shifting the input image result in a corresponding shift in the feature map?
- ☐ **Compute-bound convolution**: how does filter reuse raise arithmetic intensity, and how would comparing that intensity with the target machine's roofline balance establish whether the layer is compute bound?

These memory, compute, and data-movement patterns converge in ResNet-50 (He et al. 2016a), which anchors the chapter's compute-intensive vision lighthouse. The Residual Network (ResNet) addressed the degradation problem in deep plain networks: adding layers could increase training error despite sufficient capacity. By introducing skip connections, shortcut paths that add a block's input directly to its output and so give gradients a route around each block (developed in section 6.4),

it enabled networks of 50, 100, or even 1,000 layers. The ResNet architecture won the ImageNet 2015 competition (with 152-layer models), and ResNet-50 has become a widely used backbone and benchmark workload for computer vision.



Lighthouse 6.1: ResNet-50 (vision lighthouse)

Why it matters: ResNet-50 is a reference point for regular, convolution-heavy vision workloads. Its architecture consists almost entirely of dense convolutional layers, making it highly regular and efficient on GPUs. Under batched execution with good data reuse, ResNet-50 performance is typically limited by floating-point throughput (FLOP/s), making it a useful lighthouse for explaining data parallelism, quantization, and batching strategies. Table 6.2 summarizes the quantitative properties and their system consequences:

Table 6.2: ResNet-50 Systems Profile: Quantitative properties of the vision lighthouse and their system consequences. Under batched convolutional reuse, ResNet-50 has high effective arithmetic intensity and is typically compute-bound, so peak FP throughput on Tensor Cores often sets the ceiling before memory bandwidth does.

Property	Value	System Implication
Parameters	25.6M	102.4 MB model size at FP32; fits comfortably in GPU memory.
FLOPs/Image	8.2 GFLOP (224×224)	3×3 convolutions are the largest single kernel class at roughly 48% of MACs.
Constraint	Compute-heavy when batched	Limited by peak FLOP/s when weight and activation reuse are high; small-batch inference can move toward the memory-bound regime.
Bottleneck	FP Throughput	Benefits maximally from specialized Matrix Units (Tensor Cores).
Profile	High effective arithmetic intensity under reuse	Arithmetic intensity depends on batch size, convolution algorithm, and materialized memory traffic.

6.3.5 Depthwise separable convolutions

ResNet-50's compute-bound profile assumes abundant hardware resources, yet most inference runs on devices with power budgets three orders of magnitude smaller than a data center accelerator. Standard convolutions scale as $\mathcal{O}(N_{\text{pix}} \times K^2 \times C_{\text{in}} \times C_{\text{out}})$, a compute volume that exceeds the strict energy budgets of always-on edge devices. To bridge this gap, efficient architectures like **depthwise separable CNNs (DS-CNN)** decompose the standard convolution into two distinct operations. This factorization, introduced by Sifre in the context of feature extraction (Sifre and Mallat 2014) and popularized by MobileNet (Howard et al. 2017), decouples spatial filtering from cross-channel combination. The depthwise convolution applies filters to each input channel independently ($K \times K \times C_{\text{in}}$ parameters), while the pointwise convolution uses a 1×1 convolution to project channels to the output dimension ($1 \times 1 \times C_{\text{in}} \times C_{\text{out}}$ parameters). Per output position, a standard layer performs $K^2 C_{\text{in}} C_{\text{out}}$ multiply-accumulates, while the factorized pair performs $K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$. Dividing the two shows that the decomposition reduces parameter count and FLOPs to a fraction of roughly $\frac{1}{C_{\text{out}}} + \frac{1}{K^2}$ of standard convolution (a reduction factor approaching K^2, or about 8–9× for 3×3 filters with large C_{out}), making real-time execution feasible on resource-constrained microcontrollers.

MobileNet (Howard et al. 2017) made this factorization the basis of a model family, challenging the trend toward ever-larger networks by designing for the battery and latency budgets of smartphones and embedded devices. MobileNetV2 anchors the chapter's edge-efficiency lighthouse, reaching competitive accuracy at a fraction of ResNet-50's arithmetic, and its small compute footprint made it a natural target for compression and lower-precision deployment, which stores and computes with fewer bits per value.

↪ Compression and lower-precision deployment techniques for compact models such as MobileNet are developed in chapter 10.



Lighthouse 6.2: MobileNetV2 (efficiency lighthouse)

Why it matters: MobileNetV2 represents edge workloads that must meet a latency budget within a power envelope. Its depthwise separable convolutions trade channel-mixing capacity for fewer operations, making it a useful baseline for mobile apps, embedded vision, and neural

architecture search (NAS), automated search over model designs. Table 6.3 summarizes the efficiency lighthouse's quantitative properties:

Table 6.3: MobileNetV2 Systems Profile: Quantitative properties of the efficiency lighthouse and their system consequences. Roughly an order of magnitude smaller than ResNet-50 in both parameters and FLOPs, MobileNetV2's operation shapes and arithmetic intensity make single-image latency sensitive to operator-dispatch overhead and memory access on mobile CPUs.

Property	Value	System Implication
Parameters	3.5M	14 MB at FP32; 7.3× smaller than ResNet-50.
FLOPs/Image	600 MFLOP	13.7× fewer than ResNet-50 for similar accuracy.
Constraint	Latency Target	Single-image inference speed is the priority.
Bottleneck	Overhead/Memory Access	Operator dispatch and memory access can dominate actual compute.
Profile	Low Arithmetic Intensity	Memory access and control logic matter more than peak FLOP/s.

Set side by side, the ResNet-50 and MobileNetV2 profiles show that a model with 13.7× fewer FLOPs does not automatically run 13.7× faster. Whether operational reduction translates into wall-clock speedup depends on arithmetic intensity, operator dispatch overhead, and the hardware's roofline balance. On data center GPUs engineered for high arithmetic intensity, MobileNetV2 often achieves only a modest speedup because its fragmented depthwise operations underutilize Tensor Cores. Architectural efficiency techniques like depthwise separable convolutions and pruning (removing redundant weights or channels) alter this hardware mapping balance, and Perspective 6.1 addresses the central fallacy linking FLOP counts directly to execution latency.

Systems Perspective 6.1: Misconception: FLOPs equal speed

Fallacy: "MobileNetV2 has 13.7× fewer FLOPs than ResNet-50, so it must run 13.7× faster."

Resolution: On high-end GPUs at small batch, MobileNetV2 gains far less than its FLOP advantage suggests, and in some framework configurations runs no faster than ResNet-50. MobileNetV2's depthwise separable convolutions have low arithmetic intensity: they move more data relative to computation. GPUs optimized for dense matrix operations may use their compute units poorly on these kernels. FLOPs measure *work*; throughput depends on how well that work *maps to hardware*.

KWS (Warden 2018) pushes the same factorization to the microcontroller end of the spectrum and anchors the always-on TinyML lighthouse. A continuous audio stream is first converted by a short-time Fourier transform into a 2D time-frequency spectrogram (or Mel-frequency cepstral coefficients), so 2D convolutional filters can slide across time and frequency bins to detect acoustic phoneme signatures. Models that listen for "Hey Siri" or "Ok Google" run on always-on microcontrollers with kilobyte-scale memory and milliwatt power budgets (Warden and Situnayake 2020; C. R. Banbury et al. 2021; C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kiraly, et al. 2021), so their designers count every byte and cycle.

Lighthouse 6.3: KWS (TinyML lighthouse)

Why it matters: Keyword spotting models (like DS-CNN) represent the power-constrained end of the spectrum. Used in always-on applications like smart doorbells that wake a larger vision pipeline only after an audio trigger, these models must run on microcontrollers with milliwatt power budgets.

At 200K parameters, a DS-CNN keyword spotter's 800 KB of FP32 weights already exceeds the few hundred kilobytes of memory a typical microcontroller can devote to a model, so the model fits only after 8-bit quantization cuts it fourfold. KWS is therefore the lighthouse

for extreme quantization (INT8/INT4) and specialized architectural primitives (depthwise separable convolutions) that trade theoretical representational power for maximum efficiency per watt.

From ResNet-50's compute-heavy standard convolutions through MobileNet's depthwise separable variants to KWS's micro-power deployment, CNNs demonstrate how architectural constraints transform data structure into hardware efficiency. Local connectivity and weight sharing reduce parameter storage and expose reuse. Depth is what lets those stacks compose edges into objects, and ResNet-50's fifty layers depend on two building blocks that make depth trainable at all.

6.4 Skip Connections and Normalization

When plain convolutional networks are stacked beyond roughly 20 layers, adding more layers can make *training* error worse rather than better: gradients vanish or explode on the way back through the stack, and the scale of intermediate activations drifts on the way forward. Convolution made deep vision networks affordable in parameter count, and two building blocks made them trainable. Skip connections keep gradients alive through dozens of layers, and normalization keeps the activations those layers produce within a stable numerical range. Both arrived with deep CNNs, both carry their own memory and bandwidth costs, and both reappear in the deep families that follow, most visibly the transformer.

6.4.1 Skip connections and gradient flow

As networks grew deeper, repeated matrix multiplications and activation derivatives drove gradient signals either toward numerical underflow or toward floating-point overflow. Skip connections resolve this optimization barrier with additive identity shortcuts that bypass the layer transformations.

The depth problem follows from the chain rule, whose layer-by-layer Jacobian form appears in section 5.4.5.2. For a deep network with layers $f_1, f_2, \ldots, f_{N_L}$, the gradient of the loss $\mathcal{L}$ with respect to the weights in layer 1 is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(N_L)}} \cdot \frac{\partial \mathbf{a}^{(N_L)}}{\partial \mathbf{z}^{(N_L)}} \cdot \frac{\partial \mathbf{z}^{(N_L)}}{\partial \mathbf{a}^{(N_L-1)}} \cdot \ldots \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

where $\mathbf{z}^{(\ell)}$ represents the preactivation and $\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$ the postactivation output of layer ℓ. Each layer contributes both a weight transformation and an activation derivative $\sigma'(\mathbf{z}^{(\ell)})$. With sigmoid activations, whose derivative peaks at 0.25 (section 9), the activation-derivative factors alone contribute at most $(0.25)^{N_L}$ along a path. The full gradient also depends on the intervening weight matrices; when their transformations do not offset this attenuation, early layers receive negligible updates. Exploding gradients are the destabilizing counterpart. If each layer Jacobian expands a shared direction by about 1.5, that component grows exponentially, producing overflow, NaN (not-a-number) values, and training divergence rather than a silent stall.

ReLU removes the activation's share of this decay, since its derivative is one on active paths (section 11), and that change made networks of 10–20 layers trainable. It does not remove the problem. Inactive units pass zero gradient, and a poorly initialized weight or a large update can leave a unit permanently dead; more fundamentally, ReLU does nothing to regulate the singular values of the weight matrices, which continue to attenuate or amplify gradients across depth. The original ResNet study showed the consequence on CIFAR-10: a 56-layer plain network had higher training and test error than a 20-layer plain network throughout training (He et al. 2016a). Because the deeper network was worse on the *training* set, the failure was an optimization barrier, not overfitting.

A residual block gives the gradient a direct path back through its input. Equation 6.6 adds the identity path $\mathbf{x}$ to the residual mapping $\mathcal{F}(\mathbf{x})$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{6.6}$$

where $\mathcal{F}(\mathbf{x})$ represents the residual function (typically two convolutional layers with batch normalization and ReLU) and $\mathbf{x}$ is the identity skip connection. The original post-activation ResNet block

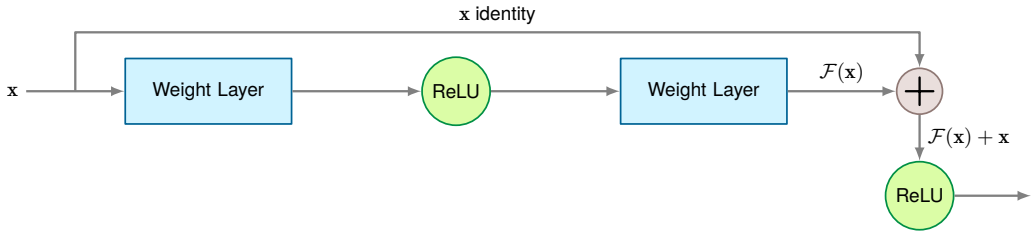


Figure 6.6: Residual Connection Block (Original Post-Activation Form): The original ResNet block (He et al. 2016a) implements $\mathbf{y} = \text{ReLU}(\mathcal{F}(\mathbf{x}) + \mathbf{x})$, where an identity shortcut bypasses the weight layers before a final ReLU activation. In later pre-activation variants (He et al. 2016b) and modern transformer blocks, the nonlinearity moves inside $\mathcal{F}(\mathbf{x})$, making the addition a completely clean identity shortcut $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ that matches the unobstructed gradient flow in equation 6.6.

(figure 6.6) routes an identity shortcut around two weight layers and the ReLU between them, adds it back at a junction, and then applies the final activation.

During backpropagation, the gradient flows through this addition, and applying the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \left(\frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + \mathbf{I} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \mathcal{F}'(\mathbf{x}) + \frac{\partial \mathcal{L}}{\partial \mathbf{y}}$$

Here $\mathbf{I}$ is the identity matrix, and the gradient decomposes into two additive terms. Even if the residual term $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \mathcal{F}'(\mathbf{x})$ attenuates, the identity term passes $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ directly to the earlier layer. While the residual branch can partially interfere with the identity signal, keeping $\|\mathcal{F}'(\mathbf{x})\|$ small keeps the block Jacobian close to identity and bounds its condition number. Through N_L residual blocks, the gradient becomes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(0)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(N_L)}} \cdot \prod_{\ell=1}^{N_L} (\mathcal{F}'_\ell(\mathbf{x}^{(\ell-1)}) + \mathbf{I})$$

Each factor $(\mathcal{F}'_\ell + \mathbf{I})$ contains the identity term, keeping the factor near identity when the residual-branch Jacobian is small. A plain network instead multiplies arbitrary layer Jacobians, and if their average gain is even slightly below one the product vanishes exponentially ($0.9^{50} \approx 0.005$). ResNets multiply near-identity factors. The deviations can still compound, and the additive path does not guarantee exact unit gain across arbitrary depth, but this structure improves conditioning enough to make networks with 100+ layers trainable. The CIFAR-10 experiments confirm it: a 56-layer residual network reached about 7.0 percent test error against 8.8 percent for its 20-layer residual counterpart, reversing the degradation seen in plain networks (He et al. 2016a). There is no single depth at which skip connections suddenly become mandatory; trainability also depends on initialization, normalization, optimizer dynamics, and block architecture (He et al. 2016b).

The shortcuts carry a concrete hardware cost that every deep network pays. Evaluating $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ requires the input tensor $\mathbf{x}$ to remain resident in memory throughout the execution of every operation in the branch $\mathcal{F}(\mathbf{x})$. This extended liveness window increases peak activation memory in accelerator HBM during the forward pass and preserves $\mathbf{x}$ for backpropagation unless activation checkpointing is used (section 8.6.5.1). The elementwise addition is also memory-bandwidth bound. It performs one addition per two loaded operands and one stored result, about 0.08 FLOP/byte in FP32 (one FLOP per 12 bytes), so un-fused residual merges become memory stalls.

Residual connections stabilize the backward gradient path, but they do not constrain the forward scale of intermediate activations. Unbounded variance across depth can still destabilize layer inputs, which calls for a complementary block: normalization.

6.4.2 Normalization and activation stability

Without normalization, small weight updates in early layers compound across depth, shifting and amplifying intermediate activations until later nonlinearities saturate or overflow and training requires tiny learning rates. Skip connections protect the backward pass, and normalization steadies the forward pass by re-centering and re-scaling those intermediate activations at each block. Batch normalization in CNNs[17] (Ioffe and Szegedy 2015), layer normalization for sequence models, and

[17] **Batch normalization (BatchNorm)**: The original normalization layer (Ioffe and Szegedy 2015), which re-scales activations using per-mini-batch statistics. In one ImageNet experiment, it reached the target accuracy with 14× fewer training steps. Its batch-size dependency and training-serving skew (switching from batch statistics to running averages at inference) are systems limitations that motivated alternatives: LayerNorm removed batch dependency for transformers, and RMSNorm removed mean centering.

[18] **RMSNorm (root mean square normalization)**: Introduced by Zhang and Sennrich (2019) at NeurIPS, RMSNorm simplifies LayerNorm by normalizing with the root mean square alone, dropping the mean-centering step. Across the paper's tested models and implementations, RMSNorm reduced overall running time by 7–64 percent relative to LayerNorm. LLaMA-family and Mixtral-style transformer reports use RMSNorm (Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023; Jiang et al. 2024), illustrating why a simpler normalization kernel matters at large language model scale.

RMSNorm[18] for large language models illustrate different choices of reduction axes and statistics.

Batch normalization standardizes activations using statistics computed across the mini-batch for each feature or channel during training. For fully connected activations, the reduction axis is the batch dimension. For convolutional activations, implementations compute per-channel statistics over both the batch and spatial positions ($B \times H_{\text{img}} \times W_{\text{img}}$). For a mini-batch $\mathcal{B} = \{a_1, \ldots, a_B\}$ of activations at a particular layer, the transformation proceeds in two stages, starting with the mini-batch mean and variance:

$$\mu_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B} a_i \qquad \sigma^2_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B}(a_i - \mu_{\mathcal{B}})^2$$

The second stage normalizes each activation over the batch and applies a learned scale and shift. The normalization step in equation 6.7 centers and scales activations, while equation 6.8 lets the network learn a useful output scale and offset:

$$\hat{a}_i = \frac{a_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \tag{6.7}$$

$$y_i = \gamma \hat{a}_i + \beta \tag{6.8}$$

The parameters γ (scale) and β (shift) are learned during training, while ϵ (typically 10^{-5}) prevents division by zero. The affine transform restores learned scale and shift flexibility, but fixed γ and β cannot exactly invert varying statistics for every batch. Normalization permits larger learning rates by preventing intermediate activations from exploding into saturating non-linear regimes, accelerating convergence relative to unnormalized networks.

While batch normalization enabled training of much deeper CNNs, it needs batch-wide statistics, which constrains distributed and sequence workloads. When a mini-batch is split across several accelerators, computing true mini-batch statistics requires an all-reduce, a collective that sums every accelerator's partial statistics and returns the total to all of them, adding communication latency to every normalized layer. If instead computed locally on small per-accelerator batches, noisy empirical statistics destabilize optimization. Furthermore, batch normalization cannot handle variable-length sequences or text generation, which produces one token at a time, without complex padding masks and training-serving skew. Layer normalization resolves these limitations by normalizing across the feature dimension rather than across the batch (Ba et al. 2016).

↪ The AllReduce collective and its communication cost are developed in section 8.7.1.1.

For an input vector $\mathbf{x} \in \mathbb{R}^d$ with d features, layer normalization first computes the mean and variance across those d components:

$$\mu_{\text{LN}} = \frac{1}{d}\sum_{i=1}^{d} x_i \qquad \sigma^2_{\text{LN}} = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu_{\text{LN}})^2$$

With those per-token statistics in hand, equation 6.9 defines the complete layer normalization operation, where $\odot$ denotes element-wise multiplication:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu_{\text{LN}}}{\sqrt{\sigma^2_{\text{LN}} + \epsilon}} \odot \gamma + \beta \tag{6.9}$$

Because each token or sample is normalized independently, the computation stays within an accelerator's on-chip SRAM and registers, with no cross-sample or cross-device communication. That independence makes layer normalization the standard choice for transformers, where sequences vary in length and text generation evaluates a single token at a time. Table 6.4 contrasts the three variants along the machine and operational dimensions that decide between them.

Table 6.4: Normalization Variant Comparison: Different normalization techniques trade off between computational efficiency, batch size sensitivity, and architectural compatibility. RMSNorm (Zhang and Sennrich 2019), used in LLaMA-family and other efficient transformer architectures (Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023), omits mean centering: $\text{RMSNorm}(\mathbf{x}) = \mathbf{x}/\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon} \odot \gamma$.

Characteristic	BatchNorm	LayerNorm	RMSNorm
Normalization Axis	Batch and, for CNNs, spatial positions	Feature dimension	Feature dimension
Batch Size Dependency	High (noisy for small batches)	None	None
Typical Use Case	CNNs, vision models	Transformers, RNNs	LLaMA, efficient transformers
Computation Cost	Higher (mean + variance)	Higher (mean + variance)	Lower (RMS only)
Training/Inference	Different (running stats)	Identical	Identical

The computational profile of normalization is defined by its arithmetic intensity. Because normalization applies elementwise operations to large activation tensors, execution is memory-bandwidth bound rather than compute bound. A straightforward LayerNorm kernel makes two reductions across the hidden dimension d, one for the mean μ_{LN} and one for the variance σ^2_{LN}, although Welford's algorithm can compute both in a single pass. RMSNorm drops the mean-centering reduction, leaving a single sum of squares, which simplifies the kernel and eases fusion with neighboring operations; its authors report 7–64 percent lower running time across the models they tested.

The table's last row is an operational cost. BatchNorm keeps nonlearned running mean and variance buffers and switches from mini-batch statistics in training to those running statistics at inference, so a mismatch between the buffers and deployment data is a frequent source of training-serving skew. LayerNorm and RMSNorm compute identically in both modes and carry no running state. With skip connections and normalization in place, depth stops being an optimization barrier.

↪ Kernel fusion, which collapses memory-bound normalization and activation chains into single kernels whose intermediates stay on chip, is developed in section 7.3.2.

Checkpoint 6.2: Making depth trainable

- ☐ **Gradient path**: can you explain why the identity term in a residual block keeps gradients from vanishing across many layers?
- ☐ **Liveness cost**: why does a skip connection raise peak activation memory, and why is an unfused residual addition memory-bandwidth bound?
- ☐ **Normalization choice**: why does layer normalization suit sequence models and token-by-token generation better than batch normalization?

Depth extends how far a network's view reaches across its input, but a convolutional stack still sees only its receptive field, whether that field spans pixels or the time bins of a spectrogram. A streaming system that must carry information across variable time spans within a fixed state budget motivates a different trade-off: updating a compact recurrent state as each observation arrives.

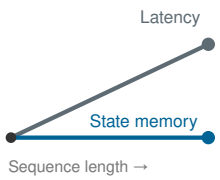
6.5 RNNs: Sequential Pattern Processing

An MLP requires a fixed-width input vector and a CNN sees only a bounded spatial window, so neither can process a continuous audio stream or a sentence of arbitrary length without padding, truncating, or stacking layers until activation memory overflows. Where convolutional networks exploit *spatial* structure, many real-world signals carry *temporal* structure, in which sequential order and prior context determine meaning. Recurrent networks process these variable-length streams within a fixed memory budget by carrying and updating a persistent state vector from one time step to the next.

Definition 6.4: Recurrent neural networks

Recurrent neural networks (RNNs) are sequence-processing architectures that maintain a hidden state $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ updated at each time step, encoding the assumption that the current output depends on all prior inputs through this fixed-size state vector.

1. **Significance**: The fixed-size state provides $\mathcal{O}(1)$ inference state memory regardless of sequence length (processing a 10,000-token sequence requires the same state memory as a 10-token sequence), but the sequential update rule forces all S recurrent transitions to execute in order. Separate per-step kernels (compiled accelerator routines, a different sense of the word from a convolution kernel) add launch overhead (L_{lat}); fusion can reduce those launches while preserving the serial dependency chain.
2. **Distinction**: Unlike MLPs, which require a fixed-dimensional input vector, or CNNs, which restrict context to a fixed spatial window, RNNs compress an unbounded history into a fixed-size bottleneck state, so they accept sequences of any length without widening the input layer. Gradient signals pass back through the recurrent steps, where repeated Jacobian products can attenuate or amplify them (derived below).
3. **Common pitfall**: A frequent misconception is that RNNs are obsolete. For streaming inference on resource-constrained hardware, such as keyword spotting on a microcontroller, a model that must keep every past input to see the whole stream needs memory that grows with stream length, which can be prohibitive. An RNN's $\mathcal{O}(1)$ state size makes it a candidate when a fixed state budget is required.



RNNs hold state memory fixed, but latency grows with sequence length.

Expanding a convolutional receptive field to cover long sequences requires either stacking dozens of layers or applying dilated kernels, both of which scale parameter count and intermediate activation storage regardless of whether the intervening tokens contain relevant information. Recurrent architectures, from simple recurrent networks to gated variants such as long short-term memory (LSTM) networks (Elman 1990; Hochreiter and Schmidhuber 1997), address this constraint by introducing a temporal inductive bias: execution order encodes causal direction, and history compresses into a recurring state. Instead of processing each input vector in isolation, the network carries an internal state vector across successive time steps. This inductive bias establishes an explicit engineering trade-off: it provides variable-length temporal context within a fixed memory budget, but it introduces loop-carried data dependencies that serialize execution on parallel accelerator hardware.

6.5.1 Pattern processing needs

A word like "bank" cannot be disambiguated on its own: "river bank" refers to a shoreline, whereas "investment bank" refers to a financial institution. Only the preceding words resolve which meaning applies. Acoustic waveforms and sensor telemetry behave the same way: a single audio sample or temperature reading is ambiguous without the history that came before it.

Data with this sequential structure imposes three requirements on a model:

1. **Variable-length inputs**: Sequences arrive with different lengths S, ruling out a single fixed-width input layer.
2. **Long-range memory**: The model must carry relevant signals across many time steps without requiring a fixed window size.
3. **Continuous state updates**: Internal representations must update as each new token arrives, folding in new evidence while dropping stale history.

A fixed-input MLP requires windowing, padding, or truncation to process variable-length sequences, and a CNN reaches a long history only by stacking or dilating layers, at a cost in activation storage. Recurrence meets all three streaming requirements by making a persistent, fixed-size state part of its dataflow.

6.5.2 Algorithmic structure

To meet the requirements of section 6.5.1, recurrent neural networks introduce directed cycles in their computational graph. Rather than mapping inputs directly to outputs through feedforward layers, an RNN maintains a persistent vector, the **hidden state**, that updates iteratively at each time step. The hidden state is a running summary of the past.

The fundamental state transition in equation 6.10 computes the current hidden state $\mathbf{h}_t$ by combining the incoming token vector $\mathbf{x}_t$ with the preceding state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \phi(\mathbf{W}_{\text{hh}}\mathbf{h}_{t-1} + \mathbf{W}_{\text{hx}}\mathbf{x}_t + \mathbf{b}_{\text{h}}) \tag{6.10}$$

where $\mathbf{h}_t$ denotes the hidden state at time t, $\mathbf{x}_t$ denotes the input at time t, $\mathbf{W}_{\text{hh}}$ contains the recurrent weights, $\mathbf{W}_{\text{hx}}$ contains the input weights, $\mathbf{b}_{\text{h}}$ is the hidden-state bias vector, and ϕ is the activation function. The equation uses column vectors; the later listings use row-major batched tensors, so their weight multiplications appear on the right. Unrolled across time (figure 6.7, right), the loop becomes a chain of transitions $\mathbf{h}_{t-1} \to \mathbf{h}_t \to \mathbf{h}_{t+1}$ in which every link reuses the same recurrent weight matrix $\mathbf{W}_{\text{hh}}$, so no step can begin until the one before it has finished.

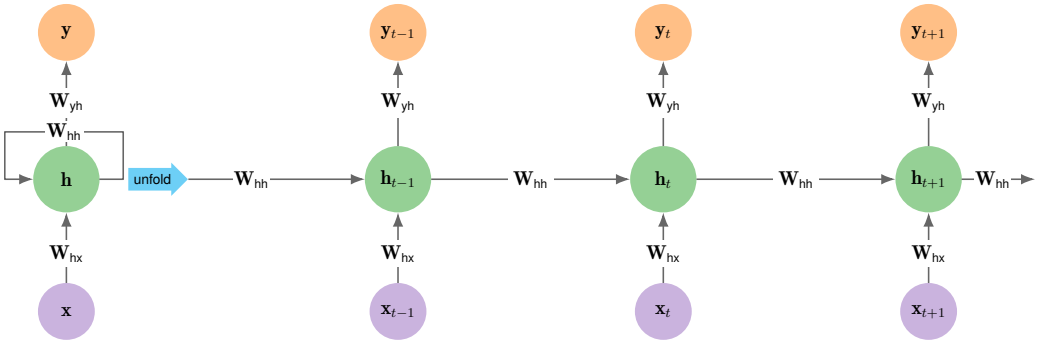


Figure 6.7: Recurrent Neural Network Unfolding: The compact recurrent loop (left) unfolds into one copy of the cell per time step (right). Three weight matrices are shared across all steps: $\mathbf{W}_{\text{hx}}$ (input-to-hidden), $\mathbf{W}_{\text{hh}}$ (hidden-to-hidden), and $\mathbf{W}_{\text{yh}}$ (hidden-to-output). These matrices are reused at every step, so the parameter count is independent of sequence length. For a 128-dimensional hidden state and a 100-dimensional input, each time step requires 16,384 MACs for recurrent connections plus 12,800 for input projection.

Unfolding the recurrent loop across S steps reveals the algorithmic properties of recurrence. Parameter count remains independent of sequence length S because the weight matrices $\mathbf{W}_{\text{hh}}$ and $\mathbf{W}_{\text{hx}}$ are shared across all time steps. However, sharing weights across time establishes a loop-carried dependency: step t cannot evaluate $\mathbf{h}_t$ until step $t-1$ finishes producing $\mathbf{h}_{t-1}$. This dependency forms a critical path of length $\mathcal{O}(S)$ serial operations that prevents hardware accelerators from parallelizing across the sequence dimension.

This sequential structure produces fundamentally different memory characteristics between inference and training. During streaming inference, an RNN evaluates one step at a time, discarding prior raw inputs and retaining only the current hidden state. State memory overhead remains strictly $\mathcal{O}(d)$ in the hidden dimension d, regardless of whether the sequence spans 10 or 10,000 steps. In contrast, training via backpropagation through time (BPTT) (Werbos 1990) unrolls the graph across the entire sequence of length S. Computing gradients via the chain rule requires evaluating the Jacobian product across all intermediate transitions:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_S} \prod_{t=1}^{S} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

Ordinary BPTT retains the intermediate activations $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_S$ for these Jacobian products. This retained-state scheme converts an $\mathcal{O}(d)$ runtime state into an $\mathcal{O}(S \cdot d)$ training footprint; recomputing saved intermediates trades additional work for memory. Repeated Jacobians combine activation derivatives with $\mathbf{W}_{\text{hh}}$, and their products can attenuate or amplify gradients over long horizons, limiting the effective context window of simple RNNs. Gated variants such as the LSTM answer

this with learned gates, elementwise valves that decide at each step how much of the stored state to keep, overwrite, or expose; a kept state passes to the next step without being multiplied by $\mathbf{W}_{hh}$, which gives gradients a path that does not shrink at every step.

6.5.3 Gated recurrence

A gate is an elementwise valve. It applies a sigmoid to a learned projection of the current input and the previous state, producing values between zero and one that multiply the quantity the gate controls, so a value near one passes that quantity through and a value near zero blocks it. LSTMs[19] (Hochreiter and Schmidhuber 1997) keep a separate cell state beside the hidden state and use three such **gates** (input, forget, and output) together with a candidate update; because the forget gate can carry the cell state forward almost unchanged, gradients cross many steps without being multiplied by $\mathbf{W}_{hh}$ each time. GRUs[20] (Cho et al. 2014) merge the same idea into two gates. For the machine, an LSTM step computes four affine transformations of the input and previous state instead of one, so it holds four times the recurrent weights, performs four times the multiply-accumulates per step, and carries twice the state (hidden and cell). Runtimes fuse the four projections into one matrix multiply four times as wide, which raises its arithmetic intensity without shortening the serial chain of steps.

6.5.4 Computational mapping

In hardware, a recurrent step is two matrix products followed by an elementwise update. Convolution reuses each weight across a grid of positions available all at once (section 6.3.4.3); a recurrent step reuses its weights only across time, and each step must wait for the state the previous step produces.

Listing 6.5 demonstrates the single-time-step mechanism using framework-level matrix operations: combine the previous hidden state with the current input, add the bias, and apply the activation to produce the next hidden state. The code is intentionally local to one step because the systems cost is not the step itself, but the dependency chain that prevents parallel execution across time.

Listing 6.5: RNN Layer Abstraction: Framework-level implementation combining two matrix multiplications, $\mathbf{h}_{t-1}\mathbf{W}_{hh}$ and $\mathbf{x}_t\mathbf{W}_{hx}$, per time step. For 128-dimensional hidden state and 100-dimensional input, each step requires 16,384 + 12,800 = 29,184 MACs, but sequential dependencies prevent parallelization across time.

```
def rnn_layer_step(x_t, h_prev, W_hh, W_hx, b):
    # x_t: input at time t (batch_size × input_dim)
    # h_prev: previous hidden state (batch_size × hidden_dim)
    # W_hh: recurrent weights (hidden_dim × hidden_dim)
    # W_hx: input weights (input_dim × hidden_dim)
    h_t = activation(matmul(h_prev, W_hh) + matmul(x_t, W_hx) + b)
    return h_t
```

At the framework level, `rnn_layer_step` hides the underlying loop structure and memory traffic behind high-level matrix multiplications (`matmul`). In hardware, evaluating this step requires moving weights and activations through the memory hierarchy. To see where execution serializes and where parallelism remains, listing 6.6 expands the matrix operations into explicit loop nests down to scalar multiply-accumulate operations.

[19] **LSTM (long short-term memory)**: Invented by Hochreiter and Schmidhuber in 1997, LSTMs introduced a "Constant Error Carousel," a gated cell state that protects error signals from exponential decay during backpropagation through time. The systems cost is that a standard LSTM computes input, forget, and output gates plus a candidate cell update, giving roughly four affine transformations per time step vs. one in a vanilla RNN. Transformers replaced LSTMs in many large-scale language workloads not because attention costs less arithmetic, since it costs more, but because it parallelizes across sequence positions where a recurrence cannot.

[20] **GRU (gated recurrent unit)**: Cho et al. (2014) describes a gated hidden unit for encoder-decoder translation that uses reset and update gates to control how the hidden state is updated. Relative to an LSTM's input, forget, and output gates plus candidate cell update, this gives a simpler gated recurrence. The broader systems lesson: architectural simplification can reduce state and matrix operations when it preserves task performance, a principle that recurs in efficiency-oriented designs from MobileNet to distilled transformers.

Listing 6.6: RNN Layer Computation: Nested loops expose the sequential dependency structure. Loop 1 enables batch parallelism, but Loops 2-3 must complete before Loop 4's activation; because each time step depends on the previous step's output, the resulting $\mathcal{O}(S)$ sequential depth prevents GPU parallelization across the time dimension.

```
def rnn_layer_compute(x_t, h_prev, W_hh, W_hx, b):
    # Initialize next hidden state
    h_t = np.zeros_like(h_prev)

    # Loop 1: Process each sequence in the batch
    for batch in range(batch_size):
        # Loop 2: Compute recurrent contribution (h_prev × W_hh)
        for i in range(hidden_dim):
            for j in range(hidden_dim):
                h_t[batch, i] += h_prev[batch, j] * W_hh[j, i]

        # Loop 3: Compute input contribution (x_t × W_hx)
        for i in range(hidden_dim):
            for j in range(input_dim):
                h_t[batch, i] += x_t[batch, j] * W_hx[j, i]

        # Loop 4: Add bias and apply activation
        for i in range(hidden_dim):
            h_t[batch, i] = activation(h_t[batch, i] + b[i])

    return h_t
```

The loop structure exposes the concurrency boundaries of recurrent execution. Batch instances in Loop 1 are independent, so sequences within a batch parallelize across accelerator cores without cross-talk. Within each batch instance, Loops 2 and 3 execute the recurrent transformation ($\mathbf{h}_{t-1}\mathbf{W}_{hh}$) and input projection ($\mathbf{x}_t\mathbf{W}_{hx}$). Both reductions must accumulate fully before Loop 4 can apply the elementwise bias addition and activation function to produce $\mathbf{h}_t$.

For an input dimension of 100 and a hidden state dimension of 128, each step executes two matrix multiplications: a 100×128 input projection and a 128×128 recurrent transformation. When batch size is large ($B \gg 1$), these operations execute as general matrix-matrix multiplications (GEMM) that reuse each loaded weight across all B samples. When batch size is small ($B = 1$, typical of real-time streaming inference), they collapse into the same weight-streaming matrix-vector products as the batch-one dense layer of equation 6.2, which starve accelerator compute units.

6.5.5 System implications

The hard physical constraint that recurrent architectures impose is sequential dependency. In the update $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t)$, the recurrent transformation at time step t needs the state produced by step $t-1$. An input sequence of length $S = 1{,}000$ therefore requires $1{,}000$ ordered state updates, even when a fused kernel executes several or all updates within one launch. Hardware can parallelize across batch instances and within each matrix operation; input projections can also be computed across time when the inputs are available. Faster arithmetic or memory shortens individual steps, while the recurrent dependency chain remains.

During inference, recurrent networks need little state memory. Because the recurrent state compresses history into a fixed-size vector $\mathbf{h}_t$, inference state memory scales as $\mathcal{O}(d)$ (for example, 2 KB for a 512-dimensional FP32 state), remaining completely flat whether sequence length S is 10 or 10,000 tokens. This constant memory footprint contrasts sharply with attention mechanisms (section 6.6), whose stored context grows with every token generated, an $\mathcal{O}(S \cdot d)$ store that the transformer section sizes. For streaming deployments on memory-constrained edge hardware, such as microcontrollers, an RNN's $\mathcal{O}(1)$ runtime state avoids the growing context store that limits attention-based models.

Recurrent execution splits the memory hierarchy between two behaviors: high temporal locality for parameters, and low operational intensity under standard framework dispatch. Weight matrices $\mathbf{W}_{hh}$ and $\mathbf{W}_{hx}$ are reused at every time step, residing comfortably in SRAM or L2 cache if the model fits on-chip. However, naively launching separate kernels for input projection, recurrent projection, bias addition, and activation at each time step incurs severe kernel-launch overhead

and forces intermediate tensors back and forth across HBM. To mitigate this memory-bandwidth bottleneck, production recurrent runtimes fuse the projections, bias addition, and activation into a single monolithic kernel that maintains $\mathbf{h}_t$ within GPU registers and shared memory throughout the sequence execution. At batch one, each recurrent step is the dense layer's matrix-vector product again, with the same energy profile. Unless the recurrent weights stay on chip from step to step, every step pays DRAM energy for every weight, about 139× the energy of its arithmetic, which is a second reason streaming runtimes keep those weights resident.

↪ What a fused kernel that holds its state in registers and shared memory costs in on-chip capacity and occupancy is developed in section 11.7.7.

Checkpoint 6.3: Recurrent state and the serial chain

Recurrence trades parallelism for a fixed memory budget. Check your understanding:

- ☐ **Serial chain**: why can a faster accelerator not remove the 1,000 ordered updates of a 1,000-step sequence, and which parts of the work can still run in parallel?
- ☐ **Fixed state**: why does an RNN's inference memory stay flat as a stream grows, while training through time stores state for every step?
- ☐ **Gates**: what does an LSTM step cost the machine compared with a plain recurrent step, and what does fusing its four projections change?

This architectural tension between fixed-capacity state and serialized execution defined sequence processing before transformers. While recurrent networks maintain constant memory overhead regardless of sequence length, the sequential dependency prevents hardware from parallelizing across time steps, the fixed-capacity state becomes an information bottleneck in which early inputs fade over long spans, and gradients through long chains of recurrent steps vanish or explode (the vanishing gradient problem) (Bengio et al. 1994). State-space models such as Mamba (Gu and Dao 2023) later revived recurrence for exactly its systems strength, a fixed-size state at decode time where an attention model's stored context grows, while training with a parallel scan instead of a step-by-step loop. Overcoming the serial chain and the fading state in general, however, required an architecture capable of establishing direct, content-addressable links across arbitrary sequence positions without intervening recurrent steps: the attention mechanism (section 6.6).

6.6 Attention: Dynamic Processing

Recurrent networks serialize execution along the sequence dimension and compress all historical context into a fixed-capacity hidden state vector (section 6.5.5). In a sentence such as "The cat, which was sitting by the window overlooking the garden, was sleeping," the subject "cat" and predicate "sleeping" are separated by ten intervening words, yet they form the core syntactic and semantic relationship. An RNN must process all intervening tokens sequentially, risking information loss across intermediate recurrent transitions while preventing hardware from parallelizing across time steps. This constraint motivates an alternative computational pattern: an operation that directly computes relevance between any two positions regardless of distance.

Attention mechanisms[21] address this limitation (Bahdanau et al. 2015) by introducing dynamic connectivity patterns that adapt to input content. The setting was sequence-to-sequence translation, built as an encoder-decoder pair. An encoder network reads the input sentence into one representation per input position, and a decoder network generates the output sentence one token at a time, carrying its own recurrent decoder state. Recurrent versions of that pair passed only the encoder's final state to the decoder. Rather than forcing all context through that single fixed-capacity vector, encoder-decoder attention computes similarity scores between the decoder state at each output step and all encoder positions, routing information dynamically across the sequence. A single decoder step compares one query with every encoder position. In full-sequence self-attention, each position queries all positions in the same sequence, so the number of score interactions grows quadratically with sequence length.

[21] **Bahdanau attention**: This approach broke the "fixed-length vector" bottleneck of prior sequence-to-sequence models. The core trade-off was accepting a linear, $\mathcal{O}(S)$ memory cost to store all input states in exchange for overcoming the information loss inherent in a single vector.

Definition 6.5: Attention mechanisms

Attention mechanisms are neural network operations that compute a weighted sum of value vectors, where the weights are derived from learned similarity scores between a query vector and a set of key vectors, enabling dynamic, content-dependent information routing between any two positions in a sequence.

1. **Significance**: Attention connects any two tokens in $\mathcal{O}(1)$ depth but computes S^2 score interactions. If those scores are materialized, a 4,096-token sequence with 16-bit scores consumes 33.6 MB per layer per head (about 16.8M scores at 2 bytes each), directly increasing the iron law's D_{vol} term.
2. **Distinction**: Unlike RNNs, which compress all prior context into a single fixed-size state vector, attention mechanisms retain every token's representation and compare tokens directly. When a model generates text one token at a time, it keeps the key and value vectors of earlier tokens rather than recomputing them, and that **key-value (KV) cache** grows as $\mathcal{O}(Sd)$; the transformer section explains generation and sizes the cache.
3. **Common pitfall**: A frequent misconception is that attention is a general-purpose weighting scheme that can be applied freely. Quadratic score computation is a hard scaling constraint: doubling the context quadruples score interactions. Naive implementations also quadruple score storage; tiled exact algorithms such as FlashAttention avoid the full matrix, while sparse variants reduce the scores computed (Child et al. 2019).

The ability of attention to connect any position to any other made the recurrent structure unnecessary for many sequence tasks. The transformer[22] (Vaswani et al. 2017) removed recurrence entirely, trading the RNN's $\mathcal{O}(S)$ sequential path for constant path length between positions within one attention layer and thereby enabling parallelization across sequence positions on high-throughput accelerators.

Attention functions as a computational primitive rather than an independent architecture family. The transformer (section 6.7), the family built around it, inherits its core systems properties directly from attention: content-dependent routing, parallel sequence execution, and quadratic score scaling. Evaluating those properties begins with the dynamic pattern-processing problem that attention addresses.

[22] **Transformer**: The founding paper, "Attention Is All You Need," made the explicit systems claim that a parallel attention mechanism could fully replace sequential recurrent processing. This architectural trade eliminates an RNN's $\mathcal{O}(S)$ path length constraint on parallelism but introduces $\mathcal{O}(S^2)$ dense score computation, a quadratic cost that continues to shape context-window engineering.

6.6.1 Pattern processing needs

In "the trophy did not fit in the suitcase because it was too big," the word "it" refers to "trophy," but if "big" changes to "small," "it" refers to "suitcase." The same positions, at the same distances, need different links depending on what the words mean. Neither a CNN's fixed spatial window nor an RNN's step-by-step hidden state can route information directly between arbitrary positions based on content. Dynamic sequence processing needs an operation that scores relationships from the tokens themselves, letting each word draw directly on the words that matter to it regardless of how far apart they sit (figure 6.8).

The bipartite connection lines in figure 6.8 illustrate how the pronoun "they" could receive information from "student" and "finish." The hypothetical weights demonstrate a direct routing path, not a measured coreference result. Pairwise connections can be evaluated in parallel, allowing a single layer to connect these positions without propagating intermediate state across intervening tokens.

Input-dependent processing extends well beyond language. In protein structure prediction, amino-acid interactions depend on chemical properties and spatial arrangement, not only chain position; in document analysis, connections between sections depend on semantic content rather than proximity. Graph convolutional networks (GCNs) (Kipf and Welling 2017) share attention's input-dependent routing but gather features along the sparse edges of each input graph, producing irregular memory access that can bind execution on memory bandwidth and cache misses rather than compute throughput.

Input dependence does not imply all-to-all connectivity. Dense attention scores every pair of sequence elements, whereas a GCN restricts aggregation to edges in the observed graph. The

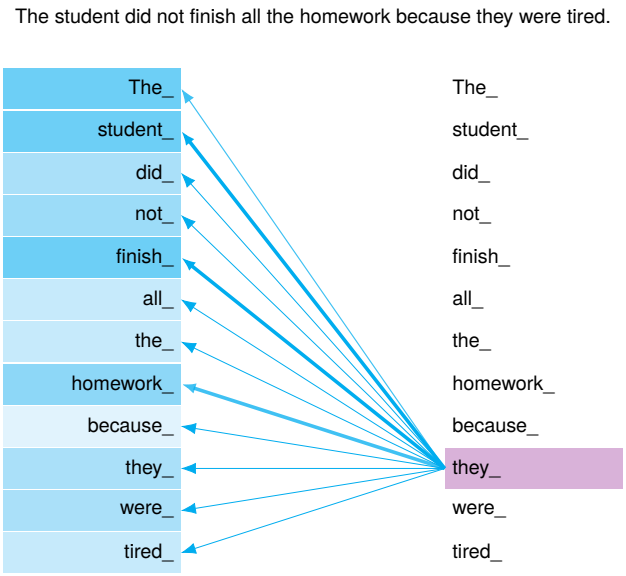


Figure 6.8: Illustrative Attention Pattern: A schematic attention pattern for the pronoun "they." Line thickness represents hypothetical attention-weight magnitude, with stronger links to "student," "finish," and "homework." One attention layer connects all positions directly and can compute their dense interactions in parallel.

common systems challenge is that the computation or memory-access pattern responds to the input rather than following a fixed stencil. Even when tensor dimensions stay fixed, content-dependent scores determine which information is emphasized. Attention provides one mechanism for learning such relationships, and dense self-attention forms the foundation of the transformer architecture.

6.6.2 Algorithmic structure

To route information dynamically (section 6.6.1), the network needs an operator that turns a variable-length sequence into content-dependent weights (Bahdanau et al. 2015). The transformer architecture computes these weights with the scaled dot-product attention operator introduced by Vaswani et al. (2017):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{head}}}}\right)\mathbf{V}$$

For this self-attention case, the inputs $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), and $\mathbf{V}$ (values)[23] are learned linear projections of the token representations ($\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$). Attention uses three separate projection matrices rather than computing dot-product affinities directly between raw tokens ($\mathbf{X}\mathbf{X}^T$) for two architectural reasons:

1. **Asymmetric relationships**: Direct token dot products $\mathbf{x}_i \cdot \mathbf{x}_j$ are symmetric ($\mathbf{x}_i \cdot \mathbf{x}_j = \mathbf{x}_j \cdot \mathbf{x}_i$), whereas relationships in language are not. In "the chef cooked the meal," the verb "cooked" looks for its direct object ("meal"), while "meal" plays a different role, so the two directions of the pair need different scores. Splitting the projection into a query (what a token looks for) and a key (what a token offers) lets $\mathbf{q}_{\text{cooked}} \cdot \mathbf{k}_{\text{meal}}$ be high while $\mathbf{q}_{\text{meal}} \cdot \mathbf{k}_{\text{cooked}}$ stays low.
2. **Separating where to look from what to pass on**: The dot products $\mathbf{Q}\mathbf{K}^T$ decide *where* to look (routing weights), while the value projection $\mathbf{V}$ decides *what* information to retrieve. Without a separate value projection, a token that serves mainly as a structural marker (such as a preposition or punctuation mark) would pass its entire raw embedding into later layers. Projecting through $\mathbf{W}_V$ lets the network strip away surface token identity and pass along only the features downstream layers need.

For an input sequence of length S projected to a dimension d_{head} (here the full width d; the transformer's multi-head attention later splits that width across several parallel attention computations

[23] **Query-key-value (QKV)**: The terminology is borrowed from information retrieval, explaining why the equation uses three distinct learned projections to calculate pairwise scores. Creating these projections requires three independent weight matrices, costing $3 \times d^2$ parameters per layer. Storing the key and value projections of earlier tokens is what produces the key-value (KV) cache of autoregressive serving (section 6.7.4.2).

called heads), the dot product $\mathbf{QK}^T$ computes all S^2 pairwise token similarities. Scaling by $1/\sqrt{d_{\text{head}}}$ prevents dot products from growing large in magnitude at high dimensions, which would push softmax into regions with vanishingly small gradients. The softmax operator[24] normalizes each row of the score matrix into a probability distribution over key positions. Finally, multiplying these normalized weights by $\mathbf{V}$ produces output vectors formed as weighted combinations of the value representations.

Figure 6.9 traces this dataflow for a sequence of length $S = 6$: cyan query rows match against red key columns to populate the 6×6 similarity matrix, whose rows subsequently weight the green value vectors to produce the contextualized output vectors on the right. Each row of the resulting $S\times S$ matrix determines how strongly an individual token attends to every token in the sequence. In a trained model, the largest affinity values reflect strong contextual dependencies, while near-zero entries suppress irrelevant positions.

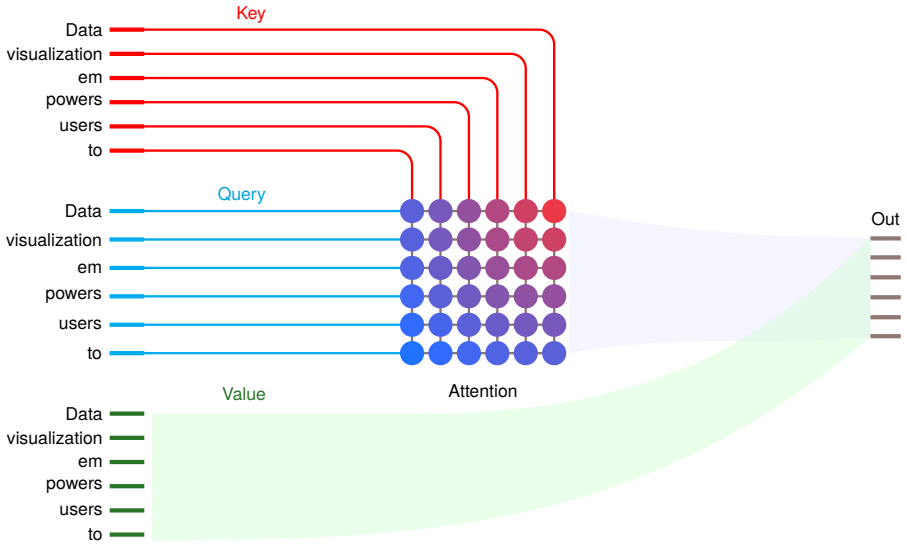


Figure 6.9: Query-Key-Value Attention Mechanism: For a 6-token sequence, queries (cyan) match against keys (red) to produce a 6×6 attention matrix with $\mathcal{O}(S^2)$ entries. The six tokens are the pieces a tokenizer cuts from "Data visualization empowers users to," which is why "empowers" appears as "em" and "powers." Cell color runs from blue to red across the grid as an illustrative gradient rather than as computed weights. Each output position aggregates information from all values (green) weighted by its own row of that matrix. The matrix structure reveals both the computational pattern (thirty-six similarity computations) and the naive memory bottleneck (storing S^2 attention weights). Adapted from Transformer Explainer (Cho et al. 2025).

While attention weights vary dynamically with the input tokens, generating the query, key, and value vectors requires static linear projections against learned parameter matrices. In accelerator implementations, these three operations are fused into a single batched matrix multiplication to maximize Tensor Core saturation and reduce kernel launch overhead. Packing the three weight matrices side by side turns three projections into one (figure 6.10): the 6×768 token embedding matrix (one learned 768-value vector per token) multiplies a 768×2304 weight matrix whose three 768-column blocks hold the query, key, and value weights, and adding the bias yields a 6×2304 output that contains all three projections, computed in a single kernel pass.

6.6.3 Computational mapping

Lowering the attention operator $\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}(\mathbf{QK}^T/\sqrt{d_{\text{head}}})\mathbf{V}$ to physical execution exposes the system cost of data-dependent connectivity. Evaluating pairwise alignments requires two distinct matrix multiplications per attention head: the score GEMM ($\mathbf{QK}^T$) and the context GEMM ($\mathbf{AV}$, where $\mathbf{A} = \text{softmax}(\mathbf{QK}^T/\sqrt{d_{\text{head}}})$). For an input of sequence length S and head dimension d_{head}, each phase executes $S^2 d_{\text{head}}$ multiply-accumulate operations, yielding $\mathcal{O}(S^2 d_{\text{head}})$ computational complexity per head.

[24] **Softmax**: Named as a "soft" (differentiable) version of argmax, with mathematical roots in Boltzmann's statistical mechanics, softmax normalizes each query row over all S scores and therefore requires a reduction over each row. Online softmax lets that reduction run over tiles with running statistics, which is why it need not force quadratic storage (section 6.6.4.1).

↪ The dot product as a similarity measure is collected for reference in section C.1.2.

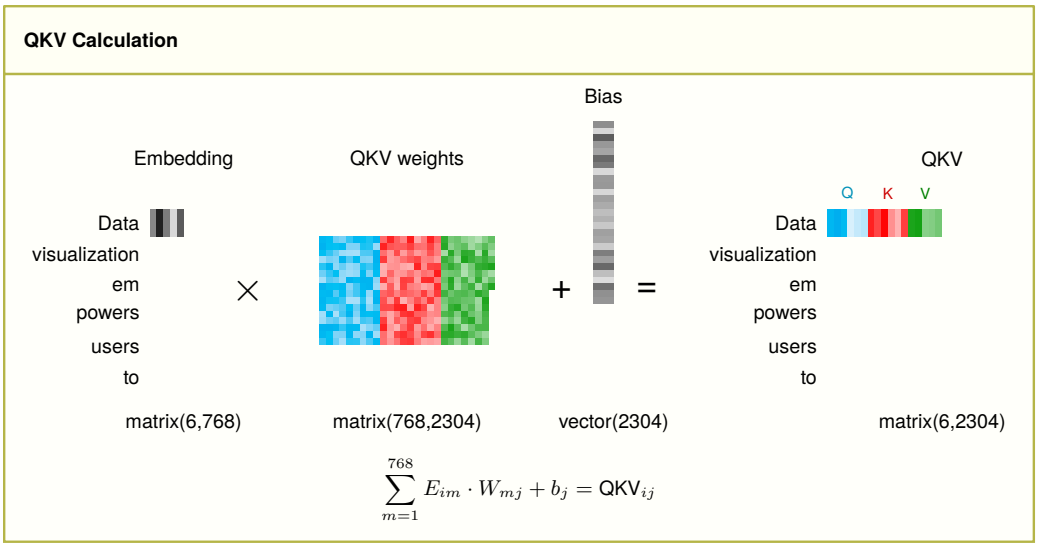


Figure 6.10: QKV Projection Computation: The embedding matrix (6×768) multiplies with QKV weight matrices (768×2304) plus bias to produce combined projections (6×2304). The 2304 output columns hold the query, key, and value projections side by side, 768 each, colored as in figure 6.9; only the first token's output row is drawn. This single batched matrix multiplication, requiring $6 \times 768 \times 2304 = 10.6$ million MACs, replaces three separate projection operations for efficiency. Adapted from Transformer Explainer (Cho et al. 2025).

Listing 6.7 contrasts the high-level matrix formulation with an explicit loop implementation that exposes this quadratic scaling. The matrix version lowers to optimized GEMM routines on accelerator Tensor Cores, whereas the loop version reveals the underlying physical execution structure: batch iterations, quadratic query-key inner products, row-wise softmax reductions, and the final value accumulation pass across all sequence positions.

Listing 6.7: Attention Computation: Two implementations showing the same $\mathcal{O}(S^2 \times d_{\text{head}})$ complexity. The matrix form (top) uses optimized GEMM, while the nested loops (bottom) expose the quadratic pairwise comparisons: for sequence length 512 and head dimension 64, computing attention scores requires $512 \times 512 \times 64 = 16.8$M MACs per attention head, plus another 16.8M MACs for value aggregation.

```
def attention_layer_matrix(Q, K, V):
    # Q, K, V: (batch_size × seq_len × d_head)
    # Compute attention scores
    scores = matmul(Q, K.transpose(-2, -1)) / sqrt(d_head)
    weights = softmax(scores)  # Normalize scores
    output = matmul(weights, V)  # Combine values
    return output

def attention_layer_compute(Q, K, V):
    # Initialize outputs
    scores = np.zeros((batch_size, seq_len, seq_len))
    outputs = np.zeros_like(V)

    # Loop 1: Process each sequence in batch
    for b in range(batch_size):
        # Loop 2: Compute attention for each query position
        for i in range(seq_len):
            # Loop 3: Compare with each key position
            for j in range(seq_len):
                # Compute attention score
                for d in range(d_head):
                    scores[b, i, j] += Q[b, i, d] * K[b, j, d]
                scores[b, i, j] /= sqrt(d_head)

        # Apply softmax to scores
        for i in range(seq_len):
            scores[b, i] = softmax(scores[b, i])

        # Loop 4: Combine values using attention weights
        for i in range(seq_len):
            for j in range(seq_len):
                for d in range(d_head):
                    outputs[b, i, d] += scores[b, i, j] * V[b, j, d]

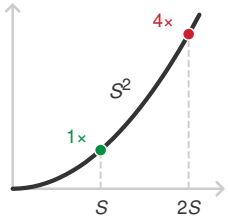
    return outputs
```

6.6.4 System implications

Attention eliminates the latency-bound serial path of RNNs, but a naive implementation replaces it with an $S \times S$ attention-score matrix that grows quadratically with sequence length S. In iron law terms, dense attention trades serial recurrent steps for pairwise score arithmetic and heavy intermediate data movement. Some of these costs are inherent to computing all-to-all interactions; others come only from materializing the full score matrix in memory.

6.6.4.1 Memory requirements

Materializing an $S \times S$ attention-weight matrix for every sequence and head creates a quadratic memory bottleneck that quickly dwarfs both the $S \times d$ activations and the fixed-size projection weights on long inputs. Tiled exact-attention kernels such as FlashAttention stream score blocks through on-chip SRAM and avoid writing the full score matrix to HBM, while still computing all S^2 dense interactions (section 8.6.4 develops the I/O-aware tiling in full, and section C.4.2 collects the online softmax recurrence for reference). This distinction separates the unavoidable quadratic compute cost of dense attention from the avoidable quadratic memory footprint of storing the full score matrix in HBM.



Doubling context quadruples score interactions and naive score storage.

Napkin Math 6.1: The quadratic bottleneck

Problem: How much memory does a materialized attention matrix require at sequence length $S = 100{,}000$ (context window)?

Math:

1. **Matrix size**: The attention score matrix ($\mathbf{QK}^T$) has dimensions $S \times S$ per head, across N_{heads} heads (12 heads in this example).
2. **Elements**: 100,000 × 100,000 × 12 heads = 1.2×10^{11} elements.
3. **Memory**: At FP16 (2 bytes/element): $1.2 \times 10^{11} \times 2$ bytes = 240 GB.
4. **Retained layers**: 240 GB per layer × 32 retained layers = 7,680 GB.

Systems insight: A materialized attention matrix consumes 240 GB per layer. Naive training that retains all 32 layers' scores for backward would require 7,680 GB, far exceeding any single GPU's capacity. This memory wall motivates two broad implementation strategies: avoid materializing the full matrix by tiling the computation, or reduce the number of scores computed in the first place.

6.6.4.2 Computation and data movement

Attention computation splits into two primary GEMM phases interleaved by a row-wise reduction: generating pairwise query-key scores and computing weighted sums over values. Query-key inner products require $B \times N_{\text{heads}} \times S^2 \times d_{\text{head}}$ MAC operations, mirrored symmetrically by the value-aggregation stage. The intervening softmax requires row-wise maximum finding, exponentiation, and summation. Unlike static weight convolutions where filter weights are reused across fixed grid positions, attention computes transient, input-dependent affinity matrices that must be refreshed for every token configuration.

Data movement in attention creates memory access patterns distinct from fixed-stencil operations. Evaluating attention requires projecting and streaming query, key, and value tensors, then routing value vectors according to dynamically evaluated scores. In naive implementations, staging the intermediate $S \times S$ score matrix in global HBM/DRAM turns score materialization into the dominant bandwidth bottleneck. While static convolutions benefit from deterministic cache lines and RNNs reuse weights across time, naive attention generates massive, short-lived activation footprints that challenge standard on-chip cache capacities.

Checkpoint 6.4: Quadratic scaling intuition

Long-context scaling is shaped by the cost of attention. Verify your intuition:

- ☐ **Quadratic growth**: can you explain why doubling sequence length quadruples dense score interactions and a materialized score matrix?
- ☐ **Tiling boundary**: can you explain why long-context dense attention remains more expensive even when tiling removes full-matrix storage?

Attention replaces the serialized hidden-state bottleneck of recurrent models with dynamic, constant-depth routing, but shifts the system burden to score arithmetic and memory bandwidth traffic. Built into a full architecture without recurrence, this primitive enables end-to-end parallel sequence processing on modern accelerators. The transformer architecture (section 6.7) builds directly on this foundation, establishing the dominant architectural paradigm for modern large-scale machine learning systems.

6.7 Transformers: Parallel Sequence Processing

Attention provides the computational primitive of dynamic, content-dependent routing between positions, yet it was originally layered on top of recurrent architectures, inheriting their sequential bottleneck. The transformer architecture removes recurrence and makes attention the operation that mixes information between positions in every layer. This enables parallel computation across

sequence positions during full-sequence processing while retaining dynamic connectivity. This architectural shift introduces distinct physical costs: quadratic attention-score cost over long inputs, a store of per-token state that grows while the model generates text, and memory-bandwidth pressure when it generates one token at a time.

Definition 6.6: Transformers

Transformers are neural network architectures that combine self-attention, feed-forward layers, residual connections, normalization, and positional information without recurrence, enabling parallel processing across sequence positions during training and prompt processing.

1. **Significance**: Parallelism is the systems payoff. An RNN processing an S-token sequence executes S sequentially dependent steps. A transformer processes all positions of a sequence concurrently, mapping token interactions to large matrix multiplications (GEMMs) that maximize accelerator compute utilization. Dense self-attention retains the pairwise score cost (section 6.6.4.1).
2. **Distinction**: Unlike attention used inside a recurrent backbone, which inherits the host architecture's $\mathcal{O}(S)$ sequential path, transformers use attention as the primary mixing operation between positions, alternating it with position-wise feed-forward networks.
3. **Common pitfall**: A frequent misconception is that transformers have "infinite context." Two distinct memory footprints matter: the $\mathcal{O}(S^2)$ attention-score activation matrix when it is materialized over a long input, and the $\mathcal{O}(S)$ key-value (KV) cache that accumulates while the model generates text. At long sequence lengths, the KV cache footprint can rival or exceed the static model weights, turning KV cache management and memory-bandwidth efficiency into dominant engineering constraints.

6.7.1 Pattern processing needs

The recurrent sequence-to-sequence hybrids that first carried attention (Sutskever et al. 2014; Bahdanau et al. 2015) fixed the information bottleneck of section 6.5.5 but not its serialization. Step t still waited on step $t-1$, so a 1,000-token input still paid 1,000 ordered state updates no matter how many execution units the accelerator offered. Removing that chain requires replacing sequential state propagation with all-to-all interactions among positions, giving up the RNN's bounded per-step memory in exchange for parallel accelerator execution. The transformer architecture (Vaswani et al. 2017) establishes self-attention as the primary mixing primitive across positions, paired with positional encodings to inject sequence order, trading the spatial parameter efficiency of CNNs and the bounded $\mathcal{O}(1)$ step state of RNNs for unconstrained sequence parallelism during training and prompt processing.

6.7.2 Algorithmic structure

The core building block of a transformer is the self-attention layer. Unlike encoder-decoder attention, where queries come from the target sequence and keys and values come from the source sequence, self-attention projects queries, keys, and values from the same input sequence $\mathbf{X}$. Drawing all three projections from the same sequence lets the model link tokens across any distance in a single step, without passing state through intermediate recurrent steps. Equation 6.11 writes this operation:

$$\text{SelfAttention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^T}{\sqrt{d_{\text{head}}}}\right)\mathbf{X}\mathbf{W}_V \tag{6.11}$$

Here, $\mathbf{X}$ is the input sequence, and $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are learned weight matrices for queries, keys, and values respectively. Without positional information or an attention mask, matrix multiplication and row-wise softmax operate symmetrically across the token dimension, making self-attention permutation-equivariant: shuffling the rows of $\mathbf{X}$ merely shuffles the output rows identically, treating the sequence as an unordered set. Positional encodings, whether added as sinusoidal vectors or applied via rotary position embeddings (Su et al. 2024), inject sequence order and relative distance.

Transformers extend single-head self-attention by executing multiple attention operations in parallel through multi-head attention. Instead of computing a single set of attention weights over d dimensions, the architecture projects queries, keys, and values into N_{heads} distinct lower-dimensional subspaces of dimension $d_{\text{head}} = d/N_{\text{heads}}$. Each head can learn a different pattern. Their outputs are concatenated and projected through an output matrix $\mathbf{W}_O$, as formalized in equation 6.12:

$$\text{MultiHead}(\mathbf{X}_Q, \mathbf{X}_K, \mathbf{X}_V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_{N_{\text{heads}}})\mathbf{W}_O \tag{6.12}$$

where each attention head is computed as follows. In this multi-head notation, $\mathbf{X}_Q$, $\mathbf{X}_K$, and $\mathbf{X}_V$ denote the source representations before the head-specific projections; for self-attention, all three are $\mathbf{X}$. The head matrices apply the projections once:

$$\text{head}_i = \text{Attention}(\mathbf{X}_Q\mathbf{W}_{Q,i}, \mathbf{X}_K\mathbf{W}_{K,i}, \mathbf{X}_V\mathbf{W}_{V,i})$$

The $\sqrt{d_{\text{head}}}$ scaling introduced with the attention operator follows from a variance argument. Assuming query and key components are independent random variables with zero mean and unit variance, their inner product $\mathbf{q}_i \cdot \mathbf{k}_j = \sum_{m=1}^{d_{\text{head}}} q_{im}k_{jm}$ has zero mean and variance d_{head}, so dividing by $\sqrt{d_{\text{head}}}$ restores unit variance and keeps softmax out of its saturated, near-zero-gradient region at large head dimensions.[25]

[25] **Attention scaling ($\sqrt{d_{\text{head}}}$)**: This normalization directly counteracts the linear growth in variance (d_{head}) of the query-key dot product, preventing the softmax function from saturating where gradients would otherwise vanish. The systems consequence is most acute in mixed-precision training: when activations grow large, unscaled dot products can produce logits that overflow the 16-bit float range, destabilizing or halting learning entirely.

Conceptually, attention implements a differentiable content-addressable memory. Whereas conventional memory architectures retrieve stored values at explicit addresses, self-attention uses the query vector $\mathbf{q}_i$ to score similarity against all candidate keys $\mathbf{K} \in \mathbb{R}^{S \times d_{\text{head}}}$. The resulting softmax distribution generates a continuous interpolation over the value vectors $\mathbf{V} \in \mathbb{R}^{S \times d_{\text{head}}}$, functioning as a differentiable lookup operation.

Although individual attention heads often specialize in distinct syntactic or semantic patterns (Clark et al. 2019), adapting the effective connectivity graph to each input sequence, empirical analyses reveal substantial head redundancy and acute numerical sensitivity in reduced-precision formats. These hardware-relevant behaviors motivate structural pruning, low-rank factorization, sparse attention patterns, and specialized quantization strategies.

To build deep networks from these dynamic projections, the transformer places self-attention inside a repeating block (figure 6.11). Each block wraps multi-head attention and position-wise feed-forward sublayers with residual connections and layer normalization. Sinusoidal positional encodings combine with token embeddings at the base of both towers, feeding repeated multi-head attention and feed-forward stages before terminating in linear and softmax output heads. Because all positions can be processed in parallel rather than sequentially, the architecture trades recurrent state for large matrix operations that map well to accelerator training. A decoder-only model such as GPT-2 keeps only the right-hand tower of this figure, without the attention that reads an encoder's output: tokens enter as embedding vectors, pass through the stack of blocks, and leave as a probability distribution over the vocabulary for the next token. The residual paths and normalization layers in this stack are shared building blocks rather than transformer inventions; section 6.4.1 and section 6.4.2 explain why they make deep stacks trainable and what each costs the machine.

Language models built from this block are usually *autoregressive*: they produce text one token at a time and feed each new token back as input for the next. Given the prompt "The cat sat on," a model predicts "the," appends it, predicts "mat" from the longer context, and continues until it emits an end token. A *causal mask* keeps that process honest. It sets the attention scores for later positions ($t' > t$) to negative infinity before the softmax, so each position attends only to itself and earlier tokens, exactly the information that will exist when the model generates.

The same mask is what lets training run in parallel. During next-token training the whole target sequence is already known, so shifting it right by one position pairs every input token with its successor (figure 6.11), and the mask guarantees that position t cannot see later tokens. An accelerator therefore evaluates every position at once with dense matrix multiplications instead of stepping through a serial time loop as an RNN must.

Generation splits into two phases with different machine profiles, drawn in figure 6.12. *Prefill* processes the whole prompt in one pass: every prompt token is known, so the model evaluates them in parallel, as in training, and stores each token's keys and values. *Decode* then produces the

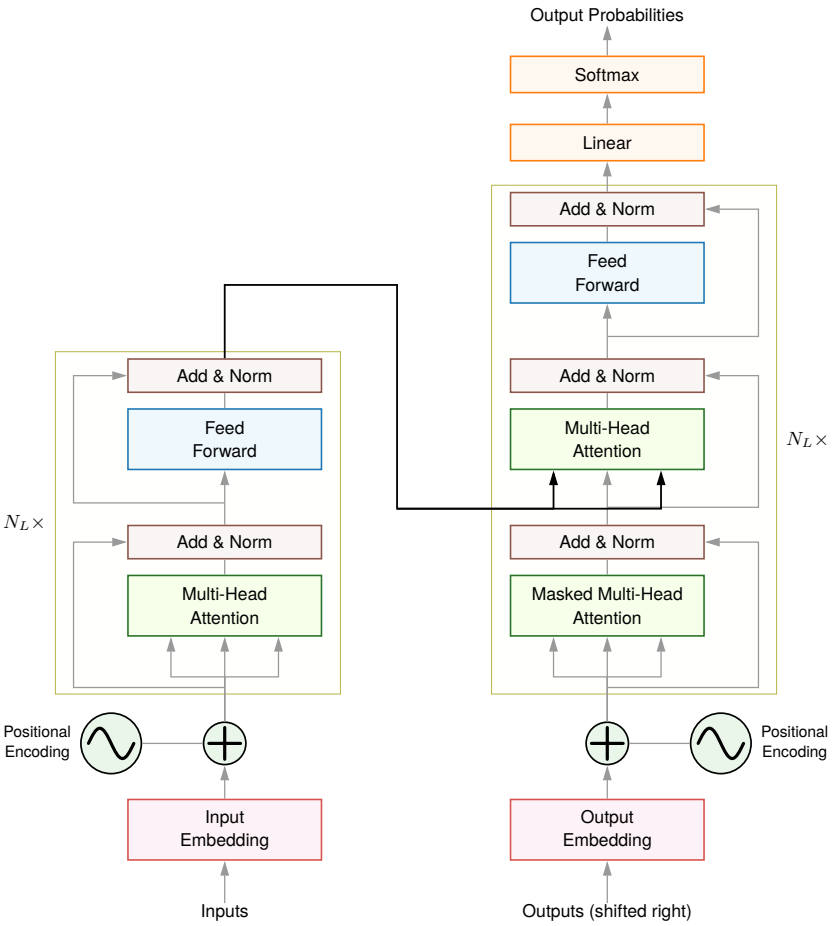


Figure 6.11: Transformer Architecture (Encoder-Decoder): The encoder (left, repeated N_L times) consists of multi-head attention followed by feed-forward layers, each with residual connections (arrows bypassing blocks) and layer normalization. The decoder (right) adds masked attention to prevent attending to future tokens during training and autoregressive generation. Positional encodings (sine waves) inject sequence order information absent from the permutation-equivariant attention operation. This design enables training parallelism across all positions while the decoder maintains autoregressive causality during inference. Adapted from Vaswani et al. (2017).

remaining tokens one per step. Each step feeds a single token through the model, so its projections shrink from matrix-matrix to matrix-vector products, and its attention reads the stored keys and values of every earlier token instead of recomputing them. That store is the key-value (KV) cache, and section 6.7.4.2 shows why decode, not prefill, usually sets a language model's serving cost.

6.7.3 Computational mapping

In code, self-attention is three projections of the same input followed by the attention operator, and multi-head attention runs several smaller copies side by side before a final projection (listing 6.8). Writing both layers side by side shows how the multi-head variant partitions the hidden dimension across heads without changing the per-head attention kernel.

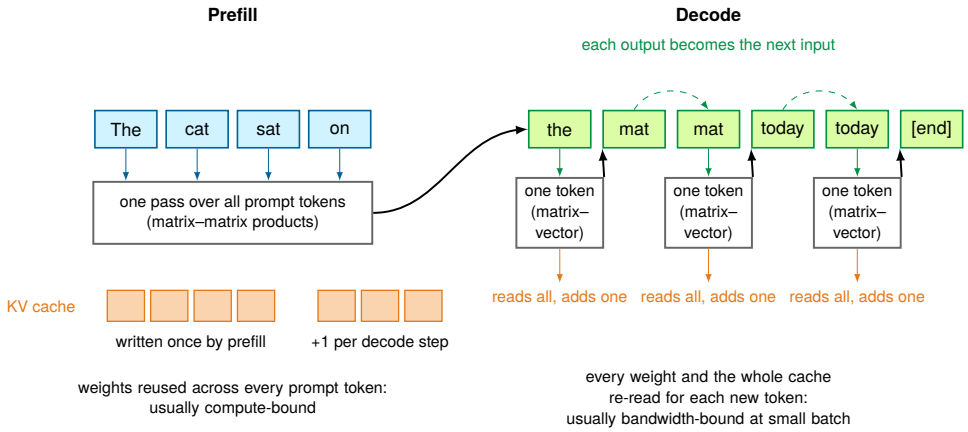


Figure 6.12: Prefill and Decode: Prefill (left) runs the whole prompt through the model in one pass of matrix-matrix products, reusing every weight across all prompt tokens, and writes each prompt token's keys and values to the KV cache. Decode (right) then generates one token per step: each output becomes the next input, each step runs matrix-vector products that re-read every weight, and each step reads the whole cache and appends one entry. The first phase is usually compute-bound; the second is usually bandwidth-bound at small batch.

Listing 6.8: Self-Attention and Multi-Head Attention: Self-attention (top) derives **Q**, **K**, and **V** from the same input **X** through three projections, then computes attention as before. Multi-head attention (bottom) runs N_{heads} parallel attention heads with dimension $d_{\text{head}} = d/N_{\text{heads}}$, then concatenates and projects. For GPT-2 (768-dim, 12 heads), each head operates on 64 dimensions, partitioning the projected feature dimension across heads while enabling diverse relationship patterns.

```
def self_attention_layer(X, W_Q, W_K, W_V, d_head):
    # X: input tensor (batch_size × seq_len × d_model)
    # W_Q, W_K, W_V: weight matrices (d_model × d_head)

    Q = matmul(X, W_Q)
    K = matmul(X, W_K)
    V = matmul(X, W_V)

    scores = matmul(Q, K.transpose(-2, -1)) / sqrt(d_head)
    attention_weights = softmax(scores, dim=-1)
    output = matmul(attention_weights, V)

    return output

def multi_head_attention(X, W_Q, W_K, W_V, W_O, num_heads, d_head):
    outputs = []
    for i in range(num_heads):
        head_output = self_attention_layer(
            X, W_Q[i], W_K[i], W_V[i], d_head
        )
        outputs.append(head_output)

    concat_output = concat(outputs, axis=-1)
    final_output = matmul(concat_output, W_O)

    return final_output
```

The attention listing accounts for information mixing across positions, but a complete block also transforms features within each position. An ordinary position-wise feed-forward network applies a dense projection from hidden width d to an intermediate layer output width d_{out}, a nonlinear activation, and a second dense projection back to d. For a sequence of length S, its tensor shapes follow $S\times d \to S\times d_{\text{out}} \to S\times d$. The same two weight matrices apply independently at every position, so the MLP accounting gives $2Sdd_{\text{out}}$ MACs and an intermediate tensor with Sd_{out} elements.

For one full-sequence block at batch one, with one standard multi-head self-attention sublayer and that two-layer feed-forward network, the query, key, value, and output projections together require $4Sd^2$ MACs. The two attention products add $2S^2d$ MACs across all heads, whose widths sum to d. Adding the feed-forward cost completes the block's matrix-product account; these counts omit biases, softmax, normalization, nonlinear activations, and residual additions, and assume dense evaluation of all position pairs. At fixed widths, projections and the feed-forward network grow linearly with sequence length, while the pairwise products grow quadratically. Which part dominates therefore depends on both the sequence length and the chosen widths, even before memory traffic is measured.

During autoregressive inference, generating output tokens step-by-step reintroduces sequential serialization. At batch one, projection matrix multiplications become matrix-vector operations with less weight reuse, making off-chip bandwidth a likely limit when weights exceed cache capacity, as quantified in section 6.7.4 via the GPT-2 XL lighthouse.

6.7.4 System implications

The quadratic bottleneck analyzed in section 6.6.4 manifests differently across execution regimes. Because full-sequence training and prompt prefill operate on known token sequences under causal masking, accelerators evaluate all positions concurrently using dense matrix multiplications. Autoregressive decoding, by contrast, must generate each token before predicting the next. That loss of token concurrency reduces weight reuse and can shift the primary performance limit from arithmetic throughput to memory bandwidth and state retrieval; sequence length, batch size, and memory hierarchy limits determine whether that transition occurs.

6.7.4.1 Training and prefill

During training and prompt prefill, all token positions are processed in parallel, allowing accelerator Tensor Cores to achieve high floating-point utilization, but dense attention's score computation grows as $\mathcal{O}(S^2)$ with sequence length S. I/O-aware tiling such as FlashAttention (Dao et al. 2022) keeps the score matrix out of HBM (section 6.6.4.1), so the quadratic arithmetic, not its storage, is the wall that remains.

That wall arrives later than its name suggests. By the block account of section 6.7.3, with a feed-forward width four times the model width, a block's projections and feed-forward network cost $12Sd^2$ multiply-accumulates against $2S^2d$ for the attention products, so attention's arithmetic dominates only once the sequence passes about six times the model width, near 9,600 tokens for GPT-2 XL. Shorter training sequences are bound by the ordinary dense matrix multiplies, and the quadratic wall is a long-context phenomenon.

Expanding sequence length S challenges accelerator memory hierarchies because full-sequence self-attention cost grows with context length. Product context windows grew from 2K tokens to beyond 128K and into the 1M-token range within a few years (OpenAI 2023c; Anthropic 2023; Google 2024a), and the decoding discussion below sizes what that growth demands of memory. Longer context windows shift complexity from external retrieval and document chunking pipelines into accelerator memory capacity and bandwidth budgets. Exact tiling changes how much attention state is stored; sparse attention instead reduces the interactions computed (section 6.6.4.1).

Example 6.4: The quadratic wall

Scenario: BERT was pretrained with 512 learned position embeddings, so its deployments enforced a hard 512-token input limit (Devlin et al. 2019).

Diagnosis: Inputs beyond the pretrained positions are undefined for the model, however much memory is available. The limit itself was chosen partly because attention cost grows quadratically with sequence length. Extending from 512 to 4,096 tokens would multiply materialized score memory by 64×.

Systems lesson: Super-linear scaling turns algorithmic complexity into a hardware constraint that becomes fixed in the model itself once pretraining chooses it. No serving-time memory fix extends BERT past 512 positions, so its longer inputs must be truncated or split into windows.

A model meant for long inputs must be pretrained with longer positions, and tiled or sparse attention is what keeps that training and its serving within the HBM activation budget.

6.7.4.2 Autoregressive decoding

Autoregressive decoding generates one token at a time and often becomes memory-bandwidth bound at batch one ($B = 1$) or small batch sizes when weights exceed on-chip memory. During prompt prefill or training, weight reuse across many token positions produces dense GEMMs with much higher arithmetic intensity. A large request batch ($B \gg 1$) supplies many such positions, but prefill can obtain this reuse even for a single request with a long prompt.

During single-token generation ($B = 1$), however, each projection layer collapses into a matrix-vector multiplication (GEMV). In the streamed-weight regime, the processor reads each projection weight from DRAM to multiply it by a single new token vector. This is the batch-one regime that equation 6.2 already derived for the MNIST layer at 0.5 FLOP/byte in FP32. Halving the weight width to 2 bytes in FP16, while still executing one multiply-accumulate operation (1 MAC = 2 FLOPs) per weight read, doubles that weight-only arithmetic intensity to exactly:

$$I_{\text{GEMV}} = \frac{2 \text{ FLOPs}}{2 \text{ bytes}} = 1.0 \text{ FLOP/byte}$$

↩ The 1.0 FLOP/byte intensity of batch-one GEMV in 16-bit precision, and 0.5 FLOP/byte in FP32, is established in section 5.5.3.

Because modern accelerators require tens to hundreds of FLOPs per byte to reach peak compute throughput, running at 1.0 FLOP/byte leaves compute cores idling for the vast majority of execution time, waiting for weights to arrive across the memory bus. To generate each token, the system must first stream every model parameter from off-chip memory into registers (for example, a 70B-parameter model streams 140 GB of FP16 weights per token, and GPT-2 XL makes that cost concrete below), then execute the projection matrix-vector products at an arithmetic intensity capped at 1.0 FLOP/byte, and finally append the new key and value vectors to the KV cache while streaming all historical keys and values from memory.

The KV cache[26] introduced with prefill and decode grows linearly with sequence length ($\mathcal{O}(N_L \times 2 \times N_{\text{heads}} \times S \times d_{\text{head}})$ per request, distinct from the $\mathcal{O}(S^2)$ attention score matrix during training), storing the key and value vectors for all previous tokens to avoid recomputing them (Pope et al. 2023; Kwon et al. 2023). Keys and values from past tokens are retained because the newly generated token's query ($\mathbf{q}_t$) must attend across all historical context ($\mathbf{k}_1 \dots \mathbf{k}_{t-1}$ and $\mathbf{v}_1 \dots \mathbf{v}_{t-1}$) to compute its next representation. Prior queries ($\mathbf{q}_1 \dots \mathbf{q}_{t-1}$) are safely discarded because past tokens never attend forward to future tokens, while caching past keys and values avoids re-running the entire model forward pass over the full prompt at every generated token. For long contexts, this cache can become massive (for example, 100+ GB), and each decoding step reads prior keys and values. The resulting bandwidth pressure depends on context length, batching, attention architecture, cache layout, and hardware.

[26] **KV cache memory scaling**: For a 7-billion-parameter transformer in FP16, model weights consume ~14 GB; a single request's cache requires 32 layers × 2 (K,V) × 32 heads × 2,048 positions × 128 dimensions × 2 bytes, or about 1.07 GB. At 8 concurrent users, the cache is ~8.6 GB, a substantial addition to the weights. Because the cache grows linearly with context and concurrency, scaling throughput forces choices such as grouped-query attention (Ainslie et al. 2023), shorter contexts, or KV paging and offloading (Kwon et al. 2023). Paging and offloading preserve model outputs, whereas grouped-query attention changes the architecture and shorter contexts truncate historical context.

↪ Budgeting KV cache memory across concurrent requests on a serving node is developed in section 13.9.5.

For one model shape, the three memory regimes diverge sharply as context grows (figure 6.13). The weights do not grow with context at all, the KV cache grows linearly with every token a request keeps, and materialized attention scores grow with the square of the context. The cache overtakes the weights near 10,000 tokens, and one layer's scores alone would fill an accelerator near 41,000 tokens, which is why serving systems size the cache per request and training kernels never materialize the scores.

GPT-2 (Radford et al. 2019) anchors the chapter's bandwidth-bound language lighthouse. It is a decoder-only transformer, the same design as today's large language models such as Llama, yet small enough that every weight and every byte of its decoding traffic can be counted by hand. Unlike BERT, an encoder-style transformer that reads its whole context in both directions, GPT-2 generates text one token at a time, so batch-one decoding reads every weight for every token it produces. Two differences from current decoders matter for the machine. GPT-2 gives every attention head its own keys and values, so it stores 307.2 KB of KV cache per token in FP16, while newer models share keys and values across groups of heads (Ainslie et al. 2023) and store several times less per parameter. Batching also amortizes weights across requests but not caches, because each sequence keeps its own, which is why serving at scale binds on KV bytes as well as weight bytes. At GPT-2 XL's size, one decoding step reads its weights in about 0.9 ms on a data-center accelerator, a few times shorter

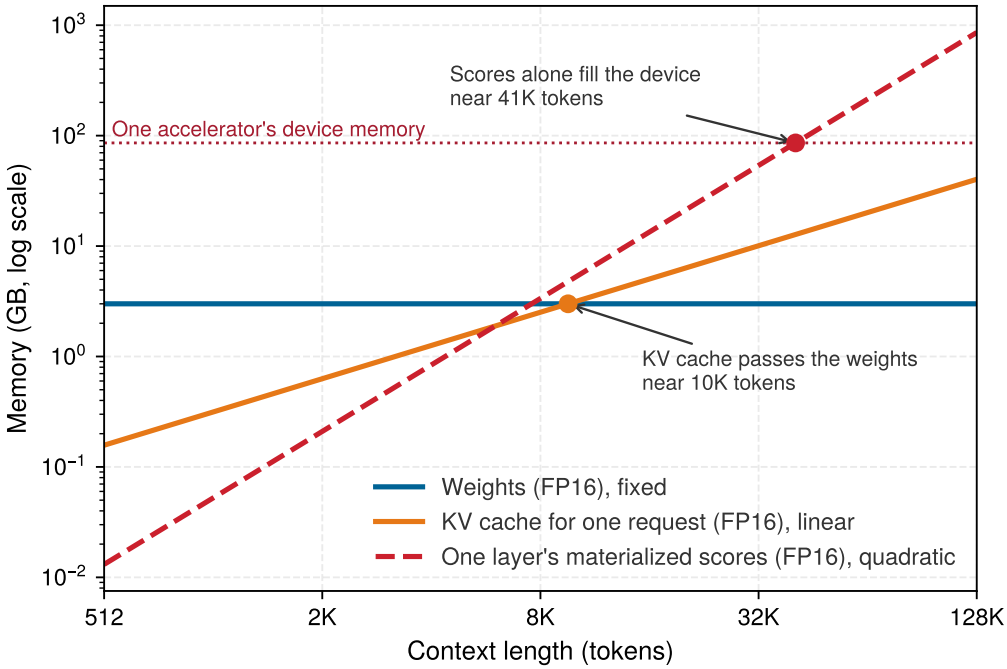


Figure 6.13: Three Memory Regimes of a Long Context: Memory against context length for a model with GPT-2 XL's shape (48 layers, width 1,600, 25 heads) in FP16, evaluated past its trained context to expose the scaling. The weights are fixed, the KV cache that serving keeps for one request grows linearly, and a single layer's materialized attention scores grow quadratically. The cache overtakes the weights near 10,000 tokens, and one layer's scores alone would fill one accelerator's device memory near 41,000 tokens, which is why tiled attention avoids materializing them.

than the time to launch the step's many small kernels (chapter 7), so bandwidth dominates cleanly only as models grow toward tens of billions of parameters.

Lighthouse 6.4: GPT-2 XL (bandwidth lighthouse)

Why it matters: GPT-2 XL exemplifies **memory-bandwidth-bound** batch-one decoding. In the weight-only model used here, each decoding step reads 6 GB of FP32 weights and performs matrix-vector operations. The resulting arithmetic intensity is about 0.5 FLOP/byte with FP32 weights or 1 FLOP/byte with FP16 weights. Batching can amortize this weight traffic, while KV-cache and activation traffic add to it. Table 6.5 summarizes the bandwidth lighthouse's quantitative properties:

Table 6.5: GPT-2 XL Systems Profile: Weight-only estimates for batch-one autoregressive decoding. Under these assumptions, each step reads the model weights at roughly 0.5–1 FLOP/byte, so HBM bandwidth strongly constrains generation throughput.

Property	Value	System Implication
Parameters	1.5B	Weight loading dominates inference latency.
Model Size	6 GB (FP32)	Fits on one GPU but saturates HBM bandwidth.
Compute	3 GFLOP/token	Low per-token compute; bottleneck is data movement, not math.
Constraint	Memory Bandwidth	Weight-bound batch-one tokens/s depends strongly on HBM bandwidth.
Profile	Bandwidth-Bound (small-batch decoding)	Larger batches can amortize weight traffic and shift the bottleneck.

Across training and inference, transformers expose different opportunities for reuse. Full-sequence processing parallelizes across token positions while dense attention adds pairwise score cost, whereas small-batch autoregressive decoding stresses memory bandwidth when weights stream from off-chip memory.

Checkpoint 6.5: Prefill and decode

- ☐ **Two regimes**: why does the same transformer bind on arithmetic during prefill but on memory bandwidth during batch-one decoding?
- ☐ **KV cache**: what does serving keep for every earlier token, why can it discard earlier queries, and how does the cache grow with context length and concurrency?
- ☐ **Long context**: in the memory-against-context figure, which of the three curves sets the limit first, and what does tiled attention change?

Yet dense, spatial, and sequence models do not cover the full spectrum of production workloads. Large-scale recommendation systems depart from standard arithmetic-dominated pipelines by coupling sparse embedding lookups (where memory capacity and irregular access distributions govern throughput) with downstream dense interaction layers. This hybrid matters at production scale: Meta reported that recommendation models accounted for most of its AI inference cycles (Gupta et al. 2020), even though such workloads receive less academic attention than language or vision models.

6.8 Sparse Architectures: Recommendation Systems

When a user opens a streaming service or e-commerce platform, the system must select a handful of recommendations from a catalog of millions within an end-to-end budget of tens of milliseconds. The engineering challenge requires mapping both users and items to dense vectors in a shared embedding space, then evaluating interaction probabilities at scale.

Large-table recommendation models expose a regime governed first by memory capacity. An accelerator engineered for dense matrix multiplication, such as an 80 GB data-center accelerator providing hundreds of teraflops of arithmetic throughput alongside 80 GB of HBM, cannot hold terabytes of embeddings locally. The sparse stage gathers rows from these tables, placing different demands on the machine than the batched convolution and streamed-weight decoding regimes examined above.

6.8.1 Pattern processing needs

A recommendation model must process user IDs across billions of accounts and item IDs across millions of products, yet raw integer IDs carry no inherent distance: user 1042 is no closer to user 1043 than to user 9,000,000, so a dense network cannot infer relationships from the numbers themselves. A single request also carries many such categorical features at once (the items a user recently viewed, the categories they follow), so one prediction can require dozens of separate ID lookups.

Embeddings solve the representation problem by mapping each discrete ID to a dense vector called an **embedding**[27] (Mikolov et al. 2013). The vectors are ordinary learned parameters: each row of an embedding table is a weight vector that gradient descent updates whenever its ID appears in a training example, so users and items with similar behavior end up near one another, and the dot product of a user vector with an item vector becomes a learned score for how well they match. The system cost is that every lookup becomes an irregular read into a table spanning tens or hundreds of gigabytes. Recommendation models therefore encounter a capacity wall and a memory-bandwidth bottleneck before the first dense feedforward layer executes.

6.8.2 Algorithmic structure

The DLRM architecture (Naumov et al. 2019) organizes this computation into a four-stage pipeline split between dense and sparse paths. Continuous features such as user age or time of day first flow through a bottom MLP, a compute-intensive but memory-light stage dominated by dense matrix

[27] **Embedding**: Deriving from the mathematical embedding of one space into another, neural embeddings map discrete tokens (such as user IDs or vocabulary words) into continuous vector spaces where geometric proximity captures semantic similarity. Word2vec popularized neural word embeddings in 2013. For systems, embedding tables create a distinctive memory access pattern: each lookup is an irregular read into a potentially terabyte-scale table, producing a sparse, bandwidth-bound workload that makes DLRM fundamentally different from compute-bound architectures like ResNet.

multiplications. In parallel, categorical IDs index into dedicated embedding tables, which is where the memory capacity wall appears.

For example, an embedding table for one billion users with 128-dimensional vectors requires $10^9 \times 128 \times 4$ bytes ≈ 512 GB of memory. This stage is memory-intensive but compute-light because each lookup is a memory copy with zero arithmetic reuse. The interaction layer then combines the dense activations from the bottom MLP with the gathered sparse embedding vectors through pairwise dot products that capture user-item correlations. Finally, a top MLP processes the concatenated interaction representations through feedforward layers to predict a target probability such as click-through rate. Each of the four stages stresses a different part of the machine (figure 6.14), and section 6.8.3 quantifies the capacity-bound profile that results.

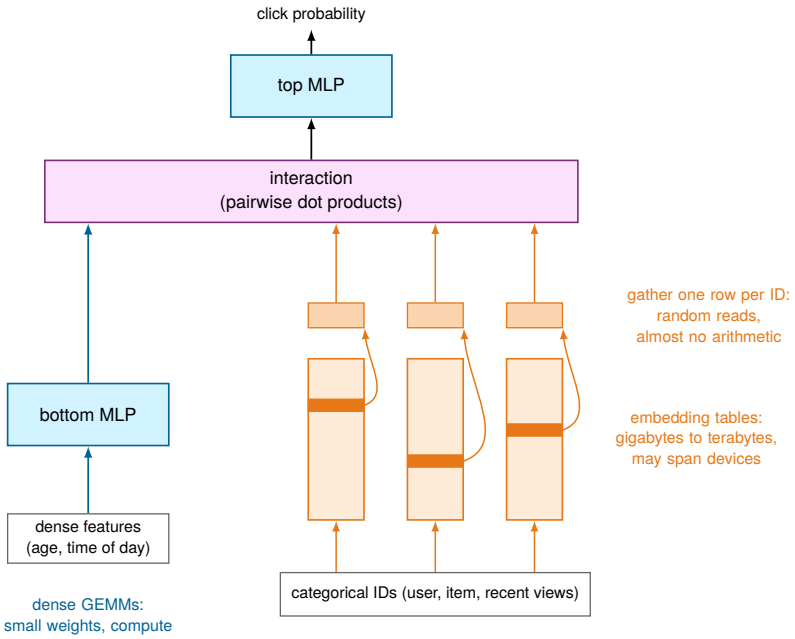


Figure 6.14: DLRM Pipeline: Dense features pass through a bottom MLP of small weight matrices, while each categorical ID selects one row of a large embedding table. The gathers are random reads with almost no arithmetic, and the tables can exceed one device's memory. An interaction stage takes pairwise dot products among the dense output and the gathered vectors, and a top MLP turns the result into a click probability.

6.8.3 Computational mapping and system implications

DLRM's computational mapping splits into two regimes that stress different hardware subsystems. The dense MLPs execute standard GEMM operations, identical to the MLP computational mapping discussed in section 6.2.4 and handled efficiently by Tensor Cores. The sparse embedding lookups, however, are qualitatively different. They are index-based memory gathers that copy parameter data with zero arithmetic reuse. An isolated lookup is bound by memory access latency; but when a batch executes hundreds of lookups across features and samples, aggregate throughput becomes memory-bandwidth bound as concurrent requests saturate the memory bus. This operational throughput limit is distinct from the capacity constraint: total table size dictates whether parameters fit in accelerator memory or must spill across PCIe to host DRAM, whereas available bandwidth governs how fast gathered rows arrive. Within each selected row, an embedding vector is contiguous (for instance, 128 FP32 elements spanning 512 bytes), allowing the downstream pooling loop to consume every transferred byte. Across rows, however, data-dependent categorical IDs lack predictable spatial locality, defeating hardware prefetchers between row accesses. Access popularity remains heavily skewed: a small set of hot rows (popular items, frequent users) shows strong temporal locality in on-chip caches, while the long tail of rarely touched rows misses to main memory. Listing 6.9 writes this sparse stage as a loop nest parallel to those of the other families.

Listing 6.9: Embedding-Bag Gather: Four nested loops expose DLRM's sparse stage. Loops 1 and 2 parallelize across samples and features, but Loop 3 takes its addresses from the input IDs, so hardware cannot predict them, and Loop 4 performs one addition per 4-byte FP32 value loaded. Batching raises that intensity only when samples share hot rows.

```
def embedding_bag_compute(sparse_ids, tables):
    # Loop 1: Process each sample in the batch (parallelizable)
    for b in range(batch_size):
        # Loop 2: Each categorical feature has its own table
        for f in range(num_features):
            pooled[b, f, :] = 0
            # Loop 3: Multi-hot feature, one gather per active ID
            for idx in sparse_ids[b][f]:  # data-dependent address
                row = tables[f][idx]  # irregular read, little reuse
                # Loop 4: Pool (sum) the gathered rows
                for d in range(embed_dim):
                    pooled[b, f, d] += row[d]
    return pooled  # feeds the interaction layer
```

DLRM (Naumov et al. 2019), which Meta open-sourced as a reference for this hybrid architecture, anchors the chapter's recommendation lighthouse. Its combination of gigabyte-scale embedding tables and small multilayer perceptrons serves as the standard benchmark for memory capacity and irregular sparse access in the data center.

↪ The share of DLRM serving latency spent on embedding lookups is developed in section 13.3.2.



Lighthouse 6.5: DLRM (recommendation lighthouse)

Why it matters: DLRM exemplifies **memory-capacity-bound** workloads. Its massive embedding tables often exceed the memory of a single accelerator or server, so the system must decide where those tables live before it can optimize arithmetic throughput. The interaction layer then gathers selected embedding vectors and combines them with dense features, making capacity and irregular data movement the dominant constraints. This capacity-first decision differs from the batched convolution and streamed-weight decoding regimes examined here, requiring different hardware and deployment choices. Table 6.6 summarizes the recommendation lighthouse's quantitative properties.

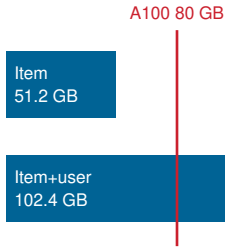
Table 6.6: DLRM Systems Profile: Quantitative properties of the recommendation lighthouse and their system consequences. Embedding tables holding billions of values push the model past single-device memory, creating a capacity-bound regime that decides which bandwidth the iron law's data term must pay.

Property	Value	System Implication
Embedding Parameters	25B	Parameters × 4 bytes; dominates total model size.
Model Size	100 GB (FP32)	May exceed one device's fast memory.
Constraint	Memory Capacity	Model size > Single GPU Memory.
Bottleneck	Irregular Data Movement	Gather operations dominate sparse feature processing.
Profile	Mixed (Sparse/Dense)	Combines memory-heavy lookups with compute-heavy MLPs.

Once embedding tables exceed one device, capacity, not arithmetic, becomes the primary systems constraint. While a ResNet-50 (102.4 MB) fits on a single accelerator and a dense language model like GPT-3 (Brown et al. 2020) (350 GB) fits within a single multi-GPU node, production recommendation models reach terabytes of embedding parameters. In iron law terms, capacity acts as a feasibility constraint that comes before either execution term. It decides where the tables live, and therefore which bandwidth the data term D_{vol}/BW must use: HBM when the tables fit, host DRAM over PCIe or a peer accelerator over NVLink when they do not.

The architecture therefore breaks the single-device assumption that holds for smaller dense models, leaving systems designers three ways to resolve the capacity wall: they can **shrink the tables** through compression, hashing, or pruning at the risk of losing information on long-tail users and items; **move the tables** to host DRAM or external key-value stores at the cost of slower

remote lookups over PCIe or Ethernet; or **partition the tables** across multiple accelerators so that no single device stores the full model, trading local memory pressure for cross-device communication. Architecturally, DLRM turns recommendation into a capacity-management challenge before it becomes a compute-optimization task. Training-time model partitioning and hardware-accelerated interconnects address these distributed layouts, and the baseline sizing calculation begins with a single table.



One embedding table fits an 80 GB A100; a second breaches the capacity wall.

Napkin Math 6.2: The capacity wall

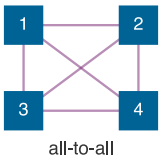
Problem: Consider a recommendation system for a store with 100M items using an embedding size of 128. How much memory does the item table alone require?

Math:

1. **Table entries**: 100M items.
2. **Vector size**: 128 elements.
3. **Precision**: FP32 (4 bytes per element).
4. **Table size**: 100M items $\times$ 128 $\times$ 4 bytes $\approx$ 51.2 GB.
5. **Capacity share**: 51.2 GB $\div$ 80 GB = 64 percent of device capacity.
6. **Two-table check**: Two tables at 64 percent each exceed 100 percent of device capacity.

Systems insight: A single embedding table for one feature (items) already consumes 64 percent of an 80 GB A100 GPU. Adding a user table of the same size means the embedding tables no longer fit on a single 80 GB accelerator, motivating table compression, off-device memory, or partitioning across devices. DLRM is capacity bound: the primary systems question is not how many FLOP/s the accelerator can deliver, but where the embedding state can physically reside.

Once the tables no longer fit on one device, sparse lookups cross memory and interconnect boundaries. A request may need rows owned by several accelerators, forcing the device executing the downstream MLP to wait while the system gathers them across PCIe or NVLink. Within a multi-accelerator server, gathering rows distributed across multiple GPUs turns memory routing into an interconnect bandwidth bottleneck, because every accelerator may need rows held by every other, an all-to-all exchange. The architectural consequence is that a recommendation model's memory layout directly determines its cross-accelerator data-transfer traffic, creating coordination overhead for both training and serving.



Sharded tables can turn lookups into all-to-all accelerator traffic.

↪ Partitioning a model that exceeds one device's memory across accelerators, and the communication that partition boundaries add, is developed in section 8.7.1.2.

Checkpoint 6.6: DLRM and capacity-bound lookups

Recommendation systems stress a different part of the machine than CNNs or transformers.

- ☐ **Capacity-bound**: Can you explain why embedding tables push DLRM into a memory capacity regime where "FLOPs" is not the binding constraint?
- ☐ **Sparse access**: Can you explain why embedding lookups defeat prefetching, and why popularity skew still lets a cache capture the hot rows?
- ☐ **Placement logic**: If a single embedding table does not fit in fast device memory, can you describe which state must move, shrink, or be partitioned?
- ☐ **Data-movement bottleneck**: Can you explain why sparse embedding lookups can dominate even when the dense MLP has modest FLOP cost?

With DLRM, the roster of five families is complete, and each has been analyzed on its own terms: the pattern it targets, the computation it performs, the mapping it induces, and the bottleneck it exposes. A deployment rarely gets to examine one family in isolation. The same accelerator that serves a vision model today may serve a language model or a recommender tomorrow, and the architecture decides which of its resources gives out first.

6.9 Comparing and Choosing Architectures

The three requests that opened this chapter can now be costed in numbers. A single ResNet-50 frame performs about 8.2 GFLOP of arithmetic against 102.4 MB of FP32 weights, and each convolutional filter is reused across its feature map, creating opportunities to keep arithmetic units busy. A single GPT-2 token at batch size one performs about 3 GFLOP/token against 3 GB of FP16 weights. Streaming those weights once gives a weight-only arithmetic intensity of roughly 1 FLOP/byte, making bandwidth a likely limit. A DLRM request gathers a handful of embedding rows from tables totaling 100 GB, more than the accelerator's 80 GB of memory, so before any question of speed arises the system must decide where those tables live.

The silicon is identical in all three cases, but the demands differ. Convolutional reuse favors arithmetic throughput, streamed token projections stress bandwidth, and oversized embedding tables impose a capacity constraint that decides which bandwidth their lookups pay. The iron law (section 1.7) then predicts which term binds, given the execution conditions and machine balance.

Each of those costs comes from a counting rule derived in its family's section. Table 6.7 collects the rules in exact form, so that a new model can be estimated on paper in the same order: count the weights, count the multiply-accumulates, count the bytes that must move or stay resident, then compare those counts with the machine.

Table 6.7: Cost Formulas by Building Block: Counting rules for inference, one per building block, in the notation of the family sections. A FLOP count is twice the multiply-accumulate count; arithmetic intensity divides FLOPs by bytes moved; and a step can finish no faster than the larger of its FLOPs divided by the peak rate and its bytes divided by the bandwidth. For GPT-2 XL, the KV-cache rule gives 307.2 KB per token in FP16, the figure quoted in the transformer section.

Building block	Weights	Multiply-accumulates per example	Bytes that move or stay resident
Dense layer	$d_{\text{in}} d_{\text{out}}$	$d_{\text{in}} d_{\text{out}}$	Every weight read once per batch, so weight-only intensity grows with batch size
Convolution layer	$K^2 C_{\text{in}} C_{\text{out}}$	$H_{\text{img}} W_{\text{img}} K^2 C_{\text{in}} C_{\text{out}}$	Output activations, $H_{\text{img}} W_{\text{img}} C_{\text{out}}$ values per example
Depthwise separable convolution	$K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$	$H_{\text{img}} W_{\text{img}} (K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}})$	Same activations as a standard layer for far fewer operations
Recurrent step	$d^2 + d d_{\text{in}}$	$d^2 + d d_{\text{in}}$ per step, S steps in order (four times for an LSTM)	State of d values at inference; Sd kept for training
Transformer block	$4d^2 + 2dd_{\text{ff}}$	$4Sd^2 + 2S^2 d + 2Sdd_{\text{ff}}$ for S tokens	S^2 scores per head if materialized
Decoding one token	All of the model's weights	About one per weight	Every weight read per step, plus a KV cache of $2N_L d$ values per token
Embedding lookup	Rows × vector width	Almost none (rows are copied)	Whole table resident: rows × width × bytes per value

6.9.1 Arithmetic intensity across workloads

Each family section measured reuse in its own terms: filters reused across a feature map, weights streamed once per decoded token, rows gathered from a table. Arithmetic intensity, the FLOP/byte ratio set against a machine's balance, puts them on one scale and shows whether each workload is likely to be compute or memory bound.

↩ Arithmetic intensity and the system balance it is compared against are established in section 2.4.

The calculation here is a weight-only proxy rather than a complete measurement of main-memory traffic. It divides floating-point work by FP32 model-weight bytes, omitting activation, intermediate, and KV-cache traffic. Batching can amortize weight traffic across examples or tokens, while additional state movement can lower realized intensity. Section C.1.5 gives the per-operation FLOP and parameter formulas used in the proxy. Under these assumptions ResNet-50's proxy exceeds GPT-2 XL's by roughly 160×, the widest gap among the lighthouses in table 6.8.

The intensity proxy is one input to architecture selection; deployment constraints extend beyond the FLOP-to-byte ratio, and batch size, precision, sequence length, and target hardware decide

whether the proxy predicts the deployed bottleneck. Table 6.8 sets the proxy beside parameter footprint, inference cost, and the governing hardware constraint for all five canonical workloads: compute throughput, memory bandwidth, memory capacity, latency, and power. For compute- and bandwidth-bound workloads, the iron law directly maps the primary bottleneck to operations (O) or data movement (D_{vol}). For capacity-, latency-, and power-constrained workloads, physical boundaries dictate where and within what budget those operations must execute, determining which optimization strategies prove effective.

Table 6.8: Lighthouse Model Comparison: Quantitative characteristics and analytical roles of the five canonical workloads. Memory is FP32 weight storage, and FLOPs are per inference except for GPT-2 XL, which is quoted per generated token; the DLRM row reports embedding-table entries rather than dense parameters. Intensity is the weight-only FP32 proxy (FLOPs per weight byte), which omits activation and KV-cache traffic; it is not defined for DLRM's embedding lookups. The bottleneck column indicates the primary system constraint each model reveals: compute-bound models like ResNet stress arithmetic throughput, while bandwidth-bound models like GPT-2 stress memory transfer rates.

Model	Domain	Params	FLOPs/Inf	Memory	Intensity (weight-only)	Bottleneck	Role in Textbook
ResNet-50	Vision	25.6M	8.2 GFLOP	102.4 MB	80.1 FLOP/byte	Compute	Dense vision throughput
GPT-2 XL	Language	1.5B	3 GFLOP/token	6 GB	0.50 FLOP/byte	Mem. Bandwidth	Token-by-token serving
DLRM	Recommender	25B	Low	100 GB	n/a	Mem. Capacity	Embedding tables and capacity planning
MobileNetV2	Edge Vision	3.5M	600 MFLOP	14 MB	42.8 FLOP/byte	Compute	Depthwise convolutions and efficiency
KWS (DS-CNN)	Audio	200K	20 MFLOP	800 KB	25 FLOP/byte	Power	Always-on power budget

Architecture selection is ultimately an engineering trade-off between math (O) and memory movement (D_{vol}). The mechanisms behind the intensities in table 6.8 explain why each lighthouse sits where it does on the intensity spectrum. ResNet-50 earns its high intensity because convolutional layers reuse each weight many times across the spatial dimensions of an image (individual deeper ResNet layers reach 100–200+ FLOP/byte). Arithmetic throughput becomes the limit when effective intensity exceeds the target machine's compute-to-bandwidth balance. GPT-2 sits at the opposite extreme during batch-one decoding with streamed weights: each projection loads weights for one token's matrix-vector product, making bandwidth a likely limit. MobileNet lands between the two at the whole-model level, with individual depthwise layers falling lower once activation traffic is included: depthwise separable convolutions reduce total O but move more data relative to that work, which fits mobile hardware well yet often "starves" high-end GPUs optimized for dense math.

Checkpoint 6.7: Arithmetic intensity and architecture

Match the architectural choice to its systems implication:

- ☐ **One accelerator, three requests**: can you say which iron-law term binds a batched ResNet-50 frame, a batch-one GPT-2 token, and a DLRM lookup, and name one change that would move each?
- ☐ **Batching**: why does batching move the dense-layer and decode workloads toward the machine's balance point while leaving the embedding gather's bottleneck in place?

Comparing this spectrum with the target machine's balance helps determine whether faster arithmetic or faster memory would improve performance. The roofline model (Williams et al. 2009) plots achievable throughput against arithmetic intensity to show where a workload turns from memory bound to compute bound. This chapter needs only the comparison introduced with

system balance, a workload's intensity set against the machine's ratio of peak compute to memory bandwidth. The full construction, including the ridge point of an H100 and how the same operation falls on either side of it, is collected for reference in section D.2.1.1.

↪ The roofline model's analytical treatment, from the memory-bound slope to the ridge point, is developed in section 11.6.

Intensity is one column of a fuller profile. Persistent state, per-request state, the axis along which the work parallelizes, and the iron-law term that binds complete the comparison.

6.9.2 Workload profiles

Table 6.9 collects that fuller profile. Each row is a family in a representative execution regime; the transformer takes two rows because prefill and decode reuse the same weights differently (section 6.7.4.2). The binding-term column applies the workload archetypes of section 2.3.2 to each family. The compute-bound rows assume sufficient reuse to exceed the machine's balance, the bandwidth-bound rows assume streamed operands, and the capacity-bound row assumes oversized tables. The columns follow a request through the machine: persistent state, per-request state, arithmetic intensity, parallel axes, and the likely limiting term under those assumptions.

Table 6.9: Architecture Workload Profiles: State, per-request activations, intensity regime, parallel axis, and binding iron-law term for each family in its common execution regime. Symbols follow the family sections: B is batch size, S sequence length, d the hidden dimension, d_{in} and d_{out} a layer's input and output widths, d_{ff} the feed-forward width, K kernel size, C_{in} and C_{out} input and output channels, and $H_{\text{img}} \times W_{\text{img}}$ the feature-map size. Intensity regimes are qualitative; table 6.8 gives weight-only proxies for the lighthouses. The $\mathcal{O}(BS^2)$ prefill term assumes materialized attention scores (section 6.6.4.1).

Family	Parameters	Activations	Intensity	Parallel Axis	Binding Term	Lighthouse
MLP	$\mathcal{O}(d_{\text{in}} d_{\text{out}})$ per layer	$\mathcal{O}(B\, d_{\text{out}})$	Low at batch one (GEMV); high when batched (GEMM)	Batch	D_{vol}/BW at small batch	None (DLRM's dense MLPs)
CNN	$\mathcal{O}(K^2 C_{\text{in}} C_{\text{out}})$ per layer, independent of image size	$\mathcal{O}(B\, H_{\text{img}} W_{\text{img}} C_{\text{out}})$	High (filters reused across positions); lower for depthwise	Batch and spatial positions	$O/(R_{\text{peak}}\, \eta_{\text{hw}})$	ResNet-50; MobileNetV2 and KWS
RNN	$\mathcal{O}(d^2 + d d_{\text{in}})$, shared across steps	$\mathcal{O}(B\, d)$ state; $\mathcal{O}(BSd)$ in training	Low at batch one; higher with batching	Batch and within-step work; recurrent time is serial	Serial step cost; dispatch adds L_{lat} unless fused	None
Transformer (prefill)	$\mathcal{O}(d^2 + d d_{\text{ff}})$ per block	$\mathcal{O}(BSd)$; $\mathcal{O}(BS^2)$ if scores are materialized	High (GEMM over all S tokens)	Batch and sequence positions	$O/(R_{\text{peak}}\, \eta_{\text{hw}})$, with an S^2 attention term	GPT-2 (prompt)
Transformer (decode)	Same weights, reread for every token	KV cache, $\mathcal{O}(BSd)$ per layer	Low (GEMV per token at small batch)	Batch only; tokens are serial	D_{vol}/BW (weights and KV cache)	GPT-2 (generation)
DLRM	Embedding tables, one row per category, dominate; dense MLPs are small	Pooled embedding vectors per feature	Very low (gathers with little arithmetic)	Batch; tables sharded across devices	Capacity first, then D_{vol}/BW	DLRM

Table 6.9 states memory as growth rates; for the lighthouses the bytes are concrete (table 6.10). Deployment usually stores weights in a narrow format, so INT8 cuts every row by four. Training moves the other way: each FP32 weight also carries a gradient and two optimizer moments, so the parameter state alone is four times the FP32 weights before any activation is saved (section 8.3.2.3 itemizes this 16 bytes per parameter), and a model that serves from one accelerator may not train on it.

Table 6.10: Lighthouse Memory: Bytes for each lighthouse's parameters at serving and training precision. Training state counts FP32 weights, gradients, and the two Adam moments, and excludes saved activations, which add to it with batch size; the fit column compares that state with one 80 GB accelerator. The last column names the per-request state that sits on top of the weights.

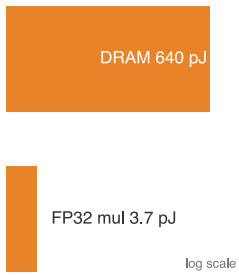
Model	FP32 Weights	INT8 Weights	Training State (16 Bytes per Parameter)	Fits One Accelerator for Training	Grows per Request With
ResNet-50	102.4 MB	25.6 MB	409.6 MB	Yes	Feature maps for each image in the batch
GPT-2 XL	6 GB	1.5 GB	24 GB	Yes	KV cache, 307.2 KB per token in FP16
DLRM	100 GB	25 GB	400 GB	No	Gathered embedding rows for each lookup
MobileNetV2	14 MB	3.5 MB	56.1 MB	Yes	Feature maps for each image
KWS (DS-CNN)	800 KB	200 KB	3.2 MB	Yes	Feature maps for each audio window

The parameter and activation columns of table 6.9 decide how data reaches the arithmetic units, and the pattern of access matters as much as the volume. Three patterns cover the table. Sequential access, the streaming of contiguous weight rows in an MLP layer or a transformer projection, lets DRAM deliver long bursts and lets hardware prefetchers run ahead, so it approaches the interface's peak bandwidth. Strided access, which a convolution produces when its window steps across a layout that does not keep the window contiguous, fetches whole cache lines to use a fraction of each; im2col lowering (section 6.3.3) moves that gather into a separate copy pass so the matrix multiply itself reads sequentially, at the cost of up to K^2-fold duplication (because overlapping sliding windows copy the same input pixels into multiple rows). Random access characterizes the DLRM row. Sparse IDs index arbitrary rows across multi-gigabyte tables; while each selected embedding vector is contiguous and fully consumed by pooling, row selection lacks predictable spatial locality between rows, which defeats hardware prefetchers and leaves sparse lookups exposed to memory latency unless many concurrent requests are in flight. Achieved bandwidth depends on vector width, alignment, and request concurrency, which is why caching popular rows is critical to reduce off-chip traffic. Attention is not random access despite its content-dependent weights, because its queries, keys, and values sit in contiguous tensors that tiled kernels stream through on-chip SRAM.

Whether an access incurs DRAM traffic at all depends on the working set, the data the execution units need at once. Small layers, such as the 784×128 reference layer at 401.4 KB in FP32, fit in on-chip SRAM. Production-width layers of tens of megabytes do not, which is why unbatched MLP inference and transformer decode stream their weights from off-chip memory on every request. Accelerators answer both ends of the table with memory hierarchy rather than arithmetic. HBM stacks DRAM dies beside the processor to supply the streaming rows at more than an order of magnitude above standard DDR channels, and multi-megabyte on-chip SRAM, often an explicitly managed scratchpad, lets compilers stage tiles so that reuse happens on chip. The intensity column measures how much a family can profit from that staging, and it doubles as an energy column.

Systems Perspective 6.2: Reuse sets each family's energy cost

Moving a 32-bit value from DRAM costs far more energy than one arithmetic operation on it (table 4.6 in section 4.9.3); against the full multiply-accumulate that a dense layer performs, the gap is about 139×, the basis used for the MLP and RNN above. How much of that cost each family pays depends on reuse, so low-intensity rows are energy-dominated by DRAM traffic. An unbatched MLP or a decode step loads each weight once per request and spends most of its energy moving it. A CNN amortizes each filter load over $H_{\text{img}} \times W_{\text{img}}$ applications, so the locality that makes it compute bound also makes it energy efficient, unless a fully materialized im2col buffer inflates temporary traffic by up to K^2. An RNN keeps its compact state in SRAM



Moving a 32-bit value from DRAM costs far more energy than one FP32 multiply.

but idles execution units across serial steps, which raises energy per token. Tiled attention keeps score blocks on chip, so its energy tracks matrix multiplication rather than DRAM round-trips.

The parallel-axis column determines how data must move among processing elements once the work is spread out. Four data movement patterns (broadcast, scatter, gather, and reduction; figure 6.15) govern how tensors travel among processing elements, whether those are units on one chip or several accelerators in one server. Batch parallelism in MLPs, CNNs, and prefill broadcasts stationary weight tiles to many units, each working on a different example or position, and compilers tile loops to reuse each broadcast value as often as possible. Spatial and model partitioning scatter a tensor across tiles or devices; mixture-of-experts models, which route each token to a few of many feedforward subnetworks, make the scatter input-dependent, so a popular expert can overload one accelerator while others idle. DLRM gathers embedding rows from wherever the sharded tables live, and each attention query gathers keys and values from every position in its sequence. Reductions combine values through sums or maxima (softmax denominators, normalization statistics, and the summing of gradients across accelerators that each trained on a different slice of the batch), and tree-structured reduction hardware finishes a reduction over n values in $\mathcal{O}(\log n)$ stages rather than $\mathcal{O}(n)$. The rows with a serial axis, RNNs and decode, are the rows these collectives cannot rescue, because no amount of broadcast or reduction bandwidth shortens a dependency chain.

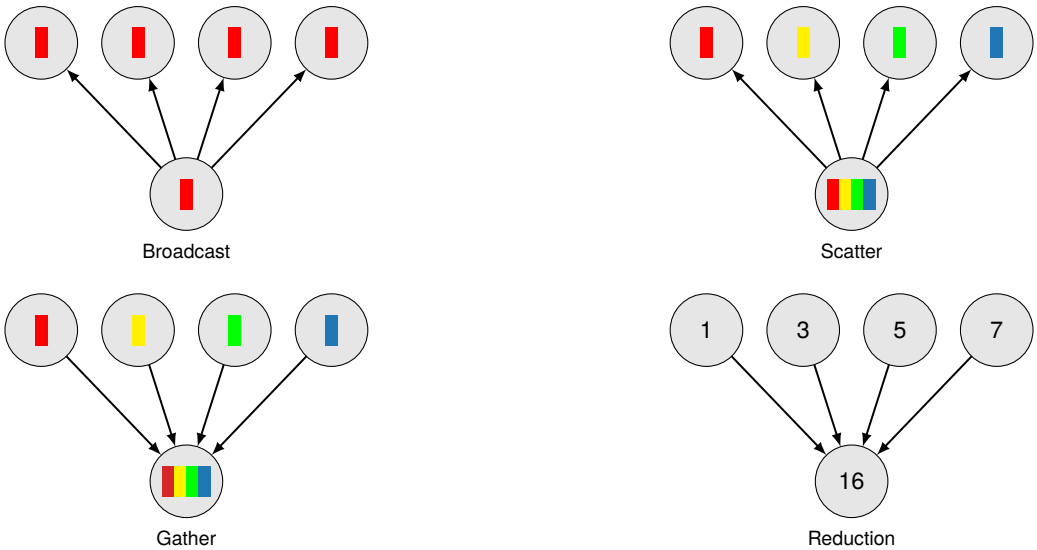


Figure 6.15: Data Movement Primitives: Four fundamental patterns govern information flow in neural network computation. Broadcast (top-left) replicates a single value to all destinations, used when sharing weights across batch elements. Scatter (top-right) distributes distinct elements to different destinations, enabling work partitioning. Gather (bottom-left) collects distributed values to a single location, as in attention pooling. Reduction (bottom-right) combines multiple values through aggregation (sum, max), appearing in gradient synchronization and attention scoring. Because moving data costs more than computing on it, these patterns are primary optimization targets.

The last column applies the iron law under each row's execution assumptions. When effective intensity exceeds the machine's balance and parallel work is sufficient (batched CNNs, prefill), the compute term $O/(R_{\text{peak}}\,\eta_{\text{hw}})$ binds. Below that balance (streamed small-batch MLPs, decode), the data term D_{vol}/BW binds, favoring fewer transferred bytes, more reuse, or higher bandwidth. RNNs add a serial chain of state updates: faster arithmetic, memory, and reduced launch overhead (L_{lat}) can shorten each step without removing the dependency. DLRM adds a constraint that sits in front of the equation. Capacity decides where the tables live and thus which bandwidth the data term pays. ResNet-50, GPT-2, and DLRM isolate these representative regimes. MobileNetV2 and KWS share the CNN row, with depthwise layers pulling their intensity down; they stand for latency and power, deployment budgets that bound the whole execution rather than one term. No lighthouse occupies the RNN row, whose serial chain the recurrent family section analyzes directly.

6.9.3 Shared building blocks

The rows of table 6.9 differ, but the operators inside them repeat. Each block was taught with the family that first needed it: dense projections with MLPs, skip connections and normalization with ResNet-50 (section 6.4.1 and section 6.4.2), and gating with recurrent networks (section 6.5.3). Table 6.11 connects each building block to a representative context and to other architectures that use it. Each block brings its hardware cost with it, so an optimization for residual additions or normalization kernels can transfer across families.

Table 6.11: Cross-Architecture Building Blocks: Engineering primitives illustrated by one architecture family and reused across many designs. The example contexts identify familiar applications rather than historical firsts. Each building block solves a recurring problem (gradient flow, activation stability, signal routing), though not every architecture uses every primitive.

Building Block	Example Context	Problem Solved	Now Used In
GEMM	MLPs	Universal function approximation	All architectures (feedforward layers)
Parameter Sharing	CNNs	Spatial efficiency	Transformers (shared projections), RNNs (weight reuse across time)
Skip Connections	ResNets (CNNs)	Gradient flow at depth	Transformers, DenseNets, U-Nets, many modern deep networks
Normalization	CNNs (BatchNorm)	Activation stability	LayerNorm (Transformers), RMSNorm (root-mean-square normalization), GroupNorm (grouped channels)
Gating	LSTMs (RNNs)	Selective signal routing	Transformers (mixture-of-experts routing), GRUs, highway networks

6.9.3.1 Dense projections

General matrix multiplication is the one primitive every row of the profile shares. It is the feedforward layer inside every transformer block, the 1×1 pointwise convolution in MobileNets, the input and recurrent projections inside every RNN cell, and the bottom and top MLPs of DLRM. Convolution reaches the same path through the im2col patch unfolding of figure 6.5, and dedicated matrix engines, such as GPU Tensor Cores and the systolic arrays of TPUs, exist because this one primitive dominates the operation count of nearly every workload. Its hardware cost depends on shape rather than on the family that invokes it. Batching therefore acts on every family that reaches this path as it acts on the dense layer measured in section 5.4.2.4, converting a matrix-vector product into a matrix-matrix product with greater reuse. The higher intensity can cross the target machine's balance point, which is why batch size changes the modeled regime of three rows of the table.

Dense connectivity supplies a common cost baseline, but each family changes a different part of the workload. Convolution shares filters across spatial positions; attention combines dense projections with pairwise interactions; and embedding lookup replaces an arithmetic transformation with an indexed read. Their parameter storage, operation counts, and transferred bytes must therefore be compared separately, as in table 6.9. Dense projections remain inside all these architectures, so improvements to their execution can transfer between families.

6.9.3.2 Gating, routing, and the transformer block

Gating generalizes beyond recurrence as the basic mechanism for data-dependent routing. Highway Networks (Srivastava et al. 2015) used gates to interpolate between a transformation and an identity bypass, a gated precursor of the residual shortcut. Encoder-decoder attention (Bahdanau et al. 2015) computes a dynamic routing distribution over input representations, and in transformers the softmax attention weights act as data-dependent routing coefficients that decide how features aggregate across tokens. Mixture-of-experts architectures extend the idea to dispatch each token to a subset of feedforward experts (Jacobs et al. 1991; Shazeer et al. 2017), which separates the weights a token reads, and so its FLOPs and bytes, from the model's total capacity. The hardware cost of gating is irregularity. Dense attention keeps static tensor shapes even though its weights vary per token, but sparse attention, expert routing, and dynamic graph execution introduce data-dependent control flow and runtime tensor sizes that defeat static memory allocation and balanced work dispatch.

↪ The irregular memory access that unstructured sparsity, embedding lookups, and dynamic routing impose on hardware is developed in section 11.5.2.1.

A transformer block is where all four building blocks meet. In the encoder tower of figure 6.11, residual paths bypass each multi-head attention and feed-forward sub-layer to preserve gradient flow

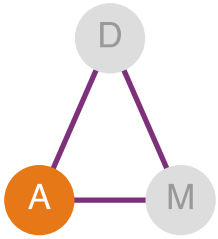
through deep stacks, dense GEMM projections carry the attention and feedforward transformations, LayerNorm or RMSNorm stabilizes activations around each sub-layer, and softmax attention weights gate information across positions. What the transformer changed was the dependency structure. Replacing recurrence with all-to-all attention cut the sequential depth for tokens to interact from $\mathcal{O}(S)$ steps to $\mathcal{O}(1)$, which let the other three blocks run at full width on parallel hardware. Vision Transformers[28] carry the same block set to images (Dosovitskiy et al. 2021).

This portability separates optimizations that transfer from those that do not. GEMM tiling and mixed-precision arithmetic help every row of table 6.9, residual memory management helps every deep network, and normalization kernel fusion helps CNNs and transformers alike. Attention-specific kernels remain tied to attention's memory pattern, but even they are built from the same GEMM and memory-access primitives. The profile and the building blocks describe what each family costs; choosing among families for a particular deployment requires turning those costs into a procedure.

28 **Vision transformers (ViTs)**: Google's 2020 ViT paper split 224×224 images into 16×16 patches (196 "tokens") and applied standard transformer attention. ViTs replace CNN's local convolutions with $\mathcal{O}(S^2)$ global attention over patch tokens. In the original study, large-scale pretraining improved ViT's competitiveness, illustrating how data and compute can compensate for weaker spatial inductive bias (Dosovitskiy et al. 2021).

6.9.4 Architecture selection framework

A wildlife monitoring sensor must classify camera-trap images within a watt-scale solar power budget, while an industrial recommendation service must retrieve terabyte-scale embeddings within a tail-latency budget of tens of milliseconds. Each constraint eliminates architectures that look competitive on unconstrained benchmarks, and a structural mismatch at this stage cannot be rescued later by compiler optimization or kernel tuning. Engineering teams nonetheless often default to architectures by popularity, deploying over-parameterized transformers where a lightweight CNN would suffice or selecting models that violate the inference latency budget. In D·A·M terms, architecture selection chooses the algorithm (A) that bridges the structure of the data (D) to the budgets of the machine (M): memory capacity, memory bandwidth, compute throughput, and thermal envelope.



Architecture selection turns on the algorithm axis, bridging data to machine.

Selection begins with data-to-architecture mapping, matching the symmetries of the input to an operator family. The useful strength of an inductive bias depends on the task (section 6.1). CNNs impose local connectivity, weight sharing, and translation equivariance, reducing parameter volume and exposing reuse for grid-like data. RNNs impose temporal recurrence, bounding inference state across sequence lengths at the cost of serialization. Transformers use content-based attention, gaining flexible interactions while paying for dense quadratic score computation. MLPs impose no specific spatial or temporal connectivity pattern, leaving those relationships to be learned. Ordered by the strength of their assumptions, the families therefore run from CNNs through RNNs and transformers to MLPs; the margin figure beside the inductive-bias definition draws the same ordering for CNNs, transformers, and MLPs. The stronger the prior, the fewer samples and the less computation a matching task needs, and the more a mismatched task loses, because a strong prior can exclude useful solutions as easily as it excludes useless ones. Model selection must test both predictive fit and the resulting machine costs, and the history of the Netflix Prize shows what happens when only the first test is run.

War Story 6.1: The ensemble Netflix did not ship (2009)

Context: Netflix awarded its $1M Grand Prize in 2009 for a 10 percent improvement in recommendation accuracy (Johnston 2012).

Mechanism: Bringing the winning improvement into production required substantial engineering effort just as Netflix's business was shifting from mailed DVDs to streaming. That shift also produced richer viewing signals and changed what personalization needed to optimize.

Impact: The additional accuracy did not justify the production effort, so Netflix did not deploy the Grand Prize code.

Fix: Netflix retained two algorithms from the earlier Progress Prize and redirected its recommendation work toward streaming-era personalization.

Systems lesson: An offline metric gain is only valuable when its integration cost and objective still match the production system.

Figure 6.16 formalizes this selection pipeline as a multi-stage triage tree: candidate architectures route first by input data structure (text, sequences, spatial grids, or tabular features) and then pass through sequential screening gates for memory capacity, compute budget, inference latency, accuracy targets, and hardware deployment readiness. High-cardinality recommendation workloads sit outside this dense flowchart and route to the sparse embedding family (section 6.8) before applying the same memory, compute, speed, accuracy, and deployment checks. If any constraint fails, the decision loop routes execution back to scale the model down, increase its capacity when accuracy falls short, or substitute an alternative architecture family.

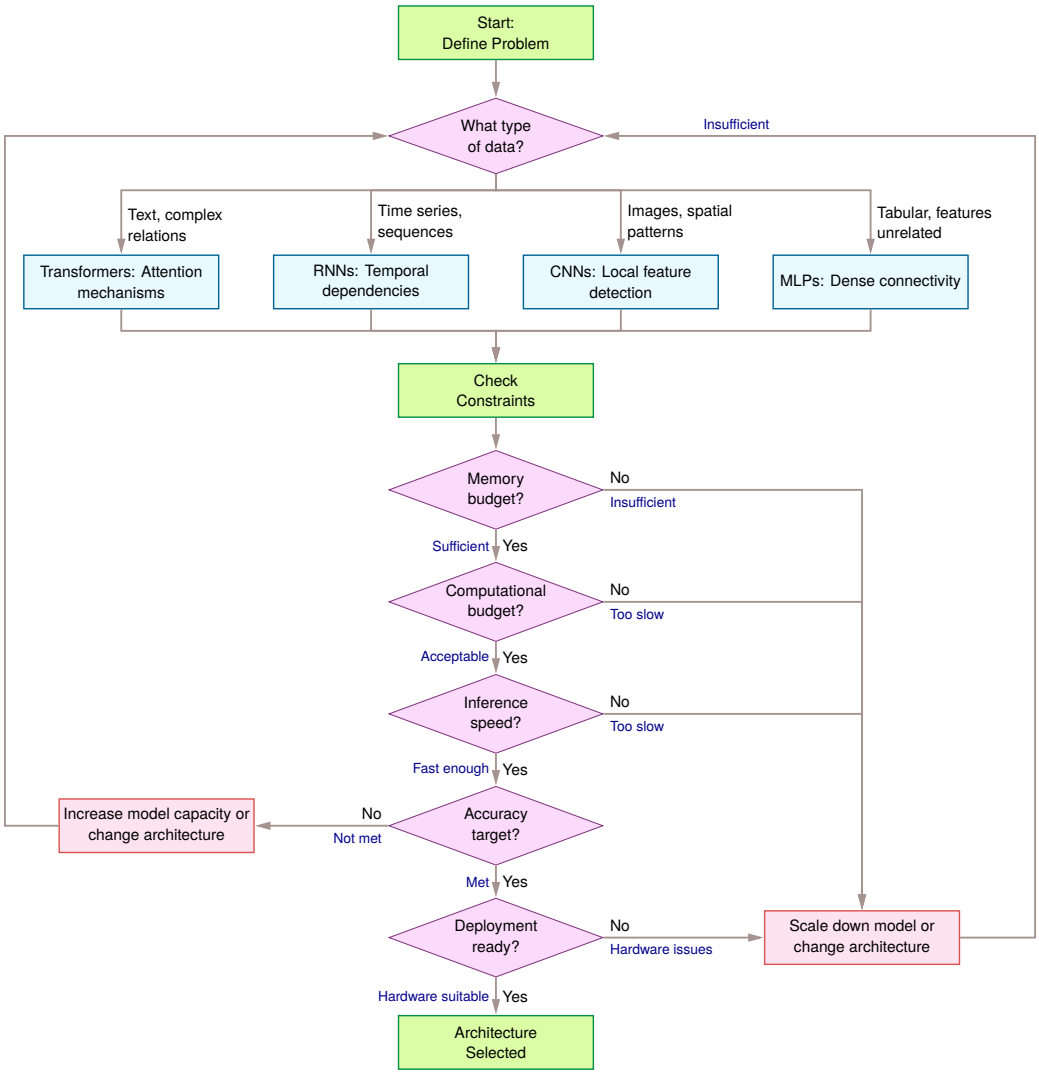


Figure 6.16: Architecture Selection Decision Framework: A systematic flowchart for choosing dense neural network architectures based on data characteristics and deployment constraints. The process begins by identifying the data type (text, sequences, images, or tabular features) to select initial candidates (transformers, RNNs, CNNs, or MLPs), then iteratively evaluates memory budget, computational cost, inference speed, accuracy targets, and hardware compatibility. High-cardinality recommendation workloads are handled separately by the sparse-embedding family, DLRM (section 6.8), then re-enter the same constraint checks.

The framework executes in three ordered phases, with model compression as the systematic way to scale a candidate down when it fails a budget:

1. **Data symmetry identification (D)**: Identify structural symmetries and invariances in the input data. Spatial locality routes candidates to CNNs; temporal sequence structure to RNNs; uncon-

strained cross-token dependency to transformers; tabular features without spatial geometry to MLPs; and high-cardinality categorical IDs to sparse embedding tables.

2. **Progressive physical constraint filtering (A × M)**: Filter candidate models against hardware budgets: static parameter storage in flash/DRAM, peak dynamic activation footprint in SRAM/HBM, arithmetic intensity against the hardware roofline, and tail latency deadlines. A violation requires scaling down model width or depth, applying quantization and compression, or switching to a lower-overhead architecture family.
3. **Empirical capacity and deployment validation**: Verify that candidate capacity meets target accuracy on representative evaluation distributions. If predictive accuracy falls short, scale model capacity and re-evaluate physical hardware constraints. If target accelerator hardware cannot sustain the required arithmetic throughput or memory bandwidth, select an alternative architecture rather than relying on marginal hyperparameter adjustments.

Kernel-level choices refine this decision. Im2col or direct convolution, fused normalization, and IO-aware tiled attention (section 8.6.4) change realized traffic and utilization, and can shift which iron-law term binds; chapter 7 and chapter 11 develop them. Architectural dependencies still matter. Batching independent recurrent sequences or fusing their operations reduces per-step cost, while the ordered state transitions remain. Candidate selection and kernel measurements therefore inform each other.

6.9.5 Architecture selection in practice

Every selection must first pass a feasibility check: whether the target hardware can sustain the required frame rate at all. A server-class vision pipeline on a mid-range GPU shows the throughput ceiling calculation in its simplest form. The wildlife monitoring deployment that follows applies the same checks under a watt-scale budget where GPUs are ruled out, which is where data structure, model capacity, and hardware budgets interact.



Napkin Math 6.3: The throughput ceiling

Problem: A real-time application must process 30 FPS of video with ResNet-50. What sustained compute throughput is required?

Math:

1. **Model cost**: ResNet-50 requires ~8.2 GFLOP per 224×224 image.
2. **Frame rate**: 30 FPS required.
3. **Sustained throughput**: 30 FPS × 8.2 GFLOP = 246 GFLOP/s.
4. **Effective throughput**: 10 TFLOP/s peak × 50–60 percent utilization = 5 TFLOP/s–6 TFLOP/s.
5. **ResNet-50 headroom**: 5 TFLOP/s ÷ 246 GFLOP/s = 20.3×.
6. **Detection-model check**: 30 FPS × 100 GFLOP = 3 TFLOP/s, leaving 1.7× headroom at the lower effective-throughput estimate.

Systems insight: A mid-range GPU delivering 10 TFLOP/s theoretical peak achieves 50–60 percent utilization in this planning scenario, yielding 5 TFLOP/s–6 TFLOP/s effective. For ResNet-50 at 30 FPS, the system has 20.3× headroom. Switching to an object detection model at 100 GFLOP per frame, however, requires 3 TFLOP/s sustained, leaving only 1.7× headroom. Batch size constraints or multi-stream processing quickly push the system toward the compute ceiling. The headroom assumes the GPU stays busy; a single 30 FPS stream runs at batch one, where kernel launches and small matrices usually hold utilization well below this planning figure.

The throughput ceiling converts an abstract compute requirement into a concrete hardware utilization percentage. A real-world deployment adds physical constraints the ceiling alone does not capture.

6.9.5.1 Worked example: Real-time wildlife monitoring

The design task is an embedded ML system that classifies wildlife species from camera trap images in an isolated national park. The deployment must execute inference entirely on-device with zero network connectivity, operate on solar-recharged battery power for six months, and achieve 90 percent accuracy across 50 target animal species. The walk-through applies the three-phase framework in five steps. Data characterization is the symmetry phase; deployment constraint profiling, candidate architecture evaluation, and hardware budget checks carry out the constraint-filtering phase; and operational risk assessment sets up the validation phase that only target-device measurement can complete.

The first step characterizes the data. The input consists of camera trap image frames (1920×1080 raw sensor resolution, downsampled to 224×224 for inference). Three properties favor a CNN candidate: spatial locality (fur textures or ear shapes occupy bounded neighborhoods), label invariance to translation (moving an animal within the frame should preserve its species label), and hierarchical composition (edges combine into textures, anatomical parts, and organisms). Convolution provides equivariant intermediate features that follow the animal's location; pooling can then support an invariant classification. Local weight sharing supplies a compact feature detector for these images.

The second step, constraint profiling, translates the deployment environment into hard hardware boundaries. Table 6.12 catalogs these physical constraints alongside the architectural requirements each imposes:

Table 6.12: Wildlife Monitoring Constraints: Five deployment constraints and their implications for the camera-trap scenario. No connectivity rules out cloud-only inference, while the power, latency, memory, and accuracy requirements must be validated on the target device.

Constraint	Requirement	Implication
Connectivity	None (offline)	All inference must run on-device
Power	~2 W average (solar + battery)	Rules out GPUs; needs a low-power SoC or edge NPU
Latency	<500 ms per detection	Allows batch size 1, no real-time streaming
Memory	512 MB RAM, 2 GB storage	Model, activations, and runtime buffers must fit locally
Accuracy	90%+ on 50 species	Requires sufficient model capacity

With physical constraints established, the third step evaluates candidate architectures against reference model baselines. The planning scenario reserves 100 MB of the available RAM for model weights, leaving the rest for activations, system software, and runtime buffers:

- **ResNet-50** (25.6M parameters, 8.2 GFLOP): A costly starting point for the 2 W envelope. Its weights fit once quantized (25.6 MB in INT8 against 102.4 MB in FP32), but its arithmetic does not: at the platform's 2 GOPS of sustained INT8 throughput assumed below, one frame takes about 4.1 seconds against a 500 ms target.
- **MobileNetV1** (4.2M parameters, 1138 MFLOP): A viable footprint and operation count. Its low arithmetic intensity, which can leave a data center GPU's matrix units idle, costs far less on a batch-one edge CPU whose much smaller compute peak is easier to keep busy. Storing weights in INT8 reduces model footprint to 4.2 MB (16.8 MB in FP32), and factorized depthwise separable convolutions drastically cut arithmetic operations.
- **KWS DS-CNN** (200K parameters, 20 MFLOP): An unmatched baseline. This model targets audio keyword spotting, so its reported size and task results do not establish accuracy on visual species classification. An adapted compact vision model would require training and evaluation on the camera-trap task.

While MobileNetV1 demonstrates the viability of depthwise separable convolutions, MobileNetV2 offers inverted residual blocks with linear bottlenecks as another compact candidate. Sizing the model with width multiplier 0.75× yields ~2.6M parameters (10.4 MB FP32, 2.6 MB with INT8 weights) and costs ~418 MFLOP at 224×224 input resolution. Its weight footprint fits the allocation; task evaluation must establish whether its capacity meets the accuracy target for the 50 species.

The fourth step, hardware sizing, checks the candidate against physical platform budgets. INT8 weights occupy 2.6 MB, the network's largest activation tensor, a $112\times112\times72$ expansion inside

its second bottleneck block, occupies 0.9 MB in INT8, and the scenario reserves 50 MB for OS and runtime buffers. Their 53.5 MB subtotal is only a single-tensor screen. Peak memory must instead include the weights, the maximum sum of simultaneously live tensor bytes, kernel workspace, and OS and runtime buffers. Inputs and outputs can coexist, and a residual path retains its input while the branch executes. The scenario has not yet established that live-tensor peak or the required workspace, so the small subtotal supports candidacy without confirming a fit in 512 MB of RAM.

For the candidate system-on-chip (SoC), an ARM Cortex-A53 at 1.2 GHz with NEON SIMD, this planning calculation assumes sustained throughput of ~2 GOPS INT8: $\frac{418\times10^{6}\ \text{INT8 ops}}{0.002\times10^{12}\ \text{INT8 ops/s}} = 209ms$ latency $\ll 500ms$ target ✓

Active power represents the final physical gate. At an estimated active inference draw of ~200 mW over 209 ms, each classification consumes 41.8 mJ. For a trigger-based rate of 100 inferences/day, model execution consumes 4.2 J/day. Full-system energy must also include image capture, sleep current, storage, and peripherals. Battery capacity, conversion losses, trigger frequency, and seasonal solar yield determine whether that daily budget supports six months of operation.

The fifth step assesses operational risks in the field. Table 6.13 pairs the primary accuracy, thermal, and coverage failure modes with corresponding systems engineering mitigations:

Table 6.13: Wildlife Camera Deployment Risks: Top accuracy, thermal, and species-coverage risks for the MobileNetV2 deployment, paired with the engineering mitigation chosen for each.

Risk	Mitigation
90% accuracy not achieved	Train on augmented dataset; consider EfficientNet-Lite if MobileNet insufficient
Thermal throttling in enclosure	Add passive heatsink; reduce inference frequency in high-temperature conditions
New species added postdeployment	Expand or retrain the output head; install the update during a physical service visit

The sizing exercise selects MobileNetV2 (0.75× width multiplier) quantized to INT8 as a candidate for an ARM Cortex-A53 SoC with 512 MB RAM. Under the assumed sustained throughput, the arithmetic estimate is approximately 209 ms per frame. Qualification now requires target-device measurements of end-to-end latency, peak memory, thermal behavior, and full-system energy, together with classification accuracy on representative camera distributions and seasonal battery tests. In production, this qualification phase regularly exposes counterintuitive trade-offs: models with fewer FLOPs can stall on bandwidth-limited hardware, expressive architectures can degrade on datasets lacking matched structure, and lab prototypes can fail against deployment memory hierarchies.

6.10 Fallacies and Pitfalls

Architecture choice is a systems decision, not a leaderboard selection. The common mistakes in architecture selection arise when teams treat architecture families as interchangeable accuracy tools while ignoring inductive bias, memory traffic, hardware mapping, and deployment state.

Fallacy: *More complex architectures always perform better than simpler ones.*

Engineers often assume that transformers outperform simpler architectures on all tasks. In production, the better choice is the simplest model that matches the data's structure and the machine's budget. Vision transformers matched CNN accuracy on ImageNet only after pretraining on far larger datasets (Dosovitskiy et al. 2021), or later with heavy augmentation and distillation; without that data or that training machinery, the CNN's locality bias wins at lower cost. The bitter lesson favors weak priors only once data and compute are large enough to pay for them, and a deployment that cannot supply that data and compute gets the cost of the weak prior without its benefit. Accuracy, training cost, and inference latency still require measurement for the specific implementation and task.

Pitfall: *Selecting architectures based solely on accuracy metrics without analyzing computational requirements.*

Practitioners choose architectures from papers reporting top-line accuracy, ignoring computational implications. RNNs expose a sequential critical path that limits parallelism, while transformers compute dense attention scores whose storage grows quadratically in naive implementations. Sequence

length 2,048 therefore requires 16× more materialized score memory than length 512. Systems that ignore these characteristics can miss latency targets, exceed memory budgets, and deliver much lower hardware utilization than expected.

Fallacy: *An architecture has one dominant bottleneck across training and inference.*

The same computation graph can occupy different systems regimes. Transformer training and prefill process many positions together and may bind on dense attention compute; small-batch autoregressive decoding repeatedly streams weights and KV-cache state and may bind on memory bandwidth. Batching, sequence length, precision, and cache layout can shift that boundary. A benchmark from one regime therefore cannot establish another's bottleneck; measure each against the iron law.

Pitfall: *Combining architectural patterns without analyzing interaction effects at the system level.*

Engineers add attention to CNNs or convolutions to transformers expecting additive benefits. Each pattern creates distinct memory access characteristics: CNNs exploit spatial locality through sliding windows, while dense attention introduces all-to-all score computation. Combining them can increase intermediate traffic and disrupt locality, so the hybrid's throughput cannot be inferred by adding the components' benefits. Adding recurrent connections to transformers likewise reintroduces sequential dependencies. Successful hybrids require profiling memory access and cache behavior before combining patterns.

Fallacy: *Architecture wins on training hardware transfer directly to deployment hardware.*

Teams design for high-end GPU clusters, then discover deployment failures on target hardware. An architecture exploiting 8× A100 GPUs (640 GB total memory) cannot deploy unchanged to a representative edge node such as the NVIDIA Jetson Orin NX (16 GB system memory). As section 6.9.4 emphasizes, architecture selection must analyze the full system stack because storage, latency, and power budgets vary by deployment target.

Pitfall: *Ignoring KV cache growth when estimating transformer serving costs.*

Teams budget transformer deployment based on model weight memory alone, overlooking the key-value (KV) cache used during autoregressive generation (Pope et al. 2023; Kwon et al. 2023). The cache grows linearly with concurrent sequences, layers, heads, and sequence length, so at long contexts or high concurrency it can become a binding serving constraint. The KV-cache footnote in section 6.7.4.2 works through the chapter's 32-layer, 32-head example. With 128-dimensional heads, 2,048-token sequences, and FP16 storage, each concurrent request holds $\approx$ 1 GB of KV cache. At 2–4 users, the cache alone consumes 2 GB–4 GB before allocator, activation, and workspace overhead. Those values consume part of the post-weight headroom of an 80 GB device and grow linearly with context and concurrency. Capacity planning must therefore budget weights, cache, and runtime buffers together, then reduce concurrency, context, or cache footprint, or add memory capacity, when the combined total approaches device memory.

6.11 Summary

Architecture is infrastructure. Choosing among MLPs, CNNs, RNNs, transformers, and recommendation models (DLRM) shapes the physical terms in the serialized form of the iron law of ML systems ($T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$). Examining each architecture through the four-part lens (pattern processing needs, algorithmic structure, computational mapping, and system implications) connects mathematical inductive bias to work, state, and dependencies. Execution conditions and the target machine then determine which term binds hardware performance.

The durable tool is arithmetic intensity read against the machine's balance. Reuse raises $I = O/D_{\text{vol}}$ and pushes a workload toward the compute term, as ResNet-50 does when batched; streamed weights lower it and expose the data term, as GPT-2 decoding does at small batch. Two families escape that single axis. A recurrent dependency chain sets a floor of S ordered updates that neither term removes, and an oversized embedding table imposes a capacity decision that fixes which bandwidth the data term pays before any arithmetic begins. Low FLOP counts, as MobileNetV2 and KWS show, help only when the resulting operation shapes still keep the hardware's functional units busy.

Key Takeaways: Architecture is infrastructure

- **Inductive bias is the unifying concept**: Every architecture encodes structural assumptions, such as locality for CNNs, sequence for RNNs, and global context for transformers. These biases trade generality for sample efficiency and determine which problems an architecture can solve efficiently.
- **Intensity against balance names the bottleneck**: A workload's arithmetic intensity, set against the machine's balance, predicts whether compute or bandwidth binds. One transformer sits on both sides: prefill reuses every weight across the prompt, while batch-one decode re-reads every weight and a growing KV cache for each token.
- **Quadratic costs are permanent constraints**: Dense transformer attention computes $\mathcal{O}(S^2)$ score interactions. Naive implementations also use quadratic score storage; tiled exact attention removes that storage cost but not the dense computation.
- **Lighthouse models isolate distinct bottlenecks**: ResNet-50 (compute), GPT-2 (bandwidth), DLRM (capacity), MobileNetV2 (compute-bound in its efficient regime, under an edge latency budget), KWS (power-bound). Each isolates one regime, and together they diagnose which physical constraint dominates a given system.
- **Depth benefits from architectural support**: Skip connections and normalization improve optimization in deep networks rather than acting as universal prerequisites beyond a fixed depth. These building blocks, illustrated here through CNNs, transfer to many deep architectures, including transformers.
- **FLOPs do not equal speed**: MobileNetV2 uses 13.7× fewer FLOPs than ResNet-50 but can gain far less than that on data center GPUs when its operation shapes and arithmetic intensity use the available compute units poorly. Architecture-hardware alignment, not operation count alone, determines throughput.
- **Architecture selection is deployment selection**: Choosing a transformer over a CNN determines memory requirements, latency floors, hardware utilization, and infrastructure costs. The architecture is the system constraint.

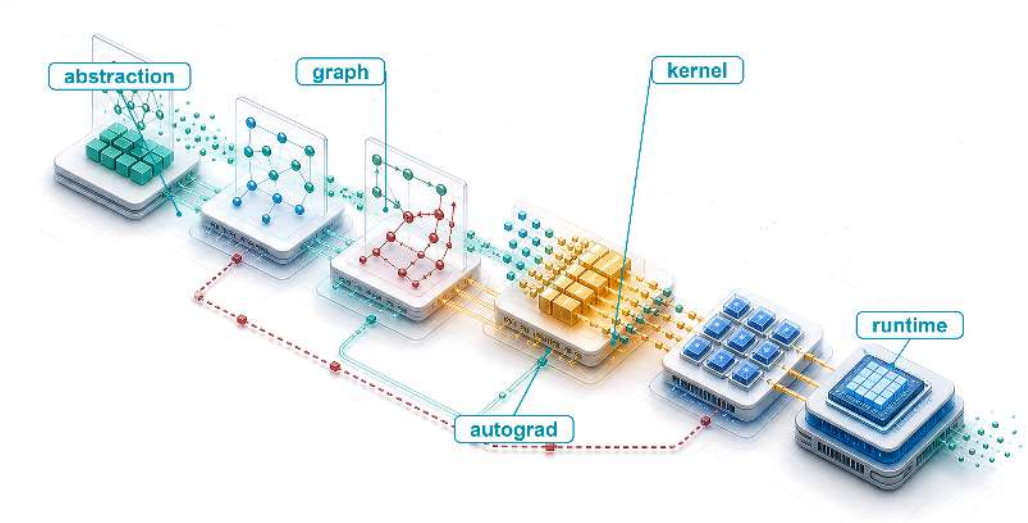
Architecture selection signs the silicon contract (principle 4) with hardware. A CNN commits to spatial locality and weight reuse; dense attention commits to $\mathcal{O}(S^2)$ score computation; and an RNN commits to serial time-step dependencies. These topological choices constrain operations O, opportunities for reducing transferred data D_{vol}, and dependency order. Realized memory footprint, bandwidth demand, and parallel efficiency also depend on the implementation. Pipelining consecutive stages across execution units overlaps work without removing the serial time-step path. Systems engineers can therefore anticipate likely bottleneck shifts before implementation by matching model inductive bias and data geometry to silicon physics.

What's Next: From blueprints to construction

Raw computational primitives (chapter 5) became concrete architectures in this chapter. Each architecture fixes *what* work must be performed, yet nothing in an architecture says when its operations run, how their gradients are computed, or which device executes them. A machine learning framework answers those three questions, acting as a compiler that lowers the architectural graph to the silicon. Chapter 7 examines how PyTorch, TensorFlow, and JAX solve these execution, differentiation, and hardware-abstraction problems, turning architectural graphs into computational graphs, autograd records, and optimized device kernels.

7

ML Frameworks

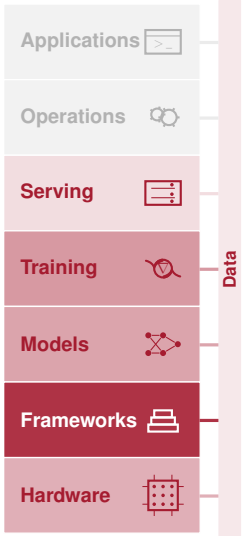


7.1 Three Framework Problems
7.2 Evolution of Framework Abstractions
7.3 Execution: From Graphs to Kernels
7.4 Differentiation: Automatic Gradient Computation
7.5 Abstraction: Tensors, Devices, and Operations
7.6 Modules: From Operations to Models
7.7 Anatomy of a Training Step
7.8 Major Frameworks: TensorFlow, PyTorch, and JAX
7.9 Choosing a Framework
7.10 Fallacies and Pitfalls
7.11 Summary

Purpose

Why does the framework silently constrain every decision that comes after it?

Model equations do not execute themselves. A framework translates an architecture into work that hardware can run, and that translation is not neutral. It decides when operations execute, what intermediate state training retains, how memory is allocated, and which deployment targets can use the resulting artifact. These choices shape more than speed. They also determine what can be debugged directly, what the compiler can optimize, and whether the same model survives the path from a research environment to a production runtime. The same model expressed through two execution paths can differ in speed or memory use by several times, a gap that no amount of model tuning recovers when the framework hides the structure a compiler needs. The abstractions that make experimentation convenient therefore create long-lived commitments. Once a project accumulates checkpoints, modules, data loaders, serving formats, and team expertise around a framework, changing it becomes a system migration rather than a library substitution, measured in engineer-months rather than afternoons. A framework well suited to rapid exploration can expose too little structure for efficient deployment, while a constrained execution path can sacrifice the flexibility needed during discovery. Because these choices meet in one software layer, a local convenience can become a downstream performance or portability constraint. The engineering question is not which framework is universally best, but which trade-offs the workload, team, and target machine can afford to make fixed. Understanding frameworks is therefore about reasoning through those system choices rather than learning transient API syntax, which changes with each release while the underlying trade-offs persist. In D·A·M terms, the framework mediates algorithm-machine co-design by turning algorithmic intent and data representation into executable work under machine constraints.



Learning Objectives

- Explain how frameworks mediate algorithm-machine co-design through execution, differentiation, and hardware abstraction
- Compare eager, graph, and just-in-time compiled execution using dispatch overhead, kernel fusion, compilation cost, and deployment constraints
- Analyze how reverse-mode differentiation computes every gradient in one backward pass and which forward state it must save or recompute
- Estimate host-to-device transfer time and explain how tensor layout, data type, and device placement determine which kernel runs and whether transfers overlap computation
- Implement module abstraction patterns for parameter discovery, mode behavior, serialization, and hooks
- Calculate training-step FLOPs, memory traffic, and dispatch overhead to identify whether a step is bound by dispatch, memory traffic, or arithmetic
- Select a development framework and deployment path based on model, hardware, team, and deployment requirements

7.1 Three Framework Problems

A few lines of Python can express one decoding step of a language model: embed the newest token, pass it through every transformer block, and project the result onto the vocabulary. The accelerator never sees those lines. It sees a stream of kernels,[1] the compiled routines it executes one launch at a time, together with memory allocations and copies, and the machine learning framework is the software that turns the first into the second. Chapter 6 fixed *what* work each architecture performs. It left open *when* each operation runs, *how* its gradients are computed, and *which* kernel on which device executes it.

GPT-2 XL, the book's bandwidth-bound lighthouse, shows what those open questions cost. One batch-one decoding step reads 3 GB of FP16 weights, which takes about 0.9 ms at the 3.35 TB/s memory bandwidth of an H100, the book's reference accelerator. Executed one operation at a time, the same step issues about 12 kernels in each of its 48 transformer blocks, 576 launches in all. At 5 µs of driver overhead per launch, launching alone takes 2.9 ms, about 3× the weight read. An eager Python framework adds 10 µs of dispatch to every operation, which raises the host's time to issue the step to 8.6 ms, about 10× the time the accelerator needs to read its weights. The accelerator spends most of the step waiting for its next instruction. Whether the step costs one millisecond or ten depends on what the framework does with those few lines of Python: run each operation as it is called, or capture the sequence, fuse it into fewer kernels, and launch it as a unit.

Every ML framework, regardless of API or design philosophy, must solve three core problems. The first is the *execution problem*: deciding when and how computation runs. A framework can execute operations immediately as written (eager execution[2]) or first construct a computational graph[3] (a record of operations and their data dependencies, developed in section 7.3.1) to optimize before launching device kernels (graph execution). The decoding step above is an execution problem, and this choice sets the trade-off between interactive debugging and whole-program compiler optimization.

With an execution strategy in place, the framework must solve the *differentiation problem*: computing gradients automatically. Because gradient-based training requires derivatives of a scalar loss with respect to millions or billions of parameters, manual calculus is impossible at scale. Frameworks implement automatic differentiation by registering derivative rules across supported operator compositions, chaining vector-Jacobian products while managing numerical precision and the memory overhead of intermediate activations retained for backward passes.

The third problem is *hardware abstraction*: targeting diverse hardware from a single interface. The same model definition should be expressible across CPUs, GPUs, Tensor Processing Units (TPUs), and mobile accelerators, even though each target presents distinct memory hierarchies, instruction sets, and interconnect bandwidths. For every operation the framework must choose which kernel runs on which device, with which memory layout and precision. Within the GPU

[1] **Kernel (GPU)**: In GPU programming, a kernel is the function dispatched to execute in parallel across thousands of threads. Each kernel launch costs about 5 µs of CPU-side overhead for parameter assembly and GPU signaling, and an eager framework's Python dispatch adds about 10 µs more before the launch, the split this chapter's overhead model uses. Small, unfused operations therefore spend more time on launch overhead (L_{lat}) than on useful arithmetic. Reducing kernel count through fusion is a direct attack on the overhead term of the iron law.

[2] **Eager execution**: This mode executes each operation immediately, which enables direct debugging with standard tools but sacrifices the global view needed for graph-level optimizations. Without capturing a larger region of computation, an eager runtime cannot fuse across that region or preplan its memory, leaving workload-dependent optimization opportunities for compilers such as `torch.compile`.

[3] **Computational graph**: Optimizing before executing is the design choice that separates graph execution from eager execution. Capturing the program as a data structure lets a framework fuse compatible operations into fewer GPU kernels before execution, reducing launch overhead and intermediate memory traffic. The engineering cost of this visibility is that the executed program differs from the source code, making debugging harder, a trade-off every graph-based framework must justify against the performance gain.

↩ Reverse-mode automatic differentiation and computational graph traversal for gradient computation are established in section 5.4.5.

domain, framework ecosystems increasingly expose custom-kernel languages such as Triton (Tillet et al. 2019), allowing developers to write high-performance, block-level kernels without hand-crafting low-level CUDA (NVIDIA's GPU programming platform (Nickolls et al. 2008)) or device assembly.

These three problems are tightly coupled: the execution model dictates when differentiation data structures are recorded, while the hardware abstraction layer determines which graph transformations can be lowered to the target. In the terms of the iron law of ML systems (principle 3), the model architecture fixes the operation count O, and the framework shapes the remaining terms. Execution strategy bears most directly on the fixed overhead L_{lat}, differentiation on the bytes moved D_{vol} (and on O, since the backward pass adds work), and abstraction on the hardware efficiency η_{hw}. These are emphases rather than exclusive assignments, since fusion, for example, cuts both launches and traffic, and the chapter uses this one mapping throughout. The framework plays the role for a model that a compiler plays for a C program, and the correspondence is literal: modern frameworks include optimizing compilers, such as Accelerated Linear Algebra (XLA) (Google 2025) and TorchInductor (Ansel et al. 2024), that lower a captured graph to machine code for the target accelerator. The formal definition below makes that role precise and separates frameworks from numerical libraries such as NumPy.

Definition 7.1: Machine learning frameworks

Machine learning frameworks are software systems that translate high-level mathematical model definitions into hardware-optimized execution plans by managing the computational graph, automatic differentiation, kernel dispatch, and memory allocation across the hardware hierarchy.

1. **Significance**: Frameworks shape every iron law term that the model itself does not fix: the bytes moved (D_{vol}), the hardware efficiency achieved (η_{hw}), and the fixed overhead paid (L_{lat}). Compiler-backed kernel fusion, for example, can eliminate writes and subsequent reads of intermediate values between compatible operations. Fusing a matrix multiplication, bias add, and rectified linear unit (ReLU) changes how the same mathematics reaches hardware without changing the model.
2. **Distinction**: Unlike a numerical library such as NumPy, which normally evaluates each operation as it is called, an ML framework can capture or defer execution to analyze a larger computational graph and apply cross-operation optimizations: kernel fusion, memory layout transformations, and parallel scheduling. Those optimizations require visibility beyond separately dispatched operations.
3. **Common pitfall**: A frequent misconception is that frameworks are interchangeable API wrappers. Framework choice determines which compiler paths are available. PyTorch can recover graph-level optimization from eager code through `torch.compile()`, while TensorFlow and JAX commonly rely on XLA-backed compilation paths that lower operations to target hardware. Moving from eager execution to a compiler path can change throughput materially, but the gain depends on model structure, shapes, operator support, and hardware.

An ML framework translates logical intent into physical execution under the constraints of the iron law, deciding how to partition computation across memory hierarchies, when to trade numerical precision for throughput, and how to schedule operations so that whichever term dominates (data movement D_{vol}, arithmetic computation, or launch latency L_{lat}) is minimized. The framework is where the governing physics developed throughout this book becomes executable code.

This chapter follows a model's graph downward, one level at a time (table 7.1): the decoding step above carries the execution level, and a two-layer MNIST training step, traced at the end, gathers every level. The execution section takes the graph to kernels and decides when each operation runs. The differentiation section builds the backward graph that training adds and accounts for the values it must keep. The abstraction section reaches the bottom of the descent, where tensors occupy device memory and kernels operate on them. Modules then assemble those pieces back into a model, and

the anatomy of one training step traces a single step through every level. The chapter closes with the major frameworks and how to choose among them.

Table 7.1: Chapter Map: Each section of the chapter lowers the model's computational graph one level closer to the hardware and names the iron law term that level most affects.

Section	Level of the graph	Question it answers	Iron law term most affected
Evolution of Framework Abstractions	Libraries, autodiff, compilers	How the levels of the stack came to exist	(context)
Execution: From Graphs to Kernels	Graph to kernels	When each operation runs, and how many launches it costs	Fixed overhead L_{lat}
Differentiation: Automatic Gradient Computation	Forward graph to backward graph	How gradients are computed, and which values must be kept	Bytes moved D_{vol}, and O
Abstraction: Tensors, Devices, and Operations	Tensors in device memory	Where each value lives and which kernel runs on it	Hardware efficiency η_{hw}
Modules: From Operations to Models	Operations to a model	How parameters and training state are organized	(organization)
Anatomy of a Training Step	Execution, differentiation, and abstraction in one step	Where one step's time and bytes go	All terms
Major Frameworks and **Choosing a Framework**	Complete systems	Which framework fits a given constraint	All terms

Automating this machinery (recording operation graphs, allocating gradient buffers, traversing dependencies in reverse, and dispatching hardware kernels) did not happen all at once. Execution, differentiation, and abstraction emerged sequentially as responses to scaling bottlenecks in earlier numerical tools, a four-decade progression from hand-tuned assembly routines to compiled graph runtimes.

7.2 Evolution of Framework Abstractions

In 1979, writing a matrix multiplication in Fortran that used the hardware efficiently required deep knowledge of cache lines, register scheduling, and vector units. By the late 2010s, a single line of Python (`torch.matmul(A, W)`) could dispatch to a tuned implementation without the programmer knowing the details of the hardware. Four decades of abstraction closed that gap, each layer removing a bottleneck that made the previous generation impractical at scale. Each layer automated a job the layer below left to the programmer. History built the layers from the bottom up, starting with tuned kernels and ending with graph compilers; the rest of this chapter walks them from the top down, starting from the graph a model defines.

The first layer established reusable hardware primitives. The Basic Linear Algebra Subprograms (BLAS)[4] standardized low-level linear-algebra operations (Lawson et al. 1979), and LAPACK (Anderson et al. 1999) built factorizations and eigensolvers on top of them. Naive nested loops saturate memory bandwidth long before the arithmetic units reach peak throughput; BLAS let numerical software delegate these operations to vendor libraries tuned for each processor's cache sizes, vector widths, and register files, and its Level 3 extension added the matrix-matrix routines, such as matrix multiplication (`C = A @ B`),[5] that dominate ML workloads (Dongarra et al. 1990). The second layer, NumPy[6] (Harris et al. 2020), wrapped those libraries in Python: the interpreter manages control flow and array metadata (shape, strides, and data type), while compiled code runs the inner loops over contiguous buffers. That division of labor, high-level intent in an interpreted language and inner loops in native code, became the template for every framework that followed.

The third layer automated gradient computation. NumPy users still derived and coded backpropagation by hand, an error-prone process that does not scale to deep networks. Deep learning frameworks such as Theano,[7] TensorFlow (Martín Abadi et al. 2016), and PyTorch (Paszke et al. 2019) made automatic differentiation a runtime primitive: by recording the forward computation as a graph, they construct the backward computation automatically. That automation let the bitter lesson operate at the pace of research. Once a new architecture no longer needed a hand-derived backward pass, general methods that scale with computation could be tried as quickly as their forward functions could be written.

[4] **BLAS (basic linear algebra subprograms)**: The 1979 API specification at the bottom layer of the stack; it fixed a set of Fortran-callable vector operations and separated the public routine names from the machine-specific implementation beneath them. Every framework above it inherits the same division: call a standard linear-algebra routine from any language and let a tuned vendor library target the processor. On NVIDIA GPUs, the tuned path for general matrix multiply (GEMM), a Level 3 routine added in 1990, is cuBLAS (NVIDIA 2024a).

[5] **GEMM**: The matrix-matrix primitive behind `C = A @ B`. Hardware vendors hand-tune GEMM for their specific chips because dense layers, attention projections, and convolution lowering all rely on matrix multiplication, making this one routine a performance floor for many frameworks above it in the stack. Its high arithmetic intensity makes GEMM the operation most able to approach peak compute throughput, while small or misaligned shapes often fall back to much lower utilization.

[6] **NumPy (numerical Python)**: In 2005, Travis Oliphant unified two competing Python array libraries (Numeric and Numarray) into a single BLAS-backed package. Its pattern of writing logic in Python and executing loops in compiled code became the design template for ML frameworks: PyTorch tensors and TensorFlow arrays extend the same n-dimensional array abstraction to GPUs.

Across that history (figure 7.1), the milestones run from BLAS and LAPACK routines through NumPy vectorization to symbolic graph capture in Theano and TensorFlow, dynamic graphs in PyTorch, and functional transformations in JAX. Each generation preserved the vendor-tuned kernel layer beneath it while raising the level of abstraction at which engineers express and differentiate models.

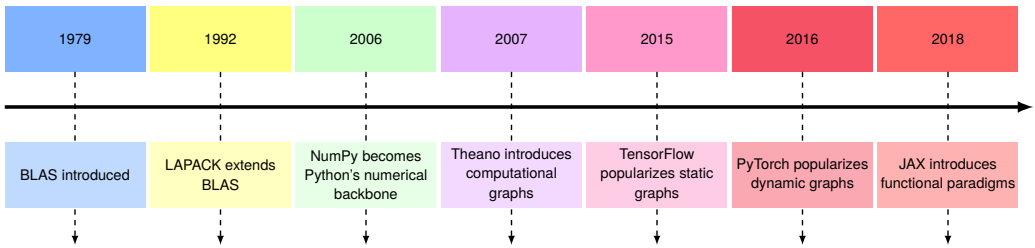


Figure 7.1: Computational Library Evolution: Numerical computing milestones from 1979 to 2018 show each era automating what the layer below exposed, from BLAS and LAPACK hardware routines to NumPy vectorized syntax and graph-based automatic differentiation in modern frameworks, trading low-level transparency for developer velocity.

Each generation hid details that consumed engineering effort in the previous one, and each introduced new trade-offs. BLAS hid assembly-level optimization but fixed the interface. NumPy hid memory management but left differentiation manual. Modern frameworks hide gradient computation but force a choice among execution models. Every layer also hides assumptions about the data it passes along: each operation expects its inputs to have a particular shape, data type, device, and, for physical quantities, unit. When nothing checks those expectations at the boundary, correct components can compose into a wrong system.

War Story 7.1: The interface that forgot its units (1999)

Context: NASA's Mars Climate Orbiter depended on ground software, spacecraft software, navigation teams, and contractor interfaces all agreeing on the physical meaning of exchanged values (Mars Climate Orbiter Mishap Investigation Board 1999).

Mechanism: Ground software supplied by Lockheed Martin computed thruster impulse in English units (pound-force seconds), while JPL navigation software consumed the values assuming SI units (newton-seconds), a 4.45× implicit unit mismatch.

Impact: On September 23, 1999, the spacecraft was lost after passing behind Mars (occultation). Its corrected 57 km periapsis, rather than the planned 226 km, was judged too low for survival.

Response: The board recommended unit-consistency checks, specification audits of transferred data, and stronger end-to-end verification of ground software.

Systems lesson: Two pieces of correct code can compose into a wrong system when the meaning of the values they exchange is implicit. ML frameworks meet the same failure whenever tensor memory layouts (such as channels-first NCHW [Batch × Channels × Height × Width] vs. channels-last NHWC [Batch × Height × Width × Channels]), quantization scale factors, or physical feature units differ across pipeline stages, producing plausible outputs that quietly corrupt downstream inference. Frameworks guard against the tensor cases by checking, at every operation, that its inputs have the expected shape, data type, and device.

A framework can run those checks as each operation executes, when the failing line and its intermediate values are directly visible, or before execution, when a captured graph can be validated and optimized as a whole. When checks and optimizations happen is the execution problem. It is the first framework design choice because runtime dispatch and graph capture determine both the debugging experience and what optimizations automatic differentiation and hardware mapping can later exploit.

7 **Theano**: Developed at the Montreal Institute for Learning Algorithms (MILA) under Yoshua Bengio starting in 2007, Theano was an early and influential Python framework that compiled symbolic mathematical expressions into optimized CPU and GPU code via computational graphs (Bergstra et al. 2010; Team et al. 2016). It demonstrated that a Python-defined computation graph could be compiled for GPU execution without requiring the researcher to write CUDA code. Mila ended active development of Theano in 2017 (Mila 2026).

↩ The bitter lesson, that general methods which exploit growing computation outlast hand-engineered approaches, is established in section 1.4.

7.3 Execution: From Graphs to Kernels

Consider two engineers writing the same neural network. The first debugs interactively, printing tensor shapes after each operation, inspecting intermediate values, and stepping through code with `pdb`. The second waits through a compilation step, then watches a model built from many small operations run faster while losing that direct, line-by-line view of intermediate state. Both are correct; they have made different choices about the **execution problem**, the question of whether operations should execute immediately as written or be recorded for later execution. It sets debugging workflow, deployment options, and peak hardware utilization. The GPT-2 XL decoding step that opened this chapter is the case in point: issued one operation at a time, it keeps the accelerator waiting on the host for most of the step.

7.3.1 Computational graphs

If a runtime sees only one Python operation at a time and forgets it as soon as the kernel returns, it cannot know which intermediate tensors later gradient rules will need or which adjacent operations could share registers on the accelerator. To plan across operations, a framework represents the program the same way chapter 6 represented model architectures, as a computational graph: a directed acyclic graph (DAG) where nodes represent operations and edges represent data dependencies. The rest of this chapter follows that graph down toward the hardware. It is the same structure that backpropagation traverses in reverse, here serving as the framework's internal model of the computation for scheduling and optimization as well.

A graph decouples the mathematical operations from the order in which hardware evaluates them. In the minimal case (figure 7.2), directed edges route input variables x and y into the central transformation operator $f(x, y)$, which emits the output variable z.

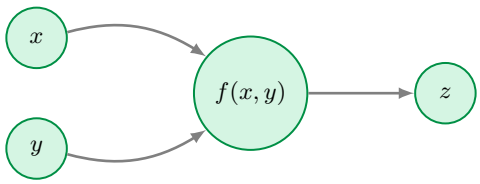


Figure 7.2: Simple Computational Graph: Representing $z = f(x, y)$ as a directed acyclic graph decouples mathematical dependencies from physical execution order, providing the structural blueprint frameworks require to track gradients and schedule operations across devices.

Real machine learning models require much more complex graph structures. Figure 7.3 partitions this system interaction into two coupled domains: the left container isolates a six-node neural network DAG with explicit dataflow edges, while the right container houses the runtime components (memory management and device placement) that interface with it.

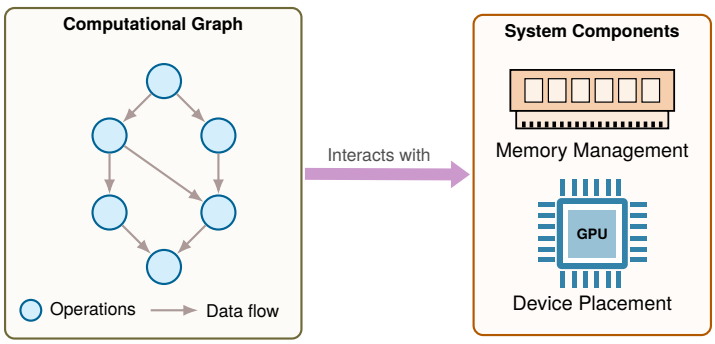


Figure 7.3: Computation Graph with System Interactions: A neural network DAG (left) interfaces with framework runtime components (right). By maintaining an explicit graph representation independent of execution, the framework can analyze tensor lifetimes for memory reuse and assign operations to target accelerators before issuing arithmetic.

This graph representation is more than a visualization; it is the data structure that enables both efficient execution and automatic differentiation. The timing of its construction forces a fundamental systems trade-off across debugging, optimization, deployment, and flexibility. Interactive debugging demands immediate execution so developers can inspect intermediate values and step through logic with standard tools. Global optimization requires observing large operator sequences at once to fuse kernels and plan memory reuse. Standalone deployment favors graphs that run without the host interpreter, while dynamic architectures require control flow that branches on computed tensor values. No single execution model serves all four demands at once.

7.3.2 The memory wall and kernel fusion

When an unoptimized runtime evaluates a sequence of three element-wise operations such as LayerNorm $\rightarrow$ Dropout $\rightarrow$ ReLU, it dispatches three separate GPU kernels: the first reads inputs from off-chip high-bandwidth memory (HBM) and writes intermediate results back to HBM, the second reads those results and writes back again, and the third reads them a final time. Shuttling intermediate tensors across the external memory bus three times dwarfs the arithmetic time and leaves compute units idle.

This bottleneck stems from the memory wall, the widening gap between compute growth and memory-bandwidth growth that equation 2.3 quantifies. An accelerator performs arithmetic far faster than it fetches operands from off-chip memory, so most individual operation types in a neural network (activations, normalizations, and other element-wise operations) are memory bound, limited by data movement, while the large matrix multiplications that dominate total arithmetic can be compute bound, limited by arithmetic throughput. Section 11.6 formalizes the boundary between the two regimes. On-chip storage (registers and SRAM scratchpads) runs at tens of terabytes per second but holds only tens of megabytes, whereas off-chip HBM or GDDR holds tens to a few hundred GB behind external buses with hundreds of cycles of latency.

Kernel fusion (also called operator fusion) fixes memory-bound chains by combining them into a single kernel that avoids intermediate HBM traffic. Instead of three round-trips to HBM, a fused kernel loads inputs from HBM once, evaluates normalization, dropout, and ReLU entirely within on-chip registers and SRAM, and writes only the final output tensor back to HBM, cutting memory traffic by roughly $3\times$. In general, for N elements of s_{elem} bytes each, fusing a chain of n_{ops} elementwise operations into one kernel reduces idealized read-and-write traffic from $2n_{\text{ops}}Ns_{\text{elem}}$ to $2Ns_{\text{elem}}$ bytes, a derivation worked in section C.3.4.2. Fusion also removes all but one of the chain's kernel launches. Attention kernels[8] use the same principle at larger scale: instead of materializing the full attention matrix in HBM, a fused implementation can keep tiles close to the compute units, cut HBM accesses by up to $9\times$, and produce workload-specific speedups from 15 percent to $3\times$ (Dao et al. 2022).

Framework compilers can fuse only operations that are visible to them simultaneously. When an eager runtime dispatches operations individually, each kernel launch executes in isolation, concealing cross-operation fusion and buffer-reuse opportunities. Capturing a broader region of computation into a deferred graph removes that isolation: the compiler can examine an entire sequence of dependencies, establish intermediate tensor lifetimes, and fuse compatible operations before dispatching work to the accelerator.

The computational graph enables global optimization, but it leaves a critical design choice unresolved: *when* the runtime constructs the graph. Consider an elementary expression such as `y = x * 2`. Evaluating the multiplication immediately yields a concrete tensor in `y`. This imperative flow is intuitive and directly debuggable, but the runtime sees only one operation at a time. Deferring execution records the operation symbolically into a dependency graph executed only upon request. That staging separates intent from evaluation, giving the compiler the end-to-end visibility required for kernel fusion and memory planning. Modern frameworks resolve this tension between *flexibility* and *optimization potential* with three execution strategies: eager execution with dynamic graphs, static computation graphs, and hybrid approaches that combine just-in-time (JIT) compilation with eager development.

↩ The memory wall and the growth-rate ratio behind it are established in section 2.2.

[8] **Fused attention kernel**: A fused attention kernel combines the $\mathbf{QK}^T$ product, softmax, and value-weighted output into a tiled implementation that keeps intermediate values in on-chip memory rather than materializing the full attention matrix in HBM. FlashAttention, introduced in section 6.6.4.1, is the canonical named example. The framework lesson is broader than the specific algorithm: fusion can shift an operation's position on the roofline model from bandwidth-limited toward throughput-limited execution by reducing round-trips through external memory.

↪ The on-chip limits that decide which intermediates a fused kernel can keep on a specific accelerator are developed in section 11.7.7.

7.3.3 Eager execution with dynamic graphs

When a tensor shape mismatches or an activation turns to `NaN` mid-layer, a deferred graph fails far from the Python line that created the operation, hiding the intermediate values that caused the fault. Eager execution, also called define-by-run, avoids that disconnect by evaluating each operation as soon as the program calls it and building the computation graph dynamically at runtime.

Example 7.1: Eager vs. graph execution code comparison

PyTorch (eager execution):

```
import torch

x = torch.tensor([1.0, 2.0])
y = x * 2
print(f"Intermediate value: {y}")  # Works immediately
z = y.sum()
```

TensorFlow 1.x (static graph):

```
import tensorflow as tf

x = tf.placeholder(tf.float32)
y = x * 2
# print(y) -> Prints Tensor("mul:0"...), not value!
z = tf.reduce_sum(y)

with tf.Session() as sess:
    result = sess.run(z, feed_dict={x: [1.0, 2.0]})
```

Systems insight: Eager execution exposes intermediate values as ordinary runtime state, which makes debugging direct. Static graphs stage computation before execution, which enables whole-graph optimization but changes the debugging model.

In this define-by-run model, when host code evaluates `y = x * 2`, the runtime dispatches the multiplication kernel to the device command queue and returns a tensor handle for subsequent operations. Device execution can continue asynchronously; dependent operations must wait for the result, and reading its value on the host can require synchronization.

Immediate evaluation gives eager code the flexibility of ordinary programming: developers can print intermediate values, use conditionals based on computed results, and debug with standard tools. The framework records operations as they happen, constructing a dynamic graph that reflects the actual execution path taken.

For gradient computation, the framework records a history of operations in what is called an autograd tape,[9] a transient data structure built during execution. Each tensor operation creates a node that records: the operation performed, references to input tensors, and how to compute gradients. These nodes form a DAG that records the actual path taken during forward-pass execution rather than a graph fixed in advance. Listing 7.1 shows how PyTorch records operations as they execute in its default eager mode.

[9] **Autograd tape**: The tape's own nodes are small; its memory cost comes from the tensors they reference. That footprint grows with model depth and sequence length as saved activations accumulate until backpropagation releases them, which is why deep models retain only selected activations and recompute the rest (activation checkpointing) to avoid out-of-memory failures.

Listing 7.1: Autograd Tape Construction: Each operation executes immediately while recording a backward node to the autograd tape for later gradient computation.

```
import torch

x = torch.tensor([1.0], requires_grad=True)
y = x * 2  # Executes immediately; records MulBackward node
z = y + 1  # Executes immediately; records AddBackward node
# The autograd tape exists NOW, built during execution
```

After these two operations, the framework has constructed an autograd tape with two nodes: one for the multiplication and one for the addition. The tape records that `z` depends on `y`, and `y` depends on `x`.

Calling `z.backward()` traverses this tape in reverse topological order, applying the chain rule at each node:

1. Compute $\frac{\partial z}{\partial z} = 1$ (seed gradient)
2. Call `AddBackward0.backward()` $\rightarrow \frac{\partial z}{\partial y_{\text{tape}}} = 1$
3. Call `MulBackward0.backward()` $\rightarrow \frac{\partial z}{\partial x} = 2$
4. Accumulate gradient in `x.grad`

After `backward()` completes, autograd normally releases the saved state needed by its backward rules. Other live references can still retain tensor storage and graph nodes. The next forward pass builds a new graph. Values needed for gradients are saved during the forward pass and remain available until backward consumes them, so their memory cost spans both phases rather than appearing only during backward.

Example 7.2: In-place operations can break gradients

Scenario: An engineer applies an in-place mutation (`x += 1`) within a custom PyTorch activation function to reduce memory allocations.

Diagnosis: In-place operations overwrite tensor memory containing forward-pass activations required by the autograd tape for backward-pass gradient computation, triggering a runtime version-counter error (PyTorch Contributors 2026a). PyTorch enforces this safety invariant by maintaining an integer version counter on each tensor that increments on every in-place write; if a saved tensor's version at backward execution differs from its version recorded during the forward pass, autograd throws an error because the activation required by the derivative rule was modified in place.

Systems lesson: Framework automatic differentiation depends on immutable forward-pass activation records. In-place writes to a saved tensor invalidate the autograd record, so PyTorch stops with an error rather than compute a wrong gradient.

In eager execution frameworks, operations evaluate imperatively as host code encounters them. The define-by-run cycle is therefore a loop of one operation at a time (figure 7.4). Python defines an operation, the runtime enqueues its device work, and control returns to Python for the next call while the device may execute asynchronously. When the runtime decides what to launch, it sees the current operation rather than a whole region of future calls.

The dynamic autograd tape expresses data-dependent behavior directly in the host language. Conditionals and loops can depend on tensor values computed during execution, enabling algorithms such as beam search, dynamic-length recurrent models, or adaptive compute paths that adjust their execution based on intermediate activations. Static graphs can also represent data-dependent control flow and dynamic dimensions through graph operations, but eager execution makes these patterns natural to write and debug in Python. Because operations are issued immediately, developers can print tensors, inspect values, and use standard debuggers (`pdb`, breakpoints) to diagnose errors much as they would in another Python program.

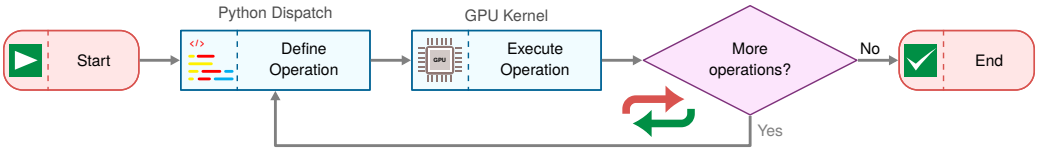


Figure 7.4: Dynamic Graph Execution Flow: In eager execution, each operation is issued as the program encounters it. Accelerator work may execute asynchronously after dispatch. This define-by-run model enables natural debugging and data-dependent control flow while limiting optimization across Python calls.

This flexibility imposes latency costs that map directly to the iron law. Each training step constructs an autograd tape dynamically, adding host-side Python dispatch latency and tape allocation to L_{lat}. Because every operation traverses the framework dispatcher before a device kernel launches, host overhead quickly dominates when individual kernels execute in single-digit microseconds. Eager execution also lacks a whole-graph view, preventing the framework from fusing element-wise chains into single GPU launches (which would cut D_{vol}) or pre-allocating a static memory arena for intermediate activations (which would cut allocation overhead in L_{lat}). The autograd tape must retain every tensor requested by registered backward rules, holding memory until backpropagation consumes it. Together, these costs impose a throughput ceiling on fine-grained workloads where dispatch latency overshadows device arithmetic.

7.3.4 Dispatch overhead in eager execution

Small op

Dispatch 94%

Large op

Compute

13%

Fixed dispatch dominates small ops and fades behind large ones.

Eager execution is slow on small operations because the host-side interpreter is slower than the device-side silicon. The **dispatch overhead ratio**, host software time T_{sw} divided by device hardware time T_{hw}, quantifies this mismatch in an additive model without host-device overlap. When the ratio exceeds 1, host dispatch takes more than half of each step. The dispatch overhead law of section 7.3.7.2 later expresses the same ratio through operation count and per-operation cost. Asynchronous execution can hide part of that host cost.

In the illustrative eager-execution model used here, every operation pays a fixed cost of approximately 15 µs for Python-side dispatch and driver launch combined, the same per-operation cost as the decoding step that opened the chapter. Actual overhead varies by framework, backend, and hardware, but its relative weight still depends on operation size. For a small operation such as a ReLU on a small vector, the kernel might execute in only 1 µs, so dispatch takes 94 percent of the modeled step time, the dispatch overhead ratio sits far above 1, and the GPU spends most of its time waiting for the next command. For a large operation such as a large matrix multiply, the kernel executes for 100 µs, dispatch drops to 13 percent of the modeled step time, and the dispatch overhead ratio falls well below 1. The operation is device-work dominated; distinguishing compute from memory limits still requires comparing its work and traffic.

Dispatch overhead explains why models with many small layers run significantly slower than their raw FLOP count predicts. Section A.5.3 places this symptom in the bottleneck taxonomy, classifying a dispatch-dominated workload as overhead-bound rather than compute-bound and showing which optimizations actually move it. Frameworks can move from *kernel-by-kernel dispatch* to *graph-level execution*, reducing repeated interpreter work and exposing operations that can be fused. A captured graph can still launch multiple kernels; fusion and runtime replay determine how much launch overhead remains. The hybrid JIT and compilation strategies in section 7.3.6 exist to address this overhead.

↩ The MNIST epoch estimate, in which this fixed per-operation cost decides training time on a CPU and an accelerator alike, is established in section 5.4.6.

7.3.5 Static computation graphs

The overhead costs of eager execution motivate the opposite design: capturing the entire computation before executing any of it. Because an eager runtime sees only one operation at a time, it cannot fuse a chain of small kernels or plan buffer reuse across a whole layer before launching work on the accelerator. Static graph execution solves this visibility problem by defining the complete computational graph as a symbolic representation first and executing it in a separate phase. This "define-then-run" execution model means the graph exists before any computation occurs, enabling ahead-of-time analysis, transformation, and global optimization that separate eager calls cannot support unless a compiler captures them.

Static graphs implement a clear separation between graph construction and execution. TensorFlow 1.x, which exemplified this approach, makes the two phases explicit in code (listing 7.2). It deliberately runs the same `x * 2` then `+ 1` computation shown under eager execution in listing 7.1, holding the arithmetic fixed so that the only thing that changes is *when* it executes: symbolic definition creates placeholders and operations without computation, while explicit execution triggers actual arithmetic.

Listing 7.2: Static Graph Two-Phase Execution: Graph construction (symbolic definition) is separated from execution (actual computation), enabling ahead-of-time optimization.

```
# Phase 1: Graph Construction (symbolic, no computation)
import tensorflow.compat.v1 as tf

tf.disable_v2_behavior()

# Define graph symbolically
x = tf.placeholder(tf.float32, shape=[1])  # Just a placeholder
y = x * 2  # Not executed, just recorded
z = y + 1  # Still no execution
# At this point, nothing has been computed

# Phase 2: Graph Execution (actual computation)
with tf.Session() as sess:
    result = sess.run(z, feed_dict={x: [1.0]})
    # Now computation happens: result = [3.0]
```

In the definition phase, the program declares operations and variables and builds the symbolic DAG without allocating any tensor memory. Only in the execution phase do data load, concrete buffers materialize on the accelerator, and the captured graph run (figure 7.5).

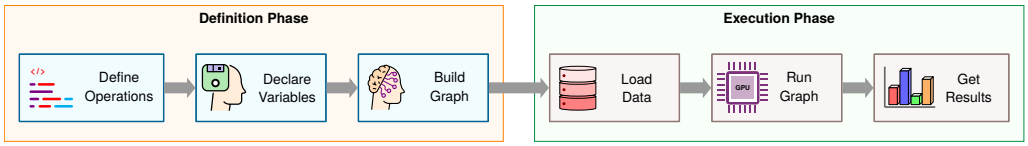


Figure 7.5: Static Graph Execution Lifecycle: Separating symbolic graph definition (left) from concrete execution (right) provides global visibility over the computation, allowing compilers to apply ahead-of-time optimizations (such as kernel fusion, buffer reuse, and layout transforms) before any tensors are materialized.

The key difference from eager execution is that during construction, `x`, `y`, and `z` are not tensors containing values but rather symbolic nodes in a graph. Operators such as `*` and `+` append transformation nodes to the graph definition without issuing arithmetic. Calling `print(y)` makes this distinction visible: it prints symbolic tensor metadata rather than numerical values. Only when the host invokes `sess.run()` does the runtime analyze, optimize, and execute the captured graph on device buffers.

Because the framework has the complete graph before execution, it can perform ahead-of-time optimization unavailable across uncaptured eager calls. The kernel fusion opportunity introduced in section 7.3.2 becomes actionable here: because the framework sees `y = x * 2` and `z = y + 1` together in the graph, it can fuse them into `z = x * 2 + 1`, eliminating the intermediate `y` and halving idealized memory traffic. When tensor shapes and lifetimes are known, the compiler can plan memory before execution and reuse buffers whose lifetimes do not overlap; dynamic dimensions may require bounds or runtime allocation. Tensor layouts can also be transformed globally (for example, NCHW to NHWC) to match hardware preferences, though physical layout conversions may still require copies. Dead-code elimination (DCE)[10] removes operations whose results are unused, and constant folding precomputes operations on constant values when their inputs are known. These optimizations map directly to iron law terms: kernel fusion can reduce D_{vol} by avoiding intermediate memory writes, constant folding reduces repeated work, planned allocation can reduce runtime overhead, and dead code elimination removes work and associated

[10] **Dead code elimination (DCE)**: Removes graph nodes whose results do not affect observable outputs or side effects. In ML graphs, dead code arises from unused branches or post-transform artifacts; side-effect analysis ensures safe removal to prevent execution of dead operations and reduce kernel launch overhead.

data movement when unused computations are present. Compilers such as **XLA**[11] (Google 2025) extend this approach, compiling captured subgraphs into fused machine code specialized for target hardware.

Static graphs can therefore approach high utilization when shapes, operators, and layouts match efficient backend paths. The cost of this performance is reduced flexibility. Standard Python control flow (`if`, `for`) cannot depend on computed tensor values in static graphs. TensorFlow provides graph-level control flow primitives (`tf.cond` and `tf.while_loop`) that support data-dependent conditions, but these require special syntax that diverges from standard Python, making code harder to write and reason about. Debugging is difficult because stack traces point to graph construction code, not execution code. Error messages often reference symbolic node names rather than the actual operations that failed.

7.3.6 Graph capture and JIT compilation

Staying in eager mode leaves small-operation workloads bound by host dispatch and unfused HBM traffic, while rewriting a research model into a static graph gives up interactive Python debugging and standard control flow. **JIT compilation** bridges that gap by letting developers write and debug ordinary eager code while capturing regions of the program at runtime for graph-level optimization. The core trade-off in graph capture is *fidelity vs. generality*. Tracing captures the execution path taken during a sample run, producing high fidelity to that path but missing branches not taken. Source-level compilation (scripting) analyzes supported program structure, preserving supported control flow but requiring a restricted language subset. Both approaches produce an intermediate representation (IR)[12] that enables graph optimizations such as kernel fusion, constant folding, dead code elimination, and buffer reuse.

The eager-vs.-compiled trade-off has a direct iron law consequence. JIT compilation amortizes the L_{lat} (dispatch overhead) across the compiled region. Longer compiled regions mean more overhead amortized per operation, which explains why graph breaks are performance-critical: each break forces a return to eager dispatch, resetting the amortization.

Graph-capture JIT systems follow the same architectural pattern across frameworks. The first execution observes tensor operations, records a graph region guarded by assumptions about shapes, dtypes, layouts, and control flow, lowers that region into an intermediate representation, applies fusion and layout optimizations, and caches executable code for later calls that satisfy the same guards. The compiled artifact is valid only while those assumptions hold, so a failed guard forces the runtime to recompile or fall back to eager execution. When eager fallback is enabled, unsupported Python code forms a graph boundary where execution returns to the eager runtime; a configuration requiring full-graph capture can instead report an error. PyTorch 2.0's `torch.compile` (Ansel et al. 2024) is a concrete instance of this pattern, but the systems idea is broader than the API: compilation pays only when captured regions are long enough and stable enough to amortize capture, lowering, code generation, and cache-management costs.

Lowering into an intermediate representation is far easier when the target vocabulary is small, and the eager frameworks a compiler starts from are not small. Figure 7.6 counts the operators that each framework defines at successive releases. PyTorch's native-function table grew from 1,064 schemas, overloads included, at version 1.0 in 2018 to 2,658 by version 2.9 in 2025 (PyTorch Contributors 2025), and TensorFlow's operator registry grew from 594 at version 1.0 in 2017 to 1,563 by version 2.20 in 2025 (TensorFlow Developers 2025). Every one of those entries is something a backend must implement, or that the framework must first functionalize and decompose.

Compiler-facing representations work from far smaller sets, which grew by tens of operators while the eager surfaces grew by thousands. The Core ATen IR, the operator set that PyTorch's compiler and export paths lower to, holds 192 operators (PyTorch Contributors 2025); the XLA compiler that TensorFlow and JAX both target defines 123 high-level optimizer (HLO) operations (OpenXLA Project 2025); and the Open Neural Network Exchange (ONNX) standard, which carries models between frameworks and runtimes, defines 203 (ONNX Project 2026). JAX starts on the compiler's side of this divide. Its user-facing operations reduce to about 135 `lax` primitives (JAX Developers 2025), a set the size of the compiler cores, so a JAX program reaches XLA with little decomposition left to do. The 13–14× gap between an eager surface and its compiler core (for TensorFlow, on its XLA path) is the work that decomposition performs. A new accelerator backend that implements a

[11] **XLA (accelerated linear algebra)**: XLA fuses subgraphs, specializes layouts, and lowers high-level operations into backend-specific code, so its fusion attacks both launch overhead (L_{lat}) and intermediate memory writes (D_{vol}). Its speedup is therefore not a fixed multiplier. Large GEMM-heavy regions may already be compute bound, while chains of small elementwise operations benefit more because fusion removes repeated trips through external memory, so gains depend on graph structure, backend support, and input-shape stability.

[12] **Intermediate representation (IR)**: The "intermediate" captures this format's architectural role: a language-independent layer that decouples the frontend (Python capture) from the backend (hardware code generation), exactly as LLVM IR decouples C/Rust/Swift frontends from x86/ARM backends. ML frameworks adopted this compiler pattern because it reduces the $\mathcal{O}(M \times N_{\text{backend}})$ cost of supporting M frontends and N_{backend} backends to $\mathcal{O}(M + N_{\text{backend}})$: a single graph capture mechanism (PyTorch's TorchDynamo, or TensorFlow's bridge into XLA) can target multiple hardware backends without rewriting the capture logic.

small core set can run models written against the full eager surface, which is why operator coverage, not peak throughput, often decides whether a model runs on a new device at all.

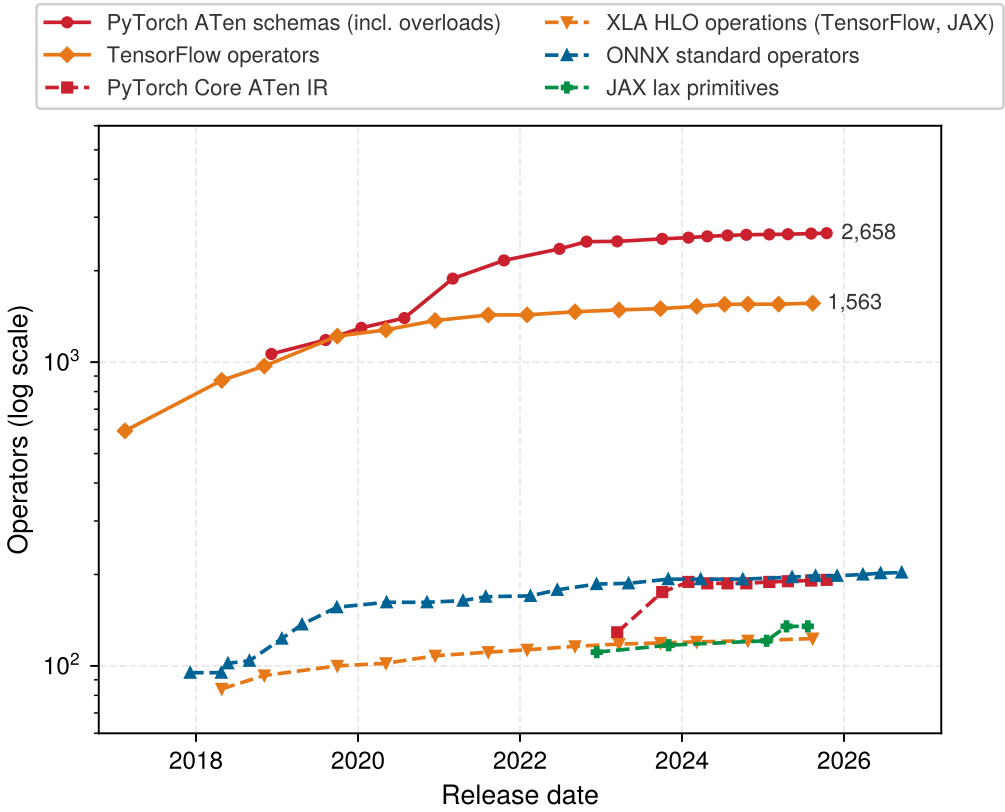


Figure 7.6: Eager Operator Surfaces versus Compiler Cores: Operators defined by the eager frameworks (PyTorch native-function schemas with overloads, and TensorFlow's registered operators) against the compiler-facing sets they lower to (PyTorch's Core ATen IR, XLA's HLO operations, the ONNX standard, and JAX's `lax` primitives) at each release, on a logarithmic axis. The eager surfaces grew past 1,500 and 2,600 entries while every compiler-facing set stayed between about 80 and 200. Counts mined from each project's repository at its release tags (PyTorch Contributors 2025; TensorFlow Developers 2025; OpenXLA Project 2025; ONNX Project 2026; JAX Developers 2025).

The two capture modes of PyTorch's TorchScript, now deprecated in favor of capture and export paths such as `torch.export` (PyTorch Contributors 2026d, 2026c), make the fidelity-generality trade-off concrete. Tracing executed a function with example inputs and recorded the tensor operations observed on that path. Scripting instead compiled a restricted, type-annotated subset of Python source, preserving supported branches in the graph at the cost of rejecting unsupported Python objects, library calls, and metaprogramming. Both produced an IR with static types in static single-assignment (SSA) form, where each variable is assigned exactly once, so fusion, constant folding, and dead code elimination could run without the Python interpreter.

The main limitation of tracing is that a trace records operations from an observed path rather than preserving arbitrary Python control flow. A data-dependent branch can therefore freeze into whichever side the example input took (listing 7.3).

Listing 7.3: Tracing a Data-Dependent Branch: Tracing records the path taken by the example input. PyTorch commonly emits a `TracerWarning` when a tensor value is converted to a Python condition because the resulting trace may not generalize.

```python
import torch


def conditional_forward(x):
    if x.sum() > 0:  # Data-dependent condition
        return x * 2
    else:
        return x * 3


traced = torch.jit.trace(conditional_forward, torch.tensor([1.0]))
# Tracing captures ONLY the x.sum() > 0 branch
# If input later has sum <= 0, traced version
# still executes x * 2 branch
```

In this example, converting the tensor condition to a Python Boolean commonly produces a `TracerWarning`; the resulting trace can still follow the captured branch for later inputs that require the other branch. Treating trace warnings as correctness failures is therefore essential when data-dependent Python control flow is present. The same hazard survives in modern graph-capture systems, which is why they guard each captured region with explicit assumptions about shapes and control flow.

Compilation therefore attacks the dispatch overhead of section 7.3.4 directly, and fusion adds a second saving in memory traffic. Notebook 7.1 quantifies both for a two-operation activation block.



Napkin Math 7.1: The physics of software overhead

Iron law connection: For small operations, the latency term (L_{lat}) in the iron law is dominated by software overhead: dispatching instructions from Python to the GPU. Each operation pays a Python dispatch cost of ~10 µs plus a kernel launch cost of ~5 µs, which together set the per-operation overhead applied in the two scenarios below. Python dispatch covers the framework call; driver launch covers submission to the device. A compiled region invoked from Python still pays an entry cost, whereas replay of a captured launch sequence removes repeated per-operation Python work and has its own submission cost.

Scenario one: Eager mode (the tiny-op trap) Consider a simple activation block, `y = relu(x + bias)`, in which the input and output each contain N elements of s_{elem} bytes each. Assume a scalar or broadcast bias remains in fast memory, so the traffic model counts the full input, intermediate, and output tensors.

- **Operations**: Two (Add, ReLU).
- Execution:
  1. Launch `Add` Kernel: 15 µs overhead.
  2. Read/Write Memory: $2Ns_{\text{elem}}$ bytes.
  3. Launch `ReLU` Kernel: 15 µs overhead.
  4. Read/Write Memory: $2Ns_{\text{elem}}$ bytes.
- **Total overhead**: 30 µs.
- **Total memory traffic**: $4Ns_{\text{elem}}$ bytes.

Scenario two: Compiled mode (fusion) The compiler fuses this into one kernel: `FusedAddRelu`. This scenario retains one ordinary Python entry and one driver launch, applying the same combined cost as eager mode to isolate the effect of reducing two calls to one. It does not model captured-graph replay.

- Execution:

1. Launch `Fused` Kernel: 15 µs overhead.
2. Read/Write Memory: $2Ns_{\text{elem}}$ bytes (the intermediate result stays on chip).

- **Total overhead**: 15 µs (2× less launch overhead).
- **Total memory traffic**: $2Ns_{\text{elem}}$ bytes (2× bandwidth efficiency).

Systems insight: Fusion saves on both terms at once: half the launches and, by the fused-chain traffic rule of section 7.3.2 with $n_{\text{ops}} = 2$, half the idealized traffic.

Scaling this arithmetic to a sequence of 5 small operations exposes the idle gaps on the accelerator timeline (figure 7.7). In eager execution, each 1 µs kernel waits on 10 µs of Python dispatch, leaving the accelerator idle for about 91 percent of the sequence; the figure isolates the Python share, and the per-kernel launch cost would widen each gap further. Compiling the sequence resolves both constraints simultaneously: the framework submits the work in a single launch of 5 µs, eliminating all intermediate host dispatch gaps, while operator fusion keeps intermediate activations in registers or cache to complete the entire sequence in 4 µs instead of the uncompiled 5 µs.

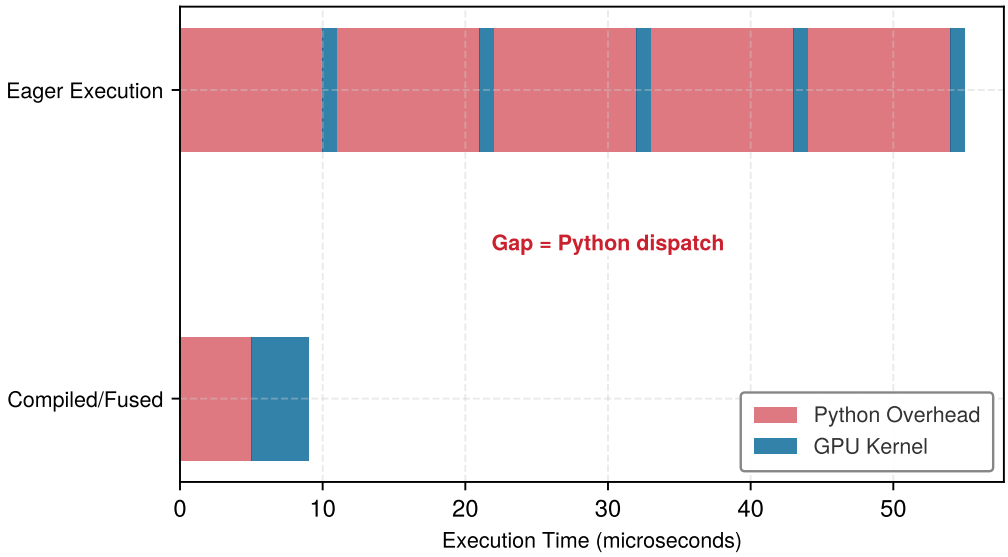


Figure 7.7: Python Dispatch Overhead: Execution timelines illustrate how host dispatch latency (L_{lat}) starves the accelerator during fine-grained operations. Eager execution incurs per-operation host overhead for each kernel, whereas compilation and fusion collapse the sequence into a single launch (drawn in the overhead color as one direct device submission), eliminating intermediate dispatch gaps and intermediate global memory round-trips.

Graph-capture compilers such as `torch.compile`[13] automate this fusion over captured regions without requiring engineers to write custom CUDA kernels.[14] However, compilation cannot span arbitrary Python code: graph breaks mark the boundary where compilation ceases to apply. Data-dependent Python control flow, unsupported library calls, I/O, custom Python objects, and highly variable shapes can shorten compiled regions. With eager fallback enabled, a break separates compiled subgraphs with standard Python execution, reintroducing interpreter dispatch and limiting fusion across the boundary. Tensor handles can pass between these regions while their storage remains on the device; host reads of device values can additionally require synchronization. This is why graph-break analysis belongs in performance engineering: the relevant metric is not whether compilation is enabled, but how much of the hot path remains inside long, stable compiled regions.

Backends occupy different points on the flexibility-performance spectrum. A **JIT** compiler traces model execution at runtime to generate optimized kernels dynamically, incurring initial warmup latency during the first pass. An **ahead-of-time (AOT)** compiler instead compiles captured computa-

[13] `torch.compile`: PyTorch's capture front end analyzes Python bytecode as each frame runs, rather than recording tensor operations alone. That vantage lets it keep capturing across Python control flow it can resolve from guarded values and insert a graph break where it cannot, which is why graph breaks rather than tracing errors are its characteristic failure mode.

[14] **CUDA (compute unified device architecture)**: NVIDIA's parallel computing platform, released in 2007, serving as the foundational layer between high-level Python operations and GPU silicon; when PyTorch executes `torch.matmul(A, W)`, the call traverses the framework's dispatcher, selects a cuBLAS kernel, and launches it on the GPU. Each launch adds the 5 µs kernel-launch share of the per-operation overhead on top of the framework's Python dispatch. For small operations, this dispatch overhead (L_{lat}) can exceed useful compute time. Fusing compatible groups of operations then reduces launch count as well as intermediate memory traffic; capturing the graph alone does not imply a single kernel.

tion before deployment, shifting compilation out of the hot path and enabling a standalone runtime when the export target supports one. A general JIT backend optimizes ordinary training and serving workloads with moderate compilation cost; a specialized inference backend can apply deeper fusion, precision lowering, and autotuning when the deployment target is fixed; an ahead-of-time mobile or embedded runtime removes even more flexibility to gain footprint and predictability. The same rule governs all of them: the narrower the target and the more stable the graph, the more optimization the compiler can safely perform.

Compilation shortens the list of kernels; a second mechanism removes the per-launch cost of whatever list remains. CUDA Graphs record a fixed sequence of kernel launches, with their arguments and memory addresses, once, and then replay the whole sequence with a single submission from the host (NVIDIA Corporation 2026a). Replay requires static shapes and fixed buffer addresses, which is why serving systems capture one graph per batch size. For the GPT-2 XL decoding step that opened the chapter, replay turns 576 separate launches into one submission, so the host's launch rate no longer sets the step time; what remains is the weight read of about 0.9 ms plus the device-side cost of running those kernels back to back, which fusion attacks. Every later decode floor in this book assumes this kind of capture.

The resulting workflow is a systems decision. Rapid prototyping favors eager execution because architecture changes and guard failures make recompilation cost visible. Long training runs and high-volume inference amortize compilation cost over many executions, provided the model has stable shapes and limited graph breaks. Debugging usually starts in eager mode because errors map directly to source code and standard debuggers such as `pdb` work mid-execution; compilation is reintroduced after the model behavior is correct and the performance bottleneck is measurable.

↩ The cost that slow experiment cycles impose on how many candidate models a team can test is established in section 3.3.

Set side by side across six dimensions (table 7.2), the three execution models differ most in when the graph is built and how much of it a compiler can see. Each row highlights how a design choice at graph construction time propagates into control flow, debugging, and runtime throughput.

Table 7.2: Execution Model Trade-Offs: Each execution model occupies a distinct position in the flexibility-optimization trade-off space. Eager execution maximizes debugging flexibility but limits cross-operation optimization; static graphs maximize optimization but require control flow to be represented in the graph; hybrid JIT compilation compiles captured regions while falling back to eager execution for unsupported patterns.

Aspect	Eager + Autograd Tape (PyTorch default)	Static graph (TensorFlow 1.x)	JIT Compilation (torch.compile)
Execution Model	Immediate	Deferred	Hybrid
Graph Construction	During forward pass	Before execution	First execution (cached)
Optimization	Per-operation kernels	Ahead-of-time	JIT compilation
Dynamic Control Flow	Python control flow	Graph control flow	Captured regions or breaks
Debugging	Easy (standard Python)	Difficult (symbolic)	Moderate (mixed)
Performance	Baseline	High (optimized)	High (compiled regions)

The same divide extends to memory planning, hardware specialization, research velocity, and integration with existing code (table 7.3). Where static capture invests upfront in planned buffer lifetimes and target-specific code generation, dynamic execution prioritizes rapid iteration and seamless composition with ordinary host code.

Table 7.3: Additional Graph Trade-Offs: Static graphs buy planned memory and hardware-specific code at the price of a define-then-run loop, while dynamic graphs buy iteration speed and natural integration with imperative code, complementing table 7.2. These trade-offs reappear when selecting frameworks in section 7.9.

Aspect	Static Graphs	Dynamic Graphs
Memory Management	Precise allocation planning, optimized memory usage	Flexible but potentially less efficient
Hardware Utilization	Can generate highly optimized hardware-specific code	May sacrifice hardware-specific optimizations
Research Velocity	Slower iteration due to define-then-run requirement	Faster prototyping and model experimentation

Aspect	Static Graphs	Dynamic Graphs
Integration with Legacy Code	More separation between definition and execution	Natural integration with imperative code

These trade-offs are not binary choices. Modern frameworks offer a spectrum of execution options, which raises the quantitative question of where on this spectrum a given project should operate.

7.3.7 Compilation and dispatch costs

Two quantitative principles make the choice of execution model measurable. The compilation continuum principle asks when compilation gains justify development cost by comparing production executions with development iterations. The dispatch overhead law writes the dispatch overhead ratio in terms of operation count and per-operation cost, giving a threshold that says when a whole workload is overhead-bound.

7.3.7.1 The compilation continuum principle

Equation 7.1 orders the execution models from maximum flexibility to maximum optimization in four positions, and each labeled arrow names the mechanism that carries a project one step rightward toward hardware:

$$\text{Eager} \xrightarrow{\text{capture}} \text{Graph JIT} \xrightarrow{\text{export}} \text{AOT Runtime} \xrightarrow{\text{specialize}} \text{Hardware} \tag{7.1}$$

Each step rightward sacrifices flexibility for performance. The practical question is *where* a given project should be placed on this continuum. The optimal compilation strategy depends on production executions, development recompilations, and their respective time costs, combined in equation 7.2:

$$\text{Compilation Benefit} = \frac{N_{\text{prod}} \cdot (T_{\text{eager}} - T_{\text{compiled}})}{T_{\text{compile}} + N_{\text{dev}} \cdot T_{\text{compile}}} \tag{7.2}$$

where:

- N_{prod} = number of production executions (dimensionless count: inference requests, training steps)
- N_{dev} = number of development iterations requiring recompilation (dimensionless count)
- T_{eager} = time per execution in eager mode (seconds)
- T_{compiled} = time per execution in compiled mode (seconds)
- T_{compile} = time per compilation or recompilation (seconds; assumed constant)

This model assumes that the compiled artifact remains valid between development changes and that the initial compilation and each recompilation have the same cost; under these assumptions, the decision rule is to compile when Compilation Benefit > 1. The ratio is dimensionless.

Table 7.4 applies that decision rule to hypothetical throughputs and compilation times across execution modes and model architectures:

Table 7.4: Illustrative Inference Throughput: The values are hypothetical inference throughputs on a data-center accelerator at batch size 32, assuming matching input shapes, precision, and numerical requirements across each row; they are assumptions, not benchmark measurements. Within this scenario, the table implies torch.compile speedups of 1.4–1.5× over eager mode and TensorRT (NVIDIA's ahead-of-time inference compiler, section 13.6) speedups of 2.3–2.7×. The setup column applies to torch.compile; TensorRT build costs require a separate measurement.

Model	Eager (examples/sec)	torch.compile (examples/sec)	TensorRT (examples/sec)	JIT Setup (seconds)
ResNet-50	1,450	2,150	3,800	15–30
BERT-Base	380	520	890	30–60
ViT-B/16	620	950	1,650	25–45
GPT-2 (124M)	180	260	420	45–90

Three cases illustrate how reuse changes the compilation decision. In research prototyping ($N_{\text{dev}} \gg N_{\text{prod}}$), teams should stay eager. If the architecture changes every few minutes, compilation

overhead can dominate. Under the scenario's 30 s compilation assumption, 10 compilations per hour consume 5 min.

For repeated execution ($N_{\text{prod}} \gg N_{\text{dev}}$), compilation can pay off because its one-time cost is amortized across many calls. In the hypothetical ResNet-50 inference scenario in table 7.4, torch.compile provides 48.3 percent higher throughput (2,150 img/s vs. 1,450 img/s). Using the scenario's 30 s compile cost, this pays off after the breakeven point in equation 7.3:

$$N_{\text{breakeven}} = \frac{T_{\text{compile}}}{T_{\text{eager}} - T_{\text{compiled}}} \tag{7.3}$$

Evaluating equation 7.3 with the hypothetical ResNet-50 values gives approximately 134,000 images. The same amortization principle applies to long training runs, using training throughput and compilation costs measured for that workload.

Long-lived production inference ($N_{\text{dev}} \approx 0$, $N_{\text{prod}} \to \infty$) extends this reuse further, providing more opportunity to amortize compilation. Aggressive autotuning can be worthwhile even when setup takes much longer, provided the model converts correctly, meets numerical requirements, and improves measured performance over the deployment lifetime.

The cases form a continuum of increasing reuse (figure 7.8). Each strategy's total time starts at its one-time compilation cost and grows at its per-execution cost, so eager wins at low execution counts, JIT wins once its compilation is amortized, and AOT wins only at the largest counts, where its higher upfront cost is fully repaid. The figure's crossovers (near 2×10^6 and 6.7×10^6 executions) come from an illustrative cost model in arbitrary units, not from the ResNet-50 scenario, whose eager-to-compiled breakeven sits near 134,000 images. What carries over is the order of the regions, not their positions.

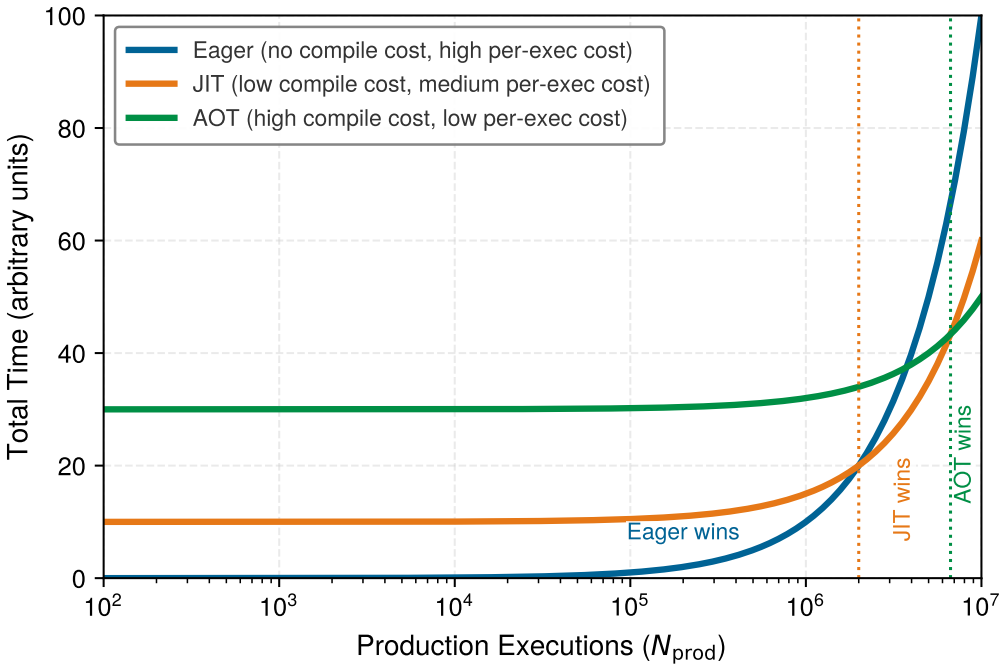


Figure 7.8: The Compilation Continuum: An illustrative cost model shows how one-time compilation cost and per-execution cost create eager, JIT, and AOT crossover regions as production executions increase. The crossover locations depend on the assumed compilation costs and per-execution rates.

These throughput differences are not uniform across workloads, so the execution strategy must align with the operational regime of each workload archetype. The appropriate strategy depends on the computational phase, the target backend, and the dominant iron law term. In batched prefill,

transformers have high arithmetic intensity that benefits from dense matrix compilation; in small-batch autoregressive decoding, however, the workload shifts into a bandwidth-bound regime where memory transfers dominate. Table 7.5 maps each recurring archetype to a candidate execution strategy based on these primary physical constraints.

↩ The workload archetypes and the iron law term that dominates each are established in section 2.3.2.

Table 7.5: Framework Execution Strategy by Workload: Each workload archetype pairs with a candidate execution strategy aligned to its dominant iron law term. Fusion cannot lift the bandwidth-bound row's weight-streaming limit; only larger batches or lower precision move it. For the power-bound row, memory capacity is a feasibility gate that must be met before any iron law term matters. For the capacity-bound row, the bandwidth and latency are those of irregular embedding gathers and the network exchange that follows them, not the streaming peak.

Archetype	Dominant Iron Law Term	Candidate Framework Strategy	Rationale
ResNet-50 (compute-bound)	$\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$ (Compute)	Compiled dense kernels	Regular dense kernels benefit from layout selection, precision lowering, and backend specialization; fusion helps most in surrounding memory- or launch-bound regions
GPT-2 (bandwidth-bound)	$\frac{D_{\text{vol}}}{\text{BW}}$ (Memory Bandwidth)	Fused attention + graph compilation	Fused attention eliminates intermediate activation round-trips, while compilation removes kernel launch overhead; streaming parameter and KV-cache traffic remain bandwidth-limited
DLRM (capacity-bound)	Capacity first, then $\frac{D_{\text{vol}}}{\text{BW}} + L_{\text{lat}}$ (gathers)	Specialized lookup kernels and measured capture	Irregular gathers limit some optimizations; measure compilation benefits in dense regions and surrounding dispatch
DS-CNN (power-bound)	Energy and memory capacity (feasibility gates)	Static-arena microcontroller runtime	Fixed memory and a small native runtime must fit the target before latency can be optimized

The rows differ in what a compiler can reach. Dense convolution exposes most of its work to fusion and layout selection, while embedding gathers and a microcontroller's fixed memory sit largely outside the graph a compiler optimizes.



Lighthouse 7.1: Framework strategy by archetype

The four recurring archetypes in table 7.5 trace one principle across the framework: compilation benefits scale with how much of the workload is optimizable.

Systems insight: In the illustrative scenario, compute-bound workloads such as the ResNet-50 row in table 7.4 benefit because their dense kernels expose substantial surface for layout selection, precision lowering, and fusion. Bandwidth-bound workloads such as small-batch GPT-2 generation require distinguishing intermediate activation traffic from parameter streaming: fusing attention kernels eliminates round-trips for transient activation tensors and removes dispatch latency, but memory bandwidth continues to bind on streaming model weights and key-value caches unless batching increases arithmetic intensity. Capacity-bound workloads such as DLRM can gain less when irregular embedding lookups leave limited compiler scope.

7.3.7.2 The dispatch overhead law

A second principle, the dispatch overhead law, emerges from equation 7.4, which identifies the regime in which framework overhead, rather than compute or memory, dominates execution time. Let N_{ops} be the number of operations (count), t_{dispatch} the per-operation dispatch overhead (seconds), and T_{compute} and T_{memory} the total compute and memory times (seconds). The ratio uses the additive, no-overlap approximation $T_{\text{hw}} \approx T_{\text{compute}} + T_{\text{memory}}$; when execution overlaps those costs, measured hardware time should replace their sum. Framework overhead dominates when operations are small relative to dispatch cost:

$$\text{Overhead Ratio} = \frac{N_{\text{ops}} \cdot t_{\text{dispatch}}}{T_{\text{compute}} + T_{\text{memory}}} \tag{7.4}$$

When Overhead Ratio > 1, the model is overhead-bound. This is the dispatch overhead ratio $T_{\text{sw}}/T_{\text{hw}}$ of section 7.3.4, written with $T_{\text{sw}} = N_{\text{ops}} \cdot t_{\text{dispatch}}$ so that operation count and per-operation cost appear explicitly. The fraction of step time spent in dispatch is Overhead Ratio$/(1+$ Overhead Ratio$)$, so a ratio above 1 means dispatch takes more than half of the step. Compilation can provide its largest gains for overhead-bound workloads because fusion reduces the number of dispatches. The training-step trace in section 7.7 works this effect through end to end, previewed by the numbers that follow.

Applying equation 7.4 means comparing the software overhead $T_{\text{sw}} = N_{\text{ops}} \cdot t_{\text{dispatch}}$ with the hardware execution time T_{hw} (both in seconds). The regime depends on the size of individual operations rather than on the model's parameter count. Notebook 7.2 contrasts a small multilayer perceptron with a large transformer layer to quantify when host overhead or device arithmetic bounds the system.

Napkin Math 7.2: Dispatch overhead at two scales

Problem: Determine the threshold where host-side Python interpreter dispatch overhead bounds end-to-end execution time relative to accelerator hardware execution.

Scenario one: Small multilayer perceptron (overhead bound)

- **Compute**: 6 ops across small matrix/element-wise operations.
- **Hardware time**: $T_{\text{hw}} \approx 2.6$ µs (mostly memory latency).
- **Software overhead**: $T_{\text{sw}} \approx 6$ ops $\times$ 15 µs/op = 90 µs.
- **Ratio**: 90 µs/2.6 µs ≈ 34.6.
- **Small-model outcome**: The system spends 97 percent of time in host-side dispatch and kernel-launch overhead. Eliminating all modeled dispatch overhead would give an upper-bound speedup of 35.6×.

Scenario two: GPT-3 layer (device bound)

- **Compute**: Huge matrix multiplications.
- **Assumptions**: Both times below are illustrative round values for one GPT-3-scale layer over a large batch, not derived from a specific accelerator. The conclusion does not hinge on them, because at the chapter's 15 µs/op the ratio would reach 1 only if the layer issued thousands of separately dispatched kernels.
- **Hardware time**: $T_{\text{hw}} \approx 100$ ms = 100000 µs.
- **Software overhead**: $T_{\text{sw}} \approx 180$ µs, one transformer block's 12 eager kernels at 15 µs/op.
- **Ratio**: 180 µs/100000 µs ≈ 0.0018.
- **Large-model outcome**: Modeled Python overhead is negligible. Compilation may still improve fusion, layout, or backend specialization; dispatch elimination contributes little.

Systems insight: Dispatch overhead is regime-dependent. Compilation can reduce host-side overhead for small-operation workloads by fusing operations into fewer launches, while large models benefit mainly from fused kernels and memory movement reductions.

7.3.8 Static runtimes for microcontrollers

The compilation continuum ends at the far edge, where the binding cost on a microcontroller is not each dispatch but the runtime itself, whose memory and latency footprint exceeds the device. While cloud frameworks such as PyTorch and TensorFlow 2.x prioritize flexibility through eager execution, TinyML[15] systems operating on microcontrollers (MCUs) with kilobytes of memory cannot afford the overhead of a Python interpreter or a fully dynamic runtime.

[15] **TinyML**: Systems designed for MCUs cannot afford a Python interpreter. Micro-runtimes instead use small C/C++ runtimes, fixed memory planning, and model-specific operators. TensorFlow Lite Micro remains interpreter-based: the application supplies a tensor arena whose buffers are planned and reused without heap allocation after setup. Predictable memory is mandatory because exceeding a 256 KB device budget can prevent initialization or inference.

Lighthouse 7.2: Lighthouse example: KWS on TinyML

Scenario: Deploying the keyword spotting (KWS) lighthouse, a DS-CNN and the book's power-bound workload, to an Nordic nRF52840 (Cortex-M4F) microcontroller with 256 KB RAM and 1 MB Flash. This is a smaller target than the ESP32-S3 of the earlier smart-doorbell scenario, chosen to show the runtime at its tightest.

Constraint: A full PyTorch and Python runtime exceeds the device's memory budget by orders of magnitude.

Framework solution: Micro-frameworks such as **TensorFlow Lite Micro (TFLM)** (David et al. 2021) solve this through a tiny interpreter-based runtime with a fixed memory discipline:

1. **Fixed memory arena**: The application supplies a contiguous tensor arena, and the framework plans and reuses buffers from that arena rather than relying on dynamic allocation during inference.
2. **Kernel selection**: Only the specific kernels used by the model (for example, Conv2D, DepthwiseConv) need to be linked or registered with the runtime.
3. **Compact interpreter execution**: The MCU runs a small C/C++ interpreter over a flat model representation, with the model and arena bound at initialization rather than assembled dynamically at runtime.

Binding constraint: The power budget makes keyword spotting a power-bound workload, but the first gate the framework must pass is memory capacity, not bandwidth or dispatch overhead, which is why table 7.5 treats it as a feasibility gate. The framework's primary job is to ensure the model's intermediate activations (the "working set") fit within the MCU's tiny SRAM.

These micro-runtimes sacrifice dynamic flexibility to reduce footprint and make memory predictable. With compact models, low-power hardware, and duty cycling, that discipline enables milliwatt-scale inference with mostly on-chip data movement. Across the full spectrum from dynamic eager execution to static graph compilation and specialized micro-runtimes, developers must therefore match the execution strategy to the binding constraint of the target.

Checkpoint 7.1: Execution models

The choice of execution mode determines both developer velocity and model performance.

Debuggability vs. speed

- ☐ **Eager mode (Python-first)**: why does executing ops one-by-one make debugging easy but optimization hard?
- ☐ **Graph mode (compiler-first)**: why does building a static graph enable kernel fusion?

Modern compromise

- ☐ **JIT compilation**: how does `torch.compile` bridge the gap?
- ☐ **Dispatch-overhead estimate**: if 6 ops each incur the chapter's 15 µs/op dispatch overhead while hardware work takes 2.6 µs, which term dominates?

The execution problem determines when computation happens and what optimizations are possible. Neural network training, however, requires a capability that no amount of clever scheduling can provide: the ability to compute gradients automatically.

7.4 Differentiation: Automatic Gradient Computation

Training requires determining how an incremental change in each of millions of parameters alters the scalar loss. Doing this manually for even a simple three-layer network requires deriving and implementing dozens of partial derivatives; for a modern transformer with billions of parameters,

manual differentiation is intractable. A framework that executes efficiently but cannot differentiate can run inference but cannot learn.

The **differentiation problem** is the task of computing gradients[16] automatically. Training requires the derivative of a scalar loss $\mathcal{L}$ with respect to every parameter. When executing the forward pass, the runtime records an execution tape and saves intermediate activation buffers for subsequent derivative evaluations. Backpropagation already showed why reverse-mode automatic differentiation (AD)[17] suits this shape of problem, since one reverse traversal from a single scalar loss yields every parameter gradient, a claim the forward-versus-reverse comparison below makes precise. Major ML frameworks therefore use reverse-mode AD for ordinary scalar-loss training (Baydin et al. 2018).

Building on the backpropagation algorithm, the systems engineering of differentiation addresses how frameworks represent computation graphs, manage memory for intermediate values, and orchestrate the backward pass efficiently across accelerators. The framework mechanizes the chain rule as large-scale systems bookkeeping: tracking operator dependencies, staging activation buffers, and dispatching backward kernels. The function in listing 7.4 reduces the core idea to three elementary operations.

Listing 7.4: Automatic Differentiation: The function $f(x) = x^2 \sin(x)$ breaks into a square, a sine, and a product, each with a known local derivative.

```
def f(x):
    a = x * x  # Square
    b = sin(x)  # Sine
    c = a * b  # Product
    return c
```

Frameworks decompose this function into elementary operations, each with a known local derivative, and then combine these local derivatives via the chain rule to compute gradients through supported compositions. The systems challenge is executing this transformation under physical hardware constraints: the framework must represent the computation graph, make necessary forward values available, and schedule backward kernels without exhausting accelerator HBM. Production runtimes address these challenges through structured graph recording, caching allocators, and reverse-mode traversal.

7.4.1 Forward and reverse mode

Computing the gradient of a scalar loss across millions of model weights one weight at a time would require millions of separate passes through the network, whereas sweeping backward from the single scalar loss recovers every weight gradient in one reverse pass. Automatic differentiation operates in these two traversal directions: forward mode, whose cost scales with the number of inputs, and reverse mode, whose cost scales with the number of outputs. Because deep learning objectives reduce a very large parameter set to a single scalar loss, this asymmetry dictates both the memory footprint and the execution schedule of the underlying framework.

7.4.1.1 Forward mode

Forward mode automatic differentiation computes directional derivatives alongside the forward calculation itself. For each intermediate variable v_i, the runtime evaluates both the value v_i and its directional derivative (tangent) $\dot{v}_i = \sum_j \frac{\partial v_i}{\partial x_j} \dot{x}_j$. Because tangents flow forward alongside the values, forward mode requires no execution tape and never retains forward activations for a later backward pass.

To see the mechanism concretely, consider computing both the value and derivative of $f(x) = x^2 \sin(x)$. Listing 7.5 shows how forward mode evaluates local derivative rules alongside each elementary operation, applying the chain and product rules at every step:

16 | **Gradients**: A gradient holds one value for every parameter it differentiates, so each weight tensor acquires a gradient tensor of the same shape. Computing gradients automatically therefore doubles the parameter-sized state before any optimizer runs, which is why training-memory accounts, such as the one in section 5.4.5.5, list gradients as a full copy of the weights.

17 | **Reverse-mode AD**: Reverse mode's single backward pass has a concrete memory cost, because the operation-dependent values that derivative rules read must be retained until the backward traversal reaches them. Activation checkpointing changes this saved set by omitting selected values and recomputing them during the backward pass.

↪ The optimization algorithms that consume these gradients are developed in section 8.3.2.

Listing 7.5: Forward Mode AD: Forward mode propagates derivatives forward through the computation graph, computing one directional derivative per seeded forward pass.

```
def f(x):  # Computing both value and derivative
    # Step 1: x -> x²
    a = x * x  # Value: x²
    da = 2 * x  # Derivative: 2x

    # Step 2: x -> sin(x)
    b = sin(x)  # Value: sin(x)
    db = cos(x)  # Derivative: cos(x)

    # Step 3: Combine using product rule
    result = a * b  # Value: x² * sin(x)
    dresult = a * db + b * da  # Derivative: x²*cos(x) + sin(x)*2x

    return result, dresult
```

Forward mode mechanizes this paired tracking through dual numbers, values of the form $v + \dot{v}\epsilon$ where $\epsilon^2 = 0$, so each operator updates the value and its tangent together without retaining a graph. Seeding $\dot{x} = 1.0$ carries the derivative through the trace (listing 7.6).

Listing 7.6: Dual-Number Trace: Evaluating the forward-mode function of listing 7.5 at $x = 2.0$ keeps a value and its derivative on every line, so one pass yields both $f(2.0) \approx 3.637$ and $f'(2.0) \approx 1.973$.

```
x, dx = 2.0, 1.0  # seed: track the derivative with respect to x

a = x * x  # value:      4.0
da = 2 * x  # derivative: 4.0
b = sin(x)  # value:      0.9093
db = cos(x)  # derivative: -0.4161
c = a * b  # value:      3.637
dc = a * db + b * da  # derivative: 1.973
```

A single forward pass with a seeded tangent vector $\dot{\mathbf{x}}$ evaluates a Jacobian-vector product (JVP), $\mathbf{J}\dot{\mathbf{x}}$, where $\mathbf{J} \in \mathbb{R}^{M \times P}$ is the Jacobian matrix. When an objective maps P parameter inputs to a single scalar loss ($M = 1$), recovering the full gradient $\nabla_\theta \mathcal{L} \in \mathbb{R}^P$ requires setting $\dot{\mathbf{x}}$ to each of the P standard basis vectors in turn. Recovering gradients with forward mode therefore requires P distinct forward evaluations (or propagating P-wide tangent bundles), scaling compute and memory traffic as $\mathcal{O}(P)$ with parameter count. For networks with millions or billions of parameters, this scaling makes forward-mode gradient calculation intractable, although forward JVPs remain effective when input dimensions are small or when evaluating directional derivatives and Hessian-vector products.

7.4.1.2 Reverse mode

An intermediate layer that maps an input vector $\mathbf{x} \in \mathbb{R}^N$ to an output $\mathbf{y} \in \mathbb{R}^M$ of width $N = M = 100{,}000$ has a full Jacobian $\mathbf{J} \in \mathbb{R}^{M \times N}$ of 10^{10} elements, which would require 40 GB of memory in FP32 just to store one layer's derivatives. Reverse-mode automatic differentiation avoids materializing this matrix and matches the many-parameters-to-one-loss shape of neural network training. Instead of propagating tangents forward from inputs, reverse mode propagates adjoints $\bar{v}_i = \partial \mathcal{L} / \partial v_i$ backward from the single scalar loss $\mathcal{L}$ to all parameters.

While forward mode computes Jacobian-vector products ($\mathbf{J}\dot{\mathbf{x}}$), reverse mode evaluates vector-Jacobian products (VJPs), $\mathbf{v}^T\mathbf{J} = \sum_{j=1}^{M} \bar{y}_j \, \partial y_j / \partial \mathbf{x}$, where $\bar{y}_j = \partial \mathcal{L} / \partial y_j$ is one element of the upstream gradient vector $\mathbf{v} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^M$. Because training optimizes a single scalar loss $\mathcal{L}$, the backward pass only ever needs to multiply this upstream gradient vector by each local operation's derivative rule, never the 40 GB full Jacobian matrix.

To see how VJPs sidestep the full matrix, consider a simple element-wise operation such as $\mathbf{y} = \text{ReLU}(\mathbf{x})$ with an input vector $\mathbf{x} = [-1.0, 2.0, 3.0]$ and an upstream gradient $\mathbf{v} = [0.5, 0.5, 0.5]$

flowing backward from the loss. Because the derivative of ReLU is 1 when $x > 0$ and 0 otherwise, the local gradient vector is $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = [0, 1, 1]$. Instead of building a 3×3 diagonal matrix and executing a matrix multiplication, the VJP computes the element-wise product $\mathbf{v} \odot \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = [0.0, 0.5, 0.5]$ directly, streaming vectors without ever materializing the massive $M \times N$ matrix in memory. Seeding the reverse sweep with $\bar{\mathcal{L}} = 1$ yields the complete parameter gradient vector $\nabla_\theta \mathcal{L}$ in a single backward pass, regardless of whether the model contains ten parameters or ten billion.

Reverse mode operates on the same three-operation function $f(x) = x^2 \sin(x)$ from listing 7.4, where x reaches the output through two distinct computational paths: the square and the sine. Algorithm 7.1 defines the reverse-mode traversal algorithm over a directed acyclic graph before tracing this concrete evaluation.

Algorithm 7.1 Reverse-mode automatic differentiation

Require: directed acyclic graph with scalar output y; nodes $v_1, \ldots, v_n$ in topological order; a local derivative rule for each node
Ensure: adjoints $\bar{v}_i = \partial y / \partial v_i$ for the requested input or parameter nodes

for $i = 1$ to n **do**
  compute v_i, record dependencies, and save values requested by its derivative rule ▷ *forward pass*
end for
$\bar{v}_i \leftarrow 0$ for all i; $\bar{y} \leftarrow 1$
for $i = n$ down to 1 **do**
  for each parent u of v_i **do**
    $\bar{u} \leftarrow \bar{u} + \bar{v}_i \, \partial v_i / \partial u$ ▷ *accumulate via the chain rule*
  end for
  release stored values once every rule that needs them has run
end for
return the adjoints for the requested nodes

The algorithmic structure of algorithm 7.1 reveals its systems cost. The forward pass records operator dependencies and buffers intermediate values required by backward derivative rules until the reverse sweep consumes them. Reverse mode trades activation storage and graph bookkeeping for a single backward pass, whose cost is a small constant multiple of the forward pass regardless of parameter count (Baur and Strassen 1983; Griewank and Walther 2008).

For the concrete function in listing 7.4 evaluated at $x = 2.0$, the forward pass records $a = x^2 = 4.0$, $b = \sin(x) \approx 0.9093$, and $c = ab \approx 3.637$. Seeding $\bar{c} = 1.0$, the product rule yields adjoints $\bar{a} = 0.9093$ and $\bar{b} = 4.0$. The square branch contributes $0.9093 \cdot 4.0 \approx 3.6372$ to $\bar{x}$, while the sine branch contributes $4.0 \cos(2.0) \approx -1.6646$. Accumulating both paths gives the exact derivative $\partial c / \partial x = \bar{x} \approx 1.973$.

This single backward traversal computes $\partial c / \partial x$ regardless of how many fan-out paths connect x to c. In a deep neural network, every weight tensor influences the scalar loss through thousands of divergent paths across successive layers; reverse mode aggregates all incoming adjoints in one reverse sweep. That one sweep, rather than one pass per parameter, is what keeps training a 175B-parameter model such as GPT-3 computationally tractable.

Executing this traversal in hardware exposes a physical constraint: evaluating local derivatives during the backward pass requires the operating point established during forward execution. The framework must therefore determine which forward tensors to persist in device memory, when to allocate them, and when to free them. The recorded graph answers the "when to free" question, because each saved tensor can be released as soon as the last backward rule that reads it has run. The peak itself is bounded separately, by splitting a large batch into micro-batches whose saved activations are freed before the next one starts while their gradients sum into the same buffers (gradient accumulation).[18]

A two-layer network makes these requirements concrete (listing 7.7). Tracing both passes explicitly shows which forward activations must survive until the reverse sweep consumes them.

[18] **Gradient accumulation**: The framework processes a large logical batch as smaller micro-batches, freeing each micro-batch's saved activations after backward while accumulating parameter-sized gradients. Processing 4,096 samples in 64-sample micro-batches can reduce the batch-dependent activation peak by up to 64× when activation memory scales linearly with batch size; weights, gradients, optimizer state, and other fixed costs do not shrink. The trade-off is more sequential work before each optimizer update.

Listing 7.7: Reverse Mode in a Neural Network: The forward pass computes and stores intermediate values; the backward pass walks the computation in reverse to produce gradients for every parameter.

```
def simple_network(x, w1, w2):
    hidden = x * w1  # First layer
    activated = max(0, hidden)  # ReLU activation
    output = activated * w2  # Second layer
    return output


# --- Forward pass stores intermediates ---
x, w1, w2 = 1.0, 2.0, 3.0
hidden = x * w1
activated = max(0, hidden)
output = activated * w2

# --- Backward pass consumes them ---
d_output = 1.0  # Seed gradient
d_w2 = activated  # = 2.0
d_activated = w2  # = 3.0
d_hidden = d_activated * (1 if hidden > 0 else 0)  # ReLU gate: 3.0
d_w1 = x * d_hidden  # = 3.0
d_x = w1 * d_hidden  # = 6.0
```

Three implementation requirements emerge from this example. First, the framework must track dependencies between operations to determine the correct reverse traversal order. Second, the information needed by each backward rule must be available, either saved during the forward pass or regenerated. The chain rule evaluates local derivatives at the forward operating point. For example, evaluating $\frac{\partial}{\partial z}\text{ReLU}(z) = \mathbb{I}(z > 0)$ requires the sign of the pre-activation z, which can be recovered from a saved value or mask, and computing a dense layer's weight gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}}$, where $\mathbf{Z}$ is the layer's pre-activation output, requires the forward input activation tensor $\mathbf{X}$. Third, every differentiated operation needs both a forward implementation and a corresponding backward rule. These requirements define the engineering surface of automatic differentiation, where retaining forward values can consume much of the training memory budget.

7.4.2 Activation memory

A 175B-parameter model in FP16 requires 350 GB just for weights, exceeding the physical memory of any single accelerator. Reverse-mode AD compounds this memory pressure by retaining operation-dependent forward activations for backward derivative rules. Saved activations weigh just as heavily on models whose weights fit easily. In a batch-64 ResNet-50 (worked below in section 7.4.3.2), saved activations exceed the FP32 weights by roughly two orders of magnitude, on top of weights, gradients, and optimizer states. Memory capacity, rather than raw floating-point throughput, frequently establishes the feasibility boundary of training.

This saved-activation footprint accumulates across successive layers, because each forward layer adds tensors that must persist until the backward sweep unwinds to that layer (listing 7.8). For a batch of B sequences of length S passing through N_L layers of width d, each saving n_{saved} tensors of s_{elem} bytes per element, the saved state is approximately

$$M_{\text{activations}} \approx B \times N_L \times S \times d \times n_{\text{saved}} \times s_{\text{elem}}$$

This total grows linearly in both batch size and network depth. Consequently, the batch size that fits on an accelerator during training is far smaller than the batch size that fits during inference, where intermediate activations can be discarded as soon as the next layer consumes them.

Listing 7.8: Activation Persistence Across Layers: The marked intermediates represent values retained for backward; peak saved-activation memory depends on their number and sizes.

```
def deep_network(x, w1, w2, w3):
    # Forward pass - must store intermediates
    hidden1 = x * w1
    activated1 = max(0, hidden1)  # Store for backward
    hidden2 = activated1 * w2
    activated2 = max(0, hidden2)  # Store for backward
    output = activated2 * w3
    return output
```

This is a capacity limit rather than the bandwidth gap of the memory wall, and frameworks relieve it through two primary architectural strategies. The first is activation checkpointing (or gradient checkpointing), a reverse-mode technique (Griewank and Walther 2000) that deep learning frameworks adopted to train deep networks in sublinear memory (T. Chen et al. 2016). Rather than buffering every intermediate activation, the runtime stores only selected segment boundary tensors and recomputes dropped intermediates on demand during the backward pass. In listing 7.9, the backward pass re-executes each forward segment to regenerate its activations rather than loading them from global memory.

↪ Activation checkpointing schedules and their recomputation cost are developed in section 8.6.5.2.

Listing 7.9: Activation Checkpointing: The forward pass stores only segment boundaries; the backward pass re-runs each segment to regenerate the dropped activations, trading recomputation for memory.

```
# Standard backward: every forward activation stays resident
h1 = layer1(x)  # kept for backward
h2 = layer2(h1)  # kept for backward
out = layer3(h2)  # kept for backward

# Checkpointed: keep only the boundary h1 and drop h2. The
# backward pass re-runs the wrapped segment to regenerate
# h2 on demand instead of holding it in memory.
h1 = layer1(x)  # boundary: kept
out = checkpoint(
    lambda a: layer3(layer2(a)), h1
)  # h2 recomputed in backward
```

The second strategy is the kernel fusion that the memory wall motivated in section 7.3.2, applied here to the body of a layer. Its capacity benefit is narrower than its traffic benefit. Temporaries that pass only between the fused operators are never allocated in HBM, so they add nothing to peak memory, while any value a backward rule reads must still be stored.[19] Rather than executing matrix multiplication, bias addition, and activation functions as separate kernels that serialize intermediate tensors to accelerator global memory (HBM), compilers fuse compatible operators into a single kernel that passes intermediate values through registers or on-chip shared memory.

The backward pass itself relies on specialized hardware-optimized kernels. Rather than synthesizing backward passes from naive elementary operations, frameworks invoke tuned backward kernels (such as cuDNN primitives) that optimize memory access strides and register usage on accelerator hardware (Chetlur et al. 2014). Combining checkpointing, kernel fusion, and specialized backward kernels enables large architectures to train within physical accelerator memory limits without incurring out-of-memory faults.

[19] **Fusion and saved state**: Fusion removes the forward round-trip of an intermediate through HBM, but it does not remove the backward pass's need for that value. If a derivative rule reads a fused intermediate, such as the pre-activation whose sign forms the ReLU mask, the fused kernel must still write that value (or a compact mask) to memory, or the backward pass must recompute it. Fusion and checkpointing therefore interact, because the compiler decides which intermediates to materialize from what the backward graph will read.

7.4.3 Autograd internals

Checkpointing, fusion, and specialized kernels address the physical bottlenecks of automatic differentiation, but production frameworks hide these mechanisms beneath a training loop of four calls that clear the gradients, run the forward pass, call backward on the loss, and step the optimizer. Behind those calls, the runtime builds and traverses a dynamic computation graph, allocates and frees intermediate buffers, schedules backward kernels on device streams, and applies a numerical-precision policy.

7.4.3.1 Recording the backward graph

If a forward operation discards its inputs and leaves no record of what function produced its output, calling `loss.backward()` has no path back to the model weights and no saved activations to plug into the chain rule. To make reverse traversal possible without ahead-of-time graph compilation, the autograd system dynamically constructs a directed acyclic graph of `Node` (or `Function`) objects during the forward pass. Each node stores an operator's backward differentiation rule and holds references to the tensors needed to evaluate its vector-Jacobian products. Because edges point backward from outputs toward inputs, the engine can walk the graph in reverse topological order as soon as it receives the initial gradient from the scalar loss; the memory that those node references keep alive is examined in section 7.4.3.2.

When gradient recording is enabled and an operation has inputs requiring gradients, its output carries a `grad_fn` referencing the backward `Function`. Each `Function` links to antecedent nodes through its `next_functions` list, forming a chain from the scalar loss back to leaf parameter tensors. Listing 7.10 inspects this pointer chain for a simple composite computation:

Listing 7.10: Reverse-Linked Graph Structure: Recorded outputs link through `grad_fn` to backward functions, forming a reverse chain from output to leaf parameters. One backward traversal computes the scalar loss gradient; its work grows with the graph.

```
import torch

x = torch.tensor([2.0], requires_grad=True)
y = x * 3
z = y.pow(2)


# Traverse the reverse-linked graph
print(z.grad_fn)  # PowBackward0
print(z.grad_fn.next_functions)  # -> MulBackward0
print(
    z.grad_fn.next_functions[0][0].next_functions
)  # -> AccumulateGrad (leaf)
```

The inspection exposes the traversal path: `PowBackward0` (from `z = y.pow(2)`) links to `MulBackward0` (from `y = x * 3`), which terminates at `AccumulateGrad` on the leaf parameter tensor `x`. Leaf tensors are graph sinks where accumulated gradients write directly into the `.grad` buffer rather than propagating further. The tuple format `(Function, index)` tracks the specific output index for multi-output operations.

This reverse-linked graph imposes a lifetime constraint: the topology and the forward information required by each backward rule must remain available until that rule executes. Host-side graph metadata (pointers and node descriptors) is distinct from the device tensor buffers it references. Retaining those buffers can consume gigabytes of accelerator capacity as model depth and batch size grow; recomputation or offloading changes where and how the required values are made available.

Production training also needs control over gradient propagation beyond a standard backward sweep, in three forms: selective gradient computation for freezing layers during transfer learning via `requires_grad=False` or `.detach()`; gradient inspection and modification for clipping, debugging, or logging intermediate adjoints via runtime hooks; and custom differentiation rules for non-standard operations such as specialized CUDA kernels or novel activation functions. These extension points operate via callbacks invoked during graph construction and reverse traversal, preserving the core engine's execution path. In PyTorch, these capabilities map to five runtime mechanisms, each with its own memory or correctness hazard (table 7.6).

Table 7.6: Autograd Control Mechanisms: Each extension point that the autograd engine exposes during graph traversal trades a specific control for a memory or correctness cost.

Mechanism	What it controls	Cost or hazard
`retain_graph=True`	Keeps saved graph state available for another backward traversal	Retains memory that would otherwise be released; the amount depends on the graph and saved values
Gradient accumulation	Successive backward passes sum into `.grad` until `zero_grad()` resets them, enabling large effective batches	Forgetting the reset silently mixes gradients across optimization steps
Custom `autograd.Function`	User-defined forward and backward rules for operations outside the built-in library	Makes the implementer decide what to save and how to differentiate
Gradient hooks	Inspecting or modifying gradients mid-traversal (clipping, logging, debugging)	Runs arbitrary Python per registered tensor on every backward pass
`.detach()`	Cuts gradient flow at a chosen boundary (frozen layers, inference outputs)	The legacy `.data` attribute bypasses autograd and silently corrupts gradients; clone before in-place mutation

Custom autograd functions move two decisions from the engine to the programmer, what to save and how to differentiate: the implementation explicitly defines which forward tensors to preserve in `ctx.save_for_backward()` and evaluates the corresponding vector-Jacobian products in `backward()`. Listing 7.11 demonstrates this interface for a custom multiply-add operation:

Listing 7.11: Custom Autograd Function: A custom multiply-add defines its own forward and backward rules and names the tensors that backward needs through `ctx.save_for_backward()`.

```python
import torch


class MultiplyAdd(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y, z):
        # Save tensors needed for backward
        ctx.save_for_backward(x, y)
        return x * y + z

    @staticmethod
    def backward(ctx, grad_output):
        # Retrieve saved tensors
        x, y = ctx.saved_tensors

        # Compute gradients using chain rule
        grad_x = grad_output * y  # dL/dx = dL/dout * dout/dx
        grad_y = grad_output * x  # dL/dy = dL/dout * dout/dy
        grad_z = grad_output  # dL/dz = dL/dout * 1

        return grad_x, grad_y, grad_z


# Usage
x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)
z = torch.tensor([1.0], requires_grad=True)

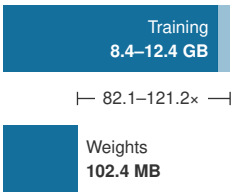
output = MultiplyAdd.apply(x, y, z)
output.backward()

print(
    x.grad, y.grad, z.grad
)  # tensor([3.]), tensor([2.]), tensor([1.])
```

7.4.3.2 Saved tensors and the memory-compute trade-off

In a retained-activation implementation, saved device tensors remain live until their backward consumers finish. These retained activations represent a primary driver of training memory overhead, compounding the footprint of weights, gradients, and optimizer states. Because different operations require different forward information (a matrix multiplication weight gradient requires input activations, while elementwise ReLU needs only the pre-activation sign), the exact memory profile depends on operator semantics and how the implementation stores that information.

Consider an illustrative ResNet-50 training scenario with 25.6M parameters (~102.4 MB of FP32 weights), batch size 64, and 224×224 images. With 8 GB–12 GB of saved activations, gradients add ~102.4 MB, and two Adaptive Moment Estimation (Adam) FP32 moment buffers add ~204.8 MB. Summing those four components gives an 8.4 GB–12.4 GB training scenario, roughly 82.1–121.2× the FP32 weight storage alone. This is a capacity-planning sum; actual peak memory depends on which components are live together. Inference requires no optimizer states, gradients, or saved backward buffers, but still has live activations and runtime workspace.



The training memory sum dwarfs FP32 weight storage alone.

Parameter count alone therefore does not tell an engineer whether training fits. Total training memory obeys the four-component formulation ($M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$) plus allocator overhead, as stated in section C.3.3.

Frameworks provide three graph-level controls to manage this trade-off. Activation checkpointing, whose mechanism listing 7.9 showed, alters what the graph retains: the engine stores selected segment boundaries and regenerates intermediate activations during the backward pass. In iron law terms, checkpointing increases the arithmetic operation count (O) to reduce peak activation storage; its effect on transfer volume (D_{vol}) depends on the recomputation path. Tensor detachment sets a gradient boundary: calling `.detach()` makes subsequent operations treat that tensor as independent of its prior computation. It does not retroactively free all previously saved state, and cutting a path changes which gradients are computed. Transfer learning shows the payoff. When a frozen feature extractor runs with gradient recording disabled and its output enters the trainable head as a constant, autograd saves none of the extractor's activations, so only the head pays the activation-storage cost. Mixed-precision training provides a third lever: representing eligible activations and weights in 16-bit formats halves their bytes relative to FP32, reducing tensor volume (D_{vol}) and enabling higher arithmetic throughput on supported Tensor Cores (R_{peak}).

The recorded graph fixes which backward kernels the device must execute (the O term), and the saved-tensor policy fixes how many bytes stay resident and move through the memory hierarchy (the D_{vol} term). Mixed precision acts on the second directly, because it shrinks every saved value and every operand a kernel reads.

↪ The decision of when these single-machine levers stop being enough and training must scale beyond one machine is developed in section 8.7.3.

7.4.3.3 Mixed-precision training support

Mixed precision (Micikevicius et al. 2018) improves two iron law terms at once. Tensor Cores run 16-bit matrix multiplications faster than FP32 units, raising R_{peak}, and eligible 16-bit activations halve the bytes stored and moved, reducing D_{vol}.

Automatic mixed-precision APIs implement this as a dispatch policy rather than a change to model code. Inside an `autocast` region, the dispatcher consults a per-operator precision list before selecting a kernel. Compute-intensive operations such as convolutions and linear projections run in FP16 or BF16, while numerically sensitive operations such as softmax, reductions, and loss calculations run in FP32. The second framework component, `GradScaler`, compensates for FP16's narrow exponent range (5 bits, with normal values bounded below by $\approx 6.10 \times 10^{-5}$). It multiplies the scalar loss by a loss-scale factor s_{loss} before backward traversal so that small gradients remain representable, divides the parameter gradients by s_{loss} before the optimizer update, and skips any step whose gradients overflow. BF16 keeps FP32's 8-bit exponent, so BF16 recipes usually run under `autocast` with the scaler disabled. The framework therefore supplies the dispatch and scaling machinery, while the choice among FP16, BF16, TF32 (an Ampere-class matrix-multiply format pairing FP32's exponent with FP16's mantissa), and FP8 and the way the loss scale adapts during training are policy decisions that section 8.6.3 develops, with the scaling mechanism itself in section 8.6.3.1. Listing 7.12 shows the two framework components working together in the FP16 path.

Listing 7.12: Mixed-Precision API: Modern frameworks provide automatic mixed-precision support through context managers that handle precision selection and numerical stability.

```
import torch
from torch.amp import autocast, GradScaler

model = MyModel().cuda()
optimizer = torch.optim.Adam(model.parameters())
scaler = GradScaler("cuda")

for inputs, targets in dataloader:
    inputs, targets = inputs.cuda(), targets.cuda()
    optimizer.zero_grad()

    # Framework automatically selects precision per operation
    with autocast(device_type="cuda", dtype=torch.float16):
        outputs = model(inputs)
        loss = criterion(outputs, targets)

    # GradScaler handles gradient scaling for numerical stability
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

Operation precision and weight storage are separate choices, so mixed precision also changes how many copies of each parameter training keeps. Under autocast the parameters stay in FP32 and are cast to 16 bits per operation, while recipes that store 16-bit working weights also keep an FP32 master copy for the update; either way the optimizer's FP32 moment buffers sit on top of gradients and saved activations. The per-parameter accounting, and the gap it opens between a model's weight file and its training footprint, is worked in section 5.4.5.5; budgeting and reducing optimizer memory is developed in section 8.3.2.5, and the training checkpoints that write this state to storage so a run can resume after a failure in section 8.4.5.

7.4.3.4 Tensor lifetimes and the caching allocator

Checkpointing and fusion (section 7.4.2) decide which values exist during a training step. The runtime still has to decide when each value's memory is reclaimed and how fresh memory is obtained. Where tensors live and how they move between host and device are the subject of section 7.5.2; the question here is narrower, namely when a value saved for the backward pass can be freed. In the retained-activation scheme of listing 7.8, forward layers add saved tensors that remain live until backward traversal consumes them. This saved-activation footprint peaks at the onset of backpropagation and recedes as intermediate buffers are released. Total training memory can peak later as gradients and temporary workspaces become live. Modern runtimes reclaim saved tensors once their backward consumers and other references no longer need them.

Allocation itself is also slow. Invoking device memory allocation and release (`cudaMalloc` and `cudaFree`) incurs substantial driver overhead, and `cudaFree` can block the host until all previously queued device work completes, draining accelerator command queues (Paszke et al. 2019). To eliminate this latency, modern frameworks intercept allocations with **caching allocators** that maintain pools of pre-reserved device memory blocks across training iterations. Suballocating from these pools amortizes device allocation costs and avoids the synchronizations that these calls can force, though varying tensor lifetimes can still induce memory fragmentation.

> Systems Perspective 7.1: Caching allocator and utilization
>
> Caching allocators primarily reduce allocation overhead and improve block reuse. Two considerations remain:
>
> 1. Allocation latency: Reusing a cached block avoids repeated device-allocation calls, whose cost and synchronization behavior depend on the allocator and runtime.

2. Fragmentation: Size classes and block splitting improve reuse but can leave free capacity in blocks that do not satisfy a request. Because each tensor requires a contiguous address range, external fragmentation occurs when variable tensor lifetimes leave small, non-contiguous free blocks in the pool; a large allocation can trigger an out-of-memory (OOM) error even when the total aggregate free memory across all blocks exceeds the requested size.

An out-of-memory error can reflect live allocations, allocator-reserved blocks, fragmentation, or another process. `nvidia-smi` reports device-level usage and is not sufficient by itself to diagnose the framework allocator; allocator-specific memory statistics are needed.

7.4.4 Tape-based and transform-based differentiation

A training iteration can run more slowly in production than in an isolated profile because the full step coordinates the allocator, the device manager, the scheduler, the data pipeline, and the optimizer, work that an operator microbenchmark does not measure. How heavy that coordination is depends on how the AD system records computation in the first place. A framework that builds a dynamic trace at runtime pays per-operation bookkeeping on every forward pass, and its caching allocator must accommodate allocation patterns that can vary with execution. A framework that captures a stable computation, by contrast, may pre-plan buffers and reuse storage across the forward and backward graphs because it can reason about lifetimes before execution. The gain depends on capture coverage, tensor lifetimes, shapes, and backend support.

The execution models of section 7.3 dictate these designs. PyTorch (Paszke et al. 2019) builds its tape during forward execution, which grants unconstrained Python control flow and direct debugging but retains tape metadata and saved tensors until backward traversal completes. Early TensorFlow (Martín Abadi et al. 2016) differentiated symbolically during graph construction, producing a fixed backward graph amenable to whole-graph dead-code elimination and memory reuse before execution; TensorFlow 2.x executes eagerly by default and compiles differentiation graphs through `tf.function` when throughput requires it (TensorFlow Developers 2024a). JAX[20] (Frostig et al. 2018) treats differentiation as a transformation that turns one function into its derivative function, which composes with vectorization (`jax.vmap()`) and compilation (`jax.jit()`) at the cost of requiring pure functions.

These implementation differences determine how much debugging visibility, compiler optimization, and deployment portability a team can expect from each framework. Each design also resolves a tension common to all of them. Backward rules need forward information, while device memory limits how much of it can be kept, so every framework must choose which activations to save, which operations to fuse, and how much recomputation to accept. Those choices decide which models can train on a given accelerator.

Checkpoint 7.2: The systems cost of gradients

Training is inherently more expensive than inference because of automatic differentiation.

Computational reality

- ☐ **Reverse mode AD**: why is it efficient for a scalar loss with many parameters?
- ☐ **Saved activations**: can you explain how saved-value memory depends on graph depth, tensor shapes, and checkpointing?

Optimization mechanics

- ☐ **Activation checkpointing**: how does recomputing activations save memory?

The execution and differentiation problems together enable the training loop: the execution model determines when computation happens, while automatic differentiation computes the gradients that drive learning. Both problems, however, assume that the same code can run across diverse hardware, and that assumption cannot be taken for granted. A model trained on an NVIDIA H100

[20] **JAX**: JAX exposes `grad`, `jit`, and `vmap` as transformations on functions; they can be composed when one transformation's outputs match what the next expects as inputs, but order matters: vectorizing gradients is not generally the same operation as differentiating a vector-valued result. Compilation can produce fused regions, library calls, and multiple kernels rather than one universal kernel. Transformed functions should be pure; Python side effects may occur during tracing but are not represented in the compiled program, while supported callbacks express intentional runtime effects.

must serve inference on a mobile phone's ARM CPU, a Google TPU, or a microcontroller with kilobytes of memory. The same `torch.matmul` call must dispatch to cuBLAS on one device and a hand-tuned ARM NEON kernel on another. This hardware diversity creates the third problem.

7.5 Abstraction: Tensors, Devices, and Operations

The preceding sections end at the level where values occupy memory and kernels run on a particular device. Modern machine learning applications demand portability across dissimilar execution targets. A model trained on data-center GPUs with tens of gigabytes of high-bandwidth memory each must deploy on an Apple neural engine, an edge TPU, or an ARM microcontroller with 256 kilobytes of SRAM. The **abstraction problem** requires frameworks to hide this underlying hardware diversity behind a uniform programming interface without sacrificing the specialized instruction sets, memory layouts, and compute pipelines of each target. GPUs expose abundant throughput-oriented parallelism but have different memory and execution semantics from CPUs. TPUs favor regular tensor programs and compiler-visible shapes. A microcontroller has kilobytes where a server has gigabytes. Frameworks must therefore balance unified developer ergonomics against target-specific performance tuning.

The problem decomposes into two interacting dimensions. The first is *data representation*: how frameworks encode tensors, parameters, and computational state in forms that work across hardware. The second is *execution mapping*: how high-level operations translate into hardware-specific implementations. These dimensions are not independent concerns. The way data is represented (memory layout, precision, device placement) directly affects *what* execution strategies are possible. A tensor stored in row-major format on a GPU requires different kernels than one in column-major format on a CPU. A model quantized to INT8 enables entirely different execution paths than FP32.

Solving the abstraction problem requires tensor representations that encode both mathematical semantics and hardware constraints, intermediate representations that enable hardware-specific compilation, and runtime systems that manage data movement across the memory hierarchy. To make this concrete, trace what must happen when a programmer writes `model(input)`. The framework must resolve four decisions in rapid succession, and the rest of this section takes them in order: data representation (tensor shape, memory layout, numeric precision), device placement and movement (the bandwidth hierarchy connecting host memory, accelerator memory, and other accelerators), input delivery (data pipelines that keep the accelerator fed), and kernel execution (dispatch, scheduling, and resource management). Organizing thousands of tensors back into one model follows in section 7.6.

7.5.1 Tensors

A ResNet-50 forward pass touches 25.6M parameters, produces intermediate activations at every layer, and must coordinate memory across CPU and GPU address spaces. Frameworks organize all of this data so that a single `model(input)` call executes millions of operations without the programmer managing a single pointer. Computational graphs specify the *logical* flow of those operations, while data structures determine how they access and manipulate data in *physical* memory, a distinction that can change throughput by an order of magnitude depending on whether data is contiguous in cache, pinned for direct memory access (DMA) transfer, or scattered across pages. To sustain terabytes per second of memory bandwidth across architectures from 1D sequences to 5D video while hiding device management behind a uniform API, frameworks build on a single data container: the tensor.

7.5.1.1 What a tensor carries

Every framework represents data with a single abstraction, the tensor, an n-dimensional array that pairs numerical values with the information needed to interpret and place them. Alongside the raw data pointer, the tensor header tracks shape, stride, element type, target device, and differentiation state so that operators can dispatch the right kernel without copying the underlying buffer.

Every computation in a neural network operates on tensors.[21] Training batches, activation maps, parameter gradients, and optimizer states are all tensors. This unified representation lets frameworks optimize a single data structure for hardware rather than managing separate containers for each role.

[21] **Tensor**: In mathematics, tensors obey coordinate-transformation laws; ML frameworks use the term more broadly for multidimensional arrays carrying shape, dtype, strides, device placement, and often automatic-differentiation metadata. A transpose commonly creates a strided view, while reshape may return either a view or a copy depending on the layout. These storage semantics make layout relevant to kernel selection and data movement.

Definition 7.2: Tensor

Tensors are n-dimensional arrays with shape, data type, stride, and device metadata. Framework runtimes use this information to select compatible operations and hardware kernels.

1. **Significance**: A contiguous FP32 tensor of shape $[1024, 1024]$ occupies $1024 \times 1024 \times 4$ = 4,194,304 bytes (about 4.2 MB). A noncontiguous view may be accepted directly by a strided kernel or may require materialization when an operation demands a supported contiguous layout.
2. **Distinction**: NumPy arrays also carry shape, dtype, and stride metadata. Framework tensors additionally integrate device placement, automatic differentiation, and accelerator dispatch.
3. **Common pitfall**: Tensor operations may return a new allocation, a view sharing existing storage, or an in-place update, depending on the operation and API variant. Memory accounting must distinguish these cases.

Because weights, gradients, master copies, optimizer moments, and saved activations are all tensors, they draw on the same device allocator and differ only in shape, dtype, device, and lifetime. That uniformity lets one caching allocator and one placement mechanism serve every role. It is also why a parameter count underestimates memory, since each weight tensor brings several companion tensors with it, as the training-versus-inference accounting of section 5.4.5.5 showed.

7.5.1.2 Rank, layout, and data type

Physical computer memory is a 1D array of linear byte addresses, whereas model inputs are 2D matrices, 3D RGB images, or 4D image batches. Tensors bridge this mismatch by pairing linear storage with coordinate axes whose count is the tensor's rank. Figure 7.9 traces this rank expansion from left to right, progressing from a Rank 0 scalar cell and Rank 1 column vector $(1, 3, 5, \ldots, 3)$ to a Rank 2 matrix grid and Rank 3 isometric volume tensor.

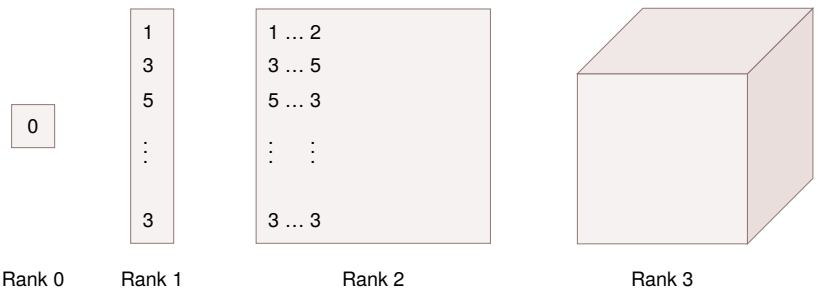


Figure 7.9: Tensor Rank Hierarchy: Ascending tensor ranks (0 to 3) generalize multi-dimensional indexing from scalars to volume tensors. Each additional rank adds an organizational axis that the framework must linearize into physical memory addresses via strides.

In vision applications, raw pixel arrays map directly onto multi-dimensional tensor layouts. For a 3×3 pixel patch (figure 7.10), planar grids for height (3 pixels vertically) and width (3 pixels horizontally) stack along depth into three color channels (red, green, and blue), forming a rank-3 coordinate space.

Framework tensors carry more than raw numbers. Each tensor stores **tensor metadata**, runtime information used to validate operations and select fast execution paths: a *shape* tuple (for example, `[64, 3, 224, 224]` for a batch of images), a *dtype* (framework literals such as `float32`, `float16`, or `int8`), and a *device* tag (CPU, cuda:0). A matrix multiplication, for instance, checks shape compatibility at dispatch time and uses the dtype to route to the correct hardware kernel, whether a standard FP32 GEMM or a Tensor Core FP16 path.

Memory layout implementation introduces distinct challenges in tensor design. While tensors provide an abstraction of multi-dimensional data, physical computer memory remains linear. Stride

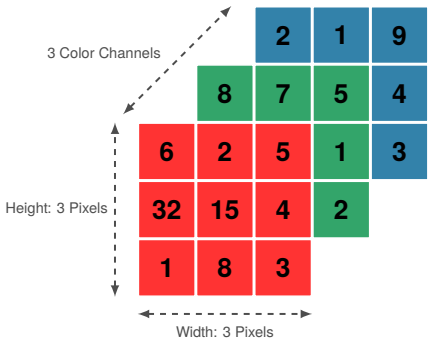


Figure 7.10: Image as an RGB Tensor: Decomposing a 2D color image into stacked height, width, and channel dimensions forms a rank-3 tensor. In ML systems, the ordering of these dimensions (such as channel-first vs. channel-last) determines memory access locality and cache efficiency during convolution operations.

patterns address this disparity by creating mappings between multi-dimensional tensor indices and linear memory addresses. Strides therefore fix the memory access pattern of every tensor operation and, with it, much of the operation's performance. Figure 7.11 contrasts how a 2×3 logical matrix maps into contiguous physical memory: row-major ordering (top right, pink) packs elements sequentially along rows ($[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]$) using strides $[3, 1]$, whereas column-major ordering (bottom right, green) serializes along columns ($[1, 4, 2, 5, 3, 6]$) using strides $[1, 2]$. In both schemes, the framework computes element offsets via $i \times \text{stride}[0] + j \times \text{stride}[1]$.

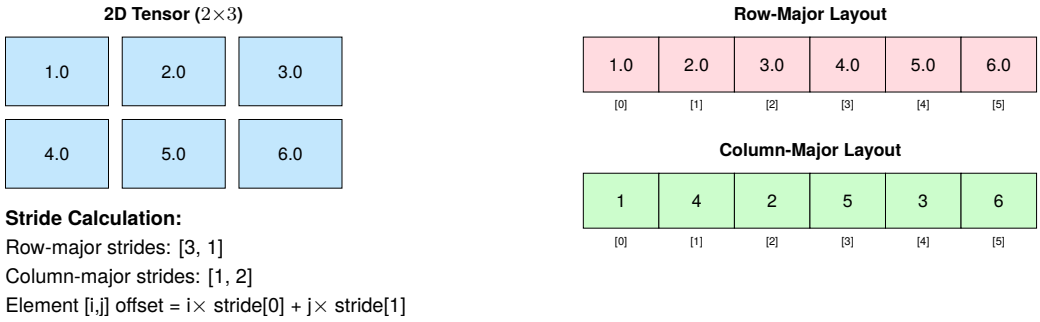


Figure 7.11: Tensor Memory Layout: A 2×3 tensor can be stored in linear memory using either row-major (C-style) or column-major (Fortran-style) ordering. Strides count elements skipped in each dimension, so i × stride[0] + j × stride[1] gives an element offset. Multiplying by the element size gives the byte offset from the storage origin. The choice of layout affects which traversals access neighboring elements efficiently.

Stride choices become performance choices. Row-major layout (used by NumPy, PyTorch) stores elements row by row, making row-wise operations more cache-friendly. Column-major layout (used by some BLAS libraries) stores elements column by column, optimizing column-wise access patterns. The stride values encode this layout information: in row-major layout for a 2×3 tensor, moving to the next row requires skipping three elements (stride[0] = 3), while moving to the next column requires skipping one element (stride[1] = 1).

These memory layout details have direct performance implications, but no ordering is universally superior. Efficient access depends on which dimension an operation traverses, the kernel's supported layouts, vectorization, and the target memory hierarchy. A mismatched layout may require strided access or a conversion, while a kernel designed for that layout can access it efficiently. PyTorch's `.transpose()` creates a view by changing stride metadata while sharing storage. A later reshape can reuse that storage only when the strides permit the requested view. Calling `.contiguous()` copies elements into a new buffer when the requested contiguous layout is not already present.

The dtype is the tensor-level lever that trades numerical range against data movement. The standard choice in machine learning has been FP32 precision, exposed in frameworks through dtype literals such as `float32`, offering a balance of precision and efficiency. Modern frameworks

extend this with specialized numeric types: integers for indexing and embedding lookups, reduced-precision floating point such as FP16 for memory-efficient training and deployment, and narrow integers such as INT8 for high-throughput inference on specialized hardware. The choice of numeric type affects both model behavior and computational efficiency. Neural network training typically requires FP32 precision for critical accumulations to maintain stable gradient computations, while inference tasks can often use lower precision (framework dtype literals such as `int8` or even `int4`, corresponding to INT8 and INT4), reducing memory usage and increasing processing speed. The autocast policy of section 7.4.3.3 combines these benefits per operation, running most computations at 16-bit precision while keeping accumulations and numerically sensitive operations in FP32. When operations mix tensors of different types, explicit casting rules must preserve numerical correctness without hiding the memory traffic and precision loss that each conversion introduces.

Tensors answer the data-representation problem by encoding shape, layout, and precision into a single abstraction. A perfectly shaped tensor on the wrong device, however, or one that must cross the PCIe-to-HBM bandwidth gap to reach the GPU, can erase every layout optimization. The next problem is placement: where data lives and how it moves.

7.5.2 Devices and memory

On an H100, the PCIe Gen5 x16 host link delivers 64 GB/s between CPU and GPU while on-package HBM3 provides 3.35 TB/s inside the GPU, a 52.3× bandwidth gap that lets a single misplaced tensor copy erase the entire speedup of GPU acceleration. Where tensor data physically resides, and how it moves between host and accelerator memory, therefore determines whether computation runs at HBM speed or stalls on the host interconnect.

7.5.2.1 Frameworks as the operating system interface

While the high-level API focuses on math, the framework backend plays the role that an operating system plays for a single machine. It schedules compute and moves data, building on the CUDA runtime's primitives for launching kernels and managing device memory.

On the data-movement side, the framework coordinates with this runtime to implement **DMA** over the PCIe bus. The bandwidth gap between the host (CPU) and device (GPU) is the first bottleneck a data-loading pipeline meets, and section D.3.2 models this transfer, separating the two constraints, $T = L_{\text{lat}} + D_{\text{vol}}/\text{BW}$, that determine whether a given data-loading strategy is limited by per-transfer latency or by sustained bandwidth. Frameworks mitigate this through pinned memory (page-locked host memory), which enables DMA transfers without pageable-memory staging. This host-device interface is what makes high-throughput training loops possible on a single machine.

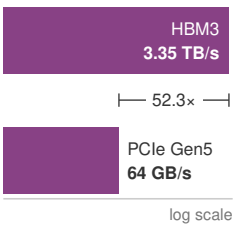
Because cross-device operations cross that 52.3× gap, the framework tracks where every tensor lives and requires that operations combine only tensors on the same device. When data must move, the framework must decide whether to block execution or overlap the transfer with other work. These decisions, invisible to most users, can move a training loop from transfer-bound execution toward substantially higher hardware utilization.

Three principles govern device and memory management: respect the bandwidth hierarchy, overlap unavoidable transfers with computation, and synchronize only where correctness requires it. The bandwidth hierarchy comes first, because the cost of moving data between devices varies by orders of magnitude with the interconnect.[22] For a 1000×1000 float32 tensor (4 MB), roughly the size of an activation tensor in a moderately sized model, the transfer times in table 7.7 separate the on-device path from every host link by more than an order of magnitude.

Table 7.7: Device Transfer Overhead: A 4 MB tensor takes far longer to cross a host link than to move within device memory. PCIe bandwidth shown is unidirectional (typical for GPU transfers), with full-duplex operation providing 2× total bandwidth; the NVLink row lists both its per-direction and bidirectional rates. These transfer costs can dominate lightweight operations, making device placement critical.

Interconnect	Bandwidth	Transfer Time	Path
PCIe Gen5 x16	64 GB/s	62.5 µs	Host to device
NVLink 4.0	450 GB/s per direction (900 GB/s bidirectional)	8.9 µs	GPU to GPU

↩ The single-node stack, in which the operating system moves data between host memory and accelerator memory, is established in section 2.4.1.



On the H100, on-device HBM3 outruns the host PCIe link by 52.3×.

[22] **NVLink**: NVIDIA's high-bandwidth GPU-to-GPU interconnect, providing 900 GB/s bidirectional bandwidth (NVLink 4.0 on the H100) compared to 128 GB/s for its PCIe Gen5 x16 host link. This 7.0× bandwidth advantage determines whether tensor parallelism, splitting one large tensor computation across multiple GPUs, is practical for a given model size: GPUs connected only by PCIe can make the D_{vol}/BW communication term dominate total training time, erasing the benefit of additional compute.

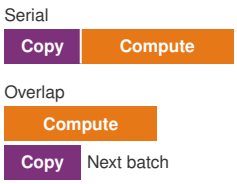
Interconnect	Bandwidth	Transfer Time	Path
HBM3	3,350 GB/s	1.2 µs	On device

These numbers connect directly to the iron law of ML systems. Every cross-device transfer contributes to $(D_{\text{vol}}/\text{BW})$ at a fraction of on-device bandwidth. Dividing 1 GB by the stated PCIe Gen5 x16 peak gives 15.6 ms, a bandwidth-only lower bound that omits protocol, launch, synchronization, topology, and contention overhead. Transfers can dominate lightweight or small-batch workloads, so deployed latency must be measured rather than inferred from peak bandwidth alone.

Device placement discipline protects the bandwidth hierarchy from accidental PCIe traffic. Every tensor carries a device attribute, and frameworks enforce a strict invariant: operations can only combine tensors on the same device, raising a `RuntimeError` when `cuda:0` and `cuda:1` tensors mix so that cross-device transfers remain explicit. Calling `.to()` on a tensor returns the original tensor when its data type and device already match the request (unless `copy=True` forces a copy) and otherwise returns a converted copy; in contrast, `nn.Module.to()` modifies registered parameters and buffers in place and returns the module.

The performance discipline follows directly from this bandwidth gap: allocate tensors on the target device from the start rather than creating them on the host and transferring them, reuse GPU memory across iterations rather than reallocating it, and colocate inputs, labels, and model parameters on the same device to eliminate implicit transfers. At 64 GB/s, violating any of these principles inserts PCIe transfers into the critical path that can dominate a training iteration that otherwise runs at 3.35 TB/s on-device.

When transfers are unavoidable, the next optimization is to hide their latency by executing them concurrently with computation. GPUs contain separate hardware units for computation (streaming multiprocessors) and data transfer (copy engines), and the framework exposes them through the CUDA stream, an execution queue whose operations run in order within the stream but concurrently with other streams. When copies and computation share one stream, they execute serially. Placing the next batch's transfer on one stream while the current batch computes on another can make the steady-state batch interval approach $\max(T_{\text{compute}}, T_{\text{transfer}})$, the larger stage time rather than their sum. Each batch still requires its own transfer to finish before its consumer runs. With sufficient independent work and hardware resources, stream-based overlap can hide much of the D_{vol}/BW penalty when computation is the longer stage (see listing 7.13).



Overlap pays the longer stage, not the sum.

The `non_blocking=True` flag requests a copy that is asynchronous with respect to the host, but real overlap also requires pinned memory. Ordinary host memory is pageable: the operating system may relocate or swap its pages at any time, and a DMA engine, which copies across PCIe without involving CPU cores, cannot handle a page fault. A copy from pageable memory therefore first stages the data synchronously into a temporary pinned buffer. Tensors placed in pinned (page-locked) host memory skip that staging, so their transfers can overlap accelerator computation. In listing 7.13 the two streams are separate; a complete pipeline must also hand each transferred batch to its consumer only after the copy completes, using the synchronization mechanisms that follow.

Listing 7.13: Overlapping Computation and Transfer: Separate streams can overlap eligible copies and computation when dependencies are synchronized correctly. Pinned host memory generally enables host-to-device copies to overlap with other device work.

```
compute_stream = torch.cuda.Stream()
transfer_stream = torch.cuda.Stream()

# Transfer next batch while computing
# current batch
with torch.cuda.stream(transfer_stream):
    next_batch = next_batch_cpu.to("cuda", non_blocking=True)

with torch.cuda.stream(compute_stream):
    output = model(current_batch)
    loss = criterion(output, labels)

# Pinned host memory can enable transfer overlap
x_pinned = torch.randn(1000, 1000).pin_memory()
x_gpu = x_pinned.to("cuda", non_blocking=True)  # Asynchronous

# A pageable source may require staging and
# should not be assumed to overlap
y_regular = torch.randn(1000, 1000)
y_gpu = y_regular.to("cuda", non_blocking=True)
```

Concurrent execution introduces ordering constraints: when one stream's output becomes another stream's input, the system must enforce a happens-before relationship without unnecessarily serializing independent work. Two synchronization mechanisms exist, and their performance consequences differ sharply. Full device synchronization (`torch.cuda.synchronize()`) blocks all streams and the CPU until every queued operation completes, creating a global serialization point that eliminates overlap. CUDA events provide the alternative: fine-grained synchronization that blocks only the dependent stream, allowing other streams and the CPU to continue execution (see listing 7.14).

Listing 7.14: CUDA Events for Synchronization: Events enable fine-grained producer-consumer patterns between streams without blocking the entire device.

```
# Create streams and event
stream1 = torch.cuda.Stream()
stream2 = torch.cuda.Stream()
event = torch.cuda.Event()

# Stream 1: producer
with torch.cuda.stream(stream1):
    result1 = expensive_computation(data1)
    event.record()  # Mark completion point

# Stream 2: consumer (waits only for stream1's event)
with torch.cuda.stream(stream2):
    event.wait()  # Block stream2 until event is recorded
    result2 = dependent_computation(result1)  # Safe to use result1
```

Full synchronization after every operation converts a concurrent pipeline into a sequential one, negating the hardware parallelism that streams expose, whereas event-based synchronization enforces only the dependencies that correctness requires. That difference creates a common operational trap: debug code left with `torch.cuda.synchronize()` in the hot path turns an overlapped pipeline into a serialized one. A timeline profiler shows the resulting gaps between kernels, while the kernel profilers and hardware counters that explain why an individual kernel underperforms belong to the profiling method of section 8.5.

7.5.2.2 Placement across accelerators

The same placement machinery extends to the one to eight accelerators of a single machine. Each device can hold a complete replica of the model and process a different slice of the batch, which requires summing every replica's gradients after the backward pass (an AllReduce), or the devices can hold different groups of layers and pass activations between them. Frameworks express these arrangements through named device groups, such as `ProcessGroup` in PyTorch or `Mesh` in JAX, and preserve the model's semantics while the hardware path underneath changes: NVLink delivers 900 GB/s of bidirectional bandwidth per H100 inside one node (table 7.7), while devices joined only by PCIe pay the slower path for every exchange. Parameter state multiplies with each arrangement, because every weight brings a gradient, optimizer moments, and often a second copy at another precision, and each replica or partition must own its share. Which arrangement to choose, and what its communication costs, is developed in section 8.7.

↪ The AllReduce collective and its communication cost are developed in section 8.7.1.1.

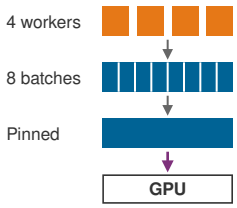
At the framework level, the central requirement is placement expressiveness on one machine with one to eight accelerators. The first arrangement is data parallelism; the second is pipeline parallelism. A third arrangement, tensor parallelism, splits individual tensor operations across devices. These are alternative placement choices, not a requirement to combine every strategy. The framework must express device ownership, transfers, and synchronization while preserving the model's semantics.

7.5.3 Data loading

Streams and events hide transfers behind computation, but scheduling cannot help if input data arrives too slowly in the first place. The data pipeline must sustain the accelerator's consumption rate. An accelerator processing 1,000 images per second at 224 by 224 resolution needs about 150.5 MB/s of sustained raw uint8 image throughput, and any shortfall idles the accelerator and lowers the effective utilization term in the iron law. The framework's contribution to this feeding problem is an abstraction that separates what a sample is from how samples reach the device. A *Dataset* defines access to individual samples, and a *DataLoader* (Paszke et al. 2019) turns that definition into a pipeline built from parallel worker processes, prefetching, and pinned staging memory.

↩ The feeding problem, in which data delivery must keep pace with accelerator consumption, is established in section 4.9.3.

Worker processes come first. With `num_workers > 0`, the DataLoader spawns CPU processes that load and preprocess samples independently, distributes sample indices to them through a shared queue, and assembles their completed samples into batches in the main process. Multiple workers overlap I/O wait with decoding and augmentation when one process is too slow to feed the accelerator; once the pipeline meets demand, extra workers add overhead rather than throughput.



Four workers fill eight prefetched batches, which drain through pinned memory to the GPU.

Prefetching sets pipeline depth. The `prefetch_factor` parameter controls how many batches each worker prepares in advance, so four workers with `prefetch_factor=2` keep up to eight batches in flight, at a cost in host memory proportional to the queued data. Prefetching raises the chance that input work overlaps accelerator computation but cannot prevent stalls when storage, decoding, or contention makes production slower than consumption.

Pinned memory completes the path to the device. The `pin_memory=True` option places batches in the page-locked host memory of section 7.5.2, so their host-to-device copies skip pageable-memory staging and can overlap device work. For a batch of 64 FP32 images at $224\times224\times3$ (38.5 MB), dividing by the H100's PCIe Gen5 x16 peak gives 0.6 ms; this is a bandwidth-only lower bound, not a measured pinned-transfer latency. The realized benefit over pageable memory depends on the platform, batch size, pipeline, and overlap. Pinned pages also reduce pageable system memory.

The DataLoader configuration in listing 7.15 is useful only when each parameter is tied to a bottleneck: `num_workers` sets loading parallelism, `prefetch_factor` sets pipeline depth, and `pin_memory` enables DMA transfers. Each worker also holds its own copy of the Dataset object and its own Python state, so the worker count trades throughput against host memory, and changes one worker makes to that state do not reach the others or the main process unless the state lives in explicitly shared or memory-mapped storage. PyTorch seeds each worker's own generator separately, but augmentation code that draws from NumPy or Python's `random` module still needs a `worker_init_fn` such as `seed_worker`.

Listing 7.15: DataLoader Throughput Configuration: Four workers with a prefetch depth of two keep up to eight batches in flight through pinned memory, and a per-worker seed function makes NumPy and Python augmentation reproducible.

```
import random

import numpy as np
import torch
from torch.utils.data import DataLoader


def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % 2**32
    np.random.seed(worker_seed)
    random.seed(worker_seed)


loader = DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
    num_workers=4,  # Parallel worker processes (mechanism 1)
    prefetch_factor=2,  # Batches prepared ahead per worker (mechanism 2)
    pin_memory=True,  # Page-locked memory for DMA (mechanism 3)
    worker_init_fn=seed_worker,  # Reproducible augmentation per worker
)

# Pipeline effect: while GPU processes batch N,
# 4 workers load batches N+1..N+8 into pinned memory,
# ready for DMA transfer when the GPU finishes.
```

The Dataset interface decides how samples can be scheduled. A *map-style* dataset implements `__len__` and `__getitem__`, so a sampler can shuffle by index, which suits data that fits in memory or supports efficient random access on disk. An *iterable-style* dataset implements `__iter__` and yields samples sequentially from a stream where random access is impractical, so any shuffling must be buffered inside the iterator. Batch collation through the `collate_fn` parameter then combines samples into a batch. The default stacks tensors along a new batch dimension, which requires identical shapes; variable-length data such as text needs custom collation that pads, sorts by length, or builds attention masks, and those choices change both memory use and throughput.

↪ Sizing worker count and prefetch depth against measured throughput and memory is developed in section 8.6.2.2.

With input delivered at the rate the accelerator consumes it, the remaining abstraction problem is execution, the translation of each operation into work that runs on the target hardware.

7.5.4 Core operations

Calling `y = torch.matmul(x, w)` without selecting a kernel matched to `x` and `w`'s device, data type, stride layout, and tile dimensions either fails at runtime or leaves Tensor Cores idle while a generic loop crawls through memory. Bridging that single Python call to thousands of parallel accelerator threads traverses three coordinated functional tiers. From left to right in figure 7.12, system-level operations (scheduling, memory management, and resource optimization) organize graph execution; basic numerical operations (GEMM, BLAS, and element-wise kernels) execute algebraic routines; and hardware operations (compute kernel management, memory abstraction, and execution control) bind the workload directly to device streams and physical silicon. The discussion that follows traces this hierarchy from the hardware kernel primitives upward to system orchestration.

7.5.4.1 Hardware abstraction operations

The hardware abstraction layer isolates framework code from platform-specific details. It solves three concrete problems: selecting the right compute kernel, moving data through the memory hierarchy, and coordinating execution across processing units.

The kernel manager dispatches each supported operation to an implementation for the current backend. For matrix multiplication, this may be a CPU BLAS backend such as Intel oneMKL or OpenBLAS (Intel Corporation 2026a; OpenBLAS Project 2026), cuBLAS on NVIDIA GPUs (NVIDIA

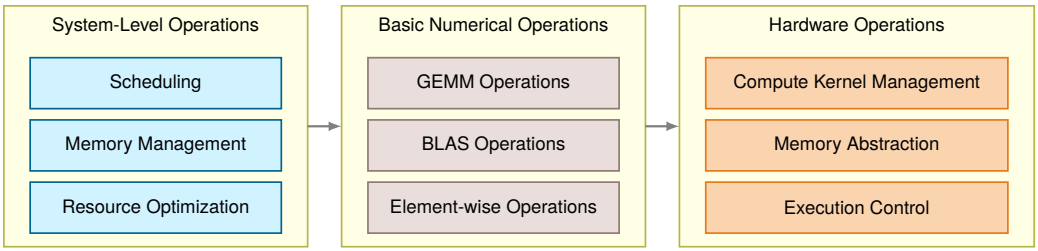


Figure 7.12: Core Operations Stack: Three functional tiers bridge high-level Python code to physical execution. Hardware operations manage low-level kernel dispatch and memory transfers, basic numerical operations execute dense GEMM and element-wise routines, and system-level operations coordinate graph scheduling and memory lifetimes across accelerators.

2024a), or accelerator-specific tensor instructions. Selection depends on dimensions, dtype, layout, backend, and library heuristics. A 4096×4096 FP16 GEMM on an H100 may use a Tensor Core path whose published dense peak is 989 TFLOP/s (Choquette 2023; NVIDIA Corporation 2022). If the backend has no implementation for an operator, execution may fall back, be partitioned, or fail, depending on the framework and runtime.

The memory abstraction layer moves tensors among pageable or pinned host memory, device memory, and unified memory, and transforms data layouts to match hardware preferences. In physical silicon, tensors do not exist as multi-dimensional matrices; they are stored as one-dimensional arrays of linear byte addresses. Layout formats dictate how dimensions map to that 1D sequence: *channels-first* NCHW places all spatial pixels of one channel contiguously before storing the next channel, whereas *channels-last* NHWC groups all channel values for a single spatial coordinate consecutively. Accelerator matrix units often prefer NHWC because loading all channels for a pixel aligns with a single, contiguous 128-byte memory transaction (a coalesced access). A framework may therefore execute kernels in NHWC for peak hardware utilization while preserving NCHW for user-facing APIs. Alignment requirements vary from 4 bytes on CPUs to 128 bytes on high-bandwidth accelerators; misaligned or strided accesses split requests across multiple memory transactions and can cut effective bandwidth substantially. The runtime also preserves dependency ordering when multiple execution units access the same tensor, preventing races during concurrent operations.

The execution controller coordinates work across multiple processing units and memory spaces. On a modern GPU, graph runtimes or explicit stream use can overlap independent kernels when dependency analysis proves they are ready and the kernels leave enough resources unused to run concurrently. Eager default-stream execution often serializes this work instead. The controller inserts synchronization barriers where data dependencies require them, tracks event completions to trigger dependent operations, and routes hardware errors (ECC failures, timeout watchdogs) to the framework's error handling path.

7.5.4.2 Basic numerical operations

With hardware abstraction managing the platform-specific details, frameworks build a layer of mathematical operations on top. GEMM dominates ML computation. The operation $\mathbf{C} = \alpha\mathbf{AW} + \beta\mathbf{C}$ accounts for the vast majority of arithmetic in neural networks: a single ResNet-50 forward pass performs approximately 8.2 GFLOP, nearly all of which reduce to GEMM. Section C.1.3 derives how GEMM arithmetic intensity scales with matrix dimension, predicting whether a given layer is compute bound or memory bound before any profiler runs. Frameworks optimize GEMM through cache-aware tiling (splitting matrices into blocks that fit in L1/L2 cache), loop unrolling for instruction-level parallelism, and shape-specific kernels. Fully connected layers use standard dense GEMM, while convolutional layers use im2col transformations that reshape input patches into matrix columns, converting convolution into GEMM.

↪ Tiling on an accelerator, where on-chip storage and the compute array fix the tile size and a dimension one element past a tile boundary can halve utilization, is developed in section 11.7.5.

Beyond GEMM, frameworks implement BLAS operations (AXPY for vector addition, GEMV for matrix-vector products) and element-wise operations (activation functions, normalization). Element-wise operations are individually cheap but collectively expensive, because each one reads and writes its full tensor; they are the fusion targets of section 7.3.2.

Training in FP32 uses 4 bytes per parameter; quantizing to INT8 reduces this to 1 byte, cutting memory by 4× and enabling 2–4× throughput improvements on hardware with INT8 acceleration.

Training typically keeps numerically sensitive accumulations in higher precision, while inference can often run many operations in FP16 or INT8 with little quality loss. Frameworks maintain separate kernel implementations for each precision format and handle workflows where different layers operate at different bit widths within a single forward pass.

7.5.4.3 System-level operations

Hardware abstraction and numerical operations provide the building blocks; system-level operations orchestrate them. The scheduler orders operations by their graph dependencies and overlaps independent work through the streams and events of section 7.5.2. The memory manager reuses freed blocks through the caching allocator of section 7.4.3.4 and, when a model exceeds device memory, executes the activation-checkpointing policy of section 7.4.2. The resource optimizer ties these decisions to the kernel selection described earlier. Its failures are the ones those mechanisms predict, since a poor schedule leaves resources idle and fragmentation or excessive live state can cause an out-of-memory error even when aggregate device capacity appears sufficient.

Checkpoint 7.3: Hardware abstraction

The abstraction problem is the bridge between *portable code* and *efficient execution*.

- ☐ **Two dimensions**: can you distinguish data representation (layout, dtype, placement) from execution mapping (kernel selection, scheduling), and explain how they constrain each other?
- ☐ **Kernel dispatch**: can you explain why the same high-level operation (for example, GEMM) needs multiple implementations (CPU vector path vs. GPU Tensor Core path) and how shapes/dtypes affect the choice?
- ☐ **Memory abstraction**: can you explain why frameworks use caching allocators and layout transforms (NCHW↔NHWC) rather than calling device allocators on every tensor?
- ☐ **Execution control**: can you describe what the runtime is doing when it overlaps independent work (streams) and inserts synchronization only where dependencies require it?

Tensors, streams, and kernel dispatch complete the descent to the hardware, but they operate at the granularity of individual buffers and operations. Model development needs the opposite direction, composing thousands of low-level operations, together with the parameters that automatic differentiation tracks, into reusable, nestable building blocks called *modules*.

7.6 Modules: From Operations to Models

A network such as ResNet-50, containing 25.6M parameters across fifty layers, requires extensive coordination: registering weights with optimizers, tracking device placement, toggling training and evaluation modes, and serializing checkpoints. Managing this manually would drown model development in error-prone tensor plumbing. The organizational layer of the framework resolves this through hierarchical composition.

Every major framework implements this through a module abstraction that binds parameter tensors, forward computation, and lifecycle hooks into a unified interface. PyTorch's `nn.Module`[23] provides an instructive case study because its design patterns recur across frameworks: Keras uses similar layer abstractions (Chollet 2018), JAX's Flax employs analogous module structures, and TensorFlow's functional API shares conceptual parallels. Three enduring design principles recur regardless of syntax or programming paradigm.

7.6.1 Automatic parameter discovery

A modern neural network may contain millions of trainable parameters spread across dozens of layers. Without automation, a programmer would need to enumerate every parameter tensor and pass it to the optimizer manually, an error-prone process that scales poorly with model complexity. Frameworks solve this through **automatic parameter discovery**: the system walks the module tree, collecting every parameter tensor so the optimizer can update them in a single call.

[23] **nn.Module**: The pattern recurs across frameworks because the organizational problem is universal. `nn.Module` automatically registers any assigned submodule or parameter into a hierarchical tree, enabling a single `.to('cuda')` call to recursively place millions of parameters onto a GPU. Keras layers, JAX Flax modules, and TensorFlow's `tf.Module` all implement the same tree-walking pattern. Without it, managing model state would require manual bookkeeping that scales linearly with architectural depth, a cost that grows prohibitive for models with hundreds of layers.

This is a tree traversal problem at its core. PyTorch's `Module.__setattr__` registers an assigned `nn.Parameter` or child `nn.Module` in the module's internal mappings. A call to `.parameters()` recursively traverses registered submodules and yields registered parameters. Other frameworks expose analogous parameter trees or collections, though their mechanisms differ.

Automatic parameter discovery gives the optimizer a complete parameter set or parameter groups, enabling grouped or fused multi-tensor update kernels when the framework and backend support them. The immediate win is correctness and composability: newly attached submodules participate in device moves, serialization, and optimizer construction without a separate manual registry. It also lets optimized update paths operate over parameter collections without per-parameter user code. Listing 7.16 demonstrates the core mechanism: attribute assignment triggers registration, and `.parameters()` returns all discovered tensors.

The distinction between *parameters* and *buffers* illustrates a subtlety of discovery. Registered parameters appear in `.parameters()` whether trainable or frozen; `requires_grad` separately controls whether autograd accumulates a gradient for them. Registered buffers move with the module and can appear in its state dictionary but are excluded from `.parameters()` and ordinary optimizer discovery. Batch-normalization running statistics are a common buffer use case.

Listing 7.16: Parameter Registration: Automatic parameter tracking through attribute assignment enables optimizer access to all trainable weights without manual enumeration.

```
import torch
import torch.nn as nn


class CustomLayer(nn.Module):
    def __init__(self, input_size, output_size):
        super().__init__()
        self.weight = nn.Parameter(
            torch.randn(output_size, input_size)
        )
        self.bias = nn.Parameter(torch.randn(output_size))
        self.register_buffer("running_mean", torch.zeros(output_size))

    def forward(self, x):
        return torch.matmul(x, self.weight.t()) + self.bias


layer = CustomLayer(10, 20)
# Framework discovers both parameters automatically:
for name, param in layer.named_parameters():
    print(f"{name}: shape {param.shape}")
```

The other major frameworks draw the same line between trainable parameters and nontrainable state through different accessors (table 7.8). Regardless of syntax, each API separates tensors that the optimizer should update from persistent buffers that merely ride along in checkpoints and device transfers.

Table 7.8: Parameter Discovery APIs Across Frameworks: Each framework exposes a different surface for distinguishing trainable parameters from nontrainable buffers, but all four solve the same underlying problem of letting optimizers find weights to update while preserving state (running statistics, frozen tensors) across forward passes. The shared pattern is structured access to parameters and persistent state. Accessor semantics differ: PyTorch also yields frozen registered parameters, so trainability remains a separate property.

Framework	Parameter Access	Nontrainable State
PyTorch	`model.parameters()`	`register_buffer()`
Keras	`layer.trainable_weights`	`layer.non_trainable_weights`
JAX/Flax	`variables["params"]` after `variables = model.init(key, x)`	Separate variable collections (for example, `batch_stats`)

Framework	Parameter Access	Nontrainable State
TensorFlow	`module.trainable_variables`	`module.non_trainable_variables`

7.6.2 Mode-dependent behavior

Training and inference require different computational behavior from the same model graph. During training, dropout layers randomly zero elements with probability $p_{\text{drop}} = \Pr(\text{drop})$ to regularize the network, while during inference those same layers must perform identity mapping to produce deterministic outputs. Batch normalization uses per-batch statistics during training but switches to accumulated running statistics during inference. If these behavioral changes are left to the programmer, forgetting a single mode switch produces silently incorrect predictions in production.

Frameworks solve this with a *state flag* that propagates through the module hierarchy. A single call to `.eval()` on the root module recursively sets `self.training = False` on every descendant, and each layer queries this flag to select its behavior. This is an instance of a broader systems principle: the same computation graph must produce different execution behavior depending on context. Compilers face the same challenge when the same source code must produce debug builds (with bounds checking and symbol tables) vs. release builds (with aggressive optimization). The flag-propagation pattern ensures correctness by centralizing the mode decision at the root rather than requiring per-layer coordination.

Evaluation mode and gradient recording are separate controls. Calling `.eval()` changes mode-dependent layer behavior but leaves autograd recording enabled. For inference that needs no gradients, `torch.no_grad()` disables that recording, avoiding the associated saved backward state while `.eval()` selects the appropriate layer behavior.

This principle extends to parameter freezing for transfer learning. Setting `requires_grad=False` prevents autograd from accumulating gradients in those parameters and removes their gradient storage. It does not necessarily remove the layer's backward computation because gradients may still need to pass through its operations to reach trainable inputs or earlier parameters. Savings therefore depend on where the frozen region lies and which tensors require gradients.

7.6.3 Hierarchical composition and serialization

Complex models compose from reusable submodules, creating a tree structure. A ResNet is not implemented as a monolithic block of operations but as a hierarchy: the root module contains a sequence of residual blocks, each block contains convolution layers and normalization layers, and each layer contains parameter tensors. This hierarchical composition must support two operations, made concrete in listing 7.17: recursive parameter collection for training and **state serialization** for checkpointing and deployment.

Listing 7.17: Nested Module Composition: Hierarchical module composition enables recursive parameter collection and a flat state mapping across the module tree.

```
import torch
import torch.nn as nn


class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        residual = x
        x = torch.relu(self.bn1(self.conv1(x)))
        x = self.bn2(self.conv2(x))
        return torch.relu(x + residual)


class ResNet(nn.Module):
    def __init__(self, num_blocks, channels=64):
        super().__init__()
        self.conv_in = nn.Conv2d(3, channels, 7, padding=3)
        self.blocks = nn.ModuleList(
            [ResidualBlock(channels) for _ in range(num_blocks)]
        )
        self.fc = nn.Linear(channels, 10)

    def forward(self, x):
        x = self.conv_in(x)
        for block in self.blocks:
            x = block(x)
        x = x.mean(dim=[2, 3])  # Global average pooling
        return self.fc(x)


model = ResNet(num_blocks=4)
total = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total}")
# state_dict() flattens the tree: 'blocks.0.conv1.weight', etc.
print(list(model.state_dict().keys())[:4])
```

Hierarchical composition mirrors the hardware memory hierarchy in a systems-relevant way: each submodule's parameters can be loaded independently, enabling placement across devices. When a model is too large for a single GPU, the framework can assign different subtrees of the module hierarchy to different devices, with the tree structure providing natural partition boundaries.

The `state_dict()` method produces an ordered mapping whose dotted keys (for example, `blocks.0.conv1.weight`) encode locations in the module hierarchy. A checkpoint for 7 billion FP16 parameters contains approximately 14 GB of weight payload before metadata or additional state, but the file format and storage path determine how those bytes are serialized. Resuming training needs more than the model's state: optimizers expose their moment buffers and step counts through their own `state_dict()`, and a training checkpoint saves both so that the run continues with the same optimizer state. `load_state_dict()` copies matching parameter and buffer values into an already-constructed module and reports missing or unexpected keys according to its strictness setting; it does not reconstruct the module hierarchy. Cross-framework exchange requires a compatible graph and operator representation in addition to weights.

The hierarchical structure also enables module-level traversal for systematic operations. Methods such as `.named_modules()` iterate the entire tree, supporting bulk transformations such as replacing all BatchNorm layers with GroupNorm or applying Xavier initialization to every Linear layer. These

traversal operations depend on the same tree structure that enables parameter discovery, illustrating how a single design decision propagates benefits across multiple use cases.

None of these three principles is specific to PyTorch. Keras layers, Flax modules, and functional approaches address the same problems of parameter management, state tracking, and composition, and differ in how they prioritize among competing solutions rather than in which problems they solve. Two practical patterns show how the principles become system controls: selective parameter freezing reduces unnecessary gradient work for transfer learning (listing 7.18), and module hooks provide noninvasive inspection (listing 7.19).

Listing 7.18: Parameter Freezing: Setting `requires_grad=False` on a pretrained backbone keeps it fixed while a newly attached head trains, and the optimizer receives only the trainable parameters.

```
import torch
import torch.nn as nn
from torchvision.models import ResNet18_Weights, resnet18

# Freeze all parameters in a pretrained
# model
pretrained_model = resnet18(weights=ResNet18_Weights.DEFAULT)

for param in pretrained_model.parameters():
    param.requires_grad = False

# Replace final layer with trainable parameters
pretrained_model.fc = nn.Linear(512, 10)  # New layer is trainable

# Only fc.parameters() will receive
# gradients during training
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad, pretrained_model.parameters()),
    lr=0.001,
)
```

Module hooks are the inspection counterpart to parameter freezing. They intercept intermediate computations without modifying model code, enabling gradient flow diagnosis and activation monitoring (listing 7.19).

Listing 7.19: Module Hooks: Forward hooks capture activations and backward hooks capture gradients during training without modifying the model's code.

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 5))


# Forward hook to inspect activations
def forward_hook(module, input, output):
    print(
        f"Layer: {module.__class__.__name__}, "
        f"Output shape: {output.shape}, "
        f"mean={output.mean():.3f}, "
        f"std={output.std():.3f}"
    )


# Backward hook to inspect gradients
def backward_hook(module, grad_input, grad_output):
    print(f"Gradient norm: {grad_output[0].norm():.3f}")


# Register hooks on specific layer
handle_fwd = model[0].register_forward_hook(forward_hook)
handle_bwd = model[0].register_full_backward_hook(backward_hook)

# Execute forward and backward pass
x = torch.randn(32, 10)
y = model(x)
loss = y.sum()
loss.backward()

# Remove hooks when done
handle_fwd.remove()
handle_bwd.remove()
```

Parameter discovery, freezing, and hooks are how the module abstraction turns thousands of tensors, each with its own shape, dtype, device, and lifetime, into one object that an optimizer can update, a serializer can save, and a profiler can inspect. With execution, differentiation, and abstraction each examined on its own, the remaining step is to watch them operate together: a single training step exercises every mechanism of this chapter at once.

7.7 Anatomy of a Training Step

In every iteration of a training loop, millions of floating-point operations, dynamic memory allocations, and hardware kernel launches must complete within a step that lasts anywhere from microseconds to seconds; for the small network below, as the analysis shows, host dispatch takes most of that time. Execution, differentiation, and hardware abstraction operate simultaneously during model training. In listing 7.20, a minimal two-layer multilayer perceptron illustrates this convergence across eight executable statements. Tracing each statement through the runtime stack connects the quantitative principles developed in section 7.3.7 to physical execution.

The network is a two-layer version of the MNIST digit classifier from chapter 5, with 784 inputs, 128 hidden units, and 10 outputs, trained at batch size 32. The example assumes trainable CUDA tensors `W1` and `W2` with shapes `(784, 128)` and `(128, 10)`, matching bias vectors `b1` and `b2`, and an optimizer already constructed over these parameters. The listing shows one iteration after that setup.

Listing 7.20: Training Step Anatomy: One minimal training iteration of a two-layer MLP exercises tensor allocation, kernel dispatch, autograd recording, gradient computation, and parameter updates.

```
# Single training step for a 2-layer MLP
x = torch.randn(32, 784, device="cuda")  # Input batch
y = torch.randint(0, 10, (32,), device="cuda")  # Labels

# Forward pass
optimizer.zero_grad()
h = torch.relu(x @ W1 + b1)  # Hidden layer
logits = h @ W2 + b2  # Output layer
loss = F.cross_entropy(logits, y)

# Backward pass
loss.backward()

# Parameter update
optimizer.step()
```

7.7.1 Forward pass

During the forward pass, executing `h = torch.relu(x @ W1 + b1)` triggers an eager execution sequence:

1. Python dispatch: The Python interpreter invokes `torch.matmul`, which routes through PyTorch's C++ dispatcher to select the CUDA backend, incurring the Python-dispatch share of the per-operation overhead (about 10 µs in this chapter's model) before issuing work to the device.
2. Kernel selection: cuBLAS selects a GEMM kernel configured for dimensions $32 \times 784 \times 128$, partitioning the computation into thread blocks sized to fit the streaming multiprocessor shared memory and L2 cache tiles.
3. Kernel launch: The host thread enqueues the launch descriptor into the GPU command stream, consuming driver launch latency while asynchronous execution allows host execution to proceed without blocking on completion.
4. GPU execution: The streaming multiprocessors stream `W1` from HBM[24] through L2 cache into registers and the FP32 arithmetic units (Tensor Cores engage only when TF32 or a 16-bit data type is enabled), evaluate the matrix product, and write the output activations back to HBM.
5. Autograd recording: Concurrently on the host, PyTorch's autograd engine instantiates an `MmBackward` node on the dynamic tape, saving references to the device tensors needed for the requested gradients. Here the weight gradient requires `x`; no input gradient is requested.

The subsequent bias addition (`+ b1`) and activation (`torch.relu`) follow an identical eager path, with each operation dispatching a separate kernel, executing round-trips to HBM, and appending a node to the tape. By the time the forward pass reaches the loss function, the autograd graph holds a complete chain of backward functions and saved activation pointers ready for reverse traversal.

7.7.2 Backward pass

Calling `loss.backward()` executes reverse-mode automatic differentiation in four stages:

1. Tape traversal: The autograd engine traverses the recorded computational graph in reverse topological order, starting from the scalar loss node.
2. Gradient computation: The engine executes the backward kernel registered to each node. Traversing backward, `CrossEntropyBackward` evaluates $\frac{\partial \mathcal{L}}{\partial \text{logits}}$ using the combined softmax-cross-entropy derivative; `MmBackward` for $\mathbf{W}_2$ computes $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \mathbf{H}^T \cdot \frac{\partial \mathcal{L}}{\partial \text{logits}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{H}}$, where $\mathbf{H}$ is the batch of hidden activations (`h`); `ReluBackward` evaluates the activation mask, zeroing gradients where forward activations satisfy $\mathbf{H} \leq 0$; and `MmBackward` for $\mathbf{W}_1$ computes $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \mathbf{X}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1}$, where $\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1$ is the pre-activation. Because the input batch $\mathbf{X}$ (`x`) does not set `requires_grad=True`, traversal terminates without evaluating $\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$.

[24] **HBM (high bandwidth memory)**: Provides 3.35 TB/s on the H100, making it the memory tier that most directly feeds accelerator arithmetic; HBM bandwidth determines whether operations are memory bound or compute bound, and its 80 GB capacity sets the hard ceiling on total live training state. Weights, activations, gradients, optimizer state, and temporary workspace must fit during execution. When they do not, the framework must resort to offloading, selective recomputation, or placement across devices, each adding complexity to what the programmer perceives as a single `loss.backward()` call.

3. Gradient accumulation: Computed parameter gradients accumulate in-place into the `.grad` fields of leaf tensors ($\mathbf{W}_1$, $\mathbf{b}_1$, $\mathbf{W}_2$, $\mathbf{b}_2$).
4. Saved-state release: As each backward node completes, the framework releases the saved tensor references that it no longer needs. Storage becomes reusable only when no other live reference retains it; in this listing, Python variables such as `h` can keep tensors alive after backward. The caching allocator can reuse released regions for subsequent operations without returning them immediately to the device.

After backward fills the parameter `.grad` buffers, `optimizer.step()` reads those gradients, updates the parameters, and advances any optimizer state. Backward itself computes gradients without applying that update, and the optimizer state the update reads is part of training memory, not of the backward pass.

7.7.3 Memory traffic

The operation counts below follow the products required by the backward traversal above. The count excludes input generation, bias additions and reductions, optimizer updates, and allocator bookkeeping. Within that boundary, each matrix multiplication reads two operands and writes one result. Counting these buffers and the corresponding arithmetic connects the graph to an execution-time estimate; loss and activation kernels receive the simpler operation-equivalent models stated below.

The traffic model assumes FP32 values occupying 4 bytes each. For `x @ W1`, it counts one read of the input matrix with shape `(32, 784)`, one read of the weight matrix with shape `(784, 128)`, and one write of the result with shape `(32, 128)`. Multiplying each shape's dimensions, summing the three element counts, and multiplying by the element width gives the table's 0.52 MB. The second matrix multiplication uses the same operand rule with its own shapes.

ReLU instead reads and writes each element of its `(32, 128)` tensor once, so its modeled traffic is twice that tensor's element count times 4 bytes, yielding 32.8 KB. These counts describe the modeled operator-buffer boundary. They assume full operand reuse within each matrix multiplication and exclude extra layout copies, workspace, and repeated tile loads; cached operands can also reduce actual HBM reads. Saving a reference for backward does not itself copy a tensor. A physical layout conversion, when required, adds its own read and write traffic to these counts.

The backward products use the same read/read/write rule. The weight gradient `h.T @ dlogits` multiplies `(128, 32)` by `(32, 10)`; the hidden gradient `dlogits @ W2.T` multiplies `(32, 10)` by `(10, 128)`; and `x.T @ dz1` multiplies `(784, 32)` by `(32, 128)`. The first two each cost as much arithmetic as the second forward GEMM, while the third matches the first forward GEMM. No `(32, 128)` by `(128, 784)` product is needed because the raw input requests no gradient. The total dense matrix work is therefore 2.013× the forward dense work. This applies the input-gradient exception derived in section 5.4.5.4 to this two-layer network.

The forward cross-entropy model assigns one operation-equivalent each to exponentiation, accumulation, and normalization per logit, and counts the logit read and saved-score write. For the nonlinear backward work, the ReLU model reads the saved activation and upstream gradient and writes a masked gradient. The simplified combined cross-entropy model reads saved probabilities and integer labels, then subtracts the target and scales by the batch mean to write the logit gradient. Actual loss implementations may split these operations across kernels. The resulting estimates, collected in table 7.9, make the arithmetic-intensity comparison reproducible without substituting for a measured memory-traffic profile.

Table 7.9: Per-Operation Roofline Analysis: Analytical work and buffer-traffic estimates separate the two-layer MLP's matrix multiplications from its low-intensity elementwise and loss operations. Each GEMM counts two operand reads and one result write; backward includes only the requested gradients. Cross-entropy and pointwise derivative work use the simplified models stated in the text. These are operator-level estimates rather than measured device instruction or HBM-transaction counts.

Component	Modeled Work	Memory Traffic	Arithmetic Intensity
MatMul (x @ W1)	$2 \times 32 \times 784 \times 128 = 6.42$ MFLOP	0.52 MB	12.4 FLOP/byte
ReLU	$32 \times 128 = 4.1$ KFLOP	32.8 KB	0.125 FLOP/byte

Component	Modeled Work	Memory Traffic	Arithmetic Intensity
MatMul (h @ W2)	$2 \times 32 \times 128 \times 10 = 81.9$ KFLOP	22.8 KB	3.6 FLOP/byte
Cross-entropy (simplified)	~0.96 KFLOP	2.6 KB	0.375 FLOP/byte
Backward `dW2`	81.9 KFLOP	22.8 KB	3.6 FLOP/byte
Backward `dh`	81.9 KFLOP	22.8 KB	3.6 FLOP/byte
Backward `dW1`	6.42 MFLOP	0.52 MB	12.4 FLOP/byte
Backward nonlinear (simplified)	4.7 KFLOP	52.0 KB	0.091 FLOP/byte

The arithmetic-intensity column helps predict each layer's regime when compared with the target machine's compute-to-bandwidth ratio. Matrix multiplications reuse operands far more than element-wise operations, but their shapes still determine whether they reach the compute roofline. Conversely, the forward ReLU and cross-entropy estimates perform negligible compute per byte transferred (0.125 FLOP/byte and 0.375 FLOP/byte), making them sensitive to memory bandwidth and, at small sizes, launch latency. In un-fused eager execution, each element-wise operation incurs a separate kernel dispatch and writes intermediate tensors to global-memory buffers between kernels; actual HBM traffic depends on caching.

The analytical model yields ~13.1 MFLOP of arithmetic work and ~1.2 MB of memory traffic for the operations included in the table. Dividing these quantities by peak H100 hardware rates determines the idealized lower bounds:

- $T_{\text{compute}} \approx 13.1$ MFLOP/67 TFLOP/s FP32 ≈ 0.2 µs
- $T_{\text{memory}} \approx 1.2$ MB/3.35 TB/s ≈ 0.4 µs
- Under a one-kernel-per-modeled-operator assumption, the 4 selected forward operators and 5 backward operators require 9 driver launches. Python enters each forward operation and calls `backward()` once, giving 5 entries. The illustrative host cost is therefore 5×10 µs plus 9×5 µs, or 95 µs.
- The excluded optimizer step is not small. An FP32 Adam update reads each parameter, its gradient, and two moments and writes three of them, 28 bytes per parameter, so the 101,770 parameters of this network move about 2.85 MB, more than the whole modeled forward and backward traffic. Counting its launches and the omitted Python calls only raises the host cost above this lower bound.

This dispatch model excludes autograd traversal overhead, allocation, and the operations omitted from the table. The backward engine launches its internal kernels without a separate Python call for each one; a profile must establish the actual launch count, particularly for loss kernels. Host costs are added without overlap here. In this model, T_{sw} exceeds the ideal lower bounds T_{compute} and T_{memory} by more than an order of magnitude, so the overhead ratio of equation 7.4 sits far above 1 and the step is overhead-bound under these assumptions. This is the small multilayer perceptron (MLP) regime of notebook 7.2, now reached from a real training loop. Profiling has a specific job here, which is to confirm that measured kernel times, achieved bandwidth, and host-device overlap match the model before acting on it. When dispatch dominates, three mechanisms attack the launch overhead from different angles: kernel fusion via `torch.compile` merges consecutive memory-bound operations into a single kernel to avoid intermediate round-trips to HBM and collapse launch boundaries; batch size scaling grows the arithmetic payload linearly while keeping dispatch overhead constant to amortize fixed launch latency across more compute work; and graph capture removes repeated Python work in captured regions while allowing CUDA Graph replay to submit a captured launch sequence with less host overhead, without requiring the entire step to become one fused kernel.

7.7.4 Dispatch to hardware

The tensor operations in this training step map to heterogeneous hardware architectures through backend abstraction layers, as table 7.10 summarizes. The displayed script uses CUDA; another target requires matching device placement and a supported backend execution path. Each backend maps common tensor semantics to silicon-specific primitives, so a single `loss.backward()` invocation

routes to different execution pipelines depending on device placement. Because differences in floating-point summation order, reduction trees, and non-deterministic atomic operations introduce numerical variation across backends, equivalence must be validated against explicit numerical tolerances rather than bitwise identity.

Table 7.10: Hardware Backends Behind a Single Code Path: Each backend implements the same tensor operations through a different library and execution mechanism, so identical Python maps onto distinct silicon.

Hardware	Backend library	Execution mechanism
CUDA GPU	cuBLAS (NVIDIA 2024a; Choquette et al. 2021)	GEMM kernels and CUDA streams for async execution
CPU	Intel oneMKL or OpenBLAS (Intel Corporation 2026a; OpenBLAS Project 2026)	Thread-level parallelism around optimized kernels
TPU	XLA (Google 2025)	Compilation to TPU-specific HLO operations
Apple Silicon	Metal Performance Shaders	MPS backend

Implementation decisions such as device placement, batch size, and compilation mode cascade through execution, differentiation, and abstraction layers simultaneously. Treating the framework as an opaque mathematical interpreter rather than a hardware-aware scheduler hides these interactions until they surface as an idle accelerator or an out-of-memory failure.



Systems Perspective 7.2: The three problems in action

In D·A·M terms, the single training step exposes how the framework balances algorithm and machine:

- Execution: Eager mode preserves imperative line-by-line debugging at the cost of per-kernel launch and dispatch latency.
- Differentiation: Reverse-mode automatic differentiation retains intermediate forward activations in HBM to compute exact vector-Jacobian products in reverse topological order.
- Abstraction: Unified tensor APIs decouple model code from heterogeneous silicon, dispatching identical operations to cuBLAS, oneMKL, or XLA-compiled HLO programs.

Resolving the friction among these dimensions requires matching operator granularity to hardware rooflines: fusing operations to eliminate memory round-trips, scaling batches to amortize launch latency, and structuring tensors to saturate accelerator execution pipelines.

The step's costs reduce to a handful of counting rules derived earlier in the chapter and collected in the chapter summary, so a training or inference step can be estimated on paper before it is profiled: count dispatched operations and compare their overhead with device time, count the passes an unfused elementwise chain makes over memory, check whether a compiled artifact will run often enough to repay its compilation, size the activations that backward must keep, and time every host-to-device copy. Applied to the two-layer MLP, the dispatch rule gives the overhead-bound verdict reached above, the elementwise-traffic rule explains why fusing its bias and ReLU kernels removes a round trip, and the saved-activation rule bounds what activation checkpointing could save. Every one of these costs exists whichever framework runs the step; frameworks differ in which of them they make easy to see and to reduce. TensorFlow, PyTorch, and JAX each answer the three framework problems in a different way.

7.8 Major Frameworks: TensorFlow, PyTorch, and JAX

A team that prototypes quickly but cannot deploy the resulting model, or deploys reliably but cannot debug training failures, has run into a framework design trade-off rather than a missing API

call. Each major framework solves all three core problems but emphasizes a different path through them. TensorFlow couples eager development to graph capture and a broad deployment ecosystem; PyTorch begins with eager execution and adds capture, compilation, and export paths; JAX organizes differentiation, vectorization, and compilation as composable function transformations. These are architectural emphases, not exclusive capabilities, and their value depends on the workload and deployment target.

7.8.1 TensorFlow

TensorFlow's architecture spans eager development and graph-based optimization across hardware from cloud TPUs to microcontrollers. TensorFlow 2 executes eagerly by default. Performance-sensitive functions can be captured with `tf.function`, and deployment formats route captured computation to server, mobile, and browser runtimes. This dual path preserves the production graph capabilities inherited from TensorFlow 1.x without requiring every TensorFlow 2 program to build a complete graph before it runs.

Within a captured `tf.function`, TensorFlow can apply graph optimizations such as constant folding, kernel fusion, and layout planning. The SavedModel serialization format is the boundary that separates training from deployment (figure 7.13). Training workflows on the left, including `tf.data` preprocessing, the Keras modeling API, and distribution strategies across CPUs, GPUs, and TPUs, serialize into one SavedModel artifact. From this central interface, graph definitions pass through supported serving or target-specific conversion paths into deployment runtimes on the right (TensorFlow Serving, TensorFlow Lite, TensorFlow.js, and native language bindings) without retaining Python runtime dependencies.

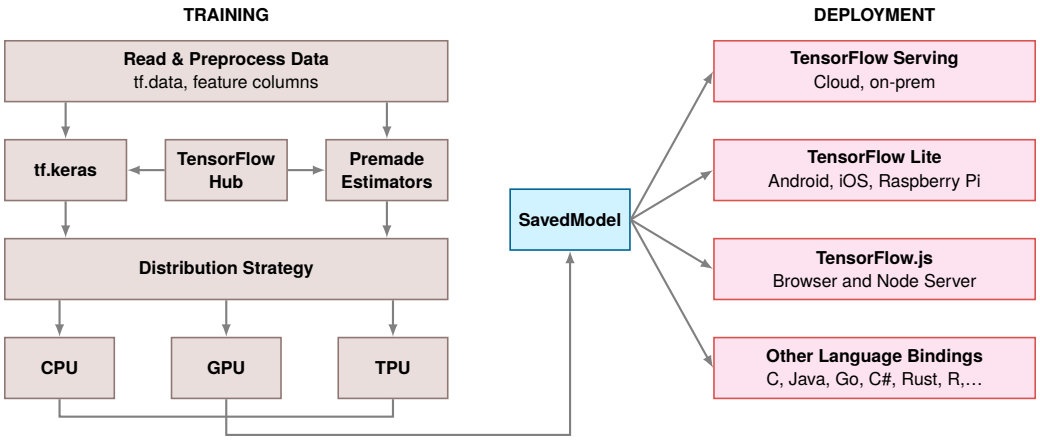


Figure 7.13: TensorFlow Training-to-Deployment Pipeline: The SavedModel format provides a boundary between training and deployment by capturing graph structure, weights, and signatures. The right-hand paths include direct serving and target-specific conversion for Lite and JavaScript runtimes. Supported operators and conversion determine whether each path can run without Python in production.

TensorFlow's answer to the execution problem is therefore two-sided: eager by default for development, with `tf.function` as the graph-conversion path for performance-sensitive code (TensorFlow Developers 2024a). Its strength is the compiled path from a captured graph to many deployment runtimes.

7.8.2 PyTorch

Where TensorFlow's design centers the path from captured graph to deployment runtime, PyTorch centers *developer experience*. PyTorch was built from the start on dynamic graphs (or "Define-by-Run"). Instead of building a graph before execution, PyTorch builds the computational graph as the code runs, because researchers need immediate feedback when experimenting with novel architectures and a define-then-run cycle slows that loop. Python loops, conditionals, and debuggers work directly inside a model's forward pass, with no special syntax and no separate compilation step, which matters most while architectures and training objectives are still changing.

PyTorch's answer to the differentiation problem is tape-based autograd (section 7.4.3.1): flexible and debuggable, but harder to optimize globally because the tape is rebuilt each iteration. Its answer to the abstraction problem is more pragmatic than comprehensive: strong GPU support through cuBLAS and cuDNN, with deployment paths including `torch.export`, edge-targeted ExecuTorch, ONNX, and specialized runtimes.

The trade-off is therefore a more fragmented deployment path. Because the graph is dynamic, the framework cannot easily perform global optimizations before execution. A model that works perfectly in development may hit performance walls in production when dispatch overhead dominates small operations. To bridge this research-to-production gap, PyTorch added graph-capture and compilation paths (`torch.compile` and export workflows) that capture a dynamic model and turn it into an optimized representation for deployment. This evolution shows how an eager framework can move toward the production end of the compilation continuum while preserving the interactive experience that motivated the design.

7.8.3 JAX

PyTorch's eager execution and TensorFlow's graph compilation represent two points on a spectrum, yet both share an imperative programming heritage where computation proceeds as a sequence of stateful operations. JAX takes a different user-facing approach, built on functional programming principles and composable program transformations rather than object-level tapes or user-authored graph APIs (Bradbury et al. 2018). Developed at Google, JAX is especially useful for work requiring custom differentiation, advanced optimization research, and large-scale distributed training.

JAX's architecture treats differentiation, vectorization, and compilation as transformations on functions. The `jax.grad` function returns a function that computes gradients, which can then be transformed again when the interfaces are compatible. For example, `vmap(grad(f))` can compute per-example gradients and `jit(vmap(grad(f)))` can compile that batched computation. Transformation order is semantically meaningful rather than arbitrary.

The programming model shifts from tracking state through objects to transforming pure functions. Where PyTorch and TensorFlow expose autograd through dynamic tapes or graph-compilation paths, JAX asks users to write pure Python functions, and automatic differentiation, vectorization, and JIT compilation all become *program transformations* that compose (listing 7.21).

Listing 7.21: JAX Function Transformations: Wrapping an ordinary loss function in `grad`, then `vmap`, then `jit` yields per-example gradients compiled through XLA.

```
import jax
import jax.numpy as jnp


def loss_fn(params, x, y):
    pred = jnp.dot(x, params["w"]) + params["b"]
    return jnp.mean((pred - y) ** 2)


# Transform: compute gradients
grad_fn = jax.grad(loss_fn)

# Transform: vectorize over batch dimension
batched_grad = jax.vmap(grad_fn, in_axes=(None, 0, 0))

# Transform: compile to XLA
fast_batched_grad = jax.jit(batched_grad)

# Compose all three: fast, batched gradient computation
```

This functional approach requires **pure functions** (no side effects) and **immutable data** (arrays cannot be modified in place). These constraints are stricter than PyTorch's mutable object model, and they are what make the transformations safe. Because function outputs depend only on in-

puts, the compiler can reorder, fuse, and parallelize operations without changing results, and one transformation can be applied to the output of another.

Each transformation does one job. `jax.grad` returns a gradient function; `jax.vmap` can vectorize a compatible function over mapped axes; and `jax.jit` can trace and compile a stable transformed function for a particular argument signature. Unlike an iterative Python loop that issues individual operations sequentially with repeated dispatch overhead, `jax.vmap` pushes the batch dimension directly into the underlying primitives, allowing compatible per-sample matrix-vector operations to become batched matrix-matrix multiplications (GEMM). The compiled program can contain multiple kernels and library calls. Across devices, JAX shards arrays over a device `Mesh` and compiles with `jax.jit`, or maps explicitly with `jax.shard_map` (the successor to the older `jax.pmap`); gradient aggregation is expressed with collectives such as `lax.psum` or `lax.pmean`. These transformations compose according to their types and semantics, and different orders can compute different quantities.

The same minimalist core delegates neural network abstractions to companion libraries (Flax and Equinox) and optimization to Optax. This separation reflects the functional philosophy: the core provides transformations, while libraries build conventional abstractions on top. The trade-off is that production readiness depends on both the transformation model and the maturity of the surrounding libraries, export paths, and operational tooling for the target environment.

The functional constraints pay off where gradient computation is itself the object of study. Custom differentiation, including higher-order gradients and custom vector-Jacobian product (VJP) and Jacobian-vector product (JVP) rules, composes cleanly because pure functions make differentiation rules predictable, which serves optimization research and scientific computing with AD. Compilation-heavy accelerator workloads gain utilization from XLA when the program fits the functional style. In exchange, JAX demands more upfront investment than PyTorch, with explicit state management, a learning curve for the functional paradigm, and compiled code that is harder to debug than eager execution. It fits teams whose requirements match these strengths rather than serving as a default.

7.8.4 Design trade-offs compared

The preceding sections described each framework's design philosophy in qualitative terms: graph-first vs. eager-first, stateful vs. functional. The useful comparison is not which framework is fastest in the abstract, because that answer changes with model shape, batch size, hardware backend, and compiler configuration. The useful comparison is what each design lets the system see and optimize. Table 7.11 therefore maps TensorFlow, PyTorch, and JAX back to the three framework problems: execution visibility, differentiation model, and hardware abstraction path.

Table 7.11: Framework Design Trade-Offs: Each column reflects a distinct answer to the three core problems. TensorFlow makes deployment and graph capture central, PyTorch makes eager execution and debugging central, and JAX makes functional transformation and compiler visibility central. The table is a diagnostic map, not a benchmark ranking.

Aspect	TensorFlow	PyTorch	JAX
Graph Type	Static roots, dynamic front end in 2.x	Dynamic	Functional transformations
Programming Model	Imperative front end, graph capture path	Imperative	Functional
Core Data Structure	Tensor with framework-managed state	Tensor with framework-managed state	Immutable array
Execution Mode	Eager by default, graph for optimization	Eager by default	Trace and just-in-time compilation
Automatic Differentiation	Reverse mode over captured computation	Reverse mode over an eager tape	Forward and reverse transformations
Hardware Abstraction	Broad deployment runtimes and XLA paths	Native GPU path plus export/compile runtimes	XLA-centered accelerator compilation
Optimization Risk	Graph capture and operator coverage	Graph breaks after eager development	Purity, shape stability, and tracing

The comparison therefore tells an engineer which constraint to profile first. In PyTorch, check whether eager dispatch or graph breaks dominate. In TensorFlow, check whether the captured graph covers the operators and deployment target. In JAX, check whether shapes and purity let XLA compile the program actually executed. A framework-level comparison or leaderboard can orient a decision, but it cannot replace profiling the specific workload on the target hardware.

The same simple network exposes how each design philosophy shapes the code. Listing 7.22 implements one neural network, a single linear layer mapping ten inputs to one output, across all three frameworks.

Listing 7.22: Hello World in Three Frameworks: One single-layer network is written three ways, in PyTorch (object-oriented), TensorFlow/Keras (declarative), and JAX (functional).

```
# PyTorch - Dynamic, Pythonic
import torch.nn as nn


class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 1)

    def forward(self, x):
        return self.fc(x)


# TensorFlow/Keras - High-level API
import tensorflow as tf

model = tf.keras.Sequential(
    [tf.keras.layers.Dense(1, input_shape=(10,))]
)

# JAX - Functional approach
import jax.numpy as jnp
from jax import random


def simple_net(params, x):
    return jnp.dot(x, params["w"]) + params["b"]


key = random.PRNGKey(0)
key_w, key_b = random.split(key)
params = {
    "w": random.normal(key_w, (10, 1)),
    "b": random.normal(key_b, (1,)),
}
```

These three implementations solve the same mathematical problem, yet each reveals where its framework keeps state and when its computation graph comes into existence. PyTorch binds state and computation together through class inheritance (`nn.Module`) and evaluates eagerly, building the graph as Python runs so that standard debuggers and control flow work naturally, at the cost of hiding the full computation from an optimizer unless a graph-capture pass is invoked. TensorFlow and Keras also execute eagerly by default in TensorFlow 2, whereas structured models can be captured with `tf.function` and exported to supported deployment runtimes whenever operator coverage and target conversion allow. By contrast, JAX treats the model as a pure function[25] over immutable arrays with no internal object state, turning `grad`, `vmap` (automatic vectorization), and `jit` (just-in-time compilation[26]) into composable transformations on stateless functions in exchange for explicit parameter passing.

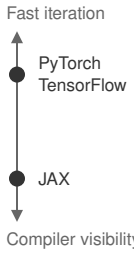
No framework maximizes all three goals simultaneously, and the ten-line program makes the trade visible in source code rather than buried in compiler internals: where state lives, when the

[25] **Pure function**: Returns outputs determined by its inputs without relying on untracked side effects; JAX transformations are designed for pure functions. Ordinary Python effects may run during tracing but are not represented in the compiled computation, and impure code can fail or behave unexpectedly. Runtime printing or external effects require supported mechanisms such as `jax.debug.print` or callbacks; random draws require explicitly managed keys.

[26] **Just-in-time (JIT) compilation**: Traces a function for an argument signature and lowers it to backend-specific executable code; the first call pays tracing and compilation costs; compatible later calls can reuse the cached executable. New shapes, dtypes, devices, or static arguments may trigger another compilation. Both compilation time and cached-call overhead depend on the program, backend, and hardware.

graph exists, and what the compiler is allowed to see. Each design trades what the compiler can see against how quickly a person can iterate. Graph capture and ahead-of-time compilation can reduce runtime overhead and intermediate data movement; eager evaluation shortens the human iteration loop, outside the iron-law runtime equation, at the cost of compiler visibility until a graph-capture path is used; functional purity gives XLA more freedom to transform a stable traced program. Each philosophy also shapes code syntax, team workflows, debugging practices, and deployment pipelines, so migration costs grow with project-specific integrations and assumptions.

7.9 Choosing a Framework



Eager defaults favor iteration; JAX's traced functions favor the compiler.

Those differences in where state lives and what the compiler sees become concrete obligations once a team commits to one framework for a project that must ship to a particular device. Framework selection decides which framework a team develops in, given the model it must express, the team that must maintain it, and the targets it must eventually run on. Development framework and deployment runtime need not be the same system: a model can be developed in PyTorch or TensorFlow and exported to a compact runtime for the device. The choice of that inference runtime is a serving decision, developed in section 13.6. The development framework still has to reach it, so the deployment target acts as a filter on which frameworks are viable.

Suppose a team deploys a fixed-shape vision model to a CPU-only embedded Linux device, with a 20 ms p99 (99th-percentile, section 12.6.2.1) inference-latency target under a specified load and a 256 MB model-and-runtime memory budget. Assume target-device trials with matching inputs, batch size, precision, and numerical tolerances establish two viable paths: PyTorch development with ONNX Runtime has 18 ms p99 latency and 140 MB peak memory use, while TensorFlow development with TensorFlow Lite has 15 ms p99 latency and 90 MB peak memory use. A path that depends on a GPU-only runtime fails the hardware filter even if its latency elsewhere is lower. These are hypothetical project measurements, not general performance rankings. The selection must now distinguish a feasible path from a worthwhile migration.

The embedded project has already fixed the deployment target, but it leaves two choices open: which development environment the team can maintain and which compatible export path meets its budget. Eager development and compiled deployment can coexist along either path, so the team need not trade away its debugging workflow to obtain a compact inference runtime.

> **Systems Perspective 7.3: Framework selection constraints**
>
> Rather than seeking the "best" framework, effective selection first eliminates candidates that cannot satisfy hard constraints: deployment target, required operations, compiler path, and team expertise. Only then should soft preferences such as performance, development speed, and ecosystem rank the survivors.

7.9.1 Framework selection criteria

The two hypothetical measurements establish latency and memory feasibility only for the tested model and inputs. A defensible choice also checks operator semantics, software dependencies, and hardware coverage. The three TensorFlow variants in table 7.12 make those filters visible within one ecosystem; the same questions apply to the PyTorch-to-ONNX path.

Table 7.12: TensorFlow Variants Across Deployment Tiers: The rows group what each variant can express (first four), what it requires of the platform (middle three), and what hardware and footprint it targets (last three). Training capability disappears across the tiers while inference and lower-precision tooling survive, which leaves operator coverage as the discriminating capability: broad in TensorFlow, inference-focused in TensorFlow Lite, and reduced to the operators one model selects in TensorFlow Lite Micro. Binary size and memory footprint depend on the build, selected operators, delegates, model, and runtime configuration rather than on the variant alone.

	TensorFlow	**TensorFlow Lite**	**TensorFlow Lite for Microcontrollers**
Training	Yes	Limited	No
Inference	Yes	Yes	Yes

	TensorFlow	**TensorFlow Lite**	**TensorFlow Lite for Microcontrollers**
Operator Coverage	Broad	Inference-focused	Model-selected subset
Native Lower-Precision Tooling	Yes	Yes	Yes
Needs an OS	Yes	Yes	No
Model Access	Filesystem or service	Platform-specific storage mapping	Platform-specific compiled or mapped storage
Accelerator Support	Yes	Delegates	Optimized kernels and integrations
Base Binary Size	Build- and package-dependent	Build- and delegate-dependent	Model- and operator-selection-dependent
Base Memory Footprint	Model-, runtime-, and allocator-dependent	Model-, delegate-, and allocator-dependent	Fixed arena plus runtime state
Optimized Architectures	x86, TPUs, GPUs	Arm Cortex-A, x86	Arm Cortex-M, DSPs, MCUs

7.9.1.1 Model requirements

For the fixed-shape vision model, the first filter is successful conversion of every required operation with acceptable numerical results. Training support is needed in the development framework but not in the deployed runtime. TensorFlow, TensorFlow Lite, and TensorFlow Lite Micro expose different operator and training capabilities. A missing inference operator, by contrast, can invalidate the deployment path even when the rest of the graph fits, which is why operator coverage is the discriminating row among the capabilities in table 7.12.

> Systems Perspective 7.4: Dynamic vs. static computational graphs
>
> The static-vs.-dynamic graph distinction (examined in section 7.3) has direct implications for model requirements analysis. Static graphs require control flow and dynamic behavior to be represented in forms understood by the graph runtime, which enables ahead-of-time optimization for deployment. Eager execution follows ordinary Python control flow directly, while graph-capture paths such as `torch.compile` and `tf.function` recover optimization opportunities when they can represent the executed program.

7.9.1.2 Software dependencies

Embedded Linux supplies operating-system services, so an OS dependency does not eliminate either candidate in this project. A bare-metal microcontroller would impose a different filter. The platform rows of table 7.12 separate that requirement from model access and accelerator integration; availability of a runtime on one tier does not imply availability on another.

The CPU-only project needs a supported CPU implementation of every required operator; it cannot count on a mobile NPU delegate or a server GPU backend. TensorFlow Lite Micro instead links a selected operator set into a small C/C++ application and can read a model FlatBuffer compiled into flash. That design removes the need for a POSIX filesystem or heap allocation during inference, but those savings do not by themselves justify selecting a microcontroller runtime for a Linux device.

7.9.1.3 Hardware constraints

A runtime that supports the operating system still has to fit alongside the model, working buffers, and application. The hardware rows of table 7.12 separate supported processor families from footprint, which depends on the actual build. The project must compare peak memory for the deployed configuration, not the size of its weight file or an empty runtime package.

The supplied trials already include model-and-runtime peak memory, so both candidates pass the project's capacity filter. That conclusion must be revisited when input shapes, selected operators, delegates, or runtime versions change. On a fixed-arena target, the analogous check is whether the planned tensor and scratch buffers fit with the rest of the application.

7.9.1.4 Production-ready evaluation factors

Satisfying model requirements, software dependencies, and hardware constraints establishes technical feasibility; production viability also depends on what the development framework's toolchain supports. If the deployment uses reduced precision, the framework must support the chosen path: post-training quantization (PTQ), which converts an already-trained model using a small representative calibration dataset to set value ranges, or quantization-aware training (QAT), which simulates low-precision arithmetic during training to recover accuracy before deployment. Toolchain stability matters as much: whether saved models remain loadable across framework and runtime updates, and whether the framework exposes the profiling hooks needed to confirm that the deployed model still meets its latency target.

↪ Post-training quantization, its calibration, and quantization-aware training are developed in section 10.3.3.

7.9.2 Development support and long-term viability assessment

Once both paths pass the technical filters, the remaining decision concerns the team's ability to maintain them. The project already has PyTorch models and an ONNX export pipeline. Adopting another development stack would add conversion validation, new debugging practices, and another set of runtime dependencies even though both candidates meet the deployment limits.

Custom C++ or CUDA operators, specialized serialization, and vendor compiler passes increase the cost of later migration. Preserving an exit path means isolating framework-specific ingestion and operator code and testing the intended export format throughout development. These commitments belong in the selection decision because the shortest inference time does not account for the work needed to keep the model deployable.

Both surviving paths meet the stated runtime limits. The team already maintains PyTorch models and an ONNX export pipeline, so it selects PyTorch with ONNX Runtime. The measured path satisfies deployment requirements without introducing a second development stack. The faster TensorFlow Lite path remains viable, but its unused latency and memory advantage does not by itself justify migration. The decision depends on retaining that headroom as the model changes. Before committing, the team must check export coverage and numerical tolerances across representative inputs and repeat the latency check under expected contention; a later operator change or tighter latency target can invalidate the choice. The choice follows the project's constraints and maintenance costs; a different team or deployment envelope can favor the other viable path.

The worked selection turned on constraints that no benchmark table records: export coverage, numerical tolerance, the team's existing pipeline, and the cost of a second stack. Engineers who reason from benchmark headlines or general-purpose programming habits fall into predictable traps.

7.10 Fallacies and Pitfalls

Framework selection involves subtle trade-offs where intuitions from conventional software engineering fail. The memory wall, kernel fusion constraints, and deployment target diversity create pitfalls that waste engineering effort and cause production systems to miss latency targets.

Fallacy: *All frameworks provide equivalent performance for the same model architecture.*

Engineers assume that a model performs the same in every framework because the mathematics is the same. The GPT-2 XL decoding step that opened this chapter refutes this: the identical step spends about 8.6 ms on the host when an eager framework issues its 576 kernels one at a time, against a weight read of about 0.9 ms once the launches are captured and replayed. In the hypothetical ResNet-50 scenario of table 7.4, compilation raises throughput by 48.3 percent and a specialized inference engine by 2.6× over eager mode. Fusion depth, graph capture, and backend libraries differ between frameworks, so organizations that assume equivalence can miss latency targets and face late framework migrations.

Pitfall: *Choosing frameworks based on popularity rather than project requirements.*

Engineers assume the most popular framework works for any project. In reality, deployment constraints dominate. A mobile runtime such as ExecuTorch or TensorFlow Lite and a microcontroller runtime such as TensorFlow Lite Micro make different assumptions about memory allocation, operator coverage, and hardware delegates; the former targets phones with GB-scale memory, while

the latter often targets devices with hundreds of KB of RAM, commonly below 256 KB for small TinyML deployments. Teams that prototype edge applications without checking the final runtime can face memory bloat that exceeds device capacity or a late framework migration after development completes. Check the deployment runtime (section 13.6) before selecting a training framework.

Fallacy: *Framework abstractions eliminate the need for systems knowledge.*

Engineers assume high-level APIs handle all optimization automatically. The training step of section 7.7 shows otherwise. Its ReLU has an arithmetic intensity of 0.125 FLOP/byte, about 160× below the H100's FP32 ridge of roughly 20 FLOP/byte, and the whole step spends about 95 µs in host dispatch against well under a microsecond of ideal device time. No API call reveals either fact. Engineers must identify the limiting resource before choosing fusion, batching, layout, or precision optimizations.

Pitfall: *Ignoring vendor lock-in from framework-specific formats.*

Engineers assume framework migration is straightforward since models are "just math." Converting TensorFlow SavedModel to PyTorch can require rewriting custom operations, validating numerical equivalence across large test suites, and retraining when operations lack exact equivalents. The ONNX interchange format (section 13.6) improves portability, but custom operators, dynamic shapes, and backend-specific optimizations can still require manual work. Organizations that ignore this during initial framework selection face costly migrations when deployment requirements change or better frameworks emerge.

Pitfall: *Increasing batch size without modeling activation memory.*

Engineers assume that if memory is available, larger batches always improve throughput. Larger mini-batches amortize fixed dispatch overhead, but they also scale the activation memory that must stay live until the backward pass consumes it. In the illustrative ResNet-50 scenario of section 7.4.3.2, batch 64 already holds 8 GB–12 GB of saved activations. Batch 256 holds 32 GB–48 GB, and batch 512 holds 64 GB–96 GB, which on an 80 GB H100 forces activation checkpointing. Recomputation then adds roughly one extra forward pass per step, which can cost more throughput than the larger batch gained, and materialized attention scores make the effect sharper for transformers because they grow with the square of sequence length. Teams that blindly maximize batch size can achieve lower throughput than smaller batches that avoid recomputation.

↪ How activation memory grows linearly with batch size and caps the batch that fits in accelerator memory is developed in section 8.3.2.1.

Fallacy: *Compilation overhead is negligible.*

Engineers assume compilation overhead is a one-time cost that pays off quickly. The hypothetical values in table 7.4 assign ResNet-50 higher compiled throughput and a compile window of 15 s to 30 s per graph change. Under those assumptions, timed with the table's inference throughputs (a training loop would add backward work but pay the same recompilation cost), a 10,000-image experiment with 10 code changes completes in 6.9 s in eager mode and 304.7 s in compiled mode, including recompilation overhead. The compiled workflow is therefore about 44.2× slower in this illustrative rapid-prototyping scenario. Teams that enable compilation during rapid prototyping can waste time waiting for recompilations that negate throughput gains.

Pitfall: *Using one execution policy for exploration and production.*

The right framework mode depends on the loop. During exploration, eager execution and small tests shorten feedback by avoiding repeated graph captures and recompilations. During production serving or long training runs, compilation can amortize setup across many requests or samples. The deciding factors are workload duration and stability: compilation pays when graph structure and shapes recur, whereas changing control flow or input shapes can invalidate cached artifacts and restart the amortization. In the hypothetical ResNet-50 scenario, one 30 s compilation pays for itself only after about 134,000 images. A prototyping loop that recompiles every few minutes never reaches that point, while a serving system passes it within minutes of steady traffic. One policy for both phases either slows research iteration or leaves production throughput unused.

7.11 Summary

Machine learning frameworks solve three core systems challenges that make deep learning tractable on modern hardware. The first is execution: deciding when and how operations run. Frameworks

balance the trade-off between eager evaluation for interactive debugging and graph execution for whole-program compilation, while modern hybrid approaches such as `torch.compile` capture graphs dynamically without sacrificing the Python control loop. The second is differentiation: computing gradients automatically. Frameworks implement reverse-mode automatic differentiation that applies the chain rule across compositions of operations, constructing an autograd tape or backward graph of vector-Jacobian products while making the required forward values available through retention or recomputation. This primitive makes training large models feasible through a single `loss.backward()` call. The third is abstraction: targeting heterogeneous hardware from a unified interface. Frameworks provide multi-dimensional tensor abstractions, intermediate representations, and runtime dispatch systems that bridge high-level model definitions to optimized device kernels across CPUs, GPUs, TPUs, and custom accelerators.

These challenges are linked through the iron law of ML systems (section 1.7): execution strategy affects dispatch latency (L_{lat}), differentiation adds arithmetic and memory traffic (D_{vol}), and abstraction influences hardware utilization efficiency (η_{hw}). These are diagnostic emphases, not exclusive assignments, since fusion changes both launches and traffic while differentiation changes both work and live storage. Workloads with little reuse are frequently memory-bandwidth bound (D_{vol}/BW). Frameworks mitigate this memory wall through kernel fusion, which merges compatible elementwise operations to keep intermediates in registers or on-chip SRAM instead of round-tripping through global memory. The separate capacity limit yields to selective recomputation, which trades additional floating-point operations (O) for lower peak activation memory.

The counting rules developed across the chapter fit on one page (table 7.13). Together they let an engineer estimate, before profiling, whether a step is bound by dispatch, by memory traffic, or by arithmetic.

Table 7.13: Key Equations: Each counting rule appears in the notation of the section that derives it, so that a step can be estimated before it is profiled. The elementwise rule assumes each of n_{ops} unfused operations reads and writes all N elements of s_{elem} bytes once; the activation rule is written for a transformer layer stack, and an MLP drops the sequence length.

Cost	Formula	Where derived
Dispatch overhead ratio	$\frac{N_{\text{ops}} \cdot t_{\text{dispatch}}}{T_{\text{compute}} + T_{\text{memory}}}$; overhead-bound above 1	equation 7.4
Elementwise chain traffic	$2n_{\text{ops}}Ns_{\text{elem}}$ bytes unfused; $2Ns_{\text{elem}}$ bytes fused	section 7.3.2
Compilation breakeven	$N_{\text{breakeven}} = \frac{T_{\text{compile}}}{T_{\text{eager}} - T_{\text{compiled}}}$ executions	equation 7.3
Saved activation bytes	$M_{\text{activations}} \approx B \times N_L \times S \times d \times n_{\text{saved}} \times s_{\text{elem}}$	section 7.4.2
Host-to-device transfer time	$T = L_{\text{lat}} + D_{\text{vol}}/\text{BW}$	section 7.5.2

Key Takeaways: The layer between math and hardware

- **Every framework solves three problems**: Execution dictates runtime dispatch, differentiation computes gradients, and abstraction targets heterogeneous hardware. TensorFlow, PyTorch, and JAX give different answers on each axis, from graph capture to eager tapes to functional transformations, and those answers fix what a team can debug, compile, and deploy long after the API syntax has changed.
- **The memory wall drives optimization**: Accelerator compute capacity has outpaced memory bandwidth, making off-chip data movement a recurring bottleneck. Kernel fusion and lower-precision formats can reduce the memory traffic term (D_{vol}/BW), while selective recomputation primarily reduces peak activation storage by adding work.
- **Compilation must amortize setup cost**: The compilation continuum principle in equation 7.2 quantifies when runtime speedup offsets compilation overhead. While dynamic workloads favor eager execution, repeatedly executed graphs benefit from JIT or ahead-of-time compilation, where the dispatch overhead law (equation 7.4) demonstrates that small operations gain disproportionately when host launch overhead (L_{lat}) is reduced.

- **Gradients cost memory as well as arithmetic**: Reverse-mode differentiation keeps the forward values that backward rules read, so saved activations grow with batch size and depth and often dominate training memory; activation checkpointing trades recomputation for that storage.
- **Where a tensor lives sets what it costs**: A host-to-device copy runs at a small fraction of on-device bandwidth, so placement, pinned memory, and stream overlap decide whether transfers hide behind computation, while layout and data type decide which kernel runs.
- **Module abstractions automate state management**: Automatic parameter registration, mode flags that propagate from the root, hierarchical serialization, and hooks let one object be updated, saved, switched between training and evaluation, and inspected without per-tensor bookkeeping.
- **A step can be estimated before it is profiled**: The chapter's cost formulas (dispatch overhead ratio, fused traffic, compilation breakeven, saved activations, transfer time) show that a small MLP training step is overhead-bound on a data-center accelerator before any profiler runs.
- **Deployment targets filter the development framework**: A framework is viable only if it can export every required operator to a runtime that fits the target device; among the viable paths, team expertise and migration cost decide.

A framework presents itself as an ergonomic programming interface, yet its true role is translating mathematical graphs into physical hardware operations. That translation determines the work, traffic, and overhead paid under the iron law of ML systems (principle 3). Eager execution prioritizes developer iteration at the cost of host dispatch latency (L_{lat}), while graph compilation trades upfront optimization time to reduce avoidable global memory traffic (D_{vol}/BW) through fused kernels. Automatic differentiation adds backward work and traffic (D_{vol}), with checkpointing trading retained activation storage against recomputed arithmetic (O). What appears to be an API convenience therefore decides, for as long as a project uses it, how much avoidable work and data movement the implementation performs within physical limits that no framework can remove.

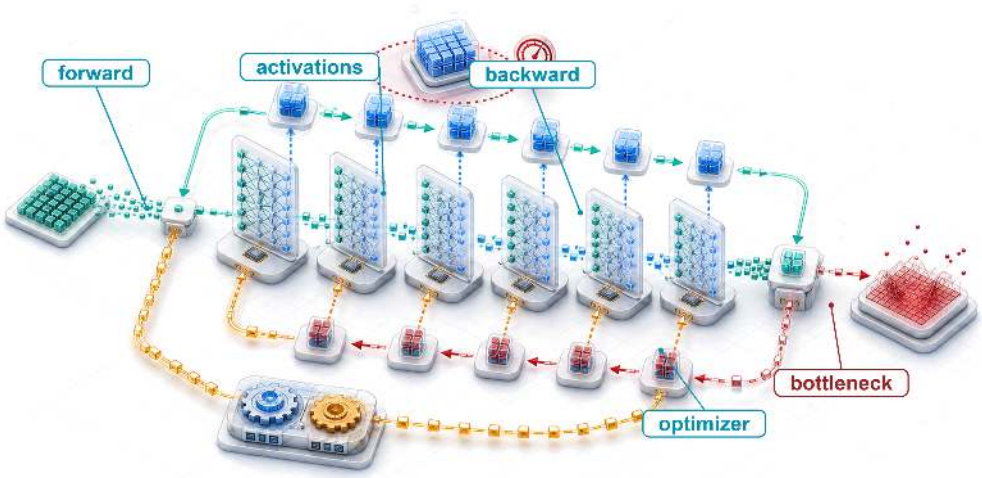
What's Next: From one training step to a training run

Frameworks translate architectures into executable kernels, and their computational graphs, autograd tapes, and dispatch pipelines are what let engineers see and control each step of training. This chapter traced a single training step through that machinery. A training run repeats that step many thousands or millions of times, and every budget decision the machinery exposes is still unmade, including which activations to checkpoint when memory runs short, how large a batch the hardware allows, and at what point a single device stops being enough. Chapter 8 makes those decisions. It opens with why training costs orders of magnitude more than inference, then budgets memory, time, and cost for real training runs and shows how far one machine can go before scaling out.

8

Model Training

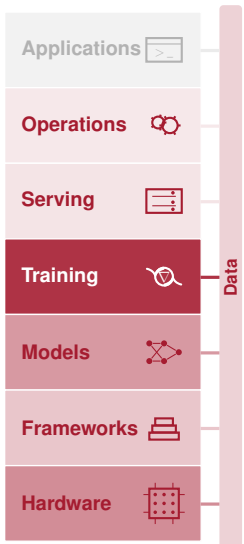
8.1 From One Step to a Training Run
8.2 Training-Time Law
8.3 Computation and Memory Costs of Training
8.4 Pipeline Architecture
8.5 Identifying Bottlenecks
8.6 Pipeline Optimizations
8.7 Scaling Training Systems
8.8 Fallacies and Pitfalls
8.9 Summary



Purpose

Why can training a model cost orders of magnitude more than one inference?

Inference computes a forward pass as data flows through the network and a prediction emerges. Training adds a backward pass and an optimizer step, retains intermediate state, and repeats that work across many examples and optimization steps. Optimizers may also maintain momentum and variance estimates whose storage exceeds the model weights, so memory rather than arithmetic often decides what fits on a device. The ratio depends on the model, workload, training duration, and inference volume, but one training run commonly costs orders of magnitude more than one inference. A lab that trains in three days can explore many more designs than one that takes a month. Over a year, that difference is more than a hundred iterations against about a dozen, and each result informs the next design while the slower lab waits weeks for every answer. A faster step is not faster training if it delays convergence, so throughput matters only relative to time to target quality. Across repeated experiments, even modest changes in convergence or utilization compound into substantial schedule and cost differences. The central systems challenge is to keep accelerators supplied with data and useful arithmetic without exhausting memory or losing time to synchronization. Which intervention helps depends on the binding compute, memory, data, or communication constraint, so profiling must precede scaling or optimization. Batch size, numerical precision, checkpointing, data loading, and parallelism must therefore be treated as coupled decisions rather than independent tuning knobs. For the systems engineer, training is where hardware and parallelism determine whether a run fits within memory, schedule, and budget, and whether the repeated optimization loop accelerates or stalls. In D·A·M terms, each precision and parallelism decision trades the mathematics of optimization against the physics of execution.



Learning Objectives

- Explain training cost asymmetry using forward, backward, optimizer-state, data, and iteration costs
- Calculate FLOPs, run time, energy, activation memory, optimizer state, and dollar cost for full training and fine-tuning
- Compare SGD, Adam, and AdamW by convergence behavior, memory overhead, and compute cost
- Design single-machine training pipelines with prefetching, overlap, batching, and checkpointed training state
- Diagnose compute-, memory-, and data-bound training bottlenecks with profiling evidence and model FLOPs utilization
- Apply mixed precision, FlashAttention, gradient accumulation, and activation checkpointing to fit accelerator memory and throughput limits
- Evaluate single-machine multi-accelerator configurations using memory, duration, communication, energy, and cost constraints, and recognize when a workload exceeds the node boundary

8.1 From One Step to a Training Run

A training run is one training step repeated millions of times, so every byte a step holds and every millisecond it spends is multiplied by the length of the run. Each step reads a batch of examples, computes a forward pass and a loss, propagates gradients backward, and updates every parameter, and the run repeats that sequence until the model reaches its quality target. The anatomy of a training step in section 7.7 traced one such step for a two-layer MNIST classifier, a smaller sibling of the three-layer network of chapter 5: 101,770 parameters, about 19.5 MFLOP per batch of 32 images by the dense-training rule (somewhat less in that chapter's exact trace, which skips the gradient of the raw input), and 1.6 MB of weights, gradients, and optimizer state. One step of GPT-2 XL over 32 sequences of 1,024 tokens costs about 295 TFLOP, roughly 15 million times the classifier's arithmetic, and its weights, gradients, and optimizer state occupy 24 GB before a single activation is stored. The framework mechanisms that made the small step possible leave every decision about the large run open: which intermediate activations to keep when high-bandwidth memory (HBM) runs out, how large a batch the memory permits, how to keep the accelerator supplied with data, and when one accelerator stops being enough. Running a model once and training it from scratch sit at opposite ends of the operational spectrum, and the size of that gap defines the discipline of training systems.

In 2019, training a GPT-2-scale model required approximately $50,000 in compute resources, whereas pretraining a frontier architecture such as GPT-4 reached an estimated $100M.[1] Although neither figure represents an audited financial disclosure, the more than three-order-of-magnitude gulf between them highlights the economic reality of deep learning. This **training cost asymmetry** reflects the gap between running a trained model on a single query and repeating the forward-backward-update loop across an entire corpus.

For GPT-2, a single forward token evaluation consumes roughly 3.00×10^9 floating-point operations, derived from standard transformer architectural accounting (Radford et al. 2019). Pretraining the full model across its WebText corpus required a cumulative budget of 1.50×10^{21} FLOPs, equivalent to approximately 167 billion token evaluations when forward and backward passes are evaluated together.

Definition 8.1: Training systems

Machine learning training systems are software-hardware systems that execute the iterative optimization loop (forward pass, loss computation, backward pass, and parameter update) to minimize a loss function over a training dataset.

[1] **Training cost scaling**: The roughly 2,000× ratio between this chapter's two cost anchors illustrates how parameter counts, training-token budgets, hardware generations, and accelerator fleets can compound; it is not a controlled comparison of disclosed training bills. OpenAI's GPT-2 report documents the model and training setup but does not disclose training cost (Radford et al. 2019). Exact GPT-4 training details were also not disclosed, so the GPT-4-class figure is explicitly an industry estimate supported by public reporting and independent infrastructure analysis (Knight 2023; Patel and Wong 2023).

1. Significance: Training holds far more state per parameter than inference. In the common mixed-precision recipe with the Adaptive Moment Estimation (Adam) optimizer, gradients and optimizer state raise the per-parameter footprint to 8× the FP16 weights an inference server stores, before any activation is counted (section 8.3.2.3 derives the multiplier). This multiplier is a major reason a model that serves on one accelerator needs several for training.
2. Distinction: Unlike inference systems, which execute a single forward pass and discard intermediate activations, standard backpropagation stores the tensors needed by the backward pass, creating a footprint that grows with model depth and batch size.
3. Common pitfall: A frequent misconception is that training failures are compute problems. A common failure is an out-of-memory (OOM) error, a memory-management problem: tensors saved for backpropagation accumulate during the forward pass and can exhaust memory before the first gradient is computed.

Three characteristics distinguish training workloads from general-purpose computing, and each demands a targeted systems intervention. First, computational intensity dominates the schedule: spreading the 1.50×10^{21} FLOPs budget over days of wall-clock time requires sustained PFLOP/s-scale throughput from hardware whose realized large-model throughput is often far below theoretical peak (Narayanan et al. 2021; Chowdhery et al. 2022), pushing the system toward higher arithmetic utilization and lower-precision numeric formats. Second, memory pressure compounds across four state tensors: storing 1.5B weights requires 6 GB in FP32, the Adam optimizer adds two state tensors per parameter that consume another 12 GB, and activation memory adds a workload-dependent footprint governed by architecture, sequence length, batch size, precision, and checkpointing. Because the combined state can exceed a single accelerator's capacity, training systems rely on gradient checkpointing,[2] which trades redundant arithmetic for reduced activation storage, and mixed-precision training,[3] which halves activation storage by casting activations to 16-bit formats while retaining FP32 master weights and optimizer moments for numerical stability.

Third, sequential data dependencies tie each gradient update to the result of the previous one, limiting how much parallelism the system can exploit across steps. Hiding the non-compute work inside each step therefore requires pipeline architectures that overlap I/O and communication with computation so that execution units do not stall waiting for input. The analysis here establishes the execution baseline on single-machine and single-node multi-GPU topologies; scaling across distributed network boundaries introduces multi-node communication fabrics and fault-tolerance protocols treated in advanced distributed systems texts.

8.1.1 Running cases: GPT-2 XL and ResNet-50

GPT-2 is the book's bandwidth-bound lighthouse. Generating one token at a time reads every weight for little arithmetic, so in inference its home regime is the bandwidth-bound one. Training changes that profile. A batch of long sequences reuses each weight across thousands of token positions, so the same model now presses on arithmetic throughput and, through activations and optimizer state, on memory capacity. The chapter uses it to test each training cost and optimization.

Lighthouse 8.1: Lighthouse example: Training GPT-2

Context: GPT-2 (1.5 billion parameters) serves as the primary case study because it is large enough to expose meaningful compute, activation, and optimizer-state constraints while remaining easier to analyze than frontier-scale models. Whether it requires distributed training depends on precision, optimizer state, activation memory, hardware capacity, and the target training time. Table 8.1 maps each model property to the systems constraint it creates:

[2] **Gradient checkpointing (activation checkpointing)**: Backpropagation retains the forward values required by each backward rule. Checkpointing saves selected values and recomputes others during backward, trading additional work for lower activation memory. The $\sqrt{N_L}$ checkpoint placement and its associated recomputation cost describe the classical schedule analyzed by T. Chen et al. (2016) rather than every checkpointing implementation. Section 8.6.5.2 works through that schedule for GPT-2's layer count.

[3] **Mixed-precision training**: Uses lower precision for selected computation and storage while retaining higher precision where accumulation or numerical range requires it. A common FP16 recipe maintains FP32 master weights and uses loss scaling to reduce gradient underflow (Micikevicius et al. 2018). BF16 ("Brain Floating Point," from Google Brain (Wang and Kanwar 2019)) matches FP32's 8-bit exponent range and therefore usually avoids dynamic loss scaling, though the exact state and accumulation precision depend on the optimizer and implementation.

↩ The feeding problem, in which data delivery must keep pace with accelerator consumption, is established in section 4.9.3.

↩ GPT-2's home regime in inference, bandwidth-bound because each generated token rereads every weight, is established in section 2.3.2.

Table 8.1: GPT-2 (1.5 Billion Parameters) Lighthouse Model Specifications: Parameter count, architecture depth, dataset size, and total training compute mapped to their systems implications. Each row pairs a model property with the engineering constraint it creates: memory footprint for weights, activation pressure from pipeline depth, I/O throughput requirements, and parallelization demand.

Property	Specification	Systems Implication
Parameters	1.5B (XL)	Requires ~3 GB (FP16) or ~6 GB (FP32) for weights alone.
Architecture	48 Layers, 1600 Dim	Deep pipeline creates heavy activation memory pressure.
Dataset	WebText (40 GB)	Preprocessing, shuffling, and delivery must sustain the selected training configuration.
Compute	~ 1.50 × 10^{21} FLOPs	Training duration depends on achieved throughput and accelerator count.

Mechanism: In dense transformer models, the arithmetic work per token is directly proportional to parameter count P. In the forward pass, each parameter participates in one multiply-accumulate operation (1 MAC = 2 FLOPs), consuming $2P$ FLOPs/token. Backpropagation computes two matrix products per parameter: one to propagate gradients backward with respect to activations ($2P$ FLOPs), and another to calculate weight gradients for the optimizer ($2P$ FLOPs). The backward pass therefore requires twice the compute of the forward pass ($4P$ FLOPs/token), yielding a total work factor of $2P + 4P = 6P$ FLOPs/token. Across a dataset of D tokens, total training work is $O \approx 6 \cdot P \cdot D$, or 1.50 × 10^{21} FLOPs.

Training GPT-2 couples this operation count with the four-term training-memory budget of equation 5.29, $M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$. Balancing execution time $T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}}$ requires orchestrating data ingestion, activation memory, and compute throughput within accelerator HBM capacity.

ResNet-50, the book's compute-bound lighthouse, carries the chapter's input pipeline. Its home regime is compute-bound because a convolution reuses each weight at every spatial position, so one 224×224 image costs about 8.2 GFLOP in the forward pass and about 24.6 GFLOP once the backward pass is added, for only 25.6 million parameters. Training it on a current data-center accelerator moves the binding term upstream. Each image must be read, decoded, and augmented on the host before the accelerator sees it, and the data pipeline (section 8.4.2) shows the accelerator consuming images faster than the host processors can prepare them. The two cases therefore cross regimes in opposite directions. GPT-2 leaves its bandwidth-bound home for arithmetic and memory capacity, while ResNet-50 leaves its compute-bound home for an input-bound one.

Every number in the chapter uses one reference machine, an NVIDIA H100 with 80 GB of HBM delivering 3.35 TB/s and a dense 16-bit arithmetic peak of 989 TFLOP/s. The H100 is a point-in-time instance of the data-center accelerator class whose design chapter 11 develops; its successors change the constants, not the reasoning.

The chapter follows GPT-2 XL's run from estimate to execution (table 8.2). The first half converts the run's operation count into time and energy, accounts for the arithmetic and bytes of one step, and assembles the pipeline that executes it, including the checkpoints that let a long run survive failure. The second half diagnoses a measured step, removes its binding term one technique at a time until GPT-2 XL trains on a single accelerator, and closes with when and how to add accelerators.

Table 8.2: Chapter Map: Each section answers one question about the training run and names the term of the training-time law, or the memory constraint, that its techniques move. Memory does not appear in the law; it decides whether a configuration runs at all.

Section	Question it answers	Law term or constraint it moves	Running case
Training-Time Law	How long, and how much energy, a run takes	All terms of $T_{\text{train}} = O/(R_{\text{peak}} \times \eta_{\text{hw}})$	GPT-2 XL run on one H100
Computation and Memory Costs of Training	What one step costs in FLOPs and in bytes	O per step; the memory constraint	GPT-2 XL step; Llama 3 8B fine-tuning

Section	Question it answers	Law term or constraint it moves	Running case
Pipeline Architecture	How data, compute, updates, and saved state flow through a step	η_{hw} lost between stages	ResNet-50 input feed; GPT-2 XL step
Identifying Bottlenecks	Which stage limits a measured step	η_{hw}, diagnosed	GPT-2 XL profile
Pipeline Optimizations	Which technique removes the binding term	R_{peak} (precision), η_{hw} (overlap, IO), memory (accumulation, checkpointing)	GPT-2 XL on one H100
Scaling Training Systems	When to add accelerators, and what synchronization costs	$R_{peak} \times \eta_{hw}$ across devices, less communication	GPT-2 XL on an eight-H100 node

8.2 Training-Time Law

Every later estimate in the chapter starts from how long the run takes, and the training-time law fixes that first. GPT-2 XL's run needs about 1.50×10^{21} FLOPs. Converting that operation count into days on an accelerator, and every loss of time into a factor engineering can recover, requires a decomposition of training time into its physical constituents in which wasted time appears as a lower utilization factor.

The iron law of ML systems (principle 3) supplies that decomposition directly once specialized to training. In its multiplicative D·A·M form, a training run processes D tokens (Data) at roughly $6P$ forward-and-backward FLOPs per token (Algorithm), for $O = 6 \cdot P \cdot D$ total operations, at a realized machine pace of $1/(R_{peak} \times \eta_{hw})$ seconds per FLOP (Machine). This training-time form of the iron law, which this chapter shortens to the training-time law, keeps that macroscopic product as the organizing equation so that every optimization technique maps to one of its factors.

Definition 8.2: The training-time law

The training-time law expresses training time through the workload's operation count and its effective achieved throughput:

$$T_{train} = \frac{O}{R_{peak} \times \eta_{hw}} \tag{8.1}$$

Here O counts useful model forward-and-backward FLOPs, excluding rematerialization. Additional execution work is reflected in effective throughput. The factor η_{hw} is an effective end-to-end utilization factor, the achieved model-operation rate divided by the relevant hardware peak. Data stalls, communication, synchronization, launch overhead, and inefficient kernels all reduce this factor. Prefetching and overlap can hide part of those costs but do not guarantee their removal.

1. Significance: In the D·A·M taxonomy, these terms map directly to the three multiplicative factors. The numerator $O = (\text{Samples/Task}) \times (\text{FLOPs/Sample})$ is governed by Data (training token count D) and Algorithm (model architecture or compression methods such as pruning and distillation), while the Machine factor $\text{Seconds/FLOP} = 1/(R_{peak} \times \eta_{hw})$ combines hardware execution units at the chosen numerical precision (R_{peak}) with effective systems utilization (η_{hw}, the primary engineering target, where Narayanan et al. (2021) measured 44 to 52 percent of theoretical peak throughput across GPT models trained on A100 clusters). Chapter 10 defines the compression techniques; here they serve only to show where such methods enter the accounting.
2. Distinction: The training-time law (equation 8.1) is the macroscopic D·A·M product form of the iron law, folding every non-arithmetic delay into η_{hw}. When η_{hw} is low, the unpacked three-term iron law exposes data movement (D_{vol}/BW), kernel computation, and fixed latency (L_{lat}) separately to diagnose which underlying bottleneck is responsible.

3. Common pitfall: A frequent misconception is that η_{hw} is fixed by hardware. It is a pipeline property, so diagnosing which factor dominates requires profiling rather than reading hardware specs.

Equation 8.1 reveals three levers for improvement: reduce total operations through data curation or algorithmic innovation, increase peak throughput through hardware selection and precision formats, or improve utilization through pipeline orchestration. Memory does not appear in the law at all. It acts as a feasibility constraint, because a configuration whose state does not fit runs at no speed. The chapter map (table 8.2) shows which section works on which factor, and table 8.8 places each optimization technique against the factor it moves, with its memory effect listed separately.

The training-time law focuses on execution efficiency, how fast the hardware processes a given workload. It does not predict how data quality, dataset size, or curriculum design determines the total operations O needed to reach a target accuracy; that workload must be supplied or estimated separately. A cleaner dataset or a better data mix can reduce the number of epochs required, shrinking O without touching hardware at all. Holding the workload fixed, the question is how to execute it as fast as possible.

↪ Data selection, which shrinks total operations (O) by keeping the examples that return the most learning per unit of compute, is developed in section 9.1.

Actual training throughput remains below the arithmetic peak whenever kernels, data delivery, or communication leave resources idle. Scaling to multiple accelerators introduces additional communication overhead, a trade-off quantified in section 8.7.

For GPT-2 XL on the reference H100, the law becomes a schedule. At the accelerator's 989 TFLOP/s dense 16-bit peak and a utilization η_{hw} of 50 percent, the high end of what large training runs sustain (Narayanan et al. 2021), the run takes about 35 days on one accelerator, about 5.1 million steps of 0.60 s each over 32 sequences of 1,024 tokens. Time converts directly into energy. Energy is power integrated over time, $E_{total} = P_{device} \times T_{train}$ at constant power, so at the H100's 700 W thermal design power the run draws about 590 kWh at the accelerator alone, about 417 J per step. Every technique that shortens the run at constant power removes energy in the same proportion, which is why the utilization factor matters to an energy budget as much as to a schedule.

↩ The iron law's first estimate of a training run, the same operation-count-to-days conversion applied to a GPT-3-class model on an accelerator cluster, is established in section 1.7.

The same conversion scales to runs far larger than GPT-2. A 70B-parameter Llama 2 model trained on 2T tokens across a 1,024-accelerator H100 cluster at the same utilization takes about 19.2 days and $1.89M of rented accelerator time, while buying that cluster costs as much as about 16.3 such runs before networking, facilities, power, and staff are counted (section 15.4.2 develops the full cost model).

Not every training workload is bound by arithmetic. DLRM, the book's capacity-bound lighthouse, holds 100B to 10T parameters, mostly in embedding tables, so its first training problem is fitting those tables in memory at all, whereas dense models such as GPT-2 meet arithmetic and activation-memory limits first.

Checkpoint 8.1: The physics of training

Training speed is governed by the utilization of hardware peaks.

Utilization gap

- ☐ **Peak vs. real**: why does sustained end-to-end utilization remain below the arithmetic peak?
- ☐ **Utilization losses**: which sources of lost time does the definition of η_{hw} fold in, and which of them can overlap hide rather than remove?
- ☐ **Order of magnitude**: a frontier run needs on the order of 10^{23} FLOPs at a utilization η_{hw} well below 1. Estimate which lever shortens wall-clock time more, doubling R_{peak} or doubling η_{hw}, and why, in practice, the two levers differ in cost and reachability even though the training-time law treats them symmetrically.

Each milestone in the history of deep learning moved the binding constraint from one factor of this law to another. Backpropagation, popularized for multilayer networks in 1986 (Rumelhart et al. 1986), made the operation count O of a deep network computable at all. AlexNet's five-to-six-day

ImageNet run on two GPUs in 2012 (Krizhevsky et al. 2012) showed that this arithmetic mapped onto the far higher R_{peak} of graphics hardware. Transformers from 2017 onward (Vaswani et al. 2017) concentrated the work into large dense matrix multiplications with large activation footprints. GPT-3's roughly 3.14×10^{23} FLOPs in 2020 (Brown et al. 2020) made utilization (η_{hw}) the difference between an affordable run and an unaffordable one, and IO-aware kernels such as FlashAttention later raised η_{hw} by attacking attention's memory traffic.

The history follows the bitter lesson. Backpropagation supplied a general recipe, and each later step let that recipe absorb more computation, more parameters, and more data. If capability tracks the computation a general method can absorb, then every point of η_{hw} lost to stalls, and every gigabyte of memory that forces a smaller model or batch, is capability forgone at a fixed budget. The first obligation of a training system is therefore to convert its compute budget into useful operations as completely as the hardware allows.

↩ The bitter lesson, that general methods which absorb more computation outperform encoded human expertise, is established in section 1.4.

The law takes a run's operation count and its achieved rate as given, and both come from the costs of individual operations. Every bottleneck traces back to specific mathematical operations: compute-intensive matrix multiplications governed by peak throughput (Goto and Geijn 2008), low-intensity normalization layers governed by memory bandwidth, and optimizer state tensors that inflate the static memory footprint. Modern accelerators have compute-to-bandwidth ratios (roofline ridge points) that far exceed the arithmetic intensity of many neural network layers, leaving memory-bound kernels starved for operands (Hennessy and Patterson 2017). Software frameworks expose these physical boundaries as configurable switches, such as mixed-precision autocasting contexts or activation checkpointing flags, and each switch must be set against the machine's measured limits rather than tuned blindly. Optimizing a training system therefore begins with what each operation costs in FLOPs and bytes.

8.3 Computation and Memory Costs of Training

In a model such as GPT-2, the tensors participating in a single training step far exceed the 30 to 50 MB of on-chip SRAM available on a modern accelerator, so executing a layer's matrix multiplication $\mathbf{Z} = \mathbf{AW}$ requires staging matrix tiles continuously across two levels of the physical memory hierarchy. Off-chip global device memory (HBM or GDDR) offers large storage capacity (typically 16 to 96 GB) to hold weights, optimizer states, and large activation tensors, whereas on-chip SRAM (registers and shared memory) sits directly on the processor die adjacent to the arithmetic execution units. Because the external memory bus bounds HBM traffic to roughly 1.5–3.5 TB/s while on-chip SRAM delivers roughly 15–20 TB/s across its 30 to 50 MB capacity, performance depends on how much work a kernel performs for every byte it fetches from HBM.

Dense matrix multiplications have high arithmetic intensity, amortizing the cost of moving bytes from HBM into SRAM over many operations and enabling specialized execution units such as Tensor Cores[4] to achieve high compute utilization. In contrast, surrounding elementwise operations (activations, normalizations, and residual additions) have low arithmetic intensity and remain bounded by memory bandwidth, creating opportunities for kernel fusion to eliminate redundant round-trips to HBM. Chapter 5 established *what* neural network operations compute and *why* they enable learning; the systems question is *what* they cost in FLOPs, memory, and bandwidth when those operations execute at scale.

4 | **Tensor Core**: A specialized matrix-multiply-accumulate unit that supports reduced-precision inputs and higher-precision accumulation. Supported tile shapes, alignment constraints, and peak-throughput ratios vary by architecture, dtype, sparsity mode, and library kernel.

Four dimensions structure this cost analysis: the FLOP counts of the matrix operations that dominate dense neural-network training, the optimizer state (the fixed bytes per parameter that every update reads and writes), the activation memory (the batch-dependent bytes that backpropagation must keep), and the arithmetic intensity that classifies each operation as compute-bound or memory-bound against the hardware's ridge point. Together, these four dimensions provide the vocabulary for analyzing the computational intensity, memory pressure, and data dependencies introduced in section 8.1.

8.3.1 Forward and backward computation

Every forward matrix multiplication in a training step must be followed by two backward matrix multiplications (one to propagate error gradients to the preceding layer and one to compute weight gradients) while intermediate activations sit in device memory waiting for their backward consumer. Although each layer applies only a linear transformation and a nonlinear activation, repeating

that forward-and-backward pair across dozens of layers and millions of steps dominates modern training infrastructure. The introduction of backpropagation[5] by Rumelhart et al. (1986) and the development of efficient matrix computation libraries such as Basic Linear Algebra Subprograms (BLAS)[6] (Dongarra et al. 1988) laid the groundwork for modern training architectures.

Forward propagation, in its simplest case, involves two operations: matrix multiplication and activation function application. Matrix multiplication implements the linear transformation at each layer. At layer ℓ, the computation can be written in row-vector form, with one example per row:

$$\mathbf{A}^{(\ell)} = f\left(\mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}\right)$$

where:

- $\mathbf{A}^{(\ell-1)}$ represents the activations from the previous layer (or the input layer for the first layer), with each row being a sample in the batch,
- $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ is the weight matrix at layer ℓ, which contains the parameters learned by the network,
- $\mathbf{b}^{(\ell)}$ is the bias vector for layer ℓ,
- $f(\cdot)$ is the activation function applied elementwise (for example, ReLU, sigmoid) to introduce nonlinearity.

8.3.1.1 Matrix operations

Section 5.4.2.2 established that forward propagation reduces to chains of matrix multiplications. Every architecture also shares a small set of building blocks, from dense projections and residual paths to normalization, each with its own cost to the machine. Training amplifies these patterns: each operation executes not once but billions of times, and each forward pass is paired with a backward pass that costs roughly twice the forward pass, tripling the work per step. Understanding which matrix operations dominate, and how their shapes change between forward and backward passes, reveals *why* specific system designs and optimizations emerged for training.

The predominance of matrix multiplication has shaped both linear algebra software and accelerator architecture. Early implementations relied on general-purpose CPU libraries, but modern training scale requires tiled, accelerator-native libraries such as cuBLAS (NVIDIA 2024a) and oneDNN that partition operands to fit cache hierarchies and feed systolic arrays.

This dense computational workload motivates blocked algorithms that parallelize across thousands of execution cores while aligning with the memory access patterns of forward and backward passes. Because backpropagation requires simultaneous access to weight matrices, incoming gradients, and stored activations, hardware architects have prioritized high memory bandwidth alongside raw arithmetic throughput. Evaluating one layer of the GPT-2 lighthouse model illustrates the scale and distribution of these operations.

Napkin Math 8.1: GPT-2 attention layer computation

Each GPT-2 layer performs attention computations that exemplify dense matrix multiplication demands. For one GPT-2 transformer layer (all heads combined) with batch size $B = 32$, sequence length $S = 1024$, and hidden dimension $d = 1600$:

Query, Key, Value Projections (the three linear transformations that create attention inputs, 3 separate matrix multiplications):

$$O_{\text{QKV}} = 2 \times 3 \times (B \times S \times d \times d)$$
$$= 2 \times 3 \times (32 \times 1024 \times 1600 \times 1600) \approx 503 \text{ billion FLOPs}$$

Attention matmuls ($\mathbf{QK}^T$ and the product of the attention probabilities with $\mathbf{V}$):

$$O_{\text{QK}} = 2 \times B \times N_{\text{heads}} \times S \times S \times d_{\text{head}} = 107.4 \text{ billion FLOPs}$$
$$O_{\text{AV}} = O_{\text{QK}}; \quad \text{pair total} \approx 214.8 \text{ billion FLOPs}$$

Output projection (attention output $\rightarrow$ hidden):

$$O_{\text{out}} = 2 \times B \times S \times d \times d \approx 168 \text{ billion FLOPs}$$

[5] **Backpropagation provenance**: The algorithm was independently derived by Linnainmaa (1970) for automatic differentiation of computer programs and by Werbos (1974) in a Harvard PhD thesis on economic modeling, over a decade before Rumelhart et al. (1986) popularized it for neural networks. A framework's backward pass traverses the computational graph in reverse topological order and applies the chain rule; independent subgraphs may expose parallel work during that traversal.

[6] **BLAS (basic linear algebra subprograms)**: The original BLAS specification standardized reusable Fortran-callable vector operations (Lawson et al. 1979); later BLAS extensions added matrix-vector and matrix-matrix routines, giving the familiar Level 1, Level 2, and Level 3 hierarchy (Dongarra et al. 1988). Training is dominated by Level 3 operations precisely because their high arithmetic intensity, $\mathcal{O}(N)$ FLOP/byte, can sustain high compute utilization. Frameworks commonly dispatch dense matrix multiplication to optimized libraries such as cuBLAS and oneDNN or to compiler-generated kernels.

↩ The building blocks every architecture shares, and what each costs the machine, are established in section 6.9.3.

Feed-Forward Network (Two linear transformations with expansion factor 4):

$$O_{\text{FFN}} \approx 16 \times B \times S \times d^2$$

Computation Scale

- Per-layer forward total (QKV 503 + attention 214.8 + output proj 168 + FFN 1,342 billion): ~2,228 GFLOP
- With 48 layers in GPT-2: ~320.8 TFLOP per training step, forward and backward. This exceeds the 295 TFLOP of the six-FLOPs-per-parameter rule used elsewhere in the chapter mostly because it adds the attention score and weighted-sum products, about 31 TFLOP per step, which grow with the square of sequence length and which that rule leaves out; the rule's parameter count also includes embedding weights that this per-layer tally omits
- Across this illustrative 50,000 steps scenario: ~16,042 PFLOP

Systems insight: The reference H100 (989 TFLOP/s dense peak with Tensor Cores, 67 TFLOP/s without) would require 0.32 s for the modeled attention-plus-FFN training step at 100 percent utilization. If the target is a step of about 200 ms, ideal arithmetic alone needs about two accelerators, and at the 30 to 50 percent utilization large runs sustain the requirement grows to 4 to 6 before communication and pipeline overhead are counted.

These FLOP counts are the compute term of the iron law made concrete, and they explain why training cost scales as a predictable function of model architecture and sequence length rather than as an unpredictable emergent property.

8.3.1.2 Matrix-vector and batched operations

Not all neural network layers execute as dense matrix-matrix multiplications. Normalization layers, bias additions, and autoregressive recurrent updates execute matrix-vector operations, which use each loaded weight once and leave a massively parallel accelerator waiting on memory. Batching turns those matrix-vector products (GEMV) into general matrix multiplications (GEMM) that reuse each loaded weight across B examples, the same conversion that raised the running dense network's arithmetic intensity at batch 32 in section 5.4.2.4. For training, the trade-off is an immediate expansion of working memory: every additional sample in the batch scales the intermediate activation volume that must reside in device memory during backpropagation. Hardware design directly mirrors this progression. Google's first Tensor Processing Unit (TPU) integrated dedicated two-dimensional systolic matrix units that required batched execution to achieve efficiency (Jouppi et al. 2017), while large-scale models such as GPT-3 use distributed clusters specifically to execute the massive GEMM patterns exposed by large global batches (Brown et al. 2020).

Matrix multiplication's dominance also explains the shape of training hardware. Its independent output elements map onto thousands of parallel arithmetic lanes, dedicated matrix units such as Tensor Cores run lower-precision matrix products several times faster than scalar arithmetic, and blocked computation keeps reused operands in on-chip memories rather than refetching them from HBM. Mixed-precision training (section 8.6.3) exploits the second property, and the realized gain of any of them still depends on the nonmatrix work, memory traffic, and input and communication costs around the matrix products.

Matrix multiplications dominate training compute, but neural networks require more than linear transformations. Between each layer's matrix operations, activation functions introduce the nonlinearity that enables networks to learn complex patterns. These functions appear computationally trivial compared to matrix multiplication, yet their implementation characteristics affect training efficiency in ways that matter at scale.

↪ Why accelerators devote their silicon to dense matrix units, and how those units are fed, is developed in section 11.4.

8.3.1.3 Activation functions

Section 5.3.2.2 established what sigmoid, tanh, ReLU, and softmax compute, why the S-curves saturate, and why a ReLU comparison costs far less silicon than an exponential. At training scale, what matters is that their arithmetic rarely sets throughput. A standalone elementwise activation performs only 1 to 4 FLOPs per element while moving 8 bytes (an FP32 read and write), roughly

0.125–0.5 FLOP/byte, so its latency is set by memory bandwidth rather than by which function it computes. GPT-2's activation, the Gaussian Error Linear Unit (GELU), shows what remains of the arithmetic choice, an exact form against a cheaper approximation.

> **Systems Perspective 8.1: The GELU activation choice**
>
> Beyond the foundational activation functions (Sigmoid, Tanh, ReLU), modern architectures increasingly adopt smoother alternatives. GPT-2 uses a GELU activation (Radford et al. 2019), the Gaussian Error Linear Unit introduced by Hendrycks and Gimpel (2016) and often implemented with the common tanh-based approximation from that formulation. Its exact definition is:
>
> $$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$$
>
> where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution.
>
> GELU applies a smooth, input-dependent scaling rather than ReLU's hard threshold. It is deterministic despite its formulation in terms of a Gaussian cumulative distribution. Whether it improves model quality relative to another activation is an empirical property of the architecture and training recipe, not a universal guarantee.
>
> Exact GELU uses `erf`, while common implementations use a tanh-based approximation such as listing 8.1. The approximation changes arithmetic cost and introduces a small numerical difference from the exact function. Its end-to-end cost depends mainly on whether the compiler fuses the activation into the preceding matrix multiplication, which removes the memory round-trip that dominates a standalone activation kernel.

Listing 8.1: GELU Approximation: Fast approximation avoids expensive erf() computation while preserving activation properties.

```
# Fast GELU approximation used in production systems
# Avoids expensive erf() computation while
# preserving activation properties
gelu_approx = (
    0.5 * x * (1 + tanh(sqrt(2 / pi) * (x + 0.044715 * x**3)))
)
```

Forward-pass matrix multiplications dominate floating-point operations, while activation functions stress memory bandwidth. Transforming these forward activations into model convergence requires propagating error gradients backward through the graph to update parameter tensors. That optimization step introduces state storage, update scheduling, and numerical stability constraints that govern training efficiency.

↩ The memory wall that bounds elementwise activation kernels, and kernel fusion as the framework's response to it, are established in section 7.3.2.

8.3.2 Optimizers and their memory

Every parameter update reads gradients, mutates persistent state tensors in High Bandwidth Memory (HBM), and writes updated weights back across the memory bus. Because modern optimizers keep one or two auxiliary state values per parameter, optimizer state often consumes more HBM capacity than the weights themselves while executing as a bandwidth-bound elementwise pass. In the D·A·M taxonomy, the choice of optimization algorithm couples the mathematical trajectory (Algorithm) directly to the working-set footprint and memory traffic (Machine).

8.3.2.1 Gradient-based optimization methods

At hardware scale, updating parameters along the steepest-descent direction collides with two physical limits: HBM capacity restricts how many activations and optimizer states remain resident, and memory bandwidth restricts how rapidly parameters update once gradients arrive. The mathematical formulation of gradient descent assumes modest networks on a single device, where neither limit binds.

↩ Gradient descent and its parameter-update rule are established in section 5.4.4.

Gradient descent In a training system, each gradient descent iteration is a chain of dependent operations (forward pass, loss, backward pass, parameter update), none of which can start before its predecessor finishes. Full-batch gradient descent, which computes each update from the entire training set, need not hold the dataset in memory at once, because examples can stream through the accelerator in micro-batches; its cost is time rather than memory. With one million training examples, every update waits for a million examples of forward and backward work, and the parameters stay frozen throughout, reducing the rate at which the model can learn from data.

Mini-batch processing Mini-batch gradient descent (definition 5.2) sits between full-batch and single-example updates, computing each update over a batch small enough to fit in memory and large enough to keep the accelerator busy (Dean et al. 2012). Single-example updates (stochastic gradient descent with a batch of one) return dense layers to the bandwidth-limited matrix-vector regime of section 8.3.1.2 and rewrite every parameter once per example. Averaging over B independent examples shrinks the standard deviation of the gradient estimate approximately as $1/\sqrt{B}$, and the batch size becomes a key system parameter, influencing both computational efficiency and memory requirements.

The choice of batch size B directly governs accelerator memory allocation. The partial decomposition in equation 8.2 separates fixed parameter and gradient storage from the batch-dependent activation working set; optimizer states, any master weights, and runtime workspaces must also enter the full budget:

$$\text{Memory Required} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{activations}}, \qquad M_{\text{activations}} \propto B \tag{8.2}$$

Because the activation term scales with B while parameter and gradient memory stay fixed, doubling the batch doubles the activation working set, and a model that fits comfortably at a small batch can exhaust the 80 GB of HBM on the reference H100 once the batch grows. Section 8.4.3.2 works through an illustrative ResNet-50 budget.

Sharing each loaded weight across the batch lets a larger batch move dense feedforward and attention projections toward the compute-bound regime; shape and the hardware ridge point determine whether compute throughput becomes limiting. Throughput and convergence follow separate curves, however. Goyal et al. (2017) trained ResNet-50 on ImageNet at $B = 8{,}192$ by scaling the learning rate linearly with batch size and adding a warmup period, a configuration-specific recipe rather than evidence of a universal critical batch size; its useful range depends on the model, data, optimizer, and target accuracy.

When the batch size necessary to saturate execution units or ensure stable convergence exceeds physical memory capacity, systems decouple the physical micro-batch size (B_{micro}) from the algorithmic batch size by accumulating gradients over several micro-batches before invoking the optimizer, which keeps peak activation storage to that of one micro-batch. Section 8.6.5 develops this gradient accumulation and the conditions under which it reproduces the large-batch gradient.

⇄ **Contrast:** Training packs large batches to amortize weight transfers and saturate arithmetic units, whereas online serving inverts the priority to per-request latency under stochastic arrivals, as analyzed in section 13.1.

8.3.2.2 Adaptive and momentum-based optimizers

Basic mini-batch stochastic gradient descent (SGD) applies a uniform scalar learning rate to every parameter tensor, which can stall training when loss curvature differs by orders of magnitude across dimensions. Momentum smooths updates using directional gradient history, RMSprop adapts individual step sizes using squared-gradient magnitudes, and Adam combines first-moment tracking with adaptive second-moment scaling (Kingma and Ba 2015). While these algorithmic enhancements can reduce the total optimization steps required to converge, they do so by introducing persistent auxiliary state that directly increases accelerator memory pressure and memory-bus traffic.

Momentum-based methods Momentum methods[7] (Polyak 1964) address SGD's oscillation problem by accumulating a velocity vector across iterations, damping high-frequency oscillations along steep ravines. In hardware, this smoothing requires allocating and maintaining a persistent velocity tensor matching the exact shape and precision of the model parameters. Parameters and this auxiliary state together occupy $2\times$ the parameter footprint before gradients and activations are counted.

Adaptive learning rate methods While momentum smooths gradient direction, it does not address the different scales of gradients across parameters. RMSprop[8] solves this by maintaining a moving average of squared gradients for each parameter, automatically reducing step sizes for parameters

7 | **Momentum**: Borrowed from physics, where momentum (mass × velocity) describes an object's tendency to continue moving. The metaphor explains the design because accumulated velocity smooths noisy gradients and can reduce the iterations needed for convergence. The systems cost is one additional velocity value per parameter, while Adam maintains two moment values per parameter.

8 | **RMSprop**: Proposed by Geoffrey Hinton in Lecture 6e of his 2012 Coursera course and never published in a peer-reviewed paper. Its per-parameter squared-gradient average is what Adam inherits as its second moment v_t, directly contributing to Adam's $3\times$ memory overhead described in section 8.3.2.3.

with historically large gradients. Adapting step sizes per parameter requires maintaining a running average of squared gradients s_t, imposing a $2\times$ static memory overhead equivalent to momentum. Furthermore, computing this running average introduces elementwise square and square-root kernels during each parameter update.

Adam optimization Adam[9] combines the benefits of both momentum and RMSprop: momentum's gradient smoothing addresses noisy updates, while RMSprop's adaptive scaling handles parameter-specific step sizes. This combination maintains two moving averages for each parameter. Here m_t and v_t are the first- and second-moment buffers, β_1 and β_2 are their decay factors, η is the learning rate, and ϵ is the numerical-stability constant in the denominator:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla\mathcal{L}(\theta_t)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla\mathcal{L}(\theta_t))^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$$

Maintaining both moment buffers triples the parameter-plus-optimizer-state footprint: the training system must allocate two persistent state tensors (m_t and v_t) alongside the parameter tensors. For a 100M-parameter model, those auxiliary vectors alone add 800 MB beyond weight storage.

[9] **Adam (adaptive moment estimation)**: The two moving averages are the first moment (momentum) and second moment (uncentered variance) of the gradients, stored for every model parameter. For a 7B model, Adam's FP32 moment tensors alone consume 56 GB; the full standard mixed-precision training state before activations is 112 GB once FP16 weights, FP16 gradients, FP32 master weights, and Adam moments are counted together.

8.3.2.3 System cost of optimizer state

Auxiliary state memory increases progressively from SGD (zero auxiliary state) through Momentum and RMSprop (one state vector per parameter) to Adam (two state vectors per parameter), as quantified in table 8.3. These memory footprints establish a trade-off against convergence rate.[10] Adam often achieves target loss in fewer optimization steps, but its static memory footprint restricts the batch size that fits in accelerator HBM and increases memory-bus traffic on every update. At GPT-2 scale, this overhead becomes a first-order memory constraint.

[10] **Convergence**: A run reaches its stopping criterion when the selected training or validation metric no longer improves enough to justify additional work. The required steps depend on the model, data, optimizer, schedule, batch size, and target quality. Fewer steps can reduce wall-clock time and cost, but adaptive optimizers add state whose memory and traffic must be included in the system budget.

Table 8.3: Optimizer Memory Footprint: Different optimization algorithms impose varying auxiliary-state costs due to the storage of intermediate values like velocities and squared gradients. The multiplier row counts parameters plus optimizer auxiliary state and excludes gradients and activations; full training memory must add those terms explicitly. These trade-offs govern resource-constrained deployments and large-scale model training.

Property	SGD	Momentum	RMSprop	Adam
Memory Overhead	None	Velocity terms	Squared gradients	Both velocity and squared gradients
Parameter + optimizer state	$1\times$	$2\times$	$2\times$	$3\times$
Dense State Access	Parameters and gradients	Adds velocity	Adds squared-gradient state	Adds first- and second-moment state
Update Work	Lowest auxiliary work	Adds momentum update	Adds adaptive scaling	Adds moment updates and bias correction
Hardware Efficiency	Varies	Varies	Varies	Varies
Convergence Behavior	Varies	Varies	Varies	Varies

Three multipliers for Adam's memory appear in this chapter, and they differ only in what they count and what they divide by. The table's $3\times$ counts FP32 weights plus two FP32 moments against the FP32 weights alone, 12 bytes per parameter against 4. Adding FP32 gradients gives the 16 bytes per parameter of plain FP32 Adam. The standard mixed-precision recipe also lands at 16 bytes (2 for FP16 weights, 2 for FP16 gradients, 4 for FP32 master weights, 8 for the moments), and it is that same 16 bytes, measured against the 2 bytes of FP16 inference weights, that the definition of training systems reports as 8×.

The costs quantified in table 8.3 create a design tension. Adam's two moment buffers may improve optimization for a given workload, but their memory can reduce the model size or batch size that fits on an accelerator. AdamW[11] (Loshchilov and Hutter 2019) decouples weight decay from Adam's

[11] **AdamW (Adam with decoupled weight decay)**: Standard ℓ_2 regularization enters the gradient and is therefore rescaled by Adam's adaptive update. AdamW applies weight decay as a separate parameter update, restoring the intended distinction between adaptive gradient scaling and decay (Loshchilov and Hutter 2019). It uses the same moment-buffer structure as Adam, though its effect on model quality remains workload dependent.

adaptive gradient update without adding another per-parameter state tensor.

Napkin Math 8.2: GPT-2 optimizer memory requirements

A representative GPT-2 XL training configuration uses the Adam optimizer with five hyperparameters:

- $\beta_1 = 0.9$ (momentum decay)
- $\beta_2 = 0.999$ (second moment decay)
- Learning rate: Warmed up from 0 to 2.5e-4 over first 500 steps, then cosine decay
- Weight decay: 0.01
- Gradient clipping: Global norm clipping at 1.0

Math:
For GPT-2's 1.5B parameters in FP32 (4 bytes each), the memory breaks down across four components:

- Parameters: 1.5B $\times$ 4 bytes = 6 GB
- Gradients: 1.5B $\times$ 4 bytes = 6 GB
- Adam State (m_t, v_t): 1.5B $\times$ 8 bytes = 12 GB
- Total static memory: 24 GB

This static training state alone, 24 GB for GPT-2, takes about 28 percent of an H100's 80 GB before any activation is stored, and how much room the activations need depends on batch size and sequence length. The activation notebook below adds the batch-dependent term at batch 32, and the pipeline optimizations then remove it piece by piece until the walkthrough (section 8.6.6) fits the step on one accelerator.

System Decisions Driven by Optimizer

1. Mixed precision training (FP16) reduces operation precision but requires keeping FP32 master weights, so the static footprint stays at ~24 GB, the same 16 bytes per parameter as FP32 Adam.
2. Gradient accumulation (section 8.6.5) builds an effective batch of $B = 512$ from micro-batches that fit in memory.

Systems insight: This illustrative configuration shows how Adam's moment buffers constrain capacity. Any convergence benefit relative to SGD or momentum must be measured under a matched model, data, schedule, batch size, and stopping criterion.

8.3.2.4 The optimizer step

Optimizer memory, once quantified, fixes what each update must read and write. In the training loop, `optimizer.zero_grad()` and `optimizer.step()` bracket each update. Gradients accumulate across `backward()` calls until they are cleared, and `step()` consumes them and mutates the optimizer's persistent state (section 7.7 traces both calls through the framework). The accumulation behavior later becomes useful for building large effective batches, but only when the loop manages that boundary deliberately. For Adam optimization, `step()` performs momentum estimation, squared-gradient tracking, bias correction, and the parameter update, and algorithm 8.1 makes that hidden state explicit so the memory cost remains visible rather than disappearing behind an API call.



Parameters (P) and gradients (G) take one share each, and Adam's moment buffers m and v add two more in this simplified static-state view.

Algorithm 8.1 Adam parameter update (one optimizer step)

Require: gradient $g_t = \nabla \mathcal{L}(\theta_t)$; step t; rate η; decays β_1, β_2; constant ϵ
Ensure: updated parameter θ_{t+1}; moment buffers m_t, v_t carried to the next step
1: $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1)\, g_t$ ▷ *first moment (momentum)*
2: $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$ ▷ *uncentered second moment*
3: $\hat{m}_t \leftarrow m_t/(1-\beta_1^t)$; $\hat{v}_t \leftarrow v_t/(1-\beta_2^t)$ ▷ *bias correction*
4: $\theta_{t+1} \leftarrow \theta_t - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$ ▷ *parameter update*

Steps 1 and 2 keep two persistent moment buffers per parameter, so parameters plus optimizer state reach roughly $3\times$ the parameter memory before gradients, activations, and FP32 master weights are counted. Beyond capacity, the optimizer step imposes a severe memory-bandwidth bottleneck. The update consists of elementwise operations with an arithmetic intensity below 1 FLOP/byte. In FP32, updating each parameter requires reading four tensors ($g_t, \theta_t, m_{t-1}, v_{t-1}$) and writing three ($m_t, v_t, \theta_{t+1}$), streaming 28 bytes across the memory bus for roughly a dozen arithmetic operations. Because matrix execution units (such as Tensor Cores) sit idle during this memory-bound phase, high-performance training systems use fused optimizer kernels to combine moment updates, bias correction, and weight writes into a single kernel launch per parameter group, minimizing round-trips through device memory.

8.3.2.5 Reducing optimizer memory

Because optimizer tensors remain resident between updates, reducing their footprint can make a larger micro-batch feasible even when the optimizer is not the longest phase of the step. At billion-parameter scale, optimizer state becomes a first-order, batch-independent term in the memory budget. As quantified in the GPT-2 worked example (section 8.3.2.3), a 1.5B model requires 12 GB for Adam optimizer state alone in FP32, in addition to parameters and gradients, before accounting for activations. This challenge has motivated memory-efficient optimizer variants: Adafactor factorizes second-moment state (Shazeer and Stern 2018), 8-bit optimizers quantize optimizer statistics (Dettmers et al. 2022), and GaLore computes updates in a low-rank space (Zhao et al. 2024).

To attribute a memory saving to the optimizer, hold the model, batch size, precision, and activation policy fixed and compare the complete memory budget. Low-rank projection buffers and other temporary state belong in that budget. A smaller optimizer-state term does not guarantee that the full training recipe fits, nor that it reaches the same validation quality in less time.

8.3.2.6 Learning rate schedules

The framework's learning rate scheduling hook changes the optimizer's trajectory without adding per-parameter state. It adjusts the learning rate η during training, letting the system shape convergence behavior while leaving the underlying optimizer equations intact.

Schedules such as cosine annealing (Loshchilov and Hutter 2017), exponential decay, or step-wise reductions implement that trajectory by changing the step size over time. A scheduler tracks lightweight scalar state (current step, epoch, warmup phase) that must be serialized into checkpoints alongside optimizer states to resume the schedule (section 8.4.5). Reproducing the training trajectory also requires the relevant random-number and data-position state and compatible execution settings. This separation lets the systems engineer combine base optimization algorithms (SGD, Adam) with scheduling strategies (cosine annealing, linear warmup) without modifying the underlying tensor update rules.

8.3.2.7 Time to target quality

A faster update is valuable only if the recipe reaches the required validation quality sooner. Optimizer state and numerical precision affect the price of each update, while the optimizer and schedule affect how many updates the run needs. Both quantities enter a time-to-target decision.

Consider two precision-and-optimizer recipes for the same model, dataset, effective batch, machine, and validation target. Both fit in memory. Table 8.4 supplies hypothetical observations for their first accepted checkpoint and calculates the total elapsed time from them.

Table 8.4: Illustrative Time-to-Quality Comparison: Step durations, target-reaching update counts, and evaluation/checkpoint overhead are supplied observations for a hypothetical workload, not benchmark results. Totals multiply updates by end-to-end step time and add overhead excluded from that step time.

Quantity	Recipe A	Recipe B
End-to-end update time	2 s	1.5 s
Updates to the same validation target	6,000	10,000
Update time × update count	200 min	250 min
Separate evaluation and checkpoint time	10 min	10 min
Total time to accepted checkpoint	210 min	260 min

Recipe A reaches the target sooner. At its shorter step duration, Recipe B would tie A's total time if it reached the target at 8,000 updates under the stated overhead. Its assumed 10,000 updates exceed that budget. On the same allocated machine, shorter total time also means fewer billed accelerator-hours.

A profile measures the cost of an update; validation measures how many updates the required quality takes. Comparing optimizers, precision, or batch schedules requires both measurements under a common acceptance criterion. A real experiment must also account for run-to-run variation and any change in evaluation frequency; an apparent crossing of the target is not evidence that a recipe reliably meets it.

Optimizer rules specify how parameters move once gradient vectors are in hand, yet computing those gradients dominates both arithmetic and memory in a billion-parameter network. The mechanics of gradient generation, rather than the optimizer update itself, therefore govern the training memory budget and step time.

8.3.3 Backpropagation and activation memory

Backpropagation evaluates parameter gradients by traversing the network's computational graph in reverse topological order, applying the chain rule layer by layer (section 5.4.5). Each backward rule reads values from the forward pass, either saved or recomputed, so tensors that inference would release after their last forward consumer stay alive until the backward pass reaches them (section 7.4.3.2). Computing $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}}$ requires the layer's forward input, and the retention of those inputs creates the training memory equation that gradient checkpointing later exploits. At the scale of GPT-2 XL, the question is what that retention costs in memory capacity and arithmetic throughput, because retained activations in HBM can become the binding memory constraint of the run.

The arithmetic side of that cost follows from the derivative equations of section 5.4.5.4. Backpropagation through a linear layer $\mathbf{Z} = \mathbf{AW}$ evaluates two matrix products, the activation gradient $\nabla_{\mathbf{A}} \mathcal{L} = (\nabla_{\mathbf{Z}} \mathcal{L})\mathbf{W}^T$ that carries the error signal upstream and the parameter gradient $\nabla_{\mathbf{W}} \mathcal{L} = \mathbf{A}^T (\nabla_{\mathbf{Z}} \mathcal{L})$ that the optimizer consumes. Each permutes the forward matrix dimensions while keeping the forward product's leading FLOP count, so backward costs about twice forward and a complete step about three times the forward FLOPs. At the scale of a transformer, that same factor of three turns the $2P$ forward FLOPs per token into the $6P$ per token behind the lighthouse budget. The memory side of the cost has no such fixed ratio, because it depends on which forward values the backward rules must read.

The three-layer MNIST classifier ($784 \to 128 \to 64 \to 10$) of chapter 5 needed only kilobytes of activation storage. GPT-2 processing a single batch of 32 sequences of 1024 tokens requires approximately 286.9 GB of saved activations. Adding parameters, gradients, master weights, and optimizer state raises the estimated training footprint to 310.9 GB, about 3.6× the H100's 80 GB before temporary workspace is counted. Section C.3.3 puts a number on each term of the full training memory equation ($M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$) for GPT-2. Modern ML frameworks automate gradient evaluation through automatic differentiation, but managing the resulting memory footprint and arithmetic schedule remains an unavoidable systems engineering responsibility.

8.3.3.1 Activation memory requirements

The four-term training-memory budget of equation 5.29 partitions training state into saved forward activations required by backward derivative rules, model parameters, parameter gradients, and optimizer state (including any higher-precision master weights), and only the first category grows with the batch. Consider a batch of training examples passing through a network. The forward pass computes and stores:

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$$
$$\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)})$$

To compute the parameter gradient $\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L} = (\mathbf{A}^{(\ell-1)})^T(\nabla_{\mathbf{Z}^{(\ell)}}\mathcal{L})$, the backward rule must read the layer's input activations $\mathbf{A}^{(\ell-1)}$. To propagate the error signal upstream through the nonlinear activation function via $\nabla_{\mathbf{Z}^{(\ell)}}\mathcal{L} = (\nabla_{\mathbf{A}^{(\ell)}}\mathcal{L}) \odot f'(\mathbf{Z}^{(\ell)})$, the kernel requires either the pre-activation tensor $\mathbf{Z}^{(\ell)}$ (as in GELU) or the post-activation output $\mathbf{A}^{(\ell)}$ (when f' can be computed from the output, as in sigmoid, or via a 1-bit boolean mask in ReLU). Fused kernels and selective recomputation can compress this saved footprint. Nevertheless, because activations scale with batch size B and sequence length S, their storage demands grow with the data dimension, whereas parameter and optimizer memory remain fixed by the model architecture. The GPT-2 calculation in notebook 8.3 decomposes this memory pressure across attention blocks, feed-forward sublayers, and optimizer states.



Napkin Math 8.3: GPT-2 activation memory breakdown

For GPT-2 with batch size B = 32, sequence length S = 1024, hidden dimension d = 1600, and 48 layers:

Math:

- Attention activations: $B \times S \times d \times 4$ (Q, K, V, output) = 32 × 1024 × 1600 × 4 × 2 bytes (FP16) = 419.4 MB
- FFN activations: $B \times S \times (4d)$ (intermediate expansion) = 32 × 1024 × 6400 × 2 bytes = 419.4 MB
- Attention scores: $5 \times N_{\text{heads}} \times S^2 \times B$ bytes, because each element of a head's $S \times S$ attention matrix is saved three times for backward: 2 bytes for the FP16 softmax probabilities, 1 byte for the dropout mask, and 2 bytes for the probabilities after dropout. With 25 heads, this term reaches 4194.3 MB per layer. It is the dominant term, quadratic in sequence length, and exactly what selective recomputation discards
- Layer norm states: Minimal (~10 MB per layer)
- Total per layer: ~5043.2 MB (attention + FFN + attention scores + layer norm states)

Result:

- Line items across all layers: 5043.2 MB × 48 layers ≈ 242.1 GB
- Total activation memory (library estimate): 286.9 GB, about 44.8 GB more than the line items because it also counts saved tensors the list omits, such as the inputs to each linear and normalization layer, the GELU output, and the dropout masks
- Parameters (FP16): 3 GB
- Gradients: 3 GB
- Master weights (FP32): 6 GB
- Adam moment state (FP32): 12 GB
- Peak memory during training: ~310.9 GB

This exceeds a single H100's 80 GB capacity.

Solutions:

1. Selective activation recomputation: Discard attention intermediates and recompute them during backward, reducing this modeled activation term by 70.2 percent to ~85.6 GB; the additional work depends on the implementation
2. Activation CPU offloading: Store some activations in CPU RAM, transfer during backward pass
3. Mixed precision: FP16 activations (already applied) vs. FP32 (would be 573.8 GB)
4. Reduced micro-batch size and gradient accumulation: Build a larger effective batch from multiple memory-feasible forward/backward passes

Systems insight: Even after selective recomputation, the modeled activations plus static state total about 109.6 GB, still above the H100's 80 GB before workspace and allocator overhead. Selective recomputation removes 70.2 percent of the activations, the same quadratic attention term that FlashAttention never stores; the segment schedule in notebook 8.8 goes further by retaining only segment inputs between passes, and the walkthrough composes these techniques until the batch-32 step fits.

This breakdown illustrates the practical engineering decisions required when accelerator memory falls short. The same trade-off between stored activations, recomputation, and batch size drives the memory-computation analysis that follows.

8.3.3.2 Memory-computation trade-offs

Training systems must balance memory usage against computational efficiency. Each forward pass generates values that backward rules may request. For a simplified network model with N_L layers, let s_ℓ denote the bytes of saved intermediate state per example at layer ℓ and $M_{\text{activations}}^{(\ell)}$ the bytes of saved activation outputs per example. Under this per-example convention, their storage scales linearly with batch size and is approximated by equation 8.3:

$$M_{\text{activations}} = B \times \sum_{\ell=1}^{N_L} \left(s_\ell + M_{\text{activations}}^{(\ell)} \right) \tag{8.3}$$

This memory requirement compounds with the weights, gradients, and optimizer memory discussed in section 8.3.2.3. Together they form the same four-term budget that equation 5.29 established and the lighthouse callout applied to GPT-2, labeled here once as equation 8.4 so that later sections can point to it:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}} \tag{8.4}$$

The runtime peak can exceed this sum because temporary workspaces, communication buffers, allocator behavior, and framework bookkeeping also occupy memory. To reduce the activation term, training systems can recompute intermediate values during backward rather than storing them. This increases computational work but can enable deeper networks or larger batches on memory-constrained hardware (T. Chen et al. 2016). Activation checkpointing, which keeps activations only at selected layers and rebuilds the rest during the backward pass, is developed in section 8.6.5.1, with its optimal spacing and its cost for GPT-2 in section 8.6.5.2.

Capacity is only one of two physical constraints imposed by memory; the other is bandwidth. Accelerators provide high matrix compute throughput, but shuttling intermediate activation tensors between off-chip High Bandwidth Memory (HBM) and on-chip SRAM frequently saturates the memory bus (Jouppi et al. 2017). Even when activations fit comfortably within HBM capacity, the time required to read and write them can dwarf the arithmetic execution time. To relieve this memory bus pressure, modern training engines fuse elementwise operations directly into preceding matrix multiplications and stage gradient computations through on-chip SRAM tiles rather than global memory. At the runtime level, framework caching allocators track tensor lifetimes across the computation graph, recycling memory blocks as soon as an activation's final backward consumer completes (Paszke et al. 2019).

8.3.3.3 Fine-tuning memory on one accelerator

GPT-2 XL's static state leaves most of an H100 for activations. One size class up, the memory constraint changes the training recipe itself. Llama 3 8B (Dubey et al. 2024), the larger member of the book's bandwidth-bound language-model family, is usually adapted to a new task rather than trained from scratch, and whether that adaptation fits on one accelerator depends on which parameters it trains.

Training scope changes which terms of the memory budget are populated. Let P count all resident model parameters, including any added head or adapters. Inference retains their weights but has $M_{\text{gradients}} = M_{\text{optimizer}} = 0$, because it neither computes parameter gradients nor updates weights. Full fine-tuning allocates gradients and optimizer state for all P parameters, following the chosen precision recipe. Head-only adaptation keeps the frozen backbone resident but allocates gradients, master weights when used, and optimizer moments only for the trainable head. Adapters, small trainable modules inserted into a frozen network, similarly restrict these update buffers to the added parameters. Their weight-memory term still includes both the frozen base and the adapters; storing fewer trainable parameters does not remove the base weights.

Saved activation memory follows the backward path rather than the trainable parameter count. When a frozen feature extractor runs without gradient recording and supplies its output as a constant to a trainable head, the head retains that boundary input and its own required backward values; the backbone has no saved backward activations. Inference also avoids saved backward state, although live forward intermediates and workspaces still consume memory. Adapters distributed through the network create a different topology. Gradients must traverse intervening frozen operations to reach earlier adapters, so those operations may still require saved tensors or recomputation. Setting `requires_grad=False` on base weights suppresses their parameter gradients, but disabling recording across that path would also cut the gradients the adapters need.

↩ Gradient-recording and detachment controls for frozen feature extractors are established in section 7.4.3.2.

The comparison therefore separates a residency decision from a differentiation decision. Subtracting $M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}}$ from device capacity gives the space available for activations and runtime overhead. A head-only configuration may fit where full fine-tuning does not, provided it meets the task's quality target; an adapter configuration must still budget the activations on its backward path. If those activations exceed the remaining space, checkpointing or a smaller micro-batch may still be necessary despite the small optimizer state. For a fixed training scope, only activation memory scales approximately linearly with batch size (equation 8.3); model parameters and optimizer state remain fixed. The Llama 3 8B case puts numbers on both decisions.

Napkin Math 8.4: Fine-tuning Llama 3 8B on one H100

Problem: Can a 8.0B-parameter Llama 3 model be fine-tuned on one H100 with 80 GB (85.9 GB in decimal units) of HBM, first with every parameter trained and then with low-rank adapters?

Given: The mixed-precision Adam recipe of section 8.3.2, a batch of 4 sequences of 2,048 tokens, and attention that never stores the quadratic score matrix.

Math:

1. **Full fine-tuning, static state**: FP16 weights 16 GB + FP16 gradients 16 GB + FP32 master weights 32 GB + Adam moments 64 GB = 128 GB, already beyond one accelerator before any activation is stored.
2. **Activations**: The per-layer estimate of section 8.3.3.1, without the quadratic attention term, gives 36.5 GB at this batch, so full fine-tuning needs about 165 GB in all.
3. **Low-rank adapters**: Low-rank adaptation (LoRA) (Hu et al. 2021) freezes the base weights and trains a pair of small matrices beside each adapted projection. Rank-16 adapters on every query and value projection add 6.8 million trainable parameters, 0.08 percent of the model, whose gradients, master copies, and moments occupy 0.11 GB.
4. **Adapter total**: Frozen BF16 weights 16 GB + adapter state 0.11 GB + activations 36.5 GB ≈ 53 GB, leaving about 33 GB for workspaces and allocator overhead.

Systems insight: Full-parameter training of this model does not fit one accelerator; it requires partitioning its state across devices with fully sharded data parallelism (FSDP) or ZeRO, which section 8.7.1.1 develops, or offloading state to host memory. Adapters change which terms of the memory budget are populated: the frozen weights stay resident, but gradients and optimizer state shrink to the adapters, and activations still follow the backward path through every frozen layer. Storing the frozen base weights in a 4-bit format shrinks the largest remaining term further (Dettmers et al. 2023), a recipe distinct from the full-parameter mixed-precision training above.

Forward propagation, gradient computation, and parameter updates define *what* training systems must compute. An operation's name alone, however, does not reveal *where* the system stalls. Matrix shape, kernel fusion, numerical precision, and the memory hierarchy determine whether arithmetic throughput or data movement binds execution. Arithmetic intensity provides the analytical tool to quantify that balance.

8.3.4 Arithmetic intensity

During a single training step on the reference H100, a large dense matrix multiplication runs near the chip's Tensor Core peak while a standalone softmax or LayerNorm kernel stalls waiting for bytes from HBM. Arithmetic intensity explains that split by comparing the operations a kernel performs to the bytes it moves across the memory bus:

$$I = \frac{O}{D_{\text{vol}}}$$

Within the roofline model, operations above the hardware ridge point have a compute-throughput ceiling, while those below it have a memory-bandwidth ceiling. Launch overhead, limited parallelism, and imperfect execution can keep measured performance below either ceiling. Section 11.6 develops the roofline model and the hardware ridge point, and section D.2.1 works through the derivation.[12]

Table 8.5 summarizes common tendencies rather than fixed measurements. Dense matrix multiplication can reuse operands enough to reach high arithmetic intensity, while standalone elementwise and reduction kernels often perform less work per byte moved. The target hardware's ridge point determines the resulting regime.

Table 8.5: Training Operation Classification Conditions: Operation families follow common arithmetic-intensity tendencies, but classification requires comparing the measured implementation with the target hardware's ridge point. Memory-sensitive kernels may benefit from reducing traffic or increasing fusion, while compute-bound kernels benefit from additional arithmetic throughput.

Operation	Arithmetic-intensity tendency	Classification condition
Dense MatMul (large)	High when shapes and tiling provide substantial operand reuse	Compute-bound only when intensity exceeds the ridge
Activation functions	Usually low as standalone elementwise kernels	Often memory-sensitive unless fusion removes traffic
LayerNorm/BatchNorm	Reduction and elementwise work with multiple tensor passes	Depends on fusion, shape, and target ridge point
Attention softmax	Reduction and normalization over the attention axis	Depends on materialization, tiling, and fusion

Hardware efficiency during model training depends directly on kernel arithmetic intensity. Figure 8.1 bounds attainable throughput across training kernels on the H100's roofline, contrasting memory-bandwidth-bound operators (softmax, LayerNorm) along the sloped ceiling against compute-bound operators (dense MatMul) on the flat 989 TFLOP/s peak.

On the reference machine the ridge in figure 8.1 sits at 295.2 FLOP/byte, the H100's 989 TFLOP/s peak divided by its 3.35 TB/s of HBM bandwidth. The operation points in this diagram are illustrative placements that show how to interpret the model, not measured GPT-2 coordinates.

[12] **Ridge point and precision**: The roofline ridge point, the arithmetic intensity threshold separating memory-bound from compute-bound operations, shifts with numerical precision. On the same accelerator, a lower-precision Tensor Core path can expose much more arithmetic throughput at roughly the same memory bandwidth, raising the ridge point substantially. Switching from TF32-style execution to BF16 mixed precision can therefore change which optimization technique yields returns, making precision selection inseparable from roofline analysis.

↩ Arithmetic intensity, the ridge point, and the memory-bound and compute-bound regimes they separate are established informally in section 2.4.2.

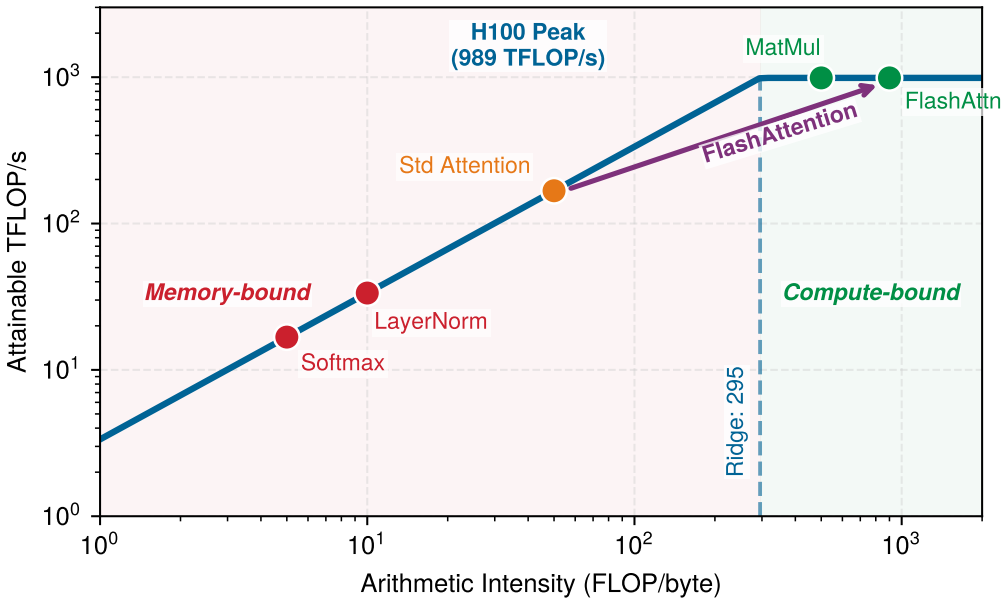


Figure 8.1: Illustrative Training Roofline: An H100 roofline with assumed operation coordinates illustrates how arithmetic intensity changes the attainable-performance ceiling. The points are not a profile of one GPT-2 execution. FlashAttention is shown moving right because it reduces HBM traffic; whether a real attention kernel crosses the ridge depends on shape, precision, hardware, and implementation.

Consider a GPT-2 attention layer that materializes the $S \times S$ attention-score matrix. For each head, the $\mathbf{QK}^T$ product costs approximately $2S^2 d_{\text{head}}$ FLOPs, while writing the FP16 score matrix and reading it back moves about $4S^2$ bytes. Counting only that score-matrix traffic gives the approximation $d_{\text{head}}/2$. For GPT-2 Small ($d = 768$ across 12 heads, so $d_{\text{head}} = 64$), this restricted accounting yields 32 FLOP/byte. The lighthouse GPT-2 XL gives the same answer, because its head dimension is also 64 and the ratio depends only on d_{head}. A complete kernel analysis must also count Q and K traffic, softmax work, additional reads and writes, cache reuse, and fusion. The $\mathcal{O}(S^2)$ materialized score traffic remains the term that FlashAttention targets.

At 295.2 FLOP/byte, the H100's ridge sits far to the right of the 32 FLOP/byte materialized attention estimate, so low-intensity operations such as attention softmax remain memory bound even when large matrix multiplications on the same device are compute bound.

> Systems Perspective 8.2: Peak FLOP/s vs. sustained performance
>
> Peak TFLOP/s is an arithmetic ceiling rather than a prediction of sustained application performance. For an implementation that is strictly bandwidth bound, raising only the arithmetic ceiling does not improve the roofline bound. Mixed-precision training can both enable faster arithmetic paths and reduce traffic for tensors actually stored and transferred at lower precision. The realized gain depends on kernel support, conversion overhead, numerical requirements, and which resource is binding. Roofline-guided optimization therefore measures the implementation before choosing fusion, precision changes, or additional compute.

Batch size also moves an operation along the intensity axis, because a larger batch spreads each weight load over more examples. Where the transition from memory-bound to compute-bound execution falls depends on the operation's shape, its precision, and the hardware.

This analysis guides optimization strategy selection. For memory-bound operations, reducing data movement through kernel fusion, reduced precision, or algorithmic improvements like FlashAttention provides the largest gains. For compute-bound operations, increasing throughput through

Tensor Cores and parallel execution matters more. The first case asks how to move fewer bytes, and the second asks how to keep more arithmetic units busy.

The trajectory vector in figure 8.1 illustrates how FlashAttention[13] can shift transformer attention toward, and for favorable shapes across, the 295.2 FLOP/byte ridge line onto the compute ceiling. By never materializing the full $S \times S$ attention matrix in HBM and instead processing tiles that fit in fast SRAM (tiling, the general technique developed in section 11.7.5), FlashAttention reduces auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$ and sharply reduces HBM traffic. The original work reports up to 3× speedups on evaluated workloads (Dao et al. 2022), with gains depending on tensor shape, hardware, and baseline. The algorithm and the conditions under which it applies are examined in detail in section 8.6.4.

[13] **FlashAttention**: The core mechanism processes attention in small tiles that fit within the accelerator's fast on-chip SRAM, avoiding writes of the full intermediate $S \times S$ matrix to slower HBM. The exact HBM IO bound depends on tile size and SRAM capacity, but the practical effect is a large reduction in memory traffic and auxiliary activation storage.

Arithmetic-intensity analysis is how the silicon contract (principle 4) is checked in training. The silicon contract names the resource a workload is built to exploit; the analysis determines which physical resource actually constrains a particular implementation. Dense matrix multiplication reaches the compute-bound regime when tensor shapes provide sufficient data reuse, while standalone normalization and elementwise activation kernels remain memory bandwidth bound.

Optimizing individual operations is necessary but insufficient. A perfectly tuned matrix multiplication achieves nothing if the accelerator sits idle waiting for the next batch of data. Isolated operation costs (dense matrix multiplication FLOPs, memory-bound activations, and optimizer state allocations) compound within the execution loop. Sustained throughput requires orchestrating these discrete operations into a balanced pipeline where upstream data delivery and downstream parameter updates prevent accelerator starvation.

8.4 Pipeline Architecture

A training step is not a single operation but a sequence of dependent stages: data must be loaded before computation can begin, forward passes must complete before backward passes start, and gradients must be computed before parameters can update. The speed of the slowest stage determines the speed of the entire system. Without pipelining, these stages serialize, idling the accelerator while the host prepares data or transfers batches across the bus.

While model execution graphs govern arithmetic on the accelerator, sustained training requires feeding those kernels across physical machine boundaries. Host CPUs ingest raw records from storage and apply preprocessing in host DRAM, while accelerators execute matrix multiplications in HBM. Data must cross the interconnect, typically a PCIe link whose bandwidth is an order of magnitude lower than device memory bandwidth. When upstream data preparation or bus transfer exceeds the duration of the forward and backward passes, the accelerator compute engines starve, reducing the hardware utilization factor η_{hw} in the training-time law.

The training system divides into three subsystems. The data pipeline handles ingestion, transformation, and batching on the host; the training loop coordinates forward passes, backward passes, and parameter updates on the accelerator; and the evaluation pipeline periodically assesses model quality on held-out data. The three subsystems decouple across the training lifecycle (figure 8.2): processed batches flow into the core forward/backward training loop while metrics stream to an asynchronous evaluation pipeline. The interfaces between them form critical synchronization boundaries where I/O latency, interconnect transfers, or synchronous validation pauses can throttle accelerator utilization. The data pipeline supplies two of the three terms of the pipeline bottleneck model defined later (equation 8.6), host preprocessing and host-to-accelerator transfer, the accelerator training loop supplies the third through matrix compute, and periodic evaluation claims shared execution time from that loop.

8.4.1 Pipeline stages and accelerator bubbles

A training step passes through four stages, a **staged system pipeline** rather than a monolithic loop. If one stage is slower than the rest, the others wait:

1. **Data Loading & Preprocessing** (CPU/Storage): Fetching raw bytes from Non-Volatile Memory Express (NVMe) storage, decoding samples, and executing transformations on CPU cores.
2. **Host-to-Device Transfer** (PCIe): Moving the preprocessed batch across the PCIe bus into GPU memory via direct memory access (DMA).

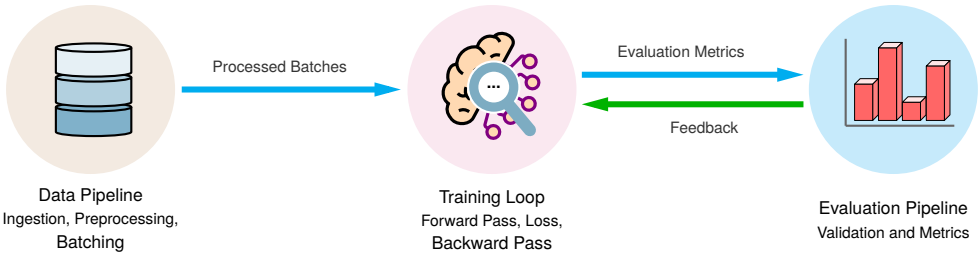


Figure 8.2: Training System Overview: The training lifecycle coordinates data preparation, core forward/backward computation, and validation evaluation. The interfaces between these three pipelines represent critical synchronization boundaries where I/O delays or evaluation pauses can throttle accelerator utilization.

3. **Forward/Backward Pass** (Accelerator): Propagating activations forward and gradients backward through layer kernels on the GPU.
4. **Parameter Synchronization** (Interconnect, present only when multiple accelerators train the model): Exchanging gradients between accelerators across the available interconnect fabric. NVLink provides up to 900 GB/s in the H100 configuration examined later, while alternative architectures rely on PCIe switches or scale-out network fabrics.

Any throughput mismatch among these stages creates **accelerator bubbles**, idle windows during which compute units stall while waiting for upstream dependencies. These bubbles stem from imbalances within the per-step pipeline. A systems engineer eliminates them through asynchronous prefetching and pipeline overlapping, staging the next batch into high-bandwidth memory before the active step completes. The first three stages form the analytical basis for the pipeline execution model in equation 8.6, where the slowest stage dictates end-to-end iteration throughput, and section 8.5 provides the profiling methods required to isolate that bottleneck.

Across the three decoupled subsystems of figure 8.2, each stage occupies a distinct hardware domain: host storage and CPU cores ingest and preprocess data, accelerator compute engines execute the model loop, and evaluation pipelines periodically preempt execution to calculate validation metrics. Any latency imbalance among them therefore becomes idle accelerator cycles. At the ingestion boundary, the data pipeline stages raw records from persistent storage into host memory, applies transformations such as augmentation and normalization (LeCun et al. 2012), and packs tensors into dense batches. These batches transfer over the host-accelerator interconnect (such as PCIe) into accelerator memory. At the validation boundary, the evaluation pipeline periodically runs held-out data through the model without computing gradients. Because validation occupies the same accelerator cores needed for training, its invocation frequency represents a direct trade-off between monitoring resolution and overall training throughput.

Within the accelerator, the training iteration forms a closed feedback loop governed by strict data and memory dependencies. Figure 8.3 traces this cyclic sequence across three stages (predict, gradients, and update), showing how predictions, the loss, and parameter gradients feed the optimizer before returning updated weights to the next iteration.

While section 8.3 counted the FLOPs and bytes of each operator in this cycle, memory residency and data dependencies govern how the hardware executes them. During the forward pass, intermediate activation tensors must remain resident in accelerator HBM because the backward pass requires them to evaluate chain-rule derivatives. Once gradients are computed, the optimizer reads both the parameters and accumulated gradients to apply the update rule, producing new weights for the subsequent iteration. The memory capacity required to hold activations, weights, and optimizer states establishes the memory floor for the entire training step.

This execution loop repeats across batches and epochs until the optimization converges. Maximizing accelerator arithmetic efficiency requires overlapping these pipeline stages across the system hierarchy. Host CPU threads prefetch and stage batch $k+1$ into page-locked (*pinned*) host memory while the accelerator executes batch k, so the DMA engine can stream it across PCIe asynchronously (section 7.5.2 explains why pinning is required). This concurrent staging hides storage latency and bus transfer time behind matrix-multiplication compute kernels. When this concurrency breaks

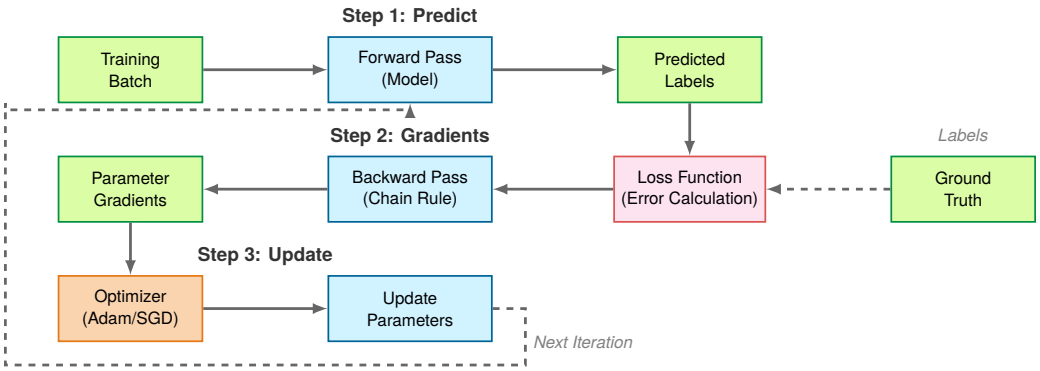


Figure 8.3: The Iterative Training Loop: Each iteration sequences forward prediction, gradient computation by backpropagation, and an optimizer update. Updated parameters return to the next forward pass while the next batch remains a separate input. Data dependencies and memory residency set the step's minimum execution time and memory floor.

down, because an I/O-bound data pipeline starves the device of input tensors or frequent evaluation stalls the update loop, the accelerator idles, degrading effective throughput.

8.4.2 Data pipeline

Keeping an accelerator's arithmetic units saturated requires streaming training examples from persistent storage into device memory at or above the model's consumption rate. Data engineering posed this as the feeding problem (section 4.9.3) and measured the throughput lost when a slow storage path cannot keep up; training is where that loss is paid on every step. ResNet-50 is the chapter's input-pipeline case, because its images are far heavier for the host to prepare than GPT-2's token IDs. The stage trace below starts with GPT-2 XL under a scenario assumption, on-the-fly single-core tokenization (production runs usually pre-tokenize), and the system implications then turn to ResNet-50. Upstream data engineering prepares and partitions datasets on storage media, while the runtime data pipeline executes concurrently with model computation on the host CPU.

The data pipeline bridges raw storage and accelerator execution across the machine boundary. Figure 8.4 partitions this data movement across three physical zones: staging raw files from persistent storage into host memory, formatting and augmenting tensors across CPU worker threads, and streaming batched inputs over the interconnect to parallel GPU accelerators.

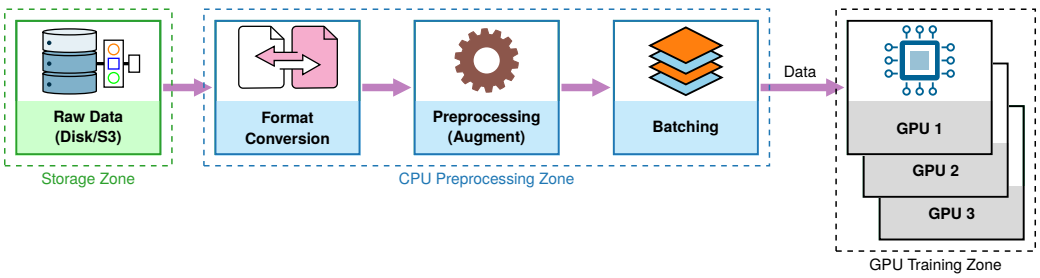


Figure 8.4: CPU-to-GPU Data Flow: Training data transitions through storage, CPU preprocessing, and GPU execution zones. Each stage represents a potential throughput gate; disk read bandwidth, transformation latency, and PCIe host-to-device transfers must collectively sustain the GPU consumption rate to keep accelerators at peak utilization.

In the storage zone, raw records reside on local NVMe drives or networked storage systems. In the CPU preprocessing zone, host worker threads decode raw bytes, apply transformations such as tokenization or data augmentation, and collate individual records into dense mini-batch tensors. In the GPU training zone, accelerators transfer those tensors across the PCIe bus and execute forward and backward passes. Because these three zones form a strict pipeline, the slowest stage dictates end-to-end training throughput. In the pipeline model developed below, the storage and CPU zones

together set the preprocessing term, the PCIe crossing sets the transfer term, and the GPU zone sets the compute term.

8.4.2.1 Core components

Data pipeline throughput originates at the storage subsystem. Upstream layout decisions, such as sequential chunking in columnar or container formats (Parquet, TFRecord, Arrow), dictate the physical I/O patterns presented to the operating system.

↩ How file format and storage layout set the data term of the iron law is established in section 4.10.2.

Storage throughput is bounded by the slower of two hardware constraints, $R_{\text{storage}} = \min(\text{BW}_{\text{disk}}, \text{BW}_{\text{network}})$, where BW_{disk} is the physical disk bandwidth and $\text{BW}_{\text{network}}$ represents the network bandwidth for distributed storage systems. In practice, training workloads rarely achieve this theoretical maximum because data shuffling can turn large sequential reads into smaller, less efficient accesses. Effective storage throughput can be modeled as $R_{\text{storage,eff}} = R_{\text{storage}} \times F_{\text{access}}$, where F_{access} depends on record size, file layout, caching, and storage hardware. When unbuffered random reads degrade F_{access}, storage bandwidth drops by an order of magnitude, transforming an I/O system that appears sufficient on paper into a primary training bottleneck.

GPT-2's 40 GB WebText corpus fits in the host memory of a single training server, so once cached it can be served from system RAM without repeated storage reads, although preprocessing and transfer costs remain. A corpus that exceeds local memory must instead be streamed, and the system must orchestrate storage reads, preprocessing, caching, and delivery throughout training. Whether multiple workers, prefetching, local caches, or distributed storage pay off then depends on which of those stages limits throughput.

8.4.2.2 Preprocessing

Once retrieved from storage, raw bytes require CPU transformation into model-ready tensors. While static feature extraction can run offline, dynamic transformations such as on-the-fly tokenization and image augmentation execute during training. Under ideal independent-worker scaling, the CPU worker pool delivers transformation capacity according to equation 8.5:

$$R_{\text{preprocessing}} = \frac{N_{\text{workers}}}{T_{\text{transform}}} \tag{8.5}$$

where N_{workers} parallel processing threads each perform transformations requiring $T_{\text{transform}}$ seconds per batch. Training architectures employ multiple workers to ensure transformation keeps pace with accelerator consumption rates.

However, transforming records assumes that storage delivers raw bytes fast enough to keep worker queues filled. Under overlapped host staging, the achieved host preparation rate cannot exceed the slower of storage retrieval and CPU transformation. Dividing effective storage throughput $R_{\text{storage,eff}}$ by the raw bytes retrieved per batch yields the storage delivery rate in batches per second. When storage delivery lags behind worker demand, additional CPU threads sit idle waiting on disk I/O, capping host throughput regardless of N_{workers}. The complete host preparation rate is therefore bounded by the lesser of the storage delivery rate and the worker transformation capacity $N_{\text{workers}}/T_{\text{transform}}$.

Preprocessed data must then transfer across the host-accelerator interconnect before computation can begin. End-to-end training throughput is consequently constrained by the slowest of three sequential stages, as formalized in equation 8.6. In this three-stage pipeline model, the host preparation rate $R_{\text{preprocessing}}$ captures the joint bound of storage delivery and CPU worker transformation, while the remaining terms capture host-to-device bus transfer and device execution:

$$R_{\text{training}} = \min\left(R_{\text{preprocessing}}, \frac{\text{BW}_{\text{host}}}{D_{\text{vol}}^{(\text{batch})}}, \frac{R_{\text{sustained}}}{O_{\text{batch}}}\right) \tag{8.6}$$

where BW_{host} is the host-to-accelerator transfer bandwidth, $D_{\text{vol}}^{(\text{batch})}$ is the input bytes per batch transferred to the device, and O_{batch} is the forward-backward work per batch in FLOPs. Every term is therefore expressed in batches per second before the minimum is taken.

Example 8.1: GPT-2 language model data pipeline

Scenario: Quantifying storage access, CPU tokenization, and PCIe host-to-device transfers supplying one H100 training GPT-2 XL at batch 32. The device service interval is the step time the training-time law gives at the chapter's utilization, about 596 ms per batch; fitting that batch in memory requires the techniques of section 8.6, but the input path does not depend on which ones are used.

Stage trace:

1. **Raw text storage**: The WebText corpus contributes about 40 GB of raw text. Sequential reads from NVMe SSD storage reach 7 GB/s. Under document sampling with access factor 0.1, random-access storage bandwidth drops to 0.70 GB/s, giving $T_{\text{read}} = D_{\text{vol}}/R_{\text{storage,eff}} \approx 0.1$ ms.
2. **Tokenization**: A BPE tokenizer (50,257 vocabulary) converts text into token IDs. For batch size $B = 32$ and sequence length $S = 1024$ (32.8K tokens total), a single CPU core processing 50K tokens/s takes $T_{\text{tokenize}} \approx 655.4$ ms per batch.
3. **Batching and padding**: Padding and packing generate an int64 tensor of shape 32 sequences $\times$ 1,024 tokens, producing 262.1 KB per batch.
4. **PCIe transfer**: Transferring the tensor across the H100's PCIe Gen5 x16 link (64 GB/s bandwidth) takes $T_{\text{transfer}} = 0.004$ ms, making bus transfer negligible.

Systems insight: Single-core CPU preparation ($T_{\text{prep}} \approx 655.4$ ms) exceeds the accelerator service interval ($T_{\text{GPU}} \approx 596$ ms), so a serial pipeline pays both costs on every step and the device service stage occupies only about 48 percent of the serial step. Slower preparation increases the exposed input wait. Scaling to 8 parallel worker processes reduces the ideal average interval between prepared batches to $T_{\text{tokenize}}/N_{\text{workers}} \approx 81.9$ ms, while prefetching overlaps T_{prep} with T_{GPU} to keep the device stage supplied throughout ideal steady-state execution, delivering 54 sequences per second on one H100. The service interval is a modeled step time, not a measurement of hardware busy time. Keeping inputs available does not establish useful FLOP utilization, which requires a separate useful-work count and measured step time. Parallel workers need not shorten the latency of an individual batch, and contention or imperfect overlap can reduce the modeled throughput.

This min-of-three relationship governs training pipeline design because system throughput equals bottleneck throughput. An accelerator with 989 TFLOP/s of compute capacity delivers zero useful work while waiting for data. Conversely, a perfectly optimized data pipeline provides no benefit if the accelerator is already compute-saturated. Balanced pipeline design aligns preprocessing capacity, transfer bandwidth, and compute throughput so that no single stage dominates iteration time. Under that scenario, the GPT-2 workload shows CPU tokenization, rather than storage retrieval or bus transfer, forming the primary throughput barrier unless parallel workers and prefetching conceal host preparation latency.

Data pipeline throughput is not the only flow that can starve the accelerator. Multi-GPU training adds a second stream of traffic: the gradient synchronization required after every step. Once synchronization time exceeds compute time, communication becomes the limiting factor. Section 8.7.2 quantifies that network wall at the point where training crosses the node boundary.

8.4.2.3 System implications

Comparing supply with demand requires both in the same unit. Grouping the two host-side terms of equation 8.6 into a single pipeline rate R_{pipeline}, in batches per second, and writing the accelerator's consumption rate as $R_{\text{accel}} = R_{\text{sustained}}/O_{\text{batch}}$, also in batches per second (where $R_{\text{sustained}}$ stays in FLOP/s), leaves a coupled system whose throughput is $\min(R_{\text{pipeline}}, R_{\text{accel}})$. When $R_{\text{pipeline}} < R_{\text{accel}}$, the accelerator sits idle waiting for data, and its busy-time fraction drops proportionally in this ideal

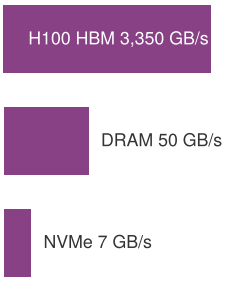
supply model, as equation 8.7 shows. Busy time counts every moment the accelerator is executing, so it is an upper bound on, and distinct from, the model-FLOP utilization defined later in the chapter:

$$\text{Accelerator Utilization} = \min\left(1, \frac{R_{\text{pipeline}}}{R_{\text{accel}}}\right) \times 100\% \tag{8.7}$$

ResNet-50 shows how quickly that drop arrives on current hardware. One training image costs about 24.6 GFLOP, so the H100's dense 16-bit peak could consume about 40,200 images/s. A well-tuned training step sustains only part of that ceiling; at the 25 percent fraction implied by the 3,200 images/s that section 4.9.3 assumed for the previous-generation A100, the H100 consumes about 10,100 images/s. A host worker that decodes and augments 375 images/s would need 28 workers per accelerator to keep up, but an eight-accelerator H100 server provides 112 host cores, 14 per accelerator. Those workers supply about 5,250 images/s, so by equation 8.7 the accelerator is busy only about 52 percent of the time. The same host fed an A100 with 9 workers and cores to spare. Upgrading the accelerator raised its appetite 3× without changing the host, which moves ResNet-50 from its home compute-bound regime into an input-bound one. Faster arithmetic adds nothing until the input pipeline is rebalanced, with more host cores per accelerator, decoding on the accelerator, or cached decoded data, and section 8.6.2 takes up the overlap that hides what remains.

8.4.2.4 Data flows

Training data traverses three memory tiers on its way from disk to accelerator, and the bandwidth gap between these tiers, spanning roughly two orders of magnitude, is the central challenge of data pipeline design. For equal byte volumes at each tier, the effective transfer rate is bounded by $\min(\text{BW}_{\text{storage}}, \text{BW}_{\text{system}}, \text{BW}_{\text{accelerator}})$, the same minimum as equation 8.6 applied to bandwidth tiers. When decoding, tokenization, or reuse changes the bytes handled per batch, each bandwidth must first be divided by its own per-batch byte volume so that the rates are comparable.



Bandwidth steps up the storage to DRAM to HBM hierarchy.

Local NVMe storage provides 7 GB/s, system memory delivers 50 GB/s, and the H100's HBM delivers 3350 GB/s. Each tier is roughly an order of magnitude or more faster than the one below it, which means data that flows freely within accelerator memory creates a severe bottleneck when it must be fetched from disk. Stated in time rather than rate, the pipeline model says that when stages overlap correctly (fetching the next batch from storage while preprocessing the current one and transferring the previous one to the accelerator), the steady-state input service interval, denoted here by iteration time, is governed by the *maximum* of the stage times, $T_{\text{iteration}} = \max(T_{\text{fetch}}, T_{\text{process}}, T_{\text{transfer}})$, rather than their *sum*. This overlap is what prefetching seeks to achieve, with each tier operating concurrently on different batches. End-to-end training must also include accelerator compute and any exposed synchronization or evaluation time.

8.4.2.5 Practical architectures

An NVMe device rated at 7 GB/s would sustain 3.50 GB/s if a workload achieved half of peak. Small random reads and shuffling can reduce throughput further; the penalty depends on access size, queue depth, file layout, and caching.

To keep accelerators fed despite this bandwidth reduction, pipeline architectures maintain multiple data buffers simultaneously: prefetch buffers loading future batches, processing buffers holding data under transformation, and transfer buffers staging data for accelerator consumption. The total host memory required scales with the per-batch memory footprint M_{batch} according to equation 8.8:

$$M_{\text{required}} = (N_{\text{prefetch}} + N_{\text{processing}} + N_{\text{transfer}}) \times M_{\text{batch}} \tag{8.8}$$

To avoid starving the accelerator, the average preprocessing service time per batch across all workers must not exceed the compute time per batch ($T_{\text{preprocessing}} \leq T_{\text{compute}}$), which restates the preprocessing term of equation 8.6 in time. If preprocessing takes 40 ms per batch on one worker while the accelerator consumes a batch every 10 ms, ideal scaling requires at least four workers; actual worker requirements depend on storage throughput, CPU contention, and transformation cost, which section 8.6.2 examines through prefetching and parallel workers.

8.4.3 Forward and backward pass

The operation counts in section 8.3.1.1 become a scheduling problem once a prepared batch reaches the forward pass: the runtime must launch kernels with enough parallel work, stage their operands, and keep the values required by backward execution.

8.4.3.1 Compute operations

A dense layer's FLOP count is already fixed by its input width, output width, and batch size. Its execution time also depends on how that work fills the accelerator. A layer with dimensions of 512×1024 at batch 64 supplies 33.6 million MACs, or 67.1 MFLOP; the kernel must divide this work among execution units without leaving a long, mostly empty final wave.

Accelerators execute these parallel workloads by grouping execution threads into fixed-size units: warps[14] of 32 threads on NVIDIA architectures, or wavefronts of 64 threads on AMD architectures. Hardware execution efficiency depends on aligning tensor dimensions and tile shapes with the thread-block geometry and the operand matrix dimensions required by specialized units like Tensor Cores.

[14] **Warp**: From textile weaving, where a "warp" is the set of threads held taut on a loom while a shuttle weaves across them. NVIDIA adopted the term because GPU threads within a warp execute the same instruction in lockstep, analogous to parallel threads moving together on the loom. An NVIDIA warp contains 32 threads.



Systems Perspective 8.3: Wave quantization and tail effects

A common mistake in ML systems is ignoring the fixed execution granularity of GPU kernels. GPU execution is quantized into waves of work, but the relevant unit is the kernel's thread or tile mapping, not necessarily one sample per thread.

Warps make the granularity concrete. In the illustrative special case where one independent work item maps to one thread, 32 items fill one warp while 33 items require a second, mostly empty warp. Neural-network kernels usually map each sample to many threads, so batch size alone does not determine this utilization.

On a large GPU such as the NVIDIA H100 with 132 Streaming Multiprocessors (SMs), where each SM schedules groups of warps, the hardware processes thousands of threads in a single wave. If the total workload slightly exceeds a wave boundary (for example, 1.01 waves), the hardware must execute a second, nearly empty wave to completion before the next task begins.

Table 8.6 quantifies that one-item-per-thread illustration. It is not a general batch-size performance model.

Table 8.6: Illustrative Warp-Tail Arithmetic: Under a one-work-item-per-thread mapping, counts just above a 32-thread boundary launch a partially filled warp, so 33 items use two warps while 64 fill the same two warps. Actual training kernels map tensors to threads and tiles in implementation-specific ways, often assigning many threads to each sample, so these values cannot be inferred from batch size alone. Profiling must identify the kernel's real mapping and wave boundaries.

Independent Work Items	Warps Needed	Lane Utilization	Relative Time
32	1	100%	1×
33	2	51.6%	~2×
64	2	100%	1×
65	3	67.7%	~1.5×

Systems insight: The table teaches the shape of the penalty, not its size for any real kernel. The actionable rule is to choose batch, sequence, and hidden dimensions that are multiples of the kernel's tile size and that divide into whole waves across the SMs, then benchmark the neighboring candidates. Profiling reveals partially filled warps or waves; batch size alone cannot.

Beyond dense matrix multiplication, distinct neural architectures introduce specialized computational patterns. Convolutional networks, for instance, slide spatial filters across multichannel input tensors. Consider an input tensor of dimensions $64 \times 224 \times 224 \times 3$ (batch size $\times$ height $\times$ width $\times$ channels) processed by 7×7 kernels. Each position requires 147 multiply-accumulate operations, and with 64 filters operating across 218×218 spatial dimensions, the arithmetic work reaches hundreds of millions of multiply-accumulate operations per image. Accelerators execute

these spatial convolutions by either lowering them into dense GEMMs through column unfolding (`im2col`) or applying Winograd minimal filtering, which reduces multiplications at the cost of extra additions and transform workspace.

The other operator families impose different scheduling constraints. Attention alternates batched matrix products with row-wise reductions, while normalization requires statistics before normalized outputs can be produced. The arithmetic-intensity analysis in section 8.3.4 explains their traffic cost; the runtime must additionally respect these reduction dependencies and choose which intermediates can remain inside a fused kernel.

Optimized operator libraries such as cuDNN (Chetlur et al. 2014) implement multiple computational algorithms for each layer type, dynamically selecting among them based on tensor dimensions, scratchpad memory limits, and hardware capabilities. For example, cuDNN chooses between direct convolution, GEMM-based lowering, fast Fourier transform (FFT) convolution, and Winograd transforms by evaluating the trade-off between workspace memory overhead and arithmetic operations (NVIDIA Corporation 2023a). These hardware utilization patterns reinforce the coupling between batch size and hardware efficiency established in section 8.3.2.1: larger batch sizes saturate accelerator compute units and amortize memory transfers, but increase intermediate activation storage.

8.4.3.2 Memory management

Memory management binds during the forward pass, when intermediate activations must be retained in high-bandwidth memory for gradient computation during backpropagation. How quickly that retained state grows depends on the architecture, as a convolutional network shows.

Consider ResNet-50 processing images at 224×224 resolution with a batch size of 32. The initial convolutional layer produces activation maps of dimension $112 \times 112 \times 64$; per image at single-precision (4 bytes), this requires approximately 3.2 MB. For an illustrative capacity budget, assume retained forward activations total 8 GB, the backward pass adds another 4 GB of activation working set (a supplied scenario assumption, distinct from stored parameter gradients), and model parameters consume 102.4 MB. This 12.1 GB subtotal excludes parameter gradients, optimizer state, and additional runtime workspace; those terms must be added before deciding whether training fits.

Under the same linear-scaling assumption, doubling the batch size to 64 doubles activation storage: forward activations reach 16 GB and the backward activation working set reaches 8 GB, raising this subtotal to 24.1 GB. In vision models with large spatial resolutions, activations dominate the memory footprint while parameter storage (102.4 MB) is negligible. For large language models, the balance inverts: GPT-3 (175B parameters) requires approximately 700 GB just for parameters in FP32 (350 GB in FP16), exceeding single-accelerator capacity before allocating any activation memory or optimizer state.

Because accelerator memory (80 GB on the reference H100) must hold activations, parameters, gradients, and optimizer states simultaneously, memory capacity rather than compute throughput frequently caps the feasible batch size. Two system-level techniques relieve this capacity bottleneck: activation checkpointing discards intermediate activations during the forward pass and recomputes them during the backward pass, trading redundant arithmetic for reduced storage; and mixed-precision training halves storage per value for tensors represented in 16-bit formats instead of FP32. Master weights and optimizer state may remain in FP32, so the saving must be calculated for each term of the memory budget. Both techniques are analyzed in detail in section 8.6.

During backpropagation (section 8.3.3), the backward pass traverses the network in reverse order, consuming retained forward activations to compute parameter and activation gradients at roughly twice the arithmetic cost of the forward pass. Memory requirements shift dynamically across layers. In ResNet-50, mid-network layers expand to 256 filters from the initial 64, where reduced spatial map dimensions partly compensate for higher channel depth. Peak memory footprint reflects retained forward activations, active gradient buffers, temporary workspaces, and the runtime allocation schedule rather than the width of a single layer alone.

8.4.4 Parameter updates

Once backward execution populates the gradient buffers, the optimizer consumes them and updates the persistent state defined in algorithm 8.1. The important pipeline boundary is the weight mutation: the next forward pass must observe the completed update. Listing 8.2 shows that boundary at the framework interface.

Listing 8.2: Parameter Update: Computes gradients and applies an optimizer step to model parameters. Repeating this cycle seeks to minimize the training objective, although improvement is not guaranteed on every step or epoch.

```
loss.backward()  # Compute gradients
optimizer.step()  # Update parameters
```

The optimizer's memory traffic is exposed on this dependency path. The FP32 Adam update, which moves 28 bytes per parameter in section 8.3.2.4, must finish before the next forward pass reads those weights. Fusing moment updates and weight writes can shorten this interval by reducing passes over HBM, even when the forward and backward matrix products already use the arithmetic units efficiently.

8.4.4.1 Batch size and parameter updates

Increasing batch size improves accelerator utilization by raising the arithmetic intensity of matrix multiplications and reducing synchronization frequency, but it also alters optimization dynamics. Scaling batch size without retuning the learning-rate schedule can degrade convergence, creating an optimization gap where hardware efficiency comes at the expense of model quality. Figure 8.5 isolates this dynamic across synthetic loss trajectories: holding the learning rate fixed while expanding from batch 32 to batch 256 leaves an acute optimization gap (dashed silver curve), whereas linearly scaling the learning rate brings convergence close to the baseline path (green curve).

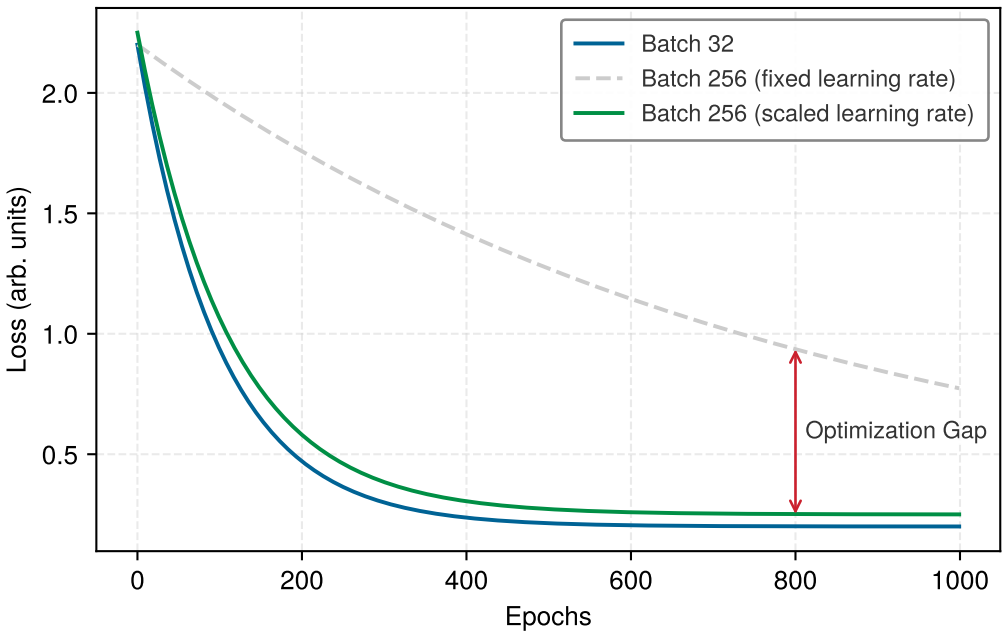


Figure 8.5: Illustrative Linear-Scaling Scenario: Synthetic training-loss curves (arbitrary units) compare a batch-32 baseline with batch 256 under a fixed or linearly scaled learning rate. The curves illustrate why increasing batch size may require retuning; they are not measured traces or a guarantee that linear scaling preserves convergence.

For a fixed token budget, doubling the global batch size halves the number of parameter updates per epoch. To traverse a comparable distance in parameter space with fewer updates, the linear scaling rule of the large-batch ResNet-50 recipe in section 8.3.2.1 scales the learning rate proportionally

with batch size ($\eta_{\text{new}} = \eta_{\text{base}} \times B_{\text{new}}/B_{\text{base}}$). Although effective across moderate batch ranges when combined with learning-rate warmup, linear scaling does not guarantee equivalent convergence or generalization. Beyond a critical batch size, gradient noise is largely eliminated, and further batch expansion yields diminishing returns per training sample while escalating memory pressure.

Beyond convergence dynamics, batch size dictates communication frequency in distributed systems. A larger global batch reduces the number of optimizer steps and gradient synchronizations per epoch. Because the communication payload of dense all-reduce synchronizations depends on parameter count rather than batch size, larger batches amortize interconnect latency over more compute FLOPs. In distributed settings, local micro-batch size and global batch size jointly determine the maximum degree of data parallelism. When memory capacity constrains the local micro-batch, gradient accumulation (section 8.6.5) decouples the effective batch size from per-device activation memory, accumulating gradients across multiple forward-backward passes before triggering an optimizer step.

8.4.5 Training state and checkpoints

A run that takes 35 days on one accelerator does not run uninterrupted. Hardware fails, jobs are preempted, and numerical instabilities corrupt weights, so a training system must save the run's state to storage at intervals and restore it exactly. That saved state is a different mechanism from the activation checkpointing of section 8.6.5.1, which recomputes values inside one step; a training checkpoint is a file that carries the run across steps.

Resuming requires restoring model weights and optimizer state in lockstep: first-moment vectors, second-moment variance estimates, and step counters. In the mixed-precision Adam recipe of section 8.3.2.3, a checkpoint holds the FP16 working weights (2 bytes per parameter), the FP32 master weights (4 bytes), and the two FP32 moments (8 bytes), 14 bytes per parameter. The FP16 gradients, which every step recomputes, are left out of the 16 bytes per parameter that training holds in device memory. For GPT-2 XL the checkpoint payload is therefore about 21 GB (3 GB + 6 GB + 12 GB), 7× the FP16 weights alone, and writing it to a local NVMe drive at 7 GB/s takes about 3 s. Checkpoint volume governs how long a save stalls the run and how long recovery takes, connecting fault tolerance to the iron law's D_{vol} term.

At the framework level, optimizers expose internal state dictionaries via `state_dict()` for serialization. Listing 8.3 demonstrates accessing optimizer state dictionaries, and listing 8.4 shows saving and restoring model parameters and optimizer states. Reproducing the training trajectory also requires the relevant random-number state, data position, learning-rate scheduler, and gradient-scaler state.

Listing 8.3: State Dictionary Interface: Optimizers expose internal state through state_dict(), enabling serialization of momentum buffers and adaptive learning rate estimates for checkpointing.

```
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Linear(10, 5)
optimizer = optim.Adam(model.parameters(), lr=0.001)

# After training steps, optimizer accumulates state
loss = model(torch.randn(3, 10)).sum()
loss.backward()
optimizer.step()

# Access state for checkpointing
state = optimizer.state_dict()
# Contains: {'state': {...}, 'param_groups': [{'lr': 0.001, ...}]}
```

Listing 8.4: Checkpoint Save and Load: Save both model parameters and optimizer state to properly resume training with correct momentum and adaptive learning rate values.

```
# Saving checkpoint
checkpoint = {
    "epoch": epoch,
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
}
torch.save(checkpoint, "checkpoint.pt")

# Resuming training
checkpoint = torch.load("checkpoint.pt")
model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
```

A checkpoint holds only persistent state. During a training step, the FP16 gradients rejoin that state in device memory and the saved activations from reverse-mode automatic differentiation sit on top of it, so device memory must hold more than the checkpoint file suggests. Notebook 8.5 works through the full comparison for a billion-parameter model.

Napkin Math 8.5: Training memory beyond the checkpoint

The frameworks chapter's ResNet-50 breakdown (section 7.4.3.1) already showed training memory several times the size of the stored weights. The following scenario extends the comparison to gradients, optimizer state, and saved activations at billion-parameter scale.

Problem: Calculate the memory that gradients, optimizer states, and forward activations add beyond raw model weights in this billion-parameter training scenario.

Math:

1. Model weights (FP16): 2 GB.
2. Gradients (FP16): 2 GB (same size as weights).
3. Master weights (FP32): 4 GB, the full-precision copy that mixed-precision training updates.
4. Optimizer states (Adam): 8 GB (momentum and variance in FP32, 4× the FP16 weight memory).
5. Activations: For a batch size of 32 and a 100-layer network, reverse-mode automatic differentiation retains the operation-dependent values required by derivative rules; activation checkpointing can omit selected values and recompute them during the backward pass.

$$M_{\text{activations}} \approx B \times N_L \times S \times d \times n_{\text{saved}} \times s_{\text{elem}}$$

For a 1024-token sequence, 1024-wide hidden state, 1 saved tensor per layer, and $s_{\text{elem}} = 2$ bytes for FP16: $32 \times 100 \times 1024 \times 1024 \times 1 \times 2 \approx$ **6.7**GB. Materialized attention terms can add a separate $B \times N_{\text{heads}} \times S^2$ component, which is why memory-efficient attention kernels matter.

Systems insight: A 2 GB model carries 16 GB of steady-state training state once gradients and optimizer buffers have been allocated, 8× its FP16 weights (16 bytes per parameter in the mixed-precision Adam recipe), while its checkpoint file holds 14 bytes per parameter. During a training step, batch-dependent activations add another 6.7 GB, raising the peak to about 20.7 GB beyond the weights. During training, data movement includes saving and retrieving these activations.

Checkpoint cadence decides how much work a failure discards. Because every update depends on the one before it, a single unmitigated numerical explosion does not cost one step; it can invali-

date every step taken since the last good checkpoint, which at frontier scale means thousands of accelerator-hours.

War Story 8.1: The PaLM loss spikes (2022)

Context: PaLM's 540-billion-parameter run used 6,144 TPU v4 chips across two 3,072-chip Pods connected through Google's data-center network and suffered severe training-instability spikes absent from smaller runs (Chowdhery et al. 2022).

Failure mode: The training loss spiked roughly 20 times despite gradient clipping. The spikes occurred at highly irregular intervals, sometimes late in training, never appeared in smaller models, and had no identified principled mitigation.

Impact: Each recovery discarded recent progress on an expensive 6,144-chip run, so reproducible checkpoints and repeatable data order were prerequisites for replay.

Response: Engineers restarted training from an uncorrupted checkpoint roughly 100 steps prior to each spike and skipped approximately 200–500 data batches around the failure window.

Systems lesson: Every step discarded in a recovery is paid for again in cluster time, so operational fault tolerance decides how much of the hardware's time turns into lasting progress. Automated loss-spike detection, checkpoint cadence, and batch skipping are core system mechanisms for keeping that lost fraction small.

PaLM's recoveries kept 6,144 chips busy on steps that were later thrown away, so the cluster's time was spent without the run advancing. In the training-time law that discarded work appears as a lower utilization factor η_{hw}, which makes checkpoint cadence a throughput decision as well as a reliability one. Frequent saves stall the run for the write time above, while sparse saves lengthen the replay after each failure.

Decomposing the training pipeline exposes the mechanics of data loading, forward and backward execution, parameter updates, and saved training state, while the computation and memory costs in section 8.3 quantify the FLOPs, memory footprint, and bandwidth each phase demands. Yet knowing how stages execute does not reveal where a specific training workload stalls. A pipeline is only as fast as its slowest stage. In an overlapped pipeline where data loading takes 120 ms and computation takes 100 ms, a 20 percent faster kernel saves nothing, because the step still takes 120 ms. Before applying optimization techniques, systems engineers must profile the execution timeline to determine whether compute, memory bandwidth, storage I/O, or network synchronization binds performance.

Checkpoint 8.2: Costing a step and building its pipeline

The first half of the chapter costed one training step and assembled the pipeline that runs it. Check that each piece can be computed before moving on to diagnosis and optimization.

Cost of a step

- ☐ **Run time and energy**: Using the training-time law, estimate how many days one accelerator needs for a run of 10^{21} FLOPs at a given peak rate and utilization, and how much energy it draws at a given device power.
- ☐ **Static state**: Explain why the common mixed-precision Adam recipe holds 16 bytes per parameter, and why switching to 16-bit arithmetic does not shrink that number.
- ☐ **Memory estimate**: For GPT-2 XL in mixed precision at batch 32 and sequence length 1,024, estimate weights, gradients, master weights, optimizer state, and activations, identify the largest term, and decide whether the total fits on an 80 GB accelerator.
- ☐ **Fine-tuning scope**: Explain why full fine-tuning of an 8-billion-parameter model overflows one 80 GB accelerator while low-rank adapters fit, and which memory terms each choice populates.

Pipeline

- ☐ **Slowest stage**: Given host preparation, transfer, and compute rates in batches per second, identify the stage that sets training throughput and compute the accelerator's busy-time fraction.
- ☐ **Training state**: Explain why a training checkpoint file is smaller than the training state in device memory, and what else must be saved to resume a run exactly.

8.5 Identifying Bottlenecks

An accelerator can report near-100 percent activity in `nvidia-smi` while spending half its cycles stalled on memory traffic, padding, or recomputing discarded activations. Knowing what each operation costs on paper does not show which resource actually saturates during a run, and raw device busy-time percentages cannot separate useful training arithmetic from overhead. Model FLOPs utilization (MFU)[15] supplies that comparison by measuring achieved model arithmetic against the hardware's peak rate.

[15] **Model FLOPs utilization (MFU)**: The PaLM paper (Chowdhery et al. 2022) defined MFU as observed throughput divided by the theoretical maximum throughput at peak FLOPs. Its 540-billion-parameter run reported 46.2 percent MFU on 6,144 TPU v4 chips. The remainder is not credited by the model-FLOP numerator, and MFU alone does not identify how much came from memory traffic, communication, pipeline bubbles, recomputation, or other work.

Definition 8.3: Model FLOPs utilization (MFU)

Model FLOPs utilization (MFU) is the efficiency metric $\text{MFU} = O_{\text{model}}/(R_{\text{peak}} \cdot T_{\text{step}})$, where O_{model} is the useful per-step model FLOP count (forward plus backward, excluding rematerialization) and T_{step} is the measured wall-clock time per training step, expressing what fraction of peak hardware throughput is doing useful model computation.

1. Significance: MFU makes the effective-throughput term in the iron law concrete. For GPT-2 XL (1.5B parameters) processing 32,768 tokens per step on an H100 (989 TFLOP/s dense FP16/BF16 Tensor Core peak), a 0.60 s step gives model FLOPs divided by the available peak-rate FLOP budget of approximately 0.50 (50 percent). The remaining 50 percent is not credited as model FLOPs and can reflect memory traffic, communication, bubbles, non-model operations, or idle time. Diagnosing that gap requires a profile rather than a universal threshold.
2. Distinction: Hardware utilization reports whether device engines are active, whereas MFU credits the analytical model-FLOP estimate used in its numerator. It can therefore exclude recomputation and padding even when those operations keep the device busy. Comparisons across accelerators remain sensitive to the selected peak-throughput specification and FLOP-counting convention.
3. Common pitfall: A frequent misconception is that 100 percent MFU is a realistic sustained target. Memory traffic, communication, and non-matrix work keep end-to-end runs below peak. Reported values depend on the model, precision, batch shape, hardware, and MFU definition, and can improve with FlashAttention and careful tuning.

Three distinct metrics are commonly referred to as "utilization" in training systems, and each addresses a different operational question. The hardware efficiency factor η_{hw} in the training-time law (equation 8.1) represents the end-to-end fraction of theoretical peak compute converted into useful model arithmetic across an entire run, whereas MFU measures this same ratio over an individual training step as the direct empirical estimator for η_{hw}. In contrast, the accelerator busy-time fraction of equation 8.7, the metric reported by standard operating-system and vendor monitoring utilities such as `nvidia-smi`, indicates only whether execution cores are active, without distinguishing useful matrix products from memory-stall cycles, padding overhead, or rematerialization. Consequently, busy-time utilization serves only as an upper bound on MFU: a data-starved pipeline has a low busy-time fraction, whereas a well-fed but memory-stalled workload shows near-100 percent busy-time utilization alongside low MFU.

Training bottlenecks can be investigated through three categories aligned with the D·A·M taxonomy (Data, Algorithm, Machine). Table 8.7 connects each axis to a common training bottleneck, its observable symptoms, and relevant optimization techniques. The axis names where the remedy usually lives rather than what the bottleneck is. Compute-bound and memory-bound execution are both

mismatches between an algorithm and a machine, as the ridge point showed, but a compute-bound kernel is most often relieved by asking for less or cheaper arithmetic (Algorithm), a memory-bound kernel by changing how the machine stages and moves bytes (Machine), and an input stall by repairing the data path (Data). These are diagnostic starting points, not exclusive mappings.

↪ The full D·A·M diagnostic framework, with its troubleshooting matrix and scorecard, is collected for reference in Appendix A.

Table 8.7: D·A·M Taxonomy Applied to Training Bottlenecks: Each row associates a D·A·M axis with one common bottleneck and its characteristic symptoms. Profiling reveals the workload's limiting constraint and guides practitioners to an appropriate optimization technique.

D·A·M Axis	Bottleneck	Symptoms	Primary Solutions
Algorithm	Compute-bound	High arithmetic-unit activity; additional bandwidth does not improve throughput; arithmetic throughput is limiting	Reduced precision, more efficient algorithms, faster compute hardware
Machine	Memory-bound	High memory-bandwidth use relative to arithmetic throughput; kernels stall on data movement	Kernel fusion, memory-efficient attention, reduced precision formats
Data	Data-bound	Periodic accelerator utilization drops to near-zero; CPU fully busy during gaps; pipeline cannot feed GPU fast enough	Prefetching, pipeline overlap, faster storage, DataLoader parallelism

Among these three regimes, data-bound stalls are the most frequently misdiagnosed because they manifest on the accelerator as idle gaps rather than saturated execution units. When host-side ingestion, decoding, or preprocessing cannot sustain the throughput demanded by device memory buses, the accelerator starves.

Example 8.2: The GIL-locked GPU

Scenario: An engineering team builds a PyTorch data-loading pipeline using standard Python `threading` for CPU-bound image preprocessing that holds the global interpreter lock (GIL).

Diagnosis: The GIL serializes these Python-level preprocessing threads, making host preparation time ($T_{\text{prep}} \approx 250$ ms) far exceed GPU execution time ($T_{\text{GPU}} \approx 12$ ms), reducing the accelerator busy-time fraction to about 4.6 percent in this serial preparation-and-compute model. Here the fraction measures time spent computing, not arithmetic efficiency relative to peak throughput.

Systems lesson: Python CPU work that holds the GIL can serialize threads and starve GPUs; native operations may release it. Process workers (`num_workers > 0`) avoid interpreter serialization, while the NVIDIA Data Loading Library (DALI) can move eligible work off the host.

In the execution timeline captured by figure 8.6, wide white intervals separate brief GPU stream activity bursts, marking periods where accelerator compute engines idle while CPU-side Python threads assemble the next batch.

Two classes of tools expose these signatures. Framework-level profilers record a timeline of operations, memory allocations, and input-pipeline stalls, which locates idle gaps and attributes them to host work. System- and kernel-level GPU profilers record kernel execution, memory transfers, synchronization points, and per-kernel counters such as achieved memory throughput and arithmetic-unit occupancy, which explains why a kernel that runs is slow. The method below uses the first class to find where time goes and the second to explain it.

↪ How profilers and benchmark harnesses turn these measurements into reproducible results is developed in section 12.5.

A disciplined profiling workflow proceeds top-down from system timelines to kernel-level performance counters, mapping observable profiler traces to the D·A·M taxonomy in table 8.7. First, timeline visualization locates idle intervals between kernel invocations. Correlating these gaps with batch preparation, transfers, host dispatch, and synchronization establishes what prevents the next kernel from running. When input starvation is confirmed, remedies include asynchronous prefetching, multi-process workers, or offloading preprocessing to the accelerator to overlap data movement with compute. Second, when kernels execute contiguously without host-induced gaps, memory throughput relative to peak bandwidth distinguishes memory-bound from compute-bound



Figure 8.6: Data-Bound Profiler Trace: In this timeline trace, isolated GPU compute blocks are separated by prominent white intervals. These idle gaps mark periods where accelerator streams stall waiting for CPU-side batch preparation and host-to-device transfers, providing a characteristic signature of an input-bound pipeline.

execution. If HBM traffic saturates while arithmetic units idle, the workload is memory-bound; kernel fusion and memory-efficient attention relieve this constraint by retaining intermediate tensors in on-chip SRAM rather than writing them back to global memory. Finally, if tensor cores operate near theoretical capacity and memory buses have spare bandwidth, the workload is compute-bound, where throughput gains require lower numerical precision, such as BF16 or FP8, to increase peak arithmetic throughput.

Applied to the chapter's two running cases, the method gives two different first moves. Before profiling anything, the FP32 GPT-2 XL step at batch 32 does not fit in an H100's memory, a capacity limit that the memory techniques below must address first. Once it fits, the timeline shows the input gaps. With single-core tokenization, host preparation takes about 655.4 ms per batch against a device step of about 596 ms (example 8.1), so a serial pipeline keeps the device stage busy only about 48 percent of each step, a Data-axis diagnosis that no kernel optimization can fix. ResNet-50 on the H100 reaches the same axis by a different route: the host cores available per accelerator supply too few decoded images, leaving the accelerator busy only about 52 percent of the time. A diagnosis names the binding resource; it does not remove it. These two profiles, read against the training-time law and the memory constraint, determine which intervention to apply first and which ones would change nothing.

8.6 Pipeline Optimizations

Even well-designed training pipelines rarely approach hardware peak throughput without targeted optimization: at a typical model-FLOP utilization of 30 percent to 50 percent, an H100 rated at 989 TFLOP/s delivers only 296.7 TFLOP/s–494.5 TFLOP/s of useful model arithmetic. These derived scenario values reflect the gaps between peak and realized throughput reported across published large-model systems (Narayanan et al. 2021; Chowdhery et al. 2022). Systems such as SuperNeurons show one version of this problem, in which memory-management bottlenecks prevent larger networks from training efficiently unless the runtime actively optimizes activation storage and transfer (L. Wang et al. 2018). Closing that utilization gap requires matching a targeted systems intervention to the specific resource that profiling shows is saturated: data movement, arithmetic throughput, or memory capacity. Table 8.8 extends the D·A·M bottleneck classification of table 8.7 by matching each bottleneck to the technique that removes it, the term of the training-time law that the technique moves, and its effect on memory, which decides feasibility rather than appearing in the law.

Table 8.8: Matching Optimizations to Bottlenecks: Each technique removes one profiled bottleneck by moving one factor of the training-time law, equation 8.1, and changes memory, which decides whether a configuration runs at all. Kernel fusion is developed with the frameworks that apply it.

Bottleneck the profile shows	Technique	Law term moved	Memory effect	Developed in
Accelerator idle while input is prepared	Prefetching and overlap	$\eta_{\text{hw}} \uparrow$	Prefetch buffers ↑ in host or device memory	section 8.6.2
Arithmetic rate at FP32	Mixed precision (FP16/BF16)	$R_{\text{peak}} \uparrow$; bytes moved ↓	Activations ↓; static state unchanged	section 8.6.3
Attention's quadratic HBM traffic	FlashAttention (IO-aware tiling)	$\eta_{\text{hw}} \uparrow$; same model FLOPs	Attention scores ↓ from quadratic to linear	section 8.6.4
Batch too large for memory	Gradient accumulation	η_{hw} may ↓ at smaller micro-batches	Activations ↓ to one micro-batch	section 8.6.5.1
Activations exceed memory	Activation checkpointing	$\eta_{\text{hw}} \downarrow$ (recomputation adds work)	Activations ↓ to segment boundaries	section 8.6.5.1
Kernel launches and intermediate traffic	Kernel fusion	$\eta_{\text{hw}} \uparrow$	Intermediates ↓	section 7.3.2

Because training throughput is governed by the system's binding constraint, applying an optimization to a nonbinding subsystem yields no end-to-end speedup. Accelerating tensor arithmetic with lower precision or FlashAttention cannot reduce step time if compute pipelines remain stalled waiting for batches from the host. Conversely, expanding prefetch buffers provides no benefit when execution is bound by HBM capacity or backward-pass recomputation. Profiling must therefore identify whether the workload is bound by input supply, memory capacity, memory bandwidth, or arithmetic throughput before an intervention is selected.

Although the governing constraint shifts with system scale (a 100 GB model faces capacity and interconnect limits that never bind a 1 GB model), identifying and mitigating each bottleneck follows consistent physical principles. Data movement latency emerges when host storage I/O and PCIe transfers cannot stage batches fast enough to keep accelerator compute units saturated. Computational throughput falls below peak hardware rates when operations execute in FP32 instead of using lower-precision Tensor Cores, or when non-fused kernels repeatedly transfer intermediate activations across the memory bus. Memory capacity constraints bind execution because accelerator HBM must simultaneously hold model parameters, optimizer states, gradients, and forward activations. Exhausting this memory caps both model depth and per-device batch size.

Because system resources are tightly coupled, relieving one constraint often exposes another. This is the conservation-of-complexity heuristic from the opening of Part I, which holds that complexity tends to move among Data, Algorithm, and Machine rather than disappear. For example, resolving host I/O stalls by aggressively prefetching batches consumes host memory; separately staged device queues also consume accelerator memory. Conversely, shrinking per-device batch sizes to fit within HBM limits lowers the arithmetic intensity of matrix multiplications, shifting execution from compute-bound matrix math into memory-bandwidth stalls. For GPT-2 XL, the profile above already ranks the first moves. If counters confirm that attention is bandwidth limited, IO-aware attention tiling (FlashAttention) targets that constraint; asynchronous prefetching hides data ingestion behind computation, and mixed precision accelerates eligible tensor operations while reducing storage for activations retained in the lower-precision format. The optimization challenge is to diagnose the governing bottleneck from execution profiles and apply targeted interventions without creating a more severe constraint elsewhere.

8.6.1 Systematic optimization framework

Tuning a training pipeline without a profile wastes work on resources that are not limiting step time or shifts the bottleneck to a neighbor. A disciplined optimization pass therefore proceeds in three steps: profile the step to locate the binding constraint, pick an intervention that relieves that specific limit, and check how combined interventions compete for shared memory and bandwidth.

First, profiling tools such as PyTorch Profiler, TensorFlow Profiler, or NVIDIA Nsight Systems record timestamps and hardware counters across each iteration. Traces capture kernel execution times on accelerator execution units, DMA transfers across PCIe or NVLink buses, and host CPU thread scheduling. Comparing elapsed time across these lanes isolates which subsystem sets the critical path. When data-loading intervals lie on the critical path and coincide with accelerator input waits, as in the GPT-2 XL profile above, the run is data-starved; speeding up matrix multiply kernels yields almost no wall-clock gain until input delivery keeps up.

Second, each diagnosed bottleneck points to a specific remedy. When the accelerator idles waiting for batches, asynchronous prefetching and double-buffering over PCIe overlap host data preparation with device compute. When kernel runtimes are bound by HBM bandwidth rather than arithmetic, kernel fusion and tiled algorithms such as FlashAttention keep intermediate activations in on-chip SRAM. When HBM capacity runs out, activation checkpointing discards and recomputes activations during the backward pass, or gradient accumulation splits each batch across sequential micro-batches to cap peak allocation. Finally, when streaming multiprocessors saturate with dense matrix math, 16-bit formats (FP16 or BF16) move eligible operations onto Tensor Cores, multiplying peak matrix throughput several-fold over FP32 CUDA cores (table 8.9) while halving the bytes moved per operand.

Third, combining interventions requires checking their side effects on shared hardware resources. Asynchronous prefetching hides data movement latency behind computation, but double-buffered batch queues consume pinned host memory and device HBM. Other pairs reinforce one another: combining 16-bit mixed precision with activation checkpointing cuts activation memory enough to admit a larger per-device batch size, raising the arithmetic intensity of matrix multiplications on underfed Tensor Cores. After each change, a fresh profile verifies that the secondary resource cost has not created a new bottleneck.

Figure 8.7 formalizes this diagnostic playbook into a triage flowchart, using low accelerator activity and high memory allocation as signals for further investigation. Idle gaps require tracing input preparation, host work, and synchronization; capacity pressure motivates memory-saving techniques. When neither signal explains a missed target, kernel profiling distinguishes bandwidth, arithmetic, and launch limits.

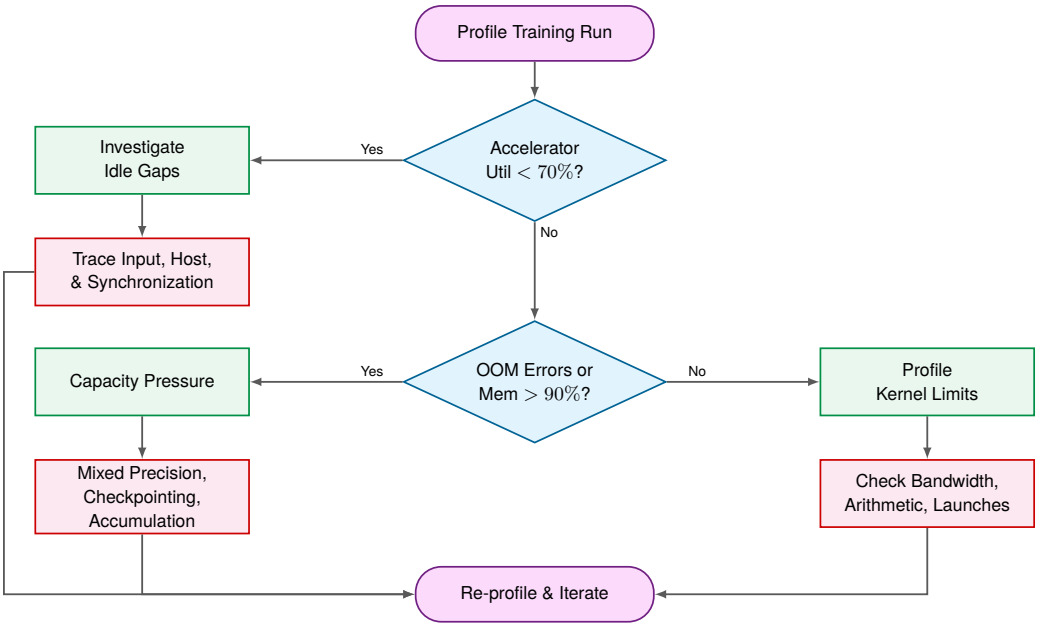


Figure 8.7: Training Optimization Decision Flowchart: A heuristic path from profiling signals to candidate optimizations. Begin with accelerator activity and memory pressure, then investigate the limiting dependency or kernel resource before selecting an intervention. The 70 percent utilization and 90 percent memory thresholds are illustrative triage values rather than universal definitions of data-, memory-, or compute-bound execution.

Each fix moves the bottleneck, so optimization is iterative and governed by Amdahl's law: a speedup applied to one stage is capped by the share of step time that stage occupies, the bound that equation 11.1 formalizes. Resolving input starvation through asynchronous prefetching, for instance, drives accelerator utilization upward until memory bandwidth or compute saturation limits step time. Each intervention requires re-profiling to identify the newly emerged critical path, continuing the cycle until accelerator execution units operate near peak theoretical throughput or application performance targets are achieved.

8.6.2 Data prefetching and overlapping

Prefetching and overlapping put the systematic framework to work on the bottleneck the data pipeline left open. ResNet-50's host workers prepare images more slowly than the H100 consumes them, so the accelerator is busy only about 52 percent of the time (section 8.4.2). That figure already assumes the host prepares the next batch while the accelerator computes the current one. Run serially, preparation and computation add, and the accelerator would be busy only about 34 percent. Overlap recovers the difference by hiding host work behind computation, but it cannot raise the supply rate itself, so closing the rest of the gap for ResNet-50 requires more host workers or decoding on the accelerator. The techniques below are most effective when profiling shows computational units idle while waiting for data.

In ML training, a "Read" stage is rarely a simple disk fetch. For image workloads it encompasses JPEG decoding, random crops, and color jitter applied on CPU cores before the tensor is ready to transfer; for language workloads it encompasses tokenization, subword encoding, and sequence padding. The Gantt schedule in figure 8.8 isolates the serial bottleneck of unbuffered ingestion. The GPU Train row remains completely idle during file-open intervals (Open 1, Open 2) and read stages (Read 1, Read 2), stretching the two-epoch schedule out to 01:35 because compute steps cannot begin until host-side decoding and transfer finish.

Prefetching addresses these inefficiencies by loading data into memory before its scheduled computation time. During the processing of the current batch, framework data pipelines such as `tf.data` load and prepare subsequent batches, maintaining a consistent supply of ready data (Murray et al. 2021).

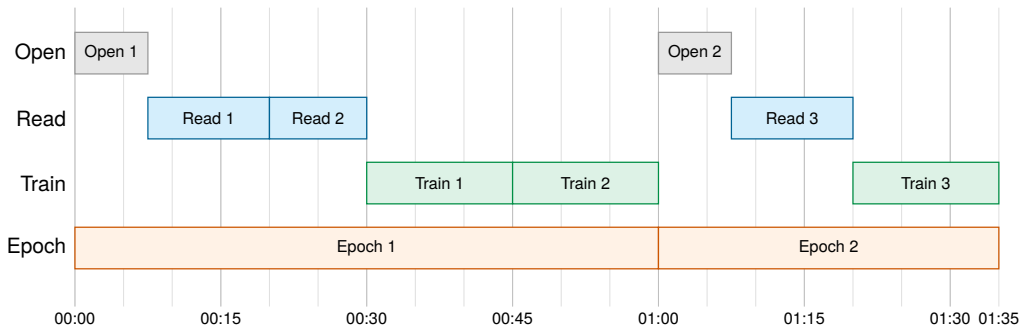


Figure 8.8: Illustrative Sequential Data Fetching: Under the assumed stage durations, file-open, read, and train operations execute serially while the GPU remains idle during file operations. The schedule spans approximately 95 min and provides a schematic baseline for the overlap comparison.

Overlapping extends prefetching by coordinating multiple pipeline stages to execute concurrently. As the system processes the current batch on the accelerator, background host threads load and preprocess subsequent batches. Aligning the serial schedule of figure 8.8 against the overlapped timeline in figure 8.9 demonstrates this latency hiding: concurrent staging of Open 2 and Read 3 behind active execution eliminates idle bubbles between training blocks (Train 1 through Train 3), compressing total wall-clock duration from approximately 95 min down to 65 min (a 31.6 percent reduction). Actual gains depend on stage balance and available concurrency.

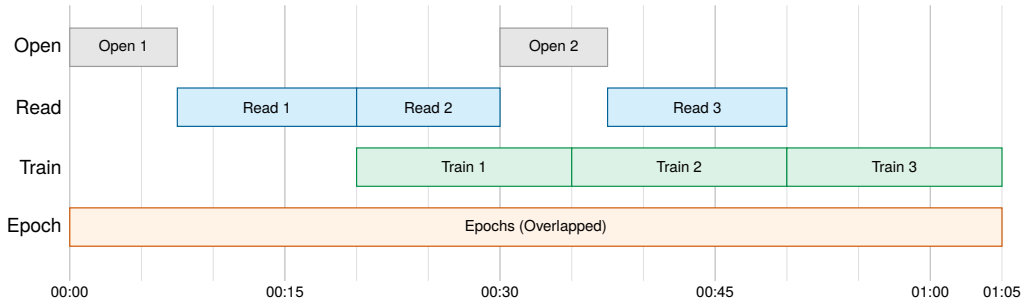


Figure 8.9: Illustrative Overlapped Data Prefetching: File-open and read work overlaps accelerator training while preserving the same individual stage durations as the serial schedule. The schematic schedule spans approximately 65 min, compared with 95 min for the serial schedule.

8.6.2.1 Prefetching mechanics

The overlapped schedule requires enough host workers and buffering to make the next batch ready before the device requests it. Worker count controls preparation capacity; queue depth absorbs variation in preparation time. Neither changes the slowest-stage bound established by equation 8.6.

Machine learning frameworks expose the control variables for this trade-off. Listing 8.5 demonstrates PyTorch's DataLoader configuration, where num_workers = 4 sets preprocessing parallelism and prefetch_factor = 2 maintains a buffer of 8 batches ready for accelerator consumption.

Listing 8.5: DataLoader Prefetch Configuration: Four workers with a prefetch factor of two maintain up to eight batches in the host-side prefetch window.

```
loader = DataLoader(
    dataset, batch_size=32, num_workers=4, prefetch_factor=2
)
```

The parameters `num_workers` and `prefetch_factor` are not generic performance toggles. They determine CPU parallelism and buffer depth, which means they shift pressure from accelerator idle time to host memory and CPU scheduling. The third knob, `pin_memory=True`, lets host-to-device copies overlap accelerator execution through DMA; its cost is page-locked host memory that the rest of the system can no longer page.

↩ The DataLoader's three knobs, worker count, prefetch depth, and pinned memory for DMA transfers, are established in section 7.5.3.

Buffer management is the main trade-off. A buffer that is too small causes the GPU to wait for data preparation, reintroducing the idle time prefetching was meant to remove. An overly large host-side buffer consumes memory that could otherwise support input processing and other host work. The right configuration coordinates CPU preparation, storage I/O, and GPU computation so each resource has work ready at the moment it can use it. These techniques yield the greatest benefit when storage access is slow, preprocessing is complex, or datasets are large.

8.6.2.2 Practical considerations

Prefetch buffers and overlap depth are tunable, so the same machinery adapts to whichever stage binds, whether slow storage, limited network bandwidth, or computational throughput. Prefetching and overlapping deliver the greatest gains when preprocessing lies on the critical path but can run concurrently with model computation. In an illustrative image pipeline, random cropping (10 ms), color jittering (15 ms), and normalization (5 ms) total 30 ms per batch. Overlap can hide up to the concurrent accelerator-computation time; any excess remains on the critical path. NLP workloads similarly benefit when tokenization and subword processing would otherwise block the training loop.

The primary trade-off is memory: prefetch buffers consume host memory, or device memory when a separate device-side queue is used, in proportion to buffer depth and batch size. With batch size 256 high-resolution images (1024×1024 pixels), one buffered batch requires approximately 3.2 GB. With num_workers = 4 and prefetch_factor = 2, the 8-batch prefetch window can hold about

25.8 GB. Tuning these parameters requires empirical testing because excessive workers contend for CPU, storage, and memory resources, while insufficient buffering reintroduces data stalls. Start with a modest worker count, increase it while measuring throughput and memory, and stop when additional workers no longer reduce accelerator idle time. When the input pipeline already exceeds compute demand, deeper prefetching adds complexity without improving throughput.

8.6.3 Mixed-precision training

Running a modern training job entirely in 32-bit floating point (FP32) leaves the accelerator's 16-bit Tensor Cores idle and doubles the byte footprint of every forward activation kept in HBM. Mixed-precision training removes both bottlenecks by executing matrix multiplications and storing activations in a 16-bit format while retaining 32-bit precision for sensitive accumulations and parameter updates. Section D.4.1 compares FP32, FP16, BF16, FP8, and INT8 precision-range trade-offs. A training recipe commonly combines FP32 with either FP16 or BF16; it does not ordinarily use FP16 and BF16 interchangeably in the same path. The resulting speed, memory, and accuracy effects depend on the workload and hardware (Micikevicius et al. 2018; Wang and Kanwar 2019; Kalamkar et al. 2019).

A neural network's stored FP32 weights require 4 bytes per parameter, while FP16 and BF16 weights use 2 bytes. For a model with 10^9 parameters, the working copy of the weights shrinks from 4 GB to 2 GB. That saving does not carry over to the static training state. The common FP16 recipe keeps FP32 master weights and FP32 Adam moments, so it spends 2 + 2 + 4 + 8 = 16 bytes per parameter on weights, gradients, master copy, and moments, exactly what FP32 Adam spends on its 4 + 4 + 8. Mixed precision's memory benefit therefore comes from the tensors that really are stored in 16 bits, above all the activations, together with the halved bytes moved for every 16-bit read, write, and gradient exchange.

Mixed precision's throughput benefit is larger than its memory benefit. Figure 8.10 plots peak dense throughput by number format across four generations of data center accelerators. From V100 to B200 the FP32 rate grew 4.8×, the FP16/BF16 tensor rate grew 18×, and the lowest supported format, 4-bit floating point (FP4) on B200, delivers 72× the V100's FP16 tensor rate. Hardware gains now arrive almost entirely through tensor units built for lower precision. On B200 the FP16/BF16 tensor path is 30× faster than FP32, so a training job that stays in FP32 leaves nearly all of the chip's arithmetic idle.

The numerical differences between these formats shape their use cases. Table 8.9 shows that BF16's 8-bit exponent gives it approximately the same normal exponent range as FP32, while FP16's 5-bit exponent gives a minimum positive normal value of about 6.1×10^{-5} and a maximum finite value of $65{,}504$. FP16 subnormals extend toward 6×10^{-8}, although their treatment depends on the operation and hardware mode. FP32 provides about seven decimal digits of precision, FP16 about three, and BF16 about two to three. BF16 therefore trades mantissa precision for the exponent range that often simplifies deep-learning training.

Table 8.9: Precision Format Comparison: The choice between FP16 and BF16 depends on whether dynamic range (BF16's strength) or precision (FP16's advantage) matters more for the specific workload. Minimum normal values shown are the practical thresholds for training, as subnormal values may flush to zero on many GPUs. On the H100, dense FP16 and BF16 Tensor Cores reach 15× the FP32 CUDA-core peak.

Property	FP32	FP16	BF16
Exponent bits	8	5	8
Mantissa bits	23	10	7
Min normal value	10^{-38}	6.1×10^{-5}	10^{-38}
H100 peak throughput ratio	1×	15×	15×

The choice between formats depends on model characteristics and hardware support. BF16's wider exponent range reduces overflow and underflow concerns, while FP16 provides more mantissa bits but often requires loss scaling. Framework autocast policies and fused kernels choose higher-precision accumulation for numerically sensitive operations when needed; loss reduction, softmax,

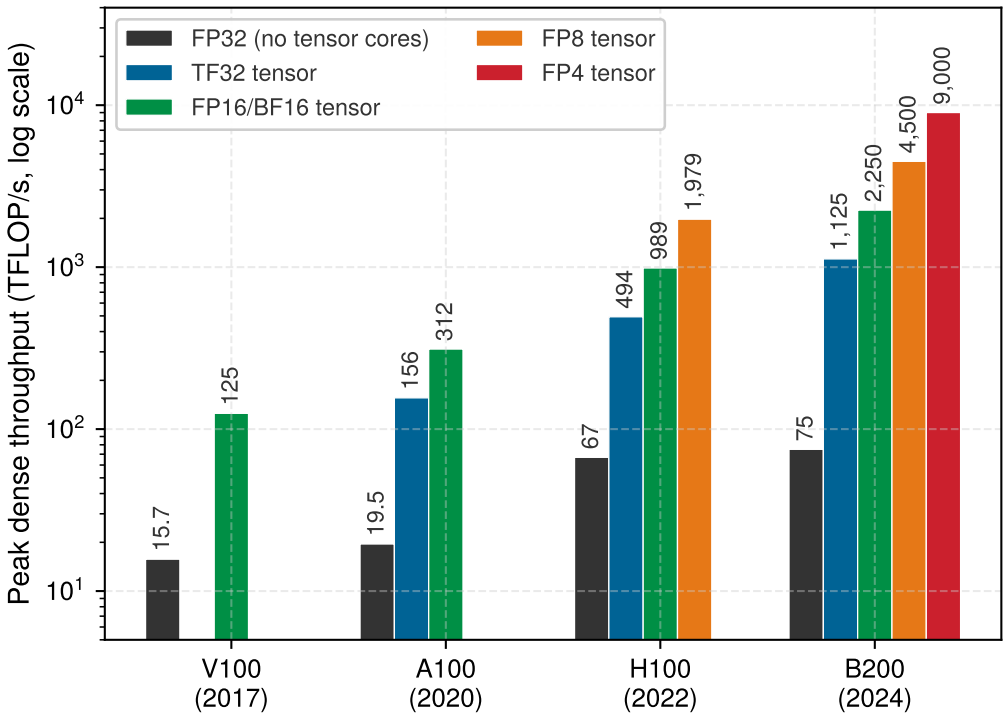


Figure 8.10: Peak Throughput by Number Format: Peak dense throughput of four generations of data center accelerators in each number format they support, on a logarithmic axis. Each step down among the tensor formats roughly doubles the peak rate on the same silicon, the first step from FP32 onto the tensor paths gains far more, and nearly all of the generational gain arrives in the 16-bit and lower formats rather than in FP32. Vendor peak specifications (NVIDIA Corporation 2020b, 2021a, 2023b, 2024).

normalization, and optimizer updates are common candidates, but their exact execution dtype is implementation-dependent.

Figure 8.11 traces this conventional FP16 execution cycle across six numbered stages: FP32 master weights are downcast to FP16 for the forward pass, loss scaling shifts small gradient values into representable range before backpropagation, and the resulting scaled gradients are unscaled and cast back to FP32 prior to the parameter update. Modern framework implementations may store or accumulate particular tensors differently, and BF16 recipes commonly omit loss scaling because of their wider exponent range.

Modern accelerator architectures pair dense matrix execution units with reduced-precision numerical formats to maximize arithmetic throughput. NVIDIA introduced Tensor Cores for FP16 operations in the Volta architecture (NVIDIA 2017) and added native BF16 execution in Ampere (NVIDIA Corporation 2020a), matching the native BF16 support Google developed for Tensor Processing Units (Wang and Kanwar 2019). These specialized functional units execute low-precision matrix multiply-accumulate operations at four to sixteen times the rate of standard 32-bit floating-point ALUs.

Mixed-precision training turns those hardware capabilities into a split numerical policy. Eligible matrix multiplications and convolutions execute in FP16 or BF16 on tensor units, while reductions, accumulations, and parameter updates retain higher precision. The performance gain comes from both arithmetic throughput and data movement: lower-precision units pack more multiply-accumulates into each cycle and each unit of silicon area, while narrower tensors halve data transfer volumes across caches and high-bandwidth memory. Higher precision remains necessary where wide reductions, small parameter updates, or sensitive normalizations can accumulate rounding error. Framework autocast policies enforce this split dynamically, selecting lower-precision execution for compute-intensive matrix multiplications while preserving FP32 for numerically fragile operations.

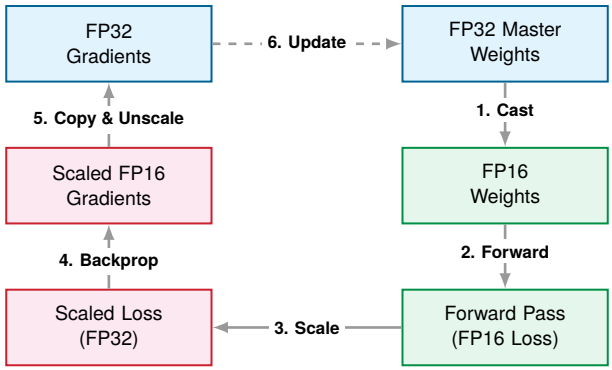


Figure 8.11: Conventional FP16 Mixed-Precision Training: A schematic six-step recipe casts FP32 master weights for lower-precision computation, scales the loss, computes scaled gradients, then unscales them for an FP32 update. Framework implementations may vary, and BF16 usually does not require this loss-scaling path.

8.6.3.1 Loss scaling

FP16's 5-bit exponent limits its representable dynamic range, creating severe underflow hazards during backpropagation. When computing gradients across deep networks, successive chain-rule multiplications frequently produce gradient magnitudes smaller than FP16's minimum positive normal value ($2^{-14} \approx 6.1 \times 10^{-5}$). While IEEE 754 FP16 defines subnormal numbers down to $\approx 6 \times 10^{-8}$, hardware accelerators often flush subnormal inputs and outputs to zero to prevent pipeline stalls. When gradients underflow to zero, weight updates stall and training fails to converge.

Loss scaling counters underflow by exploiting the linearity of differentiation: $\nabla_\theta(s_{\text{loss}} \cdot \mathcal{L}) = s_{\text{loss}} \cdot \nabla_\theta \mathcal{L}$. Multiplying the scalar loss $\mathcal{L}$ by a large scaling factor s_{loss} (such as 2^{10} or 2^{15}) prior to backpropagation shifts the gradient distribution rightward into FP16's normal representable range. Once backpropagation completes, the accumulated gradients are divided by s_{loss} (unscaled) before applying the optimizer update.

Deep learning frameworks automate this process through dynamic loss scaling; section 7.4.3.3 describes the `GradScaler` machinery that scales the loss, unscales the gradients, and skips any step whose gradients overflow. The training decision is how the scale factor adapts. Because gradient norms fluctuate during training, a static scale factor risks either underflow (if s_{loss} is too small) or numerical overflow past FP16's maximum finite value of $65{,}504$ (if s_{loss} is too large). A dynamic scaler therefore skips each overflowing step so that nonfinite gradients never reach the FP32 master weights, halves s_{loss} at each skip, and doubles s_{loss} after a configurable growth window without overflow (such as 2,000 consecutive steps). Together these adjustments keep gradients near the upper end of the dynamic range without saturating.

8.6.3.2 Mixed-precision benefits

Mixed-precision training delivers gains across three interacting system dimensions: memory footprint, arithmetic throughput, and communication volume. Halving operand width cuts the storage of every 16-bit tensor by 50 percent, so a one-billion-parameter model's working weights drop from 4 GB to 2 GB. As the 16-bytes-per-parameter accounting above showed, FP32 master weights and moments give that saving back in the static state, so the footprint that actually shrinks is the activations. Computationally, reduced precision routes dense matrix multiplication kernels to high-throughput Tensor Core pipelines (section 8.6.3.3). In distributed topologies, transferring 16-bit rather than 32-bit gradients halves collective AllReduce traffic across the interconnect. These effects compound: reclaimed activation memory permits larger micro-batches, Tensor Cores accelerate kernel execution, and narrower tensors reduce gradient exchange latency. The following case study evaluates these memory effects on GPT-2 XL at batch 32, the configuration the activation notebook left far over the H100's capacity.

Napkin Math 8.6: GPT-2 mixed precision memory savings

Problem: What does mixed precision save when GPT-2 XL (1.5B parameters) trains at batch 32 and sequence length 1,024 on one H100, and what does it leave unchanged?

FP32 baseline:

- Parameters and gradients (FP32): 12 GB (6 GB parameters + 6 GB gradients)
- Activations: ~574 GB
- Optimizer states (Adam first and second moments in FP32): 12 GB
- Total: ~598 GB

FP16 mixed precision:

- Parameters (FP16): 1.5B parameters at 2 bytes each = 3 GB
- Activations (FP16): ~287 GB
- Gradients (FP16): 3 GB
- FP32 master weights: 6 GB (for precise optimizer updates)
- Optimizer states (Adam first and second moments in FP32): 12 GB
- Total: ~311 GB, about 3.6× the H100's 80 GB

Systems insight: Mixed precision halves the activations and leaves the static state unchanged at 24 GB. At batch 32 the remaining activations still dominate, and most of them are the quadratic attention-score matrices, which FlashAttention (section 8.6.4) removes next.

Memory savings alone do not guarantee stable training. In the FP16 training recipe used here, maintaining FP32 master weights is necessary because parameter updates $\Delta w = -\eta \cdot g$ (learning rate times gradient) are typically orders of magnitude smaller than the weight values w. In FP16, where the 10-bit mantissa provides roughly three decimal digits of precision, adding an update that is more than $2^{11} \approx 2048$ times smaller than the weight magnitude results in arithmetic swamping: the updated weight rounds back to its original value. This loss of an increment through rounding is distinct from the update value itself underflowing. Retaining master weights in FP32 preserves these incremental updates across thousands of optimizer steps, casting to 16 bits only when staging weights for the forward pass. Autocast policies similarly isolate wide reductions, layer normalizations, and softmax layers in FP32 to prevent catastrophic cancellation.

Once numerical stability is established, reduced-precision arithmetic and reduced memory traffic can shorten execution when they reduce work on the critical path. Comparing complete training recipes does not isolate the contribution of precision alone; model quality, batch size, optimizer representation, and activation policy must be accounted for. The following scenario models these compound throughput, schedule, and infrastructure cost gains for a GPT-2 XL training run.

Napkin Math 8.7: GPT-2 mixed precision throughput and cost

Problem: Holding utilization fixed, how much faster and cheaper is GPT-2 XL's run on one H100 when its matrix products run in BF16 instead of in 32-bit formats?

Peak rates by path: FP32 on scalar units 67 TFLOP/s; TF32 on Tensor Cores 494 TFLOP/s; BF16 on Tensor Cores 989 TFLOP/s.

Run time and cost at 50 percent utilization and $4/hour per accelerator:

- FP32: about 518 days, $49,751
- TF32: about 70 days, $6,748
- BF16: about 35 days, $3,370

Quality gate: The comparison is valid only if a controlled validation run shows that the BF16 recipe meets the same acceptance criteria as the 32-bit baseline. The scenario does not assume a particular perplexity difference.

Systems insight: The peak ratios, 2× over TF32 and 15× over FP32, bound the gain rather than predict it. Holding utilization fixed assumes that every part of the step speeds up with the matrix units, but elementwise kernels, optimizer updates, and input waits do not run on Tensor Cores, so by Amdahl's law the measured speedup is smaller. A real decision requires measured throughput, billing, and quality validation for the target workload.

Realizing these speedups requires empirical profiling rather than assuming automatic acceleration. For small models or memory-bound workloads not dominated by large matrix multiplications, data type casting and kernel launch overheads can offset the arithmetic throughput advantage. Furthermore, whether an accelerator supports native BF16 or requires FP16 loss scaling depends on the underlying microarchitecture.

8.6.3.3 Mixed-precision hardware support

Specialized matrix execution units deliver the wall-clock speedup of mixed precision by spending fewer bits per operand. Accelerators pair these low-precision multiply units with higher-precision accumulation registers to maintain numerical stability during dense matrix reductions.

The Volta-generation Tensor Cores introduced earlier in this section perform 4×4 matrix multiply-accumulate operations in a single clock cycle, a datapath that section 11.4.2 examines in detail. For FP16 inputs, a single Tensor Core executes:

$$\mathbf{D}_{\text{tc}} = \mathbf{A}_{\text{tc}} \times \mathbf{B}_{\text{tc}} + \mathbf{C}_{\text{acc}}$$

where $\mathbf{A}_{\text{tc}}$ and $\mathbf{B}_{\text{tc}}$ are FP16 inputs and $\mathbf{C}_{\text{acc}}$ and $\mathbf{D}_{\text{tc}}$ may use FP32 accumulation. Higher-precision accumulation reduces rounding error, although it does not eliminate numerical error or cancellation.

Hardware peak specifications define the theoretical ceiling on this acceleration. The reference H100 exposes 67 TFLOP/s of FP32 throughput on standard CUDA cores and 989 TFLOP/s of dense FP16 or BF16 Tensor Core throughput, a 15× peak ratio, and adds FP8 Tensor Cores at 1,979 TFLOP/s without sparsity, about 2× its FP16/BF16 rate. These figures represent hardware ceilings rather than end-to-end training guarantees: matrix multiplications map directly to Tensor Cores, but data movement, non-Tensor-Core kernels, communication, loss scaling, and optimizer updates remain. While the batched matrix multiplication $\mathbf{QK}^T$ in transformer self-attention ($2 \times B \times N_{\text{heads}} \times S^2 \times d_{\text{head}}$ FLOPs) maps directly to these tensor units, Amdahl's law dictates that the remaining memory-bound elementwise operations, communication collectives, and optimizer updates bound the realized end-to-end speedup.

Brain Float 16 (BF16) maintains FP32's 8-bit exponent while reducing the mantissa to 7 bits. This design choice prioritizes dynamic range preservation over precision, which matters for gradient-based learning where values span many orders of magnitude. Google's TPUs natively support BF16, while NVIDIA's Ampere architecture (A100) and newer provide full hardware support.

The range advantage of BF16 appears exactly where loss scaling was needed for FP16. A gradient below FP16's smallest normal value of 6.1×10^{-5}, which some hardware flushes to zero, is still an ordinary normal number in BF16, whose smallest normal value of about 10^{-38} matches FP32's exponent range. BF16 therefore usually avoids the loss scaling needed by FP16, although accumulation precision and rounding still matter.

NVIDIA's Hopper architecture (H100) adds FP8 Tensor Core paths for 8-bit floating-point values (1.98 PFLOP/s on H100 dense vs. 0.99 PFLOP/s for FP16 dense, without sparsity); FP8's two bit layouts and the per-tensor scaling they require are developed in section 10.3.1. Table 8.10 summarizes when each precision is appropriate:

Table 8.10: Precision Selection Guide: Format choice depends on hardware support, kernel coverage, numerical behavior, and validation results. FP8 requires an explicit scaling strategy; BF16 provides a wider exponent range than FP16; FP16 often uses loss scaling; and FP32 remains available for sensitive operations or unsupported lower-precision paths.

Precision	When to Use	Hardware Requirement
FP8	Throughput-oriented training with validated scaling	Supported FP8 hardware
BF16	Wide exponent range without FP16 loss scaling	Supported BF16 accelerator
FP16	Higher mantissa precision with managed loss scaling	Supported FP16 accelerator
FP32	Higher precision or unsupported lower-precision path	GPU

In addition to accelerating compute-bound matrix operations, reduced precision relieves memory bandwidth bottlenecks by altering operational arithmetic intensity. On modern accelerators, large matrix multiplications reside in the compute-bound regime of the roofline model, whereas element-wise operations (such as bias additions, layer normalization, and residual connections) reside firmly in the memory-bound regime. The H100 illustrates the physical imbalance between arithmetic capability and memory bandwidth: its HBM3 bus supplies 3,350 GB/s, whereas sustaining its 67 TFLOP/s of FP32 throughput without operand reuse would demand 268 TB/s if every FLOP required fresh data from memory. Although operand reuse reduces actual traffic for dense matrix multiplications, operations with low arithmetic intensity remain strictly bandwidth-limited.

For memory-bound kernels, execution time is determined by bytes transferred across the memory bus rather than arithmetic clock cycles. Converting 32-bit operands to 16-bit FP16 or BF16 halves the byte volume transferred across the HBM bus, doubling the theoretical throughput of memory-bound operations even without engaging Tensor Cores. Realized bandwidth gains depend on cache hit rates, kernel fusion, and format conversion costs.

Section 7.4.3.3 describes how `autocast` and `GradScaler` implement the split between 16-bit and FP32 operations inside the framework. The policy decision that remains for the training recipe is which format to run, and listing 8.6 makes it a single switch: the FP16 path enables gradient scaling, and the BF16 path disables it.

Listing 8.6: Mixed-Precision Training: Autocast selects eligible lower-precision operations, while gradient scaling is enabled for the FP16 path and omitted for BF16.

```
import torch

model = TransformerModel().cuda()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
amp_dtype = (
    torch.float16
)  # torch.bfloat16 usually does not need loss scaling
scaler = torch.amp.GradScaler(
    "cuda", enabled=(amp_dtype == torch.float16)
)

for inputs, targets in dataloader:
    optimizer.zero_grad()

    # Automatic precision selection per operation
    with torch.amp.autocast("cuda", dtype=amp_dtype):
        output = model(inputs)
        loss = criterion(output, targets)

    # Scale loss to prevent gradient underflow
    scaler.scale(loss).backward()

    # Unscale gradients before optimizer step
    scaler.step(optimizer)
    scaler.update()  # Adjust scaling factor dynamically
```

Because hardware support for low-precision formats evolved across accelerator generations, the optimal mixed-precision recipe depends on the target architecture. Table 8.11 outlines the precision trade-offs across three generations of NVIDIA GPUs.

Table 8.11: Precision Strategy by GPU Architecture: V100 supports FP16 Tensor Core training, A100 adds native BF16, and H100 adds FP8 paths whose use requires recipe-specific validation.

Architecture	Recommended Precision	Key Considerations
V100 (Volta)	FP16 with loss scaling	No native BF16 Tensor Core path; clipping depends on the training recipe
A100 (Ampere)	BF16 or FP16	BF16 avoids FP16 loss scaling; TF32 can accelerate eligible FP32 matrix operations
H100 (Hopper)	BF16, FP16, or validated FP8	FP8 requires an FP8-aware recipe; 1,979 TFLOP/s is a peak ceiling

Throughput gains across these generations came from new tensor execution units built for narrower formats, and measured speedups also reflect software stacks, kernels, and batch dimensions that changed alongside the hardware. The peak-rate ratios in Notebook 8.7 bound what a format switch can deliver on one H100. In practice, BF16 is the default where the hardware supports it, and FP16 with loss scaling is the fallback.

8.6.4 FlashAttention: IO-aware attention optimization

Mixed-precision training addresses two bottlenecks, compute throughput because Tensor Cores run faster on 16-bit operands, and memory capacity because each value uses fewer bytes. For transformer models, a third bottleneck often dominates, and the roofline analysis has already located it. Counting only its score-matrix traffic, materialized attention in GPT-2 Small reaches about 32 FLOP/byte, well left of the H100 ridge point in figure 8.1, because its quadratic intermediate matrices are written to HBM during the forward pass and read back during backpropagation. Halving the bytes per value does not change that quadratic traffic, so even with reduced precision the compute units can sit idle while the processor waits for data.

The same quadratic term dominates GPT-2 XL's memory. At batch 32 and 1,024 tokens, the stored score, probability, and dropout matrices account for about 201 GB of the 287 GB of mixed-precision activations, about 70 percent. An attention kernel that never stores them leaves 85.6 GB of activations; with the static state the step needs 109.6 GB, still about 24 GB more than the H100's 80 GB (85.9 GB in decimal units), so removing attention's traffic is necessary but not sufficient.

FlashAttention (Dao et al. 2022) addresses this bandwidth bottleneck by treating attention as an IO problem. Processing attention in SRAM-sized tiles avoids materializing the full $S \times S$ matrix in HBM and reduces traffic without changing the mathematical output, which is how the roofline preview moved attention rightward toward, and for favorable shapes past, the ridge.

8.6.4.1 The standard attention memory bottleneck

Standard self-attention materializes an $S \times S$ score matrix and an $S \times S$ probability matrix in off-chip HBM for every attention head, turning long-sequence training into a memory-bandwidth and capacity bottleneck:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{head}}}}\right)\mathbf{V} \tag{8.9}$$

Here, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are the query, key, and value matrices for a sequence of length S, and d_{head} is the key-vector dimension used to scale the dot products before softmax. Because the intermediate product $\mathbf{Q}\mathbf{K}^T$ is written out to HBM before softmax and read back immediately afterward, attention becomes an I/O problem even when the matrix products themselves map cleanly onto Tensor Cores.

The memory bottleneck emerges from materializing the $S \times S$ intermediate matrices for scores and probabilities. For a sequence length of 4,096 with head dimension $d_{\text{head}} = 64$ (typical for a single attention head), the attention score matrix alone requires $4{,}096^2 \times 4$ bytes = 67.1 MB in FP32. This section counts FP32 scores throughout, since autocast policies commonly keep softmax in FP32;

at FP16 every figure halves, as in the roofline estimate, while the quadratic growth stays the same. With 16 heads, this grows to 1.1 GB just for intermediate attention matrices, not including the keys, queries, values, or output tensors.

GPU memory hierarchy creates the opportunity for I/O-aware attention. HBM offers large off-chip capacity, while much smaller on-chip SRAM has substantially higher bandwidth and lower latency. Attention implementations that materialize the quadratic intermediates in HBM must write and later read them during backpropagation. Because that traffic grows with the square of sequence length, it takes a growing share of step time as sequences lengthen.

The backward pass requires the information represented by the attention probabilities, but an implementation may either save those intermediates or recompute them. For $\mathbf{A} = \mathrm{softmax}(\mathbf{S})$ and $\mathbf{S} = \mathbf{Q}\mathbf{K}^T/\sqrt{d_{\text{head}}}$, one gradient expression is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Q}} = \frac{1}{\sqrt{d_{\text{head}}}} \cdot \mathrm{dsoftmax}\left(\frac{\partial \mathcal{L}}{\partial \mathbf{A}}, \mathbf{A}\right) \cdot \mathbf{K}$$

where dsoftmax denotes the softmax Jacobian-vector product. A conventional implementation may save the probability matrix for this computation; saving both probabilities and raw scores is not mathematically required. FlashAttention instead saves compact normalization statistics and recomputes tiled score and probability blocks during backward propagation, avoiding full quadratic intermediates in HBM.

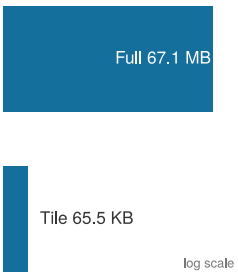
8.6.4.2 IO-aware attention through tiling

FlashAttention eliminates the need to materialize full $S \times S$ attention matrices in HBM by computing attention incrementally through tiling. Instead of computing the entire attention matrix at once, the algorithm partitions **Q**, **K**, and **V** into tiles small enough to fit in fast SRAM, computes attention scores for these tiles, and incrementally accumulates results.

The key algorithmic insight relies on the structure of equation 8.9. Softmax normalizes each row of scores by a sum over the entire row, which seems to require the whole row before any output can be formed, yet the online softmax algorithm removes that requirement. Ordinary softmax, $\mathrm{softmax}(\mathbf{x})_i = e^{\mathbf{x}_i}/\sum_j e^{\mathbf{x}_j}$, is computed stably by first subtracting the row maximum. The online form instead keeps two running statistics per row as blocks of scores arrive: the largest score seen so far, m, and the sum of exponentials taken relative to that maximum, z. When a new block raises the maximum from m to m', everything accumulated so far was exponentiated relative to the old maximum, and multiplying it by $e^{m-m'}$ re-expresses it relative to the new one. The same factor rescales the partially accumulated output. After the last block, dividing the output by z gives exactly the softmax-weighted result, as if the whole row had been seen at once.

Algorithm 8.2 decomposes this computation by partitioning the queries and the keys/values into tiles small enough to fit in SRAM, then streaming over the key/value tiles while maintaining the running softmax statistics needed to assemble each output tile, without ever forming the full score matrix.

No full $S \times S$ score or probability matrix is written to HBM. A 128×128 FP32 score tile, for example, occupies 65.5 KB compared with 67.1 MB for a 4096×4096 matrix. Other inputs, outputs, and running statistics remain, so the tile is not necessarily the largest tensor in the entire kernel.



FlashAttention swaps the full attention matrix for small SRAM tiles.

Algorithm 8.2 FlashAttention: tiled attention with online softmax

Require: queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V}$ for a length-S sequence; tile size b_{tile}
Ensure: attention output $\mathbf{O}$, without materializing the $S \times S$ score matrix

for each query tile $\mathbf{Q}_i$ **do**
  in SRAM: $\mathbf{O}_i^{\text{acc}} \leftarrow \mathbf{0}$, $m_i \leftarrow -\infty$, $z_i \leftarrow 0$
  for each key/value tile $(\mathbf{K}_j, \mathbf{V}_j)$ **do**
    load $\mathbf{Q}_i, \mathbf{K}_j, \mathbf{V}_j$ into SRAM; $\mathbf{S}_{ij} \leftarrow \mathbf{Q}_i \mathbf{K}_j^T / \sqrt{d_{\text{head}}}$
    $m_i' \leftarrow \max(m_i, \text{rowmax}(\mathbf{S}_{ij}))$ ▷ *new stable max*
    $\mathbf{O}_i^{\text{acc}} \leftarrow \mathbf{O}_i^{\text{acc}} \odot e^{m_i - m_i'} + e^{\mathbf{S}_{ij} - m_i'} \mathbf{V}_j$
    $z_i \leftarrow z_i \odot e^{m_i - m_i'} + \text{rowsum}(e^{\mathbf{S}_{ij} - m_i'})$; $m_i \leftarrow m_i'$
    discard $\mathbf{S}_{ij}$ ▷ *never written to HBM*
  end for
  $\mathbf{O}_i \leftarrow \mathbf{O}_i^{\text{acc}} / z_i$; write $\mathbf{O}_i$ to HBM
end for

8.6.4.3 Memory and IO complexity analysis

FlashAttention improves both activation memory and memory IO, which are the limiting costs in bandwidth-bound attention. Standard attention requires $\mathcal{O}(S^2)$ memory to store score matrices $\mathbf{S}$ and attention-probability matrices $\mathbf{A}$ across all sequence positions. FlashAttention reduces the stored activation footprint to $\mathcal{O}(S)$ by keeping only input and output tensors $(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O})$ plus a small SRAM buffer for the current tile.

For $S = 4096$, $d_{\text{head}} = 64$, one materialized FP32 score matrix occupies $4096^2 \times 4$ bytes = 67.1 MB per head. The three FP32 input tensors occupy $(3 \times 4096 \times 64) \times 4$ bytes ≈ 3.1 MB per head, a component-size ratio of 21.3× between one quadratic intermediate and the inputs. The comparison that decides memory is the whole per-head footprint each algorithm keeps for the backward pass. The conventional baseline assumed here stores the inputs, the output, and both the score and probability matrices, about 138.4 MB. FlashAttention stores the inputs, the output, and two numbers per row (the running maximum and sum), about 4.2 MB, roughly 32.7× smaller, and the gap widens as S grows because only the standard footprint is quadratic.

These stored-tensor totals exclude gradient buffers, temporary workspace, and the live SRAM tile, so they are not estimates of peak training memory. Implementations may also retain fewer quadratic intermediates than the conventional baseline assumed here.

The IO pattern changes for the same reason. Standard attention reads $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ from HBM, writes score, probability, and output matrices during the forward pass, then rereads those stored matrices during the backward pass before writing the gradients $d\mathbf{Q}$, $d\mathbf{K}$, and $d\mathbf{V}$ (shorthand for $\partial \mathcal{L} / \partial \mathbf{Q}$ and its counterparts). The resulting HBM traffic includes the same $\mathcal{O}(S^2)$ term that made the activation footprint large. FlashAttention forms score and probability blocks in SRAM and never writes the full $S \times S$ tensors to HBM. In the backward pass, it recomputes the needed blocks rather than reading stored full matrices. The exact HBM IO complexity depends on tile size and available SRAM because key and value blocks are streamed repeatedly across query blocks, so the bound is not simply $\mathcal{O}(S \cdot d)$.

Both approaches require $\mathcal{O}(S^2 d)$ asymptotic FLOPs for dense attention, so FlashAttention's gain comes from traffic, not arithmetic. The resulting reduction in HBM traffic can shift the kernel toward compute-bound execution and produce a net speedup, but the final bottleneck remains shape- and hardware-dependent.

8.6.4.4 Implementation and hardware utilization

FlashAttention's performance gains materialize through careful exploitation of GPU memory hierarchy. Modern frameworks can dispatch eligible attention operations among fused and fallback implementations based on hardware, inputs, and layouts; profiling verifies which path ran and whether it helped. Listing 8.7 contrasts standard and optimized attention implementations.

Listing 8.7: Attention Implementation Comparison: Standard attention materializes the full $S \times S$ matrix in HBM, while FlashAttention uses PyTorch's optimized implementation or the dedicated flash-attn library.

```
import torch
import torch.nn.functional as F


# Standard attention (materializes n-by-n matrix)
def standard_attention(q, k, v):
    # q, k, v: [batch, heads, seq_len, head_dim]
    scores = torch.matmul(q, k.transpose(-2, -1)) / (
        q.size(-1) ** 0.5
    )
    attn = F.softmax(scores, dim=-1)  # n-by-n matrix in HBM
    output = torch.matmul(attn, v)
    return output


# FlashAttention (no n-by-n materialization)
def flash_attention(q, k, v):
    # Can use FlashAttention when inputs and hardware are eligible
    output = F.scaled_dot_product_attention(q, k, v)
    return output


# Explicit FlashAttention 2 (flash-attn library)
from flash_attn import flash_attn_func


def flash_attn_2(q, k, v):
    # q, k, v: [batch, seq_len, heads, head_dim]
    # Different layout for optimized memory access
    output = flash_attn_func(q, k, v)
    return output
```

Dao et al. (2022) reports end-to-end GPT-style training speedups and separate attention-kernel benchmarks showing that IO-aware attention reduces memory traffic and improves runtime. Runtime and memory savings depend on batch size, head count, sequence length, precision, and the kernel implementation, and a capacity threshold still requires the complete training memory budget, including the state outside attention. Later versions refine the same idea: FlashAttention-2 (Dao 2023) reports a further 1.5–2$\times$ speedup through better parallelism, and FlashAttention-3 (Shah et al. 2024) adds asynchronous memory operations and FP8 support on Hopper GPUs.

8.6.4.5 When to use FlashAttention

For transformer training with long sequences, FlashAttention is usually the first attention implementation to test. It often becomes essential once sequence length pushes standard attention into an HBM-capacity or HBM-bandwidth bottleneck, especially around the multi-thousand-token regime where the $S \times S$ attention matrices dominate memory. A100- and H100-class GPUs with fast on-chip SRAM benefit most. The returns diminish for short sequences where tiling overhead is not worthwhile, on older GPU architectures without comparable SRAM bandwidth, and for nonattention architectures such as convolutional neural networks and multilayer perceptrons.

In practice, deep learning frameworks handle much of FlashAttention's integration, but the decision still depends on layout, precision, and memory budget. PyTorch's `scaled_dot_product_attention` can dispatch to FlashAttention when hardware, inputs, and backend constraints permit; hand-written attention must use an eligible primitive. Three practical checks determine whether the fast path actually runs and pays off: tensor layouts must match library expectations (contiguous memory and the expected dimension ordering), execution must use FP16 or BF16 (because FlashAttention kernels target mixed precision), and gradient checkpointing should be combined with FlashAttention whenever additional activation-memory savings are needed beyond the attention scores.

The integration is often a single-line change, swapping a manual attention call for `F.scaled_dot_product_attention`, as listing 8.7 shows. The engineering decision is still the same one established by the roofline analysis: use the optimized primitive when the bottleneck is HBM traffic, and let the library manage the tiling and SRAM scheduling needed to remove it.

8.6.4.6 IO-aware design beyond attention

FlashAttention is an instance of **IO-aware algorithm design**: because modern accelerators are compute-abundant relative to memory bandwidth, traffic governs runtime for a bandwidth-bound kernel even when FLOPs do not change. This principle extends beyond attention. In IO-aware matrix multiplication, tiling algorithms like those in CUTLASS minimize DRAM traffic by maximizing data reuse in fast caches. A naive $N \times N$ matrix multiply performs $\mathcal{O}(N^3)$ FLOPs and, without reuse, $\mathcal{O}(N^3)$ memory traffic; blocking keeps tiles in fast memory and moves traffic toward the $\mathcal{O}(N^2)$ lower bound.

The same byte-movement principle reappears in communication-efficient distributed training, where gradient compression trades extra computation (compression/decompression) for reduced network bandwidth consumption. Low-power edge devices with limited memory bandwidth benefit even more from IO-aware algorithms. Trading 10 percent more arithmetic to halve memory traffic approaches a $2\times$ energy reduction in the limit where data movement accounts for the entire budget, and falls short of that bound whenever arithmetic carries a meaningful share, because halving one additive term of a sum cannot more than halve the sum.

Eliminating off-chip materialization of the $\mathcal{O}(S^2)$ attention matrices reclaims accelerator memory across three architectural trade-offs. First, it enables longer sequence contexts on fixed hardware, with the workable context boundary determined by the complete training recipe. Second, freeing activation memory allows larger micro-batch sizes that improve hardware arithmetic saturation, provided gradient dynamics remain stable. Third, the contracted activation footprint permits deeper transformer stacks within a given memory budget, although end-to-end multi-billion-parameter feasibility continues to depend on parameter sharding, optimizer states, and remaining layer activations.

FlashAttention performs the same arithmetic but moves fewer bytes, which is why counting bytes before FLOPs is the first step on any bandwidth-bound kernel. Bandwidth is not the only memory limit, however. The capacity required to hold activations and optimizer states at the same time can still exceed the accelerator, and when models or batch sizes outgrow device memory, two complementary techniques trade computation for memory.

8.6.5 Gradient accumulation and checkpointing

FlashAttention left GPT-2 XL's batch-32 step about 24 GB over the H100's capacity, and the excess now consists of ordinary activations, the layer inputs, projections, and feed-forward intermediates that backpropagation must read. Two techniques remove it without adding hardware. Gradient accumulation builds the batch from smaller micro-batches that each fit, and activation checkpointing keeps fewer activations and recomputes the rest during the backward pass. Both were named earlier, accumulation with mini-batch processing (section 8.3.2.1) and recomputation with activation memory (section 8.3.3.2). Using them correctly requires knowing when accumulated gradients equal the large-batch gradient, what each technique costs in time, how checkpoints are placed (the $\sqrt{N_L}$ schedule), and which loss-normalization errors most often break accumulation.

8.6.5.1 Gradient accumulation and checkpointing mechanics

Gradient accumulation changes how many examples contribute to one update; checkpointing changes which activations remain resident between the forward and backward passes.

Gradient accumulation To execute large batch training on memory-constrained accelerators, systems split an effective logical batch across smaller sequential micro-batches. Three consecutive micro-batches evaluate independent losses ($\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$) and gradients ($\mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3$, where $\mathbf{g}_j = \nabla_\theta \mathcal{L}_j$), accumulating into a single unified tensor ($\mathbf{g}_1 + \mathbf{g}_2 + \mathbf{g}_3$) before triggering the optimizer step (figure 8.12).

In PyTorch, gradient accumulation is usually implemented by dividing each micro-batch loss by the number of accumulation steps and calling `optimizer.step()` only after processing the entire

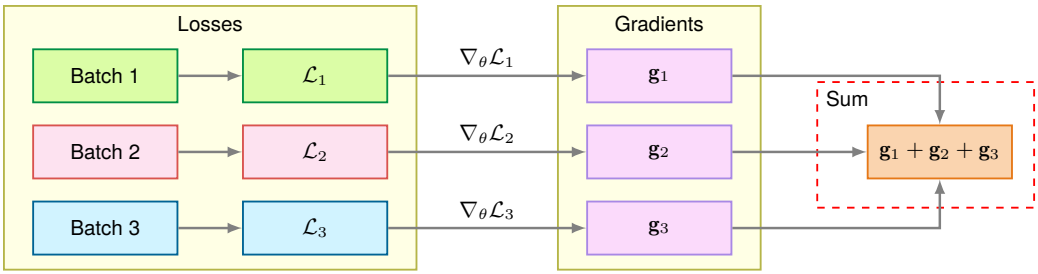


Figure 8.12: Gradient Accumulation: Three micro-batches each compute independent losses and gradients, which sum into a single combined gradient for one parameter update. This simulates training with a batch three times larger without requiring the memory to hold all samples simultaneously.

effective batch. Under that averaging convention, no learning-rate adjustment is needed solely because of accumulation. The implementation follows five steps:

1. Perform the forward pass for a micro-batch.
2. Compute the gradients during the backward pass.
3. Accumulate the gradients into a buffer without updating the model parameters.
4. Repeat steps 1–3 for all micro-batches in the effective batch.
5. Update the model parameters using the accumulated gradients after all micro-batches are processed.

Gradient accumulation can reproduce the gradient of a larger batch under controlled assumptions. For an effective batch size $B = k \times B_{\text{micro}}$ where k is the number of accumulation steps and B_{micro} is the micro-batch size, equation 8.10 confirms that the accumulated gradient equals the true batch gradient:

$$\nabla \mathcal{L}_B = \frac{1}{B} \sum_{i=1}^{B} \nabla \mathcal{L}_i = \frac{1}{k} \sum_{j=1}^{k} \left(\frac{1}{B_{\text{micro}}} \sum_{i \in \mathcal{B}_j} \nabla \mathcal{L}_i \right) \tag{8.10}$$

This equivalence holds when each example's computation is identical and the loss is averaged consistently. The right-hand side shows that averaging k micro-batch gradients (each computed over B_{micro} examples) produces the same result as computing the gradient over all $B = kB_{\text{micro}}$ examples at once. Stateful layers such as BatchNorm, stochastic operations such as dropout and data augmentation, finite-precision accumulation order, and step-indexed schedules can break exact equivalence, so large-batch accumulation still requires validation.

Gradient accumulation exchanges memory capacity for computation time. Table 8.12 separates the memory benefit from the compute and scheduling costs that remain.

Table 8.12: Gradient Accumulation Trade-Offs: Accumulation reduces resident activation memory and preserves total example-level arithmetic. Wall-clock cost depends on micro-batch efficiency, kernel-launch overhead, and whether distributed synchronization is deferred until the accumulated update.

Dimension	Effect
Memory	$\mathcal{O}(B_{\text{micro}})$ instead of $\mathcal{O}(B)$, yielding a $k\times$ reduction in activation memory
Computation	Unchanged total FLOPs, as all B examples are still processed
Time	k micro-batch forward and backward passes precede each optimizer step; smaller micro-batches can reduce utilization and add launch overhead

Accumulation preserves the total example-level arithmetic but can change efficiency because smaller micro-batches use the accelerator differently and launch kernels more often. With distributed `no_sync()` accumulation, one gradient synchronization occurs after the k local micro-batches rather than after each one. The model in equation 8.11 gives the per-update time:

$$T_{\text{effective}} = \sum_{j=1}^{k} T_{\text{micro},j} + T_{\text{sync}} + T_{\text{update}} \tag{8.11}$$

Here $T_{\text{micro},j}$ is the measured forward/backward time for micro-batch j, T_{sync} is the synchronization time paid for the accumulated update, and T_{update} is the optimizer-step time. In distributed data-parallel training, standard framework wrappers automatically fire asynchronous gradient-synchronization hooks during every backward pass. When accumulating gradients across k micro-batches, leaving these hooks active would trigger redundant AllReduce operations on every micro-batch; wrapping the first $k-1$ micro-batches in a `model.no_sync()` context suppresses this communication, synchronizing gradients only on the final k-th step. That synchronization boundary may contain several bucketed AllReduce calls.

The wall-clock penalty relative to processing the effective batch at once is workload-dependent and must be measured rather than inferred from the memory-reduction factor. When gradient accumulation is combined with distributed data parallelism across multiple machines, additional considerations arise for gradient synchronization timing and effective batch size calculation across the cluster; advanced distributed systems texts treat these patterns in depth.

Activation checkpointing Activation checkpointing reduces activation memory by trading additional FLOPs for lower HBM occupancy during backpropagation. Only selected boundary tensors are retained in device memory; intermediate activations are dropped during the forward pass and recomputed on demand as the backward pass traverses in reverse (figure 8.13, where retained tensors are solid green, dropped ones dashed gray, and recomputed ones orange). Algorithm 8.3 states the schedule layer by layer.

Algorithm 8.3 Gradient checkpointing (activation recomputation)

Require: N_L-layer network; checkpoint set $\mathcal{C} \subseteq \{1, \dots, N_L\}$ (e.g., every $\sqrt{N_L}$ layers)
Ensure: parameter gradients, at reduced peak activation memory
1: **for** $\ell = 1$ to N_L **do**
2: forward layer ℓ; store its activation only if $\ell \in \mathcal{C}$ ▷ *drop the rest*
3: **end for**
4: **for** $\ell = N_L$ down to 1 **do**
5: **if** the activation of layer ℓ was dropped **then**
6: recompute the forward segment from the nearest stored checkpoint through layer ℓ
7: **end if**
8: compute the layer's gradient; free the activation
9: **end for**
10: **return** the accumulated gradients

In this segment schedule, recomputed intermediate values remain available until their backward consumers finish; recomputing a segment afresh for each layer would add more work. For layers with comparable activation sizes and forward costs, spacing checkpoints uniformly every $\sqrt{N_L}$ layers reduces peak activation memory from $\mathcal{O}(N_L)$ to $\mathcal{O}(\sqrt{N_L})$ at the cost of one additional forward pass of arithmetic per training step (T. Chen et al. 2016). This increases executed step computation from roughly $3\times$ to $4\times$ the forward-pass FLOPs, trading extra arithmetic operations for a substantially reduced activation footprint. The useful model FLOP count O is unchanged, so the additional work enters the training-time law through effective utilization η_{hw}. Actual fit and wall-clock overhead still depend on the full memory budget and execution profile. Section 8.6.5.2 derives why $\sqrt{N_L}$ spacing minimizes this footprint and works out its cost for GPT-2's 48 layers.

Frameworks like PyTorch provide tools such as `torch.utils.checkpoint` that split a model into segments and manage the recomputation, but the recompute cost remains. Checkpointing is most effective for deep architectures with dozens or hundreds of layers, such as transformers or large convolutional networks, where activation storage exceeds the GPU's capacity before arithmetic throughput is exhausted.

The two techniques address different terms. Gradient accumulation bounds the activation memory tied to batch size, while checkpointing bounds the memory held by each layer's intermediate activations, so together they widen the range of models that fit on available hardware.

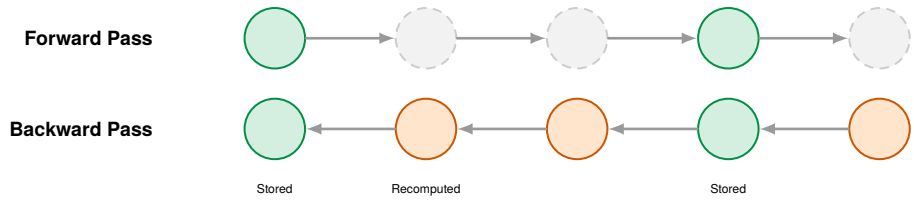


Figure 8.13: Activation Checkpointing: Trading memory usage for recomputation during backpropagation enables training deeper neural networks. By storing only a subset of activations from the forward pass and recomputing others on demand, this technique reduces peak memory requirements at the cost of increased training time.

8.6.5.2 Optimal checkpoint placement strategy

Algorithm 8.3 stated the $\sqrt{N_L}$ schedule without justifying it. For a network with N_L layers of comparable activation size and recomputation cost, each storing $M^{(\ell)}_{\text{activations}}$ bytes of activations, table 8.13 quantifies how the number and placement of checkpoints determines the memory-compute trade-off. Sub-linear checkpointing strategies can reduce memory consumption from $\mathcal{O}(N_L)$ to $\mathcal{O}(\sqrt{N_L})$ with additional forward arithmetic; the resulting wall-clock overhead depends on execution, while the reduced footprint can enable deeper models on existing hardware.

Table 8.13: Checkpointing Memory-Compute Trade-Offs: Different checkpoint strategies trade memory savings against recomputation overhead. The optimal number of checkpoints balances these factors.

Strategy	Memory Cost	Recompute Cost
No checkpointing	$N_L \times M^{(\ell)}_{\text{activations}}$	0 forward ops
One checkpoint ($n_{\text{ckpt}} = 1$)	$M^{(\ell)}_{\text{activations}} + N_L \times M^{(\ell)}_{\text{activations}}$	$(N_L - 1)$ forward ops
n_{ckpt} checkpoints	$n_{\text{ckpt}} \times M^{(\ell)}_{\text{activations}} + (N_L/n_{\text{ckpt}}) \times M^{(\ell)}_{\text{activations}}$	$(N_L - n_{\text{ckpt}})$ forward ops

The memory column counts two things, the stored checkpoints and the one segment that must be rematerialized at a time during backward, which holds N_L/n_{ckpt} layers of activations. The one-checkpoint row shows why both terms matter. With a single checkpoint the only segment spans the whole network, so backward rebuilds all N_L layers of activations at once and the peak is no lower than storing everything. Checkpoints pay off only when they are spread out so that each recomputed segment is short. The recompute column counts layer forwards. Every layer that is not itself a checkpoint is run forward once more, $(N_L - n_{\text{ckpt}})$ layer forwards in all, which is nearly one full forward pass when n_{ckpt} is much smaller than N_L.

Setting the derivative of total memory cost $(n_{\text{ckpt}} \times M^{(\ell)}_{\text{activations}} + (N_L/n_{\text{ckpt}}) \times M^{(\ell)}_{\text{activations}})$ with respect to the checkpoint count n_{ckpt} to zero yields the analytical optimum at $n_{\text{ckpt}} = \sqrt{N_L}$. In the standard $\sqrt{N_L}$ checkpointing scheme, this costs roughly one extra forward pass, about one-third more total step compute, in exchange for minimizing activation footprint under this analytical model. For GPT-2 with forty-eight transformer layers, the operational difference is substantial: uncheckpointed training requires $48 \times M^{(\ell)}_{\text{activations}}$ in full activation storage, whereas uniform checkpointing ($\sqrt{48} \approx 7$ checkpoints) requires $7 \times M^{(\ell)}_{\text{activations}} + (48/7) \times M^{(\ell)}_{\text{activations}} \approx 14 \times M^{(\ell)}_{\text{activations}}$, saving 71 percent activation memory for approximately 33 percent additional total step computation (by recomputing one forward pass over the uncheckpointed layers during backpropagation).

The GPT-2 notebook below uses eight equal segments of six layers. It applies the same boundary-plus-live-segment accounting with unequal storage costs: a segment input contains only the hidden tensor, whereas the reconstructed layer state also includes attention and feed-forward intermediates. The equal-state optimum above is therefore a useful baseline, not an optimality claim for that transformer schedule. Counting the two components separately produces the candidate memory budget without assuming a reduction factor.

Napkin Math 8.8: GPT-2 segment checkpointing

Problem: With mixed precision and FlashAttention-style attention, can a segment schedule fit GPT-2 XL's batch-32 step on one H100, and what does it cost in arithmetic?

Schedule: Partition the 48 transformer layers into 8 contiguous segments of 6 layers. During forward execution, retain each segment's input and discard its internal backward state. During backward execution, reconstruct one segment, retain its internal state while differentiating that segment, then free it before reconstructing the preceding segment. Each layer therefore executes one additional forward pass; against a forward-plus-backward step costing about three forward passes, the extra arithmetic is about one-third. This is work accounting, not a wall-clock penalty.

Math: Each retained segment input has shape batch $\times$ sequence length $\times$ hidden width, $32 \times 1{,}024 \times 1600 \times 2$ bytes. Keep all 8 inputs resident while adding the complete saved-state estimate for the 6 layers of one reconstructed segment. This deliberately counts the active segment's input in both terms, making the estimate conservative rather than omitting a live boundary.

- Retained segment inputs: 0.84 GB
- Live reconstructed segment: 10.7 GB
- Activation allowance: 11.5 GB, against 85.6 GB without checkpointing (7.4× smaller)
- Static state plus activations: 35.5 GB, leaving about 50 GB of the H100's capacity

Systems insight: Checkpointing admits the batch-32 step by replacing retained layer interiors with small boundary tensors and one live reconstructed segment. The remaining capacity must still hold output and loss buffers, temporary backward workspaces, and allocator overhead, which a runtime peak-memory measurement confirms. Gradient accumulation offers the alternative trade, keeping every activation of a smaller micro-batch and paying in update frequency instead of recomputation.

Not all operations are equally expensive to recompute, which motivates selective checkpointing: the practical rule is to spend recomputation on high-memory layers rather than on operations whose activations are already cheap to retain. Table 8.14 compares where activation memory is large enough to justify recomputation.

Table 8.14: Selective Checkpoint Placement: Placement compares the state saved at a block boundary with the work required to reconstruct it. Attention and feed-forward costs depend on sequence length, width, kernel implementation, and whether memory-efficient attention is already in use.

Layer type	Memory cost	Recompute cost	Checkpointing strategy
Attention blocks	Shape-dependent; materialized attention can add quadratic intermediates	High and sequence-length-dependent	Checkpoint block boundaries when saved state justifies recomputation
Feed-forward blocks	Often dominated by expanded hidden activations	High and width-dependent	Checkpoint when expanded activations dominate memory
Normalization	Small standalone outputs	Low	Usually handled within a larger checkpointed block

8.6.5.3 Memory and computational benefits

Both techniques reduce activation-memory requirements without adding hardware, though each targets a different dimension of the activation footprint. Gradient accumulation produces an effective batch larger than the resident micro-batch without storing all of its activations simultaneously; while a larger effective batch reduces sampling variance, it does not by itself guarantee faster convergence or better generalization. Conversely, activation checkpointing reduces the per-layer memory footprint of intermediate activations by discarding and recomputing them as needed, freeing capacity for deeper models, additional layers, or higher-resolution inputs. For lighthouse 8.1, accumulation

provides a direct way to reach a chosen effective batch size while keeping the resident micro-batch within one H100's memory.

Napkin Math 8.9: GPT-2 gradient accumulation strategy

This GPT-2-scale scenario uses the lighthouse's one-accelerator run time and a registry rental price to show gradient accumulation; it does not reconstruct GPT-2's historical training bill.

Memory Constraints

- With FlashAttention-style attention and no checkpointing, a micro-batch of $B_{\text{micro}} = 16$ needs about 67 GB, within one H100's 80 GB.
- A resident effective batch of $B = 512$ would therefore require 512 ÷ 16 = 32 accelerators.

Accumulation Configuration

- Use 8 accelerators and accumulate 4 micro-batches of 16 sequences each.
- Each accelerator processes 16 × 4 = 64 sequences per update, giving a global effective batch of 8 accelerators × 64 = 512.

A distributed `no_sync()` context can defer AllReduce until the final micro-batch.

Trade-off Analysis

- Idealized wall-clock time: 4× longer (105 hours against 26 hours) because fewer accelerators perform the same arithmetic work
- Memory overhead: accumulation normally reuses parameter-gradient buffers, although bookkeeping and temporary storage remain
- Communication: both configurations synchronize once per 512-sequence update, but participant count and topology differ
- Equal-work cost: $3,370 either way at $4/hour per accelerator, before utilization and communication effects

Why this works: Under the equivalence conditions stated above, averaged per-example gradients are additive. Each accelerator first accumulates a local gradient over its 64 examples:

$$\frac{1}{4}\sum_{j=1}^{4}\left[\frac{1}{16}\sum_{i=1}^{16}\nabla\mathcal{L}(x_{ji})\right]$$

AllReduce then averages those local gradients across 8 accelerators, producing the global effective batch:

$$\frac{1}{8}\sum_{r=1}^{8}\nabla\mathcal{L}^{(r)}_{\text{local}}, \qquad B = 8 \times 64 = 512$$

Operational comparison

Resident configuration:

- 32 accelerators × 16 sequences = 512 resident sequences per update
- Gradient synchronization spans 32 accelerators
- Hourly cost: $128/hour

Accumulated configuration:

- 8 accelerators × (16 sequences × 4 micro-batches) = 512 sequences per update
- Gradient synchronization occurs once per update across 8 accelerators, all within one server
- Hourly cost: $32/hour
- Hourly reduction: $96/hour, or 75 percent

Quality gate

- Match the examples and their order, loss normalization, optimizer state and step schedule, and stochastic-layer behavior.
- Compare held-out loss or perplexity and the optimization trajectory across repeated runs rather than assuming that the effective batch alone guarantees equivalent quality.
- Expect small round-off differences from the changed summation order; require convergence within a declared tolerance rather than bitwise identity. No perplexity result is assumed here.

Systems insight: Accumulation lowers accelerator count and hourly burn, but ideal scaling preserves accelerator-hours; actual cost depends on utilization and communication.

Listing 8.8 places one AllReduce after 4 local micro-batches, making the synchronization boundary explicit.

Listing 8.8: Gradient Accumulation Training Loop: Four local micro-batches of 16 sequences accumulate before one synchronization boundary and optimizer update across eight accelerators; the abstract AllReduce may represent multiple gradient buckets.

```
optimizer.zero_grad()
for step in range(4):  # Accumulation steps
    micro_batch = next(dataloader)  # 16 sequences
    loss = model(micro_batch) / 4  # Average effective-batch loss
    loss.backward()  # Accumulate gradients
# Now gradients represent 64 local sequences
all_reduce(gradients, op=SUM)  # Sum across 8 accelerators
gradients /= 8  # Convert the sum to the global mean
optimizer.step()  # Update with effective batch=512
```

8.6.5.4 Practical considerations

Gradient accumulation is most valuable when optimal batch sizes exceed GPU memory capacity. Transformer language models often use effective batches of hundreds of thousands to millions of tokens; if memory only permits a smaller number of sequences per device, accumulation bridges the gap without requiring additional hardware. Activation checkpointing complements this by enabling deeper architectures and is commonly used in transformer-scale training.

Both techniques introduce explicit trade-offs. Activation checkpointing recomputes discarded activations during backward propagation; with ordinary segment checkpointing, each discarded activation is typically reconstructed once, while recursive schedules may behave differently. Gradient accumulation reduces parameter-update frequency because each update follows k micro-batches. When each micro-batch loss is divided by k, the accumulated gradient follows the effective-batch mean convention. If losses are summed instead, the gradients must be rescaled before the optimizer step or the learning-rate convention changed accordingly. Framework and codebase conventions differ, making normalization a common source of subtle bugs.

8.6.6 GPT-2 optimization walkthrough

Training GPT-2 XL at batch 32 on a single H100 exposes the physical boundaries that govern large-model optimization. The tensor allocation first exceeds accelerator memory, and once it fits, a naive input pipeline starves the execution engines. Each step below fixes whatever blocks the next, memory feasibility first and execution evidence second, before any accelerator is added.

Napkin Math 8.10: GPT-2 optimization on one H100

Step 1: Establish the baseline memory footprint.

The trace starts with GPT-2 XL at batch 32 and sequence length 1,024, FP32 tensors throughout, standard attention, and a single-threaded synchronous data loader. Weights, gradients, and Adam moments take 24 GB, and FP32 activations take 574 GB, so the step needs 598 GB, about 7× the H100's 80 GB (85.9 GB in decimal units). The run fails the machine constraint before throughput matters.

Step 2: Apply mixed precision.

The first intervention targets memory feasibility rather than speed. Mixed precision (notebook 8.6) halves the activations and leaves the static state unchanged, reducing the step to 311 GB. The allocation still exceeds the accelerator.

Step 3: Replace standard attention with FlashAttention.

Never storing the quadratic score matrices removes most of the remaining activations and brings the step to 109.6 GB, still above capacity. With this kernel, the largest batch that stays resident is about 23 sequences, so the configuration overflows somewhere between batch 16 and batch 32.

Step 4: Fit the batch.

Two techniques close the gap. Accumulating two micro-batches of 16 keeps the same arithmetic per update and needs 66.8 GB. Segment checkpointing (notebook 8.8) keeps all 32 sequences resident in 35.5 GB but adds about 33 percent more arithmetic per step. The walkthrough takes accumulation, because the batch it needs fits without recomputation; checkpointing is the choice when a larger resident micro-batch matters more than arithmetic. Runtime measurement must still verify that backward workspaces, output and loss buffers, and allocator overhead fit in the remaining capacity.

Step 5: Read the profile.

Fitting the model resolves capacity but does not guarantee efficient execution. With single-core tokenization, host preparation takes longer than the device's step (example 8.1), so a serial pipeline keeps the accelerator busy only about 48 percent of the time (equation 8.7). The diagnostic focus shifts from machine memory capacity (M) to the data pipeline (D) in D·A·M terms: reducing accelerator memory further would not touch the dominant exposed delay.

Step 6: Apply prefetching and parallel preparation.

Eight worker processes, page-locked (pinned) host memory for asynchronous DMA transfers, and a prefetch queue overlap host preparation with accelerator computation, and the parallel preparation interval falls well inside the device's step. In the ideal overlapped model the accelerator is then busy for the whole step. A busy accelerator is not yet an efficient one; MFU measures how much of that busy time does useful model arithmetic, and re-profiling must establish whether memory bandwidth, arithmetic throughput, or another stage now limits execution.

Result: At the assumed 50 percent utilization, one step takes 0.60 s, the accelerator processes about 55,000 tokens/s, and the run takes about 35 days on one accelerator (about 47 days with segment checkpointing at the same executed rate). Table 8.15 collects the result.

Table 8.15: GPT-2 XL on One H100: Memory figures are calculated from the model shape; the busy-time result follows from the tokenization and step-time scenario; run time, energy, cost, and carbon follow from the assumed utilization, the accelerator's thermal design power, and registry values for facility overhead, electricity price, and grid carbon intensity. The FP32 baseline does not fit, so it has no run to measure.

Metric	FP32 baseline	Optimized	Technique applied
Static training state	24 GB	24 GB	Unchanged at 16 bytes per parameter
Activations per step	574 GB	42.8 GB	FP16, FlashAttention, two micro-batches of 16
Total memory	**598 GB**	**66.8 GB**	9× smaller
Fits one H100	No	Yes	Capacity 80 GB (85.9 GB)
Accelerator busy time	N/A	Whole step (ideal)	Prefetching, eight workers
Throughput	N/A	55,000 tokens/s	At the assumed utilization
Run time	N/A	35 days	One accelerator
Facility energy	N/A	661 kWh	Accelerator power × PUE 1.12
Electricity cost	N/A	$79	At $0.12/kWh
Carbon	N/A	0.28 t	Average grid intensity

Systems insight: The order of interventions follows a dependency, not a preference. Memory feasibility comes first because a configuration that does not fit produces no profile at all. Once it runs, the profile decides what comes next; in this scenario exposed input waits made the data pipeline the target ahead of any kernel tuning, because a faster kernel cannot shorten a step spent waiting for its input. After each intervention, re-profiling determines whether input delivery, memory bandwidth, arithmetic throughput, or another execution cost now limits progress.

This walkthrough demonstrates why training optimization is a sequential diagnostic discipline rather than an arbitrary collection of techniques. Each intervention addresses a constraint established by the memory calculation or the profile. Mixed precision halves activation storage, FlashAttention removes the quadratic attention term, and accumulation brings the batch within capacity without extra arithmetic, a 9× modeled memory reduction in all. Prefetching then addresses exposed input waits once execution begins. The result is a run of about 35 days on one accelerator.

Checkpoint 8.3: From a feasible configuration to a validated run

The walkthrough combines memory calculations with a modeled execution profile. Use the distinction to decide what an actual run must verify.

- ☐ **Evidence**: Which walkthrough values are calculated from tensor sizes, which follow from assumed utilization or tokenization rates, and which require measurement on the target system?
- ☐ **Capacity and throughput**: Why does fitting the tensor budget establish neither peak runtime memory nor efficient execution, and what must be measured next on hardware?
- ☐ **Quality gate**: Before accepting the composed optimizations, how does an engineer verify that the revised precision and effective batch preserve progress toward the same validation target?

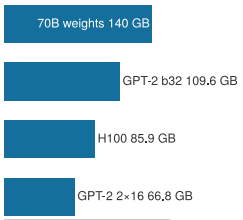
Single-accelerator optimizations address complementary constraints: mixed precision enables higher-throughput arithmetic, FlashAttention avoids materializing full attention matrices in HBM, accumulation and checkpointing bound activation memory, and asynchronous prefetching hides host input latency. Even fully optimized, however, a run of GPT-2 XL's size occupies one accelerator for about 35 days, and larger runs are bounded by the arithmetic throughput of an individual chip.

Scaling beyond single-device compute or memory capacity requires distributing computation across multiple accelerators. This architectural transition introduces inter-device communication

overhead, collective synchronization barriers, and fault-tolerance constraints that become the primary determinants of scaling efficiency.

8.7 Scaling Training Systems

The GPT-2 walkthrough fits the batch-32 step on one H100 and finishes the run in about 35 days. One accelerator cannot finish sooner, and some models cannot fit at all. Spread across the 8 accelerators of one server with perfect scaling, the same run would take about 4.4 days. A 70-billion-parameter model needs about 140 GB for its FP16 weights alone and far more for full training state, which no single accelerator holds, motivating sharding, offload, or multiple devices.



Two micro-batches of 16 fit one H100; batch 32 without checkpointing and a 70B model do not.

When a chosen workload still exceeds one device's memory or schedule, spreading it across devices provides aggregate memory and compute. Other responses include reducing the model, batch, sequence length, or training scope; offloading state; and parameter-efficient adaptation, which trains only a small set of added parameters on top of a frozen model. Scaling beyond one device begins with multi-GPU configurations inside one machine and may extend across nodes. The major parallelism strategies make different communication trade-offs; a full treatment of distributed-training implementation lies beyond this chapter.

Not all workloads benefit equally from adding GPUs. Adding accelerators helps only until communication and synchronization consume the time saved. Figure 8.14 charts this scaling decay on a log-log speedup plot up to 256 GPUs, contrasting ideal linear scaling against compute-bound (ResNet), balanced (a large language model (LLM) over NVLink), and communication-bound regimes. The widening divergence between the ideal linear dashed reference and the plateauing modeled curves illustrates the communication overhead that accumulates as synchronization compounds. In this fixed-fraction model, even unlimited parallel compute leaves the non-scaling fraction of the original runtime, so the limiting speedup is its reciprocal, the Amdahl bound of equation 11.1.

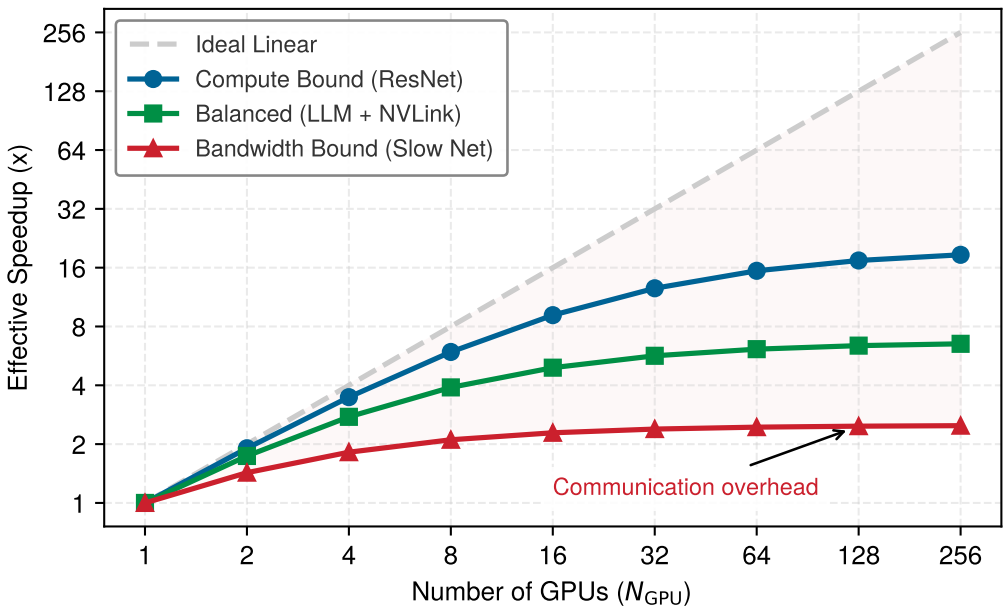


Figure 8.14: Communication Overhead: An Amdahl-style model compares ideal linear scaling against workloads with varying non-scaling communication and synchronization fractions. Real communication overhead scales dynamically with device count, network topology, message size, and overlap efficiency.

The red curve's widening gap from the dashed ideal is that communication overhead made visible, and shrinking that gap is what the distributed strategies in section 8.7.1 exist to do. Those strategies are older than deep learning: training platforms inherit dense matrix multiplication kernels and tightly synchronized parallel execution from high-performance computing (HPC), and

from warehouse-scale computing they inherit cluster orchestration and automated fault recovery for multi-day jobs, the lineage that matters once a run spans many machines.

The same inheritance appears inside one training system as pressure at three scales. At data scale, large corpora require high-throughput input pipelines to avoid accelerator starvation. At model scale, billion-parameter architectures force a choice between replicating weights across devices and partitioning the model. At infrastructure scale, coordinating multiple accelerators introduces synchronization overhead that can dominate training time. The stage interfaces formalized in section 3.4.1 govern what each scale may assume of the others.

8.7.1 Single-node multi-GPU training

Multi-GPU training within a single node introduces the core parallelism strategies. AlexNet[16] (Krizhevsky et al. 2012) split its convolutional layers across two GTX 580 GPUs because the 3 GB device memory limit prevented single-chip execution, making cross-GPU connectivity an immediate architectural constraint. Confining multi-accelerator topologies to single-node interconnects (such as PCIe or NVLink) isolates these strategies before cross-rack network fabrics enter.

The two foundational strategies partition different things. Data parallelism replicates the model and partitions the data, while model parallelism partitions model state or computation. This distinction determines memory requirements, communication patterns, and scaling behavior.

[16] **AlexNet (2012)**: The paper reports 60M parameters and a 3 GB single-GPU ceiling, but does not attribute the limit to one memory component. Its model-parallel design placed roughly half the kernels on each GPU and allowed communication only in selected layers, explicitly trading memory capacity against communication.

8.7.1.1 Data parallelism

Data parallelism replicates the entire model on each GPU, with each processing different batches. After computing gradients locally, GPUs synchronize via gradient averaging. Input batches split across parallel GPU workers for independent forward and backward passes, then converge into an AllReduce collective, which sums the gradients from every worker and returns the result to each of them, so every replica applies the identical parameter update (figure 8.15).

When a model and its optimizer state fit comfortably within single-device memory, data parallelism serves as the primary scaling baseline because local forward and backward kernels run at native memory bandwidth without inter-device synchronization. Gradient aggregation is the principal recurring communication cost; some implementations also synchronize buffers or other state. Historically, frameworks supported multi-GPU execution within a single node using multithreaded wrappers like PyTorch's `DataParallel` (DP). However, this single-process design suffers from two severe architectural bottlenecks: the Python runtime remains throttled by the Global Interpreter Lock (GIL), and a primary coordinator (GPU 0) must sequentially scatter mini-batches and gather outputs over PCIe, saturating host-device interconnect bandwidth and creating severe compute and memory imbalances. Modern systems commonly use PyTorch's `DistributedDataParallel` (DDP); TensorFlow's `MirroredStrategy` also provides replicated training but has a different execution model. With an independent Python process per accelerator, DDP avoids cross-replica contention for one Python GIL; each process maintains its own model replica and local optimizer, relying on decentralized collective AllReduce operations that bucket gradients into contiguous buffers and overlap synchronization directly with backward-pass computation.

Synchronizing gradients is the step that data parallelism adds, and for GPT-2 XL inside one server its cost can be derived directly. Each replica holds FP16 gradients of 3 GB, and every replica must end the step with their sum. Ring AllReduce (Patarasuk and Yuan 2009) arranges the N_{GPU} accelerators in a logical ring and splits the gradient into N_{GPU} chunks. In a reduce-scatter phase, each accelerator passes a chunk to its neighbor $N_{\text{GPU}} - 1$ times, adding the chunk it receives to its own, until each holds one fully summed chunk; in an all-gather phase, the summed chunks circulate $N_{\text{GPU}} - 1$ more times until every accelerator holds all of them. Each accelerator therefore sends $2(N_{\text{GPU}} - 1)/N_{\text{GPU}}$ of the gradient bytes D_{vol}, nearly twice the gradient however large the ring, and over a link of bandwidth BW the bandwidth term of the synchronization time is

$$T_{\text{comm}} \approx \frac{2(N_{\text{GPU}} - 1)}{N_{\text{GPU}}} \cdot \frac{D_{\text{vol}}}{\text{BW}} \tag{8.12}$$

For 8 H100s the factor is 1.75, so each accelerator sends 5.25 GB per step, which takes about 12 ms at 450 GB/s per direction over NVLink. Against a 596 ms step when each replica processes the full batch of 32 sequences, synchronization is about 2 percent of the step even without overlap. Splitting

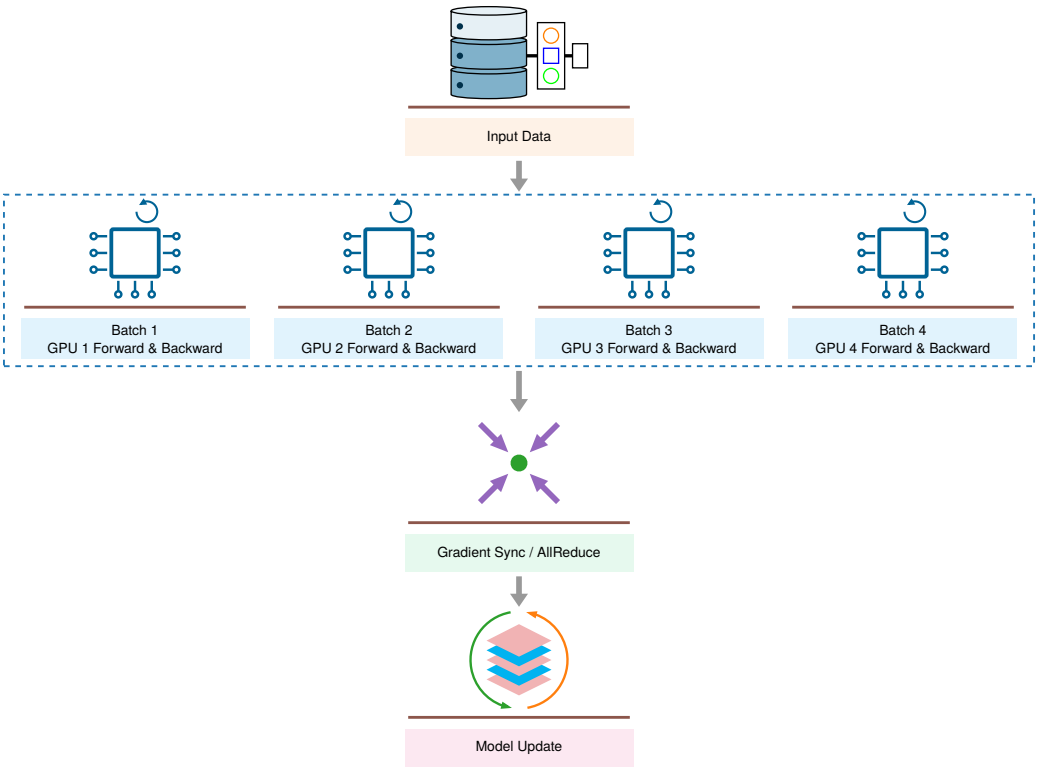


Figure 8.15: Data Parallelism: Each GPU holds a complete model copy, processes a different data shard, and synchronizes gradients. Throughput can approach linear scaling only while computation dominates synchronization, input, and imbalance costs.

a fixed batch of 32 eight ways shrinks each replica's compute to 75 ms while the gradient bytes stay the same, raising the unhidden share to about 14 percent. Data parallelism inside one server is cheap for GPT-2 XL as long as each replica keeps enough work to amortize a synchronization whose size depends only on the parameter count; frameworks hide most of what remains by launching bucketed AllReduce calls while the backward pass is still computing earlier layers' gradients.

Unsharded data parallelism has a hard constraint. Every accelerator holds a complete model replica and its full training state, 16 bytes per parameter in the mixed-precision Adam recipe, or 24 GB for GPT-2 XL on each of the 8 accelerators. The Zero Redundancy Optimizer (ZeRO) (Rajbhandari et al. 2020) removes that redundancy by sharding training state across the data-parallel replicas in three stages. ZeRO-1 shards the 12 bytes of FP32 master weights and Adam moments, leaving 8.2 GB per accelerator for GPT-2 XL; ZeRO-2 also shards the gradients, leaving 5.6 GB; and ZeRO-3, which fully sharded data parallelism (FSDP) (Zhao et al. 2023) implements, also shards the FP16 parameters, leaving 3 GB. The same arithmetic brings the Llama 3 8B full fine-tune that overflowed one accelerator in section 8.3.3.3 down to about 16 GB of static state per accelerator. The memory comes back as communication. ZeRO-1 and ZeRO-2 keep the data-parallel volume, because a reduce-scatter of gradients followed by an all-gather of updated parameters moves the same bytes as one AllReduce, while ZeRO-3 must also gather each layer's parameters before its forward and backward computation, about 1.5 times the data-parallel volume (Rajbhandari et al. 2020). A model that still exceeds local memory after sharding requires pipeline or tensor partitioning.

8.7.1.2 Model parallelism

Model parallelism partitions the model itself across GPUs, which becomes necessary when the model exceeds single-GPU memory. AlexNet adopted an early form of this strategy: selected convolutional layers split their kernels and channels across two GPUs, with communication at selected connections. Figure 8.16 illustrates a different, layerwise partition of the same broader

model-parallel idea: Device 1 through Device 3 sequentially process layers 1–16, 17–32, and 33–48, streaming boundary activations forward during the forward pass and propagating loss gradients in reverse during backpropagation.

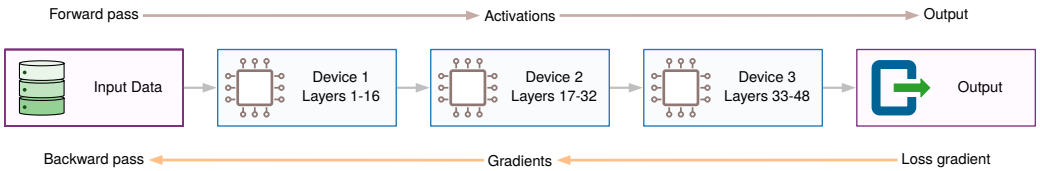


Figure 8.16: Model Parallelism: The model is partitioned across devices, with intermediate activations passing between them. This enables training models larger than single-GPU memory at the cost of sequential dependencies.

Systems Perspective 8.4: Data vs. model parallelism

When a selected workload is partitioned across N_{GPU} devices, the split may divide examples, model state, layers, or tensor operations. For a model with parameters P and batch size B, data and model parallelism provide two starting points.

Data parallelism (split the batch): For a fixed global batch, each of N_{GPU} workers processes about $1/N_{\text{GPU}}$ of the examples, but every worker holds a model replica and ordinarily its associated training state. Dense gradient communication is proportional to P per update. Scaling departs from the ideal as local batches shrink or synchronization and input costs grow.

Model parallelism (split model computation): Each accelerator stores and computes a portion of the model. An even partition may approach P/N_{GPU} parameters per worker, but embeddings, buffers, imbalance, and replicated state alter that estimate. Communication depends on the partition: layerwise pipeline parallelism exchanges activations at stage boundaries, while tensor parallelism performs collectives within partitioned layers.

Systems insight: Data parallelism is the simplest candidate when the complete training state fits per worker and more throughput is needed. Model-state, pipeline, or tensor partitioning becomes necessary when it does not, often in combination with data parallelism.

When a model exceeds the memory capacity of a single accelerator, its depth can be partitioned across sequential devices. Figure 8.17 stages this pipeline allocation across a four-accelerator execution chain, grouping 24 transformer blocks into contiguous six-block partitions (Blocks 1–6 through 19–24) that exchange boundary activations over the interconnect.

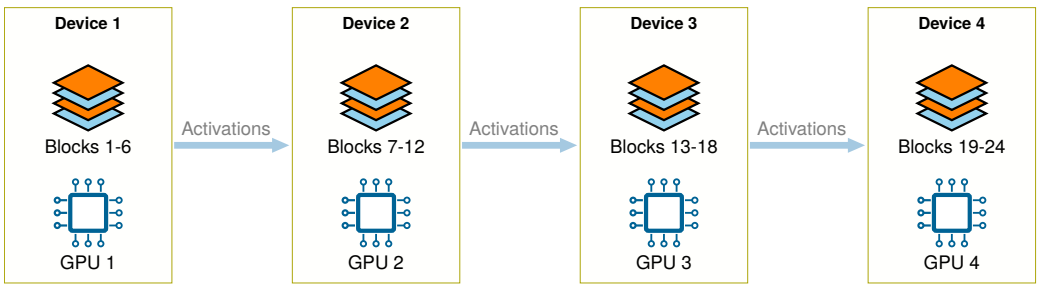


Figure 8.17: Layer-Wise Model Partitioning: Distributing consecutive transformer layers across sequential devices bounds network communication to intermediate activations (forward) and boundary gradients (backward). However, sequential layer dependencies leave downstream devices idle during forward passes unless micro-batches are pipelined.

The main bottleneck in naive layer-wise partitioning is device serialization. Because layer evaluation is strictly sequential, Device 1 computes layers 1–6 while Devices 2–4 sit idle waiting for forward activations. During the backward pass, the dependency reverses: Device 4 computes first while upstream devices wait for returning gradients. This idle duration constitutes the *pipeline*

bubble, which caps theoretical hardware utilization at $1/N_{\text{GPU}}$ under naive sequential execution. It differs from the accelerator bubbles of the staged training pipeline, which arise when one device's data-loading, transfer, and compute stages run at mismatched rates; a pipeline bubble arises even with perfectly fed devices, because each device must wait for another device's layers. Mitigating this idle time requires either pipelining micro-batches across stages or partitioning tensor dimensions within layers so that all devices compute simultaneously.

Both remedies fit inside one node. Pipeline parallelism splits each batch into micro-batches and streams them through the stages, so Device 2 works on the first micro-batch while Device 1 starts the second, and the bubble shrinks to the time spent filling and draining the pipeline at the start and end of each batch (Huang et al. 2019; Narayanan et al. 2019). Tensor parallelism instead splits the matrix operations of each layer across devices, so every device works on every layer at once (Shoeybi et al. 2019). The price is communication inside every layer, because the partial results must be combined before the next layer can begin. That frequency is why tensor parallelism is normally kept within one node, on its fastest links.

Within a node, GPUs may communicate through high-bandwidth links such as NVLink (up to 900 GB/s aggregate bidirectional bandwidth for the represented H100 configuration). Effective collective bandwidth depends on topology and contention and is lower than a raw aggregate link specification. Data parallelism communicates gradients via collective AllReduce operations, while layerwise model parallelism communicates activations point-to-point at partition boundaries. Data parallelism suits models that fit on one accelerator, whereas larger models require sharded training state or a split model.

8.7.2 Scaling beyond a single node

One machine is this book's scope, but its boundary is worth knowing. When a model's parameter footprint, optimizer state, or training duration exceeds the capacity of an 8-accelerator node, distributed training extends across machines. The strategies of this section still apply, but three things change once training leaves the node. First, bandwidth drops sharply because machines are joined by a network such as Ethernet or InfiniBand rather than NVLink, reducing link throughput by one to two orders of magnitude so that gradient synchronization that overlapped with the backward pass inside a node can come to dominate each step. Second, placement must match each strategy to a tier of the network: the frequent intra-layer communication of tensor parallelism stays inside the node, whereas the once-per-step gradient exchange of data parallelism can cross slower inter-node links. Third, hardware failure becomes a routine operational event across thousands of accelerators during a multi-day run, making automated checkpointing and restart an essential part of the training design. Each of these is a subject of its own in large-scale distributed training; within one machine the same principles appear in their simplest form, which is why the single-node case comes first.

↪ Empirical protocols for quantifying multi-accelerator scaling efficiency and communication loss are developed in section 12.5.2.2.

8.7.3 When to scale: The physical ceiling

Even a perfectly tuned single accelerator hits hard physical walls: a 70-billion-parameter model overflows 80 GB of HBM on weights alone, and a 10^{24}-FLOP pretraining run at 10^{15} sustained FLOP/s would take 32 years on one chip. Until a run meets one of those walls, however, distributing state across devices adds communication traffic, synchronization barriers, and new failure modes that can slow a job that would have fit locally. Single-device optimizations therefore come first to separate solvable local bottlenecks from genuine capacity or schedule limits: lower-precision execution (section 8.6.3) when numerical stability permits, gradient accumulation (section 8.6.5) to build target batch sizes without activation blowup, activation checkpointing (section 8.6.5.1) to trade redundant forward compute for memory headroom, and overlapped data prefetching (section 8.6.2) when host ingestion limits the critical path.

Table 8.16 structures this progression as a resource lookup keyed to measured hardware bottlenecks rather than arbitrary parameter thresholds.

Table 8.16: Scaling Decision Guidelines: Choose the least distributed configuration that satisfies measured memory, throughput, and schedule requirements. Model architecture, training state, input pipeline, hardware, and topology all affect the boundary.

Observed constraint	Candidate approach	Rationale
Training state fits one device and time is acceptable	Single GPU	Minimizes communication and operational complexity
State or throughput exceeds one device but fits one node	Single multi-GPU node	Uses fast local interconnects before crossing the network
Required state or schedule exceeds one node	Multi-node cluster	Adds aggregate memory or compute at a communication cost
Input service rate is below accelerator demand	Parallel or distributed input pipeline	Adds input throughput where profiling shows it is needed

A measured single-device baseline guides scaling decisions, while a decisive capacity or compute-time lower bound can establish the need for additional resources before every local optimization is attempted. Once local tuning is exhausted or ruled out by lower bounds, resource expansion must match the specific physical limit that has been reached. Memory exhaustion occurs when model weights, gradients, optimizer states, and activations exceed the VRAM of a single GPU: a 70-billion-parameter model requires about 140 GB in FP16 for weights alone, exceeding the reference H100's capacity and leaving little or no headroom on higher-capacity variants once training state is included. Training wall-clock time binds when the estimated duration on one device exceeds the project's deadline, as in a 10^{24}-FLOP workload that would take roughly 32 years at 10^{15} FLOP/s sustained. Input throughput binds when the host pipeline cannot sustain the aggregate accelerator consumption rate, which calls for more input-service capacity rather than more training accelerators; dataset size alone does not determine this limit because reuse, compression, locality, transformation cost, and storage bandwidth govern delivery rate.

Data parallelism also meets a statistical ceiling. Each added replica enlarges the global batch, and beyond the critical batch size met in the discussion of batch size and parameter updates (section 8.4.4.1), larger batches stop reducing the number of updates needed to reach a target loss (McCandlish et al. 2018). Past that point, more replicas raise throughput in samples per second without shortening the time to target quality, so a scaling decision needs both the hardware ceiling and the batch at which the recipe's convergence flattens.

These three limits are technical, but scaling carries a cost beyond complexity and capacity. Adding devices amplifies the energy consumption and environmental impact of training, so efficiency optimization becomes an environmental concern as much as a performance one.

Napkin Math 8.11: The carbon footprint of training

Context: Training large models requires substantial energy and compute. The physical ceiling appears once training time is converted to energy, cluster power multiplied by training time ($E_{\text{total}} = P_{\text{cluster}} \cdot T_{\text{train}}$), quantifying environmental scale:

1. Workload: Training a 7 billion-parameter model for 1 trillion tokens.
2. Compute: $O \approx 4.2 \times 10^{22}$.
3. Efficiency: 494 TFLOP/s sustained on the H100 (700 W thermal design power).
4. **Execution time**: $T_{\text{train}} \approx 0.96$ days on 1,024 GPUs.
5. **Energy footprint**: $E_{\text{total}} = P_{\text{cluster}} \cdot T_{\text{train}} = (N_{\text{GPU}} \cdot P_{\text{device}} + N_{\text{nodes}} \cdot P_{\text{host}}) \cdot T_{\text{train}}$ evaluates to (1,024 GPUs $\times$ 700 W + 128 hosts $\times$ 200 W) $\times$ 23 hours $\approx$ 17,104.8 kWh.

Impact: Under the assumed constant GPU and host-CPU power, this modeled subtotal equals 19.0 months of average household electricity use. GPU thermal design power is a scenario input rather than measured draw; memory, storage, network equipment, and facility overhead are not separately included, so the subtotal is not total facility energy.

Systems lesson: Doubling hardware utilization η_{hw} at constant cluster power P_{cluster} halves execution time T_{train} and total energy E_{total}. Under average grid carbon intensity, doubling efficiency saves ~8,552.4 kWh of electricity and avoids 3.7 t of emissions. Environmental audits must measure $E_{\text{total}} = \int P_{\text{cluster}}(t)\,dt$ rather than relying on GPU utilization metrics alone.

Scaling changes the binding constraint rather than removing it. Data parallelism adds throughput, sharded and model parallelism add memory capacity, pipeline parallelism recovers utilization lost to idle stages, and tensor parallelism makes layers larger than one device feasible, but each adds communication and coordination cost. The implementation details of multi-node distributed training, including collective communication primitives, fault tolerance mechanisms, and elastic scheduling, build directly on the single-machine principles covered throughout this chapter and are treated in depth in advanced distributed systems texts.

Checkpoint 8.4: Scaling decisions

Scaling trades compute bottlenecks for communication bottlenecks.

- ☐ **Hard limits**: Identify which resource limit binds a given training run: memory capacity, compute throughput, or input-service rate.
- ☐ **Single-node first**: Explain how a single-node baseline and relevant local optimizations inform scaling, and when a physical lower bound can justify scaling before all tuning is exhausted.
- ☐ **Data vs. model parallelism**: Justify why data parallelism fails when model states exceed single-accelerator memory, and identify what model parallelism must partition.
- ☐ **Communication overhead**: Explain why scaling efficiency degrades with accelerator count, and identify the communication metrics that dictate this ceiling.
- ☐ **Scaling ceiling**: Estimate the theoretical speedup ceiling via Amdahl's law when unparallelized synchronization accounts for 10 percent of step time, and determine where adding accelerators yields diminishing returns.

8.8 Fallacies and Pitfalls

The approach developed throughout this chapter quantifies costs through the iron law, diagnoses bottlenecks through profiling, applies targeted optimizations, and scales only when necessary. The following fallacies and pitfalls are the errors that most often waste training compute.

Fallacy: *Larger models always yield better performance.*

Teams sometimes treat scale as a monotonic lever: if a 7B-parameter model works well, a 20B-parameter model must work better. A 20B model requires approximately 320 GB of nonactivation training state in the stated mixed-precision Adam recipe (40 GB FP16 parameters + 40 GB FP16 gradients + 80 GB FP32 master weights + 160 GB Adam states). Whether it outperforms a 7B model depends on data quantity and quality, training compute, optimization, and evaluation, not on parameter count alone.

Pitfall: *Assuming distributed training automatically accelerates development.*

More accelerators do not guarantee faster training because communication, smaller local batches, input limits, and imbalance can consume the gain. The chapter's own derivation shows the mechanism: splitting GPT-2 XL's batch of 32 across eight H100s shrinks each replica's compute while the gradient bytes stay fixed, so unoverlapped synchronization grows to about 14 percent of the step. In a hypothetical 8 GPUs scenario, the same effect lets synchronization occupy 30–50 percent of step time, so speedup reaches only 4–6× rather than the ideal 8×. The decision requires measured end-to-end time and scaling efficiency.

Fallacy: *Hyperparameters transfer directly from small-scale experiments to large-scale training.*

A learning rate that works at batch size 512 may not transfer to batch size 4,096. The linear scaling rule (Goyal et al. 2017) would increase the rate from 0.1 to 0.8 for this eightfold change, often with warmup, but it is an empirical recipe rather than a requirement or convergence guarantee. Large-scale runs must validate the learning rate, schedule, optimizer state, and effective batch together.

Pitfall: *Treating mixed precision training as a simple toggle without validation.*

The H100's peak ratios bound the gain at 2× over the TF32 Tensor Core path and 15× over FP32 scalar units, but elementwise kernels, optimizer updates, and input waits do not speed up with the matrix units, so the measured gain is smaller. FP16 commonly requires loss scaling (Micikevicius et al. 2018), while BF16 usually does not (Wang and Kanwar 2019; Kalamkar et al. 2019) (section 8.6.3 compares the two formats). Either recipe can change numerical behavior, so throughput and convergence must be validated on the target workload.

Fallacy: *Memory and computation can be optimized independently.*

Memory *and* compute are coupled: in this illustrative profile, accelerator utilization drops from 90 percent at batch 256 to 60–70 percent at batch 16. Gradient accumulation (effective batch 512, physical batch 64) shows a 5-percentage-point utilization gap for 8× activation-memory reduction. Tuning these parameters independently can therefore extend training time materially, because the utilization lost at the smaller physical batch applies to every step of the run (section 8.6.5 develops this trade-off).

Pitfall: *Budgeting training memory as if it only contains model weights.*

Sizing training memory from weights alone omits gradients, optimizer state, master weights when used, activations, temporary workspaces, and allocator overhead. In the chapter's mixed-precision Adam recipe, the static state alone is 16 bytes per parameter before activations. Other optimizers and sharding strategies change this budget. Activation memory then depends on batch size, sequence length, architecture, kernels, and checkpointing; no universal multiple of the weight footprint applies.

Fallacy: *The accelerator is always the training bottleneck.*

Data loading often creates idle time, yet teams optimize computation first. On the reference H100, ResNet-50's demand outruns the host cores available to it and leaves the accelerator busy only about 52 percent of the time, and GPT-2 XL with single-core tokenization is busy only about 48 percent; a faster accelerator fixes neither, while parallel preparation and overlap (section 8.6.2) address both. Profile before assuming the GPU is the bottleneck.

Pitfall: *Optimizing kernels before profiling the input pipeline.*

Kernel-level tuning is attractive because accelerator utilization is easy to inspect, but a starved accelerator can look like an inefficient one. Before changing precision, rewriting kernels, or adding accelerators, measure dataloader throughput, host preprocessing time, and storage wait. If the accelerator is waiting for batches, feeding it reliably beats faster kernels.

8.9 Summary

Training cost is governed by three quantities that this chapter first kept separate and then joined. The training-time law, $T_{\text{train}} = O/(R_{\text{peak}} \times \eta_{\text{hw}})$, sets the time a run takes, and device power multiplied by that time sets its energy. The four-term memory budget decides what fits, and in a standard mixed-precision Adam recipe the static state alone reaches 8× the FP16 weights before a single activation is stored. The min-of-three pipeline model decides whether the accelerator is fed at all. GPT-2 XL tied the three together. Its $6PD$ operation count set a run of about 35 days on one H100, its weights, optimizer state, and activations overflowed that accelerator at batch 32 before any arithmetic began, and its single-threaded data loader could starve the device once the step fit.

Every intervention in the chapter relieves one term and burdens another, which is the conservation of complexity at work. Mixed precision raises R_{peak} and shrinks activation memory and traffic, but it leaves the static state unchanged and requires loss scaling or a wider-exponent format. Checkpointing trades extra forward arithmetic for activation memory, accumulation trades update frequency for

batch size, prefetching hides input time at the cost of host memory, and FlashAttention removes quadratic HBM traffic by recomputing tiles. Scaling to more devices adds aggregate memory and compute but turns gradient synchronization into a new term on the critical path. The profile, not the menu of techniques, decides which trade is worth making.

The same pressures shaped the hardware. Dense matrix patterns drove Tensor Cores, frameworks exposed them through mixed-precision APIs, and reduced-precision training pushed later accelerators toward BF16 and FP8. The bitter lesson explains why this matters. If capability grows with the computation a general method can absorb, every point of utilization lost to stalls is capability forgone at a fixed budget.

The counting rules developed across the chapter fit on one page (table 8.17). Together they estimate, before any profiler runs, how long a run takes, what it costs in energy, whether its step fits in memory, and which stage of its pipeline binds.

Table 8.17: Key Equations: The chapter's counting rules in the notation of the sections that derive them, so that a training run can be estimated before it is profiled. The activation rule sums per-example saved state over layers; the transformer estimates in the chapter specialize it to batch, sequence length, and hidden width.

Quantity	Formula	Where derived
Training time	$T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}}$	equation 8.1
Training work	$O \approx 6PD$ FLOPs (2 forward, 4 backward per parameter per token)	section 8.1.1
Training energy	$E_{\text{total}} = P_{\text{device}} \times T_{\text{train}}$ at constant power	section 8.2
Training memory	$M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$; 16 bytes per parameter of static state in mixed-precision Adam	equation 8.4
Activation memory	$M_{\text{activations}} = B \times \sum_{\ell=1}^{N_L} \left(s_\ell + M_{\text{activations}}^{(\ell)} \right)$	equation 8.3
Pipeline throughput	$R_{\text{training}} = \min \left(R_{\text{preprocessing}}, \frac{\text{BW}_{\text{host}}}{D_{\text{vol}}^{(\text{batch})}}, \frac{R_{\text{sustained}}}{O_{\text{batch}}} \right)$	equation 8.6
Model FLOPs utilization	$\text{MFU} = O_{\text{model}} / (R_{\text{peak}} \cdot T_{\text{step}})$	section 8.5
Checkpointed activations	$n_{\text{ckpt}} \times M_{\text{activations}}^{(\ell)} + (N_L / n_{\text{ckpt}}) \times M_{\text{activations}}^{(\ell)}$, minimized at $n_{\text{ckpt}} = \sqrt{N_L}$	table 8.13
Effective batch	$B = k \times B_{\text{micro}}$ per device	equation 8.10
Ring AllReduce time	$T_{\text{comm}} \approx \frac{2(N_{\text{GPU}}-1)}{N_{\text{GPU}}} \cdot \frac{D_{\text{vol}}}{\text{BW}}$	equation 8.12

Compute and memory deserve equal weight in that estimate, because the step that fits is not always the step that is cheapest to compute (table 8.18).

Table 8.18: Memory and Compute per Training Step: Step FLOPs follow from six FLOPs per parameter per token or three forward passes per image; static state is 16 bytes per trained parameter in the mixed-precision Adam recipe; activations come from the chapter's per-layer estimates or, for ResNet-50, its stated FP32 scenario.

Workload (step)	FLOPs per step	Static state	Activations	Fits one H100
GPT-2 XL, 32 sequences of 1,024 tokens	295 TFLOP	24 GB	287 GB stored; 85.6 GB with FlashAttention	With FlashAttention and two micro-batches of 16
ResNet-50, 32 images	787 GFLOP	0.4 GB	About 12 GB (FP32 scenario)	Yes; the binding term is the input feed
Llama 3 8B fine-tuning, 4 sequences of 2,048 tokens	395 TFLOP	128 GB (full)	36.5 GB	Only with adapters (53 GB)

Key Takeaways: Why training costs millions

- **Training cost is an iron-law budget**: $T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}}$ makes every optimization accountable: reduce work, raise effective throughput, or improve utilization. A change that misses the dominant term only moves cost around the training loop.
- **Memory determines whether a step fits**: Standard mixed-precision Adam holds 16 bytes per parameter, 8× the FP16 inference weights, before activations. Optimizer state and batch-dependent activations, not weights alone, determine whether training fits.
- **Profiles choose the remedy**: Profile, diagnose, fix, and re-profile. Compute-bound jobs need better arithmetic or algorithms; memory-bound jobs need less state or traffic; data- and communication-bound jobs need pipeline or parallelism changes.
- **Precision and IO-aware kernels shift bottlenecks**: FP16 with FP32 accumulation can improve throughput and shrink activation memory (the static state is unchanged), while FlashAttention avoids materializing the full $S \times S$ matrix in HBM; realized gains depend on workload and hardware.
- **Checkpointing trades recomputation for memory**: The segment schedule counts retained boundaries and one live reconstructed segment, reducing GPT-2 XL's batch-32 activations from 85.6 GB to 11.5 GB for about one-third more arithmetic. Use it when memory, not compute, binds.
- **Composed optimizations can postpone scale-out**: Mixed precision, FlashAttention, and gradient accumulation turn GPT-2 XL's 598 GB batch-32 step into a modeled 66.8 GB that fits one H100, for a run of about 35 days. Use relevant local levers before accepting distributed communication and operational overhead.

Training is where the iron law becomes a daily instrument. A profile read against the law names the dominant term, and the engineering task is to apply the least costly intervention that changes it before reaching for more hardware. The analysis took the model and training corpus as given, while changing how the system executed their work, including the extra arithmetic needed for recomputation. A further lever is which examples enter that workload in the first place.

What's Next: From training to data selection

Training produces learned parameters by spending compute on every example. Once the training loop is efficient, the next question is whether every example earns that cost. Repeated or low-information samples consume the same forward pass, backward pass, optimizer update, and communication as examples that add coverage or correct a weakness.

Chapter 9 therefore turns upstream from execution to workload composition, and in doing so opens the book's optimization sequence, which asks whether work is necessary before asking how to simplify or accelerate it. It starts from a supply imbalance. Accelerator fleets can grow usable compute faster than the stock of high-quality data, so extracting more learning from each example begins to matter more than collecting more examples. Its measure is the information-compute ratio, the learning an example returns per unit of training compute, and redundancy, representativeness, uncertainty, and class coverage all enter through it. A useful subset must reduce work without erasing rare cases or shifting the effective training distribution. When that balance holds, a carefully selected subset can approach full-dataset accuracy while lowering the cost of every epoch and making additional experiments affordable. The optimization target is no longer only how efficiently the system processes an example, but which examples the system should process at all.

III

OPTIMIZATION AND ACCELERATION

Part III

Optimization Principles

A working model is rarely an efficient one. Part II established how to construct ML systems that respect physical constraints; Part III addresses how to meet real-world demands on time, memory, and energy. Efficiency is also what keeps the bitter lesson affordable, because models grow with the computation available while budgets and deployment envelopes do not. Each optimization navigates a frontier, improving one metric (accuracy, latency, energy) while managing the cost to others, and each is a form of co-design. Data selection shapes the workload the algorithm sees, while compression and acceleration trade mathematical structure for physical feasibility. The principles here are the bounds and cost models of efficiency, which determine *why* some models are fast and affordable while others are slow and prohibitively expensive.

Principle 5: The Pareto frontier

Bound: After objective directions are normalized, a feasible point lies on the Pareto frontier exactly when no other point is at least as good on every objective and strictly better on one. No feasible design lies beyond that frontier, so moving along it improves one objective only by worsening another.

- **Quantization**: Trades numerical precision for reduced memory footprint.
- **Pruning**: Trades model capacity for smaller representations and can improve speed when the resulting sparsity or removed structures are supported by the target hardware.
- **Distillation**: Trades training compute for inference efficiency.

Implication: Systems engineers navigate this frontier to find the operating point appropriate to a specific deployment environment. There is no universal optimum.

Navigating the Pareto frontier requires knowing *which* resource to optimize. Before selecting a technique, engineers must diagnose whether the workload is limited by computation, memory movement, or dispatch latency. Arithmetic intensity supplies the first test. The introduction derived its threshold, the ridge point, by setting the iron law's memory term equal to its compute term, and Part II placed training kernels on either side of that threshold in a roofline plot. Here the roofline becomes an explicit rule for choosing an optimization.

Principle 6: Arithmetic intensity law

Bound: Attainable throughput (R_{attain}) is no greater than the minimum of peak compute (R_{peak}) and DRAM bandwidth (BW) times operational intensity at the cache–DRAM boundary (I) (Williams et al. 2009):

$$R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$$

Operational intensity and DRAM bandwidth must both be measured at the cache–DRAM boundary; this ideal bound excludes launch, synchronization, and other fixed overheads.

Implication: Adding peak compute does not raise the ideal roofline bound for a bandwidth-bound kernel. Engineers must identify whether the bottleneck is compute, bandwidth, or another system term before selecting an optimization.

Many important ML kernels fall on the memory-bound side of the ridge point, especially low-reuse operations such as embedding lookup, normalization, softmax, depthwise convolution, and small-batch inference paths. Dense matrix multiplications and convolutions can instead be compute-bound when batching and hardware utilization are high. This split is a consequence of the memory wall: processor speed has historically outpaced memory bandwidth, and the cumulative gap has widened over three decades (Wulf and McKee 1995; Gholami et al. 2024). Neural networks, with their large weight tensors and uneven temporal locality, are especially vulnerable, so table 8.19 maps each bottleneck regime to the optimizations that address its dominant term and the changes that leave it untouched.

Table 8.19: The Bottleneck Diagnostic: Before optimizing, identify which iron law term dominates. Optimizing the wrong term leaves the dominant bottleneck unchanged and usually yields little or no end-to-end improvement.

If the workload is…	Dominant Term	Optimization That Works	Optimization That is Wasted
Bandwidth-Bound	D_{vol}/BW	Quantization, pruning, batching	Faster accelerator (more FLOP/s)
Compute-Bound	$O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	Better kernels, Tensor Cores, faster accelerator	More memory bandwidth
Overhead-Bound	L_{lat}	Kernel fusion, async dispatch, bounded microbatching under the latency SLO	More FLOP/s or bandwidth alone

Wall-clock time is only one cost of crossing the memory wall; the energy required to move data off-chip exposes a second constraint. Even when a memory-bound kernel meets its latency target, fetching operands repeatedly from off-chip DRAM dominates the power budget on both battery-powered devices and data center racks.

Principle 7: The energy-movement principle

Decomposition: Total workload energy depends jointly on per-event energy and event counts. In the cited 45 nm reference, moving a 32-bit value from DRAM can cost roughly 100–1,000× more energy per event than a 32-bit floating-point operation, depending on operation type (Horowitz 2014). Here, D_{vol} is bytes moved and O is operations executed; E_{move} denotes the energy per byte moved from DRAM, so one 32-bit access costs $4\,E_{\text{move}}$, and E_{compute} denotes the energy of one 32-bit floating-point operation in the cited 45 nm technology.

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}, \qquad 4\,E_{\text{move}} \gg E_{\text{compute}} \quad \text{for the cited reference}$$

Implication: When memory movement dominates workload energy, optimization strategies should prioritize kernel fusion (avoiding intermediate off-chip traffic) and quantization (reducing data size); when computation dominates, reducing operation counts and improving arithmetic kernels remain central.

Even with perfect data locality and optimal bottleneck targeting, a final constraint limits how much speedup any optimization can deliver. The introduction stated Amdahl's law for an inference pipeline whose preprocessing and postprocessing capped the gain from a faster model, and later chapters applied it case by case, to data-ingestion workers and to communication overhead in training. Stated as a principle, it bounds every optimization in this part.

Principle 8: Amdahl's law

Bound: The maximum speedup of a system is limited by the fraction of the workload that cannot be accelerated (Amdahl 1967). Here, f_{parallel} is the fraction of baseline execution time accelerated and G is that fraction's speedup, either an accelerator's gain G_{accel} or a worker count N_{workers}; the fixed-work model assumes the remaining fraction is unchanged and the acceleration introduces no additional overhead.

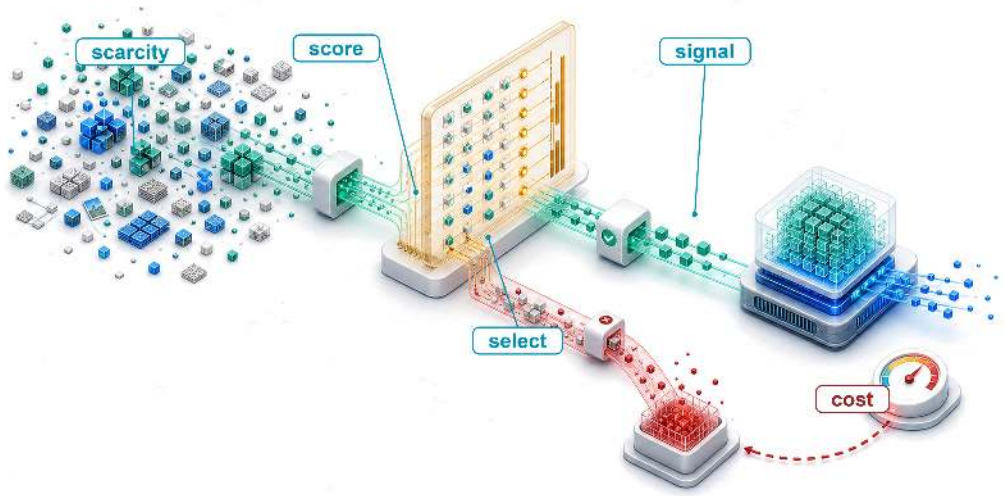
$$\text{Speedup} = \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{G}}$$

Implication: If 95 percent of a model runs 100× faster on an accelerator, the total system speedup is only about 16.8×, and no accelerator, however fast, can push it past 20×. This explains *why* data loading and preprocessing often become the ultimate bottlenecks in highly optimized systems.

Part III applies these principles across the three multiplicative factors of the iron law in the order the D·A·M taxonomy suggests (Appendix A gives the full diagnostic framework). On the data axis, chapter 9 asks first whether fewer samples can achieve the target quality (Samples/Task). Where algorithm meets machine, chapter 10 asks whether each sample's pass can use fewer operations or fewer bits per value (FLOPs/Sample and D_{vol}). On the machine axis, chapter 11 asks how to execute each remaining operation in less time (Seconds/FLOP). Each chapter ends with a favorable local number (a smaller dataset, a smaller model, or a kernel placed under its roofline), and chapter 12 closes the loop by testing whether those predicted savings survive end-to-end execution without violating quality constraints.

9

Data Selection

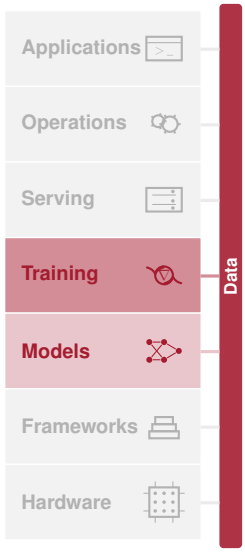


9.1 The Data Wall
9.2 Measuring Data Value
9.3 Static Pruning
9.4 Dynamic Selection
9.5 Synthetic Data Generation
9.6 Learning from Unlabeled Data
9.7 Selection Engineering
9.8 Choosing a Selection Strategy
9.9 Cross-Layer Interactions
9.10 Fallacies and Pitfalls
9.11 Summary

Purpose

How can a carefully selected subset retain most of the value of a much larger dataset?

Training pays to acquire, label, score, move, and process examples even though their learning value differs. Large datasets contain redundant, noisy, or target-misaligned samples, while scarce regions may lack the cases a model needs, such as the rare conditions under which a deployed system is most likely to fail. Training on every example treats a near-duplicate and a rare failure case as equally worth paying for, and at scale much of the compute budget can go to relearning what the model already knows. That heterogeneity turns the choice of examples into a systems optimization rather than a data-cleaning chore. The objective is therefore not the smallest dataset but the lowest end-to-end cost of reaching target quality without losing rare cases, underrepresented groups, or deployment-relevant coverage. Every strategy adds its own overhead: examples must be scored, generated, shuffled, stored, or validated, and those costs can erase the savings they promise. That judgment must be made end to end, because gains in labeling or training can be erased by selection latency, storage access, or distributed coordination. A method is worth applying only when the learning it preserves or creates exceeds the work required to apply it. This distinction matters across repeated experiments, where useful reductions compound, across adaptation, where the value of an example changes with the model and target distribution. In D·A·M terms, it is data-algorithm co-design: shape the training signal so each unit of machine work yields more useful learning.



Learning Objectives

- Explain data selection as data-algorithm co-design that controls sample value, training work, and labeling cost
- Calculate the information-compute ratio and apply the compute-optimal tokens-per-parameter check to diagnose a data-starved run
- Apply the selection inequality to test whether selection overhead beats full-dataset training cost
- Compare deduplication, coreset selection, and quality pruning for reducing redundant pretraining data
- Design curriculum, active learning, synthetic-data, or label-free strategies for changing data value during training
- Evaluate whether a selection pipeline keeps the accelerator fed under storage locality, data echoing, and multi-accelerator consistency constraints
- Select data-selection investments using return on investment, break-even, and amortization analysis

9.1 The Data Wall

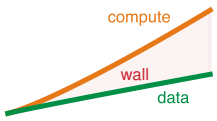
Every example a training run processes costs the same forward pass, backward pass, and optimizer update, whether it teaches the model something new or repeats what the model already knows. A duplicated image, a mislabeled record, and a rare boundary case cost the same arithmetic while contributing very different learning signals. This waste becomes acute at the **data wall**, the point at which the compute available for training grows faster than the supply of novel, high-quality examples to spend it on.[1] Chapter 8 made each of those steps as cheap as one machine allows and accounted for a whole run as roughly $6PD$ floating-point operations for P parameters and D tokens. Once each step is cheap, the next saving lies upstream, in choosing which examples deserve a step at all. For the full GPT-2 XL run that chapter 8 costed on one H100, the question becomes how many of those steps were worth taking.

Increasing the amount of training data has long been a standard way to improve models. Scaling laws[2] (Kaplan et al. 2020; Hoffmann et al. 2022) quantify how model performance can improve with dataset size, and teams have responded by collecting and generating more examples. Accelerator fleets, however, can expand usable compute faster than the supply of novel, high-quality human-generated text and images. Much of the easily accessible public web has already been incorporated into large training corpora, and expert labeling capacity grows slowly; chapter 4 measured the growth of training corpora, and figure 4.6 sets the largest of them against the estimated stock of public text, between 10^{14} and 10^{15} tokens, which they reach late this decade at the fitted rate. That is the wall at frontier scale, on fleets far larger than one machine, and it shifts engineering effort from collecting everything available to extracting more learning signal from each stored and processed sample.

GPT-2 XL shows the same wall on one machine. The arithmetic needs only the corpus size, the token budget of the original run, and the fitted compute-optimal balance between parameters and tokens.

WebText, GPT-2's training corpus (Radford et al. 2019), holds 40 GB of text, about 10 billion tokens at roughly 4 bytes per token. The compute-optimal balance fitted by the Chinchilla study, about 20 training tokens per parameter (developed in section 9.2.3), asks for 30 billion unique tokens for GPT-2's 1.5 billion parameters, 3× what WebText holds. The original run spent about 1.5×10^{21} FLOPs, which at six FLOPs per parameter per token processed about 167 billion tokens, so the model saw its corpus roughly 17 times. One machine with 8 H100 accelerators running at 50 percent model FLOPs utilization (MFU), the fraction of peak arithmetic a training run actually delivers (definition 8.3), completes that budget in about 4.4 days and 590 kWh of accelerator energy. Less than a week of one machine's compute therefore passes over the whole corpus many times. An engineering team in that position can repeat epochs over the same text, where gradient updates on familiar tokens yield diminishing loss reductions, or it can lower quality thresholds to admit uncurated web text, where

[1] **Data wall**: Unlike compute (which scales with capital expenditure) or algorithms (which improve through research), the stock of high-quality human-generated text grows slowly; Epoch AI's updated projections estimated that, under the paper's modeled consumption assumptions, models could use datasets roughly equal in size to the stock of public human-generated text between 2026 and 2032 (Villalobos et al. 2024). The point for systems design is not a fixed calendar deadline; it is that data can become a supply constraint rather than merely an economic one. The wall does not change how many operations (O) a run performs. It lowers the learning each of those operations yields, because



Compute outruns high-quality data supply; the widening gap is the data wall.

[2] **Scaling laws**: Jared Kaplan and colleagues at Johns Hopkins and OpenAI empirically demonstrated in 2020 that language model loss follows power-law relationships with model size, dataset size, and compute budget over the regimes they studied (Kaplan et al. 2020). Their fitted data exponent was approximately $\alpha = 0.095$ in $\mathcal{L} \propto D^{-\beta}$. The value is not universal, but the diminishing-return shape makes it possible to reason about when selection becomes more cost-effective than collection.

↩ GPT-2, the bandwidth-bound lighthouse for decoder-only language models, is established in section 6.7.4.2.

noisy and misaligned tokens degrade downstream evaluation. Instead of accepting either penalty, the team can invest in data selection (refining quality filters, constructing difficulty curricula, and synthesizing missing edge cases) to raise the learning return per FLOP.

When *compute* is abundant and *quality data* is scarce, the optimization priority inverts. Selection, not faster kernels, then extracts more learning per dollar, and for teams whose accelerator budget exceeds their curated corpus, the bottleneck shifts from accelerator access to the quality, legality, and diversity of the training data.

The bitter lesson applies here in data terms. A filter written from human intuition about which examples matter is the hand-built expertise the lesson warns against, whereas selection that uses the model's own signals, such as loss, gradient norms, or embedding similarity, scales with compute. Either way, improved rankings must be established against downstream quality and coverage.

Data engineering makes data correct; data selection asks which correct examples are worth training on. When selection removes processing at matched quality, it shrinks the Data factor (Samples/Task) and therefore the total operations (O) term in the iron law (principle 3). Selection therefore needs sample value stated relative to compute cost, and the information-compute ratio of section 9.2.1 measures that value as learning signal gained per unit of training compute.

↩ The bitter lesson, that general methods using more computation outperform encoded human expertise, is established in section 1.4.

Definition 9.1: Data selection

Data selection is the process of retaining, prioritizing, or generating training examples to minimize the end-to-end cost of reaching target quality while preserving required coverage.

1. **Significance**: Each excluded example removes a forward pass, a backward pass, and an optimizer update, so selection reduces Samples/Task and therefore total operations (O) directly.
2. **Distinction**: A dataset can pass every data engineering check and still contain many examples that teach the model nothing new; selection judges informativeness and diversity, not correctness.
3. **Common pitfall**: Minimizing dataset size instead of end-to-end cost. Scoring, labeling, and lost coverage can erase the savings of a smaller set.

Holding epoch count, batch size, and per-sample work fixed, a 50 percent reduction in dataset size directly halves the required forward passes, backward passes, and gradient updates. For the GPT-2 run above, it would halve both the 4.4 days on one eight-H100 machine and the 590 kWh of accelerator energy, provided the training schedule does not compensate by adding optimization steps. When data selection is combined with model compression and hardware acceleration, those reductions compound across all three axes of the iron law.

Systems Perspective 9.1: Data selection and the iron law

In the multiplicative D·A·M form of the iron law of ML systems (equation 1.2, where $T = (\text{Samples/Task}) \times (\text{FLOPs/Sample}) \times (\text{Seconds/FLOP})$), data selection reduces the first factor (Samples/Task), model-level optimizations reduce the second (FLOPs/Sample), and machine-level optimizations reduce the third (Seconds/FLOP, by increasing $R_{\text{peak}} \cdot \eta_{\text{hw}}$ and cutting memory and latency stalls). Selection removes work before those downstream optimizations execute it.

Because the three D·A·M factors multiply, their speedups compound. A 2× reduction in dataset size with 2× fewer operations per sample and 2× higher effective machine throughput yields an 8× reduction in run time because each axis divides the work or time the next axis sees. Unpacking the machine factor into the three physical terms of equation 1.3 ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$) states the condition for realizing that full product: while halving the number of training steps cuts all three terms in half, the algorithmic and hardware speedups achieve their

full multiplier only when the dominant physical bottleneck, whether compute, data movement (D_{vol}/BW), or fixed overhead (L_{lat}), falls in step.

These compute savings propagate throughout the hardware hierarchy. By decreasing total data volume (D_{vol}), deduplication and coreset selection alleviate storage bandwidth demands, network fabric ingress, and host-to-accelerator PCIe transfers that otherwise leave the accelerator's arithmetic units waiting on input. Upstream, active learning curbs annotation budgets by directing human labeling to samples chosen for uncertainty, diversity, or other task-relevant query criteria.

Yet selection is not free. Every selection strategy incurs its own runtime overhead to score, filter, shuffle, or coordinate samples. An online selection algorithm that executes additional forward passes to rank sample difficulty can consume more accelerator cycles than it saves if its selection overhead exceeds the subsequent reduction in training time. A pipeline claiming a training-time saving must make filtering plus subset training faster than full-data training at the same target quality, the selection inequality of section 9.2.6. Labeling savings and reusable representations require the broader cost accounting of section 9.8.

A learning-theory view of this problem counts the samples needed to reach a test error and treats each sample as free. The systems view counts the FLOPs, bytes, labels, seconds, and joules needed to reach the same quality, including the cost of choosing the samples. In the D·A·M taxonomy, data selection works on the data axis, deciding whether an example warrants processing at all before the algorithm fixes the operations each example costs and the machine fixes how fast those operations run. Removing an example therefore lightens both adjacent boundaries, the storage and input traffic between data and machine and the learning return between data and algorithm.

↩ The D·A·M taxonomy and its three diagnostic axes are established in section 1.5.

The engineering toolkit for intelligent data selection follows a deliberate optimization ordering, first asking whether a sample is worth processing and only then how to process the remaining workload efficiently. Static pruning acts first by fixing the retained subset before target-model training begins, even when its scoring step uses a smaller proxy model. Because a sample's usefulness changes as the model learns, dynamic selection then adapts which examples the training loop sees and which receive human labels during training through curriculum learning and active learning. Where real data still falls short after pruning and reordering, synthetic generation creates missing high-value samples on demand through augmentation, simulation, or examples labeled by a larger trained model (a *teacher*). Learning from unlabeled data complements these three levers by redefining what constitutes a supervision signal: rather than selecting among labeled examples, semi-supervised and self-supervised methods derive learning targets from the structure of unannotated data, and which levers to use, in what order, depends on the workload (figure 9.1).

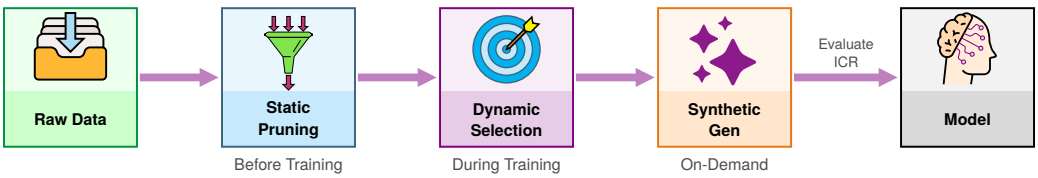


Figure 9.1: An Example Data Selection Pipeline: One possible combination of interventions for increasing data value. The three intervention points are Static Pruning before target training, Dynamic Selection during training, and Synthetic Generation on demand. The sequence is illustrative rather than required: synthetic data can enter independently and can itself be pruned or dynamically selected. The final evaluation determines whether the quality gain justifies the compute cost through the Information-Compute Ratio (ICR).

Two lighthouses carry the chapter, each in its own role. GPT-2 XL, bandwidth-bound when it decodes one token at a time but compute-bound when it trains on long batches (chapter 8), carries the wall itself, the compute-optimal diagnosis of whether a run is data-starved, and token-level deduplication. ResNet-50 on ImageNet, the book's compute-bound lighthouse, carries coresets, labels, augmentation, the input pipeline that must feed it, and the decision of which lever to fund. Its training crosses into an input-bound regime whenever the host cannot decode images as fast as the accelerator consumes them (section 8.4.1), and that crossing returns when selection runs on the machine. The measures of value in the next section use both, and both run on the book's

reference machine, the H100, so every saving can be stated in accelerator seconds and joules. The recommendation lighthouse (DLRM), MobileNetV2, and keyword spotting return only in the closing comparison of section 9.8, where each turns out to be bound by a different resource.

Every lever in the chapter removes work at one stage of the machine and adds work of its own, so each must pass a test before it is accepted (table 9.1). Each test in the right-hand column depends on measuring what an example is worth, which is the subject of the next section.

Table 9.1: Chapter Map: Each section of the chapter, the work its lever removes and adds on the machine, and the test the lever must pass before it is accepted.

Section	Lever and when it acts	Work it removes	Work it adds	Test it must pass
Measuring Data Value	Yardsticks for every lever	None	Pilot runs at several data sizes	Defines the tests below
Static Pruning	Deduplication, quality filtering, coreset; before training	Tokens and examples processed in every epoch	Hashing and proxy scoring passes	Selection inequality at matched quality and coverage
Dynamic Selection	Curriculum learning, active learning; during training	Warmup FLOPs; labels	Difficulty scoring, pool scoring rounds, retraining	Time to target quality; label cost against a random baseline
Synthetic Data Generation	Augmentation, generative synthesis, distillation; on demand	Collection and labeling of scarce cases	Host transforms, generator inference, stored teacher targets	Validation on real deployment data
Learning from Unlabeled Data	Semi-supervised and self-supervised learning	Task-specific labels	Extra training compute; pretraining	Total label and compute cost; reuse across tasks
Selection Engineering	Running any lever on the machine	Accelerator idle time	Scoring latency, random reads, worker coordination	The accelerator stays fed and the inequality holds end to end
Choosing a Selection Strategy	Costing the levers and picking among them	Investment in levers that do not pay	Engineering and infrastructure investment	Return on investment over the runs that share it

9.2 Measuring Data Value

A ResNet-50 trained on an aggressively filtered ImageNet subset can hold its aggregate accuracy while failing on rare classes that the filter discarded as low-density noise, and a GPT-2 run on a filtered corpus can cost less per epoch while needing more epochs to reach the same loss. A measurement framework is therefore needed to show what a reduction actually saves. Every data selection technique claims to improve efficiency, but only measurement separates genuine training savings from work that has merely moved to another pipeline stage, or from degradation on safety-critical tail distributions. The measures build from the information-compute ratio through a diagnostic of whether a run is data-limited, metrics that tie sample reduction to accuracy, and checks that a reduced set still covers the cases deployment needs, to the selection inequality that every training-time claim must satisfy.

9.2.1 The information-compute ratio

Retaining every example in a training corpus preserves coverage at the cost of spending FLOPs on redundant signal, whereas over-aggressive pruning saves compute at the expense of rare-slice recall. Data selection therefore moves along a Pareto frontier (principle 5) between accuracy and cost, the set of operating points that no alternative beats on one objective without losing on the other. Weighing that trade-off requires measuring how much learning signal each sample yields per unit of hardware work. Data engineering already measured task-relevant signal against one cost, the data gravity of moving bytes, in the data selection gain of equation 4.3. Selection needs the same ratio with training compute in the denominator, because a sample that is cheap to move can still cost a full forward and backward pass. That ratio is the **information-compute ratio** (ICR).

↪ Locating a model's operating point on the accuracy-efficiency frontier, and moving along it with compression, is developed in section 10.5.1.

Let $\Delta\mathcal{I}$ be the learning signal a sample or batch adds and ΔO the training operations spent processing it. In practice $\Delta\mathcal{I}$ is measured as the gain in a fixed evaluation metric, such as percentage points of validation accuracy, so ICR carries units of metric gain per FLOP. The ratio relates the two:

$$\text{ICR} = \frac{\Delta\mathcal{I}}{\Delta O}$$

A higher ICR indicates that each FLOP of training yields more measured learning signal over the stated interval. Validation gain and training FLOPs can be observed for a specified experiment with a fixed evaluation protocol; the causal contribution of an individual example generally cannot. Uncertainty and gradient-based scores instead rank candidate examples before their downstream benefit is known. Those rankings require validation against measured quality, compute cost, and deployment-critical slice coverage. The core metrics in section 9.2.4 supply complementary evidence for that evaluation rather than interchangeable measurements of per-example information, and they also track how ICR decays as the dataset grows.

9.2.2 Diminishing returns and the ICR frontier

As a training run adds more examples from the same source, a growing share of those examples repeat patterns the model has already seen, and the information-compute ratio falls. The **ICR frontier** is the dataset size beyond which the marginal ICR, the learning returned by the next FLOP spent on more data, drops below what that FLOP could return elsewhere: a larger model, a different data source, or an earlier stop.

To illustrate diminishing returns, let $\mathcal{I}(D)$ be the information content of a dataset of size D and assume $\mathcal{I}(D) \propto \log D$. This is a simplified analytical model rather than a universal law. If compute cost scales linearly with the per-sample operation count O_{sample}, then total operations are $O = O_{\text{sample}} \cdot D$, and the resulting marginal ICR follows equation 9.1:

$$\text{ICR}(D) = \frac{d\mathcal{I}/dD}{dO/dD} \approx \frac{1/D}{O_{\text{sample}}} = \frac{1}{O_{\text{sample}} \cdot D} \tag{9.1}$$

The derivative ratio describes the return on an infinitesimal increase in dataset size. For two controlled finite allocations, the corresponding incremental ratio divides the additional evaluation gain by the additional training FLOPs, using the same evaluation metric and protocol. The coreset comparison that follows instead averages gain over one epoch of each dataset; that average does not establish the return on the next increment.

Under this illustrative model, marginal ICR falls as $1/(O_{\text{sample}} \cdot D)$, so doubling the dataset halves the learning each additional FLOP yields. Inverted, the compute spent per unit of learning grows linearly with D. ICR never reaches zero in this model and the curve has no knee of its own, so the frontier is located by comparing marginal ICR with the threshold the engineer sets, not read off the curve. That saturation is distinct from the data wall (a limit on how much new, high-quality data exists), but the two compound. When new data is scarce *and* marginal data is weak, selection becomes the main lever. Beyond the frontier, additional data still adds compute but returns less learning than the threshold demands. In this regime additional data inflates the O term of the iron law without a commensurate gain in ICR. Measured in dollars rather than FLOPs, the same excess lowers the return on compute (RoC) of equation 1.6, the accuracy gained per added dollar of infrastructure.

Data selection turns the total operations (O) term in the iron law from a fixed constant into a variable. A 2× improvement in measured ICR halves the FLOPs required to reach the same measured gain. It produces the same ideal time reduction as a 2× throughput increase only when the workload is compute-bound and the saved work lies on the critical path. ICR focuses specifically on compute; the cost model of section 9.8.1 extends the same reasoning to acquisition, labeling, and storage costs.

↩ Return on compute, the economic lens that measures accuracy gains against infrastructure spending, is established in section 1.8.1.

The ratio becomes concrete on a *coreset*, a deliberately selected subset intended to preserve the full dataset's learning signal. Computing ICR for an ImageNet coreset used to train the ResNet-50 lighthouse shows how large the efficiency gap could become once dataset size, per-sample model cost, and selection strategy meet concrete FLOP budgets. The scenario uses assumed accuracy gains to illustrate the arithmetic; it is not a reported result for the scoring method (EL2N) that section 9.3.4 develops.

Napkin Math 9.1: Computing ICR: Coresets

Problem: Training the ResNet-50 lighthouse model on ImageNet for one epoch costs 3.2×10^{16} FLOPs. Under the scenario's assumed accuracy gains, how would retaining 50 percent of the data change the information-compute ratio? The calculation holds the per-sample work fixed and does not include selection overhead.

Setup:

- Dataset: ImageNet (1.28M)
- Model: ResNet-50 lighthouse (~8.2 GFLOP per forward pass, roughly 24.6 GFLOP for a forward plus backward training step, depending on implementation)
- One epoch: 1.28M $\times$ 24.6 GFLOP = 3.2×10^{16} FLOPs
- Assumed accuracy improvement per epoch (early training): 5 percentage points

Random selection (baseline):

- Process all 1.28M samples uniformly
- Accuracy gain: 5 percentage points
- $\text{ICR}_{\text{random}}$ = 5 percentage points / (3.2×10^{16} FLOPs) = 1.6×10^{-16} per FLOP

Difficulty-scored coreset (EL2N, a training-dynamics score defined in section 9.3.4; 50 percent of data):

- Process 640.6K selected samples
- Assumed accuracy gain: 4.5 percentage points (90 percent of the full-data gain)
- Compute: 640.6K $\times$ 24.6 GFLOP = 1.6×10^{16} FLOPs
- $\text{ICR}_{\text{coreset}}$ = 4.5 percentage points / (1.6×10^{16} FLOPs) = 2.9×10^{-16} per FLOP

Systems insight: Under these assumptions, the selected subset achieves 1.8× higher ICR with a 0.5 percentage points smaller accuracy gain. The comparison is learning per FLOP over one epoch of each dataset. It does not show that the coreset reaches the full-data run's final accuracy, which must be checked separately at equal quality, and a real decision also requires measured gains, selection overhead, and coverage checks.

Checkpoint 9.1: Data selection efficiency

The goal is to improve ICR subject to target quality and required coverage, then verify that the resulting pipeline reduces the cost it is intended to address.

Metric checks

- ☐ **ICR application**: Given two training runs with identical accuracy gains but different compute budgets, determine which achieved higher ICR.
- ☐ **Data efficiency**: The coreset notebook reports 1.8× higher ICR but a smaller per-epoch gain. Explain what would have to change for a 50 percent subset to match the full-data gain, which would give it 2× higher ICR and so deliver the same measured gain for half the training FLOPs.

Pipeline checks

- ☐ **The three stages**: Map static pruning, dynamic selection, and synthetic generation to the training lifecycle.

A reduction in sample count remains a claim until it is shown to translate into end-to-end efficiency without silently compromising generalization. Notebook 9.1 holds per-sample work fixed and assumes its accuracy gains. Testing the claim on a real workload first requires knowing whether

the run is limited by data at all, and then measured metrics that weigh arithmetic savings, data movement overhead, and task quality across the complete pipeline.

9.2.3 The compute-optimal frontier

Pruning a dataset when the model is already starved for unique tokens hurts accuracy rather than saving time, whereas collecting more data for an under-trained model wastes money on collection. Before evaluating any selection technique, a team must therefore test whether the training run is limited by data or by compute. Neural scaling-law experiments (Kaplan et al. 2020; Hoffmann et al. 2022) fit power-law relationships over particular model families, datasets, and compute ranges. Controlled sweeps can use those fitted relationships to compare model and data allocations, though no single ratio diagnoses every training run.

The Chinchilla study[3] (Hoffmann et al. 2022) fit a compute-optimal balance between model size and training data within its experimental regime. Allocations on either side of that fitted balance used the study's compute budget less effectively. The optimal balance defines a **compute-optimal frontier**: the best achievable performance at each compute budget when data and model size are properly balanced. Plotted over normalized training compute, the frontier separates the optimal operating envelope from the compute-starved and data-starved allocations in the suboptimal region below it (figure 9.2).

[3] **Chinchilla**: Chinchilla is a 70-billion-parameter language model trained on 1.4 trillion tokens. In the experiments reported by Hoffmann et al. (2022), it outperformed the larger GPT-3 on most evaluated tasks under a similar training-compute budget. The fitted compute-optimal allocation scaled parameters and tokens at roughly equal rates within the study's experimental regime; it is not a universal prescription for every architecture, dataset, or training recipe.

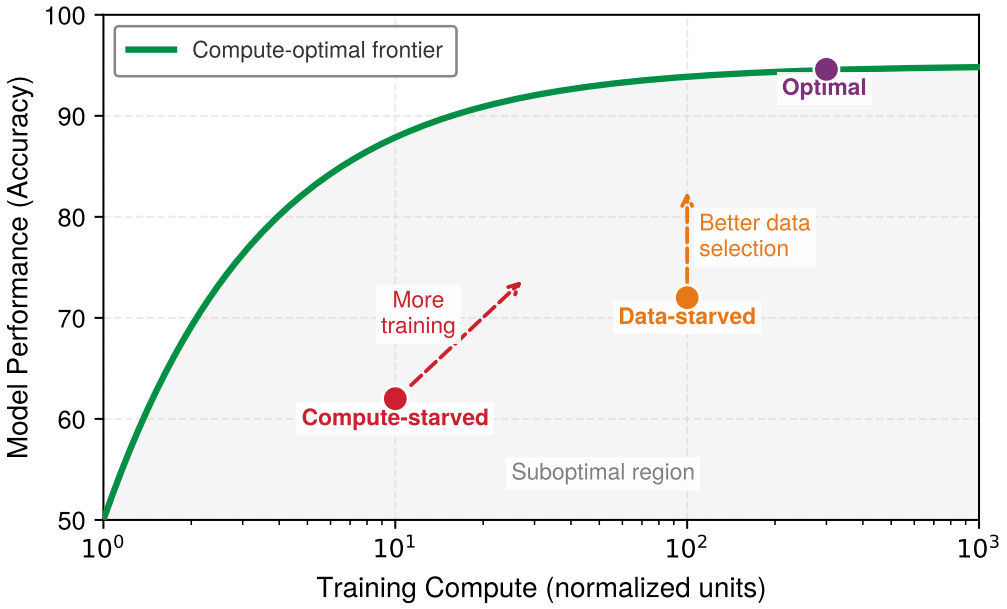


Figure 9.2: Illustrative Compute-Optimal Frontier: The green curve is a conceptual frontier over normalized compute, not a fitted result from Chinchilla. Points below it illustrate allocations that a controlled model-size and token-count sweep might identify as data- or compute-limited.

The frontier answers one allocation question, whether the next unit of compute at a given budget should go to more parameters or to more training tokens. A run is *data-starved* when it trains on too few unique, useful tokens for its parameter count, so its tokens-per-parameter ratio sits well under the fitted balance, and the same compute would yield more accuracy with more or better data. Against a frontier fitted from controlled sweeps, an operating point's location motivates further experiments. The directional vectors charted in figure 9.2 illustrate two candidate interventions: additional training compute and improved data selection at fixed compute. Controlled changes to model size, data, and training allocation determine which intervention helps; a point below the frontier does not identify a unique remedy. Points near a fitted frontier remain specific to the model family, dataset, objective, and compute range used to estimate it.

Interpreting the Chinchilla result

Within the fitted Chinchilla regime, compute-optimal model parameters and training tokens grew at roughly equal rates.[4] Training a dense transformer costs approximately $6PD$ FLOPs for P parameters and D tokens, the forward-plus-backward accounting derived for the GPT-2 lighthouse in section 8.1.1. Under an equal scaling regime where parameters and tokens grow proportionally ($P \propto D$), total compute scales quadratically ($O \propto D^2$), directly implying that optimal token count scales as the square root of the compute budget ($D_{\text{opt}} \propto \sqrt{O}$). Doubling compute therefore corresponds to about $\sqrt{2}-1$, or 41.4 percent, more tokens under those assumptions. The commonly quoted 20 tokens per parameter is a useful reference point from that study, not a universal optimum.

GPT-2 sits well short of that reference. WebText supplies about 6.7 unique tokens per parameter, a third of the fitted balance, and the original run made up the difference by repeating those tokens about 17 times (section 9.1). Repetition keeps the arithmetic budget but not the learning per FLOP, which is the regime where selection, rather than more compute, moves the run toward the frontier.

[4] **Tokens per parameter**: The ratio D/P compares training tokens with model parameters; GPT-3 used approximately 1.7 tokens per parameter (Brown et al. 2020), while Llama 2 70B used approximately 28.6 (Touvron, Martin, et al. 2023). These descriptive ratios do not establish whether either run is compute-optimal under a different architecture, corpus, objective, or hardware budget. Controlled model-size and token-count sweeps are required for that diagnosis.

Applying the diagnostic

If a training run underperforms expectations, extending one run can reveal whether additional optimization steps still help, but a plateau does not identify data starvation by itself. Learning-rate schedules, optimization limits, model capacity, and data quality can all produce a plateau. A reliable diagnosis compares controlled runs that vary model size, token count, and compute allocation while holding the evaluation protocol fixed. When controlled runs do point to the data, the remedy that keeps compute fixed is better selection, which changes the shape of the learning curve itself. Seeing that change requires switching axes, from the frontier's training compute to dataset size, where the core metrics measure what a reduction preserves.

9.2.4 Core metrics

Reporting accuracy at a single subset size hides whether a selection method learned faster early on, plateaued lower, or never reached the full dataset's target quality. Three metrics track those distinct behaviors across the learning curve and complement the measured information-compute ratio (section 9.2.1). Performance-per-data (PPD) measures average accuracy gain per sample, which becomes gain per FLOP once per-sample cost is fixed; the area under the learning curve (AULC) summarizes performance across a common dataset-size range; and the data compression ratio (DCR) measures retained sample reduction at a fixed target accuracy. None of the three establishes the causal value of an individual example.

Performance-per-data

The most direct metric, **performance-per-data (PPD)**, measures accuracy gain per sample:

$$\text{PPD}(n) = \frac{\text{Accuracy}(n) - \text{Accuracy}(0)}{n}$$

where n is the number of training samples. A higher PPD indicates greater average improvement per sample over the stated interval. Its shape must be measured because diminishing returns and their onset are workload-dependent.

Area under the learning curve

Rather than comparing at a single point, the **area under the learning curve** (AULC) integrates performance across all dataset sizes:

$$\text{AULC} = \int_0^D \text{Accuracy}(n)\, dn$$

where n is the dataset size and D is the total dataset size.

For the same metric and integration range, a higher AULC means stronger integrated performance across dataset sizes, though one strategy need not outperform the other at every size. Comparisons must use the same upper limit D or normalize the integral. Measuring AULC requires training at

several dataset sizes, so the metric itself spends compute, and when it guides a selection decision that cost belongs to the selection overhead.

In the illustrative curves of figure 9.3, efficient selection bends into an accuracy plateau near 25 percent of the data while uniform random sampling climbs gradually across the entire budget, so over the same range the efficient curve encloses the larger area. The horizontal separation between the curves is the reduction in required samples, and at an identical 20 percent data budget the vertical gap is the accuracy that selection adds without any additional compute.

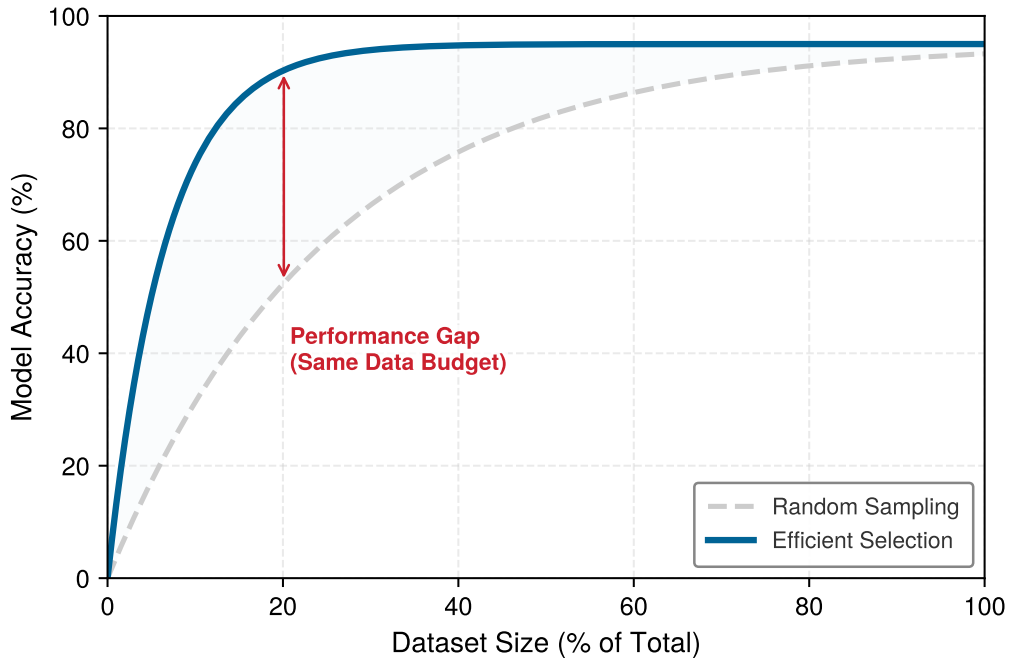


Figure 9.3: Illustrative Diminishing Returns of Data: The synthetic data-efficient curve (blue) reaches the same plateau with fewer samples than random sampling (gray). The vertical red arrow marks the modeled performance gap at a fixed dataset size; measured curves need not have this shape.

Data compression ratio

For coreset methods, the **data compression ratio** (DCR) measures how much data reduction is achieved at a target accuracy:

$$\text{DCR} = \frac{D_{\text{total}}}{D_{\text{subset}}} \text{ at Accuracy}_{\text{target}}$$

A DCR of 5× means the coreset achieves target accuracy with 20 percent of the data. Applied to the coreset notebook, the three metrics behave differently. Over one epoch, the full ImageNet set gains 5 percentage points while the half-size coreset gains 4.5 percentage points, so the coreset's PPD is higher by the same 1.8× as its ICR, because per-sample work is fixed. DCR cannot be computed from that notebook at all, because the two runs never reach a common target accuracy. Exposing that missing equal-quality check is exactly the job DCR is designed for, and even when a reduced dataset matches the full dataset's aggregate accuracy, those learning-curve metrics cannot show whether the retained subset still covers every slice deployment needs.

9.2.5 Validating a reduced dataset

When the half-size ImageNet coreset drops 640,000 images, it can hold top-1 test accuracy while quietly stripping out a rare class, shifting the training distribution away from production traffic, or

concentrating mislabeled examples in the retained subset. Validating a curated dataset therefore requires checking coverage, distribution alignment, and label quality alongside aggregate accuracy. Coverage metrics check whether coreset selection kept enough examples from each class and demographic group. Even when class counts hold, the retained subset can skew toward atypical examples, so distribution alignment metrics (such as KL divergence and the population stability index, the drift statistics of section 4.12.1) test whether the curated training set has drifted from the deployment distribution. Where selection also guides annotation, label quality metrics (inter-annotator agreement and confident learning, which flags examples whose given label disagrees with a model's confident out-of-sample prediction) verify that active learning did not introduce systematic labeling errors. A 50 percent dataset reduction is worth keeping only when all three checks pass alongside target accuracy, calibration, and robustness.

The validation target can itself be unreliable. Benchmark accuracy can overstate generalization when test sets fail to capture operational distribution shifts, demanding the same scrutiny for evaluation benchmarks as for curated training sets.

↪ Coverage, label-quality, and distribution-alignment checks are developed into a full data-benchmarking protocol in section 12.9.2.

Example 9.1: The benchmark replication gap

Scenario: Classifiers evaluated on newly collected test sets experienced 3 to 15 percentage-point accuracy drops on CIFAR-10 and 11 to 14 points on ImageNet (Recht et al. 2019).

Diagnosis: The replicated collection produced slightly harder images than the original test sets. The study found that benchmark adaptivity did not explain the gap, even though repeated use of a fixed test set remains a general evaluation risk.

Systems lesson: Evaluation-set construction affects measured generalization. Data-selection pipelines should validate curated subsets on independently collected and deployment-relevant test sets rather than trusting one long-reused benchmark.

An efficiency gain measured against a single benchmark can evaporate under distribution shift, so a curated subset must demonstrate consistent quality across multiple held-out distributions before its reported reduction ratio is trusted. Even when quality holds across those distributions, the time spent selecting the subset must still be weighed against the training time it saves.

9.2.6 The selection inequality

The efficiency measures so far count learning against training compute alone, yet a selection method also spends time choosing, and a claim that it shortens training must count that time. To reduce overall training time at a fixed quality target, selection plus subset training must take less time than training on the full corpus. Let $T_{\text{selection}}$ be total wall-clock selection overhead, including candidate scoring and any prerequisite scorer preparation charged to the training run. Let $T_{\text{train}}(D)$ be wall-clock training time for a dataset of size D, and let D_{subset} and D_{total} be the retained and full sample counts. Equation 9.2 formalizes this break-even condition as the **selection inequality**:

$$T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}}) \tag{9.2}$$

Equivalently, isolating the allowable selection overhead yields $T_{\text{selection}} < T_{\text{train}}(D_{\text{total}}) - T_{\text{train}}(D_{\text{subset}})$. The upper bound on selection latency depends strictly on how much training execution the subset eliminates; no arbitrary overhead fraction applies across workloads. When an engineering pipeline can amortize scorer preparation or checkpoint extraction across multiple downstream training runs (section 9.8.3), $T_{\text{selection}}$ reflects only the marginal scoring pass over the candidate pool. Conversely, if an auxiliary proxy must be trained from scratch for a single isolated run, that preparatory compute adds directly to $T_{\text{selection}}$. When $f(x)$ repeatedly scores the candidate pool with the full target model, those extra scoring passes and their cross-worker coordination can take longer than the training steps they remove, making the run slower overall.

Example 9.2: Selection inequality in practice

Scenario: A computer vision team selects a 128,117-image coreset (10 percent) from the 1.28 million ImageNet training images to shorten ResNet-50 training on one H100 sustaining 25 percent of its peak, the training chapter's compute-bound rate. Pretrained checkpoints for both the target architecture and a lightweight proxy are already available from earlier exploration, so this comparison accounts only for a marginal scoring pass over the candidate pool.

Diagnosis: Against a 211-minute full-data run of 100 epochs, marginal full-model scoring (Option A: target ResNet-50) adds 42 seconds of selection overhead and still saves 188.8 minutes (89.7 percent). Marginal proxy-model scoring (Option B: ResNet-18) drops selection overhead to 18 seconds and saves 189.1 minutes (89.9 percent), only 24 seconds more.

Systems lesson: Both options satisfy the selection inequality by a wide margin, because a single scoring pass costs at most one forward pass over the dataset (one-third of a full-data epoch for the target ResNet-50) and less with the proxy, while training on the 10 percent coreset shrinks every one of the 100 epochs. Scorer choice also becomes decisive when selection rescores dynamically every epoch or when the scoring architecture is significantly larger than the target, multiplying selection latency while subset training savings remain fixed.

End-to-end wall-clock time divides into selection overhead and subset training, and the split decides the outcome across three regimes (figure 9.4): lightweight proxy scoring limits selection overhead to 5 percent while training on the 40 percent subset (yielding 55 percent net savings), whereas heavyweight scoring expends 60 percent overhead on selection, wiping out all subset-training efficiency gains.

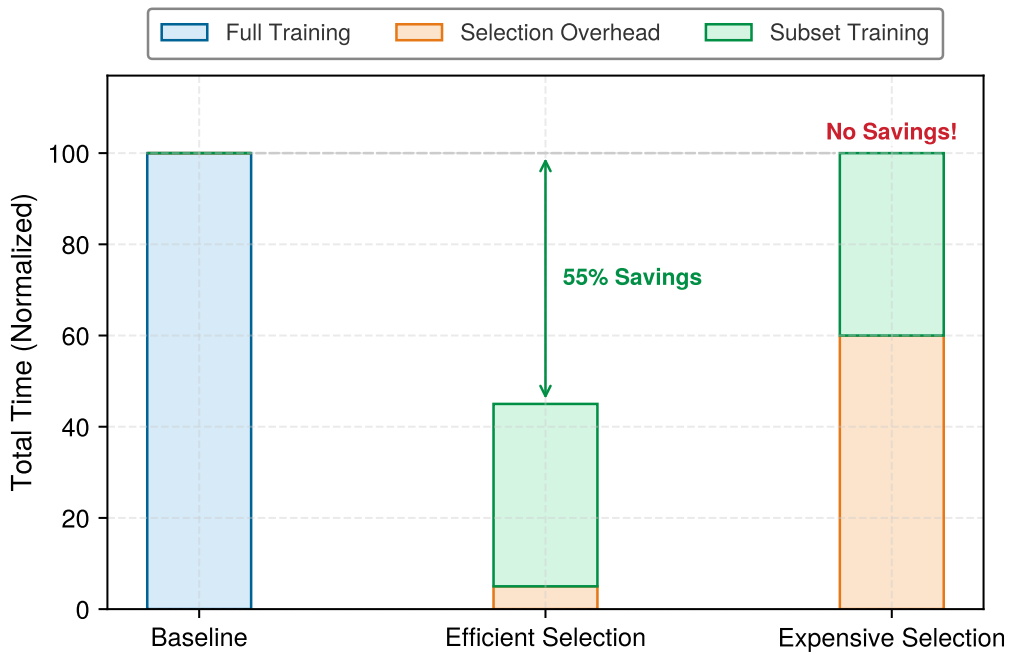


Figure 9.4: The Selection Inequality: In this illustrative example, data selection only improves end-to-end training time at matched quality if the overhead of selection plus training on the subset is less than training on the full dataset. A lightweight selection function (proxy model, cached embeddings) keeps selection overhead low; an expensive selection function (full model forward pass) can negate the savings.

This overhead compounds when selection runs dynamically across epochs rather than once offline. If per-epoch rescoring approaches the training work saved by the pruned subset, the net systems

advantage vanishes. Measured this way, a selection strategy is judged against its stated objective, counting the overhead of scoring, filtering, and staging samples alongside the training, labeling, or reuse savings it claims at target quality. Teams that optimize selection algorithms against isolated benchmarks rather than end-to-end pipeline constraints find that intuitive heuristics backfire: scoring overhead consumes accelerator cycles, optimization trajectories destabilize, or generalization silently degrades. These measures apply to the three levers in the order in which they act, and static pruning comes first because it fixes the subset before target-model training, though its scoring process may use a trained proxy.

9.3 Static Pruning

Web-scale scraping routinely gathers boilerplate text, duplicate images, and malformed records, and each one costs GPT-2 six FLOPs per parameter for every token, or ResNet-50 a full forward and backward pass, before a single useful gradient is calculated. Static pruning removes these low-value records upstream of the training loop. By dropping redundant and corrupt records before target-model training begins, static pruning shrinks the operation count (O) without changing model architectures or execution kernels.

9.3.1 The case for smaller datasets

In many training runs, a large fraction of examples contributes almost no gradient signal while still costing a full forward and backward pass. Empirical scaling studies confirm that dataset volume alone does not determine model accuracy, because large corpora carry substantial redundancy. On CIFAR-10, Paul et al. (2021) report that EL2N pruning (which scores each example by its prediction error early in training) removed half of the training data without reducing accuracy. On ImageNet-1K, Sorscher et al. (2022) report that a self-supervised prototype metric discarded 20 percent of the data without sacrificing performance. The pattern extends to language modeling, where web-scraped corpora like The Pile[5] and C4[6] contain enough duplicate and templated content to make deduplication a systems issue. In the datasets studied by Lee et al. (2022), deduplicated training reduced memorization while preserving or improving perplexity. Still, these reported gains are benchmark-specific: pruning effectiveness depends on how redundant the dataset is, which selection algorithm is used, and the capacity of the target model, so teams must validate aggressive pruning on their own task before committing to it.

Individual data points need not provide equal value for training. This heterogeneity follows in part from how classifiers learn decision boundaries. Many samples fall far from a boundary: a picture of a dog in good lighting may become easy once the model has learned its dominant features, while ambiguous and underrepresented cases can remain informative. Label quality also affects data requirements. Conflicting labels can push updates in opposing directions, so reaching a target error may require more examples or more passes over them. The size of that penalty depends on the task and training procedure. A **data quality multiplier** therefore comes from comparing learning curves at the same held-out error, rather than assigning a universal convergence rate to all noisy data.

Napkin Math 9.2: The data quality multiplier

Problem: Two hypothetical pilot learning curves reach the same held-out error using different amounts of clean and noisy data. With epoch count and per-sample work held fixed, what training-work multiplier do these sample counts imply?

Setup: Consider a representative evaluation where a fixed model architecture and training recipe reach target error $\epsilon = 0.01$ (1 percent) under clean versus noisy training partitions:

- Clean data: $D_{\text{clean}} = 10{,}000$
- Noisy data: $D_{\text{noisy}} = 16{,}000$

Result: Dividing the noisy-data count by the clean-data count gives 1.6×. Under the fixed training recipe, the noisy dataset therefore requires 1.6× as much training work to reach the same error.

5 **The Pile**: An approximately 886 GB English text corpus (reported as 825 gibibytes by the source) aggregating twenty-two subdatasets, including PubMed, ArXiv, GitHub, Project Gutenberg, Common Crawl, Stack Exchange, Wikipedia, and USPTO patents (Gao et al. 2020). Its multi-source design makes it a useful example of data diversity: each source family has a different duplication, quality, and domain-coverage profile, so selection pipelines must preserve coverage while removing redundant text.

6 **C4 (Colossal Clean Crawled Corpus)**: C4 applies filtering to Common Crawl data, including language detection, deduplication of repeated three-sentence spans, and removal of pages that fail heuristic quality checks, producing approximately 750 GB of cleaned English text (Raffel et al. 2020). Filtering has its own compute cost and can remove useful material, so its ROI and quality effects must be measured rather than inferred from corpus size alone.

Systems insight: The avoided training work sets a budget for cleaning. Cleaning pays for itself only when correction or review costs less than the work it removes, with required coverage preserved. If the two learning curves require different epoch counts, compare total examples processed to the target instead of unique sample counts. The decision depends on measured learning curves and pipeline costs, not on the presence of noise alone.

9.3.2 Data deduplication

The cheapest samples to remove are the ones that repeat what the dataset already contains. Repeated content costs a full forward and backward pass on every presentation, even when that repetition adds little useful learning signal. Exact hashing and shingle-based near-duplicate detection can identify candidates for removal without training a scoring model. Deduplication therefore comes first among the static levers, but near-duplicate thresholds must be validated so the pipeline does not remove useful distributional signal.

The simplest form of deduplication uses hash-based methods for exact matches. By computing a cryptographic hash (MD5 or SHA-256) for each sample and removing those with identical hashes, practitioners can eliminate the byte-for-byte duplicates that accumulate in large web-scraped corpora. This process is computationally cheap, scaling linearly with dataset size, and can be parallelized trivially.

Near-duplicate detection addresses content that differs at the byte level while retaining substantial token- or character-shingle overlap. For text, MinHash[7] with Locality-Sensitive Hashing[8] (LSH) approximates Jaccard similarity[9] efficiently. In practice, the pipeline converts each document into a set of overlapping character or word w-shingles, compresses these sets into fixed-length MinHash signatures where the probability of hash collision equals Jaccard similarity, and partitions the signatures into L bands of r rows using LSH. Documents matching across all rows in any single band collide in the same bucket, restricting expensive similarity verification to likely candidates and avoiding an impossible $\mathcal{O}(D^2)$ all-pairs comparison across a corpus of D web documents numbering in the billions. It can detect lightly edited content with high overlap, but it does not establish semantic paraphrase equivalence.

For images, perceptual hashing produces signatures robust to minor transformations like resizing and compression, identifying visually similar candidates stored in different formats. Embedding-based similarity can retrieve semantic near-neighbor candidates by computing dense representations (CLIP[10] (Radford et al. 2021) for images, sentence transformers for text), though proximity does not itself prove duplication and the approach incurs higher computational overhead. Across both modalities, deduplication can reduce repeated-content memorization and avoid spending FLOPs on exact or near duplicates, yet it can also alter the corpus's intentional empirical weighting, so thresholds and downstream quality must be validated rather than assuming that every duplicate is harmful.

On GPT-2's run, deduplication costs little and returns hours of accelerator time. Lee et al. (2022) found that near-duplicate removal affected between 3.04 percent (C4) and 13.63 percent (RealNews) of the corpora they studied. If WebText carried a comparable share, removing it would cut the same fraction of the 167 billion tokens GPT-2 processed, saving about 3 hours to 14 hours of the 4.4 days that one eight-H100 machine needs, or 18 kWh to 80 kWh of accelerator energy. Finding the duplicates reads the 40 GB corpus once, about 6 seconds at the 7 GB/s of the machine's local NVMe storage, and holds a 128-hash MinHash signature for each of WebText's roughly 8 million documents, about 4.1 GB of host memory. Hashing runs on host CPUs rather than on the accelerators, so the cost of deduplication lands on a different stage of the machine from the work it removes; the open question for a corpus this size is which near-duplicates are safe to remove, not whether the machine can afford to find them. Once those duplicates are gone, the next target is records that are unique but broken.

9.3.3 Data pruning by quality

A unique training example can still waste compute or actively damage the model if its content is garbled or its label contradicts the true class. **Quality-based pruning** removes samples that carry no usable signal or push gradients in the wrong direction. In supervised datasets, label error detection

↩ Deduplication as a deterministic data-cleaning step that keeps training and serving pipelines consistent is established in section 4.7.1.

7 | **MinHash**: Invented by Broder (1997) to detect duplicate web pages for AltaVista, the algorithm creates compact signatures using random hash functions such that similar documents produce similar signatures with high probability. Each signature compresses a document to a fixed-size sketch, enabling pairwise similarity estimation in $\mathcal{O}(s)$ time for sketch size s.

8 | **Locality-sensitive hashing (LSH)**: LSH hashes MinHash signatures into buckets so that similar signatures are more likely to collide. Candidate generation avoids an exhaustive all-pairs comparison, while runtime and memory depend on signature width, banding, bucket sizes, and the number of candidates that require verification.

9 | **Jaccard similarity**: Defined as $|\mathcal{A} \cap \mathcal{B}|/|\mathcal{A} \cup \mathcal{B}|$ for two sets $\mathcal{A}$ and $\mathcal{B}$, ranging from 0 (disjoint) to 1 (identical). For deduplication, the metric compares sets of shingles across documents of different lengths. The threshold must be calibrated for the shingle definition, corpus, and downstream quality because no universal cutoff separates duplicates from legitimately related documents.

10 | **CLIP (Contrastive Language-Image Pretraining)**: Pretrained on 400 million image-text pairs, CLIP maps semantically related images and text into a shared embedding space (Radford et al. 2021). Embeddings can retrieve candidates for review, but semantic proximity is not proof that two samples are duplicates. Generating embeddings also costs substantially more than computing perceptual hashes, with the ratio depending on model and hardware.

is often the highest-return form of quality pruning. Tools like Cleanlab identify samples where the assigned label may be incorrect based on model confidence patterns across training: a sample that the model consistently predicts as class A while its label says class B may be a hard boundary case, a distribution shift, a model blind spot, or an annotation mistake. Because those causes call for different remedies, the score should trigger review rather than automatic deletion, so confirmed errors can be corrected or removed without discarding hard examples.

Where label error detection checks whether the target contradicts the input, outlier removal checks whether the input itself sits far from any cluster center in feature space. Such points can indicate sensor noise, annotation errors, or file corruption, whereas in heavy-tailed domains they can also be the rare edge cases that define deployment risk, so outlier thresholds must stay conservative and the affected slices should be reviewed before removal.

Unlabeled pretraining corpora have no class labels to audit, so low-information filtering instead applies domain-specific heuristics to drop samples that carry too little signal to justify a training step. For text corpora, pipelines remove high-perplexity garbled strings (perplexity measures how surprising a text is to a language model, so high values can flag incoherent text as well as valid specialist or rare-language writing) alongside low-perplexity boilerplate or repetitive templates; for image datasets, they filter blurry, corrupted, or near-uniform frames. Because hand-written heuristics can miss subtler quality differences across billions of documents, industrial pretraining pipelines pair them with *classifier-based filtering*, which trains a lightweight linear classifier (such as fastText) using curated reference text (e.g., Wikipedia, books, and scientific papers) as positive examples and uncurated web dumps as negative examples. The classifier assigns each scraped document a continuous quality score, allowing the pipeline to drop millions of low-quality pages at gigabyte-per-second ingestion speeds before ranking the clean examples that remain.

9.3.4 Coreset selection algorithms

Once duplicates and defective records are gone, the remaining samples are unique and valid but still unequal in value, which is the heterogeneity described in section 9.3.1. Ranking them requires systematic selection criteria. **Coreset selection**[11] turns the static pruning decision into a coverage problem: keep the smallest subset that preserves the statistical properties of the entire dataset. The systems decision is where to spend the selection budget, whether on cheap coverage metrics that preserve distributional structure or on costlier training-dynamics scores that better target the decision boundary. That decision also needs a guardrail before any score is applied: classes, demographic groups, time windows, and rare failure modes that matter at deployment require minimum representation, because the highest-average-ICR subset can still remove the examples that define production risk.

Geometry-based methods select samples that cover a chosen representation without requiring target-model training. The k-Center algorithm,[12] a facility-location-style objective, selects samples that reduce the maximum distance from any point to its nearest selected center under the chosen embedding and metric. Herding takes a different approach: Welling's original procedure iteratively constructs representative pseudo-samples whose feature statistics approximate observed moments (Welling 2009), and related moment-matching objectives can guide subset selection from a fixed pool. These methods are computationally attractive because they operate on feature representations, but their scores are not inherently label-aware.

Training-dynamics methods use a proxy model to identify examples that may be important. GraNd (Gradient Normed) and **EL2N (error L2-norm)**[13] score samples by gradient magnitude or prediction error early in training (Paul et al. 2021). The exact alternative, influence functions (Koh and Liang 2017), measures how removing a sample changes validation loss, but it requires the inverse Hessian $\mathbf{H}^{-1}$ of the loss across all P parameters, with $\mathcal{O}(P^2)$ memory and $\mathcal{O}(P^3)$ arithmetic, which rules it out for models with millions or billions of parameters.

Systems methods circumvent this barrier by relying on lightweight first-order approximations: GraNd scores samples using expected gradient norm ($\|\nabla_\theta \mathcal{L}\|_2$), while EL2N observes that early in training, the gradient norm with respect to linear layers is strongly correlated with output prediction error ($\|\hat{\mathbf{y}} - \mathbf{y}\|_2$, the distance between the predicted probability vector $\hat{\mathbf{y}}$ and the one-hot label $\mathbf{y}$). This enables practitioners to score prediction error using a single forward pass without computing backward-pass gradients. Error and uncertainty differ: a confident prediction of the wrong class can

[11] **Coreset (core set)**: In computational geometry, a small subset can approximate a specified geometric objective within a controlled error factor (Agarwal et al. 2005). Machine learning adapts this idea as a selection principle, but a guarantee for an embedding and distance objective does not by itself guarantee neural-network accuracy. The retained subset must still be validated on the target task.

[12] **k-center algorithm**: Its greedy strategy iteratively picks the point farthest from the existing centers under a chosen embedding and metric. Sener and Savarese use this core-set framing for convolutional-neural-network active learning (Sener and Savarese 2018). The resulting coverage bound applies to that geometric objective, not directly to downstream accuracy or rare-class preservation.

[13] **EL2N (error L2-norm) and GraNd (gradient normed)**: These methods score samples based on their error or gradient norm early in training, identifying samples the model finds most difficult. The practicality of this approach relies on *transferability*, where scores from a small proxy model can guide data selection for a much larger target model. For instance, a proxy trained for five epochs can generate scores to curate a dataset for a full 90-epoch production training run (Paul et al. 2021).

have high EL2N despite low predictive uncertainty. High scores identify examples the proxy finds difficult, but they can also identify noise or unrepresentative samples and therefore need coverage and quality checks. The same work reports useful score transfer across several architectures, which can enable less expensive proxy-based selection. A third training-dynamics signal, forgetting events,[14] tracks how often a sample is correctly classified and later misclassified during training (Toneva et al. 2019).

[14] **Forgetting events**: This method identifies valuable examples by tracking when the model "forgets" them, transitioning from a correct to an incorrect classification during training. The central trade-off is the high cost of this analysis, which requires a full training run. However, the resulting importance scores can transfer from small proxy models to larger target models (for example, ResNet-18 to ResNet-50), enabling inexpensive proxy-based selection when ranking quality holds for the target task (Toneva et al. 2019).

Each algorithm in table 9.2 balances upfront scoring computation against downstream training reduction. Because geometry-based and training-dynamics methods optimize different proxies, their relative ranking depends on dataset scale, feature representations, target model capacity, and scoring budgets; a score is computationally justified only when downstream training savings exceed evaluation overhead.

Table 9.2: Coreset Selection Algorithm Comparison: D = dataset size, K = coreset size. Geometry-based and training-dynamics methods optimize different proxies, so their quality ranking depends on the dataset, representation, target model, and scoring budget.

Method	Compute Cost	Requires Training	Best For	Limitation
k-Center	$\mathcal{O}(D^2)$ or $\mathcal{O}(DK)$	No	Coverage, exploration	Ignores label information
Herding	$\mathcal{O}(DK)$	No	Distribution matching	Matches chosen moments
GraNd	$\mathcal{O}(\text{epochs} \times D)$	Yes (few epochs)	Decision boundaries	Requires proxy training
EL2N	$\mathcal{O}(\text{epochs} \times D)$	Yes (few epochs)	Error-based scoring	Can rank noise as hard
Forgetting	$\mathcal{O}(\text{full training})$	Yes (full)	Hard examples	Expensive to compute

Geometry-based methods prioritize coverage in a chosen feature representation, while training-dynamics methods use signals such as error or forgetting. Uncertainty sampling, which active learning uses as a query rule (section 9.4.2), concentrates the budget near the current decision boundary; EL2N instead ranks prediction error, which can also be high for confidently wrong examples away from the boundary. Figure 9.5 contrasts that boundary-focused allocation with uniform random sampling of the same five-point budget. It illustrates one family of selection rules; a geometry-based method such as k-center would instead spread its picks to cover the feature space.

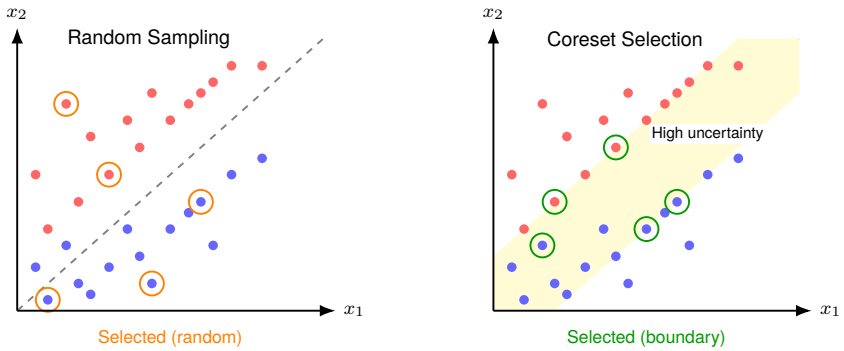


Figure 9.5: Illustrative Boundary-Focused Selection: Both panels select five samples, isolating allocation rather than budget. Random sampling distributes those choices across the feature space; a boundary-focused rule such as uncertainty sampling concentrates them in the shaded band around the current decision boundary.

EL2N with a small proxy model offers one practical balance between scoring cost and selection quality. The approach trains a lightweight model for a few epochs, computes EL2N scores, and selects a subset subject to coverage and quality checks. The proxy must provide rankings that transfer to the target model, and the investment pays off only when downstream training savings exceed selection overhead. A concrete scenario shows the workflow; whether its 10 percent subset preserves accuracy is exactly what validation must establish.

Example 9.3: Coreset selection in practice

Scenario: An engineering team aims to select a 128,117-image coreset (10 percent) from the 1.28 million ImageNet training images for faster ResNet-50 iteration.

Diagnosis: Naive random subsampling can miss rare classes and boundary examples. Training a lightweight proxy model for 5 epochs calculates EL2N error scores, prioritizing high-error examples while checking rare-class coverage.

Systems lesson: Proxy-based coreset selection replaces some full-data passes with targeted scoring compute. It accelerates full-model training only when the proxy ranking transfers and coverage checks preserve rare classes and edge cases.

Listing 9.1 turns proxy prediction error into reusable selected indices. The `compute_el2n_scores` function spends a small amount of probe compute to measure which samples still confuse a briefly trained model, and `select_coreset` retains the highest-scoring examples. This top-k sketch needs quality and coverage checks before its subset replaces the full training pool. The scoring pass requires an unshuffled dataloader so each score position remains aligned with its dataset index.

Listing 9.1: EL2N-Based Coreset Selection: The `compute_el2n_scores` function trains a proxy model for a few epochs, then measures prediction error via L2 distance from one-hot labels. High scores identify samples the proxy finds difficult, including potentially noisy examples. The `select_coreset` function illustrates ranking by this score; production use also requires coverage, noise, and target-quality validation.

```
def compute_el2n_scores(model, dataloader, num_epochs=5):
    """Compute EL2N scores.

    Returns L2 norm of (prediction - one_hot_label).
    """
    # Train proxy model for a few epochs to get meaningful predictions
    train_proxy(model, dataloader, num_epochs)

    scores = []
    model.eval()
    for x, y in dataloader:
        logits = model(x)
        probs = softmax(logits, dim=1)
        # One-hot encode labels
        one_hot = zeros_like(probs).scatter_(1, y.unsqueeze(1), 1)
        # EL2N score = L2 distance between probabilities and one-hot target
        el2n = (probs - one_hot).norm(dim=1)  # High = large prediction error
        scores.extend(el2n.tolist())
    return scores


def select_coreset(scores, dataset, fraction=0.1):
    """Select top-k samples with the largest prediction errors."""
    k = int(len(dataset) * fraction)
    # Sort by score descending (largest prediction error first)
    indices = argsort(scores, descending=True)[:k]
    return Subset(dataset, indices)


# Illustrative 10% subset; validate accuracy and coverage before use
scores = compute_el2n_scores(proxy_model, full_loader)
coreset = select_coreset(scores, full_dataset, fraction=0.1)
train_full_model(model, coreset)
```

A coverage floor changes the indices retained by the score ranking. Consider disjoint classes A and B, a total coreset budget K as in table 9.2, and task-validated minimum counts a and b for the respective classes. First reserve the highest-scoring a eligible examples from A and the highest-scoring b from B. Fill the remaining $K - a - b$ slots from the global ranking, skipping indices already

reserved. The allocation is infeasible if $K < a + b$, if either class lacks enough eligible examples to meet its floor, or if the eligible pool is smaller than K; the pipeline must revise the budget or acquire missing coverage before training.

Suppose the unconstrained top-K ranking contains only A examples and the budget can meet both floors. Reserving B's floor replaces the lowest-scoring b members of that all-A subset with the highest-scoring eligible B examples, while the remaining slots still follow the score order. Coverage thus has an explicit opportunity cost in the selection score. Overlapping demographic or operational slices require joint constraint handling because one example may satisfy several floors; summing those floors as if the slices were disjoint can reject a feasible budget. Even a feasible allocation establishes only retained counts. Independent evaluation must still establish target quality and performance on the required slices.

Together, these three static pruning techniques (deduplication, quality filtering, and coreset selection) shrink O before training begins, subject to the same fixed-schedule caveat as before: a schedule that adds compensating epochs or steps gives back part of the saving. Static pruning answers a question about *what* to keep, but it treats the answer as fixed, training every epoch on the same retained subset. A sample's usefulness, however, can change as the model learns, because examples that challenge an undertrained model may become easy after sufficient gradient updates. Dynamic selection techniques address this limitation by adapting the training data at each stage based on what the model has already mastered.

9.4 Dynamic Selection

How much a sample teaches changes as the model learns. Early in optimization, broad coverage provides stable gradients that establish general feature representations. As parameters converge, the gradient norms of mastered examples decay toward zero; on a ResNet-50 run over ImageNet, these are the clean, centered images of common classes. Executing forward and backward passes on these samples continues to consume accelerator execution cycles and high-bandwidth memory (HBM) transfers while producing negligible parameter updates, driving their effective ICR toward zero. Dynamic selection mitigates this waste by adapting batch composition to the model's current state. This shifts selection from an offline preprocessing step into the active execution loop, modifying the operation count (O) at runtime. The same adaptive logic extends to labels, where active learning decides which examples receive a human label as the model's uncertainty shifts.

9.4.1 Curriculum learning

Random shuffling feeds hard boundary cases, noisy labels, and outliers to an untrained model before its early features have stabilized, while still spending full epochs on simple examples long after the model has mastered them. **Curriculum learning**[15] (Bengio et al. 2009; Soviany et al. 2022) orders the training stream instead of shuffling it uniformly, starting with simpler examples and gradually introducing harder ones as parameters converge. Ordering the stream changes which gradients dominate at each stage of training: easy examples provide stable early signals, while examples scored as hard can contain useful boundary information, label noise, or outliers. An easy-to-hard schedule can smooth training on some workloads, but its benefit depends on the difficulty score, pacing rule, model, and data; any claimed epoch reduction must therefore be treated as illustrative rather than guaranteed.

Implementing a curriculum requires two components: a difficulty scorer that ranks samples, and a pacing function that controls how quickly hard samples are introduced. A common choice is linear pacing:

$$\text{samples}_{n_{\text{epoch}}} = \texttt{sort_by_difficulty}[: D \cdot \min(1, n_{\text{epoch}}/N_{\text{warmup}})]$$

where n_{epoch} is the current epoch, D is the total dataset size, and N_{warmup} is the number of warmup epochs before the full dataset becomes available. Early epochs train on the easiest fraction $n_{\text{epoch}}/N_{\text{warmup}}$ of the D samples; after warmup, training proceeds on the full dataset. From a systems standpoint, dynamically re-sorting multi-terabyte datasets epoch-by-epoch destroys sequential I/O read-ahead and stalls accelerators on storage latency. Production data loaders implement curricula by pre-sorting data offline into discrete difficulty-tiered shards or buckets. During training, workers stream shards sequentially from storage while maintaining local shuffle buffers within each

[15] **Curriculum learning**: From Latin *currere* ("to run"), originally meaning "the course to be run," a metaphor that maps directly to the technique, with training data as a course run in deliberate order and the easy stretches first. Curriculum learning can act as a continuation method for nonconvex optimization, with easier examples shaping the early optimization trajectory before harder examples are introduced. From a systems perspective, the ICR of a sample can vary during training, which motivates changing the mix of examples over time.

tier, preserving high sequential storage bandwidth while maintaining batch diversity and gradient stability.

The pacing rule fixes one saving before any benchmark is run. Summed over the warmup, the loader serves $\sum_{n_{\text{epoch}}=1}^{N_{\text{warmup}}} D \cdot n_{\text{epoch}}/N_{\text{warmup}} = D(N_{\text{warmup}}+1)/2$ samples in place of $N_{\text{warmup}} \cdot D$, so the warmup epochs cost roughly half their full-data FLOPs. A curriculum can therefore save compute in two distinct ways, through cheaper warmup epochs and through any reduction in the total epochs needed to reach target quality, and a claimed saving must say which of the two it measured.

The difficulty scorer is a systems choice because it trades probe-compute overhead against ordering quality, as table 9.3 shows. Loss and confidence scoring gain better ordering at the cost of extra inference, heuristics avoid compute but require domain knowledge, and self-paced scoring moves adaptation into the training loop itself.

Table 9.3: Difficulty Scoring Strategies for Curriculum Learning: The rows compare each scorer's signal source with the system prerequisite or trade-off it introduces.

Strategy	Difficulty Score	Best For
Loss-Based	Loss from probe model (low = easy)	General-purpose; requires probe training
Confidence-Based	Teacher model confidence (high = easy)	When teacher available; distillation setups
Domain Heuristics	Sentence length, image complexity	No extra compute; domain knowledge required
Self-Paced	Current model's loss (updated each epoch)	Adaptive; no probe needed

To see how curriculum savings translate to systems accounting, consider an illustrative workload where a linear-warmup curriculum reaches target accuracy in 115 epochs compared to 150 epochs for the standard shuffled baseline, an apparent 23.3 percent epoch reduction. Counting epochs alone, however, obscures the actual systems cost. Because linear pacing serves fewer samples during warmup, early epochs consume fewer FLOPs than post-warmup epochs; conversely, difficulty scoring and shard reordering introduce data-loading overhead that must be offset by the training speedup.

Alternative curriculum strategies diverge further in their computational demands. In a learned curriculum, an auxiliary network dynamically weights examples, down-weighting likely mislabeled instances, rather than enforcing a fixed difficulty schedule. Conversely, *anti-curriculum* presents the hardest examples first when boundary definition accelerates early convergence, while *self-paced learning* continuously updates sample difficulty using the current model's training loss. Because optimal sample ordering depends on the architecture, optimizer, and label noise, curriculum gains cannot be assumed from nominal epoch reductions; they must be verified by measuring end-to-end wall-clock time at the target quality metric, and they still assume that every candidate sample already has a label.

9.4.2 Active learning

Curriculum learning optimizes the order in which samples are presented but assumes all samples are already labeled. This assumption breaks down in specialized domains where labeling requires substantial expertise, time, or money. Active learning[16] (Settles 2012; Ren et al. 2021), which section 4.6.5 introduced as a way to stretch a labeling budget, becomes a selection loop here. Rather than labeling everything upfront, it shifts the optimization target from choosing which labeled samples to train on to choosing which unlabeled samples are worth labeling.

Active learning replaces one-shot collection with an iterative loop of five stages (figure 9.6): the selection strategy queries the unlabeled pool based on model uncertainty, routes ambiguous instances to a human oracle, merges newly labeled data into the training set, and retrains the model to update uncertainty scores over the remaining pool. The effectiveness of this loop depends critically on the query strategy used to select samples for annotation (Settles 2009, 2012; Ren et al. 2021). The simplest approach, uncertainty sampling, selects samples where the model is least confident, such as predictions near 0.5 probability for binary classification. Scoring with a single model is inexpensive, yet it can overconcentrate queries near one ambiguous region and miss parameters the model is confidently wrong about. Query-by-committee addresses that single-model blind spot by training

16 **Active learning**: The "active" component is the learning algorithm itself selecting which unlabeled samples a human expert should label next. This reframes the problem from a computational optimization (training on given data) to a financial one: maximizing model improvement per dollar spent on expert labeling. Querying the most informative examples can substantially reduce labeling needs compared with random sampling, but the multiplier is task-, model-, and oracle-dependent.

↩ The labeling budget, and what each label costs at scale, are established in section 4.6.2.

multiple models and selecting samples where they disagree most, capturing epistemic uncertainty at the cost of ensemble inference. When practitioners can invest even more compute per candidate, expected model change selects the samples that would induce the largest gradient update if labeled, trading per-sample gradient evaluations for a more direct estimate of parameter impact. Because any purely uncertainty- or gradient-driven rule can still cluster an entire acquisition batch inside one narrow pocket of feature space, diversity sampling complements these methods by selecting samples that are dissimilar from currently labeled data, spreading the annotation budget across the input distribution.

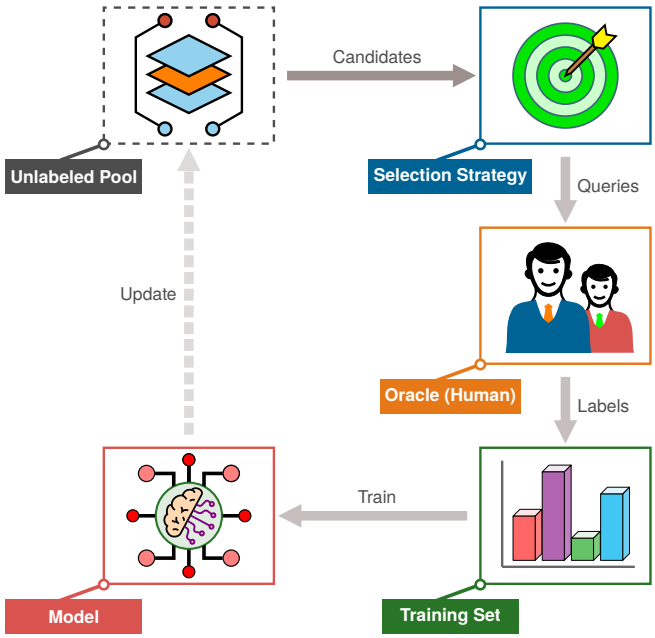


Figure 9.6: The Active Learning Closed Loop: The selection cycle trades proxy-scoring compute for queries chosen from an unlabeled pool for oracle annotation. Uncertainty, diversity, and coverage criteria seek more useful information per annotation than undirected sampling.

Active learning pays off most clearly in domains where every label requires scarce specialist judgment. When a ResNet-50-class image classifier moves from ImageNet to chest X-rays, for instance, every label requires a radiologist, and the model may be confident on common conditions but uncertain about rarer cases. By focusing human annotation on these ambiguous cases, active learning directs expensive expert time toward the examples most likely to move the decision boundary. Because annotation cost and turnaround in such domains dominate the compute needed to score candidates, routing only the examples the model is least sure about to the expert changes the economics of both labeling and per-epoch training work.

Napkin Math 9.3: The active learning ROI

Problem: A hospital team has 1 Million unlabeled scans, a \$5/label specialist-label cost, and a \$500,000 budget. How does uncertainty sampling change labeling cost and per-epoch training work relative to the budget-matched random baseline?

Scenario A: Naive Labeling

- **Cost**: Labeling all 1 Million scans would cost \$5,000,000 (10× over budget).
- **Budget-matched baseline**: $\$500,000 \div \$5/\text{label} = 100,000$ random scans.
- **Naive labeling outcome**: Random selection does not guarantee rare-pathology coverage.

Scenario B: Active Learning

- **Strategy**: Uncertainty sampling selects 50,000 scans for specialist labeling.
- **Cost**: $50,000 \times \$5/\text{label} = \$250,000$ (50 percent under budget).
- **Training work**: $100,000 \div 50,000 = 2\times$ fewer examples per epoch than the budget-matched random baseline.
- **Active learning outcome**: Matching 100,000 random scans requires empirical validation.

Systems insight: Against the budget-matched random baseline, selection saves $\$500,000 - \$250,000 = \$250,000$; accuracy still requires empirical validation.

The notebook's 2× label reduction is an assumption, and the size of that multiplier is what decides the outcome. In the illustrative learning curves of figure 9.7, uncertainty sampling reaches 90 percent accuracy with roughly 2,500 annotations while uniform random sampling needs about 10,000, a 4× gap.

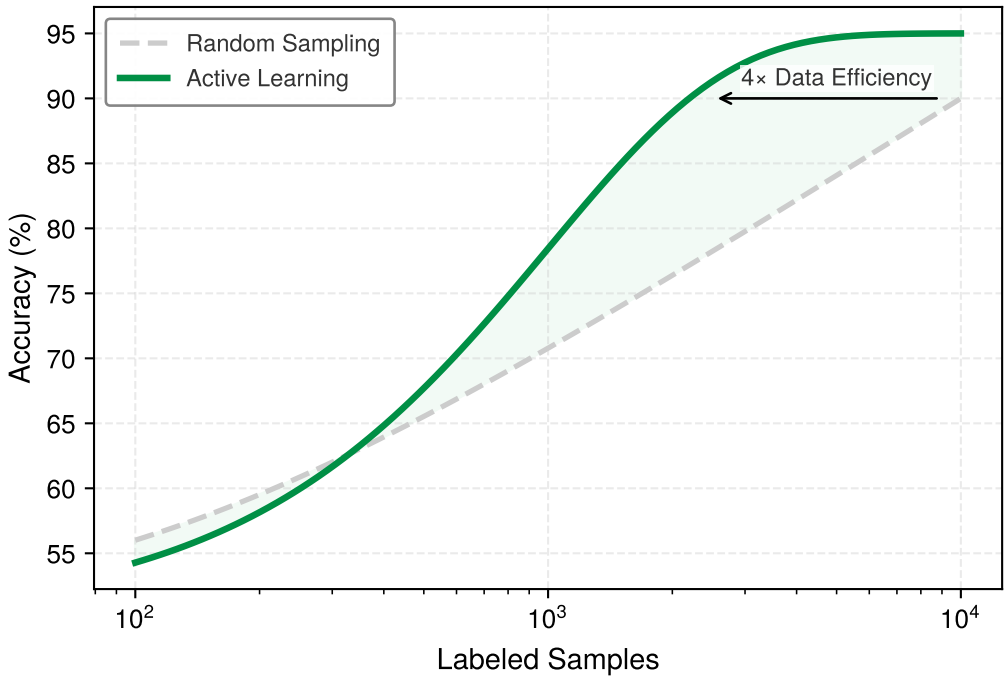


Figure 9.7: The Active Learning Multiplier: Model accuracy is plotted against labeled samples on a logarithmic axis. In this illustrative scenario, active learning (green solid) reaches the same accuracy with fewer labels than random sampling (gray dashed); the arrow marks an approximately 4× label-efficiency gap near 90 percent accuracy.

At a fixed price per label, label count and labeling dollars are the same axis, so the figure and notebook 9.3 differ only in the multiplier they assume. The notebook plans on the conservative 2×; the figure shows what a more favorable task would look like. Because the realized multiplier depends on the task, the model, and the query rule, a team budgets on the conservative value and measures the real one on its own data. Active learning also incurs scoring, acquisition, and repeated-retraining costs, so end-to-end savings can be smaller than the label-count gap; beyond saving labeling cost, however, a suitable query rule concentrates review on uncertain, diverse, or failure-relevant examples that uniform sampling rarely surfaces.

Example 9.4: Hard negative mining in a smart doorbell

Scenario: A smart doorbell vision detector receives continuous video feeds dominated by empty backgrounds (easy negatives) and normal pedestrian poses.

Diagnosis: Random frame sampling can miss rare false positives such as statues, laundry piles, and human-shaped shadows. Active learning routes uncertain predictions ("Person: 51%") and sampled production errors to human reviewers for targeted labeling.

Systems lesson: Hard-negative mining can improve labeling efficiency by spending review effort on suspected false positives and other difficult negatives. The query stream still needs diversity and coverage checks so confident but systematically wrong cases are not excluded.

Curriculum learning and active learning both choose among examples that already exist. When the pool itself lacks the cases a model needs (rare classes with too few examples, edge cases that never appear in the wild, records that privacy rules keep out of reach), no ordering or labeling policy can supply them, and the remaining lever is to create them.

9.5 Synthetic Data Generation

Real-world data collection encounters physical and legal boundaries: autonomous fleets cannot safely crash vehicles to acquire catastrophic collision trajectories, healthcare systems cannot transmit protected patient telemetry across institutional firewalls, and edge products must handle acoustic environments before any deployed fleet exists to record them. When the source corpus contains few or none of the cases deployment will face, neither static pruning nor dynamic selection can extract signal that is not there. Synthetic data generation, which section 4.4.3 introduced as one acquisition channel among several, becomes here a deliberate lever for constructing the training distribution directly. The strategy shifts from *curation*, choosing among existing records, to *creation*, making new inputs, perturbations, or supervisory targets.

9.5.1 Data augmentation

A vision model trained repeatedly on identical images can memorize their exact pixel layouts rather than the object class, yet storing multiple transformed copies of every image on disk would multiply the archive's storage footprint. **Data augmentation** avoids that storage cost by transforming existing samples on the fly during ingestion; every ResNet-50 epoch on ImageNet sees freshly cropped, flipped, and color-shifted variants of the same images. Label-preserving transformations add diversity without expanding durable storage or acquiring new data, but they move cost into the training pipeline. On host CPU worker threads, decoding and spatial transformations can saturate the cores and starve the accelerator across the PCIe bus. Offloading them to accelerator kernels relieves the CPU, yet inputs still cross the host-to-device link, and the transformations consume HBM and compute cycles that would otherwise perform gradient updates.

For image data, transformations encode invariances required at deployment (Shorten and Khoshgoftaar 2019). Geometric operations (rotation, flipping, cropping, scaling) encourage viewpoint invariance when they preserve the label, while photometric adjustments (brightness, contrast, color jitter) simulate sensor noise and varying illumination. Stronger methods mask or mix images: Cutout[17] applies random rectangular zero-masks to prevent reliance on localized features; MixUp (H. Zhang et al. 2018) constructs virtual samples by taking linear combinations of image pairs and their one-hot labels ($\tilde{x} = \lambda_{\text{mix}} x_i + (1 - \lambda_{\text{mix}}) x_j$, with mixing weight $\lambda_{\text{mix}} \in [0, 1]$); and CutMix[18] pastes rectangular patches from one image into another while blending labels proportionally to the patch area. Both mixing methods operate through linear combinations and block copies in device memory, so they add negligible FLOP overhead.

Text has no smooth geometric transformations, because tokens are discrete. Back-translation[19] paraphrases by passing target-language text through a translation model to produce synthetic source sentences. Simpler transforms such as synonym substitution, token insertion, or deletion risk corrupting labels unless validated for the task.

Automated policy searches replace hand-tuned transformation schedules. AutoAugment[20] uses reinforcement learning to search over operations, at a search cost of about 15,000 GPU-hours on

17 **Cutout**: Randomly masks square regions of input images during training, forcing the model to recognize objects from partial information rather than relying on any single discriminative region (DeVries and Taylor 2017). Unlike dropout (which zeroes neurons in feature space), Cutout operates in input space. The original Cutout experiments produced 0.3–2.0 percentage-point gains across CIFAR-10/100 and SVHN with negligible compute overhead, making it a high-ICR augmentation technique when occlusion-style invariance matches the task: more information per sample at near-zero additional cost to the pipeline.

18 **CutMix**: Replaces Cutout's zeroed-out region with a patch from a different training image, mixing labels proportionally to patch area (30 percent of image A replaced by image B yields a 70/30 label split) (Yun et al. 2019). This addresses Cutout's weakness: zeroed regions waste pixel information that could carry learning signal. The CutMix paper reports ImageNet top-1 improvements of 2.28 percentage points for ResNet-50 and 1.70 percentage points for ResNet-101, while also improving localization behavior. The method provides stronger regularization than either Cutout or MixUp alone with little additional data-pipeline cost.

19 **Back-translation**: In the method described by Sennrich et al. (2016), target-language monolingual text is translated into the source language, and the synthetic source is paired with the unchanged target for machine-translation training. The systems trade-off is latency because each synthetic source requires translation-model inference. Pipelines can precompute these examples offline rather than generating them in the data loader.

ImageNet. RandAugment[21] eliminates this search cost by parameterizing the policy with just two values: the number of transformations applied to each image and a single global distortion magnitude. Even the best augmentation schedule, however, can only perturb examples the dataset already contains.

9.5.2 Generative synthesis

Cropping or color-shifting existing images cannot invent a rare object class, an unseen weather condition, or a collision scenario that never occurred in the recorded dataset. When a pipeline needs examples that perturbations of existing records cannot supply, **generative synthesis** constructs new input-label pairs (x, y) by sampling from generative models or physical simulators. The three main engines differ in cost per sample and in how realistic their outputs are:

1. Generative Adversarial Networks (GANs) train a generator against a discriminator. Once trained, feed-forward generator inference requires a single forward pass, providing low latency and high sample throughput. However, GAN training is vulnerable to mode collapse, in which the generator covers only part of the data distribution and loses the tail variety needed for edge-case coverage.
2. Diffusion models generate samples by iteratively removing noise over many sequential steps. Latent diffusion systems such as Stable Diffusion[22] enable precise semantic conditioning from text prompts while operating in a compressed latent space. A sampling recipe with 20 to 50 denoising steps runs the backbone that many times per generated batch, so offline generation is often preferable.
3. Physical simulation engines (such as CARLA for autonomous driving or Isaac Sim and Unity for robotics) generate data without neural inference, by running physics and rendering pipelines. They produce exact ground-truth annotations (3D bounding boxes, depth maps, velocity vectors) from internal state at no labeling cost, and the bottleneck shifts to host CPU physics and graphics rendering throughput.

Regardless of the generative engine, synthetic samples inevitably diverge from the statistical distributions of real deployment environments. That divergence determines whether synthetic training transfers to production or fails on the first real-world input.

9.5.3 Bridging the domain gap

A model trained purely on synthetic data can reach high validation accuracy in the simulator while failing in production because it latched onto rendering or generator artifacts that do not exist in the real world. This **domain gap**[23] is the statistical shift between the synthetic training distribution and production deployment data. Synthetic and real feature densities can sit at different locations in feature space, misaligning the decision boundaries learned from each (figure 9.8). The marginal densities alone do not determine either decision boundary; class-conditional structure and real-data evaluation are needed to establish how the shift affects classification.

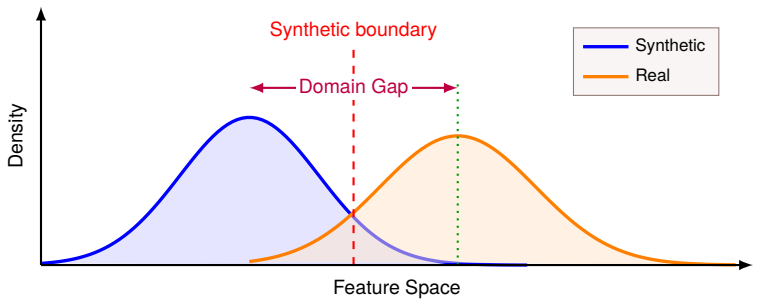


Figure 9.8: The Domain Gap Problem: Synthetic data (blue) and real data (orange) occupy shifted feature distributions. The dashed red synthetic boundary and dotted green real-data boundary illustrate possible misalignment; marginal feature densities alone do not determine class boundaries.

Two complementary strategies address this distribution mismatch. Domain randomization[24]

[20] **AutoAugment**: Treats augmentation policy design as a reinforcement learning search problem: a controller selects operations (rotate, translate, shear, equalize), their magnitudes, and application probabilities to maximize validation accuracy (Cubuk et al. 2019). The original ImageNet search used approximately 15,000 GPU-hours, motivating later methods that reduce the search space.

[21] **RandAugment**: Collapses AutoAugment's policy search to two hyperparameters, a transformation count and a shared magnitude (Cubuk et al. 2020). That small search space matches or exceeds AutoAugment on the reported benchmarks at far lower search cost, making it practical when 15,000 GPU-hours of policy search cannot be justified.

[22] **Stable Diffusion**: Performs iterative denoising in a compressed latent space, enabling targeted text-to-image examples (Rombach et al. 2022). Its iterative generation is substantially more compute-intensive than geometric transforms, so it fits cases where novelty matters more than per-sample throughput.

[23] **Domain gap**: The statistical divergence between synthetic and real data distributions, measurable with metrics such as maximum mean discrepancy (MMD) or Frechet Inception Distance (FID). For ML systems, the gap becomes training-serving skew: a simulator-trained model can validate well on synthetic data while failing silently on real deployment data.

[24] **Domain randomization**: Makes synthetic data deliberately varied by randomizing textures, colors, lighting, and physical properties. The goal is not photorealism; it is enough variation to improve coverage of deployment conditions, shifting the cost bottleneck from rendering fidelity to coverage.

takes an aggressive approach: rather than trying to match the real world precisely, it trains on varied synthetic data by randomizing lighting, textures, backgrounds, and camera parameters during generation. The aim is to make deployment conditions fall within the variation seen during training, though coverage is never guaranteed by randomization alone; the method is effective in robotics and autonomous-driving settings when the simulator spans the relevant physical and visual factors and the result is validated on real data.

Domain adaptation takes the opposite approach by explicitly aligning synthetic and real distributions. Feature alignment methods train on synthetic data while simultaneously minimizing the distance between synthetic and real feature distributions, often using adversarial training to learn domain-invariant representations. Fine-tuning offers a simpler path: pretrain on abundant synthetic data to learn general features, then fine-tune on a small real dataset to adapt to deployment conditions. Self-training combines these ideas by using a synthetic-trained model to pseudo-label real unlabeled data, then retraining on the combined labeled set. In practice, mixing synthetic and real data provides a real-data anchor while retaining some synthetic coverage: moving the mix from all-synthetic toward all-real trades a larger domain gap for a higher collection cost, and the best ratio depends on the domain, simulator fidelity, model family, and validation protocol rather than on any universal recipe.

Augmentation, noise injection, and simulation can grow a small seed set into a training corpus; the keyword spotter's data budget, including how much synthetic augmentation saves against recording real utterances, is worked through in section 4.3.1. The same creation strategy becomes riskier when samples come from learned generators. In recursive training, *model collapse*[25] can amplify errors and reduce diversity over generations. Synthetic data therefore requires provenance, mixture controls, and validation against real deployment data rather than an unverified assumption that more generated samples help. When the input examples themselves are already on disk, a trained model can instead enrich their labels.

[25] **Model collapse**: Formally analyzed by Shumailov et al. (2024), this phenomenon can arise when errors and missing tail coverage compound during recursive training. Tail distributions contain rare but important patterns that appear infrequently in training data. When generation $n+1$ trains on output from generation n under these conditions, successive generations can further compress the tails, producing increasingly homogeneous data. The degradation can be rapid in recursive training settings, which is why training on unanchored synthetic outputs risks catastrophic collapse of tail diversity.

9.5.4 Knowledge distillation

A hard one-hot label such as $[1, 0, 0]$ tells a classifier which class is right, but discards everything a stronger model knows about which wrong classes look similar. Where augmentation (section 9.5.1) and generative synthesis (section 9.5.2) create new inputs, **knowledge distillation**[26] (Hinton et al. 2015; Gou et al. 2021) creates richer targets for inputs the corpus already holds. A larger trained model (the teacher) labels each training example with its full probability distribution over classes, such as $[0.70, 0.20, 0.10]$ in place of $[1, 0, 0]$, and a compact student network trains against those soft targets, smoothed by a temperature parameter T_{distill}. Because each soft target records which wrong classes the teacher found plausible, every example carries more supervision than its one-hot label. Section 10.2.3.1 develops distillation as a compression technique, including its loss and temperature mechanics; as a data lever, its cost lies in generating, storing, and reading those targets.

Distillation trades accelerator HBM capacity against storage I/O bandwidth:

- **Online distillation**: The teacher model resides directly in accelerator memory alongside the student model during training. The system computes teacher forward passes concurrently with student execution, eliminating persistent storage overhead for soft targets. However, co-locating an overparameterized teacher consumes accelerator memory and arithmetic throughput, restricting the student's maximum per-accelerator batch size and increasing step latency.
- **Offline distillation**: The teacher generates predictions across the corpus during an offline precomputation pass, storing dense logit vectors to disk. This frees accelerator memory entirely for student training, but expands dataset storage volume and input pipeline bandwidth. For a classification vocabulary of K classes, storing uncompressed float32 logits increases the per-sample label footprint from a single integer index ($\lceil \log_2 K \rceil$ bits) to $4K$ bytes, an expansion of more than three orders of magnitude once K reaches the thousands.

[26] **Knowledge distillation**: Soft teacher probabilities reveal structural similarities across classes that one-hot encodings omit, and a temperature parameter controls how strongly those probabilities are smoothed. Distillation introduces teacher inference costs that must be justified by improved student sample efficiency or accuracy.

For ResNet-50 on ImageNet the offline footprint is concrete. Storing FP32 logits for 1,000 classes across every training image takes 5.1 GB, against 2.6 MB for the integer labels they replace, a 2,000× expansion that every epoch must read back through the input pipeline. That storage and read traffic, rather than the loss function, is what data selection must budget for. Generation and distillation both

supply labeled signal the corpus lacks; a complementary route reduces how many labels training needs in the first place, by extracting supervision from the unlabeled data the corpus already holds.

9.6 Learning from Unlabeled Data

Labels are often the scarcest input in a training pipeline. Active learning still pays an expert for every example it selects, and synthesis pays for every example it creates. Learning from unlabeled data attacks that cost from the other side. Semi-supervised learning lets a model trained on a small labeled seed supply provisional labels for the rest of the pool. Self-supervised learning removes task labels from pretraining altogether, and foundation models spread the resulting pretraining cost across many downstream tasks. Each approach trades additional training compute for fewer labels, so each is judged by total cost at matched quality rather than by label count alone.

9.6.1 Semi-supervised learning

In the chest X-ray adaptation of section 9.4.2, a hospital may hold 50,000 scans of which only 500 have been reviewed and labeled[27] by radiologists, a labeling rate of 1 percent. **Semi-supervised learning** puts the remaining 49,500 images to work by shifting the trade-off from human annotation to accelerator compute: it uses the small labeled subset to anchor training on the larger unlabeled pool, extracting supervisory signals from unannotated inputs. Models trained this way can approach fully supervised accuracy with far fewer manual labels, provided the unlabeled pool matches the target distribution and the result is validated on labeled data.

Unlabeled data, although it cannot teach the mapping from inputs to outputs, contains structural information about the input distribution $p(x)$ that can constrain the hypothesis space. Under smoothness and cluster assumptions, a decision boundary that cuts through a dense region assigns different labels to nearby inputs and is less plausible than one in a low-density region. Semi-supervised methods exploit these assumptions, which must be checked against the target task.

Three techniques implement this distribution-regularization insight. Pseudo-labeling[28] trains on the labeled subset, generates model predictions on the unlabeled pool, and incorporates samples exceeding a confidence threshold as synthetic ground truth for subsequent retraining. The confidence threshold governs a critical systems trade-off: setting it too low admits label noise that degrades learning, while setting it too high discards informative examples and wastes the unlabeled pool. Confident mistakes can survive either threshold, so confidence must be checked against labeled validation evidence rather than treated as proof of correctness.

Consistency regularization[29] enforces that the model produce invariant predictions across perturbed versions of the same input. When transformations such as cropping, rotation, or color shifts preserve the task label, the loss function penalizes prediction divergence between two stochastic views of an unlabeled sample. Hybrid approaches like FixMatch[30] unify both mechanisms: they generate a pseudo-label on a weakly augmented view when prediction confidence exceeds a threshold, then train the model to predict that label on a strongly augmented variant of the same input.

Label propagation frames semi-supervised learning as graph diffusion: it constructs an affinity graph over sample representations and propagates labels from labeled nodes to unlabeled neighbors. This approach works well when representations form well-separated clusters, though constructing and querying the similarity graph across large datasets introduces substantial memory and nearest-neighbor search overhead.

> Example 9.5: FixMatch on CIFAR-10
>
> **Scenario**: A semi-supervised image classification pipeline trains on CIFAR-10 with only 250 labeled samples (25 per class) alongside unlabeled data (Sohn et al. 2020).
>
> **Diagnosis**: The fully labeled CIFAR-10 training set contains 50K labels. FixMatch generates high-confidence pseudo-labels (>0.95 threshold) on weakly augmented unlabeled images and enforces consistency on strongly augmented variants, reaching 94.9 percent accuracy in the cited experiment.
>
> **Systems lesson**: Semi-supervised learning trades additional training compute for fewer manual labels. In an illustrative cost scenario at $1 per label, a supervised baseline with 4,000 labels

27 **Illustrative clinical labeling economics**: The review-rate and hourly-rate ranges are scenario assumptions rather than reported measurements. Under those inputs, labeling 500 scans requires 7–10 hours and costs $1,000–3,000; labeling all 50,000 costs $94,000–300,000. The arithmetic illustrates why semi-supervised learning can be attractive when expert labels are expensive, but actual review time, adjudication, prevalence, and validation requirements must be measured.

28 **Pseudo-labeling**: Uses a trained model's own confident predictions as ground-truth labels for unlabeled data; the technique's effectiveness depends on a virtuous cycle in which accurate predictions on easy unlabeled examples expand the training set and improve the model, which then labels harder examples accurately. The failure mode is equally self-reinforcing, because incorrect pseudo-labels feed the model's own misclassifications back into training through *confirmation bias*.

29 **Consistency regularization**: The method rests on a task-dependent smoothness assumption, that two inputs x_1 and x_2 related by a label-preserving perturbation should receive the same prediction. It is conceptually distinct from data augmentation (which creates more training examples) because it explicitly enforces prediction consistency as a loss term, even for unlabeled data where the "correct" label is unknown. The systems consequence is additional computation for generating and comparing predictions under multiple perturbations, a trade-off that can favor compute-rich, label-poor settings.

costs $4,000 in labels plus $50 of compute, or $4,050. FixMatch at 250 labels costs $250 in labels but 5× the compute, $250, for $500. Preparation expense falls by 8.1×; the end-to-end saving depends on the real unlabeled-data and compute costs.

The objective of semi-supervised learning is lower total labeling and compute cost at matched quality, which can justify a lower ICR when annotation savings exceed the added training expense. Neither the label reduction nor the compute increase carries over unchanged from one task to another; table 9.4 shows how far the label count fell on one benchmark.

[30] **FixMatch**: The confidence gate is strict; in the original experiments a pseudo-label is kept only when the prediction on the weak view exceeds 0.95 (Sohn et al. 2020). Every unlabeled image still costs a forward pass on its weak view whether or not it passes the gate, part of the extra training compute that buys the reduction in manual labels.

Table 9.4: FixMatch Label Efficiency on CIFAR-10: With 250 labels (0.5 percent of the dataset), FixMatch reaches 94.9 percent accuracy while using 200× fewer labels than the full training set contains.

Label Budget	Method	Accuracy	Label Efficiency
50,000 (100%)	Full training set	—	Label-count reference
4,000 (8%)	FixMatch	95.7%	12.5× more efficient
250 (0.5%)	FixMatch	94.9%	200× more efficient
40 (0.08%)	FixMatch	88.6%	1250× more efficient

The efficiency column counts labels only, relative to the full labeled set. It does not claim that FixMatch matches a fully supervised model, which would have to be compared at equal accuracy. These efficiency gains also depend on benign distribution assumptions. Semi-supervised learning assumes that unlabeled data comes from the same distribution as labeled data, and it struggles when unlabeled data contains out-of-distribution samples (the model confidently mislabels them), when class imbalance is severe (pseudo-labels amplify majority class bias), or when the labeled set does not cover all classes (preventing label propagation for unseen classes). A held-out set with true labels is the check that catches such a mismatch.

Each technique can reduce dependence on task-specific labels, but none eliminates the need for labeled evaluation. The progression raises a related possibility, that task-specific labels may not be necessary for pretraining. The structure of data itself (cat images resemble other cat images, and coherent sentences follow grammatical patterns) may provide a supervision signal, while downstream quality still requires task-aligned validation.

9.6.2 Self-supervised learning

GPT was trained to predict the next token in a sequence. BERT was trained to fill in masked tokens. Neither pretraining objective required task-specific human labels. **Self-supervised learning**[31] (SSL) generalizes this insight: by designing *pretext tasks* that derive supervision from the data's structure, models can learn reusable representations from unlabeled data at scale. Where active and semi-supervised learning reduce the demand for labels, SSL changes the accounting by making unlabeled structure the pretraining signal. Within the intervention map of figure 9.1, SSL sits beside the three levers and extends the usable pool; the question shifts from which labeled samples to train on to how unlabeled samples can support pretraining. It responds to one aspect of the data wall introduced in section 9.1 by broadening what counts as a training signal, while leaving corpus quality, licensing, coverage, and labeled evaluation requirements intact.

Labels are one form of supervision; a pretext task can also derive a learning signal from the structure of the data, as table 9.5 summarizes.

[31] **Self-supervised learning**: The pretext task, such as next-token or masked-token prediction, provides a supervisory signal derived from the data itself. This reduces dependence on task-specific human labels but can require substantial pretraining compute. Amortization depends on how broadly the resulting model is reused.

Table 9.5: Self-Supervised Pretext Tasks by Modality: Each task extracts supervision from data structure rather than human labels, enabling pretraining on large-scale unlabeled corpora.

Modality	Self-Supervised Task	Supervision Signal
Text	Masked language modeling	Predict [MASK] from context
Text	Next-token prediction	Predict next token in sequence

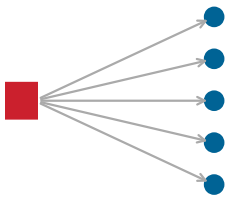
Modality	Self-Supervised Task	Supervision Signal
Images	Contrastive learning	Same image (augmented) vs. different images
Images	Masked autoencoding	Reconstruct masked patches
Multi-modal	CLIP-style alignment	Match image-text pairs

Pretext tasks generate supervision signals automatically. Masked-token objectives can encourage models to learn grammatical and semantic structure; next-token prediction teaches continuation structure; and contrastive image objectives encourage visual features that are invariant to selected transformations. These approaches build on convolutional neural network and transformer families, but the systems accounting matters: self-supervision removes task-specific annotation from the pretraining gate rather than removing data cost. Pretraining can begin on unlabeled corpora without waiting for example-level labels, but collection, filtering, licensing, storage, and processing remain. If the representation transfers, one pretraining run serves many tasks and each task needs far fewer labels; task-specific adaptation and evaluation still carry their own data requirements.

When its pretraining cost is reused across tasks, SSL supports the **foundation model paradigm**[32] (Bommasani et al. 2021). The core data selection principles (coreset selection, curriculum learning, and active learning) remain essential within this paradigm. Pretraining corpus curation applies deduplication and quality filtering at web scale, while downstream selection dictates which fine-tuning examples maximize task-specific transfer.

Pretext objectives differ in what they stress on the machine. Contrastive learning trains a model to map two augmented views of the same image to nearby representations while pushing the representations of different images apart. Those other images are the *negatives*, and the objective learns better with more of them. SimCLR (T. Chen et al. 2020) draws its negatives from the rest of the mini-batch, with batches of up to 8,192 examples, so the negative count is paid for in activation memory and, across accelerators, in all-gather traffic that collects every device's representations. Momentum Contrast (MoCo) (He et al. 2020) instead keeps recent representations in a queue produced by a slowly updated copy of the encoder (the momentum encoder), which decouples the number of negatives from batch size. Masked autoencoders split an image into small square patches, hide most of them, and train the model to reconstruct what is hidden; because the encoder sees only the visible patches, it spends far less memory and compute per image. Autoregressive pretraining, by contrast, scales with tokens. For data selection, the common point is that pretraining turns raw structure into reusable signal that multiplies the value of scarce labels; because the pretraining run itself is expensive, however, whether self-supervised pretraining pays off depends on how many downstream tasks reuse it.

[32] **Foundation model**: The name emphasizes that these models serve as a shared base for many downstream tasks, but this creates a single point of failure. Defects in the foundation model's pretraining data (biases, factual errors, memorized private content) can propagate to multiple applications built upon it, depending on transfer and downstream adaptation. From a systems perspective, this homogenization risk means that data selection quality during pretraining has an outsized blast radius: a curation error that would affect one task in the train-from-scratch paradigm can affect many downstream deployments.



One pretraining corpus defect can affect many downstream tasks.

9.6.3 Foundation-model amortization

Pretraining a self-supervised model on a massive unlabeled corpus can consume thousands of accelerator hours, an upfront bill that rarely pays off if the resulting checkpoint serves only a single downstream classifier. **Cost amortization** makes that investment economical by splitting work into two phases: an expensive pretraining pass run once on unlabeled data, and lightweight task-specific adaptations, usually fine-tuning, reused across many applications (table 9.6).

Table 9.6: Illustrative Foundation-Model Amortization: The label and compute ranges are scenario assumptions. They illustrate why reuse can lower marginal task cost when representations transfer, not a universal fine-tuning ratio.

Approach	Labels per Task	Compute per Task	Data Acquisition
Train from scratch	100K–1M labeled	100% full training	Task-specific collection
Fine-tune foundation model	100–1K labeled	1–5% of full training	Reuse pretraining corpus

Take a company building 10 specialized classifiers. Under the scenario assumptions, each task trained from scratch needs 100,000 labels at $1 per label, for $1M across all tasks and a modeled compute burden of 10,000 GPU-hours. The fine-tuning scenario instead pays 10,000 GPU-hours

↩ Fine-tuning's memory and compute cost on one accelerator, set by which parameters it updates, is established in section 8.3.3.3.

once, then assigns 1,000 labels and 50 GPU-hours of compute to each task, bringing the modeled fine-tuning label bill across all 10 tasks to $10K. Assuming both approaches reach the same target quality on every downstream task, labeling cost drops by 100× and per-task marginal compute by 20× as direct consequences of the chosen scenario values, while total compute becomes favorable only after enough downstream tasks amortize pretraining.

The compute-only crossover occurs when enough downstream tasks share the pretraining cost, provided the representations transfer at the required quality. A dollar crossover also depends on labeling savings, compute prices, and other costs that differ between the approaches. The side-by-side bar charts in figure 9.9 contrast the cost structures across ten downstream tasks: training from scratch expends a constant 1,000 GPU-hours per task (totaling 10,000 GPU-hours), whereas the foundation model approach absorbs a 10,000 GPU-hour pretraining bar upfront but collapses subsequent task fine-tuning to just 50 GPU-hours per bar.

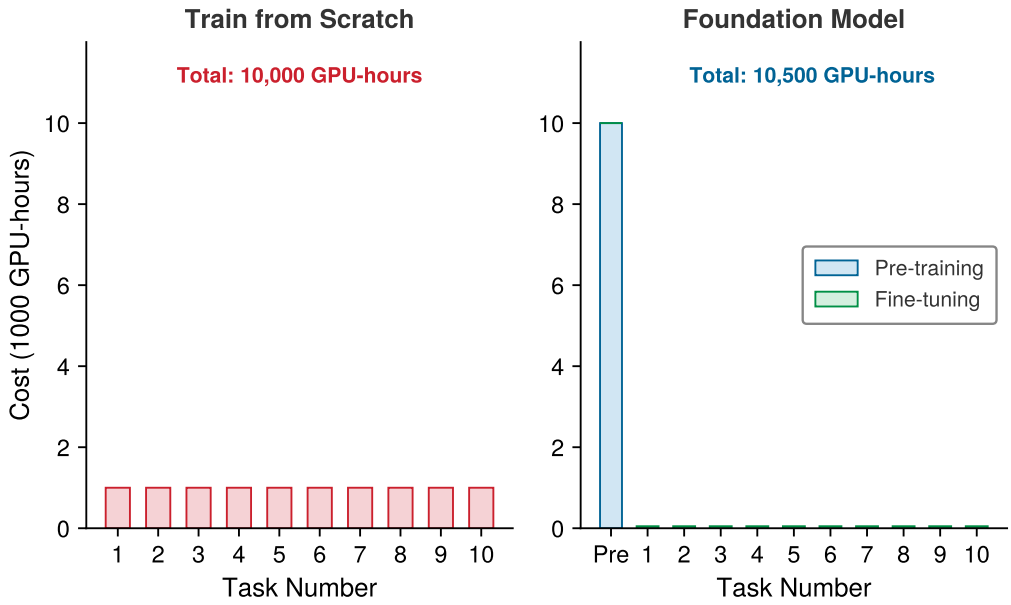


Figure 9.9: Cost Amortization in Foundation Models: In this illustrative scenario, training from scratch (left) requires 1,000 GPU-hours per task (10,000 GPU-hours total for 10 tasks). The foundation model approach (right) pays 10,000 GPU-hours upfront for pretraining but reduces each subsequent task to just 50 GPU-hours. At 10 tasks the totals are comparable (10,000 GPU-hours vs. 10,500 GPU-hours), but the per-task marginal cost drops by 20×, and the compute-only crossover favoring the foundation model occurs around 10.5 tasks. The plotted 10-task snapshot therefore remains just below that break-even point.

Every lever is now in view: pruning, reordering, labeling, creating, and learning without labels. Each has been judged so far by the work it removes; the next question is whether the machine can run it without giving that saving back.

9.7 Selection Engineering

A naive active learning loop that scans the entire dataset every epoch to select the "best" samples can turn a compute-bound training job into an I/O-bound bottleneck. **Selection engineering** is the work of making a chosen technique deliver its promised speedup on real hardware and real data pipelines. The pipeline must keep scoring cheap, reads sequential, and the accelerator fed.

The machine sets the scale for every one of those patterns. One ImageNet epoch of ResNet-50 on one H100 passes through four stages: images are read from local storage, decoded and augmented on host CPUs, copied across PCIe into accelerator memory, and consumed by the accelerator's arithmetic. Pruning shrinks the last stage directly; whether the other stages shrink, stay the same, or grow depends on how the selection is engineered (figure 9.10).

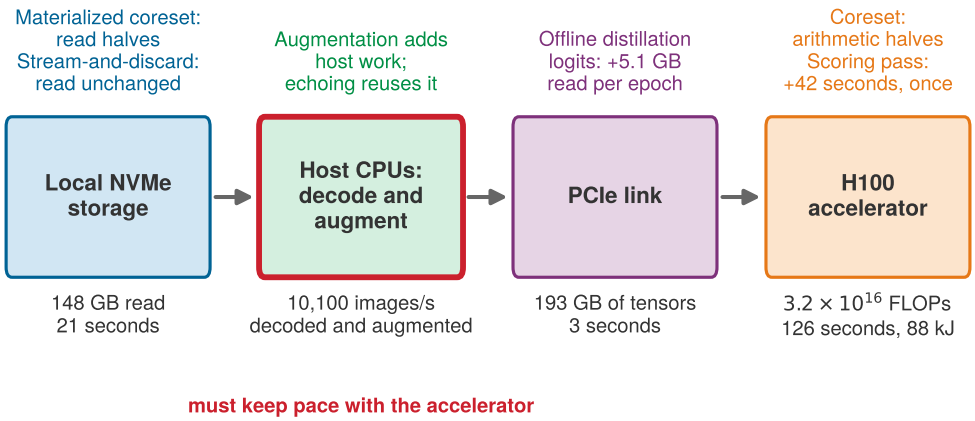


Figure 9.10: ResNet-50 Data Path on One H100: One ImageNet epoch crosses four stages: local storage, host decoding and augmentation, the PCIe link, and the accelerator. Loads under each stage are per full epoch at the compute-bound rate; notes above each stage name the lever that acts there. A materialized coreset halves the storage read and the arithmetic, stream-and-discard filtering halves only the arithmetic, offline distillation targets add bytes to every stage before the accelerator, and a scoring pass adds one third of an epoch of forward arithmetic once. The host stage, outlined in red, must decode and augment every image at the accelerator's rate.

Napkin Math 9.4: One pruned ResNet-50 epoch on the H100

Problem: ResNet-50 trains on the 1.28 million ImageNet training images for 90 epochs on one H100, and a coreset keeps 50 percent of the images. How much accelerator time and energy does the coreset save once its scoring pass is paid?

Compute: Each image costs 24.6 GFLOP forward and backward, so a full epoch is 3.2×10^{16} FLOPs. At the 25 percent of peak that ResNet-50 sustains in training (section 8.4.2), the H100 finishes it in about 126 seconds, which at its 700 W board power is about 88 kJ. The pruned epoch takes about 63 seconds and 44 kJ.

Selection: An EL2N-style scoring pass is one forward pass over the full pool, a third of an epoch's arithmetic, about 42 seconds and 29 kJ, paid once. Over 90 epochs the coreset saves about 4 MJ and 2 hours of accelerator time, 135× the scoring energy.

Data path: To stay compute-bound, the accelerator must receive about 10,100 images per second. Reading the 148 GB ImageNet archive from local NVMe at 7 GB/s takes about 21 seconds per epoch, and copying the decoded 8-bit tensors (193 GB) across PCIe at 64 GB/s takes about 3 seconds, both inside even the pruned epoch. The stage that must keep up is the host CPU, which has to decode and augment 10,100 JPEG images every second. The training chapter's host supplies only about 5,250 per accelerator, so an epoch actually takes about 244 seconds until decode capacity grows; the seconds below are the compute-bound figures selection can claim once the host keeps up.

Systems insight: Pruning halves the Data factor (Samples/Task), cutting total arithmetic O and accelerator energy in half, but it halves the storage read (D_{vol}/BW) only if the retained subset is written out as its own shards; streaming the full archive and discarding half leaves storage traffic unchanged. The seconds come from FLOPs, and whether they are realized

depends on the slowest stage of the data path, which for ResNet-50 is often host decoding (section 8.4.1).

Each lever also adds memory or storage of its own and compute or I/O of its own, and those costs land on different stages of the same data path (table 9.7).

Table 9.7: Memory and Compute Cost of Each Lever: Values for the chapter's two lighthouses on the reference H100 machine. Each lever adds memory or I/O at one stage of the data path while removing work at another, so the stage it loads must not become the new bottleneck.

Lever (lighthouse)	Memory or storage it adds	Compute or I/O it adds	Work it removes
MinHash deduplication (GPT-2, WebText)	4.1 GB of signatures in host memory	One 40 GB read (6 seconds) plus host hashing	3.04% to 13.63% of tokens processed
EL2N coreset scoring (ResNet-50, ImageNet)	One score per image (5.1 MB)	One forward pass over the pool (42 seconds, 29 kJ)	Half of every epoch (63 seconds, 44 kJ)
Coreset materialization	A 74 GB repacked subset, written once	One read and one partial write of the archive	Half of every later epoch's storage read
Stream-and-discard filtering	None	The full 148 GB read every epoch (21 seconds)	Accelerator work only; storage traffic unchanged
Offline distillation targets (ResNet-50)	5.1 GB of FP32 logits in place of 2.6 MB of labels	One teacher forward pass over the pool, then a 5.1 GB read every epoch	Teacher residency in accelerator memory during training

Those added costs surface in four places (scoring, storage reads, host preparation, and coordination across workers), and each has an engineering pattern that keeps it off the critical path.

9.7.1 The selection bottleneck

Evaluating a selection function $f(x)$ over a large candidate pool before forming every batch can easily cost more compute than training on the unselected data. While a conventional loader follows a predetermined sampling plan, active learning and adaptive curricula score candidates dynamically during training, introducing **selection latency**. In the ResNet-50 example of section 9.2.6, one static scoring pass with the target model costs a third of an epoch and is paid once. Rescoring the full ImageNet pool before every one of the 100 epochs instead adds 70 minutes of forward passes to 21 minutes of subset training, more than three times the training it serves, and cuts the saving from 90 percent to 57 percent. The selection inequality (equation 9.2) still holds, but scoring has become the largest term, and a heavier scorer or a smaller retained fraction would break it.

Engineering pipelines keep selection latency off the critical path through proxy models, cached embeddings, and indexed retrieval. A smaller proxy model curtails scoring time provided its sample ranking transfers accurately to the target architecture, and running the scoring pass in reduced numerical precision raises arithmetic throughput and cuts memory traffic when ranking fidelity permits. For similarity-based or diversity-based selection, vector indices such as FAISS[33] replace expensive corpus-wide scans with approximate nearest-neighbor retrieval over precomputed embeddings. These techniques decouple selection evaluation from primary model training and shrink $T_{\text{selection}}$, but even zero-cost scoring can still slow down training if fetching the selected indices stalls the storage layer.

9.7.2 The random-access penalty

Selecting a sparse subset of examples scattered across a dataset turns large sequential shard transfers into thousands of nonsequential reads, forfeiting hardware readahead, OS page caching, and bulk controller prefetching. Training already pays for shuffling through the access factor F_{access} that scales effective storage throughput in the data-pipeline model of section 8.4.2.1, and selection pushes that factor lower by design because the retained records no longer sit contiguously on disk. Because disk heads must physically seek and flash controllers must execute random block lookups, two subsets

33 **FAISS (Facebook AI Similarity Search)**: Provides GPU-accelerated similarity search using exact, approximate, and compressed-domain index designs (Johnson et al. 2019); Johnson, Douze, and Jegou report billion-vector graph construction on multiple GPUs, showing why vector-index infrastructure matters for web-scale selection. For data selection pipelines, FAISS-style indexing supports k-nearest-neighbor retrieval for coreset selection, embedding-based deduplication, and stratified clustering for balanced sampling. Without this infrastructure, embedding-based selection would often fall back to expensive full-corpus scans.

↩ Random access is measured in IOPS and streaming access in sequential throughput, the two storage metrics established in section 4.10.1.

of equal size can differ sharply in wall-clock training time according to their access locality. Table 9.8 quantifies this small-read penalty across storage tiers for 4 KB transfers.

Table 9.8: Illustrative Small-Read Penalty: Registry device anchors compare sequential bandwidth with random 4 KB reads under a simplified IOPS conversion. Real throughput depends on request size, queue depth, concurrency, caching, and service configuration; the cloud row is only a qualitative scenario.

Storage Tier	Sequential Throughput	Random I/O (IOPS)	Random Throughput (approx)	Random Penalty
HDD (7.2k)	~150 MB/s	~100 IOPS	~0.4 MB/s	375×
SATA SSD	~550 MB/s	~10K IOPS	~40 MB/s	13.8×
NVMe SSD	~3,500 MB/s	~500K IOPS	~2,000 MB/s	1.75×
Cloud (S3)	Configuration-dependent	Not IOPS-comparable	Configuration-dependent	Potentially extreme

Data loaders (whose general mechanism section 7.5.3 develops) avoid the problem by packing examples into large shards and shuffling inside a memory buffer. Sharded dataset formats, such as WebDataset and FFCV, pack thousands of individual examples into contiguous, sequentially readable archive chunks. Rather than fetching scattered individual samples from disk, the loader streams large sequential shards into host memory and populates a **shuffle buffer**. The training loop samples pseudo-randomly from this in-memory buffer, preserving peak storage read bandwidth while satisfying the statistical requirement of randomized mini-batches.

Converting a logical set of selected indices into sequential physical reads requires choosing between two storage strategies. For static pruning (such as a fixed coreset or a deduplicated corpus), the pipeline can materialize the subset offline by repacking the retained records into new, contiguous shards. This step incurs a one-time sequential read and write cost alongside additional disk storage, but ensures that every subsequent training epoch transfers only retained bytes from storage. Conversely, when the selection policy updates dynamically across training iterations or active-learning rounds, repeatedly repacking multi-terabyte shards on disk would choke the filesystem on write bandwidth. Data loaders instead stream the original, full-volume shards sequentially into host memory and discard unselected records before populating the shuffle buffer. This online filtering preserves sequential read throughput and eliminates seek penalties, but it leaves physical storage traffic unchanged: the host continues to read 100 percent of the corpus bytes from disk even when accelerators train on only a 10 percent subset. Systems engineers must count this ongoing read traffic on storage-bound nodes and include offline materialization costs when evaluating the selection inequality, and even after storage reads are streaming at full speed, host-side decoding and augmentation can still starve the accelerator.

Checkpoint 9.2: Selection overhead and storage access

Equation checks

- ☐ **Selection cost**: Explain why $T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}})$ must be strictly less than $T_{\text{train}}(D_{\text{total}})$ for the technique to reduce end-to-end training time.
- ☐ **Overhead management**: Explain how proxy models and cached embeddings keep $T_{\text{selection}}$ low.

Systems implications

- ☐ **I/O patterns**: Explain why nonsequential access for dynamic selection reduces dataloader throughput relative to sequential reads.

9.7.3 Data echoing

Even with sequential reads, the host CPU often cannot decode and augment samples fast enough. Heavy data transformations, such as 3D rotations, MixUp, JPEG decoding, or on-the-fly generative synthesis, demand substantial host processing per sample. When the host pipeline produces batches

slower than the accelerator executes forward and backward passes, the accelerator idles, starved for input; this is the feeding problem reappearing inside the selection loop. The resulting collapse in accelerator utilization extends training wall-clock time, neutralizing the efficiency gains achieved by pruning the dataset.

↩ Accelerator bubbles, which open when the data-loading stage cannot keep pace with the accelerator, are established in section 8.4.1.

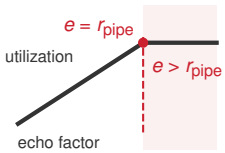
Data echoing[34] (Choi et al. 2019) reuses data multiple times before fetching new samples, trading freshness for accelerator utilization when the input pipeline is slower than training. Echoing before randomized augmentation can produce different transformed inputs on each repetition, while echoing later in the pipeline may repeat identical tensors.

[34] **Data echoing**: The key subtlety is *where* in the pipeline to insert the echo point. Echoing before augmentation (upstream echoing) can apply different random augmentations to each repetition, while echoing after augmentation feeds identical tensors to the accelerator. Useful echo factors and realized speedups depend on the workload, batch size, insertion point, and shuffling policy (Choi et al. 2019).

The optimal echo factor depends on the ratio r_{pipe} of upstream processing time to downstream training time, where T_{prep} is the time the data pipeline needs to produce a batch and T_{GPU} is the time the accelerator needs to train on it:

$$r_{\text{pipe}} = \frac{T_{\text{prep}}}{T_{\text{GPU}}}$$

In this two-stage rate model, $r_{\text{pipe}} > 1$ means the data pipeline bottlenecks training. An echo factor $e < r_{\text{pipe}}$ partially recovers idle accelerator cycles, while $e \geq r_{\text{pipe}}$ removes input waiting entirely, provided repeated samples remain statistically productive. Raising e beyond r_{pipe} adds no throughput and only reduces gradient diversity. When $r_{\text{pipe}} < 1$, the input pipeline already outruns the accelerator, so echoing has nothing to recover.



Echoing raises accelerator utilization until e reaches r_{pipe}; beyond it, nothing is gained.

Napkin Math 9.5: Worked example: Data echoing ROI

Scenario: Training ResNet-50 on ImageNet on one H100 fed by the training chapter's host, where host preparation is the slow stage (section 8.4.2).

Rates:

- Data pipeline throughput: 5,250 images/s (the host cores available to one accelerator, each reading, decoding, and augmenting about 375 images/s)
- Accelerator training throughput: 10,100 images/s (forward and backward pass at the compute-bound rate of notebook 9.4)
- Ratio $r_{\text{pipe}} = T_{\text{prep}}/T_{\text{GPU}}$ = (1/5,250 images/s) / (1/10,100 images/s) = 10,100 images/s/5,250 images/s ≈ 1.92 (accelerator waiting 48.0 percent of the time)

Without echoing:

- Effective throughput: 5,250 images/s (limited by data pipeline)
- Training time for 90 epochs: $90 \times 1.28\text{M}$ / 5,250 images/s = 21,963 seconds (6.1 hours)
- Accelerator utilization: 52.0 percent

With echo factor: $e = 2$, inserted after the measured reading, decoding, and augmentation stage.

- Each already-augmented batch is processed twice, amortizing the measured upstream work; the repeated pass introduces no fresh augmentation
- Effective throughput: 10,100 images/s, the accelerator's full rate, because an echo factor above the ratio leaves the accelerator no time waiting
- Unique images per second: 5,250 images/s (unchanged)
- Training time: $90 \times 1.28\text{M}$ / 10,100 images/s = 11,416 seconds (3.2 hours) if echoed data is equally valuable

Trade-off: Repeated samples are not guaranteed to be as useful as fresh samples; the data echoing paper evaluates echo factor, insertion point, and shuffling because those choices determine whether reuse preserves predictive performance. In one network-fed ResNet-50/ImageNet configuration, Choi et al. (2019) reports a 3.25× reduction in wall-clock time to the target metric.

Systems insight: Data echoing can trade sample diversity for accelerator utilization when the input pipeline is the bottleneck. For the training chapter's input-bound ResNet-50, overlap alone left the accelerator about half idle; an echo factor of 2 closes that gap without adding host cores. The useful echo factor must be measured for the workload because it depends on the batch size, insertion point, augmentation, shuffling, and target metric.

Echoing can introduce correlations when the same example appears within or across nearby batches, so implementations must preserve adequate shuffling and validate the target metric. Choi et al. (2019) did not observe a negative batch-normalization interaction in their ResNet-50 or Single Shot MultiBox Detector (SSD) experiments, but that result does not guarantee the same behavior for every workload.

These implementation patterns determine whether data selection yields net systems throughput. Proxy selection curtails probe compute when scoring rankings transfer to target architectures. Sharded file formats and shuffle buffers reconcile pseudo-random batch creation with hardware-level sequential streaming, while data echoing recovers stalled accelerator cycles when ingestion throughput limits the training loop. They still leave a coordination question when several accelerator workers share one host: each worker scores or trains on its own shard, yet the selected indices and scoring checkpoint must still describe one coherent dataset, because shared storage and host memory do not automatically make each worker's local view representative of the corpus.

9.7.4 Selection across multiple accelerators

When parallel worker processes divide a dataset into local shards across two to eight accelerators on one host, each worker sees only a fraction of the corpus and may score its shard with a slightly different model checkpoint. Without cross-worker coordination, shard-local coreset rankings miss rare global classes, independent hash tables fail to catch duplicates split across workers, and asynchronous active-learning workers select samples using stale uncertainty scores. While a single-process pipeline sorts one global ranking, follows one curriculum schedule, and queries one shared candidate pool, a multi-accelerator pipeline must reconstruct a representative global subset from local shard views while all workers contend for the same host memory capacity, storage bandwidth, and host-to-device links.

Whether distributed selection stays faithful to the global dataset or collapses into local shard heuristics depends on how much cross-worker coordination each technique requires against the bandwidth available to sustain it. On one host, score gathering and index dispatch compete for CPU time and memory bandwidth with decoding and batch preparation. Evaluating selection therefore requires analyzing statistical fidelity alongside these shared resources.

These are independent requirements. A coreset can remain representative yet cost too much to coordinate, while a cheap shard-local method can scale but systematically miss rare groups visible only in the global corpus. A distributed design therefore needs two acceptance tests: compare selected-data quality with a centralized or otherwise auditable reference, and compare selection overhead with the end-to-end training time it saves. Passing only one test is insufficient.

9.7.4.1 Strategies for distributed selection

In a basic data-parallel layout, each worker processes a distinct shard and model synchronization is handled separately. Data selection can add dependencies across those shards (table 9.9).

Table 9.9: Selection Dependencies in Distributed Training: The table starts from a common centralized formulation of each technique and identifies the coordination problem introduced by sharding. Distributed variants may relax rather than reproduce the centralized result exactly.

Technique	Single-Process Assumption	Distributed Challenge
Coreset Selection	Global view of dataset	Each worker sees only its shard
Active Learning	Centralized uncertainty scoring	Scoring requires model synchronization
Curriculum Learning	Global difficulty ordering	Workers may have different "hardest" samples

Technique	Single-Process Assumption	Distributed Challenge
Deduplication	One shared duplicate map	Shard-local hashes miss cross-shard duplicates

The selection dependencies admit two broad architectures, each at a different point in the trade-off between consistency and scalability. Centralized selection keeps selection global while training stays distributed. A coordinator process scores the full dataset and dispatches the selected indices to the workers, which preserves selection quality but makes the coordinator a single bottleneck that can serialize scoring or index dispatch. Whether that overhead is acceptable depends on the selection payload, refresh rate, worker count, and copies through shared host memory, and adding accelerator workers does not increase the host memory available to the coordinator. Hierarchical selection distributes the selection computation itself. Each worker selects candidates from its own shard, and the coordinator merges and reranks them before dispatching the final indices (figure 9.11). Shard-local selection reduces coordinator load but introduces a quality trade-off, because local quotas and score distributions may not preserve the global ranking or rare groups spread unevenly across shards. The merge step therefore needs normalization, coverage constraints, or a second global pass.

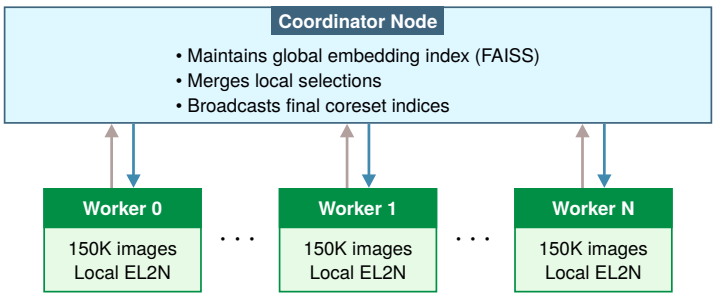


Figure 9.11: Distributed Coreset Selection Architecture: Parallel shard-local scoring reduces the work assigned to a centralized rank-merging coordinator. Workers send local EL2N scores upward; the coordinator merges them and broadcasts final coreset indices. All boxes are processes on one host; the coordinator is a selection role, and each worker drives one accelerator. The design is beneficial only when this coordination overhead remains below the training time saved by the coreset.

Shard locality affects training as well as selection. Each worker maintains an independent shuffle buffer over a non-overlapping shard, and because randomization is local to each worker's memory buffer rather than global across the corpus, rare classes or boundary cases concentrated in specific shards receive uneven exposure across workers unless the dataset is stratified and shards are balanced prior to distribution.

Cross-shard deduplication needs the same global view as hierarchical selection. Workers can compute MinHash signatures locally, but a shared merge must compare candidates across shard boundaries. Independent local hash tables cannot detect a duplicate whose counterpart belongs to another worker. The merge must fit the host's memory budget or operate in bounded batches, rather than assuming that more workers supply more host capacity.

9.7.4.2 Consistency challenges in active learning

Shard-local selection already needs a coherent score scale, but active learning introduces an additional complication because the model changes during selection. Consider what happens when Worker A scores samples using the model at step t while Worker B simultaneously updates the model to step $t+1$. Worker A's scores are now stale and may select samples that the updated model would rank differently.

The scoring schedule therefore defines the meaning of a query, not merely its cost. A reproducible system versions the model checkpoint, candidate-pool snapshot, scoring rule, and selected indices together. It can then measure how much ranking agreement decays with checkpoint age and choose a refresh interval from that evidence rather than from an arbitrary number of steps.

Several strategies mitigate this staleness problem, each with distinct overhead characteristics:

- **Synchronous scoring**: All workers pause training and score simultaneously, guaranteeing consistency but at substantial cost in accelerator utilization.
- **Periodic score refresh**: Workers re-score every k_{refresh} epochs rather than every batch, trading freshness for reduced overhead.
- **Checkpoint-robust selection**: The system selects samples that show high uncertainty under multiple model checkpoints, favoring decisions that are less sensitive to checkpoint choice. Agreement across past checkpoints is a robustness heuristic; ranking quality still needs validation as the model evolves.

Positive ROI can erode quickly when workers coordinate frequently during training. Selection across workers adds coordination overhead whose size depends on what is synchronized and how often. That overhead must remain smaller than the training time saved, and when it approaches that limit, a simpler strategy or a longer refresh interval restores the margin.

The communication path adds a further constraint. Gradient synchronization may use accelerator collectives over local interconnects, while embedding vectors, score arrays, and coreset indices may pass through host memory and CPU dispatch. Selection can therefore expose host-memory, PCIe, storage, or process-synchronization bottlenecks that the training profile alone does not reveal. Extending the design to independent machines adds network topology and remote-storage constraints; those are a further scaling problem beyond the one-host design developed here.

Operational feasibility, however, does not guarantee economic viability. A deduplication pipeline that costs more to build than one training run saves loses money on a single run and pays off only if enough runs reuse it. Deploying selection techniques therefore requires financial accounting of capital, compute, and human annotation budgets.

9.8 Choosing a Selection Strategy

The hospital team in notebook 9.3 could not label every scan, because doing so would have cost $5,000,000, 10× over its budget. Allocating capital between human annotation and accelerator compute is a primary systems decision, and a team under a fixed budget must determine whether the next dollar buys more downstream accuracy as specialist annotations or as additional accelerator hours. A cost model settles the question, and the choice among levers follows from it in four steps: the total cost of data, the break-even point of a selection pipeline, the amortization of one-time investments across the runs that share them, and a decision process worked on the ResNet-50 vision team of section 9.2.6.

9.8.1 Data costs and return on investment

Budgeting a training project from cloud GPU and object-storage invoices alone misses the licensing and human annotation bills that often dwarf compute cost. The **total cost of training data** spans acquisition, labeling, storage retention, and accelerator compute across the full data lifecycle:

$$C_{\text{total}} = C_{\text{acquire}} + C_{\text{label}} + C_{\text{store}} + C_{\text{process}}$$

Labeling (C_{label}) represents the expenditure that data selection targets most directly, whereas storage (C_{store}) and processing (C_{process}) scale mechanically with retained byte volume and the number of accelerator passes. Table 9.10 itemizes these four cost components alongside their operational drivers.

Table 9.10: Total Cost of Training Data: The four cost components span the full data lifecycle. Here D is the total sample count, D_{labeled} is the labeled subset, $D_{\text{vol,store}}$ is stored byte volume, T_{months} is retention time, N_{epochs} is the number of training passes, and O_{sample} is per-sample training work. The ranges are scenario anchors rather than universal market prices.

Component	Formula	Illustrative Range
$C_{\textbf{acquire}}$	$D \times c_{\text{sample}}$	\$0.001–\$10/sample (web scrape vs. licensed)
$C_{\textbf{label}}$	$D_{\text{labeled}} \times c_{\text{label}}$	\$0.01–\$200/sample (crowd vs. expert)
$C_{\textbf{store}}$	$D_{\text{vol,store}} \times c_{\text{storage}} \times T_{\text{months}}$	\$0.02–\$0.10/GB/month
$C_{\textbf{process}}$	$D \times N_{\text{epochs}} \times O_{\text{sample}} \times c_{\text{FLOP}}$	Proportional to training FLOPs

The interplay of these four cost terms becomes concrete when evaluating an ImageNet-scale vision model training run. Calculating each term explicitly shows how easily data acquisition and human labeling outspend the accelerator hours needed to fit the weights.

Napkin Math 9.6: Cost breakdown: ImageNet-scale training

Table 9.11 itemizes one illustrative acquisition, labeling, storage, and compute scenario at ImageNet scale. The resulting ratio follows from the stated licensing, annotation, storage, runtime, and compute-cost assumptions; it is not a typical ratio for every supervised workload.

Table 9.11: Illustrative ImageNet-Scale Cost Breakdown: Itemized acquisition, labeling, storage, and compute costs under the stated scenario assumptions. Each amount follows from the rate shown beside it, using the book's published crowd-labeling, object-storage, and cloud-GPU rates. Data costs dominate this particular calculation because licensing and annotation are included while the training run itself is short.

Cost Component	Calculation	Amount
Raw data (1.2M images)	Licensed dataset, flat fee	$50,000
Labels (crowd annotation)	1.2M × $0.05/label	$60,000
Storage (cloud object store)	150 GB × $0.02/GB/month × 12 months	$36
Training campaign (30 runs, each 100 epochs in 3.5 h on 1 H100)	105 GPU-hours × $4/GPU-hour	$421
Total		**$110,457**
Data vs. Compute ratio		99.6% data, 0.4% compute

Systems insight: Acquisition and labeling can dwarf compute cost in supervised vision workloads, as they do under these assumptions. The compute figure covers the whole training campaign at the compute-bound rate of notebook 9.4, not one run: a published model is the survivor of sweeps, restarts, and ablations, and costing a single run would understate compute roughly thirtyfold. Even so, acquisition and labeling remain the larger share. The run count is an assumption like any other, so a complete ROI calculation must state it and count both categories rather than assuming which one dominates.

Data selection techniques insert algorithmic filtering into the training pipeline to reduce downstream costs. Scoring consumes accelerator cycles, generates host-device memory traffic, or requires custom pipeline stages, so a strategy is justified only if the downstream savings exceed these overheads. Return on investment (ROI), the savings net of the investment per unit invested, makes that test explicit. Because a selection investment is often reused across a training campaign, the general form counts the N_{runs} runs that share it, the **amortized ROI**:

$$\text{Amortized ROI} = \frac{N_{\text{runs}} \times \text{Per-Run Savings} - \text{One-Time Investment}}{\text{One-Time Investment}} \times 100\% \tag{9.3}$$

A single run sets $N_{\text{runs}} = 1$ and recovers the ordinary ratio of savings minus investment to investment, whereas the balance between investment and savings varies across selection strategies. Table 9.12 maps the primary cost drivers and benefit mechanisms across four standard techniques; their net return depends on corpus volume, model scale, and existing pipeline infrastructure.

Table 9.12: ROI Profiles for Data Selection Techniques: The table identifies costs and possible savings. No technique guarantees positive ROI. A training-time claim must satisfy the selection inequality; labeling savings or reusable data investments require their corresponding total-cost comparison at target quality.

Technique	Investment (Cost)	Savings (Benefit)
Deduplication	One-time compute for hashing + infrastructure	Reduced storage and repeated-sample processing
Coreset Selection	Proxy model training + selection compute	Fewer retained samples if target quality and coverage hold

Technique	Investment (Cost)	Savings (Benefit)
Active Learning	Inference on unlabeled pool + human-in-the-loop latency	Lower labeling demand when query selection transfers
Data Augmentation	CPU/GPU cycles for transforms	Effective dataset size increase without new data acquisition

The profiles in table 9.12 compare those costs directly. Deduplication requires an upfront investment in hashing or embedding infrastructure, but can pay off across many epochs and runs by eliminating redundant storage and compute. Coreset selection demands training a proxy model to score the dataset; it only breaks even if the compute saved by dropping samples exceeds the cost of that proxy training. Active learning shifts the cost from human labelers to machine inference, generating high ROI when human time is expensive, though it introduces human-in-the-loop latency that can stall automated pipelines. Finally, data augmentation requires zero new data acquisition but shifts the burden to the CPU or GPU, trading compute cycles for effective dataset expansion. Each technique is worth adopting only when its savings exceed what it costs to set up and run, which makes locating that exact crossover threshold the next step.

9.8.2 Break-even analysis

If candidate scoring is computationally heavy or the unlabeled pool is massive, a selection policy can spend more money scoring candidates than it saves on omitted labels or training steps. The **break-even point** marks the operating boundary where the financial savings from omitted samples exactly equal the computational and operational overhead of selecting them.

Suppose labeling costs \$10/sample, active learning starts from 1,000 labeled samples (\$10,000), issues 100 queries per round at \$50 inference cost (the expense of scoring the candidate pool U with the active model), and the random-labeling baseline requires 5,000 samples for target accuracy. If active learning reaches target accuracy with only 2,000 labeled samples, the following comparison reports net savings relative to the stated labels-plus-scoring cost. This calculation includes labels and pool scoring only, assumes both methods reach the same target accuracy, and excludes retraining, engineering, storage, and annotation-delay costs. It reports net savings per dollar of that stated active-learning budget, using a different denominator from the investment ROI of equation 9.3:

$$\text{Random labeling cost} = 5{,}000 \times \$10/\text{sample} = \$50{,}000$$

$$\text{Active learning cost} = 2{,}000 \times \$10/\text{sample} + 10 \text{ rounds} \times \$50 = \$20{,}500$$

$$\text{ROI} = \frac{\$50{,}000 - \$20{,}500}{\$20{,}500} = 143.9\%$$

Within this labels-plus-scoring boundary, break-even occurs when avoided labeling costs equal selection overhead. A complete dollar comparison must also deduct any additional retraining, engineering, storage, and delay-related expense relative to the random-labeling baseline. When an unlabeled candidate pool U contains millions of uncurated examples, repeatedly scoring it across multiple rounds scales scoring cost with $|U| \times c_{\text{infer}}$, meaning an algorithmic reduction in labeled samples can be erased if proxy inference costs are not carefully bounded. When one fixed budget funds both scoring and labeling, every dollar spent scoring the pool is a dollar withdrawn from labels, so practical loops narrow U with metadata or a cheap heuristic before the active model scores it. Even a pipeline that loses money on a single run, however, can cross break-even once its output is reused across an entire training campaign.

9.8.3 Amortization across training runs

Charging the entire engineering and preprocessing bill of a static curation pipeline to a single training run makes a profitable campaign investment look like a net loss. Filters such as MinHash deduplication, exact n-gram indexing, and embedding clustering require substantial upfront engineering and compute to build once, while production model development is inherently iterative: hyperparameter search, architecture sweeps, ablation studies, and seed checks repeatedly train on the same curated corpus. The amortized ROI of equation 9.3 therefore spreads that upfront curation

investment across every run in the campaign, as table 9.13 itemizes for an illustrative MinHash deduplication pipeline.

Table 9.13: Deduplication Infrastructure Cost Components: The one-time investment covers engineering effort and initial compute; the per-run savings accrue with every subsequent training run on the deduplicated data.

Component	Cost
Build deduplication pipeline	$50,000 (engineering time)
Compute MinHash signatures (one-time)	$5,000
Per-run savings	$10,000/run

Because the initial infrastructure and signature-generation costs are fixed, each subsequent training run compounds the net financial return. Three operational factors govern whether curation infrastructure reaches break-even:

- **Repeated training runs**: Hyperparameter search, model iterations, and scheduled retraining reuse the same selection infrastructure many times.
- **Shared datasets**: A cleaned or deduplicated corpus can support multiple teams or model architectures.
- **Broadly reusable techniques**: Methods such as deduplication transfer across models, whereas task-specific coresets may not.

Table 9.14 tracks how amortized return scales with campaign run count under these cost parameters. A pipeline that loses money on a one-off experiment crosses break-even within a handful of retraining cycles and compounds its savings across larger campaigns.

Table 9.14: Amortized ROI over Multiple Training Runs: Under this scenario's costs and savings, a deduplication pipeline that loses money on its first use becomes profitable when reused across enough runs. Infrastructure investments should be evaluated over their expected lifetime rather than from one use.

Number of Runs	Amortized ROI
1 run	-81.8% (net loss)
5 runs	-9.1% (near break-even)
10 runs	+81.8% (positive)
50 runs	+809.1% (highly profitable)

Deduplication achieves high investment leverage because eliminating redundant sequences reduces host-storage footprint and avoids wasted gradient updates across every downstream architecture. Sample-selection methods conditioned on proxy gradients or uncertainty scores transfer within a model family or between related architectures, but their rankings must be revalidated when the target model, task, or data changes, which narrows their amortization window relative to deduplication. For exploratory or one-off training runs, low-overhead methods like uniform random sampling or basic augmentation may offer better ROI. A static scoring pass can still pay for itself in one run if its measured savings exceed its cost at target quality.

Checkpoint 9.3: Accepting a data intervention

A smaller training pool is one observation in the acceptance decision. Connect the measurements to the constraint the intervention was meant to relieve.

- ☐ **Time savings**: Identify the scoring, coordination, and training times needed to compare a selected subset with full-data training at the same target quality.
- ☐ **Cost objective**: Explain how an active-learning pipeline could reduce total expenditure even if it takes longer to train, and identify the costs that must enter that comparison.

□ **Coverage**: Decide whether unchanged aggregate accuracy is sufficient when a deployment-critical rare slice regresses, and name the additional validation evidence needed. For the disjoint-class allocation in section 9.3.4, explain why reducing K below $a + b$ requires revising the selection plan before training, even if every remaining example has a high score.

With time, labeling, and amortized costs expressed in one accounting, every technique in the chapter can now be weighed against the claim it makes. A training-time saving must satisfy the selection inequality, a labeling saving must beat the random-labeling baseline after scoring and any other differing costs, and a reusable investment must recover its one-time cost across the runs or tasks that share it. Costing each technique separately, however, does not say which one to reach for first when several of them compete for the same budget.

9.8.4 Decision process

When labeling budget, redundancy, rare classes, privacy, and convergence speed conflict, systems engineering requires identifying the primary binding constraint. Table 9.15 collects the three levers with order-of-magnitude ranges drawn from scenarios like those worked in this chapter; each must reduce the cost it targets after counting its overhead and preserving target quality and required coverage. Self-supervised pretraining, which changes the training signal rather than the sample set, enters the decision through the labeling-cost branch of the tree that follows.

Table 9.15: Three-Stage Data Selection Pipeline: The ranges illustrate possible outcomes rather than portable expectations. Actual gains depend on the dataset, model, selection method, and target metric.

Stage	When Applied	Techniques	Order-of-Magnitude Scenario Range
Static pruning	Before training	Coreset Selection, Deduplication, Quality Filtering	30–50% dataset reduction
Dynamic selection	During training	Curriculum Learning, Active Learning, Semi-Supervised	10–30% faster curricula; 2–100× fewer labels
Synthetic generation	On-demand	Augmentation, Generative Models, Distillation	2–10× effective data expansion

Once the dominant constraint is named, it points to a candidate technique, and the five lighthouse workloads, each bound by a different resource, show the pairing in practice. Privacy carries a condition of its own. When privacy requirements limit direct use of real records, privacy-audited synthesis can reduce that use, provided the synthetic data is checked for leakage of the records it was derived from.



Lighthouse 9.1: Lighthouse data selection

Data selection principles apply to all five lighthouse models, though the dominant constraint determines the technique. Table 9.16 maps each workload to a priority. Compute-bound ResNet-50 training benefits from coreset selection, while GPT-2 XL training relies on deduplication and quality filtering to cut the raw tokens it must process. MobileNetV2 is compute-bound in its efficient regime, and its binding deployment constraints, latency and power, apply at inference, where no choice of training data changes the cost of a prediction. The data lever that matters is therefore the one that helps a small model generalize, augmentation matched to its capacity (compact networks underfit under distortion that a large model absorbs), and the work is validating that the augmentation preserves the invariances the deployment environment requires. DLRM, the capacity-bound lighthouse, spends much of its terabyte-scale embedding capacity on cold IDs that appear rarely in training data. It uses interaction deduplication to cut redundant examples and embedding pruning to shrink those tables, but removing repeated interactions shrinks a table only when IDs disappear or the allocation, hashing, or sharing

policy changes. Finally, keyword spotting, the power-bound lighthouse, is bound for data selection by a scarce, costly seed set as much as by its power and memory budget, and it overcomes that scarcity by using augmentation and synthesis to build robust datasets from minimal seed samples.

Table 9.16: Data Selection Priorities by Lighthouse: Primary bottleneck and candidate data-selection priority for each lighthouse workload. The pairing suggests a hypothesis to test; the dominant resource constraint alone does not guarantee that a technique pays off.

Lighthouse	Primary Bottleneck	Data Selection Priority
ResNet-50	Compute	Coreset selection directly reduces training FLOPs
GPT-2 XL	Training compute (tokens × FLOPs/token)	Deduplication and quality filtering cut tokens processed; curriculum may improve token efficiency
MobileNetV2	Latency/Power	Validate augmentation against the target model and deployment invariances
DLRM	Memory capacity	Interaction deduplication cuts examples; embedding pruning shrinks tables only when cold IDs are dropped or rehashed
Keyword Spotting	Data scarcity (small seed set)	Augmentation and synthesis create datasets from minimal seeds

Across these workloads, data selection functions as a systems optimization tailored to relieve whichever physical resource binds the pipeline.

The lighthouse mapping assumes a single dominant constraint, but real projects face several at once. The triage flowchart in figure 9.12 therefore routes candidate strategies hierarchically from the root bottleneck (labeling expense, compute limits, or data scarcity) through operational checks on oracle availability, dataset redundancy, teacher or simulator support, and unlabeled-pool size to isolate actionable techniques.

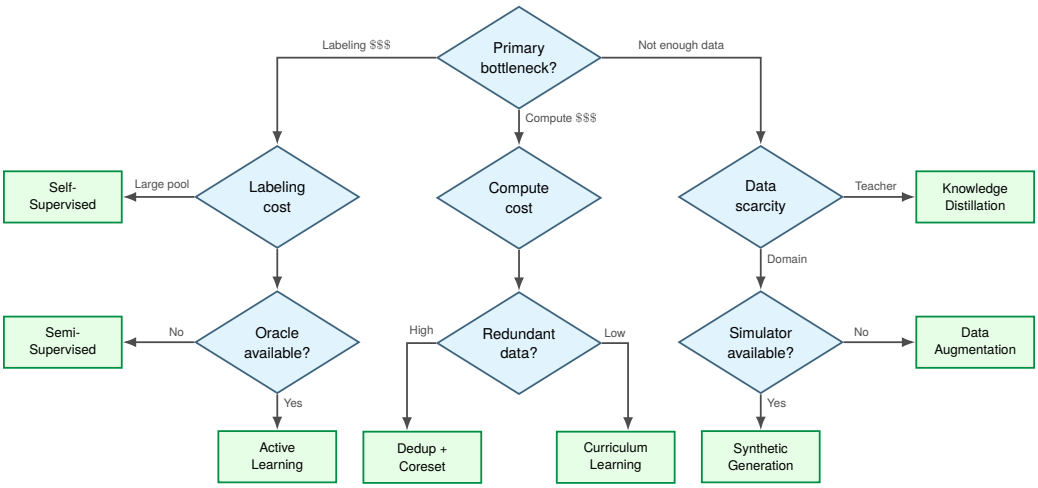


Figure 9.12: Data Selection Decision Tree: The tree begins with the primary bottleneck (labeling cost, compute cost, or data scarcity) and narrows each branch using available pools, oracles, redundancy, teachers, or simulators. Its leaves identify candidate techniques; projects with multiple binding constraints may require more than one branch.

The tree identifies candidates; an engineering decision must still test their cost and quality. The vision workload in example 9.2 provides a concrete comparison. Its target-model and proxy scorers retain the same fraction of data, but they need not retain the same examples. A cheaper ranking can therefore win the time comparison and lose the deployment-quality comparison.

Step 1: Assess the bottleneck

The vision team's stated objective is shorter training turnaround at the existing quality target. The full-data baseline takes 211 minutes, and both candidates retain 10 percent of the images. This places the decision in the compute-cost branch. The retained fraction alone does not settle it; the selection inequality and the slice-coverage check do.

A different binding constraint would change the comparison. If labels were the limiting expense, fewer annotations could justify additional training time. If controlled sweeps instead diagnosed a shortage of useful tokens, the data-scarcity branch would be relevant even with idle accelerators. Naming the objective determines which saving the candidate must actually deliver.

Step 2: Check prerequisites

Both vision candidates require access to the full image pool, a scoring model, and stable sample indices. The proxy additionally needs a ranking that transfers to the target model; its lower execution cost does not establish that prerequisite. Table 9.17 places these requirements alongside those of the alternative branches.

Table 9.17: Technique Prerequisites: Each technique carries specific infrastructure and data requirements that must be verified before implementation. A technique with excellent theoretical gains but unmet prerequisites will fail in practice, making this checklist the first step in technique selection.

Technique	Prerequisites
Active Learning	Access to oracle, unlabeled pool, retraining infrastructure
Coreset selection	Proxy model or embedding extractor, full dataset accessible
Curriculum Learning	Difficulty scoring method, pacing schedule
Semi-Supervised	Some labeled data, unlabeled data from same distribution
Self-Supervised	Large unlabeled corpus, pretraining compute budget
Augmentation	Domain knowledge of invariances, augmentation library
Synthetic Generation	Generative model or simulator, domain gap mitigation

Step 3: Estimate ROI

For the stated vision scenario, target-model scoring plus subset training takes 21.8 minutes (Option A), while proxy scoring plus subset training takes 21.4 minutes (Option B). These figures evaluate the marginal scoring pass using existing checkpoints. Both options beat the full-data baseline in the cost model. The proxy reduces only the scoring term; the retained training work is identical.

Suppose pilot validation in this hypothetical decision confirms that both runs reach the aggregate target under the modeled schedule, but only Option A meets every deployment-critical slice threshold. Option B regresses on a rare class. Option A is then the acceptable choice despite its more expensive scoring pass, saving 188.8 minutes against the baseline. Option B remains a candidate for repair, not deployment: enforce the coverage floor from section 9.3.4, rescore or enlarge the subset as needed, and repeat the time-to-quality comparison. Additional examples or passes change its cost, so the original proxy saving cannot simply be carried forward. If both candidates passed all quality gates at the stated times, Option B would win this time comparison.

A time saving must still justify any engineering investment. If the proxy checkpoint does not already exist and its training is charged entirely to a single run, that preparatory training erases Option B's modest time advantage over Option A. Section 9.8.1 accounts for that investment, and section 9.8.3 spreads one-time costs across the N_{runs} planned experiments. Reusing the proxy is economically useful only while its ranking continues to pass the same quality checks.

Step 4: Combine techniques

Data selection techniques are not mutually exclusive, but combining them does not guarantee additive gains. A representative workflow deduplicates the raw corpus, applies a coverage-aware coreset filter, orders retained batches through an empirical curriculum, and applies runtime augmentations during batch collation. Pretrained representations alter the baseline trajectory further. Every additional stage must justify its compute overhead, because cascading filters frequently discard overlapping subsets while their scoring latencies accumulate; stages compound efficiency gains only

when their effects do not overlap and their overhead remains small, so combined savings must be measured rather than assumed.

A choice made at the data layer does not stay there. This decision process weighs a technique by its own scoring cost and the training work it removes, but a smaller or reordered corpus also changes what the model learns to represent, and the compression and deployment stages downstream inherit that representation. A selection that looks optimal in isolation can therefore move cost into those later stages, so the chosen strategy must be checked against the layers it feeds.

9.9 Cross-Layer Interactions

Halving a training corpus halves the forward and backward passes, yet the resulting model may be no cheaper to compress or serve. Real ML systems combine data selection with model compression and hardware acceleration, and these cross-layer interactions amplify or undermine each other in ways no single layer reveals.

Model compression (post-training weight pruning, lower-precision representation, and distillation) acts on the trained network, and its arithmetic reductions reach serving latency only when the target execution substrate and runtime kernels support the resulting sparsity or precision patterns (Hoefler et al. 2021). The training corpus shapes the representations that compression later works on. A model trained on an uncurated, repetitive corpus can spend capacity on redundant features that post-training pruning then discards, whereas a curated corpus of diverse, informative examples can yield more compact representations from the start. That effect is an empirical hypothesis to verify, not a property of every selected dataset. The two levers therefore compose without substituting for each other. Data selection reduces training work and shapes learned representations, model compression shrinks the deployment artifact, and their combined impact is measured on serving latency and task accuracy rather than inferred from the upstream dataset reduction.

↪ Why accelerators turn structured sparsity patterns into wall-clock speedup while unstructured sparsity rarely pays off is developed in section 11.4.5.3.

↪ Co-designing post-training pruning and quantization under the silicon contract is developed in section 10.1.

Production deployments execute these techniques as one pipeline in two stages: data selection and training turn raw inputs into a trained model, and post-training compression and hardware acceleration then deploy that artifact (figure 9.13). Every stage along this end-to-end path amplifies or attenuates downstream efficiency.

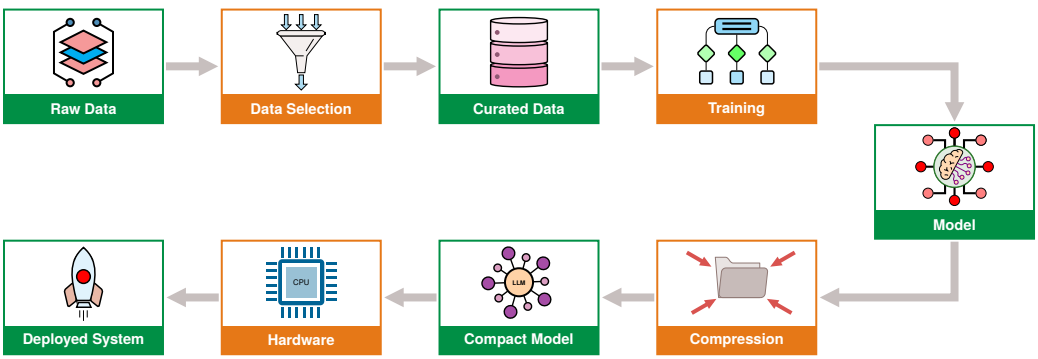


Figure 9.13: The Optimization Stack: Data artifacts flow through selection and training before the resulting model enters compression and deployment. Upstream changes propagate quality and cost effects downstream, but reducing training data does not automatically simplify the model or deployment hardware.

Tracing artifacts through the pipeline in figure 9.13 shows why data selection sits at the head of the optimization stack: pruning raw inputs reduces upstream training FLOPs, but does not automatically shrink parameter footprint, improve quantization headroom, or relax serving latency constraints in the downstream compression and hardware stages. Each downstream tier inherits both the computational savings and any representational deficits created upstream; an aggressive filter that discards rare but critical edge cases can force downstream engineers to train longer or abandon aggressive quantization to preserve task accuracy. The failure modes that follow show how data selection breaks down in production.

9.10 Fallacies and Pitfalls

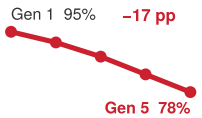
Scaling a training corpus tenfold can multiply storage, I/O, and accelerator bills while barely moving test accuracy, yet an uncalibrated selection filter can just as easily spend more time scoring candidates than it saves in training or silently erase a safety-critical rare class. These failures fall into three groups: conceptual fallacies about what data selection can achieve, implementation pitfalls that arise when correct strategies meet systems bottlenecks, and transfer errors that occur when benchmark results are applied uncritically to new domains.

Fallacy: *More data always improves the model enough to justify its cost.*

More data does not imply proportional accuracy gains because learning curves follow power-law diminishing returns. Test loss diminishes sub-linearly with dataset size ($\mathcal{L}(D) \propto D^{-\beta}$ for a fitted exponent β), meaning each successive error reduction demands a multiplicatively larger corpus. In an illustrative learning-curve scenario, scaling a dataset tenfold from 1M to 10M samples yields only 4 percentage points of accuracy gain. That $10\times$ volume expansion still costs a tenfold increase in storage I/O, host-to-device transfers, and accelerator FLOPs. Teams should measure the learning curve of their own workload rather than assume that more data delivers proportional value.

Pitfall: *Replacing real-data validation with synthetic-only training data.*

Synthetic samples cost almost nothing to acquire, so generative models can look like a replacement for data collection. Synthetic-only training fails in two ways. First, synthetic data can differ from the deployment distribution (section 9.5.3, figure 9.8), so the model learns features that fail to transfer; only real-data labels reveal the effect. Second, recursive training on model-generated data can trigger model collapse (Shumailov et al. 2024), which progressively erodes distribution tails: in this scenario, evaluation accuracy degrades from 95 percent to 78 percent across five recursive generations, suffering a 17 percentage-point collapse. Synthetic mixtures must therefore be validated against curated real data.



Illustrative model collapse: accuracy erodes with each recursive synthetic generation.

Fallacy: *Data selection is merely data cleaning.*

A dataset can be completely clean and correctly labeled yet redundant: once the model has minimized loss on a feature cluster, further identical samples add almost no gradient signal and still consume memory bandwidth and accelerator FLOPs. Cleaning corrects intrinsic defects; selection raises the learning return per FLOP by choosing examples for task-relevant information and coverage (figure 9.5; section 9.3.4). In the worked coreset scenario of notebook 9.1, the selected subset reached 1.8× higher ICR.

Pitfall: *Treating data selection as a budget-only tactic.*

Selection can look like a technique for resource-constrained teams, as if frontier foundation models could brute-force convergence by scaling the cluster. The data wall binds at every scale. At frontier scale, token consumption converges toward the finite ceiling of high-quality human text (figure 4.6); at GPT-2 scale, one eight-H100 machine passes over the whole WebText corpus about 17 times in 4.4 days (section 9.1). The economic return of selection also scales with cluster expenditure: a 10 percent efficiency gain on a $100M frontier training run saves $10M under these assumptions. Amortizing scores and cached embeddings across repeated runs (section 9.8.3) strengthens the case further.

Fallacy: *Selection overhead is too small to change training economics.*

The selection inequality (equation 9.2) requires $T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}})$ at matched quality. Consider a baseline training run requiring 8 hours, where training on an optimized coreset takes 2 hours. If a complex selection algorithm requires 10 hours to compute sample importance, the selection step alone consumes 5× the duration of the accelerated training run and exceeds the full training baseline itself, a net wall-clock loss. In the illustrative comparison, proxy-based EL2N scoring completes in 30 minutes, just 6.2 percent of the full-training baseline, which satisfies the inequality.

Pitfall: *Pruning rare classes without a coverage floor.*

Standard selection objectives minimize aggregate loss, which gives tail slices little influence over sampling. In a 1M dataset where rare classes comprise 0.1 percent of the data (1,000 examples), an unstratified 10 percent coreset retains only 100 rare-class examples in expectation, far below the

illustrative 150-sample retention floor assumed here. An appropriate floor must be validated from task-specific learning curves and critical-slice performance. Importance sampling can compensate by weighting losses by inverse selection probabilities, but extreme weights inflate gradient variance and force smaller learning rates. As recommended in section 9.3.4, production coreset pipelines enforce hard stratification floors per class or critical slice before spending the remaining selection budget.

Fallacy: *Deduplicating training data is enough to make evaluation reliable.*

Web-scale pipelines scrape training, validation, and benchmark splits from common crawls, which introduces silent cross-split leakage. When an evaluation set contains near-duplicates of training documents, the metric measures memorization rather than generalization. A deduplication study found that train-test overlap affects over 4 percent of validation examples across standard language-modeling benchmarks (Lee et al. 2022). Deduplicating the training partition alone leaves this contamination untouched; cross-partition deduplication is required, using n-gram matching or MinHash LSH (section 9.3.2). For a frozen evaluation set, the fix removes the overlapping training records and preserves the evaluation set and its provenance.

Pitfall: *Active learning without considering annotation latency.*

Textbook formulations of active learning assume an oracle that labels queries instantly. Production annotation queues introduce delays such as the illustrative 14-day latency between sample dispatch and label return assumed here. This latency breaks the active learning feedback loop in two ways. If accelerator clusters halt training while awaiting labels, expensive hardware sits idle, destroying cluster compute utilization. Conversely, if training continues asynchronously on background data while annotations are pending, model weights drift across training steps; by the time the labels arrive, the uncertainty scores that justified querying those samples are obsolete. Dispatching large batches to amortize the turnaround adds redundancy within the batch unless diversity penalties are applied.

Fallacy: *If a technique works on ImageNet, it will work on my dataset.*

CIFAR-10 and ImageNet are highly redundant across canonical object classes, which allows coreset pruning of often 30 to 50 percent without measurable accuracy loss. Histopathology, satellite imagery, and industrial defect detection have heavy-tailed feature distributions in which safety-critical patterns appear in a few rare samples. A 50 percent cut tuned on ImageNet can remove the few examples that separate subtle pathologies. Geometric coreset algorithms also depend on embeddings, and a feature extractor pretrained on natural images may not resolve fine-grained domain distinctions. Teams should measure the redundancy of their own data and validate retention across critical slices before pruning.

Pitfall: *Optimizing data selection metrics instead of deployment metrics.*

Selection algorithms optimize aggregate surrogates such as area under the learning curve (AULC) or performance-per-data (PPD, section 9.2.4). A selection policy retaining a 10 percent coreset can achieve superior aggregate PPD by prioritizing dense clusters of common, easy samples that accelerate average convergence, while discarding ambiguous edge cases and rare slices. Production failures, however, occur on tail slices, edge cases, and protected sub-populations rather than on averages. A policy without subgroup constraints can pass global validation and still fail in deployment. As the measurement framework of section 9.2 requires, policies must be evaluated on stratified, slice-level benchmarks, even when doing so slightly lowers average PPD.

9.11 Summary

Feeding every available example into a training run wastes accelerator compute, storage bandwidth, and annotation budget on redundant or noisy records because marginal learning signal varies by orders of magnitude across raw samples. Data selection therefore treats training volume as an engineering variable rather than a fixed constraint, minimizing total cost across compute, storage, labeling, energy, and wall-clock time so a curated subset can match or outperform a much larger uncurated corpus.

This systems perspective becomes actionable through three quantitative tools: the ICR, the selection inequality, and the total data cost model. The information-compute ratio quantifies learning gained per floating-point operation, providing a proxy for sample value that must be evaluated alongside coverage and target quality. The selection inequality tests a training-time claim by requiring scoring plus subset training to take less time than full-data training at matched quality. Together with return-on-investment and break-even analysis, these formulations determine whether a proposed selection mechanism saves resources or merely shifts expenditure between stages.

The three levers act at different points: static pruning removes redundancy before training, dynamic selection prioritizes informative examples during training, and synthetic generation creates data where none exists. Together, these levers answer the data wall, the gap between rapidly growing compute capacity and slowly growing high-quality data.

Learning from unlabeled data sits beside these levers rather than within them. Semi-supervised learning stretches a small labeled seed across an unlabeled pool, and self-supervised pretraining moves the bulk of representation learning into an unannotated upstream phase, so downstream tasks can reach target quality with far fewer task-specific labels; the worked amortization scenario assumes 100× fewer. The compute-only crossover occurs at roughly 10.5 tasks under the scenario assumptions. Labeling savings enter a separate dollar comparison that also requires compute prices and other differing costs.

In production, the difficult part is rarely the scoring rule itself. It is keeping selection from becoming the next bottleneck. Proxy models and cached embeddings bound scoring time, shard-based loaders keep storage access sequential, and data echoing recovers accelerator cycles when ingestion lags. On one H100, a half-size ResNet-50 coreset saves about 63 seconds and 44 kJ per epoch against a one-time scoring pass of about 42 seconds, and those seconds are realized only when the host keeps the accelerator fed. Controlled model-size and data sweeps diagnose whether a run is data-limited or compute-limited in the first place, and PPD, AULC, and DCR then show whether a reduction delivered what it promised at the binding resource.

Every row of the opening chapter map (table 9.1) has now been derived, and the formulas behind its tests fit on one page (table 9.18). Together they let an engineer estimate, before committing a training run, whether a lever saves more than it costs.

Table 9.18: Key Equations: The chapter's counting rules in the notation of the sections that derive them. The training-compute rule is the dense-transformer accounting of the GPT-2 lighthouse; the curriculum rule assumes linear pacing over N_{warmup} warmup epochs.

Quantity	Formula	Where derived
Information-compute ratio	$\text{ICR} = \Delta\mathcal{I}/\Delta O$	section 9.2.1
Marginal ICR (log-information)	$\text{ICR}(D) \approx 1/(O_{\text{sample}} \cdot D)$	equation 9.1
Training compute	$O \approx 6PD$; compute-optimal tokens $D_{\text{opt}} \propto \sqrt{O}$	section 9.2.3
Performance-per-data	$\text{PPD}(n) = [\text{Accuracy}(n) - \text{Accuracy}(0)]/n$	section 9.2.4
Data compression ratio	$\text{DCR} = D_{\text{total}}/D_{\text{subset}}$ at target accuracy	section 9.2.4
Selection inequality	$T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}})$	equation 9.2
Curriculum warmup samples	$D(N_{\text{warmup}} + 1)/2$ in place of $N_{\text{warmup}} \cdot D$	section 9.4.1
Offline distillation targets	$4K$ bytes per sample in place of $\lceil \log_2 K \rceil$ bits	section 9.5.4
Input-pipeline ratio	$r_{\text{pipe}} = T_{\text{prep}}/T_{\text{GPU}}$; echoing helps up to $e = r_{\text{pipe}}$	section 9.7.3
Total cost of data	$C_{\text{total}} = C_{\text{acquire}} + C_{\text{label}} + C_{\text{store}} + C_{\text{process}}$	section 9.8.1
Amortized return on investment	Savings across N_{runs} runs, less the one-time investment, per unit invested	equation 9.3

Key Takeaways: Curate, do not accumulate

- **Selection optimizes system cost**: The objective is minimizing total cost across compute, storage, labeling, and energy rather than reducing sample counts alone. The information-compute ratio measures learning gained per FLOP spent; total-cost accounting and pipeline measurements establish whether that gain serves the system objective.
- **Start with deduplication**: Exact deduplication is often the lowest-risk first technique because it removes repeated bytes without a learned scoring pass. Apply near-duplicate and semantic deduplication only after validating thresholds against downstream quality metrics.
- **Training-time savings must exceed selection overhead**: At matched quality, $T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}})$. Lightweight proxy models and cached embeddings keep $T_{\text{selection}}$ low, whereas unoptimized selection algorithms can consume all the compute they save.
- **Dynamic selection adapts the training mix**: Curriculum learning changes presentation order, while active learning changes which samples receive labels. Either improves efficiency when its scoring rule, coverage, and refresh overhead are validated.
- **Self-supervised pretraining can amortize adaptation cost**: Pretraining once and fine-tuning many times spreads pretraining cost across downstream tasks. The worked scenario assumes a 100× label reduction and a 20× marginal-compute reduction; realized savings depend on transfer quality and reuse.
- **Synthetic data is a supplement, not a replacement**: Synthetic samples augment scarce tail cases effectively, but cannot substitute entirely for empirical data. Unanchored synthetic training introduces domain-gap degradation, and recursive synthetic loops trigger catastrophic model collapse.
- **Validated workload reduction compounds downstream**: When selection safely eliminates examples or tokens without adding compensating steps, that work never reaches later model or machine optimizations. Dynamic scoring and synthesis add their own costs, so the combined saving must be measured rather than assumed.

Examples differ greatly in how much they teach per byte transferred and per FLOP executed, and selection spends compute on the ones that teach most. Within the D·A·M taxonomy, selection operates at the head of the system: every redundant sample or uninformative token safely eliminated directly reduces operation count (O) in the iron law of ML systems (principle 3) and can avoid downstream training reads. Scoring may still read the entire pool, and shard streaming may read discarded records, so reductions in data volume (D_{vol}) must count scoring and shard-read traffic separately. When selection overhead satisfies the inequality at matched quality, the reduction removes work before downstream compression or hardware acceleration sees it. Model-based scoring offers a way to invest additional compute in selection, consistent with the bitter lesson, but a stronger scorer justifies that investment only if its improved rankings save more downstream work than it costs. The Pareto frontier (principle 5) applies upstream of the model as well. Each retained fraction is an operating point that trades coverage and rare-slice accuracy against compute, and no single fraction is optimal for every workload.

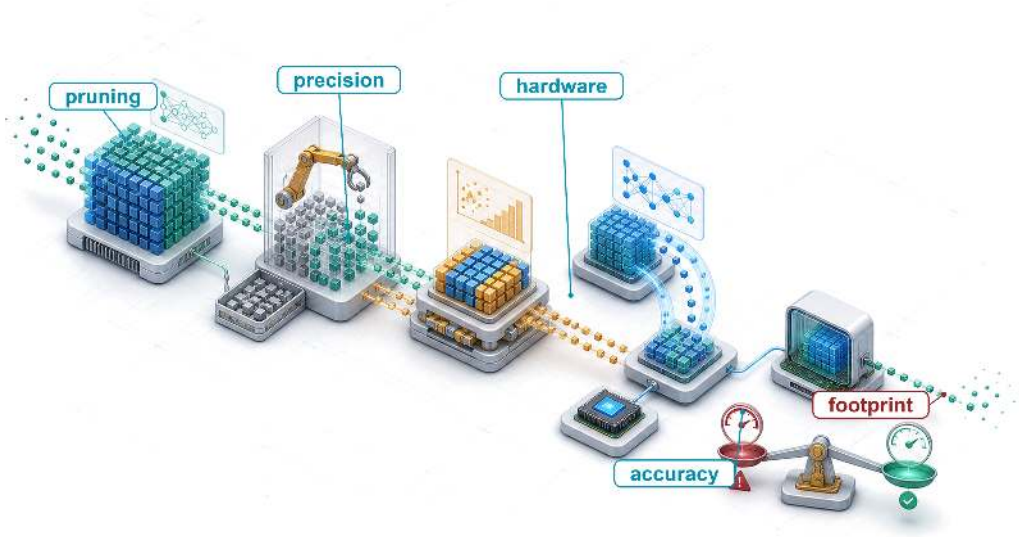
What's Next: From data to algorithms

Data selection changes *what* the system learns from. Chapter 10 turns to *how* the model represents what it has learned. It starts from the deployment envelope the model must fit and adjusts three things to fit it: the model's structure through pruning and distillation, its numerical precision through quantization, and the way the remaining operations map onto the hardware that executes them.

10

Model Compression

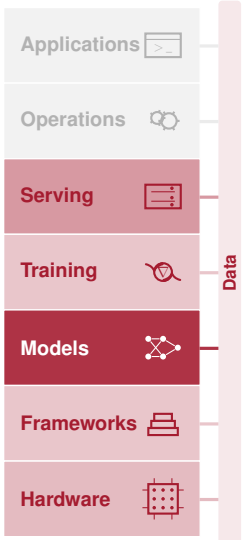
10.1 Fitting the Deployment Budget
10.2 Structural Optimization: Changing What the Model Computes
10.3 Numerical Precision: Changing How Values Are Stored
10.4 Hardware Implementation: Changing How the Model Executes
10.5 Choosing and Combining Techniques
10.6 Measuring Compression Gains
10.7 Fallacies and Pitfalls
10.8 Summary



Purpose

Why do the models that win benchmarks rarely become the models that run in production?

A capable trained model is not automatically a deployable one. Cloud, Edge, Mobile, and TinyML environments each impose physical constraints that research benchmarks ignore, including memory budgets measured in megabytes rather than gigabytes, latency targets measured in milliseconds rather than seconds, and power envelopes measured in milliwatts rather than kilowatts. Research optimizes for accuracy on held-out test sets; production optimizes for accuracy per dollar, accuracy per watt, and accuracy per millisecond. Models that win benchmarks are frequently larger, slower, and more resource-intensive than production envelopes permit. Bridging this gap requires a systematic discipline of compression that trades capabilities the deployment does not need for physical constraints it cannot violate. Because many trained models carry more numerical precision, parameters, or representational capacity than the target workload requires, some of that surplus can be removed while preserving predictive behavior, though a smaller representation is not automatically a faster one. Unless the hardware and runtime software stack exploit the new precision or structure, a nominal size reduction merely moves the bottleneck while latency remains unchanged: removing half the weights in a tensor leaves execution time and energy untouched if the underlying hardware still fetches, stores, and multiplies every zero. Applied effectively, compression reduces model size by integer factors, transforming a research artifact that runs only on data center accelerators into a production asset capable of executing on a mobile phone, sensor node, or embedded microcontroller. The engineering goal is to make the right model viable within its physical environment, which differs fundamentally from making a model as small as possible. In D·A·M terms, model compression



enacts algorithm-machine co-design directly on the network, reshaping its mathematical structure to satisfy the physical limits of the target hardware.

Learning Objectives

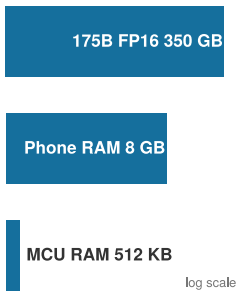
- Explain compression as algorithm-machine co-design that trades surplus capacity for memory, latency, and energy constraints
- Compare pruning, distillation, and quantization by the resource each relaxes and where each lands on the accuracy-efficiency frontier
- Calculate parameter memory, precision, and sparsity reductions to estimate best-case compression gains
- Apply post-training, quantization-aware, and weight-only strategies under accuracy and hardware constraints
- Select structured pruning and operator choices that map to available accelerator kernels
- Design compression pipelines that order pruning, distillation, and quantization and account for why their combined gains do not multiply
- Evaluate measured latency, energy, and accuracy on target hardware rather than relying on FLOP counts

10.1 Fitting the Deployment Budget

A trained model arrives with demands on memory capacity, latency, and energy that were set by the server accelerators it trained on, not by the device it must run on. The target device instead enforces a strict **deployment budget**: the memory, time, and energy it grants to each result the model produces. Memory capacity bounds the bytes of weights and activations that can stay resident, a frame rate or token rate bounds the milliseconds available for each result, and a battery or thermal envelope bounds the joules that each result may consume. Bringing a model's demands inside that budget, by changing what the model computes, how its values are stored, and how its operations execute, is the work of model compression.

Chapter 9 lowered the cost of producing a trained model by spending training compute only on examples that carry signal. Its savings end where the trained model begins. A model trained on a curated subset has the same parameter count, numerical precision, and memory footprint as one trained on the full corpus, so it can still overflow a phone's memory or leave inference bound by memory bandwidth. Chapter 11 then examines the silicon that executes a model. Compression comes before it because compression decides what that silicon is asked to run: how many operations, how many bytes per value, and which kernels.

The gap is easiest to see at its extreme. A 7-billion-parameter language model requires 14 GB merely to store its weights in FP16. A smartphone offers 8 GB of RAM shared across the operating system, background applications, and the user interface, so even if the operating system surrendered its entire memory pool to one process, the weights would not fit. Fitting in memory would only expose a second limit, the token rate. At batch size one, dense autoregressive decoding is memory-bandwidth bound because generating each token reads every weight from DRAM, and no other request shares those bytes. Over a flagship phone's 51.2 GB/s low-power DDR (LPDDR) bus, streaming 14 GB takes about 270 ms per token and about 2 J of DRAM access energy, so the memory traffic alone would draw about 8 W, above the phone's sustained thermal envelope of 3–5 W. A model of this size is the extreme case; the rest of the chapter works with models that come closer to fitting, where the engineering choices are less obvious.



Frontier weights dwarf phone and microcontroller memory; compression bridges the gap.

Under the silicon contract (principle 4), compute throughput, memory bandwidth, memory capacity, and fixed launch overhead determine which physical resource binds first for a given model on a given machine. Training sets those quantities generously. Server-class accelerators supply the aggregate memory and floating-point throughput that wide layers, high numerical precision, and large auxiliary state require, because training must hold gradients, optimizer moment buffers, and activation caches across backward passes. Inference deployment relaxes several of these demands because execution is forward-only, intermediate activation buffers can be reused after their last consumer, and optimizer states disappear, even though residual branches still extend some tensor lifetimes and autoregressive decoding retains KV state across tokens. While mixed-precision methods improve training throughput by selectively adopting FP16 or BF16, deployment

can reduce precision further, to INT8, INT4, or specialized microformats, when the target's execution units support them. Model compression is the systematic process of meeting the silicon contract of a constrained execution context, reducing parameter footprint, memory traffic, or arithmetic operations while preserving task accuracy.

↩ Mixed-precision training with FP16 and BF16, including loss scaling and FP32 accumulation, is established in section 8.6.3.

Across deployment targets, the disparity in weight memory spans about six orders of magnitude, from the 350 GB of FP16 weights in a 175-billion-parameter frontier model to the 512 KB of on-chip SRAM in an embedded microcontroller. The bitter lesson explains why this gap persists. General methods that scale with compute keep winning benchmarks, so the winning models grow with available training compute while a phone's or microcontroller's memory and power envelope stays fixed for the life of the device. Compression is the deployment-side counterweight to scale.

↩ The bitter lesson, that general methods which scale with compute keep displacing hand-engineered ones, is established in section 1.4.

The record of language models shows how much ground that counterweight can recover. Figure 10.1 follows the smallest dense model reported to score at least 60 percent on the Massive Multitask Language Understanding (MMLU) benchmark (Hendrycks et al. 2020) at each point in time. That size fell from 280 billion parameters for Gopher at the end of 2021 to 3 billion for Qwen2.5-Coder-3B, a code-specialized model, in 2024, about 93× smaller in under three years (Epoch AI 2026a). Better training data, longer training, and distillation from larger models all contributed, so the decline is not compression alone, but it shows that a capability first reached at frontier scale can later fit a far smaller deployment budget.

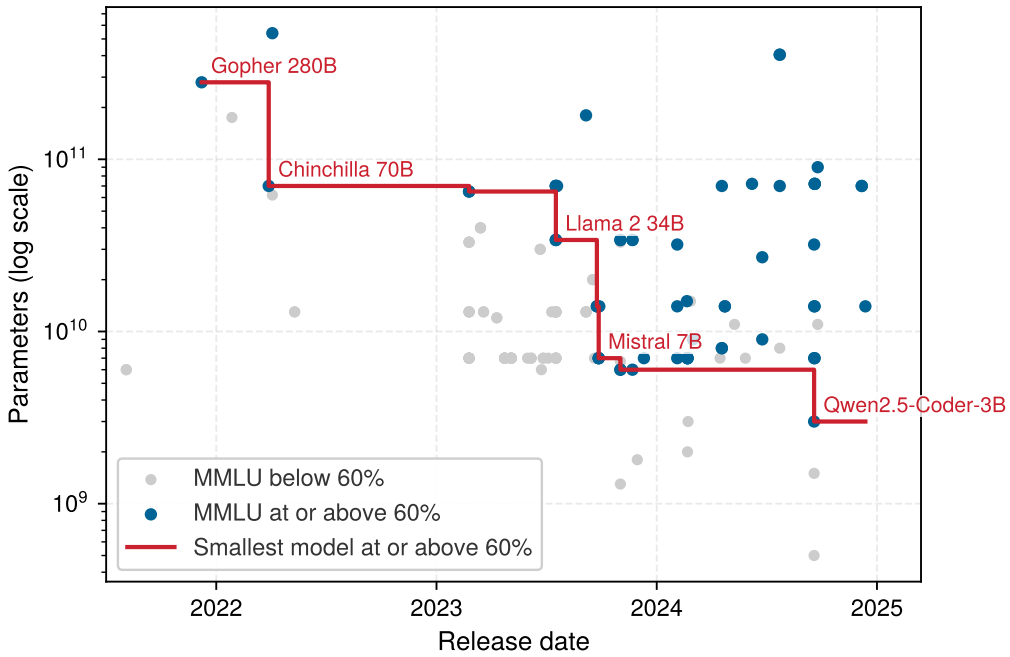


Figure 10.1: Smallest Model Reaching a Fixed Score: Parameter counts of dense language models against release date on a logarithmic axis, with models scoring at least 60 percent on the MMLU benchmark in blue and the rest in gray. The red step line tracks the smallest model at or above that score released up to each date. Scores are as reported by each source, and parameter counts come from model names, so the line is a lower envelope of reported results rather than a controlled comparison. Data from Epoch AI (2026a).

Bridging the gap between frontier models and fixed deployment targets requires an engineering framework with predictable physical trade-offs. Every compression technique removes specific structure from the model, whether unnecessary weights, surplus numerical bits, or redundant operator subgraphs. Designing a production deployment requires identifying which resource binds first on the target platform and analyzing how successive approximations compound across the memory hierarchy. The framework this chapter builds works on three tiers: what the model computes (its structure), how many bits represent each value (its precision), and how the result

executes on the target processor (its implementation). Benefits compound only when transformations relieve distinct binding bottlenecks and the runtime supports the resulting graph.

Definition 10.1: Model compression

Model compression is a family of techniques that reduce a trained model's computational cost and memory footprint by eliminating redundant parameters (pruning), reducing numerical precision (quantization), or transferring learned behavior into a smaller architecture (distillation), while preserving as much predictive accuracy as possible.

1. **Significance**: In the iron law (principle 3), compression targets both the Algorithm factor (FLOPs/Sample, and therefore total operations O) and the Machine factor (Seconds/FLOP, through its physical data-movement and compute terms). INT8 quantization of a 175-billion-parameter LLM leaves FLOPs/Sample unchanged but cuts weight memory from 350 GB (FP16) to 175 GB, a 2× reduction in D_{vol}, while dedicated low-precision matrix units can raise R_{peak} when kernels and layouts use the supported INT8 path. Unstructured pruning to 50 percent sparsity halves the nonzero parameter count, but executed operations fall only when the runtime can skip those zeros through a supported sparse format or kernel without inflating Seconds/FLOP via index-traffic stalls.
2. **Distinction**: Unlike post-training compression methods such as pruning and quantization, neural architecture search discovers efficient architectures from scratch by exploring a design space. Here, NAS is treated as a related structural optimization technique, because it changes the representation before training rather than compressing a finished model post hoc.
3. **Common pitfall**: Compression techniques do not compose without interference. Pruning changes weight distributions and operation patterns; quantization adds calibration and kernel constraints. Their combination can lose accuracy or fail to accelerate without joint validation.

The three tiers form a stack that descends from algorithmic software toward physical silicon, the same direction in which chapter 7 lowered a computational graph to kernels. At the top sits what operations the model executes, in the middle how many bits encode each operand, and at the bottom how kernels map onto the processor's compute and memory structures (figure 10.2).

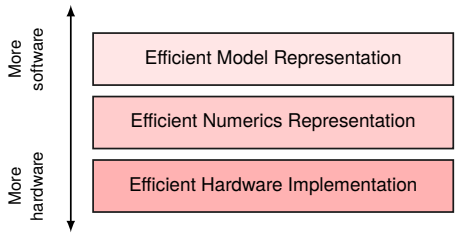


Figure 10.2: Three Tiers of Compression: Model optimization progresses from software-level representation (pruning, distillation, NAS) down through numerical precision (quantization) to hardware execution (kernel fusion, sparsity engines). Lowering abstraction moves optimizations closer to physical silicon, where representation decisions dictate achievable execution efficiency.

Each tier removes a different kind of surplus across the two levels of the iron law. The top tier, efficient model representation, removes structure and lowers the Algorithm factor (FLOPs/Sample). Pruning drops weights or whole channels that contribute little to accuracy, knowledge distillation trains a compact student to match a larger teacher, and neural architecture search (NAS) looks for layer shapes that meet an explicit latency or memory bound. The middle and bottom tiers leave FLOPs/Sample unchanged and instead lower the Machine factor (Seconds/FLOP). In the middle tier, efficient numerics representation removes bits. Quantization maps floating-point values to 8-bit or 4-bit integers, which shrinks weight memory and bus traffic (D_{vol}) by two to four times and lets

integer units execute each operation at higher peak rate (R_{peak}) and lower energy. In the bottom tier, efficient hardware implementation removes wasted execution. Kernel fusion keeps intermediate activations in on-chip memory instead of sending them back to DRAM (shrinking D_{vol}) and reduces launch overhead (L_{lat}), while sparsity pays off only when its zeros follow a pattern the hardware can skip, such as two zeros in every group of four weights, so that cutting FLOPs/Sample does not inflate Seconds/FLOP.

These three tiers are tightly coupled across the compilation and execution boundary. Pruning alters weight distributions and activation profiles, changing the dynamic ranges required for subsequent quantization-range calibration. Quantization reduces data-movement volume across the memory bus, shifting a previously memory-bound workload toward compute-bound execution where kernel fusion and tile scheduling dominate. Achieving compounding performance gains requires co-designing transformations across all three tiers rather than applying them in isolation, a composition strategy that section 10.5.3 develops through a staged pipeline.

Every technique in this chapter answers to the conservation-of-complexity heuristic that opened Part I. Complexity tends to move among the Data, Algorithm, and Machine axes rather than disappear, so engineers must account for where each technique sends cost and risk. It is a heuristic, not a literal conservation law, and removing genuinely avoidable work can reduce total cost. Pruning may reduce parameters while asking the runtime to exploit sparse structure; distillation may reduce inference cost while adding a teacher-student training phase; quantization may reduce data movement while spending numerical precision. The engineer's task is to move complexity to where the cost is lowest given deployment constraints.

Which tier of the optimization stack yields the highest return depends entirely on the physical machine hosting the workload. A data-center server equipped with several high-bandwidth memory (HBM) accelerators tolerates multi-gigabyte models but demands high token throughput under concurrency. A smartphone or an embedded microcontroller operates under severe physical ceilings on memory capacity, bus bandwidth, and thermal dissipation.

10.1.1 Deployment budgets and the running cases

Cloud serving, a phone camera, and an embedded microcontroller each break down on a different term of the deployment budget, so compression begins by asking which term a given model violates and what it costs to stop violating it. Cloud inference centers on throughput (requests per second per dollar), where supported integer quantization paths increase serving density and kernel fusion reduces per-request kernel launch overhead (Choudhary et al. 2020; Dean et al. 2018). Mobile and edge deployments must fit within tight physical memory bounds while meeting deterministic real-time deadlines. A serialized camera pipeline that must finish each frame before the next arrives at 30 frames per second has about 33 ms per frame. A pipelined design that overlaps capture, inference, and display can sustain 30 frames per second even when each frame takes longer than 33 ms end to end, so it must check frame rate and per-frame latency as two separate budgets.

TinyML makes physical fit a deployment requirement. A microcontroller with only a few hundred kilobytes of SRAM and limited model storage *cannot* accommodate a 100 MB model without a compatible placement strategy, regardless of predictive accuracy (Banbury et al. 2020). Persistent weights may reside in flash or other supported storage, while runtime buffers occupy working memory. Without paging, the peak SRAM working set must fit the available SRAM; the model artifact must separately fit its storage budget. Capacity is also a hard bound on mobile and cloud devices, though their placement options differ.

↩ The latency, memory, and power envelopes defining cloud, edge, mobile, and TinyML tiers are established in section 2.1.

Meeting real-time deadlines on commodity mobile hardware depends as much on the hardware-supported precision path as on absolute memory capacity. When execution units provide hardware acceleration exclusively for fixed-point arithmetic, precision selection dictates whether a workload executes on energy-efficient neural processing units or falls back to power-hungry general-purpose cores. That choice decides the chapter's running case, a camera pipeline that must meet an interactive frame rate within a phone's thermal budget (Example 10.1).

Example 10.1: The running case: Real-time blur on a phone

Scenario: A labeled composite. A camera application deploys a MobileNetV2-based segmentation model for real-time background blur on mid-tier smartphones at 30 FPS.

Diagnosis: An uncompressed FP32 MobileNetV2 model runs on the phone's CPU at only 8 FPS, about 125 ms per frame against a 33 ms budget, while holding the CPU at a power level the thermal envelope cannot sustain. The phone's NPU or DSP accelerates integer matrix arithmetic, not FP32, so the uncompressed model cannot use it.

Systems lesson: INT8 quantization cuts raw parameter payload by 4×, but most of the gain from 8 FPS to 35 FPS comes from what INT8 unlocks, a move from the CPU onto the integer accelerator. The precision change is the enabler; the change of processor supplies most of the speed. The frame rates are illustrative, so speed and power still require device measurement.

The chapter follows that camera pipeline through the structural and hardware tiers of the stack. MobileNetV2 is the book's compute-bound lighthouse in its efficient regime. Depthwise separable convolutions keep its arithmetic small, and at batch one on a phone its frame time goes to arithmetic and activation traffic rather than to reading weights. The precision tier needs a workload whose time is set by bytes, so it follows GPT-2 XL decoding on the same phone. GPT-2 XL is the book's bandwidth-bound lighthouse, and every token it generates streams every weight. Two callouts mark the ends of the memory constraint. DLRM (Naumov et al. 2019), the capacity-bound lighthouse, shows precision deciding whether embedding tables fit, and keyword spotting (KWS) (Y. Zhang et al. 2017), the power-bound lighthouse, shows precision deciding whether a model fits a microcontroller's SRAM and energy budget. ResNet-50, the compute-bound lighthouse, serves twice, as the teacher that distills into MobileNetV2 and as the subject of the closing INT8 deployment profile. Rather than running every technique on all five, the chapter returns to each workload where it exposes the binding resource most clearly.

Table 10.1 places the cases' weight bytes and work beside the deployment budget, and two of its rows explain how the chapter divides the work. MobileNetV2's FP32 weights stream across the phone's 51.2 GB/s memory bus in about 0.27 ms, roughly 1 percent of the 33 ms frame budget, so the frame is spent on its 600 MFLOP of arithmetic and on activation traffic, and the structural and execution tiers are the levers that matter. GPT-2 XL's FP16 weights fit in the phone's memory but take about 59 ms to stream for every generated token, so precision, which shrinks the bytes behind each weight, is the lever that matters. Even MobileNetV2 at INT8 exceeds the TinyML envelope by about 6.7×, while the purpose-built depthwise separable convolutional network (DS-CNN) keyword spotter fits it after INT8 quantization.

↩ The iron law of ML systems, which splits execution time into data-movement, compute, and fixed-overhead terms, is established in section 1.7.

Table 10.1: Deployment Budget for the Running Cases: Weight memory, work per result, and the time a flagship phone's 51.2 GB/s memory bus needs to read the weights once, against the phone and TinyML memory envelopes. Work counts two FLOPs per multiply-accumulate; GPT-2 XL's work and streaming time are per generated token, and MobileNetV2 must finish each frame within the 33 ms budget of a 30 FPS camera. DLRM represents recommendation-model embedding pressure (Naumov et al. 2019); DS-CNN represents the TinyML keyword-spotting case (Y. Zhang et al. 2017). The accounting covers weights only; activations, the KV cache, workspace, and metadata are excluded, and the DLRM row is a single reference configuration whose production counterparts can be far larger. Numbers in parentheses show how many times the model exceeds device memory.

Model and precision	Weight memory	Work per result	Weight streaming on the phone	Phone (8 GB)	TinyML (~512 KB)
MobileNetV2, FP32	14 MB	600 MFLOP per frame	0.27 ms	fits	no (26.7×)
MobileNetV2, INT8	3.5 MB	600 MFLOP per frame	0.07 ms	fits	no (6.7×)
ResNet-50, FP32 (teacher)	102.4 MB	8.2 GFLOP per image	2 ms	fits	no (195.3×)
GPT-2 XL, FP16	3 GB	3 GFLOP per token	59 ms	fits	no (5722×)
GPT-2 XL, INT4	0.75 GB	3 GFLOP per token	15 ms	fits	no (1430.5×)

Model and precision	Weight memory	Work per result	Weight streaming on the phone	Phone (8 GB)	TinyML (~512 KB)
DS-CNN (KWS), INT8	200 KB	20 MFLOP per inference	runs on a microcontroller	fits	fits
DLRM (reference)	100 GB	lookup-dominated	does not fit	no (11.6×)	no (190734.9×)

The chapter descends the optimization stack one tier at a time (table 10.2). Structural optimization changes what the model computes and carries the camera pipeline through pruning, low-rank factorization, distillation from a ResNet-50 teacher, efficient architecture design, architecture search, and input-dependent computation. Numerical precision changes how each surviving value is stored and carries GPT-2 XL decoding from numerical formats and their energy, through the quantization methods, to the conditions under which fewer bits run faster. Hardware implementation changes how the remaining operations execute, through the fusions and sparse kernels that compression makes possible and the gap between paper savings and measured ones. Choosing and combining techniques maps a binding constraint to the levers that relieve it, sequences them on the camera pipeline, and turns to the toolchains that apply them. Measuring compression gains closes the chapter on a ResNet-50 INT8 deployment profile, which chapter 11 takes up. The descent begins at the top of the stack, with the structure of the model itself.

Table 10.2: Chapter Map: Each section of the chapter works one tier of the optimization stack, from what the model computes down to how it executes, and names the iron law term that tier most affects and the condition under which its paper savings become measured ones.

Section	Tier of the stack	What changes	Iron law term most affected	Condition for a measured gain
Structural Optimization	Model representation	Which weights, channels, layers, and paths exist	Operations O, and weight bytes in D_{vol}	Removed structure leaves smaller dense tensors or a supported sparse format
Numerical Precision	Numerics representation	Bits per weight and activation	Bytes moved D_{vol}, and energy per operation	The target has a kernel path for the format
Hardware Implementation	Hardware implementation	Which kernels run, and what they skip	D_{vol}, fixed overhead L_{lat}, and efficiency η_{hw}	The compiler can fuse the pattern and the kernel can skip the zeros
Choosing and Combining Techniques	All three tiers	Which levers apply, and in what order	Whichever term binds	The binding resource is identified before any lever is applied
Measuring Compression Gains	The deployed system	Nothing in the model; the claims are measured	All terms, end to end	Measured on the target hardware against quality limits

10.2 Structural Optimization: Changing What the Model Computes

A standard ResNet-50 trained on ImageNet contains over 25 million parameters, yet with careful magnitude pruning and retraining about 80 percent of its weights can be removed for roughly a one-point loss in top-1 accuracy (Gale et al. 2019). Modern neural networks routinely carry far more parameters than deployment requires.[1] Unused capacity still consumes memory, computation, and energy. Structural optimization is the first tier of the stack, **efficient model representation**, by modifying *what* the model computes. Although additional capacity can aid optimization during training, parameters that do not improve the deployed model still impose deployment cost. The job is to remove capacity the task does not need without removing what it does need, and operating along the Pareto frontier[2] means deciding where the conservation-of-complexity heuristic sends the cost, whether into training-time discovery, runtime sparse indexing, or student-model capacity.

These techniques address the challenge through complementary structural mechanisms. Pruning eliminates redundant parameters from an existing network, and low-rank factorization stores an existing weight matrix as a product of two thin ones. Knowledge distillation transfers learned

[1] **Overparameterization**: C. Zhang et al. (2017) demonstrated that networks large enough to fit ImageNet can also memorize completely random labels, showing that training capacity can exceed the structure needed for natural labels. Pruning studies then show the deployment consequence, that trained models often contain many parameters that can be removed or sparsified with modest task loss when pruning and fine-tuning are done carefully (Gale et al. 2019; Blalock et al. 2020). The redundancy is not a universal 10× constant; it depends on architecture, dataset, sparsity pattern, and runtime support.

[2] **Pareto frontier in compression**: EfficientNet traces the frontier concretely, from B0 (77.1 percent accuracy, 390 million FLOPs) to B7 (84.4 percent, 37 billion FLOPs), a 95× compute increase for 7.3 percentage points of accuracy, quantifying how steep the trade-off becomes at the frontier's edge (Tan and Le 2019).

behavior into an independently designed compact network, efficient architecture design and neural architecture search build a network that is efficient on the target by construction (Hutter et al. 2019), and adaptive computation decides for each input how much of the network runs. In production pipelines, these techniques frequently compose, as when a NAS-discovered architecture, supervised by a distilled teacher, is pruned before final quantization.

The camera pipeline gives every structural technique the same test. MobileNetV2 must finish a frame within 33 ms on the phone, and its weights are not what fills that budget, so a structural change pays off only if it removes multiply-accumulates or activation traffic that the phone actually executes. The analysis below evaluates what structural artifact each method produces, what fine-tuning or recovery step it requires, and which hardware execution conditions allow it to deliver physical speedups. Pruning comes first because it exposes the systems constraint most directly. Eliminating mathematical parameters translates to physical hardware acceleration only when the resulting sparsity pattern aligns with the memory hierarchy and compute units of the target processor.

10.2.1 Pruning

For the camera pipeline, pruning[3] is the first lever to try because it starts from the trained MobileNetV2 rather than from a new design. Removing whole channels from its convolutions shrinks every layer that consumes them and cuts the multiply-accumulates that fill the 33 ms frame budget. Removing scattered individual weights, by contrast, leaves the same layers in place and saves bytes or time only if the runtime stores the zeros compactly and skips them. The safe removable fraction depends on the architecture, task, pruning pattern, and recovery procedure, and every pruning design turns on three choices: *what* to prune (individual weights vs. entire structures), *how* to score what is expendable (magnitude, gradients, or activations), and *when* to prune (after training, during training, or at initialization).

> **Definition 10.2: Pruning**
>
> ***Pruning*** is a model-compression technique that sparsifies the parameter space by removing the weights or structures whose removal increases the training loss least.
>
> 1. **Significance**: It can convert dense matrices into sparse structures, reducing memory footprint and total data volume (D_{vol}) when the sparse representation, metadata, and recovery procedure preserve acceptable quality.
> 2. **Distinction**: Unlike quantization, which reduces the precision of every weight, pruning reduces the count of weights by identifying and eliminating redundancy.
> 3. **Common pitfall**: A frequent misconception is that pruning "automatically" speeds up execution. In reality, without specialized sparse execution support, the resulting sparse matrices may actually run slower than dense ones due to irregular memory access patterns. A speedup requires a storage format and a kernel that skip the zeros, not merely a faster processor.

The formal pruning objective seeks a sparse parameter tensor $\hat{\mathbf{W}}$ that minimizes empirical loss subject to a parameter budget k_{nz}:

$$\min_{\hat{\mathbf{W}}} \mathcal{L}(\hat{\mathbf{W}}) \quad \text{subject to} \quad \|\hat{\mathbf{W}}\|_0 \leq k_{\text{nz}}$$

where $\|\hat{\mathbf{W}}\|_0$ is the ℓ_0 **norm** (the count of nonzero parameters). Solving this cardinality-constrained optimization is combinatorial, so practical methods use heuristics[4] such as **magnitude-based pruning**. The thresholding heuristic keeps the largest weights and zeroes the rest (listing 10.1). Applied to an 11×11 weight tensor, it collapses sub-threshold entries to zero while irregular, high-magnitude weights persist across the grid (figure 10.3).

[3] **Optimal Brain Damage**: Introduced by LeCun, Denker, et al. (1989), the method achieved 4× parameter reduction, with proportional memory savings, in a handwriting recognizer by using a diagonal approximation to second-derivative (Hessian) information to estimate the loss increase from removing each weight. A full Hessian has $\mathcal{O}(P^2)$ entries for P parameters, but Optimal Brain Damage avoids storing that full matrix by approximating its diagonal. Modern-scale pruning still favors cheaper criteria such as magnitude because even diagonal curvature estimation adds training cost.

[4] **Heuristic**: From Greek *heuriskein* (to discover), the same root as Archimedes' "eureka." In pruning, the dominant heuristic (larger magnitude means more important) works well empirically but creates a systems trap. Magnitude-based pruning applied globally can remove most parameters from overparameterized layers while leaving critical bottleneck layers largely intact, giving the appearance of aggressive compression while preserving much of the compute and memory cost in the layers that matter (Blalock et al. 2020). Iterative prune-retrain cycles with per-layer budgets are therefore often safer than naive global magnitude pruning, because each cycle lets the network redistribute importance before the next cut.

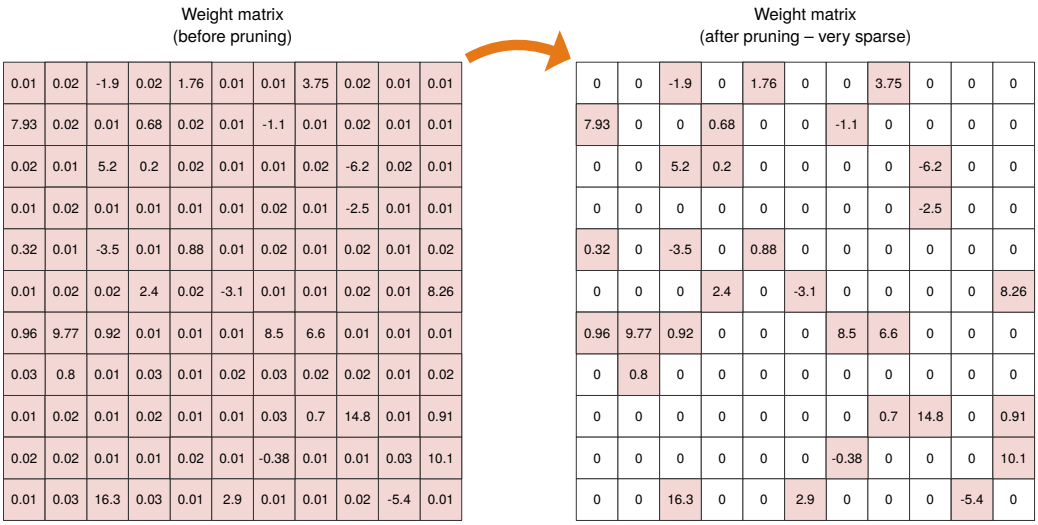
Listing 10.1: Magnitude-Based Pruning: Removes weights below a threshold to create sparse matrices, reducing the number of nonzero parameters from 9 to 4 ($k_{\text{nz}} = 4$).

```
import torch

# Original dense weight matrix
weights = torch.tensor(
    [[0.8, 0.1, -0.7], [0.05, -0.9, 0.03], [-0.6, 0.02, 0.4]]
)

# Simple magnitude-based pruning: keep only the 4 largest weights
threshold = 0.5
mask = torch.abs(weights) >= threshold
pruned_weights = weights * mask

print("Original:", weights)
print("Pruned (4 nonzeros):", pruned_weights)
```



0.01	0.02	-1.9	0.02	1.76	0.01	0.01	3.75	0.02	0.01	0.01
7.93	0.02	0.01	0.68	0.02	0.01	-1.1	0.01	0.02	0.01	0.01
0.02	0.01	5.2	0.2	0.02	0.01	0.01	0.02	-6.2	0.02	0.01
0.01	0.02	0.01	0.01	0.01	0.01	0.02	0.01	-2.5	0.01	0.01
0.32	0.01	-3.5	0.01	0.88	0.01	0.02	0.01	0.02	0.01	0.02
0.01	0.02	0.02	2.4	0.02	-3.1	0.01	0.01	0.02	0.01	8.26
0.96	9.77	0.92	0.01	0.01	0.01	8.5	6.6	0.01	0.01	0.01
0.03	0.8	0.01	0.03	0.01	0.02	0.03	0.02	0.02	0.01	0.02
0.01	0.02	0.01	0.02	0.01	0.01	0.03	0.7	14.8	0.01	0.91
0.02	0.02	0.01	0.01	0.02	0.01	-0.38	0.01	0.01	0.03	10.1
0.01	0.03	16.3	0.03	0.01	2.9	0.01	0.01	0.02	-5.4	0.01

0	0	-1.9	0	1.76	0	0	3.75	0	0	0
7.93	0	0	0.68	0	0	-1.1	0	0	0	0
0	0	5.2	0.2	0	0	0	0	-6.2	0	0
0	0	0	0	0	0	0	0	-2.5	0	0
0.32	0	-3.5	0	0.88	0	0	0	0	0	0
0	0	0	2.4	0	-3.1	0	0	0	0	8.26
0.96	9.77	0.92	0	0	0	8.5	6.6	0	0	0
0	0.8	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0.7	14.8	0	0.91
0	0	0	0	0	0	-0.38	0	0	0	10.1
0	0	16.3	0	0	2.9	0	0	0	-5.4	0

Figure 10.3: Sparse Matrix Transformation: Magnitude-based pruning replaces weights below a numerical threshold with exact zeros (white cells) while preserving salient high-magnitude connections (colored cells). The resulting sparsity cuts parameter count, but realizing hardware speedups requires translating these unstructured zero entries into sparse storage formats or specialized execution kernels.

The resulting sparse matrix retains only the high-magnitude values (colored cells) while near-zero weights become exact zeros. In models amenable to pruning, much of the representational capacity remains intact after removing low-magnitude weights, establishing magnitude thresholding as an effective baseline heuristic.

To make pruning computationally tractable during gradient-based training, practical methods often relax the combinatorial ℓ_0 constraint with continuous penalties such as ℓ_1-regularization ($\lambda_{\text{L1}}\|\mathbf{W}\|_1$), where λ_{L1} scales the sparsity penalty applied to weight tensor $\mathbf{W}$. Penalizing weight magnitudes drives non-essential parameters toward zero, making downstream thresholding easier. Practitioners typically couple this with **iterative pruning**, removing parameters in successive increments interleaved with fine-tuning cycles to recover lost accuracy (Gale et al. 2019; Blalock et al. 2020).

10.2.1.1 Target structures

The choice of what to prune depends on the deployment target's hardware constraints and which resource is the binding bottleneck. When memory capacity is primary, **neuron pruning** can provide direct relief in models whose fully connected layers dominate parameter storage. Removing entire

neurons and their associated weights and biases reduces layer width and parameter count. Profiling must first establish that these layers, rather than embeddings, activations, or another component, are the actual bottleneck.

When convolutional inference latency on commodity accelerators is the bottleneck, **channel pruning** (also called filter pruning) is a strong candidate. Eliminating entire channels or filters reduces feature-map depth and the multiply-accumulate count in subsequent layers. The resulting subnetwork remains dense and regular, so it can map to conventional GPU and Tensor Processing Unit (TPU) kernels without an unstructured sparse format. Realized latency still depends on the resulting dimensions, kernels, and memory traffic.

When a model contains removable stages, **layer pruning** removes entire layers from the network. Each removal eliminates all computation in that stage, but it also reduces representational depth and may change tensor shapes, residual paths, or downstream interfaces. The remaining layers must absorb the lost function, typically through fine-tuning or retraining. The nominal operation saving therefore does not guarantee proportional latency, because graph rewrites, kernel shapes, and memory traffic still govern execution. Layer pruning demands careful task validation and end-to-end profiling. Side by side, the two granularities differ in what they remove. An individual filter slice contracts intermediate feature depth, while an excised layer collapses a full computational stage (figure 10.4).

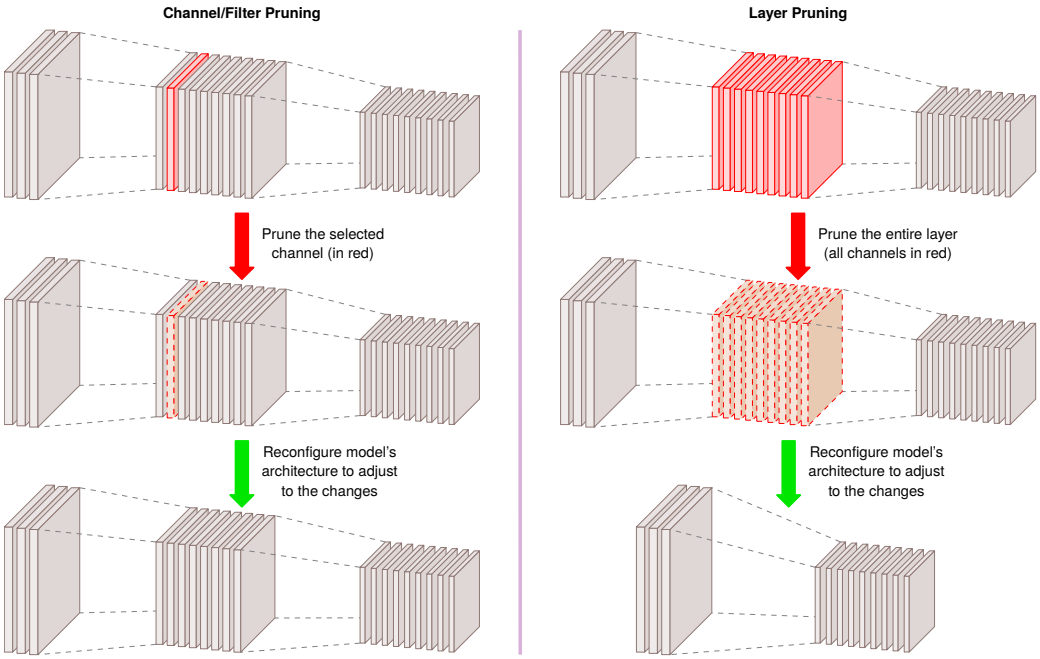


Figure 10.4: Channel vs. Layer Pruning: Structural pruning operates at different architectural granularities. Channel pruning (left) removes feature map slices within convolutional layers while preserving overall network topology and dense execution, whereas layer pruning (right) eliminates entire computational stages, requiring graph rewiring and risking greater capacity loss to achieve larger per-operation savings.

Channel and layer pruning place different demands on accelerator hardware. Channel pruning contracts the inner dimensions of dense general matrix multiply (GEMM) operations, because eliminating output channels in layer ℓ reduces the input channel dimension in layer $\ell + 1$. The tensors remain dense and avoid sparse pointer lookups, but their dimensions still decide how well single instruction, multiple data (SIMD) lanes and Tensor Core tiles fill. Pruning a 256-channel layer to 250 channels, for example, removes about 2 percent of the work yet can leave a matrix unit that processes channels in groups of 8 running a partly empty final tile, so the layer may run no faster. Pruning to 248 keeps every tile full. The camera pipeline's MobileNetV2 shows the same rule in its design, since its reference implementation rounds every channel count to a multiple of 8,

and a pruning policy for the phone keeps the pruned counts on that grid. The preferred multiple depends on the kernel and numerical format, and because it counts elements rather than bytes, it is a different constraint from cache-line or memory-transaction alignment. Layer pruning removes full computational stages and their associated launches and intermediates, but the rewritten graph must still satisfy shape, residual-connection, and kernel-layout requirements. Both approaches require quality validation.

10.2.1.2 Unstructured pruning

Unstructured pruning removes individual weights while preserving the overall network architecture. Some connections become redundant during training, contributing little to the final output. Pruning these weak connections reduces the nonzero count and can preserve most task quality after recovery.

Formalizing this process, let $\mathbf{W} \in \mathbb{R}^{m \times n}$ represent a weight matrix in a given layer. Pruning removes a subset of weights by applying a binary mask $\mathbf{M} \in \{0,1\}^{m \times n}$, yielding a pruned weight matrix:

$$\hat{\mathbf{W}} = \mathbf{M} \odot \mathbf{W}$$

where $\odot$ represents the element-wise Hadamard product. The mask $\mathbf{M}$ is constructed based on a pruning criterion, typically weight magnitude. A common approach is magnitude-based pruning, which removes a fraction ρ_{sparse} of the lowest-magnitude weights by defining a threshold δ_{prune} such that:

$$M_{i,j} = \begin{cases} 1, & \text{if } |W_{i,j}| > \delta_{\text{prune}} \\ 0, & \text{otherwise} \end{cases}$$

where δ_{prune} is chosen to ensure that only the largest $(1 - \rho_{\text{sparse}})$ fraction of weights remain. This method assumes that larger-magnitude weights contribute more to the network's function, making them preferable for retention.

Unstructured pruning reduces the count of nonzero parameters, but translating that reduction into physical storage savings requires an indexed sparse encoding such as Compressed Sparse Row (CSR) or coordinate list (COO), whose positional metadata can cancel the saving at low sparsity. Storing a dense tensor with zeroed entries saves no memory at all. Where the indexed formats break even in bytes and in time is a question about execution rather than about which weights to remove, and section 10.4.2.2 answers it.

10.2.1.3 Structured pruning

Where unstructured pruning removes individual weights across arbitrary tensor coordinates, **structured pruning** (Li et al. 2017) eliminates regular groups of parameters: channels, filters, attention heads, or entire layers. Because removing whole structural units contracts weight arrays into smaller, contiguous dense tensors, the remaining computation executes directly on standard Basic Linear Algebra Subprograms (BLAS) and convolution engines without specialized sparse libraries or metadata indexing overhead.

Between unconstrained irregular masks and coarse-grained channel excision lies hardware-aligned fine-grained regularity. Hardware-aware pruning strategies, such as *N:M structured sparsity*,[5] enforce fine-grained patterns that map directly onto accelerator execution datapaths. Under a *2:4 structured sparsity* constraint, every contiguous block of four weights must contain exactly two zeros and two nonzero values:

Original block (4 values): $[0.0, 1.4, 0.0, -2.1] \longrightarrow$ Compressed storage: $[1.4, -2.1]$ with 2-bit indices: $[01_2, 11$

Because every group of four weights compresses to exactly two values, the value payload halves, and a 2-bit index per kept value adds only a small, fixed metadata cost with no structural irregularity. This chapter uses the 2:4 pattern as the canonical compression example, and section 10.4.2 returns to what a sparse matrix unit does with it.

The regularity-compression trade-off spans three architectural granularities, from irregular connection masking to coarse-grained neuron dropping and full convolutional channel excision (figure 10.5). Moving from left to right trades per-weight freedom for dense tensor structure that standard hardware kernels can execute directly.

[5] **N:M structured sparsity**: Introduced commercially with NVIDIA's A100 GPU (2020), the 2:4 pattern was chosen because it halves multiply-accumulate operations while keeping position metadata small enough for the sparse Tensor Core path (NVIDIA Corporation 2020a; Choquette et al. 2021). This fixed ratio is a hardware constraint, not a mathematical optimum. The A100 Sparse Tensor Core path accelerates 2:4 sparse operands, yielding up to 2× math-throughput speedup over dense execution when kernels and layouts satisfy the constraint. Other ratios are not accelerated by this specific hardware path, illustrating how silicon design constrains which sparsity patterns translate to actual speedup.

↪ How sparse Tensor Cores use the 2-bit metadata to select matching activations and skip the zeroed multiplies is developed in section 11.4.5.3.

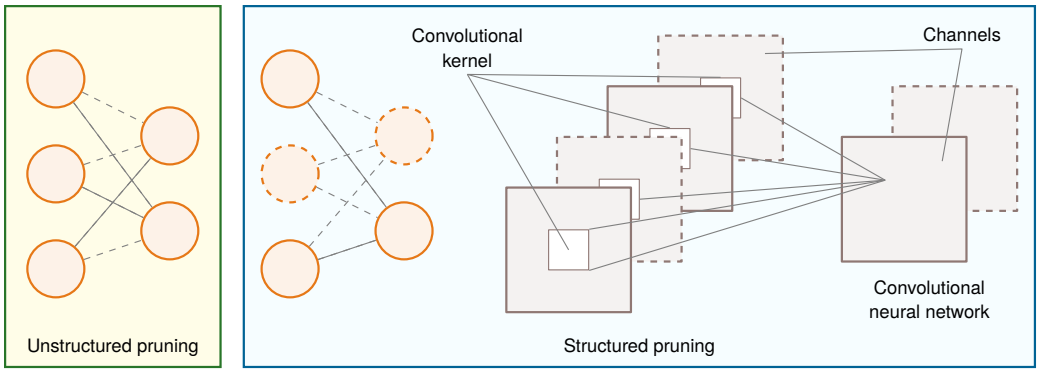


Figure 10.5: Unstructured vs. Structured Pruning: The regularity-compression trade-off across pruning granularities. Unstructured pruning (left) removes arbitrary individual connections and requires a sparse representation and matching kernel to change execution. Structured pruning of neurons (middle) or convolutional channels (right) can produce smaller dense tensors that remain compatible with standard kernels, although realized latency depends on the resulting shapes and runtime. Source: (Qi et al. 2021).

In fully connected layers (center of figure 10.5), pruning neurons eliminates the corresponding rows and columns in adjacent weight matrices. In convolutional layers (right), pruning channels removes complete 3D filter slices. Either way the subnetwork stays dense, so the remaining systems requirement is matching the resulting tensor shapes and layouts to efficient kernels, the tile-filling condition of section 10.2.1.1.

A common approach to structured pruning ranks entire neurons or filters by the magnitude of their associated weights. A low group norm is an inexpensive proxy for low importance, not proof that the unit is dispensable. The score can use an ℓ_1-norm or ℓ_2-norm over the weights associated with each unit; groups below a selected threshold become pruning candidates. Layer-wise ranking matters because raw scales can differ across layers. The method needs no representative data pass, but computing scores, choosing a threshold, materializing the smaller graph, and validating or fine-tuning the result still incur engineering and compute cost.

Another strategy is **activation-based pruning**, which evaluates neuron or filter activations over a dataset. Units that remain weak across representative inputs become candidates for removal, subject to ablation or fine-tuning checks. This method captures input-dependent behavior rather than relying only on static weights. Its ranking can still miss rare but important inputs, depend on activation scaling, or change after upstream units are removed. Activation-based pruning therefore requires representative profiling data and validation after the selected structures are removed.

Gradient-based pruning uses loss derivatives to estimate how removing a neuron or filter would change the objective. A small first-order score suggests a candidate at the current checkpoint and on the sampled data; it does not establish permanent irrelevance or capture every interaction among units. Gradient-based criteria can combine a unit's value with its gradient to rank the expected local loss change. Mathematically, this criterion approximates the change in loss $\Delta\mathcal{L}$ when zeroing parameter w_i via a first-order Taylor expansion: $\Delta\mathcal{L} \approx |w_i \cdot \frac{\partial\mathcal{L}}{\partial w_i}|$. Weights that are either near zero or have negligible gradients exert minimal influence on the objective, making them prime candidates for excision. Unlike weight-only magnitude criteria, they require backward computation and are usually integrated into training or fine-tuning. The pruned result still needs task-level validation because the score is a local approximation.

These criteria trade measurement cost against fidelity to model behavior. Magnitude scores are inexpensive and provide a useful baseline, but they ignore the input distribution. Activation scores incorporate representative inputs and expose units that are quiet on that workload, while gradient-based scores also incorporate the local loss surface at additional training cost. None dominates across models, and none establishes that removal is harmless. A sound comparison prunes to the same structural target, applies the same recovery budget, and measures task quality after removal. It then profiles the exported graph, because a better importance score can preserve quality without producing shapes that the target runtime executes efficiently. The choice therefore depends on available data and compute, the structure being removed, and the deployment's quality margin.

10.2.1.4 Pruning trade-offs

The three pruning approaches occupy different positions on the regularity-versus-granularity trade-off. Unstructured pruning can remove individual weights and therefore express fine-grained sparsity, but accelerators need matching sparse formats and kernels to skip those zeros. Structured pruning removes channels, filters, or layers and can produce smaller dense operations that conventional hardware already supports. Dynamic pruning, which section 10.2.6 develops with the other input-dependent methods, makes the executed structure depend on each input, adding control and batching overhead in exchange for conditional allocation. The three differ in what they remove, what the runtime must support, and what recovery they need (table 10.3).

These categories describe deployed execution rather than mutually exclusive training algorithms. A structured subnetwork may be learned with a dynamic schedule and then frozen for serving; classify the artifact by what its runtime actually executes.

Table 10.3: Pruning Strategies: Unstructured pruning removes individual weights but needs sparse storage and matching kernels for deployment gains; structured pruning removes whole components that map more directly to dense hardware, while dynamic pruning adds runtime adaptation and overhead.

Aspect	Unstructured Pruning	Structured Pruning	Dynamic Pruning
Removed unit	Individual weights in the model	Entire neurons, channels, filters, or layers	Adjusts pruning based on runtime conditions
Model structure	Sparse weight matrices; original architecture remains unchanged	Model architecture is modified; pruned layers are fully removed	Structure adapts dynamically
Impact on memory	Reduces the nonzero count; storage falls only with an appropriate sparse representation	Reduces model storage by removing entire components	Varies based on real-time pruning
Impact on computation	Dense work remains unless a supported sparse path skips zeros	Reduces nominal FLOPs; latency depends on resulting shapes and kernels	Varies with routing policy and controller overhead
Hardware compatibility	Requires sparse formats and matching execution support	Produces dense subnetworks, but awkward shapes can still underutilize hardware	Requires adaptive inference engines
Fine-tuning requirement	Often used to recover accuracy after pruning	Often used after structural modification	Depends on how the routing policy and subnetworks are trained
Use cases	Targets whose runtime provides sparse formats and kernels, or storage-bound high sparsity	Real-time inference optimization, mobile/edge AI, and efficient training	Adaptive AI applications, real-time systems

10.2.1.5 Pruning schedules

The pruning schedule affects how much recovery training the network receives after capacity is removed. Two primary scheduling strategies govern weight removal: iterative pruning and one-shot pruning.

Iterative pruning Iterative pruning removes capacity gradually, alternating each structural cut with fine-tuning so that the network recovers before the next cut. Across three successive prune-and-retrain cycles, accuracy drops after each cut and recovers during the fine-tuning that follows (figure 10.6).

In the illustrative workflow, a convolutional network loses six channels with recovery between successive cuts. Rather than removing all six channels in a single pass, the pipeline removes two channels per cycle across three iterations, fine-tuning the remaining parameters after each cut. The initial cut drops accuracy from 0.995 to 0.971, but brief fine-tuning restores performance to 0.992. Two subsequent prune-and-tune cycles produce a final model at 0.991 accuracy, a 0.4 percent degradation despite a 27 percent reduction in channel count. Staging structural surgery across multiple retraining intervals allows surviving weights to adjust their representations along the loss landscape.

One-shot pruning **One-shot pruning** removes multiple architectural components in a single step, followed by an extensive fine-tuning phase to recover model accuracy. This aggressive approach

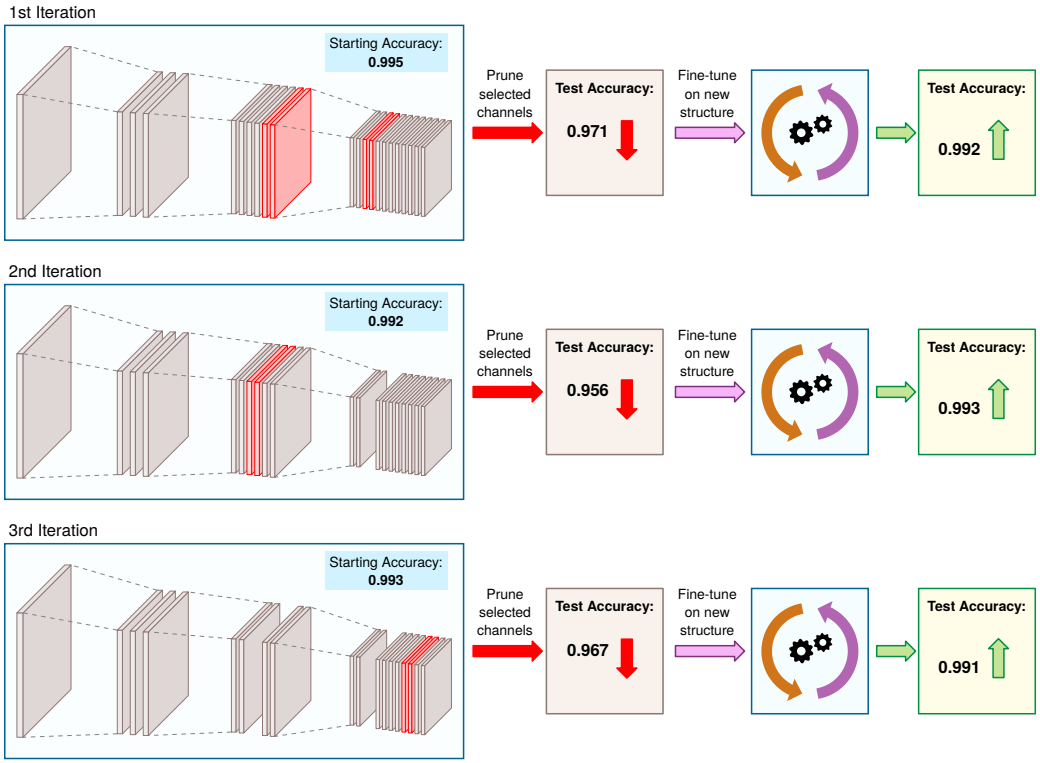


Figure 10.6: Illustrative Iterative Pruning Performance: Three rows depict successive prune-then-fine-tune cycles, each removing two of the original twenty-two channels. Accuracy drops from 0.995 to 0.971 after the first prune, recovers to 0.992 after fine-tuning, and settles at 0.991 after all three cycles, a 0.4 percent loss with 27 percent fewer channels.

compresses the model quickly but risks greater accuracy degradation, as the network must adapt to significant structural changes simultaneously.

Applying one-shot pruning to the same network illustrates the hazard of aggressive structural truncation. Removing all six highlighted channels at once (figure 10.7) causes a sharp accuracy drop from 0.995 to 0.914, from which fine-tuning recovers only partially to 0.943, leaving about a 5 percent quality penalty under the recovery procedure shown. The two figures illustrate a mechanism, not a controlled result. Both runs end with the same tensor geometry, but the iterative run also received three rounds of fine-tuning, so its better final accuracy mixes the benefit of smaller cuts with the benefit of more recovery training. A fair comparison holds the final structure and the total recovery budget fixed.

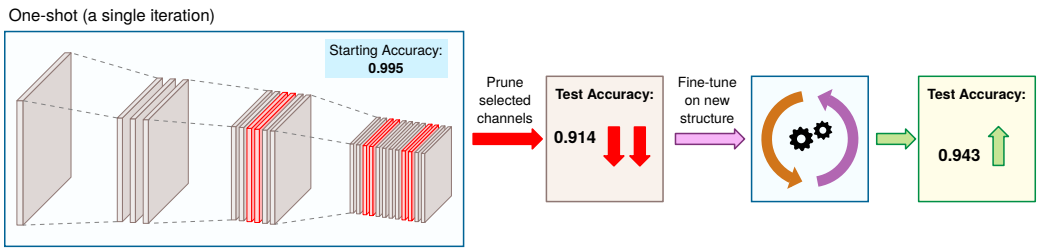


Figure 10.7: Illustrative One-Shot Pruning Impact: All six channels (27 percent) are removed simultaneously, causing accuracy to drop from 0.995 to 0.914. Fine-tuning recovers only to 0.943, a 5 percent degradation from the original accuracy.

The working rule follows from that mechanism. Start with one-shot pruning plus recovery, because it is cheapest, and accept it if the recovered model meets the quality target. Move to an iterative

schedule when it does not, which is most likely at high sparsity, where a single large cut removes capacity the network cannot rebuild in one recovery phase. Compare the two at a matched recovery budget, and spend the extra cycles only when the target accelerator can exploit the sparsity pattern that results.

10.2.1.6 Lottery ticket hypothesis (optional depth)

Standard pruning workflows assume a fully trained dense model from which weights are subsequently removed. Yet the relationship between network structure and optimization dynamics runs deeper, because pruning can reveal inherently efficient, highly trainable subnetworks embedded within the initial random parameterization.

This perspective leads to the Lottery Ticket Hypothesis[6] (LTH), which challenges conventional pruning workflows by proposing that within large neural networks, there exist small, well-initialized subnetworks ("winning tickets") that can achieve comparable accuracy to the full model when trained in isolation. Rather than viewing pruning as a post-training compression step, LTH suggests it can serve as a discovery mechanism to identify these efficient subnetworks early in training (Rachwan et al. 2022).

The discovery procedure is an iterative pruning-and-rewinding loop. After training to convergence and pruning low-magnitude connections, the surviving weights reset to their initialization state θ_0, which isolates a sparse subnetwork (figure 10.8). Repeating the cycle can isolate a sparse subnetwork that, in the reported settings, trains to accuracy comparable with the original network (Frankle and Carbin 2019). Whether the same procedure succeeds depends on architecture, scale, optimizer, and training budget.

[6] **Lottery ticket hypothesis**: Named for the intuition that training a large network is like buying many lottery tickets, most of which lose, while a few "winning tickets" (sparse subnetworks with favorable initializations) can train to comparable accuracy on their own. Frankle and Carbin (2019) established the hypothesis on smaller vision and fully connected networks; later work surveyed and extended the idea to larger settings (Rachwan et al. 2022). The systems implication is that some of the memory and compute spent training dense networks may be discoverable overhead, but the practical payoff depends on whether the winning subnetwork can be found before paying most of the original training cost.

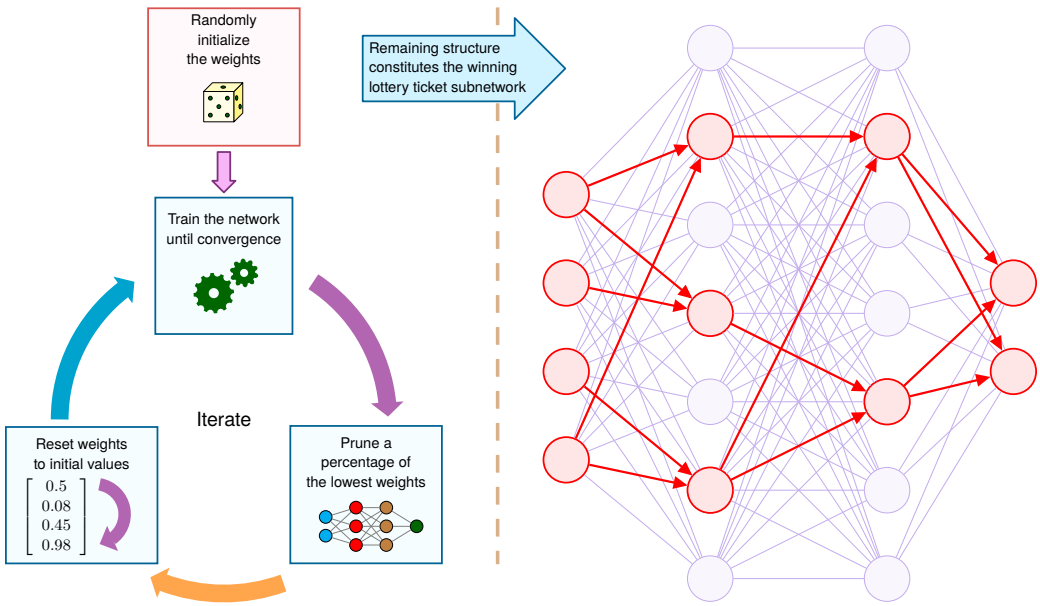


Figure 10.8: Lottery Ticket Iteration Cycle: The original procedure repeatedly trains, prunes low-magnitude weights, and resets surviving weights to their initial state θ_0. The cycle tests whether the resulting sparse subnetwork can match the dense baseline when trained in isolation; the outcome is empirical rather than guaranteed.

The Lottery Ticket Hypothesis suggests that compact, trainable subnetworks may exist inside an overparameterized initialization, which motivates direct sparse-training research. The standard discovery procedure, however, does not eliminate dense training cost. It first pays for one or more dense or partially dense runs, plus a rewind and a recovery phase per pruning cycle, to find the ticket. A winning ticket is therefore a training result. It becomes a systems win only when that total accelerator time, the final artifact size, target latency, and task quality beat the simpler alternatives: training a smaller dense model from the start, one-shot pruning with recovery, and direct sparse training.

10.2.1.7 Pruning in practice

While the Lottery Ticket Hypothesis illuminates the optimization landscape, practical serving systems care only about executable artifacts. Framework-level pruning is useful only when it materializes concrete hardware savings: a smaller dense tensor, a hardware-aligned structured sparse matrix, or an indexed sparse layout paired with matching runtime kernels. Training-time pruning often begins as mask application. The original tensor remains resident in memory, while a binary mask zeros selected weights during forward propagation. That mechanism guides gradient flow during recovery, but it delivers zero memory reduction or latency improvement on its own. Measurable efficiency gains emerge only after the masked structure is exported into the physical artifact loaded by the inference engine.

This artifact boundary is where the unstructured and structured strategies part ways. An exported unstructured mask still needs a sparse storage format and a matching kernel, whereas exported structured pruning is already a smaller dense graph. The systems audit therefore has three concrete steps. Identify what was pruned, identify the format the runtime receives, and verify that the target hardware has kernels that make the sparsity useful.

In practice, BERT-style transformers[7] have been pruned by removing redundant attention heads or intermediate dimensions, while separate distillation methods such as DistilBERT and TinyBERT train smaller dense student models that retain much of BERT's performance (Sanh et al. 2019; Jiao et al. 2020). Pruning decides which weights exist, and every weight it keeps is still stored and multiplied at full size. A second way to shrink an existing layer keeps every weight position but stores the matrix as a product of two thin matrices, which suits layers whose weights carry less information than their dimensions suggest.

↪ Benchmark protocols that separate structured from unstructured pruning and specify the hardware and sparse-kernel support behind any measured speedup are developed in section 12.9.1.3.

7 **BERT pruning**: BERT's 12 attention heads per layer can contain substantial redundancy. Michel et al. (2019) found on MultiNLI that many heads could be removed without a statistically significant performance change. The removable fraction depends on the layer and task, so deployment pruning must measure both task quality and whether the resulting structure reduces runtime work.

10.2.2 Low-rank factorization

Fetching and multiplying a full 4096×4096 weight block from DRAM spends 93.75 percent of its memory traffic and arithmetic on directions that barely change the layer's output when a rank-128 subspace already captures almost all of the transformation's variance. Deep network weight matrices often show this rapid singular-value decay, and **structured approximation** exploits it by decomposing dense $M \times N$ weight tensors into products of low-rank matrices or tensor factors that require only a fraction of the original scalar parameters (6.25 percent in the rank-128 case). Low-rank matrix factorization and multi-linear tensor decompositions translate this algebraic redundancy into reduced memory traffic and lower arithmetic cost. In the camera pipeline, the candidates are MobileNetV2's pointwise convolutions, which are matrix multiplications over channels, and factoring one trades fewer multiply-accumulates against an extra kernel and an intermediate tensor, the trade-off developed below.

10.2.2.1 Matrix factorization

Low-rank matrix factorization (LRMF) approximates weight matrices with lower-rank representations. Given a weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$, LRMF identifies factor matrices $\mathbf{U} \in \mathbb{R}^{m \times k}$ and $\mathbf{V} \in \mathbb{R}^{n \times k}$ such that:

$$\mathbf{W} \approx \mathbf{U}\mathbf{V}^T$$

where the approximation rank $k \ll \min(m, n)$. This decomposition is typically computed via singular value decomposition (SVD),[8] retaining only the k largest singular values. Writing the truncated decomposition as $\mathbf{W} \approx \mathbf{P}_{kk}\mathbf{Q}_k^T$ and splitting the singular values evenly between the two factors gives $\mathbf{U} = \mathbf{P}_{kk}^{1/2}$ and $\mathbf{V} = \mathbf{Q}_{kk}^{1/2}$. During inference, computing the layer output as $\mathbf{h} = \mathbf{U}(\mathbf{V}^T\mathbf{x})$ replaces a single large matrix-vector product with two sequential, smaller multiplications. The primary systems question is whether the reduction in parameter footprint and arithmetic operations compensates for the overhead of staging an intermediate activation and dispatching an additional kernel.

8 **Singular value decomposition (SVD)**: The Eckart-Young theorem (1936) proves that retaining the top k singular values yields an optimal rank-k approximation in the Frobenius and spectral norms. The key systems trade-off is whether the high, one-time compute cost of this factorization, $\mathcal{O}(mn \cdot \min(m, n))$, is amortized by the memory and bandwidth savings from using the smaller model in repeated inference calls.

> Systems Perspective 10.1: The bandwidth-compute trade-off
>
> Low-rank factorization can reduce both arithmetic and bandwidth demand when the retained rank is small, though it adds a second factor operation and possible kernel-launch overhead.

Storing a 4096 by 4096 matrix requires 67.1 MB (at FP32). Fetching this matrix for a single inference imposes substantial memory traffic, especially when execution is limited by physical memory bandwidth.

Factoring the matrix at rank $k = 128$ requires storing two smaller matrices (4096 by 128 and 128 by 4096), totaling only 4.2 MB, a 16× reduction in stored values. When inference uses the two factors directly, matrix-vector arithmetic also falls from $\mathcal{O}(mn)$ to $\mathcal{O}(k(m+n))$ for small k. The deployment speeds up only if the two factor operations and their intermediate traffic cost less than the original dense kernel. Explicitly materializing $\mathbf{UV}^T$ would add $\mathcal{O}(mkn)$ work and defeat the purpose.

This bandwidth-compute trade-off exemplifies the memory wall at the operator level. When batch sizes are small, execution is bottlenecked by the time required to fetch weight bytes from DRAM rather than the time required to perform multiply-accumulate operations. In production serving engines, evaluating the factored representation avoids full-rank materialization entirely. The original $m \times n$ weight matrix $\mathbf{W}$ becomes the product of a tall factor $\mathbf{U}$ ($m \times k$) and a wide factor $\mathbf{V}^T$ ($k \times n$) (figure 10.9).

↪ The AI memory wall, the regime in which arithmetic throughput outpaces the memory bandwidth that feeds it, is developed in section 11.5.1.

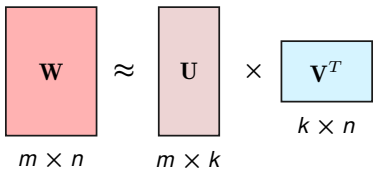


Figure 10.9: Low-Rank Factorization: A weight matrix $\mathbf{W}$ of size $m \times n$ is approximated by two smaller factors, $\mathbf{U}$ ($m \times k$) and $\mathbf{V}^T$ ($k \times n$). Storage falls from $m \times n$ to $m \times k + k \times n$ parameters, while one dense operation becomes two factor operations.

LRMF applies directly to fully connected projection layers and to flattened convolutional weight tensors. The storage and arithmetic savings derived above are only half of the accounting on a real accelerator. The intermediate activation vector $\mathbf{z} = \mathbf{V}^T\mathbf{x}$ must be staged in on-chip SRAM or round-tripped through memory, and dispatching two separate kernels incurs launch latency unless the operations are fused. Furthermore, if the approximation rank k is small or unaligned with hardware tile dimensions (such as 16×16 systolic arrays), the hardware compute units operate well below peak throughput. Selecting rank k therefore balances memory footprint and bandwidth reduction against approximation error, kernel dispatch overhead, and hardware tile utilization.

↪ How a matrix is tiled onto a fixed-size systolic array, and why a dimension just past the tile width drops utilization sharply, is developed in section 11.7.5.

10.2.2.2 Tensor decomposition (optional depth)

Tensor decomposition extends low-rank approximation to multidimensional arrays, such as four-dimensional convolutional kernels ($d_{\text{out}} \times d_{\text{in}} \times K_h \times K_w$) and multi-head attention weights. Matrix factorization could compress such a tensor only after unfolding it into a matrix, which discards which axis was which. Tensor decompositions instead keep each axis separate by representing the tensor through multilinear products of lower-dimensional factors. A three-dimensional tensor $\mathcal{Y} \in \mathbb{R}^{I \times J \times T}$, for example, decomposes into three rank-k factor matrices ($\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$), and outer products of the slice vectors $\mathbf{a}_i$, $\mathbf{b}_j$, and $\mathbf{c}_t$ reconstruct entry y_{ijt} (figure 10.10).

Formats trade parameter reduction against contraction complexity. **CP decomposition** (canonical polyadic decomposition) is the form drawn in figure 10.10. It expresses the tensor as a sum of k rank-one components, $\mathcal{Y} \approx \sum_{s=1}^{k} \mathbf{a}_s \otimes \mathbf{b}_s \otimes \mathbf{c}_s$, where $\mathbf{a}_s$, $\mathbf{b}_s$, and $\mathbf{c}_s$ are column s of $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ (Lebedev et al. 2015), giving maximal parameter reduction but often unstable rank optimization. **Tucker decomposition** keeps a small dense core tensor multiplied along each mode by a factor matrix, $\mathcal{Y} \approx \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C}$, where $\times_{n_{\text{mode}}}$ multiplies along axis n_{mode} of the tensor. **Tensor-train (TT)** stores a many-axis tensor (the order of a tensor is its number of axes) as a chain of small third-order cores, which avoids the exponential growth of Tucker cores with the number of axes.

Tensor decomposition applies to convolutional filters, attention projections, and embedding tables. Its barrier is execution cost, because accelerators are specialized for dense matrix multiplication (GEMM). Contracting decomposed tensors needs transposition kernels or many small contractions,

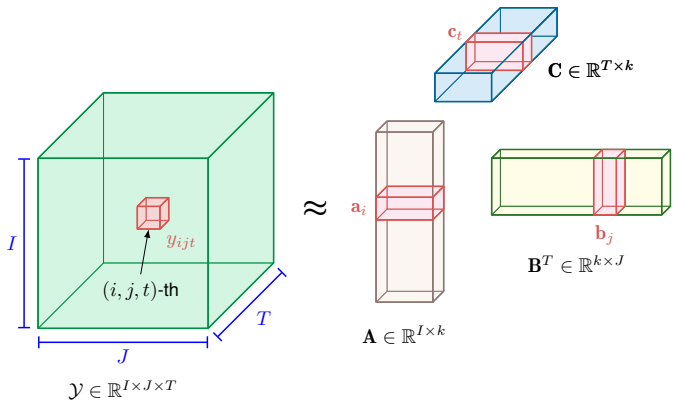


Figure 10.10: Tensor Decomposition: A three-dimensional tensor $\mathcal{Y} \in \mathbb{R}^{I \times J \times T}$ is approximated by factor matrices **A**, **B**, and **C**. The highlighted factor vectors $\mathbf{a}_i$, $\mathbf{b}_j$, and $\mathbf{c}_t$ combine across rank k to reconstruct the entry y_{ijt}. Extending low-rank approximation from two dimensions to three carries the storage and computation argument into convolutional layers. Source: (Gholami et al. 2022).

and without fused contraction kernels the data movement and launch latency can erase the arithmetic savings. LRMF and tensor decomposition differ in the data structures they apply to, the compression they achieve, and what they ask of the execution engine (table 10.4).

Table 10.4: Matrix and Tensor Factorization: LRMF applies to two-dimensional matrices, while tensor decompositions extend low-rank representations to multidimensional tensors; realized storage and latency depend on rank choice and kernel support.

Feature	Low-Rank Matrix Factorization (LRMF)	Tensor Decomposition
Applicable Data Structure	Two-dimensional matrices	Multi-dimensional tensors
Compression Mechanism	Factorizes a matrix into two or more lower-rank matrices	Decomposes a tensor into multiple lower-rank components
Common Methods	SVD; Alternating Least Squares (ALS), which fits one factor at a time	CP Decomposition, Tucker Decomposition, Tensor-Train (TT)
Compute cost	Depends on the factorization method, selected rank, and kernels	Depends on the decomposition, selected ranks, optimization procedure, and tensor-contraction kernels
Storage Reduction	Reduces storage from $\mathcal{O}(mn)$ to $\mathcal{O}(mk+kn)$	Depends on the decomposition and selected ranks; stores factor matrices and, for some methods, a core tensor
Inference execution	Replaces one dense operation with two factor operations	Introduces tensor contractions whose latency depends on available kernels
Primary Use Cases	Fully connected layers, embeddings, recommendation systems	Convolutional filters, attention mechanisms, multi-modal learning
Implementation Complexity	Easier to implement, often involves direct factorization methods	More complex, requiring iterative optimization and rank selection

In production pipelines, practitioners frequently combine these structured approximations, applying LRMF to large linear projections and embedding tables while targeting multi-dimensional convolutional layers with tensor decompositions. Under the D·A·M taxonomy, structured approximation alters the Algorithm to relieve Machine memory-capacity and bandwidth bottlenecks. When fetched weights dominate serving latency, reducing weight bytes can reduce that component of execution time. Added factor operations, intermediate tensors, and launch overhead must still be counted; they can offset the saving in either a memory-bound or compute-bound workload.

Pruning and factorization both modify the parameterization of an existing model, and neither can replace an operator that suits the target poorly. If the original operator choices, kernel dimensions, or memory access patterns are ill-suited to the target accelerator, removing or factoring weights cannot substitute more efficient primitives. When deployment limits demand a different topology altogether, the compression objective shifts from subtracting weights to training a compact model to behave like the original.

10.2.3 Knowledge distillation

The camera pipeline's MobileNetV2 does not have to learn its task from labels alone. A ResNet-50 trained for the same task needs 102.4 MB of FP32 weights and 8.2 GFLOP per image against MobileNetV2's 14 MB and 600 MFLOP (table 10.1), far too much arithmetic for the phone's frame budget, yet its output distribution carries information that labels do not. Knowledge distillation trains a compact *student* network, here MobileNetV2, to reproduce the output behavior and representations of an overparameterized *teacher*, here ResNet-50 (Hinton et al. 2015; Sanh et al. 2019). While the term borrows from chemical purification,[9] the systems mechanism is algorithmic. Distillation transfers dark knowledge across architectures, amortizing a one-time training phase across millions of lower-latency inference queries.

[9] **Distillation**: The name borrows from the chemical process of separating a mixture. In model compression, the student learns from a teacher's softened output distribution or internal representations rather than copying its parameters. Soft targets can encode relationships among classes, but the student learns only what its objective and training data expose. The metaphor is useful but not literal, since the temperature parameter comes from softmax scaling, and the size-quality trade-off depends on the student, data, objective, and training procedure (Hinton et al. 2015). Deployment savings come from the student architecture, not from the metaphor itself.

> **Definition 10.3: Knowledge distillation**
>
> ***Knowledge distillation*** is a model-compression technique that trains a smaller *student* model to match the behavior of a larger, pretrained *teacher* model.
>
> 1. **Significance**: Distillation moves cost from repeated inference to an additional training phase. A student can be substantially smaller than its teacher while retaining useful task performance; DistilBERT, for example, reports 40 percent fewer parameters, 60 percent faster inference, and 97 percent of BERT's language-understanding performance (Sanh et al. 2019). This trade-off is valuable when the student-training cost is amortized across many deployed queries.
> 2. **Distinction**: Unlike pruning, which removes parameters from an existing architecture, and quantization, which lowers numerical precision, distillation trains a new dense architecture. The student inherits behavior from the teacher's output distribution or intermediate representations rather than inheriting the teacher's full parameter count or layer structure.
> 3. **Common pitfall**: A frequent misconception is that distillation is lossless compression. In reality, the student is bounded by its own capacity, the quality of the teacher, and the match between the distillation data and deployment distribution; a student can faithfully reproduce teacher errors as well as teacher knowledge.

Data selection used the teacher's soft targets as training data (section 9.5.4); here they become a compression mechanism. Teacher models transfer semantic knowledge through nonzero probabilities assigned across non-target classes. Whereas a one-hot ground-truth label assigns a probability of one exclusively to the correct class, a teacher's distribution keeps the inter-class correlations known as **dark knowledge**, revealing that an input classified as a cat (85 percent) shares subtle semantic features with a dog (10 percent) and a fox (5 percent) (figure 10.11).

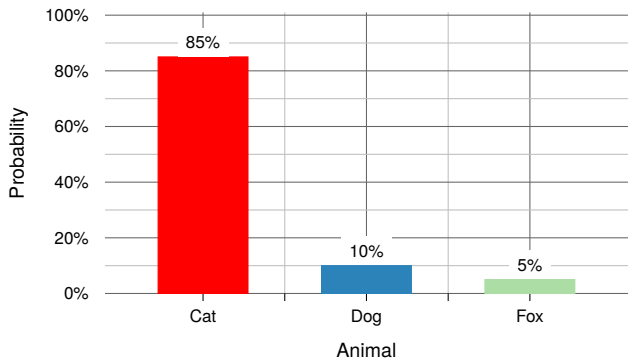


Figure 10.11: Soft Target Distribution: The teacher's relative confidence levels indicate which classes are semantically similar (for example, cat vs. dog), providing a much richer supervision signal than a binary "correct" label.

Training a student model requires balancing teacher imitation against ground-truth task supervision. Operationally, the workflow requires four decisions before training begins: selecting a teacher with the desired behavior, designing a student whose dense architecture satisfies the target accelerator's memory and compute budget, generating soft targets over the training corpus, and choosing the temperature and loss weights that govern optimization. In the resulting dual-path pipeline, an input sample x passes through both the teacher and the student, temperature-scaled soft predictions ($T_{\text{distill}} > 1$) feed the distillation loss, and unscaled student predictions ($T_{\text{distill}} = 1$) are scored against ground-truth hard labels (figure 10.12).

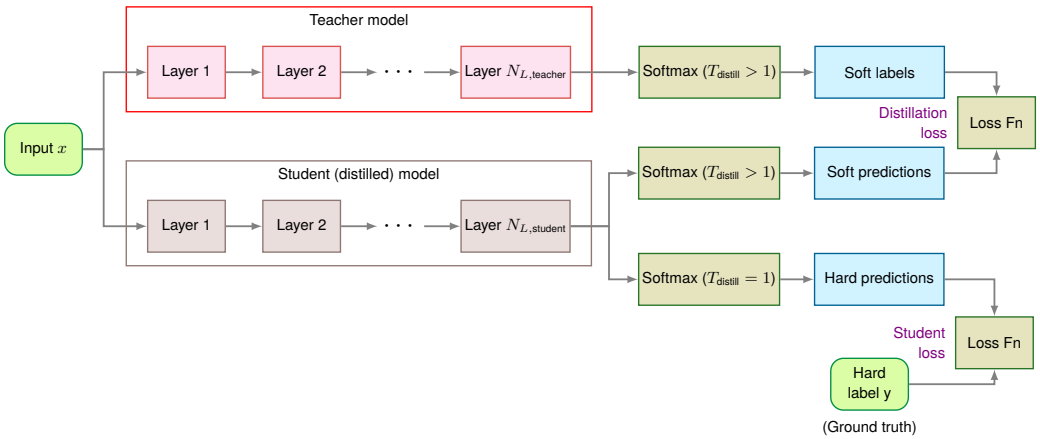


Figure 10.12: Knowledge Distillation Workflow: An input sample passes through both the teacher and the student network. The teacher produces soft labels via temperature-scaled softmax, while the student output is compared against both the soft labels (distillation loss) and the hard labels (student loss).

Deploying the distilled student requires verifying more than top-1 accuracy. Because a student can inherit the teacher's systematic biases and miscalibrations as readily as its useful representations, verification must measure calibration error, subgroup behavior, and executed latency on the target hardware.

10.2.3.1 Distillation mathematics

Standard softmax normalization converts raw model logits z_i into probabilities. Introducing a **temperature parameter**[10] T_{distill} softens the resulting probability distribution across output classes:

$$p_i^{(T_{\text{distill}})} = \frac{\exp(z_i/T_{\text{distill}})}{\sum_j \exp(z_j/T_{\text{distill}})}$$

A higher T_{distill} flattens the softmax distribution and exposes more of the dark knowledge carried in relative class probabilities, such as a teacher assigning a small but nonzero probability to dog and near-zero probability to truck when classifying an image of a cat. The total distillation loss $\mathcal{L}_{\text{distill}}$ balances standard cross-entropy with the Kullback-Leibler (KL) divergence against the teacher's softened distribution:[11]

$$\mathcal{L}_{\text{distill}} = (1-\gamma_{\text{KD}})\mathcal{L}_{\text{CE}}(\mathbf{p}_{\text{student}}, \mathbf{y}) + \gamma_{\text{KD}} T_{\text{distill}}^2 \mathcal{D}_{\text{KL}}(\mathbf{p}_{\text{teacher}}^{(T_{\text{distill}})} \| \mathbf{p}_{\text{student}}^{(T_{\text{distill}})})$$

Here $\mathbf{p}_{\text{teacher}}^{(T_{\text{distill}})}$ and $\mathbf{p}_{\text{student}}^{(T_{\text{distill}})}$ are the probability vectors computed at temperature T_{distill}, $\mathbf{y}$ is the one-hot ground-truth label vector, and $\gamma_{\text{KD}} \in [0,1]$ balances the hard-label and soft-target objectives. The scaling factor T_{distill}^2 ensures that the gradient magnitude of the distillation loss remains commensurate with the hard cross-entropy loss. Dividing logits by T_{distill} scales the derivative of the softmax probabilities by $1/T_{\text{distill}}$, and the gradient of the KL divergence with respect to the student's logits scales as $1/T_{\text{distill}}^2$ in the high-temperature limit. Multiplying by T_{distill}^2 prevents the teacher's supervisory gradient from vanishing relative to the hard-label loss as T_{distill} increases. This formulation defines response-based distillation, matching final output distributions. Modern

[10] **Temperature (softmax)**: The term comes from the Boltzmann distribution in statistical mechanics. Dividing logits by a temperature above one produces a softer distribution and can expose relative probabilities among non-target classes. At T_{distill}=1, the expression is standard softmax; increasing the temperature reduces logit gaps but does not guarantee that the revealed probabilities help the student. The useful amount of softening depends on logit scale, teacher calibration, student capacity, loss weighting, and the accompanying T_{distill}^2 factor. Temperature is therefore selected empirically rather than fixed to a universal range.

[11] **KL divergence**: Introduced by Solomon Kullback and Richard Leibler in 1951, $\mathcal{D}_{\text{KL}}(p\|q)$ quantifies the extra bits needed to encode samples from distribution p using a code optimized for q. The key asymmetric consequence: $\mathcal{D}_{\text{KL}}(\text{teacher}\|\text{student})$ penalizes the student heavily for assigning zero probability to teacher-probable outputs, forcing the student to maintain broad coverage of the teacher's distribution, including low-probability "soft labels" that carry the teacher's learned uncertainty. Distillation can affect calibration, which must be evaluated separately.

pipelines also employ feature-based distillation, training intermediate student layers to directly match the teacher's hidden representations or attention maps using mean squared error or cosine distance to guide gradient updates across deep backbones.

Efficiency gains and trade-offs Distillation's primary deployment advantage over unstructured pruning is that it produces a smaller dense model. Standard accelerators achieve peak throughput on contiguous matrix multiplications; unstructured sparse models incur irregular memory access and index decoding overheads that often negate theoretical FLOP reductions unless sparsity is extreme. A compact dense student maintains coalesced memory access and executes vendor-optimized dense GEMM kernels. However, parameter reduction alone does not guarantee lower latency, because operational intensity, layer depth, attention sequence length, and memory bandwidth limits still govern runtime. DistilBERT, for example, is about 60 percent faster (roughly 1.6×) primarily because it removes half of BERT's transformer layers (Sanh et al. 2019). The gap from the ideal 2× reflects work that halving the layers does not remove, such as the embedding layer, the task head, and fixed per-request overheads.

Distillation composes with pruning and quantization in multi-stage compression pipelines, and the order matters. Quantization usually comes last, because its calibration must see the weight and activation distributions of the final structure. Pruning first and then distilling makes the pruned network itself the student, so distillation doubles as the recovery training after the cut. Section 10.5.3 applies this order to the camera pipeline and to a modeled BERT pipeline. Structural compression can also happen earlier than fine-tuning. Gordon et al. (2020) show that BERT pruned during pretraining transfers to downstream tasks, so the structural step need not wait for a task-specific model.

These operational gains trade inference efficiency for substantial training overhead. Distillation requires running inference across the teacher model to generate soft targets and backpropagating loss through the student across millions of tokens. For large models, this auxiliary training phase consumes significant accelerator FLOPs and memory bandwidth, whereas post-training quantization, introduced later in this chapter, needs only a pass over a small calibration set. Compression effectiveness is also bounded by student capacity and teacher fidelity. An undersized student lacks the capacity to absorb the teacher's distribution, while an inaccurate or uncalibrated teacher imparts its own systematic errors to the student. Selecting an optimization strategy therefore requires evaluating task accuracy, parameter footprint, serving latency, and total training cost together, the comparison that table 10.11 in section 10.5 makes for distillation, pruning, and quantization.

Distillation still needs a student architecture, and the student's own design decides how much of the frame budget remains for everything else. Efficient architecture design and neural architecture search build that student to be efficient on the target by construction.

10.2.4 Efficient architecture design

Hardware-aware design begins before compression. If the target device cannot keep convolution kernels fed, cannot store activations without spilling, or cannot meet the power budget at the chosen input resolution, pruning and quantization only treat symptoms. The architecture itself must expose the kind of work the hardware can execute efficiently. That means choosing layer shapes, scaling rules, and operator patterns with memory bandwidth, parallelism, and energy as first-class constraints rather than post-hoc deployment checks. The efficient families of chapter 6, among them MobileNetV2's depthwise separable factorization, are best read here as responses to a measured deployment bottleneck rather than as a catalog of architectures.

10.2.4.1 Efficient design principles

The first design step is to identify what the trace shows as the limiting resource. A model can miss its target because every layer is too expensive, because one convolution family dominates arithmetic, because activations or parameters do not fit the memory hierarchy, or because the candidate architecture ignores the platform's preferred operators. Table 10.5 organizes the common responses by the bottleneck they address rather than by model family.

Table 10.5: Hardware-Aware Design Principles: A profiling trace should determine which architectural response is appropriate. MobileNet responds to redundant convolutional computation with depthwise separable convolutions, DenseNet and SqueezeNet respond to memory pressure with feature reuse and parameter reduction, and hardware-aware NAS incorporates measured platform behavior directly into the search objective.

Observed bottleneck	Architectural response	Example networks
Over-budget model scaling	Adjust depth, width, and resolution together so the model stays within the latency, memory, and power envelope.	EfficientNet
Redundant computation	Replace expensive dense operations with factorized or grouped operations that preserve useful channel mixing at lower arithmetic cost.	MobileNet, ResNeXt
Memory pressure	Reduce parameter storage or reuse features, then check peak activation lifetimes; feature reuse can retain a larger working set.	DenseNet, SqueezeNet
Platform mismatch	Include measured device latency, operator support, and power behavior in the architecture search or design loop instead of optimizing FLOPs alone.	MobileNetV3, MnasNet

These responses interact. Reducing convolutional FLOPs with depthwise separable convolutions helps only if the target runtime has efficient kernels for the resulting operators. Shrinking a model with parameter-reduction layers helps only if activation storage or memory traffic was part of the measured problem. Hardware-aware design therefore does not replace profiling; it moves profiling information earlier, into the architecture itself.

10.2.4.2 Scaling optimization

The first architectural response is global. When every stage of the profile is over budget, the problem is not one bad layer; the model is scaled incorrectly for the deployment envelope. The design task is then to distribute capacity across depth, width, and input resolution rather than choose a single parameter count: depth adds sequential stages and activation storage, whereas width exposes more parallel work within each stage at the cost of larger weight and activation tensors, and higher input resolution sharpens spatial detail by multiplying the positions every convolution must visit. Which dimension to grow therefore depends on the machine, since a wide parallel accelerator can absorb extra channels while a small edge device is limited first by memory capacity and energy per access.

Mathematically, the total FLOPs for a convolutional model can be approximated as:

$$O \propto N_L \cdot w_{\text{mult}}^2 \cdot r^2,$$

where N_L is depth (number of layers), w_{mult} is the width multiplier (channel count relative to a base model), and r is the input resolution. This expression shows why naive scaling fails. Increasing width and resolution together multiplies work quickly, and the resulting model may exceed the memory bandwidth or power budget even when the parameter count appears reasonable.

The camera pipeline's own architecture exposes one of these knobs directly. MobileNetV2's width multiplier scales every layer's channel count, and at a width of 0.75 the network shrinks from 3.5M to 2.6M parameters and from 600 MFLOP to 418 MFLOP per frame, keeping about 70 percent of the arithmetic. The square law predicts a larger cut, because depthwise layers cost in proportion to the width rather than its square, and the network's final convolution keeps its full width at reduced multipliers. A width setting is therefore a trade the phone can measure directly, in milliseconds per frame, against the accuracy the narrower network loses.

Compound scaling turns this balancing act into a controlled design rule. Instead of adjusting depth, width, and resolution independently, compound scaling grows all three dimensions by fixed ratios (α, β, γ) relative to a base model:

$$N_L = \alpha^{\phi} N_{L,0}, \quad w_{\text{mult}} = \beta^{\phi} w_{\text{mult},0}, \quad r = \gamma^{\phi} r_0$$

Here, ϕ is a scaling coefficient, and α, β, and γ (as in the original EfficientNet paper) are scaling factors determined from empirical accuracy and efficiency measurements. The rule matters because it prevents one dimension from consuming the budget before the others can contribute useful accuracy.

EfficientNet (section 10.2.5) validated this principle by using search to find a baseline architecture and then scaling it with balanced depth, width, and resolution coefficients (Tan and Le 2019). The lesson is not that every deployment should use EfficientNet. The systems lesson is that scaling is a resource-allocation decision, because the same accuracy target can imply different depth-width-resolution trade-offs depending on which resource the target platform makes scarce. Chapter 12 formalizes how to measure those trade-offs; the design principle here is to scale the dimensions against the binding resource.

The same logic extends beyond convolutional models. Transformer layers, attention heads, sequence length, and embedding width play roles analogous to depth, width, and resolution. Each increases capacity, but each stresses compute, memory bandwidth, or activation storage differently. Hardware-aware scaling keeps those dimensions tied to the measured bottleneck instead of treating model size as a single scalar.

10.2.4.3 Computation reduction

If the profile shows that a small set of convolutional operators dominates arithmetic, reducing the whole model uniformly is wasteful. The better response is to change the expensive operator. Modern efficient architectures do this by factorizing dense computations into cheaper pieces that preserve the representation needed for accuracy.

Depthwise separable convolutions, the factorization behind the MobileNetV2 and KWS lighthouses (Howard et al. 2017; Sandler et al. 2018), are the standard example. They split a standard convolution into a depthwise stage that filters each input channel separately and a pointwise 1×1 stage that mixes channels, which cuts the arithmetic to about $1/d_{\text{out}} + 1/K^2$ of a standard layer with d_{out} output channels and kernel width K, roughly one-ninth for 3×3 kernels and many output channels. What this chapter adds to that result is the deployment condition. The wall-clock benefit depends on kernel support and memory behavior. A mobile runtime with optimized depthwise kernels can convert much of this arithmetic reduction into latency savings, while a poorly supported backend may expose the factorized operations as many small, memory-bound kernels.

↩ Depthwise separable convolution and its cost reduction relative to a standard convolution are established in section 6.3.5.

Other factorization patterns follow the same logic: ResNeXt's grouped convolutions split feature maps into independent channel groups before merging them, whereas ResNet's bottleneck blocks and SqueezeNet's fire modules use 1×1 convolutions to narrow the channel dimension before an expensive spatial convolution so that both arithmetic and parameter storage fall. Because all of these rewrites help only when the spatial convolution they shrink actually dominates the trace, they yield much smaller gains once memory traffic, launch overhead, or unsupported kernels become the binding cost.

Arithmetic reduction is only one possible response. When profiling instead identifies memory capacity or data movement as the binding resource on the target device, the architecture must address that memory bottleneck directly.

10.2.4.4 Memory optimization

When the profile points to memory rather than arithmetic, the architecture must reduce the working set or the number of expensive memory accesses. Activations, feature maps, and parameters can exceed cache, SRAM, accelerator memory, or edge-device storage even when the FLOP count is acceptable. Memory-efficient architectures therefore try to preserve useful information while storing or moving less data.

DenseNet illustrates the feature-reuse response (Huang et al. 2017). In a traditional convolutional network, each layer computes a new set of feature maps, increasing the activation footprint as the network deepens. DenseNet connects layers so later computations can reuse earlier feature maps instead of relearning similar representations. In DenseNet with N_L layers and growth rate g, each layer generates g new feature maps concatenated with prior layers, causing the total number of feature maps to grow linearly:

$$\mathcal{O}(N_L g)$$

DenseNet reduces parameter redundancy through feature reuse, but concatenating retained features can increase activation storage and memory traffic. The resulting working set and runtime must be measured on the target hardware.

Parameter reduction applies the same reasoning to storage. SqueezeNet uses 1×1 convolutions to reduce the number of input channels before applying standard convolutions, making the expensive layer operate on a smaller representation (Iandola et al. 2016). The number of parameters in a standard convolutional layer is:

$$\mathcal{O}(d_{\text{in}} d_{\text{out}} K^2)$$

By reducing d_{in} using 1×1 convolutions, SqueezeNet reduces parameter count, achieving the paper's AlexNet-level accuracy target with far fewer parameters than AlexNet. That trade is attractive when the deployment constraint is flash storage, model download size, or parameter bandwidth; it is less decisive when activation memory or operator overhead dominates.

Feature reuse and parameter reduction are therefore not interchangeable recipes: reused features cut redundant representations, whereas 1×1 bottlenecks cut parameter movement at deployment, so the profile's binding memory term decides between them. Each of these manual motifs fixes one operator pattern at a time, leaving open how to choose kernel sizes, expansion ratios, and channel widths jointly across an entire network.

10.2.5 Neural architecture search

Hand-tuning kernel sizes, expansion ratios, and channel widths across twenty or more layers can explore only a handful of the billions of valid configurations that fit a phone's latency budget, leaving significant accuracy or speed on the table. **Neural architecture search (NAS)** automates this structural design by systematically traversing a bounded space of operations, topologies, and compound scaling rules prior to training (Elsken et al. 2019). Early reinforcement-learning formulations maximized task accuracy without hardware constraints (Zoph and Le 2016); subsequent weight-sharing methods amortized evaluation overhead (Pham et al. 2018); and hardware-aware NAS explicitly incorporated measured device latency and energy into the multi-objective search (Tan et al. 2019). For the camera pipeline, architecture search is already part of the model's lineage, since MnasNet and MobileNetV3, MobileNetV2's successors, came from searches that measured candidate latency on phones (Tan et al. 2019; Howard et al. 2019).

[12] **Hardware-aware NAS**: Optimizes measured latency rather than FLOPs, which can diverge by 3–5× when memory access or operator support dominates. Tan et al. (2019) fed device latency into search and found architectures 1.8× faster than MobileNetV2 at higher accuracy.

[13] **Bi-level optimization**: A formulation where one optimization problem sits inside another. In NAS, the outer level selects an architecture while the inner level trains that candidate's weights, so early methods paid a full training cost for every architecture evaluated. Weight-sharing methods such as Efficient Neural Architecture Search (ENAS) amortize one training run across many candidates, reducing search cost by roughly 1,000× (Pham et al. 2018).

Discovery runs as a closed loop. The search strategy samples candidate architectures $a \in \mathcal{A}$ from the defined search space, and the performance estimation engine evaluates their latency or accuracy and feeds the result back to refine subsequent proposals (figure 10.13). NAS[12] returns graphs that sit on the Pareto frontier under explicit resource bounds.

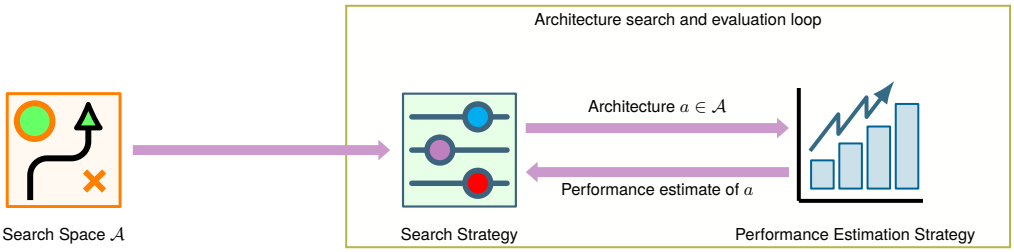


Figure 10.13: Neural Architecture Search Flow: An automated optimization loop for architecture discovery. The search strategy samples candidate subnetworks from a predefined search space, while the performance estimation engine evaluates candidates against target accuracy and hardware constraints (such as latency or energy), feeding reward signals back to steer subsequent exploration toward Pareto-optimal designs.

10.2.5.1 The NAS optimization problem (optional depth)

NAS rests on three design decisions: the search space, the search strategy, and how each candidate is evaluated. An architecture's quality cannot be known until it is trained, and training is expensive, which creates two nested decisions: which operations to include (the architecture) and which parameters work best for them (the weights).

NAS is therefore a **bi-level optimization problem**.[13] The outer loop searches the architecture space $\mathcal{A}$, while the inner loop trains candidate architectures to evaluate performance. Formally, the objective seeks the optimal architecture a^* that minimizes validation loss $\mathcal{L}_{\text{val}}$ under constraints C (latency, memory):

$$a^* = \arg\min_{a \in \mathcal{A}} \mathcal{L}_{\text{val}}(\theta^*(a), a) \quad \text{subject to} \quad C(a) \leq C_{\max}$$

where $\theta^*(a)$ represents the optimal model parameters for architecture a, obtained by minimizing training loss:

$$\theta^*(a) = \arg\min_{\theta} \mathcal{L}_{\text{train}}(\theta, a)$$

The computational bottleneck lies in this inner loop. If evaluating every candidate requires independent training from scratch, a modest search space of 10 operation choices across 20 layers yields 10^{20} possible architectures, making exhaustive enumeration impossible. Practical NAS methods overcome this combinatorial explosion through three mechanisms: restricting the search space to reusable cells, amortizing evaluation through weight sharing, or relaxing discrete choices into continuous, differentiable formulations.

10.2.5.2 Search space design (optional depth)

Many NAS systems search for reusable cells that are stacked to form complete networks, rather than searching whole architectures. A convolutional cell might choose among 3×3 convolution, 5×5 convolution, depthwise separable convolution, max pooling, or identity connections. A simplified cell with four nodes and two operations per edge yields roughly 10,000 possible cell designs, far more tractable than searching full architectures. NASNet discovered reusable normal and reduction cells this way.

Search spaces also encode hardware constraints by restricting candidate operations to primitives with efficient kernels on the target platform. Mobile search spaces, for example, prioritize depth-wise separable convolutions with bounded kernel sizes and channel counts (Zhang et al. 2020). MobileNetV3 paired this structured block-level search space with NetAdapt to prune layer channels against measured mobile CPU latency (Howard et al. 2019).

10.2.5.3 Search strategies (optional depth)

Search strategies trade search cost against architectural diversity (table 10.6). Reinforcement learning treats search as a sequential decision in which a controller proposes architectures and receives an accuracy reward. It discovered NASNet, but every reward requires training a candidate; Zoph and Le (2016) evaluated roughly 12,800 architectures on 800 GPUs for 28 days, a total of 22,400 GPU-days, a cost that pushed practical NAS toward weight sharing and cheaper performance predictors. Evolutionary algorithms instead mutate and recombine a population of candidates and keep the fittest; AmoebaNet reached ImageNet accuracy competitive with top human-designed models after 3,150 GPU-days (Real et al. 2019), a cost that needs proxy tasks, weight sharing, or massive parallelism to control. Gradient-based methods such as DARTS (Differentiable Architecture Search) (Liu et al. 2019) relax the search space into weighted combinations of all operations and optimize architecture weights and model weights jointly by gradient descent, reducing search cost from hundreds of GPU-days to one to four, though the relaxation may miss discrete patterns that discrete search finds.

Table 10.6: NAS Search Strategy Comparison: Representative trade-offs between search efficiency, use cases, and limitations for different NAS approaches. The costs are method-specific results from early reinforcement-learning and evolutionary searches and from DARTS, not universal ranges for each strategy (Zoph and Le 2016; Real et al. 2019; Liu et al. 2019).

Strategy	Search Efficiency	When to Use	Key Challenge
Reinforcement Learning	22,400 GPU-days	Novel domains, unconstrained search	High computational cost
Evolutionary Algorithms	3,150 GPU-days	Parallel infrastructure available	Requires large populations
Gradient-Based (DARTS)	1–4 GPU-days	Limited compute budget	May converge to suboptimal local minima

Hardware-aware NAS optimizes execution metrics on target silicon rather than FLOP counts, which correlate poorly with latency when memory bandwidth or launch overheads dominate. Under the arithmetic intensity law (principle 6), an operation with few FLOPs but low arithmetic intensity can run slower than a higher-FLOP operation that saturates compute. MnasNet addresses this divergence by measuring each candidate's latency directly on a mobile phone and folding it into the reward, which combines accuracy and latency through a weighted product:

$$\text{Reward}(a) = \text{Accuracy}(a) \times \left(\frac{L_{\text{lat,target}}}{L_{\text{lat,infer}}(a)} \right)^{\beta_{\text{lat}}}$$

where $L_{\text{lat,infer}}(a)$ is the measured on-device inference latency of architecture a, $L_{\text{lat,target}}$ is the latency constraint, and the exponent β_{lat} controls the accuracy-latency trade-off. Architectures that exceed the latency target are penalized. In the reported MnasNet search space, varying inverted-residual expansion ratios produced a stronger accuracy-latency trade-off than uniform expansion.

10.2.5.4 When to use NAS

Neural architecture search can discover architectures that outperform hand-designed alternatives, but its cost demands a check on whether the investment is justified. NAS can be worthwhile for novel hardware platforms with unusual constraints, such as new accelerator architectures or highly constrained edge devices, where existing architectures are poorly optimized. It can also make sense at deployment scales where small efficiency improvements justify the upfront search cost, or when architecture families for cloud, edge, and mobile deployments can amortize one search across many variants.

Conversely, custom NAS is difficult to justify when standard deployment constraints already have well-optimized architecture families. If the compute budget is only a few GPU-days, large reinforcement-learning or evolutionary searches are impractical; differentiable or weight-sharing methods such as DARTS may remain feasible, but still require deployment-scale and validation-cost justification. Rapidly changing requirements also weaken the case because the target can move before search and validation finish.

For most practitioners, existing NAS-discovered or NAS-assisted architectures provide strong baselines before funding a search from scratch. EfficientNet jointly scales depth, width, and resolution with compound coefficients (Tan and Le 2019), MobileNetV3 combines hardware-aware search with NetAdapt for phone latency targets (Howard et al. 2019), MnasNet folds measured phone latency into its reward (Tan et al. 2019), and FBNet puts device-specific latency into a mobile-CPU search objective (Wu et al. 2019). NAS has also been applied to compact transformer models under compute or memory constraints. These results show what search found under particular spaces, objectives, and measurements, not that search universally beats careful manual design. When latency, memory, or energy enters the objective and is measured on the target platform, the search evaluates candidates against the resource deployment actually constrains, yet the transfer quality of the resulting architectures still depends on task and hardware. Reserve custom NAS for unusual constraints or deployment scales that can amortize the search and validation investment. Even a searched architecture, however, still executes the same fixed graph on every input, spending as much compute on a blank background frame as on a crowded street scene.

10.2.6 Adaptive computation methods

A static 24-layer transformer executes identical multiply-accumulate (MAC) operations and streams its entire parameter footprint from HBM or off-chip DRAM for an obvious, easily classified sample as it does for an ambiguous one. Because pruning, distillation, factorization, and architecture search all produce a fixed execution graph, every input follows the same scheduled path regardless of difficulty. **Adaptive inference** replaces this open-loop pipeline with an input-dependent control loop, in which intermediate representations or runtime budgets decide whether to terminate execution early or which specialized subnetwork receives the activations. While dynamic paths can reduce average arithmetic operations and memory traffic, they introduce runtime overheads in routing logic, batch management, confidence calibration, and latency variance. The camera pipeline is a natural candidate, because many frames are easy and the phone processes one frame at a time. Dynamic pruning applies the idea at the granularity of channels and filters; early exits, conditional layers, and expert subnetworks apply it at coarser grain, where the savings per decision are larger and so is the cost of a wrong one.

10.2.6.1 Dynamic pruning

Traditional pruning methods, whether unstructured or structured, produce a **static pruning** mask or subnetwork that remains fixed during deployment, even if the mask was learned gradually during training. **Dynamic pruning** instead decides at inference time, from each input, which computation is active, so the executed subnetwork changes from one input to the next.

Dynamic pruning can therefore allocate different amounts of work to different inputs and can use, for a hard input, capacity that a static deployment mask would remove permanently. Activation-conditioned pruning, for example, deactivates neurons or channels for particular inputs (Hu et al. 2023). On a simple image with mostly uniform regions, some convolutional filters produce negligible activations, and a dynamic policy can exclude them for that frame. The exclusion reduces nominal work, but the controller sits on the critical path, input-dependent routing fragments execution across SIMD vector lanes, and any quality change remains part of the result. Training may also need routing losses or regularization so that the policy actually uses its cheaper paths. Dynamic pruning is therefore a candidate when input complexity varies enough to repay the controller and batching overhead, not a default substitute for static pruning, and its gain must be measured as latency and accuracy together across the full input distribution.

↪ Joint evaluation of compressed models on accuracy and target efficiency metrics such as latency, plotted as an efficiency-quality frontier, is measured in section 12.9.1.4.

Methods that change the mask during training are sometimes also called dynamic, but they solve a different problem. Gradual magnitude pruning progressively increases the fraction of pruned parameters while fine-tuning the survivors, which is the iterative schedule of section 10.2.1.5 run inside training, and dynamic sparse training additionally regrows connections that prove important later. Both end in a static mask for deployment, so this chapter reserves *dynamic pruning* for input-dependent execution.

10.2.6.2 Dynamic schemes

Because input samples vary widely in difficulty, spending the same compute budget on every input wastes memory bandwidth and execution cycles on simple cases. Dynamic schemes modulate

execution paths to lower amortized computation while preserving the deep network for worst-case inputs. This architectural flexibility spans four primary control variables: network depth (early exit), structural routing (conditional layer skipping), expert subnetworks (mixture-of-experts), and continuous computational allocation.

To control execution depth, **early exit** architectures attach auxiliary classification heads to intermediate layers, terminating execution when intermediate representations yield sufficient prediction confidence (Teerapittayanon, McDanel, and H. T. Kung 2017). BranchyNet implements this mechanism across convolutional backbones using shallow exit branches, whereas multi-exit vision transformers attach linear probes and entropy estimators between transformer blocks (Xin et al. 2021). When an input satisfies an exit criterion, such as softmax prediction entropy dropping below a pre-tuned threshold, the runtime halts evaluation and returns the intermediate prediction. The net operational benefit requires that the arithmetic operations and memory transfers avoided in subsequent layers exceed the computational overhead of evaluating the intermediate classifiers and gating thresholds.

The systems trade-off diverges sharply across hardware deployment targets. On single-sample edge processors (batch size $B = 1$, typical of microcontrollers and mobile NPUs), early exit immediately terminates the execution thread (Hu et al. 2020). Skipping remaining layers eliminates both the arithmetic operations and the off-chip DRAM transactions required to load subsequent layer weights into on-chip SRAM cache. Conversely, on parallel throughput accelerators (GPUs and TPUs) processing batched workloads ($B > 1$), early exit introduces severe batch fragmentation (Chen et al. 2024). If only a subset of samples in a batch reaches the exit threshold, the remaining samples must still traverse deeper layers. Unless the runtime executes dynamic batch compaction (gathering unresolved activation tensors into a contiguous memory buffer through memory copies and kernel synchronizations), inactive samples remain allocated in memory, or the accelerator must mask inactive SIMD or single instruction, multiple threads (SIMT) lanes. Consequently, tail latency for the batch remains dictated by the deepest traversing sample. Along a transformer backbone, each block taps its intermediate representation into an auxiliary classifier and a learned confidence estimator (learning to exit, LTE) that can trigger an early exit, while unresolved inputs continue to deeper layers (figure 10.14).

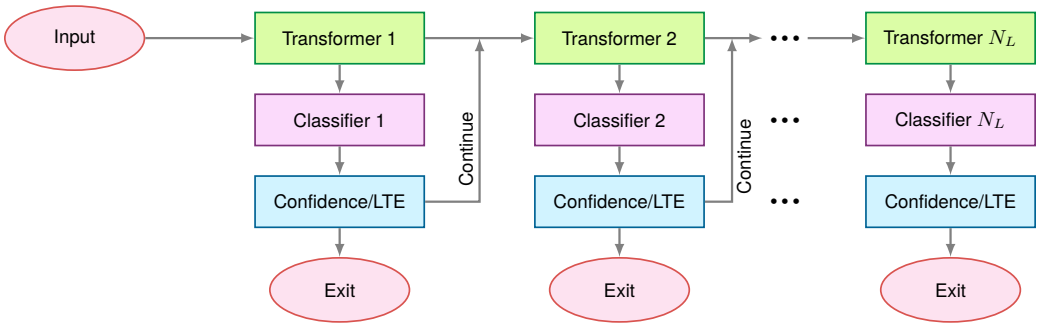


Figure 10.14: Early Exit Architecture: Transformer layers dynamically adjust computation by classifying each layer's output and enabling early termination if sufficient confidence is reached, reducing average latency and power when many inputs can exit before the final layer. Source: (Xin et al. 2021).

↪ The latency-throughput trade-offs of batching under production request streams, including service-level objective (SLO) violations, are developed in section 13.5.4.

Where early exit truncates depth along a single linear pipeline, **conditional computation** dynamically modulates the topological routing of activations through the network graph (Bengio et al. 2015). Instead of executing every block in a predefined sequence, gating functions evaluate activation tensors to bypass residual layers, alter operator kernels, or select specialized functional blocks. This converts a static directed acyclic graph (DAG) into an input-dependent dynamic graph.

SkipNet illustrates the pattern at the granularity of whole blocks. It uses lightweight recurrent or feedforward gating controllers to conditionally bypass residual blocks in convolutional networks, preserving full network depth only for computationally difficult inputs (X. Wang et al. 2018). From a hardware systems perspective, conditional routing achieves wall-clock speedups only when the cycles saved by bypassing operators surpass the latency of evaluating the gating network, executing host-device control synchronization, and managing uncoalesced memory accesses.

In a dense transformer, every token computes against every weight matrix, so parameter capacity and per-token compute grow together. The **mixture-of-experts (MoE)** architecture decouples them by replacing dense feedforward network (FFN) layers with a bank of parallel expert subnetworks coordinated by a parameterized router (Shazeer et al. 2017). Google's Switch Transformer[14] steers each token to a single expert ($k_{\text{experts}} = 1$), which reduces communication overhead relative to routing to $k_{\text{experts}} > 1$ experts (Fedus et al. 2022).

Sparse activation trades compute for memory. A token routed to one of N_{experts} experts executes arithmetic against only $1/N_{\text{experts}}$ of the expert parameters, yet an all-resident deployment keeps all N_{experts} expert matrices in device memory. In a Switch Transformer block, a Switching FFN layer sits inside the standard transformer stack, and its gating router dispatches individual tokens ("More" with $p_{\text{gate}} = 0.65$, "Parameters" with $p_{\text{gate}} = 0.8$) to distinct expert sub-blocks (FFN 2 and FFN 1) while unselected experts stay idle (figure 10.15). Because the router lies on the critical path, the runtime must gather tokens assigned to the same expert into contiguous tiles (Lepikhin et al. 2021); at low batch sizes or under a skewed routing distribution, divergent routing leaves some experts with too few tokens to keep matrix units busy and overloads others, forcing the runtime either to truncate tokens or to launch kernels that leave lanes idle.

[14] **Switch Transformer**: Fedus et al. (2022) scales to roughly 1.6 trillion total parameters while routing each token to one expert; in a compute-matched experiment, Switch-Base with 64 experts reached the T5-Base quality target in about one-seventh the training time; the trillion-parameter comparison reported roughly 4× speedup over T5-XXL. Routing each token to one expert reduces communication relative to routing to $k_{\text{experts}} > 1$ experts, but load imbalance requires auxiliary losses and capacity factors, which cap how many tokens each expert accepts per batch. This trade-off, massive parameter capacity at low per-token compute cost in exchange for complex load-balancing engineering, defines the MoE design space.

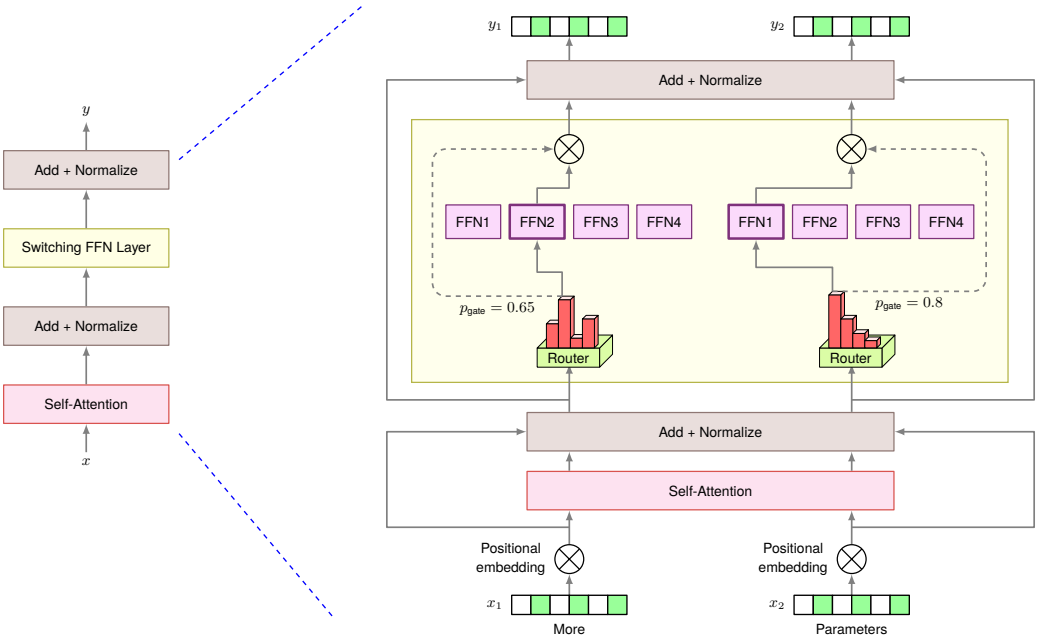


Figure 10.15: Switch Transformer Architecture: Replacing dense feedforward layers with a Switching FFN layer (left) enables sparse conditional routing. A gating network routes each token to a single expert (right) out of a large pool, decoupling total model capacity (parameters) from per-token compute cost (FLOPs). Source: (Fedus et al. 2022).

↪ How runtimes execute tensors whose shapes vary, by matching precompiled shape profiles, padding, splitting the batch, or falling back to a general kernel, is developed in section 11.8.3.2.

Early exit and MoE routing make discrete branch selections, but adaptive inference can also modulate depth or iteration counts continuously according to task difficulty (Yang et al. 2020). BlockDrop adjusts the number of active layers with a runtime policy network (Wu et al. 2018), and dynamic layer scaling adds computation until output uncertainty drops below a calibrated threshold. In an embedded vision system, high-contrast frames with clear road geometry resolve through a minimal prefix of layers, whereas degraded frames with occluded obstacles trigger deeper processing. The gain materializes only if the control evaluation remains computationally lightweight and the runtime schedules frames with matched execution depths concurrently to maintain high hardware utilization.

10.2.6.3 Implementation challenges

Realizing the efficiency gains of adaptive computation requires overcoming challenges across optimization, runtime orchestration, and hardware execution. During training, discrete routing decisions

(such as non-differentiable argmax branch selections) break standard gradient-based backpropagation. Practitioners overcome this with continuous relaxations such as the Gumbel-Softmax estimator, which replaces hard selections with smooth approximations so gradients flow, with policy gradients that reinforce routing choices that earn a reward, or with auxiliary load-balancing losses that keep routers from collapsing onto a single favored expert (Shazeer et al. 2017).

At runtime, control-flow branching conflicts with the lockstep execution of modern accelerators, where the threads of a warp execute the same instruction. When threads within a GPU warp take divergent branches, the hardware may serialize them and mask inactive threads, which erodes the theoretical FLOP savings. Gating also introduces control dependencies that can limit fusion, require token-permutation buffers, or trigger host-device synchronization. If these dispatch overheads exceed the compute time saved by skipping layers, the dynamic model has higher end-to-end latency than a dense baseline. Runtime strategies such as ragged batching and dynamic kernel launch queues manage these patterns, and they belong to the accelerator runtime rather than to the model.

Dynamic control also adds quality and security failure modes. Poorly calibrated gates underallocate computation to rare inputs, which degrades accuracy on safety-critical edge cases, and adversarial perturbations can push the gates toward truncated early exits or maximally deep paths. Evaluating adaptive models therefore cannot rely on average FLOP counts alone. Rigorous system profiling must report path distribution histograms, tail latency (p99), accuracy stratified by input difficulty, and throughput stability under dynamic batching, and a production deployment keeps monitoring route frequency and quality by route after launch.

The structural techniques covered so far (pruning, low-rank factorization, distillation, efficient architecture design, NAS, and adaptive computation) all optimize *what* computations the model performs, including which parameters exist, which connections remain, how the architecture is structured, and which paths a given input executes. These techniques can substantially reduce parameter counts and theoretical FLOPs. Regardless of structural efficiency, every surviving weight and activation must still be stored and processed at some numerical precision.

↪ Infrastructure telemetry that tracks latency distributions in deployed systems, alongside the model-quality signals production monitoring pairs with it, is developed in section 14.5.1.

Checkpoint 10.1: Choosing a structural method

Structural methods relocate deployment cost in different ways.

- ☐ **Structured vs. unstructured pruning**: compare how each pattern interacts with GPU and TPU execution.
- ☐ **Distillation vs. pruning**: explain why a dense student may preserve accuracy better than aggressive pruning while requiring additional training.
- ☐ **NAS selection**: identify when hardware-specific constraints and deployment scale justify architecture search rather than manual design.
- ☐ **Factorization cost**: explain why fewer stored parameters and MACs may not reduce latency when two factor operations add launches and an intermediate tensor.
- ☐ **Adaptive computation**: explain why early exit can cut latency at batch size one yet fragment batched execution on a GPU.

Structural optimizations alter what a model computes, but the surviving graph must still execute within the physical memory bandwidth, register files, and cache hierarchies of the target silicon. When structural pruning and distillation have carved away unnecessary parameters and layers, every remaining operation continues to incur a memory-movement and arithmetic cost determined entirely by its bit representation. Optimizing this numerical representation forms the second tier of the optimization stack, and its clearest case is a workload whose time is set by bytes rather than by arithmetic.

10.3 Numerical Precision: Changing How Values Are Stored

GPT-2 XL decoding on the phone shows what structural changes leave untouched. Its FP16 weights fit in the phone's memory, yet every generated token streams all 3 GB of them across a 51.2 GB/s bus, about 59 ms per token before any arithmetic (table 10.1). Pruning or factorizing its layers would

change which weights exist; the complementary optimization leaves the network topology unaltered and reduces the bit width of every surviving tensor. Quantization trades numerical precision for a smaller memory footprint, higher cache utilization, and higher arithmetic throughput on hardware with integer datapaths.

Definition 10.4: Quantization

Quantization is a model-compression technique that reduces information fidelity by mapping high-precision continuous values to a lower-precision discrete set.

1. **Significance**: FP32-to-INT8 conversion reduces the raw payload per quantized value by 4×. Real artifact size, traffic, and latency also depend on scales, metadata, packing, operator coverage, and the target kernels.
2. **Distinction**: Unlike pruning, which reduces the count of parameters, quantization reduces the bit depth of selected weights and/or activations.
3. **Common pitfall**: A frequent misconception is that quantization is just "rounding." In reality, it is a lossy mapping that requires careful range estimation and, often, quantization-aware training (QAT) to minimize its impact on accuracy.

Quantization[15] concerns every neural network weight and activation stored at some numerical precision: FP32 (32 bits), FP16 (16 bits), INT8 (8 bits), or lower. Bit width sets how many bytes each value occupies, how much cache it uses, and how fast specialized matrix engines can run.

Quantization transforms three physical terms of execution. First, converting FP32 to INT8 reduces raw weight payload by 4× before scale and packing metadata, allowing models to fit within constrained device memories. Second, when tensors remain packed across memory buses, memory traffic drops proportionally, directly reducing token latency in bandwidth-bound autoregressive decoding. Third, fixed-point integer arithmetic units occupy smaller silicon area and consume significantly less energy per operation than floating-point units, enabling higher compute density on supported hardware (Gupta et al. 2015; Y. E. Wang et al. 2019).

The accuracy cost varies by model, task, quantizer, calibration data, and bit width. Many convolutional neural network (CNN) inference models tolerate INT8 well; transformers, speech models, and outlier-heavy layers may need different granularity, mixed precision, or training-aware methods. The main approaches are **post-training quantization** (PTQ), **quantization-aware training** (QAT), and mixed-precision or **extreme quantization** for lower-bit deployment. They differ in training cost and control, not in a guaranteed ordering of accuracy.

The viability of each approach depends on how much precision a given model architecture can shed before quality degrades. Across precision formats from FP32 down to INT2, both CNN and transformer curves follow a low-loss plateau where INT8 quantization closely matches full-precision performance, followed by a steep accuracy cliff at lower bit widths where representational capacity breaks down. Because tolerable accuracy degradation depends on end-to-end application constraints, engineering teams must evaluate these cliff boundaries for each target workload and range-calibration recipe (figure 10.16).

10.3.1 Numerical formats

Cutting a tensor from 32 bits to 16 or 8 bits can still overflow on large activation outliers if the format lacks exponent range, or stall in software unpacking if the target processor lacks a native datapath for that bit width. How a format divides its bits between exponent range and fraction precision decides which values survive conversion, and whether the silicon provides a matching execution unit decides whether narrower values run any faster. Because neither property follows from bit count alone, selecting a numerical format requires balancing dynamic range and hardware execution support against memory traffic rather than minimizing bit width blindly. The formats commonly deployed across accelerator architectures differ in raw storage, peak arithmetic rate, and hardware support, each measured against standard 32-bit floating point (FP32) (table 10.7).

[15] **Quantization**: Rooted in Shannon's theory of representing continuous signals with discrete values (Shannon 1948), reducing FP32 to INT8 collapses over four billion representable values to just 256. Neural networks often tolerate this because trained weights concentrate information in relative magnitudes, not absolute precision: INT8 inference can stay close to the full-precision baseline with appropriate calibration or quantization-aware training, while INT4 and lower-bit methods become more architecture- and method-dependent (Jacob et al. 2018; Gholami et al. 2022; Shen et al. 2020; Lin et al. 2024). The systems consequence is that quantization viability must be validated per-model and per-task, not assumed from aggregate benchmarks.

↪ How accelerators trade precision and dynamic range for matrix-engine throughput, pairing low-precision inputs with higher-precision accumulation, is developed in section 11.4.5.

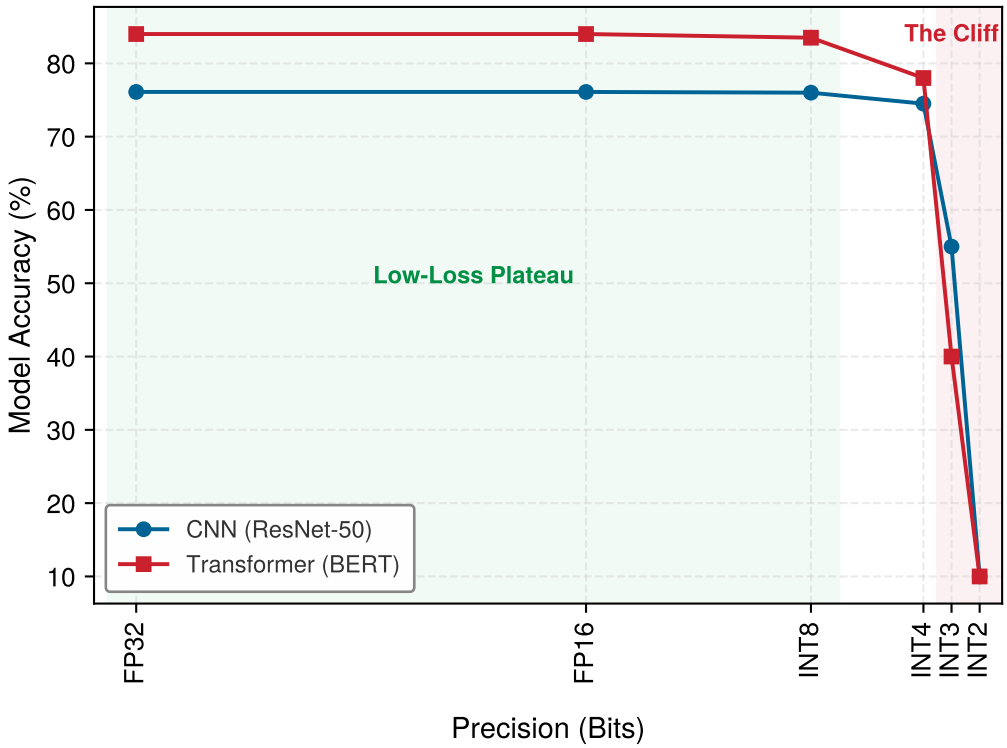


Figure 10.16: Illustrative Quantization Regimes: The curves depict a qualitative plateau followed by an accuracy cliff as bit width falls. Their breakpoints are illustrative; actual boundaries depend on the model, layer distributions, quantizer, calibration or training method, and evaluation task.

Table 10.7: Numerical Precision Formats: Storage ratios reflect raw value width; compute speed and power effects depend on hardware support and workload behavior.

Precision Format	Bit-Width	Storage Reduction (vs. FP32)	Compute Speed (vs. FP32)	Power Effect	Use Cases
FP32	32-bit	Baseline (1×)	Baseline (1×)	Baseline	Training & inference (general-purpose)
FP16	16-bit	2× smaller	Up to 2× peak on selected supported hardware	Hardware- and workload-dependent	Accelerated training, inference (NVIDIA Tensor Cores, TPUs)
BF16 (Brain Floating Point)	16-bit	2× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Training on TPUs, transformer-based models
TF32 (TensorFloat-32)	19-bit	None (stored as FP32)	Up to 8× peak on NVIDIA Ampere Tensor Cores	Hardware- and workload-dependent	Training on NVIDIA GPUs
FP8 (E4M3, E5M2)	8-bit	4× smaller	2× the dense FP16 peak on the H100	Hardware- and workload-dependent	Inference and training on recent data-center accelerators
INT8 (8-bit Integer)	8-bit	4× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Quantized inference (Edge AI, mobile AI, NPUs)
INT4 (4-bit Integer)	4-bit	8× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Ultra-low-power AI, experimental quantization

Precision Format	Bit-Width	Storage Reduction (vs. FP32)	Compute Speed (vs. FP32)	Power Effect	Use Cases
Binary/Ternary (1-bit/2-bit)	1–2-bit	16–32× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Extreme efficiency (binary/ternary neural networks)

At 16 bits, FP16 and BF16 halve memory storage relative to FP32 while targeting dedicated matrix execution units (such as NVIDIA Tensor Cores and Google TPUs) to accelerate arithmetic throughput. The range-versus-precision trade between them is established with mixed-precision training in section 8.6.3, and figure 10.17 shows the bit layouts that set it. BF16 keeps FP32's 8-bit exponent and therefore its dynamic range, whereas FP16 spends more bits on the fraction and caps finite values at 65,504, which is why FP16 training needs loss scaling and BF16 training does not. For inference, the same range limits decide whether activations with large outliers survive the narrower format. TensorFloat-32 (TF32) adopts a hybrid compromise. It retains the 8-bit exponent of FP32 and the 10-bit fraction of FP16, executing on Tensor Cores to accelerate matrix multiplication while storing values in standard 32-bit memory.

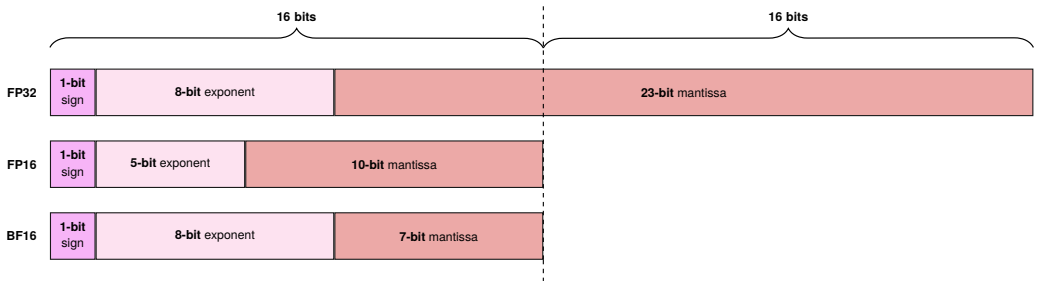


Figure 10.17: Floating-Point Bit Layouts: Bit allocation trades dynamic range against significand precision. FP32 uses 8 exponent and 23 fraction bits. BF16 retains FP32's 8-bit exponent and therefore a similar numerical range, but reduces the fraction to 7 bits. FP16 uses 5 exponent and 10 fraction bits, providing more significand precision than BF16 but a narrower range that can require loss scaling during training.

For inference workloads, INT8 precision provides a fourfold reduction in raw storage relative to FP32 when the chosen quantizer preserves acceptable prediction accuracy. Inference avoids the gradient backpropagation pass, eliminating the small-magnitude, high-dynamic-range updates that cause underflow during training. Activations and weights can still contain outliers or sensitive values, so range calibration or QAT must validate mapping them onto signed 8-bit integers ($[-128, 127]$) using uniform affine or symmetric quantization scales. In bandwidth-limited inference regimes, such as autoregressive language model token generation, this $4\times$ footprint reduction relieves memory bus contention by cutting DRAM and cache traffic. On compute-bound operators, dedicated INT8 vector and tensor units provide higher arithmetic operation density per clock cycle and per unit of silicon area than equivalent floating-point units.

FP8 gets its own row because it is the first 8-bit format that keeps an exponent. It comes in two layouts, each with one sign bit: E4M3, with four exponent and three mantissa bits, for weights and activations, and E5M2, with five exponent and two mantissa bits, for the wider range of gradients (Micikevicius et al. 2022). Like INT8, it halves the bytes of FP16, and on the reference H100 its peak matrix rate equals the INT8 rate, 2× the dense FP16 peak. Unlike INT8, it needs no zero-point, and its exponent absorbs the outliers that force an integer format to choose between clipping and resolution, so a model often converts to FP8 with less range-calibration effort than to INT8. FP8 as the default serving precision on such accelerators is developed in chapter 13. Microscaling formats carry the same idea below eight bits (Rouhani et al. 2023). A block of values, typically 32, shares one 8-bit power-of-two scale, so the elements of an MXFP8, MXFP6, or MXFP4 block borrow their range from the shared exponent and spend their few bits on precision. They are an awareness item here, because their payoff depends on accelerators with native support.

Binary and ternary formats occupy the far end of table 10.7, constraining values to $\{-1, +1\}$ or $\{-1, 0, +1\}$ so that multiplication collapses into bitwise logic or a conditional add. They promise

the largest storage ratio in the table, but byte-addressed memory and training stability make them a special case that section 10.3.4 develops in full. The keyword-spotting lighthouse shows why such formats are considered at all. Under the strict ~512 KB SRAM budget, INT8 is the baseline target, and INT4, binary, or ternary representations trade additional dynamic range to meet stricter memory and power bounds. Whereas table 10.7 counts bytes and peak rates, what each format saves in joules depends on where its bits travel, which is the subject of the energy comparison that follows.

10.3.2 Precision and energy

In silicon, energy dissipation scales directly with wire capacitance ($E \propto CV^2$), so shuttling bits across long memory buses costs orders of magnitude more joules than toggling logic gates inside an arithmetic logic unit. Because physical data movement so often dominates a workload's power budget, choosing a numerical precision is an energy decision governed by the number and location of memory accesses alongside the arithmetic performed. Reducing numerical precision directly shrinks the byte volume traversing the memory hierarchy (D_{vol}) and lowers the switching energy per ALU operation (E_{compute}), relaxing memory bandwidth and thermal bottlenecks across the hardware stack.

↪ Accelerator designs that keep operands in registers and on-chip SRAM, away from costly off-chip DRAM, are developed in section 11.2.3.

10.3.2.1 Where precision saves energy

Precision saves energy in two places, and the larger one is usually movement. The energy-movement principle (7), $E_{\text{move}} \gg E_{\text{compute}}$, says that moving an operand costs far more than operating on it (Horowitz 2014). In the reference technology model, a 32-bit read from off-chip DRAM costs about 640 pJ, roughly 2,100× the energy of the INT8 multiply-accumulate that consumes it, and the cost keeps rising as data leaves the chip for storage or the network (table 4.6). A packed INT8 stream carries one quarter of the raw value bits of an FP32 stream, so each weight or activation that crosses the memory bus costs about a quarter of the movement energy, provided transactions stay full; caches, metadata, alignment, and transaction size set the realized saving. The reference measurements predate FP8, but an FP8 value moves the same eight bits as an INT8 value, so the movement saving applies to it unchanged.

Arithmetic supplies the second saving (table 10.8). An FP32 addition costs 0.9 pJ/op and an INT8 addition 0.03 pJ/op, a ratio of 30×; an FP32 multiply costs 3.7 pJ/op and an INT8 multiply 0.2 pJ/op, a ratio of 18.5×. Every multiply-accumulate a matrix unit performs combines the two, so the figure that governs a format decision is their combination. Because INT8 products accumulate in INT32, a full INT8 multiply-accumulate uses roughly 15× less energy than its FP32 equivalent, on top of the 4× cut in bytes moved. Section D.1 collects the reference energy ratios, and section D.4 compares numerical format trade-offs.

Table 10.8: INT8 versus FP32 Energy per Operation: Reference 45 nm energy for one add, one multiply, and one multiply-accumulate at each precision. The INT8 multiply-accumulate sums its products in INT32, so its cost is an INT8 multiply plus an INT32 add, which is why its ratio is the smallest of the three. A 32-bit DRAM read, at 640 pJ, costs more than every row here combined. Total inference energy also depends on operation counts, access counts, and where operands reside (Horowitz 2014).

Operation	FP32 (pJ)	INT8 (pJ)	FP32 / INT8
Add	0.90	0.03	30×
Multiply	3.70	0.20	18.5×
Multiply-accumulate	4.60	0.30	15×

Accelerators equipped with native low-precision units (such as NVIDIA Tensor Cores, Google TPUs, or mobile NPUs) compound these arithmetic savings with reduced memory traffic. In contrast, general-purpose processors lacking native low-precision execution pipelines lose much of this benefit to the runtime overhead of bit unpacking, format conversion, and scalar fallback. Precision is therefore a model-hardware decision, not a software configuration flag.

Which saving matters depends on where the workload sits. Under the iron law of ML systems (principle 3), reducing bit width changes the data-movement (D_{vol}/BW) and compute ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) terms only through the execution path that uses the new representation. Workloads with little

The machine's memory capacity, not arithmetic, now sets the constraint.

arithmetic per byte are memory bound, so shrinking data movement dominates; workloads with more arithmetic per byte are compute bound, so arithmetic savings matter more, a distinction the roofline model in section 11.6.1 formalizes. Neither ratio predicts battery life on its own, because end-to-end energy also includes control, sensors, and idle power. When memory capacity binds, the more immediate benefit is placement, because smaller values may let the model reside in a memory tier that the uncompressed model cannot fit.

Recommendation models convert the energy argument into a placement problem, in which quantization must shrink the dominant memory structures enough to reside in the host machine's physical memory. When an embedding table outgrows a single accelerator's HBM, reducing bytes per entry determines whether the table fits on the node at all.

Lighthouse 10.1: DLRM and embedding quantization

The DLRM lighthouse presents a compression challenge where memory capacity, rather than compute throughput or memory bandwidth, is the binding constraint (Naumov et al. 2019). Even the reference configuration in table 10.1 needs 100 GB of weights, more than one accelerator's memory, and production embedding tables can grow to terabytes that must be sharded across many devices.

For DLRM, quantization targets storage density rather than arithmetic throughput. Reducing embedding-table precision from FP32 to INT8 (or lower) shrinks the memory footprint by 4–8×, allowing larger tables to fit on fewer GPUs when accuracy and lookup kernels tolerate the lower precision. Compressing the lookup tables ensures that physical device memory can accommodate the algorithmic model.

Lighthouse 10.2: Keyword spotting and the SRAM budget

The KWS lighthouse, DS-CNN, operates at the opposite extreme of the iron law from DLRM (Y. Zhang et al. 2017). Consider a deployment that keeps weights in roughly 512 KB of SRAM. Even the purpose-built DS-CNN keyword spotter overshoots that budget in FP32, where its weights occupy 800 KB. Quantized to INT8, its weights shrink to 200 KB, leaving space whose sufficiency must be checked against peak activation and runtime-buffer use. A device that reads weights directly from flash has a separate weight-storage budget; its SRAM feasibility still depends on the working set. A stricter smart-doorbell microcontroller with 256 KB SRAM leaves closer to 100 KB for model weights after runtime buffers, audio windows, and feature extraction state, motivating more aggressive INT4 or binary compression.

In FP32, even the compact DS-CNN architecture moves 4× more raw weight bytes than in INT8, and the arithmetic path can also cost more on hardware with efficient integer units. For an always-on coin-cell device, that reduction contributes to longer battery life, but sensor, wake-up, feature-extraction, and idle-power costs may dominate. Here, quantization reduces the model's data-movement term and can improve its compute term $(O/(R_{\text{peak}} \cdot \eta_{\text{hw}}))$ when the integer path is supported.

Together, DLRM and the TinyML lighthouse (10.2) illustrate the two operational extremes of this memory constraint. DLRM uses lower precision to fit embedding tables that can reach terabytes into host memory; TinyML uses lower precision to keep every inference within a tiny energy envelope and a few hundred kilobytes of on-chip SRAM. Across both extremes, reducing numerical precision attacks physical execution costs on two fronts, arithmetic energy and latency in the ALU and byte traffic across the memory bus. Both savings assume that the low-precision values still represent the model faithfully, and producing such values is the job of the quantization methods.

10.3.3 Quantization methods

Casting trained FP32 weights and activations straight to narrow integers without fitting their range clips outlier channels and rounds small values to zero, so practical quantization methods differ in

how much calibration or retraining they spend to keep that error small. Post-training quantization, quantization-aware training, and extreme low-bit formats differ chiefly in the training and validation cost they require. Representation choices, such as selecting INT8 versus INT4, mixing bit widths across sensitive layers, or choosing per-tensor versus per-channel ranges, are orthogonal options evaluated *within* these workflows rather than separate maturity levels. Ordered by the engineering effort each requires, the deployment sequence moves from fast post-training range calibration through quantization-aware retraining to custom low-bit pipelines (figure 10.18).

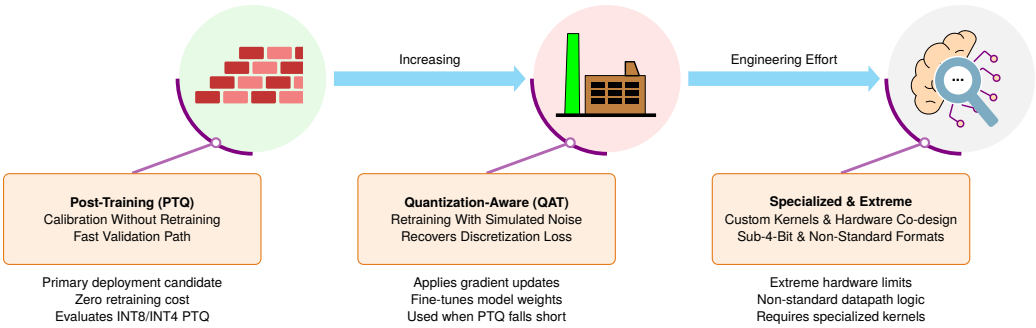


Figure 10.18: Quantization Engineering Workflow: The tiers order deployment paths by engineering and validation effort, not by format or guaranteed accuracy. PTQ serves as the initial candidate because it operates over frozen weights with zero retraining cost; QAT introduces simulated quantization into the training loop when quality recovery demands backpropagation; specialized sub-4-bit and binary formats require custom kernel execution and extensive hardware-specific validation. Representation choices such as INT8 versus INT4, per-channel scaling, and mixed precision apply across these stages rather than defining separate maturity levels.

Post-training quantization is evaluated first, because it changes representations with minimal compute and pipeline overhead. When PTQ satisfies the application's latency, energy, and quality thresholds, the optimization process stops. When post-training calibration introduces unacceptable accuracy degradation, quantization-aware training lets the weights adapt to discretization noise during backpropagation, and extreme sub-4-bit or ternary representations are reserved for targets that INT8 and INT4 cannot meet and whose hardware has kernels for them.

10.3.3.1 Post-training quantization

Retraining a large model checkpoint solely to adapt its weights to 8-bit arithmetic can cost hundreds of accelerator hours, and in many deployment pipelines the original training dataset or training cluster is not even available. **Post-training quantization (PTQ)** avoids that retraining loop altogether by converting weights and activation tensors from floating-point to fixed-point formats directly on a frozen checkpoint (Choukroun et al. 2019). Weight-only quantization requires only a single pass over static parameter tensors, while full integer quantization calibrates activation dynamic ranges using a small, unlabelled sample of representative inputs.

The principal constraint in PTQ is unmodeled discretization noise. Rounding introduces numerical error that cannot be compensated by gradient updates. While redundant architectures absorb 8-bit rounding noise with negligible accuracy loss, sensitive models or lower bit widths (such as INT4) experience degradation unless scaling granularities are refined or quantization-aware training is applied. Production runtimes such as TensorFlow Lite, Open Neural Network Exchange (ONNX) Runtime, and TensorRT provide automated calibration pipelines to profile tensor ranges and apply this conversion.

The core mechanism of PTQ is **uniform quantization**, which maps floating-point values to discrete integer levels using a consistent scaling factor. Because the interval between each quantized value is constant, uniform quantization simplifies implementation and enables efficient hardware execution. For the symmetric form shown here, s is chosen from the maximum absolute value so that the resulting integers stay within the target range. The quantized value x_q is computed as:

$$x_q = \text{round}\left(\frac{x}{s}\right)$$

where:

- x_q is the quantized integer representation,
- x is the original floating-point value,
- s is a scaling factor that maps the floating-point range to the available integer range.

Uniform quantization from FP32 to INT8 reduces the raw value payload from 32 to 8 bits per weight at the cost of a measurable quantization error, with scale metadata and packing overhead excluded (listing 10.2). Once a model is quantized, the runtime can use integer arithmetic where supported (Gholami et al. 2022).

Listing 10.2: Uniform Quantization: Converts FP32 weights to INT8, reducing raw per-weight payload from 32 to 8 bits while measuring quantization error; scale metadata and packing overhead are excluded.

```
import torch

# Original FP32 weights
weights_fp32 = torch.tensor(
    [0.127, -0.084, 0.392, -0.203], dtype=torch.float32
)
print(f"Original FP32: {weights_fp32}")
print(f"Memory per weight: 32 bits")

# Simple uniform quantization to INT8 (-128 to 127)
# Step 1: Find scale factor
max_val = weights_fp32.abs().max()
scale = max_val / 127  # 127 is max positive INT8 value

# Step 2: Quantize using our formula x_q = round(x/s)
weights_int8 = torch.round(weights_fp32 / scale).to(torch.int8)
print(f"Quantized INT8: {weights_int8}")
print(f"Memory per weight: 8 bits (reduced from 32)")

# Step 3: Dequantize to verify
weights_dequantized = weights_int8.float() * scale
print(f"Dequantized: {weights_dequantized}")
print(
    f"Quantization error: "
    f"{(weights_fp32 - weights_dequantized).abs().mean():.6f}"
)
```

Rounding to a uniform grid introduces an error with a precise bound. A uniform quantizer maps each value to the nearest of evenly spaced levels separated by a step size s (range calibration derives s from the tensor's clipping range in equation 10.1, later in this section). For any value inside the clipping range, rounding to the nearest level misses by at most half a step, so within-range error is bounded by $[-s/2, s/2]$. Aggregated across many tensors, most error concentrates inside that bound, while the tails carry the larger distortion that comes from saturation clipping and from mixing tensors that use different step sizes (figure 10.19). Many CNNs tolerate INT8 well, whereas large transformers often need outlier-aware or mixed-precision schemes because a few activation channels carry extreme values. Selecting the appropriate numerical precision therefore requires balancing accuracy constraints, hardware support, and efficiency gains.

An alternative, nonuniform quantization, assigns finer-grained precision to numerical ranges that are more densely populated, which can preserve accuracy for models whose weight distributions concentrate around specific values. Nonuniform schemes require more complex calibration and are less common in production, but they can be effective for models particularly sensitive to precision changes. More generally, sensitivity to PTQ varies by architecture, layer, and distribution. When a uniform configuration misses the quality target, the next options include finer granularity, mixed precision, weight-only methods, better calibration, or QAT.

Range calibration Mapping a tensor onto 8-bit or 4-bit integers with an ill-fitting range either saturates critical values through clipping or squanders available integer levels on sparse outliers. Quantization-range calibration prevents both failures by estimating the real-valued clipping range

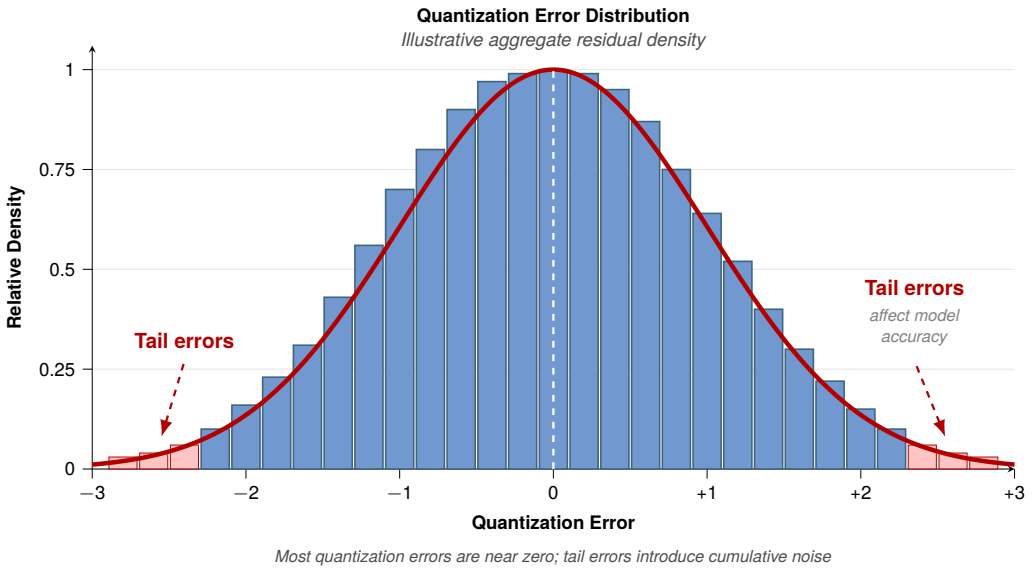


Figure 10.19: Illustrative Aggregate Quantization Residuals: Illustrative aggregate residual distribution across heterogeneous scales and clipped values. Within one unclipped uniform quantizer, rounding error is bounded by $[-s/2, s/2]$ for step size s; the displayed tails represent clipping or aggregation across different step sizes.

$[\alpha, \beta]$ for each tensor (from here on, α and β denote its lower and upper bounds). Calibration minimizes representation error over the sampled distribution, although it does not guarantee bounds on out-of-distribution deployment inputs.

Because activation tensors vary with input data, profiling requires running an unlabelled calibration dataset through the forward pass to capture dynamic ranges. In the offline conversion pipeline, the pretrained model and a representative calibration dataset pass through a profiling stage that establishes dynamic ranges, and those ranges parameterize the quantization step that yields the final quantized model (figure 10.20). Layer by layer, activation observers record value distributions from which the scale and zero-point metadata are computed (algorithm 10.1). Range calibration in this sense is distinct from the confidence calibration that asks whether a model's predicted probabilities match its observed accuracy, which returns in section 10.6.

Algorithm 10.1 Post-training quantization calibration

Require: pretrained model f_θ; representative calibration set $\mathcal{C}_{\text{cal}}$; bit width b; granularity (layer/group/channel); method (max, entropy, percentile)
Ensure: quantized weights and per-group range metadata $\{(\alpha_g, \beta_g, s_g, z_g)\}$

```
attach observers to weights and selected activation tensors, at the chosen granularity
for each batch in C_cal do ▷ calibration pass
    run f_θ in inference; record per-group value distributions
end for
for each quantization group g do
    select clipping range [α_g, β_g] by the chosen method
    derive scale s_g, zero-point z_g mapping [α_g, β_g] into the b-bit integer range
end for
quantize static weights; export activation range metadata for inference
if accuracy falls below the production threshold, widen ranges, change granularity, or move to QAT
```

The single calibration pass over $\mathcal{C}_{\text{cal}}$ avoids retraining, and FP32-to-INT8 weight quantization cuts raw stored weight values by 4× before scale, zero-point, and packing metadata. The price is

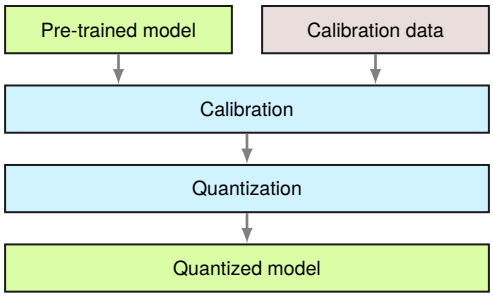


Figure 10.20: Post-Training Quantization Pipeline: PTQ converts a trained FP32 model directly to INT8 without retraining. Representative calibration data passes through the model to record activation ranges $[\alpha, \beta]$, computing scale factor s and zero-point z parameters to quantize parameters before deployment.

that the static activation ranges fixed in the loop risk saturation or wasted integer levels when the calibration data misses deployment tails. The quantization step then converts model parameters to the lower-precision format, producing the final quantized model.

Mapping an unnecessarily wide real-valued interval to the 256 discrete bins of an 8-bit integer wastes numerical resolution on empty range, whereas clipping too aggressively truncates signal. Three calibration methods balance this clipping-versus-resolution trade-off: **max** sets bounds to the absolute extremum (preserving extreme values at the cost of vulnerability to outliers), **entropy** selects bounds by minimizing the Kullback–Leibler (KL) divergence between original and quantized distributions, and **percentile** retains a chosen fraction (such as 99.9 percent) and clips the remaining tail. In one ResNet-50 layer, the dense bulk of activations clusters below 1.1, while a sparse outlier tail stretching past 2.0 motivates testing a candidate clipping boundary near 1.1 to allocate more INT8 bins to the bulk (figure 10.21). Choosing a percentile boundary requires cumulative activation counts; the illustrated boundary is not assigned a percentile rank. Task validation must establish whether clipping that tail preserves important rare cases.

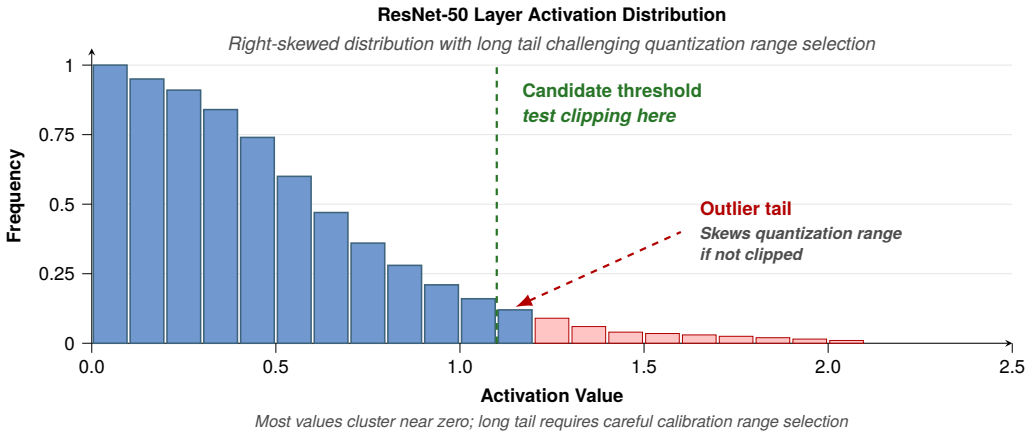


Figure 10.21: Activation Distribution: ResNet-50 layer activations have a long tail, with outlier values that can lead to inefficient precision use if not handled carefully. Source: (Wu et al. 2020).

Tuning quantization ranges Once calibration identifies clipping boundaries $[\alpha, \beta]$, the range must be mapped onto integer endpoints, and two mapping schemes dominate (figure 10.22). A **symmetric** mapping uses a zero-centered range with zero-point zero, forcing real zero to integer zero by mapping $[-1, 1]$ directly onto $[-127, 127]$. An **asymmetric** mapping keeps one scale but a generally nonzero zero-point z, which accommodates skewed activation intervals such as $[-0.5, 1.5]$ across $[-128, 127]$.

Symmetric quantization simplifies integer matrix multiplication kernels because the zero-point is zero ($z = 0$), eliminating cross-term additions during hardware accumulation. However, for

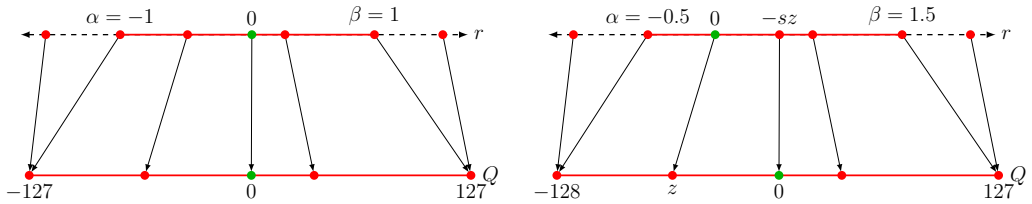


Figure 10.22: Calibration Range Selection: Symmetric vs. asymmetric affine quantization mappings. Symmetric quantization (left) forces real zero to map to integer zero, using equal-magnitude negative and positive clipping bounds at the expense of wasted dynamic range for skewed activation distributions; asymmetric quantization (right) uses an explicit zero-point offset z to map non-centered ranges $[\alpha, \beta]$ efficiently.

activations following asymmetric distributions (such as post-ReLU feature maps where all values are non-negative), symmetric quantization wastes roughly half of the integer dynamic range on unused negative values. Asymmetric quantization recovers this dynamic range by mapping $[\alpha, \beta]$ to $[0, 2^b - 1]$, but requires runtime hardware to compute offset corrections ($z \sum_k W_{ik}$ for output row i) during multiply-accumulate operations. Either mapping is fully specified by two numbers per range, a scale and a zero-point, and the uniform quantizer introduced for PTQ is the special case whose zero-point is zero.

Napkin Math 10.1: Calculating scale and zero-point

The affine quantization formula: An affine map sends a floating-point range $[\alpha, \beta]$ that includes zero to an integer range $[0, 2^b - 1]$ (for example, UINT8) via the following construction.

Analysis: Affine quantization establishes a linear mapping $x \approx s(x_q - z)$, where s is the **scale** (step size) and z is the **zero-point** (integer value corresponding to real zero). The mapping involves three computational steps:

1. **Calculate scale** (s): Divide the real range by the integer range using equation 10.1.

$$s = \frac{\beta - \alpha}{2^b - 1} \tag{10.1}$$

2. **Calculate zero-point** (z): Shift the range so that real zero maps to an integer using equation 10.2.

$$z = \text{round}\left(\frac{-\alpha}{s}\right) \tag{10.2}$$

3. **Quantize** $(x \to x_q)$: Divide by the scale, add the zero-point, then round and clamp to the integer range using equation 10.3.

$$x_q = \text{clamp}\left(\text{round}\left(\frac{x}{s} + z\right), 0, 2^b - 1\right) \tag{10.3}$$

Scenario: Consider quantizing a 2×2 activation matrix with values ranging from $\alpha = -1$ to $\beta = 3$, and the target precision is UINT8 $(b = 8)$:

$$\mathbf{X} = \begin{pmatrix} -1 & 1 \\ 0 & 3 \end{pmatrix}$$

- **Range**: $\beta - \alpha = 4$.
- **Steps**: $2^8 - 1 = 255$.
- **Scale**: $s = \frac{4}{255} \approx 0.0157$.
- **Zero-point**: $z = \text{round}(-(\alpha)/s) = \text{round}\left(-(-1)/\frac{4}{255}\right) = \text{round}(63.75) = 64$.

Applying equation 10.3 element-wise maps each floating-point entry to an integer bin:

$$\mathbf{X}_q = \begin{pmatrix} 0 & 128 \\ 64 & 255 \end{pmatrix}$$

Systems insight: The real value 0.0 is represented exactly by the integer 64. This is critical because it ensures that zero-padding (common in CNNs) is represented exactly, preventing "quantization drift" where padding introduces nonzero noise.

Granularity In convolutional filters or transformer projection matrices, weight magnitudes often vary widely from one channel to the next, so sharing a single scale factor across an entire tensor can collapse small-magnitude channels to zero or clip wider channels. Quantization granularity governs how broadly scale factors and zero-points are shared across a tensor, balancing representation resolution against metadata overhead and kernel complexity: across convolutional channels (filters 1 through d_{out}), a global layerwise threshold spans the widest distribution at the cost of resolution in narrower filters, whereas per-channel clipping boundaries adapt tightly to each filter's dynamic range at the cost of tracking additional scale vectors during matrix multiplication (figure 10.23).

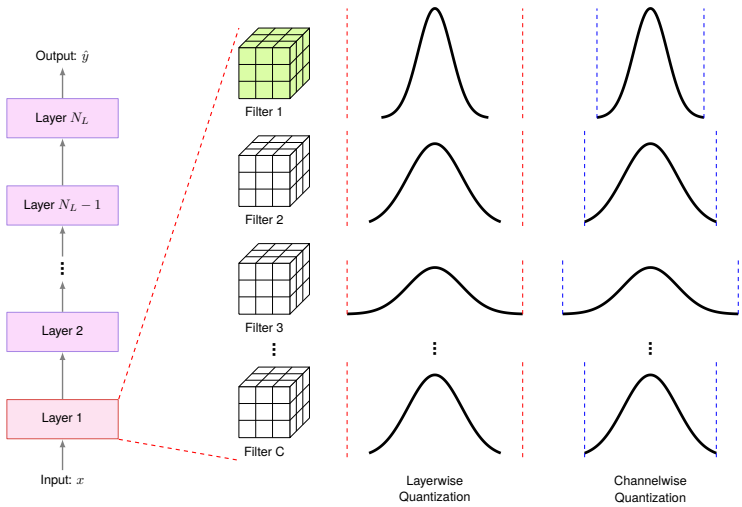


Figure 10.23: Quantization Range Variation: Different convolutional filters can have different weight ranges, motivating per-filter quantization to reduce quantization error. The differing clipping scales illustrate the choice between one layerwise range and separate channelwise ranges. Source: (Gholami et al. 2022).

Quantization granularity trades calibration and metadata overhead against representation error. More local ranges can preserve information when channels differ, but they do not improve accuracy automatically and may lack an efficient kernel path. Each additional scale also has to be stored, loaded, and applied at the granularity supported by the backend. Four levels are common, from one shared layer range to local ranges within a filter (table 10.9).

Table 10.9: Quantization Granularity Levels: Granularity trades calibration and metadata overhead against representation error, from a single per-layer range to per-filter sub-ranges.

Level	Range sharing	Trade-off
Layerwise	One range per layer	Simple but suboptimal when filter ranges vary widely
Groupwise	Filters grouped with shared ranges	Used in Q-BERT (Shen et al. 2020) for transformer attention layers
Channelwise	One range per filter	Common default; balances quantization error against scale metadata and implementation overhead

Level	Range sharing	Trade-off
Sub-channelwise	Ranges within each filter	More local ranges can reduce quantization error but increase metadata and implementation overhead

Channelwise quantization preserves accuracy over layerwise quantization when channel dynamic ranges differ, at the cost of storing additional scale vectors and executing per-channel rescaling in the kernel pipeline. Beyond scale granularity, execution efficiency depends on the distinction between static weights and dynamic activations, which impose fundamentally different memory and latency constraints on the deployment system.

Weights vs. activations Executing low-precision inference requires transforming floating-point inputs before kernel execution and requantizing intermediate accumulator outputs. During integer matrix multiplication, multiplying two 8-bit integers produces a 16-bit product. Summing these products across an inner dimension of length K causes bit-growth that quickly overflows 8-bit registers; hardware therefore accumulates into 32-bit integer registers (`INT32`). Before feeding the result into subsequent layers, requantization rescales the 32-bit accumulated sum by the layer's effective scale factor ($s_{\text{eff}} = \frac{s_W \cdot s_X}{s_Y}$) and clamps the values back into the 8-bit integer range. The full pipeline therefore has four stages (figure 10.24): quantizers convert floating-point activations and weights into INT8 operands, an integer core executes the matrix multiplication, a 32-bit accumulator holds the sum, and a requantization stage returns the result to INT8 before the nonlinear activation.

↪ How a single Tensor Core matrix instruction takes low-precision input tiles into higher-precision accumulator registers is developed in section 11.4.2.

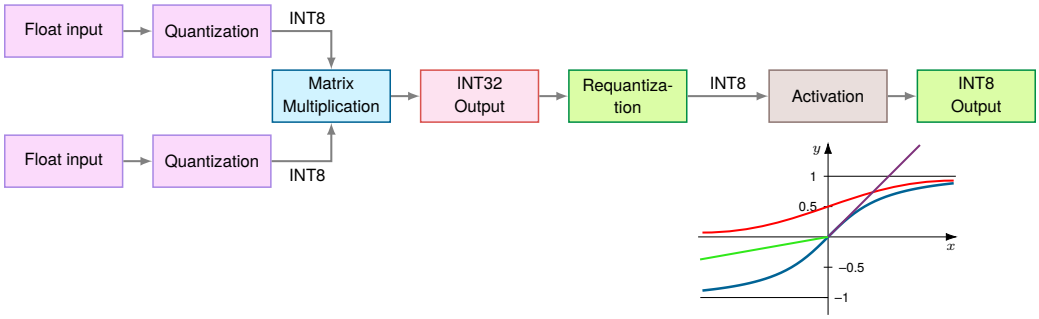


Figure 10.24: Quantized Matrix Multiplication Pipeline: Floating-point weights and activation inputs are quantized to INT8 (violet), fed into integer SIMD/Tensor Core matrix multiplication (blue), accumulated in 32-bit registers (red), and requantized back to INT8 (green) before non-linear activation. From the HarvardX TinyML course.

Activation quantization maps layer outputs to a lower-precision representation during inference. It reduces intermediate payloads and can reduce arithmetic cost on hardware with an efficient integer path, but it also introduces error between layers. For example, a CNN may convert FP32 feature maps to INT8 before later convolutions. The benefit is greatest when consecutive operators remain on the integer path; isolated quantized operators may pay conversion cost without reducing enough work. Whether activation quantization improves latency therefore depends on supported kernels, scale handling, tensor lifetimes, and any quantize or dequantize boundaries that remain in the exported graph.

Static vs. dynamic quantization The clipping range also depends on *when* activation ranges are computed, which divides activation quantization into two primary execution models: static quantization and dynamic quantization. In **static quantization**, the clipping range is precalculated during offline calibration and remains fixed during inference. This approach introduces zero runtime range-estimation overhead, because inputs are scaled by fixed factors and directly fed to integer execution units. However, static ranges risk saturation or underflow whenever production inputs deviate from the calibration distribution (Jacob et al. 2018; Yao et al. 2021).

Dynamic quantization instead calculates scaling factors and zero-points dynamically from runtime activation tensors. Adapting the clipping window to each input reduces mismatch with a fixed calibration range. Outliers can still consume resolution, and a policy that clips tails can still saturate values. The trade-off is runtime memory and latency overhead, because the hardware must perform

reduction passes over intermediate activation tensors to extract minimum and maximum values before integer arithmetic can begin. On memory-bandwidth-bound operators, this reduction pass can erode the execution advantage of lower precision.

Checkpoint 10.2: Calibration and range choices

Quantization succeeds only when the range policy matches deployment data and runtime constraints.

Range design

- ☐ **Calibration data**: explain how an unrepresentative calibration set creates clipping or wastes integer levels.
- ☐ **Granularity**: compare per-tensor and per-channel ranges by the error and metadata trade-off each creates.
- ☐ **Static vs. dynamic ranges**: select a range policy when activation distributions shift across inputs and runtime overhead is constrained.

PTQ in practice Post-training quantization balances deployment turnaround against accuracy control. PTQ requires no gradient backpropagation or retraining. Whether offline calibration is sufficient depends on the architecture, calibration data distribution, numerical quantizer design, and target hardware constraints.

The primary limitation is that PTQ freezes model parameters, preventing compensation for accuracy loss. When a quantized model falls below the production threshold, mitigations include recalibrating, refining scale granularity, adopting mixed precision, or escalating to quantization-aware training. QAT integrates precision constraints directly into the training graph using a **straight-through estimator (STE)** (section 10.3.3.2) to pass gradients through nondifferentiable rounding operators. This enables weights to adapt to low-bit representations during retraining, recovering accuracy at the cost of additional training compute (O_{train}) and hyperparameter tuning.

10.3.3.2 Quantization-aware training

When post-training quantization drops a sensitive model or a 4-bit target below its accuracy floor because frozen weights cannot adjust to coarse rounding steps, only gradient updates can recover the lost quality. Quantization-aware training incorporates numerical rounding noise directly into the optimization graph, retraining or fine-tuning a quantized checkpoint on the training dataset so its parameters adapt to low-precision discretization error (figure 10.25).

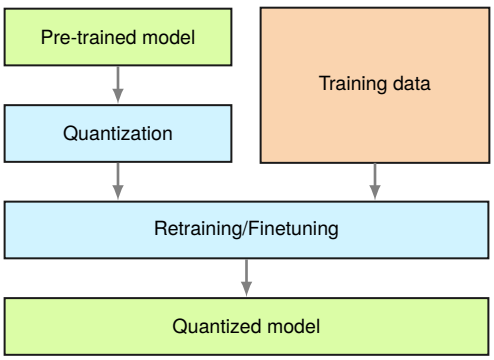


Figure 10.25: Quantization-Aware Training Pipeline: QAT inserts fake-quantization nodes into the forward graph to simulate low-precision clipping and rounding errors during training. Backpropagation updates floating-point master weights using straight-through estimators (STE), allowing network parameters to adapt to numerical noise before exporting fixed integer scales for inference.

Rather than training from scratch or cold-starting scale parameters, production workflows frequently initialize QAT using scale factors derived during an initial PTQ calibration pass. In this

two-phase pipeline, a PTQ phase feeds calibration data into the pretrained model to establish initial scale metadata, and the resulting PTQ checkpoint enters a QAT phase in which the full training data drives fine-tuning (figure 10.26).

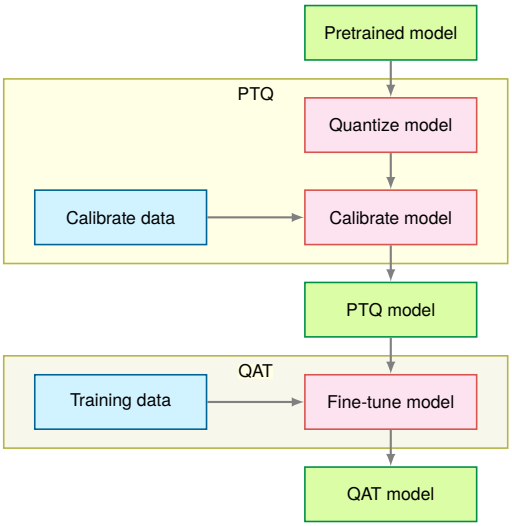


Figure 10.26: PTQ-to-QAT Pipeline: PTQ calibration can initialize ranges and scales before QAT fine-tunes the pretrained weights with simulated quantization. Calibration data feeds the PTQ stage, and training data feeds QAT.

Training mathematics During forward propagation, weights and activations are quantized and dequantized to mimic reduced precision. Let x be a full-precision value, s the scaling factor that maps floating-point values into a lower-precision range, and x_{fake} the simulated (quantized, then dequantized) value. Ignoring clipping and the zero-point for this simplified expression, the process is represented as:

$$x_{\text{fake}} = \text{round}\left(\frac{x}{s}\right) \times s$$

Although the forward pass executes with simulated quantization, computing gradients requires backpropagating through the rounding operator $\text{round}(\cdot)$. Because rounding is a piecewise-constant step function, its mathematical derivative is zero almost everywhere and undefined at step transitions, which would zero out gradients and halt gradient descent. The Straight-Through Estimator (STE) resolves this bottleneck by substituting the identity function for the rounding operator during backpropagation (Bengio, Léonard, et al. 2013).[16]

Integrating quantization effects during training lets the model adapt its weights and activation ranges to simulated low-precision error. QAT can recover quality that a post-training route loses, particularly at aggressive bit widths, but the advantage depends on the model, quantizer, calibration, and training procedure (Krishnamoorthi 2018).

Fake quantization nodes and implementation QAT implementation relies on fake quantization operations that simulate quantization during forward propagation while maintaining full precision for gradient computation. These operations insert quantize-dequantize pairs into the computational graph, creating a training-time simulation of inference-time behavior.

A fake quantization node must preserve the same errors the inference runtime will see, so its three operations model the deployment path during training:

1. **Quantization**: Map floating-point value to discrete quantization level
2. **Clipping**: Enforce range constraints based on bit width
3. **Dequantization**: Convert back to floating-point for subsequent operations

[16] **Straight-through estimator (STE)**: Proposed by Bengio, Léonard, et al. (2013), the common identity STE copies the upstream gradient through the quantizer unchanged. Because that identity is not the derivative of rounding, the estimate is biased, so QAT descends an approximation of the quantized model's loss surface rather than the surface itself.

Mathematically, for symmetric quantization with bit width b, given a floating-point input value x:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right), -2^{b-1}, 2^{b-1}-1\right)$$
$$x_{\text{fake}} = x_q \times s$$

where x_q is the quantized integer level, $s = \frac{\max(|x|)}{2^{b-1}-1}$ is the scale factor computed from the input distribution, and x_{fake} represents the fake-quantized output that mimics INT8 values but remains in floating-point format.

For asymmetric quantization supporting unsigned integers, the node applies the scale, zero-point, and quantize steps of equation 10.1, equation 10.2, and equation 10.3 to the observed range (extended to include zero) and then dequantizes:

$$s = \frac{\max(x) - \min(x)}{2^b - 1}$$
$$z = \text{round}\left(-\frac{\min(x)}{s}\right)$$
$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s} + z\right), 0, 2^b - 1\right)$$
$$x_{\text{fake}} = (x_q - z) \times s$$

where z is the zero-point offset enabling asymmetric range representation and the observed range $[\min(x), \max(x)]$ serves as the clipping range $[\alpha, \beta]$.

In the QAT forward path, fake quantization is applied to inputs and weights before convolution, simulating INT8 numerics while retaining floating-point tensors for training (listing 10.3).

Listing 10.3: QAT Convolution Forward Pass: Fake-quantizes activations and weights before convolution to simulate deployment numerics during training.

```
# Forward pass with fake quantization
def qat_conv_forward(x, weight):
    # Fake quantize input activations
    x_scale = compute_scale(x, bits=8, symmetric=False)
    x_zero = compute_zero_point(x, x_scale, bits=8)
    x_quant = fake_quantize(x, x_scale, x_zero, bits=8)

    # Fake quantize weights (typically symmetric)
    w_scale = compute_scale(weight, bits=8, symmetric=True)
    w_quant = fake_quantize(weight, w_scale, zero=0, bits=8)

    # Convolution with fake-quantized values
    output = conv2d(x_quant, w_quant)
    return output
```

During backpropagation, fake quantization nodes approximate gradients through a clipping-aware straight-through estimator:

$$\frac{\partial x_{\text{fake}}}{\partial x} = \begin{cases} 1 & \text{if } x \in [\alpha, \beta] \\ 0 & \text{otherwise} \end{cases}$$

Within the clipping range $[\alpha, \beta]$, the estimator treats quantization as an identity mapping, passing the upstream gradient $\frac{\partial \mathcal{L}}{\partial x_{\text{fake}}}$ unchanged to master weight x. Beyond clipping boundaries, the derivative evaluates to zero, blocking gradients from saturated outliers that would otherwise destabilize optimizer updates.

In practice, frameworks like PyTorch and TensorFlow implement fake quantization as custom autograd operators whose forward pass performs the quantize-dequantize round trip while the backward pass applies an STE. Scale handling depends on the method, since observers may track distributions

and later freeze, or scales may be learned. Batch normalization also requires deployment-consistent handling. Workflows freeze batch-normalization statistics and fold them into the preceding convolution before export (section 10.4.1). Freezing running statistics before export allows fake-quantization scales to match the static fused parameters executed during inference.

QAT trade-offs QAT's[17] primary advantage is additional control over quality at low precision. By incorporating simulated quantization noise during training or fine-tuning, the model can adapt to the deployed numerical path. Processors with dedicated integer units can then exploit INT8 arithmetic for faster or lower-energy inference when the exported operators are supported (Wu et al. 2020; Gholami et al. 2022). The benefit must be measured against PTQ for the same model and target, including the extra training cost and any difference in calibration, graph coverage, or kernel selection.

The cost is additional engineering complexity. QAT inserts simulated quantization into training, so teams must validate quantization schemes, scale behavior, and accuracy recovery on the target model. For very large models whose training budgets are already constrained, that overhead can make QAT impractical, and post-training routes avoid retraining altogether (Choukroun et al. 2019). QAT is therefore warranted only when the exported PTQ model misses its quality or deployment targets and training resources permit. A hybrid approach, starting with PTQ calibration and applying QAT fine-tuning only to accuracy-critical layers, often provides a useful balance.

The choice among PTQ, QAT, and dynamic quantization comes down to engineering budget, accuracy threshold, and input variability (table 10.10). Dynamic range computation is a timing choice orthogonal to PTQ versus QAT (dynamic PTQ is common), but it belongs in the comparison because it answers the third of those questions.

[17] **Quantization-aware training (QAT)**: QAT simulates low-precision behavior during training or fine-tuning so weight updates can adapt to clipping and rounding error. BERT studies report that training-aware or Hessian-aware methods can remain closer to full-precision General Language Understanding Evaluation (GLUE) baselines than simpler post-training routes in their evaluated settings (Zafrir et al. 2019; Shen et al. 2020). The exact gap depends on model, task, quantizer, bit width, training budget, and evaluation distribution; a closer benchmark result is evidence for trying QAT, not a guarantee that its added training cost or exported runtime will meet a production target.

Table 10.10: Quantization Method Comparison: The choice turns on three binding questions, namely engineering budget, accuracy threshold, and input variability, rather than on a feature checklist.

Method	Engineering effort	Accuracy preservation	Input adaptability	Reach for it when
Static PTQ	Low: no retraining	Model-dependent; parameters do not adapt	Fixed calibration range	a calibrated model meets the measured accuracy and performance targets
Static QAT	High: additional training with quantization simulation	Often stronger when PTQ falls short	Fixed calibration range	PTQ misses the accuracy target and the training budget allows
Dynamic quantization	Moderate: ranges recomputed at runtime	Model-dependent; range fits each input	Per-input range	activation ranges vary widely across inputs and runtime overhead is acceptable

PTQ serves as the low-effort baseline; QAT spends training cycles when PTQ degrades task accuracy, while dynamic quantization expends runtime memory traffic and compute to track input-dependent dynamic ranges. In practice, PTQ and QAT commonly target 8-bit or 4-bit precision, where retained accuracy depends on the model, task, and quantizer. Some deployment scenarios, however, demand even more aggressive compression, pushing precision to the absolute limits of what neural networks can tolerate.

10.3.4 Extreme quantization

On a microcontroller whose runtime buffers and audio feature extraction leave barely 100 KB of SRAM for weights, or on an energy-harvesting sensor whose power envelope cannot afford hardware multipliers, even 8-bit and 4-bit integers exceed the deployment budget. Extreme quantization pushes numerical precision down to 1 or 2 bits per value for those targets (Courbariaux et al. 2015; Zhu et al. 2017). Binarization constrains weights and activations to two values, typically encoded as $\{-1, +1\}$. In digital logic, mapping -1 to binary 0 and $+1$ to binary 1 renders scalar multiplication isomorphic to the bitwise `XNOR` operation, replacing floating-point or integer MAC units with bitwise logic and population count (`popcount`) instructions (Rastegari et al. 2016). Because a 64-bit register packs 64 binary values, a single 64-bit `XNOR` instruction computes 64 binary products in parallel

and a hardware `popcount` instruction counts the matching bits to accumulate the dot product $(2 \times \text{popcount}(\mathbf{w} \text{ XNOR } \mathbf{a}) - 64)$, eliminating hardware multipliers altogether. That efficiency comes at a steep cost in expressiveness, since 1-bit representations degrade accuracy sharply on complex perceptual and language tasks (Hubara et al. 2018).

Ternarization mitigates this capacity loss by allowing three values: $\{-1, 0, +1\}$ (Zhu et al. 2017). The zero state introduces sparsity and simplifies arithmetic. Multiplying an activation by $+1$ requires an addition, multiplying by -1 requires a subtraction, and multiplying by 0 is skipped entirely. This eliminates general-purpose multipliers, reducing the arithmetic unit to a conditional adder-subtractor accumulator. However, ternary values cannot be packed at one bit per value. They require at least two bits per weight in standard byte-aligned memory (wasting one of four possible 2-bit states) or multi-trit encoding (such as packing five trits into a single byte, since $3^5 = 243 < 256$) that introduces runtime decoding overhead. The arithmetic gains of multiplier-free execution must outweigh the instruction overhead required to decode and unpack the weights on the target architecture.

Both binary and ternary quantization introduce step functions whose derivative is zero almost everywhere ($\frac{d}{dx}\text{sign}(x) = 0$ for $x \neq 0$), causing backpropagation gradients to vanish. To train these representations, quantization-aware training again relies on the straight-through estimator (Bengio, Léonard, et al. 2013; Choi et al. 2018) of section 10.3.3.2, whose gradients reach full-precision latent weights that accumulate small optimizer steps until they cross the quantization thresholds.

10.3.4.1 Challenges and limitations

Translating sub-byte precision into actual wall-clock speedups is constrained by standard processor architectures. Native support for binary or two-bit matrix operations varies across processors and kernel libraries. On a target without a matching sub-byte path, executing these representations requires bit-shifting, masking, or unpacking routines. The instruction overhead of this bit manipulation frequently offsets memory-bandwidth savings unless workloads run on custom bit-serial accelerators, field-programmable gate arrays (FPGAs), or specialized hardware with native bitwise execution units (Umuroglu et al. 2017).

Model quality is the second barrier. Extreme quantization sharply reduces the representational capacity of a network. While low-entropy workloads such as KWS on a microcontroller SRAM budget may tolerate binary or ternary weights, complex perceptual and generative models suffer steep accuracy degradation. Binary and ternary quantization become viable candidates only when higher-precision alternatives such as INT8 or INT4 fail to satisfy physical memory or energy budgets, and when the deployment hardware can execute bit-packed operations natively (Rastegari et al. 2016; Hubara et al. 2018; Zhu et al. 2017; Umuroglu et al. 2017). Production deployment requires evaluating binary and ternary baselines against INT8, INT4, structured pruning, and compact architectural redesigns on the identical workload, measuring total binary artifact size, end-to-end latency, energy per inference, and task accuracy on target hardware. Whether a narrower format actually runs faster is the last question the precision tier must answer.

10.3.5 When fewer bits run faster

Shrinking weights from 16 bits to 8 bits fails to speed up a compute-bound matrix multiplication if the processor lacks a faster 8-bit datapath, just as faster integer ALUs fail to speed up autoregressive decoding when latency is stalled on memory-bus transfers. Every byte that quantization removes is a byte the memory system no longer moves, so whether a quantized model runs faster depends on which hardware resource binds: a compute-bound matrix multiplication speeds up only when the hardware provides a higher-throughput datapath for the narrower format, whereas a bandwidth-bound decode speeds up in proportion to the bytes removed from each token's traffic. The worked examples below bracket the payoff, first on the reference H100 for a compute-bound layer and then on the running case's phone for GPT-2 XL decoding.

Napkin Math 10.2: The quantization speedup (compute bound)

Problem: A compute-bound matrix multiplication (for example, in a transformer multilayer perceptron block at a large batch) switches from FP16 to INT8 on the reference H100. What throughput speedup can the integer matrix units deliver?

Math: On hardware with dedicated INT8 units:

1. **Integer throughput path**: The H100 rates its INT8 matrix path at 1979 TOPS against 989 TFLOP/s on the dense FP16 path (NVIDIA Corporation 2022; Choquette 2023). The peak throughput increase is that spec ratio, about 2×.
2. **Memory bandwidth**: Packed INT8 weights carry half the raw weight bytes of FP16. Their streaming time can approach half at the same achieved bandwidth, before metadata and conversion costs.
3. **Combined effect**: For compute-bound operations, the ideal throughput factor is about 2× at comparable utilization of the two arithmetic paths.

Systems insight: The speedup from quantization depends on the bottleneck. Compute-bound operations (large batch sizes, high arithmetic intensity I) can approach about 2× at comparable utilization of supported INT8 units, where the gain comes from the integer matrix path rather than reduced memory traffic. The bandwidth-bound case inverts this. Halving the bytes moved (FP16 to INT8) yields up to 2×, and larger bit-width reductions scale the gain further, as the decode example below traces.

Processors without dedicated matrix units see the same compute-bound effect through data-level parallelism in their registers, the path a phone's CPU takes before a model moves to its integer accelerator. Fixed-width vector execution units (SIMD) and systolic arrays process more operands per cycle when each operand occupies fewer bits.

Napkin Math 10.3: The SIMD multiplier

Problem: A compute-bound layer executes using INT8 rather than FP32 on the same processor. Determine the architectural source of arithmetic speedup under fixed register width.

Mechanism: SIMD execution. A CPU or GPU core processes data in fixed-width vector registers (for example, AVX-512 is 512 bits wide).

Math:

1. **Register width**: 512 bits.
2. **FP32 capacity**: 512/32 = 16 elements per vector instruction.
3. **INT8 capacity**: 512/8 = 64 elements per vector instruction.

Result: Switching to INT8 packs 4× more elements into the same register. Throughput Gain = INT8 elements/inst/FP32 elements/inst = 64/16 = 4×

Systems insight: The packing ratio gives an ideal 4× throughput factor, assuming comparable vector-instruction throughput, occupied lanes, and supported operators. It is a candidate bound for compute-bound layers, not a measured layer speedup; memory traffic and conversion overhead still affect execution.

When an operation is bandwidth bound, speedup tracks the reduction in bytes transferred per token rather than peak arithmetic throughput. In autoregressive token generation with small batch sizes, memory bus traffic dominates latency.

Napkin Math 10.4: The quantization speedup (bandwidth bound)

Problem: The running case's phone decodes GPT-2 XL (1.5B parameters) at batch size one with a full 1,024-token context. Its memory bus delivers 51.2 GB/s, and each token must read every weight and the whole KV cache. How much faster does decoding run with INT4 weights than with FP16 weights?

Math (FP16 weights):

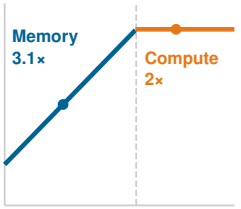
1. **Weights**: 3 GB at two bytes per parameter.
2. **KV cache**: Keys and values across 48 layers, 25 heads, and 64 dimensions per head, stored in FP16 for 1,024 tokens, occupy 0.31 GB.
3. **Total per token**: 3.3 GB, which fits the phone's 8 GB but takes 65 ms to stream, or about 15 tokens/s.

Fix (INT4 weights):

1. **Weights**: 0.75 GB at half a byte per parameter, for 1.06 GB per token with the unchanged FP16 KV cache.
2. **Speed**: Streaming that total takes 21 ms, or about 48 tokens/s.

Systems insight: INT4 weights yield a 3.1× speedup in this traffic model, below the 4× weight reduction because the FP16 KV-cache traffic is unchanged. The model already fit in memory at FP16; quantization bought speed, not feasibility. The realized speedup still depends on a kernel that unpacks INT4 weights inside the matrix multiplication rather than in a separate pass.

Across three vision networks (Inception v3, MobileNet v1, and ResNet v2), INT8 conversion contracts weight storage by exactly 4× (135 MB to 33.75 MB for the illustrative Inception bar), while the latency change varies with the architecture and its kernels (figure 10.27).



Quantization speedup depends on whether memory bandwidth or compute throughput is binding.

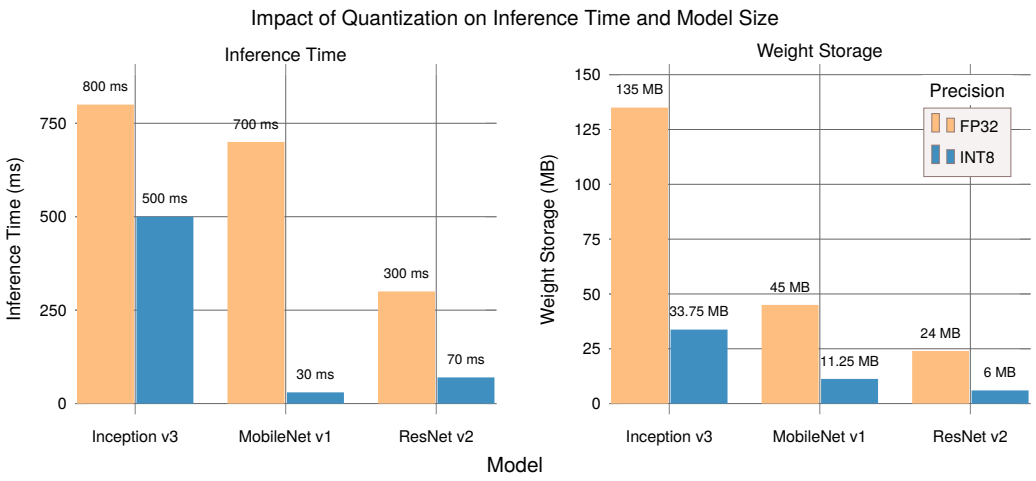


Figure 10.27: Illustrative Quantization Scenario: Paired bars separate FP32 and INT8 alternatives. The right panel shows illustrative weight storage, where reducing each weight from 32 to 8 bits produces an exact fourfold decrease; the left panel shows illustrative, architecture-dependent latency changes.

Activation-aware methods such as **activation-aware weight quantization (AWQ)**[18] target weight traffic in large language model (LLM) inference. This approach serves exactly the regime of GPT-2 XL decoding on the phone, where loading model weights limits the token rate. By protecting a small fraction of salient weight channels based on activation magnitude, AWQ supports INT4-class weight quantization while controlling task degradation. Packed low-bit weights can reduce the memory

[18] **Activation-aware weight quantization (AWQ)**: Salience is determined by activation magnitude, not weight magnitude, a distinction that matters because a small weight multiplied by a large activation produces a large output contribution. AWQ protects about 1 percent of salient weight channels through activation-aware scaling while quantizing most weights to low-bit formats, so GPT-2 XL's 3 GB of FP16 weights can move toward an INT4-class footprint of about 0.75 GB before metadata, scales, and kernel-specific packing overheads (Lin et al. 2024).

traffic required during token generation when the runtime provides a matching kernel (Lin et al. 2024).

Checkpoint 10.3: Quantization and precision checkpoint

Quantization design review criteria:

- ☐ **INT8 memory vs. energy**: Explain why INT8's per-operation energy saving can exceed 4× while its end-to-end energy saving is usually smaller.
- ☐ **QAT vs. PTQ**: Evaluate the architectural advantages of quantization-aware training over post-training quantization, identifying the accuracy and retraining cost trade-offs that justify auxiliary optimization.
- ☐ **Bit-width speedup**: For GPT-2 XL decoding on the phone, estimate FP16 versus INT4 weight bytes and the ideal weight-traffic speedup bound. Identify the conditions under which weight-traffic reduction approaches a proportional latency gain, and explain how KV traffic or a separate dequantization pass can limit that gain.
- ☐ **Compute versus bandwidth**: Explain why the same FP16-to-INT8 change can roughly double throughput on a compute-bound layer and halve time on a bandwidth-bound one for different reasons.

These speedups are bounds computed from bytes and peak rates. Whether a deployed model reaches them depends on how its compressed operations are lowered onto kernels, which is the third tier of the stack.

10.4 Hardware Implementation: Changing How the Model Executes

Pruning a model structurally to half its weights and quantizing the survivors from FP32 to INT8 cuts stored weight bytes by 2× and 4×, or 8× in all, yet in production such a model can easily run only 1.5× faster—roughly 18.8 percent of the paper projection. Profiling reveals where the missing speedup vanishes: sparse tensors continue to stream through dense kernels that cannot skip unindexed zeros, reduced-precision operators trigger unfused type conversions that flood the memory bus, and small layers spend more time launching kernels and moving intermediates than doing arithmetic. Paper metrics—parameter counts, theoretical FLOPs, and nominal bit depths—establish upper bounds that hold only when a workload is limited purely by weight traffic and every kernel exploits the new representation. Bridging the divide between paper reduction and measured hardware speedup is the job of the third tier of the stack, efficient **hardware implementation**.

The measured bottleneck selects the implementation technique. Kernel fusion consolidates memory-bound operator chains, custom dequantization loops keep packed low-precision weights inside cache, and hardware sparsity engines align with tensor layouts to bypass zeroed arithmetic.

10.4.1 Compression-specific fusion

In the camera pipeline's MobileNetV2, every inverted residual block chains a convolution, a batch normalization, and a rectified linear unit (ReLU); executed as separate kernels, each operator writes its output to accelerator global memory and the next reads that output back, paying three memory round-trips for work that could stay in on-chip registers. **Kernel fusion** (or operator fusion) eliminates those redundant memory transactions by combining adjacent operators into a single kernel (Chen et al. 2018; NVIDIA 2024c), and model compression introduces fusion requirements of its own. Folding batch normalization into a pruned convolution, keeping a dequantize step inside the matrix-multiplication kernel that consumes it, and choosing which adjacent operators of a rewritten graph can share one kernel are all fusion decisions that arise once the graph has been compressed.

That read-compute-write cycle creates memory bandwidth bottlenecks for operations with low arithmetic intensity (FLOP/byte). The traffic rule for a fused chain, established in section 7.3.2, bounds what fusion can recover. Fusing a chain of n_{ops} operations over an intermediate tensor of M_{tensor} bytes cuts idealized read-and-write traffic from $2\,n_{\text{ops}}M_{\text{tensor}}$ to $2M_{\text{tensor}}$ bytes, because the fused kernel reads its input once and writes its final output once. Weights, auxiliary inputs, cache effects, and register spills are outside this simple bound, and data dependencies, tensor shapes,

and register pressure limit how much can legally be fused. Several inference patterns nevertheless approximate the bound closely enough for fusion to matter.

Convolution-BatchNorm-ReLU fusion

The common Conv-BN-ReLU pattern can replace three launches and intermediate round-trips with one fused path when the compiler can legally fold batch normalization and combine the activation (listing 10.4). Folding is compression-specific because it must happen after the convolution's channels and scales are final. Workflows fold batch normalization into preceding convolution weights ($\mathbf{W}_{\text{fused}} = \frac{\gamma}{\sigma_{\mathcal{B}}}\mathbf{W}$) where $\sigma_{\mathcal{B}}$ is the square root of the sum of running variance and the numerical stabilizer. Folding also adjusts the convolution bias, $\mathbf{b}_{\text{fused}} = \frac{\gamma}{\sigma_{\mathcal{B}}}(\mathbf{b} - \mu_{\mathcal{B}}) + \beta$, where $\mu_{\mathcal{B}}$ is the per-channel running mean and γ and β are the learned batch-normalization scale and shift. Batch normalization's per-channel scale thus multiplies into the surviving filters, and its shift becomes the convolution's bias, so a pruned or quantized network is folded once its structure stops changing.

Listing 10.4: Conv-BN-ReLU Fusion: Combining three operations into a single kernel reduces the idealized intermediate-tensor stream from 6 transfers to 2; weights, auxiliary inputs, and spills are outside this count.

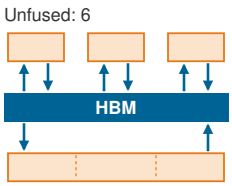
```
# Pseudocode: runtime APIs and fusion legality are
# implementation-dependent.
# === UNFUSED: 3 kernel launches, 6 memory transfers ===
conv_out = conv2d(input, weight)
bn_out = batch_norm(conv_out, ...)
relu_out = relu(bn_out)


# === FUSED: 1 kernel launch, 2 memory transfers ===
def conv_bn_relu_fused(input, weight, gamma, beta, mean, var):
    # Read input and weight once
    conv = conv2d(input, weight)

    # Apply batch norm in registers (no memory write)
    bn = gamma * (conv - mean) / sqrt(var + eps) + beta

    # Apply ReLU in registers (no memory write)
    output = relu(bn)

    # Write final result once
    return output
```



The idealized intermediate stream falls from six transfers to two.

↩ IO-aware attention tiling and its HBM-traffic accounting are established in section 8.6.4.

↪ The register pressure and accelerator-specific constraints that bound how far kernels can be fused on GPUs and TPUs are developed in section 11.7.7.1.

The intended function is preserved, although batch-normalization folding changes the arithmetic sequence, and the intermediate traffic falls by 3×. Generic bias-activation and attention fusion, by contrast, do not depend on compression and belong to the general fusion toolkit of chapter 7.

A ResNet-50 layer with 256 channels and a 28×28 spatial size puts numbers on this. Its feature map occupies 802.8 KB, so the four intermediate transfers that fusion removes amount to $4 \times 256 \times 28 \times 28 \times 4$ bytes $\approx$ 3.2 MB per layer. The full accounting also counts weights and parameters. Without fusion, each operation performs its own memory round-trip: Conv reads input (802.8 KB) plus weights (2.4 MB) and writes output (802.8 KB), totaling 4 MB. BN then reads that output, adds its parameters (2 KB), and writes again, for 1.6 MB. ReLU repeats the pattern for another 1.6 MB. The total unfused memory traffic is 7.2 MB. With fusion, the entire sequence reads input and weights once and writes the final output once, requiring only 4 MB, a 44.8 percent bandwidth reduction.

At the reference H100's 3,350 GB/s HBM bandwidth, these traffic volumes imply ideal lower bounds of 2 microseconds before fusion and 1.2 microseconds after fusion, a 1.81× ratio for the traffic component alone. These are not full layer latencies because convolution compute, cache effects, and launch overhead must also be included. Furthermore, fusion needs the computation captured as a graph, and dynamic control flow, in-place mutation, or unsupported operations can block the rewrite, so the exported plan must be inspected rather than assumed (graph capture is developed in section 7.3.6).

Quantization adds a fusion of its own. A weight-only INT8 or INT4 kernel must turn each packed integer code x_q back into a real value x before multiplying it, using the scale s and zero-point z of equation 10.1 and equation 10.2:

$$x = s(x_q - z) \tag{10.4}$$

Executed as a separate kernel, dequantization writes the full-precision weights back to memory and reads them again, restoring the traffic that quantization removed. Fused into the matrix-multiplication kernel, the dequantize step runs in registers on each tile as it arrives, so only the packed bytes cross the memory bus. That fusion is what lets GPT-2 XL's INT4 weights deliver the decode speedup computed in section 10.3.5. When a runtime cannot fuse it, the quantized model can run slower than its FP16 baseline, a failure section 10.5.4.2 examines. Fusion removes intermediate traffic between operations that must all run, leaving the complementary hardware question of how to skip operands that pruning has already set to zero.

10.4.2 Sparsity exploitation

Replacing pruned weights with zeros inside a standard dense matrix still fetches every zero from memory and multiplies it in the ALU, yielding no wall-clock speedup. Sparsity exploitation designs the storage layouts and kernels that translate **sparsity**[19]—tensors where a substantial fraction of entries are zero—into reduced memory traffic and execution time.

Formally, for a sparse weight matrix $\mathbf{W}_{\text{sparse}} \in \mathbb{R}^{m \times n}$, the nominal sparsity ratio ρ_{sparse} is:

$$\rho_{\text{sparse}} = \frac{\|\mathbf{1}_{\{(\mathbf{W}_{\text{sparse}})_{ij}=0\}}\|_0}{m \times n}$$

where $\mathbf{1}_{\{(\mathbf{W}_{\text{sparse}})_{ij}=0\}}$ is an indicator function that yields 1 if entry (i, j) is zero and 0 otherwise, and $\|\cdot\|_0$ counts nonzero entries. When magnitude pruning treats entries below δ_{prune} as removable, the thresholded sparsity ratio becomes:

$$\rho_{\text{sparse},\delta} = \frac{\|\mathbf{1}_{\{|(\mathbf{W}_{\text{sparse}})_{ij}| \leq \delta_{\text{prune}}\}}\|_0}{m \times n}$$

where δ_{prune} defines the pruning boundary. Zeros yield wall-clock acceleration only when the execution pipeline skips both the arithmetic and the associated memory traffic.

From an execution standpoint, the spatial pattern of zeros determines memory transaction efficiency. As established in section 10.2.1.2, unstructured sparsity zeroes arbitrary scalar connections, requiring pointer arrays and non-unit-stride gathers that degrade bandwidth on SIMD and systolic units. Conversely, structured patterns, such as channel pruning or $N{:}M$ constraints, preserve predictable memory layouts, aligning with standard dense kernels or dedicated hardware datapaths.

[19] **Sparsity**: From Latin *sparsus* (scattered), past participle of *spargere* (to scatter); in ML, ℓ_1 regularization such as the lasso can induce exact zeros rather than merely small values (Tibshirani 1996). Software can represent arbitrary sparsity patterns, while efficient hardware execution generally requires a supported format or structured pattern (for example, NVIDIA's 2:4 sparsity). The gap between representable and executable sparsity separates theoretical compression from realized speedup.

10.4.2.1 Sparsity utilization methods

Translating zeros into runtime speedup requires a representation that encodes nonzero positions and a kernel that decodes those coordinates to bypass unneeded computation (Hoefler et al. 2021). In a matrix-vector product, a dense $4{\times}4$ operator executes 16 multiply-accumulate operations, whereas a sparse execution path computes only the six nonzero products alongside the necessary metadata lookups:

$$\begin{bmatrix} 2 & 0 & 0 & 1 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 5 & 0 \\ 0 & 0 & 0 & 6 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} 2v_1 + v_4 \\ 3v_2 \\ 4v_1 + 5v_3 \\ 6v_4 \end{bmatrix}$$

The runtime benefit depends entirely on whether the latency eliminated by skipping 10 multiply-accumulate operations exceeds the latency added by coordinate decoding and non-contiguous activation loads. For unstructured sparsity, that trade has a computable answer in bytes and a measurable one in time.

10.4.2.2 Storage and execution break-even

A variant of the running case makes the byte side concrete. Suppose the camera team must also ship its MobileNetV2 to a wearable companion device whose microcontroller offers only 2 MB of flash memory, against the model's 14 MB of FP32 weights. Retraining a smaller architecture from scratch would require weeks of data collection and validation that the product schedule does not allow. Suppose profiling shows that about 85.7 percent of the model's weights are near zero and contribute little on the validation set. Removing those weights and fine-tuning the remainder for a few epochs leaves surviving weight values that total 2 MB with an acceptable accuracy loss. Whether the stored artifact also fits in 2 MB depends on how the surviving weights are stored. The numbers are a labeled scenario that anchors the engineering trade-off, not a universal MobileNet benchmark.

Storing only the surviving values requires an indexed sparse encoding such as **Compressed Sparse Row (CSR)** or coordinate list (COO). Because these formats must store positional metadata (such as column indices and row pointers) alongside the nonzero values, metadata overhead offsets savings at low sparsity. For example, storing a 16-bit floating-point weight with a 16-bit column index doubles the per-element byte footprint, requiring over 50 percent sparsity just to break even on artifact size (D_{vol}). Storing a dense tensor with zeroed entries saves zero memory. The wearable variant shows the consequence. Its surviving weights total 2 MB only if they cost nothing to locate. A 16-bit index beside each value adds half again to 32-bit values and doubles 16-bit ones, so the stored artifact misses the 2 MB flash budget. A denser position encoding, such as a bitmap, reduces the metadata overhead but cannot fit unchanged values that already consume the entire budget. Fitting requires further pruning or lower-precision values with room reserved for metadata, or structured removal of whole channels so that the remaining tensors can be stored without per-value indices.

↪ The memory footprint and index-pointer layouts for compressed sparse row (CSR) representations are collected for reference in section C.1.4.

Nor does unstructured sparsity translate directly to inference acceleration on modern hardware. Commodity accelerators achieve peak throughput (R_{peak}) through dense systolic arrays and wide SIMD execution units that rely on *memory coalescing*. Memory transfers use device-dependent transaction granularity, and caches store data in *cache lines*. Coalescing nearby, aligned accesses lets parallel threads share transactions more efficiently. A dense masked tensor still moves its zero entries. An indexed sparse encoding instead packs nonzero values, but Sparse Matrix-Dense Matrix Multiplication (SpMM) kernels must decode metadata and gather the matching activations. Irregular activation accesses, load imbalance, and kernel overhead can offset the skipped arithmetic even when stored weight values are contiguous. The performance break-even point depends on shape, format, batch size, and the available sparse kernel, so artifact reduction alone does not establish a speedup.

10.4.2.3 Structured patterns

Structured patterns resolve the break-even problem by constraining where zeros occur, and achieving actual speedups from them requires hardware that can efficiently skip zero-valued computations. Processor architectures differ in which patterns they exploit. Ampere-generation and later Sparse Tensor Cores exploit fine-grained 2:4 structured patterns, while systolic arrays require dense block structures. Libraries such as cuSPARSE reformulate sparse computations into patterns the hardware handles well, and MegaBlocks (Gale et al. 2022) recasts sparse mixture-of-experts training as block-sparse operations on dense tiles.

A sparse pattern earns hardware speedup only when it is regular enough for kernels to predict. Two formats make that regularity explicit: block sparse matrices and N:M sparsity patterns. Block sparse matrices group nonzeros into dense submatrices, so operations on the large sparse matrix become a smaller number of dense operations on submatrices. In the illustrated representation, 3×3 dense sub-blocks are indexed by an auxiliary coordinate array, so sub-block kernels can bypass the empty tiles (figure 10.28) (NVIDIA 2020a). The block size is a teaching choice. Actual speedup requires a block shape, layout, datatype, and kernel supported by the target library and hardware; regular blocks alone do not guarantee aligned memory transactions or Tensor Core execution.

Similarly, N:M sparsity retains at most n_{keep} nonzeros in each group of m_{group} consecutive elements; hardware-oriented pruning often enforces exactly n_{keep} retained values so every group has the same representation (Zhou et al. 2021). The deterministic format lets a supported kernel predict the value and metadata layout while retaining more capacity than removing whole channels or blocks. That

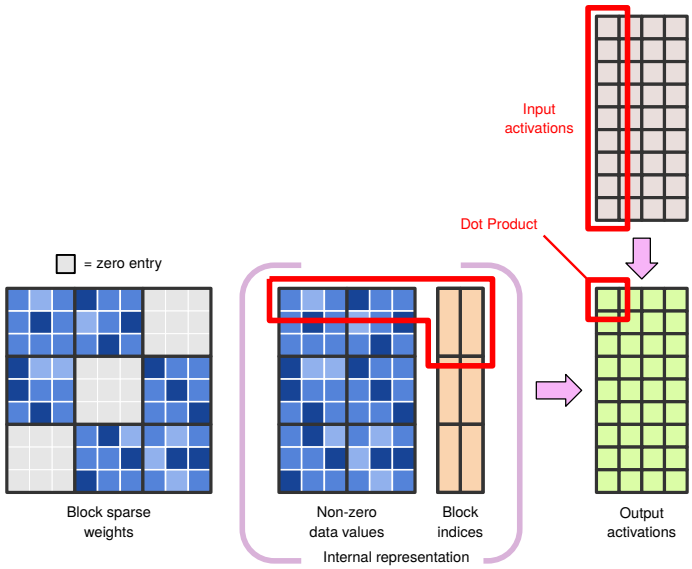


Figure 10.28: Block Sparse Representation: Grouping non-zero weights into dense sub-blocks and storing an auxiliary index array lets matching kernels skip empty blocks. The 3×3 tiles illustrate the representation; efficient GPU execution depends on supported block shapes, layouts, and kernels. Adapted from NVIDIA cuSPARSE documentation (NVIDIA 2020a).

regularity creates an executable compromise between arbitrary masks and dense computation, but only for targets with the matching path.

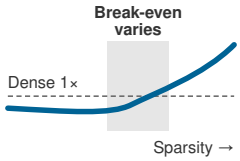
On hardware with a sparse matrix path, the 2:4 pattern of section 10.2.1.3 turns into execution savings, since its 2-bit position indices cost far less than the pointer arrays of general CSR or COO formats (NVIDIA Corporation 2020a). The sparse datapath that consumes those indices and skips the zeroed multiplies is developed in section 11.4.5.3. Learned masks need not stop at 2:4; STEP, for example, learns more general N:M masks for inference (Lu et al. 2023).

Unstructured pruning zeroes arbitrary individual weights. Those irregular locations can disrupt memory coalescing and lane utilization, while Compressed Sparse Row (CSR) or Coordinate (COO) metadata adds storage and non-unit-stride accesses. An unstructured sparse model can therefore run slower than its dense counterpart when metadata and underutilization outweigh skipped arithmetic. TPUs are useful contrast cases for dense systolic-array acceleration (Jouppi et al. 2021), but sparse acceleration still depends on a specific hardware and software path rather than accepting arbitrary masks. Field-programmable gate arrays can implement application-specific sparse formats, yet their efficiency depends on the chosen dataflow and resource budget; programmability is not a universal sparse-speedup guarantee.

Across all platforms, sparse operations can reduce memory bandwidth requirements and energy consumption when the sparse representation and kernels actually skip data movement rather than only arithmetic. This benefit compounds with quantization, since a sparse INT8 model can require less memory traffic than either technique alone when the format overhead is small enough (Hoefler et al. 2021; Gale et al. 2020).

10.4.2.4 Challenges and limitations

The dependence of every sparse speedup on a matching format and kernel explains why nominal sparsity often fails to deliver speedups in practice. First, the performance break-even point varies with tensor dimensions, sparse format, kernel implementation, and hardware architecture. When coordinate metadata and irregular activation gathers saturate cache lines, an unstructured sparse kernel can run slower than an optimized dense baseline until sparsity exceeds 80 to 90 percent. Second, the accuracy-efficiency trade-off is nonlinear, because models often tolerate moderate pruning before task quality degrades abruptly past a critical density threshold. Finally, energy savings do not scale linearly with skipped operations; irregular memory access and index decoding can consume more power on edge devices than the arithmetic they eliminate. Fusion and sparse



Unstructured sparsity pays off only after crossing a hardware- and kernel-specific break-even point.

kernels thus decide whether each technique's paper savings survive contact with the hardware. Most deployments apply several techniques at once, and choosing and ordering them is the next decision.

10.5 Choosing and Combining Techniques

When the chapter's phone decodes GPT-2 XL, its FP16 weights and KV cache claim 3.3 GB of the 8 GB that the operating system and other applications also share, each generated token waits about 65 ms while every weight streams over the 51.2 GB/s bus, and the handset sustains only 3–5 W before thermal throttling. Choosing among compression techniques—and ordering them—starts from whichever of those physical limits binds first, alongside the allowable task accuracy loss and the available training budget.

The three primary optimization levers target different levels of the system stack. Distillation alters the algorithmic architecture by training a compact dense student to emulate a larger teacher, executing efficiently on standard hardware without specialized runtime support. Pruning modifies tensor topology and operation patterns, but yields wall-clock acceleration only when the resulting sparsity aligns with hardware execution units, such as 2:4 structured sparse matrix cores or channel-aligned sub-matrices. Quantization alters data representation by reducing numerical bit width, directly lowering memory bus traffic and cache footprint while engaging higher-throughput integer datapaths. Their first-order trade-offs, with representative ranges for common choices, sit side by side in table 10.11.

Table 10.11: Optimization Trade-Offs: Systems gain, quality impact, training cost, and hardware dependency for common compression choices. Region 1 marks little or no quality loss, region 2 a moderate trade, and region 3 steep quality loss. The ranges are representative engineering values synthesized from published work on inference runtimes, quantization, pruning, low-bit LLM quantization, and distillation (NVIDIA 2024c; Jacob et al. 2018; Han et al. 2015; Lin et al. 2024; Hinton et al. 2015); actual results vary with architecture, task, range-calibration data, and implementation quality. The rows identify first-order candidates, not guaranteed outcomes, and the hardware dependency decides whether pruning or quantization changes latency at all.

Technique	Systems gain	Typical impact	Training cost	Hardware dependency	Region
Kernel fusion	10–30% latency reduction	No accuracy loss	None	The compiler must recognize the pattern	1
FP32 → BF16	2× memory, ~2× throughput	$< 0.1\%$ accuracy drop	None for inference	A native BF16 path	1
FP16 → INT8	2× memory, 2–4× throughput	Quantization error, 0.5–1% accuracy drop	Low for PTQ; higher for QAT	INT8 kernels for a runtime speedup	2
50% pruning	~2× smaller model	Capacity loss, 0.5–1% accuracy drop	Usually includes fine-tuning	Structured shapes or supported sparse kernels	2
Knowledge distillation	2–10× smaller student	Capability ceiling, 1–3% accuracy drop	Teacher inference and a student run	Low; the student is an ordinary dense model	2
4-bit quantization	4× memory reduction	Significant error, 2–5% accuracy drop	Low for PTQ; higher for QAT	Low-bit kernels with fused dequantization	2–3
90% pruning	~10× smaller model	Severe capacity loss, 5–15% accuracy drop	Iterative fine-tuning	Sparse kernels past the break-even point	3

10.5.1 The accuracy-efficiency frontier

Pushing a single compression knob too far—such as pruning 90 percent of a network's weights or quantizing sensitive layers to 4 bits without retraining—crosses a cliff where a modest extra byte saving costs 5 to 15 percentage points of task accuracy (table 10.11). Every lever in the table moves the model along this **accuracy-efficiency trade-off**: additional model capacity can improve predictive accuracy while demanding larger weight payloads, expanded activation working sets, and higher memory bandwidth during inference. On hardware with strict memory or power limits, engineers first eliminate avoidable implementation costs, then choose among feasible operating points on the Pareto frontier (principle 5).

Systems Perspective 10.2: The compression-accuracy trade-off curve

The measured trade-off is not uniform. A structure-preserving implementation improvement can move a dominated model toward the frontier with little or no accuracy loss. Once on an efficient frontier, improving one objective trades against another; aggressive structural changes can enter regions with steep quality penalties.

The engineering decision is where to stop. The "knee" of the curve marks a region where further resource savings incur a sharper quality penalty. It is a useful visual diagnostic, but accuracy points cannot be compared directly with milliseconds, bytes, or joules. Stop at an operating point that meets the deployment's quality and resource thresholds, using explicit objective weights if a further trade must be ranked.

Techniques that preserve model structure, such as kernel fusion and modest precision reduction (FP32 to BF16), can yield latency or memory gains with little quality loss when the model and execution path support them. In contrast, techniques that alter network structure, such as pruning and distillation, yield larger reductions in memory and arithmetic volume but risk significant quality degradation without careful retraining. Optimization therefore begins by identifying the binding hardware constraint—memory capacity on mobile devices, frame latency in interactive pipelines, or energy per inference on battery-powered sensors—so each tier can target the resource that actually binds: structural methods cut parameter and operation counts, quantization shrinks memory traffic and raises arithmetic intensity, and compiler-level fusion eliminates intermediate memory round-trips. The regions in table 10.11 mark where each choice typically lands on that curve, and a structured decision framework maps each binding hardware constraint to the specific levers worth testing first.

Checkpoint 10.4: The efficiency frontier

Optimization can remove avoidable costs or trade among efficient operating points.

Trade-offs

- ☐ **Accuracy vs. cost**: compare a structure-preserving optimization and a structure-altering one by naming which resource each saves and which quality risk it introduces.
- ☐ **Dominated operating points**: explain how an implementation improvement can move a dominated operating point toward the frontier without reducing accuracy.
- ☐ **The knee**: explain how a sharp change in the measured trade-off informs a decision, and state the quality and resource thresholds needed to choose an operating point.

10.5.2 Decision framework

Applying a compression technique that targets the wrong hardware bottleneck wastes engineering cycles and task accuracy: pruning FLOPs from a layer stalled on off-chip weight streaming leaves memory-bus latency intact, whereas quantizing weights to 8 bits on a compute-bound processor that lacks an integer matrix unit leaves arithmetic throughput unchanged. Identifying the binding hardware constraint first narrows the search to the optimization tiers that can relieve it. Under the D·A·M taxonomy, compression methods redistribute resource demands across Algorithm (topology, pruning, distillation, and the numerical representation of weights) and Machine (execution units and memory hierarchies); range-calibration sets are the main Data-axis input. Each system constraint maps to the optimization tiers that can relieve it (table 10.12), where the marks identify mechanisms to investigate rather than guaranteed deployment gains.

Table 10.12: Optimization Tiers: System constraints drive optimization along three core tiers: model representation, numerical precision, and hardware implementation, each addressing different resource limitations and performance goals. ✓ marks a direct lever, while △ marks a workload- or hardware-dependent benefit; for example, low-precision arithmetic helps only when the accelerator and runtime provide a supported path.

System Constraint	Model Representation	Numerical Precision	Hardware Implementation
Computational Cost	✓	△	✓
Memory and Storage	✓	✓	△
Latency and Throughput	✓	△	✓
Energy Efficiency	✓	✓	✓
Scalability	✓	△	✓

One technique can move more than one term of the iron law. For example, quantizing weights from FP32 to INT8 directly relieves on-device storage capacity. A bandwidth-limited operator approaches proportional latency savings only when the affected traffic dominates and conversion overhead does not consume the gain. A compute-limited operator needs a faster supported execution path. Unstructured pruning likewise reduces executed arithmetic only when sparse kernels exploit the removed weights; indexing and irregular memory accesses can offset the saving. The changed representation therefore supplies a candidate improvement whose effect must be measured on the target.

When persistent storage or device memory capacity binds the deployment, as over-the-air package limits and microcontroller flash budgets do, quantization provides the most direct payload reduction. Relative to FP32 storage, INT8 post-training quantization (PTQ) cuts raw weight storage by $4\times$ without requiring retraining cycles. INT4 cuts raw payload by $8\times$, though aggressive bit-width reduction increases the risk of accuracy degradation. If uniform low-bit quantization causes unacceptable accuracy loss on sensitive layers, distilling knowledge into a smaller dense architecture followed by moderate quantization preserves fidelity while satisfying the storage budget.

When latency is compute-bound, the workload operates above the accelerator's ridge point, the arithmetic intensity at which its compute and memory-bandwidth limits meet, saturating execution units. Relieving this bottleneck requires reducing executed operations or increasing arithmetic throughput. Structured pruning removes channels, attention heads, or entire layers, yielding smaller dense matrices that execute on standard libraries without sparse runtime overhead; however, pruning to dimensions that misalign with hardware warp or tile sizes (such as channel counts not divisible by 8 or 16) degrades tensor core efficiency and curtails wall-clock gains. If the target platform provides dedicated integer execution units, INT8 quantization increases the instruction throughput rate, accelerating compute-bound kernels directly. Early-exit architectures provide an alternative algorithmic path by dynamically halting inference on simpler inputs, provided that routing and pipeline synchronization overhead do not disrupt batch processing.

↪ How the ridge point shifts across accelerators and precisions is developed in section 11.6.2.

Small-batch transformer generation often exposes a weight-bandwidth bottleneck, as GPT-2 XL decoding on the phone showed, because dense decoding streams large parameter matrices for matrix-vector operations with low arithmetic intensity. When fetched weights dominate latency, weight-only quantization (such as INT4 or INT8 weights paired with FP16 activations) can approach a speedup proportional to the reduction in those bytes. KV-cache traffic and dequantization overhead limit that gain without necessarily making execution compute-bound; larger batches can increase weight reuse and shift the limiting resource.

When energy or thermal dissipation binds, quantization provides substantial benefits because moving data across off-chip memory buses consumes orders of magnitude more energy per bit than on-chip arithmetic or local SRAM access. Reducing operand bit widths directly lowers bus capacitance charging energy, while integer ALUs consume less switching energy than floating-point datapaths. Structured pruning compounds these savings by eliminating operations and memory transactions entirely. Full-system power audits must still measure peripheral components (memory refresh, host bus communication, and static idle leakage) to ensure that kernel-level energy savings translate into extended operating life.

Selection also depends on the engineering and computational budget available. Calibrated PTQ needs only a small representative dataset, and QAT adds training cost only when PTQ misses

the accuracy threshold. Knowledge distillation costs more, requiring teacher forward passes and substantial training cycles to optimize a smaller student model. Hardware-aware NAS occupies the highest cost tier, justified only when high-volume deployment amortizes the extensive compute needed to explore the architectural search space.

Applied to GPT-2 XL decoding on the phone, this framework makes INT4 weight-only quantization the first candidate, not a completed solution. It cuts the weight bytes to a quarter of FP16, but each token also streams the unchanged KV cache, and the gain holds only when dequantization is fused into the matrix multiplication (section 10.3.5). Distillation to a smaller model is another candidate if the remaining gap justifies its training cost. Each candidate must pass task-quality, latency, peak-memory, and sustained-power checks against the phone's 3–5 W envelope.

Choosing an individual technique addresses a single binding bottleneck, but production deployment budgets frequently require compounding gains across multiple dimensions. When memory capacity, bandwidth, and compute throughput are all constrained simultaneously, optimizing a system requires composing structural alterations, precision reductions, and custom execution kernels into an integrated transformation pipeline.

10.5.3 Combining and sequencing techniques

Quantizing a network to fixed 8-bit integer ranges before pruning its channels shifts the activation distributions that those ranges were calibrated to fit, while removing floating-point precision before distillation hinders gradient recovery. Because each tier changes the weight and activation distributions or tensor shapes that the next tier sees, the order in which compression techniques are applied governs both final accuracy and hardware utilization. On the camera pipeline, the sequence follows directly from those dependencies between the tiers. Channel pruning comes first, with pruned channel counts kept on a multiple of the hardware's tile width so the smaller layers still fill the phone's matrix tiles. Distillation from the ResNet-50 teacher then serves as the recovery training for the pruned network, the prune-then-distill order described in section 10.2.3. INT8 quantization comes last among the model changes, because range calibration must see the weight and activation distributions of the final structure, and because INT8 is what moves the model from the phone's CPU onto its integer accelerator. Batch-normalization folding and the remaining fusions are applied when the graph is compiled for the device. Each step is profiled on the phone before the next is taken, because each one changes the bottleneck that the next one faces. The steps also move different quantities, and most of the frame time comes off at the step that changes the processor (figure 10.29).

A modeled BERT-Base pipeline attaches numbers to the same staging. Pruning, distillation, and quantization together reduce its footprint from 440 MB to 28 MB (rounded), a 16× raw-footprint reduction. Each stage addresses a distinct bottleneck. Structural pruning removes redundant matrix multiplications, distillation supervises the compact student to recover representational capacity, and quantization packs weights and activations into lower-bit integers to relieve memory bandwidth pressure.

Example 10.2: BERT-Base mobile deployment pipeline

Scenario: A modeled deployment targets executing a 110M-parameter BERT-Base language model on resource-constrained mobile hardware. The compact graph is stipulated for this example; the figures are not reported DistilBERT results.

The FP32 teacher begins at 440 MB with assigned task accuracy 76.2 percent. Structural pruning produces a stipulated 27.5M-parameter graph, a 75 percent parameter reduction, with raw FP32 weights of 110 MB and assigned accuracy 75.1 percent.

That pruned graph is the distillation student. Teacher supervision recovers its assigned accuracy to 75.9 percent without another parameter-count reduction. Final INT8 quantization leaves the same parameter count, reduces raw weight storage to 27.5 MB, and has assigned accuracy 75.6 percent. The headline ratio uses this unrounded weight payload; scales, zero-points, packing, activations, and runtime workspace remain outside the calculation.

Diagnosis: In this modeled comparison, quantization before pruning has an assigned accuracy loss of 2.1 percent. One mechanism to investigate is whether structural changes invalidate

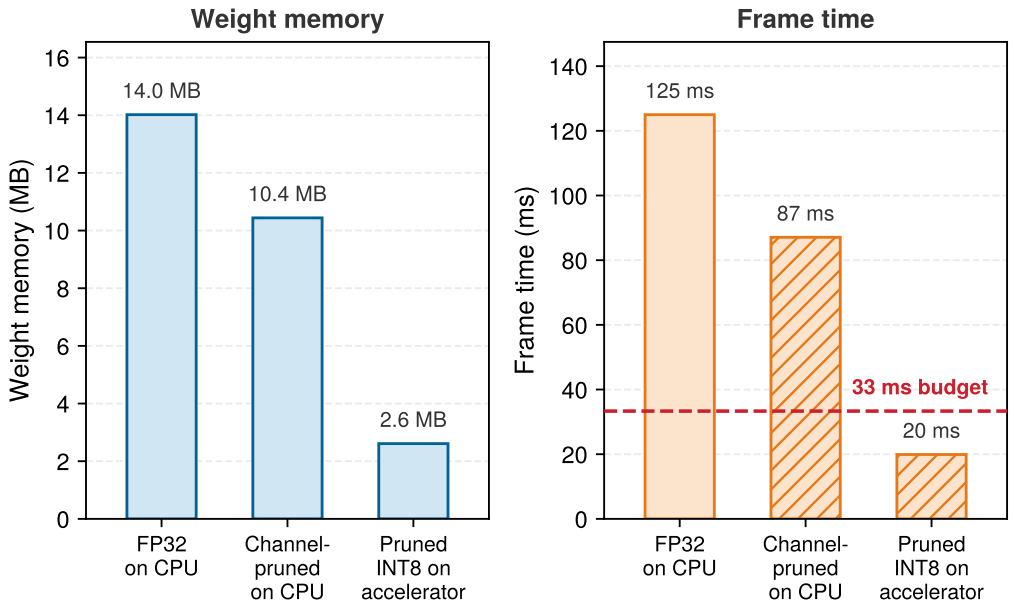


Figure 10.29: Compression Waterfall for the Camera Pipeline: MobileNetV2 weight memory (left) and frame time (right) after each model change, against the 33 ms budget of a 30 FPS camera. Weight memory follows from the parameter counts. The FP32 frame time is the running case's illustrative value, and the hatched bars scale the running case's frame times by the share of arithmetic the pruned network keeps. Channel pruning to width 0.75 keeps 70 percent of the arithmetic but leaves the frame near 87 ms on the CPU. INT8 cuts the weights by 4× and moves the model onto the integer accelerator, which brings the frame to about 20 ms. Folding and fusion, applied at compile time, are not modeled.

previously calibrated ranges. Pruning and distillation before quantization allow calibration to use the resulting distributions. These modeled values illustrate the mechanism and do not establish a universal ordering.

Systems lesson: In this modeled pipeline, pruning, then distillation, then post-training INT8 quantization produce a 16× footprint reduction (440 MB to 28 MB) while limiting task accuracy loss to 0.6 percent. Because tensor distributions and hardware kernel efficiencies vary across architectures, each intermediate artifact must be profiled on the target runtime.

Sequencing matters because these transformations interact through shared parameters and activation distributions. Structural pruning removes channels or attention heads, altering the dynamic range and variance of surviving weight tensors. Quantizing before pruning can leave ranges poorly matched to the resulting distributions, so recalibration or joint adaptation may be needed. In the staged candidate, floating-point recovery lets the pruned network adapt, and distillation supplies a teacher signal before final quantization. QAT itself updates floating-point master weights through approximate gradients; it does not require direct integer weight updates. Alternative schedules should be compared at matched recovery and calibration budgets.

Combining techniques also changes the size-quality trade-off itself (Han, Mao, et al. 2016). In one reported setting (figure 10.30), while individual SVD and pruning curves suffer steep accuracy cliffs below about 15 percent and 6 percent model size respectively, combining pruning with quantization extends the zero-loss plateau down to roughly 3 percent of original parameter footprint. This comparison does not isolate transformation order. The magnitude of these gains remains model- and runtime-specific, requiring engineers to validate each transformation stage independently.

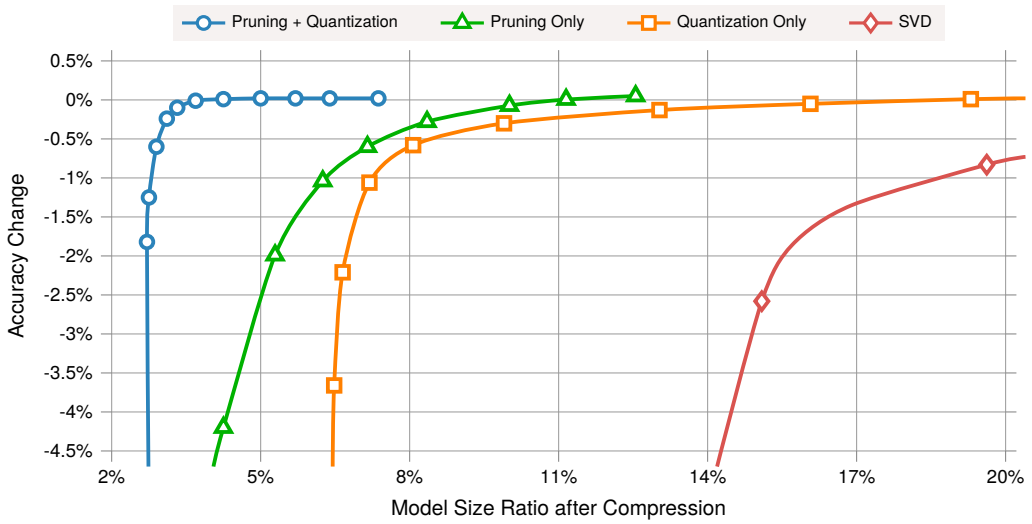


Figure 10.30: Compression Methods in One Reported Setting: The cited comparison plots accuracy change against model-size ratio for pruning, quantization, their combination, and SVD. In this setting, pruning plus quantization reaches the smallest ratio near baseline accuracy; the ordering is model- and implementation-specific. Source: (Han, Mao, et al. 2016).

> **Systems Perspective 10.3: The optimization composition problem**
>
> Optimization techniques interact through shared representations and physical resources rather than composing as independent functions. Pruning changes the surviving value distribution, tensor shapes, or sparse format; quantization adds scales and changes the available kernels; fusion changes which intermediates are materialized and can alter the profile that justified earlier choices. Composing pruning with quantization, as when 2:4 structured sparsity is paired with INT8 weights, leaves an executable that must carry both the sparsity metadata and the quantization scales, and a pruned shape that breaks the memory alignment a quantized kernel expects can erase the speedup of both (Hoefler et al. 2021). Effective optimization therefore treats the model-runtime-hardware combination as a coupled system, measures it after every transformation, and revisits the sequence when a new bottleneck appears. This makes compression a systems engineering problem as well as a machine learning problem.

A footprint reduction that compounds on paper therefore becomes a speedup only where the runtime supports every transformation in the sequence at once, which makes the toolchain that applies them part of the decision.

10.5.4 Compression toolchains

Quantizing a large model by hand would mean editing thousands of operator call sites to insert scale factors and track activation ranges. Manual graph surgery also cannot guarantee autograd consistency during retraining or the memory alignment that accelerator kernels require. Compression therefore runs through tooling, and the tool choice follows the layer of the software stack that owns the transformation. Framework APIs operate on the high-level computation graph during training and calibration, where the execution engine maintains visibility into weights, activations, and autograd state. At this stage, quantization is typically simulated, with fake-quantization nodes that clamp and round values within floating-point registers, modeling numerical precision loss while preserving differentiability.

Simulating quantization does not itself establish a physical speedup. A compiler or runtime must lower the graph into supported kernels; frameworks may integrate that stage or export to a separate deployment engine. The runtime compiler fuses scale factor arithmetic directly into matrix-multiplication compute tiles and packs sparse weights into hardware-specific metadata

layouts (such as 2:4 structured sparsity formats). This lowering eliminates round-trip memory transactions to HBM that would otherwise negate the benefits of reduced precision.

Reproducibility is the first requirement of a multi-stage pipeline. Compressing a model alters its numerical representations across both the algorithm and machine layers of the D·A·M taxonomy. Unrecorded calibration inputs or mismatched rounding modes can degrade accuracy silently without raising runtime errors, so every deployed artifact must remain traceable from its source training checkpoint, through the calibration data, pruning schedule, and fusions that produced it, to its compiled hardware execution graph.

↪ Versioned artifacts and the lineage graphs that keep deployed models traceable are developed in section 14.3.1.2.

10.5.4.1 Model optimization APIs and tools

At the model-development layer, compression algorithms require access to runtime state that downstream deployment compilers discard: gradient histories, optimizer state variables, and dynamic activation distributions. When post-training quantization or one-shot pruning degrades accuracy beyond acceptable operating thresholds, recovery requires retraining or iterative fine-tuning. Frameworks such as PyTorch and TensorFlow provide optimization APIs specifically to manage this graph-transformation boundary. These APIs inject quantization nodes and observers, reparameterize weight tensors with sparsity masks, and preserve calibration metadata alongside the trained parameters, which is what makes a compression recipe repeatable across many models.

Quantization-aware training demonstrates this graph-level intervention. Optimization APIs insert the fake-quantization operators of section 17 into the compute graph; these operators clamp and round in the forward pass and apply the straight-through estimator of section 10.3.3.2 in the backward pass. Producing a deployable low-precision artifact then takes a short sequence of graph transformations and metadata tracking (listing 10.5).

Listing 10.5: Quantization-Aware Training: Mechanism-level transformation that trains through simulated low-precision behavior and exports the calibration metadata needed for deployment.

```
choose target precision and calibration policy
insert quantize/dequantize boundaries around selected tensors
train with simulated low-precision values while keeping gradient flow intact
record scales, zero points, accumulator precision, and unsupported operations
export the calibrated graph and metadata as one reproducible artifact
```

Pruning tools implement the mask-then-export boundary of section 10.2.1.7. A mask-based API keeps the dense weight tensor $\mathbf{W}$ and a binary mask $\mathbf{M}$ of the same shape during recovery training, computing $\hat{\mathbf{W}} = \mathbf{M} \odot \mathbf{W}$ in the forward pass so the existing dense kernels and gradient updates keep working. An export pass then finalizes the transformation, either slicing away pruned channels or encoding the nonzeros in a sparse format the target's kernels accept. What must be recorded along the way, so that the compressed model can be reproduced, is short but not optional (listing 10.6).

Listing 10.6: Pruning Artifact Boundary: Applies unstructured or structured removal rules, then records the mask and fine-tuning context needed to reproduce the compressed model.

```
choose pruning granularity: individual weights, channels, blocks, or attention heads
score candidate parameters by magnitude, sensitivity, or validation loss impact
remove or mask the selected structure according to the hardware-compatible rule
fine-tune the compressed model against the original validation target
export weights, masks, sparsity pattern, and calibration evidence together
```

10.5.4.2 Hardware-specific optimization libraries

Exporting an optimized model from a training framework produces an intermediate graph representation, not an executable binary. Compilers and runtime engines such as TensorRT, Accelerated Linear Algebra (XLA), OpenVINO, and TVM own the final translation step: compiling a pruned, quantized, or fused graph into machine instructions tailored to the target accelerator. In D·A·M terms, algorithmic compression remains theoretical until the graph maps to kernels that physically exploit the target machine's memory hierarchy and execution units.

The compilation pass verifies whether nominal compression yields physical speedup. Pruning that leaves zeros in dense tensors still streams and multiplies those entries with a dense kernel. Compact sparse formats need matching kernels, such as a supported 2:4 path, while structured channel removal can produce smaller tensors for ordinary dense kernels. Quantization lowers bitwidth, but a weight-only path may unpack operands into floating-point registers before arithmetic. That unpacking adds instructions; its cost must be compared with the packed weight traffic saved.

The result can be the *runtime dequantization trap*. If an accelerator provides native INT8 matrix-multiplication kernels but lacks an INT8 implementation for an adjacent operation (such as LayerNorm, GeLU, or an attention softmax reduction), the runtime encounters a type boundary. An unfused execution path may allocate a temporary buffer, launch a dequantization kernel to recover floating-point values ($x = s(x_q - z)$, equation 10.4), execute the unsupported operator, and launch another kernel to re-quantize its output. The floating-point format and opportunities to fuse conversion depend on the runtime. These intermediate format conversions incur extra kernel launches, introduce synchronization barriers, and add temporary floating-point traffic that can make the quantized model slower than the FP16 baseline. A fused weight-only path instead unpacks values within the consuming kernel, the fusion of section 10.4.1, and retains a net memory-traffic benefit.

Low-precision kernels also require specialized tensor layouts, such as tiled or interleaved memory channels, to ensure coalesced loads into shared memory or register files. When an unsupported operator sequence blocks the fusions of section 10.4.1, the memory-bandwidth bottleneck remains intact. Whether integrated into a framework or supplied separately, the compiler and runtime determine which graph-level opportunities become an efficient machine execution plan. Whether that plan delivers the savings computed in the earlier sections is a question only measurement on the target hardware can answer.

10.6 Measuring Compression Gains

Shrinking a model's weights by $4\times$ with INT8 quantization rarely cuts end-to-end latency by $4\times$, because cache residency, memory bus contention, and kernel launch overhead absorb much of the theoretical gain. Every saving computed so far in this chapter is a paper bound, and only a measurement on the target device turns a bound into a result: profiling on the deployment hardware identifies which resource binds each layer, fixes an honest baseline, and separates model execution time from runtime and pipeline overhead.

10.6.1 Profiling and opportunity analysis

Compressing the layers with the most parameters or FLOPs can leave wall-clock latency barely changed when low-intensity operators—such as layer normalization or elementwise activations—contribute almost no FLOPs yet consume a large share of execution time by streaming whole activation tensors across memory. The top-down profiling method of section 8.5 tells compression where to act, and two of its measurements decide the technique. Static footprint (weight tensors and persistent buffers) decides whether parameters fit in fast memory or device DRAM without spilling across PCIe, while dynamic allocation (activation tensors and KV caches) sets peak residency and so bounds batch size and serving concurrency. Alongside memory residency, each operator's arithmetic intensity ($I = O/D_{\text{vol}}$) shows whether removing FLOPs or removing bytes will actually shorten it.

A profile of a Vision Transformer (ViT) on an edge accelerator illustrates this operational heterogeneity. Per token and layer, the multilayer perceptron (MLP) block costs about $8d^2$ multiply-accumulates against $4d^2$ for the query, key, value, and output projections. In a ViT-Base-sized model the MLP blocks therefore carry nearly two-thirds of FLOPs and the projections just under one-third, with the attention score and weighting products adding only a few percent at standard image resolution. Layer normalization inverts the picture, contributing almost no FLOPs yet can claim a visible share of wall-clock latency, because it streams whole activation tensors with little arithmetic per byte. The camera pipeline's MobileNetV2 splits the same way in convolutional form. Its 1×1 expansion and projection layers carry most of the multiply-accumulates (Sandler et al. 2018), while its depthwise layers, like layer normalization, do little arithmetic per byte moved and take a larger share of time than of work. Profiling isolates the governing physical bottleneck for each component, dictating targeted optimizations rather than uniform compression: structured pruning

of MLP hidden units relieves the compute-dominant blocks, kernel fusion eliminates intermediate global memory round-trips for memory-bound normalization layers, and INT8 quantization shrinks the static footprint that the MLP and projection weights dominate.



Non-model work dominates latency, so compression caps speedup at 1.25×.

Translating layer-level profiling metrics into end-to-end system speedup requires accounting for the complete serving pipeline. In production serving systems, model execution frequently represents only a fraction of total latency, with the remainder consumed by image decoding, tokenization, RPC deserialization, or network transfer. Optimizing the model in isolation encounters an Amdahl ceiling whenever non-model stages dominate the latency budget, a ceiling that section 12.3.4 measures end to end.

> **Systems Perspective 10.4: FLOPs reduction is not proportional speedup**
>
> Reducing a model's FLOPs by 50 percent does not guarantee 50 percent latency reduction. Memory-bound operations see little benefit from arithmetic reduction because data movement, not multiplication, sets their time. Amdahl's law (principle 8) applies at the system level. If model inference accounts for only 20 percent of end-to-end latency (with the remaining 80 percent spent on data loading, preprocessing, and postprocessing), then even perfect model optimization yields at most 1.25× overall speedup. Tokenization and network transfer can occupy the non-model fraction in language-model serving; image decoding, resizing, and normalization can do the same in vision pipelines. Profile the complete path on target hardware before estimating optimization benefits.

Complementing latency and throughput profiles, layer-wise sensitivity analysis quantifies how individual layers tolerate perturbation. In deep networks, different layers tolerate very different amounts of precision reduction or weight removal. Profiling accuracy degradation while systematically perturbing one layer at a time, with quantization noise or weight removal, identifies which components require high-precision representations and which can absorb aggressive 4-bit quantization or structured pruning without compromising task accuracy. Once opportunity and sensitivity profiles have guided a compression pass, the resulting candidate must be evaluated across both statistical quality and physical execution metrics.

10.6.2 Measuring optimization effectiveness

Judging a compressed model by a single scalar metric—such as top-1 accuracy or parameter count—can approve a candidate whose batch-one latency barely budges under kernel-launch overhead or whose confidence calibration drifts past safety limits on rare subgroups. Evaluating compression requires balancing statistical quality against physical execution costs across the Data, Algorithm, and Machine (D·A·M) axes. Quality baselines must capture aggregate accuracy, confidence calibration (verifying that predicted probabilities reflect empirical correctness), and performance across underrepresented input slices. Simultaneously, efficiency baselines must record peak memory residency, latency distributions (including p99 tail latency), memory bus saturation, and per-inference energy.

ResNet-50, the camera pipeline's teacher and the book's compute-bound lighthouse, shows how one compression decision ripples across physical and statistical dimensions. Quantizing it from FP32 to INT8 on the reference H100 changes latency, throughput, artifact size, energy, accuracy, and confidence calibration at once, and a deployment profile must report all six rather than an isolated accuracy number (table 10.13).

Table 10.13: ResNet-50 INT8 Deployment Profile on the Reference H100: Latency and FP32 throughput come from the modeled H100 engine profile, INT8 throughput is the MLPerf Inference measurement, and the accuracy and calibration values are illustrative. A real profile must measure all six on the target system.

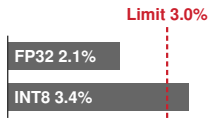
Metric	FP32 baseline	INT8 result	Deployment reading
Top-1 accuracy	76.1%	75.8% (0.3 pp drop)	Aggregate accuracy mostly holds, but subgroup checks still matter.
Batch-one latency	0.38 ms	0.30 ms (1.3×)	Fixed per-request costs dilute the gain for one image.

Metric	FP32 baseline	INT8 result	Deployment reading
Large-batch throughput	15,200 img/s	89,470 img/s (5.9×)	INT8 Tensor Cores pay off once arithmetic dominates.
Model size	102.4 MB	25.6 MB (4×)	The artifact becomes easier to cache, transmit, and deploy on edge.
Energy/inference	46.1 mJ	7.8 mJ (5.9×)	Assumes full board power at large batch; measure it directly.
Confidence-calibration error	2.1%	3.4%	Confidence calibration can drift even when accuracy looks acceptable.

Converting weights to INT8 shrinks the artifact by 4× (from 102.4 MB to 25.6 MB), but the speed gain depends on how the model is served. At batch one, a compiled INT8 engine is only 1.3× faster than its FP32 counterpart (0.30 ms against 0.38 ms), because launch overhead, per-kernel floors, and the weight read form most of a single image's time and shrink little with precision. At large batch the arithmetic dominates, and INT8 throughput of 89,470 img/s, as MLPerf Inference measured it, is 5.9× the modeled FP32 rate, a ratio between a measured rate and a modeled one from different software stacks. Energy per image follows throughput only if the board draws its full 700 W in both precisions; the 5.9× energy figure is that assumption, not a separate measurement. The runtime comparison in section 13.6 derives both latency figures from the same engine profile. While aggregate top-1 accuracy drops by only 0.3 percentage points (from 76.1 percent to 75.8 percent), confidence-calibration error (the gap between predicted probability and observed accuracy, distinct from the range calibration of quantization) increases from 2.1 percent to 3.4 percent. Post-quantization probability distributions drift, which can compromise safety-critical confidence thresholds even when top-1 classification accuracy appears preserved. Production deployment decisions require corresponding quality, latency, and energy measurements using a supported execution path on target hardware.

Suppose the application requires at least 75.5 percent top-1 accuracy and no more than 3.0 percent confidence-calibration error, measured with the same validation data and calibration metric as the profile. The INT8 candidate passes the accuracy limit but fails the calibration limit. Its modeled resource savings therefore do not justify acceptance. Retain the baseline for comparison, try confidence recalibration or a less aggressive quantization configuration, and repeat the quality checks before validating resource gains on the target runtime.

A deployment profile shows whether a compressed model meets its limits but not why it misses one. Explaining a failure such as the calibration miss above requires looking inside the model, at where clipping, rounding, and pruning masks have changed its internal values.



The INT8 candidate fails the calibration limit despite holding accuracy.

10.6.3 Diagnostic visualization

When an exported model fails a quality or latency gate—like the ResNet-50 candidate whose confidence calibration drifted to 3.4 percent—summary metrics cannot tell whether the culprit is a single activation channel saturating its clipping bounds, a pruning mask erasing an active feature detector, or an unsupported operator falling back to floating point. Diagnostic inspection looks inside the compressed graph to verify numerical integrity after framework APIs calibrate the model and hardware optimization libraries lower it to the execution substrate (section 10.5.4). While system-scale validation and artifact monitoring belong to downstream deployment infrastructure, the immediate diagnostic task is verifying that quantization clipping, pruning masks, or sparsity patterns have not perturbed internal feature distributions beyond acceptable task error tolerances.

Quantization error histograms show whether error is broadly distributed or concentrated in outliers. Activation visualizations help detect clipping, overflow, and saturation. In a grid of first-layer convolutional filters, the top rows show distinct edge, color, and frequency detectors, while the bottom row shows uniform-looking filters that appear redundant at a glance but require measured activation tracing before they can be pruned safely (figure 10.31). Framework observers and debuggers can expose quantization ranges and tensor error, while compiler or runtime inspectors can show the graph and kernels that the exported model actually executes.

Sparsity diagnostics identify candidate layers for sparse execution. The layer-wise schematic in figure 10.32 spans convolutional, fully connected, and transformer layer types and is not a

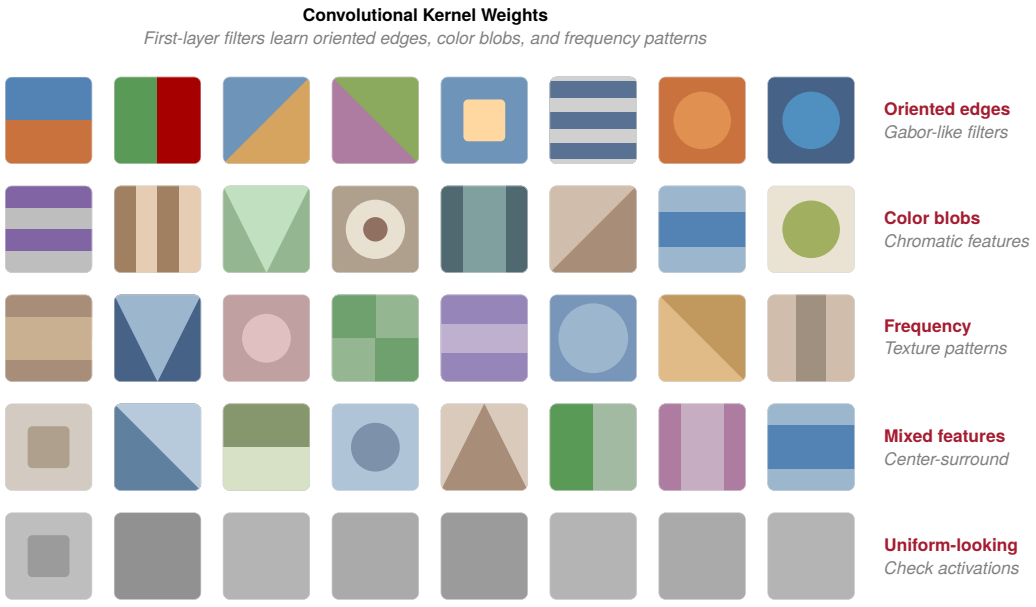


Figure 10.31: First-Layer Convolutional Filter Patterns: An 8×5 grid groups stylized kernels into edge, color, frequency, mixed, and uniform-looking patterns. Appearance alone does not establish inactivity; pruning requires measured activation or importance evidence in addition to low weight variance.

measurement from one model or pruning run. Its rows illustrate how a pruning policy might distribute zeros unevenly across architectural blocks. A real diagnostic would pair each layer's density with its storage format and target-kernel profile to determine whether those zeros reduce memory traffic or execution time. Graph viewers and sparsity inspection tools help locate the affected tensors; profiling establishes the deployment benefit.

The diagnostic value is the nonuniformity, not the exact shade of any one cell. Greater sparsity in later layers shows where the policy removed weights; it does not establish that those representations were redundant. Compare task quality after removing weights from a candidate layer and after recovery training, including performance on important input slices. Early-layer sparsity warrants the same checks because damaging feature extraction can propagate through subsequent layers. Only those comparisons can establish whether the observed removal pattern preserves required behavior. Each density also has a machine reading, because a layer whose zeros never cross the storage and execution break-even of section 10.4.2.2 saves neither bytes nor time, however dark its tile.

Checkpoint 10.5: From compression claim to deployment evidence

A compressed artifact must pass the same application requirements as its baseline.

- ☐ **Footprint and feasibility**: distinguish raw weight size from peak runtime memory, including activations, caches, metadata, and workspace.
- ☐ **Modeled and measured gains**: identify which claims in the ResNet-50 profile require target-hardware measurements, and explain why fewer FLOPs or bytes alone cannot establish latency.
- ☐ **Quality and operating limits**: specify the accuracy, confidence calibration, tail-latency, sustained-power, and energy checks needed before accepting a candidate.

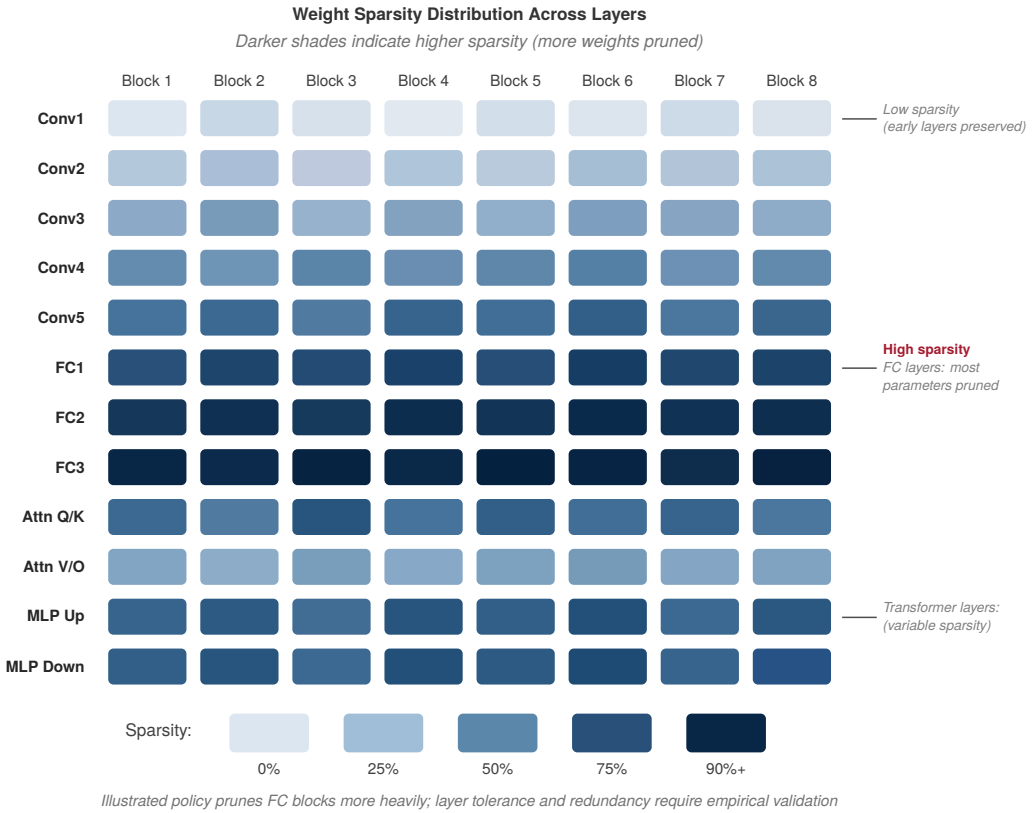


Figure 10.32: Illustrative Layer-Wise Sparsity Distribution: This schematic combines several layer types to illustrate non-uniform parameter removal. Darker tiles indicate more zero-valued weights; their shade establishes neither task redundancy nor a runtime benefit.

Implementation tools make compression repeatable, but they do not make the trade-offs disappear. Pruning alters model structure or parameter count, quantization changes numerical representation, and distillation trains the smaller artifact against a teacher's output distribution, and composing them unmeasured can produce latency regressions or silent accuracy collapse on target hardware. The methodology that makes a measured gain trustworthy, from benchmark design to the statistics of repeated runs, is developed in chapter 12. The fallacies that follow collect the places where intuitive compression assumptions fail.

10.7 Fallacies and Pitfalls

A team deploying GPT-2 XL on a phone might find that an uncalibrated 4-bit quantization cuts weight memory by 4× while generation quality collapses. Conversely, a vision team might achieve 70 percent weight sparsity through unstructured pruning only to find inference latency increases because commodity GPU kernels cannot execute irregular sparse indexing. Model compression presents numerous counterintuitive traps where theoretical parameter or FLOP reductions fail to produce wall-clock speedups. The following fallacies and pitfalls examine where standard engineering assumptions break down across the model-runtime interface.

Fallacy: *Optimization techniques can be applied independently without considering their interactions.*

Engineers assume optimization strategies compose additively: 50 percent pruning plus 4× quantization yields combined benefits. In reality, techniques interact nonlinearly and compound losses. In a representative BERT-style compression scenario, pruning to 70 percent sparsity may preserve most task performance, but applying INT8 quantization afterward can lose more accuracy than QAT on the pruned model. Knowledge distillation from heavily pruned teachers can also transfer

degenerate attention patterns that reduce student accuracy compared with distilling from dense teachers. As section 10.5.3 demonstrates, successful optimization requires coordinated application where techniques are sequenced together.

Pitfall: *Assuming theoretical metrics or compression ratios translate directly into proportional deployment speedup.*

Teams frequently reduce FLOPs or parameter counts substantially without realizing proportional latency gains on physical hardware. In one representative trace, a pruned model with 40 percent fewer parameters achieves only a 12 percent latency reduction because irregular sparsity prevents efficient execution. Similarly, reducing a transformer from 440 MB to 110 MB through INT8 quantization yields a 4× raw weight-payload reduction, yet format conversion and scalar fallback on hardware without a native low-precision datapath erode the latency gain. In the chapter's illustrative composed pipeline combining 50 percent structured pruning and INT8 quantization, the realized speedup reaches only 1.5× against an 8× paper target. Measure wall-clock latency on the target execution units rather than extrapolating from nominal compression ratios.

Fallacy: *Aggressive quantization maintains model performance with minimal accuracy loss.*

Engineers assume quantization error scales uniformly with bit width. In practice, precision reduction can show threshold effects, and the threshold depends on the model, layer, quantizer, calibration, and training method. The ResNet and BERT values in this chapter are illustrative rather than universal degradation bands. Even operations such as LayerNorm and Softmax can be implemented with integer approximations, so they do not universally require FP16; whether a low-precision implementation preserves task quality must be measured. As section 10.3.3 demonstrates, mixed-precision approaches can retain higher precision where uniform quantization fails.

Pitfall: *Defaulting to FP32 everywhere to avoid quantization risk.*

FP32 uses twice the raw storage and bandwidth of BF16, but lower-precision execution does not automatically preserve convergence or accuracy. Mixed-precision training often retains FP32 accumulators or master weights and may require loss scaling, while inference precision must be validated per model and operator. The right question is what lowest precision preserves quality on the evaluation distribution and maps efficiently to the target hardware.

Fallacy: *Post-training optimization is always enough.*

Teams often begin with post-training quantization (PTQ) because it avoids retraining, and move to quantization-aware training (QAT) only when the PTQ artifact misses its thresholds. Post-training pruning and post-hoc distillation can likewise underperform training-aware schedules at aggressive sparsity or bit width. As detailed in section 10.3.3, the production threshold, not a presumption that either path is universally superior, governs the decision.

Fallacy: *Sparse matrix representations always save memory and accelerate inference.*

Sparse formats introduce coordinate metadata that erodes storage savings at moderate densities, as the 16-bit case of section 10.4.2.2 showed; 32-bit indices make the penalty larger. To represent n_{nz} nonzero values across n_{rows} rows, a coordinate list (COO) stores explicit coordinate tuples `(row_idx, col_idx, value)`. For 32-bit floats and 32-bit integer indices, each nonzero element consumes 12 bytes (3× a dense 4-byte FP32 entry), requiring greater than 66.7 percent sparsity just to break even on memory footprint. Compressed sparse row (CSR) eliminates redundant row coordinates by storing three arrays: `values` ($n_{\text{nz}} \times 4$ B), `col_indices` ($n_{\text{nz}} \times 4$ B), and `row_pointers` ($(n_{\text{rows}} + 1) \times 4$ B). Each nonzero entry costs 8 bytes (2× a dense FP32 entry), requiring over 50 percent sparsity before the sparse structure consumes less memory than the original dense tensor; below 50 percent sparsity, CSR metadata expands the model's memory footprint.

Execution faces similar break-even constraints. Converting tensors to sparse storage as soon as pruning creates zeros degrades latency if metadata decoding, irregular gather operations, and poor cache locality outweigh the saved arithmetic. No universal sparsity threshold exists, because tensor dimensions, sparse format, runtime kernel, batch size, and hardware architecture jointly govern whether sparse execution outperforms dense execution. Specialized formats such as 2:4 structured sparsity eliminate index arrays in favor of fixed 2-bit multiplexer metadata, providing an accelerator

datapath that nearly halves weight traffic with minimal indexing overhead (section 10.2.1.3). Production workflows must therefore measure realized density and commit to a sparse representation only after verifying that the operational break-even threshold has been crossed on target hardware.

10.8 Summary

Model compression fits a trained model inside a deployment budget by working three tiers of one stack, the descent the chapter map laid out at the start (table 10.2). Structural optimization changes what the model computes, precision changes how each value is stored, and hardware implementation decides how efficiently the remaining operations execute. The running cases showed each tier where it matters most. The camera pipeline's MobileNetV2 spends its frame budget on arithmetic, so channel pruning, distillation from a ResNet-50 teacher, and an INT8 path onto the phone's integer accelerator are its levers. GPT-2 XL decoding spends its token on bytes, so INT4 weights raise its token rate by 3.1× on the same phone, and only when dequantization is fused into the matrix multiplication. The tiers can compound, but their gains do not multiply automatically. Pruning, distillation, and quantization together produce the illustrative pipeline's 16× footprint reduction from 440 MB to 28 MB; metadata, unsupported operators, and hardware mapping determine the realized deployment gain. A parameter reduction approaches the same latency reduction only when the affected work is on the critical path and the runtime can exploit the new representation. Profile on target hardware, not paper metrics.

Data selection and model compression divide the efficiency work along the D·A·M axes. Data selection acts on the Data axis and lowers the cost of producing a model by spending compute only on examples that carry signal. Compression acts where Algorithm meets Machine and lowers the cost of running the model by reshaping its structure and numerics for the target. Whether those deployment savings become real speedups still depends on the hardware and runtime that execute the compressed model. The need for them does not fade, because the bitter lesson keeps growing the models that win benchmarks while the envelopes of phones and microcontrollers stay fixed.

The counting rules developed across the chapter fit on one page (table 10.14). Together they let an engineer bound what a compression plan can save before a model is profiled on its target.

Table 10.14: Key Equations: The chapter's counting rules in the notation of the sections that derive them. Each gives a paper bound; the hardware tier explains why a measured gain can fall short of it.

Quantity	Formula	Where derived
Nominal sparsity	$\rho_{\text{sparse}} = \frac{\|\mathbf{1}_{\{(\mathbf{W}_{\text{sparse}})_{ij}=0\}}\|_0}{m \times n}$	section 10.4.2
Low-rank storage	$mk + kn$ values in place of mn	section 10.2.2.1
Depthwise separable cost	$1/d_{\text{out}} + 1/K^2$ of a standard convolution's arithmetic	section 10.2.4.3
Convolutional model scaling	$O \propto N_L \cdot w_{\text{mult}}^2 \cdot r^2$	section 10.2.4.2
Affine scale and zero-point	$s = \frac{\beta - \alpha}{2^b - 1}$, $z = \text{round}\left(\frac{-\alpha}{s}\right)$	equation 10.1, equation 10.2
Dequantization	$x = s(x_q - z)$	equation 10.4
Requantization scale	$s_{\text{eff}} = \frac{s_W \cdot s_X}{s_Y}$	section 10.3.3.1
Fused chain traffic	$2\,n_{\text{ops}} M_{\text{tensor}}$ bytes unfused; $2M_{\text{tensor}}$ fused	section 10.4.1
Bandwidth-bound decode time	D_{vol}/BW per token, where D_{vol} counts the weight bytes plus the KV-cache bytes	section 10.3.5

Key Takeaways: From benchmark winner to production model

- **Compression spends surplus capacity**: Production models trade unused parameters, precision, and capacity for latency, memory, power, or cost limits the deployment cannot violate. The target is an artifact that meets the deployment objective and all required quality and resource constraints.

- **Savings multiply only when aligned**: Structural pruning, distillation, quantization, and architecture changes can compound raw footprint reductions, as the modeled BERT mobile pipeline's 16× ratio illustrates. The gain becomes real only when task quality holds and the resulting operators match the target runtime and accelerator.
- **Precision is a deployment decision**: Where the target runtime supports it, INT8 post-training quantization is a useful first experiment because it reduces raw FP32 weight payload by 4× without retraining. QAT, distillation, or mixed precision become candidates when range calibration or layer sensitivity exposes unacceptable error.
- **Hardware decides what a saving is worth**: Unstructured sparsity and theoretical FLOP cuts do not imply latency gains unless kernels and memory layouts can exploit them. The chapter's hypothetical 8× paper target vs. 1.5× realized outcome illustrates why target-hardware profiling is mandatory.
- **End-to-end latency caps model wins**: Compression reduces end-to-end latency only when it accelerates the critical path; capacity or energy savings can remain valuable elsewhere. When inference is 20 percent of total request latency, Amdahl's law (principle 8) caps even perfect model acceleration at 1.25×, so preprocessing, dispatch, and data movement may be the true optimization target.

The techniques in this chapter differ in almost every detail, yet one rule runs beneath all of them. Each obtains a smaller, faster, or cooler model by giving up something else. Pruning gives up capacity, quantization gives up numerical precision, and distillation consumes training compute. Where the model holds no surplus to give up, task quality degrades instead. Apart from removing genuinely avoidable work, such as an unfused intermediate round-trip, compression does not make the work disappear; it moves cost or risk from one part of the system to another, and the hardware decides how much each move saves. A technique that helps on a phone can therefore be worthless on a data-center GPU. This is the conservation-of-complexity heuristic at the level of a single model. A production artifact is a model whose structure, representation, and execution path have been engineered together until they fit the silicon and application constraints that govern deployment.

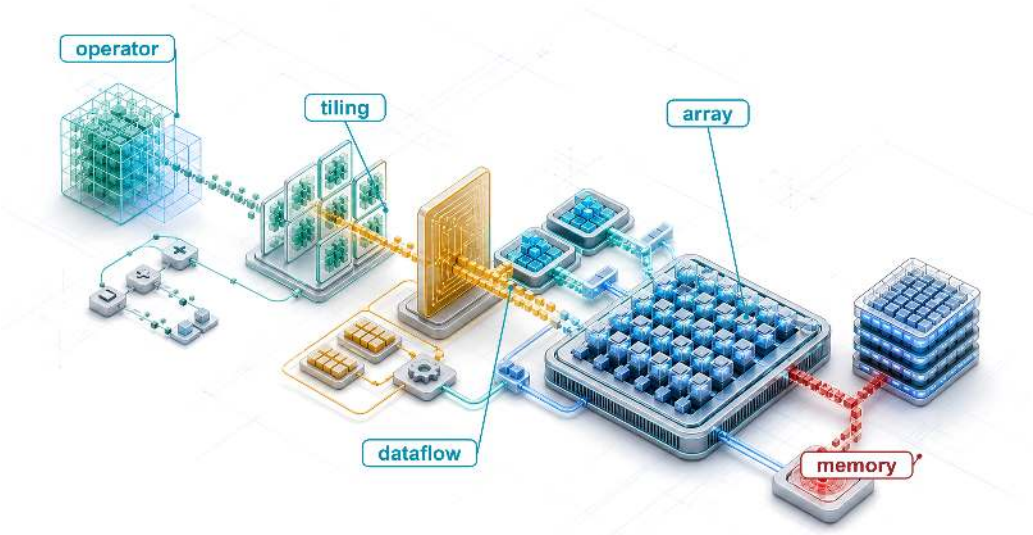


What's Next: From math to physics

These optimization techniques compress the model's logic, reducing the work the algorithm asks the machine to perform. Whether less work becomes less time depends on the silicon that executes it, and this chapter repeatedly ran into the memory wall, the regime where fetching a weight costs more than computing with it. Chapter 11 starts from that wall and from the ResNet-50 INT8 profile that closed this chapter, and explains how GPUs, TPUs, and NPUs organize tensor units, systolic arrays, and tiered SRAM/HBM hierarchies to keep arithmetic fed, which is what decides whether a quantized or sparse representation actually runs faster.

11

Hardware Acceleration

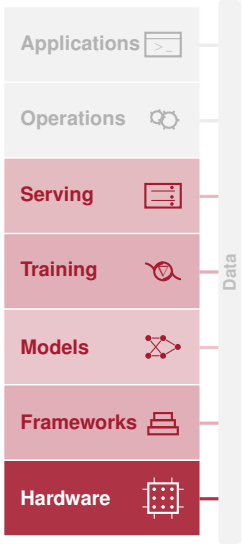


11.1 Anatomy of an Accelerator
11.2 Hardware Specialization
11.3 Compute Primitives
11.4 Compute Units
11.5 Memory Systems
11.6 Performance Bounds
11.7 Hardware Mapping
11.8 Compilers and Runtimes
11.9 Beyond One Accelerator
11.10 Heterogeneous SoC Design
11.11 Choosing an Accelerator
11.12 Fallacies and Pitfalls
11.13 Summary

Purpose

Why does moving data cost more than computing it?

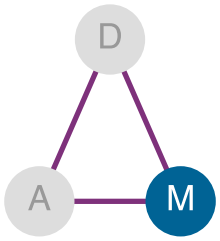
Arithmetic is inexpensive relative to data movement. In the time required to fetch a single scalar operand from off-chip DRAM, modern arithmetic units can execute dozens to hundreds of floating-point operations, and that DRAM access consumes on the order of a hundred times more energy than the computation itself. This fundamental imbalance, the memory wall, means specialized accelerators are not merely faster arithmetic engines. Rather, their microarchitectures are designed to stage, amortize, and hide data movement through wide spatial interconnects, deep on-chip memory hierarchies, and specialized execution pipelines. When general-purpose processor scaling faltered, hardware specialization enabled machine learning models to continue scaling by turning transistor budgets into high-throughput tensor datapaths. Delivered performance, however, depends on whether a model's tensor access patterns match the data paths of the underlying silicon. Hardware selection cannot be reduced to comparing peak FLOP/s: two accelerators with identical theoretical compute ratings often deliver radically different throughput on real workloads because one saturates its functional units while the other starves for data. A recommendation model dominated by sparse embedding lookups stresses memory bandwidth and interconnect latency, demanding a fundamentally different memory architecture than a dense transformer training run bounded by matrix multiply-accumulate throughput. If an operation is bandwidth bound, reducing theoretical operation count yields zero speedup because arithmetic is not the binding constraint. Matching workload characteristics to architectural dataflows determines whether an accelerator operates near its hardware ceiling or languishes at a fraction of its advertised peak. In D·A·M terms, the accelerator makes the machine constraint concrete, governing which algorithms and model architectures remain practical in production.



Learning Objectives

- Explain hardware acceleration as tensor-workload specialization and why Amdahl's law bounds its end-to-end speedup
- Compare Tensor Cores, systolic arrays, SIMD/SIMT units, and sparse execution for ML compute primitives
- Diagnose memory-wall bottlenecks using bandwidth, cache hierarchy, host-device transfer, and energy-movement costs
- Calculate arithmetic intensity and execution-path roofline ceilings, then use profiling to diagnose compute or memory limits
- Select dataflow, tiling, and mapping strategies that maximize reuse under memory-capacity constraints
- Analyze compiler and runtime optimizations that fuse kernels, plan memory, and schedule accelerators
- Evaluate accelerator choices across throughput, latency, power, cost, and deployment-context constraints

11.1 Anatomy of an Accelerator



Hardware acceleration turns on the machine axis.

Chapter 10 closed on a deployment profile for ResNet-50 quantized to INT8 (table 10.13): a model four times smaller whose large-batch throughput rises severalfold on the reference H100 while its batch-one latency barely moves. Data selection reduced the iron law's Data factor (Samples/Task) and compression reduced the Algorithm factor (FLOPs/Sample) and byte traffic (D_{vol}), but whether either saving reaches the wall clock is decided by the Machine factor (Seconds/FLOP). This chapter supplies the machine side of that profile. It explains why the same INT8 weights run faster on one execution path than on another, and why the hardware, not the model, determines how much of every compression saving survives execution. The explanation starts with the memory wall: arithmetic is cheap relative to data movement, so specialized hardware matters less because it raises compute throughput than because it organizes memory, dataflow, and parallelism so that its arithmetic units stay fed.

One workload runs through the chapter. ResNet-50 quantized to INT8 carries the book's compute-bound regime (table 1.4): its convolutions reuse each weight across many image positions, so on a data-center accelerator most of its time can go to arithmetic rather than to moving bytes. It reappears at every level of the machine, from the matrix units that execute its convolutions to the compiler that lowers its graph, and each time the question is what the INT8 path changes. GPT-2 XL decoding one token at a time supplies the single contrast. It carries the bandwidth-bound regime: every generated token streams all of the model's weights from memory for about two operations per parameter, so the machine that keeps ResNet-50 busy spends most of a decode step waiting for bytes. MobileNetV2, whose depthwise layers trade arithmetic for a small parameter footprint, and keyword spotting (KWS), the power-bound lighthouse, return where the chapter reaches phone-class and embedded chips. DLRM appears where its embedding lookups, the capacity-bound regime, break the regular access patterns that accelerators assume. The reference machine is the NVIDIA H100, the data-center accelerator this book uses for training and serving examples; its numbers are one generation's instance of limits that every generation obeys.

Definition 11.1: Hardware acceleration

Hardware acceleration is the practice of mapping work from general-purpose execution onto specialized processors or functional units optimized for a narrower operation class. For ML workloads, that specialization commonly trades some programmability for the compute density (R_{peak}) and performance-per-watt gains that regular tensor operations can exploit.

1. **Significance**: An H100 GPU delivers 989 TFLOP/s for dense FP16/BF16 matrix multiplication; against this chapter's illustrative 1–2 TFLOP/s reference-CPU baseline, this is a

490–990× gap. Its 80 billion transistors devote far more silicon to parallel arithmetic than to branch prediction, out-of-order scheduling, and large caches (Choquette 2023).

2. **Distinction**: A general-purpose CPU is designed for flexible execution, low-latency control, and varied workloads. An accelerator instead concentrates resources on a narrower operation class, so it achieves its gains only when the workload exposes the supported operations and enough parallel work to keep its arithmetic units busy.
3. **Common pitfall**: A frequent misconception is that an accelerator's advertised peak throughput is the throughput a workload receives. Delivered performance is bounded above by the lower of the compute ceiling and what memory bandwidth can feed (the roofline constraint, developed later in this chapter), so a low-arithmetic-intensity kernel can sit at a small fraction of peak FLOP/s no matter how fast the silicon is rated, because it starves for data rather than for arithmetic.

The anatomy of an accelerator is the arrangement of parts that every machine learning accelerator shares: arithmetic units built for tensor operations, on-chip memory that holds operands next to those units, a wide path to off-chip memory, and a host interface through which work arrives. Every inference and every training step on such a machine is a stream of operand movements that ends in multiply-accumulate operations, and the anatomy decides how far each operand travels before it is used. A matrix unit that can retire thousands of multiply-accumulates per cycle sits idle whenever its next operands are still in flight, and an off-chip access can take many processor cycles and cost orders of magnitude more energy than the low-precision operation it feeds. The arithmetic units are the easy part of the design; the rest of the anatomy exists to keep them fed.

The definition becomes concrete in the generic chip of figure 11.1, which serves as this chapter's map. External high-bandwidth memory (HBM) feeds shared on-chip storage (an L2 cache), which distributes operands into rows of processing elements, each equipped with a private scratchpad or cache, a matrix unit, a vector unit, and a special function unit. A host interface connects the chip to the CPU that issues its work.

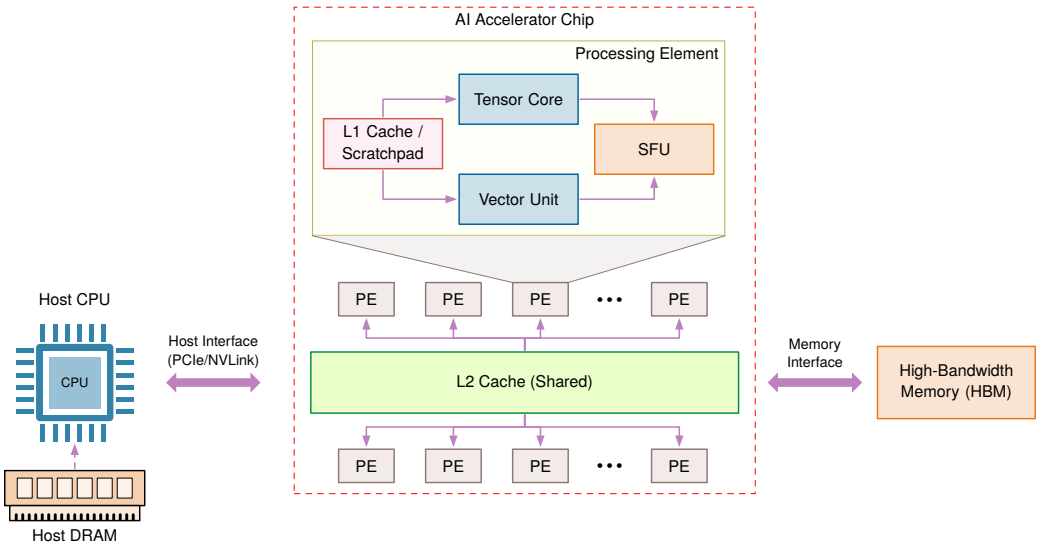


Figure 11.1: Anatomy of a Modern AI Accelerator: This generic schematic combines common components rather than depicting one product. A host interface connects the accelerator to a CPU, a high-bandwidth device-memory interface feeds shared on-chip storage, and processing elements contain matrix, vector, and special-function units with local memory. Real accelerators differ in which units they provide and whether storage is hardware-managed cache or software-managed scratchpad.

Within each processing element, specialized units partition execution across distinct mathematical primitives: dedicated matrix units (Tensor Cores) execute dense matrix multiplication, vector units

evaluate element-wise arithmetic, and special function units (SFUs) approximate transcendental operations such as activations and exponentials. Their number, grouping, and names vary by vendor; some designs arrange them into an explicit 2D systolic grid, and others distribute matrix engines among wide parallel cores.

The memory hierarchy amortizes the cost of data transport across tiers. High-bandwidth memory provides off-chip capacity and throughput, while shared on-chip storage and per-element caches or scratchpads cut off-chip traffic. Because moving a tensor tile across the package boundary costs orders of magnitude more energy than keeping it on chip, compilers and kernels must coordinate tile capacity, reuse, and prefetching to keep operands staged locally; more HBM bandwidth raises the off-chip ceiling, but without local reuse the execution units still stall. The system assumptions appendix (Appendix E) collects reference specifications for the accelerators this book uses, including the H100.

The host interface links the accelerator to the host CPU, which coordinates execution graphs, asynchronous I/O, and fallback operations while the accelerator executes dispatched kernels. Direct-memory-access engines can overlap host-device transfers with execution, but an operator unsupported by the accelerator backend may require a CPU round-trip. An operator outside the matrix-unit instruction set may instead use vector or other device kernels. These execution boundaries help set the effective speedup envelope. Kernel execution time matters little if transfers, launches, and synchronization dominate end-to-end latency. The three parts of the figure (the processing elements, the memory tiers, and the host link) are the parts this chapter takes apart in turn.

The anatomy embodies the chapter's central engineering trade-off. General-purpose processors allocate silicon area to complex control logic, speculative execution, branch prediction, and multi-level cache hierarchies to handle arbitrary control flow. An accelerator redirects that transistor budget toward dense arithmetic arrays, dedicated register files, and high-bandwidth memory paths tailored to a narrower workload class. When the software path exposes sufficient tensor parallelism, this specialization yields order-of-magnitude improvements in energy efficiency and throughput per watt. Silicon built for a narrow class of dense tensor operations pays off only because, as the bitter lesson predicts, the general methods that win by exploiting computation keep reducing to those same operations at ever larger scale. The dependence also runs the other way, because scale keeps winning only while accelerators keep lowering the time and energy of each additional operation, so the hardware this chapter studies is part of what sustains the bitter lesson itself.

↩ The bitter lesson, the observation that general methods exploiting computation win as computation grows, is established in section 1.4.

Hardware alone cannot achieve these gains. Algorithms must structure operations to expose data reuse and regular memory access, while silicon architectures must allocate physical units to the mathematical primitives and numeric precisions that algorithms actually execute. That mutual dependence motivates a complementary principle: hardware-software co-design.

Definition 11.2: Hardware-software co-design

Hardware-Software Co-design is an ML accelerator development method that crosses conventional abstraction boundaries, allowing algorithm constraints to inform silicon design and hardware capabilities to shape algorithm formulation.

1. **Significance**: On a supported tensor-core path, 8-bit integer (INT8) quantization can deliver multi-fold throughput gains because hardware executes lower-precision operations more densely and moves fewer bytes than an FP32 path; the exact gain depends on the architecture and workload (NVIDIA Corporation 2020a; Dally et al. 2021; Dally 2023).
2. **Distinction**: Unlike layered abstraction (where software calls a hardware API without knowing the silicon details), co-design exposes hardware constraints directly to algorithm and compiler authors: data alignment requirements, precision formats, and memory access patterns all become visible inputs to global cross-layer optimization.
3. **Common pitfall**: A frequent misconception is that co-design is a one-time hardware design choice. In practice, the supported features continue to evolve: NVIDIA Tensor Cores began with FP16 matrix multiplication, later generations added TF32 and broader integer support, and Ampere added acceleration for 2:4 structured sparsity (NVIDIA 2017; NVIDIA Corporation 2020a).

Co-design explains why model compression techniques deliver real speedups. Quantization shows why converting FP32 to INT8 accelerates execution on supported hardware: reduced precision cuts memory traffic, while specialized arithmetic units execute more low-precision operations per cycle than FP32 equivalents (NVIDIA Corporation 2020a). Structured pruning improves performance when it produces smaller dense shapes or a sparsity pattern recognized by silicon, whereas arbitrary unstructured sparsity incurs metadata handling and non-coalesced memory accesses that often erase theoretical gains. Evaluating accelerator performance requires tracing this path from workload to silicon: compute primitives, memory hierarchies, roofline diagnosis, mapping and dataflow, and compiler scheduling. The recurring question is why algorithmic improvements frequently fail to deliver proportional end-to-end speedups. Amdahl's law (principle 8) already answered it for model-side optimizations, since whatever an optimization leaves untouched caps the overall gain. The same bound governs hardware. The theorem below applies the principle to an accelerator, and the only new quantity is the gain, which becomes the accelerator's path gain G_{accel}.

↩ Quantization and its effect on weight payload, memory traffic, and compute cost are established in section 10.3.

Theorem 11.1: The fundamental limit of acceleration (Amdahl's law)

Hardware acceleration *does not speed up the entire system*; it speeds only the baseline-time fraction using the accelerated path. For that fraction f_{parallel} and path gain G_{accel}, **Amdahl's law** (Amdahl 1967) gives the bound in equation 11.1.

$$\text{Speedup} = \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{G_{\text{accel}}}} \tag{11.1}$$

- **Accelerated fraction** (f_{parallel}): Share of baseline time receiving the stated gain, often in matrix multiplication or another supported kernel.
- **Accelerator gain** (G_{accel}): The raw speed advantage of the GPU or Tensor Processing Unit (TPU) over the CPU for the accelerated portion of the workload.
- **Unaccelerated fraction** ($1 - f_{\text{parallel}}$): Work outside the chosen path, such as data loading, host overhead, unsupported operators, and kernel launch latency.

Pitfall: *Unaccelerated work caps total speedup.* If 10 percent of baseline time is unaffected ($f_{\text{parallel}} = 0.9$), even infinite path gain ($G_{\text{accel}} = \infty$) yields at most $10\times$ overall. Once the accelerated component is fast enough, unaffected work dominates.

Hardware acceleration targets specific terms in the iron law of ML systems, which decomposes end-to-end time into data movement (D_{vol}/BW), computation ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), and fixed latency ($L_{\text{lat}}$). While data selection reduced the number of samples processed (Samples/Task, shrinking both O and input traffic D_{vol}) and model compression reduced the operations and bytes per sample (FLOPs/Sample and weight traffic D_{vol}), hardware acceleration reduces the realized time per operation (Seconds/FLOP) by improving R_{peak}, η_{hw}, and BW. Which of these physical terms an upgrade actually improves determines its accelerated fraction under Amdahl's law.[1]

This dynamic explains why hardware upgrades frequently fail to produce proportional end-to-end speedups. Figure 11.2 maps system speedup across accelerator gain (G_{accel}) and accelerated fraction (f_{parallel}), charting where those returns diminish. Where the unaccelerated fraction $1 - f_{\text{parallel}}$ remains substantial, the white contour curves flatten horizontally into the unaccelerated-bound regime, bounding total speedup at the $1/(1 - f_{\text{parallel}})$ asymptote regardless of how far G_{accel} scales along the horizontal axis. Because f_{parallel} differs substantially across workload archetypes running on identical hardware, a workload's vertical position on this map determines whether an accelerator investment pays off or stalls on non-matrix operations.

[1] **Amdahl's law**: Maps onto the iron law only after the proposed change is specified. If a faster tensor unit reduces the computation term ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) but leaves data transfer and fixed latency unchanged, those unaffected terms bound total speedup. A different accelerator or memory system may improve bandwidth as well. The unaccelerated fraction is therefore defined by the intervention, not by a permanent classification of each term as serial.

↩ The decomposition of execution time into data, compute, and latency terms is established in section 1.7.

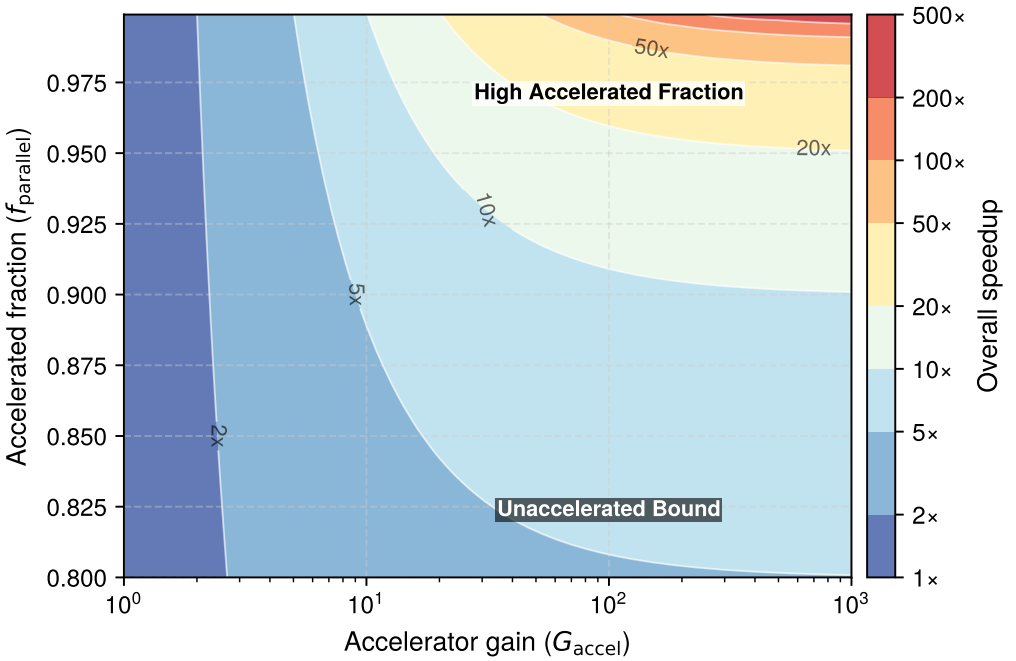


Figure 11.2: The Iron Law Heatmap: Total system speedup as a function of accelerator gain (G_{accel}) and accelerated fraction (f_{parallel}). High speedup appears only near the top-right corner, where both accelerator gain and accelerated fraction are high. At $f_{\text{parallel}} = 0.9$, for example, even an infinite gain on the accelerated path cannot exceed 10× end-to-end speedup. Contours span roughly 1×–200× speedup.

Checkpoint 11.1: The parallelism gate

These checks turn Amdahl's bound into a hardware-selection test.

Amdahl's reality

- ☐ **Serial bottlenecks**: use Amdahl's bound to explain why a 1,000× faster GPU may only speed up training by 5× when data loading is slow.
- ☐ **Speedup ceiling**: for $f_{\text{parallel}} = 0.95$ and $G_{\text{accel}} = 100$, estimate total speedup and compare it with the infinite-accelerator ceiling.
- ☐ **Workload variation**: what baseline profiling would establish the ResNet-50 and MobileNet accelerated fractions for a proposed hardware change? Explain how those fractions would determine which benefits more.

The serial bottleneck becomes concrete on real hardware, where the same accelerator that nears its parallel ceiling on one workload can stall on another. The comparison below uses the INT8 (8-bit integer) path, whose advantage over FP32 or FP16 has two sources. Each operand occupies a half or a quarter of the bytes, and an 8-bit multiplier occupies far less silicon area than a floating-point one, so the same die holds more multiply-accumulate (MAC) units. Evaluating an NVIDIA H100 on two workloads whose unaccelerated fractions differ illustrates how that fraction governs whether this peak INT8 advantage becomes realized speedup, and a second look at the decode case separates that limit from a different one, bandwidth-boundedness.

↪ Order-of-magnitude throughput, bandwidth, and capacity for each accelerator class are collected for reference in section D.3.3.

Lighthouse 11.1: An illustrative Amdahl budget on H100

Illustrative ResNet-50 inference on NVIDIA H100:

- Assume H100 provides $G_{\text{accel}} = 247\times$ on matrix multiplication relative to a server CPU without matrix extensions (1979 TOPS INT8 vs. ~8 TOPS) (Choquette 2023). The CPU's INT8 rate is a scenario assumption for this budget, not the FP32 peak of the reference CPU used elsewhere in the chapter.
- Assume $f_{\text{parallel}} = 0.95$: 95 percent of baseline time receives that gain, while 5 percent remains in data loading, preprocessing, postprocessing, and other work.

$$\text{Speedup} = \frac{1}{(1-0.95) + \frac{0.95}{247}} = \frac{1}{0.05 + 0.0038} \approx 18.6\times$$

The 247× path gain produces only 18.6× end-to-end speedup because the 5 percent unaffected fraction sets the ceiling.

Contrast with GPT-2 (autoregressive):

- For GPT-2 token generation, assume $f_{\text{parallel}} = 0.80$; the remaining 20 percent includes host overhead, sampling, and decoding work outside the accelerated matrix path.

$$\text{Speedup} = \frac{1}{(1-0.80) + \frac{0.80}{247}} = \frac{1}{0.20 + 0.0032} \approx 4.9\times$$

Less token-generation time receives the assumed gain, so even an infinite path gain yields at most $1/(1 - f_{\text{parallel}}) = 5\times$.

This calculation still overstates the decode path gain. Decode is bandwidth bound, so its accelerated path runs at the rate weights stream from memory rather than at the INT8 TOPS rate. The relevant ratio is the H100's memory bandwidth over the reference CPU socket's, 67× (a different path from the host-DRAM link used below), and INT8 helps decode on either machine by halving the bytes each weight occupies relative to FP16, not by multiplying faster. With that gain the bound falls from 4.9× to 4.7×, barely moving, because the unaccelerated fraction already dominates. Two distinct limits are at work. Bandwidth-boundedness shrinks G_{accel}, whereas the unaccelerated fraction caps what any G_{accel} can deliver. Serving optimizations attack both. Batching raises the work done per weight byte fetched, and speculative decoding, developed in chapter 13, lets a small draft model propose several tokens that the full model then verifies in one step, reducing the number of sequential full-model steps.

Whether an operation is limited by compute rate or by data movement dictates both hardware selection and software optimization. Under the arithmetic intensity law (principle 6), upgrading to an accelerator with $10\times$ higher peak compute yields negligible speedup if data delivery is the binding bottleneck. The roofline model, previewed in section 2.4.2, makes this diagnosis (Williams et al. 2009). It compares an operation's arithmetic intensity, the ratio of operations to bytes of memory traffic (FLOP/byte), with the hardware's ridge point. An operation below the ridge is bandwidth bound, the regime the roofline literature calls *memory bound*; an operation above it is compute bound. GPT-2 decode and ResNet-50 sit on opposite sides of that line, a difference the Amdahl budget above already registered in the decode path gain. Section 11.6 turns the classification into numbers once the compute units and memory systems that set the two ceilings have been examined.

The chapter builds the machine from the bottom up (table 11.1). Compute primitives and compute units supply the arithmetic side of the anatomy, ending with the INT8 and sparse paths that ResNet-50's profile depends on, and memory systems supply the other side, from registers to host memory. Performance bounds then combine the two into the roofline's ceilings and show which ResNet-50 layers clear the H100's ridge at FP16 and which fall below it at INT8; this is the chapter's midpoint, marked by a checkpoint. The second half asks how software reaches those ceilings, through hardware mapping and then the compilers and runtimes that make its choices automatically. Three shorter

↩ The arithmetic-intensity spectrum that separates the workload signatures of the major architectures is established in section 6.9.1.

sections widen the view to work beyond one accelerator, heterogeneous systems-on-chip, and the choice of an accelerator for a workload and a budget.

Table 11.1: Chapter Map: Each section supplies one part of the accelerator's anatomy and follows the chapter's INT8 ResNet-50 workload into it; the first five sections build the machine and its ceilings, and the remaining sections ask how software and system choices reach them.

Section	Part of the accelerator	Question it answers	ResNet-50 INT8 thread
Hardware Specialization	The whole chip	Why silicon is specialized for tensor work, and what a byte costs	Why its convolutions justify dedicated matrix silicon
Compute Primitives	Operations the hardware executes	Which operation classes earn dedicated units	Its convolutions lower to matrix multiplication
Compute Units	Vector, matrix, and special units	How each unit executes its primitive, including INT8 and sparse paths	The INT8 tensor path and its arithmetic floor
Memory Systems	Registers through host memory	Where operands live and what each tier costs	One activation tile's trip from HBM to registers
Performance Bounds	Compute and bandwidth ceilings	Which ceiling binds a given kernel	Which layers clear the FP16 and INT8 ridge points
Hardware Mapping	Placement, dataflow, and tiling	Which schedule captures the available reuse	Weight-stationary tiles and fused epilogues
Compilers and Runtimes	Software that lowers the graph	Who makes the mapping decisions, and when	Naive versus compiled latency
Beyond One Accelerator	Links between accelerators	What changes when one accelerator is not enough	Awareness only; ResNet-50 fits one accelerator
Heterogeneous SoC Design	Phone and embedded chips	How shared memory and power budgets change the mapping	MobileNetV2 and KWS take over
Choosing an Accelerator	The complete system	Which machine fits a workload, a latency target, and a budget	Energy and cost per request

Architectural design begins with the physical constraints that forced specialization: the breakdown of classical Dennard voltage scaling and the prohibitive energy cost of moving data across memory hierarchies. When general-purpose microarchitectures encountered these thermal and memory barriers, domain-specific hardware specialization emerged as the primary engineering path to sustain performance scaling.

11.2 Hardware Specialization

When Google deployed its first Tensor Processing Unit (TPUv1) into production data centers alongside NVIDIA K80 GPUs, the comparison showed a large efficiency gap (example 11.1). General-purpose processors dedicate substantial silicon area and power to instruction fetch, branch prediction, and cache coherence, machinery built for arbitrary code that yields poor arithmetic density on regular, dense linear algebra. When an algorithmic pattern becomes sufficiently dominant and structured, specialized hardware outperforms general-purpose platforms by replacing dynamic control machinery with dedicated arithmetic arrays and software-managed data movement.

Example 11.1: The TPUv1 vs. K80 efficiency gap

Scenario: In 2015, Google deployed its first Tensor Processing Unit (TPUv1) into production data centers, comparing its throughput and energy efficiency against general-purpose NVIDIA K80 GPUs (Jouppi et al. 2017).

Diagnosis: The K80 was a programmable GPU designed to support a broader mix of graphics and compute workloads. The TPUv1 domain-specific architecture (DSA) made a narrower, inference-focused design choice, pairing a large 8-bit systolic matrix unit with software-managed memory and simpler control.

Systems lesson: On the six Google inference workloads and contemporary CPU/GPU baselines evaluated in the TPU paper, tailoring silicon to the dominant matrix operations delivered

15–30× higher throughput and 30–80× better performance per watt (Jouppi et al. 2017). Those ratios describe the evaluated systems, not every workload.

In D·A·M terms, this efficiency gain stems from tailoring machine resources to algorithmic structure. The TPU[2] replaced hardware caches with software-managed buffers and routed data directly between neighboring arithmetic units, raising data reuse and cutting off-chip traffic. The cost is rigidity. Irregular memory access, dynamic control flow, or an unsupported numeric precision stalls a fixed-function data path and leaves expensive silicon idle. Every modern ML accelerator, from DianNao-class designs (Chen et al. 2014) to GPUs with Tensor Cores, strikes its own balance between specialization and flexibility, following a pattern that began decades before deep learning.

[2] **TPU (tensor processing unit)**: The first TPU made a deliberately narrow design choice, centering the design on a single 256×256 systolic array for 8-bit matrix multiplication and avoiding much of the control machinery required by a general-purpose core (Jouppi et al. 2017). That trade buys high compute density on dense matrix multiplication at the cost of flexibility. Irregular or branch-heavy code remains a poor fit, and compiler tiling, padding, and layer dimensions determine how fully the array is used.

11.2.1 Specialized computing

On a late-1970s microprocessor without floating-point hardware, a single 32-bit floating-point multiply required a software routine of dozens of integer shift-and-add instructions, turning scientific and signal-processing loops into an instruction-fetch bottleneck. Hardware specialization emerges whenever one computational pattern becomes a bottleneck that general-purpose processors cannot scale past. The Intel 8087 coprocessor (1980) executed floating-point operations directly in hardware (Palmer 1980), and the Intel 486DX (1989) later moved the floating-point unit onto the CPU die itself (Hennessy and Patterson 2017). That cycle—specializing silicon to relieve an acute bottleneck and integrating it once the capability proves universal—recurs in every later era, and so does its limit. Dedicated silicon removes the instruction overhead around a dominant operation, but the overall gain is bounded by how much of the run time that operation occupies.

Graphics supplied the second lesson, throughput over latency. A CPU spends most of its silicon on control logic, branch prediction, and caches that minimize the latency of a single instruction stream, which buys nothing when identical arithmetic must be applied to millions of independent pixels. GPUs reallocated that silicon to dense arrays of arithmetic units, and when unified shader architectures (2006) replaced separate vertex and pixel pipelines with arrays of identical programmable cores, the GPU became a general-purpose parallel processor (Owens et al. 2008). Digital signal processors such as Texas Instruments' TMS32010 (1983) contributed the memory organization. To meet hard real-time deadlines, they replaced caches, whose misses introduce variable latency, with software-managed scratchpads feeding dedicated MAC data paths (Lyons 2011). ML accelerators inherit both lessons.

The payoff arrived in 2012, when AlexNet (Krizhevsky et al. 2012) won the ImageNet competition after training on two consumer GPUs, the co-design result examined in section 1.3.3. Its dense 2D convolutions mapped directly onto thousands of GPU arithmetic lanes. In D·A·M terms, the machine constraint had dictated which algorithms were feasible, and aligning the algorithm's parallel structure with throughput-oriented hardware lifted it.

Across five eras from the 1980s to the 2020s (figure 11.3), specialization moved from discrete coprocessors such as the Intel 8087 FPU and TMS32010 DSP through graphics engines and dedicated media codecs to deep-learning tensor arrays and wafer-scale hardware. Each era's specialized unit targets a different bottleneck, yet the principles modern accelerators depend on (dedicating silicon to the dominant operation, staging operands in local memory, and minimizing off-chip data movement) were all in place before deep learning arrived.

11.2.2 Domain-specific architectures

A general-purpose CPU core spends most of its energy on instruction fetch and decode, branch prediction, speculative execution, and cache coherence rather than arithmetic: executing one scalar instruction can burn hundreds of picojoules of control overhead just to perform a floating-point operation that costs a few picojoules. For decades, Moore's law[3] (Moore 1998) and Dennard scaling (Dennard et al. 1974) hid that overhead by letting operating voltage fall as transistors shrank, so clock frequencies rose at constant power density. When leakage current halted voltage scaling in the mid-2000s, frequencies stalled against the power wall of section 2.2, and adding more general-purpose cores ran into Amdahl's law and **dark silicon**, the thermal limit under which only a fraction of a chip's transistors can switch at full speed simultaneously (Esmaeilzadeh et al. 2011). As Hennessy

[3] **Moore's law**: Gordon Moore's 1965 observation, revised in 1975, that the number of transistors per chip doubles roughly every two years (Moore 1998). It describes density, not performance; performance tracked density only while Dennard scaling held.

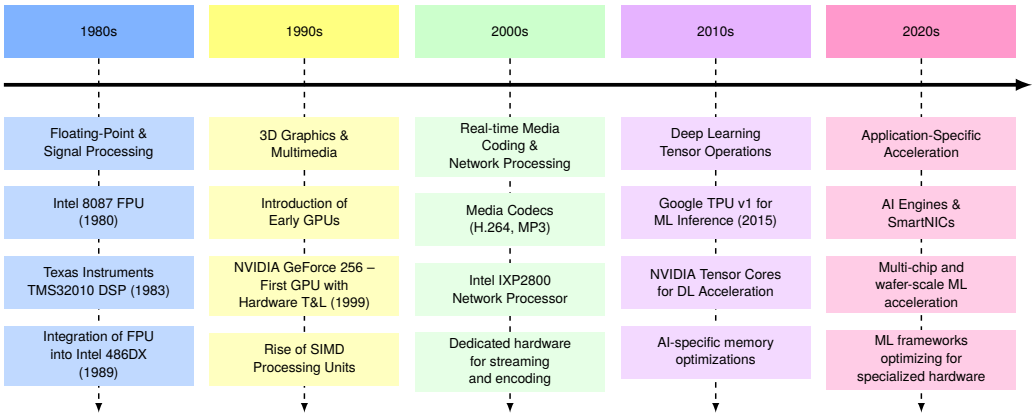


Figure 11.3: Hardware Specialization Timeline: Computing architectures evolve through recurring cycles of specialization and integration across five eras. Each era introduces domain-specific hardware to resolve an important memory or arithmetic bottleneck, from off-chip FPUs to GPUs, codecs, and TPU/Tensor Core array units. Many of these capabilities later appear alongside CPUs in broader systems, even when the specialized units remain distinct.

and Patterson (2019) argued in their Turing Lecture, the way past this wall is **domain-specific architectures (DSAs)**,[4] hardware that gives up general-purpose control machinery to spend its power budget directly on the operations a target domain performs.

A DSA wins by specializing every layer of the machine around the workload's invariants. Its data path is custom-built; matrix units in AI accelerators are often **systolic arrays**, grid-like networks of processing elements that compute and forward data directly to neighboring units, so a value fetched once from on-chip memory is reused across an entire row or column of multiply-accumulate operations instead of returning to a register file after each use. Its memory hierarchy follows the workload's reuse pattern, with software-managed scratchpads and prefetching replacing hardware-managed caches for deterministic tensor streams. Its instruction set encodes coarse tensor operations, so one macro-instruction configures thousands of arithmetic units. The same recipe produced smartphone video codecs, genomics aligners, and cryptographic application-specific integrated circuits (ASICs) wherever a kernel stabilized (Sullivan et al. 2012; Turakhia et al. 2018; Bedford Taylor 2017).

The pressure to specialize has grown because model demand has outrun single-device hardware scaling. The measured trend is the historical training-compute curve charted in chapter 1, whose fitted doubling time runs from 3.4 to about 6 months depending on the window, far faster than the roughly two-year doubling of transistor density (Amodei and Hernandez 2018a; Epoch AI 2024). No single chip closes a gap that compounds at that rate; parallelism and co-design must close the rest.

[4] **DSA (domain-specific architecture)**: A DSA that excels at dense matrix multiplication may perform worse than a CPU on irregular workloads such as graph traversal, so its gain on the target domain must justify the software, compiler, and ecosystem cost of a narrower machine (Hennessy and Patterson 2019, 2017).

↩ The measured growth of training compute and its doubling time are established in section 1.8.

11.2.3 Machine learning hardware specialization

When Google projected in 2013 that three minutes of daily voice search per user would require doubling its global CPU fleet just to run neural-network inference (example 11.2), the bottleneck was not a shortage of servers but a mismatch between neural networks and general-purpose cores. Many important neural-network workloads concentrate their work in regular tensor operations, such as dense matrix multiplications and convolutions, with substantial data parallelism, predictable reuse, and tolerance for reduced precision. Other ML operations remain irregular, bandwidth bound, or too small to fill a wide engine, so an accelerator need not speed up every part of a model; hardware built to exploit the regular portion forms the class of ML accelerators.

Example 11.2: The TPU capacity cliff

Scenario: Google projected in 2013 that user adoption of voice-search speech recognition (3 minutes/day/user) would require doubling global data center capacity to run CPU-based neural network inference (Jouppi et al. 2017).

Diagnosis: General-purpose CPU server capacity was economically unable to absorb the exponential growth in inference FLOPs, threatening a capital infrastructure bottleneck.

Systems lesson: Custom hardware acceleration becomes a candidate when workload volume makes the existing fleet uneconomical. Workload reuse, supported operations, development cost, and server total cost of ownership determine whether a custom ASIC is preferable to software changes or other hardware.

A general-purpose CPU is a poor host for this work. Take the chapter's reference CPU, the low end of the 1–2 TFLOP/s baseline used in the definition of hardware acceleration, and assume it sustains 5 percent–10 percent of its peak on a neural network workload. The upper endpoint then delivers approximately 100 GFLOP/s (billions of floating-point operations per second). CPUs optimize for single-thread performance and irregular memory access, whereas neural networks require massive parallelism and predictable data streams, and a single layer that touches gigabytes of parameters overwhelms cache hierarchies designed for smaller working sets.

The deeper mismatch is energy. Per event, accessing a 32-bit value from DRAM costs about 640 pJ in the 45 nm estimates of Horowitz (2014), roughly 173× the 3.7 pJ of one FP32 multiply, the same ratio table 4.6 reported for the data pipeline. The roughly 145-fold gap quoted with the iron law in section 1.7 comes from the same table but compared one DRAM byte, a quarter of this 32-bit access, with one FP16 operation. Exact values vary by technology node and design, but a gap of two orders of magnitude, the energy-movement principle (7), makes data movement a primary optimization target (Sze et al. 2017). Purpose-built accelerators such as TPUs sustain high utilization on dense kernels because systolic arrays and similar structures maximize data reuse while minimizing movement.

Data access can dominate the energy budget of a memory-intensive kernel.[5] On a logarithmic scale, representative 45 nm operation energies span more than four orders of magnitude, from an INT8 addition (0.03 pJ) and an FP32 multiply (3.7 pJ) to an off-chip DRAM fetch (640 pJ) (figure 11.4). This ladder is the chapter's single reference for what moving a value costs; later sections cite it rather than restate it.

Training and inference load an accelerator differently. Training computes gradients, typically in mixed precision with lower-precision tensor operations and higher-precision accumulation (Micikevicius et al. 2018), and backpropagation retains or recomputes activations, raising memory demand. Inference requires only the forward path and may use INT8 or INT4 when the model and kernels support it. The service objective, whether tail latency, throughput, cost, or energy, then decides which accelerator fits.

5 **Von Neumann bottleneck**: Separating storage from compute requires instructions and data to cross interfaces rather than remain in local registers. The exact cost of each crossing changes with technology and operation, but locality remains a first-order design concern.

↩ The activation memory that backpropagation must retain is established in section 8.3.3.1.

Definition 11.3: ML accelerator

Machine Learning Accelerators are processors or specialized units designed primarily for common neural-network tensor operations and dataflows. They pursue high R_{peak} and data reuse on matching workloads by devoting more resources to arithmetic and local data movement than a general-purpose execution path can.

1. **Significance**: An ML accelerator's defining feature is not raw arithmetic alone but a balanced path that supplies operands and retains reusable values nearby. The H100 provides 3.35 TB/s of device-memory bandwidth, about 16.8× the illustrative 200 GB/s host-DRAM path used here (Choquette 2023). This is a ceiling comparison, not a guarantee: sufficient arithmetic intensity, concurrency, and reuse are still required to approach its 989 TFLOP/s dense FP16/BF16 peak.
2. **Distinction**: The gains are conditional on operation support, parallel work, regular data access, and a software stack that selects the intended path. An ML accelerator can be orders of magnitude faster than a reference CPU on large dense matrix multiplication, yet lose much of that advantage on small, irregular, synchronization-heavy, or control-heavy workloads. Transfers and unsupported operators can dominate even when the accelerated kernel itself is fast.

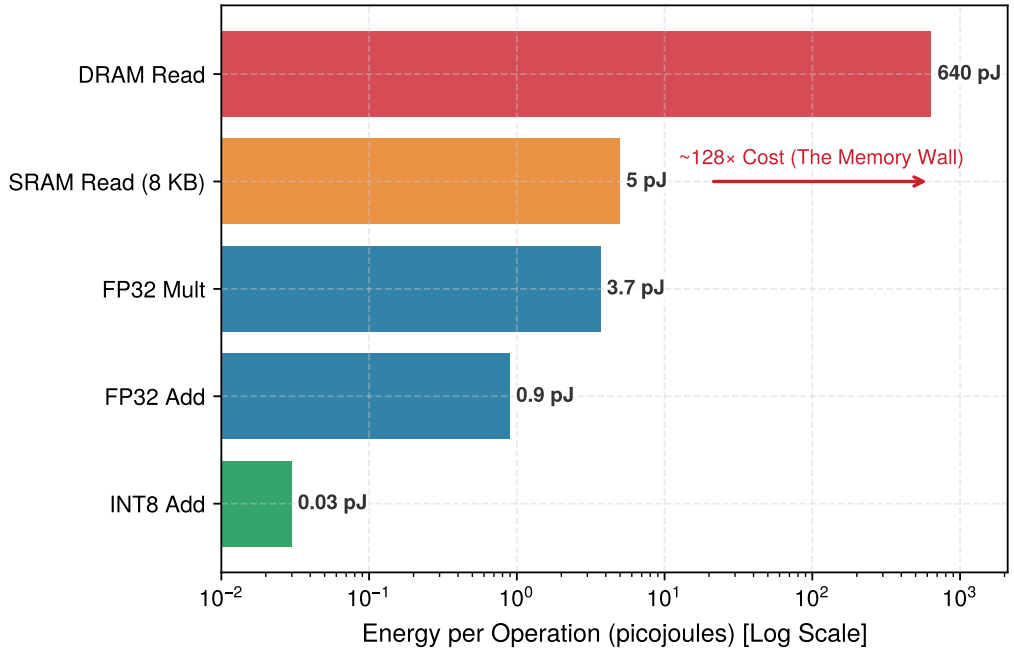


Figure 11.4: The Energy Hierarchy: Representative 45 nm energy estimates on a logarithmic scale from Horowitz (2014). An off-chip DRAM access costs about 173× an FP32 multiply and about 128× a read from an 8 KB SRAM array in this model; a small, register-adjacent L1 access (about 0.5 pJ in the registry used elsewhere in this chapter) is cheaper still. The values are not universal device specifications; they illustrate why reuse in registers and SRAM can matter more than reducing arithmetic count alone.

3. **Common pitfall**: A frequent misconception is that ML accelerators always accelerate ML. Approaching peak throughput requires a compatible precision, operation shape, layout, and enough parallel work to keep the relevant units busy, and memory and interconnect paths must sustain that work. A batch-1 autoregressive request may therefore use only a small fraction of a large training accelerator's arithmetic capacity even though the same device performs well on batched prefill or training.

Deployment context decides which constraint binds. In data centers it is often time-to-result for training large models. An NVIDIA H100 trades hundreds of watts of power for high throughput (Choquette 2023), and Google's TPUv4 makes a similar trade through systolic arrays and high-bandwidth memory (Jouppi et al. 2023); whether either lowers total cost depends on utilization, rental price, workload scaling, and the electricity rate.

At the edge, the priority shifts toward energy per inference and hard latency or power limits. A smartphone camera or always-on audio path inside a few-watt budget cannot adopt a data center memory system, so edge architectures reduce movement through local scratchpads, tight integration, and dynamic voltage and frequency scaling. Because the binding constraint differs by setting, no single architecture dominates, yet the memory wall governs both. Data center chips buy capacity and bandwidth with HBM, while edge chips depend on proximity and reuse.

Checkpoint 11.2: The accelerator gate

The energy hierarchy decides when specialization pays.

Energy hierarchy

- ☐ **Data movement cost**: explain why a DRAM access costs about two orders of magnitude more energy than the FP32 multiply that consumes its operand?
- ☐ **Architectural response**: why would keeping and reusing operands near arithmetic units reduce movement energy? Use this prediction when the following sections develop systolic arrays and Tensor Cores.

Selection logic

- ☐ **Training vs. inference**: how do training and inference objectives affect precision, memory capacity, bandwidth, and latency requirements? Explain why both phases can need high memory bandwidth.

11.2.4 The integration bottleneck

Packing thousands of multiply-accumulate units onto a die buys nothing if those units spend most cycles stalled waiting for operands to cross the memory hierarchy. From the 8087 onward, hardware has evolved to fit the algorithm's dominant bottleneck (Palmer 1980; Sze et al. 2017; Jouppi et al. 2017), and for dense matrix multiplications and convolutions the remaining difficulty is feeding the arithmetic units. The integration bottleneck is the memory wall seen from inside the accelerator: transporting operands and results through the memory hierarchy quickly and efficiently enough to keep thousands of parallel compute units busy. Specialized silicon delivers its performance-per-watt advantage only when the workload's demand concentrates into regular operations that align with the hardware's data paths.

Neural-network kernels make this integration bottleneck tractable because their arithmetic, dataflow, and numerical requirements reinforce one another. Large matrix multiplications and convolutions expose abundant data parallelism that dense processing-element arrays can execute concurrently, and because static or compiled tensor graphs move those operands along predictable paths, compilers can tile and stage transfers into local scratchpads[6] instead of relying exclusively on hardware-managed caches. Those staged operands also tolerate reduced precision, allowing supported 8-bit or 4-bit paths to pack more values into each scratchpad and multiply them more densely (Dally et al. 2021; Dally 2023). Workloads dominated by dynamic control flow, large embedding tables, small tensors, variable shapes, or strict precision requirements break these assumptions and remain bound by memory latency and irregular access.

Once arithmetic capacity is abundant, the central engineering challenge shifts to staging data close to compute. Given the 173× energy penalty of off-chip DRAM transfers, accelerator architects deliberately allocate silicon area and thermal budgets to stacked HBM, wide on-chip SRAM, and software-managed scratchpads rather than packing in additional floating-point logic.[7] Those are the memory tiers drawn in figure 11.1, and they occupy as much of the design as the arithmetic units they feed.

The specialization evident in processing elements, vector pipelines, and tiered scratchpads reflects the computational profile of neural networks, which repeatedly invoke a compact set of linear-algebraic and element-wise primitives. Within the D·A·M framework, an algorithmic change produces a wall-clock speedup only when its computational structure and memory access pattern map onto these primitives; one that cuts theoretical operations without aligning with the hardware's data paths leaves compute units starved. The next section identifies those primitives, beginning with the multiply-accumulate that carries nearly all of ResNet-50's arithmetic.

11.3 Compute Primitives

A single forward pass through a neural network layer can issue millions or billions of MAC operations, and executing them one scalar instruction at a time would swamp the processor in instruction-fetch and register-access overhead. Across fully connected, convolutional, and attention layers, learned linear transformations reduce to these repeated MACs, whose regular tensor structure exposes the parallel work and operand reuse that hardware specialization requires. MACs do not dominate every model or phase (normalization, routing, indexing, communication, and element-wise operations can become limiting), but they account for much of the peak arithmetic capacity in modern ML

6 **Scratchpad memory**: When a compiler can determine a tensor kernel's access pattern, it can explicitly stage tiles in fast, software-controlled local memory. This avoids some tag, replacement, and coherence machinery, but transfers, synchronization, banking, and capacity must be scheduled correctly. Scratchpads complement rather than universally replace caches because irregular or shared data may still benefit from hardware management. Google's TPU v1, for example, used a 24 MB software-managed Unified Buffer for intermediate activations, while weights and instructions used other paths (Jouppi et al. 2017).

7 **HBM**: The A100 and H100 generations provide 2.0–3.4 TB/s of device-memory bandwidth through stacked memory and wide interfaces, compared with 760 GB/s for the GDDR6X reference used here (NVIDIA Corporation 2020a; Choquette 2023). Raising the bandwidth ceiling can move some kernels toward the compute-bound side of the roofline, but only if their access pattern and arithmetic intensity can use it. HBM also increases packaging and system cost, so its value depends on the workload's bandwidth demand.

accelerators. Whether they dominate elapsed time still depends on tensor shape, data movement, and the surrounding operators.

The hardware units that exploit these patterns are **AI compute primitives**: specialized functional blocks, each optimized for a particular class of operation. Three primitives are especially common in accelerators, each targeting a distinct computational pattern found in neural networks; at the framework level, a single declarative statement encapsulates thousands of their arithmetic operations (listing 11.1).

Listing 11.1: Dense Layer Abstraction: High-level framework APIs encapsulate 131,072 MACs (256 inputs times 512 outputs) in a single function call, hiding the computational complexity from developers while enabling automatic hardware optimization.

```
# Framework abstracts compute-intensive operations
dense = Dense(512)(input_tensor)  # 256x512 MACs per sample
```

Beneath this API call, an autograd or compilation engine lowers the layer into a computational graph. Listing 11.2 exposes a bulk matrix multiplication and bias broadcast, followed by an optional element-wise activation; the activation is a separate stage rather than an implicit part of the shown `Dense` call.

Listing 11.2: Matrix Operation Expansion: Each dense layer decomposes into matrix multiplication and element-wise operations, exposing the dominant compute pattern that many neural-network kernels are built around.

```
# Linear transformation work scales with input_dim x output_dim x
# batch.
output = (
    matmul(input, weights) + bias
)  # Matrix multiply dominates cost
output = activation(
    output
)  # Element-wise: proportional to output_dim x batch
```

While graph representations group these steps into coarse tensor operators, execution on physical hardware requires scheduling individual scalar operations. Lowering the matrix multiplication to loop nests (listing 11.3) makes the execution structure explicit: three nested iterations traverse the batch, output feature, and reduction dimensions, establishing $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ arithmetic complexity.

Listing 11.3: Processor-Level Execution: Nested loops reveal the $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ multiply-accumulate operations that accelerators must execute, with 4.2M MACs for B=32, d_{in}=256, d_{out}=512 configurations.

```
# Total operations: batch_size × output_size × input_size MACs
for n in range(batch_size):  # Batch dimension: parallelizable
    for m in range(output_size):  # Output neurons: parallelizable
        sum = bias[m]  # Initialize accumulator
        for k in range(input_size):  # Reduction dimension: sequential
            sum += input[n, k] * weights[k, m]  # MAC operation
        output[n, m] = activation(sum)  # Nonlinear transformation
# Example work scales as batch_size × output_size ×
# input_size multiply-accumulate operations
```

These nested loops expose three recurring computational patterns: element-wise operations along vectors, matrix-level multiply-accumulate reductions, and nonlinear transformations. Their ubiquity justifies dedicated hardware execution paths: vector pipelines stream independent elements and reductions, matrix engines organize multiply-accumulate grids across spatial tiles, and special-function units approximate nonlinear math. The boundaries between these units are architectural rather than mathematical; an optimizing compiler frequently fuses operations across all three paths within a single kernel invocation to eliminate intermediate memory traffic.

11.3.1 Vector operations

In the scalar loop nest of listing 11.3, evaluating a batch of 32 samples through a 256-to-512 dense layer requires 4.2M MACs, and a scalar processor must fetch and decode separate instructions to load each input-weight pair, multiply-accumulate them, and advance the loop counters. Vector architectures eliminate this per-element control overhead by packing multiple data elements into wide register files (typically 128 to 512 bits) and applying a single decoded instruction across parallel execution lanes simultaneously.

RISC-V,[8] an open instruction-set architecture (ISA) with a standardized vector extension (Waterman et al. 2013), provides a concrete vehicle for this execution model. The contiguous loads in listing 11.4 assume that the selected weight column has been packed into a contiguous vector; the row-major `weights[k, m]` access in the scalar example is otherwise strided. In RISC-V vector assembly, code is decoupled from fixed hardware widths: the instruction `vsetvli` queries the hardware to determine how many elements fit into available registers, processes that chunk, and loops until all features are exhausted.

[8] **RISC-V (reduced instruction set computer V)**: The open ISA allows hardware teams to add custom ML instructions, including vector dot products, activation functions, and sparse tensor operations, without changing the base ISA. The trade-off is software ecosystem maturity: custom extensions require corresponding compiler, library, and runtime support, which can limit portability and increase integration work.

Listing 11.4: Vectorized Dot-Product Loop: This illustrative loop processes several element pairs per iteration and reduces them into one scalar dot product.

```
fmv.w.x fa0, zero          # Scalar dot-product accumulator
fmv.w.x ft1, zero          # Zero seed for vector reduction
loop_feature:
    vsetvli t0, feature_cnt, e32, m1, ta, ma
    vle32.v v1, (in_ptr)
    vle32.v v2, (wt_ptr)
    vfmul.vv v3, v1, v2
    vfmv.v.f v0, ft1
    vfredusum.vs v4, v3, v0
    vfmv.f.s ft0, v4
    fadd.s fa0, fa0, ft0
    slli t1, t0, 2         # Four bytes per FP32 element
    add in_ptr, in_ptr, t1
    add wt_ptr, wt_ptr, t1
    sub feature_cnt, feature_cnt, t0
    bnez feature_cnt, loop_feature
```

Each pass of the loop in listing 11.4 asks the hardware how many 32-bit elements fit in its vector registers (`vsetvli`), loads that many input and weight elements with two wide loads (`vle32.v`), multiplies them lane by lane (`vfmul.vv`), reduces the products into a partial sum (`vfredusum.vs`), and advances the pointers until no elements remain. One decoded instruction thus does the work of a whole chunk of scalar instructions.

Across iterations, the vector multiply instruction processes multiple data elements concurrently while wide loads amortize instruction decode and memory-request overhead. Because the runtime vector length dynamically adapts to the implementation's hardware width (`VLEN`), the identical binary executes correctly on both narrow embedded vector units and wide data-center pipelines. For this 256-feature workload, an 8-lane implementation breaks the reduction into approximately 524,288 vector chunks across the batch.

Beyond dense inner products, deep learning workloads rely on a broader set of vector instructions. Table 11.2 maps these primitive vector operations to the specific neural network layers that depend on them.

Table 11.2: Vector Operations: Core vector operations map directly to deep learning primitives: reductions implement pooling layers, gathers enable embedding lookups, scatters update embedding gradients, and masked operations handle attention masks. These mappings show how vector hardware supports common ML primitives.

Vector operation	Description	Neural network application
Reduction	Combines elements across a vector (for example, sum, max)	Pooling layers, attention score computation

Vector operation	Description	Neural network application
Gather	Loads multiple nonconsecutive memory elements	Embedding lookups, sparse operations
Scatter	Writes to multiple nonconsecutive memory locations	Gradient updates for embeddings
Masked operations	Selectively operates on vector elements	Attention masks, padding handling
Vector-scalar broadcast	Applies scalar to all vector elements	Bias addition, scaling operations

The benefit extends beyond instruction count. Contiguous vector loads can use memory interfaces efficiently, and control overhead is amortized across several data elements. Gather, scatter, masking, and short tails may use fewer lanes or require extra memory transactions, so vector width alone does not guarantee proportional speedup. The architectural pattern is not new. The Cray-1[9] used vector registers and pipelined functional units for scientific computing in the 1970s (Jordan 1982); modern ML processors apply related principles at much larger commercial scale. In ResNet-50 the vector path carries the ReLU after every convolution and the residual addition that closes every bottleneck block. Each touches every activation element once and performs about one operation per element, so on its own it is limited by how fast the activations stream past, not by lane count.

Vector operations excel at element-wise transformations such as activation functions, where each output depends only on *its corresponding input*. Neural networks, however, also require structured computations where each output depends on *all inputs*, the weighted sums that define layer transformations. These many-to-many operations naturally express themselves as matrix multiplications, the second compute primitive.

9 **Cray-1 vector legacy**: The Cray-1 (1975) used 64-element vector registers with pipelined functional units, allowing a stream of elements to advance through arithmetic operations without issuing one scalar instruction per element. Modern accelerators extend the same principles of wide registers, pipelined execution, and data reuse to vector and matrix tiles.

11.3.2 Matrix operations

Evaluating a dense layer one vector dot product at a time wastes reuse: each weight row is loaded for one sample and discarded before the next sample in the batch arrives, and each input vector is re-read for every output neuron. Matrix multiplication eliminates this redundant traffic by organizing work across two-dimensional tiles of weights, activations, and gradients, accounting for much of the arithmetic in dense neural networks (Goodfellow et al. 2016). Hardware and libraries tile those dimensions so blocks of weights and activations are reused across an entire tile while partial sums accumulate locally, though small or irregular matrices can still leave tile capacity unused.

11.3.2.1 Matrix operations in neural networks

Neural network computations decompose into hierarchical matrix operations. Listing 11.5 captures this hierarchy through a linear layer that transforms input features into output neurons over a batch. Its expanded mathematics uses a column-batch convention and adds an optional ReLU; this illustrates the arithmetic rather than the framework API's internal tensor layout or an activation implicit in `nn.Linear`.

Listing 11.5: Linear-Layer Operation Hierarchy: The example expands a linear layer into a matrix multiply and bias broadcast, then applies element-wise ReLU, separating the matrix and vector work that hardware must execute.

```
layer = nn.Linear(256, 512)  # Layer transforms 256 inputs to 512 outputs
output = layer(input_batch)  # Process a batch of 32 samples

# Framework Internal: Core operations (column-batch convention)
Z = matmul(weights, input)  # Matrix: transforms [256×32]
# input to [512×32] output
Z = Z + bias  # Vector: adds bias to each
# output independently
output = relu(Z)  # Vector: applies activation to
# each element independently
```

The scale of this matrix work follows from the layer dimensions. Each output neuron (512 total) must process all input features (256 total) for every sample in the batch (32 samples). The weight

matrix alone contains $256 \times 512 = 131{,}072$ parameters that define these transformations, illustrating why efficient matrix multiplication dominates performance considerations.

Neural networks employ matrix operations across architectural patterns beyond simple linear layers. Convolution operations transform into matrix multiplications through the im2col technique,[10] enabling efficient execution on matrix-optimized hardware. Listing 11.6 illustrates these applications.

Listing 11.6: Matrix Patterns Across Architectures: Linear layers, attention mechanisms, and convolutions all reduce key work to matrix multiplications, making matrix hardware the shared primitive across modern neural architectures.

```
hidden = matmul(weights, inputs)
# weights: [out_dim x in_dim], inputs: [in_dim x tokens]
# Result combines all inputs for each output

# Attention Mechanisms - Multiple matrix operations
Q = matmul(Wq, inputs)
# Project inputs to query space [query_dim x tokens]
K = matmul(Wk, inputs)
# Project inputs to key space [key_dim x tokens]
attention = matmul(Q.T, K)
# Compare all query tokens with all key tokens [tokens x tokens]

# Convolutions - Matrix multiply after reshaping
patches = im2col(input)
# Convert [H x W x C] image to matrix of patches
output = matmul(kernel, patches)
# Apply kernels to all patches simultaneously
```

[10] **Im2col (image-to-column)**: The im2col lowering of section 6.3.3 turns convolution into a matrix multiplication by unfolding each receptive-field patch into one row or column of a dense matrix. An explicit materialization can duplicate overlapping values and expand storage by up to roughly the kernel area before edge, stride, and channel effects are considered. Production libraries often use implicit-GEMM or direct-convolution kernels to obtain matrix-friendly execution without materializing the full expanded matrix.

These examples differ in tensor shape and reuse pattern, which determines how efficiently each multiplication occupies a matrix unit. Linear layers reuse one weight matrix across a batch; attention forms projections and then a token-to-token score matrix; convolution reuses each kernel across many image patches. Expressing all three as matrix multiplication exposes a common interface to hardware, but it does not make their execution costs identical. Batch size, sequence length, channel count, and tiling determine whether operands are reused locally or repeatedly fetched from memory. For ResNet-50 the lowering is nearly total: every convolution and the final fully connected layer become matrix multiplications, so almost all of its operations run on whatever matrix hardware the accelerator provides, and the INT8 profile of chapter 10 is in effect a statement about that hardware.

11.3.2.2 Matrix operations hardware acceleration

Accelerators therefore need units that handle whole tiles. Listing 11.7 demonstrates a representative dedicated matrix unit that processes an entire 16×16 block at once, illustrating why matrix instructions and Tensor Cores can deliver much higher throughput than scalar or vector-only execution paths (NVIDIA 2017; Intel Corporation 2021a).

Listing 11.7: Illustrative Matrix-Unit Operation: Representative load, matrix-multiply-accumulate, and store instructions operate on matrix blocks rather than individual scalar elements.

```
mload mr1, (weight_ptr)     # Load e.g., 16x16 block of
                            # weight matrix
mload mr2, (input_ptr)      # Load corresponding input block
matmul.mm mr3, mr1, mr2     # Multiply and accumulate entire
                            # blocks at once
mstore (output_ptr), mr3    # Store computed output block
```

The illustrative unit operates on matrix tiles rather than individual elements. A full $16 \times 16 \times 16$ tile product contains $16^3 = 4{,}096$ multiply-accumulate operations, but the cycles required depend on the actual instruction and architecture. Sustained throughput also depends on the number of matrix units, operand precision, issue rate, pipeline behavior, and whether memory can keep the units fed. Matrix units complement vector execution by accelerating structured many-to-many transformations.

Much like vector processing, matrix acceleration has deep historical roots; digital signal processors (DSPs) and graphics processors optimized matrix computations in the 1980s–1990s for image processing, scientific computing, and 3D rendering (Owens et al. 2008; Hwu 2011). Neural networks have made matrix multiplication commercially dominant, driving the integration of two-dimensional systolic arrays and matrix execution engines (such as NVIDIA Tensor Cores and Google TPUs) directly into processor datapaths.

Linear algebra constitutes only part of the neural network execution pipeline. Between linear transformations, neural networks introduce nonlinear activation functions and normalization layers. While piecewise-linear functions such as ReLU map to simple vector comparison instructions, transcendental computations (such as exponentials in softmax and GELU, or reciprocal square roots in normalization) cannot be computed directly on multiply-accumulate datapaths without incurring severe cycle penalties. Table 11.3 summarizes the execution characteristics of the three primary compute primitives and their mapping to neural network operations.

Table 11.3: Operation Characteristics: Matrix operations excel at many-to-many transformations, while vector units handle element-wise operations and reductions such as activation and normalization. Special-function paths handle transcendental arithmetic through architecture-specific approximation or dedicated logic. The operation structure determines which hardware primitive is appropriate.

Operation type	Best For	Examples	Key characteristic
Matrix Operations	Many-to-many transforms	Layer transformations, attention, convolutions	Each output depends on multiple inputs
Vector Operations	Vector and reduction work	Activation functions, layer normalization, element-wise gradients	Uses element-wise operations and vector reductions
Special-Function Paths	Transcendental arithmetic	Exponential, logarithm, reciprocal square root	Uses architecture-specific approximation or dedicated logic

11.3.3 Special function units

Evaluating an exponential in softmax, a hyperbolic tangent in GELU, or a reciprocal square root in layer normalization on standard multiply-accumulate hardware requires a multi-instruction polynomial approximation that can stall the pipeline between fast matrix kernels. Special Function Units (SFUs) and specialized instruction paths remove this bottleneck, completing the chapter's trio of processing primitives. The need is not new: floating-point coprocessors addressed scalar arithmetic bottlenecks (Palmer 1980), and digital signal processors added specialized arithmetic for signal-processing workloads (Smith 1997). An implementation may use a standalone unit, vector approximation instructions, or a library sequence, trading accuracy, latency, and throughput. In neural networks, activation, normalization, and softmax operations can become the limiting stage between matrix kernels when repeated memory passes or transcendental functions cap throughput.

11.3.3.1 Nonlinear functions

A common layer sequence illustrates how non-matrix operators enter the execution pipeline (Goodfellow et al. 2016). Listing 11.8 chains a linear transformation with ReLU activation and batch normalization, operations that appear trivial in Python framework code but impose distinct arithmetic and memory constraints on hardware.

Listing 11.8: Linear–Nonlinear Layer Sequence: A linear layer, ReLU, and BatchNorm combine matrix, element-wise, and normalization work in one common layer sequence.

```
layer = nn.Sequential(
    nn.Linear(256, 512), nn.ReLU(), nn.BatchNorm1d(512)
)
output = layer(input_tensor)
```

Listing 11.9 expands this sequence into its arithmetic primitives, isolating the element-wise thresholding of ReLU from the cross-batch reductions and reciprocal square root required by batch normalization. Here rows are samples and columns are features; `weights` has input features by output features, while `bias`, `gamma`, and `beta` contain one value per output feature. Keeping the reduced batch dimension permits those per-feature statistics to broadcast across samples.

Listing 11.9: Nonlinear Operation Expansion: Expanding the layer sequence exposes ReLU comparison, BatchNorm reductions, variance computation, and square-root normalization around the matrix multiply.

```
Z = matmul(input, weights) + bias  # Linear transformation
H = max(0, Z)  # ReLU activation
mean = reduce_mean(H, axis=0, keepdims=True)  # BatchNorm statistics
var = reduce_mean((H - mean) ** 2, axis=0, keepdims=True)  # Variance computation
output = gamma * (H - mean) / sqrt(var + eps) + beta  # Normalization
```

11.3.3.2 Hardware implementation of nonlinear functions

Translating these mathematical functions to silicon exposes sharp differences in memory traffic, instruction count, and pipeline utilization. For instance, batch normalization (Ioffe and Szegedy 2015) demands multi-pass statistics gathering across memory, variance accumulation, and a reciprocal square root, while softmax requires computing exponentials across an entire normalization dimension. While a rectified linear unit (ReLU) is mathematically simple, a naive scalar implementation incurs branch overhead or redundant memory round-trips unless fused into adjacent kernels. Listing 11.10 contrasts the memory and dependency overheads of pointwise activation against multi-pass normalization in an unfused loop model.

The label *nonlinear* hides three different execution patterns. Pointwise functions such as ReLU leave elements independent and therefore map naturally to vector lanes or parallel threads. Reductions such as the mean and variance in batch normalization introduce cross-element dependencies: partial results must be combined before later stages can proceed. Transcendental functions such as exponential, logarithm, and reciprocal square root add a numerical trade-off because their implementations balance latency, precision, and valid input range. The dependency pattern, not the mathematical label alone, determines whether vector arithmetic, a reduction tree, or a specialized approximation path is appropriate. A faster special-function path accelerates only the operation it implements; it does not remove surrounding tensor traffic or synchronization.

Listing 11.10 is intentionally dependency-explicit rather than a model of an optimized kernel. The loops expose three state lifetimes: per-element activations, per-feature accumulators, and the final normalized outputs. The normalization parameters cannot be applied until the required statistics are available, so implementations must either retain intermediate activations or recompute them. Training derives those statistics from the current batch, whereas inference can use stored statistics and reduce normalization to an element-wise affine transform. Compilers may fuse legal stages, but the data dependencies determine which transformations preserve the computation. Reading the listing as a dependency graph therefore reveals the durable hardware questions: how often each tensor crosses a memory interface, which intermediates remain local, and where reductions impose synchronization.

Listing 11.10: ReLU and BatchNorm Operations: Neural networks process input data through element-wise selections and multiple normalization passes, highlighting efficiency challenges in naive implementations.

```
for batch in range(32):
    for feature in range(512):
        # ReLU: Naive scalar compare/select; optimized kernels
        # usually implement this branchlessly.
        z = matmul_output[batch, feature]
        h = max(0.0, z)  # Conditional operation

        # BatchNorm: Multiple passes over data
        mean_sum[feature] += h  # First pass for mean
        var_sum[feature] += h * h  # Additional pass for variance

        temp[batch, feature] = h  # Extra memory storage needed

# Normalization requires complex arithmetic
for feature in range(512):
    mean = mean_sum[feature] / batch_size
    var = (var_sum[feature] / batch_size) - mean * mean

    # Square root computation: Multiple iterations
    scale = gamma[feature] / sqrt(var + eps)  # Iterative
    # approximation
    shift = beta[feature] - mean * scale

    # Additional pass over data for final computation
    for batch in range(32):
        output[batch, feature] = temp[batch, feature] * scale + shift
```

The dependency structure in listing 11.10 exposes two costs of naive execution. The pointwise ReLU and normalization scale-and-shift perform little arithmetic per accessed byte, so materializing each pass can place a low bandwidth ceiling on throughput. In this unfused loop model, `temp` is stored and reread after batch statistics are finalized; its physical traffic depends on cache residency and the chosen memory interface. Computing $1/\sqrt{\text{var}+\epsilon}$ also adds a dependent arithmetic path. The listing illustrates an iterative approximation, but actual kernels may use a reciprocal-square-root instruction and refinement, with device- and accuracy-dependent latency. Specialized function paths address this arithmetic cost, while legal fusion and local buffering reduce intermediate traffic. Neither removes the need to complete the reductions before normalization.

11.3.3.3 SFU hardware implementation

SFUs address arithmetic bottlenecks by implementing nonlinear and transcendental operations in specialized functional units rather than multi-instruction software routines. Accelerators deploy dedicated datapath circuits, such as quadratic polynomial interpolators and small lookup tables (LUTs), that evaluate functions like exponential, reciprocal square root, and logarithm within a few pipeline stages. Listing 11.11 illustrates how an instruction set exposes these operations alongside vector memory accesses.

Listing 11.11: Illustrative SFU Operations: Pseudocode shows vector nonlinear operations that may map to specialized execution paths; instruction names and latency are implementation dependent.

```
vld.v v1, (input_ptr)    # Load vector of values (pseudocode)
vrelu.v v2, v1           # Vector ReLU path
vsigm.v v3, v1           # Vector sigmoid path
vtanh.v v4, v1           # Vector tanh path
vrsqrt.v v5, v1          # Vector reciprocal-square-root path
```

Specialized function paths balance circuit area against mathematical precision. While piecewise linear activations such as ReLU require only single-cycle compare-and-select (maximum) datapath

logic, transcendental functions trade off lookup table size and polynomial interpolation degrees against approximation error. Representative mechanisms and their qualitative latencies differ by function (table 11.4).

Table 11.4: Special Function Units: Dedicated paths reduce instruction overhead for nonlinear and reduction primitives. Implementations and latency are architecture-dependent; the table describes mechanisms rather than universal cycle counts.

Function unit	Operation	Implementation strategy	Illustrative latency
Activation unit	ReLU	Compare-and-select or maximum	Low; architecture-dependent
Statistics unit	Mean, variance	Parallel reduction trees	Grows with reduction depth
Exponential unit	exp, log, sigmoid, tanh	Approximation, table lookup, and interpolation	Architecture-dependent
Root/power unit	sqrt, rsqrt	Iterative or approximation hardware	Architecture-dependent

The two lighthouses use this path very differently. ResNet-50 barely touches it: its only activation is ReLU, a comparison, and its only exponentials are the thousand in its final softmax. GPT-2 XL leans on it at every step, because each attention head applies softmax across the whole context and each feed-forward block applies GELU, so its special-function work grows with sequence length while ResNet-50's stays fixed.

Vector operations, matrix operations, and special-function paths establish the foundational arithmetic primitives of ML accelerators, but functional units alone do not dictate sustained throughput. These primitives govern *what* the silicon computes; the execution model and memory hierarchy govern *how* data feeds the datapath. A matrix unit rated for hundreds of teraFLOP/s will idle if memory bandwidth cannot sustain operand delivery, if tile dimensions mismatch register blocking, or if thread synchronization serializes execution. Peak assumes operands are always ready; the execution model decides how much of it a real workload achieves.

11.4 Compute Units

Applying ReLU to a 512-element vector shows why execution models matter: the operation is simple, but throughput depends on whether the hardware treats those 512 comparisons as scalar instructions, as lanes of a single instruction, multiple data (SIMD) unit, as GPU threads, or as tensor-program fragments. Modern AI processors package the three compute primitives into distinct execution units (vector units for vector work, matrix units such as Tensor Cores and systolic arrays for matrix work, and special function units for nonlinear functions) and then group those units into processing elements. That organization defines how computations are structured and exposed to programmers, and it explains why theoretical capability and delivered throughput diverge.

The packaging follows reuse per operand. For the reference low-precision operations, reading an operand from a register file and moving it across on-chip wires can cost more energy than the arithmetic itself, so each primitive goes to the unit whose dataflow supplies the reuse it offers. In D·A·M terms, this partition binds algorithmic primitives to machine execution paths. Routing dense matrix multiplications through vector pipelines saturates register-file read ports and collapses throughput to a small fraction of peak, while routing low-intensity elementwise work through matrix arrays leaves their lanes idle.

11.4.1 From SIMD lanes to SIMT threads

Even a wide CPU vector unit stalls when streaming through gigabytes of tensor data: cache misses freeze the pipeline for hundreds of cycles waiting on main memory, the instruction reorder buffer fills, and the SIMD lanes sit empty. The vector path of section 11.3.1 follows Flynn's SIMD execution model, which amortizes instruction fetch and decode by issuing a single decoded instruction across multiple data elements packed into wide vector registers (Flynn 1966). A 512-bit SIMD register processes sixteen 32-bit floating-point values per instruction—an idea carried into fixed-width CPU vector extensions, Arm's length-agnostic Scalable Vector Extension (SVE) (Stephens et al. 2017), and two-dimensional tile extensions such as Intel's Advanced Matrix Extensions (AMX) (Intel Corporation 2021a)—but those wide registers still live inside a CPU core built around branch

predictors, out-of-order execution logic, and deep cache hierarchies for a few sequential threads. Conditional branching within a vector also requires evaluating divergent paths under bitmasks, further cutting useful lane utilization.

Architectures built on **single instruction, multiple threads (SIMT)** execution resolve these bottlenecks by decoupling the logical programming abstraction from physical hardware execution (Lindholm et al. 2008; Nickolls et al. 2008). Instead of requiring the programmer to pack data into explicit vector registers, SIMT exposes an execution model of independent scalar threads. In GPU architectures, each Streaming Multiprocessor (SM)[11] coordinates thousands of concurrent threads. Hardware warp schedulers group these threads into warps, the 32-thread scheduling groups (wavefronts in AMD architectures) whose granularity already set lane utilization in training kernels, and every thread in a warp executes a common instruction in lockstep across parallel ALUs. This lockstep execution introduces a critical constraint known as **warp divergence**. When threads within a single warp take different paths through a conditional branch, the hardware must execute both required paths sequentially, masking off the inactive lanes during each path. This serialization reduces useful lane utilization and increases execution time, which is why performance-critical ML kernels avoid divergent branching in favor of uniform or predicated control flow.

Rather than relying on large caches to reduce memory latency, GPUs hide latency through massive thread-level concurrency. An SM statically partitions an on-chip register file across all active threads, eliminating context-saving overhead; when one warp stalls waiting hundreds of cycles for an off-chip memory fetch, the warp scheduler selects and issues instructions from another ready warp on the very next cycle. The baseline matrix multiplication kernel in listing 11.12 illustrates how this scalar thread model maps onto hardware. CUDA organizes a grid of threads into blocks: `blockIdx` locates a block in the grid, `blockDim` gives the block dimensions, and `threadIdx` locates a thread within that block. Multiplying a block coordinate by its dimension and adding the within-block coordinate gives the global row or column used by the listing.

11 | **SM (streaming multiprocessor)**: The physical hardware engine that implements the SIMT model by using warp schedulers to coordinate many parallel threads. Maintaining enough active warps can help hide instruction and memory latency, but occupancy alone does not identify the bottleneck. Low occupancy may result from register use, shared-memory use, block dimensions, or insufficient parallel work, while a memory-bound kernel can still have high occupancy.

↩ Warps as the 32-thread unit in which GPUs schedule tensor kernels are established in section 8.4.3.1.

Listing 11.12: SIMT Execution: Each thread processes a unique output element in parallel, demonstrating how SIMT enables efficient matrix multiplication on GPUs.

```
__global__ void matrix_multiply(float* C, float* A, float*
                                B, int N) {  // CUDA kernel
    // Each thread processes one output element
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    float sum = 0.0f;
    for (int k = 0; k < N; k++) {
        // Threads in a warp execute in parallel
        sum += A[row * N + k] * B[k * N + col];
    }
    C[row * N + col] = sum;
}
```

In listing 11.12, the CUDA[12] kernel assigns one output element `C[row * N + col]` to each thread. Adjacent threads in a warp share the same row index while having consecutive column indices (`threadIdx.x`). When threads fetch `B[k * N + col]`, their memory requests target contiguous 4-byte floating-point addresses. The memory controller merges these 32 individual 4-byte loads into a single 128-byte coalesced memory transaction, maximizing DRAM bus efficiency. Concurrently, all threads in the warp read the identical element `A[row * N + k]`, allowing the hardware to broadcast that single fetched value across all 32 lanes. However, this naive kernel runs into the arithmetic intensity law (principle 6). Because each inner-loop iteration performs only two floating-point operations while reloading operands from global memory without on-chip SRAM staging, the kernel remains severely memory bound. Furthermore, executing dense matrix multiplication on scalar SIMT ALUs requires one instruction fetch, decode, and issue cycle for every multiply-accumulate, saturating register file read ports. Overcoming this machine bottleneck requires moving beyond vector-of-scalars execution to specialized two-dimensional matrix units.

12 | **CUDA (compute unified device architecture)**: Released by NVIDIA in 2006, CUDA eliminated the need to disguise general-purpose computations as graphics operations, opening GPUs to scientific and ML workloads through a C-like programming model. The ecosystem it created (cuBLAS, cuDNN, TensorRT) constitutes a software ecosystem whose switching cost can lock the ML training stack to NVIDIA hardware: migrating away requires rewriting or replacing thousands of GPU-optimized kernels, a cost that often exceeds the hardware savings of competing platforms. This software lock-in, not raw silicon performance alone, helps explain why many large ML training stacks remain CUDA-centered.

11.4.2 Tensor Cores

Computing the $\mathbf{Q} \times \mathbf{K}^T$ product for a single transformer attention head over a 2,048-token sequence with 64-dimensional embeddings multiplies a 2048×64 matrix by a 64×2048 matrix: roughly 268.4M MACs, or 536.9 MFLOP when the multiply and add are counted separately. On a scalar processor executing one FLOP per cycle at 2 GHz, that single attention head would take 268.4 ms, and even on GPU SIMT lanes, issuing a separate scalar fused multiply-add for every element pair saturates instruction-issue bandwidth and register-file read ports. Dedicated matrix engines remove both bottlenecks by organizing work around multidimensional tiles and reusing fetched operands across many multiply-accumulate operations on chip. NVIDIA GPUs expose Tensor Cores, the reduced-precision matrix units[13] that mixed-precision training was built to exploit, whereas Google TPUs use matrix units built around systolic arrays; both execute tiled matrix multiplication and accumulation.[14]

Assume, for illustration, that the Tensor Cores of a single streaming multiprocessor complete that 2,048-token attention head in 0.5 ms, an implied rate of about 1.1 TFLOP/s and a small slice of a full GPU's tensor peak. The tiled path is then roughly 536.9× faster than the scalar baseline, and that speedup comes not from a higher clock frequency but from hardwiring tile-level data reuse into the execution datapath. Whether a workload reaches that rate depends on tile reuse, matrix shape, numeric precision, and memory delivery.

In NVIDIA Parallel Thread Execution (PTX), Tensor Core operations are invoked through warp-synchronous matrix instructions (`mma.sync`).[15] The mnemonic encodes the tile geometry and data types: `.m16n8k16` gives the tile dimensions ($M = 16, N = 8, K = 16$), and the trailing types (`.f32.f16.f16.f32`) declare FP16 inputs accumulated into an FP32 destination. In listing 11.13, all 32 threads in a warp execute this single instruction collectively to compute the tile product $\mathbf{D} = \mathbf{AB} + \mathbf{C}$.

Listing 11.13: Tensor Core Operation: Matrix multiplications are performed in parallel across entire matrix blocks, optimizing computational efficiency for neural network training.

```
Tensor Core Operation (example GPU PTX):
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
  {d0,d1,d2,d3},      // Destination registers
  {a0,a1,a2,a3},      // Source matrix A
  {b0,b1},            // Source matrix B
  {c0,c1,c2,c3};      // Accumulator
```

Here inputs **A** (16×16) and **B** (16×8) are FP16, while accumulator **C** and destination **D** (16×8) are FP32. A single instruction evaluates 2,048 multiply-accumulate operations that would otherwise take 64 separate scalar fused multiply-add (`fma`) instructions per thread, which relieves two bottlenecks of conventional SIMT execution: instruction issue bandwidth and register file read-port contention.

Design priorities determine how matrix engines appear in different processor families. GPU Tensor Cores preserve programmability while accelerating general-purpose deep learning kernels. TPU-style designs use large-scale matrix units arranged in systolic arrays to maximize sustained training throughput on dense tensor kernels. Mobile NPUs[16] shrink the same idea into low-power inference blocks, while server CPUs add matrix instruction extensions (AMX-class tiles) for inference and mixed workloads. Each version keeps a different amount of flexibility in exchange for removing a different amount of data movement.

Tensor Cores also changed how each accelerator generation states its peak. Successive generations combined more matrix units with narrower formats (FP16, then INT8, FP8, and FP4) and, from Ampere onward, a structured-sparsity path, so the headline rating rose by more than three orders of magnitude between 2012 and 2024 while measuring a different precision at almost every step (NVIDIA Corporation 2017, 2020a, 2024; Choquette 2023). A peak quoted without its precision and sparsity assumption therefore says little about a given workload. The generation-by-generation ratings, with their precision-specific peaks, bandwidths, and power limits, are collected with the other dated specifications in section E.2.1. Whatever the format, a Tensor Core retires a tile per

[13] **Reduced-precision ML**: Halving operand width halves the bytes per stored element and can support denser arithmetic, although realized throughput depends on the available datapath and kernel (Dally et al. 2021; Dally 2023). NVIDIA's transition from Pascal P100 vector arithmetic to mixed-precision Tensor Cores in Volta V100 increased advertised peak FP16 throughput from about 21.2 to 125 TFLOP/s, roughly 6× across those products (NVIDIA 2017). Precision selection must preserve numerical behavior and use an efficient supported path; the smallest format is not automatically the best one.

[14] **Tensor Core dimension alignment**: NVIDIA Tensor Cores are most efficient when matrix dimensions are aligned to precision- and architecture-specific multiples, such as multiples of 8 or 16 for common FP16/BF16/INT8 paths. Modern cuBLAS and cuDNN can still use Tensor Cores for many nonaligned dimensions, but poorly aligned shapes may trigger less efficient kernels, require padding, or reduce effective throughput (NVIDIA 2024a; NVIDIA Corporation 2021b). This is why model architects often choose embedding and channel dimensions such as 512 rather than 500 and why batch-size-1 inference may fail to reach peak Tensor Core utilization: alignment and arithmetic intensity jointly determine whether the hardware's matrix engines stay full.

[15] **Tensor Core**: A single Tensor Core instruction executes a complete matrix-multiply-accumulate operation on a small tile of data using a dedicated hardware block (NVIDIA 2017; NVIDIA Corporation 2020a). This approach bypasses the overhead of fetching and scheduling dozens of individual arithmetic instructions on general-purpose CUDA cores. Because these blocks constitute a large fraction of a modern accelerator's advertised tensor throughput, failing to use them can leave most of the chip's theoretical peak unavailable to the workload.

instruction only when its operands are already nearby, so accelerators never deploy matrix units on their own; the next level of the execution hierarchy exists to supply that proximity.

11.4.3 Processing elements

High-throughput matrix units cannot be fed directly from global memory: off-chip DRAM and even on-package HBM lack the bandwidth and low latency required to stream fresh operands into dense arithmetic arrays on every cycle. Accelerators resolve this bandwidth deficit by grouping Tensor Cores, vector units, and special function units alongside high-bandwidth register files and scratchpad SRAM inside modular processing elements (PEs). Staging operands inside the PE keeps data adjacent to the functional units, amortizing the latency and energy cost of external memory accesses across repeated arithmetic reuse. The term is used at two scales in this chapter. Here a PE is a tile or core that bundles several units with local memory, the role a streaming multiprocessor plays in a GPU and a core plays in a TPU; inside a systolic array (section 11.4.4), a PE is a single multiply-accumulate cell in the grid.

Processing element design varies because each architecture chooses a different balance between compute density, local memory capacity, and interconnect distance. Graphcore's Intelligence Processing Unit (IPU), for example, distributes computation across tiles backed by dedicated local SRAM (Graphcore 2020). Retaining an operand in one tile saves external traffic, but a consumer on another tile still requires an interconnect transfer. Local storage therefore shifts the mapping problem toward keeping repeated consumers close to their operands.

Across these designs, the binding physical constraint is compute density versus data locality: silicon area dedicated to arithmetic units cannot simultaneously house SRAM storage or routing wires. Packing more execution units raises peak theoretical FLOP/s, but those units idle if memory bandwidth and routing channels cannot deliver operands. A processing element's delivered efficiency therefore depends as much on its local memory hierarchy and interconnect strategy as on its raw arithmetic capability. Systolic arrays push this locality principle to its limit: rather than staging operands in a processing element's local memory and fetching them again for each operation, they pass operands directly from one processing element to its neighbor.

11.4.4 Systolic arrays

Even when operands sit in local SRAM or registers, reading two inputs and writing a partial sum back to a central register file on every multiply-accumulate burns wire energy and port bandwidth that limit how many arithmetic cells a chip can pack. While Tensor Cores amortize register access across instruction-driven micro-tiles, systolic arrays remove intermediate register-file round-trips altogether by wiring hundreds or thousands of processing elements into two-dimensional spatial grids. In a classic 2D systolic array, data streams rhythmically through adjacent cells on regular clock cycles: inputs march horizontally, partial sums accumulate vertically, and stationary weights remain anchored in place. Because every value passed from one processing element to its immediate neighbor participates in an arithmetic operation without returning to a register file or SRAM buffer, nearest-neighbor wires replace memory reads. An operand loaded once at the array edge contributes to N sequential multiply-accumulate steps across an $N \times N$ grid, driving memory traffic down by an order of magnitude. Counting DRAM accesses per multiply-accumulate with and without neighbor-to-neighbor passing makes the saving concrete.

16 **NPU (neural processing unit)**: Mobile NPUs achieve low-power inference by implementing common tensor operations in fixed-function or narrowly programmable hardware rather than as fully general GPU kernels. This architectural commitment can deliver large energy-efficiency gains for supported kernels, but it makes deployment dependent on operator coverage: unsupported functions must fall back to a CPU or GPU path that may be far less efficient for that workload (Sze et al. 2017).

Napkin Math 11.1: The energy advantage of pulsing data

Scenario: The systolic architecture improves energy efficiency by keeping operands local as work pulses through the array; the "Systolic" (heartbeat) metaphor reflects this benefit of reusing operands locally. This worked model compares systolic dataflow with a naive implementation that streams every operand through DRAM:

1. **DRAM-streaming baseline**: Loads **A**, **B**, and the running **C**, computes $\mathbf{A} \times \mathbf{B} + \mathbf{C}$, and writes **C** back without cache or register reuse.
   - **Data movement**: 3 loads + 1 write = 4 DRAM accesses (per operation).
   - **Energy**: $\approx 4 \times 640$ pJ + 1 pJ (compute) = 2561 pJ/op.

2. **Systolic array** (128 × 128): Loads **A** and **B** once at the edges. Data "pulses" through 128 processing elements.
   - **Data movement**: 2 loads per 128 operations = 0.016 DRAM accesses (per operation).
   - **Energy**: ≈ 0.016 × 640 pJ + 1 pJ (compute) ≈ 11 pJ/op.

Systems insight: Under this deliberately naive DRAM-streaming baseline, the worked model gives the systolic array a 232.8× energy advantage. The ratio measures locality, not an inherent gap between vector and systolic arithmetic.

- Concretely, a 128×128 array can sustain 16,384 MACs/cycle with a large energy saving by pulsing data through processing elements instead of repeatedly loading it from DRAM (Horowitz 2014; Jouppi et al. 2023).
- This locality lets dense arrays sustain many simultaneous MAC operations without paying a DRAM access for every operation.
- **Limitation**: The comparison assumes no effective reuse in the baseline and full reuse across the array. Caches, register blocking, array underutilization, and other dataflows change the ratio substantially.

Kung and Leiserson[17] (Kung and Leiserson 1979) first introduced systolic arrays, formalizing their use in parallel computing architectures for efficient matrix operations (Kung 1982). Unlike general-purpose execution units, systolic arrays exploit spatial and temporal locality by reusing operands as they propagate through the grid. Google's TPU exemplifies this architectural approach: in the TPUv4, a 128×128 systolic array of multiply-accumulate units processes matrix operations by streaming data through the array in a pipelined manner (Jouppi et al. 2023). Figure 11.5 traces this 2D pipelined dataflow: a control unit stages operands into input FIFO buffers that stream activation data horizontally across the processing grid, while partial sums accumulate vertically down columns into the bottom adder-accumulator chain. Each processing element performs one multiply-accumulate per cycle and passes its operands to its neighbors, so a value loaded once is reused across an entire row or column rather than refetched from memory.

[17] **Systolic array**: Kung and Leiserson used the term for arrays in which data advances through neighboring processing elements on a regular schedule (Kung and Leiserson 1979). Pipeline fill permits concurrent wavefronts and local reuse. The benefit depends on the stationary operand, tile, buffers, and utilization; it does not guarantee fixed DRAM savings. Irregular workloads and poorly aligned shapes remain a weaker fit because fill, drain, and idle elements consume capacity.

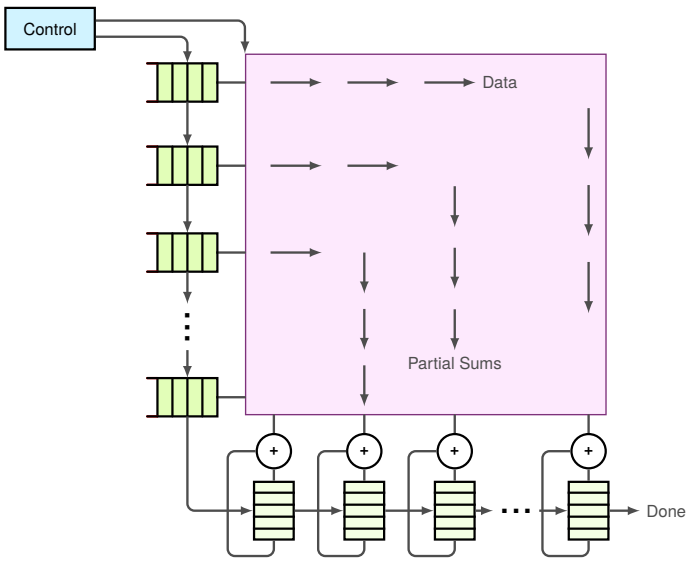


Figure 11.5: Systolic Array Dataflow Architecture: A pipelined grid of processing elements (PEs) passes operands and partial results between neighbors. The illustrated movement is one possible matrix-multiplication dataflow; designs choose which operand or partial sum remains stationary to match reuse, buffers, and tensor shape. Utilization depends on mapping and dimensions.

A 128 × 128 systolic array capable of 16,384 operations per cycle requires a continuous operand stream to maintain utilization. On-chip buffers feed activations and weights to the array edges and are replenished from off-chip memory as needed; reuse within the array avoids fetching every operand from HBM each cycle. The TPU v4's 1,200 GB/s HBM2 bandwidth helps sustain this stream, but off-chip bandwidth can still limit models whose working sets and reuse patterns exceed on-chip capacity.

> Systems Perspective 11.1: Matching architecture to workload
>
> **Challenge**: Systolic arrays must choose which operand to keep stationary in processing-element registers (weights, inputs, or partial sums), and that choice hard-codes the hardware's preference for certain model types. Section 11.7.3 develops the three options as a general mapping decision.
>
> No single accelerator architecture suits every workload. A chip optimized for weight-stationary dataflow (like early TPUs) excels at convolutional neural networks (CNNs), where filter weights are small and reused across many image patches. That same architecture stalls on large language model (LLM) autoregressive inference at batch size 1, where the entire weight matrix must be read from off-chip memory once per generated token with zero batch reuse. No choice of stationary operand recovers reuse that does not exist; the levers are batching, lower-precision weights, and memory bandwidth.

11.4.5 Numerics and sparsity on the datapath

Storing weights in INT8 or pruning half of them to zero shrinks a checkpoint on disk, but if the accelerator lacks a datapath that multiplies narrow integers or skips zero weights in silicon, the runtime must unpack those weights back to dense floating point and executes no faster than the baseline. Number formats and structured sparsity patterns were defined in chapter 10; what turns them into wall-clock speedups—including the ResNet-50 INT8 profile—is the hardware datapath that executes them. Systolic arrays and Tensor Cores often provision higher multiply-accumulate capacity for narrower formats such as FP16, BF16, INT8, and FP8, while halving or quartering the bytes moved per value relative to FP32. That speedup is not automatic: it requires a supported kernel, matching input and accumulation formats, and enough parallel work to keep the narrower datapath full without hitting another bottleneck.

11.4.5.1 Precision trade-offs

Lower precision is not free, for the range-and-precision reasons that forced loss scaling in mixed-precision training and that section 10.3.1 compared format by format. FP16 trades dynamic range for significand bits, BF16 keeps FP32's exponent range at the cost of precision, and INT8 has no exponent at all, so its usable range depends on scale, zero-point, and clipping choices. For the hardware designer, those numerical properties set the silicon cost of each datapath. A narrower format needs a smaller multiplier and moves fewer bytes, but accumulation usually stays wider, as the FP32 accumulator of the Tensor Core instruction in listing 11.13 shows, so part of the area saved in the multipliers is spent keeping sums accurate.

↩ The software side of mixed-precision training, including loss scaling, is established in section 8.6.3.

Precision support is integrated into execution units and memory paths. SIMD and SIMT lanes may support several scalar formats, while Tensor Cores (section 11.4.2) and systolic units (section 11.4.4) expose architecture-specific input and accumulation combinations. Changing storage alone may reduce bytes without producing the advertised arithmetic speedup, because the model must select a kernel that uses the efficient path. Converting FP32 weights to INT8 cuts weight traffic by 4×, but whether that moves a kernel from bandwidth bound to compute bound depends on the operation's original arithmetic intensity, the ridge point of the accelerator's INT8 path, and the overhead of quantization and dequantization (equation 10.4). Tiling and dataflow, not precision itself, determine reuse, while conversion, scaling, or fallback overhead can offset part of the gain. A format is useful only when the kernel and numerical policy can employ it safely.

11.4.5.2 Mixed-precision computing

Modern AI accelerators support mixed-precision execution, in which different stages of one computation use different numerical formats. The training recipe (FP16 or BF16 multiplications with FP32 accumulation) (Micikevicius et al. 2018; Mellempudi et al. 2019) is developed in section 8.6.3, and the inference recipe (INT8 or INT4 weights with selected activations kept wider) in section 10.3. The hardware's part is to execute several formats side by side, with an accumulator wide enough for each.

The shift toward precision diversity is evident in the evolution of AI hardware. Early architectures such as NVIDIA Volta provided Tensor Core support only for FP16, whereas later architectures, including Turing and Ampere, added integer formats and then TF32, a hybrid between FP32 and FP16, alongside BF16 (NVIDIA 2020b; NVIDIA Corporation 2017, 2018, 2020a). The generation-by-generation list of supported formats belongs with the other dated specifications in Appendix E; the durable observation is the direction of the trend, toward more formats, each narrower than the last.

The precision format used in hardware design has cascading implications across the entire system. Reducing an operand from FP32 to FP16 halves its value bytes. When those values dominate traffic, the reduction can substantially lower movement energy; the full energy budget still includes access counts, arithmetic, and other device activity (Horowitz 2014). Simultaneously, Tensor Cores and systolic arrays can pack more lower-precision multiply-accumulate units into the same silicon area, raising peak throughput (Dally et al. 2021; Dally 2023). Lower-precision integer arithmetic also consumes less energy than FP32 in representative technology studies (Horowitz 2014), and the inference-focused TPUv1 was built around 8-bit multiply units (Jouppi et al. 2017). Reduced precision can therefore cut value traffic *and* raise the supported compute ceiling at the same time.

Each narrower format raises the arithmetic ceiling again. The H100 advertises 989 TFLOP/s on its dense FP16 tensor path and 1,979 TFLOP/s on its FP8 path, an 8-bit floating-point format introduced for training and inference (Choquette 2023; Kuzmin et al. 2022; Micikevicius et al. 2022). NVIDIA's Blackwell (B200) architecture extends this trajectory with an FP4 tensor datapath (a 4-bit floating-point format that, unlike INT4, spends some of its four bits on an exponent and so trades uniform spacing for range), rated at up to 9 PFLOP/s dense or 18 PFLOP/s sparse peak throughput per chip (NVIDIA Corporation 2024). Reaching those theoretical ceilings requires numerical representations, kernel implementations, and sparsity patterns that align exactly with the specialized hardware paths. Narrower numerical formats expand the available design space, but numerical stability and dynamic range requirements dictate whether a workload can safely use FP4.

11.4.5.3 N:M structured sparsity mechanics

The sparse rating in that comparison depends on one more datapath feature, one that skips zeros only when they follow a fixed pattern. While unstructured pruning reduces model parameter counts, it rarely translates to hardware speedup because arbitrary nonzero placements destroy memory locality. Hardware accelerators resolve this with N:M structured sparsity,[18] the pruning constraint from model compression that keeps exactly n_{keep} of every m_{group} contiguous values along the reduction dimension nonzero, so memory access stays uniform and indexing predictable across parallel execution units.

Parallel hardware needs this regularity because of how sparse matrices are encoded. Classical formats such as Compressed Sparse Row (CSR) and Block Compressed Sparse Row (BSR) rely on auxiliary index arrays to locate nonzero entries, and the corresponding activation gathers can touch scattered addresses, reducing useful bytes per memory transaction. Varying numbers of nonzeros across rows also create load imbalance and divergent execution. Block sparsity groups retained values into larger dense tiles to restore regularity, at the cost of a more restrictive pruning pattern. In the block formats, a short list of nonzero block indices tells the hardware which tiles to fetch, so zero blocks are never read. Whether an indexed format saves bytes or time at a given sparsity, its break-even point, is derived in section 10.4.2.2; the datapath question here is how regular the retained values must be for parallel hardware to use them.

NVIDIA's Sparse Tensor Cores implement the 2:4 instance of the N:M pattern (NVIDIA Corporation 2020a; NVIDIA 2020a). Their input is the compressed 2:4 encoding, the two retained values of each group plus a 4-bit metadata tag (2 bits per value) recording their original positions. During execution, the memory hierarchy transfers only the compressed weights and their metadata, halving

[18] **N:M structured sparsity**: The 2:4 ratio (50 percent density) used by NVIDIA's Ampere Sparse Tensor Cores is a hardware-friendly compromise: every contiguous four-value group retains two nonzero values, preserving regular indexing while halving the dense value payload (NVIDIA Corporation 2020a). At 2:4, the metadata overhead is compact enough to store alongside the weights without overwhelming the memory-traffic savings, which is the constraint that makes the advertised 2× tensor-math throughput path plausible when kernels and model weights satisfy the pattern.

↩ The N:M and 2:4 structured sparsity patterns and their compressed storage are established in section 10.2.1.3.

↪ The CSR storage format is collected for reference in section C.1.4.

the weight-value payload while adding metadata traffic; total transferred bytes therefore fall by less than half before other overheads. Inside the Sparse Tensor Core datapath, the 4-bit metadata steers an internal multiplexer network that selects the matching two values from an uncompressed four-element activation vector. These selected activations and the two nonzero weights are then routed into a half-sized dot-product multiplier tree. Because the hardware computes only on the nonzeros while streaming compressed weights, this design doubles the peak arithmetic instruction throughput, yielding an advertised 2× tensor-math speedup over dense operations when weights are fine-tuned to satisfy the pattern (figure 11.6).

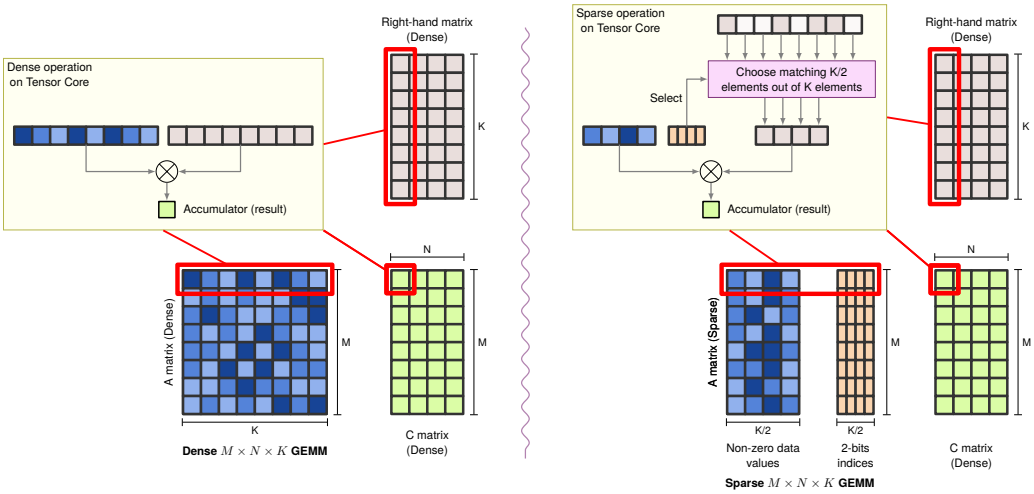


Figure 11.6: 2:4 Structured Sparsity GEMM: Left: standard dense matrix multiplication on Tensor Cores using full 8-element rows. Right: 2:4 sparse multiplication retains two nonzeros in every group of four and uses compact indices to select matching elements from the dense right-hand matrix. A supported sparse inner loop can halve the multiplied values, although metadata handling and the rest of the execution path determine end-to-end speedup. Source: PyTorch blog (PyTorch 2022).

The 2:4 pattern saves about half of the weight bytes and half of the multiplies, minus the metadata, so the reduced weight payload can lower traffic across HBM and on-chip SRAM. A roofline comparison must use consistent accounting: useful dense-equivalent operations can stay fixed while bytes decrease, whereas the count of physically executed sparse operations also decreases. Metadata and activation traffic remain in either traffic budget. Narrower values and skipped zeros therefore help when they reduce the bytes moved and keep the matrix engines fed, and the ResNet-50 INT8 profile collects that benefit only on an execution path that supports both its format and its shapes; how fast such a path can possibly run follows from one division.

Dividing a model's total operation count by peak arithmetic throughput produces an idealized lower bound on execution time, not a realistic inference latency. On the H100's INT8 tensor path, dividing ResNet-50's 8.2 GFLOP per inference by the 1,979 TOPS peak yields a floor of about 4 µs, against 8 µs at the FP16 peak. Real single-image ResNet-50 inference takes far longer than either floor. Part of that gap is kernel launch, synchronization, and host overhead that no roofline captures; part is memory traffic in low-intensity layers, because bandwidth has improved more slowly than tensor arithmetic throughput. That imbalance decides whether the advertised INT8 peak delivers high sustained utilization or sits largely idle on a given workload (Horowitz 2014; Gholami et al. 2024), and closing the gap requires the other half of the anatomy: the memory hierarchy that supplies the compute units.

11.5 Memory Systems

A neural network kernel starves whenever the memory subsystem cannot deliver operands and consume outputs fast enough to saturate arithmetic pipelines. Modern accelerators provide hundreds of teraflops (or even petaflops) on low-precision tensor datapaths (NVIDIA Corporation 2020a, 2024; Choquette 2023), yet their compute pipelines idle as soon as memory traffic saturates the bus. This

is the memory wall, the growing divergence between arithmetic capability and memory bandwidth, seen from inside the accelerator.

Because deep neural networks manipulate millions or billions of parameters across high-dimensional activation tensors, performance is frequently dictated by data movement rather than raw arithmetic capacity. Accelerators combat this memory wall through tiered storage hierarchies, software-managed scratchpads, high-bandwidth 3D stacked memory, and architectural dataflow strategies designed to maximize spatial and temporal reuse. For the chapter's ResNet-50 workload the question becomes where each activation tile lives between the moment one convolution writes it and the moment the next consumes it, and what each tier on that path costs in time and energy.

↩ The memory wall, the widening gap between compute growth and memory bandwidth growth, is established in section 2.2.

11.5.1 The AI memory wall

Upgrading to an accelerator with twice the peak FLOP/s yields zero speedup when a workload's weights and activations already saturate memory bandwidth, pinning execution time at $\frac{D_{\text{vol}}}{\text{BW}}$. On an accelerator the **AI memory wall** is chiefly this bandwidth ceiling, set by the parameter volumes and activation tensors of deep learning rather than by the single-access latency gap of the classic CPU formulation. Adding more memory capacity does not remove it either, because a model that fits comfortably in device memory still waits on the rate at which its bytes reach the compute units of section 11.4. The energy side of the wall was set out with the energy ladder in section 11.2.3: an off-chip DRAM access costs about 173× the energy of the FP32 multiply that consumes its operand (figure 11.4), and the memory wall adds the time side of the same imbalance, which grows with every hardware generation.

11.5.1.1 Quantifying the compute-memory performance gap

Over the past two decades, peak computational capabilities have grown substantially faster than DRAM bandwidth (Gholami et al. 2024). This divergence stems from physical scaling constraints: while transistor scaling and dense systolic arrays multiply arithmetic throughput, memory bandwidth remains bounded by pin counts, package-substrate routing density, and interface signaling power. Representative high-end accelerators can deliver on the order of 10^3 TFLOP/s of peak tensor throughput (for example, NVIDIA H100 delivering 989 TFLOP/s in FP16 or 1,979 TFLOP/s in dense FP8) while providing approximately 3.35 TB/s of memory bandwidth (Choquette 2023). This implies that on the order of 10^2 FLOP of work per byte moved is required to fully use the compute, a threshold section 11.6.2 calls the ridge point and that exceeds the arithmetic intensity of many practical neural network workloads.

When a workload's arithmetic intensity falls below this threshold, data transfer ($\frac{D_{\text{vol}}}{\text{BW}}$) sets the larger of the two modeled time bounds. Accelerators attempt to tolerate memory latency (typically hundreds of clock cycles for off-chip DRAM) by maintaining thousands of concurrent execution threads and switching between active warps while memory requests remain in flight. However, when memory bandwidth saturates or when register pressure limits occupancy, the scheduler runs out of independent instructions to issue. Compute pipelines then stall, leaving high-throughput tensor cores idle while waiting for operands. Simultaneously, driving wide off-chip memory buses across capacitive printed circuit board traces and PHY interfaces consumes a substantial fraction of the chip's thermal design power (TDP), directly constraining the energy budget available for arithmetic logic (Horowitz 2014; Sze et al. 2017).

The time gap and the energy ladder describe the same pressure from two sides: each byte fetched from DRAM costs more time relative to the arithmetic it feeds with every generation, on top of its energy cost. The TPUv1's performance-per-watt advantage (example 11.1) came largely from minimizing that off-chip movement through systolic arrays and large on-chip SRAM (Jouppi et al. 2017). Whether a model can exploit such a design depends on the pattern of its memory accesses, which differs sharply among parameters, activations, and optimizer state.

Hardware balance is the supply side of the memory wall; model size is the demand side. The structural widening between model footprints and available memory bandwidth is what turns the iron law's memory term into the binding constraint across generations. Parameter count is a capacity quantity. In low-batch dense decoding with weights streamed from device memory, weight traffic per generated token grows with the parameter count. Sparse expert models read selected experts rather than every stored parameter, so their total parameter counts do not directly specify per-token traffic.

Figure 11.7 plots this logarithmic divergence relative to 2014, contrasting the roughly 3,900× increase in dense model parameters (filled red points) against the 33× increase in per-device accelerator memory bandwidth (blue points) to isolate the widening shaded memory-wall gap (Krizhevsky et al. 2012; Simonyan and Zisserman 2015; Brown et al. 2020; Chowdhery et al. 2022; Dubey et al. 2024). DeepSeek-V3 appears as an open marker at its total parameter count and is left out of the trend for exactly that reason (DeepSeek-AI 2025). Even high-bandwidth accelerators such as NVIDIA B200 and AMD MI300X/MI325X cannot close this growth-rate gap through bandwidth scaling alone (NVIDIA Corporation 2024; AMD 2023). Parameter counts for proprietary systems such as GPT-4 and Gemini are not officially disclosed, so they are not plotted as factual data points.

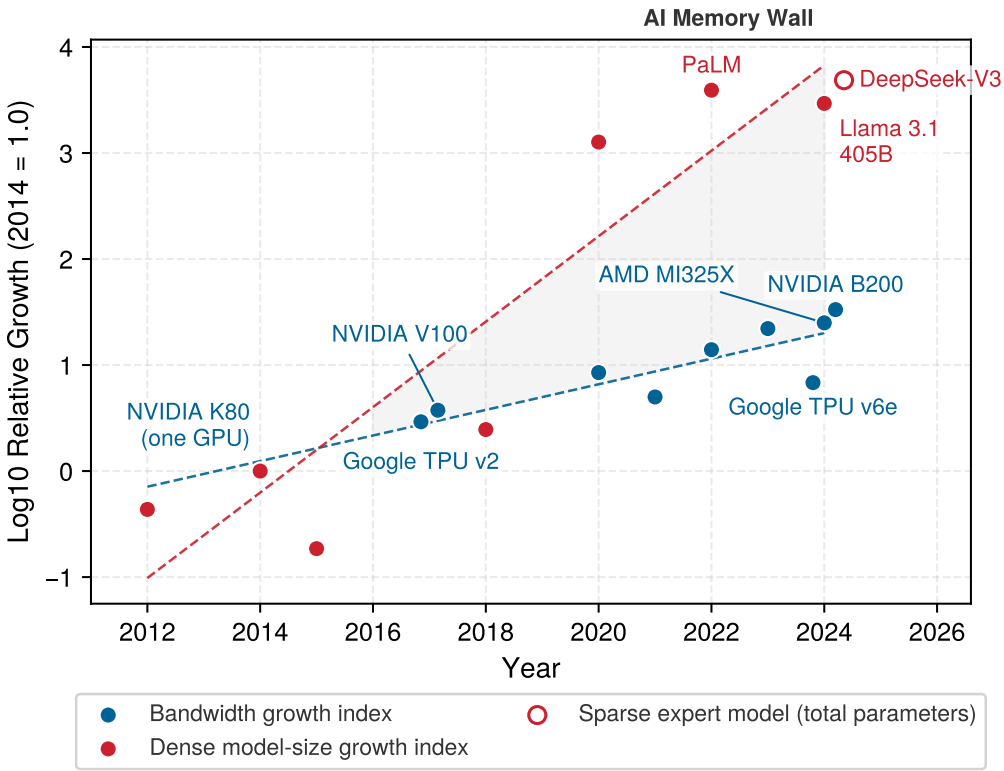


Figure 11.7: Model Size vs. Hardware Bandwidth: Both series are the base-10 logarithm of growth relative to 2014 (VGG-16 for parameters, one GPU of the dual-GPU K80 board for bandwidth), so each unit on the vertical axis is a tenfold increase. Over the plotted decade dense parameter counts grow roughly 3,900× against per-device bandwidth's 33×. The open marker is a sparse expert model plotted at total parameters and excluded from the fit. The shaded band is a gap between logarithms rather than a difference, so its width reads as a ratio, and the fitted trends describe the decade's average rates; the dense frontier has been flat since 2022. Method after Gholami et al. (2024).

11.5.2 Memory access patterns in ML workloads

In dense FP16 autoregressive generation at batch size 1, streaming a model's weights from device memory for each generated token yields roughly 1 FLOP/byte on the matrix-vector work, starving compute units before activations or gradients ever enter the picture. Across training and inference, parameters, activations, and optimizer states compete for that same memory bandwidth in distinct ways. Model parameters (weights and biases) can span hundreds of billions of values (Brown et al. 2020) that must stream from memory whenever batch reuse is low. Intermediate activations produced at each layer must also be buffered for subsequent operations, and during training those forward activations must either stay in memory until backpropagation consumes them or be recomputed on the backward pass. Training then compounds the traffic with gradients and optimizer state: backpropagation writes a gradient for every parameter, and adaptive optimizers such as Adaptive

Moment Estimation (Adam) read and write multi-word momentum states on every step, multiplying the bytes moved per parameter update (Narayanan et al. 2021; Kwon and Rhu 2018).

The physical cost of this data movement depends on where each tensor resides in the memory hierarchy. While an on-chip register access completes in a single clock cycle at sub-picojoule energy, a dependent off-chip DRAM fetch can expose hundreds of cycles of latency and expends orders of magnitude more energy. Independent requests can overlap that latency, subject to available concurrency and bandwidth. Tracing a single tensor of the chapter's ResNet-50 workload through the memory tiers during an inference pass makes this disparity concrete (lighthouse 11.2).



Lighthouse 11.2: Life of a tensor: A ResNet-50 activation map

Follow one activation map of the chapter's ResNet-50 workload, the output of a stage-2 convolution (56×56 positions by 64 channels), on its way to the next convolution's Tensor Cores on an H100:

1. **HBM**: The previous kernel wrote the map here.
   - **Size**: 201 KB per image at INT8 and 401 KB at FP16; a batch of 32 holds 6.4 MB at INT8.
   - **Latency**: A dependent off-chip request takes about 300 ns, while streaming one image's map at full bandwidth would take only about 60 ns, so the kernel must keep many requests in flight to approach the bandwidth ceiling.
   - **Energy**: About 160 pJ per byte in the 45 nm estimates used throughout the chapter, or about 32.1 µJ to read one image's map once.
2. **L2 cache**: The H100 shares 50 MB of L2 across its streaming multiprocessors (SMs), enough to hold the whole batch's map at INT8 if the next kernel consumes it soon after the previous one wrote it.
   - **Latency**: About 150 ns on Hopper.
3. **Shared memory**: One SM stages a tile of the map in its 228 KB of software-managed shared memory. A whole image's map fits at INT8 but not at FP16, so the INT8 format changes which tier can hold it.
   - **Latency**: About 17 ns on Hopper.
4. **Registers**: The Tensor Core reads its operands here.
   - **Latency**: A small number of cycles, depending on the instruction.
   - **Energy**: About 0.1 pJ per access, more than three orders of magnitude below an off-chip access.
   - **Device arithmetic peak**: 1,979 TOPS on the H100 INT8 tensor path, the rate these registers must sustain, not a register-transfer rate.

Systems insight: Latency and energy per access rise from registers to HBM. The working set's location, access count, reuse, and dependencies determine the memory contribution to time and energy; arithmetic adds its own cost. The bandwidth term of the roofline is whichever link of this ladder the working set must cross; HBM-to-L2 is one possible limiting link, not the only one.

The boundary between bandwidth-bound and compute-bound execution emerges directly from the balance between data transfer time and arithmetic execution time. For a workload requiring data volume D_{vol} (bytes) across available memory bandwidth BW (bytes/s), and executing O floating-

point operations on hardware with peak throughput R_{peak} (FLOP/s) at realized hardware utilization η_{hw}, the respective execution times are the iron law's data and compute terms:

$$T_{\text{memory}} = \frac{D_{\text{vol}}}{\text{BW}}$$
$$T_{\text{compute}} = \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$$

In a strictly sequential model without overlap, $T_{\text{memory}} > T_{\text{compute}}$ identifies bandwidth-bound execution. When hardware overlaps memory transfers with computation, the larger term sets the theoretical lower bound on execution time. In practice, however, data dependencies, memory controller queueing, and partial pipeline overlap dictate how much arithmetic hardware remains idle while waiting for operands.

11.5.2.1 Irregular memory access

Production machine learning workloads combine regular dense matrix kernels with irregular memory access patterns that defeat conventional caching and prefetching hardware. While dense operations like general matrix multiply (GEMM) and convolution stream contiguous blocks that map cleanly onto hardware caches and systolic arrays, irregular operations rely on indirect, sparse, or input-dependent memory addresses. Table 11.5 contrasts these two memory regimes across access geometry, cache locality, and hardware pressure.

Table 11.5: Regular and Irregular Memory Access: Regular dense kernels expose tileable reuse and predictable addresses, whereas irregular ML components rely more on indirect or input-dependent access, weakening caching and prefetching and adding metadata or load-balance overhead. A single model may contain both regimes.

Feature	**Regular dense kernels**	**Irregular ML components**
Access pattern	Contiguous or tiled	Indirect, scattered, or input-dependent
Cache locality	Often high after tiling	Often lower and workload-dependent
Data reuse	Structured across tiles or batches	Sparse or dynamic, depending on the operator
Dependencies	Static loop nests	Input-dependent routing or variable shapes
Examples	GEMM, convolution, dense attention	Embedding lookup, unstructured sparsity, MoE routing
Primary pressure	Compute or bandwidth, depending on arithmetic intensity	Latency, bandwidth, metadata, and load imbalance
Energy consequence	Depends on reuse and memory tier	Extra movement and metadata can increase energy

Within a single neural network, memory access behavior varies sharply across layer types. Convolutional layers have high spatial locality and register reuse as small weight kernels slide across input feature maps, whereas fully connected layers and dense attention execute regular matrix multiplications whose large parameter matrices or key-value caches can exceed on-chip SRAM capacity and become bandwidth bound when operational reuse is low. Variable sequence lengths complicate memory buffer allocation, yet the attention operations within each sequence remain regular, contiguous matrix multiplies.

True memory irregularity arises instead from unstructured sparsity,[19] embedding lookups, and dynamic token routing. Unstructured sparsity stores nonzero weights or activations in compressed formats (such as CSR or COO) that require pointer indirection to decode operand coordinates. Embedding tables break sequential transfers in a different way: sparse categorical IDs select arbitrary rows from tables spanning tens of gigabytes in DRAM, turning reads into uncoalesced, scattered fetches. DLRM, the book's capacity-bound lighthouse, is the extreme case, because its embedding tables exceed one accelerator's memory and each lookup fetches a short row from an address known only when the input arrives, defeating both caches and prefetchers. Mixture-of-Experts (MoE) models introduce a third irregularity at the routing layer, where gates direct tokens to different expert networks based on input context; although deterministic for a given input, token assignments vary across batches, disrupting prefetch pipelines and unbalancing execution queues across parallel cores.

[19] **Sparsity and memory irregularity**: Unstructured sparse formats commonly gather nonzero elements through indirect addressing, which can weaken sequential access and make metadata overhead significant. Structured sparsity preserves hardware-friendly blocks, so its access behavior differs from arbitrary pruning. Without suitable kernels or hardware support, an unstructured sparse model can run slower than its dense counterpart despite performing fewer arithmetic operations.

Irregularity degrades performance through distinct mechanisms, and treating them as one memory problem can point to the wrong optimization. Scattered addresses may turn one logical access into several memory transactions, so each fetched cache line carries few useful payload bytes. Indirection can also serialize address generation: the system must fetch an index or pointer before it knows which payload to request, limiting memory-level parallelism even when aggregate bandwidth remains available. Input-dependent routing creates a third failure mode when some processing elements receive longer queues than others; completion time follows the slowest queue while neighboring units idle. Variable-sized allocations may fragment capacity, but fragmentation is neither necessary nor sufficient for irregular access. A sequential stream can saturate bandwidth without being irregular, whereas a dependent lookup chain can stall far below peak bandwidth.

Diagnosis should therefore begin with the mechanism that prevents forward progress. A low ratio of useful payload bytes to transferred bytes indicates poor spatial utilization or coalescing. Long memory stalls alongside unused bandwidth indicate dependent misses or too few concurrent requests to hide latency. Uneven queue depths or per-route work reveal load imbalance rather than a cache-capacity problem. Cache hit rate alone cannot distinguish these cases because a miss can waste bandwidth, expose latency, or both. Each diagnosis implies a different remedy: packing or blocking improves spatial utilization, reordering and prefetching expose concurrent requests, structured sparse formats reduce indexing overhead, and routing or partitioning balances work. The memory hierarchy determines the cost of the remaining accesses, while dataflow and tensor-layout choices determine how much irregularity software can remove before execution reaches the hardware.

The optimization order follows that diagnosis. Representation comes first when the format is under system control because a block-structured sparse format or grouped routing can remove indirection before a kernel executes. Request organization comes next: accesses issued together should target adjacent addresses, and enough independent requests must remain in flight to hide unavoidable latency. Additional cache capacity or bandwidth helps only when profiling shows that the regularized request stream still exhausts that resource. More bandwidth cannot resolve a serial pointer dependency, and more capacity cannot rebalance skewed work. The objective is to expose as much regular structure as possible while confining unavoidable irregularity to the smallest portion of execution.

Software can mitigate irregular access by regularizing tensor layouts, enforcing structured sparsity, and sorting dynamic routes into uniform batches. Yet software restructuring cannot eliminate the physical penalty of data movement: whether a memory request is contiguous or scattered, every transferred byte must cross physical wires governed by finite bandwidth, propagation latency, and energy dissipation. Bounding these physical costs requires organizing storage into cascading hardware tiers. Rather than treating memory as a uniform pool, accelerator architectures deploy multilevel memory hierarchies that trade off capacity against latency and energy at every tier.

11.5.3 Memory hierarchy

No single storage technology can simultaneously satisfy the multi-terabyte-per-second bandwidth required by dense execution units and the tens of gigabytes of capacity required to hold modern model parameters. Accelerators resolve this tension through multilevel memory hierarchies. Rather than treating memory as a monolithic address space, the machine organizes storage into distinct physical tiers, each optimized for a specific balance of capacity, access latency, bandwidth, and energy dissipation.

At the apex of the hierarchy, register files and on-chip SRAM caches supply operands to arithmetic units with sub-nanosecond latency and single-digit picojoule energy costs, but their capacity is strictly limited by silicon die area. At the base, nonvolatile flash storage holds hundreds of gigabytes of model checkpoints and training datasets, but requires milliseconds or microseconds to access. Between these boundaries, software-managed scratchpads stage active tensor tiles, while device-local DRAM (such as HBM or GDDR) provides the primary working memory for parameters and activations.

This organization establishes an inverse relationship across tiers: as storage capacity expands and physical distance from arithmetic units grows, bandwidth falls, latency lengthens by orders of magnitude, and energy consumption per bit escalates. Table 11.6 compares these tiers across latency, bandwidth, capacity, and functional roles in deep learning workloads.

Table 11.6: Memory Hierarchy Trade-Offs: AI accelerators use a multilevel memory hierarchy to balance performance and capacity. Each level provides distinct latency, bandwidth, and capacity characteristics that dictate how neural network components (weights, activations, and intermediate results) should be allocated to minimize bottlenecks and maximize throughput. Relative labels describe the typical ordering; exact values depend on architecture. Section D.1.1 supplies registry-backed reference latencies.

Memory Level	**Relative latency**	Bandwidth	**Relative capacity**	**Example Use in Deep Learning**
Registers	Lowest	Highest	Tiny	Storing operands for immediate computation
L1/L2 Cache (SRAM)	Very low	High	Small	Caching frequently accessed activations and small weight blocks
Scratchpad memory	Very low	High	Small to medium	Software-managed storage for intermediate computations
Device DRAM (HBM, GDDR, or LPDDR)	High	Moderate–Very High	Large	Storing model parameters and activations that do not fit on-chip
Host DRAM (typically DDR)	High locally; transfer adds latency	Moderate	Large to very large	Staging or offloading data outside accelerator memory
Flash Storage (SSD/NVMe)	Very high	Low	Very large	Storing pretrained models and checkpoints for later loading

A recurring systems question is why accelerators cannot bypass on-chip SRAM entirely and stream all operands directly from high-capacity off-chip DRAM. A dependent DRAM request exposes the full access latency. Many independent requests can hide that latency, but their sustained operand supply remains limited by off-chip bandwidth; on-chip staging also enables reuse.

Napkin Math 11.2: Why HBM cannot feed the array directly

Problem: Why is on-chip SRAM necessary instead of fetching all data from HBM?

Math:

1. **Clock cycle**: At 2 GHz, $1/(2\ \text{GHz}) = 500$ ps.
2. **DRAM access**: Off-chip HBM sits millimeters away on the package, but row activation, DRAM-array latency, serialization, protocol traversal, and queuing add up to 100+ cycles per access.
3. **Distance, for comparison**: On an H100-class 814 mm² die, signals travel ~20 mm; at $\approx 0.5c$, $20\ \text{mm}/(0.5c) \approx 130$ ps, less than one cycle.

Systems insight: The propagation estimate alone does not explain the DRAM access cost, and it omits real interconnect delay from wiring and routing. Independent requests can overlap the 100-plus-cycle access path; a dependent request cannot. Registers and SRAM supply operands at the rate the array needs while reusing values that would otherwise cross HBM repeatedly. Transformer tiling addresses both operand supply and reuse, whereas streaming a growing KV cache can still limit inference when its required traffic exceeds the available bandwidth.

11.5.3.1 On-chip memory

The **on-chip memory** subsystem integrates registers, Static Random-Access Memory (SRAM) caches, and software-managed scratchpads directly on the accelerator die. Register files provide the immediate operand and accumulator storage needed by execution pipelines. Modern accelerators provision substantial register capacity in aggregate across the chip, but partition this pool dynamically among concurrent threads or warps. Increasing per-thread register allocation allows individual threads to retain larger matrix tiles, but limits the total number of concurrently scheduled warps, reducing the hardware's ability to hide memory latency. Conversely, underallocating registers forces the compiler to spill values to off-chip memory, converting single-cycle register accesses into multi-cycle stalls. Efficient kernel design balances register allocation to maintain maximum occupancy while keeping hot operands resident.

Hardware-managed L1 and L2 caches provide fast buffering for frequently referenced data using automatic tag checking and hardware eviction policies. However, automatic caches are poorly suited for streaming multi-gigabyte tensor operations: scanning through large activation tensors can evict reusable weight tiles before arithmetic units can reuse them. Furthermore, specialized execution pipelines impose strict layout and data-type constraints that general caches cannot satisfy automatically. As illustrated in example 11.3, hardware matrix engines require specific alignments and memory layouts; a mismatch may require padding, conversion, or a different kernel whose efficiency must be checked.

Example 11.3: Tensor Cores that never engage

Scenario: An engineering team migrates a transformer workload to NVIDIA A100 GPUs expecting automatic Tensor Core acceleration, but profiling reveals Tensor Cores remain idle (NVIDIA Corporation 2020a).

Diagnosis: The workload used shapes and data types that selected less efficient kernels rather than the intended Tensor Core path.

Systems lesson: Specialized paths have operator, precision, layout, and shape requirements. Libraries can often handle nonaligned dimensions through padding or alternative kernels, but reaching high utilization requires verifying which path was actually selected.

To avoid the eviction overhead and unpredictability of automatic caches, AI accelerators incorporate scratchpad memory. Unlike hardware caches, scratchpads are software-managed SRAM structures mapped directly into the core's address space without tag directories or hardware replacement logic. The compiler or programmer explicitly controls when tensor tiles are loaded from external DRAM, how long they reside on-chip, and when updated values are written back. In convolutional models, scratchpads retain input feature maps and weight kernels across sliding receptive windows, eliminating redundant DRAM reads (Chen et al. 2017). On NVIDIA GPUs, scratchpad memory is exposed as *shared memory*: a fast, programmer-controlled SRAM bank shared across all threads within a thread block. Custom kernels written in CUDA or Triton manage shared memory explicitly. FlashAttention demonstrates the power of this mechanism: rather than materializing the full $S \times S$ attention score matrix in off-chip HBM, the kernel tiles queries, keys, and values through shared memory and updates softmax statistics online (Dao et al. 2022). The speedup stems entirely from eliminating high-latency HBM round trips rather than reducing arithmetic operations; in fact, the algorithm recomputes intermediate attention values during the backward pass to avoid storing them to off-chip memory.

↩ FlashAttention as an IO-aware attention kernel for training is established in section 8.6.4.

11.5.3.2 Off-chip memory

When tensor working sets exceed on-chip SRAM capacity, operands and parameters reside in device-local DRAM. Rather than sequential hierarchy levels, HBM, Graphics DDR (GDDR), and Low-Power DDR (LPDDR) represent alternative memory architectures engineered for distinct cost, bandwidth, and power envelopes. High-performance accelerators use HBM, which stacks DRAM dies vertically using through-silicon vias (TSVs) and interfaces with the accelerator die across a wide silicon interposer (e.g., 1,024 to 8,192 bits). This wide bus delivers multi-terabyte-per-second bandwidth at lower clock frequencies and lower energy per bit than conventional DRAM, but requires complex 2.5D packaging. Where silicon interposers are cost-prohibitive, accelerators adopt GDDR, which uses narrower buses (e.g., 256 to 384 bits) driven at high pin speeds across standard circuit boards. Mobile and embedded accelerators use LPDDR, trading raw bandwidth for low operating voltage and reduced standby power.

When parameter and activation footprints exceed device-local DRAM capacity, the system must offload state to host DRAM (typically standard DDR). Accessing host memory requires crossing an I/O interconnect such as PCIe or a coherent fabric like NVLink or Compute Express Link (CXL). Because PCIe bandwidth (tens of gigabytes per second) is one to two orders of magnitude lower than accelerator DRAM bandwidth, unmanaged offloading stalls arithmetic execution. Efficient runtimes mitigate this penalty by overlapping asynchronous direct memory access (DMA) copies with kernel execution, prefetching upcoming layer parameters while current layers compute.

At the base of the hierarchy, nonvolatile flash storage and solid-state drives (SSDs) provide persistent, multi-terabyte capacity for datasets and training checkpoints (Narayanan et al. 2021). With read latencies measured in tens of microseconds and bus bandwidths capped at single-digit gigabytes per second, flash devices cannot participate directly in kernel execution. Checkpointing and data-loading pipelines treat flash storage as an asynchronous streaming source, staging batches through host DRAM into accelerator memory before computation begins.

The physical tiers of the memory hierarchy establish the latency floors and capacity ceilings of the accelerator, but advertised memory bandwidth is only a ceiling within them; achievable bandwidth depends on access pattern, batching, locality, and the host interface that feeds the accelerator. Peak figures assume continuous, stride-1 coalesced access across all memory channels. Real neural network workloads deviate substantially from this ideal: transformer attention, convolution, and fully connected layers realize distinct fractions of peak bandwidth because their memory access strides, tiling dimensions, and operational reuse differ. A sufficiently large, coalesced weight stream can approach peak bandwidth even at small batch size. Batching instead reuses each loaded weight across more samples, raising arithmetic intensity as analyzed under the roofline model in section 11.6. Small or irregular kernels may lack enough concurrent accesses to approach the bandwidth ceiling, making memory latency and available parallelism part of the diagnosis.

11.5.4 Host-accelerator communication

An accelerator rated for hundreds of teraflops sits idle if its input tensors are still crawling across a host PCIe bus an order of magnitude slower than its on-package HBM. In discrete accelerator systems, the host CPU and the accelerator operate as separate computational domains joined by a peripheral bus: the host manages storage I/O, network streams, and runtime orchestration in system DRAM, while the accelerator executes dense mathematical kernels over device memory (HBM or GDDR). Because those two memories occupy separate physical address spaces, computation cannot begin until tensors cross the host-accelerator interconnect, making peripheral bus bandwidth and launch latency a hard gate on end-to-end speedup.

Figure 11.8 sequences the four-stage execution trace across host and device lifelines: copying input data across PCIe to device memory via DMA (step 1), dispatching kernel instructions from the CPU (step 2), executing parallel core compute with local writeback (step 3), and copying results back to host memory (step 4). In a naive synchronous execution model, every transfer step sits directly on the critical path, stalling either the host or the accelerator until each memory transaction finishes.

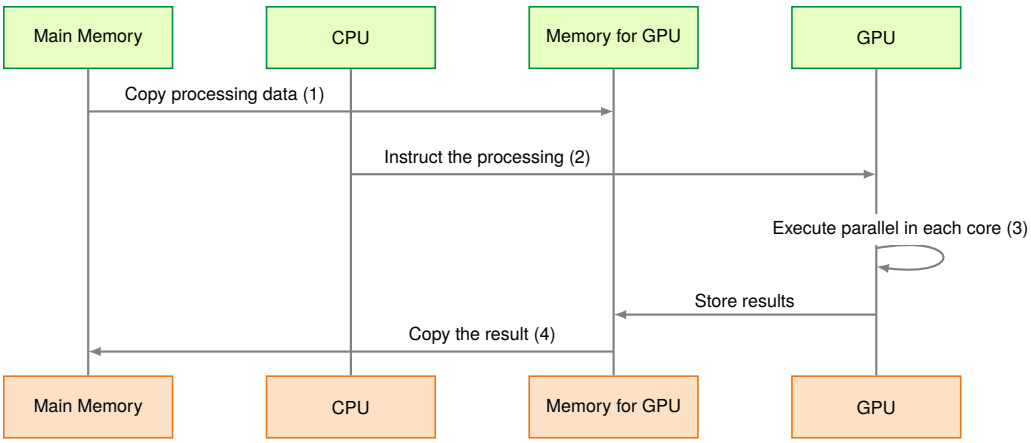


Figure 11.8: Host-Accelerator Data Transfer Sequence: Four-step execution flow between CPU host and GPU device: (1) copying input data over PCIe, (2) issuing kernel instructions, (3) GPU compute, and (4) transferring results back to host memory. Low PCIe bandwidth and host dispatch latencies can easily dominate runtime unless kernel compute time exceeds data transfer overhead.

The end-to-end latency of a kernel launch decomposes into transfer time, queue dispatch overhead, and arithmetic execution time. When tensor transfers are small or frequent, the fixed overhead of driver submission and bus round trips dominates runtime. When transfers are large, the low

bandwidth of PCIe relative to on-package HBM creates a sustained I/O bottleneck. In D·A·M terms, the Machine boundary between host and accelerator forces the Algorithm to maximize arithmetic reuse on the device once tensors arrive.

11.5.4.1 Node-level interconnect topology

Within a single machine, how bits move between GPUs and the CPU sets the next ceiling. In the 1–8 GPU regime, scaling is achieved through high-speed intra-node interconnects such as NVLink and host-to-device PCIe transfers that mitigate the memory wall. These links form a **bandwidth taper**: data-movement speed falls at each step away from the compute units, from on-package HBM through the GPU-to-GPU NVLink bridge down to the host PCIe link. The PCIe step is much slower than the accelerator-local memory and inter-GPU fabric, so any data path that touches the CPU can become a performance hazard, and keeping accelerator-to-accelerator traffic off that link is the reason NVLink exists (NVIDIA Corporation 2020c). A typical AI server is therefore not a flat mesh of connected devices, and optimizing data movement starts from its physical topology.

Within a multi-accelerator server node, two interconnect tiers define this bandwidth taper below device memory:

1. **Device-device interconnect (NVLink/Infinity Fabric)**: Modern multi-GPU nodes use specialized high-speed bridges like NVLink[20] to connect accelerators directly, bypassing the host CPU. Bandwidth ranges from 600 GB/s to 900 GB/s per GPU (NVIDIA Corporation 2020c; Choquette 2023). This link matters whenever tensors must move between accelerators within one server, including model partitioning, activation exchange, and gradient synchronization during training. The architectural principle is the boundary: traffic that stays on the accelerator fabric is far cheaper than traffic that falls back through the host.
2. **Host-device interconnect (PCIe)**: The link between the CPU and the accelerator. Bandwidth ranges from 32 to 64 GB/s (PCIe Gen4/Gen5). Host-staged training data can bottleneck here, although resident data, direct storage or network I/O, and coherent accelerator links can use other paths. Multi-GPU servers may also provide multiple PCIe links rather than a single shared 64 GB/s path.

A third tier, the network link from one server to others, continues the taper beyond the node (section 11.9). Together the levels produce a characteristic bandwidth taper:

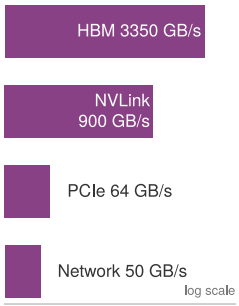
$$\begin{aligned}\text{HBM (3350 GB/s)} &\gg \text{NVLink (900 GB/s)} \\ &\gg \text{PCIe (64 GB/s)} > \text{Network (50 GB/s)}\end{aligned}$$

System efficiency depends on keeping tensors on the highest-bandwidth links available. Because PCIe and network links offer orders of magnitude less bandwidth and higher latency than on-package memory, placement and scheduling must minimize crossings across these boundaries.

11.5.4.2 Transfer optimization

The standard remedy for transfer overhead is the asynchronous pipeline of section 7.5.2. Batches staged in pinned (page-locked) host memory are copied by dedicated DMA engines while the accelerator computes on the previous batch. This double buffering conceals transfer latency behind arithmetic runtime whenever computation time exceeds transfer time ($T_{\text{compute}} \geq T_{\text{transfer}}$). When compute duration is too short to amortize the transfer, execution remains bus bound and the compute units stall.

Unified Memory takes the opposite approach, presenting a single virtual address space that spans host DRAM and device HBM while the driver migrates pages on demand whenever either processor touches an address. On-demand migration removes staging code at the cost of predictability, because each page fault stalls execution and scattered accesses can thrash pages across the PCIe bus. High-throughput production workloads therefore rely on explicit, pipelined DMA transfers and reserve Unified Memory for prototyping or for applications where programmer productivity outweighs sustained hardware utilization.



Bandwidth tapers steeply as data moves farther from the accelerator.

20 **NVLink (NVIDIA Link)**: This direct GPU-to-GPU interconnect exists to keep accelerator-to-accelerator traffic off PCIe when tensors must move inside a server. Its 600–900 GB/s aggregate bidirectional bandwidth is roughly 300–450 GB/s per direction under balanced full-duplex traffic, several times the directional bandwidth of a standard PCIe link, so workloads with frequent cross-device tensor movement can remain in the fast part of the bandwidth taper (NVIDIA Corporation 2020c; Choquette 2023). Training algorithms determine how much tensor traffic must cross this link; the hardware fact needed here is the bandwidth gap.

↩ The feeding problem, in which data delivery fails to keep pace with compute throughput, is established in section 4.9.3.

11.5.5 Model memory pressure

An optimization that relieves memory pressure for one model family can leave another untouched: caching weights helps nothing when ResNet-50's early convolutions spill large activation maps to DRAM, and spatial activation tiling helps nothing when GPT-2 decode streams gigabytes of weights for a single token. Model architecture determines which memory term binds. The architectural reasons each family stresses memory as it does were traced in section 6.9.1; the hardware consequence differs by family, and the lighthouse models of table 1.4 cover the cases.

Dense layers, the multilayer perceptron (MLP) blocks that every family contains, are bound by weight traffic at small batch. At batch size $B = 1$, each parameter loaded from off-chip memory serves a single multiply-accumulate operation (often ≤ 1 FLOP/byte), so execution sits on the bandwidth slope; batching reuses each loaded weight across B inputs and moves the layer toward the compute ceiling. Dense weight matrices regularly exceed the tens of megabytes of on-chip SRAM, so parameters reside in HBM and stream through tiles that the accelerator double-buffers into SRAM (Chen et al. 2017). The host link enters the critical path only when inputs stream from the host or when the model exceeds device memory.

Convolutional layers, the core of ResNet-50, place their memory pressure on activations rather than weights. Small filters sliding across feature maps supply three forms of reuse: neighboring windows share input activations, each filter is applied at every spatial position, and each input channel feeds every output filter. Early layers therefore carry compact parameter tensors but large activation maps that exceed on-chip SRAM, and unoptimized layer-by-layer execution spills those maps to device DRAM. Spatial tiling keeps activation tiles, with their overlapping boundary pixels, in on-chip buffers so adjacent convolutions consume them without refetching (Chen et al. 2017).

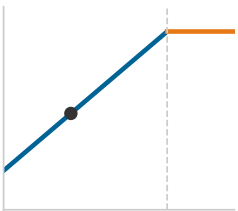
Transformers such as GPT-2 stress both capacity and bandwidth, differently in each phase. During training and prefill, standard attention materializes an $S \times S$ score matrix for sequence length S and moves $\mathcal{O}(S^2)$ bytes through HBM; IO-aware kernels such as FlashAttention (Dao et al. 2022) keep those tiles in on-chip SRAM instead (section 8.6.4). During autoregressive decode, each generated token reads the weights and the accumulated key-value (KV) cache from HBM for a vector-matrix product, which binds generation speed to memory bandwidth.

↩ How much memory one request's KV cache claims as its sequence grows is established in section 6.7.4.2.

Two other lighthouses stress memory in other ways: DLRM's sparse embedding lookups stress random-access bandwidth rather than dense compute, and MobileNetV2's depthwise convolutions trade arithmetic intensity for a smaller parameter footprint, so both reappear as memory-bound rows in the roofline operations table (table 11.7). Accelerators answer these divergent demands with the tiered hierarchy. On-chip SRAM captures high-reuse activations and tiled matrix blocks at tens of terabytes per second, HBM stages large weight tensors and sequence state, and NVLink-class links carry tensors between accelerators when a model exceeds one device, avoiding the narrower host PCIe bus. Table 11.14 later pairs each family's memory profile with the mapping that answers it. Because a reference off-chip DRAM access costs far more energy per event than arithmetic (figure 11.4), reducing repeated external accesses improves efficiency, but deciding whether a given kernel is limited by compute throughput or by memory bandwidth requires an analytical boundary. The roofline model supplies that boundary by comparing operational intensity against hardware performance limits, so that optimization effort targets the resource that actually binds.

11.6 Performance Bounds

The compute units fix how fast an accelerator can calculate, and the memory system fixes how fast it can be fed. A kernel's performance is bounded by whichever of the two runs out first. Putting numbers on that statement takes three pieces: the roofline's ceilings for each precision path, the ridge point at which they meet, and the hardware balance that moves the ridge from one generation to the next. The ResNet-50 INT8 workload meets its sharpest lesson in these bounds. The INT8 path doubles the H100's arithmetic ceiling without moving its bandwidth ceiling, so a layer that cleared the ridge at FP16 can fall below it at INT8.



Low arithmetic intensity places the modeled throughput ceiling on the bandwidth slope.

11.6.1 The roofline model

Quoting an accelerator's peak FLOP/s cannot explain why one kernel runs near the hardware ceiling while another stalls below 1 percent of peak on the same chip, because peak compute measures only what execution units can retire when operands are already in registers. Sustained throughput is

bounded by whichever limit—arithmetic capacity or memory bandwidth—runs out first. With the compute units and memory systems that set those two ceilings now in place, the **roofline model**[21] (Williams et al. 2009), which sorted training kernels in section 8.3.4, makes this diagnosis quantitative. It plots attainable performance against arithmetic intensity and identifies the lower modeled ceiling, compute or memory bandwidth, for the chosen execution path before profiling establishes which physical resource limits execution.

Performance is bounded by two ceilings, as equation 11.2 formalizes; it writes the bound of the arithmetic intensity law (principle 6) as an idealized equality that ignores launch and synchronization overheads. Here, attainable performance R_{attain} and peak compute R_{peak} are in FLOP/s (often reported as TFLOP/s), peak bandwidth BW is in bytes/s (often TB/s), and arithmetic intensity I is in FLOP/byte:

$$R_{\text{attain}} = \min(R_{\text{peak}}, \text{BW} \times I) \tag{11.2}$$

Arithmetic intensity $I = O/D_{\text{vol}}$ is the ratio of operations to bytes moved across the memory interface being modeled (definition 5.3), and it depends on the implementation as well as the operation. Under the stated traffic models, a large well-tiled 1024×1024 multiply reaches about 341.3 FLOP/byte, while standalone ReLU is about 0.125 FLOP/byte, so the two meet different ceilings on the same hardware.

The roofline visualization shows performance (TFLOP/s) on the vertical axis and arithmetic intensity (FLOP/byte) on the horizontal axis. At low arithmetic intensity, the upper bound increases linearly along the memory bandwidth slope ($\text{BW} \times I$). Above the ridge point, the upper bound is peak compute (R_{peak}). A kernel approaching the slope is bandwidth limited, while one approaching the flat roof is compute limited on the specified path. Merely lying below either line does not establish the cause; profiling explains that remaining gap. Section A.5.3 maps each regime to the optimizations that pay off and the ones that waste effort, so classifying a kernel tells the engineer whether more bandwidth, less traffic, or more arithmetic throughput is the investment to test first.

21 **Roofline model**: Introduced by Williams et al. (2009) at UC Berkeley, building on earlier I/O complexity work from the 1980s. Their specific contribution was making the compute vs. bandwidth trade-off visual and actionable: the characteristic roofline plot immediately reveals whether a kernel is compute bound (hitting the flat ceiling) or memory bound (hitting the sloped bandwidth line) and quantifies the gap to hardware limits. A kernel operating at only 50 percent of its ceiling has 2× theoretical headroom, but instruction mix, latency, and occupancy may prevent it from closing that gap.

11.6.2 Hardware ridge points

A kernel with 200 FLOP/byte that was comfortably compute bound on a V100 or A100 falls below the compute ceiling when migrated unchanged to an H100, because different devices place the compute-bandwidth boundary at different intensities. The **ridge point**, the balance point between a machine's memory and compute limits previewed alongside the iron law, is a property of the device. Computer architects call the same ratio the **hardware balance** (I_{ridge}), peak arithmetic throughput divided by peak memory bandwidth, the intensity at which the roofline's compute and bandwidth ceilings intersect:

$$I_{\text{ridge}} = \frac{R_{\text{peak}}}{\text{BW}}$$

The ratio sets a device-specific boundary. Depending on the chosen precision, the H100 requires about 295.2 FLOP/byte on its dense FP16 path (989 TFLOP/s over 3.35 TB/s) to reach its arithmetic ceiling, and twice that on its INT8 path, whereas a microcontroller may have a much lower ridge point. The same kernel can therefore be bandwidth bound on a compute-dense accelerator and compute bound on an edge processor. Hardware balance does not label either device as universally efficient; it identifies how much reuse a workload needs on that device.

Peak computational capability has expanded faster than memory bus performance across accelerator generations, and the size of that mismatch is what sets how much data reuse a kernel needs to stay fed. Across 2017–2024, NVIDIA's data center generations multiplied dense FP16 compute by 18× and memory bandwidth by only 8.9×, a 2.0× rise in the reuse a kernel needs on B200 relative to V100.

That ratio measures how far the hardware ridge point, the threshold a kernel must clear to become compute bound, has risen since the V100. Figure 11.9 plots it generation by generation, leaving out the H200 exception, so the doubling of the gap appears directly as the ridge point roughly doubling from V100 (139 FLOP/byte) to H100 (295 FLOP/byte) and then holding near that level on B200 (281 FLOP/byte). A kernel that cleared the V100 ridge comfortably can fall below the H100 ridge without any change to its code, so newer hardware demands more operational reuse before execution reaches peak throughput.

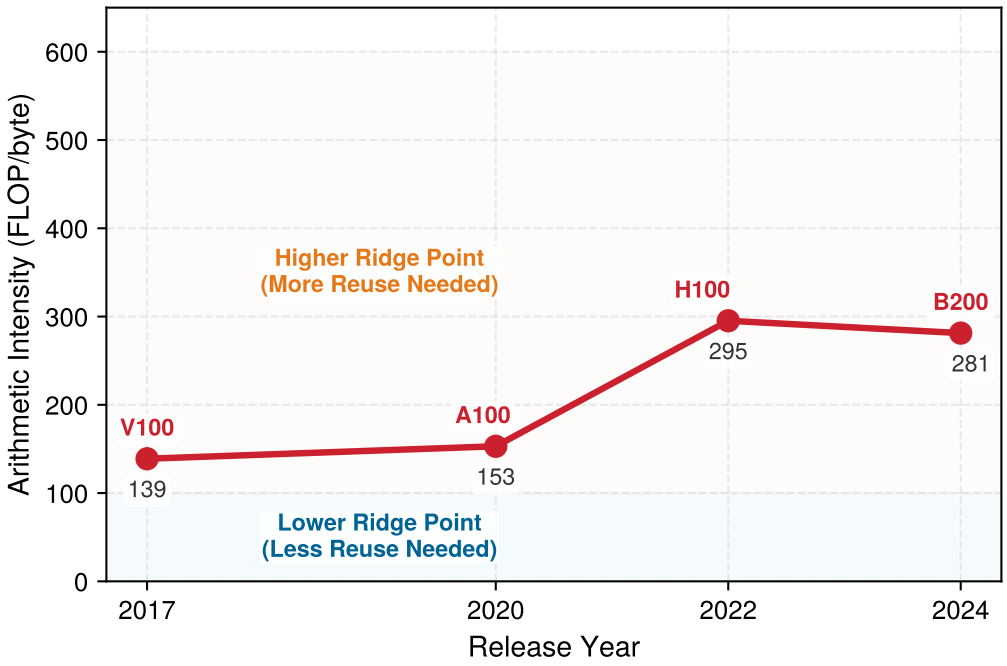


Figure 11.9: The Rising Ridge: Hardware arithmetic intensity (FLOP/byte) over time computed as each chip's dense FP16 tensor peak divided by its memory bandwidth: 125 TFLOP/s over 0.9 TB/s for V100, 312 TFLOP/s over 2.04 TB/s for A100, and 989 TFLOP/s over 3.35 TB/s for H100. This is the arithmetic intensity a kernel must reach before compute rather than bandwidth limits it. As compute capability grows faster than memory bandwidth, the ridge point rises from V100 through H100 and remains high on B200. This trend explains why architectures with high data reuse flourish while low-reuse workloads face a growing performance penalty.

The generations in figure 11.9 come from one vendor, but the divergence is industry-wide. Figure 11.10 tracks the best memory bandwidth and the best dense FP16/BF16 tensor throughput among 52 accelerators from every major vendor released since 2017 (Epoch AI 2026b). Measured from TPU v2, the best compute rate grew about 55× while the best bandwidth grew about 13×, so the ridge point of the leading devices rose several-fold; the ratio is larger than the NVIDIA-only one above because TPU v2's 16-bit baseline was low. Since early 2025 the best bandwidth has held near 8 TB/s, and the best dense FP16 rate has leveled as well, because new devices put their added arithmetic into FP8 and FP4 paths rather than 16-bit ones. For a kernel whose reuse stays fixed, this rising ridge shows up as a ceiling on achievable utilization that falls with each generation.



Napkin Math 11.3: The utilization gap

Problem: Why is it harder to get 100 percent utilization on an H100 than a V100?

Metric: The ridge point $I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$ (FLOP/byte), the number of operations the hardware must perform for every byte loaded to keep its compute units busy.

Result: The intensity needed to reach the tensor peak has increased. Under this simple HBM roofline, a kernel with $I = 200$ FLOP/byte lies above the A100 ridge but below the H100 ridge. If its intensity and implementation remain unchanged, its idealized gain is then closer to the 1.6× bandwidth ratio than the 3.2× peak-compute ratio.

Systems insight: Fusion is valuable for low-intensity chains because it can remove intermediate round-trips; it is not automatically the best optimization for every kernel.

Operations such as depthwise convolution, embedding lookup, LayerNorm, and softmax serve as low-intensity reference points that commonly fall below the HBM ridge on compute-dense ac-

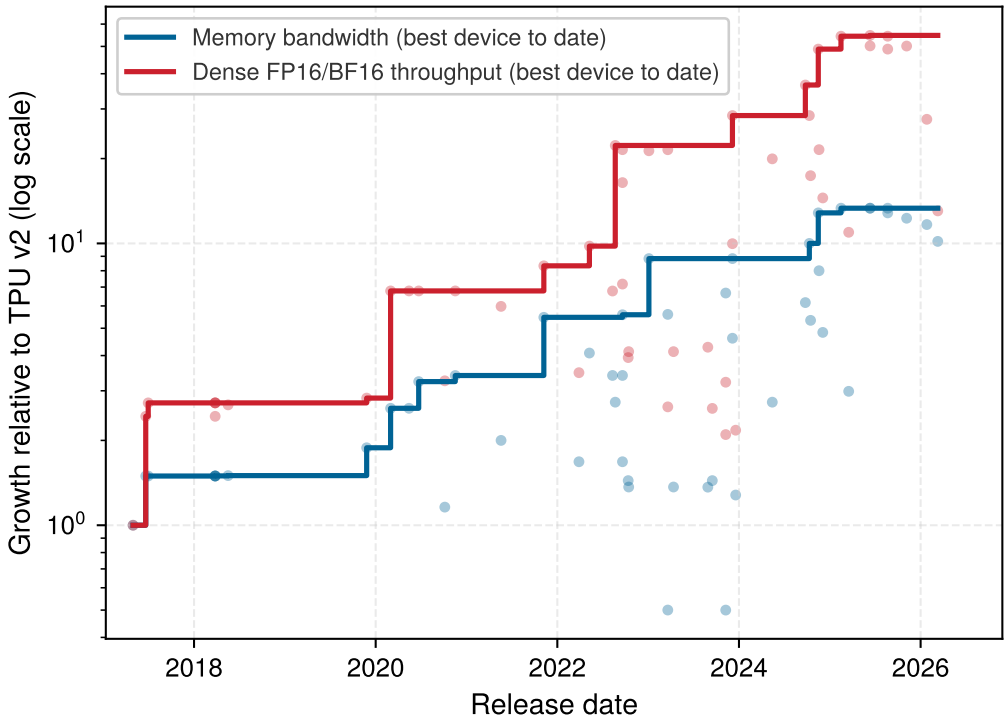


Figure 11.10: Compute and Bandwidth Across Vendors: Dense FP16/BF16 tensor throughput and memory bandwidth of machine learning accelerators from all major vendors with at least 16 GB of off-chip memory, each relative to TPU v2 (2017) on a logarithmic axis. Points are individual devices; step lines track the best device released to date. Compute pulls away from bandwidth, and both best-device envelopes level off in 2025 and 2026 as added arithmetic moves into lower-precision formats. Data from Epoch AI (2026b).

celerators under the stated traffic models. Table 11.7 maps common neural network operations to their representative roofline regimes and to the lighthouse workload that exercises each. Classification depends on the implementation, memory interface, and target path; profiling distinguishes a bandwidth ceiling from latency, limited parallelism, or other execution constraints.

Table 11.7: Operations on the Roofline: Neural network layers span a wide range of arithmetic intensities. Large, well-tiled convolutions and batched GEMMs can be compute-bound, while MobileNet depthwise layers, attention softmax, normalization, and DLRM embeddings are often memory-bound. To see how these intensity values translate into real performance predictions, a transformer layer provides a complete arithmetic-intensity calculation across its major sub-operations.

Operation	Arithmetic Intensity	**Classification**	**Lighthouse example**
Conv2D (Dense)	50–200 FLOP/byte	Straddles ridge; high-reuse cases compute-bound	ResNet-50
Dense MatMul (large batch, well-tiled)	~150–1,000+ FLOP/byte	Compute-bound once above the target's ridge	GPT-2 (batched projections)
Depthwise conv	10–20 FLOP/byte	Below modeled HBM ridge	MobileNet
Attention Softmax	Convention-dependent	Below modeled HBM ridge	GPT-2 (Generation)
LayerNorm	1–2 FLOP/byte	Below modeled HBM ridge	GPT-2/Llama
Embedding lookup	$<$ 1 FLOP/byte	Below modeled HBM ridge	DLRM

Napkin Math 11.4: Transformer layer analysis

For one GPT-2 XL layer during prefill, with hidden dimension $d = 1600$, batch size $B = 8$, sequence length $S = 1024$, and $N_{\text{heads}} = 25$ attention heads:

Attention QKV projection:

- FLOPs: $2 \times 3 \times 8 \times 1024 \times 1600 \times 1600 = 125.8$ GFLOP
- Bytes: (input + weights + output) = $(8 \times 1024 \times 1600 + 3 \times 1600 \times 1600 + 8 \times 1024 \times 1600 \times 3) \times 2 \approx 120.2$ MB
- $I = 125.8$ GFLOP / 120.2 MB = 1046.7 FLOP/byte, which is compute bound on the H100 (above the 295.2 FLOP/byte ridge point)

Softmax:

- FLOPs: $8 \times 25 \times 1024 \times 1024 \times 3 \approx 629.1$ MFLOP (exp, sum, div)
- Bytes: $8 \times 25 \times 1024 \times 1024 \times 2 \times 2 = 838.9$ MB
- $I = 629.1$ MFLOP / 838.9 MB = 0.75 FLOP/byte under this simple operation count, placing it below the reference tensor-path HBM ridge. Counting exponentials and division as multiple operations raises the numerical intensity, but those instructions use different execution units. The relevant vector, reduction, and special-function ceilings, together with profiling, determine whether bandwidth actually binds.

Systems insight: This analysis explains why FlashAttention focuses on reducing memory traffic in attention rather than reducing FLOPs.

The roofline supplies an initial resource bound; profiling determines which physical subsystem limits the kernel. When a kernel approaches its relevant off-chip bandwidth ceiling, optimization should focus on conserving memory traffic: fusing adjacent operators keeps intermediate activations in on-chip registers or shared memory (SRAM), lower-precision numeric formats reduce the physical byte count of memory transfers, and algorithmic refactorings like FlashAttention restructure access patterns to compute within SRAM tiles. When a kernel approaches the compute ceiling of its actual execution path, high data reuse keeps execution units supplied and off-chip bandwidth is no longer the binding resource. Performance here depends on saturating arithmetic execution pipelines: structuring matrix tiles to sustain Tensor Core throughput, hiding instruction latencies through software pipelining, and scaling batch sizes to distribute work across all available streaming multiprocessors.

↪ How a benchmark determines where a workload actually sits against its roofline ceiling is developed in section 12.4.3.

11.6.3 Memory bandwidth bounds

When a kernel's arithmetic intensity falls below the ridge point, adding more Tensor Cores cannot speed it up because off-chip memory bandwidth cannot deliver operands fast enough to sustain the target compute rate R_{ops} (FLOP/s). For a workload with arithmetic intensity I (FLOP/byte), keeping the compute units fed at rate R_{ops} requires the memory subsystem to transfer data at the rate defined by equation 11.3:

$$\text{BW}_{\text{req}} = \frac{R_{\text{ops}}}{I} \text{ bytes/s} \tag{11.3}$$

When this required bandwidth exceeds the hardware's peak memory bandwidth BW, memory transfers cap attainable throughput according to equation 11.4:

$$R_{\text{attain}} = \text{BW} \times I \tag{11.4}$$

Evaluating an operation against its execution path's ridge point identifies the lower modeled ceiling. Extensive data reuse can raise arithmetic intensity above that ridge, making compute the tighter bound; profiling must establish whether execution approaches it. ResNet-50's stage-3 3×3 convolution demonstrates this behavior.

Napkin Math 11.5: Convolutional layer analysis

Consider the 3×3 convolution in a stage-3 bottleneck block of ResNet-50, with input shape (batch = 32, channels = 128, height = 28, width = 28) and 128 output channels, on an H100 at FP16:

Computational requirements:

- Output size: $32 \times 128 \times 28 \times 28$ = 3.2M elements
- FLOPs per output: $128 \times 3 \times 3 \times 2$ = 2,304 (multiply-add)
- Total FLOPs: 3.2M × 2,304 = 7.4 GFLOP

Memory traffic analysis:

- Input: $32 \times 128 \times 28 \times 28 \times 2$ = 6.4 MB (FP16)
- Weights: $128 \times 128 \times 3 \times 3 \times 2 \approx$ 0.3 MB (FP16)
- Output: $32 \times 128 \times 28 \times 28 \times 2$ = 6.4 MB (FP16)
- Total: 13.1 MB

Arithmetic intensity: I = 7.4 GFLOP / 13.1 MB = 563.1 FLOP/byte

Systems insight: This is well above the H100's FP16 ridge point of 295.2 FLOP/byte, so the simple roofline model places the operation in the compute-limited region. The dense FP16 tensor peak of ~989 TFLOP/s is therefore the ceiling, but realized performance still depends on tiling, occupancy, instruction mix, and library efficiency.

The convolutional layer's high arithmetic intensity arises from spatial weight reuse: the same 3×3 kernel is applied across every spatial location in the feature map, amortizing the cost of loading weights from off-chip memory across millions of multiply-accumulate operations. Keeping active tiles in registers and shared memory reduces repeated off-chip transfers, making high arithmetic throughput possible. Whether a particular kernel then saturates the Tensor Cores depends on tiling, occupancy, and instruction mix, which the roofline does not model. Moreover, the FP16 analysis is only half of the chapter's workload, because the profile chapter 10 ended on runs ResNet-50 in INT8; quantization changes both sides of the roofline at once, and the two changes need not cancel.

Napkin Math 11.6: When INT8 moves a layer below the ridge

Problem: The H100's INT8 tensor path offers about 2× the operations per second of its FP16 path from the same 3.35 TB/s of memory bandwidth. Which ResNet-50 layers stay compute bound when the model moves from FP16 to INT8?

The two ridges: The ridge rises from 295.2 FLOP/byte at FP16 to 590.7 FLOP/byte at INT8. A layer keeps its position relative to the ridge only if every byte it moves halves along with the operand width.

Three layers at batch 32 (operations per byte, one read of input and weights and one write of the output):

- **Stage-3** 3×3 **convolution** (the layer of notebook 11.5): 563 at FP16, above the FP16 ridge. With INT8 activations it reaches 1,126, above the INT8 ridge. If its activations stay FP16 between layers, as when the kernel reads and writes 16-bit tensors for FP16 neighbors, its intensity stays near 569, below the INT8 ridge.
- **Stage-2** 3×3 **convolution** (56×56 positions, 64 channels in and out): 287 at FP16 and 574 at INT8, just below the ridge in both precisions.
- **Stage-2** 1×1 **expansion** (56×56 positions, 64 channels in and 256 out): 51 at FP16 and 102 at INT8, far below the ridge in both.

Systems insight: INT8 raises the arithmetic ceiling but leaves the bandwidth ceiling where it was, so the layers that were already bandwidth bound gain only the halved bytes, and a

compute-bound layer whose activations stay wide crosses below the new ridge. The INT8 speedup of chapter 10's profile therefore depends on whether activations travel at INT8 between layers, a decision made by the compiler and runtime (section 11.8), not by the quantized weights alone.

Not all neural network layers reuse data so heavily. Fully connected (dense) layers, such as ResNet-50's classification head, feed-forward networks, and linear projections in transformer attention blocks, lack spatial dimensions. When evaluated at small batch sizes, their arithmetic intensity drops substantially, shifting execution into the bandwidth-bound regime:

Napkin Math 11.7: Dense layer analysis

Consider ResNet-50's final fully connected layer, which maps 2,048 pooled features to 1,000 class scores for a batch of 32 images on the same H100:

Computational requirements:

- Matrix multiply: $(32 \times 2048) \times (2048 \times 1000)$
- Total FLOPs: $2 \times 32 \times 2048 \times 1000$ = 131.1 MFLOP

Memory traffic analysis:

- Input: $32 \times 2048 \times 2$ = 131.1 KB (FP16)
- Weights: $2048 \times 1000 \times 2$ = 4.1 MB (FP16)
- Output: $32 \times 1000 \times 2$ = 64 KB (FP16)
- Total: 4.3 MB

Arithmetic intensity: I = 131.1 MFLOP / 4.3 MB = 30.5 FLOP/byte

This intensity sits below the H100's ridge point of 295.2 FLOP/byte, making this operation memory bound under the stated traffic model. Attainable performance: R_{attain} = 3,350 GB/s $\times$ 30.5 FLOP/byte = 102.3 TFLOP/s

Systems insight: Attainable throughput is only 10.3 percent of peak compute capability, so at batch 32 the memory wall caps this layer long before the Tensor Cores do.

The dense layer's lower arithmetic intensity stems from limited parameter reuse. While a convolutional kernel is reused across every spatial location in an activation map, a dense weight matrix is reused only across the batch dimension. At a batch size of 32, each fetched weight contributes to 32 sample products in this operation. When the weights stream from HBM on each invocation, as modeled, the bandwidth ceiling caps throughput at 10.3 percent of peak compute capacity. The fraction is a bound rather than a measured stall count, and a layer whose weights stay cached between invocations would do better. The same arithmetic explains why transformer inference at low batch sizes is governed by memory bandwidth rather than arithmetic capability, a regime the throughput-ceiling notebook below quantifies for GPT-2.

While small-batch matrix multiplications suffer from restricted parameter reuse, elementwise and reduction operators represent an even more acute memory bottleneck. Operations such as layer normalization, activation functions, and residual additions perform only a few arithmetic operations per element loaded from memory, driving arithmetic intensity to its physical floor:

Napkin Math 11.8: LayerNorm analysis

LayerNorm over one GPT-2 XL layer's activations, with input shape (batch = 8, sequence = 1024, hidden = 1600):

Computational requirements:

- Elements: $8 \times 1024 \times 1600$ = 13.1M

- Approximate operations per element: mean reduction (1 ADD), variance (2 ADD, 1 MUL), normalization and affine transform (2 ADD, 2 MUL) ≈ 8; reciprocal-square-root overhead is amortized across each hidden vector
- Total FLOPs: 13.1M × 8 = 104.9 MFLOP

Memory traffic:

- Input: 13.1M × 2 = 26.2 MB
- Parameters (scale, bias): 1600 × 2 × 2 = 6.4 KB (negligible)
- Output: 13.1M × 2 = 26.2 MB
- Total: 52.4 MB

Arithmetic intensity: I = 104.9 MFLOP / 52.4 MB = 2.0 FLOP/byte

This arithmetic intensity sits severely below the H100 ridge point (148× below). The HBM performance upper bound is: R_{attain} = 3,350 GB/s × 2.0 FLOP/byte = 6.7 TFLOP/s

Systems insight: The simple HBM roofline bounds this standalone LayerNorm at about 0.7 percent of the H100's tensor peak. That tensor peak is a reference, not the throughput available to LayerNorm's vector and reduction instructions. Actual latency also depends on their execution ceilings, fusion, launch overhead, and which memory level supplies the data.

Standalone normalization performs minimal arithmetic relative to the memory traffic it generates, as the LayerNorm analysis demonstrates. Under the modeled unfused HBM read/write boundary, the implementation performs approximately 2 FLOP/byte, giving a low bandwidth ceiling relative to the tensor peak. Reaching that ceiling still requires enough parallel work and an efficient reduction. Cache residency changes the physical HBM byte count and can shift the relevant bound to another memory interface. In the D·A·M framework, when the machine constraint is a memory bandwidth wall, the algorithmic response is kernel fusion, which collapses adjacent projections, bias additions, normalization, and activations into one kernel so that intermediate values stay in registers or SRAM. Fusion pays off when the grouping preserves numerical behavior and fits the local register and SRAM budget, and profiling confirms that the saved traffic outweighs any loss of occupancy.

11.6.4 Optimization by intensity regime

Tuning thread-block occupancy on a bandwidth-bound LayerNorm kernel wastes engineering effort for zero speedup, just as fusing two already compute-bound matrix multiplications cannot raise throughput past the Tensor Core ceiling. The roofline's intensity regimes (table 11.8) match each bottleneck to the lever that can move it.

Table 11.8: Optimization by Arithmetic Intensity Regime: Position relative to the target execution path's ridge guides compute and traffic optimization. Profiling confirms the bottleneck; fusion removes intermediate round-trips when dependencies and local storage permit it.

Intensity regime	Typical operations	Optimization priority	Common techniques	Expected impact
Above the target path's ridge	Large convolutions	Maximize compute utilization.	Tensor Cores, thread-block tuning, and high occupancy.	Raises sustained compute utilization when the kernel already clears the ridge.
Near the target path's ridge	Medium-sized dense layers	Balance compute and memory optimization.	Larger batches, register tiling, and fusion with adjacent operations.	Can move the binding bottleneck between memory and compute.
Below the target path's ridge	Small dense layers and element-wise operations	Reduce memory traffic.	Aggressive kernel fusion, reduced precision (FP16 → INT8), and algorithmic changes.	Reduces external-memory traffic when fusion or layout changes are legal.
Low-intensity operator chains	Normalization layers and activation functions	Eliminate memory round-trips.	Fuse with adjacent operations and use in-place computation where legal.	Removes intermediate round-trips when adjacent operations can be fused.

For low-intensity operations, kernel fusion is often the decisive optimization: LayerNorm combined with the Gaussian Error Linear Unit (GELU), for example, can become a single fused kernel. One of the most accessible levers for moving an operation up and right on the roofline is batching.

Napkin Math 11.9: Batch size and arithmetic intensity

Increasing batch size improves arithmetic intensity for matrix operations by amortizing weight loading. Equation 11.5 formalizes this relationship for an idealized FP16/BF16 dense layer $(B \times d_{\text{in}}) \times (d_{\text{in}} \times d_{\text{out}})$, counting one read each of the input and weights and one write of the output, where B is batch size and d_{in} and d_{out} are the layer's input and output widths:

$$I = \frac{2Bd_{\text{in}}d_{\text{out}}}{2Bd_{\text{in}} + 2d_{\text{in}}d_{\text{out}} + 2Bd_{\text{out}}} \approx \frac{2Bd_{\text{in}}d_{\text{out}}}{2d_{\text{in}}d_{\text{out}}} = B \quad (\text{when } 2d_{\text{in}}d_{\text{out}} \gg 2B(d_{\text{in}} + d_{\text{out}})) \tag{11.5}$$

Example: Dense layer with $d_{\text{in}} = d_{\text{out}} = 2048$ (FP16)

- Batch = 1: $I \approx 1$ FLOP/byte (memory bound)
- Batch = 32: $I \approx 31$ FLOP/byte (memory bound)
- Batch = 512: $I \approx 341.3$ FLOP/byte (compute bound on the H100)

Systems insight: This explains why batching can produce large throughput improvements in production inference systems, as MLPerf Inference, the standardized benchmark suite, demonstrates by separating bulk-throughput runs from latency-constrained serving runs (Reddi et al. 2019).

↪ The MLPerf Inference execution scenarios that separate throughput runs from latency-constrained runs are developed in section 12.4.4.1.

The batch size analysis reveals why inference serving systems are designed around batching: it changes the arithmetic intensity regime of bandwidth-bound workloads. However, batching introduces latency trade-offs, since requests must wait in a queue until a batch forms. This tension between throughput (favoring large batches) and latency (favoring small batches) is a central challenge in ML serving systems, explored in depth in section 13.5.4. When service latency limits batching, batch-1 or small-batch LLM decode often retains low weight reuse and low arithmetic intensity, and the next worked model quantifies that particular regime rather than treating all LLM inference as identical.

Napkin Math 11.10: The throughput ceiling

Problem: A batch-1 GPT-2 calculation makes this bandwidth ceiling concrete: what is the maximum possible utilization of an NVIDIA H100 when running GPT-2 inference (batch size 1)?

The hardware constraints (the denominators)

- **Peak compute**: 989 TFLOP/s (FP16 Tensor Core).
- **Peak bandwidth**: 3.35 TB/s (HBM3).
- **Ridge point** ($R_{\text{peak}}/\text{BW}$): 989 TFLOP/s / 3.35 TB/s = 295.2 FLOP/byte (for FP16 Tensor Core).
  Interpretation: Saturating this chip at FP16 precision requires 295.2 FLOP/byte operations for every byte loaded. The ridge point varies by precision: FP32 operations (67 TFLOP/s peak) have a ridge point of only ~20 FLOP/byte.

The workload characteristics (the numerator)

- **Model**: GPT-2 XL (1.5 billion parameters).
- **Operation**: Autoregressive generation (1 token at a time).
- **Data movement**: Must load all weights (3 GB @ FP16) for every token.
- **Compute**: Vector-Matrix multiplication. Two FLOPs per parameter, $2P \approx 3$ GFLOP.

- **Arithmetic intensity**: 3 GFLOP / 3 GB = 1 FLOP/byte

The prediction (iron law)
Since the modeled intensity I (1) $\ll$ the ridge point I_{ridge} (295.2 FLOP/byte), batch-1 decode is bandwidth bound.

- **Maximum throughput**: 1 FLOP/byte $\times$ 3.35 TB/s = 3.4 TFLOP/s.
- **Utilization ceiling**: 3.4 TFLOP/s (Bound) / 989 TFLOP/s (Peak) $\approx 0.3\%$

Systems insight: Without batching, a ~$25,000 accelerator cannot exceed 0.3 percent of its compute peak in this weight-streaming decode scenario. Batching and weight quantization address that gap; key-value caching instead avoids recomputing prior attention states. The ceiling also assumes that the decode step is launched as a captured graph (section 7.3.6); executed one eager kernel at a time, launch overhead rather than weight streaming would set the floor.

Placed on one roofline, the layers and the decode step analyzed above fall on either side of two ridges, and only the stage-3 3×3 convolution reaches a flat ceiling (figure 11.11). Through this derivation, the roofline model provides a diagnostic framework for identifying whether operations are compute bound or memory bound. Knowing that batch-1 decode is bandwidth bound at a ceiling of 0.3 percent of peak is only the first step; the next challenge is translating this diagnosis into efficient execution plans that exploit accelerator architectures.

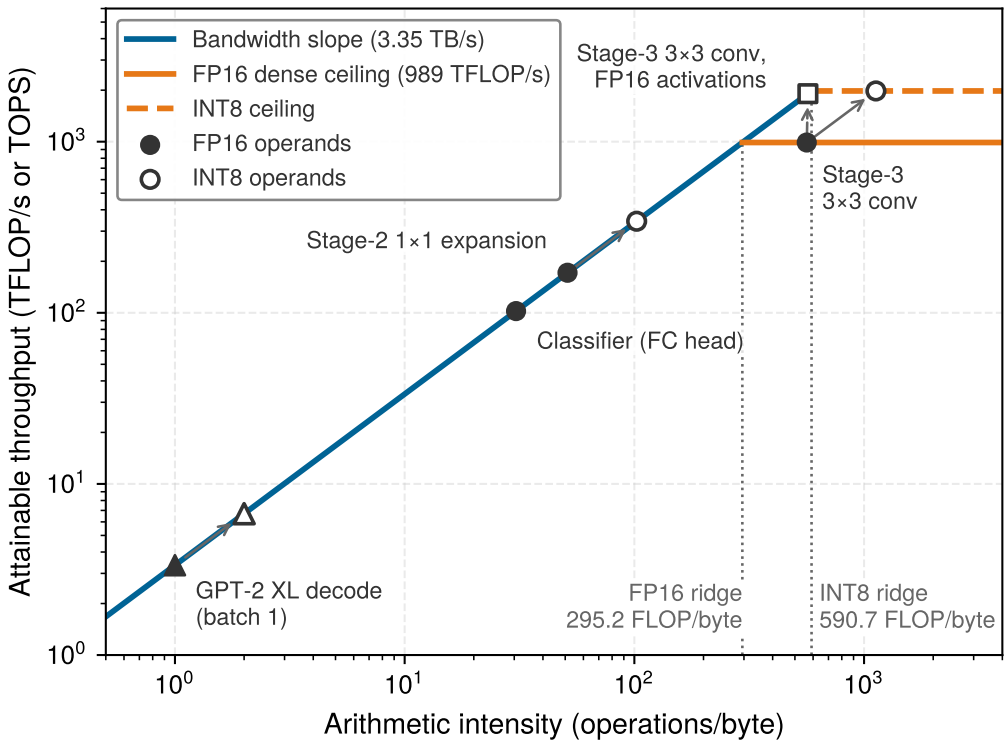


Figure 11.11: ResNet-50 and GPT-2 XL on the H100 Roofline: Attainable throughput against arithmetic intensity on log-log axes, with the dense FP16 ceiling and the INT8 ceiling sharing one bandwidth slope. INT8 raises the ridge from 295.2 FLOP/byte to 590.7 FLOP/byte. Arrows connect each layer's FP16 point to its INT8 point. ResNet-50's stage-3 convolution clears the ridge in both precisions when its activations move at the format's width, but falls below the INT8 ridge when its activations stay FP16; its stage-2 expansion and classifier sit far below either ridge, and GPT-2 XL decode sits more than an order of magnitude further left.

Checkpoint 11.3: The machine and its ceilings

The first half of the chapter has assembled the accelerator and its two ceilings. Before turning to how software reaches them, check that you can work from the anatomy to a bound.

Compute units

- ☐ **The INT8 path**: name the two sources of the INT8 tensor path's advantage over FP16, and explain why a 2:4 sparse kernel speeds up only weights that satisfy the pattern.

Memory systems

- ☐ **Operand supply**: explain why a matrix unit cannot be fed directly from HBM, and which tier a ResNet-50 activation tile occupies while a Tensor Core consumes it.

Performance bounds

- ☐ **Two ridges**: from the H100's FP16 and INT8 peaks and its memory bandwidth, compute both ridge points, then place GPT-2 XL batch-one decode and the ResNet-50 stage-3 convolution against each.
- ☐ **The crossing**: explain why a layer that is compute bound at FP16 can become bandwidth bound at INT8, and what change to its traffic would move it back above the ridge.

11.7 Hardware Mapping

Consider one of ResNet-50's 3×3 convolutions running on an accelerator tile. The mathematical operation is fixed, but the execution plan is not. One schedule can keep a filter in local registers while many output pixels stream past it; another can advance pixel by pixel and reload the same filter values repeatedly from a slower memory tier. Both schedules compute the same tensor. Only one turns the reuse in the convolution into real bandwidth savings.

Definition 11.4: Mapping in AI acceleration

Mapping in AI acceleration is the accelerator-compiler process of binding the logical computation graph to the physical hardware topology by deciding which operations execute on which processing elements, which data resides in which memory tier, and in what temporal order.

1. **Significance**: Within the D·A·M taxonomy, mapping is a machine-axis decision that helps determine how closely an operation approaches the roofline bound $\min(R_{\text{peak}}, \text{BW} \times I)$. A poor tiling choice that forces unnecessary DRAM accesses can reduce effective arithmetic intensity enough to move a nominally compute-bound operation into the bandwidth-bound regime.
2. **Distinction**: Traditional compilation often emphasizes instruction selection, register allocation, and scheduling for general-purpose processors. Accelerator mapping gives equal prominence to spatial placement and explicit data movement across a dataflow architecture. The reason is physical: as quantified by the storage-tier energy ladder in figure 11.4, an off-chip DRAM access costs about 173× the energy of an FP32 multiply, making operand placement as critical to efficiency as the execution unit that evaluates it.
3. **Common pitfall**: A frequent misconception is that mapping is automatically handled by frameworks. For general GPU workloads, compilers such as Accelerated Linear Algebra (XLA) can find strong mappings for common kernels; for specialized accelerators (systolic arrays, custom ASICs), compiler-generated mappings may still lag hand-tuned schedules because the compiler's search space is limited by the time budget at compilation.

The convolution example exposes the three decisions that recur throughout accelerator compilation. Placement assigns the multiply-accumulate work to processing elements so parallelism does not turn into idle time or interconnect congestion. Allocation keeps weights, activations, and partial sums in the memory tier where their next use will occur, rather than letting reuse spill back to

DRAM. Scheduling orders loops and kernels so the chosen placement and allocation remain valid over time. A poor choice in any one dimension can collapse a high-arithmetic-intensity operation back into a bandwidth-bound execution. In practice, these choices are too coupled for developers to manage by hand at model scale, which is why systems such as XLA, TVM, and TensorRT lower high-level models and search or select execution plans within their compile-time and hardware budgets. Section 11.8 examines that compiler support in detail.

11.7.1 Placement and allocation

Translating a model's computational graph into efficient hardware execution requires solving two tightly coupled problems. Computation placement determines which operations run on which processing elements, balancing parallelism against communication costs. Memory allocation determines where data resides within the memory hierarchy, trading capacity against access latency. These two decisions interact: placing operations on distant processing elements increases the memory bandwidth required to shuttle data between them, while allocating data to fast but small on-chip memory limits which operations can execute concurrently. Getting either wrong leaves thousands of processing elements idle or starved for data.

11.7.1.1 Computation placement

Computation placement is the process of assigning operations to an accelerator's processing elements (PEs) to expose parallelism, limit idle time, and reduce unnecessary data movement. Modern accelerators contain many such resources: the NVIDIA H100 has more than 16,000 CUDA cores and more than 500 Tensor Cores (Choquette 2023), TPUs organize thousands of multiply-accumulate units into systolic arrays (Jouppi et al. 2017), and wafer-scale processors such as Cerebras' CS-2 integrate more than 850,000 cores (Systems 2021). At these scales, placement inefficiencies become measurable because idle cores and redundant transfers waste both time and energy.

The difficulty of placement depends on workload regularity. Convolutional networks and dense transformer operations possess regular access patterns that tile predictably, though transformer architectures mix operations with distinct tensor shapes and reuse profiles. Graph neural networks (GNNs) present greater difficulty: sparse, input-dependent neighborhood aggregations introduce load imbalance and non-uniform communication across processing elements. Table 11.9 summarizes these core placement challenges and their operational impact.

Table 11.9: Computation Placement Challenges: Effective neural network deployment requires strategic allocation of computations to processing elements, balancing workload distribution, data movement costs, and hardware constraints to maximize execution efficiency. These challenges guide the design of mapping strategies that optimize resource utilization and minimize communication overhead.

Challenge	Impact on execution	Key Considerations for Placement
Workload imbalance	Some processing elements finish early while others remain overloaded, leading to idle compute resources.	Distribute operations evenly to prevent stalls and ensure full utilization of PEs.
Irregular computation patterns	Sparse graphs, routing, and variable shapes can create nonuniform work that complicates static placement.	Use adaptive placement when workload characteristics vary at run time.
Excessive data movement	Frequent memory transfers introduce latency and increase power consumption.	Keep frequently used data close to the compute units and minimize off-chip memory accesses.
Limited interconnect bandwidth	Poorly placed operations can create congestion, slowing data movement between PEs.	Optimize spatial and temporal placement to reduce communication overhead.
Model-specific execution needs	CNNs, transformers, and GNNs require different execution patterns, making a single placement strategy ineffective.	Tailor placement strategies to match the computational structure of each model type.

Static spatial mappings suffice for regular dense operations whose shapes and loop bounds are known ahead of time. Workloads with dynamic shapes, sparsity, or mixture-of-experts routing instead require runtime scheduling to balance execution across processing elements. Yet even an optimal computation placement stalls if the required operands are not staged in time: compute units can execute only as fast as operands arrive from the memory hierarchy.

11.7.1.2 Memory allocation

While computation placement determines where operations execute, memory allocation defines where data resides and how it flows through the hierarchy. The goal is to keep reused data near the processing elements without exceeding faster-tier capacity. GPUs expose global memory, shared memory, caches, and registers that kernels coordinate through tiling (NVIDIA Corporation 2020a). TPUs use on-chip SRAM buffers to stage the activations that flow horizontally through the 2D systolic-array dataflow (Jouppi et al. 2017) of figure 11.5, with partial sums flowing vertically, while wafer-scale processors partition memory and computation to control interconnect traffic (Systems 2021). General-purpose processors and accelerators both use memory hierarchies, but many accelerators expose more placement decisions to compilers and kernels. When allocation fails to exploit locality, processing elements stall waiting for operands, driving down utilization and dissipating energy across the memory bus.

The severity of these penalties varies by workload. Convolutional networks rely on structured, localized access patterns and benefit from well-defined memory layouts that support predictable reuse (Y.-H. Chen et al. 2016). Transformers require access to large parameter sets and intermediate activations, making them sensitive to capacity and bandwidth. Graph neural networks add irregular sparse structures that complicate allocation and prefetching.

Capacity is only the first allocation test. A tensor can fit in accelerator memory yet still dominate execution if each kernel rereads it from a distant tier; conversely, tiling can keep a larger working set efficient by preserving reuse locally. The allocation diagnostic is therefore traffic, not capacity alone: compilers must track how frequently values cross each memory boundary and keep high-reuse tensors in the fastest feasible tier. Systems combine static memory plans for known shapes with dynamic buffer management when shapes vary, while device capacity limits which models fit without partitioning or offload. Table 11.10 summarizes these allocation challenges across the hierarchy.

Table 11.10: Memory Allocation Challenges: Efficient memory management in AI accelerators balances data access speed with hardware constraints, mitigating performance bottlenecks caused by latency, bandwidth limitations, and irregular data patterns. Complex models such as transformers and graph networks impose variable and demanding memory requirements that amplify these challenges.

Challenge	Impact on Execution	**Key Considerations for Allocation**
High memory latency	Slow data access delays execution and reduces throughput.	Prioritize placing frequently accessed data in faster memory locations.
Limited on-chip storage	Small local memory constrains the amount of data available near compute units.	Allocate storage efficiently to maximize data availability without exceeding hardware limits.
High off-chip bandwidth demand	Frequent access to external memory increases delays and power consumption.	Reduce unnecessary memory transfers by carefully managing when and how data is moved.
Irregular memory access patterns	Some models require accessing data unpredictably, leading to inefficient memory usage.	Organize memory layout to align with access patterns and minimize unnecessary data movement.
Model-specific memory needs	Different models require different allocation strategies to optimize performance.	Tailor allocation decisions based on the structure and execution characteristics of the workload.

11.7.2 Combinatorial complexity

The convolution that opened this section illustrates why hardware mapping becomes a combinatorial search problem. Keeping a filter resident in local storage improves reuse only if the assigned processing elements have sufficient storage and if the loop schedule revisits that filter before eviction. Similarly, parallelizing across more processing elements improves throughput only until synchronization overhead and interconnect contention erode the benefit. Because changing one decision alters the feasible and efficient choices for the others, compilers must evaluate placement, allocation, and scheduling jointly rather than independently. Table 11.11 summarizes these recurring trade-offs across the execution stack.

Table 11.11: Placement-Allocation-Scheduling Trade-Offs: AI accelerator performance depends on mapping computations to hardware, allocating data to memory tiers, and scheduling execution over time. Careful consideration of these interdependent factors is essential for maximizing throughput and minimizing energy consumption.

Dimension	**Placement considerations**	**Allocation and Scheduling Considerations**
Computational granularity	Fine-grained placement enables greater parallelism but increases synchronization overhead.	Coarse-grained scheduling reduces synchronization overhead but may limit flexibility.
Spatial vs. Temporal Mapping	Spatial placement enhances parallel execution but can lead to resource contention and memory congestion.	Temporal scheduling balances resource sharing but may reduce overall throughput.
Memory and Data Locality	Placing data closer to compute units minimizes latency but may reduce overall memory availability.	Allocating data across multiple memory levels increases capacity but introduces higher access costs.
Communication and synchronization	Co-locating compute units reduces communication latency but may introduce contention.	Scheduling synchronization mechanisms mitigates stalls but can introduce additional overhead.
Dataflow and Execution Ordering	Static placement simplifies execution but limits adaptability to workload variations.	Dynamic scheduling improves adaptability but adds scheduling complexity.

11.7.2.1 Counting the search space

Evaluating these trade-offs requires navigating an unconstrained search space governed by three structural decisions: loop execution order, spatial parallelization across processing elements, and memory placement across the hierarchy. Counting each dimension on its own shows why exhaustive search is impractical.

Machine learning workloads are expressed as nested loops over tensor dimensions, and the loop execution order dictates which tensor elements remain resident in registers or SRAM between iterations. Ignoring dependences and equivalent orderings, the number of ways to arrange n_{loops} loops has the toy upper bound:

$$N_{\text{order}} = n_{\text{loops}}!$$

A convolutional layer with seven loop dimensions already has $7! = 5{,}040$ possible execution orders. If each memory level could choose an ordering independently, the toy upper bound would expand as:

$$(n_{\text{loops}}!)^{N_{\text{mem}}}$$

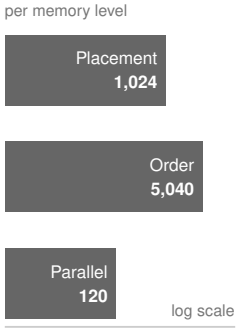
where N_{mem} is the number of memory hierarchy levels. A suboptimal ordering evicts values prematurely and multiplies DRAM traffic, whereas an optimized ordering maximizes reuse in local SRAM and caches (Sze et al. 2017).

Parallelization multiplies the count again. Parallelizing outer independent loops (such as batch or output feature channels) lets each core execute without interconnect traffic, whereas parallelizing an inner reduction loop requires cross-core synchronization and collective communication. Before legality and capacity constraints are applied, the number of ordered ways to select loops for parallel execution has the upper bound:

$$\mathcal{P}_{\text{parallel}} = \frac{n_{\text{loops}}!}{(n_{\text{loops}} - k_{\text{parallel}})!}$$

where k_{parallel} is the number of loops selected for parallel execution; three of six loops already give $6!/3! = 120$ candidates, each with different demands on synchronization, interconnect bandwidth, and PE utilization. Memory placement is the third dimension, and physical capacity bounds it. Because an off-chip DRAM fetch costs roughly 128× more energy than a read from an 8 KB SRAM array (figure 11.4), where each tensor resides directly dictates the achieved arithmetic intensity, yet an operand tile can remain in on-chip SRAM only if its working set fits the allocated buffer. Larger resident tiles raise reuse but consume scarce capacity and register pressure, potentially reducing thread concurrency or forcing spills to slower tiers.

These structural choices (loop ordering, spatial parallelization across processing elements, tile sizing, and memory placement) are deeply coupled. Choosing an output-stationary schedule, for example, pins accumulators in local registers and dictates which loops can be mapped spatially across



Per memory level, from the section's separate toy examples: $4^5 = 1{,}024$ placements, $7! = 5{,}040$ orders, $6!/3! = 120$ parallelizations. The product is raised to the number of levels, so three levels give 4^{15} placements alone.

processing elements without requiring inter-core reduction networks. Conversely, parallelizing across more execution units reduces the per-core SRAM budget available for operand staging, forcing smaller tile dimensions and increasing off-chip memory traffic.

While the unconstrained Cartesian product of these decisions suggests a massive combinatorial space, hardware capacity limits, data dependencies, and symmetry prune the vast majority of candidates as either physically illegal or strictly dominated. Exhaustive search across arbitrary loop permutations and placement choices is therefore both unnecessary and intractable. Production compilers and kernel developers manage this complexity by structuring their search around a small family of canonical dataflow patterns, spatial and temporal schedules engineered to exploit the specific reuse properties of neural network operators.

11.7.3 Dataflow optimization

As example 11.4 shows for a small convolution, two mathematically identical schedules can differ 36-fold in weight reads, and every avoided read that would have reached DRAM saves the most expensive access on the energy ladder. While spatial and temporal mapping establish where operations execute and where operands reside, **dataflow optimization** governs the exact trajectories and schedules of data moving through processing elements and the memory hierarchy to maximize operand reuse. In a systolic array or tiled vector unit, the temporal schedule and spatial routing of weights, inputs, and partial sums dictate how frequently operands must be fetched from energy-expensive DRAM rather than reused from local registers or scratchpads. By controlling off-chip traffic, the dataflow establishes the effective operational intensity of the kernel, determining whether execution achieves the compute-bound plateau or stalls in the bandwidth-bound regime of the roofline model. Optimizing compilers (section 11.8) and runtime systems (section 11.8.3) therefore tailor dataflow patterns to match the reuse characteristics of each layer to the hardware constraints of the target accelerator.

Three recurring architectural decisions structure this design space, providing a compact vocabulary for navigating dataflow choices without exhaustive search across billions of candidate schedules:

1. **Locality**: Stationary operand selection (weight-stationary, output-stationary, or input-stationary) dictates which tensor remains pinned in local registers or processing-element scratchpads, trading off memory traffic against register pressure.
2. **Organization**: Physical tensor layouts (such as channels-last `NHWC` versus channels-first `NCHW`) govern whether memory accesses match hardware memory controller burst sizes and SIMD vector widths, preventing uncoalesced memory transactions.
3. **Combination**: Kernel fusion and loop tiling partition iteration spaces and merge adjacent operations to eliminate round-trips to off-chip memory for intermediate tensors.

11.7.4 Stationary dataflows

Dataflow design exists to exploit the steep energy divergence isolated in figure 11.4, where fetching an operand from DRAM requires orders of magnitude more energy than executing an arithmetic operation on it (Y.-H. Chen et al. 2016). Whether a workload executes dense matrix operations that exceed on-chip cache capacity or irregular sparse lookups, data movement strategy dictates overall execution latency and total power consumption. If data cannot be staged and supplied to processing elements at the rate required by their arithmetic throughput, the execution units stall on memory latency, stranding theoretical compute capacity and inflating memory subsystem energy.

To prevent arithmetic pipelines from stalling, a data movement strategy must capture the inherent algorithmic reuse of tensor operations within local, low-energy storage. Listing 11.14 illustrates the canonical three-nested-loop implementation of matrix multiplication, exposing the operand reuse across weights, inputs, and partial sums that a hardware mapping must capture in local storage.

Listing 11.14: Naïve Matrix-Multiplication Dataflow: The loop nest revisits weight rows, input columns, and output accumulators, exposing the reuse that a mapping must capture in local memory.

```
# Matrix multiplication where:
# weights: [512x256] - model parameters
# input:   [256x32]  - batch of activations
# Z:       [512x32]  - output activations

# Computing each output element Z[i,j]:
for i in range(512):
    for j in range(32):
        for k in range(256):
            Z[i, j] += weights[i, k] * input[k, j]
```

Every operand in this loop nest is reused, but along a different loop. Each weight `weights[i, k]` is used once for each of the 32 output columns, each input `input[k, j]` is used once for each of the 512 output rows, and each output `Z[i, j]` accumulates 256 products before it is final. A processing element's local storage holds only a small tile at a time, so the mapping cannot capture all three reuses at once. Fetching from DRAM whichever operand is not held locally, on every use, is what turns a compute-bound kernel into a memory-bound one (Han, Liu, et al. 2016).

The central dataflow decision is therefore which operand to keep local. The loop order answers it: whatever the innermost loops touch repeatedly without changing stays resident, and everything else streams. Listing 11.15 reorders the same computation three ways, and the three orders are the three classic stationary dataflows.

Listing 11.15: Three Stationary Dataflows: The same matrix multiplication as the naïve loop, reordered three ways; the operand held fixed across the innermost loop is the one fetched once and reused from local storage, while the other two stream.

```
# Weight stationary: hold one weight, stream inputs and partial sums past it
for i in range(512):
    for k in range(256):
        w = weights[i, k]  # fetched once, reused 32 times
        for j in range(32):
            Z[i, j] += w * input[k, j]

# Output stationary: hold one partial sum, stream weights and inputs past it
for i in range(512):
    for j in range(32):
        acc = 0  # stays in a local accumulator
        for k in range(256):
            acc += weights[i, k] * input[k, j]
        Z[i, j] = acc  # written once

# Input stationary: hold one input, stream weights and partial sums past it
for k in range(256):
    for j in range(32):
        x = input[k, j]  # fetched once, reused 512 times
        for i in range(512):
            Z[i, j] += weights[i, k] * x
```

A weight-stationary dataflow keeps parameters on chip, so each weight crosses the expensive memory boundary once no matter how many inputs it touches. That pays off wherever one filter or weight tile is applied to many inputs, as in convolutions and batched matrix multiplication, and it is the canonical TPU dataflow traced cycle by cycle in section 11.7.5. The saved traffic reappears on the other operands: inputs must arrive fast enough to keep the resident weights busy, and partial sums must move between processing elements as they accumulate. The resident tile is also bounded by on-chip capacity, so a layer whose weights far exceed it must cycle weight tiles through, reloading the inputs for each one.

An output-stationary dataflow keeps each running sum in a local accumulator until all of its contributions have arrived, so every output is written to memory exactly once (Y.-H. Chen et al.

2016). It suits layers with wide fan-in, such as fully connected layers and attention projections, where writing a partial sum out after every step would cost far more than the reads it avoids. The cost is two operand streams at once, since both weights and inputs must now be delivered from memory. The naïve loop in listing 11.14 already has this order, because its innermost loop runs over k while i and j stay fixed.

An input-stationary dataflow keeps an activation tile local while weights stream past it. It pays off when one activation feeds several computations before eviction, as when a transformer token's activation feeds the query, key, and value projections. Its costs mirror weight stationary's, with weights now streaming from memory and partial sums still moving as they accumulate.

Example 11.4: Loop ordering in a small convolution

Consider a convolution applying 16 filters of size 3×3 to an 8×8 single-channel input. The computation can be expressed as five nested loops iterating over output rows (H_{out}), output columns (W_{out}), filter count (d_{out}), filter height (K_h), and filter width (K_w). The $5! = 120$ permutations are an unconstrained upper bound: legal orderings preserve the mathematical result, although floating-point reduction order can change rounding, and they generate different memory traffic.

Ordering A (weight-stationary): Place the filter loops (d_{out}, K_h, K_w) outermost and the spatial loops (H_{out}, W_{out}) innermost. Each 3×3 filter is loaded into registers once and then applied across all 36 output positions before the next filter is loaded. Total weight loads: $16 \times 9 = 144$ values, each loaded exactly once.

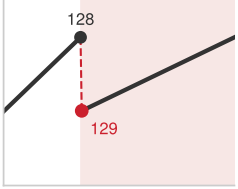
Ordering B (output-stationary, filters spilled): Place the spatial loops outermost and the filter loops innermost. For every output position, all 16 filters must be loaded, applied, and their partial sums accumulated before advancing to the next position. If the register file cannot hold all 16 filters simultaneously, filters are repeatedly fetched from cache or DRAM. In the worst case, each of the 36 output positions reloads all 144 filter weights, producing $36 \times 144 = 5{,}184$ weight reads.

Systems insight: Under these two limiting assumptions, Ordering A reduces modeled weight reads by 36× relative to the worst case for Ordering B. Real kernels also account for activation and partial-sum traffic, register capacity, cache behavior, and vectorization. The example nevertheless shows why two mathematically equivalent loop orders can place very different demands on the memory system.

The three strategies form a decision framework rather than a quality ranking. The right choice is the operand with the highest reuse relative to the on-chip storage it would occupy, which depends on layer dimensions and batch size, so compilers select among them per layer. Convolution-specific designs add a fourth label, row-stationary. The Eyeriss accelerator maps one-dimensional rows of filter weights and input activations to processing-element registers and accumulates rows of partial sums locally, a hybrid that captures more of a convolution's multidimensional reuse than fixing any single operand (Y.-H. Chen et al. 2016).

11.7.5 Tiling

↩ Tiling in its software form, the cache-aware blocking a framework applies to matrix multiplication, is established in section 7.5.4.2.



Exceeding the tile width along a single dimension tips utilization along that axis off a cliff.

A stationary dataflow decides which operand stays local; tiling decides how large a block of it local storage can hold. A fundamental mismatch exists between the *computational graph* (which sees a single 4,096 × 4,096 matrix multiplication) and the *physical silicon* (which possesses a fixed 128 × 128 systolic array). Bridging this gap requires **tiling**, which partitions a large tensor operation into tiles that fit exactly into the hardware's fast local memory (SRAM or scratchpad memory), so that each tile is reused many times before it is evicted.

To process the 4,096-wide worked-example layer on a 128-wide systolic array, the compiler decomposes the result into 1,024 output tiles. Each output tile accumulates across 32 reduction-dimension tiles, so the complete matrix multiplication executes 32,768 **A**-tile-by-**B**-tile products. This decomposition is a physical requirement imposed by memory hierarchy limits: operand tiles are fetched from high-capacity, high-latency HBM, staged in fast SRAM, and pulsed through the systolic array.

Tile dimensions balance on-chip capacity against arithmetic intensity. A larger tile reuses each loaded byte across more multiply-accumulate operations, raising operational intensity toward the compute-bound regime of the roofline model; the capacity ceiling is how much of **A**, **B**, and **C** fits simultaneously in fast on-chip memory. Tiling sustains high hardware efficiency (η_{hw}) by ensuring that for every byte loaded from main memory, the data is reused 128× within the systolic grid. When a layer dimension is not an exact multiple of the hardware tile size, the accelerator pays a *fringe penalty* in idle silicon. Consider a layer where only one output dimension expands from 128 to 129 while the other dimensions remain perfectly aligned to the 128-element tile. Processing that single extra column requires scheduling a second 128-wide tile across the systolic array, leaving 127 execution units idle during the fringe pass. Two tiles execute where one nearly sufficed, so the utilization factor along that dimension falls to $129/256$, or approximately 50 percent, creating the efficiency cliff illustrated in the margin figure. When multiple dimensions misalign simultaneously, these padding losses multiply across the tiled axes: a 129×129 output block partitioned into four 128×128 physical tiles yields a useful-area efficiency of $(129/256)^2$, or barely 25 percent. Algorithm 11.1 formalizes the resulting loop nest emitted by the compiler to orchestrate this on-chip staging and accumulation.

Algorithm 11.1 Tiled (blocked) matrix multiply

Require: $\mathbf{A} \in \mathbb{R}^{M \times K}$, $\mathbf{B} \in \mathbb{R}^{K \times N}$; tile sizes $M_{\text{tile}}, N_{\text{tile}}, K_{\text{tile}}$
Ensure: $\mathbf{C} = \mathbf{AB}$

```
for each row tile i_0 (size M_tile) do
    for each column tile j_0 (size N_tile) do
        initialize the C-tile accumulator to zero on chip
        for each k_0 over K in steps of K_tile do
            load A-tile [i_0, k_0] and B-tile [k_0, j_0] on chip
            accumulate the tile product on chip ▷ reuse N_tile/M_tile per byte
        end for
        write the C-tile back to memory
    end for
end for
```

Once operand tiles reside in fast local SRAM, the hardware executes the tile product without further off-chip memory traffic. Systolic arrays achieve computational efficiency by routing operands through a structured 2D grid of processing elements (PEs) that communicate directly with adjacent neighbors across dedicated registers, bypassing shared-memory bus contention. In a weight-stationary design, the canonical TPU arrangement of figure 11.5, weights from the **B**-tile are preloaded into local PE registers and held stationary. Input activations from the **A**-tile stream horizontally across rows from left to right, while partial sums accumulate vertically down columns. On each clock cycle, every active PE multiplies its incoming activation by its stationary weight, adds the result to the incoming partial sum from above, and passes the activation rightward to its adjacent neighbor for the next cycle.

Because operands propagate one hop per cycle, synchronized execution requires an intentional time skew. Consider a minimal 2×2 tile product where output element $C_{ij} = A_{i0}B_{0j} + A_{i1}B_{1j}$ requires two products and one accumulation. The weight matrix **B** is held stationary across four PEs: PE_{00} stores B_{00}, PE_{01} stores B_{01}, PE_{10} stores B_{10}, and PE_{11} stores B_{11}. Activation A_{i0} enters the top row and A_{i1} the second row, because each row holds the weights for one reduction index. If A_{00} enters the top row at PE_{00} in cycle 0, its partial product $A_{00}B_{00}$ moves down to PE_{10} in cycle 1. That PE holds B_{10}, so the activation it needs is A_{01}, which must therefore enter the second row at PE_{10} one cycle later than A_{00} entered the first. In cycle 1, PE_{10} adds $A_{01}B_{10}$ to the arriving partial sum and emits C_{00}. Meanwhile A_{00} has moved right to PE_{01}, and its product $A_{00}B_{01}$ reaches PE_{11} in cycle 2, just as A_{01} arrives there from PE_{10}, so C_{01} emerges one cycle after C_{00}. Inputs along each successive row and column are staggered in the same way, so operands meeting at any physical coordinate represent identical reduction indices. Local reuse amortizes the initial fill and drain overhead: each loaded activation is reused across an entire row of N_{tile} processing elements, while each stationary weight serves an entire sequence of input vectors before eviction. Sustained throughput, however,

remains bounded by off-chip memory bandwidth whenever operand replenishment cannot keep pace with array retirement (section 11.5.1).

11.7.5.1 Tiling across the memory hierarchy

The systolic case is one instance of a general rule. Tiling bounds the operand blocks actively reused at each memory tier, and the partitioning that fits a graph-level matrix onto a fixed array, with its fringe penalty and the loop nest of algorithm 11.1, recurs at every tier of the memory hierarchy. Tiling[22] restructures a computation into blocks that fit within fast local memory (caches, shared memory, or registers), so that each block is reused before eviction rather than refetched from DRAM (Lam et al. 1991).

[22] **Tiling (loop blocking)**: This restructuring partitions a computation into blocks sized for a fast local tier. A naive loop makes repeated references to the same elements; caches may capture some, while blocking deliberately concentrates reuse before eviction (Lam et al. 1991). This reduction in lower-tier traffic is a primary source of the gap between naive matrix multiplication and optimized GEMM routines.

The naïve loop nest of listing 11.14 shows the cost of leaving reuse to chance. Written as $\mathbf{C} = \mathbf{AB}$, with the weights as **A** and the input batch as **B**, it references the same elements of both operands again and again, and without tiling those repeated accesses consume unnecessary bandwidth. Some references can hit in cache, but large matrices and an unfavorable traversal order can still cause excessive lower-tier traffic. Tiling partitions this multiplication across its three principal dimensions (M, K, and N) (figure 11.12). Each highlighted region represents the working set staged in fast memory at one moment: an $M_{\text{tile}} \times K_{\text{tile}}$ block of **A** (within the green row band) multiplies a $K_{\text{tile}} \times N_{\text{tile}}$ block of **B** (within the pink column band) to accumulate one $M_{\text{tile}} \times N_{\text{tile}}$ output tile ($\text{Block}_{m,n}$, shaded blue) of **C**. Processing all inner-product operations for an active tile before evicting it amortizes off-chip DRAM latency across $M_{\text{tile}} \cdot N_{\text{tile}} \cdot K_{\text{tile}}$ arithmetic operations.

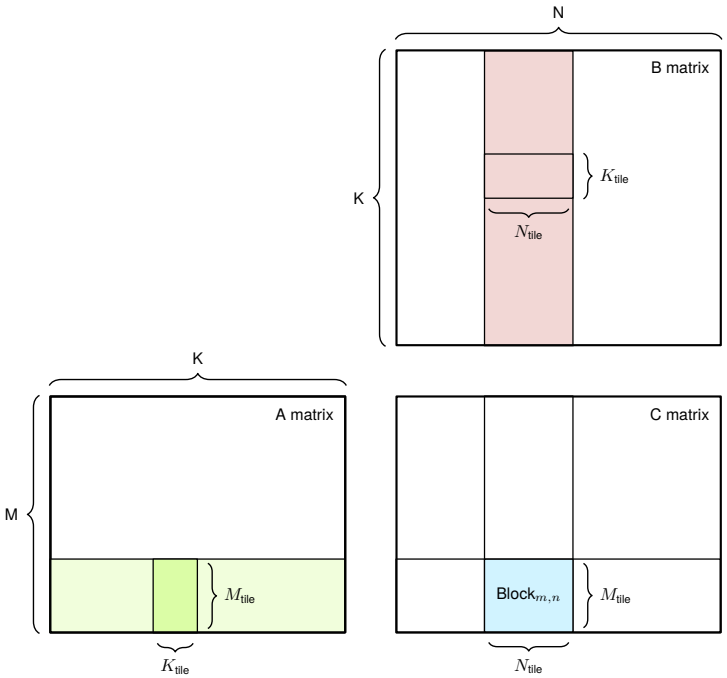


Figure 11.12: Matrix Tiling Mechanics: Partitioning large matrix multiplication ($\mathbf{C} = \mathbf{AB}$) into $M_{\text{tile}} \times K_{\text{tile}}$ and $K_{\text{tile}} \times N_{\text{tile}}$ blocks aligns memory footprints with fast on-chip SRAM/registers. Maximizing register reuse within each tile amortizes slow off-chip DRAM accesses and keeps compute engines operating at peak throughput.

Tiling fundamentals In listing 11.14, every iteration of the innermost loop fetches A_{ik} and B_{kj}, multiplies them, and updates C_{ij}. When the matrices are large, the processor reloads values it used moments earlier, and each reload that misses on chip pays the DRAM penalty in latency and energy (Horowitz 2014; Sze et al. 2017).

Performance benefits of tiling Tiling processes one submatrix at a time, so that once a block is loaded it is reused many times before the loop moves on. Listing 11.16 demonstrates cache-friendly

loop blocking: the loop bounds partition the matrices into tiles, and the hardware cache hierarchy keeps recently used tile data close to the compute units when access order has locality.

Listing 11.16: Cache-Blocked Matrix Multiplication: This high-level loop blocking approach divides matrices into smaller index ranges so hardware caches can reuse data within each tile, improving computational efficiency without explicit scratchpad loads.

```
TILE_SIZE = 32  # Choose a tile size based on hardware constraints

# Cache blocking: partition data via loop bounds.
# Loads are implicit through the hardware cache hierarchy.
for i in range(0, N, TILE_SIZE):
    for j in range(0, N, TILE_SIZE):
        for k in range(0, N, TILE_SIZE):
            # Each tile computed independently
            for ii in range(i, min(i + TILE_SIZE, N)):
                for jj in range(j, min(j + TILE_SIZE, N)):
                    for kk in range(k, min(k + TILE_SIZE, N)):
                        C[ii, jj] += A[ii, kk] * B[kk, jj]
```

The restructuring produces a chain of effects. A small tile is revisited while it is still likely to be cached, so most operand references hit in L1/L2 or shared memory instead of DRAM. DRAM traffic falls as a result, and the processing units spend less time waiting for operands. When an untiled access pattern repeatedly fetches operands from lower memory tiers, that traffic can impose a bandwidth ceiling below the arithmetic ceiling.

Tiling methods Tiling implementations differ along one axis, which is whether the hardware or the program decides what stays in fast memory. The two ends are implicit (cache) blocking and explicit (scratchpad) staging, and practical kernels often combine them.

Implicit (cache) blocking restructures only the loop bounds. The tiled matrix multiplication in listing 11.16 issues no explicit loads; its tile-shaped access pattern lets hardware caches reuse nearby values before they are evicted. It suits CPUs and GPUs with capable caches and large tensors that exceed fast-memory capacity.

Explicit (scratchpad) staging copies each tile into shared memory, a scratchpad, or registers and reorganizes the computation around the staged copy. It is the only option on accelerators whose fast memory is software managed, and it guarantees reuse that a cache might lose to eviction. Convolution is the classic case, because staging a filter once lets it serve many input regions without being refetched. Listing 11.17 loads blocks of **A** and **B** into temporary fast storage and reuses them across the inner loops, including shortened boundary tiles when N is not divisible by `TILE_SIZE` (the software counterpart of the fringe penalty that a fixed-size array pays in idle processing elements).

Listing 11.17: Explicit Tile Staging Pseudocode: A backend maps the temporary tile objects to fast storage and reuses them across inner-loop operations.

```
# Explicit tile staging: load data into fast
# temporary storage before the inner loops.
for i in range(0, N, TILE_SIZE):
    for j in range(0, N, TILE_SIZE):
        for k in range(0, N, TILE_SIZE):
            # Pseudocode: map these tile objects to fast memory
            A_tile = A[i : i + TILE_SIZE, k : k + TILE_SIZE]
            B_tile = B[k : k + TILE_SIZE, j : j + TILE_SIZE]

            # Reuse loaded tiles for all inner iterations
            for ii in range(A_tile.shape[0]):
                for jj in range(B_tile.shape[1]):
                    for kk in range(A_tile.shape[1]):
                        C[i + ii, j + jj] += (
                            A_tile[ii, kk] * B_tile[kk, jj]
                        )
```

Tiling challenges and trade-offs Tiling improves performance only when the tile matches the locality budget of the hardware. If the tile is too small, memory fetches still dominate execution time because reuse is too limited. If the tile is too large, it exceeds fast memory and causes cache thrashing or scratchpad spills. Selecting the right tile size therefore directly determines computational efficiency and memory bandwidth usage.

The capacity test counts every buffer that is live at the same time. For the tile dimensions in algorithm 11.1, the operand blocks contain $M_{\text{tile}}K_{\text{tile}}$ and $K_{\text{tile}}N_{\text{tile}}$ elements, while the output accumulator contains $M_{\text{tile}}N_{\text{tile}}$ elements. Suppose a hypothetical accelerator retains all three in one software-managed local store, uses FP16 operands and an FP32 accumulator, and loads only the current operand pair. Element storage size s_{elem} is then 2 bytes for each operand and 4 bytes for each accumulator entry. After reserving space for other live state, the tile must satisfy

$$(2\text{ bytes})M_{\text{tile}}K_{\text{tile}} + (2\text{ bytes})K_{\text{tile}}N_{\text{tile}} + (4\text{ bytes})M_{\text{tile}}N_{\text{tile}} \leq \text{available local bytes}. \tag{11.6}$$

Keeping the output dimensions fixed while doubling the reduction tile doubles both operand buffers without changing the accumulator. Reject that larger candidate if the revised left-hand side of equation 11.6 exceeds the available capacity, even if its longer reduction amortizes accumulator handling over more products. Double buffering has the same operand-capacity effect at fixed tile dimensions: retaining the next **A** and **B** tiles while computing the current pair requires a second copy of each operand buffer, while the running accumulator remains single buffered.

Actual storage tiers must be budgeted separately. A GPU kernel may stage operand tiles in shared memory while keeping output accumulators in thread registers; unused register capacity cannot make an oversized shared-memory allocation fit. Its shared-memory test counts the staged operand copies and any other shared state, while its register test counts accumulators, operand fragments, and temporary values across the participating threads. Passing both tests establishes capacity feasibility. It does not establish speed: alignment, bank conflicts, occupancy, synchronization, and measured traffic still determine whether the tile improves execution.

The tile choice also controls load balance. In architectures such as GPUs and TPUs, computations execute in parallel across thousands of processing units. If tiles are not evenly distributed, some units remain idle while others are overloaded, leading to suboptimal utilization of computational resources. Effective tile scheduling keeps parallel execution balanced and efficient.

Data movement remains a cost after tiling, even when another resource becomes the throughput limit. Although tiling reduces the number of slow memory accesses, transferring tiles between hierarchy levels still incurs latency and energy cost, especially when data falls from cache or scratchpad back to DRAM. Efficient memory prefetching and scheduling strategies minimize this residual movement and ensure that data is available when needed.

Hybrid tiling combines implicit blocking and explicit staging when neither alone captures the workload's reuse, for example relying on caches for an outer tier while staging convolution weights explicitly in a scratchpad. Register blocking, double buffering, and hierarchical tiling extend the same locality principle at smaller and larger memory tiers. AI compilers and runtime systems such as TensorFlow XLA, TVM, and MLIR automatically select these tiling strategies based on hardware constraints, enabling fine-tuned performance optimization without manual intervention. Table 11.12 compares implicit, explicit, and hybrid tiling.

Table 11.12: Tiling Strategies: Implicit, explicit, and hybrid tiling optimize memory access patterns for improved performance. Implicit (cache) blocking shapes loop bounds and lets caches decide residency, explicit (scratchpad) staging copies tiles into software-managed memory to guarantee reuse, and hybrid tiling combines both across memory tiers. AI compilers and runtime systems use these techniques to automatically optimize model execution on diverse hardware.

Aspect	**Implicit (Cache) Blocking**	**Explicit (Scratchpad) Staging**	**Hybrid tiling**
Who Decides Residency	Hardware cache replacement policy	The program, through explicit copies into software-managed memory	Caches for some tiers, explicit copies for others
Optimization Focus	Loop bounds shaped so accesses stay within cache-sized blocks	Staging tiles and reordering computation around the staged copy	Matching each memory tier to the mechanism it supports

Aspect	Implicit (Cache) Blocking	Explicit (Scratchpad) Staging	Hybrid tiling
Memory usage	Improves cache locality and reduces DRAM access	Keeps frequently used data in fast memory for multiple iterations	Minimizes data movement while ensuring high reuse
Common use cases	Matrix multiplications, CNNs, self-attention in transformers	Convolutions, recurrent neural networks (RNNs), iterative computations	AI accelerators with hierarchical memory, mixed workloads
Performance gains	Reduced memory bandwidth requirements, better cache utilization	Lower memory fetch latency, improved data locality	Maximized efficiency across multiple hardware types
Challenges	Reuse is not guaranteed; eviction can discard a tile before it is reused	Can increase register pressure, requires loop restructuring	Complexity in tuning tile size and execution order dynamically
Best when	Hardware caches are capable and the working set exceeds fast memory	Fast memory is software managed or reuse must be guaranteed	Different memory tiers favor different mechanisms

11.7.6 Memory-efficient tensor layouts

Stationary dataflows (section 11.7.4) and tiling determine which operands stay close to compute units and in what blocks; tensor layouts determine whether those operands can be accessed efficiently once they arrive. A well-chosen weight-stationary schedule still stalls if weights are laid out in a format that produces strided, non-coalesced memory fetches. Tensor layout is a kernel requirement. The physical ordering of multidimensional data in linear memory must match the memory access stride expected by the accelerator's execution units, or the processor pays in memory stalls, cache underutilization, and redundant bus transactions.

Hardware accelerators fetch data from device memory in fixed-size transactions, typically 32 to 128 bytes per transaction. When compute units request non-contiguous addresses, each transaction delivers only a fraction of useful payload, degrading effective memory bandwidth. Aligning tensor layout with the execution model lets concurrent threads or SIMD lanes access nearby aligned locations in fewer transactions; the count depends on access width and device granularity (NVIDIA Corporation 2021b).

Machine learning frameworks and compilers often manage this layout selection automatically. Optimization tools such as cuDNN for NVIDIA GPUs, XLA for TensorFlow and JAX, and MLIR-based compiler stacks transform tensor layouts as they lower operations to backend-specific kernels (NVIDIA Corporation 2021b; Google 2025; Lattner et al. 2020). In high-level frameworks, layout transformations are typically applied transparently, but developers authoring custom kernels or targeting low-level APIs such as CUDA, Metal, or OpenCL must manage tensor formats directly.

In PyTorch, for example, view operations such as `tensor.permute()` adjust tensor strides without moving data in memory, requiring an explicit `tensor.contiguous()` call to materialize a packed buffer when a downstream kernel requires contiguous storage (PyTorch Contributors 2026b). Conversely, TensorFlow defaults to channels-last (NHWC) for convolutional models on NVIDIA GPUs, warning that NCHW inputs introduce automatic transpose overhead when lowered to GPU kernels (TensorFlow Developers 2024b). Hardware-optimized libraries such as cuDNN on GPUs and oneDNN on CPUs implement tailored memory formats to maximize cache line utilization and SIMD throughput. The governing principle is to treat memory layout as an intrinsic property of the backend execution path: the fastest tensor format is the one that minimizes layout conversion overhead and makes memory accesses contiguous across hardware execution units.

11.7.6.1 Channels-last layout

In an NHWC channels-last layout, the channel index varies fastest in a contiguous tensor, placing the channel values for each spatial coordinate adjacent in memory. The label names the order in which dimensions are laid out in memory, from slowest-varying to fastest-varying, so NCHW and NHWC hold the same values in different linear orders.

Consider an RGB image tensor of shape $(\text{Height}, \text{Width}, \text{Channels}) = (3,3,3)$. The elements $X(h,w,c)$ are laid out in linear memory as follows:

$$X(0,0,0), X(0,0,1), X(0,0,2), X(0,1,0), X(0,1,1),$$
$$X(0,1,2), X(0,2,0), X(0,2,1), X(0,2,2), ...$$

All channel values for a given pixel are adjacent, and memory addresses advance across width before advancing down height. This layout supports vectorized execution when a compute kernel processes multiple channels simultaneously: CPU SIMD registers and GPU Tensor Cores can load contiguous channel vectors directly without intra-register shuffling.

The advantage of channels-last storage depends on the target operator and hardware unit. TensorFlow uses NHWC by default, whereas PyTorch provides a dedicated channels-last memory format that decouples logical dimension ordering from physical storage. Modern Tensor Core kernels in cuDNN favor NHWC because matrix-multiply units pack channel dimensions directly into accumulator tiles. When kernels require channel-contiguous inputs, passing an incompatible format forces the framework runtime to insert an explicit memory transpose, incurring a memory bandwidth penalty before computation begins.

11.7.6.2 Channels-first layout

In a contiguous NCHW channels-first layout, the spatial width index varies fastest, and all spatial values across an entire channel are stored contiguously before the next channel begins. Accelerators process data in parallel across thread warps or SIMD lanes; when consecutive threads in a warp access consecutive memory addresses, the memory controller coalesces those requests into fewer memory transactions (**memory coalescing**).[23]

[23] **Memory coalescing**: The GPU hardware mechanism that combines memory requests from threads in a warp into transactions according to address locality, alignment, access width, and device granularity. Tensor layout therefore decides how many transactions a warp's loads become, and whether a given layout helps depends on how the kernel assigns threads to tensor dimensions.

Consider the same RGB image tensor with dimensions reordered in channels-first order as $(\text{Channels}, \text{Height}, \text{Width}) = (3,3,3)$. Retaining $X(h,w,c)$ as semantic coordinate notation, the data are stored in channels-first order as follows:

$$X(0,0,0), X(0,1,0), X(0,2,0), X(1,0,0), X(1,1,0), X(1,2,0), ...,$$
$$X(0,0,1), X(0,1,1), X(0,2,1), ..., X(0,0,2), X(0,1,2), X(0,2,2), ...$$

In this format, all red channel values for the entire image are stored first, followed by all green values, and then all blue values. This arrangement allows kernels that parallelize across spatial coordinates (H,W) to issue coalesced reads along image rows, which suited earlier GPU convolution algorithms and SIMD execution pipelines (Chetlur et al. 2014).

The relative advantage of NCHW versus NHWC depends on how the underlying kernel maps computation to hardware threads. Convolutional layers evaluate filters across spatial and channel dimensions simultaneously. In classic direct or FFT-based convolutions, channels-first ordering enabled efficient spatial sliding without stride jumps. However, modern GEMM-lowered convolutions and Tensor Core pipelines map channel reduction directly to matrix multiply accumulators, favoring channels-last structures. When a model operates in NCHW but invokes a Tensor Core-optimized backend, the runtime must either run an unoptimized fallback kernel or dynamically transpose the tensor into NHWC, burning memory bandwidth in both cases.[24]

[24] **NHWC vs. NCHW**: NHWC lists dimensions as batch, height, width, channel; NCHW lists batch, channel, height, width. Physical memory format determines whether adjacent threads read adjacent addresses, so the performance effect is backend-specific. Layout-to-hardware mismatch is not a micro-optimization; it can create multi-fold performance gaps, especially when a conversion prevents Tensor Core convolution or fusion paths from being used.

11.7.6.3 Comparing channels-last and channels-first layouts

Both channels-last (NHWC) and channels-first (NCHW) layouts address specific hardware access patterns, with their efficiency determined by whether a kernel parallelizes over spatial coordinates or vectorizes across channel dimensions. Layout choice directly governs cache line utilization, memory bus transaction count, and compute unit occupancy. Table 11.13 contrasts the performance trade-offs and hardware compatibility between these two approaches.

Table 11.13: Data Layout Strategies: Channels-last (NHWC) and channels-first (NCHW) layouts optimize memory access patterns for different backend kernels. NHWC often suits CPU vectorization and modern Tensor Core convolution and fusion paths, while NCHW remains the default PyTorch tensor shape and suits older FP32 GPU kernel paths. Choosing the appropriate layout directly impacts performance by maximizing cache utilization and memory bandwidth efficiency.

Feature	**Channels-Last (NHWC)**	**Channels-First (NCHW)**
Memory storage order	Pixels are stored row-by-row, channel interleaved	All values for a given channel are stored together first
Best for	CPU loops, element-wise operations, many channels-last kernels	Many legacy GPU convolution paths and channel-first model code
Cache efficiency	High cache locality for sequential row/channel-last access	Can improve coalescing for channel-first kernels
Convolution performance	Often preferred by modern Tensor Core convolution/fusion paths	Efficient for older FP32 cuDNN paths and direct convolutions
Memory fetching	Good when kernels vectorize over adjacent channel data	Good when kernels process channel-major tiles
Default in frameworks	Common TensorFlow convention; PyTorch channels-last option	Common PyTorch tensor shape convention

While compilers such as XLA, TensorRT, and PyTorch Inductor automate layout propagation and insert layout conversions when beneficial, format conversions are not free: transposing a tensor requires reading every element from memory and writing it back in a different order, consuming memory bandwidth without executing useful arithmetic. In production deployments, matching the model's memory layout to the accelerator's native kernel format eliminates these layout transformations entirely. Even when tensor layout is perfectly aligned with the hardware, however, executing operations as isolated kernels introduces a separate bottleneck: intermediate values must be exposed through global-memory buffers between operations, even when caches absorb some physical traffic. Resolving that bottleneck requires changing how operations are scheduled rather than how tensors are laid out.

11.7.7 Kernel fusion on the accelerator

Kernel fusion[25] entered the book as a framework optimization that combines successive operations into a single kernel to avoid intermediate memory round trips. The LayerNorm analysis (notebook 11.8) shows why it matters on an accelerator. At 2.0 FLOP/byte, a standalone normalization kernel can reach at most 6.7 TFLOP/s, about 0.7 percent of the H100's tensor peak, however its arithmetic is scheduled. The hardware questions are which intermediates fusion removes from the memory bus and what the fused kernel costs in registers and occupancy.

The case for fusion rests on a traffic count. Run as separate kernels, a chain of three elementwise operations writes each intermediate result to device memory and reads it back for the next operation, whereas a fused kernel reads the input once, keeps the intermediates in registers or on-chip SRAM, and writes only the final output, which cuts idealized external traffic by roughly $3\times$. On an accelerator the saving matters most for elementwise operators, whose arithmetic intensity sits far below one FLOP per byte, so moving each intermediate across the memory bus takes longer than computing it (Jia et al. 2019). Fused execution also shortens the lifetime of intermediate buffers, although an optimizing runtime can often reuse those buffers anyway; what fusion guarantees is the removed write and read for each eliminated intermediate, and how much of that removed traffic would have reached DRAM depends on cache behavior.

11.7.7.1 Performance benefits and constraints

Because high-bandwidth memory is the scarce resource on GPUs and TPUs, eliminating redundant memory transactions translates directly into higher compute utilization (NVIDIA Corporation 2020a). Not all operations can be fused arbitrarily, however. Element-wise operations and frozen inference normalization are strong candidates because their computations do not require batch-wide reductions, whereas training-mode batch normalization computes batch statistics and is not purely element-wise. Matrix multiplications and convolutions constrain fusion further because they involve reductions and large data movement; compilers routinely fuse them with element-wise epilogues such as bias or activation, yet cannot freely merge them with unrelated global operations.

[25] **Kernel fusion**: Separate GPU kernels expose intermediate results through device memory, although caches can absorb some traffic. A fused kernel can keep intermediates in registers or local storage and avoid corresponding external write/read cycles. The resulting speedup depends on cache behavior, occupancy, launch overhead, and whether memory traffic was the bottleneck; it is not necessarily proportional to the byte reduction.

↩ Kernel fusion (also called operator fusion), with its LayerNorm, dropout, and ReLU example and the traffic count for unfused versus fused chains, is established in section 7.3.2.

Even when two operators are algebraically compatible, register pressure bounds how aggressively they should be fused. Keeping intermediate values in registers eliminates redundant memory writes, but if a fused kernel exceeds the available registers per thread, the compiler spills excess values to per-thread local memory backed by device DRAM, adding latency that can negate the benefits of fusion. On GPUs, where thread occupancy (the number of threads active in parallel) is bounded by the register file, excessive fusion can also starve parallelism. Different accelerators and compilers therefore strike different balances: NVIDIA GPUs favor warp-level parallelism where element-wise epilogue fusion is straightforward (NVIDIA Corporation 2020a), while TPUs prioritize systolic-array execution coupled to vector post-processing (Jouppi et al. 2017), and compiler stacks such as TVM, XLA, TensorRT, and MLIR use graph rewrites and engine-building heuristics to weigh memory savings against register and occupancy limits (Chen et al. 2018; Google 2025; NVIDIA 2024c; Lattner et al. 2020). When debugging performance regressions or modifying models frequently, frameworks also allow developers to disable fusion selectively so that memory savings, register usage, and hardware execution constraints can be evaluated independently.

Checkpoint 11.4: Data movement and kernel fusion

The dataflow building blocks are now in place: data locality, tensor layout, and kernel fusion. These fusion decisions are ultimately about data locality, which ties together the chapter's core data movement strategies:

- ☐ **Locality choice**: given weight-stationary, output-stationary, and input-stationary dataflows, which tensor does each hold near the compute units, and how does that choice shift where the memory traffic lands?
- ☐ **Layout match**: a kernel runs on a backend expecting NHWC but receives an NCHW tensor. Predict what happens to its memory access pattern and to delivered throughput.
- ☐ **Fusion eligibility**: for a Conv2D, a BatchNorm, and a ReLU executed in sequence, decide which pairs are worth fusing and what determines whether fusing them pays off.
- ☐ **Remaining gap**: an operation still repeatedly fetches operands because its active blocks exceed fast-memory capacity. How could tiling from section 11.7.5 change this access pattern, and what capacity and reuse evidence would justify that choice?

11.7.8 Applying mapping strategies to neural networks

While section 11.5.5 diagnosed how convolutional networks, transformers, and multilayer perceptrons stress memory subsystems, the mapping strategies above provide the systems remedies. Mapping is a systematic search over loop permutations, tiling factors, and stationarity policies that maximizes data reuse under hard SRAM and register constraints. Because each model family presents a distinct ratio of weights to activations and differing opportunities for temporal locality, optimal mapping policies diverge across workloads. Convolutional networks exploit 2D spatial weight reuse across receptive fields; transformers split between compute-bound matrix multiplications during prefill and bandwidth-bound KV-cache streaming during autoregressive decoding; and dense MLPs demand blocked GEMM execution across batch dimensions. Table 11.14 consolidates how dataflow, memory layout, kernel fusion, and tiling align with these three architectural archetypes.

11.7.8.1 Convolutional neural networks (ResNet-50)

For ResNet-50 and similar CNNs, a key mapping opportunity is spatial weight reuse. A small filter is applied across many spatial locations, so its weights participate in many multiply-accumulate operations. Weight-stationary mappings can capture that reuse by retaining filter tiles locally, while row-stationary or output-stationary mappings may better balance activation and partial-sum traffic on other architectures. Favorable convolution shapes can achieve high arithmetic intensity, which makes them well suited to matrix and systolic execution paths. The model family does not determine one universal dataflow: layer shape, batch size, local-storage capacity, and the target's supported kernels decide which reuse pattern is worth retaining.

Table 11.14: Memory Pressure and Mapping by Model Family: The first four rows profile each family's memory demands, and the remaining rows give the mapping that answers them. Standard dense transformers demand bandwidth and capacity because large weights, KV caches, and attention traffic dominate their traffic, while CNNs benefit from spatial locality and high activation reuse.

Property or technique	CNNs	Transformers	MLPs	Rationale
Weight size	Small, reused	Large, usually dense; sparse in MoE or pruned variants	Large, dense	Sets whether weights can stay on chip or must stream from HBM on every use.
Activation reuse	High	Low to medium	Low	Sets how much each loaded activation is worth keeping in local storage.
Memory access pattern	Spatial locality	Mostly regular GEMM plus KV-cache and attention traffic	Regular, sequential (streamed)	Regular patterns tile and prefetch well; KV-cache reads grow with context length.
Potential memory pressure	Feature-map movement	Memory capacity and bandwidth	Bandwidth at low batch	Starting hypotheses that execution phase, batch size, and data movement can change, not fixed bottleneck classifications.
Dataflow strategy	Weight, row, or output stationary	Phase- and kernel-dependent	Weight or output stationary	CNNs expose filter and spatial reuse; transformer attention and MLP blocks impose different traffic; dense MLPs reuse weights across a batch and accumulate outputs.
Memory-aware tensor layouts	Backend-dependent (channels-last on Tensor Core paths, NCHW on older FP32 paths)	Backend-dependent (row-major activations typical)	Row-major typical	Layout choice depends on backend kernel path and precision mode (section 11.7.6 compares the layouts); the entries name common defaults, not universal prescriptions.
Kernel Fusion	Convolution + Activation	Fused Attention	GEMM Fusion	CNNs optimize convolution+activation fusion; Transformers fuse attention mechanisms; MLPs benefit from fused matrix multiplications.
Tiling for Memory Efficiency	Explicit staging of feature-map tiles	Explicit staging of attention blocks	Blocked GEMM	CNNs stage feature-map subregions in on-chip SRAM; transformers stage query and key/value blocks in fused attention; MLPs block large matrix multiplications, implicitly or explicitly.

This spatial regularity also enables aggressive fusion and tiling. For inference with frozen BatchNorm statistics, compilers can fold or fuse convolution, normalization, and activation to avoid unnecessary intermediate writes. Training-mode BatchNorm requires reductions over batch statistics and does not have the same element-wise structure that permits fusion. Spatial tiling partitions the feature map into subregions sized to fit within on-chip SRAM, so the fused kernel processes each tile from fast memory before moving to the next.

11.7.8.2 Transformer architectures (GPT-2/Llama)

Transformer mapping depends on execution phase. During training and prefill, attention creates large intermediate matrices unless a tiled implementation avoids materializing them; during autoregressive decoding, attention reads the growing key-value (KV) cache. Each new query vector must access the stored key and value pairs across the preceding sequence. As sequences grow, the full KV cache usually lives in HBM or device DRAM, while attention kernels tile active blocks through SRAM, registers, or shared memory. This access pattern motivates staging active K/V tiles in SRAM so attention consumes them from fast local storage. During training and prefill, the memory traffic created by standard attention implementations similarly explains why fused attention kernels, such as FlashAttention (Dao et al. 2022), can deliver large performance gains: they tile the query-key product, softmax, and value summation through on-chip SRAM so the full attention matrix never reaches HBM, while the $\mathcal{O}(S^2)$ computation is preserved (section 8.6.4 develops the algorithm).

11.7.8.3 Multilayer perceptrons and DLRM

MLPs present the most straightforward mapping problem because their computation reduces largely to dense GEMM.[26] Each fully connected layer multiplies an activation matrix by a weight matrix. The

[26] **GEMM**: The operation $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$, with scalars α and β, is the dense linear-algebra primitive that many deep-learning layers lower to. Optimized GEMM libraries such as cuBLAS and oneDNN use register blocking, vectorization, and hierarchical tiling to approach hardware limits on favorable shapes (NVIDIA 2024a; Intel Corporation 2021b). Modern AI accelerators are heavily specialized for GEMM-like tiles: Tensor Cores, systolic arrays, and matrix extensions all exist to accelerate this primitive, which is why GEMM performance is an important predictor of end-to-end throughput across architectures.

weight matrix is fixed across samples, so batching reuses each loaded weight across more activations and raises arithmetic intensity. A batch size of one provides little cross-sample weight reuse and often leaves wide matrix engines underused, while larger batches can move execution toward the compute-bound regime.

↪ How GEMM arithmetic intensity scales with matrix dimension is collected for reference in section C.1.3.

Because MLP layers are typically followed by activation functions and bias additions, GEMM fusion combines these steps into a single kernel, avoiding intermediate memory writes. Blocked tiling partitions the large matrix multiplications into sub-blocks sized for the accelerator's shared memory, ensuring high cache utilization throughout computation. The simplicity of the MLP mapping, dominated by a single primitive with predictable access patterns, is precisely why hardware vendors optimize GEMM libraries so aggressively: gains in GEMM performance translate directly to MLP throughput. DLRM is not purely an MLP workload, however. Its large sparse embedding tables and feature-interaction stage can dominate capacity and memory traffic, so the dense mapping described here applies to its bottom and top MLP towers rather than to the entire model (Naumov et al. 2019).

11.7.9 Hybrid mapping strategies

The preceding architectural subsections treat each architecture in isolation, but real models rarely consist of a single layer type. A vision transformer, for example, combines a patch embedding stage, self-attention layers, and MLP blocks (Dosovitskiy et al. 2021). Those layers create different reuse patterns: the embedding stage can benefit from weight-stationary mapping, attention emphasizes activation movement and tiling, and MLP blocks demand blocked GEMM tiling and fusion. No single dataflow strategy is optimal across all these layers, so hardware mapping becomes hybrid and layer-specific.

Hybrid mapping addresses this heterogeneity by allowing the accelerator to switch strategies at layer boundaries. Each layer presents a different balance of compute intensity, data reuse, and memory access pattern, and the optimal mapping must shift accordingly (Sze et al. 2017). Rather than committing to one dataflow for the entire model, hybrid approaches consider weight stationary execution when weights can be reused locally, input-stationary execution when activation reuse justifies its storage, and output stationary execution when partial-sum traffic is costly. Layer dimensions and available local memory determine which candidate pays off.

TPU-style systolic arrays, NVIDIA GPUs, and tile-based accelerator designs expose different combinations of local memory, tensor layouts, fusion, and scheduling controls, allowing compilers and runtimes to choose layer-specific strategies rather than one global dataflow for the whole model (Jouppi et al. 2023; NVIDIA Corporation 2020a; Chen et al. 2018). These implementations depend on programmable memory hierarchies, efficient interconnects, and specialized execution pipelines. However, hybrid mapping remains a design-time optimization: in production workloads, execution conditions change dynamically due to varying input sizes, memory contention, and hardware resource availability. Machine learning compilers and runtime systems extend these static mapping choices by introducing dynamic scheduling, memory optimizations, and automatic tuning, ensuring that deep learning workloads operate efficiently across diverse accelerators and deployment environments.

11.8 Compilers and Runtimes

The roofline gives each ResNet-50 layer a ceiling, and hardware mapping showed which schedules can approach it, but neither reaches the silicon by itself. Every layer needs a tiling, a stationary operand, a tensor layout, a precision for the activations it reads and writes, and a fusion boundary, and the resulting plan must then run on a device whose input shapes, free memory, and clock rate change while it serves. Across a whole model these choices number in the thousands, which is why compilers and runtimes, not engineers, make them. Graph capture and dispatch overhead, which decide how a model reaches a compiler, belong to chapter 7, and the choice of a deployment runtime for a service belongs to section 13.6. What remains here is lowering, the path from a captured graph to the accelerator's units, and the runtime that executes the result. The decode floors this chapter quotes for GPT-2 XL assume that the step is launched as a captured graph (section 7.3.6); executed one eager kernel at a time, launch overhead rather than memory bandwidth would set the floor.

A single convolution can be implemented with dozens of valid tiling strategies, kernel variants, and memory layouts, most of which perform poorly on a given target. **Machine learning com-**

pilers navigate this space by translating high-level dataflow representations into target-specific executable code, making the mapping and dataflow decisions of the preceding sections automatically. Compiling ResNet-50 for GPU inference illustrates the core compiler stages:

1. **Graph optimization** fuses repeated Conv2D-BatchNorm-ReLU patterns into single compound kernels, eliminating intermediate round-trips to HBM.
2. **Kernel selection** chooses specialized Tensor Core implementations for compatible matrix multiplications, exploiting the high arithmetic intensity of dense convolutions.
3. **Memory planning** determines activation lifetimes to safely overlap and reuse intermediate buffer allocations.
4. **Computation scheduling** pipelines memory copies with execution across independent hardware queues to hide data transfer latencies.

↩ Graph compilation, kernel fusion, and framework dispatch overhead are established in section 7.3.

On the reference H100, the modeled batch-one forward pass falls from 3.53 ms in an eager framework to 0.33 ms as a compiled FP16 engine, a 10.6× improvement without altering the model's mathematical output. The arithmetic per image is unchanged. The gain comes from fusing the network's 174 framework kernels into 56 and launching them as one graph instead of one Python dispatch at a time, which removes the fixed cost that dominates a single image. That answers the question the chapter opened with. The INT8 arithmetic for one image takes about 4 µs on the H100 (8 µs at FP16), so even the compiled pass spends almost all of its time on launch and overhead; batch-one latency sits on the overhead floor rather than on either roofline ceiling, which is why halving the arithmetic barely moves it. The latencies are modeled rather than measured, and section 13.6 separates the same gap into the contribution of each runtime, but the mechanism is general. Graph rewrites, kernel selection, memory planning, and scheduling systematically apply kernel fusion (section 11.7.7) and memory-efficient tiling (section 11.7.5.1) to convert theoretical hardware FLOP/s into delivered throughput (Chen et al. 2018; NVIDIA 2024c).

11.8.1 ML compiler design

Machine learning compilers specialize conventional compilation techniques for programs expressed as tensor graphs and operator libraries. General-purpose compilers optimize scalar control flow, vector instructions, register allocation, and cache locality; ML compiler stacks operate at a higher semantic level, optimizing graph topology, tensor memory layouts, shape specialization, accelerator kernel dispatch, and inter-device execution planning (Li et al. 2021). Table 11.15 contrasts the emphasis of these two compiler layers. The boundary is complementary rather than disjoint: an ML compiler stack lowers high-level tensor operations into intermediate representations that feed into an LLVM, GPU, or vendor-specific backend.

Table 11.15: Compiler Optimization Priorities: ML compiler stacks add tensor- and graph-level decisions to conventional instruction- and loop-level compilation. In a production toolchain, the two layers commonly cooperate rather than replace one another.

Aspect	**Traditional compiler**	**Machine learning compiler**
Input representation	Source or intermediate-representation program	Tensor graph or tensor-level intermediate representation
Execution Model	Control flow, loops, threads, vectors, and tasks	Tensor operators, kernels, graphs, and accelerator streams
Optimization priorities	Instruction selection, loop transforms, vectorization, registers	Graph rewrites, fusion, layouts, shapes, and kernel selection
Memory management	Objects, stack/heap, caches, locality, and prefetching	Tensor lifetimes, buffer reuse, layouts, tiling, and device transfers
Target Hardware	CPUs, GPUs, and other programmable targets	CPUs, GPUs, TPUs, and custom accelerators
Compilation output	Machine code or lower-level intermediate representation	Kernels plus a hardware-specific graph or execution plan

The distinction in table 11.15 explains why compiler configuration dictates delivered performance even when the underlying neural network code remains unmodified. ML compilers manage the translation layer between high-level tensor operations and hardware execution units; when this

mapping fails to expose reuse or efficient access, execution can be limited by memory traffic, latency, or insufficient parallel work.

11.8.2 ML compilation pipeline

Machine learning models defined in modern frameworks are represented as high-level dataflow graphs describing operations on multidimensional tensors. Lowering these representations into executable code for CPUs, GPUs, TPUs, or custom ASICs requires an ML compilation pipeline (Chen et al. 2018; Google 2025; Lattner et al. 2020). A standard compilation pipeline adds a fifth stage, code generation, to the four that the ResNet-50 example illustrated, emitting target-specific machine instructions, assembly, or interpreted bytecode. Across these stages, the compiler applies the dataflow strategies developed in section 11.7.3: kernel fusion, memory tiling, and compute placement. Hardware acceleration delivers its theoretical throughput only when these compiler-driven transformations expose sufficient arithmetic intensity and align memory access with the hardware hierarchy.

11.8.2.1 Graph optimization

Machine learning frameworks represent models as high-level computation graphs, where nodes represent mathematical operators (such as convolutions, matrix multiplications, and activations) and directed edges represent tensor data dependencies. Executing each operator as a separate kernel can incur intermediate-buffer traffic and repeated launches. An edge denotes a dependency, not necessarily a new allocation or an off-chip transfer, and asynchronous dependencies need not synchronize the host. When intermediate materialization and dispatch dominate execution, graph transformations can remove substantial overhead.

Graph optimization transforms this computation graph into an efficient execution plan before hardware code generation (Chen et al. 2018; Jia et al. 2019). Working primarily at a target-independent intermediate representation (IR) level before lowering (section 11.8.2.5), graph passes apply four core transformations. Kernel fusion merges adjacent operations into a single compound kernel to eliminate off-chip memory round-trips and launch overheads: in convolutional networks, fusing Conv2D, batch normalization with frozen inference statistics, and ReLU keeps activations in local registers or shared SRAM throughout the sequence (section 11.7.7), while in transformers, pattern matchers replace multi-operator self-attention subgraphs (score matrix multiplication, scaling, masking, softmax, and multiplication by the value matrix) with IO-aware attention kernels that keep active tiles and running reduction statistics in fast on-chip memory (Dao et al. 2022). Redundant computation elimination and constant folding remove work that need not happen at runtime at all, folding frozen normalization parameters into the weights and biases of the preceding linear or convolutional layer (section 10.4.1) while pruning dead subgraphs and merging identical parallel paths through common subexpression elimination.

The remaining two passes reduce data movement and live memory footprint without changing the operator set. Algebraic simplification and layout adjustments reorder mathematically equivalent operations—pushing transpositions through elementwise ops or canceling them against inverse permutations—to avoid runtime reshape traffic, while memory-aware node reordering evaluates topological sorts of the dependency graph and schedules independent operations so intermediate activations are consumed and deallocated promptly before memory pressure forces spills to host DRAM. Production compilers execute these rewrites through pattern-matching rules, producing a coarse-grained execution graph aligned with the accelerator's bandwidth constraints before binding each node to a concrete hardware routine.

11.8.2.2 Kernel selection

Kernel selection binds each node in the optimized graph to a concrete hardware implementation. Accelerators provide dozens of valid kernel implementations for a single tensor operation, each parameterized by tile dimensions, thread block configurations, memory staging patterns, and target precision. Choosing the optimal kernel determines whether the execution engine achieves high compute utilization or stalls on uncoalesced memory accesses and pipeline bubbles (Chen et al. 2018; Zheng et al. 2020).

In most production pipelines, compilers draw from vendor-optimized kernel libraries (NVIDIA's cuDNN and cuBLAS for GPUs, Intel's oneDNN for x86) rather than synthesizing kernels from

scratch. These libraries encode microarchitectural details such as register file capacity, cache latency, and vector lane width, so the compiler can invoke pre-tuned implementations.

AI compilers trade compilation time against execution speed across three strategies when choosing among candidate kernels, with numerical precision constraining all three. Rule-based selection is the fastest at compile time, relying on static heuristics over tensor shapes, alignment, data types, and target support to pick implementations such as a Tensor Core GEMM without benchmarking a large candidate set, at the risk of missing faster configurations on irregular shapes. Cost-model selection refines that choice without running hardware benchmarks by estimating kernel latency, register pressure, cache hit rates, and arithmetic intensity with analytical models that prune unpromising candidates (Lattner et al. 2020). When compilation budgets allow hardware measurement, profile-guided selection benchmarks candidates directly on the target device: autotuners such as TVM's AutoTVM and Ansor search over tile sizes, unrolling factors, and thread configurations to converge on the fastest implementation (Chen et al. 2018; Zheng et al. 2020), and TensorRT similarly times candidate tactics against the target batch sizes. Across all three strategies, precision-aware constraints govern which candidates are legal: inference can use lower-precision paths after quality validation, but selecting INT8 without appropriate range handling degrades accuracy, whereas selecting a wider path unnecessarily forgoes available throughput.

11.8.2.3 Memory planning

Memory planning assigns physical storage locations, buffer offsets, and tensor lifetimes across the accelerator's memory hierarchy (Roesch et al. 2018; Chen et al. 2018). High-performance execution plans fail if memory management introduces uncoalesced memory transactions, bank conflicts, or unnecessary memory allocations that exhaust accelerator memory, so the compiler coordinates three core memory management mechanisms.

Tensor layout optimization chooses the physical arrangement of tensor dimensions that each kernel needs for coalesced access and vectorization, such as channels-last `NHWC` for Tensor Core convolutions (section 11.7.6) (NVIDIA Corporation 2021b; Google 2025). Compilers perform layout assignment globally across the graph to prevent back-to-back layout transformation kernels that consume memory bandwidth.

Liveness analysis and buffer reuse prevent out-of-memory errors by sharing physical memory across non-overlapping tensors. Deep neural networks generate dozens of intermediate activation tensors during forward propagation. Allocating a unique memory buffer for every tensor would quickly exceed the capacity of device memory (such as 16 to 80 GB on typical accelerator GPUs). The compiler analyzes the graph to compute the liveness interval of each tensor, from its generation to the completion of its last consumer. By modeling memory allocation as an interval graph coloring or 2D bin-packing problem, the compiler assigns multiple non-overlapping intermediate tensors to the same physical memory offsets (Roesch et al. 2018). This static reuse reduces peak device memory requirements to the working set size of concurrently active tensors, avoiding runtime allocation overhead and fragmentation.

Memory hierarchy placement optimizes the staging of data between external memory (HBM or GDDR) and on-chip scratchpad memories (SRAM or shared memory). External memory bandwidth is a scarce resource; moving intermediate tiles back and forth across the memory bus creates memory-bound stalls. Compilers synthesize tiling parameters that guarantee intermediate blocks remain resident in on-chip SRAM until all local compute operations complete, minimizing round-trip traffic across the memory bus. With memory layouts fixed and buffer addresses assigned, the compiler schedules operation execution across hardware compute resources.

11.8.2.4 Computation scheduling

Computation scheduling determines the execution timeline and hardware resource binding for operations in the compiled graph (Zheng et al. 2020). Modern accelerators feature deep hardware pipelines, multiple independent compute engines, and dedicated DMA copy engines. Effective scheduling coordinates these resources across three distinct architectural dimensions to maximize hardware occupancy, preserve data dependencies, and prevent pipeline stalls (Chen et al. 2018).

Task partitioning maps computational subgraphs onto the target's physical execution units. On GPUs, computations are partitioned into grids of thread blocks dispatched across Streaming Multiprocessors (SMs). On TPUs, matrix multiplications are partitioned into tile sequences matched to

the physical dimensions of systolic arrays (e.g., 128×128 or 256×256 processing elements) (Norrie et al. 2021). On CPUs, tasks are split into thread pools matching physical cores and vectorized to fill SIMD registers.

Execution ordering resolves dependencies while minimizing live memory pressure. Operations that depend on prior results must wait for producer kernels to complete, but independent branches (such as multi-head attention projections or parallel residual paths) can execute concurrently or interleave to maintain uniform compute utilization. Compilers analyze the dependency graph to schedule operations in an order that keeps compute pipelines full while keeping activation lifetimes short, preventing buffer bloat.

Asynchronous overlap hides memory latency by executing data transfers concurrently with compute kernels. Accelerators feature dedicated DMA engines that operate independently of arithmetic pipelines. Compilers exploit this architectural separation by scheduling memory transfers (such as host-to-device weights or inter-accelerator halo exchanges) in separate hardware queues or CUDA streams, overlapping transfer latency behind long-running compute operations (Chen et al. 2018; NVIDIA 2024c). When compute and transfer overlap completely, data movement costs are hidden behind arithmetic execution. Poor scheduling instead introduces execution bubbles where arithmetic units sit idle waiting for kernel dispatches or serialized data copies, so synchronizing kernels via explicit compiler-generated events and stream schedules ensures the hardware sustains continuous execution without CPU driver intervention (NVIDIA 2024c; Zheng et al. 2020).

11.8.2.5 Code generation

With scheduling complete, the final compilation stage translates the scheduled execution plan into target-specific machine instructions. Code generation applies traditional backend compiler passes (instruction selection, register allocation, instruction scheduling, and peephole optimization) tailored to accelerator architectures.

Instruction selection for ML targets must emit instructions that directly engage specialized matrix ISA extensions. On NVIDIA GPUs, code generators emit PTX instructions such as `mma.sync.aligned` to invoke Tensor Cores directly, as shown in listing 11.13. On Intel CPUs with Advanced Matrix Extensions (AMX), the backend targets tile-multiply instructions operating on 2D register tiles. On Arm architectures with the Scalable Matrix Extension (SME), the target is outer-product accumulation across scalable vector tiles. A backend that emits generic floating-point vector instructions instead of these matrix extensions leaves the primary compute engines idle, reducing effective throughput by an order of magnitude. For CPUs and GPUs, compilers emit machine code or optimized assembly; for TPUs, field-programmable gate arrays (FPGAs),[27] and proprietary NPUs, the output is often an optimized bytecode or binary execution package executed by a specialized hardware runtime.

[27] **Field-programmable gate array**: "Field-programmable" means the logic fabric is configurable after manufacturing, contrasting with fixed-function ASICs. FPGAs can improve performance for latency-sensitive data center services by implementing custom pipelines matched to a particular workload (Putnam et al. 2014). This reconfigurability makes FPGAs attractive for rapidly evolving ML architectures where committing to an ASIC risks obsolescence, but the requirement for hardware description languages (Verilog/VHDL) and compilation times measured in hours creates a productivity barrier that limits adoption to deployments where the efficiency benefit justifies the engineering cost.

11.8.3 Runtime support

A compiler transforms a high-level model into an execution plan tailored to target hardware, but that plan embeds static assumptions about runtime conditions: tensor shapes, workspace allocation limits, device capabilities, and execution concurrency. Ahead-of-time (AOT) compilation optimizes graph structures, fusion boundaries, and memory layouts for fixed shapes and known hardware configurations. While just-in-time (JIT) compilation can compile specialized variants dynamically as new shapes arrive, each compiled kernel still executes a fixed execution plan. Production serving breaks those assumptions: request arrival rates fluctuate, batch sizes vary, several inference workloads compete for shared accelerator memory, and thermal throttling can pull clock frequencies below their rated values. Request batching and admission control belong to the serving layer (chapter 13); the concern here is how the accelerator runtime dispatches compiled execution plans once a batch reaches the hardware.

An AI runtime bridges static graph compilation and the dynamic realities of accelerator execution. Compilers optimize and lower computational graphs under fixed structural assumptions, but production systems encounter variable batch sizes, fluctuating sequence lengths, and constrained memory budgets. The runtime executes compiled engines while managing input-dependent tensor shapes, device-memory buffers and workspace, asynchronous hardware streams, and precompiled execution profiles. In production inference stacks such as NVIDIA TensorRT, the offline builder optimizes the network and benchmarks candidate tactics to construct an execution engine, while the

runtime engine binds input buffers, selects valid optimization profiles, and dispatches kernels to the accelerator (NVIDIA Corporation 2026b, 2026c).

This execution lifecycle spans three core responsibilities. First, the runtime selects and dispatches precompiled kernel variants matched to runtime tensor dimensions and hardware capabilities. Second, it manages device memory through arena allocators and static buffer reuse, reducing repeated allocation and associated host-device synchronization overhead. Third, in distributed environments, framework runtimes coordinate cross-device execution boundaries, as demonstrated in pipeline-parallel frameworks such as GPipe (Huang et al. 2019) and automated device-placement systems (Mirhoseini et al. 2017). While serving schedulers oversee request batching and admission control across clusters, the accelerator runtime governs local execution on the device.

11.8.3.1 ML runtime architecture

General-purpose runtimes manage threads, preemptive tasks, heap allocations, and asynchronous I/O across arbitrary control-flow paths. In contrast, an AI runtime operates over dataflow graphs composed of multidimensional tensors, pre-scheduled kernel grids, device memory arenas, and hardware command queues, as contrasted in table 11.16.

Table 11.16: Runtime Execution Models: AI runtimes add tensor-, graph-, and accelerator-aware resource management to conventional runtime services. The exact boundary among framework, compiler runtime, device library, and serving scheduler varies by stack.

Aspect	**Traditional runtime**	**AI runtime**
Execution Model	Functions, threads, tasks, events, and asynchronous I/O	Tensor graphs, kernels, streams, and device events
Task scheduling	Threads and tasks across processor resources	Dependency-aware kernel and transfer dispatch
Memory Management	Objects, stacks, heaps, pools, and virtual memory	Tensor buffers, workspaces, device memory, and buffer reuse
Optimization Priorities	Responsiveness, throughput, locality, and resource sharing	Shape compatibility, transfer overlap, reuse, and utilization
Adaptability	Dynamic scheduling and allocation within program semantics	Runtime choices within compiled graphs and available kernels
Target Hardware	CPUs and heterogeneous devices	CPUs, GPUs, TPUs, and custom accelerators

Tensor memory management represents a primary architectural divergence. In traditional systems, general-purpose allocators manage variable-sized heap objects using runtime metadata. Traditional device allocation calls such as `cudaMalloc` can introduce costly driver work and synchronization. AI runtimes reduce repeated allocation overhead by combining lifetime analysis with pooled or arena allocation. For fixed-shape graphs, the compiler computes static memory offsets, mapping non-overlapping intermediate tensors to shared physical buffers. For variable-shape tensors, the runtime can carve device-memory buffers from preallocated pools, avoiding repeated driver allocation calls within the reserved capacity.

Production environments introduce physical constraints that differ from offline benchmarks. For example, an NVIDIA A100 SXM accelerator operates with a maximum TDP of 400 W, whereas the A100 PCIe variant is constrained to a 300 W TDP envelope. Because sustained clock frequencies scale with available power and thermal dissipation, a compiled kernel tuned for an SXM module may experience thermal throttling or fail to meet latency bounds on a PCIe card. The runtime cannot re-synthesize loop structures or change thread-block dimensions dynamically in response to thermal constraints; it must execute within precompiled profiles while the serving system handles request batching and admission control.

11.8.3.2 Dynamic kernel execution

While static compilation generates high-performance machine code for fixed shapes, production inference requires executing requests with dynamic dimensions. In language models, input sequence length S and batch size B vary across requests. Recompiling an execution plan for each unexpected shape can violate a request deadline: exploring loop unrolling, shared-memory tiling, and register allocation requires seconds or minutes, while serving deadlines operate on millisecond timescales.

↪ How serving runtimes form batches of varying size under latency deadlines is developed in section 13.5.3.

Profile-based inference stacks address this mismatch with precompiled shape ranges. The engine is built for a declared range of each dynamic dimension (the minimum, typical, and maximum shapes), with device-memory workspace reserved for that range. A request inside the range binds its tensors to preallocated buffers and launches precompiled kernels without compilation overhead; a request outside it must be padded, split, or routed to a slower general-purpose kernel, because the runtime cannot synthesize new tiling strategies on the fly. TensorRT's optimization profiles are the common example, and choosing and configuring such a runtime for a service is developed in section 13.6.

Variable requests also keep the host link busy, and the runtime hides those transfers with the asynchronous streams and double buffering of section 11.5.4.2, copying batch $k+1$ while batch k executes. The overlap holds only while compute duration exceeds transfer duration.

11.8.3.3 Runtime kernel selection

An operation may have multiple kernel implementations, termed tactics in TensorRT. The builder selects tactics while constructing an engine; other stacks may retain variants for runtime dispatch. The available choice depends on tensor dimensions, the compiled plan, and permitted device-memory workspace.

Operational batch size changes the available reuse and parallelism. At $B = 1$ in weight-streaming autoregressive decoding, GEMV kernels often target memory throughput. Larger prompt-processing batches can favor GEMM kernels using Tensor Core paths, register tiles, and shared memory. The transition depends on dimensions, precision, traffic, and hardware balance; no fixed batch threshold establishes compute-bound execution.

Device-memory workspace availability further constrains tactic selection. A GEMM with a small output and a long reduction dimension K launches too few thread blocks to occupy every streaming multiprocessor. A split-K variant partitions the reduction across multiprocessors at the cost of a device-memory buffer for partial sums, so a plan built under a tight workspace limit may forgo it and accept lower occupancy.

Runtime selection remains strictly bounded by the numerical formats established during compilation. In mixed-precision training and inference systems such as Megatron-LM, executing matrix multiplications in FP16 or BF16 on Tensor Cores with FP32 accumulation is a numerical requirement of the chosen recipe, validated prior to deployment (Shoeybi et al. 2019). The runtime does not monitor loss values or dynamically alter precision; it only chooses among the validated tactics that implement the specified numerical precision.

11.8.3.4 Kernel scheduling and utilization

Kernel dispatch represents the final stage of runtime execution, coordinating host-side task queues with device-side execution. The software runtime manages host-side command queues (such as CUDA streams or TPU command rings) and records hardware events or semaphores to enforce dataflow dependencies asynchronously. When independent operations (such as parallel multi-head attention projections) have no mutual data dependencies, the runtime dispatches them onto separate streams to enable concurrent execution on the accelerator. Once submitted to the device queue, however, execution control transfers entirely to hardware schedulers. On a GPU, the hardware warp scheduler and work distribution engine assign thread blocks to SMs based on register file availability, shared memory capacity, and maximum warp occupancy; the software runtime cannot schedule individual threads or dictate L1/L2 cache replacement policies. Similarly, TPU runtimes submit macro-instructions to on-chip command queues, while hardware sequencers feed systolic matrix units and vector units (Jouppi et al. 2017).

A low achieved rate alone cannot distinguish runtime gaps from a memory bottleneck. Consider three supplied illustrative profiles of low-intensity kernels, not measurements from a particular device. In the first, the timeline shows continuous kernel execution and HBM traffic close to the sustainable bandwidth ceiling. Reducing transferred bytes or increasing reuse addresses that observed limit. In the second, kernels remain active but dependent loads produce long stalls while HBM traffic stays well below its ceiling. More bandwidth alone need not help: independent requests or a different access schedule must expose enough concurrency to hide the latency. In the third, short kernels alternate with device-idle gaps while the host submits work. Reducing dispatch and synchronization overhead addresses those gaps; changing the arithmetic instruction rate does not.

The discriminating evidence is the timeline together with traffic and stall observations. If added independent requests raise bandwidth and shorten load stalls, they support the latency hypothesis; if bandwidth is already sustained near its ceiling, traffic reduction is the more direct lever. The roofline bounds the resource opportunity, while the profile identifies which mechanism leaves the kernel below it. Even with aggressive kernel fusion, optimal tactic selection, and zero-allocation memory reuse, the execution capacity of a single accelerator is physically bounded by its silicon area, thermal dissipation limit, and high-bandwidth memory capacity, so scaling beyond these physical boundaries requires distributing computation across multiple accelerators.

11.9 Beyond One Accelerator

Everything so far has assumed one accelerator, which is enough for the chapter's ResNet-50 workload and for GPT-2 XL decoding. Some workloads are not served by one device, either because their arithmetic would take too long on it or because their weights and runtime state do not fit in its memory. The techniques covered earlier remain the foundation past that point, since each device still needs an efficient dataflow, but communication adds further boundaries to the hierarchy. Moving data through on-chip storage, HBM, an intra-node link, and a network fabric generally increases latency and energy at each step, so scaling is not merely a question of adding chips.

Each step outward moves the communication boundary and changes the dominant trade-off. Inside the package, chiplet-based architectures partition a large design into smaller dies joined at relatively low latency, bypassing the manufacturing limits of a monolithic chip, while wafer-scale integration takes the opposite approach and keeps more computation on one device. At the node, a multi-accelerator server joins several chips, each with its own memory, through board- or server-level interconnects such as NVLink. A workload then splits through data parallelism (each accelerator processes different batches) or model parallelism (different accelerators hold different layers, or individual matrix multiplications are sliced across devices). When a model's weights and runtime state exceed one device's HBM, that split becomes a placement requirement rather than a speed choice. How training uses the accelerators of one machine, including how they synchronize gradients, is developed in section 8.7.1. Within a server, the engineering test compares the computation or device-memory capacity that partitioning saves with the communication it exposes, accounting for overlap and the actual directional link bandwidth.

Beyond the node, the network interface card links each machine to the others over Ethernet or InfiniBand. It is the narrowest, highest-latency step of the bandwidth taper measured in section 11.5.4.1, and traffic that must cross it leaves single-node reasoning behind. Work that spans several machines lies beyond this book's one-machine scope.

11.10 Heterogeneous SoC Design

Edge, mobile, and automotive deployments operate under strict physical constraints: sealed enclosures without active fans limit thermal dissipation to a few watts, while battery capacity caps total energy consumption. To deliver machine learning capabilities within these boundaries, heterogeneous system-on-chip (SoC) architectures integrate general-purpose CPU cores, GPU shader arrays, digital signal processors (DSPs), and specialized neural processing units (NPUs) onto a single silicon die sharing a unified memory subsystem. The workload mapping problem introduced for discrete accelerators persists, but it expands to include hardware operator coverage, cross-engine transfer overheads, real-time deadlines, and dynamic thermal throttling. Two of the book's lighthouses take over from ResNet-50 here. MobileNetV2, compute-bound in its efficient regime and built for phone cameras, runs its convolutions on a phone's NPU within a few watts. KWS, the power-bound lighthouse, listens continuously on a DSP or microcontroller within milliwatts. Both run on silicon that shares one memory system among every engine on the chip.

11.10.1 Mobile SoC architecture evolution

Mobile system-on-chip architectures implement heterogeneous computing by coupling distinct execution engines, namely multicore CPUs, wide SIMD GPU shader cores, fixed-point DSPs, and dedicated neural processing units (NPUs) over an on-chip network connected to shared low-power DRAM (LPDDR). Workload partitioning routes sensory and perceptual tasks to the compute engine best aligned with their arithmetic structure: continuous audio feature extraction and keyword

spotting run on low-power DSPs, dense convolution and matrix-multiplication layers execute on systolic or tensor-array NPUs, parallel graphics and image postprocessing map to GPUs, and sequential control flow runs on the CPU. Meeting real-time frame deadlines without triggering aggressive thermal throttling requires scheduling execution across these engines while respecting their differing power profiles.

Example 11.5: Heterogeneous microcontrollers

Scenario: An embedded smart doorbell must run periodic person detection within a tight memory, energy, and latency budget.

Diagnosis: On a supported deployment, a general-purpose Cortex-M path misses the measured energy or latency target. A micro-NPU such as Arm Ethos-U can offload compatible convolution operators, while the CPU retains control flow and unsupported operations.

Systems lesson: Micro-NPU specialization can improve the energy and latency of supported operators, but feasibility must be measured for the whole pipeline, including fallbacks and sensor processing.

Modern mobile SoCs use unified memory architectures where all processing units share a common physical address space in LPDDR. Unlike discrete accelerator architectures that incur explicit PCIe transfer latencies, unified memory allows zero-copy buffer passing between the image signal processor (ISP), CPU, GPU, and NPU. Sharing physical DRAM does not guarantee zero software overhead, however. Non-coherent engines need explicit cache maintenance (section 11.10.4), and specialized NPUs often require hardware-specific tensor layouts (such as channels-last (NHWC) with 16- or 32-byte channel alignment to feed vector register lanes), forcing memory-bound transpositions when interfacing with CPU layers structured in channels-first (NCHW) formats.

SoC architects customize these compute blocks according to domain-specific physical constraints. High-end smartphone SoCs emphasize peak burst performance and low idle leakage for interactive tasks such as real-time computational photography and on-device text generation. In contrast, embedded and automotive SoCs prioritize deterministic memory access, hardware fault detection, and bounded worst-case latency to satisfy real-time control constraints.

11.10.2 Strategies for dynamic workload distribution

Partitioning neural network workloads across heterogeneous SoC cores requires matching the arithmetic intensity and memory regularity of each computational graph operator to the microarchitecture best equipped to execute it. Consider a real-time object detection pipeline executing on a mobile SoC with an integrated CPU, GPU, and NPU. The pipeline comprises three consecutive stages: a MobileNetV2 backbone for feature extraction, nonmaximum suppression (NMS) for postprocessing candidate bounding boxes, and a display overlay to render final detections onto a camera feed. The convolutional backbone has regular, deterministic memory access and dense arithmetic suited to the NPU's fixed-function tensor engine, even though depthwise separable layers have lower arithmetic intensity than standard convolutions. NMS, by contrast, relies on score-based sorting and conditional intersection-over-union loops evaluated across variable-length candidate lists. This irregular, branch-heavy control flow maps poorly to an NPU's spatial array but executes efficiently on out-of-order CPU cores with branch prediction and hardware caching. Finally, the display overlay requires per-pixel rasterization and alpha blending across a high-resolution frame (an embarrassingly parallel, memory-bandwidth-bound task that maps directly to the GPU's rasterizers and texture units). Splitting the pipeline across the three engines, with the NPU for feature extraction, the CPU for NMS, and the GPU for display compositing, is a candidate mapping. Its compute savings must exceed transfer, layout, and synchronization costs before it improves end-to-end latency or energy over a single-engine implementation.

This pipeline partitioning reflects the broader relationship between algorithmic structures and machine architectures. Regular matrix contractions and convolutions map cleanly to systolic or vectorized NPU datapaths, whereas data-dependent control flow maps to CPU execution resources. Transformer attention layers span both paradigms: the projection GEMMs ($\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$) and attention score contractions ($\mathbf{Q}\mathbf{K}^T$, and the product of the attention weights with $\mathbf{V}$) execute efficiently on

NPU matrix units, but intermediate softmax reductions, causal masking, and layer normalizations involve elementwise operations. If the NPU runtime cannot fuse these elementwise operations into on-chip SRAM scratchpads, intermediate tensors must spill to LPDDR, eroding the accelerator's throughput advantage.

Optimal processor assignment is dynamic rather than static. As device thermal conditions deteriorate or battery reserves drop, the runtime must adapt its scheduling policy. However, migrating an executing model across processor backends carries non-trivial overhead: weights must be re-bound in memory, intermediate buffers may require layout reformatting, and device drivers incur kernel dispatch latency. If the overhead of synchronizing state across processors exceeds the compute savings of offloading, retaining execution on a single suboptimal engine can yield lower total latency and energy than continuous cross-engine migration.

Shared memory subsystems introduce cross-engine interference during simultaneous execution. Because the CPU, GPU, NPU, camera ISP, and display engine share a single LPDDR memory bus, the SoC memory controller enforces Quality of Service (QoS) priorities to satisfy real-time hardware deadlines. The display engine and camera ISP must receive guaranteed memory bandwidth and bounded latency to prevent display flicker and sensor FIFO overflow; the memory controller consequently prioritizes display and camera transactions over NPU and GPU traffic. During memory-intensive phases (such as autoregressive language model decoding where model weights must be read from DRAM for every generated token) high-priority ISP or display traffic reduces effective NPU memory bandwidth, increasing token generation latency.

Automotive computing platforms expose a stricter form of the same interference. A perception pipeline with a hard deadline may share the SoC and memory system with a cabin assistant or infotainment model. If the convenience workload streams weights from DRAM without a bandwidth limit, it can delay the perception pipeline even when the two use different compute engines. Such deployments require resource isolation and validation under concurrent load; average inference throughput alone cannot establish deadline compliance. The hardware lesson is the interference boundary: assigning work to separate engines does not isolate workloads that still share memory bandwidth.

11.10.3 Power and thermal management

Fanless mobile devices operate within strict continuous power budgets and thermal envelopes, typically dissipating no more than 3 to 5 watts sustained across the entire chassis before touch temperatures exceed safety thresholds. Although transient bursts can draw 10 to 15 watts, continuous inference rapidly saturates the thermal capacitance of the device, heating the silicon junction and requiring active power management across the SoC.

Heterogeneous SoCs manage power across distinct silicon partitions using **dynamic voltage and frequency scaling (DVFS)**, which exploits the cubic dependence of dynamic power on frequency behind the power wall (equation 2.2). Total power in complementary metal-oxide-semiconductor (CMOS) digital circuits is the sum of dynamic switching power and static leakage power:

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{leakage}} = \alpha C V^2 f + I_{\text{leak}} V$$

where α is the gate switching activity factor, C is physical load capacitance, V is supply voltage, f is clock frequency, and I_{leak} is temperature-dependent leakage current. Over an operating range where increasing clock frequency f requires supply voltage V to rise approximately proportionally, fixed activity and capacitance give the cubic approximation for dynamic power dissipation:

$$P_{\text{dynamic}} \propto f^3$$

Under that approximation, doubling clock frequency can incur an eightfold penalty in dynamic power (Le Sueur and Heiser 2010), and the same relation run in reverse is what DVFS exploits: dropping the clock lets the core voltage fall, and dynamic power falls faster than throughput. The cubic penalty also explains why ML accelerators favor massive parallel arrays of simple execution units clocked at moderate frequencies over narrow pipelines clocked at extreme frequencies.

On a heterogeneous SoC, CPU clusters, GPU shader cores, and NPU engines occupy separate power and voltage domains. Increasing the voltage and frequency of the NPU to meet a tight latency deadline consumes electrical current and thermal headroom that would otherwise be available to

the CPU or GPU. System power governors continuously adjust these operating points based on workload queues, temperature sensor readings, and frame deadline targets.

When sustained computation exceeds the chassis dissipation rate, the platform initiates thermal throttling. Because specialized NPUs achieve higher energy efficiency (TOPS/W) for tensor math than general-purpose cores, offloading matrix workloads to a CPU or GPU during a thermal event often increases total energy dissipation, exacerbating thermal accumulation. Instead, runtimes manage thermal limits by capping clock frequencies, reducing inference duty cycles (such as throttling a perception model from 30 frames per second to 15 frames per second), or dynamically switching to a smaller, pruned model checkpoint. Task migration is applied selectively to move non-tensor preprocessing away from localized silicon hot spots on the die. Battery state further shapes runtime scheduling: when battery charge drops below critical thresholds or the device enters a low-power mode, the system scheduler trades inferencing fidelity for battery life by selecting prevalidated lower-precision or sparse model variants, or by batching asynchronous inference requests to maximize the time cores spend in deep low-power sleep states.

11.10.4 Software stack challenges

Heterogeneity shifts the primary systems challenge from silicon design to software coordination. Standard programming models like OpenCL and Vulkan provide portable low-level execution abstractions across compute devices, but they do not eliminate the need for microarchitecture-specific kernel tuning. Machine learning inference runtimes partition computation graphs through hardware-specific delegates or backends targeting CPUs, GPUs, DSPs, and NPUs. When a model contains an operator unsupported by an NPU delegate (such as a custom activation, dynamic tensor shape, or non-standard reduction) the runtime splits the graph into subgraphs, inserting fallback paths to the CPU or GPU. Each boundary crossing requires synchronizing execution and translating data representations, transforming an algorithmic graph into a fragmented sequence of hardware dispatches.

Shared memory subsystems require explicit cache and memory management across non-coherent execution domains. While CPUs maintain cache coherence across their own cores via hardware snooping protocols, mobile NPUs and DSPs frequently interface with system memory as non-coherent DMA bus masters. When the CPU writes input tensors to cached DRAM and hands control to an NPU, software must explicitly flush the CPU's dirty cache lines to DRAM before NPU execution begins. Similarly, upon NPU completion, the CPU must invalidate its own cache lines over the destination memory range to prevent reading stale cached data. If unsupported operators cause an inference graph to ping-pong repeatedly between CPU and NPU, the latency of cache maintenance and intermediate DRAM round-trips can exceed the execution time of the operators themselves.

Runtimes navigate this scheduling complexity using ahead-of-time compilation, empirical profile databases, and dynamic dispatch heuristics. Device telemetry (monitoring frame completion times, core temperatures, and battery drain) informs runtime decisions to throttle clock frequencies or swap execution delegates. However, dynamic adaptation cannot replace deterministic graph lowering for latency-sensitive applications; deployment requires validating the compiled, lowered partition on physical target silicon under worst-case thermal and memory contention.

On an SoC, speed comes from keeping work and data on the right engine, not from how many engines the chip has. In the D·A·M taxonomy, an SoC represents a composite Machine layer whose components have different memory hierarchies, arithmetic strengths, and thermal envelopes. The iron law's data term and the energy-movement principle (7) together make data movement the governor of both runtime and energy: when software coordination fails to maintain tensor locality or induces frequent cross-engine fallbacks, energy dissipation is dominated by interconnect and DRAM traffic rather than arithmetic. Maximizing performance on heterogeneous SoCs requires minimizing data movement across domain boundaries while matching each algorithmic operation to the specialized datapath architected to accelerate it. From the microcontroller that runs keyword spotting to the data-center accelerator that serves GPT-2 XL, that matching ends in a choice of machine, which the next section makes explicit.

11.11 Choosing an Accelerator

Every section of this chapter ends in the same question asked at a different level: which machine should run a given workload. The question cannot be answered from a datasheet, because the resource that binds belongs to the workload on a particular machine rather than to the machine alone. The book's five lighthouses make the point on a single reference machine. Their bytes and operations differ by orders of magnitude, and each one exhausts a different resource of an H100 first (table 11.17).

Table 11.17: Lighthouse Memory and Compute: Model bytes and operations for the book's five lighthouses from the model registry, with the resource each exhausts first on one H100. The operation counts are per inference, and per generated token for GPT-2 XL decode; DLRM's bytes are dominated by its embedding tables, whose lookups perform little arithmetic.

Lighthouse	Home regime	Model bytes (FP16)	Model bytes (INT8)	Operations per inference	Binding resource on one H100
ResNet-50	Compute bound	51.2 MB	25.6 MB	8.2 GFLOP	Arithmetic, once batching amortizes the weights
MobileNetV2	Compute bound	7.0 MB	3.5 MB	0.6 GFLOP	Launch overhead and depthwise-layer bandwidth; built for a phone NPU
GPT-2 XL (decode)	Bandwidth bound	3 GB	1.5 GB	3 GFLOP per token	HBM bandwidth: every token rereads every weight
DLRM	Capacity bound	100 GB of tables	Tables dominate either way	Few per lookup	Memory capacity: tables exceed one device's 80 GB
Keyword spotting (KWS)	Power bound	400 KB	200 KB	20 MFLOP	None on an H100; built for a microcontroller's milliwatt budget

The table turns the chapter's diagnosis into a selection rule. ResNet-50 rewards arithmetic throughput on a supported INT8 path, GPT-2 XL decode rewards memory bandwidth, DLRM rewards memory capacity, and MobileNetV2 and KWS reward energy per inference on chips that never see a data center. The rest of the section applies that rule at three levels: the configurations accelerator families adopt for their target deployments, the cost of delivered throughput, and the energy each request consumes.

11.11.1 Processor configurations across deployment targets

SIMD lanes, Tensor Cores, and systolic arrays are building blocks, but their full-chip organization varies significantly across AI processors; the choice of execution units, their numerical precision support, and their connectivity shape how effectively hardware serves a given deployment. Modern AI processors make distinct design trade-offs based on their intended applications, and comparing their configurations reveals how deployment constraints drive architectural divergence. Training-optimized accelerators such as the NVIDIA A100 pack many Streaming Multiprocessors with wide SIMD units and FP16 Tensor Cores because training throughput scales with aggregate multiply-accumulate capacity across general kernels (NVIDIA Corporation 2020a), whereas Google's TPUv4 concentrates each chip into just two cores built around massive BF16 systolic arrays, trading per-thread flexibility for area and energy efficiency on dense matrix multiplications (Jouppi et al. 2023). Inference and edge targets shift the balance again: Intel's Sapphire Rapids dedicates Advanced Matrix Extensions (AMX) tile engines to INT8 and BF16 because server inference models tolerate reduced precision (Intel Corporation 2021a), and mobile neural engines shrink those matrix datapaths into low-power SoC blocks that prioritize energy per operation over peak throughput. Table 11.18 compares these architectural configurations.

Table 11.18: AI Processor Configurations: Modern AI processors prioritize different execution unit characteristics for specific workloads: NVIDIA A100 uses wide SIMD and Tensor Cores for training, Google TPUv4 emphasizes high-throughput BF16 matrix multiplication, Intel Sapphire Rapids focuses on INT8-optimized inference, and mobile NPUs prioritize low-power execution. These variations in SIMD width, Tensor Core size, and processing element count reflect the growing diversity in AI hardware architectures.

Processor	**Vector execution**	**Matrix engine**	Processing Elements	**Primary workloads**
NVIDIA A100	CUDA FP/INT lanes	Tensor Core instruction tiles	108 SMs	Training, HPC
Google TPUv4	Vector unit	128×128 BF16 systolic arrays	2 cores/chip	Training
Intel Sapphire	512-bit AVX-512	AMX tiles (up to 16 rows by 64 bytes each)	Up to 60 cores	Inference
Mobile NPU	CPU/GPU/DSP vectors	Small matrix engines	Integrated NPU blocks	Mobile inference

The pattern across these configurations reveals a consistent engineering principle: each design sacrifices generality to optimize for its target workload's dominant operation and precision. Training chips invest silicon in wide floating-point datapaths; inference chips trade precision for throughput; mobile chips trade throughput for energy efficiency. No single design dominates across all workloads, which is precisely why hardware selection depends on workload analysis rather than headline specifications.

11.11.2 Cost-performance analysis

Architectural specifications define computational potential, but deployment decisions require cost-performance analysis. Raw compute is only one input: delivered performance may be limited by arithmetic throughput, memory bandwidth, interconnect latency, software overhead, or low operational utilization. The energy gap established in section 11.2.3 drives this specialization. Because memory access costs dominate computation, accelerators achieve only a fraction of their peak throughput on memory-bound kernels. Conversely, architectures that maximize data reuse, such as systolic arrays executing dense matrix multiplications, sustain substantially higher utilization by amortizing each memory fetch across multiple arithmetic operations.

Consider an engineering team choosing between deploying a larger cluster of older accelerators or a smaller cluster of newer units. Peak FLOP/s figures mislead whenever dominant kernels are memory-bandwidth or communication bound. In a large model pipeline, dense feed-forward and attention projection layers may remain compute bound, whereas embedding lookups, layer normalization, residual additions, and small-batch autoregressive decoding often have low arithmetic intensity. Their measured bottlenecks can include memory bandwidth, dependent access latency, reductions, and launch overhead; the operator name alone does not identify the limiting resource. The decisive metric is delivered workload throughput per dollar under realistic cluster utilization, memory capacity constraints, available memory bandwidth, and scale-out interconnect efficiency.

These architectural and economic constraints explain the rapid industry adoption of newer accelerators despite substantial increases in nominal unit price. For bandwidth-bound workloads, generational improvements in memory bandwidth and compiler code generation dominate end-to-end execution time. Cloud deployments introduce additional trade-offs: hourly rental rates, dynamic instance utilization, and infrastructure overhead continuously shift the economic break-even point between purchasing on-premises hardware and leasing cloud capacity.

Price-per-peak worksheets for named products age quickly. List prices vary with vendor, region, contract terms, and procurement volume, and each product's headline peak uses a different precision, so the ratio of price to peak is meaningful for one device but not as a comparison across devices. A dated worksheet for several accelerator generations is kept with the other specifications in Appendix E. The durable rule is the one the lighthouse table states: divide cost by the throughput the workload delivers on its binding resource, not by the advertised peak.

No worksheet reduces accelerator selection to a single headline specification. A bandwidth-bound kernel benefits more from the H100's 3,350 GB/s memory subsystem than from an incremental gain in peak arithmetic, whereas a compute-bound GEMM shows the opposite sensitivity. Device memory capacity dictates whether working sets fit on-chip without spilling or partitioning, while

compiler and driver maturity determines whether the application can actually reach the advertised datapath.

11.11.3 Energy per request

At fleet scale, energy per useful inference becomes an important hardware-selection criterion. Operational emissions depend on workload throughput, measured system power, utilization, cooling overhead, and the electricity mix; embodied emissions add another lifecycle cost. Peak performance per watt is therefore a screening metric, not a substitute for end-to-end energy per request. The following scenario shows how useful throughput and a common power boundary determine the fleet needed for the same inference service.

Napkin Math 11.11: An illustrative operational-energy comparison

Problem: An inference service must complete 1 billion requests per day. Which of two hypothetical server platforms uses less operational energy for that service?

Supplied observations: For the same model, input distribution, accuracy, and latency target, a CPU server sustains 100 requests/s, and an NPU-equipped server sustains 1,000 requests/s. Their whole-server wall powers are 100 W and 200 W, respectively, including hosts and attached accelerators. These are hypothetical observations for this exercise, not product benchmarks. Assume requests arrive steadily through the day. Both fleets operate at no more than 80 percent of the supplied throughput to reserve capacity for arrival-rate variation. Assume the listed wall power throughout the 24 h day, including spare capacity. Facility cooling and embodied emissions are outside this operational boundary.

Calculation:

1. **Size the fleets from useful work.** Multiplying useful throughput by the operating fraction and the day length gives 6,912,000 requests per CPU server per day and 69,120,000 per NPU-equipped server. Divide the daily target by each capacity and round up to whole servers. The service requires 145 CPU servers or 15 NPU-equipped servers.
2. **Account for the provisioned fleet.** Server count multiplied by wall power and day length yields 348 kWh per day for CPUs and 72 kWh for NPUs. Dividing each fleet's energy by the same completed-request target gives 1.2528 J per request and 0.2592 J per request. These values include capacity headroom and integer fleet rounding.
3. **Translate the energy difference.** At a common grid factor of 0.429 kg/kWh, the NPU-equipped fleet avoids approximately 43.2 t of operational CO_2 emissions per year at the stated system boundary.

Decision: Choose the NPU-equipped fleet if the objective is to minimize operational energy under these supplied conditions. Each server draws more power, but its greater useful throughput reduces the required fleet enough to lower joules per completed request. This conclusion depends on meeting the same quality and latency targets; a faster peak arithmetic rate alone would not establish the advantage. A procurement decision must additionally compare acquisition costs, software support, and embodied emissions.

Energy per request completes the selection rule that the lighthouse table began. Hardware selection is not determined by peak speed alone: energy per useful result, carbon intensity, embodied impact, and total cost of ownership belong beside latency, throughput, and memory capacity, and each is measured against the resource the workload actually exhausts. The misconceptions that follow are the ways teams skip that measurement and choose the wrong hardware path.

11.12 Fallacies and Pitfalls

Hardware acceleration often defies intuition because headline specifications mask workload-specific bottlenecks. The fallacies and pitfalls here capture selection, scaling, and optimization errors that leave expensive compute resources underused. Sound system evaluation follows one diagnostic habit instead: characterize the workload arithmetic intensity, identify the physical bottleneck metric,

match execution to the appropriate datapath, and account for interconnect, scaling, and portability costs.

Fallacy: *More specialized hardware always provides better performance than general-purpose alternatives.*

Specialized accelerators achieve high efficiency only when workload access patterns match underlying execution datapaths: the core tenet of algorithm-machine co-design. On the roofline (section 11.6), the A100 reference tensor path has a ridge point of 153 FLOP/byte; below that ridge, increasing its 312 TFLOP/s arithmetic peak does not raise the modeled bandwidth ceiling. A transformer attention softmax with I = 2 FLOP/byte–5 FLOP/byte is capped at 4.1 TFLOP/s–10.2 TFLOP/s (3.3 percent of tensor peak) on an A100. On a CPU, whose ridge point sits around 10 FLOP/byte–20 FLOP/byte, the same traffic model places the kernel below the reference ridge, and its bandwidth ceiling represents 10 percent–50 percent of the CPU's compute capacity. These fractions are modeled bandwidth bounds, not measured utilization. When workloads have irregular memory access, tiny batch sizes, or dynamic control flow, massively parallel execution pipelines stall on memory latency and thread divergence. General-purpose CPUs with large cache hierarchies, out-of-order execution, and branch predictors often deliver superior latency and resource efficiency under these conditions. Hardware selection requires matching workload arithmetic intensity and dataflow regularity to architectural ridge points rather than assuming specialization guarantees speed.

Pitfall: *Ignoring memory bandwidth limitations when selecting acceleration strategies.*

Procurement decisions frequently target peak arithmetic throughput while overlooking memory interface constraints. Off-chip data movement imposes severe performance and energy penalties: fetching an operand from DRAM consumes roughly 640 pJ compared to 0.5 pJ for a small, register-adjacent L1 SRAM access, the cheapest tier noted in the caption of figure 11.4. Consider an accelerator providing 300 TFLOP/s of compute alongside 2 TB/s of memory bandwidth, setting a ridge point of 150 FLOP/byte. A normalization kernel such as LayerNorm with an arithmetic intensity of 1.5 FLOP/byte has a bandwidth ceiling of 3 TFLOP/s (1 percent of compute peak). Under the stated HBM streaming assumption, memory bandwidth supplies this upper bound, and adding compute units alone does not raise it.

Fallacy: *Hardware acceleration benefits scale linearly with additional accelerators.*

Distributed training need not deliver $8\times$ throughput on eight accelerators because exposed inter-chip communication can reduce parallel efficiency. Synchronizing gradients across ranks via ring AllReduce makes each rank send and receive about twice the gradient payload (equation 8.12), or 1.75× payloads across eight devices. With an interconnect such as NVLink providing 600 GB/s bidirectional bandwidth (half that per direction), transmitting a 1 GB gradient payload consumes 5.83 ms. For a model with a 50 ms compute step, this data exchange adds 11.7 percent communication overhead before counting software collective latency. Without compute-communication pipelining, an eight-accelerator node reaches only 7.2× speedup (89.6 percent parallel efficiency). Stragglers, barrier synchronizations, and collective protocol overhead widen the gap between linear expectations and physical cluster scaling.

Pitfall: *Planning accelerator capacity from peak FLOP/s specifications.*

Peak FLOP/s measures arithmetic unit capacity under ideal instruction mixes and continuous operand availability, not delivered application throughput. Realized performance depends on kernel arithmetic intensity, matrix dimensions, memory coalescing, and runtime launch overhead. The roofline model (section 11.6) bounds the gap between peak compute and bandwidth-limited execution. On an A100 GPU with a 312 TFLOP/s peak, an illustrative dense-transformer scenario supplies a sustained range of 120 TFLOP/s–180 TFLOP/s (40 percent–60 percent model FLOPs utilization). This supplied range is illustrative, not a measurement or a prediction for every model. Sizing cluster capacity from peak FLOP/s creates severe under-provisioning; capacity planning must rely on measured sustained throughput or roofline-bounded kernel projections.

Fallacy: *One peak FLOP/s rating describes a workload at every precision.*

Accelerators have separate datapaths for different precisions, and the peak throughput varies dramatically across them. An H100 delivers roughly 989 TFLOP/s in FP16 tensor operations but only about 67 TFLOP/s in FP32 CUDA-core operations: a roughly 15× gap within the same chip

(Choquette 2023). Dividing achieved BF16 model FLOP/s by the smaller FP32 peak inflates the reported utilization, while using that FP32 peak to estimate ideal training time overestimates the time. Matching against the wrong roofline misclassifies kernels as compute-bound when they are memory-bound (or vice versa). Useful model FLOP utilization divides the model's useful operations per elapsed second by the matching supported precision and execution-path peak. Hardware busy time is a different metric: an active device can spend time on transfers, redundant work, or fallback execution. Quote the denominator and precision explicitly.

Pitfall: *Deploying small-batch inference workloads on high-compute accelerators.*

Small-batch dense inference can use only a small fraction of arithmetic peak because single-sample execution lacks cross-sample parameter reuse. For a dense linear projection with widths $d_{\text{in}} = d_{\text{out}} = 2048$, a single request ($B = 1$) is modeled as streaming the entire weight tensor from device memory to execute a matrix-vector product, yielding an arithmetic intensity of only 1 FLOP/byte. At batch size $B = 256$, weight reuse amortizes memory transfers across tokens, elevating arithmetic intensity to 204.8 FLOP/byte. At $B = 1$, memory bandwidth bounds performance at 2.04 TFLOP/s on an A100 and 0.32 TFLOP/s on a T4. Although the T4 provides a lower peak of 65 TFLOP/s with a ridge point of 203.1 FLOP/byte, the two batch-1 ceilings differ by 6.4×, which is exactly their memory-bandwidth ratio; the A100's 4.8× advantage in peak compute contributes nothing. The A100 costs 6× as much as the T4, so their modeled batch-1 throughput ceilings per dollar are nearly the same. Actual procurement requires measured latency, throughput, memory fit, and the intended request mix.

Fallacy: *Vendor-specific optimizations have no long-term portability cost.*

Optimizing exclusively for a single vendor runtime captures immediate kernel efficiency but introduces severe architectural lock-in (section 11.8). Codebases saturated with proprietary device intrinsics, vendor-specific assembly, and non-standard memory layouts require months of engineering labor to port when newer, more cost-effective accelerators emerge. This technical debt delays hardware transitions and prevents multi-vendor procurement. To preserve flexibility, systems must isolate vendor-tuned micro-kernels behind hardware abstraction layers or target intermediate compiler representations (such as Triton or MLIR).

Checkpoint 11.5: Feasibility assessment: Can you run it?

Before procuring hardware, validate each physical constraint. These failure modes translate into a concrete feasibility test:

- ☐ **Memory capacity**: use $M_{\text{total}} = M_{\text{weights}} + M_{\text{KV}} + M_{\text{activations}}$ (weights, KV cache, and activation buffers) to list the inputs needed to test $M_{\text{total}} < M_{\text{device}}$ for a 7-billion-parameter Llama model with 14 GB FP16 weights on a 16 GB GPU. What sequence length, concurrency, model dimensions, and runtime workspace information is missing before a complete fit decision?
- ☐ **Bandwidth**: compute the weight-streaming lower bound $T_{\text{token}} = D_{\text{vol}}/\text{BW}$ for a 70-billion-parameter model (140 GB) on 1 TB/s memory bandwidth, then compare the result with a 50 ms latency target. Assume one dense FP16 weight read per generated token; what additional traffic and overhead would a measured latency include?
- ☐ **Compute**: identify the operation count, numerical path, and measured efficiency needed to use $T_{\text{compute}} = O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$. With serialized frame processing, 30 FPS leaves 33.3 ms per frame for the whole pipeline. How does overlapped processing change the distinction between frame latency and sustained throughput?
- ☐ **Roofline placement**: for an accelerator at roughly 2 TFLOP/s peak and 200 GB/s bandwidth, estimate the ridge-point arithmetic intensity, then place a 10 FLOP/byte kernel against it: how do the two ceilings compare, and would increasing only the compute clock raise the overall bound? What measurements would test the actual bottleneck?

11.13 Summary

Hardware acceleration co-designs compute engines, memory hierarchies, numerical formats, and dataflows. For a chosen memory interface, the iron law compares compute time $O/(R_{\text{peak}}\eta_{\text{hw}})$ with movement time D_{vol}/BW; the roofline restates that competition as $\min(R_{\text{peak}}, \text{BW} \times I)$. End-to-end latency also includes launch, synchronization, communication, and fixed overheads. Accelerators improve efficiency only when supported paths capture reuse locally. Hardware selection therefore begins with the workload's measured or modeled bottleneck, not a vendor peak. For ResNet-50, INT8 multiplies large-batch throughput because large batches sit near the arithmetic ceiling, while batch-one latency sits on the overhead floor and moves only when compilation removes launch cost.

The chapter map (table 11.1) can now be read row by row for the INT8 ResNet-50 workload that opened the chapter. Its convolutions lower onto matrix units, its INT8 path doubles the H100's arithmetic ceiling without moving the bandwidth ceiling, its activation tiles stay on chip only when the mapping and the tile sizes allow it, and whether a given layer gains from INT8 depends on whether its activations travel at INT8, a choice the compiler and runtime make. GPT-2 XL decode, the single contrast, gains from none of the arithmetic and from every byte saved.

The bounds developed across the chapter fit in one table (table 11.19). Together they let an engineer estimate, before profiling, whether a kernel on a given accelerator is limited by arithmetic, by memory bandwidth, by on-chip capacity, or by work the accelerator never touches.

Table 11.19: Key Equations: The chapter's bounds in the notation of the sections that derive them. Each ceiling is specific to one precision path and one memory interface; the tile-capacity form assumes FP16 operands and an FP32 accumulator held in one software-managed local store.

Quantity	Formula	Where derived
Amdahl bound	$\text{Speedup} = \frac{1}{(1-f_{\text{parallel}})+f_{\text{parallel}}/G_{\text{accel}}}$	equation 11.1
Memory and compute time	$T_{\text{memory}} = D_{\text{vol}}/\text{BW}$; $T_{\text{compute}} = O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	section 11.5.2
Arithmetic intensity	$I = O/D_{\text{vol}}$	definition 5.3
Ridge point (hardware balance)	$I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$, per precision path	section 11.6.2
Roofline bound	$R_{\text{attain}} = \min(R_{\text{peak}}, \text{BW} \times I)$	equation 11.2
Required bandwidth	$\text{BW}_{\text{req}} = R_{\text{ops}}/I$	equation 11.3
Dense-layer intensity	$I \approx B$ when weight traffic dominates	equation 11.5
Decode floor	$T_{\text{token}} = D_{\text{vol}}/\text{BW}$, with D_{vol} the weight bytes read per token	Notebook 11.10
Tile capacity	$(2\text{ bytes})M_{\text{tile}}K_{\text{tile}} + (2\text{ bytes})K_{\text{tile}}N_{\text{tile}} + (4\text{ bytes})M_{\text{tile}}N_{\text{tile}} \leq$ local bytes	equation 11.6
Transfer overlap	Hidden when $T_{\text{compute}} \geq T_{\text{transfer}}$	section 11.5.4.2

Key Takeaways: Moving data costs more than computing it

- **The roofline model bounds attainable performance**: Compare a kernel's arithmetic intensity with the ridge for its memory interface and supported execution path (295.2 FLOP/byte for the H100's dense FP16 tensor path and 590.7 FLOP/byte for its INT8 path). Below it, bandwidth supplies the tighter modeled ceiling; above it, compute does. Profiling establishes whether the kernel approaches that ceiling or is limited by dependencies, dispatch, or another execution resource.
- **Memory bandwidth constrains performance**: GPU compute capacity has grown faster than memory bandwidth. Many inference phases are memory bound, especially low-batch decoding and embedding lookup, while large-batch GEMMs can remain compute bound.
- **Hardware-software co-design compounds performance**: Matching algorithm patterns to architectural capabilities, such as dense GEMM on systolic or tensor units and supported sparsity on sparse paths, can produce large improvements.

- **Tensor Cores require a compatible path**: Precision, dimensions, layouts, and library support must match the architecture. Batch and shape affect reuse and utilization, but no single batch threshold guarantees peak performance.
- **End-to-end speedup stops at the next bottleneck**: Accelerating one kernel leaves launch, transfer, synchronization, communication, and unsupported work unchanged. Use Amdahl's law (principle 8) and profiling to decide whether further silicon, better mapping, or less movement will improve the complete workload.

An accelerator is not a uniformly faster computer. It provides efficient paths for particular operations and dataflows, and each of those paths is a way of moving fewer bytes per useful operation. Tiling, fusion, stationary dataflows, and systolic execution pay off only when the workload, numerical format, and software path can use them. The diagnostic habit the chapter leaves is therefore an ordering. Place the kernel on the relevant roofline to bound its performance, bound the end-to-end gain with Amdahl's law, and only then choose among more silicon, a better mapping, or less movement. Each step is a prediction that measurement must confirm, and the same habit is what keeps the bitter lesson affordable: scale keeps winning only while each additional operation gets cheaper in time and energy, and on modern accelerators the operation is rarely the expensive part; the bytes that feed it are.

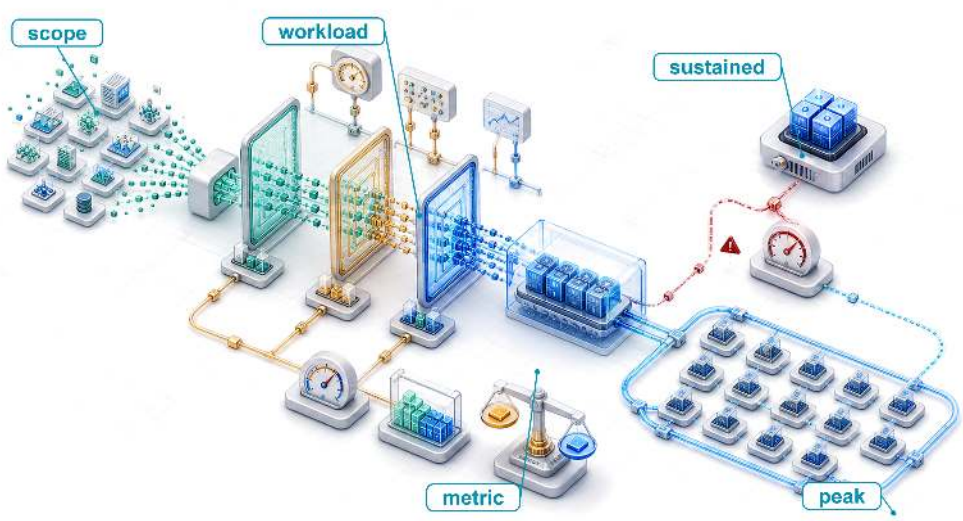


What's Next: From optimization to validation

Data selection, model compression, and hardware acceleration each end with a favorable local number. Data selection reaches target accuracy on fewer examples, compression produces a smaller artifact, and the roofline places a kernel below a ceiling that the vendor's peak overstates. Each number is a prediction made for one stage in isolation, and the stages interact once they run together on a real device under real load. Chapter 12 takes up that test, beginning with this chapter's ResNet-50 workload, whose predicted roofline position becomes the first claim a measurement must confirm. It asks which local gains survive deployment-representative conditions, and it supplies the measurement scope, standardized suites, and statistical protocols that let different systems be compared on the same terms.

12

Benchmarking

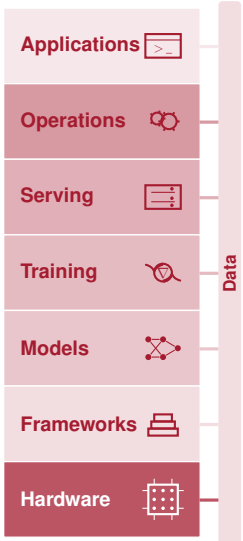


12.1 The Benchmark–Production Gap
12.2 Benchmark Carpentry
12.3 Benchmarking Granularity
12.4 Benchmark Suites
12.5 Training Benchmarks
12.6 Inference Benchmarks
12.7 Power and Energy Measurement
12.8 Benchmarking Best Practices
12.9 Model and Data Evaluation
12.10 Fallacies and Pitfalls
12.11 Summary

Purpose

How can ML systems be compared fairly when hardware, models, data, and deployment all interact?

Every optimization claim must eventually face deployment-representative conditions, and many do not survive the encounter. Each decision targets one dimension (latency, accuracy, throughput, or energy), but an ML system is the product of all these dimensions simultaneously: a pruned model may run faster on one accelerator yet slower on another, while a larger batch size improves accelerator utilization at the cost of violating a latency service-level agreement. Similarly, an edge device may advertise peak throughput that thermal throttling sharply reduces within minutes of sustained load, and a speedup measured on a warm, idle machine with synthetic inputs can vanish or reverse once real requests, cold caches, and competing jobs share the hardware. The challenge is not whether a local metric improves but whether the combined system improves under conditions that actually matter. Benchmarking makes such comparisons systematic rather than anecdotal. It requires defining what to measure, at what granularity (a single kernel, a full model, an end-to-end pipeline), and under which conditions (batch size, input distribution, thermal state, concurrent load). Without this structure, teams compare numbers that were never measured on the same terms, and decisions that looked sound in a spreadsheet collapse under production workloads. Choosing hardware, a model, or a deployment configuration on such numbers can commit an organization to millions of dollars of infrastructure that delivers a fraction of the capacity it was bought to provide. Benchmarking validates each optimization, bringing claims into contact with evidence and quantifying the gap between promise and delivery. In D·A·M terms, benchmarking holds co-design to account by revealing whether Data, Algorithm, and Machine were matched or merely assembled.



Learning Objectives

- Explain the benchmark–production gap and benchmarking as D·A·M validation that tests whether optimization claims hold under representative conditions
- Design a benchmark that turns an optimization claim into a test by fixing its task, data, model, metrics, harness, and run rules
- Select micro, macro, or end-to-end granularity based on the engineering decision being tested, and compare a roofline prediction with a measurement
- Compare training and inference benchmarks using standardized run rules, time to quality, throughput, and latency percentiles
- Measure energy per inference and average power with matched measurement boundaries, windows, and duty cycles
- Determine whether a measured difference exceeds noise using repeated runs, paired tests, and confidence intervals
- Evaluate model and data quality with confidence calibration, robustness, representativeness, and slice-level metrics, and diagnose benchmark–production gaps caused by drift, thermal throttling, and dynamic load

12.1 The Benchmark–Production Gap

A model quantized to INT8 may benchmark 2× faster on a synthetic workload and show no improvement under real traffic with variable input sizes and concurrent requests; a pruned model may hold its accuracy on the test set and fail on the edge cases the benchmark never covered. Every engineering claim about speed, size, or energy is a number measured somewhere: on a warm accelerator or a cold one, at a batch of 256 or a batch of one, on curated test images or on whatever a camera returns at night. The **benchmark–production gap** is the difference between how a machine learning system performs under the conditions a measurement fixed and how it performs under the conditions it meets in deployment. Benchmarking is the engineering discipline that measures this gap before a deployment pays for it, and that narrows it by making the measured conditions match the deployed ones.

Chapter 11 ended on predictions. Its roofline placed ResNet-50's layers against an accelerator's compute and bandwidth ceilings, and at INT8 the doubled ridge point moved some of those layers off the compute roof. Before it, chapter 10 produced an INT8 artifact a quarter of the FP32 size, and chapter 9 reached target accuracy on fewer examples. Each is a favorable local number, computed for one stage in isolation, and the stages interact once they run together on a real machine under real load. Benchmarking measures which of those numbers survive. It comes before serving and operations because every later decision, from how many accelerators to provision to when to roll a model back, rests on measurements made the way this chapter prescribes.

Definition 12.1: Machine learning benchmarking

Machine learning benchmarking is the empirical measurement of ML workloads under specified conditions, used to test claims about components, models, or end-to-end systems against representative evidence rather than peak specifications alone.

1. **Significance**: The gap between peak and sustained performance can be large. An H100 delivers 989 TFLOP/s of dense FP16 arithmetic at peak, yet ResNet-50 training at the MLPerf per-accelerator throughput of about 5,000 images/s sustains about 123 TFLOP/s of model arithmetic, a model FLOPs utilization (MFU, achieved model FLOP/s divided by the hardware's peak FLOP/s) of about 12.4 percent. Memory-bound layers, the input pipeline, and launch and synchronization overhead account for the difference. Benchmarking quantifies the actual η_{hw} gap for a workload; vendor spec sheets do not.

2. **Distinction**: ML benchmarking spans multiple scopes. Micro-benchmarks isolate operations such as matrix multiplication, model-level benchmarks evaluate complete training or inference workloads, and end-to-end benchmarks include surrounding work such as data loading, preprocessing, optimization, checkpoint I/O, or serving infrastructure. The scope must match the engineering claim.
3. **Common pitfall**: A frequent misconception is that benchmark numbers are stable references. Both the workload (new model architectures) and the hardware (new GPU generations) evolve, so a result that leads a benchmark under one version often becomes the baseline under a later version, making year-over-year comparisons meaningful only when the benchmark version is held constant.

The iron law predicts how long a workload should take, and a benchmark measures how long it does; the difference between the two is the benchmark–production gap made quantitative. For bandwidth-bound workloads, the gap between peak and sustained performance follows from the memory wall. Compute-bound workloads can approach peak, yet ResNet-50's training step shows how much of a nominally compute-bound workload still falls outside the arithmetic. Every later section that quotes a peak number is measured against this gap.

↩ The iron law, whose utilization term η_{hw} benchmarks measure against peak, is established in section 1.7.

Systems Perspective 12.1: The fallacy of peak performance

A long-standing computer architecture quip calls peak performance "the performance the manufacturer guarantees you will not exceed." For ML systems, the gap can be wide because of the memory wall. A bandwidth-bound model leaves much of the advertised FLOP/s unreachable no matter how fast the arithmetic units run. Standardized benchmarks like MLPerf require systems to run specified models and data, revealing sustained performance at a documented operating point rather than a universal rate for every workload.

ML benchmarking operates across three interdependent dimensions that map directly to the components of any deployed system. **System benchmarking** measures whether the hardware delivers promised performance under realistic workloads or whether memory bandwidth saturation and software dispatch overhead erode the gains. **Model benchmarking** measures whether optimization techniques preserve model quality across the full input distribution, not just on curated test sets. **Data benchmarking** measures whether the model generalizes to real-world data with all its noise, bias, and distributional shift. Each dimension can independently reveal problems invisible to the others, and a system that passes all three provides far stronger deployment confidence than one evaluated along any single axis. The three dimensions do not receive equal space. System benchmarking carries most of the chapter, because its measurement problems (how a benchmark is built, at what granularity it measures, which standardized suite it follows, and how it measures training, inference, and power) also shape how models and data are evaluated. Model and data evaluation follow once that machinery is in place, and the summary asks whether all three survive live traffic.

Systems Perspective 12.2: Benchmarks as moving targets

In traditional systems (for example, SPEC CPU), the benchmark is a *rigid specification*. A sorting algorithm is correct if it sorts the list. Correctness is absolute and unchanging. In ML systems, the benchmark is a *soft specification*: correctness is defined by a finite set of examples (ImageNet), and the world moves. A model that scores highly on ImageNet can still underperform on user photos taken years after the benchmark was created.

In computer architecture, engineers design for the benchmark because the benchmark represents the workload. In ML engineering, designing solely for the benchmark is overfitting. Robustness comes from acknowledging that the benchmark is only a proxy for a shifting reality.

Two lighthouse workloads carry the chapter, one per half, each chosen because it sits in a home regime and arrives with claims from the preceding chapters. ResNet-50, the book's compute-bound lighthouse, carries the measurement and training half. It is the MLPerf Training reference workload, with a published quality target (75.9 percent top-1 on ImageNet) and a published per-accelerator throughput, and its INT8 profile is the one chapter 10 modeled and chapter 11 placed on the roofline. MobileNetV2, compute-bound in its efficient regime, carries the deployment half. It runs on one edge board in a labeled composite scenario, a wildlife camera trap whose low-power host CPU and attached INT8 accelerator must classify each frame within its deadline on battery power, and the measurements made on that board recur from the harness through the energy budget. Keyword spotting (KWS), the power-bound lighthouse, carries the power section, where the question is the energy a wake-word detector spends listening rather than how fast it answers. Table 12.1 lists each workload's operations and bytes, the quantities every later benchmark either confirms or refutes.

Table 12.1: Lighthouse Operations and Bytes: Forward-pass operations per input and the bytes each workload occupies as FP32 weights, INT8 weights, and full mixed-precision training state, so that every later measurement can be checked against what the workload must compute and move. Activations and runtime buffers are excluded.

Workload	Parameters	Forward FLOPs	Weights (FP32)	Weights (INT8)	Training state (16 B/param)
ResNet-50	25.6M	8.2 GFLOP	102.4 MB	25.6 MB	409.6 MB
MobileNetV2	3.5M	600 MFLOP	14 MB	3.5 MB	56.1 MB
KWS (DS-CNN)	200K	20 MFLOP	800 KB	200 KB	3.2 MB

The table already separates the halves of the chapter. ResNet-50 does 13.7× MobileNetV2's arithmetic per image, so its training benchmarks are about whether an accelerator sustains its rate; MobileNetV2's INT8 weights fit in a few megabytes, so its deployment benchmarks are about latency, sustained thermal state, and energy on a small board.

↩ MobileNetV2, the lighthouse model for edge latency, is established in section 6.3.5.



Lighthouse 12.1: MobileNetV2 deployment validation

MobileNetV2 serves as the lighthouse example for validating the complete optimization pipeline. MobileNetV2 refines v1's depthwise separable design with inverted residuals and linear bottlenecks while maintaining a similar parameter scale (Sandler et al. 2018). It exemplifies the deployment challenges where benchmarking determines success or failure: compression can reduce model size, hardware acceleration can reduce inference latency, and only benchmarking can determine whether those gains survive the full deployment pipeline.

1. **Model compression** (chapter 10): INT8 quantization reduces this MobileNetV2 worked example from 14 MB to 3.5 MB (4× compression)
2. **Hardware acceleration** (chapter 11): in this illustrative scenario the board's attached INT8 accelerator runs inference in 2 ms against 15 ms on its host CPU
3. **Benchmarking validation**: Verify the pipeline delivers in practice

A systematic evaluation isolates the accelerator's latency from preprocessing and data transfer overhead, confirms INT8 quantization preserves accuracy on edge cases such as unusual lighting, and verifies that throughput holds on the camera trap's own frames rather than on curated ImageNet test images alone.

Each stage of that pipeline arrives with a favorable number from its own chapter, such as a smaller model file or a faster kernel. The benchmarking question is which of those local numbers still hold once the stages run together on the target device, and three principles guide how the chapter answers it.

First, *benchmarks are proxies, not truth.* Every benchmark measures specific conditions that may not match the target deployment. A system can achieve high sample throughput in Offline mode (bulk throughput with all inputs available) and much lower queries per second (QPS) in Server

mode (latency-constrained requests arriving over time). The critical question is always what the benchmark does not measure.

Second, Goodhart's Law governs benchmark rankings.[1] "When a measure becomes a target, it ceases to be a good measure." Teams that optimize for benchmark rankings often produce systems that excel in evaluation but fail in production. Benchmark-specific optimizations frequently degrade characteristics that matter for deployment: robustness, calibration, and efficiency.

Third, end-to-end beats component metrics. A component speedup is diluted by every stage of the request it does not touch, and the dilution follows Amdahl's law; section 12.3 quantifies it on a vision pipeline. These principles reappear throughout the methodology and return as misconceptions in section 12.10.

Two terms recur from here on. A service-level objective (SLO) is the target a deployed service commits to meet for a measured quantity, such as answering 99 percent of requests within a stated time; section 12.6.2.1 defines it formally. The 99th-percentile latency (p99) is the response time that 99 percent of requests meet or beat, so it describes the slowest 1 percent of requests rather than the typical one.

The chapter builds the measurement before it applies it (table 12.2). Benchmark carpentry turns a claim into a test that fixes the task, data, model, metrics, harness, and run rules. Granularity decides how much of the system a test spans, from one kernel to the whole request path, and it is where the roofline's prediction for ResNet-50 meets a measurement. Benchmark suites standardize those choices across organizations. The training and inference sections apply them to ResNet-50 and to MobileNetV2 on its edge board, and power measurement adds the joules, with keyword spotting as the case. Best practices supply the statistics that decide whether a measured difference is real, and model and data evaluation test the two dimensions that a hardware result cannot certify. The fallacies and the summary close by asking whether the evidence survives production traffic.

[1] **Goodhart's law**: Goodhart (1984) articulated the original 1975 Bank of England observation on monetary policy; Strathern (1997) generalized it into the form quoted above. The original context was macroeconomics: once a monetary aggregate became an official policy target, banks changed behavior to game the metric, destroying its predictive value. In ML, the same failure mode recurs structurally: BLEU rewards n-gram overlap (Papineni et al. 2002), ImageNet rewards performance on a fixed visual distribution (Deng et al. 2009; Recht et al. 2019), and benchmark leaderboards can incentivize test-set-specific tuning.

Table 12.2: Chapter Map: Each section turns one optimization claim into a measurement, names the granularity and unit the measurement uses, and names the trap that a careless version of the measurement falls into.

Section	Claim it tests	What is measured	Granularity	Unit	Trap it guards against
Benchmark Carpentry	ResNet-50 at INT8 is faster than at FP32	The claim restated as task, data, model, metric, harness, and run rules	Set by the claim	Stated per metric	A number reported without its conditions
Benchmarking Granularity	A kernel speedup speeds up the request	Kernel, model, and request-path time; predicted against measured roofline	Micro, macro, end to end	s, FLOP/s, B/s	A component speedup diluted by untouched stages
Benchmark Suites	Two vendors' numbers are comparable	A reference workload under shared divisions, scenarios, and run rules	Model and system	Time to quality, QPS	Favorable conditions chosen separately by each vendor
Training Benchmarks	More accelerators train a model proportionally faster	Time to target accuracy, throughput, scaling efficiency, MFU	Model on one to eight accelerators	s, images/s, percent	Throughput that never reaches the quality target
Inference Benchmarks	MobileNetV2 meets its frame deadline on the board	Latency percentiles, throughput, memory, cold start	End to end on one board	ms (p50, p99), FPS, B	A mean that hides the tail; a burst that hides throttling
Power and Energy Measurement	The device lasts on its battery	Energy over a matched window; average power across the duty cycle	Device or system boundary	J per inference, W	Efficiency quoted at an operating point the device never holds
Benchmarking Best Practices	A measured difference is real	Repeated runs, paired tests, confidence intervals	Any	Interval	A one-point gap inside sampling noise
Model and Data Evaluation	The compressed model is still right on deployed data	Confidence calibration, slices, coverage, label quality, shift	Model and dataset	ECE, percent per slice	Aggregate accuracy that hides calibration and subgroup loss

Knowing *what* to measure is only half of the problem. Measuring incorrectly, with the wrong workload, a biased baseline, or an uncontrolled variable, produces numbers that look precise and mislead the decision they inform. The history of computing benchmarks[2] is full of technically sound metrics applied with flawed methodology, from compiler-gamed Whetstone scores to throughput claims that predict nothing about sustained workloads (section 12.4.1). The defense is construction. A benchmark must be built so that it measures what its claim says, and that construction is where the chapter begins.

[2] **Benchmark**: From surveying, where a "bench mark" was a horizontal cut in stone serving as a fixed elevation reference. The term entered computing in the 1970s to describe standardized comparison points, and the surveying metaphor carries a systems lesson. Just as an elevation measurement is meaningless without a calibrated reference, an ML throughput number is meaningless without controlled workloads, thermal state, and precision settings.

12.2 Benchmark Carpentry

A claim that ResNet-50 runs faster at INT8 than at FP32 can reverse depending on whether timing starts at a JPEG on storage or at a preprocessed tensor in accelerator memory, or whether the quantized weights drop accuracy below the target floor. **Benchmark carpentry** is the craft of building a benchmark that measures what its claim says. It fixes the task, data, model, metrics, harness, system specification, and run rules so that the number the benchmark produces answers the engineering question and no other. A synthetic matrix-multiplication loop, a ResNet-50 training run over ImageNet, and a full serving pipeline all need these specifications, and changing one component changes what the next can measure. The components follow the D·A·M taxonomy. The problem definition fixes the input representation (Data), the dataset and task objective bound the feasible models (Algorithm), and the target hardware's memory, energy envelope, and deadlines bound how the model may execute (Machine).

Evaluating an ML system requires coupling statistical task metrics with physical efficiency metrics. Quantization and pruning reduce parameter footprint and memory bus traffic, but they alter numerical representations and risk degrading task accuracy. A benchmark that records execution latency without statistical accuracy (such as area under the ROC curve (AUC) or classification error) rewards aggressive approximations that produce invalid outputs; conversely, reporting accuracy without latency or memory consumption conceals whether the model satisfies the target system's physical constraints.

The measurement harness and run rules provide the experimental controls that make these evaluations reproducible. The protocol follows the question being tested. A controlled steady-state experiment can use warm-up iterations, fixed processor clocks, and pinned host memory to isolate execution differences. A deployment benchmark instead retains the relevant dynamic voltage and frequency scaling (DVFS) policy, cold starts, and competing workloads, documenting their effects. Recording tail latency distributions (p95, p99) alongside the mean exposes delays hidden by averages. Each component is built here on the claim the chapter inherited, that ResNet-50 runs faster at INT8 than at FP32, and MLPerf Tiny's anomaly-detection benchmark at the end shows every component fixed at once on a small embedded model.

12.2.1 Turning a claim into a test

Every benchmark begins as a claim. The claim from the preceding chapters, that ResNet-50 at INT8 runs faster than at FP32 without unacceptable accuracy loss, is not yet testable, because it names neither the inputs, the output, nor the conditions under which faster must hold. Fixing those boundaries is the first act of carpentry. Without an explicit task specification, hardware and software optimizations cannot be evaluated on equivalent terms: altering an input resolution, a numerical datatype, or a latency deadline changes how much work the machine does. For ResNet-50 the input is a decoded 224×224 RGB image and the output is one of a thousand ImageNet class labels. Whether the task is natural language processing (Hirschberg and Manning 2015) or computer vision (Everingham et al. 2009; Lin et al. 2014), a complete problem definition establishes three specifications: an input specification, an output specification, and a performance specification.

The input and output specifications fix the data each run must move and define the exact scope of timed execution. The input specification fixes tensor dimensions, batching behavior, numerical representations (such as FP16 or INT8), and ingestion rates. For the ResNet-50 claim, it decides whether timing starts at a JPEG on storage or at a preprocessed tensor already in accelerator memory, a choice that can change the measured time more than the precision change being tested. Conversely, the output specification defines the termination boundary of the computation. Benchmarks frequently introduce measurement bias by timing only accelerator forward passes while ignoring

the CPU post-processing required to turn raw tensor outputs into usable decisions. A robust output specification defines whether the system emits raw logits, intermediate embeddings, or final class labels, accounting for any downstream thresholding or reduction kernels that consume host memory bandwidth.

The performance specification defines the operational envelope that governs whether an optimization is viable. Unconstrained throughput numbers are meaningless if statistical accuracy degrades below acceptable thresholds, and high average throughput cannot rescue a system that violates a latency service-level agreement (SLA). For the ResNet-50 claim, the performance specification couples a quality floor (MLPerf Training requires 75.9 percent top-1, and an INT8 deployment might permit at most one percentage point of loss against FP32) with hardware execution limits: a tail-latency ceiling, sustained throughput matching the arrival rate of images, and a resident memory footprint that leaves room for activations at the chosen batch size. By defining these three boundaries up front, the benchmark ensures that the later stages, from dataset construction to measurement harness configuration, evaluate system optimizations against the true operational constraints of deployment.

12.2.2 Task and dataset

A task definition is only as rigorous as the data used to evaluate it. Standardized datasets enforce identical evaluation conditions across competing approaches, preventing teams from cherry-picking private slices that obscure comparative weaknesses. In computer vision, ImageNet (Deng et al. 2024, 2009), COCO (Lin et al. 2014), and CIFAR-10 (Krizhevsky 2009) establish reference tasks across classification and detection; in natural language processing, SQuAD[3] (Rajpurkar et al. 2016), GLUE[4] (A. Wang et al. 2018), and WikiText (Merity 2016; Merity et al. 2016) fulfill equivalent roles by standardizing comprehension, classification, and language modeling across diverse textual domains.

Dataset selection is the first place a benchmark can lose contact with deployment reality. ImageNet's validation set fixes ResNet-50's quality target, but its curated, centered photographs are not the frames a camera trap returns at dusk, a gap that section 12.9.2 returns to. A benchmark dataset must be representative enough to expose deployment failures and hard enough to separate models.

12.2.3 Model and metrics

With the task and dataset fixed, the benchmark must specify the model architecture and its baseline implementations. In systems benchmarking, the model defines the execution workload. Its computational graph determines arithmetic intensity, memory footprint, and communication topology. Selecting the benchmark model determines whether measured performance variations reflect architectural efficiency, software runtime optimizations, or raw accelerator capacity. Evaluating an architectural modification requires holding the runtime and hardware target constant. Conversely, evaluating hardware platforms or software execution engines requires standardizing on a fixed, representative model graph so that throughput differences isolate execution efficiency rather than mathematical changes to the workload.

Baseline models establish the empirical reference point for a target domain. Rather than toy algorithms, systems benchmarks rely on canonical workloads whose computational profiles and memory access patterns are well-characterized. In vision, ResNet-50 plays this role, which is why the chapter's claim is stated on it. In language processing, BERT[5] serves as a standard baseline because its multi-head self-attention and dense feed-forward blocks exercise both compute-bound matrix multiplications and memory-bandwidth-bound reduction operations. However, a baseline specification cannot stop at the mathematical graph. Two implementations of the identical architecture across different frameworks or runtimes often behave differently at runtime. Differences in kernel fusion (such as fusing normalization and activation functions directly into general matrix multiply (GEMM) output epilogues to avoid round-trips to high-bandwidth memory (HBM)), kernel launch overhead, and memory allocator behavior can exceed the performance impact of minor architectural variations. A comparison that isolates hardware must control the runtime implementation alongside the mathematical model; a runtime comparison instead names that implementation as the intervention and documents both stacks.

A benchmark must select the operational regime relevant to its engineering question. In training, memory capacity and interconnect bandwidth can be binding constraints, because storing forward activations, backward gradients, and optimizer states across multiple accelerators limits batch

[3] **SQuAD (Stanford question answering dataset)**: Introduced in 2016 with more than 100,000 question-answer pairs from Wikipedia (Rajpurkar et al. 2016). AI systems exceeded the SQuAD 1.1 human baseline of 91.2 percent F1 by 2018 (Devlin et al. 2019), but the "superhuman" result reflects a benchmarking failure mode. The task's extractive format (answers are text spans within the passage) makes it easier than open-ended question answering, inflating perceived capability relative to production NLP systems.

[4] **GLUE (general language understanding evaluation)**: Introduced in 2018 as a broad language-understanding benchmark (A. Wang et al. 2018), GLUE was quickly saturated by systems such as BERT (Devlin et al. 2019). Once GLUE became a target, leaderboard optimization reduced its discriminating power, as Goodhart's Law predicts. The pattern motivated harder follow-on evaluations such as SuperGLUE and BIG-bench.

[5] **BERT (bidirectional encoder representations from transformers)**: BERT-Large (340M parameters) is a language-processing workload in MLPerf Inference. The benchmark fixes the task, data, quality target, and applicable execution scenarios so systems can be compared on the same workload. BERT latency still depends on sequence length, batching, and implementation.

size and demands high-bandwidth collective communication. Training benchmarks therefore measure time-to-accuracy under fixed convergence criteria rather than raw iteration speed alone. In inference, backward passes and optimizer states disappear. Evaluation instead emphasizes the serving objective, such as tail latency under SLAs, while low-batch autoregressive decode often faces bandwidth pressure. Transitioning a model to production frequently requires precision reduction, quantizing FP32 or BF16 weights and activations to INT8 or FP8. Moving eligible tensors from a 16-bit to an 8-bit format halves their raw storage and can reduce traffic or raise supported arithmetic throughput. Whole-model gains depend on the execution path, and validation must establish whether numerical error stays within the quality target. A development-to-deployment comparison may need both regimes because efficient training does not establish serving feasibility; a deployment decision for a fixed pretrained model can focus on inference.

Evaluation metrics[6] translate raw model behavior into numbers that can be compared, ranked, and used to make engineering decisions. The difficulty lies in choosing the right numbers. A metric that captures accuracy but ignores latency may declare the winner to be a model too slow for production; one that rewards throughput but ignores energy may optimize for a deployment budget that does not exist. Table 12.3 categorizes evaluation measures by the failure mode each exposes and the deployment context it serves.

[6] **Metric**: In mathematics, a metric is a distance function satisfying strict axioms including the triangle inequality. ML borrows the term loosely for quantitative measures such as BLEU and perplexity, which are scoring rules rather than mathematical metrics. Leaderboard rankings can change when the evaluation protocol, dataset slice, or metric weighting changes, making the choice of metric an engineering decision that shapes which system wins, not just how performance is measured.

Table 12.3: ML Benchmarking Metric Taxonomy: Metrics organized by evaluation category, unit, and primary use case. Accuracy metrics quantify model quality, throughput and latency metrics capture system speed, and efficiency metrics combine multiple dimensions. Selecting the right metric for the deployment context is often more important than optimizing any single metric to its maximum.

Category	Metric	Unit	Primary Use Case
Accuracy	Top-1/Top-5 Accuracy	Percentage	Classification
	mAP (mean Average Precision)	0–1 score	Object detection
	BLEU/ROUGE	0–100 score	NLP generation
	Perplexity	Score (lower = better)	Language modeling
Throughput	Samples/second	Samples/s	Batch inference
	Token throughput	tokens/s	LLM inference
	Time-to-train	Hours/days	Training benchmarks
Latency	p50 latency	Milliseconds	Median response time
	p99 latency	Milliseconds	Tail latency (SLA)
	First-token latency	Milliseconds	LLM responsiveness
Efficiency	Samples/second/watt	Samples/s/W	Energy efficiency
	Accuracy/FLOP	percent/PFLOP	Algorithmic efficiency
	TCO per inference	$/inference	Economic efficiency

The metrics in table 12.3 expose physical trade-offs governed by the accelerator architecture. Throughput measures aggregate processing capacity, whereas latency measures individual request completion time. These two objectives conflict directly at the hardware boundary. Maximizing throughput requires aggregating requests into larger batches to amortize weight-loading memory bandwidth across multiple inputs, raising arithmetic intensity on tensor units. That batching mechanism inflates per-sample latency, because individual requests wait in arrival queues while a batch forms, and larger matrix operations take longer to complete. Furthermore, arithmetic mean latency masks tail behavior. Queuing jitter, memory allocation stalls, and accelerator thermal throttling cause execution times to diverge; a system with a 10 ms mean latency can still suffer a 500 ms p99 latency that violates service-level agreements. Benchmarks must therefore report distribution percentiles (p50, p95, p99) rather than averages. Compound metrics such as samples/second/watt compress multiple dimensions into a single scalar, simplifying high-level comparisons but obscuring where bottlenecks reside. Reporting atomic metrics (latency distributions, throughput, and power draw) alongside compound figures isolates which physical constraint limits the system.

↪ The batch size at which waiting for a batch to form pushes latency past its budget is measured with a batch-size sweep in section 13.5.2.

Task-specific metrics evaluate the algorithmic dimension, quantifying whether the model solves the statistical problem. Classification workloads evaluate accuracy (the fraction of correct predictions), precision, recall, and the F1 score (Sokolova and Lapalme 2009). Precision and recall isolate true positive rates from false alarms, which is essential when class distributions are imbalanced.

Continuous estimation tasks rely on Mean Squared Error (MSE) or Mean Absolute Error (MAE), while sequence generation uses specialized scoring rules such as BLEU[7] to measure modified n-gram precision against human reference translations (Papineni et al. 2002).

Task metrics evaluate algorithmic quality in isolation, but deployment viability depends on physical machine constraints. Model capacity, measured in parameter count and memory footprint, determines whether weights fit into on-chip SRAM or accelerator memory, or whether they spill across high-latency interconnects. Per-inference execution latency dictates whether a system meets real-time deadlines, while energy consumption per inference (measured in joules or millijoules) bounds thermal dissipation and battery life.

Benchmarks must also account for measurement sensitivity across software runtimes. As established in chapter 8, different frameworks handle loss computation, kernel fusion, and gradient accumulation differently, altering reported benchmark numbers. Subtle implementation differences, such as whether batch normalization is configured to use stored running statistics or current mini-batch statistics during evaluation, or how non-associative floating-point additions are reordered in reduction kernels, shift measured accuracy and latency enough to invert benchmark rankings when candidate models differ by narrow margins.

7 | **BLEU (bilingual evaluation understudy)**: Introduced by IBM in 2002, BLEU measures translation quality through modified n-gram precision with a brevity penalty against reference translations (Papineni et al. 2002). BLEU is a canonical example of Goodhart's Law in ML, because optimizing for n-gram matches can reward surface-level word overlap even when meaning, fluency, or deployment usefulness diverges from the target.

12.2.4 Benchmark harness

Metrics define *what* to measure; the benchmark harness determines *how* to measure it. The harness is the software and instrumentation scaffold that feeds inputs to the system under test, coordinates execution, collects timing and power traces, and enforces reproducibility. In an ML system, measuring execution down to the metal requires accounting for the asynchronous execution boundary between the host CPU and the accelerator. When host software initiates an inference or training step, the framework runtime merely enqueues commands onto an accelerator work queue (such as a CUDA stream or hardware command buffer) and returns control immediately. A naive timer wrapping the host-level call measures only this sub-microsecond queue dispatch latency rather than actual device execution. A rigorous harness inserts explicit device synchronization barriers (such as `torch.cuda.synchronize()`) or queries on-device timestamp events (such as `torch.cuda.Event`), ensuring that recorded timings reflect complete kernel execution and memory transfers across the bus rather than mere host enqueue overhead.

Harness architecture must align with the operational regime of the deployment target. In server environments, the harness generates concurrent request traffic using an open-loop load generator, often parameterizing inter-arrival times with a Poisson distribution.[8] An open-loop harness issues requests according to an independent arrival schedule regardless of server responsiveness. This decoupling exposes queuing delays and tail latency (such as p99 response times) when arrival bursts saturate available compute.

In contrast, a naive benchmarking client often employs a closed-loop pattern: sending a request, blocking until the response arrives, and only then issuing the next request. This design introduces the severe measurement distortion known as **coordinated omission**. Consider a server that experiences a transient 500 ms garbage collection pause or memory reallocation stall. In an open-loop production environment, independent users continue submitting queries, causing dozens of requests to accumulate in the server's ingress queue; once the pause ends, every queued query suffers inflated tail latency. In a closed-loop benchmark, however, the client blocks idly during the entire 500 ms pause, sending zero traffic while the server is struggling. The benchmark records a single slow query and zero queue backlog. By coordinating request generation with server responsiveness, the closed-loop harness artificially paces traffic, systematically erasing the very queue collapse and p99 latency spikes it was constructed to measure. Conversely, offline batch processing eliminates arrival scheduling entirely; the harness pre-stages large input batches in memory to saturate accelerator execution units, maximize memory bandwidth utilization, and measure peak sustained throughput.

In embedded and edge deployments, workloads are neither stochastic network streams nor unconstrained batch queues. Instead, inputs arrive sequentially from physical sensors (such as an audio analog-to-digital converter or an image sensor) governed by fixed hardware sampling clocks. The harness must inject these sensor streams through direct memory access into limited on-chip SRAM buffers, verifying that each inference completes within the inter-sample arrival deadline to prevent buffer overflow. Furthermore, edge harnesses must integrate physical instrumentation, such

8 | **Poisson distribution**: Named after Siméon Denis Poisson, who formalized it in 1837 while modeling wrongful conviction rates in French courts. The distribution models independent events at a constant average rate (λ_{arr}) and is a common baseline for server request arrivals. Real ML serving traffic often violates its assumptions through burstiness, correlation, and time-varying rates. A Poisson harness can therefore misestimate tail latency unless traces or stress cases cover those production patterns.

as shunt resistors or external power analyzers, to sample voltage and current across active execution cycles and low-power sleep states, capturing the true energy consumed per inference.

Reproducibility requires specifying which runtime and environmental conditions are controlled and which are part of the workload. Warm-up matters at every scale, from a single kernel to a whole run, because modern accelerators and runtime frameworks have substantial cold-start transients. Initial inference passes trigger on-demand memory allocations, dynamic just-in-time (JIT) compilation, driver initialization, and weight transfers from secondary storage into accelerator memory. For steady-state measurements, the harness executes unmeasured warm-up iterations until caches populate, memory allocators stabilize, and the intended thermal operating state is reached. Controlled experiments may fix clocks and remove competing processes; deployment tests retain and report the relevant DVFS policy and contention. Cold-start measurements time initialization separately instead of discarding it. Finally, telemetry logging must minimize the probe effect by recording execution timestamps and performance counters into non-blocking, pre-allocated memory buffers. The remaining instrumentation overhead should be checked against a minimally instrumented run.

12.2.5 Specifications and run rules

Complementing the harness that controls execution conditions, system specifications record the computational environment, the physical hardware and software stack that establishes the Machine (M) boundaries of the system, and run rules govern how the benchmark runs on it. Without both, a reported throughput or latency measurement cannot be interpreted. An execution metric reflects the interaction between an algorithm's arithmetic intensity and a machine's physical limits: peak arithmetic throughput, memory hierarchy bandwidth, and interconnect latency. A throughput figure reported without accelerator model, memory bandwidth, or operating clocks conceals whether an observed speedup stems from algorithmic efficiency or upgraded silicon, and a result reported without its seed policy conceals whether it is reproducible at all, because the stochastic processes catalogued in section 12.4.2 mean that two runs on identical hardware can produce different numbers. A protocol may fix seeds and data order when isolating system effects, or require repeated runs and quality thresholds when stochastic variation is part of the workload. Table 12.4 lists what a reproducible report records.

Table 12.4: Benchmark Specification and Run Rules: What a reproducible benchmark report records about the machine, the software, and the run, and why each item changes the measured result.

Item	What the report records	Why the result depends on it
Accelerator	Microarchitecture, active core count, base and boost clocks, power limit (TDP)	Sets the peak arithmetic rate and how long boost clocks persist
Device memory	Capacity, bus width, peak bandwidth	Sets the bandwidth ceiling and the largest batch that fits
Multi-accelerator topology	Host interface (PCIe lanes) and accelerator links (NVLink or shared PCIe switches)	All-reduce time is bounded by link bandwidth and topology
Software stack	Operating system, driver, compute runtime (CUDA or ROCm), math libraries (cuBLAS, CUTLASS)	Kernel selection, tiling, and reduction order change with library version
Framework layer	Framework version, eager or compiled execution (`torch.compile`), fusion settings	Separates framework overhead from model cost
Power and thermal	Sustained board power, throttling events, energy per sample or token	A speedup from transient clock boosting does not persist
Randomness	Seed policy, data order, known nondeterministic kernels	Two runs on identical hardware can produce different numbers
Configuration	Every hyperparameter, including the learning-rate schedule	A learning-rate change shifts convergence and final accuracy
Data	Dataset version, splits, preprocessing; any privacy or licensing limit on sharing	Different splits or preprocessing define a different workload
Code	Version of model, preprocessing, training, and evaluation code; container image	Lets another team inspect and, where access permits, reproduce the run

Reference suites may distribute containerized environments that encapsulate dependencies and configurations, while experimental logs retain training metrics, checkpoints, and any mid-run adjustments. Together, these records turn a one-time measurement into evidence another team can inspect and, when access permits, reproduce.

12.2.6 Interpreting results

Producing raw benchmark numbers requires only execution; interpreting them requires understanding the conditions that produced them, the statistical confidence behind them, and the operational boundaries that determine whether the numbers translate to production behavior. A single reported metric, such as latency or accuracy, rarely reveals whether an optimization succeeds in deployment.

Translating benchmark numbers into engineering decisions requires two methodological checks: a defined intervention and statistical rigor. First, state what changes and what remains comparable. ResNet-50 versus MobileNet can be a valid architecture comparison at matched task and quality requirements; FP32 versus INT8 can be a valid precision comparison on a supported execution path. Hold the other relevant conditions fixed or report their changes, including batch size, hardware, runtime, input distribution, and service targets. Second, distinguish steady-state execution from initialization, using the harness's warm-up protocol (section 12.2.4). A single run conflates cold-start overhead (kernel compilation, page faults, and cache warming) with sustained throughput, while a prolonged stress test may instead capture the thermal throttling examined for edge devices in section 12.6.4. Report cold-start and steady-state results according to the deployment question, and record variation across repeated trials. Whether a measured difference exceeds that variation is a statistical question, answered with the interval and paired-test rules of section 12.8.1.

> Systems Perspective 12.3: Interpreting a benchmark claim
>
> A vendor claims "Our system achieves 10,000 inferences/second on ResNet-50." Taken alone, the number cannot support deployment planning because the conditions that determine its meaning are unstated.
>
> Four unstated conditions determine what the claim means:
>
> 1. **Batch size**: Large batches amortize weight transfers and maximize compute utilization but inflate queuing delay; batch 1 minimizes latency but leaves execution units underutilized.
> 2. **Precision**: INT8 execution exploits higher-throughput integer pipelines and cuts memory traffic, but requires quantization-range calibration that can compromise numerical fidelity.
> 3. **Measurement boundary**: The measurement window must state whether it isolates pure accelerator kernel execution or includes host-to-device transfers, JPEG decoding, and tensor format conversions.
> 4. **Accuracy**: The claim must state whether the optimized model retains baseline validation accuracy (such as 76.1 percent Top-1 on ImageNet) or trades task quality for raw execution speed.
>
> **Example**: "10,000 inferences/second on ResNet-50 at batch size 32, INT8 precision, 76 percent Top-1 accuracy, including JPEG decoding, on NVIDIA H100 at 700 W thermal design power (TDP)."
>
> **Systems insight**: A throughput figure lacking batch size, precision, measurement scope, and accuracy targets is a marketing metric rather than an engineering specification.

A claim whose workload boundary, precision, and excluded costs cannot be reconstructed should be rejected. Marketing phrases compress such omissions into a few words, usually reporting the most favorable operating point without saying so; table 12.5 translates common ones into the technical caveats behind each.

Table 12.5: Decoding Vendor Benchmark Claims: Four common marketing phrases and the technical caveats that determine whether a claim describes end-to-end performance, a narrow accelerator measurement, or an unsupported efficiency comparison.

Vendor Claim	What It Often Means
"Up to 10,000 images/sec"	Peak throughput at maximum batch size, INT8, without preprocessing
"Sub-millisecond latency"	Accelerator compute only, excluding data transfer
"5× more efficient"	Per-operation efficiency, not total system efficiency
"Optimized for AI"	May only accelerate specific operations or precisions

Which metric governs viability depends on the deployment. In safety-critical applications such as medical imaging or autonomous driving, a 1 percent accuracy gain justifies substantial computational cost; in interactive search or speech transcription, tail latency within a tight service-level agreement dominates, so a minor accuracy gain is irrelevant if it breaches the latency budget. Systems evaluation must also guard against *benchmark overfitting*, in which an optimization targets the idiosyncratic computational patterns of a standardized suite at the expense of production workloads. For example, aggressive kernel fusion or operator tiling tuned exclusively for fixed sequence lengths in a benchmark harness can degrade throughput when deployed against dynamic, variable-length user requests. Robust interpretation requires validating performance across diverse input distributions, perturbed batch sizes, and representative operational duty cycles.

Checkpoint 12.1: Decoding vendor benchmark claims

When evaluating hardware or software based on vendor-reported benchmarks, check whether the claim identifies the workload, measurement boundary, and operating conditions.

- ☐ Measured quantity: Does the latency number include preprocessing and postprocessing, not just model execution?
- ☐ Precision boundary: Can the precision behind a throughput claim, such as INT4 versus INT8 or FP16, be identified?
- ☐ Workload boundary: Are the model, batch size, and input distribution used to produce the comparison explicitly named?
- ☐ Excluded costs: Are memory transfers, model loading, initialization, sustained thermal behavior, and power accounted for at the claimed performance level?
- ☐ Claimed vs. achieved: Given a 1,000 TFLOP/s peak claim and a benchmark that sustains 350 TFLOP/s, estimate the MFU and state whether that utilization ratio alone identifies the bottleneck.

12.2.7 A complete reference benchmark

MLPerf Tiny's anomaly-detection benchmark shows every component fixed at once on one small embedded model. The multi-stage pipeline in figure 12.1 traces this industrial audio benchmark across nine sequential phases, carrying the task from problem definition in the top chevrons through spectrogram extraction, autoencoder training, and INT8 quantization down to embedded deployment on an ARM mbed OS microcontroller. Each stage fixes boundary conditions for the next: the problem definition dictates the acoustic sampling rate and input representation (Data); the dataset properties and task objective determine feasible model architectures (Algorithm); and the microcontroller's hardware limits (SRAM capacity, energy envelope, and real-time latency deadlines) dictate INT8 quantization and memory-efficient kernel compilation (Machine). Here every choice constrains the next, so a benchmark that measures only classification accuracy or only raw inference latency misses cross-stage interactions, where an engineering decision at any stage narrows the feasible design space downstream.

Its problem definition fixes the same three specifications that the ResNet-50 claim needed. The input is single-channel audio streamed at 16 kHz in fixed one-second windows, which sets a rigid ingestion rate and memory buffer requirement before feature extraction begins. The output is a

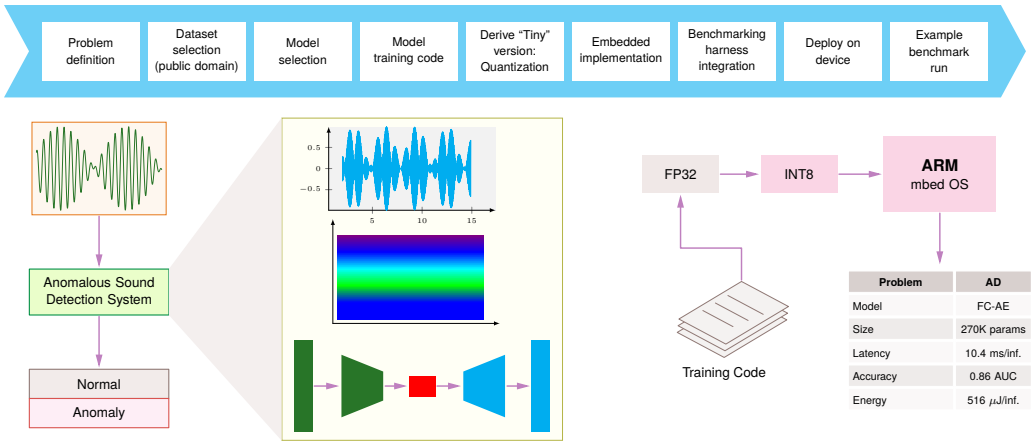


Figure 12.1: Benchmark Workflow Components: Read left to right: each stage fixes assumptions for the next, carrying an audio anomaly-detection task from problem definition to a reproducible embedded-device benchmark run. Values from C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kiraly, et al. (2021).

binary "Normal" or "Anomaly" flag, including any thresholding that turns a reconstruction error into that decision. The performance specification couples a quality metric, area under the ROC curve (AUC), with execution limits: a per-inference latency that keeps pace with the one-second input windows, sustained throughput matching the arrival rate, and a resident footprint that avoids out-of-memory faults on the microcontroller.

At the dataset selection stage (the second chevron), the benchmark pipeline ingests representative audio waveform samples of normal machine operation alongside anomalous conditions before spectrogram feature extraction. Domain-specific collections cover different audio tasks: ToyADMOS[9] (Koizumi et al. 2019) supports controlled anomaly-detection research, while Google Speech Commands (Warden 2018) supports keyword recognition. Simplified datasets like ToyADMOS are valuable for methodological development but may not capture the full complexity of production environments.

This interdependence between statistical quality and hardware constraints is evident in anomaly detection. In industrial monitoring, extreme class imbalance renders raw accuracy deceptive. If anomalies constitute only 0.1 percent of operating samples, a trivial model that constantly predicts normal operation achieves 99.9 percent accuracy while failing to detect any faults. The benchmark must instead report AUC, which scores how well the model ranks anomalous samples above normal ones across every decision threshold, alongside physical resource metrics.

This multi-metric evaluation approach governs the anomaly detection pipeline, which balances four simultaneous constraints: a model size of 270K parameters whose stored representation and runtime buffers must fit the embedded SRAM budget, a processing latency of 10.4 ms per inference to maintain real-time sampling, a detection accuracy of 0.86 AUC, and an energy consumption of 516 µJ per inference within the target battery budget. Evaluating across all four axes prevents declaring a model superior when its accuracy gain stems from an architecture too large to fit in memory or too power-hungry to survive sustained deployment.

The deployed anomaly detection workflow in figure 12.1 records the four measurements from section 12.2.3 in its terminal reporting table (lower right). For the fully connected autoencoder (FC-AE) on the ARM mbed OS target, each certifies something different. Parameter count and per-inference latency certify computational resources, the AUC certifies task effectiveness, and energy per inference certifies operational efficiency.

On a microcontroller, each metric governs a different failure: energy per inference sets battery life on untethered devices, model size decides whether weights fit in fast on-chip SRAM or spill into off-chip DRAM, and latency determines whether streaming audio is processed without dropped frames. The metrics trade against one another: reducing model size below 270K parameters can reduce memory traffic and execution time on a supported path, but risks degrading the 0.86 AUC detection accuracy. Whether these measurements constitute a passing benchmark depends on

[9] **ToyADMOS**: Developed by NTT Communications in 2019 for acoustic anomaly detection, containing audio recordings from toy car, toy conveyor, and related miniature-machine operating sounds (Koizumi et al. 2019). The "toy" prefix is intentional: the controlled environment enables reproducible benchmarking but can create a domain gap when models are moved to noisier industrial environments with different machines, sensors, vibration, and background sound.

the operational envelope; the benchmark specification standardizes the evaluation protocol, but application requirements dictate the acceptance thresholds.

The reference flow fixes the task, data, model, and measurement boundary, but it measures one model on one device at one scope. Before choosing that scope, a benchmark should pass three methodological checks.

Checkpoint 12.2: Benchmarking methodology

Three practices distinguish rigorous benchmarks from misleading ones:

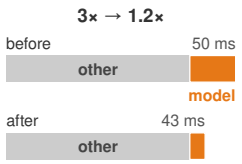
- ☐ Representative data: Compare the benchmark input distribution against the production distribution.
- ☐ Warm-up: State whether the goal is steady-state or cold-start performance, and which initialization work is timed or discarded.
- ☐ Isolation: Document the machine, runtime, and competing workloads; explain which controls isolate the intervention and which conditions represent deployment.

A benchmark that passes those checks has still made a second choice, often implicitly. It decides how much of the system it measures. A single kernel, the whole model, and the request path around the model each answer a different question, and each fails in a different way.

12.3 Benchmarking Granularity

A kernel that runs 3× faster in isolation may deliver no end-to-end speedup if the data pipeline cannot keep the accelerator fed. In an illustrative request, a 3× inference speedup applied to a 10 ms model stage inside a 50 ms request yields only about a 1.2× end-to-end improvement, or less if the optimization increases memory pressure. The dilution makes the measurement boundary a design choice. Carpentry specifies *how* a measurement is built, while **benchmarking granularity** specifies *what* physical boundary it measures. System validation spans multiple architectural scales, from individual tensor operations to complete production pipelines, with each granularity diagnosing distinct bottlenecks:

- Micro benchmarks isolate individual components: kernel execution time, memory bandwidth utilization, and single-layer numerical behavior. These diagnose *where* low-level hardware inefficiencies occur.
- Macro benchmarks evaluate composed model subsystems: full model training convergence, inference pipeline throughput, and dataset accuracy metrics. These reveal *what* architectural trade-offs exist across layer compositions.
- End-to-end benchmarks measure complete operational pipelines: client request-to-response latency including preprocessing, training time-to-accuracy including storage I/O, and model robustness on production data distributions. These show whether the combined system satisfies real-world service-level objectives.



Component speedup rarely survives as end-to-end benchmark speedup.

Optimization techniques operate across distinct granularities: kernel fusion targets micro-level execution, structured pruning alters macro-level parameter structures, and data curation determines end-to-end generalization. System validation must align with these respective boundaries. An isolated micro-benchmark may show substantial kernel speedup, yet a macro-benchmark can reveal that increased activation memory footprints force smaller batch sizes that degrade overall throughput. Similarly, an end-to-end benchmark exposes storage or host deserialization stalls that remain completely invisible at the accelerator kernel level.

Measurement perimeters widen from left to right in figure 12.2, and the figure inserts one tier between macro and end-to-end. Micro-benchmarks isolate kernel layers (ML Layers) and macro-benchmarks evaluate composed models (ML Model). An application benchmark then adds the supporting host compute around the model (AI Task), and the end-to-end benchmark adds the non-AI compute nodes the request also traverses. The three granularities in this section fold the application tier into end-to-end, but the four-tier view shows where bottlenecks migrate as the boundary widens: out of the accelerator and into the host systems around it.

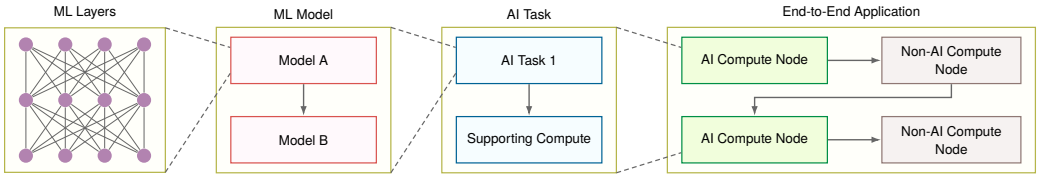


Figure 12.2: Benchmarking Granularity: Four-panel block diagram showing micro, macro, application, and end-to-end evaluation layers. Each panel maps a distinct scope of assessment, from isolated kernel operations through full-system deployment, enabling targeted optimization at every level of the ML stack.

12.3.1 Micro benchmarks

While end-to-end metrics govern service-level agreements, optimizing an accelerator pipeline requires isolating the specific mathematical operations that consume execution time and energy. Micro-benchmarks serve this diagnostic role by evaluating individual tensor operations in isolation, measuring the accelerator's hardware primitives directly. When profiling a slow inference service, macro-benchmarks report aggregate latency violations, but only micro-benchmarks reveal whether execution time is dominated by matrix multiplications, self-attention projections, memory copies across the host-accelerator interconnect, or elementwise activation kernels.

Tensor operations constitute the primary target of micro-benchmarking because matrix multiplications and convolutions dominate the arithmetic budget of deep neural networks. Highly optimized runtime libraries such as cuDNN[10] (Chetlur et al. 2014) provide hand-tuned kernel implementations tailored to specific microarchitectures. Micro-benchmarks also evaluate standalone activation functions (such as ReLU, Sigmoid, and GELU) and structural submodules (such as multi-head attention projections or transformer feed-forward blocks) across standardized input shapes. Benchmark suites such as Baidu's DeepBench (Baidu Research 2016) formalized this methodology by evaluating raw GEMM, convolution, and collective communication primitives across diverse hardware platforms, isolating silicon-level execution efficiency from higher-level framework dispatch overhead.

Measuring isolated kernels requires rigorous experimental control. Without disciplined measurement protocols, dynamic hardware frequency scaling, asynchronous execution queues, and cache state introduce transient artifacts that distort kernel runtime.

↩ The compute primitives that micro-benchmarks isolate, vector and matrix operations and special function units, are established in section 11.3.

10 **cuDNN (CUDA deep neural network library)**: Released by NVIDIA in 2014, cuDNN provides hand-tuned kernel implementations for convolutions, pooling, and normalization. The benchmarking implication: reported inference latencies depend heavily on which cuDNN version and algorithm autotuner settings were used, making cuDNN version a mandatory element of any reproducible benchmark specification.

> Systems Perspective 12.4: Micro-benchmarking rules
>
> Five rules keep a micro-benchmark measuring the kernel rather than a hardware artifact:
>
> 1. **The warm-up rule**: Do not measure cold-start iterations as steady-state performance. Modern hardware uses DVFS (dynamic voltage and frequency scaling) and dynamic frequency boosting; caches, kernels, and clocks need warm-up before the measured loop represents sustained behavior.
> 2. **The variance rule**: Report the coefficient of variation (CV) ($\text{CV} = \sigma_{\text{run}}/\mu_{\text{run}}$), where σ_{run} and μ_{run} are the standard deviation and mean across repeated runs. Investigate values above the protocol's workload-specific tolerance; common causes include background OS jitter, thermal throttling, and memory contention, and 5 percent is not a universal cutoff.
> 3. **The "speed of light" (SOL) check**: Compare the achieved throughput against the roofline. If a kernel achieves 10 TFLOP/s on an H100 (peak ~989 TFLOP/s FP16, or ~1,979 TFLOP/s FP8 dense), the diagnostic step is to identify the cause of low utilization (often kernel launch latency from too many small kernels) before optimizing the code itself.
> 4. **The cache-state rule**: Set cache state to match the claim. Flush or exceed the L2 cache when measuring cold dynamic random-access memory (DRAM) bandwidth; preserve a warmed cache when measuring steady-state reuse. Otherwise a nominal DRAM result may instead reflect cache bandwidth (~5 TB/s–10 TB/s) rather than DRAM bandwidth (~1 TB/s–2 TB/s).

5. **The synchronization barrier rule**: Accelerator kernel launches are asynchronous on the host CPU. Timing GPU operations using CPU clocks without an explicit device barrier (such as `torch.cuda.synchronize()`) measures host enqueue latency rather than kernel execution; always place synchronization barriers immediately before and after the timed region, or use GPU-side timestamp events (`cudaEventRecord`).

The roofline model[11] of section 11.6, which plots the bound of the arithmetic intensity law (principle 6), is a prediction, and a benchmark is the measurement that tests it. The roofline bounds where a kernel can sit; a benchmark determines where it does sit, and the gap between the two is the diagnostic. A kernel far below its predicted ceiling is losing time to something the model omits (launch overhead, poor coalescing, synchronization), which is exactly what profiling looks for. Making the prediction requires the workload's arithmetic intensity at a named memory interface and the target's matching compute and bandwidth limits. For an NVIDIA H100 with 989 TFLOP/s of dense FP16 Tensor Core performance (FP32 is 67 TFLOP/s)[12] and 3.35 TB/s of memory bandwidth, the ridge point (peak compute divided by peak bandwidth, the intensity at which the two ceilings meet) falls at 295 FLOP/byte. Below it, bandwidth sets the lower ceiling; above it, compute does (section 11.6.2 derives the threshold).

A profiler translates these measurement rules into physical diagnostics by decomposing kernel execution time into the three governing terms of the iron law: data movement, compute throughput, and latency overhead. Comparing the measured utilization of each resource against hardware limits reveals which term dominates the trace.

11 **Roofline model**: Williams et al. (2009) introduced the Berkeley model, named for the visual shape of its performance ceiling. Its ridge point (peak FLOP/s divided by peak bandwidth) separates bandwidth-limited from compute-limited ceilings, suggesting which interventions to test against measured execution.

12 **FLOP/s (floating-point operations per second)**: The gap between advertised peak FLOP/s and achieved FLOP/s is the central tension in hardware benchmarking. The H100 advertises 989 TFLOP/s of dense FP16 Tensor Core arithmetic, but real workloads achieve different fractions of peak depending on arithmetic intensity, memory access patterns, precision, and runtime overhead. Reporting peak FLOP/s without utilization context is the most common benchmarking distortion.

↪ The H100 ridge point, with a compute-bound matrix multiplication and a memory-bound element-wise kernel placed on either side of it, is collected for reference in section D.2.1.1.

Systems Perspective 12.5: Measuring the iron law terms

Moving from theory to trace means mapping the iron law equation onto a profiler timeline (like Nsight Systems or PyTorch Profiler).

Measuring the data term $\left(\frac{D_{\text{vol}}}{\text{BW}}\right)$

- **Signal**: Look for the "Memory Throughput" or "DRAM Bandwidth" line.
- **Formula**: $\text{BW}_{\text{effective}} = \frac{D_{\text{vol}}}{T_{\text{kernel}}}$.
- **Diagnosis**: If $\text{BW}_{\text{effective}} \approx \text{BW}_{\text{peak}}$ (for example, close to the H100's 3.35 TB/s peak), bandwidth pressure is a strong candidate bottleneck. Test whether reducing traffic or increasing bandwidth improves time; a high bandwidth reading alone does not establish that changing compute (O) has no effect.

Measuring achieved compute throughput $(R_{\textbf{peak}} \cdot \eta_{\textbf{hw}})$

- **Signal**: Distinguish "SM Active," the fraction of time the GPU's Streaming Multiprocessors (SMs), its parallel execution cores, have work resident, from executed-operation throughput. A busy multiprocessor does not imply high useful-model FLOP/s.
- **Formula**: $R_{\text{peak}} \cdot \eta_{\text{hw}} = \frac{O}{T_{\text{kernel}}}$, reported in TFLOP/s.
- **Diagnosis**: If $\eta_{\text{hw}} \ll 1$ AND $\text{BW}_{\text{effective}} \ll \text{BW}_{\text{peak}}$, neither quoted peak is being approached. Kernels too small to amortize launch latency or too few threads to occupy the SMs are candidate explanations, but low aggregate rates alone do not exclude dependency stalls or an inefficient memory-access path. Separate inter-kernel gaps from in-kernel stalls, then inspect the corresponding dispatch, dependency, occupancy, or memory-access evidence before selecting an optimization.

Measuring the latency term $(L_{\textbf{lat}})$

- **Signal**: Look for gaps (empty space) between colored kernel bars on the timeline.
- **Formula**: Kernel gap fraction $= \frac{T_{\text{gap}}}{T_{\text{kernel}} + T_{\text{gap}}}$.
- **Diagnosis**: A "Sawtooth" pattern (Compute, Gap, Compute, Gap) suggests exposed waiting; correlate CPU dispatch, dependencies, and transfers before attributing it to

software overhead. If dispatch dominates, candidates include kernel fusion, covered in section 11.7.7, or the CUDA Graphs of section 7.3.6, which capture a repeated sequence of GPU launches so the runtime can replay it with less CPU dispatch overhead.

Profilers supply the measurements this mapping needs. A framework profiler such as the PyTorch Profiler isolates the dominant layer on the step timeline, and a kernel profiler such as NVIDIA Nsight Compute explains it from on-chip performance counters, the top-down method developed on a training step in section 8.5. Counter collection and kernel replay perturb timing, so diagnostic traces are compared against a minimally instrumented run. The benchmarking use is narrower, confirming or refuting a prediction.

↩ The feeding problem, the wall-clock time an accelerator loses waiting on input I/O, is established in section 4.9.3.

These granular measurements enable precise kernel optimization, but they cannot reveal how components interact when assembled into complete models. Macro-benchmarks address this gap.

12.3.2 Macro benchmarks

Micro-benchmarks verify that isolated operators achieve high compute or memory efficiency. Macro-benchmarks determine whether those gains persist when layers are composed into a complete model. Composing layers introduces physical interactions that isolated benchmarks cannot observe: intermediate activation tensors expand peak memory footprint, sequential kernel launches expose CPU driver dispatch overhead, and memory access streams from preceding operations evict shared cache lines before downstream kernels can reuse them.

Macro-benchmarks serve a central engineering objective: evaluating model architectures and runtime configurations under standardized workloads. Model-level evaluation captures the trade-offs that emerge from layer composition: generalization accuracy on unseen validation data, dynamic memory allocation across batch sizes and context lengths, and aggregate throughput across varied request arrival patterns. These dimensions interact directly. An architecture that achieves higher accuracy may require an activation footprint that forces a smaller batch size under device memory constraints, collapsing the computational throughput that justified its adoption. ResNet-50's training step tests the roofline's prediction at exactly this granularity, because the prediction is made layer by layer and the measurement is made on the composed model.



Napkin Math 12.1: ResNet-50 on the roofline: Predicted versus measured

Problem: The roofline predicts the fastest an H100 can train ResNet-50, and MLPerf measures how fast it does. How large is the gap, and what does it say about where the time goes?

Prediction: One training image costs about 24.6 GFLOP, three times the 8.2 GFLOP forward pass, because the backward pass computes gradients with respect to both activations and weights. At training batch sizes, the convolution layers reuse each weight across hundreds of images and sit above the 295 FLOP/byte ridge, so the roofline places the step on the compute roof. Dividing the 989 TFLOP/s peak by the work per image gives a ceiling of about 40,200 images/s.

Measurement: The MLPerf Training per-accelerator result for H100-class systems is about 5,000 images/s, which delivers about 123 TFLOP/s of model arithmetic, 12.4 percent of peak.

Diagnosis: The measurement sits 8.0× below the prediction. The roofline did not err; it bounded the convolutions and omitted the rest of the step. Normalization and elementwise layers have intensities far below the ridge and run on the bandwidth slope, the host must decode and augment thousands of JPEG images every second, and every step pays launch and synchronization overhead. The prediction cannot say which of these dominates. A profiler attributes the gap among them (perspective 12.5), and section 8.5 works that attribution for a training step.

Systems insight: A prediction and a measurement together locate a bottleneck; either alone does not. The roofline says ResNet-50 could train 8.0× faster on this accelerator, and the measurement says something other than arithmetic is setting its pace.

The claim the chapter inherited, that ResNet-50 runs faster at INT8 than at FP32, needs a different measurement from this training one. An inference run under MLPerf Inference rules settles it directly. The ResNet-50 Offline scenario, run at INT8 and at the higher-precision baseline on the same accelerator with each required to hold the reference top-1 accuracy within the benchmark's quality gate, reports both throughputs under one set of rules. The chapter's own deployment measurement moves to MobileNetV2 on the camera-trap board, where the full path from claim to delivered speedup can be traced on one device.

Evaluating complete models requires standardized datasets, fixed reference tasks, and reproducible metric definitions. In computer vision, standard benchmarks evaluate models on ImageNet (Deng et al. 2024), measuring top-1 classification accuracy alongside inference latency across fixed batch sizes. In natural language processing, standardized tasks evaluate sequence-to-sequence translation and autoregressive generation, quantifying output quality against token generation rate across defined context lengths.

Several industry-standard suites establish reproducible model-level comparisons across platforms. The MLPerf suite standardizes evaluation across data centers, edge appliances, mobile devices, and microcontrollers, as detailed in section 12.6.3. For resource-constrained embedded targets, EEMBC's MLMark quantifies inference throughput and energy consumption under strict memory budgets, while the AI-Benchmark (Ignatov and Timofte 2024) suite evaluates mobile systems-on-chip across heterogeneous compute units.

12.3.3 End-to-end benchmarks

End-to-end benchmarks encompass the entire production pipeline, capturing every stage required to ingest raw data, generate model predictions, and deliver results to downstream clients. While micro- and macro-benchmarks focus on accelerator execution, real-world serving pipelines frequently find their primary bottlenecks outside the model itself.

Data preprocessing forms the initial phase of the pipeline, converting raw external inputs into model-ready tensor representations. In computer vision pipelines, image ingestion requires decoding compressed file formats (such as JPEG decompression), resizing, cropping, and color normalization. When executed on host CPUs without hardware acceleration, these preprocessing steps can take several times longer than model inference on a modern accelerator, leaving high-throughput tensor cores idle while waiting for host memory transfers. Similarly, in natural language pipelines, subword tokenization and dynamic sequence padding on the host CPU often bound serving throughput. Postprocessing introduces analogous latency bottlenecks: object detection pipelines require non-maximum suppression (NMS) and bounding-box coordinate transformations, which create sequential execution stalls if not integrated efficiently into the execution graph.

Distributed infrastructure components introduce further performance constraints beyond the compute fabric. Remote storage systems bound training throughput when reading large multimodal datasets, and network interface card (NIC) saturation in distributed clusters throttles collective communication. End-to-end benchmarks must measure pipeline throughput under realistic network latencies, concurrent query arrival patterns, and actual storage I/O constraints to ensure reproducible evaluation of the deployment as an integrated system.

Public benchmark suites rarely measure storage, networking, and compute within a single unified protocol. While MLPerf Training and Inference evaluate end-to-end scenarios more closely than isolated operator tests, their closed divisions deliberately standardize input data loading to isolate accelerator performance from storage and network variability. Consequently, production end-to-end benchmarking remains primarily an internal engineering practice, evaluated through distributed tracing and production telemetry on live query distributions.

12.3.4 End-to-end vs. component latency

Measured at the request, the critical distinction is between component latency (time spent in model computation) and end-to-end latency (total time from request arrival to response delivery). Many benchmarks report only model inference time, obscuring the remaining overhead that determines actual user experience. The overhead is not marginal: serialization, network hops, and queue wait time can dominate total request time, making model-only optimizations yield diminishing returns.

Clipper's designers measured exactly this effect. For lightweight models, CPU-side JSON deserialization and interprocess data copying took more wall-clock time than the accelerator's model execution (Crankshaw et al. 2017). Reporting model latency alone would have hidden the largest component, a serialization cost that section 13.2.2.1 develops for serving systems.

Table 12.6 gives an illustrative latency breakdown for an inference request. The model inference stage that vendors report as their "benchmark" number spans 5 to 100 ms, yet the queue wait time it sits behind ranges from 0 to over 1,000 ms. Under load, the single component a benchmark measures is dwarfed by one it never sees, so the reported number can be a small slice of what the user actually experiences.

Table 12.6: Inference Latency Breakdown: Different pipeline components contribute to end-to-end latency, and model inference (the number vendors typically report) can be only one part of total request time. Queue wait time can dominate under load, making end-to-end measurement essential for realistic performance assessment. Ranges are illustrative rather than universal.

Component	Example Range	Notes
Network round-trip	10–100 ms	Varies by region
Request parsing	0.1–1 ms	JSON/protobuf
Input preprocessing	1–50 ms	Tokenization, image resize
Queue wait time	0–1000+ ms	Load-dependent
Model inference	5–100 ms	The "benchmark"
Output postprocessing	0.5–10 ms	Decoding, format
Response serialization	0.1–1 ms	JSON/protobuf

These component-level contributions explain why optimizing any single stage yields diminishing returns on end-to-end performance, an *optimization ceiling* formalized by Amdahl's Law (principle 8). The bound itself appears as equation 11.1, where hardware acceleration states it for an accelerated path; the notebook below applies it to a measured pipeline.



Napkin Math 12.2: Amdahl's Law: Optimization ceiling

Problem: A vision pipeline spends 8 ms on preprocessing (JPEG decode, resize, normalize) and 10 ms on inference. If inference alone is optimized by 5×, how much does end-to-end latency actually improve?

Math: Assume fixed work, sequential stages, and unchanged preprocessing, with no queueing feedback. Optimizing inference from 10 ms to 2 ms reduces total latency from 18 ms to only 10 ms, a 1.8× improvement rather than 5×. Amdahl's Law formalizes this ceiling: if inference, the part being accelerated, is fraction f_{parallel} of total latency, then even infinitely fast inference yields at most $1/(1-f_{\text{parallel}})$ speedup, because the preprocessing fraction $1-f_{\text{parallel}}$ remains. With preprocessing at 44.4 percent of total latency, the remaining fraction $1-f_{\text{parallel}}$ is 0.444, which yields a maximum achievable speedup $1/(1-f_{\text{parallel}})$ of 2.25×, regardless of model optimization.

Systems insight: Aggressive model optimization yields disappointing end-to-end results whenever the nonmodel fraction dominates. Any component speedup quoted in isolation shrinks once the untouched stages are measured alongside it, and the shortfall grows as those stages take a larger share of the pipeline. Comprehensive benchmarks must either include preprocessing in measurements or state explicitly that reported speedups apply only to the inference component.

A latency report that supports a deployment decision therefore states which components it includes, the load it was measured under, and whether each figure is a component or an end-to-end number. Choosing that boundary is the trade-off between isolating a component bottleneck and measuring what the user experiences.

12.3.5 Granularity trade-offs and selection criteria

Table 12.7 contrasts the diagnostic focus, scope, and engineering trade-offs across the three benchmarking granularities. Micro-benchmarks isolate low-level operations to guide kernel optimization and hardware selection, macro-benchmarks evaluate complete models to guide architectural choices, and end-to-end benchmarks measure full pipelines to ensure production deployments satisfy real-world service requirements.

Table 12.7: Benchmarking Granularity Levels: Different benchmark scopes target distinct stages of ML system development. Micro-benchmarks isolate individual operations for low-level optimization, macro-benchmarks evaluate complete models to guide architectural choices, and end-to-end benchmarks assess full system performance in production environments.

Component	Micro Benchmarks	Macro Benchmarks	End-to-End Benchmarks
Focus	Individual operations	Complete models	Full system pipeline
Scope	Tensor ops, layers, activations	Model architecture, training, inference	Extract, transform, load; model; infrastructure
Example	Conv layer performance on cuDNN	ResNet-50 on ImageNet	Production recommendation system
Advantages	Precise bottleneck identification, Component optimization	Model architecture comparison, Standardized evaluation	Realistic performance assessment, System-wide insights
Challenges	May miss interaction effects	Limited infrastructure insights	Complex to standardize, Often proprietary
Typical Use	Hardware selection, Operation optimization	Model selection, Research comparison	Production system evaluation

Selecting a single granularity level is rarely sufficient because benchmarking presents an inherent trade-off between diagnostic isolation and production fidelity. Along the descending diagonal in figure 12.3, evaluation regimes trade diagnostic power for operational realism: micro-benchmarks (the red point at the top left) isolate kernel-level memory bandwidth and warp occupancy at the expense of end-to-end context, macro-benchmarks (the blue center point) capture composed model topologies, and end-to-end benchmarks (the green point at the bottom right) encompass live production traffic but obscure the root causes of latency spikes. Effective evaluation requires strategically combining all three tiers to verify that kernel speedups survive full-system deployment.

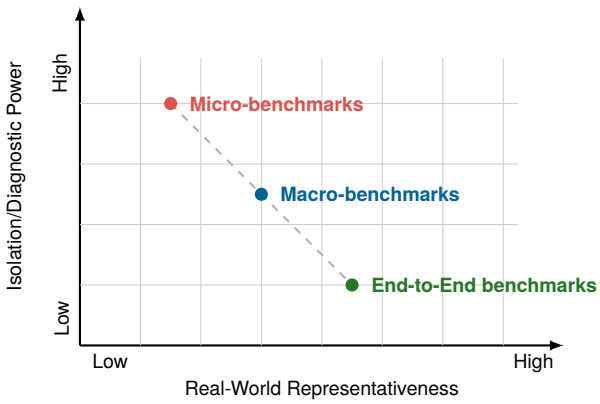


Figure 12.3: Isolation vs. Representativeness: The core trade-off in benchmarking granularity. Micro-benchmarks provide high diagnostic precision but limited real-world relevance, while end-to-end benchmarks capture realistic system behavior but offer less precise component-level insights. Effective ML system evaluation requires strategic combination of all three levels.

This trade-off maps directly onto the D·A·M taxonomy. Micro-benchmarks assess the physical capability of the Machine (M) executing a primitive Algorithm (A). Macro-benchmarks evaluate how the Algorithm (A) orchestrates tensor data flow across composed layers on the Machine (M). End-to-end benchmarks evaluate how raw Data (D) ingestion from external storage and network infrastructure interacts with both Algorithm and Machine under production load. Systematic performance engineering requires evaluating all three tiers to ensure local kernel optimizations translate into global system speedups.

Carpentry and granularity let one team build a trustworthy measurement. Comparing that measurement with another team's, or with a vendor's, requires that both were built the same way, and that agreement is what standardized benchmark suites provide.

12.4 Benchmark Suites

A team evaluating edge deployment hardware must compare five candidate system-on-chip (SoC) designs for the camera trap's next board. Vendor A reports 8 TOPS at INT8; Vendor B reports 15 TOPS at INT4; Vendor C reports inference latency on a proprietary model; Vendor D cites MLPerf scores from two generations ago; Vendor E provides only peak throughput at maximum batch size. None of these figures can be directly compared. The team cannot make an informed procurement decision because each vendor isolates a favorable metric under idiosyncratic conditions, obscuring true operating trade-offs. Benchmarking suites resolve this fragmentation by enforcing uniform measurement harnesses and standardized evaluation rules.

Each generation of benchmark suites arose to close a gap of exactly this kind, when the metrics of the day stopped predicting what users experienced. Their history explains the rules modern suites enforce.

12.4.1 From Whetstone to MLPerf

Three shifts mark that history: from isolated kernels to representative application suites, from single-metric throughput to multi-objective energy profiles, and from generic compute metrics to domain-specific execution constraints. The earliest computing benchmarks were open to benchmark gaming. Whetstone (Curnow and Wichmann 1976) used a synthetic mix of scientific operations, while LINPACK (Dongarra et al. 1979) measured dense linear-system solving. Because both relied on predictable computational loops, compiler and hardware vendors could optimize for the benchmark kernels (unrolling synthetic loops, or tailoring cache tiling to exact matrix dimensions) without improving general application performance. SPEC CPU (1989) addressed this by evaluating complete, portable software suites that stressed integer branching, irregular memory hierarchies, and floating-point pipelines (Dixit 1993). The same vulnerability applies to ML, since model-specific kernel tuning can overfit a single workload. Model compression claims therefore require validation on representative tasks such as ResNet-50 and BERT across the full software runtime.

Distributed computing added a further lesson, that component-level microbenchmarks fail to predict cluster-level throughput when interconnect latency, serialization, and collective communication dominate wall-clock time, as they do in distributed ML training. To capture these interactions, DAWNBench (Coleman et al. 2019) pioneered time-to-accuracy evaluation, tying hardware throughput to algorithmic convergence. Maximizing samples per second has no engineering value if numerical instability or poor batch sizing prevents the model from reaching its target validation accuracy. MLPerf[13] (2018) combined these lessons and institutionalized full-system measurement, multi-objective constraints, and representative workloads spanning vision, language, and recommendation (Mattson et al. 2020; Reddi et al. 2019).

Energy followed the same path. SPEC Power (2007) (Lange 2009) and Green500 (2007) (Feng and Cameron 2007) made performance per watt a ranking metric for servers and supercomputers, and MLPerf Power (MLCommons 2024b) carried it to ML workloads, whose rapid power swings and shared host components demand calibrated instruments and explicit measurement boundaries (section 12.7.4).

Energy benchmarking also exposes algorithmic efficiency. Compression and efficient architectures such as MobileNet (Howard et al. 2017; Sandler et al. 2018) save energy by reducing operations and off-chip traffic. Traffic reduction is a primary lever, because a DRAM read costs orders of magnitude more energy than an arithmetic operation (section 10.3.2.1).

As specialized accelerators emerged, generic benchmarks failed to capture domain-specific execution constraints. Data center training clusters optimize aggregate throughput within rack-level power envelopes, mobile AI operates within thermal limits (typically under 5 watts), and embedded microcontrollers function within milliwatt budgets, which decides whether a benchmark prioritizes samples per second or joules per inference. Application requirements add functional constraints such as calibration for clinical models, hard tail-latency cutoffs for trading systems, and bounded worst-case latency for autonomous vehicles.

[13] **MLPerf**: Launched in 2018 by a consortium of industry and academic institutions, MLPerf takes its name from "ML" combined with "Perf" (performance), echoing SPEC's benchmarking tradition. MLPerf's design principles (representative workloads, full-system measurement, and open submission) directly address the gaming that plagued Whetstone and LINPACK. Vendors who could previously report peak kernel throughput on cherry-picked problem sizes must now report end-to-end system performance on standardized tasks (Mattson et al. 2020; Reddi et al. 2019).

Because ML workloads couple tensor execution units, memory bandwidth, and communication fabrics, no single metric or workload can characterize a system. MLPerf therefore splits into distinct deployment suites: MLPerf Training evaluates multi-node scaling and high-bandwidth interconnects under time-to-quality targets (Mattson et al. 2020); MLPerf Inference measures throughput and latency percentiles under SLAs across server and client systems (Reddi et al. 2019); MLPerf Tiny tests ultra-low-power microcontrollers operating with tight memory constraints (C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kiraly, et al. 2021); and the cross-cutting MLPerf Power track measures useful work per watt across each environment. Reading the constraint column of table 12.8 from top to bottom, cluster-scale interconnect bandwidth gives way to latency percentiles at the edge, and finally to strict memory footprints on microcontrollers.

Table 12.8: MLPerf Benchmark Suite Variants: Each variant addresses a different deployment context, from data-center-scale training to ultra-constrained microcontroller inference, targeting specific operational constraints and measuring metrics relevant to its deployment scenario.

MLPerf Variant	Target Domain	Key Constraints	Primary Metrics
MLPerf Training	Data center	Multi-node scaling, high bandwidth interconnects	Time-to-quality, throughput (samples/sec)
MLPerf Inference	Server/Edge	Latency SLAs, throughput requirements	QPS, latency percentiles, accuracy preservation
MLPerf Tiny	MCU/IoT	Ultra-low-power inference, limited memory	Latency, accuracy, energy per inference
MLPerf Power	Cross-cutting	Energy budgets, thermal constraints	Performance/W, energy per query

Table 12.9 summarizes the progression from early synthetic loops to multi-dimensional ML systems evaluation; each row's lesson survives as a rule that modern suites enforce. Those rules become even more demanding in machine learning, where training and inference add variability that deterministic software benchmarks never faced.

Table 12.9: Benchmark Evolution: Evolution of computing benchmarks from synthetic operations to ML-specific evaluation. Each generation addressed limitations of its predecessors, culminating in MLPerf's synthesis of representative workloads, multi-objective metrics, and integrated system measurement.

Benchmark	Year	Primary Focus	Key Metric(s)	Lesson for ML Benchmarking
Whetstone	1976	Synthetic floating-point operations	MWIPS	Gaming synthetic tests undermines evaluation validity
LINPACK	1979	Linear algebra (matrix operations)	FLOP/s	Isolated operations miss system-level complexity and bottlenecks
SPEC CPU	1989	Real application workloads	SPECrate, SPECspeed	Representative workloads reveal true deployment performance
SPEC Power	2007	Server energy efficiency	ssj_ops/W across load levels	Energy efficiency requires multi-load evaluation, not just peak performance
Green500	2007	HPC energy efficiency	GFLOP/s per watt	Efficiency rankings complement raw performance rankings
MLPerf	2018	ML systems (training + inference)	Time-to-quality, QPS, latency, accuracy	Synthesizes all lessons: representative workloads + multi-objective + system

12.4.2 ML measurement challenges

Three lessons from benchmark history (representative workloads, multi-objective evaluation, and integrated system measurement) converge on a requirement unique to machine learning: managing statistical approximation. Traditional computing benchmarks specify correctness checks, which may require exact outputs or permit stated numerical tolerances. In ML systems, low-level engineering choices routinely alter numerical output: aggressive weight quantization, kernel pruning, or reduced-precision accumulators trade accuracy for execution speed and lower memory bandwidth pressure. Consequently, an ML benchmark cannot evaluate execution time in isolation; output accuracy serves as an inviolable operational constraint.

Evaluating an ML system therefore means testing the coupled Data, Algorithm, and Machine (D·A·M) choices together rather than measuring raw compute throughput. A suite earns trust by

solving three problems. It must produce valid measurements despite run-to-run noise, finite test sets, unrepresentative workloads, and misaligned metrics. It must then isolate what the hardware-software stack contributes, which is the job of a system benchmark and its roofline diagnostic. Finally, it must make results comparable across organizations, which is the job of community standardization.

ML systems combine several sources of measurement variability that classical software benchmarks were not designed to evaluate together. These fluctuations originate at both the machine and algorithmic layers. On the machine side, sustained execution pushes accelerators against thermal and power envelopes, triggering dynamic voltage and frequency scaling (DVFS) that throttles processor clocks mid-run. Host operating-system activity (thread scheduling jitter, memory paging, and PCIe bus contention during batch staging) further introduces latency variance across runs. On the algorithmic side, floating-point addition is non-associative, so reduction orders that change across parallel execution threads shift results slightly, while mini-batch shuffling, pseudo-random augmentations, stochastic dropout masks, and weight initialization perturb the trajectory that stochastic gradient descent takes across a non-convex loss surface.

Distinguishing a genuine systems optimization from background measurement noise requires isolating these overlapping sources of variance with repeated runs, paired comparisons, and confidence intervals. Those statistical controls apply to every benchmark in the chapter, so section 12.8.1 collects them in one place, including how large a test set must be before it can resolve a one-point accuracy difference.

Beyond run-to-run variance, benchmark validity depends on workload representativeness. Synthetic microbenchmarks generate uniform, statically sized tensors directly in accelerator HBM. By executing continuous, compute-bound GEMM operations in an isolated loop, microbenchmarks measure theoretical peak arithmetic throughput. However, they conceal the memory hierarchies and scheduling bottlenecks that dominate production deployments. Synthetic workloads bypass host-to-device PCIe data transfers, input decoding pipelines, dynamic memory allocation churn, and GPU cache evictions. In contrast, **trace-based benchmarking** replays timestamped request logs recorded from live serving clusters. Production traces capture Poisson and bursty request arrival distributions, variable sequence lengths, and concurrent query streams. Replaying live traces forces the serving infrastructure to manage dynamic batching (grouping requests that arrive within a short time window into one batch), key-value (KV) cache allocation under memory pressure, and queuing delays, stress-testing the system under operational conditions that synthetic steady-state loops obscure.

↩ The key-value (KV) cache, the per-request attention state that grows with every generated token, is established in section 6.7.4.

A representative workload can still mislead if the evaluation metric fails to reflect deployment constraints. When an engineering team optimizes a single algorithmic figure of merit (such as an offline BLEU score, top-1 classification accuracy, or token perplexity) in isolation from system SLOs, the metric ceases to measure genuine operational fitness. This divergence is the chapter's second principle, Goodhart's Law, operating inside a single team's choice of metric, and once the proxy becomes the sole optimization target it stops reporting overall system health. In neural machine translation, an offline accuracy optimization can break the serving deadline outright.

Example 12.1: Goodhart's Law in action

Scenario: An engineering team optimizes a neural machine translation model for offline BLEU score, increasing the beam search size (the number of candidate partial translations kept at each decoding step) from 1 to 10.

Diagnosis: The larger beam search achieves a 0.5-point BLEU gain on paper but increases candidate evaluations by 10×, increasing inference latency from 50 ms to 200 ms (4× slowdown), exceeding the serving deadline.

Systems lesson: Optimizing offline accuracy without latency constraints invites Goodhart's Law failures. Production benchmarks must enforce strict SLOs for serving latency (e.g., latency < 100 ms).

The translation example shows an algorithmic change (a tenfold wider beam) multiplying computational load and memory traffic for a marginal quality gain while violating a hardware latency

deadline. Fixed benchmarks also assess execution under an unvarying input distribution, which conceals how systems degrade under production distribution shifts. Model and data evaluation must test those distribution limits. System comparisons address a separate question by isolating infrastructure execution efficiency from model architecture and dataset variation. Standardized suites like MLPerf establish reference workloads, datasets, quality targets, and permitted implementation changes, enabling reproducible comparisons of hardware-software execution efficiency within a specified benchmark version and division.

12.4.3 System benchmarks

Hardware specifications advertise peak arithmetic rates that application workloads do not sustain—the gap between an H100's dense FP16 peak and the model arithmetic ResNet-50 training actually achieves (section 12.1). System benchmarks expose that gap by running standardized ML workloads across hardware architectures, memory systems, and interconnects rather than relying on spec-sheet peaks alone.

The underlying hardware, whether CPUs, GPUs, Tensor Processing Units (TPUs),[14] and application-specific integrated circuits (ASICs),[15] determines ML-system speed, efficiency, and scale. System benchmarks provide standardized methods for comparing hardware across AI workloads by computational throughput, memory bandwidth, power efficiency, operator coverage, and scaling (Reddi et al. 2019; Mattson et al. 2020).

System benchmarks serve two functions. For practitioners, they enable informed hardware selection by providing comparative data across configurations. For manufacturers, they quantify generational improvements and guide accelerator development. Both functions require separating what the hardware contributed from what the model and data contributed.

> **Definition 12.2: Machine learning system benchmarks**
>
> ***Machine learning system benchmarks*** are standardized evaluation protocols that hold the workload and quality target constant while varying the hardware-software stack, measuring $\eta_{\text{hw}} = R_{\text{sustained}}/R_{\text{peak}}$ (the measured sustained rate $R_{\text{sustained}}$ over the peak rate) and L_{lat} to isolate infrastructure efficiency from algorithmic improvements.
>
> 1. **Significance**: The same ResNet-50 model can report the same ImageNet Top-1 accuracy while delivering very different throughput across hardware stacks, precision formats, batch sizes, and compiler configurations. Only a system benchmark exposes that gap.
> 2. **Distinction**: Unlike algorithmic benchmarks (which vary the Data and Algorithm factors, Samples/Task and FLOPs/Sample, to improve convergence or accuracy), system benchmarks hold the algorithm and quality target fixed and vary the implementation (kernel libraries, quantization formats, batch sizes, and hardware generations) to measure how efficiently the hardware-software stack executes the workload, lowering the iron law's realized Machine factor (Seconds/FLOP) across compute ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), data movement ($D_{\text{vol}}/\text{BW}$), and fixed latency ($L_{\text{lat}}$).
> 3. **Common pitfall**: A frequent misconception is that a system benchmark result generalizes across workloads. An accelerator that achieves high utilization on ResNet-50 (a compute-friendly vision workload) may achieve much lower utilization on a recommendation system, whose embedding lookups are bound by memory capacity and bandwidth rather than arithmetic. System benchmarks are workload-specific; no single metric characterizes a hardware platform.

System benchmarks evaluate performance across scales, ranging from single-chip configurations to large distributed systems and covering both training and inference. The ImageNet challenge shows why separating the hardware's contribution from the model's is hard to do after the fact. Between 2010 and 2014, top-5 error (the blue line in figure 12.4) fell from 28.2 percent to 7.3 percent while GPU-accelerated entries (the green bars) rose from zero to over 100, with the steepest single drop at AlexNet's arrival in 2012 (Krizhevsky et al. 2012; Russakovsky et al. 2015; Gray 2015). These paired series plot contemporaneous empirical trends, not a causal attribution of how much accuracy

[14] **TPU (tensor processing unit)**: Google's custom ASIC for neural network workloads. A TPU v4 pod (4,096 chips) delivers 1.1 exaFLOP/s peak BF16 (Jouppi et al. 2023), but benchmarking TPUs requires caution: their systolic-array architecture favors regular tensor operations, so peak FLOP/s overstate performance on irregular workloads like sparse attention or dynamic control flow.

[15] **ASIC (application-specific integrated circuit)**: An ASIC's peak TOPS number applies only to the specific operators it was designed for. An unsupported layer may require a host fallback or a different implementation, adding transfer and coordination costs. This makes operator coverage an early question in any ASIC benchmark: workload compatibility can explain part of the gap between peak and achieved throughput.

gain stemmed from hardware throughput versus algorithmic design, training schedules, or dataset expansion. A system benchmark avoids that ambiguity by construction: it fixes the model and quality target so that any change in the result belongs to the hardware-software stack.

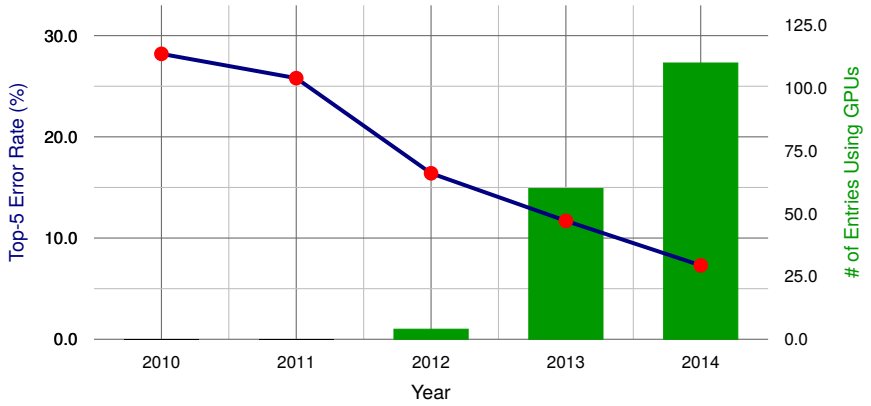


Figure 12.4: ImageNet Error and GPU Adoption: Sourced top-5 error rates are shown beside GPU use among challenge entries. The temporal association does not isolate hardware's causal contribution. Error-rate sources: (Russakovsky et al. 2015; Krizhevsky et al. 2012). Gray (2015) charts the per-year GPU entry counts.

While section 12.9.1 evaluates model compression against task accuracy baselines, system benchmarking isolates the execution efficiency of the hardware-software stack under a fixed model. Modern accelerators rarely sustain theoretical peak throughput because physical execution depends on the coupling between tensor shapes and memory hierarchies; ResNet-50 training, which the roofline places on the compute roof, sustains about 12.4 percent of an H100's peak (notebook 12.1). Parameter-heavy transformer decodes and activation-heavy convolutional passes stress entirely different levels of the memory subsystem, causing achievable throughput to diverge across precision formats.

Measuring sustained utilization rather than peak capability establishes two critical scope boundaries. First, energy consumption scales with execution efficiency, because performance per watt varies widely across hardware platforms, and an underutilized accelerator draws substantial static power for minimal computational output. Second, multi-accelerator execution introduces communication boundaries. While multi-node cluster scaling involves network-topology effects beyond this book's single-node scope, intra-node benchmarking must account for interconnect bandwidth, memory sharing across accelerators, and gradient synchronization over NVLink or PCIe.

12.4.4 Community-driven standardization

Without shared measurement boundaries, utilization numbers cannot be compared. If one benchmark includes host-side preprocessing and PCIe transfers while another measures only device kernel execution, or if one power meter reads only the accelerator voltage rail while another includes host processors and chassis fans, the resulting figures reflect incompatible boundaries rather than architectural efficiency. No single vendor can resolve this discrepancy, because each has a commercial incentive to draw the boundary around its own hardware strengths. Formal standards bodies move too slowly for hardware and model structures that change every year, so consortia such as MLCommons maintain living benchmark suites. These pair open-source reference implementations with compliance rules so that numbers from different laboratories describe comparable workloads under the same version, division, quality target, and scenario.

Standardizing system benchmarks requires formalizing the D·A·M taxonomy so that evaluations isolate the Machine without confounding changes to the Algorithm or Data. The MLPerf Training benchmark enforces this separation by measuring time-to-train to a specified quality target rather than raw floating-point throughput or epoch duration (Mattson et al. 2020). High TFLOP/s from aggressive low-precision scaling helps only if faster steps outweigh any extra work needed to reach the target, and a run that misses the target is invalid. MLPerf splits submissions into two divisions.

In the closed division, every submitter must run the reference model and reach the same quality target, so differences in time-to-train belong to the system; the open division permits model and algorithm changes, so its results compare whole solutions rather than machines. Within the closed division, the rules constrain the workload while permitting specified tunable hyperparameters and approved numerical formats. Quality and compliance checks establish which hardware, compiler, and training-configuration improvements constitute valid submissions.

The MLPerf Inference benchmark applies this discipline to production deployment constraints (Reddi et al. 2019). An accelerator cannot simply report peak throughput under infinite batching if interactive serving requires guaranteed responsiveness. The benchmark therefore fixes how queries arrive, through execution scenarios matched to physical operating regimes: one query at a time for a single interactive user, randomly arriving queries under a tail-latency bound for a shared service, and all inputs supplied upfront for bulk throughput. By standardizing these execution boundaries, community benchmarks account for queueing delays and host-to-device PCIe transfers, making the tested load and timing boundary explicit when interpreting a throughput claim.

12.4.4.1 MLPerf execution scenarios

The same hardware can report dramatically different benchmark numbers depending on *how* requests arrive, which explains why vendor claims often fail to predict production performance. Classic MLPerf Inference defines four execution scenarios that characterize distinct traffic patterns, each requiring different optimization strategies (Reddi et al. 2019). Current client and generative-AI benchmark variants also include interactive measurements for latency-sensitive large language model (LLM) workloads, where time to first token and time per output token, the two measured quantities defined in section 12.6.2.2, become central (MLCommons 2026b).

SingleStream SingleStream processes one request at a time, measuring latency for sequential inference. This scenario models mobile and embedded applications where a single user interacts with the device: a smartphone camera app classifying images, a voice assistant processing speech, or a wearable detecting gestures. The key metric is a tail percentile of per-query latency (the 90th percentile in MLPerf's rules), and inter-request batching is unavailable because the next query arrives after the previous one completes. Optimization focuses on preprocessing efficiency and power consumption rather than throughput.

MultiStream MultiStream processes multiple synchronized input streams simultaneously, modeling scenarios like autonomous vehicles with multiple cameras that must be processed together for spatial fusion. The application motivation is synchronized processing, but the benchmark rule is version-specific: current MultiStream rules issue a group of samples per query, wait for its completion before the next query, and report a specified latency percentile. Earlier versions used fixed arrival intervals. Grouped-query latency helps assess multi-input execution; it does not establish a hard-deadline guarantee for a vehicle planning loop.

Server Server generates requests following a Poisson distribution, simulating cloud API traffic where requests arrive independently and unpredictably. This scenario models web services handling millions of queries from different users. Unlike SingleStream's guaranteed sequential arrival, Server traffic creates queuing dynamics where multiple requests compete for resources. The key metrics are throughput (queries per second) and the workload's tail-latency bound, typically set at the 99th percentile, and dynamic batching can improve efficiency by grouping requests that arrive within a time window. Optimization balances throughput against latency SLOs.

Offline Offline provides all inputs upfront, measuring maximum throughput when latency constraints are removed. This scenario models batch processing pipelines: overnight data processing, scientific computing, or precomputing recommendations. With no per-query latency requirement, systems can tune batch size for sustained throughput within memory and execution limits. The key metric is pure throughput (samples per second), and optimization focuses entirely on hardware efficiency. Table 12.10 maps the classic execution scenarios, plus the newer Interactive LLM-oriented case, to their deployment contexts and optimization strategies.

Table 12.10: MLPerf Execution Scenarios: Classic MLPerf Inference scenarios map to distinct deployment contexts, each requiring different optimization strategies, and newer Interactive scenarios extend the framework to latency-sensitive LLM workloads. SingleStream and MultiStream prioritize latency, Server balances throughput and latency, Offline maximizes throughput, and Interactive emphasizes token-level responsiveness. Matching the scenario to deployment context determines which benchmark results are relevant.

Scenario	Context	Strategy	Focus
SingleStream	Mobile apps, embedded devices	No batching (batch = 1)	Preprocessing, power efficiency
MultiStream	Autonomous driving, video analytics	Synchronized sensor fusion	Grouped-query latency percentile
Server	Cloud APIs, web services	Dynamic batching with timeout	Throughput-latency trade-off tuning
Offline	Batch processing, data pipelines	Maximum batch size	Throughput, hardware utilization
Interactive	Chat, agents, local generative AI	Token streaming, KV-cache management	Time-to-first-token, time-per-output-token

Community standards fix the workload, the divisions, and the arrival pattern. The training and inference sections apply those rules to the chapter's two lighthouse workloads, beginning with the one that consumes the most arithmetic per result: training ResNet-50 to its quality target.

12.5 Training Benchmarks

Training and inference fail in opposite ways on the same accelerator. A training run stalls when synchronizing gradients across accelerators saturates the interconnect; a serving deployment violates its SLO when request bursts build a queue. The two regimes also measure different resources. Training iteratively refines parameters over billions of samples and must hold parameters, gradients, optimizer state, and the activations retained for the backward pass, a footprint that can far exceed the weights alone (table 12.1 gives ResNet-50's); its figure of merit is wall-clock time to a target accuracy. Inference evaluates fixed parameters against arriving queries, discards the backward graph and optimizer state, and is judged by latency percentiles and throughput under bursty load. A training result and an inference result on the same accelerator therefore measure different resources and cannot stand in for each other, which is why MLPerf maintains separate Training and Inference suites (Mattson et al. 2020; Reddi et al. 2019). This chapter measures training on ResNet-50 and serving on MobileNetV2 (section 12.6).

↩ The training memory budget of weights, gradients, optimizer state, and retained activations is established in section 8.3.2.5.

In an illustrative procurement failure, a team purchases a larger GPU cluster expecting proportional training-speed gains, only to discover that communication overhead and memory bottlenecks limit the actual speedup. Training benchmarks exist to catch this kind of gap before procurement. Training benchmarks span six dimensions (table 12.12), anchored by three: convergence metrics that measure learning progress, throughput metrics that measure computational efficiency, and scalability metrics that measure distributed performance.

Definition 12.3: ML training benchmarks

ML training benchmarks are machine learning system benchmarks that measure the time to reach a target quality metric (for example, a specified validation accuracy or loss threshold) on a fixed dataset and model, quantifying the rate of convergence per unit of resource.

1. **Significance**: Training benchmarks reveal large gaps invisible to hardware specs. Holding the model and quality target fixed, the time to convergence can vary widely across hardware-software stacks because training performance depends on the full pipeline: data loading $(D_{\text{vol}}/\text{BW})$, compute utilization (η_{hw}), gradient synchronization $(\text{BW}, L_{\text{lat}})$, and fault recovery overhead. A peak FLOP/s spec sheet captures none of these interactions.
2. **Distinction**: Unlike inference benchmarks, which measure per-query latency and throughput under load, training benchmarks measure time-to-accuracy across the full optimization loop: data loading, forward pass, backward pass, gradient synchronization, and optimizer step. As scale increases, communication (BW) can displace compute (R_{peak}) as the binding constraint; profiling establishes the transition for a particular workload.

3. **Common pitfall**: A frequent misconception is that training benchmarks measure "how fast the GPU runs." At large scale, interconnect bandwidth (BW) for gradient synchronization and fault tolerance overhead (checkpoint I/O, straggler mitigation) often dominate the benchmark result more than peak FLOP/s.

Training benchmarks validate whether hardware acceleration delivers promised training throughput. Accelerators and multi-chip systems all claim dramatic speedups, and training benchmarks reveal which claims hold under realistic workloads. They evaluate how hardware configurations, data loading mechanisms, and distributed training strategies perform when training production-scale models. These benchmarks are vital because training represents the largest capital expenditure in ML systems, and only rigorous time-to-accuracy measurement reveals whether that capital delivers proportional value rather than dissipating into scaling inefficiencies, memory bottlenecks, or communication overhead. The metrics themselves (time-to-accuracy, scaling efficiency, utilization, checkpoint overhead) are defined and worked at the scale of one machine with one to eight accelerators, the scale of the scaling-efficiency notebook below.

12.5.1 Training benchmark motivation

MLPerf Training (Mattson et al. 2020; MLCommons 2024c) provides the standardized framework for this time-to-quality measurement. The longitudinal scaling curves in figure 12.5 track cumulative relative performance gains across nine model workloads between 2018 and 2024 against the orange Moore's Law baseline. Every workload curve outpaces cumulative transistor scaling (which reaches 6.6× by mid-2024), led by Mask R-CNN climbing to 48.8× and ResNet reaching 34× as distributed scaling and compiler optimizations compound hardware improvements (Tschand et al. 2024). This trajectory documents improvements in the full hardware-software system beyond transistor scaling alone; the plotted results do not isolate the contribution of competitive pressure from other causes.

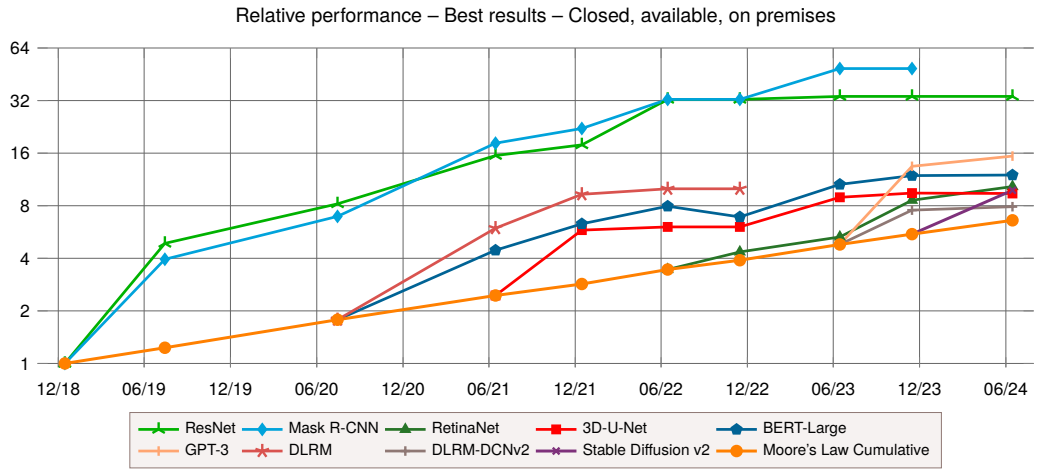


Figure 12.5: MLPerf Training Progress: Workload lines report cumulative relative performance across benchmark versions against the plotted cumulative Moore's Law baseline. Every workload's latest point exceeds the baseline at the same date, and Mask R-CNN reaches the largest plotted gain at 48.8×; the comparison captures combined hardware and software progress, not hardware scaling alone. Source: (Tschand et al. 2024).

Training benchmarks also expose inefficiencies such as slow data loading, underutilized accelerators, excessive memory overhead, and communication bottlenecks that erode scaling efficiency. Advertised hardware capability (for example, accelerator TFLOP/s) becomes a training speedup only to the extent that a benchmark verifies it under realistic conditions.

Training benchmarks serve four interconnected functions. First, they enable hardware and software optimization by providing vendor-neutral comparisons across accelerator architectures and frameworks (TensorFlow, PyTorch) on standardized tasks, guiding hardware selection for data

[16] **Mixed-precision training**: Uses lower precision for most arithmetic while preserving higher-precision accumulation where needed (Micikevicius et al. 2018). The benchmarking consequence: mixed-precision and full-precision runs are not directly comparable because reduced memory traffic and larger feasible batch sizes can change convergence dynamics. MLPerf addresses this by fixing the accuracy target, making time-to-accuracy the comparable quantity regardless of precision strategy.

centers and cloud environments. Software optimizations including mixed-precision training[16] and memory-efficient data loading are similarly quantified. Second, they evaluate scalability: adding GPUs should reduce training time proportionally, but communication overhead, synchronization latency, and memory bottlenecks limit scaling efficiency in practice. Training benchmarks quantify these losses, revealing whether infrastructure investments deliver proportional returns. Third, they provide cost and energy accountability: with a single large-scale training run consuming energy on the scale of the GPT-3 estimate in section 12.5.2.4, benchmarks that track cost per training run and power consumption per unit of progress help organizations balance computational power with sustainability goals. Finally, standardized evaluation criteria, controlled randomness, submission rules, and audits make comparisons more reproducible and constrain implementation-specific shortcuts.

12.5.2 Training metrics

From a systems perspective, training benchmarks assess the rate of convergence toward a verified quality threshold rather than the raw processing speed of an execution loop. Metrics such as sample throughput and scaling efficiency become meaningful only when the workload reaches target convergence; without that operational constraint, raw throughput rewards numerical instability and divergence. MLPerf Training formalizes this requirement by coupling wall-clock time directly to task-specific validation targets across each standardized workload.

12.5.2.1 Time and throughput

A training cluster that processes twice as many images per second by doubling the minibatch or dropping precision has not sped up training if the run requires twice as many epochs to converge—or never reaches the quality floor at all. That is why the primary metric for evaluating training efficiency is the time required to reach a predefined accuracy threshold. Training time (T_{train}) measures how long a model takes to converge to an acceptable performance level, reflecting the overall computational efficiency of the system. Let $\text{Accuracy}_{\text{val}}(t)$ be the model's validation accuracy at training time t, and let the target accuracy be the benchmark-specific threshold (for example, 75.9 percent top-1 accuracy for ResNet-50 on ImageNet in MLPerf). Equation 12.1 formally defines this metric, keeping the benchmark focused on how quickly a system achieves meaningful results:

$$T_{\text{train}} = \inf\{t \geq 0 : \text{Accuracy}_{\text{val}}(t) \geq \text{target accuracy}\} \tag{12.1}$$

A run that never reaches the target has no finite time-to-accuracy, preventing raw training speed from rewarding a system that fails the benchmark's quality criterion.

Throughput,[17] often expressed as the number of training samples processed per second, provides an additional measure of system performance. Let N_{samples} be the total number of training samples processed and T_{train} the training time from equation 12.1. Equation 12.2 makes the rate explicit by dividing processed samples by training time:

$$\text{Throughput} = \frac{N_{\text{samples}}}{T_{\text{train}}} \tag{12.2}$$

Throughput alone cannot certify system capability. A platform may sustain high sample rates using aggressive low-precision formats or enormous minibatches, yet fail to reach target accuracy because of gradient divergence or poor generalization. In MLPerf Training, for instance, a ResNet-50 benchmark on ImageNet is valid only if the model achieves the required 75.9 percent top-1 validation accuracy (Mattson et al. 2020; MLCommons 2024c). Evaluating throughput strictly as the rate of progress toward convergence ensures that reported gains reflect genuine learning productivity rather than wasted arithmetic cycles.

12.5.2.2 Scalability and parallelism

Scalability measures how effectively training performance improves as resources are added. Ideally, doubling GPU count should halve training time. In practice, communication overhead, memory bandwidth limits, and parallelization inefficiencies constrain scaling well below linear.

[17] **Throughput**: Originating in manufacturing to measure units produced per unit time, the term entered computing in the 1960s batch-processing era. The manufacturing origin carries a systems lesson: throughput and latency are inherently opposed, because batching increases throughput (more units per hour) at the cost of individual item wait time. In ML serving, this manifests as the batch-size trade-off: larger batches improve GPU utilization but increase per-request latency.

Napkin Math 12.3: Scaling efficiency calculation

Problem: A team trains an image classifier on a fixed dataset. Single-GPU training takes 24 hours. With 8 GPUs, training takes 4 hours. Is this good scaling? Where did the efficiency go?

Step 1: Define scaling efficiency. For strong scaling (fixed problem size, more processors), let $T(1)$ be the training time on a single GPU, and $T(N_{\text{GPU}})$ the training time on N_{GPU} GPUs. Equation 12.3 defines efficiency:

$$\text{Eff}_{\text{scaling}}(N_{\text{GPU}}) = \frac{T(1)}{N_{\text{GPU}} \times T(N_{\text{GPU}})} \times 100\% \tag{12.3}$$

Step 2: Calculate efficiency. $\text{Eff}_{\text{scaling}}(8) = \frac{24\,\text{hours}}{8 \times 4\,\text{hours}} \times 100\% = 24/32 = 75$ percent

With perfect scaling, 8 GPUs would complete in 3 hours (24 hours/8 GPUs). The actual 4 hours represents 75 percent efficiency.

Step 3: Investigate the efficiency loss. The two elapsed times establish the "missing" 25 percent, but do not attribute it. Table 12.11 lists illustrative contributions and profiling signals for gradient synchronization, memory copy, load imbalance, and batch-size effects. Strong-scaling comparisons hold total work fixed; time-to-quality comparisons must also account for any change in steps to convergence.

In multi-accelerator data-parallel training (such as 2, 4, or 8 GPUs connected over PCIe or NVLink within a host), each accelerator computes gradients on its own local slice of the minibatch, and the AllReduce collective of section 8.7.1.1 must sum them so every worker receives the identical global gradient average before the next step. Gradient tensors have one entry per trained parameter, but their byte payload equals weight storage only when their representations match. Moving them across interconnects adds communication whose cost depends on topology, the collective algorithm, and overlap. With fixed total work and ideal partitioning, compute time per step shrinks ($T_{\text{compute}} \propto 1/N_{\text{GPU}}$), while communication may shrink more slowly or grow. Exposed synchronization then occupies a larger fraction of the step; traces determine how much of the observed scaling loss it explains.

Table 12.11: Scaling Efficiency Loss Sources: Illustrative decomposition of the missing efficiency budget into measurable overhead categories and the corresponding measurement signal used to attribute each loss. Actual percentages depend on model, interconnect, storage, batch schedule, and framework implementation.

Source	Example Contribution	Measurement
Gradient synchronization	10–15%	AllReduce time per step
Memory copy (CPU↔GPU)	3–5%	Data transfer profiling
Load imbalance	2–5%	Per-GPU step time variance
Batch size effects	2–5%	Larger batches converge differently

Step 4: The systems insight. Scaling efficiency can decrease as N_{GPU} grows when exposed communication and other overheads fail to shrink with per-GPU compute. In this worked example, eight GPUs reach 75 percent efficiency; at larger scales, the same arithmetic makes clear why sophisticated communication and input-pipeline optimization become necessary.

MLPerf reports raw performance, while scaling efficiency can be calculated across matched submissions: a system achieving 2× throughput at 50 percent efficiency may be worse than 1.5× throughput at 90 percent efficiency, depending on cost constraints.

Parallelism strategies (data, model, and pipeline parallelism) and the synchronization each requires are developed in section 8.7.1.1; within one machine of two to eight accelerators, data parallelism is the common case. A benchmark's job is narrower: to report how much of the added capacity a configuration converts into speedup without degrading convergence. That conversion is **scaling efficiency**. While strong scaling evaluates whether adding processors reduces training time on a fixed total workload, **weak scaling** scales the problem size proportionally with processor count to

↩ The AllReduce collective that averages gradients across data-parallel workers is established in section 8.7.1.1.

keep the per-device workload constant, measuring whether the system can train larger models or ingest larger datasets within a fixed time budget without communication bottlenecks dominating.

12.5.2.3 Resource utilization

Training efficiency depends on learning speed, cluster scalability, and how effectively the hardware resources are used. Compute utilization requires an explicit numerator. Model FLOPs utilization (MFU), the metric of definition 8.3, compares the useful model operation rate with the matching hardware peak; activity counters instead report whether units are busy. MFU contrasts with hardware FLOPs utilization (HFU), which counts all executed operations including activation recomputation; recomputation adds executed FLOPs without adding useful model FLOPs for the same step. Comparing them separates useful model work from work expended to relieve memory capacity limits. Low utilization may indicate bottlenecks in data movement, memory access, or inefficient workload scheduling.

For instance, when training ResNet-50, decoding and augmenting thousands of JPEG images per second on the host can limit throughput even when the accelerators have compute to spare, the crossing from compute-bound to input-bound that section 8.4.2 traces. If storage retrieval or preprocessing cannot keep up, the accelerators wait. Profiling resource utilization identifies the bottleneck, and optimizations such as prefetching, caching, and more parallel input processing can improve sustained performance.

Memory bandwidth is another limiting factor, as deep learning models require frequent access to large volumes of data during training. If memory bandwidth saturates, increasing compute power alone will not improve training speed. Benchmarks assess how well models use available memory, ensuring that data transfer rates between storage, main memory, and processing units do not become performance bottlenecks.

I/O performance also plays a direct role in training efficiency, particularly when working with large datasets that cannot fit entirely in memory. Benchmarks evaluate the efficiency of data loading pipelines, including preprocessing operations, caching mechanisms, and storage retrieval speeds. Systems that fail to optimize data loading can experience large slowdowns, regardless of computational power.

12.5.2.4 Energy efficiency and cost

Training GPT-3 was estimated to consume 1,287 MWh of electricity (Patterson et al. 2021). At that scale, energy efficiency metrics quantify the power usage of training workloads so teams can identify systems that reach target accuracy without wasted energy. Fewer iterations reduce energy only when their total energy cost is lower; a more expensive step can offset the reduced count, so suites such as MLPerf Training measure total energy over the complete run to the same quality target. The same power accounting governs inference, where precision is one potential energy lever; section 12.7 works through why INT8 quantization cuts per-inference energy by attacking both memory traffic and arithmetic cost.

Cost considerations extend beyond electricity usage to include hardware expenses, cloud computing costs, and infrastructure maintenance. Training benchmarks provide insights into the cost-effectiveness of different hardware and software configurations by measuring training time in relation to resource expenditure. Organizations can use these benchmarks to balance performance and budget constraints when selecting training infrastructure.

12.5.2.5 Fault tolerance and robustness

Training workloads often run for extended periods, sometimes spanning days or weeks, making fault tolerance an essential consideration. A resilient system must handle unexpected failures (hardware malfunctions, network disruptions, and memory errors) without compromising accuracy convergence.

Production systems checkpoint for fault tolerance, periodically saving progress so a failure does not restart the run. On a single host, a benchmark records the save interval, the state written, and the pause each save exposes to training. Model weights and optimizer states consume storage bandwidth, and any pause extends wall-clock time to the quality target. Asynchronous saving can overlap I/O with computation, but its staging buffers and competing storage traffic remain part

of the measured workload. Cluster recovery and parallel file systems extend this problem beyond the local machine; the local benchmarking lesson is to include the intended saving behavior in sustained performance measurements. MLPerf Training primarily measures time-to-quality under standardized workloads and does not benchmark failure recovery directly.

Training runs are also where reproducibility is hardest, because stochastic processes interact with hardware-specific behavior over many hours. What a training report must preserve to make its result reproducible, the full run envelope rather than the random seed alone, is collected with the other statistical controls in section 12.8.2.

12.5.3 Training performance evaluation

A comprehensive training benchmark considers multiple dimensions of system behavior because each dimension identifies a different way hardware investment can fail to become convergence. Table 12.12 summarizes the core categories and associated metrics commonly used to benchmark system-level training performance, providing a framework for understanding how training systems behave under different workloads and configurations.

Table 12.12: Training Benchmark Dimensions: Key categories and metrics for evaluating machine learning training systems beyond simple speed, covering resource efficiency, reproducibility, and overall performance trade-offs across different training approaches and infrastructure configurations.

Category	Key Metrics	Example Benchmark Use
Training Time and Throughput	Time-to-accuracy (seconds, minutes, hours); Throughput (samples/sec)	Comparing training speed across different GPU architectures
Scalability and Parallelism	Scaling efficiency (percent of ideal speedup); Communication overhead (latency, bandwidth)	Analyzing distributed training performance for large models
Resource Utilization	Compute utilization (percent GPU/TPU usage); Memory bandwidth (GB/s); I/O efficiency (data loading speed)	Optimizing data pipelines to improve GPU utilization
Energy Efficiency and Cost	Energy consumption per run (MWh, kWh); Training throughput per watt (FLOP/s/W)	Evaluating energy-efficient training strategies
Fault Tolerance and Robustness	Checkpoint overhead (time per save); Recovery success rate (percent)	Assessing failure recovery in cloud-based training systems
Reproducibility and Standardization	Variance across runs (percent difference in accuracy, training time); Framework consistency (TensorFlow vs. PyTorch vs. JAX)	Ensuring consistency in benchmark results across hardware

The scaling notebook supplies a concrete interpretation of table 12.12. The measured completion times establish a speedup and a resource-efficiency cost, but they do not identify which overhead caused the difference. A procurement decision therefore uses the time-to-quality result first, then the utilization and communication measurements to decide whether additional hardware or a different execution configuration would address the limiting resource.

Across matched runs, GPU boost-clock[18] behavior and thermal state must follow the same protocol; otherwise a short fast run can be mistaken for a durable improvement. The repeated-run controls in section 12.8.1 determine whether the difference exceeds experimental uncertainty.

In the worked case, 8 GPUs reach 75 percent scaling efficiency. If the required quality target and completion deadline are met, the configuration clears the time constraint, but the remaining scaling overhead still matters for cost. If the deadline is missed, the efficiency ratio alone does not justify buying more accelerators. The next experiment must measure exposed synchronization, input stalls, and per-device work under the intended batch and checkpoint schedule before choosing an intervention. If convergence changes between configurations, compare complete time-to-quality runs rather than interpreting the difference as pure strong-scaling loss.

A training report can therefore support a configuration choice without claiming that every measured overhead has already been explained. Its acceptance gates concern learning; an efficiently trained model still needs separate evidence that it can serve requests under deployment constraints.

[18] **GPU boost clock**: Dynamic frequency scaling raises clocks above base when thermal and power headroom permit. Short benchmark runs can capture this burst performance, while sustained training may settle to a different operating point. The benchmark must document the measured thermal state and clock policy.

Checkpoint 12.3: Measuring a training claim

The first half of the chapter built the measurement and applied it to training. Before moving to deployment, you should be able to:

- ☐ Restate "ResNet-50 at INT8 is faster" as a benchmark by naming its input boundary, output, quality floor, and timing statistic.
- ☐ Explain why a 3× model-stage speedup inside a 50 ms request yields only about 1.2× end to end, and name the granularity that exposes it.
- ☐ Use the roofline ceiling and the measured MLPerf throughput for ResNet-50 on one accelerator to compute its utilization, and name two candidate causes of the gap.
- ☐ Explain what MLPerf's closed division holds fixed and what its open division compares instead.
- ☐ Compute strong-scaling efficiency for a run that takes 24 hours on one accelerator and 4 hours on eight, and explain why the two elapsed times alone do not attribute the loss.

12.6 Inference Benchmarks

A camera trap running MobileNetV2 on an edge board must classify each frame within a 33-millisecond deadline to keep up with a 30-frame-per-second video stream. If an unbenchmarked garbage collection pause, dynamic batching timeout, or kernel launch hiccup stretches the p99 latency well past that deadline, frames are dropped, and an animal that crosses the field of view in under a second may go unrecorded, no matter how high the board's average throughput. An autonomous vehicle's perception loop runs against the same deadline arithmetic with far higher stakes. Training benchmarks measure how quickly a system learns; inference benchmarks measure how reliably it serves under real-world operational constraints. The two regimes differ in what they tolerate: training tolerates variable iteration times as long as gradient descent converges, but inference enforces service-level objectives where every delayed prediction compromises the application.

Inference is also where the optimization chapters converge, because the accelerated hardware runs the compressed models to deliver real-time predictions. Inference benchmarks reveal whether those theoretical speedups become actual latency reductions under realistic deployment conditions. Their central quantity is end-to-end request latency $L_{\text{lat,total}}$, the full time from a request's arrival to its response, including queueing, preprocessing, model execution, and postprocessing. It is distinct from the iron law's latency term L_{lat}, which counts only the fixed overhead (orchestration, serialization, dispatch) that one execution pays regardless of how much data it moves or arithmetic it performs; queueing delay, which grows with load, belongs to $L_{\text{lat,total}}$ but not to L_{lat}.

Definition 12.4: ML inference benchmarks

ML inference benchmarks are machine learning system benchmarks that quantify a system's ability to meet latency constraints ($L_{\text{lat,total}}$) at specified throughput levels, measuring scenario-defined latency statistics, throughput (queries per second), and power efficiency across representative serving scenarios.

1. **Significance**: Inference benchmarks expose the workload- and system-specific gap between unconstrained throughput and throughput while meeting a service-level objective (SLO), such as a p99 latency target. Queuing delays push tail latency above the target at high load, and only a benchmark that enforces the target at each throughput level shows it.
2. **Distinction**: Unlike training benchmarks, which measure time-to-accuracy over a fixed dataset, inference benchmarks measure per-query response time under realistic load patterns, capturing queuing effects, batching trade-offs, and cold-start overhead that determine real-world serving economics.

3. **Common pitfall**: A frequent misconception is that average latency is a sufficient benchmark. A low mean can coexist with a long p99 tail, and at high request rates the slowest 1 percent becomes a large number of affected users. The benchmark must report the statistic the service objective names.

12.6.1 Inference benchmark motivation

Large-scale benchmark training commonly runs on dedicated data center hardware, whereas inference spans dramatically different deployment scenarios, from real-time applications like autonomous driving and conversational AI to mobile devices, IoT systems, and embedded processors. This diversity extends to hardware: while GPUs and TPUs dominate large-scale training, inference workloads often use specialized accelerators like NPUs, field-programmable gate arrays, and dedicated inference chips such as Google's Edge TPU.[19] Inference benchmarks evaluate how well hardware selection, model optimization, and data pipeline design work together across these deployment environments.

Deployment differences of this breadth motivate inference benchmarks, which evaluate the bottlenecks that emerge when models transition from development to production serving. The motivating factors parallel those for training (hardware optimization, scalability, cost, fair comparison) but differ in specifics. Software optimization frameworks apply inference-specific techniques such as kernel fusion, precision calibration, and kernel tuning, whose impact on latency, throughput, and power efficiency must be measured under realistic conditions to confirm they deliver real improvements without degrading accuracy. Auto-tuning compilers add a hidden variable: the compiler itself can require hours of optimization per model-hardware pair, meaning benchmark results reflect the tuning budget as much as the hardware capability, and comparing results across submissions requires normalizing for compiler optimization time.

Scalability concerns also shift character. Training scales by adding GPUs to reduce time-to-accuracy on a fixed workload, whereas inference must scale dynamically in response to fluctuating user demand, handling traffic spikes without violating latency guarantees. Applications that load models on demand, such as serverless deployments, also expose the model's loading time, its cold start, directly to users.

The cost and energy profile of inference differs sharply from training. Training concentrates cost in discrete runs that may recur as data, objectives, or models change; inference cost accumulates continuously with production traffic. Running an inefficient model at scale can multiply cloud compute expenses, and on battery-powered devices, excessive computation directly affects usability. Benchmarks that measure cost per inference request and efficiency per watt help organizations optimize for both performance and sustainability across deployment platforms.

MLPerf Inference extends the standardized comparison principles established for training benchmarks to deployment scenarios, defining evaluation criteria for tasks such as image classification, object detection, and speech recognition across different hardware platforms, so that inference comparisons remain reproducible under deployment constraints such as latency requirements and energy efficiency (Reddi et al. 2019).

12.6.2 Inference metrics

A camera trap must classify each frame before the next one arrives, a voice assistant must respond quickly enough that users do not perceive lag, and a recommendation engine must score enough candidates to keep pace with user scrolling. These constraints (latency and throughput) define the performance envelope within which all serving optimizations must operate. Inference metrics formalize these real-world demands into measurable quantities, and they differ from training metrics in kind, not just degree, because the optimization target shifts from convergence rate during training to latency predictability during serving. Training optimizes for throughput and time-to-accuracy; inference optimizes for latency consistency, resource efficiency, and service-level predictability, spanning cloud data centers handling millions of requests to edge devices operating under strict power constraints.

[19] **Edge TPU (tensor processing unit)**: Google's fixed-function edge AI accelerator. It illustrates a benchmarking constraint specific to fixed-function accelerators: its headline throughput applies only to quantized TensorFlow Lite models with supported operator types, so models requiring unsupported operators fall back to the host CPU or need graph rewrites before the accelerator result is meaningful.

12.6.2.1 Latency and tail latency

An inference service that averages 10 ms per request can still time out for thousands of users every hour if queueing bursts or garbage-collection pauses push the slowest 1 percent of requests past its deadline. Latency measures the time for an inference system to process an input and produce a prediction, and because averages conceal those high-percentile delays, benchmarks also report tail latency[20]—commonly the 95th percentile (p95) or 99th percentile (p99), the time within which 95 percent or 99 percent of inferences complete. Applications with hard deadlines require explicit deadline-miss or worst-case analysis beyond these percentiles. These measurements form the basis for SLOs and SLAs, which formalize performance expectations.

[20] **Tail latency**: A high-percentile response time, such as p95 or p99, determines production SLO or SLA compliance only when that objective or agreement specifies the percentile. Dean and Barroso (2013) showed that in fan-out architectures (common in recommendation systems), even 1 percent slow responses compound: a request touching 100 backend shards has about a 63 percent chance that at least one shard hits its 1 percent tail. Benchmarks reporting only mean latency hide this failure mode.

> **Definition 12.5: SLOs and SLAs**
>
> ***SLOs and SLAs*** are performance commitment specifications for production ML serving systems: a service-level objective (SLO) is a target value or range for a service-level indicator, while an SLA is a contract that specifies service objectives and the consequences of failing to meet them.
>
> 1. **Significance**: A latency SLO bounds end-to-end request latency $L_{\text{lat,total}}$ by setting a target for a specified latency indicator, and the iron law's L_{lat} is only the fixed-overhead part of that budget. Request queueing and transient spikes consume part of that budget.
> 2. **Distinction**: An SLO has no contractual consequence by itself, while missing an SLA objective triggers the consequences specified in the agreement, which may include credits or penalties. Teams commonly set the internal SLO tighter than the external agreement to leave operational headroom, though the definition does not require it.
> 3. **Common pitfall**: A frequent misconception is that meeting average latency satisfies a tail-latency SLO. SLOs can use averages, percentiles, rates, or other indicators; when the objective is defined at p99 or p99.9, an excellent mean does not establish compliance.

Engineering teams optimize toward SLOs while the business commits to SLAs. A usable latency objective also fixes the request population, the offered load, and the measurement window. Otherwise, two teams can report the same percentile threshold over different traffic mixes and reach incompatible conclusions about compliance.

> **Checkpoint 12.4: Metric selection**
>
> The metric shapes the optimization.
>
> Apply three rules before finalizing metric selection:
>
> - ☐ Throughput vs. latency: Name the objective to optimize and the quality, latency, and cost limits the benchmark must preserve.
> - ☐ Tail latency: State what p99 exposes that mean latency hides.
> - ☐ End-to-end: List the preprocessing, model execution, and postprocessing stages included in the reported latency.

At production scale, small p99 degradations affect many users. If 1 percent of requests take 10× longer (for example, 1000 ms instead of 100 ms), 10,000 requests per million are slow enough to risk timeouts and customer churn. Search engines and recommendation systems demonstrate this sensitivity: Google's search-latency experiments found measurable reductions in daily searches per user after 100–400 ms server-side delays (Brutlag 2009), which is why interactive services often treat sub-100 ms response times as a practical design target. Because occasional slow responses weigh heavily on user satisfaction, such services define percentile-based objectives, and large-scale systems may track deeper tails such as p99.9.

The challenge of meeting these tail latency targets is that the source of the tail is often architectural, not algorithmic. A garbage-collected runtime, a shared kernel driver, or a priority-inversion bug in the serving stack can inject latency spikes that no model optimization will remove.

↪ Provisioning serving capacity so that p99 latency meets its SLO under peak load is developed in section 13.4.3.

Discord's Read States service shows the effect outside machine learning. Periodic garbage-collection pauses in its Go implementation spiked p99 latency every two minutes until a rewrite in Rust removed the pauses (Howarth 2020). A managed runtime in a feature store or retrieval path can impose the same floor on an ML service's tail, and no model optimization removes it, which is why the complete serving stack belongs inside a production latency benchmark. Section 13.4.3 develops the mechanisms that shape the tail under load.

12.6.2.2 Token latency: TTFT and TPOT

Generative models are served one token at a time, so their latency splits into two measured quantities. **Time to first token (TTFT)** is the interval from a request's arrival to its first output token, dominated by prefill, the pass that processes the whole prompt in parallel. **Time per output token (TPOT)** is the mean interval between subsequent tokens, set by autoregressive decode. The gap between any two successive tokens is the **inter-token latency (ITL)**, so TPOT is the average of a request's ITLs, and the ITL distribution exposes the individual stalls that the average hides. A benchmark that reports a single aggregate throughput hides both: a model can achieve high throughput in tokens per second while still feeling sluggish if the first token arrives late, or it can minimize TTFT while generating subsequent text too slowly for interactive reading.

Token throughput translates these generation rates into user-perceived wall-clock time: for a response of 750 tokens, a generation rate of 25 tokens/s requires 30 seconds, whereas 100 tokens/s finishes in 7.5 seconds. Time-to-first-token and inter-token latency must therefore be reported together: one captures initial responsiveness, and the other captures sustained delivery speed. This latency split reflects contrasting computational regimes: time-to-first-token is governed by prompt prefill, which processes tokens in parallel and is typically compute-bound, whereas inter-token latency is governed by autoregressive decoding, where low-batch execution often streams weights and accesses a growing KV cache with limited reuse. The resulting bandwidth pressure depends on model residency, context length, batch size, and the execution path. Section 13.9.1 builds serving policies on these two measurements; the benchmarking requirement is that both are reported, under the same offered load, for every generative workload.

12.6.2.3 Throughput and resource metrics

Latency is only half of the serving envelope. Throughput counts how many inference requests a system completes per second, typically expressed as queries per second (QPS) or frames per second (FPS), and it is meaningful only together with the latency at which it was measured. Batching raises throughput by processing several inputs at once while adding wait time to each, so a throughput figure reported without its latency percentile and offered load says nothing about whether a serving deadline is met. A benchmark therefore reports **batch throughput** and per-request latency as a pair at the same offered load. Choosing the batch size that balances them is a serving policy (section 13.5.2); the benchmark's job is to measure both sides of that trade.

Speed alone is insufficient because inference optimizations can change model behavior. Reducing numerical precision can accelerate computation while cutting memory and energy, as the simplified MobileNetV2 energy estimate worked out in the power section shows (section 12.7.1), but lower-precision calculations can introduce accuracy degradation. Inference benchmarks therefore evaluate how well models perform under different numerical settings, such as FP32, FP16, and INT8.[21] Model compression techniques[22] further improve efficiency, but their impact on model accuracy varies depending on the task and dataset. Benchmarks help determine whether these optimizations are viable for deployment, ensuring that improvements in efficiency do not come at the cost of unacceptable accuracy loss.

Beyond speed and accuracy, table 12.15 lists the resource terms an inference benchmark must also report, and each exists because a system can pass the latency test and still fail to deploy. Memory footprint decides whether the model fits the device's budget at all, since inference rarely spans accelerators the way training can. Cold-start performance, the load time plus first-request latency,[23] exposes initialization cost to the requesting user. It dominates in serverless AI environments[24] and on devices that evict models to free memory. When provisioning, model transfer, runtime initialization, and the first response lie on the user request path, the benchmark must include them; a warm-instance result answers a different question. Scalability records whether added accelerators

[21] **INT8 (8-bit integer)**: INT8 sits at the aggressive end of the precision hierarchy (FP32 baseline, FP16 halves raw weight storage, INT8 quarters it), and each step demands increasing care to preserve accuracy. Post-training INT8 quantization normally fits its value ranges on a representative calibration dataset (quantization-range calibration, distinct from the confidence calibration of section 12.9.1.2); accuracy depends on the model, operator coverage, quantization method, and how well the calibration data represent deployment inputs. INT8 benchmarks must specify whether they use post-training quantization or quantization-aware training and document any range-calibration procedure.

[22] **Model compression benchmarking**: Compression impact must be measured across four dimensions simultaneously: accuracy degradation, inference speedup, memory reduction, and energy savings. A technique achieving 10× size reduction with 1 percent accuracy loss may still be unsuitable if latency misses the deployment target; unstructured pruning, for example, leaves dense execution unchanged unless a compatible sparse representation and kernel exploit the zeros.

[23] **Cold-start latency**: The initialization time from idle state includes storage reads, host-to-accelerator transfer, and framework setup. For a 7B-parameter model in FP16 (~14 GB) already resident in host memory, PCIe 4.0 transfer at 25 GB/s effective bandwidth alone takes ~560 ms; storage and initialization add to it. This lower bound makes cold-start mitigation (model caching, speculative loading) a systems design requirement.

or concurrent models raise throughput or collide on memory bandwidth, and energy per inference (joules per inference, or QPS/W in the cloud) sets battery life on mobile devices and operating cost in data centers.

12.6.3 MLPerf inference benchmarks

MLPerf Inference turns that balance among latency, throughput, memory, energy, and accuracy into scored scenarios, and it matters because deployment context changes what a result means. The benchmark, developed by MLCommons,[25] provides a standardized framework for evaluating machine learning inference performance across a range of deployment environments. MLPerf began with training benchmarks in 2018; MLPerf Inference was added later to standardize deployment-time evaluation across scenarios. The family of MLPerf inference benchmarks maps each benchmark to a deployment setting, so a score can be interpreted against the latency, throughput, memory, and power constraints the system will face.

12.6.3.1 MLPerf Inference

MLPerf Inference (Reddi et al. 2019) is the baseline inference benchmark, defining standardized scenarios for deployment-time evaluation across data-center and edge settings. Submissions follow the closed and open divisions defined in section 12.4.4. Its workloads span image classification, object detection, natural language processing, and recommendation, and its results are a common reference point for comparing accelerators and CPUs when the submission rules and scenario match the intended deployment. On recommendation workloads, scores on DLRM[26] can inform choices between accelerator generations, although the size of the gain depends on workload, software stack, and system configuration. At the same time, the cost of comprehensive benchmarking limits who can participate and how thoroughly systems are evaluated.

> **Systems Perspective 12.6: The cost of comprehensive benchmarking**
>
> Submitting to MLPerf can require dedicated engineering effort, hardware access, tuning, validation, and careful documentation across the relevant configurations. The cost depends heavily on workload and scope, but it is high enough that submissions are dominated by major technology companies and hardware vendors, while smaller organizations rely on published results rather than conducting their own comprehensive evaluations. That barrier motivates the need for more lightweight, internal benchmarking practices that organizations can use to make informed decisions without the expense of full-scale standardized benchmarking.

The rest of the MLPerf inference family narrows that baseline by deployment context. MLPerf Mobile (MLCommons 2024a) evaluates whether a model stays responsive within smartphone power and memory limits (Janapa Reddi et al. 2022). MLPerf Client (MLCommons 2026b) asks whether consumer devices can run local generative-AI workloads, which makes CPUs, discrete GPUs, and integrated NPUs part of the benchmarked system. MLPerf Tiny (C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kiraly, et al. 2021) covers embedded and ultra-low-power systems, such as IoT devices, wearables, and microcontrollers. Each variant keeps the same benchmark discipline while changing the binding resource.

12.6.4 Mobile and edge benchmarks

Unlike data centers with kilowatt-scale power delivery and active cooling infrastructure, mobile and edge deployments operate within strict milliwatt- to watt-scale power envelopes and passive thermal dissipation limits. Benchmarking these systems means reporting power consumption, inference latency, and model accuracy together, because gains in one usually come at the expense of another. Table 12.13 contrasts these operational constraints against cloud environments.

[24] **Serverless AI**: Deployment paradigm where models scale from zero instances on demand. Cold-start time varies with the model, runtime, storage path, hardware allocation, and provider. Benchmarks for intermittent workloads must report whether initialization and model loading are included, because warm-instance latency alone can understate user-perceived latency.

[25] **MLCommons**: Launched in 2020 as a nonprofit consortium evolving from the 2018 MLPerf effort, MLCommons includes members across industry and academia (MLCommons 2026a). Requiring published system specifications improves result comparability, but it does not prevent submitters from selecting which systems or workloads to enter. Published results reveal large performance differences between vendors on identical workloads, making MLCommons the closest the field has to SPEC-style like-for-like hardware comparison.

[26] **DLRM (deep learning recommendation model)**: Facebook's 2019 recommendation architecture combines embedding tables for categorical features with multilayer perceptrons for continuous features (Naumov et al. 2019). DLRM stresses benchmarks differently than vision or language models: its embedding tables can be large enough that memory capacity and bandwidth dominate compute throughput. That makes DLRM the book's capacity-bound workload in MLPerf-style inference evaluation, revealing hardware limitations invisible to compute-bound benchmarks (Reddi et al. 2019).

Table 12.13: Edge vs. Cloud Deployment Constraints: Typical cloud deployments treat power as an operating cost and latency as a service target, while edge deployments often face tighter energy and response-time limits that constrain achievable accuracy.

Constraint	Cloud Impact	Edge Impact
Power	Operational cost	Device energy budget
Latency	Service-level target	Local response deadline
Accuracy	Task-quality target	Balanced with power/latency

Consider a smartphone camera executing real-time object detection at 30 frames per second within a 3 W to 5 W thermal envelope. In this setting, an efficient architecture such as MobileNet serves as a realistic benchmark target where a larger ResNet model would violate thermal limits, despite the larger model's superior accuracy in cloud evaluations. An edge benchmark must therefore evaluate accuracy jointly with sustained power and thermal dissipation. The peak-versus-sustained gap (section 12.1) is especially acute at the edge, because a passively cooled device sheds heat poorly. As junction temperatures rise, the dynamic thermal management (DTM) controller scales down operating voltage and clock frequency, causing sustained throughput to drop sharply below burst-mode peaks. That thermal mechanism makes the cooling configuration part of the benchmark specification.

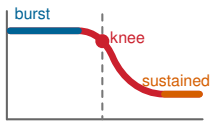
Example 12.2: Benchmarking the edge

Scenario: An edge engineering team benchmarks a candidate chip for the camera trap's board with a vendor-advertised "1 W AI" power specification.

Diagnosis: Early burst-mode testing matches vendor claims, but continuous inference loop execution builds up junction heat, triggering thermal throttling and dropping steady-state throughput.

Systems lesson: Short burst benchmarks obscure long-term thermal throttling. Benchmarking edge ML workloads requires continuous, sustained execution testing to reflect true operational hardware limits.

Throttling is the most visible way a short measurement flatters edge hardware. Idle power, duty cycle, and the data movement around the accelerator can distort a headline number too, so each needs checking before a vendor claim informs a design decision.



Burst benchmark FPS collapses once thermal throttling sets in.

Systems Perspective 12.7: Edge benchmark reality check

When evaluating edge hardware claims, four factors determine whether vendor numbers translate to real-world performance:

1. **Peak vs. sustained**: A vendor may advertise 45 TOPS peak throughput while a sustained thermal run delivers closer to 20 TOPS. In this scenario, use a sustained run longer than 30 s and verify thermal stabilization; also test the deployment duty cycle.
2. **Power at idle vs. active**: In this scenario, a device consuming 50 mW idle and 2 W active could report active draw for marketing, but if the application runs inference 1 percent of the time, effective power draw is ~69.5 mW, not 2 W.
3. **Thermal envelope**: Edge devices often operate inside a narrow TDP envelope. Exceeding it triggers throttling, so benchmark reports omitting thermal conditions are incomplete.
4. **End-to-end vs. accelerator-only**: NPU benchmarks often exclude data transfer overhead. Moving image data from camera to NPU and back can exceed inference time for small models.

A modern mobile SoC packages CPUs, GPUs, digital signal processors (DSPs), and neural processing units (NPUs) on one die that shares system memory and a common thermal budget, so an edge benchmark must measure the pipeline across those engines rather than one engine's kernel

throughput. Single-engine benchmarks miss three costs that fall between the engines: waking an accelerator from a deep sleep state, contention when several engines access shared DRAM, and converting tensor layouts between the formats different engines require. Section 11.10 develops how these SoCs partition work.

12.6.5 Building the edge-board benchmark

The camera trap's board is the case for the inference measurements above. It pairs a low-power host CPU with an attached INT8 accelerator, and the claim inherited from the compression and acceleration chapters is that INT8 MobileNetV2 on the accelerator meets the camera's frame deadline at a fraction of the host's energy. The benchmark is built in the order a careful engineer would follow: precision on the host first, then the accelerator, then the gates that decide whether the result can ship. The labeled composite values below are illustrative, not measurements of a named product.

Example 12.3: Benchmarking a vision model for edge deployment

Scenario: An edge AI team evaluates MobileNetV2 on the camera trap's host CPU, first at FP32 and then at INT8, under strict latency and model size limits (Sandler et al. 2018).

Diagnosis: FP32 execution yields 50 ms latency (20 FPS) and 14 MB size. INT8 quantization speeds up inference by 3.3× to 15 ms (66.7 FPS) and reduces model size by 4×, at a cost of 0.9 percentage points Top-1 accuracy.

Systems lesson: Edge model benchmarking requires evaluating multi-dimensional trade-offs across latency, accuracy, and memory payload. Quantization enables high-throughput edge execution when modest accuracy drops meet product requirements.

Lighthouse 12.2: MobileNetV2 on the edge accelerator

Extending lighthouse 12.1, the hardware-acceleration claim is evaluated using an illustrative SingleStream-style edge-inference scenario (Reddi et al. 2019; C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kiraly, et al. 2021).

Hardware acceleration claim: In this illustrative edge-accelerator scenario, assume INT8 MobileNetV2 inference takes ~2 ms on the accelerator, approximately 7.5× faster than the same board's low-power host CPU (~15 ms). The host CPU column is the INT8 host result of example 12.3, so both measurements describe the same board. Actual results depend on operator coverage, clock frequency, thermal state, and implementation.

Table 12.14 reports the illustrative SingleStream-style scenario assumptions across distinct measurement scopes.

Table 12.14: Edge Accelerator vs. Host CPU MobileNetV2 Scenario: Illustrative SingleStream-style latency, power, and energy values show how measurement boundaries alter comparative ratios. While end-to-end latency captures host preprocessing and link transfer overhead, the power and energy rows reflect active component execution alone rather than whole-system draw; these are illustrative values rather than controlled laboratory measurements.

Metric	Measurement Scope	Host CPU	Accelerator	Comparative Ratio
Inference latency	Isolated model compute	~15 ms	~2 ms	7.5× faster (isolated headline)
End-to-end latency	Full request path (prep + link + exec)	~18 ms	~6 ms	~3× faster (system-level)
Active power	Component during inference execution	~120 mW	~500 mW	~4.2× higher (component active)
Active energy per inference	Component active energy ($P \times t_{\text{inference}}$)	~1.8 mJ	~1 mJ	~1.8× more efficient (component active)

What this reveals: Under these assumptions, the 7.5× inference speedup narrows to ~3× end to end because preprocessing runs on the host CPU in both cases (figure 12.6). The accelerator path incurs additional latency copying each input across the host-accelerator link and retrieving the result, a transfer overhead the CPU-only path never pays.

Furthermore, the 1.8× energy advantage applies specifically to the active inference interval ($P \times t_{\text{inference}}$), measuring component-level efficiency rather than total system consumption. This metric excludes host CPU preprocessing energy, interconnect transmission power, and baseline idle draw during non-inference phases. Because end-to-end latency improves by only 3×, host baseline power consumed during these non-accelerated phases narrows the whole-system battery efficiency gain.

Workload arrival patterns determine whether component-level energy efficiency translates to extended battery life. For intermittent duty cycles, the lower active inference energy favors the accelerator, but total battery drain depends on sleep power, wake-up latency and energy overhead, host transfer costs, and any standby leakage the accelerator draws while awaiting work. For continuous streaming workloads such as video analytics, the accelerator's lower active energy per inference dominates cumulative energy consumption.

The SingleStream result illustrates why benchmarking requires matching the MLPerf scenario to the deployment context: SingleStream emphasizes latency, while Offline benchmarks would optimize for bulk throughput rather than latency bounds.

The board's INT8 results support further evaluation, but they do not establish that the camera trap's service objective has been met. The decision requires the request population, latency statistic, quality floor, and memory and energy budgets to be specified together.

Table 12.15 identifies the remaining evidence. A camera pipeline needs sustained completion times under its input cadence; an intermittent application also needs initialization and sleep-state costs. For either application, the candidate must retain the required model quality on representative inputs. The same model-level speedup can therefore support one deployment and fail another.

Table 12.15: Inference Performance Metrics: Latency, throughput, and resource usage metrics provide a quantitative basis for optimizing deployed machine learning systems and selecting appropriate hardware configurations, balancing speed, cost, and accuracy in production applications.

Category	Key Metrics	Example Benchmark Use
Latency and Tail Latency	Mean latency (ms/request); Tail latency (p95, p99, p99.9)	Evaluating interactive and soft real-time performance
Throughput and Efficiency	Queries per second (QPS); Frames per second (FPS); Batch throughput	Comparing large-scale cloud inference systems
Numerical Precision Impact	Accuracy degradation (FP32 vs. INT8); Speedup from reduced precision	Balancing accuracy vs. efficiency in optimized inference
Memory Footprint	Model size (MB/GB); RAM usage (MB); Memory bandwidth utilization	Assessing feasibility for edge and mobile deployments

Category	Key Metrics	Example Benchmark Use
Cold-Start and Load Time	Model load time (s); First inference latency (s)	Evaluating responsiveness in serverless AI
Scalability	Efficiency under load; Multi-model serving performance	Measuring robustness for dynamic, high-demand systems
Power and Energy Efficiency	Power consumption (W); Performance per W (QPS/W)	Optimizing energy use for mobile and sustainable AI

The acceptance decision can reject a candidate on any binding constraint. A lower model-only latency does not establish an end-to-end gain if preprocessing dominates, and a smaller weight file does not establish that runtime buffers fit the device. The latency decomposition and Amdahl calculation in section 12.3.4 specify which additional measurements the MobileNetV2 comparison needs.

Because physical constraints differ across environments, no single metric ordering governs all systems. Benchmarking requires establishing a priority hierarchy derived directly from the deployment specification. Table 12.16 illustrates how metric priorities shift across five operational contexts, mapping hardware and operational constraints to optimization targets.

Table 12.16: Illustrative Performance Metric Priorities by Deployment Context: A sample priority ordering shows how operational constraints can change benchmark selection and result interpretation; actual thresholds must come from the deployment specification.

Deployment Context	Primary Priority	Secondary Priority	Tertiary Priority	Key Design Constraint
Real-Time Applications	Latency	Reliability	Memory Footprint	User experience demands immediate response
Cloud-Scale Services	Throughput (QPS)	Cost Efficiency	Average Latency	Business viability requires massive scale
Edge/Mobile Devices	Power Consumption	Memory Footprint	Latency	Battery life and resource limits dominate
Training Workloads	Training Time	GPU Utilization	Memory Efficiency	Research velocity enables faster experimentation
Scientific/Medical	Accuracy	Reliability	Explainability	Correctness cannot be compromised for performance

A metric that dictates architectural success in one deployment context may be irrelevant in another. Latency ranks first for autonomous robotics, where sensor-processing pipelines must complete within strict execution deadlines to maintain vehicle stability, whereas cloud batch services accept higher queuing latency in exchange for lower operational cost per query. A 2× throughput gain represents substantial economic value in a hyperscale cloud cluster, but it provides little benefit on a battery-powered device such as the camera trap, where a 20 percent power reduction extends operational lifetime.

For the camera example, use the required deadline or percentile as the timing gate, the runtime peak allocation as the capacity gate, and the permitted quality loss as the model gate. If any gate lacks a measurement, the decision remains unresolved even when the reported inference speedup is large. A protocol should identify the missing evidence rather than treating the favorable metric as approval. An on-demand deployment adds a fourth gate, cold-start latency, measured on the user request path as section 12.6.2.3 specifies.

Beyond initialization delays, inference benchmarks become misleading when operational metrics are reported without functional validation. Numerical precision optimization introduces a significant comparability trap across accelerator platforms. Accelerators provide substantially higher INT8 operation throughput[27] than baseline FP32 floating-point rates, but these peak figures are meaningful only when the benchmark verifies accuracy retention under quantization noise, confirms hardware support across all model operators, and enforces equivalent operation definitions across architectures.

Once a candidate clears quality and capacity gates, varying offered load tests how far its timing result transfers. A runtime can sustain a favorable latency with one request in flight yet miss the

[27] **TOPS (tera operations per second)**: A measure of raw computational throughput (trillions of operations/second). The H100 delivers 1979 TOPS INT8 vs. the Apple M2 Neural Engine at 15.8 TOPS and Edge TPU at 4 TOPS, but these numbers conflate different operation types (multiply-accumulate, accumulate, or activation). TOPS comparisons across vendors are meaningful only when the operation definition, precision, and sparsity assumptions are identical, conditions rarely met in vendor specifications.

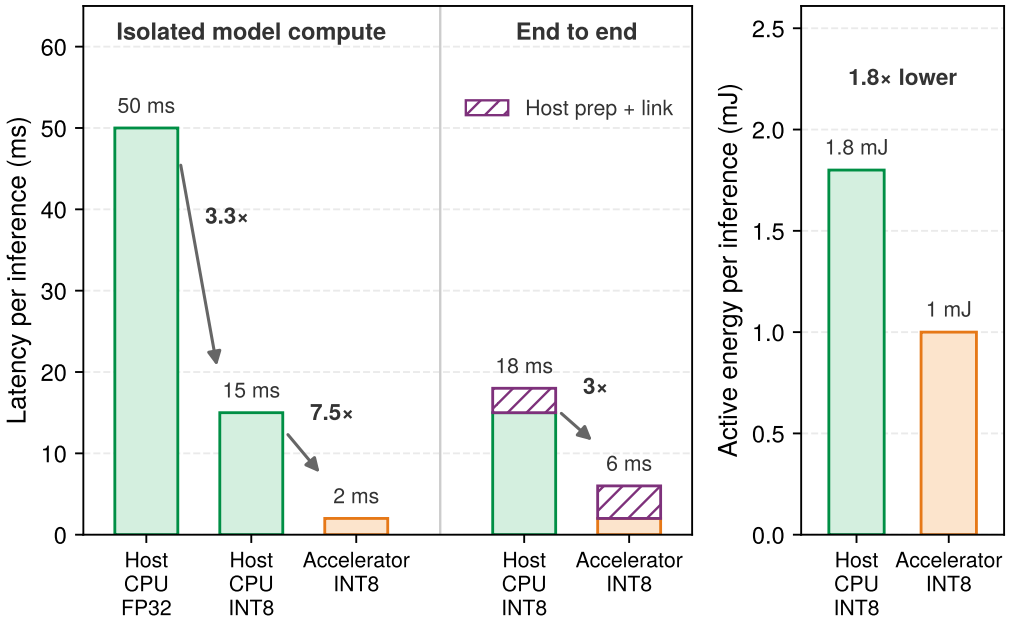


Figure 12.6: From Claimed to Delivered Speedup on the Edge Board: MobileNetV2 latency per inference (left) and active energy per inference (right) on the camera trap's board. INT8 cuts the host CPU's isolated latency by 3.3×, and the accelerator cuts it a further 7.5× when only model compute is timed. Widening the boundary to include host preprocessing and the host-accelerator link (hatched) leaves a 3× end-to-end advantage, and the active-energy advantage is 1.8× before idle and host power are counted. All values are the chapter's illustrative scenario, not measurements of a named product.

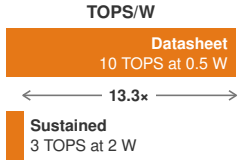
target when queues and shared memory fill. The relevant outcome is the supported load under the declared latency bound, measured with the same input mix and power policy, rather than an isolated peak throughput.

The board now has a latency, a throughput, and a memory footprint. What it lacks is the number that decides whether a battery-powered camera trap lasts a season in the field: the energy each frame costs.

12.7 Power and Energy Measurement

A chip vendor advertises "10 TOPS at 0.5 W," but under sustained inference load, thermal throttling drops actual throughput to 3 TOPS at 2 W—a 13.3× efficiency gap between the datasheet and reality that goes undetected until deployment unless power is measured under load. Energy is the one benchmark quantity that cannot be read off a clock: it must be integrated over a stated measurement boundary and window, and for a duty-cycled device it depends on how the device spends the time between inferences. KWS, the book's power-bound lighthouse, carries the measurement of energy, because a wake-word detector is judged by how long it listens on a battery rather than by how fast it answers; the edge board's frames and the data center's racks appear as contrasts.

This efficiency dimension matters directly because section 11.1 defines acceleration by its compute-density and performance-per-watt gains, making TOPS/W a primary design objective alongside raw TOPS. Power benchmarks validate whether efficiency-optimized accelerators deliver their promised energy savings. TOPS/W is particularly susceptible to gaming precisely because it is a ratio of two separately quotable peaks: a vendor can read the numerator (operations) at the batch size and precision that maximize throughput and the denominator (watts) at a near-idle operating point, so the advertised efficiency describes a state the chip never occupies under real load. Power benchmarks close that loophole by fixing the workload and the measurement window, forcing the numerator and denominator to be read at the same operating point.



Datasheet TOPS/W overstates sustained efficiency by 13.3×.

Measuring power differs from measuring execution time because power is not a single time-stamped interval: instantaneous power (supply voltage times current) fluctuates continuously across

microsecond execution phases and drifts over minutes as silicon heats up, increasing temperature-dependent leakage current. ML deployments also span a wide physical range. The dumbbell plot in figure 12.7 isolates this disparity on a logarithmic power scale, plotting minimum (blue) and maximum (red) envelopes across four deployment tiers: TinyML systems (5.6 mW to 167 mW), edge hardware (3.9 W to 1.1 kW), data center inference nodes (267 W to 6.3 kW), and large-scale training clusters (5.5 kW to 498 kW). Spanning about eight orders of magnitude from the TinyML floor to the training-cluster ceiling, this spectrum explains why no single efficiency metric serves all deployment contexts. A figure of merit optimized for multi-rack data center throughput becomes meaningless for a battery-powered keyword spotter governed by milliwatt budgets.

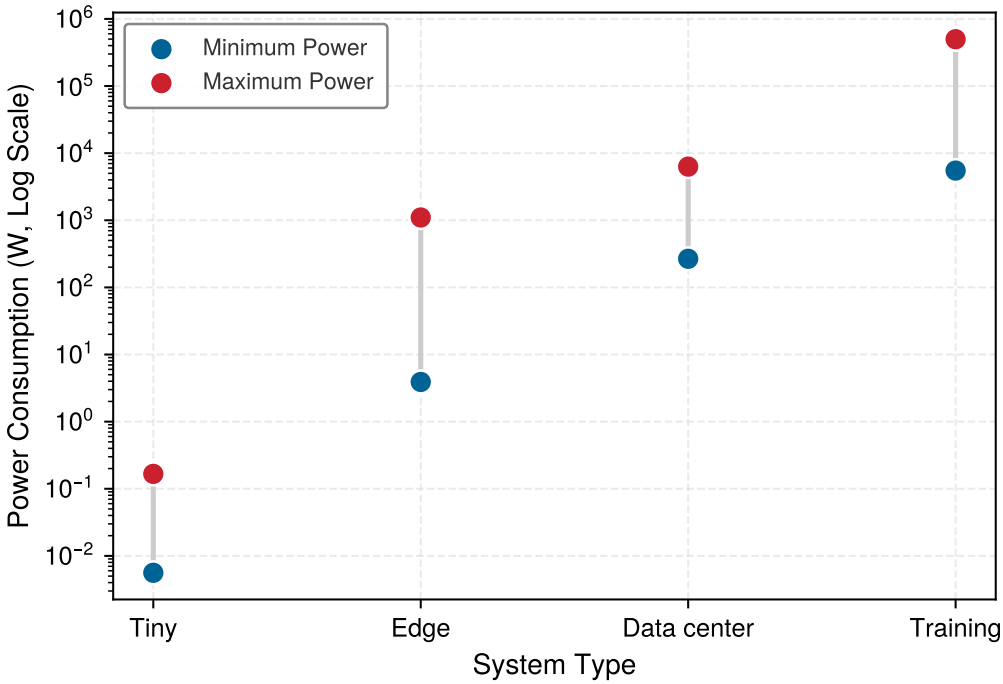


Figure 12.7: Power Consumption Differentials: Measured power spans about eight orders of magnitude across MLPerf divisions, from milliwatts for TinyML submissions through watts at the edge to kilowatts in data center inference and hundreds of kilowatts for training systems. Each pair marks the lowest and highest power among official MLPerf Power submissions in that division, after Tschand et al. (2024).

Because measured power spans from milliwatts to hundreds of kilowatts, no single instrument or measurement protocol applies everywhere. Benchmarking an always-on keyword spotter requires measuring microampere currents at millivolt rails, whereas benchmarking a server rack requires measuring high-voltage three-phase alternating current at the power distribution unit. This divergence forces benchmarking standards to establish precise physical boundaries for what hardware is included in the measured energy.

Energy accounting follows contrasting regimes across training and inference. Training energy is a capital investment amortized across the operational lifetime of the model; a large training run can consume on the order of a thousand megawatt-hours (section 12.5.2.4) (Patterson et al. 2021). Inference energy accumulates continuously with request volume, dominating total lifecycle cost as query counts scale into millions. For an isolated inference window containing one completed request, energy obeys the identity $E_{\text{inf}} = P_{\text{active}} \times L_{\text{lat,infer}}$, with P_{active} the average accelerator power during that window and $L_{\text{lat,infer}}$ its duration; the duty-cycle form used for battery accounting appears in section 12.7.2. With batching or concurrent requests, integrate energy over a common measurement window and divide by completed requests; multiplying total device power by each request's overlapping serving latency would double-count energy. For example, if measured average

accelerator power during the inference window is 300 W, a 10 ms inference uses 300 W $\times$ 0.01 s $=$ 3 J, or about 0.0008 Wh; at 100 ms, that becomes about 0.0083 Wh. An accelerator's TDP rating represents a cooling ceiling rather than dynamic power consumption, making empirical runtime power measurement essential for accurate benchmarking.

12.7.1 Power measurement boundaries

Where an engineer draws the system boundary dictates what counts as efficient. If a benchmark measures only the accelerator core, it ignores the host CPU driving the PCIe bus, the off-chip DRAM holding weights, and the cooling fans dissipating heat. Figure 12.8 partitions these measurement perimeters across three deployment tiers (Tiny, Inference, and Training), where solid olive enclosures demarcate measured components and red dashed boxes isolate excluded infrastructure such as remote storage and data-center chillers (Tschand et al. 2024).

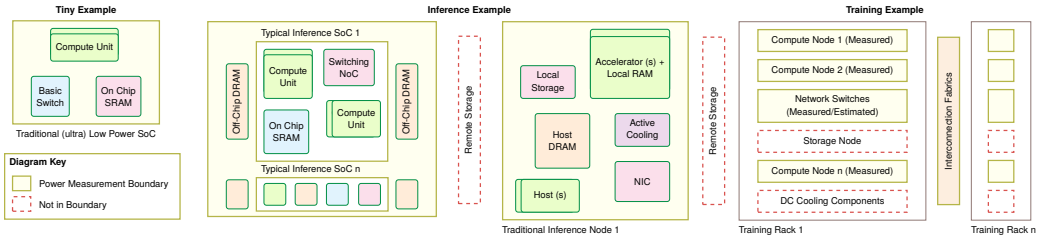


Figure 12.8: Power Measurement Boundaries: MLPerf defines system boundaries for power measurement, ranging from single-chip devices to full data center nodes, to enable fair comparisons of energy efficiency across diverse hardware platforms. These boundaries delineate which components' power consumption is included in reported metrics, impacting the interpretation of performance results. Source: (Tschand et al. 2024).

The three architectural tiers in figure 12.8 (Tiny Example, Inference Example, and Training Example) reflect distinct physical instrumentation points. In TinyML deployments, the entire low-power SoC, including compute cores, internal SRAM, and power management units, falls inside the boundary, measured directly at the DC power rail. Dedicated inference nodes expand this perimeter to include host CPUs, accelerator cards, PCIe buses, local storage, and on-chassis active cooling fans, while excluding remote storage networks. Multi-rack training deployments measure compute nodes and top-of-rack interconnect switches, but exclude centralized data center chillers, facility-level power distribution units (PDUs), and disaggregated storage clusters.

The excluded components can more than double the reported energy. Google measured the energy of the median text prompt served by its Gemini assistant under two boundaries (Elsworth et al. 2025). Counting only the active accelerators, on prompts served by its most efficient data centers, gave 0.10 Wh. Counting everything the service draws to answer that prompt gave 0.24 Wh, about 2.4× more. Of that total, the active accelerators drew 58 percent and the host CPUs and DRAM that feed them 25 percent; the remaining 17 percent went to machines held idle in reserve for traffic spikes and to facility overhead such as cooling and power conversion. An accelerator-only figure therefore understates what a deployed service consumes, and two figures measured under different boundaries are not comparable. Utilization also moves the figure, because the narrow measurement came from the most efficiently used machines. Open benchmark suites such as ML.ENERGY measure energy under realistic serving conditions so that their results reflect deployed utilization rather than an idealized run (Chung et al. 2025).

The measurement boundary determines which components are counted; workload composition determines where their energy is spent. On mobile devices running TensorFlow Mobile, data movement accounts for 57.3 percent of total inference energy (Boroumand et al. 2018), exposing why operation counts alone fail to predict power consumption; under the energy-movement principle (7), total energy depends on how many events of each kind occur and what each one costs, not on arithmetic alone. The worked example in notebook 12.4 decomposes MobileNetV2 inference energy to demonstrate how moving from FP32 to INT8 attacks both the memory-movement bottleneck and arithmetic switching costs.

Napkin Math 12.4: Why INT8 saves energy

Problem: Under the stated 45 nm component-energy model, how much does replacing FP32 with INT8 reduce MobileNetV2 weight-read and arithmetic energy?

Moving data costs far more energy than computing on it (principle 7, grounded in Horowitz's energy estimates (Horowitz 2014)). Understanding *why* quantization reduces energy consumption requires decomposing energy into its physical sources. Two dominant factors determine inference energy: compute operations and memory access.

Narrower datatypes require less switching and storage energy per operation, and off-chip memory costs far more than either. In the 45 nm component model that section 10.3.2 tabulates (table 10.8), an 8-bit multiplier uses about 18.5× less energy than a 32-bit floating-point multiplier, and reading one byte from DRAM, about 160 pJ/byte, costs over 1,600× more than a register access (Horowitz 2014). Absolute values and ratios vary with circuit design and process technology.

Math: Component energy follows the energy corollary, with a load term $D_{\text{vol}} \times E_{\text{move}}$ (model bytes times DRAM energy per byte) and a compute term $O \times E_{\text{compute}}$ (operations times energy per operation). The FP32 terms sum as 2243 µJ + 2,220 µJ = 4,463 µJ; the INT8 terms sum as 561 µJ + 120 µJ = 681 µJ. Dividing the totals gives a 6.6× reduction under this simplified model.

Table 12.17 combines the two effects in a deliberately simplified MobileNetV2 model. It counts one DRAM read per weight and charges every cataloged FLOP at the listed multiplier energy, while excluding activation traffic, cache behavior, additions as a distinct cost, control overhead, and static power:

Table 12.17: MobileNetV2 INT8 Simplified Energy Estimate: One weight read plus a multiplier-energy charge for every cataloged FLOP under a 45 nm component-energy model. This is not a whole-device inference measurement.

Component	FP32 (14 MB)	INT8 (3.5 MB)	Savings
One weight read from DRAM	2243 µJ	561 µJ	4×
Compute (600 MFLOP)	2,220 µJ	120 µJ	18.5×
Total	**4,463 µJ**	**681 µJ**	**6.6×**

Systems insight: Within this simplified model, the weight read and the arithmetic contribute comparable FP32 energy (2243 µJ and 2,220 µJ), and INT8 reduces both, taking the total from 4,463 µJ to 681 µJ. The decomposition explains the physical mechanism behind potential savings, but battery-life or per-inference claims require controlled whole-device power measurements.

Beyond individual server nodes, large-scale deployments introduce shared infrastructure that complicates boundary definition. In data centers, cooling systems and power delivery account for 20 to 30 percent of total facility electricity consumption (Barroso et al. 2019). This facility-level overhead is captured by power usage effectiveness (PUE):

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Equipment Energy}}$$

PUE values range from approximately 1.1 in hyperscale facilities using direct-to-chip liquid cooling to over 2.0 in legacy facilities relying on computer room air conditioning (Barroso et al. 2019). When an evaluation reports only server-plug energy, it excludes this facility multiplier. Furthermore, dynamic voltage and frequency scaling (DVFS) continuously modulates processor power and heat dissipation during execution (Kim et al. 2008), creating thermal feedback loops that alter the cooling overhead itself.

12.7.2 Battery and thermal benchmarking

Battery drain depends on total energy consumed over time rather than peak instantaneous power. Computational photography bursts consume multiple watts for hundreds of milliseconds during image capture, whereas background sensor processing must remain within a milliwatt-scale budget

to deliver multi-day battery life. Characterizing battery impact requires measuring the total energy consumption $E_{\text{total}} = \int P_{\text{device}}(t)\,dt$ across representative operational profiles, where $P_{\text{device}}(t)$ is the instantaneous power the device draws at time t.

The dominant operational variable is the workload duty cycle α_{duty}, defined as the fraction of time the accelerator actively runs inference. For an active power draw P_{active} and standby power draw P_{idle}, the average power is:

$$P_{\text{avg}} = \alpha_{\text{duty}} P_{\text{active}} + (1 - \alpha_{\text{duty}}) P_{\text{idle}}$$

For intermittent workloads such as the camera trap waiting for motion ($\alpha_{\text{duty}} \ll 0.01$), standby power P_{idle} and memory retention power dictate battery lifetime, making active inference efficiency secondary. For continuous pipelines like video analytics ($\alpha_{\text{duty}} \approx 1.0$), active inference energy $E_{\text{inf}} = P_{\text{active}} \cdot L_{\text{lat,infer}}$ dominates. Furthermore, elevated operating temperatures increase semiconductor leakage current, causing static power dissipation to climb as the device heats up. Benchmarking battery impact therefore requires measuring both active and standby power across the operational duty cycle while monitoring junction temperature over extended multi-minute runs.

Keyword spotting makes the point concrete. Run DS-CNN (Y. Zhang et al. 2017), the MLPerf Tiny keyword-spotting reference at 20 MFLOP per inference, on an ESP32-S3-class microcontroller board with one inference per one-second audio window. At the board's peak INT8 rate an inference takes about 8.3 ms, a duty cycle of about 0.8 percent, and costs about 3.3 mJ at the board's 0.4 W active power. Between inferences the board idles at 50 mW. Average power is therefore about 52.9 mW, and 94 percent of it is spent waiting. A benchmark that reports only energy per inference describes the remaining 6 percent of the battery drain. Active energy would overtake the idle floor only if inference occupied more than about 11 percent of the time, and real kernels running below peak raise the active share without changing the conclusion. This is why always-on audio moves to dedicated low-power silicon whose idle draw is microwatts, and why a power benchmark for a duty-cycled device must specify the duty cycle along with the inference.

12.7.3 Computational efficiency vs. power consumption

Power in CMOS logic is dynamic switching power plus static leakage power, and over the range where supply voltage tracks clock frequency, dynamic power grows roughly with the cube of frequency (section 11.10.3 derives both). Two measurement consequences follow. A measured efficiency belongs to the operating point at which it was measured, because a few percent of clock frequency moves power far more than throughput, so a boost-clock burst and a sustained run report different efficiencies for the same chip. Leakage rises with temperature, so a warm device is less efficient than a cold one at the same clock, which is why the power protocol below requires a warm-up to the intended thermal state. Across hardware generations, computations per kilowatt-hour historically doubled approximately every 1.5 years, a long-run trend known as Koomey's law (Koomey et al. 2011), so an efficiency comparison across devices must also state which generation each one represents.

Because frequency scaling quickly hits TDP ceilings, system designers optimize efficiency across the Data, Algorithm, and Machine axes. On the algorithm and data side, reduced-precision arithmetic (INT8 or FP8) directly reduces the capacitance switched in the ALUs and compresses memory traffic across interconnect buses, potentially cutting dynamic energy when the chosen format preserves the required model quality (Jacob et al. 2018; Wu et al. 2020; Gholami et al. 2021). On the machine side, the energy-delay product (EDP) measures one trade-off between energy and completion time. Its minimum need not coincide with minimum energy or maximum throughput per watt. Battery-constrained devices and power-limited data centers must choose operating points against their own latency, throughput, and energy requirements, measuring leakage and whole-system power alongside switching power.

12.7.4 Standardized power measurement

General-purpose power benchmarks such as SPECpower_ssj2008 measure server power across stepped load levels (0 to 100 percent in 10 percent increments) using steady-state transactional workloads (Lange 2009). In contrast, ML workloads swing rapidly in power: instantaneous draw alternates between compute-bound matrix multiplications (where systolic arrays or Tensor Cores

draw peak current) and memory-bound stalls or token transfers (where execution units stall waiting for DRAM or interconnect fetches).

Characterizing these fluctuations requires calibrated physical instrumentation. Software interfaces such as Intel RAPL and NVIDIA NVML expose counters or telemetry with device-specific accuracy, update rates, and polling overhead. The instrument must match the measurement boundary and required uncertainty. MLPerf power protocols use external calibrated analyzers for inference systems, while training and HPC measurements can use validated node or power-distribution telemetry; Tiny measurements use instrumentation suited to low-power devices (Tschand et al. 2024).

Resolving an individual transient requires instrument bandwidth and internal sampling appropriate to its duration; a kilohertz log cannot resolve arbitrary microsecond kernel events. Total run energy instead depends on accurate integration over the matched measurement window, which an analyzer may perform internally at a different rate from its exported readings:

$$E_{\text{total}} = \int_{t_{\text{start}}}^{t_{\text{end}}} P_{\text{device}}(t)\, dt$$

For a steady-state energy comparison, the harness separates initialization from the timed workload through three protocol stages:

1. A warm-up phase runs unmeasured inferences until caches, buffers, and temperature reach the intended operating state.
2. A sustained measurement window executes repeated inferences across minutes to average over microarchitectural phase shifts and thermal drift.
3. Hardware-to-software synchronization aligns power and workload timestamps so integrated energy and completed requests cover the same window. Cold-start or intermittent deployments require additional measurements that include initialization, sleep, and wake-up costs.

Workload structure further alters the power profile. In low-batch Transformer execution, decode often has less weight reuse than prefill and can face a lower bandwidth ceiling; their measured power profiles depend on shapes, kernels, and operating policy. DLRM embedding lookups can make memory traffic a substantial energy contributor, which must be measured alongside dense computation and static power. Batch sizing creates another trade-off: larger batches amortize parameter reads over more inputs and can lower energy per inference, while also changing activation traffic, execution time, and instantaneous power. At the edge, the duty cycle governs the outcome through the duty-cycle average power of section 12.7.2, where the keyword-spotting case shows the idle floor setting battery life.

12.7.5 MLPerf power case study

MLPerf Power (Tschand et al. 2024) translates raw electrical measurements into standardized efficiency metrics: completed inferences per second per watt under a rigorously defined system boundary. This protocol applies uniform measurement principles across data center, edge, and tiny inference scenarios, where the binding constraint shifts from rack cooling and operating cost to battery runtime and microwatt energy harvesting.

Standardizing the boundary prevents misleading comparisons across architectural families. By specifying instrumentation, measurement boundaries, matched logging windows, and the applicable workload constraints, MLPerf Power makes efficiency results interpretable within the tested scenario.

Over successive benchmark submission rounds from 2021 to 2024, standardized measurements capture how architectural improvements, compiler optimizations, and lower-precision arithmetic translate into production efficiency gains. Tracking normalized energy efficiency (samples per joule) across eight submission cycles from v1.0 to v4.1, figure 12.9 charts diverging efficiency trajectories across seven distinct MLPerf Inference workloads.

RetinaNet and ResNet show the largest plotted gains, each reaching 3× its initial normalized efficiency. BERT and GPT-J reach about 1.9×, while DLRM-v2 reaches about 1.7×, Llama 2 about 1.5×, and RNN-T about 1.3×. These trajectories combine changes in hardware, software, numerical formats, and submitted configurations. Static convolutional shapes can benefit from supported low-precision kernels and fusion; low-batch decoding and sparse embedding lookups often give

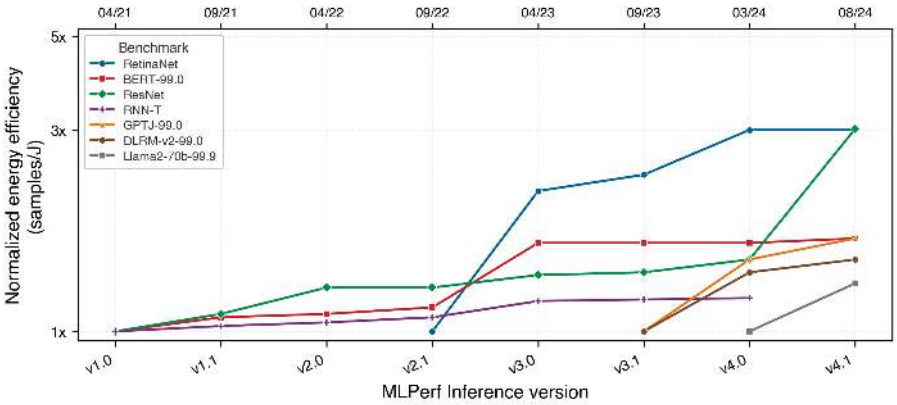


Figure 12.9: Data-Center Energy Efficiency Gains: Each series is normalized to its own first plotted version, so compare improvement within a workload rather than absolute efficiency across workloads. Source: (Tschand et al. 2024).

memory improvements greater importance. The plotted ratios alone do not isolate how much each mechanism contributed.

Timing protocols and power instrumentation provide the raw data for benchmarking. Raw data alone, however, does not guarantee sound conclusions. Converting measurements into meaningful comparisons requires understanding the systematic sources of error, bias, and misalignment that can make even carefully collected benchmark numbers misleading.

12.8 Benchmarking Best Practices

An inference stack that passes a steady-state lab run can still miss latency targets when production traffic arrives in bursts, or when the input mix shifts toward expensive examples. Training throughput, inference latency, and power efficiency each have established measurement protocols validated through MLPerf, but knowing what to measure is insufficient without understanding what benchmarks cannot capture and why this gap causes production deployments to violate their service-level objectives.

Benchmarks make simplifying assumptions that enable standardized comparison but diverge from production reality. Training suites freeze datasets and tightly specify randomness even though production data and retraining conditions evolve continuously, while inference suites emphasize a defined steady-state scenario even though production traffic arrives in bursts with variable sequence lengths, and power benchmarks control thermal environments where deployed accelerators instead face variable ambient temperatures and sustained thermal loads. Four categories of limitations (statistical, deployment-related, system design, and organizational) determine whether benchmark results translate to deployment success.

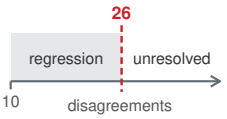
12.8.1 Statistical and methodological issues

A one-point accuracy drop on 1,000 test images or a 2 percent latency improvement between single benchmark runs can sit entirely inside sampling and system noise. Telling a genuine change from a fluke requires the same statistical apparatus for every benchmark in the chapter, from a kernel timing to an accuracy comparison.

To isolate run-to-run variability, benchmark protocols require repeated experimental trials across controlled pseudo-random seeds. The number of runs must satisfy explicit statistical-power targets, specifying the probability of detecting a stated real effect, rather than opportunistic sampling. Reporting must extend beyond single best-of-k scores or isolated arithmetic means: reporting sample variance, standard deviations, or confidence intervals quantifies stability and establishes whether an observed speedup exceeds background system jitter. Without these controls, benchmarking yields misleading conclusions. In reinforcement learning, reported performance advantages frequently evaporate into the statistical noise of random weight initializations and environment dynamics (Henderson et al. 2018). Similarly, unstandardized evaluations of generative adversarial

networks produce rank reversals across random seeds, demonstrating that uncharacterized variance undermines comparative claims (Lucic et al. 2018).

Even when run-to-run execution variance is fully controlled, benchmarking encounters a structural constraint in the evaluation dataset itself: test-set measurement capacity. A test set operates as a measurement instrument with a finite resolution limit. Because accuracy on a finite sample is a binomial proportion, empirical scores carry intrinsic sampling variance that scales inversely with sample size. When an optimization alters accuracy by one or two percentage points, the sample size and paired pattern of disagreements determine whether the test can distinguish a genuine algorithmic effect from sampling fluctuation. As analyzed in the accompanying notebook, marginal accuracy scores on a thousand-sample test set are insufficient to establish the significance of a fine-grained difference without examining the paired outcomes.



At most 26 disagreements reveal a regression; the disagreement count decides.

Napkin Math 12.5: The statistical confidence trap

Problem: A baseline classifier scores 95 percent, while a compressed version scores 94 percent on the same 1,000 images. Does this one-point drop establish a regression?

Math:

1. **Expected errors**: The scores correspond to 50 errors and 60 errors. Under the baseline binomial model, the error count has a standard deviation of about 7 errors.
2. **Difference interval** (95 percent): The same-image comparison requires paired outcomes, which the two marginal scores do not supply. For comparison, suppose the models instead achieved these scores on independent test sets sampled from the same population. Let $\hat{p}_1$ and $\hat{p}_2$ denote their accuracy estimates, and let N be the number of images evaluated for each model. Under that independent-set assumption, the standard error of their difference is

$$\mathrm{SE}(\hat{p}_1 - \hat{p}_2) = \sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{N} + \frac{\hat{p}_2(1-\hat{p}_2)}{N}}.$$

   The observed 1 percentage point difference has an approximate interval from -1 percentage point to 3 percentage points, which includes zero. Treated as independent samples, the one-point drop is indistinguishable from sampling noise. Pairing can change that verdict, because images that both models classify the same way cancel out of the comparison.
3. **Paired test on the same images**: The stated problem uses one test set, so the decisive evidence is where the two models disagree. Let n_{base} count images only the baseline classifies correctly and n_{comp} count images only the compressed model classifies correctly. Images both models get right or both get wrong carry no information about which is better, and McNemar's test (McNemar 1947) discards them. The scores fix the difference $n_{\text{base}} - n_{\text{comp}}$ at 10 (the compressed model's extra errors) but not the sum. The test statistic, without continuity correction, is

$$\chi^2 = \frac{(n_{\text{base}} - n_{\text{comp}})^2}{n_{\text{base}} + n_{\text{comp}}},$$

   and the drop is significant at 95 percent when χ^2 exceeds 3.84. That happens only when the models disagree on at most 26 images.

   Implication: The marginal scores cannot answer the question; the disagreement count decides it. If the compressed model's 10 extra errors are nearly its only disagreements with the baseline, the paired test supplies evidence of a regression at the stated significance threshold. If the models disagree on many more images, the test leaves the same one-point gap unresolved. Nonsignificance does not establish equivalence or compliance with a permitted quality-loss tolerance. A benchmark report must therefore keep per-image

outcomes, not only the two accuracies; the interval-based decision rule below separates an unresolved difference from a supported improvement.

Systems insight: A small test set limits what a benchmark can resolve. The test set, viewed as a measurement instrument, must be sized to match the precision of the change it is meant to detect. A single accuracy estimate alone needs about 1,825 images for ±1 percentage point precision, and that sample size does not by itself power a comparison between two models.

The same paired logic governs timing comparisons. Suppose an engineer measures baseline and candidate mean latency under the same request mix, offered load, and timing boundary. Each experimental trial pairs a complete baseline run with a complete candidate run, following the same warm-up and thermal-state protocol and randomizing their execution order to limit drift bias. The comparison assumes that separately repeated run pairs are independent, with persistent background load or thermal drift controlled or accounted for. Requests within one run can share queues, caches, and thermal conditions and therefore cannot automatically count as independent trials. The report states the interval method and confidence level and uses the run pairs as the repeated observations. A request-latency p99 describes the slow end of the observed request population. It is neither a confidence interval for mean latency nor a measure of uncertainty in the estimated p99.

Suppose the report supplies an uncertainty interval for the mean latency reduction, defined as baseline latency minus candidate latency. If the interval extends from a negative reduction to a positive reduction, it includes no change, and the experiment cannot resolve whether the candidate is faster. That conclusion does not establish that the implementations perform identically. If a subsequent adequately controlled comparison yields an interval entirely above zero, it supports a latency reduction under the tested conditions. Deployment still requires a prespecified minimum useful reduction, reflecting the cost of adopting the candidate. An interval entirely above zero but extending below that threshold supports a gain without establishing that the gain justifies adoption; an interval entirely above the threshold supports both claims, provided the quality and resource gates also pass.

Two further issues undermine reliability even when runs are repeated and compared correctly. Incomplete problem coverage occurs when a benchmark exercises only a narrow operating point of a system. Controlled datasets such as CIFAR-10 (Krizhevsky 2009) constrain inputs to uniform, low-resolution images (32×32 pixels) with small per-image input footprints. This constrained input shape may exercise memory differently from production images, but the complete working set still depends on the model and batch size, and allocation or kernel-launch overheads remain. A model that achieves high accuracy and throughput on a uniform, downscaled benchmark can stall in deployment when exposed to variable image resolutions, arbitrary aspect ratios, and sensor noise that force frequent kernel recompilations or fallback execution paths.

Statistical insignificance arises when evaluations rely on too few trials to distinguish algorithmic improvements from measurement noise, particularly when the evaluation medium introduces variance. Large language model evaluation exemplifies this vulnerability: whether scoring candidate generations against a reference via human preference ratings or an LLM-as-judge protocol, scores vary substantially based on prompt phrasing, judge model alignment, and response ordering. A reported two-point win can vanish under an alternative prompt template or judge configuration. Rigorous evaluation demands paired hypothesis tests and bootstrap confidence intervals over hundreds of independent prompt pairings. Omitting these confidence bounds obscures whether a reported margin reflects a genuine architectural advance or stochastic evaluation noise.

12.8.2 Reproducibility

Reproducibility studies have repeatedly shown that modest benchmark gains can disappear when random seeds, hardware, framework versions, or implementation details change (Henderson et al. 2018). The stochastic processes that section 12.4.2 identified as measurement noise interact, over a long training run, with hardware-specific behaviors (floating-point rounding, memory layout, compiler optimizations) to produce results that can vary meaningfully across environments.

A deeper layer of non-determinism comes from the parallel hardware itself. Operations such as parallel atomic additions, used during gradient accumulation for sparse embeddings in models like Graph Neural Networks, execute in non-deterministic order across threads when concurrent

updates target the same memory location. The resulting floating-point summation order changes across runs, and because floating-point addition is not associative (section 3.7.4), a different order produces bit-for-bit different gradients even with identical inputs and seeds. Enforcing bit-exact reproducibility in these cases requires a deterministic reduction path, which can reduce training throughput, a direct trade-off between reproducibility and performance that benchmark protocols must explicitly address. Without explicit controls for all these sources of variability, benchmark numbers reflect a specific confluence of conditions rather than a system's genuine capability.

MLPerf Training addresses this through standardized data preprocessing, target-quality rules, and repeated accepted runs that characterize stochastic variation (Mattson et al. 2020). The point is not merely to produce a fast run, but to show that the reported performance reflects system capability rather than a favorable combination of stochastic factors.

For a training benchmark, reproducibility is therefore the full run envelope, not the random seed alone. A credible report must preserve the model commit, dataset checksum, preprocessing pipeline, seed plan, framework and compiler versions, precision policy, batch schedule, hardware topology, thermal and power limits, and checkpoint behavior. It must also report the distribution of accepted runs rather than a single best run. Only then can the benchmark separate a real system improvement from a favorable interaction among software version, hardware state, and stochastic training path.

Reproducibility also limits comparisons across software environments. A compiler or library revision can change kernel tiling, memory access, and reduction order, so an unexplained version change is a second intervention in the experiment. The run envelope above must accompany the result; matching a model name and seed alone does not make two stacks equivalent.

12.8.3 System design challenges

Statistical rigor ensures that benchmark measurements are precise, but precise measurements of the wrong operational regime still lead to production failures. Benchmark suites must align with the physical constraints of deployment, and the mismatch is sharpest where a standardized scenario meets live serving traffic. Misalignment with real-world goals persists even when a standardized benchmark is measured correctly. Offline throughput and Server throughput answer different questions, and the Server scenario still specifies only one arrival distribution. A production trace with correlated bursts or a different input mix can produce a different queue and memory footprint. The experiment must state which operating conditions its conclusion covers, and production qualification tests those boundaries before release. The summary returns to these gaps as a release problem once model and data evaluation are in place.

Beyond deployment alignment, a further category of limitations stems directly from the physical hardware executing the workload. Accelerator behavior fluctuates with thermal conditions, operating system resource contention, and underlying microarchitectural topology.

Environmental controls must match the claim being tested. Fixed clocks, CPU affinity, and isolated memory allocations can reduce noise in a comparison of implementations, while a deployment test must retain the relevant contention and thermal policy. The sustained-state mechanisms in section 12.6.4 apply to servers as well as edge devices. A result obtained under tighter isolation supports an implementation comparison, but does not by itself establish production capacity.

The hardware lottery[28] (Hooker 2021) compounds this physical variation by coupling algorithmic viability to specialized hardware support. Modern accelerators allocate most of their silicon area to dense matrix multiplication units (such as Tensor Cores or systolic arrays) backed by wide vector units and high-bandwidth memory hierarchies. Algorithmic architectures such as dense Transformers achieve high arithmetic intensity by reusing weights across batched tokens, sustaining a large fraction of peak theoretical FLOP/s. Conversely, architectures that rely on dynamic control flow, fine-grained sparsity, or graph message passing suffer from uncoalesced memory transactions and thread divergence, underutilizing available compute silicon. Algorithmic performance on a benchmark often reflects how naturally a model's computation graph maps onto dominant accelerator execution units rather than intrinsic mathematical superiority. The lottery is how the bitter lesson shows up on a leaderboard. General methods win by riding growth in computation, but the computation that grows is the dense form the dominant accelerators supply, so the winning methods are also the ones shaped to fit it.

[28] **Hardware lottery**: Coined by Hooker (2021) to describe how algorithmic success depends on alignment with available hardware and software. Transformer workloads benefit from dense matrix operations that map efficiently to modern accelerators, while graph neural networks and sparse mixture-of-experts models can be harder to evaluate when available silicon and software stacks favor dense kernels. Hardware-specific leaderboards therefore favor architectures aligned with the measured platform and may obscure alternatives that would fare differently under other hardware assumptions.

↩ The bitter lesson, that general methods which scale with computation outrun hand-built structure, is established in section 1.4.

Hardware compatibility dependence introduces systematic biases when evaluating models across heterogeneous execution targets. An architecture optimized for GPU execution may underperform on an embedded CPU, DSP, or custom edge accelerator with differing vector widths, cache hierarchies, and quantization support. Across five hardware targets on a Google Pixel 4, figure 12.10 charts how the latency–accuracy Pareto frontier reshuffles between CPU float, CPU uint8, GPU, EdgeTPU, and DSP execution. The diagonal purple arrow links MobileNetV3 Large at roughly 75 percent top-1 accuracy, from approximately 20 ms on CPU float to 4.5 ms on the DSP. Multi-MAX reaches roughly 75 percent accuracy at 2.2 ms on EdgeTPU and 3.4 ms on DSP; its points at higher accuracy carry higher latency. Comparisons must therefore match both the model variant and the accuracy operating point. The shifted frontiers show why model selection requires measurements on the target execution path rather than a latency ranking inherited from another processor.

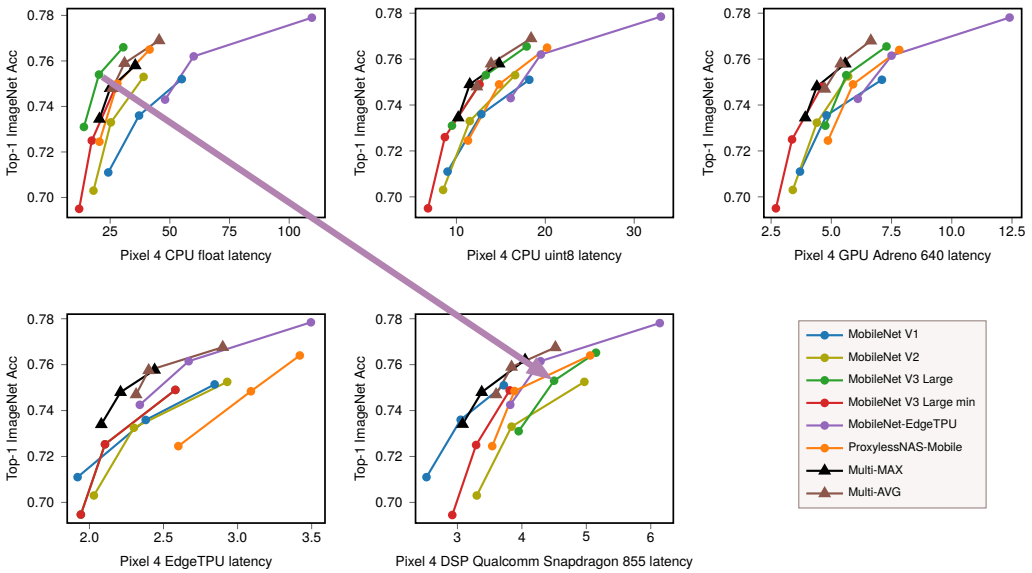


Figure 12.10: Hardware-Dependent Latency–Accuracy Trade-offs: Each panel holds top-1 ImageNet accuracy on the vertical axis while using a target-specific latency scale horizontally. The arrow links MobileNetV3 Large at roughly equal accuracy on CPU float and DSP, exposing the hardware-dependent latency shift. Source: (Chu et al. 2021).

Without benchmarking across heterogeneous hardware configurations, system designers risk selecting architectures solely because they conform to existing accelerator memory hierarchies and instruction sets. Evaluating models across diverse execution engines ensures that architectural decisions reflect computational efficiency rather than platform-specific co-design artifacts.

12.8.4 Organizational and strategic issues

The statistical, deployment, and hardware limitations concern measurement mechanics. A distinct category of failure comes from human incentives, because commercial procurement and academic leaderboards reward a high score more directly than a representative one, and those incentives bias how benchmarks are executed and reported.

12.8.4.1 Benchmark engineering

While the hardware lottery is an unintended consequence of hardware specialization, benchmark engineering is the deliberate practice of tailoring models or systems to exploit the idiosyncrasies of specific benchmark harnesses. This practice yields inflated performance metrics that collapse under production conditions.

Benchmark engineering occurs when developers tune hyperparameters, preprocessing pipelines, or execution paths specifically to maximize benchmark scores rather than generalizable performance. Legitimate optimization respects the run rules and declared workload; misleading benchmark engineering exploits restricted test conditions while making broader claims that the evidence

cannot support. For example, an inference pipeline might hardcode input tensor dimensions and pad batches specifically to match the memory stride of an accelerator's GEMM tile configuration, achieving peak throughput on fixed benchmark shapes while triggering costly kernel fallbacks and memory reallocations on arbitrary production inputs. Similarly, fine-tuning an evaluation pipeline to match the formatting biases of an automated evaluation model produces high leaderboard rankings that fail when interacting with unstructured human dialogue.

Procurement contracts and marketing claims reward exactly this tuning. It is Goodhart's Law (section 12.1) acting on an organization rather than on a single team's choice of metric.

To keep this benchmark gaming from corrupting procurement and engineering evaluations, standardized suites enforce strict run rules backed by reference implementations, submission audits, and division boundaries. Their rules forbid three classes of non-generalizable shortcuts:

1. **Precision truncation without disclosure**: A compiler silently lowers numerical precision (for instance, executing intermediate activations in FP8 or INT8 instead of BF16) to inflate arithmetic throughput, without disclosing the reduced dynamic range or verifying whether the lower precision degrades numerical stability on production workloads.
2. **Work pruning and operator elimination**: An execution engine exploits fixed benchmark test sets to skip operations required by general production inputs, such as bypassing dynamic padding removal, masking out-of-bounds tokens, or skipping input boundary validation.
3. **Benchmark fingerprinting**: A runtime inspects input tensor dimensions, dataset hash fingerprints, or memory layout patterns to identify the benchmark suite, executing hardcoded, pre-tuned execution graphs and bespoke kernel configurations that are inaccessible to arbitrary user workloads.

MLPerf detects and deters these shortcuts through its submission divisions (defined in section 12.4.4), reference-equivalence checks, and peer-reviewed code audits. The closed division permits reproducible runtime optimizations, memory layout transformations (such as converting NCHW to NHWC), hardware-generalizable kernel fusion, and approved low-precision formats with documented, verifiable output quality, while non-generalizable shortcuts remain prohibited. Broader architectural alterations belong in the open division.

12.8.4.2 Bias and over-optimization

System engineers evaluating benchmark results must distinguish legitimate architectural gains from benchmark engineering, and several practices expose the difference. Submissions that publish complete compiler flags, kernel configurations, and execution traces let independent evaluators separate general efficiency gains from harness-specific tuning. Paired results from controlled benchmarks and uncurated production traces expose discrepancies in latency distributions and memory use, and evaluating across several independently maintained suites raises the barrier against overfitting, because an optimization tailored to the quirks of one harness rarely transfers to others.

Third-party verification adds accountability. Independent audits check whether reported speedups reproduce on identical hardware, as in MLPerf's reference-versus-submission validation (section 12.8.4.1), and application-specific stress tests probe failure modes that static benchmarks omit, such as vision models under adverse imaging conditions or inference engines under skewed request traffic.

12.8.4.3 Benchmark evolution

Benchmarks are not static standards; they must evolve alongside model architectures and hardware capabilities. A benchmark that effectively separates system capabilities in one architectural generation becomes obsolete when models saturate its accuracy ceiling or when hardware innovations shift the primary system bottleneck. Stale benchmarks incentivize over-optimizing legacy computational kernels rather than advancing system efficiency.

Early deep learning benchmarks evaluated image classification and object detection on convolutional networks. As workloads expanded to large language models, recommendation systems, and generative AI, those vision benchmarks failed to capture long-context attention, autoregressive decoding, and sparse expert routing, and community benchmarks moved to language understanding (A. Wang et al. 2018, 2019) and foundation model capabilities (Liang et al. 2022). How saturation erodes a static benchmark's discriminating power is a data question, taken up in section 12.9.2.

Benchmark evolution also expands the dimensions of measurement. While early benchmarks focused on throughput and accuracy, contemporary systems require multi-objective evaluation across energy efficiency, memory footprint, and tail latency under resource constraints. Scientific instruments provide a further constraint regime in figure 12.11. Its shaded "Fast ML for Science" envelope places high-throughput trigger pipelines outside typical mobile and IoT operating points (Duarte et al. 2022). The distance between these operating points is why a benchmark must match its application.

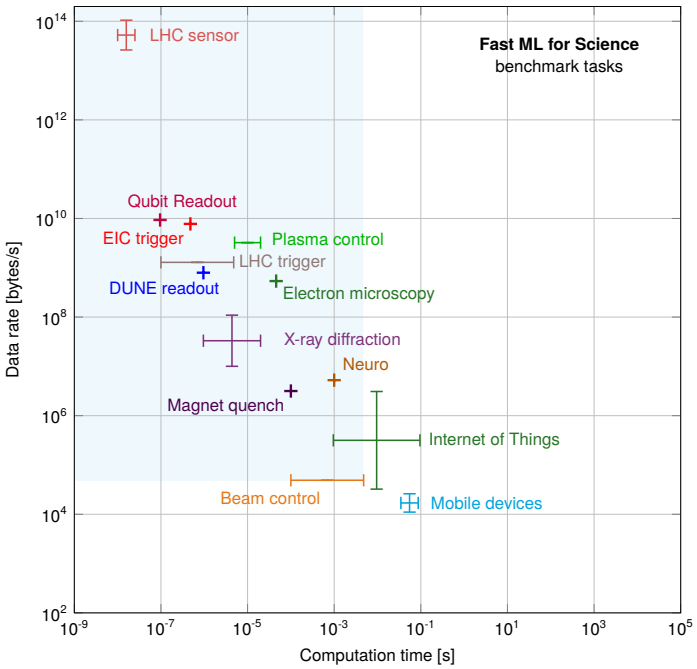


Figure 12.11: Performance Spectrum: Scientific and edge applications occupy widely separated operating points in data rate and available computation time, motivating benchmarks tied to each application's timing and data-movement constraints. Source: (Duarte et al. 2022).

Managing benchmark evolution introduces a direct trade-off between longitudinal stability and architectural relevance. Benchmarks must remain fixed long enough to allow rigorous cross-generation hardware and software comparisons. If benchmarks change too frequently, tracking historical progress becomes impossible. Conversely, freezing a benchmark indefinitely leads to metric stagnation and over-specialization. Modern benchmarking consortia balance this trade-off by operating on fixed release cycles that introduce new representative workloads while deprecating obsolete suites.

Even the most rigorous system benchmarks validate only one dimension of deployment readiness. Achieving record throughput and energy efficiency on an MLPerf workload confirms that the hardware and runtime stack can deliver high computational efficiency, but it cannot determine whether the deployed model generalizes to real-world distributions or whether the underlying dataset represents the operational population. In the D·A·M taxonomy, hardware throughput (Machine) is a necessary foundation, but algorithmic fidelity (Algorithm) and dataset integrity (Data) must hold simultaneously. Validation therefore continues with the model and the data.

12.9 Model and Data Evaluation

A compressed model running on accelerated hardware, such as the camera trap's INT8 MobileNetV2, can still fail if it was trained on unrepresentative data. System benchmarks can confirm that hardware delivers promised training throughput, inference latency, and power efficiency, but hardware validation alone cannot ensure deployment success. The optimization pipeline from Part III also included model compression and data selection, each requiring its own validation. The framework's

other two dimensions address this gap: model benchmarks verify that compression preserved accuracy and essential model properties, while data benchmarks verify that training data enables robust generalization.

12.9.1 Model benchmarking

A pruned model can maintain ImageNet top-1 accuracy while losing robustness to adversarial inputs; a quantized model can preserve average-case performance while degrading on rare edge cases; a distilled model can match its teacher's accuracy while losing confidence calibration. Model benchmarks validate whether compression preserved the properties that matter for deployment beyond top-line accuracy. Historically, benchmarks focused almost exclusively on accuracy, but compression makes multi-dimensional evaluation essential.

↩ Pruning, quantization, and distillation, the compression techniques these benchmarks validate, are established in chapter 10.

Evaluating a compressed model requires comparing its behavior against an uncompressed reference baseline. In computer vision, standard network architectures establish the accuracy and resource baselines against which compression techniques are judged: a pruned or quantized model must demonstrate how much task accuracy it sacrifices for a given reduction in latency, memory footprint, or energy consumption. Canonical reference models such as ResNet-50[29] serve as standard MLPerf workloads precisely because their moderate scale (25.6M parameters) and well-understood compute profile (8.2 GFLOP per image) make them sensitive to both hardware and software optimizations without requiring distributed multi-node setups (He et al. 2016a; Mattson et al. 2020).

29 | **ResNet (residual network)**: Introduced by He et al. (2016a), skip connections enabled deep residual networks and achieved 3.57 percent top-5 ImageNet error (ensemble), surpassing the estimated human error rate in the ImageNet challenge context (Russakovsky et al. 2015).

12.9.1.1 Accuracy metrics and their blind spots

The most common model metrics (accuracy, precision, recall, F1) each reveal different aspects of model behavior while hiding others, and understanding their blind spots is essential for compression validation. Top-k accuracy measures whether the correct label appears in the model's top-k predictions. Top-1 accuracy is strict; top-5 is lenient. The gap between them reveals ranking behavior: a model with 75 percent top-1 but 95 percent top-5 accuracy often ranks the correct class among its leading candidates without placing it first. These rank statistics do not establish whether its confidence scores are calibrated. For deployment, the acceptable gap depends on whether downstream systems can use ranked predictions or require single answers.

Precision and recall matter when classes are imbalanced or errors have asymmetric costs (Sokolova and Lapalme 2009). A camera-trap classifier with 99 percent accuracy might have 10 percent recall on a rare species (catching only one in 10 frames that contain it), an unacceptable outcome despite high accuracy. Precision (the fraction of predicted positives that are correct) and recall (the fraction of actual positives identified) expose failures that aggregate accuracy conceals.

↪ Disaggregated evaluation as a release gate, with the slices, metrics, and thresholds that decide whether a model ships, is developed in section 15.3.2.

Aggregate metrics also conceal subgroup failures. A model achieving 95 percent overall accuracy might achieve 60 percent on a critical demographic subgroup. The Gender Shades project (Buolamwini and Gebru 2018) revealed commercial gender-classification systems for facial analysis performing substantially worse on darker-skinned women than on lighter-skinned men, a disparity invisible to aggregate benchmarks. Disaggregated evaluation across deployment-relevant subgroups is essential.

12.9.1.2 Confidence calibration

For many deployment scenarios, *how confident* the model is matters as much as *what* it predicts. A well-calibrated[30] model's confidence scores correspond to actual correctness probability: among 100 predictions made with 80 percent confidence, about 80 should be correct in expectation.

Calibration failures create severe operational hazards in automated pipelines. An overconfident model automates catastrophic errors that it should defer to human review (for example, reporting 95 percent confidence but failing 30 percent of the time). Conversely, an underconfident model floods manual review queues with decisions it could safely automate (reporting 60 percent confidence while being correct 98 percent of the time).

To quantify calibration across a dataset, predictions are partitioned into N_{bins} confidence bins $\text{bin}_1, \text{bin}_2, \ldots, \text{bin}_{N_{\text{bins}}}$ (such as ten intervals of width 0.1 from 0 to 1). For each bin_m, the evaluator computes the empirical accuracy $\text{acc}(\text{bin}_m)$ (the fraction of correct predictions in that bin) and the

30 | **Calibration**: From Arabic *qalib* (a mold for casting metal) via Latin *calibrare*, originally describing the adjustment of measuring instruments against known standards. In ML, calibration ensures predicted probabilities match empirical frequencies; Guo et al. (2017) formalize this concern for modern neural networks and show that temperature scaling is a simple effective post-hoc correction. The etymology is apt: just as an uncalibrated instrument produces precise but inaccurate measurements, an uncalibrated model produces confident but unreliable predictions, causing downstream systems that threshold on confidence scores to make systematically wrong decisions.

average predicted confidence $\text{conf}(\text{bin}_m)$. **Expected calibration error (ECE)** (Pakdaman Naeini et al. 2015) measures the weighted average difference between confidence and accuracy in equation 12.4:

$$\text{ECE} = \sum_{m=1}^{N_{\text{bins}}} \frac{|\text{bin}_m|}{N} |\text{acc}(\text{bin}_m) - \text{conf}(\text{bin}_m)| \tag{12.4}$$

where N is the total sample count and $|\text{bin}_m|$ is the sample count in bin m. A measured $\text{ECE} = 0$ means confidence and accuracy agree within the chosen finite-sample bins. Population calibration implies agreement in expectation; a zero estimate does not prove calibration at every confidence level or on another distribution.

Compression frequently disrupts calibration even when preserving aggregate accuracy, a central concern when validating quantization techniques from section 10.3. When floating-point weights and activations are quantized to 8-bit integers, discrete rounding alters the numerical geometry and relative separation of final-layer logits. Often, headline top-1 accuracy barely moves, but the model becomes severely overconfident, assigning extreme probabilities to inputs where it is genuinely uncertain. Post-hoc techniques such as temperature scaling can improve calibration when fitted on separate calibration data and evaluated on an untouched representative holdout (Guo et al. 2017).

12.9.1.3 Compression benchmarks

Neural network compression (pruning, quantization, knowledge distillation, and architecture optimization) requires specialized benchmarks because compression reshapes the trade-off landscape: every byte saved or operation eliminated must be weighed against potential accuracy loss and hardware compatibility. The most basic compression metric is raw size reduction: parameter count, memory footprint in bytes, and compressed storage requirements. Size alone, however, is misleading. On ImageNet, MobileNetV2 achieves approximately 72 percent top-1 accuracy with 3.5M parameters vs. ResNet-50's 76 percent accuracy with 25.6M parameters, about 7.3× fewer parameters at comparable accuracy (Sandler et al. 2018; He et al. 2016a).

Pruning benchmarks must distinguish structured from unstructured sparsity, because they produce qualitatively different results on real hardware, as the storage and execution break-even of section 10.4.2.2 shows. Structured pruning yields smaller dense operations that conventional kernels exploit (Li et al. 2017). Unstructured pruning can reach very high sparsity but realizes speedups only with specialized sparse computation support, so protocols must specify hardware platform and software implementation (Han et al. 2015; Gale et al. 2019).

Quantization benchmarks evaluate precision reduction across data types. In the illustrative MobileNetV2 scenario, INT8 reduces raw weight storage by 4× and the assumed latency values yield the speedup in example 12.3; realized performance depends on the hardware and implementation. The precision-accuracy trade-off (Jacob et al. 2018) is worked in lighthouse 12.3 and the energy implications in section 12.7.1. Mixed-precision assignment, which keeps sensitive layers at FP16 while computation-heavy layers use INT8 or INT4, multiplies the configurations a benchmark must name. Knowledge distillation (section 10.2.3) adds a smaller student whose benchmark must verify that it generalizes rather than merely memorizing the teacher's outputs (Hinton et al. 2015).

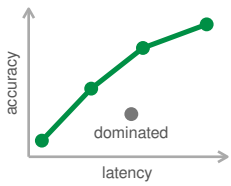
Acceleration factors vary across hardware platforms because sparse models, reduced-precision models, and efficient architectures need compatible kernels, memory layouts, and accelerator support. Current benchmark suites like MLPerf focus primarily on standardized reference models, while production deployments often use compressed or hardware-specific variants. A compression report must therefore name its representation and execution path before an efficiency-quality comparison can establish whether the variant is deployable.

12.9.1.4 Compression validation: The efficiency-quality frontier

A compression report identifies the representation and hardware path, but deployment acceptance requires deciding whether their measured gains justify changes in model behavior. Size, latency, and energy improvements must be evaluated against the quality properties and limits the application depends on.

Pareto frontier[31] evaluation (principle 5) determines whether a compressed model represents a good trade-off. Plotting accuracy against the target efficiency metric (latency, model size, energy)

[31] **Pareto frontier**: Named after economist Pareto (1896), the frontier contains all solutions where improving one objective requires degrading another. In compression benchmarking, the trade-off between adjacent candidates must be interpreted using the named axes and units. A candidate is dominated when another is at least as good on every objective and strictly better on one; frontier membership still does not establish that deployment thresholds are met.



A dominated model is matched or beaten on both accuracy and latency by a frontier model.

reveals the trade-off frontier. Models on the Pareto frontier *cannot* improve one metric without degrading the other; models below the frontier are dominated by better alternatives.

Different compression techniques make different efficiency-quality trade-offs: quantization reduces precision while often preserving aggregate accuracy (Jacob et al. 2018), pruning induces sparsity while retaining varying levels of test performance (Han et al. 2015; Gale et al. 2019), and distillation transfers knowledge from a larger teacher or ensemble into a smaller model (Hinton et al. 2015). Because these aggregate results do not establish preserved calibration or tail behavior, validation must evaluate those properties directly (Guo et al. 2017).

Evaluating calibration during compression requires a standardized protocol. Expected calibration error (ECE) compares predicted confidence with empirical accuracy, but it has no universal thresholds. The metric depends on the binning scheme, bin count, sample size, and prediction distribution. A benchmark protocol must therefore freeze the estimator and evaluate degradation relative to the uncompressed baseline under a fixed task tolerance.

Pareto optimality identifies nondominated choices, but it does not establish that any choice clears the deployment's absolute quality floor. Acceptable degradation depends on deployment context: a 2 percent accuracy drop might be acceptable for a recommendation system where users tolerate imperfect suggestions, but unacceptable for medical diagnosis where every false negative carries severe consequences. Production acceptance gates must define absolute accuracy thresholds before compression, evaluating aggregate accuracy, calibration stability, and deployment-critical slices simultaneously. The MobileNetV2 lighthouse makes this multi-dimensional validation protocol concrete.



Lighthouse 12.3: MobileNetV2 INT8 compression

Returning to lighthouse 12.1, consider an illustrative validation protocol for INT8 quantization, grounded in MobileNetV2's architecture (Sandler et al. 2018) and post-training quantization practice (Jacob et al. 2018). The values in table 12.18 are assumed. For this example, acceptance permits less than one percentage point of loss on either top-one or top-five accuracy and requires no regression in ECE or critical edge-case accuracy under the same evaluation protocol:

Precompression baseline: MobileNetV2 achieves 71.8 percent top-1 accuracy on ImageNet at 3.5M parameters (14 MB FP32).

In table 12.18, aggregate accuracy barely changes after INT8 quantization to 3.5 MB, but calibration error and edge-case accuracy reveal substantial degradation. The INT8 model's ECE rises from 0.031 to 0.089, failing this example's no-regression criterion under the fixed ECE protocol.

Table 12.18: MobileNetV2 INT8 Postcompression Metrics: Illustrative FP32 and INT8 values for top-1 and top-5 accuracy, calibration error, and edge-case accuracy, showing how aggregate accuracy can mask calibration and tail-case degradation.

Metric	FP32	INT8	Acceptable?
Top-1 accuracy	71.8%	70.9%	✓ (0.9 pp drop; below 1 percentage-point threshold)
Top-5 accuracy	91%	90.4%	✓ (below the stated accuracy-loss threshold)
Calibration ECE	0.031	0.089	No (degraded)
Edge-case accuracy	68.2%	61.4%	No (drop of 6.8 pp)

Edge-case definition: Images with >50 percent occlusion, <100 lux lighting, or $>30°$ rotation from training distribution (approximately 5 percent of real-world inputs).

What this reveals: Under these assumptions, average-case accuracy looks acceptable (0.9 percentage-point drop), but calibration degrades and edge-case accuracy drops 6.8 percentage points. If the deployment uses confidence thresholds (for example, "only act if confidence > 85 percent") or encounters many edge cases (unusual lighting, partial occlusions), INT8 MobileNetV2 could fail despite passing aggregate benchmarks.

Fix: Evaluate temperature scaling as a post-hoc calibration candidate (Guo et al. 2017). Fit a positive scalar T_{cal} on separate calibration data to divide logits before softmax: $\text{softmax}(z_i/T_{\text{cal}})$. Validate on an untouched holdout. This rescaling preserves the top-one ranking, so edge-case classification errors require separate diagnosis of quantization and data coverage, followed by correction and testing on reserved edge-case examples.

Pruning trade-offs are also nonlinear. Iterative pruning finds subnetworks that match the dense network's accuracy up to a critical sparsity and degrade sharply beyond it (Frankle and Carbin 2019), the evidence behind the lottery ticket hypothesis of section 10.2.1.6. Compression validation must therefore establish empirical trade-off curves for each specific model and architecture, identifying where the model sits on the Pareto frontier and whether further pruning yields hardware efficiency or merely degrades accuracy.

12.9.1.5 Large language model benchmarks

The compression evaluation framework applies cleanly when the task has a stable label, such as classification accuracy, detection mAP, or segmentation IoU. Large language models break that pattern. A team can choose a model because it scores well on a public benchmark, then discover in deployment that the model recognizes multiple-choice facts but cannot generate a grounded answer, responds too slowly for an interactive product, or produces confident unsafe text that the benchmark never stressed. LLM benchmarking therefore starts by naming the deployment failure that a score is meant to rule out.

The metric taxonomy in table 12.19 organizes evaluation around those failure modes rather than leaderboard rankings. Its rows use Massive Multitask Language Understanding (MMLU),[32] HELM (Holistic Evaluation of Language Models),[33] and perplexity[34] as examples of scores that answer distinct deployment questions, while the responsiveness row is measured as time to first token and time per output token, defined with the other inference metrics in section 12.6.2.2:

Table 12.19: Failure-Oriented LLM Benchmark Taxonomy: LLM metrics are useful when each score is tied to the deployment failure it is meant to rule out. Recognition, holistic behavior, corpus prediction, and responsiveness expose different risks, so no single score establishes production readiness.

Deployment failure to rule out	Metric or benchmark family	What the score reveals	What the score cannot prove
The model recognizes facts poorly	MMLU (Massive Multitask Language Understanding)	Broad factual and disciplinary knowledge across fifty-seven subjects, with scores interpretable against chance-level multiple-choice performance	Whether the model can generate grounded open-ended answers rather than choose among multiple-choice options

[32] **MMLU (massive multitask language understanding)**: Introduced by Hendrycks et al. (2020) with 15,908 multiple-choice questions across fifty-seven subjects. MMLU's benchmarking limitation is its format: multiple-choice recognition is not the same task as open-ended generation, so an MMLU score should not be read as direct evidence that a model can produce grounded free-form answers in production.

[33] **HELM (holistic evaluation of language models)**: Stanford's 2022 evaluation framework tested a broad set of models across seven dimensions (accuracy, calibration, robustness, fairness, bias, toxicity, efficiency) (Liang et al. 2022). HELM's contribution is methodological: by evaluating models that score similarly on accuracy but diverge on calibration or toxicity, it demonstrates that single-metric leaderboards systematically hide failure modes that matter for production deployment.

[34] **Perplexity**: From Latin *perplexus* (entangled); in information theory, $2^{H(p)}$ where H is entropy. A perplexity of 10 means the model is "10-way confused" on average. The systems consequence is interpretive rather than direct memory accounting: perplexity measures held-out next-token prediction on a corpus, while serving memory pressure is governed by context length, batch size, model shape, and decoding state; KV-cache management is a separate serving problem (Kwon et al. 2023).

Deployment failure to rule out	Metric or benchmark family	What the score reveals	What the score cannot prove
The model is capable but unsafe	HELM (Holistic Evaluation of Language Models)	Accuracy alongside calibration, robustness, fairness, bias, toxicity, and efficiency	Whether one aggregate score captures the deployment risk; a model can be strong on accuracy and weak on calibration, safety, cost, or prompt stability
The model predicts its corpus well	Perplexity	Held-out next-token prediction on the same corpus; a perplexity of 10 means the model is "10-way confused" on average	Whether generated answers are helpful, safe, or grounded outside that corpus
The model feels slow in use	First-token latency, inter-token latency, and token throughput	Prompt-processing delay before generation starts and decode speed after generation begins	Whether a single throughput number hides poor interactive responsiveness, especially when batching improves throughput but worsens first-token latency

A final failure mode is that a benchmark score may reflect training-data memorization rather than general capability. Benchmark contamination is especially difficult to audit in LLMs because web-scale pretraining corpora may contain benchmark questions, inflating scores through memorization rather than skill (Xu et al. 2024). Leakage detection reframes this risk as something benchmark designers can test for rather than merely suspect. Temporal holdouts use content published after the training cutoff, dynamic benchmarks generate fresh instances continuously, and contamination tests check whether the model recalls verbatim benchmark phrasing. These techniques keep the benchmark aligned with the deployment question instead of rewarding pretraining exposure to the evaluation set.

12.9.2 Data benchmarking

Model benchmarks validate whether compression preserved model quality. Model quality, however, depends on the data alongside the architecture and training procedure, and that dependency creates a failure mode that neither system nor model benchmarks can see. A perfectly preserved model trained on biased or unrepresentative data can still fail in production. Data benchmarks validate whether the efficiency strategies from chapter 9 (active learning, curriculum design, data augmentation, and synthetic data generation) produced training sets that enable reliable deployment. This is often the last validation to fail and the hardest to diagnose: a model achieving excellent accuracy on held-out test data may collapse on production inputs that the training data never adequately represented.

Within the D·A·M taxonomy, optimizing the Algorithm and the Machine cannot compensate for structural deficiencies in the Data. An accelerator cluster operating at peak arithmetic intensity only computes incorrect representations faster when training distributions are defective. A data benchmark therefore begins with an operational protocol rather than an aggregate score. The protocol defines the deployment slice the model must serve, reserves a leakage-resistant holdout, verifies duplicate and near-duplicate separation across partitions, sets minimum coverage for rare classes and subgroups, audits label quality, and establishes drift thresholds that determine when the benchmark no longer represents production. Only after those gates are explicit do aggregate metrics become interpretable.

Coverage metrics

The first question data benchmarking must answer is whether the training distribution covers the operational domain the model will encounter in production. Generalization to poorly represented input regimes is uncertain and requires direct evaluation; the mechanisms through which training coverage fails are often subtle.

Class balance is the most apparent failure mode. In a camera-trap dataset where 99 percent of frames are empty or show common species and 1 percent show the rare species the study exists to count, a degenerate model that predicts the majority class unconditionally achieves 99 percent accuracy while failing its operational purpose. Mitigating severe class imbalance requires loss reweighting, targeted oversampling, or decision threshold calibration. More insidious is subgroup

imbalance within classes: a dataset may have balanced positive and negative classes in the aggregate, yet draw negative examples disproportionately from a specific demographic cohort or geographic region. Aggregate accuracy metrics mask these systematic localized failures. When systems interact with human populations, auditing representation across demographic dimensions (such as age, geography, or dialect) is essential, though missing or noisy demographic metadata frequently complicates detection.

Feature coverage extends beyond discrete class labels into the continuous space of operational variations. While class balance is verified directly from label statistics, feature coverage requires mapping the physical parameters of the deployment environment. A vision pipeline trained exclusively on daytime captures fails under low-light sensor noise; a voice processing pipeline trained on clean studio signals degrades under acoustic reverberation and ambient noise. These environmental variations (differing sensor hardware, lighting conditions, and rare operational edge cases) fall outside discrete label distributions. Benchmarking feature coverage therefore requires specifying concrete bounds on the input space alongside domain experts before evaluating model performance.

Quality metrics

Even when a training dataset spans the operational domain, annotation errors corrupt the supervisory signal. Label errors, which average about 3 percent across ten canonical test sets and approach 6 percent on ImageNet's validation set (Northcutt et al. 2021), do not act as benign noise. Supervised optimization treats incorrect labels as ground truth. When an evaluation set contains corrupted labels, a model that learns the true underlying distribution is penalized, while a model that memorizes the annotation artifacts receives an inflated score.

For small corpora, manual audits over stratified random samples provide baseline error estimates. At scale, confident learning identifies label errors algorithmically by estimating the joint distribution of noisy annotations and true latent labels using out-of-sample predicted probabilities. When a model consistently assigns high confidence to a class that conflicts with the assigned label, the discrepancy isolates candidate annotation errors from typical optimization uncertainty. Flagging these candidates enables targeted relabeling or filtering, circumventing the prohibitive cost of exhaustive human review.

Inter-annotator agreement measures consistency across annotators prior to model training. The statistics are the ones data engineering already uses to monitor label quality in production; a data benchmark applies them once, to certify the evaluation set itself. Cohen's kappa for two raters and Fleiss' kappa for multiple raters quantify agreement above chance expectation (Cohen 1960; Fleiss 1971). When agreement metrics fall below established thresholds on tasks with objective ground truth, the failure points to ambiguous labeling taxonomies, rater fatigue, or inconsistent guidelines. While Landis and Koch's qualitative kappa bands provide a historical reference (Landis and Koch 1977), production pipelines require agreement thresholds calibrated directly to application-specific risk tolerance.

↩ Cohen's kappa, its chance-agreement correction, and label quality drift are established in section 4.6.4.

The structural form of label noise determines its downstream impact. Random label noise dilutes gradient signals across classes, increasing sample complexity but rarely teaching coherent false representations. In contrast, systematic errors, such as consistently mislabeling an underrepresented subclass or coupling a label with an environmental artifact, inject structured bias into model weights. If images of wolves photographed against snow are consistently labeled as dogs, the model learns to associate snow with the target class. In the presence of systematic label error, collecting more training data under the same labeling protocol does not correct the bias; it merely reinforces the spurious association with higher parameter certainty.

Distribution alignment

Data benchmarking must ultimately evaluate whether a model generalizes from training conditions to deployment environments. Standard evaluation protocols assume that training, validation, and test splits are independent and identically distributed (i.i.d.). In production systems, this assumption fails routinely: temporal drift alters consumer behavior, sensor degradation shifts hardware characteristics, and geographic expansion introduces unmodeled subpopulations. When the i.i.d. assumption breaks down, held-out validation accuracy may become a poor predictor of operational performance. The direction and size of the change depend on the shifted population. A

finite evaluation certifies behavior only on the population it sampled, and only within the uncertainty its sample size allows; that limit is the verification gap (principle 9).

This train-to-production alignment gap is quantified systematically by the WILDS[35] benchmark (Koh et al. 2021), which evaluates models across distribution shifts spanning hospital systems, camera trap locations, and satellite passes. On Camelyon17-WILDS, the reported ERM baseline achieved 93.2 percent in-distribution average accuracy and 70.3 percent out-of-distribution average accuracy across unseen clinical sites. This performance drop occurs despite holding the architecture, hyperparameters, and training compute constant, demonstrating a distribution-dependent performance gap for that fixed model and training procedure. The comparison does not isolate which capacity, representation, or data changes would close it.

The same benchmark's iWildCam split is the camera trap's version of the problem. Models trained on images from one set of camera-trap locations are evaluated on traps they have never seen, where background, lighting, and the mix of species all change (Koh et al. 2021). A MobileNetV2 that meets every latency and energy gate on the board can still fail this test, and no system benchmark would reveal it.

To detect changes that warrant investigation, serving pipelines can apply the statistical shift tests and the covariate, label, and concept shift taxonomy of section 4.12.1 to incoming inference streams; the benchmarking use is to decide when the evaluation set has stopped representing production. Two-sample hypothesis tests such as the Kolmogorov-Smirnov test (Berger and Zhou 2014) for univariate features and maximum mean discrepancy (Gretton et al. 2012) for multivariate embeddings detect changes in the marginal input distribution $p(x)$. Establishing covariate shift also requires the conditional target distribution $p(y \mid x)$ to remain stable; input-only tests cannot verify that condition without label evidence. Prediction entropy and confidence distributions provide additional warning signals, but shifted inputs can still produce confident predictions. Validate alert thresholds and fallback policies against labeled deployment samples; these signals alone do not guarantee detection before quality degrades.

↪ Live covariate shift detectors feed the statistical distance monitoring and automated drift alerts developed in section 22.

Distribution alignment challenges highlight a persistent tension in ML development between two paradigms: *fixing the data and iterating on models*, or *fixing the model and iterating on data*. The side-by-side schematics in figure 12.12 isolate this divergence through the placement of the padlock and feedback path. In the model-centric panel, the dataset is locked while the feedback loop iteratively refines model parameters on the CPU; in the data-centric panel, the model architecture is locked while the loop routes back to enhance training data quality, diversity, and labeling. As the bidirectional red arrow emphasizes, the two paradigms are complementary engineering strategies rather than mutually exclusive philosophies.

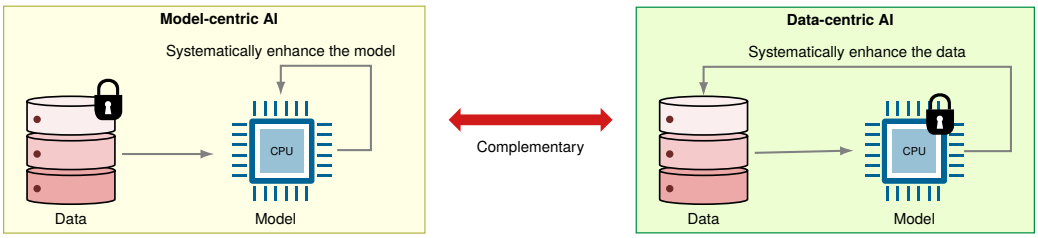


Figure 12.12: Development Paradigms: Model-centric AI iterates on architecture while holding the dataset fixed, whereas data-centric AI iterates on annotations, diversity, and bias while holding the architecture fixed. The double arrow marks the approaches as complementary.

Data-centric AI reflects an important shift in understanding that challenges the "more data is always better" assumption: dataset composition matters alongside scale. Initiatives like DataPerf (Mazumder et al. 2023) and DataComp[36] have emerged to evaluate how dataset construction affects model performance systematically. In DataComp's compute-controlled setting, a baseline that retained the top 30 percent of the candidate pool by CLIP-based filtering outperformed the unfiltered-pool baseline on aggregate downstream evaluation (Gadre et al. 2023). The result establishes the value of curation for that protocol, not a universal optimal fraction.

[35] **WILDS**: Stanford's 2021 benchmark of ten datasets with real-world distribution shifts: hospital changes (Camelyon17), wildlife camera location shifts (iWildCam), and satellite imagery temporal drift (PovertyMap). For the reported Camelyon17-WILDS ERM baseline, average accuracy fell from 93.2 percent in-distribution to 70.3 percent out-of-distribution, demonstrating that standard held-out evaluation can overestimate performance when the i.i.d. assumption fails.

[36] **DataComp**: Introduced in 2023, DataComp fixes the model family, training code, and compute budget while participants vary dataset construction. Its filtering track isolates the effect of selecting examples from a common candidate pool, making curation strategies comparable without attributing every gain to additional training compute.

Dataset saturation and dynamic benchmarks

Even when coverage, quality, and distribution alignment protocols are satisfied, static benchmarks encounter the problem of dataset saturation. As models approach empirical upper bounds on fixed test suites, leaderboard deltas compress, making it difficult to distinguish genuine capability advances from over-optimization to benchmark-specific artifacts. The capability trajectories in figure 12.13 plot this exhaustion over two decades, showing AI scores rising from normalized baselines (-100) to cross the zero line marking human performance (Maslej et al. 2024). While handwriting and speech recognition required over a decade to reach human parity (the green shaded region), reading comprehension and language understanding surged across the threshold in under two years between 2016 and 2020, rapidly exhausting the discriminating headroom of static suites.

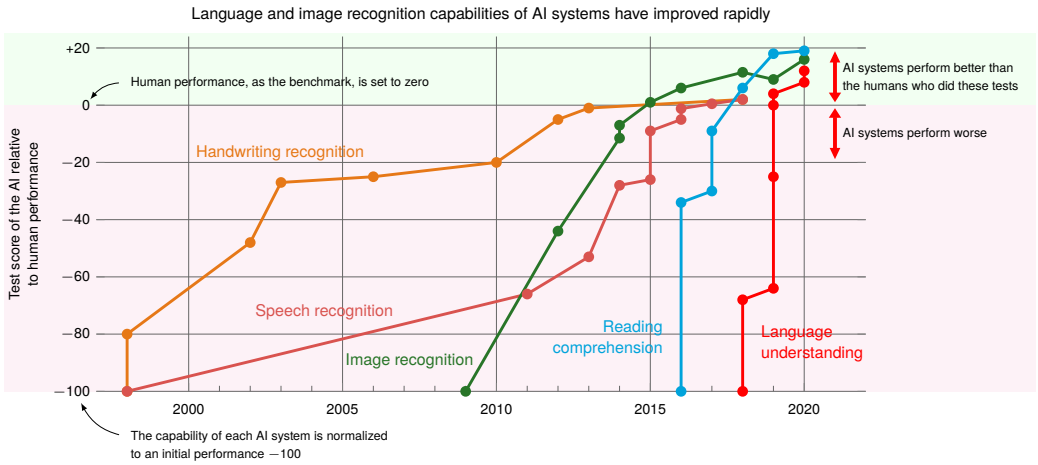


Figure 12.13: Dataset Saturation: The zero line denotes reported human performance; each curve's crossing marks a capability for which the static benchmark has less remaining discriminating headroom. The crossings do not distinguish genuine capability gains from optimization to the evaluation set. Sources: (Maslej et al. 2024; Kiela et al. 2021).

Once a static benchmark saturates, high leaderboard accuracy can mask brittle heuristics. MNIST digit recognition (LeCun et al. 1998) demonstrated early that models can exploit dataset-specific capture artifacts rather than invariant features, a concern that eventually prompted researchers to re-examine whether ImageNet had reached the end of its useful diagnostic lifetime (Beyer et al. 2020). When the evaluation distribution remains frozen, iterative model selection risks overfitting to idiosyncratic test artifacts rather than improving generalizable task capability.

Dynamic benchmarking frameworks such as Dynabench[37] (Kiela et al. 2021) address saturation by continuously evolving evaluation data. In this paradigm, human annotators generate adversarial inputs targeting the failure modes of current top-performing models, ensuring that the benchmark continues to exercise representational limits. However, dynamic benchmarks introduce a direct systems trade-off: because the evaluation distribution changes across iterations, longitudinal comparisons across model generations become difficult to calibrate. Static and dynamic benchmarks therefore serve complementary roles: static suites ensure longitudinal reproducibility on standardized tasks, while dynamic benchmarks identify newly emergent error boundaries.

37 **Dynabench**: Facebook AI Research's 2021 platform for dynamic benchmark generation, where humans craft adversarial inputs that fool current best models. Dynabench addresses the saturation problem, where very high accuracy on static benchmarks may reflect test-set familiarity rather than robust capability, but introduces its own trade-off: dynamic benchmarks are harder to compare across time because the evaluation set changes. Static and dynamic benchmarks serve complementary diagnostic roles.

12.9.3 Holistic system-model-data evaluation

Passing system, model, and data benchmarks independently is not enough. A system benchmark can validate hardware performance, a model benchmark can verify that compression preserved quality, and a data benchmark can assess training set representativeness, yet the deployed system can still fail because the three dimensions interact. Real-world AI performance emerges from that interaction, and optimizing one dimension can expose weaknesses in another.

Consider a concrete failure cascade, a composite drawn from the chapter's camera trap: an engineering team achieves target throughput on MLPerf Inference by deploying an INT8-quantized

vision model on a specialized accelerator. System benchmarks pass. However, because the quantization scale factors and zero points were fitted (quantization-range calibration) strictly against ImageNet validation samples, deploying the model in the camera trap's night-time infrared frames reveals severe accuracy degradation. Infrared frames produce activation ranges that exceed the calibrated clipping thresholds, truncating feature activations. A targeted model quality benchmark across varied exposure levels would have exposed this numerical sensitivity. Tracing the failure upstream reveals that the training pipeline contained no night-time infrared imagery, a data distribution gap that no amount of accelerator optimization or runtime kernel tuning can resolve.

Cross-dimensional coupling means that a passing benchmark in one domain can be invalidated by an unmeasured failure in another. When system benchmarks succeed alongside a model failure, the hardware delivers its promised throughput while aggressive compression degrades accuracy below deployment thresholds; when they succeed alongside a data failure, fast inference executes on benchmark inputs while training data bias causes severe accuracy disparities across demographic subgroups. Conversely, a model success paired with a system failure produces accurate predictions whose tail latency variance under concurrent load violates service-level agreements, whereas a model success paired with a data failure yields high accuracy on held-out test sets that silently degrades under production distribution shift.

These failure modes map onto the pairwise intersections of the D·A·M taxonomy (figure 1.4), though benchmark evaluation lenses do not align one-to-one with isolated physical vertices. Data benchmarks evaluate input distribution quality and coverage (D); model benchmarks assess learned algorithmic representations and numerical robustness (A); and system benchmarks expose hardware execution, kernel efficiency, and memory-interconnect traffic (M). Holistic evaluation validates the pairwise and central intersections where these domains meet, ensuring that optimizations across the Part III pipeline (data selection → model compression → hardware acceleration) do not introduce hidden cross-axis regressions.

↩ The D·A·M taxonomy and its pairwise intersections are established in section 1.5.

The D·A·M taxonomy provides a diagnostic framework for systematically identifying which axis limits performance. Table 12.20 formalizes this approach by crossing each D·A·M axis with compute, memory, and I/O constraints.

Table 12.20: D·A·M Bottleneck Diagnostic Matrix: Each cell suggests a constraint to investigate. Capacity limits determine whether the working set fits; bandwidth limits determine how quickly resident work can move. Activity counters alone do not identify the binding resource, and traces or controlled interventions must confirm a diagnosis.

Component	Compute Constraint	Memory Constraint	I/O Constraint
Data	Host preprocessing saturates compute	Capacity: working set exceeds RAM	Storage delivery cannot keep the accelerator fed
Algorithm	Required operations exceed the compute-time budget	Capacity: activations restrict feasible batch size	Exposed gradient synchronization limits step time
Machine	Useful operation rate approaches the compute roof	Bandwidth: traffic approaches the interface limit	Exposed transfers approach the interconnect limit

Applying this matrix organizes hypotheses when system telemetry diverges from theoretical hardware peaks. If an accelerator achieves only 30 percent GPU utilization during training, an engineer might prematurely blame the model architecture (Algorithm row) and attempt model surgery. Profiling often reveals instead that host-side data workers saturating CPU cores on image augmentation or tokenization cannot feed batches fast enough across PCIe (Data row, Compute Constraint column). The accelerator stalls not from algorithmic inefficiency, but from input pipeline starvation. The matrix guides investigation; traces and controlled interventions establish which physical subsystem is binding.

↪ A production troubleshooting matrix that maps common failure symptoms to a candidate D·A·M axis and the measurement that confirms it is collected for reference in section A.5.2.

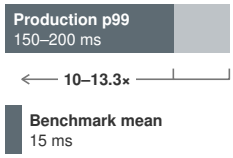
Validation under controlled laboratory conditions differs from validation in production. A laboratory protocol may fix data, simplify arrivals, or isolate execution to answer a controlled question. In production, all three assumptions can break simultaneously: data drifts, traffic spikes unpredictably, and concurrent workloads compete for shared hardware. The recurring misconceptions that follow are the ways teams forget that difference.

12.10 Fallacies and Pitfalls

Benchmarking creates false confidence when standardized measurement obscures deployment realities. Teams assume controlled evaluations predict production performance, but real systems face variability, resource constraints, and multi-objective trade-offs that benchmarks cannot capture, wasting engineering effort on systems optimized for evaluation rather than deployment.

Fallacy: *Benchmark performance directly translates to real-world application performance.*

The apparent clarity of isolated benchmark numbers leads teams to assume that measuring a model's forward pass predicts user-facing response times. It does not. In research benchmarks, a model's raw inference pass might execute in 5–10 ms. However, inside a production serving microservice, request deserialization, tokenization, feature retrieval, and network transmission overhead expand the total response time to 50–100 ms. In an illustrative model-only test, the team reports mean latency under synthetic, evenly spaced inputs (15 ms). Under the scenario's production traffic, arrival bursts and queueing backlog raise p99 tail latency to 150–200 ms (10–13.3× gap from the model-only mean). This contrast changes the statistic, measurement boundary, and traffic conditions; quantifying a slowdown requires matched end-to-end statistics under stated workload conditions. In the same illustrative comparison, a model achieving 92 percent benchmark accuracy drops to 78–82 percent when confronted with real-world distribution shifts such as noisy text and uncurated sensor inputs. Evaluating an isolated neural network kernel cannot substitute for benchmarking the complete end-to-end serving pipeline.



In this scenario, the model-only mean understates the production tail by an order of magnitude.

Pitfall: *Optimizing exclusively for benchmark metrics without considering broader system requirements.*

Benchmark leaderboards incentivize aggressive optimization, but the optimizations that climb rankings often degrade the very characteristics production demands. This exemplifies Goodhart's Law (section 12.1), whose organizational form section 12.8.4 examined. In one illustrative scenario, a team reduces inference latency from 12 ms to 8 ms through aggressive quantization, improving MLPerf ranking by 15 positions while degrading calibration such that prediction confidence scores become unreliable for downstream decision-making. Another team improves ImageNet accuracy by 2.1 percentage points through extensive hyperparameter tuning, yet the optimized model consumes 40 percent more energy and suffers 25 percent worse performance on out-of-distribution images from production cameras.

Fallacy: *Single-metric evaluation provides sufficient insight into system performance.*

A single-number claim appears deceptively simple ("94 percent accurate" or "1,200 QPS fast"), yet production success requires balancing multiple competing objectives that any single metric obscures. Modern inference systems demand evaluation across accuracy, latency, throughput, energy, and robustness dimensions (section 12.6.2). In an illustrative trade-off, a recommendation model achieving 94 percent accuracy with 180 ms p99 latency fails service-level objectives requiring p99 < 100 ms despite excellent accuracy. Conversely, a system optimized for 1,200 QPS throughput achieves this rate while consuming 4.2 W vs. 1.8 W for a slightly slower system at 1,000 QPS (2.3× power difference). For battery-powered edge devices, the 17 percent throughput loss enables 2.3× longer operation time. Different stakeholders prioritize different metrics: ML engineers focus on accuracy, infrastructure teams on throughput and cost, product managers on latency percentiles.

Pitfall: *Using outdated benchmarks that no longer reflect deployment challenges and requirements.*

Benchmarks have inertia: teams continue reporting on established benchmarks after the results cease to provide useful discrimination. Saturation occurs when multiple approaches achieve near-identical performance, eliminating useful comparison. ImageNet top-5 classification error decreased from 28.2 percent in 2010 (Russakovsky et al. 2015) to 3.57 percent by 2015 (He et al. 2016a), sharply compressing the headroom measured by that metric. As gaps narrow, the statistical confidence intervals discussed in section 12.8.1 and the result's deployment relevance matter more than leaderboard rank. Changing deployment contexts compound the problem: benchmarks designed for server hardware become misleading for edge devices with 10× less memory and 100× lower power budgets. Effective benchmarking requires retiring saturated benchmarks and developing evaluation frameworks matching target deployment realities.

Fallacy: *Research benchmarks predict production behavior under real traffic.*

A short, isolated laboratory run can miss the thermal and architectural constraints of sustained production execution. First, laboratory runs frequently suffer from initialization errors: omitting warmup runs conflates one-time JIT compilation, GPU runtime initialization, and memory allocator page faults with steady-state execution. Second, short laboratory bursts (e.g., 30 seconds) capture transient boost clock frequencies. Under sustained production load, heat accumulates faster than cooling infrastructure can dissipate it; the hardware governor triggers dynamic voltage and frequency scaling (DVFS), downclocking silicon to protect junction temperatures. As a result, a system achieving 800 QPS throughput in short burst benchmarks sustains only 400–500 QPS under continuous load with 90 percent utilization (37.5–50 percent degradation).

12.11 Summary

Benchmarking completes Part III's optimization pipeline by validating whether the efficiency gains from data selection, model compression, and hardware acceleration deliver in practice. The three benchmark lenses examine system execution, model behavior, and data representativeness separately, then test whether their assumptions survive when the complete system runs under deployment conditions. Every row of table 12.2 began as a claim and now has a measurement: ResNet-50's INT8 claim became a test with stated conditions, its roofline ceiling met an MLPerf throughput that sits 8.0× below it, MobileNetV2's accelerator speedup narrowed once the whole frame path on the edge board was timed, and keyword spotting showed that idle power, not inference, sets a duty-cycled device's battery life.

The lenses reveal different failure modes: system benchmarks expose underdelivered throughput, tail latency, and thermal behavior; model benchmarks test accuracy, calibration, robustness, and other properties that an optimization may alter; and data benchmarks examine coverage, label quality, leakage, and alignment with deployment. Standardized suites such as MLPerf Training and Inference provide comparable system evidence, whereas model and data protocols supply the quality and representativeness evidence that a hardware result cannot establish.

Optimization claims need measured execution rather than analytical proxies: wall-clock timing captures DRAM bandwidth saturation, kernel launch serialization, and host-device transfer overhead that FLOP counts omit; percentile profiling under concurrent load exposes queuing delays and thermal throttling that arithmetic means conceal; and evaluating workloads against production-representative data reveals dynamic shape variations and memory churn that uniform synthetic inputs never trigger.

A candidate release can pass the hardware, model, and data tests yet still lack evidence for the workload it will serve. A service with bursty requests needs load validation, and a camera trap deployed under changing illumination needs labeled examples from those conditions. Laboratory benchmarks establish behavior under specified conditions; a release decision must identify which of those conditions remain valid in the target deployment and test the ones that change. Timing and memory telemetry can expose overload, while silent predictive degradation requires labels or validated quality proxies, and when labels arrive late the release protocol must name that delay. A research benchmark remains useful for comparing algorithms, but a release gate for a serving system must also include load tests, tail-latency measurements, data-quality checks, failure drills, and rollback criteria. Table 12.21 pairs each simplifying benchmark assumption with the production condition that violates it and the validation step that closes the gap.

Table 12.21: Predeployment Benchmark Checklist: Each row pairs a simplifying benchmark assumption with a production condition that can violate it and the validation step that closes the gap. Each row should be independently verified against representative operational conditions.

Benchmark Assumption	Production Reality	Validation Approach
Uniform request arrival	Bursty traffic patterns	Load test with production trace replay
Clean, preprocessed inputs	Variable quality inputs	Evaluate on production data sample
Warm system state	Cold starts, cache misses	Measure cold-start performance
Isolated execution	Resource contention	Benchmark under realistic system load

Benchmark Assumption	Production Reality	Validation Approach
Fixed model version	A/B testing, gradual rollout	Establish baseline for comparison

Once a model is live, benchmarking becomes a rolling comparison against those baselines: whether the input distribution remains close to the benchmark distribution, whether latency and throughput stay inside the measured envelope, and whether model quality moves outside its expected range. Serving systems turn this measurement loop into staged rollouts, shadow evaluation (running the new model beside production without serving its outputs), and automated rollback. Benchmarking defines the performance envelope and failure thresholds; production monitoring tests live telemetry against them.

The chapter's measurement formulas fit on one page (table 12.22). Together they let an engineer state, before running a benchmark, what it will measure and what result would refute the claim.

↪ Shadow, canary, and blue-green releases and the rollback tiers that act on these thresholds are developed in section 14.4.3.

Table 12.22: Key Equations: The chapter's measurement formulas in the notation of the sections that derive them. Each states what a benchmark must measure and over which window, so that a claim can be refuted by a measurement rather than argued.

Quantity	Formula	Where derived
Time to accuracy	$T_{\text{train}} = \inf\{t \geq 0 : \text{Accuracy}_{\text{val}}(t) \geq \text{target accuracy}\}$	equation 12.1
Training throughput	$\text{Throughput} = \frac{N_{\text{samples}}}{T_{\text{train}}}$	equation 12.2
Scaling efficiency	$\text{Eff}_{\text{scaling}}(N_{\text{GPU}}) = \frac{T(1)}{N_{\text{GPU}} \times T(N_{\text{GPU}})} \times 100\%$	equation 12.3
Hardware utilization	$\eta_{\text{hw}} = R_{\text{sustained}}/R_{\text{peak}}$	section 12.4.3
Amdahl ceiling	$1/(1-f_{\text{parallel}})$ for an accelerated fraction f_{parallel} of latency	section 12.3.4
Energy per inference	$E_{\text{inf}} = P_{\text{active}} \times L_{\text{lat,infer}}$	section 12.7
Duty-cycle average power	$P_{\text{avg}} = \alpha_{\text{duty}} P_{\text{active}} + (1-\alpha_{\text{duty}}) P_{\text{idle}}$	section 12.7.2
Energy over a window	$E_{\text{total}} = \int_{t_{\text{start}}}^{t_{\text{end}}} P_{\text{device}}(t)\, dt$	section 12.7.4
Paired-test statistic	$\chi^2 = \frac{(n_{\text{base}} - n_{\text{comp}})^2}{n_{\text{base}} + n_{\text{comp}}}$	section 12.8.1
Expected calibration error	Bin-weighted mean of the gap between confidence and accuracy	equation 12.4

Key Takeaways: Measuring what matters

- **Benchmarks validate co-design**: System, model, and data benchmarks expose hardware underdelivery, compression quality loss, and distribution mismatch. A system that passes only one axis can still fail when Data, Algorithm, and Machine constraints meet under production load.
- **A claim becomes a test only when its conditions are fixed**: Benchmark carpentry restates a claim such as "ResNet-50 is faster at INT8" as a task, dataset, model, metric, harness, specification, and run rules. A number reported without those conditions cannot be reproduced, compared, or refuted.
- **A prediction and a measurement together locate a bottleneck**: The roofline places ResNet-50 training on the compute roof, yet the measured MLPerf throughput sustains only 12.4 percent of peak. The gap says time is going somewhere other than arithmetic, and profiling attributes it.
- **Proxy numbers need boundaries**: Standardized run rules make comparisons honest, but fixed workloads are still proxies. Batch size, thermal state, input distribution, concurrency, and service-deadline windows decide whether a lab result survives the benchmark–production gap.
- **Granularity trades diagnosis for realism**: Micro-benchmarks isolate kernels, macro-benchmarks expose model-level costs, and end-to-end benchmarks capture user-visible

behavior. Effective measurement stacks all three so teams can see both the symptom and the layer that caused it.

- **Tail latency often decides pass or fail**: Interactive services with percentile-based SLOs can violate their latency target while mean latency remains acceptable. Reporting percentile latency under representative load prevents a benchmark from approving a system whose mean passes while its worst-served requests violate the SLO.
- **A component speedup is diluted across the pipeline stages it does not touch**: In the chapter's vision pipeline, unchanged preprocessing at 44.4 percent of latency turns a 5× inference speedup into 1.8× end to end, and Amdahl's law (equation 11.1) caps even an infinitely fast model at 2.25×. Benchmark the whole request path before celebrating kernel speedups.
- **Energy follows the duty cycle**: For always-on keyword spotting on a microcontroller board, the idle floor accounts for 94 percent of average power, so energy per inference alone misdescribes battery life. A power benchmark states its boundary, its window, and the duty cycle.
- **A difference is real only beyond its noise**: Paired tests on per-image outcomes and intervals over repeated run pairs decide whether a one-point accuracy gap or a measured latency reduction is a finding. A test set is a measurement instrument with finite resolution.
- **Efficiency still needs quality evidence**: INT8 may cut raw weight storage 4×; the simplified component model estimates an energy reduction of about 6.6×, but whole-device measurement, confidence calibration, subgroup robustness, and edge-case behavior decide whether the compressed model is deployable.

Each chapter in this part promised fewer FLOPs, a smaller model, or higher throughput. Benchmarking tests those promises against the complete system. The gap between a claimed improvement and a measured one reveals where Data, Algorithm, and Machine were assembled rather than matched. Applied to a measured pipeline, Amdahl's law shows why an infinitely fast model can still leave the pipeline bounded by everything outside it, while tail latency shows why an average can pass even as the worst-served requests fail.



What's Next: From lab to live

The serving protocols in this chapter specified the arrival process through a load generator. Production supplies its own changing traffic, and the serving system must respond within its declared operating envelope. Part IV leaves the controlled benchmark environment, beginning with chapter 13. Engineering for arrivals no one schedules reorders the priorities that training optimized, putting per-request latency and spare headroom ahead of raw throughput.

IV

DEPLOYMENT AND OPERATIONS

Part IV

Deployment Principles

The code can be correct and the benchmarks excellent, and yet the system may fail. Part III ended with benchmarks whose load generator decides when each request arrives and whose test data stays fixed. Part IV moves from that controlled environment to production, where ML systems face the additional threat of silent decay. A machine learning system can continue to produce outputs that are confident, well-formatted, and wrong as the world drifts away from its training distribution. At deployment, the data environment escapes the engineer's control and can stress the trained algorithm and the serving machine in ways no test set anticipated. Reliability is therefore a continuous control loop of D·A·M co-design rather than a one-time release gate. The principles here define the requirements and diagnostic models for that reliability.



Principle 9: The verification gap

Policy: Statistical verification holds only over a defined deployment population p_{deploy}, task distance d_{task}, tolerance τ_{tol}, and target error ϵ, estimated by a finite-sample procedure at a stated confidence level:

$$\Pr_{(x,y)\sim p_{\text{deploy}}}[d_{\text{task}}(f(x),y) \leq \tau_{\text{tol}}] \geq 1-\epsilon$$

Implication: Deployment is not a one-way transfer; it is a control loop. Because no test suite can cover every possible real-world input, production systems must monitor their own uncertainty and fail gracefully when they drift outside their known performance envelope.

The verification gap means finite evaluation can only estimate behavior on a defined population with uncertainty. Those bounds may erode as production data diverges from the data used to set them.



Principle 10: The statistical drift diagnostic

Fitted model: Divergence between the production and baseline distributions does not by itself determine whether accuracy changes; over a measured deployment range, labeled outcomes can fit a local linear relationship:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

where P_t is the data distribution at time t after deployment, P_0 is the training (baseline) distribution, Accuracy_0 is the model's performance on P_0, $\mathcal{D}(P_t \| P_0)$ is the statistical divergence between the current and baseline distributions, and λ is fitted from labeled outcomes rather than inferred from divergence alone. Consider a credit scoring model trained on 2020 borrower behavior. Two years later, inflation rises, interest rates change, and lending policies shift. The system still produces scores, but the statistical relationship between inputs and outcomes may have changed; labeled outcome monitoring determines whether real accuracy declined while conventional error logs remained quiet. Unlike many traditional software failures, which are often surfaced by crashes, exceptions, or explicit service-health signals, ML systems can fail silently because the environment changes even when the code and infrastructure remain

unchanged. This first-order linearization applies only over the measured range; the relationship is model-dependent and may be nonlinear for large drift.

Implication: Observability must extend beyond system metrics (latency, errors) to combine statistical drift signals with labeled outcomes. A system can remain operational while prediction quality changes, but drift monitoring alone does not establish degradation.

External drift is not the only threat. Even when the world holds still, the serving pipeline itself can diverge from the model validated offline.

Principle 11: The training-serving skew diagnostic

Heuristic: Training and serving code paths can diverge on identical inputs, and that divergence is measurable without labels. For scalar model scores, define training-serving divergence as the expected absolute difference between the two functions over a stated comparison-input distribution:

$$S_{\text{skew}} = \mathbb{E}[|f_{\text{serve}}(x) - f_{\text{train}}(x)|]$$

The exact relationship between S_{skew} and accuracy depends on the loss function, decision boundary geometry, and production distribution. Unexplained divergence weakens the assumption that offline validation estimates production behavior and can cause silent accuracy loss, but S_{skew} is not a universal accuracy-loss equation. This divergence arises from inconsistent preprocessing logic, different library implementations, stale feature values, or environmental state changes between the two code paths.

Implication: Feature consistency is an architectural requirement, not merely a best practice. Feature stores are more than caches; they can reduce skew by centralizing feature definitions and retrieval. Teams still need validation for freshness, point-in-time correctness, preprocessing, model-runtime, and postprocessing parity. Even subtle differences (PIL vs. OpenCV resize, FP64 vs. FP32 normalization) can compound to produce silent accuracy degradation that standard monitoring may not detect.

Time is the nonnegotiable constraint beneath all these reliability concerns. A medical imaging system that detects tumors with 99 percent accuracy but takes 30 seconds per scan may miss its workflow deadline, and an autonomous vehicle perception model that classifies obstacles correctly in 200 ms instead of 50 ms may miss its braking deadline. Latency-sensitive deployed models operate under a strict deadline: if a correct prediction arrives too late, it has no operational value and is functionally equivalent to returning no prediction at all.

Principle 12: The latency budget principle

Policy: Under serial request-path accounting, end-to-end latency is the sum of network, preprocessing, inference, postprocessing, and queueing delays, and a deployment meets its deadline only at a stated completion quantile:

$$L_{\text{lat,total}} = L_{\text{lat,network}} + L_{\text{lat,pre}} + L_{\text{lat,infer}} + L_{\text{lat,post}} + L_{\text{lat,queue}}, \qquad \Pr[L_{\text{lat,total}} \leq T_{\text{deadline}}] \geq q$$

Implication: In latency-sensitive serving, the product selects the deadline and the acceptable completion quantile, such as 0.95, 0.99, or 0.999, and throughput is optimized within that constraint. Serving systems must implement tail-latency controls (for example, bounded-delay dynamic batching or hedged requests). Serving systems must be willing to sacrifice overall throughput to meet the latency deadline of the oldest request in the queue.

A system can satisfy its latency SLOs, detect measured distributional shifts, and maintain training-serving consistency while still causing systematic harm. The previous principles address silent

failures in correctness and service quality; this one addresses failures that degrade equity through silent amplification.



Principle 13: The bias feedback model

Fitted model: When outputs influence future inputs, today's disparity becomes an input to the next training cycle, so disparity evolves as a dynamical system rather than staying fixed. Let $\Delta_g(k) \geq 0$ denote the disparity magnitude for group g relative to a specified reference group after k fixed feedback intervals. With no effective intervention and an approximately constant dimensionless feedback factor $\alpha_{\text{fb}} > 0$, disparity may evolve as:

$$\Delta_g(k) \approx \Delta_g(0) \cdot \alpha_{\text{fb}}^k$$

where $\Delta_g(0)$ is the initial disparity magnitude relative to that reference group and α_{fb} is fitted per feedback interval. Consider a loan approval model that denies credit at higher rates to applicants from historically underserved communities. Denied applicants cannot build credit history, which makes future applications weaker, which increases future denial rates. The model's accuracy on its training distribution remains stable, but the population it serves has been reshaped by its own decisions. When $\alpha_{\text{fb}} > 1$ under these assumptions, the feedback loop is self-reinforcing; when $\alpha_{\text{fb}} \leq 1$, the modeled dynamics are stable or damped. Real deployments may also be nonlinear, saturating, or changed by intervention.

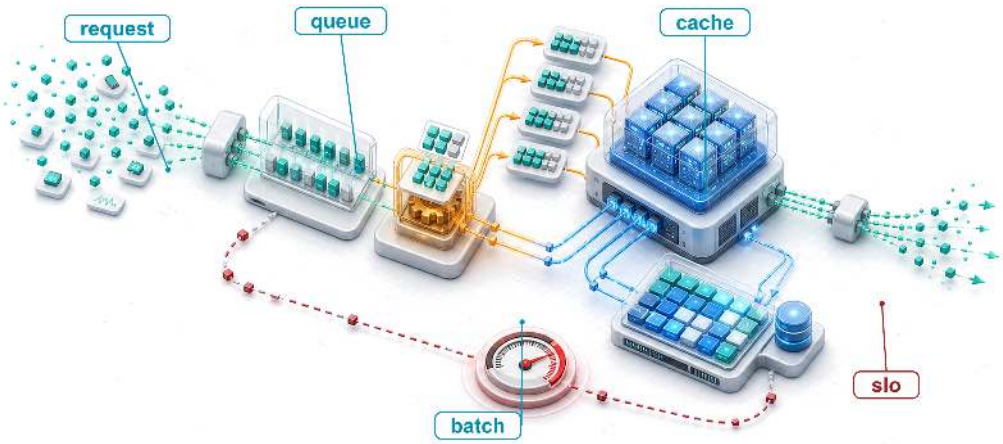
Implication: Fairness is not a postdeployment audit; it is a stability constraint on the deployment control loop. Where consequential impact warrants it and collecting or using group attributes is lawful and justified, systems must monitor disaggregated performance metrics across relevant demographic groups with the same rigor applied to latency percentiles, because a bias regression can be invisible to aggregate accuracy just as a tail-latency violation can be invisible to mean latency.

Together, the five principles define a deployment control loop that establishes what the evidence supports, observes the live system and environment, acts only when a measured failure warrants intervention, and re-evaluates the relevant service, predictive, or subgroup criterion. They separate diagnosis from response. A signal narrows the investigation but neither prescribes an automatic intervention nor proves that quality has already degraded.

Part IV makes that control loop concrete. Chapter 13 builds infrastructure that meets latency budgets under variable traffic and checks training-serving parity on the request path. Chapter 14 detects drift and skew over the model's operating lifetime while preserving the evidence needed for diagnosis and response. Chapter 15 treats fairness and feedback as measurable deployment dynamics rather than a one-time compliance check. Chapter 16 then closes the book by gathering these production controls and the quantitative bounds and models of the preceding parts into one framework for reasoning across the whole stack.

13

Model Serving

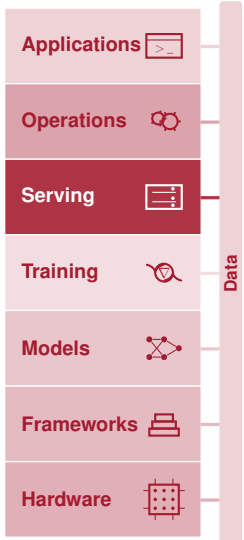


13.1 The Serving Inversion
13.2 Serving System Architecture
13.3 Request Lifecycle
13.4 Queuing Theory
13.5 Throughput Optimization
13.6 Inference Runtime Selection
13.7 Node-Level Optimization
13.8 Model Loading and Versioning
13.9 LLM Serving
13.10 Economics and Planning
13.11 Fallacies and Pitfalls
13.12 Summary

Purpose

Why does serving change the optimization priorities that made training successful?

Training and serving pursue opposing operating objectives. Training maximizes aggregate throughput across large, scheduled batches and long runs, whereas serving must answer individual, heterogeneous requests under strict latency deadlines. Training amortizes hardware initialization and weight streaming across millions of examples; serving pays its full fixed cost on every query, and small inefficiencies accumulate into machine time that must be provisioned. A system handling one billion requests a day recovers more than eleven days of machine time every day from a single millisecond saved per request. Each request also runs inside a fixed end-to-end latency budget, in which network ingress, serialization, preprocessing, queueing, and postprocessing all spend time the model cannot use. When those stages consume most of the budget, kernel optimizations barely reach the user, and a prediction that arrives after the client has timed out has no value however accurate it is. Serving is therefore a capacity problem under uncontrolled traffic as much as a latency problem. Arrival rates swing between daily peaks and troughs, so capacity provisioned for the average collapses at the peak, while capacity provisioned for the peak sits idle and costly the rest of the day. The question a serving system answers is not whether the model computes correct outputs in isolation but whether it sustains reliable, low-latency execution under production variability. In D·A·M terms, serving is where the trained algorithm meets unpredictable live data inside the machine's latency budget, and all three constraints bind on every request.



Learning Objectives

- Explain serving inversion from training throughput to per-request latency, headroom, and tail behavior
- Decompose request latency across serialization, preprocessing, inference, queuing, post-processing, and network overhead
- Apply queueing laws and simple queue models to plan capacity and admission control against percentile latency targets
- Select batching windows and inference runtimes that trade formation delay and precision for throughput within a latency budget
- Diagnose cold-start delays from weight transfer, runtime initialization, graph compilation, and warmup during model loading
- Evaluate LLM serving bottlenecks using token latency, KV-cache memory, and continuous batching constraints
- Calculate cost per inference from precision, hardware utilization, replica count, and runtime throughput

13.1 The Serving Inversion

When a user uploads a photograph and waits for its label, or a chat interface sends a prompt and waits for the first word of the reply, each request carries a strict deadline of tens to hundreds of milliseconds. Every stage between the network socket and the response draws on that single allowance: decoding the payload, waiting in a queue, moving tensors to the accelerator, running the model, and formatting the answer. Moving from offline training to live traffic triggers the **serving inversion**, a reversal of engineering priorities: training is judged by how much work the machine completes per hour, whereas serving is judged by whether each individual request returns before its deadline.

The benchmarks of chapter 12 measured models under a load generator that decided when each request arrived. Production traffic has no such scheduler. Model **serving** must sustain the operating targets that benchmarking evaluated under specified conditions while live traffic tests that operating envelope continuously, so measurement remains part of the service's lifecycle. In offline training, systems maximize aggregate throughput by packing large batches to amortize weight transfers from high-bandwidth memory (HBM) and saturate matrix arithmetic units. In online serving, requests arrive stochastically from independent clients and must return within strict service-level objectives (SLOs). The primary engineering priority therefore inverts from long-run throughput to per-request latency. The inversion restructures the balance of terms across both levels of the iron law of ML systems: shrinking a task from a large training batch to small batches or individual inputs ($B = 1$) slashes the Data factor (Samples/Task in equation 1.2) and inflates the Machine factor (Seconds/FLOP) across the unpacked terms of equation 1.3 ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). For weight-dominated operations, small batches reduce arithmetic intensity and can shift execution from compute-bound matrix multiplications toward memory-bandwidth-bound transfers (D_{vol}/BW). Simultaneously, the latency term L_{lat} expands beyond microsecond accelerator kernel launches to include network transit, request serialization, and host orchestration, often becoming the binding constraint on end-to-end response time.

↩ The multiplicative, additive, and pipelined formulations of the iron law of ML systems are established in section 1.7.

The inversion reshapes each D·A·M layer at once. In the data layer, the constraint shifts from total dataset volume to the arrival rate, payload freshness, and variable sequence dimensions of live queries, while in the algorithm layer execution becomes strictly forward-only: backpropagation state and optimizer memory vanish from the accelerator, even as dynamic input shapes introduce runtime execution variability into previously static operator graphs. In the machine layer, the operating target shifts from maximum hardware utilization to **headroom**. Whereas offline training treats an idle accelerator as wasted capital, online serving treats near-saturation as an operational hazard, because stochastic request arrivals make queue lengths and waiting times diverge nonlinearly as utilization approaches 100 percent. Maintaining adequate headroom preserves the spare capacity needed

to absorb transient traffic spikes without violating tail-latency percentiles, so serving optimizes completed requests under a strict latency deadline rather than steady-state hardware occupancy.

Meeting that latency deadline governs how engineering teams partition the end-to-end serving stack. A request's latency budget encompasses three sequential stages: host-side ingress and preprocessing (network termination, payload deserialization, and tokenization or feature transformation), accelerator kernel execution, and downstream postprocessing. Any millisecond consumed by unoptimized feature retrieval or queueing delays directly subtracts from the execution time allocated to the neural network itself. Consequently, the central engineering task in model serving is deciding which operations must remain on the synchronous request path, which tasks can be offloaded to asynchronous background pipelines, and how much headroom the system must reserve before tail-latency guarantees collapse under load. Three conditions therefore separate serving from the forward pass that a benchmark times: a per-request deadline, arrivals that no scheduler controls, and a stack in which the model is one stage among several.

Definition 13.1: Model serving

Model serving is the operational phase that provides model predictions to end-users or downstream systems under latency, throughput, availability, and cost constraints.

1. **Significance**: Online serving turns training's priority on sustained hardware efficiency (η_{hw}) into a request-latency constraint ($L_{\text{lat,total}}$), requiring a stack designed around the percentile named by its SLO.
2. **Distinction**: Unlike model training, which commonly processes scheduled batches, serving may handle stochastic online requests, synchronized streams, or offline batches.
3. **Common pitfall**: A frequent misconception is that serving is "just the forward pass." In reality, it is a systems problem: model execution is only one component of a stack that may include request routing, load balancing, and data transformation.

The SLO[1] defines the latency target that shapes every architectural decision in the serving stack, establishing the strict time budget within which all request handling must complete. When an inference server receives a request, model execution represents only one segment of an end-to-end pipeline: network ingress, payload deserialization, feature transformation, queue scheduling, and output serialization all consume time before and after the accelerator runs. The end-to-end request lifecycle partitions execution into three sequential stages (figure 13.1): input preprocessing (traditional computing), neural network evaluation (deep learning), and output postprocessing with safety validation (traditional computing). Any of these stages can become the binding latency bottleneck. Section 13.3.1 quantifies where the time goes and finds that host preprocessing can take longer than the forward pass itself.

1 **SLOs in ML serving**: An SLO is the internal target, as distinct from the contractual SLA. For ML serving, task quality and inference latency can both contribute to SLOs, creating multidimensional targets where improving one dimension (for example, deploying a larger model for accuracy) can violate another (latency).

↩ SLOs, service level agreements (SLAs), and the tail-latency percentiles they constrain are established in section 12.6.2.1.

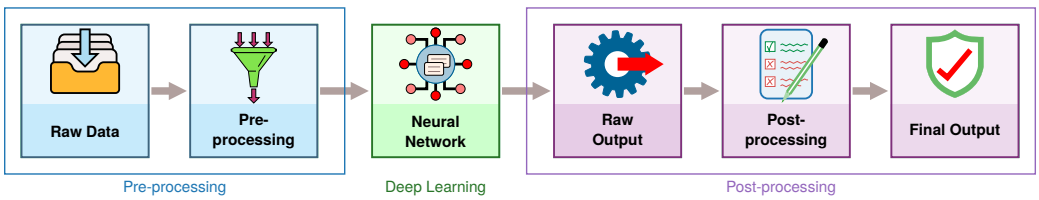


Figure 13.1: The Inference Pipeline: ML serving systems transform raw inputs into final outputs through sequential stages: preprocessing, neural network computation, and postprocessing. The neural network represents just one component; preprocessing and postprocessing make end-to-end latency depend on more than neural-network execution.

Two services carry the chapter, one per half, and each lighthouse arrives in the regime it represents. The first half follows a ResNet-50 image-classification service on one accelerator node. ResNet-50 is the book's compute-bound lighthouse. Each forward pass performs about 8.2 GFLOP of arithmetic on 102.4 MB of FP32 weights, so a large training batch keeps the matrix units busy. A served request arrives alone, however, and at batch one the same forward pass spends most of its time on weight

traffic and kernel launches rather than arithmetic. Recovering the efficiency that training enjoyed without breaking the deadline is the first half's problem, and the reference request profile introduced with the latency budget carries the service through queuing, batching, runtime selection, and replica startup. Large language model (LLM) serving then follows a GPT-2 XL chat service, the bandwidth-bound lighthouse whose decoding step section 6.7.4.2 traced. Each generated token streams all 3 GB of its FP16 weights from memory, about 0.9 ms on the book's reference H100, so memory bandwidth sets token latency and the memory each conversation holds sets concurrency. Llama 3 8B, a larger member of the same transformer family, extends that case where the per-conversation cache grows large enough to bound how many requests share the accelerator. DLRM, the capacity-bound lighthouse, appears once as a contrast in which embedding lookups rather than arithmetic dominate a request, and MobileNetV2 appears where the target device cannot hold ResNet-50 at all.

Table 13.1: Chapter Map: Each section answers one serving decision on one of the running cases and names the iron law term or resource that decision most affects. The first four rows follow a single ResNet-50 replica from its architecture to batching; the remaining rows change what the replica executes, how it starts, what it produces, and what it costs.

Section	What it decides	Running case	Iron law term or resource it targets
Serving System Architecture	How a replica turns arriving requests into accelerator work	ResNet-50 service	Fixed overhead L_{lat} (protocol, serialization, copies)
Request Lifecycle	Where one request's latency budget goes	ResNet-50 request; DLRM contrast	Every term, stage by stage
Queuing Theory	How much headroom many concurrent requests need	ResNet-50 capacity plan	Queueing delay added to L_{lat}
Throughput Optimization	How many requests share each weight read	ResNet-50 batching sweep	D_{vol}/BW and L_{lat} per request
Inference Runtime Selection	Which engine and number format execute a batch	ResNet-50 runtimes and precisions	Hardware efficiency η_{hw} and bytes D_{vol}
Node-Level Optimization	What stalls the path between network socket and accelerator	ResNet-50 node profile	Host-side L_{lat}
Model Loading and Versioning	How fast a new replica serves, and whether it computes what training computed	ResNet-50 cold start and preprocessing skew	Startup time and prediction accuracy
LLM Serving	How token streams share memory bandwidth and capacity	GPT-2 XL decode; Llama 3 8B cache	D_{vol}/BW per token; memory capacity
Economics and Planning	How many accelerators, at what cost per completed request	ResNet-50 fleet; Llama 3 8B tokens	Cost per request that meets the SLO

The architecture section (table 13.1) examines the inference server that sits between the network and the accelerator. The request lifecycle then traces one ResNet-50 request through that server and finds that the model is one slice of its latency budget. Queuing theory lets many requests arrive at once and derives the headroom that keeps their tail latency bounded, and throughput optimization raises each replica's service rate by batching, where the chapter's first half ends. The second half changes what each replica runs and how it starts: runtime and precision selection decide how fast a batch executes, node-level optimization removes the stalls between host and accelerator, and model loading decides how quickly a new replica can serve and whether it computes what training computed. LLM serving then replaces fixed outputs with token streams, and economics and planning converts the whole stack into accelerator count and cost per request. Two placement decisions precede all of these: the first is temporal, whether predictions are computed ahead of time or on demand, and the second is spatial, where the model physically runs.

13.1.1 Static vs. dynamic inference

A nightly job that precomputes product recommendations for existing users can pack thousands of items into each accelerator batch, whereas a live content-moderation API must score an uploaded image within milliseconds of arrival. Deciding whether to generate predictions before or during a user request sets the trade-off between offline hardware efficiency and online responsiveness (Google 2024b): it determines whether an accelerator runs in an unconstrained throughput regime or under the strict tail-latency deadlines of an interactive serving loop.

13.1.1.1 Static inference

Static inference (also called offline or batch inference) precomputes predictions for anticipated inputs and stores them in a retrieval store. Moving model execution entirely out of the user request path alters the underlying hardware constraints. Because batch jobs have no millisecond-level deadlines, accelerators can use large batches ($B \gg 1$) selected for throughput within memory limits. Reusing weights across samples raises arithmetic intensity for weight-dominated operations; a batch-size sweep identifies where throughput stops improving. When a client issues a request (for example, a user opening nightly generated recommendation rankings), the serving system performs a low-latency key-value lookup rather than executing a forward pass on an accelerator. The primary operational constraint shifts from accelerator latency to storage capacity and data freshness. Precomputing predictions across an entire input domain scales combinatorially ($\mathcal{O}(N_{\text{users}} \times N_{\text{items}})$ for N_{users} users and N_{items} items in recommendation systems), and stale predictions persist until the next batch refresh.

13.1.1.2 Dynamic inference

Dynamic inference (also called online or real-time inference) evaluates inputs on demand as requests arrive. On-demand execution accommodates arbitrary, previously unseen inputs within the model schema and immediately reflects newly deployed model weights without requiring precomputation. However, executing the model directly within the user request path subjects inference to the strict end-to-end latency budget set by its SLO. Because user requests arrive stochastically, the serving runtime cannot delay execution indefinitely to accumulate large batches. Serving requests individually or in small batches ($B = 1$ to $B = 4$) can expose weight traffic or launch overhead that larger batches amortize. When these costs bind execution, the device completes fewer queries per unit time, increasing provisioned cost per query. This tension between interactive response time and hardware efficiency can make dynamic serving more expensive per query than offline batching (example 13.1).

> Example 13.1: The cost of latency
>
> Latency constraints directly dictate infrastructure costs. The figures below come from the ResNet-50 reference profile that section 13.5.1 later sweeps across batch sizes. Consider a GPU server renting for $4/hour.
>
> Scenario A (low latency): Batch size 1.
>
> - Latency: 3.5 ms.
> - Throughput: 283 req/s.
> - Cost per million queries: $3.92.
>
> Scenario B (high throughput): Batch size 8.
>
> - Latency: 3.8 ms, barely above batch one, because the fixed launch cost dominates both.
> - Throughput: 2,129 req/s.
> - Cost per million queries: $0.52.
>
> **Systems insight**: Serving one image at a time saves only the difference between 3.8 ms and 3.5 ms of latency, yet multiplies the hardware cost per million queries by 8×. Engineers must quantify whether that speedup generates enough business value to justify the 8× cost increase.

Consider two contrasting deployment scenarios for a ResNet-50 image classifier. A static batch approach suits a photo management application that preclassifies every image in a user library overnight. Classifying 10,000 photos one at a time, at 3.5 ms per batch-one forward pass on the reference H100, would take 35 s; at the 89,500 images/s that MLPerf Inference (Reddi et al. 2019) measured for a compiled INT8 engine at large batch on the same accelerator, the same library takes 0.11 s, about 320× faster, and clients experience instantaneous retrieval upon browsing. Conversely, a dynamic online approach suits a content moderation service that must evaluate user-uploaded media in real time, where each image traverses the complete preprocessing, accelerator execution, and postprocessing pipeline within a 100 ms service-level objective.

These two modes optimize for fundamentally different operating regimes. Static inference maximizes accelerator utilization and amortizes memory transfers across large, offline batches, trading persistent storage capacity for zero serving latency at query time. Dynamic inference minimizes tail latency under stochastic, concurrent traffic, accepting lower compute efficiency and higher marginal cost per query to evaluate arbitrary, unpredictable inputs. Production architectures frequently synthesize both paradigms into a tiered design, in which static precomputation populates key-value caches with predictions for high-frequency queries, such as catalog items or viral media, while dynamic serving clusters evaluate the long tail of novel or personalized requests. Deciding when predictions are computed resolves only the temporal dimension of serving, however; equally binding is the spatial dimension, where the model physically executes, because physical placement dictates memory hierarchy limits, network boundaries, and the hardware budget available to the serving stack.

13.1.2 The spectrum of serving architectures

A cloud server can pool accelerators behind an HTTP endpoint at the cost of a network round trip and multi-second cold starts, whereas a hearing aid or industrial sensor microcontroller cannot afford a network hop or an operating system at all. Although "serving" often implies a networked server processing API requests, the four deployment paradigms (Cloud, Edge, Mobile, and TinyML) and the physical constraints that give rise to them—the light barrier, the power wall, and the memory wall—shape the serving stack into very different forms. Serving spans a continuum from **centralized networked microservices** (which pool accelerators to optimize throughput and cost per query at the expense of network round-trip overhead and cold-start latency) to **application-embedded edge serving** (which run models inside client processes via direct function calls, eliminating network latency and preserving privacy at the expense of strict device memory and power limits) and out to **bare-metal firmware execution** on microcontrollers. Those hardware limits do not disappear at serving time; serving adds latency SLOs and cost pressure on top of limits that training could absorb through patience. The same model may therefore require different serving strategies depending on where it runs.

↩ The light barrier, power wall, and memory wall that give rise to the four deployment paradigms are established in section 2.2.

The three architectures differ in what binds. Networked serving runs the model as a standalone service behind HTTP or gRPC, so network transfer and serialization join model execution as candidate bottlenecks; data-center accelerators support batching and concurrency, but a new replica's cold start can stretch from seconds to minutes because container startup, model loading, and warmup sit outside the steady-state request path. Application-embedded serving runs the model inside the user's application process on a phone or embedded board, behind a function call rather than a network interface. With no shared server to batch for, optimization targets energy and responsiveness (the shape that benchmarking calls SingleStream), and a passively cooled envelope of a few watts throttles sustained inference once the chassis heats. Bare-metal serving compiles the model into microcontroller firmware, often without an operating system or a dynamic allocator, so weights execute in place from flash memory, activations fit a preallocated tensor arena of a few hundred kilobytes of SRAM, and batching is impossible on a single sensor stream.

↩ The MLPerf SingleStream, MultiStream, Server, and Offline scenarios are established in section 12.4.4.1.

Embedded execution enables **zero-copy inference**. When data moves through a system, each copy consumes CPU cycles and memory bandwidth. In cloud serving, separate physical memory spaces force repeated transfers. A camera frame passes from a network socket buffer to user space, through a preprocessing buffer into page-locked host DRAM, and across PCIe into accelerator memory. A mobile pipeline can eliminate these intermediate copies when its camera, image signal processor (ISP), CPU, and NPU exchange shared buffer descriptors in unified system memory. In unified-memory system-on-chip (SoC) architectures, such as Apple A- and M-series processors or Qualcomm Snapdragon, all compute engines share a single physical DRAM pool. When software passes buffer handles rather than duplicating bytes, inference latency drops and memory bus power decreases, although format conversions may still require CPU or ISP intervention.

The deployment tiers differ first in their operating envelopes (latency targets, memory, and power), which section 2.2 derived and tabulated for each paradigm; what serving adds is the model each tier can actually run. Applying those envelopes to one image-classification workload, table 13.2 shows why cloud and mobile can adapt ResNet-50 while TinyML must substitute MobileNetV2. The cloud column runs the model through TensorRT, NVIDIA's ahead-of-time inference compiler,[2] and the

[2] **TensorRT**: TensorRT builds its engine ahead of time for one microarchitecture (such as Hopper or Blackwell) (NVIDIA 2024c). That target-specific build phase allows the compiler to generate fused kernels, select microarchitecture-specific Tensor Core execution paths, and optimize memory layouts (such as NC/32HW32). The resulting nonportable engine reduces per-request execution latency and minimizes the accelerator count needed to satisfy a target throughput.

mobile column through the TensorFlow Lite runtime; the chapter uses these engines as given until section 13.6 examines what such runtimes do to a model.

Table 13.2: Image Classification Across the Serving Spectrum: Illustrative side-by-side comparison of cloud, mobile, and TinyML serving for the same image-classification task, showing how model format, latency, throughput, memory footprint, and energy budget differ across deployment contexts.

Dimension	Cloud	Mobile	TinyML
Model format	TensorRT FP16 engine (51.2 MB)	TensorFlow Lite INT8 (25.6 MB)	Not feasible (25.6 MB); alternative: MobileNetV2 INT8 (3.5 MB)
Inference (batch-1)	0.33 ms (batch-16: 0.83 ms)	12 ms (NPU), 45 ms (CPU)	120 ms
Throughput	30,400 img/s (batched)	~80 img/s (single-stream)	~8 img/s
Memory	2 GB VRAM (batch-32)	150 MB peak (shared with app)	320 KB arena (fits in 512 KB SRAM)
Energy/inference	—	0.8 mJ (NPU), 4.2 mJ (CPU)	12 mJ

Systems Perspective 13.1: ResNet-50 across the serving spectrum

Systems insight: It is tempting to read table 13.2 as one model deployed three ways. That reading is misleading: the cloud and mobile tiers share the ResNet-50 graph but use different precision and runtimes, while the TinyML tier cannot run ResNet-50 at all and substitutes an architecture designed for its constraints. Treating these as one model hides the work that makes each deployment possible.

Whichever tier hosts it, a networked deployment runs the model inside a server process, and each such replica must turn an irregular stream of requests into work its accelerator can execute efficiently. That conversion is the job of the inference server's internal architecture.

13.2 Serving System Architecture

Inside each replica, user requests arrive in irregular bursts—milliseconds apart one moment and silent for seconds the next—whereas accelerator compute engines demand steady streams of uniform, dense batches to saturate memory bandwidth and arithmetic units. Reconciling these two incompatible rhythms at the system boundary requires a serving architecture that decouples network ingress from execution scheduling, absorbing arrival bursts and packing irregular inputs into dense batches without violating latency SLOs.

13.2.1 Internal architecture and request flow

If a web server invoked the accelerator synchronously every time a socket read completed, slow client uploads would stall the GPU and concurrent connections would contend for device memory. An inference server[3] (such as NVIDIA Triton or TensorFlow Serving) avoids this by acting as a specialized runtime scheduler that manages concurrency, device memory, and data movement between host CPUs and accelerator devices. Rather than running the model directly on the network thread, the server pipelines incoming requests through dedicated subsystems that isolate client network handling from accelerator compute. Every request traverses a six-stage runtime pipeline (figure 13.2) from network ingress and request-buffering queues through the dynamic batcher to the inference runner and hardware accelerator.

Across the stages in figure 13.2, the server handles concurrency management, request transformation, model lifecycle management, and execution scheduling. At network ingress, the server manages concurrency through asynchronous event loops (such as `epoll` or `kqueue`) and worker thread pools that handle thousands of concurrent client TCP connections without stalling accelerator execution pipelines. Decoupling network I/O from compute ensures that slow client transfers or network stalls do not starve the accelerator of ready work.

3 **Inference server**: Google's TensorFlow Serving (Olston et al. 2017) helped establish the separation of model logic from serving infrastructure; NVIDIA's Triton (NVIDIA 2024d) extends this pattern across multiple model frameworks and backends. The critical design insight is that a scheduler and dynamic batcher turn irregular single-request traffic into accelerator-friendly execution, improving utilization when latency budgets allow batching. Exact utilization gains depend on model, hardware, arrival rate, and the configured batching window.

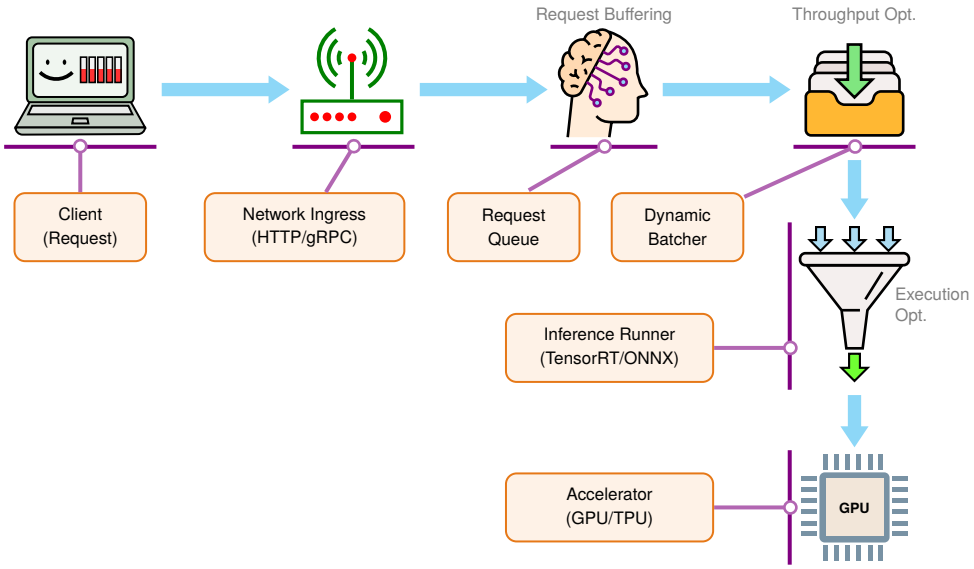


Figure 13.2: Inference Server Anatomy: A modern inference server decouples network handling from accelerator execution through a staged pipeline. Each stage isolates a concern, from absorbing bursty traffic to forming efficient batches, so the hardware accelerator stays busy despite irregular arrival patterns.

Once a payload arrives, the server transforms requests into tensors, decoding serialized network bytes into contiguous, aligned memory buffers that match the tensor layout required by the underlying execution engine. For example, four-dimensional vision tensors typically adopt NCHW[4] or NHWC formats. On modern accelerators, matrix multiplication units (such as Tensor Cores) load 8- or 16-element channel vectors using coalesced memory transactions that favor channels-last (NHWC) layout. If the execution kernel requires a different physical layout, the server must materialize that layout. Calling `torch.permute` merely creates a view with reordered strides; an explicit memory copy or transpose kernel must physically reorder the data in memory. Because such materialization performs zero arithmetic ($I \approx 0$ FLOP/byte), memory traffic and launch overhead dominate its cost. A device-local conversion consumes device memory bandwidth, whereas conversions across the host-device boundary consume PCIe bandwidth.

For those tensors to execute without startup stalls, the server also manages the model's local lifecycle. The runtime establishes the CUDA context before allocating device memory (VRAM) and streaming weights onto the accelerator, pins host memory buffers for direct memory access (DMA) transfers, and executes representative warmup passes to trigger deferred compilation and memory allocation before admitting live client traffic. While enterprise model registries, canary rollouts, and rollback policies belong to cluster-level operations (chapter 14), the serving engine governs local state, validating that active model instances are warmed, resident in accelerator memory, and ready for dispatch.

Between request buffering and the warmed model sits the execution **scheduler**, which coordinates accelerator dispatch by implementing dynamic batching (section 13.5). For weight-dominated inference at batch size $B = 1$, streaming parameters from HBM can limit execution before the arithmetic units reach peak throughput. Grouping B requests into a concurrent batch amortizes the byte transfers of reading model weights across all B inputs, increasing the arithmetic intensity ($I = O/D_{\text{vol}}$).

To balance individual request latency against batch compute efficiency, the scheduler tunes queue residency using a **batching window** (or timeout threshold). Setting the batching window to zero removes deliberate batch formation delay, but requests can still wait for an available worker. An immediately dispatched small batch may also expose too little work to use the accelerator efficiently. Conversely, setting a bounded window (such as 2 to 5 ms) allows the scheduler to buffer arriving requests until either the queue reaches a target batch size or the timeout expires. The window

[4] **NCHW and NHWC (tensor memory layouts)**: These acronyms encode the dimension order of 4D image tensors: N (batch), C (channels), H (height), and W (width). In a contiguous NCHW tensor, each channel plane is contiguous; in NHWC, channels for one spatial position are adjacent. Optimized kernels may favor either layout, and a semantic mismatch can scramble values even when the element count is correct. Serving code should therefore make both dimension meaning and any physical-layout conversion explicit.

bounds deliberate formation delay; waiting for a busy worker is a separate resource-queue delay. Batching trades that formation time for greater weight reuse and potentially higher service capacity, while total latency includes both waits and execution.

13.2.2 Interface protocols and serialization

As accelerator kernels undergo aggressive compilation (via TensorRT, TorchDynamo, or custom kernels), device execution time often drops below a single millisecond, and the host-side cost of ingesting, transporting, and deserializing payloads becomes the dominant term in end-to-end latency (the per-request overheads itemized in section 13.3.2.1). Two interface choices—the wire serialization format and the network transport protocol—determine whether that host ingress path stays in microseconds or consumes the latency budget before inference begins.

13.2.2.1 The serialization bottleneck

Machine learning payloads differ fundamentally from conventional web API payloads. They consist of multi-dimensional arrays of floating-point numbers (such as image tensors, dense embedding vectors, or token sequence logits). While text-based JSON is ubiquitous in web services, serializing dense numerical arrays into text incurs severe CPU and memory penalties. Serialization overhead manifests because JSON parsers must scan every byte sequentially, validate syntax, allocate heap nodes, and invoke string-to-float conversion routines (such as `strtod`). Each 32-bit float expands from 4 raw bytes into 7 to 15 ASCII characters including punctuation. Furthermore, raw binary data (such as compressed JPEG or raw image bytes) requires Base64 encoding to travel within JSON strings, imposing a 33 percent size inflation before parsing begins.

Binary serialization formats, such as Protocol Buffers[5] and FlatBuffers,[6] bypass this text conversion cost. Float arrays transmit directly as packed IEEE 754 byte streams with zero ASCII conversion and zero Base64 framing. Furthermore, FlatBuffers organizes serialized records with internal relative offsets, enabling zero-copy deserialization, in which the serving application reads tensor dimensions, data types, and byte offsets directly out of the network receive buffer without allocating an intermediate object graph on the heap.

13.2.2.2 REST vs. gRPC

Transport protocol selection governs connection life cycles and socket multiplexing. Conventional REST APIs typically deploy HTTP/1.1 with JSON payloads. HTTP/1.1 clients commonly use multiple persistent TCP connections for concurrency. Establishing each connection incurs handshake and, for HTTPS, TLS costs, but keep-alive reuse amortizes setup across requests. HTTP/1.1 also suffers from request-level head-of-line blocking, because responses on a single connection must return in request order. While REST APIs can adopt HTTP/2 or binary encodings, the combination of HTTP/1.1 and JSON creates significant socket overhead and CPU parsing contention when serving high-throughput tensor workloads.

In contrast, gRPC (gRPC Remote Procedure Call)[7] pairs HTTP/2 transport with Protocol Buffers serialization. HTTP/2 establishes a single long-lived TCP connection and multiplexes hundreds of concurrent request and response streams simultaneously using binary framing. This architecture multiplexes concurrent streams over a persistent connection and removes HTTP-level response-order blocking (a lost TCP packet can still stall every stream on the connection, which HTTP/3 addresses by moving to QUIC), while Protobuf enforces strongly typed contracts and serializes binary tensor payloads efficiently. Consequently, gRPC is the standard protocol for internal microservice communication in model serving clusters, where low latency, compact wire payloads, and bidirectional token streaming are primary design criteria. Even a modest embedding payload shows the gap in wire footprint and host parsing time.

> **Napkin Math 13.1: JSON vs. Protobuf serialization**
>
> Consider a request payload containing 1,000 floating point numbers (for example, an embedding vector).
>
> - **JSON**: Uses ~9 KB on the wire. Requires ~50 µs to parse.

↩ Clipper's decomposition of serving latency, in which serialization rivals model execution, is established in section 12.3.4.

[5] **Protocol Buffers (Protobuf)**: Protobuf uses a predefined schema (from a `.proto` file) to encode structured data into a compact binary format (Protocol Buffers Authors 2026). Because the schema carries field names and types, the wire payload need not repeat them as JSON does. Its wire format is still not identical to a C++ object's in-memory layout, so it requires a parsing step and does not provide the same direct zero-copy access pattern that FlatBuffers targets.

[6] **FlatBuffers**: The "flat" in the name describes the design: the binary buffer can serve as the serialized representation and the data structure being read, avoiding a separate parsing or unpacking phase for supported access patterns (FlatBuffers Authors 2026). For ML inference, this can enable zero-copy access to tensor metadata, because the serving system reads tensor shapes and offsets directly from the buffer rather than allocating a second object representation.

[7] **gRPC (gRPC remote procedure call)**: The gRPC framework pairs HTTP/2 transport with an interface definition language and message format, most commonly Protocol Buffers (gRPC Authors 2026). The size and latency benefit over REST/JSON depends on payload shape, client/server implementation, and whether serialization is a meaningful share of the end-to-end latency budget.

- **Protobuf**: Uses ~4 KB on the wire. Requires ~5 µs to parse.

Math: Switching one request to the binary payload saves 50 µs − 5 µs = 45 µs of parse time. For this illustrative system processing 10,000 requests per second, the savings compound to 10,000 × 45 µs = 0.45 s of CPU time reclaimed every wall-clock second, which is 45 percent of one core freed from serialization overhead alone.

Systems insight: This 10× scenario gain makes gRPC/Protobuf, FlatBuffers, or another binary protocol a strong candidate for high-throughput internal microservices when serialization is a visible part of the latency budget.

Architectural selection is governed by system constraints. REST over HTTP/1.1 remains practical for public-facing edge gateways where broad client compatibility and human-readable debugging take precedence over raw wire efficiency. Within cluster boundaries, gRPC over HTTP/2 with binary serialization can reduce overhead when high query volume or tensor serialization consumes a significant share of the host CPU budget; its streaming support also suits incremental token responses.

13.2.3 The load balancer layer

When traffic exceeds what a single machine can handle, adding model replicas does not prevent tail-latency spikes if requests still land on a replica that is mid-batch or still loading weights into GPU memory. Cloud and data center deployments therefore place a **load balancer** between clients and model servers to route around busy or unready replicas. A serving load balancer protects the fleet by coupling request distribution with readiness checks and traffic shifting. It routes incoming requests across replicas using policies such as round-robin or least-connections so that traffic steers around slow or saturated workers before queues build, and it backs that routing with health monitoring that inspects both process liveness and model initialization state—confirming that weights are loaded and warmup passes are complete before a replica receives live queries. The same routing control also supports progressive rollouts, shifting traffic gradually between model versions rather than cutting over abruptly.

↪ Shadow, canary, and blue-green release strategies that build on this traffic shifting are developed in section 14.4.3.

On a single machine running several model instances, such as several Open Neural Network Exchange (ONNX) Runtime sessions, the serving framework and the operating system queue requests without a separate balancer; a dedicated balancer becomes necessary once replicas span machines, and its distribution algorithms belong to that distributed setting. This chapter therefore plans capacity one node at a time. The unit of capacity is the sustained model throughput of one node, and the queueing analysis in section 13.4 determines the utilization at which that node can no longer meet its latency target and a second replica must be added behind the balancer. Protocol selection, process architecture, and the balancer in front of the replicas establish the structural skeleton of a serving deployment; evaluating actual operational performance requires tracing how latency accumulates as one request traverses each stage of a replica's runtime pipeline.

13.3 Request Lifecycle

A single HTTP request carrying a 224×224 JPEG image arrives at an inference server. Between the moment the first byte enters the network stack and the moment the classification payload departs, the request traverses a sequence of pipeline stages, grouped below into preprocessing, inference, and postprocessing, with data transfer between them. Each stage consumes a slice of the end-to-end latency budget, making granular phase decomposition essential for targeted performance tuning.

13.3.1 The latency budget

A serving system that completes thousands of inferences per second still fails its users if individual requests take two seconds to return and miss their interactive deadlines. For dynamic inference systems, the serving inversion established in section 13.1 therefore imposes a strict latency budget that shapes system design (Gujarati et al. 2020).

Relevant metrics expand from aggregate throughput and mean latency to the latency distribution. Mean and p50 describe the center, while p95 and p99 expose the slow tail. If mean latency is 50 ms

but p99 is two seconds, the p99 threshold is 40× the mean, so about 1 percent of requests are at or beyond two seconds. Whether that tail matters depends on the service and its users, which is why percentile targets should come from the SLO rather than convention.[8] Managing these percentile constraints requires decomposing the total allowed response time into a **latency budget** that allocates time across each processing phase, the serial accounting that the latency budget principle (12) states formally.

[8] **Tail latency**: Unlike averages, percentile latencies reveal the performance impact of system outliers common in ML serving, such as model cache misses or garbage collection pauses. These rare, high-latency requests can disproportionately harm user satisfaction and business outcomes. Because the magnitude is workload-specific, services validate percentile targets (p95, p99) against their own users and SLOs.

Definition 13.2: Latency budget

Latency budget is the time allocated to request stages under an end-to-end deadline; the SLO also specifies the required completion quantile.

1. **Significance**: It acts as a zero-sum constraint system where any milliseconds consumed by serialization or network overhead directly reduce the latency budget ($L_{\text{lat,total}}$) available for model inference.
2. **Distinction**: The budget allocates time across stages, while the latency SLO of section 12.6.2.1 couples the deadline (for example, < 50 ms) with a completion quantile (p99 or p99.9). A service-level indicator (**SLI**) is the measured quantity used to evaluate that objective. Tail latency spikes can stem from queueing, batch formation, cache state, interference, or data movement.
3. **Common pitfall**: A frequent misconception is that the "model" has the entire budget. In the illustrative breakdown in table 13.3, the remainder is consumed by host preprocessing and data movement (JPEG decode, resize, normalization, and PCIe transfer), while upstream network and serialization overheads (table 13.5) consume further margin.

Every serving request traverses three sequential phases that share this latency budget: preprocessing converts raw input such as image bytes or text strings into model-ready tensors, inference executes the model computation over those tensors, and postprocessing turns the resulting device outputs into user-facing responses. Accelerating model execution alone yields diminishing returns when host-side preprocessing and response formatting consume the majority of the latency budget, an immediate manifestation of Amdahl's law. As hardware compilers compress accelerator kernel runtimes into sub-millisecond durations, host-device coordination and memory transfers emerge as the primary latency bottlenecks.



Inference is one slice of the latency budget; preprocessing rivals it.

13.3.2 Stage-by-stage latency breakdown

An engineer who measures only total request duration in a vision service sees a single millisecond number that lumps together CPU-bound JPEG decompression, host-to-device PCIe transfers, accelerator kernel execution, and response formatting, making it easy to spend weeks optimizing GPU kernels that account for less than half of total response time. Timing each pipeline boundary independently isolates the actual bottleneck before optimization begins.

Example 13.2: ResNet-50: Latency budget breakdown

Table 13.3 details the per-phase latency breakdown for one ResNet-50 serving request on the reference H100, with the forward pass taken from the chapter's modeled profile and the host phases as stated assumptions.

Table 13.3: ResNet-50 latency budget: Per-phase breakdown of a single serving request on the reference H100 (modeled forward pass, assumed host phases), showing that host preprocessing alone takes longer than the ResNet-50 forward pass. The percentages expose where engineering effort pays off in this scenario. Per-phase percentages are rounded to the nearest whole number and may not sum to exactly 100.

Phase	Operation	Time	Percentage
Preprocessing	JPEG decode	3 ms	35%
Preprocessing	Resize to 224×224	1 ms	12%
Preprocessing	Normalize (mean/std)	0.5 ms	6%
Data Transfer	CPU→GPU copy	0.5 ms	6%
Inference	ResNet-50 forward pass	3.5 ms	41%
Postprocessing	Softmax + top-5	0.1 ms	~1%
Total		**8.6 ms**	**100%**

Systems insight: In this ResNet-50 serving budget, preprocessing consumes 52 percent of latency even though the forward pass performs nearly all of the arithmetic. If a compiled TensorRT engine reduces the forward pass to the modeled 0.33 ms, preprocessing would dominate at 83 percent.

Laid on the node's hardware, the same budget occupies three resources in turn, and the accelerator is busy for less than half of the request (figure 13.3). While a host CPU core decompresses the JPEG, resizes the image, and normalizes pixel values before copying the tensor across PCIe, the GPU waits completely idle for work to arrive.

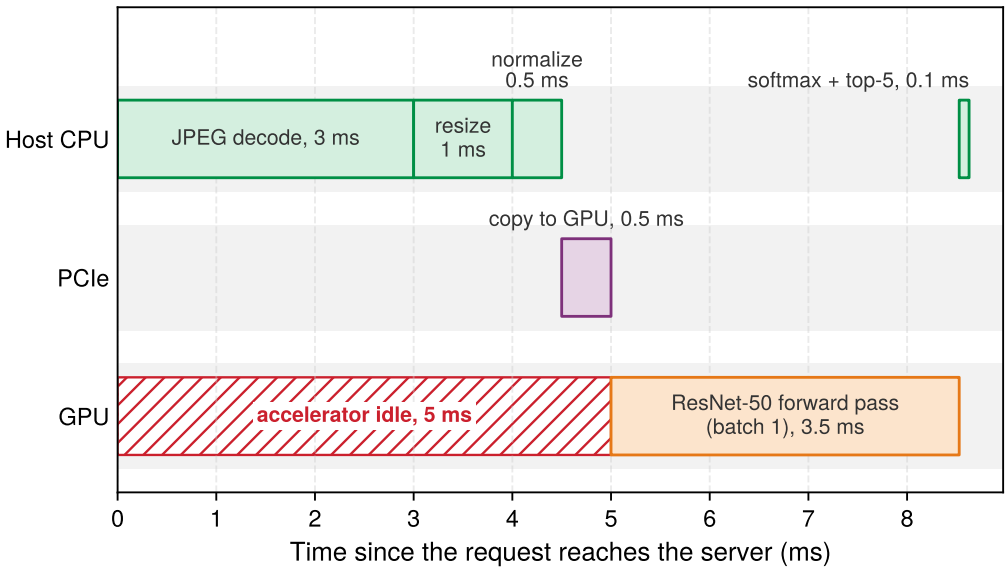


Figure 13.3: One Request on One Node: The reference ResNet-50 request drawn on the three resources it crosses. JPEG decode, resize, and normalization occupy a host CPU core, the input crosses PCIe, and the forward pass occupies the accelerator; softmax and top-5 selection are drawn on the host. Durations follow table 13.3, so the accelerator is busy for 41 percent of the 8.6 ms request and idle while the host prepares the input. Network transfer, serialization, and batching delay lie outside this budget and are not drawn.

The 3.5 ms batch-one forward pass in table 13.3 is the chapter's reference profile for ResNet-50: an eager framework running FP16 on the reference H100, modeled from the iron law's terms rather than measured, and the later ResNet-50 examples derive from it. The framework issues 174 kernels, one per operation, and each pays 10 µs of Python dispatch and 5 µs of driver launch, the per-operation costs of section 7.3.4. Each kernel also occupies the accelerator for at least an assumed 5 µs, and the arithmetic itself, at the 25 percent of peak that ResNet-50 sustains in section 8.4.2, takes about 33 µs

per image. A compiled INT8 engine anchors the other end of the range: MLPerf Inference measured 89,500 images/s on one H100 at large batch (Reddi et al. 2019). The capacity plan in section 13.4.3 uses the reference profile as the service time, the batching sweep in section 13.5.1 starts from it at batch one, and the cold-start timeline in section 13.8.1 assumes its warm steady state. The runtime and precision comparisons in section 13.6 change one term of the same model at a time, so the effect of each change is visible.

In the ResNet-50 scenario, dense convolutional arithmetic dominates the model stage's work. That does not make the stage compute bound at every batch size. At batch size one on the reference H100, issuing the kernels takes 2.6 ms, 74 percent of the forward pass, so the stage is overhead-bound and runs far below peak arithmetic throughput; batching amortizes that fixed cost and fills the machine (section 13.5.1). Changing the model architecture can move the dominant cost out of dense arithmetic entirely, and recommendation systems make this shift concrete.



Lighthouse 13.1: Lighthouse example: DLRM serving

Scenario: Serving DLRM with a 10 ms p99 latency budget.

Analysis: While ResNet-50's model stage is dominated by convolutional neural network compute, DLRM's dominant model-stage cost is embedding-table memory access. End-to-end serving bottlenecks still require measuring the full path: preprocessing, inference, postprocessing, and data movement. Table 13.4 breaks the recommendation request down by phase.

Table 13.4: DLRM serving latency: Illustrative per-phase breakdown of a recommendation request under a 10 ms p99 budget, contrasting DLRM's memory-bandwidth-bound embedding lookups against ResNet-50's convolution-dominated forward pass. Adding compute to the inference stage does not help once embedding-table bandwidth is the binding constraint.

Phase	Operation	Time	Bottleneck
Input Parsing	Request parsing	0.5 ms	CPU
Embedding Lookup	Fetch 100+ dense vectors	6 ms	memory bandwidth
Inference	MLP forward pass	1.5 ms	Compute
Postprocessing	Ranking & Filtering	1 ms	CPU
Total		**9 ms**	

Systems insight: In DLRM, the "Inference" multilayer perceptron stage is only ~17 percent of the latency. The majority of time is spent in embedding lookups, retrieving 128-dimensional embedding vectors from tables that hold about 100 GB in the reference model and reach terabytes in production. This is a memory-bandwidth and capacity-bound workload where adding more compute does not help unless the embedding tables can be served faster.

Together, table 13.3 and table 13.4 expose the same general failure mode: optimization effort goes where ML expertise applies (model quantization, pruning) while the binding constraint sits elsewhere (image decoding on CPU for ResNet-50, embedding-table memory bandwidth for DLRM). For an isolated serial request with other stage times held fixed, model-only optimization is bounded by the time spent outside the model, the Amdahl ceiling that section 12.3.4 derives for a request path. Serving modifies that familiar result in two ways. Accelerator progress tightens the ceiling, because each hardware generation shrinks the model term while the host stages stay where they were; the perspective below quantifies this for the reference request. Under load, a faster model also shortens the queue, an effect the fixed-fraction calculation cannot see and the first fallacy in section 13.11 develops.



Systems Perspective 13.2: The quantitative approach to serving

Analysis: Amdahl's law (principle 8) sets this ceiling (section D.2.3.1 provides the formal derivation). Preprocessing (4.5 ms) and data transfer (0.5 ms) consume 58 percent of total latency. Optimizing the model 10× faster (3.5 ms → 0.35 ms) yields only a 1.6× end-to-end

speedup (reducing latency from 8.6 ms to 5.5 ms). Focusing exclusively on model optimizations like quantization or pruning yields diminishing returns when non-model stages dominate execution time.

Mechanism: Domain-specific accelerators, such as the Tensor Processing Unit (TPU) and Tensor Cores, replace complex control logic with dense multiply-accumulate arrays, which shrinks the model term. The host-side stages stay on general-purpose CPUs and I/O buses, so the faster the accelerator, the larger their share of end-to-end latency.

Systems insight: Profile before optimizing. If preprocessing dominates, GPU-accelerated pipelines (NVIDIA DALI) that move preprocessing next to the accelerator may outperform model quantization, although the end-to-end gain is pipeline-specific.

13.3.2.1 Per-request overheads outside the model

Beyond model forward-pass execution, every request pays fixed overheads to the host system and network stack. When an inference query arrives, it traverses the network interface, moves across kernel and user space, deserializes from wire protocols into memory buffers, queues for batch assembly, and copies across the PCIe interconnect to accelerator device memory. These host-side operations run on general-purpose CPUs and I/O buses, which is why their share grows as the model term shrinks. Each overhead has a representative cost, a scaling behavior, and a design choice that reduces it (table 13.5).

Table 13.5: Per-Request Overheads Outside the Model: A representative breakdown of noninference latency sources. While individual components like serialization can be below 1 ms, they compound. Under the upper ends of the displayed ranges, these overheads together can rival the reference ResNet-50 forward pass. The engineering goal is to measure the actual request path and reduce the binding costs through choices such as binary payloads, persistent connections, and fewer copies.

Overhead	Typical Cost	Scaling Behavior	Reduction Strategy
Network I/O	1–5 ms	Linear with payload	Compression, Region Colocation
Serialization	50–500 µs	Linear with payload	gRPC/Protobuf (vs. JSON)
Queuing	0.1–10 ms	Diverges near capacity	Dynamic Batching, Autoscaling
Dispatch	10–50 µs	Constant per batch	Kernel Fusion (reduce launches)
Data Copy	100–500 µs	Linear with tensor	Zero-Copy/Shared Memory

13.3.2.2 The killer microseconds problem

Barroso, Patterson, and colleagues identified a critical gap in how computer systems handle latency across time scales (Barroso et al. 2017). Nanosecond-scale stalls (such as cache misses) are hidden by hardware instruction-level parallelism, while millisecond-scale delays (such as disk I/O or network RPCs) are hidden by operating system thread context switching. Operations in the microsecond range (1–100 µs) fall into the gap between these two mechanisms. They are too long to spin-wait on a CPU core without wasting compute cycles and power, yet too short for the operating system to context-switch to another thread without paying scheduling overheads that exceed the wait time itself.

Serving infrastructure operates directly in this killer microseconds regime. Using the representative ranges in table 13.5, serialization at 50–500 µs, dispatch at 10–50 µs, and data copy at 100–500 µs are each individually small, but for the 3.5 ms reference forward pass, these named microsecond-scale overheads collectively add about 5 percent to 30 percent of the latency budget before network and queuing delays are counted. No single overhead justifies optimization in isolation, yet together they determine whether the system meets its SLO.

Tackling this compound overhead demands systematic architectural intervention across the software-hardware interface. When microsecond delays accumulate across serialization, kernel dispatch, and memory copies, isolated optimizations to individual functions yield negligible improvements. Instead, high-performance serving runtimes remove the boundaries themselves by employing zero-copy memory buffers between networking and execution runtimes, maintaining persistent CUDA streams to bypass driver dispatch latencies, and migrating preprocessing kernels directly onto the accelerator to avoid intermediate host-device transfers.

13.3.3 Resolution and input size trade-offs

Bigger inputs cost more to decode and more to run, so input resolution is a serving control as well as a model property. Model architectures are often evaluated at fixed resolutions, but the serving runtime must ingest variable client inputs and rescale them before kernel execution. In convolution-dominated vision networks, floating-point operations scale quadratically with spatial dimensions ($\mathcal{O}(H \cdot W)$), whereas static costs (such as weight streaming from accelerator memory and kernel launch overheads) remain invariant. Varying input resolution therefore shifts the operational arithmetic intensity across the roofline boundary. Compute-bound stages slow down quadratically as pixel count expands, whereas memory-bound stages scale with activation memory footprint.

For convolution-dominated, compute-bound models, equation 13.1 gives a first-order approximation in which throughput scales inversely with resolution squared, comparing two input resolutions r_1 and r_2 (image side lengths in pixels):

↩ The roofline boundary and arithmetic intensity ridge points that separate compute-bound from memory-bound stages are established in section 11.6.

$$\frac{\text{Throughput}(r_2)}{\text{Throughput}(r_1)} = \left(\frac{r_1}{r_2}\right)^2 \tag{13.1}$$

Doubling resolution from 224 to 448 gives a 4× compute-scaling estimate, whereas the illustrative 3.6× scenario includes fixed overhead. Table 13.6 shows the mechanism behind that gap. Each step up in resolution reuses the same weights across more pixels, so the modeled arithmetic intensity rises and the fixed weight traffic becomes a smaller share of the work. That amortization is why the illustrative slowdown falls short of the pure compute estimate. The model counts only weights and one pass over a scaled activation term, so it establishes the direction of the effect; a profiler decides whether the executed kernels are compute or memory bound.

Table 13.6: Resolution and Modeled Arithmetic Intensity: This estimate divides FLOPs by weight bytes plus one assumed read/write of a scaled activation term. It omits aggregate intermediate-tensor, cache, and workspace traffic, so crossing a ridge point does not establish the executed-kernel bottleneck.

Resolution	Assumed Activation Term	Modeled Arith. Intensity
224×224	12.5 MB	64.4 FLOP/byte
384×384	36.7 MB	137 FLOP/byte
512×512	65.3 MB	183.9 FLOP/byte
640×640	102.0 MB	218.4 FLOP/byte

13.3.3.1 Resolution strategies in production

Different deployment contexts choose resolution according to which constraint binds first. A mobile application may accept 224×224 inputs because latency and battery life outweigh marginal accuracy gains, whereas medical-imaging workloads often step up to 512×512 or higher because diagnostic detail justifies the extra compute. Between those fixed choices, a perception pipeline may run a wide field at low resolution and reserve high-resolution crops for fine-grained recognition, while a public API that receives arbitrary user uploads can either normalize every image to a single input size or route each request to one of several validated resolution profiles.

13.3.3.2 Adaptive resolution

Adaptive resolution lets production systems select resolution dynamically based on content. One approach runs a lightweight classifier at 128×128 to categorize content type, then selects task-appropriate resolution with documents at 512×512, landscapes at 224×224, and faces at 384×384. In this illustrative scenario, the policy provides 1.4× throughput improvement with 99.2 percent accuracy retention vs. fixed high resolution. This pattern trades preprocessing cost from running the lightweight classifier for inference savings on the main model. While resolution sets the cost of the stages before the model, the stage after it is usually far cheaper, though it still crosses the device boundary and spends host CPU time and therefore belongs on the same budget.

13.3.4 Postprocessing

When a model finishes a forward pass, its raw output logits sit in accelerator memory that client applications cannot read directly; copying thousands of unreduced floats across the PCIe bus and

formatting strings on the CPU can stall the worker thread between batches. Postprocessing turns those device tensors into structured application responses across the physical boundary between the accelerator and the host CPU, and how the serving engine splits computation across that boundary determines whether the final stage finishes in microseconds or blocks the pipeline.

13.3.4.1 From logits to predictions

Classification models emit unnormalized logits across the label space, and turning those raw scores into application-ready predictions requires a chain of reduction, filtering, and calibration steps. When only the single most likely label matters, argmax selection extracts the index of the highest logit, whereas applications that present ranked alternatives use top-k extraction to keep the k largest scores alongside their class indices. Before returning those candidates, probability calibration (such as temperature scaling) rescales raw logits so that output confidences reflect empirical accuracy, allowing thresholding to discard predictions that fall below a required certainty cutoff.

Executing reduction operations on the accelerator can reduce host-device transfer volume when the saved transfer cost outweighs the device execution and launch overhead. Performing softmax, argmax, or top-k selection in device memory prunes the payload before it crosses the PCIe bus. In ResNet-50 classification, for example, running top-k on the GPU reduces the transfer payload from 1,000 float32 logits (4,000 bytes) to five values and five integer indices. For ResNet-50 image classification, listing 13.1 traces this execution path from GPU tensor operations to host CPU label lookup and response formatting.

Listing 13.1: ResNet-50 Postprocessing: Normalizes raw logits into probabilities, extracts top-k predictions, and formats the API response.

```
# Normalize raw logits into probabilities
# Input: logits tensor of shape (1, 1000), one score per ImageNet
# class
probs = torch.softmax(logits, dim=-1)  # Normalize to sum=1 on GPU

# Extract top-5 predictions for multi-class response
# topk returns (values, indices) sorted by probability
top5_probs, top5_indices = probs.topk(5)  # GPU operation
top5_probs = top5_probs.squeeze(0).tolist()
top5_indices = top5_indices.squeeze(0).tolist()

# Map class indices to human-readable labels
# imagenet_labels: list of 1000 class names from synset mapping
labels = [imagenet_labels[i] for i in top5_indices]  # CPU lookup

# Format response with predictions and metadata for API contract
response = {
    "predictions": [
        {"label": label, "confidence": float(prob)}
        for label, prob in zip(labels, top5_probs)
    ],
    "model_version": "resnet50-v2.1",  # Client-side version tracking
    "inference_time_ms": 5.2,  # Observability for latency monitoring
}
```

In listing 13.1, total postprocessing time is approximately 0.1 ms, negligible compared with preprocessing and inference. The GPU handles the normalization and top-5 reduction, so the subsequent `.tolist()` call transfers only tens of bytes across PCIe. However, `.tolist()` forces a synchronous device-to-host copy, blocking the host thread until the GPU stream finishes. In high-throughput pipelined servers, such synchronization barriers must be scheduled carefully to avoid stalling the CPU thread while it feeds subsequent requests. Fitting a calibration mapping is an offline task; applying a simple fitted mapping online is usually inexpensive, but it is valuable only when downstream decisions require confidence scores whose empirical meaning has been validated.

13.3.4.2 Output formatting

Production systems package predictions to satisfy application-level API contracts. These contracts govern JSON serialization schemas, floating-point formatting, error codes, and fallback defaults when thresholding rejects all candidates. In Python-based runtimes, repetitive JSON serialization and string allocation can consume unexpected host CPU cycles under heavy concurrency, turning a microsecond-scale formatting step into a CPU bottleneck. High-throughput servers address this by delegating serialization to native C++ runtimes or binary protocols like Protocol Buffers. Response metadata, including model version and inference execution time, completes the payload to enable downstream observability. Characterizing postprocessing closes the individual request's latency budget, though analyzing these stages strictly in isolation assumes sequential evaluation, where each transaction completes its entire lifecycle before subsequent work begins.

13.3.5 Hardware utilization and request pipelining

Executing each request in lockstep isolation leaves specialized accelerators heavily underutilized. Because network termination, payload preprocessing, neural network evaluation, and response serialization bind on distinct physical subsystems (host network interfaces, CPU cores, PCIe interconnects, and accelerator matrix units), strict serial processing forces the accelerator into idle bubbles while the host prepares incoming tensors. The central systems objective is to maximize the accelerator's **duty cycle**, the proportion of wall-clock time spent actively executing arithmetic kernels, without violating request tail-latency deadlines. Serving runtimes achieve this through **request pipelining**, overlapping asynchronous host preprocessing of forthcoming queries with concurrent accelerator execution on previously staged batches.

13.3.5.1 Overlapping I/O and compute

↩ Asynchronous host-accelerator transfer and double buffering mechanisms are established in section 11.5.4.2.

Figure 13.4 sequences execution across CPU preprocessing and GPU inference to demonstrate how request pipelining recovers lost compute capacity. In serial execution (A), each request must complete its entire lifecycle (Network → CPU Preprocessing → GPU Inference → Postprocessing) before the next request begins, leaving the GPU idle for more than 50 percent of wall-clock time. Pipelining concurrent requests (B) overlaps CPU preprocessing of upcoming requests with active GPU inference on earlier inputs, eliminating the idle bubbles and maximizing accelerator duty cycle.

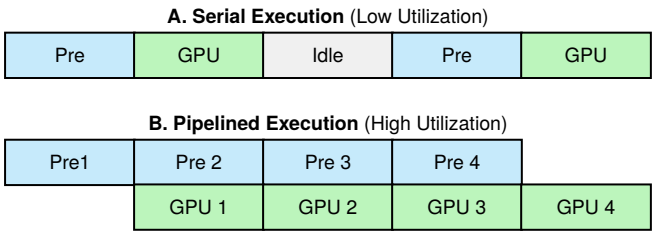


Figure 13.4: Request Pipelining: Pipelining hides latency by overlapping independent operations across different hardware resources. In pipelined execution (B), the CPU processes the next request's data while the GPU executes the current request's inference. This can increase accelerator duty cycle and throughput without changing the model.

Pipelining relies on non-blocking hardware queues and asynchronous execution streams. Instead of blocking until a GPU kernel completes, the host CPU thread enqueues memory transfers and kernel launches into an accelerator stream and immediately begins preprocessing the next incoming request. The accelerator's DMA engine and compute cores execute those queued operations independently, overlapping host preprocessing with device execution.

13.3.5.2 The systems metric: Hardware duty cycle

System efficiency here measures accelerator duty cycle rather than the fraction of peak arithmetic throughput or bandwidth achieved. In equation 13.2, compute time is the union of useful kernel-active intervals on each resource, summed across resources; overlapping kernels on one GPU are not counted twice.

$$\text{System Efficiency} = \frac{\sum T_{\text{compute}}}{\text{Wall Clock Time} \times \text{Resource Count}} \tag{13.2}$$

The reference ResNet-50 request spends 4.5 ms preprocessing on a host CPU core and 3.5 ms on the accelerator, so a serial system keeps the accelerator busy only 41 percent of the time. With at least two requests in flight in a sustained stream, the accelerator works on one request while the host prepares the next, and duty cycle rises to 78 percent after pipeline fill. It cannot reach 100 percent with one preprocessing worker, because the host stage is the slower of the two; a second worker removes that cap. A finite pair still pays pipeline fill and drain time, and stochastic service must retain enough capacity headroom to meet its tail-latency objective. If the CPU is too slow to feed the GPU, the system becomes CPU-bound, and further model optimization provides zero throughput gain; this is the Amdahl ceiling of section 13.3.2 seen from the throughput side. Hardware upgrades tighten it, because accelerator compute throughput has grown far faster than CPU single-thread performance across successive generations. A system that was compute-bound on an older accelerator may become CPU-bound after a hardware upgrade, not because preprocessing got slower but because the model got dramatically faster while the CPU did not.

Checkpoint 13.1: The latency budget

A request's latency is spent across every stage of its path, not only in the model. Use this checkpoint to locate where the budget goes before choosing an optimization.

- ☐ **Amdahl ceiling**: in the reference ResNet-50 request, why does a 10× faster model yield far less than a 10× faster response, and why does a faster accelerator make the ceiling tighter rather than looser?
- ☐ **Tail vs. mean**: can you explain why mean latency cannot demonstrate compliance with a p99 SLO?
- ☐ **Latency budget**: given a 50 ms request deadline and a combined upper bound of 18 ms for all other stages, including network transit, preprocessing, transfer, and postprocessing, how much remains for inference and queueing? Why would a measured 18 ms mean for those stages be insufficient to certify a p99 SLO?
- ☐ **Pipelining**: explain how overlapping CPU preprocessing with GPU inference raises accelerator duty cycle without changing the model, and what happens to throughput when the CPU stage is slower than the GPU stage.

Tracing an isolated request reveals the intrinsic execution time of each pipeline phase. In production, however, multiple requests arrive concurrently and contend for shared accelerators, CPU cores, and memory buses, introducing queuing delays that dwarf raw computation time.

13.4 Queuing Theory

An inference service that responds in a few milliseconds at moderate load can suffer sudden, tenfold latency spikes when traffic climbs toward capacity, even though no hardware has failed and the model's execution time has not changed. Because requests arrive in irregular bursts and contend for shared accelerator queues, host-to-device PCIe links, and memory buses, queuing theory governs how waiting times grow and why latency explodes nonlinearly as utilization approaches capacity.

13.4.1 Little's Law

Before deciding how many accelerators to provision, an engineer must know how many requests will sit inside the server at the same time; underestimating that in-flight population exhausts host request buffers, worker threads, and device activation memory under load. Given an expected arrival rate and mean response time, Little's Law determines this average concurrency directly from throughput, while queuing models predict how response times climb as utilization nears saturation.

↪ The derivation of Little's Law from conservation of requests is collected for reference in section D.2.5.

Systems Perspective 13.3: Notation alert: L vs. latency

Classical queueing texts write Little's law with single letters for the in-system count and the time in system; this book writes Q_{req} for the count and T_{lat} for the time, because the classical count letter would collide with the loss $\mathcal{L}$ and with the iron law's fixed-latency term L_{lat}. In queuing theory, T_{lat} denotes the *response time* or *time in system per request*; queue-only waiting time is W_q. Here, Q_{req} denotes the average in-system request count, λ_{arr} the arrival rate, μ the service rate of a server that handles one request at a time, and $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ the serving utilization. The subscripts distinguish queueing notation from the degradation equation's λ sensitivity parameter and keep serving utilization from occupying bare ρ. Section 13.5.2 adds the few symbols that batching needs.

Formulated by John D. C. Little (Little 1961), **Little's law** establishes an exact relationship among three macroscopic metrics in any stable queuing system:

$$Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}} \tag{13.3}$$

where Q_{req} is the average number of requests residing in the system, λ_{arr} is the arrival rate (requests per second), and T_{lat} is the average time each request spends from arrival to completion.

Little's law holds regardless of the arrival distribution, service-time distribution, or scheduling discipline; it requires only that the system is stable and that requests are neither created nor destroyed internally. For an inference service targeting 1000 QPS with a mean response time of 50 ms, Little's law dictates the average in-flight request population that host memory buffers and worker threads must sustain.

Napkin Math 13.2: Little's Law capacity sizing

Problem: How many requests are in the system on average at 1,000 QPS?

Math: Little's Law gives $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$, so average population equals throughput multiplied by mean time in system.

Given:

- **Arrival rate** (λ_{arr}), equal to throughput in a stable system: 1,000 QPS.
- **Mean time in system** (T_{lat}): 50 ms (0.05 s).

Math: Q_{req} = 1,000 QPS $\times$ 0.05 s = 50 concurrent requests on average

Systems insight: The system holds 50 requests on average across batch and queue state. That population is a memory and concurrency requirement, not a limit on the accelerator. A server whose GPU executes at most 32 requests per batch can still sustain this load, because the requests not currently executing wait in host memory; service rate, queueing behavior, and the tail distribution, not the population itself, determine whether the load is feasible.

Because equation 13.3 binds these three variables, constraining any two dictates the third. If an inference service averages 10 ms per request ($T_{\text{lat}} = 0.01$ s) and holds 50 concurrent requests on average ($Q_{\text{req}} = 50$), the sustainable arrival rate must be $\lambda_{\text{arr}} = Q_{\text{req}}/T_{\text{lat}} = 5000$ requests per second. Conversely, if host memory limits concurrency to an average of 10 requests while average time in system remains 10 ms, the system cannot exceed a throughput of 1,000 requests per second without dropping traffic.

13.4.2 The utilization-latency relationship

Little's Law describes average system inventory under steady-state conditions, but it does not reveal how latency behaves as traffic approaches system capacity. The M/M/1 queuing model (in Kendall's notation: Markovian, that is Poisson, arrivals; Markovian, that is exponential, service times; and one server) quantifies the headroom required to prevent latency collapse (Harchol-Balter 2013).[9] For a system with Poisson arrivals[10] and exponential service times, equation 13.4 gives the average time

[9] **M/M/1 queue**: Queuing theory originated with Agner Krarup Erlang's 1909 analysis of the Copenhagen Telephone Exchange, where call arrivals were modeled as memoryless (Poisson).

[10] **Poisson process**: Named after French mathematician Simeon Denis Poisson (1781–1840), this stochastic model describes events occurring continuously and independently at a constant average rate, so counts in disjoint intervals are independent.

in system:

$$T_{\text{lat}} = \frac{1}{\mu - \lambda_{\text{arr}}} = \frac{\text{service time}}{1 - \rho_{\text{serv}}} \tag{13.4}$$

where λ_{arr} is the arrival rate, μ is the service rate (requests per second the server can handle), and $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ is the utilization (the fraction of time the server is busy).

Because the denominator contains $1 - \rho_{\text{serv}}$, latency grows nonlinearly as utilization approaches unity, and small incremental increases in traffic near saturation trigger severe backpressure and queuing delays. The standard M/M/1 formulation assumes exponentially distributed service times. For compiled neural networks processing fixed-size batches and static tensor dimensions, execution times are nearly constant, making an M/D/1 queue a tighter model of mean queuing delay. The analysis here retains the M/M/1 model because it yields closed-form solutions and predicts greater mean queueing delay than M/D/1 under matched arrival rate and mean service time. In production environments, variable input lengths, dynamic batch formation, CPU preprocessing, and key-value cache lookups introduce runtime variance that quickly breaks deterministic assumptions.[11] Under this model, average time in system multiplies rapidly as utilization approaches 100 percent (table 13.7).

[11] **Kendall notation**: Kendall's A/S/c (arrival/service/servers) shorthand (Kendall 1953) is the one already used for $M/M/1$, where "M" signifies a Markovian (memoryless) process; the "D" in M/D/1 denotes deterministic service. Neither model replaces measurement of a production workload's arrival process, service-time variance, or tail behavior.

Table 13.7: Utilization-Latency Relationship: Average time in system (wait + service) as a multiple of service time for an M/M/1 queue. At 50 percent utilization, time in system is 2× service time; at 90 percent, it reaches 10×. This nonlinear growth explains why systems that perform well at moderate load suddenly violate SLOs when traffic increases: moving from 80 percent to 90 percent utilization doubles latency.

Utilization (ρ_{serv})	Latency Multiple	Example (3.5 ms service)
50%	2×	7.1 ms
70%	3.3×	11.8 ms
80%	5×	17.6 ms
90%	10×	35.3 ms
95%	20×	70.6 ms

Plotted against the serving utilization ρ_{serv}, the fraction of time the server is busy, the same $M/M/1$ model shows why the knee matters (figure 13.5). Latency stays nearly flat at low utilization and then climbs without bound as the server approaches 100 percent busy, because each new arrival is increasingly likely to find work already waiting. The 99th percentile curve rises even faster in absolute terms, staying a fixed multiple above the mean that section 13.4.3 derives next. The green zone below 50 percent utilization and the red zone above 70 percent illustrate a planning policy, not universal operating limits. This rapid compounding near saturation is why latency-sensitive systems reserve headroom rather than planning for permanently high accelerator occupancy.

13.4.3 Tail latency

↪ Why long-tailed latency distributions make high percentiles, rather than the mean, govern what users experience is collected for reference in section B.2.1.

A serving cluster running at 70 percent utilization can report a comfortable mean latency while its slowest 1 percent of requests wait more than four times as long in the queue. Because averages hide this tail contention, production SLOs specify percentile targets (p95, p99) that bound experience for the slowest requests (Dean and Barroso 2013).

The M/M/1 model that produced the mean in equation 13.4 also yields the full latency distribution, and the tail follows from it. Total request latency (queue wait plus service) is exponentially distributed with cumulative distribution function $F(t) = 1 - e^{-t/T_{\text{lat}}}$, where $T_{\text{lat}} = \frac{\text{service time}}{1-\rho_{\text{serv}}}$ is that mean response time. To derive the 99th percentile latency (p99), we solve for the time t_{99} where 99 percent of requests have completed ($F(t_{99}) = 0.99$):

$$1 - e^{-t_{99}/T_{\text{lat}}} = 0.99 \implies e^{-t_{99}/T_{\text{lat}}} = 0.01 \implies -\frac{t_{99}}{T_{\text{lat}}} = \ln(0.01) = -\ln(100) \approx -4.605$$

Solving for t_{99} and substituting the mean latency T_{lat} yields the governing expression for tail latency:

$$T_{\text{lat,p99}} \approx \frac{\text{service time}}{1 - \rho_{\text{serv}}} \cdot \ln\left(\frac{1}{1 - 0.99}\right) \approx \frac{4.6 \cdot \text{service time}}{1 - \rho_{\text{serv}}} \tag{13.5}$$

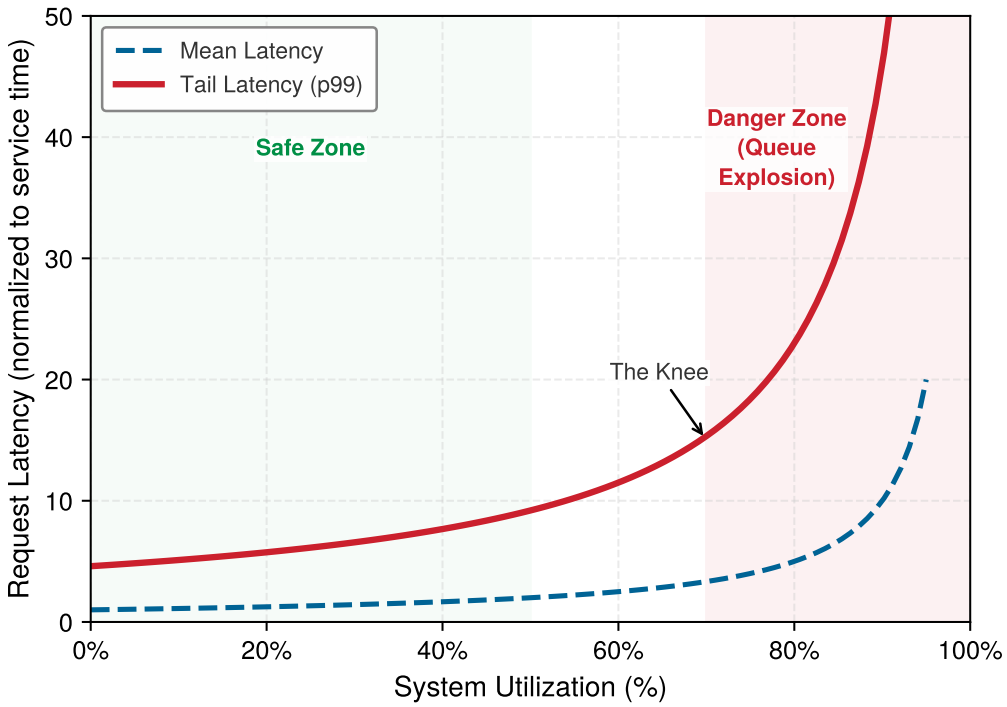


Figure 13.5: The Tail Latency Explosion: Request latency vs. serving utilization ρ_{serv}. Mean latency (blue) and the p99 approximation (red) both grow nonlinearly as utilization approaches saturation; the 70 percent marker is an illustrative planning knee rather than a model-derived breakpoint. In an $M/M/1$ queue, the 99th percentile latency scales as $-\ln(0.01) \approx 4.6\times$ mean latency, illustrating why tail delays compound violently near saturation.

At 70 percent utilization, average latency is only 3.3 times service time, while p99 latency reaches 15.3 times service time $(4.6/0.3 \approx 15.3)$. Evaluating a serving cluster by average latency therefore conceals severe tail degradation. Before exploring software mitigations like hedging, systems engineers must first provision sufficient accelerator capacity to satisfy tail-latency SLOs under peak load. For a ResNet-50 deployment, the M/M/1 tail-response-time model turns provisioning into a short sequence of steps.

Napkin Math 13.3: ResNet-50 capacity planning

Consider designing a ResNet-50 serving system with these requirements:

- **Queue-plus-GPU p99 subbudget**: 50 ms
- **Peak expected traffic**: 5,000 QPS
- **GPU service time** (batch one, the chapter's reference ResNet-50 profile): 3.5 ms

Step 0: Define the modeled request boundary. The worker model covers waiting for the GPU and the GPU forward pass only. This scenario allocates 50 ms to that portion of the request and separately assumes a combined 18 ms upper bound for network transit, serialization, preprocessing, transfer, and postprocessing. The end-to-end deadline must accommodate both allowances. A 50 ms end-to-end deadline would instead require subtracting the outside-worker allowance before applying the queue model; the arithmetic below would no longer apply. An outside-stage mean or p99 cannot substitute for the stipulated combined upper bound.

Step 1: Find safe utilization. From equation 13.5, $T_{\text{lat,p99}} \approx 4.6 \times$ service time $/ (1 - \rho_{\text{serv}})$. Setting $T_{\text{lat,p99}} \leq 50$ ms with 3.5 ms service time gives $\rho_{\text{serv}} \leq 1 - (4.6 \times 3.5\,\text{ms})/50\,\text{ms} = 0.68$

(68 percent maximum utilization). This is an M/M/1 p99 estimate under the stated arrival and service assumptions; a load test must validate the tail.

Step 2: Calculate required service rate. $\mu_{\text{required}} = 5,000\,\text{QPS}/0.68 = 7402.9\,\text{req/s}$

Step 3: Determine GPU count. In this M/M/1 model, the single-GPU rate is 283 req/s.

GPUs needed = 7402.9 req/s / 283 req/s = 26.1 $\rightarrow$ 27 GPUs

This single-GPU rate is the batch-1 service rate. Each request holds the GPU for the full service time, which keeps the M/M/1 model closed-form. Dynamic batching (section 13.5) amortizes fixed overhead across requests and raises the per-GPU rate several-fold, so a batched deployment may need fewer GPUs after its own latency distribution is validated; section 13.10.2 compares the two regimes.

Step 4: Add headroom for variance. This scenario adds 30 percent headroom for traffic spikes and variance: final count = 27 $\times$ 1.3 = 35.1, rounded up to 36.

Step 5: Verify fault tolerance. The 30 percent headroom addresses traffic variance, but production systems also need fault tolerance. With 36 GPUs, losing one leaves 35 GPUs handling 5,000 QPS. The postfailure utilization is 5,000 QPS / (283 req/s $\times$ 35) = 50.4%.

This remains well below the 68 percent safe utilization threshold, confirming N+1 redundancy is satisfied. For stricter fault tolerance requirements, N+2 redundancy (tolerating two simultaneous failures) would require 29 GPUs under the same safe-utilization threshold, or about 38 GPUs if the 30 percent headroom must remain after two simultaneous failures.

Result: Under the batch-1 M/M/1 assumptions, 36 H100 GPUs supply the modeled queue-plus-GPU capacity for 5,000 QPS within the 50 ms p99 subbudget, including one failed replica. End-to-end acceptance still requires validating the tail under load and the separately reserved outside-worker allowance.

The queuing analysis explains the capacity planning approach detailed in section 13.10.2 and connects directly to the MLPerf Server scenario. The Server scenario (section 12.4.4.1) is defined as Poisson arrivals scored by throughput against a tail-latency target. A Server result is therefore valid only when the run satisfies its percentile-latency constraint; otherwise, the target QPS must be reduced and the run repeated.

13.4.4 Tail-tolerant techniques

A capacity plan built on stationary Poisson arrivals holds only while traffic stays near the planned rate. Production traffic instead arrives in bursty, uncoordinated surges driven by user behavior, scheduled jobs, and daily cycles; when a surge pushes arrivals past the reserved headroom, accelerator service times cannot shrink to absorb the excess, so queries pile up in host and device queues and clients begin timing out.

Example 13.3: The 'Black Friday' traffic spike

Scenario: An e-commerce recommendation serving system experiences a 10× traffic surge (10,000 QPS) during Black Friday sales events.

Diagnosis: A fleet sized for normal traffic now receives more requests per second than it can complete. Utilization pins at 100 percent, and every second of the surge adds the excess arrivals to a queue that the servers cannot drain, so waiting time grows for as long as the surge lasts. In this scenario, latency reaches 10 s and clients time out. Section 13.4.2 showed why the queue starts growing sharply well before utilization reaches 100 percent.

Systems lesson: High average throughput does not prevent queueing collapse under traffic surges. Depending on the service, preserving a tail-latency objective may require load shedding, graceful degradation, or autoscaling before utilization reaches the workload's queueing knee.

Eliminating all sources of latency variability is often impractical. Production systems instead employ techniques that tolerate variability while still meeting SLOs (Dean and Barroso 2013; Dean

2012). For one serving node, the first distinction is between an isolated straggler and overload. Racing an isolated slow request can reduce its latency when spare capacity exists. When the node is overloaded, however, racing requests injects duplicate work and deepens the queue, so the system must instead protect the latency of admitted requests through graceful degradation or admission control. **Graceful degradation** returns lower-fidelity results to preserve throughput: classification pipelines fall back to cached approximations, generative models truncate decoding lengths, and model ensembles evaluate only a subset of members. Dynamically reducing active ensemble members shortens the service time $T_{\text{svc}}(B)$ of each batch of B requests, which increases the effective service rate $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$ and lowers utilization $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu_{\text{eff}}(B)$ toward a workload-appropriate operating point with more headroom, as illustrated by figure 13.5, trading a controlled accuracy reduction for SLO survival instead of an uncontrolled latency collapse. **Admission control** enforces an even stricter boundary. When the ingress queue exceeds a designated threshold, the server immediately rejects excess queries with HTTP 503 status codes rather than admitting work doomed to violate its latency deadline.

Setting this rejection threshold requires accounting for the work ahead of the arriving request. Consider a warmed, isolated worker that serves requests sequentially in first-in-first-out (FIFO) order, without batching or preemption. Assume known upper bounds on each request's service time and on every remaining stage outside the worker, including the response path. Subtract time already spent since the request began from its end-to-end latency allowance, then subtract the remaining outside-worker bound and the arriving request's service bound. The result is its waiting allowance. The worker's outstanding-work bound is the sum of the current request's residual service bound and the service bounds of queued requests. Admission passes the deadline screen only when that sum fits within the waiting allowance; a negative allowance rules out admission even to an idle worker. A request-count limit approximates this comparison only when service costs are sufficiently uniform. When they are, a practical starting point bounds the ingress queue at two or three requests per worker, the backlog a worker clears within two or three further service times, and the deadline screen and measured tails then tighten or relax that bound.

Suppose the outstanding work exceeds the full model's waiting allowance but fits the allowance obtained with a validated, faster model that is already resident and warmed. The server can admit the request through that fallback if its prediction quality meets the application's minimum requirement. Shortening the new request's service time frees waiting allowance, but does not remove work already queued ahead of it. If neither acceptable model fits, rejection avoids spending accelerator time on a result that would arrive too late. Rejection reduces availability, while fallback can reduce prediction fidelity; both outcomes must be counted alongside latency so that shedding difficult requests does not make the service appear successful merely by shrinking its measurement population.

The deadline screen is a deterministic bound only under the stated service, isolation, and scheduling assumptions. Mean service time or a per-stage p99 cannot replace those upper bounds. Variable input lengths, batch formation, and shared memory-bus contention require empirical calibration against the joint request-latency distribution. Completed-request tails must be reported together with rejection rates and fallback quality over offered traffic. Adaptive admission control can tighten the admissible backlog as measured tail latency approaches the SLO, but neither a fixed queue count nor the simplified screen certifies a production p99 target.

13.4.5 Scaling across multiple replicas

Once traffic outgrows a single node's safe utilization knee, no single-machine tuning can prevent queue build-up; higher aggregate volume requires **horizontal replication**, placing identical model replicas behind a load balancer so that each holds a complete model copy and serves disjoint queries without inter-node synchronization. (Model parallelism, which shards one large model across accelerators linked by a fabric such as NVLink, solves a different problem: fitting a model that exceeds one device's memory or latency budget.) The $M/M/c$ queuing model extends the single-server analysis to c parallel workers; pooling requests in one shared queue absorbs arrival bursts better than isolated single-server queues, provided dispatch is balanced (least-loaded routing, for example) and each replica keeps its own headroom.

13.4.5.1 The tail at scale problem

Dean and Barroso's analysis reveals why tail latency becomes critical as systems scale beyond single machines (Dean and Barroso 2013). When requests fan out to m parallel backends (querying an ensemble of models, searching sharded embedding indices, or parallelizing pipeline stages), the overall user request completes only when the slowest sub-task finishes. If sub-task completion events are independent and each has a 99 percent probability of finishing within latency threshold $L_{\text{lat,SLO}}$, the probability that all m backends finish within $L_{\text{lat,SLO}}$ is $(0.99)^m$. For a fan-out of $m = 10$, only $0.99^{10} \approx 90.4\%$ of requests meet the target; at $m = 100$, that probability drops to $0.99^{100} \approx 36.6\%$, meaning nearly two-thirds of user requests experience a tail delay. This "tail at scale" effect makes controlling single-node tail variance essential for overall system performance.

Hedging and tied requests limit the damage from a straggler replica. Under hedging, a request that has not completed within the expected time is duplicated to another server, and the client keeps whichever response arrives first.[12] With a stable latency distribution, a threshold at the historical 95th percentile duplicates about 5 percent of requests; correlated slowdowns raise that fraction and can make hedging amplify overload instead of reducing the tail. Tied requests instead enter several server queues at once and cancel the other copies when one server begins processing, which avoids waiting to detect a slow response. Both techniques create duplicate work. Cancellation before dispatch prevents a losing copy from consuming accelerator time, whereas cancellation after dispatch only discards its result, so either needs spare capacity and an explicit duplicate-work budget. Canary requests apply the same idea to fan-out, sending the request first to one or two servers and committing to the full fan-out only if they answer within the expected time.

Replicas or streams on one machine can also race requests, but sharing an accelerator or memory bus can correlate their slowdowns and exhaust the same headroom. The local admission decision remains the foundation; fleet coordination is an extension of it. Retries across a fleet obey the same duplicate-work constraint, because load balancers that retry rejected requests at other overloaded replicas amplify the overload, so replicas should share load information and shed the same fraction of requests rather than each rejecting independently. Production frameworks may use shortest-job-first or deadline-aware scheduling rather than FIFO, changing which queued work lies ahead of an arrival and requiring a corresponding admission analysis (Harchol-Balter 2013).

[12] **Hedging**: The term is borrowed from finance, where an offsetting position reduces risk; here, the redundant request insures against a slow server. The insurance is not free. For ML systems, the losing hedged request may still occupy accelerator time if inference has already launched, because ordinary GPU kernels are not cheaply canceled mid-execution. Thus, a hedging policy must budget duplicate work as well as the latency benefit.

Checkpoint 13.2: Queuing and SLO headroom

Latency SLOs are not enforced by *fast inference* alone; they are enforced by *headroom*.

- ☐ **Little's Law**: at a fixed stable throughput, can you use $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ to explain why a higher average in-system population implies a higher mean response time, even if per-request compute time is unchanged?
- ☐ **Utilization cliff**: can you explain why latency grows nonlinearly as utilization ρ_{serv} approaches one, and why production systems target a conservative ρ_{serv} rather than "100 percent busy"?
- ☐ **Queue behavior**: use the mean and p99 curves in figure 13.5 to explain nonlinear latency growth near saturation. Why is the marked 70 percent knee an illustrative planning choice rather than a model-derived breakpoint?
- ☐ **Capacity planning**: can you explain why a throughput number is only "real" if requests still meet the percentile latency SLO under load?
- ☐ **Admission decision**: for the isolated FIFO worker, how does increasing the arriving request's service bound change its waiting allowance? If a faster fallback fits behind the outstanding work, what quality and availability conditions distinguish admitting it from rejecting the request? Why would substituting measured mean times invalidate the deterministic bound?
- ☐ **Headroom estimate**: a stable service handles 2,000 requests per second and measures a 10 ms mean response time. Use $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ to estimate its average in-flight population. Why does this result alone not establish compliance with a 50 ms p99 SLO?

Headroom keeps the queue stable by leaving capacity idle, and adding replicas creates headroom by adding accelerators. The other way to keep utilization below the knee is to raise each replica's service rate, and for an accelerator that executes one request at a time the largest lever is to execute several at once.

13.5 Throughput Optimization

Consider the reference ResNet-50 classifier on an H100 GPU operating at batch size one. Each forward pass reads the model's weights from high-bandwidth memory to serve a single image and issues a long sequence of small kernels whose fixed launch cost is paid once per image, leaving neither weight traffic nor launch overhead shared across inputs. Processing requests continuously at 280 images/s, each pass spends 99 percent of its time on those fixed costs. Processing 32 images at once delivers 7,000 images/s, a 25× throughput improvement on identical hardware, because one kernel sequence and one weight read now serve thirty-two inputs. The fixed-cost share must not be confused with serving utilization. A single worker processing batch 1 at 280 requests/s is 100 percent busy in queueing terms ($\rho_{\text{serv}} = 1.0$) with zero spare serving capacity, yet nearly all of its time goes to work that a larger batch would share. **Batching**[13] is the primary lever for improving serving economics, but its operational constraints differ sharply between training and serving (Crankshaw et al. 2017). *Training* batches maximize throughput by grouping hundreds or thousands of samples together with no constraint on individual sample latency. *Serving* batches must balance accelerator saturation against individual request deadlines, ensuring that no early-arriving input waits excessively for a batch to form. An inference server resolves this tension through **dynamic batching**, adjusting batch size at runtime to match incoming request arrivals.

[13] **Batch**: From Old French *bache* (a quantity baked at one time), entering computing in the 1950s for jobs processed together without human interaction. The ML serving usage preserves the original trade-off: grouping requests amortizes fixed costs (kernel launch, weight movement) across multiple inputs, but an online request may wait for a batch to form.

> Definition 13.3: Dynamic batching
>
> ***Dynamic batching*** is the ML serving optimization that trades latency for throughput under stochastic arrival patterns.
>
> 1. **Significance**: By buffering requests into a batching window, the scheduler amortizes fixed overheads (L_{lat}) across multiple inputs, pushing the system away from the bandwidth-bound regime (BW) toward the compute-bound regime (R_{peak}).
> 2. **Distinction**: Unlike fixed-size serving batches, dynamic batching adjusts the realized batch size at inference time according to arriving traffic, a timeout, and a maximum-batch limit.
> 3. **Common pitfall**: A frequent misconception is that batching "always helps." In reality, there is a latency-throughput Pareto frontier: if the batching window is too large, the increased queuing delay may violate the system's SLO before the throughput gains are realized.

13.5.1 Why batching helps

A single request often leaves an accelerator's compute units idle because its operations expose too little parallel work to hide kernel launch overhead or reuse weights streamed from memory (though large spatial inputs or smaller edge accelerators may already saturate at batch one). For workloads that underuse the device at small batches, increasing the batch size improves hardware efficiency (Shen et al. 2019) by amortizing fixed costs across multiple requests and running inputs in parallel across the batch dimension.

Two fixed costs dominate at small batch sizes. Kernel launch overhead is the time for the CPU to prepare and submit work to the GPU. The number of launches depends on graph lowering and fusion rather than the layer count. For each launch, the CPU must assemble kernel parameters, copy them to GPU-accessible memory, and signal the GPU to begin execution. The cost per kernel does not depend on batch size. The reference ResNet-50 profile issues 174 kernels at 15 µs of host time each, 2.6 ms per forward pass. At batch size 1, this overhead far exceeds the actual compute time; at batch size 32, the same overhead is amortized across thirty-two images. Weight loading reads model parameters from GPU memory (VRAM) to the compute units. In a weight-streaming

approximation, batch size 1 reads the weights for one image; batch size 32 reuses that weight stream across thirty-two images, increasing weight-only operational intensity by 32×. Activation traffic, caches, and workspace determine the executed operation's full intensity.

↩ Per-kernel launch overhead, and why fusion and graph execution attack it, are established in section 7.3.7.2.

Larger batches improve hardware efficiency but inflate per-request latency. The engineering objective is to identify the operating point that maximizes sustainable throughput while satisfying the request deadline. For the reference ResNet-50 profile quantified in table 13.8 below, serving batch size 8 with a 10 ms batching window requires a mean formation-plus-inference budget of approximately 8.8 ms: an arriving request waits on average half the window (5 ms, as derived in section 13.5.4.1) and requires 3.8 ms of inference. If the service must meet a 20 ms SLO, the remaining 11.2 ms provides headroom for worker backlog and auxiliary pipeline stages, while delivering 7.5× higher service throughput than batch 1 on identical hardware. On this accelerator the batching window, not the computation, is most of the batch-8 budget. Batch size cannot grow indefinitely, and figure 13.6 shows why by modeling this relationship across batch sizes on a logarithmic axis. To isolate the planning knee under worst-case scheduling conditions, the figure assumes a strictly serialized single worker that waits out the full batching window before executing each batch, without pipelined overlap (Latency $= T_{\text{window}} + T_{\text{svc}}(B)$). Under this unpipelined assumption, batch-1 throughput is 74 requests/s rather than the 280 images/s sustained when inference runs back to back. Throughput rises almost in proportion to batch size while the fixed overhead dominates each batch, because each added image costs only 33 µs of device time. The planning knee sits where the batch's total per-image work equals the fixed part of its latency, the window plus the fixed cost, near batch 410: there latency is twice that of batch one and throughput is half its ceiling, and beyond it each doubling of the batch roughly doubles latency while throughput gains less and less. In this model throughput never declines; it saturates, so latency, not throughput, sets the largest useful batch.

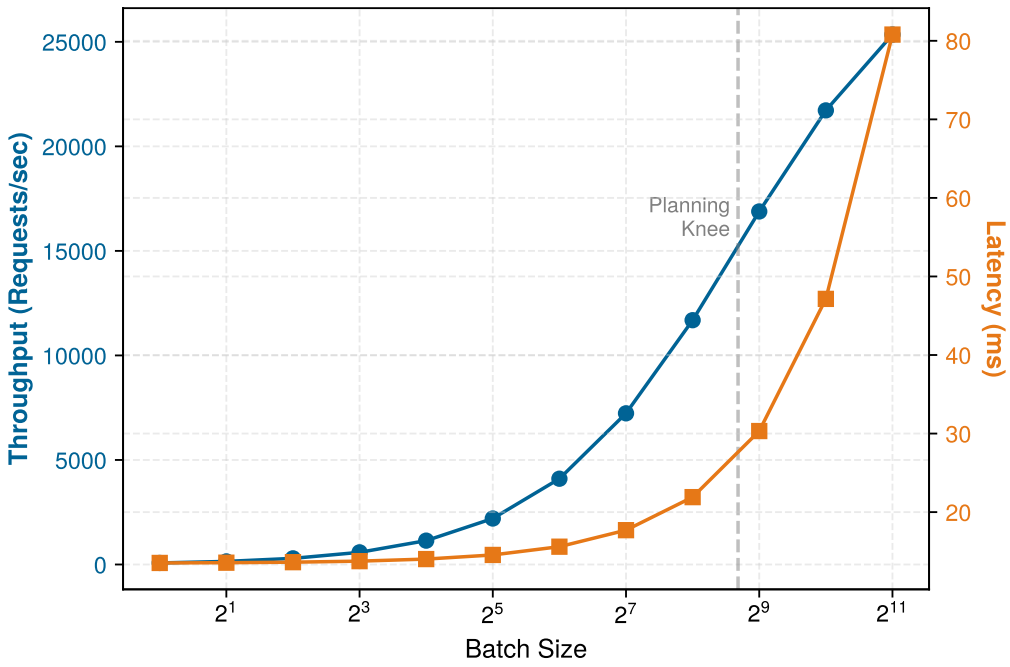


Figure 13.6: The Throughput-Latency Knee: Batch size vs. throughput (blue) and latency (orange) for the reference ResNet-50 profile on the H100. Throughput rises nearly in proportion to batch size while fixed overhead dominates, then flattens toward its ceiling. Latency grows slowly at first and roughly in proportion to batch size beyond the marked planning knee, where it is twice the batch-one latency and throughput reaches half its ceiling. Each throughput value is batch size divided by the plotted formation-plus-service latency, for a single worker that waits out a 10 ms window and then serves each batch; pipelined serving sustains higher throughput. Values are modeled, not measured.

The vertical dashed marker near batch size 410 in figure 13.6 marks the point where per-image work equals the fixed part of a batch's latency. The knee illustrates a general systems trade-off rather than a fixed operating setting. Under unpipelined serial dispatch, throughput is bounded by $B/(T_{\text{window}} + T_{\text{svc}}(B))$. In contrast, when the serving runtime pipelines batch formation with accelerator execution, saturated throughput depends solely on accelerator service time ($\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$), achieving the higher capacities documented in table 13.8.

Napkin Math 13.4: ResNet-50 batching efficiency

Table 13.8 illustrates the throughput-latency trade-off for the reference ResNet-50 profile on the H100 across batch sizes one through thirty-two. The last column is the *fixed-cost share*, the fraction of each batch's time spent on costs that do not grow with batch size: kernel issue, the per-kernel device floor, and the weight read. This share must be distinguished from the server's queueing utilization ($\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$, section 13.4.2) and its wall-clock duty cycle (equation 13.2). For example, a dedicated worker executing batch size 1 continuously operates at 100 percent queueing utilization with zero idle time between requests, yet spends 99 percent of each forward pass on fixed costs that a larger batch would share.

Math: For the batch-8 row, dividing batch latency by batch size yields per-image compute: 3.76 ms ÷ 8 images ≈ 0.47 ms per image. Dividing batch size by batch latency yields throughput: 8 images ÷ 3.76 ms ≈ 2,130 img/s. The same two relations apply to every derived row.

Table 13.8: ResNet-50 batching sweep: Modeled per-image time, throughput, and fixed-cost share across batch sizes one through thirty-two for the reference profile on the H100. Throughput grows 25× from batch one to batch thirty-two as the fixed-cost share falls from 99 percent to 77 percent, while pure inference time stretches only from 3.5 ms to 4.5 ms. The window-plus-inference column adds the full 10 ms batching window and excludes worker queueing; because the window and the fixed cost dominate it, it rises only modestly across the sweep. The fixed-cost share is distinct from server queueing utilization and wall-clock duty cycle.

Batch Size	Inference Time	Window + Inference	Per-Image Compute	Throughput	Fixed-Cost Share
1	3.53 ms	13.5 ms	3.53 ms	280 img/s	99%
4	3.63 ms	13.6 ms	0.91 ms	1,100 img/s	96%
8	3.76 ms	13.8 ms	0.47 ms	2,130 img/s	93%
16	4.02 ms	14.0 ms	0.25 ms	3,980 img/s	87%
32	4.55 ms	14.5 ms	0.14 ms	7,040 img/s	77%

The inference times exclude queue wait. The window-plus-inference column is the unpipelined worst case for a request that waits out the whole window, and section 13.5.4.1 derives the average wait a request actually sees.

Systems insight: Batch size 32 achieves 25× higher throughput than batch size 1. However, user-perceived latency includes both queue wait and inference time. With a 10 ms batching window and 4.5 ms inference, formation plus inference reaches 14.5 ms, excluding resource backlog and other request stages, vs. 3.5 ms at batch size 1.

The sweep reports seconds; the same batches look different in bytes and joules. At batch one each image carries the entire 102.4 MB FP32 weight read by itself, and at batch thirty-two it carries one thirty-second of it, while the 25 MB of activations each image writes and reads stays fixed (table 13.9). Arithmetic intensity rises from 64 FLOP/byte to 291 FLOP/byte. With the energy constants that accompanied the iron law in section 1.7, 160 pJ/byte of DRAM traffic and 2.3 pJ/FLOP of FP32 arithmetic, the energy of one classification falls from 39.2 mJ at batch one to 23.4 mJ at batch thirty-two. Data movement accounts for 52 percent of that energy at batch one and 19 percent at batch thirty-two. Past that point arithmetic dominates, and a larger batch saves almost nothing per image while still lengthening every request's wait.

Table 13.9: Memory and Compute per Image Across Batch Sizes: ResNet-50 forward pass counted in FP32 bytes and FP32 operation energy, the precision of the book's energy constants. Weight bytes divide by the batch size while activation bytes do not, so arithmetic intensity and energy per image improve until activations and arithmetic dominate. Activation traffic uses the same assumed 12.5 MB footprint, written once and read once, as the resolution estimate; time per image comes from the FP16 reference profile of table 13.8.

Batch Size	Weight Bytes per Image	Activation Bytes per Image	Arithmetic Intensity	Time per Image (Sweep)	Energy per Image	Movement Share
1	102.4 MB	25 MB	64 FLOP/byte	3.53 ms	39.2 mJ	52%
8	12.8 MB	25 MB	217 FLOP/byte	0.47 ms	24.9 mJ	24%
32	3.2 MB	25 MB	291 FLOP/byte	0.14 ms	23.4 mJ	19%

13.5.2 Formation delay and the latency-throughput frontier

The throughput gains demonstrated in table 13.8 incur a deliberate systems cost. Every request in a batch pays a latency penalty, an engineered queueing delay introduced to elevate hardware utilization. The performance gain arises from parameter reuse across inputs. Evaluating B independent queries simultaneously converts memory-bound matrix-vector products into compute-dense general matrix multiplications (GEMM), raising arithmetic intensity toward the hardware roofline ceiling (Principle 6). Conversely, the latency penalty accumulates in the queue, because assembling B distinct queries forces earlier arrivals to sit idle while waiting for subsequent requests to appear.

Formalizing this trade-off requires extending the single-server queuing notation. Let $T_{\text{svc}}(B)$ denote the execution duration for a batch of size B. The unbatched service rate is $\mu = \mu_{\text{eff}}(1) = 1/T_{\text{svc}}(1)$, while batching B inputs yields an effective aggregate throughput of $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$. In a request-triggered fixed-batching system, end-to-end response time decomposes into two distinct penalties: **batch formation delay** ($L_{\text{lat,form}}$), the time earlier requests wait while the batch accumulates to capacity, and **inference inflation**, the modest increase in kernel execution duration $T_{\text{svc}}(B)$ as batch size expands from 1 to B. Under stationary Poisson arrivals at rate λ_{arr}, the first request in a batch waits an average of $(B-1)/\lambda_{\text{arr}}$ seconds while the final arrival waits zero, yielding an average formation delay of $(B-1)/(2\lambda_{\text{arr}})$. Assuming an execution worker is immediately available upon batch assembly, equation 13.6 models the resulting latency-throughput Pareto frontier:

$$L_{\text{lat,batch}} \approx \underbrace{\frac{B-1}{2\lambda_{\text{arr}}}}_{\text{Formation delay}} + \underbrace{T_{\text{svc}}(B)}_{\text{Inference time}} \tag{13.6}$$

Equation 13.6 formalizes the component trade-off between accelerator throughput and individual request latency, assuming an idle worker is immediately available upon batch assembly. Increasing B raises arithmetic intensity and amortizes memory bandwidth across inputs, but introduces formation delay for early arrivals. Evaluated as a formation-plus-service component comparison using the service times from table 13.8 at 500 QPS, moving from batch 1 to batch 32 increases formation wait from 0 ms to 31 ms, a 10× increase in isolated stage latency (3.5 ms to 36 ms). However, batch-one service cannot sustain 500 QPS on a single worker whose peak service rate is only 280 requests/s; without batching, worker backlog diverges toward infinity, so 3.5 ms does not represent a viable steady-state response time at this arrival rate. Batching is what makes 500 QPS feasible on that hardware in the first place, demonstrating that grouping requests can lower loaded end-to-end response time even while adding formation delay. Sizing B is therefore an operational stability screen before it is an economic optimization: operators first enforce queue stability ($\lambda_{\text{arr}} < \mu_{\text{eff}}(B)$) and then select the batch size that maximizes throughput within the memory ceiling and latency SLO ($L_{\text{lat,SLO}}$). Because equation 13.6 assumes the batcher waits for a full count of B requests before launching, how serving systems relax that count-only assumption when traffic fluctuates separates the two batching policies below.

13.5.3 Static vs. dynamic batching

If an inference server holds execution until its queue accumulates a fixed batch of B requests, a lull in traffic can leave early arrivals waiting indefinitely until enough subsequent queries arrive to fill

the batch. **Static batching** (where *static* describes the batch dimension rather than the precomputed predictions of static inference) fixes the batch size at compile time so compilers can allocate static memory buffers and emit specialized compute kernels—a natural fit for offline processing and model training, where inputs are already staged, but a liability when interactive requests arrive stochastically. Conversely, setting a small fixed batch size to cap waiting time leaves accelerator compute units underutilized and fails to amortize weight transfers from high-bandwidth memory when traffic spikes.

Dynamic batching decouples batch formation from arrival determinism by buffering incoming requests behind a dual-trigger scheduler (Olston et al. 2017; NVIDIA 2024d). Rather than waiting indefinitely for a fixed tensor shape, the serving runtime makes a batch eligible for dispatch when either of two thresholds is satisfied: the queue accumulates a maximum batch size B_{max}, or a timeout window T_{window} expires relative to the oldest buffered request. When traffic bursts, the queue fills to B_{max} almost instantaneously, achieving high arithmetic intensity with minimal queuing delay. When arrival rates drop, the timeout window forces the dispatch of an undersized batch ($B < B_{\text{max}}$), bounding formation time at the expense of per-request compute efficiency. If the accelerator is busy, the ready batch can still wait for service. The timeout window T_{window} and maximum batch size B_{max} form the primary operational levers. Shorter windows limit formation delay, whereas longer windows allow larger batches when enough requests arrive. A longer window is useful only when the resulting throughput gain remains within the latency and memory constraints.

13.5.4 Dynamic batching latency-throughput trade-offs

Setting those two levers (T_{window} and B_{max}) requires accounting for what happens to the latency budget while the batcher waits: a wide window can spend a large share of the SLO accumulating requests before a single kernel launches. Because latency under load grows in the queue before it grows on the accelerator (section 13.4.4), configuring a dynamic batcher starts by separating time spent waiting from time spent in service.

Equation 13.7 decomposes the modeled batching-and-service portion of request latency into two components. Its total excludes network transit and any host stages outside the service interval:

$$L_{\text{lat,batch}} = L_{\text{lat,wait}} + T_{\text{svc}}(B) \tag{13.7}$$

where $L_{\text{lat,wait}}$ is the time spent waiting in the batching queue (corresponding to $L_{\text{lat,queue}}$ in the latency budget of principle 12) and $T_{\text{svc}}(B)$ is the service time for a batch of size B (encompassing $L_{\text{lat,infer}}$ plus portions of $L_{\text{lat,pre}}$ and $L_{\text{lat,post}}$). This wait includes formation and any subsequent wait for a worker. The batching window T_{window} bounds formation time alone; $L_{\text{lat,wait}} \leq T_{\text{window}}$ holds only when no additional resource backlog delays dispatch. Batch size affects service time through weight reuse, parallel work, and execution overhead.

13.5.4.1 Quantitative analysis of batching

Configuring a dynamic batching policy requires balancing batch formation delay against accelerator execution time and pending queue backlog. In a pure count-triggered batcher (equation 13.6), average formation delay $(B-1)/(2\lambda_{\text{arr}})$ contracts as arrival rate λ_{arr} grows, but diverges under light traffic. Conversely, a pure timer-triggered batcher dispatches on a periodic schedule of length T_{window}, decoupling formation delay from instantaneous arrival fluctuations.

To evaluate how time-based batching impacts latency budgets, consider a baseline fixed calendar window where batches dispatch periodically every T_{window} seconds and an idle execution worker is available at the boundary. Under stationary Poisson arrivals, query arrival times distribute uniformly across the interval $[0, T_{\text{window}}]$. A request arriving at offset t within the window experiences a formation wait of $T_{\text{window}} - t$. Integrating across the uniform distribution yields an expected waiting time equal to exactly half the batching window (equation 13.8):

$$\mathbb{E}[L_{\text{lat,wait}}] = \frac{T_{\text{window}}}{2} \tag{13.8}$$

A 20 ms batching window adds 10 ms average wait (up to 20 ms for the first arrival in a window; later arrivals wait less) regardless of batch size achieved. For a 50 ms mean latency SLO with 3.5 ms inference, the average wait consumes 20 percent of the latency budget before any computation begins, about 3× the forward pass itself; tail SLOs must budget the full window.

13.5.4.2 Batch size distribution

For fixed calendar windows, the number of requests collected during T_{window} follows a Poisson distribution with mean $\lambda_{\text{arr}} T_{\text{window}}$. Equation 13.9 formalizes this relationship:

$$\Pr(\text{batch size} = k) = \frac{(\lambda_{\text{arr}} T_{\text{window}})^k e^{-\lambda_{\text{arr}} T_{\text{window}}}}{k!} \tag{13.9}$$

where k is a nonnegative integer count of arrivals in the window.

With a fixed 10 ms window, the count a window collects fluctuates most at low traffic. At 50 QPS the mean batch is 0.5 with a standard deviation of 0.7, and 61 percent of windows contain no request at all, so no GPU batch launches. At 200 QPS the mean is 2 and the standard deviation 1.4, so realized batches still swing widely around the mean. At 1000 QPS the mean reaches 10 and the spread narrows relative to it, so the probability that a window collects at least twice the mean falls from 39 percent at the lowest traffic level to 0.3 percent at the highest, the law of large numbers at work.

13.5.4.3 Throughput maximization strategy

Throughput optimization requires separating per-request latency from saturated service capacity. A request waits for batch formation, then pays the service time of the formed batch. Under sustained load, however, batch formation can overlap with the previous batch's execution, so capacity is governed by the ready-batch service time:

$$\mu_{\text{eff}}(B) \approx \frac{B}{T_{\text{svc}}(B)}, \qquad \text{throughput} = \min(\lambda_{\text{arr}}, \mu_{\text{eff}}(B)) \tag{13.10}$$

In equation 13.10, λ_{arr} is the offered arrival rate and $\mu_{\text{eff}}(B)$ is the saturated service capacity for batch size B. The numerator increases linearly with batch size while service time often increases sublinearly over a useful range because larger batches amortize fixed kernel-launch overheads and DRAM weight transfers, shifting execution from memory-bandwidth-bound matrix-vector operations toward compute-bound matrix-matrix multiplication. The batching window still appears in request latency and in low-traffic regimes, where the expected batch size is limited by arrivals during the window, roughly $\lambda_{\text{arr}} T_{\text{window}}$.

To analytically evaluate batching strategies across arbitrary batch sizes without interpolating discrete benchmark tables, the reference ResNet-50 profile is affine in batch size, $T_{\text{svc}}(B) = 3.50\ \text{ms} + (0.033\ \text{ms})\,B$: 3.50 ms of fixed kernel issue, per-kernel device floor, and weight read, plus 0.033 ms of device work per image. It is the same model that produced the batching sweep (table 13.8), so the sweep and the model describe one service. This analytical service model unifies the queueing, window screening, and dispatch calculations throughout the remainder of this section. The sweep's window-plus-inference column already evaluates it for a 10 ms batching window T_{window}, pairing formation-plus-service latency with saturated pipelined capacity. The sweep's throughput gains trace directly back to the fixed-overhead term L_{lat} of the iron law (principle 3), which batching amortizes across requests.

The sweep, the formation wait, and the arrival statistics compose a calendar-window planning model. Average formation wait is half the window, Poisson arrivals make the window count fluctuate around $\lambda_{\text{arr}} T_{\text{window}}$, and saturated service capacity $\mu_{\text{eff}}(B)$ grows as batching amortizes fixed costs. An oldest-request timer, a maximum-batch trigger, or a busy worker changes the observed wait and batch distributions. Mean-latency screening can narrow candidate configurations; a percentile SLO requires their tail behavior under the intended dispatch policy and load.

13.5.4.4 Latency-constrained optimization

When latency is the binding constraint, a mean-latency approximation provides an initial screen of candidate batch sizes. For the portion of a mean latency SLO allocated to batching and service, $L_{\text{lat,budget}}$, and average wait time $T_{\text{window}}/2$, equation 13.11 defines the maximum allowable batch size B_{SLO} using a first-order average latency approximation. Other request stages require separate budget allocations. This is a feasibility bound, not a throughput optimum, and it screens a mean budget rather than certifying a p95 or p99 target:

$$B_{\text{SLO}} = \max\left\{B : \frac{T_{\text{window}}}{2} + T_{\text{svc}}(B) \le L_{\text{lat,budget}}\right\} \tag{13.11}$$

Applying equation 13.11 to the affine service model for a 50 ms mean budget shows how the choice of batching window shapes the feasible configuration space. A conservative 5 ms window adds 2.5 ms of average formation wait (up to the full 5 ms for the first request in a window) and leaves 47.5 ms for inference, enough by equation 13.11 for a batch of about 1,340 images. The configured engine maximum batch of 128, an assumed setting, binds first, so the batch is capped at 128 images and the worker sustains about 16,600 img/s when saturated. An aggressive 25 ms window adds 12.5 ms of average wait and leaves 37.5 ms, still enough for about 1,030 images, so the same engine cap binds and saturated capacity does not change. A tail SLO, unlike this mean-latency plan, must budget the full window rather than its average.

This comparison quantifies the diminishing returns of expanding the batching window. Widening the window by 20 ms buys no saturated capacity while increasing average formation wait by 10 ms. On an accelerator this fast relative to its fixed cost, the latency budget stops being the binding constraint on batch size; the engine's maximum batch and the arrival rate are. However, this initial screen relies on mean formation wait and deterministic batch sizing. In production, stochastic arrivals introduce batch-size variance that can breach tail deadlines even when the mean latency appears well within the budget, requiring the percentile envelope analysis developed next.

13.5.4.5 SLO violation analysis

Batch size variability can cause SLO violations even when mean latency appears safe. A request in a fixed window sees itself plus the other arrivals, so $B_{\text{req}} = 1 + \text{Poisson}(\lambda_{\text{arr}} T_{\text{window}})$. With an available worker at dispatch, combining the full window with the request-seen p99 batch size gives the conservative formation-plus-service envelope in equation 13.12:

$$L_{\text{lat,envelope}} = T_{\text{window}} + T_{\text{svc}}(B_{\text{req,p99}}) \tag{13.12}$$

For λ_{arr} = 500 QPS and T_{window} = 10 ms, the mean request-seen batch size is 6 and its p99 is 12. The mean latency is 8.7 ms; the conservative envelope is 13.9 ms.

The envelope is 1.6× the mean, and a configuration whose mean sits comfortably inside the SLO can still breach it when a full window coincides with an unusually large batch. Under the available-worker calendar-window assumptions, the envelope conservatively screens formation plus service because the longest window wait and largest batch need not hit the same request. A passing configuration clears only this screen; acceptance still requires evaluating worker backlog and the other request stages under the intended traffic and dispatch policy. A failing configuration can be rejected conservatively or investigated with a more faithful model and load test. Neither outcome alone establishes compliance with the end-to-end SLO.

13.5.4.6 Adaptive batching windows

Static batching windows impose a rigid trade-off that penalizes both traffic extremes. Under heavy traffic surges, a static timeout window holds early arrivals unnecessarily long after sufficient queries have already accumulated, consuming latency budget without enlarging the batch. Conversely, during off-peak periods, a fixed window frequently dispatches tiny, inefficient batches, failing to exploit accelerator arithmetic intensity. To navigate this tension, serving runtimes employ **adaptive batching** policies that dynamically modulate window duration based on instantaneous queue backlog, observed arrival rates, and the residual latency margin before the service-level objective expires. An adaptive window contracts during surges and expands during lulls, so it can lower average latency and raise realized batch size at the same time, within the remaining latency budget. Listing 13.2 implements one such heuristic.

Listing 13.2: Adaptive Batching Window: Heuristically adjusts batch timeout based on queue depth, arrival rate, and the remaining latency budget.

```
def adaptive_batching_window(
    queue_depth, arrival_rate, slo_ms, service_ms, fixed_overhead_ms
):
    """Compute a heuristic batching window.

    Based on current system state.
    """
    target_batch_size = 16  # Deployment-tuned target batch

    # Fast path: batch ready, close immediately to minimize latency
    if queue_depth >= target_batch_size:
        return 0

    # Compute maximum allowable wait from the remaining p99 budget.
    max_wait_ms = max(0, slo_ms - service_ms - fixed_overhead_ms)

    # Estimate time to accumulate target batch at current arrival
    # rate.
    # arrival_rate is requests/second, so convert seconds to
    # milliseconds.
    if arrival_rate > 0:
        requests_needed = target_batch_size - queue_depth
        estimated_wait_ms = requests_needed / arrival_rate * 1000.0
        # Return minimum of estimated wait and SLO-constrained maximum
        return min(estimated_wait_ms, max_wait_ms)

    return max_wait_ms  # Low traffic: use remaining budget to accumulate batch
```

The heuristic selects a candidate formation timer. Its inputs omit time already spent by the oldest request and work queued ahead of the batch, so the returned window does not establish a remaining deadline allowance. Deadline-aware dispatch must account for both quantities and validate the service and outside-worker budgets under load.

The nondominated batching configurations form a Pareto frontier where improving throughput requires accepting higher latency. Across five window and batch-cap configurations on the reference profile, from a 2 ms window capped at batch 8 to a 50 ms window capped at batch 128, modeled throughput rises from 2,100 img/s to 16,600 img/s, a gain of 680 percent, while the modeled p99 envelope rises from 5.8 ms to 57.7 ms, 10× its initial value. Average latency over the same range grows from 4.8 ms to 32.7 ms.

13.5.4.7 Practical configuration guidelines

The frontier, like the knee of figure 13.6, shows why configuration must work backward from the latency budget. Past the knee, widening the window buys diminishing throughput for sharply rising tail latency. In the worked scenario, reserving 30 percent of the SLO for batching wait gives $T_{\text{window}} \leq 0.3 \times L_{\text{lat,SLO}}$; another workload may allocate a different fraction after accounting for network, preprocessing, inference, and postprocessing. The traffic estimate should include peaks and burst duration rather than rely only on the mean. GPU memory supplies a separate ceiling on batch size through activations and runtime workspace. Finally, monitoring the realized batch-size and wait-time distributions reveals whether the assumed arrival model holds and whether a fixed or adaptive window is appropriate.

For an illustrative ResNet-50 serving system with a 50 ms SLO and 500 QPS traffic, the calculation provides starting estimates for two configuration knobs: the batching window and maximum batch size. Table 13.10 summarizes the resulting configuration and modeled operating point. The batch statistics and tail envelope use the calendar-window model without worker backlog; an oldest-request timer and maximum-batch trigger require a load test of their actual distributions. Its saturated-capacity row derates the ideal batch-32 rate by an assumed efficiency factor of 0.89, standing in for scheduling gaps and batches that dispatch before they fill.

Table 13.10: Practical Batching Configuration: A 30 percent allocation sets a 15 ms maximum window; this illustrative scenario selects a 12 ms window below that bound and assumes a batch-32 cap. The reported latency is a conservative envelope, not the request-latency p99.

Quantity	Value	Engineering role
Latency budget for batching	15 ms	Portion allocated to batch formation.
Maximum window	15 ms	Upper bound implied by the latency budget.
Expected request-seen batch	7	Average batch seen by an arriving request.
Request-seen p99 batch	13	Tail batch size under Poisson arrivals.
Assumed memory-limited batch	32	Scenario cap for accelerator memory.
Selected configuration	$T_{\text{window}} = 12$ ms, $B_{\max} = 32$	Candidate knob setting for load testing.
Conservative tail envelope	15.9 ms	Full window plus request-seen p99 batch service time.
Illustrative saturated capacity	6,300 img/s	Ideal batch rate derated by the efficiency factor.
Offered throughput	500 img/s	Offered load; acceptance requires load validation.



Example 13.4: From a batching window to a dispatch decision

Problem: In a hypothetical trace of the selected dual-trigger configuration, four requests arrive at 0, 2, 5, and 9 ms. No further requests arrive before dispatch. The worker becomes available at 18 ms, and the remaining response path takes 4 ms. Does the oldest request meet the 50 ms deadline under the planning service-time model?

Analysis: The batch remains below the maximum count, so the oldest-request timer makes it eligible after 12 ms. The worker is still busy, adding 6 ms of resource-queue wait. The four-request batch then takes 3.6 ms under the same linear service model used for the window analysis. Adding the remaining response path gives 25.6 ms from the oldest arrival to completion.

Result: Dispatching the four-request batch meets the oldest request's deadline with 24.4 ms of slack; the later arrivals finish with more slack. Waiting for the maximum batch would leave these requests stalled because the trace supplies no further arrivals. The timeout therefore determines eligibility, while worker availability determines actual dispatch.

Systems insight: The selected window passes this complete trace, including backlog and the response path. Establishing a percentile SLO requires repeating the accounting over the intended traffic, service-time variation, and input mix, with rejected requests and fallback quality reported alongside completed-request latency.

Working backward from the budget can also conclude that the right window is zero, because schedulers and dynamic memory allocators add coordination and synchronization overhead that hurts latency-critical or resource-constrained deployments. Under an ultra-low p99 target, even a brief formation delay consumes an unacceptable fraction of the latency budget, while in runtimes without ragged (variable-length) tensor support, batching variable-length inputs pads every sequence to the longest item in the batch—waste that single-request execution avoids completely. Memory capacity can rule out batching as well: when model weights already occupy most of the accelerator's physical memory, concurrent batch activations and dynamic KV-cache allocations risk out-of-memory aborts, forcing the runtime to serve one request at a time.

13.5.5 Traffic patterns and batching strategy

A timeout window tuned for independent web API calls will stall a multi-camera robotics rig whose frames arrive in synchronized bursts, and it is useless on a smartphone that processes one user photo at a time. Because the right batching policy depends on how requests arrive, the MLPerf inference benchmark supplies four standardized scenarios that approximate common production arrival patterns.

13.5.5.1 Server traffic (Poisson arrivals)

The MLPerf Server scenario models cloud/API-like inference traffic with Poisson arrivals (Reddi et al. 2019), the arrival model behind the window statistics of section 13.5.4.2. A fixed calendar window collects $\lambda_{\text{arr}} T_{\text{window}}$ requests on average, and that count fluctuates most, relative to its mean, at low traffic. Higher traffic can fill a desired batch sooner, permitting a shorter formation window. An oldest-request timer also includes the request that starts the window. Increasing the arrival rate therefore allows the same expected batch count with less deliberate waiting. The scheduler may instead target a larger batch if the latency and memory budgets permit it. Neither choice creates service capacity. The formed batches must complete fast enough to keep up with sustained arrivals, or resource backlog grows despite the shorter window. Window tuning must jointly check the arrival count, the service rate at the realized batch sizes, and the full request-latency distribution.

13.5.5.2 Streaming traffic (correlated arrivals)

Video analytics, robotics, and other multi-sensor systems receive inputs from several synchronized sensors, so arrivals are correlated rather than independent. The batch size equals the number of sensors and the deadline equals the frame period, both fixed outside the server, and the binding quantity becomes arrival jitter. Inference cannot start until the last frame of a set lands, so jitter consumes the frame budget before any computation runs. Such pipelines bound latency with an explicit synchronization deadline, the latest batch start that still meets the frame deadline, and a fallback, such as reusing a sensor's previous frame, when a frame misses it. The MultiStream scenario (section 12.4.4.1) approximates this shape for benchmarking.

13.5.5.3 Single-user traffic (sequential arrivals)

Synchronized sensors fix batch size and deadline outside the server. At the opposite end of the spectrum, interactive mobile and edge applications process one request stream at a time, leaving no opportunity to batch independent queries. The MLPerf SingleStream scenario captures this sequential-serving shape. Here, the optimization target shifts from a synchronization budget under a frame deadline to per-request latency and energy consumption under a rigid thermal envelope. For the smartphone vision pipeline modeled below, the primary engineering objective is not batch formation, but minimizing per-phase latency and energy dissipation.

13.5.5.4 Mobile serving constraints

Cloud services often emphasize fleet cost and latency; mobile serving adds three closely coupled constraints. The first is an energy budget that throughput alone does not capture, because each inference consumes battery. In the modeled pipeline, 2.98 mJ at 22 FPS draws about 66 mW for the inference path alone, before camera, display, ISP, and OS overhead add to that total in a full photo app. Thermal throttling can compound this limit when sustained system power reaches a device-specific thermal boundary. Memory constraints close the set because the model shares RAM with the application and operating system. Reload time depends on device storage, runtime, and model state; quantization and memory-mapped loading (section 13.8.2) can reduce footprint and startup work but do not eliminate those dependencies.

Example 13.5: ResNet-50: Mobile serving

Table 13.11 decomposes per-phase latency and energy for a single-user mobile vision inference:

Table 13.11: Mobile ResNet-50 pipeline: Illustrative per-phase latency and energy for a single-user mobile vision inference, showing that JPEG decode on the CPU dominates the energy budget even though the NPU inference stage carries the model's compute. Optimization targets shift from throughput to energy-per-inference at the edge.

Phase	Duration	Energy	Notes
Camera buffer read	8 ms	0.08 mJ	System API
JPEG decode (CPU)	15 ms	1.5 mJ	Single-threaded
Resize + Normalize	5 ms	0.4 mJ	CPU preprocessing
NPU inference	12 ms	0.8 mJ	82% utilization
Postprocess + UI	5 ms	0.2 mJ	Result rendering
Total	**45 ms**	**2.98 mJ**	22 FPS sustained

The mobile serving node is governed by four metrics:

- **Energy per inference**: 2.98 mJ enables ~12.1M inferences per 10 Wh battery (typical smartphone)
- **Modeled inference power**: 2.98 mJ / 45 ms = 66 mW, excluding the rest of the device and therefore not a guarantee of indefinite operation
- **NPU vs. CPU trade-off**: CPU fallback replaces the 12 ms, 0.8 mJ NPU inference stage with a 45 ms, 4.2 mJ CPU stage; the full pipeline would rise from 45 ms and 2.98 mJ to about 78 ms and 6.4 mJ before additional system overhead.
- **Memory footprint**: 150 MB peak (model + activations), competing with app memory

Systems insight: In this illustrative pipeline, the mobile NPU reaches 82 percent utilization at batch size one while the chapter's batch-one ResNet-50 request keeps the H100 busy only 41 percent of the time even before its tiny per-image arithmetic is counted. The contrast is workload- and hardware-specific, but it shows why a mobile accelerator designed for single-stream execution can operate efficiently without the batches used to amortize a large server GPU.

These physical constraints make edge serving optimization fundamentally different from data-center optimization. Rather than maximizing aggregate batch throughput, the objective is to sustain real-time execution within a tight thermal and battery envelope without triggering hardware throttling or starving foreground applications of system memory.

13.5.5.5 Traffic pattern summary

Traffic-adaptive batching adjusts the batching window as queue depth and request rate change. The four MLPerf inference scenarios that section 12.4.4.1 defines map onto these traffic shapes and the batching strategy each favors (table 13.12). These scenarios are benchmark abstractions rather than deployment prescriptions, so the operative batching policy must still follow the service's current concurrency, deadline, and power constraints.

Table 13.12: Traffic Patterns and Batching Strategies: The four MLPerf inference scenarios approximate distinct deployment contexts. Server traffic (cloud APIs) uses dynamic batching with a timeout; MultiStream evaluates grouped queries under its version-specific latency rules, while a production sensor deadline requires separate validation; SingleStream processes requests individually; Offline uses large batches subject to memory capacity to maximize throughput.

Scenario	Context	Strategy	Focus
Server	Cloud APIs, web services	Dynamic batching with timeout	Window tuning, utilization-latency curve
MultiStream	Grouped sensor-like inputs	Joint query processing	Group latency under benchmark rules

Scenario	Context	Strategy	Focus
SingleStream	Mobile apps, embedded devices	No batching ($B = 1$)	Preprocessing, power efficiency
Offline	Batch processing, data pipelines	Maximum batch size	Throughput, hardware utilization

The examples developed here (cloud API traffic under the Server scenario, grouped sensor inputs motivated by MultiStream, and throughput-dominated pipelines under Offline inference) all decide how requests share the accelerator. Together with the latency budget and the queueing model, they complete one replica of the ResNet-50 service, which is where the chapter's first half ends. None of them decides how efficiently the accelerator executes each batch once it forms. The same batch of the same model can run several times faster or slower depending on the engine that compiles and schedules its kernels and on the number format it computes in, and those choices are the next lever.

Checkpoint 13.3: Batching and traffic patterns

Batching is the primary lever for serving economics, but the optimal strategy depends on context.

- ☐ **Throughput-latency trade-off**: Explain why batch size 32 achieves 25× higher throughput than batch size 1, yet a production system with a 20 ms SLO might still choose batch size 8.
- ☐ **Wait vs. compute**: Given an end-to-end latency budget, can you separate batch formation, waiting for an available worker, and batch service time $T_{\text{svc}}(B)$? Which wait does a batching timeout control?
- ☐ **Dynamic vs. static batching**: Describe why static batching fails under variable traffic arrival rates, and how dynamic batching with an adaptive time window resolves queue starvation.
- ☐ **Traffic pattern matching**: Map deployment scenarios (cloud API, synchronized multi-camera stream, mobile app) to corresponding MLPerf scenarios, identifying the batching strategy suited to each operational constraint.
- ☐ **Adaptive windows**: For a desired expected batch count, explain why higher traffic permits a shorter formation window. How would you check whether the realized batches can keep up with arrivals, and what must a load test verify before accepting the configuration?
- ☐ **When not to batch**: Name a latency target, runtime limitation, or memory condition under which a batching window of zero is the right configuration.
- ☐ **Bytes and joules per image**: Using table 13.9, explain why energy per image falls steeply from batch 1 to batch 8 but only slightly from batch 8 to batch 32.
- ☐ **The first half in one request**: For one ResNet-50 request arriving at 500 QPS, name the stage that sets its latency floor, the quantity that sizes the queue's headroom, and the two things the scheduler's batch changes, and explain how each depends on the one before it.

13.6 Inference Runtime Selection

Running an unchanged ResNet-50 model through a compiled C++ engine rather than an eager Python interpreter cuts batch-one latency on the reference H100 by more than fivefold, without altering a single weight. Dynamic batching and request scheduling govern how requests are grouped and queued, but an execution engine still has to map the resulting tensor operations onto physical execution units and memory hierarchies. The **inference runtime**, the software layer that schedules tensor operations and manages device resources, therefore changes both latency and throughput for a fixed model architecture. Runtime engineering proceeds in two steps: selection chooses the execution engine that fits the deployment target, and configuration tunes that engine for the model, hardware, input shapes, and latency budget.

13.6.1 Runtime constraint map

Attempting to deploy a Python-based server runtime onto a 256 KB microcontroller fails before the first request arrives, and exporting a model with custom operators to a mobile neural processing unit (NPU) delegate can silently push unsupported layers back onto the CPU. The deployment target narrows the viable runtime family before any speed comparison matters, and each tier in table 13.2 rules out entire classes of software. The keyword-spotting microcontroller in section 7.3.8 requires a fixed memory arena and a compact C/C++ runtime, whereas a smartphone can host an application runtime that routes supported operators to accelerator delegates and falls back to another path when an operator is missing. Within each target envelope, table 13.13 lists the candidate runtime families and the conversion or execution condition each candidate must pass before latency or memory numbers can be compared.

Table 13.13: Runtime Constraint Map: Read each row left to right as one trade, in which a family gives up flexibility, gains an optimization path, and acquires a new way to fail. The middle column records where a family sits relative to an untuned eager baseline rather than a measured result, so latency, memory use, energy, and utilization still have to be measured for the specific model, hardware, precision, batch size, and compiler configuration.

Runtime family	Where it fits	Constraint anchor	Optimization path	Constraint to test
PyTorch eager	Research and iteration	Baseline latency; full Python/runtime footprint	Dynamic graphs, eager debugging	Dispatch overhead and missing graph view
Compiled PyTorch/TensorFlow	Server training and serving	Workload-dependent gains when graph capture is clean	Graph capture, fusion, layout planning	Operator coverage and graph breaks
TensorFlow Lite/Core ML	Mobile and edge inference	Target-specific mobile latency and package-size budgets	Quantization, static graphs, NPU delegates	Target-specific conversion constraints
TF Lite Micro/microTVM	Microcontroller inference	Target-specific microcontroller RAM and flash budgets	Static allocation, INT8 kernels	Selected operators and constrained memory
ONNX Runtime	Cross-framework serving	Backend-dependent; can match native runtimes for supported graphs	Standard graph format, execution providers	Export gaps and custom-operator fallbacks
TensorRT/TVM	Hardware-specialized inference	Target-dependent gains over untuned eager baselines	Kernel fusion, precision lowering, autotuning	Narrower target and conversion assumptions

A runtime can fit the device and still fail to execute the exported model. Specialized inference runtimes such as TensorRT and Apache TVM require compatible operators and conversion paths; a mobile delegate may support only part of a graph. Unsupported regions can introduce fallback execution and extra transfers. Portability therefore requires an agreement about the graph and operator semantics on both sides of the export boundary.

13.6.2 Runtime ecosystem and configuration

Serving a model directly from the Python interpreter used to train it is the fastest path to a working endpoint, but under production load the interpreter itself becomes the bottleneck: threads contend for the Global Interpreter Lock (GIL), operators launch one by one without whole-graph fusion, and dynamic allocations fragment memory. Engine selection therefore starts with the binding system constraint rather than the training framework. When rapid iteration and full support for dynamic Python control flow or custom operators matter most, PyTorch and TensorFlow models can serve directly through their native runtimes. When interpreter overhead limits throughput, serialized graph formats such as `torch.export` and TensorFlow SavedModel (with TorchScript supporting legacy deployments) capture a static operator graph for ahead-of-time optimization and multi-threaded C++ execution. Framework-native runtimes prioritize developer flexibility and broad operator coverage, whereas specialized inference engines trade that generality for whole-graph rewrites and lower launch overhead.

13.6.2.1 General-purpose optimization

The development framework and deployment runtime need not be the same system, but they must exchange a representation both can interpret. The **ONNX** format addresses this fragmentation by

standardizing model exchange across runtimes (ONNX Contributors 2019). A pipeline can train an architecture in PyTorch, export the computation graph to an ONNX intermediate representation, and execute the serialized graph in ONNX Runtime or compile it for a target accelerator. In contrast, TensorFlow Lite relies on its own conversion path, translating TensorFlow SavedModels into FlatBuffers rather than routing through ONNX. Although standard interchange formats reduce manual model conversion between development and production environments, they do not eliminate compatibility testing, operator coverage gaps, or custom-kernel fallback overhead.

The exchange is hub-and-spoke around the common ONNX intermediate representation (figure 13.7). Training frameworks such as PyTorch, TensorFlow, Keras, Caffe2, MXNet, and CNTK export inward to the shared representation, and ONNX Runtime and other target frameworks load the serialized computation graph outward without reimplementing model code.

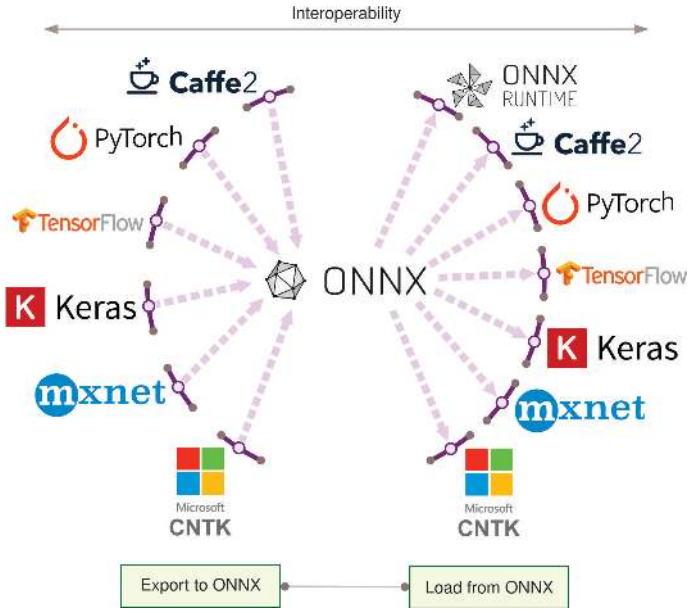


Figure 13.7: Framework Interoperability: The inward spokes represent exports into the shared ONNX format, while the outward spokes represent loads into compatible tools and runtimes; portability requires support on both sides of that boundary.

Interchange formats mitigate framework fragmentation, but an export boundary cannot salvage an architecture that violates the target's physical limits. Converting a graph to ONNX or an embedded runtime after development completes cannot resolve an excessive memory footprint, missing hardware operators, or unsupported dynamic control flow. Treating deployment as an afterthought guarantees costly rewrites, which is why hardware and runtime trade-offs need evaluating before the first line of model code is written.

When cross-hardware portability is the binding constraint, ONNX Runtime[14] provides an intermediate, hardware-agnostic execution layer (Microsoft 2024). Models export to the ONNX format, after which ONNX Runtime applies target-independent graph optimizations and delegates subgraphs to hardware-specific execution providers. This architecture enables single-format deployment across CPUs, GPUs, and custom accelerator application-specific integrated circuits (ASICs).

Portability still demands profiling operator placement across execution boundaries. If an operation is unsupported by the preferred execution provider, placement depends on the configured provider order and available implementations. A CPU fallback can introduce host-accelerator transfers and synchronization that erode the expected latency advantage; if no provider supports the operation, execution may fail. Profiling must therefore verify provider assignment and operator coverage across the entire graph, not merely confirm that the exported artifact executes without errors.

14 **ONNX Runtime**: Microsoft's inference engine applies graph optimizations such as constant folding, redundant-node elimination, and kernel fusion before handing subgraphs to execution providers. Performance relative to a tuned target-specific engine such as TensorRT depends on the workload, execution provider, and hardware.

13.6.2.2 Specialized inference engines

When latency budgets or serving costs bind more tightly than portability, specialized engines such as TensorRT for NVIDIA GPUs and OpenVINO[15] for Intel hardware optimize graphs specifically for target microarchitectures (NVIDIA 2024c; Intel Corporation 2026b; Chen et al. 2018). Rather than maintaining portable abstractions, these compilers specialize kernels directly to the hardware's register files, cache hierarchies, and matrix execution units. A specialized engine such as TensorRT or TVM offers no advantage unless its backend supports the target and its conversion path supports the model (NVIDIA 2024c; Chen et al. 2018), and ONNX reduces the exchange burden without proving either condition (ONNX Contributors 2019).

Layer fusion[16] is kernel fusion (also called operator fusion) applied when the engine is built. For the convolution → batch normalization → ReLU block, whose folding section 10.4.1 derives, the serving compiler performs the fold once, at engine build time, because batch normalization is a fixed affine transform once training ends. It absorbs the normalization into the convolution weights and bias, then fuses the ReLU into the same kernel, so the input feature map is read from HBM once and only the activated result is written back.

Kernel auto-tuning benchmarks and selects the optimal algorithm for each operation on the target microarchitecture. A single 2D convolution can be evaluated through direct convolution, GEMM transformation (im2col), Winograd minimal filtering (efficient for small 3×3 filters), or Fast Fourier Transforms (efficient for large spatial filters), alongside dozens of tile-size variations. Because the winning strategy depends on filter dimensions, batch size, and the accelerator's register and shared memory capacity, the compiler benchmarks candidate implementations against target hardware and caches the lowest-latency kernel, trading ahead-of-time compilation time for predictable serving latency. Together with operator fusion, these optimizations produce substantial speedups over an untuned framework-native baseline, though they require explicit graph compilation and may restrict dynamic tensor dimensions.

Example 13.6: ResNet-50: Runtime comparison

Table 13.14 applies the chapter's profile to each engine at batch one on the reference H100, changing one term at a time. The eager row is the reference profile. TorchScript runs the same kernels from a graph executor, removing the per-operation Python dispatch; ONNX Runtime also fuses each convolution with its normalization, activation, and residual add, cutting 174 kernels to 56; TensorRT replays the whole fused pass as one captured graph and then varies only the number format.

Table 13.14: Inference runtime comparison: Modeled batch-one latency and speedup for ResNet-50 on the reference H100 across PyTorch eager, TorchScript, ONNX Runtime, and TensorRT, from the chapter's serving profile.

Runtime	Latency	Speedup	Notes
PyTorch (eager)	3.53 ms	1×	Reference profile
TorchScript	1.79 ms	2×	No per-op Python dispatch
ONNX Runtime	0.61 ms	5.8×	Fused kernels
TensorRT FP32	0.38 ms	9.3×	One graph launch; TF32
TensorRT FP16	0.33 ms	10.6×	FP16 Tensor Cores
TensorRT INT8	0.30 ms	11.6×	Requires calibration

Systems insight: Removing per-operation overhead, not lowering precision, produces most of the batch-one speedup. TensorRT FP32 is already 9.3× faster than eager execution, and INT8 adds only 1.3× over FP32, because once host overhead is gone the per-kernel floor dominates a single image. Precision pays at large batch, where per-image work dominates; INT8 also requires calibration data and NVIDIA-specific deployment.

This optimization-compatibility trade-off governs engine selection. Higher specialization can improve hardware efficiency for supported operations and shapes but increases deployment com-

15 **OpenVINO (open visual inference and neural network optimization)**: An Intel-oriented inference toolkit that converts, optimizes, and runs models across Intel CPU, GPU, and NPU targets (Intel Corporation 2026b). Specializing for direct hardware execution departs from the portable execution guarantees of general-purpose frameworks, enabling the compiler to emit architecture-tuned vector instructions (such as AVX-512 or AMX) and exploit low-precision integer execution units. While speedups depend on the specific operator mix and target processor, this specialization can make CPU or edge serving economically viable for latency-sensitive models.

16 **Layer fusion**: The term parallels loop fusion in compiler optimization, where adjacent loops over the same array are combined into one pass to reduce memory traffic.

↩ Kernel fusion, which merges adjacent operations so intermediates stay in registers and on-chip memory instead of round-tripping through HBM, is established in section 7.3.2.

plexity, restricts dynamic control flow, and may introduce numerical deviations from the training baseline. The engineering choice depends on latency budgets, operational constraints, and how frequently model architectures evolve.

Once an execution engine is selected, configuration tunes its resource allocation to the hardware envelope. For CPU inference, thread pool sizes must match physical core counts: too few threads leave execution units idle, while too many cause CPU cache thrashing, OS scheduling delays, and lock contention across non-uniform memory access (NUMA) domains. For accelerators, static memory preallocation reserves fixed-size activation workspaces during initialization, removing runtime memory allocations (`cudaMalloc`) and page-table faults from the latency-critical request path. Execution provider priority determines which hardware backend handles each operator subgraph, while graph optimization levels balance build times against constant folding and algebraic simplification. Production deployments must benchmark these configurations across full latency distributions (p50, p99) rather than relying on framework defaults.

13.6.3 Precision selection for serving

On the reference H100, Tensor Cores execute FP16, FP8, and INT8 matrix operations, and each halving of the bytes per value doubles the dense peak rate. For ResNet-50, moving from FP32 to INT8 multiplies large-batch throughput several times over while sacrificing less than 0.4 percentage points of top-1 accuracy. Numerical precision is therefore coupled directly to infrastructure economics. At the serving layer, precision engineering introduces operational constraints: calibration data fidelity, layer sensitivity under live inputs, and dynamic precision dispatch.

↩ The quantization techniques this selection draws on, from numerical formats through calibration and quantization-aware training, are established in section 10.3.

13.6.3.1 Precision-throughput relationship

For memory-bandwidth-bound operations, reducing precision raises throughput by cutting the byte volume transferred across the memory bus. Equation 13.13 gives the bandwidth-only upper bound from precision reduction:

$$\frac{\text{Throughput}_{\text{INT8}}}{\text{Throughput}_{\text{FP32}}} = \frac{32}{8} = 4\times \text{ (theoretical maximum)} \tag{13.13}$$

Actual speedup depends on the compute-to-memory ratio, operator mix, hardware instruction throughput, and matrix alignment, falling below the bandwidth-only bound whenever arithmetic throughput or memory alignment binds. Low-precision kernels achieve peak arithmetic efficiency only when matrix dimensions align with the accelerator's warp-level execution tiles (for example, multiples of 8 or 16 elements). When dimensions do not align with hardware tiles, libraries such as cuBLAS and cuDNN pad tensors or fall back to slower execution paths.

↩ Tensor Core tile architectures and their matrix-dimension alignment constraints are established in section 11.4.2.

Example 13.7: ResNet-50: Precision trade-offs on the H100

Table 13.15 compares FP32, FP16, and two INT8 paths for ResNet-50 on the reference H100: batch-one latency for each TensorRT engine of the chapter's profile, throughput at large batch, weight bytes, and assumed accuracy:

Table 13.15: Precision trade-offs on the H100: Modeled batch-one latency for each TensorRT engine of the chapter's profile; large-batch throughput modeled at the sustained fraction for FP32 (on TF32 Tensor Cores) and FP16, and measured by MLPerf Inference for INT8; weight bytes from the registry; assumed top-1 accuracy.

Precision	Batch-1 Latency	Large-Batch Throughput	Weights	Accuracy	Calibration
FP32	0.38 ms	15,200 img/s	102.4 MB	76.13%	None
FP16	0.33 ms	30,400 img/s	51.2 MB	76.13%	None
INT8 (PTQ)	0.30 ms	89,500 img/s	25.6 MB	75.80%	1,000 samples
INT8 (QAT)	0.30 ms	89,500 img/s	25.6 MB	76.05%	Additional training

Systems insight: At batch one, INT8 is only 1.3× faster than FP32, because the fixed per-kernel cost dominates a single image. At large batch it sustains about 6× FP32's throughput and loses 0.33 percentage points of accuracy with PTQ. QAT recovers most of that loss but requires retraining.

Integer formats are not the only 8-bit option. On the reference H100, an 8-bit floating-point format (FP8, whose E4M3 layout for weights and activations section 10.3 introduced) is the natural serving default for transformer workloads. The H100's dense FP8 peak of 1,979 TFLOP/s is 2× its FP16 peak of 989 TFLOP/s, and each value occupies one byte instead of two. For the chapter's GPT-2 XL service, FP8 weights shrink from 3 GB to 1.5 GB, which lowers the bandwidth floor on a batch-one decode step from 0.9 ms to 0.45 ms. Because the format keeps a floating-point exponent, it tolerates the wide activation ranges of transformers better than INT8, though it still needs per-tensor scale factors and an accuracy check before deployment.

13.6.3.2 Precision selection constraints

Quantizing every layer of a network to INT8 indiscriminately, or calibrating scale factors on daytime studio photos before serving nighttime camera feeds, can cost several percentage points of accuracy. Precision selection is constrained by layer sensitivity, calibration data, and runtime policy. Not all layers tolerate reduced precision equally. The error bound of the uniform quantizer, half a quantization step inside the calibrated clipping range plus saturation error outside it (section 10.3, with the step from equation 10.1), limits what each layer must absorb. Layer-level output or task error depends on activation distributions and downstream sensitivity, so it must be measured rather than inferred from a universal norm law. This mechanism explains why first convolutional and final classification layers are commonly retained at FP16, while intermediate layers tolerate INT8. Initial layers process low-dimensional raw inputs (such as 3 RGB channels) before representations expand into overcomplete feature spaces that can absorb quantization noise, whereas final classification layers produce raw logits where small numerical perturbations directly shift decision boundaries and alter top-1 predictions.

Post-training quantization adds a data dependency. The calibration dataset determines the clipping thresholds $[\alpha, \beta]$ and scale factors s used for INT8 conversion, requiring it to match the statistical profile of live serving traffic rather than arbitrary training or validation splits. A calibration-serving distribution mismatch causes excessive clipping or underflow, degrading end-to-end task accuracy, an operational pitfall analyzed in section 13.11.

Production serving architectures can also turn precision from a static compilation setting into a dynamic runtime policy. When request queues are shallow and the system operates with ample latency headroom against its service level objective (SLO), the engine can dispatch requests at FP16 to maximize task accuracy. During traffic spikes that threaten queue saturation, the engine can dynamically switch execution to INT8 to preserve latency bounds. Similarly, requests that yield low classification confidence under INT8 can be selectively re-evaluated at FP16, trading a small accuracy margin for lower latency when traffic spikes.

Precision selection carries direct infrastructure sizing consequences. Under the large-batch throughput gain of table 13.15, 10 GPUs at INT8 offer the same aggregate saturated capacity as about 59 GPUs at FP32. Equal capacity does not establish equal queueing latency; service-time distributions, batching policy, and failure reserve must still satisfy the same SLO. Precision engineering

is therefore an infrastructure sizing discipline, directly coupling numerical representation to cluster capital expense and energy consumption. Yet configuring the runtime and numerical bit width addresses only the model level; achieving low-latency serving in production requires optimizing the host-to-device boundaries within the physical server node itself.

13.7 Node-Level Optimization

The reference ResNet-50 request spends 3.5 ms in the forward pass and 8.6 ms end to end. Once a compiled engine shrinks the forward pass to the modeled 0.33 ms, preprocessing alone accounts for 83 percent of what remains. An isolated model benchmark hides exactly these costs, because they sit on the host: JPEG decoding on CPU cores, copies across the PCIe bus, and the driver work that launches each kernel.

Runtime selection and precision tuning operate at the model level, determining what computation runs and what numerical format it uses. Between the model graph and the silicon, however, lies an execution layer encompassing graph compilation, bus transfers, and CPU-accelerator coordination. Node-level optimization targets this complete end-to-end request path on concrete hardware, eliminating overhead between the network socket and accelerator execution units across four primary physical boundaries:

- **Graph-to-kernel boundary**: Translating computation graphs into numerous fine-grained kernel launches incurs driver serialization overhead and forces intermediate tensors through device HBM. Graph compilers fuse adjacent operations into composite kernels that preserve intermediate activations in on-chip SRAM or register files.
- **Host-accelerator boundary**: Accelerators sit idle when the host processor fails to enqueue work ahead of execution. Unpipelined copies across host-device interconnects (such as PCIe) and blocking synchronization calls stall accelerator compute pipelines.
- **CPU execution boundary**: Host preprocessing pipelines and CPU-based inference degrade when executed through interpreted scalar code rather than vectorized single instruction, multiple data (SIMD) instruction units and NUMA-local memory allocations, and when operating-system noise preempts the threads that feed the accelerator.
- **Storage and deserialization boundary**: Initializing serving replicas requires streaming weight tensors from persistent storage into host memory. Formats requiring recursive object deserialization bottleneck CPU execution, creating cold-start latency spikes that impede autoscaling. This boundary governs how quickly a replica starts rather than how fast it serves, so section 13.8.2 develops it with the rest of model loading.

Each boundary represents a distinct physical constraint where execution stalls on memory bandwidth, bus transit, or control overhead rather than peak arithmetic throughput. The subsections that follow cross these boundaries from the accelerator outward—from runtime graph compilation and host-to-accelerator execution pipelines to CPU locality, model loading, and multi-model memory sharing.

13.7.1 Runtime graph compilation

Crossing the first of those boundaries—the graph-to-kernel interface—without compilation leaves the host CPU submitting tiny kernels one operator at a time across a 32-layer transformer or 50-layer convnet, often spending more time in the driver than the GPU spends on the math. Specialized inference engines (section 13.6) beat eager framework baselines because serving workloads run fixed operator graphs over bounded input shapes. When the operator sequence and tensor dimensions are known before live traffic arrives, compilers move graph optimization from the per-request critical path to offline deployment preparation.

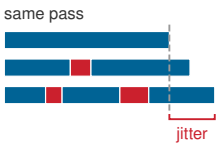
The whole-graph rewrites that make this shift pay are the ones the specialized engines apply at build time. Kernel fusion and the batch-normalization fold of section 13.6.2.2, the canonical case of compile-time constant folding, remove launches and intermediate-memory traffic from every request. Static memory planning (section 11.8.2.3) reuses one preallocated arena for tensors whose lifetimes do not overlap, which keeps allocator calls off the request path. In a serving trace, repeated launch overhead and intermediate-memory traffic point to missing fusion, and allocation stalls on the request path point to a missing memory plan.

Beyond memory planning and kernel fusion, static execution paths can use CUDA Graphs, the replay of a captured sequence of kernel launches that section 7.3.6 developed. Each eager submission pays the 5–20 µs of host launch work quantified in section 13.5.1, and total submission cost depends on the executed kernel count, not the number of layers. A decode step launching 8 to 10 sub-kernels across each of 32 transformer layers can accumulate several milliseconds of host submission work, longer than the accelerator needs for the tensor math at small batch sizes. Serving systems therefore capture the graph during warmup, once per supported batch size because replay requires static shapes and fixed buffer addresses, and dispatch it during live inference with a single driver invocation (illustratively $\sim 5\,\mu s$, depending on the graph, driver, and hardware). Replay removes repeated host submission work without eliminating device execution costs.

> **Systems Perspective 13.4: Compilation timing trade-off**
>
> Just-in-time compilation defers graph optimization until the runtime engine observes concrete input shapes and execution paths. This runtime visibility enables aggressive kernel specialization for dynamic control flow and variable sequence lengths, but it injects compilation overhead directly into the serving path, generating multi-second latency spikes on early requests.
>
> Ahead-of-time compilation executes graph analysis, kernel fusion, and hardware-specific auto-tuning prior to deployment. Pre-building removes the covered compilation work from the request path, but weights, device contexts, allocations, and representative execution paths still require loading and warmup. The compiled engine remains constrained to its supported shape profiles and may require padding or multi-profile compilation to accommodate dynamic input sizes.

13.7.2 Deterministic latency and resource isolation

Compilation fixes what the accelerator executes; it does not control how steadily the host feeds it. The server shares its host with other work. On a single host, the operating system multiplexes hardware across background telemetry daemons, logging agents, and kernel worker threads. When these unisolated processes contend for execution resources, they intermittently preempt the inference pipeline. These noisy neighbors are a primary source of **latency jitter**, where the physical time required to execute identical forward passes fluctuates unpredictably. This contention inflates 99th percentile (p99) tail latency even when average hardware utilization remains low. Resource contention thus widens tail latency through service-time variance, a physical degradation distinct from the queuing-delay accumulation along the saturation knee plotted in figure 13.5.



Noisy neighbors stretch identical forward passes unevenly, widening the tail.

Keeping latency predictable requires isolating the serving process from the operating system's default time-sharing scheduler. Serving systems such as Clockwork demonstrate that deep neural network (DNN) inference execution times can be predictable down to sub-millisecond tolerances under their controlled workload and hardware conditions (Gujarati et al. 2020). CPU affinity (thread pinning) establishes the baseline compute boundary by binding inference worker threads to designated physical cores. Without pinning, the OS scheduler periodically migrates threads between cores, losing access to warm private L1 and L2 cache contents and requiring lines to be refetched from shared L3 cache or DRAM. On multi-socket architectures, thread migration across NUMA boundaries is even more penalizing. Accesses to memory left on the previous socket cross inter-socket interconnects, adding tens of nanoseconds of latency per transaction. Pinning threads to specific cores and binding their memory allocations to local NUMA nodes reduces migration-related cache misses and remote-socket memory traffic.

Even with pinned CPU cores, memory access latency remains vulnerable to the virtual memory subsystem. Under host memory pressure, the kernel can page inactive address ranges out to disk. When the CPU touches a paged-out address, execution can stall for milliseconds while the page returns to DRAM. Host buffers used for PCIe DMA also need resident, registered backing pages during the transfer. Memory locking (`mlock`) instructs the kernel to pin specified virtual address ranges into physical host RAM, preventing page eviction for host-resident weights, input staging buffers, and offloaded key-value (KV) cache blocks. This removes eviction-related page faults at the cost of host memory flexibility; locked pages remain resident until unlocked, while cache misses

and memory contention can still delay access. Because `mlock` operates strictly on host page tables, it does not apply to accelerator HBM, which is allocated directly by the accelerator driver.

Beyond scheduled processes, asynchronous hardware interrupts introduce another source of sub-millisecond preemption. When a network interface card (NIC) receives packet bursts, it triggers hardware interrupt requests (IRQs) that force the handling CPU core to immediately suspend user-space execution and process kernel interrupt handlers. If the OS routes these interrupts to cores running the inference engine's submission loop, the CPU pauses command dispatch to the accelerator. Because GPUs process short inference layers in microseconds, even brief CPU submission stalls drain the accelerator's hardware command ring buffer, causing pipeline bubbles where compute units sit idle waiting for the next kernel descriptor. Interrupt shielding restricts IRQ affinities to dedicated housekeeping cores, reducing preemption of the GPU dispatch thread by those device interrupts.

Isolating physical cores, locking host memory, and shielding interrupt lines reduce specific sources of operating system noise, though the resulting service-time distribution still needs measurement under the intended load. Thus far, isolation has treated the host as a predictable partner that feeds a discrete accelerator; when deployment constraints or workload profiles dispense with the accelerator altogether, the host CPU becomes the executor itself.

13.7.3 CPU inference optimization

For a small fraud-detection or ranking model evaluated at batch size one, copying a single request across PCIe and launching GPU kernels through the driver can take longer than running the forward pass directly on the host processor. Deploying inference on CPUs therefore remains practical for lightweight models, sparse or bursty request streams that cannot saturate an accelerator, and services where host-to-device transfer time exceeds the math time. Yet an untuned Python CPU path leaves most of the processor's silicon idle. Modern server CPUs[17] (such as Intel Xeon and AMD EPYC) include wide SIMD vector registers and matrix units (AVX-512 and AMX), which specialized runtimes such as OpenVINO target by tiling matrix operations across L1/L2 caches and emitting packed fused multiply-add (FMA) instructions (Intel Corporation 2026b).

[17] **SIMD**: From Michael Flynn's 1966 taxonomy of computer architectures, SIMD enables one instruction to operate on multiple data elements simultaneously. Intel's AVX-512 can process sixteen FP32 values in a 512-bit vector, while AMX adds matrix tile operations. The attainable fraction of peak depends on data type, matrix shape, memory behavior, and implementation, but optimized vector kernels provide multiple-fold speedups over scalar execution.

The NUMA boundaries that thread pinning respects matter more when the CPU itself runs the model. Memory controllers are physically distributed across CPU sockets and linked by coherent interconnects (such as Intel UPI or AMD Infinity Fabric). Accessing memory attached to a remote socket incurs substantially higher latency and lower bandwidth than local channel access. An inference server must maintain NUMA awareness. Worker threads must be pinned to dedicated physical cores, and the weight tensors and request buffers those threads touch must reside in memory nodes local to that socket. Because model weights, at hundreds of megabytes to tens of gigabytes, routinely exceed per-socket L3 cache capacities, every forward pass streams weights from main RAM. Mismatched socket allocations force continuous weight streaming across the slower inter-socket interconnect, creating severe tail-latency degradation.

For lightweight models evaluated at batch size one, a CPU frequently outperforms a discrete GPU. When an accelerator processes an isolated request, the fixed overhead of transferring inputs across the host PCIe bus (several microseconds), driver queue serialization, and kernel launch overhead (5–20 µs per kernel) can easily surpass the arithmetic execution time. The CPU processes the request directly in local cache without bus transfers or driver queue delays. Determining the operational crossover requires empirical measurement across model architecture, arithmetic intensity, batch size, and latency constraints.

13.7.4 Profiling the serving node

Finding that crossover—or diagnosing why an accelerator-backed node misses its p99 target while average GPU utilization sits below 30 percent—requires inspecting where each request actually waits across the host and device. The top-down profiling method of section 8.5 applies unchanged to a serving node, where the objective is useful throughput that meets the latency SLO. The duty-cycle metric defined in equation 13.2 helps locate idle intervals, but active time alone does not measure arithmetic efficiency or prove adequate queueing headroom. Timeline profilers, such as PyTorch Profiler and NVIDIA Nsight Systems (`nsys`), record concurrent CPU threads, OS runtime calls, memory transfers, and GPU hardware queues along a synchronized timeline.

Serving traces show three recurring signatures:

- **Idle compute bubbles**: White space on the GPU execution track indicates hardware starvation. Compute warps idle while waiting for the host CPU to complete request preprocessing (e.g., tokenization or image decoding), page faults during on-demand weight paging, or blocking inter-thread synchronization.
- **Kernel launch fragmentation**: A sequence of micro-kernels, each executing in under 5–10 µs, suggests launch overhead as a candidate. Host launch timing and gaps between device kernels reveal whether submission starves the GPU; if so, graph capture or legal fusion may help.
- **Interconnect transfer stalls**: Synchronous `MemcpyHtoD` blocks on the primary CUDA stream indicate unpipelined host-to-device data movement. Overcoming this stall requires allocating page-locked (pinned) host memory and executing asynchronous transfers on dedicated copy streams to overlap bus transit with concurrent accelerator execution on preceding requests.

Example 13.8: The profiling loop

Scenario: An inference serving node reports high p99 tail latency despite low average GPU compute utilization.

Diagnosis: Timeline profiling (Nsight Systems/PyTorch Profiler) reveals large idle gaps between tiny GPU kernels caused by CPU host-side preprocessing bottlenecks and kernel launch overhead.

Systems lesson: Serving node optimization requires empirical timeline profiling. Iteratively identifying trace bottlenecks (kernel launch overhead, host-device copy stalls) guides targeted optimizations like kernel fusion and pinned memory.

Table 13.16 summarizes representative node-level optimization techniques alongside their primary target metrics and engineering costs. The table serves as a decision matrix rather than an exhaustive checklist. Engineering effort should be directed exclusively to the row that addresses the binding bottleneck identified on the measured execution trace.

Table 13.16: Node-Level Optimization Impact: Illustrative ranges for matching techniques to measured bottlenecks. Realized gains and implementation cost depend on the model, runtime, hardware, and workload.

Technique	Target Metric	Illustrative Gain	Implement. Cost	Best For
Kernel Fusion	Latency & Throughput	2–5×	Medium (Compiler)	Memory-bound layers
INT8 Quantization	Throughput	3–4×	High (Calibration)	Inference-heavy nodes
Graph Compilation	Latency	1.5–3×	Low (One-line)	Static graph models
Zero-Copy Loading	Startup Time	10–50×	Low (File format)	Autoscaling/Cold Start
CPU Pinning	Tail Latency (p99)	20–50% reduction	Low (Config)	Latency-critical apps

This diagnostic hierarchy ensures engineering resources address structural bottlenecks along the entire request path rather than applying isolated micro-optimizations. A four-level audit checkpoint structures this prioritization from external service boundaries down to device kernels.

Checkpoint 13.4: The optimization hierarchy

Optimizing inference follows the request path from the outside in across four structural levels:

- ☐ **System budget**: Partition the end-to-end latency budget between network transit, host application processing, queueing delay, and kernel execution using distributed tracing tools to verify compliance.
- ☐ **Application level**: Determine whether the arrival stream and latency budget permit batching, and test whether the gain in service capacity outweighs formation and resource-queue waiting.

- ☐ **Model level**: If dispatch or operator placement binds the request path, evaluate a compatible compiled engine or graph configuration and recheck quality across supported input shapes.
- ☐ **Kernel level**: Where intermediate traffic or launch overhead is measured, assess legal fusion candidates and verify their numerical behavior and end-to-end benefit.

A node tuned this way runs each request near its physical limit, but only once it is running. The compiled engine that makes the steady state fast took tens of seconds to build, its weights must cross storage, host memory, and the PCIe bus before the first request arrives, and its preprocessing must match what training used. How a replica reaches that steady state, and whether it computes what training computed once there, decides whether the fast node is also a correct and available one.

13.8 Model Loading and Versioning

In a representative deployment, a newly provisioned replica needs 35 seconds before it can answer its first request, most of them spent compiling a TensorRT engine, while a subtle PIL-to-OpenCV preprocessing discrepancy silently costs five percentage points of classification accuracy. Every optimization so far assumed a replica that is already running and that computes the function training produced; both assumptions fail at the edges of a replica's life. Cold start delays the moment a new replica becomes useful, so it is controlled by how weights are loaded, cached, and shared across the models and versions that occupy one machine. Skew corrupts every prediction a correctly running replica makes, so it is prevented by keeping training and serving transformations identical; serving is where its symptom appears. The path below follows a replica from storage to a warmed engine, then to the several models and versions one machine may host, and ends with that symptom.

13.8.1 Cold start and initialization dynamics

When an autoscaler spins up a new serving container during a traffic surge, that replica cannot answer a single request until its weights are loaded into memory, its execution graph is compiled, and its device buffers are initialized (Romero et al. 2021). This startup delay, known as cold start, governs how quickly a service recovers from node failures or scales out into a spike.

Definition 13.4: Cold start

Cold start is the initialization latency incurred when starting a new model replica.

1. **Significance**: It represents the fixed startup cost of loading weights, compiling graphs, and allocating buffers, which can take seconds or minutes and block elastic scaling during traffic bursts.
2. **Distinction**: Unlike inference latency ($L_{\text{lat,infer}}$), which is a per-request cost, cold start is an initialization cost paid when a replica or model instance is created, including deployment, scale-out, recovery, and on-demand model loading.
3. **Common pitfall**: A frequent misconception is that cold start is "just loading weights." In reality, it includes graph compilation and memory allocation, which can often take longer than the bandwidth-limited data transfer itself (D_{vol}/BW).

Total cold-start latency compounds across four physical operations along the initialization path: weight loading streams serialized parameters from persistent storage across the network or host I/O subsystem into host memory and then across the PCIe bus into GPU HBM, after which graph compilation lowers the framework computational graph into device-specific machine code by fusing operators and auto-tuning matrix multiplication kernels for the target accelerator. Once the compiled graph is ready, memory allocation reserves device arenas for activation scratchpads and intermediate buffers, and warmup[18] execution runs synthetic inferences to populate kernel caches, prime framework memory allocators, and complete any remaining lazy initialization before live traffic arrives.

[18] **Warmup**: Borrowed from just-in-time (JIT) compilation, where initial executions compile hot paths into optimized machine code; for ML serving, warmup inferences trigger CUDA kernel compilation, cuDNN algorithm auto-tuning, and memory pool allocation that frameworks defer until first use.

Establishing a CUDA context represents the first fixed cost on the device initialization timeline. Before launching any GPU kernel or allocating device memory, the runtime must instantiate a driver-level context: an internal control structure that manages virtual address spaces, loaded kernel binaries, and hardware event queues. Creating this context requires synchronous communication with the kernel-mode driver and allocates tens to hundreds of megabytes of device memory for bookkeeping. The 0.3–0.5 s range in table 13.17 reflects a typical data center environment; while CUDA lazy loading defers kernel module loading to shorten this initial pause, it merely defers that latency penalty to the first executed inference (NVIDIA 2026a).

In multi-process serving deployments where multiple worker processes share a single accelerator, replicating individual CUDA contexts wastes both memory capacity and initialization time. CUDA Multi-Process Service (MPS) coordinates concurrent work from multiple client processes, reducing scheduling overhead when individual processes underuse the device (NVIDIA 2026d). Its isolation behavior depends on the architecture. Pre-Volta clients shared a GPU address space; Volta clients retain separate GPU address spaces and contexts while their kernels can execute concurrently. Separate address spaces do not provide hard fault containment. A fatal GPU fault can affect other clients sharing the affected GPU, so the serving design must budget for correlated replica failures.

Example 13.9: ResNet-50: Cold start timeline

Table 13.17 traces the per-phase duration of a ResNet-50 cold start:

Table 13.17: ResNet-50 cold start timeline: Illustrative per-phase durations for weight loading, CUDA context creation, TensorRT compilation, and warmup, with totals for the optimized local case and the first-deploy cloud case showing where the dominant cost lives.

Phase	Duration	Notes
Weight loading (SSD)	0.5 s	98 MB FP32 weights from local storage
Weight loading (S3)	3–5 s	Network latency dominates for cloud storage
CUDA context	0.3–0.5 s	GPU driver initialization and memory setup
TensorRT compilation	15–30 s	Converts PyTorch model to optimized engine
Warmup (10 inferences)	0.2 s	Triggers remaining lazy initialization
Runtime overhead	0.4 s	Process startup, framework hooks, and runtime setup
Total (local, optimized)	**~1.5 s**	With precompiled TensorRT engine, warm container
Total (cloud, first deploy)	**~35 s**	Including compilation from cold state

Systems insight: Precompiling models and storing the optimized engine eliminates the 30-second compilation phase on subsequent deployments.

Without warmup, the first live request absorbs deferred JIT compilation, driver memory allocation, and kernel launch overhead. A request whose steady-state forward pass takes 3.5 ms can stall for hundreds of milliseconds during an unmitigated cold start, violating strict SLOs. Controlling this initialization overhead requires systematic selection among storage, memory, and runtime loading strategies.

13.8.2 Loading strategies and serialization formats

Once precompilation removes the 30-second compiler step from table 13.17, weight loading becomes the largest remaining term in the startup budget, and how the server schedules that read trades cold-start duration against first-request latency and memory footprint. The simplest approach, full loading, reads the entire model into memory before the server marks itself ready. This maximizes steady-state inference speed because all weights are resident in memory, but it stretches cold-start time and restricts model size to available memory. A second strategy, lazy initialization, defers compilation and allocation until first use. This minimizes startup time but shifts latency to the first request, so production systems often combine lazy initialization with synthetic warmup requests to trigger initialization before real traffic arrives.

Autoscaling systems operate as closed-loop controllers that provision or deprovision serving replicas in response to traffic demand. During demand surges, cold-start latency governs how rapidly new replicas become ready to serve traffic. For multi-gigabyte models, cold-start duration is heavily dominated by streaming serialized model weights from persistent storage into host memory. The architectural choice of serialization format determines whether this stage is bounded by storage read bandwidth or by CPU parsing overhead.

Legacy PyTorch checkpoints loaded via `torch.load()` package weights within a zip archive containing pickle-encoded object metadata. Deserializing a pickle file forces the CPU to interpret a stream of serialization opcodes sequentially, allocating individual Python objects (dictionaries, metadata wrappers, and tensor headers) and potentially copying tensor bytes into newly allocated memory. Restricted loading modes reduce arbitrary code execution risks, while memory-mapped options can avoid some eager copies. Whether parsing or storage dominates depends on the checkpoint, loading path, and hardware.

A tensor-oriented format[19] bypasses interpreter-driven object construction. Safetensors organizes checkpoints with a lightweight header containing tensor metadata followed by contiguous, uncompressed tensor byte buffers aligned to memory boundaries. Because the on-disk binary layout mirrors the in-memory array layout, the runtime can use memory mapping (`mmap(2)`), which maps file blocks directly into the process's virtual address space. Virtual memory pages fault into host RAM on access or are prefetched sequentially by the operating system, reducing Python object reconstruction and avoiding eager user-space tensor copies on the mapped path. Mapping shortens apparent startup, but a dense model eventually touches nearly all of its weights, so deferred page faults can move loading latency into early requests. The benefit is greater for sparsely accessed components or model repositories that load subsets on demand; latency-sensitive services still prefault or warm the pages needed on the request path. On systems with discrete accelerators, transferring these tensors across PCIe or NVLink is still required before computation begins, so memory-mapped host loading reduces CPU reconstruction costs without eliminating page faults, storage reads, or device initialization.

[19] **Safetensors**: The name emphasizes safety: unlike unrestricted pickle deserialization, the format does not encode executable Python objects (Hugging Face 2026). It stores tensors as contiguous raw bytes with a small header and supports memory-mapped loading. In the local benchmark example, that path is 10× faster than the compared PyTorch checkpoint path; the result is workload- and storage-specific and does not include transfer to a discrete accelerator.

Example 13.10: Loading speed: Safetensors vs. Pickle

Scenario: A serverless inference replica loads a 5 GB model checkpoint during cold-start scaling events.

Diagnosis: PyTorch `torch.load()` (Pickle) requires CPU object reconstruction, taking 15 s to initialize. Safetensors memory-mapping bypasses CPU parsing, loading weights in 1.5 s (10× faster).

Systems lesson: Tensor-oriented, memory-mapped formats can reduce CPU reconstruction and eager-copy overhead during cold start. Storage throughput, page faults, runtime initialization, and accelerator transfer still remain in the scale-out budget.

13.8.3 Model caching infrastructure

Production systems also cache model weights at the infrastructure level to shorten cold start, and the right cache follows how often the model changes. Embedding the weights in the container image yields a single deployment artifact with no network fetch at startup, but images of 10–50 GB slow every container pull, so the approach suits models that rarely update. A shared filesystem of versioned weights lets many replicas read one copy without rebuilding images, at the price of network latency in every cold start and a new availability dependency; it still requires an explicit, atomic version switch so that files never change underneath a live replica. Node-local SSD caching prepopulates Non-Volatile Memory Express (NVMe) drives on inference nodes with frequently used models and loads at several GB/s without the network, but it needs its own refresh policy as models update. Which versions those caches hold, and how a new version replaces an old one in production, belongs to the release process of section 14.4.3.

13.8.4 Multi-model serving

Dedicating an entire GPU to each regional model, A/B candidate, or ensemble stage leaves most of the fleet idle whenever traffic to individual models is uneven, yet packing several models onto one GPU without coordination risks out-of-memory crashes and noisy-neighbor latency spikes. Production systems therefore share a single machine across multiple models, making GPU memory capacity and bandwidth isolation the primary constraints. How a server manages that shared memory depends on whether models fit in VRAM simultaneously. When the combined footprint exceeds device capacity, time-multiplexing loads one model at a time and swaps weights on demand as routing changes, keeping implementation simple at the cost of swap latency. When several models fit together, memory sharing partitions GPU memory among them so that more models remain resident without swapping, even though each model receives a smaller concurrency budget. Across both regimes, model virtualization—as implemented by frameworks like Triton—decouples model lifecycle from application code by exposing repository and control APIs that load, unload, and version models dynamically (NVIDIA 2024d, 2026c, 2026b). Which policy wins depends on the arrival mix: steady traffic across models favors keeping them concurrently resident, whereas bursty, model-specific traffic favors time-multiplexing with explicit preloading to balance average latency against GPU utilization.

13.8.4.1 Partitioning one GPU across models

When multiple models or multiple instances of the same model must run concurrently on a single GPU, the hardware must partition resources between them. NVIDIA's Multi-Instance GPU[20] technology enables hardware-level isolation, dividing a data-center GPU such as the H100 into up to seven independent GPU instances, each with dedicated memory and compute resources (NVIDIA 2026e). On GPUs without MIG, CUDA stream scheduling permits overlapping or time-multiplexed work without hardware partitioning. The choice depends on whether consistent latency with MIG or maximum utilization with shared streams is the priority.

13.8.4.2 Model swapping and host memory

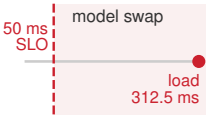
When the aggregate size of all models exceeds GPU memory capacity, the serving system must swap models between host dynamic random-access memory (DRAM) and device memory (VRAM) on demand. This introduces a new latency component determined by the PCIe bus bandwidth. For a 10 GB model on PCIe Gen4 x16 (32 GB/s theoretical bandwidth), loading takes at least 312.5 ms before deserialization, graph setup, or warmup. This raw transfer latency dwarfs a tight service-level objective such as a 50 ms deadline by more than $6\times$, rendering on-demand model swapping unviable on the interactive request path.

To accelerate this transfer path, systems keep the weights in pinned memory (page-locked host RAM). The GPU's DMA copy engines require the source pages to stay at fixed physical addresses for the whole transfer, but ordinary heap memory is pageable, and the operating system may relocate or swap its pages at any time. A transfer from a pageable buffer therefore takes two stages: the CPU first copies the data into a pinned staging buffer, and the DMA engine then moves that buffer across PCIe. Allocating model weights with `cudaHostAlloc` removes the CPU staging copy; achieved bandwidth still depends on transfer size, memory locality, and contention. What serving adds is a capacity trade-off. Pinned memory cannot be paged out or reclaimed by the OS under memory pressure, so every pinned model holds its share of physical host RAM until it is released. High-throughput serving systems therefore pin active model weights and I/O ring buffers while leaving standby model checkpoints in pageable memory.

13.8.5 Training-serving skew

Loading the exact checkpoint that training produced still does not guarantee that the server computes the same function if the host pipeline transforms raw inputs differently before those weights ever run. A model that performed well during validation degrades silently in production whenever training-serving skew causes the two paths to compute different features from the same raw input while every latency and error-rate dashboard stays green. The consistency requirement belongs to the data pipeline; serving is where a violation surfaces, and because the divergence is measurable on identical inputs without labels (principle 11), it is also one of the most preventable failures. Section 7

[20] **MIG (multi-instance GPU)**: Introduced with NVIDIA's A100 (NVIDIA Corporation 2020a) and documented across supported data-center GPUs (NVIDIA 2026e), MIG partitions a single physical GPU into independent instances, each with dedicated compute and memory resources; unlike software sharing (MPS or time-slicing), MIG provides hardware-level isolation between partitions. The trade-off is granularity, because partitions must follow fixed profiles, so resources cannot be divided arbitrarily. For multi-model serving, MIG reduces noisy-neighbor risk on shared hardware, while per-model SLO guarantees still depend on scheduler policy, load, and the selected partition profile.



Model loading lands far outside a tight serving SLO.

↩ Pinned memory and the DMA staging copy it avoids are established in section 7.5.2.1.

↩ Training-serving skew and the consistency imperative it violates are established in section 4.7.1.

catalogs the skew categories and the method that detects each. The serving-specific manifestation of skew is **preprocessing divergence**: the real-time inference pipeline processes raw data differently than the batch training pipeline, a common failure mode when training uses Python/Pandas while serving uses C++/Java or optimized inference servers.

> Example 13.11: ResNet-50: Image preprocessing skew
>
> **Scenario**: ResNet-50 vision serving models experience silent prediction accuracy degradation between offline training and online serving pipelines.
>
> **Diagnosis**: The serving pipeline implemented C++/OpenCV preprocessing (`cv2.INTER_LINEAR`, BGR order) while training used Python/PIL (`PIL.BILINEAR`, RGB order, different normalization constants).
>
> **Systems lesson**: Preprocessing divergence causes silent training-serving skew without raising runtime exceptions. Standardizing data transformation pipelines across training and serving environments prevents input distribution corruption.

A replica that loads quickly, runs predictably, and computes what training computed completes fixed-output serving. Every workload so far, however, has treated a request as one completed prediction. The fastest-growing category of serving workloads violates this assumption entirely. Large language models generate outputs token by token, with each token depending on every previous one. A single request may produce hundreds or thousands of tokens over seconds of elapsed time, yet must feel responsive from the first token onward. The shift from *fixed-output* to *variable-length, streaming-output* serving reopens each tool developed so far, from the latency budget to the runtime.

13.9 LLM Serving

An image classifier returns a single class vector in a few milliseconds and immediately frees its activation buffers, whereas a conversational LLM streams hundreds of tokens over several seconds while holding a growing key-value cache in GPU memory. LLMs make three properties far more prominent than in fixed-output serving: *autoregressive generation*[21] (each token depends on all previous tokens, making output inherently sequential), *variable-length output* (response length is unknown at request time, complicating fixed-batch assumptions), and *stateful memory* (the key-value cache grows with each generated token, creating dynamic memory pressure). Each property strains a different tool from the preceding sections: sequential generation splits the latency budget into phases, unknown output length breaks fixed-batch scheduling, and the growing cache makes memory capacity, not arithmetic, the limit on concurrency. The p50, p95, and p99 metrics used for classification serving still matter, but they apply separately to the initial prompt processing and to the subsequent token generation. The same principles of queuing theory, batching trade-offs, and latency budgets remain applicable; LLM serving adds domain-specific techniques atop them.

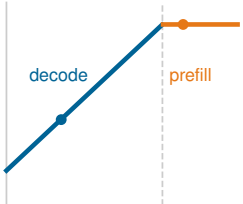
The chapter's second service carries the section. GPT-2 XL, the bandwidth-bound lighthouse, anchors the metrics, the decode floor, speculative decoding, and the energy per generated token, because its weight stream sets each of them. Llama 3 8B, introduced with the running cases, carries the cache budget, paging, and prefix reuse, because its longer contexts make memory capacity rather than bandwidth the constraint that binds first.

13.9.1 Performance metrics: TTFT and TPOT

A single end-to-end latency number cannot distinguish a chat response that begins streaming in 100 ms and flows at reading speed from one that leaves the user staring at a blank screen for two and a half seconds before dumping the entire paragraph at once. Because generative models emit a stream of tokens rather than a single output tensor, serving systems split latency into time to first token (TTFT) and time per output token (TPOT), the average of a request's inter-token latencies (ITL), as defined in section 12.6.2.2. In serving, each metric belongs to one phase. *Prefill* (processing the prompt and populating its KV cache) sets TTFT and is often compute-bound, while *decode* (generating output) sets TPOT and is often bandwidth-bound at small batches. A low TTFT therefore does

[21] **Autoregressive**: From Greek *auto-* (self) and Latin *regressus* (a going back), since the output "regresses" on itself; George Udny Yule introduced autoregressive models in 1927 for analyzing sunspot cycles. In language modeling, each output token conditions on all previously generated tokens, creating a serial dependency that prevents the parallelism exploited during training. Decode is often weight-bandwidth-bound at small batches; larger batches amortize the weight stream and can shift the bottleneck toward compute or KV-cache traffic (Pope et al. 2023).

↩ Prefill, small-batch autoregressive decode, and the KV cache that every decode step reads are established in section 6.7.4.2.



Prefill and low-batch decode often face different resource limits.

not mean a fast model. A model can have a fast TTFT but a sluggish TPOT (if the memory wall (BW) is the bottleneck), leading to a frustrating user experience where the answer starts quickly but "stutters" thereafter. Because these metrics capture distinct aspects of the user experience, interactive services set separate targets for TTFT, TPOT or ITL, and aggregate throughput.

> **Systems Perspective 13.5: LLM serving latency targets**
>
> Interactive LLM services usually need separate SLOs for responsiveness, generation fluidity, and fleet throughput. The following values are illustrative targets rather than universal requirements:
>
> - **TTFT**: $<$ 500 ms (for a 1000-token prompt)
> - **TPOT**: $<$ 50 ms (equivalent to ~20 tokens/s, faster than human reading speed)
> - **Throughput**: $>$ 1000 tokens/s aggregate across active serving replicas
>
> A single "latency" number therefore hides the prefill/decode split. TTFT is the blank-screen budget, TPOT is the reading-flow budget, and aggregate service throughput, measured as tokens/s summed across active serving replicas, determines whether those targets hold under shared load.

13.9.2 The decode floor

The chapter's bandwidth-bound lighthouse puts numbers on both phases. A GPT-2 XL chat request with a 512-token prompt performs about 1.5 TFLOP of prefill arithmetic, which an H100 sustaining 40 percent of its FP16 peak completes in about 3.9 ms, the model's share of TTFT. Each decode step then reads all 3 GB of FP16 weights to produce one token per active sequence. At batch one that read takes 0.9 ms, so the model-side TPOT can be no shorter, however many arithmetic units the accelerator adds. The floor assumes the step is launched as one captured graph. Issued operation by operation from an eager framework, the same step spends about 8.6 ms on the host submitting 576 kernels, roughly 10× the weight read, which is why every decode floor in this chapter assumes the CUDA Graph capture that section 7.3.6 developed.

Batch-one decode with FP16 weights runs at the weight-only arithmetic intensity of about 1 FLOP/byte that section 6.7.4.2 derived. The H100's ridge point sits near 295 FLOP/byte, so the step is memory-bandwidth bound by more than two orders of magnitude (Pope et al. 2023), and equation 13.14 gives its floor per token, where D_{vol} is the bytes the step reads and $\text{BW}_{\text{accelerator}}$ is the accelerator's memory bandwidth:

$$T_{\text{token}} \approx \frac{D_{\text{vol}}}{\text{BW}_{\text{accelerator}}} \tag{13.14}$$

While decode remains bandwidth bound, every step pays to move every weight, in energy as well as in time, and adding compute cores yields little latency improvement; faster memory (Machine), fewer bytes per parameter or a smaller model (Algorithm), or better batching and cache management change the bound.

Batching changes which bytes dominate. Each sequence also reads its own KV cache on every step. GPT-2 XL's attention heads each keep their own keys and values, with no grouped-query sharing, so its cache holds 307.2 KB per token of context, or 314.6 MB for a full 1,024-token context. Batching shares the weight read across sequences but not the cache reads. At batch one with a full context, a step moves 3.3 GB and takes 1.0 ms. At batch 32 it moves 13.1 GB, 77 percent of it cache, and takes 3.9 ms. Aggregate throughput rises from about 1,010 tokens/s to 8,200 tokens/s, while each sequence's TPOT stretches by 3.9×. The same trade between per-sequence cadence and aggregate throughput governs every scheduling technique that follows.

13.9.3 Decoding strategies

Switching a chat endpoint from greedy decoding to four-way beam search quadruples both the arithmetic per step and the KV-cache memory each active user holds. The sampling algorithm executed at each step therefore shapes serving cost directly. Greedy decoding takes the highest-scoring token and needs one forward pass per token, though it can fall into repetitive output because

it never revisits an early choice. Beam search keeps several candidate sequences and multiplies both computational work and KV-cache footprint by the beam width. Stochastic sampling scales the logits by a temperature and truncates the vocabulary with top-k or top-p nucleus sampling before drawing a token (Holtzman et al. 2020). For the serving system, beam width multiplies the state each request holds, while the sampling variants add only a small per-step reduction whose cost depends on its kernel implementation and on the sequence-length distribution that continuous batching must absorb.

Interactive LLM services commonly return tokens as they are produced rather than waiting for complete generation. This streaming response transforms the user experience. A two-second total generation feels responsive when tokens stream continuously, but feels broken when users stare at a blank screen for two seconds. Streaming requires infrastructure support for incremental responses and client-side rendering. Once generation streams token by token, the serving unit changes from one fixed-output prediction to a stateful sequence whose length is unknown when it is admitted and whose memory footprint grows on every step. The unknown length breaks the fixed-batch scheduling developed for fixed-output models, and the growing state turns accelerator memory into the next constraint.

13.9.4 Continuous batching

An autoregressive request finishes only when it emits an end-of-sequence token, so its length is revealed one step at a time. Extending the request-level dynamic batching of section 13.5 to an entire generation cycle keeps batch membership fixed until every sequence finishes. When sequence lengths vary, this static commitment degrades accelerator efficiency. If one sequence in a batch of eight finishes after ten tokens while others require 100 tokens, 90 percent of the compute and memory bandwidth allocated to that sequence slot is wasted on padding tokens or masked dummy operations while awaiting the slowest sequence (Yu et al. 2022).

Continuous batching[22] (also called iteration-level batching) eliminates this idle-slot waste by rescheduling batch composition at each decoding iteration (Yu et al. 2022; Kwon et al. 2023). Rather than committing to a static batch that persists across hundreds of forward passes, the serving engine re-evaluates batch membership at every token-generation step. When a sequence emits its end-of-sequence token, the scheduler immediately reclaims that execution slot, admitting an arriving request into the batch for the subsequent forward pass. Active sequences continue decoding without interruption, maintaining high accelerator utilization across variable-length workloads.

Serving runtimes such as vLLM[23] and TensorRT-LLM improve throughput via two mechanisms: reclaiming wasted compute from completed sequences and shortening queue wait times for new arrivals (Kwon et al. 2023; NVIDIA 2026g). However, admitting new requests dynamically creates a systems collision between prompt prefill and token decoding. Long-prompt prefill often exposes high arithmetic intensity and can occupy the GPU for tens of milliseconds. Low-batch decoding often has less weight reuse and is memory-bandwidth bound, advancing one token per active sequence at a time. When an engine executes a prefill in the same iteration as active decodes, the ongoing decode streams stall, causing severe spikes in inter-token latency. Sarathi-Serve resolves this interference through chunked prefill and stall-free batching (Agrawal et al. 2024). By splitting long prompts into fixed-size chunks (e.g., 512 tokens) and co-scheduling a single chunk alongside active decode requests per step, the scheduler limits the prefill work admitted per iteration, reducing interference with active decodes; measured iteration times determine the resulting token cadence.

13.9.4.1 Speculative decoding

Continuous batching and chunked prefill keep decode batches full and their cadence steady, but at small batch each generated token still pays for one full weight stream. Speculative decoding[24] attacks that bound directly. A small draft model proposes several tokens, and the target model verifies the whole proposal in one parallel pass, so a single weight stream can yield more than one accepted token. Evaluated T5-XXL workloads achieved roughly 2–3× speedups (Leviathan et al. 2023), though the gain depends on how often the target model accepts the draft.

If the draft proposes γ tokens and the target accepts each with probability α, independently of the others (Leviathan et al. 2023), one verification pass yields on average the number of tokens

[22] **Continuous batching**: NVIDIA TensorRT-LLM calls the same technique "in-flight batching" (NVIDIA 2026g). Request-level batching commits to a fixed batch for an entire generation sequence (potentially hundreds of iterations), while continuous batching reschedules at every token-generation step, a contrast that parallels preemptive OS process scheduling versus run-to-completion.

[23] **vLLM (virtual large language model (LLM))**: An open-source serving system that enables continuous batching via its PagedAttention algorithm (Kwon et al. 2023). Keeping KV-cache waste low lets it serve larger effective batches on the same hardware.

[24] **Speculative decoding**: Parallel verification is possible because the draft has already committed to all γ candidate tokens, so the target can score the whole block in one causal-attention forward pass, as it scores a prompt during prefill, rather than in γ sequential steps (Leviathan et al. 2023). The technique removes serial steps at the runtime layer without changing the model architecture.

in equation 13.15, which counts the accepted draft tokens plus the one token the target always contributes:

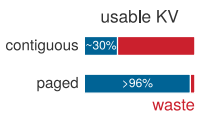
$$\mathbb{E}[\text{tokens per target pass}] = \frac{1-\alpha^{\gamma+1}}{1-\alpha} \tag{13.15}$$

Verifying $\gamma+1$ positions costs the target about one weight stream, because at batch one the pass remains bandwidth bound rather than compute bound. For the GPT-2 XL service, a GPT-2 Small draft with 248 MB of FP16 weights proposes 4 tokens per round, spending 0.30 ms on draft reads against 0.90 ms for the target's verification pass. At an assumed acceptance rate of 70 percent, a round yields 2.8 tokens on average, so the time per generated token falls from 0.90 ms to 0.43 ms, a 2.1× speedup. The gain shrinks as acceptance falls or as the batch grows. A full batch already shares each weight stream across many sequences, and verifying several positions per sequence then consumes arithmetic that a batch-one step left idle but a full batch does not.

13.9.4.2 PagedAttention

Keeping decode batches full, however, shifts the primary serving bottleneck to memory management. Because transformer generation appends attention states at every step, the KV cache for each active request grows dynamically in accelerator HBM. When sequences of varying lengths share memory, contiguous allocation produces severe external fragmentation. If a completed 100-token sequence releases its cache, an arriving 150-token request cannot occupy that freed memory because it requires a larger contiguous address range. Over time, physical memory fractures into small, noncontiguous gaps, preventing new sequences from launching even when total unallocated capacity appears sufficient. Together with the over-reservation of contiguous allocators, this memory fragmentation can waste 60 to 80 percent of KV-cache memory in naive implementations (Kwon et al. 2023), severely limiting the concurrent batch size that determines serving throughput.

PagedAttention,[25] introduced in vLLM, resolves this fragmentation by applying operating system virtual memory paging to accelerator memory (Kwon et al. 2023). In static contiguous allocation, reserving maximum sequence length memory up front causes internal waste, while freed regions may be too small or poorly placed to satisfy a new contiguous reservation. Instead of allocating one contiguous region per sequence, PagedAttention divides the KV cache into fixed-size blocks mapped through a block table. A sequence's cache can therefore occupy noncontiguous physical blocks. When a sequence completes, its blocks return to a free pool for reuse. Rather than copying noncontiguous blocks into a temporary contiguous buffer before computing attention, a specialized PagedAttention CUDA kernel traverses the block table directly, staging noncontiguous blocks into on-chip SRAM during query-key multiplication. By eliminating external fragmentation and bounding internal waste to each sequence's last, partially filled block, PagedAttention reduced KV-cache waste below 4 percent in the reported evaluation and enabled substantially higher batch concurrency on the same accelerator hardware (Kwon et al. 2023). Paging decides where cache blocks live, however; it does not reduce how many blocks each sequence needs, and that number grows with every generated token.



PagedAttention recovers the KV-cache waste of contiguous allocation.

[25] **PagedAttention**: The name directly references OS virtual memory paging, first implemented on the Atlas computer at Manchester (1962) to solve the same class of allocation problem, in which programs needed more memory than physically available, and contiguous allocation wasted space (Kilburn et al. 1962). Introduced at SOSP 2023, PagedAttention applies this six-decade-old abstraction to GPU KV-cache memory; in its evaluation, the recovered memory enabled 2–4× higher throughput on the same hardware (Kwon et al. 2023).

13.9.5 Memory and KV cache

Even after PagedAttention eliminates fragmentation waste, thirty-two long-context conversations can exhaust an 80 GB GPU's memory when the quantized model weights themselves occupy less than half the device. The KV cache, the per-sequence store of attention keys and values defined in section 6.7.4.2, grows linearly with sequence length: prefill creates cache entries for every prompt token, and each decoded token appends another entry, so a request's footprint is set by its prompt length plus the tokens it has generated so far. Summed across concurrent variable-length sequences, this dynamic state can exceed the model weights and dominate available accelerator memory, which is why the remaining memory levers either reuse cache state across requests or move it off the accelerator.

13.9.5.1 KV budget

Once contexts grow, the cache rather than the weights sets how many conversations one accelerator can hold, and Llama 3 8B makes the budget concrete. Each token stores a key vector and a value vector in every layer for every key-value head. Storing 16-bit Key and Value tensors across $N_L = 32$

layers with $H_{\text{KV}} = 8$ grouped-query attention heads of dimension $d_{\text{head}} = 128$ requires $2 \times 32 \times 8 \times 128 \times 2$ bytes $= 131{,}072$ bytes ≈ 131 KB per token per sequence. Grouped-query attention (Dubey et al. 2024) lets the model's 32 query heads share those key-value heads, which cuts the cache by 4× relative to full multi-head attention. An 8,192-token conversation therefore holds 1.07 GB. With 16.1 GB of FP16 weights resident on an H100 whose 80 GB datasheet capacity is 85.9 GB in the decimal units used for tensor footprints, about 69.8 GB remains for caches before runtime workspace, room for about 65 such conversations at once. The weights are a fixed cost; every additional conversation, and every additional thousand tokens of context, draws on the same remainder.

Larger models make the cache the binding constraint sooner. For a 70-billion-parameter-class model with grouped-query attention, 4-bit weights consume 35.3 GB of the same H100, leaving 50.6 GB for FP16 cache before runtime workspace. A batch of 32 conversations exhausts that budget at roughly 4,800 tokens of context, so serving engines trade concurrent batch size against context length to avoid out-of-memory failures (figure 13.8).

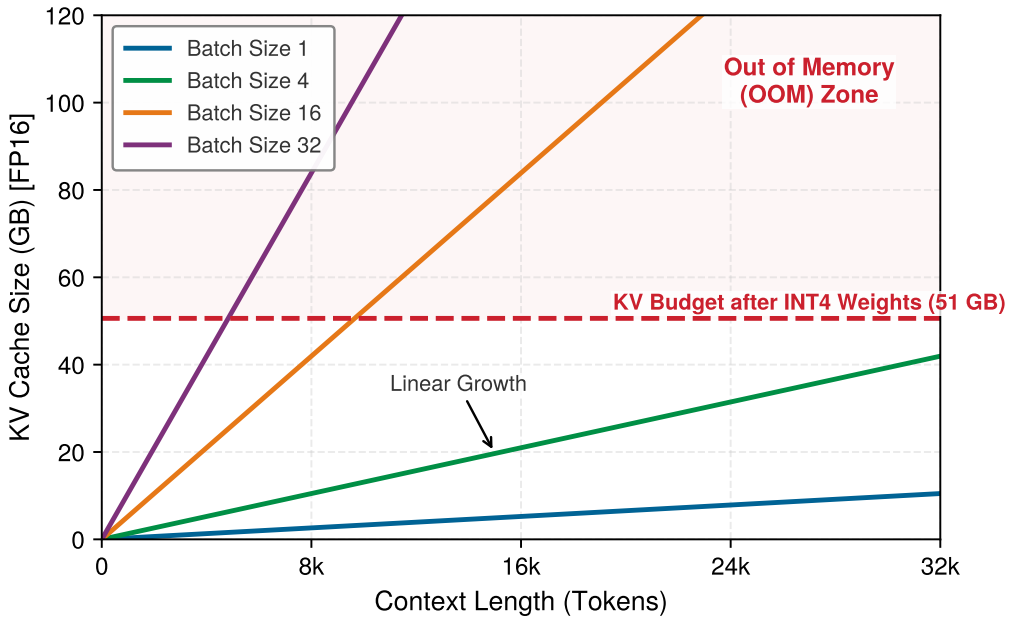


Figure 13.8: The KV-Cache Explosion: FP16 KV-cache usage vs. context length for a 70-billion-parameter-class model with GQA. On the modeled H100's 80 GB of HBM (85.9 GB decimal), 35.3 GB of INT4 weights leave 50.6 GB for KV cache before runtime overhead. Batch size 32 reaches that cache budget at roughly 4.8k tokens, forcing a trade-off between batch size and context length.

A model whose weights and cache do not fit one accelerator, even at reduced precision, can still be served from one machine by splitting it across the node's accelerators with the model parallelism of section 8.7.1.2. Serving usually favors tensor parallelism. Pipeline stages also fit, but every decoded token must pass through each stage in turn, which adds latency to every step. Tensor parallelism instead puts every accelerator to work on every layer, and each accelerator holds a slice of the weights and of the KV cache, so a 70-billion-parameter-class model at 16-bit precision, whose weights alone exceed one H100's memory, fits across the accelerators of one node with room left for cache. The cost is a collective operation after each layer's matrix multiplications. On NVLink that exchange is small next to the layer's compute; over PCIe or a network it would dominate each decode step. A serving engine therefore keeps one model's tensor-parallel group inside a single node and meets higher volume by replicating whole groups across machines rather than stretching one group across them.

13.9.5.2 Prefix caching and memory offloading

Many requests carry identical prefixes, and recomputing their cache entries wastes both prefill compute and memory. **Prefix caching** stores the KV states of common instruction prefixes (such as a

2,000-token system prompt or a shared retrieval-augmented generation (RAG) context), allowing independent requests to reuse the prefill result for an identical prefix. For n_{req} requests sharing a prefix of S_{prefix} tokens, the avoided prefill work is roughly $(n_{\text{req}} - 1)S_{\text{prefix}}$ token steps, subject to cache hits, eviction, and implementation overhead. For multi-turn conversations, the same principle lets a subsequent turn process only the new suffix after reusing compatible prior state.

When the aggregate KV cache exceeds GPU VRAM, systems can employ **KV cache offloading**. This strategy spills inactive or low-priority context windows to host CPU RAM or NVMe SSD, freeing VRAM for active generation. The reload cost is bounded below by bytes moved divided by PCIe or NVMe bandwidth, before software overhead and queueing are added. Offloading therefore prevents OOM failures and enables larger context windows, but it also creates affinity, invalidation, and hot-decode latency risks that must be budgeted explicitly.

13.9.6 Session affinity constraints

Prefix reuse and offloading both assume that the next turn of a conversation reaches the replica that holds its cache. Session affinity, routing successive requests from a client to the same physical replica, arises whenever serving state resides in local accelerator memory rather than being reconstructed on every invocation. In conversational models, this resident state is dominated by the KV cache. When turn i executes on a replica, that replica's HBM retains the projected keys and values across all prior prompt and generation tokens. Routing turn $i+1$ to a different replica discards that local memory footprint, forcing the new node to recompute the entire prefix sequence from scratch. This redundant prefill consumes memory bandwidth and tensor-core cycles that would otherwise advance active decode iterations, directly inflating time-to-first-token.

A parallel physical constraint governs personalized serving using fine-tuned Low-Rank Adaptation (LoRA) modules: small per-user adapters, each a pair of low-rank matrices whose product is added to a frozen base weight matrix. While base-model weights remain pinned in device HBM, individual user adapters must be dynamically staged into memory. If incoming requests for a user scatter across multiple replicas, each node must repeatedly transfer adapter tensors over the host-to-device PCIe bus or fetch them across the network, displacing other active working sets in memory. Stateful preprocessing pipelines, such as tokenizer vocabularies and per-session normalization state, impose the same penalty, requiring redundant CPU-side allocations and cache rebuilds whenever execution moves across node boundaries.

Strict affinity creates a direct conflict with dynamic batching. Continuous batch schedulers maximize accelerator throughput by packing concurrent requests into unified matrix multiplications, raising operational arithmetic intensity in proportion to batch size ($I \propto B$). When requests are pinned to designated replicas, the scheduler's pool of batchable candidates fragments across the cluster. A replica assigned to an idle or low-concurrency user session must either process underfilled batches, collapsing arithmetic intensity back into the memory-bandwidth-bound regime, or stall waiting for affinity-matched tokens. To prevent this collapse in utilization, production routers use soft affinity. Under soft affinity, the ingress gateway prioritizes the cache-holding replica when capacity allows, but sheds requests to alternate nodes when local queue delays exceed the physical latency required to transfer adapter weights or recompute prefix tokens.

↩ Products of two low-rank matrices, and the storage and memory traffic they save relative to a full matrix, are established in section 10.2.2.1.

> **Systems Perspective 13.6: LLM serving: Beyond the fundamentals**
>
> Each LLM serving technique answers one of two physical limits. The first is the decode bound, in which every generated token pays for a full weight stream at small batch. Continuous batching keeps decode batches full so each stream serves many sequences, chunked prefill keeps prompt processing from stalling that stream, and speculative decoding extracts more than one accepted token per stream. The second is the KV cache, which grows with sequence length and batch size and often exceeds the weights themselves; PagedAttention, prefix caching, offloading, and session affinity decide where it lives and how much of it can be reused. Weight-only quantization (such as INT4 weights with FP16 activations), the precision lever of section 13.6.3 applied to the weight stream, addresses both at once, because it shrinks the bytes streamed per token and frees capacity for cache (Lin et al. 2024).

These LLM-specific optimizations build directly on core serving mechanics: queueing models govern scheduling timeouts, batching dynamics determine throughput-latency curves, and reduced precision bounds memory traffic. Advanced treatments extend these mechanics to multi-tenant prefix sharing across fleets and multi-node tensor-parallel execution.

Every technique above changes how many sequences share each weight stream and how long each one holds its cache. Because an accelerator draws power whether a decode pass serves one sequence or a full batch, that same sharing directly governs the energy each generated token costs. Dividing node power by aggregate token throughput converts batching and prefix-caching gains into joules per token.

Napkin Math 13.5: The energy cost of a chat

Problem: How much energy does an H100-backed chat service spend per generated token and for a response with 500 tokens?

As LLMs scale, joules per token becomes a first-class operational metric alongside latency. Assuming the H100 draws its 700 W thermal design power (TDP), this scenario estimate follows from throughput and power (Choquette 2023):

1. **Throughput**: 114 concurrent requests $\times$ 8 tokens/s per request $\approx$ 912 tokens/s.
2. **Power**: 700 W (GPU) + 300 W (Host/Overhead) = 1000 W.
3. **Energy per token**: 1000 W / 912 tokens/s $\approx$ 1.0965 J/token

Systems insight: Under these assumptions, a response of 500 tokens consumes $\approx$ 548.2 J. A measured production figure is of the same order. Google reports $\approx$ 864 J (0.24 Wh) for the median Gemini text prompt, counting host, idle-reserve, and facility energy, although the study does not state how long that prompt was (Elsworth et al. 2025).

- For comparison, charging a smartphone consumes $\approx$ 40000 J.
- Boiling a cup of water consumes $\approx$ 100000 J.

The primary way to reduce J/token is to increase hardware utilization and eliminate redundant compute. If the GPU sits at 10 percent utilization due to poor batching, it still draws ~300 W and the host adds 300 W, causing energy per token to rise to 6.6 J/token (approximately 6× the baseline). Architectural optimizations like prefix caching also skip the energy-intensive prefill phase for shared context, directly reducing the energy footprint of retrieval-augmented generation (RAG) and chat applications. The serving lesson is that efficiency is not only a latency or cost metric; it also determines how much energy each useful token consumes.

Systems Perspective 13.7: Looking ahead: Spending more compute per query

Every lever so far takes the number of tokens a request generates as given, and fixed-output serving minimizes latency ($L_{\text{lat,total}} \to 0$) for a fixed amount of work per request. Reasoning models and other inference-time-compute systems make that token count a choice: they deliberately spend more computation per query to improve answer quality, generating intermediate reasoning or search tokens before the visible answer. The optimization target then expands to include answer quality, and every extra reasoning token pays the decode costs this section derived: one more step of decode work at the batch's cadence, one more KV-cache entry held until the response ends, and its share of the joules per token. Answer quality now trades directly against TPOT, cache capacity, and energy per query.

Checkpoint 13.5: LLM serving fundamentals

LLM serving introduces constraints absent from traditional model serving.

- ☐ **TTFT vs. TPOT**: Contrast the distinct user experiences captured by TTFT and TPOT, explain why long-prompt prefill often exposes more arithmetic intensity than low-batch decode, and convert a 50 ms TPOT target into its tokens-per-second equivalent.
- ☐ **Continuous batching and PagedAttention**: Contrast static batching waste under variable sequence lengths against iteration-level scheduling, and explain how PagedAttention eliminates external fragmentation in the KV cache and bounds internal waste to a single partially filled block per sequence.
- ☐ **Memory wall**: Explain why adding compute units yields negligible latency improvement when token generation is bandwidth-bound, and identify how higher-bandwidth memory, model quantization, or batching alters the operating regime.
- ☐ **Prefix caching and offloading**: Explain how caching key-value activations for shared prompt prefixes eliminates redundant prefill compute, and evaluate the latency trade-offs that govern offloading KV caches to host system memory.
- ☐ **Joules per token**: Explain why energy consumption scales with generated sequence length, and how low hardware utilization inflates the energy cost per useful token.

Every LLM serving optimization ultimately decides how many sequences share each weight stream and how much accelerator memory each sequence holds. The chat-energy calculation showed that the same sharing sets the joules spent per useful token, because each decode step streams the weights once for the whole active batch, so a well-filled batch divides that energy across sequences. Unlike fixed-output inference, LLM energy also grows with response length. Carbon accounting additionally requires an emissions factor and a defined boundary. These per-token bounds describe one accelerator. Deciding how many accelerators a service needs, and what each generated token costs, requires carrying them, together with the fixed-output results of the earlier sections, into capacity planning and unit economics.

13.10 Economics and Planning

A team that has raised an H100's ResNet-50 throughput several-fold through batching still faces a fleet-level question: does serving a 5,000 QPS target within the same p99 latency SLO cost less on a few H100s or on several times as many lower-cost T4s? Batching, precision tuning, kernel fusion, graph compilation, and the LLM levers that share each weight stream across sequences all improve single-node performance, but production deployment requires choosing how many machines to rent, of what type, and at what total bill. Serving infrastructure cost grows continuously with request volume (Zhang et al. 2019), while training cost follows a one-time dataset, model, and optimization budget. The bitter lesson sharpens this asymmetry. The methods that win are the ones that scale, so successive models grow larger, and while training pays for that scale once, serving pays for it on every request.

Tail latency dictates the technical design of a serving system, but economics dictates its operational scale. As specialized accelerators and runtime optimizations reduce the compute cycles required per forward pass, the marginal cost per inference declines.[26] Counterintuitively, reduced unit cost does not shrink aggregate serving expenditures. In accordance with Jevons paradox, lower marginal costs stimulate disproportionately higher query volumes, integrating model evaluation deeper into user-facing product loops. At hyperscale, recurring operational inference costs quickly eclipse one-time offline training investments, shifting the primary systems bottleneck toward sustained per-query efficiency.

↩ The bitter lesson, that general methods which scale with computation keep outperforming hand-engineered ones, is established in section 1.4.

[26] **Jevons paradox**: William Stanley Jevons observed in 1865 that efficiency improvements in coal-powered steam engines increased total coal consumption by making steam power economically viable for applications previously too costly (Jevons 1865); the same dynamic can apply to AI inference, where each 10× cost reduction opens application classes that were economically infeasible at the previous price point, expanding aggregate demand by more than the efficiency gain.

Example 13.12: Inference cost at Facebook

Scenario: Production ML serving systems at Meta execute tens of trillions of inferences daily across global user-facing services (Hazelwood et al. 2018).

Diagnosis: Recurring inference serving compute and energy costs dwarf initial offline model training expenses. Live request arrival patterns and strict latency bounds constrain GPU batching.

Systems lesson: Model architectures cheap to train can prove unviable to serve at fleet scale. Production serving infrastructure requires hardware-software co-design to balance tail latency, batch size, and recurring operational total cost of ownership.

The same serving-economics pressure appears in public API prices. Figure 13.9 tracks this cost collapse across representative public API list-price snapshots from 2023 through 2025 on a logarithmic scale. The steep downward trend line isolates how prompt token pricing fell by multiple orders of magnitude, from thirty dollars per million tokens for original GPT-4 down to several cents for Gemini 1.5 Flash and DeepSeek-V3. Vendor prices change frequently, so these points should be read as historical provenance for the trend rather than as current purchasing guidance (OpenAI 2023b, 2023a, 2024; OpenAI Developer Community 2024; Anthropic 2024; Google Developers Blog 2024; DeepSeek 2024). Each order-of-magnitude drop changes which applications are economically viable, and a provider can sustain such prices only if its serving stack extracts more completed requests from each accelerator-hour. Converting engineering efficiency into unit cost draws on the latency, batching, and memory mechanisms of the preceding sections.

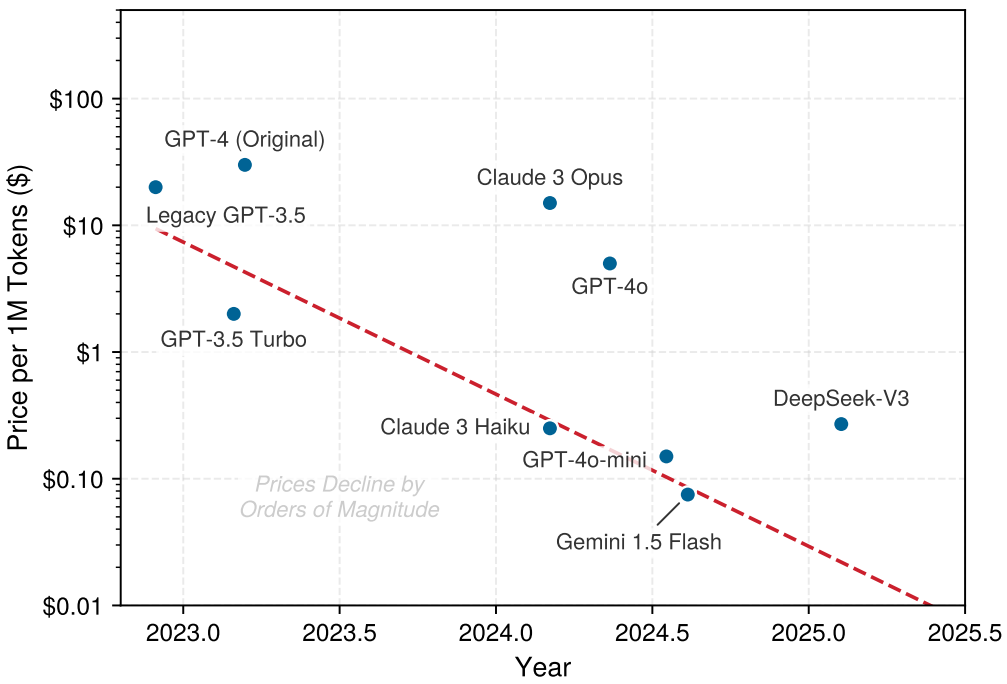


Figure 13.9: Intelligence Deflation: Representative public API input-token list prices per 1M tokens ($) over time (Log Scale). Nine model-version list prices come from the launch and pricing announcements of OpenAI, Anthropic, Google, and DeepSeek between late 2022 and 2025; the dashed line is a log-linear fit through the price floor, the points that each set a new low. The series is intended as a market trend indicator, not a controlled or current-price comparison. API token-processing prices have collapsed by multiple orders of magnitude, transforming the economics of automated AI workflows.

13.10.1 Cost per inference across hardware

Choosing serving instances by hourly rental price alone, or by peak throughput alone, misleads whenever traffic fluctuates: an H100 that costs several times as much per hour as a T4 delivers the lowest cost per million images when traffic keeps it full, but becomes the more expensive option when overnight traffic drops and the GPU sits mostly idle. The **cost per inference** divides the allocated serving cost by the number of completed inferences that satisfy the service objective. The numerator can include provisioned CPU or accelerator time, memory capacity, data transfer, storage, and orchestration. At high utilization, useful throughput spreads that provisioned cost across many requests. At low utilization, paid capacity sits idle, so the same hourly cost is divided among fewer successful inferences.

Hardware selection for model serving trades hourly rental price against parallel throughput. Although GPU instances cost several times as much per hour as commodity CPU instances, their matrix units can process enough requests per second to cut the cost per completed query by an order of magnitude. Whether CPUs or dedicated accelerators win economically depends on arithmetic intensity, batchability, and the latency deadline.

Host CPU execution is cheaper for small models, services with relaxed latency deadlines (hundreds of milliseconds), and sparse or unpredictable traffic that leaves dedicated accelerators idle. Conversely, GPU acceleration wins when models perform dense matrix operations, when service-level objectives require response times of tens of milliseconds or less, and when sustained traffic lets dynamic batching amortize kernel launch and memory transfer costs.

Beyond steady-state costs, startup time affects scaling economics. Instance launch, image pull, driver initialization, model loading, compilation, and warmup can make accelerator replicas slower to become ready than CPU replicas, although exact times depend on the platform and deployment artifact. If traffic rises faster than useful capacity can come online, latency SLOs can fail despite sufficient eventual capacity.

When accelerator replicas have a long readiness delay, predictive scaling or a warm pool may be more reliable than purely reactive scaling. For a model that runs acceptably on both device types, a hybrid design can use always-on accelerator capacity for baseline load and CPU overflow for spikes, trading a different unit cost and latency profile for faster reserve capacity. Whether this GPU+CPU hybrid works depends on model compatibility and identical output semantics across runtimes.

Example 13.13: ResNet-50: Cost analysis

Table 13.18 gives an illustrative scenario for hourly cost, throughput, and per-million-image cost when serving ResNet-50; the H100 row uses the batch-32 rate of the chapter's batching sweep:

Table 13.18: Illustrative ResNet-50 cloud inference cost comparison: Assumed hourly cost and saturated throughput, with derived cost per one million images for CPU, T4, and H100 instances.

Instance Type	Cost/Hour	Throughput	Cost per 1M Images
c5.xlarge (CPU)	$0.17	50 img/s	$0.94
g4dn.xlarge (T4 GPU)	$0.53	400 img/s	$0.37
One H100 GPU	$4	7,040 img/s	$0.16

Systems insight: At full utilization the H100 has the lowest unit cost, but its hourly rate is 7.5× the T4's, so it wins only while traffic keeps it busy. Below about 3,000 img/s of sustained traffic, T4 capacity sized to the load costs less per image than one partly idle H100. Cloud pricing varies by region and changes over time; consult current pricing for production planning.

13.10.2 Capacity planning

The unit-cost comparison in table 13.18 evaluates each instance at its saturated throughput, but a live fleet cannot run at 100 percent utilization without pushing every arrival burst past the knee of the latency curve (figure 13.5). Determining how many replicas to provision therefore combines

those per-device service rates with the queueing bounds from section 13.4. **Capacity planning** translates three inputs into replica counts: traffic patterns (peak request rate, daily and weekly cycles, growth projections), latency SLOs (p50, p95, p99 targets), and model service times across batch sizes (Harchol-Balter 2013).

The ResNet-50 capacity notebook in section 13.4.3 ran this workflow for batch-one service, deriving an M/M/1 utilization threshold of 68 percent from equation 13.5 and arriving at 36 H100s with headroom for a 5,000 QPS target. The plan extends to batched service by the same arithmetic. Applying the same utilization factor to the saturated batched rate of 7,040 img/s in table 13.18 gives 2 H100 before headroom and 3 after; the batch-1 plan needed 27 and 36.

This arithmetic compares assumed capacities under different service models; it does not establish equal p99 latency. Batching changes service times, realized batch sizes, and waiting distributions, so the factor derived for unbatched M/M/1 service cannot by itself certify the batched plan. The throughput opportunity is the batching gain of section 13.5; its usable capacity requires a load test including formation delay and worker backlog. This scenario provisions peak load plus 30 percent headroom; production margins depend on the workload and policy. Autoscaling can reduce costs during low-traffic periods while meeting latency objectives during peaks; section 14.4.1 develops the operational policy layer around these serving calculations. Capacity must be sized for requests that meet the SLO, not for raw request volume.

13.10.3 Production case study: Serving 8-billion-parameter Llama 3

When a team deploys an 8-billion-parameter Llama 3 chat service on a single H100 under a 200 ms TTFT and 20 ms TPOT budget, does GPU memory capacity, prefill compute, or decode bandwidth cap the number of requests per hour, and what does each million tokens cost? The serving bounds of section 13.9 become a dollar figure only when they meet a concrete workload, an SLO, and an hourly rental rate. This case study costs out that single-H100 deployment, reusing the decode floor and the cache budget derived earlier rather than re-deriving them. Unlike the 70-billion-parameter scale-up in figure 13.8, the 8-billion-parameter model fits on one GPU with substantial memory headroom, so batching, prompt prefill, and autoregressive decoding can be balanced on a single device without multi-node sharding.

13.10.3.1 Workload profile

The following fixed workload assumptions define the reference case used throughout the latency, memory, and economics calculations; together, they bound the analyses that follow:

- **Model**: 8-billion-parameter Llama 3 (quantized to 4-bit using activation-aware weight quantization (AWQ); see section 10.3 for quantization techniques) (Dubey et al. 2024; Lin et al. 2024).
- **Hardware**: 1× NVIDIA H100 SXM5 GPU (80 GB HBM3 device capacity, equivalent to 85.9 GB on the decimal basis used for tensor footprints, 3.35 TB/s bandwidth) (Choquette 2023).
- **Request characteristics**: 1,000-token input prompt (Prefill), 256-token generated response (Decode).
- **Target SLOs**: TTFT $<$ 200 ms, TPOT $<$ 20 ms.

13.10.3.2 Latency deconstruction

The end-to-end request latency is governed by the two-phase execution model of autoregressive transformers, applying the TTFT and TPOT metrics defined in section 13.9.1. Prefill determines whether the user sees a prompt response quickly; decode determines whether the generated stream keeps moving after it starts.

Prefill phase (time to first token) The model processes the 1,000-token prompt in parallel. Assuming 40 percent of peak compute efficiency during prefill, the model-side TTFT estimate is 41.9 ms, comfortably within the 200 ms SLO and leaving the remaining budget for network ingress, tokenization, and queueing.

Decode phase (time per output token) The model generates 256 tokens sequentially. At batch one, this phase is memory-bandwidth bound: each step streams the 4 GB weight tensor from VRAM. A batched step amortizes that stream across its active sequences.

Dividing the 4 GB weight tensor by the 3.35 TB/s bandwidth yields the theoretical floor: $T_{\text{token}} \approx$ 1.2 ms. Adding an assumed residual per-layer dispatch cost after graph capture gives a modeled TPOT of 1.57 ms. This dispatch overhead reflects the cumulative cost of launching 8–10 discrete GPU kernels (RMSNorm, RoPE, projections, attention, multilayer perceptron (MLP)) across each of the model's 32 layers. At batch size one, launching $ $250–300 small kernels adds several milliseconds of CPU-GPU submission and driver overhead, which serving engines mitigate by capturing the decode graph into a single CUDA Graph. Generating all 256 tokens therefore takes 256 tokens $\times$ 1.57 ms = 0.40 s.

13.10.3.3 Memory and throughput

With 4-bit weights occupying 4 GB, the remaining ~81.9 GB is available for the KV cache and runtime workspace, and the case study sets aside the cache reserve below while leaving the rest as workspace headroom. AWQ compresses the weights to 4 bits but leaves the KV cache in 16-bit precision, so each token still costs the approximately 0.131 MB of FP16 cache derived for this model in section 13.9.5.1. Dividing the 77.3 GB reserve by that per-token cost yields capacity for ≈ 0.6 million tokens, so at 1,256 tokens the GPU can hold a concurrent batch size of ~469 requests.

13.10.3.4 Unit economics

Consider a representative H100 SXM5 rental cost of approximately $3/hour and an assumed isolated prefill rate of 10,000 prompt tokens/s, deliberately below the rate implied by the model-side TTFT estimate above to allow for scheduling and kernel inefficiency. Dividing that rate by the 1,000-token prompt gives the prefill request capacity. For this calculation, each completed request counts both prompt and generated output tokens, totaling 1,256 tokens. Multiplying the limiting request rate by this token count and the seconds in an hour gives 45.2 million tokens/hour; dividing the hourly rental cost by that throughput gives $0.066/million tokens.

The comparison also converts KV residency into a request rate by dividing the resident request count by the modeled time to complete prefill and decode. It assumes that all those resident requests can advance at the modeled per-request pace, which is an optimistic concurrency assumption rather than a measured decode capacity. Under these assumptions, isolated prefill supplies the smaller rate and sets the displayed cost floor. Shared prefill/decode contention, batched decode timing, runtime workspace, and SLO-qualified throughput determine whether the service realizes that rate; memory fit alone does not establish it. The resulting cost floor is a bound for this workload, and a load experiment must supply the missing service capacity before the same arithmetic can price an accepted deployment configuration.

Checkpoint 13.6: Complete the LLM capacity decision

- ☐ **Define acceptance**: For the case study's 200 ms TTFT and 20 ms TPOT targets, state the required completion quantiles, availability requirement, and memory reserve before evaluating a configuration. Why do latency thresholds alone leave the SLO incomplete?
- ☐ **Measure the missing capacity**: Hold runtime, precision, hardware, and prompt/output lengths fixed. Increase offered request rate across several concurrency limits below the residency ceiling. For each warmed configuration, record TTFT and TPOT distributions, completed request rate, rejection rate, and peak allocated memory. Identify the configurations that satisfy all acceptance conditions.
- ☐ **Close the economic decision**: Recompute cost using each qualifying configuration's completed request rate and the case study's prompt-plus-output token convention. Select the lowest-cost qualifying setting for the required load. If none qualifies, identify whether lower offered load, a different batching/runtime policy, or different hardware is the next experiment. Explain why the KV-residency ceiling cannot supply the missing throughput observation.

For large language models, memory capacity (the size of the unallocated VRAM dedicated to the KV cache) determines maximum concurrent residency, while prefill compute and decode bandwidth determine realized token throughput and cost under a specific traffic mix. Memory bandwidth bounds low-batch decode latency; larger batches amortize parameter movement and shift the limit toward compute or KV-cache traffic. Physical constraints govern serving economics across the entire stack: latency budgets partition into prefill and decode phases, queueing theory dictates batch sizing and capacity headroom, and memory bandwidth bounds decode throughput. Misinterpreting these interactions leads directly to the operational fallacies and pitfalls that follow.

13.11 Fallacies and Pitfalls

Serving inverts training priorities in ways that violate intuitions from batch processing. The nonlinear relationship between utilization and latency, the hidden costs of preprocessing, and the silent failure modes of training-serving skew cause violated SLOs, wasted optimization effort, and accuracy degradation invisible to standard monitoring.

Fallacy: *Reducing model inference latency proportionally reduces user-perceived latency.*

Engineers who optimize model inference expect proportional improvement in user-perceived latency, but serving systems introduce latency sources absent from offline benchmarks. Under load, queuing delay dominates: equation 13.4 shows that at 80 percent utilization with 3.5 ms service time, average wait time is 14.1 ms before inference even begins. Reducing inference from 3.5 ms to 1.8 ms changes service time but also shifts utilization from 80 percent to 41 percent, reducing queuing wait from 14.1 ms to 1.2 ms, a 12× queuing improvement that dwarfs the 1.7 ms inference gain.

This nonlinear interaction between inference speed and queuing behavior produces an apparent paradox: the system-level speedup (17.6 ms $\rightarrow$ 3.0 ms, or 6×) far exceeds the model-level speedup (3.5 ms $\rightarrow$ 1.8 ms, or 2×). At first glance, this superlinear improvement appears to contradict Amdahl's law, which states that total system speedup is strictly bounded by the enhanced execution fraction. However, the fixed-fraction Amdahl calculation holds other stage costs unchanged; serving is a dynamic queueing system. Average queue waiting time scales nonlinearly with server utilization: $W_q \propto \frac{\rho_{\text{serv}}}{1-\rho_{\text{serv}}}$, where $\rho_{\text{serv}} = \lambda_{\text{arr}} \cdot T_{\text{svc}}(1)$ for unbatched service. When kernel optimization cuts service time $T_{\text{svc}}(1)$, it does not merely accelerate execution; it simultaneously lowers server utilization ρ_{serv}. Pulling utilization away from saturation collapses queue waiting times nonlinearly. A kernel optimization therefore shortens the request twice, once in the service time and again in the queue that the lower utilization drains. Conversely, profiling inference latency in isolation without modeling arrival load conceals the primary driver of user-perceived performance.

Pitfall: *Running serving infrastructure at high utilization to maximize cost efficiency.*

Teams target 90 percent utilization to minimize idle capacity. In production, latency degrades nonlinearly as utilization approaches capacity. Equation 13.4 shows that at 90 percent utilization, average time in system reaches 10× service time. Moving from 70 percent to 90 percent utilization cuts modeled infrastructure cost by 22.2 percent but triples average latency. For a 3.5 ms inference service, modeled p99 latency rises from ~54.1 ms to ~162 ms under the M/M/1 assumptions. In this scenario, operating at 60 to 70 percent utilization preserves substantially more latency headroom than 90 percent. A production target must instead be selected from the measured arrival process, service-time distribution, SLO, failure reserve, and cost objective.

Fallacy: *Training accuracy guarantees serving accuracy.*

Engineers assume identical model weights preserve validation-set performance. In production, preprocessing differences can shift inputs outside the training distribution. Section 13.8.5 explains how resize interpolation, numerical precision, and feature timing can create skew despite identical weights. Validation accuracy holds only over the population it sampled (principle 9), and a serving path that transforms inputs differently presents the model with a different population. The illustrative scenario here drops from 95 percent to 90 percent, a 5 percentage-point loss; these values are not a reported production incident. Production systems require consistent preprocessing and task-quality monitoring because latency and exception checks cannot detect this failure.

Pitfall: *Using average latency to evaluate serving system performance.*

Engineers monitor average latency because it trends smoothly and is simple to compute. An average, however, cannot show whether the percentile named by an SLO is failing. As section 13.4.3 demonstrates, at 70 percent utilization with 3.5 ms service time, average latency is 11.8 ms but the modeled p99 reaches 54.1 ms, a 4.6× gap invisible to mean-based monitoring. Services therefore report the relevant latency distribution and SLO percentile rather than the mean alone.

Fallacy: *Larger serving batches always improve throughput without affecting latency SLOs.*

Engineers maximize batch size assuming GPU saturation improves cost efficiency under production load. In serving systems, however, batching introduces a latency-throughput trade-off governed by queuing dynamics absent from offline benchmarks. Accumulating requests into larger batches increases wait time for early arrivals. An oldest-request window of 10 ms can add up to 10 ms of formation wait if the batch-size trigger does not fire first. Waiting for an available worker can add further delay; the combined distribution determines p99 latency. For the chapter's ResNet-50 profile on the H100 (section 13.5.5), increasing batch size from 16 to 32 raises saturated throughput by 77 percent and per-batch inference time only from 4.0 ms to 4.5 ms; at 500 QPS, however, filling the larger batch raises mean formation wait from 15 ms to 31 ms, and variable input sizes within a batch can create padding overhead that wastes compute on padding tokens. Section 13.5.4 shows why, for tight p99 targets, larger batch sizes can violate SLOs when batch formation delay plus increased per-batch inference time exceeds the latency budget. Serving batch optimization requires jointly tuning batch size, batch timeout, and concurrency against latency SLOs under realistic traffic patterns, not maximizing throughput in isolation.

Pitfall: *Calibrating quantization ranges with unrepresentative data.*

Teams may calibrate with a convenient sample even when it does not represent the supported deployment distribution. Post-training quantization determines INT8 scale factors from activation ranges in calibration data, so representative coverage matters. The illustrative scenario here drops from 76.1 percent to 72.9 percent, a 3.2 percentage-point loss; it is not a reported production incident. Effective quantization requires calibration and validation on data representative of the inputs the service is expected to handle.

Fallacy: *Cold start latency only matters for the first request.*

Engineers optimize steady-state latency assuming most requests hit warm instances. Cold starts compound during traffic spikes, deployments, and recovery, which is exactly when capacity is most needed. In this illustrative scenario, 10 instances each require 30 s of compilation, totaling 300 instance-seconds; parallel warming makes capacity useful after about 30 s, and a request that reaches a replica before warmup completes waits about 500 ms instead of 5 ms. Autoscaling policies that count only steady-state replicas therefore underestimate the capacity a spike requires. The scale-out budget must include compilation, weight loading, cache initialization, and health-check time (section 13.8.1), and warm pools of pre-initialized spare replicas, precompiled engines, staged model loading, or admission control keep new replicas from becoming another source of tail latency.

13.12 Summary

Serving moves a validated model into an operating workload. Online requests add a latency distribution, queueing, traffic variability, and failure reserve to the throughput concerns familiar from training. Little's Law relates stable long-run population, arrival rate, and response time; M/M/1 supplies an intentionally simple model of nonlinear latency growth near saturation. Neither replaces measurement, but together they turn headroom and capacity into explicit hypotheses that a load test can validate.

Effective serving optimization measures the complete request path rather than model inference alone. Protocol and serialization choices matter when transport is a visible fraction of the budget; preprocessing matters when it feeds an accelerator too slowly; batching matters only when arrivals and the SLO permit waiting. Training-serving skew adds a different failure mode. The service can remain fast and error-free while inconsistent transformations silently change its predictions.

The chosen deployment paradigm changes the feasible request path. A cloud API may use dynamic batching under a measured arrival process, synchronized sensors impose a frame deadline,

and a single-stream mobile workload may choose batch one to protect responsiveness and energy. The MLPerf scenarios provide standardized approximations of these shapes; production design still begins with the service's own traffic, hardware, and objective.

Node-level optimization connects the model's operations to the machine that executes them. Profiling reveals whether compilation, kernel fusion, precision, locality, serialization, or loading actually binds the request path. Gains matter only when they survive production input shapes, accuracy checks, and latency variation. Token-streaming language models stretch the whole toolkit at once. On the chapter's GPT-2 XL service, a batch-one decode step can be no faster than the 0.9 ms an H100 needs to read the model's weights. Latency splits into time to first token and time per output token, batching must reschedule at every generated token, and the KV cache, not arithmetic, bounds how many requests can share an accelerator. Capacity planning converts that valid throughput into replica count and unit cost, while the Llama 3 case study in section 13.10.3 separates calculable capacity bounds from the load experiment needed to establish valid throughput. A faster kernel that neither reduces provisioned capacity nor improves the SLO has not produced an economic gain.

Every row of the chapter map (table 13.1) reduces to a few counting rules, collected in table 13.19. Each formula links an observable workload parameter—arrival rate, service time, batch window, weight volume, or hourly node rate—to the operational metric it governs. Together they let an engineer estimate a service's latency budget, headroom, batch size, decode floor, and cache budget before a load test confirms them.

Table 13.19: Key Equations: The chapter's counting rules in the notation of the sections that derive them. The queueing rows assume Poisson arrivals and a single server, the batching rows assume an available worker at dispatch, and the decode floor assumes batch-one decode launched as one captured graph; each is a planning estimate that a load test must confirm.

Quantity	Formula	Where derived
Requests in the system	$Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}}$	equation 13.3
Mean time in system (M/M/1)	$T_{\text{lat}} = \frac{\text{service time}}{1-\rho_{\text{serv}}}$	equation 13.4
p99 time in system (M/M/1)	$T_{\text{lat,p99}} \approx \frac{4.6 \cdot \text{service time}}{1-\rho_{\text{serv}}}$	equation 13.5
Count-triggered batch latency	$L_{\text{lat,batch}} \approx \frac{B-1}{2\lambda_{\text{arr}}} + T_{\text{svc}}(B)$	equation 13.6
Mean wait in a fixed window	$\mathbb{E}[L_{\text{lat,wait}}] = \frac{T_{\text{window}}}{2}$	equation 13.8
Saturated batched capacity	$\mu_{\text{eff}}(B) \approx \frac{B}{T_{\text{svc}}(B)}$	equation 13.10
Tail batch envelope	$L_{\text{lat,envelope}} = T_{\text{window}} + T_{\text{svc}}(B_{\text{req,p99}})$	equation 13.12
Decode floor per token	$T_{\text{token}} \approx \frac{D_{\text{vol}}}{\text{BW}_{\text{accelerator}}}$	equation 13.14
Tokens per speculative round	$\frac{1-\alpha^{\gamma+1}}{1-\alpha}$	equation 13.15
KV cache per token	$2\, N_L\, H_{\text{KV}}\, d_{\text{head}}\, s_{\text{elem}}$ bytes	section 13.9.5.1
Cost per inference	Allocated hourly cost divided by completed inferences per hour that meet the SLO	section 13.10.1

Key Takeaways: Inverting every training priority

- **Serving is latency economics**: Training rewards throughput over long runs, but serving spends a fixed per-request budget across serialization, preprocessing, queuing, inference, postprocessing, and the network. Optimizing only model latency misses the stages users actually wait on.
- **Utilization turns into waiting**: In the chapter's M/M/1 approximation, average time in system rises from 5× service time at 80 percent utilization to 10× at 90 percent. Headroom keeps modest traffic surges from becoming SLO failures.

- **Fast models expose the rest of the pipeline**: Once inference is fast, image decode, tokenization, and other preprocessing can dominate total latency. The binding optimization becomes the request path, not the neural network kernel.
- **Batching follows traffic, not habit**: Poisson-modeled web arrivals can use dynamic batching, synchronized sensors need aligned batches, and single-stream mobile workloads may have little batching opportunity. A useful window converts available latency slack into throughput without violating the SLO.
- **Skew breaks accuracy without errors**: Resize methods, color-channel order, normalization constants, or feature definitions that differ between training and serving can change the model's effective input distribution. Shared transformations, parity tests, and production-slice monitoring reduce this risk.
- **Runtime choices become infrastructure cost**: Precision, graph compilation, kernel fusion, and serving runtime translate directly into replica count and cost per inference. Quantization and specialized runtimes can materially reduce required serving capacity when they preserve accuracy and fit the target hardware.
- **LLM serving is memory management**: Decode often reads weights from VRAM for every generated token, so token latency is bandwidth-bound unless batching changes the constraint. KV-cache layout, PagedAttention, continuous batching, precision, and runtime choice determine both concurrency and cost per token.

Training is judged by useful work completed; an online service is judged by whether requests meet their latency objective. Each stage draws on the same latency envelope: serialization, preprocessing, the queue, the model, and the response. Data determines input size and arrival shape, the algorithm determines compute and state, and the machine determines service rate and movement costs. Queueing couples all three nonlinearly, so a system within budget at moderate traffic can breach it after a small surge. Serving engineering therefore allocates the budget across the request path, preserves headroom for variability and failure, and verifies that the required fraction of requests finishes before the deadline, which is the latency budget principle (12) in operational form.

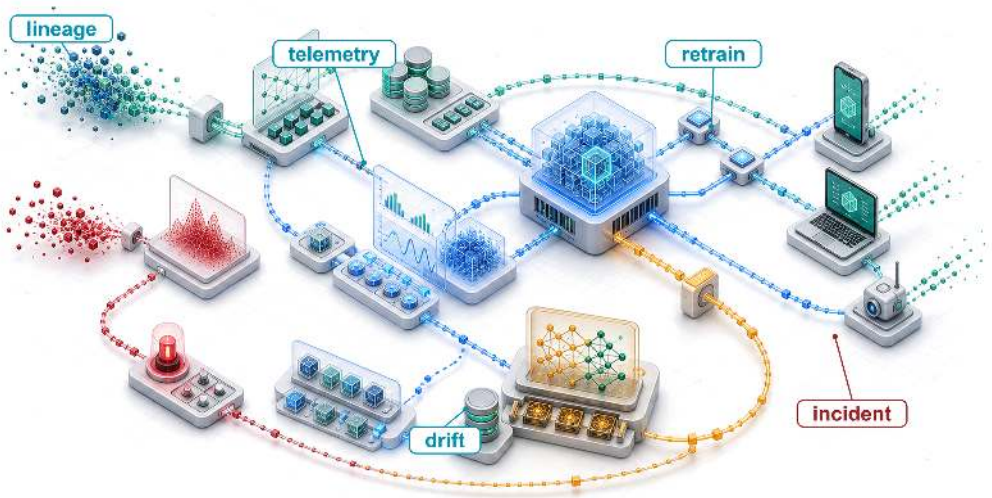


What's Next: From fast to still correct

This chapter evaluates the request path under a specified workload: latency within budget, headroom preserved, and useful throughput converted into cost. Those performance checks do not establish whether the answers are still right. Training-serving skew showed that a service can stay fast and error-free while its predictions change, and drift produces the same silence when the world moves rather than the pipeline. Chapter 14 widens the unit of concern from request-path performance to the model's operating lifetime. Monitoring tracks prediction quality alongside system health, feature stores and model registries keep serving aligned with training, and staged rollout with rollback lets a replacement model reach users without turning a bad release into an outage.

14

ML Operations

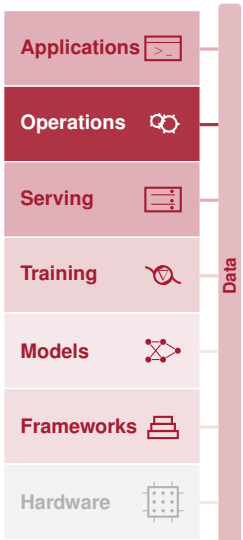


14.1 Silent Failure in Production
14.2 Technical Debt
14.3 Development Infrastructure
14.4 Releasing Models
14.5 Production Monitoring
14.6 Retraining Decisions
14.7 Incident Response and Governance
14.8 Operational Maturity
14.9 Case Studies
14.10 Fallacies and Pitfalls
14.11 Summary

Purpose

Why can an ML system be perfectly available and perfectly wrong at the same time?

Conventional monitoring often detects crashes, latency regressions, and availability failures quickly, but semantic errors can remain silent in any software system. ML systems add another silent failure mode. A model experiencing data drift can continue serving ordinary-looking predictions while accuracy degrades, triggering no alerts because every health check (latency, throughput, uptime) remains green. A fraud model can keep answering in milliseconds for months after the patterns it learned have moved elsewhere, and the first alarm may be the losses themselves. Model quality can degrade because the world changes. Customer behavior shifts, new product categories appear, seasonal patterns evolve, and the distribution the model learned from diverges from the one it now faces. Drift is not a certainty but a persistent risk for deployed models. Unlike latency or uptime, correctness may become observable only after labels or downstream outcomes arrive, so detecting degradation often requires imperfect signals and explicit investigation rather than a single threshold. Managing it requires continuous monitoring that tracks prediction quality alongside system health, drift signals that trigger investigation, retraining when outcome evidence supports it, and deployment strategies that validate new model versions against production traffic before full rollout. The gap between development and production is not a hurdle cleared once but a condition managed indefinitely, for as long as the model makes decisions that someone relies on. Machine learning operations exists because uptime without measured accuracy may still deliver wrong answers at scale. In D·A·M terms, operations makes co-design continuous, because the data environment can remain a moving target long after initial deployment.



Learning Objectives

- Explain why ML systems can remain available while prediction quality silently degrades under distribution shift
- Diagnose technical debt across data-model, model-infrastructure, and production-monitoring interface boundaries
- Design model registries, parity tests, and CI/CD pipelines that preserve training-serving consistency and reproducible rollback
- Compare canary, blue-green, shadow, and rollback strategies by the release risk they contain and the accelerator memory they hold
- Design the operations loop's layered monitoring and incident response to trace drift, skew, and degradation to a data, model, or infrastructure cause
- Apply the retraining staleness model to choose cost-aware retraining triggers and intervals
- Evaluate operational maturity and investment using model criticality, operational risk, and organizational readiness

14.1 Silent Failure in Production

A serving cluster can return HTTP 200 on every request, stay well inside its tail-latency budget, and pass every infrastructure health check while the recommendations it serves lose relevance week after week. A **silent failure** is a production model that keeps answering every request on time, raises no error, and passes every health check while its answers grow worse. Any deployed model is exposed to it, because a model is a fit to the data it trained on and the world it serves keeps moving after training ends. Chapter 13 built the request path that delivers those answers, a service that meets its latency budget, holds headroom for bursts, and turns throughput into cost per inference. A benchmark establishes performance at a point in time, and serving infrastructure verifies that the required fraction of requests finishes within the latency budget. Neither check asks whether the answers are still right. Operations is the stage of the lifecycle (chapter 3) that has no finish line. It keeps a model correct for as long as anyone relies on its decisions. The team deploys to production, and week one looks excellent. The challenge begins in week two.

Data distributions shift, user behavior changes, and the world moves on from the conditions under which the model was trained. ML models that succeed in development can fail to achieve sustained production use when postdeployment ownership and monitoring are absent. The root cause is an operational mismatch. Availability-focused monitoring tracks server uptime, request latency, and request success rates, while ML monitoring must also track statistical health, including accuracy over time, input-distribution shift, and per-segment prediction quality. A model can degrade substantially while throwing no exceptions, triggering no infrastructure alarms, and maintaining perfect uptime.

The degradation is measurable. In the Wild-Time benchmark, Yao et al. (2022) train a model on data from one period and test it both on held-out data from that period and on data from later years, across six real datasets that span yearbook portraits, satellite imagery, hospital records, news, and scientific papers. Figure 14.1 shows the result. On every dataset the same model scores lower on the later data, by 4 to 18.5 points of accuracy or of area under the curve (AUC), although nothing about the model or its serving path changed. The loss comes entirely from the world moving away from the training data, and only monitoring that compares predictions against fresh outcomes can see it.

The discipline that makes these invisible failures visible is **machine learning operations (MLOps)**. It inherits the automation and operations lineage of DevOps (Kreuzberger et al. 2023), but addresses a different failure mode. Conventional services can often be tested against deterministic code paths, while ML systems depend on training data distributions, learned parameters, and environmental conditions that shift continuously. Data distributions, learned parameters, slice-level evaluations, and runtime feedback loops therefore all function as release artifacts that govern production behavior, and a release extends far beyond an executable binary or code diff.

The operational mismatch connects directly to the book's analytical foundations. If benchmarking provides the *sensors* for the system, MLOps is the complete *control system*, and the degradation equation (equation 1.1) names the quantity it controls. Accuracy falls by a fitted sensitivity λ times

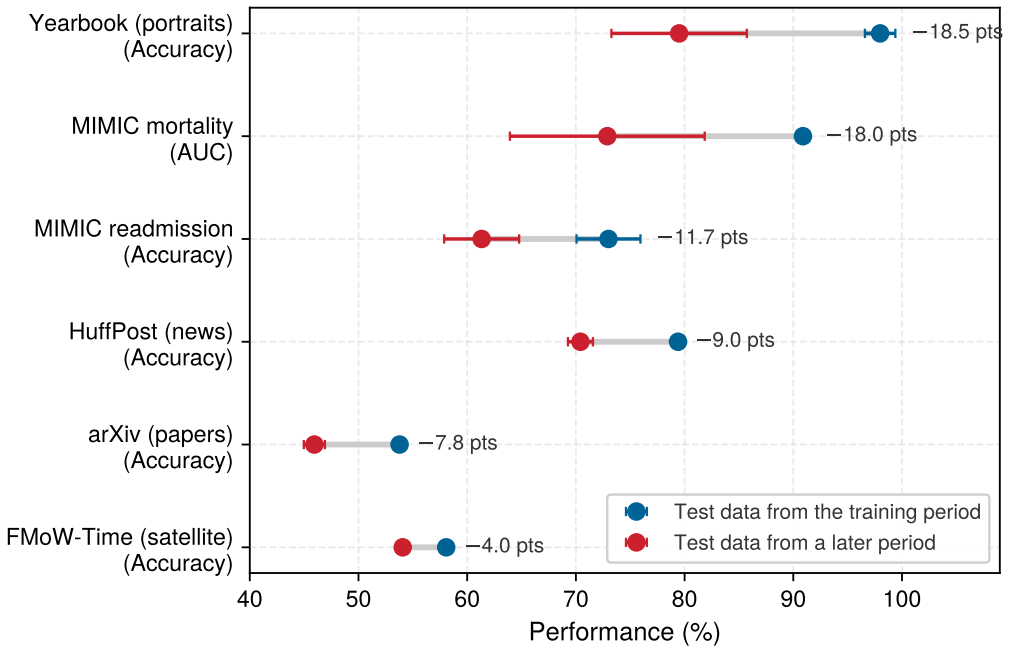


Figure 14.1: Measured Accuracy Loss over Time: For six datasets, performance of the same trained model on held-out test data from its training period (blue) and on data from a later period (red), with error bars showing one standard deviation over three training seeds. The label gives each drop in percentage points. MIMIC mortality is scored by AUC; the others by accuracy. Data from Yao et al. (2022).

the divergence between the serving and training distributions, so operations must measure that divergence continuously and estimate λ from paired drift and outcome data, because divergence alone does not prove that accuracy will fall. MLOps also formalizes interfaces and responsibilities across traditionally isolated domains (data science, machine learning engineering, and systems operations (Amershi et al. 2019)) through continuous retraining, A/B evaluation, graduated rollout, and standardized artifact tracking that supports reproducibility and auditability.

Definition 14.1: MLOps

Machine learning operations (MLOps) is the engineering practice of productionizing ML systems through CI/CD, workflow orchestration, reproducibility, versioning, collaboration, continuous training and evaluation, monitoring, and feedback loops (Kreuzberger et al. 2023).

1. **Significance**: Leaving the loop open shows up as stale predictions, silent accuracy degradation, and prolonged recovery times. Drift thresholds, retraining triggers, and mean time to recovery (MTTR) targets are deployment-specific, calibrated from prediction value, label delay, validation risk, and retraining compute expense. A retraining run is justified only when the expected accuracy gain outweighs its execution cost and rollout risk.
2. **Distinction**: DevOps monitors functional correctness, throughput, and availability. MLOps adds statistical controls for *predictive correctness*, which can decay while uptime, latency, and HTTP status codes stay healthy.
3. **Common pitfall**: A frequent misconception is that retraining on new data automatically resolves distribution shift. Retraining without diagnosing what shifted (the inputs the model receives, the relationship between those inputs and the correct answers, or both) preserves the underlying failure or amplifies collection bias. Reweighting the training

data can correct a change in input mix, but only new outcome evidence shows whether a changed input-output relationship requires adaptation; section 14.5.2 separates the two cases formally.

The loop has three stages (design, model development, and operations), but its defining feature is the return path. Runtime telemetry and retraining triggers carry live performance signals from operations back into requirements and data engineering, so deployment feeds the next design cycle instead of ending the project (figure 14.2).

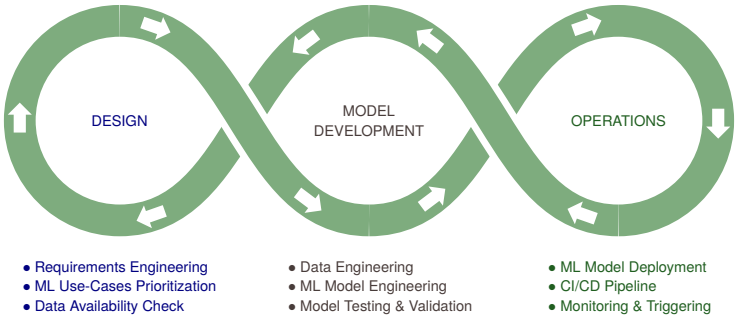


Figure 14.2: Iterative MLOps Loop: Monitoring and triggering feed production evidence back into design and model development, turning requirements, engineering, testing, and deployment into a continuous operating cycle.

The operations stage of that loop runs on every production model as five steps, which together form the **operations loop**. It detects a change in inputs, outputs, or outcomes; diagnoses which layer of the system caused it; decides whether retraining, rollback, or waiting is worth its cost; releases a replacement under controlled exposure; and recovers to a known-good version when a release misbehaves. Each step is a control that a team must build and pay for, and each consumes a concrete machine resource: telemetry bytes for detection, logged requests for diagnosis, accelerator-hours for a retraining run, and a second resident copy of the model for a safe release.

Deploying, monitoring, and maintaining one production ML system defines the fundamental operational unit: the **ML node**, a complete system comprising data pipelines, feature computation, model training, serving infrastructure, and monitoring for a single machine learning application. Platform operations at larger scale (managing hundreds of models, cross-model dependencies, multi-region coordination, and organization-wide ML platform engineering) constitute advanced topics that build on these single-model foundations.

Keeping one ML node correct decomposes into three interfaces that follow the artifact from data ingestion to live traffic and back again. At the start of the pipeline, the **data-model interface** governs the handoff between data infrastructure and model training, where the goal is feature consistency so that training and serving pipelines compute features the same way. Once a checkpoint is trained, the **model-infrastructure interface** carries those weights into a scalable service; its challenge is environment parity, because a model that works in a notebook can fail in production when versions, dependencies, or runtimes differ. Closing the loop, the **production-monitoring interface** returns statistical telemetry[1] from live serving back to training, enabling self-correction when distribution shift degrades accuracy without crashing the service.

These three interfaces determine how operational infrastructure partitions responsibility across the lifecycle. Feature stores stabilize feature computation at the data-model boundary, model registries and deployment pipelines preserve the model-infrastructure handoff, and drift monitors, retraining triggers, and governance policies connect production evidence back to a controlled response, shortening the window in which degradation goes unaddressed.

Two of the book's lighthouse workloads (section 1.7.1) carry the chapter, one for each end of the deployment spectrum. The cloud half follows a ranking service built on DLRM, the recommendation model that chapter 13 set against ResNet-50's compute-bound service. DLRM's home regime is capacity-bound. Its 100 GB of embedding tables exceed the 80 GB of one H100, so even its smallest

[1] **Telemetry**: The feedback path that can make model degradation visible before it becomes a business failure; crashes and error codes expose some software failures, but semantic errors can remain silent in any application. ML systems add distribution shift, which can go undetected without statistical telemetry such as feature distributions, prediction confidence, and drift indicators.

deployment spans two accelerators before throughput enters the calculation, and every operational control that needs a second copy of the model pays for it in memory. A ranking model is also the workload whose world moves fastest. User interests and the item catalog change daily, click labels arrive within hours, and the model's own rankings shape the clicks it later trains on. The edge half follows the keyword spotter, whose home regime is power-bound. Its 200 KB INT8 model listens continuously on a microcontroller with 4 MB of flash, where every radio transmission and flash write draws on a small energy budget. Section 4.12 left that spotter six months after launch with a detection rate four percentage points below its target, and the edge sections of this chapter pick up the drop. The device cannot be reached by hand, its audio cannot leave it, and a failed update must not disable it.

The week-two problem takes concrete shape on the ranking service in an illustrative scenario. Initial measurements show 94 percent click-prediction accuracy, p99 latency inside the service's budget, and strong performance across test segments. By week four, accuracy has dropped to 88 percent, but the infrastructure metrics show nothing wrong. By week eight, a product manager notices that recommended items convert poorly; investigation reveals the model has not adapted to a competitor's new promotion that shifted what users click. The model needed investigation six weeks ago, but no system was watching for this degradation. MLOps provides the framework to detect such changes, assess whether retraining addresses them, and validate replacement models while limiting the impact on users.

The chapter builds the operations loop in the order a team needs it (table 14.1). Technical debt explains why ML systems accumulate the hidden coupling that makes silent failure likely. Development infrastructure builds the pipeline that can rebuild and trace any model exactly, and releasing models moves a candidate onto live traffic with a tested path back. Production monitoring then watches the deployed model for the evidence the loop runs on, and retraining decisions turn that evidence into a choice to retrain, roll back, or wait. Incident response and governance handle the failures that monitoring surfaces, operational maturity decides which control a team should add next, and two case studies carry the loop to the edge and into the clinic.

Table 14.1: Chapter Map: Each section builds one step of the operations loop and names the machine resource that step consumes on the DLRM ranking service.

Section	Loop step	Question it answers	Machine cost on the ranking service
Technical Debt	Why the loop exists	Where hidden coupling accumulates	Glue code and unused features that still move bytes
Development Infrastructure	Release (prepare)	How a model is rebuilt and traced exactly	Versioned copies of data, features, and 100 GB of tables
Releasing Models	Release, recover	How a candidate reaches traffic and how it is withdrawn	A second resident copy of the tables for each release control
Production Monitoring	Detect	Whether predictions are still right	Telemetry bytes per second and their retention
Retraining Decisions	Decide	Whether a new model is worth its run	Accelerator-hours that contend with serving
Incident Response and Governance	Diagnose, recover	Which layer failed, and who acts	Replay and slice queries over logged requests
Operational Maturity	All steps	Which control to add next	Operational spend matched to the model's value
Case Studies	All steps	How the loop changes at the edge and in the clinic	Flash banks, radio energy, and clinician review time

Operating this feedback path requires adding statistical controls to the release and reliability practices of DevOps. Technical debt accumulates wherever one of the three interface boundaries erodes. Feature stores, versioning, and CI/CD pipelines restore the reproducibility needed to isolate a fault at those boundaries, while monitoring, drift detection, and staged rollout protect predictive quality once inputs begin to shift. A small set of foundational principles decides which of these controls a given model needs and when each should act.

14.1.1 Five operational principles

Without clear operating rules, a team cannot tell whether an accuracy drop came from an unrecorded data change, a mismatched serving feature, or an overdue retraining run. Five operational principles prevent these blind spots by anchoring each boundary of the ML node to a measurable quantity, from a complete artifact hash for reproducibility to net retraining value for automation (table 14.2). The sections that follow build the controls that enforce them.

Table 14.2: MLOps Principles Summary: Quick reference for the five foundational principles that guide all MLOps tooling and practice decisions.

Principle	Core Insight	Key Metric
Reproducibility	Version all artifacts	Complete artifact hash
Separation of concerns	Independent layer evolution	Layers touched per change
Consistency	Training equals serving	Feature skew rate
Observable degradation	Make failures visible	Time to detection
Cost-aware automation	Optimize total cost	Net retraining value

14.1.1.1 Reproducibility

Reproducibility requires every artifact[2] that influences model behavior to be versioned and traceable. This principle extends beyond code repositories to encompass input datasets, feature schemas, hyperparameter configurations, and execution environments. Equation 14.1 expresses this dependency formally:

$$\text{Model Output} = f(\text{Code}_v, \text{Data}_v, \text{Config}_v, \text{Environment}_v; \xi) \tag{14.1}$$

where v identifies an immutable version hash and ξ captures execution nondeterminism. Exact reproduction requires freezing all four versioned inputs alongside random seeds, dataloader sample ordering, deterministic GPU reduction kernels, and identical accelerator hardware architectures.

Fixing random seeds (such as `torch.manual_seed(42)`) still fails to guarantee bitwise reproducibility across training runs on accelerator hardware. The root cause is finite-precision arithmetic. In exact arithmetic, addition is associative, so $(a+b)+c = a+(b+c)$. Floating-point formats such as FP32, FP16, and BF16 break associativity because each intermediate sum is rounded to fit a finite mantissa width. Because concurrent atomic updates on an accelerator reach memory in an order that varies from run to run, the rounded sums differ in their least significant bits, and these differences compound over thousands of optimization steps. Two runs with identical seeds and code can therefore diverge into measurably different weights unless deterministic reduction algorithms, which fix the summation tree at the cost of parallel throughput, are enabled (section 12.8.2 develops this trade-off between reproducibility and performance).

[2] **Artifact**: Model weights are realized training outputs; the inputs and randomness that produced them cannot generally be inferred from the parameters. Versioning only code is therefore insufficient, and even versioning code, data, configuration, and environment may not guarantee exact replay when kernels or training are nondeterministic.

↩ The floating-point formats, their dynamic ranges, and where accumulated rounding error matters in training are established in section 8.6.3.

14.1.1.2 Separation of concerns

Separation of concerns decomposes MLOps systems into distinct functional layers that can evolve independently. Because data schemas, training algorithms, traffic patterns, and business rules change on different schedules, isolating each layer lets a team update one without rebuilding the others, as table 14.3 summarizes:

Table 14.3: MLOps Separation of Concerns: Each layer addresses a distinct responsibility and evolves at different rates across the data, training, serving, and monitoring layers. This separation enables independent scaling and updates, reducing blast radius when changes are required.

Layer	Responsibility	Stability
Data Layer	Feature computation, storage, serving	Changes with data schema evolution
Training Layer	Model development, hyperparameter optimization	Changes with algorithm research
Serving Layer	Inference, scaling, latency management	Changes with traffic patterns
Monitoring Layer	Drift detection, performance tracking	Changes with business requirements

14.1.1.3 Consistency imperative

The architectural boundaries in table 14.3 allow teams to upgrade serving runtimes without retraining models, adjust alert thresholds without redeployment, and modify data ingestion pipelines while preserving interface contracts. That architectural independence remains safe only when training and serving environments process features identically, establishing training-serving parity as the consistency imperative of section 4.7.1. A violation of that parity is training-serving skew, and the training-serving skew diagnostic (principle 11) detects it without waiting for labels by comparing what the training and serving code paths compute, from features through model scores, for the same inputs. The financial consequence of this discrepancy is captured in equation 14.2:

$$\text{Skew Cost} = \text{Rate}_{\text{skew}} \times Q_{\text{day}} \times C_{\text{error}} \tag{14.2}$$

where $\text{Rate}_{\text{skew}}$ is the fraction of queries with erroneous predictions caused by training-serving skew, Q_{day} is the daily query volume, and C_{error} is the average business cost incurred per erroneous prediction.

For a system serving 1,000,000 queries/day with 1 percent skew-induced errors costing $0.10 each, annual skew cost reaches $365,000. The loss recurs every year until consistency mechanisms remove its cause. Shared feature stores, unified C++ or Rust feature-extraction libraries invoked identically during offline training and online serving, and automated schema validation tests support consistency at the system boundary; matched-input tests must still verify feature values and semantics.

14.1.1.4 Observable degradation

Observable degradation requires ML systems to make silent failures visible through continuous measurement. Model accuracy degrades along a continuous statistical spectrum rather than failing as a discrete binary crash. An observed failure's time signature (an acute drop, gradual drift, or localized cohort decay) governs both the detection mechanism and the operational response. The ranking service's four-week slide from 94 to 88 percent was a gradual signature, the kind that trend analysis on accuracy catches and that no uptime or latency alert can see. Making degradation observable therefore means pairing each failure signature with the telemetry that can see it and the countermeasure it calls for, as table 14.4 outlines.

Table 14.4: Degradation Detection Strategies: Four failure signatures map to detectors and operational responses. Sudden drops require immediate diagnosis and may call for rollback, while gradual or subgroup degradation calls for diagnosis before retraining or targeted data collection.

Degradation Type	Detection Mechanism	Response Strategy
Sudden accuracy drop	Threshold alerts	Diagnose; roll back if release-related
Gradual drift	Trend analysis	Diagnose, then retrain if warranted
Subgroup degradation	Cohort monitoring	Targeted data collection
Latency increase	Percentile tracking	Infrastructure scaling

14.1.1.5 Cost-aware automation

Cost-aware automation balances computational costs against accuracy improvements. Let ΔAccuracy be the expected gain in accuracy percentage points, Value per Point the monetary value of one percentage-point gain over the decision horizon, Training Cost the compute and labor cost of a retraining run, and Deployment Risk the expected cost of validation and rollout failure. Equation 14.3 models this trade-off:

$$\text{Retrain if: } \Delta\text{Accuracy} \times \text{Value per Point} > \text{Training Cost} + \text{Deployment Risk} \tag{14.3}$$

The inequality acts as an operational decision gate rather than an unconstrained trigger. An automated pipeline should initiate retraining only when empirical evidence indicates that expected accuracy gains surpass execution cost and rollout risk. This economic trade-off governs retraining triggers, validation thresholds, and rollout gates; section 14.6 formalizes this model to calculate optimal retraining cadences. The margin by which the left side of equation 14.3 exceeds the right is the *net retraining value*, the quantity that a retraining trigger should test.

How these principles manifest in practice depends on the workload. The ranking service drifts daily as user preferences shift, while the keyword spotter may run unchanged for months between updates. The monitoring strategy must match the workload's regime and, more decisively, the labels its deployment can observe.

Lighthouse 14.1: Monitoring strategy by archetype

The dominant failure modes and monitoring priorities differ across workload archetypes. Table 14.5 compares four representative archetypes by drift pattern, monitoring metric, and example retraining trigger:

Table 14.5: Monitoring Strategy by Workload Archetype: Illustrative starting points for monitoring strategy. Real thresholds must be calibrated to the deployment's label delay, traffic volume, business risk, and alert-fatigue budget.

Archetype	Dominant Drift Pattern	Primary Monitoring Metric	Example Retraining Trigger
Compute-bound (ResNet-50)	Visual distribution shift (lighting, camera, new object classes)	Accuracy on holdout set (ground truth available)	Accuracy drops > 2% from baseline (∼monthly for stable domains)
Bandwidth-bound (GPT-2)	Vocabulary drift, topic shift, emerging entities	Perplexity on live traffic (no ground truth needed)	Perplexity increases > 10%; new vocabulary detected (∼weekly for news domains)
Capacity-bound (DLRM)	User behavior shift, item catalog churn, cold-start items	CTR/CVR delta vs. historical cohorts	Engagement drops > 5%; catalog refresh (∼daily for e-commerce)
Power-bound (KWS DS-CNN)	Acoustic environment change (noise floor shift)	Duty cycle (wakeups/hour) + false positive rate	False wake rate > 1%; battery drain exceeds spec (∼quarterly OTA update)

Systems insight: Ground-truth availability governs monitoring design. A workload's binding regime sets its cost and capacity but does not determine which quality signal is observable. Vision tasks with labels (ResNet-50) track holdout accuracy; language and recommendation services (GPT-2, DLRM) rely on perplexity or implicit clicks; microcontroller models without labels (the keyword spotter's DS-CNN) monitor duty cycles, while estimating false-wake rates requires a reference signal or reviewed examples. Retraining cadence is constrained by label delay, deployment access, validation cost, and the rate of observed change.

These principles respond to three recurring operational failures: concept drift,[3] data-quality defects (Schelter et al. 2018), and silent postdeployment degradation. Addressing them changes both what a team versions and who must coordinate alongside what it monitors: MLOps brings a broader stakeholder ecosystem together around data versioning,[4] model versioning, and continuous statistical monitoring that extend beyond traditional software engineering. By treating datasets and pipeline state as first-class versioned artifacts alongside source code, this expanded operational scope turns deployment from a linear release pipeline into a continuous, evidence-driven feedback loop.

Checkpoint 14.1: The MLOps loop

The operations loop feeds production evidence back into design.

The feedback cycle

- ☐ **Loop closure**: Explain how production telemetry returns evidence to the model lifecycle and why an alert does not by itself authorize retraining.
- ☐ **Retraining trigger**: Distinguish a distribution change from a measured quality regression, and explain why either requires diagnosis before selecting a remedy.

The artifacts

[3] **Concept drift**: Concept-drift and data-stream research formalized the problem that a model's target relationship can change after deployment (Widmer and Kubat 1996; Gama et al. 2014). In adversarial domains such as spam, fraud, and abuse detection, the distribution can actively adapt in response to the model, making continuous monitoring and retraining a structural requirement rather than an operational luxury.

[4] **DVC (Data Version Control)**: DVC brings Git-like versioning to datasets and model artifacts (Iterative 2024), closing the artifact gap that equation 14.1 formalizes. Without data versioning, the Data_v term is unrecoverable, and no combination of code commits can reconstruct the model that was deployed.

- ☐ **Artifact versioning**: Identify the data, code, configuration, weights, and environment versions needed to trace a deployed model, and explain why versioning alone does not guarantee exact replay.

Each iteration through the operational loop introduces data dependencies, model interactions, and configuration state invisible to traditional compilation and testing. When left unmanaged, these hidden dependencies accumulate into systemic technical debt that slows releases and degrades predictive accuracy. Understanding the architectural patterns of this debt is the prerequisite for designing the infrastructure that pays it down.

14.2 Technical Debt

A team that ships a model by copying feature scripts, skipping schema checks, and wiring predictions directly into downstream services gets to production quickly and then spends the next year debugging silent regressions that no unit test catches. In traditional software systems, technical debt[5] represents the future rework created by choosing expedient shortcuts over disciplined architecture (Cunningham 1992). In machine learning systems, that debt compounds through statistical coupling and data dependencies that code review and unit tests cannot detect (Sculley et al. 2015). A codebase can remain clean while the external world shifts, rendering trained parameters stale and breaking downstream consumers. The ML Test Score rubric (Breck et al. 2017) quantifies this debt; it returns with the maturity audit near the end of the chapter.

5 **Technical debt**: Ward Cunningham's 1992 WyCash experience report introduced the debt metaphor for expedient code and delayed consolidation (Cunningham 1992).

Definition 14.2: Technical debt in ML

Technical debt in machine learning is the accumulating maintenance and change cost created by implicit data dependencies, entangled features, and undeclared consumers in ML systems, where the resulting liabilities may also appear as silent accuracy degradation.

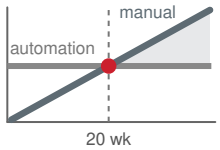
1. **Significance**: Google's analysis of production ML systems argues that model code is only a small fraction of the surrounding system; the larger operational surface includes data collection, feature extraction, configuration, serving infrastructure, monitoring, and process management (Sculley et al. 2015). ML-specific debt drivers compound this: changing one input feature can silently shift the learned representation of every other feature (entanglement), a model trained to correct another model's errors creates a fragile dependency chain (correction cascades), and downstream systems consuming model outputs without explicit contracts become undeclared consumers that break silently when the model is updated.
2. **Distinction**: ML technical debt includes system- and data-level dependencies that code review and ordinary code-focused tests may miss. It can slow development and degrade prediction quality even while availability metrics remain healthy.
3. **Common pitfall**: A frequent misconception is that "better code" solves technical debt in ML. In reality, it is a systems architecture problem: the debt accumulates when the assumptions of the training distribution (feature ranges, label meanings, data freshness) are not enforced as runtime contracts at the system boundary.

A break-even calculation makes the cost dynamics of technical debt concrete. Teams often resist automation investment because manual processes seem faster in early development, but that advantage disappears once recurring manual interventions exceed the up-front automation cost.

Napkin Math 14.1: The cumulative cost of manual operations

Problem: Why build automated pipelines when manual retraining is faster?

Physics: Manual work accumulates over time.



Cumulative manual work overtakes the one-time automation investment near week 20.

- Manual retrain: 4 hours of engineering per week.
- Pipeline build: 80 engineering hours (one-time).

Math:

- **Result**: 80 engineering hours ÷ 4 hours per week = 20 weeks.
- **Assumption**: This simplified calculation sets pipeline upkeep to zero, so automation removes the weekly task entirely.
- **Upkeep**: Real pipelines need maintenance and new features. Each week then recovers only the difference between the manual load and the upkeep, which pushes break-even later; automation still pays back whenever its weekly upkeep stays below the manual work it replaces.
- **Consequence**: After 1 year, manual teams still spend 4 hours per week on maintenance, while pipeline teams spend only their upkeep (0 recurring hours under this calculation's assumption).

Context: Over a system's lifetime, the cost of *maintaining* it can exceed the cost of *building* it. In ML, technical debt is often *data-driven* rather than *code-driven*, because correct code can still fail when the data it processes shifts. Without telemetry, the team cannot tell whether maintenance work is reducing debt or merely hiding it.

Systems insight: A team that retrains by hand eventually has no time left to ship new models, provided automation upkeep stays below the manual load it removes. Without monitoring that makes silent failures visible, the team also accumulates debt in a system that grows harder to manage.

The debt is larger than manual retraining alone, because modeling code constitutes only a small fraction of a real-world deployment. Core modeling code is one peripheral component among ten around the ML system, alongside nine supporting subsystems spanning data verification, feature extraction, configuration, and machine resource management (figure 14.3). Their presence broadens the maintenance surface; the diagram does not measure each component's cost.



Figure 14.3: Hidden Infrastructure of ML Systems: ML code is one of ten components arranged around the ML system at the center, alongside data collection, data verification, feature extraction, configuration, machine resource management, serving infrastructure, monitoring, analysis tools, and process management tools. Modeling occupies one circle and the remaining nine identify the operational surface a deployment must also carry; circle counts do not measure code volume or maintenance cost. Source: (Sculley et al. 2015).

While automation addresses the recurring operational overhead of manual retraining, it cannot prevent architectural decay. Deployed systems accumulate hidden complexity through systemic failure modes that emerge from statistical coupling rather than deterministic logic. The patterns group by where the coupling hides: in eroded boundaries and the correction cascades they invite, in interface dependencies that no import statement declares, in feedback and configuration that accumulate as the system evolves, and in the code and architecture around the model.

14.2.1 Boundary erosion

When an upstream pipeline changes how missing ages are filled or rescales a price column, every unit test in the repository can pass while the downstream model's predictions quietly shift. Statistical software loses its modular boundaries more easily than conventional software does because component behavior depends on the distribution of incoming data rather than on type signatures alone. In conventional software, modular encapsulation and typed APIs make dependencies explicit, though changes can still violate behavioral contracts. Machine learning systems blur these boundaries because an upstream formatting change or a new normalization rule can satisfy every syntactic check while shifting feature distributions and degrading downstream model accuracy. This implicit coupling through data pipelines creates unmonitored dependencies among feature engineering, training pipelines, and serving endpoints.

This erosion produces **entanglement**, where dependencies between features and components become so intertwined that local changes demand global coordination. The resulting failure mode reflects the CACE principle: *Change Anything Changes Everything*. When systems lack strict statistical checks at their boundaries, adjusting a feature encoding or data filtering threshold can shift learned weights throughout the model during retraining. With frozen weights, the same change leaves the learned parameters alone but alters the inputs the model sees, and potentially its predictions. For example, changing the binning strategy of a continuous feature shifts the feature's variance and covariance structure, causing a previously calibrated model to underperform and forcing downstream recalibrations across dependent services.

Defending against boundary erosion requires architectural modularity enforced at runtime. Components require explicit data contracts that govern statistical invariants, including allowed feature ranges, null tolerances, and drift thresholds. Enforcing separation between data ingestion, feature generation, and inference logic creates verification boundaries where data distributions can be validated before reaching model weights. Because boundary erosion remains silent during initial prototyping, schema contracts and invariant checks are the earliest point at which statistical coupling can be detected and managed.

14.2.2 Correction cascades

When boundary erosion goes unaddressed, engineering fixes frequently trigger a **correction cascade**: patching a specific model defect with an auxiliary post-filter introduces secondary errors elsewhere, requiring further patches that destabilize the entire pipeline. In ML systems, these cascades are severe because changes propagate through statistical dependencies rather than deterministic code branches. Retraining a model to correct a specific false-positive cluster may degrade accuracy across previously stable cohorts. Adding an auxiliary model to override edge-case predictions introduces uncalibrated scores that confuse downstream consumers. Repeated dependent patches can demand coordinated revalidation, with maintenance costs that may exceed those of revising the base model.

Figure 14.4 traces this failure propagation along a four-stage dependency chain (Models A through D), where each downstream component compensates for the residual errors of its predecessor. When an upstream patch alters Model A, the dashed red arcs mark every downstream correction that requires revalidation and may need retraining.

A cascade requires dependent correction models; fine-tuning or reusing an established model does not create one by itself, although reuse does inherit representations and assumptions that require evaluation on the new task. Over time, those inherited properties become rigid architectural constraints that complicate future updates.

For example, fine-tuning an existing customer churn model for a new product line inherits the base model's feature encodings and calibration thresholds. If the new product has different user engagement patterns, patching prediction errors with auxiliary post-processing rules creates a

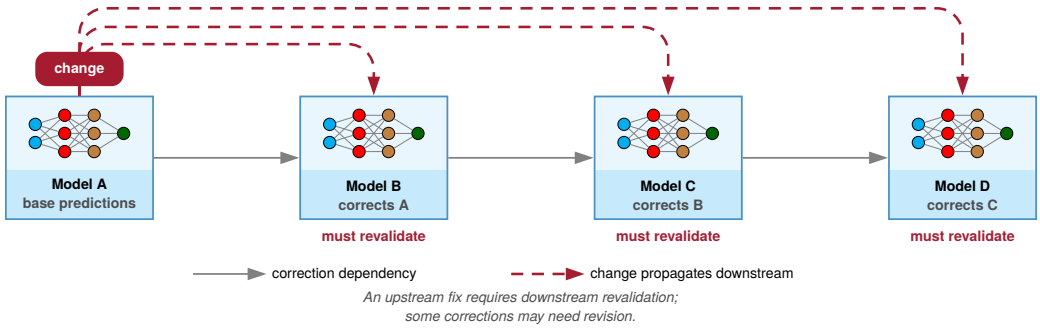


Figure 14.4: Correction Cascades: Each model is trained to correct the errors of the model before it, forming a chain of dependencies. Changing an upstream model can invalidate downstream corrections, turning one local fix into revalidation across the chain and potentially several retraining cycles.

fragile multi-stage pipeline. When accuracy degrades, engineering teams discover that the root failure lies several layers upstream in the original labeling conventions.

Mitigating correction cascades requires balancing component reuse against redesign. While fine-tuning reduces initial compute and data collection costs, training a replacement base model can remove dependencies on a chain of auxiliary corrections, while data and labeling assumptions still require validation. The architectural decision depends on task similarity, covariate alignment, and the ongoing maintenance cost of chained models. Where dependent models are unavoidable, systems must decouple them through version-pinned model artifacts and frozen intermediate representations.

14.2.3 Undeclared consumers and data dependencies

When Model A writes its predictions to a shared warehouse table and Model B later reads that column as a training feature, neither repository's import graph records the connection. Boundary erosion and correction cascades both stem from interface dependencies that bypass formal software contracts. Traditional software dependencies are declared through import statements, package managers, and RPC schemas, allowing static analysis tools to map dependency graphs. In machine learning systems, dependencies hide inside data streams, invisible to static code analysis.

Hidden coupling frequently manifests as **undeclared consumers**, where downstream services consume model predictions without formal registration or service-level agreements. When the upstream model is retrained or its output distribution shifts, these untracked consumers fail silently. For example, a credit risk score consumed by an automated marketing system can alter applicant pools, feeding shifted distributions back into future training runs, so the model increasingly trains on the applicants its own scores favored. This vulnerability is compounded by **data dependency debt**, which accumulates when ML pipelines ingest unversioned tables, unstable upstream features, or signals with volatile distributions. Eliminating this interface debt requires infrastructure-enforced boundaries: strict authentication and registration for model prediction endpoints, machine-readable schema contracts for feature stores, and automated data lineage tracking that registers consumers before granting read access.

14.2.4 Feedback loops and configuration sprawl

As an ML system evolves, even with documented interfaces, the most dangerous dynamic is the feedback loop, where a deployed model shapes the data distribution that trains its successors. In the ranking service, suggested content dictates user engagement, clicks generate subsequent training sets, and the model reinforces its own historical selections while starving unselected content of exposure. In production, this failure mode often surfaces as a widening performance gap across demographic cohorts: underrepresented groups receive worse recommendations, generating sparse engagement logs that further weaken cohort representation in subsequent retraining runs, the self-reinforcing loop that the bias feedback model (principle 13) describes. Detecting feedback loops requires cohort-stratified monitoring that tracks error distributions across user segments rather than relying on aggregate validation metrics.

↪ A worked example that fits the bias feedback factor from two audits and treats fairness as a stability property of the retraining loop is developed in section 15.3.4.

Evolution also degrades the workflows around the model. Over repeated deployment cycles, they accumulate **pipeline and configuration debt**, growing into sprawling collections of ad hoc glue scripts and divergent hyperparameter files. When pipelines lack modular interfaces, teams copy existing scripts rather than refactor brittle data loaders, producing redundant pipelines that process the same raw inputs with subtle inconsistencies. Compounding this, rapid prototyping frequently embeds business logic directly into feature preparation scripts. Controlling pipeline evolution requires workflow orchestration engines with immutable directed acyclic graph (DAG) definitions, strict parameter validation, and configuration versioning managed under the same source control as model code.

14.2.5 Code and architecture debt

Production ML repositories are rarely dominated by model definitions; instead, they fill with **glue code**, the integration wrappers written to shuttle tensors and records between general-purpose ML frameworks, feature pipelines, and low-latency serving engines (Sculley et al. 2015). Sculley et al. (2015) observe that a mature system can end up at most 5 percent machine learning code and at least 95 percent glue code. The proportion varies by system, but the asymmetry explains why integration code, not the model, dominates the maintenance surface. The resulting tight coupling to framework-specific APIs is a maintenance liability, because minor upstream framework updates force extensive rewrites of the surrounding integration scaffolding. Isolating third-party libraries behind stable internal interfaces prevents external API churn from propagating throughout the serving stack.

Rapid experimentation further burdens codebases through **dead experimental codepaths**. Research iterations leave behind legacy branches, alternative feature transforms, and abandoned scoring logic. Unlike traditional dead code that static analyzers flag and compilers prune, experimental ML codepaths often remain executable at runtime because they are guarded by dynamic configuration flags. These dormant branches expand the test matrix, complicate regression verification, and obscure the precise execution graph operating in production. Enforcing deprecation schedules and pruning retired feature flags prevents experimental residue from destabilizing the serving runtime. The same residue accumulates in data as well as code, where a retired feature can keep flowing through the pipeline long after anyone remembers why it exists.

> **Systems Perspective 14.1: Zombie features**
>
> Long-lived production models often accumulate features whose original owners, semantics, or intended use have faded. A feature can be deprecated in application code while still flowing through a feature store, training dataset, or serialized model input contract. The model may learn to ignore it, split signal across a duplicate, or depend on a preprocessing artifact that nobody still owns. Removing such a feature is therefore no longer a local cleanup. It becomes a compatibility change that can affect retraining, serving, monitoring, and downstream analysis.
>
> Features in ML systems do not disappear just because code owners stop thinking about them. Unused feature payload volume D_{vol} inflates serialization overhead and consumes memory bandwidth BW without improving accuracy. Without explicit deprecation policies and feature-store governance, models accumulate "dead code" that degrades D_{vol}/BW and complicates debugging (Sculley et al. 2015).

Underlying both patterns is **abstraction debt**. Traditional software engineering relies on mature abstractions such as objects, interfaces, and modules, whereas ML engineering lacks universal abstractions for concepts such as a feature, a training dataset slice, or a model lifecycle state. Without standardized primitives, teams frequently reinvent bespoke abstractions or abandon encapsulation altogether. Infrastructure components such as feature stores, model registries, and standardized prediction services mitigate this debt by providing uniform abstractions for feature transformation, artifact lineage, and inference routing.

Sculley et al. (2015) also list three system smells that signal mounting debt: the *Plain-Old-Data Type Smell* (passing unvalidated raw floats and string arrays rather than domain types with invariant checks), the *Multiple-Language Smell* (stitching Python, SQL, C++, and shell scripts across the training-

serving boundary), and the *Prototype Smell* (promoting throwaway research notebooks directly into production serving without re-engineering). Treating debt paydown as a routine operational discipline rather than an emergency intervention keeps these code smells from accumulating.

14.2.6 Technical debt in practice

When statistical dependencies, undeclared consumers, and unversioned configurations go unchecked in production, they surface not as compiler warnings but as commercial losses and operational incidents. Examining how those failures unfold in live systems clarifies why code-level testing alone cannot contain them.

14.2.6.1 Production debt patterns

Four documented cases show these patterns in production. The first pair shows coupling through model behavior. YouTube's recommendation system learns from noisy user-behavior signals, so its ranking objectives, per-user weighting, and live A/B evaluation are part of the system design rather than offline details (Covington et al. 2016). Its own rankings shape what users watch, which makes today's recommendations tomorrow's training signal, the feedback loop that evaluation on fixed historical logs can miss. Zillow's iBuying venture exposes the decision-coupling version of a cascade, in which a forecast drives purchases that cannot be undone when the forecast is later corrected (war story 14.1).

War Story 14.1: The Zillow Offers wind-down (2021)

Context: Zillow, a real-estate marketplace, ran Zillow Offers, an iBuying business that bought homes directly on the strength of forecasts of home prices and resale economics (Zillow Group 2021).

Mechanism: The valuation forecast did not remain a prediction. It set purchase prices, purchases became inventory, and inventory carried the forecast's error onto the balance sheet. Zillow reported that it had bought homes at prices above its revised estimates of future selling prices, while resale-capacity constraints delayed some closings (Zillow Group 2021). Revising the forecast corrected the model's view of the market but could not reverse commitments made on the earlier view, so the correction propagated downstream as losses rather than as a repair.

Impact: In November 2021 Zillow announced that it would wind down Zillow Offers, reported a $304 million third-quarter inventory write-down, and cut about 25 percent of its workforce (Zillow Group 2021).

Response: A safer design caps the inventory exposure any one forecast can create, keeps manual override between prediction and purchase, and places circuit breakers that halt purchasing when forecast uncertainty exceeds the accepted risk budget.

Systems lesson: The case does not show a chain of residual-correction models, and retraining alone would not have removed the business risk. It shows the hazard that correction cascades share: a fix applied at one stage cannot undo the state that downstream stages already built on its earlier output. When predictions trigger irreversible actions, the dependency chain runs through the physical world, and the operational controls must sit on that chain.

The second pair exposes coupling through ownership and configuration. In safety-critical driving automation, automated-control outputs, driver expectations, and subsystem responsibilities that are not specified clearly enough let operational failures cross component boundaries rather than stay local (National Transportation Safety Board 2017). Facebook's News Feed iterations show the configuration version of the same problem, in which rapid ranking changes without traceable settings and explicit objectives become hard to audit after deployment (Engineering 2016; Mosseri 2018).

In every case the remedy is structural. Mitigating technical debt requires end-to-end MLOps infrastructure: feature registries that enforce runtime schemas, metadata stores that track model-data lineage, automated CI/CD pipelines that validate statistical properties, and observability platforms that detect distribution shift before silent errors propagate downstream.

Silent interface decay, untracked consumers, and unversioned pipeline dependencies can each make a model unsafe to serve while its offline validation metrics look acceptable. Table 14.6 collects the debt patterns of this section with their root causes, failure symptoms, and mitigation strategies, the catalog of failure modes that a production-readiness rubric must detect.

Table 14.6: Technical Debt Patterns: Machine learning systems accumulate distinct forms of technical debt from data dependencies, model interactions, and evolving operational contexts. Primary debt patterns, their causes, symptoms, and recommended mitigation strategies guide practitioners in recognizing and addressing these challenges systematically.

Debt Pattern	Primary Cause	Key Symptoms	Mitigation Strategies
Boundary Erosion	Tightly coupled components, unclear interfaces	Changes cascade unpredictably, CACE principle violations	Enforce modular interfaces, design for encapsulation
Correction Cascades	Sequential model dependencies, inherited assumptions	Upstream fixes break downstream systems, escalating revisions	Careful reuse vs. redesign trade-offs, clear versioning
Undeclared Consumers	Informal output sharing, untracked dependencies	Silent breakage from model updates, hidden feedback loops	Strict access controls, formal interface contracts, usage monitoring
Data Dependency Debt	Unstable or underutilized data inputs	Model failures from data changes, brittle feature pipelines	Data versioning, lineage tracking, leave-one-out analysis
Feedback Loops	Model outputs influence future training data	Self-reinforcing behavior, hidden performance degradation	Cohort-based monitoring, canary deployments, architectural isolation
Pipeline Debt	Ad hoc workflows, lack of standard interfaces	Fragile execution, duplication, maintenance burden	Modular design, workflow orchestration tools, shared libraries
Configuration Debt	Fragmented settings, poor versioning	Irreproducible results, silent failures, tuning opacity	Version control, validation, structured formats, automation
Prototype Debt	Rapid prototyping shortcuts, tight code-logic coupling	Inflexibility as systems scale, difficult team collaboration	Flexible foundations, intentional debt tracking, planned refactoring

The mitigation column reads as a requirements list for the rest of the chapter. Shared feature definitions, versioned artifacts, and automated pipelines are development infrastructure; cohort monitoring belongs to production monitoring; and the rubric that scores a system against the whole catalog, the ML Test Score, is taken up with operational maturity (section 14.8.1), once the controls it audits have been built. Development infrastructure comes first, because a fault cannot be traced to a boundary that was never versioned.

14.3 Development Infrastructure

When a production model fails an accuracy check, an engineer who cannot recover the exact training dataset, feature transformation code, and container image that produced it can only guess which input changed. Development infrastructure turns the debt patterns diagnosed in section 14.2 into concrete enforcement points. A feature schema that drifts upstream cannot be repaired by a dashboard alone; it needs a shared contract, a versioned artifact, and a deployment path that rejects incompatible changes before they reach production. Table 14.7 maps each component directly to a foundational principle (section 14.1.1) and a specific failure mode. CI/CD pipelines also carry versioned artifacts, but the table assigns them cost-aware automation because the pipeline is where the retraining and promotion gates of equation 14.3 actually execute, automating only the changes that pass them.

Table 14.7: MLOps Infrastructure as Debt Remediation: Each infrastructure component responds to a class of technical debt observed in production ML systems. Feature stores can reduce training-serving skew by reusing feature definitions and values; versioning systems support reproducibility; CI/CD pipelines automate rollout controls; monitoring systems can surface silent degradation before users do.

Infrastructure Component	Principle Implemented	Debt Pattern Addressed
Feature stores	Consistency	Data dependency debt, training-serving skew
Versioning systems	Reproducibility	Configuration debt, correction cascades
CI/CD pipelines	Cost-aware automation	Pipeline debt, boundary erosion
Monitoring systems	Observable degradation	Feedback loops, silent failures

These operational components sit between the framework and the hardware and track what neither of them tracks, the data lineage and the cluster state. MLOps occupies the middle of the five-tier stack of figure 14.5 (ML models, frameworks, orchestration, infrastructure, and hardware). Model orchestration, which runs from data management to serving, and operational infrastructure, which runs from job scheduling to monitoring, together bridge mathematical frameworks and physical accelerator clusters and enforce the remediations of table 14.7. The tiers say *where* a component sits in the implementation stack, while the four layers of table 14.3 say *who* owns an operational responsibility; the rest of the chapter reasons with the layers and the three interfaces.

Inside the orchestration and infrastructure tiers, four architectural mechanisms execute these operational responsibilities. At the data boundary, a feature storage and retrieval layer serves a unified schema across two disparate access patterns: high-throughput analytical scans over immutable columnar files for offline training, and low-latency point lookups against key-value stores for online serving (section 14.3.1.1). To keep every transformation and checkpoint traceable, an artifact and lineage store records content-addressed hashes for dataset snapshots, code commits, container images, and model weights so that any live prediction can be audited back to its exact training inputs (section 14.3.1.2). Workflow orchestration then sequences the pipeline stages, from ingestion and validation through training and evaluation, as declared dependency graphs with retry semantics and automated triggers (section 14.3.2.1), while a compute runtime underneath schedules accelerator hardware, host memory, and processor cores across training jobs and serving endpoints under strict resource isolation (section 14.3.2.2). Specific frameworks implement pieces of this decomposition, but the underlying mechanisms and interface boundaries outlast any particular tool.

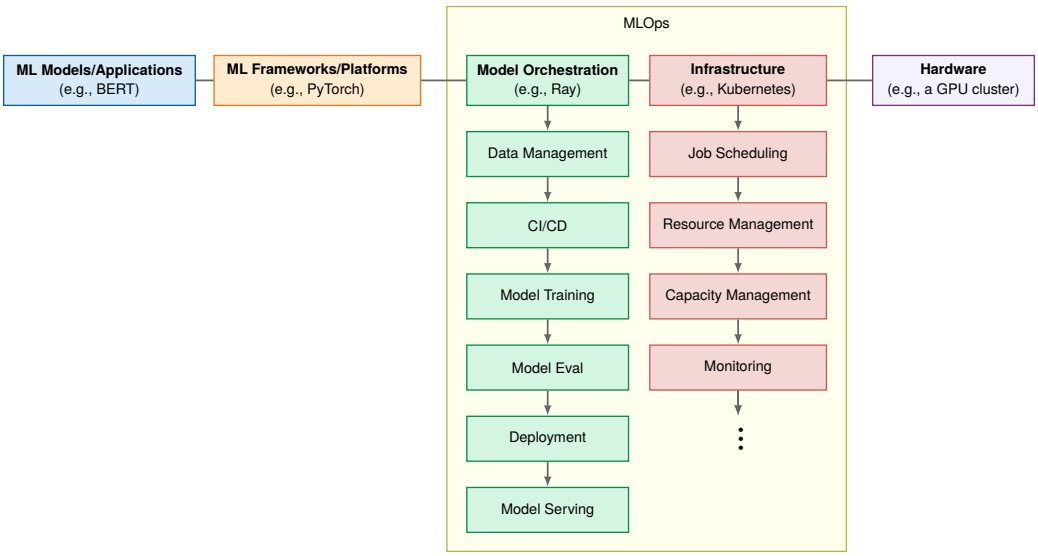


Figure 14.5: MLOps Stack Layers: Five columns organize the ML system stack: ML Models/Applications, ML Frameworks/Platforms, Model Orchestration, Infrastructure, and Hardware. MLOps spans orchestration tasks from data management through model serving and infrastructure tasks from job scheduling through monitoring, supporting automation, reproducibility, and scalable deployment.

14.3.1 Data infrastructure and preparation

If the offline training pipeline reads a backfilled warehouse table while the online serving path reads a live event stream with different null defaults, the deployed model evaluates inputs it never learned to score. Machine learning technical debt stems largely from unmanaged data boundaries: unversioned datasets erode system boundaries, inconsistent feature computation triggers correction cascades, and undocumented data dependencies create hidden downstream consumers. Data engineering built the collection, preprocessing, and feature-transformation pipelines, together with the validation and dataset versioning that make one pipeline correct and reproducible. Operations turns those pipelines into versioned interfaces that it enforces across every pipeline touching a model. Where data engineering optimizes single-pipeline correctness and ingestion throughput, MLOps data management governs cross-pipeline consistency, ensuring that offline training and online serving compute identical features.

Operations holds those interfaces to three properties: consistency, freshness, and quality. Consistency requires every artifact that influences model behavior, from raw records to engineered feature tables, to remain addressable by version, so that a checkpoint can be traced to the dataset snapshot that produced it (section 14.3.1.2), and requires training and serving to share one feature definition (section 14.3.1.1), with content-addressed dataset versioning tools such as DVC (Iterative 2024) supplying the first half of that guarantee. Because consistent definitions still degrade if inputs arrive late, freshness requires scheduled pipelines, declared in workflow orchestrators such as Apache Airflow (Apache Software Foundation 2024), to deliver current data on a known cadence so that a late run becomes a visible failure rather than a stale feature. Even timely, versioned pipelines can ingest corrupted values, so quality requires the validation checks of section 4.5.3 to run as gates that reject a corrupted batch before it reaches training or serving, while annotation platforms such as Label Studio (HumanSignal 2024) keep label corrections under version history.

↩ The collection, preprocessing, and feature-transformation pipelines that MLOps formalizes are established in chapter 4.

On the ranking service, the three properties operate in concert. Click and session events are ingested and joined with the item catalog through a scheduled pipeline (freshness). The resulting features, such as session length and rolling click rates, are stored in a feature store for both offline retraining and low-latency online lookup (consistency). Schema validation, range checks, missingness tests, and label audits reject corrupted event records before they enter the training set (quality), while dataset versioning and model-registry integration maintain traceability from raw event logs to deployed predictions.

14.3.1.1 Feature stores

The data dependency debt and training-serving skew analyzed in section 14.2 share a common root cause, inconsistent feature computation across pipeline stages. Without a unified feature layer, feature definitions are typically implemented twice: on the ranking service, a data scientist computes `user_session_length` in Python using batch aggregations over historical logs, whereas a systems engineer reimplements the calculation in Java or C++ to satisfy real-time serving latencies. Because one implementation measures wall-clock time between raw events while the other tracks server processing time and filters idle timeouts, the model trains on one definition and serves with another, degrading accuracy silently in production. Feature stores[6] address this divergence by providing an abstraction layer between data engineering and machine learning, implementing the consistency imperative through a single source of truth for feature transformations. In pipelines lacking this abstraction, duplicated feature logic across environments introduces training-serving skew, temporal data leakage, and distribution drift.

Feature stores resolve the tension between batch throughput and online latency by keeping two synchronized storage tiers behind one API, an offline store for high-throughput training scans and an online store for point lookups under a tail-latency service-level objective (SLO), both reading from the same versioned transformation logic. Historical retrieval must also be point-in-time correct, so that each training example sees only the feature values that existed at its observation time. Section 4.7.5 develops both mechanisms, including the as-of join that enforces point-in-time correctness. Operations depends on them because a ranking model trained on leaked features looks excellent offline and fails on live traffic.

Beyond consistency, feature stores support versioning, metadata management, and feature reuse across models. A ranking model and a fraud model built on the same user activity frequently

6 **Feature store**: Uber's Michelangelo platform described a centralized feature store for sharing and serving features across production models (Hermann and Del Balso 2017). At that scale, the consistency guarantee must hold under an online latency budget. A feature store differs from a shared library of feature code because its shared feature path must also serve fresh features fast enough for real-time inference.

↩ Training-serving skew, a violation of the consistency imperative, is established in section 4.7.1.

depend on overlapping features that can be centrally maintained, validated, and shared. Integration with data pipelines and model registries preserves lineage. When an upstream feature definition is updated or deprecated, downstream models can be automatically identified and scheduled for retraining.

Training-serving skew: Diagnosis and prevention Training-serving skew manifests operationally through feature store inconsistencies and pipeline divergence. Table 14.8 summarizes common causes and their detection methods:

Table 14.8: Training-Serving Skew Categories: Each category requires different detection and prevention strategies. Preprocessing and missing-data skew emerge from divergent transformation code and require parity validation, which shared feature logic, including a feature store where appropriate, can reduce. Time-dependent features need timestamp checks that both paths apply the same cutoff, and library version drift needs environment comparison because identical code can compute different values under different library versions.

Skew Type	Example	Detection Method
Feature preprocessing	Normalization uses different statistics	Same-input replay and value comparison
Missing data handling	Training fills NaN with mean; serving uses 0	Schema validation with explicit null handling
Time-dependent features	Features computed with different time cutoffs	Timestamp validation in feature pipelines
Library version drift	NumPy or Pandas version differences	Environment hash comparison

Training-serving skew case study The ranking service shows how feature divergence manifests in deployment. After a later release of the serving path, the model loses 8 percent of its accuracy despite zero alterations to model code or weights. Comparing feature distributions reveals that `user_session_length` has an empirical mean of 45 minutes in the training set but only 12 minutes in online inference. That difference starts the investigation; different user populations could produce it even with identical feature logic. Replaying the same recorded sessions through the versioned training and serving transformations, with the same available events and cutoff, reveals different durations for the same requests. Inspection of those artifacts then identifies feature-definition skew. The offline training pipeline calculated wall-clock duration from the initial event to the final event in a session, whereas the online serving path counted only foreground-active time after filtering idle periods. The model learned decision boundaries against a feature distribution that the serving infrastructure never delivers.

Feature stores resolve this discrepancy by centralizing feature logic into a single versioned definition, extending upstream data pipelines. In listing 14.1, training materializes historical features with point-in-time correctness, serving performs real-time point lookups, and both pipelines bind to the identical versioned definition hash rather than independent code paths.

Listing 14.1: Feature Store Consistency: Unified retrieval reduces training-serving skew by sharing versioned feature definitions across both pipelines.

```
feature_definitions = registry.load(version="2026-06-01")

training_features = feature_definitions.materialize_historical(
    entities=training_entities,
    at_event_time=True,
    names=["user.session_length", "user.purchase_history"],
)

serving_features = feature_definitions.lookup_online(
    entities=[{"user_id": 12345}],
    names=["user.session_length", "user.purchase_history"],
)

assert training_features.schema == serving_features.schema
assert (
    training_features.definition_hash
    == serving_features.definition_hash
)
```

By centralizing the definition of `session_length`, both offline and online pipelines execute identical transformation logic, though automated integration tests must still verify value parity. Centralized registries also record metadata and lineage, enabling rapid diagnosis when feature definitions evolve (Hermann and Del Balso 2017; Gojek and Google 2019).

The consistency imperative (section 14.1.1.3) put a number on this failure. A 1 percent skew error rate on 1,000,000 queries/day already costs $365,000 a year, while the `user_session_length` divergence above produced an 8 percent accuracy degradation. Centralized feature stores amortize the engineering cost of this infrastructure across multiple models and services, translating architectural consistency into measurable operational savings.

Skew detection in CI/CD Continuous integration pipelines must validate feature consistency before deployment. A parity test replays recorded inputs through the versioned training and serving transformations, aligns results by request identifier, and compares each feature using its declared semantics and numerical tolerance. Both paths must receive the same events available at the prediction cutoff; otherwise the test confounds implementation differences with differences in information. For a duration feature, the contract specifies the time unit and whether idle periods count. Categorical values and missingness must agree exactly; floating-point values may use a feature-specific tolerance.

A small counterexample shows why distributions cannot replace this check. Suppose the training transformation returns session durations of 12 and 45 minutes for requests A and B, respectively, while serving returns 45 and 12 minutes for those same requests. The marginal distributions are identical, so a KS comparison finds no difference, but request-keyed comparison detects both mismatches. Conversely, a new population of shorter sessions can shift the marginal distribution while both paths still agree for every replayed request. Point-in-time correctness remains a separate requirement; agreeing paths can both use information unavailable at prediction time.

Listing 14.2 supplies a complementary distribution screen using the two-sample Kolmogorov-Smirnov (KS) test that the data pipeline already uses to watch for drift (section 4.5.3). The KS statistic is the largest vertical gap between the empirical cumulative distribution functions (eCDFs) of the training and serving samples. Setting the threshold to 0.1 asserts that cumulative probabilities do not diverge by more than 10 percentage points at any value across the feature's support. In this illustrative release policy, exceeding the threshold pauses promotion for review rather than diagnosing skew. The threshold must be calibrated against sample size and feature sensitivity. Passing the screen establishes only limited marginal divergence at that threshold; it does not establish per-request parity.

Listing 14.2: Feature Distribution Screen: This function compares training and serving feature distributions using the Kolmogorov-Smirnov test, pausing promotion for review under an illustrative threshold policy.

```
def screen_feature_distributions(
    training_features, serving_features, threshold=0.1
):
    """Pause promotion for review if distributions diverge."""
    for feature in training_features.columns:
        ks_stat = ks_2samp(
            training_features[feature], serving_features[feature]
        )
        if ks_stat.statistic > threshold:
            raise DistributionReviewRequired(
                f"{feature}: KS={ks_stat.statistic:.3f}"
            )
```

14.3.1.2 Versioning and lineage

Lineage tracking and versioning implement the reproducibility principle formalized in section 14.1.1. Software artifact provenance already depends on source code, build tools, dependencies, and configuration. In machine learning systems, model behavior is the joint product of code, hyperparameter configuration, and the training dataset snapshot. Changing any single dependency produces an

entirely distinct model artifact. Each artifact gets a content hash, so any model can be traced to its inputs.

Data versioning addresses dataset mutability by retaining raw data snapshots and intermediate feature tables under content-addressable hashes, so the recorded inputs to historical runs remain retrievable. Hashes identify artifacts but cannot reconstruct missing content or remove execution nondeterminism. Model versioning registers trained weights as immutable artifacts alongside training hyperparameters, evaluation metrics, and environment dependency manifests. A **model registry**[7] is the store that holds these immutable artifacts under lifecycle states (such as staging, production, or retired) and exposes lineage graphs tracing the complete dependency chain from raw ingested records to deployed prediction endpoints (MLflow Project 2026; Google Cloud 2024).

The registry is also what lets the release machinery promote, hold, or roll back the exact artifact that was evaluated. It stores immutable version tags, links evaluation metrics to artifact hashes, and governs which checkpoint each serving tier runs (Google Cloud 2024). This distinguishes a registry from an exploratory model catalog, which indexes candidate architectures for experimentation; a model enters the registry only when it becomes a versioned candidate for deployment. Experiment tracking frameworks such as MLflow[8] (Databricks 2024) and unified distributed runtimes such as Ray[9] record the surrounding execution state, training parameters, and lineage. Together, registry governance and lineage tracking turn experimental validation into controlled release, so a tested candidate can be promoted to production, monitored under live traffic, and reversed without losing provenance.

These mechanisms construct the lineage graph of the machine learning system. This graph establishes the audit trail required to diagnose production failures: determining whether an incoming request distribution diverged from training baselines, whether upstream feature calculations changed, or whether the serving environment loaded an unvalidated weight checkpoint. By recording provenance as a required, queryable artifact, systems engineering gives incident triage a verifiable record against which to test root-cause hypotheses.

14.3.2 Continuous pipelines and automation

Feature stores and versioning systems lock down data and artifact consistency at a single point in time, whereas production models must repeat preprocessing, training, evaluation, and release without an engineer running scripts by hand and risking a skipped validation check or a mismatched dataset path. Continuous automation binds those static controls into integrated workflows that respond to new data, shifting objectives, and operational constraints (Orr et al. 2021; Google Cloud 2026a).

14.3.2.1 CI/CD pipelines

CI/CD pipelines carry that consistency from the *data* side to the *process* side, ensuring that changes flow through validated stages rather than ad hoc deployments. ML CI/CD pipelines must handle complexity absent from traditional software: data dependencies, model training workflows, and artifact versioning that couple code changes to statistical behavior changes.

A typical ML CI/CD pipeline consists of coordinated stages: checking out updated code, preprocessing input data, training a candidate model, validating performance, packaging the model, and deploying to a serving environment. In some cases, pipelines also include triggers for automatic retraining based on data drift or performance degradation. By codifying these steps, CI/CD pipelines[10] reduce manual intervention, enforce quality checks, and support continuous improvement of deployed systems.

ML-focused CI/CD layers two tiers of tooling for one reason. A general-purpose CI/CD orchestrator (GitHub Actions (GitHub, Inc. 2024b), for example) manages version-control events and execution logic, but the ML layer must additionally version data, gate on model metrics, and trigger retraining. Teams therefore add an ML platform or workflow orchestrator (Kubeflow (Authors 2024), for example) that supplies higher-level abstractions for those tasks.

Without this automation, model deployment degrades into a manual, error-prone process in which an engineer retrains locally, copies artifacts to a staging server, and promotes to production with no guarantee that the data, code, or hyperparameters match what was validated. The cost of such ad hoc workflows compounds with team size and deployment frequency, producing configuration drift and silent regressions that surface only after the model has served incorrect predictions.

[7] **Model registry**: Prevents "registry bypass," the failure mode where the undocumented production model diverges from the trained artifact through different preprocessing, stale serialization formats, or manual hotfixes applied directly to the serving endpoint. Without a registry enforcing versioned, immutable artifacts with queryable metadata and state, rollbacks require locating the correct weights from an ad hoc artifact store under incident pressure.

[8] **MLflow**: An open-source MLOps framework that couples artifact storage (S3/GCS) with metadata DBs (PostgreSQL) to record hyperparameter runs, model binary hashes, and deployment stage transitions. The systems trade-off is centralizing lineage tracking vs. incurring database connection bottlenecks during massive parallel hyperparameter sweeps.

[9] **Ray**: A distributed computing framework from UC Berkeley (Moritz et al. 2018) that provides a unified task and actor interface, backed by a distributed scheduler and fault-tolerant store. The broader MLOps lesson is that fragmented infrastructure creates translation points where preprocessing logic, normalization constants, tokenizer versions, or artifact formats can diverge silently. Shared execution abstractions can reduce that fragmentation, but training-serving skew still requires explicit consistency checks across data, features, and serving code.

A continuous training pipeline runs seven sequential stages, from dataset ingestion, data validation, and data transformation through training, evaluation, validation, and registry promotion (figure 14.6). Driven by dedicated processing, training, and evaluation engines beneath the pipeline, the workflow coordinates with surrounding dataset, model, and metadata repositories whenever an external trigger initiates retraining.

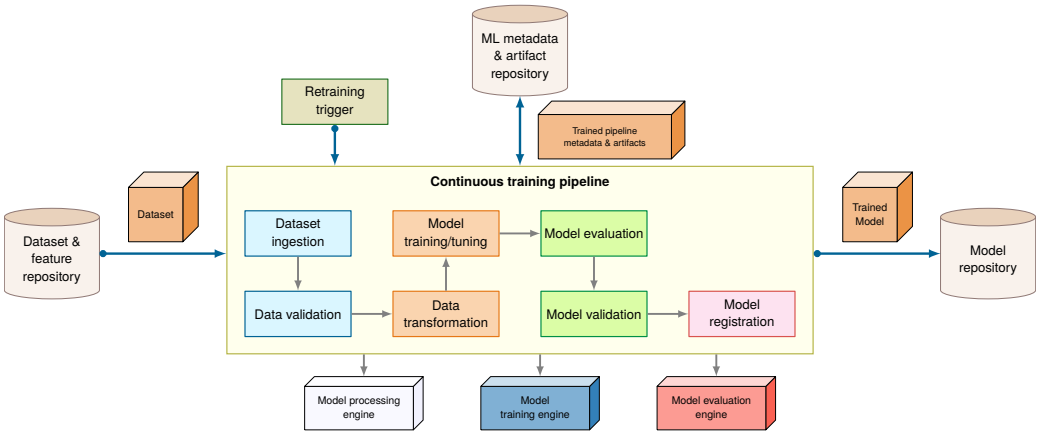


Figure 14.6: ML CI/CD Pipeline: Read the enclosed stages in execution order; the retraining trigger starts a new run, while connected repositories preserve the dataset, model, metadata, and artifacts needed to reproduce and govern it. Adapted from Google Cloud's MLOps continuous delivery and automation pipeline guidance (Google Cloud 2026a).

Consider the ranking model under active development. When a developer commits code changes, the CI pipeline pulls the specified dataset version, performs feature preprocessing, and initiates model training. Experiments are tracked in a registry such as MLflow (Databricks 2024), which logs loss curves and registers model artifacts. After passing automated offline validation, the model is containerized and deployed to a staging cluster. If the candidate satisfies staging validation, the pipeline executes a canary rollout (section 14.4.3), gradually shifting production query traffic to the new model while monitoring inference latency and error rates. If a performance regression emerges, traffic automatically rolls back to the previous model checkpoint.

CI/CD becomes foundational when systems release models repeatedly or carry meaningful production risk, turning ad hoc experimentation into structured, repeatable deployment. Scaling beyond one pipeline requires reusable components and explicitly specified artifact schemas.

> Example 14.1: Google TFX production ML pipelines
>
> **Scenario**: Google's production experience motivated TensorFlow Extended (TFX), a set of reusable components for production ML pipelines (Baylor et al. 2017). As pipelines multiply, ad hoc handoffs make validation, lineage, and reproducibility difficult.
>
> **Diagnosis**: Each stage needs explicit validation and metadata so a deployed model can be traced to the data, transformations, code, and evaluation that produced it.
>
> **Systems lesson**: Directed acyclic graph orchestration alone does not establish reproducibility. Standardized components and ML Metadata (MLMD), a store that records the inputs and outputs of every pipeline component run, preserve lineage through recorded inputs and transformations.

14.3.2.2 Training pipelines

CI/CD pipelines orchestrate the overall workflow, but training itself requires specialized compute infrastructure. Model training builds on the single-node parallel execution primitives that training systems already provide. Within MLOps, training activities become components of a reproducible pipeline supporting continual experimentation and production deployment.

[10] **Idempotency**: This property ensures that retrying a pipeline stage does not create duplicate externally visible effects, conflicting registrations, or repeated deployment actions. Producing identical models from identical inputs is a separate property (determinism and reproducibility), and the `training` stage may violate it due to randomness such as weight initialization. Production systems therefore combine stable run identifiers and atomic publication for idempotency with fixed random seeds, deterministic kernels, fixed library versions, and controlled data ordering for reproducibility.

↩ The single-node data- and model-parallel execution that training pipelines schedule is established in section 8.7.1.

Frameworks such as TensorFlow (Martín Abadi et al. 2016), PyTorch (Paszke et al. 2019), and Keras (Chollet et al. 2024) supply modular components for model architectures and optimization routines. Operational discipline determines which exploratory training routines graduate into versioned, tested retraining jobs, and under what execution triggers.

↩ The criteria for selecting a framework are established in section 7.9.1.

Training scripts and environment configurations are version-controlled with Git (Torvalds and Hamano 2024) and hosted on GitHub (GitHub, Inc. 2024a). Interactive Jupyter (Project Jupyter 2024) notebooks accelerate prototyping by coupling code, inline visualizations, and execution state, but running them unaltered in production pipelines introduces severe operational liabilities.

Notebooks in production Automated CI/CD pipelines assume deterministic and reproducible code execution, but Jupyter notebooks violate this assumption through persistent kernel state. In an interactive session, cells can be executed out of linear order, creating hidden memory dependencies that cannot be reproduced from a clean interpreter restart. A common production failure occurs when development relies on cells executed in sequence 1-3-2; a scheduled batch runner executing top-to-bottom (1-2-3) fails due to undefined variables or altered intermediate tensors.

Testing difficulties compound this state vulnerability. Standard unit-testing harnesses and mocking frameworks do not interface cleanly with interactive notebook documents. Cell-level assertions are rarely parameterized or wired into test runners, leaving notebooks significantly less tested than modular Python code.

Two tools help. Papermill enables parameterization and headless execution of notebooks, injecting runtime configurations and executing cells top-to-bottom as discrete pipeline stages (Papermill Project 2026). The nbconvert tool converts validated notebooks into static executable Python scripts for production environments (Project Jupyter 2026), exposing exported code to linters and CI test runners. Teams must still select or refactor exploratory cells and verify clean, ordered execution.

Extracting code from notebooks into tested modules introduces refactoring overhead, but it prevents hidden state assumptions from causing training-serving skew in production, a silent failure whose cost grows with every day it goes undetected, as the silent-failure calculation in the monitoring discussion shows (notebook 14.3).

Modular training logic also allows a pipeline to schedule repeatable experiments within the node's resource budget. Concurrent runs compete for accelerator memory, host preprocessing capacity, and storage bandwidth; enforcing resource limits recorded alongside each run's configuration keeps experiments within the capacity reserved for them. If execution can be interrupted, checkpoint-and-resume (section 8.4.5) preserves completed training work without requiring the entire run to restart.

How often to run this pipeline depends on evidence that only production monitoring supplies, so section 14.6 takes up the decision after monitoring. A pipeline that can run on demand still produces only candidates, and validation decides which of them may ship.

14.3.2.3 Model validation

Training pipelines produce model candidates; model validation determines which candidates merit production deployment. Unlike research evaluation, where a model that beats a benchmark on a static test set may be considered successful, production validation must assess operational readiness under representative conditions and establish the monitoring and rollback controls needed after the distribution changes. This is where MLOps manages the verification gap (principle 9) rather than closing it. No test suite covers the deployment population, so validation keeps re-estimating performance as that population changes.

The evaluation process begins with performance testing against a holdout test set sampled to represent the current production population and refreshed as that population changes. Core metrics such as accuracy, AUC, precision, recall, and F1 score (Rainio et al. 2024) are computed and tracked longitudinally to detect degradation from data drift (IBM 2024), the shift of the input distribution away from the one the model trained on. Section 14.5.2 shows how such drift appears in live telemetry.

Beyond static evaluation, production pipelines expose candidate weights to live traffic in controlled stages (shadow, canary, and blue-green deployment) while bounding the blast radius, and section 14.4.3 develops each. Validation defines the evidence every stage must produce before a candidate advances. Evaluating candidates under identical operational conditions is the prerequisite

for a sound promotion decision; a candidate that shows apparent metric gains simply because it was evaluated against a differing traffic slice or temporal window confounds model quality with data variance. Cloud ML platforms support controlled evaluation through request replay, shadow serving, and synthetic load testing. The comparison must also be recorded: experiment trackers such as Weights & Biases (Weights & Biases, Inc. 2024) log the data slice, time window, configuration, and metrics used for each candidate, so a promotion decision can be audited and repeated under the same conditions.

While automated validation gates protect against regressions on primary aggregate metrics, production validation also requires auditing slices and long-tail behavior. Automated test suites often mask severe performance drops on underrepresented subpopulations or safety-critical edge cases. Production architectures combine automated gating with policy reviews, particularly for models deployed in regulated domains. This multi-stage validation discipline bridges offline evaluation with live production monitoring, ensuring that candidate models satisfy accuracy, latency, and safety invariants before receiving unconstrained serving traffic.

14.3.3 Infrastructure integration

A versioned model checkpoint is still unsafe to deploy if the serving cluster runs a different feature definition or lacks a tested path back to the previous weights. Development infrastructure locks these pieces together across two of the three critical system interfaces: feature stores and data versioning reduce inconsistency at the data-model interface by centralizing and tracking feature access across training and serving, while CI/CD pipelines, model registries, and validation gates stabilize the model-infrastructure interface by automating the transition from trained weights to containerized services with rollback capability. Together, these controls prepare an artifact for release; the third interface, production monitoring, closes the operational loop by tracking whether predictive quality holds as inputs and operating conditions change, though the release must come first.

14.4 Releasing Models

A model that passes every offline gate has still not made a single production decision. Between the validated checkpoint and the first user-facing prediction lie packaging, conversion to a serving runtime, staged exposure to live traffic, and a path back if the new version misbehaves, and at each step the artifact users see can drift from the artifact that was evaluated. Deployment transforms a static checkpoint into a versioned, reproducible execution artifact. Serving then allocates accelerator memory, stages input batches, and schedules compute kernels to satisfy latency SLOs under dynamic request loads. Operations owns the seams between the two: proving that what ships is what was validated, limiting how much traffic a bad version can reach, and keeping rollback fast enough to use. On the ranking service each of those seams also costs memory, because any control that keeps two versions alive must hold two copies of the embedding tables.

14.4.1 Infrastructure as code

Holding two copies of the ranking tables across a staged release only works if staging and production provision the same accelerator hardware, memory limits, and scaling quotas. When teams manage clusters through manual commands or tickets instead of code, environments drift apart: a model that passed staging fails in production because an operator provisioned a different GPU type, a training job crashes because a colleague's experiment exhausted shared memory, or an inference service fails to scale because its resource quotas were set in an informal message six months earlier.

Infrastructure as code (IaC) treats infrastructure configuration as software (version-controlled, reviewed, tested, and automatically executed) rather than something configured through graphical interfaces or command-line tools. Changes are tracked, configurations can be tested before deployment, and environments can be reliably reproduced.

The specific tool matters less than the property it enforces. Infrastructure-as-code tools such as Terraform (HashiCorp 2014) version infrastructure definitions alongside application code, so a team can reproduce the accelerator type, network policy, storage permissions, and scaling limits of a training or serving environment on any cloud provider or on-premises hardware, and can audit and test those definitions in the same CI/CD pipelines as the model. Containerization provides the same reproducibility boundary for runtime dependencies. Docker (Merkel 2014) packages the model,

libraries, and serving code into an isolated unit, and orchestration systems such as Kubernetes (Cloud Native Computing Foundation 2024a) deploy that unit repeatedly while keeping resource allocation, scaling, and health management explicit.

To absorb spikes during hyperparameter tuning and surges in prediction traffic, teams use autoscaling[11] (Amazon Web Services 2024b), which adjusts compute capacity from usage metrics to balance performance and cost. The same infrastructure definitions must also cover on-premises and edge targets when latency, privacy, or regulation keeps part of a deployment off the cloud.

Infrastructure as code addresses how to provision resources reproducibly, but operators must still decide when and how much capacity to allocate. ML workloads break the assumptions of conventional web provisioning. Training jobs demand high-concurrency bursts across distributed accelerator topologies, creating friction between cluster utilization efficiency and time-to-solution. Inference services require sustained baseline allocations with rapid elastic scale-out to absorb request spikes under strict tail-latency SLOs. The utilization targets in section 14.5.1.1 turn these competing demands into measurable provisioning goals.

11 **ML autoscaling**: Autoscaling adjusts capacity based on demand signals (Amazon Web Services 2024b), but ML serving adds constraints absent from stateless web services. Autoscaling decisions must account for model loading time (cold-start overhead), GPU memory fragmentation, and batching behavior in addition to CPU utilization. Scaling up too slowly violates latency SLOs; scaling down too aggressively forces repeated cold starts that degrade p99 latency.

14.4.2 Validating the serving artifact

The artifact that reaches the serving fleet is rarely the checkpoint that passed evaluation. Before deployment, the model is usually compiled from eager framework code into a static graph for a specific runtime and often lowered in precision. Each conversion trades portability for speed, and each can change what the model computes, so the operational obligation is to prove that the converted artifact still behaves like the model that was validated.

↩ Inference runtimes and the precision choices they support for serving are established in section 13.6.

That proof must target the failure modes that conversion introduces silently. Consider a team that deploys an INT8-quantized model after verifying only throughput improvement. Classification accuracy drops on rare but high-value edge cases, and the degradation goes undetected for weeks because aggregate metrics remain within SLO bounds. The first validation layer is numerical equivalence, comparing optimized outputs against the original model on a representative test set with application-specific divergence thresholds. That check is necessary but insufficient, because a static calibration slice is not live traffic. Rare inputs, out-of-distribution examples, and subgroup-specific cases can expose quantization artifacts that aggregate test metrics hide, so the comparison must be disaggregated by the same cohorts that production monitoring tracks (section 14.5).

The second layer is operational validation. Memory footprint must be measured at peak runtime utilization, including dynamic allocations during inference, since some optimizations trade increased runtime memory for computational speed. Warm-up behavior varies by runtime: Accelerated Linear Algebra (XLA) may JIT-compile kernels during initial execution, whereas TensorRT selects tactics and builds an engine before deployment; loading that engine can still add startup latency. Runtime version compatibility then closes the loop. Deployment configurations need explicit version pinning, because even a minor runtime upgrade can change kernel selection, and with it both latency and numerical results, in an artifact whose weights never changed. An unpinned runtime is an untracked input to equation 14.1.

14.4.3 Model deployment

Consider a new version of the ranking model that beats its predecessor offline. An engineer exports the weights, copies them to a production server, and discovers that the model scores nearly every item the same, because the production server runs a different version of the feature extraction library and produces inputs the model has never seen. This scenario illustrates why deployment is a systems engineering problem rather than a simple file transfer. Packaging, testing, and tracking ML models for reliable production deployment requires treating the model weights, runtime dependencies, and preprocessing configuration as a single deployable unit. Containerizing the model artifact and serving runtime[12] packages user-space dependencies, runtime libraries, and environment variables into an immutable image, enforcing environment parity between development and cluster nodes.

Production deployment requires frameworks that handle model packaging, versioning, and integration with serving infrastructure. Tools like MLflow and model registries manage these deployment artifacts (A. Chen et al. 2020), while serving-specific frameworks handle the runtime optimization and scaling requirements. Before full-scale rollout, teams deploy updated models to staging or QA environments[13] to rigorously test performance.

12 **Containerization for ML deployment**: Docker (Merkel 2014) packages code with dependencies into portable units; Kubernetes (Burns et al. 2016) orchestrates those units across clusters. Containerization addresses the Environment_v term in equation 14.1 partially by capturing user-space dependencies as a versioned artifact; host kernels, drivers, and hardware remain external dependencies.

13 **Staging validation**: ML staging adds statistical quality checks to conventional functional and operational validation. A model can pass unit tests and still fail in production because the test data does not reflect the deployment distribution, so rollout gates must compare prediction statistics, evaluation slices, and business guardrails against calibrated thresholds rather than rely on unit tests alone.

A **staged rollout** carries that staging validation into live traffic in steps, widening a candidate's exposure only after the previous step shows no regression. Three release strategies put it into practice. **Shadow deployments**[14] duplicate live production traffic asynchronously to a candidate model without returning its inferences to users, enabling latency and output comparison without direct user exposure; accuracy still requires labels or another ground-truth reference. **Canary deployments**[15] route a small share of user traffic to the candidate and keep ramping that share toward full traffic as long as no regression appears under live load. **Blue-green deployments**[16] maintain parallel environments for router-level cutover and rapid rollback. Each strategy reduces a different release risk, but none removes the need for tested rollback procedures and production monitoring.

Release carries the canary stage of that staged rollout onto live traffic (Fowler 2014). During the limited rollout, telemetry pipelines monitor inference latency, system stability, and prediction distributions. On the ranking service, a candidate receives 5 percent of live queries while the team compares click-through rate, p99 latency, and prediction drift against the baseline model. Only after the candidate demonstrates statistical stability without latency regressions is it promoted to serve full production traffic.

> War Story 14.2: The Knight Capital error (2012)
>
> **Context**: Knight Capital Group was a major market maker in US equities handling over 17 percent of NYSE volume. In August 2012, an operational deployment updated SMARS, the firm's automated order-routing software, on seven of eight production servers but omitted the eighth (U.S. Securities and Exchange Commission 2013).
>
> **Mechanism**: The deployment repurposed a binary feature flag that, on the un-updated eighth server, activated Power Peg, an obsolete order-handling routine that had been left in the code. The stale server entered an un-throttled loop, transmitting parent order requests at $\sim 1{,}400$ orders/sec.
>
> **Impact**: In 45 minutes, the router executed over 4 million unauthorized transactions covering more than 397 million shares, incurring a $460 million loss. The firm survived only on emergency rescue financing raised days later, and lost its independence in an acquisition within months.
>
> **Response**: The incident motivates verifying that every production node runs the intended version before it receives traffic, explicit feature-flag lifecycle controls, and circuit breakers that limit the damage a faulty release can cause.
>
> **Systems lesson**: One server running the wrong version was enough to cause a $460 million loss. ML deployments share this vulnerability: a model registry version mismatch, un-synchronized feature schemas, or partial canary routing can flood production with corrupted inference requests before telemetry alerts fire.

Knight Capital shows why every node must be verified to run the intended version before traffic reaches it. Staged rollouts add a complementary protection by limiting how much traffic a bad version sees, but they can also expose load-dependent behavior. When canary deployments reveal anomalies at elevated traffic shares (such as memory exhaustion, queue spillover, or p99 tail latency inflation appearing at 30 percent traffic that remained undetectable at 5 percent), isolation requires systematic debugging across the stack. Effective diagnosis requires correlating multiple signals: performance benchmarking metrics, data distribution analysis to detect drift, and feature importance shifts that might explain degradation. Teams maintain debug toolkits including A/B test analysis frameworks, feature attribution tools, and data slice analyzers that identify which subpopulations are experiencing degraded performance. That diagnosis loop must connect directly to the release pipeline, because CI/CD integration automates deployment and rollback only when rollback is designed as part of the rollout mechanism rather than treated as an emergency script.

14.4.3.1 Rollback strategies and safety mechanisms

Without reliable rollback[17] capability, teams become deployment-averse and slow their iteration

[14] **Shadow deployment**: Economically justified when its expected reduction in rollout loss exceeds the cost of shadow infrastructure for duplicated inference without serving results.

[15] **Canary deployment**: The ML-specific challenge is that a "failure" is statistical degradation, not a deterministic crash: detecting a small accuracy difference with high confidence can require thousands of inferences, creating a tension between decision speed and statistical power that determines minimum canary duration.

[16] **Blue-green deployment**: Maintains two comparable production environments, "blue" (serving current traffic) and "green" (running the candidate), then switches traffic in a routing change once the green environment passes validation. Because rollback is a traffic flip rather than a staged drain, recovery can be faster than a gradual canary for stateless services. The trade-off is extra infrastructure during the switch, so blue-green wins over canary when brief duplicate capacity is acceptable and per-segment statistical validation is expensive.

↩ Staged rollout as the lifecycle's last validation gate, extending offline evaluation to live traffic, is established in section 3.8.2.

[17] **Rollback (from database transaction management)**: This "undo" action for deployments is complicated in ML by model-dependent state (for example, cached embeddings), which may be incompatible between model versions. This mismatch can prolong recovery and contribute to deployment aversion and slower iteration.

velocity, so effective rollback plans for three distinct scenarios. The fastest tier, **immediate rollback**, addresses critical failures detected right after deployment, such as serving errors, latency spikes, or obvious prediction failures, keeping the previous model version loaded and warm to switch traffic without cold-start delay. **Rapid rollback** handles performance degradation detected through canary metrics soon after deployment and relies on model registry integration to redeploy previous versions with minimal configuration changes. Once a release has ramped to full traffic, **delayed rollback** addresses subtler regressions that surface later in business metrics or user feedback, where reverting must also handle model-dependent data such as personalization state or cached embeddings accumulated under the new version. Table 14.9 summarizes implementation patterns for each rollback type:

Table 14.9: Rollback Patterns by Scenario: Each rollback type requires different infrastructure support and state handling strategies. Immediate rollback demands always-warm standbys; delayed rollback may require data migration procedures.

Rollback Type	Trigger	Implementation	State Handling
Immediate	Serving errors, crashes	Hot standby with fast routing switch	Verify state compatibility
Rapid	Canary metric degradation	Registry-based redeployment	Clear caches, restart sessions
Delayed	Business metric decline	Full redeployment with migration	Migrate state, replay if needed

Rollback testing Rollback procedures that have never been exercised often fail during an active incident, when cognitive load and time pressure are highest, because configuration dependencies, incompatible state, or unclear ownership only surface when a revert is actually attempted. Periodic rollback drills expose those gaps beforehand, while step-by-step runbooks ensure that the on-call responder can execute the revert without having designed the deployment. Automated rollback triggers shorten reaction time further, provided their thresholds are calibrated to the service with safeguards against flapping and the restored model is checked against live caches rather than serving predictions corrupted by incompatible feature state.

Stateful vs. stateless rollback ML systems vary in statefulness, which directly governs rollback complexity. For stateless workloads like tabular classification and static regression, rollback requires only swapping model weights or routing traffic back to the prior container, because individual inferences share no execution context. In contrast, stateful models (including sequential recommenders, session-based search rankers, and conversational systems) accumulate transient user state. Reverting such workloads can demand session resets, schema shims for cached context vectors, or explicit state migration; capturing versioned checkpoints at deployment boundaries mitigates this friction, but clean restoration remains an architectural property that requires empirical validation rather than assumption. The ranking service is the stateful case. It precalculates dense user and item embeddings and holds them in low-latency caches, so if version 2 rotates the latent vector space or changes the embedding dimension, the cached vectors become meaningless to a rolled-back version 1. A stateful rollback must therefore orchestrate atomic cache invalidation or dual-version namespace switching alongside weight redeployment to prevent scoring queries against incompatible vector spaces. In systems with feedback loops, the most complex failure mode arises when bad predictions pollute downstream feature logs or clickstream training sets during the degraded window, reverting the serving binary leaves contaminated data in the retraining corpus that must be expunged or quarantined before previous behavior is restored.

14.4.3.2 A/B testing for model validation

A/B testing provides the statistical foundation for deployment decisions by comparing model versions under controlled conditions. Unlike canary deployments (which validate operational stability), A/B tests measure whether a new model improves business outcomes with statistical confidence.

Experiment setup and decision rules A valid A/B test establishes four controls to ensure deployment decisions remain statistically sound. First, the **randomization unit** specifies whether traffic is split by user ID or individual request: user-level hashing preserves consistency across sessions

at the expense of larger sample requirements, whereas request-level splitting maximizes sample throughput but risks exposing a single user to alternating model variants. Second, the statistical power envelope determines the traffic volume required to resolve meaningful differences. Under a common-variance Normal approximation, let n be the required users per variant, δ_{MDE} the minimum detectable effect, σ the outcome standard deviation (for a conversion metric, each user's outcome is a yes-or-no event, so σ^2 is the baseline conversion rate times one minus that rate), and $z_{\alpha/2}$ and z_{β} the positive standard-Normal critical values for the chosen two-sided confidence and power. The worked scenario uses 95 percent confidence and 80 percent power. Equation 14.4 translates those design targets into required traffic before launch.

$$n = \frac{2(z_{\alpha/2} + z_{\beta})^2 \sigma^2}{\delta_{\text{MDE}}^2} \tag{14.4}$$

For a 2 percent relative lift on a 5 percent baseline conversion rate (5 percent to 5.1 percent) and 80 percent power, the experiment requires roughly 745,644 users per variant. If traffic limits the test to 25,000 users per variant instead, solving equation 14.4 for δ_{MDE} shows that the test can detect only a much larger lift, about 0.5 percentage points absolute.

Third, **guardrail metrics** define operational and behavioral boundaries that must not degrade even if the primary metric improves; a recommendation model improving click-through rate by 10 percent while increasing p99 response latency by 500 ms violates system guardrails and risks cascading timeouts across downstream services. Fourth, a fixed evaluation horizon runs the test until the preregistered sample size and a minimum duration covering weekly seasonality are reached. Stopping the moment significance first appears inflates false positive rates through repeated peeking; sequential designs permit early stopping but need specialized analysis.

Those controls establish the statistical envelope, yet ML systems add failure modes that ordinary web experiments can hide. Conversion events may arrive days after prediction, creating delayed feedback: a recommendation shown Monday can drive a purchase Friday, so the attribution window must be part of the test design. Novelty effects can also inflate early performance as users engage with fresh recommendations, which is why mature experiments include a burn-in period before measurement. Ranking systems add interference effects, because showing an item to one user can change what remains available to another, violating the independence assumption behind standard A/B analysis, and segment heterogeneity creates a further problem when an overall neutral result hides opposite effects in different cohorts. Guardrails, segment analysis, and preregistered decisions therefore belong to the experiment design, and table 14.10 turns those constraints into a deployment decision:

Table 14.10: A/B Test Decision Matrix: Deployment decisions should consider both primary metrics and guardrails. Improvements that come at the cost of guardrail violations require careful trade-off analysis rather than automatic deployment.

Primary Metric	Guardrails	Decision
Significant improvement	All pass	Ship new model
Significant improvement	Some fail	Investigate trade-offs, may need model iteration
No significant change	All pass	Evidence inconclusive; retain current model unless a prespecified equivalence or noninferiority test supports the change
Significant degradation	N/A	Do not ship; investigate root cause

The decision matrix is reliable only when the evaluation methodology is preregistered before launch, establishing expected effect sizes, the randomization unit, attribution windows, guardrail thresholds, and minimum runtimes prior to observing production traffic. Variance-reduction techniques such as CUPED (Controlled-experiment Using Pre-Experiment Data) (Deng et al. 2013) subtract the part of each user's outcome that pre-experiment behavior already predicts, shortening experiments without sacrificing power, while automated analysis pipelines protect the decision boundary from manual calculation errors and archiving negative results discourages repeating unproductive architectures. An A/B deployment decision is actionable only when the release machinery can promote, hold, or roll back the exact artifact that was evaluated, which is the guarantee

the model registry of section 14.3.1.2 provides. The keyword spotter keeps these operational goals (controlled rollout and fast rollback) while changing the constraints under which they must be met, because its devices have limited power, memory, and connectivity.

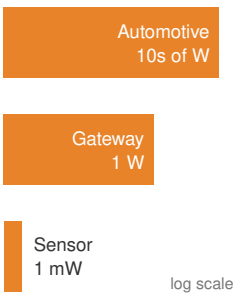
14.4.4 Edge AI deployment

Consider the keyword spotter after its detection drop. When its model degrades, an engineer cannot SSH into the device, roll back to a previous version, and restart. The device sits in someone's home on intermittent Wi-Fi, and its 200 KB model shares 4 MB of flash with the firmware that runs it. Every operational assumption from cloud MLOps (instant rollback, centralized logging, real-time monitoring) must be reimagined.

Edge AI runs machine learning inference at or near the data source rather than in centralized cloud infrastructure (Reddi et al. 2019), and its operations inherit the memory and power envelopes that separate the deployment tiers (section 2.8). A model reaches an edge device already quantized, pruned, or distilled to fit those envelopes (Warden and Situnayake 2020; David et al. 2021), and on microcontrollers (TinyML) it runs on specialized engines such as TensorFlow Lite Micro and CMSIS-NN (Lai et al. 2018). Power budgets span about four orders of magnitude, from milliwatts for sensors to tens of watts in automotive systems, and deployments split work across three tiers: sensor-level microcontrollers at 1–100 mW filter and detect, gateways at 1–10 W run heavier inference, and the cloud distributes models and retrains. Latency-critical decisions stay local while expensive work moves upward. Every operational control must fit inside the tier that runs it. An update shares flash with the running model, telemetry competes with the radio's energy budget, and a failed release must recover without a technician. Safety-critical real-time targets also add worst-case execution time (WCET) analysis to the release gate, because thermal throttling and memory contention can push inference past a deadline that average latency never reveals.

↩ The quantization, pruning, and knowledge-distillation techniques that edge deployment relies on are established in chapter 10.

Over-the-air (OTA) model updates are how operations reaches physically inaccessible devices. OTA pipelines need secure model distribution, artifact verification, and rollback mechanisms; delta or differential updates can reduce transfer volume when devices need not receive a complete model artifact. The keyword spotter's fail-safe OTA design uses dual-bank flash memory, a pattern common on microcontroller and embedded targets. The device keeps running from Bank A while it writes the candidate model and runtime to Bank B. After switching to Bank B, a hardware watchdog timer and a post-update self-test supervise the first inferences; if the new model triggers memory faults, hangs, or violates latency budgets, the bootloader automatically reverts execution to Bank A, preventing physically inaccessible devices from being bricked. Update scheduling must account for device connectivity patterns, power availability, and operational criticality.



Power budgets across the three edge tiers span about four orders of magnitude.

Monitoring and tier coordination must fit inside the same power and bandwidth budgets. Because continuous streaming would drain the battery, lightweight telemetry reports compact operational counters (inference latency, power draw, and accuracy indicators) alongside device-level health signals (thermal status, battery levels, and connectivity quality) that predict maintenance needs before a node fails. When local load or link quality shifts, edge-cloud coordination can offload heavier inference to a gateway where feature caching avoids redundant computation, while federated learning lets edge devices contribute model updates rather than raw training records, paired with secure aggregation and differential privacy to limit what those updates reveal under an explicit threat model (section 15.5.2 develops both).

Graceful degradation is a core operational pattern for edge AI. When thermal limits or battery depletion constrain hardware capacity, edge runtimes must dynamically shed load (downsampling input sample rates, bypassing speculative execution branches, or falling back to shallow heuristic models) within a fallback policy validated for the application's safety and quality requirements. Each of these controls, on the device and in the cloud, reserves resources for a version that serves little or no traffic, and the size of that reservation depends on the workload's regime.

14.4.5 Memory and compute of release controls

Every control in this section keeps a second version of the model alive, and for a capacity-bound model that second version, not the extra arithmetic, is the dominant cost (table 14.11). A shadow deployment scores every request twice. A canary, an A/B test, and a blue-green switch divert traffic rather than adding work, yet each must hold the candidate's full embedding tables in accelerator

memory whatever share of traffic it serves, and a hot standby for immediate rollback holds the previous version for the same reason. The ranking service's 5 percent canary therefore doubles the accelerators that hold tables, from two to four, while it carries a twentieth of the requests. A compute-bound ResNet-50 service pays the opposite way. Its 51.2 MB of FP16 weights fit beside the serving copy on one accelerator, so its canary costs almost no memory and only its share of compute. The keyword spotter's dual-bank update costs flash instead of accelerator memory, a second 200 KB bank that brings the model's share of the device's 4 MB to 400 KB.

Table 14.11: Release Controls on a Capacity-Bound Model: Extra resident memory and inference work that each release control adds to the DLRM ranking service on H100 accelerators, and to the keyword spotter's flash. Every server-side control holds a second full copy of the embedding tables regardless of the traffic share it serves, so release safety for this model is paid in memory rather than arithmetic.

Release control	Extra resident memory	Extra inference work	What it buys
Shadow deployment	Second copy of the tables (100 GB); 2 → 4 H100s	Every request scored twice	Output and latency comparison with no user exposure
Canary (5% of traffic)	Second copy; 2 → 4 H100s	None; a share of requests is diverted	Live quality evidence on a bounded share of users
A/B test	Second copy; 2 → 4 H100s	None; traffic is split between variants	A statistically valid outcome comparison
Blue-green switch	Full second environment; 2 → 4 H100s	Duplicate provisioned capacity during the switch	Rollback by a single routing change
Hot standby	Previous version resident; 2 → 4 H100s	None until rollback	Immediate rollback without a cold start
KWS dual-bank update	Second flash bank (200 KB in 4 MB of flash)	None	Automatic revert after a failed update

Checkpoint 14.2: Evidence for a release

- ☐ **Promotion evidence**: A shadow model has acceptable latency and similar output distributions, but outcome labels have not arrived. What has been established, and what quality evidence is still missing?
- ☐ **Recovery state**: A canary changes an embedding dimension. Which artifacts and cached state must a tested rollback restore together?
- ☐ **Converted artifact**: Explain why passing aggregate accuracy before quantization does not replace numerical, slice-level, and runtime checks after conversion.

Getting a model into production, and back out again, is only half the operational challenge. A successfully released model can still degrade through distribution shift or corrupted upstream features without raising runtime errors, and none of the controls above will notice. Production monitoring supplies the evidence that tells the team a release, a rollback, or a retraining run is needed at all.

14.5 Production Monitoring

A newly promoted ranking model can pass its canary gate on Monday, only to lose five points of accuracy by Thursday when an upstream catalog migration or a competitor's promotion alters live user traffic while every HTTP and GPU gauge stays green. *A deployed model loses accuracy at a rate that must be measured rather than assumed.* Every decision left in the operations loop (retrain, roll back, or wait) rests on evidence that only production monitoring can supply, including the decay rate that the retraining economics of section 14.6 must fit. An ML service is correct only while its live inputs resemble the data it learned from, so as they change (the silent degradation of section 1.6), predictive quality may decline. Monitoring is where the third critical interface, production-monitoring, makes

that risk observable. Staged rollout paired with input and outcome monitoring lets teams investigate failing versions and limit their impact, within the time needed to collect evidence.

14.5.1 Infrastructure telemetry and hardware utilization

When an on-call engineer is paged for a p99 latency spike on the ranking service, a single GPU busy-time gauge cannot tell them whether the accelerator is saturated with math, stalled on embedding-table memory gathers, throttling its clocks from heat, or sitting idle while a host thread waits on the feature store. Confirming whether predictive quality has changed also requires ground-truth labels that often arrive with substantial delay, so monitoring cannot wait for outcomes alone. In production, serving failures span two different physical regimes: immediate hardware execution bottlenecks and gradual distributional drift. Disentangling them requires two distinct telemetry layers: infrastructure metrics that reveal whether the serving system itself is compute-, memory-, or I/O-bound, and distribution and outcome metrics that connect data movement to model performance.

The first layer is infrastructure-level monitoring, which tracks indicators such as CPU and GPU utilization, memory bandwidth, disk consumption, network latency, and service availability. Measuring GPU utilization alone is incomplete; an accelerator reporting high execution time may still be stalled waiting for DRAM or PCIe transfers, while low utilization often signals host-side preprocessing bottlenecks. Correlating kernel execution times with memory bandwidth utilization and input pipeline queue depths distinguishes compute-, memory-, and I/O-bound behavior.

Power-efficiency metrics (such as inferences per joule or FLOP/s/W, depending on workload) add a cost-normalized view that enables mixed-workload scheduling. Thermal monitoring similarly informs operational scheduling, particularly under sustained high load where thermal throttling forces hardware clocks down, introducing tail-latency spikes that violate SLOs. An MLOps telemetry pipeline incorporates thermal headroom metrics to steer traffic across available accelerators before throttling occurs. Time-series engines such as Prometheus[18] (Cloud Native Computing Foundation 2024b), Grafana (Labs 2024), and Elastic (Elastic NV 2024) collect and visualize these operational metrics across the serving fleet.

[18] **Prometheus**: Prometheus periodically scrapes targets and stores time series for aggregation. Scrape and rule-evaluation intervals jointly affect alert latency and may miss short excursions. Monitoring per-accelerator thermals allows precise workload routing at higher data cost, while server-level aggregates can mask component-level throttling.

Hardware counters locate a bottleneck inside the accelerator, but a latency regression often originates elsewhere on the request path. Request scheduling, batching, and runtime selection belong to the serving system, and section 13.3.1 shows that model inference is only one slice of end-to-end latency, and often not the largest (principle 12). Operations uses that decomposition as a diagnostic instrument. When p99 regresses, per-stage timings for network, feature retrieval, parsing, inference, and serialization tell the on-call engineer which stage moved, and therefore which fix and which team the incident needs. Without stage-level tracing, every latency regression looks like a model problem, and the reflexive responses of rolling back or quantizing further cannot recover time lost in a feature store or a network hop (principle 8).

The serving endpoint contributes two of those stages, parsing and serialization, and its capacity model adds a third when it scales to zero. Protocol and serialization costs are developed in section 13.2.2, and the cold start that scale-to-zero capacity pays after every idle period in section 13.8.1. Operations inherits both as stages to trace, so that a regression in either is not mistaken for a model problem.

Alerts convert these signals into intervention before silent degradation reaches downstream consumers. A sustained drop in model accuracy triggers drift investigation; infrastructure alerts signal memory saturation, thermal throttling, or degraded network throughput. The latency of these alerts governs the window of undetected failure. Alerts must record the model version, affected input subpopulation, and triggering aggregation window so responders can isolate and reproduce the failure state.

Example 14.2: Recommendation monitoring at scale

Scenario: Consider the ranking service at high throughput, its latency within the SLO while recommendation quality declined as user behavior changed.

Diagnosis: Traditional infrastructure metrics (CPU utilization, HTTP error rates) remained green while cohort-level CTR fell. The engagement decline motivates investigation of traffic, feature pipelines, and ranking behavior; it does not establish classification accuracy loss.

Systems lesson: Infrastructure telemetry alone cannot establish recommendation quality. Cohort-level outcome tracking exposes localized engagement changes, while matched comparisons and mature labels are needed to assess predictive-quality changes and their causes.

14.5.1.1 Hardware utilization patterns

Standard administrative GPU utilization metrics, such as the reading from `nvidia-smi`, report the fraction of time a kernel is executing rather than the fraction of peak arithmetic throughput achieved, the distinction between busy time and model FLOPs utilization (MFU) drawn in section 8.5. The ranking service's gathers of non-contiguous embedding vectors can therefore hold a high utilization reading while every resident warp stalls on memory, and a green gauge on the operations dashboard says nothing about compute efficiency.

Distinguishing compute execution from memory or pipeline stalls requires evaluating streaming multiprocessor (SM) execution efficiency alongside memory bandwidth and I/O saturation. Table 14.12 gives illustrative screening patterns. Here GPU utilization is kernel-active time; measured FLOP rate, execution traces, and relevant stall counters are needed to test the proposed bottleneck:

↩ The analytical model separating compute-bound execution from memory-bandwidth saturation across arithmetic intensities is established in section 11.6.

Table 14.12: GPU Utilization Patterns: Illustrative utilization signatures suggest hypotheses, not unique diagnoses. High kernel-active time does not establish compute saturation. Confirm candidates with achieved arithmetic throughput, memory traffic, stall counters, and an execution trace before choosing an optimization.

Pattern	GPU Util	Memory bandwidth util.	Optimization Strategy
Possible compute limit	>85%	<70%	Check achieved FLOP rate; evaluate faster kernels or compute capacity
Possible bandwidth limit	50–85%	>85%	Check traffic and stalls; evaluate quantization or memory access
Possible input limit	<50%	<50%	Trace input starvation before changing prefetch or storage
Batch-starved	Variable (spiky)	Variable	Dynamic batching, request queuing on single server

Each candidate maps to a term of the iron law of ML systems (principle 3). The compute-bound signature points to $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$, the memory-bound signature points to D_{vol}/BW, and low utilization of both points to the data pipeline, so the trace and counters guide which term to investigate when a latency SLO is violated.

Utilization targets differ by workload. Batch training and batch (static) inference, which tolerates queuing and offline scheduling, can run above 80 to 85 percent kernel-active time while the team measures useful throughput. Online inference instead targets 50–70 percent of measured sustainable capacity at median load, holding back headroom so that bursts do not cascade into p99 timeouts (section 13.4). These figures are diagnostic starting points rather than thresholds, because kernel-active time and demand relative to capacity have different denominators. For large language model (LLM) serving, KV-cache utilization on each node, the per-request store of past attention state, is an equally important capacity-planning signal, because near the limit the scheduler must queue or reject requests, shrink batches, or shorten the admissible context (section 13.9.5 sizes and schedules the cache).

Utilization and memory telemetry also feed the number the business sees. Cost per inference, the hourly accelerator bill divided by sustained throughput, is defined and worked out in section 13.10.1; operationally it belongs on the same dashboard as the utilization signatures in table 14.12, because a rising cost per thousand inferences at constant pricing is often the first visible symptom of efficiency erosion (smaller effective batches, a shift in traffic mix, or a regression that stays inside the latency SLO).

14.5.2 Statistical monitoring and drift detection

Even when GPU utilization, memory bandwidth, and stage-level latency sit comfortably inside their operating bands, a model can still return systematically wrong predictions if live user inputs drift away from the training distribution or if an immature evaluation window compares pending outcomes against completed ones. Infrastructure telemetry shows how the serving hardware behaves; statistical monitoring shows whether the model's predictions remain correct. Unit tests can verify deterministic components but cannot by themselves establish population-level predictive performance, which must be estimated statistically. Monitoring implements observable degradation (section 14.1.1), transforming this theoretical limitation into operational practice. Once monitoring surfaces a symptom (a latency SLO miss, throughput below target, or memory creep), section A.5.3 maps that symptom to its dominant D·A·M term and tells the operator which optimizations will move the binding constraint and which will be wasted on serving infrastructure. Without continuous monitoring and the deeper **observability** it enables (the ability to infer internal state from outputs), teams lack the evidence needed to distinguish a healthy model from an unobserved regression.

Effective monitoring spans both infrastructure health and statistical model behavior. On the infrastructure side, metrics track hardware saturation, memory allocation, and latency distributions. On the model side, teams track evaluation metrics such as accuracy, precision, recall, and confusion matrix shifts (scikit-learn developers 2024) across live prediction streams.

Those quality metrics require joining predictions to outcomes under a fixed evaluation horizon. Consider a conversion model whose prediction log records a request identifier, prediction time, score, and the immutable version actually served. The outcome log records the same identifier, event time, and observation time. For an older request served by the previous version, a purchase event within the horizon supplies a positive label once observed; a completed observation window with no purchase supplies a negative label, provided outcome collection is complete. A recent request served by the candidate with no recorded purchase remains pending until its horizon and the permitted reporting delay have elapsed. The absence of a purchase record while that window is open is unknown, not a negative outcome. Joining by identifier and retaining the served version attributes each completed outcome to the model that made the prediction rather than to whichever model is deployed when the label arrives.

A quality comparison groups predictions by serving period and version, then evaluates cohorts whose outcome windows have matured under the same horizon and reporting policy, with comparable traffic conditions. Each metric and slice must report label coverage, the fraction of eligible predictions with resolved outcomes. An apparent drop caused by comparing the candidate's pending outcomes with the previous version's completed outcomes does not justify retraining; it calls for completing the window or repairing outcome collection. Even a mature window can contain selective labels if outcomes are observed only for exposed recommendations or reviewed transactions. Coverage and collection-policy checks must accompany uncertainty estimates before a quality regression supports diagnosis and a retraining decision.

Statistical monitoring has to separate two axes of distribution shift,[19] because they differ in what a monitor can observe. The three kinds of shift defined for the data pipeline in section 4.12.1, where the keyword spotter's microphone change is the worked case, fall on those two axes. Covariate shift[20] changes the input distribution $p(x)$ while the relationship between inputs and labels, $p(y \mid x)$, stays stable; label shift changes how often each label occurs while the inputs typical of each label stay the same; and concept drift changes $p(y \mid x)$ itself. Covariate shift and label shift can usually be detected without labels, from input statistics or from the model's prediction distribution, and together make up data drift. Label shift does change $p(y \mid x)$, because the probability of each label given an input follows the changed label frequencies, but its signature appears in the prediction distribution before any outcome arrives. Concept drift, a change in $p(y \mid x)$ that changed label frequencies do not explain, forms the second axis, and detecting it requires labeled outcomes. The degradation equation compares P_0 and P_t, the training-time and serving-time measurements of the data distribution in which these shifts appear.

Operations adds one distinction that the definitions alone do not make. **Model decay** is the observed decline in prediction quality over time while the model code stays unchanged, and data drift is only one possible cause of it, alongside concept drift and changes in the surrounding pipeline. Monitoring model outputs alone can miss input drift or mistake a benign output change for quality

[19] **Label-free vs. label-dependent detection**: A change in $p(x)$, or in the model's prediction distribution, can be estimated from sufficient samples without labels, although detection is not immediate. Concept drift generally requires labeled outcomes, which in high-stakes domains (medical diagnosis, fraud detection, legal decisions) can take days, weeks, or months. Proxy metrics such as prediction-confidence distributions and output entropy can provide imperfect early warnings that trade false-alarm rate for detection speed.

[20] **Covariate shift**: Importance-weighting corrections for covariate shift assume that the support of the training distribution covers the deployment distribution: every deployment input could have appeared in training, just with different probability. When deployment contains genuinely out-of-distribution inputs (new product categories, new demographics, adversarial inputs), the correction can fail and the model can produce confidently wrong outputs with no warning signal, making support coverage the hidden assumption that determines whether drift correction or full retraining may be required.

loss, so operations also tracks the input feature statistics behind the divergence $\mathcal{D}(P_t \| P_0)$, using the distance measures of feature distribution monitoring (section 22). These statistics can warn earlier than labeled performance because ground-truth feedback is often delayed.

The signature of data drift in live telemetry is a feature distribution that moves in step with a quality metric, and three synchronized telemetry tiers aligned over time expose how covariate shift erodes model performance (figure 14.7). Across the top panel, discrete request samples transition from predominantly green (online sales) to orange (in-store sales); the figure's "online store" and "offline store" labels name sales channels, not the feature-store tiers of section 14.3.1.1. The middle panel charts this shift as an expanding offline area in the sales_channel feature distribution, directly matching the steep decline of the model accuracy curve in the bottom panel. This temporal alignment motivates comparing input feature distributions with outcome metrics; their co-movement supplies a diagnostic hypothesis rather than establishing the cause by itself.

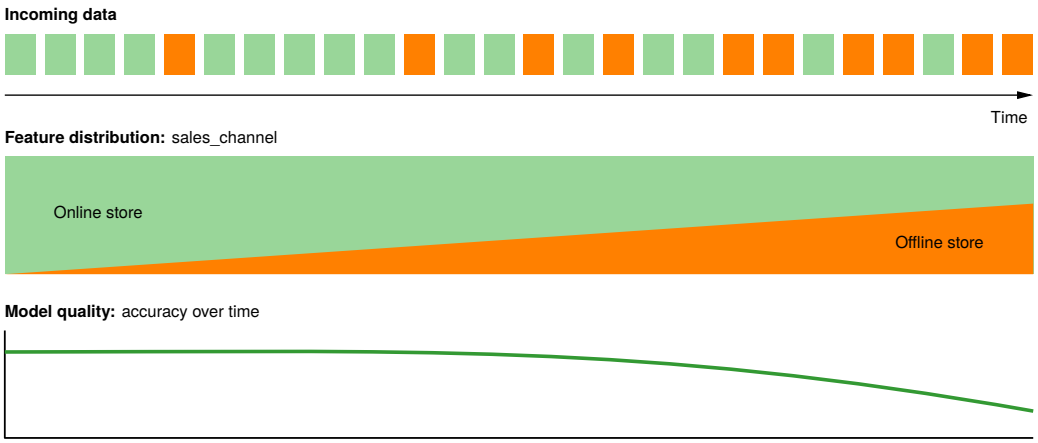


Figure 14.7: Data Drift Impact: In this illustrative scenario, incoming samples shift from predominantly online to increasingly offline sales while model accuracy declines over the same interval. Monitoring inputs and outcomes together helps determine whether the distribution shift is associated with the loss in accuracy.

The **drift detection delay**, how quickly a statistical test can confirm that predictive quality has degraded, limits statistical monitoring. The speed of confirmation is governed by sample arrival rates and statistical power requirements. Low-traffic services may wait days or weeks to accumulate enough labeled outcomes to distinguish genuine performance erosion from stochastic noise. No tooling removes this gap, because a change cannot be confirmed faster than labeled examples arrive. Monitoring architectures must therefore keep the two axes apart. Data drift can be estimated from sufficient unlabeled samples, while concept drift generally requires labeled outcomes. Neither statistical estimate is instantaneous.



Drift detection speed is bounded by the sample rate.

Napkin Math 14.2: The drift detection delay

Problem: A 95 percent-accurate model may fall by 5 percentage-point. The monitoring policy budgets 1,000 labeled examples. A power calculation of the same kind as equation 14.4 sets the minimum such a test needs; the budget is a policy choice above that floor, adding margin for label noise, dependence between outcomes, and per-slice checks. Calculate the collection duration required across high-rate and low-rate arrival scenarios.

Math:

1. **Evidence budget**: 1,000 labeled examples.
2. **High-rate case**: At 1 labeled outcome per second, collection takes 1,000 seconds ≈ 16.7 minutes.
3. **Low-rate case**: At 100 labeled outcomes per day, collection takes 10 days.

Systems insight: Detection is bounded by labeled-outcome arrival, which may lag far behind traffic. Low-volume or delayed-label domains may need days or weeks, so high-stakes systems supplement statistical monitoring with proactive model audits.

Detection delay has a direct cost. When a silent failure persists until the next review, every day of delay is lost revenue, which is the economic case for automated drift detection.

Napkin Math 14.3: The cost of silent failures

Problem: Is building an automated drift detection system worth the engineering effort?

Scenario: Consider a product recommendation engine generating $50M in annual revenue. **Failure mode**: A deployment bug causes training-serving skew, dropping recommendation quality by 5 percent. This degrades conversion rate proportionally.

Assumptions: The incident begins immediately after a scheduled check, remains detectable at the next check, and ends with an effective repair immediately after detection. The calculation therefore compares nearly full review-interval delays, rather than expected detection times for incidents arriving throughout an interval. It excludes label delay and diagnosis or recovery time.

Cost analysis:

1. **Manual ops (monthly review)**:
   - **Detection time**: ~4 weeks (28 days).
   - **Revenue loss**: $\$50\text{M} \times 0.05 \times 28 \text{ days}/365 \text{ days} \approx \$191{,}781$.
2. **Automated MLOps (daily checks)**:
   - **Detection time**: 1 day.
   - **Revenue loss**: $\$50\text{M} \times 0.05 \times 1 \text{ day}/365 \text{ days} \approx \$6{,}849$.

Systems insight: Under these timing and recovery assumptions, a single silent failure costs $184,932 more with monthly review. Across this scenario's 4 incidents/year, daily checks avoid nearly $739,726 in annual loss. An operational budget must instead use the measured time from incident onset to effective recovery, including evidence collection, diagnosis, and mitigation. An earlier alert saves money only when it enables an earlier reduction in harm.

14.5.2.1 Data quality monitoring

Waiting for labels is not the only option, because many silent failures begin as defects in the inputs themselves and can be caught at the input boundary before a prediction is served. Drift statistics reveal shifted inputs, while outcome metrics establish whether prediction quality has declined, yet by the time downstream business metrics degrade, upstream data corruption may have persisted for days or weeks. Data quality monitoring exposes these input defects before inference occurs, while output and outcome monitoring confirm whether distributional shifts impair prediction accuracy. The first input guardrail is executable validation: listing 14.3 illustrates schema expectations that reject malformed batches before inference, turning implicit data assumptions into enforceable checks.

Listing 14.3: Input Data Validation: Schema validation rules check column existence, data types, null values, and statistical bounds to catch data quality issues before they propagate to model inference.

```
schema.require_column("user_id")
schema.require_type("timestamp", "datetime")
schema.require_non_null("feature_a")

schema.require_range("age", min_value=0, max_value=120)
schema.require_mean_between(
    "purchase_amount", min_value=10, max_value=1000
)
```

Input data validation Schema validation[21] catches structural defects at ingest time. These checks enforce column existence, strict data typing, null-value limits, and numerical boundary invariants.

Feature distribution monitoring Schema validation catches structural corruption (missing columns, wrong types, null values) but cannot detect the subtler failure mode where data arrives in the correct format but from a shifted distribution. A feature representing user age can pass every schema check while its mean silently migrates from 32 to 45 over three months as marketing attracts an older demographic. This distributional shift can change model accuracy without producing a structural anomaly; labeled outcomes are needed to determine its effect.

Gradual long-term degradation evades coarse detection thresholds. Small day-to-day changes in a quality metric compound into material degradation over a year without tripping monthly alerts. Seasonal patterns compound this complexity; a model trained in summer may perform well through autumn but fail in winter conditions it never observed. Detecting such gradual degradation requires multi-timescale monitoring: performance baselines across multiple time horizons (daily, weekly, quarterly), sliding window comparisons that detect slow trends, and seasonal profiles that account for cyclical variation.

Statistical distance measures quantify this divergence by comparing serving distributions against training baselines, and four are common. The population stability index (PSI)[22] suits categorical and binned features, and the Kolmogorov-Smirnov (KS) statistic of the CI/CD distribution screen suits continuous ones. Jensen-Shannon divergence gives a symmetric, bounded comparison of probability distributions, and the Wasserstein distance, also called earth mover's distance, measures the probability mass that must move, and how far, to turn one distribution into the other. Each needs an alert threshold, and sample size makes that choice harder than it looks.

Feature-by-feature comparisons can miss changes in how inputs occur together. Consider two binary sensor flags. In the training population, equal groups have both flags off or both on; in the serving population, equal groups have one flag on and the other off. Each flag is still equally often on and off, so its individual histogram is unchanged, but the relationship between flags has reversed. Even perfect training-serving feature parity would not detect this population change. Monitoring selected feature combinations, relevant cohorts, or a joint-distribution statistic complements marginal screens; labeled outcomes are still needed to determine whether the change harms predictions.

In high-throughput production monitoring, statistical significance alone is an inadequate alert policy. With large samples, tests such as the two-sample KS test or the chi-square test (the standard test for differences in categorical frequencies) can detect small genuine differences that have little operational significance. The mechanism is the sample-size term in the test's critical value. The KS critical value c_{crit} used in the data pipeline's drift test (section 4.5.3) scales with $\sqrt{(n_0+n_t)/(n_0 n_t)}$, so it shrinks roughly as $1/\sqrt{n}$, and a monitor that compares millions of serving requests against a large training sample rejects equality for shifts far smaller than the 0.1 gate of listing 14.2. These detections are not necessarily statistical false positives, but paging on each one creates unnecessary alerts. Conversely, small-sample edge telemetry may lack power to detect a meaningful change. Monitoring must consider effect size, sampling uncertainty, and the cost of missed change together.

Production monitoring therefore combines distribution-distance measures with uncertainty and operational thresholds. The population stability index that section 4.12.1 defined for the data pipeline, the binned relative entropy between expected (training) and actual (serving) proportions over n_{bins} fixed bins, is a common choice. KL divergence, defined there alongside it, is its asymmetric relative, and PSI equals the sum of the two directed KL divergences, which is why a shift in either

[21] **Schema validation**: The rules in listing 14.3 prevent silent data contract violations, such as a feature column changing from an integer to a float. Without this input-level guardrail, downstream model monitoring cannot distinguish a data quality error from a true performance regression, masking the root cause. A schema mismatch in a critical feature can invalidate an otherwise well-formed prediction batch.

[22] **PSI**: PSI is widely used in credit-risk scorecard monitoring to compare expected and observed binned populations; Yurdakul and Naranjo analyze its statistical properties (Yurdakul and Naranjo 2020). The common 0.1 and 0.25 bands are useful operational conventions, not universal statistical laws. ML operations adopted PSI because it works on binned categorical or continuous features and provides an interpretable drift score that non-specialists can review.

direction raises it. Operations adds what the definition leaves open, the response, by turning the conventional scorecard bands into an escalation policy. An illustrative policy has three bands:

- PSI < 0.1: Below this policy's warning threshold; continue monitoring outcomes and critical slices.
- $0.1 \leq$ PSI < 0.25: Warning range; investigate population changes and upstream pipeline behavior.
- PSI ≥ 0.25: Escalation range; notify the owning team under the service's calibrated severity policy. Diagnosis determines whether intervention is needed.

Production monitors require a documented zero-bin policy, such as Laplace smoothing, because any empty bin in the denominator yields an undefined logarithm, and they must preserve fixed bin boundaries across comparison periods. PSI is a drift signal, not an automatic retraining trigger, because divergence alone does not establish that accuracy has changed (principle 10). Every threshold, whether a PSI band or a KS or Jensen-Shannon cutoff, must be calibrated per feature against sample size and the cost of missed drift, because a shift that is plainly visible on a histogram can still produce a small aggregate score.

The ranking service monitors user age as an input feature, and a shift toward older cohorts produces the bin-by-bin PSI decomposition shown in table 14.13:

Table 14.13: PSI Worked Example: User age distribution drift from training to serving, decomposed across six age bins. The total PSI is 0.029, well below the 0.1 warning threshold, even though several bins shifted by 3 percentage points. Aggregate PSI summarizes movement across bins; action depends on a calibrated threshold, sample size, and business cost.

Age Bin	Training	Serving	Difference	ln(Serving/Training)	Contribution
18–25	15%	12%	-0.03	-0.223	0.0067
26–35	25%	22%	-0.03	-0.128	0.0038
36–45	20%	18%	-0.02	-0.105	0.0021
46–55	18%	20%	+0.02	+0.105	0.0021
56–65	12%	15%	+0.03	+0.223	0.0067
66+	10%	13%	+0.03	+0.262	0.0079

Summing the six bin contributions yields a total PSI of 0.029. The difference column is in proportion units, so a 3 percentage-point shift appears as ± 0.03. Even with such shifts in several bins, the aggregate drift is well below the 0.1 warning threshold. Operational action depends on a calibrated threshold, sample size, and business cost.

↪ KL divergence, its asymmetry, and a worked drift example that computes both KL divergence and PSI are collected for reference in section B.2.3.

Data freshness monitoring Feature stores and data pipelines can become stale without triggering obvious errors. Data freshness monitoring catches that failure mode, and listing 14.4 shows a configuration that monitors feature freshness and triggers fallback behavior when data becomes stale.

Listing 14.4: Data Freshness Alert Configuration: This configuration monitors the `user_purchase_history` feature for staleness, alerting operations teams via PagerDuty and Slack and falling back to default values when the feature exceeds the maximum allowed age.

```yaml
# Example freshness alert configuration
feature: user_purchase_history
max_staleness: 6h
alert_channels: [pagerduty, slack]
on_stale:
  action: fallback_to_default
  default_value: []
```

A freshness policy turns staleness from a silent data defect into an explicit fallback, and the same pairing of a detector with a response must cover every layer of the monitoring stack, including the upstream dependencies feeding the ML system: database replication lag, feature-store ingestion watermarks, and pipeline completion status. In one representative incident, a recommendation

service detected a material shift in its `user_lifetime_value` feature within two days, tracing the anomaly to a database migration that silently altered an aggregation window. Without input-layer distribution monitoring, such upstream corruption degrades model predictions for weeks before ground-truth outcome metrics expose the failure.

14.5.3 Monitoring cost model

Every layer of that stack costs money to run, and the first cost is the telemetry itself. Collecting these signals at production scale introduces severe network and storage constraints, because capturing uncompressed telemetry for every request can saturate the very networking fabric that serves the model. These overheads force a deliberate trade-off between monitoring granularity and infrastructure operating expense.

Napkin Math 14.4: The economics of observability

Trade-off: "Measure everything" is economically infeasible at scale. Let N_{series} be the expanded series count, f_{sample} the sampling frequency per series, s_{sample} the bytes per sample, and $T_{\text{retention}}$ the retention duration; $C_{\text{ingest/sample}}$ and $C_{\text{storage/byte-time}}$ are provider-specific ingestion and storage prices. Equation 14.5 combines these terms into an operating cost rate:

$$\text{Cost rate} \approx N_{\text{series}} f_{\text{sample}} \left(C_{\text{ingest/sample}} + s_{\text{sample}} T_{\text{retention}} C_{\text{storage/byte-time}} \right) \tag{14.5}$$

The product is an operating cost rate, not a one-time setup cost.

Both data volumes follow from the same two-step arithmetic. At full fidelity, every request's telemetry enters the stream: 1M req/s × 1 KB per request = 1 GB/s. Retaining one in 60 request traces reduces average ingest to 16.7 MB/s; batching sampled traces every 60 s changes delivery timing, not the average byte rate. Table 14.14 places the two regimes side by side with their cost impact.

Table 14.14: Trace Sampling vs. Observability Cost: Retaining one in 60 request traces yields a 60× data-volume difference at 1M req/s, assuming 1 KB per request. Batch intervals do not cause the reduction.

Sampling	Granularity	Data Volume (1M req/s)	Cost Impact
All traces	Micro-bursts	~1 GB/s	High (Requires dedicated cluster)
1-in-60 traces	Trends	~16.7 MB/s	Lower (Policy-dependent)

Systems insight: Sample by outcome rather than uniformly. The worked case keeps one trace in 60 regardless of outcome; a production policy keeps 100 percent of errors and only 1 percent of successful requests, because errors are rare and carry most of the diagnostic signal. Monitor aggregate counters every 1 s and high-cardinality sketches every 60 s. The cost components below turn these choices into a monthly budget for one node.

Sampling policy alone moves telemetry volume by 60× (notebook 14.4). A complete budget must also price storage, dashboard queries, and alert evaluation, and then weigh the total against the incidents that monitoring prevents.

Cost components

Monitoring costs break down into four categories, as equation 14.6 decomposes:

$$\text{Monitoring Cost} = C_{\text{ingest}} + C_{\text{storage}} + C_{\text{compute}} + C_{\text{alert}} \tag{14.6}$$

The four C_* terms are cost components over the same accounting window: data ingestion, retained storage, query or dashboard compute, and alert-rule evaluation. Separating them matters because each scales with a different control knob. Table 14.15 provides representative unit-cost assumptions for each component, and translating these unit costs into a concrete budget estimate clarifies the real expense of monitoring even a single production model:

Table 14.15: Monitoring Cost Components: Illustrative scenario unit costs used for the worked example. Costs scale differently across components: metric ingestion scales with cardinality (number of unique metric series), storage scales with retention, and query costs scale with dashboard usage patterns.

Component	Illustrative Unit Cost	Scaling Factor
Metric Ingestion	\$0.10–0.50 per million data points	Number of metrics × sample rate
Log Storage	\$0.50–2.00 per GB/month	Log verbosity × retention period
Query Compute	\$0.01–0.05 per query	Dashboard refresh rate × users
Alert Evaluation	\$0.001–0.01 per evaluation	Number of alert rules × check frequency



Napkin Math 14.5: Single-model monitoring budget

Problem: Calculate the monthly metric ingestion, time-series storage, and query compute cost required to monitor a single ML serving node under production assumptions.

Variables:

- one model with 3 deployment variants (production, canary, staging), each emitting 50 metrics
- Metrics sampled every 15 seconds
- Retention requirement: 30 days
- 2 dashboards (model health, infrastructure), 3 team members, five-minute refresh

Metric ingestion:

- Data points per month: $3 \times 50 \times (4 \text{ samples/min} \times 60 \times 24 \times 30 \text{ days}) = 25.9\text{M}$
- Cost at \$0.30/million: \$7.8/month

Storage:

- At 8 bytes/point compressed: 25.9M × 8 bytes = 0.2 GB
- Cost at \$1/GB: \$0.21/month

Query compute:

- Queries per month: 2 dashboards × 3 users × (12 queries/hour × 8 hours/day × 22 days) = 12,672 queries/month
- Cost at \$0.02/query: \$253.4/month

Total: ~\$261.4/month for a single ML node. Alert evaluation and platform overhead are excluded.

Systems insight: Under these assumptions, cost scales linearly with identical nodes; at platform scale, query-cost optimization becomes increasingly important.

Cost optimization strategies

Each cost component in equation 14.6 maps to a distinct engineering fix. The primary driver of C_{ingest} and C_{storage} is metric cardinality. High-cardinality labels such as `user_id` or `request_id` multiply the number of series in time-series indexes, and that growth outweighs raw compute costs. Dropping unnecessary labels or aggregating them into a bounded set of buckets reduces cardinality. Hashing each identifier to another distinct identifier alone does not reduce the number of series. The second driver of storage cost is temporal resolution. Retaining raw 15-second metrics across 30 days is rarely justified. A tiered retention policy (high-resolution telemetry for recent incident debugging, downsampled rollups for historical trend analysis) preserves diagnostic fidelity while reducing retained volume. For C_{compute}, dashboard query overhead accumulates through auto-refresh loops across dozens of unattended user sessions; setting conservative refresh intervals and auto-pausing inactive sessions bounds query load. Finally, C_{alert} scales with evaluation frequency. Consolidating

overlapping threshold checks into composite multi-condition rules can reduce redundant evaluation and notifications; thresholds, ownership, and actionability still determine alert fatigue.

Cost-benefit framework

Justify monitoring investments against incident costs using the monitoring benefit-cost ratio in equation 14.7:

$$\text{Monitoring Benefit/Cost} = \frac{\text{Incidents Prevented} \times \text{Avg Incident Cost}}{\text{Annual Monitoring Cost}} \tag{14.7}$$

Suppose an average incident costs $50,000 (downtime, engineering time, and reputation) and monitoring prevents 5 incidents annually at $50,000 in monitoring cost:

$$\text{Benefit/Cost} = \frac{5 \times \$50K}{\$50K} = 5\times$$

This framework helps justify monitoring investments and prioritize which metrics deserve fine-grained observation vs. coarse sampling. The monitoring systems themselves require resilience planning to prevent operational blind spots. When primary monitoring infrastructure fails (Prometheus experiencing downtime or Grafana becoming unavailable), teams risk operating blind during critical periods. Production-grade MLOps implementations therefore maintain redundant monitoring pathways: secondary metric collectors that activate during primary system failures, local logging that persists when centralized systems fail, and heartbeat checks that detect monitoring system outages.

Robust deployments also monitor the monitor. An external watchdog checks the telemetry pipelines themselves, so a heartbeat check from an independent system catches a dead pipeline and alerts through an alternative notification path. This reduces the risk of undetected simultaneous model and telemetry failures, provided the alternative paths do not share the failed dependency.

Checkpoint 14.3: The monitoring stack

ML monitoring has several layers. The prompts in this checklist test whether each symptom can be traced to the responsible layer.

- ☐ **Infrastructure utilization**: Explain why a GPU reporting high kernel-active time can still be memory-bound, and which measurements (achieved FLOP rate, memory traffic, stall counters) would confirm it before choosing a remedy.
- ☐ **Telemetry sampling**: Explain what retaining one request trace in 60 preserves for trend monitoring, and what short-lived anomalies it can miss.
- ☐ **Data freshness**: Detect stale feature ingestion pipelines before plausible predictions mask obsolete real-world states.
- ☐ **Data drift**: Evaluate PSI or KL divergence metrics to detect when serving inputs diverge from the training baseline.
- ☐ **Model accuracy**: Account for delayed ground-truth labels when interpreting real-time task performance alerts.
- ☐ **Cohort bias**: Apply stratified cohort analysis to detect subgroup degradation hidden by aggregate top-level accuracy.

Layered monitoring converts silent degradation into signals. Infrastructure telemetry shows whether the serving path is compute-, memory-, or I/O-bound, drift statistics show that inputs have moved, and delayed outcome metrics confirm whether the movement cost accuracy. A signal is not yet a response, however. The first decision a quality signal forces is whether to retrain, a choice that weighs the value the drift is costing against the cost of a run. Alerts that point to a fault rather than to drift go to incident response, which follows that decision.

14.6 Retraining Decisions

A team that retrains the ranking model on every minor PSI spike burns thousands of dollars in accelerator time and risks training on corrupted upstream logs, while a team that waits blindly for a quarterly calendar tick lets accuracy slide six points before anyone acts. Retraining is the most expensive response the operations loop can choose, and the easiest to automate badly. Monitoring supplies two kinds of evidence, a divergence that a label-free monitor measures continuously and outcome metrics that arrive later, and the retraining decision turns them into one of three actions: retrain, roll back, or wait. The decision has three parts, taken in turn below: the candidate strategies and a gate that tests whether a run is worth its cost now, a cadence that sets how often runs should happen, and a policy comparison between a fixed schedule and an evidence trigger. On the ranking service every run is also paid in accelerator-hours on hardware that serves live traffic.

The degradation equation of equation 1.1 turns divergence into a forecast of accuracy when outcome evidence supports one:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

where λ is fitted per deployment from historical degradation data, as the statistical drift diagnostic (principle 10) requires. While not a universal law, tracking distributional divergence $\mathcal{D}(P_t \| P_0)$ can provide an earlier warning than delayed outcomes; it does not establish the direction or size of a quality change.

14.6.1 Retraining decision framework

Because distributional divergence warns early without proving how much accuracy was lost, an automated training pipeline still leaves one operational decision open: when to run. Deciding when to retrain a model means balancing accuracy degradation against compute and validation expense across three candidate policies. Table 14.16 provides illustrative schedules across domains, from daily retraining for rapidly shifting ad click prediction to quarterly updates for stable medical imaging applications:

Table 14.16: Illustrative Retraining Schedules by Domain: These represent starting points; actual cadences depend on observed drift rates and business impact, and organizations typically calibrate them through operational experience.

Domain	Illustrative Schedule	Rationale
Ad click prediction	Daily	User interests shift rapidly
Fraud detection	Weekly	Attack patterns evolve continuously
Demand forecasting	Monthly	Seasonal patterns change slowly
Medical imaging	Quarterly	Disease presentations are stable

The ranking service sits between the table's first two rows. Its users' interests shift daily, yet the economics below put its optimum at a few days. Those schedules are starting points, not invariant rules. In a *scheduled retraining* policy, pipeline execution runs on a fixed calendar cadence (daily, weekly, or monthly) regardless of incoming performance telemetry. This strategy is simple to automate and guarantees that freshly accumulated training data is integrated periodically. However, it burns accelerator hours during quiet periods and reacts slowly when distribution shifts strike between scheduled intervals.

A *triggered retraining* policy initiates pipeline execution when monitoring systems detect statistical drift or performance degradation exceeding predefined thresholds. This approach can conserve compute by responding to evidence rather than a fixed schedule, but it relies heavily on low-latency label streams or accurate proxy metrics to prevent delayed responses or false-alarm retraining runs.

A *continuous retraining* policy updates model weights incrementally as labeled records arrive, using streaming micro-batches or online learning algorithms. While this strategy minimizes staleness lag, it amplifies validation risk, because corrupted, unrepresentative, or adversarial data can poison model parameters before automated anomaly detectors or engineering teams can intervene.

The operating choice among these strategies depends on four physical and operational constraints: retraining compute cost, validation pipeline latency, automated rollback capability, and label arrival

delay. Re-optimizing a model on multi-accelerator nodes incurs tangible energy and cluster costs; triggered policies require statistically reliable proxy signals when true labels arrive with long delays; and every continuous deployment pipeline demands robust canary and rollback mechanisms. The policy should follow from these costs and the label delay, not from habit.

The economic and validation constraints become executable in listing 14.5. The decision function schedules a run only when the estimated recovered value exceeds the total cost of a run (compute, validation, deployment, and the expected cost of a bad rollout, the four terms that equation 14.10 formalizes below), and only when fresh validation data are available.

Listing 14.5: Triggered Retraining Decision: The decision combines quality loss, feature drift, prediction shift, and the cost of a run so automatic retraining fires only when the expected benefit exceeds that total cost.

```
quality_loss = baseline_accuracy - current_accuracy
feature_drift = max(population_stability_index(features))
prediction_shift = distribution_distance(
    baseline_predictions, live_predictions
)

benefit = estimate_value_recovered(
    quality_loss=quality_loss,
    feature_drift=feature_drift,
    prediction_shift=prediction_shift,
)
total_cost = (
    compute_cost + validation_cost + deployment_cost + rollout_risk
)

if benefit > total_cost and validation_data_is_fresh():
    schedule_retraining_run()
```

The gate separates detecting drift from authorizing retraining. Drift contributes to estimated benefit, while fresh validation data and positive net value determine whether automation acts. Turning that gate into a cadence requires measuring how fast an unretrained model loses value.

14.6.2 Quantitative retraining economics

To turn the trigger gate of listing 14.5 into a concrete schedule, a team must measure how fast an unretrained model loses value and weigh that loss rate against the fixed cost of a run. When outcome data support a locally exponential fit, the fitted decay rate gives a measurable timescale.[23] Its **half-life**,[24] the time for fitted accuracy to fall to half its initial value, states that timescale in intuitive units, although no production model is left to lose half its accuracy before retraining. The economically optimal interval is set by the cost of staleness and the cost of a run, and in the scenario worked below it is far shorter than the half-life. The derivation takes three steps: a staleness cost, a retraining cost, and the interval that minimizes their sum per unit time.

The staleness cost function

Model accuracy can degrade over time, creating a staleness cost. For economic planning, a team may fit the observable impact over a stable local regime as an exponential decay process. Here γ is a temporal decay rate under the assumption that measured degradation accumulates steadily; it is not determined by distribution divergence alone. The exponential model is a simplification that enables closed-form economic analysis. Let $\text{Accuracy}(t)$ represent accuracy at time t since last training, and Accuracy_0 represent initial accuracy. Equation 14.8 captures this fitted scenario, where the rate γ depends on domain volatility and observed outcomes:

$$\text{Accuracy}(t) = \text{Accuracy}_0 \cdot e^{-\gamma t} \tag{14.8}$$

This temporal model is a time-domain summary of the degradation equation (equation 1.1), which predicts accuracy from divergence, $\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$. If divergence grows at a roughly steady rate over an interval, that equation makes accuracy fall linearly in time at rate λ

[23] **System entropy**: The fitted decay rate γ measures system entropy, the silent degradation of section 1.6, in which quality decays while code and weights stay unchanged. It can differ substantially across deployments and must be estimated from outcome data. A shorter decay timescale demands more responsive monitoring, but it does not by itself require continuous training; label delay, validation risk, retraining cost, and rollback capacity determine the operational cadence.

[24] **Half-life** (from nuclear physics, where it measures the time for half of a radioactive sample to decay): In ML operations, the metaphor is a convenient approximation, not a universal law. For the exponential fit in equation 14.8, derived below, the half-life is $\ln(2)/\gamma$. Retraining cadence also depends on traffic, value, retraining cost, and risk; seasonal, abrupt, or adversarial change requires a richer drift process.

times the growth rate of $\mathcal{D}(P_t \| P_0)$, while the exponential of equation 14.8 falls at rate $\text{Accuracy}_0 \gamma$ while γt is small. Matching the two slopes shows that γ is an empirical stand-in for how fast the input distribution moves, scaled by the fitted sensitivity λ and divided by Accuracy_0. The retraining interval derived here uses the time-domain γ, because a schedule is set in days; the drift statistics of section 14.5.2 (the population stability index, the KS statistic, and KL divergence) estimate the divergence itself and alert on it directly. Steady divergence growth is an assumption rather than a law, so both forms must be checked against observed outcomes.

A longer-horizon fit would decay toward a floor, such as chance-level accuracy, rather than toward zero. The unshifted form in equation 14.8 suffices for the intervals that retraining exists to enforce, the early, small-loss regime ($\gamma t \ll 1$) on which the square-root approximation below also depends.

The cost of staleness accumulates based on the daily query volume Q_{day} and the value impact V of a unit change in accuracy fraction. Integrating the instantaneous accuracy loss $(\text{Accuracy}_0 - \text{Accuracy}(t))$ over the retraining interval T yields equation 14.9:

$$\begin{aligned} \text{Staleness Cost}(T) = \int_0^T Q_{\text{day}} \cdot V \cdot (\text{Accuracy}_0 - \text{Accuracy}(t))\, dt = \\ Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \left(T - \frac{1 - e^{-\gamma T}}{\gamma} \right) \end{aligned} \tag{14.9}$$

The integral accumulates cost over time t from 0 to T, and the closed form follows from substituting equation 14.8 for $\text{Accuracy}(t)$.

The retraining cost function

Each retraining incurs fixed costs including compute, validation, and deployment overhead. Equation 14.10 decomposes these:

$$\text{Retraining Cost} = C_{\text{compute}} + C_{\text{validation}} + C_{\text{deployment}} + C_{\text{risk}} \tag{14.10}$$

where C_{compute} is the cost of the training run itself, $C_{\text{validation}}$ is the cost of evaluating the new model before release, $C_{\text{deployment}}$ is the cost of rolling it into production, and C_{risk} is the expected cost of potential regression from the new model. These are the four terms that `total_cost` sums in listing 14.5.

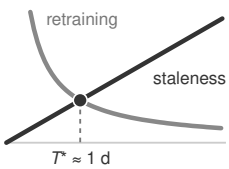
Optimal retraining interval

The optimal retraining interval T^* minimizes total cost per unit time, as equation 14.11 shows:

$$T^* = \arg\min_T \frac{\text{Staleness Cost}(T) + \text{Retraining Cost}}{T} \tag{14.11}$$

Under the assumption $\gamma T \ll 1$, a Taylor expansion of the exponential yields a closed-form approximation. Specifically, the second-order Taylor approximation $e^{-\gamma T} \approx 1 - \gamma T + \frac{1}{2}(\gamma T)^2$ reduces the bracketed staleness term $\left(T - \frac{1-e^{-\gamma T}}{\gamma}\right)$ to $\frac{1}{2}\gamma T^2$. Dividing total costs by T yields an average cost rate of $\frac{1}{2} Q_{\text{day}} V \text{Accuracy}_0 \gamma T + \frac{C}{T}$, where C is the Retraining Cost of equation 14.10. Differentiating with respect to T and setting the derivative to zero balances marginal staleness loss against amortized setup expense and yields the square-root economic interval of equation 14.12:

$$T^* \approx \sqrt{\frac{2 \cdot C}{Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}} \tag{14.12}$$



The best interval sits where amortized retraining meets accrued staleness.

An exact optimum requires minimizing equation 14.11 without the expansion. The interval grows with the cost of a run and shrinks with traffic, value, and decay rate, each under a square root.

Napkin Math 14.6: The optimal retraining interval

Problem: How often should the team retrain the model to maximize profit?

Physics: In this fitted scenario, model accuracy $\text{Accuracy}(t)$ decays exponentially at rate γ. Each successful update is assumed to restore initial accuracy with negligible training and rollout delay; traffic and value remain constant.

- Q_{day}: Daily Query Volume (Traffic).
- V: Financial value per query for a unit change in accuracy fraction. With this convention, $V = \$0.50$ means 1 percentage point of accuracy is worth $\$0.005$ per query. The Value per Point of equation 14.3 is the same quantity accumulated over a decision horizon, $V/100$ per query multiplied by the number of queries in that horizon.
- C: Fixed cost of a retraining run, including compute and operational overhead.

Formula: Equation 14.12 gives the interval T^* that minimizes the sum of staleness losses and training costs per unit time.

Math: Consider the ranking service in an illustrative scenario ($\text{Accuracy}_0 = 0.94$):

- **Traffic** (Q_{day}): 1,000,000 ranking requests/day.
- **Utility** (V): \$0.50/query for a unit accuracy change.
- **Retraining Cost** (C): \$5,000.
- **Drift Rate** (γ): 0.24 percent per day.

$$T^* \approx \sqrt{\frac{2 \times 5{,}000}{1{,}000{,}000 \times 0.50 \times 0.94 \times 0.0024}} \approx \textbf{3 days}$$

Systems insight: High traffic and valuable accuracy shorten T^*, so label availability, validation requirements, and achievable update delay become the binding limits. If T^* is less than the team's manual deployment time, the proposed cadence exceeds its operational capacity, and the team must shorten the update path or reconsider the policy and its assumptions.

With the parameters collected in table 14.17, the approximation places the economic optimum near 3 days. The ranking service's drift, a 0.24 percent daily decay fitted to its observed four-week slide, is slow enough that a daily run would not yet pay for itself at this run cost. Four times the traffic would bring the optimum to about 1.5 days, the daily cadence that table 14.16 lists for ad click prediction, while a slower-moving domain or a costlier run would push the same formula toward the table's weekly or monthly rows.

Table 14.17: Retraining Decision Parameters: Example values for the ranking service processing 1,000,000 requests daily.

Parameter	Value	Description
Q_{day}	1,000,000	Ranking requests per day
V	\$0.50/query	Value per query for a unit accuracy change
Accuracy_0	0.94	Initial accuracy
γ	0.0024	Daily decay rate (0.24% per day)
C	\$5,000	Total retraining expense

Sensitivity analysis

Under the square-root approximation, T^* scales with the square root of these parameters. Table 14.18 shows that a fourfold change in retraining cost, query volume, or decay rate moves the approximated interval twofold.

Table 14.18: Retraining Interval Sensitivity: Under the square-root approximation a fourfold change in an input moves the interval only twofold, and the directions differ. More expensive retraining lengthens the interval, while higher query volume or a faster decay rate shortens it.

Change	Effect on T^*
4× retraining cost	2× longer interval
4× query volume	2× shorter interval
4× decay rate	2× shorter interval

Accelerator-hours and contention

The run cost C bundles the four terms of equation 14.10, and on the ranking service the accelerator term is the smallest of them. In an illustrative scenario, a run that retrains on the logs gathered since the previous run occupies four H100s for six hours. Those twenty-four accelerator-hours cost $96 at the registry's training rate, about 1.9 percent of the $5,000 run. Validation, deployment, and rollout risk make up the rest, which is why automating the validation path (section 14.3.2.3) shortens the economic interval more than faster training would. The accelerator-hours matter instead as contention. The serving copy of the tables already holds two accelerators, so on an eight-accelerator node a run that holds its own copy and training state on four more leaves room for exactly one further copy of the tables while it executes (figure 14.8). That copy can be a canary, a shadow, or the hot standby of table 14.11, but not more than one of them, so a team that releases and retrains on the same node must schedule the two so that a retraining run never evicts the copy that makes rollback immediate.

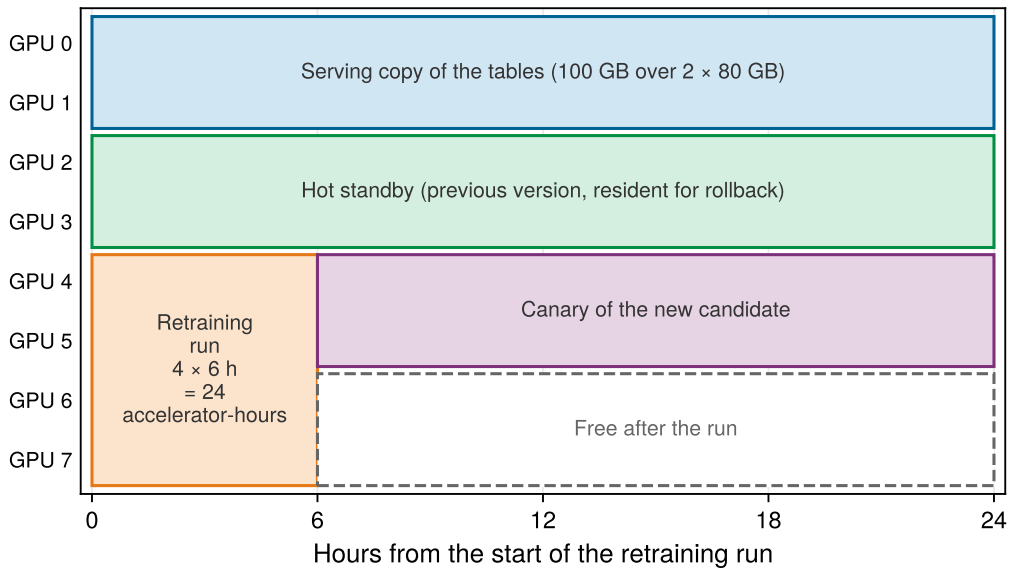


Figure 14.8: One Ranking Node's Day: Accelerator occupancy on an eight-accelerator H100 node over the day of a retraining run, under one schedule that keeps rollback immediate. The serving copy of the 100 GB tables holds 2 accelerators and the hot standby holds 2 more. The retraining run holds 4 accelerators for 6 h, so while it runs the standby is the only further copy of the tables that fits. After the run, the candidate it produces can go out as a canary on 2 of the freed accelerators. The schedule is illustrative and built from the chapter's worked numbers.

Model limitations

This closed-form framework provides a first-order approximation for operational planning, but its utility depends on several simplifying assumptions. First, the exponential decay formulation presumes gradual drift at a steady rate γ; sudden covariate shifts or acute data outages require immediate statistical change-point detection rather than smooth decay models. Second, the formulation assumes a constant marginal value V per accuracy point, whereas real-world utility often has

nonlinear thresholds or asymmetric penalties. Third, the model treats retraining events as independent resets to initial accuracy with negligible delay, setting aside update failures, time spent training and validating, and transfer learning across cycles. Finally, infrastructure costs are modeled as fixed sums, whereas cloud spot pricing, queue contention, and elastic compute availability introduce dynamic variations. Parameters improve through empirical calibration against historical telemetry as operational experience accumulates, and the framework implements cost-aware automation (section 14.1.1) with thresholds grounded in measurable business impact.

14.6.3 Scheduled versus triggered retraining

The interval derived in equation 14.12 fixes a calendar schedule from average drift, whereas the decision gate of section 14.6.1 lets observed evidence decide when to run. Over a 365-day deployment on the ranking service, the two policies trace different sawtooths against a drift threshold of 0.88 accuracy (figure 14.9). Under a scheduled policy (orange sawtooth), the model retrains at rigid 90-day intervals, repeatedly dipping below the quality floor before each reset. In contrast, a triggered policy (green sawtooth) initiates retraining the moment monitored accuracy touches the threshold, illustrating an idealized instantaneous reset; in practice the trigger is often a divergence or proxy signal calibrated against accuracy. Both curves are drawn from the temporal decay model of equation 14.8. In deployment, sample collection, label arrival, training, validation, and rollout delays can carry accuracy below the trigger before recovery; the policy does not guarantee a performance floor.

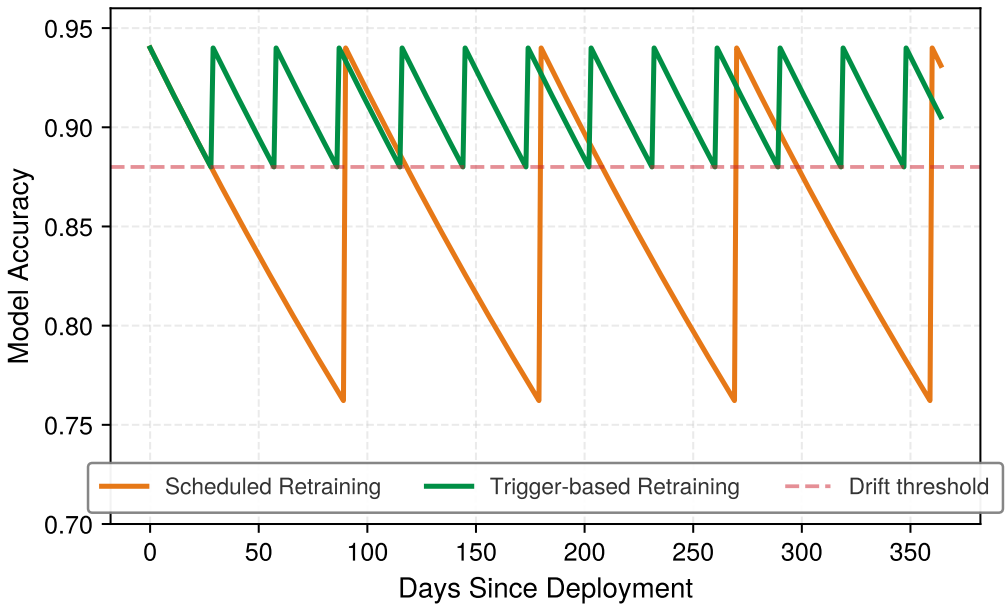


Figure 14.9: The Rotting Asset Curve: An illustrative exponential accuracy decay under two maintenance policies, both drawn as sawtooths. The scheduled policy resets every 90 days regardless of accuracy and dips below the drift threshold before each reset, while the triggered policy resets on crossing that threshold and holds a higher floor in this zero-delay model. Real response delays and update quality must be measured; neither cadence is universal.

In this zero-delay model the scheduled policy dips below the floor each cycle and the triggered one does not: scheduled retraining is easy to plan but can retrain too early or too late, whereas trigger-based retraining requires stronger telemetry but aligns intervention with observed degradation. Whichever policy fires, a retrained replacement or a rollback depends on the registry lineage of section 14.3.1.2, which links each version to the code, data, hyperparameters, and weights that produced it and so lets a responder return to a known-good checkpoint, even though lineage still cannot guarantee bitwise replay if driver versions, CUDA libraries, or nondeterministic reduction kernels vary across the fleet. Retraining answers gradual drift, whereas a sudden drop with a

cause inside the system, such as a broken feature transform, a bad release, or a corrupted upstream table, calls for a different response, because the failing layer must be found and contained, often by rollback, long before a retraining run could finish, and that is the work of incident response.

14.7 Incident Response and Governance

The ranking service from the opening of this chapter drew attention only in week eight, and a product manager noticed before any system did. Drift monitoring could have raised the alarm by week four, when accuracy had already fallen to 88 percent, but an alert only relocates the problem. Someone must then decide quickly whether the signal is real, which system caused it, and whether to roll back, and the organization must already have decided which models were allowed to ship and who owns them. Incident response supplies the first set of decisions, governance supplies the second, and the ML Test Score, taken up with operational maturity (section 14.8.1), audits whether both are in place before a failure tests them.

14.7.1 Incident response and operational practices

An unowned drift alert that fires on a Friday night without a severity policy, a diagnostic checklist, or rollback authority sits in a Slack channel until Monday while live traffic absorbs the damage. Incident response, debugging, and on-call rotation form the human half of the monitoring loop, turning a statistical signal into an engineering action within a bounded time.

14.7.1.1 Incident response for ML systems

At 2:00 AM, an on-call engineer receives an alert: the ranking service's click-through rate has dropped 12 percent over the past hour. There is no stack trace, no error log, no crashed process, just a statistical signal that something has changed. The responder must distinguish among four candidate root causes: an upstream data-pipeline failure, model drift, a seasonal traffic pattern, or statistical noise. This ambiguity is common in ML incidents, whose symptoms may appear as degradation in outcome or predictive-quality metrics rather than explicit errors, so incident response must account for statistical uncertainty.

Severity classification provides the foundation for prioritizing response in this ambiguous landscape. Table 14.19 gives one recommendation service's illustrative policy, with response deadlines from 15 minutes to 24 hours. Priority depends on affected traffic, consequences, and the need for containment. Click-through rate measures engagement rather than classification accuracy, and a population stability index value alone establishes neither quality loss nor incident severity.

Table 14.19: Incident Severity Classification for ML Systems: Illustrative response deadlines belong to this service's policy. Impact, evidence, and containment needs determine priority; drift magnitude alone does not.

Level	Criteria	Response Time	Example
P0	Service unavailable across live traffic	15 minutes	Null predictions prevent recommendation delivery
P1	Broad harmful impact needing containment	1 hour	CTR falls across major cohorts; replay confirms a broken feature contract, while longer-term outcomes remain pending
P2	Bounded impact with a validated fallback	4 hours	A localized feature defect is contained; the owner investigates remaining exposure and pending outcomes
P3	No observed impact or immediate exposure	24 hours	Training is delayed while the current release remains within its measured operating limits

Once an incident is classified, responders execute a structured response workflow where urgent containment can proceed alongside evidence collection:

1. Detection identifies which monitoring signal, affected metric, and threshold triggered the alert.
2. Impact assessment quantifies the fraction of serving traffic, user cohorts, or downstream services exposed.

3. Change correlation audits recent deployments across model weights, serving runtimes, feature transforms, and upstream data pipelines.
4. Mitigation evaluates and executes containment options, including traffic shifting, fallback enablement, or rollback to a known-good checkpoint.
5. Root cause analysis isolates whether the defect originates in data pipelines, model representations, or serving infrastructure.

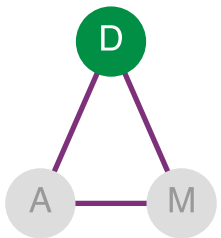
For P0 and P1 incidents, postmortem documentation is required. These postmortems analyze timelines, root causes, user impact, and preventive measures, specifically identifying which monitoring gap allowed the issue to reach production and what automated validation would have caught it earlier.

14.7.1.2 Model debugging: From detection to diagnosis

Incident response triages and mitigates; model debugging identifies root causes. Monitoring detects that something is wrong; debugging determines *why*. ML debugging must account for probabilistic and data-dependent failures in addition to conventional software defects. An incorrect prediction does not throw an exception or generate a stack trace, making systematic debugging essential for resolving ML incidents efficiently.

The debugging decision tree When model performance degrades, the following five stages provide a starting workflow; recent changes and incident severity can alter their priority. Section A.5.2 supplies a systematic diagnostic matrix that maps symptoms to D·A·M (Data · Algorithm · Machine) axes.

1. **Data pipeline integrity**: Inspect upstream data pipelines for schema changes, missing values, serialization errors, or input distribution shifts. Data integrity is an effective starting boundary because altered inputs, corrupted labels, and upstream feature changes can affect every downstream prediction.
2. **Training-serving skew**: Replay matched requests through the versioned offline and serving transformations and compare feature values, missingness, units, and time cutoffs. KS or PSI separately screen for population divergence; neither verifies that the two paths compute the same value for a given request.
3. **Subpopulation slicing**: Disaggregate performance across operational dimensions (such as geography, client hardware, or user cohorts). Localized degradation isolates data coverage gaps and slice-specific label noise that aggregate metrics conceal.
4. **Temporal degradation dynamics**: Evaluate performance trends across time. Abrupt drops prioritize recent release artifacts and upstream infrastructure outages, whereas gradual degradation motivates checking seasonality and changing predictive relationships.
5. **Model and representation attribution**: After verifying data pipeline integrity and ruling out serving skew, evaluate model-level failure modes through prediction confidence distributions, calibration curves, and feature attribution.



Production debugging starts on the data axis of D·A·M.

Slice analysis Slice analysis can guide pipeline and serving-skew investigations by testing whether the apparent system-wide problem is concentrated in a subpopulation. Performance metrics aggregated across all traffic can mask significant problems in subpopulations. **Slice analysis** exposes that masking by computing the same metric separately for each segment, and table 14.20 shows how a healthy overall accuracy can hide severe degradation in one of them.

Table 14.20: Slice Analysis Example: Overall accuracy of 91 percent appears acceptable, but tablet users (5 percent of traffic) experience 62 percent accuracy, a severe degradation masked by aggregation. Effective debugging requires systematic slice analysis across key dimensions.

User Segment	Traffic %	Accuracy	Impact
Desktop users	45%	94%	Nominal
Mobile (iOS)	30%	92%	Nominal
Mobile (Android)	20%	88%	Minor degradation

User Segment	Traffic %	Accuracy	Impact
Tablet users	5%	62%	Severe, investigate
Overall	**100%**	**91%**	**Masks tablet problem**

↪ Post-hoc feature attribution methods, Shapley value computational costs, and model interpretability boundaries are developed in section 15.3.3.1.

Feature attribution for debugging When slice analysis identifies a problematic segment, feature attribution techniques help identify which features the model relies on within its incorrect predictions. The common choice is SHAP values, feature-attribution scores that estimate how much each input feature contributed to an individual prediction, computed over the mispredicted examples of the failing slice and summarized across them. Their cost and their limits as explanations belong to responsible engineering; for debugging, one caveat governs their use. An attribution summary identifies features the model relied on within the failing slice; it does not establish whether those features caused the errors, so diagnosing stale features, missing coverage, or semantic shift requires checking feature lineage and production distributions. Attribution can also expose a zombie feature (perspective 14.1), an input the model should no longer depend on because nobody maintains it, whereas counterfactual analysis asks the complementary question about one decision at a time.

Counterfactual analysis For individual mispredictions, counterfactual analysis identifies the minimal change that would flip the prediction. If a churn prediction would flip to "engaged" when `user_session_length` rose from 12 minutes to 45, that sensitivity suggests checking for the definition mismatch illustrated in the skew case study (section 7). Matched-input comparisons and feature lineage must establish whether the churn system has the same pipeline defect before the counterfactual can guide a repair. Counterfactuals reveal which feature boundaries drive decisions and whether those boundaries make semantic sense, and counterfactuals that require implausible changes ("user age would need to be -5 years") often indicate feature engineering problems. Together, the debugging decision tree, slice analysis, feature attribution, and counterfactuals form a unified debugging toolkit that teams codify into a repeatable checklist.

Debugging checklist The diagnostic lenses identify places to investigate; a six-phase checklist organizes how to test a suspected cause. The ordering is deliberate because each phase narrows the search space for the next:

1. **Reproduce**: An ML failure that cannot be reproduced on held-out data is often data-dependent, an insight that redirects investigation toward the D·A·M taxonomy's data layer.
2. **Isolate**: Once reproduced, identifying the minimal input set that triggers the failure transforms a diffuse "the model is wrong" complaint into a specific, testable condition.
3. **Bisect**: If the failure correlates with a recent deployment, controlled comparisons of versioned model, feature-transform, and runtime artifacts help isolate which change introduced the regression.
4. **Attribute**: Feature attribution applies interpretability techniques to identify which input factors drive the erroneous behavior.
5. **Validate**: Confirming that the hypothesized root cause, when corrected, actually resolves the failure distinguishes genuine fixes from coincidental improvements.
6. **Prevent**: Converting each resolved incident into a monitoring rule or validation check systematically closes the gap between detection and recurrence. This cumulative hardening can reduce repeated failure modes over time because each incident strengthens the observability infrastructure.

The session-duration defect illustrates how these phases lead to a specific response. Suppose the CTR alert coincides with a serving-transform release on one of the ranking service's hosts. The model weights, runtime, and offline feature definition are unchanged. Request logs place the new transform on the affected sessions, and replaying those exact inputs reproduces the wall-clock versus foreground-active mismatch. Representative traces show unchanged feature-retrieval and model-execution durations, weakening a latency-regression explanation without ruling out every infrastructure fault. Mature recommendation outcomes for the new release are still pending, so the responder has established a feature-contract violation, not its eventual effect on accuracy or revenue.

The owning engineer restores the prior serving transform and its compatible feature-cache namespace, first checking the rollback on replayed requests and then exposing it through a bounded canary. Matched values agree again, and the same-input scores return to the prior artifact's outputs within the declared tolerance. Those observations demonstrate recovery of the feature contract; later comparable, mature outcome cohorts must establish whether user outcomes recover. A weights-only rollback leaves the changed feature computation in place, and immediate retraining would teach against the defect rather than repair the contract. The prevention step adds the affected idle-session inputs to parity validation and records the transform, cutoff, and cache versions needed for future replay.

14.7.1.3 On-call practices for ML systems

The debugging techniques in section 14.7.1.2 assume an engineer investigating during business hours. Production systems fail at 3:00 AM on weekends, and the person responding may not be the one who built the model. On-call practices keep someone with the right expertise available and equipped, and ML adds two difficulties to an ordinary on-call rotation. Its incidents are ambiguous. An engineer facing a 3 percent accuracy drop must decide whether it is statistical noise, a changed predictive relationship, or a failure that needs mitigation, which demands statistical context rather than log analysis. Its impact is also delayed. A ranking model that began serving slightly worse recommendations on Monday may not move revenue until Friday, by which time the window for easy diagnosis has closed, and the cause often lies in an upstream data system owned by another team.

These challenges motivate tiered escalation structures that match expertise to incident complexity. Table 14.21 illustrates one possible structure, where primary responders handle routine issues using standardized runbooks while escalation paths connect to specialists capable of deeper investigation. A parallel data on-call role can reduce time to resolution when the root cause lies in an upstream data system.

Table 14.21: ML On-Call Structure: Tiered escalation with parallel data on-call enables efficient incident response. Tier 1 handles routine issues using runbooks; Tier 2 addresses complex debugging; Tier 3 manages critical incidents requiring architectural decisions.

Tier	Responder	Responsibility
Tier 1 (Primary)	ML Engineer	Initial triage, standard runbooks, escalation decisions
Tier 2 (Escalation)	Senior ML Engineer/Data Scientist	Complex debugging, cross-system investigation, model-specific issues
Tier 3 (Critical)	ML Platform Lead	Architecture decisions, major incidents, vendor escalation
Data On-Call (Parallel)	Data Engineer	Data pipeline issues, feature store problems, upstream dependencies

Effective on-call depends on runbooks and on alert discipline. Each production model's runbook records its purpose, owner, and criticality; the latency, throughput, and accuracy ranges that define healthy behavior; past incidents and their resolutions; the diagnostic queries for recent predictions, feature distributions, and confidence scores; and the escalation and rollback criteria, including when to roll back without approval. Alert fatigue, the tendency to dismiss alerts after too many false positives, erodes all of it, so teams consolidate related alerts into one notification, use thresholds that follow weekly and seasonal patterns, retire alerts that are rarely actionable, and require a follow-up ticket before any alert is silenced. Structured shift handoffs carry active incidents, recent deployments, scheduled changes, and suppressed alerts across rotations, while limits on consecutive on-call days, compensatory time off after severe incidents, and automation that removes toil keep the rotation sustainable over years of operation. Even so, dashboards fail if no one is responsible for acting on alerts and runbooks languish if team structures do not support their use, which is why production ML operations need organizational infrastructure alongside the technical: clear governance, defined roles, and communication patterns that enable cross-functional coordination.

14.7.2 Governance and team coordination

On-call runbooks address real-time operational failures after a model is already serving traffic, whereas proactive governance sets the behavioral, legal, and ownership rules a candidate must satisfy before a data scientist hands a checkpoint to an infrastructure team for release. Without those pre-release checks, a model can ship into production with no signed feature contract, no subgroup fairness check, and no named owner, producing biased predictions or opaque decisions that neither team has the context to catch. Within an operational pipeline, governance enforces three concrete objectives: transparency through auditable training data lineage and verifiable model weights, fairness through evaluation against the application's selected and justified criteria, and compliance through adherence to privacy, retention, and auditability rules. While chapter 15 details the statistical metrics and interpretability algorithms required for these properties, MLOps provides infrastructure to collect evidence and enforce specified checks across the deployment lifecycle, alongside accountable policy and domain review.

ML governance spans the full lifecycle of the artifact, from automated pipelines that record feature schemas, training data provenance, and hyperparameter configurations during development, through prerelease validation gates that evaluate out-of-distribution robustness and selected fairness criteria, to production monitors that track changes in subgroup metrics alongside aggregate accuracy. Encoding testable governance policies in CI/CD pipelines makes release checks repeatable, while choices about acceptable risk, fairness criteria, and applicable obligations still require accountable review.

Concretely, a model registry promotion gate requires a signed feature contract, a recorded training-data lineage hash, subgroup metrics above policy thresholds, a canary SLO with automated rollback criteria, and a designated artifact owner before a model transitions from staging to production. That gate turns documented release requirements into assertions enforced by the deployment machinery. Passing those assertions establishes that the specified checks passed, not that every governance obligation has been satisfied.

Governance policies define the release criteria, but operational reliability depends on execution across organizational boundaries. Because machine learning systems span data engineering, statistical modeling, and distributed serving infrastructure, role handoffs represent the most failure-prone interfaces in the operational lifecycle. Versioned experiment stores and centralized model registries provide the shared substrate required for reproducible transitions between research and production engineering. At the data interface, formal schema specifications and lineage DAGs ensure that data scientists and platform engineers share identical assumptions regarding feature semantics, default null handling, and label arrival latency.

Titles and boundaries vary across organizations, but one common decomposition uses the five roles in table 14.22. The table maps each role to a primary responsibility without implying that every organization needs five separate positions:

Table 14.22: ML Team Roles Matrix: Clear role boundaries prevent gaps and overlaps. Data Scientists focus on model quality while ML Engineers handle productionization. Data Engineers own data pipelines while Platform Engineers own MLOps tooling. SREs ensure overall system reliability.

Role	Primary Focus	Key Deliverables	Collaboration Points
Data Scientist	Model development, experimentation, algorithm selection	Trained models, experiment results, performance benchmarks	Hands off to ML Engineer for productionization
ML Engineer	Production ML systems, training pipelines, serving infrastructure	Deployed models, training pipelines, serving systems	Receives from Data Scientist; works with Platform Engineer on infrastructure
Data Engineer	Data pipelines, feature engineering, data quality	Feature pipelines, data quality systems, feature stores	Provides data to Data Scientist; maintains feature store for ML Engineer
Platform Engineer	MLOps infrastructure, tooling, automation	CI/CD pipelines, monitoring systems, compute infrastructure	Enables ML Engineer; maintains shared infrastructure
DevOps/SRE	Reliability, incident response, system health	SLOs/SLAs, on-call procedures, runbooks	Supports all roles; owns production health

Clear role definitions matter most at handoff points, where artifacts transition between specialists. The most failure-prone transition occurs between Data Scientists and ML Engineers. An exploratory notebook script frequently fails in production environments due to undocumented data preprocessing steps, hardcoded local file paths, hidden state execution orders, or unpinned library dependencies. Similarly, the handoff from ML Engineers to SREs (Beyer et al. 2016) requires verified monitoring dashboards, active alerting rules, documented runbooks, and tested rollback procedures. Data Engineers interface with model builders through feature contracts, formal specifications of data schema, freshness SLOs, and distribution invariants that prevent upstream pipeline modifications from surfacing as silent downstream prediction failures weeks later. Teams eliminate these interface risks by requiring standardized container images, hermetic build manifests, and automated reproducibility assertions before any artifact advances to the next stage.

14.7.2.1 Stakeholder communication

While cross-functional handoffs govern internal engineering execution, stakeholder communication sets the shared expectations between technical teams and organizational decision-makers. Learned behavior adds dependence on the data and population, alongside latency-accuracy trade-offs and possible distribution drift. When stakeholders consider only service availability, misalignment can arise regarding expected predictive quality, latency budgets, and maintenance costs.

A frequent communication failure is the unconstrained request to "make the model more accurate." Model accuracy cannot be scaled independently of system costs. Boosting validation accuracy by two percentage points often requires moving to a significantly larger model architecture, incurring higher FLOP counts per sample, increasing memory bandwidth demands, and violating strict inference latency SLOs. Grounding improvement discussions in the Pareto frontier (principle 5) forces explicit decisions among model accuracy, inference latency budgets, training compute costs, and dataset acquisition expenses.

Translating statistical metrics into operational impact requires mapping errors to their consequences, which are rarely symmetric. Accuracy, ROC-AUC, and F1 treat errors symmetrically. The ranking service's errors differ in cost only mildly, so a separate illustrative case with a sharper asymmetry carries the argument. In an automated fraud detection pipeline, a false positive creates friction at checkout while a false negative allows unrecoverable theft. With missed fraud ($500 per incident) five times as costly as a blocked legitimate user ($100), the total cost across classification thresholds reaches its minimum near 0.34, well left of the symmetric 0.50 baseline (figure 14.10).

When monitoring detects distribution drift or an inference regression triggers an automated rollback, the status report must state the evidence directly: the statistical divergence metric, the demographic or tenant cohort affected, the canary rollback target, and the estimated recovery timeline. Dismissing an anomalous output distribution as noise without a hypothesis test lets a silent failure reach production decisions.

In the same fraud detection case, consider a team evaluating a proposed model upgrade. When stakeholders request improved capture rates, the engineering team frames the initiative as a concrete systems trade-off: increasing the fraction of fraud dollars captured from 92 percent to 94 percent requires integrating external feature pipelines, extending training duration by two weeks, and accepting 30 percent higher infrastructure costs. In return, the upgrade prevents an estimated $2 million in annual fraud losses while a separate assumption of a 20 percent cut in false-positive alerts removes 50,000 of them per month. Framing model improvements around financial impact, latency budgets, and compute expenditure keeps governance decisions tied to verifiable engineering trade-offs.

Checkpoint 14.4: From alert to accountable action

- ☐ **Diagnosis**: A cohort's accuracy drops while its feature distribution shifts. What additional evidence would distinguish a feature-pipeline defect from a changed predictive relationship?
- ☐ **Release authority**: Name an automated promotion check and a policy decision that its passing result cannot make on the team's behalf.

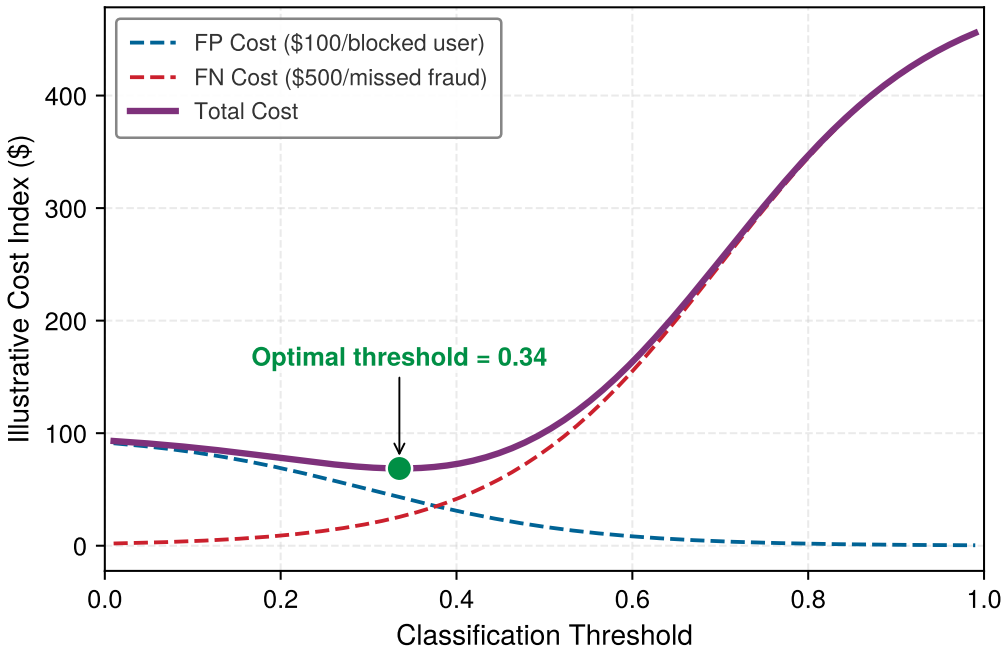


Figure 14.10: The Business Cost Curve: An equal-class-weighted illustrative cost index vs. classification threshold. Technical metrics like ROC curves hide the economic reality: errors have different costs. Here, a false negative (missed fraud) costs $500, while a false positive (blocked user) costs $100. The system flags a transaction as fraud when its score exceeds the threshold, so a lower threshold flags more transactions. Because missing fraud is $5\times$ more costly, the optimal threshold shifts left of center, making the system more aggressive at flagging suspicious transactions and accepting more false positives to minimize expensive misses. With equal costs the optimum would sit at threshold = 0.50; the asymmetry pulls it toward threshold ≈ 0.34. A population expected cost would also weight each term by class prevalence. MLOps tunes thresholds as costs and prevalence change.

14.8 Operational Maturity

An organization deploying its initial ML model might rely on a hand-run Jupyter notebook, a scheduled cron job, and minimal monitoring. A mature enterprise runs thousands of models through automated pipelines with drift detection, canary deployments, and continuous validation. Both are doing "MLOps," yet they differ greatly in reliability, cost efficiency, and deployment velocity. Empirical surveys of production deployments reveal that systems failures cluster around data dependencies, pipeline boundaries, and monitoring blind spots rather than core model architectures (Paleyes et al. 2022). Operational maturity provides the systems framework for evaluating this progression. Organizations evolve from ad hoc experimentation toward fully automated operations, and locating a system on this continuum guides resource allocation better than adopting tooling indiscriminately. Placing a system on the continuum starts with an audit of its current practices and then raises three questions in turn: what architecture the next level requires, which organizational and failure-containment patterns keep that architecture working, and how much the move is worth.

14.8.1 ML test score

A team can spend months perfecting offline hyperparameter sweeps and unit tests for its model code while leaving production monitoring and schema checks completely unbuilt, only to suffer a multi-week silent outage on the first upstream database change. Auditing the debt patterns of table 14.6 and the controls built to pay them down requires a formal technical debt assessment rubric that turns subjective readiness debates into measurable engineering gates. The ML Test Score (Breck et al. 2017) structures this evaluation across four operational categories: data tests, model tests, infrastructure tests, and monitoring tests. The rubric specifies 28 tests, seven per category. A documented manual test whose results are distributed receives 0.5 points; an automated test run

repeatedly receives 1 point, and an unperformed test receives 0. The composite score is the weakest category score, not an average:

$$\text{ML Test Score} = \min(\text{Score}_{\text{data}}, \text{Score}_{\text{model}}, \text{Score}_{\text{infra}}, \text{Score}_{\text{mon}})$$

where each category score $\text{Score}_{\text{cat}} \in [0, 7]$. A deployment with exhaustive offline model validation ($\text{Score}_{\text{model}} = 7$) but absent production telemetry ($\text{Score}_{\text{mon}} = 0$) receives an overall score of 0. The min operator is deliberate, because an unmonitored serving path can deliver corrupted predictions despite strong offline benchmark accuracy.

Three of the four suites partition production risk along the D·A·M axes, and the fourth watches all three. The data suite (D) verifies that feature distributions conform to declared schemas, privacy boundaries remain unviolated, and each feature justifies its inference latency and memory footprint. The model suite (A) confirms that architectures and hyperparameter configurations are version-controlled, staleness bounds are enforced, and offline benchmark improvements translate into live metric gains. The infrastructure suite (M) tests training reproducibility under documented execution conditions, serving-path parity between training and inference pipelines, and automated rollback mechanisms. The monitoring suite is the feedback layer across all three axes; it verifies that schema changes, invariant violations, training-serving skew, and stale weights generate actionable alerts that help limit downstream impact. Table 14.23 summarizes representative tests across each category:

Table 14.23: ML Test Score Checklist: Representative production-readiness tests from the ML Test Score rubric. The original rubric contains 28 tests grouped into four sections of seven tests each; section scores expose whether production risk comes from data validation, model validation, infrastructure, or monitoring rather than hiding the weakness in a single total. Based on Breck et al. (2017).

Category	Test	Implementation Example
Data Tests	Feature expectations are captured in schema	Great Expectations, TFX Data Validation
	All features are beneficial (no unused features)	Feature importance analysis, ablation studies
	No feature's cost exceeds its benefit	Latency/accuracy trade-off analysis
	Data pipeline has appropriate privacy controls	PII detection, access logging
Model Tests	Model spec is reviewed and checked into version control	Git-tracked model configs
	Offline and online metrics are correlated	A/B test validation of offline improvements
	All hyperparameters are tuned	Automated HPO with tracked results
	Model staleness is measured and bounded	Performance decay monitoring
Infrastructure Tests	Training is reproducible	Fixed seeds, versioned data, locked dependencies
	Model can be rolled back to previous version	Model registry with versioning
	Training and serving code paths are tested for consistency	Feature store integration tests
	Model quality is validated before serving	Automated validation gates in CI/CD
Monitoring Tests	Dependency changes result in alerts	Data schema monitoring
	Data invariants hold in training and serving	Distribution comparison tests
	Training and serving features are not skewed	Training-serving skew detection
	Model staleness is monitored	Alerts and evidence-based retraining review

Because the composite score is bounded by the minimum category, reliability audits must prioritize the bottleneck subsystem limiting the system score. Expanding a model test suite from 5 to 6 points while monitoring remains at 1 leaves the composite score unchanged, but can still improve actual coverage. The score highlights the weakest category without quantifying every reliability gain. Regular audits against this rubric reveal whether reliability risks stem from unvalidated input pipelines, offline-online distribution divergence, or uninstrumented serving paths, and so direct refactoring effort toward the debt that most threatens production. Each low category score also names the infrastructure that pays the debt down. A weak data score calls for schema contracts and shared feature definitions, a weak infrastructure score for versioned and reproducible pipelines, and a weak monitoring score for production telemetry.

14.8.2 Maturity levels

While the ML Test Score audits individual verification gates, **operational maturity** measures how cleanly those practices fit together across data pipelines, model training, and serving hardware

(Zaharia et al. 2018). Adopting an experiment tracker such as MLflow while leaving feature extraction in ad hoc notebooks and model rollout to manual file copies raises isolated test counts without reducing production outages. Although operational maturity exists on a continuum, grouping systems into three distinct maturity levels clarifies how workflows evolve from exploratory prototypes into reliable production infrastructure.

At the lowest level, workflows are ad hoc: training runs manually on local workstations, feature extraction is embedded in ephemeral scripts, and deployment relies on manual file transfers. As maturity increases, workflows become repeatable: teams introduce version-controlled pipelines, centralized artifact registries, and scheduled batch jobs. At the highest level, systems become scalable: declarative infrastructure-as-code, continuous validation pipelines, and real-time observability decouple model iteration from production serving.

Table 14.24 summarizes this architectural progression from disconnected execution toward unified lifecycle management.

Table 14.24: Maturity Progression: Machine learning operational practices evolve from manual, fragile workflows toward fully integrated, automated systems, impacting reproducibility and scalability. Key characteristics and outcomes at each maturity level emphasize architectural cohesion and lifecycle integration for building maintainable learning systems.

Maturity Level	System Characteristics	Typical Outcomes
Ad Hoc	Manual data processing, local training, no version control, unclear ownership	Fragile workflows, difficult to reproduce or debug
Repeatable	Automated training pipelines, basic CI/CD, centralized model storage, some monitoring	Improved reproducibility, limited scalability
Scalable	Fully automated workflows, integrated observability, infrastructure-as-code, governance	High reliability, rapid iteration, production-grade ML

Consider how the ranking service evolves across these maturity levels:

- **Ad hoc**: A data scientist trains a model in a Jupyter notebook, exports it as a pickle file, and hands it to an engineer who deploys it to a single server. When accuracy drops, the data scientist retrains manually by running the notebook again with fresh data. Debugging requires the original data scientist because no one else understands the preprocessing steps.
- **Repeatable**: The training script is version-controlled, with a scheduled Jenkins job that retrains monthly. Features are computed in a SQL script that engineering maintains separately. The model is deployed via container, with basic accuracy monitoring. When the feature SQL changes, the data scientist must manually verify the model still works.
- **Scalable**: Training and serving use the same feature store, reducing skew. A CI/CD pipeline investigates drift when PSI exceeds 0.25, retrains when evidence supports it, validates the new model against the baseline, and deploys via canary release. Monitoring tracks per-cohort click-through rate, triggering investigation when specific segments degrade. The entire lineage from raw data to production prediction is auditable.

Moving between maturity levels takes substantial engineering investment, and its return is a lower mean time to recovery (MTTR) and fewer silent failures, which matters most for mission-critical services. Evaluating an organization against this spectrum isolates specific architectural bottlenecks (such as manual data transfers or unversioned preprocessing logic) before they compound into production outages.

14.8.3 System design implications

The repeatable ranking service above already automates training and deployment, yet a change to its separately maintained feature SQL still requires manual validation. More frequent retraining would increase the manual validation burden without removing the mismatch risk. A versioned feature contract and matched-request parity tests address the failure boundary first; a shared feature store becomes useful when maintaining that contract across pipelines warrants the additional infrastructure. Maturity assessment therefore selects the next missing control for each system.

Figure 14.11 provides a coverage map for that assessment. Service uptime is visible at the tip, while data- and model-health failures can remain hidden below the waterline. For each failure mode, the assessment must identify the evidence that exposes it and the action that contains it.

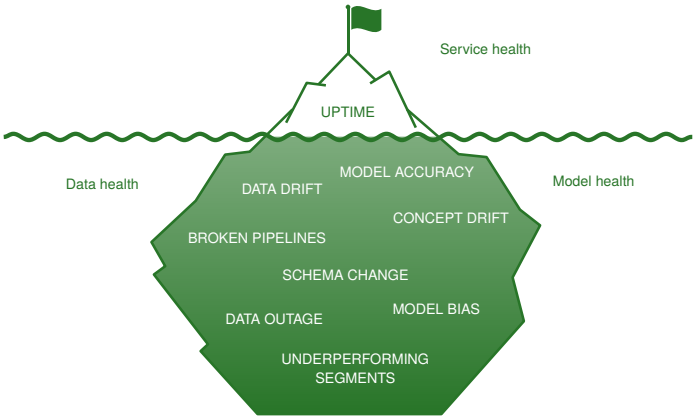


Figure 14.11: Uptime Dependency Stack: The waterline separates visible service uptime from data- and model-health failures that can remain hidden while the service stays available; the surrounding labels organize the monitoring surface into data, model, and service health.

A system that logs all three health domains can still lack a response path. If stale features are detected but no validated fallback exists, containment is the next investment. If quality outcomes are recorded without the served model version, attribution must be repaired before automatic promotion can rely on them. If a canary can exhaust the production worker's memory, resource isolation takes priority over another dashboard. Each finding identifies a specific capability to add and a failure against which to test it.

14.8.4 Design patterns and anti-patterns

Buying a commercial feature store or drift dashboard before deciding who is allowed to trip a fallback or isolate a misbehaving canary leaves the on-call team watching a failure they have no mechanism or authority to contain. Once the maturity assessment names a missing control, containment depends on an owner with the authority to change the affected system. A centralized MLOps team can maintain shared controls across models, while a federated model places engineers within product teams under shared architectural standards. Either arrangement must resolve the same handoff: who validates a fallback, who can activate it, and who confirms recovery. Buying a feature store or alerting platform before settling that handoff is the tool-first anti-pattern.

Once ownership is explicit, the residual failure determines the technical protection. Broken inputs require a way to stop harmful predictions; competing model versions require resource isolation. The cost of these controls is justified by the exposure they contain, and their effectiveness must be tested against that specific failure.

A circuit breaker automatically routes traffic away from a failing service when its error rate exceeds a threshold, and in ML serving it needs ML-specific trip conditions. Conventional circuit breakers trip when RPC timeouts or HTTP 5xx error rates exceed a set threshold. In an ML serving context, circuit breakers must also monitor invariants whose violation signals broken inputs rather than gradual drift: sudden shifts in feature bounds, missing-feature rates, or output distribution collapse. Gradual drift scores such as PSI open a diagnosis instead of tripping the breaker. When triggered, the circuit breaker bypasses the model to serve predictions from a deterministic heuristic fallback, a cached lookup table, or a simpler baseline model, preventing corrupted inferences from cascading into downstream transactional systems.

Similarly, bulkhead patterns[25] isolate candidate or canary models from primary serving paths. For the ranking service, the bulkhead must reserve a canary's full copy of the tables, 100 GB across two accelerators (table 14.11), rather than a slice proportional to its traffic share. On multi-tenant accelerator hardware, an unconstrained experimental model suffering an out-of-memory abort,

[25] **Bulkhead pattern**: This pattern partitions system resources to contain failures within isolated zones. In ML serving, a bulkhead allocates fixed accelerator memory, host RAM, and worker thread pools to experimental model versions, so a canary's resource exhaustion is less likely to terminate the primary serving path; the protection depends on the process and hardware isolation actually enforced.

memory leak, or thread-pool exhaustion can crash the shared serving runtime or stall the PCIe bus. Containerized memory limits, isolated worker queues, and supported hardware partitions can contain specific resource failures. Shared memory bandwidth, interconnects, and device-level faults still require explicit isolation analysis and testing.

Redundancy has a hard limit in ML serving. Replicating a model across workers or voting across ensemble members protects against crash faults, and agreement protocols can keep replicas consistent with one another, but agreement is not correctness. When every replica shares the same inductive biases or training-distribution blind spots, the replicas fail together and silently, and no amount of voting reveals the error before delayed ground truth arrives. Circuit breakers, bulkheads, and replication therefore strengthen an MLOps implementation only when each is applied to the failure class it can actually detect.

These patterns, like every operational practice in this chapter, must also adapt to physical, regulatory, and organizational constraints. A standard CI/CD pipeline may be infeasible without direct host access, monitoring may require indirect signals or on-device anomaly detection, and privacy regulations may limit data collection; these adaptations apply the same principles under constraint. Organizations operating many ML nodes consolidate into platform architectures with shared infrastructure and centralized governance, which raises challenges beyond single-node operations (cross-model resource allocation, system-level observability, and fault tolerance across interdependent pipelines). Sound single-model practices remain prerequisites, because gaps in monitoring, testing, or deployment multiply across the model portfolio.

14.8.5 MLOps investment economics

Spending $100K a year on an enterprise feature store and automated retraining pipeline for an internal analytics model that drives $20K in decisions is an engineering failure of its own, just as starving a $50M ranking engine of basic drift checks invites a six-figure outage. MLOps pays off when the investment matches the model's production value. This chapter has already put a number on each control separately. Training-serving skew cost a fixed fraction of daily queries (equation 14.2); automation paid back once manual hours overtook its build cost (notebook 14.1); retraining was worth a run only when recovered accuracy outweighed run cost and rollout risk (equation 14.3), with equation 14.12 setting its cadence; detection delay measured the cost of the gap between monthly and daily review (notebook 14.3); and monitoring earned its keep through the incidents it prevented (equation 14.7). An investment decision combines these separate estimates within a single accounting boundary rather than naively summing them. Because an unmitigated skew event manifests simultaneously as lost prediction value, an extended silent failure, and a production incident, adding those estimates directly would count the same underlying damage multiple times. Rigorous accounting establishes a common baseline and evaluation horizon, counts each prevented loss exactly once within a primary category, and separates routine deployment labor savings from emergency response hours already captured in incident costs. For a single ML node, the resulting accounting determines whether deployment speed, incident reduction, and monitoring coverage justify the operational spend; for a portfolio, the same discipline scales to justify platform investment.

14.8.5.1 Single-model MLOps investment

For a single production ML system, the first threshold is the annual cost of making the node observable, deployable, and recoverable. Table 14.25 summarizes the main cost categories:

Table 14.25: Single-Model MLOps Investment: Illustrative planning inputs for putting one production ML system under operational control. Open-source tooling (MLflow, Feast) can reduce software costs; cloud-managed services trade higher unit costs for reduced engineering overhead.

Component	Illustrative Assumption	Justification
CI/CD pipeline setup	$10–30K one-time	Reduces deployment time from days to hours
Monitoring and alerting	$2–10K/year	Shortens detection and response delays
Feature store (basic)	$5–20K/year	Reduces one source of training-serving skew
Model registry	$0–5K/year	Enables rollback, audit trails

Component	Illustrative Assumption	Justification
Engineering time	1–2 FTE-months setup	Initial automation and integration

14.8.5.2 Single-model ROI calculation

The return threshold then depends on model criticality. A revenue-facing model can justify more operational spend because avoided incidents and deployment-time savings have measurable value. Equation 14.13 formalizes that single-node calculation by extending the monitoring benefit-cost ratio of equation 14.7 with a second benefit term, the engineering labor that automation recovers:

$$\text{Annual Benefit/Cost} = \frac{\text{Incidents Avoided} \times \text{Avg Incident Cost} + \text{Time Savings} \times \text{Hourly Cost}}{\text{Annual MLOps Investment}} \tag{14.13}$$

where Incidents Avoided is the count of production failures the tooling prevents per year and Avg Incident Cost is the total loss per failure, so their product is the value of avoided incidents; Time Savings is the engineer-hours that automation reclaims per year from routine release and maintenance tasks (strictly distinct from emergency triage labor already subsumed under Avg Incident Cost), and Hourly Cost is the loaded labor rate, so their product is the value of recovered deployment labor; the denominator is the annualized investment, including recurring tooling and engineering costs plus setup costs amortized over the chosen planning horizon. The worked investment below applies this non-overlapping accounting boundary. The ratio expresses benefit per dollar invested, not net profit per dollar.

Consider a revenue-facing model with the following annual figures:

- 4 incidents/year avoided (at \$25K each) = \$100K saved
- 20 hours/month deployment time saved (at \$150/hr) = \$36K saved
- MLOps investment of \$30K/year

$$\text{Benefit/Cost} = \frac{\$100K + \$36K}{\$30K} = 4.53\times$$

14.8.5.3 When to invest more

The 4.53× benefit-cost ratio holds because the model is expensive enough that preventing incidents and shortening deployments outweigh the annual platform spend. The returns from single-model MLOps practices compound when teams add additional models. The transition from operating several independent ML nodes to building a centralized platform involves different economics entirely, including shared infrastructure amortization, platform team overhead, and cross-model coordination costs.

For single-model operations, invest in MLOps in proportion to model criticality. A model driving \$10M in annual revenue justifies more operational rigor than an internal analytics model. Monitoring, lineage, and repeatable deployment are often the first investments. Add a feature store when shared feature definitions and online-offline parity become material problems, and automate retraining only when evidence supports the trigger and the validation loop can contain a bad update.

The choice of operational controls also depends on what a deployment can measure and change. The case studies in section 14.9 follow the keyword spotter's edge deployment and a proposed clinical framework. Battery limits and clinical authority reshape how each system preserves evidence, validates updates, and responds to failure.

Checkpoint 14.5: Choosing the next investment

- ☐ **Accounting boundary**: Identify the setup, recurring tooling, and engineering costs needed to interpret an annual benefit-cost ratio.
- ☐ **Readiness score**: Explain how adding a useful model test can improve coverage while leaving the minimum-category ML Test Score unchanged.

- ☐ **Priority**: For a single model with unowned alerts and a tested deployment pipeline, justify the next operational investment before selecting additional tools.

14.9 Case Studies

A cloud MLOps playbook fails immediately if applied unmodified to a battery-powered voice trigger that cannot stream audio off the device, or to a clinical decision-support loop where an unreviewed weight update can harm a patient. Operational constraints in machine learning systems are governed by the physical envelope of the serving hardware and the failure consequences of the application. The chapter's keyword spotter operates under a milliwatt-scale power budget, kilobytes of on-device memory, and telemetry that must leave the device as aggregates rather than audio. In contrast, regulated AI-enabled medical devices require evidence and change controls appropriate to their intended use. FDA guidance for predetermined change control plans describes planned modifications, validation methods, and impact assessment (U.S. Food and Drug Administration 2025). These contrasting environments dictate how engineering teams manage data pipelines, model artifacts, and hardware execution across the lifecycle. Edge architectures devote most engineering effort to bridging the hardware gap between high-throughput cloud training clusters and microcontrollers, whereas clinical systems devote it to provenance tracking, cohort monitoring, and human-in-the-loop validation gates.

The edge case in section 14.9.1 follows the keyword spotter from its detection drop through deployment and monitoring, reusing the three-tier partition that a published wearable design makes concrete. The clinical case, ClinAIOps, places clinicians inside the operations loop. Despite their contrasting operating regimes, both deployments must satisfy the same core MLOps disciplines. Table 14.26 maps these foundational principles across the two environments, demonstrating how physical hardware limits and regulatory mandates alter implementation mechanisms without altering the underlying operational requirements.

Table 14.26: MLOps Principles by Case Study: Side-by-side mapping of the five foundational MLOps principles to the keyword-spotting edge design and the ClinAIOps framework, showing how domain constraints reshape implementation without changing which principles apply.

Principle	Keyword-spotting edge design	ClinAIOps
Reproducibility	Versioned audio datasets, DSP front end, and firmware alongside model weights	Audit trails, decision provenance
Separation of concerns	Independent data, training, and serving layers with an edge-specific OTA pipeline	Distinct clinical validation and deployment stages with regulatory compliance isolation
Consistency	One feature-extraction kernel shared by training and on-device inference	Standardized clinical data pipelines ensuring training-serving parity
Observable degradation	Aggregate on-device proxies (wake rate, false-wake reports, duty cycle)	Cohort-specific monitoring, outcome tracking
Cost-aware automation	Updates gated on accuracy gain against radio energy, flash wear, and rollout risk	Automated model updates with human-in-the-loop gates balancing update cost and patient risk

14.9.1 Keyword spotting at the edge

Six months after launch, the keyword spotter's detection rate had fallen four percentage points below target because of three silent changes in section 4.12: a new microphone revision with a different frequency response, evaluation labels revised under new guidelines, and an unannounced change to the device metadata schema. Operations inherits that failure on the chapter's power-bound case. The fix must reach devices that no one can touch, the evidence must be gathered without uploading audio, and a bad update must be reversible without disabling the device.

14.9.1.1 Context and motivation

Keyword spotting partitions its computation across three tiers. The microcontroller runs the audio front end and the 200 KB INT8 DS-CNN continuously in its always-on island, a companion device or hub runs a heavier verification model when the detector fires, and the cloud aggregates fleet

statistics and retrains. The partition is not peculiar to keyword spotting. A published sleep-staging wearable, the Oura Ring, uses the same three tiers, and its published evaluation makes the work of each tier concrete (Altini and Kinnunen 2021). The ring captures accelerometry, optical photoplethysmography (PPG)[26] for pulse rate and heart-rate variability, and skin temperature, and its 15–22 mAh battery rules out continuous Bluetooth Low Energy (BLE) streaming. Sensor preprocessing and lightweight activity classification run in the ring's microcontroller firmware, the heavier multi-sensor sleep-stage sequence model runs on the companion phone during periodic synchronization, and historical aggregation and retraining run in the cloud. The versioning, telemetry, over-the-air, and lifecycle controls described later are a design for the keyword spotter informed by that partition, not a reported account of any vendor's production system.

14.9.1.2 Model development and evaluation

The wearable study also shows how an edge model is evaluated against a reference standard, the step on which the keyword spotter's monitoring depends. The ring's reference labels come from polysomnography (PSG),[27] the laboratory-grade multichannel physiological recording used as the clinical reference for sleep staging. Accelerometer-only models achieved 57 percent four-stage sleep classification accuracy because movement alone cannot separate quiet wakefulness from light or REM sleep. Adding heart rate, heart-rate variability, and circadian temperature features lifted accuracy to 79 percent. The resulting 22 percentage-point gain closes roughly 84.6–88 percent of the gap between the baseline and human PSG scorers, who agree with each other at only about 82 percent to 83 percent. The evaluation paired the ring with simultaneous laboratory PSG for 106 participants and 440 nights of data (Altini and Kinnunen 2021); across 5-fold cross-validation, each added sensing modality raised four-stage accuracy, which is the evidence that justified the ring's heavier phone-tier model.

The keyword spotter faces the same evaluation problem with a weaker reference. Its labels come from reviewed recordings, and because the device uploads audio only when it wakes, a spoken wake word that the detector misses leaves no recording at all (section 4.12.1). Measuring the false rejections behind the four-point drop therefore takes a separate, consented collection, and the evaluation set must be versioned by microphone revision so that a hardware change appears as a slice rather than being averaged away. Deploying the retrained model across the tiers then adds requirements that no offline study covers: reproducible quantization, numerical validation of the converted model on the device, and a runtime energy budget.

14.9.1.3 Key operational insights

Following offline validation, the systems challenge shifts from statistical accuracy to deployment safety and runtime efficiency. The toolchain for the keyword spotter needs an automated quantization pipeline that converts FP32 weights to INT8 with calibrated scale factors, deterministic cross-compilation for the firmware that runs the model, cryptographically signed release packages for each hardware revision, and fault-tolerant OTA delivery through the dual-bank scheme of section 14.4.4, whose cost is a second 200 KB flash bank (table 14.11).

Edge MLOps operates under physical constraints distinct from server-side pipelines. Model accuracy must be co-designed with battery life, radio bandwidth, and on-device storage, and real-time ground-truth labels are nearly absent. In the power-bound row of table 14.5, health monitoring relies on operational proxies (wake rate, duty cycle, false-wake reports, and confidence distributions) rather than verified outcomes. Recovering the four-point detection drop in the field requires configuration management across every execution target (the microphone hardware revision, the DSP front end, the firmware, and the quantized model weights) so that each fielded device's combination can be traced and receive the right update.

These constraints map directly onto the five principles of table 14.26. Versioned audio datasets provide reproducibility by linking every deployed model to the recordings, labeling guidelines, and microphone revisions that trained it. Tiered software architecture enforces separation of concerns, isolating the device's audio drivers and feature extractor from the hub's verification model and the cloud's aggregation, so that firmware, model, and fallback behavior can be updated independently. A feature-extraction kernel shared between training and the device reduces implementation divergence, while parity tests verify numerical agreement between the Python pipeline and the device's DSP

[26] **Photoplethysmography (PPG)**: Optical technique that infers blood-volume changes from variations in detected light after illuminating tissue. For ML operations, PPG introduces a data quality challenge absent in controlled environments. Motion artifacts from wrist movement corrupt the signal, creating a data drift pattern where the same physiological state produces different input distributions depending on user activity. Models must either filter corrupted windows before inference or learn to be robust to motion noise, and monitoring must distinguish genuine physiological changes from artifact-induced distribution shift.

[27] **Polysomnography (PSG)**: A multi-parameter sleep study that provides the clinical ground truth data for this classification task. This "truth" is inherently noisy; expert human scorers interpreting the same PSG recordings agree with each other at about 82 percent–83 percent reliability (Altini and Kinnunen 2021). Inter-scorer agreement indicates label uncertainty but does not impose an accuracy ceiling against a fixed or adjudicated reference.

routine across firmware versions. Privacy-preserving telemetry keeps degradation observable by sending aggregate diagnostics (per-band energy histograms, wake counts, duty cycle, and battery drain) without streaming audio off the device. Finally, cost-aware automation applies to the over-the-air update, because every update must show that its accuracy gain outweighs the radio energy of reception, the flash rewrite cycles, and the risk of altering software on a device that is always listening. Where the keyword spotter shows how milliwatts, kilobytes, and radio bandwidth dictate MLOps design, moving machine learning from consumer devices into regulated healthcare shifts the primary constraint from hardware physics to patient safety, clinical validation, and algorithmic accountability, the operational domain of ClinAIOps.

14.9.2 ClinAIOps case study

On an always-listening consumer device, the engineering envelope is bounded by milliwatts, memory footprint, and radio bandwidth; in regulated medical care, the governing constraint shifts to patient safety and clinical liability, because an unreviewed model update or an uncalibrated sensor stream in a blood-pressure monitoring system can recommend an unsafe dose before a physician sees the chart. In continuous therapeutic monitoring (CTM),[28] wearable sensors stream physiological telemetry to guide personalized medical interventions. General-purpose MLOps frameworks provide lifecycle automation (orchestrating pipelines, tracking artifacts, and monitoring availability). Clinical use adds intended-use-specific evidence, oversight, and authorization requirements because predictions may influence treatment. General-purpose MLOps can incorporate human gates; the clinical framework makes their responsibilities explicit. The case is read here as an operations loop with clinicians inside it: who may change a model, on what evidence, and how a change is reversed. Whether the objectives such a system pursues, and their effects on patients, are the right ones is a question of responsibility that chapter 15 develops.

[28] **Continuous therapeutic monitoring (CTM)**: Healthcare approach using wearable sensors for real-time physiological data collection and personalized treatment adjustments. CTM forces MLOps to confront constraints absent in typical deployments: feedback loops require oversight appropriate to the intended use, training labels need clinical validation, and model changes must follow applicable authorization and change-control requirements. These constraints reshape every MLOps principle, making CTM a stress test for operational maturity.

The proposed ClinAIOps framework (Chen et al. 2023) adapts MLOps principles to these constraints by formalizing the boundaries between algorithmic inference and clinical authority. Rather than treating human oversight as an external obstacle, ClinAIOps builds multi-party feedback loops directly into the serving architecture. The framework partitions operational control across three actors (patients, clinicians, and machine learning engineers), each operating on distinct timescales with explicit safety gates.

14.9.2.1 Feedback loops

Three interlocking feedback loops organize the proposed integration of machine learning into clinical practice; their effectiveness and safety require validation in the intended setting. Figure 14.12 models these three interlocking feedback loops as a triangular protocol among the patient, clinician, and AI developer. Each pair of actors exchanges different information in both directions: the patient streams wearable sensor telemetry and receives clinician-approved therapy alerts; the AI developer provides longitudinal monitoring summaries while accepting physician-defined dosing bounds; and the clinician directly governs patient therapy regimens while establishing safety boundaries for algorithmic actuation. The outer cycle connecting all three nodes forms the overarching clinical governance framework.

Each feedback loop operates on a distinct timescale to balance responsiveness against clinical verification. The **patient treatment loop** executes continuously in real time, capturing sensor telemetry and delivering bounded adjustments for patient self-management. At an asynchronous supervisory cadence, the **clinician oversight loop** reviews aggregated trends, sets operational boundaries, and validates treatment recommendations before interventions occur. Finally, the **developer feedback loop** aggregates telemetry and clinician override patterns back to engineering teams, driving offline retraining and interface calibration.

Patient treatment loop The patient treatment loop executes localized inference over high-frequency physiological streams captured by wearable devices, such as continuous glucose monitors or optical pulse sensors, while the edge serving runtime evaluates these incoming feature vectors against recent patient history from electronic health records. In this proposed design, recommendations follow a tiered execution model as one safety control: minor adjustments that fall strictly within clinician-defined bounds are surfaced directly to the patient, whereas significant changes, and recommendations generated when model uncertainty exceeds an acceptable margin, are blocked

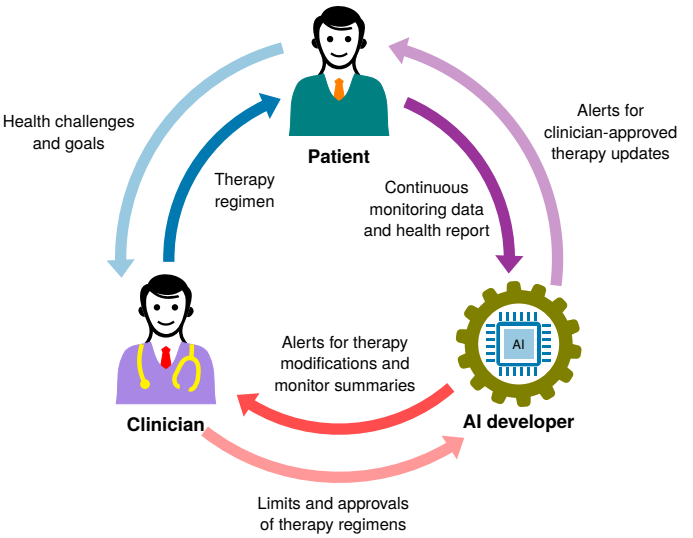


Figure 14.12: ClinAIOps Feedback Loops: The cyclical framework coordinates patients, clinicians, and AI developers to support continuous model improvement and safe clinical integration. Patients and clinicians use AI outputs in care workflows, while AI developers receive feedback from both groups to refine models and operations. Source: (Chen et al. 2023).

from automated delivery and escalated to the clinician. This structure enables rapid adaptation to dynamic physiological states within validated, authorized operating limits; those limits and the escalation mechanism must be tested.

Clinician oversight loop The clinician oversight loop provides supervisory control over automated inference. Rather than burdening medical staff with continuous raw sensor telemetry, batch aggregation pipelines distill time-series streams into longitudinal summaries, such as circadian variance, multi-day trends, and signal-quality metrics. When the system generates a therapy recommendation, such as adjusting an antihypertensive dosage for a patient with persistent hypotensive readings, the clinician evaluates the proposal alongside diagnostic factors unobserved by the wearable model and may approve, modify, or reject the recommendation. This decision is written to an audited event log, which enforces clinical accountability at the point of care and records expert feedback for later review, though approval or override is not by itself a verified patient outcome and using it as training supervision requires a defined label interpretation and validation. Clinicians also update the operational thresholds that govern the patient treatment loop, recalibrating the boundary between automated guidance and mandatory review.

Developer feedback and patient-clinician coordination Offloading continuous data logging and low-level trend detection to the AI pipeline changes the patient-clinician relationship itself. Consultations shift from routine measurement gathering to targeted clinical evaluation and goal setting, an outcome the patient and clinician loops produce together, while the third loop, developer feedback, links that clinical practice back to engineering infrastructure. Operational telemetry generated during these interactions provides engineering teams with direct visibility into production behavior: overrides, rejection frequencies, and reported adverse events motivate investigation of model behavior, workflow fit, patient mix, and sensor quality, even though they do not uniquely diagnose covariate shift. Engineers use the resulting evidence to evaluate targeted retraining, refine data validation schemas, and eliminate usability failure modes that induce alert fatigue in clinical staff.

14.9.2.2 Hypertension case example

Hypertension management illustrates how these three operational loops could coordinate care in the proposed framework. Blood pressure fluctuates continuously under exertion, stress, and circadian cycles, requiring individualized, long-term pharmacological adjustments that benefit from continuous therapeutic monitoring.

Data infrastructure Research systems estimate systolic blood pressure indirectly from electrocardiography (ECG), photoplethysmography (PPG), pulse-transit time, and heart-rate features (Q. Zhang et al. 2017). In the illustrative workflow, these signals are augmented by accelerometer streams to identify motion artifacts and self-reported medication logs. Clinical use requires rigorous calibration and prospective validation for the intended task, together with any applicable regulatory authorization; authorization alone does not establish accuracy for every cohort. When properly validated for the target cohort, this multimodal data stream is merged with electronic health records to supply feature context for inference.

Loop implementation The hypertension workflow labels each of its three panels by the two parties in the exchange (figure 14.13), so the panels do not line up one-to-one with the three loops of figure 14.12. The patient-AI panel is the patient treatment loop, the clinical-AI panel is the clinician oversight loop, and the patient-clinical panel is the coordination those two loops produce together; the developer feedback loop operates out of band, as described in section 14.9.2.1, and does not appear. Across the top of the figure, the two AI-assisted loops operate concurrently: the patient-AI panel (upper left) captures continuous blood-pressure telemetry and issues recommendations for bounded dose titrations, while the clinical-AI panel (upper right) alerts the physician to severe hypertensive excursions and accepts updated titration boundaries. Below them, the patient-clinical panel captures the resulting clinical relationship, where routine telemetry offloaded to the AI loop frees consultation appointments for longitudinal trend review, adverse-event evaluation, and shared lifestyle goal setting.

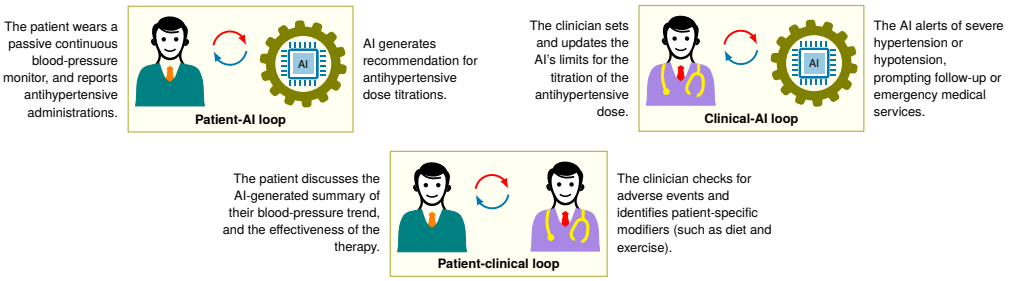


Figure 14.13: Hypertension Management Loops: The three panels, labeled patient-AI, clinical-AI, and patient-clinical, each carry a two-way exchange. In this proposed workflow, the AI issues bounded dose recommendations within validated, authorized clinician-set limits and escalates severe readings, which leaves the patient-clinical panel for trend review, adverse-event checks, and treatment decisions. The developer feedback loop of ClinAIOps is not among them and is discussed in the prose. Source: (Chen et al. 2023).

The three panels illustrate a proposed boundary for clinical automation, not an established treatment protocol: routine monitoring and bounded dose recommendations operate within validated, authorized clinician-defined thresholds, while acute escalations, adverse events, and regimen changes require explicit human authorization. That boundary is the point where ordinary MLOps practices require the additional clinical coordination summarized next.

14.9.2.3 MLOps vs. ClinAIOps comparison

The hypertension case illustrates the operational contrasts between standard software pipelines and healthcare deployments. General-purpose MLOps provides technical lifecycle controls such as automated testing, continuous integration, model registries, and drift monitoring. ClinAIOps extends this foundation into regulated clinical environments where outputs directly influence medical interventions. Table 14.27 compares the two approaches across eight operational dimensions.

Table 14.27: Clinical AI Operations: General-purpose MLOps supplies technical lifecycle controls; ClinAIOps extends them with clinical workflows, evidence, human oversight, and accountability.

	General-purpose MLOps emphasis	ClinAIOps extension
Focus	Technical model lifecycle	Human and AI decision-making

	General-purpose MLOps emphasis	ClinAIOps extension
Stakeholders	ML, data, platform, and operations teams	Adds patients, clinicians, and clinical governance
Feedback loops	Monitoring, retraining, and release	Adds treatment, clinician, and developer feedback
Objective	Reliable, governed ML operations	Safe, effective, and accountable care support
Processes	Automated pipelines and operational controls	Integrates clinical workflows and human gates
Data considerations	Lineage, quality, freshness, and access control	Adds consent, clinical provenance, and protected data
Model validation	Predictive, operational, and slice-level metrics	Adds clinical utility, safety, and cohort outcomes
Implementation	Technical integration and operational ownership	Adds clinical accountability and stakeholder incentives

The table's central distinction is that clinical deployment alters risk ownership. Statistical accuracy is a prerequisite, but it does not protect against liability when an erroneous prediction drives a medical intervention. The ClinAIOps framework therefore changes the governing constraint from device efficiency to clinical accountability. The model participates in care delivery, but it cannot own clinical decisions. Every recommendation and subsequent clinician action must be traceable to the underlying sensor inputs, model version, and inference metadata; otherwise the system cannot support regulatory audit or retrospective incident analysis. Separation of concerns shifts from an architectural preference to a safety invariant: sensor ingestion, inference scoring, clinical diagnosis, and parameter updates must be decoupled by explicit software interfaces and human approval gates.

The same accountability requirement transforms monitoring. Standardized clinical data pipelines support training-serving parity, but clinical validation requires evaluating model recommendations against standard-of-care outcomes and cohort-specific effects. Observable degradation extends beyond statistical loss and latency to clinical metrics: blood pressure control rates, adverse events, clinician overrides, and subgroup disparities. In this regime, multi-tier feedback loops are not unwanted operational latency; they are deliberate architectural controls that balance automated edge adaptation against legal and ethical accountability. Cost-aware automation operates strictly inside those gates. Model refreshes occur only when validated clinical benefits outweigh validation costs and patient risks, and low-confidence predictions trigger deterministic fallback to human clinical review.

14.9.3 Case study synthesis

The keyword spotter and ClinAIOps cases separate stable MLOps principles from the deployment constraints that reshape their implementation: copying an edge OTA pipeline into a hospital workflow or imposing clinician sign-off gates on every consumer wake-word update fails because it mistakes a domain-specific tool for the underlying principle. The keyword spotter is a resource-envelope case. Its operations must preserve reproducibility, consistency, and observable degradation while battery, telemetry, and weak ground truth limit what can be measured and updated on the device. ClinAIOps is an accountability-envelope case. The same principles apply, but validation, audit trails, and human gates dominate because the model influences clinical action.

The shared engineering lesson is that MLOps maturity is not tool accumulation. It is the ability to identify the governing constraint, choose the operational controls that match it, and preserve evidence when the model changes. Production ML systems fail when teams apply code-focused operational intuitions without accounting for statistical behavior and shifting distributions, exposing common fallacies and operational pitfalls.

Checkpoint 14.6: Transferring the operational principles

- ☐ **Evidence and design**: Separate the wearable study's reported offline accuracy from the keyword spotter's deployment and monitoring design, and name two proxy signals the spotter can report without uploading audio.

- ☐ **Feedback quality**: Explain why human scorer agreement is not a strict accuracy ceiling and why a clinician override is not automatically a verified patient outcome.
- ☐ **Shared principle**: Choose one MLOps principle and explain how battery limits and clinical accountability lead to different implementations of it.

14.10 Fallacies and Pitfalls

Operating production machine learning systems under conventional software assumptions creates distinct failure modes. The following fallacies and pitfalls highlight where standard software engineering intuitions break down against the statistical realities of deployed models.

Fallacy: *MLOps is just applying traditional DevOps practices to machine learning models.*

Engineers often assume that standard CI/CD pipelines transfer directly to machine learning. Traditional DevOps automates code builds, testing, and artifact deployment with controls for dependencies and execution environments. In an ML system, code represents only one axis of the D·A·M triad. Model execution depends jointly on code, large evolving datasets, and accelerator compute budgets. As detailed in section 14.3.2.1, ML pipelines introduce stateful retraining, multi-metric validation gates, and immutable artifact registration. Standard CI/CD engines cannot natively track data lineage, orchestrate feature stores, or detect distribution drift. When teams apply DevOps without ML-specific adaptations, they maximize the computational availability of their serving cluster while leaving statistical behavior unmonitored. The system achieves high uptime and low latency, yet silently serves degraded predictions due to unvalidated data shifts and training-serving skew. The cost is concrete. A 1 percent skew rate on 1,000,000 queries/day costs $365,000 a year (equation 14.2) while every DevOps health check stays green.

Pitfall: *Treating model deployment as a one-time event rather than an ongoing process.*

Treating deployment as a terminal milestone assumes that deployed weights remain permanently aligned with the production input distribution. Unlike compiled binaries whose execution semantics remain fixed, model quality can change as real-world distributions shift. Monitoring statistical indicators such as PSI (section 14.5.2.1) provides an early signal of feature drift, distinguishing benign variance from actionable distribution change. Crossing a warning threshold requires operational diagnosis rather than unvalidated retraining. As established in section 14.6.2, scheduling updates is an economic balancing problem. The optimal retraining interval $T^* \approx \sqrt{\frac{2C}{Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}}$ weighs the fixed compute and engineering cost C against the business losses accumulated from daily query volume Q_{day}, unit accuracy value V, and staleness decay rate γ. For the ranking service, a 0.24 percent daily decay on 1,000,000 requests a day puts that optimum near 3 days (notebook 14.6), a cadence no one-time deployment can meet. Operating a production model demands treating deployment as a continuous control loop that balances monitoring signals against retraining economics.

Fallacy: *Automated retraining ensures optimal model performance without human oversight.*

Automating pipeline execution does not eliminate the need for operational governance. Closed-loop retraining pipelines operate without semantic awareness; if upstream data ingestion suffers schema anomalies, corrupted labels, or sudden sensor drift, automated retraining bakes those errors directly into updated model weights. Furthermore, automated validation gates typically evaluate aggregate loss across a held-out test split, which easily conceals severe performance regressions on critical slices or low-frequency edge cases. For instance, retraining a ranking model on weekend traffic can degrade performance across latency-sensitive weekday workloads where user query distributions diverge, and an aggregate gate would pass it, just as the 91 percent overall accuracy in table 14.20 conceals a tablet slice at 62 percent. Resilient MLOps architectures decouple automated pipeline execution from automated production promotion, enforcing canary deployments, slice-level regression gates, and human sign-off when validation metrics deviate from expected historical envelopes.

Pitfall: *Focusing on technical infrastructure while neglecting organizational and process alignment.*

Investing in MLOps tooling without defining organizational interfaces leads to fragmented operational ownership. Machine learning systems bridge two distinct engineering cultures: model

developers optimizing offline accuracy metrics and platform engineers managing host-level latency, memory capacity, and throughput budgets. When organizations treat deployment as a handoff between isolated teams, root-cause triage stalls during production incidents. Infrastructure engineers inspect container health and GPU utilization without insight into model prediction distributions, while model authors inspect offline training curves without understanding runtime serving constraints. The ML Test Score (section 14.8.1) makes the gap measurable, because a deployment with a perfect model suite and no production monitoring scores 0 under its minimum-category rule. Tools such as feature stores and registries provide technical scaffolding, but reliability requires institutional alignment: unified SLOs spanning statistical accuracy and serving latency, joint on-call runbooks, and shared accountability for production regressions.

Fallacy: *Training and serving environments automatically remain consistent once pipelines are established.*

Establishing an initial end-to-end pipeline does not guarantee permanent parity between training and serving. Training-serving skew arises from fundamental architectural divergences between offline and online execution paths. Training typically runs as an offline batch workflow (such as distributed SQL queries or Spark pipelines) optimized for high-throughput scans over historical logs, whereas online serving runs in low-latency microservices under millisecond latency constraints. Maintaining dual implementations of feature transformations across these environments risks numerical disparities, differing missing-value semantics, or lookahead bias in historical joins. On the ranking service, the two paths computed `user_session_length` with means of 45 and 12 minutes and cost 8 percent of accuracy with no change to model code (section 7). As demonstrated in section 14.3.1.1, feature stores alleviate this divergence by unifying transformation logic and synchronizing batch and point-lookup stores. However, software pipelines alone cannot guarantee consistency; operational verification requires request-keyed value comparisons and schema assertions between offline replays and online inference inputs, alongside separate distribution monitoring to catch divergence before it degrades downstream decisions.

Pitfall: *Assuming comprehensive monitoring prevents all production incidents.*

Dashboards and alert thresholds cannot prevent production outages caused by silent data failures. Monitoring supplies evidence; reducing harm also requires a response. Input-boundary checks can catch some defects before inference, while statistical quality estimates may arrive after affected predictions have been served. Furthermore, tracking output metrics alone leaves critical blind spots along the data pipeline. As shown in section 14.5.2.1, measuring service latency and prediction distributions fails to detect input anomalies such as feature staleness. An inference service can maintain nominal p99 response times and normal prediction ranges while consuming stale user embeddings caused by upstream database replication lag or cache eviction failures. By the time downstream business metrics degrade, corrupted predictions have already impacted users; an input-boundary check acts on the first malformed request, without waiting for labeled outcomes to accumulate. Production reliability requires defensive boundary controls alongside passive observability: strict input schema enforcement, feature freshness assertions, and circuit breakers that fall back to deterministic heuristics when input quality slips below operational tolerances.

Fallacy: *Accuracy is the first production signal to monitor, so leading indicators can wait on a dashboard.*

Teams often configure accuracy dashboards as their primary operational tripwire, assuming degradation will register there first. When outcome labels arrive late, ground-truth accuracy is a lagging indicator. In production environments such as fraud detection, ad conversion, or credit underwriting, ground-truth labels arrive with substantial latency, often requiring weeks or months to accumulate through chargebacks or repayment histories. Relying solely on accuracy dashboards leaves the system operating blind throughout this label-delay window. Furthermore, aggregate accuracy can remain deceptively stable even as input distributions diverge across critical sub-populations. As detailed in section 14.5.2.1, monitoring input feature distributions with metrics like PSI or Kullback-Leibler (KL) divergence supplies indicators that can be estimated from sufficient input samples without labels. These statistical signals detect distribution shifts without waiting for labels, often enabling investigation before delayed outcome metrics are available, without guaranteeing that prediction errors have not already occurred. The delay is large. The same evidence budget that one labeled outcome per second fills in about 16.7 minutes takes 10 days at 100 outcomes a day, and in

the silent-failure scenario each incident detected at monthly rather than daily review costs $184,932 more (notebook 14.3). A leading indicator recovers that time only when it is routed to an owner with a response deadline, an escalation rule, and a diagnostic runbook; drift and freshness warnings parked on an informational dashboard accumulate unnoticed until an emergency forces a response.

Observability in production machine learning extends far beyond passive dashboard telemetry. It requires closed operational loops across the entire D·A·M architecture: enforcing input data integrity and feature stability, bounding model behavior through automated validation gates, and routing leading indicators directly to engineers empowered with diagnostic runbooks to investigate changes and limit harm.

14.11 Summary

MLOps extends software reliability with controls for learned behavior, data dependencies, and statistical uncertainty. A crashed server or network timeout turns availability dashboards red immediately, alerting operators to take corrective action. A degraded model introduces a silent statistical failure mode. The serving infrastructure returns 200 OK responses within latency deadlines, yet prediction quality can deteriorate undetected. Machine learning operations keeps that decay observable by treating model quality as a dynamic runtime variable rather than a static deployment artifact.

The five principles of table 14.2 provide an evaluation framework that applies regardless of scale or domain, and each now has a home in the operations loop. Reproducibility lives in versioned artifacts and the model registry, separation of concerns in the boundaries that releases and incidents respect, consistency in shared feature definitions checked by request-keyed parity tests, observable degradation in layered monitoring, and cost-aware automation in the retraining gate and the interval it implies.

Infrastructure components implement these principles across three critical operational boundaries: feature stores and data versioning govern the data-model interface by reducing training-serving inconsistency, CI/CD pipelines and model registries govern the model-infrastructure interface by enforcing reproducibility and enabling rollback, and monitoring systems, incident response frameworks, and on-call practices govern the production-monitoring interface by making degradation observable and actionable while the retraining decision framework connects measured degradation to economic thresholds. Domain constraints reshape *how* these principles are implemented without changing *which* principles matter. The capacity-bound ranking service pays for safe releases in memory, a second resident copy of its embedding tables for every shadow, canary, or standby, and pays for retraining in accelerator-hours that contend with those copies. The power-bound keyword spotter pays in flash, radio energy, and evidence it cannot upload. Similarly, ClinAIOps demonstrates why high-stakes clinical deployments require graceful-degradation and human-oversight controls, with three feedback loops functioning as architectural patterns rather than operational overhead.

The chapter's operating rules fit on one page (table 14.28). Together they let an engineer estimate, before an incident forces the question, what a control costs and when it pays for itself.

Table 14.28: Key Equations: The chapter's operating rules in the notation of the sections that derive them, in chapter order. The skew cost and retraining gate are stated with the principles, the sample size with A/B testing, the delay, PSI, and cost rules with monitoring, and the decay and interval with the retraining decision; the interval is the square-root approximation valid while decay over one interval is small.

Quantity	Formula	Where derived
Skew cost	$\text{Skew Cost} = \text{Rate}_{\text{skew}} \times Q_{\text{day}} \times C_{\text{error}}$	equation 14.2
Retraining gate	Retrain if: $\Delta\text{Accuracy} \times \text{Value per Point} > \text{Training Cost} + \text{Deployment Risk}$	equation 14.3
A/B sample size	$n = \frac{2(z_{\alpha/2}+z_{\beta})^2\sigma^2}{\delta^2_{\text{MDE}}}$	equation 14.4
Detection delay	Evidence budget in labeled examples divided by the labeled-outcome arrival rate	Notebook 14.2
Population stability index	$\text{PSI} = \sum_{i=1}^{n_{\text{bins}}} (\text{actual}_i - \text{expected}_i) \times \ln\left(\frac{\text{actual}_i}{\text{expected}_i}\right)$	section 4.12.1
Telemetry cost rate	$\text{Cost rate} \approx N_{\text{series}} f_{\text{sample}} \left(C_{\text{ingest/sample}} + s_{\text{sample}} T_{\text{retention}} C_{\text{storage/byte-time}}\right)$	equation 14.5

Quantity	Formula	Where derived
Monitoring benefit and cost	$\text{Monitoring Benefit/Cost} = \frac{\text{Incidents Prevented} \times \text{Avg Incident Cost}}{\text{Annual Monitoring Cost}}$	equation 14.7
Accuracy decay	$\text{Accuracy}(t) = \text{Accuracy}_0 \cdot e^{-\gamma t}$	equation 14.8
Optimal retraining interval	$T^* \approx \sqrt{\frac{2 \cdot C}{Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}}$	equation 14.12

Key Takeaways: Perfectly available, perfectly wrong

- **ML systems fail silently without outcome monitoring**: Model quality may degrade as distributional divergence $\mathcal{D}(P_t \| P_0)$ grows, although divergence alone does not determine accuracy loss. Because a serving node can maintain perfect uptime while predictions fail, tracking ground-truth outcomes is essential rather than relying on server health checks alone.
- **Training-serving skew corrupts inference without crashing**: Feature stores mitigate skew by unifying feature definitions and transformations across environments, but maintaining parity still requires continuous validation across online and offline data pipelines.
- **Deploy through graduated rollout with pretested rollback**: Canary, blue-green, and shadow deployments partition release risk across production traffic, backed by tiered rollback procedures that must be verified through regular operational drills. For a capacity-bound model, each of these controls holds a full second copy of the model in memory, whatever share of traffic it serves.
- **Retraining is an engineering optimization, not a guess**: The fitted staleness cost model ($T^* \approx \sqrt{2C/(Q_{\text{day}} \cdot V \cdot \text{Accuracy}_0 \cdot \gamma)}$) balances retraining compute against prediction quality decay, connecting update cadence to quantitative economics under its stated assumptions.
- **Stage infrastructure investments by system risk**: Instrumentation and continuous deployment form the initial operational baseline. High-consequence systems justify greater operational rigor; feature stores and automated retraining should be added as skew and scale demand.
- **The five principles transfer across domains**: Reproducibility, separation of concerns, consistency, observable degradation, and cost-aware automation remain invariants across applications; domain constraints reshape only their concrete mechanisms and tool implementations.
- **Operational maturity is organizational as well as architectural**: Managing a single model differs qualitatively from governing a production fleet. While the core principles scale, operational complexity grows with fleet size, making unified incentives and shared on-call rotations as essential as tooling.

This operational discipline distinguishes production ML engineering from development experimentation. When a deployed model degrades, systematic instrumentation enables practitioners to isolate the failure mode: identifying data drift through distribution divergence checked against labeled outcomes (principle 10), training-serving skew through preprocessing parity audits (principle 11), configuration debt through artifact provenance, and feedback loops through temporal output tracking. Treating production ML as a "deploy and forget" exercise ignores these distinct dynamics.

This operational reality roots ML engineering in the D·A·M taxonomy. In an ML system, code, weights, and accelerator hardware can remain unchanged while the data axis moves beneath them, changing the predictive behavior that the system must validate. That exposure is the other side of the bitter lesson. Behavior learned from data rather than written by hand is what lets these systems outperform hand-built rules, and it is also what lets their behavior change whenever the data does, with no line of code recording the change. The match between model and world is not

an engineering state reached once at deployment, but an ongoing cost paid continuously through telemetry, validation gates, and cost-calibrated retraining. System reliability must therefore measure prediction outcomes rather than server uptime alone, because the most dangerous failure mode in production is a green dashboard masking a drifting model. That machinery keeps a model faithful to its objective; whether the objective itself was right, and for whom, is the next question.

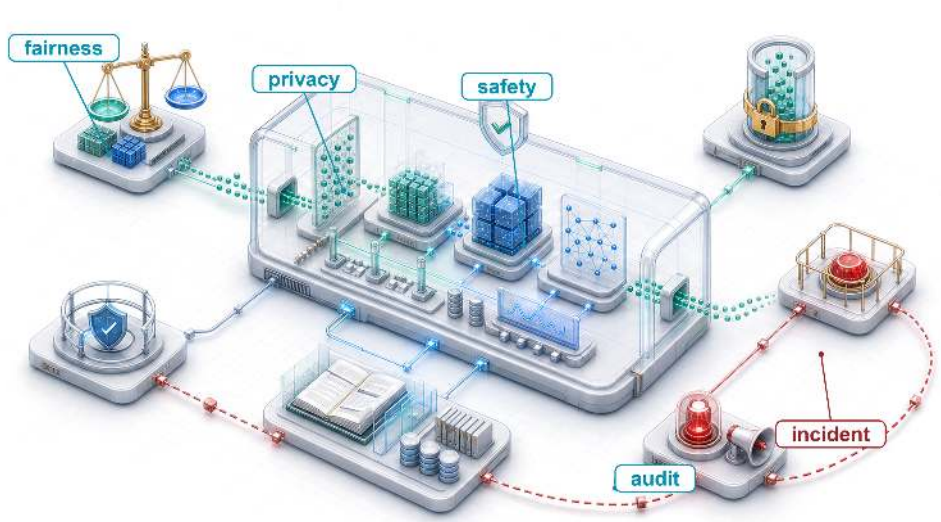


What's Next: From reliability to responsibility

What if the model never drifts and still does harm? Operations keeps a model faithful to its objective as the world shifts, but it cannot tell whether that objective was the right one. A system that tracks its training target perfectly can still reproduce historical bias, misallocate errors across groups, or mishandle personal data, while every reliability check and aggregate accuracy metric stays green. Chapter 15 takes up that specification gap, extending the monitoring discipline of this chapter from predictive quality to the outcomes a system produces for the people it affects.

15

Responsible Engineering

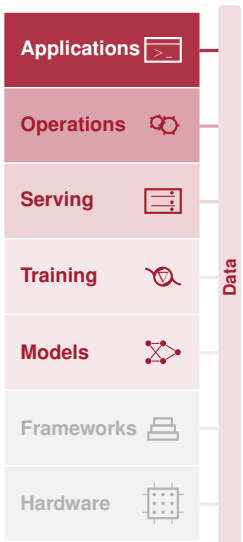


15.1 Responsibility as Systems Engineering
15.2 Engineering Responsibility Gap
15.3 Responsible Engineering Across the Lifecycle
15.4 Environmental and Cost Awareness
15.5 Data Governance and Compliance
15.6 Fallacies and Pitfalls
15.7 Summary

Purpose

Why can a system that does exactly what it was told to do still cause harm?

Latency, availability, and accuracy targets do not say whether they describe the right behavior, whom the system serves, or which costs its specification leaves unmeasured. An ML system can satisfy latency, throughput, and aggregate accuracy requirements while reproducing historical discrimination, rewarding harmful engagement, consuming unjustified energy, or using data without adequate privacy and accountability controls. Such a system has not malfunctioned; it is efficiently optimizing an incomplete specification. The omitted constraints appear as uneven outcomes across populations, lifetime cost and emissions, decisions people cannot understand or contest, and evidence that cannot be reconstructed after harm occurs. If those consequences are not defined as requirements before deployment and monitored afterward, conventional health checks can remain green while harm accumulates. A credit model that treats groups unevenly can repeat that decision millions of times before anyone aggregates the outcomes, while its dashboards report healthy latency and accuracy throughout. Responsible engineering treats such outcomes as system failures to be diagnosed, measured, and mitigated with the same rigor as latency or accuracy regressions. That work requires translating broad obligations into testable bounds, documented assumptions, traceable data, and response paths that remain enforceable in production, so that a harm found after launch can be traced and corrected. In D·A·M terms, responsibility broadens the definition of correctness: data must be examined for encoded harms and lawful use, algorithms must be bounded by defensible outcome and efficiency requirements, and machine infrastructure must monitor, document, and enforce those boundaries throughout the system lifecycle.



Learning Objectives

- Explain how a model that meets its training objective can still cause harm when that objective is a proxy that drifts from the goal
- Apply data-algorithm-machine diagnosis to localize responsibility failures in data, algorithm objectives, or monitoring infrastructure
- Calculate fairness metrics from confusion matrices and compare trade-offs on the fairness-accuracy Pareto frontier
- Design disaggregated release gates and per-group monitoring, fitting the bias feedback model to project when a passed gate will fail
- Analyze total cost, inference dominance, and carbon impact as measurable responsibility constraints
- Construct model cards, datasheets, lineage records, and audit trails for accountability
- Evaluate privacy, access-control, and compliance designs against regulatory and human-review requirements

15.1 Responsibility as Systems Engineering

A credit model decides whose loan applications are approved, a ranking model decides which items millions of people see, and a voice assistant decides when audio from a private room leaves the device. None of these outcomes appears in a loss curve or a latency dashboard, yet every deployed model settles them, and a system can meet every latency, uptime, and aggregate accuracy target while rejecting qualified minority applicants, amplifying clickbait, or leaking private audio. **Responsible engineering** is systems engineering applied to the consequences that a model's objective leaves unstated: whose errors the system makes, how much energy and money each decision draws, and whether a harmful decision can later be explained, traced, and corrected. Each of those outcomes is fixed by concrete engineering choices about data, objectives, and infrastructure.

Chapter 14 built the machinery that keeps a deployed model faithful to its objective as data and traffic change, and that machinery assumes the objective was the right one. If MLOps is the control loop for *reliability*, then responsible engineering is the control loop for *safety*. Where MLOps monitors runtime health and predictive quality to guide operational response, responsible engineering monitors outcome distributions to guide intervention when systems generate harm. In systems engineering terms, a model can pass *verification* (conforming exactly to its specified training objective) while failing *validation* (failing to satisfy operational, safety, or legal requirements) (National Aeronautics and Space Administration 2016). The verification gap (principle 9) concerns the first of these activities. It bounds how much any finite test can establish about conformance to a specification, whereas validation asks whether the specification was the right one. The failure stems from an incomplete specification rather than a fault in compiler execution or numerical optimization.

↩ Continuous telemetry and automated retraining control loops for operational reliability are established in section 14.1.1.

Traditional software engineering isolates defects within modules, though shared dependencies can still propagate faults. Machine learning systems introduce data-dependent coupling: data flows through shared representations, so corrupted or biased inputs alter the dense parameter matrices that serve every subsequent inference. A biased training dataset does not trigger a compile-time assertion or runtime crash; it silently shifts decision boundaries across the entire system. Read for responsibility rather than performance, the D·A·M taxonomy isolates where such failures originate: unrepresentative or biased inputs (Data), misaligned loss formulations (Algorithm), or telemetry blind spots and unjustified resource footprints in the serving infrastructure (Machine). Responsibility therefore has to be designed in rather than patched on after deployment.

↩ The D·A·M taxonomy, which classifies a system bottleneck along its data, algorithm, and machine axes, is established in section 1.5.

Responsible engineering expands the definition of correctness for machine learning systems. Traditional correctness (numerical stability, bounded latency, and high service availability) remains necessary, but insufficient. A production system must satisfy broader operational bounds: bounded error disparities across demographic slices, defensible energy expenditure, and verifiable data lineage. This expanded correctness applies engineering discipline to failure modes that aggregate performance metrics ignore. A p99 latency spike triggers automated alerts on an operational dashboard; a demographic fairness regression remains invisible to aggregate loss metrics until real-

world harm occurs, and when the model's decisions shape its own future training data, the hidden disparity can grow with every retraining cycle (principle 13). Both demand rigorous telemetry and automated regression testing. The optimizations of the preceding chapters inherit the same blind spot, because each was judged by the cost terms its benchmark measured.

> **Systems Perspective 15.1: The full cost of the iron law**
>
> The iron law of ML systems (principle 3) decomposes execution time into data movement, computation, and fixed overhead. Compressing models, accelerating hardware, and automating operations each optimize these terms. Yet an optimization can create or shift costs that its benchmark omits.
>
> A model quantized for edge deployment can consume less energy, but its accuracy loss need not fall evenly across demographic groups, so the compressed model must be revalidated on the same subgroup slices as the original. A ranking model such as the DLRM service, optimized for engagement, maximizes a business metric but may amplify harmful content. Responsible engineering extends this accounting to include broader impacts: the carbon cost of computation, the fairness cost of optimization choices, and the societal cost of deployment at scale. The iron law accounts for how fast a system runs and says nothing about whom it serves well.

Three running cases carry the chapter, each chosen because it exposes a different kind of responsibility failure. The first is a credit service, a labeled composite rather than a documented deployment: a classifier that approves or rejects loan applications, constructed so that every fairness criterion in the chapter can be computed from its confusion matrices. Fairness criteria need labeled decisions about people, which none of the lighthouse models makes, so the composite carries the stages that specify, measure, explain, and monitor such decisions. The second is the DLRM ranking service, the capacity-bound lighthouse, whose model alone occupies 100 GB in its registry entry, more than the 80 GB of memory on one H100. Its rankings decide what people see, so the clicks it trains on are shaped by its own earlier outputs, and its request volume makes it the chapter's case for feedback, exposure, cost, and carbon. The third is the keyword spotting (KWS) system, the power-bound lighthouse, an always-listening microphone in private homes whose design must decide what audio may leave the device, which makes it the case for privacy, lineage, and audit.

The chapter follows those cases through the lifecycle stages at which each failure is still cheap to prevent (table 15.1). Section 15.2 explains why optimization can succeed while the system fails, through proxy targets, feedback loops, and distribution shift that leave every health check green. Section 15.3 then works through four chained stages on the credit service: specifying requirements before training, measuring fairness criteria as release gates, explaining decisions and documenting models, and operating the deployed model so that its disparity stays bounded under feedback. Section 15.4 measures the energy, carbon, and money that every stage consumes on the DLRM service. Section 15.5 builds, on the KWS system, the governance infrastructure (access control, privacy, regulatory capabilities, lineage, and audit) that makes each earlier guarantee provable to someone outside the team.

Table 15.1: Chapter Map: Each section addresses one class of responsibility failure at the lifecycle stage where it is still cheap to prevent, and names the machine resources that the remedy consumes.

Section or Stage	Failure It Addresses	What It Costs the Machine	Running Case
Engineering Responsibility Gap	Proxy targets, feedback loops, and shift that leave health checks green	Nothing yet; the diagnosis locates the failure on the data, algorithm, or machine axis	Credit service, DLRM
Lifecycle: Specify	Requirements nobody wrote down: target, populations, human oversight	Labeled evaluation data for every named subgroup; reviewer hours per escalated decision	Credit service
Lifecycle: Measure	Disparities hidden by aggregate accuracy	Per-slice evaluation; mitigation training with a second network head and a larger search	Credit service

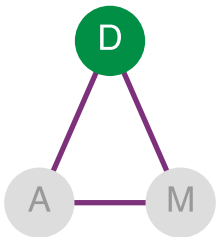
Section or Stage	Failure It Addresses	What It Costs the Machine	Running Case
Lifecycle: Explain and document	Decisions no one can contest; models reused outside their envelope	Explanation compute on the serving path, exponential in feature count for exact attribution	Credit service, MobileNetV2
Lifecycle: Operate	Disparity that grows through retraining feedback	Per-group telemetry; randomized exploration traffic served at lower expected value	Credit service, DLRM
Environmental and Cost Awareness	Energy, emissions, and cost the task does not justify	Accelerator-hours, kilowatt-hours, and kilograms of CO_2 over the service lifetime	DLRM
Data Governance and Compliance	Evidence that cannot be reconstructed after harm	Audit-log bytes, lineage storage, privacy noise and its memory cost, retraining for erasure	KWS

15.2 Engineering Responsibility Gap

The credit service can approve 90 percent of qualified majority-group applicants but only 60 percent of equally qualified minority-group applicants and still report high aggregate accuracy, because the minority group contributes only a small share of the evaluated applicants. The worked example in section 15.3.2.1 computes exactly this outcome from its confusion matrices. The **responsibility gap** between this *technical correctness* and *responsible outcomes* is the distance that conventional testing does not measure. The gap manifests through concrete mechanisms: proxy variables, feedback loops, and distribution shift, each producing harm through a distinct pathway that conventional monitoring leaves invisible.

15.2.1 When optimization succeeds but systems fail

Amazon's experimental recruiting tool (a war story in section 3.5.1), which learned to downgrade resumes that mentioned women's organizations or women's colleges, places this gap on the data axis of the D·A·M taxonomy, because its failure stemmed from historical training signal rather than an algorithmic or software defect. A model trained on a decade of historical hiring data optimized faithfully for the objective it was given, but those historical patterns encoded gender bias that the system reproduced in candidate ratings (Dastin 2018).



The Amazon failure turns on the data axis, not the algorithm.

Under empirical risk minimization, the model reduced classification loss by finding word-level correlations across the resume corpus. Because past technical hires were mostly men, words tied to women's colleges, organizations, or activities correlated negatively with positive hiring labels. The optimizer faithfully extracted the empirical distribution of the training data, but that distribution reflected past hiring practices rather than candidate competence. High-dimensional text representations pick up and amplify these statistical patterns, baking social biases directly into the vectors that downstream models consume (Bolukbasi et al. 2016).

Amazon tried to fix the model by stripping explicit gender indicators and gendered words from the training pipeline. That change failed to produce unbiased recommendations because high-dimensional feature spaces contain **proxy variables** that retain demographic signal.[1] Scrubbing isolated attributes leaves correlated features intact: ZIP codes correlate with race through residential segregation, applicant names correlate with ethnicity, and healthcare spending correlates with socioeconomic conditions. In Amazon's resume pipeline, even after explicit gender terms were removed, remaining vocabulary (attendance at specific colleges or participation in particular student organizations) continued to encode sex-correlated patterns (Dastin 2018). Removing protected attributes from training data is therefore not enough to guarantee fairness.

Closing that gap takes changes across the full system lifecycle rather than a surface-level data filter. Before deployment, evaluating each demographic subgroup separately measures how scores differ across groups, and when those slices expose a disparity, training-time mitigations such as adversarial debiasing (developed with the credit-service worked example in section 15.3.2.1) can suppress protected-attribute signal in learned representations, though they still require continuous subgroup validation. Once the model is live, human review for borderline scores provides an operational safeguard only when reviewers are trained, empowered, and audited rather than used to rubber-stamp model outputs, while downstream monitoring tracks hiring outcomes over time

[1] **Proxy variable**: Removing one proxy may have little effect when other correlated features retain similar signal. Conversely, correlation alone does not prove discrimination or identify the appropriate intervention. Protected-attribute removal, subgroup evaluation, counterfactual tests, causal analysis, and review of the full decision process provide complementary evidence; no single diagnostic is a complete defense.

to catch disparate impact that offline loss metrics conceal. Because bolting these defenses onto an existing pipeline proved unworkable, Amazon scrapped the project (Dastin 2018).

The same outcome can originate on the algorithm axis instead, in the choice of target rather than in the data. When the target itself reflects historical inequities, the training objective encodes those disparities into the model's decision boundaries, and the harm lands on the subgroup the target mismeasures. Curating the data would not repair such a failure, because the data faithfully record the wrong quantity. That algorithm-axis version appeared with life-threatening consequences in clinical care management.

War Story 15.1: The proxy variable trap (2019)

Context: In 2019, Ziad Obermeyer and colleagues at UC Berkeley audited a commercial Optum algorithm used by health systems to enroll patients with complex needs into high-risk care management programs, examining roughly fifty thousand patients across one large academic hospital (Obermeyer et al. 2019).

Mechanism: The model predicted "future healthcare cost" as a proxy for "future health need." Because the US healthcare system spends less on Black patients than on White patients with the same illness level, the algorithm learned this pattern and assigned lower risk scores to Black patients.

Impact: At any given risk score, Black patients carried substantially more chronic conditions than White patients, reducing the share of Black patients enrolled in specialized care.

Response: The study showed that replacing cost with a direct measure of health need would raise the share of Black patients identified for additional care from 17.7 percent to 46.5 percent at the analyzed threshold.

Systems lesson: Optimizing for a proxy inherits the biases of the system that generated the proxy. The proxy-target relationship must be audited across every demographic subgroup the system serves.

The recruiting tool and the care-management algorithm show that optimization success can coexist with system failure. Neither disparity requires a software bug or hardware malfunction to explain why a technically functioning model can fail its intended use. The architectural question is whether the problem specification captures acceptable outcomes. When the *technical objective* (such as minimizing empirical prediction error) diverges from the *system specification* (fair, defensible outcomes across user populations), improving the training objective can preserve or amplify harm.

Standard production telemetry cannot detect specification failures. Monitoring pipelines track inference latency, memory use, throughput, and aggregate validation loss, all of which remain green while a model systematically misallocates errors across subgroups. Preventing silent specification failures requires making responsibility metrics part of the primary test harness: verifying whether the loss function acts as a defensible proxy for the true system goal, and enforcing the justified, application-specific fairness criterion before deployment.

Checkpoint 15.1: Responsible design

Responsibility is a system property, not a model property.

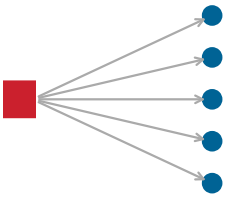
Failure modes

- ☐ **Alignment**: test whether the loss function is a defensible proxy for the system's true goal.
- ☐ **Disparate impact**: compare error rates across subgroups rather than relying on aggregate accuracy.

Check

- ☐ **Pre-mortem**: before deploying, ask: "If this system causes a scandal in six months, what likely went wrong?"

15.2.2 Silent failure modes



One upstream change; many silently affected.

↩ Drift monitors that quantify distributional divergence between serving and training data are established in section 14.5.2.

Consider the credit service after a change to its online application form alters how self-employed applicants' income is recorded. The model begins declining applicants it would have approved a week earlier, and a single upstream modification propagates across the system's blast radius (the set of downstream decisions, and the applicants behind them, that one upstream change reaches) without raising an alert. The model's confidence scores remain high, its inference latency stays within its service level agreement, and all system health checks pass green. The failure is silent. The input data distribution has shifted, but the service's pipeline monitors only service health and carries no drift monitors.

Availability-focused monitoring cannot detect this class of failure. It is the silent degradation of section 1.6, in which statistical failure modes produce degraded outputs indistinguishable from valid predictions while every kernel completes within its latency budget. The responsibility question is whom the degraded outputs reach.

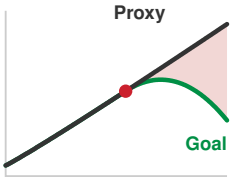
Distribution shift, a deployment input distribution that differs from the one used to develop and evaluate the model, represents the first major mechanism behind this silent degradation. Its covariate, label, and concept forms, and the divergence statistics that detect them, are developed with data engineering (see the margin). The failure is environmental. The physical distribution of inputs changed after training, yet the model has no internal telemetry to detect the discrepancy. Through the responsibility lens the pitfall sharpens, because shift often lands on one subgroup first, where a population-wide divergence statistic barely moves, and concept drift on that slice stays invisible until its outcomes are labeled. A monitor that is adequate for the population can therefore be blind to the people the shift reaches first.

↩ Covariate, label, and concept drift, with the divergence statistics that detect them, are established in section 4.12.1.

Retraining on fresh data can partially restore accuracy under distribution shift. However, retraining cannot resolve a second, more fundamental mechanism of silent failure that persists even when the data distribution is perfectly stationary: metric misalignment.

Metric misalignment occurs when the objective function optimizes an observable proxy metric that diverges from the unobserved outcome the system is intended to deliver. Goodhart's Law, the benchmark trap in which a measure optimized as a target stops measuring the goal, applies with more force here than in benchmarking, because the training objective is the target and gradient descent searches it far more exhaustively than a team tuning against a benchmark. Up to a point the proxy and the goal improve together; past that point further optimization buys proxy gains at the goal's expense.

↩ Goodhart's law, under which a benchmark metric optimized as a target ceases to measure the goal, is established in section 12.1.



Past the knee, the proxy decouples from the goal.

> Systems Perspective 15.2: The alignment gap
>
> The DLRM ranking service optimizes a proxy metric (Clicks) because the true metric (User Satisfaction) is unobservable, and the two can diverge sharply. In an illustrative version of the service, Correlation(Clicks, Satisfaction) might begin at 0.8; once ranking maximizes clicks, it may find clickbait with high clicks but low satisfaction, and the correlation could fall to 0.2.
>
> The falling correlation is the symptom. The quantity that matters is how far the proxy's movement overstates the goal's, which equation 15.1 captures for metrics normalized to a common scale:
>
> $$\text{Gap} = \mathbb{E}[\text{Proxy}] - \mathbb{E}[\text{True}] \tag{15.1}$$
>
> Suppose both metrics start at the same normalized value v, so the gap is zero. A ranking change that raises Clicks by 20 percent and lowers Satisfaction by 5 percent moves them to $1.20v$ and $0.95v$, opening a gap of $0.25v$. Every dashboard that reports Clicks shows a 20 percent win, while the goal the system exists to serve has moved the other way.
>
> **Systems insight**: Engineers cannot directly optimize an unobserved goal. They need defensible proxy validation. A randomized holdout, a small random slice of users served without the new ranking, can estimate the ranking's causal effect on the gap, but only when the holdout's measured outcome reflects the goal or a justified surrogate for it.

The alignment gap illustrates a failure originating on the algorithm axis, where the loss function optimizes a misspecified objective, so even a model that generalizes with zero test error drives

outcomes that conflict with organizational or societal goals. Distribution shift, by contrast, originates on the data axis. The model logic remains intact, but the input distribution departs from the training distribution. Both failures can be silent, but they suggest different first interventions: objective redesign for a misspecified target, or pipeline investigation and possible adaptation for shifted inputs. These remedies can be complementary when both mechanisms are present. Isolating the root cause is the job the D·A·M taxonomy already performs for performance bottlenecks, and perspective 15.3 reads the same three axes for responsibility failures.

↪ The performance-bottleneck form of the D·A·M diagnosis, from profiler symptom to binding axis, is collected for reference in Appendix A.

> **Systems Perspective 15.3: Diagnosing responsibility failures with D·A·M**
>
> When a system causes harm, use the D·A·M taxonomy to identify the root cause. Responsibility failures are rarely "algorithm bugs"; they are structural flaws along one of the three axes:
>
> - **Data (information)**: The training corpus reflects historical bias or unrepresentative sampling (such as Amazon's recruiting tool learning from biased historical applicant data).
> - **Algorithm (logic)**: The objective function optimizes an unconstrained proxy metric that amplifies harm (such as engagement optimization driving polarization).
> - **Machine (physics)**: The serving and governance infrastructure cannot see or bound the harm, through telemetry that never slices outcomes by subgroup, missing lineage and audit records, or an energy and carbon footprint that exceeds the societal benefit of deployment (such as an overparameterized foundation model serving a low-complexity task).
>
> Locating the dominant failure in the taxonomy identifies the first remediation to test: better curation (Data), safer objectives (Algorithm), or better instrumented and more efficient infrastructure (Machine). Real failures can span axes, so the intervention must still be evaluated end to end.

The D·A·M taxonomy isolates *where* failures originate; production telemetry must also catch *when* they manifest, and statistical failures (data-quality defects, distribution shift, and fairness violations) can persist for days or months behind green health checks (section 14.5). A fairness violation is the slowest of these to surface, because it is visible only on a subpopulation and only once outcomes are labeled.

A third mechanism is the feedback loop, in which model predictions actively reshape the user behavior that generates future training data. The YouTube recommendation feedback loop illustrates this dynamic at scale (M. H. Ribeiro et al. 2020).[2] M. H. Ribeiro et al. (2020) audited radicalization pathways on YouTube, finding migration from milder to more extreme channel categories and recommendation reachability between those categories. The broader systems lesson is that feedback loops can work exactly as designed while producing outcomes that conflict with societal values. Recommendation objectives must therefore be tested against downstream harms, not only against engagement proxies.

Facebook's 2018 change to News Feed ranking shows the converse decision. The team moved the ranking objective from short-term engagement proxies (clicks, watch time) toward meaningful social interactions between people, accepting lower short-term engagement to arrest the proxy's decoupling from user welfare (Mosseri 2018). The DLRM ranking service faces the same choice whenever its objective is set, because the clicks it can observe are a proxy for the value it is meant to deliver.

Silent failure modes change what verification can establish. Traditional software verification relies on deterministic assertions. A unit test asserts that an output matches an exact value, memory bounds checkers assert that buffer allocations do not overflow, and watchdog timers ensure RPC latencies do not breach service level agreements. In an ML system, correctness cannot be verified by deterministic invariants alone, because, as the credit service's form change showed, every such invariant can hold while the model serves biased, uncalibrated, or harmful predictions. What can be verified is the statistical bound of the verification gap (principle 9), which holds only over a named deployment population, task distance, and tolerance. A bound stated over the aggregate population

[2] **Goodhart's law in feedback loops**: Recommendation feedback loops are the canonical ML manifestation of Goodhart's law. Gradient descent optimizes watch-time proxies at a speed no human curator can match, and the system's own outputs reshape the training distribution. Users who consume extreme content generate data that can reinforce extremity and can decouple the proxy from user welfare faster than manual editorial processes would.

↩ The YouTube feedback loop as a production technical-debt pattern is established in section 14.2.6.1.

says nothing about a subgroup that the aggregate averages away, so a responsible specification names each consequential subgroup as a population in its own right. Turning responsibility goals into routine engineering practice means running statistical checks continuously: watching for covariate shift within each demographic slice, auditing how well proxies track the real goal, and bounding closed-loop feedback before silent errors spread through production.

15.2.3 The testing challenge

Suppose two engineers test the same credit model and agree that it should treat demographic groups fairly. One checks whether qualified applicants in both groups are approved at the same rate; the other checks whether a given risk score means the same repayment probability in both groups. When the two groups have different base repayment rates and prediction is imperfect, passing the first test forces the model to fail the second (Chouldechova 2017; Kleinberg et al. 2017). Traditional software testing never faces that bind because it checks behavior against unambiguous specifications: an arithmetic function must return the exact sum of its operands, and a relational database must preserve foreign-key integrity. Those deterministic properties map directly to executable assertions.

Responsible ML properties resist that kind of single assertion. **Fairness** has multiple mathematical definitions that can conflict under conditions such as unequal base rates and imperfect prediction (Chouldechova 2017; Kleinberg et al. 2017). What counts as fair depends on context, values, and trade-offs that technical systems cannot resolve alone. Individual fairness requires that similar individuals receive similar treatment, while group fairness requires equitable outcomes across demographic categories. These criteria can conflict, and choosing between them requires value judgments beyond the scope of optimization.

Fairness constraints and predictive performance can also trade off, by an amount that depends on the application. The Pareto frontier (principle 5) carries over unchanged, and section 15.3.2.2 measures one such trade-off once the criteria have been defined. Choosing criteria is only half of the testing problem. Aggregate metrics cannot show which groups a model fails, so testing also requires deciding which slices to evaluate, which invariances to demand, and which stresses to apply, choices that section 15.3.2 turns into release gates.

These testing choices complement traditional software testing rather than replacing it, and no single group can make them alone. Legal specialists cannot specify which demographic slices matter for a healthcare algorithm, and product managers cannot determine appropriate invariance tests for a loan model. Domain experts, affected stakeholders, policy specialists, and engineers must define the relevant harms together; engineers then encode those requirements as measurable tests and production controls. Responsibility therefore needs clear ownership within engineering as well as independent oversight outside it.

15.2.4 When responsible engineering succeeds

When the 2018 Gender Shades audit exposed error rates on darker-skinned women more than thirty times higher than on lighter-skinned men in commercial gender classifiers (Buolamwini and Gebru 2018), "be fairer" was not an instruction an engineering team could execute. Where teams have closed that gap, each documented success did the same thing: it turned a vague responsibility goal into an engineering constraint that can be specified, tested, communicated, and, when necessary, used to stop deployment. Following Gender Shades, Microsoft invested in improving gender-classification performance across demographic groups. Targeted data collection, model changes, and systematic disaggregated evaluation gave the team an explicit error-rate target, bringing audited error rates for darker-skinned subjects below 2 percent (Raji and Buolamwini 2019). Publishing these disaggregated benchmarks established a verifiable baseline and turned external audit findings into an operational release gate.

↩ Gender Shades, and the way aggregate benchmark metrics conceal subgroup disparities, are established in section 12.9.1.1.

Twitter's automatic image cropping system shows the same discipline under a different constraint. In 2020, users raised concerns about racial and gender bias in preview thumbnails. Twitter audited the system, reported measured disparities and limitations, and shifted toward uncropped previews that gave users more control (Yee et al. 2021). In that case, responsible engineering changed the product design rather than relying on an incremental adjustment to an automated ranking threshold.

Differential privacy applies the same move to privacy. It turns a privacy requirement into a mathematical bound on how much any one record can change a system's outputs, a bound that can

be budgeted and audited rather than asserted (Dwork 2008). Its mechanism and systems cost are developed with the privacy protections in section 15.5.2.

In each case the constraint sat on a D·A·M axis, whether in curated training data, a bounded loss function, or a privacy budget tracked in the runtime. Enforcing such constraints requires the operational authority to halt deployment or retire components that violate specified bounds, and that authority has to sit with the people who can still change the design.

15.2.5 Engineering leadership on responsibility

That authority belongs in engineering because a prelaunch review two weeks before release arrives after the cheap fixes have already been closed off. When a late review flags a model that cannot explain why it rejected a loan applicant or a pipeline that never recorded the demographic attributes needed to audit disparity, tuning a threshold cannot repair the design: correcting a flawed target means changing what the model predicts, and adding slice evaluation means rebuilding the data pipeline. Responsible AI engineering cannot be delegated exclusively to ethics boards or legal departments. These groups provide essential oversight, but they lack the early technical access required to catch problems while the architecture is still fluid.

Legal or ethics review can identify a problem near deployment, but it cannot recover design options the system has already foreclosed. If the team trained the model without fairness constraints, chose an architecture that cannot support interpretability requirements, or built a data pipeline without the demographic attributes needed for monitoring, review can only accept, reject, or demand expensive redesign. Engineers therefore occupy a critical position in the ML development lifecycle because their choices define the solution space for all subsequent interventions: architecture determines which fairness constraints can apply, the optimization objective determines which patterns the system learns, and the data pipeline determines whether disaggregated evaluation is possible.



Definition 15.1: Responsible AI engineering

Responsible AI engineering is the engineering discipline of designing, deploying, and maintaining systems with probabilistic outputs by turning societal and regulatory requirements into testable constraints on the D·A·M axes: permissible data contents, provenance, and composition; allowable model behavior and robustness properties; and infrastructure bounds such as latency, energy, compute budget, carbon emissions, and audit-log retention.

1. **Significance**: Each D·A·M axis acquires concrete governance constraints: the data axis is bounded by privacy regulations such as the General Data Protection Regulation (GDPR), which limits which records, fields, and features can be collected; the algorithm axis is bounded by fairness and robustness metrics (for example, positive prediction rates that differ across protected groups by no more than 5 percentage points, or accuracy that degrades by less than 2 percent when an adversary may shift every normalized input feature by at most 0.01); and the machine axis is bounded by resource and infrastructure budgets such as latency, energy per inference, carbon emissions, and audit-log retention. Violating these bounds is a system failure, not a research shortcoming.
2. **Distinction**: Unlike AI ethics (which articulates aspirational values), responsible AI engineering translates those values into measurable, testable invariants that can be verified through automated testing and continuous monitoring, using the same lifecycle practices that enforce latency SLOs.
3. **Common pitfall**: Stating a bound the system cannot measure. A disparity limit is an intention when the pipeline never collected the demographic attribute needed to check it, and a retention rule is an intention when no storage was provisioned to hold the evidence. Late-stage remediation may remain possible, but it is often narrower, slower, and more expensive.

An engineering-centered approach does not diminish the importance of diverse perspectives in identifying potential harms. Product managers, user researchers, affected communities, and policy experts contribute essential knowledge about how systems fail socially despite technical success.

Engineers translate these concerns into measurable requirements and testable properties that can be verified throughout the development lifecycle.

Organizations also need a reason to fund that work, and three reasons recur. Systematic disaggregated evaluation, documentation, and monitoring reduce the legal and financial exposure that ML system failures create, and they preserve evidence of the engineering process if problems emerge. Regulation attaches risk-management, data-governance, transparency, and human-oversight duties to high-risk applications, and controls built in from the start are cheaper than evidence reconstructed after deployment (section 15.5.3). Demonstrated practice also differentiates: organizations that can show model cards, audit trails, and published disaggregated results may qualify for deployments that competitors cannot. Each reason pays off only through repeatable workflows (checklists, documentation standards, and testing protocols) that embed responsibility constraints in the engineering lifecycle itself.

15.3 Responsible Engineering Across the Lifecycle

The recruiting tool and the care-management algorithm were both found to fail after the design options that could have addressed them were closed, so the lesson concerns timing more than vigilance. Each responsibility failure in this chapter has a point in the ML lifecycle before which it is cheap to prevent and after which it becomes expensive or impossible to repair, so responsible engineering is organized around that lifecycle rather than around a list of concerns.

Six stages follow from that premise, and the first four form a chain in which each stage depends on the output of the one before it. *Specify* comes first because a system cannot be tested against a requirement nobody wrote down; the target, the populations that matter, the evaluation data those populations demand, and the boundary of human oversight must all be fixed before training forecloses the alternatives. *Measure* turns those written requirements into release gates, evaluating each named population separately and quantifying the trade-offs among fairness criteria. Evidence that lives only in an engineer's notebook cannot be contested, audited, or reused, so *Explain and document* makes individual decisions legible to the people they affect and whole models legible to the teams that inherit them. The documented operating envelope then tells *Operate* what to watch after launch, when feedback loops and distribution shift erode a model that passed every gate. The last two stages span the whole lifecycle rather than following it, so each receives its own section, while the four chained stages structure the subsections of this one, each worked on the credit service. *Resource* accounts for the energy, carbon, and money that every stage consumes (section 15.4), and *Govern* supplies the regulatory obligations and the data infrastructure of access control, privacy, lineage, and audit that make each earlier stage provable to someone outside the team that built it (section 15.5). The four chained stages and the two that span them form one loop (figure 15.1).

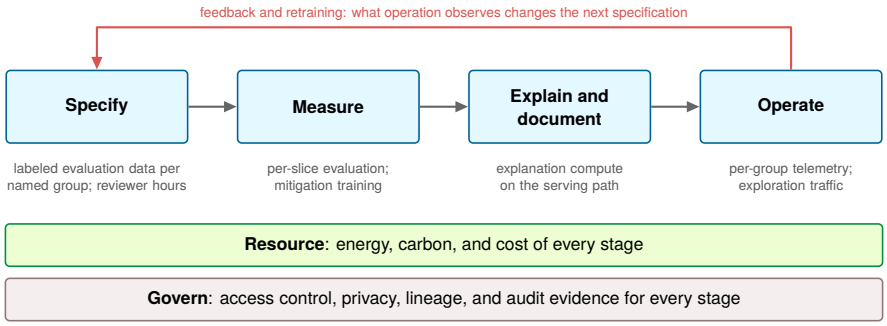


Figure 15.1: The Responsible Engineering Lifecycle: Four chained stages carry the credit service from requirement to production. Specify fixes the target, the named populations, and the oversight boundary; Measure turns them into release gates; Explain and document makes decisions and models legible; Operate watches the deployed model and feeds what it observes back into the next specification. Resource and Govern span every stage: each stage consumes accelerator-hours, energy, and money, and each must leave evidence that someone outside the team can audit. The machine cost under each stage names what the remedy consumes.

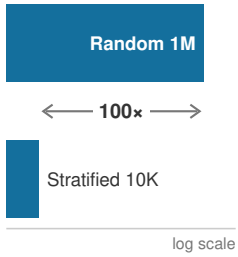
15.3.1 Specify: Requirements before training

The proxy variable trap (war story 15.1) was a specification failure. The care-management model predicted exactly what it was asked to predict, and the request was wrong. Specification is the stage with the most leverage and the least visibility, because its errors surface only after the model has learned them. Before the credit service reaches production, its team must therefore determine the provenance of the training data, identify who is represented and who is missing, anticipate failure modes, and define recourse for affected users. Table 15.2 organizes these questions by pipeline phase, distinguishing critical-path blockers from high-priority items that can proceed with documented risk acceptance. Its rows follow the pipeline rather than the six stages because specification must anticipate every phase in advance. The Evaluation row fixes what *Measure* will gate, and the Deployment and Monitoring rows fix what *Operate* will watch and how it will respond.

Table 15.2: Predeployment Assessment Framework: Critical Path items block deployment until addressed. High Priority refinements may follow launch under documented risk acceptance; a specified objective and tested monitoring and response paths remain prerequisites. Systematic coverage of responsibility concerns throughout the ML lifecycle reduces the chance that risks are overlooked.

Phase	Priority	Key Questions	Documentation Required
Data	Critical Path	Where did this data come from? Who is represented? Who is missing? What historical biases might be encoded?	Data provenance records, demographic composition analysis, collection methodology documentation
Training	High Priority	What is the system optimizing for? What does the loss implicitly penalize? How do architecture choices affect outcomes?	Objective function specification, regularization choices, hyperparameter selection rationale
Evaluation	Critical Path	Does performance hold across different user groups? What edge cases exist? How were test sets constructed?	Disaggregated metrics by demographic group, edge case testing results, test set composition analysis
Deployment	Critical Path	Who will this system affect? What happens when it fails? What recourse do affected users have?	Impact assessment, stakeholder identification, rollback procedures, user notification protocols
Monitoring	High Priority	How are anomalies detected? Who reviews system behavior? What triggers intervention?	Monitoring dashboard specifications, alert thresholds, review schedules, escalation procedures

The Training and Monitoring rows mix release prerequisites with later refinements: teams must specify the objective and test the minimum monitoring and response controls before launch, while the remaining refinements may proceed with documented risk acceptance and a remediation timeline. The Evaluation row in table 15.2 belongs to specification rather than to testing because naming the groups that matter also fixes a data budget. Showing that performance holds for each group requires a statistically valid test set per group, and the cost of that set grows with the group's rarity.



Random sampling needs 100× more images to reach a 1 percent subgroup.

Napkin Math 15.1: The statistics of representation

Problem: An engineering team needs to verify that a Face ID model works for a minority group representing 1 percent of the user base. The group's error rate must be known to within 1 percentage point at 95 percent confidence. The binomial standard error behind the statistical confidence trap of section 12.8.1 is largest at an error rate of 0.5, so the worst-case margin of error on n images is $1.96\sqrt{0.25/n}$, which falls to one percentage point near n = 10,000 images for this group.

Random sampling: To get 10,000 images of a 1 percent group in expectation via independent random sampling, the team must collect and label: $D_{\text{eval,total}}$ = 10,000 images / 0.01 = 1,000,000 images

Stratified sampling: Recruiting a representative, independently sampled evaluation set within this group requires 10,000 images.

Systems insight: Under these assumptions, natural-population sampling requires 100× as much total collection as direct subgroup recruitment, in expectation rather than as a guaranteed

quota. Targeted recruitment must still represent the deployment subgroup; active selection of difficult cases cannot be treated as an unbiased evaluation sample without accounting for its sampling design. Fairness requires intentional data engineering rather than uncurated volume.

The representation cost derived in napkin math 15.1 settles what the evaluation must contain, but even a representative evaluation cannot prevent harm at deployment when human oversight is inadequate. The Deployment row of table 15.2 forces a second decision that cannot be deferred to launch: where people sit in the decision loop once the model is live, and what that arrangement must be designed to withstand.

War Story 15.2: The automation paradox (2018)

Context: Uber's Advanced Technologies Group (ATG) was testing self-driving cars in Arizona, relying on a human safety driver to intervene if the autonomous perception system failed (National Transportation Safety Board 2019).

Mechanism: The perception system detected a pedestrian crossing the road but repeatedly toggled its classification between vehicle, bicycle, and unknown, resetting its trajectory prediction while automatic emergency braking was intentionally disabled.

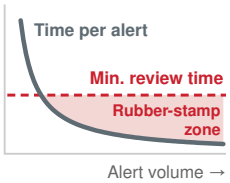
Impact: The safety driver was visually distracted by a personal phone and failed to take control, resulting in a fatal collision.

Fix: Uber retrofitted the fleet with driver-monitoring cameras, re-enabled automatic emergency braking, and established dual-operator staffing during autonomous road testing.

Systems lesson: Adding a human backup creates a new system with its own failure modes. High automation reliability can encourage complacency and reduce vigilance, so an effective fallback requires workload design, attention monitoring, and a safety case for the combined human-machine system.

For high-stakes applications, the specification should state where human oversight is required. Human-in-the-loop (HITL) systems route uncertain, high-consequence, or flagged decisions to human reviewers rather than acting autonomously; the clinician approval gates of ClinAIOps (section 14.9.2) are one proposed design. Effective HITL design must specify four requirements: the review scope (which decisions require human review), the confidence thresholds that trigger escalation, the training reviewers receive, and the mechanisms for monitoring reviewer performance. HITL is not a catch-all solution. Human reviewers can rubber-stamp automated decisions, introduce their own biases, or become overwhelmed by alert volume, so the human-machine boundary must be calibrated to the application's risks and the reviewers' capabilities.

Like an aviation preflight checklist, the assessment standardizes the questions a team must answer under schedule pressure, but answering them does not show that the system is safe. That evidence comes from measuring the model against the populations the specification named, which turns each written requirement into a gate the release must pass.



If reviewer hours stay fixed, rising alert volume eventually leaves too little time per alert for real review.

↩ The ClinAIOps clinician oversight loop, in which a clinician approves, modifies, or rejects each therapy recommendation, is established in section 14.9.2.1.

15.3.2 Measure: Fairness criteria as release gates

Turning the populations that specification named into release gates is what catches disparities that an aggregate metric hides: a commercial face classifier can post 95 percent overall accuracy while misclassifying more than a third of darker-skinned women when that group is a small share of the test set. Measurement selects the slices, metrics, and thresholds that decide whether the credit service may deploy, and responsibility properties become testable once engineers and stakeholders define criteria appropriate to the application and measure performance per subgroup. The Gender Shades audit,[3] introduced in section 15.2.4, demonstrated how **disaggregated evaluation** across demographic categories reveals disparities invisible in aggregate metrics (Buolamwini and Gebru 2018), exposing the subgroup failures that responsibility monitoring must catch before deployment. Table 15.3 shows the error-rate differences that commercial gender-classification systems produced across demographic groups. Measuring such slices carries the representation cost that specification

[3] **Gender Shades**: A 2018 study by Joy Buolamwini (MIT Media Lab) and Timnit Gebru (Microsoft Research) that audited commercial gender-classification systems from Microsoft, IBM, and Face++ using the Fitzpatrick skin type scale, a dermatological classification developed in 1975 for UV sensitivity (Fitzpatrick 1988) and repurposed here as a demographic benchmark for algorithmic auditing.

budgeted (napkin math 15.1), because small subgroups contribute few samples to a randomly drawn test set.

Table 15.3: Gender Shades Face++ Error Rates: The dark-female error rate is 43.1× the light-male rate. Source: (Buolamwini and Gebru 2018).

Demographic Group	Error Rate (%)	Relative to Light-Skinned Males
Light-skinned males	0.8%	Baseline (1.0×)
Light-skinned females	9.8%	12.2×
Dark-skinned males	0.7%	0.9×
Dark-skinned females	34.5%	43.1×

As table 15.3 quantifies, disaggregated evaluation revealed what an aggregate score concealed. The audit measured Face++ error rates of 0.8 percent for light-skinned males and 34.5 percent for dark-skinned females (accuracies of 99.2 percent and 65.5 percent). The aggregate metric gave no indication that the dark-female error rate was 43.1× the light-male rate.

No universal threshold defines acceptable disparity, but teams should establish explicit, justified bounds before deployment. A team might adopt an error-rate ratio below 1.25× or a false-positive-rate difference under 5 percentage points as an application-specific release policy. In US employment law, the disparate impact doctrine[4] shapes regulatory oversight, while the four-fifths rule[5] provides a separate, non-dispositive selection-rate diagnostic. Criteria must fit the application and its governing law, and one domain's threshold should not be borrowed for another.



Systems Perspective 15.4: The flaw of averages

A bridge that is "safe on average" but collapses under a heavy truck illustrates the **flaw of averages** (Savage 2009). An ML system that is "accurate on average" but fails for a specific demographic cohort presents the same design mistake. As with tail latency (p99), a release gate must test the conditions its aggregate metric conceals; disaggregated evaluation makes cohort-specific failures visible.

Selecting those conditions requires application knowledge. A vision classifier fails differently from a recommendation engine or an audio wake-word detector, so fairness metrics and evaluation slices must reflect the application domain. In healthcare diagnostics, demographic cohorts such as age and biological sex govern slice definitions. In content moderation, dialect and linguistic context dominate. In financial underwriting, credit regulations mandate auditing legally protected categories.



Lighthouse 15.1: Fairness concerns by lighthouse

The lighthouse workloads of section 2.3.2 were defined by the resource that binds each one, but their fairness risks come from somewhere else, from the data each learns from and the context each serves. Table 15.4 maps each lighthouse's data source and deployment context to a primary risk and metric.

Architecture alone does not determine fairness; each row identifies a deployment-specific risk to test first, not a universal property of the model family or its bottleneck.

[4] **Disparate impact**: In *Griggs v. Duke Power Co.* (1971), the US Supreme Court held under Title VII that an employment practice may be unlawful because of its operation even without discriminatory intent (Supreme Court of the United States 1971). Disparate impact is a legal claim with statute- and context-specific elements, not a synonym for any statistical disparity. Model outcomes and proxies can supply relevant evidence, but metrics alone do not establish liability.

[5] **Four-fifths rule**: The 1978 Uniform Guidelines on Employee Selection Procedures use the four-fifths rule as a practical employment-selection diagnostic (Equal Employment Opportunity Commission et al. 1978). A selection rate below 80 percent of the rate for the group with the highest rate is generally regarded as evidence of adverse impact, but the Guidelines also recognize that smaller differences may matter and that the ratio is not dispositive.

Table 15.4: Fairness Risk by Lighthouse: Fairness risks follow each lighthouse's data source and deployment context, not the resource that binds it.

Lighthouse (Home Regime)	Primary Fairness Risk	Key Evaluation Metric	Real-World Example
ResNet-50 (compute-bound)	Training data bias (underrepresentation of deployment-relevant groups)	Disaggregated accuracy by demographic group	Commercial face classifiers erred most on darker-skinned women (Gender Shades audited those systems, not ResNet-50)
GPT-2 (bandwidth-bound)	Corpus bias (overrepresentation of majority viewpoints in web text)	Toxicity rate by demographic prompt context; stereotype score	LLMs produce more toxic completions for prompts mentioning minority groups
DLRM (capacity-bound)	Feedback-loop amplification (popular items get more data)	Share of recommendation impressions by item category and supplier or creator group	Filter bubbles: the system recommends similar content to similar users, reducing discovery of niche creators
KWS DS-CNN (power-bound)	Deployment-context mismatch (trained on clean audio, deployed in noisy real-world environments)	False positive rate by acoustic environment and speaker accent	Voice assistants perform worse on accented speech; wake-word triggers on TV audio in some languages

Systems insight: Fairness evaluation must match each lighthouse's failure mode. Vision models require stratified demographic accuracy; large language models (LLMs) need toxicity and stereotype probes; recommenders need exposure audits; TinyML needs acoustic-environment tests. The KWS lighthouse faces exactly this challenge for its depthwise-separable convolutional neural network (DS-CNN). Trained on clean studio audio, it must perform equitably across accents, background noise levels, and speaker demographics in production homes, so its release gate needs accent and acoustic-environment slices, each reported with its own confidence interval.

Once the application has fixed the slices, three verification regimes expose latent responsibility failures that a uniform validation set hides. Slice-based evaluation partitions evaluation data into demographic and operational cohorts, computing metrics independently for each slice. A model may achieve 95 percent accuracy overall while dropping to 78 percent on low-income applicants or rural cohorts, a disparity that aggregate reporting never shows.

Invariance testing checks that model decisions stay unchanged when you swap features that should not matter. Swapping candidate names such as "John" and "Jamal" in a loan application must not alter approval probabilities when demographic features are legally and technically irrelevant to creditworthiness. Behavioral testing frameworks such as CheckList put this principle to work by organizing validation suites around behavioral capabilities and input swaps rather than a single static validation loss (M. T. Ribeiro et al. 2020).

Boundary and stress testing probe input regimes where empirical validation data is sparse or unrepresentative. Boundary evaluation maps behavior at the edges of the data distribution (extreme ages, rare transaction amounts, or sparse geographic categories), where prediction variance climbs sharply. Stress testing extends these margins into adversarial conditions: corrupted feature pipelines, sudden covariate shift, and synthetic edge cases designed to test safety margins under degradation. Stakeholder red-teaming complements automated suites by introducing adversarial domain probes from affected communities, surfacing failure modes that statistical benchmarks overlook.

Running these evaluations requires four linked capabilities in the testing infrastructure. It begins with stratified evaluation, which computes performance metrics separately for each cohort to expose error-rate disparities across populations, and extends to intersectional analysis across combinations of attributes so that compounding failure modes invisible in single-factor slices are caught as well. Because partitioning the test set into finer intersections leaves each slice with fewer examples, the harness must attach confidence intervals to every point estimate; the binomial resolution limit

that the statistical confidence trap of section 12.8.1 exposed for an aggregate test set binds hardest here, where each slice holds only a fraction of the evaluation data. Once the model ships, temporal monitoring carries the same cohort checks into production, catching distribution drift that degrades minority slices before it registers on aggregate health checks.

Open-source evaluation suites such as Fairlearn (Bird et al. 2020), AI Fairness 360 (Bellamy et al. 2019), and the What-If Tool (Wexler et al. 2020) compute stratified and intersectional metrics once the team has specified the metrics, slices, and alert thresholds. They compute whichever metric an engineer queries; they cannot decide which demographic boundaries matter, calibrate paging thresholds for on-call engineers, or select the legally appropriate fairness constraint.

An operational release gate must distinguish absence of evidence from evidence of parity. If a slice lacks sufficient labeled validation data to bound the error rate with statistical confidence, the CI/CD gate must report insufficient data rather than a false pass, which triggers targeted data acquisition or restricts deployment scope. Every population test therefore returns one of three results (pass, fail, or insufficient evidence), and only a pass permits release.

15.3.2.1 Worked example: Fairness analysis of the credit service

The credit service, the chapter's labeled composite, reports 85 percent accuracy on the majority group and 82.5 percent overall accuracy across the evaluated applicants, numbers that may satisfy a coarse aggregate dashboard. Evaluating the model separately across demographic cohorts exposes how aggregate metrics conceal severe disparities. This constructed evaluation assumes observed repayment outcomes for every applicant, independent of the model's approval predictions; those predictions are not the lending actions that produced the labels. The two cohorts' outcomes appear side by side, the majority cohort (Group A) in table 15.5 and the minority cohort (Group B) in table 15.6.

Table 15.5: Confusion Matrix for Group A (Majority): In this constructed scenario, loan approval outcomes for 10,000 applicants from the majority demographic group. The 90 percent true positive rate (4,500 approved of 5,000 qualified) and 20 percent false positive rate establish the baseline for fairness comparison.

	Approved (pred)	**Rejected (pred)**
Repaid (actual)	4,500 (TP)	500 (FN)
Defaulted (actual)	1,000 (FP)	4,000 (TN)

Table 15.6: Confusion Matrix for Group B (Minority): In the same constructed scenario, loan approval outcomes for 2,000 applicants from the minority demographic group. The 60 percent true positive rate (600 approved of 1,000 qualified) reveals a 30 percentage-point disparity compared with Group A; the confusion matrices alone do not identify its cause.

	Approved (pred)	**Rejected (pred)**
Repaid (actual)	600 (TP)	400 (FN)
Defaulted (actual)	200 (FP)	800 (TN)

Every rate in the calculations that follow can be reconstructed from the approved/rejected and repaid/defaulted cells of these two matrices. Three standard criteria for equitable treatment capture distinct operational objectives, and the credit service's two matrices violate all three.

Demographic parity (Dwork et al. 2012) requires equal positive selection rates across groups regardless of underlying qualification ($\Pr(\hat{Y}=1 \mid G=\text{A}) = \Pr(\hat{Y}=1 \mid G=\text{B})$). Group A receives approval at $(4{,}500+1{,}000)/10{,}000 = 55\%$, whereas Group B receives approval at $(600+200)/2{,}000 = 40\%$. The resulting 15 percentage-point disparity indicates unequal selection outcomes between cohorts.

Equal opportunity requires equal true positive rates among qualified applicants ($\Pr(\hat{Y}=1 \mid Y=1, G=\text{A}) = \Pr(\hat{Y}=1 \mid Y=1, G=\text{B})$). Group A achieves a TPR of $4{,}500/(4{,}500+500) = 90\%$, meaning 90 percent of creditworthy applicants secure loans. Group B achieves only $600/(600+400) = 60\%$. This 30 percentage-point disparity reveals that qualified applicants in Group B suffer disproportionate rejection relative to equally qualified peers in Group A.

Equalized odds tightens the constraint, demanding parity in both true positive and false positive rates ($\Pr(\hat{Y}=1 \mid Y=v, G=\text{A}) = \Pr(\hat{Y}=1 \mid Y=v, G=\text{B})$ for $v \in \{0,1\}$).[6] Neither group may then bear a disproportionate share of unjustified rejections (false negatives) or unjustified approvals (false positives). Group A yields an FPR of $1{,}000/(1{,}000+4{,}000) = 20\%$, matching Group B at $200/(200+800) = 20\%$. Although false positive rates align, the substantial true positive rate deficit violates equalized odds.

Both groups in this scenario have the same base rate (5,000 of 10,000 and 1,000 of 2,000 applicants repaid), so the impossibility results for unequal base rates do not bind here. Remediation remains possible in principle, but these matrices do not identify the cause of the disparity or which intervention would close it. Those results involve a fourth criterion, calibration within groups, which applies the probability calibration that evaluation already checks to each group separately: among applicants in group g who receive score s, the fraction who repay should be s, that is, $\Pr(Y=1 \mid \text{score}=s, G=g) = s$ for every group. Real populations often have unequal base rates, and with unequal base rates and imperfect prediction, calibration-style criteria and error-rate balance can conflict (Chouldechova 2017; Kleinberg et al. 2017). A score calibrated within each group then generally assigns the groups different false positive or false negative rates, so the operational guarantee must be selected and defended for the application rather than deduced from aggregate accuracy or calibration.

These metrics disagree because they encode different policy choices about which error rates matter most. In this scenario, the model rejects qualified applicants from Group B at a much higher rate (40 percent false negative rate vs. 10 percent) while maintaining similar false positive rates. These confusion matrices establish an operational disparity, not whether it arose from classification thresholds, labeling bias, omitted features, sampling skew, or historical discrimination. The COMPAS risk assessment system[7] shows the same disagreement in a deployed system whose groups had unequal base rates.

6 **Equalized odds**: Formalized by Hardt et al. (2016), requiring that both TPR and FPR be equal across protected groups; the weaker "equal opportunity" relaxes this to TPR alone. Equalized-odds postprocessing may require randomized, group-dependent decisions rather than deterministic thresholds alone. It also requires protected-group information at decision time and must be evaluated under applicable law; for example, US credit regulation generally restricts using protected bases to assess creditworthiness.

7 **COMPAS**: Correctional Offender Management Profiling for Alternative Sanctions.

War Story 15.3: The COMPAS recidivism algorithm audit (2016)

Context: COMPAS is a risk assessment tool used in US courtrooms to predict re-offending for bail and sentencing decisions (Angwin et al. 2016).

Mechanism: The audited scores combined approximate calibration across racial groups (a given score corresponded to about the same observed re-offense rate in every group) with unequal error rates. These observations expose a policy choice about acceptable errors; they do not establish which objective the original optimizer used.

Impact: Black defendants who did not re-offend were incorrectly flagged as high-risk at nearly twice the rate of White defendants (44.9 percent vs. 23.5 percent), while White recidivists were far more often mislabeled low-risk.

Response: The audit exposed why calibration alone could not settle whether the error distribution was acceptable. Any jurisdiction using such scores needs an explicit policy for which fairness criteria matter, independent validation across groups, and review of how the score enters the decision workflow.

Systems lesson: The scores were approximately calibrated by race but violated equalized odds, the criterion that requires equal true positive and false positive rates across groups. The credit service's confusion matrices show the error-rate side of the conflict, and COMPAS adds the calibration side, which is why the team must explicitly choose which fairness constraint governs.

Where collection and use of protected attributes are lawful and justified, production systems can automate these calculations and trigger review when disparities exceed predefined thresholds. Listing 15.1 handles the failure case explicitly, reporting undefined group rates as insufficient data rather than comparing them against a threshold. These denominator checks establish arithmetic validity only. A production release gate also needs minimum labeled sample counts and uncertainty checks for each relevant slice; a defined rate from one observation is not sufficient evidence.

Listing 15.1: Automated Fairness Monitoring: The core pattern computes per-group metrics from confusion matrices and alerts when disparities exceed application-specific thresholds. Production use requires sufficient data and lawful, justified handling of group attributes.

```
def compute_fairness_metrics(confusion_matrix):
    tp, fp, tn, fn = (
        confusion_matrix[k] for k in ["TP", "FP", "TN", "FN"]
    )
    total = tp + fp + tn + fn
    return {
        # Demographic parity
        "approval_rate": (tp + fp) / total if total else None,
        # Equal opportunity
        "tpr": tp / (tp + fn) if (tp + fn) else None,
        # Equalized odds (with TPR)
        "fpr": fp / (fp + tn) if (fp + tn) else None,
    }


# Compare groups and flag disparities exceeding threshold
for metric in ["approval_rate", "tpr", "fpr"]:
    if metrics_a[metric] is None or metrics_b[metric] is None:
        report_insufficient_data(metric)
        continue
    disparity = abs(metrics_a[metric] - metrics_b[metric])
    # e.g., 0.05 for high-stakes applications
    if disparity > FAIRNESS_THRESHOLD:
        trigger_alert(metric, disparity)
```

Automated monitoring tracks fairness metrics repeatedly at a scale manual auditing cannot reach and alerts when sufficient labeled evidence reveals a disparity. Label availability and the time needed to accumulate observations bound detection speed. The 30 percentage-point TPR disparity far exceeds this scenario's 5-percentage-point release threshold, indicating the model requires fairness intervention before deployment. Table 15.7 summarizes the computed metrics and resulting disparities across both cohorts.

Table 15.7: Fairness Metrics Summary: In the constructed credit-service scenario, approval rate and true positive rate differ across groups, while false positive rates match; the table establishes disparity but not its cause.

Metric	Group A	Group B	Disparity
Approval Rate	55%	40%	15 pp
True Positive Rate	90%	60%	30 pp
False Positive Rate	20%	20%	0 pp

Shared classification thresholds applied to differing score distributions illustrate how aggregate performance masks severe subgroup divergence (Barocas and Selbst 2016). Shifting between the two candidate decision boundaries (figure 15.2) lifts overall accuracy from 75 percent to 81.25 percent by achieving perfect separation in Subgroup A, yet simultaneously triples the misclassifications in Subgroup B. Because the two subgroups' scores are distributed differently, one shared boundary cannot sit at the best point for both, and the move that raises the global metric does so by trading away Subgroup B's outcomes. Aggregate accuracy alone cannot determine whether those outcomes are equitable for either cohort.

Mitigating these disparities means intervening before, during, or after training, and each stage imposes its own systems trade-offs. Preprocessing via sample reweighting fixes a weight for every training instance before training begins; the training loss then scales each Group B instance's penalty by its weight, amplifying gradient signal for underrepresented cohorts without discarding data.[8] The trade-off appears across the broader loss surface, where reweighting can degrade accuracy on majority slices.

[8] **Reweighting**: The technique descends from importance sampling in statistics, which corrects an estimate drawn from one distribution by weighting each sample toward a target distribution. Kamiran and Calders (2012) showed that appropriately chosen weights can reduce disparate impact from training data.

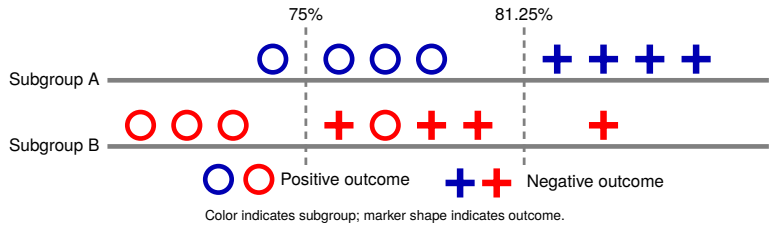


Figure 15.2: Threshold Effects on Subgroup Outcomes: Two candidate classification thresholds intersect different score distributions for Subgroups A and B. Circles mark positive outcomes (loan repayment), plus markers mark negative outcomes (default), and color distinguishes subgroup; each boundary approves the markers to its left. Each dashed line carries the overall accuracy that boundary achieves, not a score cutoff. Moving to the higher-accuracy boundary lifts overall accuracy from 75 percent to 81.25 percent, but the whole gain falls in Subgroup A, which becomes perfectly separated, while Subgroup B goes from one misclassification to three.

In-processing via adversarial debiasing turns the training loop into a minimax game between two neural networks. A primary model (encoder and predictor) transforms input features $\mathbf{x}$ (such as qualifications or income) into an intermediate latent representation $\mathbf{z}$, which a task head uses to predict the target label $\hat{y}$ under a loss $\mathcal{L}_{\text{task}}(\hat{y}, y)$ rewarding high predictive accuracy. Concurrently, an adversary head taps directly into the latent vector $\mathbf{z}$ and attempts to predict the protected attribute a (such as demographic group) by minimizing an adversarial loss $\mathcal{L}_{\text{adv}}(\hat{a}, a)$. During backpropagation, the adversary's gradient is negated as it flows into the shared feature encoder:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} - \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}$$

Here λ_{adv} weights the adversarial penalty against the task loss. The adversary head minimizes $\mathcal{L}_{\text{adv}}$ alone, trying to recover a from $\mathbf{z}$, while the encoder and task head minimize $\mathcal{L}_{\text{total}}$, so the encoder lowers its objective only by making the adversary's task harder. It is therefore forced to purge latent features that correlate with the protected attribute, producing representations that remain informative for the task while suppressing, not eliminating, protected-attribute signal under the evaluated distribution.[9] The systems cost is real: a second (adversary) head to train, adversarial gradients that destabilize convergence, and a larger hyperparameter search.

Postprocessing via threshold adjustment decouples the decision boundary across groups, lowering the approval cutoff for Group B to equalize TPR. The operational trade-off is an increase in false positives within that cohort, admitting applicants who may subsequently default. The lever is also legally constrained. In the US, group-specific cutoffs are prohibited in employment selection and restricted in credit, so this intervention often exists only in principle, and teams move instead to reweighting, feature and label repair, or objective changes.

Selecting among these interventions requires stakeholder alignment on acceptable trade-offs for the specific deployment domain. Whichever intervention is chosen, its trade-off should be stated as a number that stakeholders can argue about.

> **Checkpoint 15.2: Fairness criteria**
>
> Fairness is not a single metric; it is a constrained design choice.
>
> - ☐ **Metric definitions**: can you distinguish demographic parity, equal opportunity, equalized odds, and calibration in terms of which rates must match across groups?
> - ☐ **Impossibility trade-off**: can you explain (at a high level) why base-rate differences make it impossible to satisfy all fairness criteria simultaneously?
> - ☐ **Systems interpretation**: given the confusion matrices in table 15.5 and table 15.6, can you identify which disparity matters operationally (TPR vs. FPR vs. approval rate) and what kind of harm it represents?
> - ☐ **Engineering decision**: for a concrete high-stakes domain (credit, hiring, criminal justice), can you justify which fairness constraint you would prioritize and why?

[9] **Adversarial debiasing**: The key differentiating property is representation pressure: the adversary discourages the primary model from encoding protected-attribute information, which can reduce protected-attribute leakage and help satisfy selected fairness criteria under the evaluated distribution (B. H. Zhang et al. 2018). It does not provide a general fairness guarantee under arbitrary deployment shift, because any guarantee depends on assumptions about invariance, labels, causal structure, and the type of shift. Post-processing methods such as threshold adjustment may also be appropriate under different assumptions but must be revalidated when deployment demographics or label processes change. The cost is additional training, hyperparameter tuning, and slice-level validation rather than a universal percentage overhead.

15.3.2.2 Quantifying the fairness-accuracy trade-off

The worked example showed that fairness criteria can conflict. A separate question is how a chosen engineering constraint changes predictive utility. The Pareto frontier (principle 5) collects the configurations for which one plotted objective cannot improve without degrading another. Figure 15.3 plots a hypothetical fairness-accuracy Pareto frontier against demographic disparity, measured as the difference between two groups' positive-prediction (approval) rates, the demographic parity gap defined in section 15.3.2.1. The disparity axis is inverted so that fairer configurations lie to the right. The figure contrasts an unconstrained maximum-accuracy operating point (A) against a balanced compromise (B) and a steep accuracy collapse under strict demographic equality (C). These annotated configurations illustrate candidate policy choices along the frontier; they do not imply that unconstrained training always maximizes accuracy at high disparity, that zero disparity must reduce accuracy, or that every application has a sweet spot.

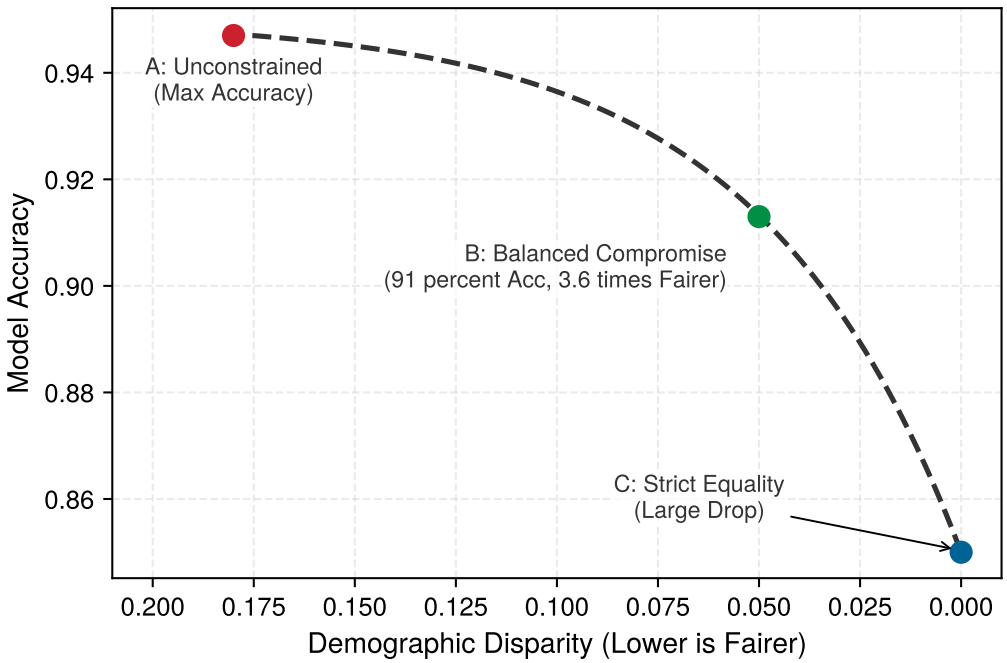


Figure 15.3: An Illustrative Fairness-Accuracy Pareto Frontier: A hypothetical relationship between model accuracy and demographic disparity. The x-axis is inverted so lower disparity lies to the right. Points A, B, and C illustrate three possible operating choices along this constructed frontier; B's label of 3.6 times fairer means its disparity (0.05) is 3.6 times smaller than A's (0.18). The curve is not an empirical or universal law.

The hypothetical frontier in figure 15.3 illustrates possible operating choices from A toward C; the deployment's measured candidate configurations, rather than that illustrative curve, determine its actual trade-offs. Quantifying this trade-off requires evaluating economic costs for the specific application. The credit service of section 15.3.2.1 makes the resulting penalty concrete because each lending outcome carries an explicit value, the margin on a repaid loan on one side and the loss on a default on the other.

Napkin Math 15.2: The price of fairness

Problem: Stakeholders demand elimination of the credit service's 30 percentage-point true positive rate (TPR) disparity. What is the "Price of Fairness" in lending outcomes?

Physics: TPRs can be equalized by adjusting the classification threshold (γ_{cls}) for the disadvantaged group.

- **Original state**: Group A (TPR = 90 percent), Group B (TPR = 60 percent, FPR = 20 percent).
- **Intervention**: Lower $\gamma_{\text{cls},g=\text{B}}$ until $\text{TPR}_{g=\text{B}} = 90$ percent. (US credit law restricts group-specific cutoffs, as the mitigation discussion noted, so the calculation quantifies the trade-off rather than a deployable intervention.)
- **The cost**: Lowering the threshold increases false positives (approving applicants who later default).

Math:

1. Under the scenario's assumed threshold response, closing the 30 percentage-point TPR gap produces a 20 percentage-point increase in false positives for the disadvantaged group.
2. If 50 percent of Group B applicants would repay, a repaid loan earns a margin of \$1,500, and a default loses \$3,000, both sides of the intervention must be counted:
   - Group B baseline utility evaluates the original expected payoff: $0.60 \times 0.50 \times \$1{,}500 - 0.20 \times 0.50 \times \$3{,}000 = \$150$ per applicant.
   - Evaluating the intervention requires counting both marginal terms: $\Delta\text{Utility} = \Delta\text{TPR} \times \text{Base Rate} \times \text{Loan Margin} - \Delta\text{FPR} \times (1 - \text{Base Rate}) \times \text{Default Loss}$.
   - The added true-positive value is \$225 per Group B applicant, while the added false-positive cost is \$300, producing a net loss of \$75.
   - Dividing this \$75 net loss by the \$150 baseline yields a 50 percent within-group utility loss. With Group B at 17 percent of applicants, the population-average loss is \$12.50 per applicant; calculating an aggregate percentage also requires the other group's baseline utility.

Systems insight: The "Price of Fairness", the fairness literature's name for the utility a system gives up to satisfy a fairness constraint, in this scenario is a 50 percent within-group utility loss, not a derivable aggregate percentage. The loss is not automatic: when a TPR gap reflects a miscalibrated threshold, closing it can raise net utility. It appears when the baseline threshold is near the utility optimum and the marginal approvals skew toward default. Engineering teams owe stakeholders both the Pareto frontier and the operational assumptions that produced it.

The calculation gives stakeholders a way to choose a point on the fairness-accuracy frontier, but it still does not explain any particular decision. When a credit-service applicant receives a rejection, stating that "the model's true positive rate for this demographic group is 60 percent compared to 90 percent for other groups" provides no actionable information. The applicant needs to know *why* the application was rejected and *what* could be changed. These questions require explainability, which is the ability to articulate which input features drove specific predictions.

15.3.3 Explain and document: Decisions and models made legible

When the credit service rejects a borrower with a bare probability score of 0.41, the applicant cannot tell whether the denial came from a high debt-to-income ratio or a corrupted credit-bureau field; six months later, when another team reuses that same classifier to screen rental tenants without knowing which populations it was tested on, the organization inherits a second failure at the model level. Explanation and documentation answer that single demand for legibility at two timescales. An explanation makes one decision understandable, at the moment it is made, to the person it affects. Documentation makes the model understandable across its lifetime to the engineers, auditors, and downstream teams who inherit it: what it was trained on, where it was validated, and what it must not be used for. Both exist for the same systems reason, since a guarantee that no one outside the team can inspect cannot be contested, audited, or safely reused.

15.3.3.1 Explainability requirements

An applicant whom the credit service denies may be entitled under applicable law to reasons or specific adverse-action factors, not merely aggregate statistics. **Explainability**[10] supports this

[10] **Explainability and interpretability**: *Interpretability* usually describes how readily a person can understand a model or its behavior, often through a constrained structure such as a short rule list or sparse linear model. *Explainability* is broader and includes post-hoc methods such as LIME and SHAP. Neither property is automatic: a linear model with opaque features can be difficult to interpret, and a feature attribution is not a causal account of a decision. The systems implication is that intrinsic constraints affect model selection, while post-hoc methods add a computation and validation path whose cost depends on the method, model, and serving workflow. GDPR access and automated-decision provisions use the phrase "meaningful information about the logic involved" for covered processing (European Parliament and Council of the European Union 2016), leaving the appropriate technical approach dependent on the decision and governing law.

capability. It enables human oversight of automated decisions, supports debugging when problems emerge, and can satisfy applicable requirements for decision transparency.

The level of explainability required varies by application context and regulatory environment, and the requirement constrains architecture rather than logging. When a rule requires specific reasons for an adverse action, a team can choose an inherently interpretable model, such as a sparse linear model, and accept whatever predictive capacity that structure gives up on the task. Alternatively, it can deploy a higher-capacity model with a post-hoc attribution method such as Shapley additive explanations (SHAP) or local interpretable model-agnostic explanations (LIME). The SHAP values that localized mispredictions during incident debugging now face an outside reader, which raises the bar. The explanation path adds serving latency, its fidelity to the model must be measured, and it yields a feature attribution rather than a causal account of the decision. Either way, the choice belongs in the design, because it determines which model family is trained and what the serving path must carry. Table 15.8 maps common deployment scenarios to their explainability needs.

↩ SHAP values, which estimate how much each input feature contributed to one prediction, are established as a debugging tool in section 14.7.1.2.

Table 15.8: Explainability Considerations by Domain: Requirements vary by decision, actor, jurisdiction, and governing rules. The engineering challenge is matching explanation mechanisms to applicable requirements while managing risks such as gaming.

Application Domain	Explainability Level	Typical Requirements
Credit decisions	Specific reasons for covered actions	Principal reasons may have to be disclosed to the applicant
Medical diagnosis	Decision support	Support clinical review and applicable documentation
Content moderation	Context-dependent	Support notices or appeals where required
Recommendation	Context-dependent	Provide transparency appropriate to the use and governing rules
Fraud detection	Controlled disclosure	Balance applicable notice duties with adversarial-gaming risk

Whatever the requirement, the candidate models lie along a gradient continuum from intrinsic interpretability to post-hoc explanation (figure 15.4). On the left, small trees and sparse linear or logistic models with meaningful features can expose decision logic to human inspection. Large trees or opaque engineered features can defeat that advantage. On the right, high-capacity representations, such as random forests, deep neural networks, and convolutional neural networks (CNNs) or transformers, often need supplementary explanation methods. LIME and SHAP require fidelity checks; attention maps alone do not establish faithful explanations. The spectrum organizes candidate approaches, not a categorical guarantee for any model family.

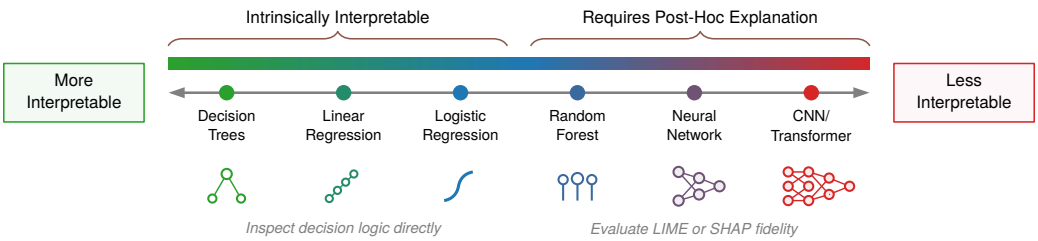


Figure 15.4: Model Interpretability Spectrum: A horizontal continuum groups decision trees, linear regression, and logistic regression as intrinsically interpretable, while random forests, neural networks, and CNNs/transformers require post-hoc explanation. The grouping is schematic: interpretability depends on model size and feature semantics, and any attribution method requires fidelity validation. It frames an auditability trade-off rather than a universal ranking of model quality.

Selecting an explainability strategy along this spectrum requires balancing explanation fidelity, serving latency, and architectural flexibility across three distinct design paradigms. Post-hoc explanation methods, including SHAP and LIME, generate feature importance scores for individual predictions without modifying model internals.[11] Model-agnostic explainers evaluate perturbed inputs or sampled feature subsets, introducing additional computation and approximation. Model-specific SHAP algorithms can exploit internal structure instead; the explanation method and where it runs determine latency and fidelity.

The explanation path has a measurable serving cost. For the credit service's 20 input features, exact Shapley attribution evaluates the model on every feature subset, 1,048,576 evaluations to explain a

11 **LIME (local interpretable model-agnostic explanations) and SHAP (SHapley additive explanations)**: LIME (Ribeiro et al. 2016) fits a local interpretable surrogate around each prediction; SHAP (Lundberg and Lee 2017) adapts Shapley values from cooperative game theory to compute feature contributions under a unified additive framework. Exact Shapley computation requires evaluating a feature's marginal contribution across all possible feature subsets, scaling exponentially with the feature count n_{feat} ($\mathcal{O}(2^{n_{\text{feat}}})$); practical production deployments therefore rely on polynomial-time model-specific algorithms (such as TreeSHAP for decision trees) or sampling approximations (such as KernelSHAP) to avoid unacceptable serving latency. The systems trade-off is that explanation fidelity, latency, and implementation complexity must be budgeted explicitly rather than treated as free.

decision that took one evaluation to make. Sampling explainers bound that work by evaluating a fixed number of subsets, but even 1,000 samples multiply the compute of every explained decision by the sample count, so the explanation path rather than the model can set the service's accelerator budget and its tail latency.

For the rejected credit application, a LIME explanation is dependable only if its local surrogate agrees with the model near the applicant and stays stable across resampling. The check fits the surrogate on perturbed applications drawn from a neighborhood of plausible applications (valid feature ranges, with dependencies such as income and debt-to-income ratio preserved). It then measures the surrogate's prediction error and approval agreement against the model on an independent set from the same neighborhood, and repeats the fit across sampling seeds and neighborhood widths to see whether the leading features and their signs persist. Acceptable error and stability must be specified before the check, and even a faithful surrogate describes model behavior without supplying a causal explanation or a legally sufficient reason.

Inherently interpretable models enforce transparency through structural constraints, using shallow decision trees, sparse linear models, or generalized additive models with inspectable feature effects. Interpretability still depends on model size and meaningful features; structural simplicity can limit expressive capacity on high-dimensional inputs, and internal representations such as attention weights do not constitute faithful post-hoc explanations of causal decision logic. Where inherently interpretable models constrain the architecture from the start, concept-based explanations probe what a trained network has already learned by projecting internal activations onto directions derived from labeled examples of a human concept, such as a "striped texture" direction in a vision model. The projection shows whether a representation encodes the concept; whether the prediction uses it requires a further test of how the output changes along that direction, and every concept needs its own annotated example set.

Example 15.1: The hospital shortcut

Scenario: Pneumonia detection vision models trained at Mount Sinai achieved high internal accuracy but failed on external hospital datasets (Zech et al. 2018).

Diagnosis: The neural network learned shortcut features (hospital-specific scanner artifacts and text tags) rather than true biological lung pathology.

Systems lesson: A model can learn whichever feature predicts the label most easily, including one unrelated to the disease. External validation is essential for detecting shortcut learning. Saliency maps, per-pixel heat maps of how strongly each input region influences the output, can provide supporting evidence but do not establish that the model learned the intended mechanism.

The hospital shortcut shows why interpretability is a systems requirement rather than presentation polish. Teams need sufficient transparency to detect and isolate spurious correlations before deployment. The choice among these paradigms depends on the application's accountability requirements. Credit law requires specific reasons for covered decisions but does not mandate a particular model architecture, and other applications face different transparency, safety, or contestability duties; section 15.5.3 maps those duties to the capabilities they demand. The spectrum does not imply "simple is always better," because a highly interpretable model that makes inaccurate predictions may also cause harm. An explanation, however, covers one decision at a time. The assumptions behind every decision the model makes need a durable record of their own, one that travels with the artifact.

15.3.3.2 Model documentation standards

When a team inherits a production model without that durable record, the artifact may post 94 percent accuracy on a designated test set while leaving the origin of the test set, the composition of the training data, and the evaluated demographic or environmental slices unrecorded, so updating or redeploying the model risks unmeasured regressions under distribution shift. **Model cards** provide standardized documentation for trained models[12] (Mitchell et al. 2019). Analogous to

[12] **Model cards**: The primary failure mode model cards address is scope creep, the gradual expansion from "it worked for case A" to "try it for case B" without revalidating intended use. In practice, cards are often written after deployment decisions are made, documenting observed behavior rather than constraining it. The companion "Datasheets for Datasets" (Gebru et al. 2021) applies the same principle to training data. Without both, the card becomes a historical record rather than an engineering guardrail.

component datasheets in computer engineering, model cards document operational boundaries, evaluation conditions, and intended operating regimes alongside the model artifact.

A complete model card covers seven core sections that together define the model's safe operating envelope. It begins with technical details (architecture, training procedures, and hyperparameters) that enable reproducibility and auditing. It specifies *intended use* alongside explicit out-of-scope applications, preventing scope creep where models trained for consumer photo indexing get repurposed for high-stakes surveillance. The card then documents which *factors* (demographic groups, environmental conditions, or sensor variations) affect performance, guiding both evaluation slicing and production monitoring.

The remaining sections quantify and bound empirical behavior. Performance metrics must report disaggregated results across the operational factors identified in section 15.3.2, because aggregate accuracy conceals systematic subpopulation disparities. Documenting the training and evaluation datasets isolates potential sampling biases and defines the distribution over which the reported metrics hold. Finally, ethical considerations, caveats, and recommendations make deployment trade-offs explicit by recording known failure modes, unmodeled edge cases, and operational mitigations.

Applying this schema to MobileNetV2 illustrates how these categories translate into concrete deployment constraints. Table 15.9 fills five of the seven sections for an edge classification service, and its accuracy entries reuse the illustrative FP32 and INT8 values of the compression validation in section 12.9.1.4.

Table 15.9: Illustrative Model Card: MobileNetV2 for Edge Deployment: Abstract model card categories translate to practical documentation that guides responsible deployment decisions. The numerical entries are scenario assumptions, not reported benchmark results.

Section	Content
Model Details	MobileNetV2 architecture with 3.5M parameters, trained on ImageNet using depthwise separable convolutions. INT8 quantized for edge deployment.
Intended Use	Real-time image classification on mobile devices with less than 50 ms latency requirement. Suitable for consumer applications including photo organization and accessibility features.
Factors	Performance varies with image quality (blur, lighting), object size in frame, and categories outside ImageNet distribution.
Metrics	70.9% top-1 accuracy on ImageNet validation after INT8 quantization (full precision: 71.8%). Accuracy varies by category: 85% on common objects, 45% on fine-grained distinctions.
Ethical Considerations	Training data reflects ImageNet biases in geographic and demographic representation. Not validated for high-stakes applications (medical diagnosis, security screening). Performance may degrade on images from underrepresented regions.

Datasheets for datasets provide the complementary record for the data tier (Gebru et al. 2021), capturing provenance, collection methodology, curation filters, and demographic composition. In D·A·M terms, a model card specifies the algorithmic and machine boundaries (including quantization precision and inference latency budgets), and a datasheet specifies the data that produced them. Together they state what the system is designed to tolerate and preserve the evidence that measurement produced, but both describe the system at the moment of release. Operation is the stage that keeps that description true once the model meets live traffic.

↩ Datasheets, as part of the governance pillar of a data pipeline, are established in section 4.2.2.

15.3.4 Operate: Fairness as loop stability

A credit model that passes its 5-percentage-point fairness gate at launch can breach that gate six quarters later without a single code change, simply because every quarterly retraining learns only from the applicants the previous model approved. A release gate measures a disparity at one instant, and the credit service then meets live traffic and closed-loop retraining that shift its behavior over time. Operating it responsibly requires the incident-response sequence that operations already runs (detection, assessment, mitigation, communication, and remediation) to treat a fairness violation or a slice-level regression as an incident in its own right, and to verify each control before launch: an alert threshold tested against historical replay, a rollback path exercised under load, and an on-call rotation with documented escalation. A requirement without a verified control is an intention rather than a safeguard, so monitoring and response must exist before the model takes production traffic, not after harm accumulates.

↩ The ML incident-response sequence of severity classification, detection, impact assessment, mitigation, and root-cause analysis is established in section 14.7.1.1.

Service-health monitoring catches crashes and timeouts, but the technical debt that ML systems accumulate surfaces as silent degradation (Sculley et al. 2015), and feedback loops can amplify small early errors across the user population. Incident response planning must therefore deploy telemetry for outcome disparities as well as input changes, with response plans that account for label delay and the evidence needed to detect harm.

Production monitoring infrastructure already tracks performance stability, input distributions, and delayed outcomes. The responsibility lens adds two requirements to that telemetry surface. Subgroup parity becomes a first-class monitor, with false-negative and false-positive rates tracked per cohort over time, because aggregate accuracy masks localized regressions. User complaints, contested decisions, and appeals feed the same surface, because statistical aggregation over millions of queries smooths away the rare harms that affected people report first. Parity monitoring matters most where the system's decisions shape its own future training data. In that regime the bias feedback model (principle 13) describes a disparity that evolves as $\Delta_g(k) \approx \Delta_g(0) \cdot \alpha_{\text{fb}}^k$ across feedback intervals rather than staying fixed. The credit service shows what that recurrence implies for a release gate.

↩ Production monitoring architectures for performance stability and distribution drift are established in section 14.5.

Napkin Math 15.3: Fairness as loop stability

Problem: After mitigation, the credit service from section 15.3.2.1 ships with a residual TPR gap of 3 percentage points, inside the scenario's 5 percentage-point release gate. Denied applicants generate no repayment labels in the routine training stream, so each quarterly retraining learns from a population the previous model selected. For this illustration, an independent audit sample provides observed repayment outcomes across both groups, including applicants not approved by the production policy. Without such outcome evidence or a justified missing-outcome correction, the all-qualified-applicant TPR gap would not be directly measurable. The next quarterly audit measures a gap of 3.3 percentage points. If the locally fitted growth factor persisted, when would the gap breach the release threshold?

Math:

1. **Fit the feedback factor**: $\alpha_{\text{fb}} = \Delta_g(1)/\Delta_g(0) = 1.10$ per quarter.
2. **Time to breach under the fitted model**: The projected gap exceeds the gate once $\Delta_g(0)\,\alpha_{\text{fb}}^k$ passes it, that is, for $k > \ln(\text{gate}/\Delta_g(0))/\ln\alpha_{\text{fb}}$. The model projects a breach after 6 quarterly retraining cycles, when the gap reaches 5.3 percentage points.
3. **Without mitigation**: The same factor would grow the original 30 percentage-point gap to 43.9 percentage points within 4 cycles, one year of retraining.
4. **Changing the loop**: If the team approves a small randomized sample of marginal applicants and observes their repayment, the loop again sees outcomes it previously censored. Suppose the refitted factor falls to 0.90; the same launch gap then decays to 1.6 percentage points over the same number of cycles.

Systems insight: A release gate passed at launch is a statement about $k = 0$. Under feedback, fairness is a stability property of the retraining loop. The projected breach time motivates a shorter audit interval, but two observations do not validate future dynamics. An intervention can change α_{fb} (through what data the loop collects) as well as $\Delta_g(0)$. The fitted factor is a local model; it must be refitted at every audit because intervention, saturation, or a shifting applicant pool changes the dynamics.

The DLRM ranking service runs the same loop with far more severe censoring. With 10 million daily users each shown 20 ranked items from a catalog of 1 million items, the service logs 200 million impressions a day, yet each user's feedback covers only 0.002 percent of the catalog, and the previous model chose every item in it. Items and creators the ranker stops showing generate no clicks and therefore no evidence that they deserved to be shown, which is the exposure form of the bias feedback model. The remedy mirrors the credit service's randomized approvals. Reserving 1 percent of slots for randomized exploration yields 2 million impressions a day that the previous model did not select, a sample from which each creator group's exposure share and engagement

can be estimated against an unbiased baseline. The cost is measurable like any other serving change, because that share of slots is served at lower expected engagement, and it is what keeping the feedback factor below one costs.

Telemetry infrastructure provides protection only when coupled with disciplined operational response. Automated dashboards require defined review cadences, clear on-call ownership, and explicit escalation procedures to ensure emerging anomalies trigger corrective engineering action rather than passive observation.

Checkpoint 15.3: Ethical deployment

Deployment is where safeguards must become operational.

Safety net

- ☐ **Rollback**: Verify that the previous model can be restored within the incident-response target.
- ☐ **Human-in-the-loop**: Verify that an operational routing path exists for human review of low-confidence predictions.

Monitoring plan

- ☐ **Silent failure**: Specify the subgroup metrics that will expose postdeployment bias.

Equitable performance across demographic cohorts addresses only one dimension of responsible engineering. The resource stage of the lifecycle addresses another, because every training run, inference request, and monitoring pipeline draws electrical power that converts to carbon emissions and operating expenditure. A pipeline can satisfy demographic parity while consuming orders of magnitude more compute than the task warrants, so responsible engineering must govern both whom the system serves and the physical resources consumed in serving them.

15.4 Environmental and Cost Awareness

In 2019, researchers estimated that development-scale training and architecture search for a large natural language processing (NLP) model could emit as much carbon as five cars over their entire lifetimes (Strubell et al. 2019). Later analysis showed that the most quoted architecture-search estimate was highly sensitive to proxy-task, hardware, data-center efficiency, and grid carbon-intensity assumptions (Patterson et al. 2021). The correction narrows the estimate without removing the point. Training runs consume megawatt-hours of electricity, inference at scale multiplies per-request inefficiencies into measurable environmental impact, and resource-intensive models exclude organizations that lack large compute budgets. The techniques that improve efficiency therefore also serve as instruments of responsible engineering, tying computational efficiency to environmental sustainability and economic accessibility. On the DLRM ranking service, request volume makes serving rather than training the term that dominates both the bill and the emissions.

15.4.1 Efficiency as responsibility

Training a single large language model consumes thousands of GPU hours and energy measured in megawatt-hours. The bitter lesson predicts that this footprint will keep growing, because the methods that win are the ones that convert additional compute into capability. Much of this expense, however, is not intrinsic to the learning task but represents *accidental complexity*: training from scratch when fine-tuning would suffice, deploying larger architectures than tasks require, and running hyperparameter searches that explore redundant configurations. Computational cost depends on engineering choices as well as model physics. Green AI treats that efficiency as a primary metric rather than an afterthought.[13]

The growth in training compute carries a price, and the price decides who can take part. Figure 15.5 plots the estimated compute cost of training notable models since 2016 in constant 2023 dollars. Across the models that Epoch AI flags as frontier at release, the cost of a training run rose about 2.9× per year, and the most expensive run in the data, Grok 4 in 2025, cost about $388M (Epoch AI 2026c). Cottier et al. (2024), using their own selection of frontier runs, estimate 2.4× per year. Either rate concentrates frontier training in the few organizations able to fund it, so efficiency is a question of access as well as of energy. For a frontier model the training run dominates the cost; for a high-traffic service such as DLRM, as section 15.4.2 shows, serving does.

At the hardware level, environmental impact scales with total electrical energy draw across the machine lifecycle, mediated by accelerator utilization, data-center cooling overhead (power usage effectiveness), and the **carbon intensity** of the local electrical grid, the mass of CO_2eq emitted per kilowatt-hour it delivers. Within the accelerator package, data movement can dominate energy dissipation. In the 45 nm reference behind the energy-movement principle (7), a single off-chip DRAM access costs two to three orders of magnitude more energy than a 32-bit floating-point operation, although total energy also depends on access width, event counts, reuse, and the device. Algorithmic and runtime optimizations reduce environmental burden only when they reduce measured watt-hours for a verified quality target under production load, curbing this physical dissipation at the silicon level.

Physical resource demands also dictate deployment accessibility. Oversized models that need multi-accelerator nodes linked by high-bandwidth interconnects (such as NVLink) just to hold their weights in memory confine deployment to centralized cloud facilities. Compressing models so their working sets fit within commodity DRAM or the unified memory of edge processors opens up who can run them. Running locally removes the need for a constant high-bandwidth network link and recurring cloud API bills, letting client devices run low-latency, private inference right where data is captured.

At production scale, where serving systems process millions of requests daily, per-query inefficiencies compound rapidly across accelerator fleets. Under the iron law of ML systems ($T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$), per-query execution latency dictates the request throughput that an individual accelerator can sustain before violating service-level latency objectives. Reducing execution time allows a smaller provisioned cluster to support the same aggregate traffic, and capacity planning can turn that reduced service demand into lower provisioned-capacity costs. A fixed fleet may also save dynamic energy, although idle and baseline power remain. A valid comparison

[13] **Green AI**: Schwartz et al. (2020) contrasted "Red AI" (performance at any cost) with "Green AI" (efficiency as primary metric). The compute-growth anchor comes from AI and Compute's 2012–2018 trend analysis (the fit that gives a 3.4-month doubling time), which reported a 300,000× increase in compute used in the largest AI training runs over that six-year window (Amodei and Hernandez 2018b); comparisons that extend to later frontier models span a longer window and report larger factors. The Green AI proposal (reporting FLOPs alongside accuracy for every published result) reframes efficiency from an engineering preference into a scientific reporting obligation, making the resource cost of marginal accuracy gains visible and comparable across research groups.

↩ The bitter lesson, the historical pattern in which methods that scale with compute overtake hand-built knowledge, is established in section 1.4.

↩ The physical energy disparity between off-chip memory access and arithmetic execution is established in section 11.2.3.

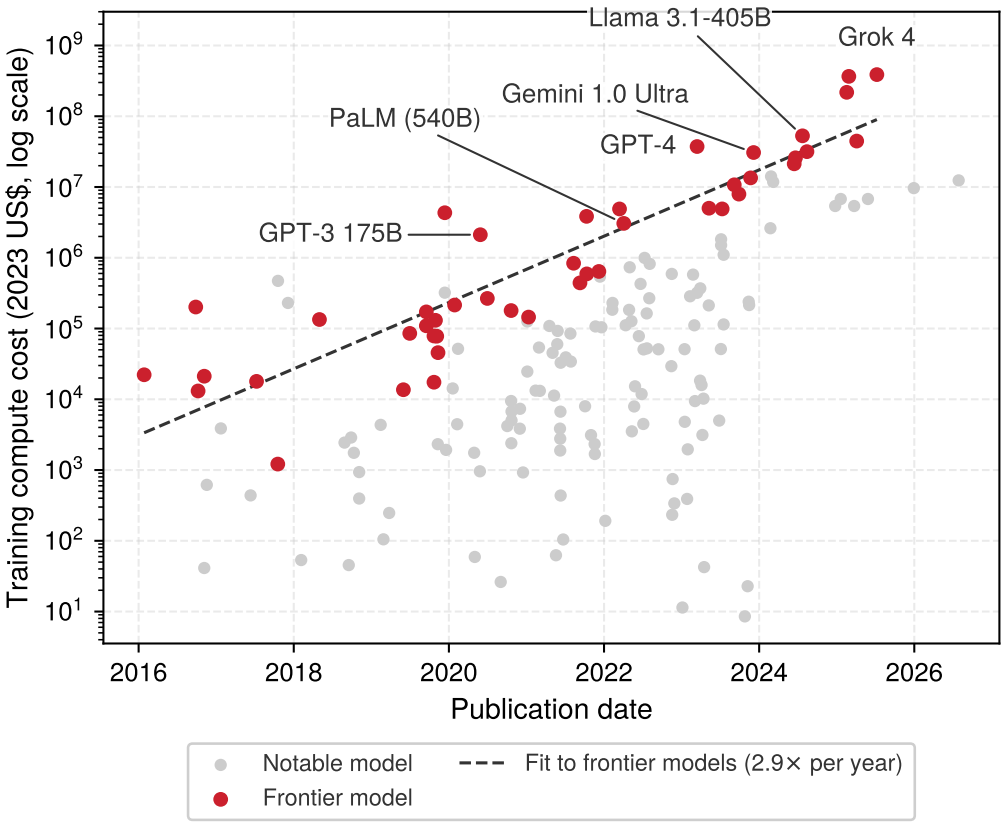


Figure 15.5: Cost of Training Notable Models: Estimated compute cost of the final training run of notable models since 2016, in constant 2023 U.S. dollars on a logarithmic axis. Red points are models flagged as frontier at release; the dashed line is a least-squares fit to them. Costs follow the amortized hardware and energy method of Cottier et al. (2024). Data from Epoch AI (2026c).

therefore measures total energy over the same traffic and quality target and distinguishes that result from savings on a capacity-based bill.

Each optimization developed earlier in this book attacks a specific term of these equations, and each reduces environmental burden only under the conditions its own chapter states. Quantization and pruning shrink the bytes moved D_{vol} and the operation count O when supported kernels retain the smaller representation, and knowledge distillation moves a large model's behavior into a smaller one (chapter 10). Selecting the data a model trains on removes training operations before they run (chapter 9), and specialized datapaths and fused kernels raise hardware efficiency η_{hw} (chapter 11).

Responsible engineering treats these optimizations as design constraints established before training and deployment, rather than post-hoc remedies applied to an oversized model. System requirements specify explicit upper bounds on latency, memory capacity, and energy consumption alongside target predictive quality, and the task is to find the most compact architecture and execution strategy that satisfies them. In practice this begins by establishing bounded machine envelopes (thermal design power, battery capacity, and peak latency) before choosing or training an architecture, and applies targeted optimizations such as quantization or kernel fusion only after the most compact adequate architecture has been selected.

Edge deployment makes these limits immediate, because passive thermal dissipation and battery capacity enforce power budgets that cannot be relaxed after the fact. A wearable with an illustrative 500 mW inference budget shows the consequence for model choice. Under the active-power profiles assumed here, the KWS lighthouse's DS-CNN (200K parameters at 50 mW) leaves a 10× power margin, while MobileNetV2 (3.5M parameters at 1.2 W) exceeds the same budget by 2.4× before

sustained thermals are considered. Fitting the envelope is necessary but not sufficient: a keyword detector does not substitute for a vision model merely because it fits, and per-inference energy compounds into aggregate emissions and operating cost once the model is deployed at scale. Techniques that lower measured energy per inference reduce lifecycle energy only when their savings outweigh any added training, preprocessing, or other workload costs, while cloud and cluster financial savings depend directly on provisioning and hardware utilization.

15.4.2 Total cost of ownership

On the DLRM ranking service, a training cycle that costs $3,200 is followed by roughly $500,000 per year in serving compute. That contrast shows how total cost of ownership,[14] the upfront and recurring expense of a system over its lifetime, can be dominated by recurring inference at high traffic. Where the cloud-versus-edge comparison used that total to choose a deployment, the lifecycle view splits one deployment's total into training cycles, recurring inference, and operations to find which term dominates. Other systems may be dominated by training, data, staffing, or idle capacity, so measurement determines where optimization should focus.

Consider the DLRM ranking service serving 10 million users daily. Training costs appear considerable: data preparation consumes 100 GPU-hours at approximately $4/hour ($400), hyperparameter search across multiple configurations requires 500 GPU-hours ($2,000), and the final training run uses 200 GPU-hours ($800). Total training cost reaches approximately $3,200.

Inference costs dominate in this scenario. With 10 million users each receiving 20 recommendations per day, the service runs 200 million inferences daily. Treating 10 milliseconds per inference as unbatched, non-overlapped dedicated GPU service demand yields approximately 23.1 GPUs running continuously. At $2.50/GPU-hour, the scenario's annual GPU cost reaches $506,944. That rate sits below the $4/hour the training runs pay because an always-on fleet is bought as continuously reserved capacity rather than on demand.

Over a three-year operational period, twelve total quarterly training cycles, including the initial cycle, produce training costs of approximately $38,400, while inference costs over the same period total $1.5M. The resulting 40:1 ratio is specific to these assumptions, but it dictates where optimization effort yields the highest return: inference latency and serving efficiency.

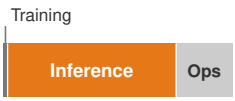
Per-query optimization becomes important when serving billions of requests. Shaving a fifth off the per-query accelerator service demand can reduce required hardware when other workload assumptions remain fixed. Hardware selection among CPUs, GPUs, and Tensor Processing Units changes costs and carbon footprint in workload- and deployment-dependent ways. Model compression through quantization and pruning can reduce high-volume inference costs when it lowers measured service demand without unacceptable quality loss.

The third term of the lifecycle total cost of ownership (TCO) is operations: monitoring, maintenance, retraining, and incident response, all of which scale with system complexity and with the rate of distribution shift in the application domain. Resources consumed by one ML system also cannot be used for other purposes, so wasteful consumption in one project constrains what other projects can attempt. Engineers should therefore evaluate return on investment (ROI), asking whether the value an ML system delivers justifies its resource consumption: a recommendation system that increases engagement by 1 percent might not justify millions of dollars in computational costs, while a medical diagnosis system that saves lives does. Deployment-to-training cost ratios vary with traffic, utilization, hardware, update cadence, staffing, and incident burden, so the right model choice depends on measured lifecycle cost and task value, not on a fixed ratio.

15.4.2.1 Accounting for cost and carbon

Converting the same scenario into carbon requires measured or estimated energy use, facility overhead, and applicable grid carbon intensity. The accounting keeps the walk-through's traffic, service demand, and training budget, adds the operations work that the walk-through left unquantified, and converts every accelerator-hour into emissions with one conversion factor. Training recurs with each retraining cycle, inference recurs with every request, and operations accumulate through telemetry, on-call engineering, and incident response, so each enters the three-year total on its own basis.

[14] **TCO (total cost of ownership).** The power, memory, and latency envelopes that separate cloud, edge, mobile, and tiny ML deployment are established in section 2.2. ML TCO also includes labeling, monitoring, retraining, remediation, energy, audits, and compliance costs that an estimate of compute alone omits. Utilization and update cadence determine how the balance between training and inference falls for a given service.



At production scale, recurring inference can dominate lifetime cost.

↩ Total cost of ownership as a comparison of cloud and edge deployments, weighing capital against operating expense, is established in section 2.5.2.

↩ Cost per inference, the serving cost that capacity planning divides among requests, is established in section 13.10.1.

Napkin Math 15.4: The carbon cost of compute

Problem: The cost accounting must convert GPU-only electrical energy into "kg CO_2eq." What carbon factor does one GPU-hour carry under the scenario baseline, excluding host, facility, and embodied impacts?

Variables:

- **Power**: 400 W per GPU (scenario baseline).
- **Intensity**: 0.4 kg/kWh CO_2eq (rounded grid baseline).

Math: Equation 15.2 captures the standard conversion:

$$\text{Carbon} = \text{Energy (kWh)} \times \text{Carbon Intensity (kg/kWh)} \tag{15.2}$$

Applying the baseline assumptions:

$$(0.4\ \text{kW} \times 1\ \text{hour}) \times 0.4\ \text{kg/kWh} = 0.16\ \text{kg}\ CO_2\text{eq per GPU-hour}$$

Systems insight: This GPU-only conversion factor lets the accounting track a modeled accelerator-carbon subtotal alongside dollar cost. Whole-system and facility emissions require incorporating the host, facility overhead, and embodied energy boundaries.

In table 15.10, inference accounts for 73.5 percent of the three-year cost, operations for 24.6 percent, and training for only 1.9 percent. Those proportions turn efficiency from a tuning preference into a responsibility check, because small serving-path reductions compound across every request the fleet handles.

Table 15.10: Three-Year Cost and Carbon of the DLRM Ranking Service: Training repeats data preparation, hyperparameter search, and final training at $4/hour for every cycle; inference converts daily requests into accelerator-hours under an unbatched, non-overlapped service model, which real batching and utilization change; operations cover monitoring, 0.5 FTE of on-call engineering, and incident response. The inference-to-training cost ratio is 40:1, and a 20 percent reduction in per-query accelerator demand saves about $304K and 19 t of modeled accelerator CO_2 under the same assumptions. The carbon column excludes operations, facility overhead, and embodied impacts.

Category	Basis	3-Year Cost	Share	Modeled Accelerator Carbon
Training	12 cycles × $3,200 per cycle	$38K	1.9%	1.5 t
Inference	556 GPU-hours/day × 365 × 3 years × $2.50/GPU-hour	$1.52M	73.5%	97.3 t
Operations	$170K/year × 3 years	$510K	24.6%	Not estimated
Total TCO		**$2.07M**	**100%**	**~98.9 t modeled subtotal**

Checkpoint 15.4: Efficiency as responsibility

Total cost of ownership reveals where responsible optimization has the most leverage.

- ☐ **Inference dominance**: under this scenario's assumptions, can you compare a 20 percent reduction in total inference cost with a 50 percent reduction in total training cost, and explain why reducing only final-run time does not establish the latter saving?
- ☐ **Carbon accounting**: can you convert GPU-hours into kg CO_2eq using power draw and carbon intensity, and explain how workload design, hardware, scheduling, and region affect the result?
- ☐ **Sufficiency test**: for a given ML system, can you justify that the model size is appropriate for the task, or identify where a simpler model would deliver comparable accuracy at a fraction of the cost?

15.4.3 Environmental impact

Two data centers can run the exact same DLRM serving fleet at the same dollar bill while one emits several times more carbon than the other because its local power grid burns coal instead of hydroelectricity, and a team that replaces working servers early to save 10 percent on electricity can raise net lifetime emissions once the carbon spent manufacturing the new silicon is counted (Gupta et al. 2022). The total-cost-of-ownership analysis in section 15.4.2 captures costs that someone pays directly, but computational resources also impose costs that no one is billed for. Operational emissions depend on measured energy, facility power usage effectiveness (PUE), and regional grid carbon intensity (gCO_2e/kWh); embodied manufacturing impacts require separate accounting. In power benchmarking, PUE separates server-plug energy from facility overhead; in carbon accounting, the same ratio, total facility energy divided by IT-equipment energy, becomes the multiplier that converts measured IT energy into the energy the facility actually draws. The same measurement boundary keeps server fans inside the IT term so that their energy is not counted twice.

A first-order operational emissions estimate is $\text{Energy}_{\text{hardware}} \times \text{PUE} \times \text{Carbon Intensity}$, where the hardware term covers the full IT-equipment boundary over the accounting window, not GPU-only energy. Operational carbon includes electricity consumed by both active work and idle provisioned equipment. In contrast, embodied carbon encompasses the greenhouse gases emitted across the hardware supply chain (silicon wafer fabrication, packaging, transport, and facility construction) before a chip executes its first cycle. Embodied carbon can account for a large share of lifetime emissions, and on low-carbon grids a majority (Gupta et al. 2022), so prematurely replacing older servers for slight operational efficiency gains can paradoxically increase net lifetime emissions. Per-request energy is measured in joules per inference, whereas joules per FLOP characterizes hardware work efficiency. An optimization that reduces TCO may not reduce energy or emissions if it changes utilization, hardware, or workload placement. Data-center electricity use makes workload design, hardware, cloud region, and timing part of responsible engineering (Henderson et al. 2020).

↩ The power measurement boundary that separates server-plug energy from facility overhead is established in section 12.7.1.

The DLRM accounting in table 15.10 shows how lopsided the carbon split can be. Under its service-demand assumptions, inference emits 63× as much modeled accelerator CO_2 as all 12 training cycles combined, so for a service of this kind the serving path, not the training run, is where carbon reductions have leverage. Efficiency optimization and environmental responsibility align when measured lifecycle energy and emissions fall for the required workload and quality target. More granular carbon accounting methodologies build on this foundation: lifecycle assessment tracks impacts across the system's full life, scope 1/2/3 emissions separate direct emissions, purchased electricity, and supply-chain or use-phase emissions, and carbon-aware scheduling shifts work toward lower-carbon times or regions.

The same physical quantities that govern performance also affect responsibility. Data movement and computation contribute to chip-level energy, while data-center emissions additionally depend on utilization, facility overhead, grid intensity, and embodied impacts. Pareto analysis applies to fairness, latency, and carbon alike, and the first question is always whether the current design lies on the frontier, because a dominated design can improve several objectives at once. *Responsible engineering extends the constrained optimization problem this book has been teaching to objectives that include societal impact alongside throughput and latency.* The magnitude of the carbon term becomes clearer in a scale calculation for training a large foundation model; under the grid assumption of napkin math 15.5, one such run exceeds a hundred passenger-car years of emissions (see the margin).



This training scenario exceeds 100 passenger-car years of emissions.

Napkin Math 15.5: The carbon cost of scale

Problem: A foundation model is being trained at the scale of GPT-3, consuming 1,287 MWh of electricity (Patterson et al. 2021). What is the operational carbon impact under the stated grid-intensity assumption?

Math:

1. **Energy consumption**: 1,287 MWh = 1,287,000 kWh.
2. **Carbon intensity**: This scenario uses ≈ 429 g/kWh CO_2 (0.429 kg/kWh) (Patterson et al. 2021), a US grid-average figure slightly above the rounded 0.4 kg/kWh baseline of

the cost accounting. Each calculation states its own grid assumption, and the difference changes the result by less than a tenth.

3. **Total emissions**: 1,287,000 kWh × 0.429 kg/kWh = 552,123 kg CO_2 (552 t).
4. **Comparison**: The scenario assumes 4.6 t CO_2 per passenger-car year. This per-year basis differs from the lifetime-car basis of the 2019 five-car estimate, so the two car comparisons are not directly comparable.

Systems insight: Under this training-energy scenario, the operational emissions equal the assumed annual emissions of 120 passenger cars. A 1 percent reduction in total training energy under the same grid mix would remove the equivalent of about 1.2 cars taken off the road for a year from the calculation.

Specification, measurement, documentation, and resource accounting define what a system must satisfy. The Govern stage of the lifecycle makes those guarantees provable to someone outside the team, which requires infrastructure that records validation results, preserves audit evidence, and enforces operational bounds in production.

15.5 Data Governance and Compliance

The keyword spotting system makes the governance problem physical. An always-listening voice assistant samples audio continuously in private homes, stores enrolled speaker profiles, and periodically syncs with cloud pipelines that retrain on population-wide audio, so every guarantee the earlier stages established now depends on what data the system may hold, who may touch it, and what evidence survives. **Data governance**, the engineering discipline that turns regulatory and ethical obligations into verifiable technical controls, is the enforcement layer that makes accountability operational. Fairness metrics, model cards, and impact assessments have practical force only if the system can prove what data it ingested, which principals accessed it, which legal basis authorized processing, and whether applicable deletion or contestability requests were executed. The legal-basis question alone can decide an enforcement outcome. In 2023, Ireland's Data Protection Commission found that Meta had relied on an invalid legal basis for personalized advertising and had violated transparency and fairness requirements, a governance failure that involved no security breach at all (Data Protection Commission 2023).

Storage architectures also serve as governance enforcement mechanisms that determine which principals access data, how queries are tracked, and whether pipelines satisfy legal requirements. Every architectural decision, from ingestion strategies through extract, transform, load (ETL) pipelines to persistent storage layout, carries governance implications that surface during regulatory audits, privacy investigations, or model recalls. Data governance translates abstract policy into concrete systems engineering through authenticated access controls on training partitions, immutable audit infrastructure that logs dataset access, privacy-preserving transformations during feature extraction, and deterministic lineage systems that trace training snapshots to deployed model checkpoints.

↩ The database, warehouse, and data-lake storage architectures, and the access patterns each one serves, are established in section 4.10.1.

Principle: Compliance as engineering constraint

Invariant: Data governance obligations are engineering requirements, not optional policy overlays.

Implication: Systems that process regulated data must map applicable duties to technical and organizational controls. Access control, erasure, contestability, audit, and lineage may be required or useful depending on the jurisdiction, data, actor, and use case. The General Data Protection Regulation (GDPR), California Consumer Privacy Act (CCPA), Brazil's General Data Protection Law (LGPD, *Lei Geral de Proteção de Dados*), and China's Personal Information Protection Law (PIPL) differ in scope and duties, so no single control checklist establishes compliance across them.

Data governance turns security, privacy, compliance, and lineage into mutually dependent engineering constraints, and the KWS system shows where each one binds. Continuous capture, enrolled

profiles, and fleet-wide retraining create obligations across consent management, data minimization, access auditing, and deletion rights, and the four domains map directly to hardware and pipeline enforcement points in this architecture. Security comes first by protecting raw features, training partitions, and model artifacts across the ingestion lifecycle through mutual authentication, role-based authorization, encryption at rest and in transit, and hardware-rooted key management: hardware keystores on edge microcontrollers protect stored speaker templates, mutual TLS authenticates telemetry in transit, and role-based access controls gate access to centralized training audio in cloud storage. Because access control alone does not limit what an authenticated consumer can learn from the records it reads, privacy regulates information exposure inside that boundary through differential privacy, k-anonymity, and automated feature redaction, keeping ambient KWS audio inside a circular SRAM buffer until an on-device trigger and bounding information leakage during fleet-wide acoustic model retraining.

Where privacy governs what leaves the device, compliance translates jurisdiction-specific statutes (such as GDPR, CCPA, and LGPD) into retention windows, consent verification, and cross-border replication rules once data arrives in the cloud, determining the legal basis for processing and the scope of any revocation or erasure workflow that removes in-scope voice profiles and acoustic data across primary stores and backups. Proving that those security, privacy, and retention rules were followed requires lineage and audit systems that render the entire data lifecycle inspectable. Retained lineage linking deployed weights through training snapshots to authorized source records supports verification of security claims and applicable obligations, and in the KWS pipeline it establishes provenance through content hashes and signed manifests that link each deployed DS-CNN binary back through specific training splits and acoustic augmentation seeds to the original raw recordings.

Governance works as an operating model rather than a single control. The framework in figure 15.6 arranges eight elements, from organizational oversight and compliance policies to data catalogs, sourcing, quality metrics, runtime operations, and security, around a central governance hub. Technical mechanisms and organizational processes must function as mutually reinforcing subsystems: encrypted storage without identity-mapped access controls still lets an insider copy data out, and compliant storage without verifiable audit trails cannot pass a regulatory inspection. Within the D·A·M taxonomy, data governance is the control plane of the Data axis. It enforces lawful sourcing, privacy bounds, and lifecycle integrity for the data that feeds the Algorithm, and it runs on Machine-axis infrastructure such as encrypted storage, access control, and audit logs.

Engineering teams do not operate in isolation. Team-level technical practices such as audit logging, bias testing, and explainable interfaces sit at the core of expanding organizational, industry, and regulatory structures, with requirements and legal mandates flowing inward across each boundary and the technical controls at the center supplying the evidence that external compliance rests on (figure 15.7). The rest of this section works at that innermost ring, starting with who may reach the data at all.

15.5.1 Security and access control architecture

When a data scientist queries the KWS feature store for training data, she can read aggregated voice features but cannot access the raw audio recordings from which they were derived, while the serving pipeline can read online features for inference without writing to the training dataset or modifying source data. That separation reflects a layered security architecture in which governance requirements translate into enforceable technical controls at each pipeline stage. Feature stores enforce role-based access control (RBAC) by mapping organizational policies to authenticated identities and database permissions. These controls operate across distinct storage tiers: object storage enforces bucket-level access policies, data warehouses implement column-level security and dynamic masking for sensitive fields, and feature stores maintain isolated read and write paths that separate low-latency online inference key-value lookups from high-throughput offline batch training extracts.

Access control mechanisms remain incomplete without encryption, which protects data at rest and in transit but does not prevent misuse after decryption or authorized access. Training corpora in data lakes use envelope encryption with keys held in a dedicated key management service, feature stores encrypt tables at rest and authenticate every service-to-service transfer, and KWS edge devices combine transport encryption for payload confidentiality with asymmetric code signing to verify

↩ The dual-storage architecture of offline columnar stores and online key-value caches is established in section 4.7.5.

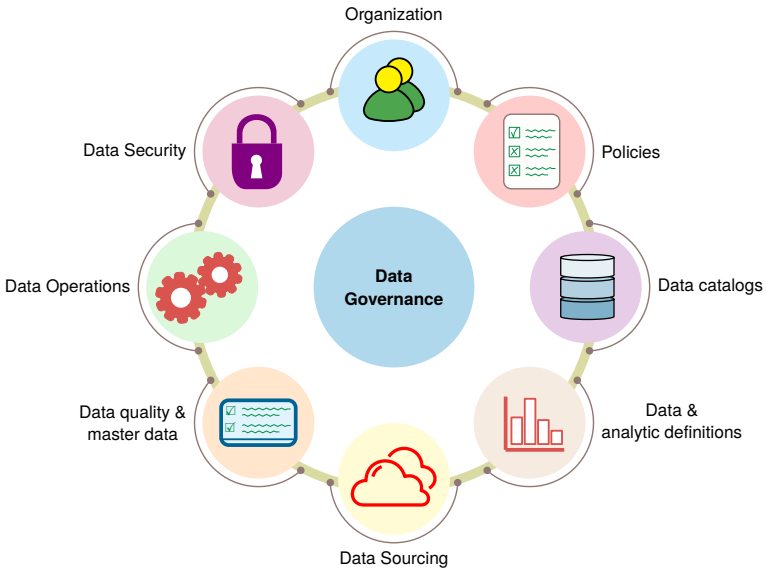


Figure 15.6: Data Governance Framework: Eight operating elements surround Data Governance as a central hub: policies, organization, security, operations, data quality and master data, sourcing, catalogs, and shared analytic definitions. The circular arrangement presents governance as a coordinated operating model rather than a single control, spanning organizational functions such as catalogs, sourcing, and analytic definitions alongside technical controls such as data security. Adapted from standard enterprise data governance frameworks.

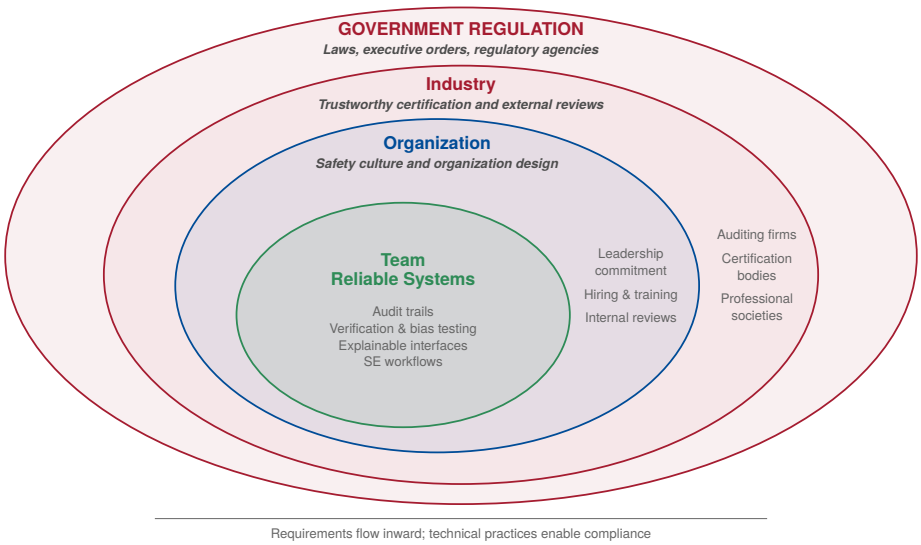


Figure 15.7: Responsible AI Governance Layers: Four nested ovals place team-level safeguards at the center, surrounded by organizational safety culture, industry certification and review, and government regulation. The nesting distinguishes implementation at the center from oversight outside it; the annotation states that requirements flow inward and technical practices enable compliance. Adapted from Shneiderman (2022).

model-update integrity. Each of these controls is only as strong as its key management and update authorization.

Protecting data while it is being computed on is a further layer, **confidential computing**. Hardware secure enclaves and trusted execution environments isolate protected memory and execution from host software outside their trust boundary, and attestation lets a client verify the approved configuration before releasing keys or data to it. The guarantee covers only the declared attacker

boundary and supported configuration, and it does not protect data from the application authorized to process it; the technique matters most when training and serving run on infrastructure that the data owner does not control.

Serialized weights and checkpoints require the same protection as training data. Deserializing an unverified checkpoint in a format that can embed executable objects (Python `pickle` archives are the common case) gives an attacker arbitrary code execution on training nodes and inference workers. Restricted serialization formats that store only tensors avoid that mechanism, and write-protected promotion gates with cryptographic signatures verify artifact integrity and authorized provenance before loading. These controls do not establish that a model is free of backdoors or that every parser and runtime is secure.

Authorized loading also leaves the model's behavior to be evaluated. Poisoned training samples can induce backdoors, perturbed inference inputs can induce misclassification (evasion attacks), and systematic queries can extract model functionality (Tramèr et al. 2016). Access control, cryptographic enclaves, and artifact verification therefore establish *who* can reach data and models and *how* they are protected across storage, transit, and execution, yet controlling access is only half the problem. Even authorized callers can compromise individual privacy if the data or model outputs themselves expose too much: prediction APIs or raw weights can reveal whether a particular record was present in training (Shokri et al. 2017), so artifact integrity and limits on information exposure require distinct controls.

15.5.2 Technical privacy protection methods

Even when role-based access control works as designed, a data scientist querying a training table does not need, and should not see, individual user records when aggregate statistics suffice, and a public prediction API can still leak whether a specific patient or customer was in the training set (Shokri et al. 2017). Privacy-preserving techniques[15] address this gap by determining what information systems expose even to authorized users, adding a second layer of protection beyond access control. Differential privacy bounds changes in an exported output distribution under a specified neighboring-dataset relation.[16] One training mechanism is Differentially Private Stochastic Gradient Descent (DP-SGD) (M. Abadi et al. 2016). It clips each sample gradient to an ℓ_2 norm threshold C, then adds calibrated Gaussian noise with distribution $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ to the sum of clipped gradients, where σ is the noise multiplier. A privacy accountant tracks the privacy loss composed across the sampled training steps and checks it against the chosen epsilon budget.

Clipping needs each sample's gradient norm, and the direct way to obtain it materializes one full gradient per sample, so gradient storage grows by the batch size. For the KWS acoustic model (200K parameters), one FP32 gradient occupies 800 KB and a batch of 256 per-sample gradients 204.8 MB; the same approach for the 1.5B-parameter GPT-2 would need 1.5 TB, far beyond the 80 GB of one H100. Specialized clipping implementations obtain the clipping factors without retaining every per-sample gradient, and small sequential microbatches trade the memory back for lost parallelism, so the cost of DP-SGD must be measured for the chosen implementation with its clipping, noise, sampling, and accounting assumptions intact.

Empirical attacks complement these formal checks. Membership inference attacks measure whether an observer can tell that a specific record was in the training set, and model inversion attacks use confidence outputs to reconstruct sensitive feature values or a training subject's likeness (Fredrikson et al. 2015). A successful attack reveals practical leakage, but no empirical attack can prove that an (ϵ, δ) bound holds.[17] Reducing confidence-score precision or restricting outputs to top-k classes reduces what some APIs expose, but neither is a complete defense; their value depends on the attacker's access and auxiliary information, while differential privacy provides a separate bound under its stated mechanism and accounting assumptions.

While differential privacy bounds information leakage from model outputs, collecting continuous raw sensor streams creates an immediate vulnerability if raw data leaves the capture device. Always-listening edge architectures, such as the lighthouse KWS system, operate under severe privacy constraints because continuous ambient microphone sampling must coexist with strict minimization and retention boundaries. To reduce routine transmission exposure, this design confines acoustic feature extraction and wake-word detection to local low-power microcontrollers or digital signal processors (DSPs). Incoming audio samples stream through a small volatile circular buffer in on-chip

[15] **Privacy-preserving techniques**: K-anonymity (Sweeney 2002) ensures each record is indistinguishable from at least $k_{\text{anon}} - 1$ others with respect to chosen quasi-identifiers, l-diversity adds attribute variety within equivalence classes (Machanavajjhala et al. 2007), and t-closeness bounds distribution distance (Li et al. 2007). These syntactic guarantees address different threats from differential privacy and do not by themselves protect against all ML-specific leakage: a model trained on de-identified data can still memorize examples or reveal membership signal under some conditions (Shokri et al. 2017). Differential privacy instead bounds the influence of one record under a specified mechanism and privacy budget, offering a semantic guarantee designed to remain meaningful in the presence of side information; the methods are therefore not a simple strength hierarchy, and the appropriate protection depends on the release, threat model, and utility requirements.

[16] **Differential privacy**: Introduced by Dwork et al. (2006), a randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ)-differential privacy if for all neighboring datasets X, X' differing by one record and output sets $\mathcal{S}$, $\Pr(\mathcal{M}(X) \in \mathcal{S}) \leq e^{\epsilon} \cdot \Pr(\mathcal{M}(X') \in \mathcal{S}) + \delta$. Here ϵ controls the multiplicative bound and δ is additive slack in the inequality; their acceptable values depend on the neighboring-dataset relation, release, and policy. The systems trade-off is utility rather than mere implementation complexity: stronger privacy often requires more noise or tighter sampling, and privacy loss composes across repeated queries or training steps. Engineers must therefore track cumulative privacy loss with a valid accountant and report the assumptions behind the chosen budget.

SRAM, where older frames are continuously overwritten by newly sampled pulse-code modulation (PCM) data. The policy allows saving or sending audio only after an on-device wake-word trigger. Because detection can produce false wakes, validation must cover both the detector and the buffer and egress controls, including which pre-trigger audio becomes accessible after activation.

Confining raw audio to the device enforces data minimization but starves centralized training of the acoustic variety that improves accuracy. **Federated learning** recovers that variety by moving training to the data. Training-capable clients such as paired phones or home hubs train on authorized local examples and send model updates instead of raw audio, and a coordinator combines those updates into a shared model.[18] The constrained KWS microcontroller only runs the result. The approach exchanges centralization risk for physical bottlenecks: every round moves parameter updates over constrained wireless uplinks, battery-powered clients drop out mid-round, data that differ from home to home make local updates diverge, and raw updates can still leak training examples through reconstruction attacks (Zhu et al. 2019). Compression of those updates reduces communication but provides no privacy guarantee by itself.

Secure aggregation lets the coordinator learn only the aggregate of participating clients' updates, under the protocol's participation, dropout, and collusion assumptions (Bonawitz et al. 2017). What that aggregate and the final model reveal remains outside its protection; differential privacy bounds that release by limiting the effect of any one record or client, provided the training mechanism and privacy accountant support the chosen neighboring relation. A design that needs both must account separately for what a curious coordinator observes during training and what recipients can infer from released checkpoints.

When triggered audio recordings or aggregated model checkpoints must be retained on centralized infrastructure for verification or downstream training, privacy protection shifts to automated lifecycle management. Automated retention workflows apply time-to-live policies across primary stores, caches, training partitions, and backups. Cryptographic erasure destroys the keys needed to decrypt covered copies; physical overwrite is a distinct storage-sanitization mechanism. Neither removes every copy automatically. The workflow must inventory replicas, key copies, and backups, define applicable retention exceptions, and verify that in-scope data is no longer recoverable under the chosen threat model.

15.5.3 Regulation as architecture

Retention and erasure pipelines exist to satisfy legal obligations as much as engineering preferences. Regulation more broadly changes responsible engineering from a best-practice argument into architecture constraints. Suppose the credit service denies an applicant, and the applicant asks why, contests the decision, and later requests access to the data used about her. The system must do more than report an accuracy score. It must produce an explanation tied to the specific decision, preserve the model and data lineage that led to that output, route the dispute to a substantive human review path, retain audit logs, and support deletion or access workflows where data rights apply. Responsible engineering now operates within explicit regulatory frameworks that turn transparency, oversight, and accountability into technical requirements.

Regulation first enters the architecture through risk classification. The EU AI Act establishes a comprehensive framework, classifying AI systems by risk level and mandating requirements accordingly.[19] The Act entered into force on August 1, 2024 and applies in phases: prohibited-practice rules began applying in 2025, while high-risk and other operator obligations phase in by system category and implementation guidance (European Parliament and Council of the European Union 2024).

For systems engineers, compliance hinges on architectural capabilities rather than fine schedules. Covered high-risk systems[20] must implement risk management, data governance, technical documentation, transparency, human oversight, and accuracy, robustness, and security requirements. A covered credit-decision system therefore needs auditability from inception: model versions, training data provenance, validation evidence, human-oversight design, logging, and postdeployment monitoring must be part of the architecture rather than documents assembled after launch.

Contestability adds a second architectural requirement. GDPR moves the same applicant workflow into data-subject rights. Article 22 grants EU data subjects the right not to be subject to decisions based solely on automated processing that produce legal or similarly significant effects, subject

17 **Membership inference attack**: The attack uses model behavior to infer whether a record appeared in training; overfitting can increase the signal (Shokri et al. 2017). In practice, higher confidence on training samples can help distinguish members from nonmembers. Its measured success depends on model access, data distribution, and defense assumptions.

18 **Federated learning**: McMahan et al. (2017) introduced Federated Averaging (FedAvg), in which each client runs several local training steps before the coordinator averages the resulting models. Averaging after many local steps rather than after every gradient cuts the number of communication rounds, which is what makes training over constrained client uplinks practical.

19 **EU AI Act (Regulation 2024/1689)**: The first comprehensive AI legal framework, defining four risk tiers with penalties that vary by infringement category. The Act has extraterritorial reach: non-EU organizations may need to comply when they place systems on the EU market or when system outputs are used in the EU. Systems engineering implications are concrete: high-risk AI requires logging infrastructure for audit trails, human oversight mechanisms built into the architecture, and CE marking (the EU conformity mark), all capabilities that must be designed in from inception, not retrofitted after deployment.

20 **High-risk AI (EU AI Act Annex III)**: Annex III enumerates specified use cases in areas including biometrics, critical infrastructure, education, employment, essential services, law enforcement, migration, and justice. Classification depends on intended use and the Article 6 criteria, including exclusions for some systems that do not materially influence decisions or pose significant risk; profiling systems within Annex III remain high-risk. Model architecture alone does not determine classification.

to specified exceptions and safeguards.[21] Article 15(1)(h) separately gives data subjects access to meaningful information about the logic involved in automated decision-making referred to in Article 22. Engineering teams should determine which decisions are covered and design the required explanation and human-review capabilities accordingly. Where a substantive review path is required, it must be operationally staffed and supported by summaries, provenance, and audit tools. That staffing scales with decision volume. If 0.1 percent of 1M daily decisions are appealed or escalated, trained reviewers must work through 1,000 cases/day, a workload assumption rather than a model-error rate, and the review tooling must surface each case's explanation, lineage, and model version fast enough for that queue to clear.

US sectoral law reaches similar capabilities through domain-specific evidence requirements, less unified than the EU AI Act. In the credit example, the Equal Credit Opportunity Act (ECOA) and its implementing Regulation B require specific principal reasons for covered adverse actions, and those reasons must accurately describe the factors actually considered or scored (Consumer Financial Protection Bureau 2023). Healthcare regulation, including the Health Insurance Portability and Accountability Act (HIPAA)[22] and Food and Drug Administration (FDA) guidance, imposes the same pattern with different artifacts: protected-health-information controls, validation records, audit logs, and incident response. Across domains, the task is to translate each obligation into a concrete capability: explanation, human review, lineage, access control, deletion, monitoring, or incident response. Because the same safeguards recur across regimes, table 15.11 organizes them by capability rather than by jurisdiction.

Table 15.11: Regulatory Obligations as System Capabilities: Each obligation, named with an example source, maps to a capability the architecture must provide and to the resource that capability consumes. The mapping is illustrative rather than a legal checklist; applicability depends on jurisdiction, actor, data, and use.

Obligation (Example Source)	Capability the System Must Provide	Machine Cost
Risk management and record keeping for high-risk systems (EU AI Act)	Versioned models, data provenance, validation evidence, automatic event logs	Log storage and lineage-graph maintenance
Contestable automated decisions (GDPR Article 22)	Substantive human-review path with case summaries, explanations, and provenance	Reviewer hours that scale with decision volume
Reasons for a decision (GDPR Article 15; ECOA adverse-action notices)	Per-decision explanation or reason codes tied to the factors actually used	Explanation compute on the serving path
Data minimization and erasure (GDPR)	Retention limits, lineage-driven deletion, retraining or unlearning when weights are in scope	Retraining accelerator-hours; deletion workflows
Protected health information (HIPAA)	Access control, encryption, audit logs, retained documentation	Audit-log bytes over the retention window
Nondiscrimination (employment, credit, and housing law)	Disaggregated evaluation and monitoring against justified thresholds	Labeled per-group evaluation data; parity telemetry

Data-protection law reaches deeper into the pipeline than decision-level duties do, because its obligations attach to the data itself rather than to a decision. When a European user invokes an applicable right to erasure under GDPR, the voice assistant must determine which personal data and downstream artifacts are in scope and execute the appropriate workflow within the regulation's response deadlines (European Parliament and Council of the European Union 2016). These obligations become architecture constraints in pipeline design, storage choices, and operational procedures. GDPR's data minimization principle requires limiting collection and retention to what is necessary for stated purposes. Article 15 access rights cover personal data undergoing processing and specified information about that processing, not every artifact merely associated with a user.

Voice assistants operating globally face overlapping regulatory regimes because requirements vary by jurisdiction and apply differently based on user age and data sensitivity. GDPR cross-border-transfer rules permit transfers through adequacy decisions, appropriate safeguards, or limited derogations rather than imposing categorical data localization. These rules can still drive regional storage, replication, and processing choices. A structured data card turns these operational metadata requirements into reviewable fields. The card in figure 15.8, adapted from the Open Images Extended

[21] **GDPR (General Data Protection Regulation) articles 15 and 22**: Article 22 restricts certain solely automated decisions with legal or similarly significant effects and, for specified exceptions, requires safeguards including human intervention, the ability to express a point of view, and the ability to contest the decision; article 15(1)(h) contains the access right to meaningful information about the logic involved in such automated decision-making (European Parliament and Council of the European Union 2016). The European Data Protection Board's guidance emphasizes that required human oversight must be substantive and not merely a "rubber-stamping" exercise (European Data Protection Board 2018).

[22] **HIPAA (Health Insurance Portability and Accountability Act)**: Enacted in 1996, with Privacy Rule and Security Rule requirements establishing standards for protected health information (United States Congress 1996; U.S. Department of Health and Human Services 2003, 2005); PHI may be used for training under applicable authorization or another permitted pathway, such as an IRB or Privacy Board waiver; de-identified data and limited data sets with data-use agreements provide additional pathways. Model outputs may remain PHI when they identify or can reasonably identify individuals, and security-rule documentation retention must be reflected in audit and evidence design.

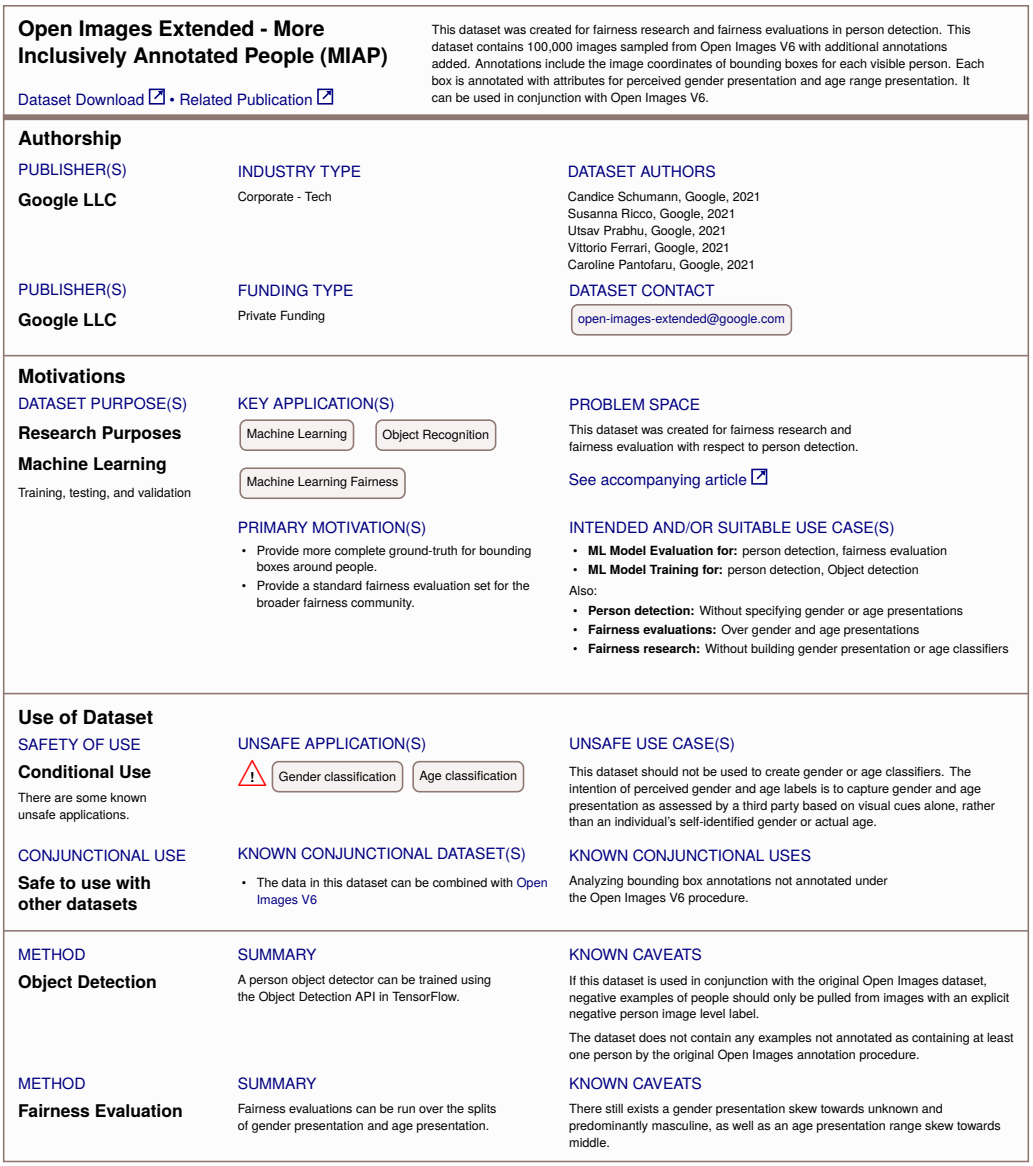


Figure 15.8: Data Governance Documentation: The populated Open Images Extended—More Inclusively Annotated People card organizes authorship, motivation, intended and unsafe uses, conjunctional use, and method caveats in one artifact. Recording permitted and prohibited uses makes the card reviewable governance evidence, though it does not establish compliance by itself. Adapted from the Open Images Extended—MIAP Data Card in Pushkarna et al. (2022).

(MIAP) specification (Pushkarna et al. 2022), partitions authorship, primary motivations, explicit safety prohibitions (such as forbidding age or gender classification), and methodological caveats. Although documenting these boundaries does not alone guarantee legal compliance, it provides the verifiable baseline necessary for pipeline ingestion checks.

The engineering response to these regulatory requirements is proactive architectural design. Teams that build documentation, monitoring, explainability, and human oversight into systems from inception demonstrate compliance efficiently. Teams that must retrofit these capabilities face expensive pipeline redesigns or deployment blocks. Nearly every regulatory obligation eventually asks the infrastructure the same two questions: which data produced this model, and who touched that data under what authority. Lineage and audit systems answer them.

15.5.4 Building data lineage infrastructure

When a regulator audits which training data produced a deployed model, or a user invokes an applicable right to erasure, the organization must answer from infrastructure rather than manual investigation. The lineage graph built for reproducibility in section 4.8.1 and section 14.3.1.2, a directed acyclic graph (DAG) of artifact versions and transformation tasks, is the same structure an audit queries.[23] Governance adds obligations to that graph rather than a new system. Ingestion records must link each KWS recording to its acquisition method and consent basis, and the use boundaries declared on a data card (figure 15.8) decide which training jobs may consume which feature partitions.

The obligation that lineage serves most directly is lineage-driven deletion. When an erasure request arrives, traversing the graph from the source recording identifies candidate derived artifacts (cached spectrograms, feature embeddings, training snapshots, and checkpoint weights) for scope determination and possible invalidation, retraining, or legal review. Deployment edges extend the same traversal to the field. Because lineage records which model version each device downloaded, a quality or compliance defect later found in the training data can be traced to the devices running the affected models and recalled there. Lineage captures provenance, but accountable operation also requires access history: who touched the data, when, and under which authority.

15.5.5 Audit infrastructure and accountability

Recording that access history for HIPAA or Sarbanes-Oxley (SOX) audits creates a write-amplification bottleneck if a production feature store logs the full feature vector and intermediate activation tensor on every read, because the audit log can outgrow the training dataset itself and blow past its storage budget while the dataset stays flat. Accountability trails[24] record access events to meet legal obligations, and for a fixed event schema, retained volume depends on recorded events per unit time, bytes per event, and the retention window. More accesses or instrumented stages raise the event count, while logging full feature payloads and intermediate activation tensors greatly increases bytes per event. Separating compact audit metadata from raw payload data controls that multiplier.

Production keyword spotting architectures resolve this trade-off through a tiered audit topology that reconciles capture fidelity with storage overhead and edge bandwidth limits. At the edge tier, microcontrollers operating within strict SRAM budgets log local state transitions, wakeup timestamps, and power states to volatile circular ring buffers. To conserve radio energy and uplink bandwidth, devices batch telemetry over secure transport rather than streaming it continuously. Scheduled maintenance windows or charging cycles can provide upload opportunities only if the required evidence survives until then.

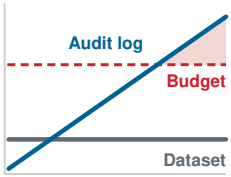
The retention requirement determines which events can use this volatile path. Dividing usable buffer capacity by the encoded telemetry byte rate gives the time to fill an empty buffer; the upload schedule must leave room for bursts and retries. Summaries may discard detail only when the retained fields still answer the specified audit questions. Events that must survive power loss require durable local storage until central storage acknowledges durable receipt, adding write energy and storage wear to the device budget. Sequence numbers and restart markers expose missing intervals after reconnection; an incomplete interval cannot establish that no incident occurred. If retention requirements exceed the available storage or uplink budget, the design must reduce optional telemetry or provision additional capacity before deployment.

Once telemetry reaches centralized infrastructure, auditing shifts from bandwidth-constrained edge devices to high-throughput feature stores. At this feature tier, offline and online feature stores emit append-only audit events for every read and write transaction. Rather than duplicating high-dimensional feature vectors, these events record requesting service identities, authorized subject pseudonyms, feature group hashes, and read timestamps. This metadata-only logging strategy enables fine-grained access verification while keeping log volume proportional to access transactions rather than payload sizes.

From feature stores, data flows into training pipelines running across accelerator clusters. At this training tier, cluster schedulers record deterministic manifests mapping training jobs directly to immutable data partition hashes, git commit SHAs, and hyperparameter configurations. While these manifests support checks of pipeline integrity and reproducibility, they reveal an inherent systems boundary: access logs confirm which records were processed, but they cannot demonstrate whether

23 **Data lineage systems**: Systems such as Apache Atlas and DataHub capture metadata about data flows from pipeline execution, creating graphs in which nodes are datasets and edges are transformations. GDPR Article 30 requires records of processing activities (European Parliament and Council of the European Union 2016), not automated model-level lineage specifically.

↩ Transformation lineage, which records the code version, parameters, and processing history behind each dataset, is established in section 4.8.1.



Audit volume can outgrow the storage budget while the dataset stays flat.

24 **Audit trail**: Audit integrity requires controls against unauthorized alteration, but retention and deletion duties vary by record type and law; append-only object storage with retention controls, or cryptographic hash chains, can provide tamper evidence without creating a universal rule that records are never deleted. A large platform may log billions of events daily; HIPAA's Security Rule also imposes multi-year documentation-retention obligations for required policies and procedures (U.S. Department of Health and Human Services 2005). Retention planning must distinguish records that must be preserved from personal data that must be minimized or deleted.

deleted user data persists within learned model weights. If the applicable erasure obligation covers a deployed model artifact, teams must evaluate retraining, validated **machine unlearning** (removing a training record's influence from learned weights without retraining from scratch (Cao and Yang 2015; Bourtoule et al. 2021)), or other legally appropriate controls; database access logs alone cannot establish removal of the training contribution.

Regulatory requirements extend audit responsibility from data access to prediction-time inference behavior. Reconstructing why the credit service denied a specific applicant in section 15.3.2.1, or why an edge voice assistant triggered a false wake-word activation, requires auditing the exact operational state at inference time. Without inference-time logging, an audit trail may confirm database access without capturing which model checkpoint, decision threshold, and runtime features produced a given decision. Saving raw input tensors for every inference request would overwhelm storage write bandwidth; production serving systems instead record compact, purpose-limited decision manifests. These manifests capture the active model version hash, runtime decision threshold, output confidence scores or reason codes, and cryptographic hashes of input features, supporting indexed retrieval of decision evidence. An input hash identifies content but cannot reconstruct it; replay also requires an appropriately retained source or an authorized recoverable representation, along with the relevant execution artifacts. Across these edge, feature, training, and serving tiers, governance uses Machine-axis logging and access control to prove Data-axis promises, yet comprehensive infrastructure still does not protect a team from its own intuitions, which the following fallacies and pitfalls collect.

15.6 Fallacies and Pitfalls

Teams can still fail after assembling assessment frameworks, fairness metrics, explainability mechanisms, efficiency analyses, and governance infrastructure. A team may retrofit fairness after benchmark success, trust aggregate accuracy, or treat compliance evidence as paperwork rather than system behavior, drawing on intuitions from traditional software engineering where bugs are local and testing is deterministic.

Fallacy: *Responsibility can be addressed after the system achieves technical objectives.*

Teams assume fairness constraints can be retrofitted once models demonstrate strong benchmark performance. In production, early architectural decisions constrain what interventions remain feasible. Amazon's recruiting tool, abandoned after remediation failed (section 15.2.1), is the case in point. Integrating fairness constraints early can be less costly than retrofitting them after data contracts, monitoring, and release gates are fixed.

Pitfall: *Relying on aggregate metrics to assess fairness.*

Engineers assume high overall accuracy indicates the system works well for all users. The flaw of averages (section 15.3.2) shows why this intuition fails, because aggregate metrics can conceal large subgroup disparities. The credit-service analysis in section 15.3.2.1 showed a 30 percentage-point TPR gap, with qualified minority applicants rejected at 4× the majority-group rate. These disparities can persist undetected when standard monitoring tracks only aggregates. Production systems require disaggregated evaluation with application-specific thresholds, such as the 5-percentage-point TPR threshold used for the credit service.

Fallacy: *Removing sensitive attributes from training data eliminates bias.*

Teams remove gender, race, and protected attributes expecting this ensures fairness. Other features can act as proxy variables when they correlate with sensitive characteristics. ZIP codes, purchase patterns, browsing history, college names, and language choices can carry indirect demographic signal. Amazon's recruiting tool kept encoding sex through college and organization names after explicit gender terms were removed (section 15.2.1), and the care-management algorithm (war story 15.1) reproduced racial disparity through its cost target, a proxy that no attribute filter would remove. Removing protected attributes does not by itself establish fairness.

Pitfall: *Treating documentation as sufficient accountability.*

Teams invest effort in model cards, then consider responsibility requirements satisfied. Documentation provides transparency (section 15.3.3.2) but not enforcement. A model card specifying "not

validated for high-stakes decisions" has no effect when the system is repurposed for loan approvals without technical restrictions. Accountability requires operational integration: monitoring dashboards, documented subgroup-disparity alert thresholds, incident response procedures, and access controls preventing deployment beyond validated use cases.

Fallacy: *Responsible AI is primarily a legal compliance issue.*

Teams treat responsibility as external oversight rather than engineering practice. Engineering decisions made months before legal review constrain the solution space more than any compliance assessment. Architecture selection determines what fairness interventions are feasible, while data pipeline design establishes whether disaggregated evaluation is even possible. As section 15.2.5 establishes, systems designed with responsibility as an engineering objective enable efficient validation; systems where responsibility is added at late-stage review face redesign or deployment with documented risks.

Pitfall: *Measuring the environmental impact of training but not inference.*

Public discourse focuses on training-run carbon, and engineers often follow this framing when assessing environmental responsibility. The illustrative TCO analysis in section 15.4.2 shows why this focus is incomplete: under its unbatched service-demand assumptions for the DLRM ranking service (table 15.10), the inference-to-training cost ratio is about 40:1, and inference emits about 63× as much CO_2 as training. A model trained periodically but served millions of times daily can have a lifecycle footprint dominated by inference rather than training. These values are scenario-dependent, but they show why lifecycle accounting must include both terms.

Fallacy: *Model weights are exempt from data governance and deletion requests.*

Teams often assume that once training data has been compiled into model weights, the data is gone and compliance obligations cannot reach the artifact. In reality, models can memorize training data, and membership inference attacks can reveal whether a sensitive record appeared in the training set (Shokri et al. 2017). Unlike a database delete, a neural network diffuses each training point's statistical influence across its parameters through gradient updates, so verifiably purging that influence forces systems trade-offs. Exact retraining from scratch on a cleaned dataset provides a reference for removing the excluded records' contribution, but it re-expends the training budget (potentially thousands of accelerator-hours) and its carbon emissions, and it does not erase other copies or facts inferable from retained data. Machine unlearning, introduced with audit infrastructure (section 15.5.5), aims to bound that cost, but verifying that an intervention removes statistical influence without degrading model utility remains an open systems challenge. Engineering teams must track data-to-model lineage and evaluate retraining, machine unlearning, or compensating controls with legal and privacy specialists whenever an erasure request affects deployed artifacts.

Across these failures, the recurring mistake is to treat responsibility as something added after the system is built rather than as a system property. Measurable constraints, continuous monitoring, and enforceable governance carry responsibility through the full lifecycle.

15.7 Summary

Responsible engineering is ML systems engineering done completely, not a separate discipline. The chapter traced a path from failure diagnosis through prevention to enforcement, beginning with the responsibility gap (the distance between technical performance and responsible outcomes) and demonstrating how proxy variables, feedback loops, and distribution shift can harm users while conventional metrics remain acceptable. The engineering response follows the model's lifecycle: specification that fixes targets, populations, and oversight before training forecloses them, fairness metrics that make disparities measurable, explanation and documentation selected for applicable stakeholder and regulatory requirements, and monitoring infrastructure that can surface silent failures and connect signals to response.

Translating responsibility concerns into measurable properties makes them tractable. A justified, application-specific bound on disparity is testable; "be fair" is not. This translation extends beyond fairness: in the chapter's illustrative TCO scenario, a 20 percent accelerator-service-demand reduction saves $304K and is modeled to avoid 19 t of CO_2 under the stated assumptions. Documentation becomes model cards with explicit intended use and known limitations. Governance becomes

access control, lineage tracking, and audit infrastructure that makes compliance evidence available rather than merely aspirational. Across these cases, abstract ethical obligations become concrete engineering requirements that can be specified, tested, monitored, and enforced.

Every row of the chapter map (table 15.1) now carries a measured machine cost: labeled evaluation data in proportion to how rare each named population is, a second network head for mitigation, explanation compute that grows exponentially with feature count for exact attribution, randomized exploration traffic to keep feedback in check, accelerator-hours and tonnes of CO_2 for the DLRM service, and log bytes, privacy noise, and retraining hours for governance. These practices share one principle: a responsibility property can be enforced only at a lifecycle stage that still has the freedom to change it. Measurement can gate only the populations that specification named, documentation can record only what measurement produced, and governance can prove only what the pipeline retained. A system that defers fairness, efficiency, transparency, or governance to a late review is therefore incomplete in the same sense as a system shipped without a latency budget; the requirement exists, but nothing in the architecture can meet it. The chapter's quantitative tools fit in one table (table 15.12), and each turns a responsibility requirement into a number a team can compute before launch and monitor after it.

Table 15.12: Key Equations: The chapter's responsibility quantities in the notation of the sections that derive them. Each converts a requirement (a fairness criterion, an evaluation budget, an explanation path, a feedback horizon, a carbon footprint, or an audit trail) into a value that can be computed before launch and monitored after it.

Quantity	Formula	Where Derived
Alignment gap	$\text{Gap} = \mathbb{E}[\text{Proxy}] - \mathbb{E}[\text{True}]$ for metrics on a common scale	equation 15.1
Demographic parity	$\Pr(\hat{Y} = 1 \mid G = \text{A}) = \Pr(\hat{Y} = 1 \mid G = \text{B})$, gated on the difference	section 15.3.2.1
Equal opportunity	$\Pr(\hat{Y} = 1 \mid Y = 1, G = \text{A}) = \Pr(\hat{Y} = 1 \mid Y = 1, G = \text{B})$	section 15.3.2.1
Equalized odds	Equal opportunity for both $v \in \{0, 1\}$, so true and false positive rates both match	section 15.3.2.1
Slice sample size	Worst-case margin $1.96\sqrt{0.25/n}$; random collection $D_{\text{eval,total}}$ = per-group target divided by the group's share	section 15.3.1
Exact attribution cost	$\mathcal{O}(2^{n_{\text{feat}}})$ model evaluations per explained decision	section 15.3.3.1
Bias feedback	$\Delta_g(k) \approx \Delta_g(0) \cdot \alpha_{\text{fb}}^k$	section 15.3.4
Operational carbon	$\text{Energy}_{\text{hardware}} \times \text{PUE} \times \text{Carbon Intensity}$	section 15.4.3
Audit log volume	Recorded events per unit time, times bytes per event, times the retention window	section 15.5.5

Key Takeaways: Reliable for whom?

- **Aggregate correctness hides harm**: A model can look accurate in aggregate while one subgroup's error rate is 43.1× another subgroup's rate, as in the Face++ Gender Shades audit. Responsible evaluation therefore starts with disaggregated and intersectional slices, not aggregate accuracy alone.
- **Responsibility becomes testable through thresholds**: "Be fair" is not testable, but bounded disparity, documented intended use, and explainability requirements are. Translating values into measurable constraints lets teams review trade-offs among fairness, accuracy, latency, and cost.
- **Efficiency is a social constraint**: An efficient model can reduce energy and cost and broaden who can deploy it. In the chapter's illustrative scenario, the inference-to-training cost ratio is 40:1, making per-query optimization responsible engineering.
- **Monitoring must watch outcomes**: Bias and privacy failures can continue with green uptime dashboards because harmful predictions look operationally normal. Production monitoring must track subgroup outcomes, data lineage, feedback loops, and incident paths with the same rigor as latency regressions.

- **Governance has to be built in**: Model cards, datasheets, access controls, erasure workflows, human-review paths, and audit trails are technical infrastructure, not optional policy overlays. Regulations such as GDPR require capabilities that should be designed into the pipeline; later remediation may remain possible but can be narrower and costlier once the system is serving decisions.

A system that does exactly what it was told is dangerous precisely because the telling is never complete. Every objective a model is given is a specification with gaps, and an optimizer is a machine for finding them. It can reproduce the bias latent in its data, chase the proxy instead of the goal, and call the result success because nothing in the objective said otherwise. Responsible engineering is the discipline of writing back in what the specification left out. Constraints and monitoring reduce the risk that optimization amplifies harms encoded in the data. The constraint is the same kind the rest of the book imposed in latency and memory, except that here it protects people the objective never named, and a model that is fast and accurate while wrong about whom it serves has not failed less than one that crashes, only more quietly.



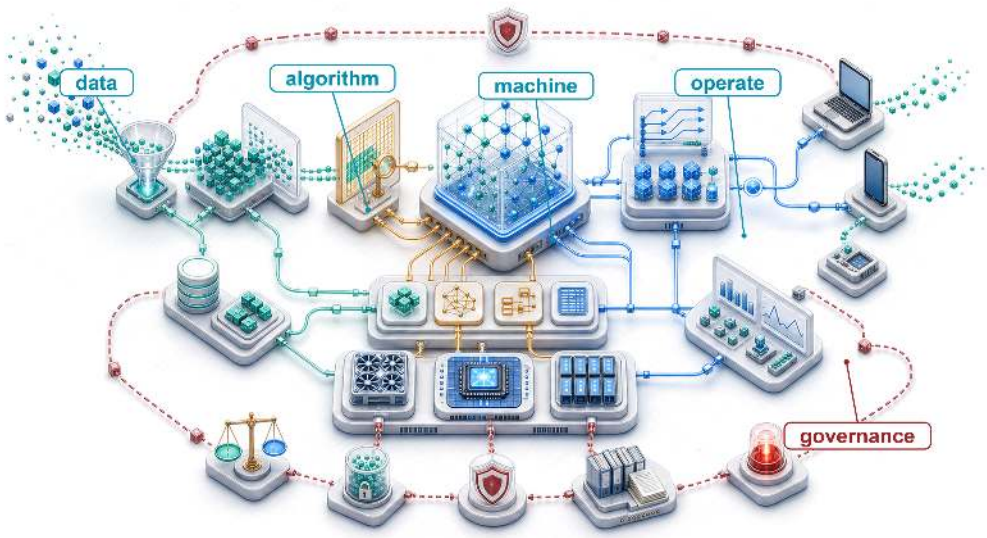
What's Next: From local targets to the whole system

Responsibility was the last constraint this book added, and it behaves like the ones before it. A disparity bound, a carbon budget, or a lineage requirement can be met only if every upstream stage preserved the freedom to meet it, exactly as a latency budget can. Every constraint so far has been taught one layer at a time, yet the costliest failures arise where layers interact, when each component meets its own target and the system still misses its goal. Chapter 16 returns to the voice-assistant request that opened the book, follows it across every layer the chapters built, and gathers their quantitative principles, from the iron law (principle 3) onward, into one framework for reasoning across the whole stack.

SYNTHESIS

16

Conclusion

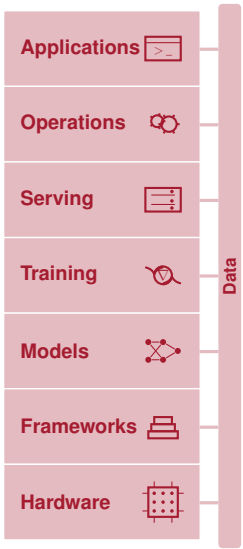


16.1 The System Is the Model
16.2 Lighthouse Models: Constraint Propagation
16.3 Thirteen Quantitative Principles
16.4 The Dual Mandate Revisited
16.5 Future Directions
16.6 The Engineering Responsibility
16.7 Fallacies and Pitfalls
16.8 Summary

Purpose

What does mastering the full stack enable that expertise in any single layer cannot?

A single production decision travels through the entire stack. A data pipeline decides which events count as training signal. That signal shapes the architecture, which in turn fixes memory footprint and arithmetic intensity. Those properties constrain hardware choice, quantization strategy, serving latency, drift monitoring, and governance obligations. Mastered individually, each layer represents a valuable skill; mastered together, they confer the ability to reason across boundaries that no single specialty sees in full. An engineer who understands only compression can shrink a model, but cannot predict whether the accuracy loss matters for the deployment context. An engineer who understands only serving can optimize latency, but cannot trace a performance regression to a data pipeline change three stages upstream. Many of the costliest production failures arise at these boundaries, where every component passes its local tests yet the system as a whole misses its target. In D·A·M terms, what endures is the ability to analyze a system before it is built, reasoning about how data, algorithm, and machine constraints will interact, where bottlenecks will emerge, and which architectural commitments will prove irreversible. The constraints that govern these interactions outlive any specific framework, hardware generation, or model family, because the cost of moving data, the speed of light, and the gap between an assumed and a measured bottleneck do not turn over with them. Thinking in systems rather than components separates an engineer who can build a part from one who can build the whole.



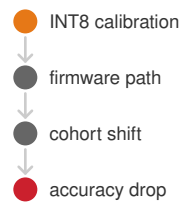
Learning Objectives

- Trace how data, architecture, compression, hardware, serving, operations, and governance decisions propagate constraints across an ML system
- Apply lighthouse-model reasoning to diagnose bottlenecks across cloud, mobile, edge, recommendation, and TinyML deployments
- Synthesize core ML systems principles into a framework for reasoning across Data, Algorithm, and Machine constraints
- Evaluate deployment trade-offs using latency budgets, memory and energy movement, drift, and responsibility constraints
- Justify an engineering approach for emerging contexts such as embodied, composed, and multi-node systems, using cross-layer evidence to identify where single-node reasoning must widen its boundary

16.1 The System Is the Model

When a user asks a smartphone voice assistant a question, learned components convert speech to text, interpret intent, retrieve information, and generate a response within seconds (section 1.1), and a failure in any stage—a drifted feature, a miscalibrated quantization range, or a queueing spike—degrades the answer even when the model weights are untouched. **The system is the model**: the behavior a user receives from a machine learning service is produced by the data pipeline, the trained weights, the execution stack, the serving path, and the monitoring loop together, never by the weights alone. Every chapter since the introduction has given one stage of that request a mechanism and a cost. Chapter 15 added the last constraint the book imposes, the obligations a system owes the people it affects, and showed that it behaves like the latency and memory budgets before it, because it can be met only where every upstream stage preserved the freedom to meet it. Each constraint was taught one layer at a time. The costliest failures arise where the layers meet.

A composite scenario, extending the calibration pitfall analyzed in section 13.11 rather than reporting any one incident, shows such a failure. To deploy an image classification model across a fleet of mobile devices, the architecture team chooses depthwise separable convolutions to factorize spatial filtering from channel projection, the compression team quantizes the resulting weights and activations to INT8 so they fit within on-chip static RAM (SRAM), and the serving team brings end-to-end p99 latency inside a 50 ms target. Each team succeeded by its local metric, yet within weeks, accuracy dropped by four percentage points on specific firmware and device cohorts. INT8 quantization fixes its clipping ranges during quantization-range calibration, permanently encoding the statistical distribution of the calibration images. A firmware-specific preprocessing routine interpolated images differently and presented an altered input distribution to the quantized model on those cohorts. No component failed in isolation, and nothing crashed. Each team validated its component against standard inputs, and while latency was evaluated end to end along the deployed request path, accuracy was measured only as an aggregate scalar over an offline evaluation set, so users on the affected cohorts simply received worse predictions that the system's monitoring was never built to see. Heterogeneous edge environments require disaggregated cohort monitoring from day one.



A calibration choice surfaces as a cohort accuracy drop three stages downstream.

Most failures of this kind start at an interface between components, so the main task is to test the interfaces before they fail. Traditional software engineering treats functional correctness as decoupled from the underlying hardware; in machine learning systems, statistical behavior is physically inseparable from the pipelines, memory hierarchies, and accelerators that train, serve, and monitor the model.

The bitter lesson teaches that general methods scaling with computation outrun hand-encoded heuristics (section 1.4). Affording that scale is the job of the iron law of ML systems (principle 3), which sits at the center of this discipline. Its three terms (data movement, compute, and overhead) serve as the primary engineering levers for quantitative analysis. Building intelligence requires both statistical algorithms and adherence to the silicon contract (principle 4), the requirement that the

resource a model saturates first be the one the executing machine supplies in abundance. Arithmetic intensity and roofline modeling convert qualitative performance intuitions into quantitative upper bounds (Williams et al. 2009).

System capabilities emerge from co-design across the Data, Algorithm, and Machine (D·A·M) triad rather than from isolated algorithmic breakthroughs, and success and failure arise from the same mechanism, cross-layer coupling. The transformer architecture (Vaswani et al. 2017) scaled into models such as GPT-3 and Llama 2 (Brown et al. 2020; Touvron, Martin, et al. 2023) only because decisions in other layers made attention affordable: IO-aware attention kernels that keep intermediate scores in on-chip memory, low-precision tensor units that raise the peak arithmetic rate, and partitioning of tensor state across accelerators.

↩ IO-aware attention tiling, which keeps attention scores in on-chip memory, is established in section 8.6.4.

Integration has concrete consequences. Software engineering often treats the "model" as a weights checkpoint, a single file of floating-point numbers. In a production environment, however, the weights are only one component of the true model, and often not the most fragile. A model that produces high benchmark accuracy fails if it ingests corrupted features, and a model that converges cleanly during training fails if it cannot satisfy deployment latency constraints.

↩ The finding that model code is a small fraction of a production ML system, whose surrounding pipelines, serving, and monitoring are larger and more enduring, is established in section 1.2.

Checkpoint 16.1: Systems thinking

The system's dependencies, feedback loops, and request path make its boundaries testable.

The integration

- ☐ **Dependencies**: can you trace a data-pipeline change through training and serving to identify its latency effect?
- ☐ **Feedback loops**: have you mapped how the model's predictions influence its future training data?

The holism

- ☐ **End-to-end**: can you trace a user request from the user interface, through the network, preprocessing, model, postprocessing, and back to the user interface?

Tracing a request end to end exposes the coupling behind such failures. Every layer of the stack enforces physical and statistical constraints on the next. Data curation and selection bound what the learner sees; architectures, training systems, and frameworks lower the resulting computation onto hardware; compression, acceleration, and benchmarking move the physical boundaries; and serving, operations, and responsible engineering hold the result to tail-latency budgets, drift, and subgroup outcomes in production. Because these stages form a tightly coupled system rather than a sequence of isolated handoffs, engineering decisions never remain localized, and an engineer who optimizes a single layer in isolation cannot predict how the change will propagate.

The rest of the chapter makes that claim quantitative. The five lighthouse models come first, each traced from its home regime to the lever that paid off and the lever that was wasted. The thirteen quantitative principles follow, regrouped by the three questions the introduction asked of every ML system: what shapes its predictions, what limits its execution, and what evidence supports its deployment. The dual mandate is then measured on two lighthouses, DLRM for trustworthy behavior and the GPT-2 family for physical deliverability. The same principles then forecast where constraints bind next, from microcontrollers to composed services and fleets. The chapter closes on the responsibility that follows from building such systems, the cross-boundary forms of the book's recurring mistakes, and a final trace of the voice-assistant request through every layer.

16.2 Lighthouse Models: Constraint Propagation

Adding peak arithmetic throughput to a batch-one large language model (LLM) decoder leaves token latency untouched because every token waits on high-bandwidth memory (HBM) weight reads; adding memory bandwidth to a batched ResNet-50 convolution buys nothing once its arithmetic intensity sits right of the ridge point. The five lighthouse models each have a home regime, the binding resource that the workload exposes under its typical operating conditions: ResNet-50 is compute-bound, GPT-2 and Llama decode is bandwidth-bound, DLRM is capacity-bound, MobileNetV2 is compute-bound in its efficient regime, and keyword spotting (KWS) is power-bound. Across the book, each lighthouse rewarded the lever aimed at its binding term and wasted the effort aimed anywhere else (table 16.1).

↩ The five lighthouse models and their diagnostic execution regimes are established in section 1.7.1.

Table 16.1: Lighthouse Constraint Traces: Each lighthouse probes one home regime, and the lever that paid off was the one aimed at that regime's binding term. A regime describes a workload on a machine rather than a model name, so a lighthouse crosses regimes when batch size, phase, or precision changes.

Lighthouse	Home Regime	Binding Term	Lever That Paid	Lever Wasted
ResNet-50	Compute-bound	Compute, $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	Batching that reuses filter weights across inputs; reduced-precision tensor arithmetic	More memory bandwidth once batched convolutions sit right of the ridge point
GPT-2/Llama decode	Bandwidth-bound	Data movement, D_{vol}/BW	Batching across requests; fewer bytes per weight; managing the per-request key-value cache	More peak arithmetic throughput, or compute-only kernel tuning at batch one
DLRM	Capacity-bound	Memory capacity, a precondition before any term applies	Placing and sharding embedding tables across memory tiers; caching frequently accessed rows	Faster dense arithmetic for the small fully connected layers
MobileNetV2	Compute-bound in its efficient regime	Compute within a phone's power envelope; depthwise layers cross to bandwidth-bound	Depthwise separable convolutions, then INT8 once depthwise layers became bandwidth-bound	Further operation cuts on layers already bound by bandwidth
Keyword spotting (KWS)	Power-bound	Energy per inference within a milliwatt budget	A sub-megabyte INT8 model with a tensor arena planned at build time	Streaming audio to the cloud, whose radio transmission costs more energy than local inference

The regime belongs to the workload on a machine, so the same model can change regime, and the regime has to be checked before an optimization is chosen. GPT-2 training is compute-bound even though its decode is bandwidth-bound, because a training step reuses each weight across every token in the batch. ResNet-50 became input-bound in chapter 8 when the data pipeline could not keep the accelerator fed, and INT8 arithmetic, which doubles the H100's peak rate without raising its memory bandwidth, doubles the ridge point, which leaves some ResNet-50 layers below it and bandwidth-bound. DLRM shows the same pattern at the large end of the deployment spectrum, where embedding capacity decides which term binds; MobileNetV2 and keyword spotting show it at the small end, where thermal and power envelopes do. A larger recommendation model, a larger decoder, or a faster vision network differs from its lighthouse in scale, not in which term binds, so the diagnosis carries forward to models that do not yet exist.

Applying each phase of the book to MobileNetV2 shows how cross-layer constraints converge on a single engineering artifact (table 16.2). Across nine phases, from the deployment tier's envelope through architecture, training, data selection, compression, acceleration, benchmarking, serving, and runtime operations, each engineering decision narrows the options available downstream.

Table 16.2: The Lighthouse Journey (MobileNetV2): Applying the book's nine phases to one model shows how decisions in one domain (for example, architecture) propagate constraints and opportunities into downstream domains (for example, hardware acceleration and monitoring).

Journey Phase	System Lens	MobileNetV2 Implementation
Foundations (chapter 2)	Deployment Tier Constraints	Mobile tier sets a watt-scale power, thermal, and battery envelope that every later phase must respect
Architecture (chapter 6)	Algorithmic Efficiency	Depthwise Separable Convolutions: 8.7× fewer FLOPs for a representative 3-by-3, 256-output-channel layer and 13.7× fewer operations than ResNet-50 at ImageNet scale
Training (chapter 8)	Throughput vs. Latency	Trained at high-throughput batch sizes for a model that will serve single requests at low latency
Data Selection (chapter 9)	Capacity-Matched Augmentation	Milder augmentation matched to a compact network's limited capacity, validated against deployment invariances such as device and lighting variation, at no inference cost
Compression (chapter 10)	Navigating the Pareto Frontier	INT8 Quantization: FP32 uses 4× and FP16 uses 2× the per-value storage of INT8, with accuracy revalidated per deployment
Acceleration (chapter 11)	Honoring the Silicon Contract	Mapping kernels to mobile neural processing units (NPUs) (for example, Apple Neural Engine) to maximize hardware execution efficiency
Benchmarking (chapter 12)	Component vs. End-to-End	Confirming that compression and accelerator gains survive the full pipeline by separating accelerator latency from preprocessing and transfer, and rechecking INT8 accuracy on edge cases such as unusual lighting
Serving (chapter 13)	Respecting the Latency Budget	p99 < 50 ms constraint; optimizing preprocessing (resize/normalize) to avoid CPU bottlenecks
Operations (chapter 14)	Managing System Entropy	Drift Monitoring: Detecting distribution shifts and tracking cohort-level accuracy across heterogeneous device populations and lighting conditions

Decisions in one row constrain options in the next. Depthwise separable convolutions cut the operation count, but depthwise filtering reduces cross-channel reuse and often leaves these layers bandwidth-bound on mobile hardware. When profiling confirms that regime, a further cut in operations buys little, and the compression row's reduction in bytes per value becomes the next lever. That lever pays only if the acceleration row confirms that the mobile NPU executes those INT8 operators, and only if the benchmarking row shows the gain surviving preprocessing and transfer. The same propagation runs through every lighthouse, and reasoning about it before profiling requires the quantitative principles the book accumulated part by part.

16.3 Thirteen Quantitative Principles

When an engineer alters an embedding table layout, doubles batch size, or quantizes an attention layer from FP16 to INT8, the performance outcome is not random; it is governed by physical bounds, empirical trade-offs, and operational feedback. Reasoning quantitatively about this behavior across the entire stack requires analytical tools with stated assumptions, and the book accumulated **thirteen quantitative principles** for that purpose across its four parts (table 16.3). They are not all the same kind of statement. Three are hard bounds (the Pareto frontier, the arithmetic intensity law, and Amdahl's Law), two are decompositions (the iron law and the energy-movement principle), two are fitted models (statistical drift and bias feedback), two are policies whose parameters a product decision sets (the verification gap and the latency budget), and four are heuristics (data-as-code, data gravity, the silicon contract, and training-serving skew). The heuristic label marks the silicon contract as a prediction about which resource an architecture saturates first, a prediction that the arithmetic intensity bound then tests. Each principle box on the part pages names its kind, and each principle is reliable only within its stated assumptions.

Table 16.3: Thirteen Quantitative Principles: Each tool appears in the part where its governing constraint or diagnostic use first becomes visible. The collection combines bounds, decompositions, fitted models, policy requirements, and heuristics. Each row holds under its stated assumptions; together they provide a shared analytical vocabulary for system design, optimization, and deployment.

#	Principle	Part	Core Equation/Statement	What It Predicts
1	Data-as-Code Principle	I: Foundations	Learned behavior ≈ Train(training data, training procedure)	Data changes behavior; other inputs also matter
2	Data-Gravity Principle	I: Foundations	Move compute toward data when repeated transfer costs exceed placement costs	Depends on volume, reuse, network, and compute mobility
3	Iron Law of ML Systems	II: Development	$T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$; overlap ranges from max to sum	D·A·M factors multiply; stages add or overlap
4	Silicon Contract	II: Development	Each architecture saturates one resource (compute, bandwidth, or capacity) first	Design must target the resource saturated first
5	Pareto Frontier	III: Optimization	No feasible point is at least as good on every objective and strictly better on one	Along the frontier, improving one objective worsens another
6	Arithmetic Intensity Law	III: Optimization	$R_{\text{attain}} \leq \min(R_{\text{peak}},\ I \times \text{BW})$	More compute cannot raise a bandwidth ceiling
7	Energy-Movement Principle	III: Optimization	$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$; dynamic RAM (DRAM) to FLOP cost ratio: 173–582×	Total energy depends on bytes moved, operations, and per-event costs
8	Amdahl's Law	III: Optimization	$\text{Speedup} = \frac{1}{(1-f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{G}}$	For fixed work, the serial fraction caps parallelism gains
9	Verification Gap	IV: Deployment	Deployment-population success probability must meet the specified target at the stated task-error tolerance	Specify distance, tolerance, population, and required success probability
10	Statistical Drift Diagnostic	IV: Deployment	$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$	A local fit; drift need not lower quality
11	Training-Serving Skew Diagnostic	IV: Deployment	$S_{\text{skew}} = \mathbb{E}[\lvert f_{\text{serve}}(x) - f_{\text{train}}(x)\rvert]$	Output mismatch signals risk, not accuracy loss
12	Latency Budget Principle	IV: Deployment	$\Pr[L_{\text{lat,total}} \leq T_{\text{deadline}}] \geq q$, where $L_{\text{lat,total}}$ sums the request-path delays	The product SLO selects q and the deadline T_{deadline}
13	Bias Feedback Model	IV: Deployment	$\Delta_g(k) \approx \Delta_g(0)\alpha_{\text{fb}}^k$ with fitted α_{fb}	Feedback may amplify harm; measure and intervene

Notation Guide for table 16.3: In deployment models, d_{task} is the task-specific distance metric, τ_{tol} is the acceptable error tolerance threshold, q is the target success probability for latency deadline T_{deadline}, and α_{fb} is the amplification factor compounding disparity Δ_g at step k. The drift row restates the degradation equation (equation 1.1), in which λ is the sensitivity fitted from labeled outcomes and $\mathcal{D}(P_t \| P_0)$ measures how far the current input distribution has moved from the training distribution. S_{skew} is the expected absolute difference between serving-path and training-path outputs on the same input. In efficiency bounds, G is the speedup factor applied to the parallelizable fraction f_{parallel}. Finite tests estimate the deployment success probability with uncertainty; an evaluation must separately state the statistical confidence level of its interval or bound.

The thirteen principles are not independent axioms. The conservation-of-complexity heuristic[1] introduced in Part I connects them into an integrated framework. In Part III, model compression demonstrated this trade-off at the scale of a single model: pruning sacrifices capacity, quantization sacrifices precision, and distillation expends training compute. At the scale of the entire system the discipline is the same. After any local simplification, trace where the displaced complexity has migrated. The heuristic serves as a diagnostic rather than a physical conservation law. While thoughtful design can eliminate accidental complexity outright, systemic complexity often shifts elsewhere, as when quantizing a model moves it into validation.

The part pages introduced the principles in the order the book needed them. Read together, they sort more usefully by the three questions the introduction asked of every ML system (section 1.12): what shapes its predictions, what limits its execution, and what evidence supports its deployment. The three groupings that follow keep each principle's number, so every principle can still be found on its part page.

1 | **Conservation-of-complexity heuristic**: Tesler's Law, a design aphorism, says that an application's irreducible complexity must be handled somewhere in the interaction among user, application, and platform (Tesler 1984). Extending it to all ML-system complexity is an analogy, which is why the heuristic directs a search for displaced costs rather than guaranteeing that an equal compensating cost exists.

What shapes predictions (principles 1 and 13)

The data-as-code principle (1) answers the first question directly. The training dataset dictates learned behavior, so a modification to data curation is a behavioral change requiring the same rigorous versioning, code review, and regression testing applied to source code. The bias feedback model (principle 13) extends that answer past launch, because a deployed model's predictions shape the data its successor trains on. It treats disparity as a quantity that compounds across retraining cycles at a fitted rate, which turns fairness into a stability question about the deployment loop. An automated credit model may satisfy throughput and convergence targets while systematically compounding disparities against underserved groups, so the feedback loop must be intercepted by disaggregated monitoring rather than discovered in aggregate accuracy.

↩ The bias feedback model, with the fitted factor that projects when a disparity that passed its release gate will breach it, is established in section 15.3.4.

What limits execution (principles 2–8)

The data-gravity principle (2) opens the second question, because data volume dictates where computation must execute. Moving a petabyte-scale dataset between cloud regions costs days of transfer time and substantial egress fees, which readily justifies moving compute to the data rather than the reverse.

The lighthouse models carry the same placement constraint into execution. In DLRM, embedding tables that outgrow a single accelerator's memory make memory capacity a binding design constraint, requiring the execution system to be engineered around where the parameters physically reside. Compute-to-data placement recurs across deployment environments precisely because moving large, low-reuse datasets saturates interconnects.

↩ The transfer time and egress cost that make a large dataset the gravitational center of an architecture are established in section 4.9.1.

The iron law (principle 3) and the silicon contract (principle 4) govern execution efficiency. The iron law's multiplicative D·A·M form connects data scale (Samples/Task), algorithmic work (FLOPs/Sample), and machine pace (Seconds/FLOP), while its three-term physical decomposition isolates data movement, computation, and overhead; the silicon contract reveals which term limits performance on a given architecture-hardware pair. Actual shapes, kernels, and runtime conditions determine which regime of table 16.1 applies. Diagnosing whether an execution path is compute-bound, bandwidth-bound, or capacity-bound determines which optimizations yield speedup and which waste engineering effort. Execution time drops most when engineers target the binding term directly rather than distributing effort uniformly.

The remaining four principles of the second question form a tightly coupled diagnostic chain. Where the Pareto frontier (principle 5) identifies the designs that no alternative beats on every objective at once—so that improving one objective along the frontier requires a trade-off against another within the same feasible design set—the arithmetic intensity law (principle 6) diagnoses whether compute or bandwidth sets the ideal ceiling at each point along it. Moving bytes across that boundary also carries a power cost, which the energy-movement principle (principle 7) captures by combining per-event costs with event counts: under the reference energy constants, one DRAM access costs about 173–582× as much as one arithmetic operation, with the DRAM-versus-FP32 ratio from hardware acceleration (chapter 11) at the lower end and the cheaper FP16 operation at the upper end, though which cost dominates a whole workload still depends on how many of each event occur. Even after memory traffic and kernel execution are tuned, Amdahl's Law (principle 8) sets the ceiling on any parallelism gain, explaining why data loading and preprocessing can become bottlenecks in highly optimized systems.

↪ The bottleneck diagnostic that pairs each binding iron-law term with the optimizations that work and those that are wasted is collected for reference in section A.5.3.

MobileNetV2 touches all four. Depthwise separable convolutions reshape the Pareto frontier (Sandler et al. 2018). INT8 quantization exploits the arithmetic intensity law by raising arithmetic intensity through reduced memory traffic (Jacob et al. 2018). Measured energy savings, where the reduced traffic and supported execution lower total consumption, align with the energy-movement principle, while Amdahl's Law bounds end-to-end speedup when host-side preprocessing remains unaccelerated. Microcontroller workloads such as KWS compress these trade-offs further, operating under sub-megabyte memories that leave little room for avoidable overhead.

↩ The compute-versus-bandwidth-boundary and hardware balance (I_{ridge}) are established in section 11.6.2.

What evidence supports deployment (principles 9–12)

The third question concerns failures that bench testing cannot rule out, because a system can work correctly on the bench yet fail silently in production. The verification gap (principle 9) specifies the required success probability over a stated deployment population and error tolerance; finite

↩ Compression moves a model along the accuracy-efficiency frontier, as established in section 10.5.1.

testing estimates whether the system meets that requirement at a stated statistical confidence level, but it cannot establish correctness for every future input. Once the model is serving live traffic, the statistical drift diagnostic (principle 10) detects when that deployment population has changed and labeled outcomes fit how much the shift costs in accuracy, whereas the training-serving skew diagnostic (principle 11) catches internal divergence when training and serving paths compute the same features differently, and the latency budget (principle 12) holds the entire request path to the tail quantile the product selects.

↩ Reusing training-time transformation parameters at serving time, the consistency imperative that prevents skew, is established in section 4.7.1.

Monitoring infrastructure, drift detection, feature stores, and disaggregated subgroup metrics exist to make these principles measurable and to trigger corrective action. A recommendation system that achieves high offline accuracy still requires parity checks when training-serving skew corrupts feature pipelines (principle 11) and outcome auditing when user behavior shifts seasonally (principle 10).

↩ The monitoring infrastructure and drift quantification that surface these silent failures are established in section 14.5.

The integrated framework

The thirteen principles are not a sequence to apply in order. They constrain one another, and a single decision activates several of them in turn.

Quantizing a model from FP16 to INT8 illustrates these mutual constraints in action. This single intervention navigates the Pareto frontier (principle 5), trading numerical precision for memory traffic, and its consequences spread across the stack. Quantization alters the model's silicon contract (principle 4), shifting its operating point on the arithmetic intensity curve (principle 6) and transforming its energy profile (principle 7). In deployment, the latency budget (principle 12) dictates whether the resulting speedup meets the service-level objective (SLO), while the verification gap (principle 9) demands verifying that the lowered serving graph preserves the empirical accuracy measured during compression testing. The opening fleet failure escaped at exactly this step. Compression testing accepted the quantized model on images from the evaluation pipeline, while the firmware path presented a different population. The training-serving skew diagnostic (principle 11), comparing model outputs from the two preprocessing paths on the same captured inputs, is the measurement that would have exposed the mismatch before the affected cohort did. In that comparison the evaluation pipeline, which produced the calibration images, plays the role the diagnostic assigns to the training path, and the firmware path plays the serving path. The verification gap names why the failure escaped; the skew diagnostic names how to catch it.

The trace touched seven of the thirteen principles, and which seven depended on the decision. Had the same model moved off the device to a cloud endpoint, data placement (principle 2) would have entered, because every input image would then cross the network. Had preprocessing stayed on the host CPU while the NPU ran the faster INT8 kernels, Amdahl's Law (principle 8) would have bounded how much of that speedup the request path could see. The principles activate in turn as a decision moves from model representation to hardware execution to production validation, so a deployment proposal can be checked against the same principles before anyone writes code.

Checkpoint 16.2: Applying the principles

A colleague proposes quantizing your model from FP16 to INT8 to reduce serving costs.

Trace the principles

- ☐ **Pareto frontier** (principle 5): What accuracy are you trading for the bandwidth gain?
- ☐ **Silicon contract** (principle 4): Does the target hardware efficiently accelerate the required INT8 operators?
- ☐ **Verification gap** (principle 9): Does the quantized serving artifact preserve the behavior accepted during compression testing?
- ☐ **Training-serving skew** (principle 11): Do the calibration images pass through the same preprocessing path that every serving device uses?
- ☐ **Latency budget** (principle 12): Does the speedup bring you within SLO, or create headroom for batching?

Regrouped by the four parts in which they were introduced, the principles form a cycle rather than a pipeline (figure 16.1). The deployment principles (9–13) turn runtime telemetry into evidence that sends work back to the data foundations, so each production lesson becomes the starting condition of the next design pass. The conservation-of-complexity heuristic sits at the center of that cycle because every pass around it asks where the complexity displaced by the previous decision has landed. Along that return path, principles 9 through 13 expose the signals that trigger targeted remediations such as canary rollbacks, fallback heuristics, dataset augmentation, model retraining, or kernel retuning, so that failures are caught before they compound.

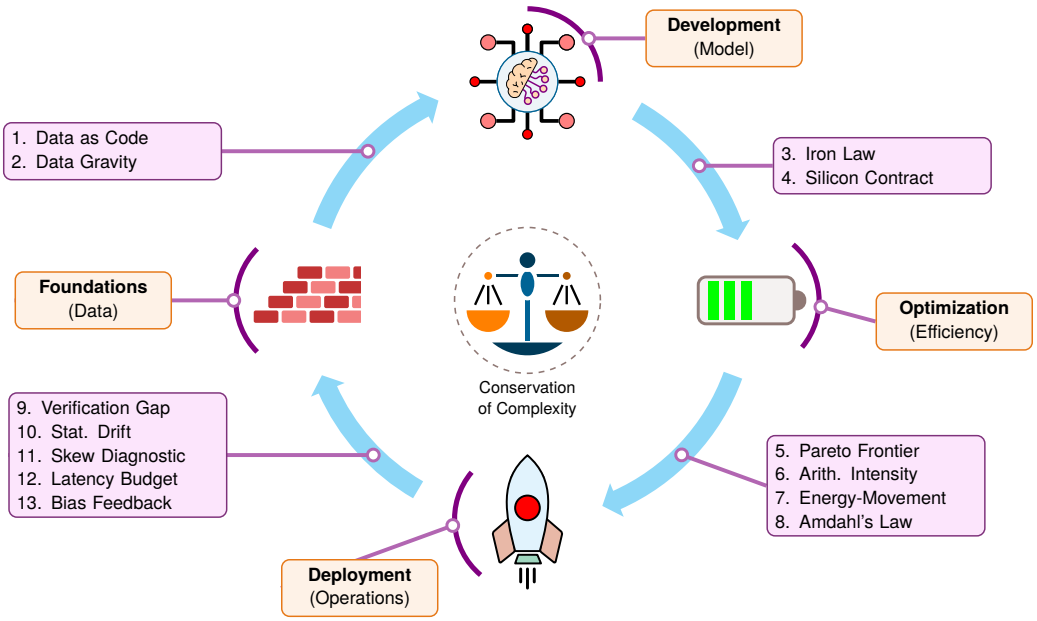


Figure 16.1: The Cycle of ML Systems (13 Principles): A four-phase systems engineering lifecycle organized around the conservation-of-complexity heuristic. The phases connect in a feedback cycle, and the thirteen principles sit along the transitions by the part that develops them, spanning bounds, decompositions, fitted models, policy requirements, and design heuristics whose assumptions must be checked. Icons mark the phases: bricks for the data foundations, a neural network for model development, a battery for the efficiency budget of optimization, and a rocket for launch into deployment.

The diagnostic framework is collected for reference in Appendix A, and the reference constants for back-of-the-envelope calculations appear in Appendix E. Tracing a quantization proposal through the principles it touches is one diagnostic pass, while the introduction posed a larger obligation that the same principles must serve, and two lighthouses can now measure it end to end.

16.4 The Dual Mandate Revisited

A recommendation model that ranks well on launch day can, six months later, be serving a population that its own rankings shaped, on hardware whose cost per query decides whether the service survives. The introduction named this pair of demands the dual mandate: learned behavior must remain trustworthy, and the machine must deliver that behavior within physical and economic limits. The thirteen principles are the book's tools for both halves, and each half can be traced end to end on a single lighthouse.

Trustworthy learned behavior

↩ The dual mandate, trustworthy learned behavior delivered within the available time, memory, energy, and cost, is established in section 1.1.

DLRM carries the first half, because several mechanisms can erode trust in its learned behavior. The degradation equation (equation 1.1) introduced a local fit relating distribution shift to observed accuracy change; the statistical drift diagnostic (principle 10) turns that idea into production telemetry, with divergence over user and item features as the early signal and delayed click and conversion outcomes as the confirmation. Training-serving skew (principle 11) attacks the same model from

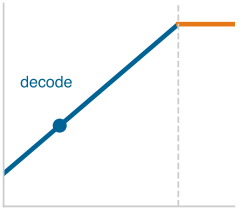
inside, when the online embedding lookup and the offline training pipeline compute a feature differently. The bias feedback model (principle 13) closes the loop, since items and creators that receive exposure generate the engagement data that earns them more exposure, and the resulting disparity can amplify or diminish across retraining cycles according to a fitted factor α_{fb}. The verification gap (principle 9) bounds what the launch evaluation actually established. Trust therefore has to be measured continuously, with alarms and a response plan, rather than certified once at launch.

Physical deliverability

GPT-2 carries the second half, and its family at scale shows the physics most sharply. Llama 2 70B is a decoder-only transformer of the same design with about 47× as many parameters as GPT-2's 1.5-billion-parameter release. The larger scale does not move batch-one decode off the bandwidth side of the ridge point, because every generated token still reads every weight once. Serving one Llama 2 70B token on two H100s makes that cost concrete.

↩ The intensity of batch-one decode, about 1 FLOP/byte in FP16 because each weight read feeds a single multiply-accumulate, is established in section 6.7.4.2.

Pricing that token repeats, at larger scale, the throughput ceiling computed for GPT-2 XL on one H100 in section 11.6.4 and the decode floor of equation 13.14. In FP16 the 70-billion-parameter model holds about 140 GB of weights, and each token costs about 140 GFLOP, two FLOPs per parameter. Sharded evenly across two NVIDIA H100s with an aggregate bandwidth of 6.70 TB/s and an aggregate FP16 peak of 1978 TFLOP/s (section E.2.2), the iron law (principle 3) gives a memory-transfer lower bound T_{memory} of about 20.9 ms against a peak-compute lower bound T_{compute} of about 0.07 ms, a ratio of 295.2×. The arithmetic intensity law (principle 6) accounts for that ratio. Intensity is still about 1 FLOP/byte, far left of the ridge in the margin roofline, so the time ratio is numerically the H100 ridge point itself. Sharding does not move the ridge, because it doubles peak rate and bandwidth together. The bound leaves out KV-cache traffic, activations, attention over the cache, interconnect communication, and dispatch overhead, so only profiling establishes the realized step time. Compute-only tuning can approach the 0.07 ms bound but cannot lower the 20.9 ms one; batching, which reuses each weight across requests, and quantization, which moves fewer bytes per weight, are the levers that can.

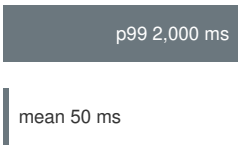


In this batch-one lower-bound model, decode sits left of the roofline ridge in the bandwidth-bound regime.

The estimate turns a physical bound into a diagnosis. The silicon contract (principle 4) exposes the architectural consequence. Accelerators are provisioned and purchased by peak arithmetic throughput, yet this workload saturates memory bandwidth first, so deliverable token generation speed is governed by bytes per second transferred from memory rather than by peak FLOP/s. The energy-movement principle (7) highlights the same weight traffic. A DRAM access costs far more energy than an arithmetic operation, by the ratio stated with that principle in section 16.3, and this workload performs only about one operation per byte fetched, so moving weights dominates the energy expenditure of each token. That ratio compares single access events with single operations, and because DRAM moves data in multi-byte bursts, an exact per-byte figure depends on transfer width and is settled by hardware telemetry.

↩ The energy ladder from registers to off-chip memory, which sets the per-access cost of moving a weight, is established in section 11.2.

Batching can amortize weight traffic across requests, and supported quantization can reduce bytes per weight. At the service level, the latency budget (principle 12) limits how far batching may go. The relevant delay now includes prompt processing and request waiting, not only token decoding. Consider an illustrative LLM chat endpoint whose mean time to first token is 50 ms but whose p99, driven by long prompts and by queueing behind long generations, reaches 2,000 ms, a tail 40× the mean (illustrated in the margin). Both values are scenario assumptions rather than outputs of a queueing model. The actual tail depends on the request mix and scheduling policy and must be measured; the general importance of tail latency is established in Dean and Barroso (2013). The product commits to the tail quantile, so the batching that improves throughput must stop where it pushes that quantile past its deadline.



Mean latency hides the tail; the selected tail quantile governs the SLO.

↩ The queueing model that relates utilization to tail latency, and the service-time assumptions under which its p99 estimate holds, is established in section 13.4.3.

The two traces use different principles, but they share one discipline: name the binding term, measure it, and budget for it. Scale keeps working only if the system can afford it and its results can still be trusted. The remaining question is where these constraints bind next, as ML systems spread across heterogeneous deployment targets, confront adversarial conditions, take on societal applications with public consequences, and grow into compositions of many components.

16.5 Future Directions

An autonomous vehicle navigating an unfamiliar intersection cannot wait for a cloud round-trip, and its sensor streams compete with the model for the same local bus; nor can a clinical assistant afford an uncalibrated diagnostic response under distribution shift. The framework is most useful when it forecasts where such physical and statistical constraints bind next. Three areas put the same physics under increasing pressure: deployment across diverse contexts, robustness under adversarial conditions (including deliberately crafted inputs (Goodfellow et al. 2015)), and societal applications whose failures carry public consequences. A fourth horizon, systems that compose multiple models, tools, and verifiers or grow beyond one machine, extends the same reasoning rather than replacing it.

Applying principles to emerging deployment contexts

Moving a workload from a data-center rack to a mobile phone or a battery-powered sensor strips away orders of magnitude in memory capacity and power budget while tightening latency deadlines. That span tests whether one quantitative framework can explain systems across contrasting resource regimes: the cloud offers abundant power and centralized hardware, edge and mobile devices operate under strict latency and battery budgets, and TinyML and embedded systems compress the same design problem into kilobytes and milliwatts.

In the cloud regime, the binding decision is how to turn abundant hardware into useful throughput, which for ResNet-50 means keeping batched kernels near the compute ceiling and for DLRM means placing embedding tables that exceed one accelerator's memory. Toward the small end of the spectrum, the MobileNetV2 and keyword-spotting lighthouses carry the forecast.

↩ The four deployment paradigms, and the physical constraints of power, latency, and memory that separate them, are established in section 2.1.

In contrast to the cloud, mobile and edge devices, the regime of the MobileNetV2 lighthouse, operate under strict power, thermal, and memory bounds that demand hardware-software co-design. Efficient network architectures combined with model compression enable deployment on platforms where a reference mobile device has about 10× smaller memory capacity and about 140× smaller power envelope than an H100-class accelerator. Edge deployment is mandatory when latency, privacy, intermittent connectivity, or egress bandwidth make centralized cloud inference infeasible. On these devices, efficiency decides whether a model runs at all.

↩ The per-device memory and power envelopes that separate the mobile tier from a data-center accelerator are established in section 2.4.2.

At the opposite extreme, TinyML and embedded systems, the domain of the keyword spotting lighthouse, operate within kilobyte SRAM budgets, milliwatt power envelopes, and multi-year operational lifecycles. At these extremes, profiling isolates memory traffic bottlenecks, custom operator lowering reduces particular dispatch and interpretation costs, and a fixed tensor arena, planned at build time from tensor lifetimes, replaces dynamic allocation so that peak memory is known before the device ships. Severe resource limitations historically catalyzed efficient architecture families such as MobileNets (Howard et al. 2017; Sandler et al. 2018) and EfficientNets (Tan and Le 2019), a direct case of machine constraints driving algorithmic innovation.

A related frontier is embodied AI and robotics, where models execute inside closed-loop sensorimotor control cycles. In this regime, the deadline T_{deadline} of the latency budget (principle 12) becomes a hard real-time safety constraint, because a tail-latency spike can cause mechanical instability or collision, and continuous high-bandwidth sensor streaming competes directly with model weight traffic across constrained on-device memory buses.

↩ The physical memory and latency envelopes that mandate compression for microcontroller and edge deployment are established in section 10.1.1.

Generative AI stresses all of these regimes at once. Autoregressive decode of the GPT-2/Llama family is bandwidth-bound at low batch, as the token estimate showed, and the serving responses to that bound each move the binding term in a different way. Batching raises weight reuse, and speculative decoding spends surplus compute on a small draft model to shorten the sequential path of the target model. Sufficiently long prompts shift the prefill phase toward the compute ceiling instead, so the same model crosses regimes within a single request.

The same physical bounds apply across these paradigms, while fitted statistical models and SLO policies remain deployment-specific. Success depends on checking those distinctions and applying the principles together rather than pursuing isolated optimizations. The more deployment contexts a system spans, the more failure surfaces it creates. Robustness, not coverage alone, therefore becomes a binding constraint at the next frontier.

↩ Continuous batching, key-value cache management, and speculative decoding for language model serving are established in section 13.9.

Building robust AI systems

Across those failure surfaces, a machine learning system can respond confidently and incorrectly while conventional infrastructure health checks remain green, allowing silent failures to persist undetected for weeks. Distribution shifts alter accuracy without code modifications, adversarial inputs (small, deliberately crafted perturbations that flip a prediction while leaving the input looking unchanged to a person) exploit vulnerabilities invisible to standard testing, and edge cases expose training-data boundaries that debugging alone cannot fix.

Finite testing estimates behavior on a defined population with uncertainty; it cannot prove correctness for every future input. Drift signals show when that population may have changed, while labeled outcomes determine whether quality changed. Together they motivate continuous monitoring as a design requirement, along with fallback policies and periodic revalidation, without claiming that every distribution shift causes degradation. The operational question is whether a failure will be detected and diagnosed before its impact spreads.

Robustness therefore demands designing for graceful degradation, not only prevention. At the single-system scale, that discipline appears as fallback paths, uncertainty thresholds, version-specific rollback policies, and monitoring hooks, with rollback reserved for a bad release rather than for external drift. At larger scale, the same logic extends to hardware redundancy and ensemble-style diversity. As AI systems assume increasingly autonomous roles in healthcare, transportation, and finance, the cost of an undetected failure grows.

↩ Incident response procedures, including the choice among rollback, fallback, and traffic reduction, are established in section 14.7.1.1.

AI for societal benefit

Robust systems are the prerequisite for deploying AI in domains where technical failures carry public consequences. The diabetic retinopathy screening case from chapter 3 shows why. A model that reaches expert-level performance on high-quality images from standardized research cameras can fail on blurry, poorly lit images from older clinic cameras, not because the algorithm is wrong but because the deployment population has moved outside the training envelope. In verification-gap terms (principle 9), the launch evaluation certified the model on the wrong population. Each societal domain stresses a different principle.

↩ The lab-to-field gap in the diabetic retinopathy screening case, where field images fall outside the training distribution, is established in section 3.6.1.

Scientific discovery workloads such as protein structure prediction, drug-interaction modeling, and materials screening can demand sustained training or inference throughput, so the iron law (principle 3) and the silicon contract (principle 4) decide how much of a search space a fixed compute budget can explore. Healthcare stresses the deployment principles instead. Calibrated uncertainty, external validation at each new site, clinical review, and continuous monitoring become safety requirements, because a diagnostic model trained on one hospital's population may degrade at another with different demographics, disease prevalence, or imaging equipment. Personalized education stresses data governance and the data-as-code principle (1), since the student-interaction records that drive personalization are both sensitive and, in effect, the program the model learns from; an interactive tutor must also meet its latency budget. A climate model that produces confident but uncalibrated projections carries the same verification problem into policy, where an overconfident forecast can steer decisions that are slow and costly to reverse.

In each of these domains the D·A·M taxonomy, the thirteen principles, and roofline analysis supply the systems questions, but the acceptable answers come from domain evidence such as clinical studies, educational outcome measurements, and physical validation, which no generic systems analysis provides. These applications also share a property that keeps their constraints tractable, because their task boundaries are fixed in advance. A diagnostic model classifies within a defined label set, and a structure predictor answers a defined kind of query. The next stress test removes that boundary.

System composition as a stress test

The most ambitious test of these principles is a system whose task boundaries are not fixed in advance. A task-general assistant or multi-component ML service may route one request through several components: retrieval, which fetches relevant documents from an index; a planner, a model that decides which steps to take; tool execution, which calls external programs such as a search engine or a calculator; generation of the answer; and a verifier, which checks the output against rules or with a second model. The governing challenge is systems engineering as much as algorithm design,

because the surrounding system must bound latency, reliability, cost, safety, and observability as work fans out across components.

System composition puts four of the principles under new load at once. As work fans out across retrieval, planning, tool execution, generation, and verification, the iron law must budget the computation and data movement of every stage, while the silicon contract fragments because each stage saturates a different resource: an index lookup leans on memory capacity and bandwidth, generation on the weight traffic of decode, and a tool call on network round trips, so no single accelerator suits the whole request. Balancing those heterogeneous stages expands the Pareto frontier from two or three metrics, such as accuracy, latency, and memory, to a larger surface that also includes safety, fairness, factuality, privacy, and cost, while statistical drift now threatens not only the final output but also retrieved documents, tool responses, and intermediate decisions.

The systems cost is visible in a single request. If an assistant fans out to retrieval, a planner, two tools, a generator, and a verifier, its realized latency follows the request's critical path. That path is the retrieval time, plus the planner's time, plus the slower of the two tool calls if they run in parallel, plus generation, verification, and the orchestration overhead between steps, and the latency budget (principle 12) must hold that sum at the selected quantile. Its reliability analysis must also account for each component, because every additional component creates another timeout, schema mismatch, stale index, or verifier false negative to monitor. Conventional microservices already face interface failures across process boundaries. Composed ML systems add probabilistic interfaces in which a large language model planner may invent a tool name that does not exist or produce JSON that deviates from the declared schema (the agreed format of each call), requiring defensive parsing, retry logic, and output validation at each interface boundary.

A composed system therefore cannot rely on a single model-quality claim; it needs interfaces whose behavior can be measured, because each interface is where one component's assumptions about another become testable (Lampson 1983). Composed systems trade monolithic simplicity for explicit coordination. Each step can be independently updated, monitored, and debugged, but each also adds an interface whose behavior must be specified and tested. The decomposition gives up latency and architectural simplicity in exchange for control and observability, a move along a Pareto frontier rather than a free gain. It also opens opportunities. Modular components can be compressed and accelerated on targeted back ends, each with its own silicon contract (principle 4) and arithmetic intensity profile, and the interfaces between them are natural observation points for drift, skew, and numerical degradation.

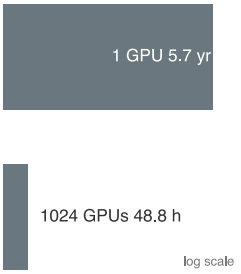


Systems Perspective 16.1: A new golden age

Hennessy and Patterson (2019) declared a "New Golden Age for Computer Architecture," driven by the end of Dennard scaling, the slowdown of Moore's Law, and new opportunities from domain-specific architectures, open instruction sets that any designer may implement without a license, and agile chip development with short, iterative hardware design cycles. Large-scale AI workloads are one domain where those pressures are visible. Building more capable AI services will not be a matter of writing a better loss function alone; it will be a systems engineering challenge involving energy efficiency, high-bandwidth interconnects, memory hierarchy design, and software stacks that keep heterogeneous hardware useful rather than idle. The thirteen principles provide a quantitative vocabulary for navigating that regime while preserving the distinction between physical bounds and deployment-specific assumptions.

A horizon note: From node to fleet

When workloads exceed a single node, the resource boundary moves outward, but bottleneck reasoning remains invariant. Local memory capacity constraints remain, while bisection bandwidth adds another limit; local exception handling expands into fleet-level fault tolerance, and training throughput becomes a distributed coordination problem. As the margin ladder illustrates, under a model in which every GPU fails independently at the same constant rate, regardless of its age, and any GPU failure halts the pool, a reference component mean time to failure (MTTF) of 5.7 years collapses into a cluster mean time between failures (MTBF) of about 48.8 hours across a 1,024-GPU pool, before accounting for correlated network or power outages. Under this model, a multi-week



Fleet scale turns rare component failures into routine system events.

training run is highly likely to encounter component failures. Such a run needs a fault-tolerance strategy, and checkpoint scheduling and recovery scope should follow the measured failure and recovery costs. At fleet scale, the analysis does not abandon single-node principles; it widens their boundary. The engineer still identifies the binding constraint, quantifies the governing cost terms in the iron law, and traces where displaced latency propagates.

This discipline rewards engineers who use bounds to guide benchmarks and measurements to test the assumptions behind those bounds. The same habit governs the last question this book raises, which is what the engineer owes the people a system affects.

16.6 The Engineering Responsibility

A model that requires an array of high-end data-center accelerators for inference excludes organizations unable to procure specialized capital;[2] upstream training data encodes historical sampling skews that distort downstream predictions; and sustained accelerator power draw directly drives facility thermal dissipation and regional grid carbon emissions. Ethical considerations cannot be separated from physical and algorithmic design, because the same decisions set both: choices from arithmetic precision to data curation distribute operational costs and societal consequences far beyond the engineering team.

Systems engineering extends beyond maximizing raw capability to operating deployments responsibly within societal and physical bounds. Production systems must achieve the arithmetic efficiency needed to run on accessible hardware, maintain fault tolerance against hardware anomalies, operate within strict thermal and power envelopes, and meet justified, application-specific fairness requirements across relevant demographic groups. Applications such as planetary climate monitoring and clinical diagnostic assistants make governance criteria first-class physical and operational constraints. Meeting them rests on three habits that outlast any model family or hardware generation: measuring hidden dependencies, bounding trade-offs against physical hardware limits, and holding production deployments to real operational constraints.

2 | **AI democratization**: Making AI accessible beyond a small number of well-resourced organizations through efficient systems engineering. Mobile-optimized models and cloud APIs can widen access, but doing so sustainably requires systematic optimization across hardware, algorithms, and infrastructure to maintain quality at scale.

The same responsibility demands continuous skepticism toward local wins. Optimizing a component in isolation is the most common way a system acquires failure modes that no team owns, and the benchmark win that does not survive production is its most familiar form. The recurring versions of that mistake are worth naming explicitly, because recognizing them early is far cheaper than a redeployment.

↩ The lifecycle that turns responsibility into written requirements, release gates, documentation, and operational monitoring is established in section 15.3.

16.7 Fallacies and Pitfalls

Fallacies and pitfalls in ML systems arise from one common source, which is treating the system as decomposable into independent parts. Each item below is the cross-boundary form of a mistake an earlier chapter met inside one layer, where optimizing one dimension, one metric, or one stage looks sufficient until that assumption meets production reality.

Fallacy: *Systems engineering complexity disappears with better tools and abstractions.*

Tools abstract complexity; they do not eliminate physical constraints. A high-level framework that hides memory management still consumes memory. An automated machine learning (AutoML) system that searches over hyperparameters still faces the Pareto frontier. Simplifying one interface may shift burden to another.

Pitfall: *Optimizing one metric without tracing displaced costs.*

When an optimization reduces latency by 50 percent, systems analysis demands asking what changed elsewhere. Quantization may add validation burden. Caching may trade memory capacity for serving speed. Some simplifications remove accidental complexity; others displace cost, and only profiling across the stack decides which occurred. Engineers who celebrate gains in one metric without tracing cross-stack effects build systems that fail in unexpected ways.

Fallacy: *Mastering individual components equals mastering the system.*

Component expertise is necessary but insufficient. An engineer who understands data pipelines, training, serving, and operations as isolated domains will still struggle with systems where a data schema change cascades through training, breaks quantization assumptions, and triggers silent

accuracy degradation in production. Integration can create more complexity than the components reveal in isolation because interfaces multiply failure modes.

Pitfall: *Scaling data collection without tracing its costs into the layers that absorb them.*

A data team that grows its collection judges the result by model quality, yet the costs land in other layers. Storage and preprocessing grow with every sample, each training epoch reads more bytes through the input pipeline, and repeatedly moving or scanning a larger dataset can exceed the cost of moving compute toward it, as the data-gravity principle (2) predicts. Within a fitted scaling regime, additional data can yield diminishing improvements (section 9.2.2) while those costs continue to grow, and coverage, data quality, model capacity, and distribution determine whether that fit remains useful for the next collection increment. If a team judges collection only by its own metric, it gathers data whose costs other teams absorb, so the marginal information value per sample has to be weighed against those downstream costs.

Fallacy: *A single accuracy metric captures model quality.*

A model evaluated solely on top-line accuracy ignores the boundaries it must cross. The composite fleet scenario that opened this chapter passed its aggregate offline evaluation while accuracy on specific firmware and device cohorts fell by four percentage points. The serving boundary imposes its own test. Under a 100 ms tail-latency SLO, a 95 percent-accurate model executing in 500 ms is infeasible, whereas a 93 percent-accurate model with a measured 50 ms tail meets that latency requirement. It is a feasible deployment only if its accuracy, subgroup behavior, resource costs, and other acceptance criteria also pass. Pareto analysis (principle 5) evaluates latency, throughput, memory footprint, energy, fairness, and serving cost together, and disaggregated evaluation exposes the subgroup disparities that an aggregate conceals.

↩ Disaggregated evaluation, which tests each relevant subgroup separately as a release gate, is established in section 15.3.2.

Pitfall: *Treating every drift alarm as an automated rollback trigger.*

The alarm fires in the monitoring layer, but its cause may sit in another layer entirely, and the response must follow the cause. Rollback is appropriate when a regression is traced to a flawed binary or pipeline release. External covariate drift, however, is not repaired by redeploying an older, equally stale checkpoint, and a shift introduced upstream, such as the firmware preprocessing path in the opening scenario, is repaired at the boundary where it entered. Safe operational responses include canary traffic shedding, fallback heuristics, prediction quarantine, or scheduled retraining once ground-truth labels confirm degradation.

Fallacy: *Optimizing the stage one team owns makes the system fast.*

Teams tend to profile the stage they own, and the model kernel is the one most visible, instrumented, and owned by the ML team, while the surrounding data path is split across storage, preprocessing, networking, and application code. Amdahl's Law (principle 8) shows what that division costs. Optimizing accelerator inference latency by 10× yields only 1.1× system speedup if CPU-bound preprocessing accounts for 90 percent of end-to-end latency. Serial fractions of that size can arise from host-side image augmentation, synchronous feature-store lookups, or tokenization that remains host bound in some implementations. End-to-end profiling across team boundaries keeps the target honest, because the stage to improve is the one that limits the system, not the one with the cleanest benchmark harness.

↩ Amdahl's law, which caps the gain from speeding up one stage by the share of request time that the other stages take, is established in section 11.1.

These fallacies and pitfalls share one root, local optimization that ignores the coupling of the system that executes it. Closing the chapter requires returning to the whole request path and measuring how those coupled stages behave together.

16.8 Summary

ML systems engineering differs from isolated component optimization by reasoning across boundaries. Because decisions couple across data, algorithm, and machine, the deployed system rather than the weights checkpoint is the model, and its costliest failures live at interfaces that no component test exercises. The thirteen principles, the conservation-of-complexity heuristic, and the lighthouse traces that carried them through the stack are the tools for reasoning about that system as a whole. Their stated assumptions distinguish exact bounds from fitted models, SLO policies, and design

↪ A diagnostic matrix that maps common production symptoms to candidate D·A·M axes and the measurements that confirm them is collected for reference in section A.5.2.

heuristics, allowing the tools to remain useful as frameworks, hardware generations, and model families change.

Key Takeaways: Reasoning across boundaries

- **Decisions couple across layers**: In the composite fleet scenario every team met its local metric while accuracy on specific device and firmware cohorts fell by four percentage points, because quantization calibration and a firmware preprocessing path met only in production. Isolated component tests miss such cross-layer interactions; cross-path parity checks, integration tests, and cohort-level outcome tracking test the governing assumptions at the interfaces.
- **Each lighthouse rewards the lever aimed at its binding term**: ResNet-50 is compute-bound, GPT-2 and Llama decode bandwidth-bound, DLRM capacity-bound, MobileNetV2 compute-bound in its efficient regime, and keyword spotting power-bound. The regime belongs to the workload on a machine, so batch size, phase, and precision can move a lighthouse across regimes, and the diagnosis carries forward to larger models of the same kind.
- **Assumptions matter across implementations**: The thirteen principles turn framework-specific craft into measurable reasoning by combining physical bounds, decompositions, fitted models, requirements, and heuristics. Apply each only within its stated scope.
- **Trace displaced costs without assuming conservation**: Compression, batching, monitoring, and governance can relocate burdens across data, algorithm, and machine. Measurement shows where each burden went.
- **Boundaries reveal the bottleneck**: In the idealized batch-one, two-H100 model, the memory-transfer lower bound for a Llama 2 70B token is about 295.2× the peak-compute lower bound, and in the illustrative LLM chat endpoint the p99 time to first token is 40× the mean. Systems thinking means measuring where physics, traffic, and users bind.
- **Both halves of the dual mandate are measurable**: Trustworthy behavior is tracked through drift, skew, and fitted feedback factors; physical deliverability through the iron law, the roofline, and energy per event. Neither is certified once at launch; both are budgets the system must keep meeting.

Hennessy and Patterson's *Computer Architecture: A Quantitative Approach* gave processor designers a shared analytical discipline by decomposing CPU time into instruction count, cycles per instruction (CPI), and clock period, so that any proposed improvement could be judged by which factor it changed (Hennessy and Patterson 2011; Hennessy and Patterson 2017). The iron law plays the same role for machine learning systems, and the thirteen principles around it extend that discipline from execution time to energy, trade-offs, and deployed behavior. Together they give a shared analytical baseline for evaluating trade-offs across data, algorithms, and machines.

The voice-assistant request that opened the introduction can now be traced through every layer, as a composite assembled from the book's lighthouses rather than from any one product's design. First, a keyword-spotting model of about 200 thousand parameters listens for the wake word on a microcontroller, its 200 KB of INT8 weights held in 1 MB of flash and its 20 MFLOP per inference repeated around the clock under a milliwatt power budget. Once the wake word triggers, an on-device speech front end, an efficient convolutional network sized like MobileNetV2, runs on the phone's neural processing unit, performing 600 MFLOP of work on 3.5 MB of INT8 weights within a 5 W envelope. When the transcribed query reaches the cloud, ranking the retrieved results falls to DLRM, whose 100 GB of embedding tables exceed one H100's 80 GB before any arithmetic begins. Finally, a GPT-2 decoder generates the response on an H100, where every token reads 3 GB of FP16 weights, a memory floor of about 0.9 ms per token that exceeds the compute floor by 295×, assuming graph capture hides dispatch overhead (section 7.3.6). Power, compute, capacity, and bandwidth bind in turn along one request, so an engineer who knows only one layer sees one of those four limits and misses the other three. The introduction's three questions take a different answer at each stage. Wake-word recordings and ranking logs shape the predictions, a different iron-law term

limits each stage's execution, and cohort-level monitoring across devices and users supplies the evidence that the whole request still deserves deployment.

What endures is the method: start from empirical evidence and physical bounds, quantify trade-offs, and treat system design as constrained optimization. This is the engineering corollary of the bitter lesson drawn from seven decades of AI research. Because general methods that scale with computation repeatedly outrun hand-crafted expertise, the durable advantage belongs to systems engineering that can absorb that computation, not to any single clever architecture (Sutton 2019). Frameworks, chips, and architectures will change; measuring against physical limits will not.

The world is rushing to build AI systems. Our task is to engineer them.

Prof. Vijay Janapa Reddi, Harvard University

BACK MATTER

A

The D·A·M Taxonomy

A system that runs slowly rarely announces why. An idle accelerator can stem from a starved input pipeline (Data), an algorithm with too little arithmetic intensity (Algorithm), or a saturated memory bus (Machine). This appendix maps profiler symptoms and measurements to the axis that binds execution, before optimization begins. Readers should arrive with the three axes from the opening chapters and the iron law that quantifies them.

How to Use This Appendix

Use the D·A·M taxonomy when a system misses a performance, cost, or quality target and the cause is not yet clear. Work through the framework in four passes:

1. **State the symptom and operating budget**: Record the missed end-to-end target, workload, and deployment context.
2. **Localize the candidate axis or boundary**: Use table A.2 for a first pass and table A.3 when the evidence crosses axes.
3. **Test the hypothesis**: Map the evidence to the iron law and arithmetic intensity, then gather confirming measurements with table A.5 and table A.6.
4. **Intervene and remeasure**: Choose a lever from table A.7, repeat the end-to-end measurement, and score the result with table A.8. If the target does not improve, revise the hypothesis.

Table A.1 maps each common need to the section that answers it. In every case the axis label is a hypothesis, not a verdict. The axes overlap, and a useful diagnosis may name both a dominant axis and a boundary effect.

Table A.1: Need-to-Section Lookup: Each row names a diagnostic need and the section of this appendix that answers it. Start from the need that matches the observed symptom.

Need	Section
Classify an optimization by the axis it acts on	section A.1
Reason about a technique that spans two or three axes	section A.2
Predict which iron-law term a change shortens	section A.3
Explain why overlapping two stages saved little time	section A.3.2
Decide whether a kernel is compute-bound or bandwidth-bound	section A.4
Narrow a symptom such as low accelerator utilization quickly	section A.5.1
Find the measurement and tool that confirm a hypothesis	section A.5.2
Choose the lever that moves the binding term	section A.5.3
Judge whether a measured system is efficient enough	section A.5.4
Avoid optimizing the wrong axis	section A.6
Follow a complete diagnosis on the reference accelerator	section A.7
Interpret a loss curve that stops improving as data grows	section A.8

A.1 The Three Axes

When a training step stalls or a serving call misses its latency budget, the cause can sit in JPEG decoding on the host, quadratic attention work in the model, or a saturated memory bus on the

accelerator. The taxonomy separates those causes along the three axes of the iron law of ML systems (section 1.7): Data is what the model learns from and what the pipeline must deliver, Algorithm is the computation the model performs, and Machine is the hardware that executes that computation. Table A.2 summarizes the role, primary physical constraint, and core optimization pathway for each axis.

Table A.2: D·A·M Axis Reference: Each axis maps to a constraint class and optimization strategy. Form a hypothesis, confirm it with measurements, then follow the chapter pointer.

Axis	Role	Physical Constraint	High-Leverage Optimization
Data (D)	What the model learns from	Volume, quality, arrival rate	Data Selection (chapter 9)
Algorithm (A)	The computation the model performs	Operations and dependencies	Model Compression (chapter 10)
Machine (M)	The hardware that executes it	Compute, memory, I/O	Hardware Acceleration (chapter 11)

This first-pass separation is useful, but production systems rarely suffer from a single axis-centered bottleneck. More often, the problem sits at the boundary between two axes, as when a data format choice determines whether the accelerator can be saturated or a pruning strategy changes the memory access pattern. Handling these cases requires mapping the intersections.

A.2 Axis Intersections

The conceptual Venn overlaps in figure 1.4 introduce the intersections of machine learning systems; the operational version is a diagnostic triad of Data, Algorithm, and Machine joined by bidirectional coupling (figure A.1). To diagnose a production bottleneck, an engineer first identifies which axis currently binds the workload, then uses the two incident edges to predict how an intervention will change memory, compute, or I/O pressure on the other two axes.

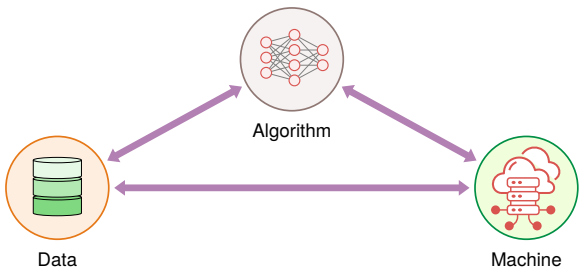


Figure A.1: The D·A·M Axis Triad: A compact diagnostic view of Data, Algorithm, and Machine as three interdependent axes. The bidirectional links emphasize that an intervention on one axis changes the constraints on the other two.

With the first-pass axis identified, the pairwise and three-way intersections expand into concrete engineering techniques and the chapters that teach them (table A.3). In the D·A·M acronym, Data, Algorithm, and Machine name the three diagnostic axes rather than single-letter algebraic variables; formal quantities follow the notation chapter, where D denotes logical dataset size (samples or tokens) and D_{vol} denotes physical data volume moved (bytes).

Table A.3: D·A·M Intersection Reference: Each zone maps specific techniques to the axes they span and the chapters that cover them. The pairwise intersections require reasoning about two domains simultaneously; the center requires all three.

Zone	Name	Key Techniques	Book Coverage
Data	Information	Storage formats, data quality, distributions	chapter 4
Algorithm	Logic	Loss functions, architectures, gradients	chapter 5, chapter 6
Data ∩ Algorithm	What to Learn From	Data selection, curriculum learning, compute-optimal scaling	chapter 9, chapter 8

Zone	Name	Key Techniques	Book Coverage
Data ∩ Machine	How to Move Information	I/O bandwidth, prefetching, data formats	chapter 4, chapter 11
Algorithm ∩ Machine	How to Execute Efficiently	Quantization, pruning, kernel fusion, mixed precision	chapter 7, chapter 10
Machine	Physics	Silicon, memory hierarchy, peak FLOP/s	chapter 11
Data ∩ Algorithm ∩ Machine	ML Systems Engineering	Iron law, Roofline, training loops, serving	chapter 8, chapter 13, chapter 12

In the D·A·M acronym, Data, Algorithm, and Machine name the three diagnostic axes rather than single-letter algebraic variables; formal quantities follow the notation chapter, where D denotes logical dataset size (samples or tokens) and D_{vol} denotes physical data volume moved (bytes). The axis-centered zones contain concepts that belong primarily to one axis: storage formats and distributions emphasize Data, loss functions and gradients emphasize Algorithm, and silicon physics and peak FLOP/s emphasize Machine. Single-domain expertise can introduce them, but deployed behavior may still depend on the other axes.

The pairwise intersections are where systems thinking begins. At Data ∩ Algorithm (*What to Learn From*), data selection, curriculum learning, active learning, and scaling laws (section A.8.1) couple information content to algorithmic capacity, because adding data that the model cannot absorb wastes compute while choosing architectures that outstrip available data wastes engineering time. Whereas that boundary governs statistical efficiency, Data ∩ Machine (*How to Move Information*) governs physical delivery through I/O bandwidth, prefetching strategies, data formats, and the energy-movement invariant, where data gravity manifests because the cost of moving bytes through the memory hierarchy decides whether the machine stays fed. Once data reaches the device, Algorithm ∩ Machine (*How to Execute Efficiently*) links mathematical structure to hardware execution across quantization, pruning, kernel fusion, mixed precision, and computational graph optimization, explaining why a pruning scheme that cuts FLOPs while fragmenting memory access can still run slower on silicon.

The center, Data ∩ Algorithm ∩ Machine, is where all three axes converge. The iron law, the roofline model, end-to-end training loops, serving pipelines, and holistic benchmarking all require simultaneous reasoning about data flow, algorithmic complexity, and hardware utilization. Whereas the intersections classify where a technique operates, isolating which axis actively throttles a workload requires translating these relationships into wall-clock execution time, which is the role of the iron law.

A.3 Iron Law Mapping

For any workload, the iron law connects the D·A·M axes to execution time T (seconds) in two steps. First, at the workload level (equation 1.2), time factors multiplicatively across the three axes:

$$T = \underbrace{\left(\frac{\text{Samples}}{\text{Task}}\right)}_{\text{Data}} \times \underbrace{\left(\frac{\text{FLOPs}}{\text{Sample}}\right)}_{\text{Algorithm}} \times \underbrace{\left(\frac{\text{Seconds}}{\text{FLOP}}\right)}_{\text{Machine}}$$

where the product of the Data and Algorithm factors sets the total arithmetic work $O = (\text{Samples/Task}) \times (\text{FLOPs/Sample})$ in FLOPs. Second, for serialized execution phases (equation 1.3), unpacking how the machine executes those operations maps work across the D·A·M interfaces into three physical time terms, where D_{vol} is bytes moved, BW is bytes per second, R_{peak} is peak throughput in FLOP/s, η_{hw} is dimensionless hardware efficiency, and L_{lat} is fixed overhead in seconds:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Data/Machine}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Algorithm/Machine}} + \underbrace{L_{\text{lat}}}_{\text{Cross-axis overhead}}$$

Dividing this serialized decomposition by O (and substituting arithmetic intensity $I = O/D_{\text{vol}}$) expresses the realized Machine factor (Seconds/FLOP) directly in terms of the three physical bottlenecks:

$$\frac{\text{Seconds}}{\text{FLOP}} = \frac{T}{O} = \underbrace{\frac{1}{I \cdot \text{BW}}}_{\text{Memory time per FLOP}} + \underbrace{\frac{1}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute time per FLOP}} + \underbrace{\frac{L_{\text{lat}}}{O}}_{\text{Overhead per FLOP}}$$

Algorithm and Machine share the compute term, separated by which variable the engineer controls. Reducing the total operations (O) is a Data or Algorithm lever, while reducing the realized time per operation (Seconds/FLOP) by raising peak throughput (R_{peak}), utilization (η_{hw}), or bandwidth (BW) is a Machine lever. This per-FLOP expression also shows why training and inference occupy different regimes under the same law. In large-batch training, high arithmetic intensity (I) and large operation counts (O) shrink the first and third terms so Seconds/FLOP approaches the compute term (section 8.2); in small-batch or single-token inference, low I and small O per request inflate the memory and overhead terms (chapter 13).

The dominant measured cost appears in one of these terms. A slow system may move too much data (D_{vol}), lack bandwidth (BW), execute too many operations (O), fail to use the hardware's peak capability (η_{hw}), or pay cross-axis overhead (L_{lat}). A workload whose time is dominated by L_{lat} is overhead-bound: kernel launches, dispatch, and synchronization, not bytes or arithmetic, set its pace. The levers below map specific optimizations to the variables they improve.

A.3.1 Component levers

- **Data lever**: Reducing the samples processed per task (Samples/Task, which shrinks O) or the moved bytes (D_{vol}) through deduplication and selection, or increasing I/O bandwidth (BW).
- **Algorithm lever**: Reducing operations per sample (FLOPs/Sample, and thus O) through pruning, distillation, or architectural refinement, or reducing the bytes each weight and activation occupies through lower-precision representation. Quantization is an Algorithm choice that shrinks the Data/Machine term D_{vol} and may raise hardware throughput R_{peak}, lowering Seconds/FLOP without reducing operation count.
- **Machine lever**: Lowering Seconds/FLOP by increasing peak throughput (R_{peak}), memory bandwidth (BW), or the utilization factor (η_{hw}) via kernel fusion and scheduling.

A.3.2 From sum to max

The additive iron law represents sequential execution. Let $T_{\text{sequential}}$ denote time without overlap and $T_{\text{pipelined}}$ denote measured time with overlap. Overlap can move the sum toward a max, but dependencies, contention, and incomplete overlap keep the measured time above this ideal lower bound:

$$T_{\text{sequential}} = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \quad \xrightarrow{\text{overlap}} \quad T_{\text{pipelined}} \geq \max\left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}\right) + L_{\text{lat}}$$

Compute should overlap with data movement wherever dependencies allow. Table A.4 summarizes key D·A·M coordination techniques that overlap work across axes.

Table A.4: D·A·M Overlap Techniques: Each technique overlaps work across axes, moving sequentially added Data/Machine and Algorithm/Machine time toward their ideal lower bound: the larger of the two terms. Dependencies and contention limit realized overlap.

Technique	D·A·M Axes Overlapped	Implementation
Prefetching	D overlaps M	DataLoader with `prefetch_factor`, `pin_memory=True`
CUDA Streams	D overlaps M	Separate streams for H2D transfer and compute
Async Gradient Sync	M (communication) overlaps A	Overlap bucketed AllReduce with remaining backward computation
Double Buffering	D overlaps M	Fill buffer N+1 while computing on buffer N

Overlaps provide the greatest relative speedup when the D·A·M time terms are reasonably balanced. If one term dominates (for example, a severely bandwidth-bound kernel), overlapping the smaller term with the larger yields negligible gain, because the max is still set by the same bottleneck. This balance occurs near $D_{\text{vol}}/\text{BW} \approx O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$. The overhead term is treated separately below.

> **Systems Perspective A.1: The overhead that cannot hide**
>
> The unhidden latency term L_{lat} (kernel launch, synchronization barriers, Python dispatch) contains serialization points that cannot be fully overlapped. Kernel fusion can reduce this term by combining operations, but it does not eliminate all launch, scheduling, or synchronization overhead.

The iron law quantifies the time each term consumes. Whether data movement or arithmetic sets the ceiling for a given kernel depends on the ratio of the two, which is arithmetic intensity.

A.4 Arithmetic Intensity Boundary

Doubling an accelerator's peak FLOP/s leaves a kernel no faster if the arithmetic units spend most of each cycle waiting for bytes from memory. The boundary between the Data/Machine memory term and the Algorithm/Machine compute term is set by the arithmetic intensity[1] (I) of the workload. A kernel whose intensity falls below the hardware's ridge point $R_{\text{peak}}/\text{BW}$ is **bandwidth-bound**, the regime the roofline literature calls *memory-bound*; this book keeps the bandwidth name so the regime is not confused with running out of memory capacity. A kernel above the ridge point is compute-bound.

The reference accelerator places the boundary concretely. An H100's dense FP16 ridge point is about 295 FLOP/byte. GPT-2 XL decoding one token at batch one reads all 3 GB of its FP16 weights and reaches an arithmetic intensity of only about 1 FLOP/byte, more than two orders of magnitude below the ridge. With kernel launches captured into a replayable graph (section 7.3.6), the weight read takes about 0.9 ms while the arithmetic needs about 3 µs, so a faster arithmetic unit would leave the per-token time unchanged. Section 11.6 develops the ridge point, hardware balance, and the regimes on either side of it, section D.2.1 carries the worked roofline template, and section 13.9.2 turns the weight read into the decode floor a serving system plans around.

The roofline gives an upper performance bound when traffic and achieved rates are measured consistently. During a production incident, screening heuristics point to the measurements to collect first, which is where diagnosis begins.

[1] **Arithmetic intensity**: This book uses the term for floating-point operations per byte transferred. Williams et al. (2009) calls the measured DRAM quantity *operational intensity* in the Roofline Model. Comparing the workload ratio with the hardware ridge point ($R_{\text{peak}}/\text{BW}$) gives an upper-bound diagnosis of bandwidth- versus compute-bound execution.

A.5 Diagnosing a Bottleneck

When a benchmark or production service misses its target, diagnosis narrows the search in four passes. Fast screening heuristics come first, followed by confirming hardware counters and profiler traces, targeted interventions on the binding term, and a stopping scorecard that decides when to stop optimizing.

A.5.1 Screening rules of thumb

Under incident pressure there is rarely time to solve the full iron law. Quantitative heuristics narrow the search space quickly; the thresholds below are screening signals to check against profiler traces and hardware counters, not conclusions.

- **Low accelerator utilization (< 80 percent)**: Screen for data, CPU, or launch starvation. Confirm with input-pipeline wait, host CPU saturation, memory-bandwidth counters, and trace gaps.
- **High accelerator utilization (> 95 percent)**: Treat this as machine bound only if compute units are busy and memory bandwidth is not saturated. If memory bandwidth is saturated, the bottleneck is still on the Data/Machine boundary.
- **Batch size of one**: Treat this as a warning that launch or dispatch overhead may dominate; confirm with traces before labeling the algorithm itself the bottleneck.
- **Low arithmetic intensity (below ridge point)**: The workload is likely bandwidth-bound (Data/Machine boundary). The precise boundary is $R_{\text{peak}}/\text{BW}$ and depends on hardware and precision.
- **Development-production mismatch**: Treat data drift as one hypothesis alongside configuration, load, dependency, and runtime differences.

Common industry labels cross the axes. Bandwidth-bound execution sits on the Data/Machine boundary because both bytes moved and available bandwidth matter. Compute-bound execution couples Algorithm work with Machine throughput. Overhead-bound execution may reflect algorithmic serial depth, runtime dispatch, synchronization, or queueing. A screened candidate becomes a diagnosis only when a measurement confirms it.

A.5.2 Confirming the hypothesis

Once a screening heuristic flags a suspect axis, the next step is to collect the hardware counter or profiler trace that can confirm or falsify it. Table A.5 maps common production failure modes to candidate axes and the measurements that validate them.

Table A.5: D·A·M Diagnostic Matrix: First hypotheses and measurements for common failures. Confirm the binding component before intervening.

Symptom	Hypothesis	Confirm With	Act After Confirmation
Low Accelerator Utilization	Data/host starvation	Trace gaps; loader, CPU, storage, launch counters	Prefetch, parallelize, or cut launches.
High Latency (p99)	Algorithm/runtime/queue	Phase p99 and queue depth	Optimize the dominant phase.
High Training Cost	Low useful-work efficiency	MFU, communication, idle time	Reduce the largest loss or resize.
High Inter-GPU Overhead	Communication bound	Device trace; collective gaps	Bucket AllReduce; tune data-parallel overlap.
Out-of-Memory (OOM)	Algorithm/Machine fit	Peak weights, states, activations	Checkpoint, shard, quantize, or shrink batch.

Each measurement in the matrix comes from a concrete hardware or runtime counter. Table A.6 connects the D·A·M components to the profiling utilities that expose them (section 8.5 develops the top-down profiling method these tools serve), and once those measurements confirm which term binds, the remaining question is which optimization will move it.

Table A.6: D·A·M Tooling Map: Profiling utilities for diagnosing bottlenecks along each D·A·M axis. Start with the primary tool for quick triage; use secondary tools for deep-dive analysis when the primary tool's output is inconclusive.

Axis	Key Metric	Primary Tool	Secondary Tool
Data	Batch Load and Wait Time	Framework profiler trace	`iostat`, `pidstat` (I/O/CPU)
Algorithm	FLOPs, Model Depth	PyTorch Profiler	DeepSpeed Flops Profiler
Machine	Accelerator Utilization, SM Occupancy	Nsight Compute	`nvidia-smi`, Nsight Systems

A.5.3 Acting on the binding term

Optimizations must target the confirmed binding term. Table A.7 covers the four regimes the book's lighthouse workloads occupy, plus the overhead-bound regime that any of them enters when the fixed latency term dominates. The compute, bandwidth, and overhead rows are named by terms of the iron law. Capacity-bound and power-bound workloads are limited by preconditions the time equation assumes are already met, that the working set fits in memory and that the energy budget can pay for the run, so either can decide feasibility before any time term is evaluated (section 2.3.2). For each regime, the table contrasts the interventions that yield end-to-end gains against those that waste engineering effort.

Table A.7: What Works vs. What Is Wasted: Optimizing a nonbinding term yields little or no end-to-end improvement. A bandwidth-bound language model will not benefit from additional peak FLOP/s alone, though an accelerator with more memory bandwidth may help. Capacity and power limits act before time does, so they are checked first.

Regime	Binding Term	Lighthouse	Optimization That Works	Optimization That Is Wasted
Compute-bound	$O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	ResNet-50; MobileNetV2; GPT-2 XL training	Better kernels, Tensor Cores, faster accelerator, lower precision	More memory bandwidth (not the binding term)
Bandwidth-bound	D_{vol}/BW	GPT-2 and Llama decode	Quantization; structured pruning; batching for weight reuse; fusion that removes intermediate traffic	Faster accelerator (more FLOP/s will not help)
Overhead-bound	L_{lat}	Any workload served at batch one; small eager kernels	Batch within the latency budget; capture graphs; fuse or dispatch asynchronously; reduce serial depth	More compute or bandwidth unless the trace shows it binding
Capacity-bound	M_{total} vs. M_{device}	DLRM embedding tables	Shard or offload tables; quantize; checkpoint activations; shrink the batch	More FLOP/s or bandwidth before the working set fits
Power-bound	E_{total} per inference vs. budget	Keyword spotting	Integer arithmetic; fewer bytes moved per inference; duty cycling; smaller models	Higher peak FLOP/s that raises power without lowering energy per inference

The lighthouse column names each regime's home workload, and a workload can cross regimes. GPT-2's decode is bandwidth-bound, yet its training is compute-bound (section A.7.3), and ResNet-50, compute-bound in its kernels, becomes input-bound when the data pipeline cannot keep pace (section A.7.1). Section 7.3.4 develops the dispatch overhead behind the overhead-bound row, and section 11.10.3 develops the power envelope behind the last. A repaired system still needs a verdict on whether it is good enough, which is what the scorecard supplies.

A.5.4 Scoring the system

Raw profiler metrics (utilization percentages, FLOP counts, bandwidth measurements) mean little until compared with the workload's targets. The efficiency indicators of the D·A·M scorecard in table A.8 set those targets and show whether a repaired system can stop or must keep investigating. They are screening signals, not universal grades. Model FLOPs utilization (MFU)[2] is especially useful for large-model training.

Table A.8: D·A·M Screening Indicators: Context-dependent signals for investigation, not universal grades.

Axis	Metric	Definition	Investigate	Strong Signal
Data	I/O Overhead	$\frac{\text{Data Wait Time}}{\text{Total Step Time}}$	$> 10\%$ (screen)	$< 1\%$ (strong)
Algorithm	Avoidable Work	Profiler or ablation finds redundant operations	Redundant work found	End-to-end work falls
Machine	MFU	$\frac{\text{Achieved model FLOP/s}}{\text{Peak FLOP/s}}$	$< 30\%$ (large models)	$> 50\%$ (same regime)

The four steps work only when they are followed in order. In practice, teams under deadline pressure skip the confirming step and optimize the wrong axis with confidence, and these failure modes are common enough to deserve their own names.

[2] **MFU**: The ratio of achieved model FLOP/s to the hardware's theoretical peak FLOP/s, introduced in the PaLM paper (Chowdhery et al. 2022). Unlike raw accelerator utilization (which counts any work the accelerator performs), MFU measures model computation rate relative to peak hardware throughput, so non-model work and system overhead are not counted as achieved model FLOPs. Section 8.5 develops MFU as a training diagnostic.

A.6 Anti-Patterns

Before committing to complex kernel optimizations, watch for these common traps that waste engineering cycles. Each anti-pattern arises when a team applies an expensive engineering lever to an axis that does not bind end-to-end execution:

- **The hardware crutch**: Buying faster accelerators (*Machine*) to fix a slow Python data loader (*Data*). The new hardware will just idle faster.
- **The model twiddle**: Changing neural architectures (*Algorithm*) when the bottleneck is actually network bandwidth or disk I/O.
- **The premature optimizer**: Writing custom CUDA kernels (*Machine*) before verifying if the Algorithm is simply doing too many unnecessary operations.

Each anti-pattern follows the same root cause: acting before diagnosing. The case studies that follow show proper diagnosis on the reference accelerator, starting from a confusing symptom and narrowing to the binding term, and the first of them puts a number on the hardware crutch.

A.7 Case Studies on H100

The three cases below run the four-step workflow on an H100, the book's reference accelerator, each with a lighthouse workload in a regime the chapters develop: ResNet-50 starved by its input pipeline, DLRM paying launch overhead at batch one, and GPT-2 XL training against the compute term. In each scenario, an initial utilization reading could easily point to the wrong axis until a second measurement isolates the true bottleneck.

A.7.1 Case 1: The starving accelerator (Data)

A team moves ResNet-50 training from an A100 to an H100 and sees no change in epoch time. A well-tuned A100 sustains about 3,200 images/s, and at the same fraction of its higher peak the H100 would train about 10,100 images/s, yet `nvidia-smi` on the new machine shows utilization oscillating far below that. Screening flags low utilization, and the trace confirms the hypothesis, showing the accelerator idle in gaps while every loader process is saturated decoding JPEGs. With 9 workers each decoding and augmenting about 375 images/s, the pipeline delivers 3,375 images/s regardless of the accelerator, a rate that kept the A100 fully busy while leaving the H100 busy only about 33 percent of the time. Because the upgrade was the hardware crutch, the useful intervention lies upstream of the model: balancing the H100 requires about 28 workers' worth of decode throughput, supplied by adding loader parallelism where host cores allow and by moving decode onto the accelerator, as the supply-and-demand pipeline model in section 8.4.2 develops.

A.7.2 Case 2: The latency cliff (cross-axis)

A recommendation service on an H100 must rank 200 candidate items per request within a 10 ms p99 budget, and it misses badly while accelerator utilization stays low because the serving code scores each candidate with its own DLRM call at batch one. Screening flags the batch size, and a trace confirms an overhead-bound regime: each call issues about 40 kernels at about 15 µs of dispatch and launch apiece, costing 0.6 ms before any device work happens, whereas reading 51.2 KB of embeddings takes only about 15.3 ns at high-bandwidth memory (HBM) bandwidth and the multilayer perceptron (MLP) needs only about 2.1 ns of arithmetic. Because the L_{lat} term exceeds the other two by more than four orders of magnitude, scoring candidates one at a time multiplies that fixed tax to about 120 ms, and a faster accelerator or wider memory bus would shrink only the nanosecond terms. Instead, batching all candidates into one call pays the overhead once to bring the request to about 0.6 ms, while capturing the call as a graph shortens what remains. The cliff runs the other way too: with large tables the same model becomes capacity-bound, the embedding reads grow, and the measurements must be repeated. Section 13.5.1 develops why batching amortizes this fixed cost and where the latency budget stops it.

A.7.3 Case 3: The compute wall (Machine)

A team trains GPT-2 XL on one H100 in TF32. Accelerator utilization is pinned near its maximum, memory-bandwidth counters are unsaturated, and the profiler reports MFU near 50 percent, the

strong-signal end of the scorecard. Because the data path keeps the accelerator fed and the trace shows no dispatch gaps, the compute term binds: whereas GPT-2's home regime in serving is bandwidth-bound decode, training at batch reuses every weight across many tokens and crosses the ridge. The run needs about 1.5 ZFLOP of arithmetic, which at this efficiency takes about 70 days, so only levers on the compute term help while adding loader workers or memory bandwidth changes nothing. Lowering precision from TF32 to BF16 where numerically safe raises the H100's Tensor Core peak by about 2× (Choquette 2023), cutting the run to about 35 days, and data parallelism across the 8 accelerators of one node brings it to about 4 days before communication losses. Section 8.1.1 develops this workload as the training running example, including the memory limits that decide what batch fits.

Taken together, the three cases show why the same utilization number can imply different next steps. Low utilization meant a starved pipeline in the first case and dispatch overhead in the second, while high utilization in the third meant the run was already doing useful work and only the compute term could shrink. The same discipline applies when the question is not how fast the system runs but how much better the model can get.

A.8 Scaling Laws and the Roofline

When a training run's held-out loss flattens despite more GPUs and longer runs, the stall can reflect either a predictable power-law return or a data-quality floor that added compute cannot move. Empirical scaling laws characterize how loss decreases as compute, data, or parameters expand, while roofline bounds identify the physical ceilings imposed by memory bandwidth and arithmetic throughput.

A.8.1 Scaling laws

Scaling laws[3] are empirical power laws that predict how held-out loss changes as resources increase within the fitted regime. Two landmark results are Kaplan scaling (Kaplan et al. 2020), which studied loss against parameter count (P), data (D), and total operations (O), and Chinchilla scaling (Hoffmann et al. 2022), which estimated a compute-optimal balance for the dense autoregressive language-model regime studied, often summarized as roughly 20 training tokens per parameter ($D \approx 20P$). Section C.4.3 derives that balance from a parametric loss surface, so a reader can see where the tokens-per-parameter ratio comes from and which fitted exponents it depends on.

As planning tools, these fits estimate how held-out loss changes with compute inside the observed regime. They do not guarantee a task error-rate reduction or extrapolate unchanged across architectures and data distributions.

A.8.2 The information roofline

When held-out loss flattens below the fitted scaling trend while tokens, parameters, and compute keep growing, adding accelerators or parameters buys no further accuracy. The **information roofline** names this limit that data quality places on what a model can learn, the counterpart to the ceiling the roofline model places on how fast it computes. Because it is a diagnostic label rather than a quantitative law, that flattening pattern tells the team to examine the data before buying more hardware, using four checks to separate the causes: deduplicating the training set and remeasuring tests whether repeated examples inflated the apparent data volume, auditing a sample of labels estimates label noise, comparing per-slice validation loss exposes coverage gaps, and comparing the training and validation distributions detects shift. If loss resumes falling after cleaning or reweighting the data at fixed compute, data quality was binding; if it does not, the cause lies in optimization, model capacity, or evaluation noise, and the scaling-law fit itself should be rechecked.

For classification under a fixed data distribution and loss, the Bayes error rate[4] is an irreducible error floor, not a ceiling. Noise, label ambiguity, coverage gaps, and distribution shift can create data-quality limits above that floor, but they do not define one universal slope or breakpoint.

Adding accelerators or parameters is futile only after measurements establish that data quality is binding. Whether debugging a training step, evaluating hardware utilization, or planning a scaling campaign, measure which axis is binding before choosing a lever.

[3] **Scaling laws**: Empirical relationships, typically power laws with a fitted negative exponent, that predict model loss as a function of dataset size, parameter count, or compute budget. Kaplan et al. (2020) studied these relationships for neural language models at OpenAI; Hoffmann et al. (2022) later estimated the compute-optimal training trade-off for its studied dense language-model regime. Section 9.2.3 develops the compute-optimal frontier and its data-starvation diagnostic in detail.

[4] **Bayes error rate**: The lowest achievable error rate under 0–1 loss for any classifier on a given data distribution, determined by class priors and the overlap between class-conditional distributions (Goodfellow et al. 2016). No amount of data, parameters, or compute can reduce error below this theoretical floor.

Checkpoint A.1: D·A·M diagnosis check

1. A training job shows 95 percent accelerator utilization but loss has plateaued for two epochs. Which D·A·M axes could be responsible, and why is utilization alone insufficient to distinguish them?
2. Your colleague suggests adding more data loader workers to a job where `nvidia-smi` shows 98 percent GPU utilization. What trace evidence is needed before deciding whether this will help?
3. An inference server meets its latency SLO at batch size 1 but fails at batch size 16. Which iron-law terms and queueing effects may have changed, and which measurements identify the binding regime?

A.9 Summary

D·A·M localizes candidate causes across Data, Algorithm, and Machine. The iron law and arithmetic intensity translate those causes into time, operations, and bytes, and the four-step workflow screens, confirms, acts on, and scores them across the four lighthouse regimes, compute-bound, bandwidth-bound, capacity-bound, and power-bound, and the overhead-bound regime that a deployment choice such as batch-one serving can create. The anti-patterns and the H100 case studies show what skipping or following that workflow costs. Scaling laws and the information roofline then separate observed improvement trajectories from data-quality limits. Use the framework as a loop rather than as a label.

Key Takeaways: Where to look first

- Begin with cross-axis hypotheses; a symptom rarely identifies its cause.
- Translate each hypothesis into the relevant iron-law term, or the capacity or power precondition, and a measurement that could falsify it.
- Use arithmetic intensity and the scorecard to identify the binding constraint, not to assign universal grades.
- Apply the corresponding lever and remeasure end to end; a local gain that leaves the binding term unchanged is not a system improvement.

B

Data Foundations

DATA problems rarely present as data problems; they surface as a training run that will not speed up, an accelerator sitting idle, accuracy eroding while dashboards stay green, or an evaluation whose verdict does not survive a second test set. The chapters use the back-of-the-envelope calculations and statistical tools that trace such symptoms back to measurable quantities without deriving them, and this appendix supplies the derivations: transfer times, decode and storage-layout costs, tail probabilities, the sample sizes that make a measured rate trustworthy, and drift divergences. The tools lean on basic probability and on the data term of the iron law.

How to Use This Appendix

This appendix is a reference organized by symptom. Each section states its tool compactly and points to the chapter that teaches the concept and applies it to a lighthouse workload, and table B.1 maps each symptom to the section that carries the tool.

Table B.1: Symptom-to-Section Lookup: Each row pairs a symptom a chapter may name with the tool in this appendix that turns it into a measurement.

When the need is…	Tool	Section
Moving a dataset between sites or regions	Transfer time D_{vol}/BW	section B.1.1
An accelerator starving behind CPU decode	Decode throughput by format	section B.1.2
Reading a few features from wide records	Row versus columnar layout	section B.1.3
A feature pipeline whose cost explodes	Selection, projection, and join costs	section B.1.4
An average that looks fine while users complain	Session-level tail probability	section B.2.1
Deciding whether a measured rate or difference is real	Confidence interval, sample size, McNemar, A/B	section B.2.2
Accuracy drifting silently in production	KL divergence, PSI, KS statistic	section B.2.3
More data that no longer improves the model	Entropy and signal per byte	section B.2.4

The first half of the appendix, Moving and Storing Data, serves the data pipelines in section 4.5. The second half, Probability and Statistics, provides the tests that decide whether a measured number can be trusted for the evaluations in section 12.8.1 and the operational monitoring in section 14.5.2.

B.1 Moving and Storing Data

Before a model can learn from a record, the record must leave storage, cross a network, and be decoded into tensors, and each step costs time. This section collects the reference cost of each step. Chapter 4 develops each of them inside a working pipeline; the numbers here are templates for checking one by hand.

B.1.1 Data gravity

Copying a petabyte of training data across a 1 Gbps wide-area link takes months, whereas shipping the job to the cluster that already holds the data takes minutes. Data gravity[1] names this constraint, which transfer-time arithmetic makes precise, and section 4.9.1 applies it to petabyte-scale training

[1] **Data gravity**: Coined by Dave McCrory in 2010 to describe how large datasets attract services and applications toward them, much as massive bodies attract smaller ones in physics (McCrory 2010).

data. Network bandwidth is finite and does not grow with compute, so moving a large dataset can cost more time and energy than moving computation to where the data already lives.

The ideal line-rate transfer time is $T = D_{\text{vol}}/\text{BW}$ after converting data volume and bandwidth to compatible units (the calculations below convert decimal bytes and bits per second explicitly). For the large volumes here, latency is negligible; section D.3.2 gives the full equation with its latency term. Table B.2 gives lower bounds; protocol overhead and endpoint limits increase them.

Table B.2: The Cost of Inertia: These transfer times explain why we often "ship compute to data" rather than "ship data to compute." Network columns are lower bounds at sustained line rate. The truck entry is an illustrative elapsed-time model for 1 PB: 8 hours to load, 32 hours in transit, and 8 hours to unload, excluding scheduling and provisioning overhead.

Data Volume	1 Gbps (Standard WAN)	10 Gbps (High-End WAN)	100 Gbps (Direct Connect)	Truck (Illustrative)
1 TB	2.2 hours	13.3 minutes	80 seconds	N/A
100 TB	9 days	22.2 hours	2.2 hours	N/A
1 PB	3 months	9 days	22.2 hours	2 days

B.1.2 Serialization cost

[2] **Serialization**: From Latin *serialis* (forming a series). The process of converting in-memory data structures into a byte stream for storage or transmission, and the reverse (*deserialization*). In ML pipelines, the choice of serialization format can dominate end-to-end training time; section 4.10.4 covers pipeline design strategies that minimize this overhead.

Even after data arrives at the machine, it must be decoded before the accelerator can use it.[2] The CPU cost of that decoding can differ by an order of magnitude across formats (table B.3). Parsing text-based formats like JavaScript Object Notation (JSON) or CSV into floating-point numbers is CPU-bound and often leaves the accelerator idle.

Table B.3: Serialization Overhead: These illustrative values show why binary columnar formats can outperform row-based text, but they are not portable benchmarks. Arrow is an in-memory/IPC format; Parquet is a compressed storage format.

Format	Decoding Speed (MB/s)	Relative CPU Decode Cost	Suitability
CSV/JSON	~100 MB/s	High	Inspection/interchange
Protobuf	~300 MB/s	Medium	RPC/Messages
Parquet	~1,000 MB/s	Low	Columnar storage

B.1.3 Row and columnar formats

Decoding speed per byte is only half of the format cost, because the storage layout also decides how many bytes a query must read from disk in the first place. Row-oriented formats such as CSV and JSON store data record-by-record, so reading the `age` field still requires parsing every attribute in each row, a layout that suits append-only logs while penalizing feature filtering. In contrast, Parquet stores compressed column chunks on disk and Arrow provides a contiguous columnar representation in memory and across inter-process communication (IPC), allowing Parquet readers to use *projection pushdown* to fetch only the requested column chunks while both formats feed *vectorized processing*. Where the row layout forces a wide scan across all fields (`ID`, `Age`, `Loc`), the columnar layout isolates the target `Age` chunk and avoids moving irrelevant attributes (figure B.1). Section 4.10.5 develops format choice as a lever on the data term of the iron law.

> **Systems Perspective B.1: The accelerator starvation problem**
>
> The choice of file format often determines whether a training pipeline is I/O bound or compute bound. Row-oriented formats force full-record scans, so storage layout compounds the decode cost whenever models consume a subset of tabular features, and the data-movement term of the iron law can silently become the critical path. Section 4.9.3 develops that starvation condition and the stage that binds it.

B.1.4 The algebra of data

Layout fixes what a scan can skip; the operations a feature pipeline applies decide how much it must read. Feature engineering turns raw records into the columns a model can learn from. A

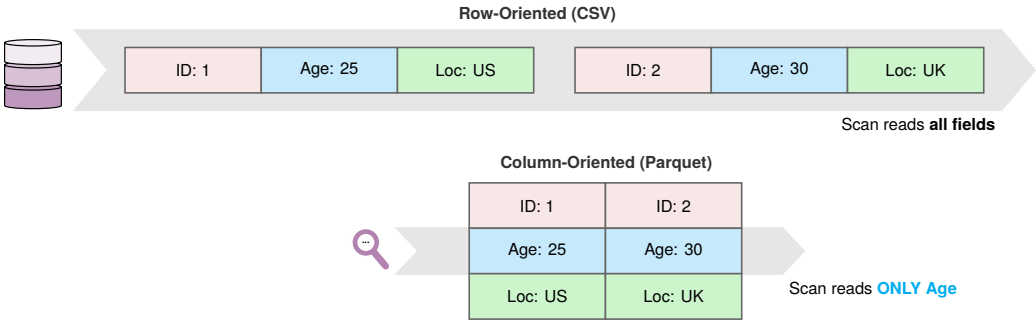


Figure B.1: Storage Layouts: Row-oriented formats group values by record, so the illustrated scan reads every field. Column-oriented formats group values by feature, so the illustrated scan reads only Age.

pipeline first narrows the population, then chooses the features, then attaches context from other tables. Those three moves correspond to three Structured Query Language (SQL) primitives, and their computational cost determines whether the feature job stays local, scans unnecessary bytes, or turns into a network shuffle.

1. **Selection (σ)**: Filtering rows (for example, `WHERE age > 30`).
   - **Cost**: For a table of N rows, $\mathcal{O}(\log N + K)$ if a B-tree index lookup returns K rows; $\mathcal{O}(N)$ for a full scan.
2. **Projection (π)**: Selecting columns (for example, `SELECT age`).
   - **Cost**: Selected chunks plus decoding and metadata in columnar formats. This row-format example reads 1 KB for a 4-byte integer, wasting 99.6 percent of the read.
3. **Join ($\bowtie$)**: Combining tables. Usually the most expensive operation.
   - **Shuffle Join**: Both tables are partitioned by key and exchanged over the network.
     - **Cost**: Reshuffling both 1 TB tables moves ~2 TB; partitioning and locality can reduce this.
   - **Broadcast Join**: One small table is sent to all workers.
     - **Cost**: $s_{\text{table}}(N_{\text{workers}} - 1)$ bytes; the s_{table}-byte table must fit on each of N_{workers} workers.

Section 4.7.4 shows when these costs push feature processing beyond one machine. Formats, serialization, and algebraic primitives determine how efficiently data moves. They say nothing about whether the numbers computed from that data can be trusted, or whether the data has changed character since training. Answering those questions requires probability and statistics.

B.2 Probability and Statistics

A 10 ms mean latency can hide a 63 percent session-level tail-spike rate, a 0.5-point accuracy change on a finite validation set can sit inside binomial noise, and a feature's mean can stay constant while its tails drift enough to degrade predictions. Four statistical tools separate those failures from noise: tail probabilities for multi-request sessions, confidence intervals and paired or online sample-size formulas for finite evaluations, divergence measures for distribution shift, and information-theoretic units for the signal a dataset carries.

B.2.1 Distributions and the long tail

In systems, the mean is often misleading. Latency distributions are often long-tailed, and user-visible services are commonly governed by high-percentile behavior rather than mean response time (Dean and Barroso 2013). A "p99" (99th percentile) latency of 500 ms means at most 1 percent of requests take longer than 500 ms; with many requests per session, the fraction of users who see at least one

slow request can be much higher. At scale (1M users), even a 1 percent affected-user rate would affect 10,000 users.

> **Napkin Math B.1: The median experience**
>
> For $N_{\text{session}} = 100$ independent requests, each with a 0.01 chance of being slow, the probability of at least one p99-tail event is $\Pr(\text{Slow}) = 1 - (0.99)^{100} \approx 1 - 0.366 = 63.4\%$.
>
> **Systems insight**: With independent requests, a 100-request session is more likely than not to include an event beyond p99. Correlated slowdowns can change this probability (Dean and Barroso 2013).

Section 13.4.3 develops tail latency for a serving system, where queuing produces the tail and request fan-out amplifies it. Tail probabilities describe one measured distribution; deciding whether a measured rate is itself trustworthy is a sampling question.

B.2.2 Sampling and estimation

Every accuracy, error rate, and conversion rate in this book is measured on a finite sample, so every one of them carries sampling error. The question that decides a release is whether a measured number, or a measured difference, is larger than that error. Four tools answer it: an interval for one rate, the sample size that makes the interval narrow enough, a paired test for two models scored on the same examples, and the traffic an online A/B test needs. Each reduces to a short formula for the number of labeled examples or user sessions an engineer must collect.

Confidence interval for a rate. A model that classifies a fraction $\hat{p}$ of N independent examples correctly has, by the normal approximation to the binomial, an interval of

$$\hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{N}} \tag{B.1}$$

at the confidence level $z_{\alpha/2}$ encodes (1.96 for 95 percent). ResNet-50 at 76 percent top-1 on the 50,000-image ImageNet validation set is therefore known only to within about 0.4 percentage points. Two models scored on independent samples of that size have a difference interval about $\sqrt{2}$ times wider, because the variances of the two estimates add, roughly 0.5 percentage points, so a smaller accuracy gap between them cannot be resolved that way. Section 12.8.1 applies the interval to benchmark reports and shows why a single accuracy number without one is not a result.

Sample size for a rate. Solving equation B.1 for N gives the number of examples a target half-width requires: the critical value squared, times $\hat{p}(1-\hat{p})$, divided by the square of the half-width. Because the half-width falls only as the square root of the count, halving it quadruples the labeling cost. A keyword spotter expected to detect 98 percent of wake words needs 3,012 labeled one-second clips to pin that rate within 0.5 percentage points (about 50 minutes of audio), and 12,047 clips (3.3 hours) to reach 0.25 percentage points. The count applies to every slice that must be certified separately. Section 15.3.1 multiplies it across demographic subgroups, where a rare group must be oversampled to reach the count at all, and section 9.2.3 needs it to confirm that a curated subset still reaches the target accuracy.

McNemar's test for two models on the same examples. When a baseline and a compressed model are scored on the same examples, the examples both get right or both get wrong say nothing about which is better. Only the discordant counts matter: n_{base} examples that only the baseline gets right and n_{comp} that only the compressed model gets right. The test statistic is

$$\chi^2 = \frac{(n_{\text{base}} - n_{\text{comp}})^2}{n_{\text{base}} + n_{\text{comp}}}, \tag{B.2}$$

and the difference is significant at 95 percent when it exceeds 3.84 (the square of 1.96). Suppose FP32 and INT8 ResNet-50 disagree on the ImageNet validation set as follows, an illustrative split: 300 images only the FP32 model gets right and 250 only the INT8 model gets right. The net loss is 50 images, 0.1 percentage points of accuracy, well inside the unpaired interval above, yet equation B.2 gives $\chi^2 = 4.55$, which exceeds the threshold. Pairing resolves a drop that independent samples

could not, which is why an evaluation harness must keep per-example outcomes rather than only the two accuracies. Section 12.8.1 works through the case in which the same marginal scores leave the verdict open, and chapter 10 produces the INT8 model whose accuracy loss the test checks.

A/B tests. An online comparison of two model versions assigns users at random to a control arm and a treatment arm and compares a rate, such as conversion, between them. With δ_{MDE} the smallest absolute difference worth detecting, σ^2 the per-user variance (the baseline rate times one minus it for a yes-or-no outcome), and z_β the critical value for the chosen power, the users needed per arm are

$$n = \frac{2(z_{\alpha/2} + z_\beta)^2 \sigma^2}{\delta_{\text{MDE}}^2}. \tag{B.3}$$

For a 2 percent relative lift on a 5 percent baseline at 95 percent confidence and 80 percent power, equation B.3 gives the release scenario of section 14.4.3.2 745,644 users per arm. A service that can route 100,000 users per day to each arm needs about 7.5 days to finish one test, and that duration, not compute, sets how often a recommendation model can be released on evidence.

Each of these tools assumes the examples come from one fixed population. When the population itself moves, the question changes from estimating a rate to detecting that the distribution has shifted.

B.2.3 Measuring drift

Comparing means is not enough to catch that shift because a distribution's tails or bin shares can move while its average stays fixed, so comparing the serving distribution (P_t) with the training distribution (P_0) requires a distance measure across the full distribution shape. Section 4.12.1 owns drift detection and its taxonomy, and section 14.5 decides what an alert should trigger; this section collects the three divergence measures they use, with one worked example.

For discrete outcomes x, equation B.4 expresses KL divergence[3] as the expected extra coding cost of approximating P_t with P_0 (Kullback and Leibler 1951; Cover and Thomas 2006). The displayed form is finite only when $P_0(x) > 0$ wherever $P_t(x) > 0$; production implementations therefore need explicit smoothing or zero handling.

$$\mathcal{D}_{\text{KL}}(P_t \| P_0) = \sum_x P_t(x) \log \frac{P_t(x)}{P_0(x)} \tag{B.4}$$

[3] **KL divergence**: Named after Solomon Kullback and Richard Leibler, who introduced it in 1951. Also called *relative entropy*, it quantifies the expected extra information needed to encode samples from one distribution using a code optimized for another; for drift monitoring in this book, the canonical direction is $\mathcal{D}_{\text{KL}}(P_t \| P_0)$. With natural logarithms, as in the example above, the unit is nats; with base-2 logarithms, the unit is bits.

Napkin Math B.2: Worked example: KL divergence for drift detection

Scenario: A keyword-spotting model was trained on audio windows binned by signal-to-noise ratio: 60 percent high, 30 percent medium, and 10 percent low. After the device maker changes the microphone in a new product generation, the windows arriving from the fleet shift to 45 percent high, 40 percent medium, and 15 percent low.

Math:

Training distribution P_0: [0.60, 0.30, 0.10]. Serving distribution P_t: [0.45, 0.40, 0.15].

$$\begin{aligned}\mathcal{D}_{\text{KL}}(P_t \| P_0) &= 0.45 \log \frac{0.45}{0.60} + 0.40 \log \frac{0.40}{0.30} + 0.15 \log \frac{0.15}{0.10} \\ &= 0.45 \times (-0.2877) + 0.40 \times (0.2877) + 0.15 \times (0.4055) \\ &= -0.1295 + 0.1151 + 0.0608 = 0.0464 \text{ nats}\end{aligned}$$

$$\begin{aligned}\mathcal{D}_{\text{KL}}(P_0 \| P_t) &= 0.60 \log \frac{0.60}{0.45} + 0.30 \log \frac{0.30}{0.40} + 0.10 \log \frac{0.10}{0.15} \\ &= 0.60 \times 0.2877 + 0.30 \times (-0.2877) + 0.10 \times (-0.4055) \\ &= 0.1726 + (-0.0863) + (-0.0405) = 0.0458 \text{ nats}\end{aligned}$$

The two directions give different values, $\mathcal{D}_{\text{KL}}(P_0 \| P_t) \neq \mathcal{D}_{\text{KL}}(P_t \| P_0)$. The population stability index (PSI) symmetrizes this by summing the two directions; the calculation below uses natural logarithms:

$$\text{PSI} = \sum (P_{0,i} - P_{t,i}) \log \frac{P_{0,i}}{P_{t,i}} = (0.15)(0.2877) + (-0.10)(-0.2877) + (-0.05)(-0.4055)$$
$$= 0.04315 + 0.02877 + 0.02027 = 0.0922$$

Systems insight: Since PSI = 0.0922 < 0.1, this drift sits just below the conventional warning threshold and well short of the critical threshold of 0.25 used in chapter 14. The right response is to keep the feature on the monitoring dashboard and look for corroborating drift signals rather than retraining from this statistic alone. The statistic is also cheap to collect on an always-on device, which can report three bin counters per interval instead of raw audio, so the check fits the power budget and raw recordings never need to leave the device.

Those PSI thresholds remain illustrative because their meaning depends on sample size, binning, smoothing, and zero handling, and because retraining requires model-quality evidence. Moreover, KL divergence and PSI compare binned or categorical distributions. For a continuous feature, such as the loudness of an audio window, the two-sample Kolmogorov-Smirnov statistic avoids binning altogether: it is the largest gap between the empirical cumulative distribution functions of a training sample and a serving sample,

$$\mathcal{D}_{\text{KS}} = \max_{x_{\text{feat}}} |F_{P_0}(x_{\text{feat}}) - F_{P_t}(x_{\text{feat}})|.$$

Its critical value shrinks with the square root of the sample sizes, so a large serving sample flags even a tiny shift as significant; whether that shift matters is a separate question. Section 4.5.3 works the test through with its critical value. All three measures rest on information theory, which supplies their units.

B.2.4 Information theory for systems

A training run can consume more examples, run longer, and still stop improving if the added data carries too little useful signal. At that point the limit is the information the data pipeline delivers to the learner, not compute or bandwidth. Section A.8.2 treats that data quality limit as a diagnostic constraint, and information theory provides the units for reasoning about it.

- **Entropy (H)**:[4] The average uncertainty in a distribution, defined as $H(X) = -\sum p(x) \log p(x)$. The log base sets the unit: natural logs give nats, while $\log_2$ gives bits. A uniform distribution has maximum entropy only over a fixed finite support. This connects directly to KL divergence above: $\mathcal{D}_{\text{KL}}$ measures the excess coding cost of using the wrong distribution.
- **Information density**: Used here as a book-level heuristic for useful learning signal per unit of storage, not as a standard information-theory quantity.
- **Signal-to-noise ratio (SNR)**: The ratio of signal power to noise power. In ML, the book uses low SNR qualitatively for data in which task-relevant structure is weak relative to noise; adding compute cannot compensate for missing task-relevant signal.

[4] **Entropy**: Shannon adopted the term in 1948 to quantify information content (Shannon 1948). In systems terms, Shannon entropy is a lower bound on average lossless code length. Practical prefix codes approach it, and block codes can approach it asymptotically. These bounds inform compression ratios and data-pipeline sizing.

B.3 Summary

When a data symptom appears, find its row in table B.1 and compute the number. Each section points back to the chapter that teaches the tool, where the lighthouse workloads show it deciding a design.

C

Algorithm Foundations

A profile showing low utilization is often misdiagnosed as a hardware limit when the cause is algorithmic: a matrix multiply shape that wastes Tensor Cores, a layout that breaks contiguity, or activation memory held for backpropagation. This appendix collects the computational complexity models, memory footprint derivations, numerical-stability rules, and proofs that the book's performance chapters lean on, enabling readers to evaluate algorithmic trade-offs before committing code to accelerators. It assumes familiarity with linear algebra and with the iron law introduced early in the book.

How to Use This Appendix

This appendix is designed as a reference. Reach for it when translating a profiler symptom ("slow matmul," "shape mismatch," "OOM during training") into a concrete computational or memory cause. Each symptom or need maps to one section (table C.1), and conventions throughout follow the book-wide notation (for example, B is reserved for batch size and BW denotes bandwidth). Every section names the chapter that teaches its concept in context; the chapter carries the argument, and this appendix carries the derivation or the reference formula.

Table C.1: Where to Look: Each row pairs a profiler symptom or a design question with the section that carries its derivation or reference formula.

Symptom or need	Section
A general matrix multiply (GEMM) runs far below peak throughput	section C.1.3
Sparsity is proposed to save memory or compute	section C.1.4
A layer's parameters and FLOPs are needed before hardware is chosen	section C.1.5
Tensor code fails on shape or contiguity, or a copy dominates a loop	section C.2.1
A broadcast allocates a much larger tensor than intended	section C.2.2
Gradients look wrong, or the backward pass costs more than expected	section C.3.2
Training runs out of memory	section C.3.3
A chain of elementwise operations is memory bound	section C.3.4.2
Softmax overflows, or the loss becomes NaN in reduced precision	section C.3.5
A bound on gradient cost relative to the forward pass	section C.4.1
Attention memory or traffic grows with sequence length	section C.4.2
A compute budget must be split between parameters and training tokens	section C.4.3

Neural networks turn linear algebra into learned behavior. Matrices transform activations, tensor layouts determine how data move through memory, and backpropagation carries error signals through the resulting computation. These foundations support the deep learning treatment in chapter 5, the framework internals in chapter 7, and the training strategies in chapter 8.

C.1 Linear Algebra

Shrinking a square matrix multiply from dimension 4,096 to 64 drops its arithmetic intensity more than tenfold below an H100's ridge point, turning a compute-bound layer into a bandwidth-bound

stall without changing a line of model logic. Most of the compute in a deep learning system is matrix multiplication, and although frameworks such as PyTorch hide the raw math, performance engineering still depends on the underlying linear algebra. Building on how numbers are stored in section D.4, this section focuses on how they are manipulated.

> **Systems Perspective C.1: Why this matters**
>
> Many dense neural networks, especially transformers and large convolutional neural networks, spend most of their compute in matrix multiplication or GEMM-like kernels: a self-attention block executes query, key, value, and output projection GEMMs alongside two attention-matrix products for scores and weighted values, and the layer's feed-forward block adds two or three more. Because those kernels dominate execution, GEMM performance characteristics explain why batch size governs throughput, why aligned layer dimensions run faster than unaligned ones, and whether a dense workload will be compute bound or bandwidth bound before any profiler trace runs.

C.1.1 Tensor operations and einsum notation

Tensor operations in deep learning contract across batch, head, sequence, and hidden dimensions at once, and Einstein summation[1] notation writes those contractions as a single index string rather than a chain of nested loops and transposes (implemented as `torch.einsum` in PyTorch and `np.einsum` in NumPy). Matrix multiplication $\mathbf{C} = \mathbf{AB}$ becomes:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

In einsum notation, this is `ik,kj->ij`. The same notation handles batched multi-head attention, `bhid,bhjd->bhij` (batch b, head h, query sequence i, key sequence j, and head dimension d). The head dimension d appears in both inputs but not in the output, so it is the index that is summed.

[1] **Einstein summation convention**: Repeated indices in a product are implicitly summed over, eliminating explicit summation signs. ML frameworks adopted the convention because it concisely expresses arbitrary tensor contractions in a single string.

C.1.2 The dot product as similarity

Comparing two token embeddings or feature vectors requires a scalar score that grows when the vectors point in the same direction and shrinks when they are orthogonal. The dot product $\mathbf{a} \cdot \mathbf{b} = \sum a_i v_i$ provides that score because it equals $\|\mathbf{a}\|\|\mathbf{b}\| \cos\phi$: for nonzero vectors, its sign distinguishes acute, right, and obtuse angles, while its magnitude also depends on both vector norms.

In attention mechanisms, query (**Q**) and key (**K**) vectors are dot-produced to compute a similarity score that determines how much each token attends to every other token. The resulting attention weights are then used to form a weighted combination of value (**V**) vectors, so every token can draw on every other token. Section 6.6 develops attention itself, including the cost of computing one score for every query-key pair.

C.1.3 General matrix multiply (GEMM)

Whether a dense layer saturates an accelerator's Tensor Cores or stalls on memory bandwidth depends on the three dimensions of its matrix multiply. Most training and inference FLOPs execute as general matrix multiplies (GEMMs):[2] for matrices of size $M \times K$ and $K \times N$, a GEMM performs $2MNK$ floating-point operations (counting multiply-accumulate as two operations).

The arithmetic intensity of GEMM scales linearly with matrix dimension. For square $N \times N$ matrices in FP16 (2 bytes/element), the ideal $\beta = 0$ bound reads **A** and **B** once and writes **C** once:

$$\text{Intensity} = \frac{O}{D_{\text{vol}}} = \frac{2N^3}{3N^2 \times 2} = \frac{N}{3} \text{ FLOP/byte}$$

Reading the old **C** when $\beta \neq 0$ changes the intensity to $N/4$ FLOP/byte. Three consequences follow for GEMM execution:

- **Larger batches can improve efficiency**: Batching increases effective matrix dimensions when it forms larger GEMMs or enables reuse; merely queuing independent small GEMMs does not raise intensity.

[2] **General matrix multiply (GEMM)**: The BLAS (Basic Linear Algebra Subprograms) family began with vector routines standardized in 1979 (Lawson et al. 1979); GEMM was standardized later as a Level 3 routine (Dongarra et al. 1988). The "GE" prefix stands for "general," as opposed to symmetric or triangular forms. GEMM computes $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$ and is a performance-critical routine in deep learning. Section 8.3.1.1 explains how GEMM shapes determine training throughput.

- **Aligned dimensions help**: Hardware Tensor Cores are optimized for precision- and architecture-specific tile multiples. Dimensions that align with these multiples avoid padding overhead and improve kernel efficiency, but they do not need to be powers of two.
- **Small matrices are inefficient**: A square GEMM with $N = 64$ has intensity $64/3 \approx 21.3$ FLOP/byte, well below the ridge point of the book's reference accelerator, the H100 (295.2 FLOP/byte). Its roofline cap is ~7.2 percent of peak; launch and tiling overhead can lower realized throughput. Section 11.6 derives the ridge point and the compute-bound and bandwidth-bound regimes on either side of it, which is the comparison each of these consequences relies on.

C.1.4 Sparse matrix formats

When most elements in a matrix are zero, specialized storage formats avoid wasting memory on zeros and enable computations that skip them entirely. The **compressed sparse row (CSR)** format uses three arrays:

- `Values`: The nonzero elements, stored in row order
- `Col_Idx`: The column index of each nonzero element
- `Row_Ptr`: The starting position in `Values` for each row (length = num_rows + 1)

CSR is useful for sparse feature matrices in recommendation pipelines and for pruned model weights. For a matrix with n_{elem} elements, n_{rows} rows, and n_{nz} nonzeros, CSR uses $\mathcal{O}(n_{\text{nz}} + n_{\text{rows}})$ storage instead of $\mathcal{O}(n_{\text{elem}})$; when n_{nz} is large relative to n_{rows}, this is often summarized as $\mathcal{O}(n_{\text{nz}})$.

To see the trade-off concretely, consider a vocabulary embedding matrix with 100,000 rows and 10,000 columns (1B parameters):

- **Dense (FP32)**: 1B parameters $\times$ 4 bytes each = 4 GB.
- **Sparse** (1 percent density): CSR stores roughly 10M entries $\times$ (4-byte value + 4-byte column index), plus one row pointer per row and a final pointer, totaling $\approx$ 80.4 MB.
- **Result**: A 50$\times$ reduction in memory footprint, fitting a model that would otherwise trigger an out-of-memory (OOM) error.

The saving holds only at low density. Each stored nonzero carries its index, so at high density CSR occupies more memory than the dense tensor it replaces. Section 10.4.2.2 finds that break-even density for pruned weights, and section 11.4.5.3 shows the fixed pattern of two nonzeros in every four that an accelerator datapath decodes without a general index.

C.1.5 Operation cost reference

Each common layer type reduces to the GEMMs and elementwise operations above, so its parameter count and forward-pass FLOPs follow from its shapes (table C.2). Given layer dimensions and input shapes, the formulas give first-order estimates of model size and compute before hardware is provisioned. The attention row is the one to watch. Sequence length contributes an S^2 term, while hidden dimension contributes d^2 projection work, so long-context models can shift the dominant cost without changing the parameter count.

Table C.2: Deep Learning Tensor Primitives: Summary of shapes, parameters, and FLOP counts. Note: B is batch size, S is sequence length, and K is kernel size. The attention FLOPs include QKV projections and score interactions. The S^2 term applies to full-sequence training and prefill, while KV-cached generation is linear per token in cached context; d^2 projection work dominates at large hidden dimensions.

Layer type	Output shape	Parameters (P)	FLOPs (per forward pass)
Linear	(B, d_{out})	$d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$ if bias is enabled	$2 \times B \times d_{\text{in}} \times d_{\text{out}}$
Conv2D	$(B, d_{\text{out}}, H_{\text{out}}, W_{\text{out}})$	$K^2 \times d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$ if bias is enabled	$2 \times B \times H_{\text{out}} \times W_{\text{out}} \times K^2 \times d_{\text{in}} \times d_{\text{out}}$
Multi-head self-attention	(B, S, d)	$4 \times d^2$, plus $4 \times d$ if projection biases are enabled	$B \times (4S^2d + 8Sd^2)$
LayerNorm	(B, S, d)	$2 \times d$ if affine scale and bias are enabled	$\mathcal{O}(B \times S \times d)$

The formulas in table C.2 specify *what* each layer computes and how many operations it performs. Expressing those computations in code raises a second question: how frameworks represent multidimensional arrays in flat memory without copying data unnecessarily.

C.2 Tensor Programming Primitives

A shape mismatch crash, a silently wrong broadcast, and a kernel running at 5 percent of peak because of a noncontiguous tensor are common ML engineering failures, and all three trace back to the same layer of abstraction. *Tensor programming* translates the abstract math of linear algebra into concrete array manipulations that run on hardware.

C.2.1 Shapes, strides, and memory layout

Frameworks represent a tensor as a view over a flat storage buffer, described by shape, stride, dtype, and offset metadata rather than as a multidimensional array of pointers. Its shape specifies the logical dimensions of the tensor (for example, `(3, 4)`), while its stride defines how many elements to skip in storage to advance one step along each dimension, which means only contiguous tensors store their logical elements in a single adjacent block.

Operations like `transpose()` or `view()` often just change the strides, not the data in memory. This is fast ($\mathcal{O}(1)$) but can lead to noncontiguous tensors that fail in kernels requiring contiguous data. In such cases, calling `contiguous()` on a tensor of N elements forces an $\mathcal{O}(N)$ memory copy that can dominate runtime if triggered repeatedly inside a loop.

The order of the strides is the tensor's memory layout. Row-major and column-major layouts of the same matrix differ only in which dimension has stride 1, and that choice directly affects cache efficiency: iterating along the stride-1 dimension touches contiguous memory locations, which is dramatically faster than strided access.

A common optimization pattern is to materialize a transposed tensor once before repeated operations to ensure contiguous access in the hot loop. The one-time copy cost is amortized across many subsequent operations. Section 7.5.1.2 introduces strides and layouts in the framework that manages them, and section 11.7.6 shows how the layout choice (channels-last versus channels-first) changes accelerator throughput for convolutions.

C.2.2 Broadcasting

Zero-copy stride manipulation also powers **broadcasting**, the rule that lets arithmetic operators combine tensors of different shapes without materializing duplicate copies of the smaller input. Frameworks compare dimensions from right to left and treat two dimensions as compatible when they are equal or when one of them is 1; a singleton dimension is then stretched virtually by setting its stride to zero, though an unintended singleton alignment, such as adding a $(B, 1)$ column vector of predictions to a $(B,)$ row vector of targets, silently allocates a (B, B) output matrix that can exhaust memory or corrupt a loss calculation.

Adding a $(3, 1)$ column tensor (**A**) to a $(1, 4)$ row tensor (**B**) expands both virtually across their complementary singleton dimensions and generates a $(3, 4)$ matrix of pairwise sums $A_i + b_j$ (figure C.1). Dashed cell boundaries mark result elements whose operands are read through stride-0 views, so neither input is copied; only the $(3, 4)$ output is allocated, and hardware repeatedly accesses identical input buffer addresses.

Consider a concrete case: tensor A has shape `(32, 1, 64)` and tensor B has shape `(1, 128, 64)`. Comparing dimensions right to left, 64 matches 64, then 1 stretches to 128, then 1 stretches to 32, yielding result shape `(32, 128, 64)`. Writing out the expanded shape before running catches silent logic bugs that accidentally allocate a large tensor (for example, a `(Batch, Batch)` matrix instead of an element-wise `(Batch)` vector).

Shapes, strides, and broadcasting govern how tensors flow through a model's forward pass. Training adds a second requirement, computing how much each parameter contributed to the error. Backpropagation meets that requirement and imposes the memory costs quantified in the training-memory decomposition below.

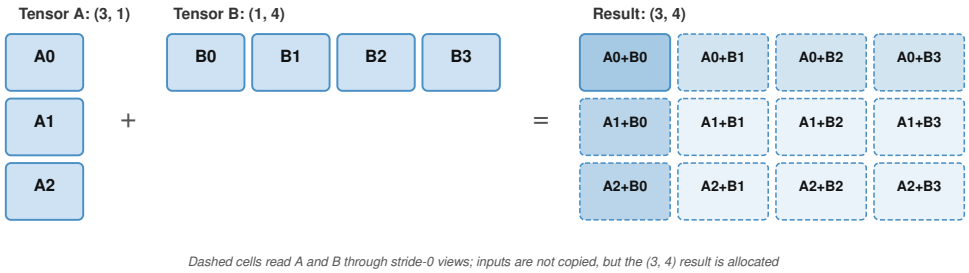


Figure C.1: Tensor Broadcasting Rules: A (3,1) column and a (1,4) row broadcast across complementary dimensions, producing the pairwise sums in a (3,4) result.

C.3 Mechanics of Learning

Training a neural network takes roughly three times the compute of a forward pass and holds gigabytes of intermediate activations in memory until the backward sweep consumes them. This section traces how the chain rule and reverse-mode automatic differentiation create that compute and memory footprint, how compilers fuse the resulting computational graph, and how the log-sum-exp shift prevents the softmax at the loss from overflowing in reduced precision.

> **Systems Perspective C.2: Why this matters**
>
> When training fails (loss goes to NaN, gradients explode, or memory runs out), understanding what backpropagation actually does is essential for diagnosing the problem. The backward graph supplies the mental model for reasoning about gradient flow and memory usage during training.

C.3.1 The chain rule and automatic differentiation

For a composed function $y = f(g(x))$, the derivative is $\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$. In a neural network, f and g are layers, and the composition can be many levels deep. For a three-layer network $y = f_3(f_2(f_1(x)))$, the chain rule extends to:

$$\frac{\partial y}{\partial x} = \frac{\partial f_3}{\partial f_2} \cdot \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial x}$$

Each factor in this product is a local derivative, computed at one layer using only that layer's inputs and outputs. This locality keeps the bookkeeping cheap: the entire network never needs to be differentiated as a monolithic function. Instead, each layer computes its own local derivative during the backward pass and multiplies it by the gradient flowing in from the layer above.

Modern frameworks use reverse-mode automatic differentiation, which computes gradients for all P parameters in a single backward pass. Starting from the output and working backward (reverse mode) requires one pass regardless of the number of parameters, whereas starting from each input and working forward (forward mode) would require P passes, one per parameter. This is why a training step is a small constant multiple of inference, commonly about 2–3× a forward pass for dense networks, rather than P passes. Section 7.4.1 shows how frameworks implement both modes and why training uses reverse mode.

C.3.2 The backpropagation algorithm

Backpropagation[3] implements the chain rule efficiently through two passes: forward to compute outputs, backward to compute gradients. The gradient edges retrace forward dependencies, meaning each backward step requires intermediate activations retained from the corresponding forward operation. Figure C.2 traces these dual dataflow passes across a two-layer network: solid gray arrows stage forward activations ($\mathbf{x} \to \mathbf{h} \to \hat{\mathbf{y}}$) through weight parameters $\mathbf{W}_1$ and $\mathbf{W}_2$ to evaluate loss $\mathcal{L}$, while curved dashed red arrows propagate intermediate adjoints ($\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}}, \frac{\partial \mathcal{L}}{\partial \mathbf{h}}, \frac{\partial \mathcal{L}}{\partial \mathbf{x}}$) in reverse order.

[3] **Backpropagation**: Short for "backward propagation of errors." The algorithm was independently discovered multiple times, by Werbos (1974) and Linnainmaa (1970) for reverse-mode differentiation and by Rumelhart et al. (1986) for neural network training. Its key insight is that computing gradients for all parameters requires one backward pass through the graph rather than one pass per parameter. Section C.4.1 derives the constant-factor work bound and a representative dense-layer cost.

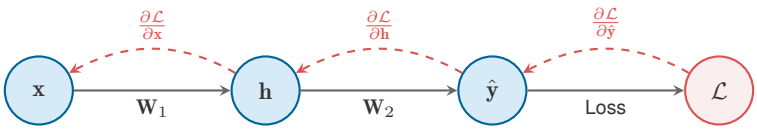


Figure C.2: Backpropagation Computational Graph: Solid gray arrows carry the forward computation from input $\mathbf{x}$ to loss $\mathcal{L}$, while dashed red arrows carry gradients backward along the same dependencies.

Forward pass

Using row-vector activations, omitting biases, and retaining intermediate activations for backward execution, computation begins at input $\mathbf{x}$. Multiplying $\mathbf{x}$ by weight matrix $\mathbf{W}_1$ yields hidden activation $\mathbf{h}$, which the execution runtime caches for backward differentiation. Multiplying $\mathbf{h}$ by $\mathbf{W}_2$ produces network output $\hat{\mathbf{y}}$, which is similarly cached. Finally, comparing $\hat{\mathbf{y}}$ to the target label produces scalar loss $\mathcal{L}$.

At this point, forward evaluation is complete, and memory holds input $\mathbf{x}$, cached activation $\mathbf{h}$, cached prediction $\hat{\mathbf{y}}$, and loss $\mathcal{L}$. For large models, these retained activation tensors frequently dominate overall memory consumption.

Backward pass

Gradient propagation begins backward from scalar loss $\mathcal{L}$. The loss function evaluates $\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}}$, the error gradient with respect to the network prediction, which starts the backward pass.

Because $\hat{\mathbf{y}} = \mathbf{h} \cdot \mathbf{W}_2$, the matrix chain rule yields two gradients at this layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \mathbf{h}^T \cdot \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \qquad \text{(weight gradient—used to update } \mathbf{W}_2\text{)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \cdot \mathbf{W}_2^T \qquad \text{(input gradient—passed backward to the previous layer)}$$

Computing $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2}$ requires cached activation $\mathbf{h}$ from the forward pass. Activations must remain resident in memory because every layer's weight gradient depends directly on that layer's input.

Continuing backward to the first layer, the linear relationship $\mathbf{h} = \mathbf{x} \cdot \mathbf{W}_1$ produces:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \mathbf{x}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}}$$

Each backward step requires two operands: the gradient flowing in from downstream layers $\left(\frac{\partial \mathcal{L}}{\partial \mathbf{h}}\right)$ and that layer's cached input ($\mathbf{x}$). Consequently, the backward pass requires roughly $2\times$ the forward pass FLOPs: each layer executes two matrix multiplications (one for the parameter gradient, one for the upstream activation gradient) rather than one. Section 5.4.5 teaches the algorithm step by step on a running network. The cached inputs that every backward step needs are the reason training memory exceeds model size, which the next section quantifies.

C.3.3 Training memory

Because the backward pass holds those intermediate activations alongside weights, gradients, and optimizer buffers, training memory is the sum of four components rather than model size alone:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$$

For a standard Adaptive Moment Estimation (Adam) optimizer (Kingma and Ba 2015) in mixed precision (Micikevicius et al. 2018):

- **Weights**: 2 bytes (FP16/BF16) or 4 bytes (FP32).
- **Gradients**: Same size as weights.
- **Optimizer State**: Adam moments require 8 bytes per parameter; an FP32 master copy adds another 4 bytes, giving 8–12 bytes per parameter.

- **Activations**: Activations typically dominate the dynamic memory footprint during training. A recomputation- or tiled-attention-friendly estimate scales as $\mathcal{O}(B \times S \times N_L \times d)$; a full no-recompute attention implementation also materializes attention-score tensors that scale with S^2.

To see how these four components interact on a single accelerator, consider a concrete language model. Once the static model state is allocated, the remaining device capacity sets a hard ceiling on the batch size that can run without recomputation.

Napkin Math C.1: Worked example: GPT-2 (1.5B) training memory

The model: GPT-2 XL has $P = 1.5 \times 10^9$ parameters, 48 layers, hidden dimension $d = 1600$.
Model state (fixed per step):

- **Weights (BF16)**: $1.5 \times 10^9 \times 2$ bytes = 3 GB
- **Gradients (BF16)**: $1.5 \times 10^9 \times 2$ bytes = 3 GB
- **Optimizer (FP32 master + momentum + variance)**: $1.5 \times 10^9 \times 12$ bytes = 18 GB
- **Total model state**: 24 GB, which fits on an 80 GB-class H100 (85.9 GB in decimal units) but leaves only 61.9 GB for activations.

Activations (scale with batch):
Per-layer retained activations for this quick estimate are approximately $12 \times B \times S \times d$ BF16 elements, or $12 \times B \times S \times d \times 2$ bytes, where B is batch size and S is sequence length. The factor twelve accounts for the major intermediate tensors retained for backpropagation: input activations, QKV projections ($3d$), attention output, FFN intermediate ($4d$), and layer norm/dropout masks. This estimate assumes the implementation does not retain the full $B \times N_{\text{heads}} \times S^2$ attention-score tensor, as in recomputation- or tiled-attention-friendly accounting. With 48 layers, batch size 8, and sequence length 1024:

$$48 \times 12 \times 8 \times 1024 \times 1600 \times 2 \text{ bytes} \approx 15.1 \text{ GB}$$

Systems insight: Total memory reaches ~39.1 GB under this activation-accounting convention, which fits on one 85.9 GB accelerator. Each additional sample adds ~1.9 GB of activations, so under this quick estimate the remaining capacity fills near batch 32; at batch 64, activations grow to ~120.8 GB, well beyond it. The estimate omits smaller retained tensors such as dropout masks and softmax inputs. The full per-layer count of 34 bytes per token per hidden unit (Korthikanti et al. 2023) fills the same room near batch 23, the limit section 8.6.6 works from. That crossover is the threshold where gradient checkpointing or another memory-saving method becomes necessary; its savings and recomputation overhead depend on checkpoint placement and implementation.

C.3.3.1 Activation explosion

While weights are fixed ($\mathcal{O}(P)$), activations grow linearly with batch size and at least linearly with sequence length; implementations that materialize attention scores add an $\mathcal{O}(S^2)$ component. As the worked example above shows, activation memory can quickly exceed the room left after model state. Gradient checkpointing[4] reduces this pressure by storing only selected activations and recomputing the rest during backpropagation (T. Chen et al. 2016), and FlashAttention removes the quadratic term by tiling attention (Dao et al. 2022). The same decomposition provides a quick feasibility check: sum the per-parameter model state, add the activation term implied by batch size and sequence length, and compare the total against the accelerator's capacity. Section 8.3.2 applies the model-state terms to optimizer choice, and section 8.3.3 develops the activation term, with section 8.6.5.1 teaching the recomputation trade-off for the training loop.

[4] **Gradient checkpointing**: Trades compute for memory. Instead of storing all activations, only a subset (checkpoints) is kept, and the missing ones are recomputed during the backward pass. This reduces memory usage from storing every layer activation to roughly the square root of the layer count at the cost of ~33 percent more compute (T. Chen et al. 2016).

C.3.4 Computational graphs and optimization

The dependency structure exposed by backpropagation also gives compilers something to optimize. ML compilers represent models as directed acyclic graphs (DAGs), and that representation enables hardware-independent transformations.

C.3.4.1 Static single assignment

To transform a DAG safely, a compiler must know which write each read depends on. Static single-assignment (SSA) form makes those dependencies explicit by assigning each variable exactly once rather than overwriting tensor variables in place, enabling safe transformations such as operator fusion.

C.3.4.2 Operator fusion

Without fusion, each operation in a chain like `MatMul` → `Add (bias)` → `ReLU` produces an intermediate tensor that is written to high-bandwidth memory (HBM) and then read back for the next operation. For elementwise operations like `Add` and `ReLU`, the compute is trivial (one FLOP per element) but the memory traffic is not (read the tensor, write it back). The arithmetic intensity of unfused elementwise operations is therefore close to zero, which makes them strongly memory bound.

Fusion combines consecutive operations into a single kernel that reads the input once, applies all operations in registers or shared memory, and writes the final result once. For a sequence of n_{ops} elementwise operations on a tensor of N elements of s_{elem} bytes each, fusion reduces memory traffic from $2n_{\text{ops}}Ns_{\text{elem}}$ bytes (each operation reads and writes) to $2Ns_{\text{elem}}$ bytes (one read, one write), a $n_{\text{ops}}\times$ reduction. Section 7.3.2 develops fusion as a framework decision, and section 11.7.7 shows the accelerator-side limits on how much a single kernel can fuse.

FlashAttention is fusion applied to attention: it computes the score, softmax, and value-product stages in SRAM tiles without materializing the full attention matrix in HBM, reducing auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$ for a fixed head dimension (Dao et al. 2022). Section C.4.2 derives the traffic bound step by step, and section 8.6.4 applies the algorithm to training.

Fused and tiled kernels keep results exact while moving fewer bytes. The softmax at the end of the forward pass raises a different question, whether its results stay finite at all.

C.3.5 Logits and numerical stability

A classifier's forward pass ends in a softmax over the final layer's outputs, and the cross-entropy loss is computed from that softmax. The softmax exponentiates those outputs, and the exponential of a large output exceeds the range of every reduced-precision format, so the softmax is a common overflow site in reduced-precision training.

Neural-network classifiers commonly output a vector of *logits*[5] $\mathbf{z}$ (unnormalized scores). For class index j, softmax normalizes z_j over all class indices k to produce a probability:

$$\text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

The problem is that if z_j is large (for example, 100), the exponential e^{z_j} overflows common training and inference formats such as FP32, BF16, and FP16. FP64 can represent this specific value, but production ML kernels rarely use FP64 for softmax. The fix is to subtract the largest logit before exponentiating, the log-sum-exp trick. The shift leaves the result mathematically unchanged and keeps every exponent at or below zero, as the worked example below illustrates for realistic logit values.[6]

> Napkin Math C.2: Log-sum-exp in action
>
> **Setup**: A three-class classifier outputs logits $\mathbf{z} = [100, 101, 102]$.
> **Without the trick (naive softmax)**:
>
> $$\exp(100) \approx 2.7 \times 10^{43}, \exp(101) \approx 7.3 \times 10^{43}, \exp(102) \approx 2.0 \times 10^{44}$$

[5] **Logit**: Joseph Berkson coined the term in 1944 by analogy with *probit*. The logit function is the inverse of the logistic (sigmoid) function: $\text{logit}(p_{\text{sig}}) = \log(p_{\text{sig}}/(1 - p_{\text{sig}}))$. In deep learning, "logits" refers more loosely to the raw, unnormalized output of the final linear layer before any activation function is applied.

[6] **Log-sum-exp**: Implemented as `torch.logsumexp` in PyTorch and `scipy.special.logsumexp` in SciPy. It relies on the identity $\log(\sum e^{z_k}) = m + \log(\sum e^{z_k - m})$, where $m = \max(z_k)$. Shifted values $z_k - m$ are ≤ 0, ensuring exponentials never overflow.

These numbers are representable in FP64 but overflow FP32 (max $\approx 3.4 \times 10^{38}$). With FP16 (max $\approx 65{,}504$), even e^{12} overflows. Raw logits rarely reach magnitude 100, but large intermediate values do arise (for example, unscaled attention scores or overconfident outputs), and because FP16 and BF16 are the standard training precisions, a naive softmax risks overflow often enough that the stable form is used by default.

With the trick: Subtract $m = \max(\mathbf{z}) = 102$:

Shifted logits: $\mathbf{z} - m = [-2, -1, 0]$

Exponentials: $\exp(-2) \approx 0.135$, $\exp(-1) \approx 0.368$, $\exp(0) = 1.0$

Sum = 1.503. LogSumExp = $102 + \log(1.503) = 102.408$.

Softmax: $[0.135/1.503,\ 0.368/1.503,\ 1.0/1.503] = [0.090,\ 0.245,\ 0.665]$

Systems insight: The exponentiated shifted values and the resulting softmax probabilities are in $[0, 1]$, so nothing overflows, even in FP16.

Section 5.4.3.2 builds the cross-entropy loss on this stable softmax. The same maximum subtraction reappears, generalized to tiles of a long sequence, in the tiled-attention derivation of section C.4.2, one of the three proofs that close this appendix.

C.4 Derivations

Without a constant-factor bound on reverse-mode differentiation, training a 175-billion-parameter model would require 175 billion forward passes per step; without an online softmax recurrence, attention would have to materialize an $S \times S$ matrix in HBM; and without a constrained-optimum derivation, a fixed compute budget could be wasted on an oversized model starved of tokens. Each subsection below states one of these three results, proves it, and closes on the systems consequence that the chapters rely on.

C.4.1 Cost of reverse-mode differentiation

Evaluating the full gradient vector $\nabla f(\mathbf{x})$ of a scalar loss function $f : \mathbb{R}^N \to \mathbb{R}$ requires compute proportional to evaluating $f(\mathbf{x})$ itself, with a constant-factor bound independent of the input dimension N. Baur and Strassen proved this complexity result in 1983. Computing a function together with its full gradient takes at most four times the operations of the function alone when every operation counts, and at most three times when only multiplications and divisions count (Baur and Strassen 1983). The per-operation accounting below is simpler than their construction and proves the looser constant of five.

C.4.1.1 Statement

Let $f : \mathbb{R}^N \to \mathbb{R}$ be a multivariate scalar-valued function represented as a computational graph $G = (V, E)$ consisting of $W(f)$ elementary operations (additions, subtractions, multiplications, divisions, and unary functions such as $\exp, \ln, \sin, \sqrt{\cdot}$).

Theorem C.1: Cheap-gradient bound

If a scalar function $f : \mathbb{R}^N \to \mathbb{R}$ can be computed in $W(f)$ elementary operations, then the full gradient vector $\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_N}\right)^T \in \mathbb{R}^N$ can be evaluated in total work $\text{Work}_{\text{Reverse}}(f, \nabla f)$ satisfying:

$$\text{Work}_{\text{Reverse}}(f, \nabla f) \leq C \cdot W(f)$$

where $C \leq 5$ is a universal constant independent of the input dimension N. Furthermore, the backward pass alone requires work $\text{Work}_{\text{Backward}}(\nabla f) \leq 4 \cdot W(f)$.

C.4.1.2 Proof

Let the computational graph $G = (V, E)$ be topologically ordered with nodes $v_{1-N}, \ldots, v_0, v_1, v_2, \ldots, v_W$, where input variables are $x_m = v_{i-N}$ for $m \in \{1, \ldots, N\}$ and the final output is $f(\mathbf{x}) = v_W$. Each intermediate node v_j is evaluated via a primitive operation:

$$v_j = \phi_j(\{v_k \mid k \in \text{Parents}(j)\})$$

Define the *adjoint variable* $\bar{v}_i$ for every node v_i as the partial derivative of the scalar output v_W with respect to v_i:

$$\bar{v}_i \triangleq \frac{\partial f}{\partial v_i} = \frac{\partial v_W}{\partial v_i}$$

By the multivariate chain rule, the adjoint $\bar{v}_i$ is the sum of partial derivatives propagated from all successor nodes (children) that consume v_i:

$$\bar{v}_i = \sum_{j \in \text{Children}(i)} \bar{v}_j \cdot \frac{\partial v_j}{\partial v_i}$$

Reverse-mode automatic differentiation initializes $\bar{v}_W = 1$ and $\bar{v}_i = 0$ for all $i < W$, then traverses the nodes in reverse topological order from v_W down to v_1. For each primitive node v_j, its fully accumulated adjoint $\bar{v}_j$ propagates error signals to its parent nodes v_k ($k \in \text{Parents}(j)$) via:

$$\bar{v}_k \leftarrow \bar{v}_k + \bar{v}_j \cdot \frac{\partial v_j}{\partial v_k}$$

We bound the backward cost one operation type at a time:

1. **Addition/Subtraction ($v_j = v_i \pm v_k$)**:
   - Local derivatives: $\frac{\partial v_j}{\partial v_i} = 1$, $\frac{\partial v_j}{\partial v_k} = \pm 1$.
   - Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j$ and $\bar{v}_k \leftarrow \bar{v}_k \pm \bar{v}_j$.
   - Cost: 2 additions in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(2) = 3$ OPs $\leq 5 \cdot 1$.
2. **Multiplication ($v_j = v_i \cdot v_k$)**:
   - Local derivatives: $\frac{\partial v_j}{\partial v_i} = v_k$, $\frac{\partial v_j}{\partial v_k} = v_i$.
   - Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j \cdot v_k$ and $\bar{v}_k \leftarrow \bar{v}_k + \bar{v}_j \cdot v_i$.
   - Cost: 2 multiplications + 2 additions = 4 operations in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(4) = 5$ OPs $\leq 5 \cdot 1$.
3. **Division ($v_j = v_i / v_k$)**:
   - Local derivatives: $\frac{\partial v_j}{\partial v_i} = \frac{1}{v_k}$, $\frac{\partial v_j}{\partial v_k} = -\frac{v_i}{v_k^2} = -\frac{v_j}{v_k}$.
   - Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \frac{\bar{v}_j}{v_k}$ and $\bar{v}_k \leftarrow \bar{v}_k - \frac{\bar{v}_j \cdot v_j}{v_k}$.
   - Cost: Compute the quotient of the incoming adjoint and the denominator once, then reuse it in both updates: add it to the numerator adjoint, and multiply it by the cached forward output before subtracting from the denominator adjoint. This requires 2 multiplications/divisions + 2 additions = 4 operations in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(4) = 5$ OPs $\leq 5 \cdot 1$.
4. **Unary transcendental operations ($v_j = \phi(v_i)$)**:
   - Local derivative: $\frac{\partial v_j}{\partial v_i} = \phi'(v_i)$.
   - Backward update: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j \cdot \phi'(v_i)$.
   - For $v_j = \exp(v_i)$, $\phi'(v_i) = v_j$, requiring 1 multiplication + 1 addition = 2 OPs.
   - For $v_j = \ln(v_i)$, $\phi'(v_i) = 1/v_i$, requiring 1 division + 1 addition = 2 OPs.

Summing across all primitive nodes in G, every forward operation generates at most 4 backward operations:

$$\text{Work}_{\text{Backward}}(\nabla f) \leq 4 \cdot W(f)$$

Adding the forward pass work $W(f)$ yields the total work bound:

$$\text{Work}_{\text{Reverse}}(f, \nabla f) = W(f) + \text{Work}_{\text{Backward}}(\nabla f) \leq 5 \cdot W(f) \qquad \blacksquare$$

> **Systems Perspective C.3: Why 175B-parameter large language models can be trained**
>
> The independence of $C \leq 5$ from input dimension N is the mathematical reason deep learning works. Consider training a transformer model with $P = 175 \times 10^9$ parameters:
>
> - **Forward pass**: Matrix multiplications $\mathbf{Y} = \mathbf{XW}$ require $2P$ FLOPs per token.
> - **Reverse-mode backward pass**: Computes both weight gradients $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$ ($2P$ FLOPs) and input activation gradients $\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \mathbf{W}^T$ ($2P$ FLOPs), totaling $4P$ FLOPs per token.
> - **Total training work**: $2P(\text{fwd}) + 4P(\text{bwd}) = 6P$ FLOPs per token. The backward compute factor is exactly $\frac{4P}{2P} = 2$, so forward plus backward costs three times the forward pass, within both the bound of $C \leq 5$ proved here and Baur and Strassen's sharper constant of four.
>
> If we instead used forward-mode automatic differentiation (AD) or finite differences to compute $\nabla f \in \mathbb{R}^P$, evaluating each partial derivative separately would require P forward passes: $\text{Work}_{\text{ForwardMode}}(\nabla f) = \mathcal{O}(P \cdot W) = \mathcal{O}(P^2)$ FLOPs. For $P = 175 \times 10^9$, that method would require $2 \times (175 \times 10^9)^2 \approx 6.1 \times 10^{22}$ FLOPs per token, P times the forward-pass work and therefore prohibitive at this scale. Reverse-mode AD replaces this $\mathcal{O}(P)$ factor with a constant multiplier of 2 for the dense-layer accounting above.

Reverse mode achieves this constant factor by keeping the forward pass's intermediate values for the backward sweep, which is why the training-memory decomposition in section C.3.3 carries an activation term. Attention adds an intermediate whose size grows with the square of the sequence length, the subject of the next derivation.

C.4.2 Tiled attention and online softmax

Standard self-attention in transformers requires materializing an $S \times S$ attention score matrix in HBM, creating an $\mathcal{O}(S^2)$ memory bandwidth bottleneck. FlashAttention (Dao et al. 2022) reduces this bottleneck by tiling query, key, and value matrices and recomputing attention using an online softmax recurrence without materializing intermediate scores to HBM. Section 8.6.4 applies the algorithm to training; this subsection proves the online softmax recurrence it relies on and then its memory-traffic bound.

C.4.2.1 Memory traffic of standard attention

Given queries $\mathbf{Q} \in \mathbb{R}^{S \times d_{\text{head}}}$, keys $\mathbf{K} \in \mathbb{R}^{S \times d_{\text{head}}}$, and values $\mathbf{V} \in \mathbb{R}^{S \times d_{\text{head}}}$ for sequence length S and head dimension d_{head}, standard attention evaluates:

$$\mathbf{S} = \mathbf{QK}^T \in \mathbb{R}^{S \times S}, \qquad \mathbf{A} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{S \times S}, \qquad \mathbf{O} = \mathbf{AV} \in \mathbb{R}^{S \times d_{\text{head}}}$$

To prevent numerical overflow, row-wise softmax subtracts the row maximum (the log-sum-exp shift of section C.3.5) $m_i = \max_{1 \leq j \leq S} S_{ij}$:

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_{k=1}^{S} \exp(S_{ik} - m_i)}$$

Standard GPU implementations execute this in three discrete kernel launches:

1. Compute $\mathbf{S} = \mathbf{QK}^T$ in SRAM, then write $\mathbf{S} \in \mathbb{R}^{S \times S}$ to HBM ($\Theta(S^2)$ writes).

2. Read $\mathbf{S}$ from HBM, compute row max m and row sum z, compute $\mathbf{A} = \text{softmax}(\mathbf{S})$, write $\mathbf{A} \in \mathbb{R}^{S\times S}$ to HBM ($\Theta(S^2)$ reads + writes).
3. Read $\mathbf{A}$ and $\mathbf{V}$ from HBM, compute $\mathbf{O} = \mathbf{AV}$, write $\mathbf{O} \in \mathbb{R}^{S\times d_{\text{head}}}$ to HBM ($\Theta(S^2 + Sd_{\text{head}})$ reads + writes).

Total HBM memory traffic is $\Theta(S^2 + Sd_{\text{head}})$ elements. For $S = 8192$, $d_{\text{head}} = 128$ in FP16, $\mathbf{S}$ requires $8192^2 \times 2$ B $= 134$ MB per head per layer. Because modern GPUs feature massive compute throughput relative to HBM bandwidth, repeatedly reading and writing intermediate matrices $\mathbf{S}$ and $\mathbf{A}$ leaves GPU execution units memory-bound.

C.4.2.2 Online softmax recurrence

Tiling softmax is difficult because computing $z_i = \sum_{k=1}^{S} \exp(S_{ik} - m_i)$ requires the global maximum m_i across all S keys. The online softmax recurrence solves this by incrementally updating normalization statistics as sub-blocks arrive.

Theorem C.2: Online softmax recurrence theorem

Let a vector $\mathbf{x} \in \mathbb{R}^n$ be partitioned into two sub-vectors $\mathbf{x}^{(1)} \in \mathbb{R}^{n_1}$ and $\mathbf{x}^{(2)} \in \mathbb{R}^{n_2}$ with $n = n_1 + n_2$. Let the local statistics for block 1 be:

$$m^{(1)} = \max_{1\le j\le n_1} x_j^{(1)}, \qquad z^{(1)} = \sum_{j=1}^{n_1} \exp\left(x_j^{(1)} - m^{(1)}\right)$$

and for block 2 be:

$$m^{(2)} = \max_{1\le j\le n_2} x_j^{(2)}, \qquad z^{(2)} = \sum_{j=1}^{n_2} \exp\left(x_j^{(2)} - m^{(2)}\right)$$

Then the combined maximum m^{new}, normalization sum z^{new}, and accumulated output vector $\mathbf{O}^{\text{new}}$ satisfy the exact recurrence relations:

$$\begin{aligned} m^{\text{new}} &= \max\left(m^{(1)}, m^{(2)}\right), \\ z^{\text{new}} &= z^{(1)} e^{m^{(1)} - m^{\text{new}}} + z^{(2)} e^{m^{(2)} - m^{\text{new}}}, \\ \mathbf{O}^{\text{new}} &= \frac{z^{(1)} e^{m^{(1)} - m^{\text{new}}}}{z^{\text{new}}} \mathbf{O}^{(1)} + \frac{e^{m^{(2)} - m^{\text{new}}}}{z^{\text{new}}} \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)}, \end{aligned}$$

where $\mathbf{O}^{(1)} = \frac{1}{z^{(1)}} \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)}$ is the partial output accumulated from block 1.

Denominator recurrence Expanding the combined sum $z^{\text{new}} = \sum_{j=1}^{n} e^{x_j - m^{\text{new}}}$:

$$\begin{aligned} z^{\text{new}} &= \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{\text{new}}} + \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{\text{new}}} \\ &= \sum_{j=1}^{n_1} e^{\left(x_j^{(1)} - m^{(1)}\right) + \left(m^{(1)} - m^{\text{new}}\right)} + \sum_{j=1}^{n_2} e^{\left(x_j^{(2)} - m^{(2)}\right) + \left(m^{(2)} - m^{\text{new}}\right)} \\ &= e^{m^{(1)} - m^{\text{new}}} \underbrace{\sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}}}_{z^{(1)}} + e^{m^{(2)} - m^{\text{new}}} \underbrace{\sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}}}_{z^{(2)}} \\ &= z^{(1)} e^{m^{(1)} - m^{\text{new}}} + z^{(2)} e^{m^{(2)} - m^{\text{new}}}. \quad \blacksquare \end{aligned}$$

Output rescaling recurrence The true normalized output over the concatenated sequence is $\mathbf{O}^{\text{new}} = \frac{1}{z^{\text{new}}} \sum_{j=1}^{n} e^{x_j - m^{\text{new}}} \mathbf{V}_j$:

$$\begin{aligned}\mathbf{O}^{\text{new}} &= \frac{1}{z^{\text{new}}} \left[\sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{\text{new}}} \mathbf{V}_j^{(1)} + \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{\text{new}}} \mathbf{V}_j^{(2)} \right] \\ &= \frac{1}{z^{\text{new}}} \left[e^{m^{(1)} - m^{\text{new}}} \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)} + e^{m^{(2)} - m^{\text{new}}} \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)} \right]\end{aligned}$$

Noting that $z^{(1)}\mathbf{O}^{(1)} = \sum_{j=1}^{n_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)}$, we substitute this directly:

$$\mathbf{O}^{\text{new}} = \frac{z^{(1)} e^{m^{(1)} - m^{\text{new}}}}{z^{\text{new}}} \mathbf{O}^{(1)} + \frac{e^{m^{(2)} - m^{\text{new}}}}{z^{\text{new}}} \sum_{j=1}^{n_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)}. \quad \blacksquare$$

C.4.2.3 Memory traffic of tiled attention

The recurrence lets each query tile be combined with one key-value tile at a time, so score tiles never need to leave SRAM. The remaining question is how many bytes then cross the HBM boundary.

Let M_{SRAM} be the fast SRAM capacity (in words). Following Algorithm 1 of Dao et al. (2022), FlashAttention sets $B_c = \lfloor M_{\text{SRAM}}/(4d_{\text{head}}) \rfloor$ and $B_r = \min(\lfloor M_{\text{SRAM}}/(4d_{\text{head}}) \rfloor, d_{\text{head}})$. The proof abstracts constant factors and buffer reuse, using $B_c = \Theta(M_{\text{SRAM}}/d_{\text{head}})$, $B_r = \Theta(\min(M_{\text{SRAM}}/d_{\text{head}}, d_{\text{head}}))$, and $B_r B_c = \mathcal{O}(M_{\text{SRAM}})$ for the on-chip working set.

The algorithm partitions $\mathbf{Q}$ into $T_r = \lceil S/B_r \rceil$ blocks of size $B_r \times d_{\text{head}}$, and partitions $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil S/B_c \rceil$ blocks of size $B_c \times d_{\text{head}}$. It executes two nested loops:

- **Outer loop** over $j = 1, \ldots, T_c$: Load $\mathbf{K}_j$ and $\mathbf{V}_j$ into SRAM.
- **Inner loop** over $i = 1, \ldots, T_r$: Load $\mathbf{Q}_i$, $\mathbf{O}_i$, and its softmax statistics; compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T$ in SRAM; then update and write $\mathbf{O}_i$ and the statistics without materializing $\mathbf{S}_{ij}$ in HBM.

This loop order is the source of the I/O reduction. A key-value tile remains resident while the algorithm visits every query tile, so each element of $\mathbf{K}$ and $\mathbf{V}$ crosses the HBM boundary once. The query and output tiles must be revisited for each key-value block, but the $B_r \times B_c$ score tile is consumed on chip and discarded. Standard attention instead writes and rereads the full $S \times S$ score and probability matrices.

Total memory access accounting Total HBM accesses comprise two components: key and value reads, where each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded once (contributing $2Sd_{\text{head}}$ accesses); and query, output, and statistics passes, where each of the T_c column blocks reads all of $\mathbf{Q}$, reads and writes $\mathbf{O}$, and updates $\mathcal{O}(S)$ softmax statistics. The matrices dominate the statistics for head dimension $d_{\text{head}} \geq 1$, so each pass contributes $\Theta(Sd_{\text{head}})$ accesses and all passes contribute $\Theta(Sd_{\text{head}}T_c)$.

Because $T_c = \lceil S/B_c \rceil = \Theta(Sd_{\text{head}}/M_{\text{SRAM}})$, the total is

$$\text{HBM}_{\text{Flash}} = \Theta\left(Sd_{\text{head}} + Sd_{\text{head}}T_c\right) = \Theta\left(Sd_{\text{head}} + \frac{S^2 d_{\text{head}}^2}{M_{\text{SRAM}}}\right) = \Theta\left(\frac{S^2 d_{\text{head}}^2}{M_{\text{SRAM}}}\right),$$

where the final equality follows for $d_{\text{head}} \leq M_{\text{SRAM}} \leq Sd_{\text{head}}$.

The bound counts communication rather than arithmetic. FlashAttention still evaluates the same dense score blocks and may recompute intermediate quantities during backpropagation; its advantage comes from keeping those intermediates out of HBM.

I/O reduction ratio Comparing standard attention I/O to FlashAttention I/O:

$$\frac{\text{HBM}_{\text{Standard}}}{\text{HBM}_{\text{Flash}}} = \frac{\Theta(S^2 + Sd_{\text{head}})}{\Theta\left(Sd_{\text{head}} + \frac{S^2 d_{\text{head}}^2}{M_{\text{SRAM}}}\right)} = \Theta\left(\frac{M_{\text{SRAM}}}{d_{\text{head}}^2}\right) \quad \text{for } S \gg \max\left(d_{\text{head}}, \frac{M_{\text{SRAM}}}{d_{\text{head}}}\right)$$

Systems Perspective C.4: A100 I/O reduction

The asymptotic ratio $M_{\text{SRAM}}/d_{\text{head}}^2$ predicts how greater SRAM capacity can reduce HBM traffic, but hidden constants and kernel details prevent it from serving as an exact transfer count. In an NVIDIA A100 measurement, Dao et al. (2022) report the effect for a forward-and-backward attention workload with sequence length 1,024, head dimension 64, 16 heads, batch size 64, a key-padding mask, and no dropout:

- **Standard attention**: 35.3 GB of HBM reads and writes; 35.1 ms runtime.
- **FlashAttention**: 4.4 GB of HBM reads and writes; 11.7 ms runtime.

The measurements show the systems consequence of the I/O bound: substantially less HBM traffic coincides with lower runtime even though FlashAttention performs additional recomputation. They do not imply that every configuration becomes compute bound; tile size, sequence length, and kernel implementation determine the realized regime.

Tiling changes how many bytes cross the memory boundary for a fixed model. The last derivation applies the same accounting one level up and asks how large the model and its training set should be for a fixed compute budget.

C.4.3 Compute-optimal scaling laws

Determining the compute-optimal allocation between model parameters and training dataset tokens for a given floating-point budget O sets how large a language model and its training set should be. This subsection derives the compute-optimal allocation using Lagrange multipliers and contrasts the original findings of Kaplan et al. (2020) with the compute-optimal corrections of Hoffmann et al. (2022). Section 9.2.3 uses the result to test whether a training strategy is limited by data or by compute.

Notation note: This derivation writes model parameters as P, dataset size in tokens as D, and the training compute budget as O total FLOPs; Kaplan et al. (2020) and Hoffmann et al. (2022) write N for parameters and C for compute. The fitted constants E, A, B, α, and β and the Lagrange multiplier λ keep the papers' letters and are local to this section, so E is a loss rather than an energy and B is a fitted constant rather than batch size.

C.4.3.1 Parametric loss surface

The cross-entropy evaluation loss $\mathcal{L}(P, D)$ of an autoregressive transformer language model trained with P non-embedding parameters on D tokens is modeled by the power-law formulation:

$$\mathcal{L}(P, D) = E + \frac{A}{P^{\alpha}} + \frac{B}{D^{\beta}}$$

where:

- E: Irreducible loss representing the entropy of natural language data.
- $\frac{A}{P^{\alpha}}$: Capacity bottleneck loss resulting from finite model parameters P.
- $\frac{B}{D^{\beta}}$: Dataset size bottleneck loss resulting from finite training tokens D.
- $A, B > 0$: Empirical scaling constants.
- $\alpha, \beta > 0$: Power-law scaling exponents for model size and dataset size, respectively.

C.4.3.2 Constrained optimization

The total floating-point operations required to train a transformer with P non-embedding parameters for D tokens is given by

$$O = 6PD \text{ FLOPs},$$

where $2PD$ accounts for the forward pass and $4PD$ accounts for the backward pass.

Given a fixed training compute budget O_0, we formulate the constrained optimization problem to minimize the reducible loss $\hat{\mathcal{L}}(P,D) = \frac{A}{P^\alpha} + \frac{B}{D^\beta}$:

$$\min_{P,D} \left(\frac{A}{P^\alpha} + \frac{B}{D^\beta} \right) \quad \text{subject to} \quad g(P,D) = 6PD - O_0 = 0$$

We construct the Lagrangian function $\mathcal{L}_{\text{Lagrange}}(P,D,\lambda)$:

$$\mathcal{L}_{\text{Lagrange}}(P,D,\lambda) = \frac{A}{P^\alpha} + \frac{B}{D^\beta} + \lambda(6PD - O_0)$$

Taking partial derivatives with respect to P, D, and λ and setting them to zero yields:

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial P} = -\alpha A P^{-\alpha-1} + 6\lambda D = 0 \implies \alpha A P^{-\alpha-1} = 6\lambda D, \tag{1}$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial D} = -\beta B D^{-\beta-1} + 6\lambda P = 0 \implies \beta B D^{-\beta-1} = 6\lambda P, \tag{2}$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial \lambda} = 6PD - O_0 = 0. \tag{3}$$

Multiplying Equation (1) by P and Equation (2) by D gives:

$$\alpha A P^{-\alpha} = 6\lambda PD,$$

$$\beta B D^{-\beta} = 6\lambda PD.$$

Equating these expressions yields the power-law optimality condition:

$$\alpha A P^{-\alpha} = \beta B D^{-\beta}.$$

This condition states that at the compute-optimal allocation, the marginal loss reduction per fractional increase in parameter count must equal the marginal loss reduction per fractional increase in dataset tokens.

C.4.3.3 Compute-optimal exponents

From $\alpha A P^{-\alpha} = \beta B D^{-\beta}$, solving for D in terms of P yields:

$$D^\beta = \left(\frac{\beta B}{\alpha A} \right) P^\alpha \implies D = \left(\frac{\beta B}{\alpha A} \right)^{1/\beta} P^{\alpha/\beta}.$$

Substituting D into the compute constraint $O = 6PD$ gives:

$$O = 6P \left(\frac{\beta B}{\alpha A} \right)^{1/\beta} P^{\alpha/\beta} = 6 \left(\frac{\beta B}{\alpha A} \right)^{1/\beta} P^{\frac{\alpha+\beta}{\beta}}.$$

Solving explicitly for the compute-optimal parameter count $P^*(O)$:

$$P^*(O) = \left(\frac{\alpha A}{\beta B} \right)^{\frac{1}{\alpha+\beta}} \left(\frac{O}{6} \right)^{\frac{\beta}{\alpha+\beta}} \propto O^{\frac{\beta}{\alpha+\beta}}.$$

By symmetry, solving for the compute-optimal token count $D^*(O)$ yields:

$$D^*(O) = \left(\frac{\beta B}{\alpha A} \right)^{\frac{1}{\alpha+\beta}} \left(\frac{O}{6} \right)^{\frac{\alpha}{\alpha+\beta}} \propto O^{\frac{\alpha}{\alpha+\beta}}.$$

Empirical exponents and the $D/P \approx 20$ rule Fitting the parametric loss surface to over 400 experimental runs, Hoffmann et al. (2022) estimated $\alpha \approx 0.34$ and $\beta \approx 0.28$. Substituting these values gives exponents close to the paper's parametric-model result:

$$\frac{\beta}{\alpha+\beta} \approx 0.45, \qquad \frac{\alpha}{\alpha+\beta} \approx 0.55$$

Thus the parametric model predicts:

$$P^*(O) \propto O^{0.45}, \qquad D^*(O) \propto O^{0.55}$$

The paper's two direct empirical approaches report near-balanced exponents of $(0.50, 0.50)$ and $(0.49, 0.51)$, which motivate the common square-root shorthand. Under the parametric fit, however, D^*/P^* grows slowly with compute rather than remaining strictly constant. Across the regime studied, roughly 20 training tokens per parameter is a useful Chinchilla heuristic, not a universal invariant.

Systems Perspective C.5: Kaplan vs. Chinchilla scaling

The difference between Kaplan et al. (2020) and Hoffmann et al. (2022) shows how the training schedule used in empirical fits can alter system design conclusions:

- **Kaplan scaling model**: Kaplan et al. (2020) fit a different loss form with its own exponents and reported compute-optimal growth of $P \propto O^{0.73}$ and $D \propto O^{0.27}$; this split does not follow from substituting exponents into the two-term surface above. This led the field to scale model sizes aggressively while holding training dataset sizes relatively small (e.g., GPT-3 175B trained on 300B tokens, giving $D/P = 1.71$).
- **Methodological divergence**: Kaplan et al. (2020) used a fixed token budget and learning-rate schedule across models. Hoffmann et al. (2022) matched the schedule to each training horizon and found that the earlier setup understated the value of training smaller models on more data.
- **Chinchilla revision**: Hoffmann et al. (2022) found near-balanced parameter and token scaling: its three approaches report exponent pairs of $(0.50, 0.50)$, $(0.49, 0.51)$, and $(0.46, 0.54)$ rather than one exact universal ratio.
- **Practical impact**: Chinchilla (70B parameters trained on 1.4T tokens, $D/P = 20$) outperformed GPT-3 (175B parameters trained on 300B tokens) across downstream benchmarks while requiring $2.5\times$ less compute during inference (inference cost scales with parameter count) and $2.5\times$ lower weight memory at the same precision (Hoffmann et al. 2022).

Summary

An algorithm becomes a systems workload through its shapes, operation graph, data dependencies, and numerical representation. Linear-algebra primitives, memory layouts, broadcasting, and sparse formats determine work, traffic, locality, and usable parallelism. Automatic differentiation and backpropagation add saved activations and backward computation; computational graphs then expose opportunities for fusion and recomputation, and the log-sum-exp shift keeps the softmax at the loss finite in reduced precision. Among the derivations, the Baur–Strassen result bounds reverse-mode overhead, while FlashAttention shows how tiling and an online recurrence can reduce HBM traffic without changing attention semantics.

The same accounting connects kernels to scaling decisions. Compute-optimal scaling turns a fixed budget into a model-data allocation, but the fitted exponents remain empirical rather than universal. Before selecting an optimization, count the operations, the bytes moved, and the state saved for the backward pass.

D

Machine Foundations

A datasheet reports peak throughput; a workload almost never reaches it. Closing that gap requires knowing which physical limit binds execution first: memory bandwidth, arithmetic throughput, fixed overhead, or energy. This appendix collects the hardware models, ratios, and order-of-magnitude values used for single-node estimates in the book, so that chapter estimates can be checked against them. Architecture essentials support readers encountering memory hierarchies and hardware cost models for the first time; familiarity with undergraduate computer architecture is helpful but not required.

How to Use This Appendix

This appendix is designed as a reference. When diagnosing performance issues, use it to translate a vague symptom ("it is slow") into a specific constraint ("bandwidth bound at batch size one") and then choose the lever that addresses it. Each need in table D.1 leads to the section that carries the model or worked template and to the chapter that applies it to the book's workloads.

Table D.1: Where to Look: Each need maps to the section of this appendix that carries the model or worked template and to the chapter that applies it to the book's workloads.

Need	Reference Section	Applied In
Sanity-check feasibility with orders of magnitude	section D.1	section 2.4
Compare access times across memory, links, and storage	section D.1.1	section 11.5.3
Decide whether a kernel is compute or bandwidth bound	section D.2.1	section 11.6
Check that a performance equation is consistent	section D.2.2	section 1.7
Predict whether more accelerators shorten a run	section D.2.3	section 8.7.1
Estimate the wall-clock time of a training run	section D.2.4	section 8.2
Size request concurrency for a serving target	section D.2.5	section 13.4.1
Explain why data movement dominates energy	section D.3.1	section 11.5.3
Separate latency-dominated from bandwidth-dominated transfers	section D.3.2	section 11.5.4
Place a device class by compute, memory, and power	section D.3.3	section 11.11
Choose a numerical format	section D.4.1	section 10.3.1
Map floating-point values to integers	section D.4.2	section 10.3

Conventions used here follow the book-wide notation (for example, B is reserved for batch size and BW for bandwidth). The values in this appendix are ratios and orders of magnitude, which change slowly because they follow from physics and circuit design. The dated specifications of particular accelerators, links, and storage devices change with every product generation; Appendix E tabulates them so a reader can substitute current values without disturbing the relationships here.

D.1 Numbers to Know

Fetching a 32-bit weight from off-chip DRAM costs hundreds of times more energy and roughly a thousand times more latency than reading it from a register, a gap that no compiler flag can erase. Like Jeff Dean's "Latency Numbers Every Programmer Should Know,"[1] the reference numbers below give the order-of-magnitude intuition that ML systems design requires. Although absolute values and ratios vary by technology and workload, the hierarchy is more durable; memorize the relationships and use the specific values as sanity checks.

[1] **Jeff Dean**: A Google Senior Fellow and one of the architects of MapReduce, BigTable, and TensorFlow. His latency numbers, originally presented with Peter Norvig around 2010, became a canonical reference for systems engineers; Colin Scott's interactive visualization shows that the *hierarchy* of latencies has stayed stable across hardware generations (Scott 2012).

> **Systems Perspective D.1: Three numbers that matter most**
>
> - **Energy ratio**: In the cited 45 nm reference, a 32-bit DRAM read uses ~581× the energy of one FP16 multiply.
> - **Training-state footprint**: Model weights (2 bytes FP16) + gradients (2 bytes FP16) + master weights (4 bytes FP32) + optimizer states for Adaptive Moment Estimation (Adam) at 8 bytes. That totals 16 bytes per parameter, so Llama 3 8B (8.03B parameters) needs 128.5 GB just to start full training, more than one accelerator's memory.
> - **Fiber propagation limit**: Light travels about 200 km/ms in fiber, so propagation sets a floor on every round trip.

Reference relationships

The numbers above mix two kinds of quantities: invariant physical or arithmetic bounds, such as propagation delay in fiber and bytes per parameter, and technology-specific energy ratios that shift with each process node while preserving their ordering. The three tables below separate those reference anchors into propagation latency, memory-to-compute energy cost, and model-sizing arithmetic.

Speed of light

No network upgrade or routing optimization can deliver a cross-country round trip in 5 ms when light in fiber takes roughly 40 ms to make the journey. Each approximate round trip in table D.2 carries that irreducible propagation component.

Table D.2: Speed of Light Reference: Light in fiber travels ~200 km/ms. These approximate round trips include reducible overhead; only propagation is irreducible.

Distance	Round-Trip Latency	Implication
Same data center	~1 ms	Distributed training feasible
Cross-country (US)	~40 ms	Reduces the remaining request budget
Cross-Atlantic	~60 ms	Further reduces the request budget
Cross-Pacific	~100 ms	Data locality is critical

Energy hierarchy

Moving data across wires costs far more energy than computing on it once it arrives in an arithmetic unit. Raising reuse per byte therefore pays off across every hardware generation (table D.3).[2]

[2] **Energy hierarchy sources**: The DRAM and arithmetic values come from Horowitz (2014) (45 nm). The L1 SRAM/register ratio is an illustrative hierarchy anchor, not a Horowitz measurement. The headline ratio compares a 32-bit DRAM access with a 16-bit floating-point multiply. Both the absolute values and the ratio vary with process, circuit, memory technology, and operand width; wire distance helps explain why movement remains expensive.

Table D.3: Energy Reference Ratios: The first three ratios use Horowitz's 45 nm measurements; the L1/register ratio is an illustrative on-chip hierarchy anchor.

Relationship	Reference Ratio	Source and Interpretation
DRAM access vs. FP16 multiply	~581×	Horowitz 45 nm; wire capacitance scales with distance
FP32 vs. INT8 energy	~18×	Horowitz 45 nm; bit width determines switching energy
FP32 vs. FP16 energy	~3.4×	Horowitz 45 nm; narrower arithmetic reduces switching and datapath energy

Relationship	Reference Ratio	Source and Interpretation
L1 SRAM vs. register	~5×	Illustrative on-chip hierarchy anchor; distance to the arithmetic unit

Scaling rules

Memory and compute requirements for training and inference follow from a few arithmetic relationships (table D.4).[3] The examples use Llama 3 8B. Section 8.3.2.3 derives the 16 bytes per parameter of training state and why switching to 16-bit arithmetic does not shrink it.

[3] **Training memory (Adam)**: The 16 bytes/parameter rule assumes mixed-precision training with Adam. ZeRO optimization can reduce per-accelerator memory by sharding optimizer states across accelerators, but the total memory across all accelerators remains ~16× parameters.

Table D.4: Scaling Rules: These are arithmetic, not hardware-specific. Training memory includes FP16 weights (2 bytes), FP16 gradients (2 bytes), FP32 master weights (4 bytes), and Adam optimizer states (8 bytes for momentum + variance).

Rule	Formula	Example (Llama 3 8B)
Inference memory (FP16)	2 bytes× parameters	16.1 GB
Inference memory (INT8)	1 byte× parameters	8.0 GB
Training memory (Adam)	16 bytes× parameters	128.5 GB
Inference FLOPs (transformer)	~2× parameters per token	16.1 GFLOP per token
Training FLOPs	~6× parameters× tokens	4.8×10^{22} FLOPs on 1T tokens

D.1.1 The latency hierarchy

Distance costs time. Each step away from the arithmetic units, from registers through on-chip cache, off-chip memory, device and host links, the network, and storage, multiplies the wait for the first byte, and the largest jumps come from crossing a physical boundary (table D.5). If a register access took ten seconds, a read from accelerator memory would take hours and a read from local flash storage would take weeks.

Table D.5: The Latency Hierarchy: Access latency by tier, as the decade that contains it, as a multiple of a register access (rounded to one significant figure), and on a human time scale where one register access takes ten seconds. The largest gaps come from crossing physical boundaries: off-chip memory, device and host links, network fabric, and storage.

Tier	Latency (Decade)	Relative to a Register	If a Register Access Took 10 s
Register	0.1–1 ns	1×	10 seconds
L1 cache	1–10 ns	3×	33.3 seconds
L2 cache	1–10 ns	10×	2.2 minutes
Accelerator memory (HBM)	100–1,000 ns	1,000×	2.8 hours
Accelerator-to-accelerator link	100–1,000 ns	2,000×	4.6 hours
Host link (PCIe)	1–10 µs	3,000×	9.3 hours
Network fabric	1–10 µs	20,000×	1.9 days
Local flash storage (NVMe)	100–1,000 µs	300,000×	38.6 days

Bandwidth falls along the same path. On the reference accelerator, high-bandwidth memory delivers about 52× the bandwidth of the host link, and a network access costs about 16× the latency of an accelerator-memory access. Data that must cross the host link or the network should cross it once and be reused many times on the device.

Illustrative latency budgets

Latency budgets depend on application-specific safety requirements, user expectations, and service-level objectives. Table D.6 lists illustrative targets rather than universal limits; section 13.3.1 shows how a serving system divides such a budget among its stages.

Table D.6: Illustrative Latency Targets: Production requirements depend on the application and its service-level objectives.

Application	Budget	Constraint
Autonomous braking	<10 ms	At 100 km/h, 10 ms = 28 cm of travel
Voice assistant	<100 ms	Human perception of "instant"
Web search	<200 ms	User patience threshold
Video streaming	<1 s	Buffer tolerance
Batch training	hours–days	Throughput dominates latency

> **Systems Perspective D.2: A note on terminology: GPUs and accelerators**
>
> Throughout this book, "accelerator" is the general term for hardware acceleration. The principles of roofline analysis, memory hierarchies, numerical precision, and performance modeling apply to GPUs, Tensor Processing Units (TPUs), NPUs, application-specific integrated circuits, and other specialized AI accelerators. Vendor-specific features such as CUDA and NVLink are identified explicitly.

Appendix E tabulates the dated accelerator specifications, including the NVIDIA H100 that serves as the book's reference machine. Next, the roofline model shows which of those specifications limits a given workload by turning compute throughput and memory bandwidth into performance ceilings.

D.2 Physics of Computing

Raw hardware specifications (FLOP/s, bytes per second, watts) are necessary but insufficient for performance reasoning. Without compact analytical models, an engineer cannot distinguish a compute-bound workload from a bandwidth-bound one, or predict whether doubling the accelerator count will halve training time. The models in this section turn those distinctions into quantitative checks.

D.2.1 The Roofline model

A kernel that achieves 1 percent of an accelerator's peak FLOP/s may be poorly written, or it may already be saturating the memory bus at the physical limit of the chip. The Roofline Model (Williams et al. 2009) separates those two cases by bounding how fast a workload can run on a given hardware target according to whether it runs out of compute or memory bandwidth first.

Every operation has an **arithmetic intensity**: the ratio of computations performed to bytes moved from memory. Matrix multiplication achieves high arithmetic intensity because each loaded element is reused across multiple multiply-accumulate steps. In contrast, element-wise operations like rectified linear unit (ReLU) have low intensity because each element is fetched from memory, evaluated once, and written back. Plotting achievable performance against arithmetic intensity yields the roofline (figure D.1): a diagonal memory-bandwidth ceiling rises linearly from the origin until it meets the horizontal peak-compute ceiling at the ridge point. A workload's arithmetic intensity determines which ceiling governs execution, and therefore whether optimizations must target memory traffic or arithmetic throughput.

The **ridge point** determines the hardware's balance. Formally, when computation and data movement overlap completely (the overlapped form of the iron law), idealized wall-clock time is bounded below by the critical path:

$$T \geq \max\left(\frac{O}{R_{\text{peak}}}, \frac{D_{\text{vol}}}{\text{BW}}\right)$$

Attainable throughput $R_{\text{attain}} = O/T$ is therefore bounded by:

$$R_{\text{attain}} \leq \frac{O}{\max\left(\frac{O}{R_{\text{peak}}}, \frac{D_{\text{vol}}}{\text{BW}}\right)} = \min\left(R_{\text{peak}}, \frac{O}{D_{\text{vol}}} \times \text{BW}\right) = \min(R_{\text{peak}}, I \times \text{BW})$$

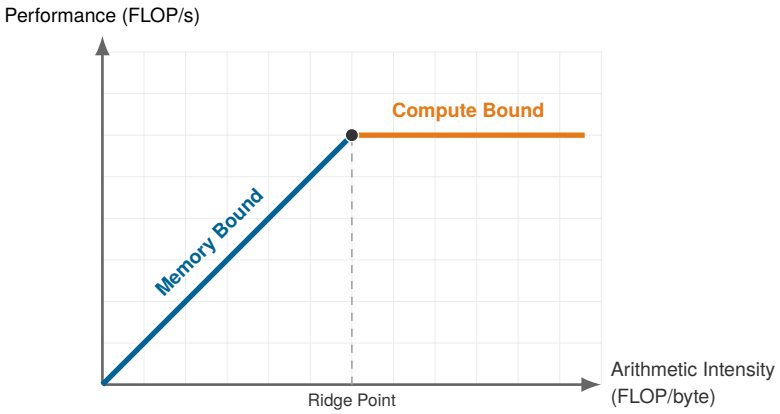


Figure D.1: The Roofline Model: The sloped memory-bandwidth ceiling and horizontal peak-compute ceiling meet at the ridge point. Plotting a workload by arithmetic intensity identifies which roofline ceiling applies.

where $I = O/D_{\text{vol}}$ is the arithmetic intensity measured at the DRAM interface. The ridge point I_{ridge} is the transition intensity where the compute and bandwidth ceilings meet:

$$R_{\text{peak}} = I_{\text{ridge}} \times \text{BW} \implies I_{\text{ridge}} = \frac{R_{\text{peak}}}{\text{BW}}$$

If a workload's arithmetic intensity falls below I_{ridge}, it is bandwidth-bound (the roofline literature says memory-bound); if above, it is compute bound. The memory-bound regime is the bandwidth-bound archetype of section 2.3.2, whose lighthouse is GPT-2 decode; the compute-bound regime is the compute-bound archetype, whose lighthouse is ResNet-50. Section 11.6 develops both regimes and the levers that move a workload between them.

> **Systems Perspective D.3: Batch size controls arithmetic intensity**
>
> For matrix multiplications, arithmetic intensity scales with the batch dimension. For $\mathbf{Y} = \mathbf{XW}$, where $\mathbf{X}$ is $(B \times d_{\text{in}})$ and $\mathbf{W}$ is $(d_{\text{in}} \times d_{\text{out}})$:
>
> - **FLOPs**: $2 \times B \times d_{\text{in}} \times d_{\text{out}}$ (multiply-adds).
> - **Bytes**: Moving $\mathbf{X}$, $\mathbf{W}$, and $\mathbf{Y}$ costs approximately $s_{\text{elem}}(Bd_{\text{in}} + d_{\text{in}}d_{\text{out}} + Bd_{\text{out}})$ bytes.
>
> Doubling B doubles FLOPs while weight traffic stays nearly constant when weights dominate, increasing arithmetic intensity. This is why batching often helps inference serving (section 13.5.1 develops the serving side, including the latency it adds), although activation traffic, cache behavior, shape, precision, and the ridge point determine whether it reaches the compute ceiling.

D.2.1.1 A concrete example: The H100 analysis

Consider an NVIDIA H100, the book's reference machine, with dense FP16 Tensor Core performance of 989 TFLOP/s and HBM3 bandwidth of 3.35 TB/s. The ridge point is 989 TFLOP/s/3.35 TB/s = 295.2 FLOP/byte (the Tera prefixes cancel, yielding FLOP/byte).

Two common operations fall on opposite sides of that ridge. General matrix multiply (GEMM) on two square matrices of size 4096 by 4096 has arithmetic intensity of approximately 1,365 FLOP/byte, so 1,365 FLOP/byte > 295.2 FLOP/byte and the operation is compute bound. An element-wise ReLU on an FP16 tensor performs 1 comparison while reading 2 bytes and writing 2 bytes (4 bytes total), yielding an arithmetic intensity of only $1/4 = 0.25$ FLOP/byte. Because 0.25 FLOP/byte $\ll$ 295.2 FLOP/byte, the operation is severely bandwidth-bound, achieving only about 0.08 percent of peak FLOP/s. This contrast is why frameworks fuse an element-wise operation into the preceding matrix multiply, so the intermediate result never makes a round trip through memory; section 7.3.2 develops fusion and the launch overhead it removes.

Precision moves the ridge. At INT8 the H100's peak rate doubles to 1,979 TOPS while its memory bandwidth stays fixed, so the ridge rises to about 591 operations per byte. A convolution layer that sits above the FP16 ridge can fall below the INT8 ridge, which is why quantized ResNet-50 inference can turn bandwidth bound on the same chip (section 11.6.2).

D.2.2 Dimensional analysis

The roofline model helps diagnose *where* a bottleneck lies. Before applying any performance equation, however, it must be verified as physically meaningful. Dimensional analysis provides this sanity check: any valid equation must be **dimensionally homogeneous**, meaning every term must resolve to the same units. If they do not, the equation contains an error.

Consider both levels of the iron law of ML systems (principle 3) introduced in section 1.7. First, in the multiplicative D·A·M form (equation 1.2), intermediate units cancel across the three factors to leave seconds per task:

$$T[\text{s}] = \frac{\text{Samples}}{\text{Task}} \times \frac{\text{FLOPs}}{\text{Sample}} \times \frac{\text{s}}{\text{FLOP}} = \frac{\text{s}}{\text{Task}}$$

Second, for serialized execution phases (equation 1.3), every additive term must resolve to time (seconds):

$$T[\text{s}] = \underbrace{\frac{D_{\text{vol}}[\text{bytes}]}{\text{BW}[\text{bytes/s}]}}_{\text{seconds}} + \underbrace{\frac{O[\text{FLOPs}]}{R_{\text{peak}}[\text{FLOP/s}] \cdot \eta_{\text{hw}}[1]}}_{\text{seconds}} + \underbrace{L_{\text{lat}}[\text{s}]}_{\text{seconds}}$$

- **Data term**: $\frac{\text{bytes}}{\text{bytes/s}} = \text{bytes} \times \frac{\text{s}}{\text{bytes}} = \text{s}$
- **Compute term**: $\frac{\text{FLOPs}}{\text{FLOP/s}} = \text{FLOPs} \times \frac{\text{s}}{\text{FLOP}} = \text{s}$
- **Latency term**: Already in seconds.

The equation is physically consistent. Apply this technique to any systems equation: if the dimensions do not match, the formula is wrong. "FLOPs" and "Bandwidth" cannot be traded directly because they have different units. Any such trade-off must convert through time, as the iron law does.

Once an equation is dimensionally sound, the next question is how its time terms scale across devices. Section D.2.3 answers that question with two complementary bounds.

D.2.3 Amdahl's law and Gustafson's law

Parallelization is the primary tool for scaling ML, but its limits depend on *how* the workload scales. The two laws differ in what they hold fixed. **Amdahl's law** governs how much faster a *fixed* task can run (optimizing latency). **Gustafson's law** governs how much *more* work can be done in the same time (optimizing throughput).

D.2.3.1 Strong scaling (Amdahl's Law)

Adding a thousand accelerators to a fixed training batch cannot shorten the step by more than 20× if 5 percent of each step is serial host dispatch or synchronization. **Strong scaling** measures how much faster a fixed-size problem runs as processors are added.

Amdahl's law (Amdahl 1967) states that speedup is limited by the serial fraction of the task.[4] Let $T(1) = 1$ be normalized single-processor execution time, composed of serial fraction $1 - f_{\text{parallel}}$ and parallelizable fraction f_{parallel}. If N_{workers} processors execute the parallel fraction with linear scaling and zero communication overhead, execution time on N_{workers} processors is:

$$T(N_{\text{workers}}) = (1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{N_{\text{workers}}}$$

The resulting theoretical speedup is:

$$\text{Speedup}(N_{\text{workers}}) = \frac{T(1)}{T(N_{\text{workers}})} = \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{N_{\text{workers}}}}$$

[4] **Gene Amdahl (1922–2015)**: A legendary computer architect at IBM, where he was the chief architect of the System/360. He later founded Amdahl Corporation to compete with IBM in the mainframe market.

As $N_{\text{workers}} \to \infty$, the term $\frac{f_{\text{parallel}}}{N_{\text{workers}}} \to 0$, and the speedup converges to $1/(1-f_{\text{parallel}})$. To see Amdahl's Law in action, suppose 5 percent of a training step is serial overhead (for example, Python global interpreter lock (GIL), kernel launch latency) and 95 percent is parallelizable matrix math:

- With $N_{\text{workers}} = 1$, speedup is 1.
- With $N_{\text{workers}} = 8$, speedup is $1/(0.05 + 0.95/8) \approx 5.9\times$.
- With $N_{\text{workers}} \to \infty$, speedup is capped at $1/0.05 = 20\times$.

No matter how many accelerators are added, this fixed workload cannot run faster than 20×. The same ceiling limits a single-node optimization: section 12.3.4 applies it to a measured pipeline, where an unchanged preprocessing stage caps the gain from a faster model.

D.2.3.2 Weak scaling (Gustafson's Law)

Instead of using 1,000 accelerators to finish a small training dataset in milliseconds, large-model training uses them to process a dataset 1,000× larger in the same wall-clock time. **Weak scaling** measures how much larger a problem can become while holding runtime fixed as processors are added.[5]

For N_{workers} processors, let $1-f_{\text{parallel}}$ be the serial fraction of execution time measured on those processors for the scaled workload, and f_{parallel} the parallel fraction. Gustafson's law (Gustafson 1988) derives "scaled speedup" by asking how long that scaled problem would take on a single processor. On N_{workers} processors, normalized runtime is $T(N_{\text{workers}}) = (1-f_{\text{parallel}}) + f_{\text{parallel}} = 1$. Executing the same total work on a single processor requires running the parallel fraction sequentially, taking $T(1) = (1-f_{\text{parallel}}) + N_{\text{workers}} \cdot f_{\text{parallel}} = N_{\text{workers}} - (1-f_{\text{parallel}})(N_{\text{workers}}-1)$. The resulting scaled speedup is:

$$\text{Scaled Speedup}(N_{\text{workers}}) = \frac{T(1)}{T(N_{\text{workers}})} = N_{\text{workers}} - (1-f_{\text{parallel}})(N_{\text{workers}}-1)$$

Because the parallel part grows linearly with N_{workers} while the serial fraction $1-f_{\text{parallel}}$ remains fixed, weak scaling can keep efficiency high as the problem grows. Using the same 5 percent serial overhead ($1-f_{\text{parallel}} = 0.05$), Gustafson's Law tells a very different story:

- With $N_{\text{workers}} = 1$, speedup is 1.
- With $N_{\text{workers}} = 8$, Scaled Speedup is $8 - 0.05 \times (7) = 8 - 0.35 = 7.65\times$.
- With $N_{\text{workers}} = 1000$, Scaled Speedup is $1000 - 0.05 \times (999) \approx 950\times$.

In weak scaling, efficiency remains high because the useful work (training the model) scales up to dwarf the fixed overheads. Section 8.7.1 shows how data-parallel training uses weak scaling to keep accelerators busy as the cluster grows.

[5] **John Gustafson**: A computer scientist known for his work in parallel computing and for introducing the Unum (universal number) format. His law was a direct response to the perceived "limits" of Amdahl's Law when applied to massive scale.

D.2.4 The training-time equation

Weak scaling explains why large training runs grow the dataset rather than shrink the wall-clock time. The training-time equation turns model size, data size, accelerator count, peak rate, and utilization into a single estimate of that wall-clock time. Section 8.2 applies it to the book's training workloads; the template below is its reference form.

Napkin Math D.1: The training time equation

Problem: How long does dense-transformer training take for a stated model size, token count, accelerator count, peak rate, and utilization?

Math:

$$T \approx \frac{6 \cdot P \cdot D}{N_{\text{GPU}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}}}$$

Variables:

- **Training FLOP factor**: The factor 6 derives from $2P$ FLOP/token for the forward pass and $4P$ FLOP/token for the backward pass ($2P$ for activation gradients and $2P$ for weight gradients), yielding $6PD$ FLOPs total across D training tokens.
- P: Number of model parameters.
- D: Number of training tokens.
- $N_{\textbf{GPU}}$: Number of accelerators.
- $R_{\textbf{peak}}$: Peak FLOP/s of one accelerator.
- $\eta_{\textbf{hw}}$: Hardware utilization, typically 30 percent–50 percent in this training estimate.

Example: Training a 1B parameter model on 20B tokens using 1 H100 (989 TFLOP/s) at 40 percent utilization. Total FLOPs $= 6 \times 1 \times 10^9 \times 2 \times 10^{10} = 1.2 \times 10^{20}$ FLOPs. Throughput $= 1 \times (9.89 \times 10^{14}) \times 0.40 \approx 3.956 \times 10^{14}$ FLOP/s. $T = \frac{1.2 \times 10^{20}}{3.956 \times 10^{14}} \approx 303,336.7$ seconds $\approx 5,055.6$ minutes.

Systems insight: Under these assumptions, training takes about 3.5 days. In D·A·M terms, this equation is the macroscopic product of Data (D tokens), Algorithm ($6P$ FLOPs/token), and Machine ($(N_{\text{GPU}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}})^{-1}$ s/FLOP): doubling the token count doubles the time, while doubling the accelerator count or the utilization halves it.

D.2.5 Little's Law

An inference server receiving 1,000 queries per second at a 50 ms average response time holds 50 requests in flight on average, and if each request occupies 1 GB of KV-cache state, a 24 GB accelerator runs out of memory before compute saturates. For capacity planning in stable inference systems, **Little's law** (Little 1961) relates long-run mean concurrency (Q_{req}), mean arrival rate (λ_{arr}), and mean time in system (T_{lat}):[6]

$$Q_{\text{req}} = \lambda_{\text{arr}} \times T_{\text{lat}}$$

[6] **John Little**: An Institute Professor at MIT and a pioneer in the field of operations research. His law, proved in 1961, is fundamental to queuing theory and is used across fields from manufacturing to computer network analysis.

The law follows from operational conservation of requests over any observation interval $[0, T_{\text{obs}}]$. Let $A(t)$ denote cumulative arrivals and $C(t)$ cumulative completions, so the instantaneous in-flight queue occupancy is $Q(t) = A(t) - C(t)$. The average number of requests in the system over $[0, T_{\text{obs}}]$ is $\bar{Q} = \frac{1}{T_{\text{obs}}} \int_0^{T_{\text{obs}}} Q(t)\, dt$. The total accumulated request-time $\int_0^{T_{\text{obs}}} Q(t)\, dt$ equals the sum of residence times $\sum_{i=1}^{N(T_{\text{obs}})} W_i$ of all $N(T_{\text{obs}})$ requests completed by time T_{obs} (plus boundary terms that vanish as $T_{\text{obs}} \to \infty$). Dividing both sides by T_{obs}:

$$\bar{Q} = \frac{1}{T_{\text{obs}}} \int_0^{T_{\text{obs}}} Q(t)\, dt = \frac{N(T_{\text{obs}})}{T_{\text{obs}}} \cdot \frac{1}{N(T_{\text{obs}})} \sum_{i=1}^{N(T_{\text{obs}})} W_i$$

In the limit as $T_{\text{obs}} \to \infty$ for a stable queue, $\lim_{T_{\text{obs}} \to \infty} \frac{N(T_{\text{obs}})}{T_{\text{obs}}} = \lambda_{\text{arr}}$ and $\lim_{N \to \infty} \frac{1}{N} \sum_{i=1}^{N} W_i = T_{\text{lat}}$, yielding $Q_{\text{req}} = \lambda_{\text{arr}} \times T_{\text{lat}}$ regardless of arrival distribution or service-time distribution. Because the identity requires no Poisson-arrival or exponential-service assumption, it applies directly to heavy-tailed large language model (LLM) generation workloads so long as the queue is stationary.

To see this in practice, consider sustaining 1,000 QPS with 50 ms average latency. The law dictates that the system must support 1000×0.05 s = 50 concurrent requests.

This sizes worker pools, but memory needs another assumption. If every in-system request simultaneously holds 1 GB of device state, the average population implies 50 GB; queued requests may instead stay in host memory or share batched state. If 24 device slots ($Q_{\text{req,max}}$, the most requests the device can hold at once) each remain occupied for 50 ms, their throughput proxy is $Q_{\text{req,max}}/T_{\text{lat}} = 24/0.05 = 480$ QPS; Little's law alone does not fix a maximum; device capacity does.

Serving systems use this relation to size request concurrency, and section 13.4.1 pairs it with a queueing model to show how latency degrades as utilization nears saturation. Together, these physics-based models (roofline, Amdahl, Gustafson, and Little) locate *where* a bottleneck lies; acting on that diagnosis requires the hardware structures that impose the bounds: the memory hierarchy, the links that move data between tiers, and the classes of hardware that package them.

D.3 Computer Architecture Essentials

A kernel can run at a small fraction of an accelerator's advertised peak when data movement, insufficient parallelism, synchronization, launch overhead, or software inefficiency stands in the way. Whereas physics sets the theoretical ceilings, computer architecture defines the machinery that determines how close a real workload can get to them across three structures: the memory hierarchy and its energy cost, the paths that move data between tiers, and the class of hardware that packages both.

D.3.1 The memory hierarchy

Computer systems rely on tiered storage architectures because no single physical medium simultaneously provides massive storage capacity and the lowest access latency. Faster, smaller, and costlier tiers sit close to the arithmetic units; larger and cheaper tiers sit farther away (figure D.2). When data movement sits on the critical execution path, staging tensors into the upper tiers avoids the orders-of-magnitude latency penalties of off-chip DRAM and persistent storage that table D.5 quantifies.

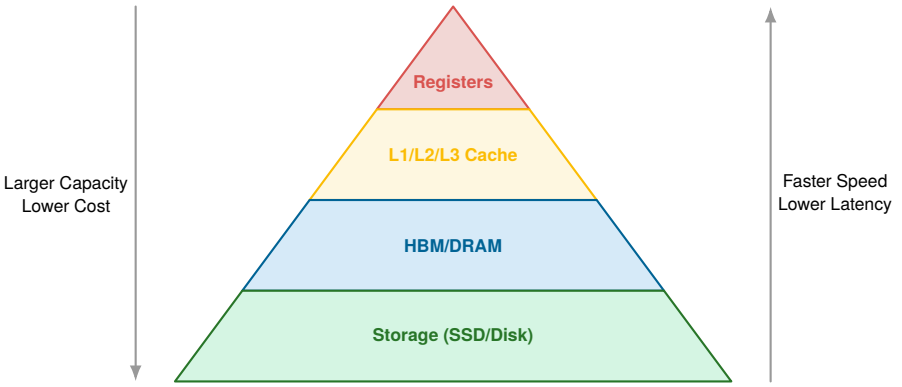


Figure D.2: The Memory Hierarchy: Performance depends on data proximity. Accessing HBM is roughly 1,000× slower than registers; accessing SSD is roughly 300,000× slower.

The memory hierarchy limits ML systems in both time and energy. Each access farther from the arithmetic units costs more energy, because the signal must drive longer, higher-capacitance wires (table D.7). Section 11.5.3 shows how accelerator design stages data in the upper tiers to avoid the off-chip cost.

Table D.7: Energy Across the Memory Hierarchy: Energy per 32-bit access in the cited 45 nm reference technology. The DRAM value is the Horowitz measurement; the register and SRAM values are illustrative hierarchy anchors. Absolute values fall with each process node, but the ordering and the gap between on-chip and off-chip access persist.

Tier	Technology	Energy per 32-bit Access	Relative to a Register
Register	Flip-flops	0.1 pJ	1×
L1 cache	SRAM	0.5 pJ	5×
L2 cache	SRAM	2 pJ	20×
Off-chip memory	DRAM/HBM	640 pJ	6,400×

Energy explains why data should stay high in the hierarchy. The table gives per-access costs, so the energy of a whole workload also depends on how many accesses reach each tier and how often data is reused. How long a transfer between tiers takes depends on two distinct properties of the path it crosses.

D.3.2 Bandwidth versus latency

Upgrading a 10 Gbps network link to 100 Gbps barely changes the delivery time of a 1 KB inference request, yet it cuts the transfer of a 1 GB checkpoint tenfold. Bandwidth (throughput) and latency

(delay) govern those two regimes as distinct constraints. For data volume D_{vol} sent over effective bandwidth BW with fixed path latency L_{lat}, and assuming the latency and serialization costs do not overlap, total transfer time is:

$$T = L_{\text{lat}} + \frac{D_{\text{vol}}}{\text{BW}}$$

The crossover point is the transfer size where the two terms are equal:

$$n^* = L_{\text{lat}} \times \text{BW}$$

For transfers below n^* (for example, small request or control messages), latency dominates. For transfers above it (for example, loading weights), bandwidth dominates.

Consider sending data over a 10 Gb/s link with 10 ms base request/response latency. The crossover size is 12.5 MB, so the dominant term depends on which side of that threshold the transfer falls.

- **Latency-dominated packet (1 KB)**: Transmission time is 1 KB $\times$ 8/10 Gb/s $\approx$ 0.8 µs. Total time is approximately 10 ms + 0.8 µs $\approx$ 10 ms. **Result**: Fixed path and protocol latency dominates; propagation is one component.
- **Bandwidth-dominated checkpoint (1 GB)**: Transmission time is 1 GB $\times$ 8/10 Gbps $\approx$ 800 ms. Total time is $\approx$ 10 ms + 800 ms = 810 ms. **Result**: Base latency is negligible; bandwidth is the bottleneck.

The classes of hardware that ML systems deploy on differ sharply in these properties. A data center accelerator and a microcontroller differ by orders of magnitude in memory bandwidth, compute, and power, and those differences decide which workloads each class can run.

D.3.3 Accelerator classes

The deployment paradigms of section 2.1 run on four classes of hardware that differ by orders of magnitude in peak rate, memory bandwidth, memory capacity, and power (table D.8). Each range spans representative devices in its class rather than any single product; section E.2.1 tabulates the products themselves.

Table D.8: Accelerator Classes: Order-of-magnitude ranges across representative devices in each class, rounded to one significant figure. Peak rate is the dense rate at each device's native ML precision (FP16 or BF16 for data center accelerators, mostly INT8 for edge, mobile, and microcontroller parts), so compare rates within a class rather than across classes. The ridge point is peak rate divided by memory bandwidth.

Class	Peak Rate	Memory Bandwidth	Memory Capacity	Power	Ridge Point
Datacenter accelerator	100–2,000 TFLOP/s	0.9–8 TB/s	30–200 GB	300–1,000 W	100–300 FLOP/byte
Edge module	4–300 TFLOP/s	8–200 GB/s	1–70 GB	2–60 W	100–1,000 FLOP/byte
Mobile SoC	20–40 TFLOP/s	50–80 GB/s	9–10 GB	5 W	300–700 FLOP/byte
Microcontroller	60–500 MFLOP/s	60–700 MB/s	0.2–4 MB	5–400 mW	0.2–6 FLOP/byte

Two patterns stand out across these classes: first, they span roughly 7 orders of magnitude in peak rate and 5 in power, so a model sized for one class rarely runs unchanged on another; second, whereas ridge points sit in the hundreds of FLOP per byte from data center to mobile, leaving the same low-intensity operations bandwidth bound on all three, a microcontroller's ridge sits within a few FLOP per byte, making nearly every layer of a keyword-spotting model compute bound and shifting the binding constraint to the power budget. Section 11.11 applies these classes to the choice of accelerator, while moving between classes also changes which numerical formats the hardware supports natively, and numerical precision directly controls *how much* data must move.

D.4 Numerical Representations

The numerical format decides how many bytes each value occupies and how much energy each operation costs. In ML systems, the choice of precision (FP32 vs. BF16 vs. INT8) is a direct trade-off between statistical fidelity and hardware throughput.

D.4.1 Floating-point formats

IEEE 754 standards and AI-specific numerical representations trade dynamic range (the span of representable values) against precision (the granularity of values within that range). Operational ranges and typical uses differ by format (table D.9), and the bit partitions of FP32, BF16, FP16, and INT8 show where each format spends its bits (figure D.3). BF16 retains FP32's full 8-bit exponent field, preserving the same dynamic range while truncating the mantissa to 7 bits, whereas FP16 widens the mantissa to 10 bits at the cost of a narrower 5-bit exponent.

Table D.9: Numerical Format Comparison: Each format trades off precision, dynamic range, memory footprint, and compute throughput.

Format	Bits	Exponent	Mantissa	Dynamic Range	Typical Use Case
FP32	32	8	23	$\sim 10^{-38}$ to 10^{38}	Training (full precision), reference inference
FP16	16	5	10	Normal: 6.10×10^{-5}–6.55×10^{4}	Training often with loss scaling, inference
BF16	16	8	7	FP32 normal exponent range	Training, usually without loss scaling
FP8	8	4 or 5	3 or 2	Varies	Training/inference on supported hardware
INT8	8	N/A	N/A	-128 to 127	Inference after quantization

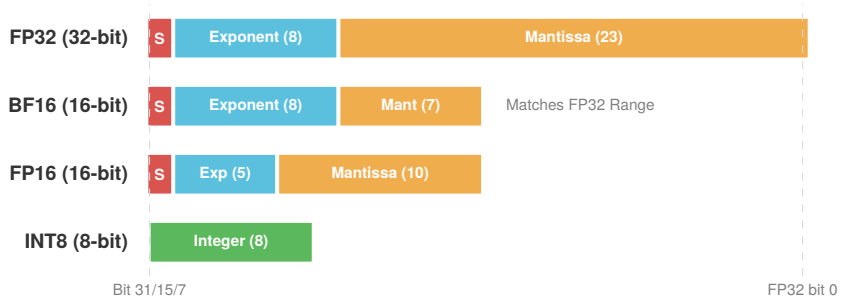


Figure D.3: Numerical Format Bit Layouts: A visual comparison of how FP32, BF16, FP16, and INT8 allocate their bits.

BF16[7] (Wang and Kanwar 2019; Kalamkar et al. 2019) occupies the same 16-bit storage class as FP16, yet the two formats behave very differently during training. Because BF16 allocates three more bits to the exponent than FP16, nominal bit width alone does not predict whether a training run will need gradient scaling.

7 **BF16 (brain floating point 16)**: Originally introduced with Google TPUv2 and later adopted by Intel, Arm, and NVIDIA, BF16 became a common training format across accelerator ecosystems. Its adoption made format support in hardware, kernels, and distributed-training libraries a portability concern rather than a vendor-specific feature.

> **Systems Perspective D.4: The dynamic range wall**
>
> The choice of numerical format is a direct application of the iron law of ML systems (principle 3). Reducing precision from FP32 to BF16 or FP16 halves the data volume term, potentially doubling throughput on memory-bound workloads. However, the *type* of 16-bit format determines the engineering complexity:
>
> - **Dynamic range (the exponent)**: BF16 preserves the eight-bit exponent of FP32 and therefore the same normal exponent range (Kalamkar et al. 2019).
> - **Precision (the mantissa)**: FP16 has a larger 10-bit mantissa than BF16 (7 bits), offering higher precision for values within its range. Its five-bit exponent, however, raises un-

derflow risk. FP16 training therefore often uses **loss scaling**, an operational step that multiplies gradients by a large constant to keep more values representable (Micikevicius et al. 2018); section 8.6.3 develops the full mixed-precision recipe.

- **Energy efficiency**: INT8 operations can be substantially more efficient than floating-point equivalents because they move less data and use simpler integer arithmetic paths. Moving to INT8 for inference is a primary lever for deploying neural networks under tight memory, latency, or power budgets (Jacob et al. 2018; Krishnamoorthi 2018); section 10.3.1 compares the formats on accuracy cost and hardware support.

D.4.2 Integer quantization

Storing weights and activations in 32-bit floating point spends four bytes per value and drives wide floating-point multipliers when 256 discrete levels often suffice for inference accuracy. Quantization cuts that cost by mapping continuous floating-point values to discrete integers, typically INT8, using either a symmetric or an asymmetric mapping to assign the floating-point range to 256 integer codes.

For symmetric quantization, let $\alpha_{\text{sym}} = \max|x|$ over the calibrated range, assume x is clipped to $[-\alpha_{\text{sym}}, \alpha_{\text{sym}}]$, and set the scale $s = \alpha_{\text{sym}}/127$. The signed INT8 code is:

$$x_q = \text{round}\left(\frac{x}{s}\right)$$

This symmetric mapping works well for weight distributions centered around zero, whereas asymmetric quantization handles distributions that are not centered (common after ReLU, which produces only nonnegative values) by shifting the range before scaling. Let α and β bound the calibrated range, clip x to $[\alpha, \beta]$, and set the scale $s = (\beta - \alpha)/255$. A common unsigned 8-bit mapping is:

$$x_q = \text{round}\left(\frac{x - \alpha}{s}\right)$$

The choice between symmetric and asymmetric quantization depends on the tensor's distribution and has measurable accuracy implications. Section 10.3 develops calibration, granularity, and the accuracy cost of each mapping on the book's workloads.

Summary

Machine-side diagnosis begins with units, ceilings, and working sets. The roofline model and the iron law identify which ceiling binds, and Amdahl's and Gustafson's laws, the memory hierarchy, the accelerator classes, and the numerical formats explain the gap between a measurement and that ceiling.

Treat peak specifications as ceilings, not predictions. Compare a measurement with the relevant ceiling, then trace the remaining gap to utilization, data movement, runtime overhead, architecture, or software. A back-of-envelope estimate is useful when it states its assumptions and units and makes the next measurement obvious. Chip specifications age with every product generation; the relationships here outlast them.

E

System Assumptions

A napkin calculation is only as trustworthy as the inputs behind it, and those inputs are easy to lose track of: an accelerator's peak rate, a model's parameter count, an electricity price, an energy cost per operation. This appendix gathers those canonical values and the unit conventions they depend on into a single reference sheet, so that a reader can check an estimate or substitute a newer figure. The chapters can then argue from numbers without restating where each one came from.

How to Use This Appendix

Every value here is dated: it belongs to a particular product, model, or price, and it changes when the next generation ships. Ratios and orders of magnitude, such as the energy gap between a DRAM access and an arithmetic operation, hold much longer, and section D.1 collects them. The NVIDIA H100 is the book's reference machine, so training and serving examples plug in the values of section E.2.2 unless a chapter names another device. Table E.1 maps each need to the section that holds the values. Section E.10 records where each value comes from, so a reader can substitute a current or local value and rerun the estimate without changing its method.

Table E.1: Where to Look: Each need maps to the section of this appendix that holds the values.

Need	Section
Convert a unit alias or check a decimal prefix	section E.1
Compare accelerator generations on one page	section E.2.1
Plug in the reference machine's peak rates, bandwidth, or ridge point	section E.2.2
Look up an older or alternative accelerator	section E.2
Size a mobile, edge, or microcontroller deployment	section E.2.9
Check which numeric formats a GPU generation supports	section E.2.10
Interpret an advertised peak-throughput claim	section E.2.11
Turn a list price into a price per unit of peak throughput	section E.2.12
Find a lighthouse's home regime and the resource that binds it	section E.3.1
Look up a model's parameters, operations, or training scale	section E.3.2
Size training-state memory per parameter	section E.4
Estimate the energy of an operation or a memory access	section E.5
Compare access latency and bandwidth across memory, links, and storage	section E.6.1
Estimate a transfer time over PCIe, NVLink, InfiniBand, or Ethernet	section E.6.2
Price electricity or data egress	section E.7
Anchor a production request volume or sensor data rate	section E.8
Follow a complete estimate built from these values	section E.9
Trace a value to its source	section E.10

E.1 Unit Conventions

Mixing binary (2^{30}-byte) capacities with decimal (10^9-byte) bandwidths shifts a memory or transfer estimate by 7 percent before any rounding, and confusing total FLOPs with FLOP/s turns a time

equation into a unit error. Every quantitative example in this book avoids those mismatches by using shared unit conventions: decimal data prefixes, distinct FLOPs and FLOP/s quantities, and the aliases in table E.2. Each row gives the multiplier k in `1 alias` $= k$ `base`. Data prefixes use decimal SI (KB $= 10^3$ bytes, not 1024). Binary IEC prefixes do not appear in the book's prose. Throughput quantities (FLOP/s, GB/s) combine these aliases with time; adding incompatible dimensions (bytes to FLOP/s) is a category error in napkin math, not a unit conversion.

Hardware capacity labels are the one deliberate exception. The reference tables retain vendor nameplate numbers and labels; the byte budgets used in calculations follow the book's device assumptions. For example, the book models an 80 GB accelerator with 80×2^{30} bytes and assigns 512×2^{10} bytes to a microcontroller with 512 KB of SRAM. These are modeled capacity assumptions, not universal guarantees inferred from the labels. Every other data quantity, including model footprints, activation sizes, and transfer volumes, uses the decimal prefixes above. Where a worked example subtracts a computed footprint from a device capacity, the capacity is converted to decimal units alongside the footprint, so that the arithmetic printed on the page is the arithmetic a reader can reproduce.

Table E.2: Unit Conventions: Decimal SI aliases and scale factors (book convention for napkin math). Throughput forms such as FLOP/s and GB/s divide work or data by time using the same conventions.

Alias	Multiplier	Base unit
`byte`	1	byte
`KB`	1000	byte
`MB`	1e+06	byte
`GB`	1e+09	byte
`TB`	1e+12	byte
`PB`	1e+15	byte
`flop`	1	flop
`GFLOPs`	1e+09	flop
`TFLOPs`	1e+12	flop
`ZFLOPs`	1e+21	flop
`param`	1	param
`Mparam`	1e+06	param
`Gbps`	1e+09	bit/s
`NS`	10^{-9}	second
`US`	10^{-6}	second
`MS`	10^{-3}	second
`second`	1	second
`hour`	3600	second
`day`	86400	second
`joule`	1	joule
`watt`	1	watt
`meter`	1	meter

E.2 Accelerator Specifications

Sizing a training cluster or a serving fleet requires a single set of per-chip ceilings so that estimates across chapters compare the workload rather than shifting hardware baselines. These **accelerator assumptions** supply the peak throughput, memory bandwidth, memory capacity, and thermal design power (TDP) for each generation the chapters cite. Values are vendor datasheet peaks, which serve as ceilings for napkin math rather than as sustained utilization, drawn from NVIDIA product documentation and IEEE Micro architecture articles (NVIDIA Corporation 2017, 2020b, 2020a, 2021a, 2018, 2024; Choquette et al. 2021; Choquette 2023), AMD MI300X documentation (AMD 2023), Google Tensor Processing Unit (TPU) publications (Jouppi et al. 2020; Jouppi et al. 2023), and Google

Cloud's current TPU v6e specification page (Google Cloud 2026b). Capacity rows keep the vendor's labeled capacity. The master matrix compares the generations on one page, and the sections after it give the per-chip values and three dated comparisons on precision support, advertised peaks, and price per unit of peak throughput. Section 11.6 develops the roofline analysis that turns these values into a regime for a given workload.

E.2.1 Master accelerator matrix

Side by side, the accelerators the book cites span more than an order of magnitude in peak rate, memory capacity, and power, yet most place their FP16 ridge point within a band of a few hundred FLOP per byte (table E.3). The ridge point divides peak FP16 throughput by memory bandwidth, so a workload whose arithmetic intensity falls below a chip's ridge is bandwidth bound on that chip and gains nothing from a faster datapath (section D.2.1 develops the model). Two chips sit far outside that band, at opposite ends. The first TPU paired an INT8-only datapath with DDR3 memory, which places its ridge point far above the band, and the wafer-scale engine keeps its weights in on-wafer SRAM, which places its ridge point far below it.

Table E.3: Master Accelerator Specification Matrix: Dense peak throughput, memory capacity and bandwidth, TDP, and FP16 ridge point (I_{ridge} = peak FP16 throughput divided by memory bandwidth) for each accelerator the book cites, read from the hardware registry. The FP8 or INT8 column gives each chip's dense low-precision peak where the registry lists one; TPU v1's ridge point uses its INT8 peak because it has no floating-point path. Memory is the vendor nameplate capacity.

Accelerator	Year	FP16/BF16 Peak	FP8 or INT8 Peak	Memory	Bandwidth	TDP	Ridge Point (I_{ridge})
NVIDIA V100	2017	125 TFLOP/s	—	32 GB	900 GB/s	300 W	138.9 FLOP/byte
NVIDIA T4	2018	65 TFLOP/s	130 TOPS (INT8)	16 GB	320 GB/s	70 W	203.1 FLOP/byte
NVIDIA A100	2020	312 TFLOP/s	624 TOPS (INT8)	80 GB	2,039 GB/s	400 W	153.0 FLOP/byte
NVIDIA H100	2022	989 TFLOP/s	1,979 TFLOP/s (FP8)	80 GB	3,350 GB/s	700 W	295.2 FLOP/byte
NVIDIA H200	2023	989 TFLOP/s	1,979 TFLOP/s (FP8)	141 GB	4,800 GB/s	700 W	206.0 FLOP/byte
NVIDIA B200	2024	2,250 TFLOP/s	4,500 TFLOP/s (FP8)	180 GB	8,000 GB/s	1,000 W	281.2 FLOP/byte
AMD Instinct MI300X	2023	1,307 TFLOP/s	2,614 TFLOP/s (FP8)	192 GB	5,300 GB/s	750 W	246.6 FLOP/byte
Intel Gaudi 2	2022	432 TFLOP/s	865 TFLOP/s (FP8)	96 GB	2,450 GB/s	600 W	176.3 FLOP/byte
Intel Gaudi 3	2024	1,835 TFLOP/s	1,835 TFLOP/s (FP8)	128 GB	3,700 GB/s	900 W	495.9 FLOP/byte
Google TPU v1	2017	—	92 TOPS (INT8)	8 GB	34 GB/s	75 W	2,705.9 FLOP/byte (INT8)
Google TPU v2	2017	46 TFLOP/s	—	16 GB	700 GB/s	280 W	65.7 FLOP/byte
Google TPU v3	2018	123 TFLOP/s	—	32 GB	900 GB/s	450 W	136.7 FLOP/byte
Google TPU v4	2021	275 TFLOP/s	—	32 GB	1,200 GB/s	200 W	229.2 FLOP/byte
Google TPU v5p	2023	459 TFLOP/s	918 TOPS (INT8)	95 GB	2,760 GB/s	300 W	166.3 FLOP/byte
Google TPU v6e	2024	918 TFLOP/s	—	32 GB	1,638 GB/s	300 W	560.4 FLOP/byte

Accelerator	Year	FP16/BF16 Peak	FP8 or INT8 Peak	Memory	Bandwidth	TDP	Ridge Point (I_{ridge})
Cerebras WSE-3	2024	125,000 TFLOP/s	—	44 GB SRAM	21 PB/s	23,000 W	6.0 FLOP/byte

E.2.2 NVIDIA H100 (reference machine)

The NVIDIA H100 is the default accelerator for the book's training and serving calculations, supplying a single baseline for peak FLOP/s, high-bandwidth memory (HBM) bandwidth, and capacity across chapters. It pairs Hopper-generation FP8 Tensor Cores and the Transformer Engine with HBM3 memory (Choquette 2023) (table E.4).

Table E.4: NVIDIA H100 (Hopper), the Reference Machine: Peak specifications from the Hopper architecture overview and datasheet (Choquette 2023), as recorded in the hardware registry. Ridge points divide each dense peak by HBM3 bandwidth. The list price is the representative range used in cost-performance examples, not a quote.

Assumption	Value
Release year	2022
Peak FP16/BF16 tensor throughput (dense)	989 TFLOP/s
Peak FP16/BF16 tensor throughput (2:4 sparse)	1,979 TFLOP/s
Peak FP8 tensor throughput (dense)	1,979 TFLOP/s
Peak INT8 tensor throughput (dense)	1,979 TOPS
Peak TF32 tensor throughput (dense)	494 TFLOP/s
Peak FP32 throughput (CUDA cores)	67 TFLOP/s
HBM3 capacity (nameplate)	80 GB
HBM3 bandwidth	3.35 TB/s
L2 cache	50 MB
Streaming multiprocessors	132
NVLink 4.0 bandwidth (bidirectional total)	900 GB/s
PCIe Gen5 x16 host link (per direction)	64 GB/s
TDP	700 W
Ridge point, FP16/BF16	295.2 FLOP/byte
Ridge point, FP8/INT8	590.7 FLOP/byte
List price (approximate)	~$25,000–30,000

The two ridge points turn these values into a classification. A kernel whose arithmetic intensity exceeds the FP16/BF16 ridge of 295.2 FLOP/byte is compute bound on the H100; one below it is bandwidth bound and runs no faster on a faster datapath. FP8 and INT8 double the peak rate while bandwidth stays fixed, so the ridge rises to 590.7 FLOP/byte and a layer that was compute bound at FP16 can fall below it after quantization. Section D.2.1.1 works the roofline example on these values, and section 11.6 develops the regimes.

E.2.3 NVIDIA V100

The Volta-generation V100 (table E.5) introduced Tensor Cores and is the accelerator of the GPT-3 training run that the book's cost and energy examples reconstruct. Its HBM2 bandwidth and power envelope serve as the early-generation baseline against which Ampere, Hopper, and Blackwell gains are measured.

Table E.5: NVIDIA V100 (Volta): Peak specs from NVIDIA's architecture whitepaper and product datasheet (NVIDIA Corporation 2017, 2020b). The V100 anchors baseline-generation comparisons.

Assumption	Value	Unit
Peak FP16 tensor throughput (V100)	125	TFLOP/s

Assumption	Value	Unit
Peak FP32 throughput (V100)	15.7	TFLOP/s
HBM bandwidth (V100)	900	GB/s
HBM capacity (V100)	32	GB
TDP (V100)	300	W

E.2.4 NVIDIA T4

Low-power inference comparisons use the Turing-generation T4 as their baseline (table E.6). Because the card fits within a low-power PCIe slot envelope without auxiliary power connectors, it anchors cost-per-query and edge-server comparisons across the serving chapters.

Table E.6: NVIDIA T4 (Turing): Peak specs from NVIDIA Tesla T4 product documentation (NVIDIA Corporation 2018). Its low TDP makes this inference accelerator a useful cost-per-inference anchor.

Assumption	Value	Unit
Peak FP16 tensor throughput (T4)	65	TFLOP/s
Peak INT8 throughput (T4)	130	TOPS
Memory bandwidth (T4)	320	GB/s
TDP (T4)	70	W

E.2.5 NVIDIA A100

The Ampere-generation A100 (table E.7) preceded the reference machine, and dated comparisons across generations use its values. It introduced TF32 and BF16 Tensor Cores alongside the HBM2e capacity tier that the H100 retained.

Table E.7: NVIDIA A100 (Ampere): Peak specs from NVIDIA's architecture whitepaper, 80 GB datasheet, and architecture article (NVIDIA Corporation 2020a, 2021a; Choquette et al. 2021). Ampere introduced TF32 Tensor Cores and the HBM capacity point that the H100 kept, with HBM3 in place of HBM2e.

Assumption	Value	Unit
Peak FP16 tensor throughput (A100)	312	TFLOP/s
Peak FP32 throughput (A100)	19.5	TFLOP/s
Peak INT8 throughput (A100)	624	TOPS
Peak TF32 throughput (A100)	156	TFLOP/s
HBM bandwidth (A100)	2039	GB/s
HBM capacity (A100)	80	GB
TDP (A100)	400	W

E.2.6 NVIDIA B200

Forward-looking capacity-planning examples rely on Blackwell-generation specs (table E.8). Its HBM3e bandwidth and native FP4 and FP8 Tensor Core rates show how memory capacity, bandwidth, and power continue to scale past Hopper.

Table E.8: NVIDIA B200 (Blackwell): Peak specs from NVIDIA Blackwell product documentation (NVIDIA Corporation 2024). Forward-looking capacity-planning examples use its HBM3e bandwidth and FP8 throughput.

Assumption	Value	Unit
Peak FP16 tensor throughput (B200)	2250	TFLOP/s
Peak FP8 tensor throughput (B200)	4500	TFLOP/s
Peak FP4 throughput (B200)	9000	TFLOP/s

Assumption	Value	Unit
HBM bandwidth (B200)	8	TB/s
HBM capacity (B200)	180	GB
TDP (B200)	1000	W

E.2.7 AMD Instinct MI300X

The MI300X capacity, bandwidth, throughput, and power specifications give a cross-vendor comparison with the H100 used elsewhere (table E.9). Its larger HBM3 capacity and memory bandwidth illustrate how an accelerator tuned for memory-bound serving shifts the single-device capacity ceiling.

Table E.9: AMD Instinct MI300X: Peak specs from AMD Instinct MI300X product documentation (AMD 2023). Its HBM capacity and bandwidth make it a cross-vendor baseline for bandwidth-bound and capacity-bound workloads.

Assumption	Value	Unit
Peak FP16 tensor throughput (MI300X)	1307	TFLOP/s
HBM bandwidth (MI300X)	5.3	TB/s
HBM capacity (MI300X)	192	GB
TDP (MI300X)	750	W

E.2.8 Google TPU v4, v5p, and v6e

When comparing training economics across accelerator families, TPUs, Google's custom application-specific integrated circuit (ASIC) accelerators, offer the main alternative to GPUs (table E.10). Each chip couples its systolic arrays to HBM and to neighboring chips over the Inter-Chip Interconnect (ICI), the TPU counterpart of NVLink.

Table E.10: Google TPU v4, v5p, and v6e (Trillium): TPU v4 values come from Google TPU publications (Jouppi et al. 2023), TPU v6e values from Google Cloud's Trillium specification page (Google Cloud 2026b), and TPU v5p values from Google Cloud's v5p specification as recorded in the hardware registry. ICI bandwidth for v5p and v6e is the bidirectional total per chip.

Assumption	TPU v4	TPU v5p	TPU v6e
Peak BF16 throughput	275 TFLOP/s	459 TFLOP/s	918 TFLOP/s
Peak INT8 throughput	—	918 TOPS	—
HBM capacity	32 GB	95 GB	32 GB
HBM bandwidth	1,200 GB/s	2,760 GB/s	1,638 GB/s
On-chip vector memory (VMEM)	—	128 MB	—
ICI bandwidth	300 GB/s	1,200 GB/s	800 GB/s
TDP	200 W	300 W	300 W

E.2.9 CPU and mobile/edge processors

Edge and mobile ML examples rely on a different performance baseline (table E.11), well below data center throughput. The microcontroller rows describe the ESP32-S3, the board on which the book's keyword-spotting examples run.

Table E.11: CPU, Mobile NPU, Edge, and Microcontroller Specs: Illustrative reference points for edge, mobile, and microcontroller examples (not vendor peaks for a single SKU, except the ESP32-S3 rows). The contrast with data-center accelerators shows why deployment target shapes every design decision.

Assumption	Value	Unit
Peak FP32 throughput (reference CPU)	1	TFLOP/s

Assumption	Value	Unit
DRAM bandwidth (reference server)	50	GB/s
Reference NPU throughput (iPhone 15 Pro)	35	TOPS
Memory bandwidth (iPhone 15 Pro)	51.2	GB/s
TDP (mobile device, reference)	5	W
Power (edge object detector, reference)	2	W
Battery capacity (phone, reference)	15	Wh
On-chip SRAM (microcontroller, ESP32-S3)	520	KiB
TDP (microcontroller, ESP32-S3)	0.4	W

E.2.10 Precision support by generation

Tensor Core format support widened with each NVIDIA generation from Volta through Ampere (table E.12). Volta supported little below FP16, whereas Ampere introduced TF32, which combines the exponent range of FP32 with the mantissa width of FP16 (NVIDIA 2020b), alongside broader support for BF16, INT8, and INT4 (NVIDIA Corporation 2017, 2018, 2020a). Later generations narrow the formats further, with FP8 on Hopper and FP4 on Blackwell (table E.4; table E.8). Section 11.4.5.1 explains why each narrower format raises peak throughput on the same silicon.

Table E.12: Precision Support Evolution: From Volta through Ampere, Tensor Core support expanded from FP16 to TF32, BF16, and integer formats, broadening the precision choices available to training and inference; the CUDA-core column shows the separate scalar/vector formats each generation supports.

Architecture	Year	Tensor Core Formats	CUDA Core Formats
Volta	2017	FP16	FP64, FP32, FP16
Turing	2018	FP16, INT8, INT4, INT1	FP64, FP32, FP16, INT8
Ampere	2020	FP64, TF32, BF16, FP16, INT8, INT4	FP64, FP32, FP16, BF16, INT8

E.2.11 Advertised peak throughput across generations

Advertised single-chip GPU throughput rose by more than three orders of magnitude across the generations plotted in figure E.1, which combine architectural scaling with lower numerical precisions and structured sparsity (NVIDIA Corporation 2017, 2020a, 2024; Choquette 2023). Because the points mix precision formats, combine FLOP/s with INT8 operations per second, and assume structured sparsity for the A100, H100, and B200, they do not measure one consistent quantity; instead, the curve compares each generation's most prominent advertised path rather than same-precision application performance. Table E.3 gives the same chips' dense, precision-specific peaks, bandwidths, and power limits.

E.2.12 Accelerator cost worksheet

An illustrative cost worksheet divides an approximate list price by a representative peak for three accelerator generations (table E.13). Because list prices vary across vendor, region, contract terms, and procurement volume while the throughput rows use distinct numerical precision modes, the resulting price-per-peak-operation ratio is meaningful within a single row to show how a price assumption enters the arithmetic, rather than as a cross-device ranking. Section 11.11.2 states the durable rule the worksheet supports, which divides cost by the throughput a workload delivers on its binding resource rather than by the advertised peak.

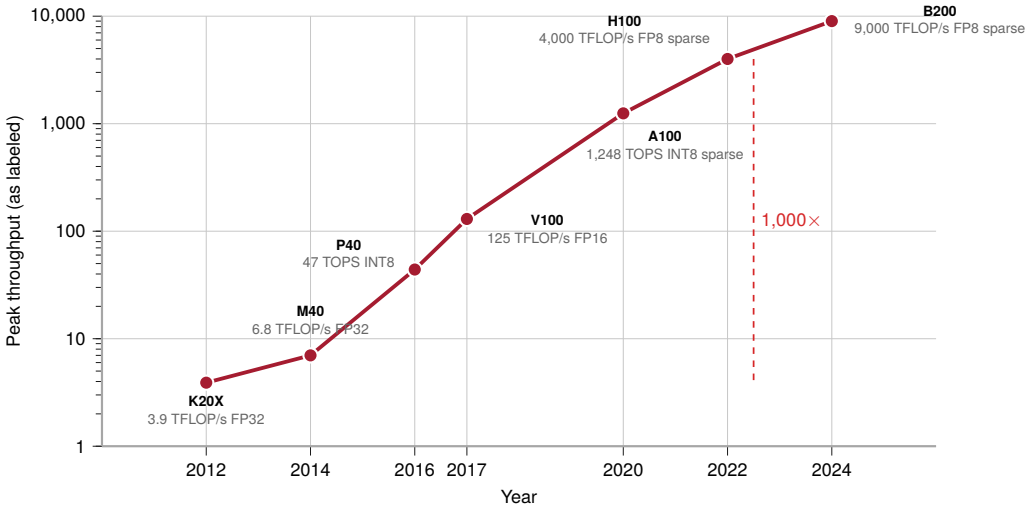


Figure E.1: Advertised GPU Peak Capability Across Generations: Single-chip values are plotted on a logarithmic axis. The precision label at each point matters because the curve does not measure same-precision scaling. The series follows Dally (2023; NVIDIA Corporation 2017, 2020a, 2024; Choquette 2023).

Table E.13: Illustrative Accelerator Cost Worksheet: Each row divides an approximate acquisition-price assumption by the representative peak mode shown. Because precision, sparsity assumptions, and price sources differ, the ratios are not comparable as workload performance. Total cost also includes utilization, memory capacity, power, cooling, interconnect, software, and infrastructure.

Accelerator	List Price	Representative Peak (precision shown)	Memory Bandwidth	Price per Peak TFLOP/s
NVIDIA V100	~$10,000	125 TFLOP/s (FP16)	900 GB/s	$80/(TFLOP/s)
NVIDIA A100	~$15,000	312 TFLOP/s (FP16)	2,039 GB/s	$48.1/(TFLOP/s)
NVIDIA H100	~$25,000–30,000	494 TFLOP/s (TF32)	3,350 GB/s	~$50.6/(TFLOP/s)

E.3 Model Specifications

The book's quantitative examples run on a small set of recurring workloads. The first table places the five lighthouse workloads in the regimes they occupy on their machines; the second gives the parameter counts, operation budgets, and training-scale anchors of every model the chapters cite.

E.3.1 Lighthouse workloads

Five lighthouse workloads recur through the book, each introduced in the regime it typifies (section 1.7.1; section 2.3.2). Because a regime belongs to a workload on a machine rather than to a model name, each row of table E.14 is keyed by model, phase, and batch, with the second GPT-2 XL row recording a crossing out of the model's home regime. Across those rows, four regimes appear: compute-bound and bandwidth-bound workloads are separated by the roofline's ridge point, whereas capacity-bound and power-bound workloads are constrained before either time term applies because the working set must fit in memory and the energy budget must cover the run. For the compute- and bandwidth-bound rows, the last column compares the time a step spends on arithmetic at peak FP16 throughput with the time it spends reading the weights once at peak HBM bandwidth on one H100, and whichever is larger names the regime; a fifth regime, overhead-bound, arises when the iron law's fixed latency term dominates and belongs to a deployment choice such as batch-one serving rather than to a lighthouse (section A.7.2).

Table E.14: Lighthouse Workloads by Model, Phase, and Batch: Each row's regime on its machine, with the registry quantities that set it. Times are idealized lower bounds at peak FP16 throughput and peak HBM bandwidth of one H100 (the phone NPU's INT8 peak for MobileNetV2); activation traffic is excluded, so the comparison establishes a regime only where the two times differ by orders of magnitude. Training costs about three forward passes, and decode costs two FLOPs per parameter per token.

Light-house	Phase	Batch	Machine	Regime	Binding Resource	What Sets the Regime
ResNet-50	Training	256 images	H100	Compute-bound	Tensor throughput	6.4 ms of arithmetic against 15.3 µs of weight reads per step
Mo-bileNetV2	Inference	1 image	Phone NPU	Compute-bound (efficient regime)	Arithmetic within a phone's power envelope	17.1 µs of arithmetic at the NPU's 35 TOPS peak, inside a 5 W device envelope
GPT-2 XL	Decode	1 sequence	H100	Bandwidth-bound	HBM bandwidth	3.0 µs of arithmetic against 0.90 ms of weight reads per token
GPT-2 XL	Training	32 sequences of 1,024 tokens	H100	Compute-bound (crossing)	Tensor throughput	298.2 ms of arithmetic against 0.90 ms of weight reads per step
Llama 3 8B	Decode, 8,192-token context	1 sequence	H100	Bandwidth-bound	HBM bandwidth and KV-cache capacity	5.1 ms reading 16.1 GB of weights and 1.07 GB of KV cache, against 16.2 µs of arithmetic per token
DLRM	Inference	Any	H100	Capacity-bound	Memory capacity	100 GB of embedding tables against 80 GB of HBM
KWS (DS-CNN)	Inference	1 audio window	ESP32-S3	Power-bound	Energy per inference	200 KB INT8 model in 520 KB of SRAM, on a board drawing 50 mW to 1.2 W

The same model changes regime when phase or batch changes. GPT-2 XL streams every weight to produce one decode token, but a training step amortizes one weight read over tens of thousands of tokens and becomes arithmetic-dominated, while Llama 3 8B's KV cache adds bytes to every decode step as the context grows. Section 11.6 places these workloads on the H100 roofline, section 6.7.4.2 sizes the KV cache that makes decode bandwidth bound, and section 13.9.5 budgets that cache against serving capacity.

E.3.2 Reference model specifications

Parameter counts, per-inference FLOP budgets, and published training-scale anchors form the **reference model assumptions** behind training-cost, memory-footprint, and inference-workload examples (table E.15). Published model sizes follow primary papers, model reports, and official documentation (Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023; Naumov et al. 2019; C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kirber, et al. 2021), whereas the inference FLOP budgets are book-derived from sequence length 128 for BERT-Base, 224×224 images for ResNet-50 and MobileNetV2, and 640×640 images for YOLOv8-Nano, counting two FLOPs per multiply-add in the vision models. For large-scale training anchors, GPT-3 training FLOPs follow (Brown et al. 2020) while its duration and accelerator count come from the published carbon reconstruction (Patterson et al. 2021), and because the GPT-4 technical report does not disclose architecture size, the GPT-4 parameter count and training GPU-days use public third-party mixture of experts (MoE) estimates (Patel and Wong 2023). Finally, the DLRM row represents the book's reference configuration, whose FP32 embedding values exceed one accelerator's memory, rather than a single production recommender.

Table E.15: Reference Model Specifications: Parameter counts and FLOP budgets for worked examples, with sources as given in the text above. GPT-4 rows cite (OpenAI et al. 2023) for the official model family and (Patel and Wong 2023) for the public MoE parameter and GPU-day estimates used in this edition.

Assumption	Value	Unit
Inference FLOPs (BERT-Base)	2.2e+10	flop
Parameters (BERT-Base)	1.1e+08	param
Parameters (Llama 3 8B)	8.03e+09	param
Hidden dimension (GPT-2 XL)	1600	-
Layers (GPT-2 XL)	48	-
Parameters (GPT-2 XL)	1.5e+09	param
Parameters (GPT-3)	1.75e+11	param
Reference training duration (GPT-3)	14.8	d
Reference training accelerators (GPT-3)	10,000	V100 GPUs
Reference training FLOPs (GPT-3)	3.14e+23	flop
Parameters (GPT-4, public MoE estimate)	1.76e+12	param
Reference training GPU-days (GPT-4)	2.5e+06	GPU-days
Inference FLOPs (ResNet-50)	8.2e+09	flop
Parameters (ResNet-50)	2.56e+07	param
Inference FLOPs (MobileNetV2)	6e+08	flop
Parameters (MobileNetV2)	3.50487e+06	param
Inference FLOPs (YOLOv8-Nano)	8.7e+09	flop
Embedding table size (DLRM, reference)	100	GB
Embedding values (DLRM, reference)	2.5e+10	embedding values
Parameters (KWS DS-CNN)	200000	param
Inference FLOPs (KWS DS-CNN)	2e+07	flop

E.4 Training Memory Conventions

Training-memory napkin math assumes the mixed-precision Adaptive Moment Estimation (Adam) storage model used in the worked training-memory estimate and several training chapters: BF16 weights, BF16 gradients, and FP32 master weights plus Adam first- and second-moment buffers (16 bytes per parameter in total). This is a book convention, not a measured hardware constant. The BF16 data path uses standard BF16 mixed-precision training (Kalamkar et al. 2019), the FP32 master copy of the weights follows earlier FP16 work (Micikevicius et al. 2018; NVIDIA 2017), and Adam's first- and second-moment buffers follow (Kingma and Ba 2015). Multiplying the bytes per parameter by the parameter count gives the nonactivation training-state footprint, and table E.16 lists the per-component byte widths. Section C.3.3 derives each component and the activation memory that adds to it, and section 8.3.2 applies the convention to the book's training runs.

Table E.16: Training Memory Conventions: Per-parameter storage for mixed-precision Adam, 16 bytes before activations. Book convention for napkin math; see (NVIDIA 2017) for mixed-precision training context.

Assumption	Value	Unit
Weight/gradient width (BF16)	2	bytes
Master weight width (FP32)	4	bytes
Master + Adam states per parameter (FP32)	12	bytes
Bytes per parameter (mixed-precision Adam)	16	bytes

E.5 Energy Constants

The specifications above bound how fast a chip can run; **energy assumptions** determine whether the design is thermally and economically viable at the level of the operation and the memory

access. Horowitz's 45 nm arithmetic and 32-bit DRAM estimates (Horowitz 2014), together with illustrative book anchors for register and SRAM access, MobileNetV2 inference, and 5G transfer, underpin the book's efficiency comparisons but are not process-independent. Access energy rises from registers through DRAM, and the arithmetic energy of an add and a multiply falls with operand width (table E.17). The access hierarchy quantifies why data reuse dominates kernel design, and section D.3.1 explains the physics behind it as ratios that hold across process nodes.

Table E.17: Energy per Operation and Access: Arithmetic and 32-bit DRAM estimates use Horowitz's 45 nm model (Horowitz 2014), and the per-byte DRAM row is derived from that estimate. Register, SRAM, MobileNetV2, and 5G rows are illustrative book anchors. The roughly 6,400× gap between a 0.1 pJ register access and a 640 pJ DRAM access explains why data reuse dominates ML kernel optimization.

Assumption	Value	Unit
Register access energy	0.1	pJ
L1 SRAM access energy	0.5	pJ
L2 SRAM access energy	2	pJ
DRAM access energy (32-bit)	640	pJ
DRAM access energy (per byte)	160	pJ/byte
FP32 add energy (45 nm)	0.9	pJ/add
FP16 add energy (45 nm)	0.4	pJ/add
INT32 add energy (45 nm)	0.1	pJ/add
INT8 add energy (45 nm)	0.03	pJ/add
FP32 multiply energy (45 nm)	3.7	pJ/multiply
FP16 multiply energy (45 nm)	1.1	pJ/multiply
INT8 multiply energy (45 nm)	0.2	pJ/multiply
MobileNetV2 inference energy (reference)	0.1	mJ
5G transfer energy per MB	100	mJ/MB

The arithmetic rows carry two orderings that hold at every process node. A multiply costs several times an add at the same precision, because multiplier area grows roughly with the square of the operand bit width b while an adder grows linearly in it, and both fall as the operand narrows (Horowitz 2014; Dally et al. 2021). Newer nodes lower every absolute value in the table, so the 45 nm figures are upper anchors for current silicon, and the orderings, not the absolute values, are what precision and reuse arguments depend on.

E.6 Interconnect and Network Bandwidth

Energy costs operate at the chip level, but real ML systems also move data across interconnects, between accelerators, across racks, and over wide-area networks. These **bandwidth assumptions** apply when chapters reason about gradient synchronization, pipeline bubbles, checkpoint I/O, or cross–data center latency. The first table gives the latency and bandwidth of each tier of one H100 node, from registers to local storage; the second gives the link families that connect accelerators, hosts, and nodes.

E.6.1 Latency and bandwidth by tier

Access latency rises by five orders of magnitude from a register to an NVMe read, while bandwidth falls from terabytes per second at HBM to single gigabytes per second at local storage (table E.18). Section D.1.1 expresses the same tiers as decades and as multiples of a register access, which change more slowly than the values here.

Table E.18: Latency and Bandwidth by Tier (H100 Node): Representative access latency and bandwidth from registers to local storage. Register, L1, and L2 latencies are generic on-chip anchors; the HBM row uses a generic off-chip DRAM latency. NVLink bandwidth is the bidirectional total; other bandwidths are per direction. Host DRAM latency is not recorded, so that cell is left empty.

Tier	Technology	Latency	Bandwidth
Registers	Flip-flops	0.3 ns	—
L1 cache	SRAM	1 ns	—
L2 cache	SRAM	4 ns	—
Accelerator memory	HBM3 (H100)	300 ns	3,350 GB/s
Accelerator link	NVLink 4.0 (H100)	500 ns	900 GB/s
Host link	PCIe Gen5 x16	1,000 ns	64 GB/s
Host memory	DDR4/DDR5	—	50 GB/s
Network	InfiniBand NDR	5 µs	50 GB/s
Local storage	NVMe SSD (Gen4)	100 µs	7 GB/s

E.6.2 Interconnect specifications

On the reference node, NVLink runs more than an order of magnitude faster than the PCIe host link, and the host link runs faster than the node's InfiniBand NDR port, so the boundary a transfer crosses sets its cost before its size does (table E.19). Every rate is quoted in GB/s for comparison, with the direction convention each vendor uses; a bidirectional total halves for a one-way transfer. NVLink and PCIe rates follow accelerator product documentation (NVIDIA Corporation 2017, 2020a; Choquette 2023); the InfiniBand architecture specification anchors the protocol family (InfiniBand Trade Association 2000), while current high-speed product families and Ethernet roadmaps anchor modern link-rate examples (NVIDIA 2026f; Ethernet Alliance 2025). Section 11.5.4 develops the transfer-time model these rates feed, and the speed of light in fiber, which bounds every cross–data center round trip, is a physical constant kept with the other durable values in section D.1.

Table E.19: Interconnect Specifications: Link-family bandwidths from accelerator product documentation, the InfiniBand architecture specification, and current product and roadmap documentation (NVIDIA Corporation 2017, 2020a; Choquette 2023; InfiniBand Trade Association 2000; NVIDIA 2026f; Ethernet Alliance 2025), as recorded in the hardware and systems registries. Network ports are converted from line rate at eight bits per byte. Latencies are representative end-to-end values where the registry records one: a generic NVLink and PCIe hop and a fabric-level network latency.

Link	Device or Generation	Bandwidth	Convention	Latency
PCIe Gen3 x16	Host link (V100)	15.75 GB/s	Per direction	—
PCIe Gen4 x16	Host link (A100)	32 GB/s	Per direction	—
PCIe Gen5 x16	Host link (H100, B200)	64 GB/s	Per direction	1,000 ns
CXL memory over PCIe Gen5	Memory expansion	64 GB/s	Per direction	—
NVLink 2.0	V100	300 GB/s	Bidirectional total	500 ns
NVLink 3.0	A100	600 GB/s	Bidirectional total	500 ns
NVLink 4.0	H100	900 GB/s	Bidirectional total	500 ns
NVLink 5.0	B200	1,800 GB/s	Bidirectional total	500 ns
ICI	TPU v4	300 GB/s	Per direction	—
ICI	TPU v5p	1,200 GB/s	Bidirectional total	—
ICI	TPU v6e	800 GB/s	Bidirectional total	—
InfiniBand EDR	Network port	12.5 GB/s	Per direction	—
InfiniBand HDR	Network port	25 GB/s	Per direction	7 µs
InfiniBand NDR	Network port	50 GB/s	Per direction	5 µs
InfiniBand XDR	Network port	100 GB/s	Per direction	5 µs
RoCE (100 GbE)	Network port	12.5 GB/s	Per direction	10 µs

Link	Device or Generation	Bandwidth	Convention	Latency
RoCE (400 GbE)	Network port	50 GB/s	Per direction	10 µs
10 GbE	Network port	1.25 GB/s	Per direction	50 µs
100 GbE	Network port	12.5 GB/s	Per direction	50 µs
400 GbE	Network port	50 GB/s	Per direction	50 µs
800 GbE	Network port	100 GB/s	Per direction	50 µs
NVMe SSD (Gen3)	Local storage	3.5 GB/s	Sequential read	—
NVMe SSD (Gen4)	Local storage	7 GB/s	Sequential read	—
NVMe SSD (Gen5)	Local storage	14 GB/s	Sequential read	—

E.7 Economic Constants

Converting a training run's megawatt-hours or a dataset's cross-region transfer volume into dollars requires baseline utility and network rates. These **pricing assumptions** underpin total cost of ownership (TCO) and energy-cost napkin math in table E.20. They are illustrative hyperscaler-order rates for ratio analysis (similar in spirit to carbon accounting examples in (Patterson et al. 2021)), not quotes for a specific region or contract. Local values should replace these anchors when absolute price dominates. Accelerator list prices are kept with each chip, in table E.4 and table E.13.

Table E.20: Economic Assumptions: Illustrative cloud electricity and egress rates for TCO napkin math (2024–2025 order of magnitude). On-premise cost structures differ.

Assumption	Value	Unit
Cloud electricity price	0.12	dollar/kWh
Cloud egress price per GB	0.09	dollar/GB

E.8 Scale References

Capacity-planning examples need request rates and data rates, so the **scale assumptions** in table E.21 fix illustrative order-of-magnitude baselines for email and search volume, autonomous-vehicle sensor streams, and standard 1080p and 4K video formats. They are magnitude anchors, not audited statistics for a specific year.

Table E.21: Production Scale and Data Rate References: Illustrative workload and sensor-rate anchors, plus standard 1080p and UHD 4K video parameters.

Assumption	Value	Unit
Gmail emails per day	1.21e+11	-
Google searches per day	1.37e+10	-
Waymo sensor data rate (low)	1	TB/h
Waymo sensor data rate (high)	19	TB/h
1080p frame width	1920	-
1080p frame height	1080	-
4K frame width	3840	-
4K frame height	2160	-
Bytes per RGB pixel	3	bytes
Video frame rate (standard)	30	Hz

E.9 Worked Estimates

A table of peak rates, byte widths, and utility prices does not by itself tell an engineer whether a kernel is bandwidth bound, whether a fine-tuning job fits on one GPU, or what a training run's electricity bill covers. The three worked estimates below combine the tables above to answer those

performance, capacity, and cost questions, and each names the section that develops its method so a reader can rerun it with substituted values.

The first estimate places two operations on the reference machine's roofline, using the method of section D.2.1. Comparing the ridge point against each operation's arithmetic intensity separates compute-bound matrix multiplies from bandwidth-bound token generation.

Napkin Math E.1: Classify a roofline regime

Problem: For the stated H100 and workload intensities, which operations are compute bound and which are bandwidth bound?

Variables: For an H100 at FP16/BF16 peak, use 989 TFLOP/s peak compute and 3.35 TB/s memory bandwidth.

Math: Divide peak FLOP/s by memory bandwidth to get the roofline ridge point: 989 TFLOP/s/3.35 TB/s $\approx$ 295.2 FLOP/byte. A large general matrix multiply (GEMM) with $N = 4096$ has intensity $N/3 \approx 1{,}365.3$ FLOP/byte.

Result: Operations above 295.2 FLOP/byte are compute bound; operations below it are bandwidth bound. The GEMM example is compute bound, while a single-token autoregressive decode at FP16, with intensity of about 1 FLOP/byte, is deeply bandwidth bound.

Systems insight: The same accelerator can be compute rich and memory constrained. Arithmetic intensity determines which resource the workload actually consumes.

The second estimate sizes training state with the convention of table E.16. Section C.3.3 derives the components, and section 8.3.3.3 shows how parameter-efficient fine-tuning brings the same model onto one accelerator.

Napkin Math E.2: Estimate training-state memory

Problem: How much model-state memory is required to fully fine-tune Llama 3 8B with mixed-precision Adam?

Variables: Mixed-precision Adam stores BF16 weights, gradients, FP32 master weights, momentum, and variance. The model-state budget is 16 bytes per parameter.

Math: For 8.03B parameters, model state is 8.03B $\times$ 16 bytes = 128.5 GB.

Result: An H100 has 80 GB of HBM, so the model state alone exceeds a single accelerator before accounting for activations.

Systems insight: Optimizer state, not weights alone, sets the floor for training memory. Capacity planning that starts from parameter bytes underestimates the real requirement.

The third estimate converts power and time into energy and cost. Section D.2.2 checks that the units of such a product reduce to energy, and section 15.4.2.1 applies the same accounting to cost and carbon at deployment scale.

Napkin Math E.3: Estimate accelerator electricity cost

Problem: What is the accelerator-only electricity cost of the GPT-3 training run?

Variables: Use 10,000 V100s, 300 W per accelerator, ~14.8 days wall-clock, and \$0.12/kWh electricity (Patterson et al. 2021).

Math: The run requires roughly 3.14×10^{23} FLOPs. The accelerator-only electricity-cost estimate is ~14.8 days wall-clock $\times$ 10,000 V100s $\times$ 24 h/day $\times$ 0.3 kW $\times$ \$0.12/kWh = ~\$127,872.

Result: The accelerator-only electricity cost is roughly \$128,000. This TDP-only estimate excludes host CPUs, networking, storage, cooling, and facility overhead. The 1,066 MWh of accelerator energy behind it covers about 83 percent of the 1,287 MWh reported for the run, which also counts host power and data center overhead. The GPU count and duration are a

published reconstruction from measured per-GPU throughput, so only their product is firm (Patterson et al. 2021).

Systems insight: Energy follows directly from power and time, while the dollar figure depends on the electricity rate assumed.

E.10 Assumption Provenance

Each assumption has a provenance record that lets readers trace the number back to its primary source. The appendix's quantitative inputs fall into evidence classes, each with its primary references (table E.22). Vendor specifications and standards define published limits, peer-reviewed measurements provide empirical anchors, and explicitly marked editorial scenarios supply illustrative values where no durable universal constant exists. Every value is read from the book's hardware, model, systems, and infrastructure registries, which record the same provenance per entry. A reader should check the class before trusting a number, because vendor peaks are ceilings, measurements are anchors, and scenarios are placeholders.

Table E.22: Assumption Provenance Catalog: Quick map from appendix section to source class and bibliography.

Appendix section	Source type	Primary references
Accelerator specifications	Vendor datasheet peaks and list-price ranges; editorial CPU/mobile anchors; release-year format support; advertised marketing peaks	(NVIDIA Corporation 2017, 2018, 2020a, 2024; NVIDIA 2020b; Choquette et al. 2021; Choquette 2023; AMD 2023; Jouppi et al. 2023; Google Cloud 2026b); editorial for CPU/mobile rows
Model specifications	Published papers, model reports, benchmark references, and official docs; GPT-3 run reconstruction; GPT-4 size from public analysis	(Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Patterson et al. 2021; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023; Naumov et al. 2019; C. Banbury, Reddi, Torelli, Holleman, Jeffries, Kirber, et al. 2021; OpenAI et al. 2023; Patel and Wong 2023)
Training memory conventions	Book convention (mixed-precision Adam layout)	(Kingma and Ba 2015; Kalamkar et al. 2019; Micikevicius et al. 2018; NVIDIA 2017)
Energy constants	Published 45 nm arithmetic/DRAM estimates; add-versus-multiply scaling	(Horowitz 2014; Dally et al. 2021) for arithmetic/DRAM; editorial for register/SRAM, MobileNetV2, and 5G rows
Interconnect bandwidth	Vendor specs; InfiniBand standard; product and roadmap documentation; representative latency tiers	(NVIDIA Corporation 2017, 2020a; Choquette 2023; InfiniBand Trade Association 2000; NVIDIA 2026f; Ethernet Alliance 2025)
Economic assumptions	Illustrative cloud/utility rates	(Patterson et al. 2021) (methodology context)
Scale references	Illustrative workload rates; standard video conventions	Editorial for workload rates; standard format definitions for video
Unit conventions	Decimal SI; book notation	Editorial
Worked estimates	Derived from the rows above	As for the inputs they use

E.11 Summary

This appendix is the reference catalog for the dated quantitative inputs used throughout the book. It runs from unit conventions and accelerator specifications through model, energy, interconnect, and cost assumptions, and closes with three worked estimates and a provenance map. The values come from vendor peaks, published measurements, standards, and editorial scenarios, so they carry different evidentiary weight and age at different rates.

Use these tables as starting inputs, not universal constants. For each estimate, record the table row, unit convention, and provenance; replace time-sensitive peaks, prices, and rates with local values; distinguish a physical ceiling from a measured operating point and a scenario assumption; and report which assumptions dominate the result. This keeps napkin math reproducible and makes clear which input controls the conclusion.

Glossary

This glossary defines key terms used throughout this book and gathers them as a compact reference. Terms are organized alphabetically so they can be consulted alongside the chapters.

3

3DMark Graphics performance benchmark suite that evaluates real-time 3D rendering capabilities, measuring triangle throughput, texture fill rates, and modern features like ray tracing and DLSS performance.

A

A/B testing A controlled experimental method for comparing two versions of a system or model by randomly dividing users into groups and measuring performance differences between the variants.

ablation study An experiment that removes, disables, or isolates one component at a time to measure its contribution to model or system behavior.

activation-based pruning A pruning method that evaluates the average activation values of neurons or filters over a dataset to identify and remove neurons that consistently produce low activations and contribute little information to the network's decision process.

activation checkpointing A memory optimization technique that reduces memory usage during backpropagation by selectively discarding and recomputing activations instead of storing all intermediate results.

activation function A mathematical function applied to the weighted sum of inputs in a neural network neuron to introduce nonlinearity, enabling the network to learn complex patterns beyond simple linear combinations.

activation memory Memory used to store intermediate layer outputs needed for gradient computation during training, scaling with batch size, sequence length, and model depth.

active learning An approach that intelligently selects the most informative examples for human annotation based on model uncertainty, reducing the amount of labeled data needed for effective training.

Adam optimization An adaptive learning rate optimization algorithm that combines momentum and RMSprop by maintaining exponentially decaying averages of both gradients and squared gradients for each parameter.

AdamW A variant of Adam that decouples weight decay from the adaptive gradient update, improving regularization while keeping the optimizer's adaptive moments.

alerting Automated notification systems that inform teams when metrics exceed predefined thresholds or anomalies are detected in production ML systems.

AlexNet A landmark convolutional neural network architecture that won the 2012 ImageNet challenge, reducing error rates from 26 percent to 16 percent and sparking the deep learning renaissance.

AllReduce A collective communication operation in distributed computing where each process contributes data and all processes receive the combined result, commonly used for gradient aggregation in distributed training.

all-to-all communication A communication pattern where every device or process exchanges data with every other device, common in sharded embedding and expert-parallel workloads.

AlphaFold A landmark AI system developed by DeepMind that predicts the three-dimensional structure of many proteins from their amino acid sequences, achieving breakthrough accuracy without solving every folding problem and demonstrating how large-scale ML systems can accelerate scientific discovery. Its predictions still require confidence interpretation and, for many uses, experimental validation.

Amdahl's Law A speedup bound showing that overall improvement is limited by the portion of a workload or pipeline that remains serial or unoptimized.

AOT compilation Ahead-of-time compilation that converts a model, graph, or program into optimized executable code before deployment or execution, trading flexibility for lower runtime overhead.

Apache Kafka A distributed streaming platform that handles real-time data feeds using a publish-subscribe messaging system, commonly used for building ML data pipelines with high throughput and fault tolerance.

Apache Spark An open-source distributed computing framework that enables large-scale data processing across clusters of computers, revolutionizing ETL operations with in-memory computing capabilities.

application-specific integrated circuit (ASIC) Custom chips designed for specific computational tasks that offer superior performance and energy efficiency compared to general-purpose processors by abandoning general-purpose flexibility. Examples include Google's TPUs, Cerebras Wafer-Scale Engine, and Bitcoin mining ASICs.

arithmetic intensity The ratio of floating-point operations to bytes of memory accessed (FLOP/byte), used in roofline analysis to determine whether workloads are memory bound or compute bound and to guide optimization priorities.

artificial general intelligence AI systems intended to match human-level performance across a broad range of cognitive tasks; the term has no universally accepted capability threshold and implies no particular architecture.

artificial intelligence The field of computer science focused on creating systems that can perform tasks typically requiring human intelligence, such as perception, reasoning, learning, and decision-making.

artificial neural network A computational model inspired by biological neural networks, consisting of interconnected nodes (neurons) organized in layers that can learn patterns from data through adjustable weights and biases.

artificial neurons Basic computational units in neural networks that mimic biological neurons, taking multiple inputs, applying weights and biases, and producing an output signal through an activation function.

attention mechanism A neural network component that computes weighted connections between elements based on their content, allowing dynamic focus on relevant parts of the input rather than fixed architectural connections.

AUC Area under the ROC curve, a threshold-independent classification metric that compares true positive and false positive rates across decision thresholds.

audit trail An append-only record of who accessed or changed data, models, or system artifacts and when, supporting accountability, debugging, and compliance.

autograd tape A transient record of operations and saved values built during forward execution so reverse-mode automatic differentiation can compute gradients during the backward pass.

automatic differentiation A computational technique that automatically calculates exact derivatives of functions implemented as computer programs by systematically applying the chain rule at the elementary operation level, essential for training neural networks through gradient-based optimization.

automatic mixed precision A training technique that automatically manages the use of different numerical precisions (FP16, FP32) to optimize memory usage and computational speed while maintaining model accuracy.

AutoML Automated machine learning that automates model design decisions, including architecture search, hyperparameter optimization, and feature selection, to reduce manual effort.

autoscaling Dynamic adjustment of compute resources based on workload demand, auto-

matically scaling up during peak usage and scaling down during low usage to optimize costs and performance.

B

backpropagation An algorithm that computes gradients of the loss function with respect to network weights by propagating error signals backward through the network layers, enabling systematic weight updates during training.

backward pass The training phase that propagates gradients from the loss back through the network to compute parameter gradients, pairing with the forward pass.

bandwidth The maximum rate of data transfer across a communication channel or memory interface, typically measured in bytes per second and critical for optimizing data movement in AI accelerators.

batch inference The process of running inference on a large dataset in bulk, typically as a scheduled job, as opposed to real-time inference on individual requests. Batch inference enables higher throughput by amortizing overhead across many inputs.

batch ingestion A data processing pattern that collects and processes data in groups or batches at scheduled intervals, suitable for scenarios where real-time processing is not critical.

batch normalization A technique that normalizes inputs to each layer to have zero mean and unit variance, which stabilizes training and often allows for higher learning rates and faster convergence, and subsequently applies a learnable scale and shift to preserve representational power.

batch processing The technique of processing multiple data samples simultaneously to amortize computation and memory access costs, improving overall throughput in neural network training and inference.

batch size The number of training examples processed simultaneously during one iteration of neural network training, affecting both computational efficiency and gradient estimation quality.

batch throughput optimization Techniques for maximizing the number of samples processed per unit time when handling multiple inputs simultaneously, using parallelism and batching efficiencies.

batched operations Matrix computations that process multiple inputs simultaneously, converting matrix-vector operations into more efficient matrix-matrix operations to improve hardware utilization.

benchmark engineering The systematic design and development of performance evaluation frameworks, involving test harness creation, metric selection, and result interpretation methodologies.

benchmark harness Systematic infrastructure component that controls test execution, manages input delivery, and collects performance measurements under controlled conditions to ensure reproducible evaluations.

benchmarking Systematic evaluation of compute performance, algorithmic effectiveness, and data quality in machine learning systems to optimize performance across diverse workloads and ensure reproducibility.

BERT Bidirectional Encoder Representations from Transformers, a transformer-based language model introduced by Google in 2018 that revolutionized natural language processing through masked language modeling pretraining.

BF16 A 16-bit floating-point format developed by Google Brain that maintains the same dynamic range as FP32 but with reduced precision, making it particularly suitable for deep learning training.

bias A learnable parameter added to the weighted sum in each neuron that shifts the activation function, allowing neurons to activate even when all inputs are zero and providing additional flexibility for the network to fit complex patterns.

binarization A 1-bit quantization technique that reduces neural network weights, activations, or both to binary values (typically -1 and +1), achieving maximum compression but often requiring specialized training procedures and hardware support.

biological neuron A cell in the nervous system that receives, processes, and transmits information through electrical and chemical signals, serving as inspiration for artificial neural networks.

bitter lesson Richard Sutton's 2019 observation that general methods using computation consistently outperform approaches encoding human expertise, suggesting that systems engineering enabling computational scale is central to AI advancement.

black box A system whose inputs and outputs are observable but whose internal workings are inaccessible or opaque. This opacity is particularly problematic when the system makes consequential decisions without explaining its reasoning.

BLAS Basic Linear Algebra Subprograms, a specification for low-level routines that perform common linear algebra operations such as vector addition, scalar multiplication, dot products, and matrix operations, forming the computational foundation of modern ML frameworks.

block sparse matrix A sparse matrix whose nonzero values appear in dense blocks rather than isolated elements, preserving structure that hardware kernels can exploit efficiently.

blue-green deployment A deployment strategy using two production environments so traffic can switch to a validated new version while the old version remains available for rollback.

bounding box A rectangular annotation that identifies object locations in images by drawing a box around each object of interest, commonly used in computer vision training datasets.

brittleness The tendency of rule-based AI systems to fail completely when encountering inputs that fall outside their programmed scenarios, no matter how similar those inputs might be to what they were designed to handle.

C

caching A technique for storing frequently accessed data in high-speed storage systems to reduce retrieval latency and improve system performance in ML pipelines.

caching allocator A framework memory manager that reuses device-memory blocks instead of calling the hardware allocator for every tensor, reducing allocation latency and fragmentation.

calibration The process in post-training quantization of analyzing a representative dataset to determine optimal quantization parameters, including scale factors and zero points, that minimize accuracy loss when converting from high to low precision.

canary deployment A deployment strategy where new model versions receive a small percentage of production traffic to validate behavior before full rollout, enabling early detection of issues with minimal user impact.

CAP theorem A distributed systems principle stating that, during a network partition, a data store cannot guarantee both linearizable consistency and availability for every request.

carbon footprint The total greenhouse gas emissions, typically measured in CO_2 equivalent, produced directly and indirectly by training and operating an ML system.

carbon intensity The emissions produced per unit of energy, usually measured as kg CO_2e per kWh, used to convert ML energy use into carbon impact.

Cerebras Wafer-Scale Engine In the CS-2 generation, a single-wafer processor containing 2.6 trillion transistors and 850,000 cores, designed to eliminate inter-device communication bottlenecks in large-scale machine learning training.

chain rule The calculus rule enabling computation of derivatives for composite functions, fundamental to backpropagation.

channelwise quantization A quantization granularity approach where each channel in a layer uses its own set of quantization parameters, providing more precise representation than layerwise quantization while maintaining hardware efficiency.

CI/CD pipelines Continuous Integration and Continuous Delivery automated workflows that streamline model development by integrating testing, validation, and deployment processes.

circuit breaker pattern A resilience pattern that stops or redirects calls to an unhealthy service after repeated failures to prevent cascading failure and enable fallback behavior.

classification labels Simple categorical annotations that assign specific tags or categories to data examples, representing the most basic form of supervised learning annotation.

CloudSuite Benchmark suite developed at EPFL that addresses modern data center workloads including web search, data analytics, and media streaming, measuring end-to-end performance across network, storage, and compute dimensions.

cold-start performance Time required for a system to transition from idle state to active execution, particularly important in serverless environments where models are loaded on demand.

collaborative filtering A technique used in recommendation systems that predicts user preferences by identifying patterns in interactions from many users, using the collective behavior of the crowd rather than just item properties.

communication overhead The time a distributed system spends synchronizing state between nodes rather than computing, set by the latency and bandwidth of the links between them.

compound AI systems AI architectures that combine multiple specialized models and components working together, rather than relying on a single monolithic model, enabling modularity, specialization, and improved interpretability.

compressed sparse row (CSR) A memory-efficient storage format for sparse matrices that uses three arrays (values, column indices, row pointers) to store only nonzero elements, reducing memory from $\mathcal{O}(n_{\text{rows}}^2)$ (for a square matrix with n_{rows} rows) to

$\mathcal{O}(n_{\text{nz}} + n_{\text{rows}})$ where n_{nz} is the number of nonzeros.

computational graph A directed graph representing the sequence of operations in neural network computation, enabling automatic differentiation.

computer engineering An engineering discipline that emerged in the late 1960s to address the growing complexity of integrating hardware and software systems, combining expertise from electrical engineering and computer science to design and build complex computing systems.

concept drift The phenomenon where $p(y \mid x)$, the statistical relationship between inputs and targets, changes over time, distinct from data drift in $p(x)$; this can, but need not, degrade a fixed model's performance. Detecting it typically requires outcome labels because input drift alone does not establish a conditional change.

conditional computation A dynamic optimization technique where different parts of a neural network are selectively activated based on input characteristics, reducing computational load by skipping unnecessary computations for specific inputs.

consensus labeling A quality control approach that collects multiple annotations for the same data point to identify controversial cases and improve label reliability through inter-annotator agreement.

containerization Packaging applications and their dependencies into portable, isolated containers using tools like Docker to ensure consistent execution across different environments.

continuous batching An LLM serving technique that adds and removes requests from an inference batch between token-generation steps as sequences start and finish.

continuous integration A software development practice where code changes are automatically integrated, tested, and validated multiple times per day to detect issues early in the development cycle.

convolution A mathematical operation that slides a filter (kernel) across input data to extract features such as edges, textures, or patterns. Convolution is fundamental to convolutional neural networks and particularly effective for processing images and spatial data.

convolutional neural network A specialized neural network architecture designed for processing grid-like data such as images, using convolutional layers that apply filters to detect local features.

coreset A small subset of a dataset selected to preserve important statistical or training properties of the full dataset while reducing training or evaluation cost.

correction cascade An ML technical-debt pattern where fixing one component creates new failures or redesign needs elsewhere because data and model dependencies propagate through the system.

covariate shift A distribution shift where input feature distributions change while the relationship between features and labels remains stable.

CP decomposition CANDECOMP/PARAFAC decomposition that expresses a tensor as a sum of rank-one components, used to compress neural network layers by reducing the number of parameters while preserving computational functionality.

credit assignment problem The challenge of determining which weights in a multi-layer network contributed to prediction errors.

CRISP-DM Cross-Industry Standard Process for Data Mining, a structured methodology developed in 1996 that defines six phases for data projects: business understanding, data understanding, data preparation, modeling, evaluation, and deployment.

critical batch size The batch size beyond which larger batches provide diminishing optimization benefit and may reduce generalization, limiting simple linear learning-rate scaling.

cross-entropy loss A loss function commonly used in classification tasks that measures the difference between predicted probability distributions and true class labels, providing strong gradients for effective learning.

crowdsourcing A collaborative data collection approach that uses distributed individuals via the internet to perform annotation tasks, enabling scalable dataset creation through platforms like Amazon Mechanical Turk.

cuBLAS NVIDIA's CUDA Basic Linear Algebra Subprograms library that provides GPU-accelerated implementations of standard linear algebra operations, enabling high-performance matrix computations on NVIDIA graphics processing units.

CUDA (Compute Unified Device Architecture) NVIDIA's parallel computing platform and programming model that enables general-purpose computing on graphics processing units (GPUs), allowing machine learning frameworks to use massive parallelism for accelerated tensor operations.

CUDA stream An ordered queue of GPU work used to schedule kernels, memory transfers, and synchronization events so independent operations can overlap.

D

D·A·M taxonomy A diagnostic framework that analyzes ML system bottlenecks through three interacting axes: Data (information flow, bounded by bandwidth), Algorithm (mathematical logic, bounded by total operations), and Machine (physical execution, bounded by peak throughput).

Dartmouth conference The legendary 8-week workshop at Dartmouth College in 1956 where AI was officially born, organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon, following their 1955 proposal, which introduced the term artificial intelligence.

data as source code The Software 2.0 principle that training data defines ML system behavior, so dataset changes act like behavior-changing code changes.

data augmentation The process of artificially expanding training datasets by creating modified versions of existing data through transformations like rotation, scaling, or noise injection.

data cascades Systemic failures where data quality issues compound over time, creating downstream negative consequences such as model failures, costly rebuilding, or project termination.

data center A facility that houses computer systems and associated components such as telecommunications and storage systems, typically containing thousands of servers for cloud computing operations.

data-centric approach A machine learning paradigm that prioritizes improving data quality, diversity, and curation rather than solely focusing on model architecture improvements to achieve better performance.

data-centric computing Systems optimized for the efficient ingestion of data and iterative refinement of model parameters, where the programmer's job is to curate data.

data contracts Explicit agreements between data producers and consumers defining schema, quality expectations, and service levels to prevent downstream breakage.

data curation The process of selecting, organizing, and maintaining high-quality datasets by removing irrelevant information, correcting errors, and ensuring data meets specific standards for machine learning applications.

data debt Accumulated data quality, documentation, schema, lineage, or freshness liabilities that compound over time and degrade model behavior.

data drift The phenomenon where the input distribution $p(x)$ changes over time even though code remains unchanged; this may affect model performance but does not guarantee degradation. It can be monitored without labels, although labels are needed to measure its effect on quality.

data echoing A training input-pipeline technique that reuses batches, often with fresh augmentation, when data preparation is slower than accelerator consumption.

data governance The framework of policies, procedures, and technologies that ensure data security, privacy, compliance, and ethical use throughout the machine learning pipeline.

data gravity The architectural pull created by large data volumes, transfer time, bandwidth limits, and egress cost, which tends to draw computation toward the data.

data ingestion The process of collecting and importing raw data from various sources into a system where it can be stored, processed, and prepared for machine learning applications.

data lake A storage repository that holds structured, semi-structured, and unstructured data in its native format, using schema-on-read approaches for flexible data analysis.

data lakehouse A storage architecture that combines data lake flexibility with warehouse-style query semantics, schema management, and transactional guarantees.

data lineage The documentation and tracking of data flow through various transformations and processes, providing visibility into data origins and modifications for compliance and debugging.

data locality The principle that data and computation should be colocated when transfer latency, bandwidth, or cost dominates remote processing.

data parallelism A distributed training strategy that splits the dataset across multiple devices while each device maintains a complete copy of the model, enabling parallel computation of gradients.

data pipeline The infrastructure and workflows that automate the movement and transformation of data from sources through processing stages to final storage or consumption.

data quality The degree to which data meets requirements for accuracy, completeness, consistency, and timeliness, directly impacting machine learning model performance.

data quality multiplier The idea that improvements in data quality can amplify gains from model and system changes because all downstream stages depend on the data.

data validation The systematic verification that collected data meets quality standards, is properly formatted, and contains accurate information suitable for machine learning model training and evaluation.

data versioning The practice of tracking and managing different versions of datasets over time, similar to code versioning, to ensure

reproducibility and enable rollback to previous data states when needed.

data warehouse A centralized repository optimized for analytical queries (OLAP) that stores integrated, structured data from multiple sources in a standardized schema.

dataflow architecture Specialized computing architecture where instruction execution is determined by data availability rather than a program counter, enabling highly parallel processing of neural network operations.

dataflow challenges Technical difficulties in managing data movement and dependencies in hardware accelerators, including memory bandwidth limitations and synchronization requirements.

datasheets for datasets Documentation for training data that captures provenance, collection methodology, demographic composition, and known limitations affecting model behavior.

dead letter queue A separate storage mechanism for data that fails processing, allowing for later analysis and potential reprocessing of problematic data without blocking the main pipeline.

deduplication The removal of exact, near-duplicate, or semantically redundant samples from a dataset to reduce storage, training compute, and leakage risk.

deep learning A subfield of machine learning that uses artificial neural networks with multiple layers to automatically learn hierarchical representations from data without explicit feature engineering.

demographic parity A fairness criterion requiring that the probability of receiving a positive prediction is independent of group membership across protected attributes.

Dennard scaling The observation that as transistors became smaller, their power density remained constant, enabling higher performance at the same power envelope. Its breakdown around 2005 ended the era of free frequency scaling.

dense layer A fully-connected neural network layer where each neuron receives input from all neurons in the previous layer, enabling comprehensive information integration across features.

deployment constraints Operational limitations such as hardware resources, network connectivity, regulatory requirements, and integration requirements that influence how machine learning models are implemented in production environments.

deployment paradigm A distinct approach to hosting and executing ML models characterized by specific resource constraints and operational properties, such as Cloud ML, Edge ML, Mobile ML, or TinyML.

depthwise separable convolution A convolution factorization that applies spatial filtering independently per channel and then mixes channels with a pointwise convolution to reduce computation.

DevOps Software development practice that combines development and operations teams to shorten development cycles and deliver high-quality software through automation and collaboration.

Dhrystone Integer-based benchmark introduced in 1984 that measures integer and string operations in DMIPS (Dhrystone MIPS), designed to complement floating-point benchmarks with typical programming constructs.

diabetic retinopathy A diabetes complication that damages blood vessels in the retina, serving as a leading cause of preventable blindness and a key application area for medical AI screening systems.

differential privacy A mathematical framework for quantifying and limiting privacy loss when releasing statistical information about datasets by bounding how much the output distribution can change when any one individual's data is added or removed.

direct memory access (DMA) A mechanism that lets a device move data to or from memory without CPU-managed copying for each transfer, enabling overlap and reducing host overhead.

disaggregated evaluation The practice of breaking down model performance metrics by demographic groups or other factors to reveal disparities that are hidden by aggregate measures.

dispatch overhead Runtime overhead from launching and orchestrating many small operations through the host framework instead of executing fused or compiled graph regions.

distributed computing An approach that processes data across multiple machines or processors simultaneously, enabling scalable handling of large datasets through frameworks like Apache Spark.

distributed intelligence The placement of computational capabilities across multiple devices and locations rather than relying on a single centralized system, enabling local processing and decision-making.

distributed training A method of training machine learning models across multiple machines or devices to handle larger datasets and models that exceed single-device computational or memory capacity.

distribution shift A change in the statistical properties of data between training and deployment, or over time during deployment. Types include covariate shift (input distribution changes), label shift (output distribution changes), and concept drift (relationship between inputs and outputs changes).

DLRM Deep Learning Recommendation Model, an architecture developed by Meta that combines categorical embeddings with a bottom multi-layer perceptron (MLP) and top MLP to handle the massive scale and sparsity of recommendation system workloads.

domain-specific architecture Hardware designs tailored to optimize specific computational workloads, trading flexibility for improved performance and energy efficiency compared to general-purpose processors.

dropout A regularization technique that randomly sets a fraction of input units to zero during training to prevent overfitting and improve generalization.

dying ReLU problem A failure mode where ReLU neurons become permanently inactive and output zero for all inputs, preventing them from contributing to learning when weighted inputs consistently produce negative preactivations.

dynamic batching A serving technique that waits briefly to group independent requests into a batch, improving throughput while adding controlled batch-formation latency.

dynamic graph A computational graph that is built and modified during program execution, allowing for flexible model architectures and easier debugging but potentially limiting optimization opportunities compared to static graphs.

dynamic pruning A pruning approach that changes which parameters or units are inactive in response to input data or training dynamics, rather than relying only on a fixed pruning decision.

dynamic quantization The process of storing neural-network weights at lower precision and determining activation quantization parameters at runtime, reducing memory use while requiring supported low-precision kernels for speedup.

dynamic random-access memory (DRAM) A type of volatile memory that stores data in capacitors and requires periodic refresh cycles, commonly used as main memory in computer systems.

dynamic voltage and frequency scaling (DVFS) Power management technique that adjusts processor voltage and clock frequency based on workload demands to optimize energy consumption while maintaining performance.

E

eager execution An execution mode where operations are evaluated immediately as they are called in the code, providing intuitive debugging and development experience but potentially sacrificing some optimization opportunities available in graph-based execution.

early exit architectures Neural network designs that include multiple prediction heads at different depths, allowing samples to exit early when confident predictions can be made, reducing average computational cost per inference.

edge computing A distributed computing paradigm that brings computation and data storage closer to the sources of data, reducing latency and bandwidth usage.

edge deployment A deployment strategy where machine learning models run locally on devices at the network edge rather than in centralized cloud servers, reducing latency and enabling operation without constant internet connectivity.

efficiency frontier The optimal trade-off curve between accuracy and computational cost (latency, energy, or memory), where no model exists that is both more accurate and more efficient.

EfficientNet A family of neural network architectures discovered through neural architecture search that achieves better accuracy-efficiency trade-offs by using compound scaling to balance network depth, width, and input resolution.

EL2N (Error L2-Norm) A sample-scoring method that ranks examples by the L2 norm of prediction error early in training, often using a proxy model, to guide data selection.

ELIZA One of the first chatbots created by MIT's Joseph Weizenbaum in 1966 that could simulate human conversation through pattern matching and substitution, notable because people began forming emotional attachments to this simple program.

embedded systems Computer systems with dedicated functions within larger mechanical or electrical systems, typically designed for specific tasks with real-time computing constraints.

embedding sharding Splitting large embedding tables across devices or nodes so they fit in aggregate memory, while adding communication to gather embeddings for each batch.

embedding table A lookup table that maps discrete IDs or tokens to dense vectors, often dominating memory in recommendation models and large vocabularies.

emergent behaviors Unexpected system-wide patterns or characteristics that arise from the interaction of individual components, often becoming apparent only when systems operate at scale or in real-world conditions.

encoder-decoder An architectural pattern where an encoder processes input into an intermediate representation and a decoder generates output from this representation, commonly used in sequence-to-sequence tasks.

end-to-end benchmarks Comprehensive evaluation methodology that assesses entire AI system pipelines including data processing, model execution, postprocessing, and infrastructure components.

energy efficiency The measure of computational work performed per unit of energy consumed, typically expressed as operations per joule and crucial for battery-powered and data center deployments.

energy per inference The total energy consumed to produce one model prediction, including model execution and relevant system overhead.

epoch One complete pass through the entire training dataset during neural network training, consisting of multiple batch iterations depending on dataset size and batch size.

equal opportunity A fairness criterion requiring equal true positive rates across different demographic groups.

equalized odds A fairness criterion requiring that both true positive and false positive rates are equal across different demographic groups.

expected calibration error (ECE) A metric measuring the gap between predicted confidence and observed accuracy across confidence bins, used to assess probability calibration.

experiment tracking The systematic recording and management of machine learning experiments, including hyperparameters, model versions, training data, and performance metrics, to enable comparison and reproducibility.

expert systems AI systems first developed in the mid-1960s that captured human expert knowledge in specific domains, exemplified by MYCIN for diagnosing blood infections, representing a shift from general AI to domain-specific applications.

extract, load, transform (ELT) A data processing paradigm that first loads raw data into the target system before applying transformations, providing flexibility for evolving analytical needs.

extract, transform, load (ETL) A traditional data processing paradigm that transforms data before loading it into a data warehouse, resulting in ready-to-query formatted data.

F

fairness laundering A failure mode where removing explicit protected attributes makes a system appear compliant while correlated proxy variables still reproduce discriminatory outcomes.

false positive rate (FPR) The proportion of actual negative cases incorrectly classified as positive, computed as false positives divided by false positives plus true negatives.

FarmBeats A Microsoft Research project that applies machine learning and IoT technologies to agriculture, using edge computing to collect real-time data on soil conditions and crop health while demonstrating distributed AI systems in challenging real-world environments.

feature engineering The process of manually designing and extracting relevant features from raw data to improve machine learning model performance, largely automated in deep learning systems.

feature map The output of a convolutional layer representing the response of learned filters to different spatial locations in the input, capturing detected features at various positions.

feature store A specialized data storage system that provides standardized, reusable features for machine learning, enabling feature sharing across multiple models and teams.

federated learning A machine learning approach that trains algorithms across decentralized edge devices or servers holding local data samples, without exchanging the raw data.

feedback loop A cyclical process where outputs influence future inputs. In ML, this includes both beneficial cycles (where model outputs inform system improvement) and harmful ones (where a model's predictions influence its own future training data, potentially reinforcing and amplifying initial biases over time).

feedforward network A neural network architecture where information flows in one direction from input to output layers without cycles, forming the foundation for many deep learning models.

field-programmable gate array (FPGA) A reconfigurable integrated circuit that can be programmed after manufacturing to implement custom digital circuits and specialized computations, offering flexibility between general-purpose processors and ASICs.

fine-tuning Adapting a pretrained model to a target task or domain by continuing training on task-specific data.

FlashAttention An IO-aware attention algorithm that tiles computation to avoid materializing the full attention matrix in high-bandwidth memory, reducing memory traffic and enabling longer sequences.

floating-point unit (FPU) A specialized processor component designed to perform arithmetic operations on floating-point numbers with high precision and efficiency.

FLOP/s Floating-Point Operations per Second, a measure of computational throughput that quantifies the number of floating-point operations a system can perform per second.

forgetting events Events where a model changes from classifying an example correctly to misclassifying it later in training, marking difficult or informative samples.

forward pass The process of computing neural network predictions by passing input data through successive layers, applying weights, biases, and activation functions at each stage to produce outputs. Also called *forward propagation*.

foundation model Large-scale machine learning models trained on broad data that can be adapted to a wide range of downstream tasks, serving as a base for specialized applications.

four pillars framework A data engineering framework organizing concerns into Quality, Reliability, Scalability, and Governance to manage the data lifecycle.

FP16 16-bit floating-point numerical representation that reduces memory usage and accelerates computation on modern hardware accelerators while maintaining acceptable precision for many machine learning applications.

FP32 32-bit floating-point numerical representation that provides standard precision for mathematical computations but requires more memory and computational resources than lower-precision formats.

FP32 to INT8 A common quantization transformation that converts 32-bit floating-point weights and activations to 8-bit integers, achieving roughly 4× memory reduction while maintaining acceptable accuracy for many models.

FP8 An 8-bit floating-point format family used on newer accelerators for high-throughput training and inference, requiring careful scaling because of limited precision and range.

framework decomposition The systematic breakdown of neural network frameworks into hardware-mappable components, enabling efficient distribution of operations across processing elements.

G

GEMM General Matrix Multiply operations that follow the pattern $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$, representing the fundamental computational kernel underlying most neural network operations including fully connected layers and convolutional layers.

GEMV General Matrix-Vector multiplication operations that compute the product of a matrix and a vector, commonly used in neural network computations and requiring careful optimization for memory access patterns.

generalization The ability of a machine learning model to perform well on unseen data that differs from the training set, often improved through diverse and high-quality training data.

generalization gap The difference between a model's performance on training data and its performance on held-out data. A large generalization gap can indicate overfitting to the training data.

generative AI A category of artificial intelligence systems capable of creating new content such as text, images, audio, or video based on learned patterns from training data.

Goodhart's Law The observation that when a measure becomes an optimization target, it can stop representing the underlying goal.

GPT-2 A 1.5-billion parameter autoregressive language model released by OpenAI in 2019, serving as a primary lighthouse model for analyzing memory bandwidth constraints in transformer inference.

graceful degradation A system design principle where services continue functioning with reduced capabilities when faced with partial failures or data unavailability.

gradient accumulation A technique that simulates larger batch sizes by accumulating gradients from multiple smaller batches before updating model parameters, enabling training with limited memory.

gradient-based pruning A pruning method that uses gradient information to rank parameters, neurons, or filters as candidates for removal.

gradient clipping An optimization technique that mitigates exploding gradients by limiting their magnitude during backpropagation, typically by scaling gradients when their norm exceeds a threshold.

gradient compression A technique used in distributed training to reduce the communication overhead by compressing gradient information exchanged between computing nodes.

gradient descent An optimization algorithm that iteratively adjusts neural network parameters in the direction that minimizes the loss function, using gradients to determine update directions and magnitudes.

gradient synchronization The process in distributed training where locally computed

gradients are aggregated across devices to ensure all devices update their parameters consistently.

GraNd (Gradient Normed) A data selection score based on the norm of an example's gradient during early training, identifying samples with larger learning signal.

graph break A point where a compiler cannot capture part of a program and must fall back to eager execution, splitting optimized graph regions.

graphics processing unit (GPU) A specialized processor originally designed for rendering graphics that provides massive parallel computing capabilities essential for efficient neural network computation, training, and inference.

Green AI A movement in AI research and practice that prioritizes computational efficiency and energy consumption as primary metrics alongside traditional performance metrics like accuracy.

Green500 Ranking system that evaluates the world's most powerful supercomputers based on energy efficiency measured in FLOP/s per watt rather than raw computational performance.

ground truth The reference target treated as correct for training or evaluation, often derived from measurement or annotation and therefore not necessarily identical to objective reality.

GRU Gated Recurrent Unit, a simplified variant of LSTM that uses fewer gates while maintaining the ability to capture long-term dependencies in sequential data.

H

hardware abstraction The layer in ML frameworks that provides a unified interface to diverse computing hardware (CPUs, GPUs, TPUs, accelerators) while handling device-specific optimizations and memory management behind the scenes.

hardware acceleration The use of specialized computing hardware to perform certain operations faster and more efficiently than software running on general-purpose processors.

hardware accelerator Specialized computing hardware designed to efficiently execute specific types of computations, such as GPUs for parallel processing or TPUs for machine learning workloads.

hardware-aware design The practice of designing neural network architectures specifically optimized for target hardware platforms, considering factors like memory hierarchy, compute units, and data movement patterns to maximize efficiency.

hidden layer An intermediate layer in a neural network between input and output layers that learns abstract representations by transforming data through learned weights and activation functions.

hidden state The internal memory of recurrent neural networks that carries information from previous time steps, enabling the network to maintain context across sequential inputs.

hierarchical processing A multi-tier system architecture where data and intelligence flow between different levels of the computing stack, from sensors to edge devices to cloud systems.

high-bandwidth memory (HBM) An advanced memory technology that provides much higher bandwidth than traditional DRAM by using 3D stacking and wide interfaces, critical for data-intensive AI workloads.

horizontal scaling Increasing system capacity by adding more machines or instances rather than upgrading existing hardware, providing better fault tolerance and load distribution.

hybrid machine learning The integration of multiple ML paradigms such as cloud, edge, mobile, and TinyML to form unified distributed systems that combine complementary strengths.

hybrid parallelism A distributed training approach that combines data parallelism and model parallelism to use the benefits of both strategies for training very large models.

hyperparameter A configuration setting that controls the learning process but is not learned from data, such as learning rate, batch size, or network architecture choices.

hyperscale data center Large-scale data center facilities containing thousands of servers and covering extensive floor space, designed to efficiently support massive computing workloads.

I

I/O bottleneck A performance limit where storage, decoding, preprocessing, or data transfer cannot supply data fast enough to keep compute resources busy.

idempotent transformation A data transformation that has the same effect when applied repeatedly as when applied once, supporting safe retries and reprocessing.

im2col A convolution-lowering technique that unfolds image patches into matrix columns so convolution can execute as GEMM.

ImageNet A massive visual database containing over 14 million labeled images across 20,000+ categories, developed by a team led by Fei-Fei Li and introduced in 2009, whose annual challenge became instrumental in driving breakthrough advances in computer vision.

imperative programming A programming paradigm in which programs specify sequences of commands that update state and direct control flow; it is distinct from whether operations execute eagerly or are staged for later execution.

inductive bias A structural assumption built into a model or architecture that restricts the functions it can learn efficiently, such as locality in CNNs.

inference The operational phase where trained neural networks make predictions on new data using fixed parameters, without weight updates.

InfiniBand A high-throughput, low-latency networking technology commonly used for multi-node accelerator clusters and distributed training.

information-compute ratio (ICR) A metric quantifying the efficiency of data selection, defined as the ratio of model performance gain to the computational cost (FLOPs) required to achieve it. Maximizing ICR is the primary goal of data selection strategies.

infrastructure as code Practice of managing and provisioning computing infrastructure through machine-readable configuration files rather than manual processes, enabling version control and automation.

INT8 An 8-bit signed-integer representation used to quantize weights, activations, or both. Each INT8 value occupies one quarter the storage of an FP32 value, and supported hardware can accelerate applicable operations, with accuracy depending on the quantization scheme.

intermediate representation (IR) A compiler-internal representation between frontend model code and backend hardware code generation, used by ML frameworks for optimization and portability.

internet of things A network of physical objects embedded with sensors, software, and other technologies that connect and exchange data with other devices and systems over the internet.

intersectional analysis Evaluation that considers combinations of demographic attributes (for example, race and gender simultaneously) to detect concentrated harms not visible in single-factor analysis.

IOPS Input/Output Operations Per Second; a storage performance metric measuring the number of read/write operations a device can handle per second, critical for random access workloads like training data loading.

Iron Law of ML Systems A quantitative framework relating workload execution time to the D·A·M taxonomy and hardware bottlenecks. Across the D·A·M axes, time factors multiplicatively as $T = (\text{Samples/Task}) \times (\text{FLOPs/Sample}) \times (\text{Seconds/FLOP})$, where the first two factors set total operations O; on physical hardware, execution unpacks into Data Movement (limited by bandwidth), Computation (limited by sustained FLOP/s), and Latency (limited by fixed overhead) as $T = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$.

iterative pruning A gradual pruning strategy that removes parameters in multiple stages with fine-tuning between each stage, allowing the model to adapt to reduced capacity and typically achieving better accuracy than one-shot pruning.

J

JAX A numerical computing library developed by Google Research that combines NumPy's API with functional programming transformations including automatic differentiation, just-in-time compilation, and automatic vectorization for high-performance machine learning research.

JIT compilation Just-In-Time compilation that analyzes and optimizes code at runtime, enabling frameworks to balance the flexibility of eager execution with the performance benefits of graph optimization by compiling frequently used functions.

K

k-anonymity A privacy technique that ensures each record in a dataset is indistinguishable from at least $k_{\text{anon}} - 1$ other records by generalizing quasi-identifiers.

kernel A small matrix of learnable weights used in convolutional layers to detect specific features through the convolution operation, also called a filter. In GPU programming, the term also denotes a function launched for parallel execution by a grid of threads.

kernel fusion An optimization technique that combines multiple computational operations into a single kernel to reduce memory transfers and improve performance on parallel processors.

keyword spotting (KWS) A technology that detects specific wake words or phrases in audio streams, typically used in voice-activated devices with constraints on power consumption and latency.

knowledge distillation A model compression technique where a smaller "student" network learns to mimic the behavior of a

larger "teacher" network by training on the teacher's soft output probabilities rather than just hard labels.

Kolmogorov-Smirnov test (K-S test) A nonparametric test that measures the maximum distance between two cumulative distributions, often used to compare feature distributions for drift.

Kullback-Leibler divergence (KL divergence) An asymmetric information-theoretic measure of how one probability distribution differs from another, used in monitoring and drift analysis.

KV cache Stored key and value tensors from previous transformer tokens used during autoregressive inference to avoid recomputing attention history.

L

L0 norm The count of nonzero elements in a vector or tensor, used in pruning to express sparsity or a target budget for remaining weights.

label quality drift Degradation in the reliability or consistency of labels over time, even when the underlying input distribution remains stable.

label shift A distribution shift where label frequencies change while the feature patterns conditioned on labels remain stable.

LAPACK Linear Algebra Package that extends BLAS with higher-level linear algebra operations including matrix decompositions, eigenvalue problems, and linear system solutions, providing essential mathematical foundations for machine learning computations.

latency The time delay between a request or event and the corresponding response or completion, critical in real-time applications where prompt responses are required.

latency constraints Real-time requirements that limit the maximum acceptable delay for model inference, driving optimization decisions in deployment scenarios where response time is critical.

layer normalization A normalization technique that normalizes inputs across the features dimension for each sample, commonly used in transformer architectures to stabilize training.

layerwise quantization A quantization granularity where all parameters within a single layer share the same quantization parameters, providing computational efficiency but potentially limiting representational precision compared to finer-grained approaches.

learning rate A hyperparameter that determines the step size for weight updates during gradient descent optimization, critically affecting training stability and convergence speed.

learning rate scheduling The systematic adjustment of learning rates during training, using strategies like step decay, exponential decay, or cosine annealing to improve convergence and final model performance.

lifecycle coherence The principle that all stages of ML development should align with overall system objectives, maintaining consistency in data handling, model architecture, and evaluation criteria.

linear scaling failure The phenomenon where increasing computing resources (for example, doubling accelerators) results in sublinear performance gains due to communication overhead and synchronization costs.

LINPACK Benchmark developed at Argonne National Laboratory that measures system performance by solving dense systems of linear equations, famous for its use in TOP500 supercomputer rankings.

Little's Law A queuing relationship stating that average concurrency equals arrival rate times average time in the system, linking throughput, latency, and capacity.

Llama Large Language Model Meta AI, a family of open foundation models that popularized efficient architectural choices like RMSNorm and SwiGLU, serving as a modern reference point for large-scale transformer efficiency.

load balancing Distribution of incoming requests across multiple server instances to prevent bottlenecks, improve response times, and ensure high availability.

locality-sensitive hashing (LSH) A hashing family that maps similar items to the same buckets with high probability, enabling scalable approximate similarity search.

logits The raw, unnormalized scores output by the last layer of a neural network before the activation function (like softmax) converts them into probabilities.

loss function A mathematical function that measures how well model outputs satisfy a training objective, providing the objective minimized by the training algorithm.

loss scaling A technique used in mixed-precision training that multiplies the loss by a large factor before backpropagation to prevent gradient underflow in reduced precision formats.

Lottery Ticket Hypothesis The theory that large neural networks contain sparse subnetworks that, when trained in isolation from proper initialization, can achieve comparable accuracy to the full network while being significantly smaller.

low-rank factorization A matrix decomposition technique that approximates large weight matrices as products of smaller matrices, reducing the number of parameters and computational operations required for neural network layers.

LSTM Long Short-Term Memory, a type of recurrent neural network architecture designed to handle long-term dependencies through gating mechanisms that control information flow.

M

machine learning A subset of artificial intelligence that enables systems to automatically improve performance on tasks through experience and data rather than explicit programming.

machine learning framework A software platform that provides tools and abstractions for designing, training, and deploying machine learning models, bridging user applications with infrastructure through computational graphs, hardware optimization, and workflow orchestration. Examples include TensorFlow, PyTorch, and JAX.

machine learning lifecycle A structured, iterative process that encompasses all stages involved in developing, deploying, and maintaining machine learning systems, from problem definition through ongoing monitoring and improvement.

machine learning operations (MLOps) The engineering discipline and set of tools focused on deploying and operating machine learning models through automation, monitoring, and management of the entire ML pipeline from development to production.

machine learning systems engineering The engineering discipline focused on building reliable, efficient, and scalable AI systems across computational platforms, spanning the entire AI lifecycle from data acquisition through deployment and operations with emphasis on resource-awareness and system-level optimization.

macro benchmarks Evaluation methodology that assesses complete machine learning models to understand how architectural choices and component interactions affect overall system behavior and performance.

magnitude-based pruning The most common pruning method that removes parameters with the smallest absolute values, based on the assumption that weights with smaller magnitudes contribute less to the model's output.

masking An anonymization technique that alters or obfuscates sensitive values so they cannot be directly traced back to the original data subject.

membership inference attack A privacy attack that tries to determine whether a specific record was included in a model's training data.

memory bandwidth Rate at which data can be read from or written to memory, measured in bytes per second, which often becomes a bottleneck in memory-intensive machine learning workloads.

memory hierarchy The organization of memory systems with different access speeds and capacities, from fast on-chip caches to slower off-chip main memory.

memory wall The widening performance gap between processor speed and memory bandwidth, where computational capacity outpaces the rate at which data can be delivered to the processor, becoming a primary bottleneck for large ML models.

metadata Descriptive information about datasets that includes details about data collection, quality metrics, validation status, and other contextual information essential for data management.

micro-batch A smaller slice of an effective batch processed sequentially for gradient accumulation or pipeline parallelism to reduce peak memory.

micro benchmarks Specialized evaluation tools that assess individual components or specific operations within machine learning systems, such as tensor operations or neural network layers.

microcontroller A small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals, commonly used in embedded systems.

MinHash A sketching method that approximates set similarity with compact signatures, commonly used for scalable near-duplicate detection.

mini-batch gradient descent A training approach that computes gradients and updates weights using a small subset of training examples simultaneously, balancing computational efficiency with gradient estimation quality.

mini-batch processing An optimization approach that computes gradients over small batches of examples, balancing the computational efficiency of batch processing with the memory constraints of stochastic methods.

mixed-precision computing A technique that uses different numerical precisions at various stages of computation, such as FP16 for matrix multiplications and FP32 for accumulations.

mixed-precision training A training methodology that combines different numerical precisions (typically FP16 and FP32) to optimize memory usage and computational speed while maintaining training stability.

ML node A single production ML application treated as an operational unit, including data pipelines, feature computation, model training, serving infrastructure, and monitoring.

ML systems Integrated computing systems comprising three core components: data that guides algorithmic behavior, learning algorithms that extract patterns from data, and computing infrastructure that enables both training and inference processes.

ML systems spectrum The range of machine learning system deployments from cloud-based systems with abundant resources to tiny embedded devices with severe constraints, each requiring different optimization strategies and trade-offs.

ML Test Score A production-readiness rubric for ML systems that evaluates data, model, infrastructure, and monitoring tests.

MLCommons Organization that develops and maintains industry-standard benchmarks for machine learning systems, including the MLPerf suite for training and inference evaluation.

MLPerf Industry-standard benchmark suite that provides standardized tests for training and inference across various deep learning workloads, enabling fair comparisons of machine learning systems.

MLPerf execution scenarios Standard MLPerf inference traffic scenarios that model sequential, synchronized, server, batch, and interactive workloads with different latency and throughput requirements.

MLPerf Inference Benchmark framework that evaluates machine learning inference performance across different deployment environments, from cloud data centers to mobile devices and embedded systems.

MLPerf Mobile Specialized benchmark that extends MLPerf evaluation to smartphones and mobile devices, measuring latency and responsiveness under strict power and memory constraints.

MLPerf Power An MLPerf methodology for measuring power and energy efficiency of ML workloads alongside performance.

MLPerf Tiny Benchmark designed for embedded and ultra-low-power AI systems such as IoT devices, wearables, and microcontrollers operating with minimal processing capabilities.

MLPerf Training Standardized benchmark that evaluates machine learning training performance by measuring time-to-accuracy, throughput, and resource utilization across different hardware platforms.

mobile machine learning The execution of machine learning models directly on portable, battery-powered devices like smartphones and tablets, enabling personalized and responsive applications.

model cards A standardized format for documenting machine learning models, capturing information essential for responsible deployment, including intended use, performance factors, and ethical considerations.

model-centric AI A research paradigm where the dataset is treated as fixed and engineering effort focuses on optimizing model architecture.

model compression Techniques used to reduce the size and computational requirements of machine learning models while seeking to preserve accuracy, enabling deployment on resource-constrained devices.

model deployment The process of integrating trained machine learning models into production systems where they can make predictions on new data and provide value to end users.

model drift The degradation of machine learning model performance over time due to changes in data patterns, user behavior, or environmental conditions that differ from the original training conditions.

model evaluation The systematic assessment of machine learning model performance using various metrics and validation techniques to determine whether the model meets requirements and is ready for deployment.

model FLOPs utilization (MFU) The ratio of the model FLOPs per second implied by observed throughput to the hardware's theoretical peak FLOPs per second, measuring how effectively a workload uses available compute for model computation.

model optimization The systematic refinement of machine learning models to enhance their efficiency while maintaining effectiveness, balancing trade-offs between accuracy, computational cost, memory usage, latency, and energy efficiency.

model parallelism A distributed training strategy that splits a neural network model across multiple devices, with each device responsible for computing a portion of the network.

model quantization The process of reducing numerical precision in machine learning models, typically from FP32 to INT8, to decrease model size and memory traffic; speed increases only with efficient low-precision kernels.

model registry Centralized repository for storing, versioning, and managing trained machine learning models with associated metadata to support model governance and deployment.

model serving Infrastructure and systems that expose deployed machine learning models through APIs to handle prediction requests at scale with appropriate latency and throughput.

model training The process of using machine learning algorithms to learn patterns from training data, adjusting model parameters to minimize prediction errors and create a functional predictive system.

model validation The process of testing machine learning models on independent datasets to assess their generalization ability and ensure they perform reliably on unseen data.

model versioning The systematic tracking and management of different versions of machine learning models, including their parameters, training data, and performance metrics, to enable comparison and rollback capabilities.

momentum An optimization technique that accumulates a velocity vector across iterations to help gradient descent navigate through local minima and accelerate convergence in consistent gradient directions.

monitoring The continuous observation and measurement of machine learning system performance, data quality, and operational metrics in production to detect issues and trigger maintenance actions.

multi-head attention An attention mechanism that uses multiple parallel attention heads, each focusing on different aspects of the input to capture diverse types of relationships simultaneously.

multilayer perceptron (MLP) A feedforward neural network with one or more hidden layers between input and output layers, capable of learning nonlinear mappings through dense connections and activation functions.

multiply-accumulate (MAC) The operation of multiplying two values and accumulating the result into a running sum, dominating linear layers and accelerator datapaths.

MYCIN One of the first large-scale expert systems developed at Stanford beginning in 1972 to diagnose blood infections, representing the shift toward capturing human expert knowledge in specific domains rather than pursuing general artificial intelligence.

N

N:M sparsity A structured sparsity pattern where exactly N of every M values are nonzero, giving hardware more regular zero patterns to exploit.

neural architecture search An automated approach that searches possible combinations of layers, connections, and architectural hyperparameters to find neural network architectures that perform well under specified objectives and constraints.

neural network A computational model consisting of interconnected nodes organized in layers that can learn to map inputs to outputs through adjustable connection weights.

neural processing unit (NPU) Specialized processors designed specifically for accelerating neural network operations and machine learning computations, optimized for parallel processing of AI workloads.

neuromorphic computing A computing approach that mimics the structure and function of biological neural networks, potentially offering more energy-efficient processing for AI applications.

NoSQL A category of database systems designed to handle large volumes of unstructured or semi-structured data with flexible schemas, often used in big data applications.

numerical precision optimization The dimension of model optimization that addresses how numerical values are represented and processed, including quantization techniques that map high-precision values to lower-bit representations.

NVLink NVIDIA's high-bandwidth GPU interconnect for intra-node communication, useful for gradient synchronization and model-parallel activation transfers.

O

observability Comprehensive monitoring approach that provides insight into system behavior through metrics, logs, and traces, enabling understanding of internal states from external outputs.

on-chip memory Fast memory integrated directly onto the processor chip, including caches and scratchpad memory, providing high bandwidth and low latency data access.

on-device learning The capability for machine learning models to adapt and learn directly on edge devices without requiring data transmission to external servers.

one-hot encoding A representation where categorical labels become vectors with a single 1 and remaining 0s.

one-shot pruning A pruning strategy where a large fraction of parameters is removed in a single step, typically followed by fine-tuning to recover accuracy, offering simplicity but potentially requiring more aggressive fine-tuning.

online analytical processing (OLAP) A database approach optimized for complex analytical queries across large datasets, typically used in data warehouses for business intelligence.

online inference Real-time prediction serving that processes individual requests with low

latency, suitable for interactive applications requiring immediate responses.

online transaction processing (OLTP) A database approach optimized for frequent, short transactions and real-time processing, commonly used in operational applications.

ONNX Open Neural Network Exchange, a standardized format for representing machine learning models that enables interoperability between different frameworks, allowing models trained in one framework to be deployed using another.

ONNX Runtime Cross-platform inference engine that optimizes machine learning models through techniques like operator fusion and kernel tuning to improve inference speed and reduce computational overhead.

optimizer An algorithm that adjusts model parameters during training to minimize the loss function, with common examples including SGD (Stochastic Gradient Descent), Adam, and RMSprop, each with different strategies for parameter updates.

optimizer state Auxiliary values maintained by an optimizer in addition to model parameters, such as Adam's momentum and variance tensors, often dominating training memory for large models.

orchestration Coordination and management of complex workflows and distributed computing tasks, often using platforms like Kubernetes for container management.

outlier detection The process of identifying data points that significantly deviate from normal patterns, which may represent errors, anomalies, or valuable rare events.

overfitting A phenomenon where a model learns specific details of training data so well that it fails to generalize to new, unseen examples, typically indicated by high training accuracy but poor validation performance.

P

padding A technique in convolutional networks that adds zeros or other values around the input borders to control the spatial dimensions of the output feature maps.

PagedAttention A KV-cache memory-management technique that stores attention context in fixed-size pages rather than contiguous per-request blocks to reduce fragmentation.

paradigm shift A structural change in scientific approach, like the shift from symbolic reasoning to statistical learning in AI during the 1990s, and from shallow to deep learning in the 2010s, requiring researchers to abandon established methods for markedly different approaches.

parallelism The simultaneous execution of multiple computational tasks or operations, central to achieving high performance in neural network processing.

parameter A learnable component of a neural network, including weights and biases, that gets adjusted during training to minimize the loss function.

Pareto frontier The set of configurations where improving one objective requires worsening another, used to reason about accuracy, latency, memory, and energy trade-offs.

partitioning A database technique that divides large datasets into smaller, manageable segments based on specific criteria to improve query performance and system scalability.

perceptron The basic building block of neural networks, consisting of weighted inputs, a bias term, and an activation function that produces a single output.

performance insights Analytical observations derived from monitoring production machine learning systems that reveal opportunities for improvement in model accuracy, system efficiency, or user experience.

performance-per-data (PPD) A metric measuring the accuracy gain per training sample, used to evaluate the quality of a dataset or selection strategy. High PPD indicates a dataset rich in information and low in redundancy.

pinned memory Page-locked host memory that enables direct, asynchronous transfers between CPU and GPU, improving data-loading throughput at the cost of reserving system memory.

pipeline jungle Anti-pattern where complex, interdependent data processing pipelines become difficult to maintain, debug, and modify, leading to technical debt and operational complexity.

pipeline parallelism A form of model parallelism where different layers of a model are placed on different devices and data flows through them in a pipeline fashion, allowing multiple batches to be processed simultaneously.

point-in-time correctness The guarantee that training examples use only feature values that would have been available at the historical prediction time, preventing leakage.

pooling A downsampling operation in convolutional networks that reduces spatial dimensions while retaining important features, commonly using max or average operations over local regions.

population stability index (PSI) A binned distribution-shift metric that compares current feature distributions against a baseline to detect population drift.

positional encoding A method used in transformer architectures to inject information about the position of tokens in a sequence, since transformers lack inherent sequential processing.

post-training quantization A quantization approach applied to already-trained models without modifying the training process, typically involving calibration on representative data to determine optimal quantization parameters.

power usage effectiveness (PUE) Metric used in data centers to measure energy efficiency, calculated as the ratio of total facility power consumption to IT equipment power consumption.

power wall The technological barrier reached around 2005 where increasing processor frequency no longer yielded performance gains without unsustainable increases in power density and heat generation, forcing a shift to parallel and specialized architectures.

precision In numerical computing, the number of significant bits or digits used to represent values, affecting computational accuracy and resource requirements in machine learning systems.

prefetching A system optimization technique that loads data into memory before it is needed, overlapping data loading with computation to reduce idle time and improve training throughput.

problem definition The initial stage of machine learning development that involves clearly specifying objectives, constraints, success metrics, and operational requirements to guide all subsequent development decisions.

processing element (PE) A repeated accelerator building block containing compute units and local storage, with arrays of PEs providing parallel tensor execution.

programmable logic controller (PLC) Industrial control systems used in manufacturing and IoT environments that can be integrated with ML models for automated decision-making in operational technology contexts.

protein folding problem The scientific challenge of predicting the three-dimensional structure and dynamics of proteins from their amino acid sequences and environments, a longstanding problem on which systems such as AlphaFold achieved breakthrough accuracy without resolving every folding question.

proxy variable A feature that indirectly carries signal about a protected or sensitive attribute even when that attribute is removed.

pseudonymization A privacy technique that replaces direct identifiers with artificial identifiers while maintaining the ability to trace records for analysis purposes.

PyTorch A deep learning framework developed by Facebook's AI Research lab that emphasizes dynamic computational graphs, eager execution, and intuitive Python integration, particularly popular for research and experimentation.

Q

quantization A model-compression technique that represents parameters or activations at lower precision, such as FP32 to INT8, reducing storage and data movement without reducing mathematical operation count; speedups require supported hardware and kernels.

quantization-aware training A training approach where quantization effects are simulated during the training process, allowing the model to adapt to reduced precision and typically achieving better accuracy than post-training quantization.

quantization granularity The level at which quantization parameters are applied, ranging from per-tensor (coarsest) to per-channel or per-group (finer), with finer granularity typically preserving more accuracy but requiring more storage.

queries per second (QPS) Performance metric that measures how many inference requests a system can process in one second, commonly used to evaluate throughput in production deployments.

query-key-value The three components of attention mechanisms where queries determine what to look for, keys represent what is available, and values contain the actual information to be weighted and combined.

R

real-time processing Processing data as it becomes available under application timing constraints; hard real-time systems guarantee worst-case deadlines, while soft real-time systems tolerate occasional misses.

receptive field The region of the input that influences a particular neuron's output, determining the spatial extent of patterns that can be detected by that neuron.

recurrent neural network A type of neural network designed for sequential data processing, featuring connections that create loops allowing information to persist across time steps.

Red AI AI research and development that prioritizes maximizing accuracy or performance without regard for the increasing computational and environmental costs required.

regularization Techniques used to prevent overfitting in neural networks by adding constraints or penalties, including methods like dropout, weight decay, and data augmentation.

ReLU Rectified Linear Unit activation function defined as $f(x) = \max(0, x)$ that introduces nonlinearity while maintaining computational efficiency and helping mitigate vanishing gradients for positive inputs.

residual connection A skip connection that adds the input of a layer to its output, enabling the training of very deep networks by mitigating the vanishing gradient problem.

ResNet Residual Network, a deep convolutional architecture built from residual blocks with skip connections, enabling the training of networks with hundreds of layers and achieving breakthrough performance.

ResNet-50 A specific 50-layer variant of the Residual Network architecture that serves as the canonical lighthouse model for compute-bound workloads, balancing depth and computational cost for vision benchmarks.

retinal fundus photographs Medical images of the interior surface of the eye, including the retina, optic disc, and blood vessels, commonly used for diagnosing eye diseases and training medical AI systems.

reverse-mode differentiation An automatic differentiation technique that computes gradients by traversing the computational graph in reverse order, highly efficient for functions with many inputs and few outputs, making it ideal for neural network training.

ridge point The arithmetic intensity at which a workload transitions from memory-bound to compute bound on a given hardware platform, calculated as peak FLOP/s divided by memory bandwidth. The inflection point on the Roofline Model diagram.

Ring AllReduce A bandwidth-efficient AllReduce algorithm where devices pass chunks around a ring and accumulate results for distributed gradient synchronization.

RMSprop An adaptive learning rate optimization algorithm that maintains a moving average of squared gradients to automatically adjust learning rates for each parameter during training.

role-based access control (RBAC) An access-control model that grants permissions according to user, service, or team roles rather than ad hoc individual grants.

rollback Process of reverting to a previous stable version of a model or system when issues are detected in production, ensuring service continuity.

roofline analysis A performance modeling technique that plots attainable performance against operational intensity, with peak compute and bandwidth forming the ceilings, to identify whether a system is memory bound or compute bound.

S

scalability The ability of machine learning systems to handle increasing amounts of data, users, or computational demands without significant degradation in performance or user experience.

scale and zero-point Quantization parameters that map real-valued tensors to integer values and back, where scale controls step size and zero-point maps real zero into the quantized range.

schema The structure and format definition of data that specifies data types, field names, and relationships, essential for data validation and processing consistency.

schema evolution The process of modifying data schemas over time while maintaining backward compatibility and ensuring continued functionality of dependent systems and applications.

schema-on-read An approach used in data lakes where data structure is defined and enforced at the time of reading rather than when storing, providing flexibility for diverse data types.

scratchpad memory Fast software-managed local memory used by accelerators to stage predictable data movement under compiler or runtime control.

segmentation maps Detailed annotations that classify objects at the pixel level, providing the most granular labeling information but requiring significantly more storage and processing resources.

self-attention An attention mechanism where queries, keys, and values all come from the same sequence, allowing each position to attend to all positions including itself.

semi-supervised learning A machine learning approach that uses both labeled and unlabeled data for training, using structural assumptions to improve model performance with limited labels.

sensitivity The proportion of actual positive cases correctly identified by a classification model, also known as true positive rate or recall; critical in medical applications where missing positive cases has severe consequences.

serverless Cloud computing model where infrastructure is automatically managed by the provider, allowing code execution without server management concerns.

service level agreement (SLA) A formal external contract specifying service commitments, often through one or more SLOs, and the consequences or penalties for noncompliance.

service-level objective (SLO) An internal measurable target for a service-level indicator such as latency, error rate, or availability that guides operations and may underpin an external SLA.

shadow deployment A deployment strategy where a new model runs in parallel with the production model, making predictions that are logged but not served to users, enabling validation without user impact.

shallow learning Machine learning approaches that use algorithms with limited complexity, such as support vector machines and decision trees, which require carefully engineered features but cannot automatically discover hierarchical representations like deep learning methods.

SHAP values Feature-attribution scores based on Shapley values that estimate each feature's contribution to a model prediction.

sigmoid An activation function that maps input values to a range between 0 and 1, historically popular but prone to vanishing gradient problems in deep networks.

silent bias Model unfairness that produces valid-looking but discriminatory outputs, evading traditional error monitoring and requiring disaggregated evaluation to detect.

silent degradation The gradual decline in ML system performance that occurs without triggering errors, exceptions, or alerts. An ML system can continue operating while producing increasingly inaccurate predictions.

silent failure A system failure mode where an ML model continues to produce plausible-looking outputs that are gradually less accurate or contextually relevant without triggering conventional error alerts.

SIMD (Single Instruction, Multiple Data) A parallel computing architecture that applies the same operation to multiple data elements simultaneously, effective for regular data-parallel computations.

SIMT (Single Instruction, Multiple Threads) An extension of SIMD that enables parallel execution across multiple independent threads, each maintaining its own state and program counter.

singular value decomposition A matrix factorization technique that decomposes a matrix into the product of three matrices, commonly used in low-rank approximations to compress neural network layers by retaining only the most significant singular values.

skip connection A direct connection that bypasses one or more layers, allowing gradients to flow more easily through deep networks and enabling better training of very deep architectures.

soft targets Teacher-model probability distributions used as supervision in knowledge distillation, carrying class-similarity information beyond hard labels.

softmax An activation function that converts logits into a probability distribution where outputs sum to 1, used in multi-class classification.

sparse Tensor Core A Tensor Core variant that accelerates supported structured sparsity patterns such as 2:4 sparsity by skipping constrained zero values.

sparsity The property of tensors in which many values are exactly zero, which can improve computational efficiency when the zeros have suitable structure and specialized sparse kernels or hardware support them; near-zero values count only after pruning.

SPEC CPU Standardized benchmark suite developed by the System Performance Evaluation Cooperative that measures processor performance using real-world applications rather than synthetic tests.

special function unit (SFU) Dedicated hardware for specialized operations such as exponentials, reciprocals, roots, and some activation-function primitives that do not map cleanly to ordinary arithmetic or matrix multiply units.

specificity The proportion of actual negative cases correctly identified by a classification model, also known as true negative rate; important for avoiding overwhelming referral systems with false positives.

SPECpower Benchmark methodology that measures server energy efficiency across varying workload levels, enabling direct comparisons of power-performance trade-offs in computing systems.

speculative decoding An optimization technique for autoregressive language models where a smaller model generates draft tokens that are then verified by a larger model, accelerating inference while maintaining quality.

speed of light (latency) The physical speed limit of information transmission (approx. 200,000 km/s in fiber), creating an irreducible lower bound on network latency that requires edge computing for applications needing sub-10 ms response times over long distances.

state dict A framework serialization mapping from module or optimizer names to tensors, used to save, load, and checkpoint models and optimizer state.

static graph A computational graph that is defined completely before execution begins, enabling comprehensive optimization and efficient deployment but requiring all oper-

ations to be specified upfront, limiting runtime flexibility.

static quantization A quantization approach where quantization parameters are determined once during calibration and remain fixed during inference, providing computational efficiency but less adaptability than dynamic approaches.

statistical learning The era of machine learning that emerged in the 1990s, shifting focus from rule-based symbolic AI to algorithms that could learn patterns from data, laying the groundwork for modern data-driven approaches to artificial intelligence.

stochastic gradient descent A variant of gradient descent that estimates gradients using individual training examples or small batches rather than the entire dataset, reducing memory requirements and enabling online learning.

Straight-Through Estimator (STE) A gradient approximation that lets training pass gradients through nondifferentiable operations such as rounding, often used in quantization-aware training.

stream ingestion A data processing pattern that handles data in real-time as it arrives, essential for applications requiring immediate processing and low-latency responses.

stream processing Real-time data processing approach that handles continuous flows of data as it arrives, enabling immediate responses to events and pattern detection.

stride The step size by which a convolutional filter moves across the input, controlling the spatial dimensions of the output and the degree of overlap between filter applications.

structured pruning A pruning approach that removes entire computational units such as neurons, channels, or layers, producing smaller dense models that are more hardware-friendly than the sparse matrices created by unstructured pruning.

supervised learning A machine learning approach where models learn from labeled training examples to make predictions on new, unlabeled data.

symbolic AI An approach to artificial intelligence that uses high-level symbolic representations of problems, logic, and search algorithms, dominant before the deep learning revolution and characterized by expert systems and rule-based reasoning.

synthetic benchmark Artificial test program designed to measure specific aspects of system performance, as opposed to benchmarks based on real-world applications and workloads.

synthetic data Artificially generated data created using algorithms, simulations, or generative models to supplement real-world datasets, addressing limitations in data availability or privacy concerns.

system entropy The tendency of ML systems to degrade over time as the world changes (drift) or as hidden dependencies accumulate (technical debt), requiring active energy (ops) to maintain order.

System-on-Chip (SoC) An integrated circuit that incorporates most or all components of a computer or electronic system, including CPU, GPU, memory, and specialized processors on a single chip. Commonly used in mobile devices and embedded systems for space and power efficiency.

systems co-design Joint design of algorithms, software, and hardware so model structure and execution fit physical capabilities and constraints.

systems integration The process of combining various components and subsystems into a unified, functional system that operates efficiently and reliably as a whole.

systems thinking An approach to understanding complex systems by considering how individual components interact and affect the whole system, particularly important in ML where data, algorithms, hardware, and deployment environments must work together effectively.

systolic array A specialized hardware architecture that efficiently performs matrix operations by streaming data through a grid of processing elements, minimizing data movement and energy consumption.

T

tail latency High-percentile response time in a system, typically measured as p95 or p99 latency, important for characterizing slow requests under peak load but not equivalent to the absolute worst case. It complements median and mean latency rather than replacing them.

tailored inference benchmarks Specialized performance tests designed for specific deployment environments or use cases, accounting for unique constraints and optimization requirements.

tanh Hyperbolic tangent activation function that maps inputs to (-1, 1), providing zero-centered outputs.

technical debt Long-term maintenance cost accumulated from expedient design decisions during development, particularly problematic in ML systems due to data dependencies and model complexity.

telemetry Automated collection and transmission of performance data and metrics from distributed systems, enabling remote monitoring and analysis.

tensor A multi-dimensional array used to represent data in neural networks, generalizing scalars (0D), vectors (1D), and matrices (2D) to higher dimensions.

Tensor Core A specialized GPU matrix unit for accelerated mixed-precision multiply-accumulate operations on tensor tiles.

tensor decomposition The extension of matrix factorization to higher-order tensors, used to compress neural network layers by representing weight tensors as combinations of smaller tensors with fewer parameters.

tensor parallelism A model parallelism strategy where individual tensor operations (like matrix multiplication) are split across multiple devices, reducing memory per device and potentially reducing large-layer latency when the reduction in computation time exceeds the added communication time.

tensor processing unit (TPU) Google's custom application-specific integrated circuit designed specifically for machine learning workloads, optimized for matrix operations and featuring systolic array architecture.

TensorFlow A comprehensive machine learning framework developed by Google that provides tools for the entire ML pipeline from research to production, featuring both eager execution and graph-based computation with extensive ecosystem support.

TensorRT NVIDIA's inference optimization library that applies techniques like operator fusion and precision reduction to accelerate deep learning inference on GPU hardware.

ternarization A low-bit quantization technique that constrains weights to three values (typically -1, 0, +1), providing significant compression while maintaining more representational capacity than binary quantization.

thermal design power (TDP) The sustained power and heat level a device or cooling system is designed to dissipate under typical maximum workload.

thermal throttling A protective mechanism in mobile and embedded devices that reduces processor clock speed and performance to prevent overheating, often limiting the sustained performance of on-device ML inference.

throughput The rate at which a system can process data or complete operations, typically measured in operations per second and a primary metric for training large models.

time per output token (TPOT) The per-token latency during autoregressive decode after the first token, reflecting decode throughput and memory-bandwidth limits.

time-to-accuracy The wall-clock time required to train a model to a specified validation accuracy.

time to first token (TTFT) The time from request arrival to the first generated token in streaming LLM serving, including prefill, scheduling, and initial response latency.

TinyML A field focused on deploying machine learning models on ultra-constrained embedded devices such as microcontrollers and sensors, operating in the milliwatt to sub-watt power range, with severe limitations on memory, power, and computational capacity. Also called *tiny machine learning*.

TOPS Tera Operations Per Second, an operation-rate unit indicating how many trillion operations a system can execute in one second. It is commonly reported for low-precision integer workloads, so the operation type and precision must be specified.

total cost of ownership (TCO) A comprehensive financial metric for ML systems encompassing training, inference, and operational costs over the system's entire lifecycle.

train-serve split A hybrid architecture pattern where computationally intensive model training occurs on powerful cloud infrastructure, while the trained model is optimized and deployed for inference on resource-constrained edge or mobile devices.

training The process of adjusting neural network parameters using data or interaction experience and an optimization procedure to minimize a loss or maximize a reward.

training-serving skew A mismatch between how features or data are computed during model training vs. serving in production, which can degrade model performance. Common causes include different preprocessing pipelines, feature computation timing, or data sources between training and inference.

transfer learning A machine learning technique that uses knowledge gained from pretrained models on related tasks, allowing faster training and better performance on new tasks with limited data by reusing learned features and representations.

transformer A neural network architecture built from attention, feed-forward blocks, residual connections, and normalization; the original design eliminates recurrence and convolution, though later variants may incorporate them.

translation equivariance A property where shifting the input causes a corresponding shift in the output feature map, distinct from invariance which discards position.

translation invariance The property that a model's output remains unchanged when the input shifts. Convolution is translation-

equivariant; pooling or global aggregation can produce approximate invariance.

Tucker decomposition A tensor decomposition method that generalizes singular value decomposition to higher-order tensors using a core tensor and factor matrices, commonly used for compressing convolutional neural network layers.

TV white spaces Unused broadcasting frequencies that can be repurposed for internet connectivity, as employed by systems like FarmBeats to extend network access to remote agricultural sensors and IoT devices.

U

uniform quantization A quantization approach where the range of values is divided into evenly spaced intervals, providing simple implementation but potentially suboptimal for nonuniform value distributions.

universal approximation theorem A theoretical result proving that neural networks with sufficient width and a suitable nonlinear activation function can approximate any continuous function on a compact domain to arbitrary accuracy.

unreasonable effectiveness of data The empirical observation that for many problems, adding more data is more effective than improving algorithms, driving the data-centric AI paradigm.

unstructured pruning A pruning approach that removes individual weights while preserving the overall network architecture, creating sparse weight matrices that require specialized hardware support to realize computational benefits.

unstructured sparsity A form of model sparsity where individual weights are set to zero without following any particular pattern, creating irregular sparsity patterns that require specialized hardware support to realize computational benefits.

V

validation issues Problems identified during model testing that indicate poor performance, overfitting, data quality problems, or other issues that must be resolved before deployment.

vanishing gradient problem A problem in deep neural networks where gradients become exponentially smaller as they propagate backward through layers, making it difficult for early layers to learn effectively.

vector operations Computational operations that process multiple data elements simultaneously, enabling efficient parallel execution of element-wise transformations in neural networks.

verification gap The mismatch between finite test-set coverage and the much larger input space an ML system may encounter in production.

versioning The practice of tracking changes to datasets, models, and pipelines over time, enabling reproducibility, rollback capabilities, and audit trails in ML systems.

virtuous cycle The self-reinforcing process in deep learning where improvements in data availability, algorithms, and computing power each enable further advances in the other areas, accelerating overall progress.

Von Neumann bottleneck The performance limitation caused by the shared bus between processor and memory in traditional computer architectures, where data movement becomes more expensive than computation.

W

warehouse-scale computer (WSC) A data center operated as a single programmable computer, treating compute, storage, networking, and failures as system-level resources.

warp A group of 32 threads on NVIDIA GPUs whose active threads execute one common instruction at a time; the fundamental unit of thread scheduling on NVIDIA GPUs.

Waymo A subsidiary of Alphabet Inc. that represents a leading deployment of machine learning systems in autonomous vehicle technology, demonstrating how ML systems can span from embedded systems to cloud infrastructure in safety-critical environments.

weak supervision An approach that uses lower-quality labels obtained more efficiently through heuristics, distant supervision, or programmatic methods rather than manual expert annotation.

web scraping An automated technique for extracting data from websites to build custom datasets, requiring careful consideration of legal, ethical, and technical constraints.

weight A learnable parameter that determines the strength of connection between neurons in different layers, adjusted during training to minimize the loss function.

weight matrix An organized collection of weights connecting one layer to another in a neural network, enabling efficient computation through matrix operations.

weight sharing The practice of using the same parameters across different spatial locations, as in convolutional networks, reducing the number of parameters while maintaining pattern detection capabilities.

Whetstone Early benchmark developed at the UK National Physical Laboratory by Curnow and Wichmann, first published as an ALGOL 60 program in 1972 and later as the canonical FORTRAN version in 1976. It measured floating-point arithmetic performance in MWIPS (millions of Whetstone instructions per second), becoming one of the first widely-adopted standardized performance tests.

workflow orchestration Automated coordination and management of complex ML pipeline sequences, ensuring proper execution order, dependency management, and error handling across distributed systems.

X

XLA Accelerated Linear Algebra, a domain-specific compiler for linear algebra operations that optimizes TensorFlow and JAX computations by generating efficient code for various hardware platforms including CPUs, GPUs, and TPUs.

Y

YOLO You Only Look Once, a family of single-stage object detection models that reframes detection as a spatial regression problem, predicting bounding boxes and class probabilities in a single forward pass to achieve high throughput on edge accelerators.

Z

zero-copy memory A memory allocation pattern that allows accelerator execution units to directly read or write host pinned memory across the system bus without an explicit staging transfer, trading bus bandwidth for reduced memory allocation footprint.

ZeRO Zero Redundancy Optimizer, a memory-efficient distributed training paradigm that partitions optimizer states, gradients, and model parameters across data-parallel ranks to eliminate memory redundancy while maintaining communication efficiency.

zero-shot learning A deployment paradigm where a pretrained model performs inference on tasks or classes unseen during supervised training, relying on learned semantic representations without fine-tuning.

References

Abadi, Martín, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. 2016. "TensorFlow: A System for Large-Scale Machine Learning." *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 265–83.

Abadi, M., A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. 2016. "Deep Learning with Differential Privacy." *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, 308–18. https://doi.org/10.1145/2976749.2978318.

Agarwal, Pankaj K., Sariel Har-Peled, and Kasturi R. Varadarajan. 2005. "Geometric Approximation via Coresets." In *Combinatorial and Computational Geometry*, edited by Jacob E. Goodman, János Pach, and Emo Welzl, vol. 52. MSRI Publications. Cambridge University Press. https://doi.org/10.1017/9781009701259.002.

Agrawal, Amey, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve." *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 117–34.

Ainslie, Joshua, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4895–901. https://doi.org/10.18653/v1/2023.emnlp-main.298.

Akidau, Tyler, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, et al. 2015. "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing." *Proceedings of the VLDB Endowment* 8 (12): 1792–803. https://doi.org/10.14778/2824032.2824076.

Altini, Marco, and Hannu Kinnunen. 2021. "The Promise of Sleep: A Multi-Sensor Approach for Accurate Sleep Stage Detection Using the Oura Ring." *Sensors* 21 (13): 4302. https://doi.org/10.3390/s21134302.

Amazon Web Services. 2024a. *Amazon Mechanical Turk*.

Amazon Web Services. 2024b. *AWS Auto Scaling*.

AMD. 2023. *AMD Instinct MI300X Accelerators*. AMD product documentation.

Amdahl, Gene M. 1967. "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities." *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference on - AFIPS '67 (Spring)*, AFIPS '67 (spring), 483–85. https://doi.org/10.1145/1465482.1465560.

Amershi, Saleema, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. "Software Engineering for Machine Learning: A Case Study." *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. https://doi.org/10.1109/icse-seip.2019.00042.

Amodei, Dario, and Danny Hernandez. 2018a. "AI and Compute." *OpenAI Blog* 2.

Amodei, Dario, and Danny Hernandez. 2018b. "AI and Compute." *OpenAI Blog* 6.

Anderson, Edward, Zhaojun Bai, Christian Bischof, L. Susan Blackford, James Demmel, Jack Dongarra, Jeremy Du Croz, et al. 1999. *LAPACK Users' Guide*. 3rd ed. SIAM. https://doi.org/10.1137/1.9780898719604.

Angwin, Julia, Jeff Larson, Surya Mattu, and Lauren Kirchner. 2016. *Machine Bias*. ProPublica investigation.

Ansel, Jason, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, et al. 2024. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation." *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 929–47. https://doi.org/10.1145/3620665.3640366.

Anthropic. 2023. *Introducing Claude 2.1*.

Anthropic. 2024. *Introducing the Next Generation of Claude*.

AnyLogic. 2024. *Synthetic Data for Artificial Intelligence*.

Apache Software Foundation. 2024. *Apache Airflow*.

Apple. 2024a. *Facial Matching Security*. Apple Platform Security.

Apple. 2024b. *The Secure Enclave*. Apple Platform Security.

Ardila, Rosana, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber. 2020. "Common Voice: A Massively-Multilingual Speech Corpus." *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 4218–22.

Armbrust, Michael, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, et al. 2020. "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores." *Proceedings of the VLDB Endowment* 13 (12): 3411–24. https://doi.org/10.14778/3415478.3415560.

Authors, Kubeflow. 2024. *Kubeflow*.

Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. "Layer Normalization." *arXiv Preprint arXiv:1607.06450*.

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. "Neural Machine Translation by Jointly Learning to Align and Translate." *International Conference on Learning Representations (ICLR)*.

Baidu Research. 2016. *DeepBench: Benchmarking Deep Learning Operations on Different Hardware*.

Bakheet, Samy, and Ayoub Al-Hamadi. 2021. "A Framework for Instantaneous Driver Drowsiness Detection Based on Improved HOG Features and Naïve Bayesian Classification." *Brain Sciences* 11 (2): 240. https://doi.org/10.3390/brainsci11020240.

Banbury, Colby R., Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, et al. 2020. "Benchmarking TinyML Systems: Challenges and Direction." *arXiv Preprint arXiv:2003.04821*.

Banbury, Colby R, Chuteng Zhou, Igor Fedorov, Ramon Matas, Urmish Thakker, Dibakar Gope, Vijay Janapa Reddi, Matthew Mattina, and Paul N Whatmough. 2021. "MicroNets: Neural Network Architectures for Deploying TinyML Applications on Commodity Microcontrollers." *Proceedings of Machine Learning and Systems* 3: 517–32.

Banbury, Colby, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, et al. 2021. "MLPerf Tiny Benchmark." *arXiv Preprint*.

Banbury, Colby, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. 2021. "MLPerf Tiny Benchmark." *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*.

Barocas, Solon, and Andrew D. Selbst. 2016. "Big Data's Disparate Impact." *California Law Review* 104: 671–732. https://doi.org/10.2139/ssrn.2477899.

Barroso, Luiz André, Urs Hölzle, and Parthasarathy Ranganathan. 2019. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*. Synthesis Lectures on Computer Architecture. Springer International Publishing. https://doi.org/10.1007/978-3-031-01761-2.

Barroso, Luiz, Mike Marty, David Patterson, and Parthasarathy Ranganathan. 2017. "Attack of the Killer Microseconds." *Communications of the ACM* 60 (4): 48–54. https://doi.org/10.1145/3015146.

Baur, Walter, and Volker Strassen. 1983. "The Complexity of Partial Derivatives." *Theoretical Computer Science* 22 (3): 317–30. https://doi.org/10.1016/0304-3975(83)90110-x.

Baydin, Atilim Gunes, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. 2018. "Automatic Differentiation in Machine Learning: A Survey." *Journal of Machine Learning Research* 18 (153): 1–43.

Baylor, Denis, Eric Breck, Heng-Tze Cheng, Noah Fiedel, Chuan Yu Foo, Zakaria Haque, Salem Haykal, et al. 2017. "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform." *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1387–95. https://doi.org/10.1145/3097983.3098021.

Bedford Taylor, Michael. 2017. "The Evolution of Bitcoin Hardware." *Computer* 50 (9): 58–66. https://doi.org/10.1109/mc.2017.3571056.

Beede, Emma, Elizabeth Baylor, Fred Hersch, Anna Iurchenko, Lauren Wilcox, Paisan Ruamviboonsuk, and Laura M. Vardoulakis. 2020. "A Human-Centered Evaluation of a Deep Learning System Deployed in Clinics for the Detection of Diabetic Retinopathy." *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*, 1–12. https://doi.org/10.1145/3313831.3376718.

Belkin, Mikhail, Daniel Hsu, Siyuan Ma, and Soumik Mandal. 2019. "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off." *Proceedings of the National Academy of Sciences* 116 (32): 15849–54. https://doi.org/10.1073/pnas.1903070116.

Bellamy, Rachel K. E., Kuntal Dey, Michael Hind, Seung Hoffman, Sylvain Houde, Karthikeyan Kannan, Pradeep Lohia, et al. 2019. "AI Fairness 360: An Extensible Toolkit for Detecting and Mitigating Algorithmic Bias." *IBM Journal of Research and Development* 63 (4/5): 4:1–15. https://doi.org/10.1147/jrd.2019.2942287.

Bender, Emily M., and Batya Friedman. 2018. "Data Statements for Natural Language Processing: Toward Mitigating System Bias and Enabling Better Science." *Transactions of the Association for Computational Linguistics* 6: 587–604. https://doi.org/10.1162/tacl_a_00041.

Bengio, Emmanuel, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. 2015. "Conditional Computation in Neural Networks for Faster Models." *arXiv Preprint arXiv:1511.06297*.

Bengio, Yoshua, Aaron Courville, and Pascal Vincent. 2013. "Representation Learning: A Review and New Perspectives." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8): 1798–828. https://doi.org/10.1109/tpami.2013.50.

Bengio, Yoshua, Nicholas Léonard, and Aaron Courville. 2013. "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation." *arXiv Preprint arXiv:1308.3432*.

Bengio, Yoshua, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. "Curriculum Learning." *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48. https://doi.org/10.1145/1553374.1553380.

Bengio, Yoshua, Patrice Simard, and Paolo Frasconi. 1994. "Learning Long-Term Dependencies with Gradient Descent Is Difficult." *IEEE Transactions on Neural Networks* 5 (2): 157–66. https://doi.org/10.1109/72.279181.

Berger, Vance W., and YanYan Zhou. 2014. "Kolmogorov–Smirnov Test: Overview." In *Wiley StatsRef: Statistics Reference Online*. Wiley. https://doi.org/10.1002/9781118445112.stat06558.

Bergstra, James, Olivier Breuleux, Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, Guillaume Desjardins, Joseph Turian, David Warde-Farley, and Yoshua Bengio. 2010. "Theano: A CPU and GPU Math Compiler in Python." *Proceedings of the Python in Science Conference* 4: 18–24. https://doi.org/10.25080/majora-92bf1922-003.

Beyer, Betsy, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.

Beyer, Lucas, Olivier J. Hénaff, Alexander Kolesnikov, Xiaohua Zhai, and Aäron van den Oord. 2020. "Are We Done with ImageNet?" *arXiv Preprint arXiv:2006.07159*.

Bird, Sarah, Miro Dudík, Richard Edgar, Brandon Horn, Roman Lutz, Vanessa Milan, Mehrnoosh Sameki, Hanna Wallach, and Kathleen Walker. 2020. "Fairlearn: A Toolkit for Assessing and Improving Fairness in AI." *Microsoft Technical Report MSR-TR-2020-32*.

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Springer.

Blalock, Davis, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. 2020. "What Is the State of Neural Network Pruning?" *Proceedings of Machine Learning and Systems (MLSys)* 2: 129–46.

Bobrow, Daniel G. 1964. "Natural Language Input for a Computer Problem Solving System." PhD thesis, Massachusetts Institute of Technology.

Boehm, Barry W. 1981. *Software Engineering Economics*. Prentice-Hall.

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam Tauman Kalai. 2016. "Man Is to Computer Programmer as Woman Is to Homemaker? Debiasing Word Embeddings." *Advances in Neural Information Processing Systems (NeurIPS)*, 4349–57.

Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, et al. 2021. "On the Opportunities and Risks of Foundation Models." *arXiv Preprint arXiv:2108.07258*.

Bonawitz, Keith, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. "Practical Secure Aggregation for Privacy-Preserving Machine Learning." *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 1175–91. https://doi.org/10.1145/3133956.3133982.

Boroumand, Amirali, Saugata Ghose, Youngsok Kim, Rachata Ausavarungnirun, Eric Shiu, Rahul Thakur, Daehyun Kim, et al. 2018. "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks." *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '18, 316–31. https://doi.org/10.1145/3173162.3173177.

Bourtoule, Lucas, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. "Machine Unlearning." *2021 IEEE Symposium on Security and Privacy (SP)*, 141–59. https://doi.org/10.1109/sp40001.2021.00019.

Bradbury, James, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*.

Breck, Eric, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. 2017. "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction." *Proceedings of the 2017 IEEE International Conference on Big Data (Big Data)*, 1123–32. https://doi.org/10.1109/bigdata.2017.8258038.

Breck, Eric, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. 2019. "Data Validation for Machine Learning." *Proceedings of the 2nd SysML Conference* 1: 334–47.

Bridle, John S. 1990. "Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition." In *Neurocomputing: Algorithms, Architectures and Applications*, edited by Françoise Fogelman Soulié and Jeanny Hérault, vol. 68. NATO ASI Series f: Computer and Systems Sciences. Springer. https://doi.org/10.1007/978-3-642-76153-9_28.

Broder, A. Z. 1997. "On the Resemblance and Containment of Documents." *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No.97TB100171)*, 21–29. https://doi.org/10.1109/sequen.1997.666900.

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. "Language Models Are Few-Shot Learners." *Advances in Neural Information Processing Systems* 33: 1877–901. https://doi.org/10.48550/arxiv.2005.14165.

Brutlag, Jake. 2009. *Speed Matters for Google Web Search*. Google Research Blog.

Buolamwini, Joy, and Timnit Gebru. 2018. "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification." *Conference on Fairness, Accountability and Transparency*, 77–91.

Burns, Brendan, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. "Borg, Omega, and Kubernetes." *Communications of the ACM* 59 (5): 50–57. https://doi.org/10.1145/2890784.

Cai, Han, Chuang Gan, Ligeng Zhu, and Song Han. 2020. "TinyTL: Reduce Activations, Not Trainable Parameters for Efficient on-Device Learning." *Advances in Neural Information Processing Systems* 33: 11285–97.

Campbell, Murray, Jr. Hoane A.Joseph, and Feng-hsiung Hsu. 2002. "Deep Blue." *Artificial Intelligence* 134 (1-2): 57–83. https://doi.org/10.1016/s0004-3702(01)00129-1.

Cao, Yinzhi, and Junfeng Yang. 2015. "Towards Making Systems Forget with Machine Unlearning." *2015 IEEE Symposium on Security and Privacy*, 463–80. https://doi.org/10.1109/sp.2015.35.

Chapman, Pete, Julian Clinton, Randy Kerber, Thomas Khabaza, Thomas Reinartz, Colin Shearer, and Rudiger Wirth. 2000. "CRISP-DM 1.0: Step-by-Step Data Mining Guide." *SPSS Inc* 1: 78.

Chellapilla, Kumar, Sidd Puri, and Patrice Simard. 2006. *High Performance Convolutional Neural Networks for Document Processing*. Tenth International Workshop on Frontiers in Handwriting Recognition.

Chen, Andrew, Andy Chow, Aaron Davidson, Arjun DCunha, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, et al. 2020. "Developments in MLflow: A System to Accelerate the Machine Learning Lifecycle." *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, 1–4. https://doi.org/10.1145/3399579.3399867.

Chen, Emma, Shvetank Prakash, Vijay Janapa Reddi, David Kim, and Pranav Rajpurkar. 2023. "A Framework for Integrating Artificial Intelligence for Clinical Care with Continuous Therapeutic Monitoring." *Nature Biomedical Engineering* 9 (4): 445–54. https://doi.org/10.1038/s41551-023-01115-0.

Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. 2021. "Evaluating Large Language Models Trained on Code." *arXiv Preprint arXiv:2107.03374*.

Chen, Tianqi, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, et al. 2018. "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning." *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, 578–94.

Chen, Tianqi, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. "Training Deep Nets with Sublinear Memory Cost." *arXiv Preprint arXiv:1604.06174*.

Chen, Tianshi, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. "DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning." *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14)*, 269–84. https://doi.org/10.1145/2541940.2541967.

Chen, Ting, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. "A Simple Framework for Contrastive Learning of Visual Representations." *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, vol. 119: 1597–607.

Chen, Yanxi, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. "EE-LLM: Large-Scale Training and Inference of Early-Exit Large Language Models with 3D Parallelism." *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Chen, Yu-Hsin, Joel S. Emer, and Vivienne Sze. 2016. "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks." *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 367–79. https://doi.org/10.1109/ISCA.2016.40.

Chen, Yu-Hsin, Joel Emer, and Vivienne Sze. 2017. "Using Dataflow to Optimize Energy Efficiency of Deep Neural Network Accelerators." *IEEE Micro* 37 (3): 12–21. https://doi.org/10.1109/mm.2017.54.

Chetlur, Sharan, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. "cuDNN: Efficient Primitives for Deep Learning." *arXiv Preprint arXiv:1410.0759*.

Child, Rewon, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. "Generating Long Sequences with Sparse Transformers." *arXiv Preprint arXiv:1904.10509*.

Cho, Aeree, Grace C. Kim, Alexander Karpekov, Alec Helbling, Zijie J. Wang, Seongmin Lee, Benjamin Hoover, and Duen Horng (Polo) Chau. 2025. "TRANSFORMER EXPLAINER: Interactive Learning of Text-Generative Models." *Proceedings of the AAAI Conference on Artificial Intelligence* 39 (28): 29625–27. https://doi.org/10.1609/aaai.v39i28.35347.

Cho, Kyunghyun, Bart van Merrienboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. "On the Properties of Neural Machine Translation: Encoder-Decoder Approaches." *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, 103–11. https://doi.org/10.3115/v1/w14-4012.

Choi, Dami, Alexandre Passos, Christopher J. Shallue, and George E. Dahl. 2019. "Faster Neural Network Training with Data Echoing." *arXiv Preprint*, ahead of print. https://doi.org/10.48550/arXiv.1907.05550.

Choi, Jungwook, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. "PACT: Parameterized Clipping Activation for Quantized Neural Networks." *arXiv Preprint*.

Chollet, François. 2018. *Deep Learning with Python*. Manning Publications.

Chollet, François et al. 2024. *Keras*.

Choquette, Jack. 2023. "NVIDIA Hopper H100 GPU: Scaling Performance." *IEEE Micro* 43 (3): 9–17. https://doi.org/10.1109/mm.2023.3256796.

Choquette, Jack, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. "NVIDIA A100 Tensor Core GPU: Performance and Innovation." *IEEE Micro* 41 (2): 29–35. https://doi.org/10.1109/mm.2021.3061394.

Choudhary, Tejalal, Vipul Mishra, Anurag Goswami, and Jagannathan Sarangapani. 2020. "A Comprehensive Survey on Model Compression and Acceleration." *Artificial Intelligence Review* 53 (7): 5113–55. https://doi.org/10.1007/s10462-020-09816-7.

Choukroun, Yoni, Eli Kravchik, Fan Yang, and Pavel Kisilev. 2019. "Low-Bit Quantization of Neural Networks for Efficient Inference." *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, 3009–18. https://doi.org/10.1109/iccvw.2019.00363.

Chouldechova, Alexandra. 2017. "Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments." *Big Data* 5 (2): 153–63. https://doi.org/10.1089/big.2016.0047.

Chow, Chao-Kong. 1970. "On Optimum Recognition Error and Reject Tradeoff." *IEEE Transactions on Information Theory* 16 (1): 41–46. https://doi.org/10.1109/TIT.1970.1054406.

Chowdhery, Aakanksha et al. 2019. "Visual Wake Words Dataset." *arXiv Preprint arXiv:1906.05721*.

Chowdhery, Aakanksha, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, et al. 2022. "PaLM: Scaling Language Modeling with Pathways." *arXiv Preprint arXiv:2204.02311*.

Chu, Grace, Okan Arikan, Gabriel Bender, Weijun Wang, Achille Brighton, Pieter-Jan Kindermans, Hanxiao Liu, Berkin Akin, Suyog Gupta, and Andrew Howard. 2021. "Discovering Multi-Hardware Mobile Models via Architecture Search." *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3016–25. https://doi.org/10.1109/cvprw53098.2021.00337.

Chung, Jae-Won, Jeff J. Ma, Ruofan Wu, Jiachen Liu, Oh Jun Kweon, Yuxuan Xia, Zhiyu Wu, and Mosharaf Chowdhury. 2025. "The ML.ENERGY Benchmark: Toward Automated Inference Energy Measurement and Optimization." *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

Clark, Kevin, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. 2019. "What Does BERT Look at? An Analysis of BERT's Attention." *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 276–86. https://doi.org/10.18653/v1/w19-4828.

Cloud Native Computing Foundation. 2024a. *Kubernetes: Production-Grade Container Orchestration*.

Cloud Native Computing Foundation. 2024b. *Prometheus: Monitoring System and Time Series Database*.

Cohen, Jacob. 1960. "A Coefficient of Agreement for Nominal Scales." *Educational and Psychological Measurement* 20 (1): 37–46. https://doi.org/10.1177/001316446002000104.

Cohen, Taco, and Max Welling. 2016. "Group Equivariant Convolutional Networks." *Proceedings of the 33rd International Conference on Machine Learning (ICML)* 48: 2990–99.

Coleman, Cody, Edward Chou, Julian Katz-Samuels, Sean Culatana, Peter Bailis, Alexander C. Berg, Robert Nowak, Roshan Sumbaly, Matei Zaharia, and I. Zeki Yalniz. 2022. "Similarity Search for Efficient Active Learning and Search of Rare Concepts." *Proceedings of the AAAI Conference on Artificial Intelligence* 36 (6): 6402–10. https://doi.org/10.1609/aaai.v36i6.20591.

Coleman, Cody, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. 2019. "Analysis of DAWNBench, a Time-to-Accuracy Machine Learning Performance Benchmark." *ACM SIGOPS Operating Systems Review* 53 (1): 14–25. https://doi.org/10.1145/3352020.3352024.

Consumer Financial Protection Bureau. 2023. *12 CFR 1002.9: Notifications (Regulation B, Equal Credit Opportunity Act)*.

Cordts, Marius, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. 2016. "The Cityscapes Dataset for Semantic Urban Scene Understanding." *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3213–23. https://doi.org/10.1109/CVPR.2016.350.

Cortes, Corinna, and Vladimir Vapnik. 1995. "Support-Vector Networks." *Machine Learning* 20 (3): 273–97. https://doi.org/10.1007/BF00994018.

Cottier, Ben, Robi Rahman, Loredana Fattorini, Nestor Maslej, Tamay Besiroglu, and David Owen. 2024. "The Rising Costs of Training Frontier AI Models." *arXiv Preprint arXiv:2405.21015*.

Courbariaux, Matthieu, Yoshua Bengio, and Jean-Pierre David. 2015. "BinaryConnect: Training Deep Neural Networks with Binary Weights During Propagations." *Advances in Neural Information Processing Systems (NeurIPS)* 28: 3123–31.

Cover, Thomas M., and Joy A. Thomas. 2006. *Elements of Information Theory*. 2nd ed. Wiley. https://doi.org/10.1002/047174882X.

Covington, Paul, Jay Adams, and Emre Sargin. 2016. "Deep Neural Networks for YouTube Recommendations." *Proceedings of the 10th ACM Conference on Recommender Systems*, 191–98. https://doi.org/10.1145/2959100.2959190.

Crankshaw, Daniel, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. "Clipper: A Low-Latency Online Prediction Serving System." *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 613–27.

CrowdFlower. 2016. *Data Science Report 2016*.

Cubuk, Ekin D., Barret Zoph, Dandelion Mané, Vijay Vasudevan, and Quoc V. Le. 2019. "AutoAugment: Learning Augmentation Strategies from Data." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 113–23. https://doi.org/10.1109/cvpr.2019.00020.

Cubuk, Ekin D., Barret Zoph, Jonathon Shlens, and Quoc V. Le. 2020. "Randaugment: Practical Automated Data Augmentation with a Reduced Search Space." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3008–17. https://doi.org/10.1109/cvprw50498.2020.00359.

Cunningham, Ward. 1992. "The WyCash Portfolio Management System." *ACM SIGPLAN OOPS Messenger* 4 (2): 29–30. https://doi.org/10.1145/157710.157715.

Curnow, H. J., and B. A. Wichmann. 1976. "A Synthetic Benchmark." *The Computer Journal* 19 (1): 43–49. https://doi.org/10.1093/comjnl/19.1.43.

Cybenko, George. 1989. "Approximation by Superpositions of a Sigmoidal Function." *Mathematics of Control, Signals, and Systems* 2 (4): 303–14. https://doi.org/10.1007/bf02551274.

Dalal, Navneet, and Bill Triggs. 2005. "Histograms of Oriented Gradients for Human Detection." *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)* 1: 886–93. https://doi.org/10.1109/cvpr.2005.177.

Dally, Bill. 2023. "Hardware for Deep Learning." *2023 IEEE Hot Chips 35 Symposium (HCS)*, 1–58. https://doi.org/10.1109/hcs59251.2023.10254716.

Dally, William J., Stephen W. Keckler, and David B. Kirk. 2021. "Evolution of the Graphics Processing Unit (GPU)." *IEEE Micro* 41 (6): 42–51. https://doi.org/10.1109/mm.2021.3113475.

Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. 2022. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 16344–59. https://doi.org/10.52202/068431-1189.

Dao, Tri. 2023. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." *arXiv Preprint arXiv:2307.08691*.

Dastin, Jeffrey. 2018. "Amazon Scraps Secret AI Recruiting Tool That Showed Bias Against Women." *Reuters*.

Data Protection Commission. 2023. *Data Protection Commission Announces Conclusion of Two Inquiries into Meta Ireland*. Regulatory press release.

Databricks. 2024. *MLflow: An Open Source Platform for the Machine Learning Lifecycle*.

David, Robert, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, et al. 2021. "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems." *Proceedings of Machine Learning and Systems* 3: 800–811.

Davis, Steven B., and Paul Mermelstein. 1980. "Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences." *IEEE Transactions on Acoustics, Speech, and Signal Processing* 28 (4): 357–66. https://doi.org/10.1109/TASSP.1980.1163420.

Dean, Jeff, David Patterson, and Cliff Young. 2018. "A New Golden Age in Computer Architecture: Empowering the Machine-Learning Revolution." *IEEE Micro* 38 (2): 21–29. https://doi.org/10.1109/mm.2018.112130030.

Dean, Jeffrey. 2012. *Achieving Rapid Response Times in Large Online Services*. Berkeley AMPLab Cloud Seminar.

Dean, Jeffrey, and Luiz André Barroso. 2013. "The Tail at Scale." *Communications of the ACM* 56 (2): 74–80. https://doi.org/10.1145/2408776.2408794.

Dean, Jeffrey, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, et al. 2012. "Large Scale Distributed Deep Networks." In *Advances in Neural Information Processing Systems (NeurIPS)*, edited by Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger, vol. 25. Curran Associates.

DeepSeek. 2024. *Introducing DeepSeek-V3*.

DeepSeek-AI. 2025. "Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures." *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 1731–45. https://doi.org/10.1145/3695053.3731412.

Deng, Alex, Ya Xu, Ron Kohavi, and Toby Walker. 2013. "Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data." *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining (WSDM)*, 123–32. https://doi.org/10.1145/2433396.2433413.

Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. "ImageNet: A Large-Scale Hierarchical Image Database." *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–55. https://doi.org/10.1109/cvpr.2009.5206848.

Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2024. *ImageNet*.

Dennard, Robert H., Frank H. Gaensslen, Hwa-Nien Yu, Victor L. Rideout, Elias Bassous, and Antoine R. LeBlanc. 1974. "Design of Ion-Implanted MOSFET's with Very Small Physical Dimensions." *IEEE J. Solid-State Circuits* 9 (5): 256–68. https://doi.org/10.1109/jssc.1974.1050511.

Dettmers, Tim, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. "8-Bit Optimizers via Block-Wise Quantization." *International Conference on Learning Representations (ICLR)*.

Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. "QLoRA: Efficient Finetuning of Quantized LLMs." *Advances in Neural Information Processing Systems (NeurIPS)* 36: 10088–115.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 4171–86. https://doi.org/10.18653/v1/n19-1423.

DeVries, Terrance, and Graham W. Taylor. 2017. "Improved Regularization of Convolutional Neural Networks with Cutout." *arXiv Preprint arXiv:1708.04552*.

Dixit, Kaivalya M. 1993. "Overview of the SPEC Benchmarks." In *The Benchmark Handbook for Database and Transaction Systems*, 2nd ed., edited by Jim Gray. Morgan Kaufmann.

Dongarra, J. J., J. R. Bunch, C. B. Moler, and G. W. Stewart. 1979. *LINPACK Users' Guide*. SIAM. https://doi.org/10.1137/1.9781611971811.

Dongarra, Jack J., Jeremy Du Croz, Sven Hammarling, and Iain S. Duff. 1990. "A Set of Level 3 Basic Linear Algebra Subprograms." *ACM Transactions on Mathematical Software* 16 (1): 1–17. https://doi.org/10.1145/77626.79170.

Dongarra, Jack J., Jeremy Du Croz, Sven Hammarling, and Richard J. Hanson. 1988. "An Extended Set of FORTRAN Basic Linear Algebra Subprograms." *ACM Transactions on Mathematical Software* 14 (1): 1–17. https://doi.org/10.1145/42288.42291.

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, et al. 2021. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." *International Conference on Learning Representations (ICLR)*.

Dua, Dheeru, and Casey Graff. 2024. *UCI Machine Learning Repository*.

Duarte, Javier, Nhan Tran, Ben Hawks, Christian Herwig, Jules Muhizi, Shvetank Prakash, and Vijay Janapa Reddi. 2022. "FastML Science Benchmarks: Accelerating Real-Time Scientific Edge Machine Learning." *arXiv Preprint arXiv:2207.07958*.

Dubey, Abhimanyu, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. *The Llama 3 Herd of Models*. arXiv preprint arXiv:2407.21783.

Dwork, Cynthia. 2008. "Differential Privacy: A Survey of Results." In *Theory and Applications of Models of Computation*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-79228-4_1.

Dwork, Cynthia, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Richard Zemel. 2012. "Fairness Through Awareness." *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, 214–26. https://doi.org/10.1145/2090236.2090255.

Dwork, Cynthia, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. "Calibrating Noise to Sensitivity in Private Data Analysis." In *Theory of Cryptography Conference (TCC)*, edited by Shai Halevi and Tal Rabin, vol. 3876. Lecture Notes in Computer Science. Springer Berlin Heidelberg. https://doi.org/10.1007/11681878_14.

Elastic NV. 2024. *Elasticsearch: Distributed Search and Analytics Engine*.

Elman, Jeffrey L. 1990. "Finding Structure in Time." *Cognitive Science* 14 (2): 179–211. https://doi.org/10.1207/s15516709cog1402_1.

Elsken, Thomas, Jan Hendrik Metzen, and Frank Hutter. 2019. "Neural Architecture Search." In *The Springer Series on Challenges in Machine Learning*, vol. 20. Springer International Publishing. https://doi.org/10.1007/978-3-030-05318-5_3.

Elsworth, Cooper, Keguo Huang, David Patterson, Ian Schneider, Robert Sedivy, Savannah Goodman, Ben Townsend, et al. 2025. "Measuring the Environmental Impact of Delivering AI at Google Scale." *arXiv Preprint arXiv:2508.15734*.

Engineering, M. 2016. *Introducing FBLearner Flow: Facebook's AI Backbone*. Engineering at Meta Blog.

Epoch AI. 2024. *Machine Learning Trends*. Epoch AI Research Database.

Epoch AI. 2026a. *Capabilities and Benchmarking*.

Epoch AI. 2026b. *Machine Learning Hardware*.

Epoch AI. 2026c. *Notable AI Models*.

Equal Employment Opportunity Commission, Civil Service Commission, Department of Labor, and Department of Justice. 1978. *Uniform Guidelines on Employee Selection Procedures*. Code of Federal Regulations.

Esmaeilzadeh, Hadi, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger. 2011. "Dark Silicon and the End of Multicore Scaling." *Proceedings of the 38th Annual International Symposium on Computer Architecture*, 365–76. https://doi.org/10.1145/2000064.2000108.

Ethernet Alliance. 2025. *2025 Ethernet Roadmap*. Ethernet Alliance roadmap.

European Data Protection Board. 2018. *Guidelines on Automated Individual Decision-Making and Profiling for the Purposes of Regulation 2016/679*.

European Parliament and Council of the European Union. 2024. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying down Harmonised Rules on Artificial Intelligence (AI Act)*. Official Journal of the European Union, L 2024/1689.

European Parliament, and Council of the European Union. 2016. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016*. Official Journal of the European Union.

European Space Agency. 1996. *Ariane 501: Presentation of Inquiry Board Report*. European Space Agency press release.

Everingham, Mark, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. 2009. "The Pascal Visual Object Classes (VOC) Challenge." *International Journal of Computer Vision* 88 (2): 303–38. https://doi.org/10.1007/s11263-009-0275-4.

Fedus, William, Barret Zoph, and Noam Shazeer. 2022. "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity." *Journal of Machine Learning Research* 23 (120): 1–39.

Feigenbaum, Edward A. 1984. "Knowledge Engineering: The Applied Side of Artificial Intelligence." *Annals of the New York Academy of Sciences* 426: 91–107. https://doi.org/10.1111/j.1749-6632.1984.tb16513.x.

Feng, Wu-chun, and Kirk W. Cameron. 2007. "The Green500 List: Encouraging Sustainable Supercomputing." *Computer* 40 (12): 50–55. https://doi.org/10.1109/MC.2007.445.

Fitzpatrick, Thomas B. 1988. "The Validity and Practicality of Sun-Reactive Skin Types i Through VI." *Archives of Dermatology* 124 (6): 869. https://doi.org/10.1001/archderm.1988.01670060015008.

FlatBuffers Authors. 2026. *FlatBuffers Documentation*.

Fleiss, Joseph L. 1971. "Measuring Nominal Scale Agreement Among Many Raters." *Psychological Bulletin* 76 (5): 378–82. https://doi.org/10.1037/h0031619.

Flynn, M. J. 1966. "Very High-Speed Computing Systems." *Proceedings of the IEEE* 54 (12): 1901–9. https://doi.org/10.1109/proc.1966.5273.

Fowler, Martin. 2014. *Canary Release*. Martin Fowler's Blog.

Frankle, Jonathan, and Michael Carbin. 2019. "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks." *International Conference on Learning Representations (ICLR)*.

Fredrikson, Matt, Somesh Jha, and Thomas Ristenpart. 2015. "Model Inversion Attacks That Exploit Confidence Information and Basic Countermeasures." *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 1322–33. https://doi.org/10.1145/2810103.2813677.

Frostig, Roy, Matthew James Johnson, and Chris Leary. 2018. "Compiling Machine Learning Programs via High-Level Tracing." *Systems for Machine Learning*.

Fukushima, Kunihiko. 1980. "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position." *Biological Cybernetics* 36 (4): 193–202. https://doi.org/10.1007/BF00344251.

Gadre, Samir Yitzhak, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, et al. 2023. "DataComp: In Search of the Next Generation of Multimodal Datasets." *Advances in Neural Information Processing Systems (NeurIPS)* 36: 27092–112. https://doi.org/10.52202/075280-1179.

Gale, Trevor, Erich Elsen, and Sara Hooker. 2019. "The State of Sparsity in Deep Neural Networks." *arXiv Preprint arXiv:1902.09574*.

Gale, Trevor, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2022. "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts." *arXiv Preprint*.

Gale, Trevor, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. "Sparse GPU Kernels for Deep Learning." *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* 2020: 8955–67. https://doi.org/10.1109/sc41405.2020.00021.

Gama, João, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. "A Survey on Concept Drift Adaptation." *ACM Computing Surveys* 46 (4): 1–37. https://doi.org/10.1145/2523813.

Gao, Leo, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, et al. 2020. "The Pile: An 800GB Dataset of Diverse Text for Language Modeling." *arXiv Preprint arXiv:2101.00027*.

Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. "Datasheets for Datasets." *Communications of the ACM* 64 (12): 86–92. https://doi.org/10.1145/3458723.

Gholami, Amir, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. "A Survey of Quantization Methods for Efficient Neural Network Inference." *arXiv Preprint arXiv:2103.13630* abs/2103.13630.

Gholami, Amir, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2022. "A Survey of Quantization Methods for Efficient Neural Network Inference." In *Low-Power Computer Vision: Improve the Efficiency of Artificial Intelligence*. Chapman; Hall/CRC. https://doi.org/10.1201/9781003162810-13.

Gholami, Amir, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. "AI and Memory Wall." *IEEE Micro* 44 (3): 33–39. https://doi.org/10.1109/mm.2024.3373763.

Ginsberg, Jeremy, Matthew H. Mohebbi, Rajan S. Patel, Lynnette Brammer, Mark S. Smolinski, and Larry Brilliant. 2009. "Detecting Influenza Epidemics Using Search Engine Query Data." *Nature* 457 (7232): 1012–14. https://doi.org/10.1038/nature07634.

GitHub, Inc. 2024a. *GitHub*.

GitHub, Inc. 2024b. *GitHub Actions*.

Glorot, Xavier, and Yoshua Bengio. 2010. "Understanding the Difficulty of Training Deep Feedforward Neural Networks." *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* 9: 249–56.

Glorot, Xavier, Antoine Bordes, and Yoshua Bengio. 2011. *Deep Sparse Rectifier Neural Networks*. International Conference on Artificial Intelligence and Statistics (AISTATS).

Gojek, and Google. 2019. *Feast: An Open Source Feature Store for Machine Learning*. Google Cloud Blog.

Goodfellow, I. J., J. Shlens, and C. Szegedy. 2015. "Explaining and Harnessing Adversarial Examples." *International Conference on Learning Representations (ICLR)*.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.

Goodhart, Charles A. E. 1984. "Problems of Monetary Management: The UK Experience." In *Monetary Theory and Practice*. Palgrave. https://doi.org/10.1007/978-1-349-17295-5_4.

Google. 2024a. *Our Next-Generation Model: Gemini 1.5*.

Google. 2024b. *Static Vs. Dynamic Inference*. Google Machine Learning Crash Course.

Google. 2025. *XLA: Optimizing Compiler for Machine Learning*.

Google Cloud. 2024. *Vertex AI Model Registry*.

Google Cloud. 2026a. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*.

Google Cloud. 2026b. *TPU v6e*.

Google Developers Blog. 2024. *Gemini 1.5 Flash Price Drop with Tuning Rollout Complete, and More*.

Gordon, Mitchell, Kevin Duh, and Nicholas Andrews. 2020. "Compressing BERT: Studying the Effects of Weight Pruning on Transfer Learning." *Proceedings of the 5th Workshop on Representation Learning for NLP*, 143–55. https://doi.org/10.18653/v1/2020.repl4nlp-1.18.

Goto, Kazushige, and Robert A. van de Geijn. 2008. "Anatomy of High-Performance Matrix Multiplication." *ACM Transactions on Mathematical Software* 34 (3): 1–25. https://doi.org/10.1145/1356052.1356053.

Gou, Jianping, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. 2021. "Knowledge Distillation: A Survey." *International Journal of Computer Vision* 129 (6): 1789–819. https://doi.org/10.1007/s11263-021-01453-z.

Goyal, Priya, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2017. "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour." *arXiv Preprint arXiv:1706.02677* abs/1706.02677.

Graphcore. 2020. *The Colossus MK2 IPU Processor*.

Gray, Allison. 2015. *NVIDIA and IBM Cloud Support ImageNet Large Scale Visual Recognition Challenge*. NVIDIA Technical Blog.

Gretton, Arthur, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. 2012. "A Kernel Two-Sample Test." *Journal of Machine Learning Research* 13 (25): 723–73.

Griewank, Andreas, and Andrea Walther. 2000. "Algorithm 799: Revolve: An Implementation of Checkpointing for the Reverse or Adjoint Mode of Computational Differentiation." *ACM Transactions on Mathematical Software* 26 (1): 19–45. https://doi.org/10.1145/347837.347846.

Griewank, Andreas, and Andrea Walther. 2008. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2nd ed. SIAM. https://doi.org/10.1137/1.9780898717761.

Groeneveld, Dirk, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, et al. 2024. "OLMo: Accelerating the Science of Language Models." *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15789–809. https://doi.org/10.18653/v1/2024.acl-long.841.

gRPC Authors. 2026. *Introduction to gRPC*.

Gu, Albert, and Tri Dao. 2023. "Mamba: Linear-Time Sequence Modeling with Selective State Spaces." *arXiv Preprint arXiv:2312.00752*.

Gudivada, Venkat N., Amy Apon, and Junhua Ding. 2017. "Data Quality Considerations for Big Data and Machine Learning: Going Beyond Data Cleaning and Transformations." *International Journal on Advances in Software* 10 (1–2): 1–20.

Gujarati, Arpan, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. "Clockwork: Predictable and Scalable DNN Inference in the Cloud." *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 443–62.

Gulshan, Varun, Lily Peng, Marc Coram, Martin C. Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, et al. 2016. "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs." *JAMA* 316 (22): 2402. https://doi.org/10.1001/jama.2016.17216.

Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. "On Calibration of Modern Neural Networks." *International Conference on Machine Learning (ICML)* 70: 1321–30.

Gupta, Suyog, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. "Deep Learning with Limited Numerical Precision." *International Conference on Machine Learning (ICML)*, 1737–46.

Gupta, Udit, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2022. "Chasing Carbon: The Elusive Environmental Footprint of Computing." *IEEE Micro* 42 (4): 37–47. https://doi.org/10.1109/MM.2022.3163226.

Gupta, Udit, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, et al. 2020. "The Architectural Implications of Facebook's DNN-Based Personalized Recommendation." *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 488–501. https://doi.org/10.1109/HPCA47549.2020.00047.

Gustafson, John L. 1988. "Reevaluating Amdahl's Law." *Communications of the ACM* 31 (5): 532–33. https://doi.org/10.1145/42411.42415.

Halevy, Alon, Peter Norvig, and Fernando Pereira. 2009. "The Unreasonable Effectiveness of Data." *IEEE Intelligent Systems* 24 (2): 8–12. https://doi.org/10.1109/mis.2009.36.

Han, Song, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. "EIE: Efficient Inference Engine on Compressed Deep Neural Network." *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 243–54. https://doi.org/10.1109/isca.2016.30.

Han, Song, Huizi Mao, and William J. Dally. 2016. "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding." *arXiv Preprint*.

Han, Song, Jeff Pool, John Tran, and William J. Dally. 2015. "Learning Both Weights and Connections for Efficient Neural Networks." *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, 1135–43.

Hanley, James A., and Abby Lippman-Hand. 1983. "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators." *JAMA* 249 (13): 1743–45.

Harchol-Balter, Mor. 2013. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press. https://doi.org/10.1017/cbo9781139226424.

Hardt, Moritz, Eric Price, and Nathan Srebro. 2016. "Equality of Opportunity in Supervised Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 29: 3315–23.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. "Array Programming with NumPy." *Nature* 585 (7825): 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Harvard Law School. 2024. *Does ChatGPT Violate New York Times Copyrights?*

HashiCorp. 2014. *Terraform: Infrastructure as Code*. Software available from https://www.terraform.io/.

Hazelwood, Kim, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, et al. 2018. "Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective." *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 620–29. https://doi.org/10.1109/hpca.2018.00059.

He, Kaiming, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. "Momentum Contrast for Unsupervised Visual Representation Learning." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9726–35. https://doi.org/10.1109/cvpr42600.2020.00975.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." *2015 IEEE International Conference on Computer Vision (ICCV)*, 1026–34. https://doi.org/10.1109/iccv.2015.123.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016a. "Deep Residual Learning for Image Recognition." *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–78. https://doi.org/10.1109/cvpr.2016.90.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016b. "Identity Mappings in Deep Residual Networks." *Computer Vision – ECCV 2016*, 630–45. https://doi.org/10.1007/978-3-319-46493-0_38.

Henderson, Peter, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. 2020. "Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning." *CoRR* abs/2002.05651 (248): 1–43. https://doi.org/10.48550/arxiv.2002.05651.

Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. "Deep Reinforcement Learning That Matters." *Proceedings of the AAAI Conference on Artificial Intelligence* 32 (1): 3207–14. https://doi.org/10.1609/aaai.v32i1.11694.

Hendler, James A. 2008. "Avoiding Another AI Winter." *IEEE Intelligent Systems* 23 (2): 2–4. https://doi.org/10.1109/MIS.2008.20.

Hendrycks, Dan, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. "Measuring Massive Multitask Language Understanding." *arXiv Preprint*.

Hendrycks, Dan, and Kevin Gimpel. 2016. "Gaussian Error Linear Units (GELUs)." *arXiv Preprint arXiv:1606.08415*.

Hennessy, J. L., and D. A. Patterson. 2011. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.

Hennessy, John L., and David A. Patterson. 2017. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann.

Hennessy, John L., and David A. Patterson. 2019. "A New Golden Age for Computer Architecture." *Communications of the ACM* 62 (2): 48–60. https://doi.org/10.1145/3282307.

Hermann, Jeremy, and Mike Del Balso. 2017. *Meet Michelangelo: Uber's Machine Learning Platform*. Uber Engineering Blog.

Hernandez, Danny, and Tom B. Brown. 2020. "Measuring the Algorithmic Efficiency of Neural Networks." *arXiv Preprint arXiv:2005.04305*, ahead of print. https://doi.org/10.48550/arxiv.2005.04305.

Hestness, Joel, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. 2017. "Deep Learning Scaling Is Predictable, Empirically." *arXiv Preprint arXiv:1712.00409*.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. "Distilling the Knowledge in a Neural Network." *arXiv Preprint*.

Hirschberg, Julia, and Christopher D. Manning. 2015. "Advances in Natural Language Processing." *Science* 349 (6245): 261–66. https://doi.org/10.1126/science.aaa8685.

Hochreiter, Sepp, and Jürgen Schmidhuber. 1997. "Long Short-Term Memory." *Neural Computation* 9 (8): 1735–80. https://doi.org/10.1162/neco.1997.9.8.1735.

Hoefler, Torsten, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. 2021. "Sparsity in Deep Learning: Pruning and Growth for Efficient Inference and Training in Neural Networks." *Journal of Machine Learning Research* 22 (241): 1–124.

Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, et al. 2022. "Training Compute-Optimal Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 30016–30. https://doi.org/10.52202/068431-2176.

Holtzman, Ari, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. "The Curious Case of Neural Text Degeneration." *Proceedings of the 8th International Conference on Learning Representations (ICLR 2020)*.

Hooker, Sara. 2021. "The Hardware Lottery." *Communications of the ACM* 64 (12): 58–65. https://doi.org/10.1145/3467017.

Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. "Multilayer Feedforward Networks Are Universal Approximators." *Neural Networks* 2 (5): 359–66. https://doi.org/10.1016/0893-6080(89)90020-8.

Horowitz, Mark. 2014. "1.1 Computing's Energy Problem (and What We Can Do about It)." *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 10–14. https://doi.org/10.1109/isscc.2014.6757323.

Howard, Andrew G., Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." *CoRR* abs/1704.04861.

Howard, Andrew, Mark Sandler, Bo Chen, Weijun Wang, Liang-Chieh Chen, Mingxing Tan, Grace Chu, et al. 2019. "Searching for MobileNetV3." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 1314–24. https://doi.org/10.1109/iccv.2019.00140.

Howarth, Jesse. 2020. "Why Discord Is Switching from Go to Rust."

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. "LoRA: Low-Rank Adaptation of Large Language Models." *arXiv Preprint arXiv:2106.09685*.

Hu, Jie, Peng Lin, Huajun Zhang, Zining Lan, Wenxin Chen, Kailiang Xie, Siyun Chen, Hao Wang, and Sheng Chang. 2023. "A Dynamic Pruning Method on Multiple Sparse Structures in Deep Neural Networks." *IEEE Access* 11: 38448–57. https://doi.org/10.1109/access.2023.3267469.

Hu, Ting-Kuei, Tianlong Chen, Haotao Wang, and Zhangyang Wang. 2020. "Triple Wins: Boosting Accuracy, Robustness and Efficiency Together by Enabling Input-Adaptive Inference." *International Conference on Learning Representations (ICLR)*.

Huang, Gao, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. 2017. "Densely Connected Convolutional Networks." *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2261–69. https://doi.org/10.1109/cvpr.2017.243.

Huang, Y., Y. Cheng, A. Bapna, O. Firat, D. Chen, M. X. Chen, H. Lee, et al. 2019. "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 103–12.

Hubara, Itay, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2018. "Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations." *Journal of Machine Learning Research* 18 (187): 1–30.

Hugging Face. 2026. *Safetensors Documentation*.

HumanSignal. 2024. *Label Studio: Open Source Data Labeling Platform*.

Hutter, Frank, Lars Kotthoff, and Joaquin Vanschoren. 2019. *Automated Machine Learning: Methods, Systems, Challenges*. In *Automated Machine Learning*. The Springer Series on Challenges in Machine Learning. Springer International Publishing. https://doi.org/10.1007/978-3-030-05318-5.

Hwu, Wen-mei W. 2011. "Introduction." In *GPU Computing Gems Emerald Edition*. Elsevier. https://doi.org/10.1016/b978-0-12-384988-5.00064-4.

Iandola, Forrest N., Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. "SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and <0.5MB Model Size." *ArXiv Preprint* abs/1602.07360.

IBM. 2024. *IBM Watson OpenScale: Data Drift Detection*.

Ignatov, Andrey, and Radu Timofte. 2024. *AI Benchmark*.

InfiniBand Trade Association. 2000. *InfiniBand Architecture Specification Volume 1*. InfiniBand Trade Association.

Intel Corporation. 2021a. "Intel Advanced Matrix Extensions (Intel AMX)." *Intel Architecture Instruction Set Extensions Programming Reference*.

Intel Corporation. 2021b. *oneDNN: Intel's Deep Learning Neural Network Library*.

Intel Corporation. 2026a. *Intel oneAPI Math Kernel Library*.

Intel Corporation. 2026b. *OpenVINO Documentation*.

Ioffe, Sergey, and Christian Szegedy. 2015. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." *Proceedings of the 32nd International Conference on Machine Learning (ICML)* 37: 448–56.

Iterative. 2024. *Data Version Control (DVC)*.

Jacob, Benoit, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–13. https://doi.org/10.1109/cvpr.2018.00286.

Jacobs, Robert A., Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. "Adaptive Mixtures of Local Experts." *Neural Computation* 3 (1): 79–87. https://doi.org/10.1162/neco.1991.3.1.79.

Janapa Reddi, Vijay et al. 2022. "MLPerf Mobile V2. 0: An Industry-Standard Benchmark Suite for Mobile Machine Learning." *Proceedings of Machine Learning and Systems (MLSys)* 4: 806–23.

Janapa Reddi, Vijay, Brian Plancher, Susan Kennedy, Laurence Moroney, Pete Warden, Lara Suzuki, Anant Agarwal, et al. 2022. "Widening Access to Applied Machine Learning with TinyML." *Harvard Data Science Review* 4 (1). https://doi.org/10.1162/99608f92.762d171a.

JAX Developers. 2025. *JAX Lax Primitives*.

Jevons, William Stanley. 1865. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal Mines*. Macmillan; Co.

Jia, Yangqing, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. "Caffe: Convolutional Architecture for Fast Feature Embedding." *Proceedings of the 22nd ACM International Conference on Multimedia*, 675–78. https://doi.org/10.1145/2647868.2654889.

Jia, Zhihao, James Thomas, Todd Warszawski, Mingyu Gao, Matei Zaharia, and Alex Aiken. 2019. "Optimizing DNN Computation with Relaxed Graph Substitutions." *Proceedings of Machine Learning and Systems (MLSys)*.

Jiang, A. Q., A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, et al. 2024. "Mixtral of Experts." *arXiv Preprint arXiv:2401.04088*.

Jiao, Xiaoqi, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. "TinyBERT: Distilling BERT for Natural Language Understanding." *Findings of the Association for Computational Linguistics: EMNLP 2020*, 4163–74. https://doi.org/10.18653/v1/2020.findings-emnlp.372.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2019. "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data* 7 (3): 535–47. https://doi.org/10.1109/tbdata.2019.2921572.

Johnston, Casey. 2012. "Netflix Never Used Its $1 Million Algorithm Due to Engineering Costs." *Ars Technica*, April.

Jordan, T. L. 1982. "A Guide to Parallel Computation and Some Cray-1 Experiences." In *Parallel Computations*. Elsevier. https://doi.org/10.1016/b978-0-12-592101-5.50006-3.

Jouppi, Norman P., Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, et al. 2021. "Ten Lessons from Three Generations Shaped Google's TPUv4i: Industrial Product." *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 1–14. https://doi.org/10.1109/isca52012.2021.00010.

Jouppi, Norman P., Doe Hyun Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. 2020. "A Domain-Specific Supercomputer for Training Deep Neural Networks." *Communications of the ACM* 63 (7): 67–78. https://doi.org/10.1145/3360307.

Jouppi, Norman P., Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, et al. 2017. "In-Datacenter Performance Analysis of a Tensor Processing Unit." *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ISCA '17, 1–12. https://doi.org/10.1145/3079856.3080246.

Jouppi, Norm, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, et al. 2023. "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings." *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 1–14. https://doi.org/10.1145/3579371.3589350.

Kalamkar, Dhiraj, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, et al. 2019. *A Study of BFLOAT16 for Deep Learning Training*.

Kamiran, Faisal, and Toon Calders. 2012. "Data Preprocessing Techniques for Classification Without Discrimination." *Knowledge and Information Systems* 33 (1): 1–33. https://doi.org/10.1007/s10115-011-0463-8.

Kang, Yiping, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. 2017. "Neurosurgeon." *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 615–29. https://doi.org/10.1145/3037697.3037698.

Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. "Scaling Laws for Neural Language Models." *ArXiv Preprint* abs/2001.08361.

Karl Rupp. 2022. *Microprocessor Trend Data*.

Karpathy, Andrej. 2017. *Software 2.0*. Medium.

Kaufman, Shachar, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. "Leakage in Data Mining: Formulation, Detection, and Avoidance." *ACM Transactions on Knowledge Discovery from Data (TKDD)* 6 (4): 1–21.

Kendall, David G. 1953. "Stochastic Processes Occurring in the Theory of Queues and Their Analysis by the Method of the Imbedded Markov Chain." *The Annals of Mathematical Statistics* 24 (3): 338–54. https://doi.org/10.1214/aoms/1177728975.

Kiela, Douwe, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, et al. 2021. "Dynabench: Rethinking Benchmarking in NLP." *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* 9: 4110–24. https://doi.org/10.18653/v1/2021.naacl-main.324.

Kilburn, Tom, David B. G. Edwards, Michael J. Lanigan, and Frank H. Sumner. 1962. "One-Level Storage System." *IRE Transactions on Electronic Computers* EC-11 (2): 223–35. https://doi.org/10.1109/TEC.1962.5219356.

Kim, Wonyoung, Meeta S. Gupta, Gu-Yeon Wei, and David Brooks. 2008. "System Level Analysis of Fast, Per-Core DVFS Using on-Chip Switching Regulators." *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, HPCA '08, 123–34. https://doi.org/10.1109/hpca.2008.4658633.

Kingma, Diederik P., and Jimmy Ba. 2015. "Adam: A Method for Stochastic Optimization." *3rd International Conference on Learning Representations (ICLR)*.

Kipf, Thomas N, and Max Welling. 2017. "Semi-Supervised Classification with Graph Convolutional Networks." *International Conference on Learning Representations (ICLR)*.

Kleinberg, Jon, Sendhil Mullainathan, and Manish Raghavan. 2017. "Inherent Trade-Offs in the Fair Determination of Risk Scores." *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, Leibniz international proceedings in informatics (LIPIcs), vol. 67: 43:1–23. https://doi.org/10.4230/LIPIcs.ITCS.2017.43.

Kleppmann, Martin. 2016. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Knight, Will. 2023. *OpenAI's CEO Says the Age of Giant AI Models Is Already over*. WIRED.

Koh, Pang Wei, and Percy Liang. 2017. "Understanding Black-Box Predictions via Influence Functions." *International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 70: 1885–94.

Koh, Pang Wei, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, et al. 2021. "WILDS: A Benchmark of in-the-Wild Distribution Shifts." In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, edited by Marina Meila and Tong Zhang, vol. 139. Proceedings of Machine Learning Research. PMLR.

Kohavi, Ron, Diane Tang, and Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press. https://doi.org/10.1017/9781108653985.

Koizumi, Yuma, Shoichiro Saito, Hisashi Uematsu, Noboru Harada, and Keisuke Imoto. 2019. "ToyADMOS: A Dataset of Miniature-Machine Operating Sounds for Anomalous Sound Detection." *2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 313–17. https://doi.org/10.1109/waspaa.2019.8937164.

Koomey, Jonathan, Stephen Berard, Marla Sanchez, and Henry Wong. 2011. "Implications of Historical Trends in the Electrical Efficiency of Computing." *IEEE Annals of the History of Computing* 33 (3): 46–54. https://doi.org/10.1109/mahc.2010.28.

Korthikanti, Vijay Anand, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. "Reducing Activation Recomputation in Large Transformer Models." *Proceedings of Machine Learning and Systems (MLSys)* 5: 341–53.

Kreuzberger, Dominik, Niklas Kühl, and Sebastian Hirschl. 2023. "Machine Learning Operations (MLOps): Overview, Definition, and Architecture." *IEEE Access* 11: 31866–79. https://doi.org/10.1109/access.2023.3262138.

Krishnamoorthi, Raghuraman. 2018. "Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper." *arXiv Preprint arXiv:1806.08342* abs/1806.08342.

Krishnan, Rayan, Pranav Rajpurkar, and Eric J. Topol. 2022. "Self-Supervised Learning in Medicine and Healthcare." *Nature Biomedical Engineering* 6 (12): 1346–52. https://doi.org/10.1038/s41551-022-00914-1.

Krizhevsky, Alex. 2009. *Learning Multiple Layers of Features from Tiny Images*. University of Toronto.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 25.

Kuhn, Max, and Kjell Johnson. 2013. *Applied Predictive Modeling*. Springer New York. https://doi.org/10.1007/978-1-4614-6849-3.

Kullback, Solomon, and Richard A. Leibler. 1951. "On Information and Sufficiency." *The Annals of Mathematical Statistics* 22 (1): 79–86. https://doi.org/10.1214/aoms/1177729694.

Kung, H. T. 1982. "Why Systolic Architectures?" *Computer* 15 (1): 37–46. https://doi.org/10.1109/mc.1982.1653825.

Kung, Hsiang Tsung, and Charles E Leiserson. 1979. "Systolic Arrays (for VLSI)." *Sparse Matrix Proceedings 1978* 1: 256–82.

Kuzmin, Andrey, Mart Van Baalen, Yuwei Ren, Markus Nagel, Jorn Peters, and Tijmen Blankevoort. 2022. "FP8 Quantization: The Power of the Exponent." *Advances in Neural Information Processing Systems (NeurIPS)*, 14651–62. https://doi.org/10.52202/068431-1065.

Kuznetsova, Alina, Hassan Rom, Neil Alldrin, Jasper Uijlings, Ivan Krasin, Jordi Pont-Tuset, Shahab Kamali, et al. 2020. "The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale." *International Journal of Computer Vision* 128 (7): 1956–81. https://doi.org/10.1007/s11263-020-01316-z.

Kwon, Woosuk, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–26. https://doi.org/10.1145/3600006.3613165.

Kwon, Youngeun, and Minsoo Rhu. 2018. "TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning." *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 740–53. https://doi.org/10.1145/3352460.3358284.

Labelbox, Inc. 2024. *Labelbox Consensus Documentation*.

Labs, Grafana. 2024. *Grafana*.

Lai, Liangzhen, Naveen Suda, and Vikas Chandra. 2018. "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs." *ArXiv Preprint* abs/1801.06601.

Lam, Monica D., Edward E. Rothberg, and Michael E. Wolf. 1991. "The Cache Performance and Optimizations of Blocked Algorithms." *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS-IV*, 63–74. https://doi.org/10.1145/106972.106981.

Lampson, Butler W. 1983. "Hints for Computer System Design." *Proceedings of the Ninth ACM Symposium on Operating Systems Principles*, 33–48. https://doi.org/10.1145/800217.806614.

Landis, J. Richard, and Gary G. Koch. 1977. "The Measurement of Observer Agreement for Categorical Data." *Biometrics* 33 (1): 159–74. https://doi.org/10.2307/2529310.

Lange, Klaus-Dieter. 2009. "Identifying Shades of Green: The SPECpower Benchmarks." *IEEE Computer* 42 (3): 95–97. https://doi.org/10.1109/mc.2009.84.

Lattner, Chris, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation." *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2–14. https://doi.org/10.1109/cgo51591.2021.9370308.

Lawson, Charles L., Richard J. Hanson, David R. Kincaid, and Fred T. Krogh. 1979. "Basic Linear Algebra Subprograms for Fortran Usage." *ACM Transactions on Mathematical Software* 5 (3): 308–23. https://doi.org/10.1145/355841.355847.

Lazer, David, Ryan Kennedy, Gary King, and Alessandro Vespignani. 2014. "The Parable of Google Flu: Traps in Big Data Analysis." *Science* 343 (6176): 1203–5. https://doi.org/10.1126/science.1248506.

Le Sueur, Etienne, and Gernot Heiser. 2010. "Dynamic Voltage and Frequency Scaling: The Laws of Diminishing Returns." *Proceedings of the 2010 International Conference on Power Aware Computing and Systems*, 1–8.

Lebedev, Vadim, Yaroslav Ganin, Maksim Rakhuba, Ivan Oseledets, and Victor Lempitsky. 2015. "Speeding up Convolutional Neural Networks Using Fine-Tuned CP-Decomposition." *3rd International Conference on Learning Representations (ICLR)*.

Leclerc, Guillaume, Andrew Ilyas, Logan Engstrom, Sung Min Park, Hadi Salman, and Aleksander Mądry. 2023. "FFCV: Accelerating Training by Removing Data Bottlenecks." *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 12011–20. https://doi.org/10.1109/cvpr52729.2023.01156.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. 2015. "Deep Learning." *Nature* 521 (7553): 436–44. https://doi.org/10.1038/nature14539.

LeCun, Yann, Bernhard Boser, John S. Denker, Donald Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. 1989. "Backpropagation Applied to Handwritten Zip Code Recognition." *Neural Computation* 1 (4): 541–51. https://doi.org/10.1162/neco.1989.1.4.541.

LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE* 86 (11): 2278–324. https://doi.org/10.1109/5.726791.

LeCun, Yann, Leon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. 2012. "Efficient BackProp." In *Neural Networks: Tricks of the Trade*, vol. 7700. Lecture Notes in Computer Science. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-35289-8_3.

LeCun, Yann, John S. Denker, and Sara A. Solla. 1989. "Optimal Brain Damage." *Advances in Neural Information Processing Systems 2 (NIPS 1989)*, 598–605.

Lee, Katherine, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. "Deduplicating Training Data Makes Language Models Better." *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8424–45. https://doi.org/10.18653/v1/2022.acl-long.577.

Lee, Peter. 2016. *Learning from Tay's Introduction*. The Official Microsoft Blog.

Lepikhin, Dmitry, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding." *International Conference on Learning Representations (ICLR)*.

Leviathan, Yaniv, Matan Kalman, and Yossi Matias. 2023. "Fast Inference from Transformers via Speculative Decoding." *International Conference on Machine Learning (ICML)*, 19274–86.

Li, Hao, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2017. "Pruning Filters for Efficient ConvNets." *International Conference on Learning Representations (ICLR)*.

Li, Mingzhen, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. "The Deep Learning Compiler: A Comprehensive Survey." *IEEE Transactions on Parallel and Distributed Systems* 32 (3): 708–27. https://doi.org/10.1109/tpds.2020.3030548.

Li, Ninghui, Tiancheng Li, and Suresh Venkatasubramanian. 2007. "t-Closeness: Privacy Beyond k-Anonymity and l-Diversity." *2007 IEEE 23rd International Conference on Data Engineering*, 106–15. https://doi.org/10.1109/ICDE.2007.367856.

Liang, Percy, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, et al. 2022. "Holistic Evaluation of Language Models." *arXiv Preprint arXiv:2211.09110*.

Lighthill, James. 1973. "Artificial Intelligence: A General Survey." In *Artificial Intelligence: A Paper Symposium*. Science Research Council.

Lin, Ji, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. "MCUNet: Tiny Deep Learning on IoT Devices." In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, Virtual*, edited by Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin. Curran Associates.

Lin, Ji, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. "AWQ: Activation-Aware Weight Quantization for on-Device LLM Compression and Acceleration." *Proceedings of Machine Learning and Systems* 6: 87–100.

Lin, Tsung-Yi, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014. "Microsoft COCO: Common Objects in Context." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-319-10602-1_48.

Lindholm, Erik, John Nickolls, Stuart Oberman, and John Montrym. 2008. "NVIDIA Tesla: A Unified Graphics and Computing Architecture." *IEEE Micro* 28 (2): 39–55. https://doi.org/10.1109/mm.2008.31.

Linnainmaa, Seppo. 1970. "The Representation of the Cumulative Rounding Error of an Algorithm as a Taylor Expansion of the Local Rounding Errors." Master's thesis, University of Helsinki.

Lipton, Zachary, Yu-Xiang Wang, and Alexander Smola. 2018. "Detecting and Correcting for Label Shift with Black Box Predictors." *International Conference on Machine Learning (ICML)*, 3122–30.

Little, John D. C. 1961. "A Proof for the Queuing Formula: $L = \lambda W$." *Operations Research* 9 (3): 383–87. https://doi.org/10.1287/opre.9.3.383.

Liu, Hanxiao, Karen Simonyan, and Yiming Yang. 2019. "DARTS: Differentiable Architecture Search." *International Conference on Learning Representations (ICLR)*.

Liu, Ze, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows." *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 9992–10002. https://doi.org/10.1109/ICCV48922.2021.00986.

Liu, Zhuang, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. 2022. "A ConvNet for the 2020s." *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11966–76. https://doi.org/10.1109/CVPR52688.2022.01167.

Loshchilov, Ilya, and Frank Hutter. 2017. "SGDR: Stochastic Gradient Descent with Warm Restarts." *Proceedings of the International Conference on Learning Representations (ICLR)*.

Loshchilov, Ilya, and Frank Hutter. 2019. "Decoupled Weight Decay Regularization." *Proceedings of the International Conference on Learning Representations (ICLR)*.

Lowe, David G. 1999. "Object Recognition from Local Scale-Invariant Features." *Proceedings of the Seventh IEEE International Conference on Computer Vision* 2: 1150–1157 vol.2. https://doi.org/10.1109/iccv.1999.790410.

Lu, Yucheng, Shivani Agrawal, Suvinay Subramanian, Oleg Rybakov, Christopher De Sa, and Amir Yazdanbakhsh. 2023. "STEP: Learning n:m Structured Sparsity Masks from Scratch with Precondition." *arXiv Preprint*.

Lucic, Mario, Karol Kurach, Marcin Michalski, Sylvain Gelly, and Olivier Bousquet. 2018. "Are GANs Created Equal? A Large-Scale Study." *Advances in Neural Information Processing Systems (NeurIPS)* 31.

Lundberg, Scott M., and Su-In Lee. 2017. "A Unified Approach to Interpreting Model Predictions." *Advances in Neural Information Processing Systems (NeurIPS)* 30: 4765–74.

Lyons, Richard G. 2011. *Understanding Digital Signal Processing*. 3rd ed. Prentice Hall.

Machanavajjhala, Ashwin, Daniel Kifer, Johannes Gehrke, and Muthu Venkitasubramaniam. 2007. "l-Diversity: Privacy Beyond k-Anonymity." *ACM Transactions on Knowledge Discovery from Data* 1 (3): 3. https://doi.org/10.1145/1217299.1217302.

Mars Climate Orbiter Mishap Investigation Board. 1999. *Mars Climate Orbiter Mishap Investigation Board Phase i Report*. National Aeronautics; Space Administration.

Maslej, Nestor, Loredana Fattorini, C. Raymond Perrault, Vanessa Parli, Anka Reuel, Erik Brynjolfsson, John Etchemendy, et al. 2024. "Artificial Intelligence Index Report 2024." In *CoRR*, abs/2405.19522. https://doi.org/10.48550/ARXIV.2405.19522.

Mattson, Peter, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, et al. 2020. "MLPerf Training Benchmark." *Proceedings of Machine Learning and Systems (MLSys)* 2: 336–49.

Mazumder, Mark. 2021. *TinyML: Multilingual Spoken Words Corpus*. tinyML Talks.

Mazumder, Mark, Colby Banbury, Xiaozhe Yao, Bojan Karlaš, et al. 2023. "DataPerf: Benchmarks for Data-Centric AI Development." *Advances in Neural Information Processing Systems* 36.

Mazumder, Mark, Sharad Chitlangia, Colby Banbury, Yiping Kang, Juan Manuel Ciro, Keith Achorn, Daniel Galvez, et al. 2021. "Multilingual Spoken Words Corpus." *Advances in Neural Information Processing Systems 35 (NeurIPS 2021) Datasets and Benchmarks Track*.

McAuliffe, Michael, Michaela Socolof, Sarah Mihuc, Michael Wagner, and Morgan Sonderegger. 2017. "Montreal Forced Aligner: Trainable Text-Speech Alignment Using Kaldi." *Annual Conference of the International Speech Communication Association (Interspeech)*, 498–502. https://doi.org/10.21437/Interspeech.2017-1386.

McCandlish, Sam, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. 2018. "An Empirical Model of Large-Batch Training." *arXiv Preprint arXiv:1812.06162*, ahead of print. https://doi.org/10.48550/arXiv.1812.06162.

McCarthy, John, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. 1955. *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 31, 1955*. No. 4. Vol. 27. Dartmouth College. https://doi.org/10.1609/aimag.v27i4.1904.

McCrory, Dave. 2010. *Data Gravity – in the Clouds*.

McCulloch, Warren S., and Walter Pitts. 1943. "A Logical Calculus of the Ideas Immanent in Nervous Activity." *The Bulletin of Mathematical Biophysics* 5 (4): 115–33. https://doi.org/10.1007/bf02478259.

McKinsey Global Institute. 2021. *The Internet of Things: Catching up to an Accelerating Opportunity*. McKinsey & Company.

McMahan, Brendan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. "Communication-Efficient Learning of Deep Networks from Decentralized Data." *International Conference on Artificial Intelligence and Statistics (AISTATS)*, Proceedings of machine learning research, vol. 54: 1273–82.

McNemar, Quinn. 1947. "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages." *Psychometrika* 12 (2): 153–57. https://doi.org/10.1007/BF02295996.

Mellempudi, Naveen, Sudarshan Srinivasan, Dipankar Das, and Bharat Kaul. 2019. "Mixed Precision Training with 8-Bit Floating Point." *arXiv Preprint arXiv:1905.12334*.

Merity, Stephen. 2016. *The WikiText Long Term Dependency Language Modeling Dataset*.

Merity, Stephen, Caiming Xiong, James Bradbury, and Richard Socher. 2016. "Pointer Sentinel Mixture Models." *arXiv Preprint arXiv:1609.07843*.

Merkel, Dirk. 2014. "Docker: Lightweight Linux Containers for Consistent Development and Deployment." *Linux Journal* 2014 (239).

Michel, Jean-Baptiste, Yuan Kui Shen, Aviva Presser Aiden, Adrian Veres, Matthew K. Gray, William Brockman, Joseph P. Pickett, et al. 2011. "Quantitative Analysis of Culture Using Millions of Digitized Books." *Science* 331 (6014): 176–82. https://doi.org/10.1126/science.1199644.

Michel, Paul, Omer Levy, and Graham Neumann. 2019. "Are Sixteen Heads Really Better Than One?" *Advances in Neural Information Processing Systems (NeurIPS)*.

Micikevicius, Paulius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, et al. 2018. "Mixed Precision Training." *International Conference on Learning Representations (ICLR)*.

Micikevicius, Paulius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, et al. 2022. "FP8 Formats for Deep Learning." *arXiv Preprint arXiv:2209.05433*.

Microsoft. 2024. *ONNX Runtime: Cross-Platform Inference and Training Machine-Learning Accelerator*. GitHub.

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Efficient Estimation of Word Representations in Vector Space." *ICLR* abs/1301.3781.

Mila. 2026. *Open Source Software*.

Minsky, Marvin. 1961. "Steps Toward Artificial Intelligence." *Proceedings of the IRE* 49 (1): 8–30. https://doi.org/10.1109/JRPROC.1961.287775.

Minsky, Marvin, and Seymour A. Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. The MIT Press.

Mirhoseini, Azalia, Hieu Pham, Quoc V. Le, Benoit Steiner, Rasmus Larsen, Yuefeng Zhou, Naveen Kumar, Mohammad Norouzi, Samy Bengio, and Jeff Dean. 2017. "Device Placement Optimization with Reinforcement Learning." *International Conference on Machine Learning (ICML)* 70: 2430–39.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. "Model Cards for Model Reporting." *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–29. https://doi.org/10.1145/3287560.3287596.

Mitchell, Tom M. 1980. *The Need for Biases in Learning Generalizations*. CBM-TR-117. Rutgers University, Department of Computer Science.

MLCommons. 2024a. *MLPerf Mobile Benchmark*.

MLCommons. 2024b. *MLPerf Power Measurement*.

MLCommons. 2024c. *MLPerf Training Benchmark*.

MLCommons. 2026a. *MLCommons: About Us*.

MLCommons. 2026b. *MLPerf Client Benchmark*.

MLflow Project. 2026. *MLflow Model Registry*.

Mohan, Jayashree, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. 2021. "Analyzing and Mitigating Data Stalls in DNN Training." *Proceedings of the VLDB Endowment* 14 (5): 771–84. https://doi.org/10.14778/3446095.3446100.

Moore, G. E. 1998. "Cramming More Components onto Integrated Circuits." *Proceedings of the IEEE* 86 (1): 82–85. https://doi.org/10.1109/jproc.1998.658762.

Moravec, Hans. 1988. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press.

Moritz, Philipp, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, et al. 2018. "Ray: A Distributed Framework for Emerging AI Applications." *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 561–77.

Mosseri, Adam. 2018. *Bringing People Closer Together*. Meta Newsroom.

Mudigere, Dheevatsa, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, et al. 2022. "Software-Hardware Co-Design for Fast and Scalable Training of Deep Learning Recommendation Models." *International Symposium on Computer Architecture (ISCA)*, 993–1011. https://doi.org/10.1145/3470496.3533727.

Murray, Derek G., Jiří Šimša, Ana Klimovic, and Ihor Indyk. 2021. "Tf.data: A Machine Learning Data Processing Framework." *Proceedings of the VLDB Endowment* 14 (12): 2945–58. https://doi.org/10.14778/3476311.3476374.

Nair, Vinod, and Geoffrey E. Hinton. 2010. "Rectified Linear Units Improve Restricted Boltzmann Machines." *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 807–14.

Nakkiran, Preetum, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. 2019. "Deep Double Descent: Where Bigger Models and More Data Hurt." *arXiv Preprint arXiv:1912.02292*.

Narayanan, Deepak, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. "PipeDream: Generalized Pipeline Parallelism for DNN Training." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 1–15. https://doi.org/10.1145/3341301.3359646.

Narayanan, Deepak, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, et al. 2021. "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15. https://doi.org/10.1145/3458817.3476209.

National Aeronautics and Space Administration. 2016. *NASA Systems Engineering Handbook*. NASA/SP-2016-6105 Rev2. National Aeronautics; Space Administration.

National Transportation Safety Board. 2017. *Collision Between a Car Operating with Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016*. HAR-17/02. National Transportation Safety Board.

National Transportation Safety Board. 2019. *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. HAR-19/03. National Transportation Safety Board.

Naumov, Maxim, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, et al. 2019. "Deep Learning Recommendation Model for Personalization and Recommendation Systems." *arXiv Preprint arXiv:1906.00091*.

Naur, Peter, and Brian Randell, eds. 1969. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968*. Scientific Affairs Division, NATO.

Nayak, Prateeth, Takuya Higuchi, Anmol Gupta, Shivesh Ranjan, Stephen Shum, Siddharth Sigtia, Erik Marchi, et al. 2022. "Improving Voice Trigger Detection with Metric Learning." *Interspeech 2022*, 1896–900. https://doi.org/10.21437/interspeech.2022-11160.

Newell, Allen, and Herbert A. Simon. 1976. "Computer Science as Empirical Inquiry: Symbols and Search." *Communications of the ACM* 19 (3): 113–26. https://doi.org/10.1145/360018.360022.

Neyshabur, Behnam, Srinadh Bhojanapalli, David McAllester, and Nati Srebro. 2017. "Exploring Generalization in Deep Learning." *Advances in Neural Information Processing Systems* 30.

Ng, Andrew. 2021. *MLOps: From Model-Centric to Data-Centric AI*. DeepLearning.AI.

Nickolls, John, Ian Buck, Michael Garland, and Kevin Skadron. 2008. "Scalable Parallel Programming with CUDA: Is CUDA the Parallel Programming Model That Application Developers Have Been Waiting For?" *Queue* 6 (2): 40–53. https://doi.org/10.1145/1365490.1365500.

Norrie, Thomas, Nishant Patil, Doe Hyun Yoon, George Kurian, Sheng Li, James Laudon, Cliff Young, Norman Jouppi, and David Patterson. 2021. "The Design Process for Google's Training Chips: TPUv2 and TPUv3." *IEEE Micro* 41 (2): 56–63. https://doi.org/10.1109/mm.2021.3058217.

Northcutt, Curtis G., Anish Athalye, and Jonas Mueller. 2021. "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks." *arXiv Preprint arXiv:2103.14749* 34: 19075–90. https://doi.org/10.48550/arxiv.2103.14749.

NVIDIA. 2017. *Training with Mixed Precision*.

NVIDIA. 2020a. *Accelerating Matrix Multiplication with Block Sparse Format and NVIDIA Tensor Cores*.

NVIDIA. 2020b. *TensorFloat-32 in the A100 GPU Accelerates AI Training, HPC up to 20x*.

NVIDIA. 2024a. *cuBLAS: CUDA Basic Linear Algebra Subprograms*.

NVIDIA. 2024b. *NVIDIA Omniverse and Simulation*.

NVIDIA. 2024c. *NVIDIA TensorRT: Programmable Inference Accelerator*.

NVIDIA. 2024d. *NVIDIA Triton Inference Server*.

NVIDIA. 2026a. *Lazy Loading: CUDA Programming Guide*.

NVIDIA. 2026b. *Model Management: NVIDIA Triton Inference Server*.

NVIDIA. 2026c. *Model Repository: NVIDIA Triton Inference Server*.

NVIDIA. 2026d. *NVIDIA CUDA Multi-Process Service*.

NVIDIA. 2026e. *NVIDIA Multi-Instance GPU User Guide*.

NVIDIA. 2026f. *NVIDIA Quantum-X800 InfiniBand Platform*. NVIDIA product documentation.

NVIDIA. 2026g. *NVIDIA TensorRT-LLM*.

NVIDIA Corporation. 2017. *NVIDIA Tesla V100 GPU Architecture*. NVIDIA Whitepaper.

NVIDIA Corporation. 2018. *NVIDIA Tesla T4 Tensor Core GPU*. NVIDIA product documentation.

NVIDIA Corporation. 2020a. *NVIDIA A100 Tensor Core GPU Architecture*. NVIDIA Whitepaper, V1.0.

NVIDIA Corporation. 2020b. *NVIDIA V100 Tensor Core GPU Datasheet*. NVIDIA product documentation.

NVIDIA Corporation. 2020c. "NVLink: Scalable High-Performance Interconnect." *NVIDIA Technical Report* 2020.

NVIDIA Corporation. 2021a. *NVIDIA A100 Tensor Core GPU Datasheet*. NVIDIA product documentation.

NVIDIA Corporation. 2021b. *NVIDIA cuDNN Developer Guide*.

NVIDIA Corporation. 2022. *NVIDIA H100 Tensor Core GPU Architecture*. NVIDIA Corporation.

NVIDIA Corporation. 2023a. *NVIDIA cuDNN Developer Guide, Version 8.9.6*.

NVIDIA Corporation. 2023b. *NVIDIA H100 Tensor Core GPU: Product Specifications*. Https://www.nvidia.com/en-us/data-center/h100/.

NVIDIA Corporation. 2024. *NVIDIA Blackwell Architecture*. NVIDIA product documentation.

NVIDIA Corporation. 2026a. *CUDA Graphs: CUDA Programming Guide*.

NVIDIA Corporation. 2026b. *How TensorRT Works*.

NVIDIA Corporation. 2026c. *Working with Dynamic Shapes*.

Nygard, Michael T. 2007. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.

Oakden-Rayner, Luke, Jared Dunnmon, Gustavo Carneiro, and Christopher Re. 2020. "Hidden Stratification Causes Clinically Meaningful Failures in Machine Learning for Medical Imaging." *Proceedings of the ACM Conference on Health, Inference, and Learning*, 151–59. https://doi.org/10.1145/3368555.3384468.

Obermeyer, Ziad, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. 2019. "Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations." *Science* 366 (6464): 447–53. https://doi.org/10.1126/science.aax2342.

Olston, Christopher, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. "TensorFlow-Serving: Flexible, High-Performance ML Serving." *CoRR* abs/1712.06139. https://doi.org/10.48550/arXiv.1712.06139.

ONNX Contributors. 2019. *ONNX: Open Neural Network Exchange*.

ONNX Project. 2026. *ONNX Operator Schemas*.

OpenAI. 2023a. *GPT-4*.

OpenAI. 2023b. *Introducing ChatGPT and Whisper APIs*.

OpenAI. 2023c. *New Models and Developer Products Announced at DevDay*.

OpenAI. 2024. *GPT-4o Mini: Advancing Cost-Efficient Intelligence*.

OpenAI Developer Community. 2024. *Announcing GPT-4o in the API*.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, et al. 2023. "GPT-4 Technical Report." *arXiv Preprint arXiv:2303.08774*, ahead of print. https://doi.org/10.48550/arXiv.2303.08774.

OpenBLAS Project. 2026. *OpenBLAS: An Optimized BLAS Library*.

OpenXLA Project. 2025. *XLA HLO Opcode List*.

Orr, Laurel, Atindriyo Sanyal, Xiao Ling, Karan Goel, and Megan Leszczynski. 2021. "Managing ML Pipelines: Feature Stores and the Coming Wave of Embedding Ecosystems." *Proceedings of the VLDB Endowment* 14 (12): 3178–81. https://doi.org/10.14778/3476311.3476402.

Our World in Data. 2024. *Historical Price of Computer Memory and Storage*.

Owens, John D., Mike Houston, David Luebke, Simon Green, John E. Stone, and James C. Phillips. 2008. "GPU Computing." *Proceedings of the IEEE* 96 (5): 879–99. https://doi.org/10.1109/jproc.2008.917757.

Pakdaman Naeini, Mahdi, Gregory Cooper, and Milos Hauskrecht. 2015. "Obtaining Well Calibrated Probabilities Using Bayesian Binning." *Proceedings of the AAAI Conference on Artificial Intelligence* 29: 2901–7. https://doi.org/10.1609/aaai.v29i1.9602.

Paleyes, Andrei, Raoul-Gabriel Urma, and Neil D. Lawrence. 2022. "Challenges in Deploying Machine Learning: A Survey of Case Studies." *ACM Computing Surveys* 55 (6): 1–29. https://doi.org/10.1145/3533378.

Palmer, John F. 1980. "The Intel 8087 Numeric Data Processor." *Proceedings of the 1980 National Computer Conference (AFIPS)*, 887–93. https://doi.org/10.1145/1500518.1500674.

Papadopoulos, Dim P., Jasper R. R. Uijlings, Frank Keller, and Vittorio Ferrari. 2017. "Extreme Clicking for Efficient Object Annotation." *2017 IEEE International Conference on Computer Vision (ICCV)*, 4940–49. https://doi.org/10.1109/ICCV.2017.528.

Papermill Project. 2026. *Papermill Documentation*.

Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. "Bleu: A Method for Automatic Evaluation of Machine Translation." *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*, 311–18. https://doi.org/10.3115/1073083.1073135.

Pareto, Vilfredo. 1896. *Cours d'économie Politique*. F. Rouge.

Park, Daniel S., William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. 2019. "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition." *Interspeech 2019*, 2613–17. https://doi.org/10.21437/interspeech.2019-2680.

Park, Intaik, Ehsan Ardestani, Damian Reeves, Sarunya Pumma, Henry Tsang, Levy Zhao, Jian He, et al. 2024. "Toward 100TB Recommendation Models with Embedding Offloading." *18th ACM Conference on Recommender Systems*, 841–43. https://doi.org/10.1145/3640457.3688037.

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, et al. 2019. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 8024–35.

Patarasuk, Pitch, and Xin Yuan. 2009. "Bandwidth Optimal All-Reduce Algorithms for Clusters of Workstations." *Journal of Parallel and Distributed Computing* 69 (2): 117–24. https://doi.org/10.1016/j.jpdc.2008.09.002.

Patel, Dylan, and Gerald Wong. 2023. *GPT-4 Architecture, Infrastructure, Training Dataset, Costs, Vision, MoE*. SemiAnalysis Blog.

Patterson, David, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. "Carbon Emissions and Large Neural Network Training." *arXiv Preprint arXiv:2104.10350*.

Paul, Mansheej, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. "Deep Learning on a Data Diet: Finding Important Examples Early in Training." *Advances in Neural Information Processing Systems (NeurIPS)* 34: 20596–607.

Peng, Lily, and Varun Gulshan. 2016. *Deep Learning for Detection of Diabetic Eye Disease*.

Pham, Hieu, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. 2018. "Efficient Neural Architecture Search via Parameter Sharing." *Proceedings of the 35th International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 80: 4095–104.

Pichai, Sundar. 2023. *Introducing Gemini: Our Largest and Most Capable AI Model*.

Pineau, Joelle, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché-Buc, Emily Fox, and Hugo Larochelle. 2021. "Improving Reproducibility in Machine Learning Research (a Report from the Neurips 2019 Reproducibility Program)." *Journal of Machine Learning Research* 22 (164): 1–20.

Polyak, Boris T. 1964. "Some Methods of Speeding up the Convergence of Iteration Methods." *USSR Computational Mathematics and Mathematical Physics* 4 (5): 1–17. https://doi.org/10.1016/0041-5553(64)90137-5.

Pope, Reiner, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. "Efficiently Scaling Transformer Inference." *Proceedings of Machine Learning and Systems (MLSys)* 5: 606–24.

Project Jupyter. 2024. *Project Jupyter*.

Project Jupyter. 2026. *nbconvert: Convert Notebooks to Other Formats*.

Protocol Buffers Authors. 2026. *Overview: Protocol Buffers*.

Pushkarna, Mahima, Andrew Zaldivar, and Oddur Kjartansson. 2022. "Data Cards: Purposeful and Transparent Dataset Documentation for Responsible AI." *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 1776–826. https://doi.org/10.1145/3531146.3533231.

Putnam, Andrew, Adrian M. Caulfield, Eric S. Chung, Derek Chiou, Kypros Constantinides, John Demme, Hadi Esmaeilzadeh, et al. 2014. "A Reconfigurable Fabric for Accelerating Large-Scale Datacenter Services." *ACM SIGARCH Computer Architecture News* 42 (3): 13–24. https://doi.org/10.1145/2678373.2665678.

PyTorch. 2022. *Accelerating Neural Network Training with Sparse Tensors*.

PyTorch Contributors. 2025. *ATen Native Functions Registry*.

PyTorch Contributors. 2026a. *PyTorch Autograd Mechanics*.

PyTorch Contributors. 2026b. *Tensor Views*.

PyTorch Contributors. 2026c. *torch.export*.

PyTorch Contributors. 2026d. *TorchScript*.

Qi, Chen, Shibo Shen, Rongpeng Li, Zhifeng Zhao, Qing Liu, Jing Liang, and Honggang Zhang. 2021. "An Efficient Pruning Scheme of Deep Neural Networks for Internet of Things Applications." *EURASIP Journal on Advances in Signal Processing* 2021 (1): 31. https://doi.org/10.1186/s13634-021-00744-4.

Quiñonero-Candela, Joaquin, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence, eds. 2009. *Dataset Shift in Machine Learning*. Neural Information Processing Series. The MIT Press.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. 2024. "Qwen2.5 Technical Report." *arXiv Preprint arXiv:2412.15115*.

Rachwan, John, Daniel Zügner, Bertrand Charpentier, Simon Geisler, Morgane Ayle, and Stephan Günnemann. 2022. "Winning the Lottery Ahead of Time: Efficient Early Network Pruning." *International Conference on Machine Learning (ICML)*, 18293–309.

Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. "Learning Transferable Visual Models from Natural Language Supervision." *International Conference on Machine Learning (ICML)*, 8748–63.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models Are Unsupervised Multitask Learners*. OpenAI.

Rae, Jack W., Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, et al. 2021. "Scaling Language Models: Methods, Analysis & Insights from Training Gopher." *arXiv Preprint arXiv:2112.11446*.

Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. 2020. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." *Journal of Machine Learning Research* 21 (140): 1–67.

Rainio, Oona, Jarmo Teuho, and Riku Klén. 2024. "Evaluation Metrics and Statistical Tests for Machine Learning." *Scientific Reports* 14 (1): 6086. https://doi.org/10.1038/s41598-024-56706-x.

Rajbhandari, Samyam, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–16. https://doi.org/10.1109/sc41405.2020.00024.

Raji, Inioluwa Deborah, and Joy Buolamwini. 2019. "Actionable Auditing: Investigating the Impact of Publicly Naming Biased Performance Results of Commercial AI Products." *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, 429–35. https://doi.org/10.1145/3306618.3314244.

Rajpurkar, Pranav, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. "SQuAD: 100,000+ Questions for Machine Comprehension of Text." *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2383–92. https://doi.org/10.18653/v1/d16-1264.

Ramesh, A., M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever. 2021. "Zero-Shot Text-to-Image Generation." In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, edited by Marina Meila and Tong Zhang, vol. 139. Proceedings of Machine Learning Research. PMLR.

Rastegari, Mohammad, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. "XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-319-46493-0_32.

Ratner, A., S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré. 2017. "Snorkel: Rapid Training Data Creation with Weak Supervision." *Proceedings of the VLDB Endowment* 11 (3): 269–82. https://doi.org/10.14778/3157794.3157797.

Ratner, Alex, Braden Hancock, Jared Dunnmon, Roger Goldman, and Christopher Ré. 2018. "Snorkel MeTaL: Weak Supervision for Multi-Task Learning." *Proceedings of the Second Workshop on Data Management for End-to-End Machine Learning*, 1–4. https://doi.org/10.1145/3209889.3209898.

Ratner, Alex, Paroma Varma, Braden Hancock, and Christopher Ré. 2019. *Weak Supervision: A New Programming Paradigm for Machine Learning*. Stanford AI Lab Blog.

Reagen, Brandon, Robert Adolf, Paul Whatmough, Gu-Yeon Wei, and David Brooks. 2017. *Deep Learning for Computer Architects*. Synthesis Lectures on Computer Architecture. Springer International Publishing. https://doi.org/10.1007/978-3-031-01756-8.

Real, Esteban, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. "Regularized Evolution for Image Classifier Architecture Search." *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01): 4780–89. https://doi.org/10.1609/aaai.v33i01.33014780.

Recht, Benjamin, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. 2019. "Do ImageNet Classifiers Generalize to ImageNet?" *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 5389–400.

Reddi, Vijay Janapa, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, et al. 2019. "MLPerf Inference Benchmark." *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 446–59. https://doi.org/10.1109/isca45697.2020.00045.

Ren, Pengzhen, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xin Wang. 2021. "A Survey of Deep Active Learning." *ACM Computing Surveys* 54 (9): 1–40. https://doi.org/10.1145/3472291.

Ribeiro, M. H., R. Ottoni, R. West, V. A. F. Almeida, and Jr. Meira Wagner. 2020. "Auditing Radicalization Pathways on YouTube." *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 131–41. https://doi.org/10.1145/3351095.3372879.

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?: Explaining the Predictions of Any Classifier." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–44. https://doi.org/10.1145/2939672.2939778.

Ribeiro, Marco Tulio, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList." *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–12. https://doi.org/10.18653/v1/2020.acl-main.442.

Robbins, Herbert, and Sutton Monro. 1951. "A Stochastic Approximation Method." *The Annals of Mathematical Statistics* 22 (3): 400–407. https://doi.org/10.1214/aoms/1177729586.

Roesch, Jared, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. "Relay: A New IR for Machine Learning Frameworks." *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 58–68. https://doi.org/10.1145/3211346.3211348.

Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. 2022. "High-Resolution Image Synthesis with Latent Diffusion Models." *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10674–85. https://doi.org/10.1109/cvpr52688.2022.01042.

Romero, Francisco, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. "INFaaS: Automated Model-Less Inference Serving." *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 397–411.

Rosenblatt, Frank. 1957. *The Perceptron: A Perceiving and Recognizing Automaton*. Report Nos. 85-460-1. Cornell Aeronautical Laboratory.

Rosenblatt, Frank. 1958. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." *Psychological Review* 65 (6): 386–408. https://doi.org/10.1037/h0042519.

Rouhani, Bita Darvish, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, et al. 2023. "Microscaling Data Formats for Deep Learning." *arXiv Preprint arXiv:2310.10537*.

Royce, Winston W. 1970. "Managing the Development of Large Software Systems." *Proceedings of IEEE WESCON* 26: 328–88.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. "Learning Representations by Back-Propagating Errors." *Nature* 323 (6088): 533–36. https://doi.org/10.1038/323533a0.

Russakovsky, Olga, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, et al. 2015. "ImageNet Large Scale Visual Recognition Challenge." *International Journal of Computer Vision* 115 (3): 211–52. https://doi.org/10.1007/s11263-015-0816-y.

Sambasivan, Nithya, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M Aroyo. 2021. "'Everyone Wants to Do the Model Work, Not the Data Work': Data Cascades in High-Stakes AI." *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–15. https://doi.org/10.1145/3411764.3445518.

Samuel, A. L. 1959. "Some Studies in Machine Learning Using the Game of Checkers." *IBM Journal of Research and Development* 3 (3): 210–29. https://doi.org/10.1147/rd.33.0210.

Sandler, Mark, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. "MobileNetV2: Inverted Residuals and Linear Bottlenecks." *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4510–20. https://doi.org/10.1109/cvpr.2018.00474.

Sanh, Victor, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter." *arXiv Preprint arXiv:1910.01108*, ahead of print. https://doi.org/10.48550/arXiv.1910.01108.

Savage, Sam L. 2009. *The Flaw of Averages: Why We Underestimate Risk in the Face of Uncertainty*. John Wiley & Sons.

Scale AI, Inc. 2024. *Scale AI Advanced Settings*.

Schelter, Sebastian, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. 2018. "Automating Large-Scale Data Quality Verification." *Proceedings of the VLDB Endowment* 11 (12): 1781–94. https://doi.org/10.14778/3229863.3229867.

Schwartz, Roy, Jesse Dodge, Noah A. Smith, and Oren Etzioni. 2020. "Green AI." *Communications of the ACM* 63 (12): 54–63. https://doi.org/10.1145/3381831.

scikit-learn developers. 2024. *Sklearn.metrics.confusion_matrix — Scikit-Learn Documentation*. Scikit-learn Documentation.

Scott, Colin. 2012. "Numbers Every Programmer Should Know by Year."

Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. 2015. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems (NeurIPS)* 28: 2503–11.

Sener, Ozan, and Silvio Savarese. 2018. "Active Learning for Convolutional Neural Networks: A Core-Set Approach." *International Conference on Learning Representations (ICLR)*.

Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016. "Improving Neural Machine Translation Models with Monolingual Data." *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 86–96. https://doi.org/10.18653/v1/P16-1009.

Settles, Burr. 2009. *Active Learning Literature Survey*. Computer Sciences Technical Report 1648. University of Wisconsin-Madison.

Settles, Burr. 2012. *Active Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Springer Cham. https://doi.org/10.1007/978-3-031-01560-1.

Sevilla, Jaime, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. 2022. "Compute Trends Across Three Eras of Machine Learning." *2022 International Joint Conference on Neural Networks (IJCNN)*, 1–8. https://doi.org/10.1109/ijcnn55064.2022.9891914.

Shah, Jay, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision." *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 68658–85. https://doi.org/10.52202/079017-2193.

Shankar, Shreya, Yoni Halpern, Eric Breck, James Atwood, Jimbo Wilson, and David Sculley. 2017. "No Classification Without Representation: Assessing Geodiversity Issues in Open Data Sets for the Developing World." *arXiv Preprint arXiv:1711.08536*.

Shannon, Claude E. 1948. "A Mathematical Theory of Communication." *Bell System Technical Journal* 27 (3): 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x.

Shazeer, Noam, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer." *International Conference on Learning Representations (ICLR)*.

Shazeer, Noam, and Mitchell Stern. 2018. "Adafactor: Adaptive Learning Rates with Sublinear Memory Cost." *International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 80: 4596–604.

Shen, Haichen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. "Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 322–37. https://doi.org/10.1145/3341301.3359658.

Shen, Sheng, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. 2020. "Q-BERT: Hessian Based Ultra Low Precision Quantization of BERT." *Proceedings of the AAAI Conference on Artificial Intelligence* 34 (05): 8815–21. https://doi.org/10.1609/aaai.v34i05.6409.

Sheng, Victor S., and Jing Zhang. 2019. "Machine Learning with Crowdsourcing: A Brief Summary of the Past Research and Future Directions." *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01): 9837–43. https://doi.org/10.1609/aaai.v33i01.33019837.

Shi, Weisong, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. "Edge Computing: Vision and Challenges." *IEEE Internet of Things Journal* 3 (5): 637–46. https://doi.org/10.1109/jiot.2016.2579198.

Shimodaira, Hidetoshi. 2000. "Improving Predictive Inference Under Covariate Shift by Weighting the Log-Likelihood Function." *Journal of Statistical Planning and Inference* 90 (2): 227–44. https://doi.org/10.1016/S0378-3758(00)00115-4.

Shneiderman, B. 2022. *Human-Centered AI*. Oxford University Press. https://doi.org/10.1093/oso/9780192845290.001.0001.

Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism." *arXiv Preprint arXiv:1909.08053*.

Shokri, Reza, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. "Membership Inference Attacks Against Machine Learning Models." *2017 IEEE Symposium on Security and Privacy (SP)*, 3–18. https://doi.org/10.1109/SP.2017.41.

Shorten, Connor, and Taghi M. Khoshgoftaar. 2019. "A Survey on Image Data Augmentation for Deep Learning." *Journal of Big Data* 6 (1): 1–48. https://doi.org/10.1186/s40537-019-0197-0.

Shortliffe, Edward H., Randall Davis, Stanton G. Axline, Bruce G. Buchanan, C.Cordell Green, and Stanley N. Cohen. 1975. "Computer-Based Consultations in Clinical Therapeutics: Explanation and Rule Acquisition Capabilities of the MYCIN System." *Computers and Biomedical Research* 8 (4): 303–20. https://doi.org/10.1016/0010-4809(75)90009-9.

Shumailov, Ilia, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. 2024. "AI Models Collapse When Trained on Recursively Generated Data." *Nature* 631 (8022): 755–59. https://doi.org/10.1038/s41586-024-07566-y.

Sifre, Laurent, and Stéphane Mallat. 2014. "Rigid-Motion Scattering for Image Classification." *arXiv Preprint arXiv:1403.1687*, ahead of print. https://doi.org/10.48550/arXiv.1403.1687.

Silver, David, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, et al. 2016. "Mastering the Game of Go with Deep Neural Networks and Tree Search." *Nature* 529 (7587): 484–89. https://doi.org/10.1038/nature16961.

Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, et al. 2017. "Mastering the Game of Go Without Human Knowledge." *Nature* 550 (7676): 354–59. https://doi.org/10.1038/nature24270.

Simonyan, Karen, and Andrew Zisserman. 2015. "Very Deep Convolutional Networks for Large-Scale Image Recognition." *arXiv Preprint*.

Siri Team. 2017. *Hey Siri: An on-Device DNN-Powered Voice Trigger for Apple's Personal Assistant*.

Smith, Steven W. 1997. "Digital Signal Processing." In *Digital Signal Processing Demystified*. Elsevier. https://doi.org/10.1016/b978-187870716-1/50004-4.

Snow, Rion, Brendan O'Connor, Daniel Jurafsky, and Andrew Y. Ng. 2008. "Cheap and Fast—but Is It Good? Evaluating Non-Expert Annotations for Natural Language Tasks." *Proceedings of the Conference on Empirical Methods in Natural Language Processing - EMNLP '08*, 254. https://doi.org/10.3115/1613715.1613751.

Sohn, Kihyuk, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. 2020. "FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 596–608.

Sokolova, Marina, and Guy Lapalme. 2009. "A Systematic Analysis of Performance Measures for Classification Tasks." *Information Processing &Amp; Management* 45 (4): 427–37. https://doi.org/10.1016/j.ipm.2009.03.002.

Sorscher, Ben, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari S. Morcos. 2022. "Beyond Neural Scaling Laws: Beating Power Law Scaling via Data Pruning." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 19523–36. https://doi.org/10.52202/068431-1419.

Soviany, Petru, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. 2022. "Curriculum Learning: A Survey." *International Journal of Computer Vision* 130 (6): 1526–65. https://doi.org/10.1007/s11263-022-01611-x.

Srinivasan, Vidhya. 2025. *AI, Personalization and the Future of Shopping*. The Keyword (Google blog).

Srivastava, Nitish, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *Journal of Machine Learning Research* 15 (1): 1929–58.

Srivastava, Rupesh Kumar, Klaus Greff, and Jürgen Schmidhuber. 2015. "Highway Networks." *arXiv Preprint arXiv:1505.00387*.

Statista Research Department. 2024. *Number of Sent and Received e-Mails Per Day Worldwide from 2017 to 2027*. Statista.

Stephens, Nigel, Stuart Biles, Matthias Boettcher, Jacob Eapen, Mbou Eyole, Giacomo Gabrielli, Matt Horsnell, et al. 2017. "The ARM Scalable Vector Extension." *IEEE Micro* 37 (2): 26–39. https://doi.org/10.1109/mm.2017.35.

Stonebraker, Mike, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, et al. 2018. "C-Store: A Column-Oriented DBMS." In *Making Databases Work: The Pragmatic Wisdom of Michael Stonebraker*. Association for Computing Machinery. https://doi.org/10.1145/3226595.3226638.

Strathern, Marilyn. 1997. "'Improving Ratings': Audit in the British University System." *European Review* 5 (3): 305–21. https://doi.org/10.1002/(SICI)1234-981X(199707)5:3<305::AID-EURO184>3.0.CO;2-4.

Stromberg, Joseph. 2015. *When Was the First Traffic Light Installed? Today in 1914*. Vox.

Strubell, Emma, Ananya Ganesh, and Andrew McCallum. 2019. "Energy and Policy Considerations for Deep Learning in NLP." *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3645–50. https://doi.org/10.18653/v1/p19-1355.

Su, Jianlin, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. "RoFormer: Enhanced Transformer with Rotary Position Embedding." *Neurocomputing* 568: 127063. https://doi.org/10.1016/j.neucom.2023.127063.

Sullivan, Gary J., Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. 2012. "Overview of the High Efficiency Video Coding (HEVC) Standard." *IEEE Transactions on Circuits and Systems for Video Technology* 22 (12): 1649–68. https://doi.org/10.1109/tcsvt.2012.2221191.

Sun, Pei, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, et al. 2020. "Scalability in Perception for Autonomous Driving: Waymo Open Dataset." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2446–54. https://doi.org/10.1109/CVPR42600.2020.00252.

Supreme Court of the United States. 1971. *Griggs v. Duke Power Co., 401 U.S. 424*. Legal decision.

Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. "Sequence to Sequence Learning with Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 27: 3104–12.

Sutton, Richard S. 2019. "The Bitter Lesson." *Incompleteideas.net* 43.

Sweeney, Latanya. 2002. "K-ANONYMITY: A MODEL for PROTECTING PRIVACY." *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 10 (05): 557–70. https://doi.org/10.1142/s0218488502001648.

Systems, Cerebras. 2021. "Wafer-Scale Deep Learning Acceleration with the Cerebras CS-2." *Cerebras Technical Paper* 2021.

Sze, Vivienne, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2017. "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." *Proceedings of the IEEE* 105 (12): 2295–329. https://doi.org/10.1109/jproc.2017.2761740.

Szegedy, Christian, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. "Going Deeper with Convolutions." *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1–9. https://doi.org/10.1109/cvpr.2015.7298594.

Tan, Mingxing, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le. 2019. "MnasNet: Platform-Aware Neural Architecture Search for Mobile." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2815–23. https://doi.org/10.1109/cvpr.2019.00293.

Tan, Mingxing, and Quoc V Le. 2019. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *International Conference on Machine Learning (ICML)*, 6105–14.

Team, The Theano Development, Rami Al-Rfou, Guillaume Alain, Amjad Almahairi, Christof Angermueller, Dzmitry Bahdanau, Nicolas Ballas, et al. 2016. "Theano: A Python Framework for Fast Computation of Mathematical Expressions." *arXiv Preprint*.

Teerapittayanon, Surat, Bradley McDanel, and H. T. Kung. 2017. "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks." *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2464–69. https://doi.org/10.1109/icpr.2016.7900006.

Teerapittayanon, Surat, Bradley McDanel, and Hsiang-Tsung Kung. 2017. "Distributed Deep Neural Networks over the Cloud, the Edge and End Devices." *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 328–39. https://doi.org/10.1109/ICDCS.2017.226.

Telgarsky, Matus. 2016. "Benefits of Depth in Neural Networks." *Proceedings of the 29th Annual Conference on Learning Theory*, 1517–39.

TensorFlow Developers. 2024a. *Better Performance with tf.function*.

TensorFlow Developers. 2024b. *Optimize TensorFlow Performance Using the Profiler*.

TensorFlow Developers. 2025. *TensorFlow Operator Registry*.

Tesler, Larry. 1984. *The Law of Conservation of Complexity*. Web page.

The Green Grid. 2007. *Green Grid Data Center Power Efficiency Metrics: PUE and DCIE*. The Green Grid.

Thyagarajan, Aditya, Elías Snorrason, Curtis G. Northcutt, and Jonas Mueller. 2022. "Identifying Incorrect Annotations in Multi-Label Classification Data." *CoRR* abs/2211.13895. https://doi.org/10.48550/ARXIV.2211.13895.

Tibshirani, Robert. 1996. "Regression Shrinkage and Selection via the Lasso." *Journal of the Royal Statistical Society: Series B (Methodological)* 58 (1): 267–88. https://doi.org/10.1111/j.2517-6161.1996.tb02080.x.

Tillet, Philippe, H. T. Kung, and David Cox. 2019. "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations." *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 10–19. https://doi.org/10.1145/3315508.3329973.

Toneva, Mariya, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J Gordon. 2019. "An Empirical Study of Example Forgetting During Deep Neural Network Learning." *International Conference on Learning Representations (ICLR)*.

Torvalds, Linus, and Junio Hamano. 2024. *Git*.

Touvron, Hugo, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, et al. 2023. "LLaMA: Open and Efficient Foundation Language Models." *arXiv Preprint arXiv:2302.13971*.

Touvron, Hugo, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. 2023. "Llama 2: Open Foundation and Fine-Tuned Chat Models." *arXiv Preprint arXiv:2307.09288*.

Tramèr, Florian, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. "Stealing Machine Learning Models via Prediction APIs." *25th USENIX Security Symposium (USENIX Security 16)*, 601–18.

Tschand, Arya, Arun Tejusve Raghunath Rajan, Sachin Idgunji, Anirban Ghosh, Jeremy Holleman, Csaba Kiraly, Pawan Ambalkar, et al. 2024. "MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from μWatts to MWatts for Sustainable AI." *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 1201–16. https://doi.org/10.1109/hpca61900.2025.00092.

Turakhia, Yatish, Gill Bejerano, and William J. Dally. 2018. "Darwin: A Genomics Co-Processor Provides up to 15,000X Acceleration on Long Read Assembly." *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 199–213. https://doi.org/10.1145/3173162.3173193.

Turing, Alan M. 1950. "I.—Computing Machinery and Intelligence." *Mind* LIX (236): 433–60. https://doi.org/10.1093/mind/lix.236.433.

Ultralytics. 2023. *YOLOv8*.

Umuroglu, Yaman, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. "Finn: A Framework for Fast, Scalable Binarized Neural Network Inference." *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 65–74. https://doi.org/10.1145/3020078.3021744.

United States Congress. 1996. *Health Insurance Portability and Accountability Act of 1996*. Public Law 104-191.

U.S. Department of Health and Human Services. 2003. "Summary of the HIPAA Privacy Rule."

U.S. Department of Health and Human Services. 2005. "Summary of the HIPAA Security Rule."

U.S. Food and Drug Administration. 2021. *Artificial Intelligence/Machine Learning (AI/ML)-Based Software as a Medical Device (SaMD) Action Plan*. U.S. Department of Health; Human Services.

U.S. Food and Drug Administration. 2025. *Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions*. Guidance for Industry and Food and Drug Administration Staff. U.S. Department of Health; Human Services.

U.S. Securities and Exchange Commission. 2013. "Securities Exchange Act of 1934, Release No. 70694: Knight Capital Americas LLC."

Vasisht, Deepak, Zerina Kapetanovic, Jongho Won, Xinxin Jin, Ranveer Chandra, Sudipta N. Sinha, Ashish Kapoor, Madhusudhan Sudarshan, and Sean Stratman. 2017. "FarmBeats: An IoT Platform for Data-Driven Agriculture." *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 515–29.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. "Attention Is All You Need." *Advances in Neural Information Processing Systems (NeurIPS)* 30: 5998–6008.

Villalobos, Pablo, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. 2024. "Position: Will We Run Out of Data? Limits of LLM Scaling Based on Human-Generated Data." *Proceedings of the 41st International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 235: 49523–44.

Viola, Paul, and Michael J. Jones. 2001. "Rapid Object Detection Using a Boosted Cascade of Simple Features." *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001* 1: I-511-I-518. https://doi.org/10.1109/cvpr.2001.990517.

Wald, Abraham. 1949. "Statistical Decision Functions." *The Annals of Mathematical Statistics* 20 (2): 165–205. https://doi.org/10.1214/aoms/1177730030.

Wang, Alex, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. "SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems." *arXiv Preprint arXiv:1905.00537*.

Wang, Alex, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding." *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 353–55. https://doi.org/10.18653/v1/w18-5446.

Wang, Linnan, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. "SuperNeurons: Dynamic GPU Memory Management for Training Deep Neural Networks." *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 41–53. https://doi.org/10.1145/3178487.3178491.

Wang, Shibo, and Pankaj Kanwar. 2019. *BFloat16: The Secret to High Performance on Cloud TPUs*.

Wang, Tianlu, Jieyu Zhao, Mark Yatskar, Kai-Wei Chang, and Vicente Ordonez. 2019. "Balanced Datasets Are Not Enough: Estimating and Mitigating Gender Bias in Deep Image Representations." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 5309–18. https://doi.org/10.1109/iccv.2019.00541.

Wang, Xin, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E. Gonzalez. 2018. "SkipNet: Learning Dynamic Routing in Convolutional Networks." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-030-01261-8_25.

Wang, Yu Emma, Gu-Yeon Wei, and David Brooks. 2019. "Benchmarking TPU, GPU, and CPU Platforms for Deep Learning." *arXiv Preprint arXiv:1907.10701*.

Wang, Zijie J., Robert Turko, Omar Shaikh, Haekyu Park, Nilaksh Das, Fred Hohman, Minsuk Kahng, and Duen Horng Polo Chau. 2021. "CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization." *IEEE Transactions on Visualization and Computer Graphics* 27 (2): 1396–406. https://doi.org/10.1109/tvcg.2020.3030418.

Warden, Pete. 2018. "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition." *arXiv Preprint arXiv:1804.03209*, ahead of print. https://doi.org/10.48550/arXiv.1804.03209.

Warden, Pete, and Daniel Situnayake. 2020. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.

Waterman, Andrew, Yunsup Lee, Rimas Avizienis, Henry Cook, David Patterson, and Krste Asanovic. 2013. "The RISC-V Instruction Set." *2013 IEEE Hot Chips 25 Symposium (HCS)*, 1–1. https://doi.org/10.1109/hotchips.2013.7478332.

Weights & Biases, Inc. 2024. *Weights & Biases: The AI Developer Platform*.

Weiser, Mark. 1991. "The Computer for the 21st Century." *Scientific American* 265 (3): 94–104. https://doi.org/10.1038/scientificamerican0991-94.

Weizenbaum, Joseph. 1966. "ELIZA–a Computer Program for the Study of Natural Language Communication Between Man and Machine." *Communications of the ACM* 9 (1): 36–45. https://doi.org/10.1145/365153.365168.

Welling, Max. 2009. "Herding Dynamical Weights to Learn." *Proceedings of the 26th Annual International Conference on Machine Learning*, 1121–28. https://doi.org/10.1145/1553374.1553517.

Wengert, Ralph E. 1964. "A Simple Automatic Derivative Evaluation Program." *Communications of the ACM* 7 (8): 463–64. https://doi.org/10.1145/355586.364791.

Werbos, Paul. 1974. "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences." PhD thesis, Harvard University.

Werbos, Paul J. 1990. "Backpropagation Through Time: What It Does and How to Do It." *Proceedings of the IEEE* 78 (10): 1550–60. https://doi.org/10.1109/5.58337.

Werchniak, Andrew, Roberto Barra Chicote, Yuriy Mishchenko, Jasha Droppo, Jeff Condal, Peng Liu, and Anish Shah. 2021. "Exploring the Application of Synthetic Audio in Training Keyword Spotters." *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 7993–96. https://doi.org/10.1109/icassp39728.2021.9413448.

Wexler, James, Mahima Pushkarna, Tolga Bolukbasi, Martin Wattenberg, Fernanda Viégas, and Jimbo Wilson. 2020. "The What-If Tool: Interactive Probing of Machine Learning Models." *IEEE Transactions on Visualization and Computer Graphics* 26 (1): 56–65. https://doi.org/10.1109/TVCG.2019.2934619.

Widmer, Gerhard, and Miroslav Kubat. 1996. "Learning in the Presence of Concept Drift and Hidden Contexts." *Machine Learning* 23 (1): 69–101. https://doi.org/10.1023/a:1018046501280.

Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76. https://doi.org/10.1145/1498765.1498785.

Wolpert, D. H., and W. G. Macready. 1997. "No Free Lunch Theorems for Optimization." *IEEE Transactions on Evolutionary Computation* 1 (1): 67–82. https://doi.org/10.1109/4235.585893.

Wu, Bichen, Kurt Keutzer, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, and Yangqing Jia. 2019. "FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10726–34. https://doi.org/10.1109/cvpr.2019.01099.

Wu, Hao, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. 2020. "Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation." *arXiv Preprint arXiv:2004.09602* abs/2004.09602.

Wu, Zuxuan, Tushar Nagarajan, Abhishek Kumar, Steven Rennie, Larry S. Davis, and Rogerio Feris. 2018. "BlockDrop: Dynamic Inference Paths in Residual Networks." *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 8817–26. https://doi.org/10.1109/cvpr.2018.00919.

Wulf, Wm. A., and Sally A. McKee. 1995. "Hitting the Memory Wall: Implications of the Obvious." *ACM SIGARCH Computer Architecture News* 23 (1): 20–24. https://doi.org/10.1145/216585.216588.

Xin, Ji, Raphael Tang, Yaoliang Yu, and Jimmy Lin. 2021. "BERxiT: Early Exiting for BERT with Better Fine-Tuning and Extension to Regression." In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, edited by Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty. Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.eacl-main.8.

Xu, Ruijie, Zengzhi Wang, Run-Ze Fan, and Pengfei Liu. 2024. "Benchmarking Benchmark Leakage in Large Language Models." *arXiv Preprint arXiv:2404.18824*.

Yang, An, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. 2025. "Qwen3 Technical Report." *arXiv Preprint arXiv:2505.09388*.

Yang, Le, Yizeng Han, Xi Chen, Shiji Song, Jifeng Dai, and Gao Huang. 2020. "Resolution Adaptive Networks for Efficient Inference." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2366–75. https://doi.org/10.1109/cvpr42600.2020.00244.

Yao, Huaxiu, Caroline Choi, Bochuan Cao, Yoonho Lee, Pang Wei Koh, and Chelsea Finn. 2022. "Wild-Time: A Benchmark of in-the-Wild Distribution Shift over Time." *arXiv Preprint arXiv:2211.14238*.

Yao, Zhewei, Amir Gholami, Sheng Shen, Kurt Keutzer, and Michael W. Mahoney. 2021. "HAWQ-V3: Dyadic Neural Network Quantization." *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 11875–86.

Yee, Kyra, Uthaipon Tantipongpipat, and Shubhanshu Mishra. 2021. "Image Cropping on Twitter: Fairness Metrics, Their Limitations, and the Importance of Representation, Design, and Agency." *Proceedings of the ACM on Human-Computer Interaction* 5 (CSCW2): 1–24. https://doi.org/10.1145/3479594.

Yosinski, Jason, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. "How Transferable Are Features in Deep Neural Networks?" *Advances in Neural Information Processing Systems* 27.

Yu, Gyeong-In, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. "Orca: A Distributed Serving System for Transformer-Based Generative Models." *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–38.

Yun, Sangdoo, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. 2019. "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 6023–32. https://doi.org/10.1109/ICCV.2019.00612.

Yurdakul, Bilal, and Joshua Naranjo. 2020. "Statistical Properties of the Population Stability Index." *The Journal of Risk Model Validation* 52. https://doi.org/10.21314/jrmv.2020.227.

Zafrir, Ofir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. "Q8BERT: Quantized 8Bit BERT." *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS Edition (EMC2-NIPS)*, 36–39. https://doi.org/10.1109/emc2-nips53020.2019.00016.

Zaharia, Matei, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, et al. 2018. "Accelerating the Machine Learning Lifecycle with MLflow." *IEEE Data Engineering Bulletin* 41 (4): 39–45.

Zaharia, Matei, Ali Ghodsi, Reynold Xin, and Michael Armbrust. 2021. "Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics." *Proceedings of CIDR* 8.

Zech, John R., Marcus A. Badgeley, Manway Liu, Anthony B. Costa, Joseph J. Titano, and Eric Karl Oermann. 2018. "Variable Generalization Performance of a Deep Learning Model to Detect Pneumonia in Chest Radiographs: A Cross-Sectional Study." *PLOS Medicine* 15 (11): e1002683. https://doi.org/10.1371/journal.pmed.1002683.

Zeiler, Matthew D., and Rob Fergus. 2014. "Visualizing and Understanding Convolutional Networks." *European Conference on Computer Vision (ECCV)*, 818–33. https://doi.org/10.1007/978-3-319-10590-1_53.

Zhang, Biao, and Rico Sennrich. 2019. "Root Mean Square Layer Normalization." *Advances in Neural Information Processing Systems (NeurIPS)*.

Zhang, Brian Hu, Blake Lemoine, and Margaret Mitchell. 2018. "Mitigating Unwanted Biases with Adversarial Learning." *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, 335–40. https://doi.org/10.1145/3278721.3278779.

Zhang, Chengliang, Minchen Yu, Wei Wang, and Feng Yan. 2019. "MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving." *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 1049–62.

Zhang, Chiyuan, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. 2017. "Understanding Deep Learning Requires Rethinking Generalization." *International Conference on Learning Representations (ICLR)*.

Zhang, Hongyi, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. 2018. *mixup: Beyond Empirical Risk Minimization*. OpenReview.net.

Zhang, Li Lyna, Yuqing Yang, Yuhang Jiang, Wenwu Zhu, and Yunxin Liu. 2020. "Fast Hardware-Aware Neural Architecture Search." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. https://doi.org/10.1109/cvprw50498.2020.00354.

Zhang, Qingxue, Dian Zhou, and Xuan Zeng. 2017. "Highly Wearable Cuff-Less Blood Pressure and Heart Rate Monitoring with Single-Arm Electrocardiogram and Photoplethysmogram Signals." *BioMedical Engineering OnLine* 16 (1): 23. https://doi.org/10.1186/s12938-017-0317-z.

Zhang, Yundong, Naveen Suda, Liangzhen Lai, and Vikas Chandra. 2017. *Hello Edge: Keyword Spotting on Microcontrollers*.

Zhao, Jiawei, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. "GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection." *arXiv Preprint*.

Zhao, Mark, Niket Agarwal, Aarti Basant, Buğra Gedik, Satadru Pan, Mustafa Ozdal, Rakesh Komuravelli, et al. 2022. "Understanding Data Storage and Ingestion for Large-Scale Deep Recommendation Model Training." *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 1042–57. https://doi.org/10.1145/3470496.3533044.

Zhao, Yanli, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, et al. 2023. "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel." *Proceedings of the VLDB Endowment* 16 (12): 3848–60. https://doi.org/10.14778/3611540.3611569.

Zheng, Lianmin, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, et al. 2020. "Ansor: Generating High-Performance Tensor Programs for Deep Learning." *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 863–79.

Zhou, Aojun, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, Wenxiu Sun, and Hongsheng Li. 2021. "Learning n:m Fine-Grained Structured Sparse Neural Networks from Scratch." *arXiv Preprint*.

Zhu, Chenzhuo, Song Han, Huizi Mao, and William J. Dally. 2017. "Trained Ternary Quantization." *International Conference on Learning Representations (ICLR)* 4 (4): 1–16.

Zhu, Ligeng, Zhijian Liu, and Song Han. 2019. "Deep Leakage from Gradients." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 14774–84.

Zillow Group. 2021. *Zillow Group Reports Third-Quarter 2021 Financial Results and Shares Plan to Wind down Zillow Offers Operations*. Investor Relations Press Release.

Zoph, Barret, and Quoc V. Le. 2016. "Neural Architecture Search with Reinforcement Learning." *International Conference on Learning Representations* 3.

Index

A/B testing
  delayed feedback, A/B testing challenges, 785
  ML deployment, 93
  randomization unit, 784
  sample size, 907
  statistical model validation, 784
Ablation study
  component isolation, 93
  definition, 93
Abstraction problem
  definition, 316
Accelerator
  anatomy, 539
  bubbles, pipeline idleness, 366
  design, workload-specific memory, 574
  memory bandwidth limitations, 361
  ML, 547
  specifications, roofline analysis example, 634
Accelerator gain
  definition, 541
Accelerator-day
  definition, 11
Activation checkpointing
  definition, 310
  memory-compute trade-off, 310, 347, 397
  optimal placement, 397
  recomputation strategy, 313
  recomputation trade-off, 372
Activation function
  definition, 182
  nonlinear transformation, 181
  softmax, 185
  training efficiency, 353
Activation memory
  definition, 205
Activation unit, 557
Activation-based pruning
  definition, 479
Active learning
  definition, 439
  label prioritization, 137
  labeling cost reduction, 424, 439
Adam optimization, *see* Adam optimizer
Adam optimizer
  convergence properties, 356
  Kingma and Ba, 356
  memory overhead, 211, 356, 357
  state buffers, 211
AdamW
  decoupled weight decay, 356
Adaptive computation
  early exit, 495
Adaptive inference
  definition, 494
Adaptive resolution
  content-based selection, 707
Admission control
  queue depth threshold, 715
Adversarial debiasing
  fairness constraints, 845
  training cost, 845
Ahead-of-time compilation, 299
  predeployment, 735
Ahead-of-time optimization, 295
AI compute primitive
  definition, 550
AI democratization, 886
AI engineering
  definition, 24
AI memory systems
  bandwidth bottleneck, 565
AI runtime, 605
  dynamic execution management, 604
  vs. traditional runtime, 605
AI winters
  systems failure, 7
AI-assisted labeling
  preannotation, 136
  scalability, 136
Alert fatigue
  operational risk, 807
AlexNet
  GPU deep learning era, 545
  ImageNet breakthrough, 6
  ImageNet revolution, 229
  multi-GPU origin, 405
Algorithm-hardware adoption lag, 179
Algorithmic efficiency
  definition, 22
Algorithmic fairness
  healthcare ML, 89
Alignment gap
  proxy metric divergence, 832
AllReduce
  collective communication, 322
  definition, 405
  gradient synchronization, 649
AlphaGo, 12
AlphaGo Zero
  self-play training, 12
Always-on sensing
  power constraints, 41
Amdahl's law
  acceleration ceiling, 541
  appendix refresher, 930
  definition, 541
  diagnostic analogy, 18
  diminishing returns, 541
  optimization ceiling, 638
  pipeline bottleneck, 73
  preprocessing bottleneck, 703, 879
  speedup limits, 73
Amdahl, Gene, *see also* Amdahl's law
Anti-curriculum
  hard examples first, 439
AOT, *see* Ahead-of-time optimization
Apache Airflow
  pipeline orchestration, 775
Apache Arrow
  in-memory columnar format, 157
Apache Atlas
  data lineage, 865
Apache TVM
  accelerator compilation, 585
  ML compiler, 729
Application-specific integrated circuit, *see* ASIC
Architecture
  building blocks, 277
  computational graph, 228
  compute vs. memory trade-off, 273
  data-to-architecture mapping, 278
  encoder-decoder, 277
  parameter efficiency, 231
  representational power vs. efficiency, 229
  selection as bias matching, 232
  systems engineering, 228
Area under the learning curve
  definition, 429
Area under the ROC curve
  threshold independence, 92
Argmax
  prediction selection, 708
Arithmetic intensity, 44
  appendix refresher, 897, 910, 928
  attention layer, 364
  bottleneck diagnosis, 879
  definition, 195
  first use, 18
  FLOP/byte ratio, 272
  GPU utilization, 225
  high intensity workloads, 41
  roofline diagnostic, 543
  threshold for compute vs. memory bound, 634
  training operations, 363
Artifact
  interdependence, 94
  lineage tracking, 94
  reproducibility, 764
  versioning, reproducibility requirement, 764
Artificial neuron
  computing primitive, 180
  definition, 180
As-of join
  definition, 146
  point-in-time correctness, 146
ASIC
  benchmarking trade-off, 643
  flexibility trade-off, 546
Assumptions
  accelerator, 938
  bandwidth, 947
  energy, 946
  nameplate capacity, 938
  pricing, 949
  reference models, 945
  scale, 949
  training memory, 946
  unit conventions, 938
Asynchronous execution
  hiding transfer latency, 320
Attention
  memory bottleneck, 391
  quadratic complexity, 391
Attention head
  specialization, 262
Attention matrix, 261
Attention Mechanism
  global token interaction, 574

memory-bound softmax, 578
Attention mechanism
Bahdanau, 254
content-addressable memory, 262
definition, 255
fused kernel, 291
information bottleneck, 254
memory pressure, 574
multi-head, 262
quadratic bottleneck, 260
query-key-value, 256
scaled dot-product, 262
self-attention, 261
similarity computation, 372
AUC
anomaly detection performance, 631
Audit trail
accountability logging, 865
inference-time logging, 866
multi-tier logging, 865
storage scale, 865
AutoAugment
search cost, 443
autocast
mixed-precision context, 313
Autograd
automatic differentiation, 210
engine, internals, 311
reverse-linked graph, 311
Autograd tape
definition, 292
dynamic graph construction, 292
memory cost, 292
reverse traversal, 331
Automatic differentiation
appendix refresher, 913
computational graph, 359
definition, 306
forward mode, 306
forward vs. reverse mode, 306
forward-mode dual numbers, 307
framework role, 286
reverse mode, 204, 306, 307
reverse-mode backpropagation, 307
Automatic mixed precision
framework support, 387
Automotive AI
real-time safety requirements, 609
Autonomous vehicle
latency requirements, 57
Autoregressive generation, 266
definition, 262
memory-bound, 41
Autoregressive model
autoregressive, serial bottleneck, 742
token generation, 744
Autoscaling
ML cold-start trade-off, 782
ML workloads, 782
startup latency, 751

Back-translation
pipeline latency, 442
text augmentation, 442
Backpressure
definition, 124
streaming systems, 124
Backpropagation
appendix refresher, 913
computational asymmetry, 307
computational cost, 359
definition, 203
gradient computation, 203
historical development, 179
historical introduction, 352
memory cost, 179
memory requirements, 359, 547
provenance, 352
through time (BPTT), 251
Backward pass
gradient computation, 372
memory operations, 372
triggering gradient computation, 331
Bandwidth
bottleneck, video streaming, 52
latency trade-off, 54, 933
PCIe, model swapping, 741
Bandwidth taper
interconnect hierarchy, 573
Bandwidth-bound workload
definition, 41
Baseline model
reference point selection, 625
Batch
loss averaging, 198
training iteration, 193
Batch inference
utilization targets, 789
Batch ingestion
definition, 123
Batch normalization, 248
hardware implementation, 555
mode-dependent behavior, 327
training speedup, 247
training stability, 185
training-serving skew, 247
Batch Size
arithmetic intensity impact, 582
Batch size, 235
arithmetic intensity effect, 364
gradient stability, 202
hardware alignment, 371
inference trade-off, 219
loss computation, 198
Batch throughput
definition, 655
Batching
batch formation delay, definition, 720
batch, small, CPU advantage, 736
etymology, 717
fallacy of larger batches, 755
hardware utilization, 353
latency penalty, 720
training vs. serving, 717
window, latency-throughput trade-off, 700
Battery
coin-cell deployment, 61
Battery life
ML impact, 58
Battery tax
always-on constraints, 58
definition, 58
Bayes error rate
appendix refresher, 901
Beam search
decoding strategy, 744
Benchmark
etymology, 624
Benchmark carpentry
definition, 624
Benchmark contamination
LLM memorization risk, 678
Benchmark coverage
incomplete problem representation, 669
Benchmark engineering
intentional optimization bias, 671
Benchmark gaming
distorting performance metrics, 672
vendor optimization for tests, 640
Benchmark harness
test infrastructure, 627
Benchmark saturation
outdated benchmarks pitfall, 683
Benchmark-production gap
definition, 620
Benchmarking
battery, duty cycle specification, 664
benchmarks as proxies, 622
definition, 620
end-to-end vs. component metrics, 623
energy, first-class metric, 640
fallacy of direct performance translation, 683
historical evolution, 640
probabilistic variability, 642
system, definition, 621
Benchmarking granularity
definition, 632
Berkson, Joseph, *see also* Logits
BERT
benchmarking baseline, 625
compression techniques, 483
BF16
appendix refresher, 935
dynamic range preservation, 389
exponent range preservation, 385
hardware support, 389
reduced-precision format, 205
Bias
activation threshold, 188
attribute removal fallacy, 866
gender discrimination, 830
geographic, training data, 118
historical data encoding, 830
mitigation strategies, 834
parameter overhead, 188
proxy variables, 830
signal aggregation, 181
Bias feedback model
disparity amplification, 879
Binarization
definition, 513
Binding constraint
definition, 13
Biological neuron
analogy limits, 180
Bitter lesson
cloud scale, 47
definition, 11
feature learning, 177
bitter lesson, The, 11, 12
BLAS
historical foundation, 288
matrix computation foundation, 352
neural networks, 234
performance foundation, 288
training compute hierarchy, 352
utilization, 234
vector and matrix operations, 324
BLEU
Goodhart's law example, 627
Blue-green deployment
definition, 783
duplicate capacity, 783
Bobrow, Daniel
STUDENT, 7
Bottleneck principle
definition, 40
Boundary erosion
dissolution of modularity, 769
Bounding box
annotation, 132
definition, 132
Break-even analysis
edge vs. cloud utilization, 50
Break-even point
data selection ROI, 457
Brittleness
expert systems, 11
Broadcasting
definition, 912
Bulkhead pattern
failure containment, 813
resource isolation, 813
Byte pair encoding
tokenization, 369

C4
quality filtering, 433
CACE principle
change propagation, 769
Cache locality, 568

Calibration
clinical trust, 95
dataset, representative traffic, 733
definition and etymology, 674
INT8 quantization, 733
production traffic mismatch, 755
release evidence, 95
Canary deployment
definition, 783
statistical validation, 783
Canary request
fan-out protection, 716
Capacity planning
infrastructure sizing, 752
Capacity-bound workload
definition, 41
Carbon footprint
energy to emissions conversion, 855
Carbon intensity
definition, 853
grid emissions, 857
US grid average, 857
Carbon-aware scheduling
lower-carbon times and regions, 857
Categorical Encoding
one-hot encoding, 142
Categorical feature, 268
Causal mask, 262
Central processing unit, *see* CPU
Chain rule
appendix refresher, 913
backpropagation foundation, 359
depth constraint, 204
gradient decomposition, 204
Channelwise quantization, *see* Quantization
CheckList, 841
Checkout-free retail
edge processing, 57
Checkpointing
definition, 374
I/O cost, 216
Chinchilla
compute-optimal scaling, 428
compute-optimal training, 428
tokens-per-parameter diagnostic, 429
Chiplet
modular die interconnect, 607
CI/CD pipeline
ML-specific adaptation, 778
CIFAR-10
classification reference dataset, 625
Circuit breaker pattern
cascade failure prevention, 813
Class balance
coverage metric, 678
Class imbalance
coverage failure mode, 678
minority-class performance, 224
Classical machine learning
feature engineering, 176
Classification label
storage requirements, 132
Cleanlab
label error detection, 435
ClinAIOps
healthcare ML operations, 818
MLOps comparison, 820
Clinician oversight loop, 818
CLIP, *see* Contrastive Language-Image Pretraining
Closed division
MLPerf submission, 656
MLPerf training, 645
Cloud infrastructure
utility model, 47
Cloud ML, 26
accelerator infrastructure, 49
characteristics, 36
consumer-scale applications, 51
data center scale, 47
definition, 47
latency limitations, 49
privacy concerns, 49
scaling limits, 67
workload archetypes, 47
Cloud pricing
cost-latency trade-off, 133
CNN, 235
definition, 236
spatial data reuse, 574
COCO
object detection dataset, 625
Code generation
hardware-specific instructions, 604
Cohen's kappa
inter-annotator agreement, 679
label quality, 135
Coin-cell battery
deployment longevity, 61
Cold start
anatomy, 738
bursty traffic impact, 755
cloud serving, 698
definition, 738
inference concern, 660
scaling events, 738
Cold start latency
physical lower bound, 655
serverless AI impact, 655
Collation
batch assembly, 323
Columnar storage
Parquet, 158
Communication overhead
scaling efficiency, 404
COMPAS
calibration and error rates, 843
recidivism algorithm audit, 843
Compilation continuum
principle, 301
Complexity tax
definition, 65
ML vs. heuristics, 65
Compositionality
definition, 190
pattern decomposition, 190
Compressed sparse row
appendix refresher, 911
definition, 520
Compression
algorithm selection, 159
Computation Placement
processing element assignment, 585
Computation scheduling
definition, 601
parallel execution, 603
Computational graph
definition (DAG), 290
framework optimization, 286
graph execution, 286
static, 294
Computational photography
pipeline constraint, 57
Compute efficiency
definition, 22
Compute-Bound
roofline classification, 575
Compute-bound
operation classification, 291
resolution scaling, 707
Compute-bound vs. memory-bound
definition, 40
memory-bound optimization, 40
training vs. inference, 44
Compute-bound workload
definition, 41
Compute-optimal frontier
Chinchilla scaling laws, 428
Computer engineering
hardware-software integration, 25
Concept drift
adversarial evolution, 766
changing relationships, 790
definition, 161
validation, 97
Concept-based explanation
human-understandable feature, 849
Conditional computation
definition, 495
Confidence interval
appendix refresher, 906
benchmark reporting, 667
Confidence threshold
automation trade-off, 220
decision making, 222
Confident learning
mislabeled example detection, 679
Confidential computing
hardware secure enclaves, 860
Configuration debt
Facebook case study, 772
parameter sprawl, 771
Confusion matrix
fairness analysis, 842
Consensus labeling
quality control, 135
Conservation of complexity
heuristic limits, 878
meta-principle, 878
model compression, 472
Consistency imperative
definition, 140
training-serving parity, 140, 765
Consistency regularization
compute trade-off, 445
semi-supervised technique, 445
Constant folding, 295
compile-time optimization, 734
Constrained hardware
inference deployment, 217
Constraint layer
definition, 88
statistical physical operational, 88
Constraint propagation principle
definition, 82
Containerization
environment parity, 782
ML deployment, 781
reproducible environments, 147
Continuous batching
LLM serving, 744
scheduling granularity, 744
Continuous therapeutic monitoring
healthcare AI, 818
human oversight, 818
Contrastive Language-Image Pretraining
semantic deduplication, 434
Contrastive Learning
batch-based methods, 447
pretext task, 447
Convergence
monitoring, 216
rate, clean vs. noisy data, 433
time-memory trade-off, 356
Convolution
computational cost, 371
data reuse, 235
depthwise, 244
depthwise separable, 244
etymology, 235
hardware optimization, 245
kernel, 238
matrix multiplication equivalence, 553
padding, 238
pointwise, 244
stride, 238
Convolutional neural network, *see* CNN
Coordinated omission
closed-loop measurement trap, 627
Coordination overhead
distributed processing, 145
multi-accelerator data selection, 455
Coreset

data efficiency, 461
definition, 435
etymology, 435
geometry origins, 435
ICR example, 426
Correction cascade
sequential dependencies, 769
Zillow case study, 772
Cosine annealing
learning rate schedule, 358
Cost amortization
self-supervised pretraining, 447
Cost modeling
data selection economics, 455
Cost per inference
operational tracking, 789
serving economics, 751
Cost-aware automation
retraining decisions, 765
Cost-Performance Analysis
accelerator economics, 612
Counterfactual analysis
prediction debugging, 806
Covariate shift
definition, 161
input distribution change, 680
CP decomposition
definition, 484
CPU
inference optimization, 736
neural network execution, 198
CPU affinity
latency reduction, 735
CPU backend
optimized BLAS libraries, 334
Cray-1
AI accelerator lineage, 552
Credit assignment problem
weight responsibility, 203
Critical batch size
throughput-convergence limit, 355
Critical path
composed request, 885
optimization, 44
Cross-entropy loss
classification loss, 199
definition, 199
information theory, 199
Cross-industry standard process for data mining
lifecycle influence, 78
Cross-layer interaction
cross-optimization effects, 462
Crowdsourcing
annotation, 121
data labeling, 121
cuBLAS
hardware-accelerated linear algebra, 352
kernel selection, 323
NVIDIA linear algebra, 602
CUDA
context, initialization overhead, 739
cores, streaming processors, 585
dispatch overhead, 299
ecosystem lock-in, 558
runtime, OS layer, 319
stream definition, 320
stream overlap, 320
CUDA Graphs
launch replay, 300
CUDA graphs
kernel launch elimination, 735
kernel launch replay, 636
CUDA MPS
context sharing, 739
cuDNN
benchmarking dependency, 633
NVIDIA deep learning library, 602
optimized implementations, 372
Curriculum learning
difficulty scorer and pacing function, 438
etymology, 438
optimization landscape, 438
Custom autograd function
backward implementation, 312
Custom differentiation rule, 311
CutMix
image augmentation, 442
label mixing, 442
Cutout
augmentation efficiency, 442
image augmentation, 442

D·A·M Taxonomy
bottleneck classification, 378
DAM taxonomy, *see* D·A·M taxonomy
D·A·M taxonomy
alignment of components, 223
appendix refresher, 893
axis intersections, 895
data selection, 424
definition, 13
diagnostic framework, 682
iron law factoring, 17
lifecycle stages, 84
ML test score, 811
responsibility diagnosis, 833
scorecard, 899
training vs. inference, 44
workload classification, 41
Dark knowledge
class correlations, 487
definition, 486
Dark Silicon
specialization driver, 545
Dartmouth Conference, 7
systems constraints, 7
Data
as source code, 13
coverage, requirements, 118
partitioning, query optimization, 159
physics of, 149
transformation, techniques, 141
Data acquisition
scalability, 120
scale vs. quality, 120
source evaluation, 118
strategies, 118
Data augmentation
definition, 442
synthetic data, 121
training diversity, 221
Data benchmarking
definition, 621
training set validation, 678
Data card
compliance documentation, 863
Data cascade
definition, 110
first use, 83
silent failure, 110
Data center
energy consumption, 857
ML infrastructure, 47
Data cleaning
error correction, 141
inconsistency removal, 141
Data collection
iron law data term, 89
Data compression ratio
definition, 430
Data curation
end-to-end generalization, 632
Data debt
definition, 162
hidden liability, 160
strategic vs. unconscious, 164
Data drift
definition, 16
degradation equation, 141
detection, 160
silent model degradation, 100
Data echoing
amortizing I/O costs, 452
GPU utilization, 452
Data engineering
definition, 108
Data freshness
SLA, 124
staleness detection, 794
Data governance
definition, 858
enforcement mechanisms, 858
model artifact compliance, 867
Data gravity
appendix refresher, 903
definition, 149
physics, 149
Data gravity principle
cloud limitations, 49
physical anchor, 879
Data labeling
clinical economics, 445
systems engineering, 131
Data lake
schema flexibility, 153
unstructured storage, 153
Data lakehouse
data gravity, 149
Data leakage
definition, 109
Data lineage
audit trail, 101
debugging and compliance, 147
definition, 147
GDPR compliance, 865
monitoring at scale, 101
processing history, 148
regulatory compliance, 101
transformation tracking, 147
Data loading
shard-based sequential I/O, 451
Data Locality
spatial and temporal, 561
Data locality
code-to-data, 150
data-to-code, 150
Data locality invariant
definition, 54
Data localization
cross-border transfer, 863
Data management
MLOps lifecycle, 775
Data minimization
privacy by design, 863
Data movement
broadcast, 276
energy dominance, 39, 588
gather, 276
optimization strategies, 588
reduction, 276
scatter, 276
Data Parallelism
training strategy, 649
Data parallelism
framework implementation, 405
gradient averaging, 405
gradient synchronization, 405
multi-accelerator scaling, 607
Data path
customization, 546
Data pipeline
architecture, 123
automated workflows, 775
definition, 123
ingestion and preprocessing, 365
storage throughput, 368
throughput optimization, 322
Data poisoning
training pipeline security, 861
Data prefetching
accelerator utilization, 383
buffer management, 383
Data quality
as code, 129
container, 129
content, 129
expectation tests, 129
mechanical vs. semantic, 129
monitoring, 792
noise penalty on convergence, 433

statistical monitoring, 127
vs. data quantity, 74
Data quality as code
definition, 129
Data quality multiplier
label noise penalty, 433
Data redundancy
empirical evidence, 433
Data reuse, 568
accelerator optimization, 589
Data science
vs. data engineering, 109
Data selection
definition, 423
first use, 22
optimization ordering, 424
Data shuffling
random access penalty, 368
Data supply rate
bandwidth constraint, 156
Data swamp
governance failure, 153
Data validation
monitoring, 127
Data versioning
DVC tool, 775
model reproducibility, 148
reproducibility, 80, 778
Data wall
definition, 422
scaling constraint, 422
zero learning signal, 426
Data warehouse
analytical queries, 152
OLAP, 152
Data-as-code principle
definition, 6
logical program, 879
Data-centric AI, 6
dataset enhancement paradigm, 680
Data-model interface
feature consistency, 762
Database
OLTP for ML, 152
DataComp
data-centric benchmark, 680
data-centric benchmarking, 680
Dataflow
optimization strategies, 589
Dataflow Architecture
definition, 584
Dataflow optimization
data movement minimization, 588
Dataflow strategy, 599
DataHub
data lineage, 865
DataLoader
configuration parameters, 384
throughput optimization, 322
DataPerf
data quality benchmark, 680
Dataset
iterable-style, 323
map-style, 323
Dataset release
definition, 146
Dataset saturation
performance capability illusion, 681
Datasheets for datasets
training data documentation, 850
DAWNBench
time-to-accuracy evaluation, 640
Dead code elimination, 295
graph optimization, 295
Dead neuron, 246
Dead-letter queue
definition, 127
Dean, Jeff
latency numbers, 926
Debugging
data pipelines, 164
Debugging checklist
runbook steps, 806
Decision framework
paradigm selection, 65
Decode, 262
Decode phase
memory-bandwidth bound, 753
sequential generation, 753
Deduplication
data selection optimization, 434
definition, 141
embedding-based, 434
hash-based, exact matching, 434
storage and I/O cost reduction, 424
Deep Blue, 12
custom chess hardware, 12
Deep learning
automatic feature learning, 177
definition, 175
first use, 10
network depth, 190
Deep learning framework
automatic differentiation, 288
DeepBench
operator-level testing, 633
Deferred execution
graph construction, 286
Degradation equation
definition, 16
distribution shift, 72
operational implementation, 760
Delta Lake
time travel, 148
Demographic drift
definition, 89
Demographic parity
definition, 842
Dennard Scaling
end of, 545
Dennard scaling
origin and breakdown, 38
Dense connectivity, 232
parameter scaling, 189
Dense Layer
roofline analysis, 580
Deployment
controlled, risk mitigation, 780
iron law overhead minimization, 98
Deployment budget
definition, 469
Deployment paradigm
definition, 25
operating regime, 36
Deployment target
cloud to edge spectrum, 729
Depth
exponential advantage, 191
Depthwise conv, 577
Depthwise Convolution
low arithmetic intensity, 576
Depthwise separable convolution, 244
mobile deployment, 85
mobile efficiency, 490
MobileNet, 244
Deterministic Transformation
reproducibility, 144
Developer feedback loop, 818
Device management
CPU-GPU transfers, 319
DevOps
MLOps extension, 760
Diabetic retinopathy
case study introduction, 81
deployment scale, 81
Differential privacy
epsilon budget, 861
epsilon trade-off, 861
formal guarantees, 861
mathematical guarantees, 834
noise injection, 861
Differentiation problem
definition, 306
Diffusion model
text-to-image synthesis, 443
Digital signal processing
multiply-accumulate units, 545
Dimensional homogeneity
appendix refresher, 930
Direct memory access, *see* DMA
Disaggregated evaluation
definition, 839
demographic stratification, 839
fairness testing, 840
Disparate impact
legal doctrine, 840
Dispatch overhead
definition, 294
law, 303
Dispatch overhead reduction
kernel launch batching, 636
Display overlay
GPU mapping, 608
Distance penalty
cloud latency, 48
Distributed computing
system-level performance, 640
Distributed data
clinic coordination cost, 90
Distributed Processing
beyond one machine, 145
scaling, 145
Distributed selection
network topology constraints, 455
Distributed training
beyond one node, 408
data selection challenges, 453
scaling efficiency, 406
Distribution shift
causes of model degradation, 832
label shift, 161
monitoring, 160
responsibility lens, 832
train-to-production alignment, 680
Diversity Sampling
active learning query strategy, 440
DLRM, 268, 270
capacity-bound benchmark, 656
capacity-bound workload, 876
embedding bottleneck, 350
I/O-bound serving, 705
interaction layer, 269
memory capacity bound, 21, 42
memory-bound constraint, 42
recommendation model benchmark, 656
DMA
computation overlap, 572
PCIe data movement interface, 319
Documentation debt
data provenance, 162
definition, 162
Domain adaptation
feature alignment, 444
Domain gap
training-serving skew, 443
Domain randomization
bridging synthetic-real gap, 443
rendering trade-off, 443
Domain-Specific Architecture, *see* DSA
Dot product
appendix refresher, 910
definition, 182
Double buffering
latency hiding, 606
Double descent, 216
DRAM
energy cost, 549
Drift
covariate shift, 790
Drift detection delay
label latency, 790
statistical monitoring, 791
Dropout
definition, 216
training vs. inference, 327
DS-CNN
monitoring strategy, 817
DSA

definition, 546
efficiency trade-off, 546
scaling law breakdown, 546
Dual mandate
definition, 4
Dual number
forward mode AD, 307
Duty cycle
accelerator utilization, 709
DVC
data versioning, 766
dataset versioning, 775
git semantics, 148
DVFS
dynamic voltage and frequency scaling, 664
power-performance envelope, 609
thermal throttling, 684
warm-up measurement artifact, 633
Dying ReLU problem
definition, 185
Dynabench
dynamic evaluation, 681
Dynamic batching
definition, 717
runtime batch sizing, 717
time window, 721
Dynamic computation, 277
Dynamic graph
definition, 292
eager execution, 292
runtime construction, 292
Dynamic inference
definition, 697
utilization targets, 789
Dynamic kernel execution
runtime adaptation, 605
Dynamic quantization
definition, 509
Dynamic random access memory, *see* DRAM
Dynamic selection
active learning, 439
curriculum learning, 438
definition, 424
training-time optimization, 438
Dynamic voltage and frequency scaling, *see* DVFS

Eager execution
define-by-run model, 292
definition, 286
optimization limitations, 291
optimization trade-off, 286
Earth mover's distance
multi-dimensional drift, 793
Edge accelerator
deployment selection, 56
Edge AI
deployment patterns, 786
Oura Ring case study, 817
Edge deployment
bandwidth constraints, 99
constraint triangle, 656
power constraints, 854
storage, 153
TinyML frameworks, 304
Edge hardware
resource limits, 55
Edge inference
power tiers, 217
Edge ML
autonomous vehicles, 57
bandwidth reduction, 52
data sovereignty, 51
definition, 51
deployment challenges, 55
distributed processing, 55
industrial IoT, 57
latency and bandwidth, 26
latency benefits, 36
privacy benefits, 54
smart retail, 57
Edge orchestration
fleet challenges, 55
Edge serving
embedded function calls, 698
Edge summary
uplink reduction, 89
Edge TPU
inference efficiency, 217
operator coverage, 653
Efficiency
Jevons paradox, 23
EfficientNet
algorithmic efficiency, 23
Einstein summation convention
definition, 910
EL2N
definition, 435
proxy transferability, 435
Element-wise operation
memory bandwidth, 324
ELIZA
brittleness, 7
ELT
iteration speed, 125
load first, 125
Embedded GPU accelerator
edge deployment, 56
Embedding
definition, 268
etymology, 268
memory access pattern, 268
table, 269
Embedding Lookup
memory-bound operation, 576
Embedding table
capacity bottleneck, 350
memory capacity bound, 41
Embodied carbon
hardware manufacturing, 857
Emergent behavior
definition, 103
Encoder-decoder
attention as routing, 277
definition, 254
Encryption
data lifecycle protection, 859
End-to-end benchmark
complete workflow measurement, 632
Energy
budget, mobile inference, 726
data movement cost, 547
Energy consumption
hardware platforms, 198
training costs, 853
Energy efficiency
TFLOPS per watt, 613
training cost accounting, 650
Energy harvesting
milliwatt threshold, 60
Energy hierarchy
reference sources, 926
Energy of transmission
local vs. cloud, 52
Energy per inference
binding constraint, 41
paradigm comparison, 54
Energy tax
definition, 19
Energy wall
battery constraints, 52
Energy-movement principle, 152
data locality, 879
Ensemble learning
accuracy vs. deployment trade-off, 92
Entanglement
ML system coupling, 769
Entropy
appendix refresher, 908
definition, 908
Environment parity
production deployment, 762
Environmental impact
carbon accounting, 857
ML systems, 853
Epoch
dataset iteration, 214
training cycle, 366
training cycles, 214
Equal opportunity
definition, 842
Equalized odds
definition, 843
postprocessing, 843
Equivariance
translation, 239
Error signal
backward propagation, 205
ETL
reprocessing cost, 125
EU AI Act
risk classification, 862
systems requirements, 862
Evaluation pipeline
validation metrics, 365
Evasion attack
inference adversarial perturbation, 861
Execution problem
definition, 290
Expected calibration error
confidence-accuracy gap, 675
Expected Model Change
active learning query strategy, 440
Experiment tracking
definition, 94
tools and practices, 94
Expert systems, 8
knowledge engineering, 176
Explainability
architecture trade-off, 847
definition and purposes, 847
post-hoc methods (SHAP, LIME), 848
Exploding gradient problem, 246
training instability, 205
Exponential unit, 557
Eyeriss
row-stationary dataflow, 590

F1 score
precision-recall harmonic mean, 626
Face ID
privacy architecture, 59
Facebook AI Similarity Search
selection infrastructure, 450
Fairness
accuracy trade-off, 846
governance objective, 808
individual vs. group, 834
mathematical definitions, 834
Fairness metrics
demographic parity, 842
equal opportunity, 842
equalized odds, 843
impossibility theorem, 843
loan approval case study, 842
FAISS, *see* Facebook AI Similarity Search
False Accepts per Hour, 113
False negative rate
fairness impact, 843
False positive rate
always-on systems, 113
Fault tolerance
training checkpoint strategy, 650, 886
Feature attribution
debugging techniques, 806
Feature Computation
hybrid patterns, 144
pipeline vs. loader, 143
Feature computation
placement, 143
Feature computation placement
definition, 143
Feature consistency
training-serving alignment, 762
Feature contract
schema specification, 809
Feature coverage

deployment variation, 679
Feature Engineering
domain knowledge, 142
Feature engineering
classical ML, 176
Feature engineering bottleneck
definition, 9
Feature extraction
hierarchical, 240
Feature importance
prediction attribution, 848
Feature learning
hierarchical, 175
Feature map, 237
Feature store
consistency, 142
consistency guarantee, 142
definition, 146
offline vs. online stores, 146
point-in-time retrieval, 149
scale, 775
training-serving bridge, 146
training-serving consistency, 775
Federated averaging
data minimization, 862
Federated learning
data minimization, 862
definition, 862
privacy preservation, 862
Feedback Loop
multi-scale temporal structure, 102
Feedback loop
problem amplification, 851
production data, 161
recommendation amplification, 833
self-reinforcing bias, 770
YouTube case study, 772
Feedforward network
architecture, 188
Feeding loss
definition, 151
Feeding problem
definition, 150
Field-programmable gate array, *see* FPGA
FixMatch
label-compute trade-off, 446
FlashAttention
2, improved parallelism, 394
3, FP8 tensor cores, 394
appendix refresher, 916
framework fusion example, 291
I/O-aware algorithm, 391
I/O-aware design, 365
IO complexity reduction, 393
memory bandwidth optimization, 391
memory optimization, 578
origin and engineering insight, 391
tiled attention implementation, 599
tiling strategy, 392
FlatBuffers
etymology, 701
zero-copy serialization, 701
Flaw of averages, 840
Flax
JAX neural network library, 326
Fleiss' kappa
inter-annotator agreement, 135
Floating point
FP32 precision, 218
Floating-point operations per second, *see* FLOP/s
Floating-point unit
history, 545
precision bottleneck, 545
FLOP/s
peak vs. achieved, 634
Flow rate
definition, 150
Food and Drug Administration
SaMD constraint, 97
Forced alignment
automated labeling, 136
speech labeling, 138
Forgetting events
coreset selection, 436
pruning trade-off, 436
Forward pass
compute operations, 371
kernel dispatch, 331
memory management, 371
Forward propagation
computation process, 194
definition, 194
layer computation, 352
matrix operations, 195
Foundation model
definition, 447
homogenization risk, 447
Four pillars framework
definition, 109
Four-fifths rule
adverse impact threshold, 840
FP16
appendix refresher, 935
dynamic range limitation, 385, 387
half precision, 205
subnormal flushing, 389
training precision, 547
FP32
gradient computation, 547
numerical precision, 205
FP32 master weights
mixed-precision training, 347
optimizer stability, 388
FP8
definition, 500
E4M3 and E5M2 formats, 500
formats, 500
hardware acceleration, 389
FPGA
reconfigurability trade-off, 604
Framework profiler
performance analysis tool, 636
timeline analysis, 736
Framework selection
constrained optimization, 339
trade-off analysis, 339
Frequency boosting
warm-up measurement artifact, 633
Freshness debt
definition, 162
distribution divergence, 162
Fully-connected layer, 232
dense connectivity, 187
Fused execution, 597

GAN
synthetic data generation, 443
Gather, 552
Gating mechanism, 252
definition, 252
sequence models, 252
GDDR
off-chip DRAM variant, 571
GDPR
Article 22 automated decisions, 862
Article 22 requirements, 863
compliance, cloud deployment, 49
data collection requirements, 118
data minimization principle, 863
GELU, 354
GEMM
appendix refresher, 910
arithmetic intensity, 324, 929
definition, 196
hardware utilization, 288
matrix multiplication, 324
ML performance predictor, 599
GEMV
definition, 196
Gender classification
demographic disparities, 834
Gender Shades
algorithmic audit methodology, 839
demographic fairness study, 674
disaggregated evaluation, 839
Generalization
vs. overfitting, 215
Generative adversarial network, *see* GAN
Generative AI
benchmark evolution, 672
GLUE
benchmark saturation, 625
language understanding benchmark, 625
Glue code
integration overhead, 771
Goodhart's law
etymology, 623
metric optimization, 832
metric trap, 623
proxy decoupling, 833
Google Flu Trends
proxy drift, 5
Google TPUv4, 612
GPT-2, 266
arithmetic intensity, 273
autoregressive bottleneck, 42
bandwidth-bound decode, 20
data selection, 422
memory bandwidth, 876
training lighthouse, 347
GPT-3, 857
training compute, 351
training cost, 650
training energy, 12
training scale, 11, 47
weight storage, 155
GPU, 853
batching utilization, 717
etymology, 4
history, 545
parallel computation, 198
training, power budget, 217
utilization monitoring, 789
GPU backend
CUDA implementation, 334
GPU boost clock
benchmark variability, 651
GPU Interconnect
switching fabric, 573
GPU memory
training constraints, 372
GPU profiler
kernel-level analysis, 379
system-level analysis, 379
timeline analysis, 736
GPU utilization
batch size dependency, 717
GPU vs. CPU
economics, 751
Graceful degradation
data pipelines, 130
definition, 130
overload handling, 715
triggers, 884
Gradient
backpropagation, 203
bias gradients, 207
chain rule, 204
error signal flow, 205
flow in residual networks, 246
memory footprint, 306
partial derivatives, 206
vanishing and exploding, 205
weight gradients, 206
Gradient accumulation
framework behavior, 357
memory budget, 203
memory efficiency, 395
memory trade-off, 308
memory-compute trade-off, 395
Gradient checkpointing
appendix refresher, 915
memory reduction, 359
Gradient compression
definition, 395

Gradient descent
  definition, 202
  full-batch computation, 355
  parameter optimization, 355
Gradient flow
  tensor detachment, 312
Gradient function node
  computation graph, 311
Gradient hook
  inspection and modification, 312
Gradient inspection, 311
Gradient instabilities
  training failure, 176
  training failure mode, 176
Gradient-based pruning
  definition, 479
GraNd
  gradient-based coreset scoring, 435
Graph break
  causes and detection, 296
Graph Compilation
  JIT overhead, 738
Graph compilation
  optimization, 734
Graph execution
  optimization advantages, 291
Graph Neural Network
  irregular computation, 585
Graph optimization
  computation graph restructuring, 602
  definition, 601
Graph retention
  multiple backward passes, 312
Graphcore
  IPU, 560
Graphics processing unit, *see* GPU
Great Expectations
  data validation, 129
Greedy decoding
  LLM generation, 743
Green AI
  compute reporting, 853
  efficiency as metric, 853
Green500
  HPC energy efficiency ranking, 640
Ground truth
  definition, 131
  proxy limitation, 131
Group-equivariant CNN, 240
gRPC
  inference protocol, 701
GRU
  efficiency, 252
  parameter efficiency, 252
GTX 580
  AlexNet training, 10
Guardrail metric
  A/B test constraint, 785
Gustafson's law
  appendix refresher, 930
Gustafson, John, *see also* Gustafson's law
Gzip
  decompression speed, 159

H100, 353, 389
  batching throughput, 717
  HBM capacity, 331
  MIG partitioning, 741
  roofline analysis, 929
  roofline prediction, 634
  Tensor Core throughput, 324
Half-life
  model decay, 799
Hardware abstraction
  framework design, 316
  two dimensions (data, execution), 316
Hardware abstraction layer
  portability, 615
Hardware acceleration
  definition, 538
Hardware accelerator, *see* Accelerator
Hardware balance, 575
Hardware cost
  deployment spectrum, 46
Hardware implementation
  theory-practice gap, 517
Hardware lottery
  architectural bias, 670
  benchmark bias, 670
Hardware Mapping
  combinatorial search space, 586
  definition, 584
Hardware mapping
  hybrid, layer-specific strategy, 600
Hardware selection
  workload matching, 614
Hardware specialization
  evolution, 544
Hardware spectrum
  deployment platforms, 46
  resource progression, 46
Hardware sustainability
  carbon footprint, 613
Hardware-Software Co-design
  definition, 540
Hardware-software co-design
  edge deployment, 883
HBM
  3D die stacking, 571
  bandwidth-cost trade-off, 549
  capacity constraint, 331
  memory bandwidth, 392, 754
  streaming weights, 275
  throughput requirements, 540
Headroom
  tail latency, 694
Healthcare AI
  deployment gap, 81
Hedging
  tail tolerance, 716
HELM
  multi-dimensional evaluation, 677
Herding
  coreset selection, 435
Heterogeneous computing
  mobile SoC, 607
Heuristic
  kernel selection trade-off, 603
  pruning, 475
Hidden state
  definition, 251
Hierarchical processing
  data flow, 69
  definition, 69
Hierarchical representation
  depth advantage, 190
Hierarchical selection
  distributed coreset, 454
High bandwidth memory, *see* HBM
High-risk AI
  classification criteria, 862
  EU classification, 836
Hinton, Geoffrey
  backpropagation, 179
HIPAA
  audit requirements, 865
  compliance, cloud deployment, 49
  ML system constraints, 863
Hochreiter, Sepp, 252
HOG
  fixed vs. learned features, 176
Holdout test set
  performance evaluation, 780
Host-Accelerator Communication
  transfer bottleneck, 572
Human-in-the-loop
  confidence-based routing, 99
  decision oversight, 839
  definition, 99
Hybrid architecture
  combining paradigms, 67
  voice assistant example, 75
  wake-word detection, 67
Hybrid machine learning, *see* Hybrid ML
Hybrid ML
  definition, 68
  hierarchical processing, 68
  integration strategies, 68
  progressive deployment, 68
  train-serve split, 68
Hybrid tiling, 594, 595
Hyperparameter
  learning rate, 202
  search cost, 91
Hypertension
  ClinAIOps case study, 819

i.i.d. assumption
  held-out evaluation limitation, 680
I/O bottleneck
  data loading, 157
  definition, 157
I/O-aware design
  algorithm optimization, 395
IB, *see* InfiniBand
Idempotency
  definition, 144
  ML pipelines, 779
  retry safety, 144
IIoT
  latency constraint, 51
im2col
  convolution to matrix, 553
  data center vs. mobile, 241
  definition, 241
  memory trade-off, 241
  memory-compute trade-off, 553
Image cropping
  racial bias, 834
ImageNet
  benchmark dataset, 119
  data scale, 119
  macro benchmark reference, 637
  scale, 21, 229
  standardized dataset, 625
  systems consequence, 229
Immutable artifact
  model registration, 778
Immutable data
  JAX, 336
Imperative programming
  framework heritage, 336
Importance sampling
  gradient variance, 464
In-memory database
  feature serving, 153
Incident response
  ML system failures, 850
  ML-specific challenges, 804
  structured processes, 804
Inductive bias
  definition, 228
  etymology, 228
  fully connected vs. convolutional, 188
  hierarchy, 278
  systems consequence, 228
  unifying concept, 228
Industry 4.0
  control loop latency, 57
Inference
  batch size selection, 219
  compiler auto-tuning, 653
  compiler, graph optimization framework, 653
  definition, 198
  deployment, 4
  deployment phase, 217
  engine, specialized, 731
  etymology, 217
  forward pass only, 198, 217
  latency optimization, 44
  latency requirements, 222
  optimization techniques, 219
  pipeline phase, 703
  real-time, storage requirements, 153
  zero-copy, mobile optimization, 698
Inference attack

privacy risk, 26
Inference inflation
definition, 720
Inference runtime
execution engine, 728
Inference server
architecture, 699
serving architecture, 699
Inference-time compute
reasoning models, 748
InfiniBand
node-to-node link, 607
Infinity Fabric
AMD interconnect, 573
Information density
definition, 908
Information roofline
definition, 901
Information theory, 908
Information-compute ratio
accuracy-cost frontier, 425
definition, 425
diminishing returns, 426
frontier, 426
Infrastructure as code
ML systems, 781
Infrastructure bottleneck
definition, 11
Initialization
Glorot, variance scaling, 205
He initialization, 205
lazy, deferred compilation, 739
Xavier/Glorot, 205
Input-stationary
dataflow strategy, 590
INT8
appendix refresher, 936
calibration dependency, 655
INT8 quantization
bandwidth trade-off, 879
Integration bottleneck
data movement cost, 549
Intel
Sapphire Rapids, 612
Intel 8087
floating-point coprocessor, 545
Intentional data engineering
fairness evaluation, 839
Inter-annotator agreement, 134
Inter-token latency
definition, 655
throughput trade-off, 655
Interface dependency
ML system challenge, 770
Intermediate representation
compiler pattern, 296
Intermittent computing
power management, 61
Interpretability
activation visualization, 224
Interpretable model
transparency by design, 849
Interrupt shielding
latency isolation, 736
Intersectional analysis
combined attribute testing, 841
Invariance testing
fairness verification, 841
IOPS
definition, 152
storage performance, 152
IoT
data wall, 51
deployment scale, 57
power-constrained environments, 656
Iron law of ML systems
appendix refresher, 895, 930
definition, 17
deployment analysis, 40
equation, 17
hardware acceleration, 541
multiplicative savings from data selection, 423
production monitoring, 789
sequential execution, 896
synthesis, 874
total operations reduction via data selection, 423
workflow mapping, 85
Iteration tax
definition, 84
Iterative pruning
definition, 476

Jaccard similarity
deduplication, 434
JAX
automatic vectorization, 337
composable program transformations, 336
composable transformations, 315
device-parallel mapping, 337
functional transformations, 336
gradient transformation, 337
JIT compilation, 337
pure functions requirement, 336
transform-based AD, 315
XLA compilation, 337
jax.grad
composable transformation, 336
Jensen-Shannon divergence
distribution shift, 793
Jetson AGX Orin
edge inference, 217
Jevons paradox
definition, 23
inference demand, 749
JIT compilation
fidelity vs. generality, 296
runtime optimization, 735
scripting, 297
shape specialization, 338
tracing, 297
JSON
serialization overhead, 701
storage overhead, 126
Jupyter Notebook
production risk, 780
Just-in-time compilation, 299

k-anonymity
privacy technique, 861
k-center
coreset selection, 435
coverage guarantee, 435
Karpathy, Andrej
Software 2.0, 4
Kendall notation
queuing models, 712
Kernel auto-tuning
algorithm selection, 731
Kernel dispatch
BLAS kernel selection, 331
hardware selection, 323
Kernel Fusion
hardware data movement, 597
Kernel fusion
backward-pass interaction, 310
compression-aware, 517
definition, 291
eliminating intermediate allocations, 310
graph compilation, 734
inference memory traffic, 219
inference optimization, 74
layer fusion, 731
memory efficiency, 597
memory traffic reduction, 597
memory-bound optimization, 582
optimizing memory-bound ops, 291
reducing memory writes, 588
serving compilers, 731
static graph optimization, 294, 295
Kernel launch
fixed overhead, 717
serving overhead, 717
Kernel Launch Latency
serial overhead, 541
Kernel launch latency
driver overhead reduction, 735
Kernel profiler
hardware-level analysis, 636
Kernel scheduling
hardware utilization, 606
Kernel selection
definition, 601
hardware-specific implementation, 602
Key management
encryption infrastructure, 859
Keyword detection rate
definition, 113
Keyword spotting, 245
automated labeling, 138
definition, 21
detection rate, 113
sub-megabyte models, 876
TinyML archetype, 42
Killer microseconds
latency gap, 706
Knowledge acquisition bottleneck
definition, 9
human throughput, 9
Knowledge distillation
benchmark evaluation, 675
definition, 486
efficiency gains, 854
etymology, 486
information density, 444
overview, 486
student model, 486
teacher model, 486
teacher-generated targets, 444
temperature parameter, 444, 487
Kolmogorov-Smirnov test
appendix refresher, 908
continuous drift, 793
covariate shift detection, 680
drift detection, 100, 127
feature distribution, 777
Kubernetes
ML orchestration, 782
Kullback, Solomon, *see also* Kullback-Leibler divergence
Kullback-Leibler divergence
appendix refresher, 907
definition, 160
distillation loss, 487
distribution comparison, 161, 793
Kung, H.T.
systolic array inventor, 561
KV cache
definition, 255
linear growth, 266
LLM memory, 745
LLM serving bottleneck, 789
memory capacity, 754
memory scaling, 266
multi-turn conversations, 747
offloading, 747
sequence length scaling, 745
session reuse, 747
transformer memory pressure, 599

l-diversity
privacy technique, 861
L0 norm
parameter count, 475
Lab-to-clinic gap
monitoring requirement, 100
Lab-to-field gap
data distribution shift, 90
definition, 90
Label propagation
graph-based semi-supervised, 445
Label quality
consensus mechanisms, 134
systematic checks, 134
Label quality drift
detection, 135
silent corruption, 135

Label shift
  definition, 161
Label Studio
  data quality, 775
Laboratory-to-deployment gap
  performance misalignment, 670
Ladder of abstraction, 288
LAPACK
  framework foundation, 288
Large language model, *see* LLM
Late correction cost
  definition, 84
Latency
  bandwidth trade-off, 933
  decision thresholds, 44
  deployment constraint, 38
  deterministic, 735
  edge deployment, 26
  first-token, LLM responsiveness, 655
  inference real-time requirement, 654
  inference vs. user-perceived, 754
  mean, monitoring pitfall, 755
  p50, median response time, 702
  p95, percentile target, 702
  percentile, p50 p95 p99 reporting, 626
  percentiles (p50, p95, p99), 702
  real-time constraints, 222
  responsiveness constraint, 38
  selection, dynamic selection bottleneck, 450
  serving constraint, 694
  throughput trade-off, 755
Latency budget
  definition, 703
  end-to-end analysis, 788
  optimization objectives, 702
  request lifecycle breakdown, 703
  tail latency constraint, 880
Latency jitter
  resource contention, 735
Latency vs. throughput
  paradigm trade-offs, 63
  trade-offs, 44
Layer
  hidden, 186
  input, 186
  organization, 186
  output, 186
Layer fusion, *see* Kernel fusion
Layer normalization, 248
  Transformer, 262
Layer sensitivity
  precision tolerance, 733
LayerNorm
  kernel fusion target, 582
  memory-bound operation, 576
Layerwise quantization, *see* Quantization
Lazy Evaluation
  data processing, 145
Learnability, 231
Learnability gap, 231
Learning rate
  cosine decay, 358
  scheduling, 358
  update magnitude, 202
  warmup, 358
Learning rate scheduling
  definition, 358
LeCun, Yann, 237
  commercial deployment, 237
  LeNet, 237
Leibler, Richard, *see also* Kullback-Leibler divergence
Leiserson, Charles E.
  systolic array inventor, 561
LeNet
  architecture comparison, 222
Li, Fei-Fei
  ImageNet creation, 119
Light barrier
  definition, 38
  latency floor, 38
  safety-critical systems, 48
Lighthouse model
  bottleneck probe, 876
  definition, 20
  reference workloads, 41
LIME
  local explanations, 848
Lineage tracking
  model provenance, 777
  model registry, 803
Linear scaling rule
  large batch training, 373
Linear transformation
  layer computation, 194
Linnainmaa, Seppo
  automatic differentiation, 179
LINPACK
  matrix operations benchmark, 640
Little's law
  appendix refresher, 932
  capacity planning, 711
  concurrency calculation, 711
Little, John, *see also* Little's law
Llama
  bandwidth-bound decode, 20
  memory-bound inference, 42
Llama 2
  70-billion-parameter bandwidth-bound workload, 882
Llama 3
  8-billion-parameter serving case study, 752
Llama 3 8B
  fine-tuning memory, 362
LLM, 266, 853
  KV-cache serving, 789
  labeling assistance, 137
LLM serving
  8-billion-parameter case study, 752
  production case study, 752
  token generation, 742
Load balancer
  serving infrastructure, 702
Locality crossover
  worked example, 55
Locality-sensitive hashing
  near-duplicate detection, 434
  probabilistic bucketing, 434
Log-sum-exp trick
  appendix refresher, 916
  numerical stability, 201
Logic unit cost
  activation functions, 185
Logits
  appendix refresher, 916
  classification output, 708
  inference optimization, 185
  logit function, 916
  raw network scores, 185
Loop Ordering
  computational efficiency, 587
LoRA
  fine-tuning memory, 362
Loss function
  cross-entropy, 199
  error measurement, 198
  etymology, 194
  landscape geometry, 194
  optimization target, 198
Loss landscape
  nonconvex optimization, 202
Loss Scaling
  gradient underflow prevention, 313
Loss scaling
  appendix refresher, 936
  gradient underflow prevention, 387
Lottery ticket hypothesis
  compression trade-off, 677
  etymology, 482
Low-rank factorization
  definition, 483
LPDDR
  low-power DRAM variant, 571
LSTM
  compute overhead, 252
  gating mechanism, 252

M/D/1 queue
  deterministic service, 712
M/M/1 queue
  Erlang origin, 711
  wait time formula, 711
M/M/c queue
  multi-server, 715
MAC
  definition, 234
  energy cost, 234
  multiply-accumulate operation, 181
  neural computation, 181
Machine learning
  definition, 7
Machine learning compiler, 601
Machine learning framework, *see* ML framework
Machine learning lifecycle, 78
  definition, 78
  iteration cycles, 83
  storage requirements, 153
  time allocation across stages, 83
Machine learning systems
  definition, 13
Machine unlearning
  definition, 866
  systems trade-offs, 867
Macro benchmark
  subsystem evaluation, 632
Magnitude-based pruning
  definition, 475
Manifold hypothesis, 231
  learnability, 231
Masked Language Modeling
  pretext task, 447
Masked operations, 552
Matrix engine, 612
Matrix multiplication
  dense operations, 371
  forward propagation, 195
  GEMM optimization, 196
  neural network workhorse, 552
  training operations, 352
Matrix operation
  hardware acceleration, 553
  hierarchical decomposition, 552
  neural network workloads, 559
Matrix tiling, 276
Maturity level
  evolutionary stage, 812
Maximum mean discrepancy
  covariate shift detection, 680
McCrory, Dave, *see also* Data gravity
McCulloch-Pitts neuron, 231
McNemar's test
  appendix refresher, 906
Mean time between failures
  fleet-scale reliability, 885
Mean time to failure
  component reliability, 885
Measurable property
  responsibility requirements, 867
Measurement boundary
  production serving energy, 663
Measurement framework
  data selection metrics, 425
Measurement variability
  sources in ML systems, 642
Mechanical Turk
  labeling scale, 121
Medical AI
  performance metrics, 92
Membership inference attack
  privacy testing, 861
  privacy validation, 862
Memory
  capacity, LLM throughput, 754
  capacity-bound workload, 270
  constraints, mobile RAM, 726

locking, mlock, 735
reuse, activation buffers, 218
sharing, GPU multi-model, 741
Memory Access
irregular patterns, 568
Memory access
random, 275
sequential, 275
strided, 275
Memory Bandwidth
achievable vs. peak, 572
scaling trends, 565
Memory bandwidth
activation function bottleneck, 354
bottleneck, 12
bottleneck mitigation, 395
bound workload, 267
LLM bottleneck, 753
LLM latency, 754
memory wall, 38
Memory coalescing
GPU optimization, 596
tensor layout impact, 596
Memory footprint
computational requirements, 190
training overhead, 205
Memory fragmentation
KV cache, 745
Memory hierarchy
appendix refresher, 933
domain-specific, 546
ML training tiers, 370
speed-capacity trade-off, 569
Memory layout
NCHW vs. NHWC, 324
row-major vs. column-major, 318
stride patterns, 317
Memory management
activation retention, 372
caching allocators, 314
device placement, 319
inference efficiency, 218
Memory mapping
on-demand model loading, 740
Memory planning
compiler phase, 603
definition, 601
tensor allocation, 734
Memory wall
AI accelerators, 565
bandwidth divergence, 38
compute-memory gap, 38
data movement bottleneck, 178
definition, 38
execution strategy impact, 291
framework execution, 291
impact on GPU utilization, 621
Memory-aware tensor layouts, 599
Memory-bandwidth bound
precision impact, 732
Memory-Bound
roofline classification, 575
Memory-bound
activation tensors, 707
operation classification, 291
Memory-computation tradeoff
training systems, 361
Metric
etymology, 626
intransitivity, 626
MFCC
audio features, 114
dimensionality reduction, 114
Micro benchmark
component isolation, 632
diagnostic purpose, 633
Micro-benchmark suite
operator-level testing, 633
Microcontroller
memory ceiling, 61
ML deployment, 62
TinyML substrate, 21
Microscaling formats, 500
Microservice
cloud model serving, 698
model serving, 698
MIG
hardware isolation, 741
multi-model serving, 741
MinHash
distributed, cross-shard deduplication, 454
near-duplicate detection, 434
signature compression, 434
Mini-batch
definition, 193
gradient descent, 202
Mini-batch gradient descent, *see* Stochastic gradient descent
Mini-batch processing
accelerator efficiency, 355
Mixed-precision
layer-wise precision assignment, 675
Mixed-precision Adam
storage model, 410, 946
Mixed-Precision Computing
training and inference, 563
Mixed-precision training
benchmarking comparability, 647
definition, 347
FP16 and FP32, 648
FP16 vs. FP32 trade-offs, 313
FP16/BF16, 364, 387
loss scaling, 387
memory savings, 385
trade-off, 347
Mixture-of-experts, *see* MoE
MixUp
image augmentation, 442
ML Accelerator
definition, 547
ML compiler
compilation pipeline, 602
definition, 601
vs. traditional compiler, 601
ML development lifecycle
vs. traditional software, 25
ML failure
data dominance, 164
ML framework
compiler analogy, 287
compiler role, 43
definition, 287
differentiation problem, 286
dynamic memory management, 361
execution problem, 286
hardware abstraction problem, 286
historical evolution, 288
on-device deployment, 59
platform comparison, 335
ML inference benchmark
definition, 652
ML node
definition, 762
ML system benchmark
definition, 643
ML test score
production readiness, 810
ML training benchmark
convergence throughput scalability, 646
definition, 646
ML workflow
definition, 79
systematic framework, 79
MLCommons
nonprofit benchmark consortium, 656
open submission, 656
MLflow
experiment tracking, 778
MLOps
anti-patterns, organizational failures, 813
centralized mlops team, organizational pattern, 813
definition, 761
DevOps comparison, 760
emergence, 760
federated, embedded engineers, 813
production deployment challenges, 760
MLOps fallacies
automated retraining sufficiency, 822
automatic consistency, 823
DevOps equivalence, 822
MLOps pitfalls
alert routing, 824
monitoring overconfidence, 823
one-time deployment, 822
organizational neglect, 822
MLP, 231
definition, 231
GEMM-dominated computation, 599
MLPerf
founding, 640
founding and purpose, 640
inference benchmarking, 582
power, ML energy measurement, 640
MLPerf client
local generative AI workloads, 656
MLPerf Inference
benchmark family evolution, 656
MultiStream scenario, 728
offline scenario, 728
server scenario, 726
server to edge evaluation, 641
serving scenarios, 725
SingleStream scenario, 726
MLPerf Mobile
on-device inference, 656
MLPerf Power
energy efficiency measurement, 641
MLPerf Tiny
embedded AI benchmark, 656
microcontroller benchmarks, 641
MLPerf Training
data center multi-node scaling, 641
standardized framework, 647
mmap
model loading, 740
MMLU
format limitation, 677
multitask language understanding, 677
MNIST, 232
dataset saturation example, 681
digit recognition, 188
Mobile machine learning, *see* Mobile ML
Mobile ML
battery constraints, 57
computational photography, 60
definition, 58
energy budget, 58
energy constraints, 36
health monitoring, 60
memory bandwidth limits, 59
NPU inference, 59
resource constraints, 26
thermal envelope, 57
voice recognition, 60
Mobile SoC
architecture evolution, 607
MobileNet, 244
MobileNetV2, 850
definition, 21
depthwise separable convolutions, 42
efficiency gains, 42
efficiency under constraint, 876
latency-constrained deployment, 244
mobile power reduction, 57
workflow constraints, 85
MoCo

queue-based dictionary, 447
Model audit
high-stakes systems, 792
Model benchmarking
definition, 621
multi-dimensional evaluation, 674
Model caching
container embedding, 740
Model card
documentation standard, 849
standardized documentation, 849
Model collapse
recursive synthetic training, 444
tail distribution compression, 444
Model compression
accuracy-efficiency trade-off, 522
definition, 471
efficient model representation, 474
first use, 22
iterative workflow, 93
mobile vision, 57
multi-dimensional benchmarking, 655
silicon contract, 470
Model debugging
decision tree, 805
probabilistic failures, 805
root cause analysis, 805
Model decay
definition, 790
Model deployment
packaging and serving, 782
Model extraction
model artifact security, 861
Model FLOPs utilization
appendix refresher, 899
definition, 378
origin, 378
PaLM benchmark, 378
Model governance
compliance objective, 808
proactive policies, 808
transparency and compliance, 808
Model inversion attack
feature reconstruction, 861
Model loading
full loading, 739
Model Memory Pressure
architecture-specific, 574
Model monitoring
drift detection, 790
performance tracking, 790
Model Optimization
conversion validation, 782
Model parallelism
layer-wise partitioning, 406
memory constraints, 406
multi-accelerator scaling, 607
Model parameter
storage requirements, 155
Model performance metrics
definition, 100
Model quantization, *see* Quantization
Model registry
centralized artifact management, 778
data artifacts, 149
definition, 778
provenance tracking, 149
version management, 778
Model serialization
state dictionary pattern, 327
Model serving
definition, 695
paradigm shift, 694
Model size
parameter count, 190
Model validation
candidate selection, 780
definition, 95
predeployment assessment, 780
Model versioning
artifact tracking, 778
requirements, 155
Model virtualization
lifecycle management, 741
Model weights
data governance obligations, 867
definition, 4
Model-centric AI
architectural innovation, 680
Model-infrastructure interface
environment parity, 762
Module abstraction
hierarchical composition, 325
PyTorch nn.Module, 325
Module hook
forward and backward, 329
MoE
definition, 496
dynamic computation paths, 568
Momentum
memory overhead, 355
optimization physics, 355
Monitoring
health, replica readiness, 702
responsible operations, 850
Monitoring ROI
calculation, 797
Moore's law
ML systems consequence, 545
slowdown, 545
vs. AI growth, 22
Moravec's paradox
compute implications, 8
Multi-chip scaling
beyond single accelerator, 607
Multi-head attention, 262
Multi-instance GPU, *see* MIG
Multi-model serving
GPU memory management, 741
Multilayer perceptron, *see* MLP
Multiply-Accumulate
MAC unit, 561
MultiStream
synchronized sensor fusion, 645
MYCIN
expert system, 8

NAS
bi-level optimization, 492
bi-level optimization cost, 491
definition, 491
hardware-aware, 491
search cost, 493
NCHW
channels-first layout, 588, 596
Network depth
training difficulty, 246
Network interface card
node interconnect, 607
Network topology
layer organization, 188
Neural architecture search, *see* NAS
Neural network
artificial neuron, 180
capacity vs. cost trade-off, 181
depth vs. width trade-off, 190
feedforward, 188
layers, 186
perceptron, 181
Neural processing unit
energy efficiency, 44
mobile devices, 59
Neuron
McCulloch-Pitts model, 180
Next-Token Prediction
pretext task, 447
NHWC
channels-last layout, 588, 596
NHWC vs. NCHW
performance impact, 596
NN
dominant MAC pattern, 549
nn.Module
hierarchical composition, 327
parameter discovery, 325
parameter management, 325
registered buffer, 326
train vs. eval modes, 327
No free lunch theorem, 232
architecture selection, 232
Non-maximum suppression
irregular memory access, 608
Non-uniform memory access, *see* NUMA
Nonlinearity
XOR problem, 183
Normalization
batch, 248
feature scaling, 141
layer, 248
parameter persistence, 141
RMSNorm, 247
Notebook to production
handoff challenges, 809
Novelty Effect
A/B testing bias, 785
NPU
efficiency trade-off, 607
inference benchmarking considerations, 653
mobile SoC, 607
neural network acceleration, 657
on-device strategy, 560
Nucleus sampling
top-p, 744
NUMA
inference latency, 736
memory locality, 736
Numerical precision optimization, *see* Quantization
NumPy
framework foundation, 288
framework lineage, 288
NVIDIA
H100, 548
NVIDIA A100, 612
NVIDIA H100
reference machine, 940
NVIDIA Jetson
edge trade-off, 90
NVIDIA Nsight
systems and compute profiling, 636
NVIDIA Nsight Systems, 382
NVLink
bandwidth hierarchy, 319
definition, 573
GPU interconnect, 408, 573
gradient synchronization, 408
intra-node scaling, 573
PCIe bottleneck, 573
training interconnect, 408
training scaling, 573
NVMe
data pipeline, 156
GPU utilization, 153
storage bandwidth, 370

Object detection
bounding boxes, 132
Observability
system state inference, 790
Observable degradation
continuous measurement, 765
Offline capability
paradigm comparison, 63
Offline scenario
maximum throughput mode, 645
OLAP, 152
OLTP, 152
On-call practice
ML systems, 807
On-call rotation
ML-specific requirements, 807
On-chip memory
definition, 570
One-hot encoding
classification labels, 199
sparsity scaling, 199
One-shot pruning
definition, 480
oneDNN

Intel optimization library, 602
Online inference, *see* Dynamic inference
Online softmax
incremental computation, 392
ONNX
cross-framework portability, 729
definition, 729
hub-and-spoke interoperability, 730
ONNX Runtime
cross-platform serving, 730
Open division
MLPerf submission, 656
OpenCL
cross-processor execution, 610
OpenVINO
Intel inference, 731
Operating system
runtime role, 43
Operational maturity
system integration, 811
Operational mismatch
ML vs. traditional software, 760
Operations loop
definition, 762
Operator fusion
appendix refresher, 916
latency gap elimination, 636
Operator kernel
dispatch hierarchy, 323
GPU definition, 286
Optimization
Amdahl's law, 73
compile-time, constant folding, 734
inference, per-query cost, 855
performance, inference, 219
Optimization difficulty, 232
Optimization stack
multiplicative effects, 462
Optimizer
adaptive learning rate, 355
momentum-based methods, 355
Optimizer state
appendix refresher, 950
memory overhead, 347
Optum, 831
ORC
columnar format, 158
Oura Ring
wearable ML case study, 817
Out-of-memory error
training constraint, 391
Outlier Detection
data cleaning, 141
Output-stationary
partial sum accumulation, 589
Over-the-air update
model distribution, 786
Overfitting
benchmark, over-optimization to test sets, 672
definition, 215
training challenge, 216
Overhead-bound
small model training, 333
Overparameterization
compression rationale, 474

P99
first use, 623
Pacing function
linear warmup, 438
Padding
batching overhead, 755
PagedAttention
memory efficiency, 745
memory fragmentation solution, 745
Pandera
schema validation, 129
Papermill
notebook parameterization, 780
Parallel Computing
Amdahl's Law, 541
Parallel fraction
accelerated path, 541
Parallel processing
neural networks, 198
Parameter
memory multiplier, 186
registration, automatic tracking, 326
reuse, hierarchical learning, 190
Parameter freezing
transfer learning, 327
Parameter tracking
transformation lineage, 147
Parameter update
optimizer step, 373
Pareto Frontier
serving trade-offs, 720
Pareto frontier, 229
accuracy-efficiency trade-off, 675
compression trade-off, 474, 675
fairness-accuracy trade-off, 846
multi-objective optimization, 879
throughput-latency, 724
Parquet
columnar format, 157, 158
I/O reduction, 145
Partial derivative
gradient computation, 206
Patient treatment loop, 818
PCIe
bandwidth generations, 573
host-device interconnect, 319, 573
host-to-device transfers, 573
Peak FLOPS
misleading metric, 614
Perceptron
computational primitive, 231
definition, 181
early neural computation, 7
etymology, 231
linear separability limit, 181
Rosenblatt, 181
Perceptual hashing
image deduplication, 434
Performance per watt
efficiency metric, 613
Performance-per-data
metric definition, 429
Periodic score refresh
active learning, 455
Peripheral component interconnect express, *see* PCIe
Perplexity
etymology, 677
filtering, low-information text removal, 435
language model prediction measure, 677
serving interpretation, 677
Personalized model
user adapters, 747
Photoplethysmography
data quality challenge, 817
wearable sensing, 820
Physical AI
real-time control constraints, 883
Physical constraints
deployment implications, 36
memory signaling, 37
permanent boundaries, 74
speed of light, 37
thermodynamics, 37
Pickle
loading overhead, 740
Pile, The
data diversity, 433
Pinned memory
definition, 319
DMA transfer, 320, 741
Pipeline
cascading failures, 110
debt, workflow fragmentation, 771
failure diagnosis, 164
failure modes, 127
overlapping, latency hiding, 383
Pipeline parallelism
micro-batch scheduling, 408
micro-batching, 408
Pipelined execution
throughput analysis, 40
Point-in-time correctness
definition, 146
time travel, 146
Poisson distribution
traffic modeling, 627
Poisson process
queuing model, 711
serving arrivals, 711
Polysomnography
label uncertainty, 817
Pooling
downsampling, 238
global average, 238
Population stability index
appendix refresher, 907
credit risk origin, 793
definition, 160
drift detection, 100, 161, 793
drift quantification, 793
Portrait Mode
pipeline latency, 60
Positive Predictive Value
prevalence dependence, 92
Post-training quantization
calibration, 733
definition, 503
overview, 498
Postmortem documentation
incident analysis, 805
Postprocessing
confidence thresholds, 222
logits to predictions, 708
response formatting, 703
Power budget
edge devices, 854
Power management
mobile AI, 609
Power usage effectiveness
data center cooling metric, 664
Power wall
definition, 38
frequency scaling, 38
mobile implications, 59
thermal limits, 38
Power-bound workload
definition, 41
Pre-annotation
workflow, 136
Precision
dynamic, adaptive quality, 733
hardware design parameter, 562
numerical format trade-offs, 934
numerical, inference optimization, 219
positive prediction accuracy, 626
reduced, architectural trade-off, 559
throughput trade-off, 732
Predictive entropy, 136
Predictive maintenance
edge deployment, 57
edge duty cycle, 57
Prefetching
accelerator optimization, 546
data loader pipeline, 322
Prefill, 262
Prefill phase
compute-bound, 752
parallel processing, 752
Prefix caching
KV cache reuse, 746
Preprocessing
digit segmentation, 222
divergence, training vs. serving, 742
GPU, accelerated pipelines, 706
image normalization, 222
latency impact, 703
Pretext Task
self-supervised learning, 447
Price of fairness

utility trade-off, 847
Privacy
always-listening devices, 861
technical protection methods, 861
technique hierarchy, 861
Problem definition
baseline comparison, 87
ML vs. traditional, 87
multi-constraint optimization, 88
ProcessGroup
distributed execution, 322
Processing element
accelerator building block, 539
Production monitoring
continuous benchmarking, 685
Production-monitoring interface
feedback loop, 762
Progressive deployment
definition, 69
model compression, 69
Projection pushdown, 904
Prometheus
monitoring trade-off, 788
Protocol Buffers
serialization, 701
Proxy metrics
definition, 100
Proxy model
practical workflow, 436
Proxy variable
bias detection, 830
bias reconstruction, 866
definition, 830
Pruning
carbon reduction, 854
channel, 477
definition, 475
dynamic, 494
layer, 477
low-information filtering, 435
neuron, 476
Optimal Brain Damage, 475
outlier removal, 435
quality, label error detection, 434
static, 494
structured, 478
unstructured, 478
Pseudo-labeling
confirmation bias, 445
semi-supervised learning, 445
semi-supervised technique, 445
PUE
deployment tiers, 46
efficiency overhead, 46
power usage effectiveness, 857
Pure function
JAX, 336
JIT requirement, 338
Python dispatch
overhead, 331
PyTorch
autograd implementation, 315
autograd system, 311
eager execution default, 335
profiler, 382
research-first design, 335
tensor layout, 595
PyTorch profiler
performance analysis, 636

QKV
KV cache, 256
Quality control
edge processing, 57
Quality debt
definition, 162
uncorrected errors, 162
Quantization
activation-aware weight (AWQ), 516
affine, 507
appendix refresher, 936
asymmetric, 506, 936
AWQ bandwidth reduction, 516
channelwise, 508
definition, 498
energy impact, 501
energy saving per operation, 501
environmental benefits, 854
extreme (INT4, binary), 498
groupwise, 508
inference precision, 319
information-theoretic origin, 498
INT8 co-design, 541
INT8 compression ratio, 622
INT8 energy savings, 664
layer sensitivity, 733
layerwise, 508
reduced precision tolerance, 549
scale, 507
serving precision, 732
sub-channelwise, 509
symmetric, 506, 936
TinyML limits, 73
zero-point, 507
Quantization calibration
entropy method, 506
max method, 506
percentile method, 506
range, 504
Quantization granularity, *see* Quantization
Quantization-aware training
accuracy preservation, 513
definition, 498
Queries per second
inference throughput metric, 655
Query key value, *see* Query-key-value
Query-by-committee
active learning strategy, 439
Query-key-value, 256
Queuing Theory
serving foundations, 755
Queuing theory
serving systems, 710
utilization divergence, 712

RandAugment
hyperparameter simplicity, 443
Random sampling
minority representation cost, 838
Random seed
benchmark protocol requirement, 667
Ray
distributed ML, 778
Reasoning model
inference-time compute, 748
Recall
positive case detection rate, 626
Receptive field, 237
depth trade-off, 237
Recommendation system
DLRM, 42
Reconstruction attack
federated learning vulnerability, 862
Recurrent neural network, *see* RNN
Red AI
performance at any cost, 853
Red teaming
stakeholder engagement, 841
Redis
online feature serving, 153
Reduction, 551
Register
fastest memory, 569
Regularization
definition, 216
preventing overfitting, 216
Regulatory compliance
AI governance, 862
architecture constraints, 863
business value, 836
EU AI Act, 862
Reliability
circuit breakers, 131
dead letter queues, 131
error handling, 130
idempotency, 144
ReLU, 246
arithmetic intensity, 929
computational efficiency, 185
conditional operation overhead, 555
dead neuron, 246
dying ReLU problem, 185
etymology, 183
hardware efficiency, 183, 353
Representation capacity, 231
Reproducibility
benchmark requirement, 627
cross-platform challenge, 670
definition, 94
experiment lineage, 94
lineage, 147
versioning principle, 764
Reproducible system artifact
definition, 94
Request pipelining
GPU utilization, 709
Request transformation
tensor formats, 700
Residual connection
Transformer, 262
Residual learning, 246
ResNet
MLPerf reference model, 674
ResNet-50, 243
activation memory, 372
arithmetic intensity, 273
batching efficiency, 717, 876
benchmarking lighthouse, 622
cloud vs. mobile, 45
compute-bound workload, 542
data selection, 424
data-loading requirements, 157
input-pipeline case, 348
iron-law workload, 20
latency budget, 704
module hierarchy, 325
systems characteristics, 42
training memory footprint, 313
weight reuse mapping, 598
Resource isolation
serving, 735
Resource utilization
compute and memory, 650
Responsibility gap
definition, 830
Responsible AI engineering
definition, 835
Responsible engineering
definition, 828
lifecycle methodology, 836
safety control loop, 828
Retraining, 16
continuous updates, 798
decision framework, 798
economics, cost-benefit optimization, 799
scheduled updates, 798
scheduled vs. triggered, 798
triggered updates, 798
Retry logic
exponential backoff, 131
Return on compute
definition, 23
Return on investment
amortized, 456
data selection framework, 456
ML systems, 855
ML systems, heuristic baseline, 855
Reweighting
bias mitigation, 844
importance sampling, 844
Ridge Point
hardware comparison, 575
Ridge point
appendix refresher, 928, 950
definition, 575
machine balance ratio, 225
precision dependence, 363
Right to erasure
data lineage requirement, 863
Ring AllReduce

bandwidth optimization, 405
RISC-V
AI customization, 551
vector extensions, 551
Risk mitigation
responsible engineering value, 836
RMSNorm, 247
efficiency, 247
LLM adoption, 247
RMSprop
adaptive step sizes, 355
origin and Adam inheritance, 355
RNN
definition, 250
sequential dependency bottleneck, 254
Role-based access control
data governance, 859
Rollback
delayed, 784
immediate, 784
ML state mismatch, 101
model state management, 783
rapid, 784
strategies, safety mechanisms, 783
Roofline Analysis
conv2d layer, 579
dense layer, 580
layer normalization, 581
Roofline model
appendix refresher, 928
architecture comparison, 273
bottleneck analysis, 44
definition, 575
etymology and origin, 634
hardware diagnosis, 543
optimization diagnostic, 575
training bottlenecks, 363
Root/power unit, 557
Rosenblatt, Frank, 231
Rotting asset curve
accuracy decay, 803
Row-oriented storage
CSV, 158
Rule-based programming
limitations, 176
Rumelhart, David
backpropagation, 179

Safetensors
secure model loading, 740
zero-copy loading, 740
Safety
human oversight requirements, 839
responsible engineering, 828
Sample complexity, 231
Sample size
rate estimation, 906
Sampled softmax
compute efficiency, 199
Sampling
temperature, top-k, top-p, 744
Samuel, Arthur
machine learning, 7
SavedModel
TensorFlow export format, 335
Scalability
distributed training metric, 648
paradigm comparison, 63
Scaling
attention, numerical stability, 262
frequency, cubic power relationship, 609, 665
hybrid, GPU+CPU, 751
infrastructure, voice assistants, 67
physical ceiling, 409
Scaling efficiency
strong scaling definition, 649
Scaling laws
appendix refresher, 901
data limitations, 74
data-compute asymmetry, 422
power-law origin, 422
Scatter, 552
Scheduler
inference server, 700
Schema contract
data producers, 163
Schema debt
definition, 162
workarounds, 162
Schema evolution
failures, 131
silent failure, 131
Schema validation
data quality, 793
ML pipelines, 127
structural data checks, 793
Schema-on-read
flexibility, 153
governance trade-off, 153
Schema-on-write
traditional databases, 153
Schmidhuber, Jürgen, 252
Scope creep
model cards, 849
Scratchpad Memory
accelerator design, 549
software-managed storage, 571
Scratchpad memory, 570
explicit control, 275
ML advantage, 549
Secure Enclave
biometric privacy, 59
Secure enclaves
trusted execution environment, 860
Security
access control architecture, 859
Selection dependency
distributed training, 453
Selection engineering
definition, 448
Selection inequality
definition, 431
Selection logic
definition, 549
Selective FP32 computation
mixed precision, 388
Selective gradient computation, 311
Self-attention, 261
Self-paced learning
adaptive difficulty, 439
Self-supervised learning
definition, 446
foundation model paradigm, 447
iron law restructuring, 446
Semantic segmentation
pixel-level labels, 132
Semi-supervised learning
consistency plus pseudo-labeling, 445
definition, 445
labeling efficiency, 136
Sensitivity
medical AI metrics, 91
Separation of concerns
MLOps layers, 764
Sequence length variability
batch waste, 744
Serial fraction
Amdahl bottleneck, 541
Serialization
definition, 904
overhead, 701
Server scenario
Poisson-distributed traffic, 645
Server traffic
Poisson process, 726
Serverless AI
cold-start benchmarking, 656
Serving
cloud/data center, 698
edge inference, embedded serving, 698
mobile/edge, 698
production deployment, 694
TinyML, 698
Serving inversion
definition, 694
throughput-to-latency shift, 694
Session affinity
sticky sessions, 747
Severity classification
incident prioritization, 804
Shadow deployment
definition, 783
inference cost, 783
Shallow learning
vs. deep learning, 175
Shannon, Claude
information theory, 199, 908
SHAP
feature attribution, 848
inference latency, 848
SHAP value
model debugging, 806
Shapley value
feature attribution, 848
Shift handoff
context transfer, 807
Shuffle buffer
approximate random sampling, 451
Sigmoid
computational cost, 184
etymology, 184
Signal-to-noise ratio
definition, 908
Silent degradation
definition, 15
Silent failure
definition, 760
detection challenges, 832
gradual degradation, 765
Silicon contract
constraint propagation, 874
definition, 37
hardware side, 44
iron law, 40
physical constraints, 37
SimCLR
batch requirements, 447
SIMD
CNN inference, 243
CPU inference, 736
CPU vectorization, 736
data-level parallelism, 551
definition, 177
edge deployment, 243
hardware execution, 557
SIMT
definition, 558
Single instruction multiple data, *see* SIMD
Single instruction multiple thread, *see* SIMT
Single-Machine Processing
scalability, 145
Single-metric evaluation
fallacy, 683
Single-node regime
definition, 27
Single-node scope
boundary, 145
Single-node stack
architecture layers, 43
definition, 43
Single-user traffic
mobile serving, 726
SingleStream
sequential inference scenario, 645
Singular value decomposition
rank-bandwidth trade-off, 483
Skip connection
cross-architecture portability, 246
gradient flow, 190, 243
SLA
data freshness, 124
definition, 654
SLI
definition, 703

Slice analysis
  subpopulation debugging, 805
SLO
  capacity planning, 710
  definition, 654
  first use, 623
  latency budget, 703
  multidimensional targets, 695
  percentile targets, 712
  vs. SLA, 695
Snappy
  decompression speed, 159
Snorkel
  weak supervision, 136
SoC
  always-on island, 114
Soft affinity
  load balancing, 747
Softmax
  appendix refresher, 916
  etymology, 185
  memory-bound operation, 576
  numerical stability, 185
  probability distribution, 185
Softmax function
  quadratic bottleneck, 257
  systems, 257
Software 1.0
  definition, 4
Software 2.0
  definition, 4
Sparse connectivity
  connection patterns, 189
Sparse Tensor Core
  2:4 constraint, 563
Sparsity
  activation sparsity, 185
  definition, 519
  hardware gap, 519
  hardware speedup requirement, 563
  memory access irregularity, 568
  N:M structured pattern, 563
  structured, 2:4 hardware support, 563
SPEC CPU
  real application workloads, 640
SPEC Power
  server energy efficiency, 640
SpecAugment
  audio augmentation, 122
Special Function Unit
  activation functions, 540
  SFU, 554
Specificity
  medical AI metrics, 92
Spectrogram
  audio features, 142
  audio representation, 142
Speculative decoding
  draft model verification, 744
  latency reduction, 744
  throughput, 744
Speech recognition
  forced alignment, 138
Speed of light check
  utilization diagnosis, 633
SQL
  feature pipeline primitives, 905
SQuAD
  reading comprehension dataset, 625
  saturation, 625
SRAM
  on-chip cache, 570
  on-chip fast memory, 570
SSD cache
  model loading, 740
Stable Diffusion
  generation cost, 443
Stage interface specification
  contracts, 84
  definition, 84
Staged rollout
  canary, 96
  definition, 783
  lifecycle preview, 96
  shadow mode, 96
  workflow, 96
Staging validation
  probabilistic adequacy, 782
Stakeholder communication
  business alignment, 809
  technical translation, 809
Staleness cost
  accuracy decay model, 799
Standardization
  feature scaling, 141
Static batching
  fixed batch size, 721
Static graph
  define-then-run model, 294
  execution model comparison, 300
  optimization benefits, 296
Static inference
  precomputed predictions, 697
Static pruning
  coreset selection, 435
  definition, 424
  pretraining filtration, 433
Static quantization, *see* Quantization
  definition, 509
Static random access memory, *see* SRAM
Static single assignment
  appendix refresher, 916
  compiler IR, 297
Statistical decision theory
  loss function origin, 194
Statistical drift risk
  accuracy erosion, 880
  degradation equation, 798
Statistical learning
  definition, 9
Statistical telemetry
  ML observability, 762
Statistical validation
  sampling strategies, 128
Statistics unit, 557
Stochastic gradient descent
  gradient estimation, 355
  mini-batch procedure, 214
  systems trade-off, 6
Storage
  activation, backpropagation, 205
  cold, archival, 149
  columnar, ML training, 152
Store-and-forward architecture
  definition, 90
  intermittent connectivity, 90
Straight-through estimator
  gradient approximation, 510
  quantization-aware training, 511
Stratified evaluation
  population-based testing, 841
Stratified sampling
  fairness evaluation, 838
Stream ingestion
  definition, 124
Streaming multiprocessor
  GPU architecture, 558
  occupancy, 558
Streaming response
  LLM serving, 744
Streaming traffic
  sensor synchronization, 726
Strong scaling
  appendix refresher, 930
  definition, 649
Structured approximation
  definition, 483
Structured pruning
  hardware alignment, 541
Structured query language
  feature pipeline primitives, 905
Structured sparsity
  N:M pattern, 478, 563
STUDENT
  symbolic AI failure mode, 7
Subgroup imbalance
  demographic disparity, 679
Subnormal numbers
  gradient underflow, 206
Supervised learning
  labeled examples, 192
Sustainability
  environmental responsibility, 853
Sutton, Richard
  bitter lesson, 11
Switch Transformer
  scaling trade-off, 496
Symbolic AI
  definition, 7
Symmetry-aware architecture, 236
Synchronization
  device vs. stream, 321
Synchronization event
  fine-grained, 321
  inter-stream, 321
Synchronous scoring
  active learning, 455
Synthetic data
  generation, 121
  rare event coverage, 122
  scalability, 121
  simulation, 122
Synthetic data generation
  definition, 424
  three-stage pipeline, 442
Synthetic Microbenchmark
  limitations, 642
System bottleneck
  dominant constraints, 46
  identifying, 40
System composition, 885
System efficiency
  feeding loss, 151
  hardware duty cycle, 709
System entropy
  model decay, 799
System is the model, The
  definition, 874
System-on-chip
  energy integration, 59
  heterogeneous AI acceleration, 607
  mobile architecture, 59
Systematic error
  learned ground truth, 679
Systems co-design
  definition, 10
Systems thinking
  definition, 102
  principles in ML, 102
Systolic Array
  data flow architecture, 560
  dataflow design, 561
  energy efficiency, 560
  TPU configuration, 561
Systolic array
  CNN data reuse, 235
  definition, 546
  etymology, 235
  TPU, 235

t-closeness
  privacy technique, 861
T4
  serving economics, 751
Tail at scale
  fan-out amplification, 716
Tail latency
  appendix refresher, 905
  fan-out amplification, 654
  p99 significance, 654
  revenue impact, 703
  utilization threshold, 712
Tanh
  etymology, 184
  zero-centered advantage, 184
Task-relevant signal
  definition, 149
  vs. Shannon entropy, 150
Technical debt
  abstraction debt, missing ML interfaces, 771
  assessment rubric, 810

data dependency debt, unstable inputs, 770
data engineering, 162
dead code, experimental ML paths, 771
definition, 767
ML compounding, 767
ML entanglement, 83
ML system complexity, 767
ML systems, 83, 851
Telemetry
ML observability, 762
Temperature
softmax distillation, 487
Temperature scaling
calibration restoration, 677
Tensor
appendix refresher, 912
data types (dtypes), 318
definition, 317
etymology, 316
metadata, 317
metadata and storage, 316
n-dimensional arrays, 316
operations, GEMM, 324
stride patterns, 318
Tensor arena
TinyML memory, 698
Tensor Core
dimension alignment, 559
dimension constraints, 234
energy efficiency, 559
hardware execution, 559
matrix multiplication, 539
matrix multiply-accumulate, 389
mixed precision, 234
mixed precision acceleration, 389
mixed-precision acceleration, 313
precision support, 559
reduced-precision matrix units, 217
throughput mechanism, 559
training throughput, 351
Tensor decomposition
definition, 484
Tensor-Train, 484
Tensor layout
hardware alignment, 595
memory-aware, 588
NCHW vs. NHWC, 700
Tensor memory layout
NCHW, 700
NHWC, 700
Tensor parallelism
intra-layer splitting, 408
operation splitting, 408
serving, 746
Tensor Processing Unit, *see* TPU
TensorFlow
full framework, 340
graph-first architecture, 335
profiler, 382
symbolic differentiation, 315
tensor layout, 595
TensorFlow Data Validation, 128
TensorFlow Lite
mobile deployment, 340
TensorFlow Lite Micro
fixed memory arena, 305
microcontroller deployment, 340
TensorFlow Serving
inference server, 699
production deployment, 335
TensorFlow.js
browser deployment, 335
TensorRT
accelerator compilation, 585
GPU optimization, 698
hardware utilization, 729
model optimization, 782
Ternarization
definition, 514
Terraform
infrastructure as code, 781
Tesler's law
conservation of complexity, 878
tf.function
graph compilation, 335
TFRecord
sequential sharding, 157
Theano
computational graph origin, 289
Thermal design power
mobile constraint, 58
Thermal management
mobile constraints, 609
Thermal throttling
impact on sustained perf, 633
intelligent migration, 610
mobile serving, 726
performance limits, 73
Thermal trip point
hardware safety, 59
Thermal wall
definition, 58
mobile constraints, 59
Thirteen quantitative principles
framework, 877
Thread pinning
CPU affinity, 736
Threshold adjustment
fairness mitigation, 845
Thresholding
confidence cutoff, 708
Throughput
accelerator parallelism, 198
definition, 648
etymology, 648
storage bandwidth, 152
training, compute vs. data, 156
vs. latency trade-off, 626
Throughput ceiling, 280
Tied request
latency reduction, 716
Tiered escalation
incident response, 807
Tiered retention
hot/warm/cold, 149
Tiered storage
edge deployment, 153
Tiling
algorithms, matrix computation, 395
attention computation, 392
definition, 590
DRAM traffic reduction, 592
explicit staging, 593
framework GEMM, 324
implicit cache blocking, 593
memory optimization, 588
memory-efficient strategies, 592
Time per output token
decode phase, 742
definition, 655
Time to first token
definition, 655
prefill phase, 742
Time travel
feature stores, 146
Time-multiplexing
model swapping, 741
Time-to-accuracy
training benchmark primary metric, 648
Timeline profiling
serving optimization, 736
TinyML, 26
always-on sensing, 36
bare-metal serving, 698
cost efficiency, 60
definition, 61
device form factor, 62
ecosystem fragmentation, 63
energy efficiency, 54
energy gap, 61
medical wearables, 63
memory planning, 304
micro-runtimes, 304
model compression, 62
operational constraints, 786
precision agriculture, 63
resource constraints, 62
training memory constraint, 62
ubiquitous sensing, 60
wake-word detection, 63
Token throughput
generation rate, 655
Tool-first anti-pattern
fragmented responsibilities, 813
Top-k
sampling, 744
TOPS
comparability limitation, 660
TOPS/W
primary design objective, 661
torch.compile
hybrid execution, 299
TorchScript
tracing and scripting, 297
Total cost of data
formula, 455
Total cost of ownership
cloud vs. edge, 50
definition, 50
inference dominance, 855
lifecycle split, 855
operational costs, 855
ToyADMOS
domain gap, 631
TPU, 855
benchmarking considerations, 643
data center integration, 548
matrix unit design, 353
origin, 545
specialization trade-off, 38
systolic array scaling, 561
XLA compilation, 334
Trace-based benchmarking
production replay, 642
Traditional compiler, 601
Traditional runtime, 605
Traffic-adaptive batching
window tuning, 727
Train-serve split
cost asymmetry, 68
definition, 68
economics, 68
Training
bidirectional computation, 44
computational cost, 351
computational intensity, 347
cost asymmetry, 346
data throughput, 44
inference vs. training, 346
infrastructure evolution, 404
mathematical foundations, 351
memory decomposition, 914
memory equation, 359
memory exhaustion threshold, 408
memory management, 361
memory pressure, 347
multi-GPU configuration, 405
scaling strategies, 404
scaling threshold, 408
step, anatomy, 330
supervised learning, 192
systems, 346
time estimation, 931
training-time law, 349
wall-clock time limit, 409
weight optimization, 202
weight update, 373
Training bottleneck
compute-bound vs. memory-bound, 378
min-of-three relationship, 369
profiling, 378
Training cost
electricity, 951
scaling trajectory, 346
Training cost asymmetry
definition, 346
Training loop
iterative learning, 214
Training memory
Adam storage model, 927

Training optimization
technique selection, 380
Training pipeline
architecture, 365
automated ML workflows, 779
data flow, 365
staged system workflow, 365
Training profiler
bottleneck analysis, 379
timeline visualization, 379
Training systems
definition, 346
Training vs. Inference
accelerator design, 547
Training vs. inference
operational differences, 217
resource differences, 217
Training-Serving Consistency, 140
Training-serving skew
accuracy impact, 775, 880
causes, 128
definition, 140
diagnosis and prevention, 776
first use, 16
serving accuracy fallacy, 754
silent accuracy degradation, 741, 776
Training-time law
definition, 349
utilization gap, 349
Transfer learning
compute efficiency, 92
definition, 92
labeling alternative, 136
Transformation lineage
definition, 147
Transformation versioning
reproducibility, 147
Transformer
architectural shift, 255
definition, 261
etymology, 255
KV-cache memory pressure, 599
recurrence removed, 260
training evolution, 351
Transistor tax
activation functions, 185
definition, 185
Translation equivariance, 237
systems trade-off, 237
Translation invariance, 232
Transparency
governance objective, 808
regulatory requirements, 848
Triton
inference server, 699
True positive rate
fairness computation, 842
Trust
competitive differentiation, 836
Tucker decomposition
definition, 484
Turing, Alan
Imitation Game, 7

Ubiquitous computing
etymology, 60
TinyML realization, 60
Uncertainty sampling
active learning query strategy, 439
Undeclared consumer
hidden model dependency, 770
Tesla case study, 772
Unified Memory
single address space, 573
Unified memory
mobile SoC, 608
Uniform quantization
definition, 503
Universal approximation theorem, 183, 231
depth vs. width, 183
systems consequence, 231
Unreasonable effectiveness of data
systems impact, 9
Unstructured pruning
hardware limitations, 521
Update rule
gradient descent, 202
Upstream echoing
definition, 452
Utilization
high utilization pitfall, 754
latency relationship, 712

Validation
accuracy plateau, 216
Vanishing gradient problem, 254
activation saturation, 184
definition, 184
exponential decay, 184
Vector execution, 612
Vector index
embedding-based selection, 450
Vector Operation
hardware acceleration, 551
Vector operation, 551, 552
Vector Unit
element-wise operations, 539
Vector-scalar broadcast, 552
Vectorization
automatic batching, 337
columnar processing, 904
Vendor lock-in
cloud deployment, 51
portability trade-off, 615
Verification gap
definition, 6
MLOps management, 780
statistical testing, 879
Version pinning
inference runtime, 782
Viola-Jones algorithm
compute efficiency, 9
Virtuous cycle
definition, 445
Vision transformer, 278
compute trade-off, 278
ViT, *see* Vision transformer
vLLM
virtual memory analogy, 744
Voice assistant
hybrid architecture, 67
Von Neumann bottleneck
ML accelerator constraint, 547
VRAM
device memory, 741

Wake-word detection
always-on power budget, 63
layered architecture, 67
Wald, Abraham
statistical decision theory, 194
Warehouse-scale computing
training evolution, 405
Warmup
cache population, 738
cold start mitigation, 738
Warmup runs
benchmarking methodology, 684
Warp
divergence penalty, 558
etymology and hardware constraint, 371
execution unit, 558
GPU execution unit, 371
Waterfall model
contrast with ML, 79
Wave quantization
accelerator utilization, 371
tail effects, 371
Waymo
domain gaps, 26
Weak scaling
appendix refresher, 931
definition, 649
Weak supervision
cost trade-off, 136
programmatic labeling, 136
Web scraping
ML training data, 120
WebDataset
sequential sharding, 157
Weight decay
definition, 216
Weight loading
cold start, 738
memory efficiency, 717
Weight matrix
layer connections, 187
Weight Reuse
spatial, 598
Weight sharing, 236
Weight update
optimization, 202
Weight-stationary
CNN optimization, 589
Weights
learnable parameters, 181
matrix organization, 187
Werbos, Paul
backpropagation, 179
Whetstone
synthetic floating-point benchmark, 640
Wildlife conservation
TinyML monitoring, 63
WILDS
distribution shift benchmark, 680
Williams, Ronald J.
backpropagation, 179
Workflow
lab-deployment gap, 81
Workflow orchestration
pipeline automation, 775
Working set size, 275
Workload archetype
definition, 41
optimization strategy, 41
Workload distribution
heterogeneous processor, 608
Workload imbalance, 585
Worst-Case Execution Time
edge AI, 786

XLA
accelerator compilation, 585
compilation speedup, 296
definition, 296
graph compilation, 296
warm-up compilation, 782
XOR problem
nonlinearity requirement, 183

YOLOv8-nano
edge inference sizing, 55

ZeRO
sharded data parallelism, 406
Zillow
coupled decisions, 772